Embedded Learning Hub

A complete systems engineering curriculum

23 tracks · 490 chapters

Generated 2026-08-23

TRACK

Start here

How the curriculum is organised, what to read in what order, and how long it takes.

4 chapters

One concept per chapter.
Zero to systems engineer.

Twenty-three tracks, 490 chapters. Each chapter explains exactly one idea in depth — what it is, why it exists, how the hardware or software actually implements it, and how it fails. Read a track in order and you get a course; search a concept and you get a reference.

No prior kernel experience needed · Foundations included · ARMv8-A / AArch64 · Linux 6.x · Windows 11 WDK · Hands-on projects

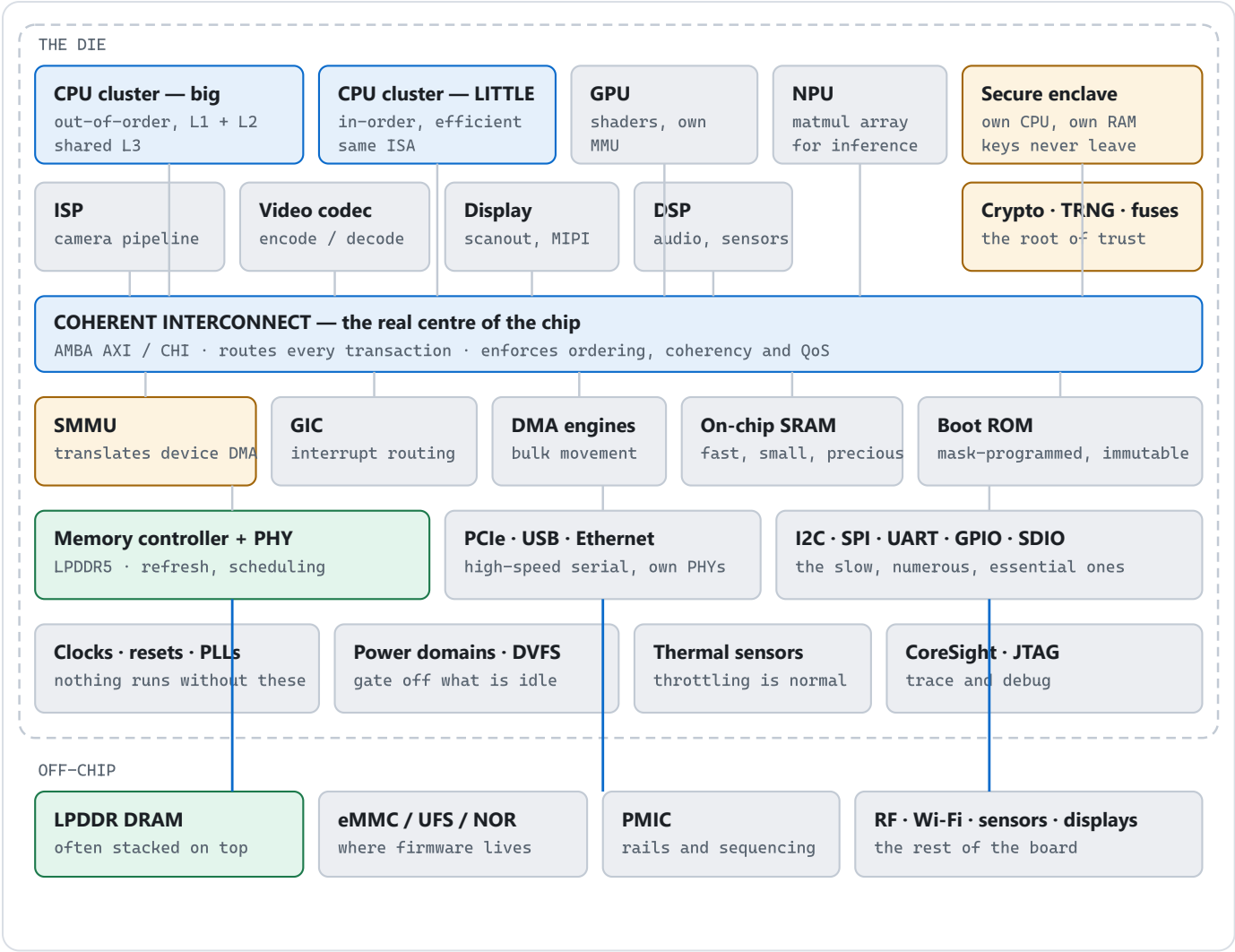
The twenty-two tracks

What a system-on-chip is

Nearly everything on this site runs on a **SoC** — a single piece of silicon holding not just processor cores but the memory controller, the graphics and AI engines, the camera pipeline, the interrupt controller, the security blocks and dozens of peripherals. A desktop PC spreads those across a motherboard. A phone, a car ECU, a router and a smart camera put them on one die, a few millimetres square.

That integration is not merely tidier. It is the reason the field looks the way it does:

Consequence of integration	What it forces on you
Wires on a die are thousands of times cheaper in power than wires on a board	Bandwidth that would be impossible off-chip, and thermal limits that arrive far sooner
Dozens of blocks share one memory system	Coherency, quality-of-service and contention become your problem, not the hardware's
Every block is a *bus master* that can read and write DRAM by itself	Memory protection must exist in the interconnect, not only in the CPU — this is why an SMMU exists
The chip is fixed at manufacture; bugs cost a mask set	Errata are worked around in firmware and drivers. A large part of systems work is compensating for silicon nobody can change
One vendor integrates blocks licensed from many others	Documentation is uneven, and the authoritative answer is often a register description rather than a manual



A representative ARM SoC. The exact blocks vary enormously; the shape — compute at the top, one coherent interconnect in the middle, memory and I/O below — does not.

THE INTERCONNECT IS THE CHIP

Beginners picture a SoC as "a CPU with peripherals attached". It is closer to a network with a CPU as one of its nodes. The GPU, the NPU, the ISP and every DMA engine issue their own memory transactions, compete for the same DRAM, and can corrupt anything they are allowed to reach. Once that clicks, the existence of the SMMU, of cache-coherency protocols, of QoS registers and of memory attributes stops feeling arbitrary.

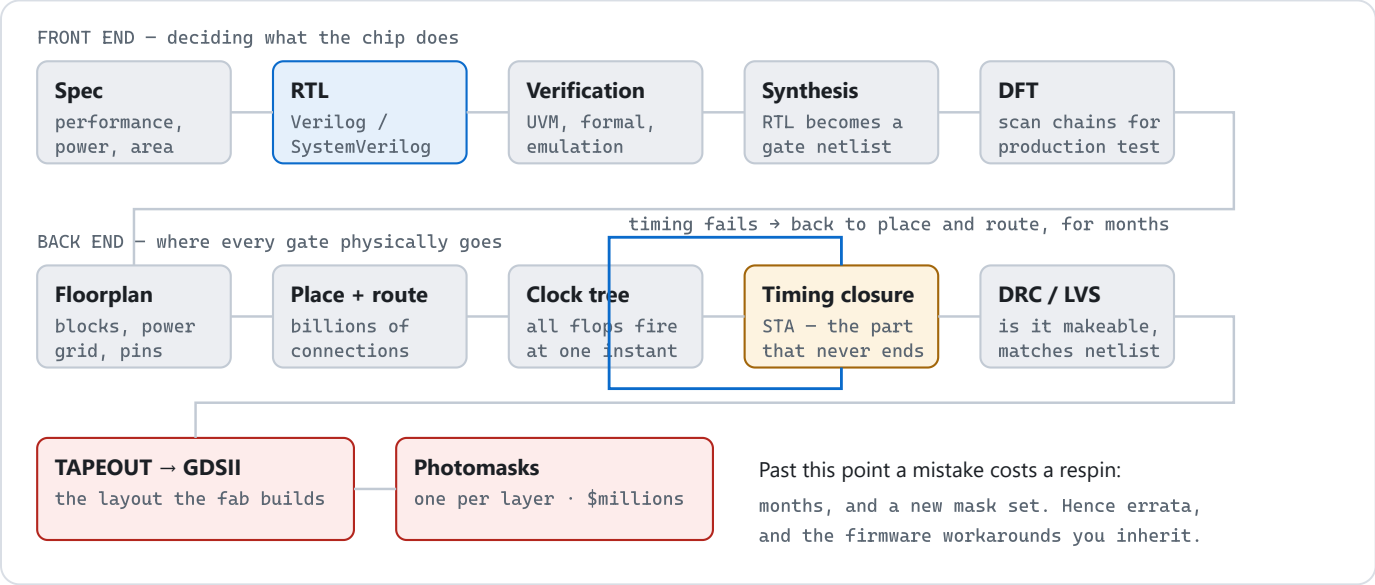
Which track covers which block

Block	Where it is taught
CPU cores, caches, pipelines, memory model	Computer architecture, then ARM architecture
Interconnect, AMBA, coherency, shareability	ARM — the AMBA and memory-model chapters
Memory controller, DRAM behaviour, DMA	Architecture and the driver tracks
GIC, timers, power states	ARM
SMMU, TrustZone, access-control firewalls	Isolation and security
Secure enclave, crypto engine, fuses, root of trust	Security, cryptography and OP-TEE
Boot ROM and the firmware chain	OP-TEE — the boot-flow chapters
Peripherals, and drivers for them	Linux or Windows kernel and drivers

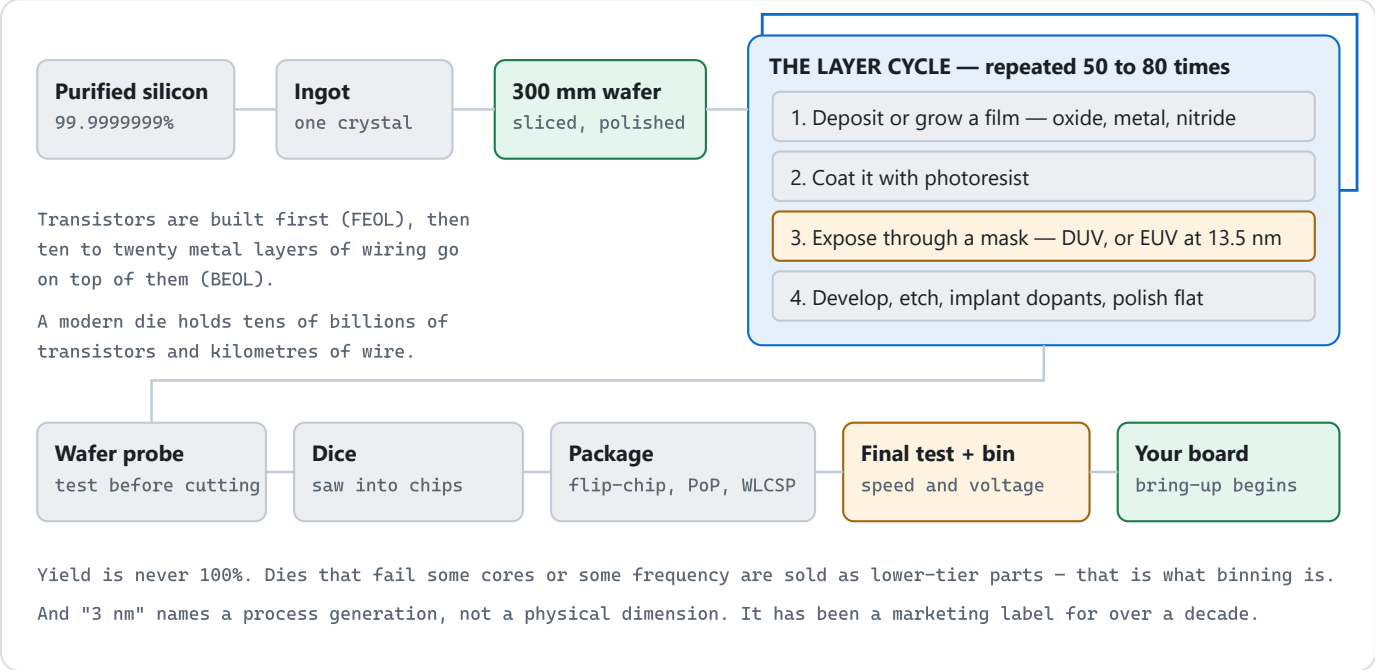
Block	Where it is taught
Running all of it before silicon exists	QEMU and virtualisation
NPU, quantisation, why inference is memory-bound	AI and large language models
CoreSight, JTAG, and finding out what went wrong	Debugging and reference

How a chip is made

Two long pipelines sit behind every SoC. The first turns an idea into a set of photographic masks; the second turns those masks into silicon. Together they take two to four years and hundreds of millions of dollars, which is why the errata document is thicker than you expect.



The design flow. Everything left of tapeout is software; everything right of it is a factory.



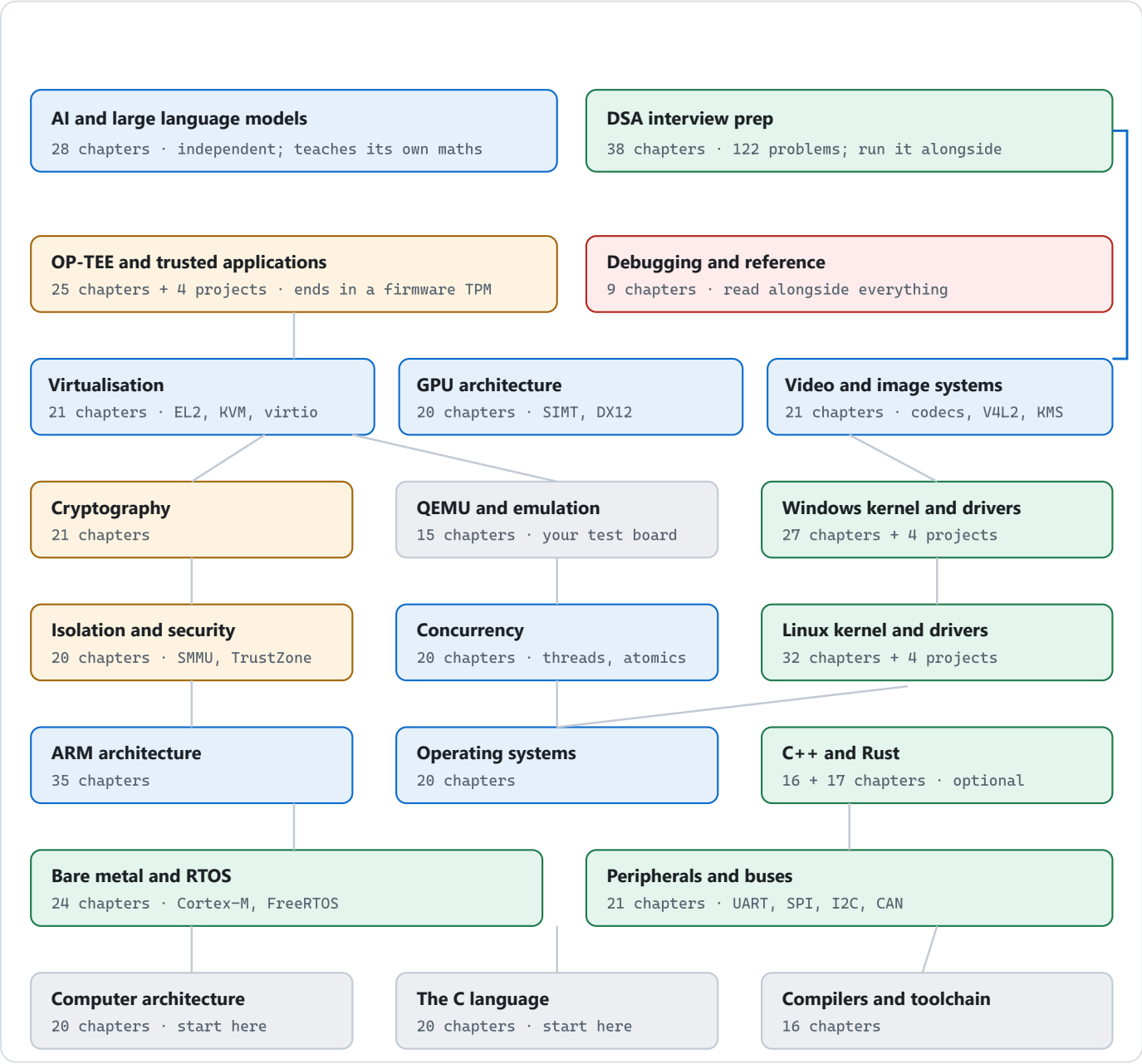
Fabrication. The cycle in the middle is the whole game: every layer of the chip is one pass through it, and a single dust particle in the wrong place kills the die.

WHY THIS MATTERS TO A DRIVER AUTHOR

Because silicon cannot be patched. When a block misbehaves — a DMA engine that ignores a coherency hint, a controller that needs a dummy read after a register write, an interrupt that fires spuriously at reset — the fix lands in firmware or in your driver, and it stays there for the life of the part. Reading an errata sheet and recognising your own bug in it is an ordinary Tuesday, and it makes far more sense once you know how little freedom the hardware team had after tapeout.

What order to read in

The tracks are not independent — later ones assume earlier ones. This is the dependency graph. Anything you read before its supports are in place will feel like memorisation instead of understanding.



Track dependencies. Two entry points (architecture and C); everything else stacks on them. The top two tracks are independent — start either at any time.

How each chapter is built

Every chapter follows the same shape, so you always know where to look for what you need:

Section	Answers
The problem	Why does this concept exist? What breaks without it?
The mechanism	How it actually works, drawn as a flow or diagram wherever possible
In practice	Real code, real register layouts, real command lines
How it fails	The bugs this concept causes, and their signatures
Check yourself	Questions you should be able to answer before moving on

> **CHAPTERS MARKED "DRAFTING"**
> The curriculum map is complete and fixed — every chapter listed is one that will exist, in that order. Chapters still being written are greyed out in the sidebar and marked *drafting* on their track page. Published chapters are finished, not stubs.

Where to start

1. **Read the prerequisites honestly.** If C pointers are shaky, the C track comes before everything else. There is no way around this.
2. **Follow the roadmap** for a week-by-week path with realistic time costs.
3. **Set up your workbench** — a cross-compiler, QEMU, and one Linux and/or Windows VM. Every project on this site runs without buying hardware.
4. **Then begin at computer architecture, chapter 01**, or jump to whichever track your job needs and backfill when you get stuck.

The learning roadmap

490 chapters is a lot of map. This page is the route through it: what to read in what order, how long each stage honestly takes, and the test that tells you a stage is finished.

The organising principle

Every track on this site answers one of four questions, and they must be answered in this order:

Question	Tracks	Why it comes when it does
What is the machine?	Computer architecture, ARM	Every abstraction above this is a lie told for convenience. You cannot debug a lie you do not know is one.
How do I express myself to it?	C, C++, Rust, Compilers	Languages are how you address the machine; the compiler is the translator whose assumptions you must share.
Who manages the machine?	Operating systems, Linux, Windows	An OS is a set of policies over the hardware. Learn the hardware, then the policy.
How do I keep it safe?	Isolation and security, Cryptography	Isolation is meaningless before you understand what is being isolated from what.
What sits above the kernel?	Virtualisation, OP-TEE, QEMU	A hypervisor and a trusted OS both use a privilege level the kernel cannot reach, so they come after you know what a kernel does. QEMU is the board you develop both on.

The AI track sits outside this ordering. It teaches its own mathematics and depends on the rest of the site only where performance meets hardware — memory bandwidth, quantisation, accelerators.

Eight phases

Phase 0 — Prerequisites (0–4 weeks)

Take the self-assessment. If you fail the C section, do the C track first and nothing else. Everything on this site is written in C or described in terms of it.

Exit test: you can write a function that packs three bitfields into a 32-bit word using shifts and masks, and explain why the compiler might reorder your two writes to a hardware register.

Phase 1 — The machine (4–7 weeks)

Computer architecture, all 20 chapters. This is the biggest single jump in the whole curriculum, and skipping it is why most self-taught engineers plateau.

Exit test: explain why a program that touches an array with stride 64 runs several times slower than one with stride 1, in terms of cache lines and prefetchers.

Phase 2 — The language, properly (4–6 weeks)

C in full, then the first half of compilers (chapters 01–06). Do these together: the compiler track explains why the C track's rules exist.

Exit test: given a small C function, predict its assembly, then check with `gcc -O2 -S` and explain every difference.

Phase 3 — The ARM architecture (5–8 weeks)

ARM, all 35 chapters. Long, but each chapter is one concept and they build tightly. Do not skip the memory-model chapters (20–22); they are where the hard bugs live.

Exit test: given a 48-bit virtual address and a set of table base registers, trace by hand which bits index which level of the page table, and say what happens at each possible failure point.

Phase 4 — Operating systems (4–6 weeks)

Operating systems for the vendor-neutral concepts, then pick Linux *or* Windows as your primary and skim the other. Both end in a complete driver you build.

Exit test: your driver deadlocks under load. Describe the tools and the order you would use them in, without opening a debugger first.

Phase 5 — Isolation and security (4–6 weeks)

Isolation and security, then cryptography. This is the part most engineers never reach, and it is what separates a driver writer from a platform engineer.

Exit test: a device DMAs to an address and faults. Name five distinct layers that could have produced that fault and the register you would read in each case.

Phase 6 — The two worlds and the machine underneath (5–8 weeks)

Two tracks that both start from the same place — an exception level above EL1 — and go in opposite directions. Virtualisation uses EL2 to run several kernels that each believe they are alone; OP-TEE uses the secure world to run one thing that the kernel cannot touch. Read QEMU alongside either: it is the board you develop both on, and it is short.

The virtualisation track needs phase 5's SMMU material for device passthrough. The OP-TEE track needs phase 5's TrustZone material and a working grasp of cryptography before its projects.

Exit test: write a trusted application and the client that drives it, then explain — with the register that enforces it — why the normal world cannot read the key it holds.

Phase 7 — Breadth and depth (ongoing)

- C++ or Rust, depending on where your industry is going. Rust if you want to be ahead of it.
- The second half of compilers — linking, loading, debug info.
- AI and LLMs, at any point; it is self-contained.
- Read real driver source. Bring up a real board. Send a patch upstream.

Realistic timeline

Phase	Chapters	Part-time (8 h/wk)	Full-time
0 · Prerequisites	0–20	0–4 weeks	0–1 week
1 · Computer architecture	20	6 weeks	2 weeks
2 · C + compiler basics	26	6 weeks	2 weeks
3 · ARM	35	8 weeks	3 weeks
4 · OS + one driver track	45–50	10 weeks	4 weeks

Phase	Chapters	Part-time (8 h/wk)	Full-time
5 · Security + crypto	41	8 weeks	3 weeks
6 · Virtualisation + OP-TEE + QEMU	60	8 weeks	3 weeks
To employable as a systems engineer		**~11 months**	**~17 weeks**

THESE NUMBERS ASSUME YOU ARE BUILDING, NOT ONLY READING

Reading a chapter takes fifteen minutes. Understanding it takes an hour of doing something with it — compiling the example, changing it until it breaks, and explaining the break. The timeline above assumes roughly one part reading to three parts hands-on. Halve the hands-on and you double the calendar time, because you will re-read everything.

Four ways people stall

COLLECTING BOARDS

A Raspberry Pi, a BeagleBone and three dev kits in a drawer is not progress. One board, broken and recovered ten times, teaches more than five untouched ones. Everything here runs in QEMU anyway.

READING THE ARM ARM COVER TO COVER

It is over 10,000 pages of reference material, not a textbook. Use it to answer a question you already have. That is what this site's ARM track is for: it gives you the questions.

SKIPPING THE MEMORY MODEL

Barriers, coherency and cache maintenance feel like optional detail right up until you have a bug that only reproduces on four cores under load. That bug costs a week. Those chapters cost a day.

WRITING DRIVERS WITHOUT READING DRIVERS

The kernel already contains a near-exact template for whatever you are writing. Professionals find it and adapt it. Search `drivers/` before you start typing.

Interview preparation runs alongside, not instead

The DSA interview prep track sits outside the seven phases deliberately. It is not a prerequisite for anything here, and nothing here is a prerequisite for it — it exists because getting the job and doing the job are measured differently, and the gap between them is real.

Start it when you have interviews booked, not before. Two to eight weeks is the usual window, and the track opens with a schedule for each. It shares the C track's assumptions about pointers and arrays and nothing else, so it can be read at any point.

The one genuine overlap is worth naming: the complexity reasoning in chapter 02 — amortised analysis, reading constraints backwards, knowing what an operation actually costs — is the same skill that tells you whether a driver's hot path is acceptable. That chapter earns its place regardless of whether you are interviewing.

Milestones worth naming

- ✓ **Junior:** you can write a working driver by adapting an existing one, and debug it with prints and a datasheet.
- ✓ **Intermediate:** you can read an unfamiliar subsystem and explain its locking, and you reach for the right tool before the debugger.
- ✓ **Senior:** you can look at an unfamiliar SoC's technical reference manual and, in an hour, identify its interconnect topology, DMA masters, isolation blocks and boot flow.
- ✓ **Beyond:** other people bring you the bugs nobody else could reproduce, and you have a method rather than a hunch.

Prerequisites & self-assessment

Be honest with this page. Starting the ARM track without solid C is the single most common way people waste three months. Everything here is teachable — but only in the right order.

The three hard requirements

1. C, at a working level

Not "I have written C". You need pointers to be boring. If any line below makes you pause, do the C track first.

```
int   a[5];
int  *p    = a;          /* what is the type of a here, and why?          */
int (*q)[5] = &a;        /* how does q differ from p?                     */
int  *r    = a + 2;      /* what address is this, in bytes?               */

char  s[] = "hi";        /* sizeof(s) is 3, not 2 — why?                  */
char *t   = "hi";        /* writing t[0] = 'H' is undefined — why?        */

struct f { uint8_t a; uint32_t b; };   /* sizeof is 8, not 5 — why?       */

void (*cb)(int) = handler;             /* declare an array of these       */

uint32_t v = 1u << 31;   /* now do 1 << 31 with a signed int. What breaks? */
```

You must be able to	Covered in
Read and write pointer declarations, including pointers to functions and to arrays	C 04, 05, 12
Predict struct size and layout from member order	C 07
Use `&`, `|`, `^`, `~`, `<<`, `>>` fluently and safely	C 08
Explain what `const`, `static` and `volatile` each mean in every position	C 09, 10
Say why signed overflow is undefined and unsigned overflow is not	C 15

2. Hexadecimal and binary fluency

You should read `0x4001_0080` and think "offset `0x80` inside the block at `0x4001_0000`" without pausing. Try these in your head:

Question	Answer
How many bytes is `0x1000`?	4096 — one page
`0xFFFF_FFFF` as an unsigned 32-bit value?	4,294,967,295 — about 4 G
Which bit does `0x0080` set?	Bit 7
`1 << 20` bytes is?	1 MiB
Mask for bits 11:4?	`0x0FF0`
Is `0x8000_0000` negative as a signed 32-bit value?	Yes — it is `INT32_MIN`

If those were slow, chapters CA 01 and CA 02 fix it in an afternoon.

3. A command line

You will live in a shell, even for Windows driver work. You need: navigating and searching a filesystem, pipes and redirection, `grep`, `find`, editing a file over SSH, and `git` at the level of clone, branch, diff, log, bisect.

Helpful but not required

Background	Where it helps	If you lack it
Any digital logic (gates, flip-flops)	Computer architecture 04–07	Those chapters teach it from zero
Basic electronics (voltage, current, pull-ups)	Board bring-up, signal debugging	Learn it when you first meet a scope
Assembly, any architecture	ARM track, compiler output	The ARM track teaches A64 from scratch
Linear algebra and calculus	AI track	AI chapters 02–11 teach exactly what is needed
Python	AI track experiments, build scripting	Pick it up alongside; it is not load-bearing elsewhere
A university OS course	Operating systems track	That track is written to replace one

> **WHAT YOU EXPLICITLY DO NOT NEED**
>
> A computer science degree. Prior kernel experience. Owning hardware. Knowing C++ or Rust. Familiarity with any specific SoC vendor. Every one of those is either taught here or genuinely optional.

The self-assessment

Score yourself honestly. Each question is answerable in two or three sentences.

Section A — C (must pass 5 of 6)

1. What is the difference between `char *p` and `char p[]` as a function parameter?
2. Why does `sizeof(struct {char a; int b;})` exceed the sum of its members?
3. What does `static` mean on a file-scope function, and on a local variable?
4. Give a concrete bug caused by omitting `volatile`.
5. What is wrong with `if (x = 5)`, and with `a[i++] = i;`?
6. How do you return two values from a C function? Give two different approaches and their trade-offs.

Section B — numbers and bits (must pass 3 of 4)

1. Convert `0b1011_0110` to hex and to decimal.
2. Write an expression that clears bits 5:3 of `reg` and sets them to the value `n`.
3. What is `-1` as an eight-bit two's-complement pattern, and why that pattern?
4. Why can `0.1 + 0.2 != 0.3` in floating point?

Section C — tools (must pass 3 of 4)

1. What does a linker do that a compiler does not?
2. What is in a `.o` file that is not in a `.c` file?
3. How would you find every use of a symbol across a 20-million-line source tree?
4. What does `git bisect` do, and when is it the right tool?

Result	Start at
Passed all three sections	Computer architecture 01 — or skim CA 01–03 and start at CA 04
Failed Section A	The C track, all of it, before anything else
Failed Section B only	CA 01–03, then continue normally
Failed Section C only	Continue, but read compiler chapters 01, 11, 12 early
Failed two or more sections	Spend a month on C and the shell. Come back. It will be a much better month than pushing on.

IF YOU ARE COMING FROM WEB OR APPLICATION DEVELOPMENT

Your instincts will fight you in three specific places, and it helps to name them in advance. **Memory is not free and not managed** — you allocate, you free, and you own the consequences. **There is no exception to catch** — a bad pointer is a crashed machine, not a stack trace. And **the abstraction is not the machine** — you will spend real time reasoning about caches, alignment and instruction ordering that a higher-level runtime hid from you. None of this is harder; it is just closer to the metal, and the feedback is less forgiving.

Setting up your workbench

Every exercise and project on this site runs without buying hardware. This page gets you a cross-compiler, an emulated ARM machine, a kernel-development host and a Windows driver target — in about an hour.

What you are building

```
YOUR MACHINE (any OS)
  │
  ├── a Linux environment            ← native, VM, or WSL2
  │     ├── native gcc + gdb          for C, C++, compiler exercises
  │     ├── aarch64 cross-compiler    for ARM assembly and bare metal
  │     ├── QEMU (system + user)      an emulated ARM64 machine
  │     └── kernel headers + build    for the Linux driver track
  │
  ├── a disposable Windows VM        ← only for the Windows track
  │     ├── test signing enabled
  │     └── kernel debugging to the host
  │
  └── optional: one real board       ← after you have done the emulated work
```

A Linux environment

Pick whichever is least friction for you. All three work for everything on this site except real hardware bring-up.

Option	Good	Watch out for
Native Linux	Fastest; USB and JTAG pass-through just work	Do not develop kernel modules on the machine you need working
A VM (VirtualBox, VMware, UTM, Hyper-V)	Snapshots. Break the kernel, roll back in seconds.	Give it 4+ cores and 8 GB if you will build kernels
WSL2 (Windows)	Zero-friction on a Windows host; excellent for C, C++, Rust, QEMU, compiler work	It runs Microsoft's kernel — you cannot `insmod` your own module without building a custom WSL kernel

```
Debian / Ubuntu — everything the site needs
```

```
# toolchain and basics
sudo apt update
sudo apt install -y build-essential git gdb make cmake pkg-config \
                    bison flex libssl-dev libelf-dev bc \
                    device-tree-compiler cscope universal-ctags

# ARM cross-compilers
sudo apt install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu \
                    gcc-arm-none-eabi          # bare metal / Cortex-M

# emulation
sudo apt install -y qemu-system-arm qemu-user-static

# kernel build (for the Linux driver track)
sudo apt install -y linux-headers-$(uname -r)

# nice to have
sudo apt install -y clang lld llvm valgrind strace ltrace linux-perf
```

Fedora equivalent

```
sudo dnf groupinstall -y "Development Tools"
sudo dnf install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu \
                    arm-none-eabi-gcc-cs qemu-system-aarch64 \
                    kernel-devel kernel-headers dtc cscope ctags \
                    openssl-devel elfutils-libelf-devel bc
```

Verify it

```
aarch64-linux-gnu-gcc --version
qemu-system-aarch64 --version
ls /lib/modules/$(uname -r)/build          # must exist for module builds

# a 30-second end-to-end check: compile for ARM64, run it under emulation
printf '#include <stdio.h>\nint main(void){printf("hello from %%s\\n","aarch64");}\n' > t.c
aarch64-linux-gnu-gcc -static t.c -o t
qemu-aarch64-static ./t
#   hello from aarch64
```

> **WHAT THE NAMING MEANS**
>
> A cross-compiler is `<arch>-<vendor>-<os>-<abi>-gcc`. `aarch64-linux-gnu-` targets 64-bit ARM running Linux with glibc. `aarch64-none-elf-` targets 64-bit ARM with *no OS* — bare metal. `arm-none-eabi-` is the 32-bit Cortex-M standard. Picking the wrong one produces link errors about missing `_start` or a libc that should not exist.

An emulated ARM64 machine

QEMU's `virt` machine is a synthetic ARM64 platform with a GIC, PCIe, an SMMU and virtio devices. It is where you will run kernels, examine exception levels, and single-step boot code.

```
# boot a kernel you built (Linux track, chapter 16)
qemu-system-aarch64 \
  -M virt -cpu cortex-a57 -smp 2 -m 1024 \
  -kernel arch/arm64/boot/Image \
  -initrd rootfs.cpio.gz \
  -append "console=ttyAMA0 earlycon" \
  -nographic

# add an emulated SMMU (security track)
  -M virt,iommu=smmuv3

# start at EL3 with firmware, rather than jumping straight to the kernel
  -M virt,secure=on,virtualization=on

# wait for a debugger before executing the first instruction
  -s -S
# then, in another terminal:
aarch64-linux-gnu-gdb vmlinux -ex 'target remote :1234' -ex 'hbreak start_kernel'
```

`-NOGRAPHIC` AND HOW TO GET OUT

`-nographic` puts the guest serial console on your terminal. To quit, press `Ctrl-A` then `x`. `Ctrl-C` goes to the guest, not to QEMU — this catches everyone once.

QEMU HAS A TRACK OF ITS OWN

The lines above are enough to get started. When you need to know what `-M virt` actually builds, how to wire a device onto a bus, why your guest has no network, or how to debug firmware and kernel together through one GDB session, the QEMU track covers it properly — including building a root filesystem, writing your own device model, and a ready-made command line for every track on this site.

Bare metal without an OS

For the ARM assembly chapters you want to run a handful of instructions with nothing underneath. This is the smallest possible setup.

`boot.S — prints a character and halts, on QEMU virt`

```
    .section .text
    .global _start
_start:
    ldr     x0, =0x09000000     // QEMU virt PL011 UART data register
    mov     w1, #'A'
    str     w1, [x0]            // memory-mapped I/O: one store = one character
1:  wfi                         // wait for interrupt = park the core
    b       1b
```

`link.ld and the build`

```
ENTRY(_start)
SECTIONS { . = 0x40000000; .text : { *(.text) } }
```

```
aarch64-none-elf-as boot.S -o boot.o        # or aarch64-linux-gnu-as
aarch64-none-elf-ld -T link.ld boot.o -o boot.elf
qemu-system-aarch64 -M virt -cpu cortex-a57 -nographic -kernel boot.elf
#   A

# inspect what you built
aarch64-none-elf-objdump -d boot.elf
aarch64-none-elf-readelf -S boot.elf
```

That is a complete embedded system: reset vector, memory-mapped I/O, and a parked core. Every chapter in the ARM track is an expansion of one of those three ideas.

Rust and C++ toolchains

```
# Rust, including a bare-metal ARM target (Rust track)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup target add aarch64-unknown-none          # bare metal
rustup target add thumbv7em-none-eabihf         # Cortex-M4F
rustup component add rust-src llvm-tools
cargo install cargo-binutils probe-rs-tools

# C++: nothing extra, but you want a recent compiler for the modern chapters
g++ --version        # 13+ for good C++20/23 coverage
clang++ --version
```

A Windows driver target

Only needed for the Windows track. Two machines: a host with the tools, and a disposable VM that you will bug-check repeatedly.

1. **On the host:** Visual Studio 2022 with "Desktop development with C++", then the Windows SDK, then the WDK — *in that order*, matching versions. The WDK adds the driver project templates.
2. **Install WinDbg:** `winget install Microsoft.WinDbg`
3. **Create the test VM** (Hyper-V, VMware or VirtualBox) and **take a snapshot immediately**. You will restore it often.
4. **On the VM,** enable test signing and kernel debugging, then reboot.

```
on the TEST VM, elevated
```

```
bcdedit /set testsigning on
bcdedit /debug on
bcdedit /dbgsettings net hostip:192.168.1.10 port:50000 key:1.2.3.4
shutdown /r /t 0
```

```
on the HOST
```

```
windbg -k net:port=50000,key=1.2.3.4
.sympath srv*C:\symbols*https://msdl.microsoft.com/download/symbols;C:\myproject\x64\Debug
.reload /f
```

> **TEST SIGNING IS A DEVELOPMENT-ONLY SETTING**
> It puts a watermark on the desktop and weakens the machine's driver integrity guarantees. Enable it only on a disposable VM, never on a machine you rely on.

Navigating large source trees

The Linux kernel is over 20 million lines. Grep alone will not do. Set up one of these before you start the Linux track — it changes the experience completely.

Tool	Setup	Strength
Elixir Bootlin	None — it is a website	Cross-referenced identifiers across every kernel version. Start here.
cscope	`make cscope` in the kernel tree	"Who calls this?" and "Where is this assigned?"
ctags	`make tags`	Jump to definition in vim/emacs
clangd	`make compile_commands.json` (kernel has a script)	Full IDE features in VS Code or any LSP editor
ripgrep	`cargo install ripgrep`	Fast enough that grepping the whole tree becomes a reflex

If you do want hardware

Buy *one* board, after you have completed at least one emulated project. A logic analyser is worth more than a second board.

Board	Best for
Raspberry Pi 4 / 5	Embedded Linux, device tree, real drivers, GPIO. The default first board.
BeagleBone Black	Excellent documentation and a clean mainline path
STM32 Nucleo / Discovery	Bare metal, Cortex-M TrustZone, on-board SWD debugger
NXP i.MX8 or ST STM32MP1 kit	A real secure world: TF-A, OP-TEE, and a resource-domain controller to study
Any 8-channel logic analyser	Seeing what your code actually does on the wire

Ready when

- ✓ `aarch64-linux-gnu-gcc` compiles a hello-world and `qemu-aarch64-static` runs it.
- ✓ The bare-metal `boot.S` above prints `A` under `qemu-system-aarch64`.
- ✓ `ls /lib/modules/$(uname -r)/build` shows a directory (Linux track only).
- ✓ WinDbg connects to your test VM and `.reload /f` succeeds (Windows track only).
- ✓ You can jump to the definition of `kmalloc` in a kernel tree in under five seconds.

TRACK

Computer architecture

How a computer works, from a single transistor-level abstraction up to a multi-core memory hierarchy. Everything else on this site assumes this track.

20 chapters

Binary, hex and number representation

A computer stores no numbers. It stores voltages, which we agree to read as bits, which we agree to group and interpret. Every bug in this chapter's territory comes from two pieces of code disagreeing about that interpretation.

The problem

A transistor is reliably good at one thing: being clearly on or clearly off. It is bad at holding "3.7 volts, meaning the digit 7". Noise, temperature and manufacturing variation smear analogue values; they barely touch a signal that only has to be distinguishably high or low.

So the machine gets two symbols. Everything else — integers, text, colours, machine instructions, your source code — is a convention layered on top of groups of those two symbols.

> **THE CENTRAL IDEA OF THIS WHOLE TRACK**
> A pattern of bits has **no inherent meaning**. `0100 0001` is 65, or the letter `A`, or a fragment of an ARM instruction, or a pixel's red channel. What it means is decided entirely by the code that reads it. "Type" in a programming language is nothing more than a compiler-enforced agreement about which interpretation applies.

Positional notation, generalised

Decimal is positional: each digit is worth its face value times a power of ten. Binary is the same system with base two, and hex with base sixteen.

```
Decimal 205
    2 × 10²  +  0 × 10¹  +  5 × 10⁰   =  200 + 0 + 5      = 205

Binary 1100 1101
    1×2⁷ 1×2⁶ 0×2⁵ 0×2⁴ 1×2³ 1×2² 0×2¹ 1×2⁰
    128 + 64 +  0 +  0 +  8 +  4 +  0 +  1              = 205

Hex 0xCD
    C × 16¹  +  D × 16⁰   =  12×16 + 13                 = 205

Same value. Three notations. The machine only ever has the middle one.
```

Why hexadecimal exists

Hex is not a third number system the machine understands — it is a shorthand for humans, chosen because **16 = 2^4**. One hex digit is exactly four bits, always, with no carrying between them. That makes the conversion mechanical rather than arithmetic:

```
  1100 1101 0011 1010
  ──┬─ ──┬─ ──┬─ ──┬──
    C    D    3    A     →  0xCD3A

Read four bits, write one hex digit. Never any arithmetic. This is why
every datasheet, debugger and hex dump uses hex and not decimal: you can
see the bits through it.
```

Hex	Binary	Dec	Hex	Binary	Dec
0	`0000`	0	8	`1000`	8
1	`0001`	1	9	`1001`	9
2	`0010`	2	A	`1010`	10
3	`0011`	3	B	`1011`	11
4	`0100`	4	C	`1100`	12
5	`0101`	5	D	`1101`	13
6	`0110`	6	E	`1110`	14
7	`0111`	7	F	`1111`	15

Memorise this table. Not because it is deep, but because fluency here is the difference between reading a register dump at a glance and translating it laboriously. It takes about twenty minutes to internalise and pays back for a career.

Widths and units

Name	Bits	Hex digits	Unsigned range	C type (typical)
Nibble	4	1	0 – 15	—
Byte	8	2	0 – 255	`uint8_t`
Halfword	16	4	0 – 65,535	`uint16_t`
Word	32	8	0 – 4,294,967,295	`uint32_t`
Doubleword	64	16	0 – ~1.8×10^{19}	`uint64_t`

> **"WORD" IS NOT A FIXED SIZE**
> In ARM documentation a word is 32 bits, always. In x86 documentation a word is 16 bits (for historical reasons) and 32 bits is a "dword". In C, `int` is whatever the platform chose. Never rely on the word "word" across ecosystems — say the bit count.

The powers you should know cold

Power	Value	Meaning in practice
2^{8}	256	One byte's worth of distinct values
2^{10}	1,024	1 KiB
2^{12}	4,096	**4 KiB — the standard page size**
2^{16}	65,536	A 16-bit address space; also the 64 KiB boundary DMA engines care about
2^{20}	1,048,576	1 MiB
2^{30}	~1.07×10^{9}	1 GiB
2^{32}	~4.29×10^{9}	4 GiB — the limit of a 32-bit address
2^{40}	~1.10×10^{12}	1 TiB; a common physical address width on ARM SoCs
2^{48}	~2.81×10^{14}	256 TiB — the default AArch64 virtual address size

KIB VS KB

KiB, MiB, GiB are powers of two (1024, 1048576, …). **KB, MB, GB** are strictly powers of ten (1000, 1000000, …). Memory is sold and addressed in powers of two; storage and network rates are usually quoted in powers of ten. This is why a "1 TB" disk shows as 931 GiB — nothing is missing, the units differ by 10%.

In practice: reading an address

Here is the skill this chapter is actually for. Given a peripheral base of `0x4001_0000` and a register at offset `0x0C`:

```
  0x4001_0000     base of the GPIO block
+ 0x0000_000C     offset of the DATA register
──────────────
  0x4001_000C     the address you load from

Now read a value out of it:  0x0000_00A5

  0x A       5
    1010   0101
    ||||   ||||
    ||||   |||└─ bit 0 = 1
    ||||   ||└── bit 1 = 0
    ||||   |└─── bit 2 = 1
    ||||   └──── bit 3 = 0
    |||└──────── bit 4 = 1
    ||└───────── bit 5 = 0
    |└────────── bit 6 = 1
    └─────────── bit 7 = 0

  "Pins 0, 2, 5 and 7 are high."   ← the whole point
```

Notice how much harder that would be if the register had been printed as decimal 165. Hex is chosen precisely so bit positions are visible.

Writing literals in C

```c
uint32_t a = 205;          /* decimal                                    */
uint32_t b = 0xCD;         /* hex — the default for anything hardware    */
uint32_t c = 0b11001101;   /* binary — C23; a GCC/Clang extension before */
uint32_t d = 0315;         /* OCTAL — a leading zero, almost never wanted */

/* digit separators, C23 and C++14, make long constants readable */
uint32_t page  = 0x0000'1000;
uint32_t mmio  = 0x4001'0080;
```

THE LEADING-ZERO TRAP

`int mode = 0755;` is octal (493 decimal) — which is what you want for a Unix file mode and almost never what you want anywhere else. `uint8_t id = 011;` is 9, not 11. A leading zero silently changes the base. This has caused real outages in configuration parsing and in IP address handling.

The same bits, different meanings

To make the opening claim concrete, take the 32-bit pattern `0x41424344` and ask four different questions of it:

Interpreted as	Value
Unsigned 32-bit integer	1,094,861,636
Signed 32-bit integer	1,094,861,636 (the top bit is 0, so identical)
Four ASCII characters	`"ABCD"`
IEEE 754 single-precision float	≈ 12.1414
An AArch64 instruction	a valid encoding — the CPU would execute it
Four bytes in memory, little-endian	`44 43 42 41` in increasing address order

Nothing in the bits distinguishes these. The distinction lives entirely in the instruction you use to read them and the type your compiler believes they have.

Byte order, briefly

A 32-bit value occupies four consecutive byte addresses. Which byte goes at the lowest address is the machine's **endianness**.

```
Storing 0x12345678 at address 0x1000

  Little-endian (ARM in practice, x86 always)
    0x1000: 78   ← least significant byte first
    0x1001: 56
    0x1002: 34
    0x1003: 12

  Big-endian ("network byte order", some legacy and some MIPS/PowerPC)
    0x1000: 12   ← most significant byte first
    0x1001: 34
    0x1002: 56
    0x1003: 78
```

It matters the moment bytes cross a boundary: a network packet, a file format, a message to a different processor on the same chip. Within one program on one machine you never notice it. This is covered in depth in chapter 08 and again in the C track.

How it fails

Symptom	Cause
A register value looks like nonsense until you print it as hex	Printing hardware state in decimal. Always `%#x` for registers.
A constant is off by a strange factor	Accidental octal from a leading zero
Values are byte-swapped over a link	Endianness mismatch; use explicit conversions, never a raw cast
A size calculation is out by 2.4%	Mixing GB and GiB
A field written to a register lands in the wrong bits	Counting bit positions from 1 instead of 0. **Bit 0 is the least significant bit.**

Check yourself

✓ Convert `0xB6` to binary and to decimal, in your head, in under ten seconds.

- ✓ Explain why hex rather than octal or decimal is used in every datasheet.
- ✓ Say what `0x1000` bytes is, and why that number appears everywhere.
- ✓ Give three different meanings the pattern `0x41424344` could have.
- ✓ Explain the difference between 1 MB and 1 MiB and where each is used.
- ✓ Say which bit `0x0040` sets, counting the way hardware does.

Signed integers and two's complement

Negative numbers do not exist in hardware. What exists is a representation clever enough that the adder does not need to know about them — and understanding why it works explains a whole family of security bugs.

The problem

You have eight bits. That is 256 distinct patterns. You want some of them to mean negative numbers. The naïve approach — steal the top bit as a sign flag — seems obvious:

```
Sign-and-magnitude (the obvious idea, and wrong)

  0000 0101  =  +5
  1000 0101  =  −5     top bit = sign, rest = magnitude

Two problems:
  1. 0000 0000 = +0  and  1000 0000 = −0.  Two zeros. Every comparison
     against zero needs a special case.
  2. Addition needs a subtractor.  5 + (−5) through a plain binary adder
     gives 1000 1010 = −10.  Wrong. The hardware must inspect the signs
     and choose between adding and subtracting.

That second point costs real gates, real area and real time.
```

The mechanism: two's complement

The trick is to choose a representation where **ordinary binary addition just works**, with no sign inspection at all. Represent $-n$ as the pattern that, added to n, produces zero after the carry out of the top bit is discarded.

```
Eight-bit two's complement: the number line wraps

  bit pattern   unsigned   signed
  0000 0000        0          0
  0000 0001        1         +1
  ...
  0111 1110      126       +126
  0111 1111      127       +127     ← largest positive
  1000 0000      128       −128     ← the wrap point
  1000 0001      129       −127
  ...
  1111 1110      254         −2
  1111 1111      255         −1     ← all ones is −1, always

The top bit still tells you the sign. But it is not a flag −
it carries a weight, and that weight is NEGATIVE:

  1111 1111  =  −128 + 64 + 32 + 16 + 8 + 4 + 2 + 1  =  −1
                 ┬┬
                 └└── the only negative weight in the whole number
```

> **THE ONE-SENTENCE DEFINITION**
>
> In an *n*-bit two's complement number, the most significant bit has weight -2^{n-1} and every other bit has its usual positive weight. That is the entire system; everything else follows.

Why addition needs no special case

```
   5 + (−5), eight bits

     0000 0101      ( +5 )
   + 1111 1011      ( −5 )
   ─────────────
   1 0000 0000
   ↑
   └── carry out of bit 7, discarded because the result is 8 bits

     result = 0000 0000 = 0   ✓

   7 + (−3)

     0000 0111      ( +7 )
   + 1111 1101      ( −3 )
   ─────────────
   1 0000 0100  →  0000 0100 = +4   ✓

The adder never looked at a sign. Subtraction is just addition of the
negation, so the CPU needs ONE arithmetic unit for both − which is why
every processor built since the 1960s uses this representation.
```

How to negate

Two equivalent recipes. Both give the same answer; use whichever is faster for you.

```
Method 1: invert all bits, then add one

    +5   =  0000 0101
  invert    1111 1010
  add 1     1111 1011  =  −5

Method 2: copy bits from the right up to and including the first 1,
          then invert everything above it

    +12  =  0000 1100
                  ┘└└  copy "100" unchanged
            1111 0100  =  −12
            └────┘ invert the rest

Sanity check: negating twice must return the original.
    −5  = 1111 1011 → invert 0000 0100 → +1 → 0000 0101 = +5  ✓
```

Ranges and the asymmetry

Width	Signed range	Unsigned range	C types
8	−128 … +127	0 … 255	`int8_t` / `uint8_t`
16	−32,768 … +32,767	0 … 65,535	`int16_t` / `uint16_t`
32	−2,147,483,648 … +2,147,483,647	0 … 4,294,967,295	`int32_t` / `uint32_t`

Width	Signed range	Unsigned range	C types
64	≈ -9.22×10^{18} … $+9.22\times10^{18}$	0 … ≈1.84×10^{19}	`int64_t` / `uint64_t`

> **THE ASYMMETRY THAT BREAKS CODE**
>
> There is one more negative number than positive. There are 128 negatives (−128…−1) and only 127 positives (1…127), because zero occupies a slot on the positive side.
>
> The consequence: `-INT_MIN` **is not representable.** `abs(INT_MIN)` returns `INT_MIN` — still negative. In C this is undefined behaviour, and on ARM the instruction simply produces `INT_MIN` back. Code of the form `if (abs(x) < limit)` can therefore be defeated by passing `INT_MIN`, which has been the root of real vulnerabilities.

Sign extension

When a narrow value is widened, the new high bits must be filled. Filling them with zero is correct for unsigned values and catastrophic for negative signed ones.

```
Widening 8-bit 0xFF (-1 signed, 255 unsigned) to 32 bits

  ZERO extension    0x000000FF  =  255      correct for uint8_t
  SIGN extension    0xFFFFFFFF  =   -1      correct for int8_t

The rule: replicate the top bit of the source into every new bit.

This is why ARM has separate instructions:
  LDRB   Wd, [Xn]     load byte, zero-extend  → unsigned
  LDRSB  Wd, [Xn]     load byte, sign-extend  → signed
  LDRSW  Xd, [Xn]     load 32-bit word, sign-extend to 64

Picking the wrong load instruction is a silent, data-dependent bug:
it only misbehaves when the value happens to have its top bit set.
```

```
/* the classic surprise */
char c = -1;                    /* char may be signed or unsigned!      */
int  i = c;                     /* -1 if signed char, 255 if unsigned   */

/* getchar() returns int, not char, precisely because of this */
int ch;
while ((ch = getchar()) != EOF)   /* EOF is -1; a char cannot hold it     */
        ...                       /*   distinctly from the byte 0xFF      */

/* always be explicit in systems code */
int8_t  signed_byte   = -1;       /* guaranteed signed   */
uint8_t unsigned_byte = 0xFF;     /* guaranteed unsigned */
```

Mixing signed and unsigned: the biggest trap in C

When C compares a signed and an unsigned value of the same rank, the signed operand is **converted to unsigned**. The negative number becomes enormous.

```
int      i = -1;
unsigned u =  1;

if (i < u)                    /* FALSE. i converts to 4294967295 */
        puts("as expected");
else
        puts("surprise");     /* this branch runs */
```

THE BUG THIS CAUSES, IN THE SHAPE IT ACTUALLY APPEARS

```
/* a length check that does not check */
int copy_in(void *dst, const void *src, int len)
{
        if (len > MAX_LEN)            /* len = -1 passes: -1 > 1024 is false */
                return -EINVAL;
        memcpy(dst, src, len);        /* memcpy takes size_t — UNSIGNED.     */
        return 0;                     /* -1 becomes SIZE_MAX. Heap destroyed. */
}
```

This exact pattern — a signed length validated against an upper bound, then passed to an unsigned API — is one of the most common serious vulnerabilities in C code. The fix is to use an unsigned type for the length *and* check both ends of the range.

```
/* the fixed version */
int copy_in(void *dst, const void *src, size_t len)
{
        if (len == 0 || len > MAX_LEN)
                return -EINVAL;
        memcpy(dst, src, len);
        return 0;
}
```

Overflow

```
Signed overflow, 8-bit:

    127 + 1  =  0111 1111 + 0000 0001  =  1000 0000  =  -128

The number line wraps from the largest positive straight to the most
negative. The carry INTO the top bit differs from the carry OUT of it —
that is exactly the hardware condition that sets the V (overflow) flag.

Unsigned overflow, 8-bit:

    255 + 1  =  1111 1111 + 0000 0001  = 1 0000 0000  →  0000 0000

Wraps to zero. Carry out of the top bit sets the C (carry) flag.
```

	Unsigned overflow	**Signed overflow**
Hardware behaviour	Wraps modulo 2^n, sets C	Wraps, sets V
C language status	**Defined** — wrapping is guaranteed	**Undefined behaviour**
What the optimiser may assume	Nothing special	That it never happens — see below

WHY SIGNED OVERFLOW BEING UB MATTERS IN PRACTICE

```
if (x + 1 < x)          /* trying to detect signed overflow */
        handle();       /* GCC deletes this branch entirely at -O2:  */
                        /* signed overflow "cannot happen", so x+1 < x */
                        /* is always false.                          */
```

The check compiles to nothing. Detect overflow with the builtins instead: `__builtin_add_overflow(a, b, &result)` returns true on overflow and is a single instruction on ARM (it reads the V flag). The kernel wraps these as `check_add_overflow()`.

The condition flags

Every arithmetic operation on ARM can set four flags, and their definitions come straight from this chapter:

Flag	Set when	Used for
N — negative	The result's top bit is 1	Signed comparisons
Z — zero	The result is all zero bits	Equality tests
C — carry	Carry out of the top bit (unsigned overflow)	Unsigned comparisons, multi-word arithmetic
V — overflow	Carry into the top bit ≠ carry out of it	Signed overflow detection

This is why ARM has two families of conditional branches: `B.HI` / `B.LO` (unsigned, test C and Z) versus `B.GT` / `B.LT` (signed, test N and V). The comparison instruction is identical; only the interpretation differs — the same theme as chapter 01.

How it fails

Symptom	Cause
A loop `for (unsigned i = n; i >= 0; i--)` never ends	An unsigned value is always ≥ 0; it wraps to `UINT_MAX`
A length check passes for a negative length	Signed/unsigned comparison conversion
A byte read as −1 instead of 255	`char` signedness is implementation-defined; use `uint8_t`
An overflow check is optimised away	Signed overflow is UB; use the compiler builtins
A value is correct on x86 and wrong on ARM	Plain `char` is signed on x86, unsigned on ARM
A timestamp difference goes hugely positive after wrap	Comparing absolute unsigned counters instead of differences; use `(int32_t)(a - b) > 0`

Check yourself

- ✓ Write −7 as an 8-bit pattern, both ways, and verify by adding it to +7.
- ✓ Explain why the top bit is not a flag but a negative weight.
- ✓ Say why there is one more negative value than positive, and name a bug it causes.
- ✓ Explain what `LDRB` does that `LDRSB` does not, and when choosing wrong is silent.
- ✓ Predict the output of `if (-1 < 1u)` and justify it from the conversion rules.
- ✓ State the difference between C and V, and which comparison instructions use each.

Floating point and IEEE 754

Floating point trades exactness for range. Understanding the trade precisely is what separates "why is my total off by a cent" from a correct design — and it is unavoidable knowledge for the AI track, where the entire performance story is about shrinking these formats.

The problem

Integers are exact but their range is fixed by their width. A 32-bit integer cannot hold the mass of an electron and the mass of the sun. Scientific notation solves this for humans:

```
   6.022 × 10²³        significand × base^exponent
   ─┬──   ─┬─┘
    │       └── move the point this many places
    └──────────── the significant digits

The insight: store the digits and the exponent separately. The point
"floats" to wherever the exponent puts it. Precision stays constant
(a fixed number of significant digits) while range becomes enormous.
```

IEEE 754 is that idea in binary, standardised in 1985 so every machine produces bit-identical results. Before it, the same program gave different answers on different hardware.

The format

```
Single precision (float, 32 bits)

  31  30      23 22                                    0
 ┌───┬──────────┬──────────────────────────────────────┐
 │ S │ exponent │            fraction                  │
 └───┴──────────┴──────────────────────────────────────┘
   1      8                     23 bits

Double precision (double, 64 bits)

  63  62         52 51                                  0
 ┌───┬─────────────┬──────────────────────────────────────┐
 │ S │  exponent   │            fraction                  │
 └───┴─────────────┴──────────────────────────────────────┘
   1       11                    52 bits

value = (−1)^S  ×  1.fraction  ×  2^(exponent − bias)

  bias = 127 for float, 1023 for double
```

The three parts

- **Sign (S)** — one bit, 0 for positive. Unlike integers, this *is* a plain flag, which is why floating point has both `+0.0` and `-0.0`.
- **Exponent** — stored with a *bias* rather than as two's complement, so that comparing two positive floats as if they were integers gives the right answer. A stored 130 with bias 127 means an actual exponent of +3.
- **Fraction (mantissa)** — the digits after an *implied* leading 1. Since any normalised binary number starts with 1, storing that bit would waste it. This "hidden bit" buys one extra bit of precision for free.

Worked example: encoding −6.25

```
1. Sign:  negative  →  S = 1

2. Convert the magnitude to binary
     6.25 = 110.01₂          (4 + 2 + 0.25)

3. Normalise: shift until there is exactly one 1 before the point
     110.01  =  1.1001 × 2²

4. Exponent: actual 2, stored = 2 + 127 = 129 = 1000 0001₂

5. Fraction: the bits after the leading 1, padded to 23 bits
     1001 0000 0000 0000 0000 000

6. Assemble
     1 | 1000 0001 | 100 1000 0000 0000 0000 0000
     = 0xC0C80000

Check it:  printf("%#x\n", *(uint32_t*)&(float){-6.25f});
```

Special values

Two exponent patterns are reserved, which is how the format encodes infinities, NaNs and very small numbers.

Exponent	Fraction	Means
All zeros	Zero	`±0.0` — the sign bit still applies
All zeros	Non-zero	**Subnormal**: no hidden bit, fills the gap around zero at reduced precision
1 … 254 (float)	Any	Normal number, as described above
All ones	Zero	`±Infinity`
All ones	Non-zero	**NaN** — Not a Number

```
1.0 / 0.0   →  +inf          /* not a trap by default: a value      */
-1.0 / 0.0  →  -inf
0.0 / 0.0   →  NaN
sqrt(-1.0)  →  NaN
inf - inf   →  NaN

/* NaN is not equal to anything, INCLUDING ITSELF */
NaN == NaN  →  false
NaN != NaN  →  true          /* the standard idiom for detecting it  */
isnan(x)                     /* the readable way                      */

/* which is why this is a real bug pattern */
if (x == x) { /* normally always true; false exactly when x is NaN */ }
```

> **NAN POISONS EVERYTHING DOWNSTREAM**
>
> Any arithmetic involving a NaN produces a NaN. A single bad sample in a signal-processing chain, or one uninitialised weight in a neural network, silently turns an entire output tensor into NaNs with no error raised anywhere. This is why "loss went to NaN" is the most common training failure in machine learning — see AI chapter 15.

Precision: what you actually get

Format	Bits (S/E/F)	Decimal digits	Max value	Typical use
`float` (FP32)	1 / 8 / 23	≈ 7	≈ 3.4×10^{38}	Graphics, DSP, default ML training
`double` (FP64)	1 / 11 / 52	≈ 16	≈ 1.8×10^{308}	Scientific computing, the C default
`_Float16` (FP16)	1 / 5 / 10	≈ 3	65,504	ML inference, graphics
`bfloat16`	1 / 8 / 7	≈ 2	≈ 3.4×10^{38}	**ML training** — FP32's range with half the bits
FP8 (E4M3 / E5M2)	1 / 4 / 3 or 1 / 5 / 2	≈ 1	448 / 57,344	Large-model inference

> **WHY BFLOAT16 BEAT FLOAT16 FOR MACHINE LEARNING**
>
> Both are 16 bits. FP16 spends 10 bits on precision and only 5 on exponent; bfloat16 spends 7 on precision and 8 on exponent — exactly FP32's exponent field. Training cares far more about *range* (gradients spanning many orders of magnitude, which underflow to zero in FP16) than about the third decimal place. bfloat16 also converts to and from FP32 by truncating or padding, with no arithmetic at all.

The thing that surprises everyone

Binary fractions can only represent sums of negative powers of two. `0.5`, `0.25` and `0.75` are exact. `0.1` is not — in binary it is a repeating fraction, exactly as 1/3 is in decimal.

```
0.1 in binary =  0.0001100110011001100110011...  (repeating "0011")

Stored as the nearest float:
    0.100000001490116119384765625

Stored as the nearest double:
    0.1000000000000000055511151231257827021181583404541015625

Therefore:
    0.1 + 0.2  ==  0.30000000000000004     not 0.3
    (0.1 + 0.2) == 0.3   →   FALSE
```

```c
/* NEVER compare floats for equality */
if (a == b)                                  /* wrong */

/* absolute tolerance — fine when the scale is known */
if (fabs(a - b) < 1e-9)

/* relative tolerance — correct in general */
if (fabs(a - b) <= 1e-9 * fmax(fabs(a), fabs(b)))

/* money: do not use floating point at all */
int64_t cents = 1999;                        /* store the smallest unit  */
```

Precision is not uniform

Because the exponent scales the whole value, the gap between representable numbers grows with magnitude. This is the most under-appreciated property of the format.

Around this value (float)	Gap to the next representable number
1.0	$\approx 1.2 \times 10^{-7}$
1,000	$\approx 6.1 \times 10^{-5}$
1,000,000	≈ 0.0625
16,777,216 (2^{24})	**2.0** — consecutive integers stop being representable
10^9	≈ 64

```
float x = 16777216.0f;      /* 2^24 */
x = x + 1.0f;               /* x is STILL 16777216.0f — the +1 vanishes */

/* the same effect in a loop: this never terminates */
for (float f = 0; f != 20000000.0f; f += 1.0f)
        ;                   /* increments stop having any effect        */
```

ABSORPTION AND CATASTROPHIC CANCELLATION

Absorption: adding a small number to a large one loses the small one entirely, as above. Summing a million small values into a running total therefore loses accuracy progressively — sum smallest first, or use Kahan summation.

Cancellation: subtracting two nearly equal numbers destroys the significant digits. `(1.0000001f - 1.0f)` keeps almost no meaningful precision, and any subsequent division amplifies the error. Rearranging the algebra to avoid the subtraction is the fix.

Floating point in hardware

An FPU is a large, power-hungry block. Whether one exists, and whether you may use it, is a real design decision:

Context	Situation
Cortex-A (application)	FPU and NEON always present; freely usable in user space
Cortex-M0/M3	No FPU. Floating point is emulated in software — hundreds of cycles per operation.
Cortex-M4F / M7	Single-precision FPU; double precision still emulated
Linux kernel	**Floating point is effectively forbidden.** FP registers are not saved on kernel entry, so using them corrupts user-space state. `kernel_fpu_begin()` exists but is heavyweight and reserved for special cases.
Interrupt handlers	Avoid. FP context save/restore adds latency to every interrupt.

```
# ARM ABI variants — a mismatch produces link errors or silent corruption
arm-none-eabi-gcc -mfloat-abi=soft      # no FPU; library calls, FP args in integer regs
arm-none-eabi-gcc -mfloat-abi=softfp    # uses the FPU, but passes FP args in integer regs
arm-none-eabi-gcc -mfloat-abi=hard      # uses the FPU and passes FP args in FP registers

# deterministic-behaviour flags worth knowing
-ffast-math          # allows reassociation; BREAKS IEEE semantics. Read before enabling.
-ffp-contract=off    # stop the compiler fusing a*b+c into one FMA (changes results)
```

FIXED POINT: THE USUAL EMBEDDED ANSWER

When there is no FPU, represent fractions as scaled integers. In Q16.16 format, a 32-bit integer holds 16 integer bits and 16 fraction bits — the value is simply the integer divided by 65,536. Addition is plain integer addition; multiplication needs a shift afterwards. It is exact, fast, and deterministic, which is why audio, motor control and DSP code often uses it even on chips that do have an FPU.

```
typedef int32_t q16;
#define Q16(x)      ((q16)((x) * 65536.0))
#define Q16_MUL(a,b) ((q16)(((int64_t)(a) * (b)) >> 16))
#define Q16_ADD(a,b) ((a) + (b))
```

How it fails

Symptom	Cause
A total is off by a fraction of a cent	Money in floating point. Use integer minor units.
`a == b` fails for values that print identically	Printing rounds; the stored values differ. Compare with a tolerance.
A loop never terminates	Float loop counter reaching the absorption limit
Everything becomes NaN after one bad input	NaN propagation; validate inputs and check with `isnan()`
Results differ between debug and release builds	`-ffast-math`, FMA contraction, or x87 excess precision
User-space maths breaks after loading a driver	Floating point used in kernel code without saving FP state
ML training loss becomes NaN or stops improving	FP16 overflow or gradient underflow; use bfloat16 or loss scaling
Sudden 100× slowdown near zero	Subnormal numbers taking a microcode path; enable flush-to-zero if acceptable

Check yourself

- ✓ Encode −6.25 as an IEEE 754 single and verify the hex.
- ✓ Explain the hidden bit and what it buys.
- ✓ Say why `0.1 + 0.2 != 0.3` without using the word "rounding" alone.
- ✓ Explain why the gap between representable floats grows with magnitude, and give a bug it causes.
- ✓ State why bfloat16 is preferred to float16 for training.
- ✓ Say why the Linux kernel forbids floating point in ordinary driver code.
- ✓ Describe Q16.16 and how to multiply two values in it.

Boolean algebra and logic gates

Everything a processor does is built from a handful of operations on single bits. This chapter is the bridge from "a transistor is a switch" to "a machine can add" — and it is also where the bit-manipulation idioms you will write every day come from.

The problem

You have bits. You need to compute with them. What is the smallest set of operations that lets you build *any* function of bits at all — arithmetic, comparison, memory addressing, instruction decoding?

The answer, from George Boole in 1847 and Claude Shannon in 1937, is remarkably small: AND, OR and NOT suffice. In fact a single gate type suffices, which is why chips are physically built from just one or two.

The gates

Gate	C operator	Truth table (A, B → Y)	Says
NOT	`~` / `!`	0→1, 1→0	Invert
AND	`&`	00→0, 01→0, 10→0, 11→1	Both
OR	`|`	00→0, 01→1, 10→1, 11→1	Either
XOR	`^`	00→0, 01→1, 10→1, 11→0	Different
NAND	`~(a & b)`	00→1, 01→1, 10→1, 11→0	Not both
NOR	`~(a | b)`	00→1, 01→0, 10→0, 11→0	Neither

NAND IS UNIVERSAL

Every one of the other gates can be built from NAND alone:

```
NOT a       = a NAND a
a AND b     = NOT(a NAND b)        = (a NAND b) NAND (a NAND b)
a OR  b     = (NOT a) NAND (NOT b) = (a NAND a) NAND (b NAND b)
```

This matters physically: in CMOS, NAND and NOR are the *natural* gates — they need four transistors — while AND and OR are built as NAND followed by an inverter and therefore cost more. Synthesis tools rewrite your logic into NANDs and NORs for exactly this reason.

Why XOR earns its own gate

XOR is "these bits differ". That single property makes it the workhorse of arithmetic, error detection and cryptography:

```
a ^ 0 == a          /* identity: XOR with zero changes nothing           */
a ^ a == 0          /* self-inverse: the basis of parity and of one-time pads */
a ^ b ^ b == a      /* apply twice and you are back — reversible encryption  */

/* toggle exactly the bits set in a mask, leaving others alone */
reg ^= (1u << 3);   /* flip bit 3 */

/* the sum bit of a binary adder IS xor — see chapter 05 */
/* parity of a word: */
int parity(uint32_t v) { return __builtin_parity(v); }
```

The algebra

These identities are not academic — they are the rewrites the compiler and the synthesis tool perform, and knowing them lets you simplify register manipulation by hand.

Law	AND form	OR form
Identity	`a & 1 = a`	`a | 0 = a`
Null	`a & 0 = 0`	`a | 1 = 1`
Idempotence	`a & a = a`	`a | a = a`
Complement	`a & ~a = 0`	`a | ~a = 1`
Commutative	`a & b = b & a`	`a | b = b | a`
Distributive	`a & (b | c) = (a & b) | (a & c)`	
Absorption	`a | (a & b) = a`	
Involution	`~~a = a`	

De Morgan's laws

```
   ~(a & b)  ==  (~a) | (~b)
   ~(a | b)  ==  (~a) & (~b)

"Not both"    is    "either not"
"Neither"     is    "not either"
```

These are how a NAND-only fabric implements OR, and how a compiler rewrites your conditionals. They also catch a very common logic bug:

```
/* "if the flag is not READY and not ERROR" */
if (!(f & READY) && !(f & ERROR))     /* correct                          */
if (!(f & (READY | ERROR)))            /* also correct — De Morgan          */
if (!(f & READY & ERROR))              /* WRONG: true unless BOTH are set   */
```

In practice: the five bit idioms

Every driver you write uses these. They follow directly from the identity and null laws above.

Goal	Idiom	Why it works
Set bits	`reg |= mask;`	`x | 1 = 1`, `x | 0 = x` — masked bits forced high, others untouched

Goal	Idiom	Why it works
Clear bits	`reg &= ~mask;`	`x & 0 = 0`, `x & 1 = x`
Toggle bits	`reg ^= mask;`	`x ^ 1 = ~x`, `x ^ 0 = x`
Test bits	`if (reg & mask)`	Non-zero exactly when at least one masked bit is set
Extract a field	`(reg >> shift) & ((1u << width) - 1)`	Shift down, then mask off everything above the field
Insert a field	`reg = (reg & ~(m << s)) | ((v & m) << s);`	Clear first, then OR in — never OR without clearing

`the field macros every embedded codebase reinvents`

```
#define BIT(n)              (1u << (n))
#define GENMASK(h, l)       (((1u << ((h) - (l) + 1)) - 1) << (l))
#define FIELD_GET(mask, v)  (((v) & (mask)) / ((mask) & -(mask)))
#define FIELD_PREP(mask, v) (((v) * ((mask) & -(mask))) & (mask))

/* mask & -mask isolates the lowest set bit, which equals 1 << shift -
   so dividing by it is the same as shifting down, with no separate
   shift constant to keep in sync. This is the Linux kernel's trick. */

#define CTRL_MODE   GENMASK(5, 3)    /* bits 5:3 → 0x38 */
#define CTRL_ENABLE BIT(0)

uint32_t r = readl(base + CTRL);
uint32_t mode = FIELD_GET(CTRL_MODE, r);           /* read the field   */
r &= ~CTRL_MODE;                                    /* clear it         */
r |= FIELD_PREP(CTRL_MODE, 3) | CTRL_ENABLE;        /* set mode=3, on   */
writel(r, base + CTRL);
```

READ-MODIFY-WRITE IS NOT ALWAYS SAFE

`reg |= BIT(3)` is three operations: load, OR, store. If the hardware also modifies that register — a status bit setting itself, or another core writing it — you will read a stale value and write it back, silently undoing the hardware's change. Write-1-to-clear registers must be written directly (`reg = BIT(3)`), never read-modify-written, or you clear every other pending bit as a side effect. This theme returns in chapter 15.

Bitwise versus logical

```
/* BITWISE: operates on every bit position independently */
uint8_t a = 0x0C, b = 0x0A;
a & b   ==  0x08          /* 00001100 & 00001010 = 00001000 */
a | b   ==  0x0E
a ^ b   ==  0x06
~a      ==  0xF3

/* LOGICAL: treats each operand as a single true/false, and SHORT-CIRCUITS */
a && b  ==  1             /* both non-zero → true */
a || b  ==  1
!a      ==  0

/* the difference that bites */
if (flags & FLAG_A)       /* correct: tests the bit               */
if (flags && FLAG_A)      /* WRONG: true whenever flags is non-zero */

/* short-circuiting is a feature, not a detail */
if (p != NULL && p->field)  /* safe: p->field only evaluated if p is valid */
if (p != NULL &  p->field)  /* CRASH: bitwise & evaluates both operands    */
```

Shifts

```
Left shift:  a << n     multiply by 2ⁿ, zeros in from the right
   0000 1011 << 2  =  0010 1100        (11 → 44)

Logical right shift:  zeros in from the left — for UNSIGNED
   1011 0000 >> 2  =  0010 1100        (176 → 44)

Arithmetic right shift: the SIGN BIT replicates — for SIGNED
   1011 0000 >> 2  =  1110 1100        (−80 → −20)

C picks which one by the operand's type. ARM has separate
instructions: LSR (logical) and ASR (arithmetic).
```

SHIFT UNDEFINED BEHAVIOUR

Three ways a shift is UB in C, all of which the optimiser is allowed to exploit:

- Shifting by an amount ≥ the operand's width. `x << 32` on a 32-bit `int` is *not* zero — on ARM the shift amount is masked to 5 bits, so it is a shift by 0.
- Shifting a negative value left.
- Shifting a value into or past the sign bit of a signed type: `1 << 31` is UB because `1` is `int`. Write `1u << 31`, and `1ull << 40` for anything above 31.

How it fails

Symptom	Cause
A condition is true whenever any flag is set	`&&` used where `&` was meant
Setting a field also corrupts a neighbouring one	OR-ing in a value without clearing the field first
Clearing one interrupt clears several	Read-modify-write applied to a W1C register

Symptom	Cause
A 64-bit mask is zero above bit 31	`1 << n` with an `int` literal; needs `1ull`
A right shift of a negative value gives a huge positive one	Value stored in an unsigned type; logical rather than arithmetic shift
A null check does not protect the dereference	`&` instead of `&&` — no short-circuit

Check yourself

- ✓ Build NOT, AND and OR from NAND gates alone.
- ✓ State both De Morgan laws and use one to simplify `!(a & b)`.
- ✓ Write the six bit idioms from memory.
- ✓ Explain why you must clear a field before OR-ing a new value in.
- ✓ Give a case where `&` and `&&` produce different behaviour, not just different values.
- ✓ Name three ways a shift expression can be undefined behaviour.

Combinational logic building blocks

Gates alone are not useful. Four standard arrangements of them — adder, multiplexer, decoder, comparator — appear in every processor ever built, and once you can see them you can read a block diagram of a datapath.

The problem

Combinational logic has no memory: the outputs are a pure function of the current inputs. Change an input, and after a propagation delay the output settles to the new answer. There is no state, no clock, nothing to remember.

That constraint is exactly what makes it composable. The question this chapter answers is: which arrangements of gates are worth building once and reusing everywhere?

The adder

Binary addition, one column at a time, is exactly the arithmetic you learned as a child — and it maps onto two gates.

```
HALF ADDER — adds two bits, produces sum and carry

   A  B │ Sum  Carry
   ─────┼────────────
   0  0 │  0     0
   0  1 │  1     0
   1  0 │  1     0
   1  1 │  0     1        ← 1+1 = 10₂

   Sum   = A XOR B        ← "they differ"
   Carry = A AND B        ← "both"

FULL ADDER — also accepts a carry in, so adders can be chained

   Sum   = A XOR B XOR Cin
   Cout  = (A AND B) OR (Cin AND (A XOR B))
```

```
RIPPLE-CARRY ADDER — chain n full adders

   A₃B₃      A₂B₂      A₁B₁      A₀B₀
    ||        ||        ||        ||
  ┌─▼─┐     ┌─▼─┐     ┌─▼─┐     ┌─▼─┐
  │ FA │◄──┤ FA │◄──┤ FA │◄──┤ FA │◄── Cin
  └─┬─┘     └─┬─┘     └─┬─┘     └─┬─┘
 Cout S₃     S₂        S₁        S₀

Simple, small — and SLOW. Bit 31's answer cannot settle until the carry
has rippled through all 31 stages below it. Delay grows linearly with
width, and this sits directly on the critical path of the whole CPU.
```

> **WHY THIS MATTERS TO SOFTWARE PEOPLE**
> The carry chain is one of the longest combinational paths in a processor, so it strongly influences the maximum clock frequency. Real designs use **carry-lookahead** or **carry-select** adders, which compute in parallel whether each stage will generate or propagate a carry, trading area for delay proportional to log n rather than n. This is the first example of a pattern you will meet constantly: *spend transistors to buy time*.

Subtraction comes free

From chapter 02: a − b = a + (~b) + 1. Feed `b` through XOR gates controlled by a "subtract" signal, and tie that same signal to the carry-in:

```
   sub = 0  →  B passes through unchanged, Cin = 0  →  A + B
   sub = 1  →  B is inverted,              Cin = 1  →  A + ~B + 1  =  A − B

One adder, one control bit, both operations. This is the heart of the ALU
and the reason two's complement won.
```

The multiplexer

A mux is a hardware `if`: it selects one of several inputs to pass to its output.

```
2-to-1 multiplexer

        A ──┐
            ├──► Y        Y = (A AND ~S) OR (B AND S)
        B ──┘
            ▲
            S              S = 0 → Y = A
                           S = 1 → Y = B

4-to-1 needs 2 select bits; n-to-1 needs ⌈log₂ n⌉.
```

Muxes are everywhere in a CPU, and naming them is how you read a datapath diagram:

Where	What it selects
ALU input	A register value, or an immediate from the instruction
Register write-back	The ALU result, or data loaded from memory
Next program counter	PC+4, or a branch target, or an exception vector
Forwarding network	The register file, or a result still in flight in the pipeline (chapter 11)

> **MUX VERSUS BRANCH, IN SOFTWARE TERMS**
> A mux computes *both* inputs and then discards one. That is precisely what a branchless conditional does: ARM's `CSEL Xd, Xn, Xm, cond` is a 64-bit multiplexer exposed as an instruction. It is often faster than a branch because it cannot be mispredicted — and it is the standard mitigation for timing side channels in cryptographic code, because its execution time does not depend on the condition.

The decoder

A decoder turns an n-bit binary number into a one-hot signal: exactly one of 2^n outputs goes high.

```
2-to-4 decoder

   A₁ A₀ │ Y₃ Y₂ Y₁ Y₀
   ──────┼────────────
    0  0 │  0  0  0  1
    0  1 │  0  0  1  0
    1  0 │  0  1  0  0
    1  1 │  1  0  0  0

   Yₖ = 1 exactly when the input equals k.
```

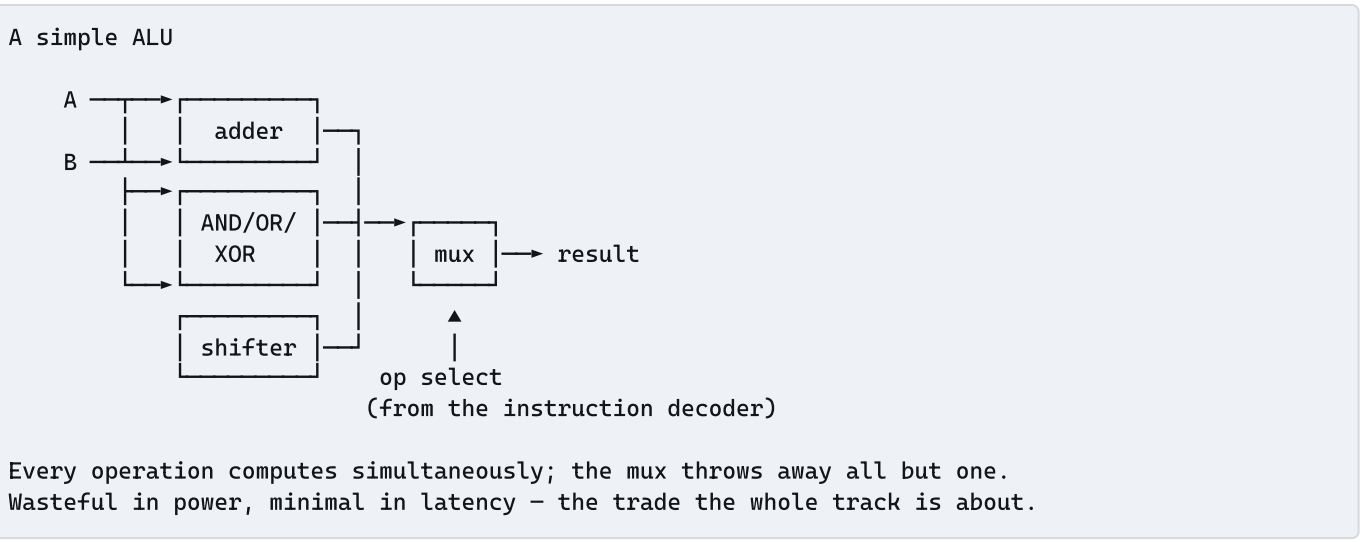
Two uses dominate, and both are ideas you meet again in software:

- **Address decoding.** The top bits of an address select which device responds. This is literally how a memory map is implemented — the `0x4001_0000` in a datasheet is a pattern the decoder matches to raise one peripheral's chip-select. See chapter 17.
- **Instruction decoding.** The opcode field indexes a decoder that enables the correct functional units. A64's fixed 32-bit encoding exists to make this decoder small and fast (chapter 09).

```
Address decoding on a small SoC

  address[31:28]  ──►  4-to-16 decoder  ──►  chip selects
      0x0            ──────────────────►  ROM
      0x2            ──────────────────►  SRAM
      0x4            ──────────────────►  peripheral bus
      0x8            ──────────────────►  DRAM controller

The "memory map" in the TRM is a description of this decoder's wiring.
```

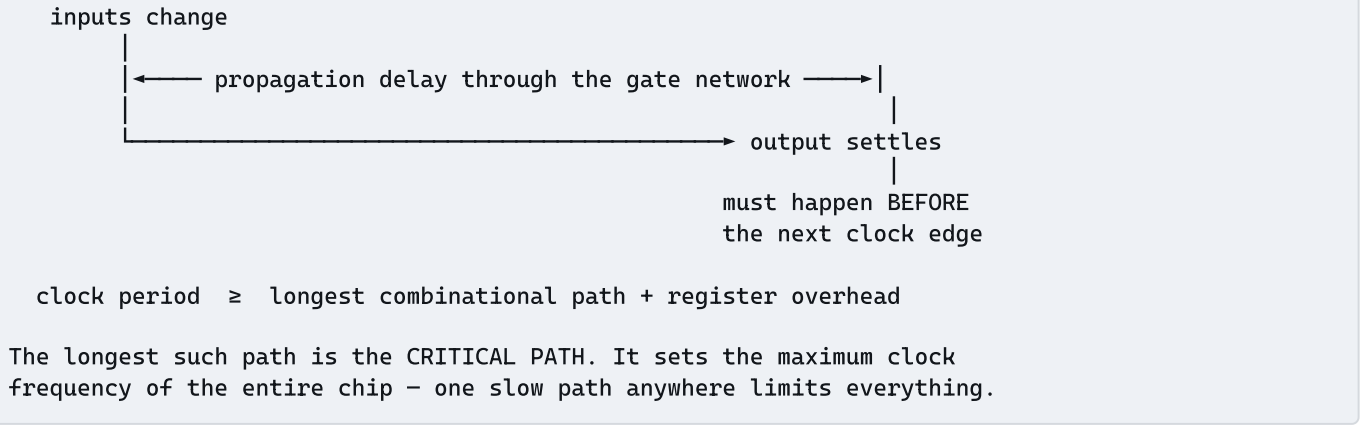
Comparators, shifters, and the ALU

Block	Built from	Note
Equality comparator	XNOR each bit pair, AND the results	"All bits match"
Magnitude comparator	A subtractor; inspect the flags	This is why `CMP` is just `SUBS` that discards its result
Barrel shifter	Layered muxes: shift by 1, 2, 4, 8, 16	Shifts any distance in one pass. ARM's is on the ALU input path, which is why `ADD X0, X1, X2, LSL #3` costs no extra cycle.
ALU	Adder + logic gates + shifter + output mux	The function-select bits come from the decoded opcode



Propagation delay: why any of this is a constraint

A gate does not switch instantly. Each contributes a few tens of picoseconds, and signals must settle before the result is captured.



> **THE CONNECTION TO PIPELINING**
> If the critical path is too long, you cannot clock faster. The fix is to cut the path in half with a register in the middle: each half now settles in half the time, so the clock can double, at the cost of taking two cycles end to end. That is pipelining, and it is the single most important idea in processor design. Chapter 06 supplies the register that makes it possible.

How it shows up in software

Observation	The combinational reason
`a * 8` compiles to a shift, folded into an addressing mode	The barrel shifter sits on the ALU input path and is free
`CSEL` beats a branch in a tight loop	A mux cannot be mispredicted
Division is 10–40× slower than multiplication	There is no cheap combinational divider; it is iterative
A comparison and a subtraction cost the same	They are the same hardware
Peripheral registers appear at "aligned, sparse" addresses	Address decoders match top bits only — full decoding costs gates
Constant-time crypto avoids data-dependent branches	Muxes take fixed time; branches do not

Check yourself

- ✓ Derive the half-adder's sum and carry equations from its truth table.
- ✓ Explain why a ripple-carry adder limits clock frequency, and what is done about it.
- ✓ Show how one adder performs both addition and subtraction.
- ✓ Name four places a multiplexer appears in a CPU datapath.
- ✓ Explain how a decoder implements a memory map.
- ✓ Define the critical path and say what it determines.
- ✓ Say why `CMP` and `SUBS` are the same instruction underneath.

Sequential logic, flip-flops and state machines

Combinational logic computes. To *remember* — and therefore to have registers, caches, program counters or anything resembling a computer — you need feedback. This chapter is where storage comes from.

The problem

Every block in chapter 05 forgets its inputs the instant they change. A machine built only from those blocks cannot hold a value, cannot count, cannot have a program counter — it can only react.

The missing ingredient is **feedback**: route an output back to an input, and the circuit can hold itself in a state.

From feedback to a latch

```
SR latch — two cross-coupled NOR gates

        S ──►┌──────┐
             │ NOR  ├──┬──► Q
        ┌───►└──────┘  │
        │              │
        │   ┌──────────┘
        │   │
        └───┼──┐
            │  │
        ┌──▼──▼┐
   R ──►│ NOR  ├────► ~Q
        └──────┘

   S  R │ Q      behaviour
   ────┼──────────────────────────
   0  0 │ hold   ← THE POINT: it remembers
   1  0 │  1     set
   0  1 │  0     reset
   1  1 │  ---   forbidden: both outputs 0, and the result on
               release depends on gate timing (a race)
```

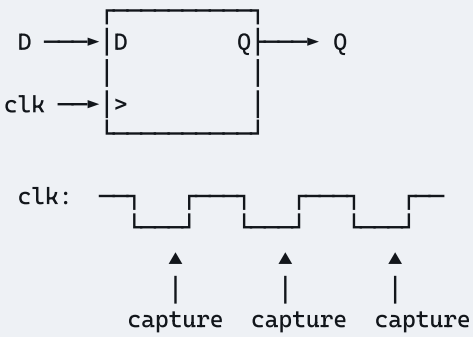
That "hold" row is the entire achievement. With `S = R = 0`, each gate's output feeds the other's input and the pair stays wherever it was. One bit is now stored — in nothing but the state of two gates driving each other.

Why latches are not enough

A plain latch is **level-sensitive**: while its enable is high, the output follows the input continuously. That transparency is dangerous — a signal can race through several latches in a single enable window, arriving somewhere it should not have reached until the next step.

Flip-flops: edge-triggered storage

```
D flip-flop — captures D only on the RISING EDGE of the clock
```




```
Between edges, the output is frozen no matter what D does.
The circuit is now SYNCHRONOUS: state changes only at known instants.
```

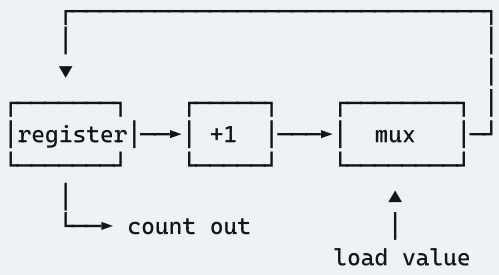
> **WHY EDGE-TRIGGERING MAKES DESIGN TRACTABLE**
>
> With edge-triggered storage, the whole machine advances in lockstep. Every combinational block gets one full clock period to settle, and the answer is sampled at a single, known instant. Reasoning reduces to a single question per path: *does this settle in time?* Without it, you would have to reason about overlapping propagation windows across the entire chip.

Registers

Put n flip-flops side by side, share the clock, and you have an n-bit register — the basic unit of storage in a processor.

Structure	Built from	In a CPU
Register	n flip-flops, common clock	`X0` – `X30`, `PC`, system registers
Register file	Registers + decoders + muxes for read/write ports	The 31 general-purpose registers, with 2 read ports and 1 write port
Shift register	Flip-flops chained output-to-input	Serial interfaces: UART, SPI, JTAG
Counter	Register + incrementer, fed back	Program counter, timers, cycle counters
Pipeline register	A register between two combinational stages	The thing that makes pipelining possible

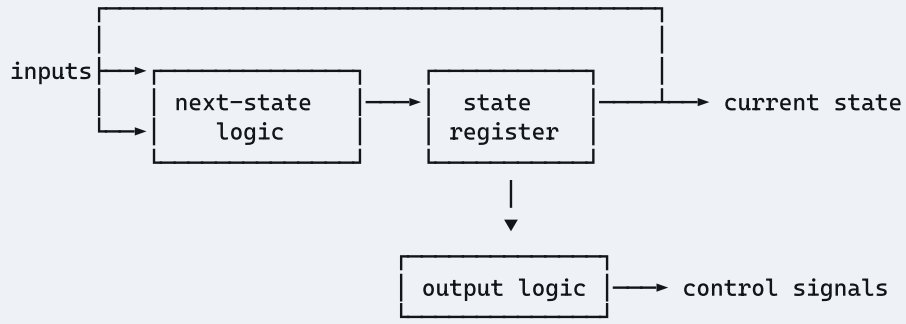
```
A counter — the simplest useful sequential circuit
```




```
This is a program counter. PC+4 each cycle, or a branch target when
the mux selects the load path.
```

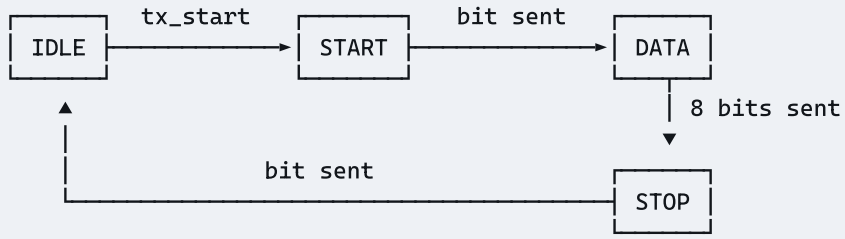
Finite state machines

Combine a register (holding "which state am I in") with combinational logic (deciding "given this state and these inputs, what next"), and you have the general model of every controller in a computer.




```
Moore machine:  outputs depend only on the state
Mealy machine:  outputs depend on state AND inputs (fewer states,
                but outputs can glitch mid-cycle)
```

A worked example: a UART transmitter




```
IDLE  : hold the line high, wait for tx_start
START : drive the line low for one bit period
DATA  : shift out 8 bits, LSB first, one per bit period
STOP  : drive high for one bit period, then return to IDLE

That is the whole transmitter. Every serial protocol you will meet is
a state machine of this shape.
```

THE SAME PATTERN IN SOFTWARE

You will write this structure constantly in drivers, and for the same reason: a device's behaviour is naturally a state machine, and modelling it as one makes illegal transitions impossible to express rather than merely unlikely.

```
enum state { IDLE, START, DATA, STOP };

static void tick(struct uart *u)
{
        switch (u->state) {
        case IDLE:  if (u->pending) { line(0); u->state = START; } break;
        case START: line(u->byte & 1); u->bit = 1; u->state = DATA;  break;
        case DATA:  line((u->byte >> u->bit) & 1);
                    if (++u->bit == 8) u->state = STOP;              break;
        case STOP:  line(1); u->state = IDLE;                        break;
        }
}
```

Windows' PnP state machine, Linux's device probe/remove lifecycle and USB's enumeration are all the same idea at a larger scale.

Reset: the state you start in

On power-up, every flip-flop holds an arbitrary value — whichever way it happened to settle. A design without reset starts in a random state and behaves unpredictably.

Kind	Behaviour	Trade-off
Asynchronous	Takes effect immediately, no clock needed	Works before the clock is running — but releasing it near a clock edge can cause metastability
Synchronous	Applied on a clock edge like any other input	Clean and timing-analysable — but useless if the clock is not yet running
Async assert, sync release	Both	The standard practice: assert immediately, release cleanly

This is why a datasheet lists a *reset value* for every register, why boot code must not assume a peripheral is in a known state unless it has reset it, and why "it works after a power cycle but not after a warm reset" is a recognisable class of bug — the two paths reset different things.

How it shows up in software

Observation	The sequential reason
A register reads back a value you never wrote	Reset value from the datasheet; the block was never programmed
A peripheral works after power cycle, fails after warm reset	Different reset domains; some state survives
You must hold a block in reset while configuring its clock	Flip-flops need a stable clock before they can capture reliably
A status bit is briefly wrong right after a write	Mealy-style output glitching, or a clock-domain crossing
A driver needs a delay after enabling a block	Internal state machine needs cycles to reach its ready state

Observation	The sequential reason
Reading a counter gives a torn value	A wide counter sampled across an edge; read high, low, then high again and retry if it changed

Check yourself

- ✓ Explain how two cross-coupled NOR gates store a bit.
- ✓ Say what is forbidden about `S = R = 1` and why.
- ✓ Give the difference between a level-sensitive latch and an edge-triggered flip-flop, and why designs use the latter.
- ✓ Draw a counter from a register and an adder.
- ✓ Draw the state diagram of a UART transmitter and name each state's job.
- ✓ Explain why every register has a documented reset value and what that means for driver code.

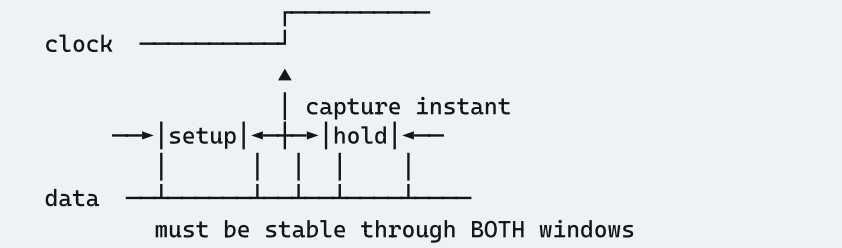
Clocks, timing and metastability

The clock is the heartbeat that makes a machine of gates behave predictably. This chapter explains what limits its speed, why crossing between two clocks is genuinely dangerous, and why so many driver bugs turn out to be "the block was not clocked".

The problem

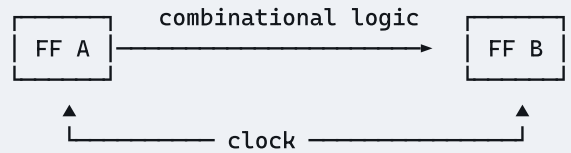
From chapter 06, a flip-flop captures its input on a clock edge. But a real flip-flop is not instantaneous. It needs its input to be *stable for a while before* the edge and to *stay stable for a while after*. Violate either window and the captured value is not merely wrong — it may be neither 0 nor 1 for a time.

Setup and hold




```
  t_setup : input must be stable this long BEFORE the edge
  t_hold  : input must remain stable this long AFTER the edge

Violate either → the flip-flop may enter a METASTABLE state:
its output hovers between logic levels for an unbounded time
before resolving randomly to 0 or 1.
```

The maximum-frequency equation




```
  T_clock  ≥  t_clk→q  +  t_logic  +  t_setup  +  t_skew

    t_clk→q : delay from the edge to A's output being valid
    t_logic : propagation through the combinational cloud  ← the big one
    t_setup : B's setup requirement
    t_skew  : difference in clock arrival time between A and B

  f_max = 1 / T_clock

The path with the largest t_logic — the CRITICAL PATH — sets f_max
for the entire chip. One slow path anywhere limits everything.
```

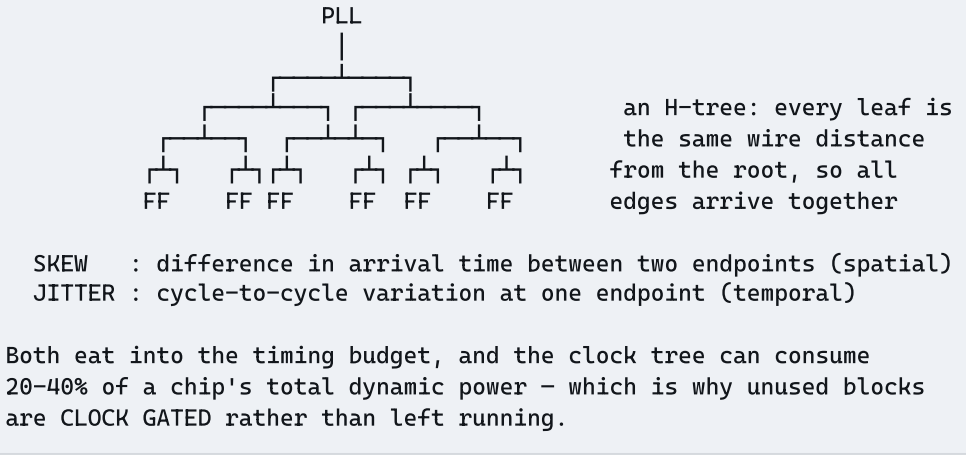
THE THREE WAYS TO GO FASTER

1. **Shorten the logic.** Better adders, fewer levels of gates.
2. **Cut the path in two with a register.** Each half now settles in half the time, so the clock can roughly double — at the cost of an extra cycle of latency. This is pipelining.
3. **Do more per cycle instead.** Wider issue, SIMD, more cores — chapter 19.

Frequency scaling stopped around 2005 because of power (chapter 20), which is why options 2 and 3 dominate every design since.

Clock distribution

A clock must reach millions of flip-flops at very nearly the same instant. It does not arrive by magic: it is driven through a large buffered tree that is among the most carefully engineered structures on the die.




```
  SKEW   : difference in arrival time between two endpoints (spatial)
  JITTER : cycle-to-cycle variation at one endpoint (temporal)

Both eat into the timing budget, and the clock tree can consume
20–40% of a chip's total dynamic power — which is why unused blocks
are CLOCK GATED rather than left running.
```

Where clocks come from

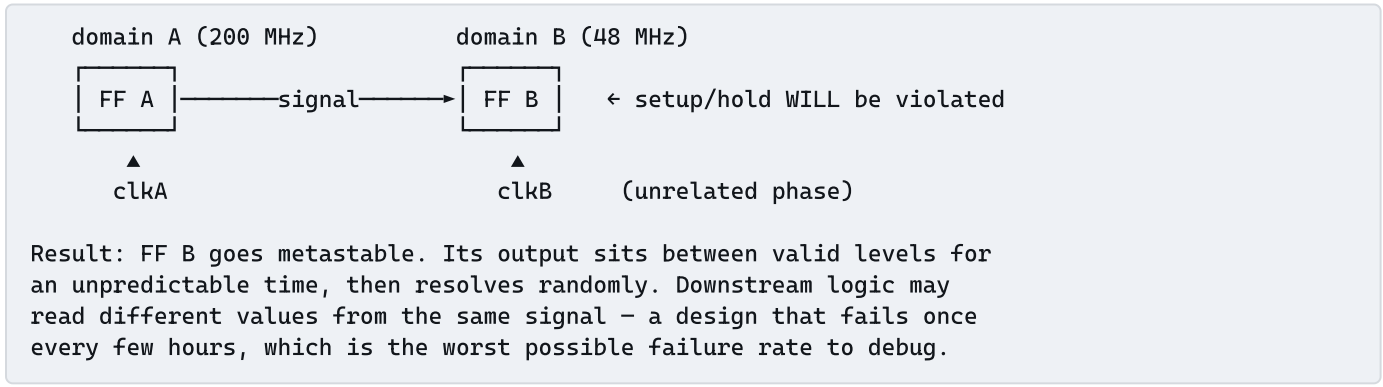
Source	Typical	Note
Crystal oscillator	12–50 MHz	The accurate reference everything derives from. Needs correct load capacitors — wrong ones give a subtly off frequency, which shows up as UART framing errors.
PLL	Multiplies to 1–3 GHz	Needs time to lock; boot code must wait for the lock bit before switching to it
Dividers	Per-peripheral	The clock tree software configures via the clock framework
RC oscillator	Low accuracy	Cheap, used for watchdogs and wake-up; too imprecise for serial links

THE NUMBER ONE BRING-UP BUG

A peripheral whose clock is not enabled reads back as all zeros, all ones, or aborts the access entirely — indistinguishable from a wrong address or a firewall denial. **Always verify the clock before debugging anything else.** On Linux, `cat /sys/kernel/debug/clk/clk_summary` shows every clock's rate and enable count. In a driver, this is what `clk_prepare_enable()` is for, and why it comes before the first register read in every `probe()` function.

Clock domain crossing

A real SoC has dozens of independent clocks: CPU, DDR, USB, audio, a slow always-on domain. When a signal passes from one to another, the receiving flip-flop's setup and hold windows are violated *eventually and unavoidably*, because the two clocks have no fixed phase relationship.



Result: FF B goes metastable. Its output sits between valid levels for an unpredictable time, then resolves randomly. Downstream logic may read different values from the same signal — a design that fails once every few hours, which is the worst possible failure rate to debug.

The standard fixes

Technique	For	How
Two-flop synchroniser	A single control bit	Chain two flip-flops in the destination domain. The first may go metastable; it is given a full clock period to resolve before the second samples it. Reduces failure probability to once in years.
Gray-coded counters	Multi-bit counters	Only one bit changes per increment, so a mid-transition sample yields either the old or the new value — never a nonsense combination
Asynchronous FIFO	Data streams	Dual-port memory with Gray-coded read/write pointers, each synchronised into the other domain
Handshake	Occasional transfers	Request/acknowledge, each synchronised — slow but simple and robust

WHY A SOFTWARE ENGINEER SHOULD CARE

Three reasons this is not just hardware trivia. First, it explains why some status registers must be read twice and compared, or why a datasheet says "this field is valid two cycles after the write" — you are seeing a synchroniser's latency. Second, the two-flop synchroniser is conceptually identical to a **memory barrier**: both are mechanisms for safely passing information between two things that do not share a timebase. Third, the "read a wide counter twice and retry if it changed" idiom in timer drivers exists precisely because the counter crosses a clock domain.

```
/* reading a 64-bit counter that updates in another clock domain */
do {
        hi1 = readl(base + CNT_HI);
        lo  = readl(base + CNT_LO);
        hi2 = readl(base + CNT_HI);
} while (hi1 != hi2);          /* retry if the low half wrapped mid-read */
```

Clock gating and dynamic scaling

Dynamic power is proportional to $C \cdot V^2 \cdot f$ — capacitance, voltage squared, frequency. Two consequences shape every modern SoC:

- **Clock gating.** Stop the clock to an idle block and its dynamic power goes to essentially zero. This is what a peripheral's "clock enable" bit does, and why disabling it makes the block's registers unreadable — there is no clock to capture the access.
- **DVFS** (dynamic voltage and frequency scaling). Lower frequency permits lower voltage, and because of the square term the energy saving is superlinear. This is what a CPU governor manipulates, and why a benchmark run at a low governor setting is not comparable to one at another.

```
Typical SoC clock hierarchy as software sees it

  crystal 24 MHz
       │
       ├─► PLL0 ──► 1.8 GHz ──► CPU cluster        [DVFS: 400 MHz – 1.8 GHz]
       │
       ├─► PLL1 ──► 800 MHz ──► DDR controller
       │
       └─► PLL2 ──► 480 MHz ──┬─► /4  → 120 MHz ──► AXI interconnect
                              ├─► /10 →  48 MHz ──► USB      [gateable]
                              └─► /30 →  16 MHz ──► APB ──┬─► UART  [gateable]
                                                          ├─► I2C   [gateable]
                                                          └─► SPI   [gateable]

Every "gateable" leaf is a clk_prepare_enable() call in some driver.
```

How it fails

Symptom	Cause
Peripheral registers read as 0 or 0xFFFFFFFF	Clock not enabled — check this first, every time
Access to a peripheral aborts	Clock or power domain off; the bus gets no response
UART output is garbage at every baud rate	Wrong crystal frequency assumed, or wrong load capacitors
System hangs immediately after switching to the PLL	Did not wait for the PLL lock bit
A failure once every few hours, never reproducible	Missing clock-domain synchroniser — classic metastability
A timer occasionally reads a wildly wrong value	Torn read across a counter's halves; use the retry loop
Works at low CPU frequency, fails at high	A timing path is marginal, or a delay loop was calibrated for one frequency
Benchmarks vary 3× run to run	DVFS and thermal throttling; pin the governor before measuring

Check yourself

- ✓ Define setup time and hold time, and say what happens when either is violated.
- ✓ Write the maximum-frequency equation and name its dominant term.
- ✓ Explain the difference between clock skew and clock jitter.
- ✓ Say why a two-flop synchroniser reduces, but never eliminates, metastability risk.
- ✓ Explain why Gray coding is used for pointers crossing clock domains.
- ✓ Give the first thing to check when a peripheral register reads as all zeros.
- ✓ Explain why lowering frequency saves more than proportional energy.

The stored-program computer

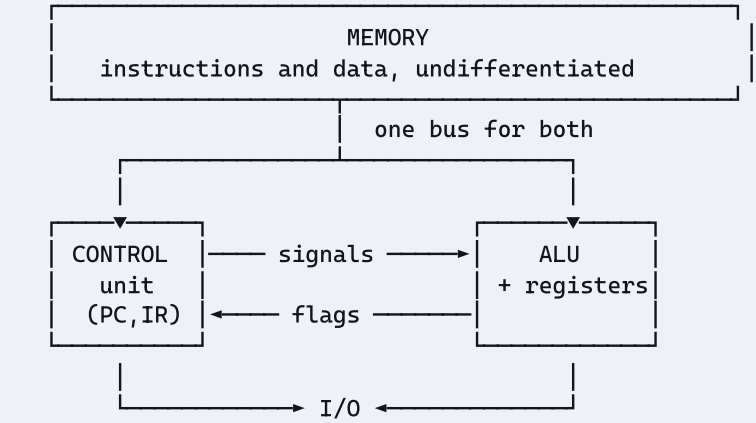
The idea that a program is just data in the same memory as everything else is what makes a computer general-purpose — and it is also the root of buffer overflows, JIT compilers, self-modifying boot code and the entire W^X security model.

The problem

Chapters 05 and 06 give you an ALU and registers. You could wire them into a machine that computes one fixed function — a calculator. To change what it does, you would rewire it. Early machines like ENIAC were literally reprogrammed by moving cables, which took days.

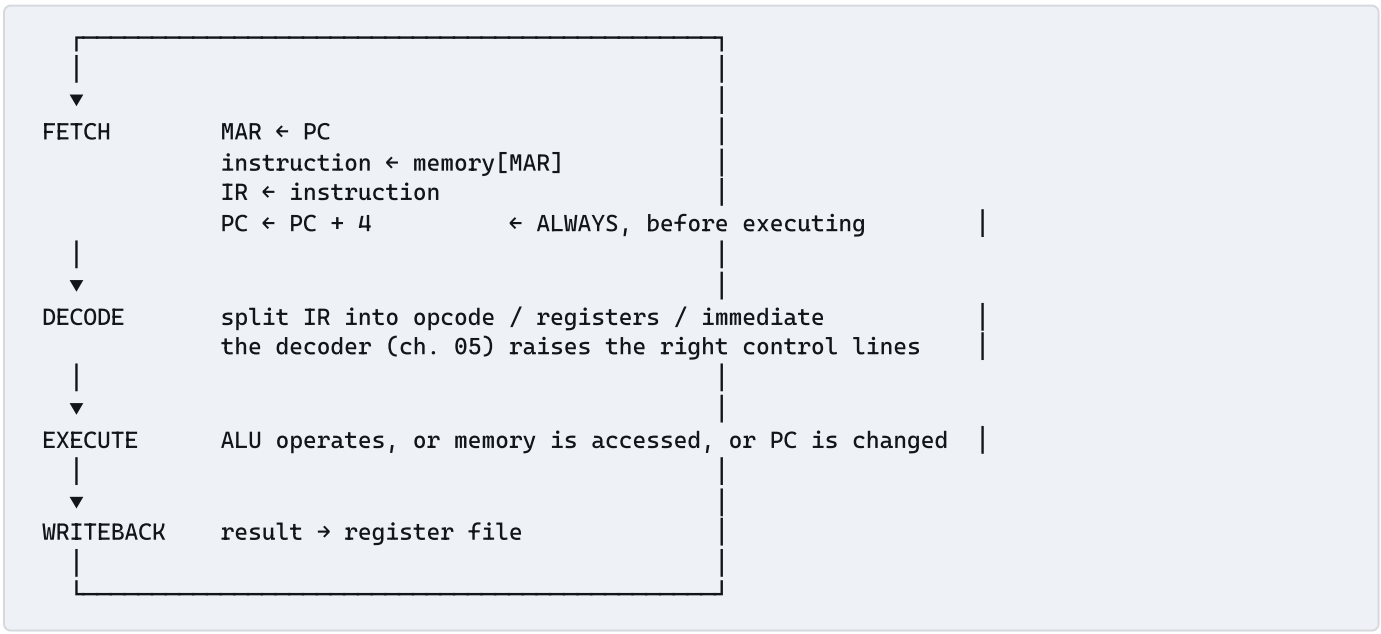
The stored-program insight, credited to the 1945 EDVAC report and usually named after von Neumann, is: **put the instructions in memory alongside the data**. Then changing the program is just changing memory contents, which the machine can do itself.

The von Neumann model




```
Five parts: memory, control unit, arithmetic unit, input, output.
Every processor since is a refinement of this drawing.
```

The fetch–decode–execute cycle



> **WHY PC IS INCREMENTED DURING FETCH, NOT AFTER EXECUTE**
> By the time a branch executes, the PC already points at the following instruction. A branch simply overwrites it. This is why a "relative branch" offset is measured from the instruction after the branch, and it is why `BL` (branch with link) can save the return address by copying the already-incremented PC into the link register — no separate addition needed. Every quirk of branch encoding traces back to this ordering.

Harvard and the modified Harvard architecture

Von Neumann's single memory port is a bottleneck: an instruction fetch and a data access compete for the same bus every cycle. The **Harvard** architecture gives instructions and data separate memories and separate buses, so both happen simultaneously.

	Von Neumann	Harvard	Modified Harvard
Memories	One, shared	Two, separate	One unified main memory, split L1 caches
Fetch + load in one cycle	No	Yes	Yes, when both hit in L1
Can code be written as data?	Yes, naturally	No, or only via special instructions	Yes, but requires explicit cache maintenance
Used by	The conceptual model	Small DSPs, some microcontrollers	**Every Cortex-A, and essentially every modern CPU**

```
Modified Harvard, as ARM actually implements it

          ┌──────────┐       ┌──────────┐
          │ L1 I-cache│       │ L1 D-cache│     separate: Harvard-like
          └──────┬───┘       └────┬─────┘
                 └──────┬─────────┘
                        ▼
                    ┌──────┐
                    │  L2  │                   unified: von Neumann-like
                    └──┬───┘
                       ▼
                    ┌──────┐
                    │ DRAM │
                    └──────┘

Best of both: parallel access at L1, one address space overall.
The COST appears the moment you write instructions as data.
```

THE CONSEQUENCE YOU WILL ACTUALLY HIT

Write instruction bytes into memory — a JIT compiler, a bootloader relocating code, a kernel patching an alternative instruction sequence, a debugger inserting a breakpoint — and those bytes sit in the **data** cache. The instruction cache and the prefetcher know nothing about them. Execute that address and you run stale instructions.

```
/* the mandatory sequence after writing code, on ARM */
dc      cvau, x0        /* clean D-cache to the point of unification  */
dsb     ish             /* wait for it to complete                    */
ic      ivau, x0        /* invalidate I-cache for that address        */
dsb     ish             /* wait                                       */
isb                     /* flush already-prefetched stale instructions*/
```

Covered in full in ARM chapter 18. Every self-modifying mechanism on the platform depends on getting this exactly right.

What the stored-program idea buys and costs

Because code is data…	You get
A program can load another program	Operating systems, loaders, dynamic linking
A program can generate code at runtime	JIT compilers, eBPF, GPU shader compilation
Code can be relocated after being loaded	Position-independent executables, ASLR, shared libraries
Code can be inspected and modified	Debuggers, breakpoints, hot patching, kernel live patching
Code can be transmitted	Firmware updates, containers, downloadable applications
Data can be executed	**Buffer overflow attacks, code injection, ROP**

W^X: taking the generality back

That last row is the problem. If an attacker can write bytes into a buffer and then redirect execution to it, the stored-program property is the vulnerability. The defence is to make memory **writable XOR executable**, never both, enforced by the MMU:

Region	Permissions	Enforced by
`.text` (code)	Read + Execute, not Write	Page table AP bits

Region	Permissions	Enforced by
`.rodata`	Read only	Page table AP bits
`.data`, `.bss`, heap, stack	Read + Write, **never Execute**	Page table UXN/PXN bits

Related hardware mitigations, all of which exist because of this chapter's central idea:

- **PXN / UXN** — privileged/unprivileged execute-never bits in ARM page descriptors.
- **PAN** — Privileged Access Never: the kernel cannot accidentally dereference a user pointer without explicitly enabling access.
- **BTI** — Branch Target Identification: an indirect branch may only land on a `BTI` instruction, which breaks jump-oriented programming.
- **Pointer authentication** — signs return addresses so a corrupted stack cannot redirect a return.

A JIT compiler must therefore map a page writable, emit code, then *remap it executable* — never holding both permissions at once. Apple platforms enforce this so strictly that they provide a dedicated API for the transition.

The memory wall

Von Neumann's single path to memory has a second consequence, and it has grown into the dominant performance constraint in computing.

```
Relative improvement since ~1980

  CPU speed     ████████████████████████████████  ~50,000×
  DRAM latency  ███                               ~10×

The gap is the "von Neumann bottleneck" or "memory wall".
A DRAM access costs 100–300 CPU cycles. The processor is not
compute-limited; it is waiting.

Everything in chapters 13–15 — caches, prefetching, the whole memory
hierarchy — exists to hide this one number.
```

> **WHY THIS MATTERS FOR AI WORKLOADS**
> Large language model inference is almost entirely memory-bandwidth-bound, not compute-bound: generating each token requires streaming the entire weight matrix through the arithmetic units, and every weight is used exactly once. This is why quantisation (making the weights smaller) speeds up inference far more than a faster multiplier would, and why accelerators are designed around memory bandwidth. See AI chapter 26.

How it shows up

Symptom	Cause
Freshly written code executes as garbage or crashes	Missing I-cache invalidate after writing instructions
A JIT works on x86 but faults on ARM	x86 keeps I-cache coherent in hardware; ARM does not
Segfault the moment a function pointer into a buffer is called	W^X: the page is not executable, which is working as intended

Symptom	Cause
A debugger breakpoint corrupts a nearby instruction	Breakpoints work by overwriting instructions — the stored-program idea in action
A performance profile shows most cycles stalled on loads	The memory wall; the fix is data layout, not faster arithmetic
Boot code hangs after relocating itself	Jumped to the new copy without cache maintenance

Check yourself

- ✓ State the stored-program idea in one sentence and name two things it makes possible.
- ✓ List the four phases of the instruction cycle and say when the PC is updated.
- ✓ Explain the difference between Harvard and modified Harvard, and which ARM uses.
- ✓ Give the exact reason writing instructions to memory requires cache maintenance.
- ✓ Explain W^X and name two ARM features that enforce it.
- ✓ Explain the memory wall and why it makes LLM inference bandwidth-bound.

Instruction set architecture: RISC vs CISC

The ISA is the contract between hardware and software — the only part of a processor that software can see, and the only part the manufacturer cannot change. This chapter explains what belongs in that contract, why the RISC/CISC argument happened, and why the answer today is stranger than either side predicted.

The problem: what must a processor promise?

A chip designer wants freedom. Caches, pipeline depth, how many instructions issue per cycle, whether execution is in order — all of that should be changeable between generations without breaking existing software.

A software author wants stability. A binary compiled in 2015 should still run in 2030.

The **instruction set architecture** is the negotiated boundary. It specifies exactly what software may rely on, and by omission grants the designer freedom over everything else.

> **ARCHITECTURE VERSUS MICROARCHITECTURE**
>
> **Architecture** is the specification: instructions, their encodings and effects, the register set, the memory model, the exception model. It is a document — for ARM, the Architecture Reference Manual.
>
> **Microarchitecture** is one implementation of that specification: Cortex-A78, Neoverse N2, Apple's M-series cores. Pipeline depth, cache sizes, branch predictor design, reorder buffer size.
>
> The distinction is the most practically useful one in this chapter. *Architectural* behaviour is portable across every implementation — if your code is architecturally correct it works everywhere. *Microarchitectural* behaviour is timing, and is chip-specific. When debugging, always ask which one you are fighting: architectural bugs reproduce anywhere, microarchitectural surprises (timing, cache effects, errata) do not.

What an ISA specifies

Element	Question it answers	AArch64's answer
Register set	How much named, fast storage is there?	31 general-purpose 64-bit registers, 32 vector registers, a zero register, separate stack pointers per privilege level
Instruction encoding	How are operations represented in memory?	Fixed 32 bits, always 4-byte aligned
Operations	What can be computed?	Arithmetic, logic, shifts, loads/stores, branches, atomics, system, SIMD
Addressing modes	How is an operand located in memory?	Base, base+offset, base+scaled index, pre/post-indexed, PC-relative
Data types	What sizes and formats are natively supported?	8/16/32/64-bit integers, IEEE 754 single/double/half, packed vectors
Memory model	What ordering do other observers see?	Weakly ordered, with explicit barriers and acquire/release accesses
Exception model	What happens on a fault or interrupt?	Four exception levels, a vector table, syndrome registers
Privilege model	Who may do what?	EL0–EL3, plus two security states

Everything *not* on that list — how many cycles an instruction takes, whether there is an L3 cache, how branches are predicted — is deliberately unspecified, and that omission is what lets a Cortex-A55 and a Cortex-X4 run identical binaries at wildly different speeds.

The CISC era and its logic

In the 1970s, memory was scarce and expensive, compilers were poor, and much code was written by hand in assembly. Those three facts pointed in one direction: make each instruction do as much as possible.

```
The CISC bet, stated plainly:

  - Programs are stored in expensive memory  →  make instructions DENSE
  - Humans write assembly                    →  make instructions EXPRESSIVE
  - Compilers are weak                       →  close the "semantic gap" in HARDWARE

Result: instructions that are almost statements in a high-level language.

  VAX-11 (the extreme case):
      MOVC3  len, src, dst      ; copy a string, one instruction
      POLYD  x, degree, table   ; evaluate a polynomial, one instruction
      INDEX  ...                ; bounds-checked array index with trap

  x86:
      REP MOVSB                 ; copy ECX bytes from ESI to EDI
      ADD  [ebx+ecx*4+8], eax   ; load, add, store — one instruction
                                ; and it can also set flags and fault mid-way
```

What went wrong with it

- **Variable-length encoding.** An x86 instruction is 1 to 15 bytes. You cannot know where instruction *n+1* begins until you have decoded instruction *n*. That makes decoding inherently serial — a severe problem once you want to decode four or eight instructions per cycle.
- **Microcode.** Complex instructions are not executed directly; they are interpreted by a small internal program. Microcode is slow, hard to pipeline, and a source of errata.
- **The 90/10 measurement.** Studies in the late 1970s found compilers used a small fraction of the available instructions. The exotic ones cost design effort, area and verification, and almost nobody emitted them.
- **Complex instructions can be slower than the equivalent sequence.** A famous finding: on some VAX models, `INDEX` was slower than the several simple instructions that did the same job — the hardware could not be optimised for every case.
- **Faults in the middle.** If `ADD [mem], eax` page-faults after the load but before the store, the machine must be able to restart or complete it precisely. This complicates the exception model enormously.

The RISC counter-argument

Berkeley RISC and Stanford MIPS, around 1980, inverted every assumption:

Principle	Reasoning
Fixed-length instructions	Decode becomes trivial and parallel; you can fetch and decode N instructions per cycle without knowing their contents
Load/store architecture	Only explicit loads and stores touch memory. Every other instruction is register-to-register, so only one class of instruction can fault on a data access.
Many registers	Keep values in registers rather than spilling to memory. RISC designs went from 8 to 32.

Principle	Reasoning
Simple addressing modes	One memory address computation per instruction, at most
No microcode	Instructions execute directly in hardware, one per cycle in the ideal case
Let the compiler do the work	Compilers had improved. Scheduling, register allocation and strength reduction move to software, where they can be improved without new silicon.

```
The same operation, both philosophies

CISC (x86-64):
    add    QWORD PTR [rbx+rcx*8+16], rax     ; one instruction
                                              ; internally: load, add, store

RISC (AArch64):
    add    x3, x1, x2, lsl #3    ; compute the address
    ldr    x4, [x3, #16]         ; LOAD
    add    x4, x4, x0            ; ADD  (registers only)
    str    x4, [x3, #16]         ; STORE

More instructions, more bytes. But each one is simple, fixed-width,
independently schedulable, and can fault in exactly one obvious place.
```

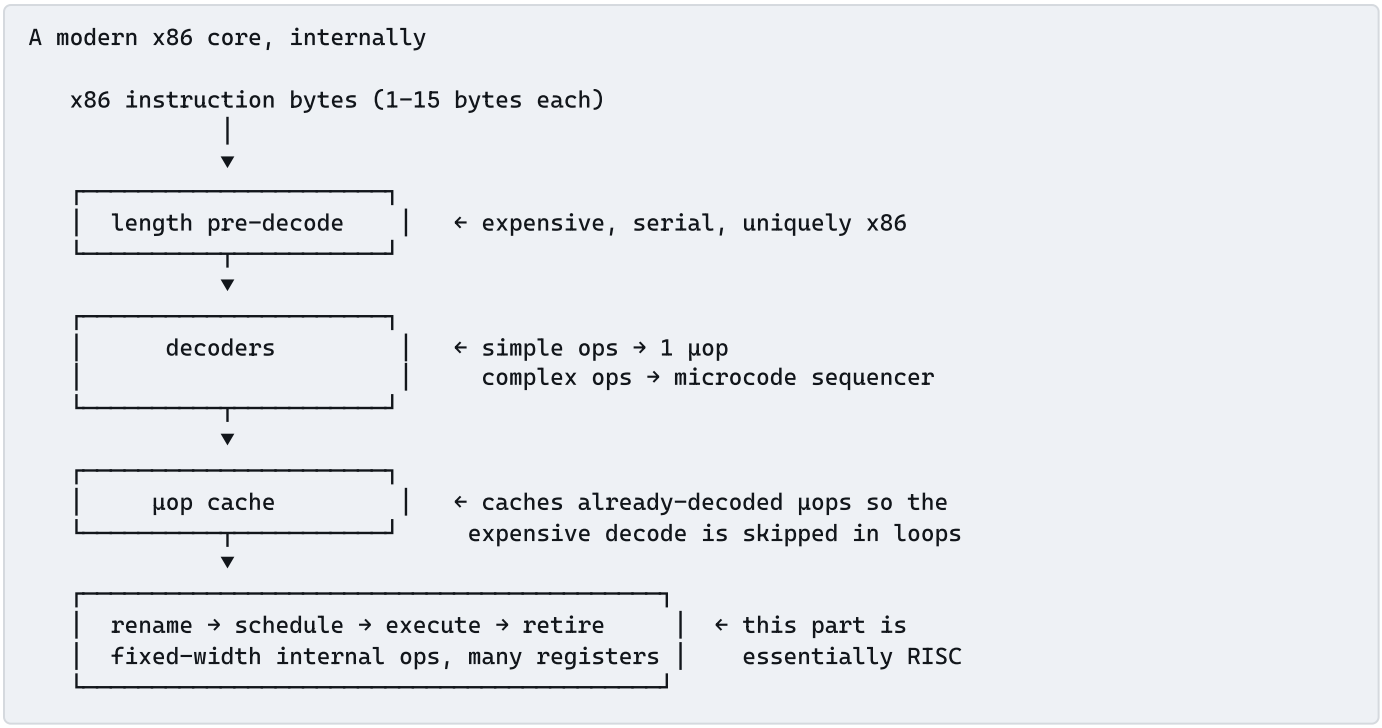
The cost RISC accepted

Code density. A fixed 32-bit encoding wastes bits on simple operations — `MOV X0, X1` uses the same four bytes as a complex shifted-add. Larger binaries mean more instruction-cache pressure, which is a real performance cost, not a cosmetic one.

ARM's answer to this was **Thumb** (1994) and later **Thumb-2**: a 16-bit encoding of the most common instructions, mixed freely with 32-bit ones, typically saving 30% of code size. Thumb is why ARM won the mobile market, where flash memory cost real money. Notably, AArch64 *removed* Thumb — by 2011 memory was cheap and decode simplicity mattered more.

Why the debate ended in a draw nobody predicted

Modern high-performance x86 processors decode their CISC instructions into internal RISC-like **micro-operations** and execute those on an out-of-order engine that would be entirely familiar to a RISC designer.



So RISC won *architecturally inside the chip*, while CISC survived *as an external interface* because of binary compatibility. Meanwhile ARM, nominally RISC, has accumulated crypto instructions, complex SIMD, atomic memory operations that execute near the memory controller, and load/store-pair instructions — none of which a 1980 RISC purist would have accepted.

> **WHERE THE DIFFERENCE STILL GENUINELY MATTERS**
>
> The lasting advantage is not instruction complexity, it is **decode**. Fixed 32-bit encoding means an ARM core can decode 8 instructions per cycle with 8 simple parallel decoders. An x86 core needs length pre-decode first, which is serial and power-hungry, and mitigates it with a µop cache — extra area and power that ARM simply does not spend. This is a large part of why ARM designs reach better performance per watt, and why Apple's cores could go so wide so cheaply.

Reading an instruction encoding

Fixed-width encoding is not just a design principle; it is something you will read directly when debugging without symbols. Here is an AArch64 instruction taken apart.

```
    ADD X0, X1, X2          encodes as 0x8B020020

 31 30 29 28    24 23 22 21 20   16 15    10 9    5 4    0
┌──┬──┬──┬─────────┬─────┬──┬───────┬────────┬──────┬──────┐
│sf│op│ S│ 0 1 0 1 │shift│ 0│   Rm  │  imm6  │  Rn  │  Rd  │
└──┴──┴──┴─────────┴─────┴──┴───────┴────────┴──────┴──────┘
  1  0  0   01011    00   0  00010   000000  00001  00000

  sf    = 1   → 64-bit operands (0 would mean 32-bit, Wn registers)
  op    = 0   → ADD (1 would be SUB)
  S     = 0   → do not set flags (1 gives ADDS)
  shift = 00  → LSL
  Rm    = 2   → X2
  imm6  = 0   → shift amount 0
  Rn    = 1   → X1
  Rd    = 0   → X0

Change sf to 0 and you have ADD W0, W1, W2 — same instruction, 32-bit.
Change S to 1 and it sets NZCV. Change op to 1 and it subtracts.

Every field is in a FIXED position. That is what makes the decoder
(chapter 05's decoder block) small and fast.
```

```
# see this for yourself
echo 'add x0, x1, x2' | aarch64-linux-gnu-as -o /tmp/t.o -
aarch64-linux-gnu-objdump -d /tmp/t.o
#   0:   8b020020        add     x0, x1, x2
```

Instruction classes every ISA has

Class	Purpose	AArch64 examples
Data processing	Compute on registers	`ADD`, `SUB`, `AND`, `ORR`, `EOR`, `MUL`, `UDIV`, `LSL`
Data movement	Load, store, register moves, constants	`LDR`, `STR`, `LDP`, `STP`, `MOV`, `MOVK`
Control flow	Change the PC	`B`, `BL`, `BR`, `RET`, `CBZ`, `TBNZ`, `B.EQ`
Comparison	Set flags or select without branching	`CMP`, `TST`, `CSEL`, `CSET`
Atomic / synchronisation	Multi-core correctness	`LDXR` / `STXR`, `CAS`, `LDADD`, `DMB`, `DSB`, `ISB`
System	Privilege, configuration, cache and TLB control	`MRS`, `MSR`, `SVC`, `ERET`, `TLBI`, `DC`, `WFI`
SIMD / vector	Data parallelism	NEON `ADD V0.4S,...`, SVE predicated operations

Notice that the last two classes are where a general-purpose ISA stops being general. System instructions are the entire subject of the ARM track's second half, and SIMD is the reason machine learning runs at all on a CPU.

Orthogonality and why compilers care

An ISA is **orthogonal** when any operation works with any register and any addressing mode, with no exceptions. Orthogonality makes a compiler's register allocator simple, because no register is special.

```
Non-orthogonal (8086, and it shaped a generation of compilers):
    - MUL always uses AX and DX implicitly
    - String instructions require SI, DI, CX
    - Only BX, SI, DI, BP may be used as base registers
    → the allocator must reason about which value must live where

Orthogonal (AArch64, nearly):
    - Any Xn can be any operand of almost any instruction
    - Exceptions are few and documented:
        · X30 is written by BL
        · X16/X17 may be clobbered by linker veneers
        · X18 is reserved on some platforms
        · SP must be 16-byte aligned at public interfaces
        · XZR reads as zero and discards writes
```

This is why the same compiler back end typically produces better code for ARM than for 32-bit x86: fewer constraints to satisfy, more registers to allocate into, less spilling to the stack.

How an ISA grows without breaking

An ISA must evolve — new workloads need new instructions — without invalidating existing binaries. The mechanisms are worth knowing because you will write the code that uses them.

1. **Reserve encoding space.** Undefined encodings are kept available and made to fault, so they can later be given meaning.
2. **Add optional extensions.** ARMv8.1 added LSE atomics; v8.3 added pointer authentication; v9 added SVE2. Each is optional at a given architecture level.
3. **Advertise features in ID registers.** `ID_AA64ISAR0_EL1` and friends report what this specific chip implements.
4. **Have software detect and adapt.** The OS reads the ID registers at boot and patches code paths accordingly.

```
/* Linux exposes CPU features to user space through HWCAP */
#include <sys/auxv.h>
#include <asm/hwcap.h>

unsigned long hw  = getauxval(AT_HWCAP);
unsigned long hw2 = getauxval(AT_HWCAP2);

if (hw & HWCAP_ATOMICS)   use_lse_atomics();    /* single-instruction atomics */
else                      use_ldxr_stxr();      /* the exclusive-monitor loop */

if (hw & HWCAP_ASIMD)     use_neon_kernel();
if (hw & HWCAP_SVE)       use_sve_kernel();
if (hw2 & HWCAP2_SVE2)    use_sve2_kernel();

/* the kernel does the same thing internally, but patches the code in
   place at boot with "alternatives" so there is no runtime branch:
   ALTERNATIVE("ldxr ...", "ldadd ...", ARM64_HAS_LSE_ATOMICS)          */
```

THE TRAP THIS CREATES

Code compiled with `-march=armv8.2-a+crypto` will execute an illegal-instruction fault on a chip lacking those extensions — usually as `SIGILL`, often long after startup, in a library you did not write. Distribution binaries therefore target a baseline and dispatch at runtime. When you see "Illegal instruction" on one machine and not another, this is almost always the reason.

How it shows up in practice

Symptom	ISA-level cause
`SIGILL` on one machine, fine on another	Built for an architecture extension the target does not implement
Disassembly is garbage after a certain point	Disassembling from a wrong offset. On ARM this self-corrects at the next 4-byte boundary; on x86 it may never resync — a direct consequence of variable-length encoding.
A constant needs four instructions to load	A 64-bit immediate does not fit in a 32-bit instruction; it is built with `MOVZ` + three `MOVK`s, or loaded from a literal pool
Code is much larger than the equivalent x86	Fixed-width encoding; consider `-Os`, and on 32-bit ARM, Thumb
A branch target "out of range" link error	Branch offsets are limited by the encoding (±128 MB for `B`); the linker inserts a veneer using `X16` / `X17`
Atomics are much faster after a kernel upgrade	The kernel began patching in LSE atomics after detecting the feature

Check yourself

- ✓ State the difference between architecture and microarchitecture, and give a bug type belonging to each.
- ✓ List six things an ISA specifies and two things it deliberately does not.
- ✓ Give the original reasoning behind CISC and explain which of its assumptions stopped being true.
- ✓ Explain why fixed-width encoding matters more for wide superscalar decode than for anything else.
- ✓ Decode `0x8B020020` field by field.
- ✓ Explain what a µop cache is compensating for.
- ✓ Describe how an ISA adds features without breaking old binaries, and the failure mode when software gets it wrong.

The datapath and the control unit

This is the chapter where a processor stops being a metaphor. Using only the blocks from chapters 05 and 06, we build a machine that actually executes instructions — and every later idea (pipelining, forwarding, speculation, exceptions) is a modification of this one drawing.

The problem

You have an ALU, a register file, muxes, decoders and memory. You have an instruction encoding from chapter 09. How do you wire them together so that fetching a 32-bit word from memory causes the right thing to happen?

The answer splits cleanly into two halves, and keeping them separate is the single most useful idea here:

DATAPATH AND CONTROL

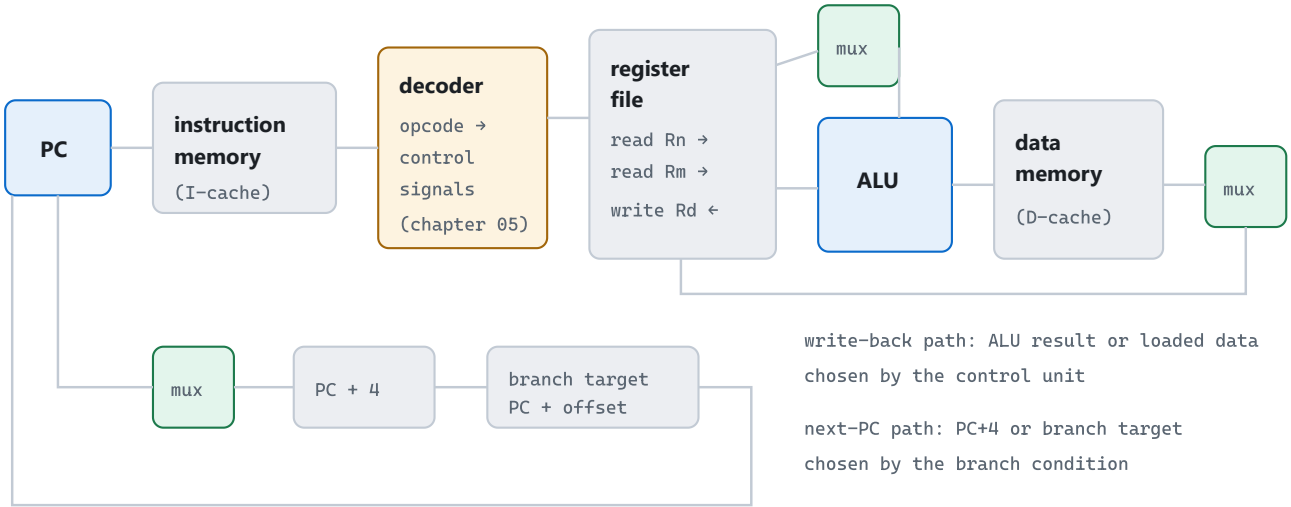
The **datapath** is where values flow: registers, ALU, memory ports, and the muxes that route between them. It is the plumbing, and it is the same for every instruction.

The **control unit** decides, for the current instruction, how every mux is set and which units are enabled. It is combinational logic driven by the opcode — the decoder from chapter 05.

The datapath makes everything *possible*; the control unit makes one thing *happen*.

A single-cycle datapath

Start with the simplest working design: every instruction completes in exactly one clock cycle.



A single-cycle datapath. Every wire exists for every instruction; the control unit decides which ones matter.

Walking three instructions through it

```
1.  ADD X0, X1, X2         register-to-register arithmetic

    PC → I-memory → instruction
    decode: opcode = ADD, Rn = 1, Rm = 2, Rd = 0
    register file: read X1 and X2
    ALU-src mux: select the REGISTER path (not the immediate)
    ALU: op = add
    mem-write: OFF, mem-read: OFF        ← data memory unused
    write-back mux: select the ALU RESULT
    register file: write X0
    next-PC mux: select PC + 4

2.  LDR X0, [X1, #16]      load

    decode: opcode = LDR, Rn = 1, Rd = 0, immediate = 16
    register file: read X1
    ALU-src mux: select the IMMEDIATE
    ALU: op = add           ← the ALU computes the ADDRESS, not a value
    data memory: read at that address
    write-back mux: select the MEMORY DATA
    register file: write X0
    next-PC mux: PC + 4

3.  CBZ X0, label          branch if zero

    decode: opcode = CBZ, Rn = 0, offset = label - PC
    register file: read X0
    ALU: compare against zero → sets the Z flag
    register write: OFF     ← nothing is written back
    next-PC mux: select the BRANCH TARGET if Z, else PC + 4
```

Notice that the ALU did three different jobs: arithmetic, address computation, and comparison. That reuse is deliberate and is why the ALU sits at the centre of the drawing.

The control unit

For this datapath, the control unit produces roughly eight signals. They are a direct function of the opcode — pure combinational logic, exactly the decoder of chapter 05.

Signal	Controls	`ADD`	`LDR`	`STR`	`CBZ`
`RegWrite`	Write the register file?	1	1	0	0
`ALUSrc`	Second ALU input: reg or immediate	reg	imm	imm	—
`ALUOp`	Which ALU function	add	add	add	sub
`MemRead`	Read data memory?	0	1	0	0
`MemWrite`	Write data memory?	0	0	1	0
`MemToReg`	Write back ALU result or memory data	ALU	mem	—	—
`Branch`	Allow the PC mux to take the target	0	0	0	1
`SetFlags`	Update NZCV	0	0	0	1

Reading that table row by row tells you what each instruction *is*, in hardware terms. A store is "a load with the memory direction reversed and no write-back". A compare is "a subtract that does not write its result". These are not analogies — they are literally the same wires with different control bits.

HARDWIRED VERSUS MICROCODED CONTROL

Hardwired: the control signals are produced by combinational logic directly from the opcode. Fast, small, inflexible. This is the RISC approach and how ARM implements the common instructions.

Microcoded: the opcode indexes a small ROM containing a *sequence* of control-signal sets, stepped through over several cycles by a microprogram counter. Slower, but it makes complex multi-step instructions possible and lets the manufacturer fix bugs by updating the microcode after shipping — which is exactly how Spectre and Meltdown mitigations were delivered on x86.

Modern chips use both: hardwired for the common cases, microcode for the complex or rare ones.

Why single-cycle is a bad design

It works, and it is easy to reason about. It is also slow, for a reason that follows directly from chapter 07.

```
The clock period must accommodate the LONGEST instruction path.

  ADD:  I-fetch  reg-read  ALU  reg-write
        250 ps + 150 ps + 200 ps + 100 ps            =  700 ps

  LDR:  I-fetch  reg-read  ALU  D-mem  reg-write
        250 ps + 150 ps + 200 ps + 250 ps + 100 ps   =  950 ps  ← longest

  clock period = 950 ps  →  f_max ≈ 1.05 GHz

Every ADD now takes 950 ps even though it needed only 700 —
250 ps of every arithmetic instruction is spent doing nothing.

Worse: each hardware unit is used for a small slice of the cycle
and sits idle for the rest. The memory port is busy for 250 of
950 ps. That is 74% waste on the most expensive resource.
```

The multi-cycle detour

One historical fix: break each instruction into several shorter cycles, and let different instructions take different numbers of cycles. `ADD` takes 4, `LDR` takes 5. The clock period drops to the longest single *step* rather than the longest instruction, and units can be reused across steps (one memory port serving both fetch and data access).

This improves clock frequency but not throughput — you are still finishing at most one instruction every 4–5 cycles. The real answer is to overlap instructions, which is chapter 11.

Inside the register file

Worth opening up, because its port count is a constraint that shapes ISAs.

```
A register file with 2 read ports and 1 write port

   read addr A (5 bits) ──► decoder ──┐
                                      ├──► mux tree ──► read data A
   read addr B (5 bits) ──► decoder ──┤
                                      └──► mux tree ──► read data B

   write addr  (5 bits) ──► decoder ──► enable one register's flip-flops
   write data           ──────────────► all registers' D inputs
   write enable         ──────────────► gates the decoder

Each additional port costs area roughly QUADRATICALLY (more wires
crossing more cells) and adds delay. This is why:

  · most instructions have at most 2 source and 1 destination register
  · LDP/STP (load/store PAIR) are special — they need extra ports,
    and are provided anyway because function prologues use them constantly
  · a wide out-of-order core needs many ports and the register file
    becomes one of the hottest, most area-expensive blocks on the die
```

> **AN ISA DESIGN DECISION YOU CAN NOW EXPLAIN**
> Why does AArch64 have no instruction with three source registers and two destinations? Because the register file ports would cost more than the instruction saves. When you see an ISA restriction, the usual explanation is a port count, a wire, or a decoder — not a lack of imagination.

Where exceptions enter the datapath

Every stage can fail, and the datapath must handle it. This is the hardware behind ARM chapter 28.

Stage	Possible failure	Datapath response
Fetch	Instruction address faults (unmapped, no-execute)	Squash the instruction; force the PC mux to the exception vector
Decode	Undefined encoding, or an instruction not permitted at this privilege	Same; record the reason in a syndrome register
Execute	Divide by zero, FP exception	Suppress the register write
Memory	Data address faults	Suppress write-back; latch the faulting address
Any time	Interrupt asserted	Complete instructions already past the point of no return, squash the rest, redirect the PC

The requirement that makes this hard is **precise exceptions**: from software's point of view, every instruction before the faulting one must have completed fully, and every instruction after it must have had no effect at all. Getting that guarantee out of a machine that runs instructions out of order and speculatively is one of the central difficulties of processor design — and the reason RISC's "only loads and stores touch memory" rule was such a good idea.

How this shows up in software

Observation	Datapath reason
`CMP` and `SUBS` have identical timing	Same wires; `CMP` just has `RegWrite = 0`
`LDR X0, [X1, X2, LSL #3]` costs no more than `LDR X0, [X1]`	The shifter and adder are already on the address path

Observation	Datapath reason
A load's result is not usable by the very next instruction without a stall	The memory stage is late in the datapath — the load-use hazard of chapter 11
Integer division takes 10–40× a multiply	No single-cycle divider exists; it iterates and does not fit the one-pass datapath
A faulting instruction reports the address that faulted, precisely	The datapath latches it and suppresses write-back
Microcode updates can change CPU behaviour after manufacture	Microcoded control paths are patchable; hardwired ones are not

Check yourself

- ✓ State the difference between datapath and control, and say which one changes per instruction.
- ✓ Trace `LDR X0, [X1, #16]` through the datapath naming every mux setting.
- ✓ Explain why the ALU is used for three unrelated purposes.
- ✓ Give the control-signal differences between `ADD` and `CMP`.
- ✓ Explain why single-cycle design wastes time, using the clock-period argument.
- ✓ Say why register-file port count constrains instruction set design.
- ✓ Define a precise exception and say why it is hard to provide.

Pipelining and hazards

Pipelining is the single most important idea in processor design, and the source of almost every counter-intuitive performance result you will ever measure. It also explains why the ARM memory model is weak, why branches are expensive, and why `ISB` exists.

The problem

Chapter 10 ended with an uncomfortable fact: in a single-cycle machine, every functional unit is idle most of the time. The memory port is busy for 250 ps out of a 950 ps cycle; the ALU for 200. You have paid for hardware that spends three quarters of its life waiting.

The insight is the same as an assembly line. You do not build one car at a time from start to finish. You divide the work into stations and keep a car at every station.

The mechanism

Insert registers (chapter 06) between the datapath stages. Each stage now works on a *different instruction* in the same cycle.

```
Classic 5-stage pipeline

  IF  — instruction fetch
  ID  — instruction decode and register read
  EX  — execute in the ALU
  MEM — data memory access
  WB  — write back to the register file

Sequential (no pipeline):

  cycle:  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
  i1:    IF  ID  EX MEM  WB
  i2:                        IF  ID  EX MEM  WB
  i3:                                        IF  ID  EX MEM  WB
                             ↑ 5 cycles per instruction

Pipelined:

  cycle:  1   2   3   4   5   6   7   8   9
  i1:    IF  ID  EX MEM  WB
  i2:        IF  ID  EX MEM  WB
  i3:            IF  ID  EX MEM  WB
  i4:                IF  ID  EX MEM  WB
  i5:                    IF  ID  EX MEM  WB
                          ↑
                    at this point FIVE instructions are in flight,
                    and one COMPLETES every cycle
```

LATENCY VERSUS THROUGHPUT

Pipelining does **not** make any individual instruction faster. Each still takes 5 stages — in fact slightly longer, because of the register overhead at each boundary. What changes is **throughput**: instructions complete at one per cycle instead of one per five.

This distinction is worth internalising because it recurs everywhere in systems work. A DMA engine does not reduce the latency of a transfer; it removes the CPU from it. An asynchronous I/O API does not make the disk faster. Confusing the two leads to optimisations that measure worse than the thing they replaced.

Why the clock also gets faster

```
Single cycle:   period = 250 + 150 + 200 + 250 + 100 = 950 ps  →  1.05 GHz

Pipelined:      period = longest SINGLE stage + register overhead
                       = 250 (IF or MEM) + 20                  →  3.7 GHz

Speedup ≈ 3.5× from frequency, plus the throughput gain.

Combined, an ideal 5-stage pipeline approaches 5× the single-cycle
machine — which is why every processor built since about 1985 is pipelined.
```

Pipeline depth in real designs

Core	Stages	Character
Cortex-M0	3	Fetch, decode, execute. Tiny, low power, very predictable timing.
Cortex-A53	8	In-order, efficiency-focused
Cortex-A78 / X-series	13–15	Out-of-order, deep front end
Pentium 4 (historical)	20–31	Chased clock frequency; mispredicts cost 30+ cycles. A cautionary tale.

Deeper is not simply better. Each extra stage shortens the clock period but increases the penalty when the pipeline must be flushed. The Pentium 4 pushed this to 31 stages, discovered the mispredict penalty was crippling, and the industry settled back to 10–15.

Hazards: when the assembly line breaks

The pipeline assumes instructions are independent. They are not. A **hazard** is any situation where the next instruction cannot execute in the next cycle.

Structural hazards

Two instructions need the same hardware in the same cycle. In the 5-stage pipeline above, IF needs memory and MEM needs memory — simultaneously.

```
  cycle:      4
  i1:        MEM   ← needs the memory port
  i4:        IF    ← also needs the memory port

Solution: separate instruction and data caches — the modified Harvard
architecture of chapter 08. This is not an optimisation; it is a
structural necessity for a pipeline.
```

Data hazards

An instruction needs a result that is still in flight.

```
    ADD X1, X2, X3      ; X1 is produced...
    SUB X4, X1, X5      ; ...and needed immediately

  cycle:    1    2    3    4    5    6
  ADD:     IF   ID   EX  MEM   WB
                           ↑ X1 written here
  SUB:          IF   ID   EX  MEM   WB
                     ↑ X1 read here — TWO CYCLES TOO EARLY

Naive fix: stall SUB for two cycles. Correct, but slow — and this
dependency pattern is extremely common in real code.
```

Forwarding (bypassing)

The result exists at the end of EX. It just has not reached the register file yet. So add wires that route it directly from the pipeline register to the ALU input — through a mux (chapter 05), naturally.

```
    ADD X1, X2, X3
    SUB X4, X1, X5

  cycle:    1    2    3    4    5
  ADD:     IF   ID   EX  MEM   WB
                     │
                     └─────── forward ──────┐
                                            ▼
  SUB:          IF   ID   EX  MEM   WB
                          ↑ uses the forwarded value — no stall

The forwarding network is a set of muxes on the ALU inputs, selecting
between the register file and results still in the pipeline. It is one
of the messiest parts of a real design and it eliminates most data
hazards entirely.
```

The load-use hazard: the one forwarding cannot fix

```
    LDR X1, [X2]        ; X1 comes from MEMORY
    ADD X3, X1, X4      ; needed in EX, one stage EARLIER

  cycle:    1    2    3    4    5
  LDR:     IF   ID   EX  MEM   WB
                          │ data available only at the END of MEM
                          └─┐
                            ▼ but ADD needs it at the START of EX
  ADD:          IF   ID  [stall]  EX   MEM   WB

You cannot forward backwards in time. One cycle of stall is unavoidable.
```

WHAT THIS MEANS FOR THE CODE YOU WRITE

The compiler schedules instructions to fill load-use gaps with unrelated work. That is why `-O2` output looks "shuffled" compared to your source: loads are hoisted early and their uses pushed later. It is also why a tight dependent pointer chase — `p = p->next` repeated — is much slower per instruction than it looks: every iteration is a load-use hazard, and on a cache miss it is hundreds of cycles rather than one.

```
/* dependent: each load must complete before the next can start */
while (p) { sum += p->val; p = p->next; }        /* latency-bound  */

/* independent: many loads in flight at once */
for (i = 0; i < n; i++) sum += a[i];             /* bandwidth-bound */

/* the second can be 10x faster for the same amount of data —
   this is why array-of-structs versus linked list matters so much */
```

Control hazards

The worst kind. When a branch is fetched, the processor does not yet know where to fetch next — the condition is not evaluated until EX.

```
    CBZ X0, target
    ADD ...             ; should these execute?
    SUB ...             ; nobody knows yet

  cycle:    1    2    3    4
  CBZ:     IF   ID   EX  ← the outcome is known only HERE
  ADD:          IF   ID  ← already fetched
  SUB:               IF  ← already fetched

If the branch is taken, both must be discarded — the pipeline is
FLUSHED and refilled from the target.

  Penalty ≈ number of stages between fetch and branch resolution.
    5-stage in-order   :  2–3 cycles
    15-stage out-of-order: 15–20 cycles
```

Branches are roughly one instruction in five. A 15-cycle penalty on 20% of instructions would triple execution time — completely unacceptable. The response is **branch prediction**, which is significant enough to have its own chapter (chapter 12).

Approach	How it works	Where used
Stall	Freeze until the outcome is known	Simple microcontrollers
Branch delay slot	The instruction after a branch *always* executes; the compiler fills the slot	MIPS, SPARC. An ISA-visible hack — never adopted by ARM, and now regretted by its users because it locks the pipeline depth into the architecture.
Predict and speculate	Guess, execute speculatively, flush if wrong	Everything modern
Predication	Execute both paths, discard one — a mux instead of a branch	ARM's `CSEL`, `CSET`; ARMv7's fully predicated instructions

Where the pipeline becomes software-visible

Pipelining is supposed to be invisible. Three places where it is not, and all three matter to systems programmers.

1. System register writes need ISB

Instructions already in the pipeline were fetched and decoded under the *old* configuration. If you enable the MMU, change `VBAR`, or alter `SCTLR`, instructions already in flight do not see the change.

```
    msr     sctlr_el1, x0       // enable the MMU
    isb                         // MANDATORY: flush the pipeline so that
                                // everything after this is re-fetched under
                                // the new configuration
    ldr     x1, [x2]            // this now goes through the MMU
```

`ISB` is a context-synchronising instruction: it discards everything prefetched and restarts the front end. Omitting it produces behaviour that varies by core and by build — the classic "works in the emulator, fails on silicon" bug.

2. Self-modifying code needs the full maintenance sequence

Instructions may already have been fetched into the pipeline and the instruction cache before you rewrote them in memory. Chapter 08 covered the cache half; `ISB` handles the pipeline half.

3. The weak memory model is a consequence of pipelining

Because instructions overlap and complete out of order, the order in which their memory effects become visible to other cores is not the program order. This is not a defect; it is the direct, unavoidable consequence of the assembly line. Recovering ordering when you need it is what barriers are for.

Quantifying it

```
   CPI = cycles per instruction

   Ideal pipeline:        CPI = 1.0
   Real:                  CPI = 1.0 + stalls per instruction

   CPI = 1
       + (branch frequency × mispredict rate × mispredict penalty)
       + (load frequency × load-use stall rate × 1)
       + (memory frequency × cache miss rate × miss penalty)      ← usually dominant
       + structural and other stalls

   Worked example:
       branches 20%, mispredict 5%, penalty 15   → 0.20 × 0.05 × 15 = 0.15
       loads 25%, load-use 30%, penalty 1        → 0.25 × 0.30 × 1  = 0.075
       memory ops 35%, L1 miss 2%, penalty 200   → 0.35 × 0.02 × 200 = 1.40

       CPI = 1 + 0.15 + 0.075 + 1.40 = 2.63

   Note which term dominates. Cache behaviour outweighs everything
   else combined — which is why chapters 13–15 exist, and why data
   layout beats instruction-level tuning almost every time.
```

```
# measure your own CPI and stall breakdown
perf stat -e cycles,instructions,branch-misses,cache-misses,stalled-cycles-frontend ./program

#  1,234,567,890  cycles
#    469,000,000  instructions      #  0.38 insn per cycle   ← CPI = 2.63
#      2,100,000  branch-misses
#      8,400,000  cache-misses
```

Beyond one instruction per cycle

If one per cycle is good, why not more? A **superscalar** processor widens every stage to handle several instructions at once.

```
  2-wide superscalar

  cycle:   1        2        3        4        5
  i1,i2:  IF/IF   ID/ID   EX/EX  MEM/MEM  WB/WB
  i3,i4:          IF/IF   ID/ID   EX/EX  MEM/MEM
                             ↑
                        two instructions complete per cycle
                        CPI = 0.5, or IPC = 2

Requirements this creates:
  · fetch 8+ bytes per cycle           → wide I-cache port
  · decode N instructions in parallel  → FIXED-WIDTH ENCODING helps enormously
  · more register file ports            → area cost (chapter 10)
  · dependency checking between the instructions issued together
  · multiple ALUs, multiple load/store units
```

Add **out-of-order execution** — buffer instructions, execute any whose operands are ready, retire them in program order to preserve precise exceptions — and you have the modern high-performance core. The reorder buffer is what keeps the *appearance* of sequential execution while the reality is anything but.

> **THE CONNECTION TO SECURITY**
> Speculative and out-of-order execution mean the processor performs work that is later discarded. The architectural state is correctly rolled back — but the *microarchitectural* state (cache contents, predictor history) is not. Spectre and Meltdown exploit exactly that gap: make the processor speculatively touch data it should not, then infer what it touched by timing the cache. This is a direct consequence of this chapter's optimisation, and it is why the distinction between architecture and microarchitecture from chapter 09 is not academic.

How it shows up in software

Observation	Pipeline reason
A branchy loop is far slower than a branchless one	Mispredict flushes; `CSEL` avoids them entirely
Traversing a linked list is much slower than an array with the same data	Dependent loads: every step is a load-use hazard plus a likely cache miss
`-O2` output has your statements reordered	The scheduler is filling hazard slots
A sorted array processes faster than an unsorted one	The branch in the loop becomes predictable — the famous Stack Overflow question
Enabling the MMU hangs on real silicon but works in QEMU	Missing `ISB`; the emulator does not model the pipeline
Two cores observe stores in different orders	Weak memory model, which is pipelining made visible
Timing varies with data values in crypto code	Data-dependent branches leaking through the predictor

Check yourself

✓ Explain why pipelining improves throughput but not latency.

- ✓ Name the five classic stages and one hazard associated with each boundary.
- ✓ Explain forwarding, and give the one hazard it cannot eliminate.
- ✓ Say why deeper pipelines are not automatically better.
- ✓ Explain why separate I- and D-caches are structurally required, not merely a speed choice.
- ✓ Write the CPI equation and say which term usually dominates.
- ✓ Explain exactly why `ISB` is required after enabling the MMU.
- ✓ Describe how pipelining makes Spectre-class attacks possible.

Branch prediction and speculation

A modern processor guesses. It guesses which way branches go, executes hundreds of instructions past the guess, and throws the work away when wrong. This is why sorted data can be faster than unsorted, and it is the mechanism behind Spectre.

The problem

Chapter 11 left us with control hazards. A branch's direction is not known until several stages after it is fetched, but the front end must fetch *something* every cycle.

```
Cost of getting it wrong, by design depth

  pipeline    branch resolved at    flush penalty
  ──────────────────────────────────────────────
  5-stage     stage 3               ~2 cycles
  10-stage    stage 7               ~7 cycles
  15-stage    stage 11              ~15 cycles
  Pentium 4   stage ~20             ~30 cycles

Branches are ~15–25% of instructions in typical code.

  With a 15-cycle penalty and NO prediction:
      CPI = 1 + 0.20 × 15 = 4.0
  With 95% prediction accuracy:
      CPI = 1 + 0.20 × 0.05 × 15 = 1.15

Prediction is not an optimisation. Without it, deep pipelines are
pointless — you would spend all your speed on flushes.
```

Static prediction

The cheapest approach: decide from the instruction alone, with no history.

Heuristic	Rationale	Accuracy
Always not-taken	Simplest; just keep fetching sequentially	~40%
Always taken	Most branches in loops are taken	~60%
Backward taken, forward not-taken	A backward branch is almost always a loop; a forward branch is usually an error path	~70%
Compiler hint	Profile-guided, encoded in the instruction or in layout	~80%

```
/* Linux and GCC let you supply the hint explicitly */
#define likely(x)   __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)

if (unlikely(ptr == NULL))          /* error path: moved out of line     */
        return -EINVAL;             /* so the fall-through is the common */
                                    /* case and stays in the I-cache     */

/* what this actually does: the compiler reorders basic blocks so the
   likely path is contiguous. On modern cores the dynamic predictor
   outperforms the hint within a few iterations — the real benefit is
   INSTRUCTION CACHE LAYOUT, not the prediction itself. */
```

> **WHERE HINTS STILL GENUINELY MATTER**
> On a small in-order core (Cortex-M, Cortex-A53) with a weak or absent dynamic predictor, and on the very first execution of any branch before history exists. In hot loops on a big core, remove them — they add noise and the hardware already knows better. Profile before adding one.

Dynamic prediction

One-bit predictor

Remember what the branch did last time. Keep a table of bits indexed by branch address.

```
   predict = last outcome

   for (i = 0; i < 10; i++) { ... }

   iterations:  T T T T T T T T T N       (N = the final exit)
   prediction:  ? T T T T T T T T T
   correct:     ✗ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✗

   Two mispredicts per loop execution: the exit, and then the first
   iteration of the NEXT execution, because the table now says "not taken".

   For a nested loop run many times, that doubling is significant.
```

Two-bit saturating counter

Require two consecutive surprises before changing the prediction. This is the fundamental unit of nearly every real predictor.

```
          taken            taken            taken
      ┌──────────┐     ┌──────────┐     ┌──────────┐
      ▼          │     ▼          │     ▼          │
  ┌──────┐     ┌──────┐     ┌──────┐     ┌──────┐
  │ 00   │     │ 01   │     │ 10   │     │ 11   │
  │ SN   │◄────│ WN   │◄────│ WT   │◄────│ ST   │
  └──────┘     └──────┘     └──────┘     └──────┘
          n-taken      n-taken      n-taken
 strongly       weakly          weakly          strongly
 NOT taken      NOT taken       TAKEN           TAKEN

 predict TAKEN if the high bit is 1.

 Same loop:  T T T T T T T T T N
 now the single N moves ST → WT, still predicting taken.
 Next execution starts correct. ONE mispredict per loop, not two.
```

Correlating and tournament predictors

Two-bit counters cannot capture patterns. Real code has branches that depend on other branches:

```
if (x < 5)  a = 1;      /* branch A */
if (y < 3)  b = 1;      /* branch B */
if (a && b) c = 1;      /* branch C — fully determined by A and B */
```

A **global history register** shifts in the outcome of every branch, forming a pattern. Indexing the counter table by (branch address XOR global history) lets the predictor learn "when the last few branches went TNT, this one is taken".

Predictor	Idea	Typical accuracy
2-bit bimodal	Counter per branch address	~85%
gshare	Index by address XOR global history	~93%
Tournament	Run a local and a global predictor, plus a meta-predictor that learns which to trust per branch	~95%
TAGE	Several tables with different history lengths; the longest matching one wins	~97–99%
Perceptron / neural	A weighted sum over a long history vector	~97–99%

Modern cores use TAGE or perceptron variants with thousands of entries and history lengths in the hundreds. The predictor can be several percent of core area — it is that valuable.

Predicting the target, not just the direction

Knowing a branch is taken is useless without knowing *where*. Three structures handle this:

Structure	Handles	How
BTB — branch target buffer	Direct branches	Cache mapping branch address → last target. Available during fetch, before decode.
Indirect predictor	`BR Xn` — virtual calls, switch tables, function pointers	History-indexed target table; harder, since the target varies
RAS — return address stack	`RET`	A hardware stack: `BL` pushes, `RET` pops. Near-perfect, because calls and returns nest.

TWO WAYS SOFTWARE BREAKS THE RETURN ADDRESS STACK

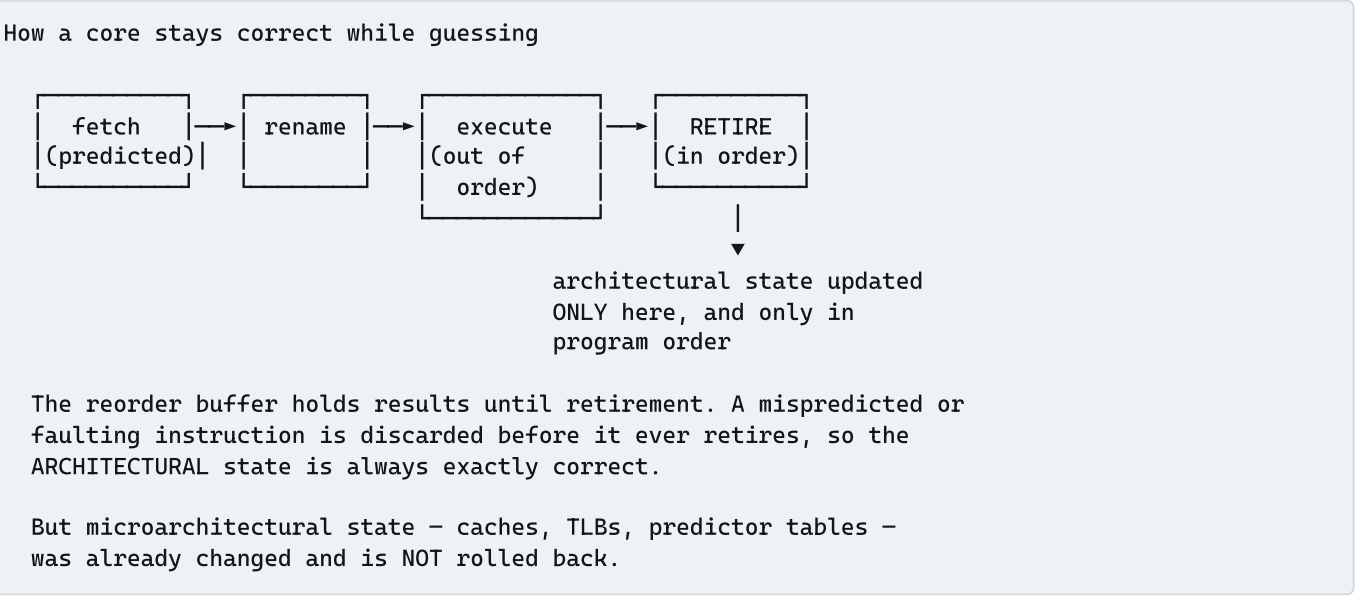
The RAS assumes calls and returns are balanced. Break that assumption and you lose a near-100% predictor:

- **Unbalanced call/return.** `setjmp` / `longjmp`, C++ exception unwinding, coroutine switches and hand-written context switches all return to somewhere the RAS did not predict. The stack stays misaligned for several subsequent returns, so the cost is more than one mispredict.
- **Using `BL` as a plain jump**, or `RET` as an indirect branch. Some hand-written assembly does this to save an instruction; it corrupts the RAS. ARM added `BLR` target hints and `RETAA` partly to keep these paths honest.

Speculation: acting on the guess

Prediction only helps if the processor *proceeds* on it. Modern cores execute far past unresolved branches, and beyond that, speculate on more than branch direction:

Kind	Guess	Recovery when wrong
Control speculation	Which way a branch goes	Flush and refetch
Memory dependence speculation	That a load does not alias an earlier pending store	Replay the load and everything after it
Value prediction	The result of a long-latency operation	Rare in shipping designs
Prefetch speculation	Which cache lines will be needed	No correctness impact — just wasted bandwidth



Spectre and Meltdown

That last line is the whole vulnerability class. Speculation is architecturally invisible and microarchitecturally loud.

```
Spectre variant 1 — bounds check bypass

  if (index < array1_size)          ← attacker trains this to predict TAKEN
          y = array2[array1[index] * 4096];

  1. Attacker calls the function repeatedly with valid indices.
     The predictor learns: this branch is taken.

  2. Attacker calls with index far out of bounds, having evicted
     array1_size from cache so the check is SLOW to resolve.

  3. The core speculates "taken", reads array1[index] — memory it
     must not read — and uses that secret byte to index array2.
     A cache line in array2 is loaded, chosen by the secret value.

  4. The check resolves. Speculation is discarded. y is never written.
     Architecturally, nothing happened.

  5. Attacker times reads across array2. One line is fast.
     Its index reveals the secret byte.

The bounds check was CORRECT. It was simply evaluated too late.
```

Variant	Mechanism	Mitigation
Spectre v1	Bounds check bypass	Index masking + `CSDB`; Linux's `array_index_nospec()`
Spectre v2	Branch target injection — poison the indirect predictor	Retpolines, `BHB` flushing, hardware `CSV2`
Meltdown	Speculative read across the privilege boundary before the permission check retires	KPTI: unmap the kernel from user page tables. Most ARM cores were never vulnerable.
MDS / L1TF	Leakage from internal buffers	Microcode buffer flushes on transitions

```
/* the Linux kernel's Spectre-v1 mitigation, and how to read it */
if (index < size) {
        index = array_index_nospec(index, size);
        /* Expands to a data dependency that forces index to 0 when out
         * of bounds — a mask computed WITHOUT a branch, so speculation
         * cannot bypass it. On arm64:
         *     cmp   x1, x2
         *     csel  x1, x1, xzr, lo     ; a mux, not a branch
         *     csdb                      ; speculation barrier
         */
        val = array[index];
}

/* ARM's speculation control instructions */
    CSDB    // consumption of speculative data barrier
    SSBB    // speculative store bypass barrier
    SB      // full speculation barrier (ARMv8.5)
```

Making your code predictable

```
/* The famous case: this loop is ~6x faster when the data is sorted */
for (i = 0; i < n; i++)
        if (data[i] >= 128)
                sum += data[i];

/* Sorted:   the branch goes one way for a long run — near-perfect prediction.
   Unsorted: 50/50 random — the predictor cannot do better than chance,
             and every other iteration costs a full flush.

   The fix that makes sorting irrelevant: remove the branch. */

for (i = 0; i < n; i++) {
        int mask = -(data[i] >= 128);   /* 0 or -1 (all ones) */
        sum += data[i] & mask;          /* a MUX, chapter 05  */
}
/* compiles to CSEL/CSINC on AArch64 — no branch, constant time */
```

Guideline	Why
Prefer predictable branches over clever ones	A branch taken 99% of the time is nearly free; a 50/50 branch is very expensive
Hoist unpredictable conditions out of hot loops	Split the loop into two predictable loops rather than one unpredictable one
Use branchless selection for random conditions	`CSEL`, arithmetic masks, `min` / `max`
Prefer `switch` over long `else if` chains	A jump table is one indirect branch instead of many conditional ones
Avoid virtual calls in the hottest loops	Indirect branches are the hardest to predict; devirtualise or batch by type
In cryptographic code, never branch on secrets	Timing and predictor state leak the secret — see crypto chapter 19

```
# measure it
perf stat -e branches,branch-misses ./program
#   1,204,000,000  branches
#      63,000,000  branch-misses   #  5.23% of all branches   ← above ~2% is worth investigating

perf record -e branch-misses ./program && perf report   # find WHICH branch
```

How it shows up

Symptom	Cause
Sorted input is several times faster than unsorted	Branch predictability, not cache behaviour
A microbenchmark is far faster than production	The benchmark's branches are trivially predictable; real inputs are not
Performance collapses after adding a rare error check	The branch became unpredictable, or the error path polluted the I-cache
Indirect calls dominate a profile	Indirect predictor misses; consider devirtualisation
A big kernel-upgrade slowdown	Spectre mitigations: retpolines and barriers cost real cycles
Cryptographic timing varies with the key	Secret-dependent branching — a genuine vulnerability
`setjmp`-heavy code has poor branch statistics	Return address stack corruption

Check yourself

✓ Compute CPI for 20% branches with a 15-cycle penalty at 100%, 90% and 99% accuracy.

- ✓ Draw the two-bit saturating counter and explain why it beats one bit on loops.
- ✓ Explain what a global history register adds that a bimodal predictor cannot capture.
- ✓ Name the three target-prediction structures and which branch type each serves.
- ✓ Give two ways software corrupts the return address stack.
- ✓ Explain Spectre v1 in five steps, and say why the bounds check was not the bug.
- ✓ Rewrite a data-dependent conditional sum as branchless code.

The memory hierarchy and locality

The gap between processor speed and memory speed is the dominant fact of modern computing. This chapter explains the illusion built to hide it, and why data layout beats almost every other optimisation you can make.

The problem

You want memory that is large, fast and cheap. Physics permits any two.

Technology	Access time	Typical size	Cost per bit	Why
Register	~0.3 ns (1 cycle)	~1 KB total	Enormous	Flip-flops right beside the ALU
SRAM (cache)	1–20 ns	32 KB – 32 MB	High	6 transistors per bit, no refresh
DRAM	50–100 ns	1–512 GB	Moderate	1 transistor + 1 capacitor, must be refreshed
NAND flash / SSD	50–100 µs	0.1–10 TB	Low	Charge trapped on a floating gate
Hard disk	5–10 ms	1–20 TB	Very low	A spinning platter and a moving arm

```
Put those numbers on a human scale. Scale one CPU cycle to one second:

  L1 cache hit          4 cycles     →  4 seconds
  L2 cache hit         12 cycles     →  12 seconds
  L3 cache hit         40 cycles     →  40 seconds
  DRAM                200 cycles     →  3 minutes 20 seconds
  SSD read       ~200,000 cycles     →  2.5 days
  Disk seek   ~20,000,000 cycles     →  8 months

A DRAM access is not "a bit slower". Relative to the core, it is a
coffee break. Everything in this chapter exists to avoid taking it.
```

Locality: the property that makes caching work

A cache is only useful if past accesses predict future ones. Real programs oblige, in two distinct ways:

THE TWO LOCALITIES

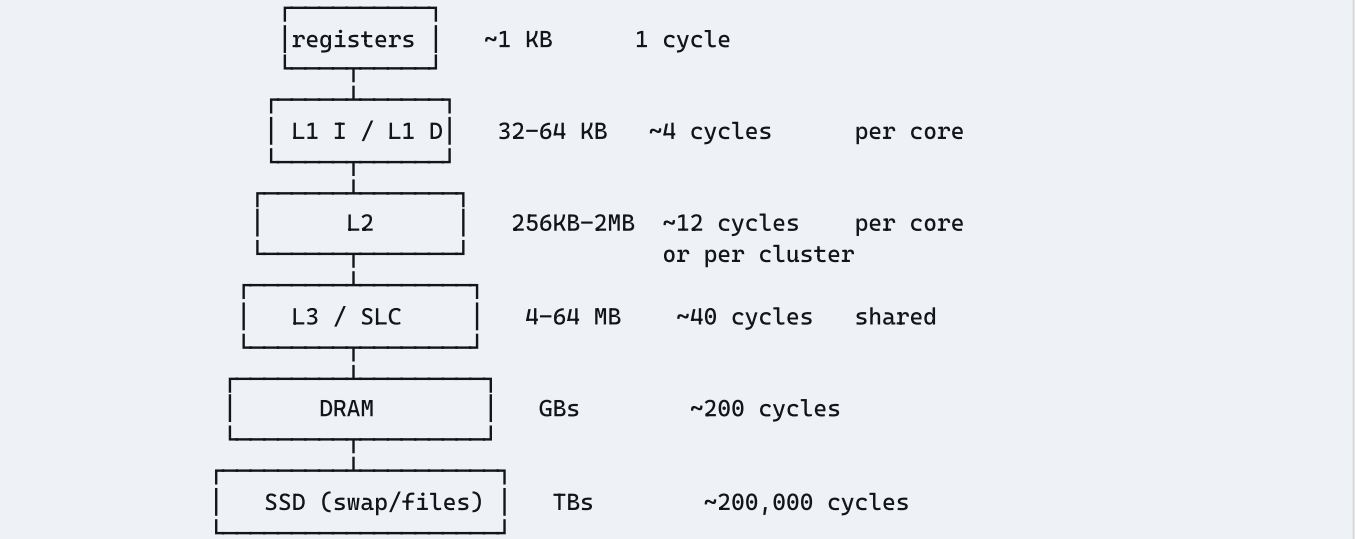
Temporal locality — if you accessed an address, you will probably access it again soon. Loop counters, the stack top, a hot data structure, function code inside a loop.

Spatial locality — if you accessed an address, you will probably access a nearby one soon. Array traversal, struct field access, sequential instruction fetch.

Caches exploit temporal locality by *keeping* recently used data, and spatial locality by *fetching a whole line* rather than a single word. Code with neither gets no benefit at all — which is why a random-access hash table over a large working set can be slower than a linear scan of an array containing the same data.

```
/* excellent locality — this is nearly free */
for (i = 0; i < n; i++)
        sum += a[i];        /* spatial: consecutive addresses
                               temporal: sum and i stay in registers */

/* terrible locality */
for (i = 0; i < n; i++)
        sum += a[perm[i]];  /* random order: every access a likely miss */

/* the classic demonstration: same work, 10x difference */
for (i = 0; i < N; i++)          for (j = 0; j < N; j++)
  for (j = 0; j < N; j++)          for (i = 0; i < N; i++)
    total += m[i][j];                total += m[i][j];

/* left: row-major, walks memory sequentially — one miss per 16 elements
   right: strides by a whole row — one miss per ELEMENT               */
```

The hierarchy




```
Each level is a CACHE for the level below it, and each is roughly
10x larger and 3–5x slower than the one above.

The same principle appears again at every scale: the page cache caches
the disk, a CDN caches an origin server, a TLB caches page table walks.
Once you see the pattern you will find it everywhere in systems.
```

Inclusive, exclusive and the "not on the critical path" question

Policy	Meaning	Trade-off
Inclusive	Everything in L1 is also in L2	Coherency is simple: snoop L3 and you know what the cores hold. Wastes capacity.
Exclusive	A line lives at exactly one level	Maximum effective capacity; coherency needs more bookkeeping.
Non-inclusive	Neither guaranteed	The common compromise on ARM designs

Quantifying it

```
Average Memory Access Time

  AMAT = hit_time + miss_rate × miss_penalty

  With a hierarchy, recurse:

  AMAT = t_L1 + m_L1 × (t_L2 + m_L2 × (t_L3 + m_L3 × t_DRAM))

  Worked example:
      t_L1 = 4,   m_L1 = 5%
      t_L2 = 12,  m_L2 = 40%     (of L1 misses)
      t_L3 = 40,  m_L3 = 50%     (of L2 misses)
      t_DRAM = 200

  AMAT = 4 + 0.05 × (12 + 0.40 × (40 + 0.50 × 200))
       = 4 + 0.05 × (12 + 0.40 × 140)
       = 4 + 0.05 × 68
       = 7.4 cycles

  Now halve the L1 miss rate to 2.5% (better data layout):
  AMAT = 4 + 0.025 × 68 = 5.7 cycles — a 23% improvement in ALL
  memory access time, from changing nothing but how data is arranged.
```

> **WHY THE MISS RATE MATTERS MORE THAN THE HIT TIME**
> Look at the leverage in that equation. Reducing L1 hit time from 4 to 3 cycles is a heroic hardware change worth ~13%. Halving the miss rate is often a source-level change worth ~23%. This is the arithmetic behind the rule of thumb that **data layout beats instruction-level optimisation** — and it is why profiling tools report cache misses so prominently.

The three kinds of miss

Type	Cause	Cure
Compulsory (cold)	First ever access to a line — nothing could have been cached	Prefetching; larger lines. Unavoidable in principle.
Capacity	The working set is larger than the cache	Shrink the working set: blocking/tiling, smaller data types, better packing
Conflict	Too many hot lines map to the same set, even though the cache is not full	Higher associativity, or change the alignment/stride of your data (chapter 14)

A fourth appears on multi-core: **coherence misses**, where another core invalidated your line. Covered in chapter 15.

Prefetching

A miss costs 200 cycles only if you wait for it. If the line arrives before you ask, the cost is zero.

Kind	Mechanism	Catches
Next-line	Fetch line N+1 whenever N is fetched	Sequential scans, instruction fetch
Stride	Detect a constant stride and extrapolate	Array walks, matrix rows, struct arrays

Kind	Mechanism	Catches
Software	`PRFM PLDL1KEEP, [addr]` — an explicit hint	Pointer chasing, irregular patterns the hardware cannot see

```
/* software prefetch: useful when you know the future and hardware does not */
for (n = list; n; n = n->next) {
        __builtin_prefetch(n->next);    /* start fetching the NEXT node   */
        process(n->data);               /* while we work on this one      */
}

/* prefetch distance matters — too close and it has not arrived,
   too far and it is evicted before use. Tune by measurement.        */
#define DIST 8
for (i = 0; i < n; i++) {
        __builtin_prefetch(&a[i + DIST]);
        heavy_work(a[i]);
}
```

PREFETCHING IS NOT FREE

Every prefetch consumes memory bandwidth and a cache line's worth of capacity. Aggressive prefetching on a bandwidth-saturated system makes things *worse* by evicting useful data and competing for the memory controller. Measure; do not assume. On modern cores the hardware stride prefetcher already handles regular patterns better than you will by hand — reserve software prefetch for genuinely irregular access.

Designing for the hierarchy

Structure of arrays versus array of structures

```
/* AoS — natural to write, poor for scanning one field */
struct particle { float x, y, z, vx, vy, vz; int id; float mass; };
struct particle p[100000];              /* 32 bytes each               */

for (i = 0; i < n; i++) p[i].x += p[i].vx;
/* each 64-byte line holds 2 particles, and you use 8 of its 64 bytes.
   87% of every line fetched is wasted bandwidth. */

/* SoA — awkward to write, excellent for scanning */
struct particles {
        float x[100000], y[100000], z[100000];
        float vx[100000], vy[100000], vz[100000];
};
for (i = 0; i < n; i++) ps.x[i] += ps.vx[i];
/* every byte of every fetched line is used, and it VECTORISES —
   the compiler can now emit NEON/SVE operating on 4–8 elements at once */
```

Blocking (tiling)

```
/* naive matrix multiply: B is walked column-wise — a miss per element */
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];

/* blocked: work on tiles small enough that A, B and C tiles all fit
   in L1 simultaneously. Same arithmetic, dramatically fewer misses. */
for (ii = 0; ii < N; ii += BS)
 for (jj = 0; jj < N; jj += BS)
  for (kk = 0; kk < N; kk += BS)
   for (i = ii; i < ii+BS; i++)
    for (j = jj; j < jj+BS; j++)
     for (k = kk; k < kk+BS; k++)
      C[i][j] += A[i][k] * B[k][j];

/* choose BS so 3 * BS * BS * sizeof(float) < L1 size.
   For 32 KB L1 and floats: BS ≈ 50, so 32 or 48 in practice.
   This is the single most important optimisation in dense linear
   algebra, and therefore in machine learning kernels — see AI ch. 04. */
```

Hot/cold splitting

```
/* before: rarely-used fields evict useful data from every line */
struct conn {
        int fd;  int state;  uint64_t bytes;      /* hot: touched per packet */
        char peer_name[64]; time_t created;       /* cold: touched on error  */
        struct sockaddr_storage addr;             /* cold                    */
};

/* after: the hot struct is small, so more connections fit per line */
struct conn {
        int fd;  int state;  uint64_t bytes;
        struct conn_cold *cold;                   /* one pointer           */
};
```

Measuring it

```
perf stat -e cache-references,cache-misses,\
L1-dcache-loads,L1-dcache-load-misses,\
LLC-loads,LLC-load-misses,dTLB-load-misses ./program

perf record -e cache-misses ./program && perf report   # which line misses?

# on ARM, the PMU exposes the same counters by architectural name
perf list | grep -i cache

# a quick empirical map of your machine's hierarchy: time a pointer
# chase over increasing working-set sizes and look for the step changes.
# The steps land exactly at your L1, L2 and L3 capacities.
```

How it shows up

Symptom	Cause
Performance drops sharply past a specific input size	The working set exceeded a cache level

Symptom	Cause
Swapping loop order changes runtime by 10×	Row-major versus column-major traversal
Linked list is far slower than an array with identical contents	No spatial locality, plus dependent loads (chapter 11)
Adding a field to a struct slows an unrelated loop	The struct crossed a cache-line boundary, halving elements per line
Profile shows high IPC but poor wall-clock throughput	Memory-bandwidth-bound; the core is busy waiting efficiently
A microbenchmark is much faster than production	The benchmark's working set fits in cache; production's does not
Aligning an array to 64 bytes gives a measurable speedup	Removed lines split across two cache lines

Check yourself

- ✓ State the two kinds of locality and give a code example of each.
- ✓ Compute AMAT for a three-level hierarchy and show which term dominates.
- ✓ Name the three classic miss types and the cure for each.
- ✓ Explain why row-major traversal beats column-major, in terms of cache lines.
- ✓ Convert an array-of-structs loop to structure-of-arrays and say what two things improve.
- ✓ Explain blocking and how to choose the block size from cache capacity.
- ✓ Say when software prefetching helps and when it hurts.

Cache organisation and policies

Chapter 13 said caches work. This chapter opens one up: how an address finds its line, why associativity exists, what happens on a write, and why two arrays exactly 4 KB apart can destroy your performance.

The problem

A cache holds a small subset of a huge address space. Two questions must be answered on every access, in about one cycle:

1. Given an address, **where would that line be** if it were cached?
2. **Is it actually there**, or is this a different address that maps to the same place?

Searching every entry would be correct and far too slow. The answer is to make the location a function of the address, and store enough extra bits to confirm identity.

Cache lines

Caches do not store individual bytes. The unit of transfer and tracking is a **line**, almost universally 64 bytes on ARM and x86.

```
Why 64 bytes and not 4?

  Larger lines:  + exploit spatial locality — fetch neighbours for free
                 + fewer tags to store (less overhead)
                 − more wasted bandwidth if you use only part of it
                 − more false sharing between cores (chapter 15)

  64 bytes is the empirical sweet spot, and it has consequences you
  can measure:

    · a 64-byte struct fits exactly one line — align it and every
      access touches one line instead of two
    · reading 1 byte costs the same as reading 64
    · two variables in the same line are, for coherency purposes,
      THE SAME VARIABLE
```

Address decomposition

An address is split into three fields. This is the whole mechanism.

```
   ┌──────────────┬──────────────┬──────────────┐
   │     TAG      │    INDEX     │    OFFSET    │
   └──────────────┴──────────────┴──────────────┘
      identity       which set     byte within
                     to look in     the line

  OFFSET = log2(line size)          64-byte line  → 6 bits
  INDEX  = log2(number of sets)
  TAG    = everything left over

Worked example: 32 KB, 8-way set-associative, 64-byte lines

  lines total = 32768 / 64        = 512
  sets        = 512 / 8           = 64 sets
  offset bits = log2(64)          = 6
  index bits  = log2(64)          = 6
  tag bits    = 48 - 6 - 6        = 36   (for a 48-bit address)

   47                    12 11    6 5    0
  ┌────────────────────────┬────────┬──────┐
  │          TAG           │ INDEX  │OFFSET│
  └────────────────────────┴────────┴──────┘
```

The lookup

```
  1. Extract INDEX from the address    → select one set
  2. Read all N ways of that set in parallel
  3. Compare each way's stored TAG against the address's TAG,
     and check its VALID bit
  4. Any match → HIT. Use OFFSET to pick bytes from that line.
     No match  → MISS. Choose a victim, evict it, fetch the line.

Steps 2 and 3 happen simultaneously across all ways — that is what
associativity costs in hardware: N comparators and N tag reads per access.
```

Associativity

Organisation	Where a line may go	Cost	Weakness
Direct-mapped (1-way)	Exactly one place	One comparator; fastest, smallest	Two hot addresses with the same index thrash forever, even with the cache 99% empty
N-way set-associative	Any of N ways in one set	N comparators, a replacement policy	Still possible to conflict with N+1 hot lines
Fully associative	Anywhere	One comparator per line — only feasible for tiny structures	Impractical above a few dozen entries; used for TLBs and victim caches

```
Direct-mapped thrashing, made concrete

  32 KB direct-mapped, 64-byte lines → 512 sets
  index = bits [14:6], so addresses 32 KB apart share an index.

  float a[8192], b[8192];        /* 32 KB each, adjacent in memory */
  for (i = 0; i < 8192; i++)
          a[i] += b[i];

  a[i] and b[i] are exactly 32768 bytes apart → SAME SET, every i.
  Each iteration evicts the other. 100% miss rate.
  Padding b by one line makes the miss rate ~6%.

This is a CONFLICT MISS: the cache is nearly empty and still missing.
```

THE POWER-OF-TWO STRIDE TRAP

This is not a historical curiosity — it bites constantly in numerical and image code. Any array whose row stride is a large power of two makes column traversal land on the same few sets.

```
float m[1024][1024];              /* row stride = 4096 bytes */
for (j = 0; j < 1024; j++)
        for (i = 0; i < 1024; i++)
                sum += m[i][j];   /* every access strides 4096 bytes */

/* With a 4096-byte set span, all 1024 accesses in the inner loop map
   to the same set. An 8-way cache holds 8 of them. Miss rate ≈ 100%.

   Fix: pad the row.  */
float m[1024][1024 + 16];         /* stride is now 4160 — indices spread */
```

The same effect explains why FFT and matrix libraries pad their leading dimensions, and why a benchmark can slow down 5× when you round an array size up to a power of two.

How much associativity is enough?

Empirically, an N-way cache of size S has about the same miss rate as a direct-mapped cache of size 2S, with diminishing returns past 8 ways. Typical modern designs: L1 4- or 8-way (latency-sensitive), L2 8- to 16-way, L3 16-way or more.

Replacement policy

Policy	Idea	Note
LRU	Evict the least recently used way	Good, but exact LRU needs log2(N!) bits per set — only practical for small N
Pseudo-LRU	A binary tree of "which half was used more recently"	N−1 bits per set; nearly as good. The common real choice.
Random	Pick a way at random	Cheap, and immune to pathological patterns. Used in some ARM L1s.
RRIP / DRRIP	Predict re-reference interval; insert streaming data as "evict soon"	Handles scans that would otherwise flush the whole cache

> **WHY LRU FAILS ON SCANS**
> Stream an array larger than the cache and LRU evicts everything useful in favour of data you will never touch again. This is why `madvise(MADV_DONTNEED)`, non-temporal stores (`STNP` on AArch64) and RRIP-style policies exist: they let a bulk copy pass through without destroying the working set of everything else.

Write policies

Two independent decisions, giving four combinations — and the choice is directly visible in your page tables.

Decision	Options
On a write hit	**Write-back**: update the line, mark it dirty, write to memory only on eviction. **Write-through**: update the line *and* memory every time.
On a write miss	**Write-allocate**: fetch the line first, then write into it. **No-write-allocate**: send the write straight to memory.

```
Write-back + write-allocate   ← the standard pairing, and the default
                                for Normal memory on ARM
  + far less memory traffic (repeated writes coalesce in the cache)
  + a write miss followed by more writes to the same line is cheap
  - memory is stale until eviction → this is exactly why DMA needs
    cache maintenance (chapter 15, and the SMMU chapter)
  - a dirty line must be written out on eviction, adding latency

Write-through + no-write-allocate
  + memory is always current → simple for DMA and for coherency
  + no dirty bits to track
  - every store goes to memory; needs a write buffer to be tolerable

The DIRTY bit is the whole difference. It is why a cache flush is
sometimes a no-op and sometimes writes megabytes to DRAM.
```

```
/* On ARM this is not a fixed property of the cache — it is chosen
   per page, via the AttrIndx field of the translation descriptor
   indexing into MAIR_EL1. See ARM chapters 11 and 14.            */

MAIR slot 0 = 0xFF   /* Normal, Inner+Outer Write-Back, read+write allocate */
MAIR slot 1 = 0x00   /* Device-nGnRnE — never cached at all                 */
MAIR slot 2 = 0x44   /* Normal, Non-cacheable                               */

/* So "is this memory write-back cached?" is a software decision made
   in a page table, not a hardware constant. Getting it wrong for a
   DMA buffer or an MMIO region is a classic driver bug.            */
```

Virtual versus physical indexing

The cache lookup needs an address — but which one, the virtual address the program used or the physical address after translation? Waiting for the MMU adds latency to every access.

Scheme	Index / Tag	Property
PIPT	Physical / Physical	Simplest to reason about; no aliasing. But the TLB lookup is on the critical path.
VIVT	Virtual / Virtual	Fastest, but must be flushed on every context switch and suffers aliasing. Largely abandoned.

Scheme	Index / Tag	Property
VIPT	Virtual / Physical	**The standard.** Index with the virtual address (available immediately) while the TLB produces the physical tag in parallel. Both finish together.

```
Why VIPT works, and its constraint

  Page offset bits are IDENTICAL in the virtual and physical address —
  translation only changes the page number.

   47                      12 11        0
  ┌──────────────────────────┬────────────┐
  │      page number         │  offset    │   ← offset is untranslated
  └──────────────────────────┴────────────┘
                             └──────┬─────┘
                                    │ if INDEX+OFFSET fit inside here,
                                    │ indexing with the VA is identical
                                    │ to indexing with the PA

  Constraint:  (cache size / associativity) ≤ page size

  4 KB pages, 64-byte lines → index+offset ≤ 12 bits
  32 KB cache: needs 8-way to keep each way at 4 KB.

This is precisely why L1 caches are "32 KB, 8-way" so consistently.
It is not a tuning choice — it is forced by the 4 KB page size.
Going to a 64 KB L1 requires 16 ways, or larger pages, or accepting
aliasing and handling it in software.
```

Inspecting a real machine

```
# Linux exposes the full cache topology
lscpu -C
#  NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL SETS
#  L1d       64K     512K    4 Data            1  256
#  L1i      128K       1M    4 Instruction     1  512
#  L2         4M      32M    8 Unified         2 8192
#  L3        16M      16M   16 Unified         3 16384

# or directly
cat
/sys/devices/system/cpu/cpu0/cache/index0/{level,type,size,ways_of_associativity,coherency_line_
size}

getconf -a | grep CACHE
```

```
/* the constant every performance-sensitive program needs */
#include <unistd.h>
long line = sysconf(_SC_LEVEL1_DCACHE_LINESIZE);   /* usually 64 */

/* C++17 gives it portably at compile time */
#include <new>
alignas(std::hardware_destructive_interference_size) struct counter { ... };

/* Linux kernel */
struct foo {
        int hot;
} ____cacheline_aligned;
```

How it shows up

Symptom	Cause
Rounding an array dimension to a power of two slows things 5×	Conflict misses from a power-of-two stride; pad the row
Two arrays are fine separately, terrible together	They map to the same sets; offset one allocation
Performance changes when an unrelated global is added	Alignment shifted, changing which sets your data lands in
A big `memcpy` slows down everything afterwards	The copy evicted the working set; use non-temporal stores
Aligning a struct to 64 bytes gives a free speedup	It no longer straddles two lines
A DMA buffer reads back stale data	Write-back cache holds the dirty line; memory has not been updated
Peripheral registers behave erratically	MMIO mapped as Normal cacheable instead of Device
L1 is always 32 KB and 8-way on every chip you check	The VIPT constraint against a 4 KB page

Check yourself

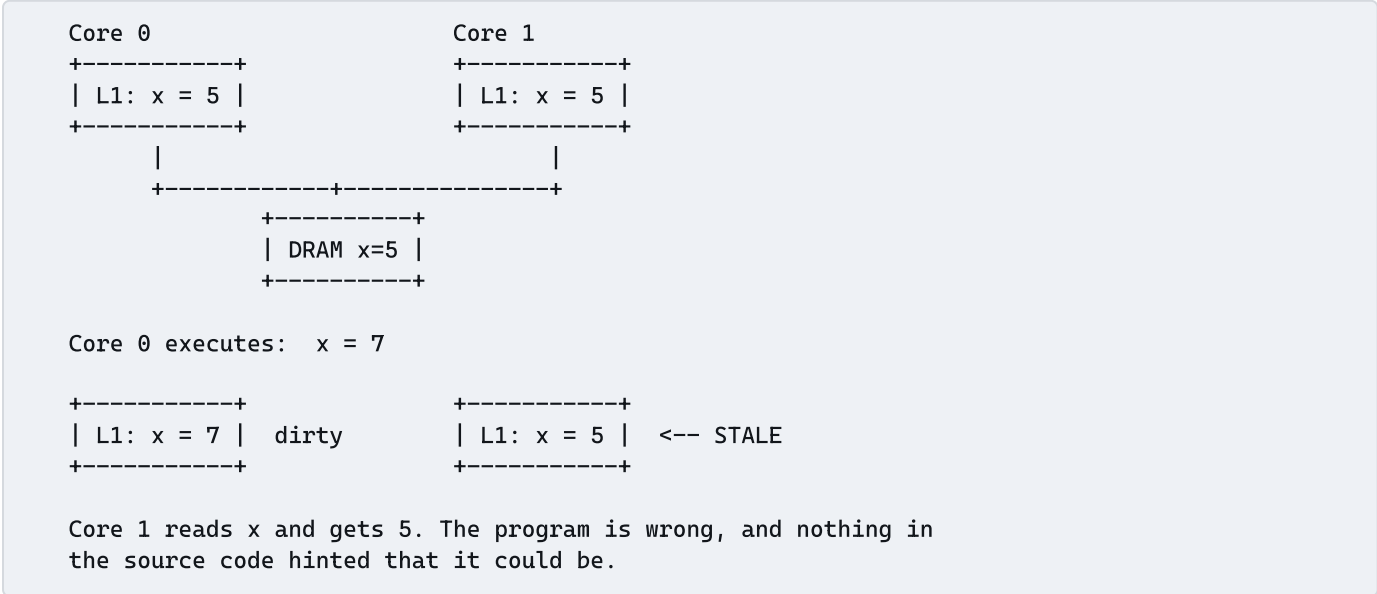
- ✓ Split a 48-bit address into tag/index/offset for a 64 KB, 4-way, 64-byte-line cache.
- ✓ Explain a conflict miss and construct two arrays that thrash a direct-mapped cache.
- ✓ Say why associativity helps and why 8-way is a common stopping point.
- ✓ Give the four write-policy combinations and which ARM uses for Normal memory.
- ✓ Explain the dirty bit and connect it to why DMA needs cache maintenance.
- ✓ Derive the VIPT constraint and use it to explain why L1 caches are 32 KB, 8-way.
- ✓ Explain why padding a matrix row can be a large optimisation.

Cache coherence and memory consistency

Two words that sound like synonyms and are not. Coherence is about one location seen by many caches; consistency is about the order of operations across different locations. Confusing them is the root of most multi-core bugs.

The problem

Give each core its own cache (chapter 13) and you have created a correctness problem: the same location now exists in several places at once, with different values.



Coherence: one location, many caches

THE COHERENCE GUARANTEE

A memory system is coherent if, for any *single* location:

1. A read returns the value of the most recent write to that location.
2. All observers agree on the *order* of writes to it (write serialisation).

Note what this does not say: nothing about the relative ordering of accesses to *different* locations. That is consistency, and it is a separate guarantee that hardware does *not* give you by default.

MESI

Each cache line carries a state. Cores watch — "snoop" — each other's transactions and change state accordingly.

State	Meaning	Others may hold it?	Memory current?
M — Modified	I have it and I changed it	No	No — I must write back
E — Exclusive	I have it, unmodified, and only I have it	No	Yes
S — Shared	I have it, unmodified, others may too	Yes	Yes
I — Invalid	No usable copy	—	—

```
The transitions that matter

  Core 0 reads x, nobody else has it   ->  I -> E
  Core 1 then reads x                  ->  Core 0: E -> S,  Core 1: I -> S
  Core 0 writes x                      ->  broadcast INVALIDATE
                                           Core 1: S -> I
                                           Core 0: S -> M
  Core 1 reads x again                 ->  Core 0 must SUPPLY the data
                                           (or write it back first)
                                           Core 0: M -> S,  Core 1: I -> S

The E state exists purely as an optimisation: if only you hold a clean
line, you may write it without broadcasting anything. Without E, every
first write to a private variable would need a bus transaction.

MOESI adds O (Owned): a dirty line may be shared, with one owner
responsible for eventual write-back. Saves memory traffic.
```

On ARM this rides the interconnect protocols from chapter 17 — ACE for smaller systems, CHI for large ones. Broadcast snooping does not scale past a modest core count, so large designs use a **directory**: a structure recording which caches hold which lines, so invalidations go point-to-point instead of to everyone.

False sharing

Coherence operates on *lines*, not variables. Two unrelated variables in the same 64-byte line are indistinguishable to the protocol.

```
/* looks perfectly parallel. It is not. */
struct counters {
        uint64_t a;    /* thread 0 increments this */
        uint64_t b;    /* thread 1 increments this */
};                     /* both in ONE cache line   */

/* Thread 0 writes a  -> invalidates the line in thread 1's cache
   Thread 1 writes b  -> invalidates the line in thread 0's cache
   ...forever. The line ping-pongs between cores every iteration.

   Measured effect: often 10-100x SLOWER than one thread doing both
   increments. You added a core and went backwards.               */

/* the fix: give each its own line */
struct counters {
        alignas(64) uint64_t a;
        alignas(64) uint64_t b;
};

/* portable spellings */
_Alignas(64)                                        /* C11           */
alignas(std::hardware_destructive_interference_size) /* C++17         */
____cacheline_aligned                               /* Linux kernel  */
__attribute__((aligned(64)))                        /* GCC / Clang   */
```

THE SIGNATURE OF FALSE SHARING

Unmistakable once you know it: **throughput gets worse as you add threads**, and the working set is tiny. Confirm with `perf c2c record` / `perf c2c report`, which names the exact cache lines suffering cross-core contention and the source lines touching them. Usual victims: per-thread statistics arrays, adjacent lock structures, and the head and tail pointers of a ring buffer.

Consistency: many locations, what order?

Coherence is not enough. Consider two *different* variables:

```
/* both start at 0 */

/* Core 0 */                    /* Core 1 */
data = 42;                      while (flag == 0)
flag = 1;                               ;
                                use(data);      /* may read 0! */
```

Every location is coherent. Yet core 1 can observe `flag == 1` while still reading `data == 0`. Coherence never promised anything about ordering *between* addresses.

Why the reordering happens

Source	What it does
Compiler	Reorders or eliminates accesses it can prove safe *single-threaded*
Store buffer	A write retires into a buffer; other cores cannot see it yet. The store to `data` can still be sitting there when `flag`'s store drains.
Out-of-order execution	Loads issue as soon as their address is known, possibly before earlier loads
Cache hierarchy	Invalidations reach different cores at different times
Interconnect	Messages take different routes and arrive out of order

Memory models, strong to weak

Model	Guarantee	Used by
Sequential consistency	One global order consistent with each program's order	No real hardware — too slow. It is the model beginners assume.
TSO (total store order)	Only store→load may be reordered; the store buffer is the sole relaxation	x86-64, SPARC
Weak / relaxed	Almost anything may be reordered unless you say otherwise	**ARM**, RISC-V, PowerPC

WHY THIS BITES WHEN PORTING FROM X86

Lock-free code that ran correctly on x86 for years can fail on ARM. x86's TSO happens to forbid the reorderings naive code depends on; ARM's weak model does not. *The code was always wrong* — x86 merely hid it. It will not reproduce reliably either: it depends on timing, core count and load, so it surfaces in production rather than in test. This is the most common serious porting bug from x86 to ARM.

Restoring order

```
Fixed with barriers

  /* Core 0 */                    /* Core 1 */
  data = 42;                      while (flag == 0)
  DMB ISHST;   <- prior stores            ;
  flag = 1;       visible before    DMB ISHLD;  <- no later load may
                  later stores                    float above this
                                   use(data);

Fixed with acquire / release - cheaper, and the modern idiom

  /* Core 0 */                    /* Core 1 */
  data = 42;                      while (LDAR(flag) == 0)   <- acquire
  STLR(flag, 1);   <- release             ;
                                  use(data);

  STLR: no EARLIER access may be reordered after it
  LDAR: no LATER   access may be reordered before it

Together they form a RELEASE-ACQUIRE PAIR: everything core 0 did
before the release is visible to core 1 after the acquire. This is
the fundamental synchronisation building block, and every mutex,
every channel and every lock-free queue is built out of it.
```

```
/* portable C11 / C++11 - compiles to LDAR/STLR on ARM, and to plain
   loads and stores on x86, where TSO already provides the ordering  */
#include <stdatomic.h>

_Atomic int flag = 0;
int data = 0;

/* producer */
data = 42;
atomic_store_explicit(&flag, 1, memory_order_release);

/* consumer */
while (atomic_load_explicit(&flag, memory_order_acquire) == 0)
        ;
use(data);                      /* guaranteed to see 42 */

/* Linux kernel spellings of the same idea */
smp_store_release(&flag, 1);
if (smp_load_acquire(&flag)) use(data);
smp_mb();  smp_rmb();  smp_wmb();      /* full / read / write barrier  */
READ_ONCE(x);  WRITE_ONCE(x, v);       /* stop the COMPILER only       */
```

`memory_order`	Guarantee	Cost on ARM
`relaxed`	Atomicity only, no ordering	Free — a plain atomic op
`acquire`	Later accesses cannot move before it	`LDAR`
`release`	Earlier accesses cannot move after it	`STLR`
`acq_rel`	Both, for read-modify-write	`LDAXR` / `STLXR` or `CAS`
`seq_cst`	Plus a single global total order	Most expensive — and the default if you omit the argument

Coherence with devices

Cores are kept coherent by hardware. Devices may not be — and whether they are is a property of the SoC, not something you may assume.

```
Non-coherent DMA: the two failure modes

  CPU writes a buffer, then starts a device READ
    · the data is dirty in L1; DRAM still holds the old bytes
    · the device reads DRAM -> gets garbage
    -> software must CLEAN (flush) the cache first

  Device WRITES a buffer, then the CPU reads it
    · the CPU may still hold a stale clean copy in L1
    · the CPU reads its cache -> gets old data
    -> software must INVALIDATE the cache first

  On ARM:  DC CVAC (clean), DC IVAC (invalidate), DC CIVAC (both)

Coherent DMA: the device participates in the coherency protocol
(ACE-Lite or CHI; marked `dma-coherent` in the device tree). No
maintenance needed - but never assume it, check.
```

```
/* Which is exactly why you NEVER hand a device a raw kmalloc pointer.
   The DMA API knows whether this device is coherent and emits the
   right maintenance - or correctly emits none.                      */

dma_addr_t d = dma_map_single(dev, buf, len, DMA_TO_DEVICE);
/* ... the device owns the buffer here; the CPU must not touch it ... */
dma_unmap_single(dev, d, len, DMA_TO_DEVICE);

/* peek mid-transfer, correctly */
dma_sync_single_for_cpu(dev, d, len, DMA_FROM_DEVICE);
inspect(buf);
dma_sync_single_for_device(dev, d, len, DMA_FROM_DEVICE);
```

Measuring coherence traffic

```
# find false sharing and cross-core line contention
perf c2c record ./program
perf c2c report --stdio
#   reports HITM (hit-modified) events: a load that hit a line
#   modified in ANOTHER core's cache - the fingerprint of sharing

# ARM PMU events worth watching
perf stat -e mem_access,l2d_cache_wb,bus_access_shared ./program

# quick triage: if throughput DROPS as you add threads and the working
# set is tiny, it is almost certainly false sharing
```

How it shows up

Symptom	Cause
Adding threads makes a parallel program slower	False sharing; pad to cache-line boundaries

Symptom	Cause
Lock-free code works on x86, fails on ARM	Missing barriers, previously hidden by TSO
A bug appears only under load on many cores	The reordering window widened; the ordering was never guaranteed
A flag is set but the data it guards is stale	Missing release/acquire pairing
DMA reads zeros or old data	Cache not cleaned before the device read
DMA results never become visible to the CPU	Cache not invalidated after the device write
Adding a `printf` makes the bug vanish	Timing changed; the race is still there
`volatile` did not fix a multi-threaded bug	`volatile` constrains the compiler, not the hardware. It is not a barrier and never was.

Check yourself

- ✓ State the difference between coherence and consistency in one sentence each.
- ✓ Walk two cores through the MESI states for read, read, write, read.
- ✓ Explain why the E state exists.
- ✓ Construct a false-sharing example, fix it, and say how you would detect it.
- ✓ Explain why the flag/data example can fail even with perfect coherence.
- ✓ Give the release-acquire pairing and state exactly what it guarantees.
- ✓ Say why lock-free code tested on x86 may fail on ARM.
- ✓ Name the two cache-maintenance operations non-coherent DMA needs, and when each applies.

Virtual memory as an architectural idea

Every address your program uses is a lie the hardware maintains. This chapter covers why that indirection exists and how it works in general; ARM chapter 12 and OS chapter 12 cover the specific mechanics.

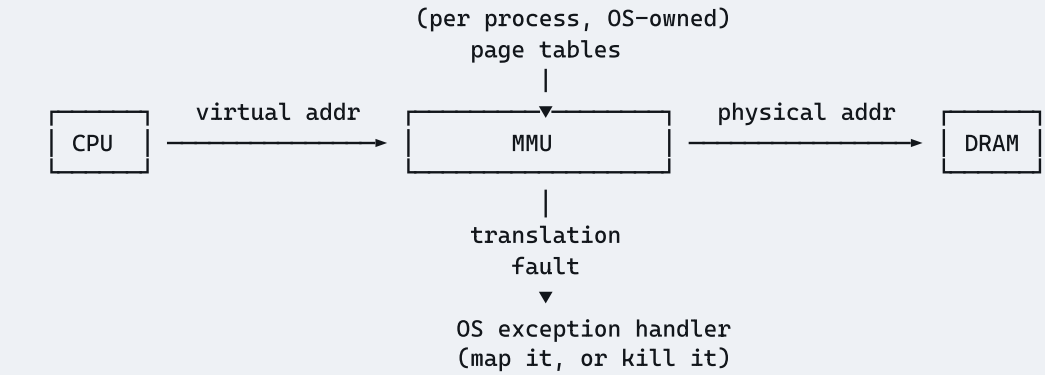
The problem

Before virtual memory, a program's addresses were physical addresses. Four consequences, all bad:

Problem	Consequence
No protection	Any program can read or write any other program's memory, and the OS's. One bug takes down everything.
No relocation	A program compiled to load at 0x10000 must load there. Two such programs cannot coexist.
Fragmentation	Free memory exists but not contiguously, so a large allocation fails while plenty is free.
Hard capacity limit	The working set must fit in physical RAM. No overcommit, no demand loading, no swap.

THE IDEA

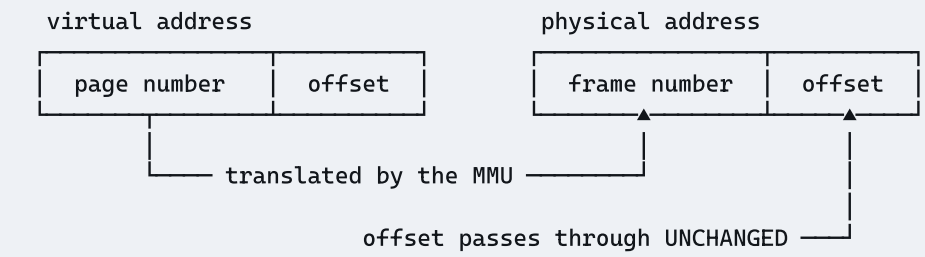
Insert a hardware translation step between the address a program emits and the address that reaches memory. Software sees **virtual addresses**; the memory system sees **physical addresses**; a per-process mapping table, controlled by the OS and consulted by hardware, connects them. Every problem above dissolves at once.



The MMU sits inside the core, in the memory stage of the pipeline (chapter 11). Every load, store and instruction fetch goes through it.

Pages: the unit of mapping

Mapping every byte individually would need a table larger than memory. So the address space is divided into fixed-size **pages**, and only the page number is translated.




```
Page size 4 KB  -> offset is 12 bits
Page size 16 KB -> offset is 14 bits
Page size 64 KB -> offset is 16 bits

Because the offset is untranslated, the low bits of a virtual and a
physical address are IDENTICAL. This single fact is what makes VIPT
caches possible (chapter 14) - the cache can start its lookup before
translation finishes.
```

Choosing a page size

	Small pages (4 KB)	Large pages (2 MB, 1 GB)
Internal fragmentation	Low — little waste per allocation	High — a 1-byte allocation can pin 2 MB
Page table size	Large — many entries	Small
TLB reach	Poor — 1024 entries cover 4 MB	Excellent — 1024 entries cover 2 GB
Fault granularity	Fine — demand paging works well	Coarse — one fault pulls in 2 MB
Typical use	General purpose	Databases, VMs, the kernel's own linear map, ML workloads

Most systems support several sizes simultaneously — Linux calls the large ones *huge pages*, ARM calls the intermediate levels *blocks*. The trade-off is always fragmentation against TLB reach.

Page tables

A flat array indexed by page number is the obvious design and is immediately impossible:

```
  48-bit address space, 4 KB pages
      2^48 / 2^12 = 2^36 pages = 68,719,476,736 entries
      x 8 bytes each = 512 GB of page table.  Per process.

The fix: exploit sparseness. Almost all of that space is unmapped, so
use a MULTI-LEVEL tree and simply omit the empty subtrees.

  VA[47:39] -> L0 entry -> points to an L1 table  (or is INVALID)
  VA[38:30] -> L1 entry -> points to an L2 table
  VA[29:21] -> L2 entry -> points to an L3 table
  VA[20:12] -> L3 entry -> the physical frame + permissions
  VA[11:0]  -> offset within the frame

A process using 8 MB needs only a handful of tables, not 512 GB.
An INVALID entry at any level prunes an entire subtree in one step.
```

Structure	How it works	Trade-off
Multi-level (radix) tree	Fixed fan-out per level, indexed by address bits	The universal choice: x86-64, ARM, RISC-V. Cost is one memory access per level on a miss.

Structure	How it works	Trade-off
Inverted page table	One entry per *physical* frame, hashed by VA	Size proportional to RAM, not address space. Awkward for shared memory. Used historically on PowerPC.
Hashed page table	Hash the VA into a bucket chain	Compact but variable-latency lookups

What an entry carries besides an address

A translation entry is not just a pointer. The permission and attribute bits are where most of the architecture's power lives:

Field	Purpose	What it enables
Valid	Is this mapping present?	Demand paging, swap, lazy allocation
Read / Write	Access permission	Read-only data, copy-on-write
Execute-never	May instructions be fetched here?	W^X — the single most effective anti-exploit measure
Privilege	User-accessible, or kernel only?	The user/kernel boundary itself
Accessed / Dirty	Has it been read? written?	Page replacement policy; knowing what must be written back
Memory attributes	Cacheable? Device? Shareable?	MMIO correctness, DMA buffers (ARM ch. 11)
Global / ASID	Shared across processes, or tagged	Avoiding a full TLB flush on context switch

> **THE INSIGHT WORTH CARRYING FORWARD**
>
> Protection and translation are the *same mechanism*. The hardware must consult the table on every access to find the physical address, so checking permissions at the same time is free. That is why virtual memory is the foundation of process isolation rather than a separate feature — and why a system without an MMU (a Cortex-M) needs an entirely different protection scheme, the MPU.

The TLB

A four-level walk costs four memory accesses. Doing that on every load would make every access five times slower. So translations are cached in the **Translation Lookaside Buffer**.

```
Typical structure

  L1 TLB    32–64 entries, fully associative, ~1 cycle
            often split: separate iTLB and dTLB
  L2 TLB    1024–2048 entries, set-associative, ~10 cycles
  miss      -> hardware page table walker -> up to 4 memory accesses
               (each of which may itself hit or miss in the caches)

TLB reach = entries x page size
  1024 entries x 4 KB  =   4 MB   <- easily exceeded
  1024 entries x 2 MB  =   2 GB   <- why huge pages help so much

A TLB miss on a workload with a large random working set can cost
more than the cache miss that follows it.
```

Context switches and tagging

Each process has its own mapping, so naively a context switch invalidates every TLB entry — and the new process then takes hundreds of misses warming it back up. Two mechanisms avoid this:

- **Address space identifiers.** Tag each entry with the owning process (ARM calls it *ASID*, x86 calls it *PCID*). Entries from several processes coexist; a switch just changes the current tag.
- **Global entries.** Kernel mappings are identical in every process, so they are marked global and match regardless of tag — no reload after a system call.

> **SOFTWARE MUST INVALIDATE; HARDWARE WILL NOT NOTICE**
> The TLB is a cache with *no coherence protocol*. If you modify a page table entry, the stale translation lives on until you explicitly invalidate it. On a multi-core system you must invalidate on *every* core that might hold it — a "TLB shootdown", which on ARM is a broadcast `TLBI` and on x86 requires an inter-processor interrupt. Forgetting this produces the classic "works on one core, fails on four" bug, and shootdown cost is a real scalability limit for `munmap`-heavy workloads.

Page faults: the useful exception

A translation failure traps to the OS, which decides what it means. Most faults are not errors — they are how features get implemented.

Fault	OS response	Feature it implements
Valid bit clear, page is on disk	Read it in, update the entry, retry	Demand paging and swap
Valid bit clear, never touched	Allocate a zero page now	Lazy allocation — why `malloc` of 1 GB is instant
Write to a read-only shared page	Copy it, make the copy writable, retry	Copy-on-write — why `fork()` is cheap
Access flag clear	Set it and retry	Tracking which pages are in use, for replacement
Write to a clean page	Set dirty, retry	Knowing which pages need writing back
Genuinely unmapped, or permission violation	SIGSEGV / bug check	Protection — catching the actual bug

```
The demand-paging round trip

  1. Instruction issues a load            VA -> MMU
  2. TLB miss -> page table walk
  3. Entry invalid -> PAGE FAULT exception, precise
  4. Hardware saves the faulting address and cause, enters the OS
  5. OS looks up its own record of what SHOULD be at that address
  6. Allocates a frame, reads from disk if needed, writes the entry
  7. Invalidates the stale TLB entry
  8. Returns from the exception
  9. The SAME instruction re-executes, and now succeeds

Step 9 is why the exception must be PRECISE (chapter 11). The core
must be able to restart the faulting instruction as though nothing
had happened - which constrains the whole out-of-order design.
```

What the indirection buys, beyond protection

Capability	Mechanism
Shared libraries	One physical copy of libc mapped into every process at whatever address suits it
Memory-mapped files	File pages mapped directly; reads become faults, writes become dirty pages

Capability	Mechanism
Guard pages	An unmapped page below each stack turns overflow into a clean fault instead of silent corruption
ASLR	Randomised mapping addresses make exploit gadget locations unpredictable
Contiguity from fragments	Physically scattered frames appear as one contiguous buffer
Overcommit	Promise more address space than you have RAM, back it only on use
Virtualisation	A second translation stage under the first (ARM ch. 16)
Device isolation	The same idea applied to DMA — the IOMMU (security ch. 02)

And what it costs

- **Latency** on every access, hidden by the TLB but never zero.
- **Memory** for the tables themselves, plus the walker's own cache pressure.
- **Determinism.** A page fault can take microseconds to milliseconds. This is why hard real-time systems lock pages (`mlockall`) or avoid an MMU entirely — see OS chapter 19.
- **Shootdown cost** on many-core systems.
- **A side channel.** Timing differences between mapped and unmapped pages leak layout information, which is what defeats naive ASLR.

Seeing it on a real system

```
# the virtual layout of a running process
cat /proc/self/maps
#   aaaab7c40000-aaaab7c48000 r-xp 00000000 fe:01 1234  /usr/bin/cat
#   aaaab7c57000-aaaab7c58000 r--p 00007000 fe:01 1234  /usr/bin/cat
#   ffff9c7f0000-ffff9c811000 r-xp 00000000 fe:01 5678  /lib/ld-linux.so
#   ffffe3d21000-ffffe3d42000 rw-p 00000000 00:00 0     [stack]
#              ^ permissions: r/w/x and p=private or s=shared

# per-page physical detail
cat /proc/self/smaps            # Rss, Pss, dirty pages, huge pages per mapping

# fault counts: minor = no disk, major = disk read required
/usr/bin/time -v ./program 2>&1 | grep -i fault
ps -o min_flt,maj_flt -p $$

# TLB pressure
perf stat -e dTLB-load-misses,iTLB-load-misses,dtlb_walk,itlb_walk ./program
```

How it shows up

Symptom	Cause
`malloc` of a huge block succeeds instantly, then crashes on write	Lazy allocation plus overcommit; memory was never really reserved
First touch of each page is slow, later touches fast	Demand paging — one minor fault per page
Huge pages give a large speedup on a big-working-set program	TLB reach, not cache behaviour
A latency spike of milliseconds in an otherwise fast loop	A major fault: the OS went to disk
Page table change works on one core, fails on four	Missing TLB shootdown

Symptom	Cause
A driver reads garbage from a peripheral	Mapped with the wrong memory attributes, not the wrong address
Stack overflow gives a clean crash rather than corruption	The guard page did its job
Real-time task misses a deadline occasionally	An unlocked page faulted; use `mlockall()`

Check yourself

- ✓ Give the four problems virtual memory solves, and say which one is about security.
- ✓ Explain why the page offset is not translated, and one thing that fact makes possible.
- ✓ Compute the size of a flat page table for a 48-bit space, and explain how a multi-level tree avoids it.
- ✓ List six fields in a page table entry besides the frame number, and what each enables.
- ✓ Define TLB reach and explain why huge pages help large working sets.
- ✓ Explain why the TLB needs explicit invalidation and what a shootdown is.
- ✓ Name four features implemented by *deliberately* taking a page fault.
- ✓ Explain why virtual memory complicates hard real-time systems.

I/O, buses and interconnects

A CPU that cannot talk to the world is a heater. This chapter covers how software reaches a device, the three ways data moves, and what actually travels on the wires between blocks — which turns out to matter enormously for security.

How software reaches a device

Two historical approaches. Only one survives in modern designs.

	Port-mapped I/O	Memory-mapped I/O
Address space	Separate, small (x86: 64 KB)	Shared with memory
Instructions	Dedicated: `IN`, `OUT`	Ordinary `LDR` / `STR`
Protection	A separate permission mechanism	The MMU, for free
Used by	Legacy x86 only	**ARM, RISC-V, and everything modern**

MEMORY-MAPPED I/O

Peripheral registers are assigned addresses in the same physical address space as DRAM. Writing to a UART is the same instruction as writing to RAM; only the address differs. Everything follows from this: device access is protected by page tables, translated by the MMU, and subject to the same memory-attribute rules as any other access.

```
A physical memory map is a static, chip-designer decision

  0x0000_0000  +--------------------------+
               |  Boot ROM                |
  0x1000_0000  +--------------------------+
               |  On-chip SRAM            |
  0x4000_0000  +--------------------------+
               |  Peripheral region       |
               |    0x4000_0000  UART0    |   each block usually gets
               |    0x4000_1000  UART1    |   at least one 4 KB page,
               |    0x4000_2000  I2C0     |   so it can be mapped and
               |    0x4800_0000  GIC      |   protected independently
  0x8000_0000  +--------------------------+
               |  DRAM                    |
               +--------------------------+

The 4 KB granularity is not an accident: it is the page size, so that
one driver can be given exactly one peripheral and nothing else.
```

Why MMIO must not be cached

```
/* Device registers are not memory. They have SIDE EFFECTS. */

status = regs->FIFO;    /* reading POPS a byte from the FIFO      */
regs->CTRL = START;     /* writing STARTS a physical operation    */
n = regs->COUNT;        /* the value changes without any write    */

/* If this region were cached and speculatable:
     - a speculative read could drain a FIFO nobody asked to read
     - a cached read would return a stale count forever
     - a write could sit in a dirty line and never reach the device
     - two writes could be merged, or reordered

   Hence a distinct memory TYPE (ARM: Device-nGnRnE), not merely
   "uncached". See ARM chapter 11.                                */

/* And hence: never dereference an MMIO pointer directly. */
u32 v = readl(base + STATUS);       /* Linux                       */
writel(v | ENABLE, base + CTRL);    /* includes the right barriers */
```

The three ways data moves

	Polling	Interrupts	DMA
Mechanism	CPU spins reading a status bit	Device signals; CPU takes an exception	Device moves data to/from memory itself
CPU cost	100% while waiting	Only on events, plus entry/exit overhead	Near zero; one interrupt on completion
Latency	Lowest and most deterministic	Interrupt latency, typically µs	Setup cost amortised over a large transfer
Best for	Very short waits; early boot before interrupts exist; hard real-time	Most peripherals	Bulk: network, storage, display, cameras
Hazard	Wastes power; can hang forever — always bound the loop	Storms if you fail to clear at the source	Coherency and address translation

```
/* polling, done correctly - ALWAYS with a timeout */
ret = readl_poll_timeout(base + STATUS, val,
                         val & STATUS_READY,   /* condition            */
                         10,                   /* µs between polls     */
                         1000);                /* µs before giving up  */
if (ret)
        return -ETIMEDOUT;      /* the device is wedged; say so
                                   rather than hanging the kernel */
```

Why DMA changes everything

```
Without DMA                      With DMA

  device -> CPU register           device -> memory, directly
         -> memory                 CPU is not involved at all

  every byte costs instructions    one setup + one interrupt

But now a SECOND entity issues addresses onto the interconnect, and
it never passed through the CPU's MMU. Two consequences:

  1. COHERENCY. The device sees DRAM; the CPU sees its cache.
     (chapter 15)

  2. PROTECTION. Every isolation guarantee built with page tables
     has a hole in it, because the device's transactions were never
     checked. This is why the IOMMU/SMMU exists.
     (security track, chapters 02-07)
```

Bus topologies

```
Shared bus            one set of wires, arbitration decides who drives
                      + simple, cheap, easy to snoop
                      - one transaction at a time; does not scale
                      e.g. classic AHB, I2C, older PCI

Crossbar              every master connected to every slave
                      + full parallelism between disjoint pairs
                      - area grows as N x M
                      e.g. small AXI interconnects

Network on chip       routers and links, packets find their way
  (NoC / mesh)        + scales to dozens of masters
                      - variable latency, needs flow control
                      e.g. ARM CMN, CHI-based fabrics

Point to point        dedicated serial links, switched
  (serial)            + very high bandwidth per pin, hot-pluggable
                      - protocol complexity, serialisation latency
                      e.g. PCIe, USB, Ethernet
```

Parallel gave way to serial

Counter-intuitively, one differential pair beats 64 parallel wires. Parallel buses are limited by *skew* — bits on different wires arrive at slightly different times, and the clock period must exceed the worst skew. Serial links embed the clock in the data, eliminating skew entirely, and then use equalisation and error correction to push the rate far higher. This is why PCIe replaced PCI, SATA replaced PATA, and USB replaced the parallel port.

AMBA: the ARM family

Protocol	Character	Typical use
APB	Simple, no bursts, one transfer at a time, low power	Slow peripheral registers: UART, I2C, GPIO, timers
AHB	Single shared bus, pipelined, supports bursts	Simpler SoCs, Cortex-M systems
AXI	Five independent channels, out-of-order via IDs, high throughput	The workhorse: memory controllers, DMA, GPU
ACE	AXI plus coherency snooping	Coherent CPU clusters

Protocol	Character	Typical use
ACE-Lite	Can snoop others but is not snooped	I/O-coherent DMA masters
CHI	Packet-based, scalable mesh, credit flow control	Large multi-core and server-class SoCs

```
AXI's five channels are independent, which is the whole point

  master                                        slave
    |--- AW: write address ------------------------->|
    |--- W:  write data ---------------------------->|
    |<-- B:  write response ------------------------|
    |--- AR: read address ------------------------->|
    |<-- R:  read data ----------------------------|

Because address and data are separate channels, a master can issue
many addresses before any data returns - OUTSTANDING TRANSACTIONS.
Because responses carry an ID, they may return OUT OF ORDER.

This is how a memory system hides 200-cycle DRAM latency: keep
dozens of requests in flight rather than waiting for each.
```

What travels besides address and data

THIS LIST IS THE FOUNDATION OF THE ENTIRE SECURITY TRACK

An AXI transaction carries far more than an address:

- `AxADDR` — the physical address.
- `AxID` — the master's transaction ID, from which the SoC derives a **master ID** or **StreamID**: *who is asking*.
- `AxPROT[1]` — the **NS bit**: Secure or Non-secure. One wire, and it is the entirety of TrustZone's bus-level enforcement.
- `AxPROT[0]` — privileged or unprivileged. `AxPROT[2]` — instruction or data.
- `AxCACHE` — cacheability and bufferability, derived from the page table's memory attributes.
- `AxLEN`, `AxSIZE`, `AxBURST` — the shape of the burst.

So the `NS` and `AttrIndx` fields you set in a page table descriptor are not abstractions — they become *physical signals*, and hardware downstream (the SMMU, the TZASC, the XPU) makes allow/deny decisions from them. Setting a bit in a descriptor is setting a wire on the interconnect.

External buses worth recognising

Bus	Wires	Speed	Character
I2C	2 (SDA, SCL)	100 kHz – 3.4 MHz	Addressed, multi-drop, open-drain. Slow and cheap: sensors, EEPROMs, PMICs.
SPI	4 (MOSI, MISO, SCK, CS)	1–100 MHz	Full duplex, one chip-select per device. Flash, displays, radios.
UART	2 (TX, RX)	9.6 k – 4 M baud	Asynchronous, no clock line, point-to-point. The debug console.
USB	Differential pair(s)	12 Mb/s – 40 Gb/s	Host-controlled, enumerable, hot-pluggable, powered.
PCIe	1–16 lanes	2–64 GB/s	Switched point-to-point, enumerable, DMA-capable, MSI interrupts.
CAN	2 (differential)	1–8 Mb/s	Multi-master, priority arbitration, robust. Automotive and industrial.

Bus	Wires	Speed	Character
Ethernet	Pairs	10 Mb/s – 800 Gb/s	Packet switched, and increasingly used inside vehicles too.

Discoverable versus described

```
DISCOVERABLE buses can be probed. Software asks, hardware answers.

  PCIe: read the configuration space at each bus/device/function
        -> vendor ID, device ID, class, BAR sizes
  USB:  the host requests descriptors from every device it detects

  -> the OS matches a driver against the reported IDs, automatically.

NON-DISCOVERABLE buses cannot. There is no way to ask address
0x40001000 whether a UART lives there - and a wrong guess may hang
the bus or trigger a side effect.

  On-SoC peripherals, I2C and SPI slaves are all in this category.

  -> the hardware must be DESCRIBED to software out of band:
       device tree (embedded Linux, ARM)
       ACPI tables  (servers, Windows on ARM, x86)
       board files  (legacy, deprecated)

This distinction is exactly why the device tree exists, and why an
embedded Linux driver looks so different from a PCIe driver.
```

Getting an interrupt from a device to a core

```
Wired interrupt (traditional)

  device asserts a physical line -> interrupt controller -> core
  · limited by pin count, so lines get shared
  · shared lines mean every handler must check "is it mine?"
    and return "not mine" if not

Message-signalled interrupt (MSI / MSI-X, and ARM's LPIs)

  device performs a WRITE to a special address, carrying a payload
  · no dedicated wires at all - it is just a bus transaction
  · thousands of distinct interrupts per device
  · no sharing, so no "is it mine?" ambiguity
  · arrives IN ORDER with the DMA data it refers to, which removes
    a classic race: with a wired interrupt the signal can arrive
    before the data it announces has landed in memory
```

The interrupt controller itself — priority, masking, routing to a specific core, security grouping — is covered in ARM chapters 23 and 24.

Inspecting a real system

```
# who owns which physical address range
cat /proc/iomem
#   40000000-40000fff : serial@40000000
#   80000000-bfffffff : System RAM

# interrupt counts per CPU — is the handler actually firing?
cat /proc/interrupts

# enumerate discoverable buses
lspci -vv          # BARs, capabilities, link speed, MSI config
lsusb -t           # topology tree

# non-discoverable hardware, as described to the kernel
ls /proc/device-tree/
dtc -I fs -O dts /proc/device-tree | less

# DMA and IOMMU groupings
ls /sys/kernel/iommu_groups/*/devices/
```

How it shows up

Symptom	Cause
Register reads return all zeros or all ones	Device unclocked or unpowered, wrong base address, or a bus firewall denying access
Writes appear to have no effect	Region mapped cacheable instead of Device; or a write-1-to-clear register
Device works in the debugger, fails at speed	Missing barriers between register writes, or a required delay
Reading a status register consumes data	Side-effecting register plus speculative or repeated reads
Interrupt fires continuously, system unresponsive	Not cleared at the device before acknowledging the controller
Interrupt never fires	Wrong number, wrong trigger type, masked, or not enabled at the device
DMA lands in the wrong place	Device programmed with a virtual or physical address instead of the DMA address
Interrupt arrives before the DMA data is visible	Wired-interrupt race; MSI avoids it by construction
PCIe device enumerates but its driver never binds	Vendor/device ID not in the driver's match table
On-SoC peripheral is invisible to Linux	Missing or `status = "disabled"` device tree node

Check yourself

- ✓ Explain memory-mapped I/O and one thing it gets for free from the MMU.
- ✓ Give three reasons MMIO must not be cached or speculated.
- ✓ Compare polling, interrupts and DMA on CPU cost, latency and failure mode.
- ✓ Explain the two problems DMA creates and name the hardware that solves each.
- ✓ Say why AXI separates address and data channels, and what that enables.
- ✓ List what an AXI transaction carries besides address and data, and why security depends on it.
- ✓ Explain discoverable versus described buses and why the device tree exists.
- ✓ Give two advantages of MSI over a wired shared interrupt.

Measuring performance: CPI, Amdahl and roofline

Optimisation without measurement is superstition. This chapter gives you the three models that let you predict what an optimisation is worth *before* you spend a week on it, and the counters that tell you where the time actually went.

The performance equation

Everything starts here. Runtime decomposes into exactly three factors, and every optimisation you will ever make attacks one of them.

```
                instructions        cycles           seconds
   time  =    ─────────────────  ×  ─────────────  ×  ─────────
                  program           instruction        cycle

                     ↑                  ↑                ↑
                  algorithm,           CPI             1 / clock
                  compiler,         (the micro-         frequency
                  ISA                architecture)

Who controls each factor:

  instruction count   you (algorithm, data structures), the compiler,
                      the ISA (CISC does more per instruction)
  CPI                 the microarchitecture, but YOU influence it
                      heavily through cache and branch behaviour
  clock period        the process node, the pipeline depth, power
                      and thermal limits (chapter 20)
```

> **THE TRAP THIS EQUATION EXPOSES**
>
> These factors are not independent, so a "win" on one can be a loss overall. Deepening the pipeline raises the clock but worsens CPI on every mispredict. A CISC instruction lowers the instruction count but may take more cycles. Vectorising cuts instruction count but may increase memory traffic. **Only end-to-end wall-clock time is real.** This is why "we reduced instruction count by 30%" is not a result — it is a hypothesis.

Why MIPS and clock speed are useless as comparisons

```
MIPS = instruction count / (runtime × 10^6)

/* Fails because: */
- a compiler that emits MORE, simpler instructions scores HIGHER
  while running slower
- different ISAs do different amounts of work per instruction
- one core at 3 GHz with CPI 4 loses to one at 2 GHz with CPI 1

/* The same applies to FLOPS in isolation, and to clock speed.
   The only honest comparison is: the same real workload,
   measured end to end, on both machines. */
```

CPI, and where the cycles go

The useful form splits CPI into a base plus the stalls you can attack:

```
   CPI = CPI_ideal + Σ (stall events per instruction × penalty)

Worked example — a memory-bound loop

   CPI_ideal                              = 0.5   (2-wide superscalar)
   L1 D-miss:   4% × 12 cycles            = 0.48
   L2 miss:     1% × 40 cycles            = 0.40
   LLC miss:    0.5% × 200 cycles         = 1.00
   branch miss: 2% × 15 cycles            = 0.30
   ──────────────────────────────────────────────
   CPI                                    = 2.68

Read the table, not the total. The LLC misses alone cost more than
everything else combined, and they are 0.5% of accesses.

Halving the LLC miss rate:      CPI 2.68 -> 2.18   (19% faster)
Perfect branch prediction:      CPI 2.68 -> 2.38   (11% faster)
Doubling the issue width:       CPI 2.68 -> 2.43   ( 9% faster)

The expensive-and-rare event wins. This is why profiling by
COUNT is misleading and profiling by COST is correct.
```

The reciprocal, **IPC** (instructions per cycle), is what most tools report. IPC above ~2 on a wide core suggests you are compute-bound; IPC below ~0.5 almost always means you are waiting on memory.

Amdahl's law: the ceiling on any optimisation

```
   If a fraction p of runtime is improved by a factor s:

                        1
   speedup  =  ──────────────────
                (1 − p) + p / s

The part you did NOT improve sets a hard ceiling:

   max speedup = 1 / (1 − p)     even with s = ∞

Examples, and the intuition they should install:

   p = 0.95, s = ∞   ->  20x     ceiling
   p = 0.90, s = ∞   ->  10x
   p = 0.75, s = ∞   ->   4x
   p = 0.50, s = ∞   ->   2x     <- half the code, infinite speedup,
                                    and you barely double performance

   p = 0.90, s = 2   ->  1.82x
   p = 0.90, s = 10  ->  5.26x
   p = 0.90, s = 100 ->  9.17x   <- 10x more effort for 1.7x more gain
```

> **WHAT AMDAHL ACTUALLY TELLS YOU TO DO**
>
> **Profile first, always.** A 10× improvement to code that is 5% of runtime yields 4.7% overall. A 30% improvement to code that is 80% of runtime yields 26%. Engineers routinely spend weeks on the former because it was the interesting problem. Measure, find the largest *p*, and attack that — then re-measure, because the profile changes as soon as you succeed.

Gustafson's counterpoint

Amdahl assumes a fixed problem size. In practice, more capable machines get used on bigger problems, and the parallel fraction often grows with problem size while the serial part stays constant. A weather model on a 1000-core cluster does not run the same simulation faster; it runs a finer grid. Both laws are correct — they answer different questions. Ask which one your situation is: *fixed work, want it sooner* (Amdahl) or *fixed time, want more work* (Gustafson).

The roofline model

The single most useful mental model for deciding whether to optimise compute or memory.

```
   Arithmetic intensity  I = FLOPs performed / bytes moved from DRAM

   Attainable GFLOP/s = min( peak_compute,  I × peak_bandwidth )

  GFLOP/s
    ▲
    │                     ┌──────────────── peak compute (the flat roof)
    │                    /
    │                   /
    │                  /  ← the slanted roof: I × peak bandwidth
    │                 /
    │                /
    │               /
    └──────────────┴──────────────────────────────────────► arithmetic intensity
                   ▲
            ridge point:  I = peak_compute / peak_bandwidth

   LEFT of the ridge  -> MEMORY BOUND.  Faster arithmetic buys nothing.
                         Improve locality, blocking, data layout,
                         compression, or fuse operations.

   RIGHT of the ridge -> COMPUTE BOUND. Bandwidth is not the issue.
                         Vectorise, use more cores, better algorithms.
```

```
/* Compute the intensity of your kernel and place it on the roofline. */

/* vector add: c[i] = a[i] + b[i]   over floats */
   FLOPs  = 1 per element
   bytes  = 12 per element  (read a, read b, write c)
   I      = 0.083 FLOP/byte        -> deeply MEMORY BOUND

/* dot product: s += a[i] * b[i] */
   FLOPs  = 2 per element
   bytes  = 8 per element
   I      = 0.25                   -> still memory bound

/* dense matrix multiply, N x N, properly blocked */
   FLOPs  = 2 N^3
   bytes  ≈ 3 N^2  (each matrix read/written once, if it fits in cache)
   I      ≈ 2N / 3                 -> grows with N: COMPUTE bound

/* This is exactly why blocked matrix multiply is the canonical
   optimisation, and why machine-learning kernels are written as
   matrix multiplies whenever possible - it moves them to the right
   of the ridge. See AI chapter 04.                                  */
```

A TYPICAL RIDGE POINT TODAY

A modern core might reach 100 GFLOP/s with 25 GB/s of bandwidth per core — a ridge at I = 4 FLOP/byte. Most real code sits at I between 0.1 and 1. **The majority of programs are memory bound**, which is why chapters 13 to 15 matter more for performance than instruction-level tricks, and why hardware keeps adding cache rather than ALUs.

Hardware performance counters

Every modern core has a PMU that counts microarchitectural events. This is ground truth, not inference.

```
# the standard first look
perf stat ./program

#   12,405,281,113  cycles                    #  2.98 GHz
#   18,204,113,782  instructions              #  1.47  insn per cycle
#    3,201,554,231  branches
#       62,119,882  branch-misses             #  1.94% of all branches
#    1,905,332,110  cache-references
#      210,553,901  cache-misses              # 11.05% of cache refs

# read it in this order:
#   IPC < 0.5      -> stalled; find out on what
#   IPC 0.5 - 1.5  -> mixed
#   IPC > 2        -> compute bound; algorithmic work is next
#   branch-misses > 2%      -> chapter 12
#   cache-miss rate high    -> chapters 13-14

# then localise it
perf record -g ./program && perf report      # which functions
perf record -e cache-misses ./program        # which lines miss
perf annotate                                # which instruction

# ARM-specific detail
perf stat -e l1d_cache_refill,l2d_cache_refill,dtlb_walk,\
stall_frontend,stall_backend ./program

# stall_frontend high  -> instruction supply: I-cache, branch mispredicts
# stall_backend  high  -> data supply or execution: D-cache, dependencies
```

Top-down analysis

Rather than guessing, classify every issue slot into one of four buckets. Intel formalised this as Top-Down; ARM's equivalent uses the stall counters above.

```
   every cycle's issue slots
            |
      ┌─────┴──────┐
   retiring     not retiring
   (useful)          |
            ┌────────┴─────────┐
      bad speculation      stalled
      (mispredicts,          |
       machine clears)   ┌───┴────┐
                      frontend    backend
                      bound       bound
                      (I-cache,   (D-cache, DRAM,
                       decode,     execution ports,
                       branch)     dependencies)

Whichever bucket dominates tells you which chapter to reread:
  frontend bound  -> code layout, I-cache, chapter 12
  backend bound   -> data layout, chapters 13-15
  bad speculation -> chapter 12
  retiring high   -> you are done tuning; change the algorithm
```

Benchmarking honestly

Mistake	Why it lies	Fix
Measuring once	Noise from scheduling, frequency, other load	Many runs; report median and spread, not the best
Reporting the mean	One outlier dominates it	Median plus p95/p99, which is what users feel
Cold vs warm confusion	First run pays page faults and cache misses	Decide which you are measuring, and say so
Dead code elimination	The compiler deletes work whose result is unused	Consume the result; use a benchmark framework's `DoNotOptimize`
Unrealistic input	Tiny inputs fit in cache; sorted inputs predict perfectly	Use production-shaped data
Frequency scaling	Turbo and thermal throttling move the clock mid-run	Pin the governor, or measure cycles rather than seconds
Ignoring variance across cores	big.LITTLE cores differ several-fold	Pin with `taskset`
Optimising the benchmark	You improved the measurement, not the product	Confirm the win on the real workload

```
# make the machine quiet before you trust a number
sudo cpupower frequency-set -g performance     # fix the governor
taskset -c 2 ./program                          # pin to one core
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'  # cold-start measurement

# measure cycles, not seconds, when the clock may move
perf stat -e cycles,instructions -r 20 ./program   # 20 runs, with stddev
```

Latency versus throughput

These are different goals and optimising one routinely harms the other. Be explicit about which you are being paid for.

	Latency	Throughput
Question	How long does one operation take?	How many per second?
Improved by	Shorter critical path, fewer dependent steps, avoiding queues	Batching, pipelining, parallelism, larger buffers
Harmed by	Batching (you wait for the batch to fill)	Small transfers, per-item overhead
Report as	Median, p95, p99, max	Operations or bytes per second
Matters most for	Interrupt handlers, control loops, interactive UI, RTOS	Bulk data, network, storage, batch compute

Little's Law connects them: $L = \lambda W$ — items in flight equal arrival rate times time in system. Raising throughput by adding queueing necessarily raises latency. In hard real-time work (OS chapter 19) only the *worst case* counts, and average throughput is irrelevant.

A workflow that works

1. **Establish a reliable measurement** of the thing you actually care about. If you cannot measure it repeatably, stop here — everything downstream is guesswork.
2. **Profile** to find where time goes. Use cost, not call count.
3. **Apply Amdahl** to the profile. Compute the best possible payoff for each candidate before you start.
4. **Classify the bottleneck** with counters or roofline: memory, compute, branch, frontend, I/O.
5. **Fix the largest one**, and only that one.
6. **Re-measure.** Confirm the predicted gain appeared. If it did not, your model was wrong — that is valuable information.
7. **Re-profile.** The bottleneck has moved. Repeat until the remaining gain is not worth the complexity.
8. **Keep the benchmark in CI** so the win does not silently regress.

How it shows up

Symptom	Cause
A big optimisation produced a tiny overall gain	Amdahl: you improved a small fraction of runtime
Vectorising gave no speedup at all	Memory bound — left of the roofline ridge
Adding cores stopped helping past four	Serial fraction, shared bandwidth, or lock contention
The benchmark improved, production did not	Unrepresentative input or a cache-resident working set
Results vary 20% between runs	Frequency scaling, core migration, other load
IPC is high but the program is slow	Executing a lot of unnecessary instructions — an algorithmic problem
IPC is very low with few cache misses	Long dependency chains, or frontend bound
Average latency is fine, users complain	You reported the mean; they experience p99

Check yourself

- ✓ Write the performance equation and say who controls each factor.

- ✓ Explain why MIPS and clock speed are not valid comparisons.
- ✓ Given a CPI breakdown, identify which stall to attack and predict the gain.
- ✓ Compute the maximum speedup when 80% of runtime is improvable.
- ✓ Define arithmetic intensity and compute it for a vector add and a matrix multiply.
- ✓ Explain what being left of the roofline ridge means for your next move.
- ✓ Name four benchmarking mistakes and how to avoid each.
- ✓ Explain why improving throughput can worsen latency.

Parallelism: ILP, SIMD, SMT and multicore

Clock speeds stopped rising around 2005. Everything since has come from doing more things at once. This chapter covers the four kinds of parallelism, which ones are free, which ones you must ask for, and why "add more cores" stops working.

Why parallelism became the only option

```
Dynamic power in CMOS:      P ≈ C × V² × f

  Raising frequency f raises power linearly — and requires raising
  voltage V to keep switching fast enough, which raises power by the
  SQUARE. So a 20% clock increase can cost ~70% more power.

  ~2005: the "power wall". Air cooling caps a chip around 100–150 W.
  Frequencies flattened at 3–4 GHz and have stayed there for 20 years.

  Meanwhile transistor counts kept doubling. The only way to spend
  them was on MORE UNITS rather than FASTER ones:

     more execution units per core        -> ILP
     wider registers                      -> SIMD
     more thread contexts per core        -> SMT
     more cores                           -> multicore
     specialised units                    -> accelerators, NPUs

This is the entire shape of modern computing, and it is a
consequence of one equation.
```

Flynn's taxonomy

	Single data	Multiple data
Single instruction	**SISD** — a classic scalar core	**SIMD** — one instruction, many elements: NEON, SVE, AVX, GPUs
Multiple instruction	**MISD** — essentially unused, except for redundant lockstep cores in safety systems	**MIMD** — multicore, multiprocessor, clusters

Instruction-level parallelism

Independent instructions in a single stream, executed simultaneously. **You do not ask for this — the hardware finds it.**

Technique	What it does	Covered in
Pipelining	Overlaps stages of consecutive instructions	ch. 11
Superscalar issue	Issues 2–8 instructions per cycle to different units	here
Out-of-order execution	Runs instructions as operands become ready, not in program order	ch. 11
Register renaming	Removes false (write-after-read/write) dependencies	ch. 11
Speculation	Executes past unresolved branches	ch. 12

```
What limits ILP, and what you can do about it

  TRUE DEPENDENCIES. A dependent chain cannot be parallelised at all.

     s = 0;
     for (i = 0; i < n; i++)
             s += a[i];        <- every add waits for the previous one
                                  4-cycle FP add latency = 4 cycles/element
                                  no matter how wide the core is

  Break the chain with accumulators - the single highest-value ILP
  optimisation available to a programmer:

     s0 = s1 = s2 = s3 = 0;
     for (i = 0; i < n; i += 4) {
             s0 += a[i];   s1 += a[i+1];      <- four INDEPENDENT chains
             s2 += a[i+2]; s3 += a[i+3];         now the 4-cycle latencies
     }                                           overlap
     s = (s0 + s1) + (s2 + s3);

  Typical result: ~4x faster, from an unchanged instruction count.

  Note the reassociation: this changes floating-point results
  (chapter 03), which is exactly why the compiler will NOT do it for
  you unless you pass -ffast-math. On integers it does it freely.
```

Beyond ~4-wide issue, ILP hits diminishing returns: real code simply does not contain enough independent work, and the hardware to find more grows quadratically. That is why cores stopped getting wider and started getting more numerous.

Data-level parallelism: SIMD

One instruction, several data elements. Cheap in hardware — one instruction fetch and decode drives many ALUs — but you or your compiler must express it.

```
   scalar                          SIMD (128-bit, 4 x float32)

   c[0] = a[0] + b[0];             ┌─────┬─────┬─────┬─────┐  a
   c[1] = a[1] + b[1];             ├─────┼─────┼─────┼─────┤
   c[2] = a[2] + b[2];             │  +  │  +  │  +  │  +  │  b
   c[3] = a[3] + b[3];             ├─────┼─────┼─────┼─────┤
                                   └─────┴─────┴─────┴─────┘  c
   4 instructions                  1 instruction

  ARM NEON      128-bit fixed        V0-V31
  ARM SVE/SVE2  128-2048, scalable   Z0-Z31 + predicates P0-P15
  x86 AVX2      256-bit
  x86 AVX-512   512-bit

SVE's key idea is VECTOR LENGTH AGNOSTIC code: the same binary runs
optimally on 128-bit and 512-bit implementations, because the loop is
written against a runtime length with predication for the tail.
No recompilation, no dispatch table.
```

```
/* Four ways to get SIMD, in order of preference */

/* 1. Let the compiler do it. Write clean, obviously independent loops. */
void add(float *restrict c, const float *a, const float *b, int n) {
        for (int i = 0; i < n; i++)
                c[i] = a[i] + b[i];
}
/* -O3 -march=armv8-a+simd emits NEON automatically.
   `restrict` is what makes it legal: it promises the arrays do not
   overlap, so the compiler may reorder the accesses. Without it the
   compiler must assume c may alias a, and vectorisation is unsafe. */

/* Check whether it actually happened - do not assume: */
//   gcc -O3 -fopt-info-vec-missed   (tells you WHY it did not)

/* 2. OpenMP, when the compiler needs permission */
#pragma omp simd
for (int i = 0; i < n; i++) c[i] = a[i] + b[i];

/* 3. Intrinsics: explicit, portable across compilers, readable */
#include <arm_neon.h>
for (int i = 0; i < n; i += 4) {
        float32x4_t va = vld1q_f32(a + i);
        float32x4_t vb = vld1q_f32(b + i);
        vst1q_f32(c + i, vaddq_f32(va, vb));
}

/* 4. Hand-written assembly. Rarely justified; hard to maintain. */
```

WHY YOUR LOOP DID NOT VECTORISE

The compiler must *prove* the transformation is safe. Common blockers, roughly in order of frequency: possible pointer aliasing (fix with `restrict`); a loop-carried dependency; a data-dependent early exit or `break`; function calls that cannot be inlined; a non-contiguous or indirect access pattern (`a[idx[i]]`); floating-point reassociation that would change results; and a trip count unknown at compile time combined with expensive tail handling. Ask the compiler with `-fopt-info-vec-missed` rather than guessing — it will name the reason.

SIMD DOES NOT RESCUE A MEMORY-BOUND LOOP

Vector add has an arithmetic intensity of about 0.08 FLOP/byte — far left of the roofline ridge (chapter 18). Making the arithmetic 4× faster changes nothing when you are waiting on DRAM. Vectorise things that are compute bound, or fix the memory behaviour first. This is the most common reason a carefully vectorised loop shows no improvement.

Thread-level parallelism on one core: SMT

Simultaneous multithreading keeps two or more architectural contexts in one physical core, so when one thread stalls the other uses the idle execution units.

Duplicated per thread	Shared between threads
Architectural registers, program counter, exception state	Execution units, caches, TLBs, branch predictors, memory bandwidth

```
Typical gain: 10–30%, for maybe 5% extra area. Excellent value —
when the threads stall a lot and do not fight over cache.

When SMT HURTS:
  · both threads are compute bound and already saturating the units
  · they have large, distinct working sets, so each halves the other's
    effective cache
  · latency matters: your thread now shares a core with an unknown
    neighbour, so worst-case timing becomes unpredictable
  · SECURITY: sharing caches and predictors is a side channel. Threads
    can observe each other. Cloud providers and hardened systems often
    disable SMT entirely for this reason.

ARM's application cores generally do NOT implement SMT — the design
choice has been more, simpler cores instead. Some Neoverse and
specialised ARM cores do. x86 calls it Hyper-Threading.
```

Multicore

```
Homogeneous                     Heterogeneous (big.LITTLE, DynamIQ)

  4x identical cores              2x big  (wide, OoO, high power)
  simple scheduling               4x LITTLE (narrow, in-order, efficient)
                                  same ISA, very different performance

  Heterogeneity buys energy efficiency: run background work on the
  LITTLE cores at a fraction of the power, migrate to big under load.

  What it costs YOU:
    · benchmark results depend on which core you landed on
    · a thread can migrate mid-measurement
    · worst-case timing analysis must assume the slowest core
    · pin with taskset when measuring anything (chapter 18)
```

Why adding cores stops helping

Limit	Mechanism	What to do
Amdahl's serial fraction	The sequential part never shrinks	Parallelise more of it, or accept the ceiling
Lock contention	Threads serialise on a shared lock	Finer-grained locks, per-CPU data, lock-free structures, RCU
Memory bandwidth	All cores share one memory controller	Improve locality; reduce bytes moved (ch. 13)
Cache coherence traffic	Shared lines ping-pong between cores	Pad to line boundaries; avoid shared writes (ch. 15)
False sharing	Unrelated variables in one cache line	Align per-thread data to 64 bytes
Load imbalance	One thread finishes last; the rest idle	Dynamic scheduling, work stealing, smaller chunks
Synchronisation overhead	Barriers and joins cost more than the work	Coarser tasks; fewer barriers
Power and thermal	All-core turbo is lower than single-core turbo	Nothing — it is physics (ch. 20)

> **NEGATIVE SCALING IS A SPECIFIC, DIAGNOSABLE BUG**
>
> If throughput *decreases* as you add threads, you are almost certainly looking at false sharing or lock contention — not at a fundamental limit. Confirm with `perf c2c` and lock profiling before concluding the problem is not parallelisable. Genuine bandwidth saturation flattens the curve; contention bends it downwards.

Parallel programming models

Model	Shape	Good for	Hazard
Shared memory threads	pthreads, `std::thread`	Fine-grained work on one machine	Races, deadlock, false sharing
Task / work stealing	OpenMP tasks, TBB, Rayon	Irregular and recursive work	Task granularity too fine
Data parallel	`#pragma omp parallel for`	Regular loops over arrays	Load imbalance; false sharing at chunk edges
Message passing	MPI, channels, actors	Distributed memory; no shared state by construction	Message overhead; harder to load balance
SPMD / GPU	CUDA, OpenCL, Vulkan compute	Massive regular data parallelism	Divergence, transfer cost, memory coalescing
Pipeline	Stages connected by queues	Streaming; heterogeneous stages	The slowest stage sets throughput

```
/* the same loop, three ways */

/* data parallel */
#pragma omp parallel for schedule(dynamic, 64) reduction(+:sum)
for (int i = 0; i < n; i++)
        sum += heavy(a[i]);
/* `dynamic` handles imbalance when heavy() varies per element;
   `reduction` gives each thread a private accumulator and combines
   them at the end - which is exactly the false-sharing fix from
   chapter 15, done for you.                                    */

/* explicit threads with per-thread state, padded */
struct alignas(64) partial { double sum; };
std::vector<partial> parts(nthreads);        /* one cache line each */

/* pipeline */
stage1 -> queue -> stage2 -> queue -> stage3
/* throughput = 1 / max(stage time). Balance the stages, and size
   the queues to absorb jitter without adding needless latency.   */
```

Measuring parallel efficiency

```
speedup    S(n) = T(1) / T(n)
efficiency E(n) = S(n) / n          # 1.0 = perfect

# plot efficiency against thread count. The SHAPE is diagnostic:
#   flat near 1.0           -> scaling well, keep going
#   gradual decline         -> Amdahl's serial fraction, or bandwidth
#   sharp knee              -> a resource saturated: bandwidth or a lock
#   DECREASING below 1 core -> false sharing or contention. A bug.

perf stat -e cycles,instructions ./program          # per-thread IPC
perf c2c record ./program                           # false sharing
perf lock record ./program && perf lock report      # lock contention
perf record -e sched:sched_switch                   # migration, imbalance
```

How it shows up

Symptom	Cause
4 threads give 1.3× speedup	Large serial fraction, or bandwidth saturation
Throughput falls as threads are added	False sharing or lock contention
Vectorised loop shows no gain	Memory bound; SIMD was never the bottleneck
Compiler refuses to vectorise a simple loop	Possible aliasing — add `restrict`
Manual unrolling with accumulators gives 4×	You broke a true dependency chain — an ILP win
Results differ between scalar and vector versions	Floating-point reassociation (ch. 03)
Benchmark varies 3× between runs	Thread landed on a LITTLE core; pin it
Disabling SMT made a latency-sensitive service faster	Contention and unpredictable sharing on the physical core
All-core workload runs at a lower clock than single-core	Power and thermal budget (ch. 20)

Check yourself

- ✓ Explain from $P \approx CV^2f$ why clock speeds stopped rising.
- ✓ Give the four kinds of parallelism and say which require programmer effort.
- ✓ Show how multiple accumulators break a dependency chain, and why the compiler will not do it for floats.
- ✓ Explain what `restrict` promises and why vectorisation needs it.
- ✓ Explain why SIMD does not help a memory-bound loop.
- ✓ Say what SMT duplicates, what it shares, and give two reasons to disable it.
- ✓ List five reasons adding cores stops helping, and how you would distinguish them.
- ✓ Interpret an efficiency-versus-threads curve that decreases below 1.0.

Power, energy and thermal limits

Power is the constraint that shapes every modern design decision — why clocks stopped rising, why phones have big and LITTLE cores, why your laptop is fast for ten seconds. For battery-powered embedded work it is often the primary specification, not an afterthought.

Power versus energy

TWO DIFFERENT QUANTITIES, TWO DIFFERENT GOALS

Power (watts) is the instantaneous rate. It determines heat, and therefore cooling, package and thermal throttling.

Energy (joules = watts × seconds) is the total consumed. It determines battery life.

They pull in different directions. Running twice as fast at three times the power *saves* energy if the device can then sleep — the "race to idle" strategy. Deciding which you are optimising is the first question in any power discussion, and getting it wrong wastes months.

Where the power goes

```
   P_total  =  P_dynamic  +  P_static

   P_dynamic = α × C × V² × f        switching: charging and
                                      discharging gate capacitance
        α  activity factor — fraction of gates toggling per cycle
        C  capacitance — grows with transistor count and wire length
        V  supply voltage
        f  clock frequency

   P_static  = I_leak × V            leakage: current through
                                      transistors that are "off"

Two facts follow, and they explain most of modern computing:

  1. V is SQUARED. Halving the voltage quarters dynamic power.
     But lower voltage means slower switching, so f must drop too.
     Reducing both gives roughly CUBIC savings — this is the entire
     basis of dynamic voltage and frequency scaling.

  2. Leakage grows as transistors shrink and as temperature rises,
     and it is paid whenever the circuit is powered at all — even
     idle. At modern nodes it can be 30–40% of total power. Hence
     POWER GATING: physically disconnect unused blocks, do not
     merely stop clocking them.
```

Why performance costs so much more than it returns

```
/* Approximate, but the shape is right and it is the shape that matters. */

  frequency   voltage   dynamic power   relative performance
  ──────────────────────────────────────────────────────────

   1.0 GHz     0.7 V        1.0 x             1.0 x
   1.5 GHz     0.8 V        1.96 x            1.5 x
   2.0 GHz     0.9 V        3.31 x            2.0 x
   2.5 GHz     1.05 V       5.63 x            2.5 x
   3.0 GHz     1.2 V        8.82 x            3.0 x

   3x the performance costs ~9x the power.
   Or, read the other way: HALF the frequency at reduced voltage
   costs about ONE EIGHTH the power.

This is why:
  · a LITTLE core at 1 GHz does background work at a tiny fraction
    of the energy a big core would spend
  · two cores at 1.5 GHz beat one at 3 GHz on both power AND
    throughput — when the work parallelises
  · "turbo" is a short-duration exception, not a sustainable state
```

The three walls

Wall	What happened	Consequence
Power wall (~2005)	Air cooling caps a chip at roughly 100–150 W; frequency scaling hit that ceiling	Clocks flattened at 3–4 GHz. Performance now comes from parallelism (ch. 19).
End of Dennard scaling (~2006)	Historically, shrinking transistors let voltage drop proportionally, keeping power per area constant. Below ~1 V, threshold voltage and leakage stopped that.	More transistors no longer come with free power savings.
Dark silicon	You can fit more transistors on a die than you can afford to switch simultaneously	Chips are full of specialised blocks — video codecs, NPUs, crypto engines — used one at a time. Specialisation became the way to spend transistors.

> **WHY ACCELERATORS EXIST**
>
> A fixed-function block doing one job — H.265 decode, AES, convolution — can be 10 to 1000× more energy efficient than a general-purpose core doing the same work, because it eliminates instruction fetch, decode, register file traffic and speculation. Dark silicon makes that trade obviously worthwhile: the block costs area, which is cheap, and saves energy, which is not. This is why a modern SoC is mostly *not* CPU, and why NPUs appeared in phones.

How hardware saves power

Technique	Mechanism	Attacks	Wake cost
Clock gating	Stop the clock to an idle block	Dynamic (f → 0)	~free, a cycle
Power gating	Disconnect the supply entirely	Dynamic *and* leakage	State is lost; µs to ms to restore
DVFS	Scale voltage and frequency together to match demand	Dynamic, cubically	µs; voltage rails settle slowly

Technique	Mechanism	Attacks	Wake cost
Heterogeneous cores	Migrate work to an efficient core	Dynamic	Migration + cache warm-up
Retention states	Low voltage that preserves SRAM contents but cannot run	Leakage, partially	Faster than full power-off
Race to idle	Finish fast, then enter a deep sleep state	Total energy	Depends on wake latency
Accelerators	Fixed-function hardware for hot workloads	Energy per operation	Setup and data movement

Idle states are a hierarchy, and deeper is not always better

```
ACPI C-states / ARM idle states — deeper saves more, wakes slower

  C0 / running     executing instructions
  C1 / WFI         clock gated, caches live          wake: ~ns
  C2               deeper clock gating               wake: ~µs
  C3               L1/L2 flushed, caches off         wake: ~10s of µs
  C6+ / power off  core powered down, state saved    wake: ~100s of µs
                   to always-on memory

  ARM's WFI (wait for interrupt) and WFE (wait for event) are the
  instructions that request this; PSCI CPU_SUSPEND selects the depth.

THE BREAK-EVEN CALCULATION that governs every idle decision:

  entering a state costs E_transition and T_wake.
  It is only worth entering if the expected idle time exceeds

        T_breakeven = E_transition / (P_shallow - P_deep)

  An OS governor predicts the next wake (from timers and past
  behaviour) and picks the deepest state whose break-even it clears.
  Predict wrong and you spend MORE energy than you saved.
```

> **WHY A PERIODIC TIMER IS EXPENSIVE OUT OF PROPORTION TO ITS WORK**
>
> A task that wakes every 1 ms to do nothing prevents the system from ever reaching a deep idle state. The work is negligible; the *lost sleep* is not. This is why Linux has a tickless kernel (`NO_HZ`), why timer coalescing and deferrable timers exist, and why a single badly written polling loop can halve a device's battery life. When profiling power, count *wakeups*, not CPU percentage — `powertop` exists precisely to show you this.

Thermal behaviour

```
   T_junction = T_ambient + P × R_thermal

     R_thermal = thermal resistance, °C per watt, from junction to air.
     Determined by package, heatsink, airflow, and PCB copper.

A phone SoC might have R_thermal ≈ 15 °C/W with no heatsink.
     5 W sustained  ->  75 °C above ambient  ->  throttling.

Which is why:

   · sustained performance is often HALF of burst performance
   · a benchmark's first 10 seconds do not predict the next 10 minutes
   · thermal throttling is a PERFORMANCE bug you cannot fix in software
   · leakage rises with temperature, and leakage generates heat —
     a positive feedback loop that must be actively broken by
     throttling, or the part destroys itself
```

Concept	Meaning
TDP	Thermal Design Power — what the cooling must remove. Not maximum power, and not average power. A design target, frequently misread as a spec.
Sustained vs burst	Short bursts exploit thermal mass; sustained load is limited by $R_{thermal}$
Throttling	Firmware reduces frequency and voltage when a temperature limit is hit
Thermal mass	Heat capacity that buys time before the limit is reached — why turbo works at all
Hot spots	Temperature is not uniform; the GPU corner may throttle while the die average looks fine

What software controls

```
Ranked by leverage. The top of this list dwarfs the bottom.

  1. DO LESS WORK.
     A better algorithm saves energy in every term at once. No power
     technique competes with not executing the instructions.

  2. MOVE LESS DATA.
     Energy per operation, roughly, at a modern node:
        32-bit integer add          ~0.1 pJ
        32-bit float multiply       ~1 pJ
        32-bit SRAM (cache) read    ~5 pJ
        32-bit DRAM read           ~200 pJ   <- 2000x the add
        wireless transmit, per bit  ~1000 pJ

     A DRAM access costs more energy than a thousand arithmetic
     operations. Cache optimisation (chapters 13-14) is therefore
     ALSO the most effective power optimisation available to you.

  3. SLEEP MORE, AND DEEPER.
     Batch work. Coalesce timers. Use interrupts, never polling.
     Minimise wakeups, not just CPU time.

  4. USE THE RIGHT UNIT.
     Offload to an accelerator, or to a LITTLE core.

  5. LET THE GOVERNOR DO ITS JOB.
     Do not pin frequency to maximum "for consistency" outside
     of a measurement session.
```

```
/* polling versus interrupts, in energy terms */

/* BAD: the core never sleeps, the whole SoC stays in a shallow state */
while (!(readl(base + STATUS) & READY))
        ;

/* BETTER: sleep until the hardware says something happened */
wait_event_interruptible(dev->wq, dev->ready);

/* In bare metal, WFI is the whole idea: */
    while (!ready) {
        __asm__ volatile("wfi");   /* clock-gate until an interrupt */
    }

/* Batching beats spreading: 100 wakeups doing 1 unit of work each
   costs far more than 1 wakeup doing 100 units, because each wakeup
   pays the transition energy and blocks deep idle.               */
```

Measuring power

```
# Linux: where are the wakeups coming from?
sudo powertop                    # per-process wakeups/second and estimated power
sudo powertop --auto-tune        # applies known-good power settings

# what state is the CPU actually in?
cpupower monitor                 # residency in each C-state and P-state
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor

# thermal zones and trip points
cat /sys/class/thermal/thermal_zone*/type
cat /sys/class/thermal/thermal_zone*/temp     # millidegrees C

# on-die energy counters, where available
perf stat -e power/energy-pkg/ ./program

# battery-powered embedded work: an external measurement is
# ground truth. A current-sense resistor plus a scope, or a
# purpose-built power analyser, shows the wakeup pattern directly
# in a way no software counter can.
```

Power in embedded design

Class	Budget	Dominant concern
Energy harvesting	µW	Sleep current. The device is asleep >99.9% of the time; active power barely matters.
Coin-cell sensor	10 µA average	Duty cycle and radio energy per packet
Wearable	mW	Wakeup frequency; always-on sensing on a low-power island
Phone	1–8 W	Thermal limits, big.LITTLE scheduling, display and radio
Automotive / industrial	W–kW	Thermal reliability over a wide temperature range

```
/* Average current for a duty-cycled sensor — the calculation that
   determines whether a product ships.                              */

   sleep current      2 µA
   active current     8 mA
   active time        20 ms per reading
   reading interval   10 s

   duty cycle = 0.020 / 10 = 0.2%
   I_avg = 0.002 × 8000 µA + 0.998 × 2 µA
         = 16 µA + 2 µA = 18 µA

   225 mAh coin cell / 0.018 mA ≈ 12,500 hours ≈ 1.4 years

   Now suppose a firmware bug leaves the radio in idle at 200 µA:
   I_avg = 218 µA  ->  43 days.

   Same code, same features, 12x shorter life. Sleep current is
   almost always the number that decides the product.            */
```

How it shows up

Symptom	Cause
Benchmark is fast for 10 s then drops 40%	Thermal throttling; the burst exploited thermal mass
All-core workload runs at a lower clock than single-core	Power budget shared across cores
Battery life is a fraction of the calculation	Sleep current higher than assumed, or a peripheral left enabled
Device never enters deep idle	A periodic timer or polling loop preventing it
Performance varies between identical units	Silicon variation, cooling differences, or ambient temperature
Results differ between runs at the same nominal frequency	DVFS transitions and core migration
Cache optimisation reduced power measurably	DRAM accesses cost ~2000× an ALU operation
An accelerator is slower but the battery lasts longer	Optimising energy, not latency — often the correct trade
Timing analysis fails on a big.LITTLE system	Worst case must assume the LITTLE core at its lowest frequency

Check yourself

- ✓ State the difference between power and energy, and give a case where they conflict.
- ✓ Write the dynamic power equation and explain why voltage is the important term.
- ✓ Explain why 3× frequency costs roughly 9× power.
- ✓ Describe the power wall, the end of Dennard scaling and dark silicon, and what each caused.
- ✓ Distinguish clock gating from power gating and say what each attacks.
- ✓ Give the break-even condition for entering a deeper idle state.
- ✓ Explain why a 1 ms periodic timer is expensive out of proportion to its work.
- ✓ Compare the energy of an integer add and a DRAM access, and say what that implies for optimisation.
- ✓ Compute average current for a duty-cycled sensor and show why sleep current dominates.

TRACK COMPLETE

That closes the computer architecture track. You now have the vocabulary every other track on this site assumes: binary representation, logic, the stored-program model, pipelines, the memory hierarchy, coherence, virtual memory, I/O, and the performance and power models used to reason about all of it.

From here, the C track makes these ideas concrete in a language, and the ARM track makes them concrete in a real architecture.

TRACK

The C language

C is the language of every kernel, driver and bootloader on this site. This track covers it to the depth a systems programmer actually needs.

20 chapters

The C abstract machine and translation phases

C does not describe what your processor does. It describes a hypothetical machine, and the compiler must only produce a program that *behaves as if* that machine ran it. Everything confusing about C — optimisation surprises, undefined behaviour, `volatile` — follows from this one idea.

What C actually is

Beginners learn C as "portable assembly": each statement becomes a few instructions. That model is wrong, and it will mislead you for years.

> **THE AS-IF RULE**
>
> The C standard defines an **abstract machine** with its own rules for evaluation, storage and side effects. A conforming implementation must reproduce the abstract machine's *observable behaviour* — and nothing else is guaranteed. Observable behaviour is:
>
> - accesses to `volatile` objects, exactly as written;
> - data written to files at program termination;
> - interactive input and output prompts, appearing before the reads they prompt.
>
> Everything else — how many instructions, which registers, whether a variable exists at all, in what order independent computations happen — is the compiler's business.

```
/* You wrote this: */
int f(void) {
        int a = 10;
        int b = 20;
        int c = a * b;
        int d[100];
        for (int i = 0; i < 100; i++) d[i] = i;
        return c;
}

/* GCC -O2 emits this: */
f:
        mov     w0, #200
        ret

/* The array, the loop, and the variables never existed. Nothing about
   them was observable, so the abstract machine permits their removal.
   The compiler is not "cheating" — it is doing exactly what C says. */
```

Why this matters immediately

You expect	The abstract machine says	Consequence
A loop that reads a hardware register will re-read it	Nothing changed the object, so one read suffices	Infinite loop unless you use `volatile` (ch. 10)
Zeroing a password buffer before `free` will happen	The buffer is dead afterwards; the store is unobservable	The `memset` is deleted; secrets remain in memory

You expect	The abstract machine says	Consequence
Signed overflow wraps like the hardware does	Signed overflow is undefined; assume it never happens	Your overflow check is deleted (ch. 15)
Two writes to different globals happen in order	Neither is observable in isolation	Another core sees them reordered (CA ch. 15)
Local variables live on the stack	Storage is only required if the address is taken	They live in registers; a debugger shows <optimized out>

THE MENTAL SHIFT

Stop asking "what instructions will this produce?" and start asking **"what am I permitted to assume?"** The compiler is not your adversary, but it is a strict contract lawyer: it will use every freedom the standard grants. Code that relies on unguaranteed behaviour works until the day you raise the optimisation level, change compilers, or upgrade GCC — and then fails in production.

The eight translation phases

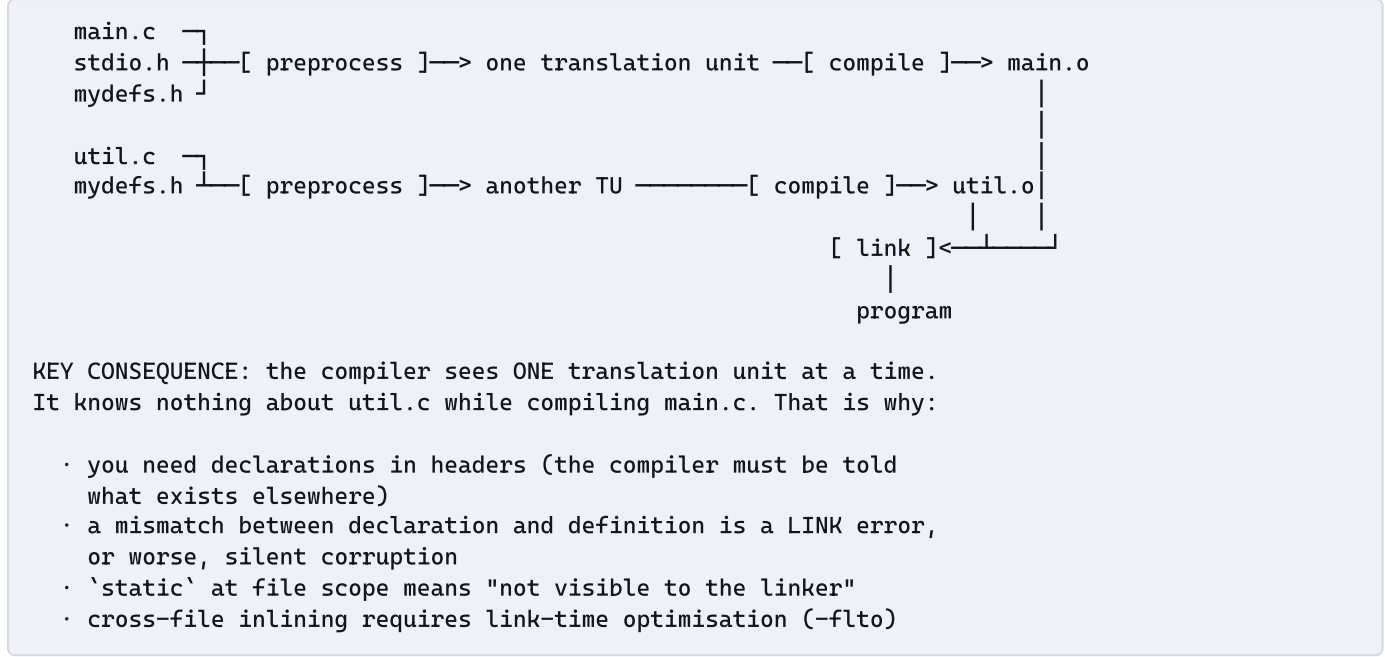
The standard defines translation as a sequence of phases. Knowing them explains a whole class of otherwise baffling errors.

```
1  Map physical source characters to the source character set.
   Trigraphs (??= for #) replaced. Removed in C23; good riddance.

2  Splice lines ending in backslash-newline.
   -> why a stray space AFTER the backslash in a multi-line macro
      breaks it: the backslash is no longer at end of line.

3  Decompose into preprocessing tokens and whitespace.
   Comments become a single space HERE — so a comment cannot be
   produced by a macro, and /* */ inside a string literal is text.

4  Execute preprocessing directives, expand macros, and recursively
   process #include files. Phases 1-4 run again on each header.
   -> #include is literal text substitution. Nothing more.

5  Convert escape sequences and characters in literals to the
   execution character set.

6  Concatenate adjacent string literals.
   -> "abc" "def" becomes "abcdef". This is why the PRIu32 macros
      work:  printf("%" PRIu32 "\n", x)

7  COMPILE. Tokens are analysed syntactically and semantically,
   and translation units become object code.

8  LINK. Resolve external references; combine translation units
   and library components into the final image.
```

```
/* Phase 2 in action — the classic multi-line macro bug */
#define SWAP(a, b) do {  \
        int t = (a);     \
        (a) = (b);       \ <-- one trailing space here
        (b) = t;         \
} while (0)
/* error: unterminated argument list / stray '\'
   because the line no longer ends in backslash-newline.       */

/* Phase 6 in action — adjacent literal concatenation */
#include <inttypes.h>
printf("count = %" PRIu32 " items\n", n);
/* PRIu32 expands to "u" (or "lu"), and phase 6 glues the three
   literals into one format string. This is how the standard makes
   printf portable across platforms with different int widths.  */
```

The translation unit

C's unit of compilation is one source file *after preprocessing* — the file plus every header it included, textually pasted in.




```
KEY CONSEQUENCE: the compiler sees ONE translation unit at a time.
It knows nothing about util.c while compiling main.c. That is why:

  · you need declarations in headers (the compiler must be told
    what exists elsewhere)
  · a mismatch between declaration and definition is a LINK error,
    or worse, silent corruption
  · `static` at file scope means "not visible to the linker"
  · cross-file inlining requires link-time optimisation (-flto)
```

```
/* the classic silent disaster this enables */

/* a.c */                          /* b.c */
int counter;                       extern short counter;   /* WRONG type */
void inc(void) { counter++; }      void reset(void) { counter = 0; }

/* Each TU compiles cleanly. The linker matches them by NAME only.
   b.c writes 2 bytes where a.c expects 4. Result: corruption that
   depends on endianness and layout.

   The fix is structural, not a matter of care: declare it ONCE, in a
   header, and include that header in BOTH files — including the one
   that defines it, so the compiler checks the definition against
   the declaration.                                                  */

/* counter.h */
extern int counter;
void inc(void);
void reset(void);
```

Sequencing and the abstract machine's clock

The abstract machine does not execute one statement at a time in a simple order. It has **sequence points** (C99) or, more precisely, a *sequenced-before* relation (C11).

Guaranteed sequenced	NOT sequenced
End of a full expression (the `;`)	Between function arguments
Between `&&` and `||` operands	Between the two operands of `+`, `*`, `=`
Between `,` operator operands	Between subexpressions generally
Before a function call's body runs	Two calls in one expression: order unspecified
Between `?:` condition and the chosen branch	

```
/* Undefined behaviour: i modified twice with no sequencing between */
i = i++ + 1;
a[i] = i++;
f(i++, i++);          /* also: argument evaluation order is unspecified */

/* Well defined, because && sequences its operands */
if (p != NULL && p->field > 0)      /* p->field is not evaluated if p is NULL */

/* Unspecified but NOT undefined: the order of the two calls */
printf("%d %d\n", g(), h());        /* g may run before or after h */

/* This is why the "clever" one-liner is a genuine bug, not a style
   issue. There is no "what it does on my compiler" — the standard
   permits anything, including formatting your disk, and in practice
   different optimisation levels give different answers.          */
```

Objects, values and lvalues

Term	Meaning
Object	A region of storage whose contents can represent values. Not "an instance of a class" — C has no classes.

Term	Meaning
Value	The meaning of an object's contents, as interpreted by its type
Lvalue	An expression that designates an object. "Locator value" — it has an address.
Rvalue	Informally, the value of an expression. The standard says "value of an expression".
Effective type	The type through which an object may be accessed. Violating it breaks strict aliasing (ch. 15).
Lifetime	The span during which storage is guaranteed reserved. Using an object outside its lifetime is undefined.

```
int x = 5;
int *p = &x;

x        /* lvalue: designates the object x        */
*p       /* lvalue: designates the object x        */
x + 1    /* not an lvalue: designates no object    */
&x       /* not an lvalue: the address is a value  */

x = 10;      /* fine: lvalue on the left  */
x + 1 = 10;  /* error: not an lvalue      */

/* Arrays are lvalues that are NOT assignable — a distinct category
   the standard calls "non-modifiable lvalue". Hence: */
int a[10], b[10];
a = b;       /* error, even though both are lvalues */
```

The abstract machine's memory model

```
C guarantees FOUR storage durations, not "the stack and the heap".
Where they physically live is the implementation's choice.

  static        exists for the whole program run
                globals, file-scope and function-scope `static`
                zero-initialised if not explicitly initialised
                -> typically .data and .bss

  thread        one instance per thread (_Thread_local, C11)
                -> typically thread-local storage blocks

  automatic     exists from block entry to block exit
                ordinary local variables
                INDETERMINATE unless you initialise them
                -> typically registers or the stack; often neither,
                   if the optimiser can prove no storage is needed

  allocated     from malloc/calloc/realloc until free
                -> typically the heap

The words "stack" and "heap" appear NOWHERE in the C standard. A
freestanding implementation (chapter 20) may have no heap at all.
```

Hosted versus freestanding

The standard defines two conformance levels, and the distinction is exactly the one that matters for kernel and bare-metal work.

	Hosted	Freestanding
Entry point	`main()` with the standard signature	Implementation-defined — `_start`, `Reset_Handler`, `DriverEntry`
Library	The complete standard library	Only headers with no runtime: `<stddef.h>`, `<stdint.h>`, `<stdbool.h>`, `<limits.h>`, `<stdarg.h>`, `<float.h>`, `<iso646.h>`, `<stdalign.h>`, `<stdnoreturn.h>`
Examples	Application on Linux or Windows	Linux kernel, bootloaders, Cortex-M firmware, UEFI

```
# building freestanding — what these flags actually mean
aarch64-none-elf-gcc \
    -ffreestanding \      # do not assume the standard library exists
    -nostdlib \           # do not link libc or the startup files
    -fno-builtin \        # do not replace calls with builtin versions
    -fno-stack-protector \
    -c main.c

# -fno-builtin matters more than it looks. Without it, GCC may turn
# a hand-written loop into a call to memcpy() — which does not exist
# in your freestanding image. Linker error, in a function you never
# called. The Linux kernel builds with -ffreestanding for this reason.
```

Standard versions

Standard	Worth knowing for
C89 / C90	Declarations at the top of a block only; no `//` comments. Still mandated in some safety-critical and legacy codebases.
C99	`//` comments, declarations anywhere, `<stdint.h>`, `<stdbool.h>`, `inline`, VLAs, designated initialisers, compound literals, `restrict`
C11	`_Atomic`, `_Thread_local`, `_Static_assert`, `_Alignas` / `_Alignof`, anonymous structs and unions, a real memory model
C17	Bug fixes only; no new features
C23	`nullptr`, `constexpr`, `typeof`, binary literals, `[[attributes]]`, `bool` / `static_assert` as keywords, trigraphs removed

```
# always state the standard explicitly; the default varies by compiler version
gcc -std=c11 -pedantic       # strict ISO C11
gcc -std=gnu11               # C11 plus GNU extensions (the Linux kernel uses gnu11/gnu17)

# GNU extensions you WILL meet in kernel code, and should recognise
typeof(x) tmp;                       /* now standard in C23    */
({ int a = f(); a * a; })            /* statement expressions  */
__attribute__((packed, aligned(4)))
[0 ... 15] = 0                       /* range initialisers     */
```

How it shows up

Symptom	Cause
Code works at `-O0`, breaks at `-O2`	You relied on behaviour the abstract machine does not guarantee
A polling loop never exits	Missing `volatile`; the compiler hoisted the load

Symptom	Cause
A debugger reports `<optimized out>`	The variable was never given storage — it was not observable
Multi-line macro fails with "stray backslash"	Trailing whitespace after the line continuation (phase 2)
Clean compile, wrong values at runtime across files	Mismatched declaration and definition; no shared header
`i = i++ + 1` differs between compilers	Undefined behaviour, not implementation choice
Link error for `memcpy` you never called	The compiler generated it; build with `-fno-builtin`
A `memset` that scrubs a key is deleted	Dead store elimination; use `memset_s` or a `volatile` pointer

Check yourself

- ✓ State the as-if rule and list what counts as observable behaviour.
- ✓ Explain why a compiler may delete an entire loop, and when it may not.
- ✓ Name the eight translation phases and one bug each of phases 2, 4 and 6 explains.
- ✓ Define a translation unit and explain why a type mismatch across files can pass the build.
- ✓ Give three expressions that are undefined because of missing sequencing.
- ✓ Name the four storage durations and say which are initialised for you.
- ✓ State two differences between hosted and freestanding, and why `-fno-builtin` matters.

Types, ranks and integer promotion

C silently converts operand types before almost every operation. The rules are precise, mostly invisible, and responsible for a remarkable share of real security vulnerabilities. This chapter makes them explicit.

The types, and what is actually guaranteed

C does not fix the size of its integer types. It fixes *minimum ranges*, and leaves the rest to the implementation.

Type	Guaranteed minimum	Typical LP64 (Linux/ARM64)	Typical LLP64 (Windows x64)
`char`	8 bits	8	8
`short`	16 bits	16	16
`int`	16 bits	32	32
`long`	32 bits	**64**	**32**
`long long`	64 bits	64	64
`void *`	—	64	64

THE SINGLE MOST EXPENSIVE ASSUMPTION IN C

`long` is 64-bit on Linux and macOS, and 32-bit on Windows. Code that stores a pointer or a file offset in a `long` compiles cleanly on both and silently truncates on one. Every large port between these platforms has hit this. The rule: **never use `long` when you mean "64 bits"** — use `int64_t`, and use `size_t` / `uintptr_t` for sizes and pointer-sized integers.

The types you should actually use

```
#include <stdint.h>
#include <stddef.h>
#include <inttypes.h>

/* exact width — use these for anything that touches hardware,
   a file format, or the wire. They are optional in the standard
   but present on every platform you will meet.                  */
int8_t  int16_t  int32_t  int64_t
uint8_t uint16_t uint32_t uint64_t

/* at-least width, possibly faster than the exact type */
int_least32_t   int_fast32_t

/* pointer-sized integers */
intptr_t  uintptr_t          /* a pointer fits in these           */
ptrdiff_t                    /* result of subtracting pointers    */

/* sizes and counts */
size_t                       /* result of sizeof; unsigned        */
ssize_t                      /* POSIX: a size or -1 for error     */

/* printing them portably — this is what <inttypes.h> is for */
printf("%" PRIu32 " %" PRIx64 "\n", a, b);
printf("%zu %td %p\n", sz, diff, (void *)ptr);
/* %zu for size_t, %td for ptrdiff_t, and cast to void* for %p —
   all three are required, and all three are commonly got wrong.  */
```

Integer promotion

> **THE RULE**
>
> In almost any arithmetic context, any operand of a type whose rank is lower than `int` — `char`, `short`, `_Bool`, bitfields — is first converted to `int`, if `int` can represent all its values. Otherwise to `unsigned int`.
>
> This happens *before* the operator is applied, and it happens whether you want it or not.

```
uint8_t a = 200, b = 100;

uint8_t c = a + b;
/* What happens:
     a is promoted to int -> 200
     b is promoted to int -> 100
     the addition is done in int -> 300      (NO wraparound here)
     the result is converted back to uint8_t -> 300 % 256 = 44
   So c == 44, but the intermediate value really was 300.        */

/* This matters when you check for overflow: */
if (a + b > 255)     /* TRUE — the comparison is done in int */
        handle_overflow();
/* ...which is what you wanted. But: */
if ((uint8_t)(a + b) < a)
        handle_overflow();   /* also works, and is the usual idiom */
```

```
/* The classic promotion surprise */
uint16_t x = 0xFFFF;
uint16_t y = ~x;                 /* what is y? */

/* Step by step:
     x promotes to int:        0x0000FFFF
     ~ applied in int:         0xFFFF0000   (int is 32-bit)
     converted back to uint16: 0x0000
   y == 0. Which is what you wanted, by luck.

   But this one is not luck: */
if (~x == 0xFFFF0000) ...        /* comparing uint16_t against int */

/* And this is a real bug: */
uint32_t z = ~x;                 /* z == 0xFFFF0000, NOT 0x0000FFFF */
/* Because ~x was computed in int, not in uint16_t. */
```

The usual arithmetic conversions

Once both operands are at least `int`, they are brought to a *common type*:

```
  1. If either is a floating type, convert to the larger floating type
     (long double > double > float).

  2. Otherwise, integer promotion is applied to both.

  3. If both have the same type, done.

  4. If both are signed, or both unsigned: convert to the HIGHER RANK.
     rank: long long > long > int > short > char

  5. MIXED SIGNEDNESS — where the bugs live:

     a) If the UNSIGNED type has rank >= the signed type's rank,
        the signed operand is converted to UNSIGNED.
                    <-- this is the dangerous case

     b) Otherwise, if the signed type can represent all values of
        the unsigned type, the unsigned operand is converted to
        SIGNED.

     c) Otherwise, both are converted to the unsigned version of
        the signed type.
```

RULE 5A, AND THE VULNERABILITY IT CREATES

```
int len = -1;
if (len < sizeof(buffer))         /* sizeof yields size_t: UNSIGNED */
        memcpy(buffer, src, len);

/* len is converted to size_t.
   -1 as an unsigned 64-bit value is 18446744073709551615.
   The comparison is FALSE... no wait — it is 18446744073709551615 <
   sizeof(buffer)? No. So the branch is not taken. Lucky.

   Now flip the comparison, which is just as natural to write: */

if (len > sizeof(buffer))
        return -EINVAL;           /* NOT taken: -1 became huge?  no —
                                     huge > sizeof, so it IS taken.  */

/* The point is that you cannot reason about it at a glance, and the
   answer changes with the direction of the comparison and the types
   involved. Compare a signed length against a signed limit, or
   validate that the length is non-negative FIRST:                  */

if (len < 0 || (size_t)len > sizeof(buffer))
        return -EINVAL;
```

CVE databases are full of this pattern. Compile with `-Wsign-compare` (included in `-Wextra`) and treat every warning as a bug, not noise.

```
/* another everyday instance of the same rule */
for (int i = 0; i < strlen(s) - 1; i++)   /* strlen returns size_t */
        ...
/* If s is empty: strlen(s) is 0, and 0 - 1 in size_t is SIZE_MAX.
   The loop runs about 18 quintillion times. */

/* fix: compute the bound once, in a signed type, or restructure */
size_t n = strlen(s);
for (size_t i = 0; i + 1 < n; i++)        /* no subtraction below zero */
        ...
```

Signed versus unsigned: choosing deliberately

	Signed	Unsigned
Overflow	**Undefined behaviour** — the compiler assumes it cannot happen	Defined: wraps modulo 2^N
Optimisation	Better: the compiler can assume `i + 1 > i`	Worse: must handle wraparound
Comparison against the other	Dangerous — see above	
Right shift	Implementation-defined for negatives (arithmetic in practice)	Defined: logical, fills with zero
Use for	Quantities that can meaningfully be negative; loop indices that count down	Bit manipulation, hardware registers, sizes, hashes, modular arithmetic

```
/* Signed overflow is UB, and the optimiser acts on it: */
int check(int x) {
        return x + 1 > x;        /* compiles to `return 1;` */
}
/* Because signed overflow "cannot happen", x + 1 > x is always true.
   Your overflow check is deleted. See chapter 15.                  */

/* Unsigned wraps predictably, which is why hardware code uses it: */
uint32_t elapsed = now - start;   /* correct even when `now` wrapped */

/* Countdown loops with unsigned indices are a perennial bug: */
for (size_t i = n - 1; i >= 0; i--)   /* i >= 0 is ALWAYS true */
        ...                            /* infinite loop         */

for (size_t i = n; i-- > 0; )          /* correct idiom          */
        use(a[i]);
```

The three character types

```
/* char, signed char and unsigned char are THREE DISTINCT TYPES.
   Whether plain `char` is signed is implementation-defined:
       x86, ARM on Windows/macOS  -> signed
       ARM on Linux, PowerPC      -> UNSIGNED
   This has broken more ports than almost anything else.        */

char c = 200;            /* signed char: this is -56. unsigned: 200 */
if (c == 200) ...        /* false on ARM Linux, true on x86         */

/* The classic getchar() bug: */
char c;
while ((c = getchar()) != EOF)   /* WRONG */
        process(c);
/* getchar returns int so it can return 256 distinct byte values PLUS
   EOF (-1). Storing it in char loses that distinction: on a signed-
   char platform, byte 0xFF becomes -1 and terminates the loop early;
   on an unsigned-char platform the loop NEVER terminates because c
   can never equal -1.                                              */

int c;                           /* correct */
while ((c = getchar()) != EOF)
        process((unsigned char)c);

/* Rules that keep you out of trouble:
     - use `unsigned char` for raw bytes and buffers
     - use `char` only for actual text
     - cast to unsigned char before passing to <ctype.h> functions:
           isalpha((unsigned char)c);
       passing a negative char there is undefined behaviour        */

/* -funsigned-char / -fsigned-char force it, but relying on a flag
   makes your code non-portable in a different way. Be explicit. */
```

Conversion rules

Conversion	Behaviour
Narrowing to unsigned	Well defined: modulo 2^N. `(uint8_t)300` is 44.

Conversion	Behaviour
Narrowing to signed	Value fits: preserved. Does not fit: **implementation-defined** (in practice, truncation with the top bit becoming the sign)
Signed → unsigned	Well defined: add 2^N until in range. `(unsigned)-1` is `UINT_MAX`.
Unsigned → signed	Fits: preserved. Does not fit: implementation-defined
Integer → float	Rounded if not exactly representable (CA ch. 03)
Float → integer	**Truncates toward zero.** Out of range or NaN is *undefined*.
Anything → `_Bool`	0 becomes false, anything else true. Not truncation.

```
(int)3.99      ->  3      /* truncation, not rounding      */
(int)-3.99     -> -3      /* toward zero, not toward -inf  */
(int)1e30      ->  UNDEFINED — does not fit in int
(_Bool)256     ->  1      /* not 0! _Bool is not a narrow int */

/* rounding, when you actually want it */
#include <math.h>
lround(3.5)    ->  4
(int)(x + 0.5) ->  wrong for negatives, and for x near the .5 boundary
```

Writing overflow checks that survive the optimiser

```
/* WRONG: relies on signed overflow, which is UB, so it is deleted */
if (a + b < a) overflow();

/* RIGHT 1: compiler builtins. Portable across GCC and Clang,
   and they map to the CPU's flags — one instruction. */
int result;
if (__builtin_add_overflow(a, b, &result))
        return -EOVERFLOW;

/* RIGHT 2: check before the operation, using the type's limits */
#include <limits.h>
if (b > 0 && a > INT_MAX - b) return -EOVERFLOW;
if (b < 0 && a < INT_MIN - b) return -EOVERFLOW;

/* RIGHT 3: for sizes, use unsigned and check the wrap — defined behaviour */
size_t total = n * size;
if (n != 0 && total / n != size)
        return -EOVERFLOW;              /* the classic malloc guard */

/* Linux kernel helpers that do exactly this */
check_add_overflow(a, b, &res);
array_size(n, size);          /* saturates to SIZE_MAX on overflow */
struct_size(p, member, n);
```

Compiler flags that catch these

```
gcc -Wall -Wextra \
    -Wsign-compare \        # signed/unsigned comparison — enable and OBEY
    -Wconversion \          # implicit narrowing conversions (noisy but valuable)
    -Wsign-conversion \
    -Wshadow \
    -Wcast-align \
    -Wpointer-arith \
    -Wstrict-overflow=5 \   # warn where the optimiser assumes no overflow
    -fno-strict-overflow    # ...or make signed overflow defined (wraps)

# find them at runtime instead
gcc -fsanitize=undefined -fsanitize=signed-integer-overflow
# UBSan reports the exact file, line and values at the moment it happens.
# Run your test suite under it — it costs a few percent and finds real bugs.
```

How it shows up

Symptom	Cause
A length check passes for a negative length	Signed converted to unsigned in the comparison
Loop over `strlen(s) - 1` runs forever on an empty string	`size_t` underflow
An overflow check compiles to `return 1`	Signed overflow is UB; the check was deleted
Code works on x86, misbehaves on ARM Linux	Plain `char` signedness differs
`getchar` loop stops early or never stops	Return value stored in `char` instead of `int`
Pointer truncated on Windows, fine on Linux	`long` is 32-bit under LLP64
`~x` on a `uint16_t` gives a huge number	Promotion to `int` before the operator
`printf("%d", sizeof(x))` prints garbage	Wrong conversion specifier; use `%zu`
Countdown loop with `size_t` never ends	`i >= 0` is always true for unsigned

Check yourself

- ✓ State what C guarantees about the size of `int` and `long`, and give the LP64/LLP64 difference.
- ✓ Explain integer promotion and predict the value of `~x` for a `uint16_t`.
- ✓ Walk through the usual arithmetic conversions for `int` versus `size_t`.
- ✓ Write a length check that is safe against a negative input.
- ✓ Explain why signed overflow being UB is useful to the optimiser and dangerous to you.
- ✓ Give the correct countdown-loop idiom for an unsigned index.
- ✓ Explain the `getchar` bug and why it fails differently on two platforms.
- ✓ Write a multiplication overflow check that the optimiser cannot remove.

Operators, precedence and evaluation order

Precedence decides how an expression is *parsed*. Evaluation order decides *when* each part runs. C fixes the first completely and leaves much of the second deliberately unspecified — and confusing the two produces bugs that change with the optimisation level.

The precedence table

```
Highest to lowest. Learn the shape, not the numbers.

 1  postfix        a[i]  f()  a.b  p->b  a++  a--  (type){init}    L→R
 2  unary          ++a  --a  +a  -a  !a  ~a  (type)a  *p  &a
                   sizeof  _Alignof                                R→L
 3  multiplicative *  /  %                                         L→R
 4  additive       +  -                                            L→R
 5  shift          <<  >>                                          L→R
 6  relational     <  <=  >  >=                                    L→R
 7  equality       ==  !=                                          L→R
 8  bitwise AND    &                                               L→R
 9  bitwise XOR    ^                                               L→R
10  bitwise OR     |                                               L→R
11  logical AND    &&                                              L→R
12  logical OR     ||                                              L→R
13  conditional    ?:                                              R→L
14  assignment     =  +=  -=  *=  /=  %=  <<=  >>=  &=  ^=  |=     R→L
15  comma          ,                                               L→R
```

THE FOUR PRECEDENCE MISTAKES THAT ACTUALLY HAPPEN

Every one of these compiles without a word from the compiler unless you enable warnings.

```
/* 1. Bitwise operators bind LOOSER than comparison.
      This is C's oldest design mistake, inherited from B. */
if (x & MASK == 0)          /* parsed as  x & (MASK == 0)  ->  x & 0  ->  0 */
if ((x & MASK) == 0)        /* what you meant                              */

/* 2. Shift binds looser than arithmetic. */
x << 2 + 3                  /* parsed as  x << (2 + 3)  =  x << 5 */
(x << 2) + 3                /* what you probably meant             */

/* 3. Dereference binds looser than postfix ++. */
*p++                        /* parsed as  *(p++)   - increment the POINTER  */
(*p)++                      /* increment the VALUE                          */

/* 4. Assignment inside a condition, without parentheses. */
while (c = getchar() != EOF)      /* c = (getchar() != EOF)  ->  c is 0 or 1 */
while ((c = getchar()) != EOF)    /* correct                                 */
```

Build with `-Wparentheses` (in `-Wall`) and it will flag most of these. The habit that removes the whole class: **parenthesise whenever mixing categories** — bitwise with comparison, shift with arithmetic, assignment inside a condition. Clarity costs nothing.

Precedence is not evaluation order

> **THE DISTINCTION**
> `a() + b() * c()` — precedence says the multiplication groups first, so the result is `a() + (b() * c())`. It says *nothing* about which function runs first. The compiler may call them in any order, and different optimisation levels genuinely produce different orders.

```
/* UNSPECIFIED: legal, but the order may vary. Do not depend on it. */
printf("%d %d\n", g(), h());        /* g and h in either order      */
int x = f() + g();                  /* likewise                     */
a[i()] = b[j()];                    /* likewise                     */

/* UNDEFINED: the standard imposes no requirements at all.
   Two modifications, or a modification and a read, of the same
   object with no sequencing between them.                          */
i = i++ + 1;
a[i] = i++;
i = ++i + i++;
f(i++, i++);

/* These are not "compiler-dependent" — they are BROKEN. The compiler
   may produce anything, and modern compilers exploit that to
   optimise. Enable -Wsequence-point (in -Wall).                     */
```

Where C does guarantee order

Construct	Guarantee	Why it matters
`&&` and `||`	Left evaluated fully first; right only if needed	Short-circuit guards: `if (p && p->x)`
`?:`	Condition first; then exactly one branch	Safe division: `d ? n/d : 0`
`,` operator	Left evaluated and discarded, then right	Multiple side effects in a `for` header
End of full expression	All side effects complete before the next statement	The basic sanity guarantee
Function call	All arguments evaluated before the body runs	But not in any particular order among themselves
C11 assignment	Right operand sequenced before the left in a compound assignment's read-modify-write	Makes `a[i++] += 1` defined in C11

```
/* Short-circuit is a guarantee you can build on */
if (p != NULL && p->len > 0 && p->data[0] == 'x')   /* safe, in order */
if (i < n && a[i] == target)                         /* bounds check first */
if (denom == 0 || total / denom > limit)             /* no divide by zero  */

/* The comma operator, used properly — rare outside for-headers */
for (i = 0, j = n - 1; i < j; i++, j--)
        swap(&a[i], &a[j]);

/* Note: the comma in a function call is a SEPARATOR, not the comma
   operator, and imposes no ordering.                               */
f(a, b);          /* separator: no sequencing between a and b */
f((a, b));        /* comma OPERATOR: one argument, b, after evaluating a */
```

Bitwise operators in practice

Op	Effect	Idiom
`&`	AND	Test or clear bits: `x & MASK`, `x &= ~BIT`
`|`	OR	Set bits: `x |= BIT`
`^`	XOR	Toggle bits: `x ^= BIT`; also equality-to-zero test
`~`	NOT	Build inverse masks: `~(1u << n)`
`<<`	Left shift	Multiply by 2^n; build masks
`>>`	Right shift	Divide by 2^n (unsigned); extract fields

SHIFT IS A MINEFIELD OF UNDEFINED BEHAVIOUR

```
x << n     /* UB if n < 0, or n >= width of the PROMOTED type of x */
x >> n     /* same                                                  */

int x = 1;
x << 32;              /* UB: int is 32 bits, so n must be 0..31      */
x << 31;              /* UB in C: shifting into the sign bit of a
                         signed type overflows. Use unsigned.        */

1 << 31               /* UB — `1` is int                            */
1u << 31              /* fine — unsigned int                        */
1ull << 40            /* fine — need the wide literal for wide shifts */

uint64_t mask = 1 << bit;     /* BUG if bit >= 32: the shift happens
                                 in int, THEN widens                */
uint64_t mask = 1ULL << bit;  /* correct                            */

/* Right shift of a NEGATIVE signed value is implementation-defined:
   arithmetic (sign-extending) on every real compiler, but not
   guaranteed. Use unsigned types for bit work — always.            */
```

The rule that avoids all of it: **do bit manipulation in unsigned types, with suffixed literals** (`1u`, `1ul`, `1ull`) wide enough for the shift.

```
/* the standard field extract/insert idioms */
#define FIELD_MASK(hi, lo)   (((1ull << ((hi) - (lo) + 1)) - 1) << (lo))
#define FIELD_GET(reg, hi, lo)  (((reg) & FIELD_MASK(hi, lo)) >> (lo))
#define FIELD_SET(reg, hi, lo, v) \
        (((reg) & ~FIELD_MASK(hi, lo)) | (((uint64_t)(v) << (lo)) & FIELD_MASK(hi, lo)))

/* Linux provides these properly, with compile-time checking: */
#include <linux/bitfield.h>
#define CTRL_SPEED  GENMASK(5, 2)
val = FIELD_GET(CTRL_SPEED, reg);
reg = (reg & ~CTRL_SPEED) | FIELD_PREP(CTRL_SPEED, 3);
```

Logical versus bitwise

```
/* && and & are NOT interchangeable, and the difference bites. */

if (p && p->valid)     /* safe: p->valid only evaluated if p is non-NULL */
if (p &  p->valid)      /* CRASH: both sides always evaluated;
                              and it computes a bitwise AND of a pointer   */

/* Type of the result differs too */
!x           /* int: 0 or 1                     */
~x           /* same type as promoted x         */
a && b       /* int: 0 or 1                     */
a &  b       /* the common type of a and b      */

/* A subtle one: !! forces a value to exactly 0 or 1 */
#define BOOL(x)  (!!(x))
flags |= BOOL(cond) << FLAG_SHIFT;    /* without !!, a value of 6 would
                                         shift into the wrong bits      */
```

Division, modulo and negative numbers

```
/* Since C99, integer division truncates TOWARD ZERO, and
   the sign of % follows the DIVIDEND.                    */

 7 /  2  ==  3       7 %  2 ==  1
-7 /  2  == -3      -7 %  2 == -1     /* not +1 */
 7 / -2  == -3       7 % -2 ==  1
-7 / -2  ==  3      -7 % -2 == -1

/* Identity that always holds:  (a/b)*b + a%b == a  */

/* So the "is it even" test is fine, but a naive modulo bucket is not: */
if (x % 2 == 1)          /* FALSE for negative odd x, which gives -1 */
if (x % 2 != 0)          /* correct */

/* Mapping into 0..n-1 for possibly-negative x: */
int idx = ((x % n) + n) % n;

/* Division by zero is UNDEFINED, not a trap you can rely on.
   INT_MIN / -1 is also undefined - the result does not fit.  */
if (b == 0 || (a == INT_MIN && b == -1))
        return -EINVAL;
```

sizeof and friends

```
sizeof(type)      /* parentheses required for a type    */
sizeof expr       /* parentheses optional for an expression */

/* sizeof does NOT evaluate its operand (except for VLAs) */
int i = 0;
size_t n = sizeof(i++);       /* i is still 0 — this is a compile-time
                                 operation, and a common interview trap */

/* Result type is size_t: unsigned. Chapter 02 explains why that matters. */
if (sizeof(a) - sizeof(b) > 0)   /* ALWAYS true — unsigned never negative */

/* The array-length macro, and why it is not safe on a pointer */
#define ARRAY_SIZE(a)  (sizeof(a) / sizeof((a)[0]))

int a[10];
ARRAY_SIZE(a);                /* 10, correct */

void f(int a[10]) {
        ARRAY_SIZE(a);        /* WRONG: `a` is a POINTER here (chapter 05).
                                 Gives sizeof(int*)/sizeof(int) = 2       */
}

/* Linux's version fails to COMPILE on a pointer, which is far better: */
#define ARRAY_SIZE(arr) \
        (sizeof(arr) / sizeof((arr)[0]) + __must_be_array(arr))

/* C11 alignment operators */
_Alignof(double)              /* alignment requirement, in bytes */
_Alignas(64) char buf[128];   /* force alignment — cache lines, DMA */
```

Compound assignment and increment

```
/* Compound assignment evaluates the left operand ONCE. This matters
   when it has side effects or is expensive.                        */
a[f()] += 1;          /* f() called once            */
a[f()] = a[f()] + 1;  /* f() called twice, in unspecified order */

/* It also performs an implicit conversion back, which can hide narrowing */
uint8_t x = 200;
x += 100;             /* computed in int (300), truncated back to 44
                         — no warning without -Wconversion             */

/* Pre versus post increment */
int i = 5;
j = i++;      /* j = 5, i = 6 — yields the OLD value */
j = ++i;      /* j = 7, i = 7 — yields the NEW value */

/* In C, on scalars, there is no performance difference: the compiler
   discards the unused old value. The habit of preferring ++i comes
   from C++, where the operand may be a heavy iterator object.       */
```

How it shows up

Symptom	Cause
A flag test is always false	`x & MASK == 0` — missing parentheses

Symptom	Cause
A loop reads one character then stops	`while (c = getchar() != EOF)`
Pointer advances when you meant to increment a value	`*p++` versus `(*p)++`
A shift produces 0 or garbage for large bit numbers	`1 << n` with `n >= 32`; needs `1ull`
`ARRAY_SIZE` returns 2 inside a function	The array parameter decayed to a pointer
Modulo bucket index is negative	`%` follows the sign of the dividend
Expression gives different results at `-O0` and `-O2`	Undefined behaviour from unsequenced modification
A size comparison is always true	`sizeof` yields unsigned; the subtraction wrapped
Crash on a null check that looks correct	`&` used where `&&` was meant — no short circuit

Check yourself

- ✓ Explain the difference between precedence and evaluation order, with an example of each mattering.
- ✓ Parse `x & MASK == 0` and say what it actually computes.
- ✓ Name the four constructs where C guarantees left-to-right evaluation.
- ✓ Give three ways a shift expression can be undefined, and the rule that avoids all of them.
- ✓ Explain why `uint64_t m = 1 << 40;` is wrong.
- ✓ Predict `-7 % 2` and write a correct positive-modulo helper.
- ✓ Explain why `sizeof(i++)` leaves `i` unchanged.
- ✓ Say why `ARRAY_SIZE` fails on a function parameter and how Linux prevents it.

Pointers from first principles

A pointer is a value that designates an object. That is all. Every difficulty people have with pointers comes from confusing the pointer with the thing it points to, or from reading declarations in the wrong direction. Both are fixable in an afternoon.

The mental model

```
Memory is a numbered array of bytes. An object occupies a run of them.
A pointer is a value that identifies where an object begins, together
with a TYPE that says how to interpret what is there.

   address:   0x1000   0x1004   0x1008   0x100C
              +--------+--------+--------+--------+
   memory:    |   42   |  ...   |  0x1000|  ...   |
              +--------+--------+--------+--------+
                  ^                |
                  |                |
                  x                p        int *p = &x;

   x     the object.        value 42.
   &x    its address.        value 0x1000.
   p     a pointer object.   value 0x1000, stored at 0x1008.
   *p    the object at that address.   value 42.
   &p    the address of the pointer itself.  value 0x1008.

The TYPE matters as much as the address:
   int *p     -> "reading 4 bytes here gives an int"
   char *c    -> "reading 1 byte here gives a char"
   Two pointers can hold the same address and mean different things.
```

THE TWO OPERATORS, SAID PRECISELY

`&x` — "the address of x". Turns an object into a pointer value.
`*p` — "the object that p points to". Turns a pointer value back into an object.

They are inverses: `*&x` is `x`. Read `*` aloud as **"the thing at"**, never as "pointer" — that single habit clears up most confusion.

Reading declarations

C declaration syntax mirrors *use*, not structure. The reliable technique is the spiral rule: start at the identifier, then move right when you can, left when you must.

```
   int *p;              p is a pointer to int
   int **p;             p is a pointer to a pointer to int
   int p[10];           p is an array of 10 ints
   int *p[10];          p is an ARRAY of 10 pointers to int
   int (*p)[10];        p is a POINTER to an array of 10 ints
   int *f(void);        f is a function returning pointer to int
   int (*f)(void);      f is a POINTER to a function returning int
   int (*f[5])(void);   f is an array of 5 pointers to functions
                          returning int
   int *(*f)(int *);    f is a pointer to a function taking int*
                          and returning int*

Why the parentheses: [] and () bind TIGHTER than *.
   int *p[10]    ==  int *(p[10])    array of pointers
   int (*p)[10]                      pointer to array

When it gets hard, stop and use a typedef. Professional code does.

   typedef int (*handler_fn)(struct device *, void *);
   handler_fn table[16];        /* obviously an array of handlers */

   cdecl(1) or `cdecl explain ...` will read any declaration aloud.
```

Where `const` goes

```
/* Read RIGHT TO LEFT from the identifier. const applies to what is
   immediately to its LEFT — unless it is leftmost, then to the right. */

const int *p;          /* p: pointer to const int   */
int const *p;          /* identical to the above    */
int *const p;          /* p: const pointer to int   */
const int *const p;    /* const pointer to const int */

const int *p;          *p = 5;   /* ERROR: object is const   */
                       p = &y;   /* fine: pointer is mutable */

int *const p = &x;     *p = 5;   /* fine: object is mutable  */
                       p = &y;   /* ERROR: pointer is const  */

/* The east-const style makes the rule uniform, which is why many
   codebases now prefer it:
        int const *p;        rather than  const int *p;
   Either is fine — be consistent.                          */
```

Pointer arithmetic

> **ARITHMETIC IS IN UNITS OF THE POINTED-TO TYPE, NOT BYTES**
>
> `p + 1` advances by `sizeof(*p)` bytes. This is what makes `a[i]` and `*(a + i)` equivalent, and it is why `void *` arithmetic is not standard C.

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;              /* points at a[0] */

p + 1                    /* a[1] — address advances by 4 bytes  */
*(p + 2)                 /* 30                                   */
p[2]                     /* identical: p[2] is *(p + 2) BY DEFINITION */
2[p]                     /* also 30 — legal, and a good argument for
                            why you should never write it        */

int *q = &a[4];
q - p                    /* 4 — a ptrdiff_t, in ELEMENTS not bytes */
(char *)q - (char *)p    /* 16 — in bytes                          */

/* void* arithmetic is a GNU extension (treats it as char*). It is
   used throughout the Linux kernel but is not portable ISO C.     */
void *v = p;
v + 1;                   /* GCC: +1 byte. MSVC/strict ISO: error   */
```

What arithmetic is legal

```
   int a[5];
   +----+----+----+----+----+----+
   |a[0]|a[1]|a[2]|a[3]|a[4]| ** |
   +----+----+----+----+----+----+
     ^                         ^
    &a[0]                   a + 5  = "one past the end"

  LEGAL:   any pointer from &a[0] to a+5 inclusive
           comparing any two of them
           dereferencing anything EXCEPT a+5

  UNDEFINED:  computing a - 1     (before the start)
              computing a + 6     (past one-past-the-end)
              dereferencing a + 5
              comparing pointers into DIFFERENT objects with < >

  The one-past-the-end rule is what makes the standard loop idiom
  legal, and it is why `end` pointers work at all:

      for (int *p = a; p != a + 5; p++)
              use(*p);
```

```
/* This looks fine and is undefined behaviour: */
for (int *p = &a[4]; p >= a; p--)     /* the final p-- computes a-1 */
        use(*p);

/* correct backwards iteration */
for (int *p = a + 5; p-- != a; )
        use(*p);
/* or just use an index */
for (size_t i = 5; i-- > 0; )
        use(a[i]);
```

Null pointers

```
int *p = NULL;        /* NULL is ((void*)0) or 0, implementation's choice */
int *p = 0;           /* the null pointer CONSTANT — same thing           */
int *p = nullptr;     /* C23, and clearer                                 */

if (p)                /* idiomatic: tests against null       */
if (p != NULL)        /* identical, more explicit            */

/* Dereferencing NULL is UNDEFINED, not "crashes with a segfault".
   On a system with memory at address 0 — many microcontrollers, and
   the x86 real-mode interrupt vector table — it silently reads
   garbage or corrupts something important.                      */

/* And the optimiser exploits it. This is a real Linux kernel CVE
   pattern (CVE-2009-1897):                                      */
struct s *p = get();
int x = p->field;          /* dereference implies p is non-NULL... */
if (p == NULL)             /* ...so the compiler DELETES this check */
        return -EINVAL;

/* Always check BEFORE dereferencing. The order is not stylistic. */

/* NULL is not necessarily all-zero bits. memset(&p, 0, sizeof p)
   is not guaranteed to produce a null pointer, though it does on
   every platform you will meet. Use explicit initialisation.      */
```

void pointers and generic code

```
/* void* is "a pointer to something unspecified". It converts to and
   from any OBJECT pointer type without a cast.                     */

void *v = &x;            /* implicit, fine                */
int *p = v;              /* implicit in C (NOT in C++)     */
int *p = (int *)v;       /* explicit — required in C++, harmless in C */

*v;                      /* ERROR: cannot dereference, size unknown  */

/* This is the mechanism behind every generic C API: */
void *malloc(size_t);
void *memcpy(void *dst, const void *src, size_t n);
int qsort_cmp(const void *a, const void *b);

/* qsort's comparator: the standard idiom for recovering the type */
static int cmp_int(const void *a, const void *b)
{
        int x = *(const int *)a;
        int y = *(const int *)b;
        return (x > y) - (x < y);    /* avoids overflow of x - y */
}
qsort(arr, n, sizeof(int), cmp_int);

/* Do NOT cast the result of malloc in C.
      int *p = malloc(n * sizeof *p);        // correct, idiomatic
   The cast is required in C++, unnecessary in C, and historically
   it hid the missing-prototype bug. Note `sizeof *p` rather than
   `sizeof(int)` — it stays correct if the type of p changes.    */
```

> **VOID* DOES NOT MEAN "FUNCTION POINTER"**
> The standard guarantees conversion between `void *` and object pointers only. Converting a function pointer through `void *` is not guaranteed — on architectures where code and data addresses differ in width or format it genuinely breaks. POSIX requires it to work for `dlsym()`, which is why that API is a documented wart. Keep function pointers in function pointer types.

Pointers to pointers

```
/* Reason 1: modifying a caller's pointer */
void alloc_buf(char **out, size_t n)
{
        *out = malloc(n);        /* write through to the caller's pointer */
}
char *buf;
alloc_buf(&buf, 100);

/* C is strictly pass-by-value. To let a function change your pointer,
   you must pass a pointer TO it. There is no other mechanism.      */

/* Reason 2: arrays of pointers — argv is the canonical example */
int main(int argc, char **argv)
/*                 char *argv[]  — identical in a parameter list */
{
        for (int i = 0; i < argc; i++)
                puts(argv[i]);
}

/* Reason 3: the linked-list deletion trick that removes all the
   special cases. Worth studying until it is obvious.             */
void remove_if(struct node **head, int target)
{
        struct node **pp = head;          /* pointer to the link itself */
        while (*pp) {
                struct node *entry = *pp;
                if (entry->value == target) {
                        *pp = entry->next;    /* unlink, whatever pp points to */
                        free(entry);
                } else {
                        pp = &entry->next;
                }
        }
}
/* No head-versus-middle special case, no `prev` pointer, no NULL
   checks for the previous node. Linus Torvalds has cited this as
   the difference between understanding pointers and not.          */
```

The four ways pointers go wrong

Fault	What it is	Detection
Dangling	Points to storage whose lifetime ended	ASan, Valgrind, KASAN
Wild	Uninitialised — contains garbage	MSan, `-Wuninitialized`, always initialise
Out of bounds	Past the end of the object	ASan, `-fsanitize=bounds`
Type-punned illegally	Accessed through an incompatible type	`-Wstrict-aliasing`, ch. 15

```
/* Dangling: returning a pointer to an automatic object */
char *bad(void)
{
        char buf[64];
        sprintf(buf, "hello");
        return buf;          /* buf's lifetime ends HERE */
}                            /* -Wreturn-local-addr catches this one */

/* Dangling: use after free */
free(p);
p->field = 1;                /* undefined; may silently work for a while */
free(p);                     /* double free: heap corruption            */

/* The discipline that prevents it: */
free(p);
p = NULL;                    /* now a later use crashes loudly and
                                a second free() is a no-op            */

/* Dangling: a pointer into a container that reallocated */
int *p = &vec->data[0];
vec_push(vec, x);            /* may realloc and move the buffer */
*p = 5;                      /* dangling - a very common real bug */

/* Wild: uninitialised */
int *p;                      /* contains whatever was on the stack */
*p = 5;                      /* undefined                          */
int *p = NULL;               /* at least it will fault predictably  */
```

restrict: promising no aliasing

```
/* Without restrict, the compiler must assume dst and src may overlap,
   so it cannot keep values in registers across the loop.          */
void copy(int *dst, const int *src, size_t n)
{
        for (size_t i = 0; i < n; i++)
                dst[i] = src[i];      /* must reload src[i] each time */
}

/* restrict promises: within this scope, the object accessed through
   this pointer is not accessed through any other pointer.          */
void copy(int *restrict dst, const int *restrict src, size_t n)
{
        for (size_t i = 0; i < n; i++)
                dst[i] = src[i];      /* now vectorisable */
}

/* This is why memcpy() and memmove() are two functions:
       memcpy  - arguments are restrict; overlapping is UNDEFINED
       memmove - handles overlap correctly, and is slower for it

   Breaking a restrict promise is undefined behaviour with no
   diagnostic. Only use it where you genuinely control both
   pointers. See chapter 10.                                      */
```

Finding pointer bugs

```
# AddressSanitizer — use-after-free, out-of-bounds, double free.
# Roughly 2x slowdown; run your whole test suite under it.
gcc -fsanitize=address -fno-omit-frame-pointer -g prog.c

# MemorySanitizer — reads of uninitialised memory (Clang only)
clang -fsanitize=memory -fno-omit-frame-pointer -g prog.c

# UndefinedBehaviorSanitizer — null deref, bad shifts, overflow
gcc -fsanitize=undefined -g prog.c

# Valgrind — no rebuild required, but ~20x slower
valgrind --leak-check=full --track-origins=yes ./prog

# Static analysis, free and fast
gcc -fanalyzer -Wall -Wextra prog.c
clang --analyze prog.c

# In the kernel: KASAN, KMSAN, and CONFIG_DEBUG_OBJECTS
```

How it shows up

Symptom	Cause
Crash far from the faulty code	Heap corruption from an earlier out-of-bounds write
Works in debug, crashes in release	Uninitialised pointer that happened to hold a valid address
Function returns garbage strings	Returned a pointer to a local buffer
Value changes without being assigned	Another pointer aliases it; or a dangling write
A null check is optimised away	You dereferenced before checking
Iterator invalidated after an append	Container reallocated; the old pointer dangles
Pointer arithmetic advances by the wrong amount	Arithmetic is in elements; check the pointer's type
`void *` arithmetic fails to compile	GNU extension; cast to `char *` for portability
Second `free()` corrupts the heap	Double free; set the pointer to NULL after freeing

Check yourself

- ✓ Explain `&` and `*` in one sentence each and say why they are inverses.
- ✓ Read aloud: `int *(*f[5])(char *)`.
- ✓ Give the difference between `const int *p` and `int *const p`.
- ✓ Explain why `p + 1` may advance by 8 bytes, and what determines it.
- ✓ State the one-past-the-end rule and show a loop that violates it.
- ✓ Explain why a null check placed after a dereference can be deleted.
- ✓ Write the pointer-to-pointer linked-list removal and say what special cases it eliminates.
- ✓ Explain what `restrict` promises and why `memcpy` and `memmove` both exist.

Arrays, decay and pointer arithmetic

"Arrays are pointers" is the most repeated falsehood in C teaching. Arrays and pointers are different types with different sizes and different semantics. They are connected by one specific rule — decay — and understanding exactly where it applies and where it does not resolves nearly every array confusion.

Arrays are not pointers

```
int a[10];
int *p = a;

sizeof(a)      /* 40 — the whole array          */
sizeof(p)      /* 8  — one pointer              */

&a             /* type int(*)[10] — pointer to array of 10 int */
&p             /* type int**                                   */

&a + 1         /* advances 40 bytes — past the whole array     */
&p + 1         /* advances 8 bytes                             */

a = p;         /* ERROR: an array is not assignable            */
p = a;         /* fine: decay                                   */
```

THE DECAY RULE, STATED EXACTLY

In most expression contexts, an expression of type "array of T" is converted to "pointer to T", pointing at the first element.

The four exceptions — the only places an array stays an array:

1. Operand of `sizeof`
2. Operand of `&` (address-of)
3. Operand of `_Alignof`
4. A string literal used to initialise a character array

Everywhere else — assignment, arithmetic, comparison, function arguments — the array decays.

```
char s[] = "hello";      /* exception 4: initialises the array,
                            so s is 6 bytes and MUTABLE          */
char *t  = "hello";      /* decay: t points at a string LITERAL,
                            which you must not modify            */

s[0] = 'H';              /* fine                                 */
t[0] = 'H';              /* UNDEFINED — literals may be in ROM,
                            and on many targets they are         */

/* Declare pointers to literals as const to make the compiler help: */
const char *t = "hello";
t[0] = 'H';              /* now a compile error, as it should be  */
```

Array parameters are a lie

```
These four declarations are IDENTICAL. The compiler rewrites all of
them to the last one.

    void f(int a[10]);
    void f(int a[100]);
    void f(int a[]);
    void f(int *a);        <- what the compiler actually sees

Consequences you must internalise:

  · sizeof(a) inside f is sizeof(int*), NOT the array size
  · the bound in the declaration is documentation, nothing more
  · the compiler will NOT check that you passed 10 elements
  · you must pass the length separately. Always.
```

```
/* the bug this causes, constantly */
void process(int a[10])
{
        size_t n = sizeof(a) / sizeof(a[0]);   /* 8/4 = 2, not 10 */
        for (size_t i = 0; i < n; i++)          /* processes 2 elements */
                ...
}
/* -Wsizeof-pointer-div warns about exactly this. Enable it. */

/* the correct signature */
void process(int *a, size_t n);
void process(size_t n, int a[n]);    /* C99: documents intent, and
                                        some compilers can check it */

/* C99 static array parameters — a promise of a minimum size.
   GCC and Clang will warn if you pass fewer or pass NULL.        */
void process(int a[static 10]);      /* "at least 10 elements"    */
```

Multidimensional arrays

```
C has no 2-D arrays. It has arrays OF arrays, stored contiguously
in row-major order.

    int m[3][4];

    memory layout, 48 bytes, one contiguous block:
    +--------------+--------------+--------------+
    | m[0][0..3]   | m[1][0..3]   | m[2][0..3]   |
    +--------------+--------------+--------------+
     ^              ^
     m[0]           m[1]        each is an int[4]

    m           type int(*)[4] after decay — pointer to a ROW
    m[0]        type int[4], decays to int*
    m[0][0]     type int
    *(*(m+1)+2) == m[1][2]

    sizeof(m)      48    the whole thing
    sizeof(m[0])   16    one row
    sizeof(m[0][0]) 4    one element

  The address computation the compiler emits for m[i][j]:
      base + (i * 4 + j) * sizeof(int)
  The COLUMN COUNT is baked into the type. That is why it cannot be
  omitted in a parameter declaration.
```

```
/* passing a 2-D array: all but the FIRST bound are required */
void f(int m[3][4]);      /* rewritten to int (*m)[4]  */
void f(int m[][4]);       /* same                      */
void f(int (*m)[4]);      /* what it really is         */
void f(int **m);          /* WRONG — a different type entirely */

/* C99 VLA parameters make it general and self-documenting */
void f(size_t rows, size_t cols, int m[rows][cols])
{
        m[i][j] = 0;      /* indexing works naturally */
}

/* the portable pre-C99 alternative: flatten it yourself */
void f(int *m, size_t rows, size_t cols)
{
        m[i * cols + j] = 0;
}
```

> **`INT **` IS NOT A 2-D ARRAY**
>
> An `int[3][4]` is one contiguous 48-byte block. An `int **` is a pointer to an array of pointers, each pointing to a separately allocated row — four allocations, scattered in memory, and an extra dereference per access. They are incompatible types and behave differently for cache locality (CA ch. 13). Passing one where the other is expected is a type error that a cast will silence and then crash.

Allocating arrays

```
/* 1-D: straightforward. Note `sizeof *a` rather than `sizeof(int)`. */
int *a = malloc(n * sizeof *a);
if (!a) return -ENOMEM;

/* Guard the multiplication — n * size can overflow (chapter 02) */
if (n > SIZE_MAX / sizeof *a) return -EOVERFLOW;
int *a = malloc(n * sizeof *a);
/* or */
int *a = calloc(n, sizeof *a);     /* calloc checks internally, and zeroes */

/* 2-D contiguous — ONE allocation, best locality, index arithmetic */
int (*m)[COLS] = malloc(rows * sizeof *m);
m[i][j] = 0;                        /* natural syntax, contiguous storage */
free(m);

/* 2-D "jagged" — N+1 allocations, worse locality, but rows may differ
   in length and can be reallocated independently                    */
int **m = malloc(rows * sizeof *m);
for (size_t i = 0; i < rows; i++)
        m[i] = malloc(cols * sizeof **m);
/* ...and you must free every row before freeing m */

/* Flat with manual indexing — one allocation, works pre-C99,
   and the shape is a runtime value                                  */
int *m = malloc(rows * cols * sizeof *m);
#define M(i, j) m[(i) * cols + (j)]
```

Variable-length arrays

```
void f(size_t n)
{
        int a[n];          /* C99 VLA: size determined at runtime */
        ...
}                          /* automatically freed at scope exit    */

/* Attractive, and largely a trap in systems code: */
```

Problem	Detail
Stack overflow	A large or attacker-controlled `n` silently blows the stack. No error, no diagnostic — just corruption or a crash. Kernel stacks are 8–16 KB.
No failure path	Unlike `malloc`, there is no way to detect that allocation failed
Optional	Made optional in C11; MSVC has never supported them
Hidden cost	Prevents some optimisations; the frame size becomes dynamic
Banned	The Linux kernel removed all VLAs in 2018 and builds with `-Wvla`. MISRA C forbids them.

```
/* Use them only for VLA PARAMETERS, which are pure documentation
   and generate no allocation:                                  */
void matmul(size_t n, size_t m, double a[n][m]);   /* good use     */

/* Otherwise: a fixed bound with a check, or heap allocation */
if (n > MAX_ITEMS) return -EINVAL;
int a[MAX_ITEMS];
/* or */
int *a = malloc(n * sizeof *a);
```

Flexible array members

```
/* The idiomatic way to allocate a header plus a variable-length
   payload in ONE allocation. C99 standardised it.              */
struct packet {
        uint32_t len;
        uint8_t  type;
        uint8_t  data[];        /* flexible array member - must be LAST,
                                   and cannot be the only member       */
};

struct packet *p = malloc(sizeof *p + payload_len);
p->len = payload_len;
memcpy(p->data, payload, payload_len);
/* one allocation, one free, contiguous, cache-friendly */

/* sizeof(struct packet) does NOT include data[] - it is 8 here.
   Guard the addition against overflow:                         */
if (payload_len > SIZE_MAX - sizeof *p) return -EOVERFLOW;

/* Linux provides helpers that do this correctly: */
p = kmalloc(struct_size(p, data, payload_len), GFP_KERNEL);

/* The old pre-C99 hack you will still meet, and should not copy: */
uint8_t data[1];        /* "struct hack" - technically UB to index past [0] */
uint8_t data[0];        /* GNU extension, same idea                          */
```

Bounds are never checked

```
int a[10];
a[10] = 0;        /* one past the end - undefined, and usually silent */
a[-1] = 0;        /* also undefined                                    */
a[1000000] = 0;   /* also undefined; may or may not fault              */

/* C performs NO bounds checking, ever. `a[i]` is defined as
   `*(a + i)` and that is all the compiler does.

   This is the root of the buffer overflow, which remains one of the
   most exploited vulnerability classes in existence - because on the
   stack, writing past an array reaches the saved return address.   */
```

```
Stack layout during a classic overflow

   higher addresses
   +---------------------+
   | saved return address|  <-- overwrite this and you control the PC
   +---------------------+
   | saved frame pointer |
   +---------------------+
   | char buf[64]        |  <-- gets(buf) writes as much as it is given
   +---------------------+
   lower addresses

Mitigations, none of which are a substitute for correct code:

  stack canaries    -fstack-protector-strong   detect on return
  non-executable stack                          make the payload harder
  ASLR                                          make addresses unknown
  FORTIFY_SOURCE    -D_FORTIFY_SOURCE=2        compile/run-time checks
  ASan              -fsanitize=address          catch it in testing
```

```
/* the discipline */
#define ARRAY_SIZE(a)  (sizeof(a) / sizeof((a)[0]))    /* on real arrays only */

for (size_t i = 0; i < ARRAY_SIZE(a); i++)   /* never a hardcoded bound */
        use(a[i]);

/* validate every index that came from outside */
if (idx >= ARRAY_SIZE(table))
        return -EINVAL;
return table[idx];

/* pass length with pointer, always */
void process(const uint8_t *data, size_t len);

/* and in security-sensitive code, mask rather than branch, so the
   check survives speculation (see CA chapter 12): */
idx = array_index_nospec(idx, ARRAY_SIZE(table));
```

Arrays of characters versus strings

```
/* A "string" in C is a convention, not a type: a char array whose
   contents end with '\0'.                                        */

char a[6] = "hello";      /* 5 chars + terminator - exactly fits  */
char b[5] = "hello";      /* legal in C! No terminator. NOT a string.
                             Passing b to strlen() runs off the end. */
char c[]  = "hello";      /* size 6, deduced - prefer this        */

sizeof("hello")   /* 6 - includes the terminator */
strlen("hello")   /* 5 - does not                */

/* This distinction is the source of a large fraction of C's
   buffer bugs. Chapter 06 covers string handling in full.     */
```

How it shows up

Symptom	Cause
`ARRAY_SIZE` returns 2 or 8 inside a function	The parameter decayed to a pointer
Writing to a string literal crashes	`char *s = "..."`; literals are read-only
Passing a 2-D array to `int **` gives garbage	Incompatible types; a cast hid the error
Stack smashing detected, at a random later point	Buffer overflow caught by the canary on return
Crash only with large input	VLA stack overflow, or a fixed buffer overrun
Struct with trailing array reads garbage	`sizeof` did not include the flexible member; allocation too small
`strlen` returns an enormous number	Missing null terminator; the array was exactly the string length
Off-by-one that only shows under ASan	Reading `a[n]`; the byte was harmlessly padding in release builds
Backwards loop crashes on the last iteration	Computing a pointer before the start of the array

Check yourself

- ✓ Give four differences between `int a[10]` and `int *p`.
- ✓ State the decay rule and its four exceptions.
- ✓ Explain why `void f(int a[10])` does not check the size, and what to write instead.
- ✓ Draw the memory layout of `int m[3][4]` and explain why the column count is part of the type.
- ✓ Give three ways to allocate a 2-D array and the trade-off of each.
- ✓ List three reasons the Linux kernel banned VLAs.
- ✓ Write a struct with a flexible array member and allocate it safely.
- ✓ Explain how an array overflow reaches the return address, and name three mitigations.

Strings and buffer safety

C has no string type. It has a convention — a byte array ending in a zero — and a library of functions that trust you to have got the sizes right. This chapter covers the convention, why the classic functions are dangerous, and what to use instead.

The convention

```
A C string is a contiguous sequence of characters terminated by '\0'.

   char s[] = "hello";

   index:   0    1    2    3    4    5
          +----+----+----+----+----+----+
          | h  | e  | l  | l  | o  | \0 |
          +----+----+----+----+----+----+
   size:  6 bytes.   length: 5 characters.

Every consequence of C string handling follows from this:

  · LENGTH IS NOT STORED. strlen() walks the array counting bytes,
    so it is O(n) and it must find a terminator or run off the end.
  · The terminator costs one byte that is easy to forget.
  · A string cannot contain a zero byte. Binary data needs a
    pointer + length, never a "string".
  · Nothing knows the CAPACITY of the buffer but you.
```

```
char a[6] = "hello";     /* exactly fits: 5 + terminator          */
char b[5] = "hello";     /* LEGAL C: no terminator. Not a string.  */
char c[]  = "hello";     /* size 6, deduced — prefer this          */
char d[10] = "hi";       /* remaining bytes zero-filled            */

sizeof("hello")          /* 6 — a compile-time array size         */
strlen("hello")          /* 5 — a runtime walk                    */

/* strlen on b runs past the end of the array: undefined behaviour,
   and in practice it returns whatever happens to follow in memory. */
```

The functions that should not exist

Function	Problem	Verdict
`gets()`	No way to specify a size. Unconditionally exploitable.	**Removed from C11.** Never use it.
`strcpy()`	Writes until the source terminator; no destination bound	Only when you have proven the size
`strcat()`	Same, plus it must walk the destination first	Same
`sprintf()`	Unbounded output	Use `snprintf()`
`strncpy()`	**Does not always terminate**, and zero-pads the whole buffer	Almost never what you want — see below
`strncat()`	The size argument is the *remaining space*, not the buffer size — universally misused	Avoid
`strtok()`	Modifies its input and holds global state; not reentrant	`strtok_r()`, or write your own

Function	Problem	Verdict
`atoi()`	No error reporting at all; UB on overflow	`strtol()` with full checking

STRNCPY IS NOT A SAFE STRCPY

```
char dst[8];
strncpy(dst, "hello world", sizeof dst);
/* dst now contains: h e l l o _ w o
   NO terminator. Any subsequent strlen(dst) or printf("%s") runs
   off the end of the buffer.                                      */

strncpy(dst, "hi", sizeof dst);
/* dst contains: h i \0 \0 \0 \0 \0 \0
   It zero-filled ALL remaining bytes — O(n) regardless of input.  */

/* strncpy was designed in 1979 for fixed-width, non-terminated
   records in early UNIX directory entries. It was never a
   "safe strcpy" and using it as one creates bugs.                */

/* If you must use it, terminate explicitly: */
strncpy(dst, src, sizeof dst - 1);
dst[sizeof dst - 1] = '\0';
```

What to use instead

snprintf — the portable workhorse

```
char buf[64];
int n = snprintf(buf, sizeof buf, "%s: %d", name, value);

/* snprintf ALWAYS terminates (when size > 0), and returns the length
   it WOULD have written. That return value is how you detect
   truncation — and it is why the return is not the bytes written.  */

if (n < 0)
        return -EIO;                    /* encoding error          */
if ((size_t)n >= sizeof buf)
        return -ENAMETOOLONG;           /* truncated               */

/* The classic bug: using the return value as a length blindly */
char buf[16];
int n = snprintf(buf, sizeof buf, "%s", long_string);
memcpy(other, buf, n);          /* n may EXCEED sizeof buf — overflow */

/* correct */
size_t len = (n < 0) ? 0 : ((size_t)n < sizeof buf ? (size_t)n : sizeof buf - 1);
```

strlcpy and strlcat — when available

```
/* From OpenBSD; in glibc since 2.38, and in the Linux kernel.
   They always terminate and return the length they TRIED to create,
   so truncation is detectable with one comparison.               */

size_t r = strlcpy(dst, src, sizeof dst);
if (r >= sizeof dst)
        return -ENAMETOOLONG;           /* truncated */

/* strscpy() is the kernel's preferred variant: it returns the
   number of bytes copied, or -E2BIG on truncation — an error code
   rather than a length you must compare.                         */
ssize_t r = strscpy(dst, src, sizeof dst);
if (r < 0)
        return r;
```

A comparison you can act on

Function	Always terminates?	Detects truncation?	Cost
`strcpy`	Yes	No bound at all	O(len)
`strncpy`	**No**	No	O(size) — always pads
`snprintf`	Yes	Yes, via return	Format parsing overhead
`strlcpy`	Yes	Yes, via return	O(src len) — reads all of src
`strscpy` (Linux)	Yes	Yes, returns `-E2BIG`	O(min(src, size))
`memcpy` + manual	You do it	You do it	Fastest, most error-prone

Parsing numbers correctly

```
/* atoi has no error reporting whatsoever and is UB on overflow. */
int n = atoi(s);          /* returns 0 for "abc" AND for "0" */

/* strtol, used properly — this is the full, correct pattern */
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int parse_int(const char *s, int *out)
{
        char *end;
        errno = 0;                      /* MUST clear it first */

        long v = strtol(s, &end, 10);    /* explicit base: 10, never 0
                                            unless you want "0x" and
                                            octal to be accepted      */

        if (end == s)          return -EINVAL;   /* no digits at all  */
        if (*end != '\0')      return -EINVAL;   /* trailing garbage  */
        if (errno == ERANGE)   return -ERANGE;   /* out of long range */
        if (v < INT_MIN || v > INT_MAX) return -ERANGE;  /* out of int */

        *out = (int)v;
        return 0;
}

/* Note the four distinct failure modes. Any parser that does not
   distinguish them will accept input it should reject.           */

/* Kernel equivalents, which get this right for you: */
kstrtoint(s, 10, &val);
kstrtoul(s, 0, &val);
```

Bytes, characters and encodings

```
A `char` is a BYTE. It is not a character.

  UTF-8 encodes a code point in 1 to 4 bytes:

    'A'   U+0041   -> 0x41                    1 byte
    'é'   U+00E9   -> 0xC3 0xA9               2 bytes
    '中'  U+4E2D   -> 0xE4 0xB8 0xAD          3 bytes
    '😜'  U+1F600  -> 0xF0 0x9F 0x98 0x80      4 bytes

  So for a UTF-8 string:
     strlen()  gives BYTES, not characters
     s[i]      indexes a byte, possibly mid-character
     toupper() on a byte of a multi-byte sequence is meaningless
     truncating at a byte boundary can produce invalid UTF-8

  What is still safe on UTF-8 without decoding:
     · comparing for equality (memcmp)
     · searching for an ASCII substring (UTF-8 is self-synchronising:
       a multi-byte sequence never contains an ASCII byte)
     · concatenation
     · sorting by byte order == sorting by code point order

  What is NOT safe: length in characters, indexing, case conversion,
  truncation, reversing, normalisation-sensitive comparison.
```

```
/* Practical advice for systems code:
     - treat text as opaque UTF-8 bytes and pass it through
     - validate UTF-8 at the boundary if you must interpret it
     - never truncate a UTF-8 string at an arbitrary byte offset
     - do not use wchar_t: it is 16-bit on Windows and 32-bit on
       Linux, which makes it useless for portable code
     - if you need real Unicode handling, use ICU or utf8proc     */

/* truncating UTF-8 safely: back up to a leading byte */
size_t utf8_truncate(const char *s, size_t max)
{
        size_t n = strnlen(s, max);
        while (n > 0 && (s[n] & 0xC0) == 0x80)   /* 10xxxxxx = continuation */
                n--;
        return n;
}
```

The ctype trap

```
#include <ctype.h>

char c = get_byte();
if (isalpha(c))          /* UNDEFINED if c is negative */

/* The <ctype.h> functions take an int that must be representable as
   `unsigned char` or equal EOF. On platforms where plain char is
   signed (x86 Linux, Windows), byte 0xE9 becomes -23, and the
   implementation indexes a lookup table out of bounds.           */

if (isalpha((unsigned char)c))     /* correct — always cast */

/* Also: these are locale-dependent. In systems code that parses
   protocols, write explicit checks instead:                      */
static inline bool is_digit(char c)  { return c >= '0' && c <= '9'; }
static inline bool is_alpha(char c)  { return (c|32) >= 'a' && (c|32) <= 'z'; }
```

Handling secrets in buffers

```
/* This does not work: */
void check_password(const char *input)
{
        char key[32];
        derive_key(key, input);
        use(key);
        memset(key, 0, sizeof key);   /* DEAD STORE — the compiler
                                         removes it, because key is
                                         never read again (chapter 01) */
}

/* Portable-ish fixes, in order of preference: */

memset_s(key, sizeof key, 0, sizeof key);      /* C11 Annex K, optional */
explicit_bzero(key, sizeof key);               /* BSD, glibc 2.25+      */
SecureZeroMemory(key, sizeof key);             /* Windows               */
memzero_explicit(key, sizeof key);             /* Linux kernel          */

/* If you have none of them, force it through a volatile pointer: */
static void *(*const volatile memset_ptr)(void *, int, size_t) = memset;
memset_ptr(key, 0, sizeof key);

/* And compare secrets in CONSTANT TIME — never with memcmp, which
   returns as soon as it finds a difference and thereby leaks how
   many bytes matched. See the cryptography track.               */
static int ct_memcmp(const void *a, const void *b, size_t n)
{
        const unsigned char *x = a, *y = b;
        unsigned char diff = 0;
        for (size_t i = 0; i < n; i++)
                diff |= x[i] ^ y[i];       /* no early exit */
        return diff;                        /* 0 == equal    */
}
```

Compiler and runtime hardening

```
# FORTIFY_SOURCE: replaces str/mem functions with checked versions
# when the destination size is known at compile time. Nearly free.
gcc -O2 -D_FORTIFY_SOURCE=3 -Wall -Wextra

# stack canaries
gcc -fstack-protector-strong

# warn about format-string problems, including non-literal formats
gcc -Wformat=2 -Wformat-security

# catch overflows in testing
gcc -fsanitize=address -g

# static analysis
gcc -fanalyzer                # GCC 10+, finds many string bugs
```

FORMAT STRING VULNERABILITIES

```
printf(user_input);              /* CATASTROPHIC */
/* If user_input contains %s the program reads an arbitrary pointer;
   %n WRITES to memory. This is a full compromise, not a crash.   */

printf("%s", user_input);        /* correct — the format is a literal */

/* -Wformat-security flags any call where the format is not a
   literal. Treat it as an error, not a warning.                  */
```

How it shows up

Symptom	Cause
`strlen` returns a huge number	Missing terminator — often from `strncpy`
Output has trailing garbage	Buffer not terminated; `%s` read past the end
"stack smashing detected" on return	Canary caught a buffer overflow
Truncation goes unnoticed until data is corrupt downstream	Return value of `snprintf` / `strlcpy` ignored
Crash copying a string that "obviously fits"	Forgot the terminator byte in the size calculation
Non-ASCII input crashes or misbehaves	`<ctype.h>` called with a negative `char`
String length differs from displayed characters	UTF-8 — bytes, not characters
A key remains in memory after being zeroed	Dead store elimination removed the `memset`
Program leaks memory contents when given `%x` input	Format string vulnerability
`atoi("abc")` silently yields 0	No error reporting; use `strtol`

Check yourself

- ✓ Explain why `strlen` is O(n) and what that implies for loops that call it.
- ✓ Give two distinct reasons `strncpy` is not a safe `strcpy`.
- ✓ Explain what `snprintf` returns and how to detect truncation correctly.

- ✓ Write a fully correct integer parser using `strtol`, naming all four failure modes.
- ✓ Say what remains safe to do with UTF-8 without decoding it, and what does not.
- ✓ Explain the `<ctype.h>` negative-`char` bug.
- ✓ Explain why `memset` on a key buffer may be deleted, and give two fixes.
- ✓ Show a format string vulnerability and its one-character fix.

Structs, unions, padding and alignment

Structs are how C describes memory layout — for data structures, for hardware register blocks, and for wire protocols. The compiler inserts invisible padding to satisfy alignment, and if you do not know where it goes, you will write structs that are twice the size they need to be and protocol parsers that silently corrupt data.

Alignment

THE REQUIREMENT

Every type has an **alignment**: a power of two that its address must be a multiple of. Typically alignment equals size for scalar types — a 4-byte `int` must sit at an address divisible by 4.

The hardware imposes this. On ARM, an unaligned access to *Device* memory always faults; on Normal memory it usually works but may be slower or split into two bus transactions. On some architectures it faults everywhere. In C, forming or dereferencing a misaligned pointer is undefined behaviour regardless of what the hardware would do.

```
#include <stdalign.h>

alignof(char)      /* 1  */
alignof(short)     /* 2  */
alignof(int)       /* 4  */
alignof(double)    /* 8  */
alignof(void *)    /* 8 on 64-bit */
alignof(max_align_t)  /* the strictest fundamental alignment: 16 typically */

/* A struct's alignment is the LARGEST alignment among its members.
   Its size is always a multiple of its alignment.               */
```

Padding, member by member

```
struct bad {
        char  a;      /* offset 0                  */
                      /* offsets 1-3: PADDING      */
        int   b;      /* offset 4  (must be /4)    */
        char  c;      /* offset 8                  */
                      /* offsets 9-11: TAIL PADDING */
};                    /* sizeof == 12, alignof == 4 */

  byte:   0    1    2    3    4    5    6    7    8    9   10   11
        +----+----+----+----+----+----+----+----+----+----+----+----+
        | a  | ..pad.. |         b         | c  |    ..pad..       |
        +----+----+----+----+----+----+----+----+----+----+----+----+

Two kinds of padding, for two different reasons:

  INTERNAL padding aligns the NEXT member.
  TAIL padding makes sizeof a multiple of alignof, so that in an
  ARRAY of this struct, every element is correctly aligned.

Reorder largest-to-smallest and both shrink:

struct good {
        int   b;      /* offset 0 */
        char  a;      /* offset 4 */
        char  c;      /* offset 5 */
                      /* offsets 6-7: tail padding */
};                    /* sizeof == 8 — 33% smaller */
```

```
/* Inspect any struct's real layout — do not guess */
#include <stddef.h>
printf("size=%zu align=%zu b@%zu c@%zu\n",
       sizeof(struct bad), alignof(struct bad),
       offsetof(struct bad, b), offsetof(struct bad, c));

/* Or have the compiler print it for you */
//   gcc -Wpadded              warns wherever padding is inserted
//   pahole ./binary           shows every struct's holes (from dwarves)
//   clang -Xclang -fdump-record-layouts
```

WHY LAYOUT MATTERS BEYOND SIZE

A struct that fits in one 64-byte cache line costs one miss; one that straddles two costs two (CA ch. 14). In the Linux kernel, shrinking a frequently allocated struct below a slab size class is a measurable performance win. `pahole` exists precisely because this is worth doing systematically — and it is a routine kind of upstream patch.

packed: powerful and dangerous

```
struct __attribute__((packed)) wire {
        uint8_t  type;
        uint32_t length;      /* now at offset 1 — MISALIGNED */
        uint16_t flags;
};                            /* sizeof == 7, alignof == 1     */

/* This removes all padding. It also creates a real hazard: */

struct wire *w = (struct wire *)buffer;
uint32_t len = w->length;    /* compiler knows this is packed, so it
                                emits byte loads + shifts. Correct,
                                but slower.                          */

uint32_t *p = &w->length;    /* -Waddress-of-packed-member:
                                a plain uint32_t* implies 4-byte
                                alignment, which this is NOT.
                                Dereferencing p is UB and FAULTS on
                                strict-alignment targets.            */
```

THE PACKED-STRUCT RULES

1. **Never take the address of a packed member** and store it in a normal pointer. Enable `-Waddress-of-packed-member` and treat it as an error.
2. **Never pass a packed member by reference** to a function expecting an aligned pointer.
3. **Packed does not fix endianness.** It controls padding only. A wire protocol still needs explicit byte-order conversion.
4. **Prefer explicit serialisation** for protocols. Packed structs work, but they trade a portability problem you can see for one you cannot.

```
/* The robust alternative: explicit serialisation. Slightly more
   code, and immune to padding, alignment AND endianness.        */

static size_t wire_parse(const uint8_t *b, size_t n, struct msg *out)
{
        if (n < 7) return 0;
        out->type   = b[0];
        out->length = ((uint32_t)b[1] << 24) | ((uint32_t)b[2] << 16) |
                      ((uint32_t)b[3] <<  8) |  (uint32_t)b[4];   /* big endian */
        out->flags  = ((uint16_t)b[5] <<  8) |  (uint16_t)b[6];
        return 7;
}

/* Or use the standard conversions when you do use a struct: */
#include <arpa/inet.h>          /* htonl, ntohl, htons, ntohs      */
#include <endian.h>             /* htobe32, le32toh, ... (glibc)   */
val = be32toh(w->length);       /* kernel: be32_to_cpu()           */
```

Structs for hardware registers

```
/* The idiomatic MMIO register block: every member volatile, gaps
   declared explicitly, and a static assertion on the layout.    */

struct uart_regs {
        volatile uint32_t DR;         /* 0x00 data                */
        volatile uint32_t RSR;        /* 0x04 receive status      */
        uint32_t          _pad0[4];   /* 0x08-0x14 reserved       */
        volatile uint32_t FR;         /* 0x18 flags               */
        uint32_t          _pad1;      /* 0x1C                     */
        volatile uint32_t IBRD;       /* 0x20 integer baud rate   */
};

_Static_assert(sizeof(struct uart_regs) == 0x24, "uart_regs layout");
_Static_assert(offsetof(struct uart_regs, FR) == 0x18, "FR offset");

#define UART0 ((struct uart_regs *)0x40000000u)

/* Why the static asserts: a silent padding change - from a member
   type edit, a compiler upgrade, or a different ABI - would shift
   every subsequent register. The assert turns a runtime mystery
   into a build failure.                                          */

/* Note: do NOT pack a register-block struct. The registers are
   naturally aligned by design; packing only invites the compiler
   to emit byte accesses, which most peripherals reject.        */
```

Unions

```
/* All members share the same storage, starting at offset 0.
   sizeof is the largest member; alignof is the strictest.     */

union value {
        uint32_t u;
        float    f;
        uint8_t  bytes[4];
};                              /* sizeof == 4 */

/* Legal use 1: saving space when only one member is live at a time,
   with a tag to say which. This is a "tagged union".            */
struct variant {
        enum { V_INT, V_FLOAT, V_PTR } kind;
        union {
                int64_t i;
                double  d;
                void   *p;
        } u;
};

/* Legal use 2: type punning. In C — unlike C++ — reading a member
   other than the one last written is DEFINED, since C99 TC3.    */
union { float f; uint32_t bits; } cvt = { .f = 1.0f };
printf("%08x\n", cvt.bits);        /* 3f800000 — well defined in C */

/* The alternative that is always correct in both languages: */
uint32_t bits;
memcpy(&bits, &f, sizeof bits);    /* compiles to zero instructions
                                      at -O2; the compiler recognises
                                      the idiom                     */

/* What is NOT legal: punning through incompatible POINTERS.
   That violates strict aliasing — see chapter 15.               */
uint32_t bits = *(uint32_t *)&f;   /* UB. Do not do this.          */

/* Anonymous unions and structs (C11) — very common in kernel code */
struct packet {
        uint8_t type;
        union {                        /* no member name needed */
                struct { uint16_t a, b; } pair;
                uint32_t raw;
        };
};
p.raw = 0;  p.pair.a = 1;             /* accessed directly */
```

Bitfields

```
struct flags {
        unsigned int enable   : 1;
        unsigned int mode     : 3;
        unsigned int priority : 4;
        unsigned int          : 0;    /* force to next storage unit */
        unsigned int count    : 16;
};
```

BITFIELDS ARE UNSUITABLE FOR HARDWARE AND WIRE FORMATS

Almost everything about them is implementation-defined:

- **Allocation order** within a storage unit — MSB-first or LSB-first — is up to the ABI, so the same struct describes different bits on big- and little-endian targets.
- **Straddling** a storage-unit boundary is implementation-defined.
- **The underlying type**: only `int`, `signed int`, `unsigned int` and `_Bool` are portable. Plain `int` bitfields may be signed or unsigned — always write `unsigned`.
- **Access is not atomic.** Reading or writing one bitfield may read-modify-write the whole storage unit, so two threads updating adjacent bitfields can lose an update — even though they touch "different variables". The C11 memory model bounds this, but it still surprises people.
- **You cannot take the address** of a bitfield.

For registers and protocols, use explicit masks and shifts. They are portable, greppable, and generate the same code.

```
/* Do this instead */
#define CTRL_ENABLE      (1u << 0)
#define CTRL_MODE_SHIFT  1
#define CTRL_MODE_MASK   (0x7u << CTRL_MODE_SHIFT)

reg = (reg & ~CTRL_MODE_MASK) | ((mode << CTRL_MODE_SHIFT) & CTRL_MODE_MASK);
reg |= CTRL_ENABLE;

/* Linux provides checked helpers */
#include <linux/bitfield.h>
#define CTRL_MODE  GENMASK(3, 1)
reg = (reg & ~CTRL_MODE) | FIELD_PREP(CTRL_MODE, mode);
mode = FIELD_GET(CTRL_MODE, reg);
```

Initialisation

```
struct point { int x, y, z; };

struct point a = {1, 2, 3};          /* positional               */
struct point b = {1};                /* y and z are ZERO, not garbage */
struct point c = {0};                /* whole struct zeroed      */
struct point d = { };                /* C23; GNU extension before */

/* Designated initialisers (C99) — use these. They survive member
   reordering, document intent, and zero everything unmentioned. */
struct point e = { .x = 1, .z = 3 };          /* y == 0 */

/* Essential for sparse structs, which is why kernel code is full
   of them: */
static const struct file_operations fops = {
        .owner   = THIS_MODULE,
        .open    = my_open,
        .read    = my_read,
        .release = my_release,
        /* every other member is NULL — safely and explicitly */
};

/* Compound literals (C99): an unnamed object you can take the
   address of. Lifetime is the enclosing block.               */
send(&(struct point){ .x = 1, .y = 2 });
configure((struct cfg){ .baud = 115200, .bits = 8 });
```

Structs cannot be compared

```
struct point a, b;
if (a == b)                    /* COMPILE ERROR — no == for structs */

if (memcmp(&a, &b, sizeof a) == 0)   /* compiles, and is WRONG */

/* memcmp compares the PADDING BYTES too, and padding contents are
   indeterminate. Two structs with identical members can compare
   unequal. Assignment copies padding unpredictably; a struct built
   field-by-field has different padding from one from an initialiser.

   Correct: compare member by member.                              */
static bool point_eq(const struct point *a, const struct point *b)
{
        return a->x == b->x && a->y == b->y && a->z == b->z;
}

/* The same reasoning applies to HASHING a struct and to sending one
   over a wire: padding bytes may leak stale stack contents — a real
   information-disclosure class in kernel ioctl handlers. Always
   memset the whole struct before filling it, when it crosses a
   trust boundary.                                                 */
struct info out;
memset(&out, 0, sizeof out);      /* not just = {0} on all paths */
out.a = ...;
copy_to_user(u, &out, sizeof out);
```

container_of

```
/* Given a pointer to a member, recover the enclosing struct.
   This is how intrusive data structures work throughout the kernel. */

#define container_of(ptr, type, member) \
        ((type *)((char *)(ptr) - offsetof(type, member)))

struct task {
        int              pid;
        struct list_head list;     /* embedded, not a pointer */
        char             name[16];
};

/* Given a struct list_head *node from a generic list walk: */
struct task *t = container_of(node, struct task, list);

/* Why this is better than a `void *data` pointer in the node:
     - no extra allocation per element
     - no extra indirection on access
     - one object can be on SEVERAL lists at once, each with its
       own embedded node
   The cost is that it is pointer arithmetic the compiler cannot
   type-check, which is why the kernel's version adds a
   __same_type() assertion.                                      */
```

How it shows up

Symptom	Cause
`sizeof` a struct is larger than the sum of its members	Padding — reorder members largest first
A wire protocol parses correctly on x86, wrongly on ARM	Endianness, or a bitfield's implementation-defined order
Fault when dereferencing a pointer to a packed member	Misaligned pointer; strict-alignment target
Register writes go to the wrong offsets after a compiler upgrade	Padding changed; add `_Static_assert` on offsets
Two identical-looking structs compare unequal	`memcmp` including indeterminate padding
Kernel leaks stack bytes to userspace	Padding in a struct copied out without being zeroed
Concurrent updates to adjacent bitfields lose one	Read-modify-write of the shared storage unit
Struct grew and throughput dropped	It now spans two cache lines
Uninitialised member despite an initialiser	Positional initialiser missing a member — use designated form

Check yourself

- ✓ Compute the size and layout of `struct { char a; int b; char c; }` and reorder it optimally.
- ✓ Explain the difference between internal and tail padding, and why tail padding must exist.
- ✓ Give three rules for using packed structs safely, and say what packed does *not* fix.
- ✓ Write a register-block struct with static assertions and explain what they protect against.
- ✓ Explain why type punning through a union is legal in C but a pointer cast is not.
- ✓ List four implementation-defined properties of bitfields.
- ✓ Explain why `memcmp` on structs is wrong, and what to do instead.

- ✓ Write `container_of` and explain why intrusive lists use it.

Bit manipulation

Embedded work is bit work. Every hardware register is a packed collection of fields, every protocol header is bit-aligned, and every flag set is a bitmask. This chapter is the complete idiom set, plus the traps that make bit code subtly wrong.

Ground rules

FOUR RULES THAT PREVENT MOST BIT BUGS

1. **Use unsigned types.** Signed shifts, signed overflow and sign extension are all undefined or implementation-defined. There is no upside.
2. **Suffix your literals.** `1u`, `1ul`, `1ull` — a bare `1` is an `int`, and `1 << 40` is undefined behaviour.
3. **Parenthesise.** Bitwise operators bind looser than comparison (ch. 03).
4. **Name your masks.** A bare `0x38` in a shift is unreviewable.

Single-bit operations

```
#define BIT(n)  (1u << (n))        /* for <32-bit fields          */
#define BIT_ULL(n) (1ull << (n))   /* when n may reach 32 or more */

x |=  BIT(n);                /* set        */
x &= ~BIT(n);                /* clear      */
x ^=  BIT(n);                /* toggle     */
if (x & BIT(n)) ...          /* test       */

/* conditionally set or clear without a branch */
x = (x & ~BIT(n)) | ((-(unsigned)cond) & BIT(n));
/* -(unsigned)cond is 0x00000000 or 0xFFFFFFFF — a mask from a bool */

/* the same idea, more readable */
if (cond) x |=  BIT(n);
else      x &= ~BIT(n);
```

THE SHIFT TRAPS, AGAIN

```
1 << 31            /* UB: shifting into the sign bit of an int */
1u << 31           /* fine                                     */
1 << 32            /* UB: n must be < width of the type        */
1ull << 40         /* fine                                     */

uint64_t m = 1 << bit;      /* BUG for bit >= 32: computed in int,
                               THEN widened                      */
uint64_t m = 1ull << bit;   /* correct                           */

x >> n             /* if x is signed and negative, the result is
                      implementation-defined (arithmetic shift in
                      practice, but do not rely on it)          */

/* And a shift by a variable that can equal the width: */
uint32_t mask = (1u << width) - 1;    /* UB when width == 32 */
uint32_t mask = width >= 32 ? ~0u : (1u << width) - 1;   /* safe */
uint32_t mask = (uint32_t)((1ull << width) - 1);          /* also safe */
```

Multi-bit fields

```
A 32-bit control register, as a TRM would document it:

  31    24 23  20 19        8 7    4 3  1  0
 +--------+------+-----------+------+----+--+
 | RESVD  | MODE |  DIVISOR  | PRIO |RSV |EN|
 +--------+------+-----------+------+----+--+

  EN       bit  0
  PRIO     bits 7:4
  DIVISOR  bits 19:8
  MODE     bits 23:20

Turn each field into a shift and a mask, once, at the top of the file.
```

```
#define CTRL_EN            BIT(0)
#define CTRL_PRIO_SHIFT    4
#define CTRL_PRIO_MASK     (0xFu   << CTRL_PRIO_SHIFT)
#define CTRL_DIV_SHIFT     8
#define CTRL_DIV_MASK      (0xFFFu << CTRL_DIV_SHIFT)
#define CTRL_MODE_SHIFT    20
#define CTRL_MODE_MASK     (0xFu   << CTRL_MODE_SHIFT)

/* extract */
unsigned prio = (reg & CTRL_PRIO_MASK) >> CTRL_PRIO_SHIFT;

/* insert - ALWAYS clear before OR-ing, and mask the value so an
   out-of-range input cannot corrupt neighbouring fields          */
reg = (reg & ~CTRL_PRIO_MASK) |
      ((prio << CTRL_PRIO_SHIFT) & CTRL_PRIO_MASK);

/* generic helpers, if you have no library */
#define FIELD_MASK(hi, lo)  (((1ull << ((hi) - (lo) + 1)) - 1) << (lo))
#define FIELD_GET(r, hi, lo)  (((r) & FIELD_MASK(hi, lo)) >> (lo))
#define FIELD_SET(r, hi, lo, v) \
        (((r) & ~FIELD_MASK(hi, lo)) | \
         (((uint64_t)(v) << (lo)) & FIELD_MASK(hi, lo)))
```

```
/* Linux's version, which you should copy the design of: the mask is
   the single source of truth and the shift is derived from it, so
   the two can never disagree.                                      */
#include <linux/bitfield.h>

#define CTRL_PRIO  GENMASK(7, 4)      /* 0x000000F0 */
#define CTRL_DIV   GENMASK(19, 8)
#define CTRL_MODE  GENMASK(23, 20)

prio = FIELD_GET(CTRL_PRIO, reg);
reg  = (reg & ~CTRL_PRIO) | FIELD_PREP(CTRL_PRIO, prio);

/* FIELD_PREP checks at COMPILE TIME that the value fits in the
   field and that the mask is a contiguous run of bits. That is
   the whole reason to prefer it over hand-rolled shifts.       */
```

Register access semantics

Type	Meaning	Correct access
RW	Read/write	Read-modify-write is fine
RO	Read only	Never write
WO	Write only; reads return garbage	Never read-modify-write — keep a shadow copy
W1C	Write 1 to clear	Write only the bits you want cleared
W1S	Write 1 to set	Same
RC	Read clears	Read exactly once and keep the value

```
/* W1C is the one people get wrong. */
reg |= IRQ_DONE;             /* WRONG: read-modify-write on W1C.
                                The read returns ALL pending bits, and
                                writing them back clears interrupts you
                                have not handled yet — silently losing
                                events.                              */

writel(IRQ_DONE, base + STATUS);   /* correct: write ONLY that bit */

/* Write-only registers need a shadow, because you cannot read back */
static uint32_t ctrl_shadow;
static void ctrl_update(uint32_t set, uint32_t clr)
{
        ctrl_shadow = (ctrl_shadow & ~clr) | set;
        writel(ctrl_shadow, base + CTRL);
}
```

Counting, finding and rounding

```
/* Use the builtins. They map to single instructions (CLZ, RBIT,
   CNT on ARM) and are far faster than a loop.                  */

__builtin_popcount(x)        /* number of set bits, unsigned int  */
__builtin_popcountll(x)      /* ...long long                      */
__builtin_clz(x)             /* count leading zeros  - UB if x==0 */
__builtin_ctz(x)             /* count trailing zeros - UB if x==0 */
__builtin_ffs(x)             /* find first set, 1-based, 0 if x==0 */
__builtin_parity(x)          /* 1 if an odd number of bits set    */

/* C23 standardises these in <stdbit.h>: */
stdc_count_ones(x)  stdc_leading_zeros(x)  stdc_trailing_zeros(x)
stdc_bit_width(x)   stdc_bit_ceil(x)       stdc_has_single_bit(x)

/* Linux equivalents that handle zero safely */
hweight32(x)   fls(x)   ffs(x)   __ffs(x)   ilog2(x)
```

```
/* Derived idioms - worth recognising instantly */

x & (x - 1)               /* clear the LOWEST set bit             */
x & -x                    /* ISOLATE the lowest set bit           */
x | (x - 1)               /* set all bits below the lowest set bit */
!(x & (x - 1)) && x       /* is x a power of two?                 */

/* iterate over set bits efficiently */
while (mask) {
        int bit = __builtin_ctzll(mask);
        handle(bit);
        mask &= mask - 1;          /* clear the one we just did */
}

/* round up to a multiple of a power of two */
#define ALIGN_UP(x, a)    (((x) + (a) - 1) & ~((a) - 1))
#define ALIGN_DOWN(x, a)  ((x) & ~((a) - 1))
#define IS_ALIGNED(x, a)  (((x) & ((a) - 1)) == 0)
/* NOTE: these require `a` to be a power of two. Assert it. */

/* round up to the next power of two */
static uint32_t next_pow2(uint32_t x)
{
        return x <= 1 ? 1 : 1u << (32 - __builtin_clz(x - 1));
}

/* ceiling division without floating point */
#define DIV_ROUND_UP(n, d)  (((n) + (d) - 1) / (d))
```

Byte order

```
/* Endianness is about the order of BYTES within a multi-byte value.
   It matters whenever bytes cross a boundary: a network, a file,
   another chip.                                                    */

uint32_t v = 0x12345678;

/* little endian (ARM, x86): low byte first  */   78 56 34 12
/* big endian ("network order"):             */   12 34 56 78

/* Never swap by hand with a cast — use the standard conversions. */
#include <arpa/inet.h>    /* htonl htons ntohl ntohs           */
#include <endian.h>       /* htobe32 htole32 be32toh le32toh   */

uint32_t on_wire = htobe32(v);       /* host -> big endian     */
uint32_t host    = be32toh(on_wire); /* big endian -> host     */

/* Linux kernel, with type annotations that sparse checks: */
__be32 x = cpu_to_be32(v);
v = be32_to_cpu(x);
/* Declaring the field as __be32 and building with `make C=1` makes
   a missing conversion a WARNING rather than a silent bug.      */

/* the builtins, if you need a raw swap */
__builtin_bswap16(x)  __builtin_bswap32(x)  __builtin_bswap64(x)

/* On a little-endian host these conversions compile to a single
   REV instruction; on a big-endian host they compile to nothing.
   There is no reason to avoid them.                             */
```

> **ENDIANNESS IS NOT THE SAME AS BIT ORDER**
>
> Within a byte, bit numbering is a documentation convention, not a hardware property — `x & 1` is the least significant bit everywhere. Endianness only reorders *bytes*. But a bitfield spanning a byte boundary is affected by both, which is one more reason bitfields are unsuitable for wire formats (ch. 07).

Bitmaps

```
/* A bitmap over N items, packed into words. */
#define BITS_PER_WORD   (8 * sizeof(unsigned long))
#define BITMAP_WORDS(n) (((n) + BITS_PER_WORD - 1) / BITS_PER_WORD)

unsigned long map[BITMAP_WORDS(1024)];

static inline void bm_set(unsigned long *m, size_t i)
{ m[i / BITS_PER_WORD] |=  1ul << (i % BITS_PER_WORD); }

static inline void bm_clear(unsigned long *m, size_t i)
{ m[i / BITS_PER_WORD] &= ~(1ul << (i % BITS_PER_WORD)); }

static inline bool bm_test(const unsigned long *m, size_t i)
{ return m[i / BITS_PER_WORD] & (1ul << (i % BITS_PER_WORD)); }

/* find the first zero bit - an allocator in four lines */
static size_t bm_find_free(const unsigned long *m, size_t n)
{
        for (size_t w = 0; w < BITMAP_WORDS(n); w++)
                if (~m[w])
                        return w * BITS_PER_WORD + __builtin_ctzl(~m[w]);
        return n;                   /* none free */
}

/* Linux has a complete, tested API - use it in kernel code:
     DECLARE_BITMAP, set_bit, clear_bit, test_and_set_bit,
     find_first_zero_bit, bitmap_weight, for_each_set_bit

   Note that set_bit() and friends are ATOMIC; __set_bit() is not.
   Choosing wrongly gives you a lost-update race under concurrency. */
```

Branchless and constant-time code

```
/* Two reasons to avoid branching on data:
     1. performance, when the branch is unpredictable (CA ch. 12)
     2. SECURITY, when the data is secret — branch timing leaks it */

/* select without a branch */
uint32_t mask = -(uint32_t)cond;          /* 0 or 0xFFFFFFFF */
result = (a & mask) | (b & ~mask);        /* cond ? a : b     */

/* min and max */
int min = b ^ ((a ^ b) & -(a < b));

/* absolute value, without a branch */
int mask = x >> (sizeof(int) * 8 - 1);    /* 0 or -1 (arithmetic shift) */
int abs  = (x + mask) ^ mask;

/* constant-time comparison — never use memcmp on secrets */
static int ct_eq(const uint8_t *a, const uint8_t *b, size_t n)
{
        uint8_t d = 0;
        for (size_t i = 0; i < n; i++)
                d |= a[i] ^ b[i];         /* no early exit */
        return d == 0;
}

/* The compiler may still reintroduce a branch. In real cryptographic
   code, verify the generated assembly, or use a vetted library.
   See the cryptography track, chapter 19.                          */
```

How it shows up

Symptom	Cause
A flag test is always false	`x & MASK == 0` — precedence
Setting bit 32 or higher does nothing	`1 << n` computed in `int`
Writing one field corrupts a neighbouring one	Missing mask on the value, or missing clear before OR
Interrupts are silently lost	Read-modify-write on a W1C status register
Reading a register changes system behaviour	Read-clear semantics; you read it twice
Protocol works locally, fails across the network	Missing byte-order conversion
Struct-based parser breaks on a different compiler	Bitfield allocation order is implementation-defined
Mask computation gives 0 for full width	`(1u << 32) - 1` is undefined
Two threads updating flags lose an update	Non-atomic read-modify-write; use `set_bit` or an atomic
Password comparison leaks timing	`memcmp` returns early on the first difference

Check yourself

- ✓ Give the four ground rules for bit code and the bug each prevents.
- ✓ Write set, clear, toggle and test for a single bit, safely for bit numbers up to 63.
- ✓ Extract and insert a 4-bit field at bits 7:4, masking the input value.
- ✓ Explain W1C semantics and why read-modify-write breaks it.

- ✓ Explain what `x & (x - 1)` and `x & -x` compute.
- ✓ Write a safe full-width mask expression for a runtime width of 0–32.
- ✓ Explain when byte-order conversion is needed and name the correct functions.
- ✓ Write a constant-time equality check and say why `memcmp` is unsuitable.

Storage duration, scope and linkage

Three independent properties that every identifier in C has, and that beginners routinely merge into one vague notion of "global". Separating them explains `static`'s two unrelated meanings, why `extern` exists, and what actually ends up in each section of your binary.

Three orthogonal questions

Property	Question it answers	Values
Storage duration	When does the object exist?	static, automatic, thread, allocated
Scope	Where is the name visible in the source?	file, block, function, function-prototype
Linkage	Do declarations in different translation units refer to the same object?	external, internal, none

```
They are independent. All eight useful combinations occur:

  int g;                     file scope, external linkage, static duration
  static int s;              file scope, INTERNAL linkage, static duration
  void f(void) {
      int a;                 block scope, no linkage, AUTOMATIC duration
      static int c;          block scope, no linkage, STATIC duration
      extern int g;          block scope, external linkage
      _Thread_local int t;   thread duration
      int *p = malloc(4);    p is automatic; *p has ALLOCATED duration
  }

The word `static` means two completely different things depending on
where it appears. That is a genuine wart in the language, not
something you are failing to understand.
```

Storage duration

Automatic

```
void f(void)
{
        int a;              /* INDETERMINATE — not zero */
        int b = 0;          /* explicitly initialised    */
        char buf[1024];     /* 1 KB of stack             */
}                           /* all cease to exist here    */

/* Reading an uninitialised automatic object is undefined behaviour,
   not "reads garbage". The compiler may assume it never happens.
   -Wuninitialized and -fsanitize=memory catch it.               */

/* Lifetime ends at the closing brace, INCLUDING inner blocks: */
int *p;
{
        int x = 5;
        p = &x;
}
*p;                         /* dangling: x's lifetime ended        */

/* Stack size is finite and often small:
     Linux userspace thread   8 MB default
     Linux KERNEL stack       8–16 KB    <- total, for the whole call chain
     Cortex-M firmware        1–8 KB
   Which is why kernel code never puts large buffers on the stack,
   and why VLAs are banned there (chapter 05).                    */
```

Static

```
int g_count;                /* zero-initialised, guaranteed */
static int s_flag = 1;
const char msg[] = "hi";

void f(void)
{
        static int calls = 0;    /* initialised ONCE, before main();
                                    retains its value between calls */
        calls++;
}

/* Static objects with no explicit initialiser are ZERO-initialised.
   This is a guarantee, unlike automatic storage.                  */

/* The initialiser must be a CONSTANT EXPRESSION — evaluated at
   translation time, not at run time:                             */
static int a = 5;              /* fine                    */
static int b = 5 * 3;          /* fine: constant folded   */
static int c = f();            /* ERROR in C (legal in C++) */
static int d = another_static; /* ERROR: not a constant     */
```

STATIC DURATION AND REENTRANCY

A function-local `static` makes the function **non-reentrant**: two threads, or an interrupt handler and mainline code, share that one object. This is why `strtok`, `asctime` and `gmtime` all have `_r` variants, and why a "just cache it in a static" optimisation is a concurrency bug waiting for a second thread. Use a caller-supplied buffer, or `_Thread_local`.

Thread

```
#include <threads.h>
_Thread_local int errno_like;      /* C11; one instance per thread   */
thread_local int x;                 /* C23 spelling / <threads.h>     */
__thread int y;                     /* GCC extension, widely used     */

/* Cost: access needs a TLS lookup — cheap but not free, and in
   shared libraries it can require a function call the first time.
   In the Linux kernel the equivalent is per-CPU data:            */
DEFINE_PER_CPU(int, my_counter);
this_cpu_inc(my_counter);       /* no locking needed: it is ours */
```

Allocated

```
void *p = malloc(n);            /* lifetime: from malloc to free     */
/* Covered fully in chapter 14. The key point here is that the
   POINTER has automatic duration while the OBJECT has allocated
   duration — two different lifetimes, and confusing them is
   exactly what a memory leak or a dangling pointer is.          */
```

Linkage, and what the linker sees

Linkage	Meaning	How to get it
External	One object shared across all translation units	File-scope declaration without `static`
Internal	One object per translation unit; invisible to the linker	File-scope `static`
None	Not shared at all	Block-scope objects, function parameters, `typedef`s

```
/* util.c */
int  public_counter;              /* external: visible everywhere    */
static int private_state;         /* internal: only this file        */
static void helper(void) { }      /* internal: not in the symbol table */
void api_function(void) { }       /* external                        */

/* main.c */
extern int public_counter;        /* refers to util.c's object       */
/* extern int private_state; */   /* LINK ERROR: undefined symbol    */
```

WHY `STATIC` AT FILE SCOPE IS GOOD PRACTICE, NOT JUST HIDING

- **Prevents name collisions** across the whole program — a real problem in large C codebases with no namespaces.
- **Enables optimisation.** The compiler can see every use of an internal-linkage function, so it can inline it aggressively, clone it, change its calling convention, or delete it entirely if unused.
- **Documents intent** — this is an implementation detail, not API.
- **Shrinks the symbol table** and the exported surface.

Rule: **everything at file scope should be `static` unless it is declared in a header.** Build with `-Wmissing-prototypes` to catch violations mechanically.

Declaration versus definition

```
/* A DECLARATION introduces a name and its type.
   A DEFINITION additionally allocates storage or supplies a body.
   You may declare a thing many times; you may define it once.    */

extern int x;               /* declaration                       */
int x;                      /* definition (tentative - see below) */
int x = 5;                  /* definition with initialiser        */

void f(void);               /* declaration (prototype)            */
void f(void) { }            /* definition                         */

/* The C-specific wrinkle: a TENTATIVE definition. */
int x;          /* at file scope, with no initialiser, this is a
                   "tentative definition". Several are allowed in
                   ONE translation unit and collapse into one.    */

/* Across translation units, C90 compilers accepted multiple
   tentative definitions and merged them - the "common symbol"
   model. GCC 10 changed the default to -fno-common, so this is
   now an error, as it always should have been:                */

/* a.c */  int counter;
/* b.c */  int counter;      /* error: multiple definition of `counter' */

/* THE CORRECT PATTERN, and the only one you should use: */

/* counter.h */
extern int counter;          /* declaration only, in the header */

/* counter.c */
#include "counter.h"         /* include your OWN header - this is
                                what makes the compiler check the
                                definition against the declaration */
int counter = 0;             /* the single definition             */
```

Header discipline

```
/* my_module.h */
#ifndef MY_MODULE_H
#define MY_MODULE_H            /* include guard — always            */

#include <stddef.h>            /* headers must be self-contained:
                                  include what YOU need, so the
                                  user need not guess the order    */

struct my_thing;               /* forward declaration is enough if
                                  callers only handle pointers      */

extern int my_global;                      /* declaration       */
int  my_init(void);                         /* declaration       */
void my_process(struct my_thing *, size_t);

/* An inline function in a header needs care in C (unlike C++): */
static inline int my_helper(int x) { return x * 2; }
/* `static inline` gives every TU its own copy with internal
   linkage. It always works, and is what you want 99% of the time.
   Plain `inline` in C requires exactly one TU to also provide an
   `extern inline` definition, which is a rule almost nobody
   remembers correctly.                                          */

#endif /* MY_MODULE_H */
```

Put in the header	Keep in the .c file
Declarations of public functions	Definitions
`extern` declarations of public variables	The one definition
Types callers must know the size of	Types callers only hold pointers to (opaque)
Macros and constants that are part of the API	Internal macros
`static inline` helpers, if small and hot	Everything else

```
/* Opaque types: the single most useful encapsulation technique in C */

/* device.h — callers see only a pointer */
struct device;                          /* incomplete type      */
struct device *device_create(void);
int  device_read(struct device *, void *, size_t);
void device_destroy(struct device *);

/* device.c — the layout is private */
struct device {
        int fd;
        char name[64];
        struct buffer *buf;
};

/* Callers cannot allocate one on the stack, cannot access members,
   and cannot break when you change the layout. The whole struct is
   an implementation detail. This is how every good C library is
   structured — FILE*, pthread_t and struct device in the kernel
   are all this pattern.                                          */
```

Where objects actually land

```
   ELF sections, and what maps to each

   .text     code, read-only, executable
             functions, and const data on some targets

   .rodata   read-only data
             string literals, const arrays, jump tables
             -> writing here faults (chapter 05)

   .data     initialised read-write static-duration objects
             int g = 5;
             -> occupies space in the BINARY (the initial values
                must be stored somewhere)

   .bss      zero-initialised static-duration objects
             int g;  static char buf[1<<20];
             -> occupies NO space in the binary, only a size.
                The loader (or startup code) zeroes it.
                This is why a 1 MB zeroed array costs nothing on disk.

   stack     automatic-duration objects
   heap      allocated-duration objects
```

```
# see it for yourself
size ./program
#    text    data     bss     dec     hex filename
#   12345     456   10240   23041    5a01 ./program

nm -C --size-sort ./program | tail -20     # largest symbols
nm ./program | grep ' [bB] '               # bss symbols
readelf -S ./program                       # section headers
objdump -t ./program                        # full symbol table

# in embedded work, `size` is a build-gate metric: .text must fit
# in flash and .data + .bss must fit in RAM.

# placing objects in a specific section, for linker scripts
__attribute__((section(".fast_code"))) void isr(void) { }
__attribute__((section(".noinit")))    uint32_t boot_count;
```

Initialisation order

```
/* Within one translation unit, static objects are initialised in
   declaration order — but in C they are all constant expressions,
   so there is no ordering hazard as there is in C++.

   What DOES have an ordering problem is constructor attributes: */

__attribute__((constructor))       void setup(void)   { }
__attribute__((constructor(101)))  void setup_early(void) { }
__attribute__((destructor))        void teardown(void) { }

/* Order ACROSS translation units is unspecified (priority values
   help, but only within one link unit). Do not build dependencies
   between constructors in different files.

   The robust pattern is explicit initialisation from main(), or
   lazy initialisation guarded by a flag:                          */

static bool ready;
static void ensure_init(void)
{
        if (!ready) { do_init(); ready = true; }
}
/* ...and note that this is NOT thread safe. For that you need
   call_once (C11) or pthread_once.                                */

#include <threads.h>
static once_flag once = ONCE_FLAG_INIT;
call_once(&once, do_init);
```

How it shows up

Symptom	Cause
"multiple definition of X" at link time	A variable defined in a header, or in two .c files
"undefined reference to X"	Declared but never defined; or defined `static` in another file
Two modules corrupt each other's data	Same external name in both; should have been `static`
Value changes unexpectedly between calls	A function-local `static` you forgot about
Function misbehaves when called from two threads	Non-reentrant: shared `static` state
Returned pointer contains garbage	Pointer to an automatic object whose lifetime ended
Binary is huge for a program with little code	Large initialised arrays in `.data`; make them zero-init or `const`
Works with GCC 9, fails to link with GCC 10	`-fno-common` became the default; fix the tentative definitions
Constructor runs before something it depends on	Cross-TU initialisation order is unspecified
Stack overflow in kernel or firmware	Large automatic buffer; kernel stacks are 8–16 KB

Check yourself

- ✓ Name the three orthogonal properties and give an example of each combination that matters.
- ✓ Explain `static`'s two distinct meanings and where each applies.
- ✓ Say which objects are zero-initialised for you and which are indeterminate.

- ✓ Explain why a function-local `static` makes a function non-reentrant.
- ✓ Give the correct header/source pattern for a shared global and say why the .c file includes its own header.
- ✓ Explain what a tentative definition is and why `-fno-common` changed things.
- ✓ Describe the opaque-type pattern and what it buys.
- ✓ Say which section a 1 MB zeroed array lands in and why it does not bloat the binary.

const, volatile and restrict

Three qualifiers, three completely different jobs. `const` is a promise you make to the compiler; `volatile` is a promise the compiler makes to you; `restrict` is a promise you make that the compiler cannot verify. Each is routinely misunderstood, and `volatile` especially so.

const

WHAT IT MEANS

`const` says "*I will not modify this object through this expression.*" It is a type-system constraint, checked at compile time. It does **not** mean the object is immutable, and it does **not** mean the value is a compile-time constant.

```
/* Placement — read right to left from the identifier */
const int *p;          /* pointer to const int: *p is read-only     */
int const *p;          /* identical                                  */
int *const p;          /* const pointer: p itself cannot be reseated */
const int *const p;    /* both                                       */

const int *p = &x;
*p = 5;                /* error: object is const through p           */
p = &y;                /* fine                                       */
x = 5;                 /* FINE — x itself is not const               */

/* That last line is the point: const on a pointer restricts what
   YOU may do through that pointer, not what the object is.       */
```

Where const earns its place

```
/* 1. API documentation the compiler enforces */
size_t strlen(const char *s);          /* will not modify s          */
void *memcpy(void *dst, const void *src, size_t n);   /* src is read */

/* 2. Genuinely read-only data goes in .rodata, saving RAM —
      which on a microcontroller is the scarce resource         */
static const uint8_t font_table[4096] = { ... };   /* flash, not RAM */

/* 3. Catching mistakes at compile time */
void process(const struct config *cfg)
{
        cfg->retries = 3;      /* error — caught immediately         */
}
```

THREE THINGS CONST IS NOT

1. **Not a compile-time constant.** `const int n = 10; int a[n];` is a VLA in C, not a fixed array. Use `#define`, an `enum`, or C23's `constexpr` for array bounds and case labels.
2. **Not immutability.** Casting it away and writing is undefined *only if the object was originally defined const*. Writing through a cast to a non-const object is legal — which is why `const` cannot be relied on for security.
3. **Not deep.** `const struct s *p` makes `p->field` const, but if `field` is a pointer, `*p->field` is still writable. C has no transitive const.

```
/* The const-correctness annoyance you will hit: C, unlike C++, does
   not implicitly convert `T**` to `const T**`. This is by design —
   allowing it would let you smuggle a const pointer into a
   non-const one — but it means some correct code needs a cast.   */
void f(const char **p);
char *argv[10];
f(argv);                    /* warning: incompatible pointer type */
f((const char **)argv);     /* explicit, and safe here            */
```

volatile

WHAT IT MEANS

`volatile` tells the compiler: *this object may change by means outside this program's control, so perform every access exactly as written — do not cache it in a register, do not eliminate it, do not duplicate it, and do not reorder it relative to other volatile accesses.*

It constrains the **compiler**. It says nothing to the CPU or the memory system.

```
/* Without volatile, this is an infinite loop at -O2 */
uint32_t *status = (uint32_t *)0x40000018;
while (!(*status & READY))     /* nothing in the loop changes *status,
        ;                         so the compiler loads it ONCE, or
                                  deletes the loop entirely          */

/* With volatile, the load happens every iteration */
volatile uint32_t *status = (volatile uint32_t *)0x40000018;
while (!(*status & READY))
        ;
```

The three legitimate uses

Use	Why
Memory-mapped I/O	Hardware changes the value; reads and writes have side effects
Variables modified by a signal handler	Must be `volatile sig_atomic_t`; the handler runs outside normal flow
Variables modified across `setjmp` / `longjmp`	Non-volatile locals have indeterminate values after a `longjmp`

VOLATILE IS NOT FOR THREADS

This is the most persistent misconception in C. `volatile` gives you *none* of what concurrent code needs:

- **No atomicity.** `volatile int x; x++;` is still a load, an add and a store. Two threads can still lose an update.
- **No memory ordering.** Volatile accesses are ordered relative to *each other*, but not relative to ordinary accesses, and the *hardware* is free to reorder everything anyway on a weakly ordered machine like ARM (CA ch. 15).
- **No visibility guarantee.** Nothing forces the store out of the store buffer or invalidates another core's cache line.

For threads, use `_Atomic` / `<stdatomic.h>`, or your platform's primitives. Linus Torvalds' well-known essay on this is `Documentation/process/volatile-considered-harmful.rst` in the kernel tree — it is worth reading.

```
/* WRONG — a classic that appears to work until it does not */
volatile bool done = false;
volatile int  result;

void producer(void) { result = compute(); done = true; }
void consumer(void) { while (!done) ; use(result); }
/* On ARM the two stores may become visible in either order, so the
   consumer can see done==true and a stale result. volatile did
   nothing to prevent it.                                          */

/* RIGHT — C11 atomics */
#include <stdatomic.h>
atomic_bool done = false;
int result;                             /* plain int is fine now */

void producer(void)
{
        result = compute();
        atomic_store_explicit(&done, true, memory_order_release);
}
void consumer(void)
{
        while (!atomic_load_explicit(&done, memory_order_acquire))
                ;
        use(result);                    /* guaranteed to see it   */
}
```

volatile in MMIO, done properly

```
/* volatile alone is not sufficient for device registers either —
   it stops the COMPILER reordering, not the CPU or the bus.     */

/* Bare metal: pair it with barriers where ordering matters */
static inline void mmio_write32(uintptr_t a, uint32_t v)
{
        *(volatile uint32_t *)a = v;
        __asm__ volatile("dsb sy" ::: "memory");   /* if it must land */
}

/* In an OS, use the accessors — they get this right for you: */
writel(val, base + CTRL);        /* Linux: volatile + barriers      */
readl(base + STATUS);
writel_relaxed(val, base + CTRL);  /* no barrier, when order is
                                      genuinely irrelevant           */
WRITE_REGISTER_ULONG(reg, val);  /* Windows                          */

/* Never hand-roll MMIO access in kernel code. The accessors also
   encode endianness and, on some architectures, special
   instructions that a plain volatile store would not use.       */
```

The volatile-qualified struct pattern

```
/* Correct: every member volatile */
struct uart_regs {
        volatile uint32_t DR;
        volatile uint32_t FR;
};

/* Also correct, and shorter: qualify the pointer */
struct uart_regs { uint32_t DR; uint32_t FR; };
volatile struct uart_regs *u = (volatile struct uart_regs *)BASE;

/* WRONG: qualifying the pointer variable rather than the target */
struct uart_regs *volatile u;      /* the POINTER is volatile;
                                      the registers are not!      */
```

restrict

WHAT IT MEANS

`restrict` on a pointer promises: *within the lifetime of this pointer, the object it points to will not be accessed through any other pointer.* It is a promise **you** make. The compiler cannot check it, and breaking it is undefined behaviour with no diagnostic.

It applies only to pointers, and it exists purely to enable optimisation.

```
/* Without restrict, the compiler must assume a and b may overlap */
void add(int *a, const int *b, size_t n)
{
        for (size_t i = 0; i < n; i++)
                a[i] += b[i];
}
/* Every iteration must RELOAD b[i], because the previous store to
   a[i] might have modified it. No vectorisation possible.       */

/* With restrict, the promise unlocks it */
void add(int *restrict a, const int *restrict b, size_t n)
{
        for (size_t i = 0; i < n; i++)
                a[i] += b[i];
}
/* Now the compiler can load a block of b, load a block of a, add
   them with SIMD, and store. Typically 4-8x faster.             */
```

```
/* The canonical illustration: two functions, one difference. */
void *memcpy (void *restrict dst, const void *restrict src, size_t n);
void *memmove(void       *dst, const void       *src, size_t n);

/* memcpy's arguments are restrict: overlapping is UNDEFINED, and
   glibc's memcpy really does misbehave on overlap — it copies in
   whatever order is fastest, including backwards.

   memmove handles overlap correctly and pays for it.

   If you are not certain the buffers are disjoint, use memmove. The
   "memcpy is faster" reasoning is not worth a corruption bug.   */
```

HOW TO BREAK A RESTRICT PROMISE WITHOUT NOTICING

```
void f(int *restrict a, int *restrict b);
int arr[10];
f(arr, arr);              /* UB — same object through both        */
f(arr, arr + 5);          /* UB if the accessed ranges overlap    */

/* Aliasing through a struct member is easy to miss: */
void g(struct s *restrict p, int *restrict q);
g(&obj, &obj.field);      /* UB — q points inside *p              */

/* Rule of thumb: only use restrict on parameters of leaf functions
   whose callers you control, or where the API contract explicitly
   forbids overlap (and is documented as doing so).             */
```

Combining qualifiers

```
/* const volatile — real and useful: a hardware register that the
   device updates but software must never write.               */
const volatile uint32_t *status = (const volatile uint32_t *)0x40000018;
uint32_t s = *status;      /* fine: reads happen every time      */
*status = 0;               /* compile error: correctly rejected  */

/* Typical for: status registers, free-running counters, ID registers */

/* volatile const is the same thing; order does not matter.       */

/* restrict + const is common in read-only source parameters */
void transform(float *restrict out, const float *restrict in, size_t n);
```

_Atomic, for completeness

```
#include <stdatomic.h>

_Atomic int counter;               /* C11 type qualifier         */
atomic_int counter;                /* the convenience typedef     */

atomic_fetch_add(&counter, 1);     /* atomic read-modify-write   */
atomic_load(&counter);
atomic_store(&counter, 0);
atomic_compare_exchange_strong(&p, &expected, desired);

/* _Atomic gives you what volatile does not:
     - indivisibility of read-modify-write
     - a memory ordering model you can reason about
     - the compiler emits LDAXR/STLXR or LSE atomics on ARM

   Note: atomic operations on large types may be implemented with a
   lock. Check with:                                          */
atomic_is_lock_free(&obj);
ATOMIC_INT_LOCK_FREE               /* compile-time: 0, 1 or 2    */
```

Need	Use	Not
Hardware register	`volatile` + barriers, or MMIO accessors	`_Atomic`
Shared between threads	`_Atomic` or a lock	`volatile`
Set by a signal handler	`volatile sig_atomic_t` or `atomic_int`	plain `int`
Read-only API parameter	`const`	—
Non-overlapping buffers	`restrict`	—
Compile-time constant	`#define`, `enum`, C23 `constexpr`	`const`

How it shows up

Symptom	Cause
Polling loop hangs at `-O2`, works at `-O0`	Missing `volatile` on the status register
Register writes never reach the device	Missing `volatile`, or missing barrier
Thread sees a stale value despite `volatile`	`volatile` is not a memory barrier; use atomics

Symptom	Cause
Counter loses increments under load	`volatile` does not make `++` atomic
`const int n` rejected as an array bound	Not a compile-time constant in C; use `enum` or `#define`
Corruption when source and destination overlap	`memcpy` with overlapping buffers; use `memmove`
Vectorisation silently disabled	Possible aliasing; add `restrict`
Wrong results after adding `restrict`	The promise was false — the buffers do alias
Warning converting `char**` to `const char**`	C does not allow it implicitly; cast explicitly
Struct members not treated as volatile	Qualified the pointer variable instead of the pointee

Check yourself

- ✓ Distinguish `const int *p`, `int *const p` and `const int *const p`.
- ✓ Give three things `const` does not provide.
- ✓ Explain exactly what `volatile` constrains, and what it does not.
- ✓ Give the three legitimate uses of `volatile`.
- ✓ Explain, with an example, why `volatile` is wrong for thread communication.
- ✓ Write a correct MMIO write and say why `volatile` alone is insufficient.
- ✓ State the `restrict` promise and show two ways to break it accidentally.
- ✓ Explain why both `memcpy` and `memmove` exist.
- ✓ Give a case where `const volatile` is exactly right.

Functions, calling conventions and varargs

A function call is a contract between two pieces of code that may have been compiled separately. This chapter covers what C guarantees about that contract, what the ABI adds, and why varargs is the least safe thing in the language.

C is pass-by-value, always

THERE ARE NO EXCEPTIONS

Every argument is copied into the function. The function receives a private copy and cannot modify the caller's object. "Pass by reference" in C is a convention: you pass a *copy of a pointer*, and the function modifies what that pointer designates.

```
void bad_swap(int a, int b) { int t = a; a = b; b = t; }
/* modifies copies; the caller sees nothing */

void good_swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
good_swap(&x, &y);
/* the POINTERS are still passed by value — but they point at the
   caller's objects, so writing through them is visible.        */

/* This is also why you need a pointer-to-pointer to change a
   caller's pointer (chapter 04):                               */
void alloc(char **out) { *out = malloc(100); }

/* And why passing a large struct by value is a real cost: */
void f(struct big s);          /* copies sizeof(struct big) bytes */
void f(const struct big *s);   /* copies 8 bytes                  */
/* Rule of thumb: pass by pointer above ~2 machine words, and use
   const to document that you will not modify it.               */
```

Prototypes matter

```
/* THREE different declarations, and only one is right in modern C */

int f();              /* pre-C23: UNSPECIFIED number of arguments.
                         No checking at all. C23: same as (void).  */
int f(void);          /* takes NO arguments. Checked.              */
int f(int, char *);   /* full prototype. Checked.                  */

/* With `int f();` the compiler cannot check your call site: */
f(1, 2, 3);           /* accepted; arguments passed per default
                         promotions; behaviour undefined if the
                         definition disagrees                      */

/* This is why every C project should build with: */
//   -Wstrict-prototypes    warn on ()
//   -Wmissing-prototypes   warn on a non-static function with no
//                          prior prototype (catches typos in names)
//   -Wold-style-definition

/* And why implicit declaration is fatal. Pre-C99, calling an
   undeclared function assumed it returned int:               */
double x = sqrt(2.0);       /* without #include <math.h>: assumed to
                               return int, so x gets garbage      */
/* C99 removed implicit declarations. GCC 14 and Clang 16 finally
   made it a hard error by default. If you see
   "implicit declaration of function", you are missing a header -
   never silence it with a cast.                                  */
```

The calling convention

C says nothing about *how* arguments reach the callee. That is defined by the platform ABI, and it is what makes separately compiled code interoperate.

```
AArch64 (AAPCS64) - the convention on every ARM64 system

  X0-X7    integer/pointer arguments 1-8, and return value in X0
  X8       indirect result pointer (for large struct returns)
  X9-X15   caller-saved temporaries
  X16,X17  IP0/IP1 - reserved for linker veneers
  X18      platform register (TLS on Windows; reserved on some OSes)
  X19-X28  CALLEE-saved: if you use them, save and restore them
  X29      FP  frame pointer
  X30      LR  link register - the return address, set by BL
  SP       stack pointer, must be 16-byte aligned at every call
  V0-V7    floating point / SIMD arguments and return
  V8-V15   callee-saved (lower 64 bits only)

  Arguments beyond 8 go on the stack, right to left.

x86-64 System V (Linux, macOS)
  RDI RSI RDX RCX R8 R9  integers;  XMM0-7 floats;  RAX returns

x86-64 Microsoft
  RCX RDX R8 R9 only - just four register arguments, plus a
  32-byte "shadow space" the caller must reserve.

  This is why calling convention mismatches are a real bug class
  in cross-platform code and in hand-written assembly.
```

```
/* what a function's prologue and epilogue are actually doing */
my_func:
        stp     x29, x30, [sp, #-32]!   /* push FP and LR, SP -= 32   */
        mov     x29, sp                 /* establish the frame        */
        stp     x19, x20, [sp, #16]     /* save callee-saved regs     */
        ...
        ldp     x19, x20, [sp, #16]     /* restore                     */
        ldp     x29, x30, [sp], #32     /* pop, SP += 32               */
        ret                             /* branch to X30               */

/* The 16-byte SP alignment requirement is not advisory. Violating
   it faults on any instruction that assumes it, and produces
   corruption that looks random. Hand-written assembly gets this
   wrong constantly.                                          */
```

Variadic functions

```
#include <stdarg.h>

int sum(int count, ...)
{
        va_list ap;
        va_start(ap, count);           /* last NAMED parameter */

        int total = 0;
        for (int i = 0; i < count; i++)
                total += va_arg(ap, int);   /* type must be correct */

        va_end(ap);                     /* required */
        return total;
}
sum(3, 10, 20, 30);
```

WHY VARARGS IS THE LEAST SAFE CONSTRUCT IN C

- **No type checking whatsoever.** `va_arg(ap, int)` on a `double` argument reads the wrong bytes from the wrong place. There is no diagnostic.
- **No count.** The function cannot know how many arguments it received. You must encode it in a named parameter or a sentinel — and if the caller lies, you read past the argument area.
- **Default argument promotions apply** and surprise people: `float` becomes `double`, and anything narrower than `int` becomes `int`. So `va_arg(ap, char)` and `va_arg(ap, float)` are always wrong.
- `NULL` **is not a pointer.** Passing bare `NULL` as a variadic sentinel may pass a 32-bit `0` where a 64-bit pointer is expected. Write `(void *)NULL`.

```
/* the promotion traps */
f(1, 'a', 3.14f);
        va_arg(ap, char)    /* WRONG: promoted to int   -> use int    */
        va_arg(ap, float)   /* WRONG: promoted to double -> use double */
        va_arg(ap, short)   /* WRONG                    -> use int    */

/* the sentinel trap */
execl("/bin/ls", "ls", NULL);          /* may pass 32-bit 0 */
execl("/bin/ls", "ls", (char *)NULL);  /* correct           */

/* Getting the compiler to help with printf-like functions: */
void my_log(const char *fmt, ...)
        __attribute__((format(printf, 1, 2)));
/* Now -Wformat checks your call sites exactly as it does printf's.
   Add this to every logging function you write - it is free, and
   it catches a genuinely common bug class.                      */

/* Forwarding varargs requires the v-variants: */
void my_log(const char *fmt, ...)
{
        va_list ap;
        va_start(ap, fmt);
        vfprintf(stderr, fmt, ap);      /* NOT fprintf(...) */
        va_end(ap);
}

/* Reusing a va_list after it has been consumed is UB. Copy it: */
va_list ap2;
va_copy(ap2, ap);
int len = vsnprintf(NULL, 0, fmt, ap);   /* measure */
char *buf = malloc(len + 1);
vsnprintf(buf, len + 1, fmt, ap2);       /* format  */
va_end(ap2);
```

ALTERNATIVES WORTH PREFERRING

Pass an array plus a count. Use a struct with designated initialisers. In C11, use `_Generic` for type-safe dispatch. Variadic macros (`__VA_ARGS__`) are checked because they expand into real calls — prefer them for logging wrappers. Reserve genuine varargs for `printf`-shaped APIs, and always attach the `format` attribute.

_Generic: type-safe dispatch

```
/* C11's answer to "I want one name for several types" */
#define abs(x) _Generic((x),        \
        int:         abs,           \
        long:        labs,          \
        long long:   llabs,         \
        float:       fabsf,         \
        double:      fabs,          \
        default:     fabs           \
)(x)

/* Selection happens at COMPILE TIME on the static type of the
   controlling expression. The unselected branches are not even
   evaluated, so this is zero-cost.                          */

/* A practical use: type-safe printing helpers */
#define print(x) _Generic((x),              \
        int:          print_int,            \
        const char *: print_str,            \
        char *:       print_str,            \
        double:       print_double          \
)(x)
```

inline and noreturn

```
/* `static inline` in a header — use this, essentially always */
static inline int max(int a, int b) { return a > b ? a : b; }

/* Plain `inline` in C has a rule almost nobody applies correctly:
   an inline definition provides no external definition, so exactly
   one TU must also provide `extern inline` (C99) or a non-inline
   definition. Get it wrong and you get "undefined reference".
   `static inline` sidesteps the whole problem.                  */

/* inline is a HINT. These are not: */
__attribute__((always_inline)) static inline void f(void);
__attribute__((noinline))      void g(void);

/* noreturn lets the compiler drop the return path and improves
   warnings about uninitialised variables after a call.        */
#include <stdnoreturn.h>
noreturn void panic(const char *msg);          /* C11 */
[[noreturn]] void panic(const char *msg);      /* C23 */
__attribute__((noreturn)) void panic(const char *);   /* GCC */
```

Attributes worth knowing

Attribute	Effect
`format(printf, m, n)`	Enables `-Wformat` checking on your own logging functions
`warn_unused_result`	Warns if the return value is ignored — essential for functions that can fail
`nonnull(1, 2)`	Documents and checks that parameters must not be NULL
`malloc`	Tells the optimiser the returned pointer does not alias anything
`pure` / `const`	No side effects / depends only on arguments — enables CSE

Attribute	Effect
`cleanup(fn)`	Calls `fn` when the variable goes out of scope — RAII in C
`weak`	The symbol may be overridden; used for optional hooks in firmware
`section("name")`	Place in a specific linker section

```
/* cleanup gives you scope-based resource management in plain C.
   The Linux kernel adopted this in 6.5 and it is spreading fast. */
static inline void free_ptr(void *p) { free(*(void **)p); }
#define _cleanup_free_ __attribute__((cleanup(free_ptr)))

void f(void)
{
        _cleanup_free_ char *buf = malloc(100);
        if (!buf) return;                  /* freed automatically */
        if (error) return;                 /* freed automatically */
        ...
}                                          /* freed automatically */

/* This removes the goto-ladder for the common single-resource case.
   It is a GCC/Clang extension, not ISO C.                          */
```

Recursion and stack budgets

```
/* Recursion is fine in application code and dangerous in systems code.
   Kernel stacks are 8-16 KB TOTAL for the entire call chain; an
   interrupt handler may nest on top of it.                         */

/* Estimate a frame's size: */
//   objdump -d prog | grep 'sub.*sp'      look for the prologue
//   gcc -fstack-usage                     emits a .su file per TU
//   gcc -Wstack-usage=1024                warn above a threshold

/* Convert recursion to iteration with an explicit stack when depth
   is input-dependent — an attacker-controlled depth is a denial of
   service at best.                                                 */

/* Tail calls MAY be optimised into a jump, but C does not
   guarantee it. Do not rely on TCO for correctness.            */
```

How it shows up

Symptom	Cause
Function modifies its argument but the caller sees no change	Pass-by-value; you need a pointer
"implicit declaration of function"	Missing header; the return type is assumed `int`
Float return value is garbage	Missing prototype, so the wrong register was read
`printf("%d", 3.14)` prints nonsense	Varargs has no type checking
Variadic function reads garbage after N arguments	Count or sentinel was wrong
Crash only on 64-bit	Bare `NULL` as a variadic sentinel
"undefined reference" for an `inline` function	Plain `inline` without an external definition; use `static inline`
Stack corruption in hand-written assembly	SP not 16-byte aligned, or a callee-saved register clobbered

Symptom	Cause
Passing a big struct is unexpectedly slow	Copied by value; pass a `const` pointer
Kernel stack overflow with deep recursion	8–16 KB total; convert to iteration

Check yourself

- ✓ Explain why C has no pass-by-reference, and how the effect is achieved.
- ✓ Give the difference between `int f()` and `int f(void)` and why it matters.
- ✓ Name the AArch64 argument registers, the callee-saved set, and the stack alignment rule.
- ✓ List four reasons varargs is unsafe.
- ✓ Explain default argument promotions and give two `va_arg` types that are always wrong.
- ✓ Show how to make the compiler type-check your own logging function.
- ✓ Explain why `static inline` is preferred over plain `inline` in headers.
- ✓ Describe the `cleanup` attribute and the problem it solves.

Function pointers and dispatch tables

Function pointers are how C does polymorphism, callbacks, plugins and state machines. Every driver interface in Linux and Windows is a struct of function pointers. Once the declaration syntax stops being frightening, the patterns are straightforward — and they are everywhere.

Declaration syntax

```
   int  f(int);            a function taking int, returning int
   int (*p)(int);          a POINTER to such a function
   int *g(int);            a function returning int*  - different!

The parentheses around *p are mandatory, because () binds tighter
than *. Without them you get "function returning a pointer".

   int (*fns[4])(int);     an array of 4 such pointers
   int (*(*p)[4])(int);    a pointer to such an array
   int (*f(void))(int);    a function returning a function pointer

At the third line, stop and use a typedef. Everyone does.
```

```
/* Two ways to name the type. Learn to read both. */
typedef int  handler_t(int);       /* the FUNCTION type      */
typedef int (*handler_ptr_t)(int); /* the POINTER type       */

handler_ptr_t p = my_func;
handler_t    *q = my_func;         /* equivalent, and arguably clearer */

/* Modern style prefers naming the function type and adding * at use,
   because it keeps the pointer-ness visible:                        */
typedef int compare_fn(const void *, const void *);
void sort(void *base, size_t n, size_t sz, compare_fn *cmp);
```

Taking and calling

```
int add(int a, int b) { return a + b; }

int (*op)(int, int) = add;      /* the & is optional...   */
int (*op)(int, int) = &add;     /* ...both are identical  */

int r = op(2, 3);               /* call: the * is optional */
int r = (*op)(2, 3);            /* ...both are identical   */

/* A function name in an expression decays to a pointer, exactly
   like an array name (chapter 05). That is why both spellings
   work. Pick one and be consistent; `op(2,3)` is more common.  */

/* Comparison and null are meaningful */
if (op == add)  ...
if (op != NULL) op(1, 2);

/* But arithmetic is NOT. You cannot do op + 1, and sizeof on a
   function type is undefined.                                   */
```

FUNCTION POINTERS ARE NOT OBJECT POINTERS

The standard guarantees conversion between `void *` and *object* pointers only. Converting a function pointer through `void *` is not guaranteed by ISO C — on architectures where code addresses have a different size or format it genuinely breaks. POSIX requires it to work so that `dlsym()` can exist, which is why that API is a documented wart. Keep function pointers in function-pointer types, and use a union or a POSIX-blessed cast if you truly must.

Callbacks

```
/* The standard library's own example */
int cmp_int(const void *a, const void *b)
{
        int x = *(const int *)a, y = *(const int *)b;
        return (x > y) - (x < y);       /* avoids overflow of x - y */
}
qsort(arr, n, sizeof arr[0], cmp_int);

/* THE design rule for callback APIs: always carry a user context.
   Without it, callers are forced into global state.            */

/* BAD - no context */
void for_each(int *a, size_t n, void (*fn)(int));

/* GOOD - an opaque pointer threaded through */
void for_each(int *a, size_t n, void (*fn)(int, void *), void *ctx)
{
        for (size_t i = 0; i < n; i++)
                fn(a[i], ctx);
}

struct acc { long total; };
static void add_to(int v, void *ctx) { ((struct acc *)ctx)->total += v; }

struct acc a = {0};
for_each(arr, n, add_to, &a);

/* This is why qsort_r/qsort_s exist: plain qsort has no context
   parameter, so comparators that need extra state must use a
   global - which is not reentrant or thread safe.             */
```

Dispatch tables: polymorphism in C

```
/* A struct of function pointers IS a vtable. This is exactly how
   C++ virtual functions are implemented, written out by hand.  */

struct file_ops {
        int     (*open)(struct file *);
        ssize_t (*read)(struct file *, void *, size_t);
        ssize_t (*write)(struct file *, const void *, size_t);
        int     (*close)(struct file *);
};

struct file {
        const struct file_ops *ops;     /* the vtable pointer */
        void *private_data;             /* the "derived" state */
};

/* Each implementation supplies its own table. Designated
   initialisers mean unimplemented operations are NULL.       */
static const struct file_ops disk_ops = {
        .open  = disk_open,
        .read  = disk_read,
        .write = disk_write,
        .close = disk_close,
};
static const struct file_ops socket_ops = {
        .open  = sock_open,
        .read  = sock_read,
        /* .write and .close are NULL */
};

/* Generic code dispatches without knowing the type: */
ssize_t file_read(struct file *f, void *buf, size_t n)
{
        if (!f->ops->read)
                return -ENOSYS;         /* ALWAYS check for NULL */
        return f->ops->read(f, buf, n);
}

/* Note `const struct file_ops *` — the table lives in .rodata,
   so it cannot be corrupted by a stray write, and it is shared
   by every instance rather than copied per object.           */
```

```
Why this is the shape of every OS driver interface

   struct file                struct file_ops (in .rodata)
   +----------------+         +---------------------------+
   | ops  ----------|-------> | open   -> disk_open       |
   | private_data   |         | read   -> disk_read       |
   +----------------+         | write  -> disk_write      |
                              | close  -> disk_close      |
   struct file                +---------------------------+
   +----------------+
   | ops  ----------|-------> (socket_ops)
   | private_data   |
   +----------------+

  One generic VFS layer, many implementations, no switch statement,
  and new implementations can be added without touching the core.

  Linux: struct file_operations, struct net_device_ops,
         struct platform_driver
  Windows: the WDF EvtIo* callbacks and the WDM MajorFunction array
  Both are this pattern.
```

Jump tables and state machines

```
/* Replacing a long switch with a table — O(1), and data-driven */

typedef void (*cmd_fn)(const char *args);

struct command {
        const char *name;
        cmd_fn      handler;
        const char *help;
};

static const struct command commands[] = {
        { "read",  cmd_read,  "read a register"  },
        { "write", cmd_write, "write a register" },
        { "dump",  cmd_dump,  "dump memory"      },
};

static int dispatch(const char *name, const char *args)
{
        for (size_t i = 0; i < ARRAY_SIZE(commands); i++)
                if (strcmp(commands[i].name, name) == 0) {
                        commands[i].handler(args);
                        return 0;
                }
        return -EINVAL;
}
/* Adding a command is one table row. The help text, the parser and
   the dispatcher all stay unchanged — this is the main practical
   benefit over a switch.                                          */
```

```
/* A state machine as a table of transitions */
typedef enum { S_IDLE, S_RUNNING, S_ERROR, S_COUNT } state_t;
typedef enum { E_START, E_STOP, E_FAULT, E_COUNT } event_t;

typedef state_t (*transition_fn)(void *ctx);

static const transition_fn table[S_COUNT][E_COUNT] = {
        [S_IDLE]    = { [E_START] = on_start,  [E_FAULT] = on_fault },
        [S_RUNNING] = { [E_STOP]  = on_stop,   [E_FAULT] = on_fault },
        [S_ERROR]   = { [E_STOP]  = on_reset },
};

state_t step(state_t s, event_t e, void *ctx)
{
        transition_fn fn = table[s][e];
        return fn ? fn(ctx) : s;        /* NULL = ignore the event */
}
/* The table IS the specification. It can be reviewed against a
   state diagram directly, which nested switches cannot.        */
```

Weak symbols and optional hooks

```
/* Firmware pattern: provide a default that the application may
   override, with no function pointer and no runtime cost.     */

__attribute__((weak)) void board_init(void) { /* default: nothing */ }

int main(void)
{
        board_init();          /* calls the app's version if it
                                  defined one, else the default   */
}

/* The linker resolves this at link time: a strong definition
   anywhere replaces the weak one. Used heavily in CMSIS and
   vendor SDKs for interrupt vectors:                          */
__attribute__((weak)) void TIM1_IRQHandler(void) { default_handler(); }
```

Safety and security

FUNCTION POINTERS ARE AN ATTACK SURFACE

An attacker who can overwrite a function pointer controls the program counter. This is why:

- **Keep vtables in `.rodata`** — declare them `const`. A read-only page cannot be corrupted by a heap or stack overflow.
- **Always NULL-check** before calling through a pointer. A partially initialised struct is a jump to address 0.
- **Validate indices** into dispatch tables before use, and mask rather than branch in security-sensitive paths (CA ch. 12).
- **Modern mitigations exist**: ARM Pointer Authentication (`PACIASP` / `AUTIASP`) signs return addresses and can sign function pointers; Branch Target Identification (`BTI`) requires indirect branch targets to be marked; Control Flow Integrity (`-fsanitize=cfi`) checks that the target has the right type.

```
/* Build with the hardening on, on aarch64: */
gcc -mbranch-protection=standard    /* PAC + BTI */
clang -fsanitize=cfi -flto -fvisibility=hidden

/* Defensive dispatch */
static int call_op(struct file *f, int op_index)
{
        if (op_index < 0 || (size_t)op_index >= ARRAY_SIZE(f->ops->table))
                return -EINVAL;                   /* bounds check    */
        cmd_fn fn = f->ops->table[op_index];
        if (!fn)
                return -ENOSYS;                   /* NULL check      */
        return fn(f);
}
```

Performance

Call kind	Cost	Why
Direct call	Cheapest; often inlined away entirely	Target known at compile time
Indirect call, well predicted	A few cycles	The indirect branch predictor learns a stable target
Indirect call, unpredictable	10–20 cycles	Mispredict flushes the pipeline (CA ch. 12)
Indirect call with CFI	Plus a type check	Worth it in security-relevant code

The real cost is usually not the branch — it is that an indirect call is an **optimisation barrier**. The compiler cannot inline it, cannot propagate constants through it, and must assume it clobbers memory. In a hot inner loop, hoisting the dispatch outside the loop, or specialising the loop per implementation, is often a large win.

```
/* dispatch per element — the pointer blocks all optimisation */
for (size_t i = 0; i < n; i++)
        ops->process(&a[i]);

/* dispatch once — now each branch can be inlined and vectorised */
if (ops->process == fast_process)
        for (size_t i = 0; i < n; i++) fast_process(&a[i]);
else
        for (size_t i = 0; i < n; i++) ops->process(&a[i]);
```

How it shows up

Symptom	Cause
Declaration will not compile	Missing parentheses: `int *f(int)` vs `int (*f)(int)`
Jump to address 0	Called through an unset function pointer; NULL-check first
Callback needs global state	API lacks a context parameter
Crash after a struct is reallocated	Stored pointer to a member; the vtable moved
Wrong function called, or immediate crash	Signature mismatch — the pointer type lied
Works with GCC, fails with a different ABI	Casting between incompatible function pointer types
Profile dominated by indirect branch misses	Unpredictable virtual dispatch; specialise the loop
Exploit overwrites a vtable	Table was writable; declare it `const`

Symptom	Cause
`dlsym` result will not convert cleanly	ISO C does not guarantee `void*` to function pointer

Check yourself

- ✓ Declare a pointer to a function taking `const void *` and returning `int`, then an array of five of them.
- ✓ Explain why `&f` and `f` are interchangeable, and `(*p)()` and `p()` too.
- ✓ Say why converting a function pointer through `void *` is not portable.
- ✓ Design a callback API and explain why the context parameter is not optional.
- ✓ Write a dispatch-table struct and show how NULL entries are handled.
- ✓ Explain why the ops table should be `const`.
- ✓ Name three hardware or compiler mitigations for function-pointer corruption.
- ✓ Explain why an indirect call in a hot loop can cost more than the branch itself.

The preprocessor

The preprocessor is a text substitution engine that runs before the compiler sees anything. It knows nothing about types, scopes or expressions. That ignorance is the source of both its power and every macro bug you will ever write.

The mental model

> **IT IS A SEPARATE LANGUAGE**
> Preprocessing happens in translation phase 4 (ch. 01), operating on *preprocessing tokens*. There are no variables, no types and no scopes — only names, replacement lists and conditional inclusion. By the time the compiler runs, every macro has vanished.

```
# See exactly what the compiler receives. Do this whenever a macro
# misbehaves — it turns a mystery into an obvious text problem.
gcc -E file.c | less
gcc -E -P file.c            # -P strips the line markers
gcc -E -dM - < /dev/null    # every predefined macro on this target
gcc -E -dM file.c | sort    # every macro visible in this TU
```

#include

```
#include <stdio.h>      /* search the SYSTEM include path first  */
#include "myheader.h"   /* search relative to THIS file first,
                           then fall back to the system path     */

/* It is literal text substitution. The entire file's contents are
   pasted at this point, and phases 1-4 run on them recursively.  */

# where does it look?
gcc -I dir      # add dir to the quoted AND angle search path
gcc -isystem d  # add d as a system dir (suppresses warnings from it)
gcc -E -v -     # print the actual search paths in order
```

Include guards

```
/* Every header needs one, or a struct definition included twice
   becomes a redefinition error.                                  */
#ifndef MYMODULE_H
#define MYMODULE_H
...
#endif /* MYMODULE_H */

/* #pragma once is shorter, supported by every compiler you will
   meet, and is not standard. It also behaves better with symlinks
   and hard links, where a guard macro can be duplicated.       */
#pragma once

/* Use whichever your project already uses. Do not mix them. */
```

HEADER HYGIENE THAT PAYS FOR ITSELF

- **Self-contained.** A header must compile on its own — include everything it needs. Test it mechanically: `gcc -fsyntax-only header.h`.
- **Minimal.** Forward-declare (`struct foo;`) rather than including, when callers only handle pointers. This is the single biggest lever on build times in large C projects.
- **Include your own header first** in the matching `.c` file. If it is not self-contained, you find out immediately.
- **No `using`-style pollution.** Do not define short unprefixed macros (`MAX` , `OK` , `ERROR`) in a public header — they will collide.

Object-like macros

```
#define BUFFER_SIZE 1024
#define PI 3.14159265358979
#define DEVICE_NAME "uart0"

/* Prefer these to `const int` for array bounds and case labels,
   because in C a const object is NOT a constant expression
   (chapter 10).                                                */
char buf[BUFFER_SIZE];

/* But for typed constants, an enum is better: it is visible to the
   debugger, obeys scope, and is type-checked.                    */
enum { BUFFER_SIZE = 1024 };
enum state { S_IDLE, S_RUN, S_DONE };

/* C23 finally gives a real alternative: */
constexpr int buffer_size = 1024;
```

Function-like macros, and the four rules

```
/* WRONG — every one of these is a real bug */
#define SQUARE(x) x * x
#define MAX(a, b) a > b ? a : b

SQUARE(2 + 3)      /* -> 2 + 3 * 2 + 3  = 11, not 25 */
10 / SQUARE(2)     /* -> 10 / 2 * 2     = 10, not 2.5 */
MAX(1, 2) + 1      /* -> 1 > 2 ? 1 : 2 + 1 = 3, not 3... by luck */
```

THE FOUR RULES

1. **Parenthesise every parameter** where it appears.
2. **Parenthesise the whole replacement.**
3. **Evaluate each parameter exactly once**, or document that you do not.
4. **Wrap multi-statement macros** in `do { } while (0)`.

```
/* Rules 1 and 2 */
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

/* Rule 3 is the one you cannot fix with parentheses: */
MAX(i++, j++)      /* one of them is incremented TWICE */
SQUARE(f())        /* f() is CALLED TWICE              */

/* GNU statement expressions solve it — used throughout Linux: */
#define MAX(a, b) __extension__ ({      \
        __auto_type _a = (a);           \
        __auto_type _b = (b);           \
        _a > _b ? _a : _b;              \
})
/* Each argument is evaluated exactly once into a temporary. The
   leading underscores avoid capturing a caller's variable named
   `a`. Not ISO C, but available in GCC and Clang.             */

/* Rule 4: why do/while(0) rather than braces alone */
#define LOG(msg) do { timestamp(); puts(msg); } while (0)

if (x) LOG("hi"); else other();
/* With bare braces this would be:
       if (x) { ... };  else other();
   — a stray semicolon before else, which is a syntax error.
   do/while(0) makes the macro a single statement that REQUIRES
   a trailing semicolon, so it composes exactly like a function
   call.                                                       */
```

Stringification and pasting

```
#define STR(x)   #x            /* # turns a parameter into a string */
#define XSTR(x)  STR(x)        /* the extra level forces expansion  */

#define VERSION 42
STR(VERSION)     /* "VERSION"  — # does NOT expand its argument */
XSTR(VERSION)    /* "42"       — the outer call expands it first */

/* This two-level idiom is essential and non-obvious. The same
   applies to token pasting:                                    */

#define CAT(a, b)   a##b       /* ## joins two tokens          */
#define XCAT(a, b)  CAT(a, b)

#define PREFIX dev
CAT(PREFIX, _init)     /* PREFIX_init — not expanded           */
XCAT(PREFIX, _init)    /* dev_init                             */

/* A practical use: generating parallel declarations */
#define DEFINE_ACCESSOR(name, type)                     \
        static type get_##name(void) { return name; }   \
        static void set_##name(type v) { name = v; }

DEFINE_ACCESSOR(baud, uint32_t)   /* get_baud / set_baud */

/* And a debug macro that shows both expression and value */
#define TRACE(expr) \
        printf("%s:%d: %s = %d\n", __FILE__, __LINE__, #expr, (expr))
TRACE(x + y);         /* main.c:42: x + y = 7 */
```

Variadic macros

```
#define LOG(fmt, ...) fprintf(stderr, fmt, __VA_ARGS__)
LOG("x=%d\n", x);
LOG("hello\n");        /* ERROR: __VA_ARGS__ is empty, leaving a
                          trailing comma                        */

/* Three fixes, in order of preference: */

/* 1. C23 / C++20: __VA_OPT__ — standard and clean */
#define LOG(fmt, ...) fprintf(stderr, fmt __VA_OPT__(,) __VA_ARGS__)

/* 2. GNU: ##__VA_ARGS__ deletes the preceding comma if empty */
#define LOG(fmt, ...) fprintf(stderr, fmt, ##__VA_ARGS__)

/* 3. Portable: require at least the format, forward the rest */
#define LOG(...) fprintf(stderr, __VA_ARGS__)

/* Counting arguments — a classic trick worth recognising */
#define NARGS(...) NARGS_(__VA_ARGS__, 5, 4, 3, 2, 1, 0)
#define NARGS_(_1, _2, _3, _4, _5, N, ...) N
NARGS(a, b, c)        /* 3 */
```

Conditional compilation

```
#if defined(__aarch64__)
        /* ARM64 */
#elif defined(__x86_64__)
        /* x86-64 */
#else
#       error "unsupported architecture"
#endif

#ifdef DEBUG
#       define DBG(...) printf(__VA_ARGS__)
#else
#       define DBG(...) ((void)0)
#endif

/* Prefer #if over #ifdef when the macro has a VALUE — it catches
   the case where the config system defines it to 0:            */
#if CONFIG_FEATURE_X        /* 0 is false; undefined is also false */
#ifdef CONFIG_FEATURE_X     /* TRUE even when defined to 0!        */

/* Useful predefined macros */
__FILE__  __LINE__  __func__  __DATE__  __TIME__
__STDC__  __STDC_VERSION__       /* 201112L = C11, 202311L = C23 */
__GNUC__  __clang__  __ARM_ARCH  __BYTE_ORDER__
__has_include(<threads.h>)       /* C23; GCC/Clang earlier        */
__has_attribute(cleanup)
__has_builtin(__builtin_expect)
```

IFDEF SPRAWL IS A MAINTENANCE DISEASE

```
/* BAD: the logic is unreadable and only one branch is ever compiled,
   so the others silently rot.                                  */
void send(void)
{
#ifdef LINUX
        write(fd, buf, n);
#elif defined(WINDOWS)
        WriteFile(h, buf, n, &w, NULL);
#endif
}

/* BETTER: isolate platform code behind one interface, implemented
   in separate FILES selected by the build system. Then every
   line compiles on its platform and the logic stays readable. */

/* port.h */
int port_send(const void *buf, size_t n);
/* port_linux.c, port_win.c — chosen in the Makefile */

/* Where a conditional is unavoidable, prefer one that keeps both
   branches compiled: */
if (IS_ENABLED(CONFIG_FEATURE_X))    /* Linux idiom: a real `if`,
        do_x();                         evaluated at compile time,
                                        but both sides type-check  */
```

Compile-time assertions

```
#include <assert.h>

/* C11: fails the BUILD, not the program. Free, and enormously
   valuable for layout assumptions.                             */
static_assert(sizeof(struct packet) == 16, "packet layout changed");
static_assert(sizeof(void *) == 8, "64-bit only");
static_assert(CHAR_BIT == 8, "assumes 8-bit bytes");
_Static_assert(offsetof(struct regs, FR) == 0x18, "FR offset");

/* Pre-C11 idiom you will still meet in older code */
#define STATIC_ASSERT(c, name) \
        typedef char static_assert_##name[(c) ? 1 : -1]

/* Put these next to every struct that describes hardware or a wire
   format (chapter 07). A compiler upgrade or an ABI change that
   shifts padding then becomes a build failure instead of a
   runtime mystery.                                             */
```

X-macros

```
/* One list, many derived artefacts, guaranteed in sync. This is the
   most useful advanced preprocessor technique.                  */

#define ERROR_LIST                      \
        X(ERR_NONE,    0, "success")    \
        X(ERR_TIMEOUT, 1, "timed out")  \
        X(ERR_NOMEM,   2, "out of memory")

/* generate the enum */
#define X(name, val, str) name = val,
enum error { ERROR_LIST };
#undef X

/* generate the string table from the SAME list */
#define X(name, val, str) [val] = str,
static const char *const error_strings[] = { ERROR_LIST };
#undef X

const char *strerr(enum error e)
{
        return (size_t)e < ARRAY_SIZE(error_strings)
                ? error_strings[e] : "unknown";
}

/* Adding an error is ONE line. The enum and the strings can never
   drift apart — which is exactly the bug this prevents.        */
```

Prefer the alternatives

Instead of	Use	Why
`#define MAX_N 10`	`enum { MAX_N = 10 };`	Typed, scoped, visible in the debugger
A short function macro	`static inline`	Type-checked, single evaluation, debuggable, same code generated
`#define SQUARE(x)`	`static inline int square(int)`	No double evaluation, no parenthesis traps
Type-dispatch macros	`_Generic` (ch. 11)	Compile-time, type-safe
`#ifdef` around logic	`if (IS_ENABLED(...))` or separate files	Both branches keep compiling
`#define DEBUG_PRINT`	A real logging function with the `format` attribute	Format checking

Macros remain irreplaceable for: include guards, conditional compilation, stringification and pasting, capturing `__FILE__`/`__LINE__` at the call site, X-macros, and anything that must work in a constant expression.

Debugging macros

```
# the first thing to do, always
gcc -E problem.c | grep -A5 'my_macro'

# clang-format the output to make it readable
gcc -E -P problem.c | clang-format

# see where a macro was defined
gcc -E -dD problem.c | grep 'define MY_MACRO'

# a macro redefined with a different value
gcc -Wall            # -Wmacro-redefined / "redefined" warning

# check a header is self-contained
gcc -fsyntax-only -x c myheader.h

# undefine from the command line
gcc -UDEBUG -DLOG_LEVEL=3
```

How it shows up

Symptom	Cause
Arithmetic macro gives a wrong result in an expression	Missing parentheses around parameters or the whole body
A side effect happens twice	Parameter used more than once in the replacement
Syntax error at `else`	Multi-statement macro not wrapped in `do { } while (0)`
Stringified macro shows the name, not the value	Missing the second expansion level
Trailing comma error with no variadic arguments	Need `__VA_OPT__` or `##__VA_ARGS__`
Feature enabled despite being configured off	`#ifdef` on a macro defined to `0`; use `#if`
"redefinition of struct" errors	Missing include guard
Header compiles in one file, fails in another	Not self-contained; depends on include order
Enum and its string table drift apart	Two hand-maintained lists; use an X-macro
Register offsets wrong after a compiler upgrade	No `static_assert` on the layout

Check yourself

- ✓ Explain why the preprocessor cannot type-check anything.
- ✓ Give the four macro rules and a bug each one prevents.
- ✓ Explain why `do { } while (0)` is the correct wrapper.
- ✓ Show why stringification needs two macro levels.
- ✓ Give the difference between `#ifdef X` and `#if X`.
- ✓ Write an X-macro generating an enum and a matching string table.
- ✓ Write a `static_assert` guarding a hardware struct's layout.
- ✓ Name three things macros are still the right tool for.

Dynamic memory and its failure modes

C gives you four functions and no safety net. Every use-after-free, double free, leak and heap overflow in the history of computing came through this interface. This chapter covers using it correctly, the ownership discipline that makes it tractable, and why embedded systems often ban it outright.

The four functions

```
void *malloc(size_t size);              /* uninitialised            */
void *calloc(size_t n, size_t size);    /* zeroed, checked multiply */
void *realloc(void *p, size_t size);    /* resize, may move         */
void  free(void *p);                    /* release; free(NULL) is a no-op */

/* correct malloc usage */
int *p = malloc(n * sizeof *p);    /* `sizeof *p`, not sizeof(int) —
                                      stays correct if the type changes */
if (!p)
        return -ENOMEM;            /* ALWAYS check                      */

/* Do NOT cast the result in C. The cast is required in C++, is
   noise in C, and historically hid the missing-prototype bug.    */
```

THE MULTIPLICATION IS THE VULNERABILITY

```
void *p = malloc(n * sizeof(struct item));
/* If n is attacker-controlled and large, n * sizeof(...) WRAPS.
   You allocate 16 bytes, then write n elements into it.
   This is a textbook heap overflow.                          */

/* calloc checks the multiplication internally — use it: */
void *p = calloc(n, sizeof(struct item));

/* or check explicitly */
if (n > SIZE_MAX / sizeof(struct item))
        return -EOVERFLOW;

/* or use a builtin */
size_t bytes;
if (__builtin_mul_overflow(n, sizeof(struct item), &bytes))
        return -EOVERFLOW;

/* Linux kernel helpers that do this for you: */
kmalloc_array(n, size, GFP_KERNEL);
kcalloc(n, size, GFP_KERNEL);
kmalloc(struct_size(p, items, n), GFP_KERNEL);   /* flexible arrays */
```

realloc, and the leak everyone writes once

```
/* WRONG: if realloc fails it returns NULL and the ORIGINAL block
   is still allocated — you have just leaked it and lost the data. */
p = realloc(p, new_size);
if (!p) return -ENOMEM;      /* the old block is now unreachable */

/* RIGHT: keep the original until you know the new one exists */
void *tmp = realloc(p, new_size);
if (!tmp) {
        free(p);             /* or keep using p — your choice, but
                                make it deliberately */
        return -ENOMEM;
}
p = tmp;

/* Other realloc facts worth knowing:
     realloc(NULL, n)  == malloc(n)
     realloc(p, 0)     is implementation-defined in C17 and
                       UNDEFINED in C23 — never rely on it as free
     the block MAY MOVE, so every pointer into it is invalidated  */

/* That last point causes real bugs: */
int *elem = &vec->data[3];
vec_push(vec, x);            /* may realloc */
*elem = 5;                   /* DANGLING — store an index, not a pointer */
```

Ownership: the discipline that makes this work

> **EVERY ALLOCATION HAS EXACTLY ONE OWNER**
>
> The owner is responsible for freeing it. Ownership may be *transferred*, but it is never shared implicitly. Write down, in the function's documentation or its name, who owns what. C gives you no enforcement, so the convention is all you have — which is precisely why it must be explicit.

```
/* Document ownership in the API. These are the three patterns: */

/* 1. Caller owns — the function borrows */
size_t parse(const char *input, struct result *out);
/*     ^ const: we only read it        ^ caller-provided storage */

/* 2. Callee allocates, caller owns — transfer OUT */
struct buffer *buffer_create(size_t n);   /* caller must
void buffer_destroy(struct buffer *b);       call destroy */

/* 3. Callee takes ownership — transfer IN */
void queue_push(struct queue *q, char *msg);   /* q now owns msg;
                                                  caller must not
                                                  free or reuse it */
/* Make this obvious in the name if it is not obvious from context: */
void queue_push_owned(struct queue *q, char *msg);

/* Matching create/destroy pairs, consistently named, are the single
   most effective convention in C memory management:            */
foo_create / foo_destroy
foo_alloc  / foo_free
foo_new    / foo_delete        /* pick ONE style per project */
```

Patterns that reduce the surface

The goto cleanup ladder

```
/* Idiomatic C, and the reason `goto` is not banned in kernel code.
   Resources are released in exactly reverse order of acquisition. */
int setup(struct dev *d)
{
        int ret;

        d->buf = malloc(SIZE);
        if (!d->buf)
                return -ENOMEM;

        d->fd = open_device();
        if (d->fd < 0) {
                ret = d->fd;
                goto err_free;
        }

        ret = start_thread(d);
        if (ret)
                goto err_close;

        return 0;

err_close:
        close(d->fd);
err_free:
        free(d->buf);
        d->buf = NULL;
        return ret;
}
/* Labels named for what they UNDO, not where they are. One exit
   point per resource. This scales to a dozen resources without
   nesting.                                                      */
```

Scope-based cleanup

```
/* GCC/Clang `cleanup` attribute — RAII in C. Adopted by the Linux
   kernel in 6.5 and spreading through systemd and glib.       */
static inline void freep(void *p) { free(*(void **)p); }
#define _cleanup_free_ __attribute__((cleanup(freep)))

int process(void)
{
        _cleanup_free_ char *buf = malloc(1024);
        if (!buf)
                return -ENOMEM;

        if (early_error)
                return -EIO;        /* freed automatically */

        return 0;                   /* freed automatically */
}

/* Linux 6.5+ provides this properly, with type safety: */
#include <linux/cleanup.h>
struct foo *f __free(kfree) = kmalloc(sizeof(*f), GFP_KERNEL);
guard(mutex)(&lock);                /* unlocked on scope exit */
```

Arena and pool allocation

```
/* When many objects share a lifetime, allocate them from one arena
   and free the whole arena at once. No per-object bookkeeping, no
   leaks, and dramatically faster.                                  */

struct arena { char *base; size_t size, used; };

void *arena_alloc(struct arena *a, size_t n)
{
        n = (n + 15) & ~(size_t)15;             /* align to 16 */
        if (a->used + n > a->size)
                return NULL;
        void *p = a->base + a->used;
        a->used += n;
        return p;
}
void arena_reset(struct arena *a) { a->used = 0; }   /* frees everything */

/* Ideal for: per-request server allocations, compiler AST nodes,
   parsing, frame-based game logic. The tradeoff is that you cannot
   free individual objects — which is exactly what removes the
   use-after-free class.                                            */

/* A fixed-size POOL is the embedded variant: pre-allocate N objects,
   hand them out from a free list, never fragment.                  */
```

The failure modes

Bug	What happens	Detection
Leak	Memory never freed; process grows until OOM	ASan (`LeakSanitizer`), Valgrind
Use after free	Reads garbage, or writes into a reallocated object	ASan, Valgrind, KASAN
Double free	Corrupts allocator metadata; often exploitable	ASan, glibc's own checks
Heap overflow	Writes past the block into adjacent data or metadata	ASan, `_FORTIFY_SOURCE`
Invalid free	Freeing a stack pointer, an interior pointer, or a non-pointer	ASan, glibc
Mismatched allocator	`free` on `new`, or `kfree` on `vmalloc`	Review; some checkers
Uninitialised read	`malloc` memory contains old data	MSan, Valgrind
Fragmentation	Free memory exists but no contiguous block is large enough	Long-running tests; arena/pool design

```
/* The habits that prevent most of these */

free(p);
p = NULL;         /* a later use crashes loudly; a second free is a no-op */

/* Free in the same layer that allocated. If a function allocates
   on some paths and not others, that is a design smell.        */

/* Never free something you did not allocate, and never hand a
   pointer to a different allocator:                            */
kfree(vmalloc_ptr);        /* kernel: corrupts the slab allocator */

/* Zero sensitive data before freeing (chapter 06) — and remember
   plain memset may be optimised away.                          */
explicit_bzero(key, sizeof key);
free(key);
```

Finding them

```
# AddressSanitizer — the default choice. ~2x slower; run everything under it.
gcc -fsanitize=address -fno-omit-frame-pointer -g prog.c
ASAN_OPTIONS=detect_leaks=1:abort_on_error=1 ./prog

# MemorySanitizer — uninitialised reads (Clang only, needs all deps rebuilt)
clang -fsanitize=memory -fno-omit-frame-pointer -g prog.c

# Valgrind — no rebuild, ~20x slower, finds slightly different things
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog

# glibc's built-in checks, free at runtime
MALLOC_CHECK_=3 ./prog
MALLOC_PERTURB_=165 ./prog      # fill freed memory with a pattern

# static analysis
gcc -fanalyzer                  # GCC 10+; finds many leaks and UAFs
clang --analyze
cppcheck --enable=all

# in the kernel
CONFIG_KASAN=y  CONFIG_DEBUG_KMEMLEAK=y  CONFIG_DEBUG_OBJECTS=y
```

Why embedded systems restrict or ban it

Problem	Consequence in an embedded context
Fragmentation	Over weeks of uptime, allocation fails despite free memory. No way to recover without a reboot.
Non-determinism	`malloc` has no bounded worst-case time. Unacceptable in a hard real-time path.
No recovery path	What does a brake controller *do* when `malloc` returns NULL?
Unbounded footprint	Static analysis cannot prove the system fits in RAM
Silent failure	Heap corruption in a system with no MMU corrupts anything, anywhere

```
/* The standard alternatives, in rough order of preference: */

/* 1. Static allocation — everything sized at build time */
static struct session sessions[MAX_SESSIONS];
static uint8_t rx_buffer[MAX_FRAME];
/* The linker tells you at BUILD TIME whether it fits. */

/* 2. Fixed-size pools */
static struct msg pool[64];
static uint64_t pool_used;             /* a bitmap */
struct msg *msg_alloc(void)
{
        int i = __builtin_ffsll(~pool_used) - 1;
        if (i < 0) return NULL;
        pool_used |= 1ull << i;
        return &pool[i];
}

/* 3. Allocate only at init, never at runtime — the MISRA and
      DO-178C compromise. Startup allocation is deterministic
      because nothing has fragmented yet.                      */

/* MISRA C:2012 Rule 21.3 bans malloc/calloc/realloc/free outright
   in safety-critical code. AUTOSAR and DO-178C impose similar
   restrictions.                                               */
```

What the allocator is doing

```
A typical glibc heap block

   +-----------+-----------------------------+---------+
   |  header   |        your data            | padding |
   |  size,    |  <- malloc returns THIS     |         |
   |  flags    |                             |         |
   +-----------+-----------------------------+---------+
    8-16 bytes

  · malloc(1) really costs 32 bytes. Many tiny allocations are
    mostly overhead — a reason to use pools or arenas.
  · The header sits immediately BEFORE your data, which is why a
    single-byte underflow corrupts allocator metadata, and why
    that is so often exploitable.
  · free() reads that header to learn the size. Passing an
    interior pointer therefore reads garbage as a size.
  · Freed blocks go on size-class free lists ("bins"); adjacent
    free blocks are coalesced. This is where fragmentation lives.
  · Large requests (>128 KB by default) go straight to mmap and
    are returned to the OS on free — which is why RSS sometimes
    drops after free and sometimes does not.

Alternative allocators worth knowing: jemalloc and tcmalloc
(better multithreaded scaling), mimalloc, and for embedded,
TLSF (bounded worst-case time) and FreeRTOS heap_4/heap_5.
```

How it shows up

Symptom	Cause
Crash far from the faulty code	Heap corruption from an earlier overflow

Symptom	Cause
"free(): invalid pointer"	Double free, interior pointer, or a stack address
"malloc(): corrupted top size"	Heap overflow overwrote allocator metadata
Process RSS grows without bound	Leak — or fragmentation preventing return to the OS
Works with small input, corrupts with large	Size calculation overflowed
Data changes after a container append	`realloc` moved the block; stored pointer dangles
Leak only on the error path	Missing cleanup in an early return; use the goto ladder
Memory lost when an allocation fails	`p = realloc(p, n)` overwrote the original pointer
Long-running device eventually fails to allocate	Fragmentation; switch to pools
Values differ between debug and release	Reading uninitialised `malloc` memory

Check yourself

- ✓ Write a correct allocation of `n` items with the overflow check.
- ✓ Show the `realloc` leak and the correct form.
- ✓ Name the three ownership patterns and how you would document each in an API.
- ✓ Write a goto cleanup ladder for three resources, in the right order.
- ✓ Explain what the `cleanup` attribute does and what problem it solves.
- ✓ Explain when an arena allocator is the right choice and what it gives up.
- ✓ Give four reasons embedded systems ban dynamic allocation, and two alternatives.
- ✓ Explain why a one-byte heap underflow is often exploitable.

Undefined, unspecified and implementation-defined behaviour

Three categories that sound similar and differ enormously. One is a portability note, one is a harmless choice, and one means the standard imposes no requirements at all — and modern optimisers act on that fact in ways that delete your safety checks.

The three categories

Category	Meaning	Must be documented?	Example
Implementation-defined	The implementation chooses, and must document its choice. Behaviour is consistent.	Yes	Size of `int`; whether plain `char` is signed; right shift of a negative value
Unspecified	The implementation chooses from several valid options, and need not document or be consistent.	No	Order of evaluation of function arguments; padding byte contents
Undefined	**The standard imposes no requirements whatsoever.** Anything may happen, including nothing, including time travel.	No	Signed overflow, null dereference, out-of-bounds access, data race

WHY UB IS NOT "WHATEVER THE HARDWARE DOES"

The common intuition — "signed overflow wraps, because the CPU wraps" — is wrong, and dangerously so. The compiler does not translate your code to instructions; it reasons about your code *assuming undefined behaviour never occurs*, and uses that assumption to justify transformations. If UB does occur, the assumptions were false, and the conclusions drawn from them can be arbitrary.

Crucially, the effects are **not local**. UB can cause code *before* the offending statement to misbehave, because the optimiser propagates the assumption backwards.

Optimisations that delete your code

```
/* 1. The overflow check that vanishes */
bool check(int a, int b) { return a + b < a; }
/* Signed overflow is UB, so the compiler assumes a + b never
   overflows. If it never overflows, a + b < a is equivalent to
   b < 0. And with -fstrict-overflow it may simplify further.
   GCC compiles the classic form `a + b < a` to `b < 0` — which
   is not the check you wrote.                                  */

/* 2. The null check that vanishes  (Linux CVE-2009-1897) */
struct tun *tun = ptr;
struct sock *sk = tun->sk;          /* dereference implies non-NULL */
if (!tun)                            /* ...so this is dead code     */
        return -EINVAL;              /* DELETED by the optimiser    */
/* Result: a null dereference that the author had explicitly
   guarded against. Order matters: check FIRST, then dereference. */

/* 3. The infinite loop that vanishes */
while (1) { }
/* A loop with no side effects and no exit is UB in C11 (6.8.5p6).
   The compiler may assume it terminates and delete it, letting
   control fall through to whatever follows.                     */

/* 4. The bounds check that vanishes */
int f(int *p, int i)
{
        int x = p[i];                /* implies i is in bounds...  */
        if (i < 0 || i >= 10)        /* ...so this is redundant    */
                return -1;
        return x;
}

/* 5. Unreachable code becomes reachable */
int f(int x)
{
        if (x > 0) return 1;
        if (x < 0) return -1;
        if (x == 0) return 0;
        /* no return here — falling off the end of a non-void
           function is UB, so the compiler may emit anything,
           including running into the NEXT function.            */
}
```

The UB you will actually hit

Undefined behaviour	Correct approach
Signed integer overflow	`__builtin_add_overflow`, or use unsigned, or check against `INT_MAX`
Division by zero; `INT_MIN / -1`	Check both before dividing
Shift ≥ width, or negative shift count	Mask the count; use unsigned types (ch. 08)
Out-of-bounds array access	Bounds-check every externally supplied index
Null pointer dereference	Check before, never after
Use after free; double free	Set to NULL after freeing (ch. 14)
Reading an uninitialised object	Initialise at declaration
Strict aliasing violation	`memcpy` or a union (below)

Undefined behaviour	Correct approach
Misaligned pointer access	`memcpy`; never cast a byte pointer to a wider type
Unsequenced modification (`i = i++`)	Split into statements (ch. 03)
Data race	`_Atomic` or a lock — `volatile` is not enough
Falling off the end of a non-void function	Return on every path; `-Wreturn-type`
Modifying a string literal	`const char *` for literals
`memcpy` with overlapping ranges	`memmove`
Passing a negative `char` to `<ctype.h>`	Cast to `unsigned char`
Freeing a non-heap or interior pointer	Free exactly what was allocated

Strict aliasing

> **THE RULE**
> An object may only be accessed through an lvalue of a *compatible* type, a signed/unsigned variant of it, a qualified version, an aggregate containing it, or **any character type**. Accessing an object through an incompatible pointer type is undefined.
>
> The rule exists so the compiler can assume that a `float *` and an `int *` never refer to the same storage — which allows it to keep values in registers across stores. Without it, every store through any pointer would invalidate every cached load.

```
/* The classic violation — "type punning" through a pointer cast */
float f = 1.0f;
uint32_t bits = *(uint32_t *)&f;      /* UNDEFINED */

/* At -O2 this may read a stale register, or reorder relative to the
   store to f, and it genuinely misbehaves on real compilers.     */

/* CORRECT 1: memcpy. Compiles to zero instructions at -O2 — the
   compiler recognises the idiom.                                 */
uint32_t bits;
memcpy(&bits, &f, sizeof bits);

/* CORRECT 2: a union. Legal in C (since C99 TC3), NOT in C++. */
union { float f; uint32_t u; } cvt = { .f = 1.0f };
uint32_t bits = cvt.u;

/* CORRECT 3: character types may alias anything — this is how
   serialisation code is allowed to work at all.               */
unsigned char *bytes = (unsigned char *)&f;
for (size_t i = 0; i < sizeof f; i++) put(bytes[i]);

/* An escape hatch, if you have inherited code you cannot fix: */
gcc -fno-strict-aliasing      /* the Linux kernel builds with this */

/* But note it costs real performance, and it is a workaround, not
   a fix. New code should be alias-correct.                     */
```

Alignment travels with aliasing

```
/* This is a second, independent bug in the same line of code */
uint8_t buf[100];
uint32_t v = *(uint32_t *)(buf + 1);    /* misaligned AND aliasing */

/* On ARM with strict alignment this faults; on x86 it is merely
   slow; and either way the compiler may assume 4-byte alignment
   and emit an instruction that requires it.                    */

/* Correct: */
uint32_t v;
memcpy(&v, buf + 1, sizeof v);          /* alignment-safe, alias-safe,
                                           and compiles to one unaligned
                                           load where the target allows */
```

Finding UB

```
# UndefinedBehaviorSanitizer — the single most valuable tool here.
# Reports the exact file, line and values at the moment it happens.
gcc -fsanitize=undefined -fno-sanitize-recover=all -g prog.c

# individual checks, if you want them separately
-fsanitize=signed-integer-overflow
-fsanitize=shift
-fsanitize=bounds
-fsanitize=alignment
-fsanitize=null
-fsanitize=object-size
-fsanitize=bool,enum
-fsanitize=float-cast-overflow

# combine with ASan for memory UB
gcc -fsanitize=address,undefined -g prog.c

# compile-time help
gcc -Wall -Wextra -Wstrict-aliasing=3 -Wcast-align \
    -Wstrict-overflow=5 -Wsequence-point -Wreturn-type \
    -Wuninitialized -Wnull-dereference -fanalyzer

# make some UB defined, where you must
-fwrapv               # signed overflow wraps (kills the optimisation
                      # AND the check-deletion behaviour)
-fno-strict-aliasing  # disable the aliasing assumption
-fno-delete-null-pointer-checks
# The Linux kernel uses all three. They are a pragmatic choice for
# a huge legacy codebase — not a licence to write UB in new code.
```

> **RUN YOUR ENTIRE TEST SUITE UNDER UBSAN AND ASAN**
> This is the highest-value habit in this chapter. UB is invisible until the day an optimiser or a compiler upgrade exposes it, and by then it is a production incident. Sanitizers cost a few times the runtime in CI and find these bugs the day they are written. Both are in GCC and Clang; there is no reason not to.

Unspecified behaviour worth knowing

```
/* Legal, but the choice is not documented and may vary. */

f(g(), h());              /* order of g and h: unspecified        */
a[i++] = i;               /* actually UNDEFINED — unsequenced      */
p1 < p2                   /* comparing pointers into different
                             objects: unspecified                  */
padding bytes             /* contents are unspecified — which is
                             why memcmp on structs is wrong
                             (chapter 07)                          */
malloc(0)                 /* may return NULL or a unique pointer   */

/* Do not write code that depends on any of these, but note that
   unlike UB, they cannot make unrelated code misbehave.        */
```

Implementation-defined behaviour worth knowing

Behaviour	Typical values	How to be portable
Size of `int`, `long`	32/64 vs 32/32	`<stdint.h>` fixed-width types
Plain `char` signedness	Signed on x86, unsigned on ARM Linux	Say `signed` / `unsigned` explicitly
Right shift of a negative value	Arithmetic everywhere in practice	Use unsigned types
Conversion of out-of-range value to signed	Truncation	Check the range first
Bitfield allocation order	Varies with ABI and endianness	Masks and shifts (ch. 07)
Struct padding	Varies with ABI	`static_assert` on offsets
`NULL`'s representation	All-zero bits, universally	Do not `memset` pointers to zero and assume NULL

How to think about it

```
The bargain C makes:

  The standard leaves things undefined so implementations can be
  FAST on wildly different hardware — one's-complement machines,
  word-addressed machines, machines that trap on overflow. The
  optimiser gets to assume the programmer stayed inside the lines.

  The programmer's side of the bargain is to STAY INSIDE THE LINES.

  What makes this hard in practice:
    · UB is silent — no warning by default
    · effects are non-local and can appear far from the cause
    · behaviour can change with optimisation level, compiler
      version, inlining decisions, or unrelated edits
    · "it worked before" is evidence of nothing

  The practical response is not to memorise the standard. It is:
    1. enable the sanitizers, in CI, on every test
    2. enable the warnings, and treat them as errors
    3. learn the ~15 UB cases in the table above by heart
    4. when something behaves differently at -O2 than at -O0,
       suspect UB FIRST, not a compiler bug
```

How it shows up

Symptom	Cause
Works at `-O0`, fails at `-O2`	UB the optimiser is now exploiting
A safety check appears to be skipped	Deleted because it was provably dead given a UB assumption
Behaviour changes after a compiler upgrade	New optimisation exploiting existing UB
Adding an unrelated `printf` "fixes" it	Inlining or register allocation changed; UB is still there
Type punning gives stale values	Strict aliasing violation
Fault reading a struct field from a byte buffer	Misaligned access; use `memcpy`
Overflow detection never triggers	Signed overflow check optimised away
Program continues past a fatal error	Fell off the end of a non-void function
Different results on ARM and x86	Implementation-defined `char` signedness, or UB
Loop that should never end runs once	Side-effect-free infinite loop assumed to terminate

Check yourself

- ✓ Define the three categories and give an example of each.
- ✓ Explain why UB effects are not local to the offending line.
- ✓ Show why `a + b < a` is not a valid overflow check, and write one that is.
- ✓ Explain the null-check deletion bug and the ordering rule that prevents it.
- ✓ State the strict aliasing rule and give three legal ways to type-pun.
- ✓ Explain why `*(uint32_t *)(buf + 1)` has two separate defects.
- ✓ Name the sanitizer flags you would put in CI and what each catches.

✓ Explain what `-fwrapv` does and why the kernel uses it.

The standard library for systems programmers

The C standard library is small, old, and uneven. Some of it is indispensable, some is actively harmful, and much of it disappears entirely in freestanding builds. This chapter is a working engineer's map of what to use, what to avoid, and what you lose in the kernel.

The headers, sorted by usefulness

Header	Contents	Freestanding?	Verdict
`<stdint.h>`	Fixed-width integer types	**Yes**	Use constantly
`<stddef.h>`	`size_t`, `ptrdiff_t`, `NULL`, `offsetof`	**Yes**	Use constantly
`<stdbool.h>`	`bool`, `true`, `false`	**Yes**	Use (built in from C23)
`<limits.h>`	`INT_MAX`, `CHAR_BIT`, …	**Yes**	Essential for range checks
`<stdarg.h>`	Varargs machinery	**Yes**	Only for printf-like APIs
`<stdalign.h>`, `<stdnoreturn.h>`	C11 keywords as macros	**Yes**	Useful
`<string.h>`	`mem*`, `str*`	No	Use `mem*`; be careful with `str*` (ch. 06)
`<stdlib.h>`	malloc, strtol, qsort, exit, abs	No	Use selectively
`<stdio.h>`	Files and formatted I/O	No	Heavy; often replaced in embedded
`<inttypes.h>`	`PRIu32` and friends	No	Use whenever you print fixed-width types
`<errno.h>`	`errno` and codes	No	Necessary evil
`<assert.h>`	`assert`, `static_assert`	Partly	`static_assert` always; `assert` carefully
`<stdatomic.h>`	C11 atomics	Mostly	Use for concurrency
`<math.h>`	Floating point	No	Pulls in a large library; often unavailable
`<time.h>`	Clocks and calendars	No	API is dated; see below
`<setjmp.h>`	Non-local jumps	No	Avoid
`<signal.h>`	Signal handling	No	Very restricted; see below
`<locale.h>`, `<wchar.h>`	Locales, wide chars	No	Avoid in systems code

The mem* functions

```
memcpy(dst, src, n)     /* non-overlapping only — restrict (ch. 10) */
memmove(dst, src, n)    /* handles overlap                          */
memset(dst, c, n)       /* c is converted to unsigned char          */
memcmp(a, b, n)         /* byte comparison; NOT for structs (ch. 07)
                           and NOT constant time (ch. 06)           */
memchr(s, c, n)         /* find a byte                              */

/* These are the fastest primitives you have. Modern implementations
   are hand-written SIMD, dispatched by CPU feature at load time.
   Do not write your own copy loop.                                 */

/* The compiler also generates calls to them from ordinary code: */
struct big a = b;            /* may become memcpy */
int arr[1000] = {0};         /* may become memset */
/* Which is why freestanding builds must PROVIDE memcpy/memset even
   if the source never calls them (chapter 20).                  */

/* memset's second argument is an int, converted to unsigned char.
   This does not do what it looks like:                          */
memset(arr, 1, n * sizeof(int));    /* sets every BYTE to 1, giving
                                       0x01010101, not 1          */
/* Only 0 is safe to splat across a multi-byte type. Use a loop
   otherwise, or memset_pattern where available.                 */
```

printf, and its cost

```
/* The format specifiers that matter, and are commonly wrong */
%d %u %x       int / unsigned / hex
%ld %lu        long
%lld %llu      long long
%zu %zd        size_t / ssize_t          <- not %d, not %lu
%td            ptrdiff_t
%p             void * — you MUST cast:  printf("%p", (void *)ptr)
%s             char * — never a non-literal FORMAT (ch. 06)
%f %e %g       double (float is promoted)
%%             a literal percent

/* fixed-width types need <inttypes.h> */
#include <inttypes.h>
printf("%" PRIu32 " %" PRIx64 "\n", a, b);

/* Field width and precision from arguments */
printf("%*d\n", width, value);
printf("%.*s\n", len, str);        /* print exactly len bytes —
                                      useful for non-terminated data */
```

> **PRINTF IS ENORMOUS**
>
> A full `printf` with floating-point support costs roughly 10–40 KB of flash and needs a sizeable stack frame. On a microcontroller with 64 KB of flash that is unacceptable. Options: `-u _printf_float` to omit float support in newlib-nano, a minimal reimplementation (`tinyprintf`, `mpaland/printf`), or a binary logging scheme that sends a format ID plus arguments and reconstructs the text on the host. The kernel's `printk` is its own implementation for exactly these reasons — it also cannot use floating point at all.

errno

```
#include <errno.h>

/* errno is thread-local (a macro expanding to a per-thread lvalue).
   The rules that people get wrong:                                  */

/* 1. It is only meaningful after a function that DOCUMENTS setting
      it, and only when that function reported failure.           */
if (fopen(...) == NULL)
        perror("fopen");            /* correct */

/* 2. A successful call may still CLOBBER errno. Never test errno
      to decide whether something failed.                         */
foo();
if (errno) ...                       /* WRONG */

/* 3. For functions that return a valid value on both success and
      failure (strtol, strtod), you MUST clear errno first:       */
errno = 0;
long v = strtol(s, &end, 10);
if (errno == ERANGE) ...

/* 4. Save it immediately if you do anything else first: */
if (write(...) < 0) {
        int saved = errno;
        cleanup();                   /* may clobber errno */
        return -saved;
}

/* Printing it */
perror("context");                   /* writes "context: message\n" */
fprintf(stderr, "%s\n", strerror(errno));       /* not thread-safe   */
strerror_r(errno, buf, sizeof buf);             /* thread-safe       */
```

The kernel does not use `errno` at all: functions return `0` or a negative errno value directly (`-EINVAL`, `-ENOMEM`). This is a strictly better design — the error is in the return value, so it cannot be clobbered or forgotten — and it is worth adopting in your own C libraries.

assert, and when to use it

```
#include <assert.h>

assert(ptr != NULL);        /* aborts if false; compiled out by NDEBUG */

/* THE RULE: assert on PROGRAMMER errors (invariants that must hold
   if the code is correct). Handle USER and ENVIRONMENT errors with
   real error paths.                                              */

assert(index < ARRAY_SIZE(table));   /* internal invariant — assert */
if (!input_valid(s)) return -EINVAL; /* external input — check      */

/* The classic bug: side effects inside assert */
assert(do_something() == 0);         /* vanishes in release builds! */
int r = do_something();
assert(r == 0);                       /* correct */

/* Give it a message by ANDing a string literal — a widely used trick */
assert(n > 0 && "count must be positive");

/* static_assert costs nothing at runtime and never disappears:  */
static_assert(sizeof(struct hdr) == 8, "header layout");

/* In the kernel: */
BUG_ON(cond);       /* panic — reserve for unrecoverable corruption */
WARN_ON(cond);      /* log a stack trace and CONTINUE — prefer this */
WARN_ON_ONCE(cond);
BUILD_BUG_ON(cond); /* compile time */
```

Time

```
#include <time.h>

/* time_t is seconds since the epoch. On 32-bit systems it overflows
   in 2038 — still a live concern on embedded hardware shipping now.
   Use 64-bit time_t (glibc 2.34+ with _TIME_BITS=64) or your own.  */

/* For MEASURING DURATIONS, never use wall-clock time — it jumps
   backwards with NTP, DST and manual changes.                    */
struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);
work();
clock_gettime(CLOCK_MONOTONIC, &t1);
double sec = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;

/* Clock choices that matter:
     CLOCK_REALTIME       wall clock; can jump. For timestamps only.
     CLOCK_MONOTONIC      never jumps; may pause in suspend. Durations.
     CLOCK_BOOTTIME       monotonic, includes suspend time.
     CLOCK_PROCESS_CPUTIME_ID   CPU time used, for profiling.       */

/* The C standard's clock() returns CPU time in CLOCKS_PER_SEC
   units and is nearly useless for wall-clock measurement.       */
```

Signals: async-signal-safety

```
/* A signal handler interrupts the program at an arbitrary point,
   possibly in the middle of malloc or printf. Almost nothing is
   safe to call from one.                                          */

/* SAFE inside a handler: a short list, including
     write(), _exit(), signal(), sigaction(), and assignment to a
     volatile sig_atomic_t.

   NOT safe: malloc, free, printf, any stdio, most of libc.      */

static volatile sig_atomic_t got_sigint = 0;

static void handler(int sig) { got_sigint = 1; }   /* that is ALL */

int main(void)
{
        struct sigaction sa = { .sa_handler = handler };
        sigemptyset(&sa.sa_mask);
        sigaction(SIGINT, &sa, NULL);      /* prefer over signal() */

        while (!got_sigint)
                do_work();                  /* the real work happens here */
}

/* The modern alternative on Linux: signalfd() or a self-pipe, which
   turn a signal into a readable file descriptor you can poll —
   removing the async-safety problem entirely.                     */
```

What to avoid outright

Avoid	Why	Instead
`gets()`	Unconditionally exploitable; removed in C11	`fgets`, `getline`
`strcpy`, `strcat`, `sprintf`	Unbounded writes	`snprintf`, `strlcpy`
`strncpy`	May not terminate; always pads	`snprintf`, `strscpy`
`atoi`, `atof`	No error reporting; UB on overflow	`strtol`, `strtod` with checks
`strtok`	Global state; modifies input	`strtok_r`, or hand-written
`rand()`	Poor quality; not thread-safe; never for security	`getrandom()`, `arc4random`, a proper PRNG
`setjmp` / `longjmp`	Skips cleanup; interacts badly with optimisation	Error returns
`system()`	Shell injection	`posix_spawn`, `fork` + `exec`
`alloca()`	Unbounded stack growth; no failure path	Fixed buffer or heap
`gmtime`, `asctime`, `strerror`	Return pointers to static buffers — not reentrant	The `_r` variants

What the kernel gives you instead

Userspace	Linux kernel	Note
`malloc` / `free`	`kmalloc` / `kfree`, `kvmalloc`, `devm_kzalloc`	Needs a GFP flag for context
`printf`	`printk`, `pr_info`, `dev_err`	No floating point
`memcpy`, `strlen`	Same names, kernel implementations	Available
`snprintf`	`snprintf`, `scnprintf`	`scnprintf` returns bytes actually written
`strtol`	`kstrtoint`, `kstrtoul`	Returns an error code, not `errno`
`assert`	`WARN_ON`, `BUG_ON`, `BUILD_BUG_ON`	Prefer `WARN_ON`
`qsort`	`sort()`	In `lib/sort.c`
`errno`	Negative return values	Better design
Floating point	**Forbidden**	FP state is not saved on kernel entry
`time()`	`ktime_get()`, `jiffies`	

How it shows up

Symptom	Cause
`printf("%d", sizeof x)` prints garbage	Wrong specifier for `size_t`; use `%zu`
Crash printing a pointer	`%p` without a `(void *)` cast
Binary too large for flash	`printf` with float support linked in
`errno` reports the wrong error	Clobbered by an intervening call; save it immediately
`strtol` range errors never detected	Forgot `errno = 0` before the call
Assertion has no effect in release	Side effect inside `assert`; `NDEBUG` removed it
Timing measurements occasionally negative	Used wall-clock time; use `CLOCK_MONOTONIC`
Deadlock or corruption inside a signal handler	Called a non-async-signal-safe function
`memset(a, 1, n)` gives 0x01010101	`memset` works on bytes
Undefined reference to `memcpy` in a bare-metal build	The compiler generated a call; provide it (ch. 20)

Check yourself

- ✓ Name the freestanding headers and say why those specific ones require no runtime.
- ✓ Give the correct specifiers for `size_t`, `uint32_t` and a pointer.
- ✓ State the four `errno` rules.
- ✓ Explain when to `assert` and when to return an error.
- ✓ Say why `assert(f() == 0)` is a bug.
- ✓ Explain which clock to use for measuring a duration, and why.
- ✓ List three functions that are safe in a signal handler and explain the constraint.
- ✓ Give five standard functions to avoid and their replacements.

Compiling and linking in practice

Most C build problems are link problems, and most link problems come from not knowing what the linker actually does. This chapter walks the pipeline from source to running image, with the diagnostic commands for each stage.

The pipeline

```
   main.c   util.c   startup.S
      |        |         |
   [ preprocess ]        |          gcc -E    -> .i
      |        |         |
   [  compile   ]        |          gcc -S    -> .s   (assembly)
      |        |         |
   [ assemble  ]  [ assemble ]      gcc -c    -> .o   (object, with
      |        |         |                             relocations)
      +--------+---------+
               |
           [   LINK   ]             ld / gcc  -> executable or .elf
               |                    resolves symbols, assigns final
               |                    addresses, applies relocations
               v
           program.elf
               |
      [ objcopy -O binary ]         -> program.bin (raw image to flash)
               |
      [ load / flash / exec ]

Each arrow can fail in a characteristic way, and knowing which stage
you are in tells you which tool to reach for.
```

```
# run one stage at a time when something is wrong
gcc -E  main.c -o main.i     # preprocessed source - macro problems
gcc -S  main.c -o main.s     # assembly            - codegen questions
gcc -c  main.c -o main.o     # object              - syntax/type errors
gcc main.o util.o -o prog    # link                - symbol errors

gcc -v main.c                # show every subprocess and its arguments
gcc -### main.c              # ...without running them
gcc -save-temps main.c       # keep .i, .s and .o alongside the output
```

Inside an object file

```
An ELF object contains SECTIONS and a SYMBOL TABLE.

  .text      machine code
  .rodata    read-only data: literals, const tables
  .data      initialised writable data
  .bss       zero-initialised data — size only, no bytes in the file
  .symtab    symbols: name, section, offset, size, binding, type
  .rela.*    RELOCATIONS: "patch this offset with the address of
             symbol S once you know it"
  .debug_*   DWARF debug information

Symbol bindings:
  GLOBAL   visible to the linker, may be referenced elsewhere
  LOCAL    file-scope `static` — invisible outside this object
  WEAK     may be overridden by a strong definition elsewhere

Symbol types you will see in nm output:
  T  in .text (defined)          t  local .text
  D  in .data                    d  local .data
  B  in .bss                     b  local .bss
  R  in .rodata
  U  UNDEFINED — needs to come from somewhere else
  W  weak
```

```
# inspect objects
nm util.o                       # symbol table
nm -C --defined-only prog       # only what this file defines
nm prog | grep ' U '            # everything still undefined
nm --size-sort -S prog | tail   # the biggest symbols

readelf -h prog                 # ELF header: type, machine, entry point
readelf -S prog                 # section headers with addresses/sizes
readelf -s prog                 # symbol table, verbose
readelf -d prog                 # dynamic section: NEEDED libraries, RPATH
readelf -r util.o               # relocations

objdump -d prog                 # disassemble
objdump -d --source prog        # interleave source (needs -g)
objdump -t prog                 # symbol table
objdump -s -j .rodata prog      # hex dump of one section

size prog                       # text/data/bss totals — the embedded
                                # build-gate metric
```

The two link errors, and how to read them

Undefined reference

```
/tmp/ccXXXX.o: in function `main':
main.c:(.text+0x18): undefined reference to `helper'
```

Cause	Check
The .c file defining it was not compiled or linked	Is `util.o` on the command line?
It is `static` in the defining file	`nm util.o | grep helper` — lower-case `t` means local

Cause	Check
Name mismatch — a typo, or C++ name mangling	`nm -C`; wrap C headers in `extern "C"`
Library not linked	Add `-lfoo`
Library listed *before* its user	**Order matters** — see below
Declared but never defined anywhere	Prototype in a header with no implementation
Wrong architecture object	`file util.o`
Compiler generated a call you did not write	`memcpy` / `memset` in freestanding builds

LINK ORDER IS NOT COMMUTATIVE

```
gcc -lm main.c        # FAILS: at the time libm is processed, nothing
                      # needs sqrt yet, so it is discarded
gcc main.c -lm        # works

/* The traditional linker makes ONE pass, left to right. When it
   reaches an archive (.a) it pulls in only the members that resolve
   currently-undefined symbols, then moves on. A library placed
   before its user contributes nothing.

   RULE: objects first, then libraries, in dependency order
   (users before providers).

   For circular dependencies between static libraries: */
gcc main.o -la -lb -la               # repeat
gcc main.o -Wl,--start-group -la -lb -Wl,--end-group   # or group them

/* Shared libraries do not have this problem in the same way, but
   keeping the order correct is still the habit to build.       */
```

Multiple definition

```
/usr/bin/ld: util.o:(.bss+0x0): multiple definition of `counter';
             main.o:(.bss+0x0): first defined here
```

```
/* Almost always: a variable DEFINED in a header. */

/* WRONG — every including file gets its own definition */
/* config.h */  int verbose = 0;

/* RIGHT */
/* config.h */  extern int verbose;
/* config.c */  int verbose = 0;

/* GCC 10 made -fno-common the default, so tentative definitions
   across translation units now error instead of silently merging.
   If an old codebase suddenly fails to link after a toolchain
   upgrade, this is why. The fix is the pattern above, not
   -fcommon.                                                    */

/* Also caused by:
     - a function DEFINED in a header without `static inline`
     - the same .c file listed twice in the build
     - a name colliding with libc (e.g. defining your own `index`) */
```

Static and shared libraries

```
# static library — an archive of object files
ar rcs libfoo.a a.o b.o c.o
ar t libfoo.a                   # list members
gcc main.o -L. -lfoo -o prog    # members are COPIED into prog

# shared library — loaded at runtime
gcc -fPIC -c a.c b.c            # position-independent code, required
gcc -shared a.o b.o -o libfoo.so.1.0
ln -s libfoo.so.1.0 libfoo.so.1
ln -s libfoo.so.1 libfoo.so
gcc main.o -L. -lfoo -o prog

# runtime lookup
ldd prog                        # which shared objects it needs
LD_LIBRARY_PATH=. ./prog        # development only
gcc ... -Wl,-rpath,'$ORIGIN/../lib'   # embed a relative search path
ldconfig -p | grep foo          # the system cache
```

	Static (.a)	Shared (.so / .dll)
Binary size	Larger — code is copied in	Smaller
Memory across processes	One copy each	Shared text pages
Deployment	Single self-contained file	Must ship and version the library
Updating the library	Requires relinking everything	Drop-in, if the ABI is stable
Startup cost	None	Dynamic symbol resolution
Call cost	Direct	Through the PLT/GOT — one indirection
Typical in embedded	**Yes** — determinism, no loader	Rare on small systems

Controlling visibility

```
/* By default every non-static symbol in a shared library is
   exported. That is slow (large dynamic symbol table), fragile
   (accidental ABI), and prevents optimisation.                */

gcc -fvisibility=hidden -shared ...     /* hide everything by default */

#define API __attribute__((visibility("default")))
API int  public_function(void);          /* explicitly exported       */
static   int private_helper(void);       /* not even in .symtab       */

/* A version script gives finer control and is what real libraries use */
/* libfoo.map */
FOO_1.0 {
    global: foo_init; foo_run; foo_free;
    local:  *;
};
gcc -shared -Wl,--version-script=libfoo.map ...

/* Benefits: smaller symbol table, faster load, a real ABI boundary,
   and the linker can inline and optimise across hidden symbols.  */
```

Link-time optimisation

```
# Without LTO the compiler sees one translation unit at a time and
# cannot inline across files. LTO defers codegen to link time.
gcc -flto -O2 -c a.c b.c
gcc -flto -O2 a.o b.o -o prog

# Typical gains: 5–20% speed, and often smaller binaries because
# unused code across the whole program can be eliminated.
# Costs: much slower and more memory-hungry links; harder debugging;
# occasionally exposes latent UB that was previously hidden by the
# TU boundary.

# also worth knowing
-ffunction-sections -fdata-sections -Wl,--gc-sections
# puts each function and object in its own section so the linker can
# discard unreferenced ones. Essential in embedded builds; often
# saves 20–40% of image size.

-Wl,--print-gc-sections    # show what was discarded
-Wl,-Map=out.map           # a full map file: what ended up where
                           # THE tool for "why is my binary so big?"
```

Linker scripts

```
/* On a hosted OS the default script is invisible. On bare metal you
   write it, and it defines the entire memory layout.           */

MEMORY {
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 512K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

ENTRY(Reset_Handler)

SECTIONS {
  .text : {
    KEEP(*(.vectors))          /* KEEP survives --gc-sections */
    *(.text*)
    *(.rodata*)
    . = ALIGN(4);
    _etext = .;                /* symbols the C code can reference */
  } > FLASH

  .data : AT (_etext) {        /* LIVES in RAM, STORED in FLASH */
    _data_start = .;
    *(.data*)
    . = ALIGN(4);
    _data_end = .;
  } > RAM

  .bss : {
    _bss_start = .;
    *(.bss*) *(COMMON)
    . = ALIGN(4);
    _bss_end = .;
  } > RAM

  _stack_top = ORIGIN(RAM) + LENGTH(RAM);
}

/* The `AT` directive is why startup code has a copy loop: the
   initial values of .data must be stored somewhere non-volatile,
   then copied to RAM before main(). Chapter 20 shows that loop. */
```

Build systems, minimally

```
# A Makefile that regenerates header dependencies automatically —
# the single most important correctness property of a C build.
CC      := aarch64-linux-gnu-gcc
CFLAGS  := -std=c11 -Wall -Wextra -O2 -MMD -MP
SRCS    := $(wildcard src/*.c)
OBJS    := $(SRCS:.c=.o)
DEPS    := $(OBJS:.o=.d)

prog: $(OBJS)
        $(CC) $^ -o $@ $(LDFLAGS)

-include $(DEPS)        # pulls in "main.o: main.c foo.h bar.h"

clean:
        rm -f $(OBJS) $(DEPS) prog

# -MMD -MP makes the compiler emit a .d file listing every header
# the object depends on. Without it, editing a header does NOT
# trigger a rebuild, and you debug a stale binary. This is the most
# common cause of "impossible" behaviour in hand-written Makefiles.
```

```
# generate compile_commands.json so clangd, clang-tidy and IDEs work
bear -- make
# or with CMake
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

A baseline flag set

```
# warnings — turn them on and treat them as errors
-Wall -Wextra -Wpedantic
-Wshadow -Wconversion -Wsign-conversion
-Wcast-align -Wcast-qual -Wpointer-arith
-Wstrict-prototypes -Wmissing-prototypes -Wold-style-definition
-Wwrite-strings -Wredundant-decls -Wnull-dereference
-Wdouble-promotion -Wformat=2
-Werror                     # in CI, at minimum

# hardening for anything that touches untrusted input
-D_FORTIFY_SOURCE=3 -O2
-fstack-protector-strong
-fstack-clash-protection
-fPIE -pie
-Wl,-z,relro,-z,now
-Wl,-z,noexecstack
-mbranch-protection=standard      # aarch64: PAC + BTI

# development and test builds
-g3 -fno-omit-frame-pointer
-fsanitize=address,undefined
-fanalyzer

# embedded
-ffunction-sections -fdata-sections -Wl,--gc-sections
-ffreestanding -nostdlib -fno-builtin
-Wl,-Map=out.map
```

How it shows up

Symptom	Cause
"undefined reference" to a function you can see	Not linked, `static`, or library order wrong
"undefined reference to `sqrt`"	`-lm` missing, or placed before the user
"multiple definition"	Variable or non-inline function defined in a header
Link fails only after a toolchain upgrade	`-fno-common` became the default
Edits to a header have no effect	Missing `-MMD -MP` dependency generation
Binary far larger than expected	No `--gc-sections`; read the map file
Runs on the build machine, "library not found" elsewhere	`LD_LIBRARY_PATH` dependency; set an RPATH or link statically
Crash before `main` on bare metal	Linker script or startup copy loop wrong
Symbol resolves to the wrong library's version	Search order; check with `ldd` and `-Wl,--trace`
LTO build fails or misbehaves	Latent UB previously hidden by the TU boundary

Check yourself

- ✓ Name the five pipeline stages and the flag that stops after each.
- ✓ Say what a relocation is and why object files need them.
- ✓ Explain why `gcc -lm main.c` fails and `gcc main.c -lm` works.
- ✓ Give three causes of "undefined reference" and the command that distinguishes them.
- ✓ Explain why defining a variable in a header causes "multiple definition".
- ✓ Compare static and shared libraries on four axes.
- ✓ Explain what `--gc-sections` needs from the compiler to be effective.
- ✓ Explain why `.data` is stored in flash but lives in RAM.
- ✓ Say what `-MMD -MP` does and what breaks without it.

Inline assembly and compiler intrinsics

Sometimes C cannot express what you need: a system register, a barrier, an atomic sequence, a SIMD instruction. Inline assembly is the escape hatch — and the most dangerous construct in the language, because a wrong constraint produces code that works until the register allocator changes its mind.

Try these first

Need	Reach for	Before
Bit counting, byte swap	`__builtin_popcount`, `__builtin_bswap32`, `<stdbit.h>`	asm
Atomics	`<stdatomic.h>`, `__atomic_*`	asm
Overflow checks	`__builtin_add_overflow`	asm
SIMD	Auto-vectorisation, then `<arm_neon.h>` intrinsics	asm
Barriers	`atomic_thread_fence`, kernel `smp_mb()`	asm
Cache and TLB maintenance	Kernel/vendor helper functions	asm
System registers	Kernel accessors (`read_sysreg()`)	asm

THE RULE

Inline assembly is a last resort. Intrinsics and builtins give the compiler full visibility — it can schedule, allocate registers, constant-fold and vectorise around them. An `asm` block is opaque: the compiler must trust your description of it, and if that description is wrong, the bug appears only at certain optimisation levels or after unrelated edits.

GCC extended asm

```
asm [volatile] ( "template"
               : output operands
               : input operands
               : clobbers
               );

%0, %1, %2 ...   refer to operands in order, outputs first
%w0              32-bit view of operand 0 (aarch64)
%x0              64-bit view

Constraint letters (aarch64):
  "r"   general register
  "m"   memory operand
  "i"   immediate constant
  "w"   SIMD/FP register
  "0"   must use the same register as operand 0

Modifiers:
  "=r"  write-only output   (assume the old value is dead)
  "+r"  read-write output
  "&r"  EARLYCLOBBER — written before all inputs are consumed
```

```
/* read a system register */
static inline uint64_t read_sctlr(void)
{
        uint64_t v;
        asm volatile("mrs %0, sctlr_el1" : "=r"(v));
        return v;
}

/* write one, then synchronise the pipeline */
static inline void write_sctlr(uint64_t v)
{
        asm volatile("msr sctlr_el1, %0\n\t"
                     "isb"
                     :: "r"(v) : "memory");
}

/* a barrier: no operands at all, only a clobber */
#define dmb_ish()  asm volatile("dmb ish" ::: "memory")

/* arithmetic with input and output */
static inline uint32_t rev32(uint32_t x)
{
        uint32_t r;
        asm("rev %w0, %w1" : "=r"(r) : "r"(x));
        return r;
}
/* ...though you should just use __builtin_bswap32(x). */
```

The four mistakes

1. FORGETTING `VOLATILE`

Without it, the compiler may delete an `asm` whose outputs are unused, hoist it out of a loop, or execute it fewer times than written. Any asm with a *side effect* — writing a system register, touching hardware, a barrier — must be `volatile`.

```
asm("msr daifset, #2");              /* may be DELETED — no outputs */
asm volatile("msr daifset, #2");     /* correct                     */

/* Conversely, a pure computation should NOT be volatile, so the
   compiler can hoist it out of loops and CSE it: */
asm("rev %w0, %w1" : "=r"(r) : "r"(x));   /* pure — leave it hoistable */
```

2. MISSING THE `"MEMORY"` CLOBBER

Tells the compiler that the asm reads or writes memory it cannot see, so it must not cache values in registers across the block, and must not reorder memory accesses past it. Every barrier, every cache operation, and any asm that dereferences a pointer needs it.

```
/* WRONG: the compiler may keep *p in a register across the barrier */
asm volatile("dsb sy");

/* RIGHT */
asm volatile("dsb sy" ::: "memory");

/* A compiler-only barrier, with no instruction at all: */
#define barrier() asm volatile("" ::: "memory")
/* This is exactly what Linux's barrier() is, and what READ_ONCE
   and WRITE_ONCE are built on.                                  */
```

3. NOT DECLARING CLOBBERED REGISTERS

If your asm writes a register the compiler thinks holds a live value, that value is silently destroyed. The symptom is corruption in unrelated code, appearing only at certain optimisation levels.

```
/* WRONG: x9 is clobbered but not declared */
asm volatile("mov x9, #0\n\t"
             "add %0, %1, x9" : "=r"(r) : "r"(x));

/* RIGHT */
asm volatile("mov x9, #0\n\t"
             "add %0, %1, x9" : "=r"(r) : "r"(x) : "x9");

/* BETTER: let the compiler choose the scratch register */
uint64_t tmp;
asm volatile("mov %1, #0\n\t"
             "add %0, %2, %1" : "=r"(r), "=&r"(tmp) : "r"(x));

/* Also clobber "cc" if you modify the condition flags: */
asm volatile("subs %0, %1, #1" : "=r"(r) : "r"(x) : "cc");
```

4. MISSING EARLYCLOBBER

By default the compiler may assign an output and an input to the *same* register, assuming inputs are fully consumed before outputs are written. If your asm writes the output before reading all inputs, you need `&`.

```
/* WRONG: %0 and %1 may be the same register, so writing %0 first
   destroys the input.                                          */
asm("mov %0, #0\n\t"
    "add %0, %0, %1" : "=r"(out) : "r"(in));

/* RIGHT */
asm("mov %0, #0\n\t"
    "add %0, %0, %1" : "=&r"(out) : "r"(in));

/* The bug is invisible until register pressure changes — which is
   why it typically appears months later after an unrelated edit. */
```

Worked examples

```
/* Interrupt masking — the shape used in every RTOS and kernel */
static inline unsigned long irq_save(void)
{
        unsigned long flags;
        asm volatile("mrs %0, daif\n\t"
                     "msr daifset, #2"      /* mask IRQ */
                     : "=r"(flags) :: "memory");
        return flags;
}
static inline void irq_restore(unsigned long flags)
{
        asm volatile("msr daif, %0" :: "r"(flags) : "memory");
}

/* An atomic increment with exclusives — for illustration; in real
   code use <stdatomic.h>, which also selects LSE on newer cores. */
static inline void atomic_inc(int *p)
{
        unsigned long tmp;
        int val;
        asm volatile("1: ldaxr %w0, %2\n\t"
                     "   add   %w0, %w0, #1\n\t"
                     "   stlxr %w1, %w0, %2\n\t"
                     "   cbnz  %w1, 1b"
                     : "=&r"(val), "=&r"(tmp), "+Q"(*p)
                     :: "memory");
}
/* Note: "+Q" is a memory operand with no offset — required for
   exclusives. "1:" and "1b" are LOCAL LABELS, mandatory in inline
   asm because the block may be duplicated by inlining or
   unrolling; a named label would then be defined twice.        */

/* SIMD is far better expressed with intrinsics */
#include <arm_neon.h>
void vadd(float *c, const float *a, const float *b, size_t n)
{
        for (size_t i = 0; i + 4 <= n; i += 4)
                vst1q_f32(c + i, vaddq_f32(vld1q_f32(a + i),
                                           vld1q_f32(b + i)));
}
/* Intrinsics are typed, schedulable, and the compiler can still
   register-allocate around them. Prefer them universally.     */
```

asm goto and basic asm

```
/* Basic asm — no operands. Only safe at FILE SCOPE or when the block
   truly touches nothing. Inside a function it is a trap, because
   the compiler assumes it does nothing.                          */
asm(".section .rodata\n\t.asciz \"build-id\"\n\t.previous");

/* asm goto — can branch to a C label. This is what makes Linux's
   static keys (runtime-patchable branches) possible.            */
static inline bool feature_enabled(void)
{
        asm goto("1: nop\n\t"
                 ".pushsection __jump_table, \"aw\"\n\t"
                 ".quad 1b, %l[l_yes]\n\t"
                 ".popsection"
                 :::: l_yes);
        return false;
l_yes:
        return true;
}
/* The nop is rewritten to a branch at runtime when the feature is
   toggled — zero cost when disabled, and no indirect call.     */
```

Naked functions and startup code

```
/* A naked function has no prologue or epilogue. Used for reset
   handlers and context switches, where the ABI does not apply. */
__attribute__((naked)) void Reset_Handler(void)
{
        asm volatile(
                "ldr r0, =_stack_top\n\t"
                "mov sp, r0\n\t"
                "bl  SystemInit\n\t"
                "bl  main\n\t"
                "b   .");
}
/* Rules: the body must be ENTIRELY asm. No local variables, no C
   statements, no return — the compiler generates nothing around
   it, so anything else has undefined behaviour.               */

/* For anything longer than a few instructions, use a separate .S
   file instead. It is easier to read, easier to debug, and the
   assembler gives better diagnostics.                          */
```

Verify what you wrote

```
# ALWAYS read the generated code after writing inline asm.
gcc -O2 -S file.c -o - | less
gcc -O2 -c file.c && objdump -d file.o

# check at several optimisation levels — constraint bugs are
# optimisation-dependent by nature
for O in 0 1 2 3 s; do gcc -O$O -S f.c -o f.$O.s; done

# godbolt.org (Compiler Explorer) is the fastest way to iterate:
# paste, see the assembly instantly, switch compilers and flags.

# in a running kernel or program
gdb ./prog
(gdb) disassemble /s my_function
(gdb) layout asm
```

TESTING CHECKLIST FOR ANY INLINE ASM

- Compiles and behaves correctly at `-O0`, `-O2`, `-Os`.
- Correct when inlined into several different call sites.
- Correct inside a loop (checks `volatile` and clobbers).
- Correct under high register pressure — call it with many live variables around it.
- Correct with LTO enabled.
- Read the disassembly at least once and confirm the register assignment is what you expect.

Portability

```
/* GCC/Clang syntax is not MSVC syntax. MSVC removed inline asm for
   x64 entirely and offers only intrinsics.                      */

#if defined(__GNUC__)
        asm volatile("dmb ish" ::: "memory");
#elif defined(_MSC_VER)
        __dmb(_ARM64_BARRIER_ISH);      /* MSVC intrinsic */
#endif

/* Isolate every asm block behind a small inline wrapper in one
   header. Then the platform conditionals live in ONE file and the
   rest of the codebase calls a normal-looking function.        */

/* arch.h */
static inline void arch_barrier(void);
static inline uint64_t arch_read_cycles(void);
static inline void arch_irq_disable(void);
```

How it shows up

Symptom	Cause
asm block disappears from the output	Missing `volatile` with unused outputs
Barrier has no effect on ordering	Missing `"memory"` clobber

Symptom	Cause
Unrelated variable corrupted	Undeclared register clobber
Works at `-O0`, breaks at `-O2`	Wrong constraints; register allocation changed
Wrong result when an input equals an output	Missing earlyclobber `&`
"symbol already defined" after inlining	Named label in asm; use local labels `1:` / `1b`
Condition flags wrong after the block	Missing `"cc"` clobber
Crash entering a naked function	Non-asm statements in the body
Fails only with LTO	Cross-TU inlining changed the surrounding register state
32-bit operation on a 64-bit register	Used `%0` where `%w0` was needed

Check yourself

- ✓ Give three alternatives to try before writing inline assembly.
- ✓ Explain when `volatile` is required and when it should be omitted.
- ✓ Explain what the `"memory"` clobber tells the compiler.
- ✓ Write a system-register read with the correct constraints.
- ✓ Explain earlyclobber and construct a case that needs it.
- ✓ Explain why inline asm must use local labels.
- ✓ Say why intrinsics are preferable to asm for SIMD.
- ✓ List the optimisation levels and conditions you would test an asm block under.

Defensive C: MISRA, assertions and static analysis

C gives you no safety net, so the profession built one out of process: coding standards, static analysis, assertions and review discipline. This chapter is what actually works in production, and what is theatre.

Defence in depth

```
Each layer catches what the one before it missed. Cost rises,
and so does the cost of the bugs each one prevents.

  1. LANGUAGE DISCIPLINE     free       consistent idioms, small
                                        functions, clear ownership
  2. COMPILER WARNINGS       free       -Wall -Wextra -Werror
  3. STATIC ANALYSIS         cheap      -fanalyzer, clang-tidy,
                                        cppcheck, commercial tools
  4. CODING STANDARD         process    MISRA, CERT C, kernel style
  5. ASSERTIONS              cheap      invariants checked at runtime
  6. SANITIZERS              CI time    ASan, UBSan, TSan, MSan
  7. UNIT + PROPERTY TESTS   effort     including fuzzing
  8. CODE REVIEW             effort     the only layer that catches
                                        DESIGN errors
  9. FORMAL METHODS          expensive  Frama-C, CBMC — for the code
                                        that must not fail

Most teams do 1, 2 and 8, and skip 3, 5 and 6 — which is precisely
where the cheapest wins are.
```

Warnings are the cheapest layer

```
# the baseline every C project should have
-Wall -Wextra -Wpedantic -Werror

# and the ones -Wall does not include but should
-Wshadow                   # a local hiding an outer name
-Wconversion               # implicit narrowing
-Wsign-conversion
-Wcast-align               # casting to a stricter alignment
-Wcast-qual                # casting away const
-Wpointer-arith
-Wstrict-prototypes        # int f() with no parameter list
-Wmissing-prototypes       # a non-static function with no prototype
-Wold-style-definition
-Wredundant-decls
-Wwrite-strings            # string literals become const char[]
-Wnull-dereference
-Wdouble-promotion         # accidental float->double on an MCU
-Wformat=2 -Wformat-security
-Wvla                      # variable-length arrays (chapter 05)
-Wswitch-enum              # a switch missing an enum value

# Treat warnings as errors in CI from day one. A codebase with
# 400 warnings has zero warnings, because nobody reads them.
```

> **ADOPTING `-WERROR` ON AN EXISTING CODEBASE**
> Do not try to fix everything at once. Enable `-Werror` only for *new* files, or use `-Werror=` for individual warnings you have already cleaned (`-Werror=return-type -Werror=implicit-function-declaration`). Ratchet forward: every warning you clear becomes an error so it can never come back. This converts an impossible cleanup into a steady one.

Static analysis

Tool	Finds	Cost
`gcc -fanalyzer`	Leaks, double free, use-after-free, null deref, some UB — with a path trace	Free; slow compile
`clang --analyze` / `scan-build`	Similar, different strengths; HTML reports	Free
`clang-tidy`	Style, modernisation, bug patterns; many CERT and MISRA checks	Free; needs `compile_commands.json`
`cppcheck`	Broad heuristic checks; MISRA add-on	Free; no build integration needed
`sparse`	Kernel-specific: `__user` / `__iomem` pointer misuse, endianness annotations, lock context	Free; `make C=1`
`smatch`	Kernel flow analysis	Free
Coverity, PVS-Studio, Polyspace, Klocwork	Deeper interprocedural analysis; certification evidence	Commercial

```
# run them in CI, not on developer laptops only
gcc -fanalyzer -Wall -Wextra -c src/*.c
scan-build make
clang-tidy src/*.c -- -Iinclude
cppcheck --enable=all --inconclusive --std=c11 src/

# kernel
make C=1 CHECK=sparse
make C=2 CF=-D__CHECK_ENDIAN__

# Static analysers produce false positives. Triage them, suppress
# with a NARROW comment and a REASON, and never with a blanket
# disable:
/* cppcheck-suppress nullPointer ; p is validated in caller, see #123 */
```

MISRA C

A coding standard for safety-related systems, originating in automotive and now used in medical, industrial and aerospace work. MISRA C:2012 has roughly 175 rules and directives, classified as *mandatory*, *required* or *advisory*.

Representative rule	Rationale
No dynamic memory (21.3)	Fragmentation and non-determinism (ch. 14)
No recursion (17.2)	Stack usage cannot be bounded statically
No `goto` (15.1, advisory)	Control-flow analysability
Single exit per function (15.5, advisory)	Simplifies reasoning; widely disputed

Representative rule	Rationale
Every `if` / `else if` chain ends in `else` (15.7)	Forces you to consider the default case
Every `switch` has a `default` (16.4)	Same
All bodies are compound statements (15.6)	Prevents the dangling-else and "goto fail" class
No implicit conversions that change signedness or width (10.x)	The bug class in chapter 02
Function return values must be used or explicitly discarded (17.7)	Ignored error codes
No `#undef`, restricted `#include` forms (20.x)	Preprocessor predictability

> **AN HONEST ASSESSMENT**
>
> MISRA's genuinely valuable rules are the ones about implicit conversion, uninitialised data, unused return values and undefined behaviour — they encode real bug classes. The controversial ones (single exit, no `goto`) can make error handling *worse* in C, since the `goto` cleanup ladder (ch. 14) is the clearest way to release resources in reverse order.
>
> MISRA is a **deviation-based** process: you are expected to violate rules where justified, and to document why. A team that treats it as an absolute prohibition ends up writing worse code to satisfy a checker. If you are not in a certified domain, adopt the conversion and initialisation rules and skip the structural dogma.

CERT C is the security-focused alternative, organised by vulnerability class rather than by language feature, and is generally a better fit for non-certified projects.

Assertions that earn their place

```
/* The distinction that matters: */
assert(...)      /* PROGRAMMER error — an invariant that must hold
                    if this code is correct. May be compiled out.  */
if (...) return -EINVAL;
                 /* EXTERNAL error — bad input, missing hardware,
                    resource exhaustion. Must ALWAYS be handled.   */

/* Assert on: */
assert(index < ARRAY_SIZE(table));     /* internal index          */
assert(state == STATE_READY);          /* state machine invariant */
assert(mutex_is_held(&lock));          /* locking discipline      */
assert(p != NULL);                     /* internal contract       */

/* Never assert on: user input, file contents, network data,
   hardware responses, or malloc results.                       */
```

```
/* In production embedded systems, assert() calling abort() is often
   wrong — you cannot just stop a motor controller. Define your own: */

#ifdef NDEBUG
#  define ASSERT(c) ((void)0)
#else
#  define ASSERT(c) ((c) ? (void)0 : assert_failed(__FILE__, __LINE__))
#endif

void assert_failed(const char *file, int line)
{
        enter_safe_state();            /* outputs off, brakes on   */
        log_persistent(file, line);    /* survive the reset        */
        system_reset();                /* recover, do not hang     */
}

/* The kernel's spectrum, and when to use each: */
BUILD_BUG_ON(cond);      /* compile time — always prefer this      */
WARN_ON(cond);           /* log a stack trace and CONTINUE — the
                            default choice for a recoverable
                            invariant violation                    */
WARN_ON_ONCE(cond);      /* ...but do not spam the log             */
BUG_ON(cond);            /* panic. Reserve for "continuing would
                            corrupt data". Rarely justified.       */
```

Design by contract

```
/**
 * ring_write() — append bytes to the ring buffer
 * @r:    ring buffer, must be non-NULL and initialised
 * @data: source buffer, must be non-NULL if @len > 0
 * @len:  number of bytes, must be <= ring_space(r)
 *
 * Context: caller must hold r->lock.
 * Return: number of bytes written, or a negative errno.
 */
ssize_t ring_write(struct ring *r, const void *data, size_t len)
{
        ASSERT(r != NULL);
        ASSERT(data != NULL || len == 0);
        ASSERT(lockdep_held(&r->lock));
        ...
}

/* Write the precondition, then assert it. The comment documents;
   the assertion enforces. Either alone is half a contract.     */

/* Let the compiler help where it can: */
void f(int *p) __attribute__((nonnull(1)));
int  g(void)   __attribute__((warn_unused_result));

/* warn_unused_result is enormously valuable on anything that can
   fail. Ignored return values are a top-tier bug source.       */
```

Defensive patterns

```
/* Validate at the trust boundary, once, thoroughly. Do not
   re-validate everywhere internally - that hides which layer owns
   the check and produces inconsistent behaviour.              */

int api_entry(const struct request *req)
{
        /* THE boundary: everything after this point may assume
           the request is well-formed.                          */
        if (!req)                       return -EINVAL;
        if (req->len > MAX_LEN)         return -EMSGSIZE;
        if (req->type >= TYPE_COUNT)    return -EINVAL;
        if (req->len > 0 && !req->data) return -EINVAL;

        return handle(req);             /* internal: asserts only */
}

/* Initialise everything at declaration */
struct config cfg = {0};      /* not: struct config cfg; */
int  ret = -EINVAL;
char *buf = NULL;

/* Const everything you do not modify - it is free documentation
   that the compiler checks.                                    */
static int compute(const struct input *in, struct output *out);

/* Fail closed, not open */
switch (mode) {
case MODE_A: return handle_a();
case MODE_B: return handle_b();
default:     return -EINVAL;      /* not: fall through to A */
}

/* Bound every loop that waits on hardware */
int timeout = 10000;
while (!(readl(base + STATUS) & READY))
        if (--timeout < 0)
                return -ETIMEDOUT;

/* Make illegal states unrepresentable where you can */
typedef struct { int v; } celsius_t;    /* not interchangeable with */
typedef struct { int v; } fahrenheit_t; /* an int, or each other    */
```

Testing C

```
# unit test frameworks that work well in C
#   Unity + CMock  — designed for embedded, no dynamic allocation
#   Check, cmocka, greatest, µnit
#   Criterion      — modern, nice output

# run everything under sanitizers in CI
gcc -fsanitize=address,undefined -g -O1 test.c -o test && ./test

# coverage — measure what the tests actually reach
gcc --coverage test.c && ./test && gcov test.c
lcov --capture --directory . --output-file cov.info
genhtml cov.info --output-directory out
# Aim for branch coverage on error paths specifically. Error paths
# are where the untested bugs live, by definition.

# FUZZING is the highest-value testing for any C code that parses
# input. It routinely finds bugs that unit tests never will.
clang -fsanitize=fuzzer,address parser_fuzz.c -o fuzz
./fuzz corpus/ -max_total_time=600
```

```
/* A libFuzzer harness is four lines. There is no excuse not to
   have one for every parser you write.                         */
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
        struct msg m;
        parse_message(data, size, &m);    /* must never crash */
        return 0;
}
```

What review is for

Tools find defects; review finds *wrong designs*. Focus review on what no tool can check:

- ✓ Is the ownership of every allocation clear and documented?
- ✓ Is every error path correct, and does it release what it acquired?
- ✓ Are the locking rules stated, and does this code follow them?
- ✓ What happens on the second call? On a concurrent call? On failure halfway through?
- ✓ Is the trust boundary explicit, and is validation on the right side of it?
- ✓ Are the integer types right, and can any arithmetic overflow?
- ✓ Does the API make misuse difficult, or merely document it?
- ✓ Will this be understandable in two years by someone who was not in this conversation?

How it shows up

Symptom	Missing layer
A bug that a warning would have caught ships	Warnings not enabled, or not errors
Codebase has hundreds of warnings	No `-Werror` ratchet; nobody reads them now
Memory bug found in production, not in test	Sanitizers not in CI

Symptom	Missing layer
Parser crashes on malformed input	No fuzzing
Error path leaks or double-frees	Error paths untested; check branch coverage
Ignored return value causes silent corruption	No `warn_unused_result`
Assertion fires in production and halts a machine	`assert` used for external input
MISRA compliance achieved, quality unchanged	Rules applied as ritual, not for their rationale
Same class of bug recurs across the codebase	Fixed instances, not the pattern; add a lint rule

Check yourself

- ✓ Name the defence layers in order of cost and say which three teams most often skip.
- ✓ Give five warning flags not included in `-Wall` and the bug each catches.
- ✓ Explain how to introduce `-Werror` into a legacy codebase.
- ✓ State the rule for when to assert versus when to return an error.
- ✓ Give two MISRA rules that encode real bug classes and one that is contested, with reasons.
- ✓ Explain where input validation belongs and why not everywhere.
- ✓ Write a libFuzzer harness for a parser.
- ✓ List four questions review should answer that no tool can.

C for freestanding and embedded targets

No operating system, no standard library, no `main()` until you arrange one. This chapter closes the C track by assembling everything into a program that runs on bare hardware — and by being explicit about what you give up.

What freestanding means

	Hosted	Freestanding
Entry point	`main(int, char **)`	Implementation-defined: `Reset_Handler`, `_start`, `DriverEntry`
Library	All of it	Only headers with no runtime component
Startup	`crt0` runs before you	You write it
Heap	Present	Only if you provide one
Examples	Linux/Windows application	Linux kernel, U-Boot, UEFI, Cortex-M firmware

```
/* The nine headers guaranteed in a freestanding implementation.
   Note what they have in common: they define TYPES and MACROS
   only — no functions requiring a runtime.                    */
<stddef.h>   <stdint.h>   <limits.h>   <float.h>   <stdbool.h>
<stdarg.h>   <stdalign.h> <stdnoreturn.h>  <iso646.h>
/* C23 adds <stdbit.h> and a few others. */

/* Everything else — string.h, stdlib.h, stdio.h, math.h — you
   either do without or provide yourself.                      */
```

```
# the flags, and what each one actually prevents
aarch64-none-elf-gcc \
    -ffreestanding \        # do not assume a hosted environment
    -nostdlib \             # do not link libc or crt startup files
    -fno-builtin \          # do not substitute builtin versions of
                            #   library functions...
    -fno-stack-protector \  # ...and do not require __stack_chk_fail
    -fno-common \
    -ffunction-sections -fdata-sections \
    -Wl,--gc-sections \
    -T link.ld

# -fno-builtin matters more than it looks: WITHOUT it GCC may turn
# a struct assignment or an array initialisation into a call to
# memcpy() or memset() that you never wrote and did not provide.
# Even WITH it, some calls can still be generated — so provide
# memcpy, memset, memmove and memcmp regardless.
```

Startup code

```
What crt0 does for you on a hosted system, and what you must do here:

   reset
     |
     1. set the stack pointer
     2. copy .data from its flash load address to RAM
     3. zero .bss
     4. (optional) enable FPU, caches, MPU/MMU
     5. call SystemInit / clock configuration
     6. run any constructors (__attribute__((constructor)))
     7. call main()
     8. if main returns — it should not — loop forever
```

```
startup.c — a complete, portable startup for Cortex-M
```

```c
#include <stdint.h>

/* Symbols defined by the linker script (chapter 17). Their ADDRESSES
   are the values you want, so declare them as arrays or take &.  */
extern uint32_t _etext, _data_start, _data_end, _bss_start, _bss_end;
extern uint32_t _stack_top;

extern int main(void);
void Reset_Handler(void);
void Default_Handler(void);

/* Every handler is weakly aliased to Default_Handler, so the
   application overrides only the ones it cares about (chapter 12). */
#define WEAK_ALIAS __attribute__((weak, alias("Default_Handler")))
void NMI_Handler(void)        WEAK_ALIAS;
void HardFault_Handler(void)  WEAK_ALIAS;
void SysTick_Handler(void)    WEAK_ALIAS;
void UART0_IRQHandler(void)   WEAK_ALIAS;

/* The vector table, placed at address 0 by the linker script.
   KEEP(*(.vectors)) in the script prevents --gc-sections from
   discarding it, since nothing references it.                  */
__attribute__((section(".vectors"), used))
void (*const vector_table[])(void) = {
        (void (*)(void))&_stack_top,   /* initial SP  */
        Reset_Handler,                  /* initial PC  */
        NMI_Handler,
        HardFault_Handler,
        0, 0, 0, 0, 0, 0, 0,            /* reserved    */
        0,                              /* SVCall      */
        0, 0,
        0,                              /* PendSV      */
        SysTick_Handler,
        UART0_IRQHandler,               /* IRQ 0       */
};

void Reset_Handler(void)
{
        /* 2. copy .data from flash to RAM */
        uint32_t *src = &_etext;
        for (uint32_t *dst = &_data_start; dst < &_data_end; )
                *dst++ = *src++;

        /* 3. zero .bss - this is why static objects are zeroed */
        for (uint32_t *p = &_bss_start; p < &_bss_end; )
                *p++ = 0;

        /* 4-5. hardware bring-up */
        SystemInit();

        /* 6. C++ / constructor support, if used */
        extern void (*__init_array_start[])(void);
        extern void (*__init_array_end[])(void);
        for (void (**f)(void) = __init_array_start;
             f < __init_array_end; f++)
                (*f)();

        /* 7. */
        main();

        /* 8. */
        for (;;)
                __asm__ volatile("wfi");
```

```
}

void Default_Handler(void)
{
        for (;;)
                ;               /* set a breakpoint here — an
                                   unexpected interrupt lands here */
}
```

> **THE COPY LOOP IS NOT OPTIONAL, AND ITS ABSENCE IS SUBTLE**
> Skip step 2 and every initialised global holds garbage. Skip step 3 and every zero-initialised global holds garbage. Both produce behaviour that looks like random corruption, works intermittently, and changes when you add an unrelated variable. If a bare-metal program behaves erratically before it does anything interesting, check the startup code first.

Providing the pieces you need

```
/* The compiler can generate calls to these four even if you never
   write them. Provide them, and mark them used so --gc-sections
   does not remove them before the reference appears.           */

void *memcpy(void *dst, const void *src, size_t n)
{
        unsigned char *d = dst;
        const unsigned char *s = src;
        while (n--) *d++ = *s++;
        return dst;
}

void *memset(void *dst, int c, size_t n)
{
        unsigned char *d = dst;
        while (n--) *d++ = (unsigned char)c;
        return dst;
}

void *memmove(void *dst, const void *src, size_t n)
{
        unsigned char *d = dst;
        const unsigned char *s = src;
        if (d < s) { while (n--) *d++ = *s++; }
        else { d += n; s += n; while (n--) *--d = *--s; }
        return dst;
}

int memcmp(const void *a, const void *b, size_t n)
{
        const unsigned char *x = a, *y = b;
        for (; n--; x++, y++) if (*x != *y) return *x - *y;
        return 0;
}

/* Compile these with -fno-tree-loop-distribute-patterns, or GCC may
   recognise the loop inside memcpy and replace it with a call to
   memcpy — infinite recursion, at -O2 only. This is a genuine and
   frequently-hit trap.                                          */
```

```
/* A minimal printf replacement — a few hundred bytes instead of
   tens of kilobytes (chapter 16).                                  */
static void putc_(char c) { while (UART->FR & FR_TXFF) ; UART->DR = c; }

void print_str(const char *s) { while (*s) putc_(*s++); }

void print_hex(uint32_t v)
{
        print_str("0x");
        for (int i = 28; i >= 0; i -= 4)
                putc_("0123456789abcdef"[(v >> i) & 0xF]);
}

void print_dec(uint32_t v)
{
        char buf[11]; int i = 0;
        if (!v) { putc_('0'); return; }
        while (v) { buf[i++] = '0' + v % 10; v /= 10; }
        while (i--) putc_(buf[i]);
}
/* No format parsing, no varargs, no floating point. Covers 95% of
   embedded logging needs at 1% of the code size.                   */
```

Living without a heap

```
/* Static allocation: the linker verifies it fits, at build time. */
static struct session sessions[MAX_SESSIONS];
static uint8_t rx_buf[MAX_FRAME];

/* Fixed-size pool with a bitmap free list */
#define POOL_N 32
static struct msg pool[POOL_N];
static uint32_t  pool_used;

struct msg *msg_alloc(void)
{
        uint32_t free = ~pool_used;
        if (!free) return NULL;
        int i = __builtin_ctz(free);
        if (i >= POOL_N) return NULL;
        pool_used |= 1u << i;
        return &pool[i];
}
void msg_free(struct msg *m)
{
        pool_used &= ~(1u << (m - pool));
}
/* Deterministic time, no fragmentation, bounded memory, and the
   failure mode is a clean NULL rather than gradual degradation. */

/* If you genuinely need a heap, provide _sbrk for newlib: */
extern uint32_t _heap_start, _stack_top;
void *_sbrk(int incr)
{
        static char *brk = (char *)&_heap_start;
        char *prev = brk;
        if (brk + incr > (char *)&_stack_top - STACK_RESERVE)
                return (void *)-1;      /* out of memory */
        brk += incr;
        return prev;
}
```

Register access, assembled

```
/* Everything from chapters 07, 08 and 10 comes together here. */

struct uart_regs {
        volatile uint32_t DR;         /* 0x00 */
        volatile uint32_t RSR;        /* 0x04 */
        uint32_t          _pad0[4];   /* 0x08 - 0x14 */
        volatile uint32_t FR;         /* 0x18 */
        uint32_t          _pad1;
        volatile uint32_t IBRD;       /* 0x20 */
        volatile uint32_t FBRD;       /* 0x24 */
        volatile uint32_t LCRH;       /* 0x28 */
        volatile uint32_t CR;         /* 0x2C */
};
_Static_assert(sizeof(struct uart_regs) == 0x30, "uart layout");
_Static_assert(offsetof(struct uart_regs, CR) == 0x2C, "CR offset");

#define UART0 ((struct uart_regs *)0x40000000u)

#define FR_TXFF     (1u << 5)
#define CR_UARTEN   (1u << 0)
#define CR_TXE      (1u << 8)

void uart_init(uint32_t baud, uint32_t clk)
{
        UART0->CR = 0;                      /* disable while configuring */
        uint32_t div = (clk * 4) / baud;
        UART0->IBRD = div >> 6;
        UART0->FBRD = div & 0x3F;
        UART0->LCRH = (3u << 5);            /* 8 bits            */
        __asm__ volatile("dsb sy" ::: "memory");   /* land before enable */
        UART0->CR = CR_UARTEN | CR_TXE;
}
```

Interrupts in bare metal

```
/* An ISR is an ordinary C function whose ADDRESS is in the vector
   table. On Cortex-M the hardware saves the caller-saved registers,
   so no special attribute is needed.                              */
void UART0_IRQHandler(void)
{
        uint32_t status = UART0->MIS;
        UART0->ICR = status;            /* W1C: clear at the SOURCE */
        if (status & INT_RX)
                ring_put(&rx_ring, UART0->DR);
}

/* Sharing data with mainline code: volatile is necessary but NOT
   sufficient (chapter 10). Either use atomics, or disable
   interrupts around the critical section.                        */
static volatile uint32_t tick;          /* written by ISR only, and
                                           32-bit aligned, so a single
                                           read is atomic on ARM    */

static _Atomic uint32_t events;         /* read-modify-write: needs
                                           a real atomic            */

/* Critical section, the portable way */
static inline uint32_t irq_save(void)
{
        uint32_t f;
        __asm__ volatile("mrs %0, primask\n\tcpsid i" : "=r"(f) :: "memory");
        return f;
}
static inline void irq_restore(uint32_t f)
{
        __asm__ volatile("msr primask, %0" :: "r"(f) : "memory");
}
/* Save-and-restore, never unconditional enable — otherwise nesting
   two critical sections re-enables interrupts too early.       */
```

The constraints to design around

Constraint	Consequence
Stack is 1–8 KB	No recursion, no VLAs, no large locals. Check with `-fstack-usage`.
No MMU	A wild pointer corrupts anything. Use the MPU where present.
Flash is 32–512 KB	`printf` and `math.h` may not fit. Watch `size` output in CI.
No OS	You write the scheduler, or use a superloop plus interrupts
Often no FPU	Software float is 10–100× slower. Use fixed point.
Watchdog	Must be kicked, and must *not* be kicked from a timer interrupt — that defeats it
Power budget	`WFI` in the idle loop, not a busy wait (CA ch. 20)

```
# the numbers to watch on every build
arm-none-eabi-size firmware.elf
#    text   data    bss    dec    hex
#   24512    108   8192  32812   802c
#   ^flash        ^^^^^^^^^^^^ RAM = data + bss (+ stack + heap)

arm-none-eabi-nm --size-sort -S firmware.elf | tail -20   # biggest symbols
arm-none-eabi-gcc -fstack-usage ...   # per-function stack, in .su files
# Fail the build if any of these exceed budget. It is the only way
# they stay within budget.
```

The two program structures

```
/* 1. Superloop plus interrupts — simple, and adequate far more
      often than people expect.                                  */
int main(void)
{
        hw_init();
        for (;;) {
                if (flag_rx)    { flag_rx = 0;   handle_rx(); }
                if (flag_tick)  { flag_tick = 0; handle_tick(); }
                kick_watchdog();
                __asm__ volatile("wfi");     /* sleep until an interrupt */
        }
}
/* ISRs stay short: set a flag, queue data, return. All real work
   happens in the loop, at a known context, with no reentrancy.  */

/* 2. An RTOS, when you need priorities and blocking */
void task_sensor(void *arg)
{
        for (;;) {
                xQueueReceive(q, &sample, portMAX_DELAY);   /* blocks */
                process(&sample);
        }
}
/* Choose an RTOS when you have several independent activities with
   different deadlines. A superloop is genuinely better when you
   do not — fewer failure modes, no stack-per-task overhead.    */
```

How it shows up

Symptom	Cause
Globals contain garbage before `main` does anything	Missing `.data` copy or `.bss` zero in startup
Undefined reference to `memcpy` you never called	Compiler generated it; provide the function
`memcpy` recurses infinitely at `-O2`	GCC recognised your loop; add `-fno-tree-loop-distribute-patterns`
Vector table missing from the image	`--gc-sections` removed it; use `KEEP()`
Hard fault immediately on reset	Stack pointer or reset vector wrong in the table
Random corruption under load	Stack overflow — no MMU to catch it; add a guard pattern
Interrupt fires repeatedly	Not cleared at the source before returning
Data shared with an ISR is stale	Missing `volatile`, or a non-atomic read-modify-write

Symptom	Cause
Image does not fit in flash	`printf` with float support; use a minimal implementation
Works on the debugger, resets in the field	Watchdog not kicked; debug halts often disable it

Check yourself

- ✓ List the freestanding headers and explain what they have in common.
- ✓ Give the eight steps of startup, in order, and what breaks if you skip steps 2 and 3.
- ✓ Explain why `-fno-builtin` is needed and why it is still not sufficient.
- ✓ Explain the `memcpy` self-recursion trap.
- ✓ Write a fixed-size pool allocator and say what it gives you over `malloc`.
- ✓ Explain why the vector table needs `KEEP()` in the linker script.
- ✓ Write a nesting-safe critical section and say why unconditional enable is wrong.
- ✓ Name the build metrics you would gate on, and where they come from.

TRACK COMPLETE

That closes the C track: the abstract machine, types and conversions, pointers and arrays, memory, undefined behaviour, the toolchain, and finally a program that runs on bare hardware with no runtime underneath it.

Next is operating systems — the concepts Linux, Windows and every RTOS share — which both driver tracks assume.

TRACK

C++

C++ where it earns its place in systems work — and an honest account of the parts to leave switched off.

16 chapters

Why C++ for systems, and what to leave out

C++ gives you real safety improvements over C at genuinely zero runtime cost — and also gives you several features that are wrong for embedded work. Knowing which is which is the whole skill, and it matters more here than in any other domain.

What C++ actually buys

```
THE FEATURES THAT COST NOTHING AT RUN TIME AND PREVENT
REAL BUGS:

  RAII                a resource is released when its owner
                      goes out of scope. No leak, on any
                      path, including exceptions and early
                      returns. This is the single biggest
                      win.
  destructors         deterministic, not garbage collected
  const correctness   enforced by the type system
  references          a pointer that cannot be null and
                      cannot be reseated
  strong typing       no implicit void* conversions; enum
                      class does not decay to int
  templates           one implementation, many types, with
                      no runtime dispatch
  constexpr           computation moved to compile time
  std::array, span    an array that knows its own size
  unique_ptr          ownership expressed in the type
  namespaces          no more my_module_do_thing()
  function overloading and default arguments

AND THE COST OF EACH, honestly: zero, provided you use them
as intended. `std::unique_ptr` compiles to exactly the same
code as a raw pointer plus a manual delete - godbolt it and
see (chapter 14).

WHAT IS *NOT* FREE, and needs a decision:
  exceptions          code size, and a hidden control flow
                      path (chapter 12)
  RTTI / dynamic_cast a type-info table per class
  virtual functions   an indirect call and a vtable pointer
                      per object (chapter 11)
  the iostreams       enormous; often 100 KB+ of code
  std::function       a heap allocation and an indirect call
  dynamic allocation  fragmentation and non-determinism
  std::string, vector heap allocation
```

THE HONEST SUMMARY

C++ is a better C for systems work — *if* you choose a subset deliberately and enforce it. Used without that discipline it produces binaries three times the size with unpredictable timing, which is why some embedded organisations ban it outright. That ban is a reasonable response to the undisciplined case and a poor response to the disciplined one.

Choosing a subset

```
# A configuration that most embedded C++ projects converge on:

    -fno-exceptions             ; chapter 12
    -fno-rtti                   ; no dynamic_cast, no typeid
    -fno-threadsafe-statics     ; if single-threaded init
    -fno-use-cxa-atexit         ; no static destructor
                                ; registration
    -ffunction-sections -fdata-sections
    -Wl,--gc-sections
    -Os or -O2
    -std=c++17 or c++20

# AND THE LIBRARY DECISION:
#   freestanding / no std::     bare metal, tightest control
#   ETL (Embedded Template Library)   fixed-capacity
#                               containers with the STL's
#                               interface
#   the STL, minus the allocating parts
#   full STL                    embedded Linux, where a heap
#                               and megabytes exist

# WHAT TO BAN, and why:
#   iostreams          enormous, and drags in locale
#   std::string        heap; use std::string_view, or a
#                      fixed-size buffer
#   std::function      heap + indirect call; use a function
#                      pointer or a template parameter
#   dynamic_cast       needs RTTI
#   exceptions         see chapter 12
#   new/delete in steady state    preallocate at init
#   std::shared_ptr    atomic refcounts; usually the wrong
#                      ownership model for embedded

# WHAT TO ENCOURAGE:
#   std::array, std::span, std::optional, std::variant
#   constexpr everything that can be
#   enum class
#   RAII wrappers for every hardware resource
#   std::unique_ptr with a custom deleter, for handles
#   templates for compile-time configuration

# WRITE IT DOWN. A one-page "C++ subset" document that says
# what is permitted and why is worth more than any amount of
# individual good judgement, because it survives staff
# changes and makes review objective.
```

RAII, immediately

```
/* THE C VERSION, and every error path is a chance to leak: */
        int do_work(void)
        {
                int fd = open("/dev/thing", O_RDWR);
                if (fd < 0) return -1;

                void *buf = malloc(SIZE);
                if (!buf) { close(fd); return -1; }

                if (configure(fd) < 0) {
                        free(buf); close(fd); return -1;
                }
                if (transfer(fd, buf) < 0) {
                        free(buf); close(fd); return -1;
                }

                free(buf);
                close(fd);
                return 0;
        }

/* THE C++ VERSION: */
        class FileDescriptor {
                int fd_;
        public:
                explicit FileDescriptor(const char *path, int flags)
                        : fd_(::open(path, flags)) {}

                ~FileDescriptor() { if (fd_ >= 0) ::close(fd_); }

                /* non-copyable, movable - chapter 03 */
                FileDescriptor(const FileDescriptor&) = delete;
                FileDescriptor& operator=(const FileDescriptor&)
                        = delete;
                FileDescriptor(FileDescriptor&& o) noexcept
                        : fd_(o.fd_) { o.fd_ = -1; }

                bool valid() const { return fd_ >= 0; }
                int  get()   const { return fd_; }
        };

        int do_work()
        {
                FileDescriptor fd("/dev/thing", O_RDWR);
                if (!fd.valid()) return -1;

                std::array<std::byte, SIZE> buf;   /* NO HEAP */

                if (configure(fd.get()) < 0) return -1;
                if (transfer(fd.get(), buf.data()) < 0)
                        return -1;

                return 0;      /* destructor closes fd, on
                                  EVERY path */
        }

/* THE GENERATED CODE IS IDENTICAL to the C version's happy
   path, plus a close in each exit block - which the C version
   also has, written by hand, four times, with one of them
   eventually forgotten.
```

```
   THIS IS THE ARGUMENT FOR C++ IN SYSTEMS WORK, and it is
   the same argument as Linux's devm_ (Linux chapter 26) and
   WDF's object parenting (Windows chapter 13): make cleanup
   structural rather than remembered.                      */
```

Proving the cost

```
/* Do not take "zero-cost" on trust. Check it (chapter 14). */

/* --- unique_ptr versus a raw pointer --- */
        void f() {
                auto p = std::make_unique<int>(42);
                use(*p);
        }
/* -O2 aarch64: */
        mov     w0, #4
        bl      operator new(unsigned long)
        mov     w1, #42
        str     w1, [x0]
        ...
        bl      operator delete(void*, unsigned long)
/* EXACTLY new/delete. No overhead, no size cost.

/* --- std::array versus a C array --- */
        int sum(std::array<int, 4> a) {
                return a[0]+a[1]+a[2]+a[3];
        }
/* compiles to the same instructions as the C version, and
   `a.size()` is a compile-time constant.

/* --- a template versus a macro --- */
        template<typename T> T max(T a, T b)
                { return a > b ? a : b; }
/* -> one `csel`, same as the macro, but type-safe and with no
   double-evaluation bug.

/* --- WHERE IT IS *NOT* FREE --- */
        std::function<int(int)> f = [x](int y) { return x + y; };
/* -> possibly a heap allocation, and an indirect call through
   a type-erased interface. Replace with a template parameter
   or a plain lambda passed by value:                */
        template<typename F> void apply(F f) { ... }
/* -> the lambda is inlined completely; zero cost.

/* THE RULE: any abstraction whose type is known at compile
   time is free. Anything that ERASES the type - virtual,
   std::function, dynamic_cast - costs an indirection and
   blocks inlining.                                        */
```

Static initialisation

```
/* A C++ global with a non-constant initialiser runs code
   before main (compilers chapter 13). On embedded this needs
   thought.                                                  */

        Device g_device{0x40000000};    /* constructor runs
                                           before main - but
                                           the hardware may
                                           not be initialised
                                           yet!             */

/* THE PROBLEMS:
     1. ORDER across translation units is unspecified
     2. it runs before your clock/PLL setup, potentially
     3. .init_array must be walked by your startup code -
        __libc_init_array() does it, and bare-metal startup
        code must call it
     4. destructors register with __cxa_atexit, which needs
        a heap on some implementations

/* THE FIXES, in order of preference: */

/* (a) make it constexpr - initialised at COMPILE time, ends
       up in .rodata or .data, no code runs:               */
        struct Config { uint32_t rate; uint8_t mode; };
        constexpr Config g_config{48000, 2};

/* (b) construct on first use - ordered by dependency: */
        Device& device() {
                static Device instance{0x40000000};
                return instance;
        }
/* (thread-safe since C++11, via a guard variable and an
   atomic - which is what -fno-threadsafe-statics removes if
   you are single-threaded.)

/* (c) explicit two-phase init - the most common embedded
       pattern: */
        Device g_device;                /* trivially
                                           constructed */
        void system_init() {
                g_device.init(0x40000000);   /* YOU choose
                                                when       */
        }

/* (d) placement new into a static buffer, when you need
       control over both timing and location:              */
        alignas(Device) static std::byte storage[sizeof(Device)];
        Device *dev = new (storage) Device{0x40000000};

/* CHECK WHAT YOU HAVE: */
        readelf -x .init_array prog.elf
        nm prog.elf | grep -i 'global.*ctor\|_GLOBAL__sub_I'
/* An empty .init_array means no static constructors, which is
   the goal for a tightly controlled firmware image.      */
```

The standard versions

Version	What it added that matters here
C++11	`auto`, move semantics, lambdas, `constexpr`, `nullptr`, `enum class`, `unique_ptr`, `array`, `<atomic>`, the memory model
C++14	Relaxed `constexpr`, generic lambdas, `make_unique`
C++17	**`string_view`, `optional`, `variant`, structured bindings, `if constexpr`, guaranteed copy elision**
C++20	`concepts`, ranges, `span`, `consteval`, designated initialisers, three-way comparison, modules
C++23	`expected`, `mdspan`, `print`, deducing `this`

```
/* C++17 IS THE PRACTICAL BASELINE for embedded work in 2026:
   universally supported by the toolchains vendors ship, and
   it contains the features that matter most - if constexpr,
   string_view, optional, and guaranteed copy elision.

   C++20's `concepts` and `span` are worth having where the
   toolchain allows; modules are still not practical in most
   embedded toolchains.

   CHECK WHAT YOUR TARGET SUPPORTS before writing the coding
   standard - a vendor GCC 8 in a BSP constrains you to C++17
   partially, and finding that out after writing the code is
   expensive.                                                */
```

How it shows up

Symptom	Cause
Binary three times larger than the C version	Exceptions, RTTI, iostreams — check the flags
100 KB of code from one `std::cout`	iostreams pulls in locale and the whole facet machinery
Unexpected heap allocation	`std::string`, `std::function`, `std::vector` growth
Crash before `main`	A static constructor touching uninitialised hardware
Static object used before construction	Initialisation order — construct on first use
Link error: `__cxa_atexit` undefined	Static destructors on a freestanding target — `-fno-use-cxa-atexit`
Link error: `__gxx_personality_v0`	Exceptions enabled but no unwinder — `-fno-exceptions`
Unpredictable timing in a control loop	Allocation, or an exception path
Template error message is 200 lines	Normal without concepts; C++20 improves it substantially
Team disagrees about which features to use	No written subset — that is the actual problem

Check yourself

- ✓ List ten C++ features that cost nothing at run time and the bug each prevents.
- ✓ List eight that are not free and say what each costs.
- ✓ Write a defensible embedded C++ subset with flags and library rules.
- ✓ Convert a C function with several error paths into an RAII version.
- ✓ Prove that `unique_ptr` and `std::array` are free, on godbolt.

- ✓ Explain why `std::function` is not, and give two replacements.
- ✓ State the rule for which abstractions are free.
- ✓ Give four ways to avoid dynamic static initialisation.
- ✓ Check whether a binary has any static constructors.
- ✓ Choose a standard version for a project and justify it against the available toolchain.

Classes, encapsulation and RAII

A class is a struct with functions, access control and a guaranteed destructor. That last property — deterministic destruction tied to scope — is what makes C++ worth using for systems code, and this chapter is about applying it to hardware.

A class is a struct that cleans up

```
class Timer {
        volatile uint32_t *base_;       /* private by default */
        uint32_t period_;

public:
        explicit Timer(uintptr_t addr, uint32_t period)
                : base_(reinterpret_cast<volatile uint32_t *>(addr)),
                  period_(period)
        {
                base_[REG_PERIOD] = period_;
                base_[REG_CTRL]   = CTRL_ENABLE;
        }

        ~Timer()
        {
                base_[REG_CTRL] = 0;    /* ALWAYS runs on scope
                                           exit                */
        }

        void restart()      { base_[REG_COUNT] = 0; }
        uint32_t value() const { return base_[REG_COUNT]; }

private:
        static constexpr size_t REG_COUNT  = 0;
        static constexpr size_t REG_PERIOD = 1;
        static constexpr size_t REG_CTRL   = 2;
};

/* `class` members are PRIVATE by default; `struct` members
   are PUBLIC. That is the only difference between the two
   keywords. Use `struct` for plain data, `class` when there
   is an invariant to protect.

/* `explicit` PREVENTS IMPLICIT CONVERSION: */
        void f(Timer t);
        f(0x40000000);          /* without explicit, this
                                   COMPILES and silently
                                   constructs a Timer from an
                                   integer                  */
/* Mark every single-argument constructor `explicit` unless
   the implicit conversion is genuinely wanted. This is a
   real bug class, and the fix costs one word.

/* `const` ON A MEMBER FUNCTION means it does not modify the
   object - and is checked. A const Timer& can call value()
   but not restart(). Mark every member function const that
   can be; it costs nothing and it documents intent
   enforceably.                                          */
```

Initialisation

```
/* MEMBER INITIALISER LISTS initialise; assignment in the body
   default-constructs and THEN assigns.                     */

/* WORSE: */
        Buffer(size_t n) {
                size_ = n;              /* size_ was already
                                           default-initialised */
                data_ = new uint8_t[n]; /* ditto */
        }

/* BETTER: */
        Buffer(size_t n)
                : size_(n), data_(new uint8_t[n]) {}

/* AND FOR A MEMBER WITH NO DEFAULT CONSTRUCTOR, or a const or
   reference member, the initialiser list is the ONLY option:
*/
        class Driver {
                Timer timer_;           /* no default ctor */
                const uint32_t id_;
                Bus& bus_;              /* a reference */
        public:
                Driver(uintptr_t addr, uint32_t id, Bus& b)
                        : timer_(addr, 1000), id_(id), bus_(b) {}
        };

/* THE ORDER TRAP: members are initialised in DECLARATION
   order, NOT in the order they appear in the initialiser
   list.                                                    */
        class Bad {
                int b_;
                int a_;
        public:
                Bad() : a_(1), b_(a_) {}   /* b_ initialised
                                              FIRST, from an
                                              uninitialised a_ */
        };
/* -Wreorder warns. Keep the list in declaration order.

/* DEFAULT MEMBER INITIALISERS (C++11) - usually the clearest:
*/
        class Config {
                uint32_t rate_  = 48000;
                uint8_t  gain_  = 0;
                bool     enabled_ = false;
        public:
                Config() = default;     /* uses the above */
                explicit Config(uint32_t rate) : rate_(rate) {}
                                        /* the others still
                                           get their defaults */
        };

/* DELEGATING CONSTRUCTORS, to avoid duplication: */
        Driver() : Driver(DEFAULT_ADDR, 0) {}
        Driver(uintptr_t addr, uint32_t id) { ... }

/* AND THE `{}` VERSUS `()` QUESTION: */
        std::vector<int> a(5, 0);       /* 5 elements, all 0 */
        std::vector<int> b{5, 0};       /* 2 elements: 5 and 0 */
/* Brace initialisation prefers an initializer_list
```

```
    constructor when one exists. Elsewhere braces are better -
    they forbid narrowing conversions:                       */
         int x{3.5};             /* ERROR - narrowing */
         int y(3.5);             /* silently 3 */
```

RAII for hardware

```
/* THE PATTERN: acquire in the constructor, release in the
   destructor. Then correctness does not depend on remembering
   anything.                                                  */

/* --- a critical section --- */
class InterruptLock {
        uint32_t saved_;
public:
        InterruptLock() : saved_(disable_interrupts()) {}
        ~InterruptLock() { restore_interrupts(saved_); }

        InterruptLock(const InterruptLock&) = delete;
        InterruptLock& operator=(const InterruptLock&) = delete;
};

        void update_shared(void)
        {
                InterruptLock lock;     /* interrupts off */

                if (error) return;      /* restored HERE */
                shared_state++;
        }                               /* and HERE */

/* Every exit path restores. There is no `goto out` to forget.

/* --- a peripheral clock --- */
class ClockGate {
        uint32_t mask_;
public:
        explicit ClockGate(uint32_t mask) : mask_(mask)
                { CLOCK->ENABLE |= mask_; }
        ~ClockGate() { CLOCK->ENABLE &= ~mask_; }
        /* non-copyable */
};

/* --- a chip-select line --- */
class ChipSelect {
        Gpio& cs_;
public:
        explicit ChipSelect(Gpio& cs) : cs_(cs) { cs_.clear(); }
        ~ChipSelect() { cs_.set(); }
};

        void spi_transfer(...)
        {
                ChipSelect cs(cs_pin);          /* asserted */
                for (auto b : data) {
                        if (!spi_write(b))
                                return;         /* DEASSERTED */
                }
        }                                       /* deasserted */

/* --- a generic scope guard, for anything without a class --- */
template<typename F>
class ScopeExit {
        F f_;
        bool active_ = true;
public:
        explicit ScopeExit(F f) : f_(std::move(f)) {}
        ~ScopeExit() { if (active_) f_(); }
```

```
        void release() { active_ = false; }

        ScopeExit(const ScopeExit&) = delete;
        ScopeExit(ScopeExit&& o) noexcept
                : f_(std::move(o.f_)), active_(o.active_)
                { o.active_ = false; }
};

template<typename F>
ScopeExit<F> make_scope_exit(F f) { return ScopeExit<F>(f); }

        auto guard = make_scope_exit([&] { cleanup(); });
        ...
        guard.release();        /* commit - do NOT clean up */

/* THE `release()` IS THE USEFUL PART: acquire resources,
   register cleanup for each, and release the guards only when
   everything succeeded. That gives transactional semantics
   with no exception handling at all.                    */
```

WHY THIS BEATS EVERY C ALTERNATIVE

The `goto out` ladder (Linux chapter 26) is correct and requires the author to maintain it as the function changes. `devm_` and WDF parenting solve it for one specific framework. RAII solves it for *every* resource, in the language, checked by the compiler, and the generated code is identical. It is the reason to consider C++ even in a codebase that uses nothing else from it.

Modelling a register block

```
/* A struct mapped onto MMIO gives you named, typed register
   access with no runtime cost.                              */

struct UartRegs {
        volatile uint32_t DR;           /* 0x00 data */
        volatile uint32_t RSR;          /* 0x04 */
        volatile uint32_t _pad0[4];
        volatile uint32_t FR;           /* 0x18 flags */
        volatile uint32_t _pad1;
        volatile uint32_t ILPR;         /* 0x20 */
        volatile uint32_t IBRD;         /* 0x24 */
        volatile uint32_t FBRD;         /* 0x28 */
        volatile uint32_t LCRH;         /* 0x2C */
        volatile uint32_t CR;           /* 0x30 */
};
static_assert(offsetof(UartRegs, CR) == 0x30,
              "register layout mismatch");
static_assert(sizeof(UartRegs) == 0x34);

/* THE static_assert IS THE POINT: get the padding wrong and
   the BUILD fails, rather than the device behaving strangely.
   Put one on every register whose offset you care about.

class Uart {
        UartRegs *regs_;
public:
        explicit Uart(uintptr_t base)
                : regs_(reinterpret_cast<UartRegs *>(base)) {}

        void put(char c) {
                while (regs_->FR & FR_TXFF) { }   /* full */
                regs_->DR = static_cast<uint32_t>(c);
        }

        void write(std::string_view s) {
                for (char c : s) put(c);
        }
};

        Uart uart{0x09000000};
        uart.write("hello\n");

/* GENERATES exactly the same loads and stores as the C
   version - `regs_->DR = c` is one `str`, and the class
   wrapper is entirely compiled away (chapter 14).

/* A SAFER ALTERNATIVE for the field manipulation, using
   templates so the shifts and masks are compile-time
   constants (chapter 07):                                */
template<uint32_t Mask, uint32_t Shift>
struct Field {
        static constexpr uint32_t get(uint32_t reg)
                { return (reg & Mask) >> Shift; }
        static constexpr uint32_t set(uint32_t reg, uint32_t v)
                { return (reg & ~Mask) | ((v << Shift) & Mask); }
};
        using Baud = Field<0x0000FF00, 8>;
        regs_->CR = Baud::set(regs_->CR, 115200 / 16);
```

Static members

```
class Counter {
        static inline int instances_ = 0;   /* C++17: define
                                               IN the class */
        int id_;
public:
        Counter() : id_(instances_++) {}
        ~Counter() { instances_--; }

        static int count() { return instances_; }
        /* a static member function - no `this`, callable as
           Counter::count()                                    */
};

/* BEFORE C++17 a static data member needed a definition in
   exactly one translation unit:                               */
        /* header */  class C { static int x; };
        /* one .cpp */ int C::x = 0;
/* `static inline` removes that, and it is one of C++17's
   most practically useful small features.

/* STATIC CONSTEXPR members need no definition at all and cost
   nothing: */
        class Uart {
                static constexpr uint32_t FR_TXFF = 1u << 5;
                static constexpr size_t   FIFO_DEPTH = 16;
        };
/* Scoped, typed, and compiled away entirely - strictly better
   than a #define in every respect.                          */
```

Hiding the implementation

```
/* THE PROBLEM: a class's private members are in its header,
   so changing them recompiles every user - and exposes
   internals.

/* PIMPL - a pointer to the implementation: */

        /* driver.hpp */
        class Driver {
                struct Impl;
                std::unique_ptr<Impl> impl_;
        public:
                Driver();
                ~Driver();              /* MUST be declared
                                           here and DEFINED in
                                           the .cpp, where
                                           Impl is complete  */
                Driver(Driver&&) noexcept;
                Driver& operator=(Driver&&) noexcept;

                void start();
                int  read();
        };

        /* driver.cpp */
        struct Driver::Impl {
                Uart uart;
                std::array<uint8_t, 256> buffer;
                ...
        };
        Driver::Driver() : impl_(std::make_unique<Impl>()) {}
        Driver::~Driver() = default;    /* here, not in the
                                           header             */
        void Driver::start() { impl_->uart.enable(); }

/* THE COST: a heap allocation and an indirection per call. On
   embedded that is usually too much, and the alternatives
   are:
     · an abstract interface with a factory (a vtable, but no
       extra allocation)
     · fixed-size storage: aligned_storage sized by a
       static_assert against sizeof(Impl)
     · simply accepting that private members are visible in
       the header, which for firmware is normally fine

   USE PIMPL for libraries with a stable ABI. Do not use it
   for firmware.                                          */
```

How it shows up

Symptom	Cause
A member has a garbage value in the constructor	Members initialise in *declaration* order — `-Wreorder`
An integer silently became an object	Non-`explicit` single-argument constructor
A resource leaks despite RAII	The object was copied — delete or implement the copy operations (ch. 03)
Destructor runs twice on the same handle	Same cause
Register writes reordered or elided	Missing `volatile` on the register struct

Symptom	Cause
Device behaves as if registers are shifted	Padding wrong — add `static_assert` on offsets
Two elements instead of five in a vector	Braces selected the `initializer_list` constructor
Narrowing conversion accepted silently	Used `()` instead of `{}`
"incomplete type" error on a Pimpl destructor	Destructor defaulted in the header
Link error on a static data member	No definition — use `static inline` (C++17)

Check yourself

- ✓ State the only difference between `class` and `struct`.
- ✓ Explain why every single-argument constructor should be `explicit`.
- ✓ Explain member initialisation order and the bug it causes.
- ✓ Say when an initialiser list is mandatory.
- ✓ Explain the `{}` versus `()` difference, both ways round.
- ✓ Write RAII wrappers for an interrupt lock, a clock gate and a chip select.
- ✓ Write a scope guard with a `release()` and explain what it enables.
- ✓ Map a register block with a struct and prove the layout with `static_assert`.
- ✓ Explain why `static constexpr` members beat `#define`.
- ✓ Explain Pimpl, its cost, and when not to use it.

Construction, destruction and the rule of zero/three/five

Six functions the compiler may generate for you, and the rules about when it does. Getting these wrong is how an RAII class silently double-frees a resource — and the correct answer is usually to write none of them.

The six special members

```
class Widget {
public:
        Widget();                                /* 1. default
                                                    constructor */
        ~Widget();                               /* 2. destructor */
        Widget(const Widget&);                   /* 3. copy ctor */
        Widget& operator=(const Widget&);        /* 4. copy
                                                    assignment  */
        Widget(Widget&&) noexcept;               /* 5. move ctor */
        Widget& operator=(Widget&&) noexcept;    /* 6. move
                                                    assignment  */
};

/* THE COMPILER GENERATES THESE IMPLICITLY, but the rules for
   WHEN are subtle and consequential:

   DEFAULT CONSTRUCTOR
        generated if you declare NO constructor at all.
        Declare any constructor and it disappears.

   DESTRUCTOR
        always generated unless you declare one.

   COPY CONSTRUCTOR and COPY ASSIGNMENT
        generated unless you declare a MOVE operation.
        DEPRECATED if you declare a destructor or the other
        copy operation - it still happens, with a warning.

   MOVE CONSTRUCTOR and MOVE ASSIGNMENT
        generated ONLY IF you declare NONE of: a destructor,
        either copy operation, or the other move operation.

   THE LAST RULE IS THE TRAP. Write a destructor - which every
   RAII class does - and you SILENTLY LOSE the move
   operations. Your class then falls back to COPYING, which
   for a resource-owning class is exactly wrong.         */
```

THE DEFAULT COPY IS A SHALLOW COPY

```
class FileHandle {
        int fd_;
public:
        explicit FileHandle(const char *p) : fd_(open(p, O_RDONLY)) {}
        ~FileHandle() { close(fd_); }
        /* copy operations are IMPLICITLY GENERATED, and copy
           fd_ verbatim */
};

        FileHandle a("/dev/thing");
        FileHandle b = a;       /* both hold the SAME fd */
        /* both destructors run -> close(fd) TWICE.
           The second close may close a DIFFERENT file that
           reused the descriptor number. */
```

This is the single most common C++ resource bug, and it is silent: no warning, correct-looking code, corruption at a distance. The fix is one of the three rules below.

The three rules

```
/* ---- RULE OF ZERO: the one to aim for ------------------ */
/* If your class has no raw resources, declare NONE of the six
   and let the compiler generate all of them correctly.    */

class Sensor {
        std::unique_ptr<Device> dev_;   /* handles its own
                                           lifetime */
        std::array<uint8_t, 64> buf_;   /* trivially copyable */
        std::string name_;              /* handles its own */
public:
        /* NOTHING. Move works, copy is correctly deleted
           (because unique_ptr is non-copyable), destruction
           is correct.                                       */
};

/* THIS IS THE GOAL. Push resource ownership into members
   that already do it right, and the enclosing class needs no
   special members at all.

/* ---- RULE OF FIVE: when you own a raw resource --------- */
class Buffer {
        uint8_t *data_ = nullptr;
        size_t   size_ = 0;

public:
        explicit Buffer(size_t n)
                : data_(new uint8_t[n]), size_(n) {}

        ~Buffer() { delete[] data_; }

        /* copy: DEEP */
        Buffer(const Buffer& o)
                : data_(new uint8_t[o.size_]), size_(o.size_)
        {
                std::copy_n(o.data_, size_, data_);
        }

        Buffer& operator=(const Buffer& o)
        {
                if (this == &o) return *this;   /* SELF-
                                                   ASSIGNMENT */
                uint8_t *p = new uint8_t[o.size_];  /* allocate
                                                       FIRST  */
                std::copy_n(o.data_, o.size_, p);
                delete[] data_;                 /* then release */
                data_ = p;
                size_ = o.size_;
                return *this;
        }

        /* move: STEAL */
        Buffer(Buffer&& o) noexcept
                : data_(o.data_), size_(o.size_)
        {
                o.data_ = nullptr;      /* leave it DESTRUCTIBLE */
                o.size_ = 0;
        }

        Buffer& operator=(Buffer&& o) noexcept
        {
```

```
                if (this == &o) return *this;
                delete[] data_;
                data_ = o.data_;
                size_ = o.size_;
                o.data_ = nullptr;
                o.size_ = 0;
                return *this;
        }
};

/* THE TWO NON-OBVIOUS REQUIREMENTS:
     1. SELF-ASSIGNMENT must work. `x = x` happens through
        references and aliases more often than you expect.
     2. THE MOVED-FROM OBJECT must be left in a valid,
        destructible state. Not "correct" - just safe to
        destroy and safe to assign to.

/* ---- RULE OF THREE: pre-C++11 ------------------------- */
/* Destructor, copy constructor, copy assignment. If you need
   one, you need all three. Still true; the rule of five just
   adds the move pair.                                       */
```

Saying what you mean

```
/* `= default` and `= delete` make intent explicit and
   checkable.                                              */

/* --- MOVE-ONLY, the right shape for most resource types --- */
class FileHandle {
        int fd_ = -1;
public:
        explicit FileHandle(const char *p) : fd_(open(p, O_RDONLY)) {}
        ~FileHandle() { if (fd_ >= 0) close(fd_); }

        FileHandle(const FileHandle&)            = delete;
        FileHandle& operator=(const FileHandle&) = delete;

        FileHandle(FileHandle&& o) noexcept : fd_(o.fd_)
                { o.fd_ = -1; }
        FileHandle& operator=(FileHandle&& o) noexcept
        {
                if (this != &o) {
                        if (fd_ >= 0) close(fd_);
                        fd_ = o.fd_;
                        o.fd_ = -1;
                }
                return *this;
        }

        int get() const { return fd_; }
};
/* Now `FileHandle b = a;` is a COMPILE ERROR, not a
   double-close. That is the whole point.

/* --- NON-COPYABLE, NON-MOVABLE: a scope guard ----------- */
class InterruptLock {
public:
        InterruptLock();
        ~InterruptLock();
        InterruptLock(const InterruptLock&)            = delete;
        InterruptLock& operator=(const InterruptLock&) = delete;
        /* declaring the copy ops suppresses the move ops too */
};

/* --- BRINGING BACK A DEFAULT that was suppressed --------- */
class C {
public:
        C(int x);               /* suppresses the default ctor */
        C() = default;          /* bring it back */
        ~C() = default;
        C(C&&) = default;       /* and the moves, which the
        C& operator=(C&&) = default;   destructor suppressed */
};

/* --- AND FOR A BASE CLASS with virtual functions
       (chapter 11) --- */
        virtual ~Base() = default;
/* A polymorphic base needs a VIRTUAL destructor, or deleting
   through a Base* does not run the derived destructor. And
   declaring it suppresses the moves, so default them
   explicitly if you want them.                            */
```

> **`NOEXCEPT` ON MOVE OPERATIONS IS NOT DECORATION**
>
> `std::vector` reallocating chooses between moving and copying its elements based on whether the move constructor is `noexcept` — because if a move can throw partway through, the vector cannot restore its previous state. A move constructor that is not marked `noexcept` causes `vector` growth to *copy* every element instead. That is a large, silent performance loss, and the fix is one keyword.

Construction and destruction order

```
class Derived : public Base {
        Member1 m1_;
        Member2 m2_;
public:
        Derived() : Base(), m1_(), m2_() { /* body */ }
};

/* CONSTRUCTION ORDER:
     1. base classes, in declaration order
     2. members, in DECLARATION order
     3. the constructor body

   DESTRUCTION IS EXACTLY REVERSED:
     1. the destructor body
     2. members, in reverse declaration order
     3. base classes, in reverse

   WHICH MEANS: a member declared later may safely depend on
   one declared earlier, and not the other way round. Order
   the declarations to match the dependencies.

/* CALLING A VIRTUAL FUNCTION IN A CONSTRUCTOR: */
class Base {
public:
        Base() { init(); }              /* calls BASE::init */
        virtual void init() { }
};
class Derived : public Base {
        int *p_;
public:
        Derived() : p_(new int) { }
        void init() override { *p_ = 1; }   /* NEVER CALLED
                                               from Base's
                                               ctor - and p_
                                               is not yet
                                               initialised
                                               anyway       */
};
/* During Base's constructor the object IS a Base; the vtable
   pointer has not yet been updated. Same in destructors.
   NEVER call a virtual function from a constructor or
   destructor - use two-phase init if you need it.       */

/* DESTRUCTION OF LOCALS is reverse of construction, which is
   exactly what RAII needs: */
        void f() {
                ClockGate  clk(CLK_UART);   /* 1st */
                Uart       uart(0x9000000); /* 2nd */
                ChipSelect cs(pin);         /* 3rd */
                ...
        }   /* cs released, then uart, then the clock -
               correct: the clock must outlive the device
               that needs it.                          */
```

Copy elision

```
/* The compiler may - and since C++17 for some cases MUST -
   construct an object directly in its destination rather than
   constructing and copying.                                  */

        Buffer make() {
                return Buffer(1024);    /* GUARANTEED elision
                                           since C++17: the
                                           object is
                                           constructed
                                           directly in the
                                           caller's storage */
        }
        Buffer b = make();      /* NO copy, NO move, not even
                                   a call to them            */

/* NAMED RETURN VALUE OPTIMISATION - allowed, not guaranteed:
*/
        Buffer make2() {
                Buffer b(1024);
                b.fill(0);
                return b;       /* NRVO usually elides */
        }

/* AND THE PESSIMISATION TO AVOID: */
        return std::move(b);    /* WORSE - the explicit move
                                   DEFEATS NRVO, forcing an
                                   actual move construction */
/* -Wpessimizing-move warns. Just `return b;` - the compiler
   already treats a returned local as an rvalue.

/* THE MECHANISM (compilers chapter 10): the caller passes a
   hidden pointer in x8, and the callee constructs the result
   there. So a "return by value" of a large object is one
   construction, in place - not a construct-then-copy.

   WHICH MEANS: returning by value is the right default, even
   for large types. The out-parameter idiom C requires is
   unnecessary in C++.                                    */
```

Checking your class

```
/* --- ASK THE TYPE SYSTEM --- */
        static_assert(std::is_nothrow_move_constructible_v<Buffer>);
        static_assert(!std::is_copy_constructible_v<FileHandle>);
        static_assert(std::is_trivially_copyable_v<Point>);
        static_assert(std::is_trivially_destructible_v<Config>);

/* `is_trivially_copyable` matters for embedded: a trivially
   copyable type can be memcpy'd, DMA'd and stored in a
   union safely. Assert it on any type you treat that way.

/* --- WARNINGS THAT CATCH THESE BUGS --- */
        -Wdeprecated-copy               /* implicit copy with a
                                           user destructor   */
        -Wdeprecated-copy-dtor
        -Wnon-virtual-dtor              /* polymorphic base
                                           without one       */
        -Wreorder                       /* initialiser order */
        -Wpessimizing-move
        -Wredundant-move
        -Weffc++                        /* GCC: a broad set of
                                           these             */

/* --- AND clang-tidy --- */
        cppcoreguidelines-special-member-functions
        performance-move-const-arg
        modernize-use-equals-default
        modernize-use-equals-delete
/* Run these. The rule-of-five check in particular finds real
   double-free bugs in existing code.                    */
```

How it shows up

Symptom	Cause
Double free, or a file descriptor closed twice	Implicit shallow copy of a resource-owning class
Use-after-free after passing an object by value	Same
`vector` growth is unexpectedly slow	Move constructor not `noexcept` — it copied instead
A move left the source usable and the destination empty	Move operations not implemented; it copied
Crash on self-assignment	No self-assignment check in copy assignment
Member is garbage in the constructor body	Declaration order versus initialiser list order
A virtual override is not called from a constructor	By design — the vtable is not yet the derived one
Derived destructor never runs	Base destructor not `virtual`
Resources released in the wrong order	Declaration order of members or locals
An expected NRVO did not happen	`return std::move(x)` defeated it
"use of deleted function"	Correct — the class is non-copyable and you tried to copy it

Check yourself

- ✓ Name the six special members and when each is implicitly generated.
- ✓ Explain why declaring a destructor suppresses the move operations, and what follows.

- ✓ Demonstrate the shallow-copy double-free and give three ways to prevent it.
- ✓ State the rules of zero, three and five, and say which to aim for.
- ✓ Write a correct rule-of-five class including self-assignment and moved-from state.
- ✓ Explain why move operations should be `noexcept`.
- ✓ State construction and destruction order, including bases and members.
- ✓ Explain why virtual calls in constructors do not dispatch as expected.
- ✓ Explain guaranteed copy elision and why `return std::move(x)` is wrong.
- ✓ Use type traits and warnings to verify a class's copy/move behaviour.

References, value categories and overload resolution

References, the five value categories, and the rules by which C++ picks one function from several candidates. This is the machinery underneath move semantics and templates, and it is where C++'s apparent unpredictability usually comes from.

References

```
        int x = 42;
        int&  r = x;            /* an lvalue reference - an
                                   ALIAS for x               */
        r = 10;                 /* x is now 10 */

/* PROPERTIES:
     · must be initialised - there is no null reference
     · cannot be reseated - `r = y` assigns to x, it does not
       make r refer to y
     · no separate object exists; the compiler implements it
       as a pointer, usually optimised away entirely

/* CONST REFERENCES bind to more things: */
        const int& cr = 42;     /* binds to a TEMPORARY, and
                                   EXTENDS ITS LIFETIME to
                                   that of cr                */
        int& bad = 42;          /* ERROR */

/* RVALUE REFERENCES (C++11) bind ONLY to rvalues: */
        int&& rr = 42;          /* OK */
        int&& bad2 = x;         /* ERROR - x is an lvalue */
        int&& ok = std::move(x); /* OK - move() casts to
                                    rvalue                   */

/* WHEN TO USE WHICH, in a function parameter:

     void f(T v)           by VALUE - for cheap types, or when
                           you need a copy anyway
     void f(const T& v)    by CONST REF - the default for
                           anything non-trivial you only read
     void f(T& v)          by REF - when you modify the
                           caller's object
     void f(T&& v)         by RVALUE REF - when you will STEAL
                           from it (chapter 05)
     void f(const T&& v)   almost never useful

/* THE THRESHOLD: pass by value for types up to about two
   registers (16 bytes on AArch64 - chapter 10's ABI), by
   const reference above that. int, float, pointers, small
   structs, string_view, span: by value. Everything else: by
   const reference.                                          */

/* THE DANGLING REFERENCE, and it is easy to write: */
        const std::string& f() {
                std::string s = "hello";
                return s;               /* DANGLING - s is
                                           destroyed         */
        }
        const int& bad = std::vector<int>{1,2,3}[0];
                                        /* the vector dies at
                                           the end of the
                                           statement         */
/* -Wreturn-local-addr and -Wdangling-gsl catch some of these;
   ASan catches the rest at run time.                       */
```

The five value categories

```
C++11 split C's lvalue/rvalue into a lattice, because move
semantics needed a third case.

                  expression
                 /          \
            glvalue          rvalue
           /       \        /     \
      lvalue        xvalue         prvalue

THE TWO PROPERTIES that define them:

  has IDENTITY?   can you take its address, does it name a
                  specific object?
  is MOVABLE?     may its resources be stolen?

  lvalue    identity, NOT movable      a named variable
  xvalue    identity, MOVABLE          std::move(x), a
                                       function returning T&&
  prvalue   NO identity, movable       42, f() returning T,
                                       a temporary

  glvalue = lvalue + xvalue     "has identity"
  rvalue  = xvalue + prvalue    "movable"

EXAMPLES:
     int x = 1;
     x                       lvalue
     42                      prvalue
     x + 1                   prvalue
     std::move(x)            xvalue
     f()   returning int     prvalue
     f()   returning int&    lvalue
     f()   returning int&&   xvalue
     arr[0]                  lvalue
     "hello"                 lvalue  (a string literal is
                                      an array object)

THE PRACTICAL RULE, which covers almost every case:
  IF IT HAS A NAME, IT IS AN LVALUE.
Even a parameter declared `T&& x` is an lvalue INSIDE the
function, because it has a name. That is precisely why
std::move is needed to pass it on (chapter 05).
```

Overload resolution

```
/* Given several candidates, C++ picks one by a defined
   procedure. Knowing it removes most surprise.            */

   1. NAME LOOKUP: gather candidates. Ordinary scope lookup,
      plus ARGUMENT-DEPENDENT LOOKUP (below).

   2. Discard candidates that cannot be called with these
      arguments (wrong arity, or no conversion exists).

   3. RANK the conversions for each argument:

        EXACT MATCH               identity, lvalue-to-rvalue,
                                  array-to-pointer,
                                  qualification (T* -> const T*)
        PROMOTION                 char -> int, float -> double
        STANDARD CONVERSION       int -> double, D* -> B*,
                                  anything -> bool
        USER-DEFINED CONVERSION   a converting constructor or
                                  a conversion operator
                                  (AT MOST ONE per argument)
        ELLIPSIS                  ...

   4. The BEST candidate is the one that is no worse for every
      argument and better for at least one. If none is, the
      call is AMBIGUOUS - an error.

   5. Templates lose to equally-good non-templates.

/* WORKED: */
        void f(int);            /* 1 */
        void f(double);         /* 2 */
        void f(const char*);    /* 3 */

        f(42);                  /* 1: exact */
        f(42.0);                /* 2: exact */
        f('a');                 /* 1: PROMOTION beats
                                   conversion             */
        f(42.0f);               /* 2: promotion */
        f("s");                 /* 3: array-to-pointer is
                                   exact                  */
        f(nullptr);             /* 3 */
        f(0);                   /* 1 - and this is why
                                   nullptr exists         */

/* AMBIGUITY: */
        void g(long);
        void g(float);
        g(42);                  /* ERROR: int -> long and
                                   int -> float are both
                                   standard conversions,
                                   neither better          */

/* THE LVALUE/RVALUE OVERLOAD, which is what move semantics
   is built on: */
        void h(const T&);       /* binds lvalues AND rvalues */
        void h(T&&);            /* binds ONLY rvalues, and
                                   PREFERRED for them       */

        T a;
        h(a);                   /* const T& */
```

```
        h(T{});                 /* T&& - better match */
        h(std::move(a));        /* T&& */
```

Argument-dependent lookup

```
/* For an unqualified call, C++ ALSO searches the namespaces
   of the argument types. Without this, operator overloading
   would not work across namespaces.                        */

        namespace lib {
                struct Widget {};
                void process(const Widget&);
                std::ostream& operator<<(std::ostream&,
                                         const Widget&);
        }

        lib::Widget w;
        process(w);             /* FOUND via ADL, without
                                   `lib::`                  */
        std::cout << w;         /* likewise - and this is why
                                   operator<< can be defined
                                   in the type's own
                                   namespace                */

/* THE IMPORTANT IDIOM it enables - customisation points: */
        template<typename T>
        void algorithm(T& a, T& b)
        {
                using std::swap;        /* the fallback */
                swap(a, b);             /* UNQUALIFIED: ADL
                                           finds a
                                           type-specific swap
                                           if one exists,
                                           otherwise
                                           std::swap        */
        }
/* Writing `std::swap(a, b)` instead would ALWAYS use the
   generic one, missing an optimised overload. The
   `using` + unqualified call pattern is the standard
   solution, and you will see it throughout the STL.

/* THE HAZARD: ADL can find candidates you did not intend,
   particularly with templates and with types from large
   namespaces. If an overload is chosen that surprises you,
   ADL is usually why - and the fix is to qualify the call.  */
```

THE TOOLS FOR "WHICH OVERLOAD WAS CHOSEN?"

```
clang++ -Xclang -ast-dump -fsyntax-only file.cpp
```

The AST names the exact function selected and every implicit conversion applied (compilers chapter 04). On an ambiguity error, Clang lists every candidate with the reason it was rejected — read that list rather than guessing. These two habits resolve almost every overload question in under a minute.

const and its propagation

```
        const int  ci = 1;
        int const  ci2 = 1;             /* the SAME thing */

        const int *p1;          /* pointer to const int -
                                   *p1 is const            */
        int const *p2;          /* identical to p1 */
        int * const p3 = &x;    /* CONST POINTER to int -
                                   p3 cannot be reseated   */
        const int * const p4 = &ci;     /* both */

/* READ RIGHT TO LEFT: `int * const` is "const pointer to
   int". Consistently declaring `int const *` rather than
   `const int *` makes the rule uniform - some codebases
   require it for that reason.

/* CONST MEMBER FUNCTIONS: */
        class Buffer {
                uint8_t *data_;
        public:
                uint8_t& at(size_t i)       { return data_[i]; }
                const uint8_t& at(size_t i) const
                                             { return data_[i]; }
        };
/* Two overloads differing only in constness - a const object
   gets the const one. Standard, and slightly tedious;
   C++23's "deducing this" collapses it into one.

/* CONSTNESS IS SHALLOW: a const member function may modify
   what a member pointer POINTS TO: */
        void Buffer::clear() const {
                data_[0] = 0;           /* LEGAL - data_ is
                                           const, *data_ is
                                           not               */
        }
/* Which is exactly the same rule as C, and a real source of
   false confidence. `std::experimental::propagate_const`
   exists for this, and is not widely used.

/* `mutable` - modifiable even in a const function: */
        class Cache {
                mutable bool valid_ = false;
                mutable int  cached_;
        public:
                int get() const {
                        if (!valid_) { cached_ = compute();
                                       valid_ = true; }
                        return cached_;
                }
        };
/* LEGITIMATE for caches, mutexes and lazily computed values -
   things that do not change the object's OBSERVABLE state.
   Anything else is a lie to the reader.                   */

/* AND THE EMBEDDED POINT: `const` on a namespace-scope object
   with a constant initialiser puts it in .rodata, which on a
   microcontroller means FLASH rather than RAM. On a part with
   32 KB of RAM that is not a stylistic matter.           */
```

Implicit conversions

```
/* C++ will convert types for you, and that is the source of
   several surprises. */

/* --- CONVERTING CONSTRUCTORS - block them --- */
        class Timeout {
                uint32_t ms_;
        public:
                Timeout(uint32_t ms) : ms_(ms) {}   /* implicit! */
        };
        void wait(Timeout t);
        wait(1000);             /* compiles - is 1000 ms, or
                                   seconds, or ticks?       */

        explicit Timeout(uint32_t ms);
        wait(1000);             /* now an ERROR */
        wait(Timeout{1000});    /* explicit at the call site */

/* --- CONVERSION OPERATORS - block those too --- */
        class Handle {
        public:
                operator int() const;   /* implicit - allows
                                           `handle + 1`, and
                                           every other
                                           nonsense           */
                explicit operator bool() const;  /* the useful
                                                    one       */
        };
        if (h) { }              /* explicit operator bool is
                                   still used HERE - a
                                   "contextual conversion" */

/* --- THE UNIFORM RULE that avoids all of it ---
     mark EVERY single-argument constructor and EVERY
     conversion operator `explicit`, except `operator bool`.
   Then conversions happen only where you wrote them.

/* --- ARITHMETIC CONVERSIONS still apply, exactly as in C
       (compilers chapter 04), including the signed/unsigned
       trap. Enable -Wconversion -Wsign-conversion.

/* --- AND enum class, which does NOT convert --- */
        enum        Old { A, B };       /* converts to int */
        enum class  New { A, B };       /* does NOT */

        int x = A;                      /* fine, and usually
                                           unwanted           */
        int y = New::A;                 /* ERROR */
        int z = static_cast<int>(New::A);   /* explicit */

        enum class Mode : uint8_t { Idle, Run, Fault };
/* An explicit underlying type is worth setting on embedded -
   it fixes the size and lets you use it in a packed
   structure.                                              */
```

How it shows up

Symptom	Cause
Garbage from a returned reference	Dangling — the referent was a local or a temporary
A move did not happen	The argument had a name, so it was an lvalue — needs `std::move`
"call of overloaded f(...) is ambiguous"	Two equally-good conversions — read Clang's candidate list
The wrong overload chosen	Promotion beat conversion, or ADL found something unexpected
An integer silently became an object	Non-`explicit` converting constructor
An object silently became an integer	Non-`explicit` conversion operator
A type-specific `swap` is never used	Called `std::swap` qualified instead of the ADL idiom
A const method modified the object	Constness is shallow — it went through a pointer member
A const array is in RAM, not flash	Non-constant initialiser, or it is not actually `const`
Comparison between enum and int compiles	Plain `enum` — use `enum class`
`f(0)` selects the pointer overload, or does not	Use `nullptr`, never `0` or `NULL`

Check yourself

- ✓ State the properties of lvalue, const-lvalue and rvalue references.
- ✓ Give the parameter-passing rule and its size threshold on AArch64.
- ✓ Name the five value categories and the two properties that define them.
- ✓ Classify eight expressions by value category.
- ✓ State the practical rule about names, and why it matters for moves.
- ✓ Walk the overload resolution procedure and rank the conversion categories.
- ✓ Explain why `f('a')` chooses the `int` overload.
- ✓ Explain ADL and write the `using std::swap` idiom correctly.
- ✓ Explain shallow constness and when `mutable` is legitimate.
- ✓ Say why every converting constructor should be `explicit`, with the one exception.

Move semantics and perfect forwarding

Transferring ownership instead of duplicating it. Move semantics is what makes `unique_ptr`, `vector` and every other resource-owning type efficient — and perfect forwarding is how a generic wrapper passes arguments on without losing information about them.

The problem it solves

```
/* Before C++11, returning a container copied it: */
        std::vector<int> make() {
                std::vector<int> v(1'000'000);
                fill(v);
                return v;               /* a COPY of a million
                                           ints, then the
                                           original is
                                           destroyed          */
        }

/* The copy is pure waste: the source is about to be
   destroyed. What we want is to STEAL its buffer.

/* WITH MOVE SEMANTICS: */
        vector(vector&& other) noexcept
                : data_(other.data_), size_(other.size_),
                  cap_(other.cap_)
        {
                other.data_ = nullptr;  /* the source no longer
                other.size_ = 0;           owns it - and is
                other.cap_  = 0;           still destructible */
        }

/* THREE POINTER ASSIGNMENTS instead of a million-element
   copy. That is the whole idea.

/* A MOVE IS NOT A SPECIAL OPERATION - it is just an OVERLOAD
   that takes an rvalue reference, and by convention steals
   rather than copies. The language provides the overload
   resolution; the stealing is yours to implement
   (chapter 03).                                          */
```

std::move

```
/* std::move does NOT move anything. It is a CAST to an rvalue
   reference, which changes which overload is selected.    */

        template<typename T>
        constexpr std::remove_reference_t<T>&&
        move(T&& t) noexcept
        {
                return static_cast<std::remove_reference_t<T>&&>(t);
        }

/* That is the entire implementation. It compiles to nothing.

/* USING IT: */
        std::vector<int> a = make();
        std::vector<int> b = a;                 /* COPY */
        std::vector<int> c = std::move(a);      /* MOVE - a is
                                                   now empty */

/* WHY IT IS NEEDED even inside a function taking T&&: */
        void sink(std::vector<int>&& v) {
                store(v);                       /* COPIES! v
                                                   HAS A NAME,
                                                   so it is an
                                                   LVALUE
                                                   (ch. 04)  */
                store(std::move(v));            /* moves */
        }
/* This is the single most common misunderstanding. An rvalue
   reference PARAMETER is an lvalue inside the function.

/* THE MOVED-FROM STATE: "valid but unspecified". You may
   destroy it and assign to it; you may not assume its value.
   For the standard containers it is in fact empty, but do not
   rely on that.                                            */
        std::string s = "hello";
        std::string t = std::move(s);
        /* s.size() is unspecified - do not read it */
        s = "new value";        /* fine - assignment is
                                   allowed */

/* WHEN NOT TO USE IT: */
        return std::move(local);        /* WRONG - defeats
                                           NRVO (chapter 03) */
        void f(const T& x) { g(std::move(x)); }
                                        /* USELESS - a const
                                           rvalue still binds
                                           to const T&, so it
                                           COPIES            */
        std::move(trivially_copyable)   /* pointless - no
                                           resources to steal */
```

Forwarding references

```
/* `T&&` in a DEDUCED context is not an rvalue reference. It
   is a FORWARDING (or "universal") reference, and it binds to
   anything.                                                  */

        template<typename T>
        void f(T&& x);          /* FORWARDING reference */

        void g(Widget&& x);     /* rvalue reference - T is not
                                   deduced                   */

        template<typename T>
        void h(std::vector<T>&& v);   /* rvalue reference -
                                         the type is not
                                         exactly T          */

/* THE RULE: `T&&` is a forwarding reference ONLY when T is a
   template parameter of that function being deduced. Also
   `auto&&`.

/* HOW IT BINDS - REFERENCE COLLAPSING: */
        T&  &   -> T&
        T&  &&  -> T&
        T&& &   -> T&
        T&& &&  -> T&&
/* "an lvalue reference wins".

        Widget w;
        f(w);                   /* T = Widget&, so
                                   T&& = Widget& && = Widget& */
        f(Widget{});            /* T = Widget, so
                                   T&& = Widget&&           */

/* SO T ENCODES WHETHER THE ARGUMENT WAS AN LVALUE OR AN
   RVALUE - and std::forward decodes it again.            */
```

Perfect forwarding

```
/* THE PROBLEM: a wrapper must pass its arguments on with
   their value category intact.                          */

/* NAIVE - always copies: */
        template<typename T>
        void wrapper(T x) { target(x); }

/* BETTER, still wrong - x has a name, so it is an lvalue: */
        template<typename T>
        void wrapper(T&& x) { target(x); }   /* always copies */

/* CORRECT: */
        template<typename T>
        void wrapper(T&& x) { target(std::forward<T>(x)); }

/* std::forward<T> casts to T&&, which after reference
   collapsing is:
        T = Widget&  -> Widget&   (stays an lvalue)
        T = Widget   -> Widget&&  (becomes an rvalue)
   Exactly preserving what was passed.

/* THE VARIADIC FORM, which is what every factory looks like:
*/
        template<typename T, typename... Args>
        std::unique_ptr<T> make_unique(Args&&... args)
        {
                return std::unique_ptr<T>(
                        new T(std::forward<Args>(args)...));
        }

        auto p = make_unique<Widget>(x, std::move(y), 42);
        /* x is copied into the constructor, y is moved, 42 is
           passed as an rvalue - each correctly               */

/* THE RULE, memorised as a pair:
     std::move     on an rvalue reference - you are done with
                   it
     std::forward  on a forwarding reference - pass on what
                   you were given
   Using move where forward belongs steals from the caller's
   lvalue, which is a real and nasty bug.                 */

/* AND THE PLACE IT MATTERS MOST - emplace: */
        std::vector<Widget> v;
        v.push_back(Widget{1, 2});      /* construct, then MOVE
                                           into the vector    */
        v.emplace_back(1, 2);           /* construct IN PLACE -
                                           no move at all     */
/* emplace_back forwards its arguments to the element's
   constructor. For a type where move is cheap the difference
   is small; for one where it is not, it is the whole cost. */
```

THE FORWARDING REFERENCE TRAP

```
template<typename T>
void f(T&& x);          // matches EVERYTHING
void f(int);            // rarely selected

f(42);                  // T&& wins: an exact match with
                        // T = int, versus a non-template
                        // that also needs no conversion –
                        // and the template's specialisation
                        // is exact for every argument type
```

A forwarding-reference overload is greedier than almost anything else, and it will silently hijack calls you intended for other overloads — including copy constructors, producing baffling errors. Constrain it with a concept (C++20) or `enable_if`, or avoid overloading on it at all.

Implementing move correctly

```
class Buffer {
        uint8_t *data_ = nullptr;
        size_t   size_ = 0;

public:
        Buffer(Buffer&& o) noexcept
                : data_(std::exchange(o.data_, nullptr)),
                  size_(std::exchange(o.size_, 0))
        {}
        /* std::exchange(obj, newval) sets obj and returns the
           OLD value - exactly the move idiom, in one call. */

        Buffer& operator=(Buffer&& o) noexcept
        {
                if (this != &o) {
                        delete[] data_;         /* release ours */
                        data_ = std::exchange(o.data_, nullptr);
                        size_ = std::exchange(o.size_, 0);
                }
                return *this;
        }
};

/* --- OR THE COPY-AND-SWAP IDIOM, which gives you both
       assignment operators from one function and is
       automatically self-assignment safe: --- */

        Buffer& operator=(Buffer o) noexcept   /* BY VALUE */
        {
                swap(*this, o);
                return *this;
        }                       /* o's destructor releases what
                                   we used to own            */

        friend void swap(Buffer& a, Buffer& b) noexcept
        {
                using std::swap;
                swap(a.data_, b.data_);
                swap(a.size_, b.size_);
        }

/* The by-value parameter is copy-constructed from an lvalue
   argument and move-constructed from an rvalue, so ONE
   operator handles both. Elegant, and slightly less efficient
   than separate operators for the copy case - which is why
   the standard library writes both.

/* --- THE noexcept REQUIREMENT, restated because it matters
       (chapter 03): --- */
        static_assert(std::is_nothrow_move_constructible_v<Buffer>);
/* Put that assertion in your test file. A move constructor
   that silently is not noexcept makes vector reallocation
   copy instead of move.

/* --- AND THE `= default` CASE --- */
        class Widget {
                std::unique_ptr<Impl> p_;
                std::string name_;
        public:
                Widget(Widget&&) noexcept = default;
```

```
                Widget& operator=(Widget&&) noexcept = default;
        };
/* Member-wise move, generated correctly. Prefer this to
   writing it by hand whenever every member is itself
   movable - which is the rule of zero again (chapter 03). */
```

Does it matter on embedded?

```
/* HONESTLY: less than in application code, because the types
   that benefit most - vector, string, shared_ptr - are often
   banned anyway (chapter 01). But it still matters in three
   places:

/* 1. MOVE-ONLY RESOURCE TYPES. This is the big one, and it
      is independent of the heap:                          */
        class DmaChannel {
                int ch_ = -1;
        public:
                explicit DmaChannel(int ch) : ch_(ch)
                        { dma_claim(ch_); }
                ~DmaChannel() { if (ch_ >= 0) dma_release(ch_); }

                DmaChannel(const DmaChannel&) = delete;
                DmaChannel(DmaChannel&& o) noexcept
                        : ch_(std::exchange(o.ch_, -1)) {}
                ...
        };

        DmaChannel acquire_channel();   /* RETURN BY VALUE,
                                           moved out, no
                                           allocation
                                           anywhere         */
        std::optional<DmaChannel> try_acquire();

/* THE OWNERSHIP IS IN THE TYPE. A DmaChannel cannot be
   accidentally copied, cannot be leaked, and can be returned
   from a factory - none of which is expressible in C without
   convention.

/* 2. FIXED-CAPACITY CONTAINERS still benefit: */
        etl::vector<Message, 16> queue;
        queue.push_back(std::move(msg));    /* moves the
                                               element's
                                               members       */

/* 3. std::optional and std::variant use moves internally, and
      both are extremely useful on embedded (chapter 15).

/* WHERE IT DOES NOT MATTER: a trivially copyable struct -
   a Point, a register value, a POD message - has no
   resources, so a move IS a copy. std::move on one is
   harmless and pointless.                                 */
```

Verifying it happens

```
/* Instrument a type and watch: */
struct Tracer {
        Tracer()                    { puts("ctor"); }
        Tracer(const Tracer&)       { puts("COPY"); }
        Tracer(Tracer&&) noexcept   { puts("move"); }
        Tracer& operator=(const Tracer&)
                { puts("COPY="); return *this; }
        Tracer& operator=(Tracer&&) noexcept
                { puts("move="); return *this; }
        ~Tracer()                   { puts("dtor"); }
};

        std::vector<Tracer> v;
        v.reserve(4);
        Tracer t;
        v.push_back(t);                 /* COPY */
        v.push_back(std::move(t));      /* move */
        v.emplace_back();               /* ctor - no copy or
                                           move at all      */

/* Drop the reserve() and watch reallocation move (or copy!)
   every element. Remove `noexcept` from the move constructor
   and watch it switch to COPY - that experiment makes the
   rule stick better than any explanation.

/* AND IN REAL CODE: */
        -Wpessimizing-move
        -Wredundant-move
        clang-tidy: performance-move-const-arg,
                    performance-unnecessary-value-param,
                    modernize-pass-by-value
        static_assert(std::is_nothrow_move_constructible_v<T>);
```

How it shows up

Symptom	Cause
A copy happens where you expected a move	The argument had a name — needs `std::move`
Same, in a template wrapper	Needs `std::forward`, not plain passing
Caller's object was emptied unexpectedly	Used `std::move` where `std::forward` belonged
`vector` growth copies elements	Move constructor not `noexcept`
Reading a moved-from object gives odd values	Its state is valid but unspecified — do not read it
Return value copied despite `std::move`	`std::move` on return defeats NRVO
`std::move` on a const object does nothing	A const rvalue binds to `const T&` — it copies
A template overload hijacks every call	Forwarding reference is too greedy — constrain it
Bizarre error mentioning the copy constructor	A forwarding-reference constructor competing with it
Move leaves the source unusable, then it is used	The moved-from object must remain valid — check your implementation

Check yourself

✓ Explain what problem move semantics solves and why a move is just an overload.

- ✓ Say what `std::move` actually does, and give three places not to use it.
- ✓ Explain why a `T&&` parameter is an lvalue inside the function.
- ✓ State when `T&&` is a forwarding reference and give the collapsing rules.
- ✓ Explain `std::forward` and the move-versus-forward rule.
- ✓ Write a correct move constructor and assignment using `std::exchange`.
- ✓ Write copy-and-swap and say what it costs.
- ✓ Explain why `noexcept` on moves changes `vector`'s behaviour.
- ✓ Give three places move semantics matters on embedded despite no heap.
- ✓ Use a tracer type to observe copies and moves, and to demonstrate the `noexcept` effect.

Templates and generic programming

Write one implementation, get a specialised copy for each type — with no runtime dispatch and no cost. Templates are how C++ achieves zero-overhead abstraction, and on embedded they are the mechanism for compile-time configuration.

Function and class templates

```
template<typename T>
constexpr const T& max(const T& a, const T& b)
{
        return a > b ? a : b;
}

        max(1, 2);              /* T = int, deduced */
        max(1.0, 2.0);          /* T = double */
        max<long>(1, 2);        /* T explicit */
        max(1, 2.0);            /* ERROR - T cannot be both */

/* EACH INSTANTIATION IS A SEPARATE FUNCTION, generated at
   compile time, and inlined like any other. `max(1,2)`
   compiles to a single `csel` - identical to the macro,
   with type checking and no double-evaluation bug.       */

template<typename T, size_t N>
class RingBuffer {
        std::array<T, N> data_{};
        size_t head_ = 0, tail_ = 0;

public:
        static constexpr size_t capacity = N;

        bool push(const T& v)
        {
                size_t next = (head_ + 1) % N;
                if (next == tail_) return false;    /* full */
                data_[head_] = v;
                head_ = next;
                return true;
        }

        bool pop(T& out)
        {
                if (head_ == tail_) return false;   /* empty */
                out = data_[tail_];
                tail_ = (tail_ + 1) % N;
                return true;
        }

        size_t size() const { return (head_ - tail_ + N) % N; }
};

        RingBuffer<uint8_t, 64>   rx;
        RingBuffer<Message, 16>   queue;

/* N IS A COMPILE-TIME CONSTANT, so:
     · the storage is a fixed array - NO HEAP
     · `% N` with N a power of two becomes an AND
     · the size is known, so bounds are checkable at compile
       time
   This is the embedded container pattern: capacity as a
   template parameter, storage inline.                    */

/* NON-TYPE TEMPLATE PARAMETERS can be integers, enums,
   pointers, and since C++20 floating point and literal class
   types:                                                  */
        template<uint32_t BaseAddr>
        class Uart {
```

```
                static constexpr auto *regs =
                        reinterpret_cast<volatile UartRegs *>(
                                BaseAddr);
        public:
                static void put(char c) { regs->DR = c; }
        };
        using Uart0 = Uart<0x09000000>;
/* The address is a constant folded into every access — no
   pointer is stored, and no register is used to hold it. */
```

Specialisation

```
/* Provide a different implementation for particular types. */

/* --- FULL SPECIALISATION --- */
template<typename T>
struct Serializer {
        static size_t write(uint8_t *buf, const T& v)
        {
                memcpy(buf, &v, sizeof(T));
                return sizeof(T);
        }
};

template<>
struct Serializer<std::string_view> {
        static size_t write(uint8_t *buf, std::string_view v)
        {
                buf[0] = static_cast<uint8_t>(v.size());
                memcpy(buf + 1, v.data(), v.size());
                return v.size() + 1;
        }
};

/* --- PARTIAL SPECIALISATION - CLASSES ONLY --- */
template<typename T>
struct Serializer<T*> {          /* any pointer */
        static size_t write(uint8_t *buf, T *p)
        {
                return Serializer<T>::write(buf, *p);
        }
};

/* FUNCTIONS CANNOT BE PARTIALLY SPECIALISED. Use overloading,
   or a class template with a static function - which is why
   the standard library's type traits are all classes.    */

/* --- TAG DISPATCH, the pre-concepts way to select an
       implementation --- */
        template<typename It>
        void advance_impl(It& it, int n,
                          std::random_access_iterator_tag)
                { it += n; }

        template<typename It>
        void advance_impl(It& it, int n,
                          std::forward_iterator_tag)
                { while (n--) ++it; }

        template<typename It>
        void advance(It& it, int n)
        {
                advance_impl(it, n,
                    typename std::iterator_traits<It>::
                        iterator_category{});
        }
/* Still worth recognising - the STL is full of it - but
   `if constexpr` and concepts have largely replaced it
   (below).                                               */
```

if constexpr

```
/* C++17's most useful template feature: a branch that is
   DISCARDED at compile time, so the untaken side need not
   even be valid for that type.                              */

template<typename T>
size_t serialise(uint8_t *buf, const T& v)
{
        if constexpr (std::is_trivially_copyable_v<T>) {
                memcpy(buf, &v, sizeof(T));
                return sizeof(T);
        } else if constexpr (has_serialise_method_v<T>) {
                return v.serialise(buf);
        } else {
                static_assert(always_false_v<T>,
                              "type is not serialisable");
        }
}

/* WITH A PLAIN `if`, every branch must COMPILE for every T,
   even the ones that never run. With `if constexpr` only the
   taken branch is instantiated - which is what makes it
   usable for dispatching on type properties.

/* AND IN EMBEDDED CONFIGURATION CODE: */
template<bool UseDma>
class Transfer {
public:
        void run(const uint8_t *data, size_t n)
        {
                if constexpr (UseDma) {
                        dma_setup(data, n);
                        dma_start();
                } else {
                        for (size_t i = 0; i < n; i++)
                                poll_write(data[i]);
                }
        }
};
/* Transfer<false> contains NO dma code at all - it is not
   compiled, so it does not exist in the binary. That is how
   you build a configurable driver with no runtime cost and
   no #ifdef.                                                */
```

`IF CONSTEXPR` REPLACES MOST `#IFDEF`

An `#ifdef` hides code from the compiler entirely, so the disabled path is never type-checked and rots silently. `if constexpr` on a `constexpr bool` is fully parsed and type-checked in every configuration, discarded only at instantiation. Same generated code, far better maintenance — and it is one of the strongest arguments for C++ over C in configurable firmware.

Concepts

```
/* C++20. Constrain a template, and get an error message that
   names the problem rather than a hundred lines of
   instantiation backtrace.                                  */

template<typename T>
concept Serialisable = requires(const T& t, uint8_t *buf) {
        { t.serialise(buf) } -> std::convertible_to<size_t>;
        { T::size } -> std::convertible_to<size_t>;
};

template<Serialisable T>
void send(const T& msg) { ... }

        send(42);
/* ERROR: constraints not satisfied
     because '42' (of type int) does not satisfy
     'Serialisable': the required expression
     't.serialise(buf)' is invalid

   Compare the pre-concepts version, which reported a failure
   deep inside send()'s body across eight levels of
   instantiation.

/* THE STANDARD CONCEPTS worth knowing: */
        std::integral, std::floating_point
        std::same_as, std::convertible_to, std::derived_from
        std::copyable, std::movable, std::default_initializable
        std::invocable, std::predicate
        std::ranges::range, std::ranges::sized_range

/* THREE WAYS TO WRITE A CONSTRAINT, all equivalent: */
        template<std::integral T> void f(T);
        template<typename T> requires std::integral<T> void f(T);
        void f(std::integral auto x);           /* terse */

/* AND CONCEPTS PARTICIPATE IN OVERLOAD RESOLUTION, with the
   MORE CONSTRAINED overload preferred: */
        template<std::integral T>       void g(T);   /* 1 */
        template<typename T>            void g(T);   /* 2 */
        g(42);                  /* 1 - more constrained */
/* Which replaces the enable_if SFINAE machinery entirely, and
   is the main reason to move to C++20 where the toolchain
   allows.                                                   */

/* THE PRE-C++20 EQUIVALENT, which you will still meet: */
        template<typename T,
                 typename = std::enable_if_t<
                        std::is_integral_v<T>>>
        void f(T x);
/* "Substitution Failure Is Not An Error" - if the enable_if
   fails to substitute, the candidate is silently removed
   rather than causing a compile error. Powerful, and the
   source of C++'s worst error messages.                  */
```

Variadic templates

```
/* Any number of arguments, type-safe, with no varargs. */

template<typename... Args>
void log(const char *fmt, Args&&... args)
{
        format_to(buffer, fmt, std::forward<Args>(args)...);
}

/* FOLD EXPRESSIONS (C++17) - the readable way to process a
   pack: */
        template<typename... Args>
        auto sum(Args... args) { return (args + ...); }
                                       /* unary right fold */

        template<typename... Args>
        bool all_valid(Args... args) { return (args.valid() && ...); }

        template<typename... Args>
        void print_all(Args&&... args)
        {
                ((std::cout << args << ' '), ...);
                                       /* comma fold */
        }

        sizeof...(Args)         /* the number of arguments */

/* A PRACTICAL EMBEDDED USE - a compile-time register field
   builder: */
        template<typename... Fields>
        constexpr uint32_t make_reg(Fields... f)
        {
                return (f.value | ...);
        }
        constexpr auto ctrl = make_reg(Enable{1}, Mode{2},
                                       Prescale{7});
/* Computed entirely at compile time; the result is one
   immediate in the instruction stream.                    */

/* PRE-C++17 RECURSION, which you will still see: */
        void print() {}                         /* base case */
        template<typename T, typename... Rest>
        void print(T first, Rest... rest)
        {
                std::cout << first;
                print(rest...);                 /* recurse */
        }
/* Fold expressions are shorter and compile faster. Use them
   where available.                                        */
```

The costs

```
/* Templates are not free in every dimension. */

/* --- CODE BLOAT --- */
        RingBuffer<uint8_t, 16>
        RingBuffer<uint8_t, 32>
        RingBuffer<uint16_t, 16>
/* THREE COMPLETE COPIES of every member function. On a
   flash-constrained part this adds up quickly.

   MITIGATIONS:
     · factor the type-independent parts into a non-template
       base:                                                */
        class RingBufferBase {          /* no template */
        protected:
                size_t head_, tail_, cap_;
                bool advance(size_t& idx);   /* shared code */
        };
        template<typename T, size_t N>
        class RingBuffer : RingBufferBase { ... };

     /* · use fewer distinct instantiations - prefer one
          capacity where you can
        · -Wl,--icf=all folds byte-identical functions
          (compilers ch. 12)
        · MEASURE: nm --size-sort -S, or bloaty            */

/* --- COMPILE TIME --- */
/* Templates are instantiated in every translation unit that
   uses them, then deduplicated by the linker. Heavy template
   use is the main cause of slow C++ builds.

   MITIGATIONS:
     extern template class RingBuffer<uint8_t, 64>;
     /* in a header - suppress instantiation here */
     template class RingBuffer<uint8_t, 64>;
     /* in ONE .cpp - instantiate it once */
     · -ftime-trace (clang) + speedscope to see where the time
       goes
     · precompiled headers, or C++20 modules if available    */

/* --- ERROR MESSAGES --- */
/* Without concepts, an error inside a template reports at the
   point of failure, not the point of misuse - hundreds of
   lines deep. Use concepts, or static_assert early: */
        template<typename T>
        void f(T x) {
                static_assert(std::is_integral_v<T>,
                              "f requires an integral type");
                ...
        }
/* One clear message instead of a wall.                    */

/* --- DEBUGGING --- */
/* Template names in a debugger and in a backtrace are
   enormous. c++filt and `set print asm-demangle` help;
   type aliases help more:                                 */
        using RxBuffer = RingBuffer<uint8_t, 64>;
```

CRTP: static polymorphism

```
/* Polymorphism with no vtable and no indirect call - the
   embedded alternative to virtual functions (chapter 11). */

template<typename Derived>
class SensorBase {
public:
        int read() {
                auto raw = self().read_raw();   /* dispatched
                                                   STATICALLY */
                return self().convert(raw);
        }

        void init() {
                self().configure();
                self().calibrate();
        }

private:
        Derived& self() { return static_cast<Derived&>(*this); }
};

class Thermistor : public SensorBase<Thermistor> {
public:
        uint16_t read_raw()          { return adc_read(CH0); }
        int      convert(uint16_t r) { return lookup(r); }
        void     configure()         { adc_init(CH0); }
        void     calibrate()         { }
};

        Thermistor t;
        int temp = t.read();

/* THE GENERATED CODE is fully inlined - read_raw and convert
   are called directly and usually vanish into the caller.
   NO vtable pointer in the object, NO indirect call, and the
   whole chain is optimisable.

/* THE COST: no runtime polymorphism. You cannot have a
   `std::vector<SensorBase*>` of mixed sensor types, because
   SensorBase<Thermistor> and SensorBase<Photodiode> are
   unrelated types.

/* WHEN TO USE WHICH:
     the set of types is KNOWN AT COMPILE TIME  -> CRTP
     the set varies at run time, or you need a
     heterogeneous container                    -> virtual
     you need both                              -> std::variant
                                                   (chapter 15)
*/
```

How it shows up

Symptom	Cause
200-line error inside the standard library	A constraint violated deep in instantiation — add concepts or a `static_assert`
"no matching function" with an obvious call	Deduction failed — often a mismatch like `max(1, 2.0)`

Symptom	Cause
Binary much larger than expected	Many instantiations — check with `nm --size-sort`
Build times growing rapidly	Template instantiation — try `extern template`, `-ftime-trace`
Template code not type-checked in some configs	You used `#ifdef` instead of `if constexpr`
Cannot partially specialise a function	Only classes can be — use overloading or a class template
An overload is never selected	A less-constrained template is winning — constrain it
SFINAE silently removes what you wanted	Substitution failure — concepts report it properly
Debugger shows unreadable type names	Introduce type aliases
CRTP object cannot go in a container with siblings	By design — different base types

Check yourself

- ✓ Write a function template and a class template with a non-type parameter.
- ✓ Explain why a template capacity parameter suits embedded containers.
- ✓ Distinguish full and partial specialisation, and say what functions cannot do.
- ✓ Explain `if constexpr` and why it beats `#ifdef`.
- ✓ Write a concept and show the error message it produces.
- ✓ Explain how concepts affect overload resolution.
- ✓ Use a fold expression to process a parameter pack.
- ✓ Name three costs of templates and a mitigation for each.
- ✓ Implement CRTP and say what it gives up compared with virtual functions.
- ✓ Measure the code size of several instantiations of one template.

`constexpr` and compile-time computation

Moving work from run time to compile time. On embedded this is not a micro-optimisation — a lookup table computed by the compiler costs zero cycles and lives in flash, and a configuration validated at compile time cannot be wrong in the field.

The keywords

```
/* --- const: "I will not modify this" ------------------- */
        const int a = 42;               /* compile-time
                                           constant here    */
        const int b = read_sensor();    /* runtime value that
                                           does not change  */

/* --- constexpr: "usable in a constant expression" ------ */
        constexpr int c = 42;
        constexpr int d = read_sensor();        /* ERROR */

        constexpr int square(int x) { return x * x; }
        constexpr int e = square(5);    /* computed at COMPILE
                                           time              */
        int n = get_n();
        int f = square(n);              /* the SAME function,
                                           called at RUN time */

/* A constexpr FUNCTION may be evaluated at compile time; it
   is not required to be. Called with runtime arguments it is
   an ordinary function.

/* --- consteval (C++20): "MUST be compile time" --------- */
        consteval int must_be_static(int x) { return x * x; }
        constexpr int g = must_be_static(5);    /* OK */
        int h = must_be_static(n);              /* ERROR */

/* Use it for anything whose whole point is compile-time
   evaluation - a hash of a string literal, a validated
   configuration, a register encoding.

/* --- constinit (C++20): "initialised at compile time" -- */
        constinit int counter = 0;      /* guaranteed static
                                           initialisation - no
                                           constructor runs
                                           before main
                                           (chapter 01)     */
/* It does NOT make the variable const; it only guarantees
   the initialisation is not dynamic. Exactly what you want
   for a mutable global on a bare-metal target.           */
```

Compile-time tables

```
/* THE HIGHEST-VALUE USE ON EMBEDDED. A table computed by the
   compiler costs nothing at run time and lives in flash.  */

/* --- CRC-32 table --- */
constexpr std::array<uint32_t, 256> make_crc_table()
{
        std::array<uint32_t, 256> table{};

        for (uint32_t i = 0; i < 256; i++) {
                uint32_t c = i;
                for (int k = 0; k < 8; k++)
                        c = (c & 1) ? (0xEDB88320u ^ (c >> 1))
                                    : (c >> 1);
                table[i] = c;
        }
        return table;
}

inline constexpr auto crc_table = make_crc_table();

uint32_t crc32(std::span<const uint8_t> data)
{
        uint32_t c = 0xFFFFFFFFu;

        for (uint8_t b : data)
                c = crc_table[(c ^ b) & 0xFF] ^ (c >> 8);

        return c ^ 0xFFFFFFFFu;
}

/* THE TABLE IS IN .rodata - flash, not RAM - fully computed.
   No initialisation code, no startup cost, and the source
   documents how it was derived rather than being 256
   inscrutable hex constants.

/* --- a sine table --- */
constexpr std::array<int16_t, 256> make_sine()
{
        std::array<int16_t, 256> t{};
        for (size_t i = 0; i < 256; i++) {
                /* a constexpr-friendly sine - std::sin is not
                   constexpr until C++26, so use a Taylor
                   series or CORDIC */
                t[i] = static_cast<int16_t>(
                        32767.0 * const_sin(2.0 * PI * i / 256));
        }
        return t;
}

/* --- a gamma correction table, from a formula --- */
template<size_t N, int GammaTimes100>
constexpr std::array<uint8_t, N> make_gamma()
{
        std::array<uint8_t, N> t{};
        for (size_t i = 0; i < N; i++)
                t[i] = static_cast<uint8_t>(
                        255.0 * const_pow(i / double(N - 1),
                                          GammaTimes100 / 100.0));
        return t;
}
```

```
        constexpr auto gamma22 = make_gamma<256, 220>();

/* CHANGE THE PARAMETER, GET A NEW TABLE. No script, no
   generated file to keep in sync, no build step. That is the
   argument.

/* CHECK IT LANDED IN FLASH: */
        nm -S prog.elf | grep crc_table
        readelf -S prog.elf         /* which section? */
/* If it is in .data rather than .rodata, something is not
   const and it is costing you RAM.                       */
```

THE EMBEDDED CASE, IN ONE SENTENCE

A `constexpr` table trades flash for RAM and cycles, is generated from readable source rather than a checked-in blob, and cannot drift out of sync with the formula that produced it. On a part with 32 KB of RAM and 512 KB of flash, that is nearly always the right trade.

What is allowed

```
/* The rules have relaxed substantially with each standard. */

/* C++11: essentially one return statement.
   C++14: loops, local variables, multiple statements, if.
   C++17: if constexpr, lambdas.
   C++20: try/catch (may not throw), dynamic allocation IF it
          is freed within the same evaluation, virtual calls,
          std::vector and std::string.
   C++23: goto, static/thread_local, more of the library.  */

/* WHAT IS STILL FORBIDDEN in a constant evaluation:
     · undefined behaviour of ANY kind - which is a feature:
       UB in a constexpr context is a COMPILE ERROR
     · reading a non-constexpr variable
     · reinterpret_cast
     · calling a non-constexpr function
     · anything touching hardware
     · allocation that escapes the evaluation

/* THE UB POINT IS WORTH DWELLING ON: */
        constexpr int f(int x) { return x + 1; }
        constexpr int bad = f(INT_MAX);
/* ERROR: overflow in constant expression
   The same expression at run time is silent UB
   (compilers chapter 15). Constant evaluation is the only
   place C++ is guaranteed to catch it.

/* DETECTING WHICH CONTEXT YOU ARE IN (C++20): */
        constexpr int fast_sqrt(int x)
        {
                if (std::is_constant_evaluated())
                        return exact_slow_sqrt(x);   /* compile
                                                        time  */
                else
                        return hardware_sqrt(x);     /* run
                                                        time  */
        }
/* C++23 spells the same thing `if consteval`.

/* AND static_assert, which is the point of much of this: */
        static_assert(crc32("hello") == 0x3610A686);
        static_assert(sizeof(Message) == 16,
                      "wire format changed");
        static_assert(TIMER_HZ % SAMPLE_RATE == 0,
                      "sample rate must divide the timer clock");
/* A configuration error that fails the BUILD is a
   configuration error that never ships.                  */
```

Compile-time configuration

```
/* Validate an entire board configuration at compile time. */

struct ClockConfig {
        uint32_t xtal_hz;
        uint32_t pll_mult;
        uint32_t pll_div;
        uint32_t ahb_div;

        constexpr uint32_t pll_hz() const
                { return xtal_hz * pll_mult / pll_div; }
        constexpr uint32_t ahb_hz() const
                { return pll_hz() / ahb_div; }

        constexpr bool valid() const
        {
                return xtal_hz >= 4'000'000
                    && xtal_hz <= 26'000'000
                    && pll_mult >= 2 && pll_mult <= 64
                    && pll_hz() <= 480'000'000
                    && ahb_hz() <= 200'000'000;
        }
};

inline constexpr ClockConfig board_clock{
        .xtal_hz  = 25'000'000,
        .pll_mult = 32,
        .pll_div  = 2,
        .ahb_div  = 2,
};
static_assert(board_clock.valid(), "invalid clock configuration");
static_assert(board_clock.ahb_hz() == 200'000'000);

/* AND DERIVED VALUES computed once, at compile time: */
        constexpr uint32_t uart_divisor(uint32_t clk,
                                        uint32_t baud)
        {
                return (clk + baud / 2) / (16 * baud);
        }
        constexpr auto UART_DIV =
                uart_divisor(board_clock.ahb_hz(), 115200);
        static_assert(UART_DIV > 0 && UART_DIV < 65536);

/* WHAT THIS REPLACES: a spreadsheet, a comment saying "if you
   change the crystal, recompute the divisors", and the bug
   where someone did not.

/* A COMPILE-TIME STRING HASH, for command dispatch with no
   strcmp at run time: */
        consteval uint32_t fnv1a(std::string_view s)
        {
                uint32_t h = 2166136261u;
                for (char c : s) {
                        h ^= static_cast<uint8_t>(c);
                        h *= 16777619u;
                }
                return h;
        }

        switch (fnv1a_runtime(input)) {
        case fnv1a("start"): do_start(); break;
```

```
        case fnv1a("stop"):  do_stop();  break;
        }
/* The case labels are integer constants; the switch is a
   jump table. And `consteval` guarantees the hashes are not
   computed at run time.                                    */
```

Type-safe units

```
/* Strong types with constexpr arithmetic: correctness with
   literally zero runtime cost.                             */

class Milliseconds {
        uint32_t v_;
public:
        explicit constexpr Milliseconds(uint32_t v) : v_(v) {}
        constexpr uint32_t count() const { return v_; }

        constexpr Milliseconds operator+(Milliseconds o) const
                { return Milliseconds{v_ + o.v_}; }
        constexpr auto operator<=>(const Milliseconds&) const
                = default;                       /* C++20 */
};

constexpr Milliseconds operator""_ms(unsigned long long v)
{
        return Milliseconds{static_cast<uint32_t>(v)};
}

        void delay(Milliseconds t);

        delay(100_ms);          /* clear */
        delay(100);             /* ERROR - explicit ctor */
        delay(1_s);             /* if you define _s too */

/* THE GENERATED CODE is identical to passing a bare uint32_t.
   The class is entirely compiled away - check on godbolt
   (chapter 14).

/* AND THE STANDARD LIBRARY ALREADY HAS THIS: */
        #include <chrono>
        using namespace std::chrono_literals;

        void delay(std::chrono::milliseconds t);
        delay(100ms);
        delay(2s);              /* converted automatically */
        delay(1500us);          /* and this */

/* std::chrono::duration is constexpr throughout, converts
   between units correctly at compile time, and refuses
   lossy conversions unless you ask with duration_cast. On
   embedded it is worth using even if you use nothing else
   from the standard library - it prevents an entire class of
   unit-confusion bug for free.                             */
```

Verifying it happened

```
/* THE RISK: you wrote constexpr, and it is being evaluated at
   run time anyway. Check.                                    */

/* 1. FORCE IT with a constexpr variable: */
        constexpr auto table = make_table();
/* If make_table() cannot be evaluated at compile time, this
   is a COMPILE ERROR - which is exactly the feedback you
   want. Assigning to `auto table = make_table();` would
   silently run it at startup.

/* 2. USE consteval where it must be compile time. */

/* 3. LOOK AT THE BINARY: */
        nm -S prog.elf | grep table
        readelf -x .rodata prog.elf | head
        objdump -s -j .rodata prog.elf
/* The table's bytes should be visible in .rodata. If the
   symbol is in .bss and there is code in .init_array, it is
   being computed at startup.

/* 4. CHECK .init_array IS EMPTY: */
        readelf -x .init_array prog.elf
/* For a well-configured firmware image it should be, or
   nearly so (chapter 01).

/* 5. AND THE COMPILE-TIME COST, which is real: */
        clang -ftime-trace ...
        /* then load the .json in speedscope or
           chrome://tracing */
/* Heavy constexpr evaluation - a large table with a complex
   formula - can add seconds to a build. Usually worth it;
   occasionally not.

/* THE LIMITS THE COMPILER IMPOSES: */
        -fconstexpr-depth=512           /* recursion */
        -fconstexpr-steps=1048576       /* clang: total
                                           operations */
        -fconstexpr-ops-limit=...       /* gcc */
/* Hitting these produces "constexpr evaluation exceeded
   maximum steps" - raise the limit, or simplify.        */
```

How it shows up

Symptom	Cause
Table is in RAM, not flash	Not `constexpr`, or not `const` — check the section
Startup code initialises a table	Computed at run time — force it with a `constexpr` variable
"call to non-constexpr function"	Something in the chain is not marked
"not a constant expression"	Reading a runtime value, or UB in the evaluation
Overflow error only in a `constexpr` context	Correct — the same code is silent UB at run time
"constexpr evaluation exceeded maximum steps"	Raise `-fconstexpr-steps`, or simplify
Build slowed by seconds	Heavy compile-time evaluation — measure with `-ftime-trace`
A configuration error reached the field	It was not `static_assert`ed

Symptom	Cause
Units confused (ms versus µs)	Bare integers — use `chrono` or a strong type
Strong-typed units cost performance	They should not — check the assembly; something is not inlining

Check yourself

- ✓ Distinguish `const`, `constexpr`, `consteval` and `constinit`.
- ✓ Write a `constexpr` function that generates a lookup table.
- ✓ Verify the table is in `.rodata` and that nothing runs at startup.
- ✓ Say what is forbidden in a constant evaluation, and why the UB rule is valuable.
- ✓ Use `static_assert` to validate a board configuration.
- ✓ Compute a derived hardware constant at compile time and assert its range.
- ✓ Write a compile-time string hash and use it in a `switch`.
- ✓ Build a strong unit type and prove it costs nothing.
- ✓ Use `std::chrono` for durations and explain what it prevents.
- ✓ Detect and fix a case where `constexpr` silently became runtime evaluation.

The standard library: containers

What each container costs in memory, allocation and cache behaviour — and which of them belong in embedded code. The right answer is usually `std::array`, and knowing why the others are wrong is more useful than knowing their interfaces.

The containers, and their real costs

Container	Layout	Allocates	Embedded verdict
`std::array<T,N>`	Contiguous, inline	**Never**	**Use freely**
`std::span<T>`	ptr + size, non-owning	Never	**Use freely**
`std::string_view`	ptr + size, non-owning	Never	**Use freely**
`std::vector<T>`	Contiguous, heap	On growth	Only with `reserve`, or not at all
`std::string`	Heap, with SSO	Above ~15 chars	Avoid; use `string_view` or a fixed buffer
`std::deque<T>`	Chunked	Per chunk	Avoid
`std::list<T>`	Nodes	**Per element**	**Avoid** — terrible cache behaviour
`std::map<K,V>`	Red-black tree	Per element	Avoid; sorted array + `binary_search`
`std::unordered_map`	Buckets + nodes	Per element	Avoid; a fixed hash table
`std::bitset<N>`	Inline bits	Never	Use freely
`std::optional<T>`	T + a bool, inline	Never	**Use freely**
`std::variant<...>`	Largest + a tag, inline	Never	**Use freely**
`std::pair`, `std::tuple`	Inline	Never	Use freely

THE PATTERN

Everything that stores its data *inline* is free and belongs in embedded code. Everything that owns heap memory is a decision. That single distinction covers the whole table, and it is why `std::array`, `span`, `optional` and `variant` are the four standard-library types worth adopting even in tightly constrained firmware.

std::array

```
#include <array>

        std::array<uint8_t, 64> buf{};          /* zeroed */
        std::array<int, 3> a{1, 2, 3};

        buf.size();             /* a compile-time constant */
        buf.data();             /* uint8_t* */
        buf[0];                 /* unchecked - same as C */
        buf.at(0);              /* THROWS if out of range -
                                   unusable with
                                   -fno-exceptions          */
        buf.front(); buf.back();
        for (auto& b : buf) { }
        std::fill(buf.begin(), buf.end(), 0);

/* IDENTICAL LAYOUT AND CODE to `uint8_t buf[64]`, plus:
     · it knows its size, so no separate length parameter
     · it does not decay to a pointer when passed
     · it can be returned by value, copied and compared
     · begin()/end() work with the algorithms

/* THE PASSING RULE: */
        void f(std::array<uint8_t, 64>& a);    /* one size
                                                  only     */
        void g(std::span<uint8_t> s);          /* ANY size -
                                                  prefer this */

/* CLASS TEMPLATE ARGUMENT DEDUCTION (C++17): */
        std::array a{1, 2, 3};          /* std::array<int,3> */

/* AND THE 2-D CASE: */
        std::array<std::array<float, 4>, 4> matrix{};
        matrix[1][2] = 3.0f;
/* Contiguous, no indirection - unlike float**.           */
```

std::span and string_view

```
#include <span>          /* C++20 */

/* A NON-OWNING view of contiguous memory: a pointer and a
   size. Replaces every (T *ptr, size_t len) pair.        */

        void process(std::span<const uint8_t> data)
        {
                for (auto b : data) { ... }
                if (data.size() < 4) return;
                auto header = data.first(4);
                auto body   = data.subspan(4);
        }

        std::array<uint8_t, 64> a;
        std::vector<uint8_t>    v;
        uint8_t                 c[32];

        process(a);             /* all three convert */
        process(v);             /* implicitly */
        process({c, 32});

/* WHY THIS MATTERS: it removes the commonest C bug shape -
   a pointer and a length passed separately, and one of them
   updated without the other.

/* FIXED-EXTENT SPANS carry the size in the type: */
        void f(std::span<uint8_t, 16> exactly16);
/* size() is then a compile-time constant, and passing the
   wrong size is a compile error.

/* string_view is the same idea for text: */
        void log(std::string_view msg);
        log("literal");                 /* no allocation, no
                                           strlen at run time -
                                           the size is known
                                           from the literal  */
        log(std::string{...});          /* also works */

/* THE LIFETIME HAZARD, and it is real: */
        std::string_view bad()
        {
                std::string s = build();
                return s;               /* DANGLING - s is
                                           destroyed         */
        }
        std::string_view sv = get_string();  /* if that returns
                                                by value,
                                                DANGLING     */
/* A view does not own. -Wdangling-gsl catches some cases;
   ASan catches the rest. Treat span and string_view as
   PARAMETERS, not as members or return values, unless you
   can see the owner's lifetime.                          */
```

std::vector, when you must

```
        std::vector<int> v;
        v.reserve(100);         /* ONE allocation, up front */
        for (int i = 0; i < 100; i++)
                v.push_back(i); /* no reallocation */

/* GROWTH: capacity doubles (or ×1.5), which means:
     · amortised O(1) push_back
     · reallocation COPIES OR MOVES every element
     · ALL ITERATORS AND POINTERS ARE INVALIDATED
     · unpredictable timing - the reallocation is a latency
       spike exactly where you cannot afford one

        v.size()        elements
        v.capacity()    allocated space
        v.shrink_to_fit()   a non-binding REQUEST
        v.clear()       size 0, capacity UNCHANGED

/* THE ITERATOR INVALIDATION TRAP: */
        for (auto it = v.begin(); it != v.end(); ++it)
                if (*it == x)
                        v.erase(it);    /* it is now INVALID */
        /* correct: */
                it = v.erase(it);       /* erase returns the
                                           next valid one    */
        /* or, better: */
        std::erase(v, x);               /* C++20 */

/* WITH A CUSTOM ALLOCATOR, vector can use a fixed pool: */
        std::pmr::monotonic_buffer_resource pool{buffer,
                                                 sizeof buffer};
        std::pmr::vector<int> v{&pool};
/* Bounded, deterministic, no global heap. Worth knowing, and
   in practice a fixed-capacity container is simpler.

/* THE EMBEDDED ALTERNATIVE - a fixed-capacity vector: */
        etl::vector<Message, 32> queue;   /* ETL */
        /* or roll your own: */
template<typename T, size_t N>
class StaticVector {
        alignas(T) std::byte storage_[N * sizeof(T)];
        size_t size_ = 0;
public:
        bool push_back(const T& v)
        {
                if (size_ >= N) return false;   /* FAILS
                                                   rather than
                                                   allocating */
                new (&data()[size_]) T(v);
                size_++;
                return true;
        }
        T *data() { return reinterpret_cast<T *>(storage_); }
        size_t size() const { return size_; }
        ...
};
/* The interface of vector, the memory behaviour of an array,
   and a bounded failure mode instead of an allocation.  */
```

Maps, and what to use instead

```
/* std::map is a red-black tree: one heap node per element,
   pointers everywhere, and a cache miss per level of the
   tree. std::unordered_map is buckets of nodes - the same
   problem plus a hash.

/* FOR A SMALL, FIXED SET, a sorted array beats both,
   dramatically: */

struct Entry { uint32_t key; Handler fn; };

inline constexpr std::array<Entry, 5> table{{
        {0x100, handle_status},
        {0x200, handle_config},
        {0x300, handle_data},
        {0x400, handle_reset},
        {0x500, handle_debug},
}};
static_assert(std::is_sorted(table.begin(), table.end(),
                [](auto& a, auto& b){ return a.key < b.key; }),
              "table must be sorted");

const Entry *lookup(uint32_t key)
{
        auto it = std::lower_bound(table.begin(), table.end(),
                        key,
                        [](const Entry& e, uint32_t k)
                                { return e.key < k; });
        return (it != table.end() && it->key == key)
                ? &*it : nullptr;
}

/* IN FLASH, no allocation, one cache line for the whole
   table, and the static_assert guarantees the sort order the
   binary search depends on.

/* FOR UP TO ABOUT 16 ENTRIES, a LINEAR SCAN is usually
   FASTER than a binary search - fewer branches, perfectly
   predictable, one cache line. Measure rather than assuming
   the asymptotically better algorithm wins at small N; on
   embedded it very often does not (CA chapter 14).

/* A FIXED HASH TABLE, when you need O(1) and have the space:
*/
template<typename K, typename V, size_t N>
class FlatMap {
        std::array<K, N> keys_{};
        std::array<V, N> vals_{};
        std::array<bool, N> used_{};
        /* open addressing, linear probing - no nodes, no
           allocation, contiguous                          */
};

/* AND FOR COMPILE-TIME LOOKUP, a perfect hash or simply a
   switch on a constexpr string hash (chapter 07) beats every
   container.                                              */
```

optional and variant

```
#include <optional>

/* "a T, or nothing" - without a sentinel value or an
   out-parameter.                                          */

        std::optional<uint32_t> read_sensor()
        {
                if (!ready()) return std::nullopt;
                return raw_value();
        }

        if (auto v = read_sensor()) {
                use(*v);
        }
        uint32_t x = read_sensor().value_or(0);

/* LAYOUT: sizeof(T) + one bool + padding. INLINE, no
   allocation. Replaces:
     · a magic sentinel (0xFFFFFFFF meaning "no reading")
     · a bool out-parameter
     · a pointer that might be null
   all of which are error-prone in ways optional is not.

#include <variant>

/* "exactly one of these types" - a tagged union with the
   tagging done correctly.                                 */

        using Event = std::variant<ButtonPress, SensorReading,
                                   Timeout>;

        void handle(const Event& e)
        {
                std::visit([](const auto& ev) {
                        using T = std::decay_t<decltype(ev)>;
                        if constexpr (std::is_same_v<T,
                                                     ButtonPress>)
                                on_button(ev);
                        else if constexpr (std::is_same_v<T,
                                                     SensorReading>)
                                on_sensor(ev);
                        else
                                on_timeout(ev);
                }, e);
        }

/* LAYOUT: as large as the largest alternative, plus a small
   index. Inline, no allocation.

   VERSUS A RAW UNION: the variant knows which member is
   active, destroys the right one, and std::visit is
   EXHAUSTIVE - add an alternative and the code that does not
   handle it fails to compile. That last property is the
   whole value.

/* THE EXCEPTIONS CAVEAT: std::get and std::visit can throw
   std::bad_variant_access. With -fno-exceptions, use
   std::get_if, which returns a pointer or null:          */
        if (auto *p = std::get_if<SensorReading>(&e))
                on_sensor(*p);
```

```
/* AND std::expected (C++23) - a value or an error, which is
   what you actually want for a fallible operation
   (chapter 12):                                          */
        std::expected<Reading, Error> read();
        if (auto r = read())  use(*r);
        else                  log(r.error());
```

Measuring the cost

```
/* Do not guess. Measure the three things that matter. */

/* --- 1. SIZE, at compile time --- */
        static_assert(sizeof(std::array<uint8_t, 64>) == 64);
        static_assert(sizeof(std::span<int>) == 16);
        static_assert(sizeof(std::optional<uint32_t>) == 8);
        static_assert(sizeof(std::string_view) == 16);
        /* and on your target: */
        static_assert(sizeof(std::vector<int>) == 24);
        static_assert(sizeof(std::string) == 32);

/* --- 2. ALLOCATIONS, at run time --- */
        static size_t g_allocs = 0;
        void *operator new(size_t n) {
                g_allocs++;
                return malloc(n);
        }
        void operator delete(void *p) noexcept { free(p); }
/* Then assert the count is zero over your hot path. On bare
   metal, simply do not link an operator new - any
   accidental allocation becomes a LINK ERROR, which is the
   strongest guarantee available.

/* --- 3. CODE SIZE --- */
        nm --size-sort -S prog.elf | tail -20
        bloaty prog.elf -d symbols
/* Watch for std::__cxx11::basic_string and the iostream
   machinery appearing - each is tens of kilobytes.

/* --- AND THE STRUCTURAL CHECK --- */
        grep -rn 'std::string\|std::vector\|std::map\|iostream' src/
/* In a codebase with a documented subset (chapter 01), this
   grep should return nothing outside the places you decided
   to allow it. Automate it in CI.                        */
```

How it shows up

Symptom	Cause
Unexpected heap allocation	`vector` growth, `string` above SSO, or a node container
Latency spike in a loop	`vector` reallocation — `reserve` up front
Crash after inserting into a container	Iterator or pointer invalidated by reallocation
Crash after erasing during iteration	Use the iterator `erase` returns, or `std::erase`
Garbage from a `string_view`	Dangling — the owner was destroyed
Poor performance despite a good algorithm	Node containers — cache misses dominate (CA ch. 14)

Symptom	Cause
Binary grew 40 KB	`std::string` or iostreams pulled in
`.at()` causes a link error	It throws; incompatible with `-fno-exceptions`
`std::get` on a variant throws	Use `std::get_if` with exceptions disabled
Binary search returns wrong results	The table is not sorted — assert it
Linear scan beats your binary search	Small N — expected; measure rather than assume

Check yourself

- ✓ Classify every standard container by whether it allocates, and state the pattern.
- ✓ Explain what `std::array` adds over a C array at zero cost.
- ✓ Use `span` to replace a pointer-and-length pair, and state its lifetime hazard.
- ✓ Explain `vector` growth, its three consequences, and how to avoid them.
- ✓ Write a fixed-capacity vector with a bounded failure mode.
- ✓ Replace a `std::map` with a sorted array plus a sortedness assertion.
- ✓ Say when a linear scan beats a binary search, and why.
- ✓ Use `optional` to replace a sentinel, and `variant` to replace a union.
- ✓ Explain why `std::visit` exhaustiveness is valuable.
- ✓ Verify container sizes with `static_assert` and prove your hot path allocates nothing.

Iterators, algorithms and ranges

The standard algorithms are templates that compile to the same loop you would write by hand — but named, correct, and hard to get wrong. Almost all of them allocate nothing, which makes this the part of the standard library that transfers straight into embedded work.

Iterators

```
An iterator generalises a pointer. The categories form a
hierarchy, and each algorithm requires a minimum category:

   input          read once, advance forward
      |           (an input stream)
   forward        read repeatedly, advance forward
      |           (forward_list, unordered_map)
   bidirectional  ...and backwards
      |           (list, map, set)
   random access  ...and jump: it += n, it2 - it1, it[n]
      |           (array, vector, deque, RAW POINTERS)
   contiguous     ...and the elements are adjacent in memory
                  (array, vector, span, pointers)  [C++20]

  A RAW POINTER IS A CONTIGUOUS ITERATOR. Every algorithm
  works on plain arrays:

        int a[10];
        std::sort(a, a + 10);
        std::sort(std::begin(a), std::end(a));

  WHY THE CATEGORY MATTERS: std::sort needs random access, so
  it cannot sort a std::list - which is why list has its own
  sort() member. std::advance is O(1) on random access and
  O(n) otherwise, which is why a loop calling it on a list is
  accidentally quadratic.

  THE RANGE CONVENTION is HALF-OPEN: [begin, end). end() is
  one past the last element and must not be dereferenced.
  This makes size = end - begin, an empty range is
  begin == end, and loops terminate naturally.
```

The algorithms worth knowing

```
#include <algorithm>
#include <numeric>

/* --- SEARCHING --- */
        std::find(b, e, value);
        std::find_if(b, e, pred);
        std::count(b, e, value);
        std::count_if(b, e, pred);
        std::all_of(b, e, pred);  any_of  none_of
        std::binary_search(b, e, value);        /* SORTED */
        std::lower_bound(b, e, value);          /* SORTED */
        std::upper_bound(b, e, value);
        std::equal_range(b, e, value);
        std::search(b, e, sb, se);              /* subsequence */
        std::min_element(b, e);  max_element  minmax_element

/* --- MODIFYING --- */
        std::copy(b, e, out);
        std::copy_if(b, e, out, pred);
        std::copy_n(b, n, out);
        std::move(b, e, out);           /* moves elements */
        std::fill(b, e, value);
        std::iota(b, e, 0);             /* 0,1,2,3,... */
        std::generate(b, e, fn);
        std::transform(b, e, out, fn);
        std::transform(b1, e1, b2, out, binary_fn);
        std::replace(b, e, old, new);
        std::reverse(b, e);
        std::rotate(b, mid, e);
        std::swap_ranges(b1, e1, b2);

/* --- REMOVING: the erase-remove idiom --- */
        v.erase(std::remove(v.begin(), v.end(), value),
                v.end());
/* std::remove does NOT remove - it SHIFTS the kept elements
   forward and returns the new logical end. The container
   still holds the old size until erase() is called. This
   surprises everyone once.
   C++20 has the sane spelling:                          */
        std::erase(v, value);
        std::erase_if(v, pred);

/* --- SORTING --- */
        std::sort(b, e);                /* introsort, O(n log n),
                                           NOT stable      */
        std::stable_sort(b, e);         /* may ALLOCATE */
        std::partial_sort(b, mid, e);   /* top-k only */
        std::nth_element(b, nth, e);    /* O(n) selection */
        std::is_sorted(b, e);
        std::partition(b, e, pred);
        std::stable_partition(b, e, pred);   /* may allocate */

/* --- NUMERIC --- */
        std::accumulate(b, e, init);
        std::accumulate(b, e, init, binary_op);
        std::reduce(b, e);              /* C++17, may be
                                           parallel/reordered */
        std::inner_product(b1, e1, b2, init);
        std::partial_sum(b, e, out);
        std::adjacent_difference(b, e, out);
```

```
        std::clamp(v, lo, hi);          /* <algorithm> */
        std::gcd(a, b);  std::lcm(a, b);  /* <numeric> */

/* --- COMPARING --- */
        std::equal(b1, e1, b2);
        std::lexicographical_compare(b1, e1, b2, e2);
        std::mismatch(b1, e1, b2);

/* WHICH ONES ALLOCATE - the only three that matter for
   embedded:
        std::stable_sort
        std::stable_partition
        std::inplace_merge
   Every other algorithm in this list allocates nothing.  */
```

WHY USE THEM RATHER THAN A HAND-WRITTEN LOOP

They compile to the same instructions — `std::find` on a `span<int>` is byte-identical to the loop you would write, and often better because the implementation is specialised for contiguous ranges and vectorises. What you gain is a *name* at the call site, no off-by-one, no inverted comparison, and correct behaviour on empty ranges. That last one is where hand-written loops actually fail.

Lambdas

```
        auto is_even = [](int x) { return x % 2 == 0; };

        int threshold = 10;
        auto above = [threshold](int x) { return x > threshold; };
                                        /* capture BY VALUE */
        auto count_it = [&count](int x) { if (x) count++; };
                                        /* capture BY REFERENCE */

/* CAPTURE FORMS: */
        [ ]             nothing
        [x]             x by value
        [&x]            x by reference
        [=]             everything used, by value    (avoid)
        [&]             everything used, by reference (avoid)
        [this]          the enclosing object's members
        [*this]         a COPY of the enclosing object (C++17)
        [x = expr]      init capture: a new variable
        [p = std::move(ptr)]    move into the lambda

/* PREFER EXPLICIT CAPTURES. `[&]` in a lambda that outlives
   the scope is a dangling reference, and the capture list is
   the only place a reader can see what the lambda depends
   on.

/* WHAT A LAMBDA IS: a compiler-generated class with an
   operator(), and one member per capture. */
        auto f = [a, b](int x) { return a * x + b; };
/* becomes, in effect: */
        struct __lambda {
                int a, b;
                int operator()(int x) const { return a*x + b; }
        };
/* A CAPTURELESS lambda is an empty class, converts to a plain
   function pointer, and costs nothing at all:            */
        void (*fp)(int) = [](int x) { handle(x); };
/* which is how you pass one as a C callback (chapter 16).

/* GENERIC LAMBDAS (C++14): */
        auto print = [](const auto& x) { log(x); };
/* `auto` parameters make operator() a template.

/* THE COST QUESTION: a lambda passed to an algorithm as a
   template parameter is INLINED COMPLETELY - zero cost.
   Stored in a std::function it is type-erased: possibly a
   heap allocation and an indirect call. That difference is
   the whole story (chapter 01).                          */

        template<typename F> void each(F f);   /* free */
        void each(std::function<void(int)> f); /* not free */
```

Ranges

```
#include <ranges>       /* C++20 */
#include <algorithm>

/* 1. ALGORITHMS TAKE A RANGE, not two iterators: */
        std::ranges::sort(v);
        std::ranges::find(v, 42);
        std::ranges::copy(v, out);

/* Shorter, and it cannot mismatch begin and end from
   different containers - a real bug the iterator form
   permits.

/* 2. PROJECTIONS - apply a function before comparing: */
        struct Reading { uint32_t time; int16_t value; };
        std::vector<Reading> readings;

        std::ranges::sort(readings, {}, &Reading::time);
                                /* sort by .time, with the
                                   default comparator      */
        auto it = std::ranges::find(readings, 100,
                                    &Reading::value);
        auto m = std::ranges::max_element(readings, {},
                                          &Reading::value);

/* This replaces a lambda per call, and reads better.

/* 3. VIEWS - lazy, composable, non-owning: */
        namespace v = std::views;

        for (int x : v | v::filter([](int i){ return i > 0; })
                       | v::transform([](int i){ return i * 2; })
                       | v::take(10))
                use(x);

/* LAZY means no intermediate container is created - the
   pipeline is fused into one loop, and generates the same
   code as the hand-written version.

        v::iota(0, 10)          /* 0..9 without a container */
        v::reverse
        v::drop(n)  v::take(n)
        v::take_while(pred)  v::drop_while(pred)
        v::split(delim)
        v::join
        v::keys  v::values      /* for pairs */
        v::enumerate            /* C++23: index and value */
        v::chunk(n)  v::slide(n)    /* C++23 */
        v::zip                  /* C++23 */

/* AN EMBEDDED-RELEVANT EXAMPLE: */
        for (auto [i, sample] : v::enumerate(buffer
                                | v::filter(is_valid)
                                | v::take(MAX_SAMPLES)))
                process(i, sample);

/* THE CAVEATS FOR EMBEDDED:
     · ranges support in older toolchains is patchy; check
       before committing to it
     · compile times increase noticeably
     · error messages, without concepts support, are poor
```

```
    · debugging a pipeline is harder than debugging a loop
    · SOME views allocate (ranges::to, split on some
      implementations) — check what you use

  THE VERDICT: ranges are excellent, and worth adopting on
  embedded Linux with a modern toolchain. On a
  microcontroller with a vendor GCC 8, they are not
  available anyway.                                    */
```

Algorithms on embedded

```
/* Almost all of them are directly usable. Some examples that
   earn their place: */

/* --- on a fixed buffer --- */
        std::array<int16_t, 256> samples;

        auto peak = *std::max_element(samples.begin(),
                                      samples.end());
        auto sum  = std::accumulate(samples.begin(),
                                    samples.end(), 0);
        std::transform(samples.begin(), samples.end(),
                       output.begin(),
                       [](int16_t s) { return s * gain >> 8; });

/* --- on a span, so it works with any buffer --- */
        int16_t rms(std::span<const int16_t> s)
        {
                auto sq = std::inner_product(
                        s.begin(), s.end(), s.begin(), 0LL);
                return isqrt(sq / s.size());
        }

/* --- searching a constexpr table (chapter 08) --- */
        auto it = std::lower_bound(table.begin(), table.end(),
                                   key, by_key);

/* --- and the compile-time versions --- */
        static_assert(std::is_sorted(table.begin(),
                                     table.end(), by_key));
/* Most algorithms became constexpr in C++20, so you can run
   them on constexpr data and assert the result.

/* WHAT TO WATCH:

   1. THE THREE ALLOCATING ALGORITHMS - stable_sort,
      stable_partition, inplace_merge. Use sort and partition
      instead unless stability matters.

   2. std::sort IS INTROSORT: quicksort, falling back to
      heapsort, with insertion sort for small ranges. Its
      WORST CASE is O(n log n), so it is real-time-safe in
      complexity - but its constant factor and code size are
      substantial. For fewer than about 32 elements, an
      insertion sort you write yourself is smaller and
      faster.

   3. CODE SIZE: each instantiation of sort for a distinct
      type and comparator is a separate copy of a fairly large
      function. Check with nm --size-sort.

   4. std::reduce and the parallel algorithms (std::execution)
      have no place on a microcontroller - and on embedded
      Linux they need a threading runtime.               */
```

Writing your own

```
/* Follow the same conventions and your code composes with
   the standard library. */

template<typename It, typename T>
constexpr It find_last(It first, It last, const T& value)
{
        It result = last;
        for (; first != last; ++first)
                if (*first == value)
                        result = first;
        return result;
}

/* THE CONVENTIONS:
     · take [first, last), return an iterator
     · return `last` to mean "not found"
     · take the predicate or comparator as a template
       parameter, never std::function
     · mark it constexpr if it can be
     · require the WEAKEST iterator category that works

/* A RANGE-BASED VERSION, and a concept to constrain it: */
template<std::ranges::forward_range R, typename T>
constexpr auto find_last(R&& r, const T& value)
{
        return find_last(std::ranges::begin(r),
                         std::ranges::end(r), value);
}

/* AND A CUSTOM ITERATOR, when you need to present something
   as a range - a hardware FIFO, for example: */
class FifoIterator {
        Fifo *f_;
        bool  at_end_;
public:
        using iterator_category = std::input_iterator_tag;
        using value_type        = uint8_t;
        using difference_type   = std::ptrdiff_t;
        using pointer           = const uint8_t *;
        using reference         = uint8_t;

        uint8_t operator*() const  { return f_->peek(); }
        FifoIterator& operator++() { f_->pop();
                                     at_end_ = f_->empty();
                                     return *this; }
        bool operator==(const FifoIterator& o) const
                { return at_end_ == o.at_end_; }
};
/* Now std::copy, std::find and the rest work on your FIFO,
   with no allocation and no intermediate buffer.        */
```

How it shows up

Symptom	Cause
`std::remove` did not change the size	It never does — pair it with `erase`, or use `std::erase`
`binary_search` gives wrong answers	The range is not sorted by the same comparator

Symptom	Cause
Quadratic performance with a list	`std::advance` or indexing on a non-random-access iterator
"no matching function" for `std::sort`	The iterator is not random access
Unexpected allocation	`stable_sort`, `stable_partition` or `inplace_merge`
Lambda captured a dangling reference	`[&]` on a lambda that outlived the scope
Lambda in `std::function` allocates	Type erasure — pass it as a template parameter instead
Algorithm crashes on an empty range	`max_element` and friends return `end()` — check before dereferencing
Binary grew after using `std::sort`	Introsort is large; for small N write an insertion sort
Ranges pipeline will not compile	Toolchain support, or a view requiring a stronger range
Mismatched `begin` / `end` from two containers	Use the ranges overloads, which make it impossible

Check yourself

- ✓ Name the iterator categories and say which algorithms each supports.
- ✓ Explain the half-open range convention and what it buys.
- ✓ List twenty algorithms and what each does.
- ✓ Explain the erase-remove idiom and why `remove` does not remove.
- ✓ Name the three allocating algorithms.
- ✓ Explain lambda capture forms and why `[&]` is risky.
- ✓ Say what a lambda compiles to, and when it is free versus not.
- ✓ Use ranges with projections and a view pipeline.
- ✓ State the embedded caveats for ranges.
- ✓ Write an algorithm and a custom iterator that follow the standard conventions.

Smart pointers and ownership

Making ownership visible in the type system. `unique_ptr` costs nothing and prevents leaks; `shared_ptr` costs atomics and is usually the wrong answer. On embedded, the most valuable use is not the heap at all — it is wrapping hardware handles.

Expressing ownership

```
A raw pointer says nothing about ownership. These do:

  T*                  a NON-OWNING observer. Someone else
                      frees it. May be null.
  T&                  a non-owning observer that cannot be
                      null and cannot be reseated.
  std::unique_ptr<T>  EXCLUSIVE ownership. Freed when it
                      goes out of scope. Move-only.
  std::shared_ptr<T>  SHARED ownership. Freed when the last
                      owner goes. Atomic refcount.
  std::weak_ptr<T>    a non-owning reference to a shared
                      object, which knows whether it is
                      still alive.
  std::optional<T>    a value that may be absent, stored
                      INLINE - not a pointer at all.
  std::span<T>        a non-owning view of a contiguous
                      range.

THE CONVENTION THIS ENABLES, and it is worth adopting
wholesale:

  · a function taking `unique_ptr<T>` by value TAKES
    OWNERSHIP
  · a function taking `T*` or `T&` BORROWS - the caller
    keeps ownership and must outlive the call
  · a function returning `unique_ptr<T>` TRANSFERS
    ownership
  · a raw pointer stored as a member means "someone else
    owns this, and I depend on their lifetime"

Once a codebase follows this, ownership is READABLE FROM
THE SIGNATURE, which is most of what documentation used to
have to carry.
```

unique_ptr

```
#include <memory>

        auto p = std::make_unique<Widget>(arg1, arg2);
        p->method();
        Widget& w = *p;
        Widget *raw = p.get();          /* observe, do not own */

        auto q = std::move(p);          /* transfer - p is now
                                           null              */
        p.reset();                      /* delete and null */
        Widget *released = p.release(); /* give up ownership;
                                           YOU must delete    */

        std::unique_ptr<Widget[]> arr =
                std::make_unique<Widget[]>(10);   /* uses
                                                     delete[] */

/* SIZE: exactly sizeof(T*). No overhead at all, provided the
   deleter is stateless. Verify: */
        static_assert(sizeof(std::unique_ptr<int>)
                      == sizeof(int *));

/* THE GENERATED CODE is identical to a raw pointer plus a
   manual delete on every exit path - including the ones you
   would have forgotten (chapter 01).

/* MOVE-ONLY BY DESIGN: */
        auto a = std::make_unique<int>(1);
        auto b = a;                     /* COMPILE ERROR */
        auto c = std::move(a);          /* fine */

/* IN A FACTORY, which is its most natural use: */
        std::unique_ptr<Driver> create_driver(DeviceType t)
        {
                switch (t) {
                case SPI: return std::make_unique<SpiDriver>();
                case I2C: return std::make_unique<I2cDriver>();
                }
                return nullptr;
        }
/* The caller owns the result and cannot forget to delete it.
   Note the return type is the BASE class - which requires a
   virtual destructor (chapter 11).                        */
```

Custom deleters — the embedded use

```
/* THE MOST VALUABLE PATTERN IN THIS CHAPTER FOR EMBEDDED
   WORK: unique_ptr does not have to own heap memory. It can
   own ANY resource with a release function.              */

/* --- a file descriptor --- */
        struct FdDeleter {
                using pointer = int;    /* the "pointer" type
                                           is an int!        */
                void operator()(int fd) const
                        { if (fd >= 0) ::close(fd); }
        };
        using UniqueFd = std::unique_ptr<int, FdDeleter>;

        UniqueFd fd{::open("/dev/thing", O_RDWR)};
        if (fd.get() < 0) return -1;
        ioctl(fd.get(), ...);
        /* closed automatically, on every path */

/* THE `using pointer = int;` IS THE TRICK: unique_ptr uses
   Deleter::pointer if it exists, so the "pointer" can be any
   handle type - an int, a HANDLE, a DMA channel number.

/* --- a C library handle --- */
        struct FileCloser {
                void operator()(FILE *f) const { fclose(f); }
        };
        std::unique_ptr<FILE, FileCloser> f{fopen(path, "r")};

        /* or, more briefly, with a function pointer type: */
        std::unique_ptr<FILE, decltype(&fclose)>
                f{fopen(path, "r"), &fclose};
/* NOTE: this second form makes the unique_ptr TWO pointers
   wide, because the deleter has state. Prefer the stateless
   functor.                                                 */

/* --- a hardware resource --- */
        struct DmaRereleaser {
                using pointer = DmaChannel;
                void operator()(DmaChannel c) const
                        { dma_release(c); }
        };
        using UniqueDma = std::unique_ptr<DmaChannel,
                                          DmaReleaser>;

        UniqueDma claim_dma()
        {
                return UniqueDma{dma_claim()};
        }
        /* move-only, released automatically, no heap involved
           anywhere */

/* --- and a placement-new object in a static pool --- */
        struct PoolDeleter {
                Pool *pool;
                void operator()(Widget *w) const
                {
                        w->~Widget();
                        pool->free(w);
                }
        };
```

```
        std::unique_ptr<Widget, PoolDeleter>
                make_pooled(Pool& p)
        {
                void *mem = p.alloc(sizeof(Widget));
                return {new (mem) Widget{}, PoolDeleter{&p}};
        }
/* RAII, ownership transfer, exception safety - with a fixed
   pool and no global heap. This is unique_ptr earning its
   place on a microcontroller.                          */
```

THE POINT FOR EMBEDDED

`unique_ptr` is not a heap tool. It is an *ownership* tool that happens to default to the heap. With a custom deleter it wraps file descriptors, DMA channels, hardware locks, pool allocations and vendor SDK handles — giving move-only semantics, automatic release and a signature that documents who owns what, at exactly zero runtime cost.

shared_ptr, and why to hesitate

```
        auto p = std::make_shared<Widget>(args);
        auto q = p;                     /* refcount 2 */
        p.use_count();

/* WHAT IT COSTS:
     · TWO pointers: the object, and the CONTROL BLOCK
     · the control block holds a strong count, a weak count,
       and a deleter
     · every copy and destruction is an ATOMIC increment or
       decrement - a full memory barrier on ARM
       (ARM chapter 22)
     · make_shared does ONE allocation for both; constructing
       from a raw pointer does TWO

/* WHY IT IS USUALLY THE WRONG ANSWER:

   1. THE ATOMICS ARE NOT FREE. Copying a shared_ptr in a hot
      path is a measurable cost, and on a multi-core system
      the cache line containing the refcount ping-pongs
      between cores (CA chapter 15).

   2. IT HIDES THE OWNERSHIP GRAPH. "Who frees this?" becomes
      unanswerable, and destruction happens at an
      unpredictable point, possibly on an unexpected thread.

   3. CYCLES LEAK. Two objects holding shared_ptr to each
      other never reach zero.

   4. IT IS OFTEN CHOSEN TO AVOID THINKING about lifetimes,
      which is exactly when it goes wrong.

/* WHEN IT IS GENUINELY RIGHT: several independent owners,
   with no clear "last one", and a lifetime that really is
   dynamic - a cache of shared resources, a plugin registry.
   That is rarer than its usage suggests.

/* BREAKING A CYCLE with weak_ptr: */
        struct Node {
                std::shared_ptr<Node> child;
                std::weak_ptr<Node>   parent;   /* NOT shared */
        };

        if (auto p = node->parent.lock()) {  /* promote to
                                                shared, or get
                                                null           */
                use(*p);
        }

/* ON EMBEDDED: shared_ptr is rarely appropriate. If you find
   yourself reaching for it, ask whether the real answer is a
   fixed pool of objects with indices, or a single owner and
   borrowed references.                                    */
```

Using them correctly

```
/* --- 1. ALWAYS make_unique / make_shared --- */
        auto p = std::make_unique<Widget>(args);   /* yes */
        std::unique_ptr<Widget> q{new Widget(args)}; /* avoid */
/* make_shared is also ONE allocation instead of two. And in
   C++14 and earlier, the make_ forms avoided a real exception
   -safety hole in expressions like f(unique_ptr(new A),
   unique_ptr(new B)).

/* --- 2. NEVER two owners from one raw pointer --- */
        Widget *raw = new Widget;
        std::unique_ptr<Widget> a{raw};
        std::unique_ptr<Widget> b{raw};   /* DOUBLE DELETE */

/* --- 3. PASS BY REFERENCE unless transferring ownership --- */
        void use(const Widget& w);              /* borrows */
        void use(Widget *w);                    /* borrows,
                                                   nullable  */
        void take(std::unique_ptr<Widget> w);   /* TAKES */
        void reseat(std::unique_ptr<Widget>& w);/* may replace */

        void bad(const std::unique_ptr<Widget>& w);
/* That last one is a common mistake: it borrows, but forces
   the caller to have a unique_ptr specifically. Take
   `const Widget&` instead - then any owner works.

/* --- 4. DO NOT store a raw pointer to something you do not
       control --- */
        class Bad {
                Widget *w_;             /* who owns this?
                                           will it outlive
                                           me?               */
        };
/* If the answer is "the caller guarantees it", say so in a
   comment and take it as a reference in the constructor. If
   it is not guaranteed, the design is wrong.

/* --- 5. FOR A POLYMORPHIC BASE, virtual destructor --- */
        class Base { public: virtual ~Base() = default; };
        std::unique_ptr<Base> p = std::make_unique<Derived>();
/* Without the virtual destructor, ~Derived never runs.
   (shared_ptr is the exception - it captures the deleter at
   construction, so it destroys correctly even without one.
   Do not rely on that.)                                  */
```

Ownership without a heap

```
/* On a target with no dynamic allocation, ownership still
   matters. The patterns: */

/* --- 1. A STATIC POOL WITH INDICES --- */
template<typename T, size_t N>
class Pool {
        std::array<std::optional<T>, N> slots_{};
public:
        struct Handle {
                uint8_t index = 0xFF;
                bool valid() const { return index < N; }
        };

        Handle acquire()
        {
                for (size_t i = 0; i < N; i++)
                        if (!slots_[i]) {
                                slots_[i].emplace();
                                return {static_cast<uint8_t>(i)};
                        }
                return {};      /* exhausted - a BOUNDED
                                   failure                  */
        }

        void release(Handle h)
                { if (h.valid()) slots_[h.index].reset(); }

        T *get(Handle h)
                { return h.valid() && slots_[h.index]
                         ? &*slots_[h.index] : nullptr; }
};
/* A HANDLE IS SMALLER THAN A POINTER, survives the pool
   moving, and can be validated. A generation counter added to
   the handle catches use-after-release:                   */
        struct Handle { uint8_t index; uint8_t generation; };
/* which is the standard "generational index" pattern, and it
   turns a dangling reference into a detectable error.

/* --- 2. SINGLE OWNER, BORROWED REFERENCES --- */
        class System {
                Uart   uart_{UART0_BASE};   /* System OWNS
                Sensor sensor_{&uart_};        these  */
                Logger logger_{&uart_};
        };
/* Members are constructed in declaration order and destroyed
   in reverse (chapter 03), so `uart_` outlives its users by
   construction. No allocation, no smart pointers, and the
   lifetimes are provable by reading the class.

/* --- 3. STATIC STORAGE WITH PLACEMENT NEW --- */
        alignas(Driver) static std::byte storage[sizeof(Driver)];
        Driver *drv = nullptr;

        void init() { drv = new (storage) Driver{cfg}; }
        void deinit() { drv->~Driver(); drv = nullptr; }
/* You control exactly when construction happens - useful when
   it must be after clock setup (chapter 01).            */
```

How it shows up

Symptom	Cause
Double free	Two smart pointers constructed from one raw pointer
Derived destructor never runs	Base destructor not `virtual`
Memory grows steadily	`shared_ptr` cycle — break it with `weak_ptr`
Hot path slower than expected	Copying `shared_ptr` — atomic refcounts
Cache line contention across cores	Shared refcount being modified by several threads
Destructor runs on an unexpected thread	`shared_ptr` — the last owner destroys it, wherever that is
`unique_ptr` is 16 bytes	A stateful deleter — use a stateless functor
"use of deleted function" copying a `unique_ptr`	Correct — it is move-only
Caller must have a `unique_ptr` to call your function	Take `const T&` instead
Use-after-free with a pool	No generation counter in the handle
Object outlives what it points to	Raw member pointer with no lifetime guarantee

Check yourself

- ✓ Say what each pointer type communicates about ownership.
- ✓ State the parameter conventions for taking versus borrowing.
- ✓ Prove `unique_ptr` has no size or code overhead.
- ✓ Write a custom deleter for a file descriptor and for a hardware channel.
- ✓ Explain the `using pointer =` trick.
- ✓ List four costs of `shared_ptr` and say when it is genuinely right.
- ✓ Break a reference cycle with `weak_ptr`.
- ✓ Give five rules for using smart pointers correctly.
- ✓ Design a static pool with generational handles and say what it detects.
- ✓ Express ownership in a heap-free system three different ways.

Inheritance, virtual functions and the vtable

How runtime polymorphism is implemented, exactly what it costs, and the four alternatives that give you the same flexibility without an indirect call. On embedded the answer is often "not virtual" — but you need to know the mechanism to make that judgement.

The vtable

```
class Shape {
public:
        virtual ~Shape() = default;
        virtual int area() const = 0;
        virtual void draw() const = 0;
};

class Circle : public Shape {
        int r_;
public:
        int area() const override { return 3 * r_ * r_; }
        void draw() const override { ... }
};

OBJECT LAYOUT of a Circle on AArch64:

   +0   __vptr    -----------+   8 bytes, ALWAYS FIRST
   +8   r_                   |   4 bytes (+ 4 padding)
   = 16 bytes total          |
                             v
   Circle's VTABLE (one per CLASS, in .rodata):
      [0]  offset-to-top   0
      [1]  typeinfo ptr    -> Circle's type_info (RTTI)
      [2]  &Circle::~Circle          (complete)
      [3]  &Circle::~Circle          (deleting)
      [4]  &Circle::area
      [5]  &Circle::draw

A CALL:
     shape->area();
compiles to:
     ldr  x8, [x0]           ; load the vptr
     ldr  x8, [x8, #32]      ; load the slot
     blr  x8                 ; indirect call

THREE INSTRUCTIONS AND TWO DEPENDENT LOADS, versus one
`bl` for a direct call.

THE COSTS, precisely:
  · 8 bytes per OBJECT for the vptr
  · one vtable per CLASS in .rodata
  · two dependent loads before the call - a potential
    cache miss each
  · an INDIRECT branch, which the predictor must learn
    (CA chapter 12)
  · NO INLINING, unless the compiler can prove the dynamic
    type - which blocks every downstream optimisation

THAT LAST POINT IS USUALLY THE LARGEST COST. The call
itself is a few cycles; the lost inlining can be a factor
of two on small functions.
```

The rules that matter

```
/* --- 1. A POLYMORPHIC BASE NEEDS A VIRTUAL DESTRUCTOR --- */
        class Base { public: ~Base(); };        /* NOT virtual */
        Base *p = new Derived;
        delete p;               /* ~Derived NEVER RUNS -
                                   UNDEFINED BEHAVIOUR       */

        class Base { public: virtual ~Base() = default; };
/* -Wnon-virtual-dtor / -Wdelete-non-virtual-dtor catch this.
   Enable them.

   THE EXCEPTION: if a base is never deleted through a base
   pointer, a PROTECTED non-virtual destructor is correct and
   cheaper:                                                  */
        class Interface {
        protected:
                ~Interface() = default;         /* cannot
                                                   delete
                                                   through
                                                   this      */
        public:
                virtual void run() = 0;
        };

/* --- 2. ALWAYS WRITE `override` --- */
        class Derived : public Base {
                void draw() const override;     /* checked */
                void draw();                    /* silently a
                                                   NEW
                                                   function -
                                                   const
                                                   mismatch  */
        };
/* -Wsuggest-override, -Winconsistent-missing-override.
   `override` turns a silent bug into a compile error, and
   costs one word.

/* --- 3. `final` enables DEVIRTUALISATION --- */
        class Circle final : public Shape { ... };
        void f(Circle& c) { c.area(); }   /* the compiler KNOWS
                                             the type -> a
                                             DIRECT, INLINABLE
                                             call            */
/* Mark leaf classes and non-overridden virtuals `final`. It
   is free and it recovers the inlining.

/* --- 4. NEVER CALL A VIRTUAL FROM A CONSTRUCTOR --- */
/* During Base's constructor the vptr points at BASE's vtable
   (chapter 03). The override is not called. Use two-phase
   init.

/* --- 5. SLICING --- */
        void f(Shape s);        /* BY VALUE - only the Shape
                                   part is copied           */
        Circle c;
        f(c);                   /* SLICED - r_ is lost, and
                                   the vptr is Shape's      */
/* Always pass polymorphic types by reference or pointer.
   Making the base abstract prevents it entirely, because
   `Shape s` will not compile.                              */
```

```
/* --- 6. DEFAULT ARGUMENTS ARE STATICALLY BOUND --- */
        class Base    { virtual void f(int x = 1); };
        class Derived { void f(int x = 2) override; };
        Base *p = new Derived;
        p->f();                 /* calls Derived::f(1) - the
                                   DEFAULT came from Base   */
/* Do not use default arguments on virtual functions.      */
```

Multiple and virtual inheritance

```
/* MULTIPLE INHERITANCE from interfaces is fine and cheap: */
        class Readable  { public: virtual int read() = 0; };
        class Writable  { public: virtual int write(int) = 0; };
        class Port : public Readable, public Writable { ... };

/* LAYOUT: TWO vptrs, one per base. And a `Writable*` obtained
   from a `Port*` is at a NON-ZERO OFFSET, so:            */
        Port p;
        Writable *w = &p;       /* w != (void*)&p ! */
/* The compiler adjusts the pointer. Casting back and forth
   with reinterpret_cast rather than static_cast breaks
   silently - which is one reason C-style casts are dangerous
   in C++.

/* AND THE THUNK: calling through Writable* needs `this`
   adjusted before entering Port's function, so the vtable
   entry points at a small thunk that does the subtraction.
   You will see these in disassembly as tiny functions named
   `non-virtual thunk to Port::write`.

/* VIRTUAL INHERITANCE - the diamond: */
        class Base { int x; };
        class A : virtual public Base {};
        class B : virtual public Base {};
        class D : public A, public B {};    /* ONE Base */

/* THE COST is substantial: a virtual base pointer per
   subobject, an extra indirection to reach Base's members,
   and complicated construction ordering (the most-derived
   class constructs the virtual base).

   ON EMBEDDED: avoid virtual inheritance entirely. If you
   need the diamond, you almost certainly want composition or
   pure interfaces with no data.                          */

/* THE INTERFACE RULE that avoids all of this:
     a class with virtual functions has NO DATA MEMBERS
     a class with data members has NO VIRTUAL FUNCTIONS and is
     not inherited from
   Then multiple inheritance is only ever from interfaces, and
   the diamond cannot arise.                              */
```

> **PREFER COMPOSITION TO INHERITANCE**
> Inheritance couples the derived class to the base's implementation, its layout, and its future changes. Composition — holding a member and forwarding — couples only to the interface. The C++ Core Guidelines put it as: use inheritance for *substitutability* (an interface), and composition for *reuse*. Almost every "share this code between classes" problem is composition, not inheritance.

The four alternatives

```
/* --- 1. TEMPLATES: static polymorphism (chapter 06) --- */
template<typename Sensor>
void sample_loop(Sensor& s)
{
        while (running)
                log(s.read());          /* resolved at COMPILE
                                           time, INLINED     */
}
/* ZERO cost, full inlining. The type set must be known at
   compile time, and each instantiation is a separate copy of
   the code.

/* --- 2. CRTP (chapter 06) --- */
template<typename D>
class SensorBase {
        D& self() { return static_cast<D&>(*this); }
public:
        int read() { return self().convert(self().read_raw()); }
};
/* An interface with shared implementation, statically
   dispatched.

/* --- 3. std::variant + std::visit (chapter 08) --- */
        using AnySensor = std::variant<Thermistor, Photodiode,
                                       Hall>;

        std::array<AnySensor, 4> sensors;

        for (auto& s : sensors)
                std::visit([](auto& sensor)
                        { log(sensor.read()); }, s);

/* A HETEROGENEOUS CONTAINER with no heap and no vtable. The
   dispatch is a jump table on the variant's index - one
   indirect branch, but the target is a known small set and
   the functions CAN be inlined into the visitor.

   THE TRADE: the type list is closed. Adding a sensor type
   means recompiling, which for firmware is fine and for a
   plugin system is not.

/* --- 4. A FUNCTION POINTER TABLE: the C way, still good --- */
        struct DriverOps {
                int  (*read)(void *ctx);
                int  (*write)(void *ctx, int v);
                void (*close)(void *ctx);
        };
        struct Driver {
                const DriverOps *ops;
                void *ctx;
        };
        d.ops->read(d.ctx);
/* Exactly a hand-rolled vtable - and this is precisely what
   the Linux kernel's file_operations is (Linux chapter 20).
   It is explicit, has no language machinery behind it, and
   costs the same indirect call.

/* CHOOSING:

     types known at compile time, one type per call site
```

```
          -> TEMPLATES
     interface with shared implementation, static
          -> CRTP
     small closed set, heterogeneous container, no heap
          -> VARIANT
     open set, runtime plugins, ABI boundary
          -> VIRTUAL (or an ops table)

   AND THE HONEST NOTE: virtual functions are not expensive
   in absolute terms. If the function does meaningful work,
   the indirect call is noise. The cost that matters is lost
   inlining on SMALL functions called in a loop - and that is
   the case where an alternative pays.                    */
```

Measuring the cost

```
/* --- SIZE --- */
        static_assert(sizeof(NonPolymorphic) == 4);
        static_assert(sizeof(Polymorphic) == 16);
                        /* 8 for the vptr + 4 + padding */

/* On an object you allocate a thousand of, 8 bytes each is
   8 KB. On a microcontroller that is a real consideration.

/* --- SEE THE VTABLES --- */
        clang++ -Xclang -fdump-vtable-layouts -c file.cpp
        nm -C prog.elf | grep vtable
        objdump -s -j .rodata prog.elf     /* they live here */

/* --- CODE SIZE --- */
        bloaty prog.elf -d symbols
/* Look for: vtables, typeinfo objects, thunks, and the
   __cxa_pure_virtual handler. With -fno-rtti the typeinfo
   disappears; with -fno-exceptions much of the rest does.

/* --- WHETHER A CALL WAS DEVIRTUALISED --- */
        gcc -fdump-ipa-devirt
        clang -Rpass=devirt
/* Or simply read the assembly (compilers chapter 16): a `bl`
   to a named function was devirtualised; `ldr`+`blr` was not.

/* --- WHAT HELPS DEVIRTUALISATION --- */
        final on classes and on individual virtual functions
        -fwhole-program or LTO
        anonymous namespaces, which prove no external
                overrides exist
        passing concrete types rather than base references
        where the call site knows

/* --- AND THE FLAGS THAT REMOVE THE MACHINERY --- */
        -fno-rtti               /* no typeinfo, no
                                   dynamic_cast            */
        -fno-exceptions
        -fno-threadsafe-statics
/* All three are standard for embedded C++ (chapter 01), and
   together they typically remove tens of kilobytes.      */
```

RTTI

```
#include <typeinfo>

        Shape *s = get_shape();

        if (Circle *c = dynamic_cast<Circle *>(s)) {
                use(*c);                /* it was a Circle */
        }
        Circle& r = dynamic_cast<Circle&>(*s);   /* THROWS
                                                    on failure */

        if (typeid(*s) == typeid(Circle)) { ... }
        const char *name = typeid(*s).name();   /* MANGLED */

/* THE COST: a typeinfo object per polymorphic class, plus a
   runtime walk of the inheritance graph for dynamic_cast -
   which is not constant time with multiple inheritance.

/* AND -fno-rtti REMOVES IT ENTIRELY, which is what embedded
   builds do. Then dynamic_cast and typeid are compile
   errors.

/* THE ALTERNATIVE when you need to recover the concrete type:
   put a tag in the base and use static_cast:              */
        class Shape {
        public:
                enum class Kind { Circle, Square };
                Kind kind() const { return kind_; }
                virtual ~Shape() = default;
        protected:
                explicit Shape(Kind k) : kind_(k) {}
        private:
                Kind kind_;
        };

        if (s->kind() == Shape::Kind::Circle)
                auto *c = static_cast<Circle *>(s);

/* This is LLVM's own idiom - its `isa<>` and `dyn_cast<>`
   are exactly this, built on a kind field, and LLVM is
   compiled with -fno-rtti for the reason above.

/* AND THE BETTER ANSWER, usually: if you are asking what
   concrete type something is, the interface is probably
   missing a virtual function that would let you avoid
   asking.                                                 */
```

How it shows up

Symptom	Cause
Derived destructor never runs; memory leaks	Base destructor not `virtual`
An override is silently not called	Signature mismatch — `override` would have caught it
Object loses its derived data	Slicing — passed by value
Wrong default argument used	Defaults are statically bound on virtual functions
Virtual call in constructor calls the base version	The vptr is not yet the derived one

Symptom	Cause
Pointer comparison fails after a cast	Multiple inheritance — a base pointer is at an offset
Crash after `reinterpret_cast` between base and derived	Same — use `static_cast`
Objects 8 bytes larger than expected	The vptr
Small virtual function is a bottleneck	Lost inlining — mark `final`, or use a template
`dynamic_cast` is a compile error	`-fno-rtti` — use a kind tag
Link error: `__cxa_pure_virtual`	A pure virtual called during construction, or missing runtime support
Tens of kilobytes of typeinfo in the binary	RTTI enabled — turn it off

Check yourself

- ✓ Draw an object's layout and its vtable, and write out the three instructions of a virtual call.
- ✓ List the five costs of a virtual call and say which usually dominates.
- ✓ Explain why a polymorphic base needs a virtual destructor, and the protected exception.
- ✓ Explain what `override` and `final` each buy.
- ✓ Explain slicing and how to make it impossible.
- ✓ Explain why a base pointer under multiple inheritance is at an offset.
- ✓ State the interface rule that prevents the diamond.
- ✓ Give four alternatives to virtual dispatch and say when each fits.
- ✓ Measure vtable and typeinfo cost in a real binary.
- ✓ Replace `dynamic_cast` with a kind tag and explain LLVM's use of it.

Exceptions and error-handling strategies

How C++ exceptions actually work, what they cost, and why most embedded projects disable them. Then the alternatives — and one of them, `std::expected`, is genuinely better than either exceptions or error codes.

How they work

```
throw MyError{42};
     |
1. ALLOCATE the exception object. __cxa_allocate_exception
   uses malloc, falling back to an emergency static buffer.
     |
2. __cxa_throw begins UNWINDING.
     |
3. PHASE 1 - SEARCH. Walk up the stack using the .eh_frame
   unwind tables (compilers chapter 14), asking each frame's
   "personality routine" whether it has a matching handler.
   NOTHING IS DESTROYED YET.
     |
4. If no handler is found -> std::terminate. This is why an
   uncaught exception can abort with the stack intact.
     |
5. PHASE 2 - CLEANUP. Walk up again, this time running each
   frame's destructors and landing pads, and transfer
   control to the handler.
     |
catch (const MyError& e) { ... }

THE TWO-PHASE DESIGN exists so that if no handler exists,
the program terminates with the throwing frame still on the
stack - which makes the core dump useful.

THE IMPLEMENTATION IS TABLE-DRIVEN ("zero-cost"):
  · NO code on the non-throwing path. No flag to set, no
    branch, no register saved. A function that never throws
    runs exactly as fast as the C equivalent.
  · The cost is entirely in DATA: .eh_frame and
    .gcc_except_table describe, for every address, what to
    destroy and where the handlers are.

SO THE HONEST COST is:
  RUNTIME (happy path)   zero
  RUNTIME (throwing)     microseconds - table lookup,
                         unwinding, allocation. NOT bounded.
  CODE SIZE              typically 10-20% larger binary
  RAM                    a heap allocation per throw
  DETERMINISM            none - the unwind time depends on
                         stack depth and table contents
```

Why embedded disables them

```
/* -fno-exceptions is the default choice in embedded C++, and
   the reasons are concrete rather than dogmatic:         */

   1. CODE SIZE. The tables are 10-20% of the binary, and the
      unwinder itself (libgcc_eh or libunwind) is tens of
      kilobytes. On a 256 KB part that is decisive.

   2. NON-DETERMINISM. Throw time depends on stack depth, the
      number of frames with destructors, and table lookups.
      A hard real-time deadline cannot accommodate an
      unbounded operation on any path.

   3. THE ALLOCATION. __cxa_allocate_exception calls malloc.
      On a system with no heap that is a link error; on one
      with a heap it is an allocation on the error path, which
      is exactly where you least want one.

   4. WRITING EXCEPTION-SAFE CODE IS HARD, and the
      consequences of getting it wrong are subtle - a
      half-updated data structure after an unwind is worse
      than an error code that was ignored.

   5. THE ECOSYSTEM. Vendor SDKs, RTOS APIs and HALs are C.
      An exception cannot propagate through a C stack frame
      (there are no unwind tables), so it terminates.

   BUILD WITH:
        -fno-exceptions -fno-unwind-tables
        -fno-asynchronous-unwind-tables

   WHAT CHANGES:
     · `throw` and `try`/`catch` are compile errors
     · std::terminate is called where the library would throw
     · new returns nullptr if you use std::nothrow, otherwise
       it terminates - so use the nothrow form
     · vector::at, optional::value, variant::get become
       unusable - use operator[], *opt, get_if
     · noexcept is implied everywhere

   MEASURE IT ON YOUR OWN CODE before deciding. On embedded
   Linux with megabytes of flash, exceptions may be entirely
   affordable, and the argument is then only about
   determinism.                                           */
```

> **THE ONE THING EXCEPTIONS GENUINELY DO BETTER**
>
> They cannot be ignored. An error code returned and not checked is silent; an exception not caught terminates loudly. And they propagate through call chains that have no way to return a value — most importantly constructors, which is why a constructor that can fail either throws or requires a two-phase init. Every alternative below has to answer that second point.

std::expected

```
#include <expected>     /* C++23; std::optional plus an error */

/* THE BEST OF BOTH: a value or an error, in the return type,
   with no allocation, no unwinding, and no way to ignore it
   accidentally.                                              */

enum class Error { Timeout, Nack, BusBusy, BadArgument };

std::expected<uint16_t, Error> read_register(uint8_t addr)
{
        if (addr > MAX_REG)
                return std::unexpected(Error::BadArgument);

        if (!bus_acquire())
                return std::unexpected(Error::BusBusy);

        auto v = bus_read(addr);
        if (!v)
                return std::unexpected(Error::Timeout);

        return *v;              /* the success value */
}

        auto r = read_register(0x10);
        if (r) {
                use(*r);
        } else {
                log(r.error());
        }

        uint16_t v = read_register(0x10).value_or(0);

/* AND IT COMPOSES - monadic operations (C++23): */
        auto result = read_register(0x10)
                .and_then([](uint16_t v)
                        { return validate(v); })
                .transform([](uint16_t v)
                        { return v * 2; })
                .or_else([](Error e)
                        { return fallback(e); });

/* LAYOUT: a union of the value and the error, plus a
   discriminant. INLINE, no allocation - typically
   max(sizeof(T), sizeof(E)) + alignment.

   static_assert(sizeof(std::expected<uint16_t, Error>) == 4);

/* MARK IT [[nodiscard]] and ignoring it becomes a warning: */
        [[nodiscard]] std::expected<uint16_t, Error>
                read_register(uint8_t addr);

/* IF YOUR TOOLCHAIN LACKS C++23, tl::expected is a
   header-only implementation with the same interface, and
   it is widely used in embedded projects for exactly this
   reason.                                                 */
```

The other strategies

```
/* --- 1. ERROR CODES, the C way ------------------------- */
        Status read(uint8_t addr, uint16_t *out);
        if (read(0x10, &v) != Status::Ok) { ... }
/* PROS: universal, no machinery, works with C.
   CONS: out-parameters, ignorable, no composition, and the
   value cannot be const.
   FIX THE IGNORABILITY: */
        [[nodiscard]] Status read(...);

/* --- 2. optional, when there is only one failure mode --- */
        std::optional<uint16_t> read(uint8_t addr);
/* Simpler than expected when the caller does not need to
   know WHY.

/* --- 3. AN ERROR STATE ON THE OBJECT, sticky ------------ */
class Bus {
        Error last_ = Error::None;
public:
        uint16_t read(uint8_t a)
        {
                if (last_ != Error::None) return 0;   /* already
                                                         failed */
                ...
        }
        Error error() const { return last_; }
        void clear_error() { last_ = Error::None; }
        explicit operator bool() const
                { return last_ == Error::None; }
};
        bus.write(0x10, 1);
        bus.write(0x11, 2);
        bus.write(0x12, 3);
        if (!bus) handle(bus.error());   /* CHECK ONCE, after
                                            a sequence       */
/* THE STICKY-ERROR PATTERN is genuinely good for a sequence
   of hardware operations: no per-call checking, and the
   first failure suppresses the rest so nothing operates on
   garbage. It is how iostreams work, and how many HAL
   layers should.

/* --- 4. ASSERT AND ABORT, for programmer errors ---------- */
        void set_channel(uint8_t ch)
        {
                assert(ch < NUM_CHANNELS);   /* a BUG, not a
                                                runtime error */
                ...
        }
/* DISTINGUISH THE TWO CASES, and this is the important
   distinction in the whole chapter:

     A PROGRAMMER ERROR - a precondition violated, an invalid
     index, a null where one is impossible - is a BUG. Assert,
     and fail fast. Returning an error code invites the caller
     to continue with a broken invariant.

     A RUNTIME ERROR - the hardware timed out, the input was
     malformed, the resource is busy - is an EXPECTED
     CONDITION. Return it.

   Conflating them produces either code full of unreachable
```

```
   error handling, or bugs papered over as failures.      */

/* --- 5. CONSTRUCTORS THAT CANNOT FAIL ------------------- */
/* The problem exceptions solve. Without them: */

        /* (a) two-phase init */
        class Device {
                bool ok_ = false;
        public:
                Device() = default;
                bool init(uintptr_t base);   /* returns success */
                explicit operator bool() const { return ok_; }
        };

        /* (b) a static factory returning expected */
        class Device {
                explicit Device(uintptr_t base);   /* private,
                                                      cannot
                                                      fail    */
        public:
                static std::expected<Device, Error>
                create(uintptr_t base)
                {
                        if (!probe(base))
                                return std::unexpected(
                                        Error::NotPresent);
                        return Device{base};
                }
        };
        auto d = Device::create(0x40000000);

/* (b) IS THE BETTER PATTERN: there is no "constructed but
   invalid" state to check for, and the type system enforces
   that you handled the failure before you have a Device.  */
```

Exception safety, if you do use them

```
/* FOUR GUARANTEES a function can offer: */

   NOTHROW      it never throws. Mark it `noexcept`.
   STRONG       if it throws, the program state is UNCHANGED -
                the operation is transactional.
   BASIC        if it throws, everything is still VALID and
                destructible, but possibly modified. No leaks,
                no broken invariants.
   NONE         all bets are off. Unacceptable.

/* THE BASIC GUARANTEE IS THE MINIMUM. Achieve it with RAII
   (chapter 02): if every resource is owned by an object,
   unwinding releases everything automatically.

/* THE STRONG GUARANTEE, via copy-and-swap: */
        void Container::insert(const T& v)
        {
                Container tmp = *this;  /* may throw - harmless */
                tmp.do_insert(v);       /* may throw - harmless */
                swap(*this, tmp);       /* noexcept - COMMITS */
        }
/* Do all the throwing work on a copy, then commit with an
   operation that cannot fail. Expensive, and the only
   general way to be transactional.

/* noexcept, and why it matters: */
        void f() noexcept;
/* If f throws anyway, std::terminate is called IMMEDIATELY -
   no unwinding. So `noexcept` is a promise, not a filter.

   MARK noexcept:
     · move constructors and move assignment (chapter 03 -
       vector depends on it)
     · destructors (implicitly noexcept already)
     · swap
     · anything genuinely trivial
   DO NOT mark it speculatively: removing it later is an ABI
   break.

/* DESTRUCTORS MUST NOT THROW. They are implicitly noexcept,
   and a throwing destructor during unwinding terminates the
   program. If a destructor performs an operation that can
   fail - closing a file, flushing a buffer - either swallow
   the error or provide an explicit close() the user calls
   first.                                                  */
```

Interoperating with C

```
/* An exception must NEVER propagate through a C stack frame.
   There are no unwind tables for it, and the behaviour is
   undefined - in practice, terminate.                    */

/* SO EVERY C CALLBACK NEEDS A BARRIER: */
extern "C" int my_callback(void *ctx, int arg) noexcept
{
        try {
                auto *self = static_cast<Handler *>(ctx);
                return self->handle(arg);
        } catch (const std::exception& e) {
                log_error(e.what());
                return -1;
        } catch (...) {
                return -1;
        }
}

/* THE `noexcept` ON AN extern "C" FUNCTION is the belt and
   braces: even if a catch is missed, terminate happens HERE,
   at a diagnosable point, rather than corrupting the C
   caller's stack.

/* AND THE SAME AT EVERY BOUNDARY:
     · a signal handler
     · a thread entry function
     · an interrupt handler (in the rare case C++ is used
       there)
     · anything registered with a C library
     · a DLL/shared-library export

   RULE: `extern "C"` and `noexcept` go together, with a
   catch-all inside.                                      */
```

Choosing

Context	Strategy
Bare metal, hard real-time	`-fno-exceptions`; `expected` or error codes; assert for bugs
Bare metal, soft real-time	Same — the code size argument usually decides it
Embedded Linux application	Either. `expected` for recoverable errors, exceptions for genuinely exceptional ones
A library with a C API	No exceptions across the boundary; translate at the edge
Kernel or driver code	Never — no runtime support exists
Constructor that can fail	A static factory returning `expected`
Programmer error	`assert`, always — never an error return
Existing codebase	Match it. Mixing two strategies is worse than either

How it shows up

Symptom	Cause
Link error: `__gxx_personality_v0`	Exceptions enabled but no unwinder linked — use `-fno-exceptions`

Symptom	Cause
Link error: `__cxa_throw`, `_Unwind_Resume`	Same
Binary 20% larger than expected	Unwind tables — check with `size -A`
Terminate with no message	An exception escaped a `noexcept` function or a C boundary
Terminate during stack unwinding	A destructor threw
Crash on the error path only	Allocation failure inside `__cxa_allocate_exception`
`vector::at` causes a link error	It throws — use `operator[]`
`new` terminates instead of returning null	Use `new (std::nothrow)`
Data structure half-updated after an error	Only the basic guarantee was provided where strong was needed
`vector` copies instead of moving	Move constructor not `noexcept` (ch. 03)
An error was silently ignored	No `[[nodiscard]]` on the return type
Latency spike on an error path	Unwinding — the reason real-time systems disable it

Check yourself

- ✓ Explain the two-phase unwinding process and why it is two phases.
- ✓ State the four costs of exceptions precisely, including the zero one.
- ✓ Give five reasons embedded projects disable them, and what changes when you do.
- ✓ Say the one thing exceptions do that alternatives must answer for.
- ✓ Use `std::expected` including its monadic operations, and state its layout.
- ✓ Distinguish a programmer error from a runtime error, and handle each correctly.
- ✓ Give two ways to handle a constructor that can fail, and say which is better.
- ✓ Name the four exception-safety guarantees and achieve the strong one.
- ✓ Say when to mark `noexcept` and why it is an ABI commitment.
- ✓ Write a correct C callback barrier.

The C++ memory model, threads and atomics

C++11 gave the language a formal memory model — the first time C or C++ said anything precise about what threads may observe. It maps directly onto the ARM barriers from ARM chapter 21, and it is what makes portable lock-free code possible at all.

The memory model

```
THE CENTRAL RULE:

  A DATA RACE IS UNDEFINED BEHAVIOUR.

A data race exists when two threads access the same memory
location, at least one writes, and the accesses are not
ordered by a happens-before relationship.

NOT "you get one value or the other". Undefined - the
compiler may cache the value in a register forever, split a
store into two, invent a store, or fold the branch away
(compilers chapter 15).

SO EVERY SHARED MUTABLE LOCATION must be either:
  · protected by a mutex, or
  · declared std::atomic

`volatile` IS NOT THE ANSWER. volatile means "this access
is observable, do not elide or reorder it relative to other
volatile accesses". It says NOTHING about atomicity or
about ordering relative to non-volatile accesses. It is for
MMIO (chapter 02), not for threads. This is the single most
common misconception in C++ concurrency.

HAPPENS-BEFORE is established by:
  · sequenced-before, within one thread
  · a mutex unlock, then a lock of the same mutex
  · a release store, then an acquire load that reads it
  · thread creation, and join
  · and transitively through all of the above
```

Atomics

```
#include <atomic>

        std::atomic<int> counter{0};

        counter++;                      /* atomic RMW */
        counter.fetch_add(1);
        counter.store(5);
        int v = counter.load();
        int old = counter.exchange(10);

        int expected = 5;
        if (counter.compare_exchange_strong(expected, 10)) {
                /* it WAS 5, now it is 10 */
        } else {
                /* it was not; `expected` now holds the actual
                   value - note the IN-OUT parameter        */
        }

/* THE WEAK FORM may fail SPURIOUSLY, and is cheaper on
   LL/SC architectures like ARM - so always use it in a
   loop: */
        int old = counter.load();
        int next;
        do {
                next = compute(old);
        } while (!counter.compare_exchange_weak(old, next));

/* IS IT LOCK-FREE? */
        static_assert(std::atomic<int>::is_always_lock_free);
        std::atomic<BigStruct> x;
        assert(x.is_lock_free());       /* probably FALSE -
                                           it uses a hidden
                                           MUTEX             */
/* An atomic larger than the machine's widest atomic
   instruction is implemented with a lock, which on embedded
   is almost never acceptable. Assert lock-freedom on
   anything you rely on.

   ON AARCH64: 1, 2, 4, 8 bytes are lock-free natively, and
   16 bytes are with LSE (ARMv8.1) via CASP.

/* std::atomic_flag is the only type GUARANTEED lock-free
   everywhere: */
        std::atomic_flag lock = ATOMIC_FLAG_INIT;
        while (lock.test_and_set(std::memory_order_acquire))
                ;                       /* a spinlock */
        ...
        lock.clear(std::memory_order_release);
```

Memory orderings

```
/* Six orderings, from strongest to weakest. Each maps to
   specific ARM instructions.                              */

   memory_order_seq_cst      the DEFAULT. Sequential
                             consistency: a single total order
                             of all seq_cst operations, seen
                             identically by every thread.
        ARM: ldar / stlr, plus a dmb ish for stores on some
        implementations.

   memory_order_acquire      a load. Nothing after it in
                             program order may move before it.
        ARM: ldar

   memory_order_release      a store. Nothing before it may
                             move after it.
        ARM: stlr

   memory_order_acq_rel      for read-modify-write: both.
        ARM: ldaxr / stlxr

   memory_order_relaxed      atomicity only. NO ordering
                             guarantees at all.
        ARM: a plain ldr / str

   memory_order_consume      deprecated in practice - every
                             implementation promotes it to
                             acquire.

/* THE RELEASE/ACQUIRE PAIR is the workhorse, and it is
   exactly the ARM pattern from chapter 21: */

        /* producer */
        data = 42;                              /* plain */
        ready.store(true, std::memory_order_release);

        /* consumer */
        if (ready.load(std::memory_order_acquire))
                use(data);      /* GUARANTEED to see 42 */

/* THE RELEASE STORE publishes everything written before it;
   the ACQUIRE LOAD that observes it sees all of that. This is
   RCU's rcu_assign_pointer/rcu_dereference (Linux chapter 12)
   expressed in the language.

/* RELAXED, and when it is correct: */
        std::atomic<uint64_t> packet_count{0};
        packet_count.fetch_add(1, std::memory_order_relaxed);
/* A statistic that nothing else depends on: you need the
   increment to be atomic, and you do not care when other
   threads see it. Relaxed is a plain `ldxr/stxr` (or an LSE
   `ldadd`) with no barrier - meaningfully cheaper.

/* THE COST ON ARM, roughly:
     relaxed     ~ the same as a plain access
     acquire     ldar - a few cycles
     release     stlr - a few cycles
     seq_cst     as above, plus possibly a dmb - tens of
                 cycles under contention
```

```
   USE THE DEFAULT (seq_cst) unless you have measured a
   problem AND can prove the weaker ordering is correct.
   Reasoning about relaxed orderings is genuinely difficult,
   and the failures are unreproducible.                  */
```

THE `VOLATILE` TRAP, STATED ONCE MORE

```
volatile bool ready = false;    // WRONG for threads
ready = true;                   // not atomic, no ordering
```

`volatile` prevents the compiler eliding the access. It does **not** make it atomic, and it emits no barrier — so on ARM the CPU may still reorder it relative to the data it was meant to publish. Java and C# gave `volatile` acquire/release semantics; C and C++ did not. Use `std::atomic` for threads and `volatile` for MMIO, and never confuse them.

Threads and mutexes

```
#include <thread>
#include <mutex>
#include <condition_variable>

        std::thread t{[]{ work(); }};
        std::thread t2{worker_fn, arg1, arg2};

        t.join();               /* wait for it */
        t.detach();             /* let it run independently */
/* A std::thread that is neither joined nor detached calls
   std::terminate in its destructor. That is deliberate:
   silently abandoning a thread is worse.

        std::jthread jt{[](std::stop_token st) {
                while (!st.stop_requested()) work();
        }};                     /* C++20 - joins automatically
                                   in its destructor, and
                                   carries a stop token     */

/* MUTEXES, always via a guard (RAII, chapter 02): */
        std::mutex m;

        {
                std::lock_guard lk{m};          /* C++17 CTAD */
                shared_data++;
        }                                       /* unlocked */

        std::unique_lock lk{m};         /* movable, can unlock
                                           and relock        */
        lk.unlock();
        lk.lock();

        std::scoped_lock lk{m1, m2};    /* SEVERAL mutexes,
                                           deadlock-free
                                           ordering          */

        std::shared_mutex sm;
        { std::shared_lock r{sm}; }     /* many readers */
        { std::unique_lock w{sm}; }     /* one writer */

/* NEVER lock and unlock by hand. An early return or an
   exception between them leaves it locked forever.

/* CONDITION VARIABLES: */
        std::mutex m;
        std::condition_variable cv;
        bool ready = false;

        /* waiter */
        {
                std::unique_lock lk{m};
                cv.wait(lk, []{ return ready; });   /* the
                                                       PREDICATE
                                                       form   */
                consume();
        }

        /* notifier */
        {
                std::lock_guard lk{m};
```

```
                ready = true;
        }
        cv.notify_one();

/* THE PREDICATE FORM IS MANDATORY. Without it a SPURIOUS
   WAKEUP - which is permitted and does happen - returns from
   wait() with the condition false. The predicate overload
   loops for you.

/* AND THE LOST-WAKEUP RULE: set the condition WHILE HOLDING
   the mutex, then notify. Notifying without the lock is
   allowed and sometimes faster, but setting the flag without
   it is a race.                                            */
```

Patterns

```
/* --- SINGLE-PRODUCER SINGLE-CONSUMER RING, lock-free ---- */
template<typename T, size_t N>
class SpscQueue {
        static_assert((N & (N - 1)) == 0, "N must be a power
                                           of two");
        std::array<T, N> buf_;
        alignas(64) std::atomic<size_t> head_{0};   /* producer */
        alignas(64) std::atomic<size_t> tail_{0};   /* consumer */

public:
        bool push(const T& v)
        {
                size_t h = head_.load(std::memory_order_relaxed);
                size_t t = tail_.load(std::memory_order_acquire);

                if (h - t >= N) return false;   /* full */

                buf_[h & (N - 1)] = v;
                head_.store(h + 1, std::memory_order_release);
                return true;
        }

        bool pop(T& out)
        {
                size_t t = tail_.load(std::memory_order_relaxed);
                size_t h = head_.load(std::memory_order_acquire);

                if (h == t) return false;       /* empty */

                out = buf_[t & (N - 1)];
                tail_.store(t + 1, std::memory_order_release);
                return true;
        }
};

/* THE THREE THINGS THAT MAKE IT CORRECT:
     1. each index is written by ONE thread only, so no CAS is
        needed
     2. the release store publishes the DATA written before
        it; the acquire load on the other side sees it
     3. alignas(64) puts head_ and tail_ on SEPARATE CACHE
        LINES - without it, the producer's store to head_
        invalidates the consumer's cached tail_ on every
        operation. FALSE SHARING (CA chapter 15), and it can
        cost an order of magnitude.

/* --- A SPINLOCK, when a mutex is too heavy --------------- */
class Spinlock {
        std::atomic_flag flag_ = ATOMIC_FLAG_INIT;
public:
        void lock()
        {
                while (flag_.test_and_set(
                                std::memory_order_acquire)) {
                        while (flag_.test(
                                std::memory_order_relaxed))
                                __builtin_arm_yield();
                                /* the ARM `yield` hint -
                                   relaxes the pipeline and
                                   helps SMT/cluster peers  */
```

```
                }
        }
        void unlock()
                { flag_.clear(std::memory_order_release); }
};
/* THE TEST-AND-TEST-AND-SET SHAPE matters: spinning on
   test_and_set writes the cache line every iteration,
   bouncing it between cores. Spinning on a relaxed `test`
   only reads, so the line stays shared until it is actually
   free.

/* --- CALL ONCE --- */
        std::once_flag flag;
        std::call_once(flag, []{ expensive_init(); });

/* --- AND THE SIMPLEST CORRECT SINGLETON --- */
        Device& device() {
                static Device instance;  /* thread-safe
                                            initialisation is
                                            GUARANTEED since
                                            C++11            */
                return instance;
        }
/* -fno-threadsafe-statics removes the guard variable, which
   is correct only if you know initialisation is
   single-threaded (chapter 01).                          */
```

On embedded

```
/* std::thread needs an OS. On bare metal you have interrupts
   and possibly an RTOS instead - but the MEMORY MODEL still
   applies, and atomics are still the right tool.        */

/* --- SHARING WITH AN INTERRUPT HANDLER --- */
        std::atomic<uint32_t> sample_count{0};
        std::atomic<bool>     data_ready{false};
        Sample buffer[N];               /* plain */

        extern "C" void ADC_IRQHandler(void)
        {
                buffer[idx] = read_adc();       /* plain
                                                   stores    */
                data_ready.store(true,
                        std::memory_order_release);
        }

        void main_loop(void)
        {
                if (data_ready.load(std::memory_order_acquire)) {
                        process(buffer);        /* sees the
                                                   stores    */
                        data_ready.store(false,
                                std::memory_order_relaxed);
                }
        }

/* ON A SINGLE CORE, an interrupt cannot interleave with
   itself, so the ordering concern is only the COMPILER's -
   and atomic gives you that for free, portably, instead of a
   `volatile` plus a hand-written barrier.

   ON A MULTI-CORE MCU or an AMP system, the full memory model
   applies exactly as on a big system.

/* --- WHAT std::atomic COMPILES TO on an ARMv7-M --- */
        std::atomic<uint32_t> x;
        x.fetch_add(1);
   ->   a LDREX/STREX loop - no lock, no library call.
        std::atomic<uint64_t> y;        /* on 32-bit ARM */
        y.fetch_add(1);
   ->   possibly a LIBRARY CALL with a lock. Check with
        is_always_lock_free before relying on it.

/* --- AND THE CRITICAL-SECTION ALTERNATIVE --- */
        class InterruptLock { ... };    /* chapter 02 */
        {
                InterruptLock guard;
                shared_struct.field_a = 1;      /* a MULTI-WORD
                shared_struct.field_b = 2;         update      */
        }
/* For anything larger than one atomic word on a single-core
   part, disabling interrupts briefly is simpler and cheaper
   than a lock-free algorithm. Use atomics for single values
   and interrupt masking for compound updates.           */
```

Finding races

```
# ThreadSanitizer - the only tool that reliably finds these:
    g++ -fsanitize=thread -g -O1 prog.cpp
    ./prog

    WARNING: ThreadSanitizer: data race
      Write of size 4 at 0x... by thread T2:
        #0 worker() prog.cpp:15
      Previous read of size 4 at 0x... by thread T1:
        #0 main prog.cpp:22
      Location is global 'counter' of size 4

# ~10x slower, ~5x memory. Run your test suite under it
# nightly; it finds races that will never reproduce under
# ordinary testing.

# HELGRIND, for lock-order problems:
    valgrind --tool=helgrind ./prog

# AND THE STATIC CHECKS:
    clang -Wthread-safety, with annotations:
        class Cache {
                std::mutex m_;
                int data_ GUARDED_BY(m_);
                void update() REQUIRES(m_);
        };
# The compiler then verifies at BUILD TIME that data_ is only
# touched with m_ held. This is genuinely excellent and badly
# underused - it turns a class of race into a compile error.

# ON EMBEDDED, where sanitizers are unavailable:
#   · reason explicitly about every shared variable and write
#     the reasoning down
#   · make everything shared either atomic or interrupt-masked
#   · use static analysis and code review
#   · stress test with an interrupt rate far above the
#     expected one
#   · and prefer designs with a single owner per datum, so
#     the question does not arise
```

How it shows up

Symptom	Cause
Works in debug, fails in release	A race the optimiser exposed — `volatile` instead of `atomic`
Works on one core, fails on many	Missing barriers; the ordering was accidental
A flag is never observed as set	The compiler cached it in a register — needs `atomic`
Data visible after the flag, but stale	Missing release/acquire pairing
Spurious wakeup processes garbage	Condition variable wait without a predicate
Deadlock acquiring two mutexes	Inconsistent order — use `std::scoped_lock`
Terminate on thread destruction	Neither joined nor detached — use `jthread`
Atomic operations are surprisingly slow	`seq_cst` under contention, or not lock-free
Queue throughput collapses with two cores	False sharing — `alignas(64)` the indices

Symptom	Cause
A 64-bit atomic calls into the library	Not lock-free on that target — assert it
Race found by TSan that never reproduces	Real. Fix it — reproducibility is not the test

Check yourself

- ✓ State what a data race is and what the standard says happens.
- ✓ Explain precisely why `volatile` is not a threading tool.
- ✓ Name the ways happens-before is established.
- ✓ Use `compare_exchange` correctly, and say why the weak form needs a loop.
- ✓ Name the six memory orderings and their ARM instructions.
- ✓ Write a release/acquire pair and say exactly what it guarantees.
- ✓ Say when `relaxed` is correct.
- ✓ Explain why a condition variable wait needs a predicate.
- ✓ Write an SPSC ring and justify each ordering and the `alignas`.
- ✓ Explain test-and-test-and-set and the cache-line reason for it.
- ✓ Use atomics to share data with an interrupt handler, and say when to mask instead.
- ✓ Run TSan and use `-Wthread-safety` annotations.

Zero-cost abstraction: reading the generated assembly

C++'s central claim is that its abstractions cost nothing you would not have paid by hand. That claim is checkable in about thirty seconds per case, and this chapter is a series of those checks — the ones that hold, and the ones that do not.

The claim, stated precisely

```
Stroustrup's formulation:

  "What you don't use, you don't pay for. And what you do
   use, you couldn't hand-code any better."

THE FIRST HALF IS ABOUT ABSENCE: a program that uses no
virtual functions carries no vtables; one that throws no
exceptions pays nothing on the happy path.

THE SECOND HALF IS ABOUT PRESENCE: an abstraction whose
types are known at compile time is compiled away entirely.

WHAT MAKES AN ABSTRACTION FREE:
  · the type is known at compile time
  · the function is visible for inlining
  · no indirection the compiler cannot see through
  · no allocation

WHAT MAKES ONE COST:
  · TYPE ERASURE — virtual, std::function, dynamic_cast.
    The compiler cannot know the target, so it cannot
    inline.
  · ALLOCATION — vector, string, shared_ptr, exceptions
  · ATOMIC OPERATIONS — shared_ptr's refcount
  · CODE the abstraction adds that you would not have
    written: bounds checks, refcounts, unwind tables

THE METHOD FOR THE REST OF THIS CHAPTER: write both
versions, compile both at -O2 for aarch64, and diff the
assembly (compilers chapter 16).
```

The ones that are free

```
/* --- std::array versus a C array ------------------------ */
        int sum_c(const int *a, size_t n) {
                int s = 0;
                for (size_t i = 0; i < n; i++) s += a[i];
                return s;
        }
        int sum_cpp(const std::array<int, 4>& a) {
                return std::accumulate(a.begin(), a.end(), 0);
        }
/* -O2: sum_cpp is BETTER - the size is known, so it fully
   unrolls:
        ldp  w1, w2, [x0]
        ldp  w3, w4, [x0, #8]
        add  w1, w1, w2
        add  w3, w3, w4
        add  w0, w1, w3
        ret
   Five instructions, no loop. The C version cannot do this
   because n is a runtime value.

/* --- unique_ptr versus new/delete ----------------------- */
        void f_c()   { Widget *p = new Widget; use(p);
                       delete p; }
        void f_cpp() { auto p = std::make_unique<Widget>();
                       use(p.get()); }
/* IDENTICAL assembly. Byte for byte.

/* --- a class wrapper around a register ------------------ */
        class Reg {
                volatile uint32_t *p_;
        public:
                explicit Reg(uintptr_t a)
                        : p_(reinterpret_cast<volatile uint32_t *>(a)) {}
                uint32_t read() const  { return *p_; }
                void write(uint32_t v) { *p_ = v; }
        };
        void f(Reg r) { r.write(r.read() | 1); }
/* ->  ldr  w1, [x0]
        orr  w1, w1, #1
        str  w1, [x0]
        ret
   Identical to the C version. The class does not exist at run
   time.

/* --- a strong type (chapter 07) ------------------------- */
        void delay_c(uint32_t ms);
        void delay_cpp(Milliseconds ms);
        delay_cpp(100_ms);
/* ->  mov  w0, #100
        b    delay
   The literal suffix, the class, the constructor - all gone.

/* --- a lambda in an algorithm --------------------------- */
        auto n = std::count_if(v.begin(), v.end(),
                               [](int x){ return x > 10; });
/* Compiles to the same loop as a hand-written one, and often
   vectorises where a function pointer would not - because
   the predicate is INLINED and therefore visible to the
   vectoriser (compilers chapter 07).
```

```
/* --- CRTP versus virtual (chapter 06) ------------------- */
        struct Base { virtual int f() = 0; };
        struct D : Base { int f() override { return 42; } };
        int use_virt(Base& b) { return b.f() + 1; }
/* ->  ldr  x8, [x0]        ; vptr
        ldr  x8, [x8]        ; slot
        blr  x8              ; INDIRECT CALL
        add  w0, w0, #1

        template<typename D> struct CBase {
                int f() { return static_cast<D&>(*this).impl(); }
        };
        struct CD : CBase<CD> { int impl() { return 42; } };
        int use_crtp(CD& d) { return d.f() + 1; }
/* ->  mov  w0, #43
        ret
   The entire call chain constant-folded away.           */
```

CHECK IT YOURSELF, IN THIRTY SECONDS

Open godbolt.org, set the compiler to *armv8-a clang* or *ARM64 gcc*, flags to `-O2`, and paste both versions side by side. Every claim in this chapter is verifiable that way, and the habit of verifying rather than believing is the actual skill being taught.

The ones that are not

```
/* --- std::function ---------------------------------------- */
        void apply_fn(std::function<int(int)> f, int x)
                { use(f(x)); }
/* ->  a call through a type-erased vtable, plus possibly a
        heap allocation when the std::function was
        constructed. NOT inlinable.

        template<typename F>
        void apply_tmpl(F f, int x) { use(f(x)); }
        apply_tmpl([](int y){ return y * 2; }, 5);
/* ->  mov  w0, #10
        b    use
   Fully inlined and folded. The SAME lambda, two orders of
   magnitude apart, purely because of type erasure.

   THE EMBEDDED-FRIENDLY MIDDLE GROUND - a fixed-capacity
   inplace_function, or simply a function pointer plus a
   context pointer (the C ops-table idiom, chapter 11).

/* --- shared_ptr ------------------------------------------- */
        void f(std::shared_ptr<Widget> p) { use(*p); }
/* Passing by value: an ATOMIC increment on entry and an
   atomic decrement on exit, each a full barrier on ARM. Pass
   `const shared_ptr&` or, better, `Widget&`.

/* --- std::string ------------------------------------------ */
        void log_cpp(const std::string& s);
        log_cpp("literal");
/* Constructs a temporary std::string: possibly an allocation,
   a copy, and a destruction. Use string_view (chapter 08):
        void log_sv(std::string_view s);
        log_sv("literal");
   ->  two immediates - a pointer and a length. No
        allocation, no strlen at run time.

/* --- .at() versus operator[] ------------------------------ */
        int a(std::vector<int>& v, size_t i) { return v.at(i); }
/* ->  a comparison, a branch, and a call to
        __throw_out_of_range - plus the unwind machinery.
        int b(std::vector<int>& v, size_t i) { return v[i]; }
/* ->  one `ldr`.
   Both are correct choices; know which you are paying for.

/* --- iostreams -------------------------------------------- */
        std::cout << "value: " << x << '\n';
/* Pulls in locale, facets, the whole formatting machinery -
   commonly 100 KB+ on a bare-metal target. std::format
   (C++20) is smaller but still substantial; a hand-rolled
   printf-style function or std::to_chars is what embedded
   code should use.
        std::to_chars(buf, buf + n, x);   /* no allocation, no
                                             locale, fast     */
```

Measuring properly

```
# --- CODE SIZE, per symbol ---
    nm --size-sort -S -C prog.elf | tail -20
    bloaty prog.elf -d symbols,compileunits
    size -A prog.elf

# --- DIFF TWO VERSIONS ---
    g++ -O2 -S -o a.s a.cpp
    g++ -O2 -S -o b.s b.cpp
    diff -u a.s b.s

# --- COUNT INSTRUCTIONS IN A FUNCTION ---
    objdump -d prog.elf | awk '/<myfunc>:/,/^$/' | wc -l

# --- PROVE NO ALLOCATION HAPPENS ---
    /* on bare metal, simply do not provide operator new:
       any accidental allocation becomes a LINK ERROR - the
       strongest guarantee available */

    /* on a hosted system, count them: */
    static std::atomic<size_t> g_allocs{0};
    void *operator new(size_t n) { g_allocs++; return malloc(n); }
    void operator delete(void *p) noexcept { free(p); }

    size_t before = g_allocs;
    hot_path();
    assert(g_allocs == before);         /* in a test */

# --- PROVE A TYPE IS WHAT YOU THINK ---
    static_assert(sizeof(Wrapper) == sizeof(uint32_t));
    static_assert(std::is_trivially_copyable_v<Message>);
    static_assert(std::is_standard_layout_v<RegBlock>);
    static_assert(alignof(Buffer) == 64);

# --- AND ACTUAL TIME, which is the only thing that
#     ultimately matters ---
    perf stat -r 10 ./prog
    /* cycles, instructions, IPC, cache-misses,
       branch-misses (compilers chapter 16) */
```

When the abstraction is not free

```
/* Usually because inlining did not happen. The causes, in
   order of frequency: */

   1. THE DEFINITION IS NOT VISIBLE.
        A member function defined in a .cpp cannot be inlined
        into another translation unit without LTO
        (compilers chapter 12). Define small functions in the
        HEADER.

   2. IT IS TOO LARGE for the inliner's threshold.
        Split it: a small inlinable wrapper around a large
        out-of-line body.
                inline int fast_path(int x) {
                        if (likely(x < 16)) return table[x];
                        return slow_path(x);   /* not inlined,
                                                  and that is
                                                  fine       */
                }

   3. IT IS VIRTUAL and the type is not provable.
        Mark the class or the function `final` (chapter 11),
        or use a template.

   4. IT IS CALLED THROUGH A FUNCTION POINTER or
      std::function.

   5. THE ADDRESS OF A LOCAL ESCAPED, so the variable cannot
      be promoted to a register (compilers chapter 05).

   6. -O0. Nothing is inlined; measuring there tells you
      nothing.

/* FORCE IT, when you have measured a reason: */
        __attribute__((always_inline)) inline void f();
        [[gnu::always_inline]] inline void f();
/* AND FORBID IT, for cold paths that would pollute the
   I-cache: */
        __attribute__((noinline)) void error_path();

/* ASK WHY IT DID NOT HAPPEN: */
        clang -Rpass-missed=inline -Rpass-analysis=inline
        gcc -fopt-info-inline-missed
/* which prints the reason per call site - "cost exceeds
   threshold", "function not inlinable", "recursive". That
   output is the answer far more often than guessing.    */
```

A worked comparison

```
/* A realistic embedded case: a GPIO abstraction. */

/* --- C --- */
        #define GPIO_BASE 0x40020000
        #define GPIO_ODR  (*(volatile uint32_t *)(GPIO_BASE + 0x14))
        static inline void gpio_set(int pin)
                { GPIO_ODR |= (1u << pin); }

/* --- C++ --- */
template<uintptr_t Base>
class GpioPort {
        struct Regs {
                volatile uint32_t MODER, OTYPER, OSPEEDR,
                                  PUPDR, IDR, ODR;
        };
        static Regs *regs()
                { return reinterpret_cast<Regs *>(Base); }
public:
        template<uint8_t Pin>
        class Pin_t {
                static_assert(Pin < 16, "invalid pin");
        public:
                static void set()    { regs()->ODR |=  (1u << Pin); }
                static void clear()  { regs()->ODR &= ~(1u << Pin); }
                static bool read()   { return regs()->IDR &
                                              (1u << Pin); }
        };
};
        using PortA = GpioPort<0x40020000>;
        using LedPin = PortA::Pin_t<5>;

        LedPin::set();

/* BOTH COMPILE TO: */
        movz w8, #0x2000, lsl #16       ; the base
        movk w8, #0x14                  ; + ODR offset
        ldr  w9, [x8]
        orr  w9, w9, #0x20              ; bit 5
        str  w9, [x8]

/* IDENTICAL. And the C++ version additionally gives you:
     · a static_assert on the pin number - an invalid pin is
       a BUILD error
     · named, typed registers instead of offsets
     · LedPin as a TYPE, passable to templates
     · no macros, so no namespace pollution and no
       double-evaluation hazard
     · the port and pin cannot be mismatched, because they
       are one type

   THAT IS THE ARGUMENT for C++ in firmware, in one example:
   the same instructions, with errors moved from run time to
   compile time.                                          */
```

A verification checklist

Claim	How to check
"This wrapper is free"	Diff the assembly against the raw version
"No allocation in this path"	Instrument `operator new`, or omit it entirely
"This type is one word"	`static_assert(sizeof(T) == ...)`
"This is inlined"	Look for the `bl`; or `-Rpass=inline`
"This call is devirtualised"	A named `bl` rather than `ldr` + `blr`
"This is computed at compile time"	It is in `.rodata`; `readelf -x`
"This loop vectorises"	`v` registers in the output; `-fopt-info-vec`
"Exceptions cost nothing here"	`size -A` with and without `-fno-exceptions`
"This is faster"	`perf stat -r 10` — the only claim that needs timing

How it shows up

Symptom	Cause
An abstraction that should be free is not	Inlining did not happen — ask the compiler why
A wrapper adds instructions	Definition not visible; needs to be in the header or LTO
Unexpected `bl` to `operator new`	A hidden allocation — `string`, `function`, `vector`
Indirect call where you expected direct	Virtual and not devirtualised — mark `final`
A lambda is not inlined	Stored in `std::function` — pass as a template parameter
Binary grew after adding one header	iostreams, or a heavy template instantiation
Measured at `-O0` and concluded C++ is slow	Measure at the level you ship
A benchmark shows zero time	The whole thing was optimised away — the result is unused
Assembly identical but the program is slower	Memory bound, not code bound — check `perf stat`

Check yourself

- ✓ State the zero-cost claim precisely and say what makes an abstraction free.
- ✓ Verify on godbolt that `unique_ptr`, `array` and a register wrapper are free.
- ✓ Show that a lambda as a template parameter is free and in `std::function` is not.
- ✓ Name five abstractions that cost, and what each costs.
- ✓ Prove a code path performs no allocation.
- ✓ List six reasons inlining fails, and ask the compiler which applies.
- ✓ Write a templated GPIO abstraction and prove it matches the C version.
- ✓ Say what that abstraction buys beyond the identical instructions.
- ✓ Work the verification checklist against a claim in your own code.

Modern C++ for embedded

A practical assembly of everything in this track: the features worth adopting, the patterns that recur in real firmware, and a coding standard you could hand to a team on Monday.

The adoption list

Feature	Since	Why it earns its place
`enum class`	C++11	No implicit int conversion; explicit underlying type
`nullptr`	C++11	Overload resolution works correctly
`constexpr`	C++11/14	Tables and configuration in flash (ch. 07)
`static_assert`	C++11	Build-time validation of layouts and configuration
`std::array`	C++11	An array that knows its size, at zero cost
`std::unique_ptr`	C++11	Ownership in the type, with custom deleters for handles
`override`, `final`	C++11	Catch signature mistakes; enable devirtualisation
`= delete`	C++11	Make copying a compile error
`<atomic>`	C++11	A real memory model (ch. 13)
`<chrono>`	C++11	Unit-safe durations, entirely `constexpr`
`std::string_view`	C++17	Strings with no allocation
`std::optional`	C++17	Absence without a sentinel
`std::variant`	C++17	A tagged union, exhaustively checked
`if constexpr`	C++17	Replaces most `#ifdef` (ch. 06)
Structured bindings	C++17	Readable destructuring
`[[nodiscard]]`	C++17	Error returns cannot be silently dropped
Guaranteed copy elision	C++17	Return by value is free
`std::span`	C++20	Replaces every pointer-plus-length pair
Concepts	C++20	Comprehensible template errors
Designated initialisers	C++20	Readable configuration structs
`consteval`, `constinit`	C++20	Guarantees about when initialisation happens
`std::bit_cast`	C++20	Type punning without UB (compilers ch. 15)
`std::expected`	C++23	The best error-handling primitive available

Patterns that recur

```
/* --- 1. STRUCTURED BINDINGS + designated initialisers --- */
        struct Config { uint32_t rate; uint8_t gain;
                        bool enabled; };

        constexpr Config cfg{
                .rate    = 48000,       /* C++20 - order must
                .gain    = 3,              match declaration */
                .enabled = true,
        };

        auto [rate, gain, enabled] = cfg;   /* C++17 */

        for (auto& [key, value] : map) { ... }

        if (auto [it, ok] = set.insert(x); ok) { ... }
                                        /* init-statement in
                                           an if - C++17     */

/* --- 2. A TYPED REGISTER, with compile-time field
       checking --- */
template<uint32_t Addr>
struct Register {
        static uint32_t read()
                { return *reinterpret_cast<volatile uint32_t *>(Addr); }
        static void write(uint32_t v)
                { *reinterpret_cast<volatile uint32_t *>(Addr) = v; }

        template<uint8_t Hi, uint8_t Lo>
        struct Field {
                static_assert(Hi < 32 && Lo <= Hi);
                static constexpr uint32_t mask =
                        ((1u << (Hi - Lo + 1)) - 1) << Lo;

                static uint32_t get()
                        { return (read() & mask) >> Lo; }

                static void set(uint32_t v)
                {
                        write((read() & ~mask)
                              | ((v << Lo) & mask));
                }
        };
};
        using CTRL   = Register<0x40000000>;
        using Enable = CTRL::Field<0, 0>;
        using Mode   = CTRL::Field<5, 4>;

        Enable::set(1);
        auto m = Mode::get();
/* Shifts and masks computed at compile time; an out-of-range
   field is a build error (chapter 14 proves the cost).

/* --- 3. A STATE MACHINE with variant ---------------------- */
        struct Idle {};
        struct Connecting { uint32_t started; };
        struct Connected  { uint8_t session; };
        struct Error      { ErrorCode code; };

        using State = std::variant<Idle, Connecting,
                                   Connected, Error>;
```

```
        State next(const State& s, Event e)
        {
                return std::visit([&](const auto& st) -> State {
                        using T = std::decay_t<decltype(st)>;

                        if constexpr (std::is_same_v<T, Idle>) {
                                if (e == Event::Connect)
                                        return Connecting{now()};
                        } else if constexpr (
                                std::is_same_v<T, Connecting>) {
                                if (e == Event::Ack)
                                        return Connected{alloc()};
                                if (now() - st.started > TIMEOUT)
                                        return Error{
                                             ErrorCode::Timeout};
                        }
                        return st;      /* unchanged */
                }, s);
        }
/* EACH STATE CARRIES ITS OWN DATA, the compiler checks that
   every state is handled, and there is no heap. Adding a
   state without handling it fails to compile.

/* --- 4. SPAN EVERYWHERE at interfaces ------------------- */
        size_t encode(std::span<uint8_t> out,
                      std::span<const Sample> in);
/* No length parameters to get out of sync, works with arrays,
   vectors and raw buffers alike.

/* --- 5. CHRONO for every duration ----------------------- */
        using namespace std::chrono_literals;

        void set_timeout(std::chrono::milliseconds t);
        set_timeout(500ms);
        set_timeout(2s);        /* converted correctly */
        set_timeout(500);       /* COMPILE ERROR - good */

        constexpr auto PERIOD = 10ms;
        constexpr auto TICKS  = PERIOD / 1ms;   /* = 10 */

/* --- 6. [[nodiscard]] on every fallible operation -------- */
        [[nodiscard]] std::expected<Reading, Error> read();
        [[nodiscard]] bool try_lock();
        read();                 /* WARNING */

/* --- 7. A COMPILE-TIME DISPATCH TABLE ------------------- */
        constexpr std::array handlers{
                &handle_status, &handle_config, &handle_data,
        };
        static_assert(handlers.size() == NUM_COMMANDS);
        if (cmd < handlers.size())
                handlers[cmd]();
/* In flash, bounds-checked at build time, one indirect
   call.                                                    */
```

Freestanding C++

```
/* On bare metal, much of the standard library is unavailable -
   but the LANGUAGE is entirely available, and the
   header-only parts of the library mostly are too.       */

/* AVAILABLE without a hosted implementation: */
        <type_traits>   <limits>      <cstddef>
        <cstdint>       <utility>     <initializer_list>
        <atomic>        <bit>         <concepts>
        <compare>       <ratio>       <array> (mostly)
        <span>          <string_view> (mostly)
        <algorithm>     (the non-allocating parts)
        <chrono>        (the duration parts)
        <optional>      <variant>     (with -fno-exceptions
                                       caveats)

/* WHAT YOU MUST PROVIDE YOURSELF: */
        /* the runtime support the compiler assumes exists */
        extern "C" void __cxa_pure_virtual() { for (;;) ; }
        extern "C" int  __cxa_atexit(void (*)(void *), void *,
                                     void *) { return 0; }
        extern "C" void __cxa_guard_acquire() { }
        /* or build with -fno-use-cxa-atexit
           -fno-threadsafe-statics and avoid needing them */

        /* operator new, if anything uses it - or DELIBERATELY
           OMIT IT so any use is a link error (chapter 14) */
        void *operator new(size_t) = delete;
        void  operator delete(void *) noexcept = delete;

        /* memcpy, memset - the compiler emits calls to these
           even in freestanding mode (compilers chapter 07) */
        extern "C" void *memcpy(void *d, const void *s, size_t n);

/* THE STARTUP SEQUENCE that must run before main: */
        extern void (*__init_array_start[])(void);
        extern void (*__init_array_end[])(void);

        static void call_constructors(void)
        {
                for (auto **p = __init_array_start;
                     p < __init_array_end; p++)
                   (*p)();
        }
/* Called from the reset handler after copying .data and
   zeroing .bss (compilers chapter 12). If .init_array is
   empty - the goal (chapter 01) - this loop does nothing and
   costs nothing.

/* THE FLAGS: */
        -ffreestanding -fno-exceptions -fno-rtti
        -fno-threadsafe-statics -fno-use-cxa-atexit
        -fno-unwind-tables -fno-asynchronous-unwind-tables
        -ffunction-sections -fdata-sections
        -Wl,--gc-sections -nostdlib
        -Os -std=c++17
```

FIXED-CAPACITY LIBRARY CHOICES

ETL (Embedded Template Library) gives you the STL's interfaces with fixed capacity and no heap: `etl::vector<T, N>`, `etl::map`, `etl::string`, `etl::optional`, state machines and message routing. **Boost.SML** for state machines. **fmt** or `std::to_chars` instead of iostreams. **tl::expected** if you are pre-C++23. Each is header-only and designed for exactly this environment.

A coding standard

```
# Embedded C++ Coding Standard
# ============================

## Build flags (mandatory)
  -std=c++17            (or c++20 where the toolchain allows)
  -fno-exceptions -fno-rtti
  -fno-threadsafe-statics -fno-use-cxa-atexit
  -Wall -Wextra -Wpedantic -Werror
  -Wshadow -Wconversion -Wsign-conversion
  -Wnon-virtual-dtor -Wold-style-cast -Wcast-align
  -Wuseless-cast -Wdouble-promotion -Wformat=2
  -Wsuggest-override
  -ffunction-sections -fdata-sections -Wl,--gc-sections

## Forbidden
  - dynamic allocation after initialisation
  - std::string, std::vector, std::map, std::function
  - iostreams
  - exceptions, dynamic_cast, typeid
  - virtual inheritance
  - C-style casts
  - macros where constexpr, inline or a template works
  - `using namespace` at file scope in a header
  - raw owning pointers

## Required
  - every resource wrapped in RAII
  - every single-argument constructor `explicit`
  - `override` on every override
  - `final` on every leaf class
  - `[[nodiscard]]` on every fallible function
  - `const` on every member function that does not mutate
  - `constexpr` wherever it compiles
  - `static_assert` on every struct whose layout matters
  - fixed-width types (uint32_t), never `int` for hardware
  - std::span or std::array at every buffer interface
  - std::chrono for every duration

## Preferred
  - the rule of zero (chapter 03)
  - composition over inheritance
  - templates or variant over virtual, where the type set is
    closed
  - std::expected or an error enum over error codes
  - free functions over member functions where possible
  - anonymous namespaces over `static` at file scope

## Review checklist
  - does it allocate?          (grep for new, make_unique)
  - can it fail silently?      ([[nodiscard]]?)
  - is every path bounded?     (no unbounded loops or waits)
  - is shared state atomic or masked?  (chapter 13)
  - does the binary grow?      (size -A, before and after)

# ONE PAGE. Enforceable by the compiler for most of it, and by
# grep in CI for the rest. That is the point - a standard
# nobody can check is a standard nobody follows.
```

Migrating a C codebase

```
/* Incremental, in this order - each step is independently
   valuable and independently revertible.                  */

   1. COMPILE THE EXISTING C AS C++. Fix what breaks:
        · implicit void* conversions need casts
        · `class`, `new`, `template` etc. as identifiers
        · stricter enum rules
        · missing casts on malloc
      This alone finds real bugs, and it is the whole step -
      do not change anything else yet.

   2. REPLACE MACROS with constexpr and inline functions.
      Typed, scoped, debuggable.

   3. WRAP RESOURCES IN RAII, one at a time. Each conversion
      removes a class of leak. Start with the ones whose
      cleanup is most often forgotten.

   4. REPLACE ARRAYS AND POINTER PAIRS with std::array and
      std::span at interfaces.

   5. INTRODUCE enum class for state and mode values.

   6. ADD [[nodiscard]] to fallible functions and fix the
      warnings.

   7. REPLACE FUNCTION-POINTER TABLES with templates or
      variant WHERE THE TYPE SET IS CLOSED - and leave them
      alone where it is not.

   8. ONLY THEN consider classes with virtual functions, and
      only where runtime polymorphism is genuinely needed.

/* WHAT NOT TO DO:
     · rewrite everything at once
     · introduce exceptions into a codebase that does not
       expect them
     · use inheritance to share implementation
     · adopt the whole standard library because it is there

/* AND MEASURE AT EVERY STEP:
        size -A before.elf after.elf
        perf stat, or a cycle counter on the target
   A migration that grows the binary by 40% has gone wrong,
   and you want to know which step did it.               */
```

How it shows up

Symptom	Cause
Link error: `__cxa_pure_virtual`	Missing freestanding runtime support
Link error: `operator new`	Something allocates — find it, that is the point
Link error: `__cxa_guard_acquire`	A function-local static — add `-fno-threadsafe-statics`
Code runs before `main` unexpectedly	A static constructor — use `constexpr` or `constinit`
Binary grew after migration	Measure per step; usually one specific feature

Symptom	Cause
`std::optional::value()` is a link error	It throws — use `*opt`
Designated initialisers rejected	C++20, and the order must match declaration order
Concepts unavailable	Toolchain predates C++20 — use `static_assert`
Team disagrees about C++ features	No written standard — that is the actual problem
Warnings ignored across the codebase	No `-Werror` in CI

Check yourself

- ✓ Name twenty modern features worth adopting and what each buys.
- ✓ Write a typed register abstraction with compile-time field checking.
- ✓ Implement a state machine with `variant` and explain the exhaustiveness guarantee.
- ✓ List what is available in freestanding C++ and what you must supply.
- ✓ Write the reset-handler sequence including `.init_array`.
- ✓ Produce a one-page coding standard with mandatory flags and a review checklist.
- ✓ Explain why omitting `operator new` is a strong guarantee.
- ✓ Give the eight migration steps in order and say what each is worth alone.
- ✓ Name three fixed-capacity libraries and what each replaces.
- ✓ Measure a migration step's effect on binary size.

Interoperating with C

Every embedded C++ project sits on top of C: a vendor HAL, an RTOS, a driver, libc. Mixing them safely is a small set of rules about linkage, layout and lifetimes — and getting them wrong produces link errors at best and silent corruption at worst.

extern "C"

```
/* C++ MANGLES function names to encode types, so overloads
   get distinct symbols. C does not. `extern "C"` says "use C
   linkage for this".                                        */

        void foo(int);          /* C++: _Z3fooi */
        extern "C" void foo(int);   /* C: foo */

        nm prog.o
          0000000000000000 T _Z3fooi
          0000000000000000 T foo
        c++filt _Z3fooi
          foo(int)

/* THE HEADER PATTERN, so one header works for both
   languages: */

        /* driver.h */
        #ifndef DRIVER_H
        #define DRIVER_H

        #include <stdint.h>

        #ifdef __cplusplus
        extern "C" {
        #endif

        int  driver_init(uintptr_t base);
        int  driver_read(uint8_t *buf, size_t n);
        void driver_close(void);

        #ifdef __cplusplus
        }
        #endif

        #endif

/* WHAT extern "C" DOES:
     · disables name mangling
     · uses the C calling convention (identical on AArch64,
       but not on every platform)
   WHAT IT DOES NOT DO:
     · it does NOT change the language rules inside the
       function - you may still use C++ features
     · it does NOT allow OVERLOADING, because there is only
       one symbol name available:                          */
        extern "C" void f(int);
        extern "C" void f(double);      /* ERROR */

/* CALLING C++ FROM C: expose a C-linkage wrapper. */
        /* wrapper.hpp */
        class Driver { ... };

        /* wrapper.cpp */
        extern "C" void *driver_create(uintptr_t base)
        {
                return new (std::nothrow) Driver{base};
        }
        extern "C" int driver_read(void *h, uint8_t *b, size_t n)
        {
                return static_cast<Driver *>(h)->read(b, n);
```

```
        }
        extern "C" void driver_destroy(void *h)
        {
                delete static_cast<Driver *>(h);
        }
/* An OPAQUE HANDLE plus C-linkage functions is the standard
   shape, and it is exactly what a C library with a struct
   pointer looks like from the outside.                   */
```

Layout compatibility

```
/* For C and C++ to agree on a struct, it must be STANDARD
   LAYOUT: no virtual functions, no virtual bases, all
   non-static data members with the same access control, no
   base class with data members.                          */

        struct Message {               /* shareable */
                uint32_t id;
                uint16_t length;
                uint8_t  payload[64];
        };
        static_assert(std::is_standard_layout_v<Message>);
        static_assert(std::is_trivially_copyable_v<Message>);
        static_assert(sizeof(Message) == 72);
        static_assert(offsetof(Message, payload) == 8);

/* THOSE FOUR ASSERTIONS are what makes it safe to pass this
   struct to C, memcpy it, or DMA it. Put them next to every
   type that crosses the boundary.

/* WHAT BREAKS LAYOUT COMPATIBILITY: */
        struct Bad {
                uint32_t id;
                virtual void f();      /* adds a VPTR at
                                          offset 0       */
        };
        struct Bad2 {
        public:  uint32_t a;
        private: uint32_t b;           /* mixed access -
                                          order is
                                          unspecified    */
        };

/* WHAT IS SAFE to add to a standard-layout struct:
     · member FUNCTIONS (non-virtual) - they are not stored
     · static members
     · constructors and destructors - though adding them
       makes it non-TRIVIAL, so it can no longer be memcpy'd
       or DMA'd

        struct Message {
                uint32_t id;
                uint16_t length;
                uint8_t  payload[64];

                /* these cost NOTHING and change no layout: */
                std::span<const uint8_t> data() const
                        { return {payload, length}; }
                bool valid() const { return length <= 64; }
        };
/* SAME BYTES, better interface. This is a genuinely
   underused technique: take the C struct exactly as it is
   and add methods.                                       */

/* AND THE ENUM QUESTION: */
        enum class Mode : uint8_t { Idle, Run };
/* An enum class with an explicit underlying type has exactly
   that representation, so it is safe in a shared struct - C
   sees a uint8_t.                                        */
```

STANDARD LAYOUT VERSUS TRIVIALLY COPYABLE

Two different guarantees. **Standard layout** means C and C++ agree on where the members are, so the struct can cross the boundary. **Trivially copyable** means it can be `memcpy`'d, DMA'd or stored in a union — no user-provided copy, move or destructor. A struct with a constructor is standard layout but not trivially copyable, and DMA into it is undefined. Assert both where both matter.

Callbacks

```
/* C libraries take function pointers, usually with a `void
   *context`. C++ has three ways to supply one.           */

        typedef void (*event_cb)(void *ctx, int event);
        void register_handler(event_cb cb, void *ctx);

/* --- 1. A CAPTURELESS LAMBDA converts to a function
       pointer --- */
        register_handler([](void *ctx, int ev) {
                static_cast<Handler *>(ctx)->on_event(ev);
        }, &my_handler);
/* Clean, and the lambda has C++ linkage - which is
   technically not the same as `extern "C"` and is accepted
   by every real compiler. For strict correctness on unusual
   platforms, use form 2.

/* --- 2. A STATIC MEMBER, or a free extern "C" function --- */
        class Handler {
                void on_event(int ev);
        public:
                static void trampoline(void *ctx, int ev)
                {
                        static_cast<Handler *>(ctx)->on_event(ev);
                }
                void install() { register_handler(trampoline,
                                                  this); }
        };

/* --- 3. A TEMPLATE TRAMPOLINE, generated per type --- */
        template<typename T, void (T::*M)(int)>
        void trampoline(void *ctx, int ev)
        {
                (static_cast<T *>(ctx)->*M)(ev);
        }
        register_handler(trampoline<Handler,
                                    &Handler::on_event>,
                         &h);
/* Fully inlined, no runtime indirection beyond the C call
   itself.

/* --- THE THREE RULES for every C callback --- */

/* (a) NO EXCEPTIONS MAY ESCAPE (chapter 12): */
        extern "C" void cb(void *ctx, int ev) noexcept
        {
                try   { handler(ctx, ev); }
                catch (...) { log("callback threw"); }
        }

/* (b) THE CONTEXT MUST OUTLIVE THE REGISTRATION. A callback
       registered with `this` and never unregistered before
       the object is destroyed is a dangling pointer waiting
       for an event. Unregister in the destructor:        */
        ~Handler() { unregister_handler(trampoline, this); }

/* (c) A NON-CAPTURELESS LAMBDA CANNOT CONVERT. */
        int x = 5;
        register_handler([x](void*, int){ use(x); }, nullptr);
        /* ERROR - it has state, so it is not a plain
```

```
            function. Put the state in the context pointer,
            which is what the context pointer is FOR.     */
```

Wrapping an RTOS or HAL

```
/* The most valuable thing C++ does to a C API: wrap it in
   RAII once, and every user gets correct cleanup.        */

/* --- a FreeRTOS mutex --- */
class Mutex {
        SemaphoreHandle_t h_;
public:
        Mutex() : h_(xSemaphoreCreateMutex()) {
                configASSERT(h_ != nullptr);
        }
        ~Mutex() { vSemaphoreDelete(h_); }

        Mutex(const Mutex&) = delete;
        Mutex& operator=(const Mutex&) = delete;

        void lock()   { xSemaphoreTake(h_, portMAX_DELAY); }
        void unlock() { xSemaphoreGive(h_); }
        bool try_lock_for(TickType_t t)
                { return xSemaphoreTake(h_, t) == pdTRUE; }
};
/* NOW std::lock_guard WORKS ON IT - the standard library's
   locking types only require lock() and unlock():       */
        Mutex m;
        {
                std::lock_guard lk{m};
                shared++;
        }
        std::scoped_lock lk{m1, m2};    /* deadlock-free
                                           ordering, free   */

/* --- a task --- */
class Task {
        TaskHandle_t h_ = nullptr;

        template<typename F>
        static void entry(void *p)
        {
                (*static_cast<F *>(p))();
                vTaskDelete(nullptr);
        }
public:
        template<typename F>
        Task(F&& f, const char *name, uint16_t stack,
             UBaseType_t prio)
        {
                /* the functor must outlive the task - store
                   it, or require a captureless lambda    */
                xTaskCreate(entry<std::decay_t<F>>, name,
                            stack, &f, prio, &h_);
        }
        ~Task() { if (h_) vTaskDelete(h_); }
};

/* --- a vendor HAL handle --- */
class SpiBus {
        SPI_HandleTypeDef h_{};
public:
        explicit SpiBus(SPI_TypeDef *inst)
        {
                h_.Instance = inst;
```

```
                h_.Init = { ... };
                if (HAL_SPI_Init(&h_) != HAL_OK)
                        panic("SPI init failed");
        }
        ~SpiBus() { HAL_SPI_DeInit(&h_); }

        SpiBus(const SpiBus&) = delete;

        [[nodiscard]] std::expected<void, HAL_StatusTypeDef>
        transfer(std::span<const uint8_t> tx,
                 std::span<uint8_t> rx)
        {
                auto st = HAL_SPI_TransmitReceive(&h_,
                        const_cast<uint8_t *>(tx.data()),
                        rx.data(),
                        static_cast<uint16_t>(tx.size()),
                        HAL_MAX_DELAY);
                if (st != HAL_OK) return std::unexpected(st);
                return {};
        }
};

/* THE PATTERN IN ONE SENTENCE: keep the C API exactly as it
   is, and add a thin class that gives it a destructor, a
   span-based interface, and an error type that cannot be
   ignored. The generated code is the same; the failure
   modes are gone.                                          */
```

The pitfalls

```
/* --- 1. C++ IS NOT A SUPERSET OF C. These differ: --- */
        void *p = malloc(n);            /* C: fine.
                                           C++: ERROR, needs a
                                           cast              */
        int f();                        /* C: unspecified
                                           args. C++: no args */
        char c[] = "x";  sizeof('a');   /* C: 4. C++: 1 */
        enum E { A }; int x = A;        /* both fine, but
                                           enum class differs */
        struct S { };                   /* C: size may be 0.
                                           C++: size 1      */
        /* and C++ has more keywords: class, new, delete,
           template, typename, this, operator, ...          */

/* --- 2. INITIALISATION ORDER ACROSS THE BOUNDARY --- */
/* A C++ static constructor may run before the C library it
   depends on is initialised (chapter 01). Use constinit,
   lazy initialisation, or explicit init functions.

/* --- 3. `restrict` IS NOT IN C++ --- */
        void f(int * restrict p);       /* C only */
        void f(int * __restrict p);     /* the compiler
                                           extension both
                                           accept           */

/* --- 4. VARIABLE-LENGTH ARRAYS ARE NOT IN C++ --- */
        void f(int n) { int a[n]; }     /* C99; a GNU
                                           extension in C++ */
/* Use std::array with a fixed bound, or alloca, or a static
   buffer.

/* --- 5. COMPOUND LITERALS DIFFER --- */
        f((struct S){1, 2});            /* C99. In C++ use a
                                           temporary:       */
        f(S{1, 2});

/* --- 6. DESIGNATED INITIALISERS --- */
        struct S s = { .b = 2, .a = 1 };  /* C: ANY order.
                                             C++20: must
                                             match
                                             DECLARATION
                                             order         */

/* --- 7. const AT FILE SCOPE --- */
        const int x = 1;        /* C: EXTERNAL linkage.
                                   C++: INTERNAL linkage    */
/* So a `const int` in a header is one object per translation
   unit in C++ and a duplicate-definition error in C. Use
   `inline constexpr` in C++17.

/* --- 8. STRICTER TYPE RULES generally --- */
        int *p = (void *)q;             /* needs a cast */
        enum E e = 1;                   /* needs a cast */
        char *s = "literal";            /* C++: ERROR, use
                                           const char *     */
/* Every one of these is C++ catching something C permitted,
   which is why step 1 of a migration (chapter 15) is
   valuable on its own.                                     */
```

Building mixed projects

```
# --- CMake ---
    project(firmware C CXX ASM)

    set(CMAKE_C_STANDARD 11)
    set(CMAKE_CXX_STANDARD 17)
    set(CMAKE_CXX_STANDARD_REQUIRED ON)
    set(CMAKE_CXX_EXTENSIONS OFF)

    add_executable(fw
        main.cpp driver.cpp        # C++
        vendor_hal.c startup.c     # C
        vectors.S)                 # assembly

    target_compile_options(fw PRIVATE
        $<$<COMPILE_LANGUAGE:CXX>:-fno-exceptions -fno-rtti>
        $<$<COMPILE_LANGUAGE:C>:-std=c11>
        -Wall -Wextra -Os)

# --- THE LINKER MUST BE THE C++ ONE ---
#   g++, not gcc - it links libstdc++ and, crucially, the
#   static-constructor machinery. Linking with gcc gives
#   undefined references to __cxa_* and silently skips
#   .init_array.
#   CMake does this automatically when any source is C++.

# --- CHECK WHAT LANGUAGE A SYMBOL HAS ---
    nm -C prog.o | grep myfunc
    #   `myfunc`           C linkage
    #   `myfunc(int)`      C++ linkage (demangled)

# --- AND THE COMMONEST LINK ERROR ---
#   "undefined reference to `driver_init'"
#   where driver_init is defined in a .c file
#   -> the C++ caller saw a declaration WITHOUT extern "C",
#      so it looked for the mangled name. Add the
#      #ifdef __cplusplus guard to the header.
#
#   THE REVERSE: a C file calling a C++ function that was not
#   declared extern "C" - same error, same cause.
```

How it shows up

Symptom	Cause
"undefined reference" to a function that clearly exists	Missing `extern "C"` — check with `nm -C`
Link error only when the caller is C++	Same, in the header
Undefined `__cxa_*` symbols	Linked with `gcc` instead of `g++`
Static constructors never run	Same, or `.init_array` not walked in startup code
Struct fields wrong across the boundary	Not standard layout — assert it
DMA into a struct corrupts it	Not trivially copyable — it has a constructor
Terminate inside a C library	An exception escaped a callback
Crash in a callback after an object was destroyed	Not unregistered in the destructor
A capturing lambda will not convert to a function pointer	By design — use the context pointer

Symptom	Cause
Duplicate symbol for a `const` in a header	Different linkage rules — use `inline constexpr`
C code compiles, the same code as C++ does not	Stricter rules — each one is finding something

Check yourself

- ✓ Explain what `extern "C"` does and does not do.
- ✓ Write a header usable from both languages.
- ✓ Expose a C++ class through an opaque-handle C API.
- ✓ Distinguish standard layout from trivially copyable and say when each matters.
- ✓ Add methods to a C struct without changing its layout.
- ✓ Give three ways to supply a C callback from C++, and the three rules for all of them.
- ✓ Wrap an RTOS mutex so `std::lock_guard` works with it.
- ✓ Name eight ways C++ differs from C in existing code.
- ✓ Explain why the linker must be `g++`.
- ✓ Diagnose an `extern "C"` link error using `nm`.

> **END OF THE C++ TRACK**
>
> Sixteen chapters: what C++ buys, what it costs, how to verify both, and a subset you can defend in review. Next: Rust, which takes the same goal — zero-cost abstraction over hardware — and reaches it by making the compiler prove memory safety rather than trusting discipline.

TRACK

Rust

The memory-safety model that is now in the Linux kernel, explained as a systems programmer needs it rather than as a tutorial.

17 chapters

Why Rust for systems programming

Rust's claim is specific: memory safety and data-race freedom, proved at compile time, with no garbage collector and no runtime cost. This chapter is about what that claim actually covers, what it does not, and why the Linux kernel accepted it.

The claim, precisely

```
IN SAFE RUST, the following are IMPOSSIBLE:

  use after free            dangling pointers
  double free               buffer overflow
  null pointer dereference  uninitialised memory reads
  data races                iterator invalidation

Not "unlikely", not "caught by tools" - REJECTED BY THE
COMPILER. The program does not build.

WHAT IS STILL POSSIBLE, and this matters:
  · memory LEAKS (safe, just wasteful)
  · deadlocks
  · integer overflow (panics in debug, wraps in release -
    both defined)
  · logic errors
  · panics, which abort or unwind
  · resource exhaustion
  · anything inside an `unsafe` block

THE COST: none at run time. No GC, no runtime, no hidden
allocation. Rust compiles through LLVM to the same
instructions C would (chapter 06 of the compilers track),
and often better - because its aliasing rules are stronger
than C's, so LLVM can optimise more aggressively.

WHERE THE COST IS PAID: at COMPILE TIME, in your head. The
borrow checker rejects programs that are actually correct
but that it cannot prove correct, and learning to write in
a shape it accepts takes weeks.
```

Why the industry moved

```
/* THE DATA, which is what changed minds:

   · MICROSOFT: ~70% of CVEs assigned each year are memory
     safety issues. Consistent across a decade.
   · GOOGLE (Chrome): ~70% of serious security bugs are
     memory safety.
   · GOOGLE (Android): memory-safety vulnerabilities fell
     from 76% of the total to 24% as the proportion of new
     code written in Rust and Java grew - WITHOUT rewriting
     the old code.
   · THE US GOVERNMENT (ONCD, 2024) recommended memory-safe
     languages for new critical software.

   THE ANDROID RESULT IS THE IMPORTANT ONE: the drop came
   from NEW code being memory-safe. Vulnerability density is
   highest in new code, so replacing the growth edge rather
   than rewriting the base is the effective strategy - which
   is exactly the position an existing C codebase is in.

/* AND THE COUNTER-ARGUMENTS, stated fairly:
      · the learning curve is real, and steeper than any
        mainstream language
      · compile times are slow
      · the embedded ecosystem is younger than C's by three
        decades
      · vendor SDKs, HALs and certified toolchains are C
      · safety-certification stories (ISO 26262, DO-178C) are
        emerging but not yet routine
      · `unsafe` is required for hardware access, so the
        guarantee is only as good as the audit of those
        blocks                                              */
```

Against C and C++

	C	C++	Rust
Memory safety	None	Opt-in, by discipline	**Guaranteed in safe code**
Data-race freedom	None	None	**Guaranteed in safe code**
Resource cleanup	Manual	RAII	RAII, and *enforced* — no copy of an owner
Null	Any pointer	Any pointer	`Option<&T>` — the check is the type
Error handling	Return codes	Exceptions or codes	`Result`, `#[must_use]`
Undefined behaviour	Extensive	Extensive	Only inside `unsafe`
Generics	Macros	Templates (duck-typed)	Traits — *constrained*, so errors are local
Build system	Make, CMake, …	Same	**Cargo**, uniform
Dependencies	System packages	Same, or vendored	crates.io, versioned
Testing	Third-party	Third-party	Built in: `#[test]`, doctests
Toolchain maturity	50 years	40 years	10 years
Compile speed	Fast	Slow	Slow
Embedded vendor support	Universal	Common	Growing

The mechanism, in one page

```
/* Three rules do almost all the work. Chapters 02-04 cover
   each properly; this is the shape.                        */

/* --- 1. OWNERSHIP: every value has exactly one owner --- */
        let s = String::from("hello");
        let t = s;              // MOVED. `s` is no longer
                                // usable.
        println!("{}", s);      // COMPILE ERROR

/* When the owner goes out of scope, the value is dropped.
   Exactly once, automatically, on every path. This is C++'s
   RAII with the copy-that-double-frees made impossible
   (C++ chapter 03).

/* --- 2. BORROWING: many readers XOR one writer --- */
        let mut v = vec![1, 2, 3];

        let a = &v;             // shared borrow
        let b = &v;             // another - fine
        println!("{:?} {:?}", a, b);

        let m = &mut v;         // exclusive borrow
        let n = &v;             // ERROR - cannot borrow while
                                // mutably borrowed

/* THIS ONE RULE eliminates data races AND iterator
   invalidation AND aliasing bugs, because all three are the
   same underlying problem: two references to one thing, one
   of them writing.

/* --- 3. LIFETIMES: a reference cannot outlive its
       referent --- */
        let r;
        {
                let x = 5;
                r = &x;
        }                       // x dropped
        println!("{}", r);      // ERROR - r outlives x

/* THE COMPILER TRACKS THIS ACROSS FUNCTION BOUNDARIES, which
   is what makes returning a reference safe.

/* AND THE PAYOFF, in one example - this C bug: */
        char *f(void) {
                char buf[64];
                sprintf(buf, "hi");
                return buf;             /* DANGLING */
        }
/* is, in Rust, not a bug you have to find. It does not
   compile.                                                 */
```

> **THE TRADE, STATED HONESTLY**
>
> The borrow checker rejects some programs that are correct. Learning Rust is largely learning which program *shapes* it can prove correct, and restructuring toward them. That is a real cost, paid up front, in exchange for a class of bug that never reaches production. Whether that trade is worth it depends on how expensive your bugs are — which for firmware, kernels and anything network-facing is usually "very".

Rust on embedded

```
/* THE PARTS THAT FIT EMBEDDED WELL:

   · no_std - the core language and `core` with no OS, no
     allocator, no runtime (chapter 15). Rust was designed for
     this from the start.
   · ZERO-COST ABSTRACTIONS, verified the same way as C++'s
     (C++ chapter 14) - and the compiler is LLVM, so the
     optimiser is the same one.
   · TYPE-STATE: encode hardware state in the type system, so
     "configure the pin before using it" is checked at
     compile time rather than documented.
   · No hidden allocation. If it allocates, you can see it.
   · Cargo, which after CMake and vendor IDEs is a relief.
   · Excellent cross-compilation: rustup target add
     thumbv7em-none-eabihf, and it works.

/* THE PARTS THAT DO NOT FIT YET:
   · vendor HALs and SDKs are C - so every project has an FFI
     layer (chapter 12), and that layer is `unsafe`
   · certified toolchains (Ferrocene exists; the ecosystem
     around it is young)
   · debugging and trace tooling is thinner than for C
   · binary size can be larger without care - though
     opt-level="z", LTO and panic=abort close most of the gap
   · the async story on bare metal is excellent but recent
     (Embassy - chapter 16)

/* WHERE IT IS ALREADY IN PRODUCTION:
     Android (Binder, Keystore, the Bluetooth stack, parts of
       the kernel)
     Windows (parts of the kernel and font parsing)
     the LINUX KERNEL (chapter 17)
     AWS Firecracker, Cloudflare's proxy, Discord, Dropbox
     Oxide Computer's firmware (Hubris - an entire embedded OS)
     Espressif, Nordic and STM32 HALs

/* THE HONEST RECOMMENDATION for a 2026 embedded team:
      · a NEW project on a well-supported chip: seriously
        consider it
      · an EXISTING C codebase: consider Rust for new
        components with a clean interface, not a rewrite
      · a project constrained by a vendor SDK or certification:
        C or C++, and revisit in two years
      · and LEARN IT REGARDLESS - the ideas improve how you
        write C++, and the kernel now accepts it            */
```

Getting started

```
# --- INSTALL ---
    curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
    rustup update
    rustup component add rust-src rustfmt clippy
    rustup target add aarch64-unknown-linux-gnu
    rustup target add thumbv7em-none-eabihf   # Cortex-M4F

# --- A HOSTED PROJECT ---
    cargo new hello
    cd hello
    cargo run
    cargo build --release
    cargo test
    cargo clippy -- -D warnings         # the linter. Use it.
    cargo fmt                           # one formatting
                                        # standard, no debate

# --- CROSS-COMPILE ---
    cargo build --target aarch64-unknown-linux-gnu --release

# --- THE FILES ---
    Cargo.toml          dependencies, metadata, profiles
    Cargo.lock          exact versions - COMMIT this for a
                        binary, not for a library
    src/main.rs         the binary crate root
    src/lib.rs          the library crate root

# --- A FIRST PROGRAM, showing the shape ---
fn main() {
    let data = vec![3, 1, 4, 1, 5, 9, 2, 6];

    let sum: i32 = data.iter().sum();
    let max = data.iter().max().expect("non-empty");

    let mut sorted = data.clone();
    sorted.sort();

    println!("sum={sum} max={max} sorted={sorted:?}");

    match find_first_even(&data) {
        Some(x) => println!("first even: {x}"),
        None    => println!("no even numbers"),
    }
}

fn find_first_even(v: &[i32]) -> Option<&i32> {
    v.iter().find(|&&x| x % 2 == 0)
}

/* THINGS TO NOTICE ALREADY:
     `&[i32]`      a SLICE - a pointer and a length, like
                   std::span (C++ chapter 08)
     Option<&i32>  "a reference or nothing" - and the compiler
                   forces you to handle both cases
     `match`       exhaustive: forget a case and it does not
                   compile
     no semicolon  on the last expression of a function - it
                   is the return value
     `{sum}`       inline format arguments                */
```

Reading Rust as a C programmer

```
    fn name(arg: Type) -> Ret { }       // function
    let x = 5;                          // immutable BY
                                        // DEFAULT
    let mut y = 5;                      // mutable
    const MAX: u32 = 100;               // compile-time
    static GLOBAL: u32 = 0;             // a static

    &T          a shared reference       (const T* that cannot
                                         be null)
    &mut T      an exclusive reference   (T* that cannot be
                                         null, and is the ONLY
                                         one)
    *const T    a raw pointer           (needs unsafe to
                                         deref)
    Box<T>      an owned heap value      (unique_ptr)
    Vec<T>      a growable array        (vector)
    &[T]        a slice                 (span)
    &str        a string slice          (string_view)
    String      an owned string
    Option<T>   Some(T) or None         (optional)
    Result<T,E> Ok(T) or Err(E)         (expected)

    struct S { a: u32 }                 // a struct
    enum E { A, B(u32), C { x: u8 } }   // a TAGGED UNION -
                                        // variants carry data
    impl S { fn method(&self) {} }      // methods, defined
                                        // separately
    trait T { fn f(&self); }            // an interface
    impl T for S { fn f(&self) {} }     // implementing it

    match x {                           // exhaustive
        0 => a(),
        1..=9 => b(),
        _ => c(),
    }

    if let Some(v) = opt { }            // match one pattern
    while let Some(v) = it.next() { }

    v.iter().map(f).filter(g).sum()     // iterator chains -
                                        // lazy, fused, zero
                                        // cost

/* THE TWO THINGS THAT LOOK STRANGEST AT FIRST:
     1. `impl` blocks separate from the struct - which means a
        type can have methods added by anyone who imports the
        trait
     2. no `return` on the last expression - the value of the
        final expression IS the return value. `return` exists
        for early exits.                                    */
```

How it shows up

Symptom	Meaning
"borrow of moved value"	Ownership transferred — clone, borrow, or restructure (ch. 02)
"cannot borrow as mutable more than once"	The aliasing rule (ch. 03)

Symptom	Meaning
"does not live long enough"	A reference outliving its referent (ch. 04)
"missing lifetime specifier"	The compiler cannot infer which input a returned reference came from
"non-exhaustive patterns"	A `match` arm is missing — the feature working
"cannot find function" for a trait method	The trait is not in scope — `use` it
Fighting the borrow checker constantly	Normal for the first weeks; usually the design wants restructuring
Binary much larger than the C equivalent	Panic machinery and formatting — see chapter 15
Slow compile times	Real. Use `cargo check` while iterating
Everything requires `unsafe`	You are writing C in Rust — find the safe abstraction

Check yourself

- ✓ State exactly what safe Rust guarantees, and five things it does not.
- ✓ Explain why the Android result is the important data point.
- ✓ Compare Rust with C and C++ across ten dimensions.
- ✓ State the three rules — ownership, borrowing, lifetimes — and what each eliminates.
- ✓ Explain the trade the borrow checker imposes.
- ✓ Name five things that fit embedded well and four that do not yet.
- ✓ Give a defensible recommendation for three different project situations.
- ✓ Install the toolchain and cross-compile for an ARM target.
- ✓ Read a page of Rust and map each construct to its C or C++ equivalent.
- ✓ Explain what `Option<&T>` replaces and why it is safer.

Ownership

One value, one owner, dropped exactly once. This is C++'s RAII with the loopholes closed — and it is the rule from which use-after-free, double-free and leaks all become impossible rather than merely unlikely.

The three rules

```
1. Every value has exactly ONE OWNER.
2. There can be only one owner at a time.
3. When the owner goes out of scope, the value is DROPPED.

{
     let s = String::from("hello");  // s owns the heap
                                     // buffer
     use(&s);
}                                    // s dropped here.
                                     // The buffer is
                                     // freed. Exactly
                                     // once, on every
                                     // path.

COMPARE C++ (chapter 03 of that track):
     std::string s = "hello";        // same
     std::string t = s;              // a COPY - two
                                     // buffers
COMPARE RUST:
     let s = String::from("hello");
     let t = s;                      // a MOVE - one
                                     // buffer, and `s` is
                                     // dead

THE DIFFERENCE IS THE DEFAULT. C++ copies unless you say
move; Rust moves unless the type opts into Copy. So the
accidental deep copy and the accidental shallow copy - the
two classic C++ bugs - are both eliminated by the same
choice.
```

Moves

```
    let s1 = String::from("hello");
    let s2 = s1;                        // MOVE
    println!("{s1}");                   // ERROR:
                                        // borrow of moved value

/* THE ERROR MESSAGE, which is genuinely helpful:

     error[E0382]: borrow of moved value: `s1`
       --> src/main.rs:3:20
        |
      1 |     let s1 = String::from("hello");
        |         -- move occurs because `s1` has type
        |            `String`, which does not implement the
        |            `Copy` trait
      2 |     let s2 = s1;
        |              -- value moved here
      3 |     println!("{s1}");
        |                ^^^^ value borrowed here after move
        |
      help: consider cloning the value
        |
      2 |     let s2 = s1.clone();

/* MOVES HAPPEN ON:
     assignment            let b = a;
     passing by value      f(a);
     returning             return a;
     pushing into a
       collection          v.push(a);
     pattern matching by
       value               match a { ... }

/* WHAT A MOVE COSTS: a memcpy of the value itself - for a
   String that is three words (pointer, length, capacity).
   THE HEAP BUFFER IS NOT COPIED. And the compiler usually
   elides even the memcpy.

/* THE MOVED-FROM VALUE IS GONE, not "valid but unspecified"
   as in C++ - the compiler prevents you from touching it at
   all. That is strictly stronger.                          */

/* PASSING TO A FUNCTION MOVES: */
    fn consume(s: String) { }           // takes OWNERSHIP

    let s = String::from("hi");
    consume(s);
    println!("{s}");                    // ERROR

/* SO IF THE CALLER NEEDS IT AFTERWARDS, borrow instead
   (chapter 03): */
    fn inspect(s: &String) { }          // BORROWS
    inspect(&s);
    println!("{s}");                    // fine

/* OR RETURN IT BACK - the pattern before borrowing existed,
   and still occasionally useful: */
    fn transform(mut s: String) -> String { s.push('!'); s }
    let s = transform(s);
```

Copy types

```
    let x = 5;
    let y = x;                          // COPIED, not moved
    println!("{x}");                    // fine - x is still
                                        // valid

/* TYPES THAT IMPLEMENT `Copy` are duplicated bitwise instead
   of moved. The rule: a type may be Copy only if every part
   of it is Copy and it has no Drop implementation.

   COPY TYPES:
     all integers, floats, bool, char
     &T (shared references)
     raw pointers
     arrays and tuples of Copy types
     any struct you derive Copy on

   NOT COPY:
     String, Vec<T>, Box<T>      - they own a resource
     &mut T                      - copying it would create
                                   two exclusive references
     anything with a Drop impl   - copying would double-drop

/* DERIVING IT: */
    #[derive(Copy, Clone, Debug, PartialEq)]
    struct Point { x: i32, y: i32 }

    let p = Point { x: 1, y: 2 };
    let q = p;                          // copied
    println!("{p:?} {q:?}");            // both usable

/* Copy REQUIRES Clone - Copy is "Clone, and it is a bitwise
   memcpy". You always derive both together.

/* WHEN TO DERIVE Copy: for small, plain-data types - a
   register value, a coordinate, an enum of states, a message
   header. It makes them behave like C values, which is what
   you want.
   WHEN NOT TO: anything large (a copy becomes implicit and
   invisible), or anything that owns a resource.

/* AND `Clone` FOR EXPLICIT DEEP COPIES: */
    let s2 = s1.clone();                // explicit,
                                        // potentially
                                        // expensive, VISIBLE
/* The visibility is the point: in C++ a deep copy can happen
   in a function call you did not look at. In Rust it says
   `.clone()`.                                              */
```

THE `.CLONE()` ESCAPE HATCH

When the borrow checker refuses, cloning usually makes it compile. That is fine while learning — but treat a `.clone()` you added to silence an error as a note to revisit. Frequently the real fix is to borrow instead of own, or to restructure so one function owns the data and the others take references. Reaching for `clone` reflexively produces Rust that works and performs like a garbage-collected language.

Drop

```
    struct Uart { base: usize }

    impl Drop for Uart {
        fn drop(&mut self) {
            // called automatically when the owner goes out of
            // scope
            unsafe { write_reg(self.base + CTRL, 0); }
        }
    }

    {
        let uart = Uart { base: 0x4000_0000 };
        uart.write(b"hello");
    }   // drop() runs HERE, on every path

/* DROP ORDER:
     · local variables in REVERSE declaration order
     · struct fields in DECLARATION order
     · the outer value's drop() runs BEFORE its fields are
       dropped

    struct Device {
        clock: ClockGate,       // dropped SECOND
        uart:  Uart,            // dropped FIRST
    }
/* Wait - fields drop in declaration order, so `clock` drops
   first. If the UART needs the clock enabled to be shut down
   cleanly, declare it the other way round:              */
    struct Device {
        uart:  Uart,            // dropped first - good
        clock: ClockGate,       // dropped second
    }

/* YOU CANNOT CALL drop() DIRECTLY: */
    uart.drop();                          // ERROR
    drop(uart);                           // this instead - it
                                          // MOVES the value
                                          // into a function
                                          // that does nothing,
                                          // so it drops there

/* AND `std::mem::forget` LEAKS DELIBERATELY: */
    std::mem::forget(uart);               // drop is NOT called
/* Which is SAFE Rust - leaking is not a memory-safety
   violation. It matters for FFI: when you hand ownership to
   C, you forget the Rust value so it is not dropped twice
   (chapter 12).

/* A TYPE WITH Drop CANNOT BE Copy - which the compiler
   enforces, because a bitwise copy followed by two drops is
   exactly the C++ double-free (C++ chapter 03).        */
```

Ownership of hardware

```
/* THE EMBEDDED APPLICATION: make a peripheral a value that
   can only be owned once. Then "two drivers configured the
   same UART" is a compile error.                          */

    pub struct Uart0 { _private: () }   // cannot be
                                        // constructed outside
                                        // this module

    pub struct Peripherals {
        pub uart0: Uart0,
        pub spi1:  Spi1,
        pub gpio:  Gpio,
    }

    static TAKEN: AtomicBool = AtomicBool::new(false);

    impl Peripherals {
        pub fn take() -> Option<Self> {
            if TAKEN.swap(true, Ordering::SeqCst) {
                None                    // ALREADY TAKEN
            } else {
                Some(unsafe { Self::steal() })
            }
        }
    }

    fn main() -> ! {
        let p = Peripherals::take().unwrap();

        let uart = UartDriver::new(p.uart0);   // MOVES the
                                               // token in
        // p.uart0 is now gone - nobody else can construct a
        // second UartDriver for it, at all.
        loop { }
    }

/* THIS IS THE SINGLETON PATTERN the whole embedded Rust
   ecosystem is built on (chapter 16). The peripheral token
   is a zero-sized type - it occupies no memory and generates
   no code - and moving it expresses exclusive access in a way
   the compiler checks.

/* AND TYPE-STATE, taking it further: encode the CONFIGURATION
   in the type, so an unconfigured pin cannot be used.    */

    struct Pin<MODE> { pin: u8, _mode: PhantomData<MODE> }
    struct Input;
    struct Output;

    impl Pin<Input> {
        fn into_output(self) -> Pin<Output> {
            configure_as_output(self.pin);
            Pin { pin: self.pin, _mode: PhantomData }
        }
        fn read(&self) -> bool { read_pin(self.pin) }
    }

    impl Pin<Output> {
        fn set_high(&mut self) { write_pin(self.pin, true); }
        // NO read() - an output pin cannot be read
```

```
    }

    let pin: Pin<Input> = ...;
    pin.set_high();             // COMPILE ERROR
    let mut pin = pin.into_output();
    pin.set_high();             // fine

/* PhantomData IS ZERO-SIZED, so Pin<Input> and Pin<Output>
   are both one byte and generate identical code. The
   distinction exists only at compile time - which is the
   definition of a zero-cost abstraction (C++ chapter 14, the
   same idea).                                              */
```

Scopes and shadowing

```
    let x = 5;
    let x = x + 1;              // SHADOWING - a new variable,
    let x = x * 2;              // same name. Not mutation.
    println!("{x}");            // 12

/* SHADOWING lets you change the TYPE, which mutation cannot:
*/
    let input = "42";                   // &str
    let input: u32 = input.parse()?;    // now u32

/* IDIOMATIC AND USEFUL - it avoids `input_str` and
   `input_num`. Distinct from `mut`, which keeps one variable
   and changes its value.

/* BLOCKS ARE EXPRESSIONS: */
    let y = {
        let a = compute();
        a * 2                   // no semicolon = the block's
                                // value
    };

/* WHICH IS WHY: */
    let max = if a > b { a } else { b };
    let msg = match code {
        0 => "ok",
        _ => "error",
    };
/* `if` and `match` are expressions, so there is no ternary
   operator and no need for one.

/* AND THE SEMICOLON RULE, which trips everyone once: */
    fn f() -> i32 { 5 }         // returns 5
    fn g() -> i32 { 5; }        // ERROR - `5;` is a
                                // STATEMENT, and the block's
                                // value is () - the unit
                                // type
```

A mental model

```
WHEN THE BORROW CHECKER REJECTS SOMETHING, ask in order:

1. Does this function need to OWN the value, or only READ
   it?
      -> if only read, take &T (chapter 03)

2. Does it need to MODIFY it in place?
      -> take &mut T

3. Does it need to KEEP it beyond the call?
      -> take T by value; the caller gives it up

4. Do TWO parts of the program genuinely need to own it?
      -> they cannot. Either
           · one owns and the other borrows, or
           · restructure so there is a clear owner, or
           · use Rc/Arc for genuinely shared ownership
             (chapter 09) - which is the C++ shared_ptr
             answer, with the same caveats

5. Am I fighting a design problem rather than the
   compiler?
      -> frequently yes. "Who owns this?" is a question C
         lets you avoid answering and Rust does not, and
         answering it usually improves the design.

THE COMMONEST BEGINNER SHAPE that fights the checker:
a struct holding references to other parts of the same
structure. Rust makes self-referential structures hard on
purpose, because they are exactly what goes wrong when
something moves. Use indices into an arena instead - which
is the same advice as the generational-handle pattern in
C++ chapter 10.
```

How it shows up

Error	Meaning and fix
"use of moved value"	You gave it away. Borrow, clone, or return it back
"cannot move out of borrowed content"	You have `&T` and tried to take ownership — clone, or take `T`
"cannot move out of index"	Moving one element out of a `Vec` — use `swap_remove`, `take`, or clone
"cannot move out of `self`"	The method takes `&self`; it needs `self`
"the trait `Copy` is not implemented"	The type owns a resource — that is why it moves
"cannot implement Copy for a type with Drop"	Correct — that combination is a double free
Value dropped earlier than expected	Field or variable declaration order
Resource not released	`mem::forget`, an `Rc` cycle, or an early `drop` you did not intend
Peripheral `take()` returns `None`	Something already took it — working as designed
A function returns `()` unexpectedly	A stray semicolon on the last expression

Check yourself

- ✓ State the three ownership rules and what each prevents.
- ✓ Explain why move-by-default eliminates two distinct C++ bugs.
- ✓ List the operations that move a value.
- ✓ Say which types are `Copy` and which cannot be, with reasons.
- ✓ Explain when to derive `Copy` and when not to.
- ✓ Implement `Drop` and state the drop order rules.
- ✓ Explain why `Drop` and `Copy` are mutually exclusive.
- ✓ Write a peripheral singleton with a `take()` and explain what it guarantees.
- ✓ Write a type-state pin and explain why it costs nothing.
- ✓ Work the five-question mental model on a borrow-checker error.

Borrowing and references

Access without ownership, governed by one rule: many readers *or* one writer, never both. That single constraint eliminates data races, iterator invalidation and aliasing bugs simultaneously — because all three are the same problem.

The rule

```
AT ANY GIVEN TIME, for any given value, you may have
EITHER:

  · any number of SHARED references    &T
  · exactly one EXCLUSIVE reference    &mut T

but never both, and never two exclusive ones.

AND every reference must be VALID for its whole lifetime -
it cannot outlive what it points to (chapter 04).

     let mut v = vec![1, 2, 3];

     let a = &v;             // shared
     let b = &v;             // another shared - FINE
     println!("{a:?} {b:?}");

     let m = &mut v;         // exclusive - fine, because
                             // a and b are no longer USED
     m.push(4);

     println!("{a:?}");      // ERROR if this came after
                             // the &mut

NON-LEXICAL LIFETIMES: a borrow ends at its LAST USE, not
at the end of the scope. So this compiles:

     let a = &v;
     println!("{a:?}");      // a's last use
     let m = &mut v;         // fine - a is dead
     m.push(4);

which is why modern Rust is far less irritating than the
2015-era descriptions suggest.

THE NAMES: `&T` is often called "immutable" and `&mut T`
"mutable", but SHARED and EXCLUSIVE are the accurate
words - the real distinction is aliasing, not mutation.
Cell and Mutex allow mutation through &T (chapter 09),
which is only confusing if you learned the wrong names.
```

What it eliminates

```
/* --- 1. ITERATOR INVALIDATION --- */
    let mut v = vec![1, 2, 3];
    for x in &v {               // v is borrowed SHARED here
        v.push(*x);             // ERROR - push needs &mut
    }
/* THE C++ EQUIVALENT compiles, runs, and corrupts memory
   when the vector reallocates (C++ chapter 08). Here it is a
   compile error, and the error message says exactly why.

/* --- 2. USE AFTER FREE --- */
    let r;
    {
        let x = 5;
        r = &x;
    }
    println!("{r}");            // ERROR: `x` does not live
                                // long enough

/* --- 3. DATA RACES --- */
    let mut data = vec![1, 2, 3];
    thread::spawn(|| data.push(4));     // ERROR - the closure
    data.push(5);                       // would need &mut
                                        // while main also
                                        // has it
/* The SAME RULE that prevents iterator invalidation prevents
   data races, because they are the same shape: two paths to
   one value, one of them writing.

/* --- 4. ALIASING BUGS --- */
    fn f(a: &mut i32, b: &mut i32) { *a = 1; *b = 2; }
    let mut x = 0;
    f(&mut x, &mut x);          // ERROR - two exclusive
                                // borrows
/* In C this is legal, and the compiler must assume a and b
   may alias - which prevents optimisation. Rust's guarantee
   is why rustc emits LLVM's `noalias` on &mut parameters,
   and why Rust sometimes optimises BETTER than the
   equivalent C.                                          */
```

Borrowing in functions

```
    fn read(v: &Vec<i32>) -> i32 { v[0] }       // shared
    fn modify(v: &mut Vec<i32>) { v.push(4); }  // exclusive
    fn consume(v: Vec<i32>) { }                 // takes
                                                // ownership

    let mut v = vec![1, 2, 3];
    read(&v);
    modify(&mut v);
    println!("{v:?}");          // still usable
    consume(v);
    println!("{v:?}");          // ERROR - moved

/* THE PARAMETER RULE, and it is the whole design decision:
     need to READ it              -> &T
     need to MODIFY it in place   -> &mut T
     need to KEEP it              -> T
   Exactly the C++ const-ref / ref / value decision
   (C++ chapter 04), but enforced.

/* PREFER SLICES TO CONCRETE TYPES: */
    fn sum(v: &Vec<i32>) -> i32 { ... }         // narrow
    fn sum(v: &[i32])    -> i32 { ... }         // BETTER

    sum(&vec);                  // Vec derefs to a slice
    sum(&array);                // arrays too
    sum(&vec[1..3]);            // and sub-slices

    fn greet(s: &String) { }    // narrow
    fn greet(s: &str)    { }    // BETTER - accepts String,
                                // &str and literals
/* This is exactly the std::span / std::string_view advice
   from C++ chapter 08, and for the same reason.

/* METHODS take self in one of four ways: */
    impl Buffer {
        fn len(&self) -> usize { ... }          // read
        fn push(&mut self, b: u8) { ... }       // modify
        fn into_vec(self) -> Vec<u8> { ... }    // CONSUMES
        fn new() -> Self { ... }                // no self -
                                                // an
                                                // associated
                                                // function
    }
/* `into_*` by convention means "consumes self". `as_*` means
   a cheap borrowed view. `to_*` means an expensive
   conversion producing an owned value. Following those
   conventions makes an API readable at a glance.        */
```

THE MUTABILITY IS ON THE BINDING, NOT THE TYPE

```
let v = vec![1];        // immutable binding
v.push(2);              // ERROR
let mut v = vec![1];    // mutable binding
v.push(2);              // fine

fn f(v: &mut Vec<i32>)  // an exclusive reference
let mut r = &v;         // a MUTABLE BINDING holding a
                        // SHARED reference - r can be
                        // reassigned, *r cannot be written
```

`mut` in a `let` and `mut` in `&mut` are different things, and reading them as one is a common early confusion.

Working with the checker

```
/* --- 1. TWO FIELDS OF ONE STRUCT: fine --- */
    struct S { a: Vec<i32>, b: Vec<i32> }
    let ra = &mut s.a;
    let rb = &mut s.b;          // FINE - disjoint fields
/* The compiler tracks fields separately. But NOT through a
   method: */
    let ra = s.get_a_mut();     // borrows ALL of s
    let rb = s.get_b_mut();     // ERROR
/* Fix by having one method return both, or by borrowing the
   fields directly.

/* --- 2. SPLITTING A SLICE --- */
    let (left, right) = v.split_at_mut(mid);
    // two &mut into DISJOINT halves - the standard library
    // provides this because it needs unsafe internally, and
    // it is proved correct once so you never need to.

    for chunk in v.chunks_mut(64) { ... }
    let [a, b] = v.get_disjoint_mut([0, 5]).unwrap();  // 1.86

/* --- 3. A LOOP THAT MODIFIES WHAT IT ITERATES --- */
    // WRONG:
    for x in &v { if bad(x) { v.remove(...); } }
    // RIGHT:
    v.retain(|x| !bad(x));
    // or collect the indices first, or iterate mutably:
    for x in &mut v { *x += 1; }
    // or drain:
    let removed: Vec<_> = v.drain(..).filter(keep).collect();

/* --- 4. RETURNING A REFERENCE INTO A LOCAL: impossible --- */
    fn f() -> &str {
        let s = String::from("hi");
        &s                      // ERROR
    }
    fn f() -> String { String::from("hi") }     // return the
                                                // value

/* --- 5. THE SELF-REFERENTIAL STRUCT: do not --- */
    struct Bad<'a> { data: Vec<u8>, slice: &'a [u8] }
/* If `slice` points into `data`, moving the struct
   invalidates it. Rust makes this hard on purpose. Use
   INDICES instead:                                    */
    struct Good { data: Vec<u8>, range: Range<usize> }
/* which is the arena/handle pattern again.

/* --- 6. WHEN YOU GENUINELY NEED SHARED MUTATION --- */
    RefCell<T>      runtime-checked borrows, single-threaded
    Cell<T>         get/set of Copy values, no borrows at all
    Mutex<T>        across threads
    RwLock<T>       many readers or one writer, across
                    threads
/* All four are "interior mutability" - mutation through &T,
   with the aliasing rule enforced at RUN TIME instead of
   compile time (chapter 09).                          */
```

Reborrowing

```
/* A `&mut` can be temporarily lent out. The compiler does
   this implicitly, and knowing it explains several
   otherwise-confusing errors.                              */

    fn f(v: &mut Vec<i32>) {
        g(v);                   // implicitly REBORROWS:
                                // g(&mut *v)
        v.push(1);              // v is usable again after g
                                // returns
    }
    fn g(v: &mut Vec<i32>) { }

/* WITHOUT reborrowing this would MOVE the &mut into g and v
   would be unusable afterwards - which is what happens when
   the type is not obviously a reference:                */

    fn h<T>(x: T) { }
    fn f(v: &mut Vec<i32>) {
        h(v);                   // MOVES - T is inferred as
                                // &mut Vec, no reborrow
        v.push(1);              // ERROR
    }
    // fix: h(&mut *v);         // explicit reborrow

/* AND IN A LOOP, the classic: */
    let mut current = &mut head;
    loop {
        current = &mut current.next;     // moves out of
                                         // current
    }
/* Linked-list traversal with &mut is genuinely awkward, and
   is one of the shapes where an index-based arena is simply
   better in Rust.                                          */
```

The optimisation payoff

```
/* Rust's aliasing guarantee is stronger than C's, and LLVM
   uses it.                                                  */

    // C: the compiler CANNOT assume a and b are distinct
    void add(int *a, const int *b, int n) {
            for (int i = 0; i < n; i++)
                    a[i] += b[i];   // must reload b[i] in
                                    // case a[i] aliased it
    }
    // which is why `restrict` exists (compilers chapter 07).

    // Rust: &mut is UNIQUE by construction
    fn add(a: &mut [i32], b: &[i32]) {
        for i in 0..a.len().min(b.len()) {
            a[i] += b[i];
        }
    }
/* rustc emits `noalias` on the &mut parameter, so LLVM knows
   they are disjoint and vectorises without a runtime alias
   check - the same code C gets only with `restrict`, and
   without the risk of the promise being false.

/* CHECK IT: */
    cargo asm --rust my_crate::add
    # or paste into godbolt with the Rust compiler selected

/* THE BOUNDS CHECK QUESTION, which everyone asks:
     a[i] IS bounds-checked. But:
       · the check is a compare and a branch, predicted
         perfectly
       · LLVM removes it whenever it can prove the index is in
         range - which iterators always can
       · so idiomatic Rust (iterators, chapter 08) usually has
         NO bounds checks at all:                          */
    for (x, y) in a.iter_mut().zip(b.iter()) { *x += *y; }
/* -> no bounds checks, vectorised, identical to the C with
   restrict.

   THE RULE: write iterator chains, not index loops, and the
   bounds-check cost disappears along with the aliasing
   question.                                                 */
```

How it shows up

Error	Meaning and fix
"cannot borrow as mutable because it is also borrowed as immutable"	A shared borrow is still live — shorten it, or restructure
"cannot borrow as mutable more than once"	Two `&mut` — split the data, or use `split_at_mut`
"cannot borrow as mutable, not declared as mutable"	Missing `mut` on the `let`
"cannot assign to `*x` which is behind a `&` reference"	You have a shared reference; you need `&mut`
"cannot move out of borrowed content"	Clone, or take ownership instead of borrowing
"does not live long enough"	The reference outlives its referent (ch. 04)

Error	Meaning and fix
Two disjoint fields refuse to borrow	A method borrowed all of `self` — borrow the fields directly
A `&mut` becomes unusable after a call	It was moved, not reborrowed — write `&mut *x`
Linked-list traversal will not compile	Expected — use indices into an arena
Index loop is slower than expected	Bounds checks — use iterators

Check yourself

- ✓ State the borrowing rule using the words shared and exclusive.
- ✓ Explain non-lexical lifetimes and why they matter in practice.
- ✓ Show how one rule prevents iterator invalidation, use-after-free and data races.
- ✓ Give the parameter-passing rule for the three reference kinds.
- ✓ Explain why `&[T]` and `&str` are better parameters than `&Vec<T>` and `&String`.
- ✓ Distinguish `mut` on a binding from `&mut`.
- ✓ Give six techniques for restructuring code the checker rejects.
- ✓ Explain reborrowing and when it does not happen implicitly.
- ✓ Explain why `&mut` lets LLVM optimise better than C.
- ✓ Explain when bounds checks cost nothing, and how to write code that way.

Lifetimes

Lifetimes are how the compiler proves a reference never outlives what it points to. Most of the time they are inferred and invisible; when you must write them, they are describing a relationship you already understood implicitly.

What a lifetime is

```
A lifetime is a REGION OF CODE for which a reference is
valid. It is not a runtime value - it exists only during
type checking, and generates no code whatsoever.

     {
         let r;                       // ---------+-- 'a
         {                            //          |
             let x = 5;               // -+-- 'b  |
             r = &x;                  //  |       |
         }                            // -+       |
         println!("{r}");             //          |
     }                                // ---------+

`r` has lifetime 'a; `x` has lifetime 'b. The reference
`&x` cannot outlive 'b, but it is used within 'a. Since
'b is SHORTER than 'a, this is rejected:

     error[E0597]: `x` does not live long enough

THE COMPILER IS ANSWERING ONE QUESTION: for every
reference, is the thing it points to still alive at every
point where the reference is used? That is the whole
feature.

AND THE SYNTAX, when you must write it:
     &'a T           a shared reference living at least 'a
     &'a mut T       an exclusive one
     'static         lives for the whole program
```

Elision — why you rarely write them

```
/* Three rules let the compiler infer lifetimes in almost
   every function signature.                                */

   1. Each elided lifetime in a PARAMETER gets its own
      distinct lifetime.
   2. If there is EXACTLY ONE input lifetime, it is assigned
      to all elided output lifetimes.
   3. If one of the parameters is `&self` or `&mut self`, ITS
      lifetime is assigned to all elided output lifetimes.

/* SO THESE ARE ALL FINE without annotation: */

    fn first(s: &str) -> &str { &s[..1] }
    // rule 2: one input, so the output borrows from it

    fn len(s: &str) -> usize { s.len() }
    // no output reference - nothing to infer

    impl Buffer {
        fn as_slice(&self) -> &[u8] { &self.data }
        // rule 3: the output borrows from self
    }

/* AND THIS ONE IS NOT, which is the case you will meet: */

    fn longest(a: &str, b: &str) -> &str {      // ERROR
        if a.len() > b.len() { a } else { b }
    }
    // error[E0106]: missing lifetime specifier
    //   this function's return type contains a borrowed
    //   value, but the signature does not say whether it is
    //   borrowed from `a` or `b`

/* Rule 1 gives a and b DIFFERENT lifetimes, and rule 2 does
   not apply because there are two. So you must say: */

    fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
        if a.len() > b.len() { a } else { b }
    }

/* WHAT THAT SIGNATURE MEANS: the returned reference is valid
   for as long as BOTH inputs are - i.e. for the shorter of
   the two. It is not choosing a lifetime; it is stating a
   CONSTRAINT the caller must satisfy.

/* AND IF ONLY ONE INPUT IS ACTUALLY BORROWED FROM, say so -
   it is less restrictive for the caller: */
    fn prefix<'a>(s: &'a str, sep: &str) -> &'a str {
        s.split(sep).next().unwrap_or(s)
    }
    // `sep` can now be a shorter-lived temporary          */
```

> **A LIFETIME ANNOTATION DOES NOT CHANGE ANYTHING AT RUN TIME**
>
> It does not extend a value's life, allocate anything, or emit code. It is a *claim about a relationship* that the compiler then verifies. Writing `<'a>` never fixes a problem by itself — it either states a relationship that is true, and the code compiles, or one that is false, and the error moves to the call site.

Lifetimes in structs

```
/* A struct holding a reference must declare the lifetime,
   because the struct cannot outlive what it borrows.     */

    struct Parser<'a> {
        input: &'a str,
        pos: usize,
    }

    impl<'a> Parser<'a> {
        fn new(input: &'a str) -> Self {
            Parser { input, pos: 0 }
        }

        fn next_token(&mut self) -> Option<&'a str> {
            // returns a slice of the ORIGINAL input, not of
            // self - so it lives as long as 'a, and the
            // caller may hold it after calling next_token
            // again
            ...
        }
    }

    let text = String::from("a b c");
    let mut p = Parser::new(&text);
    let t = p.next_token();
    // drop(text);              // ERROR - p and t borrow it

/* NOTE THE DIFFERENCE between these two: */
    fn next_a(&mut self) -> Option<&'a str>    // borrows the
                                               // INPUT
    fn next_b(&mut self) -> Option<&str>       // elision
                                               // rule 3:
                                               // borrows SELF
/* The second one keeps `self` borrowed for as long as the
   result is held, so you cannot call it twice and keep both
   results. The first can. Choosing correctly is the main
   reason to write lifetimes explicitly on methods.

/* AND THE GENERAL ADVICE FOR EMBEDDED: a struct holding
   references is often better as a struct holding INDICES or
   owning its data. Borrowing structs are excellent for
   parsers and views over a buffer; they are awkward for
   long-lived state.                                       */
```

'static

```
/* 'static means "valid for the entire program". Two distinct
   things use it, and confusing them causes real trouble. */

/* --- 1. &'static T - a reference that lives forever --- */
    let s: &'static str = "hello";      // string literals are
                                        // in .rodata
    static TABLE: [u8; 256] = [0; 256];
    let t: &'static [u8] = &TABLE;

/* On embedded this is common and useful: */
    static mut BUFFER: [u8; 1024] = [0; 1024];
    // `static mut` requires unsafe to access - use a
    // StaticCell or a Mutex<RefCell<..>> instead
    // (chapter 15)

/* --- 2. T: 'static - a TYPE BOUND --- */
    fn spawn<F: FnOnce() + Send + 'static>(f: F);

/* This does NOT mean "lives forever". It means "contains no
   references with a lifetime shorter than 'static" - i.e. it
   either owns all its data, or only holds &'static
   references.

   String is 'static. i32 is 'static. Vec<String> is 'static.
   &'a str is NOT 'static unless 'a is.

   THAT IS WHY thread::spawn requires it: the thread may
   outlive the spawning scope, so its closure must not borrow
   anything local. Scoped threads (chapter 13) lift the
   requirement by guaranteeing the join happens first.

/* THE ANTI-PATTERN: adding `+ 'static` to silence an error.
   It usually pushes the problem to the caller, who then has
   to clone or leak. Work out what the real lifetime
   relationship is instead.                              */
```

Bounds and subtyping

```
/* --- LIFETIME BOUNDS --- */
    fn f<'a, 'b: 'a>(x: &'a str, y: &'b str) -> &'a str
/* `'b: 'a` reads "'b OUTLIVES 'a" - 'b is at least as long.
   So a &'b can be used where a &'a is expected.

    struct Wrapper<'a, T: 'a> { r: &'a T }
/* `T: 'a` means T contains no references shorter than 'a.
   Since Rust 2018 this is inferred in most positions and you
   rarely write it.

/* --- SUBTYPING: a longer lifetime coerces to a shorter --- */
    let long = String::from("x");
    {
        let r: &str = &long;    // &'long coerces to a shorter
                                // lifetime here
    }
/* This is why passing a &'static where &'a is wanted always
   works.

/* --- VARIANCE, which you will meet in error messages --- */
    &'a T           COVARIANT in 'a and in T
                    (a longer 'a works where a shorter is
                     wanted)
    &'a mut T       covariant in 'a, INVARIANT in T
    Cell<T>         invariant in T
    fn(T) -> U      CONTRAVARIANT in T, covariant in U

/* WHY &mut T IS INVARIANT IN T: if it were covariant you
   could write a shorter-lived reference through a
   &mut &'static T, creating a dangling &'static. The
   compiler forbids it, and the resulting error mentions
   "lifetime may not live long enough" in a way that only
   makes sense once you know variance exists.

/* YOU DO NOT NEED TO MEMORISE THIS. You need to recognise
   that an error about a &mut of a reference type is probably
   variance, and that the fix is usually to restructure
   rather than to add annotations.                       */
```

Practical patterns

```
/* --- 1. A ZERO-COPY PARSER, the best use of lifetimes --- */
    struct Packet<'a> {
        header: &'a [u8; 8],
        payload: &'a [u8],
    }

    fn parse(buf: &[u8]) -> Option<Packet<'_>> {
        if buf.len() < 8 { return None; }
        let (h, rest) = buf.split_at(8);
        Some(Packet {
            header: h.try_into().ok()?,
            payload: rest,
        })
    }
/* NO ALLOCATION, no copying - the packet is a VIEW into the
   receive buffer, and the compiler guarantees the buffer
   outlives it. That is a genuinely better answer than either
   copying (C++ safe version) or a raw pointer (C version).

   `'_` is the ANONYMOUS lifetime - "infer it, but I
   acknowledge one exists". Prefer it to omitting the
   parameter entirely; `-> Option<Packet>` is deprecated
   style for exactly this reason.

/* --- 2. A BUILDER THAT BORROWS --- */
    struct Config<'a> { name: &'a str, retries: u32 }

    impl<'a> Config<'a> {
        fn name(mut self, n: &'a str) -> Self {
            self.name = n; self
        }
        fn retries(mut self, r: u32) -> Self {
            self.retries = r; self
        }
    }
    let c = Config::default().name("dev").retries(3);
/* Taking `self` by value and returning it makes the chain
   work without any borrowing at all.

/* --- 3. AVOIDING LIFETIMES: own it, or index it --- */
    struct Owned { data: Vec<u8> }              // no
                                                // lifetimes
    struct Indexed { range: Range<usize> }      // no
                                                // lifetimes
/* When a struct with a lifetime becomes awkward - which
   happens when it needs to be stored, sent between threads,
   or held in a collection - one of these two is the answer.

/* --- 4. WHEN A REFERENCE MUST OUTLIVE ITS SCOPE --- */
    let leaked: &'static mut [u8] =
        Box::leak(vec![0u8; 1024].into_boxed_slice());
/* Deliberately leaks a heap allocation to obtain a 'static
   reference. Legitimate for one-time initialisation at
   startup - a DMA buffer, a global table - and a red flag
   anywhere else.

   ON no_std, StaticCell does the same thing without a heap
   (chapter 15).                                          */
```

Reading the errors

```
error[E0597]: `x` does not live long enough
  --> src/main.rs:6:13
   |
 5 |         let x = 5;
   |             - binding `x` declared here
 6 |         r = &x;
   |             ^^ borrowed value does not live long enough
 7 |     }
   |     - `x` dropped here while still borrowed
 8 |     println!("{r}");
   |               --- borrow later used here

/* THE THREE SPANS tell you everything: where it was
   borrowed, where it was dropped, and where the borrow was
   still needed. Read them in that order and the fix is
   usually obvious - either move the value out, extend its
   scope, or stop holding the reference.

/* THE OTHER COMMON ONES:

   E0106  missing lifetime specifier
          -> the compiler cannot tell which input the output
             borrows from. Add <'a> and say.

   E0621  explicit lifetime required
          -> a returned reference needs a longer lifetime
             than the signature promises.

   E0716  temporary value dropped while borrowed
          -> you borrowed from something that dies at the end
             of the statement:
                 let r = &String::from("x");    // ERROR
                 let s = String::from("x");
                 let r = &s;                    // fine

   E0759  argument requires that it is borrowed for 'static
          -> usually thread::spawn or a trait object; own the
             data or use a scoped thread.

/* AND THE PRACTICAL ADVICE: when stuck, try in this order:
     1. does the function really need to return a reference,
        or can it return an owned value?
     2. can the caller keep ownership and pass &T in?
     3. can the struct own its data instead of borrowing?
     4. can indices replace the references?
     5. only then, write the lifetime annotations.
   Four out of five times one of the first four is the right
   answer, and the annotation was a symptom.             */
```

How it shows up

Error	Meaning and fix
E0106 missing lifetime specifier	Two inputs, one output reference — say which it comes from
E0597 does not live long enough	Read the three spans; usually restructure or own the data
E0716 temporary dropped while borrowed	Bind the temporary to a variable first

Error	Meaning and fix
E0759 requires 'static	A thread or trait object — own it, or use a scoped thread
"lifetime may not live long enough"	Often variance, through a `&mut` of a reference type
A method keeps `self` borrowed too long	Elision rule 3 — annotate the output to borrow the input instead
A struct with a lifetime cannot be stored	Make it own its data, or use indices
Adding `'static` moves the error to the caller	It was the wrong fix — find the real relationship
Everything needs `.clone()`	The ownership design wants revisiting, not more clones

Check yourself

- ✓ Explain what a lifetime is and that it generates no code.
- ✓ State the three elision rules and apply them to five signatures.
- ✓ Explain what `fn longest<'a>(a: &'a str, b: &'a str) -> &'a str` claims.
- ✓ Write a struct that borrows, and explain what it constrains.
- ✓ Distinguish `&'static T` from `T: 'static`.
- ✓ Explain why `thread::spawn` requires `'static`.
- ✓ Explain `'b: 'a` and why `&mut T` is invariant in T.
- ✓ Write a zero-copy parser using the anonymous lifetime.
- ✓ Give four ways to avoid needing lifetime annotations at all.
- ✓ Read an E0597 error and identify the three spans.

Structs, enums and pattern matching

Rust's enums carry data and its `match` is exhaustive. Together they let you make invalid states unrepresentable — which is a stronger guarantee than any amount of validation, and the single most useful design technique the language offers.

Structs

```
    struct Point { x: i32, y: i32 }             // named fields
    struct Wrapper(u32);                        // TUPLE struct
    struct Marker;                              // UNIT struct -
                                                // zero sized

    let p = Point { x: 1, y: 2 };
    let w = Wrapper(42);
    println!("{}", w.0);

    let x = 1; let y = 2;
    let p = Point { x, y };                     // field
                                                // shorthand

    let q = Point { x: 5, ..p };                // struct
                                                // update
                                                // syntax

/* METHODS live in an `impl` block, separate from the data: */
    impl Point {
        fn new(x: i32, y: i32) -> Self { Self { x, y } }
                                        // an ASSOCIATED
                                        // function - no self
        fn magnitude(&self) -> f64 { ... }      // borrows
        fn translate(&mut self, dx: i32) { ... }// mutates
        fn into_tuple(self) -> (i32, i32) { ... }// consumes
    }
    let p = Point::new(1, 2);           // :: for associated
    let m = p.magnitude();              // .  for methods

/* NEWTYPES - a tuple struct wrapping one value - are the
   idiomatic way to get a distinct type at zero cost: */
    struct Celsius(f32);
    struct Fahrenheit(f32);
    fn set_target(t: Celsius) { }
    set_target(Fahrenheit(98.6));       // COMPILE ERROR
/* Same representation, different type. This is the C++
   strong-typedef pattern (C++ chapter 07) with less
   ceremony, and it is used pervasively in embedded Rust for
   units, register values and hardware identifiers.

/* DERIVES do the boilerplate: */
    #[derive(Debug, Clone, Copy, PartialEq, Eq,
             PartialOrd, Ord, Hash, Default)]
    struct Config { rate: u32, gain: u8 }

    println!("{c:?}");          // Debug
    println!("{c:#?}");         // pretty-printed Debug
    let d = Config::default();  // Default - zeroes
    if a == b { }               // PartialEq
```

Enums that carry data

```
/* THE CENTRAL DIFFERENCE FROM C: a Rust enum is a TAGGED
   UNION, and each variant may carry different data.      */

    enum Message {
        Quit,                                   // no data
        Move { x: i32, y: i32 },                // named
                                                // fields
        Write(String),                          // one value
        Colour(u8, u8, u8),                     // a tuple
    }

/* IN C YOU WOULD WRITE: */
        struct Message {
                enum { QUIT, MOVE, WRITE, COLOUR } tag;
                union {
                        struct { int x, y; } move_;
                        char *write;
                        struct { uint8_t r,g,b; } colour;
                } data;
        };
/* ...and nothing stops you reading `data.write` when the tag
   says MOVE. Rust makes that impossible: you can only reach
   the payload through a `match` that has already checked the
   tag.

/* SIZE: as large as the biggest variant, plus a
   discriminant - and Rust applies NICHE OPTIMISATION: */
    assert_eq!(size_of::<Option<&u8>>(), size_of::<&u8>());
/* A reference can never be null, so `None` is represented by
   the null bit pattern - Option<&T> is EXACTLY the size of a
   pointer. Same for Option<Box<T>>, Option<NonZeroU32>, and
   any enum with an unused bit pattern available. So the
   safety costs nothing.

/* C-LIKE ENUMS with explicit values, for hardware: */
    #[derive(Clone, Copy, PartialEq)]
    #[repr(u8)]
    enum Mode { Idle = 0, Run = 1, Fault = 0xFF }

    let raw = Mode::Run as u8;          // enum -> integer:
                                        // free
    let m = Mode::try_from(raw)?;       // integer -> enum:
                                        // CHECKED
/* `#[repr(u8)]` fixes the representation, which matters for
   FFI and for register fields (chapter 12). Converting FROM
   an integer must be fallible, because not every value is
   valid - and that is exactly the check C omits.        */
```

Pattern matching

```
    match msg {
        Message::Quit => cleanup(),
        Message::Move { x, y } => move_to(x, y),
        Message::Write(text) => print(&text),
        Message::Colour(r, g, b) => set_colour(r, g, b),
    }
/* EXHAUSTIVE: omit a variant and it does not compile. Add a
   variant to the enum later, and every match that does not
   handle it fails to build - which is precisely the review
   you want.

/* PATTERN FORMS: */
    match x {
        0 => "zero",
        1 | 2 | 3 => "small",           // alternatives
        4..=9 => "medium",              // an inclusive RANGE
        n if n < 0 => "negative",       // a GUARD
        _ => "large",                   // the wildcard
    }

    match point {
        Point { x: 0, y: 0 } => "origin",
        Point { x: 0, y } => "on the y axis",
        Point { x, .. } => "elsewhere",         // .. ignores
                                                // the rest
    }

    match slice {
        [] => "empty",
        [one] => "one element",
        [first, .., last] => "several",         // slice
                                                // patterns
        [first, rest @ ..] => "head and tail",  // BINDING with
                                                // @
    }

    match opt {
        Some(n @ 1..=9) => println!("single digit {n}"),
        Some(n) => println!("{n}"),
        None => println!("nothing"),
    }

/* MATCH ERGONOMICS - matching on a reference binds
   references, so you rarely need to write & or *: */
    match &opt {
        Some(v) => use(v),      // v is &T automatically
        None => {}
    }

/* THE SHORTHANDS: */
    if let Some(v) = opt { use(v); }
    else { handle_none(); }

    let Some(v) = opt else { return; };  // LET-ELSE - the
                                         // else branch must
                                         // diverge

    while let Some(item) = queue.pop() { process(item); }

    // and matches! for a boolean test:
```

```
    if matches!(state, State::Running | State::Paused) { }

/* LET-ELSE IS THE ONE TO ADOPT: it replaces the pyramid of
   `if let` nesting with early returns, which reads exactly
   like the guard-clause style good C already uses.      */
```

MAKE INVALID STATES UNREPRESENTABLE

```
// BAD: four booleans = sixteen states, twelve invalid
struct Conn { connected: bool, authed: bool,
              error: bool, session: Option<u32> }

// GOOD: exactly the states that exist
enum Conn {
    Disconnected,
    Connecting { started: Instant },
    Authenticated { session: u32 },
    Failed(Error),
}
```

The second version cannot represent "authenticated but not connected", so no code needs to check for it and no test needs to cover it. This is the highest-leverage design technique in the language, and it transfers directly to C++ (`std::variant`) and even to C with discipline.

Modelling hardware

```
/* --- A REGISTER FIELD as an enum --- */
    #[repr(u8)]
    #[derive(Clone, Copy, PartialEq, Debug)]
    pub enum Prescaler { Div1 = 0, Div2 = 1, Div4 = 2,
                         Div8 = 3 }

    impl Prescaler {
        pub const fn divisor(self) -> u32 {
            1 << (self as u8)
        }
    }
/* An invalid prescaler value cannot exist, and the divisor
   is computed at compile time.

/* --- A COMMAND PROTOCOL --- */
    enum Command {
        Read  { addr: u16, len: u8 },
        Write { addr: u16, data: [u8; 4] },
        Reset,
        SetMode(Mode),
    }

    fn encode(cmd: &Command, out: &mut [u8]) -> usize {
        match cmd {
            Command::Read { addr, len } => {
                out[0] = 0x01;
                out[1..3].copy_from_slice(&addr.to_le_bytes());
                out[3] = *len;
                4
            }
            Command::Write { addr, data } => {
                out[0] = 0x02;
                out[1..3].copy_from_slice(&addr.to_le_bytes());
                out[3..7].copy_from_slice(data);
                7
            }
            Command::Reset => { out[0] = 0x03; 1 }
            Command::SetMode(m) => { out[0] = 0x04;
                                     out[1] = *m as u8; 2 }
        }
    }
/* Add a command and every encoder, decoder and handler that
   does not cover it fails to compile. In C that is a
   comment; here it is the build.

/* --- A STATE MACHINE, where the state OWNS its data --- */
    enum State {
        Idle,
        Sampling { buffer: [u16; 64], count: usize },
        Transmitting { remaining: usize },
        Error(ErrorKind),
    }

    fn step(state: State, ev: Event) -> State {
        match (state, ev) {
            (State::Idle, Event::Start) =>
                State::Sampling { buffer: [0; 64], count: 0 },

            (State::Sampling { mut buffer, count },
             Event::Sample(v)) if count < 64 => {
```

```
                buffer[count] = v;
                State::Sampling { buffer, count: count + 1 }
            }

            (State::Sampling { count: 64, .. },
             Event::Sample(_)) =>
                State::Transmitting { remaining: 64 },

            (s, Event::Reset) => { let _ = s; State::Idle }

            (s, _) => s,        // unchanged
        }
    }
/* MATCHING ON A TUPLE of (state, event) is the idiomatic
   transition table, and it is checked for exhaustiveness
   across BOTH dimensions.

   NOTE the buffer lives INSIDE the Sampling variant - so it
   does not exist while Idle, and cannot be touched from any
   other state. That is the same "invalid states
   unrepresentable" idea applied to memory.             */
```

Traits on your types, briefly

```
/* Full coverage is chapter 06; the ones you implement on
   almost every type: */

    use core::fmt;

    impl fmt::Display for Temperature {
        fn fmt(&self, f: &mut fmt::Formatter<'_>)
                -> fmt::Result {
            write!(f, "{:.1}°C", self.0)
        }
    }
    // Display is for USERS; Debug (usually derived) is for
    // PROGRAMMERS.

    impl From<u16> for Raw {
        fn from(v: u16) -> Self { Raw(v) }
    }
    // implementing From gives you Into for free:
    let r: Raw = 42u16.into();

    impl TryFrom<u8> for Mode {
        type Error = ();
        fn try_from(v: u8) -> Result<Self, ()> {
            match v {
                0 => Ok(Mode::Idle),
                1 => Ok(Mode::Run),
                0xFF => Ok(Mode::Fault),
                _ => Err(()),
            }
        }
    }
    // the CHECKED integer-to-enum conversion from above

    impl Default for Config {
        fn default() -> Self {
            Config { rate: 48_000, gain: 0 }
        }
    }

/* AND THE DERIVE LIST worth putting on most data types: */
    #[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
/* Debug for diagnostics, Clone/Copy if it is plain data,
   PartialEq for tests, Default if there is a sensible one. */
```

How it shows up

Error	Meaning and fix
"non-exhaustive patterns: `X` not covered"	Add the arm — the feature working as intended
"unreachable pattern"	An earlier arm already matches everything this one would
"cannot move out of ... which is behind a reference"	Matching by value on a `&T` — bind by reference or clone
"binding moves value" in a match arm	Use `ref`, or match on `&value`
`Option<T>` is larger than expected	No niche available — check with `size_of`
Enum discriminant differs from C	Missing `#[repr(u8)]` or `#[repr(C)]`

Error	Meaning and fix
A struct's layout differs from C	Rust reorders fields by default — use `#[repr(C)]`
Two units confused at a call site	Use newtypes
Many booleans, most combinations invalid	Replace with an enum
Adding a variant broke twenty files	Exactly right — those were the places that needed updating

Check yourself

- ✓ Write named, tuple and unit structs, and say when each fits.
- ✓ Explain newtypes and give an embedded use.
- ✓ Explain how a Rust enum differs from a C one, and what it makes impossible.
- ✓ Explain niche optimisation and demonstrate it with `size_of`.
- ✓ Use `#[repr(u8)]` and explain why integer-to-enum must be fallible.
- ✓ Write matches using ranges, guards, bindings, slice patterns and `..`.
- ✓ Use `let-else` to flatten nested `if let`.
- ✓ Redesign a struct of booleans as an enum and say what that removes.
- ✓ Write a state machine matching on `(state, event)`.
- ✓ Implement `Display`, `From`, `TryFrom` and `Default`.

Traits and generics

Rust's abstraction mechanism: interfaces that types opt into, and generics constrained by them. Unlike C++ templates, the constraints are checked when the generic is *defined*, so the errors appear where the mistake is.

Traits

```
    trait Sensor {
        fn read(&mut self) -> u16;              // required

        fn read_averaged(&mut self, n: u8) -> u16 {
            let mut sum = 0u32;                 // a DEFAULT
            for _ in 0..n {                     // method
                sum += self.read() as u32;
            }
            (sum / n as u32) as u16
        }

        const MAX_VALUE: u16 = 4095;            // associated
                                                // constant
        type Unit;                              // associated
                                                // TYPE
    }

    struct Thermistor { channel: u8 }

    impl Sensor for Thermistor {
        type Unit = Celsius;
        fn read(&mut self) -> u16 { adc_read(self.channel) }
        // read_averaged comes free from the default
    }

/* THE COHERENCE (ORPHAN) RULE: you may implement a trait for
   a type only if you own the TRAIT or you own the TYPE. So
   nobody can implement a foreign trait for a foreign type,
   which guarantees exactly one implementation exists
   anywhere in the program.

   THE WORKAROUND when you need to: a newtype (chapter 05).
        struct MyVec(Vec<u8>);
        impl SomeForeignTrait for MyVec { ... }

/* A TRAIT MUST BE IN SCOPE to call its methods: */
    use my_crate::Sensor;       // without this, `.read()` is
                                // "method not found"
/* Which is why the standard library has PRELUDES, and why
   embedded HAL crates tell you to `use embedded_hal::prelude
   ::*`.                                                  */
```

Generics

```
    fn largest<T: PartialOrd>(list: &[T]) -> &T {
        let mut max = &list[0];
        for x in list {
            if x > max { max = x; }
        }
        max
    }

/* THE BOUND `T: PartialOrd` IS CHECKED AT DEFINITION. If the
   body used `+` you would get an error HERE, not at some
   distant instantiation - which is the central difference
   from C++ templates (C++ chapter 06), and the reason Rust's
   generic errors are readable.

/* THREE SYNTAXES for the same thing: */
    fn f<T: Display + Clone>(x: T) { }
    fn f<T>(x: T) where T: Display + Clone { }
    fn f(x: impl Display + Clone) { }           // "impl
                                                // Trait" -
                                                // shortest,
                                                // but the
                                                // caller
                                                // cannot
                                                // turbofish it

/* GENERIC STRUCTS AND IMPLS: */
    struct RingBuffer<T, const N: usize> {
        data: [T; N],
        head: usize,
        tail: usize,
    }

    impl<T: Copy + Default, const N: usize> RingBuffer<T, N> {
        pub const fn capacity(&self) -> usize { N }

        pub fn push(&mut self, v: T) -> Result<(), T> {
            let next = (self.head + 1) % N;
            if next == self.tail { return Err(v); }
            self.data[self.head] = v;
            self.head = next;
            Ok(())
        }
    }

    let mut rb: RingBuffer<u8, 64> = RingBuffer::new();

/* CONST GENERICS (`const N: usize`) are the embedded feature:
   a fixed-capacity container with the size in the type, no
   heap, and `% N` folded to an AND when N is a power of two.
   Exactly the C++ template-parameter pattern (C++ ch. 06).

/* MONOMORPHISATION: the compiler generates a separate copy
   for each concrete type used - same as C++ templates, same
   code-size consideration, same zero runtime cost.       */
```

CONSTRAINED GENERICS VERSUS DUCK-TYPED TEMPLATES

A C++ template is checked when instantiated, so an error inside `std::sort` reports a hundred lines deep. A Rust generic is checked when *defined*: if the body calls a method the bound does not promise, the error is on that line, in your function. That is why C++20 added concepts (C++ chapter 06) — it is retrofitting the same property.

Static versus dynamic dispatch

```
/* --- STATIC: one copy per type, fully inlinable --- */
    fn sample<S: Sensor>(s: &mut S) -> u16 { s.read() }

    sample(&mut thermistor);    // generates sample::<Thermistor>
    sample(&mut photodiode);    // and sample::<Photodiode>

/* -> a direct call, usually inlined to nothing. Zero cost,
   more code.

/* --- DYNAMIC: one copy, a vtable --- */
    fn sample_dyn(s: &mut dyn Sensor) -> u16 { s.read() }

    let sensors: [&mut dyn Sensor; 3] = [&mut a, &mut b,
                                         &mut c];
    for s in sensors { sample_dyn(s); }

/* A `&dyn Trait` is a FAT POINTER - two words: the data
   pointer and a vtable pointer. Note the difference from
   C++ (C++ chapter 11), where the vptr is INSIDE the object:
     · a Rust object has no vtable pointer unless you make a
       trait object of it, so non-polymorphic use costs
       nothing
     · but &dyn T is 16 bytes rather than 8

    assert_eq!(size_of::<&u8>(), 8);
    assert_eq!(size_of::<&dyn Sensor>(), 16);

/* OBJECT SAFETY - a trait can only be a `dyn` object if:
     · no generic methods
     · no methods taking or returning `Self` by value
     · no associated constants (before Rust 1.79)
     · every method is dispatchable
   `Clone` is not object safe, for instance, because clone()
   returns Self.

/* CHOOSING - the same decision as C++ chapter 11:
     the type set is known at compile time  -> GENERICS
     a heterogeneous collection, or a plugin -> dyn
     code size matters more than speed       -> dyn
     it is in a hot path                     -> generics

   ON EMBEDDED, generics dominate: the peripheral set is
   known, and monomorphisation lets the optimiser fold the
   whole driver stack into a few instructions.

/* AND `impl Trait` IN RETURN POSITION, which is static
   dispatch with the type hidden: */
    fn make_sensor() -> impl Sensor { Thermistor::new(0) }
/* One concrete type, chosen by the function, opaque to the
   caller. No allocation, no vtable. For a heterogeneous
   return you need Box<dyn Sensor> instead.             */
```

The traits that matter

Trait	Meaning
`Copy`, `Clone`	Bitwise duplication; explicit duplication (ch. 02)

Trait	Meaning
`Drop`	Run code when the value dies
`Debug`, `Display`	Formatting for programmers; for users
`Default`	A sensible zero value
`PartialEq`, `Eq`	Equality; equality that is reflexive (so `f32` is only `PartialEq`)
`PartialOrd`, `Ord`	Ordering, total ordering
`Hash`	Usable as a map key
`From`, `Into`	Infallible conversion — implement `From`, get `Into` free
`TryFrom`, `TryInto`	Fallible conversion
`Deref`, `DerefMut`	Behave like a reference to something else
`Index`, `IndexMut`	`x[i]`
`Iterator`, `IntoIterator`	Iteration (ch. 08)
`Send`, `Sync`	Thread safety (ch. 13)
`Sized`	Known size — an implicit bound on every generic parameter
`Fn`, `FnMut`, `FnOnce`	Callables (ch. 08)
Operator traits	`Add`, `Sub`, `BitOr`, `Not`, ...

```
/* OPERATOR OVERLOADING is just implementing a trait: */
    use core::ops::{Add, BitOr};

    #[derive(Clone, Copy)]
    struct Flags(u32);

    impl BitOr for Flags {
        type Output = Flags;
        fn bitor(self, rhs: Flags) -> Flags {
                Flags(self.0 | rhs.0) }
    }
    let f = Flags(1) | Flags(2);

/* Deref, and its one legitimate use - smart pointers: */
    impl Deref for MyBox<T> {
        type Target = T;
        fn deref(&self) -> &T { &self.0 }
    }
/* DO NOT use Deref to fake inheritance. It is for types that
   ARE a pointer to something, and abusing it produces
   confusing method resolution.                          */
```

The embedded-hal traits

```
/* The embedded ecosystem is built on one set of traits, so a
   driver written once works on every chip (chapter 16).  */

    use embedded_hal::digital::{OutputPin, InputPin};
    use embedded_hal::spi::SpiDevice;
    use embedded_hal::delay::DelayNs;

    pub struct Display<SPI, DC, RST> {
        spi: SPI,
        dc:  DC,
        rst: RST,
    }

    impl<SPI, DC, RST, E> Display<SPI, DC, RST>
    where
        SPI: SpiDevice<Error = E>,
        DC:  OutputPin,
        RST: OutputPin,
    {
        pub fn init(&mut self, delay: &mut impl DelayNs)
                -> Result<(), E>
        {
            self.rst.set_low().ok();
            delay.delay_ms(10);
            self.rst.set_high().ok();
            delay.delay_ms(120);

            self.dc.set_low().ok();             // command mode
            self.spi.write(&[0x11])?;           // sleep out
            Ok(())
        }
    }

/* THIS DRIVER COMPILES FOR EVERY CHIP whose HAL implements
   the traits - STM32, nRF, ESP32, RP2040 - with no
   conditional compilation and no porting layer. That is the
   ecosystem's central achievement, and it has no equivalent
   in the C world, where every vendor HAL has its own
   incompatible API.

   AND IT IS ZERO COST: SPI, DC and RST are concrete types at
   the call site, so every trait call is direct and inlined.
   The generated code is identical to a chip-specific
   driver.                                                */
```

Patterns worth knowing

```
/* --- BLANKET IMPLEMENTATIONS --- */
    trait Loggable { fn log(&self); }

    impl<T: Display> Loggable for T {
        fn log(&self) { println!("{self}"); }
    }
/* EVERY type implementing Display now implements Loggable.
   The standard library uses this heavily - `impl<T> Into<U>
   for T where U: From<T>` is why implementing From gives you
   Into.

/* --- MARKER TRAITS: no methods, only a promise --- */
    unsafe trait Zeroable {}            // "all-zero bits is a
                                        // valid value"
    unsafe impl Zeroable for u32 {}
    fn zeroed<T: Zeroable>() -> T { unsafe { core::mem::zeroed() } }
/* `unsafe trait` means implementing it is a promise the
   compiler cannot check - which is how Send and Sync work
   (chapter 13).

/* --- EXTENSION TRAITS: add methods to a foreign type --- */
    trait BitExt { fn bit(self, n: u8) -> bool; }
    impl BitExt for u32 {
        fn bit(self, n: u8) -> bool { (self >> n) & 1 != 0 }
    }
    if reg.bit(5) { }
/* Legal because you own the trait. Extremely useful for
   register manipulation.

/* --- SEALED TRAITS: public to use, private to implement --- */
    mod private { pub trait Sealed {} }
    pub trait Register: private::Sealed { ... }
    impl private::Sealed for CTRL {}
    impl Register for CTRL { ... }
/* Nobody outside your crate can implement Register, because
   they cannot name Sealed. Used throughout the PAC crates
   (chapter 16) so the register set is closed.

/* --- ASSOCIATED TYPES versus GENERIC PARAMETERS --- */
    trait Container { type Item; fn get(&self)
                                      -> Option<&Self::Item>; }
    trait Convert<T> { fn convert(&self) -> T; }
/* USE AN ASSOCIATED TYPE when there is exactly one sensible
   choice per implementing type (Iterator::Item). Use a
   GENERIC PARAMETER when a type may implement it several
   ways (From<u8> and From<u16> for the same type). Getting
   this wrong makes an API either too rigid or ambiguous. */
```

How it shows up

Error	Meaning and fix
"no method named `x` found"	The trait is not in scope — `use` it
"the trait bound `T: Foo` is not satisfied"	Add the bound, or implement the trait
"only traits defined in the current crate can be implemented"	The orphan rule — use a newtype
"the trait cannot be made into an object"	Not object safe — use generics, or restructure the trait

Error	Meaning and fix
"type annotations needed"	Inference is ambiguous — turbofish it: `x.parse::<u32>()`
"conflicting implementations"	Two impls could apply — usually a blanket impl clashing
"the size for values of type `T` cannot be known"	Needs `?Sized`, or a reference
Binary larger than expected	Monomorphisation — consider `dyn` for cold paths
A generic function will not inline	Check it is not accidentally a trait object
An API is too rigid or too ambiguous	Associated type versus generic parameter chosen wrongly

Check yourself

- ✓ Define a trait with a required method, a default method and an associated type.
- ✓ State the orphan rule and its newtype workaround.
- ✓ Explain why a trait must be in scope to call its methods.
- ✓ Write the same bound three ways and say when each is preferable.
- ✓ Explain why Rust's generic errors appear at the definition and C++'s at instantiation.
- ✓ Use const generics for a fixed-capacity container.
- ✓ Compare `&dyn Trait` with C++'s vptr, including the size difference.
- ✓ State the object-safety rules and why `Clone` fails them.
- ✓ Write a driver generic over `embedded-hal` traits and say what it buys.
- ✓ Use a blanket impl, an extension trait and a sealed trait.

Error handling with `Option` and `Result`

No exceptions, no error codes to forget. Failure is in the return type, the compiler makes you handle it, and the `?` operator makes propagation as terse as ignoring it would be in C.

Two types

```
    enum Option<T> { Some(T), None }
    enum Result<T, E> { Ok(T), Err(E) }

/* THAT IS ALL THEY ARE - ordinary enums (chapter 05), with a
   great many convenience methods.

     Option    "a value, or nothing" - absence is not an error
     Result    "a value, or an error explaining why"

   USE Option WHEN: a lookup misses, a buffer is empty, a
   configuration field is unset, a conversion has one obvious
   failure mode.
   USE Result WHEN: the caller needs to know WHY, or wants to
   propagate.

/* THERE IS NO NULL. `Option<&T>` is the same size as `&T`
   (niche optimisation, chapter 05), so the safety is free -
   and the check is not optional:                          */

    fn find(&self, id: u32) -> Option<&Device>;

    let d = self.find(id);
    d.name();                   // COMPILE ERROR - it is an
                                // Option
    if let Some(d) = self.find(id) { d.name(); }

/* COMPARE C, where the null check is a convention nobody
   enforces, and C++, where std::optional is opt-in and the
   raw pointer remains available.

/* AND #[must_use]: Result is marked with it, so ignoring one
   is a warning: */
    write_register(0x10, 5);    // warning: unused
                                // `Result` that must be used
    let _ = write_register(0x10, 5);    // explicit "I am
                                        // ignoring this"  */
```

The `?` operator

```
/* WITHOUT it, propagation is verbose: */
    fn read_config() -> Result<Config, Error> {
        let raw = match read_flash(CONFIG_ADDR) {
            Ok(v) => v,
            Err(e) => return Err(e),
        };
        let parsed = match parse(&raw) {
            Ok(v) => v,
            Err(e) => return Err(e),
        };
        Ok(parsed)
    }

/* WITH it: */
    fn read_config() -> Result<Config, Error> {
        let raw = read_flash(CONFIG_ADDR)?;
        let parsed = parse(&raw)?;
        Ok(parsed)
    }

/* `expr?` means: if Ok(v), evaluate to v; if Err(e), RETURN
   Err(e.into()) from the enclosing function.

   THE `.into()` IS THE IMPORTANT PART: the error is converted
   via the From trait (chapter 06), so a function returning
   `Result<_, MyError>` can use `?` on anything whose error
   type converts into MyError.                              */

    #[derive(Debug)]
    enum Error {
        Flash(FlashError),
        Parse(ParseError),
        Timeout,
    }

    impl From<FlashError> for Error {
        fn from(e: FlashError) -> Self { Error::Flash(e) }
    }
    impl From<ParseError> for Error {
        fn from(e: ParseError) -> Self { Error::Parse(e) }
    }
/* Now `?` converts automatically at every call site, and
   adding a new error source is one From impl.

/* `?` ALSO WORKS ON Option, inside a function returning
   Option: */
    fn first_even(v: &[i32]) -> Option<i32> {
        let x = v.iter().find(|x| *x % 2 == 0)?;
        Some(*x * 2)
    }

/* AND THE CHAIN, which is where it earns its keep: */
    fn handle(&mut self, cmd: &[u8]) -> Result<Response, Error> {
        let parsed = Command::parse(cmd)?;
        let result = self.execute(parsed)?;
        let encoded = result.encode()?;
        self.transmit(&encoded)?;
        Ok(Response::Ok)
    }
```

```
/* Four fallible operations, four characters of error
   handling, and not one of them can be forgotten.       */
```

WHY THIS BEATS BOTH ALTERNATIVES

An error code can be ignored silently — and in C, routinely is. An exception cannot be ignored but is invisible in the signature and unwinds unpredictably (C++ chapter 12). `Result` is in the type, cannot be dropped without a warning, costs nothing at run time, and propagates in one character. It is the design C++23's `std::expected` is copying.

The methods worth knowing

```
/* --- EXTRACTING --- */
    opt.unwrap()                // PANICS on None
    opt.expect("msg")           // panics with your message
    opt.unwrap_or(default)
    opt.unwrap_or_else(|| compute())
    opt.unwrap_or_default()

    res.unwrap()                // panics, printing the Err
    res.expect("msg")
    res.unwrap_or(default)
    res.unwrap_err()            // panics if Ok

/* USE `expect` OVER `unwrap` - the message tells the next
   reader WHY you believed it could not fail:            */
    let p = Peripherals::take()
        .expect("peripherals taken twice");

/* --- TRANSFORMING --- */
    opt.map(|v| v * 2)                  // Option<T> ->
                                        // Option<U>
    opt.and_then(|v| lookup(v))         // flat_map - avoids
                                        // Option<Option<T>>
    opt.filter(|v| *v > 0)
    opt.or(other)
    opt.take()                          // leaves None behind
    opt.replace(v)
    opt.get_or_insert_with(|| make())

    res.map(|v| v * 2)
    res.map_err(|e| Error::Wrapped(e))  // convert the ERROR
    res.and_then(|v| next_step(v))
    res.ok()                            // Result -> Option,
                                        // discarding the
                                        // error
    opt.ok_or(Error::Missing)           // Option -> Result
    opt.ok_or_else(|| Error::Missing)

/* --- INSPECTING --- */
    opt.is_some()  opt.is_none()
    res.is_ok()    res.is_err()
    opt.as_ref()                        // Option<T> ->
                                        // Option<&T>
    opt.as_mut()
    res.as_ref()
    opt.iter()                          // 0 or 1 items

/* --- COLLECTING, which is a genuinely useful trick --- */
    let results: Vec<Result<u32, E>> = ...;
    let all: Result<Vec<u32>, E> = results.into_iter().collect();
/* Collects into Ok(vec) if EVERY element is Ok, or the FIRST
   Err. One line replaces a loop with an early return.

    let vals: Vec<u32> = results.into_iter()
                               .flatten()  // keep the Oks
                               .collect();

/* --- THE LAZY VARIANTS --- */
    opt.unwrap_or(expensive())          // ALWAYS evaluates
    opt.unwrap_or_else(|| expensive())  // only on None
/* The `_else` and `_with` suffixes always mean "lazy". On
```

```
    embedded, where the default might be a hardware read,
    this matters.                                          */
```

Designing an error type

```
/* --- FOR no_std / EMBEDDED: a small enum, Copy if
       possible --- */
    #[derive(Debug, Clone, Copy, PartialEq, Eq)]
    pub enum Error {
        Timeout,
        BusError,
        Nack,
        InvalidAddress(u8),
        BufferTooSmall { needed: usize, got: usize },
    }

    impl core::fmt::Display for Error {
        fn fmt(&self, f: &mut core::fmt::Formatter<'_>)
                -> core::fmt::Result {
            match self {
                Error::Timeout => write!(f, "timeout"),
                Error::Nack => write!(f, "not acknowledged"),
                Error::InvalidAddress(a) =>
                        write!(f, "invalid address {a:#x}"),
                Error::BufferTooSmall { needed, got } =>
                        write!(f, "buffer too small: need \
                                   {needed}, got {got}"),
                _ => write!(f, "bus error"),
            }
        }
    }

/* SIZE MATTERS on embedded: keep the variants small, and
   prefer Copy so the error can be returned without a move.
   `Result<(), Error>` with a Copy enum is typically one or
   two bytes.

/* --- THE thiserror CRATE removes the boilerplate (std or
       no_std with `core-error`): --- */
    #[derive(Debug, thiserror::Error)]
    pub enum Error {
        #[error("timeout after {0} ms")]
        Timeout(u32),

        #[error("SPI failure")]
        Spi(#[from] SpiError),           // generates the From
                                         // impl AND the
                                         // source chain
    }

/* --- FOR APPLICATIONS, anyhow: --- */
    fn main() -> anyhow::Result<()> {
        let cfg = read_config()
            .context("reading configuration")?;
        run(cfg).context("running")?;
        Ok(())
    }
/* anyhow::Error boxes any error and accumulates CONTEXT, so
   the final message is "running: reading configuration: file
   not found". Perfect for a binary; wrong for a library,
   because callers cannot match on the error.

/* THE RULE: LIBRARIES define a concrete enum so callers can
```

```
   match. APPLICATIONS use anyhow so they can report. Embedded
   firmware is a library in this sense - define the enum. */
```

Panics

```
/* A panic is for a BUG, not a runtime failure - the same
   distinction as assert versus error return (C++ chapter 12).
*/

    panic!("invariant violated: {state:?}");
    assert!(index < LEN, "index {index} out of range");
    assert_eq!(a, b);
    debug_assert!(expensive_check());   // debug builds only
    unreachable!("state machine reached an impossible state");
    todo!()  unimplemented!()

/* WHAT HAPPENS: by default the stack UNWINDS, running Drop
   for every live value, then the thread dies. With
   `panic = "abort"` it terminates immediately.

/* THESE PANIC, and knowing which is important on embedded:
     unwrap() / expect() on None or Err
     array or slice indexing out of range
     integer division by zero
     INTEGER OVERFLOW in debug builds (release wraps)
     .unwrap() on a poisoned Mutex

/* AVOIDING PANICS in code that must not: */
    v.get(i)                    // Option, not a panic
    v.get_mut(i)
    a.checked_add(b)            // Option
    a.saturating_add(b)         // clamps
    a.wrapping_add(b)           // explicit wrap
    a.overflowing_add(b)        // (value, did_overflow)
    a.checked_div(b)            // None on zero

/* ON EMBEDDED (chapter 15): */
    #[panic_handler]
    fn panic(info: &PanicInfo) -> ! {
        // log it somewhere persistent, then reset
        write_panic_log(info);
        cortex_m::peripheral::SCB::sys_reset();
    }

    // in Cargo.toml
    [profile.release]
    panic = "abort"             // no unwinder - smaller
    overflow-checks = false     // or true, if you prefer the
                                // panic

/* AND THE panic-never TRICK: if you can prove no panic is
   reachable, the `panic-never` crate makes any residual panic
   a LINK ERROR. Combined with careful use of `get` and
   `checked_*`, that gives a build-time guarantee of no
   panics - which for safety-critical firmware is a strong
   property.                                              */
```

Patterns

```
/* --- 1. FALLIBLE CONSTRUCTION --- */
    impl Uart {
        pub fn new(base: usize, baud: u32)
                -> Result<Self, Error>
        {
            if baud == 0 || baud > MAX_BAUD {
                return Err(Error::InvalidBaud);
            }
            let uart = Uart { base };
            uart.configure(baud)?;
            Ok(uart)
        }
    }
/* No "constructed but invalid" state - if you have a Uart, it
   works. Same argument as the C++ static-factory pattern.

/* --- 2. RETRY WITH A BOUND --- */
    fn read_with_retry(&mut self, n: u8)
            -> Result<u16, Error>
    {
        let mut last = Error::Timeout;
        for _ in 0..n {
            match self.read() {
                Ok(v) => return Ok(v),
                Err(e) => last = e,
            }
        }
        Err(last)
    }

/* --- 3. CLEANUP ON ERROR is automatic --- */
    fn transfer(&mut self) -> Result<(), Error> {
        let _cs = self.cs.assert();     // a Drop guard
        self.write(&[0x01])?;           // early return here
        self.write(&[0x02])?;           // or here
        Ok(())
    }                                   // _cs drops on EVERY
                                        // path, including the
                                        // ? returns
/* This is why RAII and `?` compose so well: the early return
   is exactly as safe as the normal one.

/* --- 4. MATCHING ON THE ERROR --- */
    match sensor.read() {
        Ok(v) => process(v),
        Err(Error::Timeout) => retry(),
        Err(Error::Nack) => reset_bus(),
        Err(e) => fatal(e),
    }
/* A concrete error enum makes this possible. anyhow does
   not - which is the reason libraries should not use it.

/* --- 5. TESTS --- */
    #[test]
    fn rejects_zero_baud() {
        assert!(matches!(Uart::new(0x4000, 0),
                         Err(Error::InvalidBaud)));
    }

    #[test]
```

```
    fn works() -> Result<(), Error> {
        let u = Uart::new(0x4000, 115200)?;   // `?` in a test
        assert_eq!(u.baud(), 115200);
        Ok(())
    }
```

How it shows up

Symptom	Cause and fix
"unused `Result` that must be used"	Handle it, or write `let _ =` deliberately
"the `?` operator can only be used in a function that returns Result"	Change the return type, or handle it explicitly
"`?` couldn't convert the error"	Missing `From` impl for that error type
"cannot use `?` on an Option in a Result function"	Use `.ok_or(...)?`
Panic: "called `unwrap()` on a `None`"	Handle the case, or use `expect` with a reason
Panic: "index out of bounds"	Use `.get()`
Panic in release but not debug, or vice versa	Integer overflow — checked in debug, wrapping in release
Binary large because of panic machinery	`panic = "abort"`, and avoid formatting in the handler
Cannot match on a library's errors	It used `anyhow` — a library design mistake
Error type is large and copied everywhere	Box the payload, or shrink the variants

Check yourself

- ✓ Say when to use `Option` and when `Result`.
- ✓ Explain why `Option<&T>` costs nothing.
- ✓ Explain exactly what `?` does, including the conversion.
- ✓ Write `From` impls so `?` works across three error types.
- ✓ Use ten combinator methods and say which are lazy.
- ✓ Collect a `Vec<Result>` into a `Result<Vec>`.
- ✓ Design an error enum suitable for `no_std`.
- ✓ Say why libraries define enums and applications use `anyhow`.
- ✓ List what panics, and the non-panicking alternative for each.
- ✓ Write a `#[panic_handler]` and explain `panic = "abort"`.
- ✓ Show why RAII and `?` compose safely.

Collections, iterators and closures

Rust's iterators are lazy, composable and compile to the same loop you would write by hand — usually without bounds checks. On embedded they are the main reason idiomatic Rust is fast, and the main reason it needs no heap.

The Iterator trait

```
    trait Iterator {
        type Item;
        fn next(&mut self) -> Option<Self::Item>;

        // ...and about seventy provided methods, all built
        // on next()
    }

/* THAT IS THE WHOLE INTERFACE. Implement `next` and you get
   map, filter, zip, take, fold, sum and everything else for
   free.                                                    */

    struct Countdown(u32);

    impl Iterator for Countdown {
        type Item = u32;
        fn next(&mut self) -> Option<u32> {
            if self.0 == 0 { None }
            else { self.0 -= 1; Some(self.0 + 1) }
        }
    }

    for x in Countdown(3) { print!("{x} "); }   // 3 2 1
    let v: Vec<u32> = Countdown(3).collect();
    let s: u32 = Countdown(100).filter(|x| x % 3 == 0).sum();

/* THREE WAYS TO ITERATE A COLLECTION, and choosing wrongly
   is the commonest early mistake:                          */

    for x in &v      { }        // x: &T        - borrows
    for x in &mut v  { }        // x: &mut T    - borrows
                                //                mutably
    for x in v       { }        // x: T         - CONSUMES v

    v.iter()                    // -> &T
    v.iter_mut()                // -> &mut T
    v.into_iter()               // -> T, consumes

/* `for x in v` MOVES v. If you use v afterwards, the error
   is "use of moved value" and the fix is `&v`.          */
```

Adapters and consumers

```
/* ADAPTERS are lazy - they build a new iterator and do
   nothing until consumed.                                  */

    .map(|x| x * 2)
    .filter(|x| *x > 0)
    .filter_map(|x| x.checked_mul(2))   // filter + map, on
                                        // Option
    .flat_map(|x| x.iter())
    .flatten()                          // Iterator<Iterator>
                                        // -> Iterator
    .take(5)  .skip(5)
    .take_while(|x| *x < 10)
    .skip_while(|x| *x < 10)
    .step_by(2)
    .chain(other)                       // concatenate
    .zip(other)                         // pairs; stops at the
                                        // shorter
    .enumerate()                        // (index, item)
    .rev()                              // needs
                                        // DoubleEndedIterator
    .peekable()                         // .peek() without
                                        // consuming
    .windows(3)   .chunks(4)            // on SLICES, not
                                        // iterators
    .scan(state, f)                     // stateful map
    .inspect(|x| println!("{x:?}"))     // debugging, no
                                        // effect

/* CONSUMERS drive the whole chain: */
    .collect::<Vec<_>>()
    .sum::<i32>()   .product()
    .count()  .last()  .nth(3)
    .min()  .max()  .min_by_key(|x| x.len())
    .fold(init, |acc, x| acc + x)
    .try_fold(init, |acc, x| acc.checked_add(x))
    .reduce(|a, b| a.max(b))            // fold with no init
    .any(|x| x > 0)   .all(|x| x > 0)   // SHORT-CIRCUIT
    .find(|x| pred(x))   .position(|x| pred(x))
    .for_each(|x| do_it(x))
    .partition(|x| pred(x))             // into two
                                        // collections
    .unzip()

/* THE LAZINESS MATTERS: */
    let it = v.iter().map(|x| expensive(x));    // NOTHING
                                                // happens
    let first = it.take(1).collect::<Vec<_>>(); // ONE call to
                                                // expensive
/* And short-circuiting consumers stop early:               */
    v.iter().any(|x| *x < 0)            // stops at the first
                                        // negative

/* COLLECT IS POLYMORPHIC in its target: */
    let v: Vec<i32>              = it.collect();
    let s: String                = chars.collect();
    let m: HashMap<K, V>         = pairs.collect();
    let r: Result<Vec<i32>, E>   = results.collect();  // ch. 07
    let (a, b): (Vec<_>, Vec<_>) = it.unzip();
```

WHY ITERATORS BEAT INDEX LOOPS

An index loop needs a bounds check per access, and the compiler can only remove it if it can prove the index is in range. An iterator *is* the proof — it never produces an out-of-range index — so there is nothing to check. The result is that `a.iter_mut().zip(b).for_each(|(x, y)| *x += y)` vectorises and `for i in 0..n { a[i] += b[i] }` may not. On embedded this is the difference between idiomatic and fast, and it is the same code.

Closures

```
    let add = |a, b| a + b;
    let double = |x: i32| -> i32 { x * 2 };

    let factor = 3;
    let scale = |x| x * factor;         // CAPTURES factor

/* THREE TRAITS, by what the closure does with its captures:

     FnOnce   consumes them - callable once
     FnMut    mutates them  - callable repeatedly, needs
              &mut
     Fn       only reads them - callable repeatedly, from &

   Every closure implements FnOnce; those that do not consume
   also implement FnMut; those that do not mutate also
   implement Fn. So `Fn: FnMut: FnOnce`.

    fn call_once<F: FnOnce()>(f: F) { f(); }
    fn call_many<F: FnMut()>(mut f: F) { f(); f(); }
    fn call_shared<F: Fn()>(f: &F) { f(); f(); }

/* CAPTURE MODE IS INFERRED - by reference if that suffices,
   by mutable reference if needed, by value if required.
   `move` forces capture by value:                        */

    let data = vec![1, 2, 3];
    let closure = move || println!("{data:?}");
    // data is MOVED into the closure - required for
    // thread::spawn (chapter 13), because the closure may
    // outlive this scope

/* A CLOSURE IS A STRUCT with one field per capture, plus an
   impl of the call trait - exactly like a C++ lambda
   (C++ chapter 09). A closure capturing nothing is
   zero-sized and coerces to a plain function pointer:    */

    let f: fn(i32) -> i32 = |x| x + 1;
    register_callback(f);       // usable as a C callback
                                // (chapter 12)

/* PASSING CLOSURES:
     fn f<F: Fn(u32)>(cb: F)            // generic - INLINED,
                                        // zero cost
     fn f(cb: &dyn Fn(u32))             // trait object - an
                                        // indirect call
     fn f(cb: Box<dyn Fn(u32)>)         // heap - avoid on
                                        // embedded

   The first is what you want. The same static-versus-dynamic
   trade as C++'s template parameter versus std::function
   (C++ chapter 14).                                       */
```

Collections

Type	Allocates	Embedded
`[T; N]`	No — inline	**Use freely**

Type	Allocates	Embedded
`&[T]`, `&mut [T]`	No — a view	**Use freely**
`&str`	No	**Use freely**
`Vec<T>`	Yes	Needs `alloc`
`String`	Yes	Needs `alloc`
`VecDeque`, `BTreeMap`, `HashMap`	Yes	Needs `alloc`; `HashMap` needs `std`
`heapless::Vec<T, N>`	**No**	**The embedded answer**
`heapless::String<N>`	No	Same
`heapless::spsc::Queue`	No	Lock-free, for ISR-to-main

```
/* THE heapless CRATE gives you the standard interfaces with
   the capacity in the type (chapter 06's const generics): */

    use heapless::Vec;

    let mut v: Vec<u8, 64> = Vec::new();
    v.push(1).unwrap();                 // Result - it can be
                                        // FULL
    v.extend_from_slice(&[2, 3])?;
    let s: &[u8] = &v;                  // derefs to a slice
    for x in &v { }                     // iterates like Vec

    use heapless::String;
    let mut s: String<32> = String::new();
    write!(s, "value: {}", x).ok();     // core::fmt::Write

    use heapless::spsc::Queue;
    static mut Q: Queue<Event, 16> = Queue::new();
    let (mut prod, mut cons) = unsafe { Q.split() };
    // prod goes to the ISR, cons to the main loop -
    // lock-free, no critical section (chapter 13)

/* THE KEY DIFFERENCE FROM std: push returns Result, because
   a fixed-capacity container can be full. That is the
   BOUNDED FAILURE you want on embedded, instead of an
   allocation that might fail or a reallocation that costs
   unpredictable time.                                    */
```

Slices

```
    let a = [1, 2, 3, 4, 5];
    let s: &[i32] = &a;
    let mid = &a[1..4];         // [2, 3, 4]
    let head = &a[..2];
    let tail = &a[2..];

/* A SLICE IS A FAT POINTER - a pointer and a length, 16
   bytes. Exactly std::span (C++ chapter 08), and the same
   advice: take &[T] in parameters, not &Vec<T>.

/* THE METHODS worth knowing: */
    s.len()  s.is_empty()  s.first()  s.last()
    s.get(i)                        // Option - NO PANIC
    s.split_at(mid)  s.split_at_mut(mid)
    s.chunks(n)  s.chunks_exact(n)  s.windows(n)
    s.iter().rev()
    s.contains(&x)  s.starts_with(&pre)
    s.binary_search(&x)             // Result<index,
                                    // insertion_point>
    s.sort()  s.sort_unstable()     // unstable does NOT
                                    // allocate - prefer it
    s.copy_from_slice(&other)       // lengths must MATCH
    s.fill(0)
    s.rotate_left(n)
    s.split_first()  s.split_last()

/* PARSING A BUFFER, zero-copy: */
    fn parse(buf: &[u8]) -> Option<(&[u8; 4], &[u8])> {
        let (h, rest) = buf.split_first_chunk::<4>()?;
        Some((h, rest))
    }
/* No allocation, no copying, and the lifetime ties the
   result to the buffer (chapter 04).

/* AND THE BYTE CONVERSIONS, which embedded code needs
   constantly: */
    u32::from_le_bytes(bytes.try_into().ok()?)
    u32::from_be_bytes(...)
    value.to_le_bytes()                 // -> [u8; 4]
    s.copy_from_slice(&value.to_be_bytes());
/* Explicit endianness, checked length. Compare the C
   memcpy-and-hope, or the pointer cast that is undefined
   behaviour (compilers chapter 15).                     */
```

The generated code

```
/* --- A CHAIN COMPILES TO ONE LOOP --- */
    fn sum_evens(v: &[i32]) -> i32 {
        v.iter().filter(|x| *x % 2 == 0).sum()
    }
/* -O2 aarch64: one loop, no closure objects, no
   intermediate collection - and it vectorises. The
   adapters are structs whose next() methods inline into
   one another completely.

/* --- BOUNDS CHECKS DISAPPEAR --- */
    fn add_indexed(a: &mut [i32], b: &[i32]) {
        for i in 0..a.len() { a[i] += b[i]; }
                               // b[i] IS bounds-checked -
                               // the compiler cannot prove
                               // b is long enough
    }
    fn add_iter(a: &mut [i32], b: &[i32]) {
        for (x, y) in a.iter_mut().zip(b) { *x += *y; }
                               // NO checks - zip stops at
                               // the shorter
    }
/* The second is shorter, safer AND faster. That combination
   is unusual and worth internalising.

/* --- VERIFY IT --- */
    cargo asm my_crate::sum_evens --rust
    cargo bloat --release -n 20
    # or godbolt with the Rust compiler

/* --- AND THE ONE THING TO WATCH --- */
    let v: Vec<i32> = it.collect();     // ALLOCATES
    let n = it.count();                 // does not
/* `collect` is where a chain stops being free. On embedded,
   collect into a heapless::Vec (which returns Result), or
   consume with fold/sum/for_each and never materialise the
   intermediate at all.                                    */
```

How it shows up

Symptom	Cause and fix
"use of moved value" after a `for` loop	`for x in v` consumed it — use `&v`
"cannot borrow as mutable" inside a loop over the same collection	Use `iter_mut`, `retain`, or collect indices first
An iterator chain does nothing	Adapters are lazy — it was never consumed
"type annotations needed" on `collect`	Say the target: `collect::<Vec<_>>()`
"expected closure, found a different closure"	Every closure has a unique type — use a generic parameter or `impl Fn`
"cannot move out of captured variable"	The closure needs `move`, or is `FnOnce` where `Fn` is required
Closure will not convert to a function pointer	It captures — put the state in a context parameter
Unexpected allocation	`collect`, `Box`, `String`, or `sort` (stable)
Index loop slower than expected	Bounds checks — use iterators

Symptom	Cause and fix
`heapless::Vec::push` returns `Err`	Full — the bounded failure, by design
`copy_from_slice` panics	Lengths differ — check first, or use `chunks`

Check yourself

- ✓ Implement `Iterator` for a custom type and say what you get free.
- ✓ Distinguish `iter`, `iter_mut` and `into_iter`.
- ✓ Name fifteen adapters and ten consumers.
- ✓ Explain laziness and short-circuiting with an example of each.
- ✓ Explain why iterators eliminate bounds checks and index loops may not.
- ✓ Name the three closure traits and their relationship.
- ✓ Say when `move` is required and what a captureless closure becomes.
- ✓ Use `heapless` containers and explain the `Result` from `push`.
- ✓ Parse a byte buffer zero-copy with slice methods and explicit endianness.
- ✓ Verify that an iterator chain compiles to one loop with no allocation.

`Box`, `Rc`, `RefCell` and interior mutability

When the ownership rules from chapters 02 and 03 are too rigid for the design, these types relax them — some at compile time, some by moving the check to run time. Knowing which is which is the whole chapter.

Box

```
    let b = Box::new(42);       // one value, on the heap
    let v = *b;                 // deref

/* THE SAME THING AS std::unique_ptr (C++ chapter 10): one
   owner, freed on drop, zero overhead beyond the allocation
   itself. sizeof(Box<T>) == sizeof(&T).

/* THE THREE REASONS TO USE IT:

/* 1. A RECURSIVE TYPE, which otherwise has infinite size: */
    enum List { Cons(i32, Box<List>), Nil }
    // without Box: "recursive type has infinite size"

/* 2. A TRAIT OBJECT that must be owned: */
    let sensors: Vec<Box<dyn Sensor>> = vec![
        Box::new(Thermistor::new(0)),
        Box::new(Photodiode::new(1)),
    ];

/* 3. MOVING A LARGE VALUE cheaply - moving a Box moves one
      pointer, not the payload.

/* AND THE CONVERSIONS that matter for FFI (chapter 12): */
    let raw: *mut T = Box::into_raw(b);  // give up ownership
    let b = unsafe { Box::from_raw(raw) };// take it back
/* into_raw LEAKS unless something later calls from_raw. That
   is exactly how you hand a Rust-allocated object to C and
   get it back.                                            */
```

Interior mutability

```
THE PROBLEM: sometimes you need to mutate through a shared
reference. A cache, a refcount, a peripheral behind a
global.

THE RULE (chapter 03) says &T means no mutation. Interior
mutability types are the sanctioned exception: they
provide `fn set(&self, ...)` and uphold the aliasing
guarantee some other way.

  UnsafeCell<T>   the PRIMITIVE. The only legal way to get
                  &mut from &. Everything below is built
                  on it.
  Cell<T>         get/set by VALUE for Copy types. No
                  references handed out, so no aliasing
                  question. Zero cost.
  RefCell<T>      hands out &T and &mut T, with the
                  borrow rule checked AT RUN TIME.
                  PANICS on violation.
  Mutex<T>        the same, across threads, by blocking.
  RwLock<T>       many readers or one writer, across
                  threads.
  Atomic*         lock-free single values.

THE PATTERN: the compile-time check is moved to run time,
or to a lock. You pay either a panic risk or a
synchronisation cost, and you get flexibility.

THE CHOICE, in one line:
  Copy value, single thread    -> Cell
  needs references, single
    thread                     -> RefCell
  across threads               -> Mutex, RwLock or Atomic
  performance-critical single
    value                      -> Atomic
```

```
/* --- Cell: no borrows, so no possible failure --- */
    use core::cell::Cell;

    struct Counter { count: Cell<u32> }

    impl Counter {
        fn increment(&self) {           // note: &self, not
                                        // &mut self
            self.count.set(self.count.get() + 1);
        }
    }
/* Cell::get COPIES the value out; set replaces it. No
   reference to the interior ever exists, so there is nothing
   to alias. Zero cost - it compiles to a plain load and
   store.

/* --- RefCell: borrows, checked at run time --- */
    use core::cell::RefCell;

    let c = RefCell::new(vec![1, 2, 3]);

    {
        let r = c.borrow();             // Ref<Vec<i32>>
        println!("{}", r.len());
    }                                   // released here

    c.borrow_mut().push(4);             // RefMut

    let a = c.borrow();
    let b = c.borrow_mut();             // PANIC: already
                                        // borrowed

    // the non-panicking forms:
    if let Ok(mut m) = c.try_borrow_mut() { m.push(5); }

/* RefCell KEEPS A COUNTER of active borrows and panics if
   the rule is violated. So the guarantee is preserved - you
   just find out at run time instead of compile time.

   COST: one usize per RefCell, and a check per borrow.

   ON EMBEDDED: a panic in a RefCell is a bug that reaches
   the field. Prefer designs where the compiler can check;
   use try_borrow when a runtime failure is genuinely
   possible.                                              */
```

`UNSAFECELL` IS THE ONLY LEGAL ROUTE

Obtaining `&mut T` from `&T` by any other means — a pointer cast, `transmute` — is undefined behaviour, because the compiler emits `noalias` on shared references and will optimise on that assumption. `UnsafeCell` is the marker that tells it not to. Every safe interior-mutability type wraps one, and any `unsafe` code of your own that mutates through a shared reference must too.

Rc and Arc

```
    use std::rc::Rc;            // single-threaded
    use std::sync::Arc;         // Atomic - across threads

    let a = Rc::new(vec![1, 2, 3]);
    let b = Rc::clone(&a);      // refcount 2 - NOT a deep
                                // copy
    println!("{}", Rc::strong_count(&a));   // 2

/* SHARED OWNERSHIP: the value lives until the last Rc drops.
   The same as std::shared_ptr (C++ chapter 10), with the
   same caveats:

     · Rc is NOT thread-safe (no atomics) - and the compiler
       enforces that, because Rc is not Send (chapter 13).
       Arc is, and pays for atomic refcounting.
     · CYCLES LEAK. Two Rcs pointing at each other never
       reach zero.
     · Rc gives you &T only. For mutation you need
       Rc<RefCell<T>> - which is the shape most "I need
       shared mutable state" designs end up in, and which
       should prompt you to reconsider the design.

/* BREAKING A CYCLE with Weak: */
    use std::rc::{Rc, Weak};

    struct Node {
        children: RefCell<Vec<Rc<Node>>>,
        parent: RefCell<Weak<Node>>,    // NOT Rc
    }

    if let Some(p) = node.parent.borrow().upgrade() {
        // Weak::upgrade returns Option<Rc<T>> - None if the
        // target is gone
    }

/* ON EMBEDDED: Rc and Arc need `alloc`, and the atomic
   refcounting in Arc is a real cost. Ask first whether:
     · one clear owner plus borrowed references works
     · a static arena with indices works (chapter 04)
     · the sharing is actually needed
   Reaching for Rc<RefCell<T>> is usually a sign the
   ownership design has not been decided.               */
```

Globals on embedded

```
/* THE PROBLEM: a peripheral is a global, and an interrupt
   handler needs to reach it. `static mut` is unsafe to
   touch, and as of Rust 2024 taking a reference to one is a
   hard error.                                              */

/* --- THE OLD WAY, now forbidden --- */
    static mut UART: Option<Uart> = None;
    unsafe { UART.as_mut().unwrap().write(b"x"); }
                                // undefined behaviour if an
                                // interrupt races

/* --- 1. Mutex<RefCell<T>> with a critical section --- */
    use cortex_m::interrupt::{self, Mutex};
    use core::cell::RefCell;

    static UART: Mutex<RefCell<Option<Uart>>> =
        Mutex::new(RefCell::new(None));

    fn init(uart: Uart) {
        interrupt::free(|cs| {
            UART.borrow(cs).replace(Some(uart));
        });
    }

    #[interrupt]
    fn USART1() {
        interrupt::free(|cs| {
            if let Some(u) = UART.borrow(cs).borrow_mut()
                                 .as_mut() {
                u.handle_irq();
            }
        });
    }

/* `cortex_m::interrupt::Mutex` is NOT a blocking mutex - it
   requires a CriticalSection token, proving interrupts are
   disabled. So access is sound by construction, and the
   token costs nothing at run time. This is the standard
   pattern in the embedded ecosystem.

/* --- 2. StaticCell: initialise once, get &'static mut --- */
    use static_cell::StaticCell;

    static BUFFER: StaticCell<[u8; 1024]> = StaticCell::new();

    let buf: &'static mut [u8; 1024] = BUFFER.init([0; 1024]);
/* Panics if called twice. Gives you a 'static mutable
   reference without a heap - which is exactly what a DMA
   buffer or a network stack needs (chapter 16).

/* --- 3. Atomics, for simple flags and counters --- */
    use core::sync::atomic::{AtomicU32, AtomicBool, Ordering};

    static COUNT: AtomicU32 = AtomicU32::new(0);
    static READY: AtomicBool = AtomicBool::new(false);

    #[interrupt]
    fn TIMER() {
        COUNT.fetch_add(1, Ordering::Relaxed);
        READY.store(true, Ordering::Release);
```

```
    }

    fn main_loop() {
        if READY.load(Ordering::Acquire) {
            // sees everything the ISR wrote before the store
            // (chapter 13)
        }
    }
/* No critical section, no panic risk, no allocation. For a
   flag or a counter this is strictly the best option.

/* --- 4. RTIC, which removes the question entirely --- */
    #[app(device = stm32f4xx_hal::pac)]
    mod app {
        #[shared] struct Shared { uart: Uart }
        #[task(binds = USART1, shared = [uart])]
        fn on_uart(mut cx: on_uart::Context) {
            cx.shared.uart.lock(|u| u.handle_irq());
        }
    }
/* RTIC computes the required critical sections at COMPILE
   time from the priority ceiling, so the lock is often free
   (chapter 16).                                          */
```

Deref and custom smart pointers

```
    use core::ops::{Deref, DerefMut};

    struct Guard<'a, T> { inner: &'a mut T }

    impl<T> Deref for Guard<'_, T> {
        type Target = T;
        fn deref(&self) -> &T { self.inner }
    }
    impl<T> DerefMut for Guard<'_, T> {
        fn deref_mut(&mut self) -> &mut T { self.inner }
    }

    impl<T> Drop for Guard<'_, T> {
        fn drop(&mut self) { release_hardware(); }
    }

    let g = acquire();          // returns a Guard
    g.method();                 // DEREF COERCION - calls
                                // T::method
                                // released when g drops

/* DEREF COERCION is why `&String` works where `&str` is
   wanted, and `&Vec<T>` where `&[T]` is - and why you should
   take the slice types in parameters (chapter 03).

/* THE RULE FOR IMPLEMENTING Deref: only for types that ARE a
   pointer or a guard. Using it to fake inheritance produces
   confusing method resolution, exactly as in C++
   (C++ chapter 06).

/* THE GUARD PATTERN IS THE USEFUL ONE, and it is how Mutex
   works: lock() returns a guard, the guard derefs to the
   data, and dropping it unlocks. RAII plus Deref gives you
   an API that cannot be used without holding the lock.  */
```

Choosing

Need	Use	Cost
One owner, heap	`Box<T>`	The allocation
Mutate a Copy value through `&`	`Cell<T>`	None
Mutate anything through `&`, one thread	`RefCell<T>`	A counter, and a panic risk
Shared ownership, one thread	`Rc<T>`	Refcount, cycles leak
Shared ownership, threads	`Arc<T>`	Atomic refcount
Shared mutable, threads	`Arc<Mutex<T>>`	Allocation + locking
A flag or counter, ISR-shared	`Atomic*`	Near zero
A peripheral, ISR-shared	`Mutex<RefCell<T>>` + critical section	Interrupt latency
A one-time `'static` buffer, no heap	`StaticCell`	None
All of the above, managed	RTIC or Embassy	None — computed at compile time

How it shows up

Symptom	Cause and fix
Panic: "already borrowed: BorrowMutError"	Two `RefCell` borrows overlap — use `try_borrow_mut`, or restructure
Panic: "already mutably borrowed"	Same, the other way round
"recursive type has infinite size"	Add a `Box`
Memory grows steadily with `Rc`	A cycle — make one direction `Weak`
"`Rc<T>` cannot be sent between threads"	Correct — use `Arc`
"cannot borrow as mutable behind a `&`"	You need interior mutability, or `&mut self`
"creating a mutable reference to a `static mut`"	Forbidden in Rust 2024 — use `Mutex<RefCell>` or an atomic
Data corruption between an ISR and main	No critical section, or the wrong ordering on an atomic
`StaticCell::init` panics	Called twice — by design
Everything is `Rc<RefCell<T>>`	The ownership design needs revisiting

Check yourself

- ✓ Give the three reasons to use `Box` and its FFI conversions.
- ✓ Explain interior mutability and name the five types that provide it.
- ✓ Explain why `Cell` cannot fail and `RefCell` can.
- ✓ Explain why `UnsafeCell` is the only legal route, in terms of `noalias`.
- ✓ Contrast `Rc` with `Arc` and say how the compiler enforces the difference.
- ✓ Break a reference cycle with `Weak`.
- ✓ Give four ways to share a peripheral with an interrupt handler on embedded.
- ✓ Explain what the `CriticalSection` token proves and why it is free.
- ✓ Explain `StaticCell` and what it replaces.
- ✓ Implement a guard with `Deref` and `Drop`, and say what it guarantees.

Modules, crates and Cargo

Rust's code organisation and build system. After CMake, vendor IDEs and hand-written Makefiles, Cargo is the single feature most likely to make an embedded engineer want to switch — and the module system is where privacy and API boundaries live.

Modules

```
/* A module is a namespace and a PRIVACY BOUNDARY. Everything
   is private by default.                                    */

    mod uart {
        pub struct Uart { base: usize }     // the TYPE is
                                            // public
                                            // `base` is not

        impl Uart {
            pub fn new(base: usize) -> Self { Uart { base } }
            pub fn write(&mut self, b: u8) { self.put(b); }
            fn put(&mut self, b: u8) { /* private */ }
        }

        pub(crate) fn debug_dump() { }      // visible in this
                                            // CRATE only
        pub(super) fn helper() { }          // visible to the
                                            // PARENT module
        pub(in crate::drivers) fn x() { }   // a specific path
    }

    use uart::Uart;
    let mut u = Uart::new(0x4000_0000);
    u.write(b'x');
    u.put(b'y');                // ERROR - private

/* PRIVACY IS PER-MODULE, not per-type: a child module can
   see its ancestors' private items. So `mod tests` inside a
   module can test private functions - which is why Rust's
   unit tests live in the file they test.

/* FILE LAYOUT: */
    src/
      main.rs           the binary crate root
      lib.rs            the library crate root
      uart.rs           -> `mod uart;`
      drivers/
        mod.rs          -> `mod drivers;` (older style)
        spi.rs          -> `mod spi;` inside drivers
      drivers.rs        -> the 2018+ style: a sibling file
                           instead of drivers/mod.rs

    // in lib.rs
    mod uart;                   // private module
    pub mod drivers;            // public module
    pub use uart::Uart;         // RE-EXPORT: users write
                                // my_crate::Uart, not
                                // my_crate::uart::Uart

/* THE RE-EXPORT IS THE API DESIGN TOOL: organise the source
   however you like, and present a flat, deliberate public
   surface with `pub use`.

/* PATHS: */
    use crate::drivers::spi::Spi;       // absolute, from the
                                        // crate root
    use super::helper;                  // the parent module
    use self::inner::thing;             // this module
    use core::fmt::Write;               // another crate
```

```
    use heapless::{Vec, String};        // grouped
    use embedded_hal::digital::*;       // glob - use
                                        // sparingly, except
                                        // for preludes
    use core::result::Result as R;      // rename
```

Crates and workspaces

```
# A CRATE is a compilation unit: one library or one binary.
# A PACKAGE is one Cargo.toml, containing up to one library
# and any number of binaries.

    my-project/
      Cargo.toml
      src/
        lib.rs            the library crate
        main.rs           a binary crate
        bin/
          tool.rs         another binary: cargo run --bin tool
      tests/              INTEGRATION tests - one crate each
      benches/
      examples/           cargo run --example blink

# A WORKSPACE shares one target directory and one lock file
# across several packages - which is how a real firmware
# project is laid out:

    # Cargo.toml at the root
    [workspace]
    resolver = "2"
    members = ["firmware", "protocol", "hal-ext"]

    [workspace.dependencies]
    heapless = "0.8"
    embedded-hal = "1.0"

    # firmware/Cargo.toml
    [dependencies]
    protocol = { path = "../protocol" }
    heapless.workspace = true

# WHY IT MATTERS ON EMBEDDED: the protocol crate can be
# `no_std` and tested on the HOST with `cargo test`, while
# the firmware crate cross-compiles. That split - testable
# logic in a portable crate, hardware in a target crate - is
# the single most valuable structural decision in an embedded
# Rust project.
```

Cargo.toml

```
[package]
name = "firmware"
version = "0.1.0"
edition = "2021"                # 2015/2018/2021/2024 - opt-in
                                # language changes
rust-version = "1.75"           # the MSRV

[dependencies]
cortex-m = "0.7"
cortex-m-rt = "0.7"
embedded-hal = "1.0"
heapless = { version = "0.8", default-features = false }
nb = "1.0"
defmt = "0.3"

[dependencies.stm32f4xx-hal]
version = "0.21"
features = ["stm32f411", "rtic"]

[dev-dependencies]                      # tests and examples
                                        # only
proptest = "1"

[build-dependencies]                    # for build.rs
cc = "1"

[features]
default = ["logging"]
logging = ["dep:defmt"]
debug-uart = []

[profile.release]
opt-level = "z"                 # optimise for SIZE
                                # ("s" is less aggressive)
lto = "fat"                     # link-time optimisation
codegen-units = 1               # one unit - better
                                # optimisation, slower build
panic = "abort"                 # no unwinder (chapter 07)
debug = true                    # KEEP symbols - they cost
                                # nothing in flash, and
                                # defmt/probe-rs need them
overflow-checks = false

[profile.dev]
opt-level = 1                   # -O0 firmware is often too
                                # slow to run at all

# VERSION REQUIREMENTS:
#   "1.0"     ^1.0  - any 1.x, compatible
#   "=1.0.3"  exactly
#   "~1.0.3"  >=1.0.3, <1.1.0
#   "*"       anything - never do this
#
# Cargo.lock records the EXACT resolved versions.
# COMMIT IT for a binary; do NOT for a library.
```

> **THE RELEASE PROFILE IS THE SIZE LEVER**
>
> `opt-level = "z"`, `lto = "fat"`, `codegen-units = 1` and `panic = "abort"` together typically halve a firmware binary compared with the defaults. Set them before concluding Rust produces large binaries — the default release profile is tuned for server workloads, not for a 256 KB part.

Features

```
    [features]
    default = ["std"]
    std = ["alloc", "serde/std"]
    alloc = []
    logging = ["dep:defmt"]
    hardware-crc = []

    [dependencies]
    defmt = { version = "0.3", optional = true }

/* IN CODE: */
    #[cfg(feature = "logging")]
    fn log(msg: &str) { defmt::info!("{}", msg); }

    #[cfg(not(feature = "logging"))]
    fn log(_: &str) { }

    #[cfg(all(feature = "std", not(feature = "no-alloc")))]
    use std::vec::Vec;

/* AND THE STANDARD no_std SHAPE: */
    #![cfg_attr(not(feature = "std"), no_std)]

/* FEATURES MUST BE ADDITIVE. If two crates depend on yours
   with different features, Cargo UNIFIES them - enabling the
   union. So a feature that REMOVES functionality breaks that
   crate for everyone else in the build. Never write
   `no-floats`; write `floats` and leave it off by default.

/* THE OTHER cfg CONDITIONS: */
    #[cfg(target_arch = "arm")]
    #[cfg(target_os = "none")]
    #[cfg(target_pointer_width = "32")]
    #[cfg(debug_assertions)]
    #[cfg(test)]

/* AND ON THE COMMAND LINE: */
    cargo build --features "logging,hardware-crc"
    cargo build --no-default-features
    cargo build --all-features
```

The commands

```
# --- DAILY ---
    cargo check                 # TYPE-CHECK only - 5x faster
                                # than build. Use this while
                                # iterating.
    cargo build
    cargo build --release
    cargo run
    cargo test
    cargo clippy -- -D warnings # the linter. Genuinely
                                # excellent - it finds real
                                # bugs, not just style.
    cargo fmt                   # one format, no arguments
    cargo doc --open            # generate and view docs

# --- CROSS-COMPILATION ---
    rustup target add thumbv7em-none-eabihf
    cargo build --target thumbv7em-none-eabihf --release

    # .cargo/config.toml, so you can omit --target:
    [build]
    target = "thumbv7em-none-eabihf"

    [target.thumbv7em-none-eabihf]
    runner = "probe-rs run --chip STM32F411RETx"
    rustflags = ["-C", "link-arg=-Tlink.x"]

    # then `cargo run` FLASHES AND RUNS ON THE TARGET, with
    # defmt output streaming back. That single line of
    # configuration replaces an IDE.

# --- EMBEDDED TOOLING ---
    cargo install probe-rs-tools cargo-binutils cargo-bloat \
                  flip-link cargo-embed
    rustup component add llvm-tools

    cargo size --release -- -A          # section sizes
    cargo bloat --release -n 20         # what is taking space
    cargo nm --release
    cargo objdump --release -- -d
    probe-rs run --chip STM32F411RETx target/.../firmware

# --- MAINTENANCE ---
    cargo update                # update within the
                                # constraints
    cargo tree                  # the dependency graph
    cargo tree -d               # DUPLICATE versions - a
                                # common cause of bloat
    cargo audit                 # known vulnerabilities
    cargo deny check            # licences, bans, advisories
    cargo outdated
    cargo expand                # show macro expansions -
                                # invaluable for debugging
                                # derive macros
```

Testing

```
/* UNIT TESTS live beside the code, and can see private
   items: */
    fn parse(s: &str) -> Option<u32> { ... }

    #[cfg(test)]
    mod tests {
        use super::*;

        #[test]
        fn parses_decimal() {
            assert_eq!(parse("42"), Some(42));
        }

        #[test]
        fn rejects_garbage() {
            assert!(parse("x").is_none());
        }

        #[test]
        #[should_panic(expected = "index out of range")]
        fn panics_on_bad_index() { lookup(999); }

        #[test]
        fn returns_result() -> Result<(), Error> {
            let v = fallible()?;         // `?` works in tests
            assert_eq!(v, 1);
            Ok(())
        }
    }

/* DOC TESTS - examples in documentation that are COMPILED
   AND RUN: */
    /// Converts a raw ADC reading to millivolts.
    ///
    /// # Examples
    /// ```
    /// use my_crate::to_mv;
    /// assert_eq!(to_mv(2048), 1650);
    /// ```
    pub fn to_mv(raw: u16) -> u16 { ... }

/* DOCUMENTATION THAT CANNOT GO STALE, because `cargo test`
   runs it. This is one of Rust's genuinely distinctive
   features and it is worth using everywhere.

/* THE EMBEDDED TESTING STRATEGY: */
    1. put the LOGIC in a no_std crate with no hardware
       dependencies - protocol parsing, state machines,
       filters, checksums
    2. `cargo test` it on the HOST, where you have a debugger
       and instant feedback
    3. keep the hardware layer thin, and test it on target
       with defmt-test or embedded-test
    4. use traits (chapter 06) so the logic can be tested
       against a MOCK implementation:

    #[cfg(test)]
    struct MockSpi { written: Vec<u8> }
    impl SpiDevice for MockSpi { ... }
```

```
    #[test]
    fn sends_init_sequence() {
        let mut spi = MockSpi::default();
        let mut d = Display::new(&mut spi);
        d.init().unwrap();
        assert_eq!(spi.written, &[0x11, 0x29]);
    }
/* THIS IS THE BIGGEST PRACTICAL WIN in embedded Rust: most
   of your firmware becomes testable on a laptop, in
   milliseconds, without hardware.                       */
```

build.rs and documentation

```
/* build.rs runs BEFORE compilation - for generated code,
   linker scripts, or compiling C.                          */

    // build.rs
    use std::{env, fs, path::PathBuf};

    fn main() {
        let out = PathBuf::from(env::var("OUT_DIR").unwrap());

        // put the linker script where the linker will find it
        fs::write(out.join("memory.x"),
                  include_bytes!("memory.x")).unwrap();
        println!("cargo:rustc-link-search={}", out.display());
        println!("cargo:rerun-if-changed=memory.x");

        // compile a C file from the vendor SDK (chapter 12)
        cc::Build::new()
            .file("vendor/hal.c")
            .include("vendor/include")
            .compile("vendorhal");
    }

/* THE `cargo:` DIRECTIVES it can emit:
     rustc-link-lib, rustc-link-search, rustc-cfg,
     rustc-env, rerun-if-changed, rerun-if-env-changed,
     warning

/* DOCUMENTATION: */
    //! Crate-level documentation, at the top of lib.rs.

    /// Documentation for the item that follows.
    ///
    /// # Errors
    /// Returns [`Error::Timeout`] if the device does not
    /// respond within 10 ms.
    ///
    /// # Safety
    /// (for unsafe fn) The caller must guarantee ...
    ///
    /// # Panics
    /// Panics if `len` exceeds the buffer capacity.
    pub fn read(&mut self, len: usize) -> Result<&[u8], Error>

    #![deny(missing_docs)]      // require docs on every
                                // public item

    cargo doc --open
/* The generated documentation links types, shows source, and
   runs the examples. `#[deny(missing_docs)]` on a library
   crate is worth adopting from the first commit.        */
```

How it shows up

Symptom	Cause and fix
"module not found"	Missing `mod x;` in the parent, or the file is misnamed
"function is private"	Add `pub`, or `pub(crate)` if it is internal

Symptom	Cause and fix
"unresolved import"	Wrong path — remember `crate::`, `super::`, `self::`
A trait method is "not found"	The trait is not in scope (ch. 06)
Two versions of the same crate linked	`cargo tree -d` — align the version requirements
A feature enabled unexpectedly	Feature unification — check `cargo tree -f`
Disabling a feature breaks another crate	The feature is subtractive — it should be additive
Release binary far larger than expected	Default profile — set `opt-level="z"`, LTO, `panic="abort"`
Builds are slow	Use `cargo check`; raise `codegen-units` in dev
`cargo test` fails to link on an embedded target	Tests need a host target — put logic in a portable crate
Linker script not found	`build.rs` must copy it to `OUT_DIR` and add a link-search path

Check yourself

- ✓ Explain module privacy, including what a child module can see.
- ✓ Use `pub use` to design a public API independent of the file layout.
- ✓ Distinguish a crate, a package and a workspace.
- ✓ Lay out a firmware project so the logic is host-testable.
- ✓ Write a release profile tuned for size and say what each setting does.
- ✓ Explain why features must be additive and what happens if they are not.
- ✓ Configure `.cargo/config.toml` so `cargo run` flashes the target.
- ✓ Write unit tests, a doc test, and a test using a mock trait implementation.
- ✓ Use `cargo size`, `cargo bloat` and `cargo tree -d`.
- ✓ Write a `build.rs` that installs a linker script and compiles a C file.

Unsafe Rust and the unsafe boundary

Hardware access requires operations the compiler cannot verify. `unsafe` is where you take responsibility for them — and the discipline that makes Rust worthwhile is confining it behind safe abstractions rather than spreading it through the codebase.

What unsafe actually permits

```
`unsafe` enables exactly FIVE things:

  1. dereference a RAW POINTER (*const T, *mut T)
  2. call an `unsafe fn` (including every extern "C"
     function)
  3. access or modify a `static mut`
  4. implement an `unsafe trait` (Send, Sync, and your own)
  5. access a union field

IT DOES NOT:
  · turn off the borrow checker
  · turn off lifetimes
  · turn off type checking
  · make anything faster
  · permit undefined behaviour

THAT LAST POINT IS THE ONE PEOPLE GET WRONG. `unsafe` does
not grant permission to break the rules; it moves
RESPONSIBILITY FOR UPHOLDING THEM from the compiler to you.
Breaking them inside `unsafe` is still undefined behaviour,
with all the consequences from compilers chapter 15 - and
worse, because the rest of the program was optimised on
the assumption that you did not.

THE PROPERTIES YOU MUST UPHOLD:
  · no dangling or misaligned pointer dereferences
  · no aliasing violations: never two &mut, never &mut
    alongside &
  · no data races
  · every value must be VALID for its type - no bool that
    is 2, no char that is not a scalar value, no
    uninitialised reads
  · no breaking of any safe abstraction's invariants
```

Raw pointers

```
    let x = 42;
    let p: *const i32 = &x;
    let mut y = 42;
    let q: *mut i32 = &mut y;

    // creating them is SAFE; dereferencing is not
    unsafe { println!("{}", *p); }
    unsafe { *q = 43; }

/* RAW POINTERS, unlike references:
     · may be null
     · may dangle
     · have no lifetime
     · may alias freely
     · are not automatically dereferenced

/* THE OPERATIONS worth knowing: */
    core::ptr::null::<u8>()
    core::ptr::null_mut::<u8>()
    p.is_null()
    p.offset(3)                // in ELEMENTS, and it is UB
                               // to go out of bounds
    p.add(3)  p.sub(3)
    p.wrapping_add(3)          // no UB, but the result may
                               // not be dereferenceable

    unsafe { core::ptr::read(p) }
    unsafe { core::ptr::write(q, 5) }
    unsafe { core::ptr::read_volatile(p) }      // for MMIO
    unsafe { core::ptr::write_volatile(q, 5) }
    unsafe { core::ptr::copy(src, dst, n) }     // memmove
    unsafe { core::ptr::copy_nonoverlapping(src, dst, n) }
                                                // memcpy
    unsafe { core::ptr::swap(a, b) }

/* ALIGNMENT MATTERS AND IS CHECKED IN DEBUG BUILDS: */
    let buf: [u8; 8] = [0; 8];
    let p = buf.as_ptr() as *const u32;
    unsafe { *p }              // UB if buf is not 4-aligned
    // correct:
    unsafe { p.read_unaligned() }
    // or, better, no unsafe at all:
    u32::from_le_bytes(buf[0..4].try_into().unwrap())

/* AND `addr_of!` FOR UNALIGNED OR PACKED FIELDS - creating
   a reference to a misaligned field is UB even without
   dereferencing it: */
    use core::ptr::addr_of;
    let p = unsafe { addr_of!((*packed_ptr).field) };
    let v = unsafe { p.read_unaligned() };
```

The unsafe boundary

```
THE CENTRAL DISCIPLINE:

  unsafe code should be SMALL, LOCAL, and WRAPPED IN A SAFE
  API whose invariants you can state and check.

The goal is not to eliminate unsafe — hardware access
requires it. The goal is that the REST of the program
never needs it, so that a memory-safety bug can only be in
the few hundred lines you audited.

BAD:                          GOOD:
┌──────────────────────┐      ┌──────────────────────┐
│ unsafe everywhere    │      │   safe application   │
│ every driver, every  │      ├──────────────────────┤
│ access, no           │      │   safe driver API    │
│ invariants written   │      ├──────────────────────┤
│ down                 │      │ unsafe: 20 lines,    │
│                      │      │ documented, tested   │
└──────────────────────┘      └──────────────────────┘

THE TEST: can you state, in one sentence, what the caller
must guarantee? If yes, that sentence is the safety
comment, and the wrapper can be safe. If no, the
abstraction is wrong.
```

```
/* --- A SAFE MMIO WRAPPER, the canonical example --- */

pub struct Uart {
    base: *mut u32,
}

// SAFETY: the caller of `new` guarantees exclusive access to
// this peripheral, so sending it between contexts is sound.
unsafe impl Send for Uart {}

impl Uart {
    /// # Safety
    /// `base` must be the address of a UART peripheral of
    /// this type, correctly clocked, and no other Uart must
    /// exist for the same address.
    pub const unsafe fn new(base: usize) -> Self {
        Uart { base: base as *mut u32 }
    }

    // SAFE from here on - the constructor took the promise
    pub fn write_byte(&mut self, b: u8) {
        while self.flags() & FR_TXFF != 0 {}
        // SAFETY: base is valid by the constructor's
        // contract; DR is within the peripheral's register
        // block
        unsafe { self.base.add(REG_DR)
                          .write_volatile(b as u32) };
    }

    fn flags(&self) -> u32 {
        // SAFETY: as above
        unsafe { self.base.add(REG_FR).read_volatile() }
    }
}

/* TWO UNSAFE BLOCKS, each one line, each with a SAFETY
   comment. Everything above is safe, and a user of this
   driver writes no unsafe at all.

/* THE CONVENTIONS the ecosystem follows, and clippy enforces:
     · `# Safety` doc section on every `unsafe fn`, stating
       what the CALLER must guarantee
     · `// SAFETY:` comment on every `unsafe` BLOCK, stating
       why the requirements hold HERE
     · #![deny(unsafe_op_in_unsafe_fn)] - so an unsafe fn
       still needs explicit blocks inside, rather than being
       unsafe throughout
     · #![warn(clippy::undocumented_unsafe_blocks)]        */
```

THE TWO COMMENT KINDS, AND WHY BOTH

`# Safety` on a function is a *precondition*: what the caller must ensure. `// SAFETY:` on a block is a *proof*: why the preconditions are met at this site. A codebase with both can be audited by reading only the unsafe blocks and checking each proof — which is exactly the property that makes the guarantee worth anything.

Hardware access patterns

```
/* --- 1. VOLATILE, always, for MMIO --- */
    unsafe { ptr.write_volatile(v) }
    unsafe { ptr.read_volatile() }
/* A plain `*p = v` may be elided, reordered or merged - the
   same rule as C's volatile (C++ chapter 02). Rust has no
   `volatile` type qualifier; it has volatile OPERATIONS,
   which is the more honest design.

/* --- 2. USE A PAC, and write no unsafe at all --- */
    let p = pac::Peripherals::take().unwrap();
    p.gpioa.moder.modify(|_, w| w.moder5().output());
    p.gpioa.odr.write(|w| w.odr5().set_bit());
/* The svd2rust-generated PAC (chapter 16) contains the
   unsafe; you get a typed, field-named, safe API. This is
   what "confine unsafe to a small audited layer" looks like
   when the layer is generated from the vendor's own SVD
   file.

/* --- 3. A CRITICAL SECTION for shared state --- */
    critical_section::with(|cs| {
        SHARED.borrow(cs).borrow_mut().update();
    });
/* The `cs` token proves interrupts are disabled
   (chapter 09). No unsafe needed at the use site.

/* --- 4. A STATIC BUFFER without `static mut` --- */
    use static_cell::StaticCell;
    static BUF: StaticCell<[u8; 1024]> = StaticCell::new();
    let buf: &'static mut [u8; 1024] = BUF.init([0; 1024]);
/* Safe, panics on double init, no heap.

/* --- 5. TRANSMUTE: almost always the wrong answer --- */
    let f: f32 = unsafe { core::mem::transmute(bits) };  // no
    let f = f32::from_bits(bits);                        // yes
    let bytes = unsafe { core::mem::transmute::<u32,
                                    [u8;4]>(x) };        // no
    let bytes = x.to_le_bytes();                         // yes
    let s = unsafe { core::str::from_utf8_unchecked(b) }; // no
    let s = core::str::from_utf8(b)?;                    // yes
/* transmute is the most dangerous function in Rust: it
   reinterprets bits with NO checking of size, validity or
   alignment. There is nearly always a safe alternative, and
   where there is not, `bytemuck` or `zerocopy` provide
   audited, trait-gated versions.

/* --- 6. slice::from_raw_parts, for a DMA buffer --- */
    // SAFETY: the DMA engine has finished (we checked the
    // completion flag), `ptr` points to `len` initialised
    // bytes within BUFFER, and no other reference to that
    // region exists.
    let data = unsafe {
        core::slice::from_raw_parts(ptr, len)
    };
/* One of the legitimate uses. Note the safety comment covers
   validity, initialisation, length AND aliasing - all four
   are required.                                          */
```

Unions and uninitialised memory

```
/* --- UNIONS, for FFI compatibility --- */
    #[repr(C)]
    union Register {
        word: u32,
        bytes: [u8; 4],
    }
    let r = Register { word: 0x1234_5678 };
    let b = unsafe { r.bytes };         // reading ANY field
                                        // is unsafe
/* Prefer an enum (chapter 05) unless you are matching a C
   layout. For bit reinterpretation, to_le_bytes is safe and
   clearer.

/* --- MaybeUninit, for genuinely uninitialised memory --- */
    use core::mem::MaybeUninit;

    let mut buf: [MaybeUninit<u8>; 1024] =
        unsafe { MaybeUninit::uninit().assume_init() };
        // ^ sound ONLY because MaybeUninit<u8> has no
        //   validity requirement

    let n = read_into(&mut buf);

    // SAFETY: read_into initialised the first `n` elements
    let data = unsafe {
        core::slice::from_raw_parts(buf.as_ptr() as *const u8,
                                    n)
    };

/* NEVER `mem::uninitialized()` - it is deprecated and always
   UB for types with validity requirements. And note that
   reading uninitialised memory is UB even for u8: the value
   is "poison" (compilers chapter 15), not a random number.

/* THE SAFE ALTERNATIVE, and usually the right one: */
    let mut buf = [0u8; 1024];          // zeroing 1 KB costs
                                        // ~100 cycles
/* Measure before deciding the initialisation is too
   expensive. It usually is not.                          */
```

Verifying unsafe code

```
# --- MIRI: an interpreter that detects UB ---
    rustup +nightly component add miri
    cargo +nightly miri test

# It catches:
#   out-of-bounds accesses
#   use-after-free
#   invalid values (a bool that is 2)
#   misaligned accesses
#   ALIASING VIOLATIONS, under Stacked Borrows / Tree Borrows
#   data races
#   uninitialised reads
#   memory leaks
#
# MIRI IS THE MOST IMPORTANT TOOL IN THIS CHAPTER. It finds
# aliasing bugs that no amount of testing would, because the
# consequence of an aliasing violation is a wrong
# optimisation, not a crash.
#
# LIMITATION: it cannot run code that touches real hardware or
# calls into C. So structure the unsafe helpers so they can be
# tested against a mock on the host - the same argument as
# chapter 10's testing strategy.

# --- THE SANITIZERS ---
    RUSTFLAGS="-Zsanitizer=address" cargo +nightly test
    RUSTFLAGS="-Zsanitizer=thread"  cargo +nightly test

# --- CLIPPY LINTS worth denying ---
    #![deny(unsafe_op_in_unsafe_fn)]
    #![warn(clippy::undocumented_unsafe_blocks)]
    #![warn(clippy::multiple_unsafe_ops_per_block)]
    #![warn(clippy::missing_safety_doc)]
    #![warn(clippy::ptr_as_ptr, clippy::cast_ptr_alignment)]

# --- AND THE STRUCTURAL CHECK ---
    cargo geiger                # counts unsafe per dependency
    grep -rn 'unsafe' src/ | wc -l
# Track that number. In a well-structured embedded crate it
# should be in the tens and concentrated in a few files, not
# spread across every driver.
```

A review checklist

Check	Why
Is there a `# Safety` section on every `unsafe fn`?	The caller cannot uphold an unstated contract
Is there a `// SAFETY:` on every block?	It is the proof that makes the audit possible
Is the block as small as it can be?	Large blocks hide the operation that matters
Is every pointer non-null, aligned and in-bounds?	All three are separate UB conditions
Could two `&mut` to the same data exist?	The commonest and most damaging violation
Is the memory initialised before being read?	Uninitialised reads are UB even for `u8`
Are all values valid for their type?	A `bool` that is 2 is UB immediately
Does an interrupt or another core race here?	Data races are UB regardless of the outcome

Check	Why
Is `write_volatile` used for MMIO?	A plain write may be elided or reordered
Is there a safe alternative?	Usually yes — `from_bits`, `to_le_bytes`, a PAC
Does Miri pass?	It finds what testing cannot
Is the safe wrapper's invariant actually upheld?	A safe API over unsound unsafe is worse than no wrapper

How it shows up

Symptom	Cause and fix
"this operation is unsafe"	Wrap it in a block — and write the SAFETY comment
Works in debug, wrong in release	Aliasing violation the optimiser exploited — run Miri
Miri: "attempting a read access ... tag does not exist"	A pointer derived from a reference that has since been invalidated
Miri: "constructing invalid value"	A value outside its type's valid range
Misaligned access fault on ARM	Cast a `u8` pointer to a wider type — use `read_unaligned`
Register write has no effect	Not `volatile` — the compiler removed it
Corruption only with interrupts enabled	Missing critical section or atomic
Undefined behaviour after `transmute`	Sizes, validity or alignment mismatched — use a safe conversion
Safe API turns out to be unsound	The invariant was not actually enforced — the worst failure mode
`unsafe` appearing throughout the codebase	The abstraction boundary is missing

Check yourself

- ✓ Name the five things `unsafe` permits, and four it does not.
- ✓ List the properties you must uphold inside an unsafe block.
- ✓ Explain the unsafe boundary and the one-sentence test for a correct wrapper.
- ✓ Write a safe MMIO wrapper with correct `# Safety` and `// SAFETY:` comments.
- ✓ Explain why volatile operations rather than a volatile type.
- ✓ Give safe alternatives to `transmute` for four common cases.
- ✓ Explain `MaybeUninit` and why reading uninitialised memory is UB.
- ✓ Run Miri and say what it catches that testing cannot.
- ✓ Enable the four lints that enforce unsafe discipline.
- ✓ Work the twelve-point review checklist on a real unsafe block.

FFI: calling C and being called by C

Every real embedded Rust project sits on a C vendor SDK. This chapter is the boundary: how to call C safely, how to be called from C, and the layout, ownership and panic rules that make the seam sound rather than merely working.

Calling C

```
    // declare what the C side provides
    extern "C" {
        fn HAL_GPIO_WritePin(port: *mut GPIO_TypeDef,
                             pin: u16, state: u32);
        fn HAL_Delay(ms: u32);
        fn malloc(size: usize) -> *mut core::ffi::c_void;
    }

    // every call is unsafe - the compiler cannot check
    // anything about C
    unsafe { HAL_Delay(100); }

    // Rust 2024: `unsafe extern` makes the declaration itself
    // an unsafe act, which is more honest
    unsafe extern "C" {
        safe fn abs(x: i32) -> i32;     // and individual
                                        // functions may be
                                        // marked safe
        unsafe fn free(p: *mut c_void);
    }

/* THE C TYPES, from core::ffi: */
    c_char  c_schar  c_uchar
    c_short c_ushort c_int  c_uint
    c_long  c_ulong  c_longlong c_ulonglong
    c_float c_double
    c_void
    CStr    CString  (CString needs alloc)

/* USE THESE, not u8/i32 - c_char is i8 on x86 and u8 on ARM,
   and c_long is 32 bits on Windows and 64 on Linux
   (compilers chapter 10). Getting it wrong compiles and then
   misbehaves.

/* LINKING: */
    #[link(name = "vendorhal")]         // -lvendorhal
    extern "C" { ... }

    #[link(name = "m")]
    #[link(name = "static_thing", kind = "static")]

    // or from build.rs (chapter 10):
    println!("cargo:rustc-link-lib=static=vendorhal");
    println!("cargo:rustc-link-search=native=vendor/lib");
```

Layout

```
/* RUST REORDERS STRUCT FIELDS BY DEFAULT to minimise
   padding. For anything crossing the boundary, say
   #[repr(C)].                                          */

    #[repr(C)]
    pub struct Message {
        pub id: u32,
        pub length: u16,
        pub flags: u8,
        pub _pad: u8,               // explicit, so both sides
                                    // agree
        pub payload: [u8; 64],
    }
    const _: () = assert!(core::mem::size_of::<Message>() == 72);
    const _: () = assert!(core::mem::offset_of!(Message,
                                            payload) == 8);

/* THE `const _: () = assert!(...)` IDIOM is a compile-time
   check in a const context - the Rust equivalent of
   static_assert, and it should be on every FFI struct
   (C++ chapter 16 makes the same argument).

/* THE repr ATTRIBUTES: */
    #[repr(C)]              C-compatible layout
    #[repr(transparent)]    exactly like the single field -
                            for newtypes crossing FFI
    #[repr(packed)]         no padding. DANGEROUS - taking a
                            reference to a misaligned field is
                            UB (chapter 11)
    #[repr(align(64))]      minimum alignment
    #[repr(u8)] / #[repr(i32)]      a fixed enum
                                    discriminant
    #[repr(C, packed)]      combined

/* ENUMS ACROSS FFI: */
    #[repr(C)]
    #[derive(Clone, Copy, PartialEq)]
    pub enum Status { Ok = 0, Error = 1, Timeout = 2 }

/* BUT RECEIVING one from C is unsound if C can send a value
   outside the set - a Rust enum with an invalid discriminant
   is immediate UB. So take an integer and convert: */
    #[no_mangle]
    pub extern "C" fn handle(raw: u32) -> i32 {
        match Status::try_from(raw) {
            Ok(s) => process(s),
            Err(_) => -1,
        }
    }

/* AND NEWTYPES with repr(transparent), which cost nothing: */
    #[repr(transparent)]
    pub struct Handle(*mut c_void);
/* Passed in a register exactly like the raw pointer, but a
   distinct type on the Rust side.

/* AUTOMATE IT: bindgen generates all of this from the C
   headers (below), and it gets the platform-dependent types
   right.                                               */
```

Strings and slices

```
/* RUST STRINGS ARE UTF-8 AND NOT NUL-TERMINATED. C strings
   are NUL-terminated and of no particular encoding. The
   conversion is not free and can fail.                    */

    use core::ffi::{CStr, c_char};

/* --- C -> Rust --- */
    // SAFETY: `p` points to a NUL-terminated string that
    // remains valid for the duration of this call
    let s: &CStr = unsafe { CStr::from_ptr(p) };
    let bytes: &[u8] = s.to_bytes();
    let text: &str = s.to_str()?;       // FAILS on invalid
                                        // UTF-8

/* --- Rust -> C --- */
    // a literal, NUL-terminated at compile time:
    let s = c"hello";                   // Rust 1.77 - &CStr
    unsafe { c_puts(s.as_ptr()); }

    // or, with alloc:
    let owned = CString::new("hello")?; // fails if it
                                        // contains a NUL
    unsafe { c_puts(owned.as_ptr()); }
    // `owned` MUST outlive the call - this is wrong:
    unsafe { c_puts(CString::new("x")?.as_ptr()); }
                                // the CString is dropped at
                                // the end of the statement,
                                // leaving a dangling pointer

/* --- SLICES --- */
    // C -> Rust
    #[no_mangle]
    pub extern "C" fn process(data: *const u8, len: usize)
            -> i32
    {
        if data.is_null() { return -1; }
        // SAFETY: the caller guarantees `data` points to
        // `len` initialised bytes that are not mutated for
        // the duration of this call
        let s = unsafe { core::slice::from_raw_parts(data,
                                                     len) };
        do_work(s)
    }

    // Rust -> C
    let v = [1u8, 2, 3];
    unsafe { c_send(v.as_ptr(), v.len()); }

/* THE THREE THINGS TO CHECK on every incoming pointer:
     1. is it null?
     2. is the length plausible? (a bound, not just non-zero)
     3. is the alignment right for the element type?
   And then state in the SAFETY comment what you are trusting
   the caller for - because you cannot check the rest.    */
```

THE FFI BOUNDARY IS A TRUST BOUNDARY

Everything arriving from C is unvalidated: pointers may be null or dangling, lengths may be wrong, enum values may be out of range, strings may not be NUL-terminated. This is exactly the position a kernel driver is in with respect to userspace (Linux chapter 21), and it deserves the same discipline: check what you can, document what you cannot, and keep the unchecked surface small.

Being called from C

```
    #[no_mangle]
    pub extern "C" fn rust_process(data: *const u8,
                                   len: usize) -> i32
    {
        // 1. VALIDATE
        if data.is_null() || len == 0 || len > MAX_LEN {
            return -1;
        }

        // 2. CONVERT
        // SAFETY: checked non-null; the caller guarantees
        // `len` valid bytes
        let s = unsafe { core::slice::from_raw_parts(data,
                                                     len) };

        // 3. CATCH PANICS - unwinding into C is UB
        let result = core::panic::catch_unwind(|| {
            do_work(s)
        });

        match result {
            Ok(Ok(v))  => v as i32,
            Ok(Err(_)) => -2,
            Err(_)     => -3,       // it panicked
        }
    }

/* `#[no_mangle]` gives it a plain symbol name.
   `extern "C"` gives it the C calling convention.
   BOTH are required.

/* THE PANIC RULE: a panic that unwinds across an FFI boundary
   is UNDEFINED BEHAVIOUR, exactly like a C++ exception
   escaping into C (C++ chapter 16). Three ways to prevent
   it:
     1. panic = "abort" in the profile - the embedded answer
     2. catch_unwind at every entry point - needs std
     3. extern "C-unwind" if you WANT unwinding to propagate,
        and the C side is built to handle it

/* --- HANDING OWNERSHIP TO C, and getting it back --- */
    #[no_mangle]
    pub extern "C" fn device_create(base: usize)
            -> *mut Device
    {
        match Device::new(base) {
            Ok(d) => Box::into_raw(Box::new(d)),    // LEAKS,
                                                    // deliberately
            Err(_) => core::ptr::null_mut(),
        }
    }

    #[no_mangle]
    pub unsafe extern "C" fn device_destroy(d: *mut Device) {
        if !d.is_null() {
            // SAFETY: `d` came from device_create and has not
            // been destroyed
            drop(unsafe { Box::from_raw(d) });
        }
    }
```

```
    #[no_mangle]
    pub unsafe extern "C" fn device_read(d: *mut Device,
                                         out: *mut u16) -> i32
    {
        if d.is_null() || out.is_null() { return -1; }
        // SAFETY: as above
        let dev = unsafe { &mut *d };
        match dev.read() {
            Ok(v) => { unsafe { out.write(v) }; 0 }
            Err(_) => -1,
        }
    }

/* THE OPAQUE-HANDLE PATTERN, identical to the C++ one
   (C++ chapter 16): create returns a pointer, operations
   take it, destroy frees it. `Box::into_raw` and
   `from_raw` are the ownership transfer, and forgetting the
   destroy is a leak while calling it twice is a
   double-free.

/* GENERATE THE C HEADER automatically: */
    # cbindgen.toml + build.rs
    cbindgen --config cbindgen.toml --crate my_crate \
             --output include/my_crate.h
/* So the header cannot drift from the Rust definitions.  */
```

Callbacks

```
/* --- REGISTERING A RUST FUNCTION AS A C CALLBACK --- */

    type Callback = unsafe extern "C" fn(ctx: *mut c_void,
                                         event: u32);

    extern "C" {
        fn register_handler(cb: Callback, ctx: *mut c_void);
    }

    struct Handler { count: u32 }

    unsafe extern "C" fn trampoline(ctx: *mut c_void,
                                    event: u32)
    {
        if ctx.is_null() { return; }
        // SAFETY: `ctx` is the Handler pointer we registered,
        // it is still alive (we unregister in Drop), and this
        // callback is not re-entrant
        let h = unsafe { &mut *(ctx as *mut Handler) };
        h.count += 1;
        h.on_event(event);
    }

    impl Handler {
        fn install(&mut self) {
            unsafe {
                register_handler(trampoline,
                                 self as *mut _ as *mut c_void);
            }
        }
    }
    impl Drop for Handler {
        fn drop(&mut self) {
            unsafe { unregister_handler(trampoline); }
        }
    }

/* THE THREE RULES, the same as C++ chapter 16's:
     1. no panic may escape
     2. the context must outlive the registration - hence the
        Drop impl
     3. a capturing closure cannot be a C function pointer;
        put the state in the context

/* A CAPTURELESS CLOSURE does coerce: */
    let f: extern "C" fn(u32) = |x| handle(x);

/* AND THE PINNING PROBLEM: `self as *mut _` is only valid
   while the Handler does not MOVE. If it might, use
   Box::into_raw so the address is stable, or Pin.       */
```

bindgen and cc

```
# --- bindgen: generate Rust declarations from C headers ---
    # build.rs
    let bindings = bindgen::Builder::default()
        .header("vendor/hal.h")
        .clang_arg("-Ivendor/include")
        .clang_arg("--target=thumbv7em-none-eabihf")
        .use_core()                     // no_std
        .ctypes_prefix("core::ffi")
        .derive_default(true)
        .derive_debug(true)
        .allowlist_function("HAL_.*")   // do not import the
        .allowlist_type("GPIO_.*")      // whole SDK
        .blocklist_type("__va_list")
        .parse_callbacks(Box::new(
            bindgen::CargoCallbacks::new()))
        .generate()?;

    bindings.write_to_file(out.join("bindings.rs"))?;

    // in lib.rs
    #[allow(non_upper_case_globals, non_camel_case_types,
            non_snake_case, dead_code)]
    mod bindings {
        include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
    }

# ALLOWLISTING MATTERS: without it, bindgen imports the
# entire transitive header closure - thousands of items,
# a slow build, and name collisions.

# --- cc: compile C sources into the crate ---
    cc::Build::new()
        .file("vendor/hal.c")
        .include("vendor/include")
        .define("STM32F411xE", None)
        .flag("-mcpu=cortex-m4")
        .compile("vendorhal");

# --- THE RESULTING PROJECT SHAPE, which is what a real
#     embedded Rust project on a vendor SDK looks like:

    my-hal-sys/         raw bindgen output. unsafe, ugly,
                        generated. Nobody reads it.
      build.rs
      vendor/           the C SDK
    my-hal/             a SAFE wrapper. Hand-written, small,
                        every unsafe documented (chapter 11).
    firmware/           the application. NO unsafe at all.

/* THE `-sys` CRATE CONVENTION is universal in the Rust
   ecosystem: `foo-sys` is the raw bindings, `foo` is the
   safe wrapper. Following it makes your crate legible to
   everyone else.                                        */
```

The pitfalls

```
/* --- 1. UNWINDING ACROSS FFI: UB. panic = "abort", or
       catch_unwind at every entry.

/* --- 2. LAYOUT MISMATCH: silent corruption. Always
       #[repr(C)], and assert sizes and offsets.

/* --- 3. INVALID VALUES FROM C: */
    #[no_mangle] pub extern "C" fn f(b: bool) { }
/* If C passes 2, that is instant UB - a bool must be 0 or 1.
   Take u8 and convert. Same for char, enums, references and
   NonNull.

/* --- 4. NULL WHERE A REFERENCE IS EXPECTED: */
    #[no_mangle] pub extern "C" fn f(p: &Device) { }   // UB if
                                                       // C
                                                       // passes
                                                       // NULL
    #[no_mangle] pub extern "C" fn f(p: *const Device) { }
/* Take a raw pointer and check. Or Option<&Device>, which is
   FFI-safe and represents null as None (chapter 05's niche
   optimisation) - that is the elegant answer.

/* --- 5. LIFETIME ASSUMPTIONS: C gives you a pointer with no
       lifetime. Copy the data, or document precisely how long
       the caller must keep it valid.

/* --- 6. THREAD SAFETY: C code may call your function from
       any thread or interrupt. Nothing in the type system
       tells you. Assume the worst.

/* --- 7. `static mut` IN GENERATED BINDINGS: bindgen emits
       these for C globals. Wrap access, or use an atomic.

/* --- 8. ALIGNMENT: a *const u8 from C may not be aligned for
       a wider type. read_unaligned, or from_le_bytes.

/* --- 9. VARIADIC FUNCTIONS: printf-style functions can be
       declared but are awkward and easy to misuse. Prefer a
       fixed-argument wrapper on the C side.

/* --- 10. THE ONE THAT COSTS MOST TIME: a C header that
       differs from the compiled library. bindgen reads the
       header; the linker uses the binary. If they disagree,
       everything compiles and the layout is wrong. Rebuild
       both from the same source.                        */
```

How it shows up

Symptom	Cause and fix
"undefined reference" at link time	Missing `#[no_mangle]`, or the library is not linked
Struct fields wrong across the boundary	Missing `#[repr(C)]` — Rust reordered them
Works on x86, wrong on ARM	`c_char` signedness, or `c_long` width
Crash in `CStr::from_ptr`	Not NUL-terminated, or the pointer is dangling
Dangling pointer from a `CString`	The temporary was dropped at the end of the statement

Symptom	Cause and fix
UB with no obvious cause	An invalid `bool`, `char` or enum from C
Abort inside a C caller	A panic unwound across FFI
Double free	`Box::from_raw` called twice, or on a pointer not from `into_raw`
Memory grows	`into_raw` with no matching destroy
Callback crashes after the object moved	The context pointer was to a stack value — box it
bindgen output is enormous	No allowlist
Everything compiles, layout is silently wrong	Header and library built from different sources

Check yourself

- ✓ Declare and call a C function, using the correct `core::ffi` types.
- ✓ Explain why `c_char` and `c_long` matter.
- ✓ Write an FFI struct with `#[repr(C)]` and compile-time size assertions.
- ✓ Convert strings and slices in both directions, and name the three checks on an incoming pointer.
- ✓ Export a Rust function with validation, conversion and panic handling.
- ✓ Implement the opaque-handle ownership pattern with `into_raw` / `from_raw`.
- ✓ Register a Rust callback with a C library and state the three rules.
- ✓ Configure bindgen with an allowlist and explain the `-sys` crate convention.
- ✓ List ten FFI pitfalls and the fix for each.
- ✓ Explain why `Option<&T>` is the elegant answer to nullable parameters.

Fearless concurrency: `Send`, `Sync`, threads and channels

Rust's data-race guarantee comes from two marker traits and the borrow checker. That is the whole mechanism — and it means a program that compiles cannot have a data race, which is a guarantee no other systems language offers.

Send and Sync

```
Send    it is safe to MOVE this type to another thread
Sync    it is safe to SHARE &T with another thread
        (equivalently: &T is Send)

BOTH ARE AUTO TRAITS: the compiler implements them
automatically for any type whose components all implement
them. You almost never write them.

WHAT IS NOT Send:
  Rc<T>           - a non-atomic refcount
  *const T, *mut T - raw pointers, conservatively
  MutexGuard      - some platforms require unlock on the
                    locking thread

WHAT IS NOT Sync:
  Cell<T>, RefCell<T>  - unsynchronised interior
                         mutability
  Rc<T>
  raw pointers

WHAT IS BOTH: almost everything else. i32, String,
Vec<T> (if T is), Arc<T> (if T is Send+Sync), Mutex<T>
(if T is Send), atomics.

AND THE COMBINATION THAT MATTERS:
     Mutex<T>   is Sync if T: Send
because the mutex provides the synchronisation, so sharing
a reference to it is safe as long as the contents can move
between threads.

THE ENFORCEMENT IS AUTOMATIC:

     let rc = Rc::new(5);
     thread::spawn(move || { println!("{rc}"); });
     // error: `Rc<i32>` cannot be sent between threads
     //        safely
     //        required because it appears within the type
     //        of the closure

You did not annotate anything. The compiler worked out
that the closure captures an Rc, that Rc is not Send, and
that spawn requires Send. THAT is the whole data-race
guarantee.
```

Threads

```
    use std::thread;

    let h = thread::spawn(|| {
        expensive_work()
    });
    let result = h.join().unwrap();     // Result - Err if it
                                        // panicked

/* THE SIGNATURE explains the constraints: */
    pub fn spawn<F, T>(f: F) -> JoinHandle<T>
    where
        F: FnOnce() -> T + Send + 'static,
        T: Send + 'static;

/*   Send      the closure moves to another thread
     'static   the thread may outlive this scope, so the
               closure must not borrow anything local
               (chapter 04)

/* WHICH IS WHY `move` IS USUALLY REQUIRED: */
    let data = vec![1, 2, 3];
    thread::spawn(move || println!("{data:?}"));   // data
                                                   // MOVED in

/* SCOPED THREADS lift the 'static requirement, because the
   scope guarantees they are joined before it ends: */
    let data = vec![1, 2, 3];
    thread::scope(|s| {
        s.spawn(|| println!("{data:?}"));   // BORROWS - no
                                            // move, no clone
        s.spawn(|| println!("{}", data.len()));
    });                                     // both joined
                                            // here
    println!("{data:?}");                   // still usable

/* Two shared borrows in two threads - allowed, because
   neither mutates. The borrow checker's rule extends across
   threads with no extra machinery.                      */
```

Sharing data

```
    use std::sync::{Arc, Mutex, RwLock};

/* --- Arc<Mutex<T>>: the standard shape --- */
    let counter = Arc::new(Mutex::new(0));

    let mut handles = vec![];
    for _ in 0..10 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut n = c.lock().unwrap();
            *n += 1;
        }));                    // the GUARD drops here,
                                // unlocking
    }
    for h in handles { h.join().unwrap(); }
    println!("{}", *counter.lock().unwrap());   // 10

/* THE GUARD IS THE POINT: `lock()` returns a
   MutexGuard<T> which Derefs to T and unlocks on Drop
   (chapter 09). YOU CANNOT ACCESS THE DATA WITHOUT HOLDING
   THE LOCK - the API makes it impossible, rather than
   documenting it.

   Compare C++ (C++ chapter 13), where std::mutex and the
   data it protects are separate objects and nothing connects
   them. Rust's Mutex<T> CONTAINS the data.

/* POISONING: if a thread panics while holding the lock, the
   mutex is poisoned and later lock() calls return Err -
   because the data may be in a half-updated state.       */
    match m.lock() {
        Ok(g) => use(g),
        Err(poisoned) => {
            let g = poisoned.into_inner();  // use it anyway
            recover(g);
        }
    }

/* RwLock, for read-heavy data: */
    let cfg = Arc::new(RwLock::new(Config::default()));
    { let r = cfg.read().unwrap(); use(&r); }   // many
    { let mut w = cfg.write().unwrap(); w.rate = 48000; }
                                               // one

/* ATOMICS, for single values (C++ chapter 13 covers the
   memory model, which is the SAME model): */
    use std::sync::atomic::{AtomicU32, AtomicBool, Ordering};

    static COUNT: AtomicU32 = AtomicU32::new(0);
    COUNT.fetch_add(1, Ordering::Relaxed);

    READY.store(true, Ordering::Release);
    if READY.load(Ordering::Acquire) { /* sees prior writes */ }

    let ok = FLAG.compare_exchange(false, true,
                                   Ordering::AcqRel,
                                   Ordering::Acquire).is_ok();

/* THE ORDERINGS AND THEIR ARM INSTRUCTIONS are identical to
   C++'s (C++ chapter 13, ARM chapter 21): Relaxed is a plain
```

```
   access, Acquire is ldar, Release is stlr, SeqCst adds a
   barrier. Use SeqCst unless you have measured a reason.  */
```

> **WHY "FEARLESS" IS NOT MARKETING**
> In C or C++, a data race is undefined behaviour you discover through corruption months later, and ThreadSanitizer finds only races your tests happen to execute. In Rust, the same mistake is a compile error with a message naming the type that is not `Send`. The class of bug does not become easier to debug — it becomes impossible to write outside `unsafe`.

Channels

```
    use std::sync::mpsc;

    let (tx, rx) = mpsc::channel();     // unbounded, multi-
                                        // producer,
                                        // single-consumer

    for i in 0..3 {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(format!("from {i}")).unwrap();
        });
    }
    drop(tx);                   // drop the ORIGINAL, or rx
                                // never sees a disconnect

    for msg in rx {             // iterates until every sender
        println!("{msg}");      // is dropped
    }

    let (tx, rx) = mpsc::sync_channel(4);   // BOUNDED - send
                                            // blocks when
                                            // full

    rx.recv()                   // blocks; Err when all
                                // senders drop
    rx.try_recv()               // non-blocking
    rx.recv_timeout(dur)

/* CHANNELS TRANSFER OWNERSHIP. `send(v)` moves v; the
   receiver owns it afterwards. No sharing, therefore no
   locking, therefore no race - which is why "share memory by
   communicating" works so cleanly here.

/* AND THE BETTER CRATE: */
    use crossbeam_channel as chan;
    let (tx, rx) = chan::bounded(16);
    chan::select! {                     // wait on several
        recv(rx1) -> msg => handle(msg),
        recv(rx2) -> msg => handle(msg),
        default(Duration::from_millis(10)) => timeout(),
    }
/* crossbeam adds multi-CONSUMER, select, and better
   performance. It is what most production code uses.    */
```

Concurrency without an OS

```
/* On bare metal there are no threads - there are interrupts,
   and possibly several cores. Send and Sync still apply, and
   the ecosystem uses them.                                  */

/* --- 1. ATOMICS, for flags and counters --- */
    use core::sync::atomic::{AtomicU32, Ordering};

    static TICKS: AtomicU32 = AtomicU32::new(0);

    #[interrupt]
    fn SysTick() { TICKS.fetch_add(1, Ordering::Relaxed); }

    fn now() -> u32 { TICKS.load(Ordering::Relaxed) }

/* NOTE: not every target has atomics. Cortex-M0 lacks
   LDREX/STREX, so `portable-atomic` provides them via
   critical sections:                                        */
    use portable_atomic::AtomicU64;     // works everywhere

/* --- 2. CRITICAL SECTIONS, for compound state --- */
    use critical_section::Mutex;
    use core::cell::RefCell;

    static SHARED: Mutex<RefCell<State>> =
        Mutex::new(RefCell::new(State::new()));

    #[interrupt]
    fn TIM2() {
        critical_section::with(|cs| {
            SHARED.borrow(cs).borrow_mut().tick();
        });
    }
/* `critical_section::Mutex` is not a blocking lock - it
   requires a token proving interrupts are disabled
   (chapter 09). Zero cost, and sound by construction.

/* --- 3. A LOCK-FREE SPSC QUEUE, ISR to main --- */
    use heapless::spsc::{Queue, Producer, Consumer};

    static mut Q: Queue<Event, 16> = Queue::new();

    // in init, once:
    let (mut prod, mut cons) = unsafe { Q.split() };
    // `prod` goes to the ISR, `cons` to the main loop -
    // and the TYPES enforce that only one end does each

    #[interrupt]
    fn EXTI0() {
        PRODUCER.enqueue(Event::Button).ok();   // never
                                                // blocks
    }
    fn main_loop() {
        while let Some(ev) = CONSUMER.dequeue() { handle(ev); }
    }
/* Single-producer single-consumer needs no lock at all - the
   same design as the C++ ring in C++ chapter 13, with the
   producer and consumer halves as distinct TYPES so the
   invariant cannot be violated.

/* --- 4. RTIC: priorities and resources, checked at compile
```

```
       time --- */
    #[app(device = pac, peripherals = true)]
    mod app {
        #[shared] struct Shared { counter: u32 }
        #[local]  struct Local  { led: Led }

        #[task(binds = TIM2, shared = [counter], priority = 2)]
        fn timer(mut cx: timer::Context) {
            cx.shared.counter.lock(|c| *c += 1);
        }

        #[idle(shared = [counter])]
        fn idle(mut cx: idle::Context) -> ! {
            loop {
                let n = cx.shared.counter.lock(|c| *c);
                ...
            }
        }
    }
/* RTIC applies the STACK RESOURCE POLICY: it computes each
   resource's priority ceiling at compile time and raises
   BASEPRI only as far as needed - so a resource shared only
   between equal-priority tasks needs NO critical section at
   all. Deadlock is impossible by construction, and the
   analysis is done by the macro, not at run time
   (chapter 16).                                          */
```

Implementing Send and Sync

```
/* Occasionally you must assert it yourself - almost always
   because a raw pointer made the type conservatively
   non-Send.                                              */

    pub struct Uart { base: *mut u32 }

    // SAFETY: `base` points to a memory-mapped peripheral.
    // The Uart is created once via a token that guarantees
    // exclusive ownership, so moving it between contexts
    // cannot create aliasing.
    unsafe impl Send for Uart {}

    // NOT Sync: two &Uart in two contexts could interleave
    // register writes. Callers must wrap it in a Mutex.

/* THE DISCIPLINE: `unsafe impl Send` and `unsafe impl Sync`
   are promises the compiler cannot check (chapter 11).
   Write the SAFETY comment, and state which of the two you
   are NOT implementing and why - that second half is what
   readers need.

/* AND THE OPPOSITE - explicitly opting OUT: */
    use core::marker::PhantomData;
    pub struct NotSync {
        data: u32,
        _marker: PhantomData<*const ()>,    // raw pointer:
                                            // not Send, not
                                            // Sync
    }
/* PhantomData with a chosen type is how you control auto
   traits without any runtime cost. `PhantomData<&'a T>`
   also expresses a lifetime relationship for a type that
   holds no reference - which is what type-state patterns
   need (chapter 02).                                     */
```

How it shows up

Error	Meaning and fix
"`Rc<T>` cannot be sent between threads safely"	Use `Arc`
"`RefCell<T>` cannot be shared between threads"	Use `Mutex` or `RwLock`
"`*mut T` cannot be sent"	Wrap it and `unsafe impl Send`, with a justification
"closure may outlive the current function"	Add `move`, or use a scoped thread
"borrowed data escapes outside of function"	The `'static` bound — own the data or scope the thread
`lock()` returns `Err`	Poisoned — a previous holder panicked
Deadlock	Rust does **not** prevent this — establish a lock order
Receiver never terminates	A sender clone was not dropped
`fetch_add` unavailable on Cortex-M0	No LDREX — use `portable-atomic`
Corruption between an ISR and main	No critical section, or the wrong atomic ordering
RTIC reports a priority error	The analysis found a real ceiling violation at compile time

Check yourself

- ✓ Define `Send` and `Sync` and say which common types lack each.
- ✓ Explain how the compiler derives them and why you rarely write them.
- ✓ Explain why `thread::spawn` requires `Send + 'static`.
- ✓ Use scoped threads to borrow local data, and say why it is sound.
- ✓ Explain why `Mutex<T>` containing the data beats a separate mutex.
- ✓ Explain lock poisoning and how to recover.
- ✓ Use a channel and explain why ownership transfer removes the need for locking.
- ✓ Give four concurrency mechanisms for bare metal and when each fits.
- ✓ Explain what RTIC computes at compile time and what it eliminates.
- ✓ Write a justified `unsafe impl Send` and say what you deliberately did not implement.

Async Rust

Concurrency without threads: a state machine the compiler writes for you. On embedded this is transformative — Embassy gives you cooperative multitasking with no RTOS, no per-task stacks, and code that reads like blocking code.

The model

```
An `async fn` does not run when called. It returns a
FUTURE - a value representing work not yet done.

     async fn read_sensor() -> u16 { ... }

     let fut = read_sensor();        // NOTHING HAS
                                     // HAPPENED
     let v = fut.await;              // now it runs, and
                                     // may yield

THE FUTURE TRAIT, which is the whole mechanism:

     trait Future {
         type Output;
         fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
                 -> Poll<Self::Output>;
     }
     enum Poll<T> { Ready(T), Pending }

The EXECUTOR calls poll(). If the future returns Pending,
it has arranged - via the Waker in the Context - to be
woken when progress is possible, and the executor runs
something else. When woken, poll() is called again.

THE COMPILER TRANSFORMS an async fn into a STATE MACHINE:

     async fn f() {
         let a = step1().await;      // state 0
         let b = step2(a).await;     // state 1
         finish(b);                  // state 2
     }

becomes, in effect:

     enum F { S0, S1 { a: A }, S2 { b: B }, Done }
     impl Future for F {
         fn poll(...) -> Poll<()> {
             loop { match self.state {
                 S0 => match step1().poll() {
                     Pending => return Pending,
                     Ready(a) => self.state = S1 { a },
                 },
                 ...
             }}
         }
     }

THE SIZE OF THAT ENUM is the size of the future - it holds
every local that is live across an await point. That is
why async needs no per-task stack: the state that would
have lived on a stack lives in the future instead, sized
exactly, at compile time.

AND WHY THAT MATTERS ON EMBEDDED: an RTOS gives every task
a stack, sized by guesswork and usually over-provisioned.
Embassy's tasks are futures - typically tens or a few
hundred bytes, exactly measured by the compiler.
```

Writing async code

```
    async fn fetch(id: u32) -> Result<Data, Error> {
        let conn = connect().await?;            // `?` works
        let raw = conn.request(id).await?;
        Ok(parse(raw)?)
    }

/* IT READS LIKE BLOCKING CODE. That is the point - the
   alternative is a hand-written state machine or a chain of
   callbacks, both of which obscure the control flow.

/* CONCURRENCY WITHIN ONE TASK: */
    use futures::future::join;

    let (a, b) = join(read_temp(), read_pressure()).await;
    // both progress; completes when BOTH do

    use futures::future::select;
    match select(read_sensor(), timer(Duration::from_ms(100)))
              .await {
        Either::Left((v, _))  => got_value(v),
        Either::Right((_, _)) => timed_out(),
    }
    // whichever finishes first; the other is DROPPED

/* IN EMBASSY the same things read better: */
    use embassy_futures::{join::join, select::{select,
                                               Either}};

    match select(uart.read(&mut buf),
                 Timer::after_millis(100)).await {
        Either::First(r) => handle(r?),
        Either::Second(_) => return Err(Error::Timeout),
    }
/* A TIMEOUT ON ANY OPERATION, in three lines, with no timer
   callbacks and no state to unwind. This single pattern is
   worth the whole model on embedded.

/* CANCELLATION IS DROPPING THE FUTURE: */
    let fut = long_operation();
    drop(fut);                  // it simply stops - its Drop
                                // runs, releasing whatever it
                                // held
/* WHICH IS WHY `select` can abandon the loser safely, and
   why every async operation must be CANCEL SAFE: it must be
   correct to stop at any await point. A future that has
   started a DMA transfer must stop it in Drop, or the
   hardware writes into a buffer that no longer exists.
   THIS IS THE MAIN CORRECTNESS HAZARD in embedded async, and
   it is the one to think about when writing a driver.   */
```

> **ASYNC IS CONCURRENCY, NOT PARALLELISM**
>
> On a single core, an executor interleaves tasks at await points — one runs at a time, and a task that never awaits blocks everything. That is cooperative multitasking, and it means no preemption, no data races between tasks on one executor, and no locking needed for data shared between them. It also means a compute-bound loop must yield explicitly.

Embassy

```
#![no_std]
#![no_main]

use embassy_executor::Spawner;
use embassy_stm32::gpio::{Level, Output, Speed};
use embassy_time::{Duration, Timer};

#[embassy_executor::task]
async fn blink(mut led: Output<'static>) {
    loop {
        led.set_high();
        Timer::after_millis(500).await;      // YIELDS - the
        led.set_low();                       // executor runs
        Timer::after_millis(500).await;      // other tasks
    }
}

#[embassy_executor::task]
async fn sensor(mut i2c: I2c<'static, Async>) {
    let mut buf = [0u8; 2];
    loop {
        i2c.write_read(ADDR, &[REG_TEMP], &mut buf)
           .await
           .unwrap();                         // an actual DMA
                                              // transfer,
                                              // awaited
        let temp = i16::from_be_bytes(buf);
        CHANNEL.send(Reading::Temp(temp)).await;
        Timer::after_secs(1).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());

    let led = Output::new(p.PC13, Level::Low, Speed::Low);
    spawner.spawn(blink(led)).unwrap();
    spawner.spawn(sensor(i2c)).unwrap();

    loop {
        match CHANNEL.receive().await {
            Reading::Temp(t) => log(t),
        }
    }
}

/* WHAT IS HAPPENING UNDERNEATH:

   · `Timer::after_millis(500).await` registers with a
     hardware timer and returns Pending. The executor finds
     nothing else runnable and executes WFI - the core sleeps
     until an interrupt. Power consumption drops to
     microamps.

   · `i2c.write_read(...).await` starts a DMA transfer and
     returns Pending. The DMA-complete interrupt wakes the
     task. NO POLLING, NO BUSY-WAIT, and the core sleeps
     during the transfer.
```

```
   · Each task is a future stored in a statically allocated
     slot sized by the compiler. NO HEAP, NO PER-TASK STACK.

   THE COMPARISON WITH AN RTOS:
     FreeRTOS: 8 tasks x 1 KB stack = 8 KB of RAM, guessed,
               plus context-switch cost, plus a scheduler
     Embassy:  8 futures x ~100 bytes = under 1 KB, exact,
               with switches that are ordinary function
               returns

/* THE SYNCHRONISATION PRIMITIVES, all no-alloc: */
    use embassy_sync::{channel::Channel, mutex::Mutex,
                       signal::Signal, pubsub::PubSubChannel};
    use embassy_sync::blocking_mutex::raw::ThreadModeRawMutex;

    static CHANNEL: Channel<ThreadModeRawMutex, Reading, 8>
                = Channel::new();
    static SIGNAL: Signal<ThreadModeRawMutex, ()>
                = Signal::new();

    // an ASYNC mutex - awaiting the lock yields rather than
    // blocking
    static SHARED: Mutex<ThreadModeRawMutex, State>
                = Mutex::new(State::new());
    { let mut s = SHARED.lock().await; s.update(); }
```

Pin, briefly

```
/* A future that holds a reference into itself - which the
   state machine transformation produces whenever a local
   reference lives across an await - must not MOVE, or the
   reference dangles.

   `Pin<P>` is the type-level promise that the pointee will
   not move.                                                */

    async fn f() {
        let data = [0u8; 64];
        let slice = &data[..];          // a reference INTO
                                        // the future
        io.read(slice).await;           // live across an
                                        // await ->
                                        // self-referential
    }

/* IN PRACTICE YOU RARELY TOUCH Pin:
     · `.await` handles pinning for you
     · pin_mut! or Box::pin when you must hold a future
     · implementing Future by hand is where you meet it

    use futures::pin_mut;
    let fut = do_thing();
    pin_mut!(fut);              // pins it to the stack
    poll_fn(|cx| fut.as_mut().poll(cx)).await;

/* `Unpin` MEANS "moving me is fine" - most types are Unpin,
   and only self-referential ones are not. If an error
   mentions Unpin, you are holding a future that needs
   pinning; pin_mut! or Box::pin is the fix.             */
```

Async on Linux

```
/* For embedded LINUX rather than bare metal, tokio is the
   standard runtime. */

    #[tokio::main]
    async fn main() -> Result<()> {
        let listener = TcpListener::bind("0.0.0.0:8080")
                           .await?;
        loop {
            let (socket, _) = listener.accept().await?;
            tokio::spawn(async move { handle(socket).await });
        }
    }

/* TOKIO gives you: a multi-threaded work-stealing executor,
   async file and socket I/O, timers, channels, and
   synchronisation primitives. It is what a network service
   on an embedded Linux gateway should use.

/* THE CATCH - BLOCKING IN AN ASYNC CONTEXT: */
    async fn bad() {
        std::thread::sleep(Duration::from_secs(1));  // BLOCKS
                                                     // THE
                                                     // EXECUTOR
        let data = std::fs::read("file")?;           // ditto
    }
    async fn good() {
        tokio::time::sleep(Duration::from_secs(1)).await;
        let data = tokio::fs::read("file").await?;
        // and for genuinely blocking work:
        let r = tokio::task::spawn_blocking(|| heavy_cpu())
                    .await?;
    }
/* A blocking call in an async task stalls every other task
   on that executor thread. This is the commonest async bug,
   in every language that has it.

/* AND THE RUNTIME QUESTION: async code is tied to its
   runtime's primitives, so a library written against tokio
   does not work under async-std or Embassy. Libraries should
   be runtime-agnostic where possible - which in practice
   means depending only on `futures` traits.            */
```

When to use it

Situation	Approach
Many I/O-bound tasks, bare metal	**Embassy** — this is its case
Hard real-time, priority-driven	**RTIC** — preemptive, with compile-time analysis
A simple superloop with two or three activities	A plain loop with state; async adds nothing
Network service on embedded Linux	**tokio**
CPU-bound work	Threads, or `spawn_blocking` — async does not help
Existing RTOS codebase	Keep the RTOS; async is not a drop-in replacement
Very tight flash budget	Measure — Embassy is small but not free

```
/* THE HONEST TRADE-OFFS:

   IN FAVOUR:
     · code that reads sequentially for logic that is
       concurrent
     · no per-task stacks - typically a large RAM saving
     · automatic power management: the executor sleeps when
       nothing is ready
     · timeouts and cancellation are three lines
     · no priority inversion, no preemption races between
       tasks on one executor

   AGAINST:
     · the state machine transformation makes debugging
       harder - a backtrace shows the executor, not your
       logic
     · CANCEL SAFETY is a real hazard: every await point is a
       possible cancellation point
     · a future's size can surprise you - a large array
       across an await lives in the future
     · no preemption, so a long computation starves
       everything; you must yield explicitly
     · error messages involving Pin and generic futures are
       poor
     · it is newer than the alternatives

   MEASURE THE FUTURE SIZES, which is the one number that
   catches people out:                                    */
        const _: () = assert!(
            core::mem::size_of_val(&my_task()) < 512);
/* A future holding a 4 KB buffer across an await is 4 KB of
   static RAM. Box it, or restructure so the buffer is not
   live across the await.                                 */
```

How it shows up

Symptom	Cause and fix
The async function never runs	The future was created and not awaited
Everything stalls	A blocking call, or a compute loop with no await
"future cannot be sent between threads"	It holds a non-`Send` value across an await
"`X` is not `Unpin`"	Pin it: `pin_mut!` or `Box::pin`
DMA writes into a freed buffer	The future was cancelled without stopping the transfer — cancel safety
Data lost on a timeout	The abandoned future had consumed input — restructure to be cancel safe
Static RAM much larger than expected	A large local live across an await
Backtrace shows only the executor	Expected — instrument with `defmt` instead
Deadlock on an async mutex	Held across an await that waits on the holder
Power consumption never drops	A task is polling rather than awaiting a real interrupt

Check yourself

- ✓ Explain what an `async fn` returns and when the work happens.
- ✓ Describe the state machine transformation and where the state lives.

- ✓ Explain why async needs no per-task stack, and compare with an RTOS.
- ✓ Use `join` and `select`, and implement a timeout.
- ✓ Explain cancel safety and give a hardware example where it matters.
- ✓ Write an Embassy task and explain what happens at each await.
- ✓ Explain why the core can sleep during an awaited DMA transfer.
- ✓ Say what `Pin` is for and when you actually meet it.
- ✓ Name the commonest async bug on a hosted runtime and its fix.
- ✓ Decide between Embassy, RTIC, an RTOS and a superloop for four situations.
- ✓ Measure a future's size and say what makes it large.

`no_std` and bare-metal Rust

Rust without an operating system: what you lose, what you keep, and the boilerplate that turns a crate into a firmware image. Unlike C++, Rust was designed for this case from the start, and the result is genuinely comfortable.

The three layers

```
core          ALWAYS AVAILABLE. No OS, no allocator.
              Option, Result, Iterator, slices, str,
              integers and floats, core::fmt, atomics,
              mem, ptr, marker traits, panic machinery.
              Most of the LANGUAGE lives here.

alloc         needs a GLOBAL ALLOCATOR, nothing more.
              Box, Vec, String, BTreeMap, Rc, Arc.
              Opt in with `extern crate alloc;`.

std           needs an OS. Files, sockets, threads,
              processes, time, environment, HashMap
              (because of random seeding).

`#![no_std]` OPTS OUT OF std, keeping core. Add `alloc`
back if you want heap collections and can supply an
allocator.

THE PRACTICAL POINT: `core` is not a cut-down subset -
it contains iterators, the whole Option/Result API,
formatting, atomics and the numeric traits. Idiomatic
Rust works almost unchanged; what you lose is I/O,
threads and heap collections.

COMPARE C++ FREESTANDING (C++ chapter 15), where you must
check header by header what is available and supply
several runtime functions yourself.
```

A minimal firmware crate

```
#![no_std]
#![no_main]

use panic_halt as _;            // supplies #[panic_handler]
use cortex_m_rt::entry;         // supplies the reset vector
use cortex_m::asm;

#[entry]
fn main() -> ! {                // NEVER returns
    let p = pac::Peripherals::take().unwrap();

    p.RCC.ahb1enr.modify(|_, w| w.gpiocen().set_bit());
    p.GPIOC.moder.modify(|_, w| w.moder13().output());

    loop {
        p.GPIOC.bsrr.write(|w| w.bs13().set_bit());
        asm::delay(8_000_000);
        p.GPIOC.bsrr.write(|w| w.br13().set_bit());
        asm::delay(8_000_000);
    }
}

/* `#![no_main]` because the C `main` signature and the
   pre-main runtime do not exist. `#[entry]` from
   cortex-m-rt provides the reset handler, which zeroes .bss,
   copies .data from flash, initialises the FPU, and calls
   your function (compilers chapter 12).

/* THE PANIC HANDLER is mandatory - core cannot provide one
   because the right behaviour is application-specific: */
    use panic_halt as _;        // spin forever
    use panic_reset as _;       // system reset
    use panic_probe as _;       // print over RTT, then halt
    use panic_persist as _;     // save the message to RAM
                                // across a reset

/* OR WRITE ONE, which is what a product does: */
    #[panic_handler]
    fn panic(info: &core::panic::PanicInfo) -> ! {
        // save the location for the next boot to report
        cortex_m::interrupt::disable();
        write_panic_record(info.location());
        cortex_m::peripheral::SCB::sys_reset();
    }

/* AND THE INTERRUPT HANDLERS: */
    use cortex_m_rt::exception;
    use stm32f4xx_hal::pac::interrupt;

    #[exception]
    fn SysTick() { TICKS.fetch_add(1, Ordering::Relaxed); }

    #[interrupt]
    fn USART1() { critical_section::with(|cs| { ... }); }

    #[exception]
    unsafe fn HardFault(ef: &ExceptionFrame) -> ! {
        // ef holds the stacked registers - log them
        log_fault(ef);
        loop {}
```

```
    }
/* The names are checked against the device's vector table by
   the PAC, so a typo is a compile error rather than an
   interrupt that never fires.                          */
```

Project layout

```
# Cargo.toml
    [dependencies]
    cortex-m = "0.7"
    cortex-m-rt = "0.7"
    embedded-hal = "1.0"
    stm32f4xx-hal = { version = "0.21",
                      features = ["stm32f411"] }
    heapless = "0.8"
    defmt = "0.3"
    defmt-rtt = "0.4"
    panic-probe = { version = "0.3", features = ["print-defmt"] }
    critical-section = "1.1"

    [profile.release]
    opt-level = "z"
    lto = "fat"
    codegen-units = 1
    panic = "abort"
    debug = 2                   # symbols cost nothing in
                                # flash and defmt needs them
    overflow-checks = false

    [profile.dev]
    opt-level = 1               # -O0 firmware is often too
                                # slow to run

# .cargo/config.toml
    [build]
    target = "thumbv7em-none-eabihf"

    [target.thumbv7em-none-eabihf]
    runner = "probe-rs run --chip STM32F411RETx"
    rustflags = [
      "-C", "link-arg=-Tlink.x",        # from cortex-m-rt
      "-C", "link-arg=-Tdefmt.x",       # for defmt
      "-C", "linker=flip-link",         # STACK OVERFLOW
                                        # PROTECTION
    ]

# memory.x - the memory map, read by cortex-m-rt's link.x
    MEMORY
    {
      FLASH : ORIGIN = 0x08000000, LENGTH = 512K
      RAM   : ORIGIN = 0x20000000, LENGTH = 128K
    }

# build.rs copies memory.x into OUT_DIR (chapter 10).

# THEN:
    cargo run --release
# builds, flashes and runs on the target, streaming defmt
# logs back. One command.

/* flip-link IS WORTH SPECIAL MENTION: it moves the stack
   BELOW static memory, so a stack overflow hits an unmapped
   region and faults immediately instead of silently
   corrupting .bss. That is a free fix for the single worst
   embedded failure mode (Linux chapter 02 describes the same
   problem), and it costs nothing.                          */
```

> **THE THREE TOOLS THAT CHANGE THE EXPERIENCE**
>
> **probe-rs** — flash, run, debug and stream logs with one command, no vendor IDE. **defmt** — deferred formatting: the format string stays in the ELF and only the arguments go over the wire, so a log line costs a few bytes and a handful of cycles. **flip-link** — stack overflow becomes a fault instead of corruption. Together they are why embedded Rust tooling is frequently better than the C equivalent, not merely comparable.

What you lose, and the replacement

Absent	Use instead
`Vec<T>`, `String`	`heapless::Vec<T, N>`, `heapless::String<N>`
`HashMap`	`heapless::FnvIndexMap`, or a sorted array
`println!`	`defmt::info!`, or `write!` to a UART
`std::error::Error`	Your own enum (ch. 07)
`std::time::Instant`	`embassy_time::Instant`, or a tick counter
`std::thread`	Interrupts, RTIC tasks, or Embassy tasks
`std::sync::Mutex`	`critical_section::Mutex`, `embassy_sync::Mutex`
Floating-point `f64::sin` etc.	`libm`, or fixed-point
`rand`	The hardware RNG, or a PRNG crate with `default-features=false`
`std::io`	`embedded-io`, `core::fmt::Write`

```
/* MAKING A LIBRARY WORK BOTH WAYS - the convention every
   good crate follows: */

    #![cfg_attr(not(feature = "std"), no_std)]

    #[cfg(feature = "alloc")]
    extern crate alloc;

    #[cfg(feature = "alloc")]
    use alloc::vec::Vec;

    // Cargo.toml
    [features]
    default = ["std"]
    std = ["alloc"]
    alloc = []

/* AND THE CHECK: `cargo build --no-default-features` in CI,
   so a std-only dependency cannot creep in unnoticed.

/* USING A DEPENDENCY IN no_std: */
    serde = { version = "1", default-features = false,
              features = ["derive"] }
/* `default-features = false` is the crucial part - most
   crates enable std by default. If a crate does not support
   no_std, that is usually a hard blocker.               */
```

Adding a heap, if you must

```
    #![no_std]
    extern crate alloc;

    use embedded_alloc::LlffHeap as Heap;
    use alloc::{vec::Vec, string::String};

    #[global_allocator]
    static HEAP: Heap = Heap::empty();

    #[entry]
    fn main() -> ! {
        {
            use core::mem::MaybeUninit;
            const HEAP_SIZE: usize = 8 * 1024;
            static mut MEM: [MaybeUninit<u8>; HEAP_SIZE] =
                [MaybeUninit::uninit(); HEAP_SIZE];
            unsafe {
                HEAP.init(core::ptr::addr_of_mut!(MEM) as usize,
                          HEAP_SIZE)
            }
        }

        let mut v = Vec::new();
        v.push(42);
        ...
    }

/* HANDLING EXHAUSTION: */
    let mut v = Vec::new();
    v.try_reserve(100)?;                // Result, not a panic
    // otherwise allocation failure calls the OOM handler,
    // which by default panics

/* SHOULD YOU? Usually not.
     · fragmentation is unbounded over a long uptime
     · allocation time is not deterministic
     · failure handling is awkward
     · heapless gives you the same interfaces with a bounded
       failure mode (chapter 08)

   WHEN IT IS REASONABLE: a device with megabytes of RAM
   running a network stack or a filesystem, where the working
   set genuinely varies. On a 64 KB microcontroller, use
   heapless and static allocation.                          */
```

Controlling size

```
# --- MEASURE FIRST ---
    cargo size --release -- -A
       .vector_table  1024
       .text         18432
       .rodata        2048
       .data           128
       .bss           4096

    cargo bloat --release -n 30
    cargo bloat --release --crates       # by dependency

# --- THE BIGGEST WINS, in order ---

# 1. THE PROFILE (chapter 10) - typically halves the binary:
    opt-level = "z", lto = "fat", codegen-units = 1,
    panic = "abort"

# 2. AVOID CORE::FMT. The formatting machinery is often the
#    single largest item in a small firmware:
    panic!("bad value: {}", x);         // pulls in fmt
    panic!("bad value");                // does not
    defmt::info!("value {=u32}", x);    // FORMATS ON THE
                                        // HOST - the string
                                        // never enters the
                                        // binary

# 3. defmt INSTEAD OF ANY OTHER LOGGING. It is the difference
#    between a few hundred bytes and several kilobytes.

# 4. PANICS. Every `unwrap`, every index, every slice
#    operation can panic, and each panic site carries a
#    location string. Use `get()`, `checked_*` and explicit
#    handling (chapter 07). Then:
    panic-never = "0.1"
#    makes any REACHABLE panic a LINK ERROR - which is a
#    build-time proof that no panic can occur.

# 5. build-std, to rebuild core with your profile:
    cargo +nightly build -Z build-std=core \
        -Z build-std-features=panic_immediate_abort \
        --target thumbv7em-none-eabihf --release
#    `panic_immediate_abort` removes the panic formatting
#    from core itself. Often several kilobytes.

# 6. --gc-sections is on by default via link.x, but check:
    cargo nm --release | sort -k1

# A REALISTIC RESULT: a blinky is around 2 KB; a firmware
# with a HAL, defmt logging and a USB stack is 30-60 KB.
# Comparable with C once the profile is right, and the
# difference is usually formatting rather than the language.
```

Testing and debugging

```
/* --- HOST TESTS for the portable logic (chapter 10) --- */
    #[cfg(test)]
    mod tests {
        use super::*;
        #[test]
        fn crc_matches_reference() {
            assert_eq!(crc32(b"hello"), 0x3610A686);
        }
    }
    // cargo test --target x86_64-unknown-linux-gnu
    // put the logic in a portable crate so this works

/* --- ON-TARGET TESTS --- */
    #[defmt_test::tests]
    mod tests {
        #[test]
        fn uart_loopback() {
            let mut u = setup_uart();
            u.write(b'x').unwrap();
            assert_eq!(u.read().unwrap(), b'x');
        }
    }
    // cargo test --target thumbv7em-none-eabihf
    // runs on the DEVICE, results streamed back over RTT

/* --- LOGGING --- */
    defmt::trace!("entering state {}", state);
    defmt::debug!("adc = {=u16}", raw);
    defmt::info!("ready");
    defmt::warn!("retry {=u8}/{=u8}", n, max);
    defmt::error!("fault: {}", e);
    defmt::assert!(x < 10, "x too large: {}", x);

    #[derive(defmt::Format)]        // like Debug, for defmt
    struct Reading { temp: i16, humidity: u8 }

    // filter at build time:
    DEFMT_LOG=info cargo run --release

/* --- DEBUGGING --- */
    probe-rs run --chip STM32F411RETx target/.../fw
    probe-rs attach --chip STM32F411RETx
    cargo embed                     # a TUI with RTT

    # GDB, when you need it:
    probe-rs gdb --chip STM32F411RETx
    (gdb) target remote :1337
    (gdb) monitor reset halt
    (gdb) break main
    (gdb) continue

/* --- AND THE HARD FAULT HANDLER, which is worth writing
       properly: */
    #[exception]
    unsafe fn HardFault(ef: &ExceptionFrame) -> ! {
        defmt::error!("HardFault at pc={=u32:#x} lr={=u32:#x}",
                      ef.pc(), ef.lr());
        // and the fault status registers (ARM chapter 29)
        let cfsr = (*SCB::PTR).cfsr.read();
        defmt::error!("CFSR = {=u32:#x}", cfsr);
```

```
        loop {}
    }
/* Which turns "it hangs" into "it faulted at this address
   because of an unaligned access" - the same diagnosis
   process as ARM chapter 29, with the registers named.  */
```

How it shows up

Symptom	Cause and fix
"can't find crate for `std`"	A dependency needs std — set `default-features = false`
"`#[panic_handler]` function required"	Add `use panic_halt as _;` or write one
"language item required: `eh_personality`"	Set `panic = "abort"`
"undefined reference to `memcpy`"	Needs a `compiler_builtins` or the target's runtime
Linker error about `link.x` or memory regions	`memory.x` missing or not copied by `build.rs`
Binary far larger than expected	Profile not tuned, or `core::fmt` pulled in
Firmware hangs immediately	Stack overflow — use `flip-link`; or clocks not configured
Interrupt never fires	Not unmasked in the NVIC, or the handler name is wrong
Panic with no message	`panic-halt` — switch to `panic-probe` while developing
`Peripherals::take()` returns `None`	Called twice — by design (ch. 02)
Allocation fails at run time	Heap too small, or fragmented — prefer `heapless`
defmt output is garbled	The ELF and the running binary differ — reflash

Check yourself

- ✓ Distinguish `core`, `alloc` and `std`, and say what each needs.
- ✓ Write a minimal `no_std` firmware crate with an entry point and a panic handler.
- ✓ Explain what `cortex-m-rt` does before your `main` runs.
- ✓ Configure `Cargo.toml`, `.cargo/config.toml` and `memory.x`.
- ✓ Explain what `flip-link` prevents and why it is free.
- ✓ Name eight `std` facilities and their `no_std` replacements.
- ✓ Make a library build both with and without `std`, and test both in CI.
- ✓ Add a global allocator, and give three reasons not to.
- ✓ List six size-reduction techniques in order of effect.
- ✓ Explain why `defmt` is smaller than any alternative.
- ✓ Write a useful HardFault handler.

The embedded Rust ecosystem: PAC, HAL, RTIC, Embassy

The layered architecture that lets a driver written once run on every supported chip. This is the ecosystem's central achievement, and it has no equivalent in the C world where every vendor HAL is incompatible with every other.

The layers

```
+-------------------------------------------------+
|  APPLICATION                                    |
|  your firmware, or RTIC / Embassy tasks         |
+-------------------------------------------------+
|  DEVICE DRIVERS - portable                      |
|  generic over embedded-hal traits, so ONE driver  |
|  works on every chip                            |
+-------------------------------------------------+
|  embedded-hal   - THE TRAIT DEFINITIONS         |
|  SpiDevice, I2c, OutputPin, DelayNs, ...        |
+-------------------------------------------------+
|  HAL - per chip family                          |
|  stm32f4xx-hal, nrf52840-hal, esp-hal, rp2040-hal |
|  implements the traits; owns clocks, pins, DMA    |
+-------------------------------------------------+
|  PAC - Peripheral Access Crate                  |
|  GENERATED from the vendor's SVD file by svd2rust.|
|  Typed, safe register access. Contains the unsafe.|
+-------------------------------------------------+
|  cortex-m / riscv - the core itself             |
|  NVIC, SysTick, SCB, critical sections          |
+-------------------------------------------------+
|  cortex-m-rt - the startup and vector table     |
+-------------------------------------------------+

THE PAYOFF: a driver for an SPI display depends only on
embedded-hal. It compiles for STM32, nRF, ESP32 and RP2040
with no #ifdef, no porting layer, and no runtime cost -
because the concrete SPI type is a generic parameter,
monomorphised at the call site (chapter 06).

COMPARE C, where an ST driver uses HAL_SPI_Transmit, a
Nordic one uses nrfx_spim_xfer, and porting means
rewriting. crates.io has hundreds of sensor and display
drivers that work everywhere, because they were written
against one trait set.
```

The PAC

```
/* GENERATED from the vendor's CMSIS-SVD file - the same XML
   that generates the C headers - by svd2rust. So it covers
   every register and field the vendor documented.        */

    use stm32f4xx_hal::pac;

    let p = pac::Peripherals::take().unwrap();  // SINGLETON
                                                // (chapter 02)

    // enable the GPIOC clock
    p.RCC.ahb1enr.modify(|_, w| w.gpiocen().set_bit());

    // set PC13 as output
    p.GPIOC.moder.modify(|_, w| w.moder13().output());

    // write it
    p.GPIOC.bsrr.write(|w| w.bs13().set_bit());

    // read a field
    let mode = p.GPIOC.moder.read().moder13().bits();

/* THE THREE ACCESS METHODS, and the distinction matters: */
    reg.read()                  // -> R, with field accessors
    reg.write(|w| ...)          // RESET value, then your
                                // changes. Fields you do not
                                // mention are RESET.
    reg.modify(|r, w| ...)      // read-modify-write. Fields
                                // you do not mention are
                                // PRESERVED.
    unsafe { reg.write_with_zero(|w| ...) }   // start from
                                              // zero

/* USING `write` WHERE `modify` WAS MEANT is the commonest PAC
   bug: it silently resets every other field in the register.

/* WHAT THE TYPE SYSTEM GIVES YOU:
     · a field name for every documented bit
     · enum values for enumerated fields:
         w.moder13().output()  not  w.bits(0b01)
     · WRITE-ONLY fields cannot be read
     · READ-ONLY fields cannot be written
     · out-of-range values are compile errors
     · a register that does not exist on this part is not in
       the crate at all

/* GENERATE ONE for a chip that has no crate: */
    svd2rust -i vendor.svd --target cortex-m
    form -i lib.rs -o src/ && rm lib.rs
    cargo fmt

/* THE PAC IS TOO LOW-LEVEL FOR APPLICATION CODE - it is the
   register map, not a driver. Use the HAL, and drop to the
   PAC for the occasional register the HAL does not expose.
   Every HAL re-exports its PAC for exactly that.        */
```

The HAL

```
    use stm32f4xx_hal::{pac, prelude::*, gpio::Output,
                        spi::Spi};

    let dp = pac::Peripherals::take().unwrap();

    // 1. CLOCKS - a builder that VALIDATES the configuration
    let rcc = dp.RCC.constrain();
    let clocks = rcc.cfgr
        .use_hse(25.MHz())
        .sysclk(100.MHz())
        .pclk1(50.MHz())
        .freeze();              // computes the PLL settings,
                                // PANICS if impossible

    // 2. SPLIT the GPIO port into individual PINS
    let gpioa = dp.GPIOA.split();
    let gpioc = dp.GPIOC.split();

    // 3. CONFIGURE pins - the TYPE changes (chapter 02)
    let mut led = gpioc.pc13.into_push_pull_output();
    let sck  = gpioa.pa5.into_alternate();
    let miso = gpioa.pa6.into_alternate();
    let mosi = gpioa.pa7.into_alternate();

    // 4. CONSTRUCT a peripheral, CONSUMING the pins
    let spi = Spi::new(dp.SPI1, (sck, miso, mosi),
                       Mode { polarity: IdleLow,
                              phase: CaptureOnFirstTransition },
                       1.MHz(), &clocks);

    // gpioa.pa5 no longer exists - it was MOVED into the SPI.
    // Nothing else can reconfigure it. That is the ownership
    // model applied to hardware (chapter 02).

    led.set_high();
    spi.write(&[0x01, 0x02]).unwrap();

/* WHAT THE HAL ADDS OVER THE PAC:
     · clock tree configuration, computed and validated
     · pin type-states, so a misconfigured pin is a compile
       error
     · peripherals that CONSUME their pins, preventing
       conflicts
     · implementations of the embedded-hal traits
     · DMA integration
     · sensible defaults

/* AND THE TYPE-STATE PAYOFF: */
    let pin = gpioa.pa5.into_floating_input();
    spi = Spi::new(dp.SPI1, (pin, ...), ...);
    // ERROR: expected Pin<'A', 5, Alternate<5>>,
    //        found Pin<'A', 5, Input<Floating>>
/* A wrong pin configuration - the single commonest embedded
   bring-up bug - is caught by the compiler with a message
   naming the expected mode.                              */
```

OWNERSHIP APPLIED TO HARDWARE

The `Peripherals::take()` singleton, the `split()` that yields one value per pin, and constructors that *consume* their pins together mean that two drivers cannot claim the same hardware. Not by convention, not by review — the second attempt does not compile. That is the single most valuable thing the embedded Rust ecosystem does, and it is a direct application of chapter 02.

Writing a portable driver

```
use embedded_hal::spi::SpiDevice;
use embedded_hal::digital::OutputPin;
use embedded_hal::delay::DelayNs;

pub struct Display<SPI, DC, RST> {
    spi: SPI,
    dc: DC,
    rst: RST,
}

impl<SPI, DC, RST, E> Display<SPI, DC, RST>
where
    SPI: SpiDevice<Error = E>,
    DC:  OutputPin,
    RST: OutputPin,
{
    pub fn new(spi: SPI, dc: DC, rst: RST) -> Self {
        Self { spi, dc, rst }
    }

    pub fn init(&mut self, delay: &mut impl DelayNs)
            -> Result<(), Error<E>>
    {
        self.rst.set_low().map_err(|_| Error::Pin)?;
        delay.delay_ms(10);
        self.rst.set_high().map_err(|_| Error::Pin)?;
        delay.delay_ms(120);

        self.command(0x11)?;            // sleep out
        delay.delay_ms(120);
        self.command(0x29)?;            // display on
        Ok(())
    }

    fn command(&mut self, c: u8) -> Result<(), Error<E>> {
        self.dc.set_low().map_err(|_| Error::Pin)?;
        self.spi.write(&[c]).map_err(Error::Spi)
    }

    pub fn data(&mut self, d: &[u8]) -> Result<(), Error<E>> {
        self.dc.set_high().map_err(|_| Error::Pin)?;
        self.spi.write(d).map_err(Error::Spi)
    }
}

/* THIS COMPILES for STM32, nRF52, ESP32, RP2040 and every
   other chip with a HAL - unchanged. Publish it to crates.io
   and it works for everyone.

/* THE embedded-hal 1.0 TRAITS worth knowing: */
    digital::{InputPin, OutputPin, StatefulOutputPin}
    spi::{SpiBus, SpiDevice}        // Device includes CS
                                    // management
    i2c::I2c
    delay::DelayNs
    pwm::SetDutyCycle
    embedded-hal-async::*           // the async versions,
                                    // for Embassy
    embedded-io::{Read, Write}      // byte streams
    embedded-can, embedded-storage, embedded-nal
```

```
/* THE DRIVER-AUTHOR CONVENTIONS:
     · be generic over the traits, never over a concrete HAL
     · take ownership of the peripherals you need
     · define your own error enum, wrapping the bus error
     · provide both blocking and async versions where it
       matters
     · no_std, and no default features that need std
     · document the wiring in the crate docs             */
```

RTIC

```
use rtic::app;

#[app(device = stm32f4xx_hal::pac, peripherals = true,
      dispatchers = [EXTI0, EXTI1])]
mod app {
    use super::*;

    #[shared]                   // shared between tasks -
    struct Shared {             // access requires .lock()
        counter: u32,
    }

    #[local]                    // owned by exactly one task -
    struct Local {              // NO locking needed
        led: Output<'static>,
    }

    #[init]
    fn init(cx: init::Context) -> (Shared, Local) {
        let dp = cx.device;
        // ... configure clocks, pins, peripherals ...
        blink::spawn().unwrap();
        (Shared { counter: 0 }, Local { led })
    }

    #[task(binds = TIM2, shared = [counter], priority = 3)]
    fn on_timer(mut cx: on_timer::Context) {
        cx.shared.counter.lock(|c| *c += 1);
    }

    #[task(local = [led], priority = 1)]
    async fn blink(cx: blink::Context) {
        loop {
            cx.local.led.toggle();      // NO LOCK - it is
                                        // local
            Mono::delay(500.millis()).await;
        }
    }

    #[idle(shared = [counter])]
    fn idle(mut cx: idle::Context) -> ! {
        loop {
            let n = cx.shared.counter.lock(|c| *c);
            cortex_m::asm::wfi();       // sleep until an
                                        // interrupt
        }
    }
}

/* WHAT RTIC DOES AT COMPILE TIME:

   · PRIORITY ASSIGNMENT: each task has a fixed priority,
     mapped onto NVIC priorities.
   · RESOURCE ANALYSIS: it computes each shared resource's
     PRIORITY CEILING - the highest priority of any task
     using it - and `lock()` raises BASEPRI only to that
     ceiling.
   · SO: a resource shared only between tasks of EQUAL
     priority needs NO critical section at all, and lock()
     compiles to nothing. A resource shared between priority
```

```
    1 and 3 raises BASEPRI to 3 for the duration — blocking
    only what could conflict, not all interrupts.
  · DEADLOCK IS IMPOSSIBLE: the Stack Resource Policy
    guarantees it, and the analysis is done by the macro.
  · `#[local]` resources need no synchronisation at all,
    because the type system proves one owner.

  THIS IS A REAL-TIME KERNEL WITH NO RUNTIME. There is no
  scheduler loop, no task control blocks, no context
  switching code — the NVIC does the scheduling, and RTIC
  only computes the correct priority masks. The generated
  code is what a careful engineer would hand-write, and the
  analysis is what they would get wrong.                */
```

Embassy

```
use embassy_executor::Spawner;
use embassy_stm32::{gpio::{Level, Output, Speed}, i2c::I2c};
use embassy_sync::{channel::Channel,
                   blocking_mutex::raw::ThreadModeRawMutex};
use embassy_time::{Duration, Timer};

static CHANNEL: Channel<ThreadModeRawMutex, Reading, 8>
        = Channel::new();

#[embassy_executor::task]
async fn blink(mut led: Output<'static>) {
    loop {
        led.toggle();
        Timer::after_millis(500).await;
    }
}

#[embassy_executor::task]
async fn sensor(mut i2c: I2c<'static, Async>) {
    let mut buf = [0u8; 2];
    loop {
        // a real DMA transfer - the core SLEEPS during it
        i2c.write_read(0x48, &[0x00], &mut buf).await.unwrap();
        CHANNEL.send(Reading(i16::from_be_bytes(buf))).await;
        Timer::after_secs(1).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());

    spawner.spawn(blink(Output::new(p.PC13, Level::Low,
                                    Speed::Low))).unwrap();
    spawner.spawn(sensor(i2c)).unwrap();

    loop {
        let r = CHANNEL.receive().await;
        defmt::info!("reading: {}", r.0);
    }
}

/* WHAT EMBASSY PROVIDES:
     · an async executor with no heap and no per-task stacks
       (chapter 14)
     · HALs for STM32, nRF, RP2040, ESP32 with ASYNC drivers
       that use DMA and interrupts rather than polling
     · embassy-time: timers, delays, timeouts
     · embassy-sync: Channel, Mutex, Signal, PubSub - all
       no-alloc
     · embassy-net: a full TCP/IP stack
     · embassy-usb: a USB device stack
     · AUTOMATIC POWER MANAGEMENT: the executor executes WFI
       when nothing is ready, so an idle system draws
       microamps with no code from you

/* RTIC OR EMBASSY?

     RTIC     PREEMPTIVE, priority-driven, hard real-time.
              Compile-time resource analysis. Best when
```

```
               latency bounds matter and the work is
               interrupt-shaped.

     EMBASSY   COOPERATIVE, async. Best when the work is
               I/O-shaped - network, USB, several sensors -
               and readability matters. Sequential-looking
               code for concurrent logic.

     BOTH TOGETHER: RTIC can run an Embassy executor in a
     low-priority task, giving hard real-time for the
     critical path and async for everything else. That
     combination is increasingly the answer for a complex
     device.                                                */
```

Tooling and where to look

```
# --- INSTALL ---
    rustup target add thumbv7em-none-eabihf   # Cortex-M4F
    rustup target add thumbv6m-none-eabi      # M0/M0+
    rustup target add riscv32imac-unknown-none-elf
    cargo install probe-rs-tools cargo-binutils cargo-embed \
                  flip-link cargo-generate
    rustup component add llvm-tools

# --- START A PROJECT from a template ---
    cargo generate --git https://github.com/rust-embedded/cortex-m-quickstart
    cargo generate --git https://github.com/embassy-rs/embassy \
                   template

# --- THE DAILY LOOP ---
    cargo run --release         # build, flash, run, stream
                                # logs
    cargo embed                 # a TUI with RTT
    probe-rs attach --chip STM32F411RETx
    cargo size --release -- -A
    cargo bloat --release -n 20

# --- WHERE TO LOOK ---
    https://docs.rust-embedded.org/book/       the Embedded
                                               Book
    https://docs.rust-embedded.org/discovery/  a hands-on
                                               course
    https://embassy.dev/book/
    https://rtic.rs/
    https://github.com/rust-embedded/awesome-embedded-rust
    the Matrix room #rust-embedded:matrix.org

# --- MATURITY, honestly, as of 2026 ---
    EXCELLENT:  Cortex-M (STM32, nRF, RP2040/2350), ESP32,
                RISC-V
    GOOD:       Cortex-A bare metal, drivers for common
                sensors and displays
    EMERGING:   safety certification (Ferrocene is qualified
                for ISO 26262 and IEC 61508), vendor support,
                DSP-heavy work
    THIN:       obscure MCUs, some vendor radio stacks,
                certified medical and avionics toolchains

# CHECK THAT YOUR SPECIFIC CHIP HAS A MAINTAINED HAL before
# committing. A PAC alone is workable but means writing the
# HAL layer yourself.
```

How it shows up

Symptom	Cause and fix
`Peripherals::take()` returns `None`	Called twice — by design
"expected `Alternate<5>`, found `Input`"	Pin not configured for that peripheral — the type-state working
A register field was reset unexpectedly	Used `write` where `modify` was meant
The peripheral does nothing	Clock not enabled — the HAL usually does it, the PAC does not
`clocks.freeze()` panics	The requested clock tree is unachievable — read the constraint it names
A driver will not compile on another chip	It depends on a concrete HAL rather than `embedded-hal`
RTIC reports a priority error	A real ceiling violation, found at compile time
An RTIC task never runs	Not spawned, or no dispatcher interrupt free
Embassy task stalls everything	A blocking call or a loop with no await (ch. 14)
Power never drops	Polling instead of awaiting a real interrupt
No HAL for your chip	Generate a PAC with svd2rust and write the thin HAL layer

Check yourself

- ✓ Draw the layer diagram and say what each layer contributes.
- ✓ Explain what a PAC is generated from and what its type system prevents.
- ✓ Distinguish `read`, `write` and `modify`, and the bug from confusing two.
- ✓ Explain how `take`, `split` and consuming constructors prevent hardware conflicts.
- ✓ Show how pin type-states catch a misconfiguration at compile time.
- ✓ Write a driver generic over `embedded-hal` and say why it is portable and free.
- ✓ Explain what RTIC computes at compile time and why some locks cost nothing.
- ✓ Explain why RTIC cannot deadlock.
- ✓ Write an Embassy application with two tasks and a channel.
- ✓ Choose between RTIC and Embassy for two different products, and say when to use both.

Rust in the Linux kernel

Since 6.1 the kernel accepts Rust drivers. This chapter is what that actually means: how the abstractions are built, what a Rust driver looks like beside the C one from Linux chapter 19, and an honest account of the state of it.

Why the kernel accepted it

```
THE ARGUMENT, made over several years and finally
accepted:

  · roughly two thirds of kernel CVEs are memory safety
    issues - use-after-free, buffer overflow, race
    conditions on refcounts
  · drivers are the largest and least-reviewed part of the
    tree, and where most of those bugs live
  · the existing tooling - KASAN, lockdep, sparse, smatch,
    syzkaller (Linux chapter 27) - finds a great deal and
    still misses a great deal
  · a language that makes the class impossible is
    structurally better than one that makes it detectable

THE CONDITIONS ATTACHED, which shape everything below:

  1. NO REWRITE. C remains the kernel's language; Rust is
     for NEW code, chiefly drivers.
  2. NO SECOND ABI. Rust code calls the existing C
     interfaces through bindings; it does not define a
     parallel kernel API.
  3. C DEVELOPERS ARE NOT REQUIRED TO LEARN RUST. If a C
     change breaks Rust code, fixing it is the Rust
     maintainers' problem.
  4. NO PANICS. A kernel panic is unacceptable; the
     abstractions must return errors instead.
  5. NO IMPLICIT ALLOCATION FAILURE. Kernel allocation can
     fail, so the standard library's infallible
     collections are not usable - the kernel has its own.

TIMELINE:
  6.1 (2022)   the infrastructure merged
  6.3–6.7      abstractions grow; the first real drivers
  6.8+         Binder, the Nova GPU driver, PHY drivers,
               null_blk
  6.12+        Rust is production-supported on several
               architectures

AND THE REALISTIC STATUS in 2026: several shipping
drivers, an actively growing abstraction layer, and the
Android Binder rewrite as the flagship. Not yet routine,
and no longer experimental.
```

What differs from ordinary Rust

```
/* --- 1. NO std, NO alloc-as-you-know-it --- */
    #![no_std]
/* The kernel provides its own `kernel` crate, containing the
   allocation types, synchronisation primitives and
   abstractions. `alloc`'s infallible Vec::push is unusable
   because kernel allocation FAILS.                       */

    use kernel::alloc::{flags, KVec};

    let mut v = KVec::new();
    v.push(42, flags::GFP_KERNEL)?;     // FALLIBLE - returns
                                        // Result

/* NOTE the GFP flag as a parameter - the allocation context
   question from Linux chapter 06 is in the type system,
   where the compiler can see it.

/* --- 2. NO PANICS --- */
    v[0]                         // indexing PANICS - avoid
    v.get(0).ok_or(EINVAL)?      // this instead
/* The kernel builds with panic=abort, and an abort is a
   kernel panic. Every abstraction is written to return
   Result rather than panic, and the same discipline applies
   to driver code (chapter 07).

/* --- 3. THE ERROR TYPE IS AN ERRNO --- */
    use kernel::error::{code::*, Result};

    fn probe(...) -> Result {
        if bad { return Err(EINVAL); }
        let p = ioremap(addr).ok_or(ENOMEM)?;
        Ok(())
    }
/* `Result` defaults to `Result<()>` with the kernel's Error
   type, which wraps a negative errno - so it converts
   directly at the C boundary.

/* --- 4. PINNED INITIALISATION --- */
    #[pin_data]
    struct MyDevice {
        #[pin] lock: SpinLock<State>,
        base: IoMem<0x1000>,
    }
/* Kernel objects frequently contain self-referential
   structures - a list_head that points at itself, a lock
   with an address registered with lockdep. They cannot be
   moved after initialisation. The `pin-init` crate makes
   in-place, fallible construction expressible, which is
   machinery ordinary Rust rarely needs (chapter 14's Pin,
   applied).

/* --- 5. NO EXTERNAL CRATES --- */
/* Everything must be in-tree. No crates.io, no Cargo
   dependency resolution - the kernel build system compiles
   the Rust files directly.                               */
```

A driver, beside the C version

```
// SPDX-License-Identifier: GPL-2.0
//! A minimal platform driver.

use kernel::{
    c_str, device::Core, of, platform, prelude::*,
    sync::aref::ARef,
};

struct MyDriver {
    pdev: ARef<platform::Device>,
    base: IoMem<0x1000>,
}

kernel::of_device_table!(
    OF_TABLE,
    MODULE_OF_TABLE,
    <MyDriver as platform::Driver>::IdInfo,
    [
        (of::DeviceId::new(c_str!("vendor,my-device")), ()),
    ]
);

impl platform::Driver for MyDriver {
    type IdInfo = ();
    const OF_ID_TABLE: Option<of::IdTable<Self::IdInfo>>
        = Some(&OF_TABLE);

    fn probe(
        pdev: &platform::Device<Core>,
        _info: Option<&Self::IdInfo>,
    ) -> Result<Pin<KBox<Self>>> {
        dev_info!(pdev.as_ref(), "probing\n");

        let base = pdev.io_request_and_map(0, c_str!("mydev"))?;

        let id = base.read32(REG_ID);
        if id != EXPECTED_ID {
            dev_err!(pdev.as_ref(), "bad id {:#x}\n", id);
            return Err(ENODEV);
        }

        KBox::pin_init(
            try_pin_init!(MyDriver {
                pdev: pdev.into(),
                base,
            }),
            GFP_KERNEL,
        )
    }
}

impl Drop for MyDriver {
    fn drop(&mut self) {
        // the equivalent of remove() - runs automatically
        dev_info!(self.pdev.as_ref(), "removing\n");
    }
}

kernel::module_platform_driver! {
    type: MyDriver,
```

```
    name: "my_driver",
    author: "Your Name",
    description: "Example platform driver",
    license: "GPL",
}

/* COMPARE THE C VERSION (Linux chapter 19) POINT BY POINT:

   · NO GOTO LADDER. The `?` operator returns early, and Drop
     releases everything acquired so far. The error path
     cannot be wrong because it is not written
     (Linux chapter 26's devm_, but from the language).

   · NO remove() FUNCTION. Drop is the teardown, and it runs
     in reverse construction order, automatically.

   · io_request_and_map RETURNS A TYPE whose Drop unmaps.
     Forgetting is impossible.

   · IoMem<0x1000> CARRIES THE SIZE IN THE TYPE, so
     read32(offset) is bounds-checked at COMPILE TIME where
     the offset is a constant.

   · THE ID TABLE is checked by the macro rather than being a
     hand-written array with a sentinel that can be omitted
     (Linux chapter 17's classic crash).

   · dev_err! is the same dev_err(), with format checking.

   THE C DRIVER IS ABOUT 90 LINES; THE RUST ONE IS ABOUT 60,
   AND THE 30 LINES REMOVED ARE THE ERROR PATHS - which is
   exactly where the bugs were.                          */
```

WHAT THE ABSTRACTION LAYER IS DOING

Between the Rust driver and the C kernel sits a hand-written layer of `unsafe` code that wraps each C interface in a safe Rust API — checking null, enforcing lifetimes, tying refcounts to `Drop`. That layer is the project: writing it correctly, for each subsystem, is most of the work and most of the review. The driver above is safe *because* someone proved the wrapper's invariants once (chapter 11).

The abstractions

```
/* --- SYNCHRONISATION, with the guard pattern --- */
    use kernel::sync::{Mutex, SpinLock, new_mutex};

    #[pin_data]
    struct Shared {
        #[pin] data: Mutex<State>,
    }

    let guard = shared.data.lock();
    guard.field = 5;                    // the guard DEREFS to
                                        // State
    // unlocked when the guard drops - no unlock to forget

/* THE TYPE SYSTEM ENFORCES what lockdep checks at run time
   in C (Linux chapter 11): you cannot touch the data without
   the guard, and the guard cannot outlive the lock.

/* --- REFERENCE COUNTING --- */
    use kernel::sync::aref::ARef;

    let dev: ARef<Device> = pdev.into();
/* ARef is a refcounted reference to a kernel object whose
   Drop calls put_device(). The get/put imbalance that
   leaks a device or frees it early (Windows chapter 03 has
   the same failure mode) is not expressible.

/* --- MMIO --- */
    let base: IoMem<0x1000> = pdev.io_request_and_map(0,
                                        c_str!("mydev"))?;
    let v = base.read32(0x04);          // compile-time bounds
                                        // check
    base.write32(0x08, value);
    base.try_read32(offset)?;           // runtime check, for
                                        // a dynamic offset
/* Volatile by construction, sized by the type, and the
   mapping released on Drop.

/* --- WORK QUEUES, TIMERS, WAIT QUEUES --- */
    use kernel::workqueue::{self, Work, WorkItem};
    use kernel::time::{Delta, Instant};
    use kernel::sync::CondVar;

/* --- CHAR DEVICES and file operations --- */
    use kernel::miscdevice::{MiscDevice, MiscDeviceOptions};

    #[vtable]
    impl MiscDevice for MyDevice {
        type Ptr = Pin<KBox<Self>>;

        fn open(_file: &File, _misc: &MiscDeviceRegistration<Self>)
                -> Result<Self::Ptr> { ... }

        fn ioctl(me: Pin<&Self>, _file: &File,
                 cmd: u32, arg: usize) -> Result<isize> {
            match cmd {
                MY_IOCTL_GET => { ... }
                _ => Err(ENOTTY),
            }
        }
    }
```

```
/* --- USER MEMORY, where the type system earns its keep --- */
    use kernel::uaccess::{UserSlice, UserSliceReader};

    fn read_from_user(ptr: UserPtr, len: usize) -> Result<KVec<u8>> {
        let mut reader = UserSlice::new(ptr, len).reader();
        let mut buf = KVec::with_capacity(len, GFP_KERNEL)?;
        reader.read_all(&mut buf, GFP_KERNEL)?;
        Ok(buf)
    }
/* A UserPtr is a DISTINCT TYPE from a kernel pointer. You
   cannot dereference it; you must go through the checked
   readers. The `__user` annotation that sparse checks in C
   (Linux chapter 08) is a real type here, enforced by the
   compiler rather than by an optional static analyser.  */
```

Building it

```
# --- PREREQUISITES ---
    rustup override set $(scripts/min-tool-version.sh rustc)
    rustup component add rust-src
    cargo install --locked bindgen-cli
    rustup component add clippy rustfmt

# --- CHECK the toolchain is acceptable ---
    make LLVM=1 rustavailable
    # prints exactly what is missing if anything is

# --- CONFIGURE ---
    make LLVM=1 menuconfig
    #   General setup ---> Rust support   [*]
    #   (needs CONFIG_MODVERSIONS=n on some versions)

# --- BUILD ---
    make LLVM=1 -j$(nproc)
    make LLVM=1 rust-analyzer        # generates
                                     # rust-project.json for
                                     # IDE support

# --- THE SAMPLES, which are the best documentation ---
    samples/rust/rust_minimal.rs
    samples/rust/rust_print.rs
    samples/rust/rust_misc_device.rs
    samples/rust/rust_driver_platform.rs
    samples/rust/rust_driver_pci.rs

# --- THE SOURCE TO READ ---
    rust/kernel/            the abstraction layer
    rust/bindings/          bindgen output for the C headers
    rust/macros/            #[vtable], module!, pin_data
    Documentation/rust/     coding guidelines and the design
                            rationale

# --- WRITING A DRIVER ---
    drivers/foo/foo.rs
    # in the Makefile:
    obj-$(CONFIG_FOO) += foo.o
    # in Kconfig:
    config FOO
        tristate "Foo driver"
        depends on RUST

# LLVM=1 IS EFFECTIVELY REQUIRED: rustc uses LLVM, and mixing
# GCC-compiled C with LLVM-compiled Rust works but is less
# well tested, particularly for LTO.
```

An honest assessment

```
/* WHAT WORKS WELL TODAY:
     · character devices and misc devices
     · platform and PCI drivers
     · the synchronisation abstractions - Mutex, SpinLock,
       CondVar, ARef
     · allocation with proper failure handling
     · Binder (Android's IPC) - the largest Rust component in
       the kernel, and in production on millions of devices
     · the Nova GPU driver and several network PHY drivers

/* WHAT IS STILL THIN:
     · many subsystems have no abstractions yet - writing a
       driver for one means writing the wrapper too, which is
       the hard part
     · filesystems, block layer, and much of networking
     · documentation, compared with the C side
     · the abstraction API changes between releases, so
       out-of-tree Rust drivers churn more than C ones

/* THE POLITICAL SITUATION, stated neutrally: some
   maintainers welcome it and some do not, and the "no C
   developer is required to learn Rust" rule means the Rust
   side absorbs the maintenance cost of C changes. That
   arrangement has produced friction in specific subsystems.
   It has also produced shipping code.

/* WHAT IT MEANS FOR YOU:

     LEARNING IT: worthwhile. The abstractions are a
     well-written example of the safe-wrapper discipline
     (chapter 11), and reading rust/kernel/ teaches both Rust
     and the kernel.

     WRITING A DRIVER TODAY: reasonable IF your subsystem has
     abstractions. Check rust/kernel/ first. If it does not,
     you are writing the abstraction layer, which is a much
     larger project and needs to be upstreamed to be useful.

     FOR A PRODUCT: viable for Android-adjacent work, and
     early for most other things. The C path is still the
     default, and choosing Rust should be a decision with a
     reason rather than a preference.

     AND THE DIRECTION IS CLEAR: the trend across Android,
     Windows, and now Linux is that new systems code in
     memory-unsafe languages increasingly needs
     justification. That is worth positioning for regardless
     of what you write this year.                        */
```

How it shows up

Symptom	Cause and fix
`make rustavailable` fails	It names exactly what is missing — usually the rustc version or `rust-src`
"Rust support" not offered in menuconfig	Toolchain check failed, or a conflicting config option
Abstraction does not exist for your subsystem	You must write and upstream it — check `rust/kernel/` first

Symptom	Cause and fix
Code breaks on a kernel upgrade	The abstraction API is not stable — expect churn
Allocation compiles but should not	Use the kernel's fallible `KVec`, not `alloc`'s
A panic reaches the kernel	An `unwrap` or an index — use `get` and `?`
"cannot move out of pinned value"	Kernel objects are pinned — use `pin_init`
Cannot dereference a user pointer	Correct — go through `UserSlice`
Symbol not found at module load	The C symbol is not exported, or not bound in `rust/bindings`
Building with GCC produces oddities	Use `LLVM=1`

A project that uses all of this

OP-TEE chapter 25 builds a signing service in which every layer is Rust: a user-mode driver, a kernel module of the kind this chapter describes, and a trusted application in the secure world. It is also an honest look at the rough edges — the kernel module has to carry its own FFI bindings, because there is no `kernel::tee` abstraction upstream, and the `MiscDevice` and `UserSlice` APIs used here are still moving between releases.

Check yourself

- ✓ Give the argument for Rust in the kernel and the five conditions attached.
- ✓ Name five ways kernel Rust differs from ordinary Rust.
- ✓ Explain why `KVec::push` takes a GFP flag and returns `Result`.
- ✓ Write a platform driver and compare it point by point with the C version.
- ✓ Explain what replaces the `goto` ladder and the `remove()` function.
- ✓ Explain what the abstraction layer is and why it is the real work.
- ✓ Say what `ARef`, `IoMem` and `UserSlice` each make impossible.
- ✓ Explain why kernel objects need pinned initialisation.
- ✓ Build a kernel with Rust support and run a sample.
- ✓ Give an honest recommendation for three different project situations.

> **END OF THE RUST TRACK**
> Seventeen chapters, from ownership to the kernel. You have the model, the embedded ecosystem, the unsafe discipline and the FFI boundary — enough to write a firmware image or a driver and to judge when Rust is the right choice. Next: cryptography, where correctness matters more than anywhere else in this curriculum.

TRACK

Bare metal & RTOS

Cortex-M from the reset vector up: startup code, linker scripts, interrupts, peripherals, and a preemptive RTOS you build yourself.

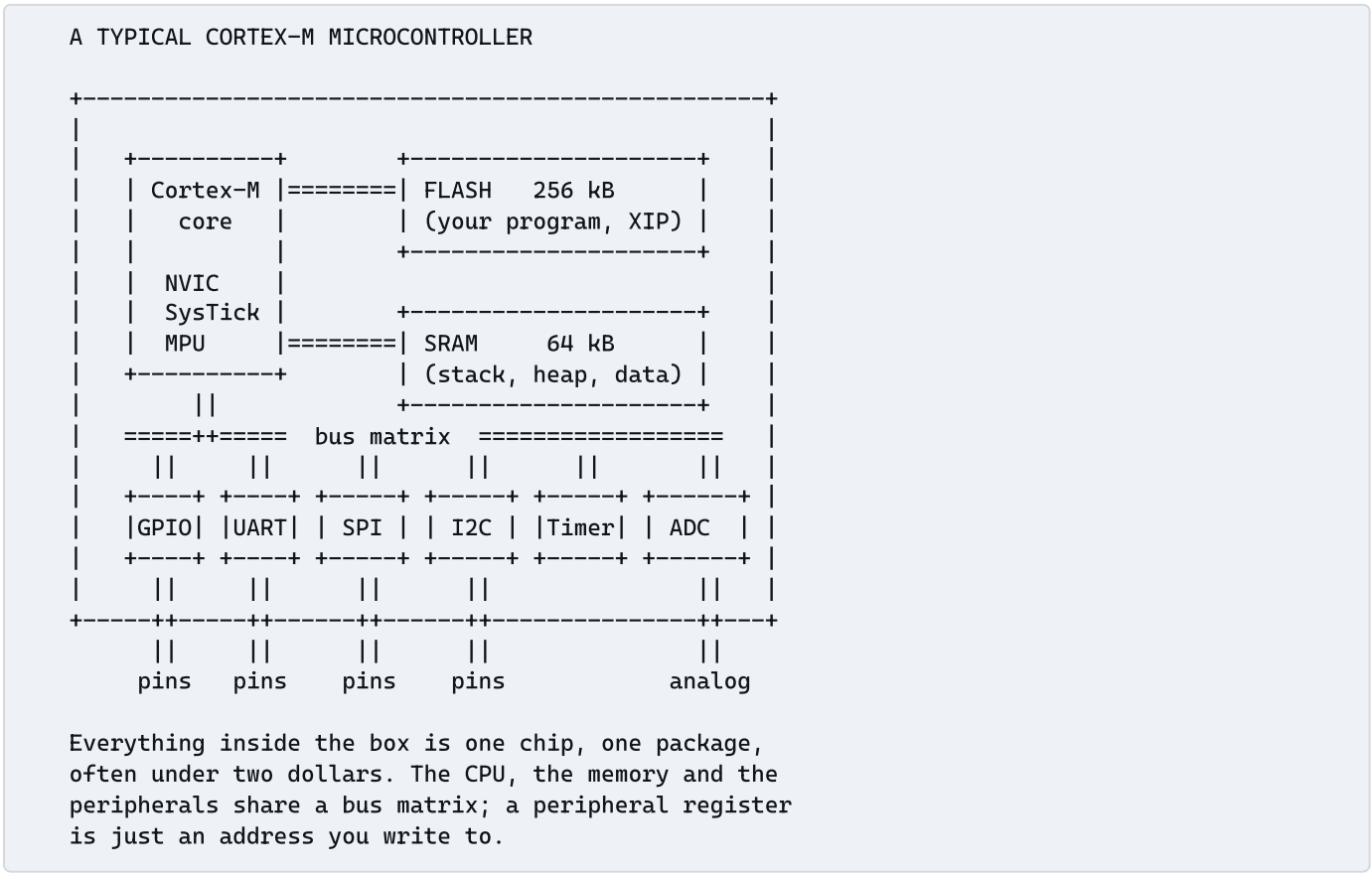
24 chapters

What a microcontroller is, and how it differs

Everything else on this site so far has assumed a processor with an MMU, hundreds of megabytes of DRAM and an operating system underneath you. A microcontroller has none of those. This chapter establishes what you actually get, because every technique in this track follows from the constraints.

One chip, one program

A microcontroller is a complete computer on a single die: a CPU core, its memory, and the peripherals that connect it to the outside world. It boots from its own on-chip flash into your code, and your code is the only code. There is no bootloader chain to negotiate with, no kernel to ask permission from, and no other process competing for the CPU — unless you write one.



The consequence that matters most: **there is no virtual memory**. A pointer holds a physical address. Address `0x40020000` is not "wherever the kernel decided to map the GPIO block" — it is the GPIO block, permanently, as documented in the reference manual. This makes the machine dramatically easier to reason about and dramatically less forgiving, in exactly the ways you would expect.

Against an application processor

	Microcontroller (Cortex-M)	Application processor (Cortex-A)
Clock	16–500 MHz	1–3 GHz
RAM	4 kB – 1 MB, on-chip SRAM	512 MB – 16 GB, external DRAM
Program store	On-chip flash, executed in place	eMMC or SSD, copied to DRAM

	Microcontroller (Cortex-M)	Application processor (Cortex-A)
MMU	None. An optional MPU with ~8 regions	Full MMU, multi-level page tables
Privilege levels	Two (Thread/Handler, privileged/unprivileged)	Four (EL0–EL3) plus secure world
Caches	Often none; sometimes a small I-cache	L1/L2/L3, coherent, with a whole protocol
Operating system	None, or an RTOS you link into your binary	Linux, measured in tens of millions of lines
Boot time	Microseconds to milliseconds	Hundreds of milliseconds to minutes
Interrupt latency	12 cycles, deterministic	Microseconds, and not guaranteed
Sleep current	Hundreds of nanoamps	Milliamps at best
Unit cost	$0.30 – $10	$5 – $200

> **THE TRADE IS DETERMINISM AGAINST CAPABILITY**
>
> A Cortex-A core is faster in every benchmark and worse at every deadline. Its caches, branch predictors, MMU and scheduler all improve *average* performance by introducing variance. A Cortex-M has no cache to miss, no TLB to walk and no scheduler to preempt you, so the time between an electrical edge on a pin and the first instruction of your handler is a number you can look up and rely on. When the requirement is "the motor commutates within 5 µs, every time, for ten years", that number is the whole product.

Execute in place, and why RAM is precious

Your program is not loaded into RAM. The CPU fetches instructions directly from flash over a dedicated bus — *execute in place*, XIP. Flash is comparatively plentiful and slow; SRAM is fast and scarce. That inverts habits learned on a desktop:

```
/* On a microcontroller these two lines have completely
   different costs.                                        */

const char msg[] = "sensor fault";       /* Ideally: flash. */
char       buf[] = "sensor fault";       /* Always: SRAM,   */
                                         /* AND a copy of   */
                                         /* the initialiser */
                                         /* in flash too.   */

/* The second costs 13 bytes of your 64 kB of SRAM, plus 13
   bytes of flash for the initialiser that startup code
   copies into it (chapter 06). If it does not need to be
   written, it must be const - that is not a style rule
   here, it is a memory budget.

   Toolchains will place const data in flash only if you let
   them. On AVR and some others you need an explicit
   attribute; on Cortex-M, const alone is enough.          */

const char *p = msg;    /* p is 4 bytes of SRAM pointing at flash */
```

A 64 kB SRAM budget sounds generous until you count: every task's stack, every DMA buffer, every driver's ring buffer, the heap if you have one, and every non-const global. Running out is not an `ENOMEM` you can handle — it is a stack that silently grows into your variables. Chapter 20 is about making that impossible rather than unlikely.

A peripheral is an address

On Linux you request a GPIO from a subsystem that arbitrates between drivers. Here, you write to a register. The entire abstraction is a struct overlaid on a fixed address:

```
/* Memory-mapped I/O, the whole idea in one block. */

#include <stdint.h>

typedef struct {
    volatile uint32_t MODER;    /* 0x00  mode: in/out/alt/analog */
    volatile uint32_t OTYPER;   /* 0x04  push-pull or open-drain */
    volatile uint32_t OSPEEDR;  /* 0x08  slew rate                */
    volatile uint32_t PUPDR;    /* 0x0C  pull-up / pull-down      */
    volatile uint32_t IDR;      /* 0x10  input data   (read-only) */
    volatile uint32_t ODR;      /* 0x14  output data              */
    volatile uint32_t BSRR;     /* 0x18  atomic set/reset         */
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) 0x40020000UL)

/* Set PA5 high. One store. No system call, no driver, no
   lock, and roughly two clock cycles.                     */
GPIOA->BSRR = (1U << 5);

/* volatile is mandatory, not decorative. Without it the
   compiler may cache the value in a register, reorder the
   access, or delete it entirely - all legal, because C's
   abstract machine has no concept of a wire attached to
   memory. See the C track, chapter 10.                    */
```

> **`VOLATILE` IS NECESSARY BUT NOT SUFFICIENT**
>
> It stops the *compiler* reordering and eliding accesses. It says nothing to the *hardware* about write buffers or bus ordering, and nothing about atomicity. Two `volatile` writes to different peripherals can still land out of order at the far side of a bus bridge. On Cortex-M that is rare enough to ignore for GPIO and common enough to matter when you are clearing an interrupt flag and returning from the handler — a case we return to in chapter 15.

Nothing underneath you

The most disorienting part of moving from Linux to bare metal is how much simply is not there.

```
WHAT YOU LOSE                  WHAT YOU GAIN

printf to a console       ->   a UART you configure first
                               (chapter 16 has cheaper options)

malloc that works         ->   a heap you must justify,
                               size, and usually avoid

"segmentation fault"      ->   silent corruption, or a
                               HardFault you must decode
                               yourself (chapter 15)

a scheduler               ->   a superloop, until you write
                               one (chapters 17-18)

drivers for everything    ->   the reference manual and
                               1200 pages of register tables

process isolation         ->   one address space; any bug
                               can reach any byte

time from the OS          ->   a timer you configure, whose
                               accuracy is your crystal's

The trade is total visibility. There is no layer between
your code and the silicon that you cannot read, single-step,
and change. Every problem is ultimately findable, because
there is nowhere for it to hide.
```

Choosing between them

The decision is usually made for you by three constraints, in this order:

- **Power.** If the product runs from a coin cell for a year, it is a microcontroller. No application processor idles in the microamp range. This one constraint decides more designs than any other.
- **Determinism.** If missing a deadline is a safety event rather than a dropped frame, you want a core with no cache and no scheduler — or a system that has both, with an MCU beside it doing the part that must not miss.
- **What the software needs.** A TCP/IP stack, a filesystem and a display fit on a modern MCU. A web browser, a camera pipeline and a package manager do not. If you need Linux's driver ecosystem, you need Linux, and you have chosen an application processor.

Plenty of real products contain both: an application processor for the interface and connectivity, and a microcontroller for the real-time control loop and power management, talking over SPI or I2C. Knowing both sides of that link, and why the split exists, is most of what this track and the next one are for.

How it shows up

Symptom	Cause
Variables change value with no code writing them	Stack overflow — no MMU means no guard page. Size the stacks (chapter 20)
A loop reading a status register never exits	Missing `volatile`; the compiler hoisted the load out of the loop
Program works in debug, fails at `-O2`	Almost always `volatile`, timing assumptions, or undefined behaviour the optimiser exploited

Symptom	Cause
Linker reports "region RAM overflowed"	Too many non-const globals or too large a buffer; check what should have been `const`
Adding one small array breaks unrelated code	You were already out of SRAM; the new array moved the collision somewhere visible
`printf` hangs or emits nothing	No console exists until you configure a UART and retarget the C library (chapter 16)
Code runs once then the chip appears dead	A fault handler with an empty `while(1)` — the default trap (chapter 15)
Timing differs between two identical boards	The crystal, its load capacitors, or the internal oscillator's tolerance (chapter 09)
Sleep current is 3 mA instead of 2 µA	A GPIO floating, a peripheral clock left enabled, or the debugger still attached (chapter 13)
Writing a peripheral register has no effect	Its clock is not enabled. On most MCUs a peripheral with no clock silently ignores writes (chapter 09)

Check yourself

- ✓ Name the four things a microcontroller integrates onto one die.
- ✓ Explain what "execute in place" means and why it makes SRAM the scarce resource.
- ✓ State what a pointer holds on a system with no MMU, and one consequence for debugging.
- ✓ Give three concrete differences between a Cortex-M and a Cortex-A that affect interrupt latency.
- ✓ Explain why `const` is a memory-budget decision here rather than a style preference.
- ✓ Say what `volatile` guarantees, and name one thing it does not.
- ✓ Describe what happens on a stack overflow with no MMU, and why it is hard to find.
- ✓ List the three constraints that usually decide MCU versus application processor.
- ✓ Explain why a peripheral register write can silently do nothing.
- ✓ Give an example of a product that would contain both kinds of processor, and say what each does.

The Cortex-M profile: ARMv7-M and ARMv8-M

The ARM track on this site covers the A profile — AArch64, exception levels, page tables. The M profile is a different architecture with a different instruction set, a different exception model and a different memory map. This chapter is the map of what M means, and which M you are holding.

Three profiles, one company

ARM defines three architecture profiles, and they are not variants of each other. Code, tooling and mental models transfer only partly between them.

Profile	For	Defining feature
A — Application	Phones, servers, anything running Linux	MMU, virtual memory, exception levels EL0–EL3
R — Real-time	Storage controllers, basebands, safety islands	MPU rather than MMU, but A-like performance and often lockstep cores
M — Microcontroller	Everything in this track	Thumb-only, hardware exception entry, deterministic latency, tiny gate count

The ARM architecture track teaches the A profile. Almost none of its exception-level material applies here: an M-profile core has no EL1, no `SPSR_EL1`, no vector base you set up with a table of branch instructions, and no MMU chapter to read. What transfers is the memory model vocabulary, the barrier instructions, and the general shape of a load/store architecture.

Which M profile is this chip?

```
ARCHITECTURE          CORES              WHAT IT ADDS

ARMv6-M               Cortex-M0          Thumb + a few
                      Cortex-M0+         32-bit instrs.
                      Cortex-M1          Smallest possible.
                                         No bit-banding,
                                         8 IRQ priorities
                                         (2 bits), fixed
                                         vector table.

ARMv7-M               Cortex-M3          Full Thumb-2,
                                         hardware divide,
                                         bit-banding,
                                         up to 256 IRQs,
                                         relocatable VTOR.

ARMv7E-M              Cortex-M4          + DSP (SIMD) instrs
                      Cortex-M7          + optional FPU
                                         M7: caches, dual
                                         issue, TCM.

ARMv8-M Baseline      Cortex-M23         v6-M + TrustZone-M

ARMv8-M Mainline      Cortex-M33         v7E-M + TrustZone-M
                      Cortex-M35P        + stack limit regs

ARMv8.1-M             Cortex-M55         + Helium (MVE),
                      Cortex-M85         vector extension
                                         for ML and DSP

The practical split: v6-M is deliberately minimal and you
will notice the missing instructions; v7-M and above are
comfortable. TrustZone-M appears only at v8-M.
```

READING A PART NUMBER

"STM32F407" tells you the vendor (ST), the family (F4) and the peripheral set — but the core is only implied. The reference manual's first pages state it: Cortex-M4 with FPU, ARMv7E-M. That distinction decides whether you can use `__disable_irq` with a BASEPRI level, whether `float` costs one cycle or forty, and whether the vector table can be moved. Establish it before writing a line of code.

Thumb-2, and only Thumb-2

An M-profile core executes Thumb instructions exclusively. There is no A32/ARM state to switch into, and no AArch64 at all. Attempting to switch to ARM state is not a mode change — it is a fault.

```
/* Thumb-2 mixes 16-bit and 32-bit encodings freely. The
   assembler picks; you can force it with .w and .n suffixes. */

    movs  r0, #1          @ 16-bit encoding, 2 bytes
    mov.w r0, #0x12345678 @ 32-bit encoding, 4 bytes
    add   r1, r2, r3      @ assembler chooses

/* Every branch target must be Thumb, and the architecture
   encodes that in the LOW BIT of the address:              */

    ldr   r0, =my_function + 1   @ bit 0 SET = Thumb
    bx    r0                     @ branch and exchange

/* This is why every entry in the vector table is an odd
   number, and why a function pointer that looks "one byte
   off" is correct. Clear that bit and the core takes a
   UsageFault with INVSTATE set - it believes you asked it to
   execute ARM-state code, which it cannot do.

   You will meet this exactly once, when hand-writing a
   vector table or a context switch, and it will cost you an
   afternoon if nobody warned you.                          */
```

The core peripherals are part of the architecture

This is the single most useful structural fact about the M profile, and it has no equivalent on Cortex-A. The interrupt controller, the system timer, the fault registers and the MPU are defined by the *architecture*, at fixed addresses, identically on every vendor's chip.

```
   THE ARCHITECTURALLY FIXED MEMORY MAP

   0x00000000  +---------------------------+
               | Code      (flash, XIP)    |  512 MB region
   0x20000000  +---------------------------+
               | SRAM                      |  512 MB region
   0x40000000  +---------------------------+
               | Peripheral (vendor's)     |  512 MB region
   0x60000000  +---------------------------+
               | External RAM              |
   0xA0000000  +---------------------------+
               | External device           |
   0xE0000000  +---------------------------+
               | Private Peripheral Bus    |  <-- the core's
               |   0xE000E010  SysTick     |      own blocks
               |   0xE000E100  NVIC        |
               |   0xE000ED00  SCB         |
               |   0xE000ED90  MPU         |
               |   0xE0001000  DWT         |
   0xFFFFFFFF  +---------------------------+

   Regions also carry default memory ATTRIBUTES: the
   Peripheral and Device regions are non-cacheable and
   strongly ordered, so a store to 0x40020000 is not
   buffered the way a store to SRAM might be. You get
   sane peripheral behaviour without configuring anything.
```

So `NVIC_EnableIRQ()` is the same code on an ST, an NXP and a Nordic part, because the NVIC is at `0xE000E100` on all of them. Everything above `0xE0000000` is ARM's; the `0x40000000` region is the vendor's, and that is where the reference manual takes over from the architecture reference manual.

> **TWO MANUALS, AND KNOWING WHICH ONE ANSWERS YOUR QUESTION**
> The *ARMv7-M Architecture Reference Manual* defines the core: instructions, exceptions, NVIC, SysTick, MPU, fault registers. The *vendor reference manual* defines everything hanging off the bus matrix: clocks, GPIO, UART, timers, DMA. When a fault register puzzles you, reach for ARM's manual. When a peripheral does not respond, reach for the vendor's. Reaching for the wrong one is the most common way to waste an hour.

Hardware exception entry

On Cortex-A, taking an exception gives you a branch to a vector and the job of saving state yourself, in assembly. On Cortex-M the hardware does it: it pushes eight registers onto the active stack, loads the handler's address from the vector table, and enters it. Your handler is an ordinary C function.

```
/* This is a complete, correct interrupt handler. There is no
   assembly wrapper, no manual register save, no eret.       */

void TIM2_IRQHandler(void)
{
    TIM2->SR = ~TIM_SR_UIF;   /* clear the flag in hardware */
    ticks++;
}

/* The hardware stacked R0-R3, R12, LR, PC and xPSR on entry
   and will unstack them on return. Those are exactly the
   caller-saved registers of the ARM procedure call standard,
   which is why a plain C function is a valid handler: the
   compiler already preserves everything else.

   This is the "AAPCS-compatible exception model", and it is
   the reason M-profile interrupt latency is a fixed 12
   cycles rather than "however long your prologue takes".    */
```

Chapter 07 covers the vector table and NVIC in detail, and chapter 08 covers what happens when a second interrupt arrives during the first — where tail-chaining and late-arrival make the model faster than the naive description suggests.

Floating point, DSP and Helium

```
/* Three separate optional extensions. Knowing which you have
   changes how you write numerical code.                      */

/* 1. FPU (M4F, M7, M33). Single precision in one cycle.
      Double precision is software on almost everything.     */
float  a = x * y + z;   /* 1 instruction: VFMA.F32          */
double b = x * y + z;   /* ~100 cycles: a library call       */

/* 2. DSP extension (M4, M7, M33). SIMD within 32-bit
      registers, and a single-cycle multiply-accumulate.     */
sum = __SMLAD(pair1, pair2, sum);  /* two 16x16 MACs at once */

/* 3. Helium / MVE (M55, M85). True 128-bit vectors,
      the M-profile answer to NEON.                          */

/* If the part has no FPU, every float operation becomes a
   call into libgcc. Code written assuming hardware float can
   be 40x slower on an M0 without a single warning - it
   compiles and runs, it is simply slow. This is the classic
   "port the algorithm to the cheaper chip" trap.

   Fixed-point arithmetic is the answer, and it is why Q15
   and Q31 formats persist in embedded DSP code.             */
```

> **AN FPU CHANGES YOUR CONTEXT SWITCH**
> The FPU adds 33 more registers. If more than one task uses floating point, the RTOS must save and restore them, which roughly triples context-switch cost. Cortex-M supports *lazy stacking* — reserving the space but only writing the registers when a second FP-using context actually appears — and it is enabled by default. If you hand-write a context switch (chapter 18) and forget the FP registers exist, you get corruption that only appears once two tasks both do float maths.

How it shows up

Symptom	Cause
UsageFault with `INVSTATE` set	Branched to an address with bit 0 clear — the core thinks you want ARM state
Hard fault immediately on the first interrupt	Vector table entries written without the Thumb bit set
Code from a tutorial does not compile for this part	It targets v7-M; you have a v6-M core without those instructions
Setting `VTOR` has no effect	ARMv6-M has no VTOR — the vector table is fixed at address 0
Float maths is inexplicably slow	No FPU, or the FPU is present but not enabled in `CPACR`
Enabling the FPU causes faults on first float use	`CPACR` written but no barrier before the next FP instruction
Corruption once two tasks use `float`	Context switch does not save the FP register bank
Priority levels behave coarsely	The part implements only 3 or 4 priority bits, not 8 (chapter 08)
Code portable between vendors, drivers are not	Correct and expected: the core is ARM's, the peripherals are the vendor's
`__disable_irq` works, BASEPRI does not	BASEPRI does not exist on ARMv6-M

Check yourself

- ✓ Name the three ARM architecture profiles and the defining feature of each.
- ✓ State which ARM architecture version introduced TrustZone-M.
- ✓ Explain why every vector table entry is an odd address.
- ✓ Say what happens if you branch to an address with bit 0 clear.
- ✓ Give the address of the NVIC and explain why it is the same on every vendor's chip.
- ✓ Explain which manual answers a question about a fault register, and which about a UART.
- ✓ List the eight registers the hardware stacks on exception entry, and say why those eight.
- ✓ Explain why an ordinary C function can serve as an interrupt handler on Cortex-M.
- ✓ Describe what lazy FP stacking does and the bug it hides in a hand-written context switch.
- ✓ Explain why a `double` can be a hundred times slower than a `float` here.

Registers, modes and the two stack pointers

Sixteen registers, two modes, two privilege levels and two stack pointers. That is the entire programmer's model, and the whole of an RTOS context switch is built from it. This chapter is the reference you will come back to when writing chapter 18.

The register file

```
+-------+
|  R0   |  \
|  R1   |   |  Caller-saved (AAPCS): arguments and
|  R2   |   |  scratch. The HARDWARE stacks these
|  R3   |  /   on exception entry.
+-------+
|  R4   |  \
|  R5   |   |
|  R6   |   |
|  R7   |   |  Callee-saved. The hardware does NOT
|  R8   |   |  stack these. If your context switch
|  R9   |   |  needs them saved, you save them.
|  R10  |   |
|  R11  |  /
+-------+
|  R12  |     IP, intra-procedure scratch. Hardware-stacked.
+-------+
|  R13  |     SP - banked! MSP or PSP (see below)
+-------+
|  R14  |     LR, link register. Hardware-stacked.
+-------+
|  R15  |     PC. Hardware-stacked.
+-------+
|  xPSR |     Program status. Hardware-stacked.
+-------+

The eight hardware-stacked registers are exactly
R0-R3, R12, LR, PC, xPSR. That set is chosen to match
the procedure call standard's caller-saved set, which
is what makes a plain C function a legal handler.
```

Note what is absent compared with AArch64: no `X0–X30`, no separate `SP_EL0` and `SP_EL1` per exception level, no `ELR`. The M profile is deliberately small.

xPSR: three registers at one address

```
/* xPSR is the union of three views, and you can read them
   individually or together.                                  */

/*  31 30 29 28 27      24        16      9  8       0
   +--+--+--+--+--+-----+--+-------+------+---------+
   | N| Z| C| V| Q|     |T |  ICI  |      | ISR_NUM |
   +--+--+--+--+--+-----+--+-------+------+---------+
     APSR (flags)        ^   EPSR         IPSR
                         |
                    THUMB BIT - must be 1, always.        */

/* APSR - condition flags, exactly as on any ARM core.
   Q is the DSP saturation sticky flag.

   IPSR - which exception is executing. 0 means Thread mode,
   i.e. ordinary code. This is how you answer "am I in an
   interrupt?" at runtime:                                  */

static inline int in_handler_mode(void)
{
    uint32_t ipsr;
    __asm volatile ("mrs %0, ipsr" : "=r" (ipsr));
    return ipsr != 0;
}

/* An RTOS uses precisely this to decide whether a call is
   the ISR-safe variant. FreeRTOS's configASSERT checks on
   xQueueSend versus xQueueSendFromISR come down to this
   register (chapter 19).

   EPSR - execution state, including the T bit. It cannot be
   written directly by software; it is restored from the
   stacked xPSR on exception return. Clearing the T bit in a
   stacked frame is a reliable way to fault on return.       */
```

Two modes, two privilege levels

```
                  PRIVILEGED          UNPRIVILEGED

   HANDLER mode   always here         cannot happen
   (in an
    exception)    Uses MSP.
                  IPSR != 0.

   THREAD mode    reset lands here    optional; set by
   (ordinary                          writing CONTROL.nPRIV
    code)         Uses MSP or PSP,
                  chosen by
                  CONTROL.SPSEL      An RTOS runs tasks
                                     here, with the MPU
                                     enforcing isolation
                                     (chapter 14).

   Handler mode is ALWAYS privileged. There is no way to take
   an interrupt into unprivileged code, which is why fault
   handlers can always reach the fault registers.
```

```
/* The CONTROL register - only three bits, both of which
   you will use in an RTOS.                                    */

/*   bit 2  FPCA   floating-point context active (read-only
                   to you; the hardware sets it)
     bit 1  SPSEL  0 = use MSP in Thread mode
                   1 = use PSP in Thread mode
     bit 0  nPRIV  0 = Thread mode privileged
                   1 = Thread mode unprivileged            */

static inline void switch_to_psp_unprivileged(void)
{
    __asm volatile ("msr control, %0" :: "r" (0x3) : "memory");
    __asm volatile ("isb");          /* MANDATORY - see below */
}

/* The ISB is not optional. Writing CONTROL changes how the
   very next instruction fetches its stack; without a barrier
   the pipeline may already hold instructions decoded under
   the old setting. This is the single most common omission
   in hand-written RTOS startup code, and it produces a fault
   that moves when you add a printf.                           */
```

> **YOU CANNOT RAISE YOUR OWN PRIVILEGE**
> `CONTROL.nPRIV` can be set from privileged code to drop to unprivileged, but unprivileged code cannot clear it — the write is ignored. The only way back to privileged execution is to take an exception, which is exactly what an `SVC` instruction is for. That is the entire system-call mechanism of an M-profile RTOS, and it is the same idea as `svc` on Cortex-A with about one percent of the machinery.

Two stack pointers, and why

R13 is banked: there are two physical registers, and which one you see as `SP` depends on the mode and `CONTROL.SPSEL`.

```
   MSP - Main Stack Pointer
     Reset value loaded from vector table entry 0.
     Used by: Handler mode, always.
              Thread mode when SPSEL = 0.
     Holds: interrupt frames, and the main() stack in a
            bare-metal program.

   PSP - Process Stack Pointer
     Not initialised by hardware; you set it.
     Used by: Thread mode when SPSEL = 1.
     Holds: the current task's stack in an RTOS.

   BARE METAL              RTOS
   +-----------+           +-----------+   +-----------+
   |   main()  | MSP       | task A    |   | task B    | PSP
   |   + ISRs  |           | stack     |   | stack     | (switched)
   +-----------+           +-----------+   +-----------+
                                 |
                           +-----------+
                           |   ISRs    | MSP (shared, one only)
                           +-----------+

   The split means an interrupt storm cannot overflow a
   task's stack into another task's data - ISRs have their
   own stack. It also means you size the MSP for the worst
   nesting depth of interrupts, and each PSP for one task.
   That is a far easier budget to get right than one stack
   for everything.
```

```
/* Reading and writing the banked pointers. */

static inline uint32_t get_psp(void)
{
    uint32_t r; __asm volatile ("mrs %0, psp" : "=r" (r)); return r;
}
static inline void set_psp(uint32_t v)
{
    __asm volatile ("msr psp, %0" :: "r" (v));
}

/* Note there is no barrier needed after MSR PSP itself -
   you are not changing WHICH stack pointer is in use, only
   its value. The barrier is needed after writing CONTROL,
   which changes the selection.                              */
```

EXC_RETURN: the magic LR value

On exception entry the hardware does not put a return address in `LR`. It puts a special pattern with the top 24 bits set, encoding which stack to unwind from and whether FP state is present.

```
/* EXC_RETURN values you will actually see (ARMv7-M):

   0xFFFFFFF1  return to Handler mode, MSP,  no FP frame
   0xFFFFFFF9  return to Thread  mode, MSP,  no FP frame
   0xFFFFFFFD  return to Thread  mode, PSP,  no FP frame
   0xFFFFFFE9  return to Thread  mode, MSP,  with FP frame
   0xFFFFFFED  return to Thread  mode, PSP,  with FP frame

     bit 2 = 1  ->  return uses PSP
     bit 4 = 0  ->  extended (FP) stack frame
     bit 3 = 1  ->  returning to Thread mode               */

/* Executing BX LR with this value in LR is what returns from
   an exception - there is no ERET instruction. The core sees
   the 0xFF... pattern in a branch target and interprets it
   as "unstack and return" rather than an address.

   A context switch works by CHANGING which stack PSP points
   at while inside PendSV, then returning with 0xFFFFFFFD.
   The hardware unstacks the new task's frame and the CPU is
   simply running different code. Chapter 18 builds this.    */
```

THIS IS THE WHOLE TRICK OF AN M-PROFILE CONTEXT SWITCH

There is no privileged "load all registers" instruction and no dedicated switch mechanism. You save R4–R11 by hand, change `PSP`, restore R4–R11 from the new stack, and `BX` to `EXC_RETURN`. The hardware's automatic unstacking does the rest. Once you see that the exception return path *is* the switch, the twenty lines of assembly in chapter 18 stop looking like magic.

Interrupt masking registers

```
/* Three ways to turn interrupts off, in increasing subtlety. */

/* PRIMASK - one bit. Blocks everything except NMI and
   HardFault. The blunt instrument.                          */
__disable_irq();          /* cpsid i  -> PRIMASK = 1 */
/* ... critical section ... */
__enable_irq();           /* cpsie i  -> PRIMASK = 0 */

/* FAULTMASK - like PRIMASK but also blocks HardFault.
   Cleared automatically on exception return. Rarely used
   outside fault recovery.                                   */

/* BASEPRI - block only interrupts at or below a priority.
   ARMv7-M and above only. This is what an RTOS uses, so
   that high-priority "zero-latency" interrupts keep running
   through kernel critical sections.                         */
__set_BASEPRI(5 << (8 - __NVIC_PRIO_BITS));
/* ... kernel critical section ...
   Interrupts of priority 0-4 still fire and must NOT call
   any RTOS API. Priorities 5+ are blocked and may.          */
__set_BASEPRI(0);

/* Note the shift. Priority bits are stored in the HIGH bits
   of the byte, so a part with 4 implemented bits treats
   0x50 and 0x5F identically. Forgetting the shift is why
   "my BASEPRI does nothing" is a common question - writing
   5 instead of 5<<4 sets a priority so low it blocks
   nothing. Chapter 08 covers this properly.                 */
```

How it shows up

Symptom	Cause
Fault a few instructions after writing `CONTROL`	Missing `ISB`; the pipeline still holds instructions decoded for the old stack
Fault moves or vanishes when you add a print	Same missing barrier — you changed the pipeline timing
Task stacks corrupt each other under interrupt load	Tasks and ISRs sharing MSP; move tasks to PSP
Unprivileged code cannot regain privilege	Correct and by design — use `SVC`
Context switch corrupts variables in some tasks	R4–R11 not saved; the hardware only stacks the caller-saved eight
Corruption appears only once two tasks use float	FP registers not saved; check `EXC_RETURN` bit 4
Fault on exception return	Stacked xPSR has the T bit clear, or `EXC_RETURN` was overwritten
`BASEPRI` appears to do nothing	Value not shifted into the implemented high priority bits
`BASEPRI` does not exist	ARMv6-M core — only `PRIMASK` is available
RTOS assert fires inside an ISR	Called the non-`FromISR` API; the kernel checked `IPSR`

Check yourself

- ✓ List the eight registers stacked by hardware and explain why that particular set.
- ✓ Say which registers a context switch must save by hand, and why.
- ✓ Explain how to determine at runtime whether you are in an interrupt.

- ✓ State the two modes and two privilege levels, and which combination cannot occur.
- ✓ Explain why an `ISB` is required after writing `CONTROL`.
- ✓ Give two concrete benefits of running tasks on PSP and interrupts on MSP.
- ✓ Explain how unprivileged code can legitimately reach privileged execution.
- ✓ Decode `0xFFFFFFFD` as an `EXC_RETURN` value.
- ✓ Describe how changing PSP inside PendSV performs a task switch.
- ✓ Explain the difference between PRIMASK and BASEPRI, and why an RTOS prefers the latter.

Reset: from power-on to your first instruction

On Linux the boot sequence is a chain of loaders you inherit. Here it is eleven microseconds long, entirely yours, and defined by two 32-bit words at address zero. This chapter follows the electrons from the power rail to `main()`.

The sequence

```
POWER APPLIED
     |
     v
+----------------------+
| Brown-out reset holds |  The BOR circuit keeps the core
| the core in reset     |  in reset until VDD crosses the
| until VDD is stable   |  threshold. Typically 1.7-2.0 V.
+----------------------+
     |
     v
+----------------------+
| Internal oscillator   |  The chip starts on an internal
| starts (HSI/MSI)      |  RC oscillator - fast to start,
|                       |  a few percent accurate. Your
|                       |  crystal is not running yet.
+----------------------+
     |
     v
+----------------------+
| Option bytes / boot   |  Boot pins or option bits select
| pins sampled          |  what is aliased to 0x00000000:
|                       |  main flash, system ROM (the
|                       |  vendor's serial bootloader), or
|                       |  SRAM.
+----------------------+
     |
     v
+----------------------+
| READ TWO WORDS        |  This is the whole handoff.
|  [0x00000000] -> MSP  |  Word 0: initial stack pointer
|  [0x00000004] -> PC   |  Word 1: Reset_Handler address
+----------------------+
     |
     v
Reset_Handler executes  (chapter 06 - copy .data, zero .bss)
     |
     v
SystemInit()            (chapter 09 - clocks and PLL)
     |
     v
main()

Elapsed: microseconds. There is no bootloader, no device
tree, no ELF loader, no relocation. The hardware loads two
registers and starts fetching.
```

> **THE FIRST TWO WORDS ARE THE ENTIRE BOOT PROTOCOL**
> Everything else — the vendor's HAL, CMSIS, your RTOS — is software convention layered on top. The architecture's contract is: at the address aliased to zero, put a stack pointer and an entry address. If the first word is not a valid, aligned, writable RAM address, the core faults before executing one instruction of your code, and it faults in a way that is very hard to debug because nothing of yours has run yet.

What actually sits at address zero

```
/* The start of a real vector table, in C. The array IS the
   boot protocol - the linker places it at the base of
   flash, and the hardware reads it directly.               */

extern uint32_t _estack;          /* from the linker script */
void Reset_Handler(void);
void NMI_Handler(void);
void HardFault_Handler(void);

__attribute__((section(".isr_vector"), used))
void (* const g_vectors[])(void) = {
    (void (*)(void)) &_estack,    /* 0x00: initial MSP      */
    Reset_Handler,                /* 0x04: entry point      */
    NMI_Handler,                  /* 0x08                   */
    HardFault_Handler,            /* 0x0C                   */
    MemManage_Handler,            /* 0x10                   */
    BusFault_Handler,             /* 0x14                   */
    UsageFault_Handler,           /* 0x18                   */
    0, 0, 0, 0,                   /* 0x1C-0x28 reserved     */
    SVC_Handler,                  /* 0x2C                   */
    DebugMon_Handler,             /* 0x30                   */
    0,                            /* 0x34 reserved          */
    PendSV_Handler,               /* 0x38                   */
    SysTick_Handler,              /* 0x3C                   */
    /* 0x40 onward: the vendor's peripheral interrupts      */
    WWDG_IRQHandler,
    PVD_IRQHandler,
    /* ... typically 60-100 more ...                        */
};

/* Two things that trip people up:

   1. The first entry is DATA (an address), not a function
      pointer. The cast is a lie that keeps the array one
      type. Some projects use a union to be honest about it.

   2. Every other entry is a function pointer, and the
      compiler automatically sets bit 0 for Thumb (chapter
      02). Hand-writing this table in assembly means setting
      that bit yourself, or faulting on the first interrupt.

   3. `used` matters. Nothing in C refers to g_vectors, so
      LTO or --gc-sections will happily delete it. The
      KEEP() directive in the linker script is the other
      half of this belt-and-braces (chapter 05).            */
```

Why the stack pointer points at the end of RAM

```
0x20010000  +------------------+  <- _estack (64 kB part)
            |                  |
            |   STACK          |  grows DOWNWARD
            |     |            |
            |     v            |
            |                  |
            |   (free)         |
            |                  |
            |     ^            |
            |     |            |
            |   HEAP           |  grows upward, if you
            |                  |  have one at all
            +------------------+  <- _end
            |   .bss           |  zeroed by startup
            +------------------+
            |   .data          |  copied from flash
0x20000000  +------------------+

The stack starts at the TOP because it grows down. Set
_estack to the last address + 1: on a part with 64 kB of
SRAM at 0x20000000, that is 0x20010000, not 0x2000FFFF.
The SP is pre-decremented on push, so the first push
writes to 0x2000FFFC - inside RAM. Off-by-one here gives
you a fault on the very first push.

Note what is NOT here: any guard between stack and heap.
When they meet, they simply overwrite each other. The MPU
(chapter 14) is how you get a real guard.
```

Boot mode selection

Almost every MCU can boot from somewhere other than your flash, and this is how a factory programs a blank chip and how you recover a brick.

Boot source	Selected by	Used for
Main flash	Default; BOOT0 low	Normal operation
System memory (ROM bootloader)	BOOT0 high, or an option byte	Factory programming and recovery over UART, USB DFU or CAN
SRAM	Boot pins, or a debugger writing it directly	Loading code without wearing out flash during development
External flash (QSPI)	Option bytes plus a memory-mapped controller	Programs too large for internal flash

> **THE ROM BOOTLOADER IS YOUR RECOVERY PATH — PROTECT ACCESS TO IT**
>
> If your firmware disables the debug port and then fails to boot, the ROM bootloader entered via BOOT0 is often the only way back. Designs that route BOOT0 to a test pad rather than a hard ground are field-recoverable; designs that tie it low permanently and also disable SWD are one bad update away from scrap. This is a schematic decision made in week one that determines whether a firmware bug costs an hour or a production run.

Moving the vector table

Address `0x00000000` is an *alias*. On most parts flash also appears at its true address (often `0x08000000`), and the alias points wherever the boot mode selected. Once running, you can relocate the table entirely with `VTOR`:

```
/* ARMv7-M and above. ARMv6-M has no VTOR - the table is
   fixed at 0, and a bootloader on M0 must use other tricks
   (usually remapping SRAM to address 0).                  */

#define SCB_VTOR (*(volatile uint32_t *) 0xE000ED08)

static void jump_to_application(uint32_t app_base)
{
    uint32_t app_sp  = *(volatile uint32_t *) (app_base + 0);
    uint32_t app_pc  = *(volatile uint32_t *) (app_base + 4);

    __disable_irq();                  /* no interrupts mid-jump   */
    /* ... de-init every peripheral the bootloader enabled ...     */

    SCB_VTOR = app_base;              /* point at the app's table  */
    __asm volatile ("dsb");
    __asm volatile ("isb");

    __set_MSP(app_sp);                /* adopt the app's stack     */
    ((void (*)(void)) app_pc)();      /* never returns             */
}

/* This is a complete bootloader-to-application handoff, and
   nearly every field-update mechanism ends in these lines.

   The classic bug: forgetting to de-initialise. A timer the
   bootloader left running fires an interrupt as soon as the
   application enables interrupts, vectors through the
   application's table to a handler the application never
   installed, and lands in the default infinite loop. The
   symptom is "the app works when flashed directly but hangs
   when launched from the bootloader" - and the cause is
   always something left enabled.

   VTOR also has an alignment requirement: the table must be
   aligned to the next power of two above its size. For 100
   entries (400 bytes) that means 512-byte alignment. Get it
   wrong and the low bits are ignored, so the table lands
   somewhere you did not intend.                            */
```

What the debugger does differently

Attaching a debugger changes the reset sequence, which is why "it works under the debugger" is such a common and such a misleading observation.

```
/* Under a debugger, a typical "load and run" does:

   1. Halt the core (possibly via a reset-and-halt sequence
      that catches the core before Reset_Handler).
   2. Write the .text and .data sections into flash.
   3. Set the PC and SP directly - often SKIPPING the
      hardware's read of the first two words.
   4. Release the core.

   Consequences you will meet:

   - A corrupt first word in the vector table works fine
     under the debugger and bricks on a cold boot, because
     the debugger set SP itself.

   - Peripherals keep their state across a warm reset that
     the debugger issues, but not across a power cycle. Code
     that depends on a peripheral's reset value can work in
     one and fail in the other.

   - The debug unit keeps clocks running in sleep modes to
     stay attached, so current measurements are meaningless
     with SWD connected (chapter 13).

   - Some parts hold the watchdog frozen while halted. Your
     code passes single-stepping and resets in the field.

   RULE: before believing anything about startup, power-cycle
   the board with the debugger detached. Every startup bug
   worth its name hides in the difference.                  */
```

How it shows up

Symptom	Cause
Faults before any of your code runs	First vector-table word is not a valid, aligned RAM address
Fault on the very first push	`_estack` set to last address rather than last address + 1
Works under debugger, dead on power-up	Debugger set SP and PC directly, masking a broken vector table
Vector table missing from the binary	No `KEEP()` in the linker script and no `used` attribute — garbage collected
Hard fault on the first interrupt	Hand-written table without the Thumb bit set in entries
App hangs only when launched from bootloader	A peripheral or interrupt left enabled by the bootloader
Setting `VTOR` partially works	Table not aligned to the next power of two above its size
`VTOR` does not exist	ARMv6-M core; relocate by remapping SRAM to address 0
Chip appears bricked after a bad update	Debug disabled and BOOT0 not accessible — the recovery path was designed out
Behaviour differs between warm and cold reset	Peripheral state persists across warm reset; test both

Check yourself

- ✓ State exactly what the hardware does with the first two words at the boot address.
- ✓ Explain why `_estack` is one past the last RAM address.
- ✓ Say why the first vector table entry is data while the rest are function pointers.
- ✓ Explain why the vector table needs `KEEP()` or `used`.

- ✓ Describe how boot mode selection works and why BOOT0 access matters for recovery.
- ✓ List the four steps of a bootloader-to-application handoff.
- ✓ Explain the most common bootloader hand-off bug and its symptom.
- ✓ State `VTOR`'s alignment requirement for a 400-byte table.
- ✓ Give three ways a debugger changes the startup sequence.
- ✓ Explain why you must power-cycle with the debugger detached before trusting startup code.

The memory map and the linker script

On a hosted system the linker script is a detail you never see. Here it is a design document: it decides what lives in flash, what lives in RAM, where the stack ends, and what happens when you run out. This chapter reads one line by line.

Why you cannot avoid this file

The compilers track covers linking in general: sections, symbols, relocation. What changes on bare metal is that *nothing supplies a default that works*. There is no loader to place your segments at run time, so the addresses baked in at link time are the addresses used forever. The linker script is where you state them.

```
THE THREE JOBS OF A BARE-METAL LINKER SCRIPT

1. Declare the physical memories and their sizes.
      "256 kB of flash at 0x08000000,
        64 kB of SRAM  at 0x20000000"

2. Assign each input section to one of them.
      .text   -> flash
      .rodata -> flash
      .data   -> RAM, but its INITIAL VALUES in flash
      .bss    -> RAM, no initial values at all

3. Export symbols so startup code can find the boundaries.
      _sdata, _edata, _sbss, _ebss, _estack, _sidata

Job 3 is the one that surprises people: the linker script
and the startup code (chapter 06) are two halves of one
mechanism. Neither makes sense alone.
```

Load address versus virtual address

This is the concept the whole file turns on, and the one that makes `.data` comprehensible.

```
A variable with an initial value:

    int counter = 42;

It must be WRITABLE, so at run time it must live in RAM.
But RAM is volatile - at power-on it contains garbage.
So the value 42 has to be stored somewhere non-volatile.

FLASH (what is programmed)      RAM (at run time)
+---------------------+         +------------------+
| .text  (code)       |         |                  |
+---------------------+         |                  |
| .rodata (consts)    |         |                  |
+---------------------+         +------------------+
| .data INITIALISERS  | ------> | .data  counter=42|
|   the value 42      |  copy   +------------------+
+---------------------+  at     | .bss   (zeroed)  |
                         boot   +------------------+

VMA - Virtual Memory Address: where the code EXPECTS the
      variable to be. For .data, that is in RAM.
LMA - Load Memory Address: where the value is STORED in
      the image. For .data, that is in flash.

For .text and .rodata, VMA == LMA - they are used where
they sit. For .data they differ, and the gap between them
is exactly the copy loop in Reset_Handler.

.bss has no LMA at all. It occupies no space in the image
because its contents are known: zero. That is why a
100 kB array of zeros adds nothing to your binary, and a
100 kB array of ones adds 100 kB.
```

A complete linker script

```
/* GNU ld script for a Cortex-M4 with 256 kB flash, 64 kB SRAM.
   This is the whole file - they are shorter than their
   reputation suggests.                                        */

ENTRY(Reset_Handler)

MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 256K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
}

/* Stack and heap sizes, checked at link time below. */
_Min_Heap_Size  = 0x200;    /*  512 bytes */
_Min_Stack_Size = 0x800;    /* 2048 bytes */

SECTIONS
{
  /* Vector table FIRST, at the very base of flash.
     KEEP stops --gc-sections deleting it: nothing in C
     refers to it, so it looks like dead code.               */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector))
    . = ALIGN(4);
  } >FLASH

  /* Code and read-only data. */
  .text :
  {
    . = ALIGN(4);
    *(.text)
    *(.text*)
    *(.glue_7)  *(.glue_7t)
    *(.eh_frame)
    KEEP(*(.init))
    KEEP(*(.fini))
    . = ALIGN(4);
    _etext = .;
  } >FLASH

  .rodata :
  {
    . = ALIGN(4);
    *(.rodata)
    *(.rodata*)
    . = ALIGN(4);
  } >FLASH

  /* THE IMPORTANT ONE.
     AT> FLASH sets the LMA; >RAM sets the VMA.
     _sidata is where the initialisers live in flash;
     _sdata..._edata is where they must end up in RAM.       */
  _sidata = LOADADDR(.data);

  .data :
  {
    . = ALIGN(4);
    _sdata = .;
```

```
    *(.data)
    *(.data*)
    . = ALIGN(4);
    _edata = .;
  } >RAM AT> FLASH

  /* Zero-initialised data. No AT> clause: no load address,
     no space in the image.                                    */
  .bss :
  {
    . = ALIGN(4);
    _sbss = .;
    __bss_start__ = _sbss;
    *(.bss)
    *(.bss*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;
    __bss_end__ = _ebss;
  } >RAM

  /* Prove at LINK TIME that stack and heap still fit.
     Without this you discover the overflow at run time,
     as corruption. With it, the build fails.                  */
  ._user_heap_stack :
  {
    . = ALIGN(8);
    PROVIDE ( end = . );
    . = . + _Min_Heap_Size;
    . = . + _Min_Stack_Size;
    . = ALIGN(8);
  } >RAM

  _estack = ORIGIN(RAM) + LENGTH(RAM);   /* top of RAM */

  /DISCARD/ : { libc.a ( * ) libm.a ( * ) libgcc.a ( * ) }
  .ARM.attributes 0 : { *(.ARM.attributes) }
}
```

THE `._USER_HEAP_STACK` SECTION IS THE BEST LINE IN THE FILE

It allocates nothing and generates no code. Its only purpose is to make the linker fail if the remaining RAM cannot hold your declared minimum stack and heap. Without it, an over-large array links successfully and corrupts the stack at run time — a bug that presents as random misbehaviour weeks later. With it, you get `region RAM overflowed by 312 bytes` at build time. Trading a runtime mystery for a compile error is the whole art of embedded engineering, and this is the cheapest such trade available.

Putting specific things in specific places

```
/* Four placements you will actually need. */

/* 1. A buffer in a particular RAM bank - often required
      because only one bank is reachable by the DMA engine,
      or because a second bank is faster.                    */
__attribute__((section(".sram2")))
static uint8_t dma_buffer[1024];
/* with, in the script:
     RAM2 (rw) : ORIGIN = 0x2001C000, LENGTH = 16K
     .sram2 (NOLOAD) : { *(.sram2) } >RAM2                   */

/* 2. Data that must SURVIVE a reset - a crash log, a boot
      counter. Place it in a NOLOAD section so startup code
      never zeroes it.                                       */
__attribute__((section(".noinit")))
static uint32_t crash_count;
/*   .noinit (NOLOAD) : { *(.noinit) } >RAM
     NOLOAD means: reserve the address range, put nothing in
     the image. Combined with excluding it from the .bss
     zeroing loop, the value survives a warm reset.          */

/* 3. Code that must run FROM RAM - a flash-erase routine
      cannot execute from the flash it is erasing.           */
__attribute__((section(".ramfunc"), long_call, noinline))
void erase_sector(uint32_t addr) { /* ... */ }
/*   .ramfunc : { *(.ramfunc) } >RAM AT> FLASH
     Same LMA/VMA trick as .data - startup copies it.        */

/* 4. A fixed-address configuration block the bootloader and
      application both know about.                           */
__attribute__((section(".config"), used))
const struct config cfg = { .magic = 0xC0FFEE };
/*   .config 0x0801F000 : { KEEP(*(.config)) } >FLASH        */
```

Reading the output

```
$ arm-none-eabi-size -A firmware.elf
section          size      addr
.isr_vector       428   0x8000000
.text           28964   0x80001ac
.rodata          1832   0x8007270
.data             264  0x20000000     <- counts against BOTH
.bss             8192  0x20000108
._user_heap_stack 2560  0x20002108

/* FLASH used = .isr_vector + .text + .rodata + .data
              = 428 + 28964 + 1832 + 264 = 31488 of 262144

   RAM used   = .data + .bss + heap + stack
              = 264 + 8192 + 2560 = 11016 of 65536

   .data is counted TWICE - once in flash for the
   initialisers, once in RAM for the variables. This is the
   single most common arithmetic error when budgeting.      */

$ arm-none-eabi-nm --size-sort -S firmware.elf | tail -5
20000908 00000800 b uart_rx_ring
20001108 00001000 b sensor_history
/* --size-sort finds what is eating your RAM. 'b' is .bss,
   'd' is .data, 't' is .text. When the linker says RAM
   overflowed, this command names the culprit in seconds.   */

$ arm-none-eabi-objdump -h firmware.elf
/* Shows VMA and LMA side by side. If .data's LMA is not in
   flash, your AT> clause is wrong and the initialisers are
   not in the image at all - variables will hold garbage.   */
```

> **THE MAP FILE IS WORTH MORE THAN IT LOOKS**
>
> Pass `-Wl,-Map=firmware.map` and you get a file naming every symbol, its address, its size, and crucially *which object file pulled it in*. When a 4 kB printf implementation appears in a project that never calls printf, the map file's "referenced by" chain is what tells you it came in through `assert`. This is the single most under-used diagnostic in embedded development.

How it shows up

Symptom	Cause
`region RAM overflowed by N bytes`	Working as intended — use `nm --size-sort` to find the largest objects
Initialised globals contain garbage	`.data` has no `AT>` clause, or startup does not run the copy loop
Globals are zero when they should be 42	Section landed in `.bss`; check the map file
Vector table absent from the binary	Missing `KEEP()` — `--gc-sections` removed it
Binary far larger than expected	A large array initialised to non-zero values lives in flash as well as RAM
DMA transfers silently produce nothing	Buffer in a RAM bank the DMA engine cannot reach — place it explicitly
Flash erase routine hangs the core	Code executing from the flash being erased; needs `.ramfunc`
Crash log cleared by the reset it recorded	Variable in `.bss`, so startup zeroed it; needs `.noinit`
Linker succeeds, stack corrupts at run time	No `._user_heap_stack` check — add it

Symptom	Cause
Unexplained 4 kB of code you never called	Pulled in transitively; the map file's reference chain names it

Check yourself

- ✓ State the three jobs of a bare-metal linker script.
- ✓ Define VMA and LMA and say which sections have them differ.
- ✓ Explain why `.bss` occupies no space in the image but `.data` does.
- ✓ Explain why an array of 100 kB of zeros is free but 100 kB of ones is not.
- ✓ Say what the `AT>` clause does and what breaks without it.
- ✓ Explain what `._user_heap_stack` achieves given it emits no code.
- ✓ Give the command that finds which symbols are consuming RAM.
- ✓ Explain why `.data` counts against both flash and RAM budgets.
- ✓ Describe when you need `NOLOAD` and when you need `.ramfunc`.
- ✓ Say what the map file tells you that `size` does not.

Startup code: .data, .bss and the C runtime

C promises that globals with initialisers hold their values and globals without them are zero. On a hosted system the loader keeps that promise. Here, roughly forty lines of code you own keep it — and if they are wrong, C stops being C.

The C runtime contract

```
WHAT THE C STANDARD PROMISES        WHO DELIVERS IT HERE

int a = 42;                         Reset_Handler's
  -> a == 42 at main()              copy loop

int b;                              Reset_Handler's
  -> b == 0 at main()               zero loop

static int c = 7;                   same copy loop
  -> c == 7 at main()

C++ global constructors run         Reset_Handler calling
  before main()                     __libc_init_array

Before Reset_Handler completes, NONE of these hold. Any
code that runs earlier - and on some parts SystemInit()
does - is running in a language that merely resembles C.
Globals read there return whatever the SRAM powered up
with, which on many parts is genuinely random.
```

THE RULE THAT FOLLOWS

Do not touch initialised globals before the copy and zero loops have run. This bites hardest in `SystemInit()`, which many vendor templates call *before* the loops so that clocks are up early. If `SystemInit()` writes `SystemCoreClock` — a global — the subsequent `.data` copy overwrites it with the flash initialiser, and every delay in your program is wrong by whatever ratio the reset clock differs from the configured one. It is a genuinely common bug and it looks like a hardware fault.

Reset_Handler, complete

```
/* Portable C startup. Every symbol here comes from the
   linker script in chapter 05.                              */

extern uint32_t _sidata;   /* .data initialisers, in flash  */
extern uint32_t _sdata;    /* .data start, in RAM           */
extern uint32_t _edata;    /* .data end,   in RAM           */
extern uint32_t _sbss;     /* .bss  start, in RAM           */
extern uint32_t _ebss;     /* .bss  end,   in RAM           */

extern void SystemInit(void);
extern void __libc_init_array(void);
extern int  main(void);

__attribute__((naked, noreturn)) void Reset_Handler(void)
{
    /* The stack pointer is already loaded by hardware from
       vector[0], so we may call functions immediately.

       'naked' because we must not have a prologue that
       touches a stack we have not validated, and because
       this function never returns.                          */

    /* 1. Copy .data from flash to RAM. */
    uint32_t *src = &_sidata;
    uint32_t *dst = &_sdata;
    while (dst < &_edata) {
        *dst++ = *src++;
    }

    /* 2. Zero .bss. */
    dst = &_sbss;
    while (dst < &_ebss) {
        *dst++ = 0;
    }

    /* 3. NOW globals are valid. Clocks may safely use them. */
    SystemInit();

    /* 4. Run C++ constructors and anything in .init_array.
          Harmless in a pure C project - the array is empty. */
    __libc_init_array();

    /* 5. Go. */
    (void) main();

    /* main() returning is a bug on bare metal - there is
       nothing to return to. Trap rather than fall off the
       end into whatever bytes follow.                       */
    for (;;) { }
}
```

> **WHY THE LOOPS ARE WORD-WIDE**
>
> The linker script aligns every boundary to 4 bytes, so copying `uint32_t` at a time is safe and four times faster than bytes. If you ever change those `ALIGN(4)` directives, these loops become unaligned accesses — which on Cortex-M0 is a fault, and on M3 and above is merely slow. Keep the two files in agreement; they are one mechanism, as chapter 05 argued.

Default handlers and the weak-symbol trick

The vector table names a hundred handlers. You will implement three. The remaining ninety-seven still need addresses, and the standard mechanism is weak aliasing.

```
/* Every handler is declared weak and aliased to one default.
   If you define a real one anywhere in the project, the
   linker prefers yours; otherwise the alias stands.        */

void Default_Handler(void)
{
    /* Deliberately NOT an empty while(1). See below. */
    __asm volatile ("bkpt #0");   /* halt if a debugger is attached */
    for (;;) { }
}

#define ALIAS_DEFAULT __attribute__((weak, alias("Default_Handler")))

void NMI_Handler(void)          ALIAS_DEFAULT;
void HardFault_Handler(void)    ALIAS_DEFAULT;
void SVC_Handler(void)          ALIAS_DEFAULT;
void PendSV_Handler(void)       ALIAS_DEFAULT;
void SysTick_Handler(void)      ALIAS_DEFAULT;
void USART1_IRQHandler(void)    ALIAS_DEFAULT;
/* ... and so on for every peripheral ... */

/* Now, elsewhere in the project:

     void USART1_IRQHandler(void) { ... }

   silently and correctly replaces the default. No
   registration call, no table edit. This is why adding an
   interrupt handler on Cortex-M is just defining a function
   with the right name - and why a TYPO in that name is such
   a nasty bug: your function is never called, the default
   handler runs instead, and nothing warns you.

   Guard against it: after enabling an interrupt, assert that
   the vector table entry is not Default_Handler.           */

void assert_handler_installed(IRQn_Type irq)
{
    uint32_t *vt = (uint32_t *) SCB->VTOR;
    void (*installed)(void) = (void (*)(void)) vt[16 + irq];
    configASSERT(installed != Default_Handler);
}
```

> **AN EMPTY `WHILE(1)` IS THE WORST POSSIBLE DEFAULT HANDLER**
> The chip stops, the watchdog eventually resets it, and the reset cause says "watchdog" — telling you nothing. A `bkpt` halts into the debugger if one is attached and is a no-op if not, so you land exactly at the fault with the stack intact. Better still, record the active `IPSR` to a `.noinit` variable first, so a field unit's next boot can report which unhandled interrupt killed it. Chapter 15 builds that out.

The same thing in assembly

Vendor templates usually ship assembly. It is worth being able to read it, because when the C version misbehaves the assembly is the reference.

```
    .syntax unified
    .cpu    cortex-m4
    .thumb

    .global Reset_Handler
    .type   Reset_Handler, %function
Reset_Handler:
    ldr   sp, =_estack          @ redundant - hardware did it,
                                @ but harmless and explicit

    movs  r1, #0                @ offset into .data
    b     LoopCopyDataInit

CopyDataInit:
    ldr   r3, =_sidata          @ source base (flash)
    ldr   r3, [r3, r1]          @ load word at source+offset
    str   r3, [r0, r1]          @ store to dest+offset
    adds  r1, r1, #4

LoopCopyDataInit:
    ldr   r0, =_sdata
    ldr   r3, =_edata
    adds  r2, r0, r1
    cmp   r2, r3
    bcc   CopyDataInit          @ branch if carry clear (unsigned <)

    ldr   r2, =_sbss
    b     LoopFillZerobss

FillZerobss:
    movs  r3, #0
    str   r3, [r2], #4          @ store and post-increment

LoopFillZerobss:
    ldr   r3, =_ebss
    cmp   r2, r3
    bcc   FillZerobss

    bl    SystemInit
    bl    __libc_init_array
    bl    main
    b     .                     @ trap if main returns

/* Same five steps. The 'bcc' (branch if carry clear) is the
   unsigned less-than test - a detail worth knowing because
   using 'blt' (signed) here is a real bug on parts where
   RAM addresses have the top bit set.                      */
```

Retargeting the C library

`printf`, `malloc` and `assert` all assume an operating system. Newlib resolves that by calling out to a small set of stubs you must supply.

```
/* The minimum set to make newlib link and behave. */

#include <errno.h>
#include <sys/stat.h>

/* Where printf's bytes go. */
int _write(int fd, char *buf, int len)
{
    (void) fd;
    for (int i = 0; i < len; i++) {
        uart_putc(buf[i]);
    }
    return len;
}

/* The heap. sbrk grows it; this version refuses to collide
   with the stack, which the naive vendor version does not. */
extern uint32_t _end;      /* end of .bss, from linker      */
extern uint32_t _estack;

void *_sbrk(ptrdiff_t incr)
{
    static char *brk = NULL;
    if (brk == NULL) {
        brk = (char *) &_end;
    }

    char *sp = (char *) __get_MSP();
    if (brk + incr > sp - 512) {      /* keep 512 B for stack */
        errno = ENOMEM;
        return (void *) -1;
    }

    char *prev = brk;
    brk += incr;
    return prev;
}

int _close(int fd)                { (void)fd; return -1; }
int _fstat(int fd, struct stat *st){ (void)fd; st->st_mode = S_IFCHR; return 0; }
int _isatty(int fd)               { (void)fd; return 1; }
int _lseek(int fd, int p, int w)  { (void)fd;(void)p;(void)w; return 0; }
int _read(int fd, char *b, int l) { (void)fd;(void)b;(void)l; return 0; }

/* Two things worth knowing:

   1. printf with %f pulls in roughly 8-10 kB of float
      formatting. Link with -u _printf_float only if you
      need it, or use a tiny printf implementation. On a
      part with 32 kB of flash this single decision can be
      a third of your budget.

   2. Use -specs=nano.specs for newlib-nano: a much smaller
      libc with a reduced printf. Almost always the right
      default on an MCU.                                  */
```

How it shows up

Symptom	Cause
Initialised globals hold garbage	Copy loop missing, or `_sidata` wrong because the `AT>` clause is absent

Symptom	Cause
A global set in `SystemInit()` reverts	`SystemInit()` ran before the copy loop and was overwritten by it
Uninitialised globals are not zero	Zero loop missing, or `_sbss` / `_ebss` not exported
Every delay is wrong by a fixed ratio	`SystemCoreClock` clobbered by the `.data` copy
C++ objects behave as if never constructed	`__libc_init_array()` not called
An ISR you wrote is never entered	Handler name misspelled; the weak default is still linked
Chip resets periodically, cause reads "watchdog"	An unhandled interrupt spinning in an empty default handler
Unaligned access fault in startup	Section boundaries not 4-byte aligned but loops copy words
`malloc` succeeds then the stack corrupts	Vendor `_sbrk` with no stack collision check
Binary grows 10 kB after adding one `printf`	Float formatting pulled in; use newlib-nano or a tiny printf

Check yourself

- ✓ List the five steps of `Reset_Handler` in order.
- ✓ Explain what the C standard promises about globals and who delivers it here.
- ✓ Say why touching globals in `SystemInit()` can be a bug.
- ✓ Explain why `Reset_Handler` is declared `naked`.
- ✓ Describe how weak aliasing lets you install a handler by defining a function.
- ✓ Explain the failure mode of a misspelled handler name and how to detect it.
- ✓ Give two reasons an empty `while(1)` is a bad default handler.
- ✓ Explain why the copy loops move words rather than bytes, and what constrains that.
- ✓ Name three newlib stubs and what each is for.
- ✓ Say what `-specs=nano.specs` changes and why it matters on a small part.

The vector table and the NVIC

The Nested Vectored Interrupt Controller is part of the ARM architecture, at a fixed address, identical on every vendor's chip. It is also the reason Cortex-M interrupt latency is a number you can quote rather than a distribution you must measure.

From pin to handler

```
A signal arrives. Six things must all be true or nothing
happens - and "nothing happens" is the most common
interrupt bug there is.

1. PERIPHERAL         The peripheral must generate the
   enable             event at all: clock on, configured,
                      running.
     |
     v
2. PERIPHERAL         The peripheral's own interrupt
   interrupt enable   enable bit (e.g. USART_CR1.RXNEIE).
     |                Separate from the event itself.
     v
3. Peripheral sets    A status flag latches. This flag is
   its STATUS FLAG    what you must clear in the handler.
     |
     v
4. NVIC ENABLE        NVIC_ISER bit for that IRQ number.
     |                Set with NVIC_EnableIRQ().
     v
5. NVIC PRIORITY      Must be numerically lower (= more
   versus current     urgent) than whatever is running,
   execution          and not blocked by BASEPRI/PRIMASK.
     |
     v
6. PRIMASK clear      Global enable. cpsie i.
     |
     v
HANDLER RUNS - and must clear the flag from step 3, or it
will run again immediately, forever.

Debugging checklist: read the peripheral status register,
NVIC_ISER, NVIC_ISPR (pending), and PRIMASK. One of those
four answers "why is my interrupt not firing" every time.
```

Two numbering schemes

Exception numbers and IRQ numbers are different, offset by 16, and mixing them up is a rite of passage.

```
/* EXCEPTION number: the index into the vector table.
   IRQ number:       what the NVIC API takes.

   exception = irq + 16

   Exception  Vector offset  IRQ    Name
      1        0x04          -      Reset
      2        0x08          -14    NMI
      3        0x0C          -13    HardFault
      4        0x10          -12    MemManage
      5        0x14          -11    BusFault
      6        0x18          -10    UsageFault
     11        0x2C           -5    SVCall
     14        0x38           -2    PendSV
     15        0x3C           -1    SysTick
     16        0x40            0    first peripheral IRQ
     17        0x44            1    second peripheral IRQ

   System exceptions get NEGATIVE IRQ numbers, which is how
   the CMSIS API distinguishes them - NVIC_SetPriority takes
   an IRQn_Type that may be negative, and internally routes
   to SHPR (system handler priority) instead of IPR.

   So to find the vector table slot for USART1 with IRQ 37:
       offset = (37 + 16) * 4 = 0xD4                        */

/* Reading the currently active exception at runtime: */
uint32_t active = SCB->ICSR & SCB_ICSR_VECTACTIVE_Msk;
/* This is an EXCEPTION number, so subtract 16 for the IRQ. */
```

The NVIC registers

```
/* All at 0xE000E100. Note the SET/CLEAR pair design: you
   never read-modify-write, so there is no race with an
   interrupt that fires between the read and the write.     */

/*  ISER[]  0xE000E100  Interrupt Set Enable   - write 1 to enable
    ICER[]  0xE000E180  Interrupt Clear Enable - write 1 to disable
    ISPR[]  0xE000E200  Interrupt Set Pending  - write 1 to force
    ICPR[]  0xE000E280  Interrupt Clear Pending- write 1 to drop
    IABR[]  0xE000E300  Interrupt Active (read only)
    IPR[]   0xE000E400  Interrupt Priority, one BYTE per IRQ    */

/* Writing 0 to any of these does nothing. That is the point:
   to disable IRQ 37 you write bit 37 of ICER, you do NOT
   read ISER, clear a bit and write it back. The latter would
   lose any interrupt enabled by a higher-priority handler
   that preempted you mid-sequence.                          */

static inline void my_enable_irq(int irq)
{
    NVIC->ISER[irq >> 5] = 1UL << (irq & 0x1F);
}
static inline void my_disable_irq(int irq)
{
    NVIC->ICER[irq >> 5] = 1UL << (irq & 0x1F);
    __asm volatile ("dsb");   /* ensure it takes effect...   */
    __asm volatile ("isb");   /* ...before the next instr    */
}

/* The barriers after DISABLE are the subtle part. Without
   them an interrupt already on its way can still be taken
   after your ICER write returns - the write is posted, the
   NVIC has not yet acted, and the pipeline continues. Code
   that disables an interrupt and then frees the buffer that
   interrupt uses will, rarely, use-after-free. This is one
   of the genuinely hard-to-reproduce bare-metal bugs.       */
```

SET/CLEAR REGISTER PAIRS ARE A HARDWARE IDIOM WORTH RECOGNISING

You will see the same design in GPIO (`BSRR`), in DMA, and in the GIC on Cortex-A. The reason is always the same: a single write that affects exactly the bits you name is atomic with respect to anything else touching that register, so no lock is needed. When you meet a peripheral with only a read-write control register and no set/clear pair, that is a signal you will need a critical section around it.

Pending, active, and the difference

```
PENDING  the NVIC has latched a request but the handler
         has not started. Bit in ISPR/ICPR.

ACTIVE   the handler has started and has not returned.
         Bit in IABR. Stays set through preemption.

Both can be set at once: a handler is running (active)
and the same interrupt has been requested again (pending).
When the handler returns, it is immediately re-entered.

time -->
         req         req2
          |           |
pending  _|--|________|-------|______
active   ____|--------|-------|______
             handler   handler
               #1        #2

This is why clearing the PERIPHERAL flag is not the same
as clearing the NVIC pending bit, and why the ORDER
matters:

  WRONG                       RIGHT
  NVIC_ClearPendingIRQ(n);    USARTx->ICR = flag;  /* source */
  USARTx->ICR = flag;         /* NVIC pending clears itself  */

Clear the source first. If you clear NVIC pending while
the peripheral flag is still asserted, the peripheral
immediately re-asserts it and you have achieved nothing.
```

The write-buffer race at handler exit

```
/* The most notorious Cortex-M interrupt bug. */

void USART1_IRQHandler(void)
{
    if (USART1->ISR & USART_ISR_RXNE) {
        ring_put(USART1->RDR);      /* reading RDR clears RXNE */
    }
    /* Handler returns here. */
}

/* On a part where the peripheral sits behind a write buffer
   or a slow bus bridge, the flag-clearing write may still be
   in flight when the exception return completes. The NVIC
   samples the (still asserted) line and re-enters the
   handler. The second entry finds nothing to do - a
   "spurious" interrupt. Usually harmless; occasionally it
   corrupts a state machine that counts entries.

   The fix is a barrier that forces the write to complete
   before the return:                                       */

void USART1_IRQHandler_fixed(void)
{
    if (USART1->ISR & USART_ISR_RXNE) {
        ring_put(USART1->RDR);
    }
    __asm volatile ("dsb");   /* drain the write buffer */
}

/* ARM's own guidance is that this is only required on
   specific parts, and vendors document it inconsistently.
   Pragmatically: if you see doubled interrupts under load,
   add the DSB before spending a day on anything else.
   The cost is a few cycles.                                */
```

Where the twelve cycles go

```
INTERRUPT LATENCY, Cortex-M3/M4, zero wait states:

cycle  0   request asserted
1–8        hardware stacks 8 registers (R0–R3, R12,
           LR, PC, xPSR) onto the current stack
(overlapped) vector fetch from the table
9–11       pipeline refill at the handler address
12         first instruction of your handler executes

TAIL-CHAINING: if a second interrupt is pending when the
first returns, the hardware does NOT unstack and restack.
It jumps straight to the next handler in 6 cycles.

LATE ARRIVAL: if a higher-priority interrupt arrives
DURING the stacking of a lower one, the hardware
redirects to the higher one without restarting the push.

+-------------+---------+-------------+
| naive model | stack 8 | unstack 8   |  ~24 cycles
+-------------+---------+-------------+   between handlers
| actual      | stack 8 | tail-chain  |  ~6 cycles
+-------------+---------+-------------+

Flash wait states change all of these numbers. At 168 MHz
with 5 wait states the vector fetch alone costs more than
the whole sequence above. Enable the flash prefetch and
ART accelerator, or put the vector table in SRAM if
latency truly matters.
```

How it shows up

Symptom	Cause
Interrupt never fires	One of the six gates — check peripheral enable, NVIC ISER, PRIMASK, in that order
Handler runs once then loops forever	Peripheral status flag never cleared in the handler
Handler runs twice per event	Write-buffer race at exit — add a `DSB`
Clearing NVIC pending has no effect	Peripheral flag still asserted; clear the source first
Wrong handler entered	Exception number used where an IRQ number was wanted, or vice versa (off by 16)
Disabling an interrupt does not take effect at once	Missing `DSB` / `ISB` after the `ICER` write
Rare use-after-free in an ISR-owned buffer	Same missing barrier — an in-flight interrupt arrived after disable
An interrupt is lost under load	Pending is one bit deep; a third request while one is active and one pending is dropped
Latency far worse than 12 cycles	Flash wait states; enable prefetch or move the table to SRAM
Enabling one interrupt disables another	Read-modify-write on `ISER` instead of a single-bit write

Check yourself

- ✓ List the six conditions that must hold for an interrupt to reach its handler.
- ✓ Convert IRQ number 37 to a vector table byte offset.

- ✓ Explain why system exceptions have negative IRQ numbers in CMSIS.
- ✓ Explain why NVIC uses set/clear register pairs rather than read-modify-write.
- ✓ Distinguish pending from active, and describe a state where both are set.
- ✓ Say which to clear first: the peripheral flag or the NVIC pending bit, and why.
- ✓ Describe the write-buffer race at handler exit and its fix.
- ✓ Explain why barriers are needed after disabling an interrupt.
- ✓ Explain tail-chaining and give its cycle count against the naive model.
- ✓ Name two things that make real latency worse than the architectural 12 cycles.

Priorities, preemption and tail-chaining

Interrupt priority on Cortex-M is where three separate confusions meet: lower numbers are more urgent, only the top bits of the priority byte are implemented, and the value splits into two fields whose boundary you configure. Getting all three right is what makes an RTOS safe.

Lower is more urgent

```
/* Priority 0 is the HIGHEST priority. This is inverted
   relative to almost every other scheduler you have met,
   including the RTOS running on top of it - FreeRTOS task
   priorities run the other way, where higher is more urgent.

   In a FreeRTOS project you therefore hold BOTH conventions
   in your head at once:

     NVIC priority 0   = most urgent interrupt
     FreeRTOS priority 0 = least urgent task (the idle task)

   Mixing them up is common enough that FreeRTOS ships an
   assertion specifically to catch it.                      */

NVIC_SetPriority(TIM2_IRQn,   0);  /* most urgent  */
NVIC_SetPriority(USART1_IRQn, 5);
NVIC_SetPriority(EXTI0_IRQn, 15);  /* least urgent */

/* Reset default is 0 for everything, meaning every interrupt
   starts at maximum urgency and none can preempt another.
   Leaving the defaults is a real design decision that you
   should make deliberately rather than by omission.        */
```

Only the top bits exist

```
The priority register is a byte per interrupt, but silicon
vendors implement only the top 3, 4 or 5 bits. The rest
read as zero and ignore writes.

A part with 4 implemented bits (STM32, most Cortex-M4):

 bit  7  6  5  4  3  2  1  0
     +--+--+--+--+--+--+--+--+
     |  IMPLEMENTED |  ZERO  |
     +--+--+--+--+--+--+--+--+
      <- 16 levels ->

So the usable values are 0x00, 0x10, 0x20 ... 0xF0.

CMSIS hides this. NVIC_SetPriority(irq, 5) shifts for you:

    IPR[irq] = (5 << (8 - __NVIC_PRIO_BITS)) & 0xFF
             = (5 << 4) = 0x50

But when you write BASEPRI YOURSELF, nothing shifts for
you, and this is the single most common bug in
hand-written critical sections:

  __set_BASEPRI(5);        /* WRONG - 5 = 0x05, and with  */
                           /* 4 bits implemented that is  */
                           /* priority 0. Blocks nothing  */
                           /* except... nothing.          */

  __set_BASEPRI(5 << 4);   /* RIGHT - blocks priority 5   */
                           /* and numerically higher.     */

Always write it as (n << (8 - __NVIC_PRIO_BITS)) so the
code survives a move to a part with different bits.
```

CHECK `__NVIC_PRIO_BITS` FOR YOUR PART BEFORE WRITING ANY PRIORITY CODE

It is defined in the vendor's device header. Cortex-M0 has 2 bits (4 levels), most M3/M4 parts have 4 (16 levels), some have 3 or 5. Code that assumes 4 and runs on a part with 3 silently collapses two of your priority levels into one, which turns a carefully designed preemption scheme into an unpredictable one — and the symptom is intermittent priority inversion under load, not a clean failure.

Group and sub-priority

The implemented bits split into two fields. Only the group field decides preemption; the sub-priority field breaks ties among simultaneously pending interrupts.

```
With 4 implemented bits and PRIGROUP set for 2/2:

 bit   7  6  5  4
     +----+----+
     |GRP |SUB |
     +----+----+
       ^     ^
       |     +-- ties only: which of two SIMULTANEOUSLY
       |         pending interrupts runs first. Does NOT
       |         allow preemption.
       |
       +-- preemption: an interrupt with a lower GROUP
           value preempts one that is running.

Example, groups 0-3 and subs 0-3:

  A: group 1, sub 3   ->  priority 0x70
  B: group 1, sub 0   ->  priority 0x40
  C: group 0, sub 3   ->  priority 0x30

B does NOT preempt A - same group. It waits.
C DOES preempt both  - lower group.
If A and B arrive together, B runs first (lower sub).

SCB->AIRCR PRIGROUP field sets the split:

  PRIGROUP  group bits  sub bits   (4-bit part)
     3          4          0        <- all preemption
     4          3          1
     5          2          2
     6          1          3
     7          0          4        <- no preemption at all

NVIC_SetPriorityGrouping(3) - all bits are group, no
sub-priority - is the sane default and what most RTOS
ports require. FreeRTOS's port assumes it and asserts if
you have configured otherwise.
```

Nesting, and what it costs

```
/* Preemption is automatic and needs no code. It also costs
   stack, and that is the part people forget.               */

/* Each nesting level consumes:
     32 bytes  basic frame (8 registers)
   + 72 bytes  if the FPU context is stacked too
   + whatever the handler's own locals need

   With 4 priority groups and FP in use, worst case is
   4 * (104 + locals) of MSP. If your MSP is 1 kB and each
   handler uses 100 bytes of locals, you are at 816 bytes
   before counting the code that was interrupted.

   This is why chapter 20 sizes the main stack from the
   interrupt nesting depth, not from a guess.               */

/* To FORBID nesting entirely, give every interrupt the same
   group priority. Simple, predictable, and adequate for
   most designs:                                            */

NVIC_SetPriorityGrouping(3);           /* 4 group bits      */
for (int irq = 0; irq < 82; irq++) {
    NVIC_SetPriority((IRQn_Type) irq, 5);   /* all equal    */
}
/* Now nothing preempts anything. Latency for the second
   interrupt becomes the duration of the first, which you
   must then bound - the classic trade.                     */
```

THE RTOS PRIORITY BOUNDARY

An RTOS kernel protects its data with `BASEPRI`, which means interrupts above that threshold run *through* kernel critical sections and therefore must never call a kernel API. This splits your interrupts into two populations, and the split is a design decision you make once:

```
    priority 0  +---------------------------+
                |  ZERO-LATENCY interrupts  |  Never blocked.
                |  Motor commutation, fault |  MUST NOT call
                |  detection, safety stop.  |  any RTOS API.
                |  Not blocked by BASEPRI.  |
    priority 4  +---------------------------+
                |  KERNEL-AWARE interrupts  |  Blocked during
                |  UART, SPI, timers, most  |  kernel critical
                |  peripheral handlers.     |  sections. MAY
                |  MAY call *FromISR APIs.  |  call FromISR.
    priority 15 +---------------------------+

    In FreeRTOS the boundary is
    configMAX_SYSCALL_INTERRUPT_PRIORITY. Calling an API from
    above the line corrupts kernel state in a way that
    manifests hours later; configASSERT catches it
    immediately, which is why you leave it enabled in
    development.
```

Tail-chaining and late arrival

```
NAIVE MODEL (what you would implement in software)

IRQ A ----+                    +---- IRQ B
          v                    v
[stack 8][handler A][unstack 8][stack 8][handler B][unstack 8]
 12 cyc              12 cyc     12 cyc              12 cyc

ACTUAL HARDWARE - TAIL-CHAINING

[stack 8][handler A][ 6 cyc ][handler B][unstack 8]
                     tail-chain: the frame on the stack
                     is already correct for B, so the
                     hardware skips unstack+restack.

LATE ARRIVAL

IRQ A arrives, hardware begins stacking...
     ...IRQ B (higher priority) arrives at cycle 4...
The hardware does NOT abandon or restart the push. It
completes the stacking and then vectors to B instead of
A. A remains pending and tail-chains after B.

Net effect: the expensive part (pushing 8 registers)
happens once per BURST of interrupts rather than once
per interrupt. Under heavy load a Cortex-M spends far
less time on entry/exit than the per-interrupt figure
suggests - which is precisely why the profile is used
for interrupt-dense control applications.
```

How it shows up

Symptom	Cause
Critical section protects nothing	`BASEPRI` written without shifting into the implemented bits
Two priority levels behave as one	Part implements fewer bits than assumed; check `__NVIC_PRIO_BITS`
A higher-priority interrupt does not preempt	Same group priority — only sub-priority differs, which never preempts
Nothing preempts anything	All priorities left at the reset default of 0
Stack overflow only under interrupt load	Nesting depth times frame size exceeds MSP; bound the depth
RTOS state corrupts hours into a run	An API called from an interrupt above the syscall threshold
`configASSERT` fires on an ISR call	Exactly the above, caught early — fix the priority or use the `FromISR` variant
Behaviour changes when priorities are renumbered	Priority grouping is not what you assumed; set it explicitly at startup
Latency fine in isolation, poor in the system	A lower-priority handler is long and cannot be preempted by an equal-group peer
FreeRTOS asserts at startup about grouping	Port requires all bits as group priority — call `NVIC_SetPriorityGrouping(3)`

Check yourself

- ✓ State which numeric direction is more urgent for NVIC, and for FreeRTOS tasks.
- ✓ Explain why only the top bits of the priority byte are implemented.

- ✓ Write a correct `BASEPRI` value for priority 5 on a part with 4 priority bits.
- ✓ Distinguish group priority from sub-priority in terms of preemption.
- ✓ Say what `NVIC_SetPriorityGrouping(3)` configures and why RTOS ports want it.
- ✓ Calculate worst-case MSP usage for 4 nesting levels with FP context stacked.
- ✓ Explain the two interrupt populations an RTOS creates and the rule for each.
- ✓ Describe tail-chaining and give its cycle cost against the naive model.
- ✓ Explain late arrival and what happens to the interrupt that was displaced.
- ✓ Give a way to forbid interrupt nesting entirely, and the cost of doing so.

The clock tree: oscillators, PLLs and dividers

Almost every "the peripheral does not respond" bug on a microcontroller is a clock bug. This chapter covers where clocks come from, how they are multiplied and divided to reach every block, and the two ordering rules that must never be violated.

Where clocks come from

```
FOUR SOURCES, in rough order of accuracy and cost:

INTERNAL RC  (HSI, MSI)      1–3% accurate, sometimes 0.1%
  +--[~~~]--+               after factory trim.
                            Starts in microseconds.
                            Drifts with temperature.
                            Free – no external parts.
                            THE CHIP BOOTS ON THIS.

CRYSTAL      (HSE)           10–50 ppm. Needs two load
  +--[X]--+                  capacitors chosen to match
                             the crystal's spec.
                             Starts in 1–10 ms.
                             Required for USB, Ethernet,
                             CAN and anything with a
                             protocol-defined bit rate.

LOW-SPEED    (LSE)           32.768 kHz, 20 ppm.
  +--[x]--+                  2^15 Hz, so a 15-bit counter
                             overflows at exactly 1 Hz.
                             Runs in the lowest power
                             modes; drives the RTC.

EXTERNAL     (bypass mode)   A clock signal from another
  --->                       chip. Used when one oscillator
                             serves a whole board.

Accuracy is a REQUIREMENT, not a preference. UART at 3%
error fails at 115200 baud (chapter bus-02 does the
arithmetic). USB requires 0.25%. Sleep-mode wake timing
needs the LSE or you drift seconds per day.
```

THE INTERNAL OSCILLATOR IS NOT GOOD ENOUGH FOR A UART, USUALLY

A 1% clock error plus a 1% error at the other end plus baud-rate rounding can exceed the ~2% that asynchronous serial framing tolerates. Many parts can trim the HSI against the LSE at runtime to reach 0.1%, which is enough. Otherwise you fit a crystal. "It works on my desk and fails in the cold chamber" is this problem: the RC oscillator's temperature coefficient moved you outside the window.

The PLL and the tree

```
A realistic STM32F4 clock tree, simplified:

HSI 16 MHz --+
             |    +----------------------------+
HSE  8 MHz --+-->-| /M  ->  xN  ->  /P         |--> PLLCLK
                  |  VCO input   VCO output    |     168 MHz
                  +----------------------------+
                     |
                     v
               +-----------+
HSI ---------> |  SYSCLK   |  <- mux: HSI, HSE or PLL
HSE ---------> |   MUX     |
PLL ---------> +-----------+
                     |
                     v  /AHBPRE (1,2,4...512)
               +-----------+
               |  HCLK     |  168 MHz -> core, DMA, memory
               +-----------+
                  |     |
       /APB1PRE   |     |   /APB2PRE
                  v     v
           +--------+ +--------+
           | PCLK1  | | PCLK2  |
           | 42 MHz | | 84 MHz |
           +--------+ +--------+
            I2C,UART2  UART1,SPI1
            TIM2-7     TIM1,TIM8

THE TIMER EXCEPTION, and it catches everyone:
if an APB prescaler is not 1, the TIMER clock on that bus
is PCLK x2, not PCLK. So with PCLK1 = 42 MHz, TIM2 counts
at 84 MHz. Getting this wrong makes every period exactly
half or double what you intended.

CONSTRAINTS the reference manual imposes on the PLL:
  VCO input  (HSE/M)     must be 1-2 MHz
  VCO output (that x N)  must be 100-432 MHz
  PLLCLK     (that /P)   must be <= 168 MHz
Pick M, N, P to satisfy all three simultaneously. Vendor
clock-configuration tools exist precisely because this is
a constraint-satisfaction problem, not a calculation.
```

Two ordering rules

```
/* RULE 1: raise flash wait states BEFORE raising the clock,
   and lower them AFTER lowering it.

   Flash cannot be read at 168 MHz with zero wait states. If
   you switch SYSCLK to the PLL while the flash controller
   still expects 16 MHz timing, the very next instruction
   fetch returns garbage and you fault - before any code of
   yours can report it.                                      */

void clock_init_168mhz(void)
{
    /* 1. Turn on HSE and WAIT for it. */
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY)) { }

    /* 2. Flash: 5 wait states + caches + prefetch, FIRST. */
    FLASH->ACR = FLASH_ACR_LATENCY_5WS
               | FLASH_ACR_PRFTEN
               | FLASH_ACR_ICEN
               | FLASH_ACR_DCEN;
    while ((FLASH->ACR & FLASH_ACR_LATENCY) != FLASH_ACR_LATENCY_5WS) { }

    /* 3. Bus prescalers BEFORE the core speeds up, so no bus
          ever momentarily exceeds its maximum.              */
    RCC->CFGR |= RCC_CFGR_HPRE_DIV1     /* AHB  = 168 MHz */
              |  RCC_CFGR_PPRE1_DIV4    /* APB1 =  42 MHz */
              |  RCC_CFGR_PPRE2_DIV2;   /* APB2 =  84 MHz */

    /* 4. Configure and start the PLL: 8/8 = 1, x336, /2. */
    RCC->PLLCFGR = (8u)            /* M */
                 | (336u << 6)     /* N */
                 | (0u   << 16)    /* P = 2, encoded as 0 */
                 | RCC_PLLCFGR_PLLSRC_HSE
                 | (7u << 24);     /* Q = 7, for USB 48 MHz */
    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) { }

    /* 5. Only now switch SYSCLK, and confirm it took. */
    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) { }

    SystemCoreClock = 168000000u;
}

/* RULE 2: every WAIT loop above is mandatory. An oscillator
   takes milliseconds to stabilise; a PLL takes hundreds of
   microseconds to lock. Switching to an unlocked PLL gives
   you an unstable clock, which gives you behaviour that
   changes with temperature and supply voltage and looks
   exactly like a hardware fault.                            */
```

> **EVERY WAIT LOOP NEEDS A TIMEOUT IN PRODUCTION CODE**
>
> The loops above hang forever if the crystal never starts — a cracked crystal, a bad solder joint, or the wrong load capacitors. A field unit that hangs in `clock_init` is indistinguishable from a dead board. Real firmware counts iterations, and on timeout falls back to the internal oscillator and sets a fault flag, so the device boots degraded and can *report* the failure rather than appearing dead.

Peripheral clock gating

```
/* Every peripheral has an individual clock enable, and it
   is OFF at reset. A peripheral with no clock does not
   fault when you write its registers - it silently ignores
   them, and reads return zero.

   This is the number one cause of "my GPIO does nothing".  */

RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

/* THE DELAY THAT IS NOT SUPERSTITION:
   the enable write is posted on the bus. On several ST
   families the peripheral needs a couple of clock cycles
   before its registers respond. Writing GPIOA->MODER on the
   very next instruction can be lost.

   The idiomatic fix is a dummy read-back, which forces the
   write to complete before proceeding:                    */

volatile uint32_t tmp = RCC->AHB1ENR;   /* read back */
(void) tmp;

/* ST's own HAL does exactly this, in the __HAL_RCC_*_CLK_ENABLE
   macros. If you write registers directly and skip it, you
   get an initialisation that works at -O0 and fails at -O2,
   because the optimiser tightened the instruction spacing.

   Gating is also your main static power lever: an unused
   peripheral with its clock off draws essentially nothing.
   Chapter 13 uses this heavily.                            */
```

Verifying the clock is what you think

```
/* NEVER trust the arithmetic. Measure. Two ways: */

/* 1. MCO - route a clock to a pin and put a scope on it.
      Almost every part can output SYSCLK/2, /4 etc.        */
RCC->CFGR |= RCC_CFGR_MCO1_1        /* source: PLL          */
          |  RCC_CFGR_MCO1PRE_DIV4; /* divide so it fits    */
/* configure PA8 as alternate function 0 - now PA8 carries
   SYSCLK/4 and one scope reading confirms the whole tree.  */

/* 2. Blink at a known rate and time it with a stopwatch.
      Crude, needs no equipment, and catches factor-of-two
      errors instantly - which are the errors you actually
      make (the APB timer doubling, or a /2 you forgot).    */
for (;;) {
    GPIOA->ODR ^= (1u << 5);
    for (volatile int i = 0; i < 168000000 / 20; i++) { }
}
/* Should toggle at roughly 10 Hz if SYSCLK is 168 MHz and
   the loop is ~2 cycles. If you count 5 Hz, you are at
   84 MHz and your PLL config is off by one divider.        */

/* SystemCoreClock is a VARIABLE, not a measurement. It is
   whatever the last line of your clock init assigned. If
   the PLL silently failed to lock and you fell back to HSI,
   SystemCoreClock still reads 168000000 and every delay in
   the system is wrong by 10x while the variable insists
   everything is fine.                                       */
```

How it shows up

Symptom	Cause
Peripheral registers read zero, writes ignored	Peripheral clock not enabled in the `RCC` gate register
Init works at `-O0`, fails at `-O2`	No read-back after the clock enable; the next write outran the bus
Hard fault immediately on switching to the PLL	Flash wait states not raised first
Every timer period is half or double	APB prescaler timer-doubling rule
Board hangs at boot, appears dead	A wait loop on an oscillator that never started, with no timeout
UART garbles at high baud, fine at low	Clock accuracy — internal RC where a crystal is needed
Works at room temperature, fails in the cold	RC oscillator drift moved baud error outside tolerance
Delays wrong by exactly 10x	PLL never locked, running on HSI, while `SystemCoreClock` still claims otherwise
USB enumerates intermittently or not at all	48 MHz USB clock not exact; check the PLL Q divider
RTC drifts seconds per day	Running from the internal oscillator rather than the 32.768 kHz LSE

Check yourself

- ✓ Name four clock sources and give one application that requires each.
- ✓ Explain why the chip boots on the internal RC oscillator.
- ✓ State the rule about flash wait states and clock changes, in both directions.
- ✓ Explain the APB timer-doubling rule and the bug it causes.

- ✓ List the three PLL constraints that must hold simultaneously.
- ✓ Say why every oscillator wait loop needs a timeout in shipped firmware.
- ✓ Explain what happens when you write a peripheral register with its clock gated off.
- ✓ Explain why a read-back after a clock-enable write is not superstition.
- ✓ Give two ways to verify the actual clock frequency on hardware.
- ✓ Explain why `SystemCoreClock` can be confidently wrong.

GPIO and pinmux from the register level

A pin is the only part of a microcontroller that touches the physical world, and it is massively multiplexed: one ball of solder, a dozen possible functions, and a set of electrical properties you must configure or regret. This is the simplest peripheral and the one worth understanding exactly.

What is behind a pin

```
                        VDD
                         |
                      [pull-up]  <- PUPDR
                         |
   alt.fn out --+        |        +---> IDR (input data)
   ODR ---------+-->[ P-MOS ]     |
                |    output    +--+---> alt.fn in
                |    driver    |  |
   MODER -------+-->[ N-MOS ]--+--+---> ADC (analog)
                         |         [Schmitt trigger]
                      [pull-down] <- PUPDR
                         |
                        GND
                         |
                      === PIN ===

FIVE independent decisions per pin:

MODER    input / output / alternate function / analog
OTYPER   push-pull (drives both rails) or open-drain
         (drives low only; needs a pull-up for high)
OSPEEDR  slew rate - how fast the edge is
PUPDR    internal pull-up, pull-down, or neither
AFR      WHICH alternate function, when MODER says AF

Getting MODER right and the other four wrong is the
normal failure mode: the pin half-works.
```

Configuring a pin properly

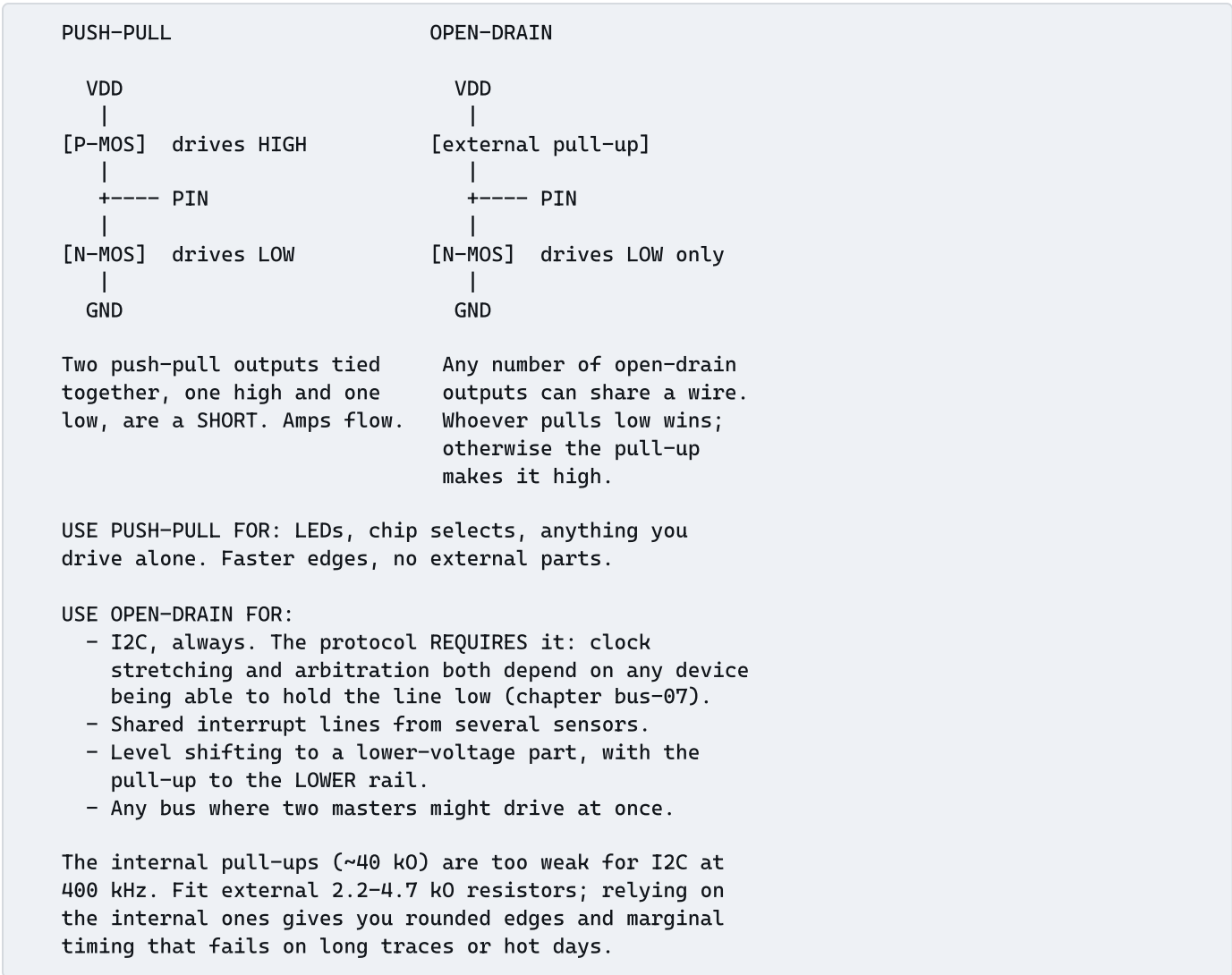
```
/* PA5 as a push-pull output - the LED case. */

RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
(void) RCC->AHB1ENR;                  /* read back (ch 09)  */

/* Two bits per pin: clear then set. Never just OR - the
   reset value for some pins is not 00, notably the SWD
   pins PA13/PA14 which come up as alternate function.     */
GPIOA->MODER   &= ~(3u << (5 * 2));
GPIOA->MODER   |=  (1u << (5 * 2));   /* 01 = output       */

GPIOA->OTYPER  &= ~(1u << 5);         /* 0 = push-pull     */

GPIOA->OSPEEDR &= ~(3u << (5 * 2));   /* 00 = low speed    */
                                       /* for an LED this is */
                                       /* the RIGHT choice   */
GPIOA->PUPDR   &= ~(3u << (5 * 2));   /* 00 = no pull      */

/* PA9 as USART1_TX - an alternate function. */
GPIOA->MODER   &= ~(3u << (9 * 2));
GPIOA->MODER   |=  (2u << (9 * 2));   /* 10 = alt function */
GPIOA->OSPEEDR |=  (3u << (9 * 2));   /* fast edges for a  */
                                       /* real signal        */
/* AFR[0] covers pins 0-7, AFR[1] covers 8-15, 4 bits each. */
GPIOA->AFR[1]  &= ~(0xFu << ((9 - 8) * 4));
GPIOA->AFR[1]  |=  (7u   << ((9 - 8) * 4));  /* AF7=USART1 */

/* WHICH AF number? Not guessable. It is a table in the
   DATASHEET (not the reference manual) called "Alternate
   function mapping". Every pin has a different mapping, and
   a given peripheral appears on only a few pins. This table
   is what pin-planning tools automate.                    */
```

> **SLEW RATE IS AN EMC DECISION, NOT A PERFORMANCE ONE**
>
> A fast edge radiates. Setting every pin to maximum speed "to be safe" is how boards fail radiated-emissions testing, and the fix at that stage costs a respin or a pile of series resistors. Use the slowest setting that meets the timing requirement: low speed for LEDs and slow logic, high speed only for a clock or a bus running near its limit. Chapter bus-19 covers the physics.

BSRR: setting pins without a race

```
/* The obvious way is wrong in the presence of interrupts: */

GPIOA->ODR |=  (1u << 5);      /* read, modify, write      */

/* If an interrupt fires between the read and the write and
   changes another pin in the same port, that change is lost
   when this write completes. A classic lost-update race,
   and it produces LEDs that occasionally stay on and relays
   that occasionally do not fire.                           */

/* BSRR is a WRITE-ONLY register with set bits in the low
   half and reset bits in the high half. One store, no read,
   no race - and it affects only the bits you name.         */

GPIOA->BSRR = (1u << 5);           /* set PA5              */
GPIOA->BSRR = (1u << (5 + 16));    /* clear PA5            */
GPIOA->BSRR = (1u << 5) | (1u << (6 + 16));  /* both, in
                                      the same instant     */

/* If both the set and reset bit for one pin are written,
   SET WINS. Documented, and occasionally useful.

   This is the same set/clear idiom as the NVIC's ISER/ICER
   (chapter 07). Recognising it is a general skill: when a
   peripheral offers a write-1-to-act register, use it and
   delete your critical section.                            */

/* Reading is unambiguous - IDR always reflects the PIN,
   even when the pin is configured as an output. That is
   how you detect a short: drive high, read low.            */
if (!(GPIOA->IDR & (1u << 5))) {
    fault_log(FAULT_LED_SHORTED);
}
```

Push-pull versus open-drain



Pins as interrupt sources

```
/* EXTI: external interrupt lines. The mux is per PIN NUMBER
   across ports, and that constraint bites.                */

/* Route PC13 to EXTI line 13. */
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
SYSCFG->EXTICR[13 / 4] &= ~(0xFu << ((13 % 4) * 4));
SYSCFG->EXTICR[13 / 4] |=  (2u   << ((13 % 4) * 4));  /* port C */

EXTI->IMR  |=  (1u << 13);      /* unmask                   */
EXTI->FTSR |=  (1u << 13);      /* falling edge             */
EXTI->RTSR &= ~(1u << 13);      /* not rising               */

NVIC_SetPriority(EXTI15_10_IRQn, 10);
NVIC_EnableIRQ(EXTI15_10_IRQn);

/* THE CONSTRAINT: EXTI line N serves pin N of exactly ONE
   port. You can use PA0 or PB0 or PC0 as an interrupt - but
   not two of them at once. Discovering this after the PCB
   is made is a genuinely common and expensive mistake, and
   it is why pin assignment happens before layout.

   Note also that lines 5-9 and 10-15 SHARE a single NVIC
   vector, so the handler must demultiplex:                */

void EXTI15_10_IRQHandler(void)
{
    if (EXTI->PR & (1u << 13)) {
        EXTI->PR = (1u << 13);   /* write 1 to CLEAR */
        button_pressed();
    }
    if (EXTI->PR & (1u << 14)) {
        EXTI->PR = (1u << 14);
        lid_switch_changed();
    }
}

/* "Write 1 to clear" is a third register idiom, alongside
   read-modify-write and set/clear pairs. Writing 0 does
   nothing; writing ~0 clears everything, which is a
   convenient bug when you meant to clear one flag.        */
```

> **DEBOUNCING BELONGS IN SOFTWARE, AND NOT IN THE ISR**
>
> A mechanical switch bounces for 1–20 ms, producing dozens of edges. An EXTI handler that increments a counter will see all of them. The standard fix is to have the ISR disable its own EXTI line and start a one-shot timer; the timer's handler samples the pin once, acts, and re-enables EXTI. That gives you exactly one event per press, costs no polling, and keeps the ISR short. A `for`-loop delay inside the handler "works" and blocks every other interrupt for 20 ms — an eternity.

How it shows up

Symptom	Cause
Pin does nothing at all	GPIO port clock not enabled (chapter 09)
Debugger disconnects when you configure a pin	You reconfigured PA13/PA14 — the SWD pins, which default to alternate function

Symptom	Cause
Alternate function pin behaves as GPIO	`MODER` set to AF but `AFR` left at 0, selecting a different function
Occasional lost pin changes	Read-modify-write on `ODR` racing an ISR — use `BSRR`
Input reads as random noise	Floating pin — enable a pull-up or pull-down
I2C works at 100 kHz, fails at 400 kHz	Relying on internal pull-ups; fit external resistors
Two outputs get hot and the rail sags	Two push-pull outputs on one net, driving opposite
Board fails radiated emissions testing	Every pin set to maximum slew rate
One button press counts as several	Switch bounce, undebounced
PA0 and PB0 cannot both be interrupts	EXTI line 0 serves one port only — a layout-time constraint

Check yourself

- ✓ Name the five per-pin settings and what each controls.
- ✓ Explain why you clear before setting a two-bit `MODER` field.
- ✓ Explain what `BSRR` is for and the race it eliminates.
- ✓ Say what happens when both set and reset bits for a pin are written together.
- ✓ Give three situations that require open-drain and one that requires push-pull.
- ✓ Explain why internal pull-ups are inadequate for I2C at 400 kHz.
- ✓ Explain why slew rate is an EMC decision.
- ✓ State the EXTI multiplexing constraint and when it must be resolved.
- ✓ Describe a debouncing scheme that does not block in the ISR.
- ✓ Explain how to detect a shorted output pin in software.

Timers, PWM, input capture and SysTick

A timer is a counter with comparators attached, and almost every real-time behaviour you will ever build is one of four configurations of that idea. Once you see the shape, every vendor's 60-page timer chapter becomes navigable.

One counter, four uses

```
   THE CORE: a counter clocked by (timer clock / PSC+1),
   counting up to ARR, then wrapping and raising an update
   event.

   CNT
    ^
ARR |    /|    /|    /|
    |   / |   / |   / |
CCR |--/--|--/--|--/--|---  compare value
    | /   | /   | /   |
   0+-----+-----+-----+--> time
         UEV   UEV   UEV    update event (interrupt/DMA)

   FOUR USES OF THE SAME HARDWARE:

   1. PERIODIC INTERRUPT   ignore CCR, act on the update
                           event. Your system tick.

   2. PWM OUTPUT           the output pin is high while
                           CNT < CCR, low after. Duty
                           cycle = CCR/ARR. Set and forget:
                           no CPU involvement at all.

   3. INPUT CAPTURE        an edge on a pin COPIES CNT into
                           CCR and raises an interrupt. You
                           read a timestamp, not a value -
                           this is how you measure a pulse
                           without polling.

   4. OUTPUT COMPARE       toggle/set/clear a pin the instant
                           CNT reaches CCR, in hardware, with
                           zero jitter from interrupt latency.

   Configurations 2, 3 and 4 involve NO CPU once started.
   That is the entire point: the timer does the timing, and
   the CPU is free or asleep.
```

The arithmetic, done once

```
/* f_event = f_timer / ((PSC + 1) * (ARR + 1))

   Both registers hold "value minus one" because a counter
   from 0 to N inclusive has N+1 states. Forgetting the -1
   gives an error of one part in ARR - invisible at
   ARR=65535, badly wrong at ARR=1.                          */

/* GOAL: a 1 kHz update event from a 84 MHz timer clock.
   (Remember the APB doubling rule from chapter 09: PCLK1 is
   42 MHz, so TIM2's clock is 84 MHz.)

   84e6 / 1000 = 84000 counts needed.
   84000 > 65535, so it will not fit in ARR alone.

   PSC = 83  -> timer counts at 84e6/84 = 1 MHz
   ARR = 999 -> update every 1000 counts = 1 kHz          */

RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
(void) RCC->APB1ENR;

TIM2->PSC = 83;
TIM2->ARR = 999;

/* PSC is BUFFERED: writes take effect at the next update
   event, not immediately. To apply it now, force an update: */
TIM2->EGR = TIM_EGR_UG;
/* ...which also sets the update interrupt flag, so clear it
   before enabling the interrupt or you take one spurious
   interrupt the instant you enable it.                      */
TIM2->SR = ~TIM_SR_UIF;

TIM2->DIER |= TIM_DIER_UIE;
NVIC_EnableIRQ(TIM2_IRQn);
TIM2->CR1 |= TIM_CR1_CEN;

void TIM2_IRQHandler(void)
{
    TIM2->SR = ~TIM_SR_UIF;    /* clear FIRST (chapter 07)  */
    tick_count++;
}

/* Choosing PSC vs ARR when several combinations work:
   prefer the LARGEST ARR. Resolution is 1/ARR, so
   PSC=83, ARR=999 gives 0.1% duty steps for PWM, while
   PSC=8399, ARR=9 gives 10% steps. Same frequency, wildly
   different usefulness.                                     */
```

PWM, and why it needs no CPU

```
/* PWM on TIM3 channel 1 -> PA6, 20 kHz, for a motor or LED. */

TIM3->PSC = 0;                    /* full 84 MHz            */
TIM3->ARR = 4199;                 /* 84e6/4200 = 20 kHz     */

/* Mode 110 = PWM mode 1: active while CNT < CCR.
   OCxPE = preload enable: writes to CCR take effect at the
   next update, so a duty change never produces a runt
   pulse mid-cycle. Always enable it for PWM.               */
TIM3->CCMR1 |= (6u << 4) | TIM_CCMR1_OC1PE;
TIM3->CCER  |= TIM_CCER_CC1E;     /* enable the output      */
TIM3->CR1   |= TIM_CR1_ARPE;      /* preload ARR too        */
TIM3->EGR    = TIM_EGR_UG;
TIM3->CR1   |= TIM_CR1_CEN;

/* Duty cycle is now ONE STORE, any time, from anywhere: */
TIM3->CCR1 = 2100;                /* 50%                    */

/* No interrupt, no ISR, no jitter. The pin toggles at
   exactly the right instant because a comparator does it.

   ADVANCED TIMERS (TIM1, TIM8) add what motor control
   needs and general timers lack:

     - COMPLEMENTARY outputs (CH1 and CH1N) for a half
       bridge, with hardware DEAD TIME between them so the
       high and low transistors are never on together. That
       dead time is what stops a shoot-through that destroys
       the bridge - it must be hardware, because a software
       delay can be preempted.

     - BREAK INPUT: a fault pin that forces all outputs to a
       safe state in hardware, within nanoseconds, with no
       CPU involvement. This is a safety mechanism; software
       cannot react fast enough to save a motor.

     - REPETITION COUNTER: update event every N cycles
       rather than every cycle.                            */
TIM1->BDTR = (72u << 0)           /* dead time             */
           | TIM_BDTR_BKE         /* break enabled         */
           | TIM_BDTR_MOE;        /* master output enable  */
```

> **THE GENERAL PRINCIPLE: PUSH TIMING INTO HARDWARE**
>
> Any time you find yourself writing "wait, then toggle a pin", ask whether a timer channel can do it. Software timing is subject to interrupt latency, cache effects and scheduler jitter; a comparator is subject to none of them. The difference between a competent embedded design and an amateur one is very often just how much of the timing was handed to peripherals — and it is also what lets the CPU sleep, which chapter 13 depends on entirely.

Input capture: measuring without polling

```
/* Measure the period and duty of an incoming signal in
   hardware. PWM INPUT MODE uses two channels on one pin.  */

/* CH1 captures rising edges -> period
   CH2 captures falling edges -> high time                  */
TIM4->CCMR1 = (1u << 0)      /* CC1S: IC1 mapped to TI1    */
            | (2u << 8);     /* CC2S: IC2 mapped to TI1    */
TIM4->CCER  = TIM_CCER_CC1E              /* CH1 rising     */
            | TIM_CCER_CC2E | TIM_CCER_CC2P;  /* CH2 falling */
TIM4->SMCR  = (5u << 4)      /* TS: trigger = TI1FP1       */
            | (4u << 0);     /* SMS: reset mode            */
TIM4->CR1  |= TIM_CR1_CEN;

/* Now, with no CPU involvement at all:
     CCR1 = period in timer ticks
     CCR2 = high time in timer ticks
     duty = CCR2 / CCR1

   Reset mode means each rising edge zeroes the counter, so
   CCR1 is a period rather than a running timestamp. This is
   how you read an RC receiver, a tachometer, an ultrasonic
   rangefinder's echo pulse, or any sensor with a
   pulse-width output - accurately, while the CPU sleeps.

   THE OVERFLOW PROBLEM: if the signal stops, CNT wraps and
   CCR1 holds a stale value that looks plausible. Always
   enable the update interrupt as a timeout and treat "no
   capture since last update" as signal lost. Reading a
   dead sensor as a valid measurement is worse than reading
   an error.                                                */
```

SysTick, and why an RTOS wants it

```
/* SysTick is architectural (chapter 02): same registers on
   every Cortex-M, at 0xE000E010. A 24-bit down-counter.
   It exists so an RTOS port is portable across vendors.   */

SysTick->LOAD = (SystemCoreClock / 1000u) - 1u;  /* 1 ms   */
SysTick->VAL  = 0;
SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk   /* core clock */
              | SysTick_CTRL_TICKINT_Msk
              | SysTick_CTRL_ENABLE_Msk;

volatile uint32_t g_ms;
void SysTick_Handler(void) { g_ms++; }   /* flag auto-clears */

/* 24 bits: at 168 MHz the maximum period is
   16777216/168e6 = 99.8 ms. It cannot be your only
   long-duration timer.

   THE OVERFLOW BUG, and the fix:                          */

/* WRONG - breaks when g_ms wraps at 49.7 days */
if (g_ms > deadline) { ... }

/* RIGHT - unsigned subtraction wraps correctly */
if ((int32_t)(g_ms - deadline) >= 0) { ... }

/* The second form works across the wrap because the
   subtraction is done modulo 2^32 and then interpreted as
   signed. It is correct for any interval under ~24 days.
   Every timekeeping bug of the "device fails after seven
   weeks of uptime" kind is the first form.                */

/* Reading g_ms outside the ISR needs care on a 32-bit
   counter it is a single aligned access, so it is atomic on
   Cortex-M. A 64-bit counter is NOT - it needs a critical
   section or a seqlock, or you will read a torn value once
   every few billion reads, which is once a week in a busy
   system.                                                  */
```

How it shows up

Symptom	Cause
Period is exactly half or double	APB timer-clock doubling rule (chapter 09)
Frequency off by one part in ARR	Forgot that PSC and ARR hold value minus one
Prescaler change has no effect	PSC is buffered — force an update with `EGR.UG`
One spurious interrupt right after enabling	`EGR.UG` set the update flag; clear it before enabling the interrupt
PWM duty steps are very coarse	ARR too small — trade prescaler for counter range
Glitch or runt pulse when changing duty	CCR preload (`OCxPE`) not enabled
PWM configured but the pin stays low	On an advanced timer, `BDTR.MOE` not set
Motor bridge destroys itself	No hardware dead time between complementary outputs
Dead sensor reads as a valid measurement	Stale capture register; no update-event timeout
Device misbehaves after 49.7 days	Tick comparison with `>` instead of wrap-safe subtraction

Check yourself

- ✓ Give the formula relating timer clock, PSC, ARR and event frequency.
- ✓ Explain why both PSC and ARR are stored as value minus one.
- ✓ Choose PSC and ARR for 1 kHz from an 84 MHz timer clock, and justify the split.
- ✓ Explain why `EGR.UG` is needed and the side effect it has.
- ✓ Explain what CCR preload prevents.
- ✓ Name the three features advanced timers add and what each is for.
- ✓ Explain why hardware dead time cannot be replaced by a software delay.
- ✓ Describe PWM input mode and what CCR1 and CCR2 hold.
- ✓ Explain the stale-capture failure and how to detect signal loss.
- ✓ Write a wrap-safe deadline comparison and explain why it works.

Polling, interrupts and DMA: three ways to move data

Every peripheral transfer is one of three mechanisms, and choosing wrongly is the most common cause of a design that works on the bench and falls over in the field. This chapter is the decision, with the arithmetic that makes it.

The three mechanisms

```
POLLING
  CPU: while (!(flag)) { }  then read
  Cost: 100% of a core, doing nothing useful
  Latency: as good as it gets - you are already looking
  Use when: the wait is shorter than an interrupt entry
            (~12 cycles + your prologue), or during init,
            or in a hard real-time loop that has nothing
            else to do.

INTERRUPT
  CPU: handler runs per BYTE or per EVENT
  Cost: ~20-40 cycles of overhead per event
  Latency: 12 cycles + whatever is blocking you
  Use when: events are infrequent relative to the
            overhead, or their timing is unpredictable.

DMA
  CPU: configure once, then nothing until the block ends
  Cost: bus bandwidth, contention with the CPU
  Latency: irrelevant per byte; one interrupt per BLOCK
  Use when: the data is a block, or the rate is high
            enough that per-event interrupts would
            saturate the CPU.

THE ARITHMETIC THAT DECIDES:

  UART at 115200 baud = 11520 bytes/s
    per-byte interrupt at 30 cycles = 345,600 cycles/s
    on a 168 MHz core = 0.2% CPU. Interrupts are fine.

  UART at 3 Mbaud = 300,000 bytes/s
    per-byte interrupt at 30 cycles = 9,000,000 cycles/s
    = 5.4% CPU, and 300k interrupts/s starves everything
    else. Use DMA.

  ADC at 1 Msample/s
    per-sample interrupt = 30,000,000 cycles/s = 18% CPU
    doing nothing but copying. Use DMA. Always.

Do this calculation. It takes thirty seconds and it is
the difference between a design that scales and one that
is rewritten under deadline.
```

Polling, done correctly

```
/* Polling gets a bad name because it is usually written
   without a timeout. This version cannot hang.            */

int uart_send_blocking(const uint8_t *p, size_t n, uint32_t ms)
{
    uint32_t deadline = g_ms + ms;

    for (size_t i = 0; i < n; i++) {
        while (!(USART1->ISR & USART_ISR_TXE)) {
            if ((int32_t)(g_ms - deadline) >= 0) {
                return -ETIMEDOUT;      /* the crucial part */
            }
        }
        USART1->TDR = p[i];
    }
    return 0;
}

/* Three rules for any polling loop that ships:

   1. A TIMEOUT, always. A peripheral that never sets its
      flag - because a clock is off, a cable is unplugged,
      or the far end died - turns a poll into a hang. This
      is the single most common cause of a field unit that
      "just stops".

   2. The flag variable must be volatile, or read through a
      volatile pointer. Otherwise the compiler may hoist the
      load out of the loop and spin forever on a stale value
      (see the C track, chapter 10).

   3. Consider what else is starved. A 10 ms poll with
      interrupts enabled is fine; the same poll inside a
      critical section delays every interrupt by 10 ms.     */
```

DMA: the controller does the copying

```
/* DMA is a second bus master. It reads and writes memory
   without the CPU, arbitrating for the bus alongside it.  */

/* UART receive into a circular buffer - the configuration
   worth memorising, because it never overruns.            */

DMA2_Stream5->PAR  = (uint32_t) &USART1->RDR;   /* source */
DMA2_Stream5->M0AR = (uint32_t) rx_buffer;      /* dest   */
DMA2_Stream5->NDTR = sizeof(rx_buffer);
DMA2_Stream5->CR   = (4u << 25)          /* channel 4      */
                   | DMA_SxCR_MINC       /* increment mem  */
                   | DMA_SxCR_CIRC       /* CIRCULAR       */
                   | DMA_SxCR_TCIE       /* half/full IRQ  */
                   | DMA_SxCR_HTIE;
USART1->CR3 |= USART_CR3_DMAR;
DMA2_Stream5->CR |= DMA_SxCR_EN;

/* CIRCULAR mode means NDTR reloads automatically at the end
   and the controller wraps to the buffer start. Combined
   with the half-transfer interrupt you get a classic
   double-buffer: process the first half while the second
   fills, and vice versa. The UART can now receive
   continuously, forever, with two interrupts per buffer
   instead of one per byte.

   Add the UART's IDLE-line interrupt and you also learn
   when a variable-length message ENDED, which is what makes
   this pattern work for protocols rather than just streams:

     bytes_received = sizeof(rx_buffer) - DMA2_Stream5->NDTR;

   That expression, read in the IDLE handler, tells you
   exactly how much arrived. This combination - circular DMA
   plus IDLE detection - is the standard high-performance
   UART receive design, and it is worth knowing by heart.  */
```

FOUR DMA RULES THAT ARE NOT OPTIONAL

1. THE BUFFER MUST BE REACHABLE.
   Many parts have RAM banks the DMA engine cannot see -
   CCM RAM on STM32F4 is the classic. A buffer there
   produces a transfer error or silent zeros. Place DMA
   buffers explicitly (chapter 05).

2. THE BUFFER MUST NOT BE ON THE STACK.
   A local array goes out of scope when the function
   returns; the DMA controller keeps writing to it. This
   corrupts whatever occupies that stack space next, at
   a delay, and is genuinely one of the hardest embedded
   bugs to find. Use static or heap.

3. CACHE MAINTENANCE ON M7.
   Cortex-M7 has data caches. Before a memory-to-
   peripheral transfer, CLEAN the cache so the DMA sees
   your writes. After a peripheral-to-memory transfer,
   INVALIDATE so the CPU does not read stale lines.
   Miss this and it works at -O0 and fails at -O2, or
   works until the buffer happens to stay in cache.

     SCB_CleanDCache_by_Addr((uint32_t*)buf, len);
     ... start TX DMA ...

     ... RX DMA completes ...
     SCB_InvalidateDCache_by_Addr((uint32_t*)buf, len);

   M0/M3/M4 have no data cache, so this is one of the
   few places where code does not port upward cleanly.

4. ALIGNMENT AND WIDTH MUST AGREE.
   A 32-bit peripheral width with a byte-aligned buffer
   is a transfer error, not a slow transfer.

The compiler does not know about DMA

```
/* DMA writes memory behind the compiler's back. Every
   variable the DMA controller touches must be volatile, and
   volatile alone is still not quite enough.               */

static volatile uint8_t rx_buffer[256];   /* volatile: yes */
static volatile int     rx_done;

void DMA2_Stream5_IRQHandler(void)
{
    DMA2->HIFCR = DMA_HIFCR_CTCIF5;
    rx_done = 1;
}

void consumer(void)
{
    while (!rx_done) { }        /* volatile: the load repeats */

    __asm volatile ("dmb");     /* ordering: see below        */
    process(rx_buffer);
}

/* The DMB matters. volatile guarantees the COMPILER does
   not reorder or elide the accesses. It says nothing about
   the memory system. On Cortex-M the core is largely
   in-order and this is usually benign, but write buffers
   and the bus matrix can still reorder the DMA's buffer
   writes relative to its completion signal.

   The rule: volatile for the compiler, a barrier for the
   hardware. They solve different problems and you often
   need both. This is exactly the distinction the ARM track
   makes in its barriers chapter, and it applies here in
   miniature.                                              */
```

A decision table

Situation	Mechanism	Why
Reading a status register during init	Polling	Happens once; an ISR would be more code than the wait
Waiting ~1 µs for a peripheral to be ready	Polling	Shorter than interrupt entry and exit
Button press	Interrupt	Rare and unpredictable; polling wastes power
UART console at 115200	Interrupt	0.2% CPU — simplicity wins
UART at 1 Mbaud or above	DMA	Interrupt rate would dominate the CPU
ADC sampling continuously	DMA, circular	Per-sample interrupts are unaffordable at any real rate
SPI display refresh	DMA	Large block, no per-byte decision to make
I2C register read of 2 bytes	Interrupt	DMA setup costs more than the transfer
Audio in or out	DMA, double-buffered	Hard deadline, continuous, unattended
Motor commutation on a hall edge	Interrupt, highest priority	Latency is the entire requirement

How it shows up

Symptom	Cause
Device stops responding, no crash	A polling loop with no timeout waiting on a flag that never sets
Poll loop never exits despite the flag setting	Missing `volatile`; the load was hoisted
DMA completes but the buffer holds zeros	Buffer in RAM the DMA engine cannot reach
Random corruption some time after a transfer	DMA buffer was a stack local that went out of scope
Works at `-O0`, fails at `-O2`, Cortex-M7	Missing cache clean or invalidate around the transfer
DMA reports a transfer error immediately	Alignment or width mismatch between peripheral and memory
UART drops bytes under load	Per-byte interrupts cannot keep up — move to circular DMA
Received message length is unknown	Use IDLE-line detection with `NDTR` to find the boundary
System responsive on the bench, not in the field	Interrupt load calculated for the demo rate, not the real one
Occasional stale data read after DMA completes	No barrier between the completion flag and the buffer read

Check yourself

- ✓ Give the CPU cost of per-byte interrupts for a 3 Mbaud UART on a 168 MHz core.
- ✓ State the three rules for a polling loop that ships.
- ✓ Explain why a polling loop without a timeout is a field-reliability problem.
- ✓ Describe circular DMA plus IDLE detection and what problem it solves.
- ✓ Write the expression that gives bytes received so far from `NDTR`.
- ✓ Explain why a DMA buffer must not be a stack local.
- ✓ State the cache maintenance required around DMA on Cortex-M7, in both directions.
- ✓ Explain why `volatile` and a barrier solve different problems.
- ✓ Choose a mechanism for a 2-byte I2C register read and justify it.
- ✓ Explain why DMA buffers sometimes must be placed in a specific RAM bank.

Low-power modes and the current budget

Battery life is arithmetic, not craft. This chapter covers the sleep modes a Cortex-M offers, the duty-cycle calculation that decides whether your product lasts a year or a fortnight, and the handful of mistakes that account for nearly every "why is it drawing 3 mA" question.

The modes

```
Typical STM32L4-class figures. Every vendor names these
differently; the STRUCTURE is universal.

MODE        CPU   RAM   PERIPH  WAKE     CURRENT  WAKE TIME
-----------------------------------------------------------
RUN         on    on    on      -        10 mA    -
SLEEP       off   on    on      any IRQ   2 mA    ~0 cycles
STOP        off   on    few     EXTI,RTC  2 uA    ~5 us
STANDBY     off   LOST  none    pin,RTC   0.3 uA  ~250 us
                  (few                             + full
                   backup                          reboot
                   regs                            from
                   survive)                        reset

THE TWO THINGS THAT CHANGE BETWEEN ROWS:

1. What still has a CLOCK. In SLEEP the core clock stops
   but peripherals run, so a UART keeps receiving and a
   DMA transfer keeps going. That is why SLEEP + DMA is
   such a powerful combination.

2. Whether SRAM keeps its CONTENT. STANDBY loses it.
   Waking from STANDBY re-runs Reset_Handler and your
   program starts from the top - it is a reboot, not a
   resume. Anything you must remember goes in backup
   registers or the RTC's domain.

The jump from SLEEP to STOP is a factor of a thousand.
That is the decision that matters; everything else is
refinement.
```

Entering a mode

```
/* WFI - wait for interrupt. The core halts until an
   interrupt arrives. This is the whole mechanism.         */

__WFI();

/* SLEEP-ON-EXIT: an elegant pattern for a purely
   event-driven device. The core re-enters sleep
   automatically when an ISR returns, so main() never runs
   again after setup and there is no idle loop at all.     */
SCB->SCR |= SCB_SCR_SLEEPONEXIT_Msk;

/* Deeper modes set SLEEPDEEP and then WFI. */
static void enter_stop_mode(void)
{
    /* Select STOP with the regulator in low-power mode. */
    PWR->CR1 &= ~PWR_CR1_LPMS;
    PWR->CR1 |=  PWR_CR1_LPMS_STOP2;

    SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
    __DSB();                       /* drain writes first   */
    __WFI();
    SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;

    /* CRITICAL: STOP mode usually switches SYSCLK back to
       the internal oscillator on wake. Your 168 MHz PLL is
       off and every timing assumption is wrong until you
       restore it.                                          */
    clock_init_168mhz();
}
```

THE WFI RACE, AND THE STANDARD FIX

```
THE BUG

  while (!work_pending) {
      __WFI();          <-- interrupt fires HERE, between
  }                         the test and the WFI. The ISR
                            sets work_pending and returns.
                            WFI then sleeps anyway, and
                            nothing will wake us until the
                            NEXT interrupt - which may
                            never come.

Symptom: the device hangs roughly once a day, in a way
that never reproduces on the bench.

THE FIX: disable interrupts around the test. WFI wakes on
a pending interrupt even when PRIMASK masks it - that is
an architectural guarantee, and it exists precisely for
this pattern.

  __disable_irq();
  if (!work_pending) {
      __WFI();          /* wakes on pending IRQ even    */
  }                     /* though PRIMASK blocks entry  */
  __enable_irq();       /* NOW the ISR actually runs    */

The interrupt is taken immediately after the enable. No
window, no lost wakeup.
```

The arithmetic that decides the product

```
/* A sensor that wakes every 10 s, measures for 20 ms, and
   transmits for 100 ms once a minute.

   AVERAGE CURRENT = sum of (current * time) / period

   Per minute:
     6 wakes  x 20 ms x 10 mA  = 1200 mA-ms  measuring
     1 TX     x 100 ms x 30 mA = 3000 mA-ms  radio
     remainder: 60000 - 220 ms x 2 uA
                = 59780 ms x 0.002 mA = 120 mA-ms  sleeping
     ------------------------------------------------
     total = 4320 mA-ms per 60000 ms
     average = 0.072 mA = 72 uA

   A 220 mAh coin cell / 0.072 mA = 3055 hours = 127 days.

   NOW RAISE SLEEP CURRENT FROM 2 uA TO 50 uA - one pin left
   floating, say:
     sleep contribution = 59780 x 0.050 = 2989 mA-ms
     total = 7189, average = 0.120 mA
     battery life = 76 days.

   A single floating pin cost 40% of the product's life. It
   would never be noticed in testing, because the device
   works perfectly.

   THE LESSON: in a duty-cycled design the sleep current
   dominates the total even though it is the smallest
   number, because it is multiplied by the largest time.
   Optimise the term with the biggest product, which is
   almost never the one that feels slow.                  */
```

Where the microamps actually go

```
IN ROUGH ORDER OF HOW OFTEN THEY ARE THE CULPRIT:

1. THE DEBUGGER IS ATTACHED.
   SWD keeps clocks alive. Measurements with a debugger
   connected are meaningless - often 100x too high.
   Measure on battery, detached, always.

2. A FLOATING INPUT PIN.
   An unconnected CMOS input drifts near the switching
   threshold and both transistors conduct. Microamps to
   milliamps, per pin. EVERY unused pin must be an
   output, or an input with a pull enabled, or analog.
   Analog mode is the lowest of the three because it
   disconnects the Schmitt trigger entirely.

3. A PULL-UP FIGHTING A DRIVEN LOW.
   A 4.7 kO pull-up to 3.3 V held low is 700 uA,
   continuously. I2C buses idle high for this reason; if
   you sleep with SDA held low you pay for it all night.

4. A PERIPHERAL CLOCK LEFT ENABLED.
   Gate everything you are not using (chapter 09). Some
   peripherals also need explicit disabling, not just
   clock gating - an ADC's analog front end draws current
   from the reference regardless of its clock.

5. AN LED.
   2 mA is 30x your entire sleep budget. Indicator LEDs
   must be duty-cycled or omitted on battery products.

6. AN EXTERNAL SENSOR STILL POWERED.
   Its datasheet's "sleep current" may exceed your MCU's
   by an order of magnitude. Consider power-gating it
   with a MOSFET so its current is genuinely zero.

7. THE REGULATOR'S OWN QUIESCENT CURRENT.
   A regulator with 50 uA quiescent draw makes a 2 uA MCU
   irrelevant. This is a schematic decision that caps
   your achievable sleep current before firmware runs.
```

> **MEASURE, DO NOT MODEL**
>
> Sleep-current bugs are invisible to every software tool. You need an ammeter that resolves microamps and milliamps in the same trace — a dedicated low-power analyser, or a sense resistor and a scope. The characteristic finding is a device that sleeps beautifully in one state and not in another, because one code path leaves a peripheral enabled. Plot current against time across a full duty cycle and the extra current will be a visible plateau you can align with your code.

Waking up, and RTC scheduling

```
/* From STOP or STANDBY, the wake sources are limited to
   what still has a clock: the RTC, a handful of EXTI pins,
   and sometimes a low-power UART or comparator.           */

static void sleep_for_seconds(uint32_t s)
{
    RTC->WPR = 0xCA; RTC->WPR = 0x53;     /* unlock         */
    RTC->CR &= ~RTC_CR_WUTE;
    while (!(RTC->ISR & RTC_ISR_WUTWF)) { }

    RTC->WUTR = s - 1;                    /* 1 Hz clock     */
    RTC->CR |= RTC_CR_WUCKSEL_2;
    RTC->CR |= RTC_CR_WUTE | RTC_CR_WUTIE;
    RTC->WPR = 0xFF;                      /* relock         */

    EXTI->IMR  |= EXTI_IMR_MR22;          /* RTC wakeup line */
    EXTI->RTSR |= EXTI_RTSR_TR22;

    enter_stop_mode();
}

/* THE 32.768 kHz CRYSTAL EARNS ITS PLACE HERE. The internal
   low-speed RC drifts several percent with temperature, so
   a device that wakes "every 10 seconds" wakes every 9.7 to
   10.3. Over a day that is minutes of accumulated error -
   fatal for a synchronised sensor network, irrelevant for a
   thermostat. Know which you are building.

   WAKE TIME COSTS ENERGY TOO. Waking from STANDBY takes
   ~250 us at full run current before your code executes,
   plus PLL lock time. For very short duty cycles STOP can
   beat STANDBY despite its higher sleep current, because
   you spend less time paying the wake penalty. Compute
   both; the crossover is often around a 1-second period. */
```

How it shows up

Symptom	Cause
Sleep current 100x the datasheet figure	Debugger attached — measure detached, on battery
Current varies between identical units	Floating pins; the threshold is near the switching point
Device hangs about once a day	The WFI race — test the flag with interrupts disabled
Timing wrong after waking from STOP	Clock fell back to the internal oscillator; restore the PLL
Variables lost after a deep sleep	STANDBY loses SRAM — it is a reboot, not a resume
Battery life half the calculation	The sleep-current term dominates; find the extra microamps
Steady 700 µA that appears only at night	A pull-up fighting a line left driven low
Wake interval drifts over hours	RTC on the internal RC rather than the 32.768 kHz crystal
Sleep current fine, average current poor	Wake penalty dominates — compare STOP against STANDBY
Cannot get below 50 µA whatever you do	The regulator's quiescent current, not the MCU

Check yourself

- ✓ Name the four sleep modes and the two properties that change between them.
- ✓ Explain why SLEEP mode still allows a DMA transfer to complete.
- ✓ Say what is lost entering STANDBY and what that means for your program's structure.
- ✓ Describe the WFI race and write the correct pattern.
- ✓ Explain why WFI wakes on an interrupt that PRIMASK is masking.
- ✓ Compute average current for a device with a given duty cycle.
- ✓ Explain why sleep current dominates a duty-cycled budget despite being the smallest number.
- ✓ Give three reasons an unused pin costs current, and the best configuration for one.
- ✓ Explain why measurements with a debugger attached are meaningless.
- ✓ Say when STOP can beat STANDBY on average current.

The MPU: memory protection without an MMU

An MPU cannot translate addresses, so it gives you no virtual memory and no isolation between identical address spaces. What it does give you is the ability to turn silent corruption into an immediate, localised fault — which on a system with no `SIGSEGV` is worth a great deal.

What it is and is not

	MMU (Cortex-A)	MPU (Cortex-M)
Translates addresses	Yes — virtual to physical	No. Addresses are physical, always
Granularity	4 kB pages, millions of them	8 or 16 regions, sized by powers of two
Configuration store	Page tables in memory	A handful of registers
Per-process address space	Yes	No — one physical space, different permissions
Cost to reconfigure	TLB maintenance	A few register writes; cheap enough per context switch
What it buys you	Isolation and virtual memory	Fault-on-mistake, and privilege separation

So the MPU is not a security boundary in the way TrustZone is (that is a separate mechanism), and it is not virtual memory. It is a programmable set of rules about which addresses may be read, written or executed, and by whom.

The three things it is genuinely worth doing

```
1. STACK GUARD - the highest-value use by far.

   Place a small, no-access region immediately below each
   stack. Overflow hits it and takes a MemManage fault
   AT THE INSTRUCTION THAT OVERFLOWED, with the faulting
   task identifiable.

   Without it:  the stack grows into .bss, a variable
                changes for no reason, and you spend
                three days.
   With it:     an immediate fault naming the culprit.

   +----------------+ <- stack top
   |  task stack    |
   |       |        |
   |       v        |
   +----------------+ <- stack bottom
   |  GUARD 32 B    |  MPU: no access
   +----------------+
   |  other data    |
   +----------------+

2. EXECUTE NEVER on RAM and peripherals.

   Mark everything that is not flash as XN. A corrupted
   function pointer or a smashed return address then
   faults instead of executing whatever bytes it found.
   This is the single cheapest robustness measure
   available, and it costs one region.

3. READ-ONLY on .rodata and the vector table.

   Catches wild pointer writes at the instant they
   happen rather than when the corrupted value is later
   used.

Everything else - full task isolation, unprivileged
tasks, per-task peripheral permissions - is real but
costs regions and design effort. These three are close
to free.
```

Configuring a region

```
/* ARMv7-M MPU. Regions are numbered, sized by powers of two,
   and must be aligned to their own size - a 1 kB region
   must start on a 1 kB boundary. That constraint drives
   your linker script (chapter 05).                        */

static void mpu_region(uint32_t n, uint32_t base,
                       uint32_t size_log2, uint32_t attr)
{
    MPU->RNR  = n;
    MPU->RBAR = base & MPU_RBAR_ADDR_Msk;
    MPU->RASR = attr
              | ((size_log2 - 1u) << MPU_RASR_SIZE_Pos)
              | MPU_RASR_ENABLE_Msk;
}

/* Access permission field (AP), bits 24-26 of RASR:
     000  no access at all
     001  privileged RW,        unprivileged none
     010  privileged RW,        unprivileged RO
     011  full access           (RW for both)
     101  privileged RO,        unprivileged none
     110  read-only             (RO for both)             */

#define AP_NONE     (0u << 24)
#define AP_PRIV_RW  (1u << 24)
#define AP_FULL     (3u << 24)
#define AP_RO       (6u << 24)
#define XN          (1u << 28)
#define MEM_NORMAL  ((1u << 17) | (1u << 18))  /* C and B  */

void mpu_init(void)
{
    extern uint32_t _stack_guard;   /* from the linker script */

    MPU->CTRL = 0;                  /* disable while editing  */
    __DSB(); __ISB();

    /* 0: all of SRAM, full access, execute never.           */
    mpu_region(0, 0x20000000, 17, AP_FULL | XN | MEM_NORMAL);

    /* 1: the guard band below the stack. Higher-numbered
          regions WIN where they overlap, so this carves a
          hole out of region 0.                            */
    mpu_region(1, (uint32_t) &_stack_guard, 5, AP_NONE);

    /* 2: peripherals, privileged only, execute never.       */
    mpu_region(2, 0x40000000, 29, AP_PRIV_RW | XN);

    /* PRIVDEFENA: privileged code keeps the default memory
       map for anything no region covers. Without it, any
       address you did not explicitly describe faults - which
       is stricter, and a good way to discover you forgot
       the flash region.                                    */
    MPU->CTRL = MPU_CTRL_ENABLE_Msk | MPU_CTRL_PRIVDEFENA_Msk;
    __DSB(); __ISB();
}

/* THE BARRIERS ARE MANDATORY. Enabling the MPU changes how
   the very next fetch is permission-checked. Without DSB
```

```
   and ISB you get a fault whose address makes no sense,
   and which moves when you change unrelated code.         */
```

> **ARMV8-M REDESIGNED THIS, AND FOR THE BETTER**
> ARMv8-M (Cortex-M23/M33/M55) replaces power-of-two sizing with plain base and limit addresses, so a region can be any size at any 32-byte-aligned address. It also moves memory attributes into an indexed `MAIR` register, as on Cortex-A. If you have written v7-M MPU code, the v8-M version is substantially simpler and the alignment gymnastics disappear. Check which you have before copying an example — the register layouts are incompatible.

Per-task protection in an RTOS

```
/* The MPU is cheap enough to reprogram on every context
   switch, which is what FreeRTOS-MPU and Zephyr's userspace
   do. Each task carries its own region set.               */

typedef struct {
    uint32_t rbar;
    uint32_t rasr;
} mpu_region_t;

typedef struct {
    uint32_t     *stack_top;
    mpu_region_t  regions[3];   /* task stack + 2 private   */
} tcb_t;

/* Inside the context switch (chapter 18), after selecting
   the next task and before returning:                     */
static inline void mpu_load(const tcb_t *t)
{
    MPU->CTRL = 0;
    for (int i = 0; i < 3; i++) {
        MPU->RNR  = 4 + i;          /* task regions: 4,5,6  */
        MPU->RBAR = t->regions[i].rbar;
        MPU->RASR = t->regions[i].rasr;
    }
    MPU->CTRL = MPU_CTRL_ENABLE_Msk | MPU_CTRL_PRIVDEFENA_Msk;
    __DSB(); __ISB();
}

/* Roughly 20 cycles. Against a context switch of ~100
   cycles that is affordable, and it buys you a task that
   physically cannot corrupt another task's stack.

   THE HONEST LIMITATION: with 8 regions and 3-4 spent on
   global policy, you have 4-5 per task. A task needing its
   stack, a DMA buffer, two peripherals and a shared queue
   has already run out. Full isolation on Cortex-M is a real
   design constraint, not a checkbox - which is why most
   projects use the MPU for the three high-value cases above
   and stop there.                                          */
```

Reading a MemManage fault

```
/* The MPU reports through MemManage, which is a
   configurable fault - you must ENABLE it, or it escalates
   to HardFault and you lose the diagnostic detail.        */

SCB->SHCSR |= SCB_SHCSR_MEMFAULTENA_Msk;

void MemManage_Handler(void)
{
    uint32_t mmfsr = (SCB->CFSR & 0xFFu);

    /*  bit 0  IACCVIOL   instruction fetch violation - you
                          jumped into an XN region
        bit 1  DACCVIOL   data access violation
        bit 3  MUNSTKERR  fault while unstacking on return
        bit 4  MSTKERR    fault while STACKING on entry -
                          this is your stack guard firing
        bit 7  MMARVALID  MMFAR holds the faulting address  */

    uint32_t addr = (mmfsr & (1u << 7)) ? SCB->MMFAR : 0xFFFFFFFF;

    fault_record(mmfsr, addr, current_task_id());
    __asm volatile ("bkpt #0");
}

/* MSTKERR is the one to recognise on sight: the hardware
   could not push the exception frame because the stack
   pointer had already run into the guard region. That is a
   stack overflow, caught at the moment it happened, naming
   the task. Compare with the alternative - silent
   corruption discovered days later - and the MPU has paid
   for itself with this one bit.                            */
```

How it shows up

Symptom	Cause
Fault immediately on enabling the MPU	A region you need is not covered and `PRIVDEFENA` is off
Fault address makes no sense	Missing `DSB` / `ISB` after writing `CTRL`
Region appears to have no effect	Base address not aligned to the region size — low bits ignored
Two regions overlap unexpectedly	Higher region number wins; check the ordering
MemManage never fires, HardFault does	`MEMFAULTENA` not set — the fault escalated
`MMFAR` holds nonsense	`MMARVALID` is clear; the address was not captured
Stack overflow still silent	Guard region sized smaller than one exception frame — use at least 32 bytes
Example code does not compile for this part	ARMv8-M MPU registers differ from ARMv7-M entirely
Run out of regions adding one task	The real constraint — reduce per-task regions or drop full isolation
Faults appear only after enabling optimisation	Inlining moved a buffer; the region no longer covers it

Check yourself

- ✓ State two things an MPU does not do that an MMU does.
- ✓ Name the three highest-value uses of an MPU and why each is nearly free.

- ✓ Explain how a stack guard region converts a silent bug into an immediate fault.
- ✓ State the alignment rule for an ARMv7-M region and its effect on the linker script.
- ✓ Say which region wins where two overlap.
- ✓ Explain what `PRIVDEFENA` does and the trade-off in leaving it off.
- ✓ Explain why `DSB` and `ISB` are required around MPU changes.
- ✓ Say what `MSTKERR` means and why it is the most useful fault bit.
- ✓ Explain why MemManage faults sometimes appear as HardFaults.
- ✓ Give the practical limit on per-task isolation with 8 regions.

Decoding a HardFault

A HardFault is not a mystery. The hardware records what happened, where, and in which instruction — you simply have to read four registers and know where the stack frame is. This chapter turns the most feared event on Cortex-M into a five-minute procedure.

The fault taxonomy

```
CONFIGURABLE FAULTS - specific, informative, and DISABLED
at reset:

  MemManage   MPU permission violation (chapter 14)
  BusFault    the bus said no: bad address, peripheral
              with no clock, external memory not ready
  UsageFault  undefined instruction, illegal state,
              unaligned access, divide by zero

HARDFAULT - the catch-all. You land here when:

  a) the specific fault is not enabled (the default!), so
     it ESCALATES; or
  b) a fault occurs inside a fault handler; or
  c) a fault occurs at a priority that cannot be
     preempted - "priority escalation".

FIRST ACTION IN ANY NEW PROJECT: enable the specific
faults, so you get the informative one instead of the
catch-all.

  SCB->SHCSR |= SCB_SHCSR_USGFAULTENA_Msk
             |  SCB_SHCSR_BUSFAULTENA_Msk
             |  SCB_SHCSR_MEMFAULTENA_Msk;

This one line converts "HardFault, no information" into
"UsageFault, UNALIGNED, at address X" and saves hours on
every future bug.
```

The Configurable Fault Status Register

```
/* CFSR at 0xE000ED28 packs all three status bytes. */

/*  MMFSR   bits 0-7    MemManage
      0  IACCVIOL   instruction fetch from XN region
      1  DACCVIOL   data access violation
      3  MUNSTKERR  fault unstacking on exception return
      4  MSTKERR    fault stacking on entry = STACK OVERFLOW
      5  MLSPERR    fault during lazy FP stacking
      7  MMARVALID  MMFAR is valid

    BFSR    bits 8-15   BusFault
      8  IBUSERR       instruction bus error
      9  PRECISERR     data error, BFAR holds the address
     10  IMPRECISERR   data error, address UNKNOWN (see below)
     11  UNSTKERR      unstacking error
     12  STKERR        stacking error
     13  LSPERR        lazy FP stacking error
     15  BFARVALID     BFAR is valid

    UFSR    bits 16-31  UsageFault
     16  UNDEFINSTR    undefined instruction
     17  INVSTATE      invalid state - usually the Thumb bit
     18  INVPC         invalid PC on exception return
     19  NOCP          coprocessor absent - FPU not enabled
     24  UNALIGNED     unaligned access
     25  DIVBYZERO     divide by zero (if DIV_0_TRP set)  */

/* The three that account for most real faults:

   PRECISERR + BFARVALID
     A bad pointer, and BFAR names the address. Usually a
     null pointer (address near 0) or a peripheral whose
     clock is off (address in 0x4xxxxxxx).

   IMPRECISERR
     The write was buffered; the core moved on before the
     bus reported the error, so the address is LOST. Fix by
     putting a DSB after suspect writes to force precision
     while debugging, or set DISDEFWBUF in ACTLR to disable
     write buffering entirely - slow, but every fault
     becomes precise.

   MSTKERR
     Stack overflow, caught by an MPU guard region. The most
     valuable single bit on the chip.                      */
```

Reading the stacked frame

The hardware pushed eight registers before entering the handler. The stacked PC is the instruction that faulted, and that is what you actually want.

```
/* Step 1: a naked handler that finds the right stack and
   hands it to C. EXC_RETURN bit 2 says which one.         */

__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile (
        "tst   lr, #4            \n"  /* bit 2 of EXC_RETURN */
        "ite   eq                \n"
        "mrseq r0, msp           \n"  /* clear -> MSP        */
        "mrsne r0, psp           \n"  /* set   -> PSP        */
        "b     hardfault_report  \n"
    );
}

/* Step 2: the frame, in the order the hardware pushed it. */
typedef struct {
    uint32_t r0, r1, r2, r3;
    uint32_t r12;
    uint32_t lr;      /* return address of the faulting fn */
    uint32_t pc;      /* THE FAULTING INSTRUCTION          */
    uint32_t psr;
} frame_t;

void hardfault_report(frame_t *f)
{
    uint32_t cfsr = SCB->CFSR;
    uint32_t hfsr = SCB->HFSR;

    /* HFSR bit 30 = FORCED: a configurable fault escalated.
       If this is set, CFSR tells you the real cause.      */

    fault_log_t *log = &g_crash_log;      /* in .noinit    */
    log->magic = 0xDEADBEEF;
    log->pc    = f->pc;
    log->lr    = f->lr;
    log->cfsr  = cfsr;
    log->hfsr  = hfsr;
    log->bfar  = (cfsr & (1u << 15)) ? SCB->BFAR : 0;
    log->mmfar = (cfsr & (1u << 7))  ? SCB->MMFAR : 0;

    __asm volatile ("bkpt #0");   /* halt if debugger present */
    NVIC_SystemReset();           /* otherwise reboot          */
}

/* Step 3: turn the PC into a line of source.

     $ arm-none-eabi-addr2line -e firmware.elf -f -C 0x080012a4
     i2c_read
     /home/user/proj/src/i2c.c:142

   That is the whole procedure. Fault -> PC -> source line,
   in about a minute once the handler is in place.         */
```

PUT THE CRASH LOG IN `.NOINIT` AND REPORT IT AFTER REBOOT

A fault handler that only halts is useful on your desk and useless in the field. Writing the log to a `.noinit` section (chapter 05) means it survives the reset; early in the next boot you check the magic number, transmit or store the record, and clear it. Now a unit that crashed in a customer's cabinet tells you the faulting PC when it comes back up. This costs about forty lines and is the difference between diagnosing a field failure and asking for the unit back.

What actually causes them

Fault bits	Almost always means
`PRECISERR`, BFAR near 0	Null pointer dereference — an `malloc`/lookup that returned NULL and went unchecked
`PRECISERR`, BFAR in 0x4xxxxxxx	Peripheral register accessed with its clock gated off
`IMPRECISERR`	A buffered write went bad earlier; add `DSB` or disable write buffering to localise it
`MSTKERR`	Stack overflow — the MPU guard did its job
`INVSTATE`	Branch to an address with the Thumb bit clear — a corrupted or hand-built function pointer
`UNDEFINSTR`	Executing data: a wild jump, or a corrupted return address
`NOCP`	Float instruction with the FPU not enabled in `CPACR`
`UNALIGNED`	A packed struct member or a cast to a wider type at an odd address
`INVPC`	Corrupted `EXC_RETURN`, usually a context-switch bug
`FORCED` only, CFSR zero	A fault inside a fault handler — check that your handler does nothing that can fault

The imprecise fault, and how to corner it

```
/* IMPRECISERR is the frustrating one: the address is gone
   and the stacked PC points several instructions past the
   real culprit. Three techniques, in order of cost:       */

/* 1. Disable the write buffer. Every fault becomes precise.
      Costs perhaps 10-20% performance - fine for a debug
      build.                                               */
SCnSCB->ACTLR |= SCnSCB_ACTLR_DISDEFWBUF_Msk;

/* 2. Bisect with barriers. Put __DSB() after groups of
      suspect writes; the fault becomes precise at whichever
      barrier follows the bad one.                          */

/* 3. Use the DWT watchpoint unit to trap writes to a
      specific address, which catches the corruptor rather
      than the victim - covered in chapter 16.              */

/* The single most common source: writing to a peripheral
   whose clock you have not enabled. The write is posted,
   the bus rejects it later, and the fault arrives with no
   address. If you see IMPRECISERR in code that just
   initialised a peripheral, check the RCC enable and the
   read-back (chapter 09) before anything else.            */
```

How it shows up

Symptom	Cause
HardFault with CFSR all zero	Fault inside a handler, or a double fault — check `HFSR`
Always HardFault, never a specific fault	Configurable faults not enabled in `SHCSR`
Stacked PC points at innocent code	Imprecise bus fault — disable write buffering to localise
Fault handler itself faults	It touched a peripheral, used floating point, or overflowed the stack further
Debugger shows the handler but no frame	Read the frame via MSP/PSP as the naked handler above does

Symptom	Cause
Fault only in release builds	Inlining changed stack usage, or exposed undefined behaviour
Fault on the first floating-point operation	`NOCP` — enable the FPU in `CPACR` with a barrier after
Fault after a context switch	`INVPC` — `EXC_RETURN` or the saved frame is corrupt
Unit crashes in the field, works on the bench	Add a `.noinit` crash log and report it after reboot
Crash log is empty after reset	Log placed in `.bss`, so startup zeroed it

Check yourself

- ✓ Name the three configurable faults and say why they are usually disabled.
- ✓ Give the one line that makes future faults far more informative.
- ✓ List three ways a fault escalates to HardFault.
- ✓ Explain how to determine which stack pointer holds the exception frame.
- ✓ Give the layout of the stacked frame and say which field you want most.
- ✓ Give the command that turns a faulting PC into a source line.
- ✓ Distinguish `PRECISERR` from `IMPRECISERR` and say why one loses the address.
- ✓ Name two techniques for localising an imprecise fault.
- ✓ Say what `MSTKERR` indicates and what must be configured for it to occur.
- ✓ Explain how to make a crash log survive the reset that follows it.

Debugging bare metal: SWD, breakpoints and semihosting

There is no `gdb` attaching to a process here — there is a two-wire connection to the silicon itself, and a debug unit built into the core. This chapter covers what that unit can do, and the techniques that work when stopping the CPU is not an option.

SWD and what is behind it

```
Two wires plus ground. SWCLK and SWDIO.

HOST                    TARGET
+--------+  SWCLK   +---------------------------------+
| probe  |--------->--| SW-DP  (debug port)            |
| ST-Link|  SWDIO   |    |                            |
| J-Link |<-------->--|    +-- AHB-AP (memory access)  |
| CMSIS  |          |    |      |                     |
| -DAP   |          |    |      +--> the whole bus:   |
+--------+          |    |           RAM, flash,      |
                    |    |           peripherals      |
                    |    |                            |
                    |    +-- core debug registers:    |
                    |         halt, step, breakpoints |
                    |                                 |
                    |    FPB   6 hardware breakpoints |
                    |    DWT   4 watchpoints, cycle   |
                    |          counter, profiling     |
                    |    ITM   instrumentation trace  |
                   +---------------------------------+

The key property: the AHB-AP is a BUS MASTER. The probe
can read and write any address WITHOUT halting the core.
That is what makes live variable watching possible, and
it is the basis of the best debugging techniques here.
```

Breakpoints and watchpoints are finite

```
/* HARDWARE BREAKPOINTS: the FPB unit compares the fetch
   address against a small set of registers. Cortex-M3/M4
   give you SIX. Cortex-M0 gives you FOUR.

   They work in flash, which software breakpoints cannot -
   a software breakpoint replaces the instruction with BKPT,
   and you cannot rewrite flash cheaply. So on an MCU the
   limit is hard: your seventh breakpoint silently does not
   work, or your debugger refuses.

   WATCHPOINTS: the DWT unit, FOUR comparators, trigger on a
   data ADDRESS being read or written. These are the most
   under-used feature in embedded debugging.               */

/* THE CANONICAL USE: "this variable is being corrupted and
   I have no idea by what."                                */

(gdb) watch g_sensor_state
(gdb) continue
/* Hardware Watchpoint 2: g_sensor_state
   Old value = 3
   New value = 51966
   memcpy () at string.c:71                                */

/* Three seconds instead of three days. A watchpoint answers
   "who wrote here" directly, and it is the correct first
   move for any corruption bug rather than the last resort
   it usually is.

   You can also set one from code, which is useful when the
   corruption happens before the debugger connects:        */
DWT->COMP0     = (uint32_t) &g_sensor_state;
DWT->MASK0     = 0;
DWT->FUNCTION0 = (6u << 0);   /* watch for writes */
```

> **HALTING CHANGES THE SYSTEM**
> Stopping the core stops your code, but not the world. A motor keeps spinning, a PWM output freezes at whatever level it held, a UART peer times out, a watchdog fires, and any peripheral with a FIFO overflows. For anything with a control loop or a protocol partner, halting produces a state that could never occur at run time — and you then debug the artefact rather than the bug. This is why the non-stop techniques below matter more on an MCU than on a workstation.

Getting output, cheapest first

```
/* 1. A GPIO PIN AND A SCOPE.  Cost: ~2 cycles.
      The fastest instrumentation that exists. Toggle a pin
      at the top and bottom of an ISR and you can MEASURE
      its duration and jitter directly - something no
      software tool gives you.                              */
#define PROBE_HI()  (GPIOB->BSRR = (1u << 0))
#define PROBE_LO()  (GPIOB->BSRR = (1u << 16))

void TIM2_IRQHandler(void) {
    PROBE_HI();
    /* ... work ... */
    PROBE_LO();
}
/* The scope now shows execution time, frequency, and
   whether two ISRs ever overlap. For real-time work this
   beats every other technique.                             */

/* 2. ITM / SWO.  Cost: ~5 cycles per word.
      A dedicated trace pin. Non-blocking: writes go to a
      FIFO and are dropped if it is full, so your timing is
      barely perturbed.                                     */
static inline void itm_putc(char c) {
    if ((ITM->TCR & 1) && (ITM->TER & 1)) {
        while (ITM->PORT[0].u32 == 0) { }
        ITM->PORT[0].u8 = (uint8_t) c;
    }
}
/* SWO needs the trace clock configured to match the probe's
   expectation - the usual reason "SWO produces garbage".   */

/* 3. RTT (Segger's Real Time Transfer).  Cost: ~1 us.
      A ring buffer in RAM that the probe reads over SWD
      WITHOUT halting - using the AHB-AP's bus mastering.
      No extra pin, works on any part with SWD, and is
      roughly a thousand times faster than semihosting.
      For most projects this is the right answer.           */

/* 4. UART printf.  Cost: 87 us per character at 115200.
      Blocking, and slow enough to change timing profoundly.
      A printf in an ISR that runs at 10 kHz is an instant
      system failure. Use DMA if you must have it.          */

/* 5. SEMIHOSTING.  Cost: MILLISECONDS per call.
      The core executes BKPT 0xAB and the DEBUGGER performs
      the I/O. Convenient, needs no hardware, and:

        - HALTS THE CPU for every character
        - HANGS THE CHIP if no debugger is attached, because
          BKPT with no debugger escalates to HardFault

      That second point ships products that work on the
      bench and brick in the field. Use semihosting for
      early bring-up only, and make sure it cannot survive
      into a release build.                                 */
```

Measuring time exactly

```
/* The DWT cycle counter is a free, exact, 32-bit tally of
   core cycles. It is the correct tool for micro-benchmarks
   and for proving a deadline is met.                      */

static void cycle_counter_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->LAR   = 0xC5ACCE55;      /* unlock on some parts   */
    DWT->CYCCNT = 0;
    DWT->CTRL  |= DWT_CTRL_CYCCNTENA_Msk;
}

uint32_t t0 = DWT->CYCCNT;
do_the_thing();
uint32_t elapsed = DWT->CYCCNT - t0;   /* wrap-safe subtraction */

/* At 168 MHz one cycle is 5.95 ns, and the counter wraps
   every 25.6 seconds. The subtraction above is correct
   across the wrap for any interval shorter than that.

   Use it to record a WORST CASE rather than an average -
   the maximum is what a deadline cares about:             */
static uint32_t worst;
if (elapsed > worst) { worst = elapsed; }

/* This turns "the ISR is fast enough" from an opinion into
   a number you can assert on. Combined with the priority
   analysis of chapter 08, it is most of what real-time
   verification amounts to on a small system.              */
```

When there is no debugger at all

```
   The situations that actually matter in production:

   FIELD FAILURE          -> .noinit crash log, reported on
                             the next boot (chapter 15)

   TIMING-DEPENDENT BUG   -> GPIO probe + scope; halting
                             would destroy the evidence

   HAPPENS ONCE A WEEK    -> RTT ring buffer left running,
                             read whenever it triggers

   DEBUG PORT DISABLED    -> a fault log over whatever
   (security)                interface the product already
                             has, gated behind
                             authentication

   BOOTS AND DIES BEFORE  -> toggle a pin at successive
   THE PROBE CONNECTS        points in startup; the last
                             one that toggles brackets it.
                             Crude, and it always works.

   The common thread: design the observability in before you
   need it. A product with a crash log, a probe pin and an
   RTT buffer is debuggable in the field. One without them
   requires the unit back on your desk - and it will then
   fail to reproduce.
```

How it shows up

Symptom	Cause
Seventh breakpoint does not trigger	Only six hardware comparators exist; flash cannot hold software breakpoints
Chip hangs when run without a debugger	Semihosting left enabled — `BKPT` escalates to HardFault
Bug disappears while single-stepping	Halting changed the timing; use a GPIO probe instead
Motor runs away when you hit a breakpoint	Peripherals do not halt with the core — configure debug-freeze bits
Watchdog resets during a debug session	Enable the debug freeze for the watchdog in `DBGMCU`
SWO output is garbage	Trace clock frequency does not match the probe's setting
Adding `printf` makes the bug vanish	UART output is slow enough to change the timing you are chasing
Cannot connect after flashing new firmware	Code reconfigured or disabled the SWD pins — use connect-under-reset
Cycle counter always reads zero	`TRCENA` not set, or the DWT lock not written
Corruption source unknown after days of work	A watchpoint would have named it in seconds — use it first

Check yourself

- ✓ Explain what the AHB-AP allows a probe to do without halting the core.
- ✓ State how many hardware breakpoints and watchpoints a typical Cortex-M4 provides.
- ✓ Explain why software breakpoints are not an option in flash.
- ✓ Give the first debugging move for a variable being corrupted by unknown code.
- ✓ List three things that go wrong in the system when you halt the core.
- ✓ Rank GPIO, ITM, RTT, UART and semihosting by cost per output.
- ✓ Explain why semihosting can brick a device in the field.
- ✓ Show how to measure a function's worst-case duration in cycles.
- ✓ Explain why the DWT counter subtraction is safe across its wrap.
- ✓ Name three observability features worth designing in before you need them.

Why you need a scheduler: the superloop and its limits

Most embedded products ship with no RTOS at all, and that is often the right decision. This chapter covers the architectures between "one loop" and "preemptive kernel", what each costs, and the specific symptom that tells you it is time to move up.

The superloop

```
/* The architecture behind more shipped firmware than every
   RTOS combined.                                            */

int main(void)
{
    clock_init();
    gpio_init();
    uart_init();

    for (;;) {
        poll_sensors();
        update_control();
        service_comms();
        update_display();
    }
}

/* WHAT IS GOOD ABOUT IT

   - No concurrency. Nothing preempts anything, so there are
     no races between the four functions and no locks. Every
     data-sharing bug you have ever read about is absent by
     construction.
   - Trivially analysable. The worst-case loop time is the
     sum of four numbers.
   - No kernel: no scheduler RAM, no per-task stacks, no
     kernel code. On a part with 8 kB of RAM this is not a
     preference, it is the only option.
   - Nothing to learn or misconfigure.

   WHAT IS BAD ABOUT IT

   - LATENCY IS THE WHOLE LOOP. If update_display() takes
     40 ms, the control loop cannot respond faster than that,
     no matter how urgent.
   - One slow function degrades everything.
   - Any blocking call is fatal to responsiveness.        */
```

THE RULE THAT MAKES A SUPERLOOP WORK

Every function in the loop must return quickly and never block. That single discipline is what keeps the architecture viable, and it is why superloop code is written as state machines rather than sequences. A function that waits — for a UART byte, an I2C transfer, a sensor conversion — has stolen the whole loop's responsiveness. The moment you cannot express something without waiting, you have found the boundary of the architecture.

Non-blocking state machines

```
/* THE BLOCKING VERSION - 210 ms of the loop, unavailable
   to everything else.                                      */
void read_sensor_blocking(void)
{
    i2c_write(ADDR, CMD_START);
    delay_ms(200);                    /* conversion time    */
    value = i2c_read16(ADDR, REG_DATA);
}

/* THE STATE MACHINE - returns in microseconds, every time. */
typedef enum { S_IDLE, S_WAIT, S_READ } sensor_state_t;

void read_sensor_step(void)
{
    static sensor_state_t st = S_IDLE;
    static uint32_t       t0;

    switch (st) {
    case S_IDLE:
        i2c_write(ADDR, CMD_START);
        t0 = g_ms;
        st = S_WAIT;
        break;

    case S_WAIT:
        if ((int32_t)(g_ms - (t0 + 200)) >= 0) {   /* wrap-safe */
            st = S_READ;
        }
        break;                        /* returns immediately */

    case S_READ:
        value = i2c_read16(ADDR, REG_DATA);
        st = S_IDLE;
        break;
    }
}

/* This is the central skill of superloop design, and it is
   also its central cost: the CONTROL FLOW IS INSIDE OUT.
   The sequence "start, wait, read" is obvious in the
   blocking version and scattered across a switch in the
   non-blocking one. With three such sequences interleaved,
   each with error handling and timeouts, the state count
   multiplies and the code becomes genuinely hard to follow.

   THAT is the real argument for an RTOS. Not performance -
   readability. An RTOS lets you write the blocking version
   and have it be correct, because the kernel switches away
   from a blocked task instead of the whole system waiting. */
```

The spectrum

```
1. SUPERLOOP
   for(;;) { a(); b(); c(); }
   Latency: whole loop.  RAM: one stack.
   Use when: every function is short, few deadlines.

2. SUPERLOOP + INTERRUPTS  <- most shipped firmware
   ISRs handle urgent work; the loop handles the rest;
   they communicate through flags and ring buffers.
   Latency: microseconds for ISR work, whole loop for
            the rest. RAM: one stack + ISR frames.
   Use when: the urgent work is SHORT. This covers a
            remarkable proportion of real products.

3. COOPERATIVE SCHEDULER
   Tasks yield explicitly; the scheduler picks the next.
   No preemption, so still no data races between tasks.
   Latency: longest run between yields.
   RAM: one stack per task.
   Use when: you want task structure but cannot afford
            preemption's synchronisation burden.

4. PREEMPTIVE RTOS
   The kernel switches on a tick or an event; a
   high-priority task runs as soon as it is ready.
   Latency: bounded by the kernel, typically a few us.
   RAM: one stack per task + kernel overhead (2-10 kB).
   Cost: RACES ARE NOW POSSIBLE. Every shared variable
         needs a mutex, a queue, or an atomic.
   Use when: multiple independent deadlines, or blocking
            sequences you want written as sequences.

Moving from 2 to 4 buys bounded latency and pays for it
in synchronisation bugs. That is a real trade and the
step should be deliberate.
```

The pattern that delays the decision

```
/* Most "we need an RTOS" moments are actually "we need to
   get work out of the ISR". Deferred processing solves that
   without a kernel.                                        */

static volatile uint8_t  ring[256];
static volatile uint8_t  head, tail;   /* single producer,  */
                                       /* single consumer   */

void USART1_IRQHandler(void)           /* PRODUCER: ISR     */
{
    uint8_t next = (uint8_t)(head + 1);
    if (next != tail) {                /* drop if full      */
        ring[head] = (uint8_t) USART1->RDR;
        head = next;                   /* publish LAST      */
    }
}

void service_comms(void)               /* CONSUMER: loop    */
{
    while (tail != head) {
        parse_byte(ring[tail]);
        tail = (uint8_t)(tail + 1);
    }
}

/* WHY THIS IS SAFE WITHOUT A LOCK:
   - exactly one writer of head (the ISR), one writer of
     tail (the loop);
   - both are single bytes, so reads and writes are atomic
     on Cortex-M;
   - head is published AFTER the data is written, so the
     consumer never sees an index for a byte not yet
     stored. On Cortex-M's ordering this holds; on a weakly
     ordered core you would need a release barrier here,
     which is exactly what the ARM track's barriers chapter
     is about.
   - the size is a power of two, so the wrap is free and
     the index arithmetic cannot go wrong.

   THE ISR IS NOW ~10 CYCLES. All the parsing happens in the
   loop, at whatever rate it manages, and a burst is
   absorbed by the buffer. This single pattern removes the
   need for an RTOS in a large fraction of designs.        */
```

When to actually move up

Symptom	Fix
One function occasionally takes 50 ms and everything stutters	Split it into a state machine first; an RTOS only hides the problem
ISR does too much work	Deferred processing with a ring buffer — no kernel needed
Two independent deadlines that cannot both be met by one loop	A preemptive RTOS. This is the real reason
Sequential protocols with many wait states become unreadable	A preemptive RTOS, for readability
A third-party stack (TCP/IP, BLE, USB) demands one	Use the one it expects — fighting it costs more than it saves

Symptom	Fix
You want "threads" because it feels more professional	Not a reason. The superloop is a legitimate architecture
RAM is under 8 kB	You cannot afford a preemptive kernel; stay with 1 or 2
Safety certification is required	Either — but the superloop is far easier to argue for, which is why safety-critical code often uses it

AN RTOS DOES NOT MAKE A SLOW SYSTEM FAST

If your control loop misses its deadline because the computation takes too long, adding a scheduler adds overhead and changes nothing. An RTOS reallocates *latency*, letting urgent work preempt unimportant work. It cannot create CPU time. Diagnose which problem you have — measure with the cycle counter from chapter 16 — before assuming a kernel is the answer.

How it shows up

Symptom	Cause
Response time equals the slowest function	Inherent to a superloop — that is the architecture
Display update makes the control loop jitter	A long non-blocking-but-slow function; split it
Serial bytes lost during a long operation	No ring buffer; the ISR had nowhere to put them
Adding a feature broke unrelated timing	Loop time grew; every deadline is coupled to every function
Code became unreadable after three state machines	The genuine signal to consider an RTOS
Races appeared right after adopting an RTOS	Expected — preemption is what you bought; add proper synchronisation
RTOS added and the system got slower	The problem was throughput, not latency
Kernel does not fit in RAM	Below roughly 8 kB a preemptive kernel is not viable
Ring buffer occasionally loses a byte	Index published before the data was written, or a size that is not a power of two
Two producers into one ring buffer corrupt it	The lock-free argument holds for one producer only

Check yourself

- ✓ State the one discipline that makes a superloop viable.
- ✓ Give two genuine advantages of a superloop over an RTOS.
- ✓ Convert a blocking sensor read into a non-blocking state machine.
- ✓ Explain the real cost of state machines, and why it argues for an RTOS.
- ✓ Name the four architectures on the spectrum and the latency of each.
- ✓ Explain why the single-producer ring buffer needs no lock on Cortex-M.
- ✓ Say why the head index must be published after the data.
- ✓ Give the one symptom that genuinely calls for a preemptive kernel.
- ✓ Explain why an RTOS cannot fix a throughput problem.
- ✓ Say what a preemptive kernel costs you that a superloop does not.

The context switch, PendSV and the scheduler core

Twenty instructions separate "a program" from "an operating system". This chapter builds the switch itself: why it must run in PendSV, why PendSV must be the lowest priority in the system, and what the hardware does for you at each end.

The whole idea in one picture

```
A "context" is just a stack with registers saved on it.
Switching tasks means changing which stack PSP points at.

TASK A's STACK              TASK B's STACK
+--------------+            +--------------+
|   xPSR       | \          |   xPSR       | \
|   PC         |  |         |   PC         |  |  hardware
|   LR         |  | pushed  |   LR         |  |  pushes
|   R12        |  | by HW   |   R12        |  |  and pops
|   R3         |  |         |   R3         |  |  these 8
|   R2         |  |         |   R2         |  |
|   R1         |  |         |   R1         |  |
|   R0         | /          |   R0         | /
+--------------+            +--------------+
|   R11        | \          |   R11        | \
|   R10        |  |         |   R10        |  |  YOU push
|   R9         |  | pushed  |   R9         |  |  and pop
|   R8         |  | by YOU  |   R8         |  |  these 8
|   R7         |  |         |   R7         |  |
|   R6         |  |         |   R6         |  |
|   R5         |  |         |   R5         |  |
|   R4         | /          |   R4         | /
+--------------+ <- saved    +--------------+ <- saved
     PSP_A         in TCB        PSP_B         in TCB

THE SWITCH:
  1. hardware pushed 8 registers onto A's stack already
  2. push R4-R11 onto A's stack
  3. save PSP into A's TCB
  4. pick the next task
  5. load PSP from B's TCB
  6. pop R4-R11 from B's stack
  7. BX to EXC_RETURN - hardware pops B's 8 registers

Step 7 returns to whatever B was doing, because B's PC is
in the frame the hardware just popped. There is no jump
to task B; you simply return from an interrupt into it.
```

Why PendSV

```
PendSV is an exception whose ONLY purpose is deferring
work to the lowest priority. Set its priority to 0xFF -
the lowest possible - and it runs only when no other
exception is active.

WITHOUT PendSV, switching directly in SysTick:

  SysTick fires -> switch to task B -> return
  But an interrupt was PREEMPTED by SysTick and is still
  ACTIVE. Returning into task B while that interrupt's
  frame is on the stack corrupts the exception nesting,
  and the core takes a UsageFault (INVPC) or worse.

WITH PendSV:

time -->
  UART IRQ  |====================|
  SysTick        |==|  (sets PendSV pending, returns)
  PendSV                         |=====|  switch here
  task B                               |=========

  Every other exception drains first. PendSV runs last,
  with a clean nesting state, and the switch is safe.

THE RULE: PendSV must be the LOWEST priority exception in
the system. If anything is lower, it can be preempted by
the switch and the same corruption returns.

  NVIC_SetPriority(PendSV_IRQn, 0xFF);
  NVIC_SetPriority(SysTick_IRQn, 0xFE);

This is why every Cortex-M RTOS port uses PendSV. It is
not a convention; it is the only correct place.
```

The switch, in full

```
/* This is a complete, working PendSV handler for ARMv7-M.
   'naked' because we must control every instruction - a
   compiler prologue would corrupt the frame we are building. */

__attribute__((naked)) void PendSV_Handler(void)
{
    __asm volatile (
    /* --- SAVE the outgoing task ------------------------- */
    "   mrs   r0, psp              \n"  /* current task SP   */
    "   cbz   r0, restore          \n"  /* first switch: no  */
                                        /* task to save      */
    "   subs  r0, r0, #32          \n"  /* room for R4-R11   */
    "   stm   r0, {r4-r11}         \n"

    "   ldr   r1, =current_tcb     \n"
    "   ldr   r2, [r1]             \n"  /* r2 = current TCB  */
    "   str   r0, [r2]             \n"  /* TCB->sp = r0      */
                                        /* sp is at offset 0 */

    /* --- CHOOSE the next task --------------------------- */
    "restore:                      \n"
    "   push  {lr}                 \n"  /* EXC_RETURN        */
    "   bl    scheduler_next       \n"  /* C function        */
    "   pop   {lr}                 \n"

    /* --- RESTORE the incoming task ---------------------- */
    "   ldr   r1, =current_tcb     \n"
    "   ldr   r2, [r1]             \n"
    "   ldr   r0, [r2]             \n"  /* r0 = TCB->sp      */
    "   ldm   r0, {r4-r11}         \n"
    "   adds  r0, r0, #32          \n"
    "   msr   psp, r0              \n"

    "   isb                        \n"
    "   bx    lr                   \n"  /* EXC_RETURN ->     */
                                        /* hardware unstacks */
    );
}

/* Points worth dwelling on:

   - scheduler_next() is ordinary C. It reads the ready list
     and writes current_tcb. All the policy lives there; the
     assembly is pure mechanism.

   - LR must be preserved across the call, hence push/pop.
     It holds EXC_RETURN, and losing it means you cannot
     return from the exception at all.

   - 'sp' is deliberately the FIRST member of the TCB, so
     the offset is zero and the assembly needs no knowledge
     of the struct layout. Every RTOS does this.

   - This code does NOT save the FP registers. On a part
     with an FPU where two tasks use float, add:

         tst   lr, #0x10          @ EXC_RETURN bit 4
         it    eq
         vstmdbeq r0!, {s16-s31}  @ save if FP frame present
```

```
    and the mirror on restore. Omitting it is the classic
    bug that only appears when a second task uses float
    (chapter 02).                                          */
```

Manufacturing the first stack frame

A task that has never run has no saved context. You fabricate one that looks exactly like a real switch, so the first restore starts it at its entry point.

```
uint32_t *task_stack_init(uint32_t *stack_top,
                          void (*entry)(void *), void *arg)
{
    /* 8-byte alignment is required by AAPCS. */
    uint32_t *sp = (uint32_t *)((uint32_t) stack_top & ~7u);

    /* --- the frame the HARDWARE will pop, in its order --- */
    *(--sp) = 0x01000000u;            /* xPSR: THUMB BIT SET */
    *(--sp) = (uint32_t) entry & ~1u; /* PC                  */
    *(--sp) = (uint32_t) task_exit;   /* LR: where a task    */
                                      /* goes if it returns  */
    *(--sp) = 0x12121212u;            /* R12                 */
    *(--sp) = 0x03030303u;            /* R3                  */
    *(--sp) = 0x02020202u;            /* R2                  */
    *(--sp) = 0x01010101u;            /* R1                  */
    *(--sp) = (uint32_t) arg;         /* R0 = first argument */

    /* --- the registers WE pop ---------------------------- */
    *(--sp) = 0x11111111u;            /* R11                  */
    *(--sp) = 0x10101010u;            /* R10                  */
    *(--sp) = 0x09090909u;            /* R9                   */
    *(--sp) = 0x08080808u;            /* R8                   */
    *(--sp) = 0x07070707u;            /* R7                   */
    *(--sp) = 0x06060606u;            /* R6                   */
    *(--sp) = 0x05050505u;            /* R5                   */
    *(--sp) = 0x04040404u;            /* R4                   */

    return sp;                        /* store in TCB->sp    */
}

/* THREE THINGS THAT MUST BE RIGHT:

   1. xPSR MUST have bit 24 (T) set. Clear it and the core
      faults with INVSTATE the instant the task starts
      (chapter 02). This is the single most common bug when
      writing a scheduler from scratch.

   2. The PC entry must have bit 0 CLEAR here, even though
      function pointers carry it set. The hardware takes the
      T state from xPSR, not from the PC, and a set bit 0 in
      the stacked PC is architecturally unpredictable.

   3. LR should point at a function that handles a task
      returning. A task that falls off the end otherwise
      branches to whatever 0x0E0E0E0E-style filler you put
      there. Make it explicit:                              */

static void task_exit(void)
{
    task_delete(current_tcb);
    for (;;) { }        /* never reached; the switch away
                           happens inside task_delete       */
}

/* The recognisable filler values are a debugging gift: in a
   fault dump, seeing 0x07070707 in R7 tells you instantly
   that this task has never actually run.                   */
```

Starting the scheduler

```
/* Getting from main() - running on MSP, privileged - into
   the first task on PSP is a one-time bootstrap.          */

void scheduler_start(void)
{
    NVIC_SetPriority(PendSV_IRQn,  0xFF);   /* lowest       */
    NVIC_SetPriority(SysTick_IRQn, 0xFE);

    current_tcb = scheduler_highest_ready();

    /* Point PSP at the first task's frame, skipping the 8
       registers the hardware will pop for us.              */
    __set_PSP((uint32_t) current_tcb->sp + 32);

    /* Switch Thread mode to PSP. */
    __set_CONTROL(0x02);
    __ISB();                    /* MANDATORY (chapter 03)   */

    systick_init(1000);         /* 1 kHz tick               */

    /* Jump to the task entry. From here the only way back
       into main() is a bug.                                */
    void (*entry)(void *) = current_tcb->entry;
    entry(current_tcb->arg);
}

/* THE TICK. SysTick does no switching itself - it accounts
   for time and, if that changed who should run, requests a
   PendSV. That request is a single register write:        */

void SysTick_Handler(void)
{
    g_ticks++;
    if (scheduler_tick()) {                 /* returns 1 if
                                               a switch is
                                               needed      */
        SCB->ICSR = SCB_ICSR_PENDSVSET_Msk;  /* request it  */
    }
}

/* Any code, including any ISR, can request a switch with
   that one store. A UART handler that unblocks a waiting
   task does exactly this, which is how a task becomes
   runnable "immediately" rather than at the next tick.    */
```

> **MECHANISM AND POLICY ARE CLEANLY SEPARATED**
>
> The assembly above is the entire *mechanism*, and it never changes. Round-robin, fixed-priority preemptive, rate-monotonic, or EDF are all just different implementations of `scheduler_next()` — ordinary C, testable on a host, with no assembly involved. When you read FreeRTOS or Zephyr, this is the seam to look for first: `port.c` holds the twenty instructions, and everything else is policy on top.

How it shows up

Symptom	Cause
Fault the instant the first task starts	T bit not set in the fabricated xPSR
UsageFault with `INVPC`	Corrupted `EXC_RETURN` — usually LR not preserved across the scheduler call
Fault when switching from inside an ISR	Switching somewhere other than PendSV
Rare corruption under interrupt load	PendSV is not the lowest priority exception
Variables corrupt across switches	R4–R11 not saved, or the wrong number of words adjusted
Corruption only when two tasks use float	FP registers not saved; test `EXC_RETURN` bit 4
Task runs once then the system dies	Task returned; LR in the fabricated frame pointed at nothing valid
First switch faults, later ones work	No null check for "no outgoing task" on the very first entry
Scheduler never runs after `CONTROL` write	Missing `ISB`
Registers full of `0x07070707` in a dump	The task never ran — your filler values telling you exactly that

Check yourself

- ✓ State which eight registers the hardware saves and which eight you must save.
- ✓ Explain why the switch must happen in PendSV rather than SysTick.
- ✓ Say what priority PendSV must have and what breaks otherwise.
- ✓ Explain why returning with `EXC_RETURN` starts the new task.
- ✓ Explain why `sp` is the first member of the TCB.
- ✓ List the three things that must be right in a fabricated first frame.
- ✓ Say why the stacked PC must have bit 0 clear while function pointers have it set.
- ✓ Explain why LR must be pushed around the call to the scheduler.
- ✓ Give the single store that requests a context switch from any ISR.
- ✓ Explain the mechanism/policy split and where to find it in a real RTOS.

RTOS primitives: queues, semaphores and ISR-safe APIs

The concurrency track covers mutexes and condition variables in general. What is different on a microcontroller is that half your concurrency is with interrupts rather than tasks — and an interrupt cannot block, which changes every API it touches.

Why ISRs need their own API

```
A TASK that cannot proceed BLOCKS: the kernel removes it
from the ready list and switches to someone else. It
resumes later, where it left off.

AN ISR CANNOT DO THIS. There is nothing to switch to -
an interrupt is not a schedulable entity, it has no TCB
and no stack of its own to preserve. Blocking in an ISR
would suspend whatever task happened to be interrupted,
which is not the ISR and has done nothing wrong.

So every kernel splits its API:

  xQueueSend(q, &item, portMAX_DELAY)
     may block. TASK ONLY.

  xQueueSendFromISR(q, &item, &woken)
     never blocks; returns pdFAIL if full. ISR ONLY.

And the FromISR variants have an extra job: reporting
whether they made a higher-priority task ready, so the
ISR can request a context switch on the way out.

  void UART_IRQHandler(void)
  {
      BaseType_t woken = pdFALSE;
      xQueueSendFromISR(q, &byte, &woken);
      portYIELD_FROM_ISR(woken);   /* sets PendSV if so  */
  }

Omit that last line and the woken task waits until the
next tick - up to a full millisecond of latency, for no
reason, and only under some timings. It is a classic
intermittent-latency bug.
```

THE PRIORITY CEILING FOR ISR CALLS

An ISR may only call kernel APIs if its priority is at or below `configMAX_SYSCALL_INTERRUPT_PRIORITY` (chapter 08). Above that line the kernel's `BASEPRI` critical sections do not block it, so it can interrupt the kernel mid-update and corrupt a list. The damage appears much later, in unrelated code. Keep `configASSERT` enabled in development — FreeRTOS checks this on every ISR API call and catches the mistake immediately rather than three weeks later.

The queue is the primitive you want

```
/* A queue copies data and handles the blocking, the
   waking, and the mutual exclusion in one object. Prefer
   it over a shared buffer plus a semaphore almost always. */

QueueHandle_t sensor_q;

void app_init(void)
{
    sensor_q = xQueueCreate(10, sizeof(sensor_reading_t));
}

/* PRODUCER - an ISR. */
void ADC_IRQHandler(void)
{
    BaseType_t woken = pdFALSE;
    sensor_reading_t r = { .ts = g_ticks, .value = ADC1->DR };

    /* Never blocks. If the queue is full the reading is
       dropped, and that is a DESIGN DECISION you should
       make explicitly - count the drops.                  */
    if (xQueueSendFromISR(sensor_q, &r, &woken) != pdTRUE) {
        g_dropped++;
    }
    portYIELD_FROM_ISR(woken);
}

/* CONSUMER - a task. */
void sensor_task(void *arg)
{
    sensor_reading_t r;
    for (;;) {
        if (xQueueReceive(sensor_q, &r, pdMS_TO_TICKS(100))) {
            process(&r);
        } else {
            handle_sensor_timeout();     /* the timeout is  */
        }                                /* not optional    */
    }
}

/* WHY A QUEUE BEATS A SHARED BUFFER:
   - it COPIES, so there is no lifetime question about the
     producer's data;
   - the blocking receive means the consumer uses zero CPU
     while waiting, which is what lets the system sleep;
   - full and empty are handled once, correctly, rather
     than in every caller;
   - a timeout is available, so a stuck producer becomes a
     reported error rather than a hang.

   THE COST IS THE COPY. For large items, queue a POINTER
   to a buffer from a pool instead - but then you own the
   lifetime again, and the pool needs its own discipline. */
```

Semaphores, mutexes, and the difference that matters

```
/* BINARY SEMAPHORE - signalling. No owner.
   Use for: ISR says "something happened" to a task.        */
SemaphoreHandle_t data_ready;

void DMA1_IRQHandler(void) {
    BaseType_t woken = pdFALSE;
    xSemaphoreGiveFromISR(data_ready, &woken);
    portYIELD_FROM_ISR(woken);
}
void consumer(void *a) {
    for (;;) {
        xSemaphoreTake(data_ready, portMAX_DELAY);
        process_buffer();
    }
}

/* COUNTING SEMAPHORE - resource pools, or counting events
   that may arrive faster than they are consumed.          */
SemaphoreHandle_t slots = xSemaphoreCreateCounting(5, 5);

/* MUTEX - mutual exclusion. HAS AN OWNER, and that is the
   whole difference.                                       */
SemaphoreHandle_t spi_lock = xSemaphoreCreateMutex();

void use_spi(void) {
    xSemaphoreTake(spi_lock, portMAX_DELAY);
    spi_transfer(...);
    xSemaphoreGive(spi_lock);
}

/* BECAUSE A MUTEX HAS AN OWNER IT CAN DO TWO THINGS A
   SEMAPHORE CANNOT:

   1. PRIORITY INHERITANCE. While a low-priority task holds
      a mutex a high-priority task wants, the holder is
      temporarily raised to the waiter's priority. This
      bounds priority inversion - the failure that reset
      Mars Pathfinder (see the concurrency track, chapter
      15). A binary semaphore has no owner, so it cannot
      do this, and using one for mutual exclusion silently
      gives up the protection.

   2. RECURSION. xSemaphoreCreateRecursiveMutex lets the
      owner take it again without deadlocking.

   THE RULE:
     mutual exclusion -> MUTEX, always.
     signalling       -> BINARY SEMAPHORE.
   Using a semaphore as a lock is the single most common
   RTOS design error, and it works fine until the day a
   priority inversion costs you a deadline.

   NEVER take a mutex from an ISR. There is no owner to
   record, no priority to inherit, and no way to block.
   There is deliberately no xSemaphoreTakeFromISR.         */
```

Task notifications: the cheap path

```
/* A direct-to-task signal with no separate kernel object.
   Roughly 45x faster than a semaphore and it costs no RAM
   beyond a word already in the TCB.                        */

void DMA1_IRQHandler(void)
{
    BaseType_t woken = pdFALSE;
    vTaskNotifyGiveFromISR(dma_task_handle, &woken);
    portYIELD_FROM_ISR(woken);
}

void dma_task(void *arg)
{
    for (;;) {
        /* pdTRUE = clear the count on exit, i.e. act as a
           binary semaphore.                                 */
        if (ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(500))) {
            process_buffer();
        } else {
            dma_timeout_recovery();
        }
    }
}

/* THE LIMITATION that decides whether you can use it: a
   notification targets ONE SPECIFIC TASK. There is no
   broadcast and no queueing of senders. If two tasks might
   wait for the same event, or the sender does not know the
   receiver, use a semaphore or queue.

   For the extremely common ISR-to-one-task pattern -
   which is most of embedded signalling - notifications are
   strictly better and under-used.                          */
```

Choosing

Need	Use	Not
ISR hands data to one task	Queue, or notification if the data fits in a word	A global plus a flag
ISR signals one known task	Task notification	Semaphore — slower, more RAM
ISR signals whichever task is waiting	Binary semaphore	Notification — it needs a specific target
Protect a shared peripheral	Mutex	Binary semaphore — no priority inheritance
Limit concurrent users of a pool	Counting semaphore	A hand-rolled counter
Wait for several conditions	Event group	Several semaphores and polling
Producer/consumer of large buffers	Queue of pointers plus a pool	Queue of the buffers themselves — the copy cost
Mutual exclusion inside an ISR	Disable interrupts briefly	Any kernel object — ISRs cannot block

How it shows up

Symptom	Cause
Latency sometimes a full tick longer than expected	`portYIELD_FROM_ISR` omitted after a `FromISR` call
Kernel lists corrupt after hours of running	An ISR above the syscall priority called a kernel API
`configASSERT` fires on an ISR API call	Exactly that, caught early — fix the priority
System hangs when a task holds a lock	Blocking API called from an ISR, or a mutex taken recursively without the recursive variant
A high-priority task misses deadlines under load	Binary semaphore used for mutual exclusion — no priority inheritance
Queue sends fail silently under burst load	Queue full and the return value ignored — count the drops
Task waits forever	`portMAX_DELAY` with no timeout path; a lost signal is unrecoverable
Data corrupt when queueing pointers	Producer freed or reused the buffer before the consumer finished
Notification lost when two tasks wait	Notifications target one task only
Everything works until a third task is added	An implicit assumption about who runs when — usually a missing lock

Check yourself

- ✓ Explain why an ISR cannot block, in terms of what blocking actually does.
- ✓ Say what the extra out-parameter of a `FromISR` call is for.
- ✓ Describe the bug caused by omitting `portYIELD_FROM_ISR`.
- ✓ State the priority rule for ISRs that call kernel APIs, and why.
- ✓ Give three reasons a queue beats a shared buffer plus a flag.
- ✓ State the two capabilities a mutex has that a binary semaphore lacks.
- ✓ Explain why there is no `xSemaphoreTakeFromISR`.
- ✓ Say when a task notification is the right choice and when it is not.
- ✓ Explain the risk of queueing pointers rather than data.
- ✓ Give the correct mutual-exclusion mechanism inside an ISR.

Stack sizing, priority assignment and inversion

Two numbers per task decide whether a system is reliable: how much stack it gets and what priority it runs at. Both are usually guessed. This chapter replaces the guessing with measurement for one and a theorem for the other.

Why stack sizing is hard here

```
ON LINUX                        ON AN MCU

8 MB default stack              512 bytes to 4 kB
Guard page: overflow            Nothing. The stack grows
traps immediately               into the next task's data
Grows on demand                 Fixed at creation
One bad recursion = crash       One bad recursion = a
                                variable changes somewhere
                                else, hours later

WHAT CONSUMES A TASK'S STACK:

  local variables in every frame of the deepest call path
+ saved registers at each call (up to 32 bytes)
+ compiler temporaries and spills
+ ONE EXCEPTION FRAME if an interrupt lands while this
  task runs (32 bytes, or 104 with FP state)
+ the nesting depth of interrupts, IF they run on the
  task stack

That last point is why PSP matters (chapter 03): with
tasks on PSP and interrupts on MSP, interrupt nesting is
charged to ONE main stack you size once, rather than to
every task stack separately. Without the split, every
task stack must carry the worst-case interrupt depth.
```

Measuring, three ways

```
/* 1. WATERMARKING - the standard technique. Fill the stack
      with a pattern at creation; later, count how much is
      still intact.                                          */

void stack_fill(uint32_t *base, size_t words)
{
    for (size_t i = 0; i < words; i++) {
        base[i] = 0xA5A5A5A5u;
    }
}

size_t stack_unused_words(const uint32_t *base, size_t words)
{
    size_t i = 0;
    while (i < words && base[i] == 0xA5A5A5A5u) {
        i++;
    }
    return i;      /* words never touched */
}

/* FreeRTOS does this for you:
     uxTaskGetStackHighWaterMark(handle)
   returns the minimum free WORDS ever observed. Print it
   for every task after a long soak test under peak load.

   THE TRAP: this reports the worst case SEEN, not the
   worst case POSSIBLE. If the error path that calls
   snprintf never ran during your test, its 200 bytes are
   not in the number. Exercise error paths deliberately
   before trusting a watermark.                           */

/* 2. STATIC ANALYSIS - the only way to bound the worst
      case rather than sample it.                            */

/*   $ arm-none-eabi-gcc -fstack-usage ...
     produces a .su file per translation unit:

       i2c.c:142:5:i2c_read    48   static
       parse.c:88:1:parse_msg  120  dynamic,bounded

     Combine with the call graph and you get a true maximum.
     Tools: puncover, StackAnalyzer, or a script over the
     .su files plus objdump's call graph.

     "dynamic" entries - alloca, VLAs, recursion - cannot be
     bounded. That is the real reason those constructs are
     banned in embedded coding standards, MISRA included.  */

/* 3. THE MPU GUARD - not a measurement, a safety net.
      A no-access region below each stack (chapter 14)
      converts overflow into an immediate, attributable
      fault. Use it alongside watermarking, not instead:
      watermarking tells you how close you are, the guard
      catches the day you are wrong.                         */
```

A PRACTICAL PROCEDURE

Start every task at 1 kB. Run the system under peak load, including every error path you can provoke, for long enough to see the rare cases. Read the high-water marks. Set each stack to the observed maximum plus 50%, rounded up to a sensible boundary. Add an MPU guard region. Re-check the watermarks in every release. This is not elegant, but it is what reliable products actually do, and it beats both guessing and the false confidence of a single short test run.

Assigning priorities: rate-monotonic

```
/* There IS a right answer for periodic tasks, and it is a
   theorem rather than an opinion.

   RATE-MONOTONIC: the shorter the period, the higher the
   priority. Not "the more important" - the more FREQUENT.

   Liu and Layland proved this is OPTIMAL among fixed-
   priority schemes: if any fixed-priority assignment can
   meet all deadlines, rate-monotonic can.

   SCHEDULABILITY TEST:

     U = sum(Ci / Ti)          C = worst-case compute time
                               T = period

     Schedulable if U <= n(2^(1/n) - 1)

       n=1  100%      n=4  75.7%
       n=2  82.8%     n=5  74.3%
       n=3  78.0%     n->inf  69.3%   (ln 2)

   WORKED EXAMPLE:

     control loop   T=10 ms   C=2 ms   ->  0.200
     sensor read    T=50 ms   C=5 ms   ->  0.100
     comms          T=100 ms  C=20 ms  ->  0.200
     display        T=200 ms  C=30 ms  ->  0.150
                                           -----
                                    U =    0.650

     n=4 bound is 0.757, and 0.650 <= 0.757, so ALL
     deadlines are met with priorities in period order:
     control > sensor > comms > display.

   The bound is SUFFICIENT, not necessary: above it a system
   may still be schedulable, but you must then do exact
   response-time analysis rather than the simple test.

   THIS IS WHY 69% IS A FAMOUS NUMBER. A fixed-priority
   real-time system that must always meet deadlines cannot
   safely be planned above roughly 70% CPU utilisation.
   Designing for 95% is designing for missed deadlines.   */
```

Priority inversion, in miniature

```
   The concurrency track covers this fully (chapter 15).
   The MCU-specific points:

   1. USE A MUTEX, NOT A SEMAPHORE, for mutual exclusion.
      Only a mutex has an owner, so only a mutex can
      implement priority inheritance (chapter 19).

   2. CHECK IT IS ENABLED.
        configUSE_MUTEXES = 1
      Without it xSemaphoreCreateMutex is compiled out and
      you silently get a binary semaphore.

   3. THE ISR IS ALWAYS HIGHEST.
      Any interrupt outranks every task, always. A long ISR
      delays your most urgent task no matter what priority
      you gave it. Interrupt duration is therefore part of
      your schedulability arithmetic, not separate from it:

        U = sum(Ci/Ti) for tasks
          + sum(Ei/Pi) for INTERRUPTS

      A 20 us ISR at 10 kHz is 20% utilisation before any
      task runs. This is routinely forgotten, and it is
      why a system that analyses as 65% utilised misses
      deadlines at 85%.

   4. MEASURE C, DO NOT ESTIMATE IT.
      Use the DWT cycle counter (chapter 16) and record the
      MAXIMUM, not the average. A deadline cares only about
      the worst case, and the worst case is often 5-10x the
      typical one because of a cache miss, a flash wait
      state, or an error path.
```

Rules that survive contact with real systems

Rule	Why
Fewer tasks is better	Each costs a stack, a TCB and a synchronisation surface. Two tasks that always run together should be one
Never `portMAX_DELAY` without a recovery path	A lost signal becomes a permanently blocked task and a silent partial failure
The idle task must actually run	If it never does, you are at 100% CPU and have no margin — and cannot sleep
Keep ISRs to tens of microseconds	They preempt every task; their duration is everyone's jitter
Do not use priority to express importance	Priority expresses urgency. An important but slow task belongs low
Avoid dynamic creation after startup	Fragmentation and unbounded failure modes; create everything up front
One priority per task, no two equal	Equal priorities mean round-robin, which makes response times depend on arrival order
Budget for 70%, not 95%	The rate-monotonic bound — above it, deadlines are not guaranteed

How it shows up

Symptom	Cause
A variable in another task changes spontaneously	Stack overflow — add MPU guards and check watermarks
Crash only when an error path runs	That path's stack usage was never in the watermark
Stack usage cannot be bounded statically	Recursion, `alloca` or a VLA — remove them
Deadlines missed at 85% CPU despite analysis	Interrupt time not included in the utilisation sum
System analyses fine but misses deadlines rarely	Worst-case compute times estimated rather than measured
High-priority task blocked by a low one	Priority inversion — use a mutex and enable inheritance
Mutex behaves like a semaphore	`configUSE_MUTEXES` is 0
A task never runs again	Blocked on `portMAX_DELAY` for a signal that was lost
Response time depends on which task started first	Two tasks at equal priority round-robining
Device cannot enter sleep	The idle task never runs; something polls instead of blocking

Check yourself

- ✓ List five things that consume a task's stack.
- ✓ Explain why running tasks on PSP reduces total stack requirements.
- ✓ Describe watermarking and state its one important limitation.
- ✓ Explain why recursion and VLAs are banned by embedded coding standards.
- ✓ State the rate-monotonic priority rule in one sentence.
- ✓ Give the schedulability bound for four tasks and apply it to an example.
- ✓ Explain why 69% is a significant number for fixed-priority systems.
- ✓ Show how interrupt time enters the utilisation calculation.
- ✓ Explain why you must measure worst-case rather than average compute time.
- ✓ Give three rules for priority assignment and justify each.

FreeRTOS in practice

FreeRTOS is the most widely deployed RTOS in the world, and its whole configuration surface is one header file. This chapter covers the settings that matter, the port layer where chapter 18's assembly actually lives, and the handful of mistakes that account for most FreeRTOS bugs.

The shape of it

```
FreeRTOS-Kernel/
  tasks.c        the scheduler, ready lists, TCBs
  queue.c        queues, semaphores, mutexes - all one
                 implementation underneath
  timers.c       software timers, run by a daemon task
  event_groups.c
  list.c         the intrusive linked list everything uses
  portable/
    GCC/ARM_CM4F/
      port.c     <- chapter 18's PendSV_Handler lives here
      portmacro.h   critical sections, BASEPRI, yields
    MemMang/
      heap_1.c   allocate only, never free
      heap_2.c   free, no coalescing (deprecated)
      heap_3.c   wraps malloc/free
      heap_4.c   free WITH coalescing  <- the usual choice
      heap_5.c   heap_4 across several memory regions

Your project adds exactly one file:
  FreeRTOSConfig.h

Reading tasks.c and port.c is genuinely worthwhile - it
is a few thousand lines of clear C, and after chapter 18
the port layer will look familiar rather than magical.
```

The settings that matter

```
/* FreeRTOSConfig.h - the ones worth understanding. */

#define configCPU_CLOCK_HZ            168000000
#define configTICK_RATE_HZ            1000    /* 1 ms tick   */
#define configMAX_PRIORITIES          8
#define configMINIMAL_STACK_SIZE      128     /* WORDS = 512 B */
#define configTOTAL_HEAP_SIZE         (24 * 1024)

/* ---- THE TWO THAT CAUSE THE MOST TROUBLE ---------------- */

/* Must match the silicon. Read it from the vendor header;
   do not assume 4 (chapter 08).                            */
#define configPRIO_BITS               4

/* The ISR priority ceiling. Interrupts numerically BELOW
   this value (= higher urgency) must NEVER call an API.

   Note this is the RAW value, shifted into the implemented
   bits - not the CMSIS-style logical priority.             */
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY  5
#define configMAX_SYSCALL_INTERRUPT_PRIORITY \
    (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS))

/* ---- KEEP THESE ON IN DEVELOPMENT ----------------------- */

#define configASSERT(x) if ((x) == 0) { taskDISABLE_INTERRUPTS(); \
                                        for (;;); }
/* configASSERT catches: API called from a too-high-priority
   ISR, blocking call from an ISR, invalid priority, and
   several others - each of which is otherwise a bug that
   appears hours later in unrelated code. Turning it off to
   save flash is a false economy until you are shipping.    */

#define configCHECK_FOR_STACK_OVERFLOW   2   /* pattern check */
#define configUSE_MALLOC_FAILED_HOOK     1
#define configUSE_MUTEXES                1   /* or mutexes    */
                                             /* become binary */
                                             /* semaphores    */
#define configUSE_TRACE_FACILITY         1
#define configGENERATE_RUN_TIME_STATS    1

void vApplicationStackOverflowHook(TaskHandle_t t, char *name)
{
    /* Record to .noinit and reset - chapter 15. */
    strncpy(g_crash.task, name, sizeof(g_crash.task));
    g_crash.magic = CRASH_MAGIC_STACK;
    NVIC_SystemReset();
}
```

THREE LINES THAT MUST BE IN YOUR CONFIG, AND USUALLY ARE NOT

The FreeRTOS Cortex-M port installs its own SVC, PendSV and SysTick handlers. If your vendor's startup file also defines them — and ST's does — you get either a duplicate-symbol error or, worse, the vendor's stubs silently winning. The standard fix is these three defines, which redirect the names:

```
#define vPortSVCHandler      SVC_Handler
#define xPortPendSVHandler   PendSV_Handler
#define xPortSysTickHandler  SysTick_Handler
```

Omitting them produces a system that builds cleanly and never switches tasks. It is the most common first-time FreeRTOS failure by a wide margin.

Static allocation

```
/* Dynamic creation is convenient and, in a system that must
   not fail, a liability: xTaskCreate can return failure at
   run time, and a heap can fragment. Static allocation
   moves every failure to LINK TIME.                        */

#define configSUPPORT_STATIC_ALLOCATION  1
#define configSUPPORT_DYNAMIC_ALLOCATION  0   /* no heap at all */

static StaticTask_t  sensor_tcb;
static StackType_t   sensor_stack[512];

static StaticQueue_t reading_q_buf;
static uint8_t       reading_q_storage[10 * sizeof(reading_t)];

void app_init(void)
{
    TaskHandle_t h = xTaskCreateStatic(
        sensor_task, "sensor", 512, NULL, 3,
        sensor_stack, &sensor_tcb);
    configASSERT(h != NULL);        /* cannot actually fail  */

    QueueHandle_t q = xQueueCreateStatic(
        10, sizeof(reading_t), reading_q_storage, &reading_q_buf);
    configASSERT(q != NULL);
}

/* With dynamic allocation disabled entirely, the linker
   reports your true RAM usage and there is no heap to
   exhaust, fragment, or analyse. Safety-critical and
   certified projects require this; ordinary projects
   benefit from it and rarely bother.

   The cost: everything must be sized at compile time, which
   is a constraint you would want anyway on a part with
   64 kB of RAM.                                            */
```

The idle task and tickless sleep

```
/* The idle hook runs whenever nothing else is ready. It is
   where a battery-powered product sleeps.                  */

void vApplicationIdleHook(void)
{
    __WFI();       /* wake on the next interrupt or tick    */
}

/* But a 1 kHz tick wakes the core a thousand times a second
   whether or not there is work - which on a battery product
   dominates the current budget (chapter 13).

   TICKLESS IDLE fixes this: the kernel computes how long
   until the next task is due, programs a wake-up timer for
   exactly that, sleeps deeply, and then CORRECTS the tick
   count on waking so time still adds up.                   */

#define configUSE_TICKLESS_IDLE            1
#define configEXPECTED_IDLE_TIME_BEFORE_SLEEP  5   /* ticks   */

/* The generic implementation uses SysTick, which does not
   run in STOP mode. For real low power you supply
   portSUPPRESS_TICKS_AND_SLEEP() yourself, using the RTC as
   the wake source. Vendor ports usually provide one - check
   before writing it.

   Verify with an ammeter, not by reading the code. The
   difference between "tickless is enabled" and "tickless is
   working" is a factor of a hundred in average current, and
   only a current trace tells you which you have.           */
```

Debugging aids worth enabling

```
/* 1. RUN-TIME STATS - where the CPU actually goes. */
vTaskGetRunTimeStats(buf);
/*   Task        Abs Time      % Time
     IDLE        14893921      89%
     sensor        892134       5%
     comms         721002       4%
     display       201883       1%

   Needs a timer 10-100x faster than the tick, wired up via
   portCONFIGURE_TIMER_FOR_RUN_TIME_STATS. Worth the effort:
   it answers "is there headroom" definitively, and the idle
   percentage is your real margin.                          */

/* 2. HIGH WATER MARKS - chapter 20's measurement. */
UBaseType_t free_words = uxTaskGetStackHighWaterMark(handle);

/* 3. TASK LIST */
vTaskList(buf);
/*   Name      State  Prio  Stack  Num
     sensor    B      3     412    1      B = Blocked
     comms     R      2     288    2      R = Ready
     display   B      1     501    3      S = Suspended

   A task stuck in R that never becomes X (running) is a
   priority problem. A task in B forever is a lost signal.  */

/* 4. TRACEALYZER / SystemView
   Instrumented trace showing every switch, block and ISR on
   a timeline. This is how you SEE priority inversion rather
   than infer it, and it is worth the licence cost on any
   project where timing is the difficulty.                  */
```

How it shows up

Symptom	Cause
Scheduler starts, no task ever runs	Handler name defines missing; the vendor's stub handlers won
Hard fault as soon as the scheduler starts	Interrupt priority grouping not all-preemption; call `NVIC_SetPriorityGrouping(3)`
`configASSERT` fires in `port.c`	Usually an ISR priority above the syscall ceiling — read the line, it is precise
Mutexes give no priority inheritance	`configUSE_MUTEXES` is 0
`xTaskCreate` returns failure	Heap exhausted — raise `configTOTAL_HEAP_SIZE` or go static
Random corruption, no pattern	Stack overflow — enable `configCHECK_FOR_STACK_OVERFLOW` 2
Tickless enabled but current unchanged	Generic implementation using SysTick, which stops in deep sleep
Timings wrong by a constant factor	`configCPU_CLOCK_HZ` does not match the actual clock
Software timer callbacks are late	The timer daemon task's priority is too low
Everything works until heap fragments	`heap_2` in use — switch to `heap_4` or static

Check yourself

✓ Name the five heap implementations and say which is the usual choice.

- ✓ Explain what the three handler-name defines do and what happens without them.
- ✓ Explain the difference between the logical and raw forms of the syscall priority.
- ✓ Give four things `configASSERT` catches.
- ✓ Explain why `configUSE_MUTEXES` being 0 is dangerous rather than merely limiting.
- ✓ State two advantages of static allocation over dynamic.
- ✓ Explain what tickless idle does and why it needs tick correction.
- ✓ Say why enabling tickless idle may not reduce current, and how to check.
- ✓ Describe what run-time stats tell you that stack watermarks do not.
- ✓ Interpret a task shown as Blocked forever in `vTaskList`.

Zephyr in practice

Zephyr is not FreeRTOS with more features — it is a different proposition. FreeRTOS gives you a scheduler and leaves the system to you; Zephyr gives you a whole operating system with device drivers, a device tree, a build system and a networking stack, and asks you to work its way.

The essential difference

	FreeRTOS	Zephyr
What it is	A scheduler and IPC primitives	A complete RTOS with drivers, subsystems and a build system
Hardware description	Your code, vendor HAL	Device tree, as in Linux
Configuration	One header	Kconfig, as in Linux
Build	Your Makefile or IDE	`west` plus CMake
Drivers	Vendor HAL, per-chip	Portable driver API across all supported chips
Footprint	~6 kB flash minimum	~20 kB minimum, realistically much more
Learning curve	An afternoon	Genuinely a week or two
Board support	You write it	750+ boards supported upstream

The trade is legibility against leverage. FreeRTOS is small enough to read in a day; you will never wonder what it is doing. Zephyr abstracts the hardware so completely that the same application source builds for an nRF52, an STM32 and a RISC-V part — but understanding why something does not work means learning device tree, Kconfig and the driver model first.

Device tree: hardware as data

```
/* Zephyr borrows the device tree wholesale from Linux (see
   the Linux track, chapter 18). Hardware is DESCRIBED, not
   coded.                                                    */

/* boards/arm/my_board/my_board.dts */
/ {
    aliases {
        status-led = &led0;
        sensor-bus = &i2c1;
    };

    leds {
        compatible = "gpio-leds";
        led0: led_0 {
            gpios = <&gpioa 5 GPIO_ACTIVE_HIGH>;
            label = "Status LED";
        };
    };
};

&i2c1 {
    status = "okay";
    clock-frequency = <I2C_BITRATE_FAST>;

    bme280@76 {
        compatible = "bosch,bme280";
        reg = <0x76>;
    };
};

/* The application never names a pin, a port or a register: */

#include <zephyr/drivers/gpio.h>

static const struct gpio_dt_spec led =
    GPIO_DT_SPEC_GET(DT_ALIAS(status_led), gpios);

int main(void)
{
    if (!gpio_is_ready_dt(&led)) {
        return -ENODEV;
    }
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);

    for (;;) {
        gpio_pin_toggle_dt(&led);
        k_msleep(500);
    }
}

/* This source is portable across every supported board. Move
   to different silicon and you change the .dts, not the
   application. That is the leverage Zephyr is selling, and
   on a product line with several hardware variants it is a
   very large saving.

   The macros are resolved at COMPILE time - devicetree.h is
   generated from the .dts during the build - so there is no
   runtime parsing cost, unlike Linux.                       */
```

> **DEVICE TREE ERRORS ARE COMPILE-TIME AND CRYPTIC**
> A missing node or a typo'd alias produces an error inside a generated header, pointing at a macro expansion rather than your `.dts`. The habit that saves hours: after any device tree change, read `build/zephyr/zephyr.dts` — the fully merged and resolved tree — rather than your fragment. It shows what the build actually concluded, including everything the SoC's own `.dtsi` files contributed.

The kernel API

```
/* Same concepts as chapter 19, different spelling, and
   static definition is idiomatic rather than exceptional.  */

#include <zephyr/kernel.h>

/* Threads. Note K_THREAD_DEFINE creates it at BOOT with no
   runtime allocation - the Zephyr default style.           */
K_THREAD_DEFINE(sensor_tid, 1024, sensor_thread,
                NULL, NULL, NULL,
                5,        /* priority: LOWER = more urgent,
                             the OPPOSITE of FreeRTOS and the
                             same as the NVIC. Cooperative
                             priorities are negative.        */
                0, 0);

/* Message queue - the queue of chapter 19. */
K_MSGQ_DEFINE(sensor_q, sizeof(struct reading), 10, 4);

void sensor_thread(void *a, void *b, void *c)
{
    struct reading r;
    while (1) {
        if (k_msgq_get(&sensor_q, &r, K_MSEC(100)) == 0) {
            process(&r);
        }
    }
}

/* From an ISR - note there is no separate FromISR API.
   Zephyr's calls detect ISR context themselves and require
   only that you pass K_NO_WAIT.                            */
void my_isr(const void *arg)
{
    struct reading r = read_hw();
    k_msgq_put(&sensor_q, &r, K_NO_WAIT);   /* must not block */
}

/* Other primitives, mapping onto chapter 19:
     k_mutex      mutex, with priority inheritance
     k_sem        counting semaphore
     k_event      event flags
     k_fifo/lifo  linked-list queues, no copy
     k_work       deferred work to a system workqueue -
                  the standard "get it out of the ISR"
                  mechanism, equivalent to a Linux workqueue

   K_FOREVER, K_NO_WAIT and K_MSEC(n) replace FreeRTOS's
   portMAX_DELAY and tick arithmetic, and are type-safe.    */
```

Kconfig and west

```
# prj.conf — application configuration, Kconfig syntax
CONFIG_GPIO=y
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_BME280=y
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3
CONFIG_MAIN_STACK_SIZE=2048
CONFIG_THREAD_ANALYZER=y          # stack watermarks, ch 20
CONFIG_THREAD_ANALYZER_AUTO=y

# Build and flash
$ west build -b nucleo_f429zi app/
$ west flash
$ west debug

# Inspect what Kconfig actually resolved to — symbols have
# dependencies, and one you set may be silently disabled
# because its prerequisite is off. This file is the truth:
$ less build/zephyr/.config

# Change configuration interactively, which also SHOWS the
# dependency that is blocking a symbol:
$ west build -t menuconfig

/* The most common Zephyr frustration by far: "I set
   CONFIG_FOO=y and it had no effect." Almost always FOO
   depends on BAR which is not enabled, and Kconfig silently
   dropped it. menuconfig's search (press /) tells you the
   dependency chain immediately.                          */
```

Choosing between them

Situation	Choice
32 kB flash, one board, fixed scope	FreeRTOS, or no RTOS at all
You need BLE, Wi-Fi, USB or TCP/IP	Zephyr — the stacks are integrated and maintained
Several hardware variants of one product	Zephyr — the device tree pays for itself quickly
Vendor supplies a FreeRTOS BSP and examples	FreeRTOS — swimming with the current
You must read and justify every line for certification	FreeRTOS, or a certified kernel like SAFERTOS
Long-lived product needing upstream security fixes	Zephyr — an active foundation and a real release cadence
Team already knows Linux device tree and Kconfig	Zephyr — the concepts transfer directly
You need it working this week	FreeRTOS

How it shows up

Symptom	Cause
`CONFIG_FOO=y` has no effect	An unmet Kconfig dependency; check `build/zephyr/.config`
Cryptic error inside a generated header	Device tree node or alias missing — read the merged `zephyr.dts`
`gpio_is_ready_dt` returns false	The controller's node has `status = "disabled"`

Symptom	Cause
Device not found at runtime	Driver not enabled in Kconfig even though the node exists
Priorities behave backwards versus FreeRTOS	Zephyr uses lower = more urgent, and negative for cooperative
Kernel call hangs the system from an ISR	A timeout other than `K_NO_WAIT` passed in interrupt context
Build succeeds, board does nothing	Wrong board target — the binary is for different silicon
Binary far larger than expected	Logging, shell or asserts left enabled in a release build
Stack overflow in the system workqueue	Long work items — raise `CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE`
Cannot find where a pin is configured	It is in the pinctrl node of the board's `.dts`, not in code

Check yourself

- ✓ State the essential difference in what FreeRTOS and Zephyr each provide.
- ✓ Explain what the device tree buys you and what it costs.
- ✓ Say why Zephyr's device tree has no runtime parsing cost, unlike Linux's.
- ✓ Give the file to read when a device tree change does not behave as expected.
- ✓ State Zephyr's priority direction and contrast it with FreeRTOS.
- ✓ Explain why Zephyr has no separate `FromISR` API.
- ✓ Name the Zephyr equivalent of deferring work out of an ISR.
- ✓ Describe the most common Kconfig frustration and how to diagnose it.
- ✓ Give three situations that clearly favour Zephyr.
- ✓ Give three situations that clearly favour FreeRTOS.

Project: bare-metal bring-up, every register explained

A complete firmware image with no vendor HAL, no CMSIS beyond the register definitions, and no library you did not write. It boots, configures its clocks, drives a UART, samples an ADC over DMA, and reports. Roughly 400 lines, and every one of them is explainable.

What it does, and what to run it on

```
TARGET: STM32F411 "Black Pill" or any Nucleo-F4. The
register names below are F4-family; the STRUCTURE ports
to any Cortex-M with a page of renaming.

NO HARDWARE? QEMU emulates several Cortex-M boards:

  qemu-system-arm -M netduinoplus2 -cpu cortex-m4 \
                  -nographic -kernel firmware.elf

The QEMU model implements the core, NVIC, SysTick and
USART faithfully. It does NOT model the RCC clock tree
or the ADC properly, so parts 3 and 6 need real silicon
to be meaningful. Everything else runs.

BEHAVIOUR:
  - boots to a 100 MHz PLL clock
  - blinks an LED at exactly 1 Hz from SysTick
  - prints a banner and a reading once a second over
    UART at 115200
  - samples an internal temperature sensor via DMA
  - reports its own stack high-water mark
  - decodes and reports any fault, then resets

BUILD:
  $ make
  $ arm-none-eabi-size firmware.elf
  $ st-flash write firmware.bin 0x08000000
```

The files

```
project/
  Makefile
  linker.ld          chapter 05
  startup.c          chapters 04, 06 - vectors + Reset_Handler
  main.c             the application
  clock.c            chapter 09
  uart.c             chapters 10, 12
  adc_dma.c          chapter 12
  fault.c            chapter 15
  stm32f411.h        register definitions (from CMSIS, or
                     write the handful you need)

/* Makefile - the whole thing. */
CROSS   = arm-none-eabi-
CFLAGS  = -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard \
          -Og -g3 -Wall -Wextra -Werror \
          -ffunction-sections -fdata-sections \
          -fstack-usage -std=c11
LDFLAGS = -T linker.ld -nostartfiles -Wl,--gc-sections \
          -Wl,-Map=firmware.map -specs=nano.specs

OBJS = startup.o main.o clock.o uart.o adc_dma.o fault.o

firmware.elf: $(OBJS) linker.ld
        $(CROSS)gcc $(CFLAGS) $(LDFLAGS) $(OBJS) -o $@
        $(CROSS)objcopy -O binary $@ firmware.bin
        $(CROSS)size $@

%.o: %.c
        $(CROSS)gcc $(CFLAGS) -c $< -o $@

/* -Werror is deliberate. On bare metal a warning about an
   uninitialised variable or a discarded qualifier is
   usually a real bug, and there is no runtime to catch it.
   -fstack-usage produces the .su files chapter 20 uses.   */
```

Part 1: startup

```
/* startup.c - chapters 04 and 06, condensed. */

#include <stdint.h>

extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss, _estack;
extern int main(void);
void SystemInit(void);

void Default_Handler(void) { __asm volatile ("bkpt #0"); for(;;){} }

#define WEAK_DEF __attribute__((weak, alias("Default_Handler")))
void NMI_Handler(void)        WEAK_DEF;
void HardFault_Handler(void)  WEAK_DEF;
void MemManage_Handler(void)  WEAK_DEF;
void BusFault_Handler(void)   WEAK_DEF;
void UsageFault_Handler(void) WEAK_DEF;
void SVC_Handler(void)        WEAK_DEF;
void PendSV_Handler(void)     WEAK_DEF;
void SysTick_Handler(void)    WEAK_DEF;
void USART2_IRQHandler(void)  WEAK_DEF;
void DMA2_Stream0_IRQHandler(void) WEAK_DEF;

__attribute__((noreturn)) void Reset_Handler(void)
{
    uint32_t *src = &_sidata, *dst = &_sdata;
    while (dst < &_edata) { *dst++ = *src++; }   /* .data     */
    dst = &_sbss;
    while (dst < &_ebss)  { *dst++ = 0; }        /* .bss      */

    /* Fill the unused stack with a pattern so we can measure
       the high-water mark later (chapter 20). We are running
       ON this stack, so stop short of the current SP.        */
    extern uint32_t _sstack;
    uint32_t sp = __get_MSP();
    for (uint32_t *p = &_sstack; p < (uint32_t *)(sp - 64); p++) {
        *p = 0xA5A5A5A5u;
    }

    SystemInit();          /* AFTER .data - chapter 06        */
    (void) main();
    for (;;) { }
}

__attribute__((section(".isr_vector"), used))
void (* const g_vectors[])(void) = {
    (void (*)(void)) &_estack,
    Reset_Handler,  NMI_Handler,       HardFault_Handler,
    MemManage_Handler, BusFault_Handler, UsageFault_Handler,
    0, 0, 0, 0,
    SVC_Handler,    Default_Handler,   0,
    PendSV_Handler, SysTick_Handler,
    /* IRQ 0.. - only the two we use are named; the rest
       default. Index = IRQ + 16 (chapter 07).               */
    [16 + 38] = USART2_IRQHandler,
    [16 + 56] = DMA2_Stream0_IRQHandler,
};

/* The designated-initialiser syntax [16 + 38] = ... is a
```

```
    C99 feature that saves listing eighty zeros, and it makes
    the IRQ-to-slot arithmetic explicit in the source.       */
```

Part 2: clocks, with a fallback

```
/* clock.c - chapter 09, including the timeout that chapter
   insisted on.                                            */

uint32_t SystemCoreClock = 16000000u;   /* HSI, until proven */
volatile uint8_t g_clock_degraded;

static int wait_flag(volatile uint32_t *reg, uint32_t mask,
                     uint32_t tries)
{
    while (!(*reg & mask)) {
        if (--tries == 0u) { return -1; }
    }
    return 0;
}

void SystemInit(void)
{
    /* FPU on, before any float code. Barriers required.    */
    SCB->CPACR |= (0xFu << 20);
    __DSB(); __ISB();

    /* Enable the specific faults - chapter 15's first move. */
    SCB->SHCSR |= SCB_SHCSR_USGFAULTENA_Msk
               |  SCB_SHCSR_BUSFAULTENA_Msk
               |  SCB_SHCSR_MEMFAULTENA_Msk;

    RCC->CR |= RCC_CR_HSEON;
    if (wait_flag(&RCC->CR, RCC_CR_HSERDY, 100000u) < 0) {
        g_clock_degraded = 1;      /* run on HSI at 16 MHz  */
        return;                    /* the board still boots */
    }

    FLASH->ACR = FLASH_ACR_LATENCY_3WS | FLASH_ACR_PRFTEN
               | FLASH_ACR_ICEN | FLASH_ACR_DCEN;

    RCC->CFGR |= RCC_CFGR_PPRE1_DIV2;   /* APB1 = 50 MHz    */

    /* 25 MHz HSE: /25 = 1 MHz, x200 = 200 MHz, /2 = 100 MHz */
    RCC->PLLCFGR = 25u | (200u << 6) | (0u << 16)
                 | RCC_PLLCFGR_PLLSRC_HSE | (4u << 24);
    RCC->CR |= RCC_CR_PLLON;
    if (wait_flag(&RCC->CR, RCC_CR_PLLRDY, 100000u) < 0) {
        g_clock_degraded = 1;
        return;
    }

    RCC->CFGR = (RCC->CFGR & ~RCC_CFGR_SW) | RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) { }

    SystemCoreClock = 100000000u;
}

/* A field unit whose crystal has cracked now boots at 16 MHz
   with g_clock_degraded set, prints a warning over its UART
   (at a baud rate computed from the ACTUAL clock, so it is
   still readable) and keeps working. Compare with hanging
   forever in a wait loop.                                 */
```

Part 3: UART, with a lock-free ring

```
/* uart.c - chapters 10, 12, 17. */

#define TXBUF 256                          /* power of two   */
static volatile uint8_t txb[TXBUF];
static volatile uint16_t txhead, txtail;

void uart_init(uint32_t baud)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
    (void) RCC->APB1ENR;                   /* read back      */

    /* PA2 = TX, AF7. Chapter 10's sequence.                 */
    GPIOA->MODER   = (GPIOA->MODER   & ~(3u << 4)) | (2u << 4);
    GPIOA->OSPEEDR |= (3u << 4);
    GPIOA->AFR[0]  = (GPIOA->AFR[0]  & ~(0xFu << 8)) | (7u << 8);

    /* APB1 is SystemCoreClock/2, and USART2 lives on APB1.
       Using the variable means a degraded clock still gives
       a correct baud rate.                                  */
    USART2->BRR = ((SystemCoreClock / 2u) + baud / 2u) / baud;
    USART2->CR1 = USART_CR1_TE | USART_CR1_UE;

    NVIC_SetPriority(USART2_IRQn, 6);
    NVIC_EnableIRQ(USART2_IRQn);
}

void uart_putc(char c)
{
    uint16_t next = (uint16_t)((txhead + 1u) & (TXBUF - 1u));
    while (next == txtail) { }      /* full: wait for the ISR */
    txb[txhead] = (uint8_t) c;
    txhead = next;                  /* publish AFTER the data */
    USART2->CR1 |= USART_CR1_TXEIE; /* kick the ISR           */
}

void USART2_IRQHandler(void)
{
    if (USART2->SR & USART_SR_TXE) {
        if (txtail != txhead) {
            USART2->DR = txb[txtail];
            txtail = (uint16_t)((txtail + 1u) & (TXBUF - 1u));
        } else {
            USART2->CR1 &= ~USART_CR1_TXEIE;   /* nothing left */
        }
    }
}

/* uart_putc returns in nanoseconds unless the buffer is
   full, so main() is never blocked by 87 us per character.
   The single-producer/single-consumer argument from chapter
   17 applies: one writer of txhead, one of txtail.        */
```

Part 4: main

```
/* main.c */

volatile uint32_t g_ms;
void SysTick_Handler(void) { g_ms++; }

int main(void)
{
    /* SysTick: 1 kHz from whatever clock we ended up with. */
    SysTick->LOAD = (SystemCoreClock / 1000u) - 1u;
    SysTick->VAL  = 0;
    SysTick->CTRL = 7u;      /* core clock | tick int | enable */

    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
    (void) RCC->AHB1ENR;
    GPIOC->MODER = (GPIOC->MODER & ~(3u << 26)) | (1u << 26);

    uart_init(115200);
    adc_dma_init();

    fault_report_previous();     /* chapter 15's .noinit log */

    uart_puts("\r\n=== bare-metal bring-up ===\r\n");
    if (g_clock_degraded) {
        uart_puts("WARNING: HSE failed, running on HSI\r\n");
    }
    uart_printf("SYSCLK  %lu Hz\r\n", SystemCoreClock);

    uint32_t next = 0;
    for (;;) {
        if ((int32_t)(g_ms - next) >= 0) {   /* wrap-safe    */
            next += 500u;
            GPIOC->ODR ^= (1u << 13);       /* 1 Hz square   */

            uart_printf("t=%lus  temp=%d.%02u C  stack_free=%u\r\n",
                        g_ms / 1000u,
                        adc_temperature_c() / 100,
                        (unsigned)(adc_temperature_c() % 100),
                        (unsigned) stack_unused_bytes());
        }
        __WFI();      /* sleep until SysTick or the UART ISR */
    }
}

/* The __WFI() is the point. This loop uses well under 1% of
   the CPU: everything is interrupt- or DMA-driven, and the
   core sleeps between events. That is what all twenty-two
   preceding chapters were building toward.                */
```

Exercises, in order of difficulty

- **Verify the clock.** Route MCO to a pin and confirm 100 MHz with a scope, or time the LED against a stopwatch for 60 seconds. Do not trust `SystemCoreClock`.
- **Break the vector table.** Corrupt the first word, flash it, and observe the difference between running under the debugger and cold booting (chapter 04).
- **Provoke each fault.** Write to address 0, execute from RAM, divide by zero with `DIV_0_TRP` set. Confirm your handler decodes each correctly (chapter 15).

- **Measure the ISR.** Toggle a spare pin at the top and bottom of the UART handler and read its duration on a scope. Then compute the maximum baud rate this design supports.
- **Add the MPU stack guard** from chapter 14, then deliberately recurse until it fires.
- **Convert the UART to DMA** (chapter 12) and compare CPU load at 921600 baud.
- **Make it sleep properly.** Move to STOP mode between samples with an RTC wake-up, and measure the average current. Aim for under 10 µA.

How it shows up

Symptom	Cause
Nothing at all happens	Vector table missing — check `KEEP()` and the map file
Works under debugger, dead on power-up	Vector table word 0 wrong; the debugger set SP itself
UART emits garbage	`BRR` computed from the wrong clock — check the APB prescaler
UART emits nothing	GPIO alternate function number wrong; verify AF7 in the *datasheet*
LED blinks at half or double rate	`SystemCoreClock` does not match reality; measure it
Board hangs before the banner	A wait loop with no timeout — this project's `wait_flag` exists for that
Fault handler itself faults	It called `uart_printf`, which touches a peripheral; log to `.noinit` instead
Stack free reads as the whole stack	Fill pattern written after the stack was already used
ADC reads a constant value	DMA buffer in unreachable RAM, or the ADC clock is gated
Everything works, current is 20 mA	No `__WFI()`, or a debugger still attached

Check yourself

- ✓ Explain every line of the vector table, including the designated initialisers.
- ✓ Say why `SystemInit()` is called after the `.data` copy.
- ✓ Explain how a cracked crystal leaves this firmware still able to report the fault.
- ✓ Justify computing `BRR` from `SystemCoreClock` rather than a constant.
- ✓ Explain why `uart_putc` does not block in normal operation.
- ✓ Explain why `txhead` is published after the data byte is stored.
- ✓ Explain the wrap-safe comparison in the main loop.
- ✓ Say what `__WFI()` achieves here and what would break without it.
- ✓ Explain why `-Werror` is a reasonable default on bare metal.
- ✓ Describe how you would verify the clock frequency without trusting any variable.

Project: a preemptive RTOS from scratch

About 300 lines gives you a real preemptive kernel: fixed-priority scheduling, blocking delays, mutexes with priority inheritance, and message queues. Writing one is the fastest way to stop finding FreeRTOS mysterious, because every line of it will be something you have already decided.

Scope

```
WHAT IT HAS                    WHAT IT DELIBERATELY LACKS

fixed-priority preemption      dynamic task creation
round-robin within a priority  task deletion
k_sleep with a tick wheel      software timers
counting semaphores            event groups
mutexes + priority inheritance recursive mutexes
fixed-size message queues      stack overflow detection
idle task with WFI             (add the MPU guard from
                                chapter 14 as an exercise)

Everything is STATICALLY allocated - tasks are declared at
compile time. That removes an entire class of failure and
keeps the code readable.

Target: any Cortex-M3/M4. Runs under QEMU:
  qemu-system-arm -M netduinoplus2 -cpu cortex-m4 \
                  -nographic -kernel kernel.elf
```

Part 1: the task control block

```
/* kernel.h */

#define MAX_TASKS      8
#define MAX_PRIO       8

typedef enum {
    T_READY, T_RUNNING, T_BLOCKED, T_SLEEPING
} tstate_t;

typedef struct tcb {
    uint32_t     *sp;          /* MUST be offset 0 - ch 18   */
    tstate_t      state;
    uint8_t       prio;        /* current, may be inherited  */
    uint8_t       base_prio;   /* what it was declared with  */
    uint32_t      wake_tick;
    struct tcb   *next;        /* ready or wait list link    */
    void         *wait_obj;    /* what it is blocked on      */
    uint32_t     *stack_base;
    uint32_t      stack_words;
    const char   *name;
} tcb_t;

extern tcb_t *current_tcb;     /* the assembly reads this    */

/* Ready lists: one singly-linked list per priority, plus a
   bitmap so "highest ready priority" is one instruction.   */
static tcb_t   *ready[MAX_PRIO];
static uint32_t ready_bits;

static inline int highest_ready(void)
{
    /* CLZ: count leading zeros. Bit 31 = priority 0.
       31 - clz gives the highest set bit, and one
       instruction replaces a loop. This is exactly the
       trick FreeRTOS uses in its optimised port.           */
    return ready_bits ? (31 - __builtin_clz(ready_bits)) : -1;
}

static void ready_add(tcb_t *t)
{
    t->state = T_READY;
    t->next  = ready[t->prio];
    ready[t->prio] = t;
    ready_bits |= (1u << t->prio);
}

static tcb_t *ready_pop(int prio)
{
    tcb_t *t = ready[prio];
    if (t) {
        ready[prio] = t->next;
        if (!ready[prio]) { ready_bits &= ~(1u << prio); }
    }
    return t;
}
```

Part 2: the scheduler

```
/* This is the C function chapter 18's assembly calls. All
   the POLICY is here; the assembly is pure mechanism.     */

void scheduler_next(void)
{
    /* Put the outgoing task back if it is still runnable. */
    if (current_tcb && current_tcb->state == T_RUNNING) {
        ready_add(current_tcb);
    }

    int p = highest_ready();
    /* p is never -1: the idle task is always ready at
       priority 0, which is why an idle task must exist.   */
    tcb_t *next = ready_pop(p);
    next->state = T_RUNNING;
    current_tcb = next;
}

/* Requesting a switch. One store, callable from anywhere -
   task code or any ISR (chapter 18).                      */
static inline void yield(void)
{
    SCB->ICSR = SCB_ICSR_PENDSVSET_Msk;
    __DSB(); __ISB();
}

/* The tick: accounting plus wake-ups. */
volatile uint32_t g_ticks;

void SysTick_Handler(void)
{
    g_ticks++;
    int woke = 0;

    for (int i = 0; i < MAX_TASKS; i++) {
        tcb_t *t = &g_tasks[i];
        if (t->state == T_SLEEPING &&
            (int32_t)(g_ticks - t->wake_tick) >= 0) {  /* wrap-safe */
            ready_add(t);
            woke = 1;
        }
    }

    /* Switch if someone woke, or to round-robin the current
       priority level.                                      */
    if (woke || ready[current_tcb->prio]) {
        yield();
    }
}

/* The O(MAX_TASKS) scan is fine for 8 tasks and wrong for
   200. A real kernel keeps a delta list sorted by wake time
   so the tick is O(1). Worth implementing as an exercise -
   it is the difference between a toy and a kernel.        */
```

Part 3: the switch

```
/* Chapter 18's handler, unchanged. This is the only
   assembly in the project.                              */

__attribute__((naked)) void PendSV_Handler(void)
{
    __asm volatile (
    "   mrs   r0, psp             \n"
    "   cbz   r0, 1f              \n"
    "   subs  r0, r0, #32         \n"
    "   stm   r0, {r4-r11}        \n"
    "   ldr   r1, =current_tcb    \n"
    "   ldr   r2, [r1]            \n"
    "   str   r0, [r2]            \n"   /* sp at offset 0    */
    "1:                           \n"
    "   push  {lr}                \n"
    "   bl    scheduler_next      \n"
    "   pop   {lr}                \n"
    "   ldr   r1, =current_tcb    \n"
    "   ldr   r2, [r1]            \n"
    "   ldr   r0, [r2]            \n"
    "   ldm   r0, {r4-r11}        \n"
    "   adds  r0, r0, #32         \n"
    "   msr   psp, r0             \n"
    "   isb                       \n"
    "   bx    lr                  \n"
    );
}

/* Task creation: fabricate a frame (chapter 18). */
void task_create(tcb_t *t, const char *name,
                 void (*entry)(void *), void *arg,
                 uint8_t prio, uint32_t *stack, uint32_t words)
{
    for (uint32_t i = 0; i < words; i++) {
        stack[i] = 0xA5A5A5A5u;              /* watermark    */
    }

    uint32_t *sp = (uint32_t *)(((uint32_t)(stack + words)) & ~7u);
    *(--sp) = 0x01000000u;                   /* xPSR: T BIT  */
    *(--sp) = (uint32_t) entry & ~1u;        /* PC           */
    *(--sp) = (uint32_t) task_exit;          /* LR           */
    *(--sp) = 0; *(--sp) = 0; *(--sp) = 0; *(--sp) = 0;
    *(--sp) = (uint32_t) arg;                /* R0           */
    for (int i = 0; i < 8; i++) { *(--sp) = 0; }  /* R4-R11  */

    t->sp = sp;  t->name = name;
    t->prio = t->base_prio = prio;
    t->stack_base = stack;  t->stack_words = words;
    ready_add(t);
}
```

Part 4: blocking, and a mutex that inherits

```
/* Critical sections use BASEPRI, so high-priority interrupts
   keep running through them (chapters 03, 08).            */

#define KERNEL_PRIO  (5u << (8 - __NVIC_PRIO_BITS))

static inline uint32_t enter_critical(void)
{
    uint32_t old = __get_BASEPRI();
    __set_BASEPRI(KERNEL_PRIO);
    __DSB(); __ISB();
    return old;
}
static inline void exit_critical(uint32_t old)
{
    __set_BASEPRI(old);
}

void k_sleep(uint32_t ms)
{
    uint32_t s = enter_critical();
    current_tcb->wake_tick = g_ticks + ms;
    current_tcb->state     = T_SLEEPING;
    exit_critical(s);
    yield();                      /* switches away here     */
}

/* MUTEX WITH PRIORITY INHERITANCE - the feature that
   distinguishes a mutex from a semaphore (chapter 19).    */

typedef struct {
    tcb_t *owner;
    tcb_t *waiters;
    int    locked;
} mutex_t;

void mutex_lock(mutex_t *m)
{
    uint32_t s = enter_critical();

    if (!m->locked) {
        m->locked = 1;
        m->owner  = current_tcb;
        exit_critical(s);
        return;
    }

    /* THE INHERITANCE. If we outrank the owner, lend it our
       priority so it cannot be preempted by anything in
       between - which is exactly the unbounded inversion of
       the concurrency track, chapter 15.                   */
    if (current_tcb->prio > m->owner->prio) {
        /* remove, re-prioritise, re-insert: the owner may be
           sitting in a ready list under its old priority.   */
        ready_remove(m->owner);
        m->owner->prio = current_tcb->prio;
        if (m->owner->state == T_READY) { ready_add(m->owner); }
    }

    current_tcb->state    = T_BLOCKED;
    current_tcb->wait_obj = m;
```

```
    current_tcb->next     = m->waiters;
    m->waiters            = current_tcb;

    exit_critical(s);
    yield();
}

void mutex_unlock(mutex_t *m)
{
    uint32_t s = enter_critical();

    /* Drop any inherited priority first. */
    current_tcb->prio = current_tcb->base_prio;

    tcb_t *best = NULL, **link = &m->waiters;
    for (tcb_t **p = &m->waiters; *p; p = &(*p)->next) {
        if (!best || (*p)->prio > best->prio) { best = *p; link = p; }
    }

    if (best) {
        *link     = best->next;
        m->owner  = best;
        ready_add(best);
    } else {
        m->locked = 0;
        m->owner  = NULL;
    }

    exit_critical(s);
    yield();
}
```

WHERE THIS KERNEL IS HONESTLY SIMPLIFIED

Inheritance here is one level deep: if the owner is itself blocked on a second mutex, the boost does not propagate down the chain. Real kernels handle that transitively. The kernel also has no deadlock detection, no timeout on `mutex_lock`, and an O(n) tick. Each is a deliberate omission you can now see the shape of — which is the point of writing one. Read FreeRTOS's `xTaskPriorityInherit` afterwards and it will read as a careful version of code you have already written.

Part 5: it running

```
static uint32_t s_hi[256], s_lo[256], s_idle[64];
static tcb_t    t_hi, t_lo, t_idle;
static mutex_t  bus;

static void hi_task(void *a)
{
    for (;;) {
        k_sleep(100);
        uint32_t t0 = DWT->CYCCNT;
        mutex_lock(&bus);
        use_shared_bus();
        mutex_unlock(&bus);
        record_max(DWT->CYCCNT - t0);   /* blocking time     */
    }
}

static void lo_task(void *a)
{
    for (;;) {
        mutex_lock(&bus);
        busy_work(50);           /* hold it a long time      */
        mutex_unlock(&bus);
        k_sleep(10);
    }
}

static void idle_task(void *a) { for (;;) { __WFI(); } }

int main(void)
{
    clock_init();  uart_init(115200);  cycle_counter_init();

    task_create(&t_hi,   "hi",   hi_task,   NULL, 5, s_hi,   256);
    task_create(&t_lo,   "lo",   lo_task,   NULL, 2, s_lo,   256);
    task_create(&t_idle, "idle", idle_task, NULL, 0, s_idle,  64);

    NVIC_SetPriority(PendSV_IRQn,  0xFF);   /* LOWEST - ch 18 */
    NVIC_SetPriority(SysTick_IRQn, 0xFE);

    current_tcb = ready_pop(highest_ready());
    current_tcb->state = T_RUNNING;
    __set_PSP((uint32_t) current_tcb->sp + 32);
    __set_CONTROL(0x02);
    __ISB();                                /* MANDATORY      */

    SysTick->LOAD = (SystemCoreClock / 1000u) - 1u;
    SysTick->VAL  = 0;
    SysTick->CTRL = 7u;

    ((void (*)(void *)) hi_task)(NULL);      /* never returns  */
    for (;;) { }
}

/* THE EXPERIMENT WORTH RUNNING: add a medium-priority task
   at priority 3 that does nothing but busy_work. Then
   disable the inheritance block in mutex_lock and compare
   the recorded worst-case blocking time of hi_task.

   Without inheritance it grows without bound as the medium
   task takes more CPU - that is unbounded priority
```

```
    inversion, reproduced on your own desk in about twenty
    lines. Re-enable inheritance and it collapses to the
    length of lo_task's critical section. Seeing that number
    change is worth more than any amount of reading about
    Mars Pathfinder.                                        */
```

Exercises

- **Reproduce unbounded inversion** as described above, and plot worst-case blocking against the medium task's CPU share.
- **Add a message queue** with blocking send and receive, and the wake-on-send that makes a waiting receiver runnable immediately rather than at the next tick.
- **Replace the O(n) tick** with a delta list sorted by wake time.
- **Add the MPU stack guard** from chapter 14, one region per task, reloaded in `scheduler_next`. Then recurse until it fires and confirm the fault names the right task.
- **Add FP context saving** and verify with two tasks doing float maths that corruption appears without it and vanishes with it.
- **Make inheritance transitive** across a chain of two mutexes.
- **Measure the switch.** Toggle a pin in PendSV and count cycles with DWT. Compare against FreeRTOS on the same board.

How it shows up

Symptom	Cause
Fault the moment the first task starts	T bit missing from the fabricated xPSR
Faults after the first switch, not before	No null check for the first entry, or `sp` not at TCB offset 0
Corruption under interrupt load	PendSV not the lowest priority exception
Scheduler picks the wrong task	`ready_bits` not cleared when a list empties
Everything hangs when all tasks sleep	No idle task — `highest_ready()` returns −1
System stops after ~49 days of ticks	Tick comparison not wrap-safe
A woken task waits up to a full tick	No `yield()` after making it ready
Priority stays boosted after unlock	`base_prio` not restored in `mutex_unlock`
Inheritance has no effect	Owner left in a ready list under its old priority — remove and re-add
Corruption once two tasks use `float`	FP registers not saved in the switch

Check yourself

- ✓ Explain why `sp` must be the first member of the TCB.
- ✓ Explain the CLZ trick for finding the highest ready priority.
- ✓ Say why an idle task is structurally required, not merely convenient.
- ✓ Explain why critical sections use BASEPRI rather than PRIMASK.
- ✓ Describe what priority inheritance does and why the owner must be re-inserted.
- ✓ Explain why `base_prio` is stored separately from `prio`.

- ✓ Say why the tick's sleep scan is wrap-safe and what it costs at scale.
- ✓ Describe the experiment that demonstrates unbounded priority inversion.
- ✓ Name three ways this kernel is simplified relative to FreeRTOS.
- ✓ Explain why `yield()` is safe to call from an ISR.

TRACK

Peripherals & buses

The serial protocols an embedded engineer touches daily — UART, SPI, I2C, CAN, USB and MIPI — at the bit level, and how to debug them.

21 chapters

How a peripheral attaches: registers, IRQs and DMA

Every serial controller in this track — UART, SPI, I2C, CAN, USB — presents the same four surfaces to software. Learn the shape once and each new peripheral becomes a question of which bits, not how it works.

The universal shape

```
FOUR SURFACES, and every peripheral has all four.

1. CONTROL registers     you write: enable, mode, speed
2. STATUS registers      you read:  ready, busy, error
3. DATA registers        the actual bytes, in and out
4. INTERRUPT enables     which status bits raise an IRQ

+-------------------------------------------------+
|  CPU        writes CR1  ---------------+        |
|             reads  SR   <----------+   |        |
|             writes DR   -------+   |   |        |
+--------------------------------|--|---|--------+
                                 v  |   v
+-------------------------------------------------+
|  PERIPHERAL                                     |
|    +---------+   +-----------+   +------------+ |
|    | control |   |  SHIFT    |   |   status   | |
|    | logic   |-->|  REGISTER |-->|   flags    | |
|    +---------+   +-----------+   +------------+ |
|                       |               |         |
+-----------------------|---------------|---------+
                        v               v
                      PINS            NVIC

THE SHIFT REGISTER IS THE HEART. A serial peripheral is a
parallel-to-serial converter with timing logic: you write
a byte, it clocks the bits out one at a time at the
configured rate. Everything else is bookkeeping around
that.

THE DOUBLE BUFFER that confuses everyone:

  DR (data register)  -->  SHIFT REGISTER  --> pin

Two stages. "TXE" (transmit empty) means DR is free for
the NEXT byte - the previous one is still shifting out.
"TC" (transmission complete) means the shift register is
empty too and the line is genuinely idle.

Get these confused and you disable the transmitter, or
turn off an RS-485 driver, while the last byte is still
on the wire. Truncated final characters are ALWAYS this
bug (chapter 14).
```

Reading a register map

```
/* Vendor headers give you a struct; the reference manual
   gives you the meaning. Both are needed.                  */

typedef struct {
    volatile uint32_t CR1;    /* control 1: enable, parity   */
    volatile uint32_t CR2;    /* control 2: stop bits, addr  */
    volatile uint32_t CR3;    /* control 3: flow, DMA        */
    volatile uint32_t BRR;    /* baud rate divisor           */
    volatile uint32_t GTPR;
    volatile uint32_t RTOR;
    volatile uint32_t RQR;
    volatile uint32_t ISR;    /* status - READ ONLY          */
    volatile uint32_t ICR;    /* interrupt CLEAR             */
    volatile uint32_t RDR;    /* receive data  - READ ONLY   */
    volatile uint32_t TDR;    /* transmit data - WRITE ONLY  */
} USART_TypeDef;

/* FOUR CLEARING IDIOMS, and you must know which applies:

   1. WRITE 1 TO CLEAR (w1c)
        USART1->ICR = USART_ICR_TCCF;
      Writing 0 does nothing. Writing ~0 clears everything -
      an easy accidental bug.

   2. READ TO CLEAR
        (void) USART1->RDR;        // clears RXNE
      Dangerous: a debugger displaying the register CLEARS
      THE FLAG. "It works until I watch it" is this.

   3. READ ONE, THEN READ/WRITE ANOTHER
        status = USART1->SR; (void) USART1->DR;
      The classic F1-era sequence. Skipping the first read
      leaves the flag set forever.

   4. WRITE 0 TO CLEAR (rc_w0)
        TIM2->SR = ~TIM_SR_UIF;
      Note the complement: write ones everywhere EXCEPT the
      bit you are clearing. Writing 0 directly would clear
      every flag, including ones set since you read.

   Using the wrong idiom gives you an interrupt that fires
   forever, or a flag you never see. Check the "Access"
   column in the reference manual's register table - it is
   there for exactly this.                                  */
```

> **READ-TO-CLEAR REGISTERS AND DEBUGGERS DO NOT MIX**
> If a status flag clears on read, then a debugger's register view, a watch expression, or a memory window covering that address will clear it behind your back. The peripheral then behaves differently under observation. When a bug vanishes the moment you look at a peripheral register, this is nearly always why — close the register view and use a variable you copy the status into instead.

The initialisation order that always works

```
/* Get this order wrong and the peripheral silently ignores
   you. It is the same for every controller in this track.  */

/* 1. ENABLE THE PERIPHERAL CLOCK, and read it back.
      Without a clock, register writes are discarded and
      reads return zero (mcu chapter 09).                   */
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
(void) RCC->APB1ENR;

/* 2. CONFIGURE THE PINS - mode, alternate function, speed,
      pull. Do this BEFORE enabling the peripheral, or the
      first edge it drives comes from an unconfigured pin
      (mcu chapter 10).                                     */
gpio_config_af(GPIOA, 2, AF7, PUSHPULL, FAST);

/* 3. CONFIGURE THE PERIPHERAL WHILE IT IS DISABLED.
      Most controllers latch configuration only when going
      from disabled to enabled, and many IGNORE writes to
      config fields while enabled - silently.               */
USART2->CR1 = 0;                      /* ensure disabled   */
USART2->BRR = brr_for(115200);
USART2->CR2 = 0;                      /* 1 stop bit        */
USART2->CR3 = 0;

/* 4. ENABLE INTERRUPTS AT THE PERIPHERAL, then set priority
      and enable at the NVIC. This order matters if the line
      could already be active.                              */
USART2->CR1 |= USART_CR1_RXNEIE;
NVIC_SetPriority(USART2_IRQn, 6);
NVIC_EnableIRQ(USART2_IRQn);

/* 5. ENABLE THE PERIPHERAL LAST. */
USART2->CR1 |= USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;

/* THE RULE: clock, pins, config, interrupts, enable.
   Deviating from it produces "works sometimes" behaviour
   that depends on optimisation level and what the pin was
   doing beforehand.                                        */
```

Synchronous, asynchronous, and why it decides everything

```
SYNCHRONOUS - a clock line is transmitted alongside data.
  SPI, I2C, USB (embedded in the encoding), MIPI.

  The receiver samples on the clock edge the transmitter
  provided, so the two need no agreement about rate.
  Speed can be changed freely and can reach hundreds of
  MHz. Costs a pin.

    CLK  __|--|__|--|__|--|__
    DATA ___X====X====X====X_
             ^    ^    ^     sampled here

ASYNCHRONOUS - no clock line. Both ends must independently
  agree on the rate BEFORE any data flows.
  UART, RS-485, CAN.

  The receiver finds the start edge, then samples at
  pre-agreed intervals based on its OWN clock.

    DATA --|_______X===X===X===X---
           start   ^   ^   ^   ^
                   sampled on internal timing

  This is why UART needs an accurate clock and SPI does
  not (mcu chapter 09), why UART has a baud rate you must
  configure identically at both ends, and why UART tops
  out in the low megabits while SPI reaches 100 Mbit.

THE THIRD CATEGORY: self-clocking codes. CAN and USB
embed clock information in the data by guaranteeing
transitions - bit stuffing in CAN, NRZI plus stuffing in
USB. The receiver recovers the clock from the edges, so
you get asynchronous wiring with synchronous accuracy.
Chapters 09 and 12 cover these.
```

Error flags are the real interface

```
/* Every peripheral in this track has an error status you
   are supposed to check and most code ignores.            */

void USART2_IRQHandler(void)
{
    uint32_t isr = USART2->ISR;

    /* CHECK ERRORS FIRST. An overrun leaves RXNE set with
       stale data; handling the byte before the error means
       you process garbage and never clear the condition.  */
    if (isr & (USART_ISR_ORE | USART_ISR_FE |
               USART_ISR_NE  | USART_ISR_PE)) {
        if (isr & USART_ISR_ORE) { stats.overrun++; }
        if (isr & USART_ISR_FE)  { stats.framing++; }
        if (isr & USART_ISR_NE)  { stats.noise++;   }
        if (isr & USART_ISR_PE)  { stats.parity++;  }

        USART2->ICR = USART_ICR_ORECF | USART_ICR_FECF |
                      USART_ICR_NECF  | USART_ICR_PECF;
        (void) USART2->RDR;         /* discard the bad byte */
        return;
    }

    if (isr & USART_ISR_RXNE) {
        ring_put((uint8_t) USART2->RDR);
    }
}

/* WHAT EACH ERROR ACTUALLY TELLS YOU - this is diagnosis,
   not bookkeeping:

     OVERRUN   your software is too slow. The fix is DMA or
               a higher interrupt priority, not more retries.
     FRAMING   the stop bit was not high: baud rate mismatch,
               a clock error, or noise. Check both ends.
     NOISE     the three samples per bit disagreed. Electrical:
               long wires, no ground, missing termination.
     PARITY    a bit flipped. Rare, and means real corruption.

   Counting these in a struct you can print is one of the
   highest-value things you can add to a driver. Without it
   "the link is unreliable" is unactionable; with it you
   know within seconds whether the problem is your code,
   the baud rate, or the cable.                            */
```

How it shows up

Symptom	Cause
Register writes have no effect, reads give zero	Peripheral clock not enabled
Configuration ignored	Written while the peripheral was enabled
Last character of every message lost	Waited for TXE rather than TC before disabling
Interrupt fires continuously	Wrong clearing idiom for that status register
Bug vanishes when the debugger shows the register	Read-to-clear flag being cleared by the debugger
Flag never clears despite writing to it	Register is read-to-clear, or needs the two-step sequence

Symptom	Cause
All flags cleared when you meant to clear one	Wrote 0 to an `rc_w0` register instead of the complement
Receive stops permanently after a burst	Overrun set and never cleared — the flag blocks further reception
Spurious first byte on startup	Peripheral enabled before its pins were configured
"The link is unreliable"	No error counters — add them before doing anything else

Check yourself

- ✓ Name the four register surfaces every serial peripheral presents.
- ✓ Explain the difference between TXE and TC and the bug that confuses them.
- ✓ List the four flag-clearing idioms and how to tell which applies.
- ✓ Explain why read-to-clear registers misbehave under a debugger.
- ✓ Give the five-step initialisation order and what breaks if you reorder it.
- ✓ Explain why a synchronous bus needs no clock accuracy agreement.
- ✓ Say why UART tops out far below SPI in achievable rate.
- ✓ Explain what self-clocking codes achieve and name two buses that use them.
- ✓ Say what an overrun error tells you about your software, specifically.
- ✓ Distinguish framing, noise and parity errors as diagnostic signals.

UART: framing, baud rate and error budget

The oldest interface still in daily use, and the one where an arithmetic error of two percent turns working communication into garbage. This chapter covers the frame, the sampling, and the calculation that tells you in advance whether a given clock can reach a given baud rate.

The frame

```
Idle line is HIGH. One character, 8N1, value 0x41 ('A'):

     idle    start  b0 b1 b2 b3 b4 b5 b6 b7  stop  idle
----------+      +--+--+--+--+--+--+--+--+     +---------
          |      |  |  |  |  |  |  |  |  |     |
          +------+  +--+--+--+--+--+--+--+-----+
           START   1  0  0  0  0  0  1  0   STOP
                   <-- LSB FIRST -->

0x41 = 0100 0001 binary, sent LSB first: 1,0,0,0,0,0,1,0

THE START BIT IS THE WHOLE MECHANISM. It is the only
guaranteed edge, and everything the receiver does is
timed from it. An idle line and a start bit are
distinguishable because idle is high and start is low -
that is why the idle state is defined as high.

FRAME NOTATION: 8N1 = 8 data bits, No parity, 1 stop bit.
Others exist (7E1, 8N2, 9-bit for addressing) but 8N1 is
the overwhelming default.

OVERHEAD: 10 bits carry 8 bits of data. So
  bytes/second = baud / 10
115200 baud = 11520 bytes/s. This division by ten is the
most useful UART arithmetic you will do.
```

How the receiver stays aligned

```
The receiver has NO clock from the transmitter. It
oversamples - typically 16x the bit rate.

1. Wait for the falling edge (start bit).
2. Count 8 sample-clocks to reach the MIDDLE of the
   start bit. Verify it is still low; if not, it was
   noise - abandon and resume hunting.
3. Thereafter sample every 16 clocks, taking three
   samples around the centre (7, 8, 9) and majority-voting
   them. Disagreement sets the NOISE flag.
4. After the last data bit, check the stop bit is HIGH.
   If not: FRAMING ERROR.

     start bit
---+        +----
   |        |
   +--------+
   ^   ^^^  ^
   |   789  |
 edge  vote |
detected    next bit boundary

WHY ERROR ACCUMULATES: the receiver re-synchronises only
on the START bit. Over the following 9 bit times, any
difference between the two clocks accumulates. By the
stop bit the sampling point has drifted by 9.5 bit times
times the total error.

TOLERANCE: the sample must still land inside the correct
bit, so the accumulated drift must stay under half a bit:

  9.5 x total_error < 0.5
  total_error < 5.3%

In practice, with the three-sample vote needing all of
its window inside the bit, the usable budget is about
+/-2%, SHARED between the two ends. So roughly 1% each -
which is why the internal RC oscillator is marginal and
a crystal is not a luxury (mcu chapter 09).
```

Computing the divisor, and its error

```
/* The divisor is an integer, so almost no baud rate is
   exactly achievable. The question is always: how far off?  */

/*   BRR = f_pclk / baud       (rounded to nearest)
     actual_baud = f_pclk / BRR
     error = (actual - wanted) / wanted                    */

static uint16_t brr_for(uint32_t pclk, uint32_t baud)
{
    return (uint16_t)((pclk + baud / 2u) / baud);   /* round */
}

/* WORKED: 16 MHz PCLK, 115200 baud.

     16000000 / 115200 = 138.89
     rounded            = 139
     actual = 16000000 / 139 = 115108 baud
     error  = (115108 - 115200) / 115200 = -0.08%

   Excellent. Now 8 MHz PCLK, 115200 baud:

     8000000 / 115200 = 69.44
     rounded          = 69
     actual = 8000000 / 69 = 115942
     error  = +0.64%

   Still fine alone, but add a 1% RC oscillator at each end
   and you are at 2.6% - outside budget, and it will fail
   intermittently and temperature-dependently.

   Now the classic disaster, 8 MHz and 921600:

     8000000 / 921600 = 8.68
     rounded          = 9
     actual = 8000000 / 9 = 888889
     error  = -3.55%

   Broken before you add any oscillator error at all. The
   divisor is small, so rounding it costs an enormous
   fraction. THIS IS WHY HIGH BAUD RATES NEED HIGH CLOCKS:
   not for throughput, but for divisor RESOLUTION.         */

/* ALWAYS CHECK AT COMPILE TIME. */
#define BAUD        115200u
#define PCLK        16000000u
#define BRR_VAL     ((PCLK + BAUD/2u) / BAUD)
#define ACTUAL      (PCLK / BRR_VAL)
#define ERR_PPT     (((int32_t)ACTUAL - (int32_t)BAUD) * 1000 / (int32_t)BAUD)
_Static_assert(ERR_PPT < 15 && ERR_PPT > -15,
               "UART baud error exceeds 1.5% - pick another "
               "clock or baud rate");

/* Twelve lines that turn an intermittent field failure into
   a build error. Put this in every UART driver you write. */
```

WHY 115200, AND WHY THE ODD NUMBERS

The classic rates are divisions of 1.8432 MHz, the crystal fitted to the original 8250 UART, chosen precisely because it divides evenly to 110, 300, 1200, 9600, 19200, 38400, 57600 and 115200 with zero error. When you fit a 8 MHz or 16 MHz crystal instead you inherit rounding error at every rate. Parts intended for serial work often offer 7.3728 MHz or 11.0592 MHz crystals for this reason — they are 1.8432 MHz multiples, and they make the error exactly zero.

Fractional dividers and oversampling

```
/* Modern UARTs improve resolution two ways. */

/* 1. FRACTIONAL BRR. The divisor has a fractional part -
      on STM32, 4 bits of sixteenths.                      */
/*   BRR = mantissa << 4 | fraction
     16 MHz, 115200: 138.89 -> mantissa 138, fraction 14/16
                     = 138.875, actual = 115207 baud
                     error = +0.006%
   An order of magnitude better than integer rounding.     */

/* 2. OVERSAMPLING BY 8 INSTEAD OF 16. Halves the divisor,
      doubling the maximum baud rate for a given clock.     */
USART1->CR1 |= USART_CR1_OVER8;
/*   Costs noise immunity: with 8 samples per bit the vote
     window is narrower and the tolerance drops from about
     3.5% to 2%. Use it only when you need the rate and the
     link is short and clean.                              */

/* 3. AUTOBAUD. Some UARTs measure the rate from a known
      first character - typically 0x55 ('U'), which is
      alternating bits and gives the most edges to measure. */
USART1->CR2 |= USART_CR2_ABREN | USART_CR2_ABRMODE_0;
/*   Useful when the peer's rate is unknown or configurable.
     The peer must send the sync character first, so both
     ends need to agree on the protocol even if not the
     rate.                                                 */
```

Parity, and what it is worth

```
/* Parity adds one bit making the count of ones even or odd.
   It detects any ODD number of bit errors and misses every
   even number.                                              */

USART1->CR1 |= USART_CR1_PCE;      /* parity enable          */
USART1->CR1 &= ~USART_CR1_PS;      /* 0 = even, 1 = odd      */

/* THE TRAP: on most STM32 UARTs the parity bit is taken
   from the DATA LENGTH, not added to it. For 8 data bits
   plus parity you must set the word length to NINE:         */
USART1->CR1 |= USART_CR1_M0;       /* 9-bit word             */
/*   Forget this and you send 7 data bits plus parity, and
     every byte above 0x7F is corrupted while ASCII appears
     to work perfectly. A memorable afternoon.               */

/* IS IT WORTH IT? Rarely, on a modern link.
   - It costs 10% of your throughput (11 bits per byte).
   - It catches only single-bit errors, and real-world
     serial corruption is usually a burst that flips
     several bits.
   - Any protocol carrying data worth protecting should have
     a CRC over the whole message, which is strictly better.

   Use parity when a legacy peer demands it. For new
   designs, a CRC-16 over the frame gives far better
   detection for similar overhead on messages of any
   length.                                                   */
```

How it shows up

Symptom	Cause
Consistent garbage, same wrong bytes each time	Baud rate mismatch — a specific ratio, often 2x
Mostly correct with occasional wrong characters	Baud error near the tolerance limit; check the divisor arithmetic
Works at 9600, fails at 115200	Divisor resolution — the error grows as the divisor shrinks
Works warm, fails cold	RC oscillator drift pushed total error outside budget
Framing errors on every byte	Wrong stop-bit count, wrong word length, or inverted polarity
ASCII works, bytes above 0x7F corrupt	Parity enabled without extending word length to 9
Noise flag set frequently	Electrical: long unshielded wire, missing ground, or no common reference
Nothing received at all, TX works	TX and RX not crossed, or the peer expects a null modem crossover
Occasional single-character loss under load	Overrun — move to DMA or raise the ISR priority
Autobaud never locks	Peer's first character is not the expected sync pattern

Check yourself

- ✓ Draw an 8N1 frame for a given byte, LSB first, with correct idle levels.
- ✓ Explain why the idle state is high and what that buys the start bit.
- ✓ Convert a baud rate to bytes per second and say why the factor is ten.
- ✓ Describe the receiver's oversampling and majority vote.

- ✓ Derive the tolerance limit from the accumulated drift over 9.5 bit times.
- ✓ Compute the baud error for 8 MHz at 921600 and explain why it fails.
- ✓ Explain why high baud rates need high peripheral clocks.
- ✓ Say why 1.8432 MHz and its multiples appear on serial-oriented boards.
- ✓ State the cost of oversampling by 8 rather than 16.
- ✓ Explain the parity word-length trap and its characteristic symptom.

UART in practice: flow control, DMA and ring buffers

The frame from the previous chapter is the easy part. Making a UART link that does not drop bytes under load, recovers from a peer that disappears, and finds message boundaries in a byte stream is the engineering — and it is the same work for every stream protocol you will meet.

Why bytes get lost

```
A UART has ONE receive holding register (or a shallow
FIFO). At 115200 baud a byte arrives every 87 us. If your
software has not read the previous byte by then, it is
OVERWRITTEN and the OVERRUN flag sets.

87 us is a long time - about 14,000 cycles at 168 MHz -
which is why interrupt-per-byte works fine at this rate.

At 3 Mbaud a byte arrives every 3.3 us: 550 cycles. An
ISR with a 12-cycle entry, a prologue, a ring-buffer
write and an exit is perhaps 40 cycles, so it still fits
- until something disables interrupts for 10 us. Then you
lose three bytes and the protocol desynchronises.

WHAT DISABLES INTERRUPTS LONG ENOUGH TO HURT:
  - a flash write or erase (can stall the bus for MS)
  - a long critical section in another driver
  - a higher-priority ISR doing too much
  - a debugger halting the core

THE FIX IS NOT "OPTIMISE THE ISR". It is DMA, which
removes software from the per-byte path entirely. Once
the DMA controller is writing to memory, an interrupt
disabled for 10 us costs you nothing at all.
```

The ring buffer, done right

```
/* Single producer (ISR), single consumer (task). Power-of-
   two size so the mask is free. No lock needed - the
   argument is in the mcu track, chapter 17.               */

#define RXSZ 512                       /* MUST be 2^n       */
typedef struct {
    uint8_t  buf[RXSZ];
    volatile uint16_t head;            /* written by ISR    */
    volatile uint16_t tail;            /* written by task   */
    volatile uint32_t dropped;
} ring_t;

static inline int ring_put(ring_t *r, uint8_t c)
{
    uint16_t next = (uint16_t)((r->head + 1u) & (RXSZ - 1u));
    if (next == r->tail) {
        r->dropped++;                  /* COUNT it          */
        return -1;
    }
    r->buf[r->head] = c;
    r->head = next;                    /* publish AFTER     */
    return 0;
}

static inline int ring_get(ring_t *r, uint8_t *c)
{
    if (r->tail == r->head) { return -1; }
    *c = r->buf[r->tail];
    r->tail = (uint16_t)((r->tail + 1u) & (RXSZ - 1u));
    return 0;
}

/* THREE THINGS THAT MAKE THIS CORRECT:

   1. head is published AFTER the data byte is stored. The
      consumer therefore never sees an index for a byte not
      yet written.

   2. The buffer is full when next == tail, which wastes one
      slot but avoids the ambiguity of head == tail meaning
      both empty and full. The alternative - a separate
      count - is a variable BOTH sides write, and needs a
      lock.

   3. dropped is counted, not ignored. A link that silently
      discards data under load is unbudgetable; one that
      reports it tells you exactly how much bigger the
      buffer needs to be.

   SIZING: buffer must cover the longest time the consumer
   can be delayed, times the byte rate.
     115200 baud = 11.5 bytes/ms
     consumer delayed up to 20 ms
     -> 230 bytes minimum. Round up to 512.              */
```

Circular DMA plus IDLE: the design worth memorising

```
/* This combination receives continuously, at any rate, with
   no per-byte software, AND tells you where messages end.
   It is the standard answer for a serious UART link.      */

static uint8_t dma_buf[512];
static uint16_t last_pos;

void uart_dma_rx_init(void)
{
    /* Circular DMA: NDTR reloads and the address wraps. */
    DMA1_Stream5->PAR  = (uint32_t) &USART2->RDR;
    DMA1_Stream5->M0AR = (uint32_t) dma_buf;
    DMA1_Stream5->NDTR = sizeof(dma_buf);
    DMA1_Stream5->CR   = (4u << 25) | DMA_SxCR_MINC
                       | DMA_SxCR_CIRC | DMA_SxCR_TCIE
                       | DMA_SxCR_HTIE;
    DMA1_Stream5->CR  |= DMA_SxCR_EN;

    USART2->CR3 |= USART_CR3_DMAR;
    USART2->CR1 |= USART_CR1_IDLEIE;   /* THE KEY PIECE     */
}

/* IDLE fires when the line has been quiet for one full
   character time after data. That is exactly "the message
   ended", for any protocol with gaps between messages.   */
void USART2_IRQHandler(void)
{
    if (USART2->ISR & USART_ISR_IDLE) {
        USART2->ICR = USART_ICR_IDLECF;
        uart_process_upto(sizeof(dma_buf) - DMA1_Stream5->NDTR);
    }
}

/* NDTR counts DOWN from the buffer size, so
     bytes_written_so_far = size - NDTR
   and the new data is everything between last_pos and that
   position, handling the wrap.                            */
static void uart_process_upto(uint16_t pos)
{
    if (pos == last_pos) { return; }

    if (pos > last_pos) {
        parse(&dma_buf[last_pos], pos - last_pos);
    } else {                                  /* wrapped    */
        parse(&dma_buf[last_pos], sizeof(dma_buf) - last_pos);
        if (pos > 0) { parse(&dma_buf[0], pos); }
    }
    last_pos = pos;
}

/* Also call uart_process_upto from the DMA half-transfer
   and transfer-complete interrupts, so a message longer
   than half the buffer is processed before it is
   overwritten. Three interrupt sources, one function.

   RESULT: zero per-byte CPU, message boundaries detected in
   hardware, and a link that survives the CPU being busy for
   milliseconds. This handles 3 Mbaud comfortably on a part
   that could not manage 921600 with per-byte interrupts.  */
```

> **IDLE DETECTION IS THE ANSWER TO "HOW DO I KNOW THE MESSAGE ENDED?"**
> A UART is a byte stream with no framing above the character. Every protocol on top must delimit messages somehow, and there are only four options: a fixed length, a length field near the start, a terminator byte, or a timing gap. IDLE detection implements the last of these in hardware, for free, and it is why Modbus RTU (chapter 14) specifies a 3.5-character silent interval as its frame delimiter — it was designed for exactly this mechanism.

Flow control

```
HARDWARE (RTS/CTS) - two extra wires, and the only kind
that works reliably.

  Device A                    Device B
  RTS (out) ----------------> CTS (in)
  CTS (in)  <---------------- RTS (out)

  "RTS" now means "I am Ready To Receive" (the name is
  historical and misleading). A drives RTS low when its
  buffer has room; B's UART checks CTS before starting
  each character and pauses if it is high.

  ENTIRELY IN HARDWARE once enabled:
    USART1->CR3 |= USART_CR3_RTSE | USART_CR3_CTSE;

  The peripheral asserts RTS from its own FIFO level, so
  it works even if your software is completely stalled.
  This is the only flow control that survives a
  millisecond-long interrupt lockout.

SOFTWARE (XON/XOFF) - in-band, no extra wires.
  Receiver sends 0x13 (XOFF) to pause, 0x11 (XON) to
  resume. Problems:
    - those two byte values can never appear in your data,
      so it is unusable for binary
    - the XOFF itself takes a character time to send and
      another to be noticed, so several bytes still
      arrive after it
    - if XON is lost, the link deadlocks permanently
  Use only with legacy equipment that requires it.

NONE - the default, and correct when the receiver is
  always faster than the sender, or the protocol has its
  own acknowledgement and retry at a higher layer.
```

Making the link survive reality

```
/* Four things that separate a driver that works on the
   bench from one that works in a cabinet for five years. */

/* 1. NEVER BLOCK FOREVER. */
int uart_send(const uint8_t *p, size_t n, uint32_t timeout_ms)
{
    uint32_t deadline = g_ms + timeout_ms;
    while (n--) {
        while (!(USART2->ISR & USART_ISR_TXE)) {
            if ((int32_t)(g_ms - deadline) >= 0) { return -ETIMEDOUT; }
        }
        USART2->TDR = *p++;
    }
    return 0;
}

/* 2. RECOVER FROM ERRORS, DO NOT JUST COUNT THEM.
      An overrun on some parts BLOCKS further reception
      until cleared. If your error path forgets to clear it,
      the link dies permanently after the first burst.     */
static void uart_error_recover(void)
{
    uint32_t isr = USART2->ISR;
    if (isr & (USART_ISR_ORE|USART_ISR_FE|USART_ISR_NE|USART_ISR_PE)) {
        USART2->ICR = USART_ICR_ORECF | USART_ICR_FECF
                    | USART_ICR_NECF  | USART_ICR_PECF;
        (void) USART2->RDR;
        stats.resyncs++;
        parser_reset();          /* the stream is desynced  */
    }
}

/* 3. RESYNCHRONISE THE PARSER. After any error the byte
      stream has lost alignment. A parser that keeps its
      state machine across an error will mis-frame every
      subsequent message. Reset to hunting-for-header.     */

/* 4. TIME OUT A PARTIAL MESSAGE. If a header arrives and
      the rest never does, the parser must not wait
      indefinitely holding a buffer.                       */
if (parser.state != HUNTING &&
    (int32_t)(g_ms - parser.started_ms - 100) >= 0) {
    stats.partial_timeouts++;
    parser_reset();
}

/* Every one of these four is learned the hard way. A UART
   link with no timeout, no error recovery, no resync and
   no partial-message timeout will work perfectly until the
   day a connector is jostled, and then stay broken until
   someone power-cycles it.                                */
```

How it shows up

Symptom	Cause
Link dies permanently after a burst	Overrun flag never cleared — it blocks further reception
Bytes lost only when writing to flash	Flash operations stall the bus; use DMA for receive

Symptom	Cause
Every message after an error is mis-framed	Parser state not reset on error
Task blocks forever holding a lock	A send with no timeout and a peer that stopped reading
Ring buffer occasionally corrupt	Head published before the data, or two producers
Buffer never reports full but data is missing	Drops not counted — add the counter
Messages merge together	No delimiter; use IDLE detection or a length field
DMA works but long messages corrupt	Half-transfer interrupt not handled — data overwritten before processing
XON/XOFF corrupts binary data	0x11 and 0x13 appearing in the payload
Flow control enabled but bytes still lost	RTS/CTS wired straight through instead of crossed

Check yourself

- ✓ Compute the inter-byte time at 3 Mbaud and say why DMA becomes necessary.
- ✓ Explain why DMA makes a 10 µs interrupt lockout harmless.
- ✓ Give the three properties that make the single-producer ring buffer correct.
- ✓ Explain why the buffer wastes one slot rather than using a count.
- ✓ Size a receive buffer from baud rate and worst-case consumer delay.
- ✓ Write the expression for bytes received so far from `NDTR`.
- ✓ Explain what IDLE detection does and name the four ways to delimit messages.
- ✓ Explain why RTS/CTS works when software flow control does not.
- ✓ Say why XON/XOFF cannot carry binary data.
- ✓ List the four robustness measures a shipping UART driver needs.

SPI: modes 0-3, chip select and timing

SPI is the simplest bus in this track and the one most often mis-wired, because it has no addressing, no acknowledgement, no error detection, and four clock configurations of which exactly one is right for your device.

Four wires and a shift register

```
MASTER                          SLAVE
+----------+                    +----------+
|  SCLK  --+------------------->-+-- SCLK   |
|  MOSI  --+------------------->-+-- MOSI   |
|  MISO  <-+-------------------<-+-- MISO   |
|  CS    --+------------------->-+-- CS     |  active LOW
+----------+                    +----------+

IT IS ONE CIRCULAR SHIFT REGISTER SPANNING BOTH CHIPS:

  master reg [10110010] --MOSI--> slave reg [01001101]
                 ^                              |
                 +---------MISO-----------------+

Every clock edge shifts one bit out of each and into the
other. After 8 clocks the two bytes have been EXCHANGED.

CONSEQUENCE THAT SURPRISES EVERYONE: you cannot receive
without transmitting. To read a byte you must clock out
a dummy byte - conventionally 0x00 or 0xFF. There is no
"read" operation, only "exchange".

uint8_t spi_xfer(uint8_t out) {
    while (!(SPI1->SR & SPI_SR_TXE)) { }
    *(volatile uint8_t *)&SPI1->DR = out;
    while (!(SPI1->SR & SPI_SR_RXNE)) { }
    return *(volatile uint8_t *)&SPI1->DR;
}
uint8_t v = spi_xfer(0xFF);      /* "read" */

NO ADDRESSING - selection is the CS pin, so N devices
need N chip-select pins.
NO ACKNOWLEDGEMENT - the master cannot tell whether a
slave is present, powered, or listening.
NO ERROR DETECTION - none at all.
NO SPEED NEGOTIATION - you must know the slave's limit.

In exchange: full duplex, trivially simple, and fast.
50-100 MHz is routine.
```

The four modes

```
Two independent bits give four combinations:

CPOL - clock polarity: idle level of SCLK
       0 = idle low,  1 = idle high
CPHA - clock phase:   which edge SAMPLES
       0 = sample on the FIRST edge
       1 = sample on the SECOND edge

MODE 0  CPOL=0 CPHA=0   sample on RISING    <- most common
  SCLK  ___|--|__|--|__
  MOSI  X====X====X====
        ^    ^    ^        sampled on rising edge

MODE 1  CPOL=0 CPHA=1   sample on FALLING
  SCLK  ___|--|__|--|__
  MOSI  X====X====X====
           ^    ^          sampled on falling edge

MODE 2  CPOL=1 CPHA=0   idle high, sample FALLING
  SCLK  ---|__|--|__|--
        ^    ^    ^

MODE 3  CPOL=1 CPHA=1   idle high, sample RISING
  SCLK  ---|__|--|__|--
           ^    ^

MODE 0 and MODE 3 cover the large majority of devices.

THE RULE: data is SET UP on one edge and SAMPLED on the
other. Both ends must agree, and the datasheet always
states it - usually as a timing diagram rather than a
mode number, so you may have to read the diagram and
derive the mode yourself.

WRONG MODE SYMPTOMS, and they are diagnostic:
  - every byte shifted by one bit (0x55 reads as 0xAA
    or 0x2A) -> CPHA is wrong
  - first byte wrong, rest correct -> CPHA wrong, and
    the device recovered
  - all 0x00 or all 0xFF -> often CS, wiring, or power,
    not mode
  - works at low speed, fails fast -> timing, not mode
    (see below)
```

Configuring it

```
void spi_init(uint32_t hz)
{
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
    (void) RCC->APB2ENR;

    /* PA5 SCK, PA6 MISO, PA7 MOSI - AF5. CS is a plain GPIO
       we drive ourselves (see below).                      */
    gpio_af(GPIOA, 5, 5); gpio_af(GPIOA, 6, 5); gpio_af(GPIOA, 7, 5);
    gpio_output(GPIOB, 6);                 /* CS            */
    GPIOB->BSRR = (1u << 6);               /* idle HIGH     */

    /* Prescaler: PCLK2 / 2^(BR+1). At 84 MHz PCLK2,
       BR=3 gives 84/16 = 5.25 MHz.                         */
    uint32_t br = 0;
    while ((84000000u >> (br + 1)) > hz && br < 7) { br++; }

    SPI1->CR1 = SPI_CR1_MSTR              /* master         */
              | (br << 3)
              | SPI_CR1_SSM | SPI_CR1_SSI /* software NSS   */
              /* CPOL=0, CPHA=0 -> mode 0: both bits clear  */
              ;
    SPI1->CR2 = 0;                        /* 8-bit          */
    SPI1->CR1 |= SPI_CR1_SPE;
}

/* SSM + SSI IS NOT OPTIONAL. Without software slave
   management the hardware NSS input, if it goes low,
   silently demotes the peripheral from master to slave and
   everything stops. This is one of the most common
   first-time SPI failures on STM32, and the symptom -
   "MOSI never toggles" - gives no hint of the cause.      */
```

> **DRIVE CHIP SELECT YOURSELF, IN SOFTWARE**
> Hardware CS on most microcontrollers deasserts between bytes, which breaks any device expecting a multi-byte transaction to be framed by one continuous CS assertion — which is almost all of them. A GPIO you control explicitly lets you hold CS low across a command byte, an address and a data burst. It also lets you meet the setup and hold times a datasheet specifies around the CS edges.

A transaction, with the timing that matters

```
/* Reading a register from a typical sensor. */

uint8_t sensor_read_reg(uint8_t reg)
{
    uint8_t v;

    CS_LOW();
    delay_ns(50);              /* t_CSS: CS setup before
                                  the first clock edge     */

    (void) spi_xfer(reg | 0x80);   /* bit 7 = read         */
    v = spi_xfer(0xFF);             /* dummy out, data in   */

    /* CRITICAL: wait for the shift register to EMPTY before
       raising CS. TXE means the DATA REGISTER is free, not
       that the byte has left the pin (chapter 01).        */
    while (SPI1->SR & SPI_SR_BSY) { }
    delay_ns(50);              /* t_CSH: CS hold           */
    CS_HIGH();

    return v;
}

/* RAISING CS ON TXE INSTEAD OF BSY TRUNCATES THE LAST BYTE.
   The symptom is a device that mostly works but ignores
   the final byte of every write - so single-register
   writes fail while burst writes appear to work except for
   the last register. Memorably confusing.

   BURST TRANSFERS: most devices auto-increment their
   internal address, so one CS assertion can read many
   registers. This is much faster than N transactions and
   is how you read a FIFO:                                 */

void sensor_read_burst(uint8_t start, uint8_t *dst, size_t n)
{
    CS_LOW();
    (void) spi_xfer(start | 0xC0);    /* read + increment  */
    while (n--) { *dst++ = spi_xfer(0xFF); }
    while (SPI1->SR & SPI_SR_BSY) { }
    CS_HIGH();
}
```

Why it works slowly and fails fast

```
The most common SPI failure after mode errors.

THE ROUND TRIP that limits speed:

  master drives SCLK edge
    -> propagates down the trace        (~5 ns / metre...
                                         but 1-2 ns typical)
    -> slave's t_V: time to put data
       on MISO after the edge           (often 20-40 ns!)
    -> propagates back
    -> must arrive before the master's
       setup time at the NEXT edge

  So:  t_prop_out + t_V + t_prop_back + t_setup < half a
       clock period

  With t_V = 30 ns and small propagation, half a period
  must exceed ~35 ns, so the period is 70 ns and the
  maximum clock is about 14 MHz - regardless of what the
  master can generate.

THIS IS WHY "it works at 1 MHz and fails at 20 MHz" is
almost never a software problem. Check:
  - the slave's maximum SCLK in its datasheet
  - t_V (output valid delay) which often binds first
  - trace length, especially through a connector or
    ribbon cable
  - whether MISO has a weak pull-up fighting the driver

RULES OF THUMB:
  - on-board, short traces: 10-25 MHz usually fine
  - through a connector or 10 cm of ribbon: 1-4 MHz
  - always bring a new device up at 1 MHz, confirm it
    works, THEN raise the clock while watching for
    errors. Starting fast conflates wiring bugs with
    timing bugs.
```

How it shows up

Symptom	Cause
Every byte shifted by one bit	Wrong CPHA
Reads return all 0x00	MISO not connected, device unpowered, or CS never asserted
Reads return all 0xFF	MISO floating with a pull-up; the device is not driving
MOSI and SCLK never toggle	NSS pulled low without `SSM` / `SSI` — demoted to slave
Last byte of every write ignored	CS raised on TXE instead of waiting for `BSY`
Multi-byte transactions fail, single bytes work	Hardware CS deasserting between bytes
Works at 1 MHz, fails at 10 MHz	Slave's output valid delay or trace length — a timing budget
Works with a scope probe attached	The probe's capacitance changed edge timing — you are marginal
Two devices, one always fails	Both chip selects asserted, or a shared CS
Data corrupt only during other activity	DMA or interrupt disturbing a bit-banged CS, or a shared bus without a lock

Check yourself

- ✓ Explain why SPI cannot read without writing.
- ✓ Define CPOL and CPHA and give the four mode combinations.
- ✓ Say which two modes cover most devices.
- ✓ Give the diagnostic symptom of a wrong CPHA setting.
- ✓ Explain why chip select should be a software GPIO.
- ✓ Explain what `SSM` and `SSI` do and the failure without them.
- ✓ Explain why you must wait for `BSY` rather than TXE before raising CS.
- ✓ Describe the round-trip timing budget that limits SPI clock speed.
- ✓ Explain why a device may work at 1 MHz and fail at 20 MHz.
- ✓ Give the bring-up procedure for a new SPI device.

SPI in practice: DMA, multi-slave and quad SPI

SPI's simplicity means all the engineering lives outside the protocol: how to move a megabyte without occupying the CPU, how to share one bus between drivers that do not know about each other, and what changes when the bus grows to four data lines.

Full-duplex DMA

```
/* SPI always exchanges, so a DMA transfer needs BOTH
   directions configured even if you only care about one.
   Miss the RX stream and the receive FIFO overruns, which
   on most parts wedges the peripheral.                    */

static uint8_t tx_buf[256], rx_buf[256];

void spi_dma_xfer(const uint8_t *tx, uint8_t *rx, size_t n)
{
    /* RX FIRST. It must be armed before any clock edge, or
       the first bytes arrive with nowhere to go.          */
    DMA2_Stream0->PAR  = (uint32_t) &SPI1->DR;
    DMA2_Stream0->M0AR = (uint32_t) rx;
    DMA2_Stream0->NDTR = n;
    DMA2_Stream0->CR   = (3u << 25) | DMA_SxCR_MINC
                       | DMA_SxCR_TCIE;      /* periph->mem */
    DMA2_Stream0->CR  |= DMA_SxCR_EN;

    DMA2_Stream3->PAR  = (uint32_t) &SPI1->DR;
    DMA2_Stream3->M0AR = (uint32_t) tx;
    DMA2_Stream3->NDTR = n;
    DMA2_Stream3->CR   = (3u << 25) | DMA_SxCR_MINC
                       | DMA_SxCR_DIR_0;     /* mem->periph */
    DMA2_Stream3->CR  |= DMA_SxCR_EN;

    CS_LOW();
    SPI1->CR2 |= SPI_CR2_RXDMAEN | SPI_CR2_TXDMAEN;
}

/* COMPLETION IS THE RX INTERRUPT, NOT TX. TX completes when
   the last byte enters the shift register; RX completes
   when the last byte has fully arrived. Raising CS on TX
   completion truncates the transfer (chapter 04).        */

void DMA2_Stream0_IRQHandler(void)
{
    DMA2->LIFCR = DMA_LIFCR_CTCIF0;
    while (SPI1->SR & SPI_SR_BSY) { }     /* belt and braces */
    CS_HIGH();
    SPI1->CR2 &= ~(SPI_CR2_RXDMAEN | SPI_CR2_TXDMAEN);
    xfer_done = 1;
}

/* WRITE-ONLY TRANSFERS still need somewhere for the
   incoming bytes to go. Two options:
     - a real rx buffer you ignore, or
     - a ONE-BYTE dummy with MINC cleared, so every incoming
       byte overwrites the same location. This is the
       idiomatic trick and it costs one byte of RAM:      */
DMA2_Stream0->M0AR = (uint32_t) &dummy;
DMA2_Stream0->CR   = (3u << 25) | DMA_SxCR_TCIE;  /* no MINC */

/* READ-ONLY transfers use the mirror image: a single
   constant 0xFF source with MINC cleared on the TX stream. */
```

CORTEX-M7 CACHE MAINTENANCE APPLIES HERE IN FULL

Before a transmit DMA, clean the cache so the controller sees your data. After a receive DMA, invalidate so the CPU does not read stale lines. Both operations work on cache-line granularity (32 bytes), so a buffer that shares a cache line with another variable can have that variable clobbered by the invalidate. Align DMA buffers to 32 bytes and pad their length to a multiple of 32 — the alternative is a corruption bug that depends on the linker's variable placement.

Sharing the bus

```
MULTI-SLAVE: one bus, N chip selects.

           +-------- SCLK/MOSI/MISO --------+
           |            |            |      |
MASTER  CS1|         CS2|         CS3|      |
  |--------+            |            |      |
  |---------------------+            |      |
  |----------------------------------+      |
                                        sensor A
                                        sensor B
                                        flash

THE RULE: exactly one CS low at any instant. Two devices
driving MISO simultaneously is a short between two
push-pull outputs.

THE PROBLEM: devices may need DIFFERENT modes and speeds.
Sensor A wants mode 0 at 10 MHz; the flash wants mode 3
at 50 MHz. The controller has one configuration.

THE SOLUTION: reconfigure per transaction, under a lock.
This is exactly what Linux's SPI subsystem does with
spi_transfer and per-device spi_device structs (see the
Linux track, chapter 19) - and the reason that
abstraction exists is this problem.
```

```
/* A bus manager. In an RTOS this is a mutex; bare metal, a
   flag plus a critical section.                          */

typedef struct {
    uint32_t cr1;               /* mode + speed for this   */
    GPIO_TypeDef *cs_port;
    uint16_t      cs_pin;
} spi_device_t;

static SemaphoreHandle_t spi_lock;

int spi_transact(const spi_device_t *dev,
                 const uint8_t *tx, uint8_t *rx, size_t n)
{
    if (xSemaphoreTake(spi_lock, pdMS_TO_TICKS(100)) != pdTRUE) {
        return -EBUSY;          /* TIMEOUT, not forever    */
    }

    /* Reconfigure for THIS device. SPE must be off to
       change mode - the peripheral latches CR1 on enable
       (chapter 01).                                       */
    SPI1->CR1 &= ~SPI_CR1_SPE;
    SPI1->CR1  = dev->cr1;
    SPI1->CR1 |= SPI_CR1_SPE;

    dev->cs_port->BSRR = dev->cs_pin << 16;   /* CS low     */
    int r = spi_xfer_blocking(tx, rx, n);
    while (SPI1->SR & SPI_SR_BSY) { }
    dev->cs_port->BSRR = dev->cs_pin;         /* CS high    */

    xSemaphoreGive(spi_lock);
    return r;
}

/* WITHOUT THE LOCK: two tasks interleave, the mode changes
   mid-transaction, and two chip selects end up low at once.
   The symptom is corruption that appears only when both
   devices are used concurrently - which is to say, only in
   the integrated system and never in the unit test.       */
```

Daisy chaining

```
Some devices - shift registers, LED drivers, some ADCs -
support chaining, so N devices need only ONE chip select.

MASTER MOSI -->[ dev 1 ]-->[ dev 2 ]-->[ dev 3 ]--> MISO
                    ^           ^           ^
                    +-----------+-----------+
                            CS (shared)

Data shifts THROUGH each device into the next. To load
three 8-bit devices you clock 24 bits with CS low, then
raise CS - which latches each device's own byte.

THE ORDER IS REVERSED: the FIRST byte you send ends up in
the LAST device, because it has been pushed furthest
along the chain.

  uint8_t frame[3] = { dev3_data, dev2_data, dev1_data };
  CS_LOW();
  spi_write(frame, 3);
  while (SPI1->SR & SPI_SR_BSY) { }
  CS_HIGH();          /* latches all three at once      */

Getting the order backwards is the universal first bug,
and it looks like the devices are wired in the wrong
sequence.

TRADE-OFF: one CS pin and simultaneous update, against
having to rewrite the whole chain to change one device
and a maximum length set by accumulated propagation
delay.
```

Quad SPI

```
/* When SPI is used for program storage, one data line is
   the bottleneck. Quad SPI uses four bidirectional lines. */

/*   SPI      1 bit  per clock
     DUAL     2 bits per clock
     QUAD     4 bits per clock
     OCTAL    8 bits per clock  (xSPI / HyperBus)

   At 100 MHz: single = 12.5 MB/s, quad = 50 MB/s.

   THE KILLER FEATURE IS MEMORY-MAPPED MODE. The controller
   translates bus reads into flash commands automatically,
   so external flash appears at an address:               */

QUADSPI->CCR = QUADSPI_CCR_FMODE       /* memory-mapped    */
             | (3u << 24)              /* 4-line data      */
             | (2u << 8)               /* 1-line address   */
             | 0xEB;                   /* fast read quad   */

/* Now this just works, executing directly from external
   flash (execute-in-place, mcu chapter 01):              */
const uint8_t *font = (const uint8_t *) 0x90000000;
uint8_t glyph = font[offset];

/* WHAT IT COSTS:
   - reads are slower than internal flash, so cache matters
     enormously; on M7 enable the D-cache for that region
   - the first access after idle pays the command overhead
   - you CANNOT write while executing from it, so a
     firmware update to QSPI must run from internal flash
     or RAM (the .ramfunc trick, mcu chapter 05)
   - erase takes hundreds of milliseconds during which
     nothing can read it

   That last point is the design constraint people miss:
   put code and rarely-changing assets in QSPI, and never
   put anything you must write during normal operation
   alongside code you must execute.                       */
```

How it shows up

Symptom	Cause
SPI wedges after the first DMA transfer	RX DMA not configured; the receive FIFO overran
Last bytes of a DMA transfer lost	CS raised on TX completion rather than RX
DMA corrupts a nearby variable on M7	Cache invalidate hit a shared cache line — align and pad buffers to 32 bytes
Corruption only when two devices are used together	No bus lock; mode changed mid-transaction
Mode change has no effect	Written while `SPE` was set
Two devices respond at once	Two chip selects low — check every CS is deasserted on init
Daisy chain devices show each other's data	Byte order reversed — the first byte sent travels furthest
QSPI memory-mapped reads return garbage	Wrong read command opcode or dummy-cycle count for that flash
System hangs during a QSPI erase	Code executing from the flash being erased
Throughput far below the clock rate	Per-byte CPU overhead; the bus is idle between bytes

Check yourself

- ✓ Explain why a write-only SPI DMA transfer still needs an RX stream.
- ✓ Give the one-byte trick for discarding received data.
- ✓ Say which DMA completion signals the end of a transfer, and why.
- ✓ Explain the cache-line hazard for DMA buffers on Cortex-M7.
- ✓ State the invariant for chip selects on a shared bus.
- ✓ Explain why per-device reconfiguration needs a lock.
- ✓ Explain why `SPE` must be cleared to change mode.
- ✓ Describe daisy chaining and the byte-order rule.
- ✓ Say what memory-mapped QSPI enables and give its throughput advantage.
- ✓ Explain why a firmware update to QSPI cannot execute from QSPI.

I2C: start, stop, addressing and ACK

Two wires, up to 127 devices, and an acknowledgement on every byte — I2C gives you far more than SPI for two fewer pins, and charges for it in protocol complexity and a set of failure modes that have no SPI equivalent.

Two wires, open drain

```
                  VDD
                   |
             +-----+------+
             |            |
           [Rp]          [Rp]        2.2k - 4.7k
             |            |
SDA ---------+------------|----------+----------+
SCL ---------------------- +----------|----------|--+
             |            |          |          |  |
         +---+---+     +---+---+  +---+---+  +---+--+--+
         |MASTER |     |slave 1|  |slave 2|  | slave 3 |
         +-------+     +-------+  +-------+  +---------+

EVERY DEVICE IS OPEN-DRAIN: it can pull a line LOW, or
release it. Nobody ever DRIVES a line high - the pull-up
resistors do that (mcu chapter 10).

THIS ONE DECISION EXPLAINS ALMOST EVERYTHING ABOUT I2C:

- Two devices can transmit simultaneously without damage,
  because low always wins. That makes ARBITRATION
  possible (chapter 07).
- A slave can hold SCL low to say "wait", which is CLOCK
  STRETCHING (chapter 07).
- A receiver can pull SDA low during the 9th bit to
  ACKNOWLEDGE.
- Rise time is set by Rp and the bus capacitance, so the
  RESISTOR VALUE LIMITS THE SPEED. This is why the
  internal pull-ups (~40 kO) cannot do 400 kHz.

  t_rise ~ 0.8 x Rp x C_bus
  400 kHz needs t_rise < 300 ns
  With C_bus = 100 pF:  Rp < 3.75 kO

SPEEDS: 100 kHz standard, 400 kHz fast, 1 MHz fast-plus,
3.4 MHz high-speed (rare, needs a current-source pull-up).
```

Start, stop, and the bytes between

```
START and STOP are the only times SDA changes while SCL
is HIGH. During data, SDA may only change while SCL is
LOW. That distinction is the entire framing mechanism.

START:  SDA falls while SCL high
STOP:   SDA rises while SCL high

  SDA  ---+                          +---
          |                          |
          +--------------------------+
  SCL  ------+                    +------
             |                    |
             +--------------------+
           ^                        ^
        START                    STOP

A COMPLETE WRITE OF ONE BYTE TO REGISTER 0x10:

S | 0x76<<1|W | A | 0x10 | A | 0x42 | A | P
^       ^       ^    ^      ^    ^     ^   ^
|       |       |    |      |    |     |   stop
|       |       |    |      |    |     slave ACKs data
|       |       |    |      |    data byte
|       |       |    |      slave ACKs register
|       |       |    register address
|       |       slave ACKs its address
|       address + direction bit (0 = write)
start

THE 9TH BIT OF EVERY BYTE IS THE ACKNOWLEDGEMENT. The
transmitter releases SDA; the receiver pulls it LOW to
say "got it". A high 9th bit is a NACK.

ACK = 0 (SDA low). This inversion catches everyone.
```

The repeated start

```
Reading a register needs BOTH directions: write the
register number, then read the value. A STOP between them
would release the bus and let another master interleave -
so I2C provides a REPEATED START, which changes direction
without releasing.

S | addr|W | A | 0x10 | A | Sr | addr|R | A | data | N | P
                            ^                      ^
                            |                      master
                            repeated start,        NACKs the
                            no STOP first          LAST byte

THE MASTER NACKS THE FINAL BYTE IT READS. This is how it
says "stop sending" - the slave sees the NACK and
releases SDA so the master can generate a STOP. ACK the
last byte instead and the slave keeps driving SDA,
the STOP cannot be generated, and the bus hangs.

Multi-byte read: ACK every byte except the last.

  for (i = 0; i < n; i++) {
      data[i] = i2c_read_byte(i < n - 1);   /* ack flag */
  }

THE ALTERNATIVE some code uses - stop, then start again -
works with most devices and fails with some, because it
allows another master in between and because a few
devices reset their internal address pointer on STOP.
Use a repeated start; it is what the datasheet's timing
diagram shows.
```

Addressing

```
/* 7-bit addresses, shifted left, with the direction bit in
   position 0. This shift is the single most common source
   of confusion in I2C code.                              */

/*   Datasheet says: "the device address is 0x76"
     On the wire:     0xEC for write (0x76 << 1 | 0)
                      0xED for read  (0x76 << 1 | 1)

   Some datasheets quote the SHIFTED value, so "address
   0xEC" means the same device. If a scan finds a device at
   half your expected address - or double it - this is why.

   ALWAYS SCAN. Twelve lines that answer "is it even there":  */

void i2c_scan(void)
{
    for (uint8_t a = 0x08; a < 0x78; a++) {   /* reserved
                                                 outside     */
        i2c_start();
        int ack = i2c_send_byte(a << 1);      /* write        */
        i2c_stop();
        if (ack) { printf("found 0x%02X\r\n", a); }
    }
}

/* RESERVED ADDRESSES - do not use, and do not scan:
     0x00       general call / start byte
     0x01       CBUS
     0x02-0x03  reserved
     0x04-0x07  Hs-mode master code
     0x78-0x7B  10-bit addressing
     0x7C-0x7F  reserved

   ADDRESS COLLISIONS are I2C's practical limit. Many sensors
   offer only two addresses via an ADDR pin, so three
   identical sensors will not fit on one bus. Solutions:
     - a second I2C peripheral
     - an I2C multiplexer (TCA9548A and similar)
     - devices with more address pins
   Discovering this after the PCB is made is a common and
   expensive mistake - check address availability during
   schematic review, not bring-up.                         */
```

NACK IS INFORMATION, NOT FAILURE

A NACK on the address byte means no device answered — wrong address, unpowered, or not connected. A NACK on a data byte means the device is refusing, usually because it is busy: an EEPROM performing an internal write cycle NACKs its address until the write completes, and the correct response is to retry. This is called acknowledge polling, and it is the documented way to wait for an EEPROM write rather than a fixed delay. Treat NACK as a status code with a meaning that depends on when it arrived.

A blocking master transaction

```
/* Every step has a timeout. An I2C bus with a confused
   slave WILL hang otherwise, and hang forever.            */

#define I2C_TIMEOUT_MS 10

static int wait_flag(volatile uint32_t *r, uint32_t m, int set)
{
    uint32_t deadline = g_ms + I2C_TIMEOUT_MS;
    while (((*r & m) != 0) != (set != 0)) {
        if ((int32_t)(g_ms - deadline) >= 0) { return -ETIMEDOUT; }
    }
    return 0;
}

int i2c_write_reg(uint8_t addr, uint8_t reg, uint8_t val)
{
    int rc;

    I2C1->CR2 = (addr << 1) | (2u << 16)     /* 2 bytes      */
              | I2C_CR2_START | I2C_CR2_AUTOEND;

    if ((rc = wait_flag(&I2C1->ISR, I2C_ISR_TXIS, 1)) < 0) { goto err; }
    if (I2C1->ISR & I2C_ISR_NACKF) { rc = -ENXIO; goto err; }
    I2C1->TXDR = reg;

    if ((rc = wait_flag(&I2C1->ISR, I2C_ISR_TXIS, 1)) < 0) { goto err; }
    I2C1->TXDR = val;

    if ((rc = wait_flag(&I2C1->ISR, I2C_ISR_STOPF, 1)) < 0) { goto err; }
    I2C1->ICR = I2C_ICR_STOPCF;
    return 0;

err:
    i2c_recover();          /* chapter 07 - do NOT skip this */
    return rc;
}

int i2c_read_regs(uint8_t addr, uint8_t reg, uint8_t *d, size_t n)
{
    /* Phase 1: write the register pointer, NO autoend -
       we want a repeated start.                            */
    I2C1->CR2 = (addr << 1) | (1u << 16) | I2C_CR2_START;
    if (wait_flag(&I2C1->ISR, I2C_ISR_TXIS, 1) < 0) { goto err; }
    if (I2C1->ISR & I2C_ISR_NACKF) { goto err; }
    I2C1->TXDR = reg;
    if (wait_flag(&I2C1->ISR, I2C_ISR_TC, 1) < 0) { goto err; }

    /* Phase 2: repeated start, read direction. */
    I2C1->CR2 = (addr << 1) | I2C_CR2_RD_WRN | (n << 16)
              | I2C_CR2_START | I2C_CR2_AUTOEND;

    for (size_t i = 0; i < n; i++) {
        if (wait_flag(&I2C1->ISR, I2C_ISR_RXNE, 1) < 0) { goto err; }
        d[i] = (uint8_t) I2C1->RXDR;
    }
    if (wait_flag(&I2C1->ISR, I2C_ISR_STOPF, 1) < 0) { goto err; }
    I2C1->ICR = I2C_ICR_STOPCF;
    return 0;

err:
```

```
    i2c_recover();
    return -EIO;
}

/* The v2 peripheral handles ACK/NACK of the final byte from
   the NBYTES count, which is why the loop above has no
   explicit ack flag. Older peripherals require you to clear
   the ACK bit before reading the second-to-last byte -
   check which generation you have.                        */
```

How it shows up

Symptom	Cause
Scan finds a device at half the expected address	Datasheet quoted the shifted 8-bit address
NACK on the address byte	No device there — wrong address, unpowered, or not connected
NACK on a data byte to an EEPROM	Internal write in progress; retry (acknowledge polling)
Bus hangs after a multi-byte read	Last byte ACKed instead of NACKed; the slave still drives SDA
Works at 100 kHz, fails at 400 kHz	Pull-up too weak — rise time exceeds the specification
Nothing works, both lines read low	No pull-ups fitted at all
Reads return the previous register's value	STOP used instead of a repeated start; the pointer reset
Three identical sensors will not fit	Address collision — needs a mux or a second bus
Driver hangs forever on one bad transaction	A wait loop with no timeout
First transaction after reset always fails	Bus left mid-transaction by the previous run — needs recovery (chapter 07)

Check yourself

- ✓ Explain why open-drain wiring is required and name three capabilities it enables.
- ✓ Compute the maximum pull-up resistance for 400 kHz with 100 pF of bus capacitance.
- ✓ State the rule distinguishing START and STOP from data transitions.
- ✓ Explain why ACK is a low level rather than a high one.
- ✓ Explain what a repeated start is for and what breaks if you use STOP instead.
- ✓ Say why the master must NACK the last byte it reads.
- ✓ Explain the address-shift confusion and how a bus scan resolves it.
- ✓ Distinguish a NACK on the address byte from a NACK on a data byte.
- ✓ Describe acknowledge polling and what it replaces.
- ✓ Explain why every I2C wait loop needs a timeout.

I2C: clock stretching, arbitration and bus recovery

I2C's open-drain wiring gives it three capabilities no other simple bus has, and each brings a failure mode. A stuck bus that needs software to unstick it is the one every embedded engineer eventually meets at three in the morning.

Clock stretching

```
A slave that needs more time HOLDS SCL LOW after the
acknowledge bit. The master, which only RELEASES SCL and
waits for it to rise, sees it stay low and waits.

  master releases SCL --+
                        |
  SCL  ______/----------+__________/------
                        ^          ^
                slave holds it   slave releases,
                low here         clock resumes

Entirely automatic in hardware IF the master implements
it: the master must read SCL back after releasing it and
not proceed until it is actually high.

WHO STRETCHES:
  - EEPROMs during an internal write
  - sensors computing a conversion
  - MCUs acting as I2C slaves, between the interrupt and
    the software loading the next byte - this one can
    stretch for a LONG time
  - bit-banged slaves, unpredictably

THE HAZARDS:

1. SOME MASTERS DO NOT SUPPORT IT. A bit-banged master
   that drives SCL push-pull instead of releasing it will
   fight the slave, and both are damaged or the data is
   corrupt. Bit-banged I2C MUST use open-drain or
   input/output-low switching, never push-pull high.

2. IT CAN BE UNBOUNDED. A slave that stretches forever -
   because its firmware crashed mid-transaction - hangs
   the master indefinitely. This is why every wait needs
   a timeout (chapter 06) and why SMBus, unlike I2C,
   specifies a 25 ms maximum.

3. SOME PERIPHERALS HAVE ERRATA. Several MCU I2C blocks
   have documented bugs when a slave stretches at a
   particular moment. Read the errata sheet - this is one
   of the few times it genuinely matters.
```

Multi-master arbitration

```
Two masters may start transmitting at the same instant.
Because the bus is open-drain, LOW ALWAYS WINS, and that
is enough to resolve it with no extra protocol.

Master A sends address 0x50 = 1010000
Master B sends address 0x44 = 1000100

bit:        6  5  4  3  2  1  0
A drives:   1  0  1  0  0  0  0
B drives:   1  0  0  0  1  0  0
BUS is:     1  0  0  ...
                  ^
                  A wanted 1, bus shows 0.

At bit 4, A releases SDA (writing a 1) but reads back a
0 - B is pulling it low. A has LOST ARBITRATION. It
immediately stops driving and becomes a slave for the
rest of the transaction.

PROPERTIES THAT MAKE THIS ELEGANT:
  - no data is lost: B's message continues uncorrupted,
    and B does not even know a collision occurred
  - no time is wasted on a retry of the winner
  - lower addresses win, so priority is implicit

This is the same mechanism CAN uses, and for the same
reason (chapter 09). Both are "wired-AND" buses where a
dominant level overrides a recessive one.

IN PRACTICE multi-master I2C is uncommon and often
avoided, because the error handling is fiddly and few
designs need it. But the ARBITRATION LOST flag still
fires in single-master systems - and when it does, it
means something is corrupting SDA. Treat it as a wiring
or noise diagnosis, not a multi-master event.
```

The stuck bus, and how to unstick it

```
THE FAILURE:

1. The master is reading. The slave is transmitting and
   has pulled SDA LOW for a zero bit.
2. The MASTER RESETS - watchdog, crash, debugger, or
   power glitch on the master only.
3. The master restarts and stops clocking.
4. The slave is still mid-byte, still holding SDA low,
   waiting for the clock that will never come.
5. SDA is stuck low. The master cannot generate a START,
   because START requires SDA to FALL - and it is already
   low. The bus is dead until power-cycled.

This is extremely common in development, because you
reset the master constantly and the sensor never does.
"I2C only works after a full power cycle" is exactly
this.

THE RECOVERY, from the I2C specification:

1. Switch SCL and SDA to GPIO, open-drain.
2. Clock SCL up to 9 times, checking SDA after each.
   The slave advances through its byte and, after at
   most 9 clocks, finishes and releases SDA.
3. Once SDA is high, generate a manual STOP: SDA low,
   SCL high, then SDA high.
4. Re-initialise the peripheral.

RUN THIS AT EVERY BOOT, unconditionally, before the
first transaction. It costs 100 microseconds and it
eliminates a whole class of "needs a power cycle" bug.
```

```
/* The recovery, complete. */

void i2c_bus_recover(void)
{
    /* Peripheral off; take the pins as open-drain GPIO. */
    I2C1->CR1 &= ~I2C_CR1_PE;
    gpio_open_drain_output(SCL_PORT, SCL_PIN);
    gpio_open_drain_output(SDA_PORT, SDA_PIN);
    SDA_HIGH(); SCL_HIGH();
    delay_us(10);

    /* Up to 9 clocks to walk the slave out of its byte. */
    for (int i = 0; i < 9 && !SDA_READ(); i++) {
        SCL_LOW();  delay_us(5);
        SCL_HIGH(); delay_us(5);
    }

    /* Manual STOP: SDA must RISE while SCL is high. */
    SDA_LOW();  delay_us(5);
    SCL_HIGH(); delay_us(5);
    SDA_HIGH(); delay_us(5);

    if (!SDA_READ()) {
        /* Still stuck: the slave is not merely mid-byte, it
           is crashed or the line is shorted. If the design
           has a power-gate FET on the sensor rail, this is
           where you use it - the ONLY remaining fix.       */
        stats.i2c_unrecoverable++;
        sensor_power_cycle();
    }

    gpio_af(SCL_PORT, SCL_PIN, I2C_AF);
    gpio_af(SDA_PORT, SDA_PIN, I2C_AF);
    i2c_init();
}

/* THE DESIGN LESSON: put a MOSFET in the sensor's supply.
   It costs a few cents and turns "unrecoverable, RMA the
   unit" into "power-cycle that rail and continue". Any
   product with an I2C sensor it cannot reach physically
   should have one.                                        */
```

NEVER LEAVE I2C WITHOUT A TIMEOUT AND A RECOVERY PATH

Of all the buses in this track, I2C is the one that hangs. A slave holding SCL, a slave holding SDA, a missing pull-up, a noise glitch that desynchronises a state machine — each produces a wait that never ends. A driver that blocks forever in a wait loop turns a recoverable bus glitch into a dead product. Every wait gets a deadline; every timeout calls the recovery above; every recovery failure is counted and reported.

Electrical realities

```
I2C IS THE MOST ELECTRICALLY FRAGILE BUS HERE. It is slow,
high-impedance, and has no error detection whatsoever -
a corrupted bit is simply wrong data with a valid ACK.

THE BUS CAPACITANCE BUDGET: 400 pF maximum, total.
  - each device pin        ~10 pF
  - PCB trace           ~1 pF per cm
  - a ribbon cable      ~50-100 pF per metre

So 30 cm of PCB plus six devices is ~90 pF: fine. Add a
metre of ribbon and you are at 190 pF and the rise time
has doubled. Two metres and you are out of specification
entirely.

I2C WAS DESIGNED FOR CHIPS ON ONE BOARD. Running it
between boards, through a connector, or down a cable is
the single most common cause of intermittent I2C. If you
must:
  - lower the speed to 100 kHz
  - lower the pull-ups to 2.2k (more current, faster edge)
  - use a bus buffer/extender (P82B715 and similar)
  - or use a bus designed for it: RS-485 or CAN
    (chapters 14, 09)

ENABLE THE DIGITAL NOISE FILTER if the peripheral has one:
  I2C1->CR1 |= (2u << I2C_CR1_DNF_Pos);   /* 2 clocks   */
It suppresses spikes shorter than N clock periods, at the
cost of a little timing margin. Cheap insurance.
```

How it shows up

Symptom	Cause
Works only after a full power cycle	Bus stuck from a master reset mid-read — add boot-time recovery
SDA measured low with nothing running	A slave mid-byte, waiting for clocks
Master hangs forever on one transaction	Slave stretching indefinitely, and no timeout
Arbitration-lost flag in a single-master system	Noise or a wiring fault corrupting SDA — not a protocol event
Bit-banged master corrupts data with some slaves	SCL driven push-pull, fighting a stretching slave
Intermittent failures with a cable	Bus capacitance over budget; slow down or use a buffer
Fails only when a motor runs	Coupled noise; add the digital filter and check grounding
Occasional wrong sensor values, no error flags	I2C has no error detection — add a checksum at the protocol level or sanity-check the range
Recovery clocks do not free the bus	Slave is crashed rather than mid-byte; power-cycle the rail
Works on the bench, fails in the enclosure	Longer wiring, more capacitance, more coupled noise

Check yourself

- ✓ Explain clock stretching and what the master must do to support it.
- ✓ Give three kinds of device that stretch and one that may stretch unboundedly.

- ✓ Explain why a bit-banged master must never drive SCL push-pull high.
- ✓ Walk through arbitration between two masters and say which wins.
- ✓ Explain what an arbitration-lost flag means in a single-master system.
- ✓ Describe exactly how a bus becomes stuck with SDA low.
- ✓ Give the four steps of the standard bus recovery, and say why nine clocks.
- ✓ Explain why a power-gate FET on the sensor rail is worth its cost.
- ✓ State the bus capacitance budget and estimate it for a metre of ribbon cable.
- ✓ Explain why a corrupted I2C bit produces wrong data rather than an error.

Choosing between UART, SPI and I2C

Three buses, overlapping capabilities, and a choice usually made by whichever the sensor happens to support. When you do have a choice — and in your own designs you always do — these are the axes that matter.

Side by side

	UART	SPI	I2C
Wires (2 devices)	2 (+ 2 for flow control)	4	2
Wires (N devices)	2 per device	3 + N	2, always
Typical speed	115 kbit – 3 Mbit	10–100 Mbit	100 k – 1 Mbit
Duplex	Full	Full	Half
Clock	None — both ends must agree	Shared	Shared
Addressing	None (point to point)	Chip select pin	7-bit address
Acknowledgement	None	None	Per byte
Error detection	Parity, framing, noise flags	None	None
Multi-master	No	No	Yes, with arbitration
Distance	Metres (more with RS-485)	Centimetres	Centimetres
Can hang the bus	No	No	Yes
Driver complexity	Low	Very low	High

The decision, in order

```
1. IS THE PEER OFF-BOARD OR MORE THAN ~30 CM AWAY?
     -> NOT SPI, NOT I2C. Both are chip-to-chip buses on
        one board. Use UART, or RS-485/CAN for real
        distance (chapters 14, 09).

2. DO YOU NEED MORE THAN ~1 MBIT/S?
     -> SPI. Nothing else here comes close. Displays,
        external flash, high-rate ADCs, radio modules.

3. ARE PINS THE BINDING CONSTRAINT?
     -> I2C. Ten sensors on two wires versus thirteen
        pins for SPI. On a 20-pin part this decides it.

4. IS THIS A CONSOLE, A LOG, OR A MODEM-STYLE DEVICE?
     -> UART. GPS, GSM, Bluetooth modules and debug
        consoles are all UART by convention, and fighting
        the convention costs more than it saves.

5. DOES THE DEVICE NEED TO INITIATE?
     -> UART, or add an interrupt line. SPI and I2C
        slaves cannot speak unprompted; the near-universal
        solution is a separate INT pin that tells the
        master to come and read - which quietly costs you
        the pin you saved by choosing I2C.

6. OTHERWISE
     -> I2C for slow sensors, SPI for anything else.

AND THE HONEST ANSWER MOST OF THE TIME: the sensor you
want supports one bus, and that decides it. The choice
above matters when you are designing the protocol between
two chips you control - an MCU talking to an application
processor, or two MCUs in one product.
```

The interrupt line nobody counts

```
/* SPI and I2C are strictly master-driven. A sensor with new
   data has no way to say so, which leaves three options.  */

/* 1. POLLING - simple, wasteful, and adds latency. */
for (;;) {
    if (i2c_read_reg(ADDR, STATUS) & DATA_READY) {
        read_and_process();
    }
    delay_ms(10);          /* 100 transactions/s doing
                              nothing, and up to 10 ms of
                              latency                     */
}

/* 2. AN INTERRUPT PIN - one extra wire, and the right
      answer nearly always.                               */
void EXTI9_5_IRQHandler(void)
{
    EXTI->PR = (1u << 7);
    /* Do NOT do I2C in the ISR - it blocks for hundreds of
       microseconds. Signal a task (mcu chapter 19).       */
    vTaskNotifyGiveFromISR(sensor_task, &woken);
    portYIELD_FROM_ISR(woken);
}

/* Now: zero bus traffic when idle, microseconds of latency,
   and the CPU can sleep. The pin you spent buys back more
   than it cost.

   SHARED INTERRUPT LINES: several open-drain INT outputs
   can share one MCU pin with a pull-up - any device pulling
   low signals, and the handler polls each device's status
   to find out who. This is exactly the wired-OR idea from
   chapter 06, and it keeps the pin count at one regardless
   of device count.                                        */

/* 3. A FIFO IN THE SENSOR - increasingly common and the
      best of both. The device buffers 32-512 samples and
      interrupts only when the FIFO passes a watermark. The
      master then reads a burst.

      This is how a modern IMU is used: instead of 1000
      interrupts per second for 1000 samples, you get 10
      interrupts and read 100 samples each time. The CPU
      sleeps 99% of the time and the bus carries the same
      data in a tenth of the transactions.

      If a sensor offers a FIFO, use it. It is usually the
      difference between a design that meets its power
      budget and one that does not.                       */
```

COUNT THE REAL PIN COST

I2C's two-wire claim assumes no interrupt lines and no address conflicts. Three sensors that each need an interrupt pin cost 2 + 3 = 5 pins, against SPI's 3 + 3 = 6 — and if two of them share a fixed address you also need a multiplexer, which costs a part, board area and another two wires. Do this arithmetic with the actual devices before committing the schematic, not after.

Mixing them, and level shifting

```
A real board runs all three at once, and the interesting
problem becomes voltage domains.

MCU at 3.3 V, sensor at 1.8 V:

PUSH-PULL BUSES (SPI, UART) need a proper level shifter -
a dedicated translator IC, or a resistor divider for
3.3 -> 1.8 in the slow direction only. A divider cannot
shift UP, and it slows the edge, so it is unsuitable for
fast SPI.

OPEN-DRAIN BUSES (I2C) shift with ONE MOSFET PER LINE:

     3.3V                    1.8V
      |                       |
    [4.7k]                  [4.7k]
      |                       |
SDA_3V3 ----+           +---- SDA_1V8
            |           |
          +-+-----------+-+
          | G   MOSFET    |    gate tied to 1.8 V
          | S           D |
          +---------------+

Either side pulling low pulls the other low, through the
body diode and then the channel. Elegant, cheap, and the
standard solution - the BSS138 in this configuration
appears on an enormous number of boards.

THIS IS ANOTHER CONSEQUENCE OF OPEN-DRAIN, and a real
advantage of I2C in mixed-voltage designs.

ALWAYS CHECK: is the sensor 5 V tolerant? Many 3.3 V
parts are not, and an I2C bus pulled up to 5 V will
damage them - slowly, so the board works for a week.
```

How it shows up

Symptom	Cause
I2C unreliable between two boards	Wrong bus for the distance — use UART, RS-485 or a buffer
SPI display too slow to refresh	Clock too low, or per-byte CPU overhead; use DMA
Ran out of pins late in the design	Interrupt lines not counted in the original bus choice
High CPU load just polling a sensor	No interrupt line and no FIFO — add one
Power budget missed by a wide margin	Waking per sample instead of per FIFO watermark
Two identical sensors cannot coexist	Fixed I2C address; needs a mux or a different bus
Level shifter works for I2C, not for SPI	MOSFET shifters are for open-drain only
Board works for a week then a sensor dies	Non-5 V-tolerant part on a 5 V pulled-up bus
Sensor data always one sample stale	Polling loop reading before the conversion completes
ISR takes hundreds of microseconds	Bus transaction inside the interrupt handler — defer it to a task

Check yourself

- ✓ Give the wire count for five devices on each of the three buses.
- ✓ Say which of the three can hang the bus and why.
- ✓ State the first question to ask when choosing, and what it rules out.
- ✓ Explain why a slave device cannot initiate on SPI or I2C.
- ✓ Describe the three ways to learn that a sensor has new data.
- ✓ Explain how a shared open-drain interrupt line works.
- ✓ Explain why a sensor FIFO can decide whether a power budget is met.
- ✓ Do the real pin arithmetic for three interrupt-driven sensors on I2C versus SPI.
- ✓ Explain why one MOSFET level-shifts I2C but not SPI.
- ✓ Say what to check before pulling an I2C bus up to 5 V.

CAN: arbitration and the bit-level protocol

CAN was designed for a car in 1986 and every design decision shows it: no addresses, no master, priority resolved without losing a bit of bandwidth, and error handling that assumes nodes will fail and the bus must survive. It is the most thoroughly engineered bus in this track.

The physical layer

```
A DIFFERENTIAL PAIR, terminated at both ends.

node A        node B        node C
  |             |             |
[xcvr]        [xcvr]        [xcvr]
  |  |          |  |          |  |
[120]+----------+--+----------+--[120]
  CANH ==================================
  CANL ==================================

TWO BUS STATES, and the naming is the key to everything:

RECESSIVE (logical 1)   CANH = CANL = 2.5 V
                        differential = 0 V
                        This is what the bus does when
                        NOBODY drives it.

DOMINANT  (logical 0)   CANH = 3.5 V, CANL = 1.5 V
                        differential = 2 V
                        Any node driving dominant
                        OVERRIDES every recessive node.

IT IS A WIRED-AND BUS: dominant wins, exactly like I2C's
open-drain low (chapter 07). Everything below follows
from that one property.

DIFFERENTIAL SIGNALLING is why CAN works in a car: noise
couples equally into both wires, so the DIFFERENCE is
unaffected. Common-mode rejection of tens of volts, over
40 metres, next to an ignition coil.

TERMINATION: 120 ohms at each END of the bus, not at each
node. Two resistors total, giving 60 ohms as seen by a
transmitter. Missing one causes reflections and
intermittent errors; fitting three drops the impedance
too low and transmitters cannot pull the bus dominant.
"Measure 60 ohms across a powered-down bus" is the first
diagnostic anyone does.
```

The frame

```
STANDARD DATA FRAME (11-bit identifier):

+---+-----------+---+---+------+--------+-----+---+---+---+
|SOF| ID 11 bits|RTR|IDE| r0   | DLC 4  |DATA |CRC|ACK|EOF|
+---+-----------+---+---+------+--------+-----+---+---+---+
  1      11      1   1    1        4    0-64   16  2   7

SOF   start of frame, one dominant bit
ID    IDENTIFIER - not an address. It names the MESSAGE,
      not the sender or receiver.
RTR   remote transmission request (asking for data)
IDE   0 = 11-bit ID, 1 = 29-bit extended
DLC   data length, 0-8 bytes
CRC   15-bit CRC + delimiter
ACK   THE CLEVER BIT - see below
EOF   7 recessive bits

THERE IS NO SOURCE OR DESTINATION ADDRESS. A node
broadcasts "engine speed is 3000 rpm" with ID 0x0C0, and
every node that cares filters for that ID. Adding a
listener requires no change anywhere else - which is why
a car can gain a new module without reflashing the
others.

THE ACK SLOT: the transmitter sends a RECESSIVE bit, and
EVERY node that received the frame with a valid CRC
drives it DOMINANT. So:

  - one ACK bit is enough for any number of receivers
  - the transmitter learns "at least one node heard me"
  - it does NOT learn who, or whether the intended
    recipient did

A SINGLE NODE ALONE ON A BUS CANNOT TRANSMIT SUCCESSFULLY
- nobody acknowledges, so it retries forever and
eventually goes error-passive (chapter 10). This is the
most common CAN bring-up confusion: your code is correct,
but you need a second node, or loopback mode, to test.
```

Arbitration without losing bandwidth

```
Every node may start transmitting when the bus is idle.
Collisions are resolved DURING the identifier, bit by bit.

Node A sends ID 0x123 = 001 0010 0011
Node B sends ID 0x121 = 001 0010 0001

bit:      10 9 8 7 6 5 4 3 2 1 0
A drives:  0 0 1 0 0 1 0 0 0 1 1
B drives:  0 0 1 0 0 1 0 0 0 0 1
BUS:       0 0 1 0 0 1 0 0 0 0 ...
                           ^
                      A sends recessive(1),
                      reads back dominant(0).
                      A LOSES, stops immediately.

Every transmitter monitors the bus while sending. Sending
recessive and reading dominant means someone with a lower
ID is talking; you withdraw at once and retry later.

WHY THIS IS BETTER THAN ETHERNET'S APPROACH:
  - NON-DESTRUCTIVE: B's frame continues uncorrupted. No
    collision, no backoff, no wasted bus time.
  - DETERMINISTIC: the lowest ID always wins, so the
    highest-priority message has a computable worst-case
    latency. Ethernet's CSMA/CD has none.
  - FREE: it costs no extra bits; the identifier does
    double duty as the priority.

LOWER ID = HIGHER PRIORITY. This makes ID assignment a
REAL-TIME DESIGN DECISION, not an arbitrary labelling.
Safety-critical messages get low IDs; diagnostics get
high ones. Get it backwards and a diagnostic tool can
delay a brake message.

THE COST: a node with a high ID can be starved
indefinitely on a busy bus. Bus load is therefore budgeted
- 30-40% is a normal design target, and above 70% the
low-priority messages become unpredictable.
```

BIT RATE IS CAPPED BY THE PHYSICS OF ARBITRATION

For arbitration to work, every node must sample each identifier bit *after* the signal has propagated to the far end of the bus and back. So one bit time must exceed the round-trip delay — which ties bit rate to bus length: 1 Mbit/s for 40 m, 500 kbit/s for 100 m, 125 kbit/s for 500 m. This is not a signal integrity limit you can engineer around with better cable; it is inherent to resolving arbitration within a bit. CAN FD (chapter 11) works around it by switching to a faster rate only *after* arbitration has finished.

Bit stuffing and self-clocking

```
/* CAN has no clock line, and no start bit per byte. The
   receiver recovers timing from EDGES - so the protocol
   guarantees edges happen.                                */

/* BIT STUFFING: after 5 consecutive bits of the same value,
   the transmitter inserts one bit of the OPPOSITE value.
   The receiver removes it.

     data:    1 1 1 1 1 1 1 0 0 0 0 0 0
     on wire: 1 1 1 1 1 0 1 1 0 0 0 0 0 1
                        ^             ^
                    stuffed       stuffed

   CONSEQUENCES:

   1. A frame's duration is VARIABLE. The same 8 bytes take
      different times depending on the bit pattern. Worst
      case for a standard frame is about 135 bit times
      against a nominal 111. Real-time analysis must use the
      worst case, not the nominal - a 20% error otherwise.

   2. SIX consecutive identical bits is by definition an
      ERROR, and that is exactly how errors are signalled:
      an ERROR FRAME is six dominant bits, deliberately
      violating the stuffing rule so every node notices
      immediately.

   3. Stuffing does not apply to the CRC delimiter, ACK and
      EOF fields, which are fixed-form.                    */

/* THE ERROR FRAME is what makes CAN robust. Any node that
   detects a violation - stuff error, CRC error, form error,
   ACK error, or bit error - immediately transmits six
   dominant bits. This DESTROYS the frame in progress for
   everyone, so all nodes discard it consistently and the
   transmitter retries automatically.

   No software involvement. A corrupted message is retried
   in microseconds by hardware. This is why CAN is used
   where it is, and it is also why a single faulty node can
   destroy every frame on the bus - which is what error
   confinement in chapter 10 exists to prevent.            */
```

How it shows up

Symptom	Cause
Single node cannot transmit anything	Nobody to ACK — add a second node or use loopback mode
Bus reads 120 Ω instead of 60 Ω	One terminator missing
Bus reads 40 Ω	Three terminators fitted — only the two ends should have one
Works on the bench, errors in the vehicle	Bit rate too high for the bus length, or a stub too long
Low-priority messages sometimes never sent	Bus load too high — starvation is inherent to priority arbitration
Timing analysis wrong by ~20%	Nominal frame length used instead of the worst case with stuffing

Symptom	Cause
One faulty node kills all traffic	It is generating error frames — error confinement should eventually silence it (chapter 10)
A diagnostic tool delays control messages	ID assignment backwards — diagnostics need high IDs
Frames received with wrong data	Bit timing sample point misconfigured (chapter 10)
Node transmits but nothing receives	CANH and CANL swapped

Check yourself

- ✓ Define dominant and recessive and say which wins.
- ✓ Explain why CAN uses differential signalling.
- ✓ State how many terminators a bus needs, where, and what to measure.
- ✓ Explain what a CAN identifier names, given there are no addresses.
- ✓ Explain the ACK slot and why one bit serves any number of receivers.
- ✓ Say why a single node alone on a bus cannot transmit.
- ✓ Walk through arbitration between two IDs and say which wins and why.
- ✓ Give two reasons CAN arbitration beats Ethernet's collision handling.
- ✓ Explain bit stuffing and its effect on worst-case frame duration.
- ✓ Explain how an error frame works and why it needs no software.

CAN: bit timing, error states and bus-off

Two things distinguish a CAN node that works from one that mostly works: a sample point set where the standard wants it, and an understanding of the error counters that decide whether a misbehaving node is allowed to keep talking.

The bit, divided

```
A CAN bit is divided into TIME QUANTA (tq), and the
division is what you configure - not a baud rate divisor.

|<--------------- one bit time ------------------>|
+--------+-----------------+--------------------+
|  SYNC  |    PROP + PS1   |        PS2         |
|  1 tq  |                 |                    |
+--------+-----------------+--------------------+
                           ^
                      SAMPLE POINT

SYNC_SEG   always 1 tq. Edges are expected here.
PROP_SEG   compensates for the physical propagation
           delay to the far end of the bus and back.
PHASE_SEG1 buffer, can be LENGTHENED to resync.
PHASE_SEG2 buffer, can be SHORTENED to resync.

SAMPLE POINT = (SYNC + PROP + PS1) / total, as a
percentage of the bit.

TARGET: 75-80% for most rates; CANopen and DeviceNet
specify 87.5%. LATE, not in the middle - because
arbitration requires the signal to have propagated to
the far end AND BACK before anyone samples (chapter 09).

THIS IS THE SINGLE MOST COMMON CAN CONFIGURATION ERROR.
A sample point at 50% works perfectly on a short bench
bus and fails on a long vehicle harness, because the
round trip no longer fits. The symptom is a bus that
works until the cable gets longer.

SJW (SynchronisationJumpWidth): how many tq the resync
may adjust by, 1-4. It absorbs the accumulated difference
between two nodes' oscillators. Set it to 1 for tight
clocks, higher if you must tolerate sloppy ones.
```

Calculating it

```
/* GOAL: 500 kbit/s from a 42 MHz peripheral clock, sample
   point at 87.5%.                                          */

/*  tq per bit is chosen first. More tq gives finer control
    of the sample point; fewer allows higher bit rates.
    8–25 is the usual range; 16 is a good default.

    prescaler = f_clk / (bitrate * tq_per_bit)
              = 42e6 / (500000 * 16)
              = 5.25            -> NOT AN INTEGER.

    Try tq = 14:
              = 42e6 / (500000 * 14) = 6      -> exact.

    Now divide 14 tq to place the sample point:
      SYNC = 1
      TSEG1 (prop + ps1) = 11
      TSEG2 (ps2)        = 2
      total = 1 + 11 + 2 = 14
      sample = (1 + 11) / 14 = 85.7%     -> close enough

    Registers hold "value minus one" (mcu chapter 11):     */

CAN1->BTR = ((6u - 1u)  << 0)      /* BRP  = 6            */
          | ((11u - 1u) << 16)     /* TS1  = 11 tq        */
          | ((2u - 1u)  << 20)     /* TS2  = 2 tq         */
          | ((1u - 1u)  << 24);    /* SJW  = 1 tq         */

/* VERIFY BY MEASUREMENT, not by arithmetic. Put a scope on
   CANH and measure one bit's width: it must be 2.00 us for
   500 kbit/s. An error of even 1% will cause errors on a
   long bus with a node at the tolerance limit.

   EVERY NODE ON THE BUS MUST USE THE SAME BIT RATE, and
   should use a similar sample point. They need NOT use the
   same tq count or prescaler - only the resulting bit time
   and a compatible sample point matter.

   ONLINE CALCULATORS EXIST and are worth using: the
   constraint set (integer prescaler, tq range, sample point
   target, SJW) is genuinely fiddly and errors are silent. */
```

BIT TIMING CAN ONLY BE CHANGED IN INITIALISATION MODE

The `BTR` register is writable only while the peripheral is in initialisation mode, and writes are silently ignored otherwise. Code that configures timing after starting the controller produces a node running at whatever the reset default was — usually wrong, and usually diagnosed as a wiring problem. Set timing before leaving init, and read the register back to confirm it took.

Error confinement

```
CAN assumes nodes will fail, and contains them
automatically. Two counters per node:

  TEC - transmit error counter
  REC - receive error counter

ROUGHLY: an error while transmitting adds 8 to TEC; an
error while receiving adds 1 to REC; a SUCCESS subtracts
1 from the relevant counter.

The 8:1 asymmetry is deliberate: a node that is itself
broken accumulates errors eight times faster than the
innocent nodes that merely witness them. The faulty node
is therefore confined first, and the bus survives.

THREE STATES:

ERROR ACTIVE      TEC < 128 and REC < 128
  Normal. Transmits ACTIVE error frames (6 dominant
  bits), which destroy the offending frame for everyone.

ERROR PASSIVE     TEC >= 128 or REC >= 128
  Still communicates, but transmits PASSIVE error frames
  (6 RECESSIVE bits) which do not disturb others. It
  also waits 8 extra bit times before transmitting,
  yielding to healthy nodes.

BUS-OFF           TEC >= 256
  Disconnected. Transmits nothing, ACKs nothing, has no
  effect on the bus at all.

+---------------+  TEC or REC >= 128   +----------------+
| ERROR ACTIVE  |--------------------> | ERROR PASSIVE  |
|               | <------------------- |                |
+---------------+  both < 128          +----------------+
                                                |
                                     TEC >= 256 |
                                                v
                                       +----------------+
                                       |    BUS-OFF     |
                                       +----------------+
                                                |
                         128 x 11 recessive bits, |
                         and software permission  |
                                                v
                                          ERROR ACTIVE

RECOVERY FROM BUS-OFF requires observing 128 occurrences
of 11 consecutive recessive bits - i.e. the bus being
idle - which takes at least 128 x 11 bit times. At
500 kbit/s that is about 2.8 ms.
```

Handling bus-off in software

```
/* Whether recovery is automatic is a configuration choice,
   and the right answer depends on the product.            */

/* AUTOMATIC (ABOM): the hardware recovers as soon as the
   128x11 condition is met. Convenient, and it hides the
   fault - a node with a broken transceiver will cycle
   in and out of bus-off forever, and nobody notices.      */
CAN1->MCR |= CAN_MCR_ABOM;

/* MANUAL: software decides. Preferred in anything that
   must report faults, because it lets you count and
   rate-limit.                                             */
void CAN1_SCE_IRQHandler(void)
{
    uint32_t esr = CAN1->ESR;

    if (esr & CAN_ESR_BOFF) {
        stats.bus_off_count++;
        stats.last_lec = (esr >> 4) & 7u;   /* last error code */

        if (stats.bus_off_count < 5) {
            can_restart();                  /* try again    */
        } else {
            /* Repeated bus-off is a HARDWARE fault: a
               shorted bus, a missing terminator, a wrong
               bit rate, or a dead transceiver. Retrying
               forever hides it. Report and stay off.      */
            fault_latch(FAULT_CAN_BUS);
        }
    }
}

/* THE LAST ERROR CODE names the cause, and it is genuinely
   diagnostic:

     1 STUFF   more than 5 identical bits -> noise, or a
               node at the wrong bit rate
     2 FORM    a fixed-form field was wrong -> bit rate or
               a node speaking a different protocol
     3 ACK     nobody acknowledged -> you are alone on the
               bus, or the others are bus-off, or CANH/CANL
               are swapped
     4 BIT1    sent recessive, read dominant -> another
               node is stuck dominant, or a short to ground
     5 BIT0    sent dominant, read recessive -> your
               transceiver is not driving; check its
               standby pin and supply
     6 CRC     corruption in transit -> noise or termination

   ACK error on a bench setup with one node is normal and
   expected (chapter 09). ACK error in a running vehicle
   means everyone else has gone quiet.                     */
```

Acceptance filters

```
/* A busy bus carries thousands of frames per second, and
   most are not yours. Filtering in HARDWARE is essential -
   waking the CPU for every frame would dominate the load. */

/* MASK MODE: check only the bits the mask selects.
     accept if (received_id & mask) == (filter_id & mask)  */

/* Accept 0x100-0x10F: the low nibble is "don't care".     */
CAN1->sFilterRegister[0].FR1 = (0x100u << 21);   /* id      */
CAN1->sFilterRegister[0].FR2 = (0x7F0u << 21);   /* mask    */

/* LIST MODE: accept exactly these identifiers, no mask.
   Use when you want a handful of unrelated IDs.           */

/* DESIGN CONSEQUENCE: assign IDs so that the messages one
   node cares about share a bit pattern. If a node needs
   0x101, 0x205 and 0x3C2, no single mask captures them and
   you burn three filter banks. If those messages were
   numbered 0x110, 0x111, 0x112 one filter does it.

   This is why CAN ID allocation is a system design task
   done up front, in a spreadsheet, balancing:
     - priority (lower = more urgent, chapter 09)
     - filterability (group by consumer)
     - room to grow

   Retrofitting an ID scheme after the software is written
   is genuinely painful, which is why standard higher-layer
   protocols - CANopen, J1939 - specify one for you.       */
```

How it shows up

Symptom	Cause
Works on the bench, fails with a long harness	Sample point too early — set it to 75–87.5%
Bit timing register has no effect	Written outside initialisation mode
Errors between two specific nodes only	Their bit rates differ slightly; measure a bit with a scope
Node goes bus-off immediately	Alone on the bus (ACK error), or CANH/CANL swapped
LEC reads BIT0 persistently	Transceiver not driving — check its standby/enable pin and supply
LEC reads BIT1 persistently	Another node stuck dominant, or a short to ground
Node cycles in and out of bus-off unnoticed	Automatic recovery hiding a hardware fault — count and latch
CPU saturated by receive interrupts	Filters too permissive; filter in hardware
Cannot filter the messages a node needs	ID allocation not designed for filterability
One node's errors kill the whole bus	It is error-active and sending error frames; wait for confinement or find it

Check yourself

- ✓ Name the four segments of a CAN bit and say which two absorb resynchronisation.
- ✓ Explain why the sample point is late rather than centred.
- ✓ Calculate BRP, TSEG1 and TSEG2 for 500 kbit/s from a 42 MHz clock.

- ✓ Say what must match between nodes and what need not.
- ✓ Explain the 8:1 asymmetry between transmit and receive error counters.
- ✓ Name the three error states and the thresholds between them.
- ✓ Explain the difference between active and passive error frames.
- ✓ State what bus-off recovery requires and how long it takes at 500 kbit/s.
- ✓ Interpret an ACK error, a BIT0 error and a BIT1 error as diagnoses.
- ✓ Explain why CAN ID allocation must consider filterability, not just priority.

CAN FD, and CAN in a vehicle

Classical CAN carries 8 bytes at 500 kbit/s, which was generous in 1991 and is not now. CAN FD raises both limits without giving up arbitration — and the higher-layer protocols built on top are where an embedded engineer actually works.

What CAN FD changes

```
TWO CHANGES, and the first is the clever one.

1. DUAL BIT RATE. Arbitration happens at the classical
   rate; the data phase switches to a faster one.

   |<-- arbitration -->|<--- data --->|<-- ack -->|
   |     500 kbit/s    |   2-8 Mbit/s  | 500 kbit/s|
   +-------------------+---------------+-----------+
    ID, RTR, control    DATA + CRC      ACK, EOF

   WHY THIS WORKS: the round-trip constraint (chapter 09)
   applies only while several nodes might be driving at
   once - during arbitration. Once one node has won, it
   is alone on the bus and no round trip is needed, so
   the bit time can shrink to whatever the physical layer
   supports.

   A genuinely elegant piece of protocol design: it
   removes the bandwidth limit without touching the
   property that made CAN worth using.

2. LONGER FRAMES. Up to 64 bytes rather than 8. DLC
   values above 8 are encoded non-linearly:
     9-15 mean 12, 16, 20, 24, 32, 48, 64 bytes.
   So the jump is 8, 12, 16, 20, 24, 32, 48, 64 - you
   cannot ask for 40.

ALSO: a stronger CRC (17 or 21 bit), and no RTR frames.

THROUGHPUT: 8 bytes at 500 kbit/s is about 3.6 kB/s of
payload after overhead. 64 bytes with a 2 Mbit/s data
phase is roughly 25x that, and it means a firmware
update over CAN goes from hours to minutes.
```

CLASSICAL AND FD NODES CANNOT SHARE A BUS

A classical CAN controller sees an FD frame's control bits as a form error and transmits an error frame, destroying it — so one legacy node makes FD traffic impossible. Some transceivers and controllers support "CAN FD tolerant" mode, where classical nodes ignore FD frames instead of objecting, but this must be supported by every node on the bus. In practice a vehicle migrates one whole bus segment at a time, with a gateway translating between them.

The higher-layer protocols

```
Raw CAN gives you 8 (or 64) bytes with an identifier.
Everything else is a protocol on top, and you will
almost always be implementing one of these:

ISO-TP (ISO 15765-2) - TRANSPORT
  Segments messages longer than one frame, up to 4095
  bytes, with flow control. The foundation of diagnostics.

    SINGLE FRAME    [0x0n][ up to 7 bytes ]
    FIRST FRAME     [0x1L][LL][ 6 bytes ]
    FLOW CONTROL    [0x30][BS][STmin]     <- receiver
    CONSECUTIVE     [0x2n][ 7 bytes ]        controls
                                             the rate

  The receiver dictates block size and minimum separation
  time, so a slow ECU can throttle a fast tester. This is
  the piece people re-implement badly; use a library.

UDS (ISO 14229) - DIAGNOSTICS
  Request/response services over ISO-TP. Read fault
  codes, read data by identifier, flash firmware, run
  routines. Service 0x22 (ReadDataByIdentifier) and 0x2E
  (WriteDataByIdentifier) are most of what a tool does.

J1939 - HEAVY VEHICLES
  29-bit IDs carrying structured meaning: priority,
  PGN (parameter group number), source address. Standard
  PGNs mean engine speed is the same message on every
  truck, regardless of manufacturer.

CANopen - INDUSTRIAL
  Object dictionary, PDOs for real-time data, SDOs for
  configuration, standard device profiles. Common in
  motion control and industrial automation.

AUTOSAR - AUTOMOTIVE SOFTWARE ARCHITECTURE
  Not a CAN protocol as such, but the framework most
  automotive ECU software is written inside, with CAN as
  one of its communication stacks.
```

ISO-TP, because you will meet it

```
/* Receiving a multi-frame message. The state machine is
   small; the timeouts are what make it correct.          */

typedef enum { TP_IDLE, TP_RECEIVING } tp_state_t;

typedef struct {
    tp_state_t state;
    uint8_t    buf[4095];
    uint16_t   expected, received;
    uint8_t    next_sn;          /* sequence number 0-15   */
    uint32_t   last_frame_ms;
} tp_rx_t;

void tp_on_frame(tp_rx_t *tp, const uint8_t *d, uint8_t len)
{
    uint8_t type = d[0] >> 4;

    switch (type) {
    case 0x0:                                  /* SINGLE   */
        deliver(&d[1], d[0] & 0x0F);
        tp->state = TP_IDLE;
        break;

    case 0x1:                                  /* FIRST    */
        tp->expected = (uint16_t)(((d[0] & 0x0F) << 8) | d[1]);
        if (tp->expected > sizeof(tp->buf)) {
            tp_send_fc_overflow();             /* refuse    */
            return;
        }
        memcpy(tp->buf, &d[2], 6);
        tp->received = 6;
        tp->next_sn  = 1;
        tp->state    = TP_RECEIVING;
        tp->last_frame_ms = g_ms;
        tp_send_flow_control(0 /* BS: unlimited */, 0 /* STmin */);
        break;

    case 0x2: {                                /* CONSEC   */
        if (tp->state != TP_RECEIVING) { return; }  /* stray */

        uint8_t sn = d[0] & 0x0F;
        if (sn != tp->next_sn) {
            /* A frame was LOST. The sequence number exists
               precisely to detect this; abort rather than
               deliver a message with a hole in it.        */
            stats.tp_sequence_errors++;
            tp->state = TP_IDLE;
            return;
        }
        tp->next_sn = (uint8_t)((sn + 1u) & 0x0F);

        uint16_t n = tp->expected - tp->received;
        if (n > 7) { n = 7; }
        memcpy(&tp->buf[tp->received], &d[1], n);
        tp->received += n;
        tp->last_frame_ms = g_ms;

        if (tp->received >= tp->expected) {
            deliver(tp->buf, tp->expected);
            tp->state = TP_IDLE;
```

```
        }
        break;
    }
    default:
        break;
    }
}

/* THE TIMEOUT IS NOT OPTIONAL. If the sender dies mid
   message, this state machine waits forever holding a 4 kB
   buffer. ISO 15765-2 specifies N_Cr = 1000 ms between
   consecutive frames.                                    */
void tp_poll(tp_rx_t *tp)
{
    if (tp->state == TP_RECEIVING &&
        (int32_t)(g_ms - tp->last_frame_ms - 1000) >= 0) {
        stats.tp_timeouts++;
        tp->state = TP_IDLE;
    }
}
```

What a vehicle bus actually looks like

```
A car has SEVERAL buses, segregated by criticality, with
a gateway between them.

+-----------------------------------------------------+
|                     GATEWAY ECU                     |
+--+--------------+--------------+-------------+-----+
   |              |              |             |
POWERTRAIN     CHASSIS         BODY         DIAGNOSTIC
500 kbit/s     500 kbit/s      125 kbit/s   OBD-II port
or CAN FD                      or LIN
   |              |              |
engine ECU     ABS            doors, lights
transmission   steering       climate, seats
               airbag

WHY SEGREGATED:
  - a fault on the body bus must not affect braking
  - bandwidth: the powertrain bus carries 10 ms cyclic
    messages; the body bus can be far slower and cheaper
  - SECURITY: the OBD-II port is physically accessible
    to anyone, so the gateway is a firewall. The famous
    2015 Jeep Cherokee remote attack pivoted from the
    infotainment system onto the CAN bus - after which
    the protocol offers no defence at all, because CAN
    has NO AUTHENTICATION WHATSOEVER.

THIS IS THE SECURITY REALITY OF CAN: any node can
transmit any identifier. There is no source
authentication, no encryption, and no way for a receiver
to know a frame is genuine. Security must come from:
  - segmentation and a gateway that filters
  - SecOC (AUTOSAR Secure Onboard Communication), which
    adds a truncated MAC and a freshness counter to
    critical messages
  - physical access control

If you are designing a CAN system today and it carries
anything that matters, assume the bus is hostile and
authenticate at the application layer. The cryptography
track's MAC chapter is directly relevant.
```

How it shows up

Symptom	Cause
FD frames destroyed by error frames	A classical node on the bus objecting to FD control bits
Cannot request a 40-byte frame	FD lengths are quantised: 32 then 48
FD works at low data rate, fails at 5 Mbit/s	Transceiver not rated for it, or stub lengths too long
Multi-frame messages truncated	Sequence number wrap not handled — it is modulo 16
ISO-TP holds a buffer forever	No N_Cr timeout between consecutive frames
Fast tester overruns a slow ECU	Flow control STmin ignored by the sender
Diagnostics work on one tool, not another	Block size or separation time handling differs
Firmware update over CAN takes hours	Classical CAN at 500 kbit/s — this is what FD is for
Spoofed messages accepted as genuine	CAN has no authentication; add SecOC or authenticate at the application layer

Symptom	Cause
A body-bus fault affects the powertrain	Buses not segregated, or the gateway is not filtering

Check yourself

- ✓ Explain why CAN FD can use a faster bit rate during the data phase.
- ✓ State the two limits CAN FD raises and by how much.
- ✓ Explain why FD data lengths above 8 bytes are quantised.
- ✓ Say what happens when a classical node shares a bus with FD traffic.
- ✓ Name the four ISO-TP frame types and what each carries.
- ✓ Explain what flow control lets the receiver dictate, and why that matters.
- ✓ Explain the purpose of the sequence number and the correct response to a gap.
- ✓ Say why the N_Cr timeout is required and what it protects.
- ✓ Give two reasons a vehicle has several segregated CAN buses.
- ✓ State CAN's security properties and name two ways to compensate.

USB: the bus, descriptors and enumeration

USB is by far the most complex bus in this track, and the one where a microcontroller almost never implements the hard parts itself. What you must understand is the model — endpoints, transfer types and descriptors — because that is the vocabulary every USB stack expects you to speak.

The model

```
USB IS STRICTLY HOST-DRIVEN. A device NEVER initiates.
The host polls; the device answers. Even an "interrupt"
endpoint is polled - the name is historical.

HOST                                DEVICE
+--------+                          +--------------+
|        |  ---- token: IN, ep 1 ---> | endpoint 1 IN|
|        |  <--- data packet -------- |              |
|        |  ---- ACK --------------->  |              |
+--------+                          +--------------+

ENDPOINT: a buffer in the device with a number (0-15) and
a direction. IN means device-to-host; OUT means
host-to-device. Directions are ALWAYS named from the
HOST's point of view, which is the first thing to get
straight.

ENDPOINT 0 is special: bidirectional, always present, and
carries all control traffic including enumeration.

INTERFACE: a group of endpoints implementing one function.
CONFIGURATION: a set of interfaces. Devices almost always
have exactly one configuration; the flexibility is
rarely used.

A composite device - say a USB headset - has several
interfaces in one configuration: audio out, audio in,
and HID for the volume buttons.
```

Four transfer types

Type	Guarantees	Used for
Control	Reliable, retried; 10% of bandwidth reserved	Enumeration and configuration, on endpoint 0
Bulk	Reliable, retried; no timing guarantee at all	Mass storage, serial ports, printers — anything where correctness beats latency
Interrupt	Reliable, retried; guaranteed poll interval	Keyboards, mice, sensors — small data at a bounded latency
Isochronous	Guaranteed bandwidth, *no retries*, no error recovery	Audio and video, where a late packet is worse than a lost one

```
THE TRADE-OFF THAT DEFINES USB:

  BULK gets whatever bandwidth is left over, but it never
  loses data. A mass-storage transfer may stall for
  milliseconds if the bus is busy; it will not corrupt.

  ISOCHRONOUS reserves bandwidth in every frame and
  never retries. A dropped audio sample is a click; a
  late one would be a gap in playback, which is worse.
  So it simply does not retry.

Choosing wrongly is a common design error: implementing
an audio device over bulk gives you glitch-free data with
unpredictable latency, which for audio is exactly the
wrong trade.

SPEEDS:
  Low   1.5 Mbit/s   HID only, 8-byte packets
  Full  12 Mbit/s    most MCU peripherals, 64-byte max
  High  480 Mbit/s   needs a ULPI PHY, 512-byte bulk
  Super 5 Gbit/s+    a different physical layer entirely,
                     essentially never on an MCU
```

Enumeration, step by step

```
What happens in the second after you plug the cable in.

1. ATTACH. The device pulls D+ (full speed) or D- (low
   speed) to 3.3 V through 1.5 kO. The host sees the
   line go high and knows something is there - and which
   speed it claims.

2. RESET. The host drives both lines low for 10 ms. The
   device resets its USB state machine and adopts
   ADDRESS 0.

3. GET_DESCRIPTOR (Device), 8 bytes. The host asks for
   the first 8 bytes only, because byte 7 is
   bMaxPacketSize0 and it does not yet know how large a
   packet the device can accept.

4. RESET again.

5. SET_ADDRESS. The host assigns 1-127. The device must
   complete the STATUS stage of this request at the OLD
   address, then switch. Getting that ordering wrong is
   a classic stack bug.

6. GET_DESCRIPTOR (Device), all 18 bytes. Now the host
   has VID, PID and class.

7. GET_DESCRIPTOR (Configuration), 9 bytes, then again
   for the full length reported. This returns the
   configuration, all interfaces, and all endpoints as
   ONE concatenated blob.

8. GET_DESCRIPTOR (String) for manufacturer, product,
   serial - in UTF-16LE.

9. The host matches VID/PID/class against its drivers
   and loads one.

10. SET_CONFIGURATION(1). The device is now CONFIGURED
    and its non-zero endpoints become usable.

IF ENUMERATION FAILS the host reports "device not
recognised" and gives you nothing else. A protocol
analyser or Wireshark's USB capture is the only
practical way to see which step failed.
```

Descriptors are the device's declaration

```
/* Descriptors are packed structs the device returns. They
   ARE the device's interface contract - everything the host
   knows comes from them.                                    */

typedef struct __attribute__((packed)) {
    uint8_t  bLength;             /* 18                     */
    uint8_t  bDescriptorType;     /* 1 = DEVICE             */
    uint16_t bcdUSB;              /* 0x0200 = USB 2.0       */
    uint8_t  bDeviceClass;        /* 0 = see interface      */
    uint8_t  bDeviceSubClass;
    uint8_t  bDeviceProtocol;
    uint8_t  bMaxPacketSize0;     /* 8, 16, 32 or 64        */
    uint16_t idVendor;            /* assigned by USB-IF     */
    uint16_t idProduct;
    uint16_t bcdDevice;
    uint8_t  iManufacturer;       /* string INDEX, not text */
    uint8_t  iProduct;
    uint8_t  iSerialNumber;
    uint8_t  bNumConfigurations;  /* almost always 1         */
} usb_device_descriptor_t;

/* A CDC-ACM (virtual serial port) configuration, as the
   host receives it - one contiguous blob.                */
static const uint8_t config_desc[] = {
    /* Configuration: 9 bytes, total length 67, 2 interfaces */
    9, 0x02, 67, 0, 2, 1, 0, 0xC0, 50,

    /* Interface 0: communications class, 1 endpoint      */
    9, 0x04, 0, 0, 1, 0x02, 0x02, 0x01, 0,
    /*   CDC functional descriptors (5+4+5+5 bytes)       */
    5, 0x24, 0x00, 0x10, 0x01,
    4, 0x24, 0x02, 0x02,
    5, 0x24, 0x06, 0, 1,
    5, 0x24, 0x01, 0, 1,
    /*   Endpoint 0x82: interrupt IN, 8 bytes, 255 ms     */
    7, 0x05, 0x82, 0x03, 8, 0, 255,

    /* Interface 1: CDC data, 2 endpoints                 */
    9, 0x04, 1, 0, 2, 0x0A, 0, 0, 0,
    /*   Endpoint 0x01: bulk OUT, 64 bytes                */
    7, 0x05, 0x01, 0x02, 64, 0, 0,
    /*   Endpoint 0x81: bulk IN, 64 bytes                 */
    7, 0x05, 0x81, 0x02, 64, 0, 0,
};

/* ENDPOINT ADDRESS ENCODING: bit 7 is direction.
     0x01 = endpoint 1, OUT
     0x81 = endpoint 1, IN
   These are DIFFERENT endpoints that share a number.

   THE TOTAL LENGTH FIELD (67 here) MUST BE EXACT. The host
   requests that many bytes; if the descriptors actually sum
   to a different number, enumeration fails in a way that
   gives no useful error. Compute it with sizeof() rather
   than by hand:

     #define CONFIG_TOTAL_LEN (sizeof(config_desc))
     ... and place it with a _Static_assert to check.     */
```

> **VENDOR IDS COST MONEY, AND THERE IS A LEGITIMATE ALTERNATIVE**
> A USB Vendor ID from the USB-IF costs several thousand dollars, and using someone else's without permission means your device can collide with theirs and cannot legally carry the USB logo. For hobby and internal work, pid.codes allocates free PIDs under a shared VID for open-source projects. For a product, budget for a real VID — or use a USB-to-serial bridge chip whose vendor's VID/PID you are licensed to use by buying the chip.

What this means on a microcontroller

```
/* You will use a stack. TinyUSB, the vendor's middleware,
   or Zephyr's - not your own. USB is a case where writing
   it yourself is a month of work to reach parity.

   WHAT YOU ACTUALLY DO:                                    */

/* 1. Provide descriptors. */
uint8_t const *tud_descriptor_device_cb(void) {
    return (uint8_t const *) &device_desc;
}

/* 2. Implement class callbacks. */
void tud_cdc_rx_cb(uint8_t itf) {
    uint8_t buf[64];
    uint32_t n = tud_cdc_read(buf, sizeof(buf));
    handle_input(buf, n);
}

/* 3. Service the stack, from a task or the main loop. */
for (;;) {
    tud_task();
    application_step();
}

/* THE HARDWARE REQUIREMENTS people underestimate:

   - AN ACCURATE CLOCK. USB full speed needs 0.25%. The
     internal RC oscillator is not sufficient unless the
     part has crystal-less USB, where it trims the RC
     against the host's start-of-frame packets. Otherwise:
     a crystal, and one whose frequency divides to exactly
     48 MHz (mcu chapter 09).

   - CORRECT D+/D- ROUTING. 90 ohm differential impedance,
     matched lengths, no stubs. USB is fast enough that
     layout matters - this is not a bus you can wire with
     jumper leads and expect to work.

   - VBUS SENSING if bus-powered, so the device knows when
     it is actually attached.

   - THE 1.5k PULL-UP, ideally software-controlled so the
     device can force a re-enumeration by dropping off the
     bus and reattaching. Without it, changing descriptors
     requires physically unplugging.                        */
```

How it shows up

Symptom	Cause
"Device not recognised", nothing else	Enumeration failed — capture it with a protocol analyser or Wireshark
Enumeration fails after the configuration request	`wTotalLength` does not match the actual descriptor bytes
Works on one host, not another	Descriptor that is technically invalid; Linux is more forgiving than Windows
Device enumerates then disconnects repeatedly	Clock accuracy — check the crystal and the 48 MHz derivation
Intermittent at high data rates only	D+/D− routing: impedance, length matching, or a stub
Endpoint 0x81 and 0x01 confused	Bit 7 is direction; they are different endpoints
Audio clicks and dropouts	Bulk used where isochronous is needed
Descriptor change needs a physical unplug	No software control of the D+ pull-up
Device works only when self-powered	VBUS sensing absent or miswired
Windows caches an old descriptor	It keys on VID/PID — bump `bcdDevice` or clear the registry entry during development

Check yourself

- ✓ Explain why a USB device can never initiate a transfer.
- ✓ Say whose point of view IN and OUT are named from.
- ✓ Name the four transfer types and the guarantee each makes.
- ✓ Explain why isochronous transfers do not retry.
- ✓ Explain why the host asks for only 8 bytes of the device descriptor first.
- ✓ Say what must happen at the old address during SET_ADDRESS.
- ✓ Explain what `wTotalLength` covers and the failure when it is wrong.
- ✓ Decode endpoint addresses 0x01 and 0x81.
- ✓ State USB full speed's clock accuracy requirement and two ways to meet it.
- ✓ Explain why a software-controlled pull-up is worth having.

USB device classes and writing a device

The single most valuable thing to know about USB is which standard class to claim, because claiming the right one means every operating system already has your driver. Getting this choice right is the difference between shipping a product and shipping a product plus three signed drivers.

The classes worth knowing

Class	Driver on Windows/macOS/Linux	Use for
CDC-ACM (serial)	Built in everywhere; Windows needs an INF before 10	A virtual COM port — debug consoles, legacy replacement
HID	Built in, no driver, no INF, ever	Keyboards, mice, and *any* low-rate custom data
MSC (mass storage)	Built in everywhere	Exposing a filesystem — logs, firmware drop-in
Audio (UAC)	Built in; UAC1 more compatible than UAC2	Speakers, microphones
Video (UVC)	Built in everywhere	Cameras
DFU	Needs WinUSB or a tool like dfu-util	Firmware update
Vendor-specific	WinUSB via a WCID descriptor; libusb elsewhere	High throughput custom protocols

```
THE DECISION:

Need a debug console?          -> CDC-ACM
Custom data, under ~64 kB/s?   -> HID
Custom data, high rate?        -> vendor-specific + WCID
Exposing files?                -> MSC
Firmware update?               -> DFU, or MSC drag-and-drop

HID IS UNDERRATED FOR CUSTOM DATA. It needs no driver, no
INF, no signing and no administrator rights on any
operating system, and browsers can talk to it directly
through WebHID. The cost is throughput: 64 KB/s at full
speed, 24 MB/s at high speed but only with careful
descriptor work.

For a configuration tool, a dongle, or a sensor that
reports a few kilobytes a second, HID removes an entire
category of deployment problem. Reach for it before
reaching for a vendor-specific interface.
```

Making a vendor-specific device driverless on Windows

```
/* Historically a vendor-specific device meant shipping a
   signed .inf and .cat. Microsoft OS descriptors (WCID)
   remove that: the device TELLS Windows to bind WinUSB.  */

/* Windows requests string descriptor 0xEE at enumeration.
   Return this magic string and it will follow up with
   vendor requests.                                       */
static const uint8_t msos_string[] = {
    0x12, 0x03,
    'M',0, 'S',0, 'F',0, 'T',0, '1',0, '0',0, '0',0,
    MSOS_VENDOR_CODE, 0x00
};

/* Then answer the compatible-ID request with "WINUSB". */
static const uint8_t msos_compat_id[] = {
    0x28,0,0,0,        /* dwLength = 40                   */
    0x00,0x01,         /* bcdVersion 1.0                  */
    0x04,0x00,         /* wIndex: compatible ID           */
    0x01,              /* one section                     */
    0,0,0,0,0,0,0,     /* reserved                        */
    0x00,              /* interface 0                     */
    0x01,              /* reserved                        */
    'W','I','N','U','S','B',0,0,   /* compatible ID       */
    0,0,0,0,0,0,0,0,               /* sub-compatible ID   */
    0,0,0,0,0,0                    /* reserved            */
};

/* Result: plug it in and Windows binds WinUSB with no
   driver install and no administrator rights. libusb then
   talks to it identically on Windows, macOS and Linux.

   ADD A DEVICE INTERFACE GUID via the extended properties
   descriptor and your application can find the device by
   GUID rather than enumerating everything.

   THIS IS THE MODERN ANSWER for custom USB devices, and
   it is much less widely known than it should be. TinyUSB
   and most stacks support it with a few lines of
   configuration.                                         */
```

A custom HID device

```
/* The HID report descriptor is a small stack-based language
   describing your data's structure. It is the fiddliest
   part of HID and the reason people avoid it - but a
   generic in/out report is a fixed template you can copy. */

static const uint8_t hid_report_desc[] = {
    0x06, 0x00, 0xFF,   /* Usage Page (vendor-defined)      */
    0x09, 0x01,         /* Usage (1)                        */
    0xA1, 0x01,         /* Collection (Application)         */

    0x09, 0x02,         /*   Usage (2) - input report       */
    0x15, 0x00,         /*   Logical Minimum (0)            */
    0x26, 0xFF, 0x00,   /*   Logical Maximum (255)          */
    0x75, 0x08,         /*   Report Size (8 bits)           */
    0x95, 0x40,         /*   Report Count (64 bytes)        */
    0x81, 0x02,         /*   Input (Data,Var,Abs)           */

    0x09, 0x03,         /*   Usage (3) - output report      */
    0x15, 0x00,
    0x26, 0xFF, 0x00,
    0x75, 0x08,
    0x95, 0x40,
    0x91, 0x02,         /*   Output (Data,Var,Abs)          */

    0xC0                /* End Collection                   */
};

/* 64 bytes in and 64 bytes out, opaque to the host. Now:  */

void tud_hid_set_report_cb(uint8_t itf, uint8_t id,
                           hid_report_type_t type,
                           uint8_t const *buf, uint16_t len)
{
    handle_command(buf, len);          /* host -> device   */
}

void send_reading(const reading_t *r)
{
    uint8_t rpt[64] = {0};
    memcpy(rpt, r, sizeof(*r));
    tud_hid_report(0, rpt, sizeof(rpt));   /* device -> host */
}

/* USE A DESCRIPTOR TOOL. The HID Descriptor Tool from
   USB-IF, or hidrd, will validate what you write. A subtly
   wrong report descriptor produces a device that enumerates
   successfully and then delivers reports the host silently
   discards - which is a miserable thing to debug by
   inspection.                                              */
```

WINDOWS CACHES DESCRIPTORS AGAINST VID/PID

During development, changing your descriptors and re-plugging often changes nothing, because Windows remembers the device from the first time it saw that VID/PID and does not re-read. Increment `bcdDevice` on every descriptor change, or delete the device from Device Manager with "delete the driver software" ticked, or use a different PID for each iteration. Hours are lost to this every year, and the symptom — "my change had no effect" — points at your code rather than the host.

Debugging USB

```
IN ORDER OF WHAT YOU SHOULD REACH FOR:

1. lsusb -v (Linux) / USB Tree View (Windows)
   Shows the descriptors as the host parsed them. If
   your device appears with the right VID/PID and the
   endpoints you expect, enumeration succeeded and the
   problem is above that layer.

2. WIRESHARK WITH usbmon (Linux) or USBPcap (Windows)
   Every packet, free, no hardware. This shows you which
   control transfer failed, and it is the tool that
   answers "why does it not enumerate".

3. dmesg (Linux)
   The kernel narrates enumeration failures in plain
   language: "device descriptor read/64, error -71".
   Often names the problem outright.

4. A HARDWARE PROTOCOL ANALYSER
   Beagle, Ellisys and similar. Needed only for
   electrical or timing problems - a device that
   enumerates on one host and not another, or one that
   fails at high speed. Expensive, and rarely the first
   resort.

5. GPIO + SCOPE inside your callbacks
   When the stack's behaviour itself is in question,
   toggling a pin in the endpoint callback tells you
   whether it is being called at all, and how often
   (mcu chapter 16).

THE ORDER MATTERS. People reach for the analyser first
and Wireshark last, when Wireshark answers the question
nine times out of ten and costs nothing.
```

How it shows up

Symptom	Cause
Descriptor changes have no effect	Windows cached the device — bump `bcdDevice` or remove it in Device Manager
Enumerates but the host discards every report	HID report descriptor subtly invalid — validate it with a tool
Needs a driver install on Windows	Vendor-specific without WCID — add the Microsoft OS descriptors
Works on Linux, not Windows	Descriptor technically invalid; Linux is more tolerant
Throughput far below expectations on HID	Poll interval too long, or report size too small for the rate
CDC device needs an INF on old Windows	Expected before Windows 10; ship one or use HID
Device disappears under load	Stack not serviced often enough — call the task loop more frequently
`dmesg` shows "error -71"	Electrical or clock problem, not a descriptor problem
Audio device stutters	Isochronous endpoint not filled every frame — the deadline is hard
Cannot open the device without root on Linux	Missing udev rule — ship one with the product

Check yourself

- ✓ Name four standard classes with built-in drivers on every OS.
- ✓ Explain why HID is attractive for custom low-rate data.
- ✓ State HID's practical throughput limit at full speed.
- ✓ Explain what WCID descriptors achieve on Windows.
- ✓ Say which string descriptor index Windows requests for OS descriptors.
- ✓ Explain what a HID report descriptor is and the failure mode when it is wrong.
- ✓ Describe why Windows appears to ignore your descriptor changes.
- ✓ Give the order in which to reach for USB debugging tools, and why.
- ✓ Say what `lsusb -v` tells you about whether enumeration succeeded.
- ✓ Explain when a hardware protocol analyser is actually necessary.

RS-485, Modbus and industrial serial

A UART reaches a few metres. RS-485 takes the same UART and carries it a kilometre, to 32 devices, through a factory full of electrical noise — for the cost of one transceiver chip and one piece of software discipline you must get exactly right.

What RS-485 changes

```
RS-232 (what a plain UART pin is, roughly)
  Single-ended: the voltage is measured against GROUND.
  Noise couples onto the signal wire and is
  indistinguishable from data. Ground offset between two
  buildings can exceed the signal itself.
  Reach: ~15 m at 9600 baud. One device per wire.

RS-485
  DIFFERENTIAL: two wires, A and B. The receiver reads
  the DIFFERENCE. Noise couples equally onto both and
  cancels. Same idea as CAN (chapter 09).
  Reach: 1200 m at 100 kbit/s. 32 standard loads,
  more with low-load transceivers.

           +--[120]--+                    +--[120]--+
           |         |                    |         |
  A  ------+---------+--------------------+---------+---
  B  ------+---------+--------------------+---------+---
           |         |                    |
        +---+---+  +--+----+           +---+---+
        |master |  |slave 1|    ...    |slave n|
        +-------+  +-------+           +-------+

HALF DUPLEX, and this is the crucial difference from
everything else in this track: ONE PAIR, shared, and only
one transmitter may be enabled at a time. There is NO
arbitration - unlike CAN or I2C, two simultaneous
transmitters simply corrupt each other and may damage the
drivers.

So the protocol on top MUST guarantee that only one node
talks at a time. In practice that means a master polls
and slaves answer only when addressed - which is exactly
what Modbus does.

RS-422 is the full-duplex cousin: two pairs, one master
transmitting to many receivers. No direction control
needed, but no multi-drop from the slaves either.
```

Direction control, the thing everyone gets wrong

```
/* The transceiver has a DRIVER ENABLE pin. Assert it to
   transmit, release it to listen. The timing of the
   RELEASE is where the bugs live.                          */

#define DE_HIGH()  (GPIOB->BSRR = (1u << 1))       /* TX     */
#define DE_LOW()   (GPIOB->BSRR = (1u << (1 + 16)))/* RX     */

/* WRONG - truncates the last byte, always.                 */
void rs485_send_broken(const uint8_t *p, size_t n)
{
    DE_HIGH();
    while (n--) {
        while (!(USART2->ISR & USART_ISR_TXE)) { }
        USART2->TDR = *p++;
    }
    DE_LOW();          /* TXE means the DATA REGISTER is
                          free. The last byte is STILL IN
                          THE SHIFT REGISTER (chapter 01).
                          Disabling the driver now cuts it
                          off mid-character.                */
}

/* RIGHT - wait for TRANSMISSION COMPLETE.                  */
void rs485_send(const uint8_t *p, size_t n)
{
    DE_HIGH();
    delay_us(1);                       /* driver settle    */

    while (n--) {
        while (!(USART2->ISR & USART_ISR_TXE)) { }
        USART2->TDR = *p++;
    }

    while (!(USART2->ISR & USART_ISR_TC)) { }   /* THE FIX  */
    DE_LOW();
}

/* BETTER STILL - let the hardware do it. Many modern UARTs
   drive the DE pin themselves with programmable setup and
   hold times, which removes the software race entirely.  */
USART2->CR3 |= USART_CR3_DEM;              /* DE mode      */
USART2->CR1 |= (8u << USART_CR1_DEAT_Pos)  /* assertion    */
            |  (8u << USART_CR1_DEDT_Pos); /* deassertion  */
/* Now DE is an alternate function pin and the timing is
   exact regardless of interrupts. If your part has this,
   use it - it eliminates a whole class of intermittent
   "the last byte is sometimes wrong" bugs.

   WITH DMA the same trap appears: the DMA transfer-complete
   interrupt fires when the last byte enters the UART, not
   when it leaves the pin. Wait for TC in the DMA callback
   before dropping DE.                                      */
```

FAIL-SAFE BIASING: AN IDLE BUS IS NOT A SAFE BUS

When no driver is enabled, the A/B pair floats. Noise then flips the receiver randomly and every node sees garbage that may resemble a start bit — so every node's UART reports framing errors continuously, and a receiver may lock onto noise as though it were a frame. The fix is fail-safe biasing: a pull-up on A and a pull-down on B, typically 560 Ω to 1 kΩ, fitted *once* on the bus, holding the idle state at a defined logical high. Many transceivers include internal fail-safe; many do not. If an RS-485 bus produces garbage when nothing is transmitting, this is the first thing to check.

Modbus RTU

```
Published in 1979 and still the default in industrial
automation, because it is simple enough to implement in
an afternoon and everything supports it.

FRAME:
+---------+----------+---------------+---------+
| ADDRESS | FUNCTION |      DATA     | CRC-16  |
|  1 byte |  1 byte  |   0-252 bytes | 2 bytes |
+---------+----------+---------------+---------+

FRAMING BY SILENCE: a frame starts after 3.5 character
times of silence and ends the same way. There is no start
or end delimiter byte at all.

AT 9600 BAUD, 8N1: one character is 11 bits = 1.146 ms
  t3.5 = 4.01 ms
  t1.5 = 1.72 ms  (max gap WITHIN a frame)

ABOVE 19200 BAUD the standard fixes these at 1.75 ms and
750 us rather than scaling them, because the character
times become impractically short.

THIS IS EXACTLY WHAT UART IDLE DETECTION IS FOR
(chapter 03). Modbus RTU was designed around a timing
gap, and the IDLE interrupt implements it in hardware.
The alternative - a timer restarted on every byte - works
but costs an interrupt per character.

COMMON FUNCTION CODES:
  0x01  read coils            (bits, read/write)
  0x02  read discrete inputs  (bits, read only)
  0x03  read holding registers  (16-bit, read/write)
  0x04  read input registers    (16-bit, read only)
  0x05  write single coil
  0x06  write single register
  0x10  write multiple registers

ADDRESS 0 IS BROADCAST - all slaves act, none reply.
Addresses 1-247 are valid slaves.

EXCEPTION RESPONSE: the slave replies with the function
code OR 0x80, plus an exception code. 0x02 is "illegal
data address", which is what you get when your register
map does not match the master's expectation - the single
most common Modbus integration problem.
```

```
/* CRC-16/MODBUS. Note the reflected polynomial 0xA001 and
   the 0xFFFF seed - both are specific to Modbus and using
   the wrong CRC-16 variant is a classic error, because
   there are at least a dozen of them.                    */

static uint16_t modbus_crc(const uint8_t *d, size_t n)
{
    uint16_t crc = 0xFFFFu;
    while (n--) {
        crc ^= *d++;
        for (int i = 0; i < 8; i++) {
            crc = (crc & 1u) ? (uint16_t)((crc >> 1) ^ 0xA001u)
                             : (uint16_t)(crc >> 1);
        }
    }
    return crc;                     /* transmitted LOW byte
                                       first - the ONLY
                                       little-endian field
                                       in the protocol      */
}

/* EVERYTHING ELSE IN MODBUS IS BIG-ENDIAN. Register values,
   addresses, counts: all high byte first. The CRC alone is
   low byte first. This inconsistency is a permanent source
   of bugs and it is not a mistake in your reading - it
   really is like that.                                    */

/* A slave, receiving via IDLE-triggered DMA (chapter 03). */
void modbus_on_frame(const uint8_t *f, uint16_t len)
{
    if (len < 4) { return; }                /* runt        */
    if (f[0] != MY_ADDRESS && f[0] != 0) { return; }

    uint16_t rx_crc = (uint16_t)(f[len-2] | (f[len-1] << 8));
    if (rx_crc != modbus_crc(f, len - 2u)) {
        stats.crc_errors++;
        return;               /* SILENTLY DISCARD - a bad
                                 CRC may mean the frame was
                                 not even for us            */
    }

    switch (f[1]) {
    case 0x03: handle_read_holding(f, len);  break;
    case 0x06: handle_write_single(f, len);  break;
    default:
        modbus_exception(f[1], 0x01);       /* illegal fn  */
    }
}

/* NEVER REPLY TO A BAD CRC. You cannot trust the address
   byte either, so replying risks two slaves transmitting
   at once - which on a half-duplex bus corrupts the next
   valid exchange too.                                     */
```

How it shows up

Symptom	Cause
Last byte of every reply corrupt	DE dropped on TXE instead of TC
Continuous framing errors when idle	No fail-safe biasing — the bus floats

Symptom	Cause
Works with two nodes, fails with ten	Termination, biasing, or too many unit loads
Intermittent errors at the far end only	Missing terminator, or a stub too long
Exception code 0x02 on every read	Register map offset — vendors disagree about 0-based versus 1-based
CRC never matches	Wrong CRC-16 variant, or byte order of the CRC field
Values are byte-swapped	Modbus is big-endian except the CRC
Two slaves reply at once	Duplicate addresses, or a slave replying to a bad CRC
Frames merge together	Inter-frame silence not enforced — use IDLE detection
Master times out on a slow slave	Response timeout shorter than the slave's processing time

Check yourself

- ✓ Explain why differential signalling reaches a kilometre where single-ended reaches fifteen metres.
- ✓ Say why RS-485 has no arbitration and what that requires of the protocol above it.
- ✓ Explain the difference between TXE and TC for driver-enable timing.
- ✓ Describe hardware DE control and why it is preferable.
- ✓ Explain fail-safe biasing and the symptom of its absence.
- ✓ State how Modbus RTU delimits frames and compute t3.5 at 9600 baud.
- ✓ Say which UART feature implements Modbus framing in hardware.
- ✓ Give the byte order of Modbus fields, including the exception.
- ✓ Explain why a slave must not reply to a frame with a bad CRC.
- ✓ Say what exception code 0x02 usually indicates in an integration.

MIPI CSI-2 and camera interfaces

A 12-megapixel sensor at 30 frames per second produces about 5 gigabits every second. No bus in the preceding chapters is within two orders of magnitude of that, which is why camera interfaces are a different kind of engineering — and why the control path and the data path are separate buses.

Why cameras need their own interface

```
THE ARITHMETIC:

  4000 x 3000 pixels x 10 bits x 30 fps
    = 3.6 Gbit/s of raw sensor data

SPI at 100 Mbit/s would need 36 lanes.
USB high speed at 480 Mbit/s cannot do it at all.

THE OLD ANSWER - PARALLEL DVP (digital video port):

  D0-D7 (or D0-D11)   data
  PCLK                pixel clock
  HSYNC               line valid
  VSYNC               frame valid

  12-16 pins, and every one of them switching at the
  pixel clock. At VGA/30fps that is fine; at anything
  modern the pin count and the EMI are both unacceptable.
  Still common on small MCU-class sensors (OV7670 and
  similar) and worth recognising.

THE MODERN ANSWER - MIPI CSI-2 over D-PHY:

  A clock lane plus 1-4 data lanes, DIFFERENTIAL, each
  lane carrying 1-2.5 Gbit/s.

  4 lanes x 2.5 Gbit/s = 10 Gbit/s on 10 wires.

PLUS A SEPARATE I2C BUS (called CCI - camera control
interface) for configuration. This split is the thing to
internalise:

  CSI-2  = pixels, one direction, very fast
  I2C    = registers, bidirectional, slow

You configure the sensor over I2C - resolution, exposure,
gain, frame rate - and it then streams pixels over CSI-2.
Two completely different buses to one chip, and bringing
up a camera means getting both right independently.
```

D-PHY's two modes

```
The physical layer switches between two electrical modes
on the same wires, and this is what makes it efficient.

HIGH-SPEED (HS) mode
  Differential, 200 mV swing around 200 mV common mode.
  Low voltage means low power and low EMI at gigabit
  rates. Used for the actual pixel burst.

LOW-POWER (LP) mode
  Single-ended, 1.2 V, both wires used independently as
  two slow signals. Used between bursts for control,
  and to signal entry into HS mode.

  LP-11  both high    = stop state, the idle condition
  LP-01, LP-00        = the HS entry sequence
  LP-10  turnaround (for bidirectional lane 0)

A FRAME LOOKS LIKE:

  LP-11 ... [HS burst: line 1] ... LP-11 ...
            [HS burst: line 2] ... LP-11 ...

The link drops to low power between every LINE. On a
battery device this matters enormously - the high-speed
drivers are only active while pixels are actually moving.

THIS IS WHY CSI-2 BRING-UP FAILS IN A CHARACTERISTIC WAY:
if the lanes never leave LP-11, the sensor is not
streaming at all - check the I2C configuration and the
sensor's clock and reset pins. If you see HS bursts but
no valid frames, the problem is packet-level: lane count,
data type, or virtual channel mismatch.
```

The packet layer

```
/* CSI-2 carries SHORT and LONG packets, distinguished by
   the data type field.                                      */

/* SHORT PACKET - 4 bytes, synchronisation only:
     Frame Start (0x00), Frame End (0x01),
     Line Start (0x02),  Line End (0x03)

   +--------+--------+--------+--------+
   | DI     | frame number    | ECC    |
   +--------+--------+--------+--------+

   LONG PACKET - carries pixel data:

   +--------+-----------------+--------+=========+--------+
   | DI     | word count      | ECC    | payload |CRC-16  |
   +--------+-----------------+--------+=========+--------+
     header: 4 bytes                     variable   2 bytes

   DI (data identifier) = virtual channel (2 bits) +
                          data type (6 bits)

   DATA TYPES you will meet:
     0x2A  RAW8      0x2B  RAW10     0x2C  RAW12
     0x1E  YUV422 8-bit
     0x24  RGB888
     0x30-0x37  user defined

   THE ECC on the header is a Hamming code that corrects one
   bit error and detects two. The header must survive - a
   corrupted word count would desynchronise the whole
   stream - so it gets stronger protection than the payload,
   which only has a CRC.

   VIRTUAL CHANNELS let several image streams share one
   physical link: a sensor sending both full-resolution and
   a downscaled preview, or two sensors through an
   aggregator. The receiver demultiplexes by VC number.

   RAW FORMATS ARE PACKED. RAW10 puts four pixels in five
   bytes:

     byte 0: P0[9:2]
     byte 1: P1[9:2]
     byte 2: P2[9:2]
     byte 3: P3[9:2]
     byte 4: P3[1:0] P2[1:0] P1[1:0] P0[1:0]

   Software that assumes two bytes per pixel gets an image
   that looks correct but with wrong colours and a
   progressive horizontal skew - a very recognisable
   symptom once you have seen it.                            */
```

THE SENSOR OUTPUTS BAYER, NOT COLOUR

Almost every image sensor has a colour filter array — one red, one blue and two green filters per 2×2 group — so each pixel measures only one colour. Turning that into RGB is *demosaicing*, and it is one stage of an image signal processor pipeline that also does black level, lens shading, white balance, denoise, gamma and sharpening. On an application processor the ISP is dedicated hardware; on a microcontroller there is usually none, which is why MCU camera work uses sensors that output YUV or RGB directly and accept the sensor's own internal processing. The video track covers the pipeline properly.

Bringing up a camera

```
/* The order matters, and each step has a distinct failure
   signature. Work through them in this sequence.         */

/* 1. POWER AND CLOCKS. Sensors need several rails in a
      documented ORDER, and an external master clock
      (XCLK/MCLK, often 24 MHz) before they respond to
      anything.                                           */
gpio_set(SENSOR_PWDN, 0);       /* release power-down      */
delay_ms(5);
mclk_start(24000000);           /* usually a timer PWM     */
delay_ms(5);
gpio_set(SENSOR_RESET, 1);      /* release reset AFTER
                                   the clock is running    */
delay_ms(20);

/* 2. I2C PROBE. Read the chip ID register. If this fails,
      nothing else can work - stop here and fix it. This
      single check separates "camera problem" from "camera
      not even present".                                  */
uint16_t id = sensor_read16(0x300A);
if (id != EXPECTED_ID) { return -ENODEV; }

/* 3. CONFIGURE. Sensor vendors supply a register list -
      often hundreds of undocumented address/value pairs.
      This is normal and you will not be able to derive it;
      start from the vendor's list for a known mode.       */
sensor_write_array(mode_1920x1080_30fps);

/* 4. RECEIVER CONFIGURATION MUST MATCH EXACTLY:
        - number of lanes
        - data type (RAW10 vs RAW12 vs YUV422)
        - virtual channel
        - line length and frame height
        - D-PHY timing (t_HS-SETTLE especially)
      A mismatch in ANY of these gives you no frames, or
      corrupt ones, with no error message beyond a receiver
      error counter.                                       */

/* 5. CHECK THE RECEIVER'S ERROR COUNTERS FIRST when
      debugging. On Linux:
        $ dmesg | grep -i csi
        $ media-ctl -p          # the pipeline topology
        $ v4l2-ctl --list-formats-ext
      ECC and CRC error counts tell you immediately whether
      the problem is electrical (errors climbing) or
      configuration (no errors, no frames).                */
```

How it shows up

Symptom	Cause
Sensor does not respond on I2C	Master clock not running, or reset released before the clock
Lanes stay in LP-11	Sensor not streaming — check I2C configuration and the mode
HS bursts present but no valid frames	Lane count, data type or virtual channel mismatch
Image skewed progressively across the frame	Line length wrong, or packed RAW format unpacked as two bytes per pixel
Colours wrong and image looks striped	RAW10/RAW12 packing not handled
Image is greyscale mosaic	Bayer data not demosaiced — there is no ISP in the path
ECC errors climbing	Electrical: lane length matching, impedance, or D-PHY timing
Works at low resolution, fails at high	Insufficient lanes or bandwidth for the pixel rate
Frames arrive but tear or duplicate	Receiver buffer bandwidth, or DMA not keeping up
Two sensors, only one works	Same virtual channel, or an I2C address collision on the control bus

Check yourself

- ✓ Compute the raw data rate for a 12 MP sensor at 30 fps and 10 bits.
- ✓ Explain why the control path and the data path are different buses.
- ✓ Name D-PHY's two electrical modes and what each is used for.
- ✓ Say what it means diagnostically if the lanes never leave LP-11.
- ✓ Distinguish short packets from long packets and give two examples of each.
- ✓ Explain why the packet header has ECC while the payload has only a CRC.
- ✓ Say what virtual channels are for.
- ✓ Describe RAW10 packing and the symptom of unpacking it wrongly.
- ✓ Explain what demosaicing is and why an MCU usually avoids needing it.
- ✓ Give the camera bring-up order and the failure signature of each step.

MIPI DSI and display interfaces

Displays are CSI-2 in reverse, with one important addition: some panels have memory and some do not, and that single fact decides whether your firmware sends a frame when something changes or sixty times a second forever.

The display interface spectrum

```
In increasing order of bandwidth and complexity:

SPI                  up to ~320x240
  A display controller with its own frame buffer. You
  send only what changed. Simple, slow, and completely
  adequate for a small UI. ILI9341, ST7789 and similar.

PARALLEL 8080/6800   up to ~800x480
  8 or 16 data lines plus control strobes. Still a
  controller with memory; just a wider path to it.
  Often driven by an MCU's FSMC/FMC memory controller,
  so writing a pixel is a store to an address.

RGB PARALLEL (DPI)   up to ~1024x600
  NO MEMORY IN THE PANEL. 16-24 data lines plus HSYNC,
  VSYNC, DE and PCLK, and you must supply every pixel of
  every frame, continuously, forever. Needs an LCD
  controller peripheral (LTDC) and a frame buffer in RAM.

MIPI DSI             1080p and beyond
  1-4 differential lanes, same D-PHY as CSI-2
  (chapter 15). Panels may or may not have memory.

LVDS / eDP           laptop-class panels
  Different physical layer, same fundamental split.

THE SPLIT THAT MATTERS MORE THAN THE WIRING:

COMMAND MODE / with GRAM
  The panel has RAM. You write a region when it changes
  and stop. The panel refreshes itself.
  -> low average power, simple firmware, ideal for a
     mostly-static UI.

VIDEO MODE / no memory
  You stream every frame at the refresh rate, with
  blanking intervals, exactly like an old CRT.
  -> constant bandwidth and power whether anything
     changes or not, but no tearing and no panel RAM cost.

A wearable with a static watch face wants command mode.
A device playing video wants video mode. Choosing the
panel decides which firmware you write.
```

Video mode timing

```
Video mode inherits CRT vocabulary, and the blanking
intervals are real time you must budget for.

|<----------------- HTOTAL ------------------>|
|<-HSA->|<-HBP->|<--- HACT --->|<-HFP->|
+-------+-------+---------------+-------+
| sync  | back  |   PIXELS      | front |
|       | porch |               | porch |
+-------+-------+---------------+-------+

Same structure vertically: VSA, VBP, VACT, VFP.

PIXEL CLOCK = HTOTAL x VTOTAL x refresh_rate

WORKED, for 800x480 at 60 Hz:
  HTOTAL = 48 + 40 + 800 + 40 = 928
  VTOTAL = 3 + 29 + 480 + 13  = 525
  PCLK   = 928 x 525 x 60 = 29.2 MHz

BANDWIDTH at 24 bits per pixel:
  29.2e6 x 24 = 701 Mbit/s
Two DSI lanes at 500 Mbit/s each covers it.

NOTE THE OVERHEAD: 928x525 = 487,200 clocks to deliver
384,000 pixels. Twenty percent of your bandwidth is
blanking. The porch values come from the PANEL DATASHEET
and are not negotiable - they are the time the panel's
internal circuitry needs.

FRAME BUFFER COST:
  800 x 480 x 3 bytes = 1.1 MB
That does not fit in any microcontroller's SRAM, which
is why RGB parallel and DSI video mode need external
SDRAM. This single number is often what forces a design
from an MCU to an application processor - or to a
command-mode panel with its own memory.
```

Talking to the panel

```
/* DSI carries both pixels and commands on the same lanes.
   DCS (Display Command Set) is the standard command
   vocabulary, and lane 0 is bidirectional so the panel can
   answer.                                                  */

/* Standard DCS commands - the same on every compliant panel */
#define DCS_ENTER_SLEEP_MODE   0x10
#define DCS_EXIT_SLEEP_MODE    0x11
#define DCS_SET_DISPLAY_OFF    0x28
#define DCS_SET_DISPLAY_ON     0x29
#define DCS_SET_COLUMN_ADDRESS 0x2A
#define DCS_SET_PAGE_ADDRESS   0x2B
#define DCS_WRITE_MEMORY_START 0x2C
#define DCS_SET_TEAR_ON        0x35

static void panel_init(void)
{
    dsi_dcs_write(DCS_EXIT_SLEEP_MODE, NULL, 0);
    delay_ms(120);              /* datasheet: mandatory     */

    /* Then the VENDOR's initialisation sequence: dozens of
       manufacturer-specific commands setting gamma curves,
       power rails and timing. Undocumented, supplied as a
       blob, and NOT derivable. Every panel needs its own,
       and the panel vendor must give you one - if they
       will not, choose another panel.                      */
    panel_vendor_init_sequence();

    dsi_dcs_write(DCS_SET_DISPLAY_ON, NULL, 0);
}

/* COMMAND MODE UPDATE - only the changed rectangle. */
void panel_blit(uint16_t x, uint16_t y, uint16_t w, uint16_t h,
                const uint16_t *pixels)
{
    uint8_t col[4] = { x >> 8, x & 0xFF,
                       (x+w-1) >> 8, (x+w-1) & 0xFF };
    uint8_t page[4] = { y >> 8, y & 0xFF,
                        (y+h-1) >> 8, (y+h-1) & 0xFF };

    dsi_dcs_write(DCS_SET_COLUMN_ADDRESS, col,  4);
    dsi_dcs_write(DCS_SET_PAGE_ADDRESS,   page, 4);
    dsi_dcs_write_memory(pixels, (size_t) w * h * 2u);
}

/* Updating a 100x20 status area costs 4 kB instead of the
   750 kB a full 800x480 frame would - a 180x saving, and
   the difference between a responsive UI and a slideshow
   on a slow link.                                          */
```

TEARING, AND THE TE SIGNAL

If you write the panel's memory while it is scanning that region out, the top of the display shows the new image and the bottom the old — a visible tear that moves. Command-mode panels provide a *tearing effect* output that pulses at the start of vertical blanking; wait for it before starting an update and the panel scans out only complete frames. It costs one pin and one interrupt. Video-mode panels avoid the problem differently, by double-buffering in the host and switching buffers during blanking.

Displays on a microcontroller

```
/* Two viable strategies, and the choice follows from the
   frame buffer arithmetic above.                          */

/* STRATEGY 1: SPI or parallel panel WITH memory.
   No frame buffer in RAM at all - draw directly into the
   panel, one region at a time.                            */
void draw_button(int x, int y, const char *label)
{
    panel_fill_rect(x, y, 100, 40, COLOUR_BLUE);
    panel_draw_text(x + 10, y + 12, label, COLOUR_WHITE);
}
/* Works on a part with 32 kB of RAM. The limit is redraw
   speed: at 40 MHz SPI, a full 320x240 16-bit refresh is
   1.5 Mbit = 38 ms. Fine for a UI, useless for animation. */

/* STRATEGY 2: RGB parallel with an LTDC and external SDRAM.
   A real frame buffer, hardware layer compositing, and
   DMA2D for accelerated fills and blits.                  */
LTDC_Layer1->CFBAR = (uint32_t) framebuffer;
DMA2D->CR = DMA2D_CR_MODE_R2M;      /* register to memory  */
DMA2D->OCOLR = 0xFF0000FF;
DMA2D->CR |= DMA2D_CR_START;        /* hardware fill       */

/* PARTIAL REFRESH IS THE KEY OPTIMISATION EITHER WAY. Track
   dirty rectangles and update only those. A UI that redraws
   the whole screen for a blinking cursor is the most common
   embedded graphics performance mistake, and the fix is
   almost always bookkeeping rather than a faster bus.

   LVGL and similar libraries do this dirty-rectangle
   tracking for you and are worth using rather than
   reimplementing.                                          */
```

How it shows up

Symptom	Cause
Panel stays black, backlight on	Vendor init sequence missing or incomplete
Image rolls or is offset	Porch timings do not match the panel datasheet
Image tears horizontally	Writing during scan-out — use the TE signal or double-buffer
Colours inverted or byte-swapped	Pixel format mismatch: RGB565 versus BGR565, or endianness
Bottom of the screen shows garbage	Frame buffer smaller than the configured resolution
UI feels sluggish	Full-screen redraws; track dirty rectangles
LTDC underrun errors	SDRAM bandwidth exhausted — reduce colour depth or refresh rate
Works at 30 Hz, fails at 60 Hz	Pixel clock or lane bandwidth exceeded
Panel needs a different init after a power cycle	Sleep-out delay too short — the 120 ms is mandatory
Cannot fit a frame buffer	Correct — use a command-mode panel with its own memory

Check yourself

✓ Name five display interfaces in order of bandwidth.

- ✓ Explain the difference between command mode and video mode.
- ✓ Say which mode suits a mostly-static UI and why.
- ✓ Compute the pixel clock for 800×480 at 60 Hz with given porches.
- ✓ Explain what fraction of bandwidth blanking consumes and why it exists.
- ✓ Compute the frame buffer size for 800×480 at 24 bits and say what it implies.
- ✓ Explain what DCS is and why a vendor init sequence is still needed.
- ✓ Describe tearing and the two ways to prevent it.
- ✓ Give the bandwidth saving from a partial update of a small region.
- ✓ Explain why dirty-rectangle tracking matters more than bus speed.

Ethernet, the MAC/PHY boundary and MDIO

Ethernet on a microcontroller is three separate things that must all work: a MAC inside the chip, a PHY outside it, and a management bus connecting them. Most bring-up problems are at one of the two boundaries, and knowing which one you are looking at is most of the diagnosis.

MAC and PHY

```
+-------------------------------------------------+
|  MICROCONTROLLER                                |
|                                                 |
|   TCP/IP stack (lwIP)                           |
|        |                                        |
|   +----+-----+                                  |
|   |   MAC    |  frames, addresses, CRC, DMA     |
|   +----+-----+                                  |
|        |                                        |
+--------|----------------+----------------------+
         | RMII/MII       | MDIO (2 wires)
         | (7 or 18 wires)|
+--------+----------------+----------------------+
|   PHY  (external chip)                          |
|   encoding, analog drive, link negotiation      |
+--------+----------------------------------------+
         |
     [magnetics]      transformer - isolation and
         |            common-mode rejection
       RJ45

THE DIVISION OF LABOUR:

MAC (in the MCU)     frame assembly, source/destination
                     addresses, CRC, collision handling,
                     DMA descriptors

PHY (external)       4B/5B or similar line coding,
                     analog line driving, auto-
                     negotiation of speed and duplex,
                     link detection

MDIO/MDC             a tiny two-wire management bus so
                     the MAC can read and write the PHY's
                     registers. Essentially I2C's job,
                     with its own protocol.

MAGNETICS            galvanic isolation to 1.5 kV, and
                     common-mode rejection. NOT optional -
                     omitting them is a real design error
                     that works on a bench and destroys
                     ports in the field.
```

MII versus RMII

```
MII    18 wires, 25 MHz clock, 4 bits per direction
RMII    7 wires, 50 MHz clock, 2 bits per direction
RGMII  12 wires, 125 MHz DDR, for gigabit

RMII is what you will use on a microcontroller: half the
pins for the same 100 Mbit/s.

RMII SIGNALS:
  REF_CLK    50 MHz, SHARED by MAC and PHY
  TXD0, TXD1
  TX_EN
  RXD0, RXD1
  CRS_DV     carrier sense / data valid

THE REF_CLK IS THE CLASSIC BRING-UP PROBLEM. Both ends
must use the SAME 50 MHz clock, and there are two valid
topologies:

  A) A 50 MHz oscillator feeds both the PHY and the MCU's
     REF_CLK input. Simple and robust.

  B) The MCU outputs 50 MHz on its MCO pin to the PHY,
     derived from its own PLL.

Topology B requires that the MCU's clock configuration
produce EXACTLY 50 MHz, and it must be running before the
PHY is released from reset. Getting this wrong gives a
PHY that never establishes a link, or a link that comes
up and passes no traffic - and neither symptom points at
the clock.

MEASURE REF_CLK WITH A SCOPE during bring-up. It is one
probe and it eliminates the most likely cause first.
```

MDIO: reading the PHY

```
/* MDIO is a 2-wire bus: MDC (clock, up to 2.5 MHz) and
   MDIO (bidirectional data). The MAC has a register pair
   that drives it for you.                                  */

static uint16_t phy_read(uint8_t phy_addr, uint8_t reg)
{
    ETH->MACMIIAR = (phy_addr << 11) | (reg << 6)
                  | ETH_MACMIIAR_CR_Div42 | ETH_MACMIIAR_MB;
    while (ETH->MACMIIAR & ETH_MACMIIAR_MB) { }   /* + timeout! */
    return (uint16_t) ETH->MACMIIDR;
}

/* THE STANDARD REGISTERS - IEEE 802.3 defines 0-15
   identically for every PHY ever made, which means this
   code works with any of them:                             */

/*  0  BMCR    control: reset, speed, duplex, autoneg
    1  BMSR    status: LINK UP, autoneg complete
    2  PHYIDR1 |  OUI and model - identifies the chip
    3  PHYIDR2 |
    4  ANAR    what we advertise
    5  ANLPAR  what the link partner advertises
   16+ vendor specific - including, annoyingly, the
       resolved speed and duplex on most PHYs           */

int phy_link_status(uint8_t addr, int *speed, int *full_duplex)
{
    /* BMSR bit 2 is LATCHING LOW: it reports "link has been
       down since the last read". Read it TWICE to get the
       current state - a genuine and very commonly missed
       detail.                                              */
    (void) phy_read(addr, 1);
    uint16_t bmsr = phy_read(addr, 1);

    if (!(bmsr & (1u << 2))) { return 0; }        /* down    */
    if (!(bmsr & (1u << 5))) { return 0; }        /* autoneg
                                                     not done */

    /* Resolved speed/duplex is vendor-specific. For the
       common LAN8742 it is register 31.                    */
    uint16_t sr = phy_read(addr, 31);
    *speed       = ((sr >> 2) & 3u) <= 1 ? 10 : 100;
    *full_duplex = (sr >> 4) & 1u;
    return 1;
}

/* FINDING THE PHY ADDRESS: it is set by strapping pins and
   is usually 0 or 1, but vendors vary. Scan for it - the
   same approach as an I2C scan (chapter 06):              */
int phy_scan(void)
{
    for (uint8_t a = 0; a < 32; a++) {
        uint16_t id = phy_read(a, 2);
        if (id != 0xFFFF && id != 0x0000) { return a; }
    }
    return -1;      /* nothing responded: check MDC, the
                       PHY's reset pin, and its supply     */
}
```

THE BRING-UP ORDER THAT ISOLATES FAULTS

Check them in this sequence and each step rules out a layer: **1.** Is REF_CLK present and 50 MHz? **2.** Does an MDIO scan find the PHY? **3.** Does the PHY report link up and auto-negotiation complete? **4.** Does the MAC receive any frames at all (check the receive descriptor count, not the IP stack)? **5.** Does a broadcast ARP appear? Only then look at lwIP. Debugging from the TCP layer downward is why Ethernet bring-up has a reputation for taking days.

Descriptors and buffers

```
/* The MAC does not have registers you write frames into.
   It walks a RING of DESCRIPTORS in memory, each pointing
   at a buffer - the same pattern as any high-throughput
   DMA engine (mcu chapter 12).                              */

typedef struct {
    volatile uint32_t status;      /* OWN bit + flags      */
    uint32_t          ctrl_len;
    uint32_t          buf1_addr;
    uint32_t          next_desc;
} eth_desc_t;

#define DESC_OWN  (1u << 31)       /* 1 = DMA owns it      */

/* THE OWNERSHIP BIT IS THE WHOLE PROTOCOL:
     OWN = 1  the DMA engine may use this descriptor
     OWN = 0  software owns it

   RX: software clears OWN after processing, handing the
       buffer back. FORGET THIS AND RECEPTION STOPS after
       exactly as many frames as you have descriptors -
       a very recognisable symptom.

   TX: software sets OWN to hand the frame over.          */

static eth_desc_t rx_desc[4] __attribute__((aligned(4)));
static uint8_t    rx_buf[4][1524] __attribute__((aligned(4)));

void eth_rx_poll(void)
{
    eth_desc_t *d = &rx_desc[rx_index];
    if (d->status & DESC_OWN) { return; }      /* DMA still
                                                  owns it   */
    uint16_t len = (uint16_t)((d->status >> 16) & 0x3FFF);
    lwip_input(rx_buf[rx_index], len - 4u);    /* strip CRC  */

    d->status = DESC_OWN;                      /* GIVE IT BACK */
    rx_index = (rx_index + 1u) & 3u;
    ETH->DMARPDR = 0;                          /* poll demand */
}

/* MEMORY PLACEMENT MATTERS, exactly as in mcu chapter 12:
   - descriptors and buffers must be in RAM the Ethernet
     DMA can reach. On STM32F7/H7 the default SRAM often is
     NOT reachable - use a specific region and place it in
     the linker script.
   - on Cortex-M7, either mark that region non-cacheable in
     the MPU, or do explicit cache maintenance around every
     descriptor access. Getting this wrong produces an
     Ethernet port that works for a few seconds and then
     stops, which is a memorably frustrating bug.          */
```

How it shows up

Symptom	Cause
MDIO scan finds nothing	No REF_CLK, PHY held in reset, or MDC divider misconfigured
PHY found but link never comes up	REF_CLK wrong frequency, magnetics or cable fault

Symptom	Cause
Link up, no frames received	RMII pin configuration, or descriptors in unreachable RAM
Receives exactly N frames then stops	OWN bit not returned to the DMA — N is your descriptor count
Works a few seconds then stops, Cortex-M7	Cache coherency on descriptors — mark the region non-cacheable
Link status flaps in software but not physically	BMSR link bit is latching low; read it twice
Works at 10 Mbit, fails at 100	Signal integrity, or a duplex mismatch from failed auto-negotiation
Ports damaged in the field	Magnetics omitted or wrongly specified
Occasional CRC errors under load	REF_CLK jitter, or clock topology B with a marginal PLL
TCP is slow but ping is fine	Descriptor count too low, or buffers being copied unnecessarily

Check yourself

- ✓ State which functions belong to the MAC and which to the PHY.
- ✓ Explain what the magnetics do and why they are not optional.
- ✓ Give the wire count and clock rate for MII and RMII.
- ✓ Describe the two valid REF_CLK topologies and the risk of each.
- ✓ Explain what MDIO is for and which registers are standardised.
- ✓ Explain why the BMSR link bit must be read twice.
- ✓ Describe how to find an unknown PHY address.
- ✓ Explain the descriptor ownership bit and the symptom of not returning it.
- ✓ Say why descriptor placement matters on an STM32H7.
- ✓ Give the five-step bring-up order and what each step rules out.

Reading a datasheet and a timing diagram

A datasheet is not a manual and is not meant to be read in order. It is a reference organised by an internal logic that nobody explains, and knowing that logic — plus how to read a timing diagram — is a skill that saves more time than any amount of coding ability.

What is in one, and the order to read it

```
THE STANDARD STRUCTURE:

 1. Features / description     marketing. Skim.
 2. Pinout and pin functions   check package variants
 3. Absolute maximum ratings   DESTRUCTION limits
 4. Recommended operating      the ranges you must
    conditions                 actually design within
 5. Electrical characteristics DC: voltages, currents
 6. Timing characteristics     AC: the numbers that
                               decide your driver
 7. Functional description     how it works
 8. Register map               what you write
 9. Application information    reference schematic
10. Package / ordering         which part to buy

THE ORDER TO ACTUALLY READ IT, first time:

1. BLOCK DIAGRAM (in section 7). Five minutes, and it
   orients everything else.
2. PIN TABLE. Which pins must be tied, which float,
   which are supplies. Note every pin marked "must be
   connected to ground" - these are usually thermal pads
   or unused supplies and omitting them is fatal.
3. RECOMMENDED OPERATING CONDITIONS. Not absolute
   maximums - those are where it BREAKS, not where it
   WORKS.
4. THE INTERFACE TIMING DIAGRAM for your bus.
5. THE REGISTER MAP, only for the registers you need.
6. APPLICATION SCHEMATIC. Copy it. The decoupling and
   the pull-ups shown there are requirements, not
   suggestions.

ABSOLUTE MAXIMUM vs RECOMMENDED, because this confusion
destroys parts:

  Absolute max VIN:     -0.3 V to VDD + 0.3 V
    Beyond this the part may be damaged. Not "may
    misbehave" - damaged, permanently, possibly slowly.

  Recommended VIH:      0.7 x VDD minimum
    Below this a logic high may not be read as high.

A 3.3 V signal into a 1.8 V part exceeds absolute maximum
by 1.2 V. It will appear to work. It will fail in
weeks.
```

Reading a timing diagram

```
A typical SPI write, with the symbols that matter:

        |<-tCSS->|                    |<-tCSH->|
CS   ---+        +--------------------+        +-------
        |________|                    |________|
                 :                    :
SCLK  ___________:__|--|__|--|__|--|__:___________
                 :  ^     ^     ^     :
                 :  |<tSU>|<tH>|     :
MOSI  XXXXXXXXXX:XX===X=====X=====XXX:XXXXXXXXXXX
                 :                    :
MISO  ---------- ZZZ==X=====X=====ZZZ ------------
                    |<tV>|

THE FIVE SYMBOLS YOU MEET EVERYWHERE:

tSU  SETUP TIME. How long data must be STABLE BEFORE the
     clock edge. YOU must guarantee this.

tH   HOLD TIME. How long data must remain stable AFTER
     the edge. YOU must guarantee this.

tV   VALID / OUTPUT DELAY. How long AFTER the edge before
     the device's output is valid. THE DEVICE imposes
     this on you, and it is what limits clock speed
     (chapter 04).

tCSS/tCSH  Chip select setup and hold, around the
     transaction.

tPD  PROPAGATION DELAY, generally.

THE ASYMMETRY IS THE KEY INSIGHT:
  - SETUP and HOLD are YOUR obligations to the device.
  - VALID DELAY is the device's obligation to you, and
    it constrains how fast you may clock.

WAVEFORM NOTATION:
  ===X===   valid data, value irrelevant
  XXXXXX    don't care / undefined
  ZZZZZZ    high impedance, nobody driving
  dashed    optional or conditional

MIN, TYP, MAX - and which one to design against:

  For times YOU must meet (setup, hold): use the MAX
    required value. Meeting the typical is not enough.
  For times the DEVICE promises (valid delay): use the
    MAX, because that is the worst case you must
    tolerate.
  TYP is for estimating average behaviour. NEVER design
    a timing budget from typical values - the part that
    fails will be the one at the limit, and there will
    be one in every thousand.
```

A worked timing check

```
/* QUESTION: can I clock this ADC at 20 MHz?

   FROM THE DATASHEET:
     tV  (SCLK falling to MISO valid)   max 25 ns
     tSU (MISO setup at the MCU)        min  5 ns
     board propagation, both ways       ~2 ns

   AT 20 MHz the period is 50 ns, so half a period - the
   time between the driving edge and the sampling edge - is
   25 ns.

   BUDGET:
     tV                     25 ns
     propagation out+back    2 ns
     MCU setup requirement   5 ns
                           -------
     total                  32 ns   needed
     available              25 ns

   DOES NOT FIT. 32 > 25.

   MAXIMUM WORKABLE CLOCK:
     half period must be >= 32 ns
     period >= 64 ns
     f <= 15.6 MHz

   So 12.5 MHz (the next prescaler step down) is the answer.

   WHAT IF I NEED 20 MHz ANYWAY? Three options, in order of
   preference:
     1. Use the MCU's "sample at the falling edge instead"
        option if it has one - some SPI peripherals allow
        delaying the sample point by half a clock, which
        doubles the available window.
     2. Shorten the traces.
     3. A faster ADC.

   THIS CALCULATION TAKES TWO MINUTES and it is the
   difference between choosing a clock rate by reasoning
   and choosing it by trying values until one works. The
   second approach leaves you at an unknown margin, and the
   failures appear in production across temperature.       */
```

THE THREE DOCUMENTS, AND WHICH ANSWERS WHAT

For a microcontroller the information is split across at least three files, and reaching for the wrong one wastes an hour: the **datasheet** holds pinout, electrical limits, timing and package — anything about a specific part number; the **reference manual** holds peripheral descriptions and register maps, shared across a whole family; the **errata sheet** holds the bugs. Read the errata before debugging anything that "should work" — silicon bugs in I2C peripherals, DMA engines and low-power modes are common, documented, and invisible unless you look.

What datasheets do not say plainly

```
THINGS THAT ARE THERE BUT EASY TO MISS:

"This pin must be left unconnected"
  Really means it. Often an internal test or regulator
  node; grounding it can damage the part.

A CONDITIONS COLUMN on every specification
  "IOL = 4 mA, VDD = 3.0 V, TA = 25 C". Your conditions
  differ, so the number differs. Check the conditions
  before trusting the value.

"Guaranteed by design" / "not tested"
  This parameter is not measured in production. Treat it
  with more caution than a tested one.

POWER SEQUENCING
  Often a small note, occasionally a diagram: "VDDIO
  must not exceed VDDCORE by more than 0.3 V during
  ramp". Violating it can latch up the part on every
  power-on, and the symptom is a board that sometimes
  does not start.

The RESET timing
  Minimum reset pulse width and minimum time after
  release before the first transaction. Both are
  routinely violated by init code that runs too soon.

FOOTNOTES UNDER THE TABLES
  Genuinely important information lives there:
  "(3) Not available in the 20-pin package."

THINGS THAT ARE NOT THERE AT ALL, and you must ask the
vendor or find out empirically:
  - the full register map, on some sensors
  - initialisation sequences (chapter 16's panel blob)
  - real power consumption in your exact use case
  - behaviour outside the specified ranges
  - how the part fails
```

How it shows up

Symptom	Cause
Part works for weeks then dies	Absolute maximum exceeded — usually an input above VDD
Works on the bench, fails in production	Designed against typical rather than worst-case timing
Works at room temperature only	Specification conditions differ from your operating range
One board in fifty does not start	Power sequencing or reset timing violated
Pin has an unexpected function	Package-variant footnote under the pin table
Peripheral behaves against the manual	Check the errata sheet before your own code
Bus fails above a certain clock	Output valid delay — do the timing budget
Measured current far above the datasheet	Different conditions: clock, temperature, or a pin loaded
Sensor needs an undocumented init blob	Normal — obtain it from the vendor
Damaged part after grounding a spare pin	"Leave unconnected" meant it

Check yourself

- ✓ Give the order to read a datasheet for the first time and why the block diagram comes first.
- ✓ Distinguish absolute maximum from recommended operating conditions.
- ✓ Define setup, hold and valid delay, and say which are your obligations.
- ✓ Explain why valid delay limits clock speed but setup time does not.
- ✓ Say which of min, typ and max to design against, for each kind of parameter.
- ✓ Work a timing budget to find the maximum clock for a given valid delay.
- ✓ Name the three documents for a microcontroller and what each answers.
- ✓ Explain what "guaranteed by design" implies.
- ✓ Give two things a datasheet often does not tell you at all.
- ✓ Explain why the conditions column matters as much as the value.

Signal integrity for firmware engineers

You will not design the board, but you will be handed bugs that live in the copper, and you need to recognise them before spending a week in the debugger. This chapter is the minimum electrical model that lets you say "this is not a software problem" with evidence.

It is the edge rate, not the clock rate

```
The intuition that misleads everyone: "this bus is only
100 kHz, so signal integrity does not apply."

IT IS THE RISE TIME THAT MATTERS, NOT THE FREQUENCY. A
100 kHz square wave with a 1 ns edge contains energy up
to hundreds of megahertz - and it is that energy which
radiates, reflects and couples.

  f_knee ~ 0.5 / t_rise

  t_rise = 1 ns   ->  f_knee = 500 MHz
  t_rise = 10 ns  ->  f_knee =  50 MHz

WHEN IS A TRACE A TRANSMISSION LINE?

  When its propagation delay exceeds about a sixth of
  the rise time.

  Signals travel at roughly 15 cm/ns on FR4.

  t_rise = 1 ns   -> critical length ~2.5 cm
  t_rise = 5 ns   -> critical length ~12 cm

So a 1 ns edge on a 5 cm trace IS a transmission line and
will reflect, whatever the clock rate. THIS IS WHY SLEW
RATE CONTROL EXISTS on GPIO pins (mcu chapter 10), and
why setting everything to maximum speed causes problems
that setting it to minimum does not.

THE FIRMWARE LEVER: you control the slew rate setting.
Use the slowest that meets timing. It costs nothing and
it removes an entire class of problem.
```

Reflections and termination

```
A signal reaching an impedance change reflects part of
its energy back.

  reflection coefficient = (Z_load - Z_line) /
                           (Z_load + Z_line)

OPEN END (Z_load infinite): coefficient = +1, the wave
reflects entirely and ADDS - overshoot, then ringing as
it bounces.

  ideal            actual
   ___              _  _
  |                | \/ \____
  |                |
__|              __|

Overshoot above VDD + 0.3 V exceeds absolute maximum
(chapter 18) and stresses the input protection diodes.
Ringing that crosses the input threshold several times
gives you MULTIPLE CLOCK EDGES from one transition -
which a counter or a shift register will faithfully
count.

THE FIRMWARE-VISIBLE SYMPTOM: an SPI device that
occasionally reads one bit too many, or a counter that
increments by two. Before assuming your code is wrong,
put a scope on the clock.

SERIES TERMINATION is the usual fix and the one you will
see on schematics: a resistor at the SOURCE, sized so
R_series + R_driver = Z_line (typically 50 ohms, so a
22-33 ohm resistor).

  driver --[33R]---------------------- receiver
           ^
      absorbs the reflection on its return

If a bus is marginal and there are footprints for series
resistors that were left at 0 ohms, fitting the right
value is often the whole fix.
```

Ground is the part nobody thinks about

```
CURRENT FLOWS IN LOOPS. The return path is as real as
the outgoing signal, and it takes the path of LEAST
INDUCTANCE - which at speed is directly under the trace,
not the shortest path on the plane.

GOOD:                    BAD:
+----------+             +----------+
|  signal  |             |  signal  |
+==========+             +==========+
|  ground  |             |   SPLIT  |  <- the return
|  plane   |             |  ______  |     current must
+----------+             +----------+     detour around

A split in the ground plane under a fast signal forces
the return current to detour, creating a large loop that
radiates and picks up noise. This is the single most
common cause of a board that fails EMC testing.

FOR FIRMWARE ENGINEERS, THE PRACTICAL VERSIONS:

1. CONNECT THE GROUNDS. Two boards linked by a UART with
   no shared ground reference do not have a defined
   signal level. This is the number one cause of
   "serial works on the desk, not in the rack".

2. SCOPE PROBE GROUNDING. The long ground lead on a
   probe is an inductive loop that ADDS ringing to what
   you measure. If you see 1 GHz ringing on a 1 MHz
   signal, suspect the probe before the board - use the
   short ground spring.

3. DECOUPLING IS A GROUND ISSUE. A 100 nF capacitor next
   to every supply pin gives switching current a short
   local loop instead of a trip across the board. When a
   chip misbehaves under load and the schematic shows
   one shared capacitor, that is a real finding.

4. CURRENT RETURNS THROUGH THE GROUND. A motor's ground
   current flowing under your I2C traces couples into
   them. "I2C fails when the motor runs" (chapter 07) is
   usually a ground-routing problem, not a filtering
   one.
```

WHAT YOU CAN ACTUALLY CHANGE FROM FIRMWARE

Four levers, and they are worth trying before escalating: **slew rate** — set every pin to the slowest that meets timing; **clock rate** — halve it and see whether the errors vanish, which localises the problem to timing or integrity immediately; **digital filters** — enable the peripheral's noise filter where it has one (I2C's DNF, CAN's sample point); and **error counters** — instrument every bus so "unreliable" becomes a number you can correlate with events like a motor starting.

Crosstalk and what it looks like in software

```
Two parallel traces couple capacitively and inductively.
A fast edge on one induces a pulse on its neighbour.

aggressor  ____|-----
               |
victim     ____/\_____   a glitch at the aggressor's edge

COUPLING FALLS OFF FAST WITH SEPARATION. The usual rule
is 3x the trace width between traces, and more for a
clock. Coupling also grows with the parallel run LENGTH -
two traces side by side for 10 cm couple far more than
for 1 cm.

RIBBON CABLES ARE THE WORST CASE: every conductor
parallel to every other, for the whole length, with no
ground plane. A ribbon carrying a clock next to an I2C
line is a reliable source of intermittent faults. The
standard fix is alternating signal and ground conductors
in the ribbon, which halves your conductor count and is
worth it.

SOFTWARE-VISIBLE SYMPTOMS OF CROSSTALK:

  - a bus that only errors when ANOTHER bus is active.
    Correlate: stop the other traffic and see if the
    errors stop. This single experiment identifies
    crosstalk faster than any measurement.

  - an interrupt that fires with no cause. An EXTI pin
    next to a switching signal will trigger on the
    neighbour's edge. If a GPIO interrupt count climbs
    when a motor or display is active, it is coupling.

  - a reset line glitching. Reset inputs are usually
    high impedance and unfiltered; a coupled pulse
    reboots the board with no crash log. If a board
    resets with no fault recorded and the reset-cause
    register says "pin reset", suspect the reset line
    rather than your firmware.
```

How it shows up

Symptom	Cause
Bus errors vanish when you halve the clock	Timing or integrity, not logic — stop debugging the code
Shift register reads one bit too many	Ringing on the clock crossing the threshold twice
Serial works on the desk, fails in the rack	No shared ground reference between the two boards
Bus A errors only when bus B is active	Crosstalk — correlate by stopping B
GPIO interrupt fires with no external event	Coupling from a neighbouring switching line
Board resets, no fault log, cause is "pin reset"	Glitch on the reset line, not a firmware crash
1 GHz ringing seen on a 1 MHz signal	Scope probe ground lead — use the spring
Chip misbehaves only under load	Inadequate decoupling near its supply pins
Failed radiated emissions testing	Slew rates too fast, or a split ground plane under a fast trace

Symptom	Cause
Intermittent faults with a ribbon cable	Crosstalk — interleave grounds or shorten the run

Check yourself

- ✓ Explain why edge rate rather than clock rate determines signal integrity.
- ✓ Compute the knee frequency for a 1 ns rise time.
- ✓ Give the critical trace length for a 5 ns edge and say what it means.
- ✓ Explain what reflects at an open-ended trace and the software-visible symptom.
- ✓ Describe series termination and how the resistor value is chosen.
- ✓ Explain where return current flows at speed and why plane splits matter.
- ✓ Give the most common cause of serial failing between two chassis.
- ✓ Describe the experiment that identifies crosstalk in one step.
- ✓ Explain why a scope can show ringing that is not really there.
- ✓ List the four things a firmware engineer can change to improve integrity.

Debugging a bus with a scope and a logic analyser

Every bus in this track eventually fails in a way that source code cannot explain. The difference between an hour and a week is knowing which instrument answers which question, and having a procedure rather than a hunch.

Which instrument

Instrument	Answers	Cannot answer
Logic analyser (~$20 clone + sigrok/PulseView)	What bytes were sent, protocol decode, timing between events, long captures	Anything about signal *shape* — it only sees above or below a threshold
Oscilloscope	Edge shape, ringing, overshoot, noise, actual voltage levels	Long captures; decoding many bytes is painful
Protocol analyser (USB, CAN)	Full protocol semantics, errors, timing, at speed	Costs real money; unnecessary for simple buses
GPIO probe pin	What your software was doing when the bus event happened	Anything about the bus itself

```
THE DECISION IS ONE QUESTION:

  "Is the DATA wrong, or is the SIGNAL wrong?"

DATA WRONG  (wrong bytes, wrong sequence, missing ACK,
             wrong address, protocol violation)
  -> LOGIC ANALYSER. It decodes, it captures for
     seconds, and a $20 one is entirely sufficient.

SIGNAL WRONG (works at low speed, intermittent, one
              board in ten, errors when a motor runs)
  -> OSCILLOSCOPE. You need to see the shape.

BUY THE LOGIC ANALYSER FIRST. An 8-channel USB clone
with PulseView decodes UART, SPI, I2C, CAN, 1-Wire and
dozens more, captures at 24 MHz, and costs less than an
hour of your time. There is no reason to debug a serial
bus without one.
```

The procedure

```
/* Work in this order. Each step rules out a layer, and
   skipping ahead is how days are lost.                    */

/* 1. IS ANYTHING HAPPENING AT ALL?
      Probe the clock. No clock means the problem is
      configuration - peripheral clock, pin alternate
      function, or the peripheral never enabled (chapter
      01). Zero point debugging protocol logic when nothing
      is being transmitted.                                */

/* 2. IS THE IDLE STATE CORRECT?
      UART   idle HIGH
      I2C    both lines HIGH (if not: no pull-ups, or a
             stuck bus - chapter 07)
      SPI    CS high, SCLK at its CPOL level
      CAN    ~2.5 V on both lines, 0 V differential

      A wrong idle state is a wiring or configuration fault
      and nothing above it can work.                       */

/* 3. IS THE TIMING RIGHT?
      Measure one bit. Not the average - one bit, with
      cursors.
        UART 115200 -> 8.68 us
        SPI 1 MHz   -> 1.00 us
        I2C 400 kHz -> 2.50 us
      A 10% error here is the whole bug (chapter 02).      */

/* 4. ARE THE LEVELS RIGHT?
      VOH should be near VDD, VOL near 0. If a "high" is
      2.0 V on a 3.3 V system, you have a pull-up fighting
      a driver, a level-shifter problem, or a short.       */

/* 5. DECODE IT.
      Now turn on protocol decoding and read the bytes.
      Compare against what your code intended to send. This
      is where you discover the address was shifted, the
      register number was wrong, or the byte order was
      reversed.                                            */

/* 6. CORRELATE WITH SOFTWARE.
      Add a GPIO probe pin toggled in your driver, captured
      on a spare analyser channel. Now you can see "the
      driver entered the ISR here, and the bus glitched
      there" on one timeline. This is the technique that
      solves the genuinely hard ones.                      */
```

Triggering is the whole skill

```
A fault that happens once an hour cannot be found by
watching. You must arm the instrument to catch it.

TRIGGER ON THE ERROR, NOT THE TRAFFIC:

I2C     trigger on a NACK. PulseView and most scopes
        support this directly, and it catches exactly
        the transaction that failed.

UART    trigger on a specific byte value, or on a
        framing error if the scope supports it.

SPI     trigger on a CS edge, then look at what came
        after.

CAN     trigger on an error frame.

THE UNIVERSAL TECHNIQUE - SOFTWARE-TRIGGERED CAPTURE:

  When your driver detects the error, toggle a GPIO.
  Trigger the analyser on that pin. You now capture
  precisely the failing event, with your own code
  deciding what counts as a failure.

    if (i2c_status & NACKF) {
        TRIGGER_PIN_HIGH();     /* arm the capture */
        TRIGGER_PIN_LOW();
        stats.nack++;
    }

  Set the analyser to capture with 90% PRE-TRIGGER, so
  you see everything that led up to the failure rather
  than what followed it. The cause is always before the
  symptom.

This one technique - your software telling the
instrument when to look - turns an intermittent fault
into a reproducible capture, and it works for every bus
in this track.
```

> **CAPTURE THE WORKING CASE FIRST**
>
> Before capturing a failure, capture a success and save it. Half of bus debugging is noticing a difference, and you cannot see a difference without a reference. It also validates your instrument setup — if you cannot capture and decode a transaction that definitely works, then a failed capture tells you nothing about the bus, only about your probe placement or threshold setting.

Recognisable pictures

```
WHAT YOU SEE                    WHAT IT MEANS

Clean edges, wrong bytes        Software: wrong register,
                                wrong address, wrong
                                byte order

Slow rounded rising edges,      Pull-up too weak, or
sharp falling edges             too much capacitance
(open-drain buses)              (chapter 07)

Ringing after each edge         Reflection; slew rate too
                                fast or missing series
                                termination (chapter 19)

Signal never reaches VDD        Pull-up fighting a driver,
                                or a level-shifter fault

Bits present but one extra      Ringing crossing the
clock counted                   threshold twice

Correct data, no ACK            Device not there,
                                unpowered, or the address
                                is shifted (chapter 06)

Everything correct, device      Timing violation: setup,
ignores it                      hold or valid delay
                                (chapter 18)

Bus idle-low when nothing       Missing pull-ups (I2C) or
is transmitting                 fail-safe bias (RS-485,
                                chapter 14)

Last byte truncated             TXE used where TC was
                                needed (chapters 01, 14)

Traffic stops after N frames    Descriptor or buffer
                                ownership not returned
                                (chapter 17)
```

How it shows up

Symptom	Cause
Analyser decodes nothing	Threshold wrong for the logic level, or ground not connected
Capture is all noise	Ground lead not attached to the board's ground
Fault never appears while watching	Trigger on the error, not the traffic
Captured the failure but not the cause	Use pre-trigger capture — the cause precedes the symptom
Ringing appears only with the probe attached	Probe ground lead inductance (chapter 19)
Signal changes when probed	Probe capacitance loading a marginal bus — you were already at the limit
Decoded bytes differ from what the code sent	Byte order, address shift, or a masking error
Everything looks correct but the device ignores it	A timing parameter, not a data problem
Cannot tell whether it is hardware or software	Add a GPIO probe and correlate on one timeline
Intermittent, cannot reproduce	Software-triggered capture on your own error detection

Check yourself

- ✓ State the one question that decides between a scope and a logic analyser.
- ✓ Give the six-step debugging procedure in order.
- ✓ Say what a missing clock signal tells you immediately.
- ✓ Give the correct idle state for each of UART, SPI, I2C and CAN.
- ✓ Give the bit time for 115200 baud and for 400 kHz I2C.
- ✓ Explain the software-triggered capture technique.
- ✓ Explain why pre-trigger capture matters.
- ✓ Say why you should capture a working transaction first.
- ✓ Interpret slow rising edges with sharp falling edges on an open-drain bus.
- ✓ Explain what it means when a signal changes as soon as you probe it.

Project: an I2C sensor driver, from datasheet to data

A complete production-quality driver for a real sensor: bus recovery, error counting, compensation arithmetic straight from the datasheet, and a state machine that never blocks. This is what the whole track has been for.

The target

```
SENSOR: Bosch BME280 — temperature, pressure, humidity.
Chosen because it is cheap, ubiquitous, well documented,
and its compensation arithmetic is genuinely non-trivial
in an instructive way.

ADDRESS: 0x76 or 0x77, selected by the SDO pin.
INTERFACE: I2C (or SPI — the register map is identical,
           which is common and worth noticing).

WHAT THE DRIVER DOES:
  - recovers a stuck bus at startup (chapter 07)
  - probes and verifies the chip ID
  - reads factory calibration
  - configures oversampling and filtering
  - reads all three measurements in ONE burst
  - applies the datasheet's compensation formulas
  - counts every error class
  - never blocks longer than one transaction timeout

WITHOUT HARDWARE: the compensation arithmetic can be
tested on a host with the datasheet's own worked example,
which Bosch provides precisely so you can validate an
implementation. That test is in the exercises and it is
worth doing first — it is the part most likely to be
subtly wrong.
```

Part 1: the register map

```
/* bme280.h - straight from section 5.3 of the datasheet. */

#define BME280_ADDR_LOW    0x76
#define BME280_ADDR_HIGH   0x77
#define BME280_CHIP_ID     0x60      /* the value, not a reg */

#define REG_ID             0xD0
#define REG_RESET          0xE0
#define REG_CALIB00        0x88      /* 26 bytes            */
#define REG_CALIB26        0xE1      /* 7 more bytes        */
#define REG_CTRL_HUM       0xF2
#define REG_STATUS         0xF3
#define REG_CTRL_MEAS      0xF4
#define REG_CONFIG         0xF5
#define REG_PRESS_MSB      0xF7      /* start of the 8-byte
                                        burst              */

/* Calibration, as the datasheet names them. Signedness is
   NOT uniform and getting one wrong gives readings that
   look plausible and drift with temperature - the worst
   kind of bug. Copy this table carefully.                 */
typedef struct {
    uint16_t T1;  int16_t T2, T3;
    uint16_t P1;  int16_t P2, P3, P4, P5, P6, P7, P8, P9;
    uint8_t  H1;  int16_t H2;  uint8_t H3;
    int16_t  H4, H5;  int8_t H6;
    int32_t  t_fine;          /* carries temperature into
                                 the pressure and humidity
                                 compensation - a shared
                                 intermediate, and the
                                 reason temperature must be
                                 compensated FIRST         */
} bme280_calib_t;

typedef struct {
    uint8_t          addr;
    bme280_calib_t   cal;
    struct {
        uint32_t nack, timeout, recover, crc_range, ok;
    } stats;
} bme280_t;
```

Part 2: probe and initialise

```
int bme280_init(bme280_t *d)
{
    /* STEP 0: recover the bus BEFORE the first transaction.
       If the MCU reset mid-read last time, SDA is stuck low
       and nothing will work (chapter 07). This costs 100 us
       and eliminates "only works after a power cycle".    */
    i2c_bus_recover();

    /* STEP 1: find it. Try both addresses rather than
       assuming - the SDO pin may be strapped either way,
       and this makes one driver work on two boards.       */
    for (uint8_t a = BME280_ADDR_LOW; a <= BME280_ADDR_HIGH; a++) {
        uint8_t id;
        if (i2c_read_regs(a, REG_ID, &id, 1) == 0 &&
            id == BME280_CHIP_ID) {
            d->addr = a;
            goto found;
        }
    }
    return -ENODEV;        /* stop here. Nothing above this
                              layer can work.              */
found:

    /* STEP 2: soft reset, then WAIT. The datasheet gives
       2 ms; poll the status register rather than trusting
       a fixed delay.                                       */
    i2c_write_reg(d->addr, REG_RESET, 0xB6);
    uint32_t deadline = g_ms + 10;
    do {
        uint8_t st;
        if (i2c_read_regs(d->addr, REG_STATUS, &st, 1) == 0 &&
            !(st & 0x01)) { break; }          /* not updating */
        if ((int32_t)(g_ms - deadline) >= 0) { return -ETIMEDOUT; }
    } while (1);

    /* STEP 3: calibration. Two non-contiguous blocks, and
       the H4/H5 fields are packed across nibble boundaries -
       a detail that is easy to get wrong and produces
       humidity readings that are wildly off.               */
    uint8_t c1[26], c2[7];
    if (i2c_read_regs(d->addr, REG_CALIB00, c1, 26) < 0) { return -EIO; }
    if (i2c_read_regs(d->addr, REG_CALIB26, c2,  7) < 0) { return -EIO; }

    d->cal.T1 = (uint16_t)(c1[0]  | (c1[1]  << 8));
    d->cal.T2 = (int16_t) (c1[2]  | (c1[3]  << 8));
    d->cal.T3 = (int16_t) (c1[4]  | (c1[5]  << 8));
    d->cal.P1 = (uint16_t)(c1[6]  | (c1[7]  << 8));
    d->cal.P2 = (int16_t) (c1[8]  | (c1[9]  << 8));
    /* ... P3-P9 likewise ... */
    d->cal.H1 = c1[25];
    d->cal.H2 = (int16_t)(c2[0] | (c2[1] << 8));
    d->cal.H3 = c2[2];
    /* THE PACKED PAIR: H4 is 12 bits split 8+4, H5 is 12
       bits split 4+8, sharing byte c2[4].                  */
    d->cal.H4 = (int16_t)((c2[3] << 4) | (c2[4] & 0x0F));
    d->cal.H5 = (int16_t)((c2[5] << 4) | (c2[4] >> 4));
    d->cal.H6 = (int8_t)  c2[6];

    /* STEP 4: configure. CTRL_HUM MUST be written BEFORE
       CTRL_MEAS - the datasheet says so, and writing them
```

```
       in the other order silently leaves humidity
       oversampling at its previous value. A footnote that
       costs an afternoon (chapter 18).                    */
    i2c_write_reg(d->addr, REG_CTRL_HUM,  0x01);  /* x1 hum   */
    i2c_write_reg(d->addr, REG_CONFIG,    0xA0);  /* 1s, IIR4 */
    i2c_write_reg(d->addr, REG_CTRL_MEAS, 0x27);  /* x1,x1,
                                                     normal   */
    return 0;
}
```

Part 3: one burst, three measurements

```
/* All eight data registers in ONE transaction. This is not
   an optimisation - it is a CORRECTNESS requirement. The
   sensor shadows its output registers during a burst, so
   reading them separately can return values from two
   different measurements: a temperature from one sample
   and a pressure from the next.

   The datasheet says so, and it is a general property of
   sensors with multi-register values. Always read related
   registers in one burst.                                  */

int bme280_read(bme280_t *d, int32_t *temp_c100,
                uint32_t *press_pa, uint32_t *hum_1024)
{
    uint8_t b[8];
    int rc = i2c_read_regs(d->addr, REG_PRESS_MSB, b, 8);
    if (rc == -ENXIO)     { d->stats.nack++;    return rc; }
    if (rc == -ETIMEDOUT) { d->stats.timeout++;
                            i2c_bus_recover();
                            d->stats.recover++;  return rc; }
    if (rc < 0)           { return rc; }

    /* 20-bit values, left-aligned in 3 bytes. */
    int32_t adc_p = (int32_t)(((uint32_t)b[0] << 12) |
                              ((uint32_t)b[1] << 4)  | (b[2] >> 4));
    int32_t adc_t = (int32_t)(((uint32_t)b[3] << 12) |
                              ((uint32_t)b[4] << 4)  | (b[5] >> 4));
    int32_t adc_h = (int32_t)(((uint32_t)b[6] << 8)  |  b[7]);

    /* 0x80000 and 0x8000 are the "measurement skipped"
       sentinels. Treating them as data gives you a
       plausible-looking wrong answer.                      */
    if (adc_t == 0x80000 || adc_p == 0x80000 || adc_h == 0x8000) {
        d->stats.crc_range++;
        return -EAGAIN;
    }

    *temp_c100 = bme280_compensate_t(&d->cal, adc_t);  /* FIRST */
    *press_pa  = bme280_compensate_p(&d->cal, adc_p);
    *hum_1024  = bme280_compensate_h(&d->cal, adc_h);

    /* Range check against the datasheet's specified limits.
       A sensor that has come loose reads a value that is
       numerically valid and physically impossible.         */
    if (*temp_c100 < -4000 || *temp_c100 > 8500) {
        d->stats.crc_range++;
        return -ERANGE;
    }

    d->stats.ok++;
    return 0;
}

/* INTEGER COMPENSATION, from datasheet section 4.2.3.
   Fixed-point throughout: no float, so it runs on an M0.  */

int32_t bme280_compensate_t(bme280_calib_t *c, int32_t adc_T)
{
    int32_t v1 = ((((adc_T >> 3) - ((int32_t)c->T1 << 1))) *
                  ((int32_t)c->T2)) >> 11;
```

```
    int32_t v2 = (((((adc_T >> 4) - ((int32_t)c->T1)) *
                    ((adc_T >> 4) - ((int32_t)c->T1))) >> 12) *
                  ((int32_t)c->T3)) >> 14;
    c->t_fine = v1 + v2;              /* SHARED - see below  */
    return (c->t_fine * 5 + 128) >> 8;   /* 0.01 C units    */
}

uint32_t bme280_compensate_p(bme280_calib_t *c, int32_t adc_P)
{
    int64_t v1 = ((int64_t)c->t_fine) - 128000;   /* t_fine! */
    int64_t v2 = v1 * v1 * (int64_t)c->P6;
    v2 = v2 + ((v1 * (int64_t)c->P5) << 17);
    v2 = v2 + (((int64_t)c->P4) << 35);
    v1 = ((v1 * v1 * (int64_t)c->P3) >> 8) +
         ((v1 * (int64_t)c->P2) << 12);
    v1 = (((((int64_t)1) << 47) + v1)) * ((int64_t)c->P1) >> 33;

    if (v1 == 0) { return 0; }        /* AVOID DIVIDE BY ZERO -
                                         the datasheet's own
                                         guard, and a real
                                         possibility with
                                         corrupt calibration */

    int64_t p = 1048576 - adc_P;
    p = (((p << 31) - v2) * 3125) / v1;
    v1 = (((int64_t)c->P9) * (p >> 13) * (p >> 13)) >> 25;
    v2 = (((int64_t)c->P8) * p) >> 19;
    p = ((p + v1 + v2) >> 8) + (((int64_t)c->P7) << 4);
    return (uint32_t)(p >> 8);        /* Pa                  */
}

/* NOTE t_fine: temperature compensation computes it, and
   pressure and humidity both consume it. So temperature
   MUST be compensated first, every cycle. Call them out of
   order and pressure is wrong in a way that tracks
   temperature - which looks like a sensor fault rather
   than a software one.

   NOTE the 64-bit arithmetic. On a Cortex-M0 without a
   hardware divider this is genuinely expensive - hundreds
   of cycles. Bosch also publishes a 32-bit version with
   slightly reduced accuracy for exactly this reason.      */
```

Part 4: never block

```
/* The blocking version above is fine in an RTOS task. In a
   superloop (mcu chapter 17) it must become a state
   machine.                                                */

typedef enum { S_IDLE, S_TRIGGER, S_WAIT, S_READ } st_t;

void bme280_step(bme280_t *d)
{
    static st_t st = S_IDLE;
    static uint32_t t0;

    switch (st) {
    case S_IDLE:
        if ((int32_t)(g_ms - t0 - 1000) < 0) { break; }
        st = S_TRIGGER;
        break;

    case S_TRIGGER:
        /* forced mode: one measurement, then sleep */
        i2c_write_reg(d->addr, REG_CTRL_MEAS, 0x25);
        t0 = g_ms;
        st = S_WAIT;
        break;

    case S_WAIT:
        /* Max measurement time from datasheet section 9.1,
           for this oversampling: 9.3 ms. Use the DATASHEET
           number, not a guess.                             */
        if ((int32_t)(g_ms - t0 - 10) >= 0) { st = S_READ; }
        break;

    case S_READ: {
        int32_t t; uint32_t p, h;
        if (bme280_read(d, &t, &p, &h) == 0) {
            publish(t, p, h);
        }
        t0 = g_ms;
        st = S_IDLE;
        break;
    }
    }
}

/* FORCED MODE rather than normal mode is the right default
   for a battery device: the sensor measures once and
   returns to sleep at 0.1 uA, instead of free-running.
   Combined with the MCU sleeping between measurements
   (mcu chapter 13), the whole system draws microamps
   between samples.                                        */
```

Exercises

- **Validate the compensation on a host first.** Bosch's datasheet gives a worked example with known calibration constants and raw values. Compile the compensation functions for your PC and check them against it before touching hardware — this separates arithmetic bugs from bus bugs completely.
- **Break the bus deliberately.** Reset the MCU mid-read with a debugger. Confirm the bus is stuck, then confirm your recovery frees it.

- **Read the registers separately** instead of in one burst, then heat the sensor while changing pressure, and look for the inconsistent pairs.
- **Capture a transaction** with a logic analyser (chapter 20) and hand-decode the address, register and data against the datasheet.
- **Remove the pull-ups** and observe the edges on a scope; then fit 10 kΩ and try 400 kHz (chapter 07).
- **Port it to SPI.** The register map is identical; only the transport changes. Structure the driver so this is a 30-line change — that separation is the point.
- **Add a second sensor** at the other address and confirm both work on one bus.

How it shows up

Symptom	Cause
Chip ID read fails	Wrong address, unpowered, or a stuck bus — run recovery first
Temperature plausible, pressure wildly wrong	Compensation called out of order; `t_fine` is stale
Humidity wildly wrong	H4/H5 nibble packing decoded incorrectly
Readings drift with temperature in a way the sensor should correct	A calibration constant's signedness is wrong
Humidity oversampling has no effect	`CTRL_HUM` written after `CTRL_MEAS`
Occasional mismatched temperature and pressure	Registers read separately rather than in one burst
Reads 0x80000 sometimes	Measurement skipped or not ready — treat as a sentinel, not data
Divide-by-zero fault	Corrupt calibration data; keep the datasheet's guard
Works after power cycle only	No bus recovery at startup
Very slow on a Cortex-M0	64-bit compensation without a hardware divider — use the 32-bit variant

Check yourself

- ✓ Explain why bus recovery runs before the first transaction, not after the first failure.
- ✓ Say why the driver probes both possible addresses.
- ✓ Explain why related registers must be read in one burst.
- ✓ Explain what `t_fine` is and the ordering constraint it imposes.
- ✓ Describe the H4/H5 packing and why it is easy to get wrong.
- ✓ Say why `CTRL_HUM` must be written before `CTRL_MEAS`.
- ✓ Explain the role of the 0x80000 sentinel value.
- ✓ Justify keeping the divide-by-zero guard from the datasheet.
- ✓ Explain why forced mode suits a battery device.
- ✓ Say how you would validate the compensation arithmetic without any hardware.

TRACK

Operating systems

The OS concepts that are true of Linux, Windows and every RTOS alike — the vocabulary both driver tracks assume.

20 chapters

What an operating system is

An OS has exactly two jobs, and every feature it has is an instance of one of them. Getting this framing right makes the rest of the track — scheduling, memory, files, drivers — read as variations on a theme rather than a pile of unrelated machinery.

The two jobs

1. ABSTRACTION
Turn awkward, varied hardware into a uniform interface. A program calls `read()` and does not care whether the bytes come from an NVMe SSD, a network socket, a USB stick or a pipe. Without this, every application would need a driver for every device.

2. Arbitration
Share finite resources among competing users, safely and fairly. CPU time, memory, devices, files — all must be divided so that one program cannot starve, corrupt or spy on another.

```
Every OS subsystem is one of those two, applied to one resource:

  resource          abstraction              arbitration
  ──────────────────────────────────────────────────────────
  CPU               process / thread         scheduler
  memory            virtual address space    allocator + MMU
  storage           file, directory          permissions, quotas
  network           socket                   ports, firewall, qdisc
  devices           /dev node, handle        drivers, permissions
  time              timers, clocks           timer wheel

If you can place a feature in that table, you understand what it
is for. If you cannot, it is probably policy, not mechanism.
```

Mechanism versus policy

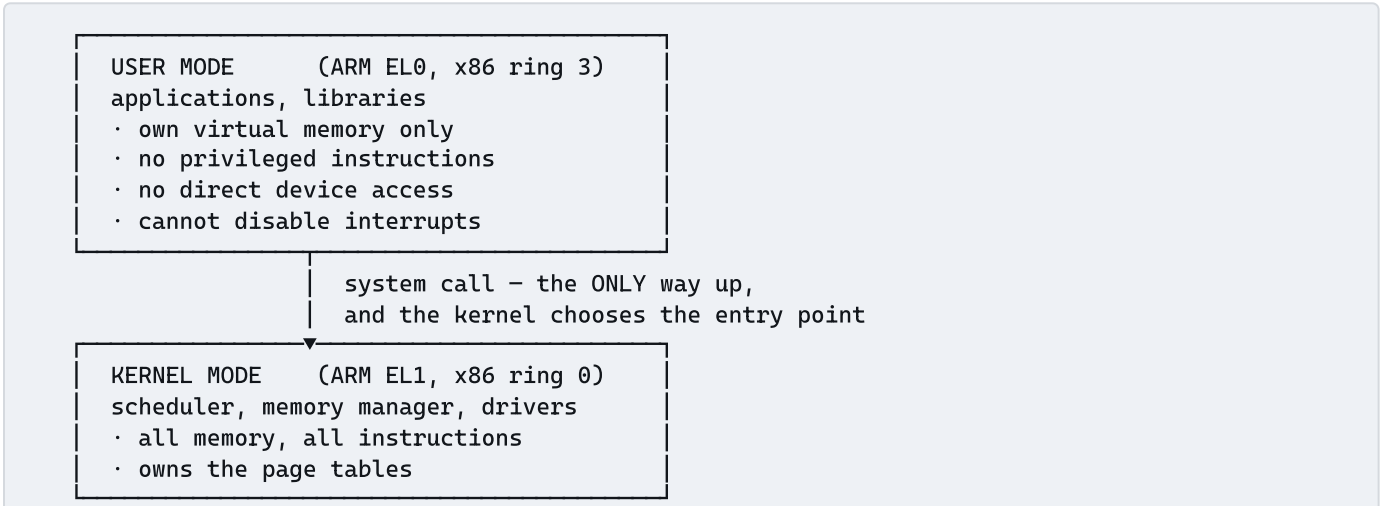
A design principle that runs through every good OS, and the reason Linux and Windows differ so much in places while doing identical things underneath.

	Mechanism	Policy
Question	*How* is it done?	*What* should be done?
Example	Context switching: save registers, swap page tables	Which thread runs next
Example	Page tables and fault handling	Which page to evict
Example	The block layer's request queue	The I/O scheduler that orders it
Changes	Rarely; it is close to hardware	Often; it is tuning and workload-dependent

Separating them is why Linux can ship several schedulers and several I/O elevators over the same core machinery, and why you can change one with a `sysctl` rather than a rebuild.

The privilege boundary

Arbitration requires enforcement, and enforcement requires that applications *cannot* bypass the OS. That needs hardware support — the exception levels of CA chapter 16 and ARM chapter 26.



> **WHY "THE OS PROTECTS YOU" IS IMPRECISE**
>
> The OS *configures* protection; the **hardware** enforces it. A kernel bug can disable it, and a system without an MMU (a Cortex-M) has no enforcement at all — which is why a wild pointer on a microcontroller can silently corrupt anything, and why RTOS "protection" is advisory unless an MPU is configured.

What an OS actually provides

Service	Abstraction offered	Chapter
Process management	Create, run, wait for and terminate programs	04, 05
Scheduling	The illusion of simultaneous execution	06, 07
Memory management	A private, large, contiguous address space	11–13
File systems	Named, persistent, hierarchical byte streams	14, 15
I/O and drivers	Uniform device interfaces	16
IPC	Safe communication between isolated programs	08
Synchronisation	Coordinated access to shared state	09, 10
Time	Clocks, timers, sleeping	17
Security	Identity, permissions, isolation	20
Networking	Sockets over protocol stacks	—

Kinds of operating system

Type	Optimises for	Examples	Typical footprint
General purpose	Throughput, fairness, broad hardware support	Linux, Windows, macOS, BSD	MBs–GBs
Real-time (RTOS)	*Predictability* — bounded worst-case latency	FreeRTOS, Zephyr, ThreadX, QNX, VxWorks	KBs–MBs
Embedded Linux	General purpose on constrained hardware	Yocto, Buildroot, Android	MBs
Hypervisor	Isolating whole OSes	KVM, Xen, Hyper-V, Hafnium	Varies
Unikernel	One application, no separation	MirageOS, Unikraft	KBs–MBs
Bare metal	No OS at all — a superloop	Cortex-M firmware	KBs

> **REAL-TIME DOES NOT MEAN FAST**
>
> It means **predictable**. An RTOS that guarantees a 50 µs worst-case interrupt latency is "real-time"; Linux with a 5 µs *average* and an occasional 5 ms outlier is not. For a general-purpose OS the metric is throughput and the average case; for an RTOS it is the *worst* case, and average performance is almost irrelevant. Design decisions follow directly: no demand paging, no unbounded allocators, priority inheritance on every lock, bounded interrupt disable times.

How the OS gets control

```
   power on
      │
   firmware / boot ROM        immutable, verifies the next stage
      │
   bootloader                 U-Boot, GRUB, UEFI
      │  loads the kernel image and a hardware description
      │  (device tree or ACPI tables)
      ▼
   kernel entry               still running on firmware's assumptions
      │  set up its own page tables, turn on the MMU,
      │  discover CPUs and memory, initialise subsystems
      ▼
   first user process         init / systemd / launchd
      │  everything after this point is scheduled normally
      ▼
   the system is "up"

Detail in os-18. The important structural point: the kernel is not
"always running". It is a library of handlers that runs when a
system call, an interrupt or a fault transfers control to it — and
otherwise the CPU is executing user code, or idle.
```

The kernel is event-driven

```
Three, and only three, ways the kernel gains control:

  1. SYSTEM CALL       user code deliberately asks          (ch. 03)
                       synchronous, on the caller's behalf

  2. INTERRUPT         a device signals asynchronously      (ch. 17)
                       unrelated to what was running

  3. EXCEPTION/FAULT   the current instruction failed       (CA ch. 16)
                       page fault, illegal instruction, alignment

  Everything the kernel does is triggered by one of these. There is
  no "kernel main loop" — except the idle task, whose entire job is
  to execute a low-power wait instruction until an interrupt arrives.

  This is why kernel code is written as short, non-blocking handlers,
  and why "what context am I in?" is the first question you ask about
  any kernel function (chapters 09 and 16).
```

Linux and Windows, same problems

Concept	Linux	Windows
Schedulable unit	`task_struct`	`KTHREAD`
Address space	`mm_struct`	Process + VAD tree
Open file	File descriptor (an `int`)	`HANDLE`
Kernel entry	`svc` / `syscall`	`svc` / `syscall`
Loadable driver	`.ko` module	`.sys` driver
Deferred work	softirq, workqueue	DPC, work item
Context constraint	"atomic context"	IRQL
Fatal error	panic / oops	Bug check (BSOD)
Uniform I/O object	Everything is a file	Everything is an object with a handle

The vocabulary differs; the problems are identical. This track teaches the concepts once so that both driver tracks are a matter of learning names rather than ideas.

What the abstraction costs

Cost	Magnitude	When it matters
System call	~50–500 ns	High-frequency I/O — hence `io_uring`, batching, vDSO
Context switch	~1–5 µs, plus cache and TLB refill	Many threads, or fine-grained switching
Page fault (minor)	~1–5 µs	First touch of each page
Page fault (major)	Milliseconds	Any real-time deadline
Scheduling jitter	Microseconds to milliseconds	Control loops, audio, motion
Memory overhead	Kernel, page tables, per-task structures	Small embedded systems

These costs explain the escape hatches that exist in every OS: `mmap` instead of `read`, `io_uring` instead of syscall-per-operation, huge pages, `mlockall()`, CPU isolation, kernel bypass (DPDK, SPDK), and ultimately choosing an RTOS or bare metal when the OS's guarantees are worth less than its overhead.

How it shows up

Symptom	Underlying OS concept
A tight loop of small reads is slow	System call overhead; batch or use `mmap`
Occasional millisecond latency spike	Major page fault, or preemption by a higher-priority task
Program cannot access a device	Arbitration: permissions on the device node
One process cannot see another's memory	Abstraction working as designed
Kernel bug takes the whole machine down	No privilege boundary *inside* the kernel
An RTOS meets deadlines that Linux misses	Worst case versus average case
Real-time task misses a deadline once an hour	Unlocked pages, or an unbounded kernel path
Firmware corrupts unrelated data	No MMU; nothing enforces isolation

Check yourself

- ✓ State the two jobs of an OS and place four subsystems into that framing.
- ✓ Distinguish mechanism from policy, with two examples.
- ✓ Name the three hardware features that make the privilege boundary enforceable.
- ✓ Explain why "real-time" means predictable, not fast, and give a design consequence.
- ✓ Name the only three ways the kernel gains control.
- ✓ Give the Windows term for four Linux concepts.
- ✓ Give approximate costs for a system call, a context switch and a major page fault.

Kernel architectures: monolithic, micro, hybrid

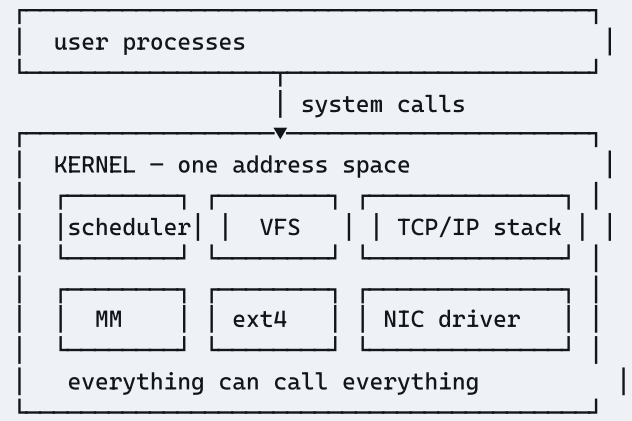
Where you draw the line between "in the kernel" and "outside it" determines the system's performance, its reliability and its attack surface. This is the oldest live argument in operating systems, and the answer genuinely depends on what you are building.

The question

An OS provides file systems, drivers, networking, memory management and scheduling. Which of those run in kernel mode, with full privilege and no isolation from each other — and which run as ordinary isolated processes?

```
The trade-off, stated once:

  IN THE KERNEL              function call: ~1 ns
                             full access to everything
                             a bug is a system crash
                             a vulnerability is total compromise

  OUT OF THE KERNEL          IPC round trip: ~1–10 µs
                             isolated address space
                             a bug kills one service
                             a vulnerability is contained

  Everything below is a different answer to: how much performance
  is that isolation worth?
```

Monolithic



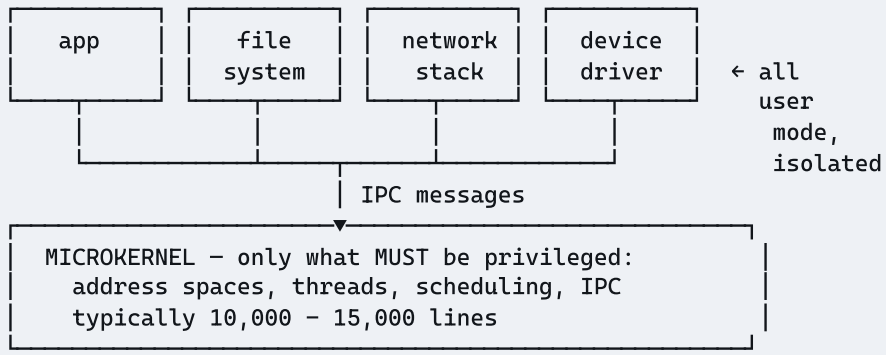
Linux, BSD, older UNIX. Loadable modules do NOT change this:
a .ko is linked into the same address space at runtime.

Advantage	Disadvantage
Subsystem calls are direct function calls — no marshalling, no context switch	Any driver bug can corrupt any kernel data
Shared data structures are cheap to access	A driver crash panics the machine
Simple to write and to reason about performance	A driver vulnerability is full system compromise
Mature, well-optimised implementations exist	Millions of lines at maximum privilege

Advantage	Disadvantage
Modules give runtime flexibility without IPC cost	No stable internal API; everything must be rebuilt together

> **WHY LINUX IS MONOLITHIC AND STAYS THAT WAY**
>
> Roughly 70% of the Linux tree is drivers, all running at EL1 with no isolation from each other. That is a deliberate trade: driver authors get direct access and full speed, and the project accepts that a bad driver can panic the system. Mitigations added over time — `lockdep`, KASAN, the driver model, and increasingly Rust for new drivers — attack the *bug rate* rather than the architecture.

Microkernel

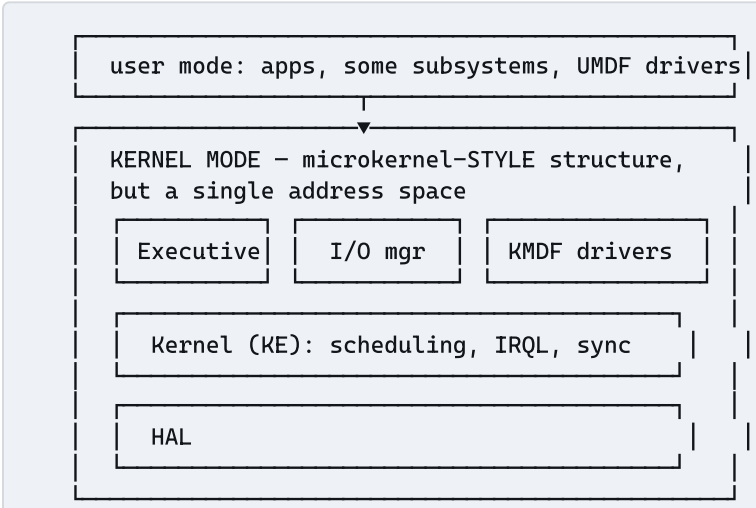


seL4, QNX, MINIX 3, L4 family, Zircon (Fuchsia)

Advantage	Disadvantage
A driver crash kills one process; a supervisor can restart it	Every cross-component call is IPC: two context switches minimum
Trusted computing base is small enough to *formally verify* (seL4 is proved correct)	Data must be copied or shared explicitly
Components can be replaced or upgraded independently	More complex to design: everything is a protocol
Strong fault isolation and least privilege throughout	Historically 10–50% slower on I/O-heavy workloads
Natural fit for certification (DO-178C, IEC 61508)	Small ecosystem; fewer drivers exist

```
Why the performance gap narrowed but did not close

  1980s L3/L4 work showed IPC could be made ~20x faster than
  Mach's, by making it synchronous, register-based and avoiding
  the scheduler. Modern microkernels do a round trip in a few
  hundred nanoseconds, not microseconds.

  What still costs:
    · two address-space switches -> TLB pressure, even with ASIDs
    · cache footprint of crossing between components
    · data copies, unless shared memory is set up (which erodes
      the isolation you paid for)
    · the scheduler is now on the critical path of every I/O

  Where it wins outright: when a fault MUST be contained.
  seL4 in avionics and automotive; QNX in vehicles and medical
  devices. Isolation is the product, and the cycles are the price.
```

Hybrid




```
Windows NT, macOS/XNU (a Mach microkernel with BSD and
drivers linked into the same address space).
```

"Hybrid" mostly means: *organised* like a microkernel — layered, with defined interfaces and object abstractions — but *running* like a monolith, in one address space, for speed. Windows adds a genuine escape hatch with **UMDF**: user-mode drivers for classes of device (USB, HID, sensors) where a crash must not be fatal. That is the microkernel argument applied selectively, where the performance cost is acceptable.

Other shapes

Architecture	Idea	Used for
Exokernel	The kernel only multiplexes hardware securely; applications implement their own abstractions in libraries	Research; influenced kernel-bypass networking (DPDK) and unikernels
Unikernel	One application linked with just the OS functions it needs, in a single address space, no user/kernel split	Cloud appliances, MirageOS, Unikraft — tiny images, fast boot, minimal attack surface
Library OS	OS services as a library the application links; isolation is provided from outside	Graphene, Windows Drawbridge, WASM runtimes

Architecture	Idea	Used for
Nanokernel / separation kernel	Only spatial and temporal partitioning; everything else is a partition	Avionics (ARINC 653), mixed-criticality automotive
No kernel	A superloop plus interrupts	Cortex-M firmware (C ch. 20)

Loadable modules

```
# Linux — the module runs in the SAME address space at full privilege.
# This is convenience, not isolation.
sudo insmod driver.ko          # link into the running kernel
lsmod                          # what is loaded, and who depends on it
sudo rmmod driver              # unlink
modinfo driver.ko              # metadata, parameters, dependencies

# Windows
sc create mydrv type= kernel binPath= C:\drv.sys
sc start mydrv
pnputil /add-driver mydrv.inf /install
```

Benefit	Cost
Smaller base image; load only what the hardware needs	Symbol resolution and relocation at load time
Third-party drivers without rebuilding the kernel	A module can do anything — signing becomes essential
Development without rebooting	No stable in-kernel ABI in Linux: rebuild per version
Unload to reclaim memory	Unload races are a classic source of use-after-free

Windows *does* maintain a stable driver ABI across releases, which is why a driver from years ago often still loads. Linux deliberately does not, to force drivers upstream where they are maintained with the API. Both positions are defensible and both have real costs — this is one of the clearest philosophical differences between the two systems.

Isolating drivers without leaving the kernel

Technique	Protects against	Where
User-mode drivers	Crashes and most vulnerabilities	UMDF (Windows), FUSE, UIO/VFIO, libusb
IOMMU / SMMU	DMA from a buggy or hostile device	Both — see the security track
Memory-safe languages	Whole classes of memory bugs	Rust in Linux 6.1+, Windows drivers
eBPF	Arbitrary code — a verifier proves termination and memory safety before loading	Linux
Driver signing	Unauthorised code	Both; mandatory on Windows x64
Sanitisers and static analysis	Bugs, before shipping	KASAN, Driver Verifier, SDV

eBPF is the most interesting of these: it lets you run user-supplied code *in* the kernel while retaining safety, by proving properties statically instead of isolating at runtime. It is effectively a fourth answer to the original question.

Choosing, in practice

If you need…	Choose	Because
Broad hardware support, good throughput	Monolithic (Linux)	Ecosystem and raw performance
Certification, or provable isolation	Microkernel (seL4, QNX)	Small verifiable TCB; fault containment
Mixed criticality on one SoC	Separation kernel or hypervisor	Spatial and temporal partitioning
Hard real-time, small footprint	RTOS or bare metal	No unbounded paths; predictable
A single cloud service, minimal surface	Unikernel	No unused code, fast boot
Windows compatibility	NT hybrid	It is the platform

THE PATTERN WORTH CARRYING AWAY

Every architecture here is answering "where do I put the isolation boundary?" — and the same question recurs at every level of this site. TrustZone puts it between two worlds; the SMMU puts it in front of DMA; the MMU puts it between processes; a hypervisor puts it between kernels. The mechanism differs; the reasoning about cost versus containment is identical every time.

How it shows up

Symptom	Architectural cause
One buggy driver panics the whole machine	Monolithic: no isolation between kernel components
A driver crash only kills a service	User-mode driver (UMDF, FUSE)
Module will not load after a kernel upgrade	No stable in-kernel ABI in Linux
A Windows driver from 2015 still loads	NT maintains a stable driver ABI
Microkernel system is slower on I/O benchmarks	IPC and context switches on every operation
Filesystem in userspace is slower than in-kernel	Same reason: FUSE crosses the boundary per operation
Kernel-bypass networking is dramatically faster	Exokernel-style: removes the kernel from the data path
Use-after-free on module unload	Unload raced with in-flight work

Check yourself

- ✓ State the single trade-off that all kernel architectures are answering.
- ✓ Give two advantages and two disadvantages of a monolithic kernel.
- ✓ Explain what a microkernel keeps in the kernel and why that list is short.
- ✓ Say why microkernel IPC cost narrowed but never vanished.
- ✓ Explain in what sense Windows NT is "hybrid".
- ✓ Explain why a Linux module is not an isolation boundary.
- ✓ Give three ways to isolate drivers without moving them out of the kernel.
- ✓ Explain why Linux refuses a stable in-kernel ABI and what that buys.

The user/kernel boundary and system calls

The system call is the one door between an application and the kernel. Everything an OS offers passes through it, every argument crossing it is hostile until validated, and its cost shapes the design of every high-performance I/O interface ever built.

Why a special mechanism

A function call cannot cross the privilege boundary. If user code could simply `call` a kernel address, it could call *any* kernel address — landing in the middle of a function, skipping permission checks, with attacker-chosen register state. Isolation would be decorative.

THE THREE REQUIREMENTS

1. **Privilege escalation** must happen atomically with the control transfer.
2. **The entry point is chosen by the kernel**, not the caller. User code supplies a *number*, not an address.
3. **The kernel must be able to return** to exactly where the caller was, at its original privilege.

A synchronous exception provides all three, which is why every architecture implements system calls as a deliberate trap.

The mechanism

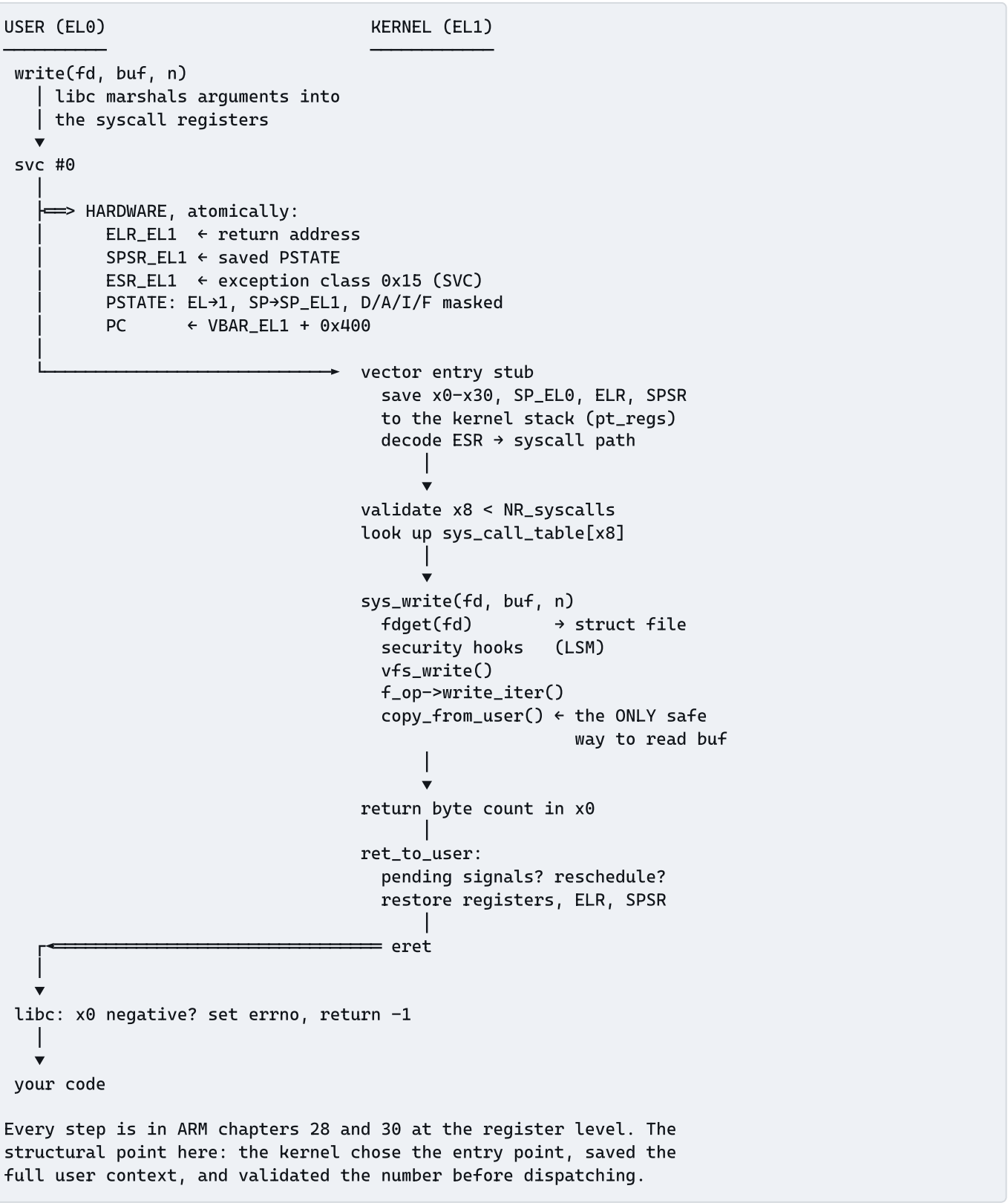
Architecture	Instruction	Number in	Arguments in	Return in
AArch64 (Linux)	`svc #0`	`x8`	`x0` – `x5`	`x0`
x86-64 (Linux)	`syscall`	`rax`	`rdi rsi rdx r10 r8 r9`	`rax`
ARM32 (Linux)	`svc #0`	`r7`	`r0` – `r5`	`r0`
RISC-V	`ecall`	`a7`	`a0` – `a5`	`a0`

THE SYSCALL ABI IS NOT THE C ABI

Note the differences: on x86-64 the fourth argument is in `r10`, not `rcx`, because the `syscall` instruction clobbers `rcx` with the return address. On AArch64 the syscall number sits in `x8`, outside the normal argument registers. Kernel entry code must know both conventions, and this is exactly the mismatch that makes hand-written syscall wrappers error-prone — use libc's.

```
/* what write(1, "hi\n", 3) compiles to on AArch64 Linux */
        mov     x0, #1                  // fd
        adr     x1, msg                 // buf
        mov     x2, #3                  // count
        mov     x8, #64                 // __NR_write
        svc     #0
        // x0 now holds the byte count, or a negative errno
```

The full round trip



Every step is in ARM chapters 28 and 30 at the register level. The structural point here: the kernel chose the entry point, saved the full user context, and validated the number before dispatching.

Validating what crosses

> **EVERY BYTE FROM USER SPACE IS HOSTILE**
> A system call argument is attacker-controlled input arriving at the highest privilege level. The kernel must assume the caller is malicious — not merely buggy — because a local user exploiting a validation gap gets kernel code execution.

```
/* NEVER dereference a user pointer directly. */
long sys_example(void __user *ubuf, size_t len)
{
        char kbuf[256];

        /* WRONG — the pointer may reference kernel memory, unmapped
           memory, or memory that another thread unmaps mid-call. */
        memcpy(kbuf, ubuf, len);

        /* RIGHT */
        if (len > sizeof kbuf)
                return -EINVAL;                 /* bound FIRST      */
        if (copy_from_user(kbuf, ubuf, len))
                return -EFAULT;                 /* returns bytes NOT copied */
        ...
}

/* copy_from_user / copy_to_user do three things you cannot skip:
     1. check the address really lies in user space
     2. install an exception fixup, so a fault returns -EFAULT
        instead of oopsing
     3. respect PAN/SMAP, the hardware feature that blocks kernel
        access to user pages outside these helpers          */
```

The five validation rules

Rule	Failure if ignored
Bound every length before using it	Kernel buffer overflow
Copy before validating, never validate in place	TOCTOU: another thread changes the value between check and use
Check every index and enum against its range	Out-of-bounds table access
Zero output structures fully before filling them	Padding bytes leak kernel stack contents to userspace
Reject unknown flags rather than ignoring them	Cannot add meanings later without breaking programs

```
/* Rule 2, the TOCTOU trap, written out */
struct req *r = (struct req __user *)arg;

if (r->len > MAX) return -EINVAL;      /* read #1 — validate */
process(r->buf, r->len);                /* read #2 — USE      */
/* Another thread in the same process can change r->len between
   the two reads. The check applied to a value that no longer
   exists. This is a real and repeatedly exploited pattern.   */

/* Correct: one copy, then validate the copy */
struct req k;
if (copy_from_user(&k, r, sizeof k)) return -EFAULT;
if (k.len > MAX) return -EINVAL;
process(k.buf, k.len);

/* Rule 4, the info leak */
struct info out;
memset(&out, 0, sizeof out);           /* padding too (C ch. 07) */
out.a = x; out.b = y;
if (copy_to_user(u, &out, sizeof out)) return -EFAULT;
```

The ABI is forever

> **"WE DO NOT BREAK USERSPACE"**
> Linus Torvalds' rule, and the reason Linux binaries from 1995 still run. A system call's number, argument layout and semantics are permanent once released. You cannot change a struct, repurpose a field, or tighten validation in a way that rejects previously-accepted input.

```
/* This is why the syscall table has entries like: */
open, openat, openat2                  /* three generations */
stat, stat64, fstatat, statx
clone, clone3
epoll_create, epoll_create1
dup, dup2, dup3

/* Each new one adds a flags argument or a size field, because the
   old one could not be extended. Designing for extension from the
   start is the lesson:                                          */

struct my_args {
        __u32 size;         /* sizeof(struct) — lets the kernel tell
                               which version the caller compiled against */
        __u32 flags;        /* reject unknown bits, so new ones are safe */
        __u64 field_a;
        __u64 field_b;
        /* new fields appended here in future versions */
};

long sys_myop(struct my_args __user *uargs, size_t usize)
{
        struct my_args a = {};

        if (usize > PAGE_SIZE) return -E2BIG;
        /* copy_struct_from_user handles version skew both ways:
           zero-fills a smaller old struct, and requires that any
           extra bytes from a newer caller are all zero.        */
        if (copy_struct_from_user(&a, sizeof a, uargs, usize))
                return -EFAULT;

        if (a.flags & ~MY_VALID_FLAGS) return -EINVAL;   /* rule 5 */
        ...
}
```

Cost, and the interfaces built to avoid it

```
A minimal syscall costs roughly 50–500 ns:

   trap and privilege change          ~20–50 cycles
   save and restore user registers    ~30–60 cycles
   table lookup and dispatch          ~10 cycles
   pipeline and branch predictor cost variable
   Spectre/Meltdown mitigations       +100–400 cycles on affected CPUs

That is 100–1000x a function call. For a program doing millions of
small operations it dominates everything else.

Four families of workaround, all worth recognising:

  BATCH        do more per call
               writev, readv, sendmmsg, io_submit

  AVOID        remove the kernel from the path entirely
               mmap (page-fault once, then plain loads/stores)
               vDSO (see below)
               DPDK, SPDK — poll the device from user space

  AMORTISE     submit many operations, collect many completions
               io_uring: shared ring buffers, often ZERO syscalls
               in steady state

  MOVE IN      run the logic in the kernel instead
               eBPF: verified user code, executed kernel-side
```

```
/* The vDSO — a small shared object the kernel maps into every
   process, containing implementations that need no privilege.  */
clock_gettime(CLOCK_MONOTONIC, &ts);
/* Reads a kernel-maintained page of timekeeping data plus the
   hardware counter, in USER MODE. No trap at all. Roughly 25 ns
   instead of 300 ns.                                           */

cat /proc/self/maps | grep vdso
LD_SHOW_AUXV=1 /bin/true | grep VDSO
```

Observing system calls

```
# trace every syscall a process makes
strace ./program
strace -c ./program              # summary: counts, time, errors
strace -f -e trace=openat,read,write ./program
strace -p 1234                   # attach to a running process

# lower overhead, system-wide
perf trace ./program
perf stat -e raw_syscalls:sys_enter ./program

# who is making the most syscalls, system-wide
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
        @[comm, args->id] = count(); }'

# Windows
# Process Monitor, or Event Tracing for Windows (ETW)

# the syscall table itself
ausyscall --dump                       # numbers to names
cat /usr/include/asm-generic/unistd.h  # canonical numbering
```

> **`STRACE -C` FIRST**
> When a program is inexplicably slow, run `strace -c` before anything else. A count of two million `read()` calls of 1 byte each, or thousands of failed `stat()` calls probing for a file, diagnoses the problem in seconds — and both are extremely common.

Windows

```
Win32 is NOT the syscall layer. There are two hops:

  application
      |  CreateFile(...)                    documented, stable API
      v
  kernel32.dll / kernelbase.dll
      |  NtCreateFile(...)                  Native API
      v
  ntdll.dll
      |  mov eax, 0x55        <- syscall NUMBER, and it CHANGES
      |  syscall                 between Windows versions
      v
  === kernel ===
  KiSystemService -> SSDT lookup -> NtCreateFile

Consequences:
  · Windows' stable ABI is Win32, not the syscall interface
  · calling syscalls directly is unsupported and breaks on update
    (which is why malware does it, and why EDR products watch for it)
  · Linux is the opposite: the SYSCALL is stable and libc is a
    convenience wrapper
```

How it shows up

Symptom	Cause
Program is slow with low CPU usage	Syscall-bound; check `strace -c`
Reading a file byte-by-byte is 100× slower	One syscall per byte; buffer it
Performance dropped after a kernel security update	Spectre/Meltdown mitigations on syscall entry
Kernel oops on a bad pointer from userspace	Direct dereference instead of `copy_from_user`
Vulnerability from a length passed by the caller	Length used before being bounded
Exploit changes a value after validation	TOCTOU: validated in place instead of after copying
Kernel stack bytes appear in userspace output	Struct padding not zeroed
New kernel flag breaks an old program	Unknown flags were ignored rather than rejected
Windows software breaks after an update	Called syscalls directly instead of via Win32
`clock_gettime` is unexpectedly cheap	Served by the vDSO, no trap

Check yourself

- ✓ Give three reasons a system call cannot be an ordinary function call.
- ✓ Name the AArch64 syscall instruction and which register carries the number.
- ✓ Walk the round trip from `svc #0` to `eret`, naming what hardware saves.
- ✓ Give the five rules for validating user input, and the failure each prevents.
- ✓ Explain the TOCTOU pattern and the fix.
- ✓ Explain why `openat2` exists alongside `open`.
- ✓ Design an extensible syscall argument struct and say how version skew is handled.
- ✓ Name four techniques for reducing syscall cost, with an example of each.
- ✓ Explain why Windows' stable ABI is Win32 rather than the syscall layer.

Processes and the process control block

A process is a program plus everything the OS must remember about it. This chapter covers what that state actually is, how processes are created and destroyed, and the lifecycle bugs — zombies, orphans, fork-in-a-threaded-program — that come from getting it wrong.

What a process is

> **PROGRAM VERSUS PROCESS**
> A **program** is a file on disk: instructions and initial data. A **process** is a program in execution, together with all the state the OS maintains on its behalf. One program can back many processes; each has its own private state.

```
A process is the sum of:

  ADDRESS SPACE          its private virtual memory map
    text                 code, read-only, shared between instances
    data + bss           initialised and zeroed globals
    heap                 grows upward, via brk/mmap
    memory mappings      files, shared libraries, anonymous regions
    stack(s)             one per thread, grows downward

  EXECUTION STATE        registers, PC, stack pointer, flags
                         (per thread — see chapter 05)

  KERNEL RESOURCES       open file descriptors
                         current working directory, root directory
                         signal handlers and masks
                         timers
                         namespaces, cgroup membership

  IDENTITY               PID, PPID, UID/GID, capabilities,
                         security label (SELinux, AppArmor)

  ACCOUNTING             CPU time used, memory usage, priority,
                         start time, resource limits
```

The process control block

All of that lives in one kernel structure: `task_struct` in Linux, `EPROCESS` plus `KPROCESS` in Windows. It is the definitive answer to "what does the kernel know about this program?".

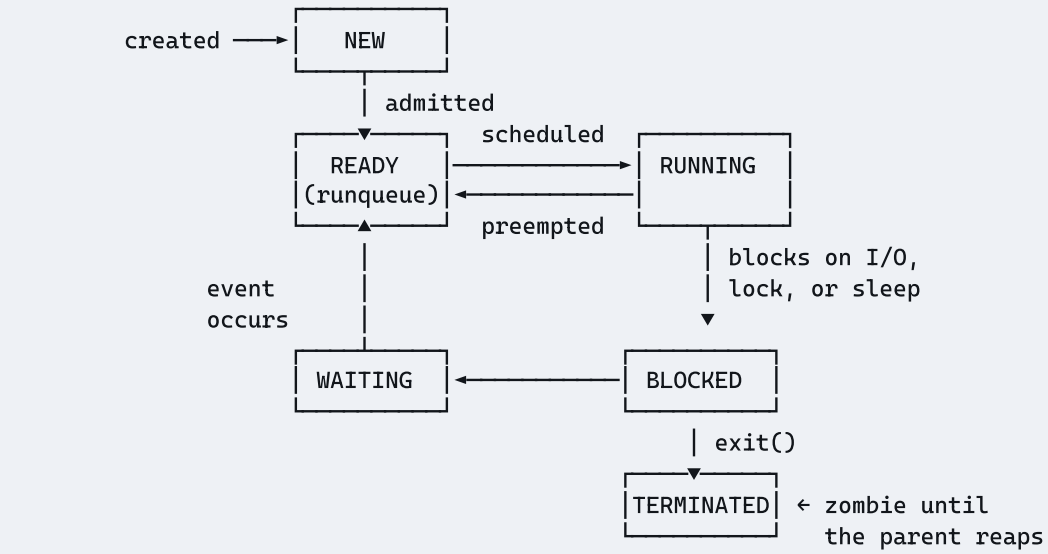
```
/* Linux: include/linux/sched.h — several hundred fields. The ones
   that matter for understanding the model:                    */

struct task_struct {
        /* ---- scheduling ---- */
        unsigned int          __state;        /* RUNNING, INTERRUPTIBLE... */
        int                   prio, static_prio, normal_prio;
        const struct sched_class *sched_class;
        struct sched_entity   se;             /* CFS/EEVDF bookkeeping */
        cpumask_t             cpus_mask;      /* affinity              */

        /* ---- memory ---- */
        struct mm_struct     *mm;             /* NULL for kernel threads */
        struct mm_struct     *active_mm;

        /* ---- identity and relationships ---- */
        pid_t                 pid, tgid;      /* thread id, THREAD GROUP id */
        struct task_struct __rcu *real_parent;
        struct list_head      children, sibling;
        const struct cred    *cred;           /* uid, gid, capabilities */

        /* ---- resources ---- */
        struct files_struct  *files;          /* fd table              */
        struct fs_struct     *fs;             /* cwd, root             */
        struct signal_struct *signal;
        struct sighand_struct *sighand;

        /* ---- kernel-side execution ---- */
        void                 *stack;          /* 8-16 KB kernel stack  */
        struct thread_struct  thread;         /* saved registers       */

        char                  comm[16];       /* the name in `ps`      */
};

/* `current` is a macro giving the running task. On arm64 it reads
   SP_EL0, which the kernel repurposes to hold the task pointer —
   so it is a single register read, not a lookup.               */
```

THE KEY STRUCTURAL INSIGHT

In Linux there is **no separate process structure**. A process is a group of `task_struct`s sharing a `tgid`, an `mm_struct` and a `files_struct`. Threads and processes are the same object with different sharing. This is why `getpid()` returns `tgid` while `gettid()` returns `pid`, and it is the single most important thing to understand about the Linux model (chapter 05).

Windows takes the opposite approach: `EPROCESS` owns the address space and handle table, and `ETHREAD` objects belong to it. Both designs work; the Linux one is simpler internally, the Windows one maps more directly onto the conceptual model.

Process states




```
Linux state names, and what they mean in `ps`:

  R  TASK_RUNNING            running OR ready — the runqueue
  S  TASK_INTERRUPTIBLE      sleeping; a signal will wake it
  D  TASK_UNINTERRUPTIBLE    sleeping; signals will NOT wake it
                             usually blocked on disk or NFS.
                             A process stuck in D cannot be killed —
                             not even by SIGKILL. This is why
                             "unkillable process" happens.
  T  TASK_STOPPED            SIGSTOP / ptrace
  Z  EXIT_ZOMBIE             finished, awaiting reaping
  I  TASK_IDLE               idle kernel thread
```

Creating a process

Unix: fork + exec

```
pid_t pid = fork();          /* duplicate the calling process */

if (pid < 0) {
        perror("fork");                     /* failed              */
} else if (pid == 0) {
        /* CHILD: fork returns 0 here */
        execl("/bin/ls", "ls", "-l", (char *)NULL);
        _exit(127);                          /* only if exec failed */
} else {
        /* PARENT: fork returns the child's pid */
        int status;
        waitpid(pid, &status, 0);
}
```

```
Why two calls instead of one "spawn"?

  Because the gap between fork and exec is where you configure the
  child, using ORDINARY code rather than a hundred spawn() flags:

      pid = fork();
      if (pid == 0) {
              close(0);  open("input.txt", O_RDONLY);   /* redirect */
              dup2(pipefd[1], 1);                        /* pipe out */
              setuid(nobody);                            /* drop privs */
              chdir("/srv");
              setrlimit(RLIMIT_CPU, &lim);
              execv(path, argv);
      }

  That is the entire design of the Unix shell — pipes, redirection
  and job control all live in that gap. Windows' CreateProcess takes
  a large parameter block instead, and cannot express arbitrary
  setup.

Cost, and why it is not as bad as it sounds:
  fork does NOT copy memory. It marks every page COPY-ON-WRITE and
  copies page TABLES only (chapter 13). Pages are duplicated lazily,
  on first write — and exec discards the address space immediately
  anyway, so most are never copied at all.

  posix_spawn() and vfork() exist for the case where even that is
  too much; posix_spawn is the portable choice.
```

FORK() IN A THREADED PROGRAM

Only the calling thread survives into the child. Every other thread vanishes *mid-operation* — including any that held a lock. If a thread was inside `malloc` when you forked, the child's heap lock is held by a thread that does not exist, and the next allocation deadlocks forever.

Between `fork` and `exec` in a multithreaded process you may call only **async-signal-safe** functions — a short list that excludes `malloc`, `printf` and most of libc. In practice: fork, immediately exec, and use `posix_spawn()` if you can, which handles this correctly.

Windows: CreateProcess

```
STARTUPINFO si = { .cb = sizeof si };
PROCESS_INFORMATION pi;

/* one call creates and starts a NEW program — no fork equivalent */
CreateProcess(
        L"C:\\Windows\\System32\\cmd.exe",   /* image        */
        cmdline,                              /* command line */
        NULL, NULL,                           /* security     */
        TRUE,                                 /* inherit handles */
        0, NULL, NULL,
        &si, &pi);

WaitForSingleObject(pi.hProcess, INFINITE);
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);      /* handles are refcounted — close both */
```

	fork + exec	CreateProcess
Configuration	Arbitrary code between the two calls	A fixed parameter block
Cost	Two syscalls; COW page-table copy	One call, no duplication
Inheritance	Everything, then close what you do not want	Opt in, per handle
Threads	Dangerous — only the caller survives	No issue
Expressiveness	Higher: pipes, redirection, privilege drop	Lower, but safer by default

Termination, zombies and orphans

```
exit(0);       /* flushes stdio, runs atexit handlers, then _exit  */
_exit(0);      /* immediate; no cleanup — correct after a failed exec */

/* Kernel-side, exit() does:
     · close all file descriptors
     · release the address space
     · send SIGCHLD to the parent
     · re-parent any children to init (or the nearest subreaper)
     · become a ZOMBIE, keeping only the exit status  */
```

```
ZOMBIE — finished, but not yet reaped

  A dead process keeps a task_struct alive purely so the parent can
  read its exit status. It holds no memory, no fds, no CPU — but it
  does hold a PID.

  It is reaped when the parent calls wait()/waitpid(). If the parent
  never does, zombies accumulate until the PID space is exhausted
  and no process can be created system-wide.

  ps aux | grep 'Z'          <- state Z, "<defunct>"

  Fixes, in order of preference:
    · call waitpid() — ideally from a SIGCHLD handler, in a loop:
          while (waitpid(-1, &st, WNOHANG) > 0) ;
    · signal(SIGCHLD, SIG_IGN)  — the kernel reaps automatically
    · double-fork, so the grandchild is orphaned to init


ORPHAN — parent died first

  The child is re-parented to PID 1 (or the nearest process that
  called PR_SET_CHILD_SUBREAPER). init reaps continuously, so
  orphans are harmless. This is the normal way daemons detach.
```

The process tree

```
pstree -p                  # the whole hierarchy
ps -ef --forest
ps -o pid,ppid,pgid,sid,tty,stat,cmd

/* Four grouping levels, each with a purpose: */
PID      one process (in Linux, one thread group)
PGID     process group — a pipeline. Ctrl+C signals the whole group.
SID      session — a login. Has one controlling terminal.
         Detaching from it (setsid) is how a daemon is born.
Namespace  a private view of PIDs, mounts, network, users.
           This is the mechanism containers are built from.
```

```
/* Classic daemonisation, and what each step is for */
if (fork() > 0) _exit(0);      /* parent exits; child is orphaned  */
setsid();                       /* new session: no controlling tty  */
if (fork() > 0) _exit(0);      /* second fork: cannot ever reacquire
                                   a terminal                       */
chdir("/");                     /* do not pin a filesystem          */
umask(0);
close(0); close(1); close(2);   /* reopen on /dev/null              */

/* Modern systems: do NOT do this. Let systemd manage the lifecycle
   (Type=simple) and log to stdout. The double-fork dance exists to
   solve problems a service manager already solves.               */
```

Inspecting a process

```
# everything the kernel knows, as files
ls /proc/self/
cat /proc/PID/status        # state, threads, uid, memory summary
cat /proc/PID/maps          # the virtual address space
cat /proc/PID/smaps         # per-mapping RSS, PSS, dirty pages
ls  /proc/PID/fd/           # open file descriptors, as symlinks
cat /proc/PID/cmdline       # NUL-separated argv
cat /proc/PID/environ
cat /proc/PID/limits        # rlimits
cat /proc/PID/stack         # kernel stack — WHY is it in D state?
ls  /proc/PID/task/         # one directory per thread

# resource usage
ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,etime,cmd -p PID
top -H -p PID               # per-thread view
/usr/bin/time -v ./program  # peak RSS, faults, context switches

# limits
ulimit -a
prlimit --pid PID
cat /sys/fs/cgroup/.../memory.max
```

DIAGNOSING AN UNKILLABLE PROCESS

A process in state **D** ignores `SIGKILL` because it is blocked in the kernel in a non-interruptible sleep. Read `/proc/PID/stack` and `/proc/PID/wchan` to see exactly which kernel function it is waiting in — almost always a storage or network filesystem call. The fix is to address the underlying I/O (a hung NFS mount, a failing disk), not the process.

Resource limits

Limit	Controls	Typical symptom when hit
`RLIMIT_NOFILE`	Open file descriptors	`EMFILE`: "too many open files" — usually an fd leak
`RLIMIT_NPROC`	Processes per user	`EAGAIN` from `fork`; often a fork bomb or leak
`RLIMIT_STACK`	Stack size	`SIGSEGV` on deep recursion
`RLIMIT_AS`	Total virtual address space	`malloc` returns NULL
`RLIMIT_CORE`	Core dump size	No core file after a crash
cgroup `memory.max`	Physical memory (containers)	OOM kill of the container
cgroup `cpu.max`	CPU bandwidth	Throttling; latency spikes

How it shows up

Symptom	Cause
Hundreds of `<defunct>` processes	Parent never calls `wait()`
"Cannot fork: resource temporarily unavailable"	PID exhaustion from zombies, or `RLIMIT_NPROC`
Process ignores `SIGKILL`	State D — uninterruptible I/O; check `/proc/PID/stack`
Child deadlocks after `fork` in a threaded program	Inherited a lock held by a thread that no longer exists
"Too many open files"	fd leak; check `ls /proc/PID/fd | wc -l`
Daemon dies when the terminal closes	Never called `setsid()`; still in the session
Child inherits an fd it should not have	Missing `O_CLOEXEC` / `FD_CLOEXEC`
Exit status is 0 despite a failure	Only the low 8 bits are returned; use the `WIFEXITED` macros
Memory usage looks doubled after `fork`	Shared COW pages counted twice; read PSS in `smaps`

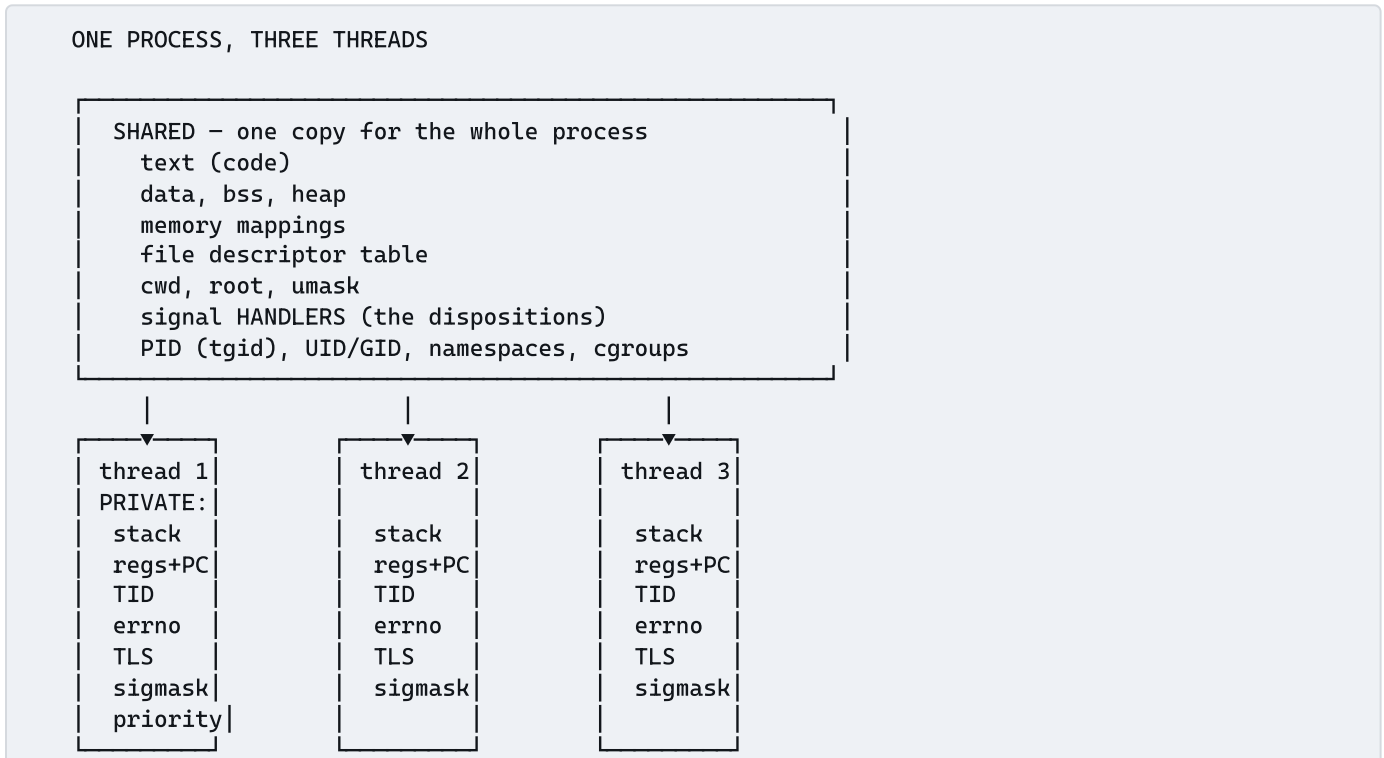
Check yourself

- ✓ Distinguish a program from a process and list four categories of process state.
- ✓ Explain why Linux has no separate process structure, and what `tgid` means.
- ✓ Name the Linux process states and explain why D-state processes cannot be killed.
- ✓ Explain what the gap between `fork` and `exec` is for, with two examples.
- ✓ Explain why `fork` is cheap despite duplicating a process.
- ✓ Explain why `fork` in a threaded program is dangerous, and the safe rule.
- ✓ Define zombie and orphan, and give two ways to prevent zombie accumulation.
- ✓ Explain what each of PID, PGID, SID and namespace groups, and what each is used for.

Threads and threading models

A thread is an independent execution context sharing an address space with its siblings. That sharing is the whole point and the whole danger: communication is free, and so is corruption.

What is shared and what is not



Note the asymmetry in signals: the HANDLER is shared, the MASK is per-thread. That is a genuine source of confusion.

THE TRADE

Threads communicate through ordinary memory — a pointer write is all it takes, at nanosecond cost. Processes must use IPC, which is safe but costs microseconds. Threads buy you cheap communication and pay for it with the entire class of data races, deadlocks and memory-ordering bugs from CA chapter 15.

The Linux model

```
/* In Linux, threads and processes are the SAME KERNEL OBJECT.
   Both are created by clone(); the flags decide what is shared. */

clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND |
      CLONE_THREAD | CLONE_SYSVSEM | CLONE_SETTLS, ...)
      /* ^ a THREAD: shares everything */

clone(SIGCHLD, ...)          /* == fork(): shares nothing */

/* So a "process" is a group of tasks with the same tgid. */
getpid()   /* returns tgid - the same for every thread */
gettid()   /* returns pid  - unique per thread          */

ls /proc/1234/task/          /* one directory per thread */
ps -L -p 1234                /* list threads             */
top -H                       /* per-thread view          */
```

> **WHY THIS DESIGN**
>
> Because the kernel needs one scheduler, one context-switch path and one wait mechanism regardless of how much is shared, the sharing is expressed as flags rather than as a separate object type. Windows takes the other approach — `EPROCESS` owns `ETHREAD`s — which matches intuition more closely at the cost of duplicating machinery.

Threading models

Model	Mapping	Property
1:1 (kernel threads)	Each user thread is one kernel thread	True parallelism; blocking is fine; context switch costs a syscall. **Linux, Windows, macOS all use this.**
N:1 (green threads)	Many user threads on one kernel thread	Very cheap switching; *one blocking call stops everything*; no multicore. Historical.
M:N	M user threads over N kernel threads	Best of both in theory; scheduler interaction is notoriously hard. Go's runtime does this well; Solaris and NPTL abandoned it.
Async / coroutines	Cooperative, one OS thread per core	No preemption, no locks needed within a task, but a blocking call stalls the whole executor

```
Why M:N mostly failed, and where it works

  The kernel scheduler and the user scheduler each make decisions
  without knowing the other's state. Classic failure: a user thread
  blocks in a syscall, taking its kernel thread with it, and the
  user scheduler cannot see that its worker vanished.

  Go succeeds because its runtime OWNS the blocking calls: every
  channel operation, network read and syscall goes through the
  runtime, which can park the goroutine and reuse the OS thread.
  That is only possible in a language with a controlled runtime.

  In C and C++ you get 1:1, and async is a library (io_uring,
  libuv, ASIO) that you must use consistently or not at all.
```

Creating threads

```
#include <pthread.h>

static void *worker(void *arg)
{
        int id = *(int *)arg;
        /* ... */
        return (void *)(intptr_t)result;
}

pthread_t t[4];
int ids[4];
for (int i = 0; i < 4; i++) {
        ids[i] = i;                     /* NOT &i - see below */
        if (pthread_create(&t[i], NULL, worker, &ids[i]) != 0)
                return -1;              /* returns errno, does NOT set it */
}
for (int i = 0; i < 4; i++) {
        void *ret;
        pthread_join(t[i], &ret);
}

/* The classic bug: passing &i, the LOOP VARIABLE. Every thread
   receives the same address, and reads whatever value it holds
   when the thread happens to run. Pass a distinct object per
   thread, or pass the value cast to intptr_t.               */
```

```
/* C11 threads - standard but thinly supported; pthreads remains
   the practical choice on POSIX.                              */
#include <threads.h>
thrd_create(&t, worker, arg);
thrd_join(t, &res);

/* Windows */
HANDLE h = CreateThread(NULL, 0, ThreadProc, param, 0, &tid);
WaitForSingleObject(h, INFINITE);
CloseHandle(h);
/* Prefer _beginthreadex in C code: it initialises the CRT's
   per-thread state (errno, strtok buffers). Raw CreateThread
   leaks that state and breaks some CRT functions.           */
```

Detached versus joinable

```
/* A joinable thread's exit status and stack are retained until
   someone joins it. Never joining leaks — the thread equivalent
   of a zombie process.                                          */

pthread_join(t, &ret);          /* wait and reclaim */

/* Or detach, if you never need the result: */
pthread_detach(t);
/* or at creation */
pthread_attr_t a;
pthread_attr_init(&a);
pthread_attr_setdetachstate(&a, PTHREAD_CREATE_DETACHED);
pthread_attr_setstacksize(&a, 256 * 1024);   /* default is 8 MB! */
pthread_create(&t, &a, worker, arg);
pthread_attr_destroy(&a);

/* Stack size matters: 8 MB of VIRTUAL address space per thread is
   fine on 64-bit, but on 32-bit it caps you at ~380 threads, and
   the guard page plus first-touch cost is real. Size it.      */
```

Thread-local storage

```
_Thread_local int counter;          /* C11              */
thread_local int counter;            /* C23 / <threads.h> */
__thread int counter;                /* GCC extension     */
__declspec(thread) int counter;      /* MSVC              */

/* Each thread gets its own instance, with static duration.
   Used for: errno, per-thread caches, random number state,
   the current transaction or request context.            */

/* Dynamic TLS, when the key is not known at compile time: */
pthread_key_t key;
pthread_key_create(&key, destructor);   /* destructor runs at thread exit */
pthread_setspecific(key, ptr);
void *p = pthread_getspecific(key);

/* Cost: a TLS access is a few instructions via a dedicated register
   (TPIDR_EL0 on AArch64). In a shared library with dynamic TLS it
   can require a function call on first access per thread.       */
```

Thread safety and reentrancy

Property	Means
Thread-safe	Correct when called concurrently from several threads — usually via internal locking
Reentrant	Correct when interrupted and called again, including from a signal handler. Stricter: implies no locks and no static state.
Async-signal-safe	Safe to call from a signal handler. Stricter still — see C ch. 16.

```
/* NOT thread-safe: returns a pointer to static storage */
char *strtok(char *s, const char *d);
struct tm *localtime(const time_t *t);
char *strerror(int e);
struct passwd *getpwnam(const char *n);

/* the _r variants take caller-supplied storage instead */
strtok_r(s, d, &saveptr);
localtime_r(t, &tm_buf);
strerror_r(e, buf, sizeof buf);
getpwnam_r(n, &pwd, buf, sizeof buf, &result);

/* Writing thread-safe code: three strategies, in order of
   preference.                                          */

/* 1. NO SHARED STATE — by far the best. Give each thread its own
      data and combine at the end.                          */
struct alignas(64) partial { long sum; };   /* padded: CA ch. 15 */
struct partial results[NTHREADS];

/* 2. IMMUTABLE SHARED STATE — read-only after construction needs
      no synchronisation at all.                            */
static const struct config *cfg;   /* set once before threads start */

/* 3. EXPLICIT SYNCHRONISATION — locks, atomics, channels.
      Necessary, but every lock is a chance to deadlock.    */
```

Signals and threads

```
/* The rules, which are unintuitive:
     · signal HANDLERS are per-PROCESS (shared)
     · signal MASKS are per-THREAD
     · a signal sent to the process is delivered to ANY ONE thread
       that has it unblocked — you do not choose which
     · pthread_kill() targets a specific thread             */

/* The standard pattern: block signals everywhere, handle them in
   one dedicated thread. This removes async-signal-safety from the
   problem entirely.                                         */
sigset_t set;
sigemptyset(&set);
sigaddset(&set, SIGINT);
sigaddset(&set, SIGTERM);
pthread_sigmask(SIG_BLOCK, &set, NULL);   /* BEFORE creating threads,
                                             so all inherit the mask */

/* ...then in one thread: */
int sig;
sigwait(&set, &sig);              /* synchronous, ordinary context —
                                     you may call anything here      */
shutdown_gracefully();

/* Linux alternative: signalfd() turns signals into a readable fd
   you can put in an epoll loop.                             */
```

Cancellation

```
pthread_cancel(t);              /* request cancellation */

/* Deferred (the default): the thread dies at the next CANCELLATION
   POINT — a blocking call such as read, write, sleep, wait.  */
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);

/* Asynchronous: dies immediately, anywhere. Almost never usable —
   it can die holding a lock or half-way through an allocation. */
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);

/* Cleanup handlers run on cancellation */
pthread_cleanup_push(unlock_it, &mutex);
        /* ... critical section ... */
pthread_cleanup_pop(1);         /* 1 = also run it on normal exit */
```

PREFER COOPERATIVE SHUTDOWN

`pthread_cancel` is difficult to use correctly: every resource must have a cleanup handler, and a cancellation point can appear inside library code you did not write. In practice, use an atomic flag plus a wakeup mechanism (a pipe, an eventfd, a condition variable) and let the thread exit its own loop. Windows does not offer cancellation at all — `TerminateThread` is documented as unsafe and should never be used.

Thread pools

```
Creating a thread per unit of work does not scale:

   thread creation      ~10–100 µs and ~8 MB of virtual address space
   context switching    grows with thread count
   cache footprint      each thread evicts the others
   scheduler overhead   O(log n) or worse per switch

A POOL fixes the count and queues the work:

   [ task queue ] ──► worker 1
                  ├──► worker 2
                  ├──► worker 3
                  └──► worker 4      (typically nproc, or nproc+1)

Sizing:
   CPU-bound      ≈ number of cores
   I/O-bound      more, since most workers are blocked
   mixed          separate pools, so slow I/O cannot starve compute

Beware of the work-queue lock becoming the bottleneck — that is
what work-stealing deques solve, and why Rayon, TBB and Go's
scheduler all use per-worker queues.
```

Debugging threaded programs

```
# ThreadSanitizer — finds data races that have not yet manifested.
# The single most valuable tool here. ~5–15x slowdown.
gcc -fsanitize=thread -g prog.c && ./prog

# Helgrind / DRD — race and lock-order detection, no rebuild needed
valgrind --tool=helgrind ./prog

# gdb
info threads
thread 3
thread apply all bt          # backtrace of EVERY thread — the first
                             # thing to run on a deadlock

# what are the threads doing right now?
top -H -p PID
ps -L -o pid,tid,psr,pcpu,stat,wchan,comm -p PID
cat /proc/PID/task/TID/stack        # kernel-side, for D state

# contention
perf lock record ./prog && perf lock report
perf sched latency
```

How it shows up

Symptom	Cause
All threads get the same argument value	Passed `&i`, the shared loop variable
Adding threads makes it slower	False sharing or lock contention (CA ch. 15)
Works with 2 threads, fails with 16	A race whose window widened
Fails only on ARM, not x86	Missing barriers; TSO hid it
Memory grows with each completed thread	Joinable threads never joined
Cannot create more than ~380 threads on 32-bit	8 MB default stack exhausting address space
Signal handled by an unexpected thread	Process-directed signals go to any unblocked thread
`strtok` returns wrong results	Static state shared between threads; use `strtok_r`
Deadlock after cancelling a thread	Cancelled while holding a lock
Child process hangs after `fork`	Inherited a lock held by a thread that did not survive

Check yourself

- ✓ List what threads share and what is private, including the signal asymmetry.
- ✓ Explain why Linux uses one object for threads and processes, and what `tgid` distinguishes.
- ✓ Compare 1:1, N:1 and M:N, and say why M:N works for Go but not for C.
- ✓ Give the `&i` bug and two correct ways to pass per-thread data.
- ✓ Explain joinable versus detached and the leak that follows from confusing them.
- ✓ Distinguish thread-safe, reentrant and async-signal-safe.
- ✓ Describe the standard pattern for handling signals in a threaded program.
- ✓ Explain why `pthread_cancel` is hard to use and what to do instead.

CPU scheduling algorithms

The scheduler decides which thread runs next, and it must satisfy goals that directly contradict each other. Understanding which goal a scheduler is optimising explains every design decision it makes — and tells you which one your workload actually needs.

Goals that conflict

Goal	Definition	Conflicts with
Throughput	Work completed per second	Fairness, latency — switching costs time
Latency / response	Time from ready to running	Throughput — frequent preemption thrashes caches
Fairness	Proportional share of CPU	Priority, throughput
Priority	Important work first	Fairness — and risks starvation
Determinism	Bounded worst case	Nearly everything else
Efficiency	Low scheduler overhead itself	Sophistication of the policy

THE ONE QUESTION TO ASK

"Is this system optimising the *average* case or the *worst* case?" A general-purpose OS optimises the average — it will happily add 10 ms of jitter to gain 5% throughput. An RTOS optimises the worst case and gives up throughput without hesitation. Almost every argument about scheduler behaviour is really a disagreement about which of these the system should do.

Preemptive versus cooperative

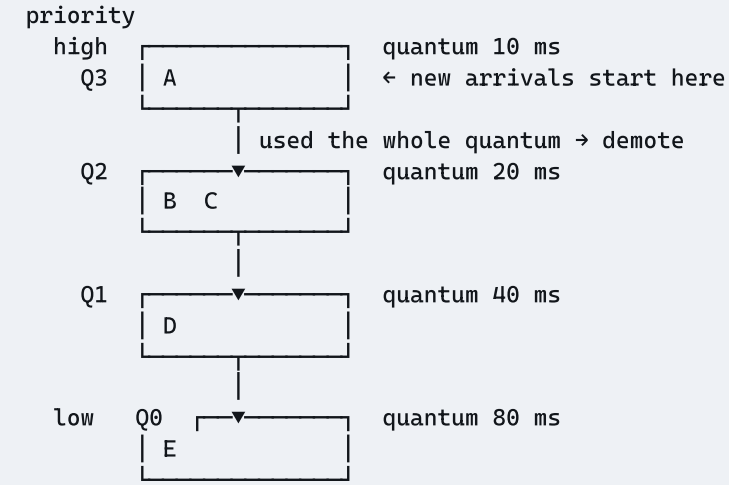
```
COOPERATIVE           a thread runs until it yields or blocks
                      + no preemption points, so no locks needed
                        between yields — much simpler reasoning
                      + very low overhead
                      − one runaway thread hangs the system
                      − a buggy thread can never be recovered
                      used by: async runtimes, some RTOSes,
                               classic Mac OS and Windows 3.x

PREEMPTIVE            a timer interrupt can switch at any moment
                      + one thread cannot monopolise the CPU
                      + latency is bounded by the tick or better
                      − every shared structure needs synchronisation
                      − context switch cost on every switch
                      used by: every general-purpose OS

Linux preemption levels, chosen at build time:
  PREEMPT_NONE        kernel code never preempted — best throughput
  PREEMPT_VOLUNTARY   explicit reschedule points in long loops
  PREEMPT             kernel is preemptible except in critical
                      sections — the desktop default
  PREEMPT_RT          almost fully preemptible: most spinlocks
                      become sleeping mutexes and IRQ handlers
                      become threads. Latency in tens of µs,
                      at a throughput cost.
```

The classic algorithms

Algorithm	Rule	Strength	Weakness
FCFS / FIFO	First to arrive runs to completion	Trivially simple, no starvation	*Convoy effect*: one long job blocks everything behind it
SJF	Shortest job first	*Provably optimal* average wait time	Requires knowing job length; starves long jobs
SRTF	Preemptive SJF	Even better average	Same problems, plus more switching
Round robin	Each gets one time slice, cyclically	Fair, responsive, no starvation	Quantum choice is a hard trade-off
Priority	Highest priority runs	Expresses importance	Starvation; priority inversion
MLFQ	Several queues; demote CPU hogs, promote I/O-bound tasks	Approximates SJF *without* knowing job lengths	Tunable in many ways; gameable
Fair-share / CFS	Track CPU consumed; run whoever is furthest behind	Fair by construction; no heuristics to tune	Not real-time
Lottery / stride	Tickets proportional to share	Simple proportional sharing	Randomness gives poor short-term fairness

```
The quantum trade-off, made concrete

  Context switch cost ≈ 1–5 µs (plus cache and TLB refill)

    quantum 1 ms   →  overhead 0.1–0.5%   latency up to 1 ms
    quantum 10 ms  →  overhead 0.01–0.05% latency up to 10 ms
    quantum 100 µs →  overhead 1–5%       latency up to 100 µs

  Interactive work wants short quanta; batch work wants long ones.
  This is precisely why MLFQ exists: give short quanta at high
  priority to tasks that block quickly, and long quanta at low
  priority to tasks that burn their whole slice.
```

Multi-level feedback queue




```
Rules:
 1. run the highest non-empty queue, round robin within it
 2. a job that uses its full quantum is DEMOTED (it is CPU-bound)
 3. a job that blocks early stays or is PROMOTED (it is I/O-bound,
    and is probably interactive)
 4. periodically boost everything back to the top — this prevents
    starvation, and defeats gaming rule 3 by yielding just before
    the quantum expires

This infers job length from OBSERVED behaviour, which is why it
approximates SJF without an oracle. Windows and older Linux both
used variants of this.
```

Fair scheduling: CFS and EEVDF

```
/* CFS (Completely Fair Scheduler), Linux 2.6.23–6.5, replaced the
   heuristic MLFQ with a single idea:                              */

   VIRTUAL RUNTIME (vruntime) = actual runtime, weighted by nice

       vruntime += delta_exec * (NICE_0_WEIGHT / task_weight)

   · a low-nice (high-priority) task has a large weight, so its
     vruntime advances SLOWLY, so it is picked more often
   · always run the task with the SMALLEST vruntime
   · tasks are kept in a red-black tree keyed by vruntime, so
     "smallest" is O(1) to find and O(log n) to reinsert

   No heuristics, no priority boosting, no interactivity guessing —
   just: whoever has had the least CPU (weighted) runs next.

   nice values map to weights on a ~1.25x-per-step curve:
       nice  0 → weight 1024
       nice -5 → weight 3121   (~3x more CPU)
       nice  5 → weight  335   (~3x less)
       nice ±1 is roughly a 10% change in share
```

```
/* EEVDF (Earliest Eligible Virtual Deadline First) replaced CFS in
   Linux 6.6. Same fairness core, plus an explicit LATENCY term:

     · each task gets a virtual deadline based on its requested
       latency (settable per task via sched_setattr)
     · among ELIGIBLE tasks (those not ahead of their fair share),
       run the earliest deadline
     · so a task can ask for low latency WITHOUT asking for more
       CPU — which CFS could not express

   This matters for audio, games and interactive work, which want
   to be scheduled QUICKLY but not necessarily OFTEN.          */

chrt --help                      # see and set policies
nice -n 10 ./batch_job
renice -n -5 -p 1234
schedtool -n -5 -e ./program
```

Real-time scheduling

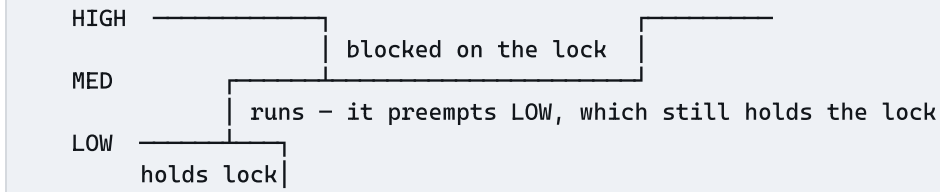
Policy	Rule	Use for
`SCHED_FIFO`	Priority 1–99. Runs until it blocks or yields. **No time slicing.**	Short, bounded, critical work
`SCHED_RR`	Same, plus round-robin among equal priorities	Several equal-priority RT tasks
`SCHED_DEADLINE`	Specify runtime, deadline and period explicitly. EDF + constant bandwidth server.	Periodic tasks with real deadlines
`SCHED_OTHER`	The fair scheduler	Everything else
`SCHED_IDLE`	Only when nothing else wants the CPU	Background scavenging

```
#include <sched.h>
struct sched_param p = { .sched_priority = 50 };
sched_setscheduler(0, SCHED_FIFO, &p);

/* SCHED_DEADLINE — say what you actually need */
struct sched_attr attr = {
        .size          = sizeof attr,
        .sched_policy  = SCHED_DEADLINE,
        .sched_runtime =  2 * 1000 * 1000,   /*  2 ms of CPU     */
        .sched_deadline = 8 * 1000 * 1000,   /*  within 8 ms     */
        .sched_period  = 10 * 1000 * 1000,   /*  every 10 ms     */
};
syscall(SYS_sched_setattr, 0, &attr, 0);
/* The kernel performs an ADMISSION TEST: if total utilisation
   would exceed capacity, the call fails rather than silently
   missing deadlines. That is what makes it genuinely real-time. */
```

A `SCHED_FIFO` INFINITE LOOP HANGS THE MACHINE

It never yields, and nothing at lower priority — including your shell — will ever run again. Protections: `sched_rt_runtime_us` reserves a slice for non-RT tasks (95% RT by default), and you should always test RT code on an isolated CPU. Reserve `SCHED_FIFO` for short, provably bounded work, and always pair it with `mlockall()` so a page fault cannot introduce unbounded delay.

Priority inversion




```
HIGH waits on LOW; MED preempts LOW; therefore MED delays HIGH
indefinitely. Priority is effectively inverted.

This is not theoretical: it nearly ended the Mars Pathfinder
mission in 1997. The fix was uploaded from Earth.

SOLUTIONS
 Priority inheritance  the lock holder temporarily inherits the
                       highest priority among its waiters.
                       → PTHREAD_PRIO_INHERIT, and RT-mutexes
                         throughout PREEMPT_RT
 Priority ceiling      the lock carries a priority; anyone taking
                       it is raised to that level immediately
 Avoid sharing         lock-free structures, or per-CPU data
```

```
pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_setprotocol(&a, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&m, &a);
/* Costs a little on every lock and unlock. Mandatory when RT and
   non-RT threads share a mutex.                                  */
```

Multiprocessor scheduling

Concern	Mechanism
Per-CPU runqueues	Avoids one global lock. Linux has one runqueue per CPU.
Load balancing	Periodically migrate tasks between runqueues; also on wake-up and fork
Cache affinity	Prefer the CPU a task last ran on — its cache is still warm. Migration costs a full cache refill.
Scheduling domains	A hierarchy — SMT sibling → core → cluster → NUMA node — with balancing cheapest at the closest level
NUMA	Keep a task near its memory; migrating across nodes can double memory latency
Heterogeneous cores	big.LITTLE: energy-aware placement. Small tasks to LITTLE, bursty ones to big.

```
# pin to specific CPUs — essential for benchmarking and RT
taskset -c 2,3 ./program
taskset -pc 2 1234

# isolate CPUs from the general scheduler entirely (kernel cmdline)
isolcpus=2,3 nohz_full=2,3 rcu_nocbs=2,3
# then place your RT threads there with taskset. This is the
# standard recipe for low-latency work on Linux.

# NUMA placement
numactl --cpunodebind=0 --membind=0 ./program
numastat -p 1234
```

Observing the scheduler

```
# where is the time going?
vmstat 1                    # r = runnable, cs = context switches/s
pidstat -w -p PID 1         # voluntary vs involuntary switches
                            #   voluntary   = blocked on I/O or a lock
                            #   involuntary = preempted (CPU contention)

# per-task scheduler statistics
cat /proc/PID/sched         # vruntime, wait time, migrations
cat /proc/PID/status | grep -i ctxt
cat /proc/schedstat

# tracing
perf sched record ./workload && perf sched latency
perf sched timehist
trace-cmd record -e sched:'*' -- ./workload

# latency measurement for RT tuning
cyclictest -t4 -p 90 -i 200 -m -D 60
#   reports min/avg/MAX latency. The MAX is the number that matters.
```

READING THE TWO SWITCH COUNTS

High **voluntary** context switches means the task blocks a lot — it is I/O-bound, and the fix is usually fewer, larger operations. High **involuntary** switches means it is being preempted — the machine is oversubscribed, or something with higher priority keeps arriving. They call for completely different responses, and `pidstat -w` distinguishes them in one line.

How it shows up

Symptom	Cause
Occasional multi-millisecond latency spikes	Preemption, or a major page fault; use RT policy plus `mlockall`
System unresponsive but CPU is idle	Tasks blocked on I/O (D state), not on CPU
Machine hangs after starting an RT thread	`SCHED_FIFO` loop that never yields
High-priority task delayed by a low-priority one	Priority inversion; enable priority inheritance
Benchmark varies 3× between runs	Migration between big and LITTLE cores; pin with `taskset`
Throughput drops when threads exceed core count	Context-switch overhead and cache thrashing
`nice` has little effect	The task is I/O-bound; nice only allocates CPU
Performance worse after adding NUMA nodes	Memory allocated on a remote node; use `numactl`
Audio glitches under load	Latency, not throughput; needs an RT policy or EEVDF latency hint

Check yourself

- ✓ Name four scheduling goals and a pair that directly conflict.
- ✓ Explain the convoy effect and which algorithm suffers from it.
- ✓ Explain why SJF is optimal and why it cannot be implemented directly.
- ✓ Describe MLFQ's four rules and why rule 4 exists.

- ✓ Define vruntime and explain how nice values change the share.
- ✓ Say what EEVDF adds over CFS and which workloads care.
- ✓ Explain priority inversion and two mechanisms that solve it.
- ✓ Explain the difference between voluntary and involuntary context switches, and what each implies.

Context switching, step by step

The mechanism that creates the illusion of concurrency. It is conceptually simple — save one thread's state, restore another's — and the details of what counts as "state", and what it costs beyond the registers, determine the performance of the entire system.

When a switch happens

```
Only ever from inside the kernel, and only for three reasons —
matching the three ways the kernel gains control (chapter 01):

  1. THE THREAD BLOCKS            voluntary
     read() with no data ready, lock contended, sleep(), wait()
     → mark it not-runnable, pick someone else

  2. A TIMER INTERRUPT FIRES      involuntary, preemption
     the quantum expired, or a higher-priority task woke
     → the scheduler is invoked on the way out of the interrupt

  3. A THREAD BECOMES RUNNABLE    involuntary, preemption
     an interrupt completes I/O and wakes a higher-priority thread
     → preempt immediately, if the policy allows

Linux marks the current task with TIF_NEED_RESCHED rather than
switching immediately. The actual switch happens at the next safe
point — on the way back to user space, or when a preemption-disabled
region ends. That is why PREEMPT_NONE vs PREEMPT changes latency
without changing the mechanism.
```

What "context" means

State	Saved by	Cost
General-purpose registers	Software (the switch routine)	~30 stores + 30 loads
PC and stack pointer	Software	Included above
PSTATE / flags	Hardware on exception entry	Free
FP / SIMD registers	Software, usually **lazily**	512+ bytes; deferred until used
Page tables (`TTBR0`)	Software, only on *process* switch	One register write, plus TLB effects
Thread-local base (`TPIDR_EL0`)	Software	One register
Kernel stack pointer	Software	One register
Caches and TLBs	**Nobody** — they are just wrong afterwards	**The dominant cost**

THE REGISTER SAVE IS NOT THE EXPENSIVE PART

Saving and restoring 31 registers is perhaps 100 cycles. The real cost is that the new thread arrives to find the caches full of the *old* thread's data, the branch predictor trained on the old thread's branches, and possibly a cold TLB. Refilling those costs thousands of cycles and does not appear anywhere in the switch routine. This is why the measured cost of a context switch ranges from ~1 µs to ~50 µs depending entirely on working-set size.

Thread switch versus process switch

```
SAME PROCESS (thread → thread)          DIFFERENT PROCESS
  save/restore registers                  save/restore registers
  swap kernel stack                       swap kernel stack
  swap TLS register                       swap TLS register
  - page tables UNCHANGED -               SWAP TTBR0_EL1
  - TLB stays valid -                     TLB entries now belong to
  - caches stay warm-ish -                  another ASID
                                          caches likely cold

  ~1-2 µs                                 ~2-5 µs, plus a much
                                          larger refill penalty

This is the concrete reason threads are "cheaper" than processes,
and why thread pools beat process pools for fine-grained work.
```

ASIDs make process switches survivable

```
/* Without address space identifiers, changing TTBR0 would require
   flushing the entire TLB - and the new process would then take
   hundreds of TLB misses warming it back up.

   With ASIDs (ARM) / PCIDs (x86), each TLB entry is tagged with
   its owning address space. A switch changes the current ASID;
   entries from BOTH processes coexist. No flush.              */

msr     ttbr0_el1, x0        /* x0[63:48] holds the ASID */
isb

/* Kernel mappings are marked GLOBAL (nG=0), so they match under any
   ASID and survive every switch - which is why a syscall does not
   disturb the TLB at all.                                      */

/* ASIDs are a limited resource (8 or 16 bits). When they are
   exhausted the kernel rolls over the generation and DOES flush
   everything. This is rare and shows up as a periodic latency
   blip on systems with very many processes.                   */
```

> **KPTI MADE THIS WORSE**
>
> Kernel Page Table Isolation, the Meltdown mitigation, gives the kernel a *separate* page table so that kernel addresses are not mapped while user code runs. Every system call and every interrupt therefore switches address space twice. On affected CPUs this added roughly 5–30% to syscall-heavy workloads. ARM cores were mostly not vulnerable and do not pay this; most x86 cores do, unless mitigated in hardware.

The switch itself

```
/* Linux arm64, simplified from arch/arm64/kernel/entry.S.
   Only the CALLEE-SAVED registers need saving here — this runs as
   an ordinary C function call, so the caller already saved the
   volatile ones per the ABI (C chapter 11).                    */

SYM_FUNC_START(cpu_switch_to)
        /* x0 = prev task, x1 = next task */
        mov     x10, #THREAD_CPU_CONTEXT
        add     x8, x0, x10                 /* &prev->thread.cpu_context */
        mov     x9, sp

        stp     x19, x20, [x8], #16         /* save callee-saved */
        stp     x21, x22, [x8], #16
        stp     x23, x24, [x8], #16
        stp     x25, x26, [x8], #16
        stp     x27, x28, [x8], #16
        stp     x29, x9,  [x8], #16         /* FP and SP         */
        str     lr, [x8]                     /* return address    */

        add     x8, x1, x10                 /* &next->thread.cpu_context */
        ldp     x19, x20, [x8], #16         /* restore           */
        ldp     x21, x22, [x8], #16
        ldp     x23, x24, [x8], #16
        ldp     x25, x26, [x8], #16
        ldp     x27, x28, [x8], #16
        ldp     x29, x9,  [x8], #16
        ldr     lr, [x8]
        mov     sp, x9

        msr     sp_el0, x1                  /* `current` for the new task */
        ret                                  /* returns into NEXT's flow */
SYM_FUNC_END(cpu_switch_to)
```

> **THE TRICK WORTH APPRECIATING**
>
> `cpu_switch_to` is called by one thread and *returns into a different one*. The `ret` at the end jumps to the link register restored from the next task's saved context — which is wherever *that* thread last called this function. Every thread's stack is paused mid-call in the same place. A newly created thread has its saved context hand-crafted so that this `ret` lands on its entry function instead.

Lazy FPU switching

```
/* FP/SIMD state is large — 32 x 128-bit V registers, or far more
   with SVE (up to 2048 bits each). Saving it on every switch would
   dominate the cost, and most threads never touch it.        */

/* The optimisation:
     1. on switch, DISABLE FP access for the incoming thread
        (CPACR_EL1.FPEN = 0)
     2. if it never uses FP, nothing was saved or restored — free
     3. if it executes an FP instruction, a trap fires; the kernel
        then saves the previous owner's state, restores this
        thread's, enables FP, and retries the instruction     */

/* Consequences you can observe:
     · the FIRST FP instruction after a switch is expensive
     · in the kernel, FP is FORBIDDEN by default precisely because
       its state is not saved on kernel entry (C chapter 16).
       Kernel code that needs it must bracket it explicitly:  */
kernel_neon_begin();
    /* SIMD-accelerated crypto or checksum here */
kernel_neon_end();
```

Measuring the cost

```
Direct cost — the switch routine itself
    save/restore registers            ~100 cycles
    scheduler decision (EEVDF)        ~200–500 cycles
    page table swap (process switch)  ~50 cycles + barriers

Indirect cost — the part nobody counts, and the part that matters
    cold L1/L2 for the new working set   100s–1000s of cycles
    TLB refill                            100s of cycles
    branch predictor retraining           100s of cycles

  Measured end to end:
    thread ⇔ thread, tiny working set      ~1 µs
    process ⇔ process, tiny working set    ~2 µs
    process ⇔ process, 1 MB working set    ~10–50 µs

The variance IS the point: the switch has a fixed cost and a
working-set-dependent cost, and the second one dominates.
```

```
# count them
vmstat 1                        # cs column: switches per second
pidstat -w -p PID 1             # voluntary vs involuntary, per task
perf stat -e context-switches,cpu-migrations ./program
cat /proc/PID/status | grep ctxt

# see them
perf sched record ./workload && perf sched timehist
trace-cmd record -e sched:sched_switch -- ./workload

# measure the raw cost on your machine
perf bench sched pipe           # ping-pong through a pipe
perf bench sched messaging

# a rough rule: if `cs` is above ~50,000/s on a few cores, switching
# itself is a measurable fraction of your CPU budget.
```

Reducing switch cost

Technique	Effect
Fewer threads — a pool sized to core count	Removes switching that exists only because of oversubscription
Batch I/O — `io_uring`, `readv`	Fewer blocking points, so fewer voluntary switches
Async / coroutines	Switching in user space costs ~20 ns, not ~1 µs — no kernel involved
CPU pinning (`taskset`)	Prevents migration, preserves cache affinity
Longer quanta for batch work	Fewer preemptions; hurts latency
Spin briefly before blocking	Adaptive mutexes: if the lock will be free within ~1 µs, spinning is cheaper than switching twice
Busy-poll for ultra-low latency	Never sleep at all — burns a core to remove the wake-up path entirely
Huge pages	Fewer TLB entries to refill after a switch

```
/* Adaptive mutexes make the spin-versus-block decision automatically:
   spin while the lock HOLDER IS RUNNING on another CPU (it will
   likely release soon), block if the holder is descheduled (it
   will not).                                                  */

/* glibc: */
pthread_mutexattr_settype(&a, PTHREAD_MUTEX_ADAPTIVE_NP);

/* This is why uncontended and briefly-contended mutexes are
   nearly free, and heavily-contended ones are catastrophic —
   the cost cliff is exactly one context switch pair.         */
```

Context switching in an RTOS

```
/* On a Cortex-M with no MMU, a switch is far simpler and far
   faster — typically 50–200 cycles.                          */

void PendSV_Handler(void)      /* the standard RTOS switch hook */
{
        /* hardware already stacked R0–R3, R12, LR, PC, xPSR */
        __asm volatile (
        "   mrs   r0, psp            \n"   /* process stack pointer */
        "   stmdb r0!, {r4–r11}      \n"   /* save the rest         */
        "   ldr   r1, =current_tcb   \n"
        "   ldr   r2, [r1]           \n"
        "   str   r0, [r2]           \n"   /* save SP into the TCB  */
        "   bl    select_next_task   \n"
        "   ldr   r2, [r1]           \n"
        "   ldr   r0, [r2]           \n"   /* new SP                */
        "   ldmia r0!, {r4–r11}      \n"
        "   msr   psp, r0            \n"
        "   bx    lr                 \n"); /* hardware unstacks     */
}

/* Why it is so much cheaper:
     · no page tables, no TLB, no ASID
     · caches are small or absent, so little to lose
     · the hardware stacks half the registers for you
     · no scheduler tree — often a simple priority bitmap

   PendSV is used because it is the LOWEST-priority exception, so
   the switch happens only after all pending interrupts are handled.
   Switching from inside an ISR would otherwise nest badly.     */
```

How it shows up

Symptom	Cause
Throughput falls as threads exceed core count	Switching overhead and cache thrashing
Very high `cs` in `vmstat`	Oversubscription, or a lock ping-ponging between threads
High *voluntary* switches	Blocking frequently — batch the I/O
High *involuntary* switches	CPU contention — reduce thread count or pin
Latency improves dramatically after pinning	Migration was destroying cache affinity
Syscall-heavy workload slowed after a kernel update	KPTI: two address-space switches per syscall
First floating-point operation after a switch is slow	Lazy FPU restore trap
Async runtime far outperforms threads on I/O	User-space switching is ~50× cheaper
Periodic latency blip on a many-process system	ASID rollover forcing a full TLB flush
Kernel oops after using floating point in a driver	FP state not saved on kernel entry; use `kernel_neon_begin()`

Check yourself

- ✓ Name the three events that cause a context switch and which are voluntary.
- ✓ List what is saved on a switch and identify the item nobody saves.
- ✓ Explain why the register save is not the dominant cost.

- ✓ Give the difference between a thread switch and a process switch.
- ✓ Explain what ASIDs do and what happens when they run out.
- ✓ Explain lazy FPU switching and why kernel code cannot use floating point freely.
- ✓ Explain how `cpu_switch_to` can return into a different thread.
- ✓ Give four ways to reduce context-switch cost, and when each applies.

Interprocess communication

Isolation is the point of a process, so any communication must be arranged deliberately through the kernel. The mechanisms differ in throughput, latency, direction, naming and what they can carry — and choosing wrongly is the difference between microseconds and milliseconds.

The choices that matter

Question	Options	Consequence
Data or control?	Bulk bytes vs a signal or event	Shared memory vs eventfd
How much data?	Bytes vs megabytes	Copy cost decides everything
Same machine?	Local vs network	Unix sockets vs TCP
Related processes?	Parent/child vs unrelated	Anonymous pipe vs a named object
Message or stream?	Preserved boundaries vs a byte stream	Datagram vs stream socket
One-to-one or many?	Point-to-point vs multicast	Pipe vs pub/sub
Synchronisation included?	Blocking built in, or your problem	Pipes block; shared memory does not

```
Rough cost, same machine, small messages:

  shared memory + atomics    ~20–100 ns     no syscall in steady state
  futex wake                 ~1–5 µs        only when actually blocking
  pipe / socketpair          ~5–15 µs       two copies + two switches
  Unix domain socket         ~5–20 µs
  POSIX message queue        ~5–20 µs
  D-Bus / Binder             ~50–200 µs     plus marshalling
  loopback TCP               ~20–50 µs      full protocol stack

The pattern: anything that copies through the kernel costs
microseconds. Shared memory costs nanoseconds but hands you the
entire synchronisation problem in exchange.
```

Pipes

```
int fd[2];
pipe(fd);                       /* fd[0] read end, fd[1] write end */

if (fork() == 0) {
        close(fd[0]);                   /* child writes           */
        dup2(fd[1], STDOUT_FILENO);     /* stdout → pipe          */
        close(fd[1]);
        execlp("ls", "ls", NULL);
}
close(fd[1]);                           /* parent MUST close its
                                           write end, or it never
                                           sees EOF               */
char buf[4096];
ssize_t n;
while ((n = read(fd[0], buf, sizeof buf)) > 0)
        process(buf, n);
/* read returns 0 (EOF) only when ALL write ends are closed */
```

Property	Behaviour
Direction	Unidirectional. Two pipes for bidirectional, or use `socketpair()`.
Naming	Anonymous — inherited across `fork`. `mkfifo()` gives a named pipe for unrelated processes.
Boundaries	A byte stream. Two writes may be read as one. **Frame it yourself.**
Capacity	64 KB by default (`F_SETPIPE_SZ` to change). Writer blocks when full; reader blocks when empty.
Atomicity	Writes up to `PIPE_BUF` (4096) are atomic — they will not interleave with other writers.
Closed reader	Writing gives `SIGPIPE`, which kills the process by default. Almost always you want `signal(SIGPIPE, SIG_IGN)` and to handle `EPIPE`.

THE TWO PIPE BUGS EVERYONE WRITES

Forgetting to close unused ends. If the parent keeps its copy of the write end open, the reader never sees EOF and hangs forever. Every process must close every descriptor it does not use.

Assuming message boundaries. A pipe is a byte stream. If you `write()` two 100-byte messages, the reader may get 200 bytes in one `read()`, or 37 then 163. Prefix each message with a length, or use a datagram mechanism.

Sockets

```
/* Unix domain sockets: the default choice for local IPC.
   Same API as network sockets, but no protocol stack overhead. */

int sv[2];
socketpair(AF_UNIX, SOCK_STREAM, 0, sv);   /* bidirectional pipe */

/* Named, for unrelated processes */
int s = socket(AF_UNIX, SOCK_STREAM, 0);
struct sockaddr_un a = { .sun_family = AF_UNIX };
strcpy(a.sun_path, "/run/myservice.sock");
bind(s, (struct sockaddr *)&a, sizeof a);
listen(s, 16);
int c = accept(s, NULL, NULL);

/* SOCK_DGRAM preserves message boundaries;
   SOCK_SEQPACKET gives boundaries AND reliable ordering.     */
```

```
/* The killer feature: passing FILE DESCRIPTORS between processes.
   No other local IPC mechanism can do this.                    */

struct msghdr msg = {0};
char cmsgbuf[CMSG_SPACE(sizeof(int))];
msg.msg_control = cmsgbuf;
msg.msg_controllen = sizeof cmsgbuf;

struct cmsghdr *cm = CMSG_FIRSTHDR(&msg);
cm->cmsg_level = SOL_SOCKET;
cm->cmsg_type  = SCM_RIGHTS;              /* "here is an fd"      */
cm->cmsg_len   = CMSG_LEN(sizeof(int));
memcpy(CMSG_DATA(cm), &fd_to_send, sizeof(int));

sendmsg(sock, &msg, 0);

/* The receiver gets a NEW descriptor in its own table, referring
   to the same open file. This is how systemd socket activation,
   privilege separation, Wayland buffer passing and container
   runtimes all work — a privileged process opens the resource
   and hands the descriptor to an unprivileged one.
   SCM_CREDENTIALS similarly passes verified pid/uid/gid.      */
```

Shared memory

```
/* POSIX shared memory — the fastest mechanism, and the one with
   no synchronisation included.                                 */

int fd = shm_open("/myshm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, SIZE);
void *p = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
close(fd);                      /* the mapping keeps it alive     */
/* ... both processes now see the same physical pages ...        */
munmap(p, SIZE);
shm_unlink("/myshm");           /* remove the name                */

/* Anonymous shared memory, for related processes: */
void *p = mmap(NULL, SIZE, PROT_READ | PROT_WRITE,
               MAP_SHARED | MAP_ANONYMOUS, -1, 0);
fork();                         /* both see the same pages        */

/* memfd_create: an anonymous, sealable region you can pass by fd */
int fd = memfd_create("buf", MFD_ALLOW_SEALING);
fcntl(fd, F_ADD_SEALS, F_SEAL_SHRINK | F_SEAL_GROW);
/* Sealing lets the receiver TRUST that the sender cannot shrink
   the region underneath it — otherwise a malicious sender causes
   SIGBUS in the receiver. This is what Android and Wayland use. */
```

SHARED MEMORY GIVES YOU NO SYNCHRONISATION WHATSOEVER

You get raw memory shared between address spaces. Everything from CA chapter 15 applies in full: you need atomics with correct memory ordering, or a lock placed *in the shared region itself*. And because the peer may be malicious, you must additionally validate everything you read — a shared-memory protocol is a trust boundary.

```
/* A lock that lives in the shared region */
pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_setpshared(&a, PTHREAD_PROCESS_SHARED);   /* essential */
pthread_mutexattr_setrobust(&a, PTHREAD_MUTEX_ROBUST);      /* see below */
pthread_mutex_init(&shared->lock, &a);

/* ROBUST matters: if the peer dies holding the lock, a normal
   mutex is stuck forever. A robust mutex returns EOWNERDEAD to
   the next acquirer, which can repair the state and call
   pthread_mutex_consistent().                                */
```

Signalling without data

Mechanism	Use	Note
`eventfd`	A counter you can `poll()`	The standard way to wake an event loop from another thread or process
`signalfd`	Signals as a readable fd	Removes async-signal-safety from the problem entirely
`timerfd`	Timers as a readable fd	Integrates cleanly with `epoll`
`futex`	Fast userspace mutex	The primitive under every Linux lock — see below
POSIX semaphores	`sem_open`, `sem_wait`	Named or in shared memory
Signals	`kill(pid, SIGUSR1)`	No payload, can be coalesced, async-safety restrictions

```
The futex, which is worth understanding once

  A mutex that never contends should cost ZERO syscalls. A mutex
  that does contend must eventually sleep, which requires the kernel.
  futex gives you both:

    lock:   atomic compare-and-swap in USER SPACE
            uncontended → done, ~20 ns, no syscall
            contended   → futex(FUTEX_WAIT) → kernel puts you to sleep

    unlock: atomic store in user space
            if waiters were recorded → futex(FUTEX_WAKE)

  The kernel is involved ONLY on contention. This is why an
  uncontended pthread_mutex_lock is nearly free and a contended
  one costs microseconds — the cliff is exactly one syscall pair.

  Every Linux mutex, condition variable, semaphore and barrier is
  built on this.
```

System V IPC

```
/* The 1983 API: message queues, semaphores and shared memory,
   named by integer keys.                                      */
key_t key = ftok("/some/path", 'A');
int shmid = shmget(key, SIZE, IPC_CREAT | 0600);
void *p = shmat(shmid, NULL, 0);

/* Avoid in new code:
     · a global integer namespace with collisions and no cleanup
     · objects PERSIST after all processes exit (ipcs / ipcrm)
     · not pollable — cannot be used with epoll
     · clumsier API

   POSIX equivalents (shm_open, mq_open, sem_open) are namespaced
   under a filesystem path, follow fd semantics, and are pollable.
   Use those.                                                  */

ipcs -a          # list leftover System V objects
ipcrm -m ID      # clean them up manually
```

Higher-level IPC

System	Built on	Adds
D-Bus	Unix sockets	Naming, method calls, signals, introspection, activation. Desktop and systemd.
Binder	A kernel driver	Android's IPC: reference-counted object passing, one-copy transfers, priority inheritance across calls
gRPC / Cap'n Proto	Sockets	Schema, serialisation, versioning, works locally or over a network
Wayland	Unix sockets + `SCM_RIGHTS`	Passes graphics buffers by fd — zero copy for pixels
io_uring	Shared memory rings	Kernel/user "IPC" with often zero syscalls in steady state
virtio	Shared memory rings	Guest ↔ host communication; the same ring design as io_uring

> **THE RECURRING DESIGN**
>
> Notice that io_uring, virtio, Binder and every high-performance IPC converge on the same structure: a **shared-memory ring buffer** with atomic head and tail indices, plus a doorbell (an eventfd, an MMIO write, a futex) used only when the peer needs waking. Bulk data by shared memory; wake-ups by syscall, as rarely as possible. Once you recognise this shape you will see it everywhere in systems design.

Choosing

```
  Need to wake an event loop?              eventfd
  Parent/child, one direction, a stream?   pipe
  Bidirectional, local, need fd passing?   Unix socket (socketpair)
  Message boundaries matter?               SOCK_SEQPACKET or datagram
  Megabytes of data?                       shared memory + a ring
  Across machines?                         TCP, or a real RPC framework
  Unrelated processes, need naming?        named socket or shm_open
  Structured API with versioning?          D-Bus, gRPC
  Absolute lowest latency?                 shared memory + busy poll

Default answer for local IPC on Linux: a Unix domain socket. It is
bidirectional, ordered, has a sane API, passes credentials and file
descriptors, works with epoll, and cleans up when the process dies.
Reach for shared memory only when you have measured that it matters.
```

Debugging IPC

```
# what is open, and connected to what
lsof -p PID
ls -l /proc/PID/fd/
ss -xp                          # Unix sockets and their peers
lsof -U

# System V leftovers
ipcs -a

# shared mappings
cat /proc/PID/maps | grep -E 'shm|SYSV|memfd'
cat /proc/PID/smaps             # Shared_Clean / Shared_Dirty per mapping

# trace the traffic
strace -e trace=read,write,sendmsg,recvmsg -p PID
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_futex { @[comm] = count(); }'

# a hang: where is each side blocked?
cat /proc/PID/stack
cat /proc/PID/wchan
gdb -p PID -batch -ex 'thread apply all bt'
```

How it shows up

Symptom	Cause
Reader hangs, never gets EOF	Some process still holds the write end open
Process dies silently on write	`SIGPIPE` — the reader closed; ignore it and handle `EPIPE`
Messages arrive merged or split	Stream semantics; add length framing
Writer blocks forever	Pipe full and the reader is not consuming — a flow-control bug
Deadlock with two pipes	Both sides blocked writing while neither reads; use non-blocking + `poll`
Shared-memory data occasionally corrupt	No synchronisation or wrong memory ordering
Peer crashed and the lock is stuck	Non-robust process-shared mutex
`SIGBUS` reading shared memory	Sender truncated it; use `memfd` seals

Symptom	Cause
System V objects accumulate after restarts	They persist beyond process lifetime; use POSIX instead
Loopback TCP much slower than expected	Full protocol stack; use a Unix socket

Check yourself

- ✓ Give the seven questions that determine which IPC mechanism to use.
- ✓ Explain why a pipe reader may never see EOF, and the rule that prevents it.
- ✓ Explain why pipes do not preserve message boundaries and what to do about it.
- ✓ Describe `SCM_RIGHTS` and name two systems that depend on it.
- ✓ Say what shared memory does *not* provide, and two things you must add.
- ✓ Explain robust mutexes and the failure they address.
- ✓ Explain how a futex avoids syscalls in the uncontended case.
- ✓ Describe the ring-buffer-plus-doorbell pattern and name three systems using it.

Synchronisation primitives

Shared mutable state plus concurrency equals corruption, unless something enforces order. This chapter covers the primitives, what each one is actually for, and — the part that matters most in kernel work — which are legal in which execution context.

The problem, precisely

```
/* A shared counter, incremented by two threads. */
counter++;

/* which is really three operations (C chapter 01, CA chapter 09) */
        ldr  w1, [x0]        /* LOAD  */
        add  w1, w1, #1      /* MODIFY */
        str  w1, [x0]        /* STORE */

/* Interleaved:
     T1: load  → 5
     T2: load  → 5
     T1: add   → 6
     T2: add   → 6
     T1: store → 6
     T2: store → 6          one increment LOST                */
```

THREE PROPERTIES YOU MAY NEED, AND THEY ARE SEPARATE

- **Atomicity** — the operation completes indivisibly. Nothing observes a half-done state.
- **Mutual exclusion** — at most one thread in a region at a time.
- **Ordering** — other threads observe operations in the intended order (CA ch. 15).

Atomics give atomicity and, with the right memory order, ordering. Locks give all three. Neither is a substitute for the other, and the classic mistake is assuming that "atomic" implies "correct".

Atomic operations

```
#include <stdatomic.h>

atomic_int counter = 0;
atomic_fetch_add(&counter, 1);              /* indivisible */

/* the primitive everything else is built from */
int expected = old;
if (atomic_compare_exchange_strong(&v, &expected, desired)) {
        /* v was `old`, now it is `desired` */
} else {
        /* v was not `old`; expected now holds the actual value */
}

/* the CAS loop — the shape of every lock-free algorithm */
int old, new;
do {
        old = atomic_load(&v);
        new = f(old);                       /* any computation */
} while (!atomic_compare_exchange_weak(&v, &old, new));
/* _weak may fail spuriously but is cheaper in a loop; _strong
   does not, and is right when there is no loop.              */

/* Check what is actually lock-free on your target: */
atomic_is_lock_free(&obj);
ATOMIC_INT_LOCK_FREE            /* 2 = always, 1 = sometimes, 0 = never */
/* A "lock-free" atomic on a large struct may be implemented with a
   hidden mutex — which is a correctness surprise in a signal
   handler or an interrupt context.                            */
```

Memory order	Guarantee	Use for
`relaxed`	Atomicity only	Counters and statistics nobody synchronises on
`acquire`	Later accesses cannot move before it	Reading a flag that guards data
`release`	Earlier accesses cannot move after it	Setting a flag after writing data
`acq_rel`	Both, for read-modify-write	Lock acquire/release, reference counts that free
`seq_cst`	Plus one global total order	The safe default; the most expensive

Mutexes

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);
        /* critical section */
pthread_mutex_unlock(&m);

/* Properties that matter:
     · OWNERSHIP — only the locker may unlock. This is what allows
       priority inheritance and deadlock detection.
     · SLEEPS on contention (via futex — OS chapter 08)
     · uncontended cost ~20 ns; contended cost ~1–5 µs        */

/* Types */
PTHREAD_MUTEX_NORMAL        /* self-deadlock on recursive lock   */
PTHREAD_MUTEX_ERRORCHECK    /* returns EDEADLK instead — use in
                               debug builds                      */
PTHREAD_MUTEX_RECURSIVE     /* same thread may relock. Usually a
                               design smell: it hides unclear
                               ownership of the invariant.       */
PTHREAD_MUTEX_ADAPTIVE_NP   /* spin briefly, then sleep          */
```

```
/* Always release on every path. In C, the goto ladder or the
   cleanup attribute (C chapters 11 and 14):                    */
static inline void unlock_p(pthread_mutex_t **m) { pthread_mutex_unlock(*m); }
#define LOCKED(mp) \
        pthread_mutex_t *_l __attribute__((cleanup(unlock_p))) = (mp); \
        pthread_mutex_lock(_l)

void f(void)
{
        LOCKED(&m);
        if (error) return;          /* unlocked automatically */
        ...
}                                   /* unlocked automatically */

/* Linux kernel 6.5+ has the same idea, type-safe: */
guard(mutex)(&lock);
scoped_guard(spinlock_irqsave, &lock) { ... }
```

Reader-writer locks

```
pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;

pthread_rwlock_rdlock(&rw);     /* many readers concurrently */
pthread_rwlock_unlock(&rw);

pthread_rwlock_wrlock(&rw);     /* exclusive */
pthread_rwlock_unlock(&rw);
```

READER-WRITER LOCKS ARE USUALLY SLOWER THAN YOU EXPECT

They are only a win when reads are both *frequent* and *long*. For short critical sections the extra bookkeeping — and the cache-line contention on the shared reader count, which every reader must write — often costs more than a plain mutex. They also introduce a writer-starvation policy question that the implementation answers for you, usually badly.

Measure before choosing one. If reads dominate and are short, RCU or a seqlock is almost always the better answer.

Condition variables

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  cv = PTHREAD_COND_INITIALIZER;
bool ready = false;

/* WAITER — note the loop, which is not optional */
pthread_mutex_lock(&m);
while (!ready)                          /* while, NEVER if */
        pthread_cond_wait(&cv, &m);     /* atomically unlocks m,
                                           sleeps, relocks on wake */
consume();
pthread_mutex_unlock(&m);

/* SIGNALLER */
pthread_mutex_lock(&m);
ready = true;
pthread_mutex_unlock(&m);
pthread_cond_signal(&cv);               /* or _broadcast for all */
```

ALWAYS `WHILE`, NEVER `IF`

Three independent reasons the predicate can be false after a wake-up:

1. **Spurious wake-ups** are explicitly permitted by POSIX and do occur.
2. **Another waiter got there first** and consumed the item — with `broadcast`, all waiters wake but only one can proceed.
3. **The condition changed again** between the signal and your reacquiring the mutex.

Using `if` produces a bug that appears under load, months later, and is nearly impossible to reproduce.

```
/* The three rules for condition variables:
     1. always associated with a mutex, and the predicate is
        protected by that mutex
     2. wait in a while loop on the predicate
     3. hold the mutex when changing the predicate

   Signal inside or outside the lock? Either is correct.
   Inside is simpler to reason about; outside avoids the waiter
   waking and immediately blocking on the mutex you still hold.
   Modern implementations optimise both cases.                */
```

Semaphores

```
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 3);             /* 0 = threads only, 3 permits */

sem_wait(&s);                   /* take a permit, or block */
        /* at most 3 threads here */
sem_post(&s);                   /* return a permit */

/* Unlike a mutex, a semaphore has NO OWNER: any thread may post.
   That makes it the right tool for:
     · limiting concurrency to N (a resource pool)
     · signalling between threads (post from one, wait in another)
     · counting available items in a producer/consumer queue

   And the wrong tool for mutual exclusion — a mutex's ownership
   is what enables priority inheritance and error checking.    */

/* sem_post IS async-signal-safe, which makes semaphores the one
   correct way to wake a thread from a signal handler.        */
```

Kernel context: the rule that dominates

> **CAN THIS CODE SLEEP?**
>
> Everything about kernel synchronisation follows from this one question. If you are in a context that cannot sleep — an interrupt handler, holding a spinlock, or above DISPATCH_LEVEL on Windows — then any primitive that might block is a bug. It will deadlock, or trigger "scheduling while atomic".

```
   Does this code need to sleep while holding the lock?
   (kmalloc(GFP_KERNEL), copy_to_user, I/O, another mutex)
        │
        ├── YES ──► MUTEX  (Linux)  /  FAST_MUTEX, ERESOURCE (Windows)
        │           process context only; never in an ISR
        │
        └── NO ───► is the data ALSO touched by an interrupt handler?
                    │
                    ├── NO ──► spin_lock()
                    │
                    └── YES ─► spin_lock_irqsave()
                               disables interrupts on THIS core, so
                               the handler cannot deadlock against you

   Special cases:
     read-mostly, rare writes    → RCU
     a simple counter or flag    → atomic_t, no lock at all
     waiting for an event        → wait_event() + wake_up()
     one-shot completion         → struct completion
     read-mostly, short reads    → seqlock
```

```
/* Linux */
spinlock_t lock;
unsigned long flags;

spin_lock_irqsave(&lock, flags);        /* save, disable, lock   */
        /* short, non-sleeping critical section ONLY */
spin_unlock_irqrestore(&lock, flags);   /* unlock, restore state */

/* Why save-and-restore rather than plain disable/enable: if
   interrupts were ALREADY disabled by an outer critical section,
   unconditionally re-enabling them at the inner unlock is a
   serious bug.                                                  */

/* Windows equivalents */
KIRQL old;
KeAcquireSpinLock(&lock, &old);          /* raises to DISPATCH_LEVEL */
KeReleaseSpinLock(&lock, old);
KeAcquireSpinLockAtDpcLevel(&lock);      /* already at DISPATCH     */
```

Linux	Windows	Sleeps?	Context
`spinlock_t`	`KSPIN_LOCK`	No	Any
`mutex`	`FAST_MUTEX`	Yes	Process only
`rw_semaphore`	`ERESOURCE`	Yes	Process only
`completion`	`KEVENT`	Yes (waiter)	Wait: process. Signal: any.
`atomic_t`	`Interlocked*`	No	Any
RCU	—	Readers: no	Any

RCU and seqlocks

```
/* RCU: Read-Copy-Update. Readers pay ZERO synchronisation cost —
   no atomics, no cache-line contention, no barriers on most
   architectures. Writers do all the work.                    */

/* reader */
rcu_read_lock();                       /* just disables preemption */
struct foo *p = rcu_dereference(gp);
if (p) use(p->field);
rcu_read_unlock();

/* writer */
struct foo *new = kmalloc(...);
*new = *old;                           /* COPY */
new->field = value;                    /* UPDATE the copy */
rcu_assign_pointer(gp, new);           /* publish, with a release */
synchronize_rcu();                     /* wait until every pre-existing
                                          reader has finished       */
kfree(old);                            /* now provably safe */

/* The insight: instead of preventing readers from seeing a torn
   update, never modify in place. Readers see either the old
   object or the new one, both consistent. Reclamation is
   deferred until no reader can hold a reference.

   Ideal for: routing tables, module lists, configuration —
   anything read constantly and updated rarely.
   Bad for: frequent writes (each one allocates and waits).   */
```

```
/* Seqlock: readers retry if a write occurred during their read.
   Very cheap reads, but readers must be side-effect free.     */

/* reader */
unsigned seq;
do {
        seq = read_seqbegin(&sl);
        a = shared.x;  b = shared.y;    /* may read a torn pair */
} while (read_seqretry(&sl, seq));      /* ...and retry if so   */

/* writer */
write_seqlock(&sl);
shared.x = 1;  shared.y = 2;
write_sequnlock(&sl);

/* Used for timekeeping: readers vastly outnumber writers and must
   never block a writer. Note readers can starve under constant
   writing, and must not have side effects since they may run
   several times.                                              */
```

Rules that prevent most bugs

Rule	Prevents
Document what each lock protects, next to its declaration	Ambiguous ownership — the root of most locking bugs
Acquire multiple locks in one global order, always	Deadlock (ch. 10)
Keep critical sections short	Contention, and latency spikes

Rule	Prevents
Never call unknown code holding a lock	Lock-order inversion via a callback
Never sleep holding a spinlock	Deadlock, "scheduling while atomic"
Never return from a function holding a lock it acquired	Unbalanced locking; use scope guards
Prefer no sharing, then immutable sharing, then locks	The whole problem class
One lock per data structure, not per function	Locks that protect nothing coherent

Finding synchronisation bugs

```
# ThreadSanitizer — finds races that have not yet manifested.
# The highest-value tool in this chapter.
gcc -fsanitize=thread -g prog.c && ./prog

# lock order and race detection without rebuilding
valgrind --tool=helgrind ./prog
valgrind --tool=drd ./prog

# contention
perf lock record ./prog && perf lock report
mutrace ./prog

# a hang: get every thread's stack
gdb -p PID -batch -ex 'thread apply all bt'

# kernel
CONFIG_PROVE_LOCKING=y          # lockdep: proves lock ORDER correctness
                                # across the whole kernel, and reports a
                                # potential deadlock the FIRST time an
                                # inconsistent order is observed —
                                # before it ever deadlocks
CONFIG_DEBUG_ATOMIC_SLEEP=y     # catches sleeping in atomic context
CONFIG_KCSAN=y                  # data race detector
```

> **LOCKDEP IS THE SINGLE BEST THING ABOUT KERNEL DEVELOPMENT**
> It builds a graph of every lock acquisition order observed at runtime and reports a cycle the first time one appears — even if the deadlock did not actually occur. A potential deadlock that would surface once a year in production becomes a splat in `dmesg` on your first test run. Always develop with it enabled.

How it shows up

Symptom	Cause
Counter is occasionally wrong	Unsynchronised read-modify-write
Works with 2 threads, fails with 16	A race whose window widened
Wakes up but the condition is false	`if` instead of `while` around `cond_wait`
Hang with all threads in `pthread_mutex_lock`	Deadlock — get all backtraces
"scheduling while atomic" / bug check 0xA	Slept in an interrupt or spinlock context
Adding a reader-writer lock made it slower	Short critical sections; the bookkeeping dominates
Performance collapses past N threads	Lock contention; the cliff is one context switch pair

Symptom	Cause
Interrupts stay disabled after unlock	Used `spin_unlock_irq` where `irqrestore` was needed
Fails on ARM, passes on x86	Missing memory ordering, hidden by TSO
Crash freeing an object another thread is reading	Missing RCU grace period or reference count

Check yourself

- ✓ Name the three properties concurrent code may need and say which primitives provide each.
- ✓ Write a CAS loop and explain `_weak` versus `_strong`.
- ✓ Give three reasons a condition-variable predicate must be rechecked in a loop.
- ✓ Explain the difference between a mutex and a semaphore, and when ownership matters.
- ✓ Give the decision procedure for choosing a kernel lock.
- ✓ Explain why `spin_lock_irqsave` saves the previous state.
- ✓ Explain how RCU makes readers free, and what writers pay instead.
- ✓ Say what lockdep proves and why it finds deadlocks that never happened.

Classic concurrency problems and deadlock

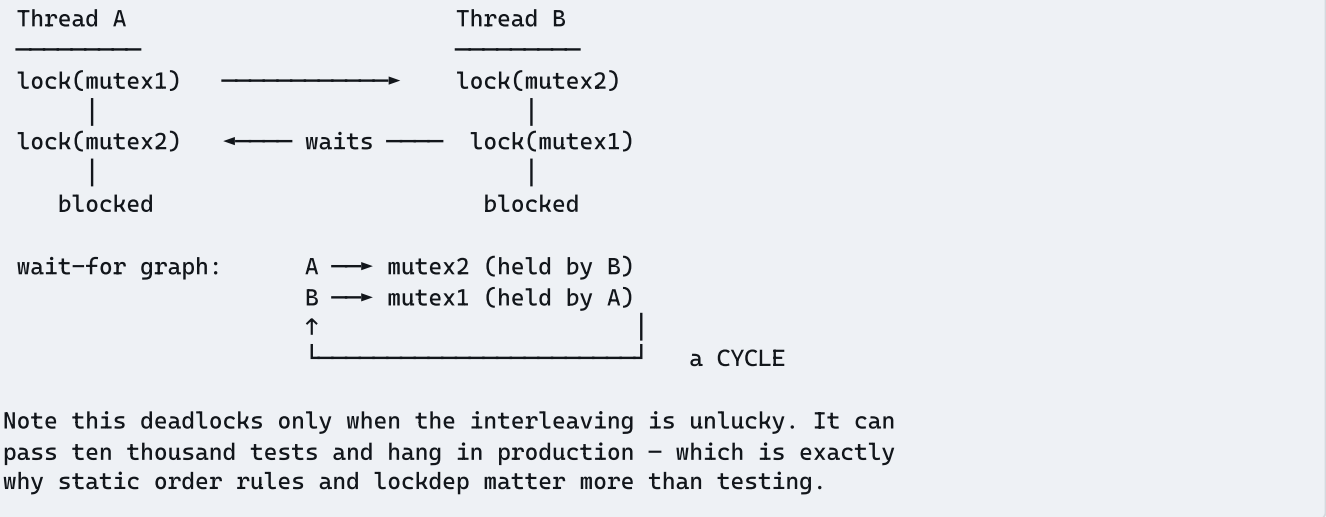
Deadlock has exactly four necessary conditions, and breaking any one of them prevents it. The classic textbook problems are not academic exercises — each is a real pattern you will meet, and each has a standard solution worth knowing by name.

The four conditions

COFFMAN'S CONDITIONS — ALL FOUR MUST HOLD SIMULTANEOUSLY

1. **Mutual exclusion** — a resource is held exclusively.
2. **Hold and wait** — a thread holds one resource while waiting for another.
3. **No preemption** — a resource cannot be forcibly taken away.
4. **Circular wait** — a cycle exists in the wait-for graph.

Break *any* one and deadlock becomes impossible. Every prevention technique below is an attack on one of these four.




```
Note this deadlocks only when the interleaving is unlucky. It can
pass ten thousand tests and hang in production — which is exactly
why static order rules and lockdep matter more than testing.
```

Breaking each condition

Attack	Technique	Practicality
Mutual exclusion	Lock-free structures, immutable data, per-thread copies	Best when applicable — removes the problem entirely
Hold and wait	Acquire all locks at once, or none. Release everything before retrying.	Often impractical: you rarely know the full set in advance
No preemption	`trylock` + back off, releasing what you hold	Workable, but risks livelock without randomised backoff
Circular wait	**A global lock order, obeyed everywhere**	**The standard solution in real systems**

```
/* LOCK ORDERING — the practical answer. Define a total order and
   document it where the locks are declared.                    */

/*
 * Lock order (acquire in this order, release in reverse):
 *      1. device->config_lock
 *      2. device->queue_lock
 *      3. buffer->lock
 */

/* When the order is dynamic — two objects of the same type — order
   by ADDRESS. Any consistent total order works.               */
void transfer(struct account *a, struct account *b, int amount)
{
        struct account *first  = (a < b) ? a : b;
        struct account *second = (a < b) ? b : a;

        if (a == b) return;             /* do not self-deadlock */

        pthread_mutex_lock(&first->lock);
        pthread_mutex_lock(&second->lock);
        a->balance -= amount;
        b->balance += amount;
        pthread_mutex_unlock(&second->lock);
        pthread_mutex_unlock(&first->lock);
}
/* Ordering by address is the standard trick; the kernel's
   double_lock_balance() and Linux's inode locking both do it.  */
```

```
/* TRYLOCK AND BACK OFF — when ordering is impossible */
for (;;) {
        pthread_mutex_lock(&m1);
        if (pthread_mutex_trylock(&m2) == 0)
                break;                          /* got both      */
        pthread_mutex_unlock(&m1);              /* release ALL   */
        usleep(rand() % 100);                    /* random backoff */
}
/* The random backoff is not optional: without it, two threads can
   repeatedly grab and release in lockstep forever — LIVELOCK.
   They are running, consuming CPU, and making no progress.    */
```

Detection and recovery

```
/* Rather than preventing deadlock, detect it and break the cycle. */

/* 1. Timeouts — crude but effective in application code */
struct timespec ts;
clock_gettime(CLOCK_REALTIME, &ts);
ts.tv_sec += 5;
if (pthread_mutex_timedlock(&m, &ts) == ETIMEDOUT) {
        release_everything();
        return -EAGAIN;                 /* retry from the top */
}

/* 2. Error-checking mutexes catch self-deadlock immediately */
pthread_mutexattr_settype(&a, PTHREAD_MUTEX_ERRORCHECK);
if (pthread_mutex_lock(&m) == EDEADLK) ...

/* 3. lockdep (kernel) — builds the lock-order graph at runtime and
      reports a cycle the FIRST time an inconsistent order is seen,
      whether or not it deadlocked. This is the gold standard.   */

/* Databases do full cycle detection in the wait-for graph and
   abort one transaction as the "victim" — which is why you must
   handle deadlock-retry errors in any transactional code.     */
```

Livelock and starvation

	Deadlock	Livelock	Starvation
Threads are	Blocked	Running	Running or blocked
CPU usage	Zero	**High**	Normal
Progress	None, ever	None, ever	Some threads progress; one never does
Cause	Circular wait	Repeated mutual back-off	Unfair scheduling or lock policy
Fix	Lock ordering	Randomised backoff, or a queue	Fair/ticket locks, aging, priority boost

```
/* Starvation from an unfair lock: a thread on the same core keeps
   reacquiring before a waiter on another core is scheduled.  */

/* A TICKET LOCK gives strict FIFO fairness: */
typedef struct { atomic_uint next, now; } ticket_lock_t;

void lock(ticket_lock_t *l) {
        unsigned my = atomic_fetch_add(&l->next, 1);
        while (atomic_load_explicit(&l->now, memory_order_acquire) != my)
                cpu_relax();
}
void unlock(ticket_lock_t *l) {
        atomic_fetch_add_explicit(&l->now, 1, memory_order_release);
}
/* Fair, but every waiter spins on the SAME cache line — the
   coherence traffic scales badly (CA chapter 15). MCS and qspinlock
   fix that by giving each waiter its own line to spin on; Linux
   uses qspinlock for exactly this reason.                    */
```

Producer–consumer

```
/* The most common real pattern: one side generates, the other
   consumes, with a bounded buffer in between.                */

#define N 16
struct item buf[N];
size_t head, tail, count;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full  = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void produce(struct item it)
{
        pthread_mutex_lock(&m);
        while (count == N)                      /* while, not if */
                pthread_cond_wait(&not_full, &m);
        buf[head] = it;
        head = (head + 1) % N;
        count++;
        pthread_mutex_unlock(&m);
        pthread_cond_signal(&not_empty);
}

struct item consume(void)
{
        pthread_mutex_lock(&m);
        while (count == 0)
                pthread_cond_wait(&not_empty, &m);
        struct item it = buf[tail];
        tail = (tail + 1) % N;
        count--;
        pthread_mutex_unlock(&m);
        pthread_cond_signal(&not_full);
        return it;
}

/* TWO condition variables, not one. With a single cv and
   broadcast, a producer can wake another producer, which finds
   the buffer still full and sleeps again — wasted wake-ups, and
   in the signal (non-broadcast) case, a lost wake-up that hangs
   the system.                                                  */
```

Readers–writers

```
Three policies, and the trade-off is unavoidable:

  READER PREFERENCE    a new reader may enter while others read
                       → maximum read throughput
                       → WRITERS CAN STARVE indefinitely

  WRITER PREFERENCE    a waiting writer blocks new readers
                       → writers always progress
                       → readers can starve under heavy writing

  FAIR (queued)        requests served in arrival order
                       → nobody starves
                       → lower throughput than reader preference

  glibc's pthread_rwlock defaults to reader preference; the
  PTHREAD_RWLOCK_PREFER_WRITER_NONRECURSIVE_NP attribute changes it.
  Knowing which you have matters: a monitoring thread reading a
  structure in a tight loop can permanently block the writer.
```

Dining philosophers

```
   Five philosophers, five forks, each needs both neighbours' forks.

        P0
     F4    F0
   P4        P1
     F3    F1
        P2  F2  P3

  Naive: everyone picks up their LEFT fork, then their RIGHT.
  If all five do it simultaneously, each holds one fork and waits
  forever. All four Coffman conditions, in five lines of code.

  SOLUTIONS, each attacking a different condition:

  1. RESOURCE ORDERING     number the forks; always take the
                           lower-numbered one first.
                           → breaks circular wait. THE standard fix.

  2. LIMIT CONCURRENCY     a semaphore allowing only 4 diners at once.
                           → at least one can always get both forks;
                             breaks hold-and-wait in effect.

  3. ASYMMETRY             odd philosophers take left first,
                           even take right first.
                           → breaks the cycle. Same idea as (1).

  4. ATOMIC ACQUISITION    take both forks or neither, under a
                           global lock.
                           → breaks hold-and-wait. Simple, but
                             serialises everything.

  The point of the problem: it is the minimal example of resource
  ordering, and (1) is what you will actually use.
```

ABBA deadlock across layers

```
/* The dangerous version in real systems: the cycle spans two
   subsystems, and neither author can see the other's order.  */

/* Path A: filesystem operation */
lock(inode->i_mutex);
        lock(page->lock);        /* fs → mm */

/* Path B: memory reclaim */
lock(page->lock);
        lock(inode->i_mutex);    /* mm → fs */

/* Neither file's code looks wrong in isolation. This is why:
     · lock ordering must be documented GLOBALLY, not per-module
     · callbacks are dangerous: calling unknown code while holding
       a lock lets the callee acquire locks you did not expect
     · lockdep finds these, and almost nothing else does      */

/* The general rule: NEVER call into another subsystem, or into a
   caller-supplied callback, while holding a lock.            */
```

Priority inversion, again

Covered in chapter 06, but it belongs in the deadlock family: a high-priority thread is blocked indefinitely, not by a cycle but by a scheduling interaction. The fix — priority inheritance — is a property of the *lock*, which is why an RTOS mutex differs from a plain one, and why `PTHREAD_PRIO_INHERIT` exists.

Avoiding locks entirely

Guarantee	Meaning
Blocking	A suspended thread can block others indefinitely
Obstruction-free	A thread running alone completes
Lock-free	*Some* thread always makes progress — no deadlock possible
Wait-free	*Every* thread completes in bounded steps — the strongest, and rarest

```
/* A lock-free stack push */
void push(struct node *n)
{
        struct node *old;
        do {
                old = atomic_load(&head);
                n->next = old;
        } while (!atomic_compare_exchange_weak(&head, &old, n));
}

/* Pop is much harder - the ABA problem: between reading head and
   the CAS, another thread could pop A, pop B, and push A back.
   Your CAS succeeds because the pointer matches, but the list
   behind it is now wrong.

   Solutions: tagged pointers (a counter in the unused bits),
   hazard pointers, epoch-based reclamation, or RCU.

   The lesson: lock-free code is genuinely hard and easy to get
   subtly wrong. Use a reviewed library, or RCU, unless you have
   measured that lock contention is the actual bottleneck.   */
```

Debugging a hang

```
# 1. Is it deadlock or livelock? Look at CPU usage.
top -H -p PID          # 0% = deadlock;  100% = livelock or spin

# 2. Get every thread's stack. This usually answers it outright.
gdb -p PID -batch -ex 'thread apply all bt'
eu-stack -p PID
py-spy dump --pid PID              # Python
jstack PID                          # Java - reports deadlocks directly

# 3. What is each thread waiting on?
cat /proc/PID/task/*/stack          # kernel side
cat /proc/PID/task/*/wchan
ls -l /proc/PID/task/*/status | head

# 4. Reproduce it deliberately
valgrind --tool=helgrind ./prog     # lock order violations
gcc -fsanitize=thread               # races and some lock issues

# 5. Kernel
dmesg | grep -A40 'possible circular locking dependency'
# lockdep prints the exact two acquisition orders and the stacks
# that established them. It is nearly always sufficient.
```

> **THE FIRST COMMAND TO RUN ON ANY HANG**
>
> `thread apply all bt`. Two threads blocked in `pthread_mutex_lock` on different mutexes, each holding the other's, is unmistakable in the output. Most hangs are diagnosed in under a minute by that one command — before reaching for any specialised tool.

How it shows up

Symptom	Cause
Hang at 0% CPU, all threads blocked	Deadlock; get all backtraces
Hang at 100% CPU, no progress	Livelock, or spinning on a lock never released
One thread never progresses; others fine	Starvation; use a fair lock or aging
Hangs once a week under load	An order violation on a rare code path — enable lockdep or Helgrind
Hangs only after adding a callback	The callee took a lock, creating a cycle across layers
Deadlock between filesystem and memory code	Classic ABBA across subsystems
Self-deadlock on re-entering a function	Non-recursive mutex; fix the design, do not make it recursive
Writer never runs while readers are busy	Reader-preference rwlock
Lock-free stack corrupts under load	ABA problem; needs tagging or safe reclamation
Transaction aborts with "deadlock detected"	The database broke the cycle; retry the transaction

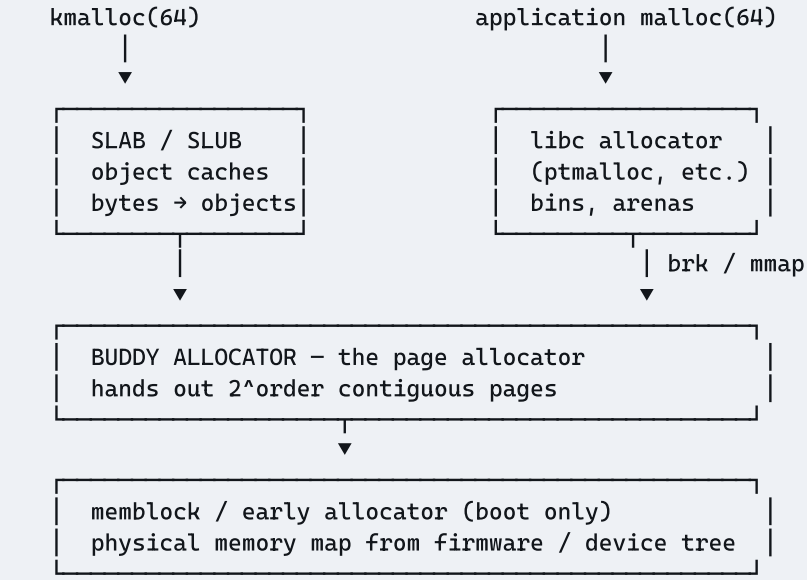
Check yourself

- ✓ State the four Coffman conditions and one way to break each.
- ✓ Explain why lock ordering is the practical solution, and how to order two objects of the same type.
- ✓ Explain why trylock-and-retry needs randomised backoff.
- ✓ Distinguish deadlock, livelock and starvation by CPU usage and cause.
- ✓ Explain why producer–consumer needs two condition variables.
- ✓ Give the three readers–writers policies and who starves under each.
- ✓ Give two solutions to dining philosophers and which condition each breaks.
- ✓ Explain ABBA deadlock across subsystems and the rule that prevents it.
- ✓ Explain the ABA problem and name two mitigations.

Physical memory management and allocators

Before virtual memory can hand out address spaces, something must track which physical pages are free. This chapter covers that layer — the page allocator, the object allocators built on it, and fragmentation, which is the problem all of them exist to fight.

The allocator stack



Each layer solves the layer above's granularity problem: the buddy allocator deals in pages, but most allocations are tens of bytes.

The buddy allocator

```
Free lists, one per ORDER, where order n means 2^n contiguous pages.

  order 0:  4 KB      order 3:  32 KB     order 6:  256 KB
  order 1:  8 KB      order 4:  64 KB     order 7:  512 KB
  order 2: 16 KB      order 5: 128 KB     ... up to order 10 (4 MB)

ALLOCATION of order 2, when only an order-4 block is free:

   order 4:  [================ 64 KB ================]
                        split
   order 3:  [====== 32 KB ======][====== 32 KB ======]
                  split                  ↑ freed to order-3 list
   order 2:  [= 16 =][= 16 =]
              return   ↑ freed to order-2 list

FREEING: when a block is released, check whether its BUDDY — the
adjacent block of the same size, found by flipping one address bit —
is also free. If so, merge them and repeat upward.

   buddy_address = block_address XOR (block_size)

That single XOR is why the scheme is fast: finding the buddy is one
arithmetic operation, not a search.

  + coalescing is automatic and cheap
  + finding a block is O(1) amortised
  - INTERNAL fragmentation: a 5-page request takes an order-3
    block (8 pages), wasting 3
  - EXTERNAL fragmentation: many scattered order-0 blocks cannot
    satisfy an order-5 request even with plenty free
```

```
# see it live
cat /proc/buddyinfo
# Node 0, zone   Normal   1024   512   256   64   16   4   1   0   0   0   0
#                         ^order0 ^1    ^2   ...                   ^order10
# Lots of low-order blocks and zeros at high orders means the memory
# is FRAGMENTED: plenty free, but no large contiguous runs.

cat /proc/pagetypeinfo     # per migrate-type breakdown
cat /proc/zoneinfo         # free pages, watermarks, per zone
```

Zones and NUMA nodes

Zone	Range	Exists because
`ZONE_DMA`	First 16 MB (x86)	Ancient ISA devices could only address 24 bits
`ZONE_DMA32`	First 4 GB	Devices with 32-bit DMA addressing
`ZONE_NORMAL`	The rest	General use
`ZONE_HIGHMEM`	Above the kernel's direct map	32-bit only; gone on 64-bit
`ZONE_MOVABLE`	Pseudo-zone	Pages that can be migrated — enables hotplug and defragmentation

```
/* Zones are why allocation can fail with memory apparently free:
   a device needing DMA32 memory cannot use pages above 4 GB.
   This is exactly what dma_set_mask() communicates.          */

/* NUMA: on multi-socket systems, memory attached to another
   socket costs 1.5-2x the latency. The allocator prefers the
   local node.                                                */
numactl --hardware              # node distances and free memory
numastat -p PID                 # where THIS process's pages live
cat /proc/PID/numa_maps
```

Slab allocators

```
The problem: kmalloc(64) from a 4 KB page allocator wastes 98%.
Plus, initialising a complex object is expensive if repeated.

The slab idea (Bonwick, 1994):
  · allocate a PAGE (or several), carve it into equal-size objects
  · keep a free list threaded through the unused objects
  · allocation = pop the list; free = push it. O(1), no search.
  · objects of one TYPE share a cache, so they stay hot in CPU cache

   ┌──────────── one slab (4 KB page) ────────────┐
   │ obj0 | obj1 | obj2 | obj3 | ... | objN | meta │
   └──┬──┴──────┬──┴──────────────────────────────┘
      └── free ──┘  a singly-linked list through free objects

Linux has had three implementations:
  SLAB   original; per-CPU and per-node queues, complex
  SLUB   the current default; simpler, better multicore scaling,
         less metadata, better debugging
  SLOB   tiny, for very small systems (removed in 6.4)
```

```
/* generic sizes - kmalloc uses power-of-two caches */
void *p = kmalloc(100, GFP_KERNEL);      /* comes from kmalloc-128,
                                            so 28 bytes are wasted */

/* a dedicated cache, when you allocate one type constantly */
static struct kmem_cache *my_cache;

my_cache = kmem_cache_create("my_objects",
                             sizeof(struct my_object),
                             0,                     /* alignment  */
                             SLAB_HWCACHE_ALIGN,    /* flags      */
                             NULL);                 /* constructor */

struct my_object *o = kmem_cache_alloc(my_cache, GFP_KERNEL);
kmem_cache_free(my_cache, o);
kmem_cache_destroy(my_cache);

/* Benefits over kmalloc: exact sizing (no power-of-two waste),
   a named entry in /proc/slabinfo for leak hunting, per-type
   locality, and optional constructors.                       */
```

```
# which caches are consuming memory?
sudo slabtop -s c              # sorted by cache size
cat /proc/slabinfo

# a steadily growing cache with no corresponding workload growth
# is a kernel memory leak, and slabtop names the object type —
# which usually identifies the subsystem immediately.
```

Fragmentation

Kind	Meaning	Mitigation
Internal	Allocated more than requested; the remainder is unusable	Size-class tuning, dedicated slab caches
External	Enough total free memory, but not contiguous	Compaction, migration, avoiding large allocations

```
External fragmentation, made concrete

   memory:  [used][free][used][free][used][free][used][free]
              4KB   4KB   4KB   4KB   4KB   4KB   4KB   4KB

   16 KB free in total. An order-2 (16 KB contiguous) request FAILS.

Linux's defences:

  MIGRATE TYPES     group pages by movability, so unmovable kernel
                    allocations do not scatter through regions that
                    would otherwise be compactable
                      MIGRATE_UNMOVABLE   kernel data, page tables
                      MIGRATE_MOVABLE     user pages, page cache
                      MIGRATE_RECLAIMABLE caches that can be dropped

  COMPACTION        physically MOVE movable pages to create
                    contiguous free runs, updating page tables and
                    reverse mappings as it goes
                      echo 1 > /proc/sys/vm/compact_memory

  CMA               reserve a region at boot that is normally used
                    for movable pages but can be reclaimed on demand
                    for large contiguous DMA buffers — cameras,
                    video codecs, GPUs

  This is why kmalloc of large sizes can fail on a long-running
  system, and why kvmalloc() exists: try kmalloc (contiguous),
  fall back to vmalloc (virtually contiguous only).           */
```

vmalloc versus kmalloc

	`kmalloc`	`vmalloc`
Physically contiguous	**Yes**	No
Virtually contiguous	Yes	Yes
Usable for DMA	**Yes**	**No**
Max practical size	A few MB, and fails under fragmentation	Large — limited by the vmalloc area
Cost	Fast	Slower: page table setup, TLB pressure
Can sleep	Depends on GFP flag	Always — never in atomic context

```
void *p = kvmalloc(size, GFP_KERNEL);   /* try kmalloc, fall back
                                           to vmalloc              */
kvfree(p);
/* Use this whenever the buffer is large and does NOT need to be
   physically contiguous. It is the right default for most
   variable-size kernel allocations.                              */
```

Reclaim and the OOM killer

```
Watermarks per zone drive reclaim:

   free pages
      |
   HIGH  ──────  above this: no reclaim needed
      |
   LOW   ──────  kswapd wakes and reclaims in the BACKGROUND
      |
   MIN   ──────  allocators reclaim SYNCHRONOUSLY (direct reclaim)
      |          - the allocating task now pays the cost, and
      |            latency spikes
      0          allocation fails → OOM killer

What gets reclaimed, cheapest first:
  1. clean page cache          just drop it, it is on disk
  2. clean anonymous pages     only if swap exists
  3. dirty page cache          must write back first
  4. dirty anonymous pages     swap out
  5. slab caches               shrinkers: dentry, inode, fs caches

  echo 3 > /proc/sys/vm/drop_caches     # drop clean caches (testing)
  vm.swappiness                          # anon vs file cache bias
  vm.min_free_kbytes                     # raise for network/RT work
                                         # to avoid direct reclaim
```

```
# when reclaim fails: the OOM killer picks a victim by oom_score
cat /proc/PID/oom_score
echo -1000 > /proc/PID/oom_score_adj    # never kill this process
dmesg | grep -A20 'Out of memory'

# In containers, the cgroup limit triggers a per-cgroup OOM long
# before the machine is out of memory:
cat /sys/fs/cgroup/.../memory.max
cat /sys/fs/cgroup/.../memory.events    # oom_kill counter
```

READING "FREE" CORRECTLY

```
free -h
#               total   used   free   shared  buff/cache  available
# Mem:            16G    4G     1G      200M        11G         11G
#                            ^^^      low, and PERFECTLY NORMAL
#                                                      ^^^
#                                          THIS is what matters

/* "free" memory is wasted memory — Linux uses everything spare for
   page cache, which it can drop instantly. Look at AVAILABLE, which
   accounts for reclaimable cache. Alarm on `free` being low is the
   single most common Linux memory misdiagnosis.                 */
```

The user-space side

```
/* malloc does not call the kernel for every allocation. */

brk / sbrk        grow the heap contiguously. Simple, but memory is
                  only returned to the OS from the TOP — one live
                  allocation at the end pins everything below it.

mmap              anonymous mappings. glibc uses this for requests
                  above MMAP_THRESHOLD (128 KB by default), and
                  returns them to the OS immediately on free.

/* Which explains a common observation: freeing a lot of memory
   sometimes reduces RSS and sometimes does not. Small allocations
   from the brk heap are recycled internally; large mmap'd ones
   go straight back.                                          */

mallopt(M_MMAP_THRESHOLD, 64*1024);
malloc_trim(0);                  /* ask glibc to return free pages */
MALLOC_ARENA_MAX=2 ./program     /* limit per-thread arenas —
                                    often cuts RSS substantially in
                                    highly threaded programs      */
```

Embedded systems

```
/* No MMU means no virtual memory, so physical fragmentation is
   permanent and unrecoverable. This is why (C chapter 14):     */

/* Static allocation — the linker proves it fits at BUILD time */
static struct session sessions[MAX_SESSIONS];

/* Fixed-size pools — O(1), no fragmentation, bounded */
static struct msg pool[64];
static uint64_t  used;

/* TLSF (Two-Level Segregated Fit) — a real-time heap with
   BOUNDED worst-case allocation time, unlike a general malloc */

/* FreeRTOS ships five heap implementations, and choosing between
   them is a real design decision:
     heap_1  allocate only, never free — safest
     heap_2  free supported, no coalescing — fragments
     heap_4  coalescing — the usual choice
     heap_5  heap_4 across multiple memory regions            */
```

How it shows up

Symptom	Cause
Allocation fails with plenty of free memory	Fragmentation, or a zone constraint (DMA32)
`kmalloc` of 1 MB fails after weeks of uptime	External fragmentation; use `kvmalloc`
Latency spikes under memory pressure	Direct reclaim; raise `vm.min_free_kbytes`
Process killed with plenty of RAM shown as cached	cgroup limit, not machine memory
"free" shows almost no free memory	Page cache — read the *available* column
A slab cache grows without bound	Kernel leak; `slabtop` names the object type

Symptom	Cause
RSS does not fall after freeing memory	brk heap cannot return non-top blocks; try `malloc_trim`
Threaded server uses far more RSS than expected	Per-thread malloc arenas; set `MALLOC_ARENA_MAX`
DMA buffer allocation fails on an embedded board	No CMA reservation, or the region is exhausted
Device fails after weeks, works after reboot	Fragmentation in a no-MMU system; use pools

Check yourself

- ✓ Draw the allocator stack and say what problem each layer solves.
- ✓ Explain buddy allocation, including how the buddy address is computed.
- ✓ Explain what `/proc/buddyinfo` tells you about fragmentation.
- ✓ Say why zones exist and give a case where allocation fails despite free memory.
- ✓ Explain the slab idea and two advantages of a dedicated cache over `kmalloc`.
- ✓ Distinguish internal from external fragmentation and name Linux's three defences against the latter.
- ✓ Give three differences between `kmalloc` and `vmalloc`.
- ✓ Explain the watermarks and why direct reclaim causes latency spikes.
- ✓ Explain why low "free" memory on Linux is normal.

Virtual memory, paging and page replacement

CA chapter 16 covered the hardware mechanism. This chapter is the software policy: which pages live in RAM, which are evicted, and how the OS makes a page fault into a feature rather than an error.

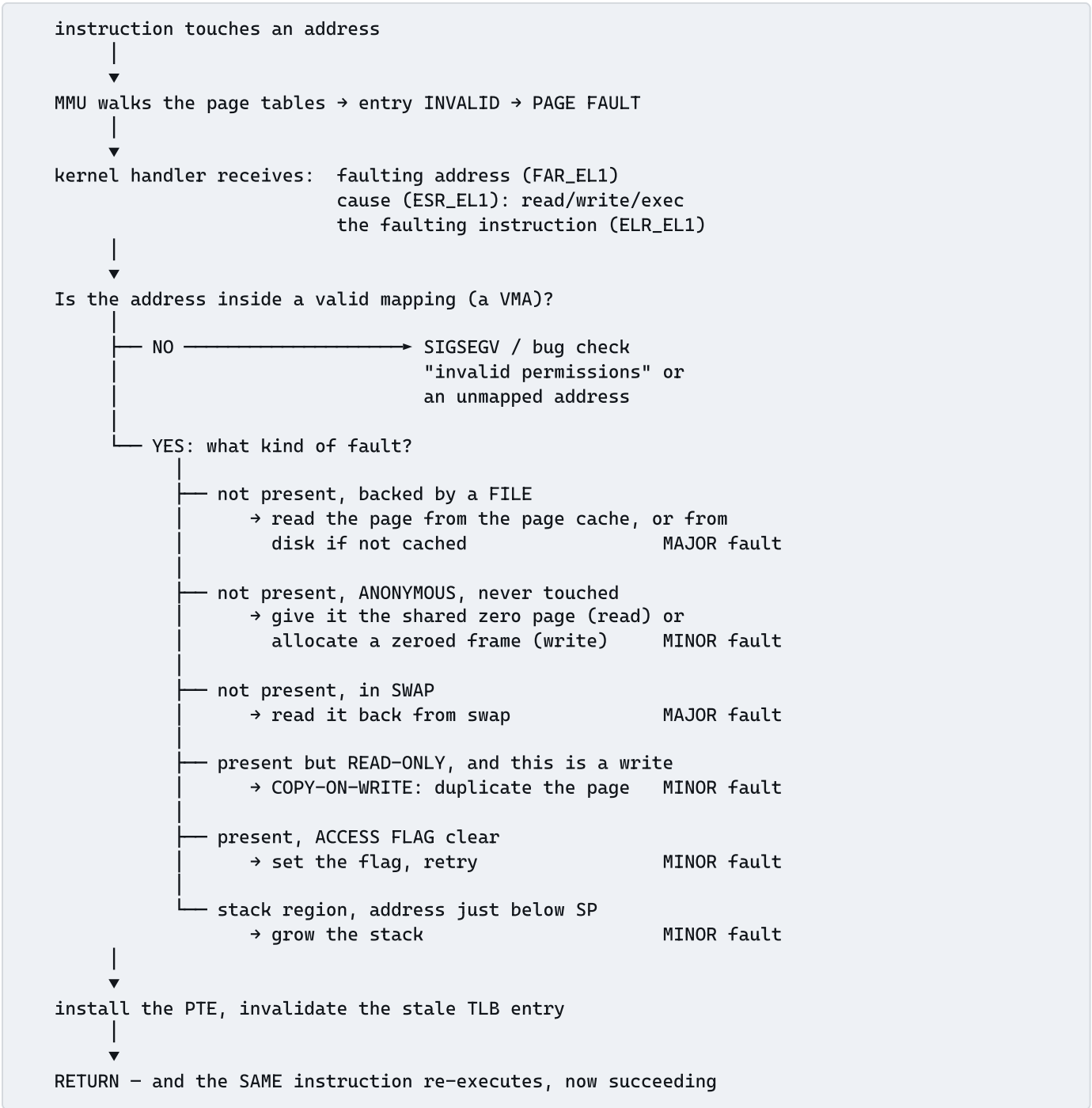
Demand paging

THE IDEA

Do not load anything until it is touched. Mark pages invalid, and let the hardware tell you — via a page fault — exactly which page a program actually needs. Programs typically use a small fraction of what they map, so this converts a large up-front cost into a series of small on-demand ones.

```
Executing a 200 MB binary

  WITHOUT demand paging: read 200 MB from disk, then start.
                         Slow to start, and most of it is never used.

  WITH demand paging:    map the file, start immediately.
                         Fault in ~2 MB of actually-executed code.

  This is also why:
    · a huge malloc() returns instantly — nothing is allocated yet
    · a program's RSS is far smaller than its VSZ
    · the first run of a program is slower than the second
      (the second finds the pages in the page cache)
```

The page fault handler



	Minor fault	Major fault
Disk I/O	None	**Yes**
Cost	~1–5 µs	~100 µs (SSD) to ~10 ms (disk)
Causes	First touch, COW, access-flag, page already in cache	Reading from a file or from swap
Acceptable in real time?	Marginal	**Never**

```
# count them
/usr/bin/time -v ./program | grep -i faults
#   Major (requiring I/O) page faults: 12
#   Minor (reclaiming a frame) page faults: 45231

ps -o min_flt,maj_flt,cmd -p PID
perf stat -e page-faults,major-faults ./program
sar -B 1                      # system-wide fault and paging rates
```

How the kernel knows what should be there

```
/* The page tables say what IS mapped. A separate structure says
   what SHOULD be mappable — that is what the fault handler
   consults.                                                    */

struct vm_area_struct {          /* one per contiguous mapping */
        unsigned long vm_start, vm_end;
        unsigned long vm_flags;          /* READ|WRITE|EXEC|SHARED */
        struct file  *vm_file;           /* backing file, or NULL
                                            for anonymous          */
        unsigned long vm_pgoff;          /* offset within the file */
        const struct vm_operations_struct *vm_ops;   /* ->fault()  */
};
/* Kept in a maple tree (formerly a red-black tree) per mm_struct,
   so "which VMA contains this address?" is O(log n).          */

/* Windows: the same idea, called a Virtual Address Descriptor. */
```

```
cat /proc/self/maps
# 5584f2a00000-5584f2a02000 r-xp 00000000 fe:01 1234  /usr/bin/cat
# 5584f2c01000-5584f2c02000 rw-p 00001000 fe:01 1234  /usr/bin/cat
# 5584f4b1e000-5584f4b3f000 rw-p 00000000 00:00 0     [heap]
# 7ffd8a3c0000-7ffd8a3e1000 rw-p 00000000 00:00 0     [stack]
#      ^start        ^end   ^perms ^offset ^dev ^inode ^backing
#                   p = private (COW),  s = shared

cat /proc/self/smaps          # per-VMA: Rss, Pss, Shared/Private,
                              # Dirty/Clean, THP usage, swap
```

Page replacement policy

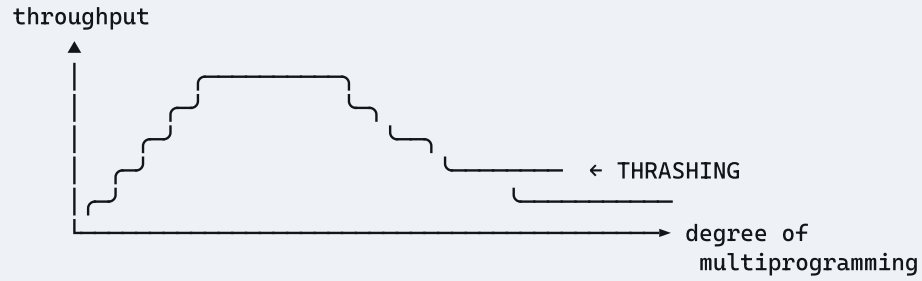
When memory is full, which page is evicted? The theoretically optimal choice — the page used furthest in the future — requires knowing the future, so every real policy approximates it.

Policy	Rule	Problem
OPT / Belady	Evict the page used furthest in the future	Unimplementable; used only as a benchmark
FIFO	Evict the oldest	Ignores usage. Suffers *Belady's anomaly*: more frames can mean *more* faults.
LRU	Evict the least recently used	Good, but exact LRU needs a timestamp on every access — far too expensive in hardware
Clock / second chance	Circular scan; if the access bit is set, clear it and move on	Cheap LRU approximation. The classic implementation.

Policy	Rule	Problem
Two-list (Linux)	Active and inactive LRU lists; promotion on second reference	Resists scan pollution
LFU / ARC / MGLRU	Track frequency as well as recency	More state; better under mixed workloads

```
Linux's two-list approach, and the problem it solves

   ACTIVE list      pages referenced more than once — the working set
       ▲   │
       │   │ aging pushes down when the active list grows too large
       │   ▼
   INACTIVE list    candidates for eviction
           │
           ▼ evicted (clean: dropped; dirty: written back first)

  A page enters the INACTIVE list on first touch. It is promoted to
  ACTIVE only on a SECOND reference.

  Why: a large sequential scan — `cat bigfile`, a backup — touches
  millions of pages exactly once. With plain LRU those pages would
  evict your entire working set. With two lists they never reach
  the active list, and the working set survives. This is called
  scan resistance, and it is why `grep -r /` does not destroy your
  machine's performance for the next ten minutes.

  MGLRU (Multi-Gen LRU, Linux 6.1+) generalises this to several
  generations, with much lower overhead at scale.
```

Thrashing and the working set




```
  Beyond a point, adding processes means no process's working set
  fits. Every process faults, the disk saturates, and CPU
  utilisation COLLAPSES while the system appears maximally busy.

  The classic failure signature:
     · load average very high
     · CPU mostly in iowait
     · si/so columns in vmstat non-zero and large
     · everything, including your shell, responds in seconds

  WORKING SET: the set of pages a process has touched in the last
  T units of time. If the sum of all working sets exceeds RAM,
  thrashing is inevitable regardless of policy.

  Responses: reduce concurrency, add memory, use cgroup limits so
  one process cannot drag the system down, or enable PSI-based
  pressure handling.
```

```
# detect it
vmstat 1
#  r  b   swpd   free  buff  cache   si   so    bi    bo  cs us sy id wa
#              ^^^^                  ^^   ^^                          ^^
#   si/so = swap in/out per second. Sustained non-zero = thrashing.
#   wa    = iowait. High wa with low us/sy means waiting on disk.

# PSI — the modern, direct measure of pressure (Linux 4.20+)
cat /proc/pressure/memory
# some avg10=45.23 avg60=30.12 avg300=12.45
#      ^ % of time at least one task was stalled on memory
# This is far more actionable than free/available, and cgroup-aware.
```

Swap

Question	Answer
What is swapped?	Only **anonymous** pages. File-backed pages are simply dropped and re-read from the file.
Is swap bad?	No. Swapping *out* idle pages to make room for cache is good. Swapping *in* continuously is thrashing.
Should I disable it?	Usually not — without swap the kernel cannot evict idle anonymous memory at all, so it evicts *file* pages instead, hurting more. Do disable it for hard real-time and for latency-critical databases with their own caching.
`vm.swappiness`	0–200; the bias toward reclaiming anonymous versus file pages. Default 60. Lower means prefer dropping cache.
zram / zswap	Compressed swap in RAM. Excellent on memory-constrained devices — CPU is cheaper than I/O.

```
swapon --show
cat /proc/PID/status | grep VmSwap        # how much THIS process has swapped
smem -s swap -r                           # per-process swap usage

# real-time and latency-critical work: pin everything resident
mlockall(MCL_CURRENT | MCL_FUTURE);
# ...and check RLIMIT_MEMLOCK allows it. Without this, a single
# major fault can blow a millisecond deadline.
```

Huge pages

```
/* TLB reach (CA chapter 16): 1024 entries x 4 KB = 4 MB.
   With 2 MB pages the same TLB covers 2 GB.                  */

/* Transparent Huge Pages — automatic, no code change */
cat /sys/kernel/mm/transparent_hugepage/enabled
#   [always] madvise never
madvise(p, len, MADV_HUGEPAGE);      /* opt in, per region */

/* Explicit huge pages — reserved, guaranteed, never swapped */
echo 1024 > /proc/sys/vm/nr_hugepages
mmap(NULL, len, PROT_READ|PROT_WRITE,
     MAP_PRIVATE|MAP_ANONYMOUS|MAP_HUGETLB, -1, 0);
```

> **THP IS NOT FREE**
> Allocating a 2 MB huge page requires 2 MB of *contiguous* physical memory. Under fragmentation the kernel may enter synchronous compaction to find it, stalling the faulting process for milliseconds. Databases (MongoDB, Redis, Oracle) routinely recommend `transparent_hugepage=madvise` or `never` for exactly this reason — the latency variance is worse than the TLB benefit. Explicit `hugetlbfs` pages, reserved at boot, avoid the problem entirely.

Controlling paging from user space

```
madvise(p, len, MADV_WILLNEED);   /* prefetch: fault it in now */
madvise(p, len, MADV_DONTNEED);   /* discard: anonymous pages are
                                     freed and re-fault as ZERO   */
madvise(p, len, MADV_SEQUENTIAL); /* aggressive readahead         */
madvise(p, len, MADV_RANDOM);     /* disable readahead            */
madvise(p, len, MADV_FREE);       /* may be reclaimed lazily -
                                     faster than DONTNEED         */
madvise(p, len, MADV_HUGEPAGE);
madvise(p, len, MADV_COLD);       /* deprioritise, do not free    */

posix_fadvise(fd, off, len, POSIX_FADV_DONTNEED);   /* drop from cache */
posix_fadvise(fd, off, len, POSIX_FADV_WILLNEED);

mlock(p, len);                    /* pin: never evicted           */
mlockall(MCL_CURRENT | MCL_FUTURE);
mmap(..., MAP_POPULATE, ...);     /* prefault the whole mapping   */
```

Handling faults in user space

```
/* userfaultfd lets a user process handle its own page faults.
   The mechanism behind live VM migration, distributed shared
   memory, and CRIU checkpoint/restore.                        */

int uffd = syscall(SYS_userfaultfd, O_CLOEXEC);
/* register a region; a helper thread reads fault events from uffd
   and supplies pages with UFFDIO_COPY. The faulting thread blocks
   until the page arrives - post-copy migration in a nutshell. */
```

How it shows up

Symptom	Cause
First run slow, second run fast	Major faults on the first; page cache hits on the second
`malloc` of 10 GB succeeds instantly, crashes later	Lazy allocation plus overcommit; memory arrives on first touch
RSS far below VSZ	Normal — most mapped pages are never touched
System unresponsive, CPU mostly iowait	Thrashing; check `vmstat` si/so and PSI
Millisecond latency spike in a tight loop	Major fault; use `mlockall`
Backup job destroys database performance	Scan pollution — mitigated by two-list LRU, and by `POSIX_FADV_DONTNEED`
Random multi-millisecond stalls on a busy server	THP compaction; set `transparent_hugepage=madvise`
Disabling swap made things worse	The kernel now evicts file cache instead of idle anonymous pages

Symptom	Cause
Memory freed but RSS unchanged	`MADV_FREE` defers reclamation, or the allocator kept it
Real-time task misses a deadline hourly	An unlocked page faulted; `mlockall(MCL_FUTURE)`

Check yourself

- ✓ Explain demand paging and three observable consequences of it.
- ✓ Walk the page fault handler's decision tree and name six distinct fault causes.
- ✓ Distinguish minor from major faults by cost and cause.
- ✓ Explain what a VMA is and why the page tables alone are insufficient.
- ✓ Explain Belady's anomaly and which policy suffers it.
- ✓ Explain Linux's two-list LRU and the specific problem it solves.
- ✓ Describe thrashing, its signature in `vmstat`, and three responses.
- ✓ Explain why disabling swap can make memory pressure worse.
- ✓ Give two reasons THP can hurt latency, and the mitigation.

Memory-mapped files and copy-on-write

Two applications of the same trick: make the page tables lie, and let a fault fix things up on demand. Together they explain why `fork()` is cheap, why shared libraries cost one copy of memory system-wide, and why `mmap` is sometimes far faster than `read` and sometimes much slower.

Mapping a file into memory

```
int fd = open("data.bin", O_RDONLY);
struct stat st;
fstat(fd, &st);

void *p = mmap(NULL,            /* let the kernel choose the address */
               st.st_size,      /* length                            */
               PROT_READ,       /* protection                        */
               MAP_PRIVATE,     /* private (COW) or MAP_SHARED       */
               fd, 0);          /* file and offset (page-aligned)    */
close(fd);                      /* the MAPPING keeps the file alive  */

/* the file's bytes are now simply memory */
uint32_t magic = *(uint32_t *)p;
for (size_t i = 0; i < st.st_size; i++)
        checksum += ((uint8_t *)p)[i];

munmap(p, st.st_size);
```

```
What actually happened:

  mmap() created a VMA and returned. NO data was read.
  Every page is marked invalid in the page tables.

  The first access to each page faults:
     · kernel looks up the VMA → this is file-backed at offset N
     · finds the page in the PAGE CACHE, or reads it from disk
     · installs a PTE pointing at that page-cache page
     · retries the instruction

  Consequences:
     · the mapping is the SAME physical memory as the page cache,
       so no copy is made
     · two processes mapping the same file share those frames
     · a 100 GB file can be mapped on a machine with 8 GB of RAM
     · reading sequentially triggers readahead, exactly as read()
       would
```

The four combinations

	File-backed	Anonymous
MAP_PRIVATE	Copy-on-write view of a file. Writes are private and never reach disk. *Used for: executables, shared libraries.*	Ordinary private memory. *Used for:* `malloc` *of large blocks, thread stacks.*
MAP_SHARED	Writes go to the file and are visible to every other mapper. *Used for: databases, IPC over a file.*	Shared memory between related processes. *Used for:* `fork` *-based IPC (ch. 08).*

```
/* MAP_PRIVATE on a file is where the two ideas meet: */
void *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_PRIVATE, fd, 0);
p[0] = 'X';
/* The page was shared read-only with the page cache. Writing it
   triggers a COW fault: the kernel allocates a fresh frame, copies
   the contents, and remaps THIS process's PTE to the copy.
   The file on disk is untouched. Other mappers see the original. */
```

Copy-on-write

```
   BEFORE fork()
   parent PTEs ──────►  [ page A ]  [ page B ]  [ page C ]
                          rw           rw          rw

   AFTER fork() — no data copied, only page TABLES
   parent PTEs ──┐
                 ├────►  [ page A ]  [ page B ]  [ page C ]
   child  PTEs ──┘         RO,cow      RO,cow     RO,cow
                           refcount 2  refcount 2 refcount 2

   Both mappings are marked READ-ONLY, with a flag saying "this is
   COW, not genuinely read-only".

   CHILD WRITES page B
        │ write to a read-only page → PAGE FAULT
        ▼
   kernel: the VMA says writable, and the PTE says COW
        · allocate a new frame
        · copy page B into it
        · point the CHILD's PTE at the copy, mark it rw
        · decrement the original's refcount to 1
        · since only the parent now maps it, restore ITS PTE to rw
        · retry the instruction

   parent PTEs ──────►  [ page A ]  [ page B  ]  [ page C ]
                          RO,cow       rw          RO,cow
   child  PTEs ──────►  [ page A ]  [ page B' ]  [ page C ]
                          RO,cow       rw          RO,cow

  Only the pages actually written are ever duplicated. A process
  that forks and immediately execs copies almost nothing.
```

Where else COW appears

Use	How
`fork()`	The canonical case
Shared libraries	Text is shared read-only; the data segment is COW, so each process gets private globals from one on-disk copy
The zero page	All freshly allocated anonymous pages map one shared zero page read-only until written
KSM	Kernel Samepage Merging: identical pages across VMs are merged and made COW
Filesystem snapshots	btrfs, ZFS, LVM — the same idea at block granularity
Container images	overlayfs: the lower layer is shared, writes go to an upper layer

THE FORK MEMORY-ACCOUNTING CONFUSION

After `fork()`, `ps` shows both processes with the same RSS — but they share almost all of it, so the machine's real usage barely changed. RSS double-counts shared pages. Use **PSS** (Proportional Set Size), which divides each shared page's size by the number of sharers:

```
grep -H '^Pss:' /proc/*/smaps_rollup | sort -k2 -n -r | head
smem -k -s pss                    # per-process, sorted by PSS
/* PSS is the only per-process number that sums to something
   meaningful across all processes.                          */
```

Why COW is security-sensitive

```
/* Dirty COW (CVE-2016-5195) — a race between the COW fault handler
   and madvise(MADV_DONTNEED), exploitable to write to a read-only
   file mapping, giving local root on essentially every Linux
   system for nine years.

   The lesson is general: COW is a place where the page tables
   deliberately disagree with the VMA's stated permissions, and
   the window between fault and fixup is a race that must be
   handled with great care. Several later CVEs (Dirty Pipe,
   CVE-2022-0847) follow the same shape.                     */
```

mmap versus read/write

	`mmap`	`read` / `write`
Copies	Zero — you touch page-cache pages directly	One copy, cache → your buffer
Syscalls	One, then page faults	One per call
Random access	**Excellent** — just a pointer	Needs `lseek` + `read` per access
Sequential streaming	Often *slower* — fault per page	**Better** — large reads amortise everything
Small files	Setup and teardown dominate	Better
Error handling	`SIGBUS` on I/O error — hard to handle	`-EIO` return value
File growth	Must re-map; risk of `SIGBUS` past EOF	Just works
Memory pressure	Counts against the process; can be reclaimed under you	Bounded by your buffer

THE HONEST SUMMARY

`mmap` is a win for **random access to large files** and for **sharing**. It is frequently a loss for sequential streaming, because a page fault every 4 KB costs more than one `read()` of 1 MB. Linus Torvalds has made this point repeatedly, and benchmarks bear it out. Use `mmap` when you would otherwise be seeking; use `read` with a large buffer when you are streaming.

```
/* If you do use mmap, tell the kernel your access pattern */
madvise(p, len, MADV_SEQUENTIAL);   /* aggressive readahead, drop behind */
madvise(p, len, MADV_RANDOM);       /* disable readahead                 */
madvise(p, len, MADV_WILLNEED);     /* prefault now                      */
mmap(..., MAP_POPULATE, ...);       /* prefault the whole mapping        */

/* Handling SIGBUS is genuinely awkward: it fires on a truncated
   file or an I/O error, from anywhere the mapping is touched. In
   robust code, either avoid mmap for untrusted files, or install
   a handler and use siglongjmp - which is its own problem.    */
```

Zero-copy I/O

```
/* The general goal: avoid copying data through user space at all. */

/* sendfile: file → socket, entirely in the kernel */
sendfile(out_fd, in_fd, &offset, count);
/* saves two copies and two context switches versus read+write.
   This is how static web servers serve files.                */

/* splice: move data between fds via a pipe, without copying */
splice(fd_in, NULL, pipefd[1], NULL, len, SPLICE_F_MOVE);
splice(pipefd[0], NULL, fd_out, NULL, len, SPLICE_F_MOVE);

/* vmsplice: map user pages into a pipe */
/* copy_file_range: server-side copy, may be offloaded to the
   filesystem or even the storage device (reflink on btrfs/XFS) */
copy_file_range(fd_in, &off_in, fd_out, &off_out, len, 0);

/* io_uring: registered buffers plus fixed files remove even the
   per-operation syscall (OS chapter 08).                      */
```

Direct access and persistent memory

```
/* DAX: for byte-addressable persistent memory, the page cache is
   pointless - map the device's pages directly.                */
mount -o dax /dev/pmem0 /mnt

/* A store instruction now writes persistent storage. Which raises
   a genuinely new problem: ordering and durability at CPU cache
   granularity. You need explicit flushes:                     */
    dc cvap, x0        /* clean to point of persistence (ARM)   */
    clwb               /* cache line write back (x86)           */
    sfence
/* Getting this wrong gives torn, non-atomic updates that survive
   a power cut - a corruption class that did not exist before. */
```

Observing mappings

```
cat /proc/PID/maps               # all VMAs
cat /proc/PID/smaps              # per-VMA detail:
#   Rss, Pss, Shared_Clean/Dirty, Private_Clean/Dirty,
#   Referenced, Anonymous, Swap, THPeligible
cat /proc/PID/smaps_rollup       # totals — much cheaper to parse

pmap -x PID                      # readable summary
pmap -XX PID                     # everything

# is this page shared, and with whom?
sudo ./page-types -p PID         # from linux/tools/vm

# how much of the page cache does a file occupy?
vmtouch -v /path/to/file
fincore /path/to/file

# force a file out of the cache, for cold benchmarking
dd of=/path/to/file oflag=nocache conv=notrunc,fdatasync count=0
```

How it shows up

Symptom	Cause
RSS appears to double after `fork`	Shared COW pages counted twice; read PSS
`mmap` slower than `read` for streaming	One fault per page versus one large read
`SIGBUS` touching a mapping	File truncated under you, or an I/O error
Writes to a `MAP_PRIVATE` file mapping vanish	Correct — private mappings never write back
Two processes mapping the same file do not see each other's writes	Used `MAP_PRIVATE`; need `MAP_SHARED`
Memory usage grows when reading a large file	Page cache; reclaimable, not a leak
First write after `fork` is slow	COW fault plus a page copy
Shared library uses one copy of text, many of data	Text shared, data segment is COW — as designed
Data corrupted after power loss on pmem	Missing cache flush to the point of persistence
Container image uses less disk than the sum of layers	overlayfs COW at file granularity

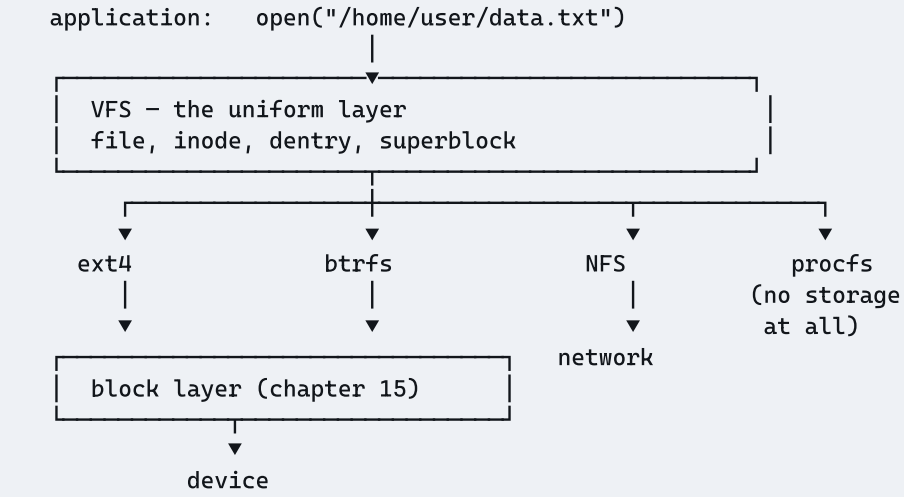
Check yourself

- ✓ Explain what `mmap` does at call time versus at first touch.
- ✓ Give the four private/shared × file/anonymous combinations and a use for each.
- ✓ Walk through a COW fault step by step, including what happens to the last sharer.
- ✓ Explain why RSS is misleading after `fork` and what to use instead.
- ✓ Name four mechanisms besides `fork` that use copy-on-write.
- ✓ Say when `mmap` beats `read` and when it does not.
- ✓ Explain why `SIGBUS` makes `mmap` error handling hard.
- ✓ Name three zero-copy mechanisms and what each avoids copying.

File systems

A filesystem turns a flat array of blocks into named, persistent, hierarchical objects — and must survive power loss halfway through any operation. That durability requirement is what makes filesystem design hard, and what every design below is really answering.

What a filesystem provides




```
The VFS is why `cat` works on a disk file, a network mount, a pipe
and /proc/cpuinfo without knowing the difference.
```

The four VFS objects

Object	Represents	Lifetime
`struct super_block`	A mounted filesystem	From mount to unmount
`struct inode`	A file's *metadata*: size, permissions, timestamps, block pointers. **Not the name.**	Cached; one per file
`struct dentry`	One path component and its inode — the name-to-inode binding	Cached aggressively (the dcache)
`struct file`	An *open* file: the current offset, mode, and the `f_op` table	Per `open()`

NAMES LIVE IN DIRECTORIES, NOT IN FILES

An inode has no name. A **directory** is a file whose contents map names to inode numbers. This single design decision explains hard links (several names, one inode), why `rename()` within a filesystem is instantaneous (it edits directory entries only), why deleting a file with an open descriptor does not free the space (the inode's link count reached zero but its *open* count did not), and why you cannot hard-link across filesystems (inode numbers are only meaningful within one).

```
ls -li                  # the leading number is the INODE number
stat file.txt           # inode, links, size, blocks, timestamps
                        # note: Links: 2 means two names for this inode

ln  file.txt hard       # another NAME for the same inode
ln -s file.txt soft     # a small file whose CONTENTS are a path

df -i                   # inode usage — a filesystem can run out of
                        # inodes while blocks remain free, which is a
                        # confusing "No space left on device" with
                        # df showing plenty of space

# a deleted-but-open file still consumes space:
lsof +L1                # files with link count 0 that are still open
ls -l /proc/PID/fd/     # shows "(deleted)"
```

On-disk layout

```
A classic UNIX-style filesystem (ext4 simplified)

  +-----------+-------------+-------------+---------------------+
  | superblock| inode bitmap| inode table |     data blocks     |
  |           | block bitmap|             |                     |
  +-----------+-------------+-------------+---------------------+

  superblock   size, block size, free counts, mount state, UUID.
               Replicated, because losing it loses everything.
  bitmaps      which inodes and blocks are free
  inode table  fixed-size records, indexed by inode number
  data blocks  file contents and directory contents

  ext4 divides all of this into BLOCK GROUPS so metadata sits near
  the data it describes, reducing seeks on rotational media.

How an inode finds its data:

  small file    12 direct block pointers            → up to 48 KB
  larger        1 single-indirect (a block of ptrs) → +4 MB
                1 double-indirect                    → +4 GB
                1 triple-indirect                    → +4 TB

  ext4 replaces most of this with EXTENTS: (start, length) pairs.
  One extent can describe 128 MB, so a large contiguous file needs
  a handful of records instead of millions of pointers.
```

Crash consistency: the real problem

```
Appending a block to a file requires THREE updates:

   1. mark the block used in the block bitmap
   2. add the block pointer to the inode
   3. update the inode's size

Power fails between any two:

   after 1 only  → a block is leaked. Wasteful, not dangerous.
   after 2 only  → the inode points at a block marked FREE.
                   It will be handed to another file. Two files now
                   share a block: SILENT CORRUPTION.
   after 3 only  → size claims data that was never written; the
                   file contains whatever was previously in that
                   block — possibly another user's data. A
                   security bug, not just a correctness one.

There is no ordering of three independent writes that is safe,
because the disk may reorder them and may not complete any of them.
Every solution below adds a mechanism to make a MULTI-BLOCK update
appear ATOMIC.
```

Approach	Mechanism	Cost	Used by
fsck	Repair inconsistencies after the fact by scanning everything	Hours on a large filesystem; may lose data	ext2, FAT
Journalling	Write intent to a log first; replay it after a crash	Metadata written twice	ext3/4, XFS, NTFS
Copy-on-write	Never overwrite; write new blocks and atomically switch the root pointer	Fragmentation; more space	btrfs, ZFS, APFS
Log-structured	The whole filesystem is an append-only log	Needs garbage collection	F2FS, NILFS, and every SSD's FTL internally
Soft updates	Carefully order writes so any prefix is consistent	Extremely intricate to implement	FFS on BSD

```
/* Journalling modes in ext4 — the trade-off made explicit */

data=journal      /* both data AND metadata to the journal first.
                     Safest, and roughly halves write throughput.  */
data=ordered      /* DEFAULT. Metadata journalled; data blocks are
                     forced to disk BEFORE the metadata that refers
                     to them commits. Prevents the "size claims
                     unwritten data" case above.                   */
data=writeback    /* metadata journalled, data unordered. Fastest;
                     after a crash a file can contain stale blocks
                     — including another file's old contents.      */

mount -o data=ordered,barrier=1 /dev/sda1 /mnt
/* barrier=1 (the default) issues cache flushes so the DEVICE does
   not reorder the journal commit ahead of the data. Turning it off
   is fast and unsafe, and is a very common cause of "journalling
   did not protect me".                                          */
```

fsync and what "written" means

```
write(fd, buf, n);      /* returns as soon as the data is in the
                           PAGE CACHE. Nothing has reached disk. */

fsync(fd);              /* force this file's data AND metadata out,
                           and flush the device's write cache      */
fdatasync(fd);          /* data plus only the metadata needed to
                           read it back — skips timestamp updates,
                           so it is measurably faster              */
sync();                 /* everything, everywhere. Returns before
                           completion on some systems.             */

/* The durable-rename idiom — the standard way to update a file
   atomically. Every step matters.                              */
fd = open("file.tmp", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, data, len);
fsync(fd);                          /* 1. contents are durable    */
close(fd);
rename("file.tmp", "file");         /* 2. atomic swap of the name */
dirfd = open(".", O_RDONLY);
fsync(dirfd);                       /* 3. the RENAME is durable   */
close(dirfd);

/* Skipping step 3 is the classic bug: the rename is a directory
   metadata change, and until the directory is synced a crash can
   leave you with neither the old file nor the new one.        */
```

EVERY LAYER HAS A CACHE THAT CAN LIE

Application buffer → libc `stdio` buffer → kernel page cache → device write cache → platters or flash. `fflush()` only empties the second. `fsync()` pushes through the third and asks the fourth to flush — but consumer drives have been known to acknowledge a flush without performing one, and a virtualisation layer or a RAID controller with a non-battery-backed cache can do the same. If durability genuinely matters, test it: write, cut power, verify.

Filesystems worth knowing

Filesystem	Design	Use it for
ext4	Journalled, extents, block groups	The safe Linux default
XFS	Journalled, B+trees, allocation groups, excellent parallelism	Large files, many threads, big storage
btrfs	COW, snapshots, checksums, integrated RAID, compression	Snapshots and data integrity
ZFS	COW, checksums everywhere, pooled storage, self-healing	Data you cannot lose; licence keeps it out of the kernel
F2FS	Log-structured, flash-aware	Phones, embedded eMMC/SD
UBIFS / JFFS2	Operate on *raw NAND*, doing their own wear levelling	Embedded devices with bare flash and no FTL
SquashFS	Read-only, compressed	Firmware images, live systems, container layers
tmpfs	Page cache with no backing store	`/tmp`, `/run`, `/dev/shm`
overlayfs	Union: read-only lower + writable upper, COW per file	Containers, live images

Filesystem	Design	Use it for
FAT / exFAT	No journal, no permissions, universally supported	Interchange only — never for anything that matters
procfs / sysfs / debugfs	No storage; files are generated on read	Kernel interfaces (ch. 16)

Flash storage changes the rules

```
NAND flash cannot overwrite. It can only:
   · READ a page          (4–16 KB)
   · PROGRAM a page       (once, after erase)
   · ERASE a BLOCK        (128 KB – 4 MB) — and blocks wear out
                            after ~1,000–100,000 erase cycles

So a "write 4 KB at offset X" becomes:
   read the whole block → erase it → reprogram it with the change
   = WRITE AMPLIFICATION, and it consumes erase cycles.

TWO PLACES this is solved:

  MANAGED FLASH (SSD, eMMC, SD, UFS)
      an on-device Flash Translation Layer does log-structured
      writes, wear levelling, garbage collection and bad-block
      management, and presents a normal block device.
      → use ext4, XFS, F2FS. Issue TRIM/discard so the FTL knows
        which blocks are free:  mount -o discard, or fstrim weekly.

  RAW NAND (embedded, no FTL)
      the FILESYSTEM must do all of that itself.
      → UBI + UBIFS, or JFFS2. Using ext4 on raw NAND will destroy
        the flash and lose data.

Either way: the number of WRITES matters more than their size, and
excessive logging or a chatty database can wear out an embedded
device's flash in months.
```

Performance

```
# the dentry cache is why repeated path lookups are free
cat /proc/sys/fs/dentry-state
slabtop | grep -E 'dentry|inode'

# and why a `find /` inflates memory — it populates the dcache
echo 2 > /proc/sys/vm/drop_caches    # drop dentries and inodes

# readahead
blockdev --getra /dev/sda
posix_fadvise(fd, 0, 0, POSIX_FADV_SEQUENTIAL);

# many small files are dominated by metadata, not data
#   1 file  of 1 GB   : ~1 open, sequential reads
#   1M files of 1 KB  : ~1M opens, 1M inode reads, 1M dentry lookups
# This is why extracting a source tarball is slower than copying a
# disk image of the same size, and why build systems are IOPS-bound.

# measure
iostat -x 1
fio --name=test --rw=randread --bs=4k --numjobs=4 --size=1G
strace -c -e trace=openat,read,write,fsync ./program
```

How it shows up

Symptom	Cause
"No space left" but `df` shows free space	Inodes exhausted — check `df -i`
Deleting files does not free space	A process still holds the file open; `lsof +L1`
Data lost after power cut despite journalling	Device write cache not flushed, or barriers disabled
File exists but is full of zeros after a crash	`data=writeback`, or missing `fsync` before rename
Config file lost entirely after a crash	Renamed without syncing the directory
Extracting many small files is far slower than one big one	Metadata operations dominate; IOPS-bound
Embedded device's flash fails after months	Write amplification and no wear levelling; wrong filesystem for raw NAND
SSD slows down over time	TRIM not issued; the FTL cannot garbage collect
Hard link across filesystems fails	Inode numbers are per-filesystem
Memory usage grows during a large `find`	Dentry and inode cache — reclaimable

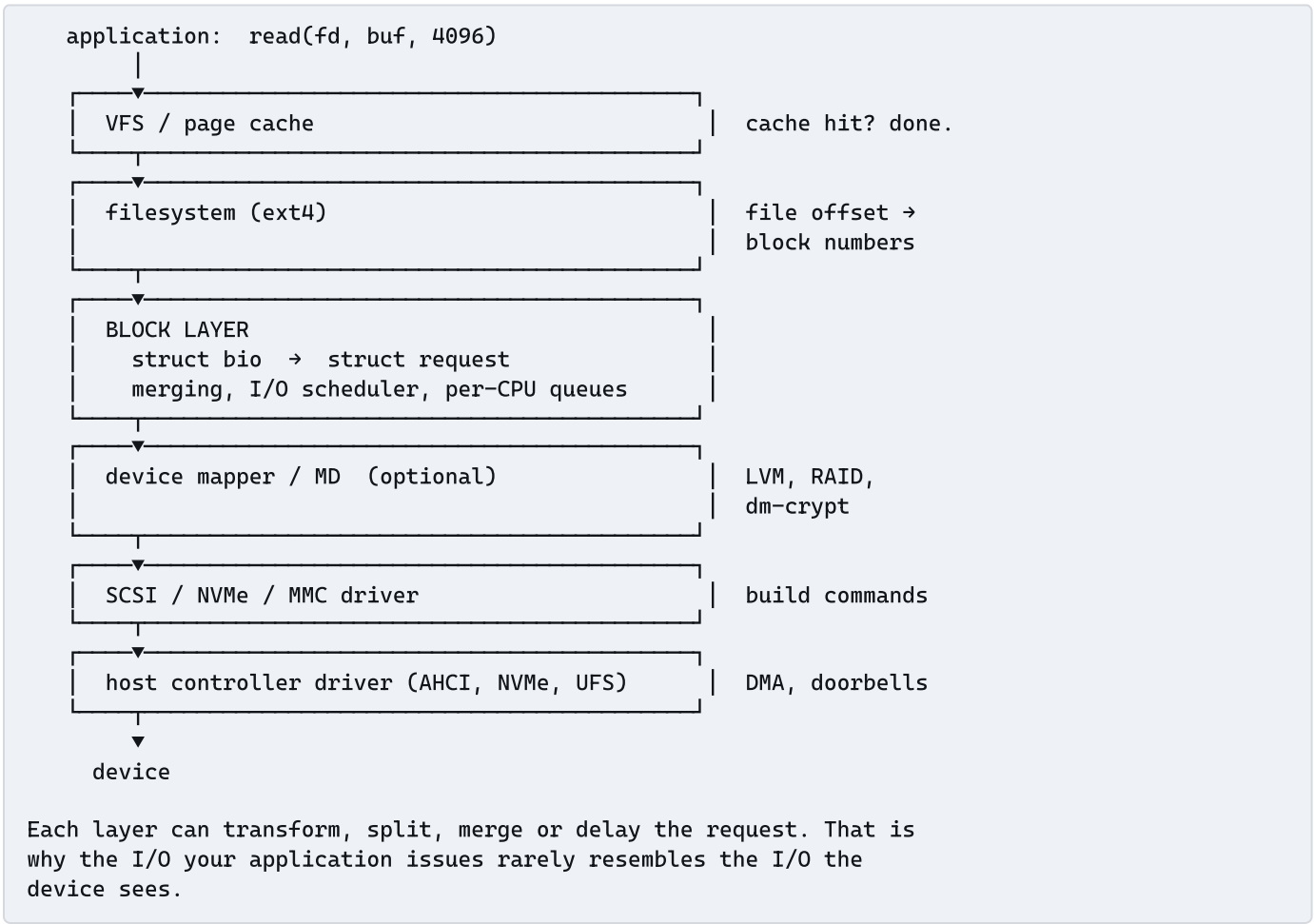
Check yourself

- ✓ Name the four VFS objects and what each represents.
- ✓ Explain why names live in directories and give three consequences.
- ✓ Describe the three-write append problem and why no ordering is safe.
- ✓ Compare journalling and copy-on-write as crash-consistency strategies.
- ✓ Explain the three ext4 journalling modes and their trade-offs.
- ✓ Write the durable-rename idiom and say what each step protects.
- ✓ Explain write amplification and the two places it can be handled.
- ✓ Say why ext4 on raw NAND is a mistake, and what to use instead.

The block layer and the storage stack

Between a filesystem's "read block 12345" and a device actually moving data sits a stack whose job is to merge, reorder, queue and account for requests. Understanding it explains most storage performance behaviour, and all storage latency surprises.

The stack



Each layer can transform, split, merge or delay the request. That is why the I/O your application issues rarely resembles the I/O the device sees.

bio and request

```
/* A `bio` is the unit of I/O submitted to the block layer: a
   target device, a starting sector, and a scatter-gather list of
   memory pages.                                                 */

struct bio {
        struct block_device *bi_bdev;
        blk_opf_t            bi_opf;      /* READ, WRITE, FLUSH,
                                             DISCARD, ZONE_APPEND */
        struct bvec_iter     bi_iter;     /* sector, size, index  */
        struct bio_vec      *bi_io_vec;   /* page, offset, len[]  */
        bio_end_io_t        *bi_end_io;   /* completion callback  */
};

/* Note the scatter-gather list: the pages need not be contiguous
   in physical memory. The device's DMA engine walks the list.
   This is why the DMA API and the SMMU matter here — see the
   security track.                                              */

/* A `request` is one or more merged bios, ready for a driver. */
```

Merging and scheduling

```
The block layer's two big optimisations:

  MERGING — adjacent requests become one
     read sectors 100-107    ┐
     read sectors 108-115    ├─► one request, sectors 100-131
     read sectors 116-123    │
     read sectors 124-131    ┘
     · front merge, back merge, or a completely new request
     · limited by max_sectors_kb and max_segments

  SORTING / SCHEDULING — reorder to suit the device
     on a rotational disk, seeking is 5-10 ms and sequential
     access is ~1000x faster, so ordering by sector is decisive.
     On an SSD, "seeking" is free and the scheduler mostly gets
     out of the way.

  This is why `iostat` often shows fewer, larger requests reaching
  the device than the application issued — and why an application
  doing 4 KB sequential writes can still achieve full bandwidth.
```

Scheduler	Idea	Use for
`none`	No reordering; straight to the device	**NVMe** — the device has thousands of queue entries and reorders better than you can
`mq-deadline`	Two sorted queues plus expiry deadlines, so nothing starves	SATA SSDs, mixed workloads, latency-sensitive work
`bfq`	Budget Fair Queueing — proportional share per process, strong interactivity	Desktops, rotational disks, containers needing fairness
`kyber`	Latency-target based; throttles to hit a goal	Fast devices with a latency SLO

```
cat /sys/block/nvme0n1/queue/scheduler
#   [none] mq-deadline kyber bfq
echo mq-deadline > /sys/block/sda/queue/scheduler

# the queue tunables that actually matter
/sys/block/X/queue/
    nr_requests          queue depth at the block layer
    read_ahead_kb        sequential prefetch (128 default)
    max_sectors_kb       largest single request
    rotational           1 = disk, 0 = SSD; affects scheduler defaults
    add_random           entropy contribution — turn off on SSDs
    nomerges             disable merging (debugging only)
    write_cache          "write back" or "write through"
```

blk-mq: why the old design broke

```
SINGLE-QUEUE (pre-2013)
   all CPUs ──► one request queue, one spinlock ──► device
   Fine at 200 IOPS. At 1,000,000 IOPS the lock IS the bottleneck,
   and every submission bounces a cache line between sockets.

MULTI-QUEUE (blk-mq)
   CPU0 ─► software queue 0 ┐
   CPU1 ─► software queue 1 ├─► hardware queue 0 ─► device
   CPU2 ─► software queue 2 ├─► hardware queue 1 ─► (NVMe has up
   CPU3 ─► software queue 3 ┘                        to 64K queues)

   · per-CPU submission: no shared lock on the fast path
   · completion interrupts are steered back to the submitting CPU,
     so the completion finds the data cache-warm
   · maps naturally onto NVMe, which was designed with the same model

  This is the same insight as per-CPU runqueues in the scheduler
  (OS chapter 06): at scale, the shared structure is the problem.
```

Device characteristics

	HDD	SATA SSD	NVMe SSD	Optane / PMEM
Random read latency	5–10 ms	~100 µs	~20–80 µs	<1 µs
Random IOPS	~100	~50–100 k	~500 k–1 M+	Millions
Sequential	100–250 MB/s	~550 MB/s	2–14 GB/s	Very high
Seek penalty	**Enormous**	None	None	None
Queue depth benefit	Small	Large	**Essential**	—
Bottleneck	Mechanics	Controller, SATA link	Sometimes the *software* stack	Software, entirely

> **THE INVERSION**
>
> On a hard disk, software overhead was irrelevant — a 10 µs kernel path next to a 10 ms seek. On NVMe the device responds in 20 µs and the kernel path can be a comparable fraction of that. This is why `io_uring`, polled I/O and SPDK exist: at these speeds the operating system itself is the thing to optimise away.

```
# polled I/O: spin instead of taking a completion interrupt.
# Burns CPU, removes interrupt and context-switch latency.
echo 1 > /sys/block/nvme0n1/queue/io_poll
# with io_uring: IORING_SETUP_IOPOLL
```

Queue depth and Little's Law

```
   throughput = queue_depth / latency          (Little's Law)

   NVMe device: 80 µs latency per operation

     QD = 1    →  1 / 80µs   =    12,500 IOPS
     QD = 32   →  32 / 80µs  =   400,000 IOPS
     QD = 128  →  128 / 80µs = 1,600,000 IOPS (until saturation)

  A single-threaded, synchronous application can NEVER exceed
  QD=1 performance, no matter how fast the device is. This is the
  single most common storage benchmarking mistake — measuring a
  million-IOPS drive with `dd` and concluding it does 12,500.

  To get depth you need: async I/O (io_uring, libaio), multiple
  threads, or readahead doing it for you.

  But depth costs LATENCY: at QD=128 each request waits behind 127
  others. Throughput and latency trade off exactly as in CA
  chapter 18.
```

```
# measure properly, with depth and parallelism
fio --name=randread --ioengine=io_uring --direct=1 \
    --rw=randread --bs=4k --iodepth=32 --numjobs=4 \
    --size=4G --runtime=60 --group_reporting

# --direct=1 bypasses the page cache. Without it you are
# benchmarking RAM, which is the second most common mistake.
```

Direct I/O

```
int fd = open("file", O_RDWR | O_DIRECT);
/* Bypasses the page cache entirely. Requires that the buffer,
   the file offset and the length are all aligned to the device's
   logical block size — usually 512 or 4096 bytes.            */

void *buf;
posix_memalign(&buf, 4096, size);      /* aligned allocation */
pread(fd, buf, 4096, offset);          /* all three aligned  */

/* Use O_DIRECT when:
     · the application has its own cache (databases)
     · the data will not be re-read, and caching it would evict
       something useful
     · you are benchmarking the device

   Do NOT use it as a general speed-up. It removes readahead and
   write coalescing, so naive sequential access gets SLOWER.  */
```

Device mapper and RAID

Target	Does
`dm-linear`	Maps a range to another device — the building block of LVM
`dm-crypt`	Transparent encryption (LUKS)
`dm-thin`	Thin provisioning and snapshots
`dm-cache`, `bcache`	Use an SSD as a cache in front of a disk
`dm-verity`	Cryptographically verified read-only device — Android Verified Boot and confidential container images depend on it
`md`	Software RAID 0/1/5/6/10

```
lsblk                          # the full stack, visually
dmsetup table                  # device mapper targets in use
cat /proc/mdstat               # RAID status and rebuild progress

/* RAID 5's write hole: a partial stripe update that is
   interrupted by power loss leaves data and parity inconsistent,
   and the inconsistency is invisible until a later rebuild
   reconstructs wrong data. Journals, battery-backed cache or a
   COW filesystem (btrfs/ZFS RAID) address it.              */
```

I/O control and isolation

```
# cgroup v2 io controller — limit or weight a container's I/O
echo "259:0 rbps=10485760 wbps=5242880 riops=1000 wiops=500" \
     > /sys/fs/cgroup/mygroup/io.max
echo "default 100" > /sys/fs/cgroup/mygroup/io.weight

# io.latency: guarantee a latency target for a priority group,
# throttling others when it is exceeded
echo "259:0 target=10000" > /sys/fs/cgroup/critical/io.latency

# per-process I/O priority (works with BFQ / CFQ-style schedulers)
ionice -c 3 -p PID             # idle class
ionice -c 2 -n 7 ./backup      # best-effort, lowest priority
```

Observing

```
iostat -xz 1
#  Device  r/s   w/s  rkB/s  wkB/s  rareq-sz  aqu-sz  r_await w_await %util
#                                             ^depth  ^latency        ^busy
#  %util is MISLEADING on SSDs: a device that can serve 32 requests
#  in parallel shows 100% "utilisation" while nearly idle. Use
#  aqu-sz and await instead.

iotop -o                        # which processes are doing I/O
pidstat -d 1

# per-request tracing
blktrace -d /dev/sda -o - | blkparse -i -
btt -i trace.bin                # latency breakdown by stage:
                                # Q2G queue→get, G2I insert,
                                # I2D issue→device, D2C device→complete
                                # This tells you WHICH LAYER is slow.

# eBPF one-liners
biolatency-bpfcc                # latency histogram
biosnoop-bpfcc                  # every I/O with process and latency
biotop-bpfcc

# where is the time going overall?
cat /proc/pressure/io           # PSI — stall percentage
```

DIAGNOSING "THE DISK IS SLOW"

Work down the stack. `iostat` `await` high but `aqu-sz` low means the *device* is slow. `aqu-sz` high means you are queueing — the device is saturated, or the scheduler is holding requests. If application latency is high but `await` is low, the problem is above the block layer: filesystem metadata, `fsync`, or the page cache. `btt` resolves this precisely by breaking one request's life into stages.

How it shows up

Symptom	Cause
NVMe benchmarks at a fraction of its rating	Queue depth 1 — synchronous single-threaded I/O
Benchmark shows impossible throughput	Hitting the page cache; use `--direct=1`
`%util` is 100% but the device is fine	`%util` is meaningless for parallel devices
`O_DIRECT` made things slower	Lost readahead and write coalescing
`O_DIRECT` returns `EINVAL`	Buffer, offset or length not block-aligned
Write latency spikes every few seconds	Writeback flushing dirty pages; tune `vm.dirty_*`
Backup job destroys interactive latency	No I/O isolation; use `ionice`, BFQ or cgroup `io.latency`
SSD gets slower over months	No TRIM; the FTL cannot garbage collect
RAID 5 array corrupt after power loss	The write hole
Random reads far slower than sequential on a disk	Seek time — the fundamental HDD characteristic

Check yourself

- ✓ Name the layers between `read()` and the device, and what each may change.
- ✓ Explain what a `bio` contains and why it is a scatter-gather list.
- ✓ Explain merging and why it matters more on rotational media.
- ✓ Say which I/O scheduler suits NVMe and why.
- ✓ Explain why blk-mq replaced the single request queue.
- ✓ Use Little's Law to explain why a synchronous program cannot saturate an NVMe drive.
- ✓ Give two legitimate uses of `O_DIRECT` and one common misuse.
- ✓ Explain why `%util` is misleading on SSDs and what to read instead.

The I/O subsystem and device drivers

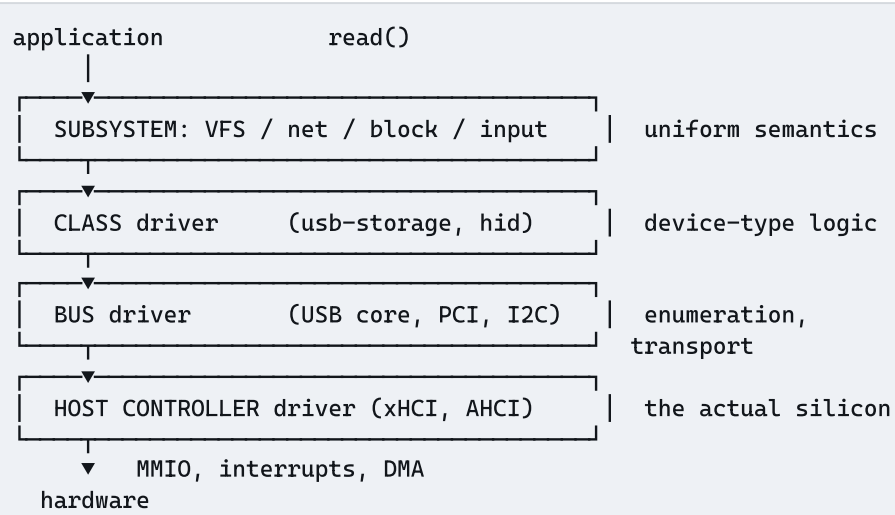
A driver is the adapter between one piece of hardware and one kernel subsystem. This chapter covers the concepts every driver model shares — device classes, the discovery and binding lifecycle, I/O models, and the layering that lets one `read()` reach a hundred different kinds of device.

What a driver actually does

TWO INTERFACES, ONE TRANSLATOR

Upward, a driver implements a **subsystem interface** — the operations a network stack, a block layer or a VFS expects. Downward, it speaks the **device's register protocol**. Everything else in a driver is lifecycle and error handling.

This is why "write a driver" is not one skill: you must know the subsystem's contract as well as the hardware's. Reading an existing driver for the same subsystem is nearly always the fastest route.




```
A USB flash drive traverses every layer: VFS → block → SCSI →
usb-storage → USB core → xHCI → the controller's registers.
Each layer is reusable, which is why adding one new USB device
usually requires no new code at all.
```

The three device classes

	Character	Block	Network
Access	Byte stream, usually sequential	Fixed-size blocks, random access	Packets
Buffered by the kernel	Usually not	Page cache	Socket buffers
Appears as	`/dev/ttyS0`	`/dev/sda`	**No `/dev` node** — `eth0`
Interface	`file_operations`	`block_device_operations` + request queue	`net_device_ops`

	Character	Block	Network
Examples	Serial, input, GPIO, sensors, your custom device	Disks, SD, NVMe	Ethernet, Wi-Fi, modems

Network devices are the odd one out: they are not files, because a packet interface does not fit `read`/`write` semantics well and because routing decisions happen above the device. Sockets are the file-like abstraction instead.

Discovery and binding

```
Two worlds, and the difference drives everything:

  DISCOVERABLE BUSES — hardware can be interrogated
      PCI:  read configuration space → vendor ID, device ID, class,
            BAR sizes and types
      USB:  request descriptors from the device
      → the bus driver enumerates, and the core matches a driver
        by ID. Hotplug works naturally.

  NON-DISCOVERABLE — you cannot ask
      on-SoC peripherals, I2C and SPI slaves.
      There is no safe way to probe address 0x40001000; a wrong
      guess can hang the bus or trigger a side effect.
      → hardware must be DESCRIBED out of band:
            device tree (embedded, ARM)
            ACPI tables (servers, x86, Windows on ARM)

This is exactly why the device tree exists, and why an embedded
Linux driver looks structurally different from a PCIe driver.
```

```
   HARDWARE SIDE                    DRIVER SIDE
   device tree node                 module loads
   or PCI enumeration               registers a driver with
        │                           its match table
        ▼                                 │
   struct device created                  ▼
   added to the bus                 struct device_driver added
        └──────────────┬──────────────────┘
                       ▼
             bus->match(dev, drv)
             compare compatible strings / vendor+device IDs
                       │
                       ▼ match
             driver->probe(dev)
             ── get resources: registers, IRQ, clocks, regulators
             ── map MMIO
             ── initialise the hardware
             ── register with the SUBSYSTEM
                       │
                       ▼
                  in service
                       │
             driver->remove(dev)   on unbind, unload, or hot-unplug
             ── quiesce hardware FIRST, then release, in reverse
```

PROBE ORDERING IS NOT GUARANTEED

Your driver may probe before the clock controller or regulator it depends on exists. Every modern driver model has an answer: Linux returns `-EPROBE_DEFER` and the core retries later; Windows' PnP manager starts devices as their dependencies become available. Never assume initialisation order, and never work around it with `late_initcall` — that is a race, not a fix.

The four resources every driver needs

Resource	Why	Failure if missing
Registers (MMIO)	To control the device	Fault, or reads of garbage
Interrupt	To learn about completions	Hangs waiting forever, or polls wastefully
Clocks	An unclocked block does not respond	Registers read as 0 or abort
Power / regulators	Same	Same

```
/* The order matters: enable clocks and power BEFORE touching any
   register. This is the single most common bring-up bug, and it
   presents identically to a wrong base address.                */

regulator_enable(vdd);
clk_prepare_enable(clk);
reset_control_deassert(rst);
/* ... only now is it safe to read the ID register ... */
id = readl(base + REG_ID);
```

The three I/O models

	Polling	Interrupt	DMA
CPU cost	100% while waiting	Per event	Near zero
Latency	Lowest, most deterministic	Interrupt entry overhead	Setup cost, amortised
Best for	Very short waits, early boot, hard real time, ultra-high-rate NICs	Most peripherals	Bulk transfer
Hazard	Wastes power; must be bounded	Storms if not cleared at the source	Coherency and address translation

```
/* Polling, done correctly — ALWAYS with a timeout */
ret = readl_poll_timeout(base + STATUS, val, val & READY, 10, 1000);
if (ret)
        return -ETIMEDOUT;      /* say so, rather than hanging */

/* Hybrid models are common in practice:
     NAPI (network)   take one interrupt, then POLL until the
                      queue drains. Interrupt latency at low
                      rates, no interrupt storm at high rates.
     Adaptive coalescing  the device delays interrupts until N
                      packets or T microseconds.               */
```

DMA from the driver's point of view

```
The three rules, which are the same on every OS:

  1. Program the device with the address the DEVICE must use —
     Linux calls it dma_addr_t, Windows calls it a logical address.
     It is NOT a virtual address and NOT necessarily a physical one:
     with an IOMMU it is an IOVA.

  2. Respect BUFFER OWNERSHIP. Between map and unmap the buffer
     belongs to the device. Touching it from the CPU without an
     explicit sync gives stale or corrupt data on non-coherent
     systems (CA chapter 15).

  3. Never DMA to stack memory or vmalloc memory. Stack buffers may
     share a cache line with unrelated data; vmalloc memory is not
     physically contiguous.

  And the reason an SMMU/IOMMU exists at all: a device issuing
  addresses bypasses the CPU's MMU entirely, so every isolation
  guarantee built with page tables has a hole in it unless
  something checks those transactions. Security track, ch. 02–07.
```

Drivers outside the kernel

Mechanism	Gives user space	Used for
UIO	Mapped registers and interrupt notification via a file descriptor	Simple custom hardware
VFIO	The same, plus IOMMU-enforced DMA isolation	Device passthrough to VMs; DPDK, SPDK
libusb	Direct USB transfers	Instruments, programmers, cameras
spidev / i2c-dev	Raw bus transactions	Prototyping, one-off sensors
gpiod	GPIO lines by fd (replaces the deprecated sysfs interface)	Scripts, tooling
FUSE	A filesystem implemented in user space	sshfs, s3fs, experimentation
UMDF (Windows)	A full driver framework in user mode	USB, HID, sensors — the recommended default

```
The trade-off (OS chapter 02, again):

  IN KERNEL       full speed, direct access, no context switches
                  a bug panics the machine
                  a vulnerability is total compromise

  IN USER SPACE   a crash kills one process
                  can use any language, any library, a debugger
                  can be restarted
                  syscall or mapping overhead per operation
                  needs an IOMMU to make DMA safe

  DPDK and SPDK take the user-space path AND remove the overhead,
  by mapping the device with VFIO and POLLING it from a dedicated
  core — no syscalls, no interrupts, no kernel in the data path.
  They trade a whole CPU core for latency and throughput.
```

Exposing a device to user space

Interface	Good for	Caution
Device node + `read` / `write`	Streams of data	Needs framing if messages have boundaries
ioctl / `DeviceIoControl`	Arbitrary commands with structured arguments	The struct is permanent ABI — design for extension
sysfs attributes	One value per file; scriptable configuration and status	It is ABI: cannot be changed once shipped
debugfs	Anything, during development	No stability guarantee — never depend on it
mmap	Zero-copy access to buffers or registers	Exposes hardware directly; validate carefully
netlink / uevents	Asynchronous notifications to user space	
A subsystem (IIO, input, v4l2, hwmon)	**Preferred.** Existing tools and libraries work immediately	Must fit the subsystem's model

> **USE AN EXISTING SUBSYSTEM IF ONE FITS**
>
> A temperature sensor registered with **hwmon** is immediately visible to `sensors`, monitoring agents and fan control. The same sensor behind a custom ioctl needs bespoke tooling forever. Before inventing an interface, search for a subsystem that already models your device class — IIO for sensors, input for buttons and touch, v4l2 for video, ALSA for audio, hwmon for monitoring, rtc, watchdog, pwm, led. This is the difference between a driver that gets merged and one that does not.

Getting removal right

```
/* Removal is where most driver bugs live, because it races with
   everything still in flight.                                    */

static void my_remove(struct device *dev)
{
        struct my_dev *d = dev_get_drvdata(dev);

        /* 1. STOP the hardware generating events, FIRST */
        writel(0, d->regs + REG_IRQ_ENABLE);
        writel(0, d->regs + REG_CTRL);

        /* 2. unregister from the subsystem, so no NEW requests
              can arrive */
        unregister_device(&d->cdev);

        /* 3. wait for in-flight work to finish */
        cancel_work_sync(&d->work);
        del_timer_sync(&d->timer);

        /* 4. release resources in reverse order of acquisition */
        clk_disable_unprepare(d->clk);
        /* devm_ resources released automatically after this   */
}

/* Get the ORDER wrong and an interrupt fires after you freed the
   structure it references — a use-after-free that only appears
   under load.                                                    */
```

```
# the single best driver test there is: bind/unbind in a loop
for i in $(seq 100); do
  echo 40010000.widget > /sys/bus/platform/drivers/mydrv/unbind
  echo 40010000.widget > /sys/bus/platform/drivers/mydrv/bind
done
dmesg | grep -iE 'kasan|use-after|warn|leak'
# Nearly every resource leak, missing-quiesce and lifetime bug
# surfaces here and almost nowhere else. Run it with KASAN on.
```

Observing the I/O subsystem

```
lspci -vv                   # PCI devices, BARs, capabilities, driver
lsusb -t                    # USB topology
ls /sys/bus/*/devices/      # everything the device model knows
ls /sys/bus/platform/drivers/mydrv/    # what bound to this driver
ls /sys/class/               # by device class

cat /proc/interrupts        # is the handler firing?
cat /proc/iomem             # who owns which physical range
cat /proc/devices           # major numbers
ls /sys/kernel/iommu_groups/*/devices/

udevadm info -a -n /dev/ttyUSB0     # every attribute, for writing rules
udevadm monitor                      # watch hotplug events live

dmesg -w                             # driver messages as they happen
```

How it shows up

Symptom	Cause
Device present but no driver bound	Match table missing the ID or compatible string
Registers read all zeros or abort	Clock or power not enabled — check this before the address
Probe fails intermittently at boot	Dependency not ready; return `-EPROBE_DEFER`
Interrupt count never increases	Wrong number, wrong trigger type, or not enabled at the device
Interrupt fires continuously	Not cleared at the source before acknowledging
DMA writes land in the wrong place	Programmed with a virtual or physical address instead of the DMA address
Crash on module unload under load	Removal order wrong; something still in flight
Works once, fails on re-bind	State not reset in probe, or leaked in remove
Custom tooling needed for a common device type	Should have used an existing subsystem
User-space driver cannot DMA safely	Needs VFIO with an IOMMU, not UIO

Check yourself

- ✓ State a driver's two interfaces and why writing one needs two kinds of knowledge.
- ✓ Give the three device classes and say why network devices have no `/dev` node.
- ✓ Explain discoverable versus described buses and why the device tree exists.
- ✓ Walk the discovery-to-probe binding flow.
- ✓ Name the four resources a driver acquires and the ordering rule between them.

- ✓ Compare polling, interrupts and DMA, and describe one hybrid model.
- ✓ Give the three DMA rules and the reason an IOMMU exists.
- ✓ Give the correct order of operations in `remove()` and the bug that wrong order causes.

Interrupts, timers and timekeeping

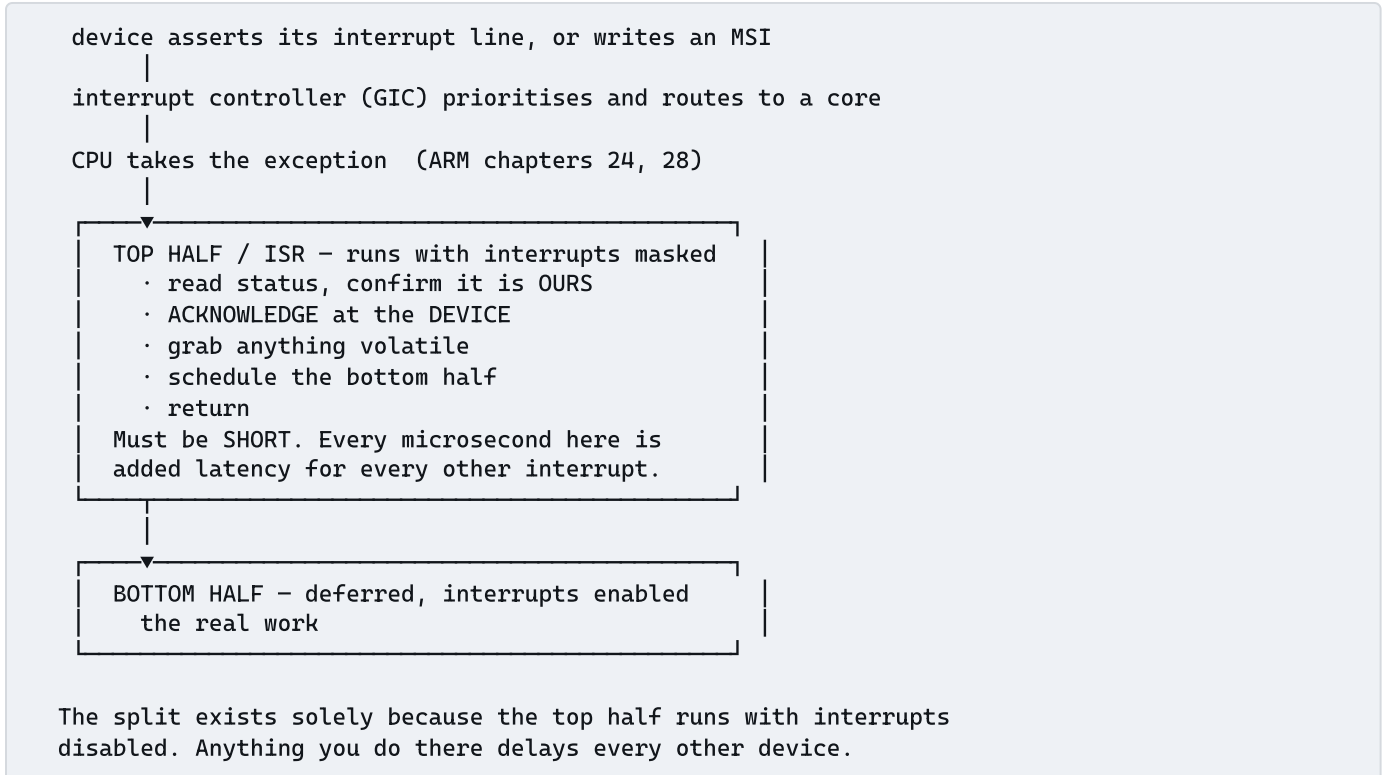
Interrupts are how hardware gets the kernel's attention, and timers are how the kernel gets its own. Both are governed by one constraint: the code that runs in interrupt context is severely restricted, and everything else about the design follows from working around that.

Interrupt context

THE RULE EVERYTHING ELSE FOLLOWS FROM

An interrupt handler runs in a *borrowed* context: on some arbitrary thread's kernel stack, with interrupts partly or fully masked, and with no task to schedule away from. Therefore it **cannot sleep**. No mutex, no `GFP_KERNEL` allocation, no `copy_to_user`, no waiting for I/O. Violating this deadlocks the machine.

Windows expresses the same constraint as **IRQL**: at DIRQL and DISPATCH_LEVEL you may not touch pageable memory or wait. Linux calls it "atomic context". The concept is identical.



Deferred work

Linux	Windows	Context	Can sleep?	Use for
softirq	—	Atomic	No	Networking, block completion. Statically defined; you will not add one.
tasklet	**DPC**	Atomic	No	Legacy in Linux (being removed); the standard mechanism on Windows
workqueue	**work item**	Process (kernel thread)	**Yes**	Anything that may block

Linux	Windows	Context	Can sleep?	Use for
threaded IRQ	—	Process (dedicated thread)	**Yes**	**The modern default** for device drivers

```
/* Threaded IRQ: a tiny hard handler plus a sleepable thread.
   This is what you should reach for first.                  */
devm_request_threaded_irq(dev, irq,
                          my_hard_isr,   /* atomic: ack, return */
                          my_thread_fn,  /* may sleep           */
                          IRQF_ONESHOT, dev_name(dev), d);

static irqreturn_t my_hard_isr(int irq, void *p)
{
        struct my_dev *d = p;
        u32 st = readl(d->regs + STATUS);

        if (!(st & IRQ_PENDING))
                return IRQ_NONE;        /* not ours — ESSENTIAL on a
                                           shared line, or the core
                                           cannot detect a stuck IRQ */
        writel(st, d->regs + STATUS);   /* W1C: clear at the SOURCE  */
        d->last_status = st;
        return IRQ_WAKE_THREAD;
}

/* PREEMPT_RT makes ALL interrupt handlers threaded by default —
   which is how it achieves bounded latency, at some throughput
   cost.                                                      */
```

THE INTERRUPT STORM

Clear the interrupt at the controller but not at the *device*, and a level-triggered line re-asserts immediately. The core does nothing but enter and leave the handler forever; the system appears hung with 100% CPU in the ISR. Always clear the condition at the source *first*, then acknowledge upward. Linux's spurious-interrupt detector will eventually disable the line and log `"nobody cared"` — which is a diagnosis, not a fix.

Edge versus level

	Level-triggered	Edge-triggered
Asserted	While the condition holds	On the transition only
Sharing	**Possible** — each handler checks and clears its own	Unsafe: two simultaneous edges register as one
Missed interrupts	Cannot be missed — the line stays asserted	**Can be lost** if masked during the edge
Failure mode	Storm, if never cleared	Silent hang, waiting for an event that already happened

```
/* The classic edge-triggered race, and the standard fix: */
static irqreturn_t isr(int irq, void *p)
{
        do {
                status = readl(STATUS);
                handle(status);
                writel(status, STATUS);        /* clear */
        } while (readl(STATUS) & PENDING);     /* RE-CHECK before
                                                  returning — a new
                                                  event may have
                                                  arrived while we
                                                  were handling      */
        return IRQ_HANDLED;
}
/* Without the loop, an event arriving between handle() and return
   produces no new edge and is lost forever. The device then waits
   for a response that never comes.                                 */
```

MSI: interrupts as memory writes

```
WIRED interrupt              MESSAGE-SIGNALLED (MSI/MSI-X, ARM LPI)
  a physical line              a WRITE to a special address
  pin-limited → shared         thousands per device, never shared
  needs "is it mine?"          the vector identifies the source
  can arrive BEFORE the DMA    arrives IN ORDER with the DMA data,
  data it announces            because it is the same bus stream

That last point is not a minor detail. With a wired interrupt, a
driver must issue a read (a "PCI posting flush") to guarantee the
DMA data has landed before it reads the buffer. With MSI the
ordering is free, which removes a subtle and common data-corruption
bug.

  cat /proc/interrupts        # PCI-MSI entries vs IO-APIC/GIC-SPI
  lspci -vv | grep -A3 MSI-X
```

Interrupt routing and affinity

```
# which CPU handles which interrupt?
cat /proc/interrupts
#            CPU0    CPU1    CPU2    CPU3
#  24:     102394       0       0       0   GICv3  35 Level  eth0

# all on CPU0 is a scalability problem. Spread them:
cat /proc/irq/24/smp_affinity_list
echo 2 > /proc/irq/24/smp_affinity_list      # pin to CPU2

# irqbalance does this automatically — and is usually wrong for
# latency-sensitive or RT workloads. Disable it and pin by hand:
systemctl stop irqbalance

# keep interrupts OFF your real-time cores entirely
# kernel cmdline: isolcpus=2,3 nohz_full=2,3 irqaffinity=0,1

# network multi-queue: one IRQ per queue, pinned per core, with RSS
# steering flows to queues. This is how a NIC scales past one core.
ethtool -l eth0
ethtool -L eth0 combined 4
```

Timer hardware

Role	Provides	ARM example
Clocksource	A monotonically increasing counter — "what time is it?"	`CNTPCT_EL0`, the architected generic timer
Clockevent	Programmable "interrupt me at time T"	`CNTP_CVAL_EL0` comparator

```
cat /sys/devices/system/clocksource/clocksource0/available_clocksource
#   arch_sys_counter        (ARM generic timer — the good one)
#   tsc hpet acpi_pm        (x86 options)

/* A good clocksource must be: monotonic, constant-rate (not
   affected by CPU frequency), synchronised across cores, and
   cheap to read. Early x86 TSC failed several of these, which is
   why HPET existed and why "unstable TSC" messages mattered.

   ARM's generic timer is architected: same behaviour on every
   implementation, readable from EL0 without a trap, which is what
   makes the vDSO clock_gettime possible (OS chapter 03).      */
```

The tick, and doing without it

```
PERIODIC TICK — the traditional design
   a timer interrupt every 1/HZ seconds (HZ = 100, 250 or 1000)
   each tick: update jiffies, run expired timers, account CPU
              time, check whether to preempt

   Simple. But it wakes the CPU 1000 times a second even when
   completely idle, preventing deep sleep states (CA chapter 20).

TICKLESS IDLE (NO_HZ_IDLE) — the default
   when a CPU goes idle, program the timer for the NEXT actual
   deadline instead of the next tick. An idle laptop can sleep
   for seconds at a time.

FULL TICKLESS (NO_HZ_FULL)
   suppress the tick even while RUNNING, if only one task is
   runnable on that CPU. Removes ~1000 interruptions per second
   from a latency-critical thread.
      kernel cmdline: nohz_full=2,3 rcu_nocbs=2,3 isolcpus=2,3
   Combined with IRQ affinity and a pinned RT thread, this is the
   standard recipe for microsecond-class latency on Linux.
```

```
/* jiffies: the tick counter. Still used, and still a trap. */
unsigned long timeout = jiffies + msecs_to_jiffies(500);

if (time_after(jiffies, timeout))       /* CORRECT              */
if (jiffies > timeout)                  /* WRONG — breaks on the
                                           32-bit wrap, which on a
                                           1000 HZ system happens
                                           every 49.7 days      */
/* time_after/time_before do signed subtraction, which is correct
   across wraparound. Use them always.                          */
```

Kernel timers

```
/* Low resolution — jiffy granularity, very cheap, batched */
struct timer_list t;
timer_setup(&t, callback, 0);
mod_timer(&t, jiffies + msecs_to_jiffies(100));
del_timer_sync(&t);            /* MUST be _sync before freeing —
                                  waits for a running callback   */

/* High resolution — nanosecond, backed by a clockevent */
struct hrtimer h;
hrtimer_init(&h, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
h.function = callback;
hrtimer_start(&h, ms_to_ktime(10), HRTIMER_MODE_REL);
hrtimer_cancel(&h);

/* Short delays, no timer needed */
udelay(50);       /* BUSY-WAIT. Atomic context OK. Keep under
                     ~100 µs — you are burning a CPU.           */
usleep_range(100, 200);   /* SLEEPS. Process context only. The
                             range lets the kernel BATCH your
                             wakeup with others, saving power.  */
msleep(20);

/* Never usleep_range with a zero-width range: it defeats batching
   and wakes the CPU precisely for you alone.                   */
```

Need to wait	Use
< 10 µs	`udelay` / `ndelay` — busy-wait
10 µs – 20 ms	`usleep_range`
> 20 ms	`msleep`
Until an event	`wait_event_*` — not a timer at all
Precise, repeating	`hrtimer`
Coarse, cancellable	`timer_list`

Time in user space

```
struct timespec ts;
clock_gettime(CLOCK_MONOTONIC, &ts);      /* durations — NEVER jumps */
clock_gettime(CLOCK_REALTIME, &ts);       /* wall clock — CAN jump   */
clock_gettime(CLOCK_BOOTTIME, &ts);       /* monotonic + suspend time */

/* Served by the vDSO: no syscall, ~25 ns. */

/* precise periodic work, without drift */
struct timespec next;
clock_gettime(CLOCK_MONOTONIC, &next);
for (;;) {
        next.tv_nsec += 1000000;                /* +1 ms */
        if (next.tv_nsec >= 1000000000) { next.tv_nsec -= 1000000000; next.tv_sec++; }
        clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
        do_work();
}
/* ABSOLUTE deadlines. Sleeping for a RELATIVE 1 ms accumulates the
   work time plus wakeup jitter into permanent drift.          */

/* timerfd: a timer as a file descriptor — integrates with epoll */
int tfd = timerfd_create(CLOCK_MONOTONIC, 0);
timerfd_settime(tfd, 0, &spec, NULL);
```

NEVER MEASURE DURATIONS WITH `CLOCK_REALTIME`

NTP steps it, daylight saving changes it, an administrator can set it backwards, and a virtual machine can have it jump on migration. Durations computed from it go negative or enormous. Use `CLOCK_MONOTONIC` for anything you subtract, and `CLOCK_REALTIME` only for timestamps a human will read.

Interrupt latency

```
   device asserts
        |  ← hardware propagation, GIC arbitration
        |  ← INTERRUPTS DISABLED somewhere in the kernel  ◄── the
        |     (a spinlock, a critical section)                variable
        |  ← exception entry, register save                   part
        |  ← handler dispatch
        ▼
   your ISR's first instruction

  Total latency = hardware + longest interrupts-disabled region
                + entry cost

  The DOMINANT and most variable term is the interrupt-disabled
  time somewhere else in the kernel. This is exactly what
  PREEMPT_RT attacks: it converts most spinlocks to sleeping
  mutexes and threads the handlers, bounding that term.

  Measuring it:
     cyclictest -t4 -p 90 -i 200 -m -D 3600
     → report the MAXIMUM, not the average. The average is
       always fine; the maximum is what breaks deadlines.

     ftrace: irqsoff and preemptoff tracers find the exact
     longest region and its stack trace.
         echo irqsoff > /sys/kernel/debug/tracing/current_tracer
         cat /sys/kernel/debug/tracing/trace
```

How it shows up

Symptom	Cause
System hangs at 100% CPU in an ISR	Interrupt storm — not cleared at the device
`"irq N: nobody cared"` in dmesg	Handler returned `IRQ_HANDLED` without handling it, or a shared line with no owner
Device stops responding after high load	Lost edge-triggered interrupt; add the re-check loop
Interrupt count never increases	Wrong number, wrong trigger type, or masked at the device
All interrupts land on CPU0	No affinity spreading; the core saturates
Latency spikes only on some cores	Interrupts not steered away from the RT cores
"scheduling while atomic" / bug check 0xA	Slept in interrupt context
Timer callback runs after the structure is freed	Used `del_timer` instead of `del_timer_sync`
Periodic task drifts over hours	Relative sleeps instead of absolute deadlines
Measured duration is negative	Used `CLOCK_REALTIME`; NTP stepped it
Idle laptop has poor battery life	A periodic timer preventing deep idle (CA ch. 20)

Check yourself

- ✓ Explain why an interrupt handler cannot sleep, and name the Windows equivalent of the rule.
- ✓ Give the top-half/bottom-half split and four deferred-work mechanisms.
- ✓ Explain the interrupt storm and the ordering that prevents it.

- ✓ Compare edge and level triggering on sharing and on missed interrupts.
- ✓ Give two advantages of MSI, including the ordering property.
- ✓ Distinguish clocksource from clockevent.
- ✓ Explain tickless operation and what `nohz_full` buys.
- ✓ Explain why `jiffies > timeout` is wrong.
- ✓ Say which term dominates interrupt latency and how PREEMPT_RT addresses it.

Boot and initialisation

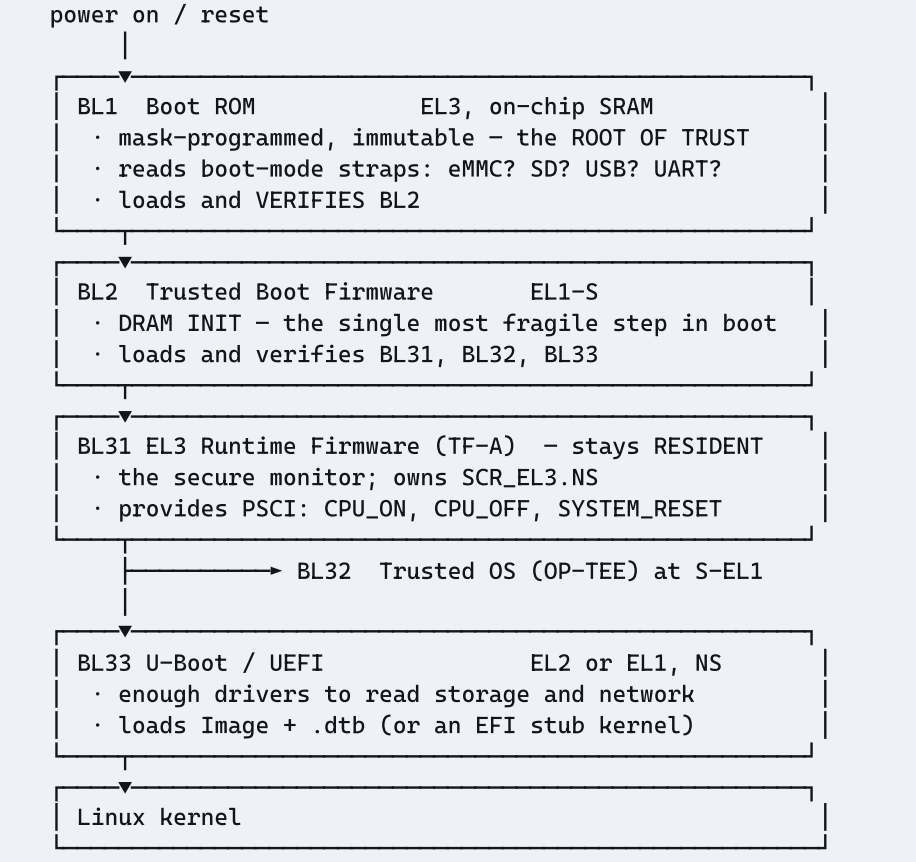
Boot is a chain of increasingly capable programs, each loading and validating the next. It is where firmware, security and the OS meet, and where bring-up problems are most opaque — because when it fails you often have no output at all.

The bootstrap problem

WHY THERE ARE SO MANY STAGES

At reset the CPU has no DRAM, no stack, no filesystem and no drivers — only a fixed address it starts executing from. Each stage exists to make the *next* one possible: on-chip ROM runs from a few kilobytes of SRAM and its only job is to initialise enough to load a slightly larger program; that program initialises DRAM so a full bootloader will fit; the bootloader has drivers enough to read a filesystem and load a kernel. Each stage is bigger and more capable because the previous one earned it the resources.

ARM64 embedded boot



The security dimension of this chain — signature verification at every arrow, anti-rollback, fuses — is the subject of the security track. Here we care about the mechanics.

The kernel's entry contract

```
/* Documented in Documentation/arch/arm64/booting.rst. The
   bootloader MUST deliver exactly this state:              */

    x0 = physical address of the flattened device tree blob
    x1 = x2 = x3 = 0                      (reserved, must be zero)

    MMU        OFF
    D-cache    OFF   (and the image must be cache-clean to PoC)
    I-cache    may be on or off
    interrupts masked at the CPU
    running at EL2 if available (preferred — enables KVM), else EL1
    all secondary CPUs parked, released via PSCI or a spin-table

/* Getting any of this wrong gives a kernel that hangs before it can
   print anything — which is why `earlycon` exists and why a serial
   console is the first thing you bring up on new hardware.     */
```

Inside the kernel

```
   head.S — arch/arm64/kernel/head.S
     · if at EL2 and not using KVM, drop to EL1
     · set up an IDENTITY map plus the initial kernel page tables
     · enable the MMU
     · BRANCH TO VIRTUAL ADDRESSES  ◄── the moment the kernel
       (__primary_switch)               starts living in its own
     · zero .bss, set up the initial stack       address space
        │
        ▼
   start_kernel()  — init/main.c
     setup_arch()        parse the DTB or ACPI, reserve memory,
                         discover CPUs, set up earlycon
     mm_init()           hand memory to the buddy allocator, slab
     sched_init()        runqueues and the idle task
     init_IRQ()          probe and initialise the GIC
     time_init()         start the generic timer
     rest_init()         create PID 1 and kthreadd, then become
                         the idle task forever
        │
        ▼
   kernel_init()  (PID 1, still in kernel mode)
     do_initcalls()      run every initcall level in order
     smp_init()          bring up secondary CPUs via PSCI CPU_ON
     mount the root filesystem
     free the initrd
     execve("/sbin/init")  ◄── PID 1 becomes a user process and
                               NEVER returns to the kernel
```

```
/* initcalls: drivers are not called explicitly. Each registers via
   a macro; the linker collects them into ordered sections.     */
early_initcall()      /* level 0 */
core_initcall()       /* 1 */
postcore_initcall()   /* 2 */
arch_initcall()       /* 3 */
subsys_initcall()     /* 4 — bus types, classes */
fs_initcall()         /* 5 */
device_initcall()     /* 6 — most drivers; module_init maps here */
late_initcall()       /* 7 */

/* Within a level, order follows LINK order, which follows the
   Makefile. Depending on that is a bug. Express real dependencies
   with -EPROBE_DEFER (OS chapter 16) instead.                 */
```

initramfs and the root filesystem

```
The chicken-and-egg problem:
   to mount the root filesystem you need its driver
   the driver is a module
   the module lives on the root filesystem

INITRAMFS solves it: a cpio archive, loaded into RAM by the
bootloader (or built into the kernel), containing exactly the
modules and tools needed to reach the real root.

   bootloader loads Image + initramfs
        |
   kernel unpacks the cpio into a tmpfs — this IS the initial root
        |
   /init runs (a shell script, or systemd)
     · load storage, RAID, LVM, crypto modules
     · assemble arrays, unlock LUKS, activate LVM
     · possibly fetch a network root
     · mount the real root at /sysroot
         |
   switch_root /sysroot /sbin/init
     · move the mounts, free the initramfs memory, exec the real init

Embedded systems often skip it entirely: build the storage driver
into the kernel and boot straight to a fixed root. Fewer moving
parts, faster boot.
```

```
# the kernel command line drives most of this
root=/dev/mmcblk0p2 rootwait rw
root=PARTUUID=abcd-1234        # stable across device renumbering
rootfstype=ext4
console=ttyAMA0,115200 earlycon
init=/bin/sh                   # emergency: skip the init system
initcall_debug                 # print every initcall and its timing
loglevel=8 ignore_loglevel

# see what you booted with
cat /proc/cmdline
```

User space boot

Init system	Model	Suits
SysV init	Numbered shell scripts, run sequentially	Historical; simple to read, slow
BusyBox init	Minimal, `/etc/inittab`	Embedded — small and predictable
systemd	Declarative units, dependency graph, parallel start, socket and bus activation	Desktops and servers; increasingly embedded
OpenRC / runit / s6	Supervision-focused, lighter	Minimal systems, containers

```
# systemd's key idea: SOCKET ACTIVATION removes ordering entirely.
# systemd creates the listening socket first, then starts the
# service lazily and passes it the fd (SCM_RIGHTS, OS chapter 08).
# Clients can connect before the service has started — their
# connections simply queue. Nothing needs to wait for anything.

systemd-analyze                # total boot time, split by phase
systemd-analyze blame          # slowest units
systemd-analyze critical-chain  # the dependency path that determined
                               # total boot time — the only one worth
                               # optimising
systemd-analyze plot > boot.svg
```

Bringing up secondary CPUs

```
/* Only CPU0 runs at reset. The others are parked. */

/* PSCI — the standard mechanism, implemented by EL3 firmware */
    SMC #0 with x0 = PSCI_CPU_ON (0xC4000003)
               x1 = target MPIDR
               x2 = entry point physical address
               x3 = context id
/* The firmware releases that core, which enters at the given
   address with the MMU off, exactly like the primary CPU did. */

/* The kernel then, per CPU:
     · set up its per-CPU page tables and stack
     · enable the MMU
     · initialise its GIC redistributor and local timer
     · calibrate, and mark itself online
     · enter the idle loop, ready for the scheduler          */

/* Older platforms used a SPIN-TABLE: secondaries poll a memory
   location written by the primary. PSCI is preferred because it
   also handles CPU_OFF, SUSPEND and SYSTEM_RESET uniformly.  */

cat /sys/devices/system/cpu/online
echo 0 > /sys/devices/system/cpu/cpu3/online     # hotplug offline
```

Fast boot

Technique	Saves
Skip the bootloader's countdown and menu	Seconds — usually the largest single win

Technique	Saves
Compile in the storage driver; drop initramfs	Hundreds of ms
Trim the kernel config aggressively	Decompression and initcall time
Choose the decompressor deliberately (LZ4 > gzip > XZ for speed)	Hundreds of ms; trades image size
`quiet` on the command line	Serial console output is astonishingly slow at 115200
Defer non-essential drivers to after userspace starts	Perceived boot time
Suspend-to-RAM or hibernate instead of a cold boot	Nearly everything
Execute-in-place (XIP) from NOR flash	The whole copy-and-decompress step

```
# find where the time actually goes
dmesg | head -50                      # timestamps are seconds since start
systemd-analyze blame
cat /proc/cmdline                      # is initcall_debug on?

# with initcall_debug, dmesg shows every initcall's duration:
dmesg | grep 'initcall' | sort -t= -k2 -rn | head -20

# a 5-second gap in dmesg timestamps is almost always a probe
# timeout — a device that is not there, or is not responding.
```

When boot fails silently

```
Work down the chain. At each step, the question is "did we get
this far?", and the answer needs SOME form of output.

  1. No bootloader output at all
       → power rails, clocks, reset, boot-mode straps, or the
         boot medium. Attach JTAG: can the probe see the core's
         IDCODE? If not, stop and fix that first.

  2. Bootloader runs, kernel produces nothing
       → almost always the console. Add `earlycon` and check that
         the device tree's chosen/stdout-path names the right UART
         at the right address.
       → or the entry contract was violated (x0 not the DTB, MMU
         left on, D-cache left on)

  3. Kernel prints, then stops at "Starting kernel..."
       → DTB wrong or absent; decompression failure; wrong load
         address; memory node describing memory that is not there

  4. Kernel boots, panics with "Unable to mount root fs"
       → the storage driver is a module and there is no initramfs,
         or `root=` names the wrong device. `rootwait` helps when
         the device appears late.

  5. Boots to a shell but a device is missing
       → the device tree node, its status property, or driver
         binding (OS chapter 16)

Tools: earlycon, `initcall_debug`, `ignore_loglevel`, a JTAG probe,
and a serial cable — which remains the single most valuable piece
of bring-up equipment.
```

How it shows up

Symptom	Cause
No output whatsoever	Console not configured; add `earlycon` and check `stdout-path`
Hangs right after "Starting kernel"	Entry contract violated, or the DTB is wrong
"Unable to mount root fs on unknown-block(0,0)"	Storage driver missing, or wrong `root=`
"Kernel panic - not syncing: No working init found"	Root mounted but `/sbin/init` is absent or not executable
Boots from SD but not eMMC	Boot-mode straps, or partition/offset differences
Random crashes under load, boots fine	Marginal DRAM timing from BL2 — test at temperature
Multi-second gap in dmesg timestamps	A probe timeout on a device that is not present
Only one CPU online	PSCI not working, or the CPU nodes in the DTB are wrong
Boot is slow with a serial console attached	Console output at 115200 dominates; add `quiet`
Works from a development board, fails on production silicon	Fuses changed the boot or security configuration

Check yourself

- ✓ Explain why boot needs multiple stages rather than one program.
- ✓ Name the ARM boot stages and what each is responsible for.
- ✓ State the ARM64 kernel entry contract and one failure caused by breaking it.
- ✓ Explain what happens between `head.S` and `start_kernel`.
- ✓ Explain the initramfs chicken-and-egg problem and how `switch_root` resolves it.
- ✓ Explain socket activation and why it removes startup ordering.
- ✓ Describe how secondary CPUs are started with PSCI.
- ✓ Give a five-step procedure for diagnosing a silent boot failure.

Real-time operating systems and latency

Real-time means *predictable*, not fast. An RTOS trades throughput, memory efficiency and flexibility for one property: a bounded worst case. This chapter covers what that bound costs, how to measure it, and when Linux can deliver it instead.

What real-time means

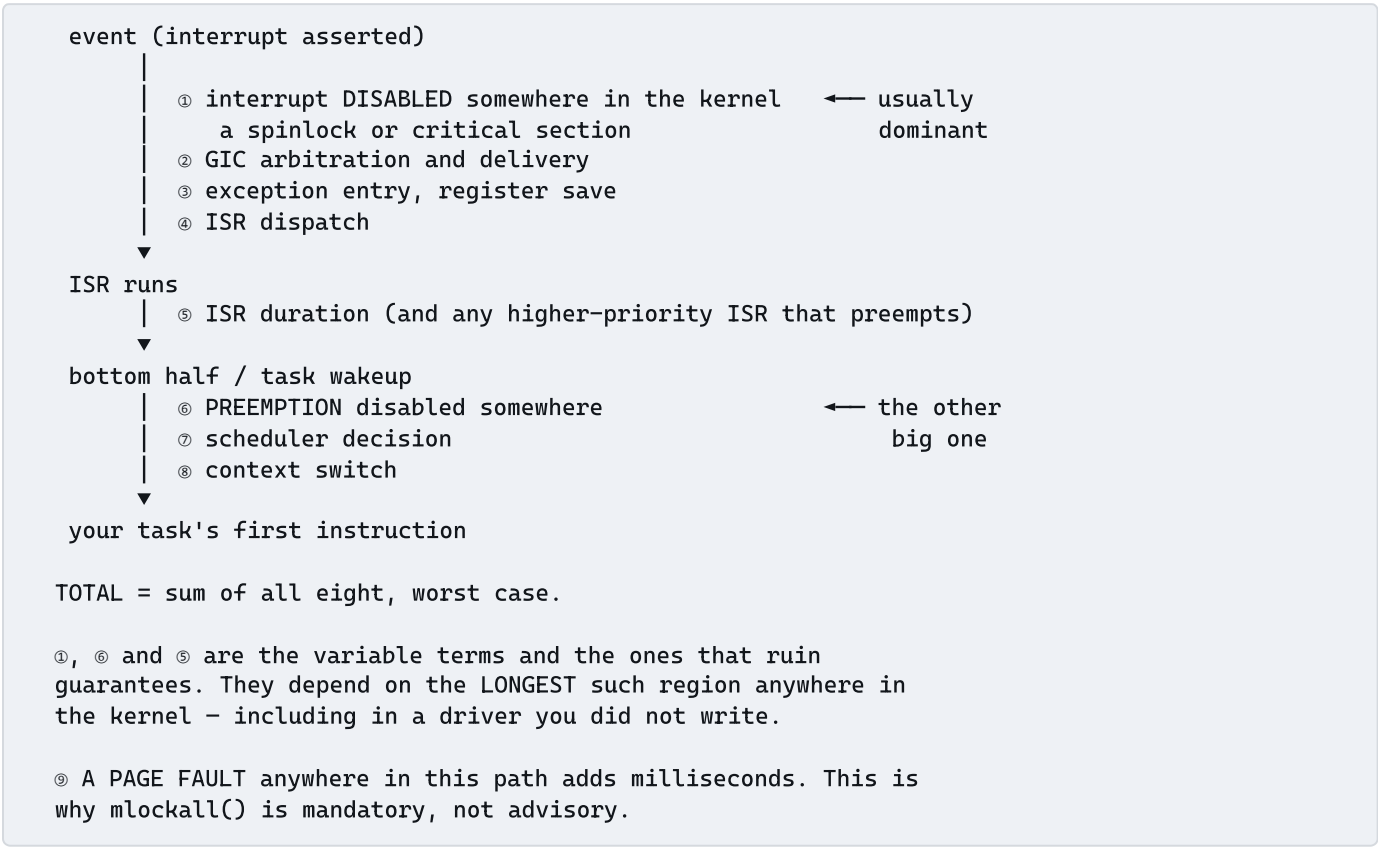
THE DISTINCTION

A system is real-time if correctness depends on *when* a result is produced, not only on the result itself. A motor controller that computes the right current 2 ms late has failed, however correct the number.

The metric is therefore the **worst case**, never the average. A system with 5 µs average latency and one 8 ms outlier per hour is *not* real-time; one with a guaranteed 200 µs bound is.

Class	Missing a deadline means	Examples
Hard	System failure. Possibly injury or loss of life.	Airbag deployment, flight control, engine timing, medical infusion
Firm	The result becomes worthless, but no damage	Video frame rendering, a missed radar sweep
Soft	Quality degrades proportionally	Audio glitches, UI stutter, VoIP jitter

Where latency comes from



How an RTOS achieves a bound

Design choice	Bounds	Costs
Strict priority preemption, no fairness	The highest-priority ready task always runs	Lower-priority tasks can starve — by design
No demand paging, often no MMU	Removes page-fault latency entirely	No isolation between tasks
Static allocation only	No allocator variance, no fragmentation	Everything sized at build time
Bounded interrupt disable regions	Term ① above	Careful hand-auditing of every critical section
Priority inheritance on every mutex	Priority inversion (ch. 06)	A few cycles per lock and unlock
O(1) scheduler — a priority bitmap	Term ⑦	Limited number of priorities
Small kernel	Few paths to audit; analysable	Few features

```
/* A typical RTOS scheduler decision, in constant time: */
static uint32_t ready_bitmap;              /* one bit per priority */

static inline int highest_ready(void)
{
        return 31 - __builtin_clz(ready_bitmap);   /* ONE instruction */
}
/* Compare with EEVDF's red-black tree walk. The RTOS gives up all
   fairness and gains a provable, tiny, constant bound.        */
```

The RTOSes you will meet

RTOS	Footprint	Character
FreeRTOS	5–10 KB	The de-facto MCU standard. Tasks, queues, semaphores, timers. MIT licensed, now maintained by AWS.
Zephyr	10–100 KB	Linux Foundation. Device-tree driven, Kconfig, extensive driver model and networking. The modern choice for new MCU work.
ThreadX / Azure RTOS	~10 KB	Safety-certified, very small, widely deployed
QNX	MBs	Microkernel, POSIX, hard real-time. Automotive infotainment and instrument clusters.
VxWorks	MBs	Aerospace and defence; long certification pedigree
RTEMS	MBs	Open source, spaceflight heritage
seL4	~10 KLOC	Formally verified microkernel with proven WCET bounds
Linux + PREEMPT_RT	MBs	Soft to firm real-time with a full OS underneath

The RTOS programming model

```
/* FreeRTOS — the shape is essentially the same in every RTOS */

void sensor_task(void *arg)
{
        TickType_t next = xTaskGetTickCount();
        for (;;) {
                read_sensor();
                process();
                /* absolute deadline — no drift accumulation */
                vTaskDelayUntil(&next, pdMS_TO_TICKS(10));
        }
}

xTaskCreate(sensor_task, "sensor",
            256,            /* stack in WORDS — sized per task */
            NULL,
            3,              /* priority: higher number = higher */
            &handle);
vTaskStartScheduler();      /* never returns */

/* ISR → task communication. Note the FromISR suffix: separate
   entry points that never block, and that request a context
   switch on return.                                          */
void UART_IRQHandler(void)
{
        BaseType_t woken = pdFALSE;
        xQueueSendFromISR(rx_queue, &byte, &woken);
        portYIELD_FROM_ISR(woken);      /* switch immediately if a
                                           higher-priority task woke */
}

/* Calling the blocking variant from an ISR is the classic RTOS
   bug: it asserts, or silently corrupts the scheduler state. */
```

PER-TASK STACKS ARE THE MEMORY PROBLEM

Every task needs its own stack, sized for its worst-case call depth *plus* any interrupt frame that can nest on it. Twenty tasks at 1 KB each is 20 KB — often more than the application's data. Undersize one and you get silent corruption of the neighbouring task's stack, which presents as impossible behaviour far from the cause.

Use `uxTaskGetStackHighWaterMark()`, enable stack-overflow checking during development, and fill stacks with a known pattern so you can measure real usage rather than guess.

Worst-case execution time

```
Hard real-time requires a PROVEN upper bound on execution time —
not a measured maximum. That is genuinely hard, because modern
hardware is designed to make the AVERAGE case fast:

  caches            a miss is 100x a hit. WCET must assume misses.
  branch prediction a mispredict costs 15+ cycles. Assume worst.
  out-of-order      timing depends on surrounding instructions
  DRAM refresh      periodic stalls outside your control
  shared buses      another master can delay you
  SMT / big.LITTLE  a sibling thread steals resources; cores differ
  DVFS              the clock itself varies

  Approaches:
    STATIC ANALYSIS   model the pipeline and caches, prove a bound
                      (aiT, Bound-T). Sound, pessimistic, expensive.
    MEASUREMENT       run many times, take the max, add margin.
                      Practical, and NOT a proof.
    HYBRID            measure segments, compose analytically.

  Simplifying for analysability is usually cheaper than analysing
  complexity: disable caches on the critical path, lock cache ways,
  use TCM/scratchpad memory, pin to one core, avoid dynamic
  dispatch, and forbid recursion and unbounded loops.
```

Will the task set meet its deadlines?

```
/* RATE MONOTONIC — static priorities, shorter period = higher
   priority. Optimal among fixed-priority schemes.            */

   Sufficient condition (Liu & Layland):
        U = Σ (Cᵢ / Tᵢ)  ≤  n(2^(1/n) − 1)

        n=1  → 1.000      n=3  → 0.780
        n=2  → 0.828      n→∞  → 0.693

   Worked example, three periodic tasks:
        T1: C=1 ms,  T=4 ms   → 0.250
        T2: C=2 ms,  T=8 ms   → 0.250
        T3: C=4 ms,  T=16 ms  → 0.250
        U = 0.75  ≤  0.780    → schedulable, guaranteed.

/* EARLIEST DEADLINE FIRST — dynamic priorities.
        schedulable iff  U ≤ 1.0
   Strictly better utilisation, but behaves badly on overload:
   under RM the low-priority tasks fail first and predictably;
   under EDF a transient overload can cause a cascade of misses.

   Linux's SCHED_DEADLINE implements EDF with a constant-bandwidth
   server and an ADMISSION TEST, so requesting more than the system
   can guarantee FAILS at setup rather than silently missing
   deadlines later (OS chapter 06).                           */
```

Real-time Linux

```
# PREEMPT_RT, merged into mainline in 6.12. What it changes:
#   · most spinlocks become sleeping rt_mutexes
#   · interrupt handlers become kernel threads by default
#   · priority inheritance everywhere
#   · bounded interrupt-disable regions throughout
#   Result: typical worst case in the tens of microseconds,
#   at roughly 10–30% throughput cost.

# The full recipe for low latency on Linux:

# 1. kernel command line — isolate cores completely
isolcpus=2,3 nohz_full=2,3 rcu_nocbs=2,3 irqaffinity=0,1 \
    intel_pstate=disable processor.max_cstate=1 idle=poll

# 2. keep interrupts off the RT cores
for i in /proc/irq/*/smp_affinity_list; do echo 0-1 > $i 2>/dev/null; done
systemctl stop irqbalance

# 3. fix the clock, disable deep idle
cpupower frequency-set -g performance

# 4. in the application
mlockall(MCL_CURRENT | MCL_FUTURE);        /* no page faults    */
sched_setscheduler(0, SCHED_FIFO, &param); /* priority 80-90    */
CPU_SET(2, &mask); sched_setaffinity(...); /* pin to an isolated core */
/* pre-fault and pre-allocate EVERYTHING before the real-time loop */

# 5. measure — the only step that proves anything
cyclictest -t2 -p 90 -a 2,3 -i 200 -m -D 24h
#   report the MAX column. Run for hours, under representative load.
```

Configuration	Typical worst-case latency
Stock desktop kernel	Milliseconds, occasionally tens
`PREEMPT` (low-latency desktop)	Hundreds of µs
`PREEMPT_RT`, tuned and isolated	10–50 µs
Bare-metal RTOS on a Cortex-M	1–10 µs
Interrupt handler alone, no OS	< 1 µs

Mixed criticality

```
Modern SoCs run real-time and general-purpose workloads together.
Four ways to combine them, in increasing order of isolation:

  1. SEPARATE CORES on one SoC (AMP)
       Cortex-A runs Linux, Cortex-M/R runs the RTOS.
       Communication via shared memory + mailbox (RPMsg, OpenAMP).
       Very common: motor control beside a Linux HMI.

  2. HYPERVISOR PARTITIONING
       Xen/Jailhouse/Hafnium gives each guest dedicated cores,
       memory and devices. Strong spatial isolation.

  3. PREEMPT_RT alone
       One kernel, RT priorities plus core isolation.
       Simplest; isolation is only as good as the kernel.

  4. DUAL KERNEL (Xenomai, RTAI)
       A tiny RT co-kernel takes interrupts FIRST and runs Linux
       as its lowest-priority task. Best Linux-hosted latency;
       a separate API and real maintenance cost.

  In all four, the hard part is shared HARDWARE: the memory
  controller, caches, the interconnect. A Linux core saturating
  DRAM bandwidth can inflate the RT core's WCET even with perfect
  logical isolation. Cache partitioning and memory bandwidth
  regulation exist for exactly this.
```

How it shows up

Symptom	Cause
Deadline missed once an hour	A page fault — `mlockall` not used, or a lazily faulted stack
Latency fine on the bench, bad in the field	Untested load path; measure under representative traffic
Occasional millisecond spike on PREEMPT_RT	A long interrupt-disabled region — find it with the `irqsoff` tracer
High-priority task delayed by a low-priority one	Priority inversion; enable priority inheritance
Adding a background job broke real-time timing	Shared memory bandwidth or cache pressure, not scheduling
RTOS task corrupts another's data	Stack overflow; check high-water marks
System hangs after raising a task's priority	A `SCHED_FIFO` loop that never yields
Timing varies 3× between runs	Migrated to a LITTLE core, or DVFS changed the clock
Works with the debugger attached, fails without	The debugger changed cache or timing behaviour
Periodic task drifts over hours	Relative delays instead of absolute deadlines

Check yourself

- ✓ Explain why real-time means predictable and why the average is irrelevant.
- ✓ Distinguish hard, firm and soft real-time with an example of each.
- ✓ Name the eight contributors to interrupt-to-task latency and the two that dominate.
- ✓ Give four RTOS design choices and what each one costs.
- ✓ Explain why `FromISR` variants exist.

- ✓ Explain why WCET analysis is hard on modern hardware, and three ways to simplify it.
- ✓ Apply the rate-monotonic bound to a three-task set.
- ✓ Give the full recipe for low-latency Linux, and the one step that proves it worked.

Virtualisation, containers and isolation

Three different answers to "run this workload without letting it interfere with that one" — with three very different isolation strengths and costs. This chapter closes the OS track by making the trade-off explicit.

The isolation spectrum

```
weaker isolation, lower cost ──────────► stronger, higher cost

  process        namespace/       VM              separate
                 container                        hardware
  ┌──────┐      ┌──────────┐    ┌──────────┐    ┌──────────┐
  │ MMU  │      │ MMU +    │    │ MMU +    │    │ physical │
  │ page │      │ kernel   │    │ 2-stage  │    │ air gap  │
  │tables│      │ namespaces│   │ + EL2    │    │          │
  └──────┘      └──────────┘    └──────────┘    └──────────┘
  shared        SHARED          own kernel      own everything
  kernel        KERNEL          shared hardware

  attack surface: the shared component
  ~30M lines      ~30M lines      ~50K lines      none
  (the kernel)    (the kernel)    (hypervisor)

The whole argument between containers and VMs is that middle row:
a container escape needs one kernel bug; a VM escape needs a
hypervisor bug, and there is far less hypervisor to get wrong.
```

Virtual machines

Type	Runs on	Examples
Type 1 (bare metal)	Directly on hardware	Xen, Hyper-V, ESXi, Jailhouse, Hafnium
Type 2 (hosted)	On a host OS	VirtualBox, VMware Workstation, QEMU/TCG
Hybrid	A host kernel module *becomes* the hypervisor	**KVM** — Linux is both host OS and Type 1 hypervisor

```
The three things a hypervisor must virtualise:

  CPU     the guest kernel expects to run privileged instructions.
          Modern hardware makes this cheap: the guest runs at EL1
          natively, and only "sensitive" operations TRAP to EL2.
          (ARM VHE, Intel VT-x, AMD-V)

  MEMORY  the guest believes it owns physical memory. A SECOND
          translation stage maps guest-physical (IPA) to real
          physical, owned by the hypervisor and unchangeable by
          the guest.
              guest VA ─[stage 1, guest]─► IPA ─[stage 2, HV]─► PA
          ARM chapter 16 covers the mechanics.

  I/O     the guest programs devices that do not exist.
          Either emulate them, paravirtualise them, or hand over
          a real one (below).
```

How a trap works

```
/* The guest kernel writes to what it believes is a UART register. */

        writel(1, uart_base + UART_CR);      /* guest, at EL1 */

/* 1. stage-1 translation: guest VA → IPA, using the guest's own
      page tables. Succeeds — the guest mapped it.
   2. stage-2 lookup of that IPA in VTTBR_EL2. The hypervisor
      deliberately left it UNMAPPED.
   3. stage-2 data abort → taken to EL2, not EL1.
   4. the hypervisor reads:
         HPFAR_EL2   the faulting IPA — which device register
         ESR_EL2.ISS ISV/SAS/SRT/WnR — size, direction, and which
                     guest register held the value
   5. it emulates the UART in software and updates its model.
   6. it advances the guest's ELR_EL2 past the faulting
      instruction and ERETs.

   The guest never knows. This is exactly how QEMU/KVM presents
   virtual hardware — and why emulated I/O is slow: every register
   access is a full world switch.                              */
```

Three ways to give a VM I/O

Approach	Mechanism	Cost	Guest needs
Full emulation	Trap every register access; emulate a real device (e1000, IDE)	Very high — a trap per access	Nothing; stock drivers work
Paravirtualisation	`virtio`: a shared-memory ring plus a doorbell	Low — batched, few exits	A virtio driver
Passthrough	Assign the real device to the guest via the IOMMU	Near zero	The real driver; the device is exclusive
SR-IOV	The device presents multiple virtual functions, one per guest	Near zero	Device support

```
virtio is the same design as io_uring and every high-performance
IPC (OS chapter 08):

   guest driver                          host / device backend
   ┌──────────────┐                      ┌──────────────────┐
   │ available    │──── shared ─────────▶│  reads requests  │
   │   ring       │     memory           │                  │
   │ used ring    │◀─────────────────────│  writes results  │
   └──────┬───────┘                      └─────────▲────────┘
          │ doorbell: ONE MMIO write               │ interrupt
          │ per BATCH, not per request             │ (or polling)
          └────────────────────────────────────────┘

  Bulk data by shared memory; exits only when the peer needs
  waking. Recognise this shape — it is the answer to nearly every
  "cross a boundary at high rate" problem in systems design.
```

PASSTHROUGH REQUIRES AN IOMMU, WITHOUT EXCEPTION

A device assigned to a guest is programmed by the guest with *guest-physical* addresses. Without an IOMMU translating and constraining its DMA, the device would write those addresses as real physical addresses — scribbling over the host or another guest. The IOMMU's stage-2 translation is what makes passthrough safe, and it is the single most important reason the SMMU exists. Covered fully in the security track.

Containers

A CONTAINER IS NOT A THING

There is no container object in the Linux kernel. A "container" is an ordinary process started with a particular combination of **namespaces** (what it can see), **cgroups** (what it can use) and **capabilities/seccomp/LSM** (what it can do). Docker and Podman are tools that assemble that combination; the kernel only sees processes.

Namespace	Isolates
`mnt`	The mount table — its own filesystem view
`pid`	Process IDs — the container's init is PID 1, and it cannot see host processes
`net`	Interfaces, routes, firewall rules, ports
`uts`	Hostname and domain name
`ipc`	System V IPC and POSIX message queues
`user`	UID/GID mapping — root inside, unprivileged outside. **The one that makes rootless containers possible.**
`cgroup`	The cgroup hierarchy view
`time`	Boot and monotonic clock offsets

```
# build one by hand — this IS a container
sudo unshare --pid --net --mount --uts --ipc --fork --mount-proc \
     chroot /path/to/rootfs /bin/sh

# inspect any process's namespaces
ls -l /proc/self/ns/
readlink /proc/1/ns/net /proc/$$/ns/net    # same link = same namespace

nsenter -t PID -n -m -p /bin/sh            # join an existing one
```

```
# cgroups v2 — the resource half
mkdir /sys/fs/cgroup/myapp
echo $$              > /sys/fs/cgroup/myapp/cgroup.procs
echo "200000 1000000" > /sys/fs/cgroup/myapp/cpu.max     # 20% of one CPU
echo 512M            > /sys/fs/cgroup/myapp/memory.max
echo "259:0 rbps=10485760" > /sys/fs/cgroup/myapp/io.max

cat /sys/fs/cgroup/myapp/memory.current
cat /sys/fs/cgroup/myapp/memory.events     # oom_kill counter
cat /sys/fs/cgroup/myapp/cpu.stat          # throttling counters
```

THE CONTAINER IS STILL USING THE HOST KERNEL

`uname -r` inside a container reports the *host's* kernel. Consequences that surprise people constantly:

- A kernel exploit escapes the container. This is the entire isolation argument against containers for untrusted workloads.
- Kernel modules, `sysctl` settings and the clock are shared or host-controlled.
- `/proc/cpuinfo` and `free` show **host** values, not the cgroup limit — which is why JVMs and thread pools historically sized themselves for 64 cores inside a 2-core container. Modern runtimes read the cgroup files; older code and hand-rolled `nproc` logic do not.

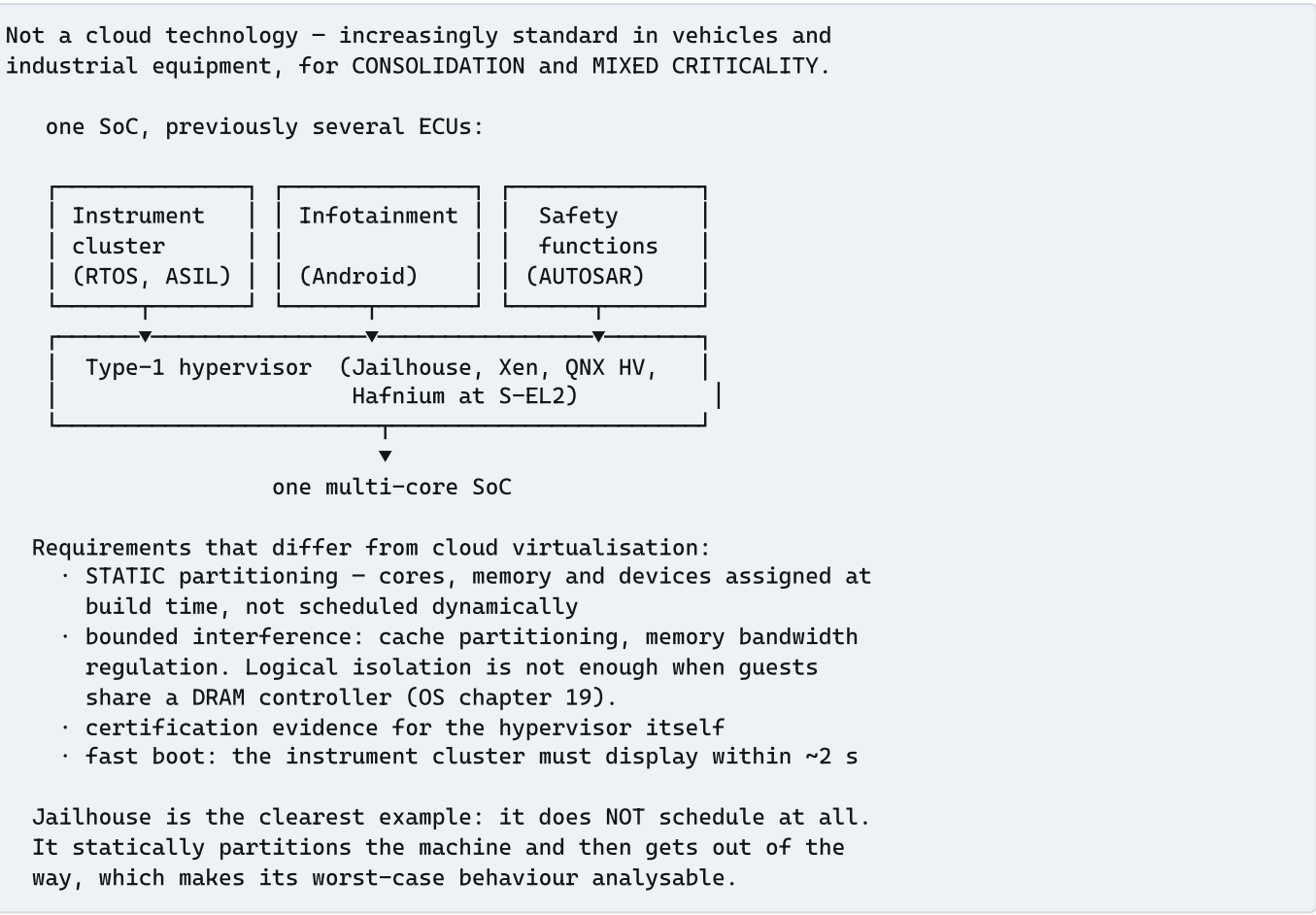
Narrowing what a container can do

```
/* Namespaces isolate what you SEE. These restrict what you can DO. */

/* 1. Capabilities — split root's power into ~40 pieces */
capsh --print
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE ...
/* CAP_SYS_ADMIN is so broad it is effectively root — never grant it */

/* 2. seccomp — filter system calls with a BPF program */
docker run --security-opt seccomp=profile.json ...
/* Docker's default profile blocks ~44 syscalls. A tight custom
   profile is one of the highest-value hardening steps available. */

/* 3. LSM — SELinux or AppArmor mandatory access control */

/* 4. no_new_privs — setuid binaries cannot regain privilege */
prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);

/* 5. USER NAMESPACES — root in the container maps to an
      unprivileged host UID. Even a container escape lands as
      nobody. This is the strongest single control, and what
      rootless Podman is built on.                          */
podman run --userns=keep-id ...
```

The middle ground

Technology	Idea	Trade-off
Kata Containers	Each container runs in a lightweight VM	Container UX, VM isolation; ~100 ms extra startup
Firecracker	A minimal VMM — ~50 K lines, five emulated devices	~125 ms boot, ~5 MB overhead. Powers AWS Lambda.
gVisor	A user-space kernel intercepting syscalls	Small kernel attack surface; syscall overhead and imperfect compatibility
WebAssembly	A sandboxed bytecode VM with capability-based I/O (WASI)	Microsecond startup, tiny images; a restricted programming model
Unikernel	The application linked with only the OS functions it needs	Minimal surface, fast boot; hard to debug and operate

Virtualisation in embedded systems



What each approach costs

	Process	Container	VM	Unikernel
Startup	~1 ms	~50–500 ms	~1–30 s (Firecracker ~125 ms)	~10 ms
Memory overhead	None	Negligible	Guest kernel + VMM, ~100 MB+	~1–10 MB
CPU overhead	None	Negligible	2–10%, more with emulated I/O	Low
Isolation strength	MMU only	Kernel-enforced; shared kernel	**Hardware-enforced**	Hypervisor-enforced
Different OS?	No	No — same kernel	**Yes**	N/A
Density	Thousands	Hundreds	Tens	Hundreds

How it shows up

Symptom	Cause
App sees 64 CPUs inside a 2-CPU container	`/proc/cpuinfo` is not namespaced; read the cgroup limits
Container OOM-killed with free host memory	The cgroup limit, not machine memory
Container CPU throttled at low utilisation	`cpu.max` quota exhausted within the period; check `cpu.stat`
Emulated network in a VM is very slow	Full device emulation; switch to virtio
Passthrough device fails to attach	No IOMMU enabled, or the device shares an IOMMU group
Guest crashes only under memory pressure	Host overcommit and ballooning

Symptom	Cause
`uname -r` in a container shows the host kernel	Correct — containers share the kernel
Container escape via a kernel bug	The shared-kernel trade-off; use a VM for untrusted code
RT guest misses deadlines despite dedicated cores	Shared memory bandwidth or cache; needs partitioning
Nested virtualisation is very slow	Each level multiplies trap cost

Check yourself

- ✓ Order process, container and VM isolation by strength and name the shared component in each.
- ✓ Name the three things a hypervisor must virtualise and how each is done on ARM.
- ✓ Walk through a trapped MMIO access from guest instruction to `ERET`.
- ✓ Compare emulation, paravirtualisation and passthrough on cost and guest requirements.
- ✓ Explain why passthrough requires an IOMMU.
- ✓ Explain what a container actually is in kernel terms.
- ✓ Name five namespaces and what each isolates.
- ✓ Give three consequences of containers sharing the host kernel.
- ✓ Say why embedded virtualisation favours static partitioning.

TRACK COMPLETE

That closes the operating systems track: what an OS is, kernel architectures, system calls, processes and threads, scheduling and context switching, IPC, synchronisation and deadlock, memory management and paging, filesystems and storage, the I/O subsystem, interrupts and time, boot, real-time, and virtualisation.

Next is the ARM architecture track — 35 chapters taking the concepts from computer architecture and operating systems down to the actual registers, page table descriptors, barriers and exception vectors of a real machine.

TRACK

Concurrency & multithreading

What actually happens when two things run at once: the hardware reordering underneath you, the primitives that tame it, and the bugs that only appear on the third Tuesday in production.

20 chapters

Why concurrency exists, and what it costs

Concurrency is not a feature you add for speed. It is a structural decision that buys you three specific things and charges you in a currency — correctness you can no longer verify by reading the code — that most teams underestimate until the first bug that only reproduces in production.

The three reasons, and only three

```
1. LATENCY HIDING - something is slow and not the CPU

     thread A: read from disk ................. 10 ms
     thread B: meanwhile, do actual work

   This is why servers are threaded. It has nothing to do
   with having more cores; a single core wins here too,
   because the alternative is a CPU sitting idle.

2. THROUGHPUT - the work genuinely divides

     8 cores, 8 independent chunks -> up to 8x

   This is the only reason that needs multiple cores, and
   the only one where Amdahl's law bites.

3. STRUCTURE - the problem IS concurrent

     a UI that must stay responsive while working
     a device that raises interrupts whenever it likes
     ten clients that arrive independently

   Here concurrency is not an optimisation. Modelling these
   sequentially produces worse code, not faster code.

IF YOUR REASON IS NOT ON THIS LIST, do not add threads.
"It felt like it should be faster" is not a reason, and
parallel code that is slower than the serial version is the
single most common outcome of a first attempt.
```

CONCURRENCY IS NOT PARALLELISM

Concurrency is a way to structure a program: several independent activities, whose execution may interleave. **Parallelism** is executing several things at literally the same instant, which needs hardware. A single-core machine can be highly concurrent and not at all parallel. Reasons 1 and 3 above are concurrency; reason 2 is parallelism. Confusing them is why people expect a thread pool to speed up a program that was waiting on a network socket.

What the speed-up is actually bounded by

```
/* Amdahl's law. If a fraction p of a program can be
   parallelised and the rest cannot:                        */

        speedup(N) = 1 / ( (1 - p) + p/N )

   p = 0.95, N = 8   ->  5.9x     (not 8x)
   p = 0.95, N = 32  ->  12.5x
   p = 0.95, N = INF ->  20x      <-- the ceiling, for ever

   p = 0.50, N = INF ->  2x

/* The 5% you did not parallelise caps you at 20x no matter
   how many cores you buy. And "the rest" includes things
   people forget to count:

     - process startup and teardown
     - reading the input, writing the output
     - the lock everyone contends on
     - the memory allocator
     - the one std::cout in the inner loop

   Gustafson's law is the optimistic counterpart: if you scale
   the PROBLEM with the machine rather than fixing it, the
   serial fraction shrinks in relative terms and the ceiling
   moves. Both are true. Amdahl describes making a fixed job
   faster; Gustafson describes doing a bigger job in the same
   time. Know which one you are actually doing.               */
```

The bill

Cost	What it looks like
Correctness stops being local	A function that is correct on its own can be wrong when two threads call it. You can no longer verify code by reading it in isolation, which is the single biggest cost and the hardest to price
Bugs become non-deterministic	A race that fires once in 10^8 iterations appears weekly in production and never on your laptop. "Cannot reproduce" becomes a normal state
Tests stop proving things	A passing test means "this interleaving worked", not "the code is correct". See chapter 20 for what to do instead
Memory traffic multiplies	Two threads writing near each other can be *slower* than one thread doing all the work — false sharing
Synchronisation is not free	An uncontended mutex is ~20 ns; a contended one that sleeps is ~1–10 µs. Lock a mutex per element of a million-element loop and you have written something slower than the serial version by an order of magnitude
Latency gets a tail	Average throughput improves; the 99.9th percentile gets worse, because now there is queueing and there are lock convoys
Debugging tools stop helping	A breakpoint changes the timing and the bug goes away. `printf` is a memory barrier and a syscall

```
/* Rough costs on a modern out-of-order core. Ratios, not
   measurements - benchmark your own platform.              */

  an L1 cache hit ........................   ~1 ns
  an uncontended atomic increment ........   ~5-20 ns
  an uncontended mutex lock + unlock .....   ~20 ns
  a cache line bouncing between two cores ..  ~50-200 ns
  a contended mutex that must sleep ......   ~1-10 us
  a thread context switch ................   ~1-5 us
  creating a thread ......................   ~10-100 us
  a condition-variable wake-up ...........   ~5-50 us

/* The two lines worth memorising:

   * creating a thread costs about as much as 10,000 mutex
     operations. This is why thread POOLS exist (chapter 17).

   * moving a cache line between cores costs more than a
     hundred arithmetic operations. This is why data layout
     decides parallel performance more often than algorithm
     choice does (chapter 16).                               */
```

THE MOST COMMON FIRST MISTAKE

Taking a loop, splitting it across eight threads, and finding it runs slower. Almost always one of three causes: the threads are contending on a shared counter or allocator; they are writing to adjacent memory and bouncing a cache line; or the work per item is smaller than the synchronisation cost. All three are covered later in this track, and all three are invisible in the source code.

The models you will meet

Model	Idea	Where it wins
Shared memory + locks	Threads see the same memory; mutexes keep them out of each other's way	The default in C, C++, Java, Rust. Fast, and the easiest to get subtly wrong
Message passing	No shared state; send values down channels	Go, Erlang, Rust channels. Whole bug classes vanish; copying costs
Event loop	One thread, a queue of callbacks, never block	Node.js, embedded superloops, GUI toolkits. No data races by construction; one slow callback stalls everything
async/await	Cooperative tasks that yield at explicit points	Rust, C#, Python. Millions of concurrent waits on a few threads (chapter 18)
Data parallel	One operation over a large array, split automatically	SIMD, OpenMP, GPUs. See the GPU track
Interrupt-driven	Hardware preempts you at any instruction	Every embedded system, whether you chose it or not

THE EMBEDDED CASE IS NOT OPTIONAL

Even a bare-metal program with no threads is concurrent: an interrupt can fire between any two instructions, and the handler shares memory with the code it interrupted. Everything in this track about races, atomicity and memory ordering applies, minus the scheduler. If you have ever written `volatile` hoping it would fix something, chapters 11 and 12 are for you.

Why it is hard, specifically

```
Two threads run this. `counter` starts at 0.

    counter = counter + 1;

One line. It compiles to three operations:

    LDR  w0, [counter]      // read
    ADD  w0, w0, #1         // modify
    STR  w0, [counter]      // write

An interleaving that loses an increment:

  thread A          thread B          counter
  --------          --------          -------
  LDR  -> 0                              0
                    LDR  -> 0            0
  ADD  -> 1                              0
                    ADD  -> 1            0
  STR                                    1
                    STR                  1     <-- should be 2

Now note three things that make this worse than it looks:

1. This interleaving is RARE. The window is nanoseconds.
   Your test passes ten million times and fails in
   production.

2. The compiler may keep `counter` in a register across the
   whole loop, so the write happens ONCE, at the end.

3. The hardware may reorder your loads and stores relative
   to other variables entirely (chapter 12).

Points 2 and 3 mean you cannot reason about concurrency by
reading the source at all. You are reasoning about a
contract between you, the compiler and the CPU - and that
contract is the memory model.
```

Rules that survive contact with real systems

1. **Do not share what you can copy.** The cheapest synchronisation is the kind you deleted. Give each thread its own data and merge at the end.

2. **Do not communicate by sharing memory; share memory by communicating.** Go's slogan, and it is right far more often than shared-mutable-state advocates admit.

3. **Immutable data needs no locks.** If nothing writes after publication, every reader is safe for free.

4. **One lock, one clearly named invariant.** If you cannot state in a sentence what a lock protects, nobody can use it correctly, including you in six months.

5. **Measure before and after.** Parallel code that is not faster is strictly worse than serial code, and you will not know which you have without a number.

6. **Prefer coarse locking first.** One big lock that is correct beats fine-grained locking that is subtly wrong. Make it correct, measure, then split only what the profile demands.

What this track assumes and what it adds

The operating systems track already introduced threads, synchronisation primitives and deadlock as OS concepts. The C++ and Rust tracks cover their respective language models. This track is the practitioner's version: what the hardware really does, which primitive to reach for and why, the failure modes that only appear under load, and how to find them deliberately rather than by waiting.

How it shows up

Symptom	Cause
Parallel version is slower than serial	Contention, false sharing, or work per item smaller than the sync cost
Speed-up plateaus at 4x on 32 cores	Amdahl — the serial fraction, often the allocator or the I/O
Works on your laptop, fails on the server	More cores, different timing, a wider window for the race
Fails only in release builds	The optimiser hoisted a load out of a loop that had no barrier
Adding a `printf` makes it work	The syscall is a barrier and a scheduling point — the bug is real and you have just hidden it
Average latency improved, p99 got much worse	Queueing and lock convoys
A counter is slightly wrong under load	Non-atomic read-modify-write
Hangs once a week with no CPU usage	Deadlock (chapter 14)
Hangs once a week at 100% CPU	Livelock, or a spin that never yields

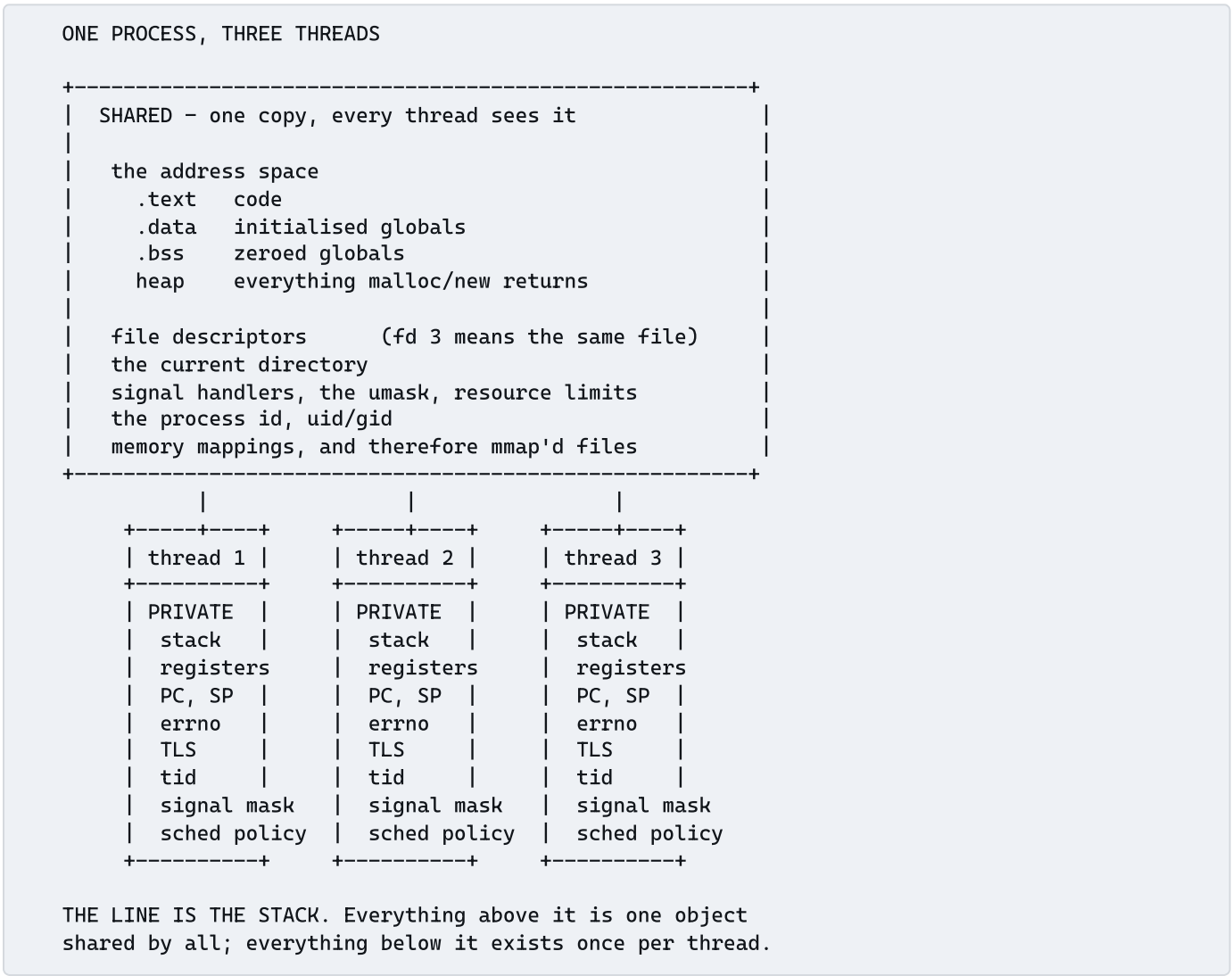
Check yourself

- ✓ Give the three legitimate reasons to introduce concurrency, and a real example of each.
- ✓ Distinguish concurrency from parallelism and say which one a single-core machine can offer.
- ✓ State Amdahl's law and compute the ceiling for `p = 0.9`.
- ✓ Name three costs of concurrency that do not appear in a profiler.
- ✓ Order these by cost: thread creation, uncontended mutex, cache-line transfer, atomic increment.
- ✓ Explain why creating a thread per work item is usually wrong.
- ✓ Show the interleaving that loses an increment, and name two further reasons source-level reasoning fails.
- ✓ Name four concurrency models and one system that uses each.
- ✓ Explain why a bare-metal program with interrupts is concurrent.
- ✓ State the six rules, and say which one you would break first under deadline pressure — and what it would cost.

Threads, and exactly what they share

Every concurrency bug is ultimately a disagreement about what is shared. This chapter draws the line precisely — what one thread can see of another's state, what it cannot, and the handful of per-thread things people assume are private and are not.

The picture



"PRIVATE" MEANS "NOT SHARED BY DEFAULT", NOT "UNREACHABLE"

A thread's stack is ordinary memory in the same address space. Pass a pointer to a local variable to another thread and it can read and write it — there is no hardware boundary. Stack-private is a *convention* the compiler relies on, not a protection. Which is why returning a pointer to a local, or handing one to a thread that outlives the frame, is a use-after-free with extra steps.

The things people get wrong

Thing	Shared or private?	Why it surprises people
`errno`	**Private**	It looks like a global. It is a macro expanding to `(*__errno_location())`, which is thread-local. This was fixed decades ago and people still expect the old behaviour
File descriptors	**Shared**	Including the *file offset*. Two threads doing `read()` on the same fd interleave and each gets part of the data. Use `pread` / `pwrite`
The heap	**Shared**	Which is why `malloc` must be thread-safe, and why it is often a contention point (chapter 16)
Signal handlers	**Shared** (the disposition)	But the signal *mask* is per-thread, and which thread receives a process-directed signal is unspecified
The current working directory	**Shared**	One thread calling `chdir()` changes every other thread's relative paths, mid-operation
The FPU / SIMD registers	**Private**	Saved and restored per thread — but on some embedded targets only if the kernel was built to do so
`static` locals in a function	**Shared**	The classic accidental global. `static char buf[256]` inside a helper makes that helper unusable concurrently
Anything returning a pointer to internal state	**Shared**	`strtok`, `asctime`, `gethostbyname`… the `_r` suffix exists for exactly this reason
C++ function-local `static` init	Shared, but **thread-safe since C++11**	The compiler emits a guard. It is safe, and it is also a hidden lock in your hot path

```
/* The static-buffer bug, which every codebase has once. */

const char *format_id(int id)
{
        static char buf[32];              /* SHARED. Disaster. */
        snprintf(buf, sizeof buf, "id-%d", id);
        return buf;
}

/* Two threads:
     A: format_id(1)  -> writes "id-1"
     B: format_id(2)  -> overwrites with "id-2"
     A: prints buf    -> "id-2"

   No crash. No sanitiser hit unless you look for it. Just
   quietly wrong output, occasionally, under load.

   THREE FIXES, in order of preference:                     */

/* 1. Caller owns the buffer. Boring, obvious, always right. */
void format_id_r(int id, char *out, size_t n)
{
        snprintf(out, n, "id-%d", id);
}

/* 2. Thread-local storage. Works, but the lifetime rules are
      now subtle and each thread pays for the buffer.        */
const char *format_id_tls(int id)
{
        static _Thread_local char buf[32];   /* C11          */
        snprintf(buf, sizeof buf, "id-%d", id);
        return buf;
}

/* 3. Allocate and make the caller free it. Correct, and now
      you have an allocation in a hot path.                  */
```

Thread-local storage

```
/* One name, one variable per thread, resolved at access. */

  C11         _Thread_local int depth;
  C++11       thread_local int depth;
  GCC/Clang   __thread int depth;          /* older, faster,
                                              no destructors */
  POSIX       pthread_key_create(&key, dtor);
              pthread_setspecific(key, p);
              pthread_getspecific(key);
  Windows     __declspec(thread) int depth;
              TlsAlloc/TlsGetValue/TlsSetValue

/* WHAT IT IS FOR

     per-thread scratch buffers
     per-thread random-number state
     a "current transaction" or "current request" pointer
     recursion depth counters, logging context
     per-thread allocator caches - this is how modern mallocs
       avoid a global lock

   WHAT IT COSTS

     * access is not free: often a register read plus an
       offset, sometimes a function call (__tls_get_addr) for
       dynamically loaded modules
     * every thread pays the memory, even threads that never
       touch it
     * destructors run at thread exit - but NOT if the thread
       is killed, and not for the main thread on some
       platforms
     * it is invisible shared-nothing state, which makes code
       hard to reason about and impossible to move between
       threads

   THE TRAP: thread_local in a THREAD POOL is per-WORKER, not
   per-TASK. Task 1 leaves state behind that task 2 sees, and
   the two tasks are unrelated. If you use TLS for request
   context in a pool, you must clear it on every task
   boundary (chapter 17).                                  */
```

STACK SIZE IS A REAL CONSTRAINT, AND THE DEFAULT IS NOT WHAT YOU THINK

Linux gives a pthread 8 MB by default (`ulimit -s`), Windows gives 1 MB reserved, and an RTOS gives you whatever you asked for — often 512 bytes. Spawn a thousand threads on Linux and you have reserved 8 GB of address space; it is virtual and mostly unbacked, but on a 32-bit target it is fatal. Meanwhile a deep recursion or a large local array on an embedded target silently runs off the end of a small stack and corrupts whatever is next in RAM. Set it deliberately with `pthread_attr_setstacksize`.

What "shared" means for the compiler

```
/* Sharing is not just about which memory two threads can
   reach. It is about what the COMPILER is allowed to assume.

   In single-threaded code the compiler may assume nothing
   else changes memory. That assumption is what makes
   optimisation possible - and it is exactly wrong under
   concurrency.                                          */

int flag = 0;

void waiter(void)
{
        while (!flag)          /* the compiler sees no writes  */
                ;              /* to `flag` in this loop, so   */
        do_work();             /* it may hoist the load OUT:   */
}

/*     if (!flag) for (;;) ;    <- an infinite loop, legally  */

/* `volatile` stops that hoist, and does NOTHING ELSE:
     - it does not make the access atomic
     - it does not order it against other accesses
     - it does not stop the CPU reordering

   volatile is for memory-mapped registers and for variables
   changed by a signal handler on the SAME thread. It is not a
   concurrency tool, and treating it as one is the second most
   common mistake in this whole track.

   The right answer is an atomic with an ordering:           */

#include <stdatomic.h>
atomic_int flag = 0;

void waiter(void)
{
        while (!atomic_load_explicit(&flag, memory_order_acquire))
                ;
        do_work();       /* now guaranteed to see the writes
                            the setter made before storing    */
}

/* Chapters 11 and 12 are entirely about that last guarantee. */
```

Threads below user space

```
The same word means different things at different levels.

USER THREAD (pthread, std::thread, HANDLE)
  scheduled by the kernel, has a kernel task behind it.
  On Linux, 1:1 - one pthread is one task_struct.

KERNEL THREAD (kthread)
  lives entirely in the kernel, no user address space.
  Workqueues, kswapd, ksoftirqd (Linux chapter 14).

INTERRUPT CONTEXT
  not a thread at all. Cannot sleep, cannot take a mutex,
  borrowed the stack of whatever it interrupted. Everything
  in this track applies, but only the primitives that do
  not sleep are legal - spinlocks, atomics.

HARDWARE THREAD (SMT / hyperthreading)
  two architectural register sets sharing one core's
  execution units. To software it is two CPUs; to the cache
  it is one. Two "parallel" threads on one core's siblings
  share L1 and can contend invisibly.

FIBER / COROUTINE / GREEN THREAD
  scheduled in USER SPACE, no kernel involvement. Cheap to
  switch, cannot use more than one core by itself, and a
  blocking syscall stalls every one of them (chapter 18).

Knowing which of these you have decides which primitives
are legal. Taking a sleeping mutex in interrupt context is
not slow - it is a kernel bug that panics.
```

How it shows up

Symptom	Cause
A function returns another thread's data	A `static` buffer inside it
Two threads reading one file each get half the data	Shared fd, shared offset — use `pread`
Relative paths break intermittently	Another thread called `chdir`
A signal is handled by the wrong thread	Process-directed signals go to any thread that has not masked them
Stack corruption on an embedded target	Default thread stack far smaller than the locals in use
Address space exhausted with a few thousand threads	8 MB reserved per thread on a 32-bit build
Request context leaks between unrelated requests	`thread_local` in a thread pool, not cleared per task
A polling loop never exits in release builds	Non-atomic flag; the load was hoisted out of the loop
`volatile` "fixed" a race, then it came back	It never fixed it — it changed the timing
Kernel panic taking a mutex in an interrupt	Sleeping primitive in non-sleepable context

Check yourself

- ✓ List what threads in one process share and what is per-thread, from memory.
- ✓ Explain why a thread's stack is "private" but not protected.

- ✓ Say whether `errno` is shared, and how that is implemented.
- ✓ Explain what goes wrong when two threads `read()` the same descriptor.
- ✓ Find the `static` buffer bug in a function and give three fixes, ranked.
- ✓ Give three good uses for thread-local storage and one trap it creates in a thread pool.
- ✓ State the default thread stack size on Linux and on Windows, and why it matters at scale.
- ✓ Explain precisely what `volatile` does and what it does not.
- ✓ Show why a non-atomic polling loop can become an infinite loop after optimisation.
- ✓ Distinguish a user thread, a kernel thread, interrupt context, an SMT sibling and a coroutine, and say which synchronisation primitives are legal in each.

Creating, joining and cancelling threads

Starting a thread is three lines in any language. Ending one correctly is where the bodies are buried: lifetimes that outlive the data they point at, detached threads still running during static destruction, and cancellation, which is almost always the wrong tool.

The same idea, four spellings

```
/* POSIX */
#include <pthread.h>

static void *worker(void *arg)
{
        int id = *(int *)arg;
        printf("worker %d\n", id);
        return (void *)(intptr_t)(id * 2);      /* the result  */
}

int main(void)
{
        pthread_t t;
        int id = 7;

        /* &id is a pointer to a LOCAL. Safe only because we
           join before main's frame goes away. See below.     */
        if (pthread_create(&t, NULL, worker, &id) != 0)
                return 1;      /* returns errno, does NOT set it */

        void *ret;
        pthread_join(t, &ret);
        printf("result %d\n", (int)(intptr_t)ret);
        return 0;
}
```

```
// C++11 and later - the same thing, with types and RAII
#include <thread>
#include <future>

int worker(int id) { return id * 2; }

int main()
{
    std::thread t(worker, 7);   // arguments are COPIED by value
    t.join();                   // or t.detach() - one or the
                                // other, or std::terminate

    // When you want a result, do not hand-roll it:
    std::future<int> f = std::async(std::launch::async, worker, 7);
    int result = f.get();       // blocks, and rethrows any
                                // exception the worker threw
}

// std::jthread (C++20) joins in its destructor and carries a
// stop_token. It is what std::thread should have been.
#include <stop_token>
std::jthread jt([](std::stop_token st) {
    while (!st.stop_requested()) { /* work */ }
});
// no join needed: the destructor requests stop, then joins.
```

```
// Rust - the lifetime problem solved by the type system
use std::thread;

let data = vec![1, 2, 3];
let h = thread::spawn(move || {          // `move` transfers
    data.iter().sum::<i32>()             // ownership in
});                                      // -- no dangling
let total: i32 = h.join().unwrap();      // possible

// Borrowing across threads needs a SCOPE, which guarantees
// the threads end before the borrow does (Rust 1.63+):
let v = vec![1, 2, 3];
thread::scope(|s| {
    s.spawn(|| println!("{:?}", &v));    // borrow is fine:
    s.spawn(|| println!("{}", v.len())); // scope joins all
});                                      // before returning
```

```
/* Windows */
#include <windows.h>
#include <process.h>

static unsigned __stdcall worker(void *arg)
{
        return (unsigned)(uintptr_t)arg * 2;
}

int main(void)
{
        /* _beginthreadex, NOT CreateThread, for anything that
           touches the CRT: it allocates the per-thread CRT
           state that errno, strtok and friends need.         */
        uintptr_t h = _beginthreadex(NULL, 0, worker,
                                     (void *)(uintptr_t)7, 0, NULL);
        WaitForSingleObject((HANDLE)h, INFINITE);

        DWORD code = 0;
        GetExitCodeThread((HANDLE)h, &code);
        CloseHandle((HANDLE)h);        /* a HANDLE is a resource;
                                          leaking it leaks the
                                          thread object          */
        return 0;
}
```

JOIN OR DETACH — THERE IS NO THIRD OPTION

Every thread you create holds resources until someone collects it. `join` waits and collects; `detach` says "collect yourself when you finish". Do neither and you leak a thread control block per thread, which on a long-running server is a slow death. C++ chose to make this loud: `~std::thread` on a joinable thread calls `std::terminate`, deliberately, so the bug is a crash rather than a leak.

The lifetime bug

```
THE MISTAKE

  void start(void)
  {
          int id = 7;                     // on start's stack
          pthread_t t;
          pthread_create(&t, NULL, worker, &id);
          pthread_detach(t);
  }                                       // frame destroyed
                                          // HERE
  worker() now reads &id -> whatever the next call put there

The window is tiny and the value is often still correct,
which is what makes it survive testing.

THREE FIXES

1. JOIN before the frame ends.
     pthread_join(t, NULL);   // id is alive until here

2. HEAP-ALLOCATE the argument, worker frees it.
     struct arg *a = malloc(sizeof *a);
     a->id = 7;
     pthread_create(&t, NULL, worker, a);
     // worker: ... free(a);
     // and if pthread_create FAILS, free it yourself

3. PASS BY VALUE where the API allows.
     pthread_create takes void*, so you can smuggle an int:
     pthread_create(&t, NULL, worker, (void *)(intptr_t)7);
     Legal, ugly, and it does not scale past one small value.

C++ picks fix 3 by default - std::thread COPIES its
arguments - which is why passing a reference needs the
explicit std::ref(), and why doing so reintroduces the bug.

Rust picks fix 1 and 2 and makes the compiler check, which
is the entire point of `move` and thread::scope.
```

Cancellation, and why you should not

```
/* POSIX cancellation exists. Almost every use of it is wrong. */

    pthread_cancel(t);          /* politely ask                */

/* What actually happens depends on two per-thread settings:

     PTHREAD_CANCEL_DEFERRED   (default)
       the thread dies at the next CANCELLATION POINT - a list
       of ~100 functions including read, write, sleep, wait,
       printf, malloc-sometimes. Between them it is immune.

     PTHREAD_CANCEL_ASYNCHRONOUS
       the thread dies at literally any instruction. Which
       means it can die holding a mutex, halfway through a
       malloc, with a file half-written. Almost nothing is
       async-cancel-safe. Do not use this.

   Even deferred cancellation leaves you needing cleanup
   handlers for every resource:                              */

static void *worker(void *arg)
{
        char *buf = malloc(1024);
        pthread_cleanup_push(free, buf);       /* runs on cancel */

        pthread_mutex_lock(&m);
        pthread_cleanup_push(unlock_helper, &m);

        read(fd, buf, 1024);                   /* cancellation
                                                  point         */
        pthread_cleanup_pop(1);                /* unlock         */
        pthread_cleanup_pop(1);                /* free           */
        return NULL;
}

/* And in C++ this is worse than it looks: on glibc, cancellation
   is implemented by throwing a special exception through your
   frames. A `catch (...)` that does not rethrow swallows it and
   the thread refuses to die.                                    */
```

USE A STOP FLAG INSTEAD. ALWAYS.

Cooperative cancellation is simpler, portable, testable and correct: an atomic flag the worker checks at points *it* chooses, so it is never interrupted mid-invariant. Every modern design does this — `std::stop_token`, Rust channels, Go's `context`, .NET's `CancellationToken`. The only thing `pthread_cancel` gives you is the ability to interrupt a blocking syscall, and the right answer there is a self-pipe, an `eventfd`, or a timeout.

```
/* The pattern to use, in full. */

struct worker {
        atomic_bool     stop;
        int             wake_fd[2];   /* self-pipe            */
        pthread_t       tid;
};

static void *run(void *arg)
{
        struct worker *w = arg;
        struct pollfd fds[2] = {
                { .fd = w->sock,        .events = POLLIN },
                { .fd = w->wake_fd[0],  .events = POLLIN },
        };

        while (!atomic_load_explicit(&w->stop, memory_order_relaxed)) {
                if (poll(fds, 2, -1) < 0) {
                        if (errno == EINTR) continue;
                        break;
                }
                if (fds[1].revents)      /* woken to stop      */
                        break;
                handle(w->sock);         /* never interrupted
                                            mid-operation       */
        }
        cleanup(w);                      /* ordinary code, no
                                            cleanup handlers    */
        return NULL;
}

void worker_stop(struct worker *w)
{
        atomic_store_explicit(&w->stop, true, memory_order_relaxed);
        write(w->wake_fd[1], "x", 1);    /* break the poll     */
        pthread_join(w->tid, NULL);      /* and WAIT for it    */
}

/* The join at the end is not optional. "I set the flag" is not
   "it has stopped", and shutting down while a worker still
   touches your data is the commonest shutdown crash there is. */
```

Shutdown order

```
The rule: a thread must not outlive anything it points at.

BAD, and extremely common:

  main() returns
    -> static destructors run
    -> a detached worker wakes up
    -> touches a logger that no longer exists
    -> crash during exit, in a stack trace that makes no
       sense because the frames are gone

THE ORDER THAT WORKS

  1. tell every worker to stop        (set flags, wake them)
  2. JOIN every worker                (now none are running)
  3. tear down what they used         (queues, loggers, pools)
  4. return from main

Steps 1 and 2 are separate for a reason: signalling all of
them before joining any of them means shutdown takes as long
as the slowest worker, not the sum of all of them.

AND NOTE: detached threads cannot be joined, so a design
with detached threads has NO correct shutdown. That is the
real argument against detach - not the leak.
```

How it shows up

Symptom	Cause
Worker sees a garbage argument, occasionally	Pointer to a caller's local; the frame is gone
`std::terminate` at a scope exit	A joinable `std::thread` destroyed
Thread count grows until the process dies	Threads neither joined nor detached
`pthread_create` returns 11 (EAGAIN)	Out of threads or address space — check `ulimit -u` and the stack size
Crash during process exit, in destructor code	A detached thread still running past static destruction
Cancelled thread never dies	It never reaches a cancellation point, or a `catch(...)` swallowed the unwind
Deadlock after cancelling a thread	It died holding a mutex
`errno` or `strtok` misbehaves on Windows	`CreateThread` instead of `_beginthreadex` — no CRT state
Shutdown takes N × the worker timeout	Signalling and joining one worker at a time instead of all, then all
Handle leak on Windows	`CloseHandle` not called on the thread handle

Check yourself

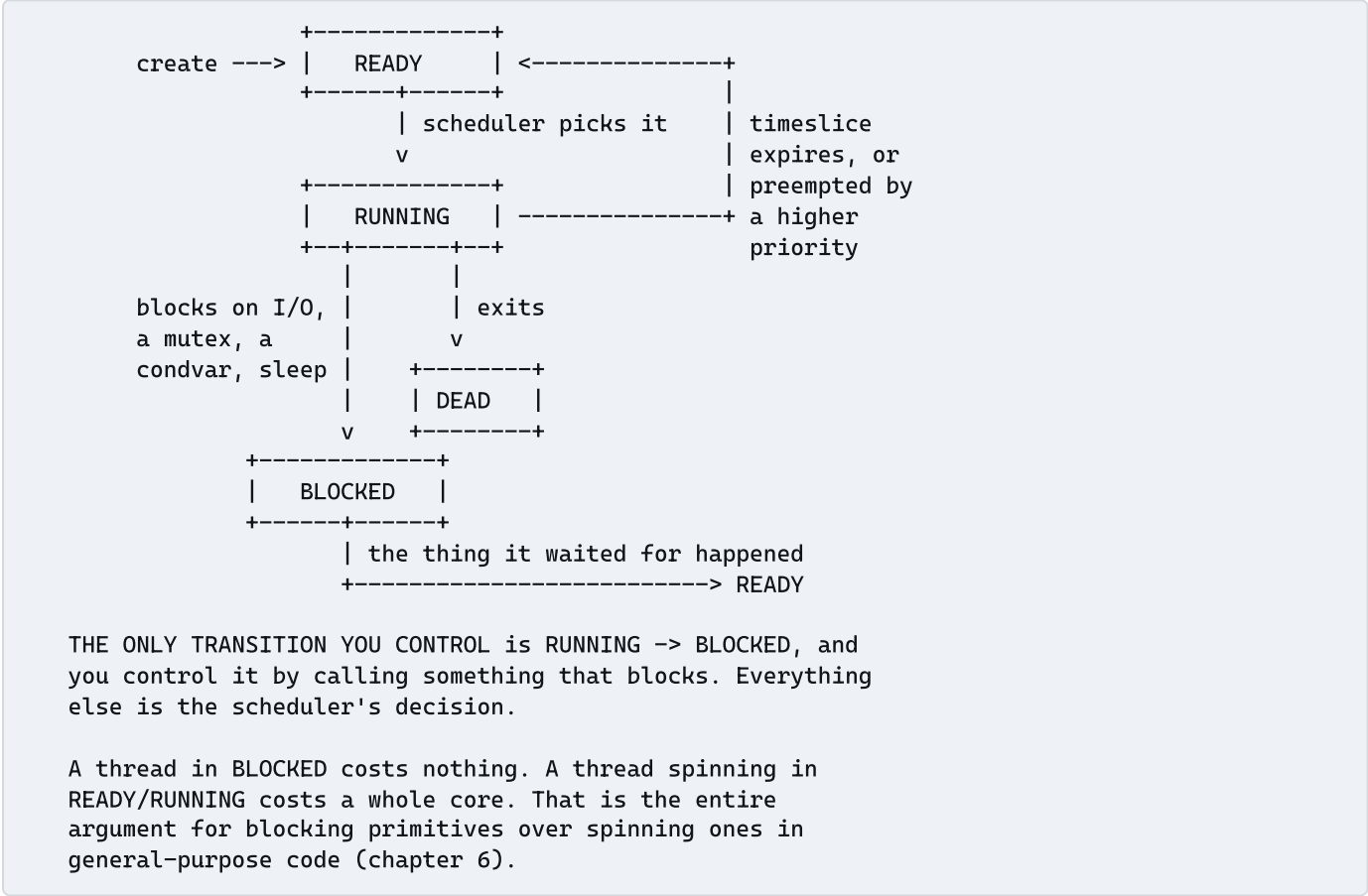
- ✓ Write thread creation and join in POSIX, C++ and Rust from memory.
- ✓ Explain why every thread must be joined or detached, and what C++ does if you do neither.
- ✓ Show the argument-lifetime bug and give three fixes, saying which language picks which.
- ✓ Explain what `std::jthread` adds over `std::thread`.
- ✓ Say why `_beginthreadex` is preferred to `CreateThread`.

- ✓ Describe deferred versus asynchronous cancellation and why the latter is unusable.
- ✓ Write the stop-flag-plus-self-pipe pattern and say why the join at the end is required.
- ✓ Give the four-step shutdown order and explain why signalling and joining are separate phases.
- ✓ Explain why a design built on detached threads has no correct shutdown.

How the scheduler decides who runs

You do not control when your threads run. Understanding the policy that does — and the four or five knobs you are actually allowed to turn — is the difference between tuning a system and superstitiously adding `sleep(0)` until the symptom moves.

The state machine



Policies you can actually choose

Policy	Behaviour	Use it when
`SCHED_OTHER` / CFS	Fair-share by weight. Every runnable thread makes progress; `nice` adjusts the share	Default. Almost always right
`SCHED_FIFO`	Real-time. Runs until it blocks or yields. A higher-priority FIFO thread preempts instantly and **never** gives the CPU back on its own	Hard deadlines, and only with a bounded work loop
`SCHED_RR`	FIFO plus a timeslice among equal priorities	Several equal-priority real-time threads
`SCHED_DEADLINE`	You declare (runtime, deadline, period); the kernel admits you or refuses	Periodic real-time work where you can state the budget
`SCHED_IDLE`	Runs only when nothing else wants the CPU	Background scrubbing, prefetch
`SCHED_BATCH`	CFS, but assumed non-interactive; fewer preemptions	Long compute jobs where throughput beats latency

```
/* Setting a real-time policy. Needs CAP_SYS_NICE or root, and
   a rlimit (RLIMIT_RTPRIO) that allows it.                    */

#include <pthread.h>
#include <sched.h>

int make_realtime(pthread_t t, int prio)
{
        struct sched_param p = { .sched_priority = prio };
        /* prio 1..99 for FIFO/RR; higher number = higher
           priority, the opposite of `nice`.                  */
        return pthread_setschedparam(t, SCHED_FIFO, &p);
}

/* THE FOOTGUN, and it is a big one:

   A SCHED_FIFO thread that busy-loops on a single-core system
   locks the machine. Nothing else - not your shell, not the
   watchdog - will ever run. Linux ships a safety valve:

     /proc/sys/kernel/sched_rt_runtime_us  = 950000
     /proc/sys/kernel/sched_rt_period_us   = 1000000

   which caps all real-time threads at 95% of each second so
   you can still get a shell. Turn that off (set runtime to -1)
   only if you are certain, and keep a serial console.

   THE RULE FOR SCHED_FIFO: the loop body must be bounded and
   must end in a blocking call. If it can spin, it will.     */
```

PRIORITY IS NOT URGENCY

Raising a thread's priority does not make it faster; it makes it *starve everything below it*. On a loaded system that is usually how you turn a latency problem into an unresponsive machine. Before reaching for priorities, check whether the thread is actually runnable — a thread blocked on I/O gains nothing from being important. And a high-priority thread that takes a lock a low-priority thread holds gets priority inversion, which is worse than the original problem.

Affinity, and why the scheduler moves you

```
/* By default a thread may run on any CPU, and the scheduler
   WILL move it to balance load. That migration costs you:
   cold L1, cold L2, cold TLB, and possibly a different NUMA
   node from your memory.                                    */

#define _GNU_SOURCE
#include <sched.h>

void pin_to(int cpu)
{
        cpu_set_t set;
        CPU_ZERO(&set);
        CPU_SET(cpu, &set);
        sched_setaffinity(0, sizeof set, &set);   /* 0 = self  */
}

# From the shell:
$ taskset -c 2,3 ./myapp          # start pinned
$ taskset -cp 2 1234              # pin a running thread
$ numactl --cpunodebind=0 --membind=0 ./myapp

# See where threads actually ran:
$ ps -eLo pid,tid,psr,comm | grep myapp
$ perf stat -e migrations,context-switches ./myapp

/* WHEN PINNING HELPS
     - real-time threads that must not be migrated
     - one thread per core, no oversubscription, cache-heavy
     - NUMA: keeping a thread on the node holding its memory
     - isolating a latency-critical thread with isolcpus /
       nohz_full so the kernel stops interrupting it

   WHEN IT HURTS
     - you pinned two hot threads to SMT siblings of one core
       and they now fight over one L1
     - you pinned everything and the scheduler can no longer
       balance a load you did not predict
     - a pinned thread blocks and its core idles while another
       core has a run queue three deep

   Measure. Pinning is one of the few tuning knobs that is
   just as likely to hurt as help.                           */
```

Yielding, sleeping, and spinning

```
/* Four ways to give up the CPU, and only two are usually right. */

  sched_yield()           "I am still ready; run someone else."
                          Under CFS this often means the
                          scheduler picks YOU again immediately,
                          because you still have the most
                          credit. As a synchronisation tool it
                          is a coin flip. Legitimate use: a
                          spin-wait's back-off path, and little
                          else.

  nanosleep(1ms)          Genuinely blocks. Precise to the
                          timer resolution, which under CFS
                          means "at least 1ms, possibly 4ms".
                          Fine for polling something slow.
                          Terrible inside a lock.

  a blocking wait         futex / condvar / poll. The thread
                          leaves the run queue entirely and is
                          woken by the event itself. THIS IS
                          THE RIGHT ANSWER for waiting on
                          another thread.

  a spin loop             Burns a core to save a few hundred
                          nanoseconds of wake-up latency. Right
                          only when the wait is provably shorter
                          than a context switch (~1-5 us), and
                          only with a pause instruction:

                              while (locked)
                                      __builtin_ia32_pause();  /* x86 */
                              /* or on ARM: */
                              __asm__ volatile("yield");
                              /* or WFE, which actually sleeps
                                 the core until an event       */

/* The pause/yield hint matters more than it looks: it stops
   the core speculating down the loop, frees the SMT sibling's
   execution units, and cuts the power draw of the spin by a
   large factor.                                              */
```

> **THE SCHEDULER IS NOT YOUR SYNCHRONISATION PRIMITIVE**
> Code that works because "thread A is higher priority so it runs first" is not synchronised; it is lucky. Add a core, change a kernel version, run under a hypervisor where your vCPU is descheduled (virtualisation chapter 8), and the assumption evaporates. If order matters, express it with a primitive that guarantees it.

Watching it happen

```
# Is my thread running, blocked, or fighting for a core?
$ ps -eLo pid,tid,stat,psr,pcpu,comm | grep myapp
#   R = running/ready   S = interruptible sleep
#   D = uninterruptible (usually disk)   T = stopped

# How often is it being switched out, and voluntarily?
$ cat /proc/$(pidof myapp)/status | grep ctxt_switches
voluntary_ctxt_switches:     4821    # it blocked - good
nonvoluntary_ctxt_switches:  91024   # it was preempted - it
                                     # wants more CPU than it
                                     # is getting

# Where did the time actually go?
$ perf sched record -- ./myapp
$ perf sched latency --sort max
# per-task: runtime, number of switches, and worst-case delay
# between "became runnable" and "actually ran"

# Scheduling latency for a real-time thread:
$ cyclictest -t1 -p 80 -i 1000 -l 100000 -m
T: 0 (1234) P:80 I:1000 C: 100000 Min: 8 Act: 12 Avg: 14 Max: 71

# A high nonvoluntary count plus high `perf sched latency` max
# means oversubscription: you have more runnable threads than
# cores. No amount of priority tuning fixes that - reduce the
# thread count.
```

How it shows up

Symptom	Cause
Machine locks up entirely	A `SCHED_FIFO` thread spinning, with the RT throttle disabled
Raising priority made latency worse	Priority inversion, or starving the thread that produces the work
Huge `nonvoluntary_ctxt_switches`	More runnable threads than cores
Throughput drops when you add threads	Oversubscription plus cache thrash from migration
Latency spikes every few hundred milliseconds	Migration, or a timer tick on a core you did not isolate
Pinning made it slower	Two hot threads on SMT siblings sharing one L1
`sched_yield` loop burns a core and never progresses	CFS re-picks the yielding thread; it is not a handoff
Works on bare metal, misses deadlines in a VM	The vCPU is a thread on the host and gets descheduled
`pthread_setschedparam` returns EPERM	No `CAP_SYS_NICE`, or `RLIMIT_RTPRIO` is zero
Timing changes completely under a debugger	The debugger is a scheduling event; the original timing was never guaranteed

Check yourself

- ✓ Draw the thread state machine and say which transition you control.
- ✓ Explain why a blocked thread is free and a spinning one is not.
- ✓ Name four scheduling policies and when each is appropriate.

- ✓ Explain what the RT throttle protects you from and why disabling it is dangerous.
- ✓ State the rule a `SCHED_FIFO` loop body must obey.
- ✓ Give two cases where pinning helps and two where it hurts.
- ✓ Explain why `sched_yield` is unreliable as a synchronisation mechanism.
- ✓ Say when a spin loop is justified, and what instruction belongs in it.
- ✓ Interpret a high `nonvoluntary_ctxt_switches` count.
- ✓ Explain why "higher priority so it runs first" is not synchronisation.

Race conditions and data races

Two different bugs share one word. A *data race* is undefined behaviour and a tool can find it. A *race condition* is a logic error that no tool can find, because the code is doing exactly what you wrote. Telling them apart tells you which of the two entirely different fixes you need.

The two bugs

```
DATA RACE - a language-level violation

  two threads access the same memory location
  at least one writes
  they are not ordered by any synchronisation

  -> UNDEFINED BEHAVIOUR in C, C++ and Rust-with-unsafe.
     Not "you get one value or the other". The compiler
     was allowed to assume this never happens, so it may
     have optimised on that assumption.

  FOUND BY: ThreadSanitizer, Helgrind, the Go race
            detector, Rust's type system at compile time.
  FIXED BY: a lock, an atomic, or not sharing.


RACE CONDITION - a logic error about ordering

  the program's correctness depends on which of several
  legal orderings actually happens.

  -> WELL-DEFINED BEHAVIOUR. Every individual operation is
     properly synchronised. The SEQUENCE is wrong.

  FOUND BY: thinking, review, stress tests, and production.
  FIXED BY: making the composite operation atomic, or
            removing the assumption about order.


EVERY DATA RACE IS A RACE CONDITION.
NOT EVERY RACE CONDITION IS A DATA RACE.

And the second kind is the one that survives to production,
precisely because your tools said the code was clean.
```

```
/* A DATA RACE. TSan finds this in seconds. */

int counter;                            /* plain int          */

void *worker(void *_) {
        for (int i = 0; i < 100000; i++)
                counter++;              /* unsynchronised RMW  */
        return NULL;
}
/* Fix: make it atomic, or hold a lock. */


/* A RACE CONDITION with NO data race. TSan says nothing. */

std::mutex m;
std::map<std::string, int> cache;

int get_or_create(const std::string &k)
{
        {   std::lock_guard<std::mutex> g(m);
            auto it = cache.find(k);
            if (it != cache.end())
                    return it->second;      /* properly locked  */
        }                                   /* LOCK RELEASED    */

        int v = expensive_compute(k);       /* slow, unlocked   */

        {   std::lock_guard<std::mutex> g(m);
            cache[k] = v;                   /* properly locked  */
            return v;
        }
}

/* Every access to `cache` is under the mutex. No data race.
   But two threads calling get_or_create("x") simultaneously
   both miss, both compute, and both insert. If
   expensive_compute has a side effect - allocating an id,
   opening a connection, charging a card - you have just done
   it twice.

   THE BUG IS THE GAP between the two locked regions. */
```

> **THE TEST: COULD AN ADVERSARY SCHEDULE THIS?**
> Imagine an adversary who chooses when every thread is preempted, at instruction granularity, with full knowledge of your code. Would the program still be correct? Not "is it likely" — the window in the example above is milliseconds wide, and under load it is hit constantly. Any reasoning of the form "the chance of that is tiny" is a bug you have decided not to fix.

The shapes races take

Shape	Example	Fix
Check-then-act	`if (!exists(f)) create(f);`	One atomic operation: `O_CREAT|O_EXCL`, `try_emplace`, compare-and-swap
Read-modify-write	`x = x + 1`	An atomic RMW, or a lock
Lost update	Two threads read a record, both edit, both save	Optimistic concurrency: a version number checked on write

Shape	Example	Fix
Time-of-check to time-of-use	`access()` then `open()`	Never re-resolve; use the handle you already checked. A security bug, not just a correctness one
Publication race	Set a pointer, then initialise the object	Initialise fully, then publish with a release store (chapter 12)
Iterator invalidation	One thread iterates while another inserts	Hold the lock for the whole iteration, or copy, or use a concurrent container
Lifetime race	Thread B uses an object thread A is destroying	Shared ownership (`shared_ptr` , refcount) or a join before teardown
Lazy init	Double-checked locking, written by hand	`std::call_once` , a function-local `static` , or an acquire/release pair done properly

Double-checked locking, the canonical wrong answer

```
/* THE BROKEN VERSION. Published in books for a decade. */

Singleton *instance;                    /* plain pointer      */
std::mutex m;

Singleton *get(void)
{
        if (!instance) {                /* 1. unsynchronised
                                              read            */
                std::lock_guard<std::mutex> g(m);
                if (!instance)
                        instance = new Singleton();   /* 2. */
        }
        return instance;
}

/* TWO independent defects:

   (a) Line 1 reads `instance` with no synchronisation while
       line 2 writes it. That is a DATA RACE: undefined
       behaviour, full stop.

   (b) Even if the read were somehow fine, `new Singleton()`
       is three steps:
              allocate
              construct
              assign to instance
       and nothing forces that order. A compiler or CPU may
       publish the pointer BEFORE the constructor has run.
       Another thread sees non-null, returns it, and uses a
       half-built object.

   Defect (b) is why this is famous: the code looks right, the
   bug is invisible in the source, and it appears only on
   weakly-ordered hardware like ARM - so it ships from an x86
   developer's laptop and fails on the phone.               */


/* CORRECT, and simplest: */
Singleton &get(void)
{
        static Singleton instance;   /* C++11 guarantees this
                                        is thread-safe and
                                        initialised exactly
                                        once                  */
        return instance;
}

/* CORRECT, when you need the pointer form: */
std::atomic<Singleton *> instance{nullptr};
std::mutex m;

Singleton *get(void)
{
        Singleton *p = instance.load(std::memory_order_acquire);
        if (!p) {
                std::lock_guard<std::mutex> g(m);
                p = instance.load(std::memory_order_relaxed);
                if (!p) {
                        p = new Singleton();
```

```
                        /* release: everything the constructor
                           wrote is visible to any thread that
                           acquires this pointer.             */
                        instance.store(p, std::memory_order_release);
                }
        }
        return p;
}
```

Finding them on purpose

```
# ThreadSanitizer - finds DATA races, not race conditions.
# Roughly 5-15x slower, 5-10x more memory. Worth every cycle.
$ gcc -fsanitize=thread -g -O1 app.c -o app
$ ./app
WARNING: ThreadSanitizer: data race (pid=4211)
  Write of size 4 at 0x5586f4a0 by thread T2:
    #0 worker app.c:12
  Previous read of size 4 at 0x5586f4a0 by thread T1:
    #0 worker app.c:12
  Location is global 'counter' of size 4

# It only reports races it OBSERVES. Code paths not executed
# are not checked, so run it against your real workload.

$ valgrind --tool=helgrind ./app       # slower, no rebuild
$ go test -race ./...                  # built in
$ cargo +nightly build -Z sanitizer=thread

# For RACE CONDITIONS, no tool. What actually works:
#   * run with more threads than cores, so preemption is
#     constant rather than rare
#   * inject delays at suspicious points and see what breaks
#   * stress in a loop for hours; log the interleaving
#   * model-check the protocol (TLA+, Spin) if it is critical

# Widening the window deliberately - the cheapest technique
# in this whole chapter:
        auto it = cache.find(k);
        if (it != cache.end()) return it->second;
+       std::this_thread::sleep_for(std::chrono::milliseconds(50));
        int v = expensive_compute(k);
# If the bug is a gap between two locked regions, a sleep in
# the gap turns "once a week" into "every single time".
```

> **A CLEAN TSAN RUN IS NOT A CLEAN PROGRAM**
>
> ThreadSanitizer proves the absence of data races *on the paths it executed*. It says nothing about race conditions, nothing about paths you did not run, and nothing about ordering bugs across processes or against hardware. Treat it as a lint that catches an important, mechanical class of bug — not as a proof.

How it shows up

Symptom	Cause
A counter is slightly low under load	Non-atomic read-modify-write; increments lost
A cached value is computed twice	Check-then-act with an unlocked gap

Symptom	Cause
A user is charged twice	The same gap, with a side effect in it
Crash using a freshly published object	Publication race — the pointer was visible before the constructor finished
Fails on ARM, never on x86	Weaker memory ordering exposes a publication or reordering bug
Fails only with optimisation on	The compiler exploited the undefined behaviour a data race permitted
Iterator crash in a container that is "locked"	The lock was released between obtaining the iterator and using it
Use-after-free during shutdown	Lifetime race — no join before teardown
A file is created twice, or a symlink attack succeeds	TOCTOU between check and use
TSan is clean and the bug persists	It is a race condition, not a data race — different bug, different fix

Check yourself

- ✓ Define a data race and a race condition precisely, and say which tools find which.
- ✓ Explain why a data race is undefined behaviour rather than "you get one of two values".
- ✓ Write a race condition that contains no data race.
- ✓ State the adversarial-scheduler test and apply it to a piece of your own code.
- ✓ Name six shapes races take and give the fix for each.
- ✓ Give both defects in hand-written double-checked locking.
- ✓ Explain why the double-checked locking bug ships from x86 and fails on ARM.
- ✓ Write the two correct forms of lazy initialisation in C++.
- ✓ Explain what a clean ThreadSanitizer run does and does not prove.
- ✓ Describe the delay-injection technique and why it is so effective on gap bugs.

Mutual exclusion from first principles

Before you can use a mutex well you have to know what one is made of. This chapter builds mutual exclusion up from nothing — the software-only algorithms that almost work, the hardware instruction that makes it possible, and the futex trick that makes it fast.

What we are trying to buy

```
A correct mutual-exclusion protocol needs four properties.
Dropping any one of them is a real bug with a real name.

1. MUTUAL EXCLUSION
     at most one thread in the critical section at a time.
     Without it: the thing you were trying to prevent.

2. PROGRESS  (deadlock freedom)
     if nobody is in the critical section and somebody
     wants in, somebody gets in.
     Without it: deadlock (chapter 14).

3. BOUNDED WAITING  (starvation freedom)
     a thread that wants in gets in after a bounded number
     of other entries.
     Without it: starvation - a thread waits for ever
     while others take turns ahead of it.

4. NO ASSUMPTIONS ABOUT SPEED
     correctness must not depend on relative thread speeds,
     core counts, or the scheduler.
     Without it: works on your laptop.

Note what is NOT on the list: fairness in the FIFO sense,
and performance. Most real mutexes deliberately give up
strict fairness to go faster (see barging, below).
```

Software-only: interesting, and not what you use

```
/* PETERSON'S ALGORITHM - two threads, no special instruction.
   Correct on a sequentially consistent machine.            */

atomic_bool want[2] = { false, false };
atomic_int  turn    = 0;

void lock(int me)
{
        int other = 1 - me;
        want[me] = true;        /* I want in                  */
        turn = other;           /* but you go first           */
        while (want[other] && turn == other)
                ;               /* spin                       */
}

void unlock(int me) { want[me] = false; }

/* WHY IT WORKS: the `turn` write is a deliberate deference.
   If both threads arrive together, both set turn, and the
   LAST write wins - so exactly one of them sees turn pointing
   away from itself and proceeds.

   WHY YOU WILL NEVER USE IT:

   1. It is for exactly two threads. Extending it (Lamport's
      bakery algorithm) needs O(n) storage and O(n) scanning
      per acquisition.

   2. It spins. Every waiter burns a core.

   3. THE REAL KILLER: it is only correct under sequential
      consistency, and no real CPU provides that. On ARM or
      x86 the store to `want[me]` and the load of
      `want[other]` can be reordered, and both threads enter.
      Making it correct needs full barriers in both places -
      at which point you are paying more than a real mutex
      costs.

   It is on the exam because it proves mutual exclusion is
   possible without hardware help. It is not in your code
   because hardware help is cheaper.                        */
```

The instruction that changes everything

```
/* What we need is one indivisible read-modify-write. Every
   architecture provides one; they differ in shape.        */

/* x86: exchange, with an implicit lock prefix. */
        xchg    eax, [lock]        ; atomically swap

/* x86: compare-and-swap, the general form. */
        lock cmpxchg [ptr], ebx    ; if [ptr]==eax, [ptr]=ebx

/* ARM: LOAD-LINKED / STORE-CONDITIONAL. No single atomic RMW -
   instead a pair, where the store FAILS if anyone touched the
   line in between.                                          */
retry:  ldaxr   w1, [x0]           ; load-acquire exclusive
        cmp     w1, #0
        b.ne    retry              ; already locked, spin
        mov     w2, #1
        stlxr   w3, w2, [x0]       ; store-release exclusive
        cbnz    w3, retry          ; w3 != 0 means we lost the
                                   ; reservation - try again

/* ARMv8.1-A LSE adds true single-instruction atomics, which
   scale far better under contention because they can be done
   at the point of coherency instead of bouncing the line:   */
        swpal   w1, w2, [x0]       ; swap
        casal   w1, w2, [x0]       ; compare-and-swap
        ldaddal w1, w2, [x0]       ; atomic add

/* In C, you write none of this: */
        while (atomic_exchange_explicit(&lock, 1,
                                        memory_order_acquire))
                ;                          /* spin lock       */
        /* critical section */
        atomic_store_explicit(&lock, 0, memory_order_release);
```

> **WHY ACQUIRE AND RELEASE, SPECIFICALLY**
>
> The lock is not just about exclusion — it is about *visibility*. Acquiring with `memory_order_acquire` guarantees that everything the previous holder did before releasing is visible to you afterwards. Releasing with `memory_order_release` guarantees your writes are visible to the next acquirer. A "mutex" that excluded correctly but gave no ordering guarantee would still let you read stale data. Chapter 12 is this idea in full.

A spinlock, properly written

```
/* The naive version, and why each fix matters. */

/* v1: correct, and terrible under contention. */
void lock_v1(atomic_flag *l) {
        while (atomic_flag_test_and_set_explicit(l, memory_order_acquire))
                ;
}
/* Every waiter hammers the cache line with an atomic RMW.
   With 16 waiters the line ping-pongs 16 ways and the HOLDER
   cannot even write its own data. Contention makes it slower
   the more threads you add.                                  */

/* v2: test-and-test-and-set. Spin on a plain LOAD, which can
   sit in the local cache in Shared state and generate no bus
   traffic at all. Only attempt the expensive RMW when it
   looks free.                                                */
void lock_v2(atomic_bool *l) {
        for (;;) {
                if (!atomic_exchange_explicit(l, true,
                                              memory_order_acquire))
                        return;
                while (atomic_load_explicit(l, memory_order_relaxed))
                        cpu_relax();          /* pause / yield  */
        }
}

/* v3: add back-off, so the thundering herd on release does not
   all collide at once. */
void lock_v3(atomic_bool *l) {
        unsigned backoff = 1;
        for (;;) {
                if (!atomic_exchange_explicit(l, true,
                                              memory_order_acquire))
                        return;
                while (atomic_load_explicit(l, memory_order_relaxed)) {
                        for (unsigned i = 0; i < backoff; i++)
                                cpu_relax();
                        if (backoff < 1024) backoff *= 2;
                }
        }
}

/* v4: give up and sleep, because spinning past a few hundred
   cycles is worse than a context switch. That is a MUTEX. */

/* And note none of v1-v3 is FAIR: a thread that just released
   can immediately re-acquire while an older waiter is still
   backing off. A TICKET LOCK fixes that:                     */

struct ticket { atomic_uint next, now_serving; };

void ticket_lock(struct ticket *t) {
        unsigned me = atomic_fetch_add_explicit(&t->next, 1,
                                        memory_order_relaxed);
        while (atomic_load_explicit(&t->now_serving,
                                    memory_order_acquire) != me)
                cpu_relax();
}
void ticket_unlock(struct ticket *t) {
        atomic_fetch_add_explicit(&t->now_serving, 1,
```

```
                                    memory_order_release);
}
/* Strict FIFO, and every waiter still spins on one shared
   line. MCS and CLH locks fix that too by giving each waiter
   its own line to spin on - which is what the kernel uses for
   contended spinlocks.                                      */
```

> **NEVER SPIN WHERE YOU CAN BE PREEMPTED**
> A spinlock assumes the holder is *running* and will release shortly. If the holder is descheduled — because you are in user space, or in a VM whose vCPU was taken away (virtualisation chapter 8) — every waiter burns a full timeslice waiting for a thread that cannot run. This is lock-holder preemption, and it is why spinlocks belong in the kernel with preemption disabled, not in application code.

The futex: spin a little, then sleep

```
   A real mutex wants both: no syscall in the fast path, and
   no spinning in the slow one. Linux's futex gives exactly
   that by making the KERNEL involved only on contention.

   state: 0 = free, 1 = locked, 2 = locked and someone waits

   LOCK
     CAS 0 -> 1 succeeds?          done. NO SYSCALL AT ALL.
                                   This is the common case and
                                   it costs ~20 ns.
     otherwise:
       spin briefly (adaptive)
       still locked?
         set state to 2  ("I am going to sleep")
         futex(FUTEX_WAIT, addr, 2)
           kernel: "is *addr still 2? then queue me and sleep"
           - the recheck inside the kernel is what closes the
             race between deciding to sleep and sleeping

   UNLOCK
     if state was 1:  store 0.     NO SYSCALL.
     if state was 2:  store 0, then
                      futex(FUTEX_WAKE, addr, 1)

   THE WHOLE TRICK is that "2" records the existence of
   waiters, so the unlock path knows whether a syscall is
   needed. An uncontended mutex never enters the kernel in
   either direction.

   Windows: WaitOnAddress / WakeByAddressSingle.
   macOS:   __ulock_wait / os_unfair_lock.
   Same idea everywhere.
```

How it shows up

Symptom	Cause
Hand-rolled lock works on x86, fails on ARM	Missing barriers; the algorithm assumed sequential consistency
Spinlock gets slower as threads are added	Test-and-set hammering one cache line — needs test-and-test-and-set
One thread never acquires while others cycle	Barging; the lock is unfair. Use a ticket lock

Symptom	Cause
A core is pinned at 100% doing nothing	Spinning on a lock whose holder is descheduled
Latency spikes to whole milliseconds	Lock-holder preemption, or spinning past the point where sleeping was cheaper
Reading stale data despite holding the lock	Lock implemented with relaxed atomics — exclusion without ordering
Wake-up is missed and a thread sleeps for ever	The check-then-sleep race; futex's in-kernel recheck exists to prevent exactly this
Performance collapses at exactly N threads	N is the point where the contended path starts entering the kernel

Check yourself

- ✓ State the four properties a mutual-exclusion protocol must have, and name the bug you get without each.
- ✓ Explain how Peterson's algorithm achieves exclusion, and the three reasons it is unusable.
- ✓ Name the atomic primitive on x86 and the ARM equivalent, and say why LL/SC can fail spuriously.
- ✓ Explain what LSE atomics improve over LL/SC under contention.
- ✓ Explain why a lock needs acquire and release semantics, not just exclusion.
- ✓ Improve a naive test-and-set spinlock twice and justify each change.
- ✓ Write a ticket lock and say what it fixes and what it does not.
- ✓ Explain lock-holder preemption and why spinlocks are a kernel construct.
- ✓ Describe the three futex states and say why the "2" state exists.
- ✓ Explain what the kernel-side recheck in `FUTEX_WAIT` prevents.

Mutexes in practice

The primitive is simple; using it well is a set of habits. What a lock protects, how long you hold it, whether you can take two, and never — under any circumstances — releasing it by hand on an error path.

A lock protects an invariant, not a variable

```
THE QUESTION THAT PREVENTS MOST LOCK BUGS

  "What is true when this lock is not held?"

If you cannot answer in one sentence, the lock is
decoration.

BAD: "m protects `count`."
     Then what protects the relationship between `count`
     and the buffer it counts? Two locks, two invariants,
     and a window between them.

GOOD: "m protects the queue: `head`, `tail`, `count` and
       the elements between head and tail are mutually
       consistent whenever m is not held."

Now every rule follows from the sentence:
  - you may not read `count` without m, because the
    invariant only holds while m is unheld and you are
    observing it mid-update otherwise
  - you must restore ALL of it before unlocking
  - a function that returns a pointer INTO the queue has
    leaked the invariant past the lock, and is wrong

WRITE THE SENTENCE IN A COMMENT NEXT TO THE LOCK. It is the
single highest-value comment in concurrent code, and the
one people never write.
```

```
/* What that looks like in a header. */

struct queue {
        /* @lock protects every field below it: head, tail and
         * count are mutually consistent, and elements
         * [head, tail) are initialised, whenever @lock is not
         * held. Callers must not retain pointers to elements
         * after releasing @lock.
         */
        pthread_mutex_t lock;
        struct item    *head, *tail;
        size_t          count;
};

/* The kernel writes this as a lockdep-checkable annotation;
   C++ has an analyser attribute for it:                    */

class Queue {
    std::mutex m_;
    size_t     count_ GUARDED_BY(m_);      /* clang thread
                                              safety analysis */
public:
    size_t size() { std::lock_guard<std::mutex> g(m_);
                    return count_; }
};
/* -Wthread-safety turns "read count_ without m_" into a
   COMPILE ERROR. It is underused and nearly free.          */
```

Never unlock by hand

```
/* THE BUG, in every codebase that uses manual unlock. */

int process(struct queue *q, int x)
{
        pthread_mutex_lock(&q->lock);

        if (x < 0)
                return -EINVAL;              /* LOCK LEAKED   */

        if (q->count == 0) {
                pthread_mutex_unlock(&q->lock);
                return -EAGAIN;              /* fine, for now */
        }

        do_work(q);                          /* if this throws
                                                or longjmps:
                                                LOCK LEAKED   */
        pthread_mutex_unlock(&q->lock);
        return 0;
}
/* Someone adds a fourth early return in six months, and the
   process deadlocks the next time that path is taken.      */

/* C++: RAII. The destructor cannot be forgotten and runs on
   exceptions too. */
int process(Queue &q, int x)
{
        std::lock_guard<std::mutex> g(q.m);   /* one line     */
        if (x < 0)      return -EINVAL;       /* unlocked      */
        if (q.empty())  return -EAGAIN;       /* unlocked      */
        do_work(q);                           /* throws? still
                                                 unlocked      */
        return 0;
}
/* std::scoped_lock (C++17) is the same and takes N mutexes
   deadlock-free. Prefer it: std::lock_guard has no advantage. */

/* Rust: the data is INSIDE the mutex, so there is no way to
   touch it without locking, and the guard unlocks on drop. */
let q = Mutex::new(Queue::new());
{
    let mut g = q.lock().unwrap();
    g.push(item);
}   // unlocked here, always

/* C, with GCC/Clang: */
static inline void unlock_p(pthread_mutex_t **m) {
        pthread_mutex_unlock(*m);
}
#define LOCKED(mtx) \
        pthread_mutex_t *__l __attribute__((cleanup(unlock_p))) \
                = &(mtx); pthread_mutex_lock(__l)

int process(struct queue *q, int x)
{
        LOCKED(q->lock);
        if (x < 0) return -EINVAL;      /* unlocked correctly */
        ...
}
```

RUST'S REAL CONTRIBUTION

Other languages give you a lock and a separate piece of data and trust you to pair them. Rust makes `Mutex<T>` *own* the `T`, so accessing the data without holding the lock is not a bug you might make — it is code that does not compile. That single design choice removes the entire "forgot to take the lock" category, which is most of this chapter.

Which mutex

Kind	What it adds	Cost
Normal / fast	Nothing. Undefined behaviour if you relock or unlock someone else's	Cheapest. The default
Error-checking	Returns `EDEADLK` on self-relock, `EPERM` on foreign unlock	A branch. Use in debug builds
Recursive	The owner may lock it again; counts	An owner field and a counter — and see the warning below
Timed	`pthread_mutex_timedlock`, `try_lock_for`	Nothing extra when uncontended
Robust	If the owner dies, the next locker gets `EOWNERDEAD` and may recover	Only for shared-memory IPC mutexes
Priority-inheriting	Boosts the holder to the highest waiter's priority	Essential for real-time (chapter 15)
Adaptive	Spins briefly before sleeping	Great for short critical sections, wasteful for long ones
Shared / process-shared	Lives in shared memory across processes	Must be robust, or a crash wedges everyone

RECURSIVE MUTEXES ARE A DESIGN SMELL

They exist so that a function holding a lock can call another function that takes the same lock. That situation means you do not know your own locking structure. The real cost is that the invariant is only guaranteed at the *outermost* unlock — so the inner function sees the data mid-update and cannot tell. The fix is to split each public function into a locking wrapper and an unlocked `_locked` worker, which also documents which is which.

```
/* The pattern that removes the need for recursion. */

static void queue_push_locked(struct queue *q, struct item *i)
{
        /* caller MUST hold q->lock. Name says so. */
        ...
}

void queue_push(struct queue *q, struct item *i)
{
        LOCKED(q->lock);
        queue_push_locked(q, i);
}

void queue_push_many(struct queue *q, struct item **v, size_t n)
{
        LOCKED(q->lock);                    /* taken ONCE     */
        for (size_t k = 0; k < n; k++)
                queue_push_locked(q, v[k]);
}
```

How much to protect, and for how long

```
/* TOO COARSE: one lock for everything.
   Correct, simple, and it serialises your whole program. */

/* TOO FINE: a lock per element.
   More memory than the data, more time in locking than in
   work, and now you can deadlock by taking two. */

/* HOLD TIME is usually the bigger lever than granularity: */

/* BAD - I/O inside the critical section. */
{
        std::lock_guard<std::mutex> g(m);
        auto v = cache[k];
        write_to_disk(v);            /* 10 ms, holding the lock,
                                        blocking every other
                                        thread                 */
}

/* GOOD - copy out, then work. */
Value v;
{
        std::lock_guard<std::mutex> g(m);
        v = cache[k];                /* microseconds          */
}
write_to_disk(v);                    /* nobody is blocked     */

/* THE RULES

   1. Never call into unknown code while holding a lock. A
      callback, a virtual function, an allocator or a logger
      may take another lock and deadlock you (chapter 14).
   2. Never do I/O or sleep while holding a lock.
   3. Never allocate while holding a hot lock if you can
      avoid it - malloc has its own locks.
   4. Copy small things out and release; do not hold a lock
      across expensive work.
   5. If the critical section is one atomic-sized update, use
      an atomic instead (chapter 11).                        */
```

Measuring contention

```
# Is the lock actually contended, or are you guessing?

$ perf lock record ./app && perf lock report
 Name   acquired  contended  avg wait    max wait   total wait
 &q->lock  184291      41082   1.42 us   9.81 ms     58.3 ms

$ perf stat -e 'syscalls:sys_enter_futex' -p $(pidof app)
# a large futex count means real contention: the fast path is
# not being taken

# In the kernel:
$ echo 1 > /proc/sys/kernel/lock_stat
$ cat /proc/lock_stat

# The number that matters is CONTENDED / ACQUIRED. Below a few
# percent, leave it alone. Above ten percent, either shrink
# the critical section or split the lock - and measure again,
# because splitting a lock often just moves the queue.
```

How it shows up

Symptom	Cause
Deadlock on an uncommon error path	Manual unlock missed on an early return
Deadlock after an exception	Manual unlock skipped by the unwind — use RAII
Data corruption despite "everything is locked"	Two locks, two invariants, and a gap between them
A getter returns a torn value	Read without the lock because "it is just one field"
Crash using a pointer obtained under a lock	The invariant escaped the critical section
Throughput collapses under load	Long hold times — usually I/O or allocation inside the lock
Deadlock when a callback fires	Unknown code called while holding a lock
Works, then deadlocks after a refactor adds a call	A recursive-mutex design that was hiding the structure
`EDEADLK` from `pthread_mutex_lock`	Self-relock on an error-checking mutex — a real bug it just caught for you
Everything hangs after a process crashes	Process-shared mutex that was not robust

Check yourself

- ✓ State the sentence that should be written next to every lock, and write one for a real structure.
- ✓ Show the manual-unlock bug and give the fix in C, C++ and Rust.
- ✓ Explain what Rust's `Mutex<T>` makes impossible that other languages permit.
- ✓ Name six kinds of mutex and what each adds.
- ✓ Explain why recursive mutexes are a design smell and give the refactor that removes them.
- ✓ Give five rules about what must never happen inside a critical section.
- ✓ Explain why hold time usually matters more than lock granularity.
- ✓ Measure lock contention and say what ratio would make you act.
- ✓ Explain why splitting a contended lock sometimes changes nothing.

Condition variables and waiting for a state

A mutex answers "may I touch this?". A condition variable answers "has it become what I need yet?" — without burning a core asking. It is the primitive people most often get wrong, and every mistake comes from forgetting that the condition variable itself holds no state at all.

The problem it solves

```
You want: "wait until the queue is non-empty."

ATTEMPT 1 - spin
    while (queue_empty(q))
            ;
  Burns a core. On one core, may never progress.

ATTEMPT 2 - spin with a lock
    for (;;) {
            lock(m);
            if (!queue_empty(q)) break;
            unlock(m);
    }
  Burns a core AND hammers the lock, so the producer
  cannot get in to add anything.

ATTEMPT 3 - poll with a sleep
    for (;;) {
            lock(m);
            if (!queue_empty(q)) break;
            unlock(m);
            sleep_ms(1);
    }
  Works. Adds up to 1 ms of latency to every item and wakes
  up pointlessly all night. This is what people ship when
  they do not know about condition variables.

WHAT YOU ACTUALLY WANT
    lock(m);
    while (queue_empty(q))
            cond_wait(cv, m);      /* atomically: unlock m,
                                      sleep, and relock m
                                      on wake              */
    take_item(q);
    unlock(m);

Zero CPU while waiting, and wake-up latency measured in
microseconds.
```

THE ATOMICITY THAT MAKES IT WORK

`cond_wait` must release the mutex and begin waiting *as one indivisible step*. If there were any gap, a producer could add an item and signal in that gap — and the consumer would then go to sleep waiting for an event that already happened, for ever. This is the **lost wake-up**, and avoiding it is the entire reason the API takes the mutex as an argument rather than you unlocking it yourself.

The three rules

```
/* RULE 1: ALWAYS wait in a loop, never an if. */

        lock(m);
        while (!condition)          /* while, not if           */
                cond_wait(cv, m);
        /* condition is true here */
        unlock(m);

/* Three separate reasons, any one of which is sufficient:

   (a) SPURIOUS WAKE-UPS. POSIX explicitly permits cond_wait
       to return without any signal at all. This is not a bug
       in the implementation; it is licensed by the standard
       so that implementations can be fast (and it genuinely
       happens on signal delivery).

   (b) BARGING. Between the signaller's signal and your
       actually running, a THIRD thread may acquire the mutex
       and consume the item. You wake up, the queue is empty
       again. Most implementations are not FIFO handoff.

   (c) BROADCAST. If the signaller uses broadcast, every
       waiter wakes but only some can proceed.

   A codegen or review rule of "cond_wait is only ever called
   inside a while loop testing the predicate" catches almost
   every condition-variable bug there is.                    */


/* RULE 2: hold the mutex when you change the predicate. */

        lock(m);
        queue_push(q, item);        /* change the state ...    */
        unlock(m);
        cond_signal(cv);            /* ... then signal         */

/* Signalling inside or outside the lock are both correct.
   Outside avoids the waiter waking straight into a locked
   mutex ("hurry up and wait"), which some implementations
   optimise anyway. Inside is required if the condition
   variable or the predicate might be destroyed by a woken
   waiter - which is a real hazard on shutdown.

   THE THING THAT IS NEVER CORRECT is changing the predicate
   WITHOUT the mutex and then signalling: the waiter may test
   the predicate between your write and your signal, and
   sleep for ever.                                           */


/* RULE 3: one condition variable per predicate. */

/* WRONG - one cv for two different conditions */
        while (queue_full(q))  cond_wait(cv, m);   /* producer */
        while (queue_empty(q)) cond_wait(cv, m);   /* consumer */
/* signal() may wake a producer when you meant a consumer, and
   the woken thread goes straight back to sleep, having
   consumed the signal. Deadlock with a full queue.
```

```
   Either use TWO condition variables, or use broadcast and
   accept the thundering herd.                              */
```

The bounded buffer, written properly

```
/* The canonical example, and a good template. */

#define CAP 256

struct bbuf {
        pthread_mutex_t m;
        pthread_cond_t  not_empty;      /* two predicates,     */
        pthread_cond_t  not_full;       /* two variables       */
        void           *slot[CAP];
        size_t          head, tail, count;
        bool            closed;         /* for clean shutdown  */
};

int bbuf_put(struct bbuf *b, void *item)
{
        pthread_mutex_lock(&b->m);

        while (b->count == CAP && !b->closed)
                pthread_cond_wait(&b->not_full, &b->m);

        if (b->closed) {                        /* shutdown path */
                pthread_mutex_unlock(&b->m);
                return -EPIPE;
        }

        b->slot[b->tail] = item;
        b->tail = (b->tail + 1) % CAP;
        b->count++;
        pthread_mutex_unlock(&b->m);

        pthread_cond_signal(&b->not_empty);    /* exactly one
                                                  consumer can
                                                  proceed       */
        return 0;
}

void *bbuf_get(struct bbuf *b)
{
        pthread_mutex_lock(&b->m);

        while (b->count == 0 && !b->closed)
                pthread_cond_wait(&b->not_empty, &b->m);

        if (b->count == 0) {                    /* closed, empty */
                pthread_mutex_unlock(&b->m);
                return NULL;
        }

        void *item = b->slot[b->head];
        b->head = (b->head + 1) % CAP;
        b->count--;
        pthread_mutex_unlock(&b->m);

        pthread_cond_signal(&b->not_full);
        return item;
}

/* Shutdown. Note the BROADCAST: every waiter, on both
   variables, must be released - and `closed` is part of every
   predicate above precisely so they can be.                */
```

```
void bbuf_close(struct bbuf *b)
{
        pthread_mutex_lock(&b->m);
        b->closed = true;
        pthread_mutex_unlock(&b->m);
        pthread_cond_broadcast(&b->not_empty);
        pthread_cond_broadcast(&b->not_full);
}
```

```
// C++, where the predicate overload removes the loop from
// your code but not from the semantics.
#include <condition_variable>

std::mutex m;
std::condition_variable cv;
std::queue<int> q;
bool closed = false;

void producer(int v) {
    { std::lock_guard<std::mutex> g(m); q.push(v); }
    cv.notify_one();
}

std::optional<int> consumer() {
    std::unique_lock<std::mutex> lk(m);      // unique_lock, not
                                             // lock_guard: wait
                                             // must unlock it
    cv.wait(lk, [&]{ return !q.empty() || closed; });
    //           ^ this IS the while loop, written once
    if (q.empty()) return std::nullopt;
    int v = q.front(); q.pop();
    return v;
}

// With a timeout - note wait_for returns the predicate's value,
// so you can tell timeout from success without a second check:
if (!cv.wait_for(lk, std::chrono::seconds(5),
                 [&]{ return !q.empty(); }))
        return timeout();
```

> **`WAIT` NEEDS `UNIQUE_LOCK`, AND THAT IS NOT ARBITRARY**
>
> `std::lock_guard` cannot unlock; `cv.wait` must. That is why the type differs, and why the compiler stops you. Similarly, waiting on a condition variable with a `recursive_mutex` locked twice releases it only once — so you sleep still holding it, and nobody can ever signal you. Another reason recursive mutexes cause trouble.

Signal or broadcast

	`signal` / `notify_one`	`broadcast` / `notify_all`
Wakes	One waiter (unspecified which)	Every waiter
Correct when	Every waiter waits for the same predicate and one item satisfies exactly one	Waiters have different predicates, or one event satisfies many
Cost	One wake-up	N wake-ups, N−1 of which relock, re-test and sleep again — the thundering herd
Risk	Waking the wrong waiter if predicates differ, losing the signal	Only performance

The safe default when unsure is `broadcast`: it is never *incorrect*, only slower. Use `signal` when you can state, in one sentence, that every waiter on this variable is waiting for exactly the same thing and one event satisfies exactly one of them.

How it shows up

Symptom	Cause
Consumer proceeds with an empty queue	`if` instead of `while` — spurious wake-up or barging
Everything hangs, no CPU used	Lost wake-up: the predicate was changed without the mutex, or signalled before any waiter existed
Hangs only under load	One condition variable serving two predicates; a signal woke the wrong kind of waiter
Hangs on shutdown	Waiters still blocked; needs a `closed` flag in every predicate plus a broadcast
Latency spikes with many consumers	Thundering herd from `broadcast`
Sleeps for ever holding the mutex	Waiting with a recursive mutex locked more than once
Works, but one core sits at 100%	Someone replaced the wait with a polling loop
Crash inside `cond_signal` at shutdown	A woken waiter destroyed the condition variable while the signaller was still inside it — signal under the lock
Occasional 1 ms latency per item	A poll-and-sleep loop, not a condition variable

Check yourself

- ✓ Explain what a condition variable adds over a mutex, and why it holds no state itself.
- ✓ Explain the atomic unlock-and-wait, and the bug that exists without it.
- ✓ Give three independent reasons the wait must be in a `while` loop.
- ✓ State what must be true when you change a predicate, and the failure when it is not.
- ✓ Explain why one condition variable per predicate matters, with the deadlock it prevents.
- ✓ Write a bounded buffer with clean shutdown from memory.
- ✓ Say why `cv.wait` requires `unique_lock` rather than `lock_guard`.
- ✓ Give the rule for choosing `signal` over `broadcast`.
- ✓ Explain the risk of signalling outside the lock during shutdown.

Semaphores, barriers and latches

Three primitives that are not mutexes. A semaphore counts permits; a latch is a one-shot gate; a barrier is a rendezvous. Reaching for the right one turns several pages of condition-variable code into a line, and reaching for the wrong one produces the most durable deadlocks in this track.

Semaphores: counting permits

```
A semaphore is an integer with two atomic operations and a
waiting rule.

  wait / P / acquire / down
      if count > 0:  count--,  proceed
      else:          block until someone posts

  post / V / release / up
      count++,  wake one waiter if any

THREE DISTINCT USES, and they look nothing alike:

1. COUNTING A RESOURCE    initial = N
     N identical things (connections, buffers, slots).
     wait = take one, post = give one back.

2. SIGNALLING             initial = 0
     "an event happened". Producer posts, consumer waits.
     The count IS the number of pending events, so unlike a
     condition variable a semaphore REMEMBERS a post that
     arrives before anyone waits. No lost wake-ups.

3. MUTUAL EXCLUSION       initial = 1
     A "binary semaphore". Works, and you should not:
     it has no owner, so no priority inheritance, no
     deadlock detection, no error checking, and anyone may
     post a semaphore they never waited on.

THE KEY DIFFERENCE FROM A CONDITION VARIABLE:
  a condvar is stateless and a signal to nobody is lost.
  a semaphore is stateful and a post to nobody is stored.
That single property decides which one your problem wants.
```

```
/* POSIX */
#include <semaphore.h>

sem_t slots;
sem_init(&slots, 0 /* not process-shared */, 8 /* permits */);

sem_wait(&slots);        /* take one, blocking            */
sem_trywait(&slots);     /* or fail with EAGAIN           */
sem_timedwait(&slots, &abs_timeout);
/* ... use the resource ... */
sem_post(&slots);        /* give it back                  */

/* sem_post is ASYNC-SIGNAL-SAFE - almost uniquely among
   synchronisation primitives. It is the only correct way to
   wake a thread from a signal handler.                    */

/* C++20 */
#include <semaphore>
std::counting_semaphore<8> slots{8};
std::binary_semaphore     signal{0};

slots.acquire();
slots.release();
if (slots.try_acquire_for(std::chrono::milliseconds(10))) { ... }

/* Windows */
HANDLE s = CreateSemaphore(NULL, 8, 8, NULL);
WaitForSingleObject(s, INFINITE);
ReleaseSemaphore(s, 1, NULL);
```

THE RULE THAT CATCHES MOST SEMAPHORE BUGS

Wrap it. A raw `sem_wait` / `sem_post` pair scattered across a function has exactly the problem manual `unlock` has, plus no owner to blame. Use a RAII guard, and make the semaphore private to the type that owns the resource it counts — never a global that any code may post.

```
/* Semaphore as a connection pool - use 1, the good one. */

class Pool {
    std::counting_semaphore<> permits_;
    std::mutex                m_;
    std::vector<Conn>         free_;
public:
    explicit Pool(int n) : permits_(n) { /* fill free_ */ }

    class Lease {                       /* RAII, so a throw or
                                           an early return
                                           cannot leak a permit */
        Pool *p_; Conn c_;
    public:
        Lease(Pool *p, Conn c) : p_(p), c_(std::move(c)) {}
        ~Lease() { if (p_) p_->give_back(std::move(c_)); }
        Lease(Lease&& o) : p_(o.p_), c_(std::move(o.c_))
                           { o.p_ = nullptr; }
        Conn &get() { return c_; }
    };

    Lease take() {
        permits_.acquire();             /* blocks if all N are
                                           out - THIS is the
                                           backpressure         */
        std::lock_guard<std::mutex> g(m_);
        Conn c = std::move(free_.back());
        free_.pop_back();
        return Lease(this, std::move(c));
    }
private:
    void give_back(Conn c) {
        { std::lock_guard<std::mutex> g(m_); free_.push_back(std::move(c)); }
        permits_.release();             /* release AFTER the
                                           object is back in
                                           the list             */
    }
};
```

Latches: a gate that opens once

```
/* A latch counts DOWN to zero and then stays open for ever.
   Single use. Nobody resets it.                              */

#include <latch>                        /* C++20             */

std::latch ready(3);                    /* three participants */

/* worker */
        do_setup();
        ready.count_down();             /* "I am ready"        */
        ready.wait();                   /* wait for the others */
        /* all three are past setup here */

/* or, from a coordinator that does not participate: */
        ready.wait();                   /* until all 3 counted */

/* arrive_and_wait() is count_down() + wait() in one call.  */

/* USES
     - start N workers, wait until all have initialised
     - fan out N requests, wait for all N to complete
     - hold workers at a starting line for a benchmark

   Before C++20, or in C, it is a counter plus a condvar: */

struct latch {
        pthread_mutex_t m;
        pthread_cond_t  cv;
        unsigned        n;
};
void latch_count_down(struct latch *l) {
        pthread_mutex_lock(&l->m);
        if (l->n > 0 && --l->n == 0)
                pthread_cond_broadcast(&l->cv);   /* broadcast:
                                                     everyone is
                                                     released    */
        pthread_mutex_unlock(&l->m);
}
void latch_wait(struct latch *l) {
        pthread_mutex_lock(&l->m);
        while (l->n > 0)
                pthread_cond_wait(&l->cv, &l->m);
        pthread_mutex_unlock(&l->m);
}
```

Barriers: a rendezvous, repeatedly

```
/* A barrier is a REUSABLE latch. Every participant waits
   until all arrive; then all are released and the barrier
   resets for the next round.                                */

#include <barrier>                      /* C++20             */

std::barrier sync(4, [] {               /* completion function:
        swap_buffers();                    runs ONCE per phase,
        ++generation;                      by one thread, while
});                                        all others wait    */

void worker(int id) {
        for (int step = 0; step < 1000; step++) {
                compute_my_share(id);
                sync.arrive_and_wait();     /* phase boundary  */
                /* every thread's writes from this phase are
                   visible to every thread in the next one    */
        }
}

/* pthread barriers, the same thing: */
pthread_barrier_t b;
pthread_barrier_init(&b, NULL, 4);
int r = pthread_barrier_wait(&b);
if (r == PTHREAD_BARRIER_SERIAL_THREAD) {
        /* exactly ONE thread gets this - do the once-per-phase
           work here */
}

/* WHAT BARRIERS ARE FOR: bulk-synchronous parallelism.
   Iterative solvers, physics steps, image filters, any
   "everyone computes, then everyone exchanges" loop.

   THE COST: every phase runs at the speed of the SLOWEST
   participant. With 16 threads and one that is 10% slow,
   you lose 10% of all 16. Load imbalance is the whole game.

   THE DEADLOCK: if any participant leaves early - returns,
   throws, hits an error path - every other thread waits for
   ever. arrive_and_drop() exists for exactly this, and code
   that can throw between arrivals needs it.               */
```

> **THE THREE WAYS BARRIERS DEADLOCK**
>
> **1.** A participant exits the loop early on an error and never arrives again. **2.** The count does not match the number of threads — off by one, or a thread that failed to start. **3.** Two barriers used in different orders by different threads. All three present identically: everything hangs, no CPU used, and a stack dump shows every thread in `barrier_wait`.

Choosing

You want to...	Use
Protect data from concurrent access	**Mutex** — it has an owner
Wait for a state to become true	**Condition variable**

You want to…	Use
Limit concurrency to N	**Counting semaphore**
Signal from an interrupt or signal handler	**Semaphore** — the only async-signal-safe one
Wake a thread for an event that may arrive first	**Semaphore**, which stores the post
Wait for N things to finish, once	**Latch**
Synchronise phases of a loop, repeatedly	**Barrier**
Run something exactly once	`call_once` / `pthread_once`
Deliver a single value to a single waiter	`future` / `promise`

How it shows up

Symptom	Cause
Pool exhausts and never recovers	A permit leaked on an error path — needs RAII
More than N concurrent users of an N-permit resource	Somebody posted without waiting; a semaphore has no owner to stop them
Real-time thread misses deadlines behind a binary semaphore	No priority inheritance — use a mutex (chapter 15)
Everything hangs at a barrier	Wrong participant count, or one thread left early
Barrier phase takes as long as the slowest thread every time	Working as designed — fix load balance, not the barrier
Latch never opens	Fewer `count_down` calls than the initial count, often an early return
Deadlock on shutdown with waiters on a semaphore	Semaphores have no "close" — post N times or use a different primitive
Signal handler deadlocks the process	It took a mutex or signalled a condvar; only `sem_post` is safe there
Counter drifts over days	Unbalanced wait/post, usually one path that returns without releasing

Check yourself

- ✓ Define a semaphore's two operations and give the three distinct uses.
- ✓ State the key difference between a semaphore and a condition variable, and when it decides the choice.
- ✓ Explain why a binary semaphore is a poor mutex — give three specific losses.
- ✓ Say why `sem_post` is the only safe wake-up from a signal handler.
- ✓ Write a permit-leaking bug and fix it with RAII.
- ✓ Distinguish a latch from a barrier in one sentence.
- ✓ Explain what a barrier's completion function is for.
- ✓ Give the three ways a barrier deadlocks and how each presents.
- ✓ Explain why a barrier makes every phase run at the slowest participant's speed.
- ✓ Pick the right primitive for six of the situations in the table without looking.

Reader-writer locks, seqlocks and RCU

Most shared data is read far more often than it is written. Three primitives exploit that, each trading a different thing away — and the first one, the obvious one, is usually the worst of the three.

Reader-writer locks, and why they disappoint

```
/* The idea: many readers OR one writer. */

pthread_rwlock_t l = PTHREAD_RWLOCK_INITIALIZER;

pthread_rwlock_rdlock(&l);      /* shared                    */
        read_things();
pthread_rwlock_unlock(&l);

pthread_rwlock_wrlock(&l);      /* exclusive                 */
        write_things();
pthread_rwlock_unlock(&l);

/* C++17: */
std::shared_mutex m;
{ std::shared_lock  g(m);  read_things();  }   /* many         */
{ std::unique_lock  g(m);  write_things(); }   /* one          */

/* WHY IT OFTEN LOSES TO A PLAIN MUTEX:

   1. Taking a read lock is a WRITE to the lock's state - the
      reader count. So every reader dirties the same cache
      line, and 16 readers "reading in parallel" bounce one
      line 16 ways. Under load a shared_mutex read lock can be
      SLOWER than an exclusive mutex, because a mutex at least
      only bounces the line once per holder.

   2. It is bigger and its uncontended path is longer: more
      state to update, more branches.

   3. Fairness is a policy you did not choose. Reader-preferring
      implementations starve writers indefinitely under a steady
      read load. Writer-preferring ones make readers queue
      behind a writer that has not started yet.

   4. There is no atomic upgrade. read -> write means unlock,
      relock, and RE-VALIDATE everything you read, because the
      world changed in the gap. Implementations that offer
      upgrade usually deadlock if two readers try at once.

   USE IT WHEN: read critical sections are LONG (microseconds,
   not nanoseconds) and writes are genuinely rare. Below that,
   measure against a plain mutex before believing it helps. */
```

THE RULE OF THUMB

If the read critical section is shorter than about a microsecond, a plain mutex usually wins. The reader-writer lock only starts paying when the work under the read lock is long enough to dwarf the cost of the reader-count update — and by then you should ask whether you could copy the data out under a mutex instead.

Seqlocks: readers that never write

```
The insight: let readers do NOTHING to shared state, and
detect afterwards whether they were interfered with.

A counter, incremented on entry and exit of every write.

  even  = stable
  odd   = a write is in progress

WRITER                          READER
  seq++            (odd)        do {
  write the data                    s1 = seq;
  seq++            (even)           if (s1 & 1) continue;
                                    ... read the data ...
                                    s2 = seq;
                                  } while (s1 != s2);

If seq changed, or was odd, the read was torn - so retry.

WHAT YOU GAIN
  readers issue ZERO writes. No cache line is dirtied, so
  N readers genuinely scale. This is why Linux uses it for
  the system time (gettimeofday) - read constantly by every
  core, written once per tick.

WHAT YOU GIVE UP
  * readers may retry, so a reader can starve under a
    constant write load
  * the data being read MUST be safe to read while torn -
    no pointer chasing, no unbounded loops on a value that
    might be garbage mid-write
  * writers still need a mutex among themselves; a seqlock
    does not order writers
  * it is not a general-purpose lock. It suits small,
    fixed-size, self-contained records.
```

```
/* A seqlock in C11 atomics. */

typedef struct { atomic_uint seq; struct data d; pthread_mutex_t wm; } seqlock;

void seq_write(seqlock *l, const struct data *src)
{
        pthread_mutex_lock(&l->wm);            /* writers only */
        atomic_fetch_add_explicit(&l->seq, 1, memory_order_relaxed);
        atomic_thread_fence(memory_order_release);   /* odd BEFORE
                                                        the data */
        l->d = *src;
        atomic_thread_fence(memory_order_release);   /* data BEFORE
                                                        even     */
        atomic_fetch_add_explicit(&l->seq, 1, memory_order_relaxed);
        pthread_mutex_unlock(&l->wm);
}

struct data seq_read(seqlock *l)
{
        struct data out;
        unsigned s1, s2;
        do {
                s1 = atomic_load_explicit(&l->seq, memory_order_acquire);
                if (s1 & 1u)
                        continue;              /* write in flight */
                out = l->d;                    /* may be torn     */
                atomic_thread_fence(memory_order_acquire);
                s2 = atomic_load_explicit(&l->seq, memory_order_relaxed);
        } while (s1 != s2);
        return out;
}

/* Note: strictly, reading l->d while a writer writes it is a
   data race by the letter of the C standard, and TSan will say
   so. Real implementations use atomic loads per field or
   compiler-specific escape hatches. This is one of the few
   places where the model and the practice diverge, and the
   kernel's read_seqcount_begin/retry is the reference. */
```

RCU: readers that do not even check

```
Read-Copy-Update. The most extreme point on the curve:
readers pay LITERALLY NOTHING - no atomic, no branch, in
the classic kernel build not a single instruction.

THE IDEA
  Never modify data in place. Instead:
    1. COPY the object
    2. modify the copy
    3. publish the new pointer atomically
    4. WAIT until every reader that might still hold the
       old pointer has finished
    5. free the old one

Step 4 is the whole trick, and it is called the GRACE
PERIOD.

  writer:  new = copy(old)
           modify(new)
           rcu_assign_pointer(p, new)     ; release store
           synchronize_rcu()              ; wait for readers
           kfree(old)

  reader:  rcu_read_lock()                ; often a no-op
           q = rcu_dereference(p)
           use(q)                         ; q stays valid
           rcu_read_unlock()              ; often a no-op

HOW THE GRACE PERIOD IS DETECTED
  In the kernel's classic RCU, a reader may not sleep or be
  preempted inside a read-side section. So once every CPU
  has passed through a context switch, no reader from
  before the update can still be running. That is a
  QUIESCENT STATE, and observing one per CPU ends the
  grace period. No reference counts, no atomics, no
  communication from readers at all.

THE COSTS
  * writers are expensive and must wait, possibly
    milliseconds
  * memory is freed late, so usage spikes under write load
  * readers may see stale data - RCU gives you a consistent
    SNAPSHOT, not the latest value
  * read sections have hard rules (no sleeping in classic
    RCU) that the compiler cannot check

WHERE IT WINS: routing tables, module lists, security
policy, the dcache - anything read on every operation by
every core and updated rarely.
```

```
/* Userspace RCU (liburcu) - the same API outside the kernel. */
#include <urcu.h>

struct config *global_cfg;

/* reader - the hot path */
void handle_request(void)
{
        rcu_read_lock();
        struct config *c = rcu_dereference(global_cfg);
        use(c->timeout, c->retries);
        rcu_read_unlock();
}

/* writer - rare */
void update_config(int timeout)
{
        struct config *old, *new = malloc(sizeof *new);

        pthread_mutex_lock(&writer_lock);      /* writers still
                                                  serialise      */
        old = global_cfg;
        *new = *old;                           /* COPY           */
        new->timeout = timeout;                /* UPDATE         */
        rcu_assign_pointer(global_cfg, new);   /* PUBLISH        */
        pthread_mutex_unlock(&writer_lock);

        synchronize_rcu();                     /* WAIT           */
        free(old);                             /* now safe       */
}

/* call_rcu(&old->head, free_cb) defers the free instead of
   blocking the writer - what you use when the writer is on a
   latency-sensitive path.                                    */
```

Choosing

	Mutex	RW lock	Seqlock	RCU
Reader cost	One atomic	One atomic, contended	Two plain loads	≈ nothing
Readers scale?	No	Poorly	Yes	Perfectly
Writer cost	Low	Low	Low	High — grace period
Readers can starve?	No	Possible	**Yes**, under write load	No
Readers see	Latest	Latest	Latest, or retry	A recent snapshot
Read section may sleep	Yes	Yes	No	No (classic RCU)
Memory freed	Immediately	Immediately	Immediately	Late
Good for	Almost everything	Long reads, rare writes	Small fixed records	Pointer-published structures read by every core

> **TRY THE BORING ANSWER FIRST**
> The order to reach for these is: plain mutex → copy the data out under a mutex → RW lock → seqlock → RCU. Each step buys read scalability and costs correctness margin. Most code that reaches for RCU would have been fine copying a small struct under a mutex, and would have been far easier to review. Move down the list only when a profile says the read path is the bottleneck.

How it shows up

Symptom	Cause
RW lock is slower than the mutex it replaced	Short read sections; the reader-count line is the bottleneck
Writers never get in	Reader-preferring RW lock under continuous read load
Deadlock upgrading read to write	Two readers both trying to upgrade
Stale data after a documented update	RCU is working as designed — readers see a snapshot
Memory usage spikes under write load	RCU frees late; use `call_rcu` and rate-limit writers
Reader crashes dereferencing a freed pointer	Missing `rcu_read_lock`, or the writer freed before the grace period
Kernel hang in `synchronize_rcu`	A read-side section that slept or never ended
Seqlock reader spins for ever	Writers never stop; the reader is starved
Seqlock reader crashes	It followed a pointer read from torn data
TSan reports a race on a seqlock	Genuine, by the letter of the standard — this is a known model/practice gap

Check yourself

- ✓ Explain why taking a read lock is a write, and what that costs.
- ✓ Give four reasons a reader-writer lock can lose to a plain mutex.
- ✓ Explain why there is no safe atomic read-to-write upgrade.
- ✓ Describe the seqlock protocol and say what readers give up.
- ✓ Explain why seqlock data must be safe to read while torn.
- ✓ State the five steps of an RCU update and name the one that is the trick.
- ✓ Explain how a grace period is detected without readers signalling anything.
- ✓ Give three costs of RCU.
- ✓ Order the four primitives by reader cost and by review difficulty.
- ✓ Say which primitive you would use for a routing table, for the system clock, and for a config struct read once per request.

Atomic operations and the CAS loop

An atomic operation is one that no other thread can observe half-finished. That is a smaller guarantee than people assume — it says nothing about *ordering*, and it does not compose. Both of those gaps are where the bugs live.

What atomic buys, and what it does not

```
ATOMIC GIVES YOU

  indivisibility   no thread sees a half-written value.
                   No torn reads, no lost read-modify-write.

  a defined value  the result is one of the values some
                   thread actually wrote - never a mixture.

ATOMIC DOES NOT GIVE YOU

  ordering         unless you ask. atomic_fetch_add with
                   memory_order_relaxed orders NOTHING
                   around it (chapter 12).

  composition      two atomic operations are not one atomic
                   operation:

                       if (atomic_load(&a) == 0)     // 1
                               atomic_store(&b, 1);  // 2

                   anything can happen between 1 and 2.
                   This is the check-then-act race with
                   extra syntax.

  atomicity of the SURROUNDING LOGIC. An atomic counter
  that is correct tells you nothing about the invariant it
  was counting.

THE ONE OPERATION THAT COMPOSES is compare-and-swap, because
it folds the check and the act into a single indivisible
step. Everything lock-free is built out of it.
```

The operation set

```
#include <stdatomic.h>                /* C11                   */

atomic_int   counter = 0;
atomic_bool  flag    = false;
_Atomic(struct point) pt;             /* may be lock-free or
                                         not - see below      */

/* LOAD / STORE - indivisible, but nothing more by default. */
        int v = atomic_load(&counter);
        atomic_store(&counter, 42);

/* READ-MODIFY-WRITE - the useful ones. Each returns the OLD
   value, which is what makes them composable into algorithms. */
        old = atomic_fetch_add(&counter,  1);
        old = atomic_fetch_sub(&counter,  1);
        old = atomic_fetch_or (&flags, MASK);
        old = atomic_fetch_and(&flags, ~MASK);
        old = atomic_fetch_xor(&flags, MASK);
        old = atomic_exchange (&ptr, newptr);

/* COMPARE-AND-SWAP - the universal primitive. */
        bool ok = atomic_compare_exchange_strong(&p, &expected, desired);
        /* if (*p == expected) { *p = desired; return true; }
           else                { expected = *p; return false; }

           NOTE: `expected` is updated on failure. That is not
           a wart - it saves you a reload in the retry loop.  */

/* C++ is the same set, spelled as methods: */
        std::atomic<int> c{0};
        c.fetch_add(1, std::memory_order_relaxed);
        c.compare_exchange_weak(exp, des);

/* Rust: */
        use std::sync::atomic::{AtomicUsize, Ordering};
        let c = AtomicUsize::new(0);
        c.fetch_add(1, Ordering::Relaxed);
        c.compare_exchange_weak(exp, des, Ordering::AcqRel,
                                Ordering::Relaxed)
```

> **`STRONG` VERSUS `WEAK`**
> `compare_exchange_weak` may fail *spuriously* — return false even when the value matched. That sounds useless until you remember ARM's LL/SC (chapter 6): the store-exclusive fails if anything touched the cache line, including an unrelated access. The weak form maps to one LL/SC attempt; the strong form has to add a retry loop inside. **Use `weak` inside a loop you were writing anyway; use `strong` when a single attempt must be conclusive.** On x86 they compile identically.

The CAS loop

```
/* The shape of every lock-free update. Learn it once. */

        T old = atomic_load_explicit(&obj, memory_order_relaxed);
        T new;
        do {
                new = compute_new_from(old);     /* pure! */
        } while (!atomic_compare_exchange_weak_explicit(
                        &obj, &old, new,
                        memory_order_acq_rel,    /* on success */
                        memory_order_relaxed));  /* on failure */

/* WHY IT IS CORRECT: the CAS only succeeds if `obj` is still
   `old` - i.e. nobody changed it while you were computing.
   If someone did, `old` is refreshed with the current value
   and you recompute. Progress is guaranteed for the system
   (someone always wins) but not for any given thread.

   THREE RULES FOR THE BODY:

   1. compute_new_from() MUST be pure and cheap. It runs again
      on every failure. Anything with a side effect - logging,
      allocating, incrementing another counter - happens
      multiple times.

   2. It must not be able to loop for ever. Under heavy
      contention a thread can lose repeatedly; if that matters,
      add back-off, or use a lock.

   3. Do NOT reload `old` yourself at the top of the loop.
      compare_exchange already wrote the current value into it.
      Reloading opens a window and is a common source of
      subtle livelock.                                        */

/* A worked example: atomically clamp-and-add. */
int add_clamped(atomic_int *c, int delta, int max)
{
        int old = atomic_load_explicit(c, memory_order_relaxed);
        int new;
        do {
                if (old >= max)
                        return -1;               /* full        */
                new = old + delta;
                if (new > max) new = max;
        } while (!atomic_compare_exchange_weak_explicit(
                        c, &old, new,
                        memory_order_acq_rel, memory_order_relaxed));
        return new;
}
/* You cannot write that with fetch_add. This is exactly the
   class of problem CAS exists for: a read-modify-write whose
   "modify" involves a decision.                           */
```

Is it actually lock-free?

```
/* An "atomic" type is not necessarily implemented in hardware.
   If the type is too big, the compiler emits a hidden LOCK. */

        _Atomic(struct { long a, b, c; }) big;   /* 24 bytes    */

        printf("%d\n", atomic_is_lock_free(&big));   /* 0       */

/* C++ has the compile-time version, which is what you want in
   a static_assert: */
        static_assert(std::atomic<MyType>::is_always_lock_free,
                      "this type needs a hidden mutex");

/* WHAT IS LOCK-FREE, TYPICALLY

     1, 2, 4, 8 bytes         yes, everywhere
     16 bytes                 x86-64: yes, via CMPXCHG16B
                              ARM64:  yes, via LDXP/STXP or
                                      CASP (LSE)
                              but often only if you compile
                              with -mcx16 / -march=armv8.1-a
     anything larger          NO - a lock table, keyed by
                              address, inside libatomic

   WHY IT MATTERS

     * a hidden lock is not async-signal-safe, so your
       "atomic" is illegal in a signal handler
     * it is not usable across processes in shared memory:
       the lock table is per-process, so two processes
       "atomically" updating the same shared struct do not
       exclude each other AT ALL
     * performance falls off a cliff you cannot see in the
       source

   Check it. Once, in a static_assert, and never worry again. */
```

> **`VOLATILE` IS STILL NOT ATOMIC**
>
> Worth repeating from chapter 2 because it is the most durable misconception in this field. `volatile` stops the compiler caching a value in a register. It does not make the access indivisible, does not stop the CPU reordering, and does not give any ordering guarantee. On MSVC with `/volatile:ms` it happens to add acquire/release semantics — which is why code written for Windows breaks when ported, and why `/volatile:iso` exists.

What atomics are actually for

Use	Operation	Ordering usually needed
Statistics counters	`fetch_add`	`relaxed` — nobody makes a decision on the value
Reference counts, increment	`fetch_add`	`relaxed` — you already hold a reference
Reference counts, decrement	`fetch_sub`	`acq_rel`, or release plus an acquire fence before destroying
A "stop" flag	`store` / `load`	`relaxed` is usually fine; `release` / `acquire` if data goes with it
Publishing a pointer	`store`	**`release`**, always (chapter 12)

Use	Operation	Ordering usually needed
Sequence numbers, ID allocation	`fetch_add`	`relaxed`
Spinlock acquire / release	`exchange` / `store`	`acquire` / `release`
Lock-free stack, queue	`compare_exchange`	`acq_rel`
One-time initialisation	`compare_exchange`	`acq_rel`, or just use `call_once`

```
/* The reference-count idiom, written correctly. It is the
   most common real use of non-trivial ordering.           */

void ref_get(struct obj *o)
{
        /* relaxed: we already hold a reference, so the object
           cannot die under us, and no data is being published */
        atomic_fetch_add_explicit(&o->refs, 1, memory_order_relaxed);
}

void ref_put(struct obj *o)
{
        if (atomic_fetch_sub_explicit(&o->refs, 1,
                                      memory_order_release) == 1) {
                /* We dropped the last reference. The release
                   above published OUR writes. Now we must
                   ACQUIRE everyone else's before we destroy,
                   or we may free memory whose last write we
                   never saw.                                  */
                atomic_thread_fence(memory_order_acquire);
                destroy(o);
        }
}
/* This exact pattern is in the C++ standard's shared_ptr
   discussion, in Boost, and in the Linux kernel's kref. If you
   find yourself writing a refcount by hand, copy it exactly. */
```

How it shows up

Symptom	Cause
Counter is correct but the logic around it is not	Two atomic operations treated as one — use CAS
CAS loop performs a side effect several times	An impure body; it re-runs on every failure
CAS loop spins for ever under load	No back-off; a thread keeps losing. Livelock
CAS never succeeds	`expected` reloaded manually at the top of the loop, or `weak` used where a single conclusive attempt was needed
"Atomic" struct is slow and not signal-safe	Too large — `libatomic` is using a hidden lock
Two processes corrupt a shared atomic struct	Same cause: the hidden lock table is per-process
Works on x86, breaks on ARM	Relaxed ordering that x86's strong model was hiding
Use-after-free on the last `put`	Missing acquire fence before destroy in the refcount
Works with MSVC, breaks with GCC	Relied on `/volatile:ms` giving acquire/release semantics

Symptom	Cause
Link error on `__atomic_load_16`	16-byte atomics without `-mcx16` / `-latomic`

Check yourself

- ✓ State exactly what atomicity guarantees and the two things it does not.
- ✓ Show why two atomic operations are not one, with a concrete race.
- ✓ Explain why compare-and-swap is the primitive everything lock-free is built from.
- ✓ Write the CAS loop from memory and give the three rules for its body.
- ✓ Explain why `expected` is updated on failure and why reloading it manually is a bug.
- ✓ Distinguish `compare_exchange_weak` from `strong`, and relate it to LL/SC.
- ✓ Explain how an "atomic" type can contain a hidden lock and give two consequences.
- ✓ Say why `volatile` is not a substitute, including the MSVC caveat.
- ✓ Write the reference-count get/put pair with correct orderings and justify each.
- ✓ Give the right memory order for a statistics counter and for publishing a pointer, with reasons.

Memory ordering and happens-before

Your program's memory operations do not happen in the order you wrote them. The compiler reorders, the CPU reorders, and the cache system makes writes visible to different cores at different times. A memory model is the contract that says which of those reorderings you are allowed to notice — and *happens-before* is the only relation in it that matters.

Three layers of reordering

```
YOU WROTE                  WHAT ACTUALLY RUNS

  a = 1;                     the COMPILER may:
  b = 2;                       reorder independent stores
  x = c;                       keep a value in a register
                               delete a "redundant" load
                               hoist a load out of a loop
                               merge two stores into one

                             the CPU may:
                               execute out of order
                               speculate past a branch
                               buffer stores and drain later
                               prefetch loads early

                             the MEMORY SYSTEM may:
                               make your store visible to
                               core 2 before core 3 sees it

NONE of this is a bug. All of it is required for
performance, and all of it is invisible in single-threaded
code because each layer preserves the illusion for the
thread doing the work.

Concurrency is where the illusion leaks - and the memory
model is the exact specification of how much it may leak.
```

Architecture	Reordering allowed	Practical effect
Sequential consistency	None	The mental model everyone has. No real CPU provides it
x86-64 (TSO)	Store→Load only	Strong. Most sloppy code accidentally works, which is why it ships
ARM64	Almost anything	Weak. The same code fails, and this is where your bug is discovered
RISC-V (RVWMO)	Almost anything	Weak, similar to ARM
POWER	Anything, plus non-multi-copy-atomic	Weakest in common use

Happens-before is the whole model

```
The C, C++, Rust, Java and Go memory models all say the
same thing:

  If A happens-before B, then B sees A's effects.
  If neither happens-before the other AND one is a write,
  that is a DATA RACE - undefined behaviour.

HOW YOU ESTABLISH HAPPENS-BEFORE - and there are only four
ways:

1. PROGRAM ORDER, within one thread.
     a = 1; b = 2;      -> a=1 happens-before b=2
     (as OBSERVED by that thread. Others may disagree.)

2. RELEASE / ACQUIRE on the same variable.
     thread A:  ... writes ...
                atomic_store(&flag, 1, release);
     thread B:  while (!atomic_load(&flag, acquire)) ;
                ... reads ...
     -> everything before A's store happens-before
        everything after B's load. THIS IS THE ONE YOU USE.

3. THREAD CREATION AND JOIN.
     everything before pthread_create happens-before the
     new thread's first instruction.
     everything in a thread happens-before pthread_join
     returns.

4. MUTEX UNLOCK / LOCK on the same mutex.
     which is just case 2 with nicer syntax - unlock is a
     release, lock is an acquire.

TRANSITIVITY: happens-before chains. If A hb B and B hb C
then A hb C. That is what makes a mutex work across three
threads.

AND NOTHING ELSE ESTABLISHES IT. Not "thread B started
later". Not "the sleep was long enough". Not "it is only
one byte".
```

The six orderings

```
/* From weakest to strongest. */

memory_order_relaxed
        Atomicity only. No ordering with anything.
        USE FOR: counters nobody makes decisions on, and the
        increment side of a refcount.
        DO NOT USE FOR: publishing anything.

memory_order_acquire        (loads, and RMW)
        Nothing after this in program order may be reordered
        BEFORE it. Pairs with a release on the same variable
        to give you everything the releaser wrote.
        THINK: "I am picking up the data."

memory_order_release        (stores, and RMW)
        Nothing before this in program order may be reordered
        AFTER it.
        THINK: "I am handing over the data."

memory_order_acq_rel        (RMW only)
        Both. For a read-modify-write that both consumes and
        publishes - a CAS in a lock-free stack, a refcount
        decrement.

memory_order_seq_cst        (the default)
        Acquire+release PLUS a single total order that ALL
        seq_cst operations agree on, across all variables.
        The only one that gives you the naive mental model.
        Costs a full barrier (DMB ISH on ARM, MFENCE-ish on
        x86 for stores).

memory_order_consume
        Intended as a cheaper acquire for pointer chasing.
        Every real compiler promotes it to acquire because
        the specification is unimplementable as written.
        DO NOT USE IT. It is being deprecated.
```

> **THE DEFAULT IS `SEQ_CST`, AND THAT IS THE RIGHT DEFAULT**
>
> Write `atomic_load(&x)` and you get sequential consistency. It is the slowest ordering and the only one that behaves the way you expect. **Start there.** Weaken an ordering only when a profile shows the barrier costs you and you can write down the happens-before chain that still holds. Code littered with `relaxed` written by someone who could not articulate why is the least reviewable code in existence.

The publication pattern

```
/* This is 90% of the memory ordering you will ever need. */

struct config *cfg;                     /* the payload         */
atomic_bool ready = false;

/* PUBLISHER */
void publish(void)
{
        struct config *c = malloc(sizeof *c);
        c->timeout = 30;                /* (1) */
        c->retries = 3;                 /* (2) */
        cfg = c;                        /* (3) */

        atomic_store_explicit(&ready, true, memory_order_release);
        /* RELEASE: (1),(2),(3) cannot move after this line, and
           are guaranteed visible to anyone who ACQUIRES `ready`
           and sees true.                                        */
}

/* CONSUMER */
void consume(void)
{
        if (atomic_load_explicit(&ready, memory_order_acquire)) {
                /* ACQUIRE: nothing below may move above. Having
                   seen `true`, we are guaranteed to see (1),(2)
                   and (3).                                     */
                use(cfg->timeout, cfg->retries);   /* safe      */
        }
}

/* WHAT GOES WRONG WITH relaxed

        atomic_store_explicit(&ready, true, memory_order_relaxed);

   The store to `ready` may become visible BEFORE the stores to
   c->timeout, c->retries, or even before `cfg` itself. The
   consumer sees ready==true, dereferences a pointer that is
   still null or points at uninitialised memory, and crashes.

   ON x86 this almost never happens, because TSO does not
   reorder store-store. ON ARM it happens immediately. That
   asymmetry is why this bug reaches production: it is written
   on a laptop and found on a phone.                          */
```

The one x86 also gets wrong

```
/* Store-Load is reordered even on x86. This is Dekker's
   pattern, and it is the classic demonstration.            */

atomic_int x = 0, y = 0;
int r1, r2;

/* thread A */              /* thread B */
x = 1;                      y = 1;
r1 = y;                     r2 = x;

/* CAN r1 == 0 && r2 == 0 ?

   With sequential consistency: NO. Someone stored first, and
   the other thread must see it.

   In reality: YES, on every real CPU including x86.

   Both stores sit in their core's store buffer. Both loads
   read from L1, which has not seen the other core's store
   yet. Both read 0.

   This is the ONLY reordering x86 permits, and it is enough
   to break hand-rolled synchronisation. It is why
   seq_cst STORES are expensive on x86 - the compiler must
   emit an MFENCE or use XCHG to drain the store buffer.

   THE FIX: use seq_cst on both (the default), or insert
   atomic_thread_fence(memory_order_seq_cst) between the store
   and the load in each thread.                              */
```

What it compiles to

```
/* ARM64 - the orderings map to distinct instructions, which
   is why the choice actually costs something.              */

  relaxed load        LDR   x0, [x1]
  acquire load        LDAR  x0, [x1]          ; load-acquire
  seq_cst load        LDAR  x0, [x1]          ; same

  relaxed store       STR   x0, [x1]
  release store       STLR  x0, [x1]          ; store-release
  seq_cst store       STLR  x0, [x1]          ; same
                      (with -moutline-atomics or older targets
                       you may instead see DMB ISH + STR)

  fence(acquire)      DMB ISHLD
  fence(release)      DMB ISH
  fence(seq_cst)      DMB ISH

  relaxed fetch_add   LDXR/ADD/STXR loop, or LDADD  (LSE)
  acq_rel fetch_add   LDAXR/ADD/STLXR,   or LDADDAL (LSE)

/* x86-64 - almost everything is free, which is the trap. */

  relaxed load        MOV
  acquire load        MOV                     ; free!
  seq_cst load        MOV                     ; free!

  relaxed store       MOV
  release store       MOV                     ; free!
  seq_cst store       XCHG  (or MOV + MFENCE) ; NOT free

/* Read that x86 column again. Acquire and release cost
   NOTHING on x86 - so code that omits them behaves
   identically. The bug is not merely hidden; it is
   unobservable, on that machine, for ever.                */
```

> **TEST ON WEAK HARDWARE, OR YOU HAVE NOT TESTED**
> An arm64 machine — a Raspberry Pi, an Apple Silicon Mac, a Graviton instance — will surface ordering bugs that x86 physically cannot exhibit. If your product ships on ARM and your CI is x86-only, your CI is not testing the memory model at all. Add one weak-ordered runner; it is the highest-value CI change a concurrent codebase can make.

How it shows up

Symptom	Cause
Crash dereferencing a just-published pointer	Relaxed store where a release was needed
Reader sees the flag but stale data	Missing acquire on the reader side
Works on x86, fails on ARM/Apple Silicon	Store-store or load-load reordering that TSO forbids
Fails only at -O2	The compiler hoisted a non-atomic load out of a loop
Both threads take the "other one went first" branch	Store-Load reordering — needs `seq_cst` or a fence
Adding a `printf` fixes it	The syscall is a full barrier
Use-after-free on the last refcount drop	Release on the decrement without the acquire fence before destroy

Symptom	Cause
Performance regression after "fixing" ordering	Every operation promoted to `seq_cst`; on ARM each store became an `STLR` or a `DMB`
Code using `memory_order_consume` behaves as acquire	Correct — every compiler promotes it

Check yourself

- ✓ Name the three layers that reorder your memory operations and give an example of each.
- ✓ State the happens-before rule and the four ways to establish it.
- ✓ Explain why transitivity of happens-before is what makes a mutex work.
- ✓ List the six memory orderings and say what each guarantees.
- ✓ Say which ordering is the default and why that is the right default.
- ✓ Write the publication pattern with correct orderings and explain what breaks with `relaxed`.
- ✓ Explain why the publication bug ships from x86 and fails on ARM.
- ✓ Give the Dekker example and say why both threads can read zero even on x86.
- ✓ Say what an acquire load and a release store compile to on ARM64 and on x86-64.
- ✓ Explain why acquire/release being free on x86 makes the bug worse, not better.

Lock-free and wait-free data structures

Lock-free does not mean fast, and it does not mean easy. It means one specific progress guarantee, bought with a memory-reclamation problem that has no cheap solution. This chapter is mostly about why you should not write these, and what to do when you must.

The guarantees, precisely

```
BLOCKING (a mutex)
  If the thread holding the lock is descheduled, killed, or
  page-faults, NOBODY makes progress.

OBSTRUCTION-FREE
  A thread makes progress if it runs alone for long enough.
  Weakest useful guarantee; rarely used on its own.

LOCK-FREE
  At least ONE thread makes progress, always.
  Individual threads may starve indefinitely.
  -> the system never stalls because a thread was
     descheduled at a bad moment.

WAIT-FREE
  EVERY thread completes in a BOUNDED number of steps,
  regardless of what other threads do.
  -> no starvation, and a hard latency bound.
  -> almost always slower in the common case, because
     bounding the worst case costs work in every case.

WHAT THIS BUYS YOU, and it is narrower than the marketing:

  * immunity to lock-holder preemption. Matters enormously
    in a hypervisor guest (virtualisation chapter 8) or a
    system with mixed priorities.
  * usable from an interrupt handler or a signal handler,
    where you cannot take a sleeping lock.
  * no deadlock, ever - there is no lock to order.
  * a real worst-case bound (wait-free only), which is why
    hard real-time cares.

WHAT IT DOES NOT BUY

  * throughput. Under contention a lock-free structure
    does the same cache-line bouncing a spinlock does, and
    often more, because losers retry.
  * simplicity. See the rest of this chapter.
```

THE HONEST RECOMMENDATION

Use a mutex. If profiling proves the lock is the bottleneck, first try: sharding the data, per-thread accumulation with a merge, a read-mostly structure (chapter 10), or an off-the-shelf concurrent container. Write a lock-free structure yourself only if none of those fit and you have a memory-reclamation scheme, a weak-hardware test rig, and a reviewer who has done it before. This is not conservatism — published, peer-reviewed lock-free algorithms have shipped with bugs found years later.

A lock-free stack, and its famous bug

```
/* The Treiber stack. Four lines, and one of them is wrong. */

struct node { struct node *next; void *val; };
_Atomic(struct node *) top;

void push(void *v)
{
        struct node *n = malloc(sizeof *n);
        n->val = v;
        n->next = atomic_load_explicit(&top, memory_order_relaxed);
        while (!atomic_compare_exchange_weak_explicit(
                        &top, &n->next, n,
                        memory_order_release,      /* publish n */
                        memory_order_relaxed))
                ;                                  /* n->next was
                                                      refreshed  */
}

void *pop(void)
{
        struct node *t = atomic_load_explicit(&top, memory_order_acquire);
        while (t && !atomic_compare_exchange_weak_explicit(
                        &top, &t, t->next,
                        memory_order_acquire, memory_order_acquire))
                ;
        if (!t) return NULL;
        void *v = t->val;
        free(t);                    /* <-- THE BUG IS HERE      */
        return v;
}
```

```
   THE ABA PROBLEM

   stack:  A -> B -> C

   thread 1: reads top = A, reads A->next = B.
             about to CAS(top, A, B)...
             ...and is preempted here.

   thread 2: pop() -> gets A, frees it
             pop() -> gets B, frees it
             push(X)
             push(A)      <-- malloc RETURNS THE SAME ADDRESS

   stack:  A -> X -> C          (B is freed!)

   thread 1: resumes. CAS(top, A, B) - top IS still A, so the
             CAS SUCCEEDS.

   stack:  B -> ???             B is freed memory.

   The value was A, then changed, then became A again. The CAS
   cannot tell. Hence A-B-A.

   AND NOTE the second bug hiding in the same window: thread 1
   read `t->next` from a node thread 2 already freed. That is
   a use-after-free even before the ABA fires.
```

The real problem: when can you free?

```
/* "Is any other thread still looking at this node?" has no
   cheap answer, and every solution is a whole subsystem.    */

/* 1. TAGGED POINTERS - fixes ABA, not use-after-free.
      Pack a counter beside the pointer and CAS both at once. */

struct tagged { struct node *ptr; uintptr_t tag; };
_Atomic(struct tagged) top;          /* needs 128-bit CAS:
                                        CMPXCHG16B / CASP     */
/* Every store increments tag, so A-with-tag-7 never equals
   A-with-tag-9. Cheap and effective - but a reader can still
   dereference a node that was freed between its load and its
   CAS. Only sufficient if nodes are never actually freed
   (a free list you own).                                     */

/* 2. HAZARD POINTERS - each thread publishes what it is
      looking at; the reclaimer checks before freeing.       */

_Atomic(struct node *) hazard[MAX_THREADS];

void *pop(void)
{
        struct node *t;
        do {
                t = atomic_load(&top);
                if (!t) return NULL;
                atomic_store(&hazard[my_id], t);   /* announce  */
                if (atomic_load(&top) != t)        /* re-verify */
                        continue;                  /* it moved  */
        } while (!atomic_compare_exchange_weak(&top, &t, t->next));

        atomic_store(&hazard[my_id], NULL);
        retire(t);          /* deferred: free only when no
                               hazard[] entry points at it      */
        return t->val;
}
/* Bounded memory, O(threads) scan per reclaim, and a store +
   fence on every read. C++26 has std::hazard_pointer. */

/* 3. EPOCH-BASED RECLAMATION - readers announce an epoch;
      memory from epoch N is freed once every thread has been
      seen in epoch N+2. This is crossbeam's approach in Rust,
      and it is essentially userspace RCU (chapter 10).
      Cheaper reads than hazard pointers, unbounded memory if
      a thread stalls inside a critical section.             */

/* 4. NEVER FREE - a preallocated pool and a free list.
      The only approach that is genuinely simple, and the one
      real-time systems actually use. Bounded, predictable,
      and it turns the problem into capacity planning.       */

/* 5. A GARBAGE COLLECTOR - which is why lock-free structures
      are dramatically easier in Java, Go and C#. The hardest
      part of the problem is already solved for you.         */
```

RECLAMATION IS THE WHOLE DIFFICULTY

Every lock-free algorithm you read in a paper assumes garbage collection, or waves at "a safe memory reclamation scheme". That hand-wave is 80% of the implementation work and essentially all of the bugs. If you are evaluating a lock-free library, the first question is not "is the algorithm correct" — it is "which reclamation scheme, and what happens when a thread stalls inside a read".

The one you should actually write

```
/* A single-producer single-consumer ring buffer. Lock-free,
   wait-free, no reclamation problem, ~30 lines, and it covers
   a large fraction of real use cases: an audio callback, an
   ISR to a worker, a logging thread.

   It works because with exactly one producer and one consumer,
   each index has exactly one writer.                          */

#define CAP 1024                    /* power of two           */

struct spsc {
        void       *slot[CAP];
        alignas(64) atomic_size_t head;   /* consumer writes   */
        alignas(64) atomic_size_t tail;   /* producer writes   */
        /* alignas(64) keeps them on separate cache lines -
           without it the two threads fight over one line and
           the whole thing runs at a fraction of the speed
           (chapter 16).                                       */
};

bool spsc_push(struct spsc *q, void *v)
{
        size_t t = atomic_load_explicit(&q->tail, memory_order_relaxed);
        size_t h = atomic_load_explicit(&q->head, memory_order_acquire);
        if (t - h == CAP)
                return false;                   /* full        */

        q->slot[t & (CAP - 1)] = v;             /* write data  */
        atomic_store_explicit(&q->tail, t + 1,
                              memory_order_release);  /* publish */
        return true;
}

bool spsc_pop(struct spsc *q, void **out)
{
        size_t h = atomic_load_explicit(&q->head, memory_order_relaxed);
        size_t t = atomic_load_explicit(&q->tail, memory_order_acquire);
        if (h == t)
                return false;                   /* empty       */

        *out = q->slot[h & (CAP - 1)];
        atomic_store_explicit(&q->head, h + 1,
                              memory_order_release);
        return true;
}

/* Note the indices MONOTONICALLY INCREASE and are masked only
   on use. That is what makes full and empty distinguishable
   without wasting a slot, and unsigned wraparound is
   well-defined so it is correct for ever.                     */
```

How it shows up

Symptom	Cause
Rare corruption in a lock-free stack	ABA — a node was freed and reallocated at the same address
Use-after-free reading `t->next`	No reclamation scheme; the node was freed between the load and the CAS
Memory grows without bound	Epoch reclamation with a thread stalled inside a critical section
Lock-free version slower than the mutex	Contention causes retries; losers redo the work
SPSC queue is 10x slower than expected	Head and tail on the same cache line — needs `alignas(64)`
Works on x86, corrupts on ARM	Missing acquire/release; TSO was hiding it
One thread starves for seconds	Lock-free guarantees system progress, not per-thread — use wait-free if that matters
Link error on 128-bit CAS	Tagged pointers without `-mcx16` or `-march=armv8.1-a`
Queue reports full when it is empty	Indices masked before comparison instead of on use

Check yourself

- ✓ Define blocking, lock-free and wait-free in terms of progress guarantees.
- ✓ Give three things lock-free genuinely buys and two it does not.
- ✓ Walk through the ABA sequence step by step.
- ✓ Identify the second bug that occurs in the same window as ABA.
- ✓ Name five reclamation approaches and the cost of each.
- ✓ Explain why lock-free structures are far easier in Java or Go.
- ✓ Write an SPSC ring buffer with correct orderings from memory.
- ✓ Explain why the SPSC queue has no reclamation problem.
- ✓ Say why the indices increase monotonically and are masked only on use.
- ✓ Explain the `alignas(64)` and what happens without it.

Deadlock, livelock and starvation

Three ways for a program to stop making progress, distinguishable in about ten seconds if you know what to look at. Deadlock is also the one concurrency bug with a complete, mechanical prevention strategy — which is why it is inexcusable in new code.

The four conditions

```
Coffman's conditions. Deadlock requires ALL FOUR.
Break any one and it cannot happen.

1. MUTUAL EXCLUSION
     the resource cannot be shared.
     Break it: use immutable data, or copies, or a
     lock-free structure. Often impossible.

2. HOLD AND WAIT
     a thread holds one resource while requesting another.
     Break it: take all locks at once (std::scoped_lock),
     or take none and retry.

3. NO PREEMPTION
     a resource cannot be taken away from its holder.
     Break it: try_lock with a timeout, and back off.

4. CIRCULAR WAIT
     a cycle in the "waits for" graph.
     Break it: a GLOBAL LOCK ORDER.
     <-- THIS IS THE ONE YOU BREAK. Always.

Condition 4 is the practical target because it costs
nothing at runtime, needs no error path, and can be
checked mechanically.
```

```
/* THE CANONICAL DEADLOCK */

void transfer(Account &from, Account &to, int amount)
{
        std::lock_guard<std::mutex> g1(from.m);   /* A then B */
        std::lock_guard<std::mutex> g2(to.m);
        from.balance -= amount;
        to.balance   += amount;
}

/* thread 1: transfer(alice, bob, 100)   locks alice, wants bob
   thread 2: transfer(bob, alice, 50)    locks bob,  wants alice
   -> a cycle. Both wait for ever.                          */


/* FIX 1: a global order. Any total order will do, as long as
   everyone uses the same one. Address works and needs no
   extra state.                                             */
void transfer(Account &from, Account &to, int amount)
{
        Account *a = &from, *b = &to;
        if (a > b) std::swap(a, b);          /* ORDER BY ADDRESS */

        std::lock_guard<std::mutex> g1(a->m);
        std::lock_guard<std::mutex> g2(b->m);
        from.balance -= amount;
        to.balance   += amount;
}
/* Careful: transfer(x, x) now locks the same mutex twice.
   Guard it: if (&from == &to) return; */


/* FIX 2: let the library do it. std::scoped_lock takes N
   mutexes with a deadlock-avoidance algorithm (repeated
   try_lock with back-off).                                 */
void transfer(Account &from, Account &to, int amount)
{
        std::scoped_lock g(from.m, to.m);    /* C++17, safe in
                                                any order       */
        from.balance -= amount;
        to.balance   += amount;
}
/* This is the right default. std::lock() is the C++11 form. */
```

WRITE THE LOCK ORDER DOWN

Not in a wiki — in a header, next to the locks, as a numbered list. "Locks are acquired in this order: (1) `registry.m`, (2) `account.m`, (3) `logger.m`. Never take a lower-numbered lock while holding a higher-numbered one." That comment is checkable in review and by tooling. A lock order that exists only in people's heads has already been violated somewhere.

The deadlocks you did not write

```
/* The cycle is rarely two mutexes in one function. It is
   almost always a call you did not know took a lock.       */

/* 1. THE CALLBACK. */
void Subject::notify() {
        std::lock_guard g(m_);
        for (auto *o : observers_)
                o->on_event();     /* unknown code. It may call
                                      back into US and try to
                                      take m_. Or take ITS lock,
                                      while another thread does
                                      the reverse.             */
}
/* FIX: copy the observer list under the lock, release, then
   call. Or dispatch onto a queue.                          */
void Subject::notify() {
        std::vector<Observer *> copy;
        { std::lock_guard g(m_); copy = observers_; }
        for (auto *o : copy) o->on_event();     /* unlocked    */
}

/* 2. THE ALLOCATOR. malloc takes locks. Allocating inside
      your lock creates an order between your lock and the
      allocator's - and the allocator is taken by everything. */

/* 3. THE LOGGER. Same shape, and worse because logging is
      added to error paths, which is where lock state is
      already unusual.                                       */

/* 4. RECURSION THROUGH A LIBRARY. You call into a framework
      holding your lock; it calls your callback on another
      path that takes the same lock.                         */

/* 5. FORK. In a multithreaded process, fork() gives the child
      ONE thread and a copy of every mutex - including ones
      that were locked by threads that do not exist in the
      child. The child deadlocks on the first malloc.
      This is why POSIX says only async-signal-safe functions
      are legal between fork and exec, and why
      pthread_atfork exists.                                 */

/* 6. SIGNAL HANDLERS. A handler that takes a lock the
      interrupted thread already holds deadlocks that thread
      against itself.                                        */
```

Telling the three apart

	Deadlock	Livelock	Starvation
CPU usage	**Zero**	**100%**	Normal
Progress	None, ever	None, but busy	Some threads progress, one does not
Threads are	Blocked in a wait	Running, retrying	Runnable but never chosen
Typical cause	Circular wait	Mutual back-off, or a CAS loop that always loses	Unfair lock, priority, or a hot producer
Recovers alone?	Never	Sometimes, with randomised back-off	Sometimes, when load drops

```
# Deadlock: attach and look. Every thread will be in a wait.
$ gdb -p $(pidof app)
(gdb) thread apply all bt
Thread 3: __lll_lock_wait () ... at accounts.cpp:41
Thread 2: __lll_lock_wait () ... at accounts.cpp:41
# Two threads, same line, both waiting. That is your cycle.

$ eu-stack -p $(pidof app)            # lighter than gdb
$ py-spy dump --pid $(pidof app)      # Python
$ jstack $(pidof java)                # Java PRINTS the cycle:
#   "Found one Java-level deadlock"

# Live detection - use it in every debug build:
$ valgrind --tool=helgrind ./app
==1234== Thread #2: lock order "0x5a1000 before 0x5a1040" violated

# The kernel's version, and the best of them:
$ echo 1 > /proc/sys/kernel/lock_stat      # with CONFIG_LOCKDEP
# lockdep learns the lock order from observed acquisitions and
# reports a POTENTIAL inversion the first time it sees one -
# even if the deadlock never actually happens. That is the
# property you want: it finds the bug on the run where it did
# NOT deadlock.

# In C++, the same idea, free:
$ clang++ -Wthread-safety -Wthread-safety-analysis ...
# with GUARDED_BY / ACQUIRED_BEFORE annotations.
```

TIMEOUTS ARE A DIAGNOSTIC, NOT A FIX

`try_lock_for(5s)` converts a hang into an error you can log — genuinely useful, and it makes production survivable. It does not remove the cycle. Code that retries after a timeout can livelock instead, and code that gives up leaves the invariant half-updated. Use timeouts to detect and report; fix the order.

Livelock and starvation

```
/* LIVELOCK: everyone is polite, nobody progresses. */

bool transfer(Account &a, Account &b) {
        while (true) {
                std::unique_lock la(a.m);
                std::unique_lock lb(b.m, std::try_to_lock);
                if (lb.owns_lock()) { do_it(); return true; }
                /* release both and retry - no deadlock ... */
        }   /* ... but two threads can retry in lockstep for
               ever, each releasing just as the other tries.  */
}
/* FIX: randomised exponential back-off, so the two threads
   desynchronise. This is precisely why Ethernet uses it. */
                std::this_thread::sleep_for(
                        std::chrono::microseconds(rand() % (1 << n)));

/* STARVATION: progress happens, but not for you. */

/* Causes:
     - a reader-preferring RW lock and continuous readers
       (chapter 10)
     - an unfair mutex where a barging thread re-acquires
       before the queued waiter is scheduled
     - priority: a SCHED_FIFO thread starving everything below
     - a CAS loop where one thread always loses to a faster
       core, or to a core with better cache locality

   Fixes: ticket locks, fair queueing, priority ageing, or
   accepting it and measuring the p99.9 rather than the mean. */
```

The rules that prevent all of this

1. **Define a global lock order and document it in the header.** Break circular wait and the other three conditions do not matter.
2. **Take multiple locks with `scoped_lock`,** never by hand.
3. **Never call unknown code while holding a lock.** No callbacks, no virtual calls, no allocation in hot paths, no logging.
4. **Never hold a lock across I/O or a sleep.**
5. **Prefer one coarse lock** until a profile forces you to split it. Two locks are more than twice the risk of one.
6. **Run lockdep, Helgrind or `-Wthread-safety` in CI.** All three find inversions on runs that did not hang.
7. **Add timeouts in production** so a hang becomes a logged error with a stack, not a mystery.

How it shows up

Symptom	Cause
Hang, zero CPU, two threads on the same lock line	Classic circular wait
Hang, 100% CPU	Livelock, or a CAS loop that never wins

Symptom	Cause
One request in a thousand takes 30 seconds	Starvation under an unfair lock
Deadlock only when logging is enabled	The logger's lock introduced a new order
Deadlock only under memory pressure	An allocation inside a lock, and the allocator took its own
Child process hangs after `fork`	A mutex copied in a locked state; only the forking thread survived
Hang inside a signal handler	The handler took a lock the interrupted thread held
Deadlock appears after adding an observer	A callback invoked under the lock
Works for months, deadlocks after a refactor	Two call paths now acquire in opposite orders
Helgrind reports an inversion that never hangs	Correct and valuable — it found a latent cycle

Check yourself

- ✓ State the four Coffman conditions and say which one you break in practice, and why that one.
- ✓ Write the classic transfer deadlock and give two distinct fixes.
- ✓ Explain the edge case in ordering by address and how to guard it.
- ✓ Give six ways a lock gets taken that you did not write.
- ✓ Explain why `fork` in a multithreaded process is dangerous.
- ✓ Distinguish deadlock, livelock and starvation by CPU usage and thread state.
- ✓ Describe how lockdep finds a deadlock on a run that did not deadlock.
- ✓ Explain why a timeout is a diagnostic rather than a fix.
- ✓ Explain why livelock needs randomised back-off specifically.
- ✓ List the seven prevention rules and identify which two you currently violate.

Priority inversion and real-time concurrency

A high-priority thread blocked behind a low-priority one is not a scheduling curiosity; it is the failure that put Mars Pathfinder into a reset loop on another planet. This chapter covers the mechanism, the two standard fixes, and what "real-time" actually demands of concurrent code.

The mechanism

```
Three threads. H high, M medium, L low. H and L share a
mutex; M shares nothing.

time -->

L    [--lock--][ running ]
M                        [======== running, forever =======]
H              [wants the lock ... blocked ... blocked ....]

1. L takes the mutex and starts work.
2. H becomes runnable, preempts L, and blocks on the mutex.
3. L is runnable again - but M, which is higher priority
   than L, preempts it.
4. L never runs, so it never releases the mutex, so H never
   runs.

H is now effectively running at L's priority, and is
blocked by M - a thread it outranks and shares nothing
with. That is UNBOUNDED PRIORITY INVERSION: the delay has
no upper bound, because any number of medium-priority
threads can keep arriving.

BOUNDED inversion - H waiting while L finishes its critical
section - is unavoidable and fine. You bound it by keeping
critical sections short. It is the UNBOUNDED case, caused
by a third thread, that must be designed out.
```

MARS PATHFINDER, 1997

The lander began resetting on the Martian surface. A high-priority bus-management task shared a mutex with a low-priority meteorological task; a medium-priority communications task preempted the low one, the high-priority task missed its deadline, and the watchdog rebooted the spacecraft. The fix was uploaded from Earth: enable priority inheritance on that mutex — a flag that VxWorks supported and that had been left off.

The two protocols

```
/* PRIORITY INHERITANCE (PI)
   While L holds a mutex that H wants, L runs at H's priority.
   As soon as it releases, it drops back.                    */

pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_setprotocol(&a, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&m, &a);

/*   L    [--lock--][boosted to H's priority][unlock]
     M                                      [ runs ]
     H              [blocked][----------------------][runs]

   M can no longer preempt L, because L is temporarily as
   important as H. The inversion is now BOUNDED by the length
   of L's critical section.

   Cost: the scheduler must track the boost, and inheritance
   is TRANSITIVE - if L is itself blocked on a mutex held by
   K, K gets boosted too. Chains are handled but cost more.

   This is the default choice, and what Linux's PREEMPT_RT,
   VxWorks and QNX use.                                      */


/* PRIORITY CEILING (PCP / immediate ceiling)
   Every mutex is given a CEILING: the priority of the
   highest-priority thread that will ever take it. Any thread
   that acquires it is immediately raised to the ceiling.   */

pthread_mutexattr_setprotocol(&a, PTHREAD_PRIO_PROTECT);
pthread_mutexattr_setprioceiling(&a, 80);

/*   Advantages over inheritance:
       * no boost tracking at runtime - the raise is
         unconditional and cheap
       * prevents DEADLOCK among ceiling-protected mutexes,
         provably, because a thread can only ever block on a
         mutex whose ceiling exceeds its current priority
       * better worst-case analysis for certification

     Disadvantages:
       * you must know every user of every mutex in advance,
         and keep the ceilings correct as the code changes
       * a low-priority thread is boosted even when no
         high-priority thread wants the lock - it "wastes"
         priority

   Use PCP in certified systems where the thread set is fixed
   and analysed. Use PI everywhere else.                     */
```

> **ONLY MUTEXES CAN DO THIS**
>
> Priority inheritance requires an **owner**. A semaphore has none — anyone may post it — so there is nobody to boost. The same applies to condition variables, barriers, and hand-rolled spinlocks. If you are using a binary semaphore as a mutex in a real-time system (chapter 9), you have no protection against unbounded inversion. This is the single strongest argument for using a real mutex.

What real-time actually requires

```
"Real-time" does not mean fast. It means BOUNDED.
A system with a guaranteed 10 ms response is real-time.
One that is usually 10 us and occasionally 200 ms is not.

THE THINGS THAT MUST BE BOUNDED

1. every critical section          measurable, short
2. priority inversion              PI or PCP mutexes
3. memory allocation               PREALLOCATE. malloc has
                                   unbounded worst case and
                                   its own locks
4. page faults                     mlockall(MCL_CURRENT |
                                   MCL_FUTURE) - a demand
                                   page fault is milliseconds
5. interrupt latency               threaded IRQs, or a small
                                   bounded top half
6. the scheduler itself            SCHED_FIFO/RR/DEADLINE,
                                   not CFS
7. lock acquisition                fair (ticket/FIFO) locks,
                                   or no locks at all
8. cache and TLB behaviour         pinning, and sometimes
                                   cache partitioning

MISS ANY ONE and the system has an unbounded tail, however
good the other seven are. This is why real-time work feels
like removing things rather than adding them.
```

```
/* The standard preamble for a real-time thread on Linux. */

#include <sched.h>
#include <sys/mman.h>
#include <malloc.h>

void rt_setup(void)
{
        /* 1. Never be swapped or demand-paged. */
        mlockall(MCL_CURRENT | MCL_FUTURE);

        /* 2. Stop the allocator returning memory to the OS and
              having to fault it back in. */
        mallopt(M_TRIM_THRESHOLD, -1);
        mallopt(M_MMAP_MAX, 0);

        /* 3. Touch the whole stack now, so no fault later. */
        char scratch[512 * 1024];
        memset(scratch, 0, sizeof scratch);

        /* 4. Real-time policy. */
        struct sched_param p = { .sched_priority = 80 };
        pthread_setschedparam(pthread_self(), SCHED_FIFO, &p);

        /* 5. Pin, and use a core isolated with isolcpus and
              nohz_full so the kernel stops interrupting it. */
        cpu_set_t s; CPU_ZERO(&s); CPU_SET(3, &s);
        sched_setaffinity(0, sizeof s, &s);
}

/* And a PI mutex for anything shared with a lower-priority
   thread: */
pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_setprotocol(&a, PTHREAD_PRIO_INHERIT);
pthread_mutex_init(&shared_lock, &a);

/* THEN MEASURE, because every one of those steps is a claim: */
$ cyclictest -t1 -p80 -a3 -i200 -m -n -l 10000000
T: 0 (1234) P:80 I:200 C:10000000 Min:2 Act:3 Avg:3 Max:14
/* Max is the number that matters. Run it for hours, under
   load, with the real workload running. A Max that grows the
   longer you run means something is unbounded.             */
```

Inversion outside the scheduler

```
/* The same shape appears wherever a queue is shared. */

  A LOCK CONVOY
    Many threads pass one lock in strict FIFO order. Each
    releases, the next is woken, but the releaser immediately
    tries again and now queues at the back. Throughput
    collapses to one handoff per context switch (~µs) even
    though the critical section is nanoseconds.
    FIX: barging (unfair) locks, or shard the data.

  THE THUNDERING HERD
    N threads woken for one item; N-1 immediately sleep again.
    FIX: signal instead of broadcast (chapter 8), or a queue
    per worker.

  I/O PRIORITY INVERSION
    A high-priority thread blocks on a page from a file whose
    read was queued behind a low-priority thread's bulk copy.
    FIX: ionice, cgroup I/O weights, or read ahead.

  MEMORY ALLOCATOR INVERSION
    Your RT thread calls malloc, which takes the arena lock a
    low-priority thread holds while the allocator does an
    mmap. Unbounded, and invisible in your source.
    FIX: preallocate. This is why RT code has pools.

  In every case the pattern is: a scarce serialised resource,
  and a scheduling policy that does not know about it.      */
```

How it shows up

Symptom	Cause
High-priority thread misses deadlines under load	Unbounded priority inversion — enable `PTHREAD_PRIO_INHERIT`
Watchdog resets the system intermittently	The Pathfinder failure, exactly
Raising a thread's priority made latency worse	It now blocks on a lock held by something it outranks
Latency fine in test, terrible in production	Medium-priority load absent in test is the third thread in the inversion
Occasional millisecond spikes in an RT loop	A page fault, a `malloc`, or a migration — one of the eight bounds is missing
Binary semaphore gives no protection	Semaphores have no owner, so no inheritance
Throughput collapses at a specific thread count	A lock convoy
`cyclictest` Max grows the longer you run	Something is unbounded; the average is irrelevant
Works on bare metal, misses deadlines in a VM	The vCPU is descheduled by the host (virtualisation chapter 8)
`pthread_mutexattr_setprotocol` returns ENOTSUP	The platform has no PI mutexes — a design constraint you must plan around

Check yourself

- ✓ Draw the three-thread inversion and say which thread causes the unbounded part.
- ✓ Distinguish bounded from unbounded inversion and say which is acceptable.

- ✓ Explain priority inheritance and what "transitive" means for it.
- ✓ Explain priority ceiling, and give one advantage and one disadvantage over inheritance.
- ✓ Explain why a semaphore cannot offer either protocol.
- ✓ Define real-time in one sentence without using the word "fast".
- ✓ List the eight things that must be bounded, and name what breaks each.
- ✓ Write the real-time thread preamble and justify every step.
- ✓ Explain what `cyclictest`'s Max tells you that Avg does not.
- ✓ Describe a lock convoy and how it differs from ordinary contention.

Cache effects: false sharing and contention

The most common reason parallel code is slower than serial code has nothing to do with locks or algorithms. It is that two cores are fighting over one 64-byte line of memory that they do not even logically share — and nothing in the source code shows it.

Why a cache line is the unit of everything

```
Coherence protocols operate on CACHE LINES, not variables.
A line is 64 bytes on x86 and most ARM (128 on Apple
Silicon's performance cores, which catches people out).

MESI, simplified. Each line, in each core's cache, is in
one of four states:

  M odified   I have it, it is dirty, nobody else has it
  E xclusive  I have it, it is clean, nobody else has it
  S hared     several caches have it, all clean
  I nvalid    I do not have it

THE RULE THAT COSTS YOU:
  to WRITE a line, a core must own it EXCLUSIVELY.
  So every write must INVALIDATE every other copy.

TWO CORES READING the same line:
  both in S. No traffic at all after the first fetch.
  Reads scale perfectly.

TWO CORES WRITING the same line, alternately:
  core 1 writes -> must invalidate core 2's copy
  core 2 writes -> must invalidate core 1's copy
  ... the line PING-PONGS between them.

  Each transfer costs 50-200 cycles on the same die, and
  far more across sockets. Do it in a tight loop and the
  two cores together are slower than one core alone.
```

False sharing

```
/* TRUE sharing: two threads use the same variable. The cost
   is inherent - you asked for it.

   FALSE sharing: two threads use DIFFERENT variables that
   happen to sit in the same cache line. The cost is pure
   accident, and pure waste.                                */

struct counters {
        long a;          /* thread 0 increments this  */
        long b;          /* thread 1 increments this  */
};                       /* 16 bytes: ONE cache line  */

/* Threads 0 and 1 touch nothing in common. Logically there is
   no sharing at all. But every increment of `a` invalidates
   the line holding `b`, so thread 1 must re-fetch it - and
   vice versa, millions of times per second.

   Measured effect: commonly 5-20x slower than the same code
   with the fields separated. On a 2-socket machine, worse. */

/* THE FIX: pad to a cache line. */

#define CACHELINE 64

struct counters {
        alignas(CACHELINE) long a;
        alignas(CACHELINE) long b;
};                       /* 128 bytes, two lines, zero
                            interference                    */

/* C++17 gives you the portable constants: */
#include <new>
struct counters {
        alignas(std::hardware_destructive_interference_size) long a;
        alignas(std::hardware_destructive_interference_size) long b;
};
/* hardware_destructive_interference_size  = keep apart
   hardware_constructive_interference_size = keep together   */

/* Linux kernel: */
        struct foo {
                int a;
                ____cacheline_aligned int b;
        };
/* Or the whole-struct form, for per-CPU data:              */
        DEFINE_PER_CPU_ALIGNED(struct stats, my_stats);
```

> **THE DIAGNOSTIC QUESTION**
> "Do two threads write to memory within 64 bytes of each other?" If yes, and it is in a hot loop, that is almost certainly your problem. Note it is *within 64 bytes*, not *the same variable* — adjacent array elements, adjacent struct members, and two objects from the same allocation batch all qualify.

Where it hides

```
/* 1. AN ARRAY INDEXED BY THREAD ID - the classic. */
long results[NUM_THREADS];            /* 8 threads = 64 bytes
                                         = ONE line. Every
                                         thread fights every
                                         other thread.        */
/* FIX: */
struct padded { alignas(64) long v; };
struct padded results[NUM_THREADS];

/* 2. A LOCK NEXT TO THE DATA IT PROTECTS. */
struct node {
        pthread_mutex_t lock;
        int             data;         /* same line as the lock */
};
/* Waiters spinning on the lock word invalidate the line, so
   the HOLDER - who is trying to work on `data` - keeps losing
   its cache line to the very threads waiting for it. Separate
   them, or accept it for cold locks.                         */

/* 3. HEAD AND TAIL OF A QUEUE. Producer writes tail, consumer
      writes head. Adjacent -> every push fights every pop.
      This is why the SPSC queue in chapter 13 has
      alignas(64) on both.                                    */

/* 4. REFERENCE COUNTS INSIDE SHARED OBJECTS. Many readers
      touching an object they only READ still write its
      refcount, so a "read-only" workload writes constantly.
      This is a large part of why shared_ptr copies are
      expensive, and why RCU (chapter 10) is worth the
      complexity for read-mostly data.                        */

/* 5. FALSE SHARING YOU INHERITED. Two unrelated globals the
      linker placed adjacently. Two objects malloc returned
      from the same 64-byte chunk. A vector<bool> where
      "different" elements are bits of one byte.              */

/* 6. std::atomic MEMBERS IN A STRUCT. Atomics make the
      invalidation explicit and frequent - false sharing
      between two atomics is the worst case of all.           */
```

Measuring it

```
# The definitive counter on x86: HITM - a cache hit in a
# MODIFIED line in ANOTHER core's cache. That is the ping-pong.

$ perf c2c record ./app
$ perf c2c report --stdio
# Shared Data Cache Line Table
#  Total  Tot  ----- LLC Load Hitm -----
#  Rmt  Lcl   Hitm    Total  Lcl  Rmt   Data address
#  ...  ...   82.4%    1621  1338  283   0x55d4a2f01840
#
#   ----- Offset ---  Code address   Symbol
#         0x0            worker+0x24  results[tid]++
#         0x8            worker+0x24  results[tid]++
#
# TWO DIFFERENT OFFSETS in ONE line, both hot, both written.
# That is false sharing, named and located.

# Cheaper first check:
$ perf stat -e cache-misses,cache-references,\
LLC-load-misses,LLC-store-misses ./app

# On ARM, the equivalent counters vary by implementation:
$ perf list | grep -i 'snoop\|coheren'

# THE ZERO-TOOL TEST, and it is a good one:
#   pad the suspect structure to 64 bytes and re-run.
#   If it gets dramatically faster, you found it.
#   If nothing changes, it was not false sharing - stop
#   guessing and go back to the profiler.
```

Designing for it

```
/* THE GENERAL PRINCIPLE: separate by WRITER, not by meaning. */

/* BAD: grouped by what the fields mean. */
struct connection {
        int             fd;             /* read by all       */
        atomic_long     bytes_in;       /* written by reader */
        atomic_long     bytes_out;      /* written by writer */
        atomic_int      refs;           /* written by all    */
        struct sockaddr peer;           /* read by all       */
};

/* GOOD: grouped by who writes them. */
struct connection {
        /* immutable after construction - shared freely, and
           never invalidated. Put it first and it stays hot. */
        int             fd;
        struct sockaddr peer;

        alignas(64) atomic_long bytes_in;    /* reader thread */
        alignas(64) atomic_long bytes_out;   /* writer thread */
        alignas(64) atomic_int  refs;        /* everyone      */
};

/* AND THE BETTER ANSWER: do not share the counters at all. */

struct stats { alignas(64) long n, bytes; };
struct stats per_thread[NUM_THREADS];

void record(int tid, long b) {          /* no atomics, no
        per_thread[tid].n++;               contention, plain
        per_thread[tid].bytes += b;        increments        */
}

long total_n(void) {                    /* pay once, when you
        long t = 0;                        actually need it   */
        for (int i = 0; i < NUM_THREADS; i++)
                t += per_thread[i].n;
        return t;
}
/* Sharding beats optimising the shared case. A per-thread
   counter with a periodic merge is faster than the cleverest
   atomic, because it generates no coherence traffic at all. */
```

> **DO NOT PAD EVERYTHING**
>
> Padding costs memory and cache capacity. A struct padded from 24 to 128 bytes means five times fewer fit in L2, and a traversal that was cache-friendly becomes cache-hostile. Pad only fields that are *written by different threads in a hot loop*. Everything else benefits from being packed — `hardware_constructive_interference_size` exists to say "keep these together" for exactly that reason.

How it shows up

Symptom	Cause
Parallel version slower than serial, no locks involved	False sharing
Scaling is negative past 2–4 threads	A line ping-ponging between more and more cores

Symptom	Cause
Adding a field to a struct halved throughput	It pushed two hot fields into one line, or shifted alignment
Performance changes with unrelated code edits	The linker moved two globals into the same line
Much worse on a 2-socket machine	Cross-socket coherence traffic is far more expensive
Worse on Apple Silicon than expected	128-byte lines; 64-byte padding is not enough
Atomic counter is the hottest line in `perf c2c`	True sharing — shard it instead
A "read-only" workload generates write traffic	Reference counts, or a shared iterator
Padding made it slower	You padded a cold structure and destroyed its cache density
Lock holder is starved by its own waiters	Lock word in the same line as the protected data

Check yourself

- ✓ Explain why writing requires exclusive ownership of a line, and what that costs.
- ✓ Distinguish true from false sharing with an example of each.
- ✓ Explain why `long results[8]` indexed by thread id is a performance disaster.
- ✓ Name six places false sharing hides.
- ✓ Give the two C++17 interference-size constants and say what each is for.
- ✓ Describe what `perf c2c` shows and what "two offsets in one line" means.
- ✓ Give the zero-tool test for false sharing and how to interpret both outcomes.
- ✓ Restructure a struct to group by writer rather than by meaning.
- ✓ Explain why sharding beats optimising the shared counter.
- ✓ Say when padding is wrong, and what it costs.

Thread pools, work queues and work stealing

Creating a thread costs about as much as ten thousand mutex operations, so you create them once and feed them work. That single decision brings its own set of problems: how many threads, what happens when the queue fills, and the deadlock that every pool eventually hits.

The shape

```
    submit(task) ---> +---------------------------+
                      |      TASK QUEUE           |
                      +---------------------------+
                         |        |        |
                      +--v--+  +--v--+  +--v--+
                      | W 0 |  | W 1 |  | W 2 |    N workers,
                      +-----+  +-----+  +-----+    created once

    Each worker:
         for (;;) {
                 task = queue.pop();     /* blocks when empty  */
                 if (!task) break;       /* shutdown sentinel  */
                 task();
         }

    WHAT THE POOL BUYS
      * thread creation cost paid once, not per task
      * a bound on concurrency - N threads, not N-thousand
      * a natural place to apply backpressure
      * warm caches: a worker keeps running the same code

    WHAT IT COSTS
      * the queue is a contention point
      * a blocking task occupies a worker doing nothing
      * thread-local state now belongs to the WORKER, not the
        task (chapter 2)
      * and the deadlock in the "recursion" section below
```

How many threads

```
/* THE ONLY TWO RULES YOU NEED */

/* CPU-BOUND work: N = number of cores.
   More threads do not create more CPU; they add context
   switches and cache pressure. */
        unsigned n = std::thread::hardware_concurrency();
        /* subtract one if the main thread also works */

/* I/O-BOUND work: N = cores / (1 - blocked_fraction)
   A thread blocked 90% of the time uses 10% of a core, so
   you need ~10x more threads to keep the cores busy. */

        /* measured: each task spends 45 ms waiting, 5 ms
           computing -> blocked_fraction = 0.9
           8 cores / (1 - 0.9) = 80 threads               */

/* AND THE REAL ADVICE: if the answer is "80 threads", you
   probably want async I/O and a small pool instead
   (chapter 18). Eighty stacks is 640 MB of reserved address
   space and eighty things for the scheduler to consider. */

/* MIXED WORKLOADS: use SEPARATE POOLS, sized differently.
   One CPU pool of `cores` threads, one I/O pool of many.
   A single pool sized for I/O will thrash on CPU work; sized
   for CPU it will stall on I/O.                           */

/* MEASURE, do not calculate:
     * are all workers busy?         -> add threads
     * are cores idle with work queued?  -> workers are
                                           blocking
     * is throughput flat but latency rising? -> the queue is
                                           growing; you are
                                           past capacity     */
```

A pool, complete

```
#include <thread>
#include <mutex>
#include <condition_variable>
#include <deque>
#include <functional>
#include <future>
#include <vector>

class ThreadPool {
public:
    explicit ThreadPool(unsigned n, size_t max_queue = 4096)
        : max_(max_queue)
    {
        for (unsigned i = 0; i < n; i++)
            workers_.emplace_back([this] { run(); });
    }

    ~ThreadPool() { shutdown(); }

    /* Returns a future, so the caller can wait for a result -
       and, crucially, so an exception in the task surfaces at
       get() instead of calling std::terminate. */
    template <class F>
    auto submit(F f) -> std::future<decltype(f())>
    {
        using R = decltype(f());
        auto task = std::make_shared<std::packaged_task<R()>>(
                        std::move(f));
        auto fut = task->get_future();
        {
            std::unique_lock<std::mutex> lk(m_);

            /* BACKPRESSURE. An unbounded queue is not a
               feature - it converts "too slow" into "out of
               memory", much later, somewhere else. */
            not_full_.wait(lk, [this] {
                    return q_.size() < max_ || stop_; });
            if (stop_) throw std::runtime_error("pool stopped");

            q_.emplace_back([task] { (*task)(); });
        }
        not_empty_.notify_one();
        return fut;
    }

    void shutdown()
    {
        { std::lock_guard<std::mutex> g(m_);
          if (stop_) return;
          stop_ = true; }
        not_empty_.notify_all();      /* release every waiter  */
        not_full_.notify_all();
        for (auto &t : workers_)      /* THEN join. Signal all,
            if (t.joinable()) t.join();  then join all
                                         (chapter 3).          */
        workers_.clear();
    }

private:
    void run()
```

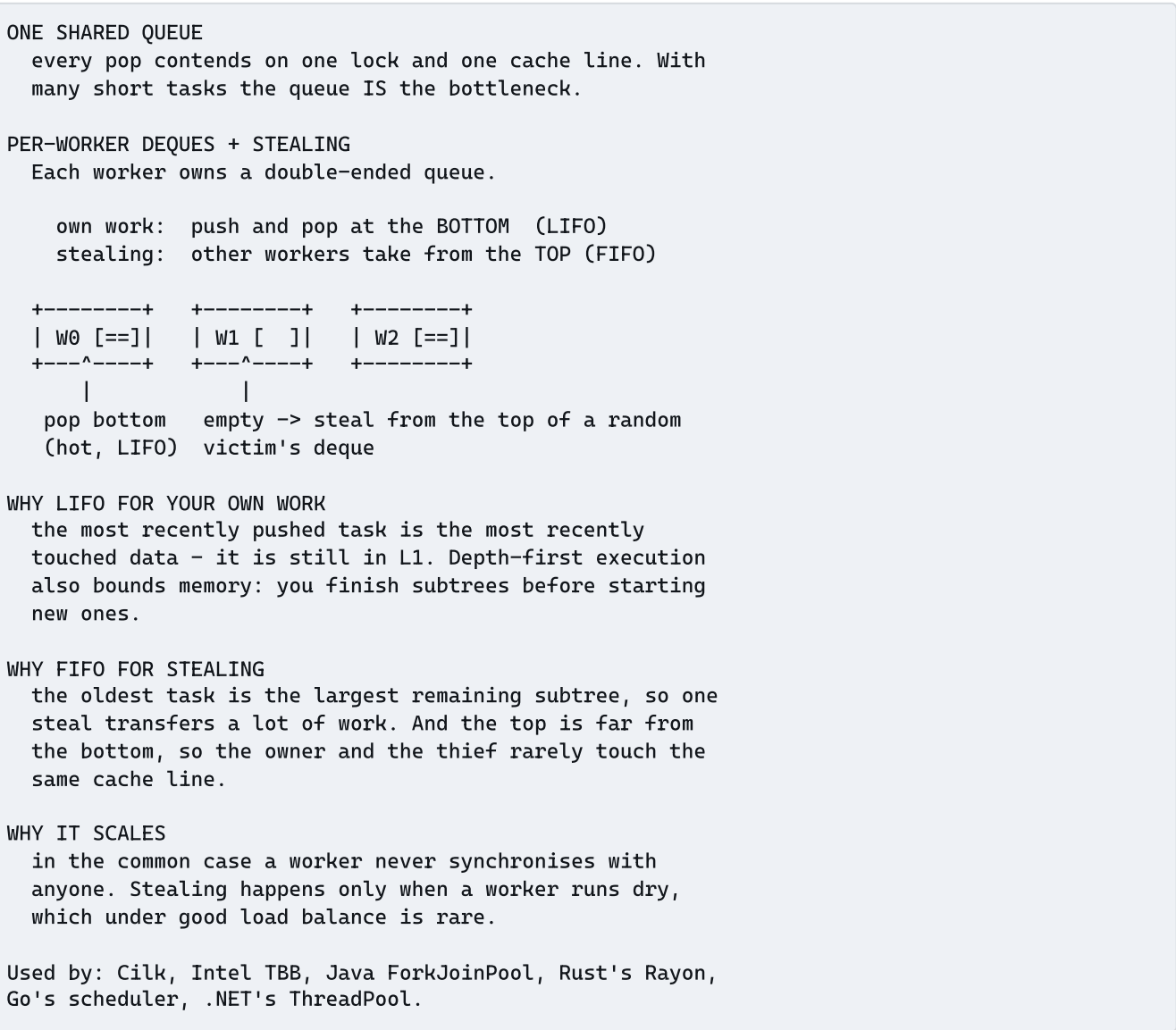
```
    {
        for (;;) {
            std::function<void()> task;
            {
                std::unique_lock<std::mutex> lk(m_);
                not_empty_.wait(lk, [this] {
                        return !q_.empty() || stop_; });
                if (q_.empty()) return;     /* stopping, drained */
                task = std::move(q_.front());
                q_.pop_front();
            }
            not_full_.notify_one();

            /* A task that throws must not kill the worker, or
               the pool silently shrinks to zero and everything
               hangs. packaged_task captures the exception into
               the future; this catch is the belt to that
               braces. */
            try { task(); }
            catch (...) { /* log it */ }
        }
    }

    std::mutex                          m_;
    std::condition_variable             not_empty_, not_full_;
    std::deque<std::function<void()>> q_;
    std::vector<std::thread>            workers_;
    size_t                              max_;
    bool                                stop_ = false;
};
```

THE DEADLOCK EVERY POOL EVENTUALLY HITS

A task submits another task to the *same* pool and waits for it. With N workers, N such tasks occupy every worker, and the sub-tasks they are waiting for can never be scheduled. The pool is full of threads waiting for work that cannot start. It works fine in testing with one request and deadlocks under load.

```
/* THE DEADLOCK */
pool.submit([&] {
        auto f = pool.submit([] { return part_two(); });
        return part_one() + f.get();     /* <-- waits, holding
                                               a worker            */
});

/* FIXES, in order of preference:

   1. DO NOT BLOCK IN A TASK. Restructure so the continuation
      is itself a task, triggered by completion. This is what
      async/await and future.then() are for (chapter 18).

   2. SEPARATE POOLS. Parent tasks in pool A, children in
      pool B. The cycle is broken because they are different
      resources - the same reasoning as lock ordering.

   3. HELP WHILE WAITING. The waiting thread runs queued tasks
      instead of blocking. This is what work-stealing runtimes
      (TBB, Rayon, Go, .NET) do, and it is why they tolerate
      nested parallelism. Note it changes the semantics: your
      task can now be re-entered on the same thread.

   4. AN UNBOUNDED POOL that grows when all workers block.
      .NET's injection heuristic. Effective, and it makes
      thread counts unpredictable.                          */
```

Work stealing



> **THREAD-LOCAL STATE BELONGS TO THE WORKER, NOT THE TASK**
> A `thread_local` holding "the current request id" is set by task 1 and still there when unrelated task 2 runs on the same worker. With work stealing it is worse: a task can migrate mid-tree, so even "the same task" may see different thread-local state. Either clear it at every task boundary, or pass context explicitly. This is a common source of logs attributing work to the wrong request.

How it shows up

Symptom	Cause
Pool hangs under load, fine in tests	Nested submit-and-wait exhausting the workers
Pool silently stops processing	A task threw and killed its worker; repeat until zero remain
Memory grows until the process dies	Unbounded queue absorbing a permanent overload
Cores idle with a long queue	Workers blocked on I/O — wrong pool size, or wrong model
Throughput flat, latency climbing	Past capacity; the queue is the delay
Queue lock is the hottest thing in the profile	Tasks too small — batch them, or use work stealing

Symptom	Cause
Log lines attributed to the wrong request	`thread_local` context not cleared between tasks
Shutdown hangs	Workers blocked in `wait` with no stop flag in the predicate
Adding threads made it slower	Context switching and cache pressure past the core count
Deadlock only with work stealing enabled	A task was re-entered on a thread already inside it

Check yourself

- ✓ Give three things a pool buys and three it costs.
- ✓ Size a pool for CPU-bound and for I/O-bound work, and say when the I/O answer means you want async instead.
- ✓ Explain why an unbounded task queue is a bug, not a feature.
- ✓ Explain what happens when a task throws, and two places to catch it.
- ✓ Describe the nested-submit deadlock and give four fixes in order of preference.
- ✓ Explain why work stealing uses LIFO locally and FIFO for thieves.
- ✓ Explain why work stealing scales when a single shared queue does not.
- ✓ Say why `thread_local` is dangerous in a pool, and worse with stealing.
- ✓ Write the shutdown sequence and explain the order of its steps.

Asynchronous models: events, coroutines, async/await

A thread blocked on a socket costs a megabyte of stack and a scheduler slot to do nothing. Asynchronous models remove that cost by making waiting explicit — and charge you in a different currency: your code turns inside out, and the function colour problem arrives.

The problem with a thread per connection

```
10,000 concurrent connections, one thread each:

  10,000 x 8 MB reserved stack   = 80 GB of address space
  10,000 kernel task structures
  a run queue the scheduler must reason about
  context switches whenever any of them wakes

Most of those threads are doing NOTHING - waiting for a
packet. The work is tiny; the waiting is everything.

THE ASYNC ALTERNATIVE

  one thread (or a few), and a readiness list:

    for (;;) {
            events = wait_for_readiness(all_fds);
            for (e : events)
                    handle(e);      /* never blocks */
    }

  10,000 connections =  ~10,000 small state objects,
  a few hundred bytes each, and one thread.

THE TRADE

  you may NEVER BLOCK inside a handler. One blocking call -
  a DNS lookup, a disk read, a mutex - stalls every other
  connection on that thread. In a threaded model it stalls
  only itself.

  That single rule is what makes async both fast and
  hostile to write.
```

Readiness versus completion

	Readiness (`epoll`, `kqueue`)	**Completion (`io_uring`, IOCP)**
Kernel tells you	"you can read now"	"your read is done, here is the data"
You then	Call `read()`, which returns immediately	Nothing — the buffer is already filled
Syscalls per operation	Two: wait, then read	Can be zero — submit and reap in batches
Buffers	Allocated when the data arrives	Must be committed *before*, and pinned until completion
Regular files	**Does not work** — a file is always "ready", so `read()` blocks anyway	Works properly

	Readiness (`epoll` , `kqueue`)	Completion (`io_uring` , IOCP)
Where	Linux, BSD, macOS	Windows (IOCP) since the 1990s, Linux (`io_uring`) since 5.1

```
/* The readiness loop, which is what most async runtimes wrap. */

int ep = epoll_create1(0);

struct epoll_event ev = {
        .events = EPOLLIN | EPOLLET,     /* edge-triggered     */
        .data.ptr = conn,
};
epoll_ctl(ep, EPOLL_CTL_ADD, conn->fd, &ev);

for (;;) {
        struct epoll_event out[64];
        int n = epoll_wait(ep, out, 64, -1);
        for (int i = 0; i < n; i++)
                handle(out[i].data.ptr);
}

/* LEVEL vs EDGE triggered - the classic epoll bug:

     LEVEL (default): reports while data remains. Forgiving.
     EDGE  (EPOLLET): reports only on a CHANGE. Fewer wake-ups,
                      and you MUST read until EAGAIN or you
                      will never be told again - the connection
                      simply goes silent and stays silent.

   With EPOLLET the loop is:
        while ((n = read(fd, buf, sizeof buf)) > 0) consume();
        if (n < 0 && errno != EAGAIN) error();

   Forget the loop and you get a hang that appears only when a
   message spans two reads.                                    */
```

Callbacks, and why they became coroutines

```
/* CALLBACK STYLE - the control flow is inside out. */

read_async(sock, buf, [](int n) {
        parse_async(buf, n, [](Request r) {
                db_query_async(r, [](Row row) {
                        render_async(row, [](string html) {
                                write_async(sock, html, [](int) {
                                        /* five levels deep, and
                                           now try adding error
                                           handling to each   */
                                });
                        });
                });
        });
});

/* PROBLEMS
     * error handling at every level, or errors vanish
     * no try/catch, no RAII scope - the stack is gone by the
       time the callback runs
     * lifetimes are manual: who owns `buf` across five
       callbacks? Use-after-free is the default outcome
     * a loop becomes a recursive callback chain
     * debugging: the stack trace shows the event loop, not
       how you got there                                    */


/* ASYNC/AWAIT - the same machine, written as straight code. */

async Task<void> serve(Socket sock) {
        try {
                auto n    = co_await read_async(sock, buf);
                auto req  = parse(buf, n);
                auto row  = co_await db_query_async(req);
                auto html = render(row);
                co_await write_async(sock, html);
        } catch (const IoError &e) {          /* works again    */
                log(e);
        }
}   /* destructors run. RAII works. Loops are loops. */

/* WHAT THE COMPILER DOES: splits the function at every await
   into a state machine, and heap-allocates the locals that
   live across a suspension point. `co_await` is "return to
   the caller, and resume here when the awaited thing
   completes".                                             */
```

STACKLESS VERSUS STACKFUL

Stackless coroutines (C++20, Rust, C#, JavaScript) store only the variables that survive a suspension — tens of bytes, and the compiler computes the size. You can only suspend inside a coroutine, which is why `await` is contagious. **Stackful** coroutines and goroutines get a real (growable) stack, so any function can yield — no colouring — at the cost of kilobytes each and a runtime that manages stacks. Go chose stackful; almost everyone else chose stackless.

The function colour problem

```
/* An async function can only be awaited from an async
   function. So async spreads up the call graph until it
   reaches main.                                          */

        int compute(int x);                    /* "red"  - sync */
        async Task<int> fetch(int x);          /* "blue" - async */

        /* calling fetch from compute: impossible without
           blocking, which defeats the purpose             */

/* CONSEQUENCES

   1. TWO ECOSYSTEMS. Rust has `Read` and `AsyncRead`,
      `Mutex` and `tokio::Mutex`. Python has requests and
      aiohttp. Libraries get written twice.

   2. THE BRIDGE IS DANGEROUS. block_on() from inside an async
      context deadlocks the executor - the thread that would
      run the future is the one blocked waiting for it.

   3. sleep() IS A BUG. std::thread::sleep in an async task
      blocks the whole executor thread and every other task on
      it. Every runtime ships its own: tokio::time::sleep.

   4. A LOCK HELD ACROSS AN AWAIT is held for an unbounded
      time, and in a work-stealing executor may be released
      on a different thread than took it - which is undefined
      for many mutex implementations.

   THE RULE: never hold a std mutex across an await. Either
   scope it so it is released first, or use the runtime's
   async-aware mutex.                                     */

        /* WRONG */
        let g = mutex.lock().unwrap();
        let v = fetch().await;              /* held across await */

        /* RIGHT */
        let v = fetch().await;
        { let mut g = mutex.lock().unwrap(); *g = v; }

        /* OR, when you genuinely must hold it: */
        let mut g = async_mutex.lock().await;
        let v = fetch().await;              /* fine: this mutex
                                               yields properly  */
```

Choosing a model

Model	Concurrency ceiling	Use when
Thread per task	Thousands	Simple, CPU-bound, or moderate concurrency. Still the right default for most programs
Thread pool	Thousands of tasks, N threads	Bounded CPU work (chapter 17)
Event loop + callbacks	Hundreds of thousands	You have no coroutine support. Expect the pyramid

Model	Concurrency ceiling	Use when
async/await	Hundreds of thousands	High-concurrency I/O in a language that supports it well
Goroutines / green threads	Millions	You can choose the language. No colouring, blocking calls are handled by the runtime
Virtual threads (Java 21)	Millions	Existing blocking code, made scalable without rewriting it — the most interesting recent development here

> **ASYNC IS FOR I/O CONCURRENCY, NOT FOR SPEED**
>
> An async runtime does not make anything compute faster; it removes the cost of *waiting*. If your bottleneck is CPU, async adds state-machine overhead and buys nothing — you want a thread pool. If your bottleneck is ten thousand idle sockets, async is transformative. Diagnosing which you have, before rewriting, saves a great deal of regret.

How it shows up

Symptom	Cause
One slow request stalls every other connection	A blocking call inside a handler
A connection goes silent and never recovers	Edge-triggered epoll without reading to `EAGAIN`
Deadlock calling `block_on` inside a task	Blocking the executor thread that would run the future
Everything stops for exactly one second	`thread::sleep` instead of the runtime's sleep
Panic or UB unlocking a mutex	Held across an `await`, released on a different thread
Use-after-free in a callback	A buffer whose owner returned before the callback ran
Memory grows with connection count faster than expected	Large futures — a big local held across an await is stored in the state machine
Stack traces show only the event loop	Inherent to callbacks; async/await restores useful traces
Async rewrite made it slower	The workload was CPU-bound
Works on Linux, poor on Windows	A readiness design ported to a completion-based OS

Check yourself

- ✓ Compute the address space cost of 10,000 threads and explain why async avoids it.
- ✓ State the one rule an event-loop handler must never break.
- ✓ Distinguish readiness from completion notification and give an OS for each.
- ✓ Explain why `epoll` does not work for regular files.
- ✓ Describe the edge-triggered bug and the loop that prevents it.
- ✓ Give four problems with callback style that async/await fixes.
- ✓ Explain stackless versus stackful coroutines and which languages chose which.
- ✓ Define the function colour problem and give three consequences.
- ✓ Explain why holding a standard mutex across an `await` is wrong.
- ✓ Say when async is the wrong tool, and what to use instead.

Parallel patterns and how to decompose work

Almost every parallel program is one of six shapes. Recognising which one you have tells you where the synchronisation goes, what the scaling limit is, and — usually — that you can avoid sharing altogether.

The six

```
1. EMBARRASSINGLY PARALLEL / MAP
     f(x) for every x, independently.
     NO shared state, NO synchronisation, linear scaling
     until you hit memory bandwidth.
     -> image filters, hashing files, per-request work.

2. REDUCE
     combine N values into one with an associative op.
     Tree-shaped: log N depth, and the only sync is at the
     joins.
     -> sum, max, count, histogram merge.

3. MAP-REDUCE (scatter-gather)
     map, then reduce. The overwhelmingly common shape.
     -> "count word frequencies", "sum per customer".

4. PIPELINE
     stage1 -> stage2 -> stage3, each on its own thread,
     items flowing through.
     Throughput = the SLOWEST stage. Latency = the sum.
     -> decode -> filter -> encode; parse -> plan -> exec.

5. FORK-JOIN / DIVIDE AND CONQUER
     split, recurse in parallel, combine. Naturally
     matches work stealing (chapter 17).
     -> quicksort, mergesort, tree traversal, FFT.

6. PRODUCER-CONSUMER
     decouple rates with a queue. Not really about speed -
     about smoothing bursts and isolating failure.
     -> logging, request queues, sensor to processor.

AND THE ONE THAT IS NOT A PATTERN:
   "put a mutex around the shared thing and run it on
   8 threads". That is not a decomposition, it is a hope.
```

Map, and the mistake in it

```
/* THE OBVIOUS VERSION - and why it is slow. */

std::atomic<long> total{0};

parallel_for(0, n, [&](int i) {
        total += expensive(data[i]);     /* EVERY iteration
                                            writes one atomic  */
});
/* All threads hammer one cache line. Often SLOWER than the
   serial loop (chapter 16).                                   */

/* THE FIX: privatise, then combine. Once. */

struct alignas(64) Partial { long v = 0; };
std::vector<Partial> partial(nthreads);

parallel_for_chunked(0, n, [&](int lo, int hi, int tid) {
        long local = 0;                  /* a REGISTER, not
                                            even memory        */
        for (int i = lo; i < hi; i++)
                local += expensive(data[i]);
        partial[tid].v = local;          /* ONE write, ever    */
});
long total = 0;
for (auto &p : partial) total += p.v;    /* nthreads adds      */

/* The pattern, in one line: ACCUMULATE PRIVATELY, MERGE ONCE.
   It applies to counters, histograms, string builders, output
   buffers, allocators, random number generators - anything
   where the combine is associative.                          */

/* And the libraries already do it: */
        std::reduce(std::execution::par, v.begin(), v.end(), 0L);
        v.par_iter().map(expensive).sum::<i64>();      // Rayon
        #pragma omp parallel for reduction(+:total)    // OpenMP
```

GRANULARITY IS THE SINGLE BIGGEST LEVER

Too fine and scheduling overhead dominates: a task that takes 100 ns cannot pay for a 1 µs enqueue. Too coarse and the tail dominates: 8 chunks on 8 cores means one slow chunk leaves 7 cores idle. The usual sweet spot is **tasks of roughly 10–100 µs**, and **4–8× more chunks than cores** so the scheduler can balance. Both numbers are worth measuring rather than trusting.

Pipelines

```
/* Three stages, bounded queues between them. */

BoundedQueue<Raw>     q1(64);
BoundedQueue<Decoded> q2(64);

std::thread reader([&] {
        Raw r;
        while (read_next(&r)) q1.push(r);
        q1.close();                       /* propagate the end  */
});
std::thread decoder([&] {
        Raw r;
        while (q1.pop(&r)) q2.push(decode(r));
        q2.close();
});
std::thread writer([&] {
        Decoded d;
        while (q2.pop(&d)) write_out(d);
});

/* THE PROPERTIES

   throughput = 1 / (slowest stage time)
        adding threads to the FAST stages changes nothing.
        Find the slow stage and parallelise THAT - a pipeline
        where stage 2 is the bottleneck becomes
        stage1 -> [stage2 x 4] -> stage3.

   latency = sum of all stages + queueing
        a pipeline makes throughput better and latency WORSE.
        If you care about latency, this is the wrong pattern.

   BOUNDED queues are mandatory
        an unbounded queue between a fast producer and a slow
        consumer converts backpressure into memory exhaustion,
        far away from the cause.

   CLOSING must propagate
        each stage closes its output when its input closes and
        it has drained. Miss one and shutdown hangs -
        the commonest pipeline bug.

   ORDER is not preserved once you widen a stage.
        If output order matters you need sequence numbers and
        a reorder buffer at the end, which is real work.     */
```

Fork-join

```
/* Parallel mergesort, with the two details that matter. */

void psort(int *a, int n, int depth)
{
        /* DETAIL 1: a cutoff. Below it, go serial. Without
           this you create a task per ELEMENT and spend all
           your time scheduling. */
        if (n < 4096) { std::sort(a, a + n); return; }

        /* DETAIL 2: bound the depth, or the task count
           explodes. depth 0 means "no more forking". */
        int mid = n / 2;
        if (depth > 0) {
                auto left = std::async(std::launch::async,
                                       [&]{ psort(a, mid, depth-1); });
                psort(a + mid, n - mid, depth - 1);
                left.get();                    /* join */
        } else {
                psort(a, mid, 0);
                psort(a + mid, n - mid, 0);
        }
        std::inplace_merge(a, a + mid, a + n);
}
/* call with depth = log2(hardware_concurrency) + 1 */

/* WITH A WORK-STEALING RUNTIME you can skip the depth bound -
   that is exactly what stealing is for. Rayon: */
        fn psort(a: &mut [i32]) {
            if a.len() < 4096 { a.sort(); return; }
            let mid = a.len() / 2;
            let (l, r) = a.split_at_mut(mid);
            rayon::join(|| psort(l), || psort(r));   /* steals  */
            merge(l, r);
        }
/* rayon::join is nearly free when nobody steals - it pushes
   the right half and runs the left inline. The cutoff is
   still needed; the depth bound is not.                    */
```

Choosing a decomposition

```
THREE WAYS TO CUT A PROBLEM

BY DATA        each thread gets a slice of the data and
               runs the same code.
               + scales with data size
               + easy to balance
               - needs the data to be splittable
               -> the default. Try this first.

BY TASK        each thread runs different code.
               + suits heterogeneous work
               - scales to the NUMBER OF TASKS, not cores
               - balancing is manual and usually wrong
               -> pipelines, and GUI-plus-worker.

BY PIPELINE    each thread is a stage.
               + good for streaming, bounded memory
               - throughput capped by the slowest stage
               - latency increases

THE QUESTIONS THAT PICK ONE

  Is the work uniform?        yes -> data
  Is there a natural stream?  yes -> pipeline
  Is the work heterogeneous?  yes -> task
  Does order matter?          if yes, avoid widening stages
  Is state shared?            if yes, can you privatise it?
                              (usually yes - do that first)
```

LOAD IMBALANCE BEATS ALMOST EVERY OTHER INEFFICIENCY

Eight chunks on eight cores where one chunk takes twice as long means seven cores idle for half the run — a 43% loss, invisible in any lock profile. Fix it with more, smaller chunks plus dynamic scheduling (`schedule(dynamic)` in OpenMP, work stealing elsewhere) rather than by trying to predict chunk costs. Static partitioning is only right when every item genuinely costs the same.

How it shows up

Symptom	Cause
Parallel map slower than serial	Shared accumulator — privatise and merge
Speed-up stops at 2–3× on 16 cores	Load imbalance, or memory bandwidth saturation
Most time in the scheduler	Tasks too small; raise the cutoff
Cores idle at the end of every phase	Too few chunks; use 4–8× the core count
Adding threads to a pipeline changes nothing	You widened a stage that was not the bottleneck
Pipeline consumes all memory	Unbounded queue absorbing a rate mismatch
Pipeline hangs at shutdown	Close not propagated through every stage
Output order scrambled after optimising	A widened stage; needs sequence numbers
Fork-join creates millions of tasks	No cutoff, or no depth bound without work stealing
Scales on one machine, not on the bigger one	NUMA — threads on one node, memory on another

Check yourself

- ✓ Name the six patterns and give a real workload for each.
- ✓ Explain why a shared atomic accumulator in a parallel map is usually a mistake.
- ✓ State the privatise-and-merge rule and list five things it applies to.
- ✓ Give the two granularity failure modes and the rough numbers that avoid both.
- ✓ State how pipeline throughput and latency are each determined.
- ✓ Explain why bounded queues are mandatory in a pipeline.
- ✓ Explain why close must propagate, and what happens if it does not.
- ✓ Give the two details a fork-join implementation needs, and say which one work stealing removes.
- ✓ Compare data, task and pipeline decomposition and say which to try first.
- ✓ Quantify the cost of one chunk in eight taking twice as long.

Finding concurrency bugs on purpose

A test that passes tells you one interleaving worked. The discipline that finds concurrency bugs is not better luck — it is making the rare interleaving common, on hardware that permits it, with tools that see what your eyes cannot.

The method

```
FIRST: WHICH BUG IS IT?  Ten seconds of observation saves
an afternoon.

  hung, 0% CPU           -> deadlock (chapter 14)
  hung, 100% CPU         -> livelock, or a spin/CAS loop
  wrong answer, no crash -> race condition or lost update
  crash in random places -> data race, or use-after-free
  one thread never runs  -> starvation
  slow, not wrong        -> contention or false sharing
  fine alone, wrong in
    parallel             -> shared state you forgot about

THEN, IN THIS ORDER:

1. Make it reproduce more often   (this chapter)
2. Run a sanitiser                (finds data races)
3. Read the code for the invariant, not the syntax
4. Only then, attach a debugger

Reaching for gdb first is the classic mistake: a breakpoint
changes the timing, and the bug you were chasing stops
happening.
```

Making it reproduce

```
# 1. MORE THREADS THAN CORES. Forces constant preemption,
#    so the interleavings you never see become normal.
$ ./app --threads=64          # on 8 cores

# 2. PIN EVERYTHING TO ONE CORE. Removes true parallelism and
#    maximises preemption at arbitrary instructions. Finds a
#    completely different set of bugs from (1) - run both.
$ taskset -c 0 ./app

# 3. RUN ON WEAK HARDWARE. An arm64 box exhibits reorderings
#    x86 physically cannot (chapter 12). If you ship on ARM
#    and test on x86, you are not testing the memory model.

# 4. SLOW DOWN THE SUSPECT WINDOW. The single highest-value
#    technique in this chapter:
      auto it = cache.find(k);
      if (it != cache.end()) return it->second;
+     std::this_thread::sleep_for(std::chrono::milliseconds(50));
      auto v = compute(k);
#    A gap bug that happened once a week now happens every
#    time. Delete the sleep once you have the fix.

# 5. RANDOMISE SCHEDULING. Inject a yield at every
#    synchronisation point, under a flag:
      #ifdef FUZZ_SCHED
      #define SYNC_POINT() do { if (rand() % 4 == 0) \
                                std::this_thread::yield(); } while (0)
      #else
      #define SYNC_POINT() ((void)0)
      #endif

# 6. STRESS IN A LOOP, and record the seed.
$ for i in $(seq 1 100000); do ./test --seed=$i || \
      { echo "FAILED seed=$i"; break; }; done

# 7. RUN UNDER LOAD. Many races need a competitor:
$ stress-ng --cpu 8 --vm 2 & ./app
```

> **PIN TO ONE CORE *AND* OVERSUBSCRIBE — THEY FIND DIFFERENT BUGS**
>
> One core with many threads gives you frequent, arbitrary preemption: it finds check-then-act gaps, lost updates and non-reentrant code. Many cores give you true simultaneity and memory reordering: it finds publication bugs, false sharing and missing barriers. A bug found by one is often invisible to the other, so a stress suite should do both.

The tools

Tool	Finds	Cost
`-fsanitize=thread`	Data races, and lock-order inversions	5–15× slower, 5–10× memory
`-fsanitize=address`	Use-after-free, overflow — often the visible symptom of a race	~2× slower
`valgrind --tool=helgrind`	Races and lock-order violations, no rebuild	20–50× slower
`valgrind --tool=drd`	Similar, better on some patterns, lower memory	Similar

Tool	Finds	Cost
`-Wthread-safety` (clang)	Missing locks — **at compile time**	Free. Needs annotations
Kernel lockdep	Potential deadlocks on runs that did not deadlock	Debug kernel only
`perf c2c`	False sharing, by address and line	Low
`perf lock`	Contention: which lock, how long	Low
rr	**Deterministic record and replay** — reverse-execute a captured failure	~2×, x86 only
TLA+ / Spin	Design-level protocol bugs, exhaustively	Your time; finds bugs no test can

```
# ThreadSanitizer, in practice.
$ g++ -fsanitize=thread -g -O1 -fno-omit-frame-pointer app.cpp
$ TSAN_OPTIONS="halt_on_error=1 history_size=7" ./app

WARNING: ThreadSanitizer: data race (pid=4211)
  Write of size 8 at 0x7b0400000f00 by thread T2:
    #0 Cache::insert(...) cache.cpp:88
    #1 worker(...) main.cpp:41
  Previous read of size 8 at 0x7b0400000f00 by thread T1:
    #0 Cache::lookup(...) cache.cpp:62
  Location is heap block of size 96 allocated by main thread
  Mutexes locked by thread T2: M12 (cache.cpp:20)
  Mutexes locked by thread T1: (none)      <-- THE ANSWER

# That last pair of lines is what you read first: one side
# held the mutex, the other did not.

# rr, when a bug is rare but you caught it once:
$ rr record ./app
$ rr replay
(rr) continue                 # runs to the crash, identically
(rr) reverse-continue         # run BACKWARDS to the previous
(rr) watch -l ptr             # write to that address
# Deterministic replay of a race is close to magic: the
# recording contains the exact interleaving that failed.
```

SANITISERS DO NOT COMPOSE, AND THEY MISS RACE CONDITIONS

TSan and ASan cannot be enabled together — build twice. TSan needs the *whole* program instrumented; an uninstrumented third-party library produces false negatives and sometimes false positives. And most importantly: TSan finds **data races**, not **race conditions** (chapter 5). A clean TSan run on properly-locked code that has a gap between two critical sections proves nothing at all.

Designing so the bug cannot happen

```
/* Cheaper than any tool in the table above. */

/* 1. DON'T SHARE. Per-thread state merged at the end
      (chapter 19). No sharing, no race, nothing to test.

   2. IMMUTABLE AFTER PUBLICATION. Build fully, publish with a
      release store, never mutate. Readers need no lock at all.

   3. OWNERSHIP, NOT LOCKING. One thread owns each object;
      others send messages. Rust enforces this; in C++ you can
      approximate it with move-only types and a queue.

   4. MAKE ILLEGAL STATES UNREPRESENTABLE. Put the data inside
      the mutex so there is no way to touch it unlocked -
      Rust's Mutex<T>, or in C++:                              */

template <class T>
class Guarded {
    std::mutex m_;
    T          v_;
public:
    template <class F>
    auto with(F f) -> decltype(f(v_)) {
        std::lock_guard<std::mutex> g(m_);
        return f(v_);                /* the ONLY way in       */
    }
};
Guarded<std::map<int,int>> cache;
cache.with([&](auto &m) { m[k] = v; });
/* You cannot forget the lock, because there is no accessor
   that does not take it. The whole class of bug is gone.    */

/* 5. ANNOTATE, and let the compiler check.                  */
class Cache {
    std::mutex m_;
    std::map<int,int> data_ GUARDED_BY(m_);
    int hits_ GUARDED_BY(m_);
public:
    int hits() { return hits_; }   /* -Wthread-safety: ERROR,
                                      reading hits_ without m_ */
};

/* 6. ONE INVARIANT PER LOCK, written down (chapter 7).      */
```

What to run in CI

```
# The realistic set, in cost order. Even the first three
# catch most of it.

  every commit:
    -Wthread-safety                       free, compile time
    unit tests under TSan                 slow but bounded
    unit tests with threads=4x cores      finds gap bugs

  nightly:
    the full suite under TSan and under ASan (separate runs)
    the full suite pinned to one core
    the full suite on an ARM64 runner      <-- the highest-value
                                               single addition
    a stress soak, several hours, seeds logged

  before a release:
    Helgrind on the critical paths
    a load test measuring p99.9, not the mean

# And record every intermittent failure with its seed and its
# machine. Three occurrences of "flaky test" with the same
# stack is not flakiness; it is a bug with a low reproduction
# rate, and it will reach production at scale.
```

> **NEVER MARK A CONCURRENCY TEST FLAKY**
>
> A test that fails once in fifty runs is telling you the truth about a bug that will happen once in fifty thousand production requests — which, at a thousand requests a second, is once a minute. Retrying the test until it passes converts a caught bug into a shipped one. Quarantine it if you must ship, but track it as a defect, not as noise.

How it shows up

Symptom	What to do
"Cannot reproduce"	Oversubscribe, pin to one core, add a sleep in the suspect gap
Fails in CI, never locally	Different core count or a slower machine — widening the window
Fails only in release builds	The optimiser exploited undefined behaviour a data race permitted
Disappears under the debugger	Timing changed. Use logging or `rr` instead
Disappears when you add a `printf`	The syscall is a barrier — you moved the bug, not fixed it
TSan clean, bug persists	It is a race condition, not a data race
TSan reports a race in a third-party library	Uninstrumented code, or a genuine bug — `suppressions` file, then investigate
Only fails on arm64	A memory-ordering bug x86 cannot exhibit
Only fails at high load	Contention widens every window; also check queue bounds
Test marked flaky six months ago	Look at it. It is almost certainly your production incident

Check yourself

- ✓ Classify a hung program by CPU usage and say what each case means.

- ✓ Give the four-step order for attacking a concurrency bug, and why the debugger is last.
- ✓ Name seven ways to make a rare interleaving common.
- ✓ Explain why pinning to one core and oversubscribing find different bugs.
- ✓ Read a TSan report and identify the two lines that matter most.
- ✓ State three limits of ThreadSanitizer.
- ✓ Explain what `rr` gives you that a debugger cannot.
- ✓ Write a `Guarded<T>` wrapper and explain which bug class it deletes.
- ✓ Give the CI set you would actually run, and name the single highest-value addition.
- ✓ Explain why a flaky concurrency test is a production incident in waiting.

TRACK

Compilers & toolchain

What happens between your source file and a running instruction — and why the optimiser sometimes deletes your code.

16 chapters

What a compiler is and its phases

Between your source file and a running instruction there are about a dozen distinct transformations. Knowing which one is responsible for a given behaviour is what lets you answer "why did it do that?" instead of guessing.

The pipeline

```
   hello.c
      |
      v
   PREPROCESSOR              #include, #define, #if
      |                      -> a single translation unit
      v  hello.i
   LEXER                     characters -> TOKENS
      |                      (chapter 02)
      v
   PARSER                    tokens -> a PARSE TREE
      |                      (chapter 03)
      v
   SEMANTIC ANALYSIS         types, scopes, name resolution
      |                      -> an annotated AST (chapter 04)
      v
   IR GENERATION             AST -> an INTERMEDIATE
      |                      REPRESENTATION (chapter 05)
      v
   ===== THE MIDDLE END: machine-independent =====
      |
   OPTIMISATION PASSES       constant folding, dead code,
      |                      inlining, loop transforms
      |                      (chapters 06-07)
      v
   ===== THE BACK END: machine-specific =====
      |
   INSTRUCTION SELECTION     IR -> target instructions
      |                      (chapter 08)
      v
   SCHEDULING                reorder for the pipeline
      |
      v
   REGISTER ALLOCATION       virtual -> physical registers
      |                      (chapter 09)
      v
   ASSEMBLY EMISSION
      |
      v  hello.s
   ASSEMBLER                 text -> an OBJECT FILE
      |                      (chapter 11)
      v  hello.o
   LINKER                    objects + libraries -> an
      |                      executable (chapter 12)
      v  a.out
   LOADER                    map it, relocate, resolve
                             (chapter 13)

   THE SPLIT THAT MATTERS: front end (language-specific),
   middle end (neither), back end (target-specific). It is why
   GCC supports many languages on many targets with N+M
   components rather than N*M.
```

Watching each phase

```
# --- Preprocessor only ---
    gcc -E hello.c -o hello.i
    ; expands includes and macros. When a macro misbehaves,
    ; look here FIRST - the answer is always visible.
    gcc -E -P hello.c | less     ; -P drops line markers

# --- Tokens ---
    gcc -E -dD hello.c           ; with all macro definitions
    clang -cc1 -dump-tokens hello.c

# --- The parse tree / AST ---
    clang -Xclang -ast-dump -fsyntax-only hello.c
    gcc -fdump-tree-original-raw hello.c

# --- The IR ---
    clang -S -emit-llvm hello.c -o hello.ll   ; LLVM IR
    gcc -fdump-tree-all hello.c               ; GIMPLE, every
                                              ; pass
    gcc -fdump-rtl-all hello.c                ; RTL

# --- Assembly ---
    gcc -S hello.c -o hello.s
    gcc -S -O2 -fverbose-asm hello.c          ; with comments

# --- Object file ---
    gcc -c hello.c -o hello.o
    objdump -d hello.o
    readelf -a hello.o

# --- The whole thing, with every step shown ---
    gcc -v hello.c
    ; prints the actual cc1, as and ld command lines - which
    ; is how you find out what flags your build system is
    ; really passing.

# THE SINGLE MOST USEFUL TOOL for this whole track:
    https://godbolt.org        (Compiler Explorer)
# Source on the left, assembly on the right, colour-matched,
# for any compiler and any target including aarch64. Open it
# now and keep it open while reading these chapters.
```

> **COMPILER EXPLORER CHANGES HOW YOU LEARN THIS**
> Every claim in this track is checkable in about ten seconds: paste the code, pick `armv8-a clang` or `aarch64 gcc`, set `-O2`, and read the output. Do that rather than believing the text — including when the text is right, because the habit is what makes the knowledge stick.

Front end, middle end, back end

	Front end	Middle end	Back end
Knows about	The *language*	Neither	The *target*
Input	Source text	IR	IR
Output	IR	Better IR	Machine code
Errors from here	Syntax, type errors	Rarely any	"Cannot allocate register"

	Front end	Middle end	Back end
GCC	c, c++, fortran, go, d…	GIMPLE passes	x86, aarch64, riscv…
LLVM	clang, rustc, swift…	LLVM IR passes	LLVM backends

```
/* WHY THE SPLIT PAYS: LLVM has one optimiser. Rust, Swift,
   Julia, Zig, clang and a dozen others all feed it LLVM IR
   and inherit every optimisation anyone writes.

   AND WHY IT LEAKS: the middle end optimises the IR under the
   assumption that the source program has NO UNDEFINED
   BEHAVIOUR. That assumption is what makes aggressive
   optimisation legal, and it is why UB has effects that look
   insane at the source level (chapter 15).                */
```

GCC and LLVM

```
/* You will use both. The differences that matter in
   practice:                                              */

   GCC
     · the default for the Linux kernel and most embedded
       Linux toolchains
     · GIMPLE (tree-based) then RTL (low-level) IR
     · a monolithic program
     · often marginally better code on some targets,
       especially with -O2 on ARM
     · inline assembly and __attribute__ syntax are the de
       facto standard that Clang copies

   LLVM/CLANG
     · a LIBRARY, so tools are built on it: clangd, clang-tidy,
       clang-format, sanitizers, static analysers
     · LLVM IR: typed, SSA, human-readable, stable enough to
       write tools against
     · far better diagnostics - error messages that point at
       the right thing
     · the sanitizers (ASan, UBSan, TSan, MSan) originate here
     · what Rust, Swift and Zig use

/* IN EMBEDDED WORK you will typically compile the kernel and
   the BSP with GCC (because that is what the vendor
   supports), and use Clang's tooling - clangd for navigation,
   clang-tidy and the sanitizers for analysis - alongside it.
   The Linux kernel now builds with either.

   AND FOR LEARNING: read Clang's LLVM IR output. It is far
   more legible than GIMPLE and the optimisation passes are
   individually observable.                               */
```

Optimisation levels

Flag	Does	Use for
`-O0`	Nothing. Every variable in memory, code matches source line for line	Debugging. **The kernel will not build with it**
`-O1`	Cheap optimisations, fast compile	Rarely chosen deliberately

Flag	Does	Use for
`-O2`	The standard. Inlining, vectorisation, most passes	**Almost everything**
`-O3`	`-O2` plus aggressive inlining and vectorisation	Measured hot paths; can be *slower* from cache pressure
`-Os`	Optimise for size	Flash-constrained microcontrollers
`-Oz`	Size above all (Clang)	Extremely tight parts
`-Og`	Optimise, but keep debugging usable	Debug builds of code that must run at speed
`-Ofast`	`-O3` plus **unsafe maths**	Almost never — it breaks IEEE 754 (CA ch. 03)

```
/* WHY -O0 IS NOT "SAFE":

   · the kernel REQUIRES optimisation. Inline assembly
     constraints, __builtin_constant_p, and many macros only
     work when constants fold. Linux does not build at -O0.
   · unoptimised code is 3–10x slower and much larger, which
     on an embedded target may not fit or may miss deadlines.
   · a BUG THAT ONLY APPEARS AT -O2 is usually undefined
     behaviour in your code (chapter 15), not a compiler bug.
     Turning optimisation off hides it; it does not fix it.

/* THE RIGHT DEBUG BUILD is -Og -g, or -O2 -g with the
   understanding that variables may be optimised out. Modern
   DWARF and gdb handle -O2 far better than they used to
   (chapter 14).                                          */
```

Flags worth knowing

```
# --- Warnings: turn them on, all of them ---
    -Wall -Wextra              ; the baseline. Not optional.
    -Werror                    ; in CI
    -Wshadow                   ; a shadowed variable is
                               ; almost always a bug
    -Wconversion               ; implicit narrowing
    -Wsign-conversion
    -Wcast-align               ; important on ARM
    -Wpointer-arith
    -Wvla                      ; variable-length arrays
    -Wframe-larger-than=1024   ; stack usage (Linux ch. 02)
    -Wstack-usage=1024

# --- Standards ---
    -std=c11 / -std=gnu11 / -std=c17
    -pedantic                  ; reject GNU extensions

# --- Code generation ---
    -fno-strict-aliasing       ; the kernel uses this; see
                               ; chapter 15
    -fno-common
    -fstack-protector-strong
    -fno-omit-frame-pointer    ; makes profiling and
                               ; backtraces work
    -ffunction-sections -fdata-sections
        + -Wl,--gc-sections    ; drop unused code at link time
                               ; (chapter 12)
    -flto                      ; link-time optimisation

# --- Target selection, ARM ---
    -march=armv8-a+crc+crypto
    -mtune=cortex-a53
    -mcpu=cortex-a53+crc       ; sets both
    -mgeneral-regs-only        ; no FP/SIMD - REQUIRED for
                               ; kernel code (Linux ch. 02)

# --- Freestanding, for firmware ---
    -ffreestanding -nostdlib -nostdinc
    -fno-builtin

# --- Diagnostics about what the optimiser did ---
    -fopt-info-vec             ; which loops vectorised
    -fopt-info-vec-missed      ; and which did NOT, with the
                               ; reason (chapter 07)
    -fopt-info-inline
    clang: -Rpass=.* -Rpass-missed=.* -Rpass-analysis=.*
```

Translation units and the build

```
   foo.c  bar.c  baz.c
     |      |      |
     |  each compiled SEPARATELY, knowing nothing about the
     |  others - this is the C model
     v      v      v
   foo.o  bar.o  baz.o
     \      |      /
      \     |     /
       LINKER  +  libc.a, libm.so
          |
          v
       a.out

  CONSEQUENCES OF SEPARATE COMPILATION:

    · the compiler cannot inline across files, or prove
      anything about a function it has not seen
      -> which is why `static` matters: a static function has
         no external callers, so the compiler may inline it,
         specialise it, or delete it entirely
      -> and why LTO exists (chapter 12): defer optimisation
         to link time, when everything IS visible

    · a declaration mismatch between files is NOT caught by
      the compiler - only by the linker, and only sometimes
      -> which is what header files are for, and why a
         function should be declared in exactly one header
         that both the definition and the callers include

    · build times scale with the number of translation units
      times the size of the headers each pulls in
      -> which is why include hygiene matters on large
         projects, and why C++ modules were invented
```

How it shows up

Symptom	Which phase
"expected ';' before..."	Parser — but the cause is often a bad macro; check `-E`
A macro expands wrongly	Preprocessor — `gcc -E` shows the truth
"incompatible pointer types"	Semantic analysis
Code works at `-O0`, breaks at `-O2`	Almost always UB in your code (ch. 15)
A variable "optimised out" in gdb	Register allocation — build with `-Og`
"undefined reference to ..."	Linker — the symbol was never defined (ch. 12)
"multiple definition of ..."	Linker — defined in two objects
Runs on the host, crashes on target	Wrong `-march`, or an alignment assumption
Binary far larger than expected	No `--gc-sections`, or debug info not stripped
A loop did not vectorise	`-fopt-info-vec-missed` names the reason
Illegal instruction on target	`-march` allowed something the CPU lacks

Check yourself

- ✓ Name every phase from source to running instruction, in order.
- ✓ Explain the front/middle/back-end split and what it buys.
- ✓ Dump the output of five different phases for one source file.
- ✓ Use Compiler Explorer to check a claim about generated code.
- ✓ Say when to prefer GCC and when Clang, in embedded work.
- ✓ Explain why the kernel cannot be built at `-O0`.
- ✓ Explain why a bug appearing only at `-O2` is usually yours.
- ✓ List ten warning flags and what each catches.
- ✓ Explain what separate compilation prevents the compiler from doing, and what `static` and LTO recover.
- ✓ Attribute ten error messages to the phase that produced them.

Lexical analysis

Turning a stream of characters into a stream of tokens. The simplest phase of a compiler, and the one whose rules produce the most surprising error messages when you break them.

What a token is

```
SOURCE:      int count = x + 42;   /* a comment */

TOKENS:
   KEYWORD      "int"
   IDENTIFIER   "count"
   PUNCTUATOR   "="
   IDENTIFIER   "x"
   PUNCTUATOR   "+"
   NUMBER       "42"        value 42, type int
   PUNCTUATOR   ";"
   (the comment is discarded, replaced by one space)

EACH TOKEN CARRIES:
   a KIND
   the TEXT (its "spelling")
   a SOURCE LOCATION - file, line, column
   for literals, a computed VALUE and TYPE

THE SOURCE LOCATION is why error messages can point at the
right character, and why it must be preserved all the way
through the front end. A compiler that loses it produces
the "error somewhere in this file" diagnostics that older
toolchains were notorious for.

C's TOKEN CATEGORIES (C11 6.4):
   keywords          int, if, while, _Atomic, ...
   identifiers       count, printf, _my_var
   constants         42, 0x2A, 3.14, 'c', 1e-9
   string literals   "hello"
   punctuators       + - * / = == { } [ ] ; ...
   header names      <stdio.h>  (only after #include)
```

The maximal munch rule

```
/* THE RULE: at each point, take the LONGEST sequence of
   characters that forms a valid token. Greedily, without
   looking at whether the result makes sense.              */

        x+++y

/* Munched maximally: */
        x  ++  +  y        -> (x++) + y
/* NOT: */
        x  +  ++y

/* AND THE FAMOUS ONE: */
        a = b //*divide*/c;
             ^^
/* `//` starts a line comment in C99 and later, so this is: */
        a = b;
/* In C89, where // was not a comment, it was: */
        a = b / c;
/* The same source, two meanings, decided by the standard
   version. Real code was broken by this transition.      */

/* THE DIVISION TRAP: */
        int x = a /*p;      /* `/*` opens a comment! */
/* intended: a divided by *p. Write it as: */
        int x = a / *p;
        int x = a / (*p);

/* AND IN C++ BEFORE C++11: */
        vector<vector<int>> v;
                        ^^
/* `>>` munched as the right-shift operator, not two closing
   angle brackets. You had to write `> >`. C++11 special-cased
   it - which tells you the rule is a genuine design tension,
   not a curiosity.                                        */

/* THE PRACTICAL LESSON: whitespace is not decoration. When an
   expression parses strangely, add spaces between operators
   and look again.                                          */
```

The translation phases

```
C defines EIGHT translation phases, in order (C11 5.1.1.2).
Knowing them explains several otherwise baffling behaviours:

  1. Map physical characters to the source character set.
     Replace trigraphs (??= for #). Removed in C23.

  2. DELETE backslash-newline pairs - LINE SPLICING.
     Happens BEFORE tokenising, which is why:

         int x = 1; \
         int y = 2;

     is one line, and why a stray space AFTER the backslash
     breaks a macro continuation with a confusing error.

  3. Decompose into PREPROCESSING TOKENS and whitespace.
     Each comment becomes one space.

  4. Execute PREPROCESSING DIRECTIVES and expand macros.
     #include files are processed recursively through phases
     1-4.

  5. Convert escape sequences in character and string
     literals.

  6. CONCATENATE ADJACENT STRING LITERALS:
         "hello" " " "world"   ->   "hello world"
     which is why this works:
         #define PREFIX "app: "
         puts(PREFIX "started");

  7. Convert preprocessing tokens to TOKENS. Syntax and
     semantic analysis. -> a translation unit

  8. Resolve external references. LINK. -> a program

  PHASES 1-6 ARE THE PREPROCESSOR. Phase 7 is the compiler
  proper. Phase 8 is the linker (chapter 12).

  THE CONSEQUENCE PEOPLE MEET: the preprocessor operates on
  PREPROCESSING TOKENS, which are not quite tokens. A number
  like 0x1p+3 is one pp-number even though it would not be a
  valid integer, which is why some macro tricks work and
  others produce "invalid suffix" errors.
```

The hard cases

```
/* --- 1. IS IT A TYPE OR A VARIABLE? --------------------- */
       foo * bar;
/* This is EITHER:
     a declaration: `bar` is a pointer to `foo`
     an expression: `foo` multiplied by `bar`
   depending on whether `foo` is a typedef name.

   THE LEXER CANNOT KNOW. It must ask the symbol table, which
   is built by the parser. This is "the lexer hack": C
   compilers feed type information BACKWARDS from the parser
   to the lexer, so it can emit TYPEDEF_NAME or IDENTIFIER.

   This is why C's grammar is not context-free, and why
   `typedef` makes C genuinely harder to parse than languages
   that spell declarations differently.                    */

/* --- 2. NUMERIC LITERALS -------------------------------- */
       42          int
       42u         unsigned
       42l  42L    long
       42ull       unsigned long long
       0x2A        hex
       052         OCTAL - leading zero! A classic bug:
                   permissions written as 0644 are octal
                   (correct), but a zero-padded decimal like
                   0100 is 64, not 100.
       0b101010    binary - a GNU extension, standard in C23
       1'000'000   digit separators (C++14, C23)
       3.14        double
       3.14f       float
       1e-9        double
       0x1.8p3     hex float: 1.5 * 2^3 = 12.0

/* --- 3. CHARACTER AND STRING ESCAPES -------------------- */
       '\n' '\t' '\\' '\'' '\0'
       '\x41'      hex - CONSUMES AS MANY HEX DIGITS AS IT
                   CAN, so "\x41BC" is ONE character with
                   value 0x41BC (implementation-defined
                   overflow), not 'A' followed by "BC".
                   Write "\x41" "BC" instead.
       '\101'      octal - exactly up to three digits
       u8"..."     UTF-8
       L"..."      wide
       R"(raw)"    raw string (C++)

/* --- 4. THE `>>` AND `<:` DIGRAPHS ---------------------- */
       <: :> <% %> %: %:%:
/* alternative spellings of [ ] { } # ## for keyboards
   lacking those characters. Still in the standard. They very
   occasionally surprise someone using `<:` in a template
   argument.                                               */
```

> **THE OCTAL TRAP IS A REAL EMBEDDED BUG**
>
> `#define TIMEOUT_MS 0100` is 64, not 100. It appears in configuration constants, in zero-padded register values copied from a datasheet table, and in permission masks where octal *was* intended but the reader does not realise. Never zero-pad a decimal constant, and when you do mean octal, use `0o` (C23) or a comment.

Writing one

```
/* Lexers are simple enough to write by hand, and most
   production compilers do - it is faster and gives better
   error messages than a generated one.                    */

typedef enum {
        TOK_EOF, TOK_IDENT, TOK_NUMBER, TOK_STRING,
        TOK_PLUS, TOK_MINUS, TOK_STAR, TOK_SLASH,
        TOK_LPAREN, TOK_RPAREN, TOK_SEMI, TOK_EQ, TOK_EQEQ,
        TOK_KEYWORD_IF, TOK_KEYWORD_WHILE, /* ... */
} TokenKind;

typedef struct {
        TokenKind kind;
        const char *start;      /* into the source buffer */
        size_t      length;
        int         line, column;
        union { long ival; double dval; };
} Token;

static Token next_token(Lexer *lx)
{
        skip_whitespace_and_comments(lx);

        if (at_end(lx))
                return make(lx, TOK_EOF);

        char c = advance(lx);

        /* identifiers and keywords */
        if (isalpha(c) || c == '_') {
                while (isalnum(peek(lx)) || peek(lx) == '_')
                        advance(lx);
                /* look it up in a keyword table - a perfect
                   hash, or just a sorted bsearch           */
                return make_ident_or_keyword(lx);
        }

        /* numbers */
        if (isdigit(c))
                return scan_number(lx);

        /* strings */
        if (c == '"')
                return scan_string(lx);

        /* MAXIMAL MUNCH on operators: check the longest
           first                                            */
        switch (c) {
        case '=':
                if (match(lx, '=')) return make(lx, TOK_EQEQ);
                return make(lx, TOK_EQ);
        case '+': return make(lx, TOK_PLUS);
        case '(': return make(lx, TOK_LPAREN);
        ...
        }

        return error_token(lx, "unexpected character");
}

/* NOTE `start` and `length` POINT INTO THE SOURCE rather than
```

```
   copying the text. That is how real compilers avoid
   allocating a string per token, and it is why the source
   buffer must outlive the tokens.

   THE OTHER PRODUCTION DETAIL: a keyword is just an
   identifier whose spelling is in a table. Do not write a
   separate scanner for each.                              */
```

Generated lexers

```
/* flex / lex - a regular expression per token kind: */

%%
[ \t\n]+                { /* skip */ }
"/*"([^*]|\*[^/])*"*/"  { /* comment */ }
"if"                    { return TOK_IF; }
"while"                 { return TOK_WHILE; }
[a-zA-Z_][a-zA-Z0-9_]*  { return TOK_IDENT; }
[0-9]+                  { yylval.ival = atol(yytext);
                          return TOK_NUMBER; }
"=="                    { return TOK_EQEQ; }
"="                     { return TOK_EQ; }
"+"                     { return TOK_PLUS; }
.                       { return TOK_ERROR; }
%%

/* flex compiles these into a DFA - a state machine that
   consumes one character per step, in constant time. That is
   why lexing is linear and fast.

   THE RULE ORDER MATTERS: flex prefers the longest match, and
   among equal-length matches, the rule listed FIRST. So
   keywords must come before the identifier rule, or every
   keyword lexes as an identifier.

/* WHEN TO USE A GENERATOR: prototypes, small languages,
   configuration file parsers. For a production compiler,
   hand-written wins on error messages, on incremental
   re-lexing for an IDE, and on the context sensitivity C
   needs anyway.                                           */
```

How it shows up

Symptom	Cause
"unterminated comment"	A `/*` inside what you meant as division — `a / *p`
Half the file vanishes	An unclosed comment swallowing to the next `*/`
Macro continuation breaks	Whitespace after the backslash — line splicing is phase 2
Constant is smaller than written	Leading zero — it is octal
"invalid suffix on integer constant"	Token pasting produced something that is not a valid number
`x+++y` parses unexpectedly	Maximal munch — add spaces
String escape consumes too much	`\x` is greedy — split the literal
"expected ';' before '*'"	A typedef name the compiler does not know about — a missing include
Two adjacent strings merge unexpectedly	Phase 6 concatenation, often from a macro

Symptom	Cause
Error points at the wrong line	Line splicing or a macro moved things — check `gcc -E`

Check yourself

- ✓ Tokenise a line of C by hand, including the source locations.
- ✓ State the maximal munch rule and give three cases where it surprises.
- ✓ List C's eight translation phases and say which are the preprocessor's.
- ✓ Explain why line splicing happens before tokenising, and the bug that follows.
- ✓ Explain string literal concatenation and one place it is useful.
- ✓ Explain the lexer hack and why C's grammar is not context-free.
- ✓ Decode six numeric literal forms, including the octal trap.
- ✓ Explain why `"\x41BC"` is not what you expect.
- ✓ Write a hand-rolled lexer for a small expression language.
- ✓ Explain why flex rule order matters for keywords.

Grammars and parsing

Turning a flat stream of tokens into a tree that reflects the structure of the program. This is where operator precedence comes from, why C's declaration syntax is famously awkward, and how a compiler decides what your code means.

Grammars

```
A GRAMMAR is a set of rules that generate a language. Written
in BNF or EBNF:

   expr    -> expr '+' term
            | expr '-' term
            | term

   term    -> term '*' factor
            | term '/' factor
            | factor

   factor  -> NUMBER
            | IDENT
            | '(' expr ')'

  TERMINALS are tokens: '+', NUMBER, IDENT.
  NON-TERMINALS are the named rules: expr, term, factor.

  THE STRUCTURE ENCODES PRECEDENCE. Because `term` appears
  inside `expr`, multiplication binds tighter than addition:
  the only way to derive `1 + 2 * 3` is with `2 * 3` as one
  term. Precedence is not a table bolted on afterwards - it is
  the SHAPE of the grammar.

  AND ASSOCIATIVITY comes from the recursion side:

     expr -> expr '+' term      LEFT recursive  -> left assoc
                                (a+b)+c
     expr -> term '+' expr      RIGHT recursive -> right assoc
                                a+(b+c)

  So `a - b - c` means `(a-b)-c` because the grammar is
  left-recursive, and `a = b = c` means `a = (b = c)` because
  assignment's rule is right-recursive.
```

Ambiguity

```
/* A grammar is AMBIGUOUS if some input has more than one
   parse tree. The classic:                                  */

        stmt -> 'if' '(' expr ')' stmt
              | 'if' '(' expr ')' stmt 'else' stmt
              | ...

        if (a) if (b) x(); else y();

/* Two readings:
     if (a) { if (b) x(); else y(); }     <-- the rule
     if (a) { if (b) x(); } else y();

   THE DANGLING ELSE. Every C-family language resolves it the
   same way: an `else` binds to the NEAREST unmatched `if`.
   That is a disambiguation rule imposed on top of an
   ambiguous grammar, not something the grammar itself
   expresses.

   AND IT IS WHY braces are worth writing even for
   single statements - not because the rule is unclear, but
   because the reader has to apply it.                       */

/* THE OTHER AMBIGUITY C HAS: the typedef problem
   (chapter 02).                                              */
        (foo)(bar);
/* a cast of `bar` to type `foo`, or a call of function `foo`
   with argument `bar`? Only the symbol table knows.         */

/* AND C++ ADDS the "most vexing parse": */
        Widget w(Gadget());
/* Declares a FUNCTION `w` taking a pointer-to-function
   returning Gadget, and returning Widget. Almost never what
   was meant. C++11's brace initialisation exists partly to
   escape it:                                                */
        Widget w{Gadget{}};
```

Recursive descent

```
/* One function per grammar rule. Readable, debuggable, gives
   excellent error messages - which is why GCC, Clang and most
   production compilers use it despite the theory favouring
   generated parsers.                                        */

static Node *parse_expr(Parser *p);

static Node *parse_primary(Parser *p)
{
        if (match(p, TOK_NUMBER))
                return make_number(previous(p));

        if (match(p, TOK_IDENT))
                return make_ident(previous(p));

        if (match(p, TOK_LPAREN)) {
                Node *e = parse_expr(p);
                expect(p, TOK_RPAREN, "expected ')'");
                return e;
        }

        error(p, "expected an expression");
        return NULL;
}

static Node *parse_unary(Parser *p)
{
        if (match(p, TOK_MINUS) || match(p, TOK_BANG)) {
                Token op = previous(p);
                Node *operand = parse_unary(p);   /* RIGHT
                                                     assoc  */
                return make_unary(op, operand);
        }
        return parse_primary(p);
}

static Node *parse_term(Parser *p)
{
        Node *left = parse_unary(p);

        /* LEFT associativity via a LOOP, not recursion -
           left-recursive grammar rules cannot be written
           directly in recursive descent because they would
           recurse forever.                                */
        while (match(p, TOK_STAR) || match(p, TOK_SLASH)) {
                Token op = previous(p);
                Node *right = parse_unary(p);
                left = make_binary(left, op, right);
        }
        return left;
}

static Node *parse_expr(Parser *p)
{
        Node *left = parse_term(p);

        while (match(p, TOK_PLUS) || match(p, TOK_MINUS)) {
                Token op = previous(p);
                Node *right = parse_term(p);
                left = make_binary(left, op, right);
```

```
        }
        return left;
}

/* THE PATTERN: each precedence level is a function that calls
   the NEXT TIGHTER level, then loops on its own operators.
   Add a level by adding a function in the chain.

   THE LEFT-RECURSION RULE is the one thing to remember:
   rewrite `expr -> expr op term` as a loop.             */
```

Precedence climbing

```
/* C has fifteen precedence levels. Fifteen near-identical
   functions is tedious and slow. Pratt parsing (precedence
   climbing) collapses them into one loop driven by a table. */

static int precedence_of(TokenKind k)
{
        switch (k) {
        case TOK_OROR:   return 1;
        case TOK_ANDAND: return 2;
        case TOK_OR:     return 3;
        case TOK_XOR:    return 4;
        case TOK_AND:    return 5;
        case TOK_EQEQ: case TOK_NEQ:  return 6;
        case TOK_LT: case TOK_GT:
        case TOK_LE: case TOK_GE:     return 7;
        case TOK_SHL: case TOK_SHR:   return 8;
        case TOK_PLUS: case TOK_MINUS: return 9;
        case TOK_STAR: case TOK_SLASH:
        case TOK_PERCENT:              return 10;
        default: return 0;      /* not a binary operator */
        }
}

static Node *parse_binary(Parser *p, int min_prec)
{
        Node *left = parse_unary(p);

        for (;;) {
                int prec = precedence_of(peek(p).kind);

                if (prec == 0 || prec < min_prec)
                        break;

                Token op = advance(p);

                /* LEFT associative: the right operand may only
                   contain operators of HIGHER precedence, so
                   pass prec + 1.
                   For a RIGHT associative operator, pass prec
                   instead.                                    */
                Node *right = parse_binary(p, prec + 1);

                left = make_binary(left, op, right);
        }
        return left;
}

        /* the whole expression grammar: */
        Node *e = parse_binary(p, 1);

/* ONE FUNCTION, one table. Adding an operator is a table
   entry. This is what most modern hand-written parsers use,
   and it is worth understanding because the `prec + 1` versus
   `prec` distinction is exactly where associativity lives. */
```

PRECEDENCE AND ASSOCIATIVITY ARE GRAMMAR, NOT CONVENTION

When you wonder why `a & b == c` means `a & (b == c)` — a genuine C wart that has caused real bugs — the answer is that `==` sits at a higher precedence level than `&` in the grammar C inherited from B. It is not a rule someone can change; it is the shape of the language. Parenthesise bitwise operators.

LR parsing and generators

```
/* The other family: BOTTOM-UP. Instead of predicting which
   rule applies, shift tokens onto a stack and REDUCE when the
   top of the stack matches a rule's right-hand side.       */

   Parsing `1 + 2 * 3`:

     stack                  input          action
     (empty)                1 + 2 * 3      shift
     1                      + 2 * 3        reduce factor
     factor                 + 2 * 3        reduce term
     term                   + 2 * 3        reduce expr
     expr                   + 2 * 3        shift
     expr +                 2 * 3          shift
     expr + 2               * 3            reduce factor,term
     expr + term            * 3            SHIFT (not reduce!)
     expr + term *          3              shift
     expr + term * 3        (end)          reduce factor
     expr + term * factor   (end)          reduce term
     expr + term            (end)          reduce expr
     expr                   (end)          ACCEPT

   THE INTERESTING STEP is `expr + term` with `*` next: the
   parser could reduce `expr + term` to `expr`, or shift the
   `*`. It shifts, because `*` binds tighter. That decision -
   SHIFT/REDUCE - is what a precedence declaration resolves in
   a generator.

/* yacc / bison: */
%left  '+' '-'
%left  '*' '/'          /* later = tighter */
%right UMINUS

%%
expr : expr '+' expr    { $$ = mk(ADD, $1, $3); }
     | expr '-' expr    { $$ = mk(SUB, $1, $3); }
     | expr '*' expr    { $$ = mk(MUL, $1, $3); }
     | expr '/' expr    { $$ = mk(DIV, $1, $3); }
     | '-' expr %prec UMINUS { $$ = mk(NEG, $2); }
     | '(' expr ')'     { $$ = $2; }
     | NUMBER           { $$ = mk_num($1); }
     ;

/* THE GRAMMAR IS AMBIGUOUS as written - and the %left
   declarations resolve every conflict. That is the appeal:
   the grammar reads like the language reference.

   THE CONFLICTS YOU WILL SEE:
     shift/reduce   usually benign; bison shifts by default,
                    which is correct for dangling-else
     reduce/reduce  ALWAYS a real ambiguity. Fix the grammar.

   Run `bison -Wcounterexamples` - it prints an actual input
   that triggers each conflict, which turns a cryptic warning
   into a concrete problem.                                 */

/* WHY PRODUCTION COMPILERS STILL USE RECURSIVE DESCENT:
     · error messages: a hand-written parser knows what it
       expected and can suggest a fix
     · error RECOVERY: continuing sensibly after a mistake
     · context sensitivity: C's typedef problem
```

```
     · incremental re-parsing, for IDEs
   GCC moved from bison to recursive descent for C++ in 2004
   for exactly these reasons.                                */
```

C declaration syntax

```
/* C's declarations follow the "declaration mirrors use"
   principle, which makes complex ones hard to read.       */

        int    *p;              /* *p is an int             */
        int    a[10];           /* a[i] is an int           */
        int    f(void);         /* f() is an int            */
        int   *f(void);         /* *f() is an int -> f
                                   returns int*             */
        int  (*f)(void);        /* (*f)() is an int -> f is a
                                   pointer to function      */
        int   *a[10];           /* array of 10 int*         */
        int  (*a)[10];          /* pointer to array of 10    */
        int (*f[10])(void);     /* array of 10 function
                                   pointers                 */
        int (*(*f)(void))[10];  /* pointer to function
                                   returning pointer to
                                   array of 10 int          */

/* THE READING RULE - "spiral" or "right-left":
     1. start at the identifier
     2. go RIGHT while you can: [] means array, () means
        function
     3. then go LEFT: * means pointer
     4. parentheses override, innermost first             */

        int (*f[10])(void);
             ^ f
              [10]     -> array of 10
            *          -> pointers to
                (void) -> functions taking void
        int            -> returning int

/* THE TOOL: */
        cdecl> explain int (*(*f)(void))[10]
        declare f as pointer to function (void) returning
        pointer to array 10 of int

/* THE PRACTICAL ADVICE: use typedefs. */
        typedef int (*handler_fn)(void);
        handler_fn table[10];        /* obviously an array of
                                        10 handlers          */

/* WHY IT IS LIKE THIS: Dennis Ritchie chose it so that a
   declaration looks like the expression that uses the
   variable. It is internally consistent, and it composes
   badly - which is why every later language (Go, Rust, even
   C++'s trailing return types) spells declarations
   differently.                                             */
```

Error recovery

```
/* A compiler that stops at the first error is unusable. Good
   parsers RECOVER and keep going, without producing a
   cascade of bogus follow-on errors.                      */

/* PANIC MODE - the standard technique: */
static void synchronise(Parser *p)
{
        advance(p);

        while (!at_end(p)) {
                /* a semicolon ends a statement - resume there */
                if (previous(p).kind == TOK_SEMI)
                        return;

                /* or at a token that clearly STARTS something */
                switch (peek(p).kind) {
                case TOK_IF: case TOK_WHILE: case TOK_FOR:
                case TOK_RETURN: case TOK_STRUCT:
                case TOK_RBRACE:
                        return;
                default:
                        advance(p);
                }
        }
}

/* AND SUPPRESS CASCADES: after reporting an error, do not
   report another until the parser has successfully consumed
   something. Otherwise one missing brace produces two hundred
   messages and the real error scrolls off the screen.

/* GOOD ERROR MESSAGES, which Clang made the standard:
     · point at the exact column, with a caret
     · say what was EXPECTED, not just what was found
     · suggest a FIX when the situation is unambiguous
     · show the relevant note - "to match this '(' here"

     error: expected ';' after expression
        x = y + 1
                  ^
                  ;

   Compare "parse error before '}'" - which was normal in
   1998 and tells the reader almost nothing.             */
```

How it shows up

Symptom	Cause
Hundreds of errors from one typo	Poor error recovery, or a missing brace/parenthesis early on — fix the first one
"expected ';' before '*'"	A typedef name the parser does not know — a missing include
Expression evaluates in the wrong order	Precedence — particularly bitwise versus comparison
`else` attached to the wrong `if`	Dangling else — use braces
A declaration is read as a function	C++'s most vexing parse — use brace initialisation
Cannot read a declaration	Apply the right-left rule, or introduce a typedef

Symptom	Cause
bison reports shift/reduce conflicts	Usually benign; check with `-Wcounterexamples`
bison reports reduce/reduce conflicts	A real ambiguity — fix the grammar
Error points past the real problem	The parser only noticed when the construct failed to close

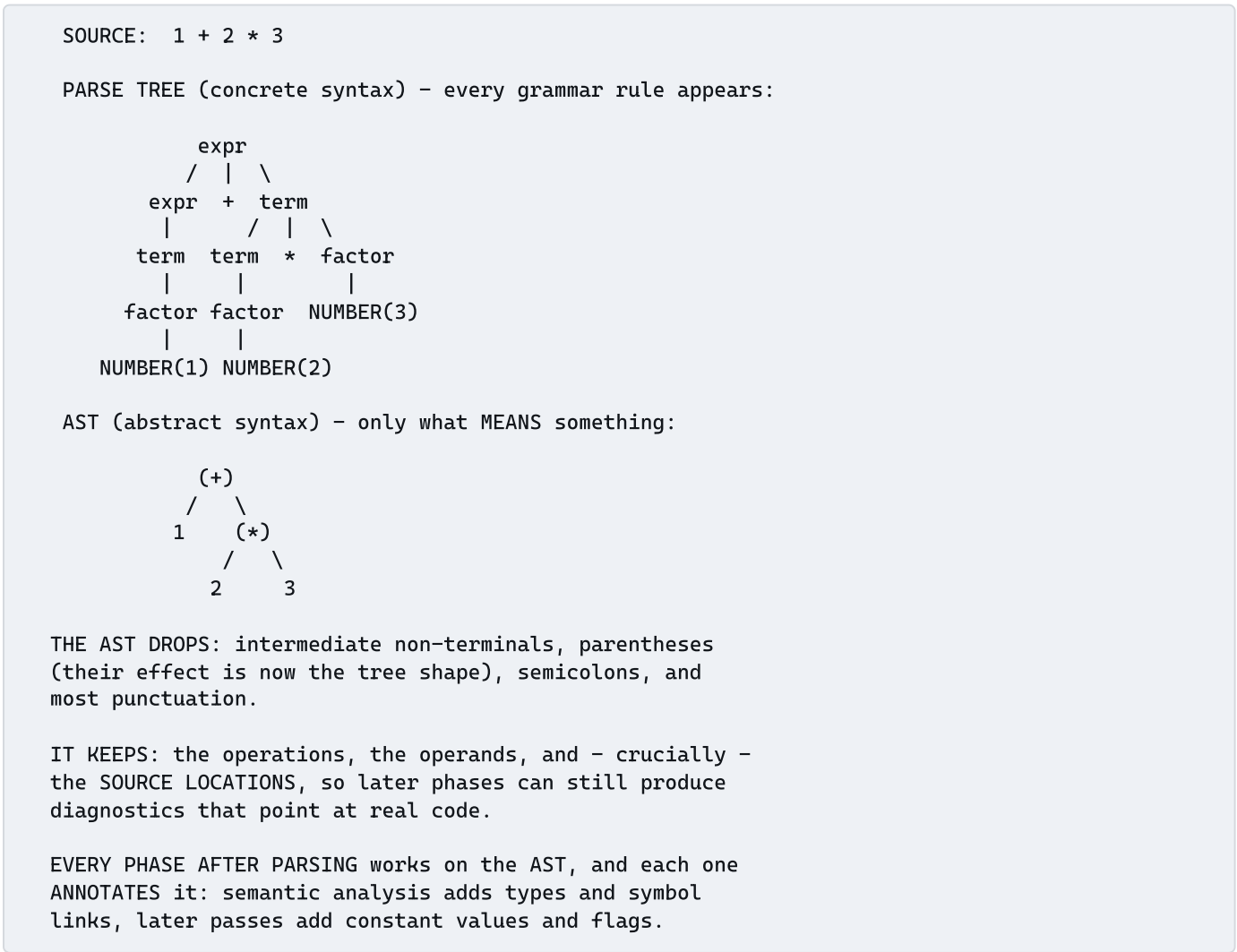
Check yourself

- ✓ Write a grammar whose shape encodes precedence and associativity.
- ✓ Explain how the recursion side determines associativity.
- ✓ Give three ambiguities in C or C++ and how each is resolved.
- ✓ Write a recursive descent parser for an expression language.
- ✓ Explain why left-recursive rules become loops.
- ✓ Implement precedence climbing and say where associativity lives in it.
- ✓ Trace an LR parse and identify a shift/reduce decision.
- ✓ Say why production compilers prefer recursive descent.
- ✓ Read four complex C declarations using the right-left rule.
- ✓ Explain panic-mode recovery and cascade suppression.

Abstract syntax trees and semantic analysis

The parse tree says what the program *looks like*. Semantic analysis works out what it *means*: which declaration each name refers to, what type every expression has, and whether the program is legal at all.

Parse tree versus AST



```
/* A representative AST node: */

typedef enum {
        NODE_INT_LITERAL, NODE_IDENT, NODE_BINARY,
        NODE_UNARY, NODE_CALL, NODE_MEMBER, NODE_CAST,
        NODE_IF, NODE_WHILE, NODE_RETURN, NODE_BLOCK,
        NODE_VAR_DECL, NODE_FUNC_DECL,
} NodeKind;

typedef struct Node {
        NodeKind    kind;
        SourceRange range;      /* for diagnostics */
        Type       *type;       /* filled in by semantic
                                   analysis - NULL until then */

        union {
                struct { long value; } int_lit;
                struct { Symbol *sym; } ident;   /* <- resolved
                                                    by name
                                                    lookup    */
                struct { TokenKind op;
                         struct Node *lhs, *rhs; } binary;
                struct { struct Node *fn;
                         struct Node **args;
                         int nargs; } call;
                struct { struct Node *cond, *then_br,
                                     *else_br; } if_stmt;
                ...
        };
} Node;

/* THE TWO FIELDS THAT MATTER MOST are `type` and, for
   identifiers, `sym`. Before semantic analysis both are
   empty; afterwards, every expression has a type and every
   name points at its declaration. That is what "semantic
   analysis" produces.                                    */
```

Symbol tables and scope

```
/* NAME RESOLUTION: given an identifier, find its
   declaration.                                          */

typedef struct Scope {
        struct Scope *parent;
        HashMap      *symbols;     /* name -> Symbol* */
} Scope;

static Symbol *lookup(Scope *s, const char *name)
{
        for (; s != NULL; s = s->parent) {
                Symbol *sym = map_get(s->symbols, name);
                if (sym)
                        return sym;       /* INNERMOST wins */
        }
        return NULL;      /* undeclared */
}

/* THE SCOPE CHAIN gives shadowing for free: an inner
   declaration hides an outer one, because lookup stops at the
   first match.                                          */

        int x = 1;              /* file scope */
        void f(void) {
                int x = 2;      /* shadows it */
                {
                        int x = 3;   /* shadows again */
                        printf("%d", x);   /* 3 */
                }
                printf("%d", x);           /* 2 */
        }

/* WHICH IS WHY -Wshadow IS WORTH ENABLING: shadowing is legal
   and occasionally intended, and far more often a bug where
   someone redeclared a variable they meant to assign.     */

/* C's SCOPE KINDS:
     file scope        outside any function
     block scope       inside { }
     function scope    LABELS ONLY - a goto label is visible
                       throughout its function regardless of
                       nesting
     function prototype scope   parameter names in a
                       declaration, which is why they can be
                       omitted

/* AND C's FOUR NAMESPACES - genuinely separate, so this is
   all legal and refers to four different things: */
        struct foo { int foo; };
        typedef struct foo foo;
        foo foo;
        goto foo;
        foo: ;
/*   1. ordinary identifiers: variables, functions, typedefs,
        enum constants
     2. TAGS: struct, union, enum names - which is why you
        write `struct foo` rather than `foo` without a typedef
     3. MEMBERS: one namespace per struct or union
     4. LABELS                                          */
```

Type checking

```
/* Every expression gets a type, computed bottom-up. */

static Type *check_binary(Node *n, Scope *scope)
{
        Type *lt = check_expr(n->binary.lhs, scope);
        Type *rt = check_expr(n->binary.rhs, scope);

        switch (n->binary.op) {
        case TOK_PLUS:
                /* POINTER ARITHMETIC is a special case: */
                if (is_pointer(lt) && is_integer(rt))
                        return lt;
                if (is_integer(lt) && is_pointer(rt))
                        return rt;
                if (is_arithmetic(lt) && is_arithmetic(rt))
                        return usual_arithmetic_conversions(lt, rt);
                error(n, "invalid operands to binary +");
                return type_error;

        case TOK_MINUS:
                if (is_pointer(lt) && is_pointer(rt)) {
                        if (!compatible(pointee(lt), pointee(rt)))
                                error(n, "incompatible pointers");
                        return type_ptrdiff_t;   /* NOT a
                                                    pointer! */
                }
                ...

        case TOK_EQEQ: case TOK_LT:
                ...
                return type_int;      /* comparisons yield int
                                         in C, bool in C++    */
        }
}

/* THE USUAL ARITHMETIC CONVERSIONS - the rules that decide
   what `int + unsigned` means (C track chapter 02): */

   1. if either is long double, convert both to long double
   2. else if either is double, ...
   3. else if either is float, ...
   4. else INTEGER PROMOTIONS on both (anything narrower than
      int becomes int)
   5. if both are now the same type, done
   6. if both signed or both unsigned: the lower rank
      converts to the higher
   7. if the UNSIGNED type has rank >= the signed type:
      the signed converts to UNSIGNED       <-- the trap
   8. if the signed type can represent every value of the
      unsigned type: unsigned converts to signed
   9. otherwise both convert to the unsigned version of the
      signed type

/* RULE 7 IS THE BUG SOURCE: */
        int i = -1;
        unsigned u = 1;
        if (i < u)              /* i converts to UNSIGNED */
                puts("less");   /* NOT PRINTED: -1 becomes
                                   4294967295             */
```

```
/* THE COMPILER IS FOLLOWING THE STANDARD EXACTLY. -Wextra
   warns (-Wsign-compare). Enable it.                    */
```

> **IMPLICIT CONVERSIONS ARE WHERE C BUGS LIVE**
> The conversion rules are consistent, documented, and almost nobody has them memorised. The practical defence is not memorisation: it is `-Wconversion -Wsign-compare -Wsign-conversion`, plus the habit of using explicitly sized types (`uint32_t`, not `unsigned`) so the rank question is at least visible in the source.

Lvalues and value categories

```
/* An LVALUE designates an object - something with an address
   you could take. An RVALUE is a value.                  */

        int x, a[10], *p;

        x           lvalue
        x + 1       rvalue
        a           lvalue (of array type - but it DECAYS to
                    a pointer rvalue in most contexts)
        a[0]        lvalue
        *p          lvalue
        &x          rvalue (a pointer VALUE)
        f()         rvalue in C; may be an lvalue in C++ if it
                    returns a reference
        (x = 5)     rvalue in C - which is why `(x = 5) = 6`
                    is illegal

/* WHERE IT MATTERS - the compiler must check: */
        x = 5;          /* left side must be a MODIFIABLE
                           lvalue                          */
        5 = x;          /* error: not an lvalue */
        &5;             /* error: cannot take the address */
        const int c = 1;
        c = 2;          /* error: not MODIFIABLE */
        x++;            /* needs a modifiable lvalue */

/* "MODIFIABLE LVALUE" excludes: arrays, const-qualified
   objects, incomplete types, and structs containing a const
   member.

/* AND THE ARRAY DECAY RULE, which surprises everyone once: */
        void f(int a[10]) { sizeof(a); }
/* `a` here is a POINTER - the [10] is ignored. sizeof(a) is
   8, not 40. Array parameters do not exist in C; they are
   silently rewritten to pointers.                        */

/* C++ REFINES this into five value categories - lvalue,
   xvalue, prvalue, glvalue, rvalue - to support move
   semantics (C++ track chapter 05). The C model is the
   simple ancestor.                                        */
```

What semantic analysis checks

Check	Example error
Name is declared	"undeclared identifier 'foo'"

Check	Example error
No duplicate declaration in a scope	"redefinition of 'x'"
Types are compatible	"incompatible pointer types"
Assignment target is a modifiable lvalue	"lvalue required as left operand"
Function call arity and argument types	"too few arguments to function"
Return type matches	"returning 'int *' from a function returning 'int'"
Struct member exists	"no member named 'x' in 'struct foo'"
Type is complete where required	"invalid use of incomplete type"
Constant expressions really are constant	"variably modified type at file scope"
Case labels are unique and constant	"duplicate case value"
`break` / `continue` are in a loop	"break statement not within loop or switch"
Qualifiers are respected	"assignment discards 'const' qualifier"

Flow-sensitive analysis

```
/* Some checks need to know the ORDER of execution, not just
   the tree shape. These are done on a control flow graph.  */

/* --- Uninitialised use --- */
        int x;
        if (cond)
                x = 1;
        printf("%d", x);        /* MAY be uninitialised */

/* gcc -Wmaybe-uninitialized. It is a MAY analysis, so it has
   false positives and false negatives - the exact question is
   undecidable. That is why the warning is separate from
   -Wuninitialized (which fires only when it is certain).  */

/* --- Missing return --- */
        int f(int x) {
                if (x > 0)
                        return 1;
                /* falls off the end -> UNDEFINED BEHAVIOUR if
                   the value is used                         */
        }
/* -Wreturn-type. In C++ this is worse than in C: the
   optimiser may assume the path is unreachable and delete
   the branch that reaches it (chapter 15).              */

/* --- Unreachable code --- */
        return 1;
        printf("never");        /* -Wunreachable-code */

/* --- Fallthrough in a switch --- */
        switch (x) {
        case 1:
                do_a();
                /* fallthrough - intentional? */
        case 2:
                do_b();
                break;
        }
/* -Wimplicit-fallthrough. Silence it deliberately: */
        __attribute__((fallthrough));
        /* or, C23: */
        [[fallthrough]];

/* --- Definite assignment, in other languages --- */
/* Java and C# REQUIRE definite assignment and reject the
   program otherwise. Rust goes further with ownership and
   borrow checking (Rust track chapter 04). C only warns,
   which is a language design choice with real consequences
   for how many bugs reach production.                    */
```

Looking at the AST

```
# Clang prints a genuinely readable AST:
    clang -Xclang -ast-dump -fsyntax-only test.c

    TranslationUnitDecl
    `-FunctionDecl 0x... <test.c:1:1, line:4:1> f 'int (int)'
      |-ParmVarDecl 0x... x 'int'
      `-CompoundStmt
        `-ReturnStmt
          `-BinaryOperator 'int' '+'
            |-ImplicitCastExpr 'int' <LValueToRValue>
            | `-DeclRefExpr 'int' lvalue ParmVar 0x... 'x'
            `-IntegerLiteral 'int' 1

# READ THE ANNOTATIONS - they are the output of semantic
# analysis:
#   'int (int)'          the function's computed type
#   ImplicitCastExpr     a conversion the compiler INSERTED
#   <LValueToRValue>     the specific conversion kind
#   lvalue               the value category
#   DeclRefExpr ... 0x...  the resolved declaration - the
#                          address links back to the ParmVarDecl

# THE IMPLICIT CASTS ARE THE INTERESTING PART. Every
# conversion C performs silently appears here as a node. If
# you have ever wondered where an unexpected promotion came
# from, this dump answers it exactly.

    clang -Xclang -ast-dump=json -fsyntax-only test.c
    ; machine-readable, for tooling

    gcc -fdump-tree-original-raw test.c
    ; GCC's equivalent, considerably less readable

# AND FOR C++, where the answer is much less obvious:
    clang++ -Xclang -ast-dump -fsyntax-only test.cpp
    ; shows which overload was selected, which template was
    ; instantiated, and every implicit conversion - the three
    ; things hardest to determine by reading.
```

How it shows up

Symptom	Cause
"undeclared identifier"	Missing include, typo, or a scope you did not expect
Comparison behaves backwards	Signed/unsigned conversion — enable `-Wsign-compare`
`sizeof` on a parameter is 8	Array decay — array parameters are pointers
"lvalue required"	Assigning to something with no address
"incompatible pointer types"	A real type error — never silence it with a cast without understanding it
Value changes unexpectedly in arithmetic	Integer promotion or the usual arithmetic conversions
Wrong variable modified	Shadowing — `-Wshadow`
Works, but a warning about maybe-uninitialised	Possibly a false positive; possibly a real path you have not considered
Missing return causes odd behaviour at `-O2`	UB — the optimiser assumed the path was unreachable

Symptom	Cause
Cannot tell which overload was chosen (C++)	Dump the AST

Check yourself

- ✓ Draw the parse tree and the AST for one expression, and say what was dropped.
- ✓ Explain how a scope chain gives shadowing, and why `-Wshadow` is worth enabling.
- ✓ Name C's four namespaces and give an example using all of them.
- ✓ State the usual arithmetic conversions and demonstrate the signed/unsigned trap.
- ✓ Classify six expressions as lvalues or rvalues.
- ✓ Explain array decay and its effect on `sizeof`.
- ✓ List ten checks semantic analysis performs.
- ✓ Explain why uninitialised-use analysis has false positives.
- ✓ Dump a Clang AST and identify every implicit conversion in it.
- ✓ Explain what the resolved `DeclRefExpr` address tells you.

Intermediate representations and SSA

The form a compiler thinks in. Static Single Assignment makes data flow explicit, which is what turns most optimisations from difficult analyses into simple graph walks — and it is why modern compilers optimise as well as they do.

Why an IR at all

```
Without an IR, supporting L languages on T targets needs
L * T compilers.

With one:   L front ends  +  1 optimiser  +  T back ends

AND THE IR MUST BE:
  · LANGUAGE-NEUTRAL: no C-specific or Rust-specific
    concepts
  · TARGET-NEUTRAL: no x86 or ARM assumptions, at least
    until the back end
  · SIMPLE: few operation kinds, so a pass handles all of
    them
  · ANALYSABLE: data flow and control flow explicit
  · TYPED: enough to reason about sizes and aliasing

THE LEVELS a compiler passes through:

  HIGH      close to the AST. Types, loops, structured
            control flow. GCC's GENERIC.
     |
  MID       three-address code, basic blocks, SSA.
            Most optimisation happens here.
            GCC's GIMPLE, LLVM IR.
     |
  LOW       close to machine code, but still virtual
            registers. GCC's RTL, LLVM's MachineIR.
     |
  MACHINE   real instructions, real registers.
```

Three-address code

```
/* SOURCE: */
        x = (a + b) * (c - d);

/* THREE-ADDRESS CODE - each instruction does ONE operation
   with at most three operands:                              */

        t1 = a + b
        t2 = c - d
        t3 = t1 * t2
        x  = t3

/* THE FORM: `result = operand1 op operand2`. Nested
   expressions become sequences of temporaries.

   WHY: every instruction is independently analysable and
   movable. A tree expression has to be walked; a flat list of
   simple operations can be reordered, deleted, or replaced
   one at a time.

/* THE OPERATION KINDS a mid-level IR needs, and it is a
   short list:
        arithmetic     add sub mul div rem
        bitwise        and or xor shl shr
        comparison     eq ne lt le gt ge
        memory         load store
        control        br  cond_br  ret  switch
        call
        conversion     trunc, extend, bitcast, int-to-float
        address        getelementptr / array_ref

   Everything in every source language reduces to these.  */
```

Basic blocks and the control flow graph

```
A BASIC BLOCK is a maximal straight-line sequence:
  - entered only at the first instruction
  - left only at the last

  SOURCE:
      if (a > 0)
              b = 1;
      else
              b = 2;
      c = b + 1;

  CFG:
                  +-------------+
                  | entry:      |
                  |  t = a > 0  |
                  |  br t, L1,L2|
                  +--+-------+--+
                     |       |
               true  |       |  false
                     v       v
            +-------+--+ +--+-------+
            | L1:      | | L2:      |
            |  b1 = 1  | |  b2 = 2  |
            |  br L3   | |  br L3   |
            +-------+--+ +--+-------+
                    |       |
                    +---+---+
                        v
                  +-----+--------+
                  | L3:          |
                  |  b3 = phi(...)|
                  |  c = b3 + 1  |
                  +--------------+

  THE CFG IS THE FOUNDATION of every flow analysis:
  reachability, liveness, dominance, loop detection.

  KEY RELATIONSHIPS:
    PREDECESSORS / SUCCESSORS   the edges
    DOMINATES     A dominates B if EVERY path from entry to B
                  passes through A. This is what makes
                  "definition reaches use" decidable, and it is
                  what SSA construction is built on.
    POST-DOMINATES  the mirror image, used for sinking code
    LOOP          a back edge to a dominator
    NATURAL LOOP  a loop with a single entry (a header) -
                  which is what loop optimisations require
                  (chapter 07)
```

Static Single Assignment

```
/* THE RULE: every variable is assigned EXACTLY ONCE. Reassign
   and you get a new name.                                    */

/* NOT SSA: */
        x = 1;
        x = x + 1;
        y = x * 2;

/* SSA: */
        x1 = 1
        x2 = x1 + 1
        y1 = x2 * 2

/* WHY THIS IS TRANSFORMATIVE:

   Before SSA, "what value does x have here?" requires a
   dataflow analysis over the whole function - which
   assignments reach this use, along which paths.

   In SSA, x2 has exactly ONE definition, and it is right
   there. THE NAME IS THE ANSWER. Use-def chains become
   pointers rather than analyses.

   CONSTANT PROPAGATION becomes: look at the definition; if
   it is a constant, substitute.
   DEAD CODE ELIMINATION becomes: does this definition have
   any uses? No -> delete.
   VALUE NUMBERING becomes: do two definitions compute the
   same expression from the same operands? -> they are equal.

   Each of these was a whole-function analysis; in SSA they
   are local.                                              */
```

Phi nodes

```
/* THE PROBLEM: at a merge point, which definition applies? */

        if (cond)
                x = 1;
        else
                x = 2;
        y = x;          /* which x? */

/* THE SOLUTION: a PHI FUNCTION at the top of the merge
   block, selecting by which predecessor you came from.    */

   entry:
        br cond, L1, L2

   L1:
        x1 = 1
        br L3

   L2:
        x2 = 2
        br L3

   L3:
        x3 = phi [x1, L1], [x2, L2]
        y1 = x3

/* A PHI IS NOT A REAL INSTRUCTION. It is a notation meaning
   "the value depends on the incoming edge". The back end
   removes them by inserting COPIES at the end of each
   predecessor block - "phi elimination" - and then register
   allocation usually makes those copies free by assigning
   x1, x2 and x3 the same register (chapter 09).

/* PHIS ARE ALWAYS AT THE TOP of a block, and they all execute
   "simultaneously" - a phi may reference a value defined by
   another phi in the same block, and the semantics are that
   all read the OLD values. That matters for loop-carried
   dependencies:                                            */

   loop:
        i2 = phi [0, entry], [i3, loop]
        s2 = phi [0, entry], [s3, loop]
        s3 = s2 + i2
        i3 = i2 + 1
        c  = i3 < n
        br c, loop, exit

/* Read that and the loop's structure is completely explicit:
   two loop-carried values, their initial values, and their
   updates. That legibility is why loop optimisations
   (chapter 07) are practical.                             */
```

WHERE PHI NODES GO: THE DOMINANCE FRONTIER

A phi for variable `x` is needed in block `B` exactly when two different definitions of `x` can reach `B` along different paths. The precise answer is the *iterated dominance frontier* of the blocks that define `x` — the first blocks where a definition's dominance "runs out". Cytron's 1991 algorithm computes this efficiently, and it is what made SSA practical rather than merely elegant.

LLVM IR

```
; C source:
;     int sum(int *a, int n) {
;             int s = 0;
;             for (int i = 0; i < n; i++)
;                     s += a[i];
;             return s;
;     }

; clang -S -emit-llvm -O1 sum.c -o -

define i32 @sum(ptr noundef %a, i32 noundef %n) {
entry:
  %cmp = icmp sgt i32 %n, 0
  br i1 %cmp, label %loop.preheader, label %exit

loop.preheader:
  %wide.n = zext i32 %n to i64
  br label %loop

loop:
  %i    = phi i64 [ 0, %loop.preheader ], [ %i.next, %loop ]
  %s    = phi i32 [ 0, %loop.preheader ], [ %s.next, %loop ]
  %ptr  = getelementptr inbounds i32, ptr %a, i64 %i
  %v    = load i32, ptr %ptr, align 4
  %s.next = add nsw i32 %s, %v
  %i.next = add nuw nsw i64 %i, 1
  %done = icmp eq i64 %i.next, %wide.n
  br i1 %done, label %exit, label %loop

exit:
  %result = phi i32 [ 0, %entry ], [ %s.next, %loop ]
  ret i32 %result
}

/* READ THE ANNOTATIONS - they carry the optimiser's
   knowledge:

     i32, i64, ptr    explicit types on everything
     %name            an SSA value; %0, %1 for unnamed
     phi              the loop-carried values, explicit
     getelementptr    address computation, type-aware. It
                      knows the element size, so no manual
                      multiply appears.
     inbounds         "this address stays within the object" -
                      a PROMISE from the front end, derived
                      from C's rules. It licenses
                      optimisations that would otherwise be
                      illegal.
     nsw / nuw        "no signed/unsigned wrap" - i.e. signed
                      overflow is UB in C, so the optimiser
                      may assume it does not happen
                      (chapter 15). THIS IS WHERE UB ENTERS
                      THE IR.
     noundef          the argument is not undef or poison
     align 4          the load's alignment

   THE `nsw` FLAG IS THE ONE TO UNDERSTAND. It is how
   "signed overflow is undefined" travels from the C standard
```

```
   into the optimiser, and it is why `i + 1 > i` can be folded
   to `true` for signed i but not for unsigned.          */
```

Memory and the SSA boundary

```
/* SSA works for SCALARS in registers. Memory is different:
   two pointers might alias, so a store through one may
   change what a load through the other sees. SSA cannot
   express that with names alone.                         */

/* SO CLANG STARTS by putting everything in memory: */

define i32 @f(i32 %x) {
  %x.addr = alloca i32            ; a stack slot
  store i32 %x, ptr %x.addr
  %1 = load i32, ptr %x.addr
  %2 = add i32 %1, 1
  store i32 %2, ptr %x.addr
  %3 = load i32, ptr %x.addr
  ret i32 %3
}

/* ...and then the `mem2reg` (or SROA) pass PROMOTES stack
   slots whose address is never taken into SSA values:    */

define i32 @f(i32 %x) {
  %1 = add i32 %x, 1
  ret i32 %1
}

/* THIS IS WHY `&x` MATTERS FOR PERFORMANCE. Take the address
   of a local and pass it somewhere the compiler cannot see,
   and the variable can no longer be promoted - it must live
   in memory, be reloaded after every call that might modify
   it, and cannot participate in most optimisations.

   THE PRACTICAL RULE: do not take the address of a local
   unless you need to. `int i` in a loop is free; `int i;
   f(&i);` is not.

/* AND FOR MEMORY THAT STAYS IN MEMORY, the optimiser needs
   ALIAS ANALYSIS: can this store affect that load? The
   answers come from
     · type-based rules (strict aliasing - chapter 15)
     · `restrict`, which promises no aliasing
     · escape analysis: an object whose address never leaves
       the function cannot be touched from outside
   and the quality of those answers is most of what separates
   a good optimiser from a mediocre one.                  */
```

GCC's IRs

```
# GCC uses several, in sequence:

# GENERIC - a language-independent tree, close to the AST
    gcc -fdump-tree-original test.c

# GIMPLE - three-address, then SSA form
    gcc -fdump-tree-gimple test.c
    gcc -fdump-tree-ssa test.c
    gcc -fdump-tree-optimized test.c    ; after all tree passes

    ;; sum (int * a, int n)
    ;; basic block 3:
    ;;   # i_15 = PHI <0(2), i_11(3)>
    ;;   # s_16 = PHI <0(2), s_10(3)>
    ;;   _1 = (long unsigned int) i_15;
    ;;   _2 = _1 * 4;
    ;;   _3 = a_8(D) + _2;
    ;;   _4 = *_3;
    ;;   s_10 = s_16 + _4;
    ;;   i_11 = i_15 + 1;
    ;;   if (i_11 < n_9(D)) goto bb 3; else goto bb 4;

# NOTE the naming: `i_15` is version 15 of `i` - GCC keeps the
# source name, which makes GIMPLE dumps easier to relate back
# to code than LLVM's numbered values. `(D)` marks a default
# definition, i.e. a parameter.

# RTL - low level, close to machine instructions
    gcc -fdump-rtl-all test.c

# EVERY PASS, in order, with statistics:
    gcc -fdump-tree-all-details test.c
    gcc -fopt-info-all test.c

# THE PRACTICAL USE: when a loop does not optimise as you
# expect, dump `-fdump-tree-optimized` and look at what the
# middle end actually produced. It is frequently obvious in
# the dump and invisible in the assembly.
```

How it shows up

Symptom	Cause
A local variable is never in a register	Its address was taken — `mem2reg` could not promote it
A value is reloaded after every call	The compiler cannot prove the call does not modify it
A loop is not optimised	Not a natural loop, or a memory dependency it cannot disprove
`i + 1 > i` folds to true for `int`	The `nsw` flag — signed overflow is UB (ch. 15)
The same code is faster with `restrict`	Alias analysis could not prove independence without it
Assembly does not match the source at all	Read the IR dump — the transformation is visible there
Optimisation differs between GCC and Clang	Different alias analysis and pass ordering, not a bug in either
Struct fields not kept in registers	SROA could not split it — usually because its address escaped

Check yourself

- ✓ Explain why an IR exists and what properties it needs.
- ✓ Convert an expression to three-address code.
- ✓ Draw a control flow graph and identify dominance and a natural loop.
- ✓ Convert straight-line code to SSA and explain what SSA makes cheap.
- ✓ Explain what a phi node means and how the back end removes it.
- ✓ Read an LLVM IR loop and identify the loop-carried values.
- ✓ Explain `nsw`, `inbounds` and where they come from.
- ✓ Explain `mem2reg` and why taking an address costs performance.
- ✓ Say what alias analysis is for and three sources of its answers.
- ✓ Dump GIMPLE for a loop and relate each statement to the source.

Machine-independent optimisation

What the middle end actually does to your code, why each transformation is legal, and the one assumption underneath all of it: that your program has no undefined behaviour.

The rule every optimisation obeys

THE AS-IF RULE

A compiler may perform any transformation, however drastic, provided the *observable behaviour* of the program is unchanged. In C, observable behaviour is: I/O, accesses to `volatile` objects, and the final state at program termination.

Everything else — the order of computations, whether a variable exists at all, how many times a loop body runs, whether a function call happens — is fair game. And because **undefined behaviour has no defined observable behaviour**, a program containing UB places no constraint on the compiler at all along that path (chapter 15).

The local transformations

```
/* --- CONSTANT FOLDING: evaluate at compile time --------- */
       int x = 2 * 3 + 4;              ->  int x = 10;
       if (sizeof(int) == 4)           ->  if (1)

/* --- CONSTANT PROPAGATION ------------------------------- */
       x = 5;
       y = x + 1;      ->  y = 6;
       z = y * 2;      ->  z = 12;

/* --- ALGEBRAIC SIMPLIFICATION --------------------------- */
       x * 1     -> x            x + 0     -> x
       x * 0     -> 0            x - x     -> 0
       x / 1     -> x            x & x     -> x
       x | 0     -> x            x ^ x     -> 0
       !!x       -> x != 0       -(-x)     -> x
       (x+1)-1   -> x            <-- ONLY if overflow is UB!

/* --- STRENGTH REDUCTION: cheaper operations ------------- */
       x * 8       ->  x << 3
       x / 8       ->  x >> 3      (unsigned; signed needs a
                                    correction for negatives)
       x % 8       ->  x & 7       (unsigned)
       x * 10      ->  (x << 3) + (x << 1)
       pow(x, 2)   ->  x * x

/* THE SIGNED DIVISION ONE is worth seeing, because it
   explains a common "why is this so many instructions?": */
       int f(int x) { return x / 8; }
   ->  add   w8, w0, #7        ; if negative, bias by 7
       cmp   w0, #0
       csel  w8, w8, w0, lt    ; select the biased value
       asr   w0, w8, #3        ; arithmetic shift
   Four instructions, because C rounds toward zero and a
   plain shift rounds toward negative infinity. Unsigned
   division by 8 is ONE instruction. That is a real reason to
   use unsigned types for indices and sizes.

/* --- COMMON SUBEXPRESSION ELIMINATION ------------------- */
       a = b * c + g;
       d = b * c * e;
   ->  t = b * c;
       a = t + g;
       d = t * e;

/* --- COPY PROPAGATION ----------------------------------- */
       y = x;
       z = y + 1;      ->  z = x + 1;

/* --- DEAD CODE ELIMINATION ------------------------------ */
       x = compute();          /* if x is never used AND
                                  compute() has no side
                                  effects -> delete both  */

/* --- DEAD STORE ELIMINATION ----------------------------- */
       x = 1;
       x = 2;          ->  x = 2;
/* AND THE SECURITY PROBLEM this causes: */
       memset(key, 0, sizeof(key));    /* DELETED - the
                                          buffer is never
                                          read afterwards */
```

```
        free(key);
/* Use explicit_bzero(), memset_s(), SecureZeroMemory(), or a
   volatile pointer. This is a real, exploited bug class
   (isolation chapter 16).                                  */
```

Inlining

```
/* THE MOST IMPORTANT OPTIMISATION, because it enables all the
   others. */

        static int square(int x) { return x * x; }
        int f(int a) { return square(a) + square(3); }

/* after inlining: */
        int f(int a) { return a * a + 3 * 3; }
/* after constant folding: */
        int f(int a) { return a * a + 9; }

/* THE COMPOUNDING EFFECT is the point: inlining exposes the
   arguments as constants, which enables folding, which
   enables branch elimination, which enables dead code
   removal, which may enable more inlining.

/* WHAT THE HEURISTIC WEIGHS:
     · the callee's size (LLVM's cost model, GCC's
       -finline-limit)
     · how many call sites there are
     · whether the callee is `static` and called once - then
       inlining is FREE, because the out-of-line copy can be
       deleted
     · whether arguments are constants at the call site
     · whether it is in a loop
     · profile data, if available (PGO)

/* THE ATTRIBUTES: */
        static inline int f(void) { ... }
        /* `inline` is a HINT, and in C it primarily affects
           linkage rules, not the decision. */

        __attribute__((always_inline))     /* force it */
        __attribute__((noinline))          /* forbid it */
        __attribute__((flatten))           /* inline
                                              everything this
                                              function calls */

/* WHEN NOT TO INLINE - and these are real:
     · large functions called from many places: I-cache
       pressure costs more than the call (CA chapter 14)
     · cold error paths: inlining them pushes the hot path out
       of the cache. Mark them noinline.
     · anything you need to see in a profile or a backtrace

/* THE EMBEDDED CONSIDERATION: -O2 inlines aggressively and
   can grow code substantially. On a flash-constrained part
   measure with `size` and consider -Os, which uses a much
   more conservative inlining threshold.                    */
```

Global transformations

```
/* --- GLOBAL VALUE NUMBERING ------------------------------ */
/* CSE across basic blocks, using SSA to identify expressions
   that provably compute the same value.                    */

/* --- PARTIAL REDUNDANCY ELIMINATION ---------------------- */
        if (cond)
                x = a + b;      /* computed here */
        y = a + b;              /* redundant on that path,
                                   not on the other        */
   ->   if (cond)  { t = a + b; x = t; }
        else       { t = a + b; }
        y = t;
/* Hoist the computation onto the path that lacked it, so it
   is computed exactly once on every path.                  */

/* --- TAIL CALL ELIMINATION ------------------------------- */
        int f(int n, int acc) {
                if (n == 0) return acc;
                return f(n - 1, acc * n);    /* TAIL call */
        }
   ->   a LOOP. No stack growth.
/* Which is why the recursive version is as fast as the
   iterative one at -O2, and why it stack-overflows at -O0.
   C does not GUARANTEE this - it is an optimisation, not a
   language rule, unlike in Scheme.                         */

/* --- JUMP THREADING -------------------------------------- */
        if (a) x = 1; else x = 2;
        if (a) y = 3; else y = 4;
   ->   if (a) { x = 1; y = 3; } else { x = 2; y = 4; }
/* The second test is known on each path.                   */

/* --- IF-CONVERSION: branches to conditional moves ------- */
        if (a > b) m = a; else m = b;
   ->   cmp  w0, w1
        csel w0, w0, w1, gt          ; ONE instruction, no
                                     ; branch, no misprediction
/* On ARM this is very effective because of the rich
   conditional-select instructions (ARM chapter 07). It is
   also why a "branchless" hand-optimisation is usually
   unnecessary: the compiler already did it.
   BUT: it evaluates BOTH sides, so it is only legal when
   both are side-effect free and safe to execute. This is
   also why if-conversion is NOT constant-time-safe in the
   way people assume for crypto (cryptography ch. 19).   */

/* --- FUNCTION SPECIALISATION (IPA-CP) -------------------- */
        static void f(int mode) { if (mode) ... else ... }
        f(1); f(1); f(0);
   ->   two specialised copies, each with the branch folded
        away.

/* --- DEVIRTUALISATION (C++) ------------------------------ */
/* If the dynamic type is provable, replace an indirect call
   through a vtable with a direct call - which then allows
   inlining. Enormously important for C++ performance
   (C++ track chapter 07).                                  */
```

Pass ordering

```
Optimisations ENABLE one another, so order matters and no
single order is optimal for all programs. A representative
LLVM -O2 pipeline:

   SROA                     split aggregates, promote to SSA
   EarlyCSE
   InstCombine              peephole on the IR
   SimplifyCFG              merge and delete blocks
      |
   INLINING  <--------------+  (the "CGSCC" pass manager runs
      |                     |   this bottom-up over the call
   more InstCombine,        |   graph, so a callee is
   SimplifyCFG, SROA -------+   optimised before its caller
      |                         is considered for inlining)
   GVN                      global value numbering
   MemCpyOpt
   DSE                      dead store elimination
      |
   LOOP PASSES:
     LoopRotate             turn while into do-while
     LICM                   hoist invariants (chapter 07)
     LoopUnswitch
     IndVarSimplify         canonicalise induction variables
     LoopIdiom              recognise memset/memcpy patterns
     LoopDeletion
     LoopUnroll
     LoopVectorize          (chapter 07)
     SLPVectorize
      |
   InstCombine, SimplifyFG  again - the loop passes exposed
      |                     new opportunities
   ADCE                     aggressive dead code elimination
      |
   -> the back end (chapters 08-09)

  NOTE HOW OFTEN InstCombine AND SimplifyCFG REPEAT. Cleanup
  passes run after every major transformation because each one
  leaves debris the next pass would otherwise trip over.

  SEE THE REAL ONE:
     llvm: opt -O2 -debug-pass-manager < in.ll > /dev/null
     gcc:  -fdump-tree-all -fdump-ipa-all
```

Link-time optimisation

```
# The problem: separate compilation (chapter 01) means the
# compiler cannot inline across translation units, or prove
# anything about a function defined elsewhere.

# LTO defers optimisation until link time, when everything is
# visible.

    gcc -flto -O2 -c a.c b.c
    gcc -flto -O2 a.o b.o -o prog
    ; the .o files contain IR, not machine code

    clang -flto=thin -O2 ...     ; ThinLTO: parallel, scalable,
                                 ; nearly as effective, much
                                 ; faster to link

# WHAT IT ENABLES:
#   cross-module inlining
#   whole-program dead code elimination - unused functions
#     across the whole program disappear
#   better devirtualisation
#   global constant propagation
#   identical code folding

# TYPICAL RESULTS: 5-15% faster, and often SMALLER because of
# the dead code elimination - which on an embedded target may
# matter more than the speed.

# THE COSTS:
#   link time grows substantially (ThinLTO mitigates this)
#   memory use at link time
#   debugging is harder - inlining across files confuses
#     backtraces
#   it EXPOSES UB that separate compilation hid. Code that
#     "worked" may now break, because the optimiser can finally
#     see the contradiction (chapter 15).

# THE LAST POINT IS THE ONE TO EXPECT: enabling LTO on an old
# codebase frequently surfaces latent bugs. They were always
# bugs.
```

Profile-guided optimisation

```
# The compiler's heuristics guess which branches are taken and
# which functions are hot. PGO replaces the guess with
# measurement.

# 1. build instrumented
    gcc -fprofile-generate -O2 prog.c -o prog
    clang -fprofile-instr-generate -O2 prog.c -o prog

# 2. RUN IT on a REPRESENTATIVE workload
    ./prog typical-input

# 3. rebuild using the data
    gcc -fprofile-use -O2 prog.c -o prog
    clang -fprofile-instr-use=prog.profdata -O2 prog.c -o prog

# WHAT IT IMPROVES:
#   branch layout - the hot path becomes the fall-through, so
#     the I-cache holds contiguous hot code (CA chapter 12)
#   inlining decisions - inline what is actually hot
#   hot/cold splitting - cold code moved to a separate section
#   loop unrolling based on real trip counts
#   register allocation priorities

# TYPICAL: 10-20% on branch-heavy code. Sometimes much more on
# interpreters and parsers.

# SAMPLE-BASED PGO, which avoids the instrumented build:
    perf record -b ./prog
    create_llvm_prof --binary=./prog --out=prog.prof
    clang -fprofile-sample-use=prog.prof ...
# Lower overhead, usable on production workloads, slightly
# less accurate.

# THE CAVEAT: a profile from an unrepresentative workload can
# make things WORSE, because the compiler will optimise
# confidently for the wrong case. Profile what you actually
# run.
```

Watching it happen

```
# Which optimisations fired, and which did not and why:
    gcc -fopt-info-all
    gcc -fopt-info-inline-optimized
    gcc -fopt-info-vec-missed

    clang -Rpass=inline
    clang -Rpass-missed=loop-vectorize
    clang -Rpass-analysis=loop-vectorize
    ; the -missed and -analysis forms print the REASON, which
    ; is the useful part

# Step through the IR one pass at a time:
    opt -O2 -print-after-all input.ll 2> passes.txt
    opt -passes='sroa,instcombine' -S input.ll
    ; run exactly the passes you want, and see the effect of
    ; each. This is the best way to understand what a pass
    ; actually does.

# GCC's equivalent:
    gcc -fdump-tree-all-details
    ls *.c.*t.*        ; one file per pass, in order

# AND ALWAYS: godbolt.org, with the "Optimization Pipeline"
# view enabled - it shows the IR after every pass, diffed.
```

How it shows up

Symptom	Cause
A `memset` that clears a key is gone	Dead store elimination — use `explicit_bzero`
A loop that should run is deleted	Its result is unused, or it contains UB
Signed division is four instructions	C's round-toward-zero rule — use unsigned where correct
A "branchless" hand-optimisation is slower	The compiler already did if-conversion, better
Recursive code overflows only at `-O0`	Tail call elimination at higher levels
Code grows a lot at `-O2`	Inlining — try `-Os` on constrained targets
Hot path slower after inlining	I-cache pressure — mark cold paths `noinline`
LTO breaks a working program	Latent UB the optimiser can now see
PGO makes it slower	Unrepresentative profile
A function does not appear in a profile	Inlined — `-fno-inline` to check
Assembly bears no resemblance to the source	Working as intended; read the IR to follow it

Check yourself

- ✓ State the as-if rule and what counts as observable behaviour in C.
- ✓ Name eight local optimisations and give an example of each.
- ✓ Explain why signed division by a power of two costs more than unsigned.
- ✓ Explain why dead store elimination is a security problem, and the fix.
- ✓ Explain why inlining is the most important optimisation.

- ✓ Give three cases where inlining should be prevented.
- ✓ Explain tail call elimination and why it is not guaranteed in C.
- ✓ Explain if-conversion and one situation where it is not what you want.
- ✓ Say why cleanup passes repeat throughout the pipeline.
- ✓ Explain what LTO enables and why it exposes latent bugs.
- ✓ Run a PGO build and say what it improves.

Loop optimisation and vectorisation

Programs spend most of their time in loops, so compilers spend most of their effort there. This chapter covers the transformations, why a loop fails to vectorise, and how to find out which reason applies to yours.

The loop transformations

```
/* --- LICM: loop-invariant code motion -------------------- */
        for (i = 0; i < n; i++)
                a[i] = b[i] * (x * y);      /* x*y each time */
   ->   t = x * y;
        for (i = 0; i < n; i++)
                a[i] = b[i] * t;

/* LICM is why manually hoisting is usually unnecessary - AND
   why it sometimes IS necessary: the compiler can only hoist
   what it can prove is invariant. A call it cannot see into,
   or a load it cannot prove is unaliased, stays inside.  */

/* --- STRENGTH REDUCTION on induction variables ---------- */
        for (i = 0; i < n; i++)
                a[i * 4] = 0;           /* a multiply per
                                           iteration        */
   ->   for (p = a; p < a + n*4; p += 4)
                *p = 0;                 /* an add           */

/* --- LOOP ROTATION: while -> do-while ------------------- */
        while (cond) { body }
   ->   if (cond) { do { body } while (cond); }
/* One test per iteration instead of a test plus a jump, and
   the loop body becomes a single block ending in a
   conditional branch - which is what most later loop passes
   require. Nearly every loop is rotated first.          */

/* --- LOOP UNROLLING ------------------------------------- */
        for (i = 0; i < n; i++)
                a[i] = b[i] + 1;
   ->   for (i = 0; i < n - 3; i += 4) {
                a[i]   = b[i]   + 1;
                a[i+1] = b[i+1] + 1;
                a[i+2] = b[i+2] + 1;
                a[i+3] = b[i+3] + 1;
        }
        for (; i < n; i++)              /* the EPILOGUE, for
                a[i] = b[i] + 1;           the remainder    */

/* WHY: fewer branches, more instruction-level parallelism,
   and it exposes vectorisation. COST: code size, and on a
   small I-cache that can lose more than it gains.
        #pragma GCC unroll 4
        #pragma clang loop unroll_count(4)
        #pragma clang loop unroll(disable)              */

/* --- LOOP FUSION ---------------------------------------- */
        for (i = 0; i < n; i++) a[i] = b[i];
        for (i = 0; i < n; i++) c[i] = a[i] * 2;
   ->   for (i = 0; i < n; i++) { a[i] = b[i];
                                  c[i] = a[i] * 2; }
/* One pass over memory instead of two - a cache win
   (CA chapter 14), often larger than the arithmetic saving. */

/* --- LOOP INTERCHANGE ----------------------------------- */
        for (j = 0; j < n; j++)
                for (i = 0; i < n; i++)
                        a[i][j] = 0;    /* STRIDE n - a cache
                                           miss per element */
   ->   for (i = 0; i < n; i++)
```

```
                for (j = 0; j < n; j++)
                        a[i][j] = 0;    /* stride 1          */
/* THE SINGLE BIGGEST WIN available on array code, and one the
   compiler often cannot do because it cannot prove the
   interchange is legal. Writing the loops in the right order
   is worth more than any flag.                              */

/* --- LOOP UNSWITCHING ----------------------------------- */
        for (i = 0; i < n; i++) {
                if (flag) do_a(i); else do_b(i);
        }
   ->   if (flag) for (i = 0; i < n; i++) do_a(i);
        else      for (i = 0; i < n; i++) do_b(i);
/* Hoist a loop-invariant condition out, at the cost of
   duplicating the loop.                                      */

/* --- IDIOM RECOGNITION ---------------------------------- */
        for (i = 0; i < n; i++) a[i] = 0;
   ->   memset(a, 0, n * sizeof(*a));
        for (i = 0; i < n; i++) a[i] = b[i];
   ->   memcpy(a, b, n * sizeof(*a));
/* Which is why hand-written copy loops are not faster than
   memcpy - they usually BECOME memcpy. And why
   -ffreestanding code still needs memcpy to exist
   (C track chapter 20).                                      */
```

Vectorisation

```
SIMD: one instruction, several data elements.

   SCALAR, 4 iterations:
      ldr  w1, [x0]        a[0]
      add  w1, w1, #1
      str  w1, [x2]
      ... x4

   VECTOR, 1 iteration:
      ldr  q0, [x0]        a[0..3], 128 bits at once
      add  v0.4s, v0.4s, v1.4s      four adds
      str  q0, [x2]

  ARM's SIMD:
     NEON (Advanced SIMD)   fixed 128-bit registers v0-v31.
                            Universally available on ARMv8-A.
     SVE / SVE2             VECTOR-LENGTH AGNOSTIC: the same
                            binary runs on 128-bit and
                            2048-bit implementations, using
                            predicates and a loop that adapts.
                            Server and newer mobile parts.

  ELEMENT ARRANGEMENTS on a 128-bit NEON register:
     v0.16b   16 x  8-bit
     v0.8h     8 x 16-bit
     v0.4s     4 x 32-bit
     v0.2d     2 x 64-bit

  THE THEORETICAL SPEEDUP is the element count - 4x for
  32-bit. THE REAL SPEEDUP is usually 2-3x, because loads,
  stores and loop overhead do not vectorise, and memory
  bandwidth often becomes the limit (CA chapter 18).
```

Why a loop does not vectorise

```
/* --- 1. LOOP-CARRIED DEPENDENCE ------------------------- */
        for (i = 1; i < n; i++)
                a[i] = a[i-1] + 1;      /* each iteration
                                           needs the previous */
/* Genuinely not vectorisable. Restructure the algorithm, or
   accept it.                                               */

/* BUT THIS ONE IS FINE: */
        for (i = 1; i < n; i++)
                a[i-1] = a[i] + 1;      /* reads ahead of
                                           writes - safe    */

/* --- 2. POSSIBLE ALIASING ------------------------------- */
        void add(int *a, int *b, int *c, int n) {
                for (int i = 0; i < n; i++)
                        a[i] = b[i] + c[i];
        }
/* The compiler cannot prove a, b and c do not overlap. It
   will either refuse, or - more commonly at -O3 - emit a
   RUNTIME ALIAS CHECK and two versions of the loop.

   THE FIX: */
        void add(int * restrict a, const int * restrict b,
                 const int * restrict c, int n)
/* `restrict` PROMISES no aliasing through these pointers for
   the object's lifetime. Break the promise and you get
   undefined behaviour and wrong results - so only use it
   when it is true.                                          */

/* --- 3. NON-CONTIGUOUS ACCESS --------------------------- */
        for (i = 0; i < n; i++)
                sum += a[i].field;      /* stride = sizeof
                                           the struct        */
/* Gather/scatter is possible on SVE but slow, and NEON has no
   general gather. THE FIX is a data layout change:
   structure-of-arrays instead of array-of-structures.    */

        /* AoS - poor for SIMD */
        struct P { float x, y, z; } points[N];
        /* SoA - vectorises perfectly */
        struct { float x[N], y[N], z[N]; } points;

/* --- 4. CONTROL FLOW INSIDE THE LOOP -------------------- */
        for (i = 0; i < n; i++)
                if (a[i] > 0)
                        b[i] = a[i];
/* Vectorisable via MASKING - compute all lanes, then blend by
   a predicate. NEON can do this; SVE does it natively. But an
   early `break` cannot be vectorised at all, because the trip
   count is not known in advance.                          */

/* --- 5. FUNCTION CALLS ---------------------------------- */
        for (i = 0; i < n; i++)
                a[i] = sinf(b[i]);
/* Needs a VECTORISED VERSION of the function. libmvec
   provides these for the maths functions; -ffast-math or
   -fveclib=libmvec may enable their use. A call to a
   function the compiler cannot see blocks vectorisation
   entirely.                                               */
```

```
/* --- 6. UNKNOWN TRIP COUNT OR COMPLEX EXIT ------------- */
        while (*p) { ... p++; }     /* length unknown */
/* Some of these can be vectorised speculatively; most are
   not.                                                    */

/* --- 7. REDUCTIONS ON FLOATING POINT ------------------- */
        for (i = 0; i < n; i++) sum += a[i];
/* FP addition is NOT ASSOCIATIVE (CA chapter 03), so
   reordering changes the result. The compiler will not do it
   by default. Options:
        -ffast-math          (dangerous - relaxes everything)
        -fassociative-math   (narrower)
        #pragma omp simd reduction(+:sum)   (explicit consent)
   INTEGER reductions vectorise freely, because integer
   addition IS associative - modulo wrapping, which is why
   unsigned is easier here too.                            */
```

ASK THE COMPILER WHY

```
gcc  -O3 -fopt-info-vec-missed
clang -O3 -Rpass-missed=loop-vectorize \
          -Rpass-analysis=loop-vectorize
```

These print the specific reason each loop failed: "loop not vectorized: cannot identify array bounds", "unsafe dependent memory operations", "value that could not be identified as reduction". That output turns vectorisation from guesswork into a checklist, and it is the single most useful flag in this chapter.

Helping the compiler

```
/* --- restrict, when it is true ------------------------- */
        void saxpy(int n, float a,
                   const float * restrict x,
                   float * restrict y)
        {
                for (int i = 0; i < n; i++)
                        y[i] = a * x[i] + y[i];
        }

/* --- tell it about alignment --------------------------- */
        float *p = __builtin_assume_aligned(ptr, 16);
        alignas(16) float buf[N];
/* An aligned vector load is faster than an unaligned one on
   many cores, and knowing the alignment lets the compiler
   skip the prologue that handles the misaligned head.   */

/* --- pragmas ------------------------------------------- */
        #pragma GCC ivdep               /* ignore assumed
                                           dependences      */
        #pragma clang loop vectorize(enable)
        #pragma clang loop vectorize_width(4) interleave_count(2)
        #pragma omp simd                /* OpenMP - portable */
        #pragma omp simd reduction(+:sum)

/* --- write loops the compiler can analyse -------------- */
        /* HARD: */
        for (p = start; p != end; p = p->next)   /* pointer
                                                    chasing */
        /* EASY: */
        for (i = 0; i < n; i++)                  /* counted,
                                                    contiguous */

        /* HARD: a loop with several exits */
        /* EASY: one entry, one exit, known trip count */

/* --- and when nothing works, intrinsics ---------------- */
#include <arm_neon.h>

void add_neon(int32_t *a, const int32_t *b,
              const int32_t *c, int n)
{
        int i;

        for (i = 0; i <= n - 4; i += 4) {
                int32x4_t vb = vld1q_s32(b + i);
                int32x4_t vc = vld1q_s32(c + i);
                int32x4_t va = vaddq_s32(vb, vc);
                vst1q_s32(a + i, va);
        }
        for (; i < n; i++)              /* the tail */
                a[i] = b[i] + c[i];
}

/* INTRINSICS ARE THE LAST RESORT: they are target-specific,
   they do not adapt to a wider vector unit, and they are
   easy to get subtly wrong. Try, in order:
     1. restructure the loop and the data layout
     2. restrict, alignment, pragmas
     3. intrinsics
     4. hand-written assembly
```

```
   and MEASURE at each step - the auto-vectoriser is
   frequently within 10% of hand-written intrinsics on simple
   kernels, and it improves with every compiler release. */
```

Reading vectorised output

```
; float sum(const float *a, int n)
;     { float s = 0; for (int i=0;i<n;i++) s += a[i];
;       return s; }
; aarch64, -O3 -ffast-math

sum:
        cmp     w1, #0
        b.le    .Lzero
        ...
        movi    v0.4s, #0               ; accumulator, 4 lanes
        movi    v1.4s, #0               ; a SECOND accumulator
.Lvector:
        ldp     q2, q3, [x0], #32       ; load 8 floats
        fadd    v0.4s, v0.4s, v2.4s     ; interleaved, to hide
        fadd    v1.4s, v1.4s, v3.4s     ; FP latency
        subs    x8, x8, #8
        b.ne    .Lvector

        fadd    v0.4s, v0.4s, v1.4s     ; combine accumulators
        faddp   v0.4s, v0.4s, v0.4s     ; horizontal reduce
        faddp   s0, v0.2s               ; ...to one scalar
        ...
.Lscalar:                               ; the epilogue, for
        ldr     s1, [x0], #4            ; the remaining
        fadd    s0, s0, s1              ; elements
        subs    w8, w8, #1
        b.ne    .Lscalar
        ret

/* THE SHAPE TO RECOGNISE:
     1. a guard for small n
     2. possibly a runtime alias check
     3. the VECTOR BODY, often unrolled with several
        accumulators to hide latency (ARM chapter 10)
     4. a HORIZONTAL REDUCTION to combine lanes
     5. a SCALAR EPILOGUE for the remainder

   `q0`, `v0.4s`, `faddp` - the presence of these means it
   vectorised. All-scalar `s0`, `w0` means it did not.

   AND NOTE `-ffast-math` was required for this one, because
   the reduction reorders FP additions.                  */
```

How it shows up

Symptom	Cause
Loop not vectorised, no obvious reason	Ask: `-fopt-info-vec-missed`
"unsafe dependent memory operations"	Possible aliasing — try `restrict`
Two copies of the loop in the assembly	Runtime alias check — `restrict` removes it
Vectorises for `float` but not the reduction	FP associativity — needs explicit consent

Symptom	Cause
Vectorised but no faster	Memory bandwidth bound, not compute bound
Slower after `-O3`	Code growth from unrolling; I-cache pressure
Array-of-structs loop will not vectorise	Strided access — restructure to struct-of-arrays
Cache misses dominate	Loop order — interchange so the innermost stride is 1
Hand-written copy loop no faster than `memcpy`	It became `memcpy`
Results differ with `-ffast-math`	Expected — it permits reassociation (CA ch. 03)
Intrinsics beaten by autovectorisation	Common. Measure before hand-writing

Check yourself

- ✓ Name eight loop transformations and what each buys.
- ✓ Explain why loop rotation happens before other loop passes.
- ✓ Explain why loop interchange is often the largest available win.
- ✓ Explain idiom recognition and its consequence for freestanding builds.
- ✓ Describe NEON's element arrangements and how SVE differs.
- ✓ List seven reasons a loop fails to vectorise, with a fix for each.
- ✓ Use `-fopt-info-vec-missed` on a real loop and act on the output.
- ✓ Explain what `restrict` promises and the cost of lying.
- ✓ Convert an array-of-structures layout to structure-of-arrays and explain why it helps.
- ✓ Recognise the five parts of a vectorised loop in assembly.

Instruction selection and scheduling

The back end's first two jobs: choosing which machine instructions implement the IR, and deciding what order to issue them in so the pipeline stays busy. This is where target knowledge finally enters the compiler.

Instruction selection

```
The IR says WHAT to compute. Instruction selection decides
WHICH instructions do it - and there are usually several
answers.

   IR:      %1 = mul i32 %a, 4
            %2 = add i32 %b, %1

   CANDIDATES on AArch64:

     (a)  lsl  w1, w0, #2          ; a * 4
          add  w2, w3, w1
          -> 2 instructions

     (b)  add  w2, w3, w0, lsl #2  ; ONE instruction:
          -> the shifted-register addressing form

   ARM's instruction set has many such fused forms, so
   selection matters more than on a simpler ISA:

     add  x0, x1, x2, lsl #3       shifted operand
     madd w0, w1, w2, w3           multiply-add
     msub w0, w1, w2, w3           multiply-subtract
     ubfx w0, w1, #4, #8           bitfield extract
     bfi  w0, w1, #4, #8           bitfield insert
     csel w0, w1, w2, eq           conditional select
     csinc, csneg, csinv           select-and-modify
     ldp/stp  x0, x1, [x2]         load/store PAIR
     ldr  w0, [x1, w2, sxtw #2]    scaled, sign-extended
                                   index

   RECOGNISING THESE IS THE JOB. A pattern-matching selector
   covers the IR graph with the cheapest set of patterns that
   tile it.
```

Tree tiling

```
/* The classic formulation: the IR is a tree (or DAG); each
   instruction is a PATTERN with a COST; find the cheapest
   cover.                                                    */

   IR TREE for  a[i] = b + 1 :

              store
             /     \
          add       add
         /   \     /   \
       a    shl   b     1
           /   \
          i     2

   PATTERNS available:
      "add reg, reg"                    cost 1
      "add reg, reg, lsl #n"            cost 1   <- covers 3
                                                    nodes
      "str reg, [reg, reg, lsl #n]"     cost 1   <- covers 5
                                                    nodes!
      "lsl reg, reg, #n"                cost 1

   THE GREEDY COVER using the third pattern:
      add  w1, w2, #1              ; b + 1
      str  w1, [x0, x3, lsl #2]    ; a[i] = ...
   TWO instructions for eight IR nodes.

/* THE ALGORITHMS:
     · maximal munch - greedily take the largest matching
       pattern at each node. Simple, near-optimal in practice.
     · dynamic programming (BURG, iburg) - provably optimal
       tiling for a tree, in linear time
     · DAG-based selection (LLVM's SelectionDAG) - handles
       shared subexpressions
     · GlobalISel (LLVM's newer path) - works on the whole
       function, better for cross-block patterns and faster
       to compile

/* IN LLVM you write patterns in TableGen: */
   def : Pat<(add GPR32:$a, (shl GPR32:$b, imm:$c)),
             (ADDWrs GPR32:$a, GPR32:$b, imm:$c)>;
/* and the build generates the matcher. AArch64's target
   description is roughly 40,000 lines of this - which is a
   measure of how much of a compiler is target knowledge. */
```

Peephole optimisation

```
/* After selection, a sliding window over the instruction
   stream replaces short sequences with better ones.       */

        mov  w0, w0                     ->  (delete)
        add  w0, w0, #0                 ->  (delete)

        str  w0, [x1]                   ->  str w0, [x1]
        ldr  w0, [x1]                       (the load is
                                             redundant)

        mov  w1, #0                     ->  str wzr, [x0]
        str  w1, [x0]                       (use the zero
                                             register)

        cmp  w0, #0                     ->  cbz w0, .L1
        b.eq .L1                            (compare-and-
                                             branch)

        sub  w0, w0, #1                 ->  subs w0, w0, #1
        cmp  w0, #0                         b.ne .L1
        b.ne .L1                            (subs sets the
                                             flags)

        ldr  w0, [x1]                   ->  ldp w0, w1, [x1]
        ldr  w1, [x1, #4]                   (paired load)

/* ARM-SPECIFIC ONES worth recognising in output:
     wzr / xzr        the ZERO REGISTER - reading gives 0,
                      writing discards. `mov w0, wzr` is how
                      you zero a register, and `str wzr, [x0]`
                      stores zero with no register needed.
     cbz / cbnz       compare-with-zero and branch, one
                      instruction
     tbz / tbnz       test a single BIT and branch
     ldp / stp        load/store pair - half the instructions
                      for prologues and epilogues
     ret              a specialised `br x30`                */

/* THE PEEPHOLE PASS IS WHY hand-optimised assembly rarely
   beats the compiler on straight-line code: every one of
   these substitutions is applied automatically and
   exhaustively.                                           */
```

Instruction scheduling

```
The processor is pipelined (CA chapter 11). An instruction
that needs a result not yet available STALLS.

   POORLY SCHEDULED:
      ldr  w0, [x1]        ; 4-cycle load latency
      add  w0, w0, #1      ; STALLS 3 cycles waiting for w0
      ldr  w2, [x3]
      add  w2, w2, #1      ; STALLS again
      -> ~14 cycles

   SCHEDULED:
      ldr  w0, [x1]
      ldr  w2, [x3]        ; issue the second load while the
                           ; first is in flight
      add  w0, w0, #1      ; by now w0 is ready
      add  w2, w2, #1
      -> ~8 cycles

  SAME INSTRUCTIONS, same result, fewer stalls.

  THE SCHEDULER NEEDS A MACHINE MODEL:
     each instruction's LATENCY (when its result is ready)
     each instruction's THROUGHPUT (how often it can issue)
     which FUNCTIONAL UNITS it uses, and how many there are
     the issue WIDTH
     forwarding paths

  LLVM's is in the target's TableGen; you can see it:
     llvm-mca -mcpu=cortex-a72 input.s
  which prints a cycle-by-cycle simulation, the bottleneck
  resource, and the achieved IPC. It is the best tool for
  "why is this loop slower than I expect".
```

```
/* LIST SCHEDULING, the standard algorithm:

   1. build a DEPENDENCE DAG for the basic block
        true dependence   (read-after-write) - real
        anti-dependence   (write-after-read) - removable by
                          renaming
        output dependence (write-after-write) - ditto
        memory dependences, where aliasing is unclear

   2. compute each node's PRIORITY - usually the critical
      path length to the end of the block

   3. repeatedly: from the instructions whose operands are
      ready, issue the highest priority one

   THE TENSION WITH REGISTER ALLOCATION (chapter 09):
   spreading instructions apart to hide latency INCREASES the
   number of simultaneously live values, which increases
   register pressure, which may cause spills that cost more
   than the stalls saved. Compilers schedule before and after
   allocation for this reason, and the pre-allocation
   scheduler is deliberately register-pressure-aware.     */

/* WHY IT MATTERS LESS THAN IT USED TO:
   an out-of-order core (CA chapter 12) reorders instructions
   itself, within its window. So scheduling matters most on:
     · in-order cores - Cortex-A53, A55, most microcontrollers
     · very long dependence chains that exceed the window
     · code where the compiler can see further ahead than the
       hardware can

   ON A CORTEX-A53 (in-order) good scheduling can be worth
   20-30%. On a Cortex-X (aggressively out-of-order) it is
   often under 5%. -mtune= tells the compiler which model to
   use, and it is worth setting correctly even when -march
   must stay generic.                                     */
```

`-MARCH` VERSUS `-MTUNE`

`-march` says which instructions may be *emitted* — get it wrong and you get an illegal instruction on older hardware. `-mtune` says which core to *optimise for* — scheduling, inlining thresholds, alignment. Tuning for the wrong core costs performance; targeting the wrong architecture costs correctness. For a product shipping on one SoC, set both: `-mcpu=cortex-a53` sets them together.

Software pipelining

```
/* Overlap iterations, so the loop body issues work from
   several iterations at once and no stage waits.          */

   NAIVE:  for each i:  load a[i] ; compute ; store b[i]

           |load|      |          |
           |    |compute|         |
           |    |      |store|    |
                             |load|      |
   -> the load latency is exposed every iteration.

   PIPELINED:
           |load i+2 | compute i+1 | store i |
           |load i+3 | compute i+2 | store i+1|
   -> each iteration issues a load for a FUTURE element, a
      compute for the previous one, and a store for the one
      before that. The pipeline is always full.

   REQUIRES a PROLOGUE to fill the pipeline and an EPILOGUE to
   drain it, which is why software-pipelined loops look
   complicated in the assembly.

/* GCC: -fmodulo-sched, and it is not on by default at -O2
   because it increases register pressure and code size.
   LLVM does it for some targets via the MachinePipeliner.

/* YOU CAN DO IT BY HAND, and on an in-order DSP-like core it
   is sometimes worth it: */
        t = a[0];
        for (i = 0; i < n - 1; i++) {
                u = a[i + 1];           /* load for NEXT */
                b[i] = f(t);            /* compute for THIS */
                t = u;
        }
        b[n-1] = f(t);
/* But measure first. On an out-of-order core the hardware
   already does this.                                      */
```

Reading the output

```
; int dot(const int *a, const int *b, int n)
; aarch64 -O2 -mcpu=cortex-a53

dot:
        cmp     w2, #0
        b.le    .Lzero
        mov     x3, #0                  ; i
        mov     w4, #0                  ; sum
.Lloop:
        ldr     w5, [x0, x3, lsl #2]    ; a[i] - SCALED INDEX
        ldr     w6, [x1, x3, lsl #2]    ; b[i]
        add     x3, x3, #1              ; i++  - issued while
                                        ; the loads are in
                                        ; flight
        cmp     x3, x2, sxtw
        madd    w4, w5, w6, w4          ; sum += a[i]*b[i]
                                        ; ONE instruction
        b.lt    .Lloop
        mov     w0, w4
        ret
.Lzero:
        mov     w0, #0
        ret

/* WHAT TO NOTICE:

   `[x0, x3, lsl #2]`   the index scaling is FREE, folded
                        into the addressing mode. No separate
                        multiply or shift.
   `madd`               multiply-accumulate in one
                        instruction - selection recognised the
                        pattern.
   `add x3, x3, #1` placed BETWEEN the loads and their use -
                        the scheduler hiding load latency.
   `sxtw`               sign-extend w2 to 64 bits, folded into
                        the compare.
   `cmp` then `b.lt`    the flags are set once and used once.

   FIVE INSTRUCTIONS in the loop body for a load, a load, a
   multiply, an add, an index update and a branch. That is
   what good selection plus scheduling looks like, and it is
   why hand-writing this in assembly is unlikely to help. */

# --- ANALYSE IT ---
    llvm-mca -mcpu=cortex-a53 -timeline dot.s
    ; cycle-by-cycle, with the bottleneck identified

    perf stat -e cycles,instructions,stalled-cycles-frontend ./prog
    perf annotate                ; per-instruction cost
```

How it shows up

Symptom	Cause
Illegal instruction on target	`-march` allowed something the CPU lacks
Slower than expected on an in-order core	Wrong `-mtune`; the schedule targets a different pipeline
Many separate shifts and adds for an index	The addressing mode was not matched — often an unusual element size

Symptom	Cause
Multiply and add not fused	Rounding rules for FP, or the intermediate value is used elsewhere
Long stalls on loads	Dependence chain the scheduler could not break — or a real cache miss
Loop faster with manual reordering	Register pressure prevented the scheduler from doing it
Assembly full of prologue/epilogue code	Software pipelining or vectorisation — normal
Hand-written assembly no faster	Peephole plus selection already found those patterns
IPC far below the issue width	Run `llvm-mca` — it names the bottleneck resource

Check yourself

- ✓ Explain instruction selection and give three ARM fused forms it must recognise.
- ✓ Tile a small IR tree and count the instructions saved.
- ✓ Name four peephole substitutions and the ARM features they exploit.
- ✓ Explain what the zero register buys.
- ✓ Explain why scheduling matters and what a machine model must contain.
- ✓ Describe list scheduling and its tension with register allocation.
- ✓ Say when scheduling matters most and least, and why.
- ✓ Distinguish `-march` from `-mtune` and the failure each wrong choice causes.
- ✓ Explain software pipelining and why it needs a prologue and epilogue.
- ✓ Read a compiled loop and identify selection and scheduling decisions in it.

Register allocation

The IR has unlimited virtual registers; the machine has 31. Deciding which values live in registers and which get pushed to memory is an NP-complete problem the compiler solves thousands of times a second — and it explains why your variable "does not exist" in the debugger.

The problem

```
AFTER instruction selection:

   %1 = ldr [%a]
   %2 = ldr [%b]
   %3 = add %1, %2
   %4 = ldr [%c]
   %5 = mul %3, %4
   str [%d], %5

Six virtual registers. AArch64 has:

   x0–x7     arguments and results     CALLER-saved
   x8        indirect result location
   x9–x15    temporaries               CALLER-saved
   x16, x17  IP0, IP1 – linker veneers
   x18       PLATFORM register (TLS on some ABIs;
             RESERVED – never use it)
   x19–x28   CALLEE-saved
   x29       frame pointer
   x30       link register
   sp/xzr    stack pointer / zero register

   v0–v31    FP/SIMD; v8–v15's LOW 64 BITS are callee-saved

So roughly 29 usable general-purpose registers – generous
by historical standards, and still not unlimited.

THE JOB: map virtual registers to physical ones, such that
two values that are LIVE AT THE SAME TIME never share a
register. When that is impossible, SPILL one to the stack.
```

Liveness

```
/* A value is LIVE from its definition until its LAST USE. */

        a = 1;          /* a live from here            */
        b = 2;          /* b live                      */
        c = a + b;      /* a and b DIE here; c is born */
        d = c * 2;      /* c dies                      */
        return d;       /* d dies                      */

   LIVE RANGES:
        a:  |---|
        b:   |--|
        c:      |--|
        d:         |--|

   MAXIMUM SIMULTANEOUSLY LIVE = 2. Two registers suffice.

/* COMPUTED BY BACKWARD DATAFLOW over the CFG: */

        live_out(B) = union of live_in(S) for successors S
        live_in(B)  = used(B) ∪ (live_out(B) − defined(B))

/* Iterate to a fixed point. In SSA (chapter 05) this is
   cheaper, because each value has exactly one definition and
   the live range is the region dominated by it that reaches a
   use.                                                    */

/* REGISTER PRESSURE at a point = the number of values live
   there. Where pressure exceeds the register count, a spill
   is unavoidable. THAT is what the allocator is measuring,
   and it is why:
     · long-lived variables cost more than short-lived ones
     · a value used once far from its definition is expensive
     · calls are pressure points, because they clobber every
       caller-saved register                              */
```

Graph colouring

```
Chaitin's insight (1981): this is GRAPH COLOURING.

  Build an INTERFERENCE GRAPH: a node per value, an edge
  between any two values live at the same time.

     a --- b
      \   /
       \ /
        c --- d

  Colour the graph with K colours (K = number of registers)
  such that no two adjacent nodes share a colour.

  With K = 2:  a=red, b=blue, c=red, d=blue.  Done.

  THE ALGORITHM (Chaitin-Briggs):

   1. BUILD the interference graph.
   2. SIMPLIFY: repeatedly remove any node with FEWER THAN K
      neighbours, pushing it on a stack. Such a node can
      always be coloured later, whatever its neighbours get.
   3. If only nodes with >= K neighbours remain, SPILL one -
      choose by a heuristic: highest degree, lowest use
      count, not inside a loop.
   4. When the graph is empty, POP the stack and assign each
      node a colour differing from its already-coloured
      neighbours.
   5. If a spill was chosen, rewrite the code with store/load
      around it and START AGAIN - the new loads and stores
      create new, very short live ranges.

  GRAPH COLOURING IS NP-COMPLETE, so this is a heuristic. It
  is also why compilation time can jump on functions with
  enormous basic blocks - the graph gets large.

  THE OTHER ALGORITHM you will meet: LINEAR SCAN. Sort live
  intervals by start point, sweep, assign the first free
  register, spill the interval ending latest. Far faster,
  slightly worse code. LLVM's -O0 uses "fast" allocation;
  -O2 uses a greedy allocator that is closer to colouring in
  quality and closer to linear scan in speed.
```

Spilling

```
/* When a value cannot get a register, it lives on the STACK,
   loaded before each use and stored after each definition. */

/* WITHOUT spilling: */
        add  w0, w1, w2

/* WITH the result spilled: */
        add  w0, w1, w2
        str  w0, [sp, #16]              ; SPILL
        ...
        ldr  w0, [sp, #16]              ; RELOAD
        mul  w3, w0, w4

/* THE COST: two memory accesses. L1 hits, so a few cycles
   each - but in a hot loop, per iteration, it adds up
   quickly.

/* HOW THE ALLOCATOR CHOOSES what to spill:
        spill cost ≈ (loads + stores) × 10^(loop depth)
                     ──────────────────────────────────
                                 degree
   So a value used once outside a loop is cheap to spill; one
   used every iteration of a triple-nested loop is not.

/* SPILL SLOTS live in the stack frame, which is why a
   register-pressured function has a large frame - and why
   -Wframe-larger-than can fire on code with no large locals
   at all (Linux chapter 02).

/* WHAT CAUSES PRESSURE, in order of how often it is the
   cause:
     1. too many live variables - long functions doing many
        things at once
     2. aggressive inlining, which merges several functions'
        pressure into one
     3. unrolling and vectorisation, which multiply the live
        values per iteration
     4. large structs passed or returned by value
     5. many function arguments (more than 8 on AArch64 go on
        the stack anyway)
     6. taking the address of locals, which prevents promotion
        entirely (chapter 05)

/* AND THE FIX, in the same order: split the function. A
   200-line function with 40 live variables will spill; four
   50-line functions will not, and the compiler will inline
   them back if it helps.                                  */
```

> **"OPTIMIZED OUT" IN THE DEBUGGER**
>
> When gdb says `<optimized out>`, the variable has no single location — it lives in a register for part of its range, in a different register later, on the stack briefly, and nowhere at all where the compiler proved it unnecessary. DWARF location lists (chapter 14) can describe this, and modern compilers emit them, but not for values that genuinely no longer exist. Build with `-Og` when you need to inspect everything.

Calling conventions constrain it

```
/* The ABI (chapter 10) divides registers into two classes,
   and the allocator must respect it. */

   CALLER-SAVED (volatile): x0-x18, v0-v7, v16-v31
        A call may destroy them. If a value must survive a
        call, the CALLER saves it - or the allocator puts it
        in a callee-saved register instead.

   CALLEE-SAVED (non-volatile): x19-x28, v8-v15 (low 64 bits)
        A function using one must save and restore it in its
        prologue and epilogue.

/* THE ALLOCATION STRATEGY that follows:

     a value NOT live across any call
          -> prefer a CALLER-saved register: free, no
             prologue cost

     a value live ACROSS a call
          -> prefer a CALLEE-saved register: pay once in the
             prologue rather than at every call site       */

/* WHICH IS WHY a leaf function looks like this: */
        f:
                add   w0, w0, #1
                ret
/* no prologue at all - it uses only caller-saved registers
   and calls nothing.

/* AND A FUNCTION WITH A CALL IN A LOOP looks like this: */
        g:
                stp   x29, x30, [sp, #-32]!    ; FP and LR
                mov   x29, sp
                stp   x19, x20, [sp, #16]      ; callee-saved
                mov   w19, w0                  ; keep the arg
                                               ; across calls
        .Lloop:
                bl    helper                   ; clobbers
                                               ; x0-x18
                add   w19, w19, w0             ; w19 SURVIVED
                ...
                ldp   x19, x20, [sp, #16]
                ldp   x29, x30, [sp], #32
                ret

/* THE `stp`/`ldp` PAIRS are the prologue and epilogue, and
   their size tells you immediately how many callee-saved
   registers the function needed - which is a proxy for how
   much simultaneous state it carries.                  */
```

Coalescing and rematerialisation

```
/* --- COALESCING: eliminate copies ----------------------- */

/* Phi elimination (chapter 05) inserts copies: */
        mov  w1, w0
        ...uses of w1...

/* If w0 and w1 do NOT interfere, give them the same register
   and delete the move. That is coalescing, and it is why
   phi nodes usually cost nothing.

   THE TENSION: coalescing MERGES two nodes in the
   interference graph, and the merged node has the union of
   both neighbour sets - a higher degree, which may make the
   graph uncolourable and force a spill. So aggressive
   coalescing can be worse than none.

   CONSERVATIVE COALESCING (Briggs): only merge if the result
   has fewer than K neighbours of significant degree. This is
   what production allocators use.                          */

/* --- REMATERIALISATION: recompute instead of reload ----- */

/* Spilling a constant is silly: */
        mov  w0, #42
        str  w0, [sp, #8]       ; spill
        ...
        ldr  w0, [sp, #8]       ; reload

/* Just recompute it: */
        mov  w0, #42
        ...
        mov  w0, #42            ; REMATERIALISED - one
                                ; instruction, no memory

/* REMATERIALISABLE VALUES: constants, addresses of globals,
   the frame pointer plus an offset, and anything computed
   from values that are still live. The allocator prefers
   rematerialisation to spilling whenever it applies, which
   is why constants almost never occupy a register across a
   call.                                                    */

/* --- LIVE RANGE SPLITTING ------------------------------ */
/* Instead of spilling a value for its whole range, split the
   range at the pressure point:                             */
        v is live from A to Z
        pressure is high only between M and N
   ->   keep v in a register A..M
        spill it M..N
        reload into a (possibly different) register N..Z
/* This is what LLVM's greedy allocator does, and it is the
   main reason its output beats a naive colouring
   allocator.                                               */
```

Seeing the pressure

```
# --- How big is the frame? Spills live there. ---
    objdump -d prog | grep -A2 '<myfunc>:'
    ; look at the first `sub sp, sp, #N` or `stp ... [sp,#-N]!`

    gcc -fstack-usage prog.c
    ; writes prog.su:  prog.c:10:5:myfunc  128  static

    gcc -Wstack-usage=256
    gcc -Wframe-larger-than=1024      ; the kernel sets this

# --- Count the spills directly ---
    llvm: -mllvm -stats 2>&1 | grep -i spill
      12 regalloc - Number of spill slots allocated
      34 regalloc - Number of spills inserted
      41 regalloc - Number of reloads inserted

    gcc: -fdump-rtl-ira -fdump-rtl-reload

# --- In the assembly, look for str/ldr to [sp, #N] ---
# that are not obviously part of the ABI. A loop body with
# stack traffic in it is spilling.

# --- Reduce pressure, in order of effectiveness ---
#   1. split the function
#   2. shorten variable lifetimes - declare at first use,
#      not at the top
#   3. avoid taking the address of locals
#   4. pass large structs by pointer, not by value
#   5. reduce unrolling: #pragma clang loop unroll(disable)
#   6. reduce inlining: __attribute__((noinline)) on the
#      callee, or -finline-limit=
#   7. on FP-heavy code, check whether v8-v15 spilling is the
#      issue - those are callee-saved and expensive

# THE FIRST ONE IS ALMOST ALWAYS THE ANSWER. Register
# pressure is a symptom of a function doing too much.
```

How it shows up

Symptom	Cause
`<optimized out>` in gdb	The value has no single home — use `-Og`
Loop body contains `str` / `ldr` to the stack	Spilling — reduce pressure
Large stack frame with no large locals	Spill slots
`-Wframe-larger-than` fires unexpectedly	Same
Performance drops after inlining	Merged pressure caused spills
Performance drops after unrolling	More live values per iteration
A function got much slower after adding one variable	It crossed the pressure threshold
Compile time spikes on one function	A huge basic block; the interference graph is large
Long prologue with many `stp` pairs	Many callee-saved registers needed

Symptom	Cause
Constants reloaded from the stack	Rematerialisation did not apply — often an address that is not statically known
Code uses `x18` and crashes on some platforms	It is reserved — never allocate it by hand

Check yourself

- ✓ State AArch64's register classes and which are callee-saved.
- ✓ Compute live ranges for a straight-line sequence and the maximum pressure.
- ✓ Explain graph colouring allocation and the simplify/spill/select loop.
- ✓ Explain why the algorithm restarts after inserting spill code.
- ✓ Contrast graph colouring with linear scan.
- ✓ Give the spill cost heuristic and say why loop depth dominates it.
- ✓ List six causes of register pressure and the fix for each.
- ✓ Explain why a value live across a call prefers a callee-saved register.
- ✓ Explain coalescing and why aggressive coalescing can hurt.
- ✓ Explain rematerialisation and when it applies.
- ✓ Measure spills in a real function and reduce them.

Calling conventions and the ABI

The contract that lets separately compiled code work together: where arguments go, who saves what, how the stack is laid out. Violate it and you get corruption that looks like a hardware fault — and every language's foreign function interface is built on it.

What an ABI covers

```
An API is a source-level contract: function names, types,
semantics. An ABI is a BINARY contract:

  · sizes and alignments of every type
  · struct layout and padding rules
  · where arguments are passed, and in what order
  · where return values come back
  · which registers are preserved across a call
  · stack frame layout, alignment, and direction
  · the stack unwinding format, for exceptions
  · name mangling (C++)
  · object file format, relocations, symbol visibility
  · dynamic linking: PLT, GOT, TLS model
  · system call numbering and convention

THE RELEVANT DOCUMENT for ARM 64-bit is:
  "Procedure Call Standard for the Arm 64-bit Architecture"
  (AAPCS64), plus the ELF ABI supplement.

TWO OBJECT FILES AGREE ON AN ABI OR THEY DO NOT WORK. This
is why you cannot link an armhf library into an aarch64
binary, why C++ objects from different compilers may not
link, and why the kernel's syscall interface is frozen
forever (Linux chapter 02).
```

AAPCS64: the register map

Register	Role	Preserved?
`x0` – `x7`	Arguments 1–8; `x0` (– `x1`) returns the result	No
`x8`	Indirect result location (for large returns)	No
`x9` – `x15`	Temporaries	No
`x16`, `x17`	IP0/IP1 — the linker may clobber these in veneers	No
`x18`	**Platform register** — reserved. TLS on Windows/Darwin, shadow call stack on some Linux configs	Do not touch
`x19` – `x28`	General purpose	**Yes**
`x29`	Frame pointer	**Yes**
`x30`	Link register — the return address	**Yes**
`sp`	Stack pointer, **16-byte aligned**	Yes
`v0` – `v7`	FP/SIMD arguments and results	No
`v8` – `v15`	FP/SIMD	**Low 64 bits only**
`v16` – `v31`	FP/SIMD temporaries	No

```
/* THE v8-v15 RULE IS A REAL TRAP: only the LOW 64 BITS are
   callee-saved. A function that uses the full 128-bit v8 and
   restores only the low half has corrupted its caller's
   upper half. Hand-written SIMD assembly must save the whole
   register if it uses the whole register.               */

/* AND x18: on Linux it is usually free, but on Windows on
   ARM and on Darwin it holds the TEB/TLS pointer, and with
   -fsanitize=shadow-call-stack it holds the shadow stack
   pointer. Compile with -ffixed-x18 if you are writing code
   that must be portable across those. Never allocate it in
   hand-written assembly.                                 */
```

Argument passing

```
/* THE ALGORITHM, simplified but accurate for the common
   cases:

   NGRN = next general register number (starts at 0)
   NSRN = next SIMD register number (starts at 0)
   NSAA = next stacked argument address

   For each argument, in order:
     · integer or pointer, fits in 8 bytes
           -> x[NGRN++], if NGRN < 8; else stack
     · 16-byte integer (__int128)
           -> an even-numbered register PAIR
     · float, double, or a short vector
           -> v[NSRN++], if NSRN < 8; else stack
     · HFA/HVA - a struct of 1-4 identical float/vector
       members
           -> consecutive v registers, one per member
     · other struct, <= 16 bytes
           -> passed in 1 or 2 general registers, by value
     · other struct, > 16 bytes
           -> a COPY is made, and its ADDRESS is passed in a
              register. The callee may modify the copy.
     · variadic arguments after the named ones
           -> on the stack, on most platforms (Darwin differs)
*/

/* WORKED EXAMPLE: */
        struct Small { int a, b; };            /* 8 bytes  */
        struct Big   { int a[10]; };           /* 40 bytes */
        struct Vec3  { float x, y, z; };       /* an HFA    */

        void f(int a, double b, struct Small c,
               struct Big d, struct Vec3 e, int f);

/*      a -> w0
        b -> d0
        c -> x1              (8 bytes, packed into one reg)
        d -> x2 = &copy_of_d (too large - by reference)
        e -> s1, s2, s3      (HFA - one register per member)
        f -> w3                                          */

/* THE HFA RULE IS WHY a 3-float vector type is efficient on
   ARM and a 3-int one is not: floats get three registers,
   ints get packed and spilled. It is also why graphics and
   DSP code on ARM tends to use float structs.           */

/* VARIADIC FUNCTIONS: the named arguments follow the normal
   rules, everything after goes on the stack in a defined
   layout, and va_arg walks it. THIS IS WHY:
     · printf("%d", someFloat) is UB - the float was passed in
       a v register or promoted to double on the stack, and
       %d reads an integer slot
     · a function must have a prototype in scope: without it,
       the compiler cannot know whether an argument is named
       or variadic                                        */
```

Return values

```
        int f(void);                    -> w0
        long f(void);                   -> x0
        void *f(void);                  -> x0
        float f(void);                  -> s0
        double f(void);                 -> d0
        __int128 f(void);               -> x0, x1
        struct Small f(void);           -> x0 (and x1 if 9–16
                                           bytes)
        struct Vec3 f(void);            -> s0, s1, s2 (HFA)

        struct Big f(void);
/* THE CALLER allocates space, passes its address in x8, and
   the callee writes the result there. So:            */

        struct Big result = f();
   compiles to:
        add   x8, sp, #16          ; where to put it
        bl    f
        ; f writes 40 bytes at [x8]

/* WHICH IS WHY returning a large struct by value is not as
   expensive as people assume — there is one copy, into a
   destination the caller chose, not two. And why NAMED RETURN
   VALUE OPTIMISATION matters in C++ (C++ chapter 05): if the
   callee builds the result directly at [x8], there is no copy
   at all.                                                */
```

The stack frame

```
higher addresses
+---------------------------+
|  caller's frame           |
+---------------------------+  <- sp on entry, 16-aligned
|  stacked arguments        |     (arguments 9+)
+---------------------------+
|  x29 (saved FP)           |  <- x29 points HERE
|  x30 (saved LR)           |
+---------------------------+
|  other callee-saved regs  |
|  x19-x28, v8-v15          |
+---------------------------+
|  local variables          |
|  spill slots (chapter 09) |
+---------------------------+
|  outgoing arguments for   |
|  calls this function makes|
+---------------------------+  <- sp, always 16-aligned
lower addresses

THE STANDARD PROLOGUE:
     stp   x29, x30, [sp, #-48]!   ; push FP and LR,
                                   ; allocate 48 bytes
     mov   x29, sp                 ; establish the frame
     stp   x19, x20, [sp, #16]     ; save callee-saved
     str   x21, [sp, #32]

THE EPILOGUE, exactly reversed:
     ldr   x21, [sp, #32]
     ldp   x19, x20, [sp, #16]
     ldp   x29, x30, [sp], #48     ; pop and deallocate
     ret                           ; = br x30

THE FRAME POINTER CHAIN: each x29 points at the previous
x29, forming a linked list. That is how a backtrace works
without debug information - walk the chain, read the saved
x30 at each level. -fomit-frame-pointer breaks it, which is
why profilers and the kernel build with
-fno-omit-frame-pointer.

16-BYTE ALIGNMENT IS MANDATORY at every public interface.
Misalign the stack and you get a fault the first time
something executes a 128-bit SIMD load/store relative to
sp - typically deep inside memcpy, far from the cause.
```

Type layout

```
/* SIZES AND ALIGNMENTS on AArch64 Linux (LP64): */

        char            1    align 1
        short           2    align 2
        int             4    align 4
        long            8    align 8      <- 4 on Windows!
        long long       8    align 8
        pointer         8    align 8
        float           4    align 4
        double          8    align 8
        long double    16    align 16     <- 8 on some ABIs
        bool            1    align 1

/* THE `long` DIFFERENCE is the classic portability bug:
     LP64  (Linux, macOS):  long = 8
     LLP64 (Windows):       long = 4
   Use int64_t/uint64_t/intptr_t and the question does not
   arise.                                                  */

/* STRUCT LAYOUT: members in declaration order, each at the
   next offset satisfying its alignment; the struct's
   alignment is the maximum of its members'; the size is
   rounded up to that.                                     */

        struct A {              /* 24 bytes */
                char  a;        /* offset 0            */
                                /* 7 bytes PADDING     */
                long  b;        /* offset 8            */
                char  c;        /* offset 16           */
                                /* 7 bytes TAIL PADDING*/
        };

        struct B {              /* 16 bytes - REORDERED */
                long  b;        /* offset 0            */
                char  a;        /* offset 8            */
                char  c;        /* offset 9            */
                                /* 6 bytes padding     */
        };

/* SORTING MEMBERS LARGEST-FIRST is the simplest layout
   optimisation available, and it costs nothing. On an array
   of a million elements, 24 versus 16 bytes is 8 MB and a
   third more cache lines (CA chapter 14).

   SEE THE LAYOUT:
        pahole -C A prog.o
        clang -Xclang -fdump-record-layouts -c prog.c        */

/* PACKING, and why it is dangerous: */
        struct __attribute__((packed)) Wire {
                uint8_t  type;
                uint32_t value;         /* offset 1 -
                                           MISALIGNED */
        };
/* On ARM, an unaligned access to Normal memory works but may
   be slower; to DEVICE memory it FAULTS (ARM chapter 11).
   And taking a POINTER to a packed member and dereferencing
   it is undefined behaviour - the compiler warns with
   -Waddress-of-packed-member for exactly this reason.
```

```
   FOR WIRE FORMATS, prefer explicit serialisation:      */
        static void encode(uint8_t *buf, const struct Msg *m)
        {
                buf[0] = m->type;
                put_le32(buf + 1, m->value);
        }
/* Slower to write, correct everywhere, and endian-explicit. */
```

THE C++ ADDITIONS

C++ layers name mangling (so overloads and templates get distinct symbols), vtable layout, RTTI representation, exception unwinding tables, and the rules for empty base optimisation on top of the C ABI. The Itanium C++ ABI is what GCC and Clang both implement on Linux and ARM, which is why their objects interoperate — and MSVC's differs entirely, which is why they do not on Windows.

Inline assembly and the ABI

```
/* Writing assembly means obeying the ABI yourself. */

static inline uint64_t read_cntvct(void)
{
        uint64_t val;

        __asm__ volatile("mrs %0, cntvct_el0"
                         : "=r" (val)     /* output */
                         :                /* no inputs */
                         : "memory");     /* clobbers */
        return val;
}

/* THE CONSTRAINT LETTERS you need on AArch64:
     "r"    a general register
     "w"    a SIMD/FP register
     "m"    a memory operand
     "i"    an immediate constant
     "I"    a 12-bit immediate valid for add/sub
     "0"    same as operand 0 (tied)
     "+r"   read AND written
     "=r"   write only

/* THE CLOBBER LIST is where mistakes happen:
     · list EVERY register you modify that is not an output
     · "memory" means "this may read or write memory the
       compiler does not know about" - it prevents the
       compiler from caching loads across the asm and from
       reordering memory operations around it
     · "cc" means the condition flags are modified

/* FORGETTING A CLOBBER produces the worst class of bug: the
   compiler keeps a value in a register you overwrote, and the
   corruption appears somewhere unrelated, only at -O2, only
   sometimes.

/* AND `volatile` on the asm means "do not delete this even if
   its outputs are unused, and do not move it". Required for
   anything with a side effect - an MMIO access, a barrier, a
   system register write.                                  */

/* A COMPLETE EXAMPLE with input, output and clobbers: */
static inline void write_ttbr0(uint64_t val)
{
        __asm__ volatile("msr ttbr0_el1, %0\n"
                         "isb"
                         :
                         : "r" (val)
                         : "memory");
}

/* FOR A WHOLE FUNCTION IN ASSEMBLY, the ABI is entirely
   yours: save x19-x28 if you use them, keep sp 16-aligned,
   return through x30, put the result in x0. Get any of it
   wrong and the failure appears in the CALLER.          */
```

How it shows up

Symptom	Cause
Corruption in the caller after a call	Callee-saved register not restored — often hand-written assembly
Corruption only in SIMD code	Only the low 64 bits of `v8`–`v15` are preserved
Fault inside `memcpy` with a sane pointer	Stack not 16-byte aligned
Works at `-O0`, corrupts at `-O2`	Missing clobber in inline assembly
Wrong value from `printf`	Argument type does not match the format — variadic slots
Struct different size on two platforms	`long`, or different alignment rules
Struct larger than the sum of its members	Padding — reorder largest first
Unaligned access fault	A packed struct member, or a cast to a wider type
Backtrace stops after one frame	`-fomit-frame-pointer`, or hand-written assembly with no frame
C++ objects will not link across compilers	Different C++ ABI
Crash only on Windows/Darwin ARM	Used `x18`
Passing a 3-float struct is fast, 3-int is not	The HFA rule

Check yourself

- ✓ List what an ABI specifies beyond an API.
- ✓ State AArch64's register roles from memory, including `x8` and `x18`.
- ✓ Explain the `v8`–`v15` partial-preservation rule and the bug it causes.
- ✓ Work out where each argument of a mixed-type function is passed.
- ✓ Explain HFAs and why they favour float structs on ARM.
- ✓ Explain how a large struct is returned and why NRVO matters.
- ✓ Draw a stack frame and write a correct prologue and epilogue.
- ✓ Explain the frame pointer chain and what breaks it.
- ✓ Lay out a struct by hand and reorder it to save space.
- ✓ Explain why packed structs are dangerous on ARM.
- ✓ Write inline assembly with correct constraints and clobbers, and say what `"memory"` prevents.

Object files, symbols and relocation

What the assembler produces: machine code with holes in it, a list of the names it defines, a list of the names it needs, and instructions for filling the holes. Understanding this makes linker errors readable and explains where every byte of your binary comes from.

ELF structure

```
 +-----------------------------+
 |  ELF HEADER                 |  magic, class (32/64),
 |                             |  endianness, type, machine,
 |                             |  entry point, offsets to
 |                             |  the two tables below
 +-----------------------------+
 |  PROGRAM HEADERS            |  for the LOADER: which
 |  (segments)                 |  chunks to map, where, with
 |                             |  what permissions.
 |                             |  Absent in a .o file.
 +-----------------------------+
 |  .text      code            |
 |  .rodata    constants       |
 |  .data      initialised     |
 |  .bss       zero-initialised|  (occupies NO file space)
 |  .symtab    symbol table    |
 |  .strtab    strings         |
 |  .rela.text relocations     |
 |  .debug_*   DWARF (ch. 14)  |
 |  ...                        |
 +-----------------------------+
 |  SECTION HEADERS            |  for the LINKER: name,
 |                             |  type, flags, size, offset
 +-----------------------------+

TWO VIEWS OF THE SAME BYTES:
   SECTIONS  are what the LINKER manipulates
   SEGMENTS  are what the LOADER maps

A relocatable object (.o) has sections and no segments.
An executable has both. The linker's job is largely to
combine sections into segments.

INSPECT:
   readelf -h prog.o        ; header
   readelf -S prog.o        ; sections
   readelf -l prog          ; segments
   readelf -s prog.o        ; symbols
   readelf -r prog.o        ; relocations
   objdump -d prog.o        ; disassembly
   size prog.o              ; text/data/bss totals
```

Sections

```
/* WHERE EACH DECLARATION LANDS: */

        int   global_init  = 42;        /* .data           */
        int   global_zero;              /* .bss            */
        const int global_ro = 7;        /* .rodata         */
        static int file_static = 1;     /* .data, local    */
        char *s = "hello";              /* the POINTER in
                                           .data, the STRING
                                           in .rodata      */

        void f(void) { }                /* .text           */

        void g(void) {
                static int counter;     /* .bss - a static
                                           local is a global
                                           with local scope */
                int local;              /* the STACK - not in
                                           the object file  */
        }

/* .bss COSTS NO FILE SPACE. It records "reserve N zeroed
   bytes at this address"; the loader provides zeroed pages.
   Which is why a program with a 1 GB zero array is a small
   file, and why moving a large zero-initialised array into
   .data by writing `= {0}`... does nothing, because the
   compiler recognises all-zero initialisers and puts it in
   .bss anyway.                                              */

/* SECTION FLAGS - what readelf -S shows: */
        A   ALLOC      occupies memory at run time
        X   EXECINSTR  executable
        W   WRITE      writable
        M   MERGE      identical entries may be merged
        S   STRINGS    contains NUL-terminated strings

        .text    AX     read + execute, not writable (W^X)
        .rodata  A      read only
        .data    WA     read + write
        .bss     WA     read + write, NOBITS type
        .symtab  (none) not loaded at run time

/* CUSTOM SECTIONS, which embedded code uses constantly: */
        __attribute__((section(".isr_vector")))
        const void *vectors[] = { &stack_top, reset_handler };

        __attribute__((section(".fastcode")))
        void hot_function(void) { ... }

/* and the linker script places them (chapter 12).

/* THE PATTERN FOR REGISTRATION TABLES, used by the kernel's
   initcalls (Linux chapter 04) and by test frameworks: */

        #define REGISTER(fn) \
            static void (*const _reg_##fn)(void) \
                __attribute__((section("my_table"), used)) = fn;

/* The linker gathers every entry into one contiguous section
   and defines __start_my_table and __stop_my_table
```

```
    automatically, so you can iterate the table without
    maintaining a list.                                    */
```

Symbols

```
$ readelf -s prog.o

Num: Value  Size Type    Bind   Vis      Ndx Name
  0: 0         0 NOTYPE  LOCAL  DEFAULT  UND
  5: 0         4 OBJECT  LOCAL  DEFAULT    3 file_static
  8: 0        24 FUNC    GLOBAL DEFAULT    1 main
  9: 0         4 OBJECT  GLOBAL DEFAULT    3 global_init
 10: 0         0 NOTYPE  GLOBAL DEFAULT  UND printf
 11: 0         4 OBJECT  WEAK   DEFAULT  COM global_zero

/* THE FIELDS THAT MATTER:

   TYPE     FUNC, OBJECT (data), SECTION, FILE, NOTYPE
   BIND     LOCAL   - not visible outside this object
                      (`static`)
            GLOBAL  - visible; duplicate definitions are an
                      error
            WEAK    - may be overridden by a GLOBAL of the
                      same name; if never defined, resolves
                      to 0
   NDX      the section it lives in, or:
            UND     - UNDEFINED. This object NEEDS it.
            ABS     - an absolute value
            COM     - a COMMON symbol (see below)
   VIS      DEFAULT, HIDDEN, PROTECTED, INTERNAL

/* UND SYMBOLS ARE THE UNRESOLVED REFERENCES. Every
   "undefined reference to X" from the linker means: some
   object had X as UND, and nothing supplied a definition
   (chapter 12).

/* WEAK SYMBOLS, and why embedded code uses them: */
        __attribute__((weak)) void board_init(void) { }
        /* a default that a board file may override */

        __attribute__((weak)) extern int optional_feature;
        if (&optional_feature)          /* NULL if absent */
                use_it();

/* This is exactly how a vendor SDK provides overridable
   default handlers, and how the kernel provides
   architecture-overridable functions.

/* COMMON SYMBOLS - a historical C wart: */
        int x;          /* at file scope, in TWO .c files */
/* Traditionally this was legal: both become COMMON, and the
   linker merges them into one object. That silently merges
   what were meant to be two separate variables.
   -fno-common is now the DEFAULT (GCC 10+), making it an
   error. Do not turn it off.                              */

/* VISIBILITY, for shared libraries: */
        __attribute__((visibility("hidden")))
        void internal_helper(void);
/* Not exported from the .so: faster calls (no PLT), smaller
   dynamic symbol table, and a real API boundary. Build with
   -fvisibility=hidden and mark exports explicitly - this is
   what well-behaved libraries do.                         */
```

Relocations

```
/* The assembler does not know where anything will end up. So
   it leaves HOLES and records how to fill them.          */

        extern int counter;
        void inc(void) { counter++; }

$ objdump -d inc.o
0000000000000000 <inc>:
   0:   90000000        adrp    x0, 0 <inc>       ; hole
   4:   f9400001        ldr     x1, [x0]          ; hole
   8:   b9400020        ldr     w0, [x1]
   c:   11000400        add     w0, w0, #0x1
  10:   b9000020        str     w0, [x1]
  14:   d65f03c0        ret

$ readelf -r inc.o
Offset  Type                    Symbol
000000  R_AARCH64_ADR_GOT_PAGE  counter
000004  R_AARCH64_LD64_GOT_LO12_NC  counter

/* THE `adrp x0, 0` has a ZERO where the address should be.
   The relocation says: "at offset 0, patch in the page
   address of counter's GOT entry, using the ADRP encoding".

/* A RELOCATION IS: (offset, type, symbol, addend)
     offset  where in the section to patch
     type    HOW to patch - which bits, what encoding, what
             arithmetic
     symbol  what to patch in
     addend  a constant to add

/* AARCH64 RELOCATION TYPES you will see:
     R_AARCH64_CALL26          bl/b, +/-128 MB range
     R_AARCH64_JUMP26          b
     R_AARCH64_ADR_PREL_PG_HI21   adrp - the page address
     R_AARCH64_ADD_ABS_LO12_NC    add - the page offset
     R_AARCH64_LDST32_ABS_LO12_NC ldr/str offset
     R_AARCH64_ABS64              a full 64-bit address, in
                                  data
     R_AARCH64_ADR_GOT_PAGE       via the GOT (chapter 13)
     R_AARCH64_TLSLE_*            thread-local storage

/* THE ADRP + ADD PAIR is the standard AArch64 idiom for
   forming an address:
        adrp x0, symbol          ; the 4 KB PAGE containing it
        add  x0, x0, :lo12:symbol ; plus the offset within
   Two instructions, +/-4 GB range, position independent.
   You will see it constantly in ARM64 disassembly.      */

/* THE 128 MB LIMIT ON `bl` matters in large binaries: if a
   call is further than that, the linker inserts a VENEER - a
   small stub that does a full 64-bit indirect branch, using
   x16/x17 (chapter 10). That is what those registers are
   reserved for, and it is why a huge kernel or browser binary
   contains thousands of tiny thunks.                    */
```

> **READING A RELOCATION TELLS YOU WHAT THE LINKER WILL DO**
> When a symbol resolves to the wrong thing, or a call goes to address zero, `readelf -r` on the object shows exactly which reference the linker was asked to patch and with what. That is usually faster than reasoning about link order, and it distinguishes "the reference is wrong" from "the definition is missing".

What the assembler does

```
/* Two passes, conceptually:

   PASS 1: read the source, build the symbol table, assign
           offsets. Every instruction's size must be known -
           which is why ARM's fixed 32-bit instructions make
           this trivial and x86's variable-length ones do not.

   PASS 2: emit the encodings. For anything whose value is not
           yet known, emit zero and record a relocation.

/* ASSEMBLER DIRECTIVES - not instructions, but commands to
   the assembler: */

         .text                   ; switch to the .text section
         .section .rodata
         .globl  main            ; make `main` GLOBAL
         .type   main, %function
         .size   main, .-main    ; current position minus start
         .align  3               ; align to 2^3 = 8 bytes
         .p2align 4,,15
         .word   0x12345678      ; emit 4 bytes
         .quad   symbol          ; emit 8 bytes + a relocation
         .byte   1, 2, 3
         .asciz  "hello"         ; string with a NUL
         .space  100             ; 100 zero bytes
         .equ    CONST, 42       ; a symbolic constant
         .macro  / .endm         ; assembler macros
         .cfi_startproc          ; unwind information
         .cfi_def_cfa_offset 32  ; (chapter 14)
         .cfi_offset 29, -32
         .cfi_endproc

/* THE .cfi_* DIRECTIVES generate .eh_frame, which is how
   backtraces and C++ exceptions work without frame pointers.
   Hand-written assembly that omits them is invisible to
   unwinders - which is why kernel assembly routines carry
   them, and why a missing one breaks perf's call graphs. */

/* THE LOCAL LABEL CONVENTION:
         .L...      a local label, NOT in the symbol table
         1:  2:     numeric labels, referenced as 1b (back)
                    or 1f (forward) - reusable, and the usual
                    choice inside macros                    */
```

The toolchain

```
# --- What is in this object? ---
    readelf -a prog.o | less
    objdump -x prog.o
    nm prog.o                   ; symbols, compactly
      0000000000000000 T main       T = text, global
      0000000000000000 D global      D = data, global
      0000000000000004 B counter     B = bss
      0000000000000000 R constant    R = rodata
                       U printf      U = UNDEFINED
      0000000000000000 t helper      lowercase = LOCAL
      0000000000000000 W weak_fn     W = weak

    nm -C prog.o                ; DEMANGLE C++ names
    nm --defined-only prog.o
    nm -u prog.o                ; undefined only - "what does
                                ; this object need?"

# --- Disassemble ---
    objdump -d prog.o           ; code only
    objdump -D prog.o           ; ALL sections, including data
    objdump -dS prog.o          ; interleaved with SOURCE
                                ; (needs -g)
    objdump -drwC prog.o        ; with relocations, demangled

    llvm-objdump -d --mcpu=cortex-a53 prog.o

# --- Where did the size go? ---
    size prog.o
       text    data     bss     dec     hex filename
       1234     100     256    1590     636 prog.o

    size -A prog                ; PER SECTION
    nm --size-sort -S prog | tail -20      ; the biggest
                                           ; symbols
    bloaty prog                 ; the best tool - attributes
                                ; every byte to a source file

# --- Manipulate ---
    strip prog                  ; remove symbols
    strip --strip-debug prog    ; keep the symbol table
    objcopy -O binary prog.elf prog.bin      ; for flashing
    objcopy --only-keep-debug prog prog.debug
    objcopy --add-gnu-debuglink=prog.debug prog
    ; ship a stripped binary plus separate debug info

    ar rcs libfoo.a a.o b.o     ; a static library is just an
    ar t libfoo.a               ; archive of object files
    ranlib libfoo.a             ; build the symbol index
```

How it shows up

Symptom	Cause
"undefined reference to X"	X is UND in some object and defined nowhere — `nm -u` finds who needs it
"multiple definition of X"	Two objects define it — likely a definition in a header
Two variables silently became one	Common symbols — build with `-fno-common`

Symptom	Cause
A weak default is not overridden	The strong definition is in a library the linker did not pull in (ch. 12)
Binary far larger than expected	Debug info, or no `--gc-sections`. Run `bloaty`
A 1 GB array makes a small file	It is in `.bss` — correct
"relocation truncated to fit"	Target out of range — usually a large binary or a wrong memory model
Thousands of tiny stubs in the disassembly	Linker veneers for out-of-range calls
Backtrace stops in assembly code	Missing `.cfi_*` directives
A symbol is exported that should not be	Default visibility — use `-fvisibility=hidden`
C++ symbol names are unreadable	Mangled — use `nm -C` or `c++filt`
A custom section is dropped	Missing `used` attribute, or `--gc-sections` removed it

Check yourself

- ✓ Describe ELF's structure and the difference between sections and segments.
- ✓ Say which section each kind of declaration lands in, and why `.bss` is free.
- ✓ Use a custom section plus `__start_` / `__stop_` to build a registration table.
- ✓ Read a symbol table and explain TYPE, BIND and NDX.
- ✓ Explain weak symbols and give two embedded uses.
- ✓ Explain common symbols and why `-fno-common` is now the default.
- ✓ Explain what a relocation contains and read one with `readelf -r`.
- ✓ Explain the `adrp` + `add` idiom and the 128 MB branch limit.
- ✓ Say what `.cfi_*` directives produce and what breaks without them.
- ✓ Use `nm`, `objdump`, `size` and `bloaty` to account for a binary's contents.

Static and dynamic linking

Combining object files into a program: symbol resolution, the archive rules that make link order matter, linker scripts for bare metal, and the shared-library machinery that resolves calls at run time.

What the linker does

```
INPUT:   a.o  b.o  libc.a  libfoo.so
OUTPUT:  a.out

1. READ every input, collecting sections and symbols.

2. RESOLVE symbols: for every UND reference, find a
   definition.

3. For ARCHIVES (.a), pull in ONLY the members that
   resolve a currently-undefined symbol. This is where
   link order comes from.

4. LAYOUT: assign an address to every output section,
   merging like-named input sections.
      .text  = a.o(.text) + b.o(.text) + ...
      .data  = a.o(.data) + b.o(.data) + ...

5. RELOCATE: now that every address is known, patch every
   hole (chapter 11).

6. Emit PROGRAM HEADERS describing what the loader should
   map, with what permissions.

THE ORDER OF STEPS 2 AND 3 IS THE WHOLE STORY behind the
most confusing linker errors, so it is worth being precise:
the linker processes inputs LEFT TO RIGHT, maintaining a
set of currently-undefined symbols. An archive is scanned
ONCE, at the position where it appears.
```

Link order

```
# THE RULE: a library must appear AFTER the objects that need
# it.

    gcc main.o -lfoo            # WORKS
    gcc -lfoo main.o            # "undefined reference to
                                #  foo_init"

/* WHY: when the linker reaches -lfoo in the second case,
   nothing is undefined yet, so it takes NOTHING from the
   archive. Then main.o introduces undefined symbols, and the
   archive is already behind it.                          */

# CIRCULAR DEPENDENCIES between two libraries:
    gcc main.o -lfoo -lbar -lfoo      # repeat it
    gcc main.o -Wl,--start-group -lfoo -lbar -Wl,--end-group
    ; the group is rescanned until no new symbols are resolved

# THE GRANULARITY IS THE ARCHIVE MEMBER (one .o), not the
# function. Pulling in one function from a member drags in the
# whole member - and everything IT references. This is why:
#   · a static library should have one function per source
#     file if size matters
#   · linking a "small" helper can pull in printf, and with it
#     the entire formatted-output machinery and locale support

# FORCE INCLUSION, when a member is needed for its side
# effects (a constructor, a registration table):
    -Wl,--whole-archive -lfoo -Wl,--no-whole-archive
    -u symbol_name              ; pretend this symbol is
                                ; undefined, forcing the
                                ; member in

# THIS IS A REAL EMBEDDED PROBLEM: a driver registered by a
# constructor in a static library is silently absent, because
# nothing referenced it. --whole-archive or an explicit -u is
# the fix.
```

Static libraries

```
# A .a is just an ar archive of .o files, plus an index.

    ar rcs libmath.a add.o sub.o mul.o
    ar t libmath.a              ; list members
    ar x libmath.a add.o        ; extract one
    nm libmath.a                ; symbols, per member
    ranlib libmath.a            ; rebuild the symbol index

    gcc main.c -L. -lmath -o prog
    gcc -static main.c -o prog  ; link EVERYTHING statically

# PROPERTIES:
#   + no runtime dependencies; one file to deploy
#   + no dynamic loader cost at startup
#   + the linker can do whole-program optimisation
#     (--gc-sections, LTO)
#   + no symbol interposition, so calls are direct
#   - larger binaries if many programs share the library
#   - a library fix requires relinking every program
#   - glibc's static linking has caveats: getaddrinfo, NSS
#     and dlopen still need the matching shared libraries at
#     run time, which defeats the point

# FOR FULLY STATIC BINARIES, use musl:
    musl-gcc -static main.c -o prog
    ; or a musl-based cross toolchain. This is what most
    ; container and embedded images do.

# EMBEDDED PRACTICE: static linking is the norm. There is no
# dynamic loader on bare metal, the memory saving of sharing
# does not apply with one program, and the determinism is
# worth a great deal.
```

Dead code elimination at link time

```
# By default the linker keeps whole SECTIONS. Put each
# function and variable in its own section and it can drop
# the unused ones.

    gcc -ffunction-sections -fdata-sections -c *.c
    gcc -Wl,--gc-sections *.o -o prog

    # see what was dropped:
    gcc -Wl,--gc-sections -Wl,--print-gc-sections ...

# TYPICAL SAVING on embedded code: 20-50%. On a
# flash-constrained part this is the single highest-value
# linker flag.

# KEEP something the linker would drop:
    __attribute__((used))              /* in the source */
    KEEP(*(.isr_vector))               /* in the linker
                                          script - REQUIRED
                                          for vector tables */
    -Wl,--undefined=symbol

# AND THE OTHER SIZE TOOLS:
    -Wl,--icf=all              ; identical code folding: merge
                               ; byte-identical functions
                               ; (lld and gold)
    -flto                      ; whole-program optimisation
                               ; (chapter 06)
    -Os / -Oz
    strip / --strip-debug

# MEASURE, do not guess:
    size -A prog
    bloaty prog -d compileunits,symbols
    ; bloaty attributes every byte to a source file and
    ; symbol, which turns "the binary is too big" into a
    ; ranked list.
```

Linker scripts

```
/* On bare metal you must tell the linker where memory IS.
   This is a realistic script for a Cortex-M or an ARM64
   firmware image.                                          */

ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 512K
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

_stack_size = 4K;

SECTIONS
{
    .isr_vector :
    {
        KEEP(*(.isr_vector))    /* KEEP: never gc this */
    } > FLASH

    .text :
    {
        *(.text .text.*)
        *(.rodata .rodata.*)
        . = ALIGN(4);
        _etext = .;             /* define a symbol at the
                                   current location         */
    } > FLASH

    /* .data is STORED in flash, RUN from RAM - so it has two
       addresses: a Load Memory Address and a Virtual Memory
       Address.                                              */
    .data : AT (_etext)
    {
        _sdata = .;
        *(.data .data.*)
        . = ALIGN(4);
        _edata = .;
    } > RAM

    .bss :
    {
        _sbss = .;
        *(.bss .bss.*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    .stack (NOLOAD) :
    {
        . = ALIGN(8);
        . = . + _stack_size;
        _stack_top = .;
    } > RAM

    /DISCARD/ : { *(.comment) *(.ARM.attributes) }
}
```

```
/* THE LMA/VMA DISTINCTION IS THE KEY IDEA for firmware:
   initialised data must be STORED somewhere non-volatile and
   COPIED to RAM before main() runs. The startup code does
   it:                                                      */

void Reset_Handler(void)
{
        extern uint32_t _etext, _sdata, _edata, _sbss, _ebss;
        uint32_t *src = &_etext;
        uint32_t *dst = &_sdata;

        while (dst < &_edata)           /* copy .data from
                dst++ = src++;             flash to RAM     */

        dst = &_sbss;
        while (dst < &_ebss)            /* zero .bss */
                *dst++ = 0;

        __libc_init_array();            /* C++ constructors */
        main();
        while (1) ;
}

/* USEFUL SCRIPT CONSTRUCTS:
     .           the current location counter
     ALIGN(n)
     ADDR(sec), LOADADDR(sec), SIZEOF(sec)
     PROVIDE(sym = value)    define only if not already
     ASSERT(cond, "message") FAIL THE LINK on a condition -
                             e.g. ASSERT(_ebss < _stack_top,
                             "RAM overflow")
     NOLOAD                  reserve space, emit no data

/* THE ASSERT IS UNDERUSED and worth adopting: a link that
   fails with "RAM overflow" is far better than a firmware
   image that overwrites its own stack at run time.      */

# See the default script:
    ld --verbose
    gcc -Wl,--verbose ...
# and the final layout:
    gcc -Wl,-Map=out.map ...
    ; the map file shows every section's address and size, and
    ; WHY each object was included. Read it when something is
    ; unexpectedly present.
```

Shared libraries

```
# Build one:
    gcc -fPIC -c foo.c bar.c
    gcc -shared -Wl,-soname,libfoo.so.1 \
        -o libfoo.so.1.2.3 foo.o bar.o
    ln -s libfoo.so.1.2.3 libfoo.so.1
    ln -s libfoo.so.1 libfoo.so

# THE THREE NAMES:
#   libfoo.so           the LINKER name - what -lfoo finds
#   libfoo.so.1         the SONAME - what the loader records
#                       in the executable and searches for
#   libfoo.so.1.2.3     the real file
#
# THE SONAME IS THE ABI VERSION. Change the ABI
# incompatibly -> bump it to .so.2, and old binaries keep
# using .so.1. Get this wrong and you break every program on
# the system.

# -fPIC IS REQUIRED for shared library code: the library may
# be mapped at any address, so it must not contain absolute
# addresses (chapter 13).

# --- Inspecting ---
    ldd prog                    ; what it needs
    readelf -d prog             ; the DYNAMIC section:
                                ; NEEDED, SONAME, RPATH
    readelf --dyn-syms libfoo.so
    objdump -T libfoo.so        ; the dynamic symbol table

# --- Finding libraries at run time, in order ---
#   1. DT_RPATH in the executable (deprecated)
#   2. LD_LIBRARY_PATH
#   3. DT_RUNPATH
#   4. /etc/ld.so.cache        (built by ldconfig)
#   5. /lib, /usr/lib
#
    LD_LIBRARY_PATH=/opt/lib ./prog
    LD_DEBUG=libs ./prog        ; trace the search
    LD_DEBUG=all ./prog
    ldconfig -p | grep foo

# --- $ORIGIN, for relocatable installs ---
    gcc -Wl,-rpath,'$ORIGIN/../lib' ...
    ; resolves relative to the EXECUTABLE's location, so the
    ; whole tree can be moved. This is how self-contained
    ; application bundles work.
```

Symbol interposition

```
/* In a dynamic link, the FIRST definition found wins - across
   the whole process. That is interposition, and it is both a
   feature and a hazard.                                      */

# LD_PRELOAD: insert a library before all others
    LD_PRELOAD=./mymalloc.so ./prog
    ; every call to malloc now goes to yours

/* WRITING ONE: */
#define _GNU_SOURCE
#include <dlfcn.h>

void *malloc(size_t size)
{
        static void *(*real_malloc)(size_t);

        if (!real_malloc)
                real_malloc = dlsym(RTLD_NEXT, "malloc");

        void *p = real_malloc(size);
        log_allocation(p, size);
        return p;
}

/* USES: profiling, leak tracking, fault injection, sandboxing,
   and - the reason security people care - hijacking.

/* THE COST OF ALLOWING IT: every call to a global function
   from within a shared library must go through the PLT, in
   case someone interposed. That is why:
        -fvisibility=hidden          hide internals
        -Bsymbolic                   bind internal references
                                     directly
        __attribute__((visibility("hidden")))
   are worth using: they make intra-library calls direct, and
   they stop your internals becoming part of your ABI.

/* VERSION SCRIPTS - the disciplined way: */
        {
          global:
            foo_init;
            foo_process;
            foo_shutdown;
          local:
            *;                  /* everything else hidden */
        };
    gcc -shared -Wl,--version-script=foo.map ...

/* This is what a well-maintained library ships. It makes the
   exported set explicit and reviewable, rather than
   "whatever happened not to be static".                */
```

> **STATIC OR SHARED, FOR EMBEDDED WORK**
> On bare metal there is no choice: static. On embedded Linux, the trade is real — shared libraries save flash and RAM when several programs use them, and static linking gives determinism, faster startup, no version skew, and lets LTO and `--gc-sections` see the whole program. Most modern embedded images static-link the application against musl and keep only a handful of shared libraries. Measure both for your image; the answer depends on how many binaries you ship.

The linkers

Linker	Notes
`ld.bfd`	The classic GNU linker. Most features, most target support, slowest.
`ld.gold`	Faster, ELF only. Now deprecated and unmaintained.
`lld`	LLVM's. Very fast, good diagnostics, ICF, ThinLTO. `-fuse-ld=lld`
`mold`	The fastest available; a drop-in for most uses. `-fuse-ld=mold`

```
# Switching is usually one flag and worth trying on a large
# project — link time can drop by 5–10x:
    gcc -fuse-ld=lld ...
    gcc -fuse-ld=mold ...
    ld --version

# WHEN IT MATTERS: an incremental build's wall time is often
# dominated by the link, not the compile. On a kernel or a
# large C++ application the difference between bfd and mold is
# minutes per build.
```

How it shows up

Symptom	Cause
"undefined reference" despite the library being listed	Library before the object that needs it — put `-l` last
"undefined reference" only in a static build	Circular archive dependency — use `--start-group`
A constructor in a static library never runs	Nothing referenced that member — `--whole-archive` or `-u`
Vector table missing from the image	Garbage collected — needs `KEEP()`
Firmware runs from flash but variables are wrong	`.data` not copied from LMA to VMA by the startup code
Silent stack/heap collision at run time	No `ASSERT` in the linker script
"cannot find -lfoo"	Missing `-L`, or no `libfoo.so` symlink
Runs on the build host, "cannot open shared object" on target	Library not deployed, or the SONAME differs
Program picks up the wrong library version	Search order — check with `LD_DEBUG=libs`
Upgrading a library breaks every binary	SONAME not bumped on an incompatible change
Internal function called through the PLT	Default visibility — use a version script
Binary much bigger than expected	No `--gc-sections`; run `bloaty`

Check yourself

- ✓ List the linker's six steps in order.
- ✓ Explain why link order matters and give three ways to handle circularity.
- ✓ Explain archive-member granularity and its size consequence.
- ✓ Say why a constructor in a static library may never run, and two fixes.
- ✓ Enable section-level garbage collection and measure the saving.
- ✓ Write a linker script with MEMORY, SECTIONS and an ASSERT.
- ✓ Explain LMA versus VMA and write the startup code that bridges them.
- ✓ Explain the three shared-library names and what the SONAME governs.
- ✓ Trace the runtime library search with `LD_DEBUG`.
- ✓ Explain symbol interposition, its cost, and how a version script controls it.
- ✓ Compare static and shared linking for an embedded image.

Loaders, PIE and address space layout

From an executable file to a running process: what the kernel maps, what the dynamic loader does before `main`, and how position-independent code makes address space randomisation possible.

What happens on exec

```
execve("./prog", argv, envp)
     |
KERNEL:
  · parse the ELF header, check the magic and machine
  · tear down the old address space
  · MAP the PT_LOAD segments at their requested addresses,
    with the requested permissions
  · set up the stack: argv, envp, and the AUXILIARY VECTOR
  · if there is a PT_INTERP segment, ALSO map the
    interpreter named in it - normally
    /lib/ld-linux-aarch64.so.1
  · set PC to the INTERPRETER's entry point, not the
    program's
     |
DYNAMIC LOADER (ld.so):
  · relocate ITSELF first
  · read DT_NEEDED from the executable: which libraries
  · find and map each one, recursively
  · perform RELOCATIONS across all of them
  · initialise TLS
  · run each library's initialisers, then the
    executable's
  · jump to the executable's entry point
     |
_start  (in crt1.o)
  · set up the frame, align the stack
  · call __libc_start_main(main, argc, argv, ...)
     |
__libc_start_main
  · initialise libc: stdio, locale, the heap
  · register the atexit handler for destructors
  · call the .init_array constructors
  · call main(argc, argv, envp)
  · call exit(main's return value)

FOR A STATIC BINARY there is no PT_INTERP, so the kernel
jumps straight to _start and the whole loader phase does
not exist. That is most of the startup-time difference.

WATCH IT:  LD_DEBUG=all ./prog 2>&1 | less
           strace ./prog
```

Position independence and ASLR

```
/* THREE KINDS OF BINARY: */

   ET_EXEC       fixed load address, baked into the file.
                 Cannot be randomised. Historical default.

   ET_DYN + PIE  POSITION INDEPENDENT EXECUTABLE. Can be
                 loaded anywhere. The default on every modern
                 distribution.

   ET_DYN        a shared library. Always position
                 independent.

/* BUILD THEM: */
        gcc -fPIE -pie prog.c -o prog       ; PIE (default)
        gcc -fno-pie -no-pie prog.c -o prog ; fixed
        gcc -fPIC -shared foo.c -o libfoo.so

/* WHY PIE MATTERS: ADDRESS SPACE LAYOUT RANDOMISATION. If the
   attacker does not know where your code is, return-oriented
   programming becomes far harder - they cannot construct a
   chain of gadget addresses without an information leak
   first.

   WHAT GETS RANDOMISED on Linux:
        the executable itself   (only if PIE)
        every shared library
        the heap (brk)
        mmap regions
        the stack
        vdso

   CHECK:
        cat /proc/sys/kernel/randomize_va_space
             0 = off, 1 = partial, 2 = full (the default)
        cat /proc/self/maps          ; twice - the addresses
                                     ; differ
        setarch -R ./prog            ; run with ASLR disabled,
                                     ; for reproducible
                                     ; debugging

/* THE COST: PIE code must reach its own globals without
   absolute addresses. On AArch64 that costs almost nothing -
   `adrp`+`add` is PC-relative and already the normal idiom
   (chapter 11). On 32-bit x86 it cost a dedicated register
   and 5-10% performance, which is why the transition took so
   long there and was trivial on ARM.                    */
```

The GOT and the PLT

```
A call to a function in another shared object cannot be a
direct branch: the address is unknown until load time, and the
code page must stay read-only and shared between processes.

  THE SOLUTION: indirection through two tables.

   GOT   Global Offset Table - in .data, WRITABLE. Holds
         resolved ADDRESSES.
   PLT   Procedure Linkage Table - in .text, executable. A
         small stub per external function.

  CALLING printf:

     your code:
         bl   printf@plt

     printf@plt:                    (in .plt)
         adrp x16, :got:printf
         ldr  x17, [x16, :got_lo12:printf]   ; load from GOT
         br   x17                            ; jump there

     GOT entry for printf:
         initially -> the PLT resolver stub
         after the first call -> the real address of printf

  LAZY BINDING: the first call goes to the resolver, which
  finds printf, WRITES its address into the GOT, and jumps to
  it. Every later call loads the now-correct address and goes
  straight there.

  WHY LAZY: a program using 5,000 library functions but
  calling 200 of them resolves only 200. Startup is faster.

  WHY YOU MIGHT NOT WANT IT:
        LD_BIND_NOW=1 ./prog
        gcc -Wl,-z,now -Wl,-z,relro
  Resolve everything at startup, then make the GOT READ-ONLY
  (RELRO). This closes "GOT overwrite" attacks, in which an
  attacker with an arbitrary write redirects a library call.
  FULL RELRO is standard hardening and costs a little startup
  time.

  CHECK IT:
        readelf -d prog | grep -E 'BIND_NOW|FLAGS'
        checksec --file=prog
```

Thread-local storage

```
        __thread int counter;           /* GNU */
        _Thread_local int counter;      /* C11 */
        thread_local int counter;       /* C++11 */

/* Each thread gets its own instance. The implementation
   depends on how much the compiler knows at compile time -
   there are four MODELS, from fastest to most general: */

   LOCAL-EXEC       the variable is in the main executable.
                    Its offset from the thread pointer is
                    known at LINK time.
                        mrs  x0, tpidr_el0
                        add  x0, x0, #:tprel_hi12:counter
                        add  x0, x0, #:tprel_lo12_nc:counter
                    TWO instructions. The fastest.

   INITIAL-EXEC     in a library loaded at startup. The offset
                    comes from the GOT.
                        mrs  x0, tpidr_el0
                        adrp x1, :gottprel:counter
                        ldr  x1, [x1, :gottprel_lo12:counter]
                        add  x0, x0, x1

   LOCAL-DYNAMIC    several variables in a dlopen'd library.
   GENERAL-DYNAMIC  the fully general case. Calls
                    __tls_get_addr - a FUNCTION CALL for every
                    access.

/* SELECT ONE: */
        gcc -ftls-model=initial-exec ...
        __attribute__((tls_model("initial-exec")))

/* THE PRACTICAL POINT: a `__thread` variable in a library
   that might be dlopen'd uses general-dynamic and costs a
   function call per access. In a hot path that is
   significant, and forcing initial-exec is the fix - at the
   cost of the library no longer being dlopen-able.

   ON AARCH64 the thread pointer is TPIDR_EL0, readable from
   user mode with one instruction, which is why local-exec TLS
   is nearly free.                                          */
```

Constructors and destructors

```
/* Code that runs before main and after it returns. */

        __attribute__((constructor))
        static void my_init(void) { ... }

        __attribute__((destructor))
        static void my_fini(void) { ... }

        __attribute__((constructor(101)))   /* priority:
                                               lower runs
                                               first          */
        static void very_early(void) { ... }

/* HOW: pointers are placed in .init_array / .fini_array, and
   the loader (or __libc_csu_init for a static binary) walks
   the array.

        readelf -x .init_array prog

/* C++ STATIC INITIALISATION uses the same mechanism: a
   constructor for every namespace-scope object with a
   non-constant initialiser.

/* THE STATIC INITIALISATION ORDER FIASCO: the order of
   constructors ACROSS translation units is UNSPECIFIED. So: */

        /* a.cpp */  Logger logger;
        /* b.cpp */  Config config{logger};   /* MAY run first,
                                                   using an
                                                   unconstructed
                                                   logger        */

/* THE FIX - construct on first use, which is guaranteed to be
   ordered by the dependency itself: */
        Logger &get_logger(void)
        {
                static Logger instance;   /* constructed on the
                                             first call, and
                                             thread-safe since
                                             C++11             */
                return instance;
        }

/* AND IN A SHARED LIBRARY, constructors run at dlopen and
   destructors at dlclose - so a library that registers itself
   with a constructor works, and one that assumes main() has
   already run does not.                                      */
```

The process address space

```
$ cat /proc/self/maps

aaaab2340000-aaaab2341000 r-xp 00000000 fe:01 12345 /bin/cat
aaaab2350000-aaaab2351000 r--p 00000000 fe:01 12345 /bin/cat
aaaab2351000-aaaab2352000 rw-p 00001000 fe:01 12345 /bin/cat
aaaad4f60000-aaaad4f81000 rw-p 00000000 00:00 0     [heap]
ffff8a100000-ffff8a280000 r-xp 00000000 fe:01 23456 /lib/libc.so.6
ffff8a280000-ffff8a290000 r--p 00180000 fe:01 23456 /lib/libc.so.6
ffff8a290000-ffff8a292000 rw-p 00190000 fe:01 23456 /lib/libc.so.6
ffff8a2a0000-ffff8a2c0000 r-xp 00000000 fe:01 34567 /lib/ld-linux-aarch64.so.1
ffffc4d20000-ffffc4d41000 rw-p 00000000 00:00 0     [stack]
ffffa3b40000-ffffa3b41000 r-xp 00000000 00:00 0     [vdso]

/* READ THE PERMISSIONS COLUMN:
     r-xp   code: readable, executable, NOT writable
     r--p   read-only data, including relocated-then-RELRO
     rw-p   writable data
     p      private (copy-on-write); s would be shared

   NO SEGMENT IS BOTH `w` AND `x`. That is W^X, and it is
   what makes injected shellcode unusable without a further
   step. A binary that needs writable-executable memory - a
   JIT - must mprotect explicitly, and hardened systems
   forbid it.

/* [vdso] - a page the kernel maps into every process
   containing fast implementations of gettimeofday,
   clock_gettime and getcpu, which read the clock directly
   without a syscall. That is why clock_gettime costs about
   25 ns rather than 500 (ARM chapter 32).

/* THE FILE OFFSETS reveal the segment structure: libc appears
   three times, at offsets 0, 0x180000 and 0x190000 - the
   same file mapped three times with different permissions,
   which is exactly the PT_LOAD segments from chapter 11. */
```

HARDENING FLAGS, AND WHAT EACH STOPS

```
-fstack-protector-strong    stack smashing
-D_FORTIFY_SOURCE=2         buffer overflows in libc calls
-fPIE -pie                  enables ASLR of the executable
-Wl,-z,relro,-z,now         GOT overwrite
-Wl,-z,noexecstack          executable stack
-fstack-clash-protection    stack-guard-page jumping
-fcf-protection / -mbranch-protection=standard
                            ROP/JOP, via BTI and PAC on ARMv8.3+
```

Check what a binary actually has with `checksec --file=prog`. On ARMv8.3 and later, `-mbranch-protection=standard` enables pointer authentication and branch target identification, which are the strongest of these and cost very little.

Loading at run time

```
#include <dlfcn.h>

        void *h = dlopen("libplugin.so", RTLD_NOW | RTLD_LOCAL);
        if (!h) {
                fprintf(stderr, "%s\n", dlerror());
                return -1;
        }

        /* the cast is ugly and POSIX-required */
        void (*fn)(int) = (void (*)(int))dlsym(h, "plugin_run");
        if (!fn) { ... }

        fn(42);
        dlclose(h);

/* THE FLAGS:
     RTLD_NOW     resolve everything at dlopen. Fails
                  immediately on a missing symbol.
     RTLD_LAZY    resolve on first use. Fails LATER, at the
                  call - much harder to diagnose.
     RTLD_GLOBAL  the library's symbols become available to
                  subsequently loaded libraries
     RTLD_LOCAL   they do not. The safe default.
     RTLD_NODELETE  never unload

/* PREFER RTLD_NOW | RTLD_LOCAL. Lazy binding in a plugin
   means a typo in an exported name crashes in production
   rather than failing at load.

/* THE C++ PROBLEM: dlsym takes a mangled name. Export plugin
   entry points as extern "C", or provide a single registration
   function.

/* AND THE UNLOAD HAZARD: after dlclose, every pointer into
   the library - function pointers, vtables, static data,
   string literals - is dangling. A callback registered with
   another subsystem and never unregistered becomes a crash
   at an arbitrary later moment. Plugin systems commonly use
   RTLD_NODELETE for exactly this reason.              */
```

How it shows up

Symptom	Cause
"cannot open shared object file"	Not on the search path — `LD_DEBUG=libs` shows where it looked
Wrong library version loaded	Search order; check `ldd` and `LD_LIBRARY_PATH`
Addresses differ every run	ASLR — use `setarch -R` for reproducible debugging
Crash at a call, long after a successful `dlopen`	`RTLD_LAZY` hid a missing symbol
Crash after `dlclose`	A dangling pointer into the unloaded library
`dlsym` returns NULL for a C++ function	Name mangling — use `extern "C"`
Static object used before construction	Static initialisation order fiasco — construct on first use
Thread-local access is slow	General-dynamic TLS model — try `initial-exec`
Startup is slow	Many libraries and relocations; consider static linking or prelinking

Symptom	Cause
Exploit mitigations absent	Check with `checksec`; the flags are not all on by default
JIT fails with a permission error	W^X — the page must be `mprotect`ed, and hardened systems may refuse

Check yourself

- ✓ Trace execution from `execve` to `main`, naming each participant.
- ✓ Explain what changes for a statically linked binary.
- ✓ Distinguish ET_EXEC, PIE and shared objects, and say why PIE enables ASLR.
- ✓ Explain the GOT and PLT and how lazy binding works.
- ✓ Explain RELRO and the attack it prevents.
- ✓ Name the four TLS models and the cost of each.
- ✓ Explain how constructors are implemented and the static initialisation order fiasco.
- ✓ Read `/proc/self/maps` and explain the permissions and repeated file mappings.
- ✓ Explain what the vdso is for.
- ✓ List six hardening flags and what each stops.
- ✓ Use `dlopen` safely and name three hazards.

Debug information and how debuggers work

How a debugger maps machine addresses back to source lines, finds variables that live in registers, and walks a stack. DWARF is the format, and understanding it explains both what debuggers can do and why optimised code is harder to inspect.

DWARF

```
Debug information lives in ELF sections that are NOT
loaded at run time:

  .debug_info     the DIE tree: types, variables,
                  functions, scopes
  .debug_abbrev   the abbreviation table it is encoded
                  against
  .debug_line     the LINE TABLE: address -> file:line
  .debug_str      strings
  .debug_loclists variable LOCATIONS, per address range
  .debug_ranges   non-contiguous address ranges
  .debug_frame    stack unwinding
  .eh_frame       unwinding, LOADED (used by C++
                  exceptions, so it must exist at run
                  time)

THE CENTRAL STRUCTURE is a tree of DIEs - Debugging
Information Entries. Each has a TAG and ATTRIBUTES:

  DW_TAG_compile_unit
    DW_AT_name       "prog.c"
    DW_AT_comp_dir   "/home/me/src"
    DW_AT_producer   "GNU C17 13.2.0 -O2 -g"
    |
    +- DW_TAG_base_type
    |    DW_AT_name "int"  DW_AT_byte_size 4
    |    DW_AT_encoding DW_ATE_signed
    |
    +- DW_TAG_structure_type
    |    DW_AT_name "point"  DW_AT_byte_size 8
    |    +- DW_TAG_member  "x"  DW_AT_data_member_location 0
    |    +- DW_TAG_member  "y"  DW_AT_data_member_location 4
    |
    +- DW_TAG_subprogram
         DW_AT_name "main"
         DW_AT_low_pc  0x1000
         DW_AT_high_pc 0x1080
         DW_AT_frame_base  DW_OP_call_frame_cfa
         +- DW_TAG_formal_parameter "argc"
         |    DW_AT_location  ...
         +- DW_TAG_variable "sum"
              DW_AT_location  ...

READ IT:
   readelf --debug-dump=info prog | less
   llvm-dwarfdump prog
   objdump --dwarf=info prog
   dwarfdump prog
```

The line table

```
/* .debug_line maps ADDRESSES to source positions. It is
   encoded as a bytecode program for a state machine -
   because a naive table would be enormous.               */

$ readelf --debug-dump=decodedline prog

File name  Line number  Starting address
prog.c              10             0x1000
prog.c              11             0x1008
prog.c              12             0x1010
prog.c              11             0x1020    <- back to 11!
prog.c              14             0x1030

/* THE OUT-OF-ORDER ENTRY at 0x1020 is the optimiser: code
   from line 11 was scheduled after code from line 12. That is
   why single-stepping optimised code jumps around, and it is
   not a bug in the debugger.

/* EACH ROW ALSO CARRIES FLAGS:
     is_stmt          a good place for a breakpoint
     basic_block
     end_sequence
     prologue_end     <-- where a `break func` actually lands
     epilogue_begin

   `prologue_end` is why a breakpoint on a function stops
   AFTER the prologue, so the arguments are already in their
   final locations. Without it the debugger would show
   garbage for every parameter.

/* WITH INLINING, one address may map to SEVERAL source
   positions - the inlined line and the call site. DWARF
   expresses this with DW_TAG_inlined_subroutine, and it is
   how a backtrace can show:

       #0  helper (x=5) at helper.c:12
       #1  caller () at main.c:30
   even though `helper` was inlined and there is no stack
   frame for it. gdb calls these "inline frames".        */
```

Where variables live

```
/* At -O0 a variable has one home: a stack slot. Simple.

   At -O2 it moves: a register here, a different register
   there, spilled briefly, and gone entirely where the
   compiler proved it unnecessary. DWARF describes this with a
   LOCATION LIST - an expression per address range.       */

$ llvm-dwarfdump --debug-loclists prog

0x0000000c:
  [0x1000, 0x1018): DW_OP_reg0 W0
  [0x1018, 0x1030): DW_OP_reg19 W19
  [0x1030, 0x1040): DW_OP_fbreg -20
  [0x1040, 0x1050): <empty>

/* READS AS: between 0x1000 and 0x1018 the variable is in w0;
   then in w19; then at frame base minus 20; then NOWHERE -
   which is what gdb reports as `<optimized out>`.

/* THE EXPRESSIONS are a small stack language, DWARF
   expressions:
        DW_OP_reg3           it IS register 3
        DW_OP_breg3 -16      at [register 3 - 16]
        DW_OP_fbreg -20      at [frame base - 20]
        DW_OP_addr 0x4000    at a fixed address
        DW_OP_piece          the value is SPLIT across
                             several locations
        DW_OP_stack_value    the value is computed, and does
                             not exist in memory at all

   `DW_OP_stack_value` is how a debugger can show a variable
   that was optimised into an expression - "x is y*2" - even
   though nothing stores x.

/* THIS IS WHY -Og EXISTS: it optimises, but avoids
   transformations that destroy variable locations. For
   debugging code that must run at speed, -Og -g is the right
   answer, not -O0.                                       */
```

> **WHY `<OPTIMIZED OUT>` IS NOT A COMPILER BUG**
>
> If the compiler proved a variable's value is never needed at a point, it does not exist there. There is no location to report because there is no value. DWARF can describe registers, memory, split values and even computed values — but not values that were never computed. That is the honest answer, and the fix is `-Og`, not distrust of the debugger.

Stack unwinding

```
/* To produce a backtrace, a debugger must find each caller's
   frame. Two ways:

   1. THE FRAME POINTER CHAIN (chapter 10). x29 points at the
      saved x29, forming a linked list; x30 is beside it.
      Simple, fast, and REQUIRES -fno-omit-frame-pointer.

   2. CFI - CALL FRAME INFORMATION. A table describing, for
      every address, how to compute the CANONICAL FRAME
      ADDRESS and where each saved register is. No frame
      pointer needed, and it works mid-prologue.        */

$ readelf --debug-dump=frames-interp prog

   LOC     CFA      x19 x29 x30 ra
0000001000 sp+0                  u   (nothing saved yet)
0000001004 sp+32    c-32 c-24  c-8     (after the stp)
0000001008 x29+32   c-32 c-24  c-8     (after mov x29,sp)
...
0000001040 sp+0                  u   (after the epilogue)

/* READS AS: at 0x1004 the canonical frame address is sp+32,
   x19 is saved at CFA-32, and the return address at CFA-8.
   An unwinder reads that row and steps to the caller.

/* THE ASSEMBLER DIRECTIVES that generate it (chapter 11): */
        .cfi_startproc
        stp     x29, x30, [sp, #-32]!
        .cfi_def_cfa_offset 32
        .cfi_offset 29, -32
        .cfi_offset 30, -24
        mov     x29, sp
        .cfi_def_cfa_register 29
        ...
        .cfi_endproc

/* HAND-WRITTEN ASSEMBLY WITHOUT THESE IS INVISIBLE to
   unwinders: the backtrace stops there, C++ exceptions
   cannot propagate through it, and perf's call graphs are
   truncated. Every kernel and libc assembly routine carries
   them for this reason.

/* .eh_frame VERSUS .debug_frame: identical in structure, but
   .eh_frame is ALLOCATED - it is present in a stripped
   binary, because C++ exception handling needs it at run
   time. Which is why a stripped C++ binary can still produce
   a backtrace, and a stripped C binary compiled with
   -fno-asynchronous-unwind-tables cannot.              */
```

How a debugger works

```
/* --- BREAKPOINTS --- */
/* Replace the instruction at the target with a trap. On
   AArch64 that is BRK #0 (0xd4200000).                    */

        1. read and SAVE the original 4 bytes
        2. write BRK #0 there
        3. continue
        4. the CPU traps; the kernel sends SIGTRAP; the
           debugger is notified via waitpid
        5. to continue: restore the original instruction,
           single-step over it, re-insert the trap, resume

/* HARDWARE BREAKPOINTS use the debug registers instead - no
   memory modification, so they work on read-only or flash
   memory. AArch64 provides a handful (typically 6 breakpoint
   and 4 watchpoint registers), which is why gdb limits you.

/* --- WATCHPOINTS --- */
        watch myvar
/* Hardware watchpoints use the same debug registers and trap
   on ACCESS. SOFTWARE watchpoints single-step the entire
   program and compare after each instruction - correct, and
   thousands of times slower. gdb tells you which it used.

/* --- SINGLE STEPPING --- */
        PSTATE.SS, or ptrace(PTRACE_SINGLESTEP)
/* `step` (source line) is implemented as: repeatedly
   single-step until the line table says we left the current
   line. Which is why stepping optimised code jumps around -
   the line table genuinely does.

/* --- THE PTRACE INTERFACE --- */
        ptrace(PTRACE_ATTACH, pid, 0, 0);
        waitpid(pid, &status, 0);
        ptrace(PTRACE_PEEKDATA, pid, addr, 0);
        ptrace(PTRACE_POKEDATA, pid, addr, value);
        ptrace(PTRACE_GETREGSET, pid, NT_PRSTATUS, &iov);
        ptrace(PTRACE_CONT, pid, 0, signal);
        ptrace(PTRACE_DETACH, pid, 0, 0);

/* THE WHOLE DEBUGGER is ptrace plus DWARF plus a great deal
   of careful bookkeeping. A minimal one is a few hundred
   lines, and writing one is the fastest way to understand
   what gdb is doing.

/* WHY YOU MAY NOT BE ABLE TO ATTACH: */
        cat /proc/sys/kernel/yama/ptrace_scope
        # 0 = any process may attach to any of its own
        # 1 = only to descendants (the default) - which is why
        #     `gdb -p` on an unrelated process fails
        # 2 = admin only,  3 = nobody                      */
```

Managing debug info

```
# Debug info is LARGE - often several times the code size.

# --- SPLIT IT OUT, ship the binary, keep the symbols ---
    objcopy --only-keep-debug prog prog.debug
    objcopy --strip-debug prog
    objcopy --add-gnu-debuglink=prog.debug prog
    ; gdb finds prog.debug automatically via the debuglink,
    ; or via a build-id path in /usr/lib/debug

    readelf -n prog | grep -A1 'Build ID'
    ; the build-id is how distributions match a binary to its
    ; separately-packaged debug symbols

# --- SPLIT DWARF: keep it out of the .o files entirely ---
    gcc -g -gsplit-dwarf -c prog.c
    ; produces prog.dwo alongside prog.o. The linker never
    ; reads it, so LINK TIME DROPS SUBSTANTIALLY on large
    ; projects - often the single biggest build-time win for
    ; a debug build of a large C++ codebase.

# --- COMPRESS ---
    gcc -gz              ; compressed debug sections
    -Wl,--compress-debug-sections=zlib

# --- DWARF VERSION ---
    -gdwarf-4  -gdwarf-5
    ; DWARF 5 is smaller and better, but old tools may not
    ; read it. Check what your target's gdb supports.

# --- LEVELS ---
    -g0    none
    -g1    line tables and function names only - enough for
           BACKTRACES, a fraction of the size. Good for
           production builds.
    -g     (= -g2) the normal level
    -g3    plus MACRO definitions, so gdb can expand macros

# THE PRODUCTION RECOMMENDATION: build with -g, split the
# debug info out, ship the stripped binary, and archive the
# .debug file with the build. Then a crash dump from the
# field is analysable, and the shipped image is small.
```

Beyond gdb

```
# DWARF is a general format, and many tools consume it:

    pahole -C mystruct prog.o    ; STRUCT LAYOUT with holes
                                 ; marked - the best tool for
                                 ; finding padding
                                 ; (chapter 10)
    pahole --reorganize          ; suggests a better order

    perf report --call-graph dwarf
    ; unwinds using CFI rather than frame pointers, so it
    ; works on binaries built with -fomit-frame-pointer

    bpftrace, systemtap
    ; use DWARF to attach probes to source lines and read
    ; local variables in a running kernel

    addr2line -e prog -f -i 0x1234
    ; -i shows INLINE frames too - essential for making sense
    ; of an optimised backtrace

    llvm-symbolizer < addresses.txt
    ; what the sanitizers use to turn addresses into
    ; file:line

    eu-stack, gcore, coredumpctl
    ; post-mortem analysis

# AND THE KERNEL:
    CONFIG_DEBUG_INFO_DWARF5=y
    CONFIG_GDB_SCRIPTS=y         ; lx-dmesg, lx-symbols
    CONFIG_DEBUG_INFO_BTF=y      ; BTF - a compact type format
                                 ; derived from DWARF, used by
                                 ; BPF and by kernel tooling.
                                 ; A fraction of DWARF's size,
                                 ; enough for type
                                 ; introspection.
```

How it shows up

Symptom	Cause
"No symbol table is loaded"	Built without `-g`, or stripped and the debuglink is missing
`<optimized out>`	The value does not exist there — use `-Og`
Stepping jumps around	The line table genuinely is out of order after scheduling
A breakpoint lands in the wrong place	Line-table granularity; try an address breakpoint
Backtrace stops in assembly	Missing `.cfi_*` directives
Backtrace stops after one frame	`-fomit-frame-pointer` and no CFI, or a corrupted stack
Inlined function missing from a backtrace	Use `addr2line -i`, or a debugger that shows inline frames
Cannot set more than a few watchpoints	Hardware debug register limit
Watchpoint makes the program crawl	Fell back to a software watchpoint
`gdb -p` refuses to attach	`ptrace_scope` — check the yama setting
Debug build links very slowly	Debug info volume — use `-gsplit-dwarf`

Symptom	Cause
Symbols do not match the binary	Different build — check the build-id

Check yourself

- ✓ Name DWARF's main sections and what each holds.
- ✓ Read a DIE tree and identify a function, its parameters and its types.
- ✓ Explain the line table and why `prologue_end` exists.
- ✓ Explain location lists and what `<optimized out>` really means.
- ✓ Explain the two unwinding mechanisms and when each is available.
- ✓ Say what `.cfi_*` directives produce and what breaks without them.
- ✓ Explain how a software breakpoint is implemented, step by step.
- ✓ Explain when hardware breakpoints are required and why they are limited.
- ✓ Split debug info out of a binary and have gdb find it again.
- ✓ Use `pahole` and `addr2line -i` on a real binary.

Undefined behaviour and the optimiser

Why a compiler deletes your bounds check, why a bug appears only at `-O2`, and why "it worked before" is not evidence. Undefined behaviour is not a bug in the compiler — it is a licence you granted it, and this chapter is about what it does with that licence.

The contract

```
THE C STANDARD says: if a program executes undefined
behaviour, the standard imposes NO REQUIREMENTS on what
happens.

THE OPTIMISER READS THAT CONTRAPOSITIVELY:

  "The programmer promised this program has no UB.
   Therefore I may ASSUME the conditions that would cause
   UB never occur, and optimise on that assumption."

That is not aggressive interpretation - it is the only
interpretation that makes optimisation possible at all.
Every optimisation is "this transformation preserves
behaviour for all DEFINED executions".

THE CONSEQUENCES that surprise people:

  · UB is not "does something unpredictable at that line".
    It can affect code BEFORE it, because the compiler
    reasons backwards.

  · UB in a branch that is never taken can still change
    your program, because the compiler used it to conclude
    the branch is unreachable.

  · "It works" is not evidence of correctness. It is
    evidence that this compiler, at this optimisation
    level, this week, happened to choose an outcome you
    liked.

FOUR CATEGORIES, and the difference matters:
  UNDEFINED         anything may happen
  UNSPECIFIED       one of several behaviours, no record
                    (argument evaluation order)
  IMPLEMENTATION-DEFINED   one of several, DOCUMENTED
                    (whether char is signed)
  LOCALE-SPECIFIC
```

The ones that bite

```
/* --- 1. SIGNED INTEGER OVERFLOW ------------------------- */
        int f(int x) { return x + 1 > x; }
   ->   mov w0, #1
        ret
/* ALWAYS TRUE. Signed overflow is UB, so the compiler may
   assume x + 1 > x. For UNSIGNED it must emit a real
   comparison, because unsigned wrapping is DEFINED.

        for (int i = 0; i <= n; i++)
/* With int i, the compiler may assume i never wraps, which is
   what lets it prove the loop terminates and unroll it. With
   `unsigned i` and n == UINT_MAX, the loop is infinite and
   the compiler must preserve that.

   THE FLAG: -fwrapv makes signed overflow wrap (defined),
   at some optimisation cost. -ftrapv traps. The kernel uses
   -fno-strict-overflow.

/* --- 2. NULL DEREFERENCE IMPLIES NON-NULL --------------- */
        void f(int *p) {
                int x = *p;             /* dereference */
                if (p == NULL)          /* DELETED - if p were
                        return;            NULL we already had
                *p = x + 1;                UB               */
        }
/* The check is removed. This exact pattern caused
   CVE-2009-1897 in the Linux kernel: a dereference before a
   NULL check let the compiler delete the check, and the
   dereference of a mapped NULL page became exploitable.

/* --- 3. STRICT ALIASING --------------------------------- */
        float bits_to_float(uint32_t u)
        {
                return *(float *)&u;    /* UB */
        }
/* Objects of different types may not alias, so the compiler
   may assume the float load does not see the uint32_t store,
   and reorder or elide them.

   THE CORRECT WAYS: */
        memcpy(&f, &u, sizeof f);       /* always correct, and
                                           compiles to zero
                                           instructions      */
        f = __builtin_bit_cast(float, u);   /* C++20 / clang */
        union { uint32_t u; float f; } c = { .u = u };
        return c.f;                     /* legal in C, NOT in
                                           C++               */

/* char, unsigned char and std::byte may alias ANYTHING - that
   is the exception that makes memcpy work.

   THE KERNEL BUILDS WITH -fno-strict-aliasing because it
   type-puns deliberately in many places. Application code
   should NOT rely on that flag.

/* --- 4. OUT-OF-BOUNDS ACCESS ---------------------------- */
        int a[10];
        a[10] = 0;              /* UB, and the compiler may
                                   assume it does not happen */
/* Which is why a bounds check AFTER the access is deleted,
```

```
   and why `if (i < 10 || i > 10)` can be simplified in
   surprising ways once the compiler knows an access at a[i]
   already happened.

/* --- 5. UNINITIALISED VALUES ---------------------------- */
        int x;
        if (x) ...              /* UB - and `x` is not merely
                                   a random number: it may
                                   behave INCONSISTENTLY,
                                   because it is "poison" in
                                   the IR and may be folded
                                   differently at each use  */

/* --- 6. SHIFT COUNT ------------------------------------- */
        1 << 32                 /* UB for 32-bit int */
        x >> -1                 /* UB */
        1 << 31                 /* UB in C99 for signed int;
                                   use 1u << 31            */
/* And note the HARDWARE differs: ARM masks the shift count to
   the register width, x86 masks to 5 or 6 bits. So the same
   UB gives different answers on different targets.

/* --- 7. INFINITE LOOPS WITHOUT SIDE EFFECTS ------------- */
        while (1) { }           /* UB in C++; in C, UB only if
                                   the condition is not a
                                   constant                */
/* The compiler may assume any loop terminates, and delete or
   fall through it. This has produced genuinely astonishing
   output when someone wrote an empty spin loop.

/* --- 8. DATA RACES -------------------------------------- */
        /* two threads, no synchronisation, one writes */
/* UB. Not "you get one value or the other" - the compiler may
   split a store into two, invent a store, or cache the value
   in a register indefinitely. Use _Atomic, or a lock.

/* --- 9. UNSEQUENCED MODIFICATION ------------------------ */
        i = i++ + 1;            /* UB */
        f(i++, i++);            /* UB */
        a[i] = i++;             /* UB */
/* Not merely "unspecified order" - genuinely undefined.

/* --- 10. FUNCTION POINTER TYPE MISMATCH ----------------- */
        void (*f)(void) = (void (*)(void))some_int_fn;
        f();                    /* UB */
/* And with control-flow integrity enabled, it now TRAPS -
   which is how a long-latent bug becomes a crash after a
   toolchain upgrade.                                      */
```

THE PATTERN BEHIND ALL OF THEM

The compiler is not looking for your UB and punishing it. It is doing ordinary reasoning — "this pointer was dereferenced, so it is non-null"; "this loop counter is `int`, so it does not wrap" — and propagating the conclusion. The transformation is correct for every program without UB. Yours has UB, so it is outside the set the compiler promised anything about.

Finding it

```
# --- UBSAN: the single most valuable tool here ---
    gcc -fsanitize=undefined -fno-sanitize-recover=all -g
    clang -fsanitize=undefined,integer,nullability -g

# It instruments the program and reports at RUN TIME:
    prog.c:10:12: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'

# WHAT IT CHECKS:
    signed integer overflow      shift out of range
    null dereference             misaligned access
    array bounds (with known sizes)
    division by zero             invalid enum/bool values
    float-to-int overflow        unreachable code reached
    object size violations       vptr misuse (C++)
    function pointer type mismatch

# COST: roughly 20% slower. Cheap enough to run all your tests
# under. THE MINIMAL PRODUCTION SUBSET:
    -fsanitize=undefined -fsanitize-minimal-runtime \
    -fno-sanitize-recover=all
# which traps rather than printing, and costs very little.

# --- ASAN: memory errors ---
    -fsanitize=address -fno-omit-frame-pointer
# use-after-free, buffer overflow, double free, leaks.
# ~2x slower, 3x memory. Reports with the allocation and free
# stacks.

# --- TSAN: data races ---
    -fsanitize=thread
# ~10x slower, but it finds races that no amount of testing
# will.

# --- MSAN: uninitialised reads (clang) ---
    -fsanitize=memory
# Requires ALL code including libraries to be instrumented,
# which is the practical obstacle.

# --- STATIC ANALYSIS ---
    gcc -fanalyzer             ; GCC 10+, genuinely good
    clang --analyze
    clang-tidy
    cppcheck
    scan-build make

# --- COMPILER WARNINGS THAT SPECIFICALLY CATCH UB ---
    -Wall -Wextra
    -Wstrict-aliasing=2
    -Warray-bounds=2
    -Wnull-dereference
    -Wshift-overflow=2
    -Wsequence-point
    -Wuninitialized -Wmaybe-uninitialized

# THE COMBINATION THAT FINDS THE MOST, in practice:
#   -Wall -Wextra -Werror in CI
#   + all tests under UBSan and ASan
#   + a nightly TSan run if threaded
```

```
# That catches the overwhelming majority of what this chapter
# describes.
```

Writing code that has none

```
/* --- CHECK BEFORE, NOT AFTER --- */
        if (p == NULL) return;
        int x = *p;                     /* right order */

/* --- CHECK OVERFLOW WITHOUT CAUSING IT --- */
        /* WRONG: */
        if (a + b < a) overflow();      /* the add is already
                                           UB               */
        /* RIGHT: */
        if (a > INT_MAX - b) overflow();
        /* OR, best: */
        int result;
        if (__builtin_add_overflow(a, b, &result))
                overflow();
        /* which compiles to one add plus a branch on the
           carry flag - no cost at all on ARM             */

/* --- USE UNSIGNED FOR BIT MANIPULATION --- */
        uint32_t mask = 1u << 31;       /* defined */
        int32_t  bad  = 1 << 31;        /* UB */

/* --- TYPE-PUN WITH memcpy --- */
        uint32_t u;
        memcpy(&u, &f, sizeof u);
        /* every compiler recognises this and emits ZERO
           instructions for it when the types are
           register-sized                                 */

/* --- PASS SIZES, ALWAYS --- */
        void f(int *a, size_t n);       /* not `int a[]` and
                                           a guess          */

/* --- USE THE SAFE STRING FUNCTIONS --- */
        strscpy(dst, src, sizeof dst);      /* kernel */
        snprintf(buf, sizeof buf, ...);
        /* never strcpy, strcat, sprintf, gets */

/* --- ENABLE THE FLAGS THAT NARROW UB --- */
        -fwrapv                 signed overflow wraps
        -fno-strict-aliasing    type punning permitted
        -fno-delete-null-pointer-checks
        -ftrivial-auto-var-init=zero     initialise locals
/* THE KERNEL USES ALL FOUR. They cost some optimisation and
   remove whole bug classes, which for a codebase of that size
   and lifetime is clearly the right trade. For your code,
   decide deliberately rather than by default.

/* --- AND THE STRUCTURAL ONES --- */
/*   · prefer array indices to pointer arithmetic
     · keep buffer sizes next to buffers, or use a struct
     · make invalid states unrepresentable where you can
     · initialise at declaration
     · one exit point for resources (C ch. 26 / cmp ch. 06) */
```

The language design answer

Language	Approach to UB
C, C++	Extensive UB, for optimisation freedom and to accommodate every historical machine
Java, C#	Almost none: bounds checked, integers wrap, null throws. Pays runtime cost
Go	Almost none; bounds checked, but data races are still UB
Rust	None in *safe* code — the compiler proves it away. UB is confined to `unsafe` blocks, which are auditable and rare (Rust ch. 02)
Zig	UB is checked in debug builds, assumed in release — an explicit, per-build choice
Ada/SPARK	Checked by default; provably absent with formal verification

This is the strongest single argument for Rust in systems programming, and it is worth stating precisely: Rust does not eliminate the hardware realities — it moves the burden of proving the absence of UB from the programmer's discipline to the compiler's type system, and confines the cases it cannot prove to blocks you must write `unsafe` on. The bugs in this chapter do not become easier to find; they become impossible to write outside a marked region.

How it shows up

Symptom	Likely UB
Works at `-O0`, fails at `-O2`	Almost certainly UB somewhere
A NULL check is missing from the assembly	A dereference before it
A bounds check disappears	An out-of-bounds access elsewhere let the compiler conclude the index range
A loop is deleted or becomes infinite	Signed overflow, or a loop with no side effects
A comparison folds to a constant	Signed overflow assumption
Type punning gives stale values	Strict aliasing
Different results on ARM and x86	Shift count, or signedness of `char`
Breaks after a compiler upgrade	A new optimisation exploited existing UB
Breaks when LTO is enabled	The optimiser can finally see the contradiction
Crashes only under load, in threads	A data race
Traps after enabling CFI	Function pointer type mismatch
Value differs between two uses	Uninitialised — it is poison, not a fixed random number

Check yourself

- ✓ State the contract and explain why the optimiser's reading is the only workable one.
- ✓ Distinguish undefined, unspecified and implementation-defined behaviour.
- ✓ Explain why `x + 1 > x` folds to true for `int` and not `unsigned`.
- ✓ Explain the deleted-null-check pattern and the kernel CVE it caused.
- ✓ Explain strict aliasing and give three correct ways to type-pun.
- ✓ Explain why UB can affect code before it in the source.
- ✓ List ten kinds of UB in C and recognise each in code.

- ✓ Enable UBSan and ASan and run a test suite under them.
- ✓ Rewrite an overflow check so it does not itself overflow.
- ✓ Name four flags that narrow UB and say what each costs.
- ✓ Explain how Rust changes the situation, precisely.

Reading compiler output like a professional

The practical skill this whole track exists to give you: look at generated AArch64 assembly and understand what the compiler did and why. It is the only way to settle a performance argument, and it is faster than you expect once you know the patterns.

The workflow

```
# --- godbolt.org, for anything small ---
# Source left, assembly right, colour-matched by line. Set the
# compiler to "ARM64 gcc" or "armv8-a clang", flags to -O2.
# Nothing else comes close for exploring a question.

# --- Locally, for real code ---
    gcc -S -O2 -fverbose-asm prog.c -o prog.s
    ; -fverbose-asm annotates each instruction with the
    ; variable names involved

    objdump -dS --demangle prog.o | less
    ; disassembly INTERLEAVED with source (needs -g)

    objdump -d --no-show-raw-insn --demangle prog.o

# --- One function only ---
    objdump -d prog | awk '/<myfunc>:/,/^$/'
    gdb -batch -ex 'disassemble /s myfunc' prog

# --- With performance data attached ---
    perf record ./prog
    perf annotate myfunc
    ; shows the percentage of samples on each instruction -
    ; which turns "this function is slow" into "this load is
    ; slow"

# --- Cycle-level analysis ---
    llvm-mca -mcpu=cortex-a53 -timeline prog.s
    ; a pipeline simulation with the bottleneck named

# --- ALWAYS COMPILE WITH THE FLAGS YOU SHIP. Reading -O0
# output tells you nothing about your release build, and
# reading -O2 output while shipping -Os tells you almost as
# little.
```

Recognising the skeleton

```
; int add(int a, int b) { return a + b; }
add:
        add     w0, w0, w1              ; args in w0,w1;
        ret                             ; result in w0

; A LEAF FUNCTION: no prologue, no stack, no calls. If you see
; this shape, the function is as cheap as it can be.

; ------------------------------------------------------------
; int call_it(int x) { return helper(x) + 1; }
call_it:
        stp     x29, x30, [sp, #-16]!   ; PROLOGUE: save FP,LR
        mov     x29, sp
        bl      helper                  ; the call
        add     w0, w0, #1
        ldp     x29, x30, [sp], #16     ; EPILOGUE
        ret

; A NON-LEAF FUNCTION. The stp/ldp pair is the giveaway.
; ------------------------------------------------------------
; A function needing callee-saved registers:
big:
        stp     x29, x30, [sp, #-64]!
        mov     x29, sp
        stp     x19, x20, [sp, #16]     ; more saves ->
        stp     x21, x22, [sp, #32]     ; more simultaneous
        str     x23, [sp, #48]          ; live state
        ...
; THE PROLOGUE SIZE IS A PRESSURE GAUGE (chapter 09). Five
; saved registers and a 64-byte frame means the function is
; carrying a lot at once - a candidate for splitting.
```

The patterns worth knowing

You see	It means
`ldr w0, [x1, x2, lsl #2]`	Array index, scaling folded in — free
`adrp x0, sym` + `add x0, x0, :lo12:sym`	Forming a global's address — PC-relative
`madd`, `msub`	Multiply-accumulate fused into one instruction
`csel`, `csinc`, `cinc`	A branch was if-converted — no misprediction
`ubfx`, `sbfx`, `bfi`	Bitfield extract/insert in one instruction
`ldp` / `stp`	Paired access — two loads for the price of one
`cbz`, `tbz`	Compare-with-zero or test-bit and branch
`wzr`, `xzr`	The zero register — no register needed to store 0
`v0.4s`, `q0`, `faddp`	**Vectorised** (ch. 07)
`bl memcpy` where you wrote a loop	Idiom recognition
`str` / `ldr` to `[sp, #n]` in a loop	**Spilling** (ch. 09)
`dmb ish`, `dsb`, `isb`	A memory barrier (ARM ch. 21)
`ldaxr` / `stlxr` loop	An atomic read-modify-write (ARM ch. 22)

You see	It means
`paciasp` / `autiasp`	Pointer authentication — return-address signing
`bti c`	Branch target identification — an indirect-branch landing pad
`brk #0x3e8` / `udf`	A trap the compiler inserted — UBSan, or unreachable code

A worked example

```
/* SOURCE: */
int count_set_bits(const uint32_t *data, int n)
{
        int count = 0;

        for (int i = 0; i < n; i++)
                count += __builtin_popcount(data[i]);

        return count;
}

; aarch64 gcc -O2 -mcpu=cortex-a72
count_set_bits:
        cmp     w1, #0
        b.le    .Lreturn_zero           ; guard: n <= 0
        mov     x2, #0                  ; i
        mov     w3, #0                  ; count
.Lloop:
        ldr     w4, [x0, x2, lsl #2]    ; data[i] - scaled
        fmov    s0, w4                  ; move to a SIMD reg
        cnt     v0.8b, v0.8b            ; popcount per BYTE
        addv    b0, v0.8b               ; sum the bytes
        fmov    w4, s0                  ; back to a GP reg
        add     x2, x2, #1
        add     w3, w3, w4
        cmp     x2, x1, sxtw
        b.lt    .Lloop
        mov     w0, w3
        ret
.Lreturn_zero:
        mov     w0, #0
        ret

/* WHAT THIS TELLS YOU:

   1. __builtin_popcount became `cnt` + `addv` on the SIMD
      unit, because AArch64 has no scalar population count.
      Four instructions including the register moves.

   2. THE fmov PAIR IS THE COST. Moving between general and
      SIMD registers has real latency (typically 3-5 cycles
      each way on an A72), and it happens EVERY iteration.

   3. IT DID NOT VECTORISE. Processing four words at once
      would amortise the moves, and it did not - because the
      reduction plus the per-element move confused the
      vectoriser.

   4. THE FIX, and this is the point of reading the output:
      restructure so the popcount stays in the vector domain: */

int count_set_bits_v2(const uint32_t *data, int n)
{
        uint32x4_t acc = vdupq_n_u32(0);
        int i;

        for (i = 0; i <= n - 4; i += 4) {
                uint8x16_t v = vreinterpretq_u8_u32(
                                        vld1q_u32(data + i));
                uint8x16_t c = vcntq_u8(v);
```

```
                acc = vaddq_u32(acc,
                        vpaddlq_u16(vpaddlq_u8(c)));
        }

        int count = vaddvq_u32(acc);

        for (; i < n; i++)              /* the tail */
                count += __builtin_popcount(data[i]);

        return count;
}
/* Four words per iteration, no domain crossing in the loop,
   one horizontal reduction at the end. Typically 3–4x faster.

   YOU WOULD NOT HAVE KNOWN TO DO THIS without reading the
   assembly. The source looked fine.                       */
```

WHAT READING ASSEMBLY IS ACTUALLY FOR

Not to hand-optimise — the compiler usually wins at that. It is to answer three questions no other tool answers: *did the optimisation I expected happen?*, *what is this loop actually doing per iteration?*, and *is the cost where I think it is?* Every performance investigation that gets past guesswork goes through this step.

The questions to ask

```
/* Work down this list on any hot function: */

1. IS IT A LEAF? No stp/ldp prologue means no calls, no
   frame. Good.

2. HOW BIG IS THE FRAME? `sub sp, sp, #N` or the stp
   immediate. Large frames mean spilling or large locals.

3. WHAT IS IN THE LOOP BODY? Count the instructions between
   the label and the backward branch. That is your per-
   iteration cost, roughly.

4. ANY STACK TRAFFIC IN THE LOOP? str/ldr to [sp, #n] means
   spilling. Reduce pressure (chapter 09).

5. ANY CALLS IN THE LOOP? `bl` prevents most optimisation
   across it and clobbers the caller-saved registers.

6. DID IT VECTORISE? Look for v registers and q loads. If not,
   ask why with -fopt-info-vec-missed (chapter 07).

7. ARE THE ADDRESSING MODES BEING USED? `[x0, x1, lsl #2]`
   good; a separate lsl then add means something prevented
   folding.

8. ARE THERE REDUNDANT MOVES? A `mov` between two registers
   that could have been the same one suggests a coalescing
   failure or a domain crossing.

9. ANY BARRIERS OR ATOMICS? dmb/ldaxr are expensive and often
   unintended - a stray volatile or an atomic where a plain
   variable would do.

10. IS THE HOT PATH THE FALL-THROUGH? A taken branch on the
    common path costs more than a not-taken one. PGO fixes
    this (chapter 06); __builtin_expect hints it.

11. WHAT DOES perf annotate SAY? The instruction with the
    samples on it is often a load - i.e. the problem is
    memory, not code (CA chapter 14), and no amount of
    instruction tuning will help.
```

Comparing two versions

```
# The most productive use of all: does my change help?

    gcc -O2 -S before.c -o before.s
    gcc -O2 -S after.c  -o after.s
    diff -u before.s after.s

# Or count instructions in the hot loop:
    objdump -d prog | awk '/\.Lloop/,/b\.lt/' | wc -l

# Or measure sizes:
    size -A before.o after.o
    nm --size-sort -S prog | tail

# AND THEN MEASURE THE ACTUAL PROGRAM, because instruction
# count is a proxy, not the answer:
    perf stat -r 10 ./prog
      cycles, instructions, IPC, branch-misses,
      cache-misses, stalled-cycles

# THE THREE NUMBERS THAT DIAGNOSE MOST THINGS:
#   IPC low + high cache-misses    -> memory bound
#   IPC low + high branch-misses   -> branchy code
#   IPC high but slow              -> genuinely doing too much
#                                     work; look at the
#                                     algorithm

# A CHANGE THAT REMOVES INSTRUCTIONS AND DOES NOT REMOVE TIME
# means the bottleneck is elsewhere. That is worth
# discovering before spending a day on it.
```

Traps when reading

```
/* --- 1. THE FUNCTION IS NOT THERE --- */
/* It was inlined. Look at the caller, or build with
   -fno-inline to see it in isolation - accepting that you are
   now looking at different code.

/* --- 2. THE LOOP IS NOT THERE --- */
/* Constant-folded, deleted as dead, or turned into a closed
   form. If you wrote a benchmark whose result is unused, the
   compiler deleted the whole thing. Defeat it: */
        __asm__ volatile("" : : "r"(result) : "memory");
        /* or use a benchmark framework's DoNotOptimize() */

/* --- 3. THE CODE IS DUPLICATED --- */
/* Loop unswitching, vectorisation with a scalar epilogue,
   runtime alias checks (chapter 07) - all produce several
   copies. Find the one that runs.

/* --- 4. YOU ARE READING -O0 --- */
/* Every variable in memory, every expression separate. It
   tells you nothing about your release build.

/* --- 5. LINE ATTRIBUTION IS APPROXIMATE --- */
/* After scheduling, an instruction's source line is where it
   CAME FROM, not where it runs. objdump -S interleaving looks
   scrambled for this reason (chapter 14).

/* --- 6. THE STATIC FUNCTION VANISHED --- */
/* If a `static` function has one caller it is usually inlined
   and the out-of-line copy deleted. Add
   __attribute__((noinline)) temporarily to study it.

/* --- 7. YOU ARE ON THE WRONG TARGET --- */
/* godbolt defaults to x86-64. Check the compiler selection.
   Half of all "the compiler did something strange" reports
   are this.                                              */
```

A practice routine

```
# Twenty minutes, and it teaches more than reading about it.

# 1. Open godbolt.org, set ARM64 gcc, -O2.

# 2. Type each of these and read the output:
    int f(int x) { return x * 2; }          ; lsl
    int f(int x) { return x / 2; }          ; the signed
                                            ; correction
    unsigned f(unsigned x) { return x / 2; } ; one lsr
    int f(int x) { return x % 7; }          ; magic-number
                                            ; division
    int f(int a, int b) { return a > b ? a : b; }  ; csel
    int f(int *p) { return p ? *p : 0; }
    int f(void) { int a[4] = {1,2,3,4};
                  return a[0]+a[1]+a[2]+a[3]; }    ; folded?

# 3. Then a loop, and check whether it vectorises:
    void f(float *a, float *b, int n) {
            for (int i = 0; i < n; i++) a[i] += b[i];
    }
    ; add `restrict` and watch the alias check disappear

# 4. Then something from your own codebase.

# THE HABIT TO BUILD: whenever you wonder "is this fast?" or
# "did that help?", spend two minutes looking rather than
# reasoning. It is almost always faster than the argument,
# and it is always more reliable.
```

How it shows up

Symptom	What to look for
Function slower than expected	Stack traffic in the loop; calls; missing vectorisation
Your optimisation changed nothing	The compiler was already doing it — diff the assembly
Benchmark shows zero time	The loop was deleted; the result is unused
Cannot find the function	Inlined
Several copies of the loop	Unswitching, vectorisation epilogue, or alias-check versioning
Unexpected `bl` to a libc function	Idiom recognition, or a division/modulo helper
Unexpected barriers	A stray `volatile`, or an atomic type
Register moves everywhere	Domain crossing (GP↔SIMD), or coalescing failure
Assembly looks fine but it is still slow	Memory bound — check `perf stat` cache misses
x86 instructions in the output	Wrong target selected

Check yourself

- ✓ Get from a source file to annotated assembly three different ways.
- ✓ Recognise a leaf function, a non-leaf, and a register-pressured one at a glance.
- ✓ Identify fifteen AArch64 patterns and say what each implies.

- ✓ Work the eleven diagnostic questions on a real hot function.
- ✓ Spot spilling, vectorisation and idiom recognition in output.
- ✓ Diff two versions of a function and quantify the change.
- ✓ Use `perf stat` to decide whether you are memory or compute bound.
- ✓ Name seven traps that make assembly misleading, and how to avoid each.
- ✓ Find a real optimisation opportunity in your own code by reading its output.
- ✓ Explain what reading assembly is for, and what it is not for.

> **END OF THE COMPILERS TRACK**
>
> Sixteen chapters, from preprocessing to reading the result. You now know what happens to your source, which phase to blame for a given behaviour, and how to check rather than guess. Next: C++, where the same machinery has to cope with a far larger language.

TRACK

ARM architecture

The A-profile architecture concept by concept: registers, translation, caches, ordering, interrupts and the exception model.

35 chapters

What ARM is: architecture versus microarchitecture

Arm Ltd does not make chips. It writes a specification and licenses core designs that implement it. Keeping those two things apart is the first thing to get right, because it tells you which of your bugs are portable and which are specific to the silicon in front of you.

Two products, often confused

	Architecture	Microarchitecture
Is	A specification — a contract about behaviour	An implementation — actual logic gates
Defines	Instructions, registers, memory model, exception behaviour	Pipeline depth, cache sizes, prefetchers, branch predictors
Document	The *Arm Architecture Reference Manual* ("the ARM ARM")	A core's Technical Reference Manual, plus its errata
Determines	**Correctness**	**Performance**
Examples	ARMv8-A, ARMv9-A, ARMv8-M	Cortex-A78, Neoverse N2, Apple Firestorm, Qualcomm Oryon

WHY THIS MATTERS WHEN YOU ARE DEBUGGING

Ask which one you are fighting.

Architectural problems are portable: if your code violates the memory model, it is wrong on every ARM implementation, and the fix is in your code. **Microarchitectural** surprises — timing, cache line effects, an erratum — are specific to one core, and the fix may be a workaround, a different compiler flag, or a kernel quirk.

A useful test: *would this still be a bug on a hypothetical, perfectly simple in-order implementation of the same architecture?* If yes, it is architectural and it is your problem to fix properly.

The licensing model, and why it shapes the ecosystem

Licence type	Buys	Licensees
Core licence	A ready-made core design (Cortex-A78) to drop into your SoC	Most SoC vendors: NXP, ST, TI, Rockchip, MediaTek
Architecture licence	The right to design your *own* core implementing the ARM architecture	Apple, Qualcomm, Nvidia, Ampere, Fujitsu
Subsystem / CSS	A pre-integrated cluster, interconnect and memory system	Vendors wanting a shorter path to silicon

The architecture licence is why Apple's cores are dramatically faster than a Cortex core of the same generation while running identical binaries: same contract, very different implementation. It is also why "ARM performance" is a meaningless phrase — you must name the core.

```
One architecture, many implementations, all running your binary:

   ARMv8-A specification
        │
        ├── Cortex-A53   in-order, 8-stage, tiny, power-efficient
        ├── Cortex-A78   out-of-order, 4-wide, deep, fast
        ├── Neoverse V2  server-class, wide vectors, huge caches
        ├── Apple M-series  very wide, very large reorder buffer
        └── Qualcomm Oryon  custom, architecture-licensed

Same instructions. Same page table format. Same barriers.
10-50x difference in single-thread performance, and completely
different cache hierarchies.
```

What the architecture actually specifies

- **The instruction set** — encodings and precise semantics.
- **The register file** — what exists and what each register means.
- **The memory model** — what reorderings are permitted, and what barriers guarantee (ch. 20).
- **The exception model** — exception levels, vectors, what hardware saves (ch. 26).
- **Address translation** — page table formats, descriptor bits, TLB maintenance (ch. 13).
- **System registers** — names, fields, and which level may access them.
- **Required behaviour on reset**, and what is *IMPLEMENTATION DEFINED*.

> **IMPLEMENTATION DEFINED IS A FORMAL TERM**
>
> The ARM ARM uses precise vocabulary that is worth learning, because it tells you exactly how much you may rely on something:
>
> - **IMPLEMENTATION DEFINED** — the implementer chooses and must document it. Cache size, number of breakpoints, whether a feature exists.
> - **UNPREDICTABLE** — behaviour is not specified, but must not be a security hole or hang the machine. Never write code that relies on it.
> - **CONSTRAINED UNPREDICTABLE** — one of a specified, enumerated set of behaviours. Still do not rely on it.
> - **RES0 / RES1** — reserved bits you must write as zero or one respectively, so that future architecture versions can use them without breaking your code. Ignoring this is why some old firmware fails on new silicon.

Never assume — ask the CPU

```
/* The architecture provides ID registers describing exactly which
   optional features this implementation has. Software must query
   them rather than assume, because the SAME architecture version
   allows different feature sets.                                  */

MIDR_EL1          /* implementer, part number, revision — "who am I" */
MPIDR_EL1         /* affinity: which core in which cluster           */
ID_AA64ISAR0_EL1  /* atomics (LSE), CRC32, crypto, SHA               */
ID_AA64ISAR1_EL1  /* pointer authentication, JSCVT, DPB              */
ID_AA64MMFR0_EL1  /* physical address size, granule sizes supported  */
ID_AA64MMFR1_EL1  /* VHE, hardware access/dirty bit management       */
ID_AA64PFR0_EL1   /* which exception levels exist, SVE, FP           */
CTR_EL0           /* cache line sizes — readable from EL0            */

/* Example: does this core have the Large System Extensions? */
static bool has_lse(void)
{
        uint64_t isar0;
        asm volatile("mrs %0, id_aa64isar0_el1" : "=r"(isar0));
        return ((isar0 >> 20) & 0xF) >= 2;   /* Atomic field */
}
/* Linux and Windows both do exactly this at boot and PATCH the
   binary — replacing an LDXR/STXR loop with a single LDADD where
   the hardware supports it. One kernel image, optimal on each. */
```

```
# see what your machine reports
cat /proc/cpuinfo
#   CPU implementer : 0x41        (0x41 = 'A' = Arm Ltd; 0x61 = Apple)
#   CPU architecture: 8
#   CPU variant     : 0x1
#   CPU part        : 0xd0b       (0xd0b = Cortex-A76)
#   Features        : fp asimd atomics crc32 sha1 sha2 ...
#                     ^^^^^^^ "atomics" means LSE is present

lscpu
cat /sys/devices/system/cpu/cpu0/regs/identification/midr_el1
```

Errata

> **READ THE ERRATA DOCUMENT. ALWAYS.**
> Every core has documented hardware bugs, published as an errata notice, each with a software workaround. They are not rare and they are not theoretical — the Linux kernel contains hundreds of them, gated by `MIDR` checks so the workaround applies only on affected revisions.

```
grep -rn "ARM64_WORKAROUND" arch/arm64/kernel/cpu_errata.c | head
cat /sys/devices/system/cpu/vulnerabilities/*

/* A real example: Cortex-A53 erratum 843419 — a specific sequence
   of ADRP plus a load can compute the wrong address. The fix is a
   LINKER workaround (--fix-cortex-a53-843419), which is why some
   toolchains pass it by default for that core.

   The lesson: when hardware behaves impossibly, check the errata
   before assuming your code is at fault. And when bringing up new
   silicon, read the whole document once, up front.          */
```

Why ARM ended up everywhere

Property	Consequence
Licensable IP	Any company can build a custom SoC without designing a CPU. This, more than any technical property, is why ARM won mobile and embedded.
Power efficiency	Simple decode, load/store design, and a culture of performance-per-watt
Scalability	The same architecture family spans a coin-cell sensor and a 128-core server
Clean 64-bit ISA	AArch64 was designed fresh in 2011, without decades of compatibility baggage
Ecosystem	Linux, Windows, Android, every RTOS, every toolchain

Using the ARM ARM

```
It is over 12,000 pages. It is a REFERENCE, not a textbook, and
reading it linearly is a mistake beginners make once.

  How to use it:
    · start from a QUESTION ("what does ESR_EL1 bit 24 mean?")
    · search the PDF, or use developer.arm.com's HTML version
    · the register descriptions (section D) are the most-used part
    · pseudocode in the manual is NORMATIVE — when prose and
      pseudocode disagree, the pseudocode wins

  Start instead with:
    "Arm Cortex-A Series Programmer's Guide for ARMv8-A"
      — a few hundred readable pages, written as a tutorial
    "Learn the architecture" guides on developer.arm.com
      — short, topic-focused, and genuinely good

  Then use the ARM ARM to answer the questions those raise.
```

How it shows up

Symptom	Cause
Code works on one ARM board, fails on another	Architectural violation exposed by a different implementation, or a missing feature check
Illegal instruction on some cores only	Used an optional feature without querying the ID registers
Timing-dependent bug appears only on the big cores	Microarchitectural: deeper out-of-order window widens the race
Hardware behaves impossibly	Check the errata before doubting yourself
Firmware fails on newer silicon	RES0 bits written as non-zero
Benchmark differs 10× between two "ARM" chips	Different microarchitectures — the name of the core matters
Atomics are slow under contention	Exclusive-based path; the core lacks LSE, or the kernel did not patch

Check yourself

- ✓ State the difference between architecture and microarchitecture, and give an example bug of each kind.
- ✓ Explain what an architecture licence permits that a core licence does not.
- ✓ Define IMPLEMENTATION DEFINED, UNPREDICTABLE and RES0, and say how much you may rely on each.
- ✓ Name three ID registers and what they tell you.
- ✓ Explain why a kernel patches its own code at boot after reading ID registers.
- ✓ Say why errata documents matter and give one example of a linker-level workaround.
- ✓ Explain the right way to use the ARM ARM.

Profiles and architecture versions

"ARM" names a family, not a thing. A Cortex-M0 and a Neoverse V2 share a vendor and very little else. This chapter maps the territory so that when someone says "ARMv8" you know which of several very different machines they mean.

The three profiles

	A — Application	R — Real-time	M — Microcontroller
Optimised for	Throughput, rich OS	Deterministic latency	Cost, power, small code
Memory protection	**Full MMU**, virtual memory	MPU, sometimes MMU	Optional MPU, **no MMU**
Privilege model	EL0–EL3	EL0–EL2 (v8-R)	Thread / Handler mode, optional Secure
Instruction set	A64 (and A32/T32)	A32/T32, A64 in v8-R	**Thumb-2 only**
Interrupt controller	GIC	GIC	**NVIC** — vectored, in-core
Typical software	Linux, Android, Windows	RTOS, bare metal, safety islands	Bare metal, FreeRTOS, Zephyr
Examples	Cortex-A53/A78, Neoverse	Cortex-R5/R52/R82	Cortex-M0+/M4/M33/M55

THE SINGLE DISTINCTION THAT MATTERS MOST

MMU or no MMU. With one, you get virtual memory, process isolation, demand paging and everything in the OS track. Without one, every address is physical, a wild pointer can corrupt anything, and "protection" means an MPU checking region permissions with no translation. Almost every other difference follows from this one.

M-profile: different enough to surprise you

```
Cortex-M is not "a small Cortex-A". Concretely different:

  EXCEPTIONS       the hardware stacks R0-R3, R12, LR, PC, xPSR
                   automatically, so an ISR is a PLAIN C FUNCTION.
                   No vector table of code — a table of ADDRESSES.

  VECTOR TABLE     entry 0 is the initial STACK POINTER, entry 1 is
                   the reset handler. The CPU loads SP from memory
                   before executing anything.

  NVIC             priorities, tail-chaining (skip the unstack/stack
                   between back-to-back interrupts), late arrival.
                   Interrupt latency ~12 cycles, deterministic.

  TWO STACKS       MSP (main, used by handlers) and PSP (process).
                   An RTOS gives each task a PSP and keeps MSP for
                   exceptions.

  BIT-BANDING      (v7-M) an alias region where each word maps to
                   one bit — atomic single-bit access without
                   read-modify-write.

  TRUSTZONE-M      a function CALL across the boundary, not an SMC.
                   Tens of cycles, not thousands (ch. 15 of the
                   security track).

  NO A64           Thumb-2 only: mixed 16- and 32-bit encodings,
                   chosen for code density.
```

Architecture versions

Version	Year	Brought
ARMv7-A	2005	32-bit only. Processor *modes* (User, FIQ, IRQ, SVC, Abort, Undef, System, Monitor, Hyp) with banked registers. CP15 coprocessor for system control. NEON, TrustZone, LPAE.
ARMv8-A	2011	**AArch64.** A clean 64-bit ISA, 31 general registers, exception levels replacing modes, system registers replacing CP15, a formally specified memory model.
ARMv8.1	2014	**LSE atomics**, VHE (host OS runs at EL2), hardware access/dirty bit management
ARMv8.2	2016	SVE, half-precision FP, RAS extensions, 52-bit addresses
ARMv8.3	2016	**Pointer Authentication** (PAC), nested virtualisation, JavaScript FP conversion
ARMv8.4	2017	**Secure EL2**, memory partitioning and monitoring (MPAM)
ARMv8.5	2018	**BTI** (Branch Target Identification), **MTE** (Memory Tagging), speculation barriers
ARMv9-A	2021	SVE2 mandatory, **CCA / Realm Management Extension** (a fourth security state), BTI and MTE promoted to baseline

> **VERSIONS ARE CUMULATIVE AND MOSTLY OPTIONAL**
> ARMv8.5 includes everything from 8.1 to 8.4. But many individual features remain *optional* even within a version, which is why software must query the ID registers rather than infer capability from a version number (chapter 01). "This is an ARMv8.2 core" does not tell you whether it has SVE.

Execution states

```
ARMv8-A defines TWO execution states, and a core may implement
one or both:

  AArch64                        AArch32
    A64 instruction set            A32 (ARM) + T32 (Thumb)
    31 x 64-bit registers          16 x 32-bit registers
    fixed 32-bit encodings         mixed 16/32-bit (T32)
    EL0-EL3                        modes mapped onto ELs
    no conditional execution       full predication (IT blocks)
    system registers               CP15 coprocessor

  Switching states happens ONLY on an exception boundary:
    · taking an exception can move AArch32 → AArch64
    · ERET can move AArch64 → AArch32
    · a HIGHER EL may be AArch64 while a lower one is AArch32,
      never the reverse

  So a 64-bit kernel can run 32-bit userspace, but a 32-bit kernel
  cannot run 64-bit applications.

  AArch32 support is now commonly OMITTED at EL0 in new cores
  (Cortex-A510 onward, Apple silicon). 32-bit ARM Linux binaries
  simply will not run there — a real migration issue for
  long-lived embedded products.
```

The security features, briefly

Feature	Defends against	How
PAC (v8.3)	ROP/JOP — corrupting return addresses and function pointers	A cryptographic signature in the unused top bits of a pointer, checked before use
BTI (v8.5)	Jump-oriented programming	An indirect branch may only land on a `BTI` instruction
MTE (v8.5)	Buffer overflow and use-after-free	A 4-bit tag on each 16-byte granule and in each pointer; hardware compares them
PAN (v8.1)	Kernel accidentally dereferencing a user pointer	Kernel access to user memory faults unless explicitly enabled
CCA / RME (v9)	An untrusted hypervisor or host	A fourth world ("Realm") isolated from both Normal and Secure

```
# build with the hardening on
gcc -mbranch-protection=standard    # PAC on returns + BTI
gcc -march=armv8.5-a+memtag         # MTE

# MTE is the most interesting of these: it makes ASan-class
# detection cheap enough to run in PRODUCTION. Android ships it.
cat /proc/cpuinfo | grep -o 'paca\|bti\|mte'
```

Decoding the names

```
Cortex-A78AE
|      | | └── AE = Automotive Enhanced (lockstep, safety)
|      | └──── generation number: higher is newer/bigger
|      └─────── profile: A, R or M
└──────────────── Arm's own core family

Neoverse N2 / V2 / E2
  N = balanced (scale-out cloud)
  V = maximum performance (HPC, high-end server)
  E = efficiency (edge, networking)

Cortex-M33 / M55 / M85
  M23, M33  → ARMv8-M with TrustZone-M
  M55, M85  → ARMv8.1-M with Helium (M-profile vector extension)

DynamIQ Shared Unit (DSU)
  the cluster that ties heterogeneous cores together with a
  shared L3 — the successor to big.LITTLE's separate clusters

Suffixes worth recognising:
  -AE   automotive, functional safety
  -AP   with parity/ECC for safety
  +crypto / +sve / +simd  in compiler -march strings
```

Telling the compiler what to target

```
# architecture version — what INSTRUCTIONS may be emitted
-march=armv8-a
-march=armv8.2-a+crypto+fp16
-march=armv9-a+sve2

# tune for a specific microarchitecture — SAME instructions,
# different scheduling and cost model
-mtune=cortex-a76

# both at once
-mcpu=cortex-a76           # implies -march and -mtune
-mcpu=cortex-a55+crypto

# big.LITTLE: build for the LEAST capable core you must run on,
# tune for the one that matters
-mcpu=cortex-a55 -mtune=cortex-a76

# M-profile
-mcpu=cortex-m4 -mfpu=fpv4-sp-d16 -mfloat-abi=hard -mthumb

# runtime dispatch instead: build a baseline binary and select
# optimised routines after reading the ID registers. This is what
# glibc's ifunc mechanism and every crypto library do.
gcc -march=armv8-a ... ; if (has_crypto()) use_aes_instructions();
```

> **`-MARCH=NATIVE` IS A TRAP IN EMBEDDED WORK**
>
> It targets the build machine, not the deployment target — and on a big.LITTLE system it may target whichever core the compiler happened to run on. The result runs until it is scheduled onto a LITTLE core lacking a feature, then takes an illegal-instruction fault. Always name the target explicitly.

Which profile for which job

Requirement	Profile	Why
Run Linux or Android	**A**	Needs an MMU
Hard real-time, microsecond deadlines	**R** or **M**	Deterministic interrupt latency, no paging
Battery-powered sensor, years of life	**M**	Microamp sleep currents
Motor control beside a rich UI	**A** + **M/R** on one SoC	Asymmetric multiprocessing (OS ch. 19)
Functional safety certification	**R** (-AE parts)	Lockstep cores, ECC, safety documentation
Server or cloud	**Neoverse**	Core count, memory bandwidth, RAS

How it shows up

Symptom	Cause
Illegal instruction on some cores, not others	Optional feature used without an ID-register check
32-bit binary will not run on a new ARM64 chip	AArch32 at EL0 not implemented
Code built with `-march=native` faults in the field	Targeted the build machine, or the big core only
Cortex-M interrupt handler needs no special attribute	Hardware stacks the caller-saved registers
MMU-based isolation unavailable on a Cortex-M	M-profile has no MMU — MPU only
Atomics are slow on an older core	No LSE; falls back to exclusive loops
Two "ARMv8" chips differ 10× in performance	Different microarchitectures entirely
Security hardening flags do nothing	The core predates PAC/BTI/MTE

Check yourself

- ✓ Give the three profiles, their memory-protection model, and typical software for each.
- ✓ Name four concrete ways M-profile differs from A-profile.
- ✓ Say what ARMv8-A changed relative to ARMv7-A, in four points.
- ✓ Explain why a version number does not tell you which features are present.
- ✓ State the rule about which exception levels may be AArch32 versus AArch64.
- ✓ Explain what PAC, BTI and MTE each defend against.
- ✓ Give the difference between `-march`, `-mtune` and `-mcpu`.
- ✓ Explain why `-march=native` is dangerous for embedded targets.

Load/store architecture and the RISC philosophy

One design rule shapes everything else about ARM: only load and store instructions touch memory. Everything else operates on registers. This chapter draws out the consequences, several of which are the direct cause of bug classes you will meet for the rest of your career.

The rule

LOAD/STORE ARCHITECTURE

Memory is accessed *only* by the `LDR` and `STR` instruction families. Every arithmetic, logical and comparison instruction reads its operands from registers and writes its result to a register. There is no `ADD [address], #1`.

```
/* C source */
counter++;

/* AArch64 — three separate instructions, plus address formation */
        adrp    x0, counter               // page address of the symbol
        add     x0, x0, :lo12:counter     // + low 12 bits = full address
        ldr     w1, [x0]                  // LOAD    memory → register
        add     w1, w1, #1                // MODIFY  register only
        str     w1, [x0]                  // STORE   register → memory

/* x86, by contrast, can do it in one instruction: */
        incl    counter(%rip)             // read-modify-write in one op
/* — but note that even on x86 this is NOT atomic without a LOCK
   prefix. The hardware still performs three steps internally.  */
```

What follows from it

1. Read-modify-write is never atomic

```
   Core 0                    Core 1
   ldr  w1, [x0]   → 5
                             ldr  w1, [x0]   → 5
   add  w1, w1, #1 → 6
                             add  w1, w1, #1 → 6
   str  w1, [x0]   → 6
                             str  w1, [x0]   → 6

   Two increments, one result. This is not an ARM quirk — it is
   the direct, visible consequence of the load/store rule, and it
   is why the atomics of chapter 22 exist as separate instructions
   rather than as a property of ADD.
```

2. Address formation is explicit

```
/* A 64-bit address does not fit in a 32-bit instruction, so ARM
   builds addresses from pieces. Three common idioms:          */

/* PC-relative, ±4 GB — how globals are reached in position-
   independent code. ADRP gives the 4 KB PAGE address; the low
   bits come from an ADD or the load's own offset.            */
        adrp    x0, symbol
        add     x0, x0, :lo12:symbol

/* Small immediate offset, folded into the load */
        ldr     x1, [x0, #16]

/* Large constants are built 16 bits at a time */
        movz    x0, #0x1234, lsl #16
        movk    x0, #0x5678

/* This is why disassembly shows ADRP/ADD pairs everywhere, and
   why the linker has relocation types specifically for splitting
   an address across two instructions.                         */
```

3. Fixed 32-bit instruction width

Consequence	Effect
Decode is trivial	Several instructions can be decoded in parallel with no serial dependency — a major reason ARM cores go wide cheaply
Disassembly is unambiguous	You can start disassembling at any 4-byte-aligned address and be correct. On x86, you cannot.
No unaligned instructions	Every instruction address is 4-byte aligned; the low two bits of the PC are always zero
Immediates are limited	Only 12 bits for arithmetic, so large constants need several instructions or a literal pool
Code density is worse than x86	Which is exactly why Thumb-2 exists for M-profile, mixing 16- and 32-bit encodings

4. Memory access is visible and countable

```
/* Because every access is an explicit instruction, you can SEE the
   memory traffic in the disassembly. This makes ARM assembly
   unusually readable for performance work.                    */

objdump -d prog | grep -cE '\b(ldr|ldp|str|stp)'   /* count accesses */

/* And it makes the compiler's decisions visible: if a variable
   stays in a register across a loop, there are no LDRs inside
   the loop. If `volatile` forces a reload, you see one per
   iteration. This is the fastest way to confirm what volatile
   and restrict actually did (C chapters 10 and 18).         */
```

RISC in practice, not in slogans

Classic RISC claim	Reality on modern ARM
"Reduced instruction *set*"	AArch64 has well over a thousand instructions. The set is not small.
"One instruction per cycle"	Wide superscalar cores retire several per cycle; some instructions take many
"Simple instructions only"	`FJCVTZS` (JavaScript float-to-int), `AESE`, `SHA256H`, `LD64B` — these are not simple

Classic RISC claim	Reality on modern ARM
"No microcode"	Largely true, and it does matter: decode stays cheap
"Load/store architecture"	**True, and the one that actually matters**
"Fixed-width instructions"	True for A64; false for Thumb-2
"Many general-purpose registers"	True — 31, versus x86-64's 16

> **THE RISC/CISC DISTINCTION HAS LARGELY DISSOLVED**
>
> Modern x86 cores decode complex instructions into RISC-like micro-operations and execute those. Modern ARM cores have complex instructions. What remains genuinely different is the *front end*: fixed-width, regularly encoded instructions are much cheaper to decode in parallel, which lets ARM cores go wide with less power. That, plus load/store discipline, is the real substance behind the label.

Why 31 registers changes how code looks

```
/* With 16 registers (x86-64), the compiler spills to the stack
   constantly. With 31, most functions keep everything in
   registers. Compare a small function:                          */

/* AArch64 — no memory traffic at all inside the function */
sum4:   add     w0, w0, w1
        add     w0, w0, w2
        add     w0, w0, w3
        ret

/* And the calling convention passes EIGHT arguments in registers
   (chapter 09), so most calls involve no stack traffic either.

   Practical consequence: on AArch64, seeing LDR/STR of stack
   slots inside a hot loop usually means something is wrong —
   an address was taken, aliasing blocked an optimisation, or
   the function is genuinely register-starved.                */
```

Orthogonality and its limits

```
/* Most instructions accept any register in any operand position —
   which makes the ISA easy to generate code for. But there are
   real exceptions worth knowing:                                */

/* XZR/WZR — register 31 reads as zero and discards writes.
   This gives you several idioms for free:                       */
        mov     x0, xzr          // x0 = 0
        subs    xzr, x1, x2      // compare, discard the result
                                 //   (this IS how CMP is encoded)
        str     xzr, [x0]        // store zero, no register needed

/* But in SOME encodings, register 31 means the STACK POINTER
   instead of the zero register. Which one depends on the
   instruction:                                                  */
        add     x0, sp, #16      // here 31 = SP
        add     x0, x1, xzr      // here 31 = XZR

/* This is a genuine trap when hand-writing or reading assembly,
   and the reason disassemblers print SP and XZR explicitly.  */
```

No predication — and what replaced it

```
/* ARMv7 could predicate almost ANY instruction:
        ADDEQ r0, r1, r2      @ execute only if Z is set
   This saved branches, but cost 4 encoding bits on every single
   instruction and complicated out-of-order execution.

   A64 dropped it. Instead there are conditional SELECT
   instructions, which are branchless without needing predication
   everywhere:                                                  */

        cmp     w0, w1
        csel    w2, w3, w4, gt      // w2 = (w0 > w1) ? w3 : w4
        cset    w2, eq              // w2 = (Z) ? 1 : 0
        csinc   w2, w3, w4, ne      // w2 = ne ? w3 : w4 + 1
        csinv, csneg, cinc, cneg

/* This is how the compiler turns a small if/else into branchless
   code — and why the branchless idioms in CA chapter 12 and C
   chapter 08 compile so well on AArch64.                       */

int max(int a, int b) { return a > b ? a : b; }
/*      cmp     w0, w1
        csel    w0, w0, w1, gt
        ret                    — no branch, no mispredict      */
```

Flags are set explicitly

```
/* Most instructions do NOT touch the condition flags. You opt in
   with an S suffix. This is the opposite of x86, where nearly
   everything clobbers EFLAGS.                                  */

        add     w0, w1, w2       // flags unchanged
        adds    w0, w1, w2       // sets N, Z, C, V
        subs    xzr, x1, x2      // == cmp x1, x2
        ands    wzr, w1, w2      // == tst w1, w2

/* Why it matters: because flags survive across unrelated
   instructions, the compiler can compute a condition early,
   do other work, and branch on it later — which schedules
   better on a deep pipeline. It also means an overflow check
   needs an explicit ADDS, which is exactly what
   __builtin_add_overflow generates (C chapter 02).          */
```

Reading AArch64 disassembly

```
# the fastest way to learn: compile and look
gcc -O2 -S -o - test.c              # assembly source
gcc -O2 -c test.c && objdump -d test.o
objdump -d --source prog            # interleaved with C (needs -g)

# godbolt.org is the fastest iteration loop for this

# what to look for, in order:
#   1. LDR/STR inside loops  → memory traffic; is it necessary?
#   2. B / B.cond            → branches; predictable?
#   3. CSEL / CSET           → the compiler went branchless
#   4. ADRP/ADD pairs        → global variable access
#   5. STP/LDP at entry/exit → the prologue and epilogue
#   6. BL                    → calls that were not inlined
#   7. LD1/ST1/V registers   → it vectorised
```

> **THE HABIT WORTH BUILDING**
> When a performance question comes up, look at the assembly before theorising. AArch64 output is regular enough to read without being an assembly programmer, and it answers questions — did it vectorise, did `restrict` help, is that load hoisted out of the loop — that no amount of reasoning about the source will settle.

How it shows up

Symptom	Cause
A shared counter loses increments	Read-modify-write is three instructions; needs an atomic
Disassembly is full of ADRP/ADD pairs	Normal — that is how PC-relative addresses are built
A loop reloads a variable every iteration	`volatile`, or possible aliasing the compiler cannot rule out
Hand-written assembly corrupts the stack	Confused XZR with SP in an encoding where register 31 means SP
An overflow check compiles away	No `ADDS` emitted because signed overflow is UB (C ch. 15)
Branchless code on x86 becomes branches on ARM	Or the reverse — check for `CSEL` in the output
Code size larger than the x86 equivalent	Fixed 32-bit encodings; use Thumb-2 on M-profile
Stack traffic in a small hot function	Address taken, or register pressure — look for the spill

Check yourself

- ✓ State the load/store rule and show `counter++` as instructions.
- ✓ Explain why atomicity requires dedicated instructions, from that rule.
- ✓ Explain what an `ADRP` / `ADD` pair does and why it takes two instructions.
- ✓ Give three consequences of fixed 32-bit instruction width.
- ✓ Explain the dual meaning of register 31 and when each applies.
- ✓ Say what replaced ARMv7's predication and why the change was made.
- ✓ Explain why the `S` suffix exists and one place it matters.
- ✓ List five things to look for when reading AArch64 disassembly.

The AArch64 register file

Thirty-one general-purpose registers, a stack pointer per exception level, a zero register that is also sometimes the stack pointer, and a vector file that may be 128 or 2048 bits wide. This chapter is the map you will refer back to constantly.

General-purpose registers

```
X0 — X30      31 registers, 64 bits each
W0 — W30      the LOW 32 BITS of the same registers

┌──────────────────────────────────────────────────┐
│                    X0  (64 bits)                 │
│                ┌──────────────────────────────┐  │
│  upper 32      │        W0  (32 bits)         │  │
└────────────────┴──────────────────────────────┘──┘

CRITICAL RULE: writing Wn ZEROES the upper 32 bits of Xn.
               There is no way to write only the low half.

     mov     x0, #0xFFFFFFFFFFFFFFFF
     mov     w0, #1                    // x0 is now 0x1, NOT
                                       // 0xFFFFFFFF00000001

This is a deliberate design choice: it removes the partial-
register dependency stalls that plague x86, where writing EAX
must merge with the old RAX. It also means 32-bit arithmetic is
free — no sign or zero extension needed afterwards.
```

Roles assigned by the ABI

Register	Role	Preserved across a call?
`X0` – `X7`	Arguments 1–8, and return value in `X0` (`X0` / `X1` for 128-bit)	No — caller-saved
`X8`	Indirect result location (large struct return). On Linux, also the **syscall number**.	No
`X9` – `X15`	Temporaries	No — caller-saved
`X16`, `X17`	IP0, IP1 — intra-procedure-call scratch, used by linker veneers and PLT stubs	No; may be clobbered by the linker
`X18`	Platform register. Reserved on some OSes; TLS on Windows; kernel shadow-call-stack pointer on Android.	Platform-defined
`X19` – `X28`	General use	**Yes — callee-saved**
`X29`	FP — frame pointer	Yes
`X30`	LR — link register, written by `BL`	Yes, by convention (saved in the prologue)

> **`X18` IS A PORTABILITY TRAP**
>
> The ABI says "platform register" and each OS decides. Windows uses it for TLS; Android's kernel uses it for the shadow call stack; some RTOSes leave it general. Hand-written assembly or a compiler configured with the wrong `-ffixed-x18` setting produces corruption that only appears on one platform. If you write assembly that must be portable, do not touch `X18`.

The special registers

```
/* XZR / WZR — register encoding 31, in most contexts */
        mov     x0, xzr           // x0 = 0, no immediate needed
        str     xzr, [x1]         // store 64 bits of zero
        subs    xzr, x1, x2       // compare: this IS how CMP encodes
        ands    wzr, w1, w2       // test:    this IS how TST encodes

/* SP — register encoding 31, in OTHER contexts */
        add     x0, sp, #16       // 31 here means SP
        mov     x0, sp            // encoded as `add x0, sp, #0`
        sub     sp, sp, #64       // allocate stack

/* The SAME encoding means two different things depending on the
   instruction. This is why disassemblers print them explicitly,
   and why hand-written assembly gets it wrong.               */

/* PC — the program counter is NOT a general register in AArch64.
   You cannot write it directly (unlike ARMv7, where R15 was the
   PC and `mov pc, lr` was a return). It changes only via
   branches, and it can be READ only PC-relatively:          */
        adr     x0, .              // address of this instruction
        adrp    x0, symbol         // page address, ±4 GB range
```

Stack pointers per exception level

```
   SP_EL0    used by EL0, and optionally by higher ELs
   SP_EL1    the kernel's stack
   SP_EL2    the hypervisor's stack
   SP_EL3    the secure monitor's stack

   PSTATE.SPSel selects which one "SP" refers to at the current EL:
        SPSel = 0  →  always SP_EL0        (called "t", thread)
        SPSel = 1  →  SP_ELx for this EL   (called "h", handler)

   This is why exception return states are written EL1t and EL1h,
   and why the vector table has separate entries for exceptions
   taken with SP_EL0 versus SP_ELx (chapter 27).

   WHY IT EXISTS: on exception entry the hardware switches to the
   handler's own stack automatically. Without this, a user process
   with a corrupt SP would take the kernel down with it — the
   kernel would try to save state onto an invalid stack.

   Linux reads `current` from SP_EL0 while running at EL1, using
   the otherwise-unused register as a per-CPU pointer to the
   current task_struct. That is why `current` is one register
   read rather than a lookup (OS chapter 04).
```

SIMD and floating-point registers

```
   V0 — V31      32 registers, 128 bits each

   The SAME register, named by the width you are using:

   ┌──────────────────────────────────────────┐
   │               Q0   (128 bits)            │
   │                    ┌──────────────────────┤
   │                    │    D0  (64 bits)     │
   │                    │           ┌──────────┤
   │                    │           │ S0 (32)  │
   │                    │           │     ┌────┤
   │                    │           │     │H0  │ (16)
   │                    │           │     │  ┌─┤
   │                    │           │     │  │B0│ (8)
   └────────────────────┴───────────┴─────┴──┴─┘

   Bn  8-bit    Hn 16-bit    Sn 32-bit (float)
   Dn 64-bit (double)        Qn 128-bit (full vector)

   Or as VECTORS with a lane specifier:
        V0.16B   16 bytes        V0.8H   8 half-words
        V0.4S    4 words         V0.2D   2 double-words

   Writing Sn or Dn ZEROES the upper bits of Vn — the same rule
   as W registers.

   FPCR / FPSR   control and status: rounding mode, exception
                 flags, flush-to-zero. Note these are NOT in
                 PSTATE and are saved separately.
```

SVE: registers whose width is unknown at compile time

```
/* The Scalable Vector Extension changes the model fundamentally: */

   Z0 – Z31     vector registers, 128 to 2048 bits — the width is
                an IMPLEMENTATION CHOICE, not known when you compile
   P0 – P15     predicate registers, one bit per vector byte
   FFR          first-fault register, for speculative loads

/* Vector-length-agnostic code: the same binary runs optimally on
   any width, because the loop asks the hardware how wide it is:  */

        whilelo p0.s, x0, x1        // predicate = lanes still in range
1:      ld1w    z0.s, p0/z, [x2, x0, lsl #2]
        ld1w    z1.s, p0/z, [x3, x0, lsl #2]
        fadd    z0.s, z0.s, z1.s
        st1w    z0.s, p0,   [x4, x0, lsl #2]
        incw    x0                  // advance by the ACTUAL width
        whilelo p0.s, x0, x1         // recompute the predicate
        b.first 1b

/* No scalar tail loop is needed — the predicate handles the
   remainder. Compare with NEON, which needs an epilogue for
   elements that do not fill a full 128-bit vector.          */

/* Z0–Z31 ALIAS V0–V31: the low 128 bits of Zn IS Vn. */
```

System registers are not general registers

```
/* Configuration lives in SYSTEM registers, which are a separate
   namespace and are accessed only with MRS and MSR (chapter 06). */

        mrs     x0, sctlr_el1        // System register → general
        msr     sctlr_el1, x0        // General → System register

/* They are named REG_ELx where x is the LOWEST exception level
   that may access them. SCTLR_EL1 is accessible at EL1, EL2 and
   EL3 — but not EL0.

   You cannot use a system register as an operand to ADD. You must
   move it to a general register first. This is a common early
   mistake when hand-writing boot code.                       */
```

No banked general registers

> **THE MOST CONSEQUENTIAL DIFFERENCE FROM ARMV7**
>
> ARMv7 banked several registers per processor mode: entering IRQ mode gave you a private `R13` and `R14` automatically. **AArch64 does not bank `X0`–`X30` at all.**
>
> When an exception is taken, the handler begins executing with the interrupted code's register values still live. Nothing is saved for you except `ELR`, `SPSR` and the syndrome registers. Saving and restoring the general registers is *entirely* software's job — and that saved set is `struct pt_regs` in Linux, the trap frame in Windows.
>
> This is why every AArch64 exception vector begins with thirty-odd store instructions (chapter 28), and why getting that stub wrong corrupts the interrupted program in ways that are extremely hard to diagnose.

Register conventions in practice

```
/* A standard prologue and epilogue — read this until it is
   instantly recognisable in any disassembly.               */

my_func:
        stp     x29, x30, [sp, #-32]!   // push FP,LR; pre-decrement SP
        mov     x29, sp                 // establish the frame pointer
        stp     x19, x20, [sp, #16]     // save the callee-saved regs
                                        //   this function will use
        ...
        ldp     x19, x20, [sp, #16]     // restore
        ldp     x29, x30, [sp], #32     // pop FP,LR; post-increment SP
        ret                             // == BR X30, with a hint

/* STP/LDP move TWO registers in one instruction — which is why
   prologues save in pairs, and why the stack is always adjusted
   in multiples of 16 (chapter 09 covers the alignment rule). */

/* A LEAF function — one that calls nothing — needs no prologue
   at all, because LR is not clobbered:                        */
square: mul     w0, w0, w0
        ret
```

Inspecting registers

```
# gdb
info registers                # X0–X30, SP, PC, PSTATE
info registers x0 x1 sp
info all-registers            # including V0–V31 and system regs
p/x $x0                       # one register, in hex
p $sp
info frame                    # the current frame's saved registers

# vector registers
p $v0.d                       # as two doubles
p $v0.s                       # as four floats
p/x $v0.b                     # as sixteen bytes

# system registers, where the target exposes them
p/x $SCTLR_EL1
p/x $CPSR                     # PSTATE, on some gdb versions

# in a core dump or an oops, Linux prints them all:
dmesg | grep -A20 'pc :'
#   pc : ffff800008123456
#   lr : ffff8000081234ab
#   sp : ffff800009abc000
#   x29: ...  x28: ...  x27: ...
```

How it shows up

Symptom	Cause
Upper 32 bits of a value unexpectedly zero	A `W`-register write zeroed them — by design
Hand-written assembly corrupts the caller	Clobbered a callee-saved register (`X19`–`X28`) without saving it
Assembly works on Linux, breaks on Windows	Used `X18`, which is the platform register
Stack corruption in an exception handler	Confused XZR with SP in an encoding where 31 means SP
Interrupted program corrupted after an ISR	Vector stub failed to save all general registers — nothing is banked
Alignment fault on a function call	SP not 16-byte aligned
FP values wrong after a context switch	FPCR/FPSR not saved; they are not part of PSTATE
SVE code faults on a narrower machine	Assumed a fixed vector length instead of being length-agnostic
Cannot use a system register in an ADD	Must `MRS` it into a general register first

Check yourself

- ✓ State what happens to the upper 32 bits when you write a `W` register, and why that design was chosen.
- ✓ Name the argument registers, the callee-saved set, and what `X30` holds.
- ✓ Explain the dual meaning of register encoding 31.
- ✓ Explain why there is a stack pointer per exception level.
- ✓ Give the `t` / `h` distinction in EL1t and EL1h.
- ✓ Explain how `V0`, `D0` and `S0` relate.
- ✓ Say what SVE changes about writing vector code.

✓ Explain why AArch64 exception handlers begin with many store instructions.

PSTATE and the condition flags

ARMv7 had one CPSR register holding all processor state. AArch64 broke it into independently accessible fields called PSTATE. Understanding those fields — especially the interrupt masks — is what lets you read an exception handler, a critical section, or a crash dump.

What PSTATE is

> **NOT A REGISTER**
> PSTATE is an *abstraction*: a collection of processor state bits, each accessed through its own system-register name. There is no instruction that reads all of PSTATE at once during normal execution. It is only gathered into a single value when an exception is taken, at which point the hardware writes it into `SPSR_ELx` — and `ERET` restores it from there.

Field	Meaning	Accessed via
`N Z C V`	Condition flags	`NZCV`
`D A I F`	Exception masks: Debug, SError, IRQ, FIQ	`DAIF`, `DAIFSet`, `DAIFClr`
`EL`	Current exception level, 0–3	`CurrentEL`
`SPSel`	Which stack pointer is in use	`SPSel`
`nRW`	Execution state: AArch64 or AArch32	Only in SPSR
`PAN`	Privileged Access Never	`PAN` (v8.1)
`UAO`	User Access Override	`UAO` (v8.2)
`SSBS`	Speculative Store Bypass Safe	`SSBS` (v8.5)
`BTYPE`	Branch Target type, for BTI	Set by hardware (v8.5)
`TCO`	Tag Check Override, for MTE	`TCO` (v8.5)
`IL`, `SS`	Illegal execution state, software step	Debug

The condition flags

```
  N  Negative   the result's sign bit was 1
  Z  Zero       the result was zero
  C  Carry      unsigned overflow (add) / NO borrow (subtract)
  V  oVerflow   SIGNED overflow

Set only by instructions with the S suffix, or by CMP/CMN/TST/TEQ
(which are themselves encoded as SUBS/ADDS/ANDS with XZR as the
destination — see chapter 03).

THE POINT of having both C and V: one comparison sets both, so
the same CMP serves signed and unsigned tests. The condition code
you branch on decides which interpretation applies.
```

Code	Meaning	Flags	Use for
`EQ` / `NE`	Equal / not equal	Z=1 / Z=0	Either signedness

Code	Meaning	Flags	Use for
`HS` (`CS`) / `LO` (`CC`)	≥ / <	C=1 / C=0	**Unsigned**
`HI` / `LS`	> / ≤	C=1&Z=0 / C=0|Z=1	**Unsigned**
`GE` / `LT`	≥ / <	N=V / N≠V	**Signed**
`GT` / `LE`	> / ≤	Z=0&N=V / Z=1|N≠V	**Signed**
`MI` / `PL`	Negative / positive-or-zero	N=1 / N=0	Sign test
`VS` / `VC`	Overflow set / clear	V=1 / V=0	Signed overflow detection
`AL` / `NV`	Always / never	—	Unconditional

```
/* Why the signed/unsigned distinction is not academic: */

        mov     w0, #-1             // 0xFFFFFFFF
        mov     w1, #1
        cmp     w0, w1

        b.lt    signed_less         // TAKEN:   -1 < 1        ✓
        b.lo    unsigned_less       // NOT taken: 0xFFFFFFFF > 1 ✓

/* Both are correct — for their respective interpretations. Using
   the wrong condition is the assembly equivalent of the signed/
   unsigned comparison bug in C chapter 02, and it is exactly as
   easy to get wrong.                                          */

/* Detecting signed overflow explicitly: */
        adds    w0, w1, w2
        b.vs    overflow_handler    // V set = signed overflow
/* This is what __builtin_add_overflow compiles to (C ch. 02). */
```

```
/* Flags survive across unrelated instructions, because most
   instructions do NOT set them. That enables useful scheduling: */

        cmp     x0, x1              // compute the condition EARLY
        ldr     x2, [x3]            // ...do a long-latency load...
        add     x4, x5, x6          // ...and other work...
        b.eq    equal               // ...then branch. The load has
                                    //    had time to complete.
/* On x86 nearly every instruction clobbers EFLAGS, so this
   separation is not possible. It is a small but real advantage
   of the explicit-S design.                                   */
```

The interrupt masks

```
   D  Debug     mask debug exceptions
   A  SError    mask System Errors (imprecise external aborts)
   I  IRQ       mask normal interrupts
   F  FIQ       mask fast interrupts

   1 = MASKED (disabled).  0 = enabled.

   In DAIF, the bits sit at:   D=9  A=8  I=7  F=6

        msr     daifset, #2      // set I  → DISABLE IRQ
        msr     daifclr, #2      // clear I → ENABLE IRQ
        msr     daifset, #3      // set I and F
        msr     daifset, #0xF    // mask everything

   The immediate is a 4-bit mask over {D,A,I,F} = bits {3,2,1,0}
   of the immediate. So #2 = 0b0010 = the I bit.
```

```
/* Save-and-restore is the ONLY correct pattern for a critical
   section. Unconditionally re-enabling is a serious bug.     */

static inline unsigned long irq_save(void)
{
        unsigned long flags;
        asm volatile("mrs %0, daif\n\t"
                     "msr daifset, #2"
                     : "=r"(flags) :: "memory");
        return flags;
}

static inline void irq_restore(unsigned long flags)
{
        asm volatile("msr daif, %0" :: "r"(flags) : "memory");
}

/* Why unconditional enable is wrong:

        outer_lock();          // disables IRQs
            inner_lock();      // disables IRQs
            inner_unlock();    // ENABLES IRQs  ← WRONG
        ...                    // now running unprotected inside
                               //    the OUTER critical section
        outer_unlock();

   With save/restore, the inner unlock restores "already disabled"
   and nesting works. This is exactly why Linux has
   spin_lock_irqsave() and not just spin_lock_irq()
   (OS chapter 09).                                          */
```

HARDWARE MASKS D, A, I AND F ON EVERY EXCEPTION ENTRY

This is not optional and not configurable. The reason is that `ELR_ELx` and `SPSR_ELx` are single registers, not a stack: a second exception arriving before your handler saved them would overwrite the return address, and you could never get back. Your handler saves them to the kernel stack first, and only *then* may unmask to allow nesting. Getting this order wrong destroys the interrupted context (chapter 28).

IRQ versus FIQ

	IRQ	FIQ
ARMv7	Standard interrupt	"Fast": extra banked registers, last vector entry, higher priority
AArch64	Standard interrupt	**No banked registers, no speed advantage** — just a second, separately maskable signal
Conventional use	Non-secure world	**Secure world**: Group 0 GIC interrupts
Routing	`SCR_EL3.IRQ`, `HCR_EL2.IMO`	`SCR_EL3.FIQ`, `HCR_EL2.FMO`

The practical consequence: on a TrustZone system, a Non-secure OS cannot mask secure interrupts by clearing `I`, because they arrive as FIQ and are configured to be taken to EL3. That separation is what lets the secure world preempt the normal world reliably (security track).

PAN, UAO and the kernel/user boundary

```
/* PAN (Privileged Access Never, ARMv8.1) — when set, EL1 and EL2
   accesses to memory marked user-accessible FAULT.

   Why: it turns "kernel accidentally dereferenced a user pointer"
   from a silent, exploitable bug into an immediate fault. It is
   the ARM equivalent of x86's SMAP.                          */

        msr     pan, #1              // enable PAN

/* Legitimate access to user memory then requires explicitly
   lifting it, which is exactly what copy_to_user does:      */
        msr     pan, #0
        // ... the actual copy, with an exception fixup installed
        msr     pan, #1

/* UAO (User Access Override, ARMv8.2) makes the unprivileged
   load/store instructions (LDTR/STTR) behave as privileged,
   which lets the kernel use ONE code path for both cases.  */

/* This is why you must never dereference a __user pointer
   directly (OS chapter 03) — on a PAN-enabled system it does not
   silently work, it faults.                                  */
```

Reading PSTATE in a crash dump

```
/* A Linux arm64 oops prints the saved PSTATE: */

pstate: 60400009 (nZCv daif +PAN -UAO -TCO -DIT -SSBS BTYPE=--)
        ^^^^^^^^  ^^^^ ^^^^  ^^^^
        raw value |    |     features
                  |    └── masks: lowercase = CLEAR (enabled)
                  |                UPPERCASE = SET (masked)
                  └─────── flags:  lowercase = clear, upper = set

/* Reading it:
     nZCv   → N=0, Z=1, C=1, V=0   (the last comparison was equal)
     daif   → all lowercase = interrupts were ENABLED
     DAIF   → all uppercase = all masked (typical inside a handler)
     +PAN   → PAN was active, so this was NOT a user-memory access

   The low bits of the raw value encode the EL and SPSel:
     0x...09 = 0b1001 → EL2h    0x...05 = 0b0101 → EL1h
     0x...04 = 0b0100 → EL1t    0x...00 = 0b0000 → EL0t

   So `pstate: ...09` means the fault happened at EL2 using
   SP_EL2 — which immediately tells you it was hypervisor code,
   not the guest.                                          */
```

```
# in gdb
p/x $cpsr                    # PSTATE, on targets that expose it
p $pstate

# in Linux, at runtime
cat /proc/cpuinfo | grep Features       # pan, uao, ssbs, bti...

# check what the kernel enabled
dmesg | grep -iE 'PAN|SSBS|BTI|MTE'
```

SPSR: PSTATE, frozen

```
On exception entry the hardware packs PSTATE into SPSR_ELx:

   31 30 29 28        21 20    12 11 10  9  8  7  6  5  4  3   0
  ┌──┬──┬──┬──┬─────────┬──┬─────┬─────┬──┬──┬──┬──┬──┬──┬───────┐
  │ N│ Z│ C│ V│  ...    │IL│ ... │ ... │ D│ A│ I│ F│ 0│nRW│ M[3:0]│
  └──┴──┴──┴──┴─────────┴──┴─────┴─────┴──┴──┴──┴──┴──┴──┴───────┘

   M[3:0] encodes the EL and stack selection to return to:
        0b0000  EL0t        0b0100  EL1t        0b1000  EL2t
                            0b0101  EL1h        0b1001  EL2h
        0b1100  EL3t        0b1101  EL3h

   ERET restores PSTATE from SPSR_ELx and PC from ELR_ELx, both
   ATOMICALLY. That atomicity is the whole point: there is no
   window in which the privilege has changed but the PC has not.

   Firmware uses this to DROP to a lower EL during boot: write the
   target state into SPSR_EL3, the entry point into ELR_EL3, then
   ERET (chapter 28).
```

How it shows up

Symptom	Cause
Comparison behaves oddly at the sign boundary	Used a signed condition for unsigned values, or the reverse
Interrupts stay disabled after a critical section	Unconditional enable instead of save/restore
Nested critical sections lose protection	Same — the inner unlock re-enabled prematurely
Corrupted return address in a nested exception	Unmasked interrupts before saving ELR/SPSR
Kernel faults dereferencing a user pointer	PAN is doing its job — use `copy_from_user`
Secure interrupt not maskable by the normal OS	Delivered as FIQ, routed to EL3 — by design
Overflow check never fires	No `S` suffix, so V was never set
Crash dump shows `pstate: ...09`	The fault was at EL2h — hypervisor, not guest
`ERET` returns to the wrong level	Wrong `M[3:0]` encoding in SPSR

Check yourself

- ✓ Explain why PSTATE is not a single register and when it becomes one.
- ✓ Give the four condition flags and say why both C and V are needed.
- ✓ Choose the right condition code for a signed and an unsigned "less than".
- ✓ Write a nesting-safe critical section and explain why unconditional enable is wrong.
- ✓ Explain why hardware masks DAIF on exception entry.
- ✓ State how FIQ differs from IRQ on AArch64 and what it is used for.
- ✓ Explain what PAN prevents and how `copy_to_user` works with it.
- ✓ Decode `pstate: 60400009 (nZCv daif +PAN)` into plain English.

System registers, MRS and MSR

Every piece of processor configuration — the MMU, caches, exception vectors, timers, security state — lives in a system register. There are hundreds. This chapter covers the naming scheme, the access rules, and the one instruction sequence you must get right or your configuration will not take effect.

The naming scheme

> `NAME_ELX`
>
> The suffix names the **lowest exception level that may access the register** — not the level it applies to. `SCTLR_EL1` is accessible from EL1, EL2 and EL3; it is *not* accessible from EL0. A register with no suffix, or `_EL0`, is accessible from user space.

```
   TTBR0_EL1        translation table base, accessible from EL1+
   TTBR0_EL2        the EL2 equivalent — a SEPARATE register
   VTTBR_EL2        stage-2 table base; only meaningful at EL2
   CNTPCT_EL0       physical counter — readable from USER SPACE
   SCTLR_EL3        secure monitor's system control

   Registers ending _EL1 often have _EL2 and _EL3 counterparts.
   They are DIFFERENT REGISTERS with independent contents, not
   different views of one register. Writing SCTLR_EL1 does not
   affect SCTLR_EL2.

   Special forms:
     _EL12, _EL02    accessed from EL2 with VHE enabled, these
                     alias the EL1/EL0 registers — how a host
                     kernel running at EL2 manipulates guest state
     ICC_*_EL1       GIC CPU interface (chapter 23)
     ID_*_EL1        read-only feature identification (chapter 01)
```

MRS and MSR

```
        mrs     Xt, <systemreg>      // System register → general
        msr     <systemreg>, Xt      // General → System register

/* Mnemonic: MRS = "Move Register from System"
             MSR = "Move System Register"                    */

/* A read-modify-write is three instructions, because system
   registers cannot be ALU operands (chapter 04):            */
        mrs     x0, sctlr_el1
        orr     x0, x0, #(1 << 0)      // set the M bit
        msr     sctlr_el1, x0
        isb                             // ← see below

/* There is also an immediate form for PSTATE fields only: */
        msr     daifset, #2
        msr     spsel, #1
        msr     pan, #1
```

The rule that catches everyone

> **A SYSTEM REGISTER WRITE NEEDS AN `ISB` TO TAKE EFFECT**
>
> Instructions already in the pipeline were fetched, decoded and possibly executed under the *old* configuration. Writing `SCTLR_EL1` to enable the MMU does not retroactively change how the next few instructions were translated.
>
> **ISB** — Instruction Synchronization Barrier — flushes the pipeline so that everything after it is re-fetched and re-decoded under the new context. Without it, behaviour depends on pipeline depth and varies between cores, which is exactly the kind of bug that appears on one board and not another.

```
/* The canonical sequence, and why each part is there: */

        /* 1. make sure prior memory writes (e.g. the page tables
              you just built) are visible to the table walker  */
        dsb     ish

        /* 2. change the configuration */
        msr     ttbr0_el1, x0
        msr     tcr_el1, x1
        msr     sctlr_el1, x2

        /* 3. flush the pipeline so it takes effect NOW */
        isb

/* Which registers need it? Anything that changes how instructions
   are FETCHED, TRANSLATED or PERMITTED:
     SCTLR, TTBR, TCR, MAIR, VBAR, SCR, HCR, CPACR, CPTR, TPIDR

   Registers that only affect a later explicit read - CNTFRQ,
   most ID registers - do not.

   When in doubt, add the ISB. It costs a pipeline flush; the bug
   costs a week.                                              */
```

The registers you will actually use

Register	Controls	Chapter
`SCTLR_EL1`	System Control: `M` MMU enable, `C` data cache, `I` instruction cache, `A` alignment check, `WXN` write-implies-execute-never, `SA` stack alignment	12
`TTBR0_EL1` / `TTBR1_EL1`	Translation table base — low and high address ranges. The ASID lives in `TTBR0[63:48]`.	13
`TCR_EL1`	Translation Control: VA size (`T0SZ` / `T1SZ`), granule size, walk cacheability and shareability, top-byte-ignore	13
`MAIR_EL1`	Memory Attribute Indirection — eight 8-bit slots that descriptors index by number	11
`VBAR_EL1`	Vector Base Address — where the exception vector table lives	27
`ESR_EL1`	Exception Syndrome — *why* the last exception happened	29
`FAR_EL1`	Fault Address — *which address* faulted	29
`ELR_EL1` / `SPSR_EL1`	Saved return address and saved PSTATE	28

Register	Controls	Chapter
`MPIDR_EL1`	Which core am I? Affinity levels, used by SMP startup	33
`CurrentEL`	The exception level right now, in bits [3:2]	26
`CNTFRQ_EL0`, `CNTPCT_EL0`	Generic timer frequency and count	32
`CTR_EL0`	Cache line sizes — readable from EL0 so userspace can do cache maintenance correctly	18
`TPIDR_EL0`	Thread pointer — this is where TLS lives	OS 05
`SCR_EL3`	Secure Configuration: the `NS` bit, interrupt routing, `SMD`	security
`HCR_EL2`	Hypervisor Configuration: stage-2 enable, trap control, virtual interrupts	31

SCTLR_EL1, field by field

```
The single most important register in early boot. The bits you
will actually touch:

   bit  0  M      MMU enable
   bit  1  A      Alignment check — fault on ANY unaligned access
   bit  2  C      Data cache enable
   bit  3  SA     Stack alignment check (SP must be 16-byte aligned)
   bit  4  SA0    Stack alignment check for EL0
   bit 12  I      Instruction cache enable
   bit 19  WXN    Writable implies eXecute-Never — a W^X hardening
   bit 25  EE     Exception endianness
   bit 39  TCF    Tag check fault behaviour (MTE)

  Reset value has M, C and I CLEAR: the MMU and caches are OFF
  when your boot code starts. Turning them on, in the right order
  and with the right barriers, is the job of early boot.

  A common bring-up bug: enabling C (data cache) before the MMU.
  With the MMU off, all memory is treated as Device-nGnRnE and
  the data cache does nothing — so it appears harmless. Then you
  enable the MMU and the previously-uncached writes are suddenly
  cacheable, and stale data appears. Enable M and C together,
  after the tables are built and a DSB has completed.
```

Access can be trapped

```
/* A higher exception level can configure a system register access
   at a LOWER level to trap instead of executing. This is the
   entire basis of virtualisation (chapter 31).              */

/* Example: HCR_EL2.TVM traps guest writes to virtual-memory
   control registers, so the hypervisor sees every TTBR change. */
        mrs     x0, hcr_el2
        orr     x0, x0, #(1 << 26)      // TVM
        msr     hcr_el2, x0
        isb

/* Now a guest at EL1 executing `msr ttbr0_el1, x0` takes a
   synchronous exception to EL2 with ESR_EL2.EC = 0x18
   (trapped MSR/MRS). The hypervisor decodes the syndrome to
   learn which register and which general register were involved,
   emulates the write, advances ELR_EL2 by 4, and ERETs.     */

/* Other common trap controls:
     CPACR_EL1.FPEN     trap FP/SIMD — used for lazy FPU switching
     CPTR_EL2/EL3       the EL2/EL3 equivalents
     HCR_EL2.TGE        route ALL EL0 exceptions to EL2
     MDCR_EL2/EL3       trap debug and performance monitor access
     SCR_EL3.SMD        disable SMC entirely                  */
```

Accessing them from C

```
/* Inline assembly, with the constraints from C chapter 18: */

#define read_sysreg(r) ({                       \
        uint64_t __v;                           \
        asm volatile("mrs %0, " #r : "=r"(__v)); \
        __v;                                    \
})

#define write_sysreg(v, r) do {                 \
        uint64_t __v = (uint64_t)(v);           \
        asm volatile("msr " #r ", %0" :: "r"(__v)); \
} while (0)

uint64_t sctlr = read_sysreg(sctlr_el1);
write_sysreg(sctlr | 1, sctlr_el1);
asm volatile("isb");

/* Linux provides exactly this in arch/arm64/include/asm/sysreg.h,
   plus read_sysreg_s() for registers the assembler does not know
   by name (encoded as S<op0>_<op1>_<CRn>_<CRm>_<op2>).       */

/* Note: `volatile` is REQUIRED here. Without it the compiler may
   delete an MSR whose result is unused, or hoist an MRS out of a
   loop. A register write with a side effect needs it always.  */
```

The encoding, and unnamed registers

```
A system register is identified by five fields:

        S<op0>_<op1>_C<CRn>_C<CRm>_<op2>

Assemblers know the names for architected registers. For newer or
IMPLEMENTATION DEFINED ones, you encode them directly:

        mrs     x0, S3_1_C15_C2_0        // an IMP DEF register

Linux uses this for registers newer than the minimum supported
binutils version:

        #define SYS_ID_AA64ISAR2_EL1  sys_reg(3, 0, 0, 6, 2)
        read_sysreg_s(SYS_ID_AA64ISAR2_EL1);

If you see S3_x_Cy_Cz_w in a disassembly, look it up in the ARM ARM
register index - it is an architected register your disassembler
did not recognise, and it is often the interesting one.
```

System registers and context switching

```
/* Which system registers must be saved on a context switch?
   Only those that are PER-CONTEXT:                          */

   TTBR0_EL1        per address space - this IS the switch
   TPIDR_EL0        thread pointer (TLS)
   TPIDRRO_EL0      read-only thread pointer
   FPCR / FPSR      FP control and status
   (PAC keys, if used)

/* NOT saved, because they are global to the kernel:         */
   SCTLR_EL1, MAIR_EL1, TCR_EL1, VBAR_EL1, TTBR1_EL1

/* This is why a thread switch within one process is cheaper than
   a process switch: TTBR0 does not change, so the TLB entries
   remain valid (OS chapter 07).

   A VM switch is far more expensive: the hypervisor must save and
   restore essentially the entire EL1 system register file, which
   is dozens of registers - one reason VM exits are costly.  */
```

Reading them at runtime

```
# Linux exposes some via sysfs
cat /sys/devices/system/cpu/cpu0/regs/identification/midr_el1
cat /sys/devices/system/cpu/cpu0/regs/identification/revidr_el1

# gdb, on a bare-metal or kernel target
p/x $SCTLR_EL1
p/x $TTBR0_EL1
p/x $ESR_EL1
info all-registers | grep -i el1

# in a kernel oops, the relevant ones are printed for you
dmesg | grep -E 'ESR|FAR|pstate'

# a small kernel module or bare-metal routine to dump the
# configuration is worth having during bring-up:
pr_info("SCTLR=%016llx TCR=%016llx MAIR=%016llx TTBR0=%016llx\n",
        read_sysreg(sctlr_el1), read_sysreg(tcr_el1),
        read_sysreg(mair_el1),  read_sysreg(ttbr0_el1));
```

How it shows up

Symptom	Cause
Configuration change has no effect	Missing `ISB` after the `MSR`
Works on one core, fails on another	Missing barrier; behaviour depends on pipeline depth
Undefined instruction on `MRS`	Accessed a register from too low an exception level
Guest hangs writing a system register	A trap is configured and the hypervisor is not emulating it
Stale data after enabling the MMU	Cache enabled before translation was configured
Alignment faults everywhere after boot	`SCTLR.A` set, enabling strict alignment checking
Compiler removed your `MSR`	Missing `volatile` on the inline assembly
Disassembly shows `S3_0_C0_C6_2`	An architected register the assembler does not know — look it up
TLS broken after a hand-written context switch	`TPIDR_EL0` not saved and restored

Check yourself

- ✓ Explain what the `_ELx` suffix means, and whether `SCTLR_EL1` is one register or several.
- ✓ Write a read-modify-write of a system register and explain why it takes three instructions.
- ✓ Explain why `ISB` is needed and name four registers that require it.
- ✓ Give five `SCTLR_EL1` bits and what each controls.
- ✓ Explain how a hypervisor intercepts a guest's system register write.
- ✓ Say which system registers are per-context and which are global, and why that matters for switch cost.
- ✓ Explain why inline assembly for `MSR` needs `volatile`.
- ✓ Explain what `S3_1_C15_C2_0` is.

The A64 instruction set

You do not need to memorise a thousand instructions. You need to recognise the dozen families, know the idioms the compiler emits constantly, and be able to look up the rest. This chapter is the working subset.

The families

Family	Representative	Notes
Data processing (register)	`ADD` `SUB` `AND` `ORR` `EOR` `MUL` `UDIV` `SDIV`	Suffix `S` sets flags
Data processing (immediate)	`ADD Xd, Xn, #imm12`	12-bit immediate, optionally shifted left 12
Move	`MOV` `MOVZ` `MOVK` `MOVN`	Build 64-bit constants 16 bits at a time
Shift / rotate	`LSL` `LSR` `ASR` `ROR`	Aliases of `UBFM` / `SBFM`
Bitfield	`UBFX` `SBFX` `BFI` `UBFIZ`	Extract and insert in one instruction
Load / store	`LDR` `STR` `LDP` `STP` `LDRB` `LDRSW`	Chapter 08
Branch	`B` `BL` `BR` `BLR` `RET` `CBZ` `TBZ` `B.cond`	See below
Conditional select	`CSEL` `CSET` `CSINC` `CINC` `CSNEG`	Branchless selection
Address formation	`ADR` `ADRP`	PC-relative, ±1 MB and ±4 GB
Atomics	`LDXR` / `STXR`, `LDADD`, `CAS`, `SWP`	Chapter 22
Barriers	`DMB` `DSB` `ISB`	Chapter 21
System	`MRS` `MSR` `SVC` `HVC` `SMC` `ERET` `WFI` `TLBI` `DC` `IC`	The systems programmer's set
SIMD / FP	`FADD` `FMUL` `LD1` `ST1` `DUP` `ADDV`	Operate on `V` registers
Crypto / CRC	`AESE` `SHA256H` `PMULL` `CRC32`	Optional — check the ID registers

Building constants

```
/* A 32-bit instruction cannot hold a 64-bit immediate, so large
   constants are assembled from 16-bit pieces.               */

        movz    x0, #0x1234, lsl #48    // zero the rest, set bits 63:48
        movk    x0, #0x5678, lsl #32    // keep the rest, set bits 47:32
        movk    x0, #0x9abc, lsl #16
        movk    x0, #0xdef0             // x0 = 0x123456789abcdef0

        movn    x0, #0                  // MOVe Negated: x0 = ~0 = -1

/* The assembler's `MOV Xd, #imm` picks whichever of MOVZ, MOVN or
   an ORR-with-bitmask-immediate encodes the value, or emits
   several instructions if none does.                        */

/* BITMASK IMMEDIATES are a genuinely clever encoding: logical
   instructions can take any value that is a repeating pattern of
   set bits. So these are ONE instruction each:              */
        and     x0, x0, #0xFFFFFFFF     // repeating pattern ✓
        orr     x0, x0, #0x0F0F0F0F0F0F0F0F
        and     x0, x0, #0xFFF          // ✓
/* ...but this needs a MOV first, because 0x12345 is not a
   repeating pattern:                                         */
        mov     x1, #0x12345
        and     x0, x0, x1

/* This is why some masks compile to one instruction and visually
   similar ones compile to two. It is not arbitrary.         */
```

Branches

Instruction	Does	Range
`B label`	Unconditional branch	±128 MB
`BL label`	Branch with link — sets `X30`	±128 MB
`BR Xn`	Indirect branch	Anywhere
`BLR Xn`	Indirect call	Anywhere
`RET {Xn}`	Return — `BR X30` plus a predictor hint	Anywhere
`B.cond label`	Conditional on NZCV	±1 MB
`CBZ/CBNZ Xn, label`	Compare with zero and branch — **no flags needed**	±1 MB
`TBZ/TBNZ Xn, #b, label`	Test one bit and branch — no flags needed	±32 KB

```
/* CBZ and TBZ are very common because they need no CMP first: */
        cbz     x0, is_null             // if (p == NULL) goto...
        tbnz    w1, #5, bit_set         // if (flags & (1 << 5)) goto...

/* RET is BR X30 with a hint. The hint matters: it tells the
   return-address stack predictor to POP, which is why using
   BR X30 instead of RET degrades prediction (CA chapter 12). */

/* The ±128 MB limit on B/BL is why LINKER VENEERS exist: if a
   call target is further away, the linker inserts a trampoline
   that loads the full address into X16 and does BR X16. That is
   what X16/X17 are reserved for (chapter 04).               */
```

Conditional select: the branchless workhorse

```
        csel    Xd, Xn, Xm, cond    // Xd = cond ? Xn : Xm
        csinc   Xd, Xn, Xm, cond    // Xd = cond ? Xn : Xm + 1
        csinv   Xd, Xn, Xm, cond    // Xd = cond ? Xn : ~Xm
        csneg   Xd, Xn, Xm, cond    // Xd = cond ? Xn : -Xm

/* And their aliases, which are what you usually see: */
        cset    Xd, cond            // Xd = cond ? 1 : 0
        csetm   Xd, cond            // Xd = cond ? -1 : 0   ← a MASK
        cinc    Xd, Xn, cond        // Xd = cond ? Xn+1 : Xn
        cneg, cinv

/* Source and result: */
int max(int a, int b) { return a > b ? a : b; }
/*      cmp     w0, w1
        csel    w0, w0, w1, gt
        ret                                                 */

int sign(int x) { return (x > 0) - (x < 0); }
/*      cmp     w0, #0
        cset    w1, gt
        csinv   w0, w1, wzr, ge      // clever: -1 when negative
        ret                                                 */

/* CSETM is exactly the mask idiom from C chapter 08:
        mask = -(unsigned)cond;   → csetm
   which is why branchless C compiles so cleanly here.      */
```

Bitfield instructions

```
/* ARM does field extract and insert in ONE instruction, which is
   why the mask-and-shift idioms of C chapter 08 are cheap.  */

        ubfx    w0, w1, #4, #3       // w0 = (w1 >> 4) & 0x7  (unsigned)
        sbfx    w0, w1, #4, #3       // ...sign-extended
        bfi     w0, w1, #4, #3       // insert w1's low 3 bits at 4
        bfxil   w0, w1, #4, #3       // extract and insert low
        ubfiz   w0, w1, #4, #3       // extract, insert, zero the rest

/* Extensions are also bitfield ops: */
        uxtb    w0, w1               // zero-extend byte
        sxtw    x0, w1               // sign-extend word to 64
        lsl     x0, x1, #3           // alias of UBFM

/* Bit counting and reversal, all single instructions: */
        clz     x0, x1               // count leading zeros
        rbit    x0, x1               // reverse bit order
        rev     x0, x1               // reverse BYTE order (endian swap)
        rev16, rev32
        cnt     v0.8b, v1.8b         // popcount, per byte (SIMD)

/* __builtin_clz → CLZ.  __builtin_bswap32 → REV.
   __builtin_ctz → RBIT then CLZ. This is why those builtins are
   free on ARM and you should always prefer them (C ch. 08). */
```

Arithmetic worth knowing

```
/* Multiply-accumulate in one instruction — the basis of every
   DSP and matrix kernel:                                       */
        madd    x0, x1, x2, x3       // x0 = x3 + x1 * x2
        msub    x0, x1, x2, x3       // x0 = x3 - x1 * x2
        mneg    x0, x1, x2           // x0 = -(x1 * x2)

/* Widening multiply: 32x32 → 64, no overflow */
        umull   x0, w1, w2
        smull   x0, w1, w2
        umulh   x0, x1, x2           // HIGH 64 bits of a 64x64 product
        smulh   x0, x1, x2

/* Division is NOT cheap — roughly 10-20 cycles, and there is no
   remainder instruction. `a % b` becomes:                      */
        sdiv    x2, x0, x1
        msub    x2, x2, x1, x0       // x2 = x0 - (x0/x1)*x1

/* Which is why dividing by a constant compiles to a multiply by
   a magic reciprocal plus shifts, and why power-of-two divisors
   become a single shift.                                       */

/* Shifted register operands are free — folded into the ALU: */
        add     x0, x1, x2, lsl #3   // x0 = x1 + (x2 << 3)
        sub     x0, x1, x2, asr #2
/* This is how array indexing compiles: base + index*8 is ONE
   instruction, not two.                                        */
```

System instructions

Instruction	Effect
`SVC #imm`	Supervisor call → exception to EL1. System calls.
`HVC #imm`	Hypervisor call → EL2
`SMC #imm`	Secure monitor call → EL3
`ERET`	Exception return: PC from `ELR`, PSTATE from `SPSR`, atomically
`WFI`	Wait For Interrupt — enter a low-power state until one arrives
`WFE` / `SEV`	Wait For Event / Send Event — used by spinlocks to avoid burning power
`YIELD`	A hint that this thread is spinning; useful on SMT
`NOP`	No operation — also a patch point for runtime code modification
`BRK #imm`	Software breakpoint → debug exception
`HLT #imm`	Halt into the external debugger
`UDF`	Permanently undefined — used to trap deliberately
`TLBI`, `DC`, `IC`, `AT`	TLB, data cache, instruction cache maintenance; address translation

```
/* The WFE/SEV spinlock idiom — spin without burning power: */
1:      ldaxr   w1, [x0]
        cbnz    w1, 2f              // locked? go wait
        stxr    w2, w3, [x0]
        cbnz    w2, 1b
        ...                          // got it
2:      wfe                          // sleep until an event
        b       1b
/* The unlock does SEV (or a store-release, which generates an
   event on the exclusive monitor). This is why ARM spinlocks
   consume far less power than a naive busy loop.            */
```

Compiler idioms you will see constantly

```
Learn to recognise these on sight — they cover most disassembly:

  stp x29, x30, [sp, #-N]!     function prologue
  mov x29, sp
  ldp x29, x30, [sp], #N       epilogue
  ret

  adrp x0, sym                 global variable access
  add  x0, x0, :lo12:sym       (or the offset folded into the LDR)

  cmp  w0, w1                  comparison + branchless select
  csel w0, w0, w1, gt

  ldr  x0, [x1, x2, lsl #3]    array[i] for 8-byte elements

  cbz  x0, .Lnull              null check

  bl   memcpy                  a struct copy the compiler lowered

  ld1 {v0.4s}, [x0], #16       vectorised loop body
  fadd v0.4s, v0.4s, v1.4s
  st1 {v0.4s}, [x1], #16

  nop                          a patch point: static keys, ftrace,
                               or alternative instruction patching
```

Looking things up

```
# the fastest reference loop
#   godbolt.org — paste C, see the assembly, hover for docs
#   developer.arm.com/documentation — searchable A64 ISA guide

# locally
aarch64-linux-gnu-objdump -d prog | less
gcc -O2 -S -o - test.c

# decode a single instruction word
echo '.word 0xd65f03c0' > t.s && as -o t.o t.s && objdump -d t.o
#   d65f03c0  →  ret

# the ARM ARM's index of instructions by encoding is section C6.
# When you meet an unfamiliar mnemonic, search there rather than
# guessing from the name — several are aliases of something else.
```

How it shows up

Symptom	Cause
A simple mask compiles to two instructions	The value is not a valid bitmask immediate
"relocation truncated to fit" at link time	Branch target beyond ±128 MB; the linker needs a veneer
Poor branch prediction on returns	Used `BR X30` instead of `RET`
Modulo is unexpectedly slow	No remainder instruction — it is `SDIV` plus `MSUB`
Compiler emitted `CSEL` where you expected a branch	It went branchless — usually what you want

Symptom	Cause
Illegal instruction on some cores	Optional feature (crypto, LSE, SVE) without an ID check
Spin loop burns power	No `WFE`; use the event-based idiom
Unexplained `NOP`s in the disassembly	Patch points for ftrace, static keys or alternatives

Check yourself

- ✓ Name the instruction families and give one representative of each.
- ✓ Explain how a 64-bit constant is built and why some masks are one instruction.
- ✓ Give three branch instructions that do not need a preceding `CMP`.
- ✓ Explain why `RET` is preferable to `BR X30`.
- ✓ Write branchless `max` and identify the instruction the compiler uses.
- ✓ Explain what `CSETM` produces and which C idiom maps to it.
- ✓ Show how `a % b` is computed and why it costs two instructions.
- ✓ Explain the `WFE`/`SEV` spinlock idiom and what it saves.

Addressing modes

Since only loads and stores touch memory, how they compute an address is where a surprising amount of performance lives. Array indexing, struct access, stack frames and pointer walking each map onto a specific mode, and recognising them makes disassembly readable.

The modes

Syntax	Address used	Side effect on `Xn`	Compiles from
`[Xn]`	Xn	None	`*p`
`[Xn, #imm]`	Xn + imm	None	`p->field`, `a[3]`
`[Xn, Xm]`	Xn + Xm	None	`p[i]` for byte arrays
`[Xn, Xm, LSL #s]`	Xn + (Xm << s)	None	**`a[i]`** — the scaled index
`[Xn, Wm, SXTW #s]`	Xn + (sign-extend Wm << s)	None	`a[i]` with a 32-bit signed index
`[Xn, #imm]!`	Xn + imm	**Xn += imm first**	Push; `*++p`
`[Xn], #imm`	Xn	**Xn += imm after**	Pop; `*p++`
`label` (PC-relative)	PC ± imm19×4	None	Literal pools, `ADR`

```
Pre-index versus post-index — the distinction that trips people up

   PRE-INDEX   [x0, #8]!      "update, then use"
        x0 = x0 + 8
        access [x0]

   POST-INDEX  [x0], #8       "use, then update"
        access [x0]
        x0 = x0 + 8

  Mnemonic: the ! is where the update happens relative to the
  bracket. Inside the bracket → before. After the bracket → after.

  This is exactly *++p versus *p++ in C, and it is why a linked-
  list walk or a memcpy loop needs no separate ADD instruction.
```

The scaled index, and why it matters

```
/* Array indexing is ONE instruction, because the shift is free —
   it happens in the address generation unit, not the ALU.   */

long a[100];
long f(long *a, int i) { return a[i]; }
/*      ldr     x0, [x0, w1, sxtw #3]     // x0 + (sign-ext w1)*8
        ret                                                    */

/* The shift amount matches the element size:
        #0  → 1-byte elements       #2  → 4-byte (int, float)
        #1  → 2-byte                #3  → 8-byte (long, pointer)
        #4  → 16-byte (Q registers)

   Note SXTW: a signed 32-bit index must be sign-extended to 64
   bits. If you index with a size_t (unsigned 64-bit) instead,
   the extension disappears:                                   */

long g(long *a, size_t i) { return a[i]; }
/*      ldr     x0, [x0, x1, lsl #3]      // no extension needed */

/* This is a real, if small, reason to prefer size_t for indices
   in hot loops — and it also removes the signed-overflow UB
   from the index arithmetic (C chapter 15).                */
```

Immediate offset ranges

```
The encodable range depends on the ACCESS SIZE, because the
offset is scaled by it:

   LDRB/STRB    unsigned imm12, scaled by 1   →  0 .. 4095
   LDRH/STRH    unsigned imm12, scaled by 2   →  0 .. 8190,  even
   LDR/STR (W)  unsigned imm12, scaled by 4   →  0 .. 16380, mult of 4
   LDR/STR (X)  unsigned imm12, scaled by 8   →  0 .. 32760, mult of 8

   Unscaled signed form (LDUR/STUR): -256 .. 255, any alignment.
   The assembler picks it automatically when needed.

   LDP/STP      signed imm7, scaled          →  -512 .. 504 for X

WHY THIS MATTERS IN PRACTICE:

  · a struct field at offset 32760 is one instruction; at 32768
    it needs an extra ADD. Very large structs cost more to access.

  · this is why the compiler sometimes rebases a pointer into the
    middle of a large structure — to bring fields into range.

  · stack frames larger than ~32 KB generate extra address
    arithmetic on every local variable access. Another reason
    large stack arrays are a bad idea (C chapter 09).
```

Load and store pair

```
/* LDP/STP move TWO registers in ONE instruction. This is why
   prologues save registers in pairs and why the stack moves in
   multiples of 16 bytes.                                      */

        stp     x29, x30, [sp, #-32]!   // push both, pre-decrement
        stp     x19, x20, [sp, #16]
        ...
        ldp     x19, x20, [sp, #16]
        ldp     x29, x30, [sp], #32     // pop both, post-increment

/* Also used for bulk copies - a 16-byte-at-a-time memcpy: */
1:      ldp     x2, x3, [x1], #16
        stp     x2, x3, [x0], #16
        subs    x4, x4, #16
        b.ne    1b

/* And with Q registers, 32 bytes at a time: */
        ldp     q0, q1, [x1], #32
        stp     q0, q1, [x0], #32

/* Constraint: both registers must be the same size, and the
   offset is a signed 7-bit value scaled by the access size -
   a much narrower range than a single LDR.                  */
```

PC-relative addressing

```
        adr     x0, label        // PC ± 1 MB, byte granular
        adrp    x0, symbol       // PC ± 4 GB, PAGE granular (4 KB)

/* ADRP gives the address of the 4 KB PAGE containing the symbol,
   with the low 12 bits ZEROED. The low bits are supplied
   separately:                                              */

        adrp    x0, my_global
        add     x0, x0, :lo12:my_global     // full address
        ldr     w1, [x0]

/* Or folded into the load's own offset — one instruction fewer: */
        adrp    x0, my_global
        ldr     w1, [x0, :lo12:my_global]

/* Why the split: a single 32-bit instruction cannot encode a
   ±4 GB byte offset. Splitting it into a page offset (21 bits)
   plus a 12-bit low part covers the full range.

   This is also why the LINKER has relocation types specifically
   for the two halves (R_AARCH64_ADR_PREL_PG_HI21 and
   R_AARCH64_ADD_ABS_LO12_NC), and why an ADRP/ADD pair must not
   be separated by the compiler — Cortex-A53 erratum 843419 is
   precisely a bug in that sequence (chapter 01).           */

/* LITERAL POOLS: a PC-relative load of a constant stored in the
   code stream. The assembler's = syntax generates one:      */
        ldr     x0, =0x123456789abcdef0
/*  becomes:
        ldr     x0, .Lpool
        ...
    .Lpool: .quad 0x123456789abcdef0                         */
```

Access sizes and extension

```
        ldrb    w0, [x1]        // load byte,  ZERO-extend to 32
        ldrh    w0, [x1]        // load half,  zero-extend
        ldr     w0, [x1]        // load word   (upper 32 of x0 zeroed)
        ldr     x0, [x1]        // load 64 bits

        ldrsb   w0, [x1]        // load byte,  SIGN-extend to 32
        ldrsb   x0, [x1]        // ...to 64
        ldrsh   x0, [x1]
        ldrsw   x0, [x1]        // load word, sign-extend to 64
                                //   — very common for int → long

        strb    w0, [x1]        // store the low 8 bits
        strh    w0, [x1]
        str     w0, [x1]        // store 32
        str     x0, [x1]        // store 64

/* Note there is no STRSB: sign extension is a property of
   LOADING, not storing. And remember that writing Wn zeroes the
   top half of Xn (chapter 04), so LDR Wn is implicitly a
   zero-extending 64-bit write.                             */
```

Ordered, exclusive and non-temporal

Form	Meaning	Chapter
`LDAR` / `STLR`	Load-acquire / store-release — ordering attached to the access	21
`LDXR` / `STXR`	Exclusive, for atomic sequences	22
`LDAXR` / `STLXR`	Both: exclusive and ordered	22
`LDNP` / `STNP`	Non-temporal pair — a hint not to pollute the cache	17
`LDTR` / `STTR`	Unprivileged — behaves as an EL0 access even when executed at EL1. Used by `copy_to_user`.	05
`PRFM`	Prefetch hint: `PLDL1KEEP`, `PSTL2STRM`...	CA 13

```
/* LDTR/STTR are worth understanding: they let kernel code access
   user memory with USER permissions, so a malicious pointer
   pointing at kernel memory faults rather than succeeding. That
   is how copy_from_user enforces the boundary in hardware
   rather than by checking the address (OS chapter 03).      */

/* Non-temporal, for streaming through data you will not reuse: */
1:      ldnp    q0, q1, [x1], #32
        stnp    q0, q1, [x0], #32
        subs    x2, x2, #32
        b.ne    1b
/* Avoids evicting the working set for a bulk copy — the same
   idea as MADV_DONTNEED at a different level.               */
```

Reading addressing in disassembly

```
/* Each pattern tells you what the source code was doing: */

  ldr  x0, [x1]                 *p
  ldr  x0, [x1, #24]            p->field  (at offset 24)
  ldr  x0, [x1, x2, lsl #3]     a[i]      (8-byte elements)
  ldr  x0, [x1], #8             *p++      (pointer walk)
  ldr  x0, [x1, #8]!            *++p
  ldp  x0, x1, [x2, #16]        two adjacent fields, one instruction
  ldr  x0, [sp, #40]            a local variable, or a spilled reg
  adrp x0, sym / add x0,x0,:lo12: a global
  ldr  w0, [x1, w2, sxtw #2]    a[i] with an int index

/* And the absence of loads inside a loop tells you the compiler
   kept everything in registers — which is the goal.        */
```

THE ADDRESSING MODE REVEALS THE DATA STRUCTURE

When reverse-engineering or reviewing generated code, the addressing mode is the strongest clue about the underlying type. A constant offset means a struct field; a scaled register offset means an array; post-index means a sequential walk; an indirect `BLR` after a load means a function pointer or vtable (C ch. 12). Reading these patterns is a large part of what makes disassembly useful rather than opaque.

How it shows up

Symptom	Cause
Extra `ADD` before every struct access	Field offset beyond the immediate range — the struct is very large
Extra sign-extension in an indexing loop	Index is `int`; use `size_t`
Pointer advances unexpectedly	Post-index side effect — `[Xn], #imm` updates `Xn`
Stack access needs extra arithmetic	Frame larger than the encodable offset range
Alignment fault on a Device-memory access	Unaligned MMIO always faults (ch. 11)
Link error about relocation truncation	Symbol beyond `ADRP`'s ±4 GB, or a branch beyond ±128 MB
Kernel accepts a user pointer into kernel memory	Used a normal load instead of `LDTR` / `copy_from_user`
A bulk copy evicts the working set	Use `LDNP` / `STNP`

Check yourself

- ✓ Name the addressing modes and the C construct each typically compiles from.
- ✓ Explain pre-index versus post-index and the C operators they correspond to.
- ✓ Explain why `a[i]` is one instruction and what the shift amount encodes.
- ✓ Say why the immediate offset range differs between `LDRB` and `LDR X`.
- ✓ Explain what `ADRP` produces and why the low bits are separate.
- ✓ Give three load forms that sign-extend and say why there is no store equivalent.
- ✓ Explain what `LDTR` is for and how it enforces the user/kernel boundary.
- ✓ Given `ldr x0, [x1, x2, lsl #3]`, describe the source construct.

The procedure call standard (AAPCS64)

The ABI is the contract that lets separately compiled code interoperate. You need it to write assembly that C can call, to read stack traces, to debug a corrupted frame, and to understand why a struct passed by value sometimes costs nothing and sometimes costs a memory copy.

Register roles

```
  X0–X7     arguments 1–8, and return value in X0
            (X0:X1 for a 128-bit return)          CALLER-saved
  X8        indirect result location register
            – and, on Linux, the syscall number   CALLER-saved
  X9–X15    temporaries                           CALLER-saved
  X16,X17   IP0/IP1 – linker veneers, PLT stubs   may be clobbered
                                                  by the LINKER
  X18       platform register – OS-specific       don't touch
  X19–X28   general use                           CALLEE-saved
  X29       FP  frame pointer                     CALLEE-saved
  X30       LR  link register                     CALLEE-saved
  SP        stack pointer                         CALLEE-saved

  V0–V7     FP/SIMD arguments and return          CALLER-saved
  V8–V15    LOWER 64 BITS callee-saved            partially saved
  V16–V31   temporaries                           CALLER-saved

Note the oddity in V8–V15: only the low 64 bits are preserved.
Assembly that uses the full 128 bits of V8 must save it itself.
This catches out hand-written SIMD kernels regularly.
```

CALLER-SAVED VERSUS CALLEE-SAVED

Caller-saved (volatile): if you need the value after a call, *you* save it. The callee may destroy it freely.

Callee-saved (non-volatile): if a function uses one, it must restore the original value before returning. The caller may assume it survives.

The split exists so neither side saves more than necessary: values needed across a call go in callee-saved registers, short-lived temporaries go in caller-saved ones, and the compiler allocates accordingly.

The stack

```
   higher addresses
   ┌──────────────────────────┐
   │ caller's frame           │
   ├──────────────────────────┤
   │ arguments 9, 10, ...     │  ← beyond the 8 register arguments
   ├──────────────────────────┤  ← SP on entry, 16-byte aligned
   │ saved X29 (caller's FP)  │  ┐
   │ saved X30 (return addr)  │  │ this frame
   ├──────────────────────────┤  │
   │ saved X19–X28 (as used)  │  │
   ├──────────────────────────┤  │
   │ local variables          │  │
   │ spilled temporaries      │  ┘
   └──────────────────────────┘  ← SP now
   lower addresses            (grows DOWN)

RULES:
  · SP must be 16-BYTE ALIGNED at every public interface.
    Not 8. Sixteen. This is not advisory — instructions that
    assume it will fault, and the fault looks like random
    corruption.
  · SP must always be valid: an exception can occur at any
    instruction and the handler may use it.
  · The RED ZONE that exists on x86-64 does NOT exist here.
    You may not write below SP.
```

```
/* A standard non-leaf function */
func:
        stp     x29, x30, [sp, #-32]!   // allocate 32, save FP and LR
        mov     x29, sp                 // establish the frame pointer
        stp     x19, x20, [sp, #16]     // save what we will use
        ...
        ldp     x19, x20, [sp, #16]
        ldp     x29, x30, [sp], #32     // restore, deallocate
        ret

/* A LEAF function — calls nothing, so LR is untouched and no
   frame is needed at all:                                    */
square: mul     w0, w0, w0
        ret

/* Large frames need an explicit adjustment, because the
   pre-index immediate is limited to ±512 for STP:           */
big:    stp     x29, x30, [sp, #-16]!
        mov     x29, sp
        sub     sp, sp, #4096           // room for locals
        ...
        add     sp, sp, #4096
        ldp     x29, x30, [sp], #16
        ret
```

The frame pointer chain

```
X29 points at a two-word record: {caller's X29, return address}
Each record points to the previous one, forming a linked list:

     X29 ──► ┌─────────────────┐
             │ caller's X29 ├──► ┌─────────────────┐
             │ return addr  │    │ caller's X29 ├──► ...
             └─────────────────┘    │ return addr  │
                                    └─────────────────┘

THIS is how a debugger or profiler walks a stack:
     fp = X29
     while (fp) {
             print_symbol(*(fp + 8));   // return address
             fp = *(fp + 0);            // previous frame
     }

Which is why -fomit-frame-pointer breaks naive stack walking,
and why perf and the kernel want -fno-omit-frame-pointer.
The alternative is DWARF CFI unwinding, which is exact but
requires parsing .eh_frame — far more expensive, and not
usable from an interrupt handler.

Linux arm64 builds with frame pointers ON for this reason.
```

How arguments are assigned

```
/* The rules, applied left to right: */

/* 1. Integers and pointers → the next free X register (X0–X7) */
void f(int a, long b, void *c, char d);
/*     w0      x1       x2      w3                            */

/* 2. Floats and vectors → the next free V register (V0–V7),
      in a SEPARATE sequence from the integer registers        */
void g(int a, double b, int c, float d);
/*     w0      d0        w1      s1
   Note: integers and floats do not compete for slots.        */

/* 3. Small structs (≤ 16 bytes) → passed in up to two registers */
struct pair { long a, b; };
void h(struct pair p);
/*     x0 = p.a, x1 = p.b   — no memory involved              */

/* 4. HFAs/HVAs — a struct of 1–4 identical float members —
      go in consecutive V registers                            */
struct vec3 { float x, y, z; };
void i(struct vec3 v);
/*     s0, s1, s2                                             */

/* 5. Larger structs → COPIED TO THE STACK, and a pointer to
      that copy is passed                                      */
struct big { long a[10]; };
void j(struct big b);
/*     the caller memcpys 80 bytes onto the stack             */

/* 6. Large RETURN values → the caller allocates space and
      passes its address in X8                                 */
struct big k(void);
/*     caller: add x8, sp, #N ; bl k
       callee writes the result through x8                    */

/* 7. Remaining arguments → the stack, right after the frame  */
```

Return type	Returned in
`int`, `long`, pointer	`X0`
128-bit integer	`X0` : `X1`
`float`, `double`	`S0` / `D0`
Struct ≤ 16 bytes	`X0` : `X1`
HFA of ≤ 4 floats	`V0` – `V3`
Anything larger	Memory, via the pointer in `X8`
`void`	Nothing

> **THE PRACTICAL CONSEQUENCE**
>
> A struct of two `long`s costs nothing to pass — it goes in two registers. A struct of three costs a stack copy on every call. That 16-byte boundary is a real, measurable cliff, and it is a good reason to keep small value types genuinely small, or to pass a `const` pointer instead (C ch. 11).

Variadic functions

```
/* Variadic arguments follow the same register rules, but the
   callee must be able to walk them, so va_start saves ALL the
   argument registers into a "register save area" on the stack. */

int printf(const char *fmt, ...);
/*  X0 = fmt, then X1-X7 and V0-V7 hold the variadic args.
    va_start spills X1-X7 and V0-V7 to the stack, then va_arg
    walks that area and then the overflow stack area.        */

/* Which is why:
     · a variadic function has a larger prologue than it looks
     · Apple's arm64 ABI differs here: it passes ALL variadic
       arguments on the STACK, which is a real portability
       difference for hand-written assembly and for JITs
     · default argument promotions apply: float → double,
       narrow ints → int (C chapter 11)                      */

/* The va_list layout is defined by the ABI: */
typedef struct {
        void *__stack;          // next stack argument
        void *__gr_top;         // top of the general register area
        void *__vr_top;         // top of the vector register area
        int   __gr_offs;        // offset into the GP area (negative)
        int   __vr_offs;
} va_list[1];
```

Writing assembly C can call

```
        .text
        .global my_memcpy
        .type   my_memcpy, %function
my_memcpy:
        // x0 = dst, x1 = src, x2 = n   - per the ABI
        // returns dst in x0, so save it
        mov     x3, x0                  // x3 is caller-saved: free

        cbz     x2, 2f
1:      ldrb    w4, [x1], #1            // x4, x5: caller-saved
        strb    w4, [x3], #1
        subs    x2, x2, #1
        b.ne    1b
2:      ret                             // x0 still holds dst
        .size   my_memcpy, . - my_memcpy

/* CHECKLIST for hand-written assembly:
     ☐ arguments read from X0-X7 in order
     ☐ return value placed in X0
     ☐ X19-X28, X29, X30 saved if used
     ☐ low 64 bits of V8-V15 saved if used
     ☐ SP 16-byte aligned at every call you make
     ☐ SP restored exactly before RET
     ☐ X18 untouched
     ☐ .type %function and .size set, so the debugger and
       unwinder recognise it                                 */
```

```
/* Calling assembly from C: declare it with a matching prototype.
   A mismatch is a LINK-time success and a RUNTIME disaster -
   the linker matches by NAME only (C chapter 01).           */

extern void *my_memcpy(void *dst, const void *src, size_t n);

/* And from C++, wrap it: */
extern "C" void *my_memcpy(void *, const void *, size_t);
```

Stack unwinding

```
/* Two mechanisms, with different trade-offs: */

/* 1. FRAME POINTER walking - fast, simple, works anywhere,
      including in an interrupt handler. Requires that every
      frame in the chain has a frame pointer.               */
        -fno-omit-frame-pointer

/* 2. DWARF CFI (.eh_frame) - exact, works without frame
      pointers, but requires parsing unwind tables. Used by
      C++ exceptions, backtrace(), and gdb.                 */
        -funwind-tables
        readelf --debug-dump=frames prog

/* In the kernel, arm64 supports both: frame-pointer unwinding
   for oops traces and ftrace, and now ORC-style tables for
   reliable stacktraces used by live patching.

   The practical rule: if your profiler shows broken or
   truncated stacks, either build with frame pointers or ensure
   the profiler uses DWARF:                                  */
perf record --call-graph=fp   ./prog     # needs frame pointers
perf record --call-graph=dwarf ./prog    # slower, more accurate
```

Pointer authentication and the ABI

```
/* With -mbranch-protection=standard, the prologue and epilogue
   gain two instructions:                                    */

func:   paciasp                         // sign LR using SP as context
        stp     x29, x30, [sp, #-16]!
        ...
        ldp     x29, x30, [sp], #16
        autiasp                         // authenticate LR
        ret                             // faults if it was tampered

/* The signature occupies the unused upper bits of the pointer.
   An attacker who overwrites the saved LR cannot produce a valid
   signature, so the AUT fails and the RET faults.

   Note the interaction with unwinding: a debugger walking the
   stack sees SIGNED return addresses and must strip the
   authentication bits (XPACI) before symbolising them. Older
   tools show garbage addresses on PAC-enabled binaries.    */

/* BTI adds a landing-pad requirement on indirect branches: */
func:   bti     c                       // valid target for BLR
        ...
```

Debugging ABI problems

```
# check the arguments a function actually received
gdb) info args
gdb) info frame
gdb) p $x0
gdb) bt

# the frame pointer chain, by hand
gdb) p/x $x29
gdb) x/2gx $x29          # {caller's fp, return address}

# see what the compiler expects
gcc -O2 -S test.c        # look at the prologue and argument use

# verify alignment at a call site
gdb) p $sp % 16          # must be 0

# a corrupted stack trace, or a return to a garbage address,
# almost always means:
#   · SP misaligned
#   · a callee-saved register clobbered
#   · the frame pointer not established or not restored
#   · a prototype mismatch between caller and callee
```

How it shows up

Symptom	Cause
Caller's local variables corrupted after a call	Callee clobbered a callee-saved register
Return to a garbage address	`X30` not saved, or the stack was corrupted
Alignment fault deep inside libc	SP not 16-byte aligned at the call
Float arguments arrive as garbage	Missing prototype, so they went in the wrong register class
Struct-by-value call is unexpectedly slow	Larger than 16 bytes — a memory copy per call
Assembly works on Linux, fails on Windows or macOS	`X18` use, or Apple's variadic-on-stack rule
SIMD kernel corrupts the caller	Used the full 128 bits of `V8`–`V15`; only the low 64 are preserved
Stack traces truncated in perf	Built with `-fomit-frame-pointer`
Backtrace shows nonsense addresses on a hardened build	PAC-signed return addresses not stripped by the tool

Check yourself

- ✓ Name the argument registers, the return register, and the callee-saved set.
- ✓ Explain caller-saved versus callee-saved and why both exist.
- ✓ State the stack alignment rule and one symptom of violating it.
- ✓ Draw the frame pointer chain and show how a debugger walks it.
- ✓ Explain how a 12-byte struct, a 24-byte struct and a `struct vec3` are each passed.
- ✓ Say what `X8` is for on a large struct return.
- ✓ Give the checklist for hand-written assembly that C will call.
- ✓ Explain what `PACIASP` / `AUTIASP` do and how they interact with unwinding.

How an instruction executes

CA chapter 11 covered pipelining in general. This chapter follows one AArch64 instruction through a real core, showing where the MMU, the caches and the exception machinery each enter the picture — and what can go wrong at each point.

The pipeline, concretely

```
   ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐
   │ FETCH   │→│ DECODE  │→│ RENAME  │→│ ISSUE   │→│EXECUTE  │→│ RETIRE  │
   │         │ │         │ │         │ │         │ │ MEMORY  │ │         │
   └─────────┘ └─────────┘ └─────────┘ └─────────┘ └─────────┘ └─────────┘
        ▲           ▲                     ▲           ▲           ▲
        |           |                     |           |           |
   branch       fixed 32-bit          scheduler   MMU + caches  in ORDER,
   predictor    encoding makes        picks ready               architectural
   + I-TLB      this cheap and        instructions              state updates
   + I-cache    parallel                                        HERE ONLY

  Depth varies enormously by core:
     Cortex-A53     8 stages, in-order, 2-wide
     Cortex-A78    13 stages, out-of-order, 4-wide fetch/6-wide issue
     Neoverse V2   deeper still, 8-wide

  Same architecture. Same binary. Very different behaviour — the
  distinction from chapter 01.
```

Fetch: where translation begins

```
/* The PC is a VIRTUAL address. Before any instruction bytes can be
   read, it must be translated.                                    */

   PC → I-TLB lookup
          hit  → physical address in ~1 cycle
          miss → hardware page table walk (chapter 13)
                 walk fails → INSTRUCTION ABORT
                              ESR.EC = 0x20 (lower EL) or 0x21
        → L1 instruction cache
          hit  → up to 4-8 instructions returned
          miss → L2 → interconnect → DRAM

/* In parallel, the branch predictor is already producing the NEXT
   fetch address, so the front end never waits for decode to
   discover that an instruction was a branch (CA chapter 12).  */

/* Three things can go wrong here:
     · translation fault      → instruction abort
     · permission fault       → the page is XN (execute-never)
     · the page is Device     → instruction fetch from Device
                                memory is UNPREDICTABLE. Never
                                execute from an MMIO mapping.  */
```

Decode and rename

```
/* Fixed 32-bit encodings mean the decoder can split N instructions
   simultaneously with no serial dependency - the single biggest
   front-end advantage of the ISA (chapter 03).               */

/* Two things happen that software cannot see but must understand: */

/* 1. CRACKING - some instructions become several micro-operations */
        ldp     x0, x1, [x2]        // may become two loads
        stp     x0, x1, [x2, #16]!  // store + address update

/* 2. REGISTER RENAMING - architectural registers are mapped onto
      a much larger physical register file.                    */

        add     x0, x1, x2      // x0 → physical reg p37
        ldr     x0, [x3]        // x0 → physical reg p52  (a NEW one)
        add     x4, x0, #1      //         reads p52

/* Renaming removes WAR and WAW hazards entirely: the second write
   to x0 does not have to wait for the first, because they are
   different physical registers. Only TRUE dependencies (RAW)
   remain, and those are the ones you can attack in source code
   with multiple accumulators (CA chapter 19).               */
```

The memory stage, in detail

```
This is where most of the interesting failures live.

   effective address computed (base + offset, or base + index<<s)
        |
        ▼
   ① ALIGNMENT CHECK
        Device memory: ANY unaligned access faults
        Normal memory: allowed, unless SCTLR.A is set
        SP-relative:   SP must be 16-byte aligned if SCTLR.SA
        |
        ▼
   ② TRANSLATION — D-TLB, tagged by ASID
        miss → hardware table walk, up to 4 memory accesses
        invalid descriptor → TRANSLATION FAULT
              ESR.EC = 0x24/0x25, FAR = the address
        |
        ▼
   ③ PERMISSION CHECK
        AP bits vs current EL and access type
        PXN/UXN for instruction fetch
        PAN: may EL1 touch this user page right now? (chapter 05)
        fail → PERMISSION FAULT
        |
        ▼
   ④ STAGE 2 — only if a hypervisor is active
        IPA → PA via VTTBR_EL2, second permission check
        fail → exception to EL2, not EL1 (chapter 16)
        |
        ▼
   ⑤ MEMORY TYPE from MAIR, via the descriptor's AttrIndx
        Normal cacheable  → probe the cache
        Device            → bypass the cache, no speculation,
                            no merging (chapter 11)
        |
        ▼
   ⑥ CACHE — L1 hit? L2? L3? interconnect?
        |
        ▼
   ⑦ INTERCONNECT — AXI/CHI transaction carrying
        address, master ID, NS bit, cacheability
        SMMU may translate it (for non-CPU masters)
        XPU may REFUSE it → external abort, DFSC = 0b010000
        |
        ▼
   ⑧ data returned, or the store retires

Every fault code you will decode in chapter 29 corresponds to a
specific step in this list.
```

Out-of-order execution and its limits

```
/* Instructions EXECUTE when their operands are ready, in any
   order. They RETIRE strictly in program order.             */

        ldr     x0, [x1]        // L2 miss: 40 cycles
        add     x2, x3, x4      // independent — executes NOW
        add     x5, x6, x7      // independent — executes NOW
        add     x8, x0, #1      // DEPENDENT on x0 — must wait

/* The reorder buffer holds results until retirement. Two
   properties follow, and both matter:                       */

/* 1. PRECISE EXCEPTIONS. If the load faults, everything after it
      is discarded before retiring, so the architectural state
      looks exactly as if execution stopped at the load. That is
      what makes demand paging possible: the OS fixes the mapping
      and the SAME instruction re-executes (OS chapter 12).  */

/* 2. SPECULATIVE state is NOT rolled back. Caches, TLBs and
      predictor tables were already changed. This is the entire
      basis of Spectre (CA chapter 12), and the reason ARM added
      CSDB, SB and SSBS.                                     */

/* What LIMITS the reordering window:
     · reorder buffer size (~100–500 entries)
     · load/store queue depth
     · physical register file size
     · a full BARRIER, which drains it deliberately (chapter 21) */
```

Where stalls come from

Cause	Cost	Source-level fix
L1 data miss	~12 cycles	Locality, layout (CA 13)
Last-level miss → DRAM	~200 cycles	Blocking, prefetch, smaller working set
TLB miss → page walk	10s–100s of cycles	Huge pages
Branch mispredict	10–20 cycles	Branchless code, predictable branches
True dependency chain	Latency of each link	**Multiple accumulators** (CA 19)
Divide	10–20 cycles, not pipelined	Avoid, or use reciprocal multiply
Store-to-load forwarding failure	10s of cycles	Avoid overlapping partial stores then wide loads
`ISB` / `DSB`	Full drain — expensive	Use the weakest barrier that suffices
Exclusive-loop contention	Unbounded retries	LSE atomics (ch. 22)

```
/* The dependency chain is the one you control most directly: */

/* 4-cycle FP add latency, serialised — 4 cycles per element */
for (i = 0; i < n; i++) sum += a[i];

/* four independent chains — the latencies overlap */
for (i = 0; i < n; i += 4) {
        s0 += a[i];   s1 += a[i+1];
        s2 += a[i+2]; s3 += a[i+3];
}
sum = (s0 + s1) + (s2 + s3);
/* Typically ~4x faster with an UNCHANGED instruction count. The
   compiler will not do this for floats without -ffast-math,
   because it changes the rounding (CA chapter 03).         */
```

Measuring what the pipeline is doing

```
# the top-down view: where are the issue slots going?
perf stat -e cycles,instructions,stall_frontend,stall_backend ./prog

#   stall_frontend high → INSTRUCTION supply
#        I-cache misses, branch mispredicts, ITLB misses
#        fix: code layout, PGO, fewer/predictable branches
#
#   stall_backend high  → DATA supply or execution
#        D-cache misses, dependency chains, divides
#        fix: data layout, more parallelism, better algorithms

# drill down
perf stat -e l1d_cache_refill,l2d_cache_refill,ll_cache_miss_rd \
          -e dtlb_walk,itlb_walk \
          -e br_mis_pred_retired,br_retired \
          -e inst_spec,inst_retired ./prog

#   inst_spec vs inst_retired = how much work was SPECULATED and
#   thrown away. A large gap means heavy mispredict waste.

# which instruction?
perf record -e l1d_cache_refill ./prog && perf annotate

# ARM's own tooling gives a full top-down breakdown
#   topdown-tool (from Arm's telemetry solution)
#   Arm Streamline, Arm MAP
```

> **IPC IS THE FIRST NUMBER TO LOOK AT**
>
> Instructions per cycle tells you immediately which world you are in. Below ~0.5 on a wide core means you are stalled — find out on what. Above ~2 means the core is busy and further gains must come from doing *less work*, i.e. a better algorithm. Between the two, look at the stall counters. This one number saves a great deal of misdirected optimisation.

Where exceptions fit

```
Any stage can raise an exception:

  FETCH    instruction abort — translation or permission
  DECODE   undefined instruction; trapped by a higher EL
  EXECUTE  FP exception, divide-by-zero (where trapped)
  MEMORY   data abort, alignment fault, external abort
  any      IRQ/FIQ arriving asynchronously
  any      SError — an imprecise external abort, often from
           a write posted long ago

SYNCHRONOUS exceptions are precise: the instruction is
identified exactly, and ELR points at it (or past it, for SVC).

ASYNCHRONOUS ones are taken between instructions, at a point
the hardware chooses.

SError is the awkward case: the reported PC has NOTHING to do
with the cause, because the failing transaction retired long
before the error came back. This is why external aborts from a
bus firewall (security chapter 20) are so hard to attribute,
and why an ESB (Error Synchronization Barrier) is used to
contain them at known points.

Chapter 28 covers the entry sequence; chapter 29 covers
decoding what happened.
```

How it shows up

Symptom	Cause
IPC below 0.5 with few cache misses	Long dependency chains, or frontend-bound
Loop 4× faster after manual unrolling	Broke a true dependency chain
Large gap between `inst_spec` and `inst_retired`	Heavy branch misprediction; work thrown away
Unaligned access faults on one region only	That region is mapped as Device memory
Fetching instructions from a peripheral hangs	Execution from Device memory is unpredictable
Fault address unrelated to the reported PC	SError — an imprecise, asynchronous abort
Code faster on a big core than expected	Deeper reorder window hides the latency
Barrier in a hot loop destroys throughput	`DSB` / `ISB` drains the pipeline; use a weaker one
Timing-dependent bug only on out-of-order cores	Reordering the in-order core happened not to perform

Check yourself

- ✓ Name the pipeline stages and say which two involve address translation.
- ✓ Explain what register renaming removes and what it cannot remove.
- ✓ List the eight checks a load passes through in the memory stage.
- ✓ Explain precise exceptions and why demand paging depends on them.
- ✓ Explain why speculative state is not rolled back, and one consequence.
- ✓ Give five sources of pipeline stalls and the source-level fix for each.

- ✓ Say what `stall_frontend` versus `stall_backend` tells you to do.
- ✓ Explain why SError is harder to diagnose than a data abort.

Memory types: Normal versus Device

ARM tags every region of memory with a type that changes what the hardware is permitted to do with accesses to it — reorder them, merge them, speculate them, cache them. Choosing the wrong type is one of the most common and most confusing driver bugs, because the symptom rarely points at the cause.

Why a type is needed at all

RAM AND REGISTERS ARE NOT THE SAME THING

Reading RAM twice gives the same answer and costs nothing. Reading a FIFO register twice *consumes two bytes*. Writing RAM has no effect until someone reads it; writing a control register starts a motor.

The optimisations that make RAM fast — caching, speculation, write merging, reordering — are all *incorrect* for a peripheral. So the architecture provides a type that switches them off, and the page table says which is which.

The two types

	Normal	Device
Used for	RAM, code, data, DMA buffers	Peripheral registers (MMIO)
Reordering	Freely, subject to the memory model	Not reordered relative to other Device accesses
Speculative access	Allowed — the CPU may read addresses you never asked for	**Never**
Merging / splitting	Allowed	Depends on sub-type
Repeating an access	Allowed	Not allowed
Unaligned access	Allowed	**Always faults**
Cacheable	Yes — write-back, write-through, or non-cacheable	Never
Exclusives (`LDXR`)	Supported	**Not supported** — undefined behaviour
Instruction fetch	Yes	Unpredictable — never execute from Device memory

Device sub-types

```
Written Device-<G|nG><R|nR><E|nE>, each letter a permission:

   G / nG   GATHERING - may two accesses be merged into one
                        larger access, or one access split?
   R / nR   REORDERING - may accesses to DIFFERENT addresses in
                        this region be reordered with each other?
   E / nE   EARLY WRITE ACKNOWLEDGEMENT - may a write be
                        acknowledged before it reaches the
                        endpoint (i.e. buffered en route)?

   Device-nGnRnE   nothing allowed. The strictest.
                   Use for: control registers where every write
                   must land, in order, before the next.
                   MAIR encoding 0x00

   Device-nGnRE    early ack permitted, still no gathering or
                   reordering. The common default for most
                   peripherals - noticeably faster than nGnRnE
                   because writes need not stall.
                   MAIR encoding 0x04

   Device-nGRE     reordering permitted
                   MAIR encoding 0x08

   Device-GRE      everything permitted. Use for framebuffers and
                   large streaming windows, where throughput
                   matters and individual accesses have no
                   side effects.
                   MAIR encoding 0x0C

  Ordering of strictness: nGnRnE > nGnRE > nGRE > GRE

  Note that even the most relaxed Device type is NOT cacheable and
  NOT speculatable. Those two properties define "Device".
```

Normal memory cacheability

```
/* Normal memory has independent INNER and OUTER cacheability
   attributes, each one of:                                    */

   Non-cacheable
   Write-Through,  Read-Allocate / Write-Allocate
   Write-Back,     Read-Allocate / Write-Allocate    ← the usual

/* "Inner" and "Outer" are implementation-defined boundaries.
   Typically Inner = the CPU cluster's caches (L1, L2), Outer =
   a system-level cache beyond it. Most software sets them the
   same.                                                       */

/* And a SHAREABILITY attribute, which is separate:           */
   Non-shareable       no coherency maintained
   Inner Shareable     coherent within the inner domain — this is
                       what SMP kernel memory uses
   Outer Shareable     coherent across a wider domain, including
                       some DMA masters

/* Shareability determines how far a barrier or a TLBI
   propagates (chapters 19 and 21). Getting it wrong gives you
   coherency bugs that only appear on multi-core.             */
```

MAIR: how a descriptor names a type

```
A page table descriptor has only 3 bits for attributes (AttrIndx).
Three bits cannot encode all the combinations above, so ARM uses
INDIRECTION:

   MAIR_EL1 holds EIGHT 8-bit attribute values, one per slot.
   The descriptor's AttrIndx selects a slot.

   MAIR_EL1 = 0x00000000_0044FF04
                            || ||
                            || |└─ slot 0 = 0x04 Device-nGnRE
                            || └── slot 1 = 0xFF Normal WB RW-alloc
                            |└──── slot 2 = 0x44 Normal Non-cacheable
                            └───── slot 3 = 0x00 Device-nGnRnE

   Encoding of each byte:
      0x00        Device-nGnRnE
      0x04        Device-nGnRE
      0x08        Device-nGRE
      0x0C        Device-GRE
      0xFF        Normal, Inner+Outer WB, RW-allocate
      0x44        Normal, Inner+Outer Non-cacheable
      high nibble = Outer, low nibble = Inner (for Normal)

   Change MAIR and EVERY page using that slot changes type at
   once — which is convenient, and also a way to break the whole
   system with one register write.
```

```
/* Setting it up, in boot code */
#define MAIR_DEVICE_nGnRnE  0x00
#define MAIR_DEVICE_nGnRE   0x04
#define MAIR_NORMAL_NC      0x44
#define MAIR_NORMAL_WB      0xFF

#define IDX_DEVICE_nGnRnE   0
#define IDX_DEVICE_nGnRE    1
#define IDX_NORMAL_NC       2
#define IDX_NORMAL_WB       3

        uint64_t mair =
                ((uint64_t)MAIR_DEVICE_nGnRnE << (8 * IDX_DEVICE_nGnRnE)) |
                ((uint64_t)MAIR_DEVICE_nGnRE  << (8 * IDX_DEVICE_nGnRE))  |
                ((uint64_t)MAIR_NORMAL_NC     << (8 * IDX_NORMAL_NC))     |
                ((uint64_t)MAIR_NORMAL_WB     << (8 * IDX_NORMAL_WB));

        asm volatile("msr mair_el1, %0" :: "r"(mair));
        asm volatile("isb");            /* chapter 06 */

/* then a descriptor references a slot: */
        pte = pa | PTE_VALID | PTE_AF
                 | PTE_ATTR(IDX_DEVICE_nGnRE)   /* bits [4:2] */
                 | PTE_UXN | PTE_PXN;           /* never execute MMIO */
```

The bugs, and why they are confusing

MMIO MAPPED AS NORMAL CACHEABLE

This is the classic, and it produces four distinct failure modes that look unrelated:

1. **Writes never reach the device.** They sit in a dirty cache line until eviction — which may be never, or may happen minutes later at an unrelated moment.
2. **Reads return stale values forever.** The first read caches the line; every subsequent read hits the cache. A status register appears frozen.
3. **A FIFO drains spontaneously.** Speculative execution reads addresses the program never touched. Reading a FIFO register *pops* it, so data vanishes with no code responsible.
4. **Writes are merged or reordered.** A sequence that must be written in order arrives coalesced, and the device sees an illegal state transition.

The tell: it often works under a debugger (single-stepping defeats the caching and speculation) and fails at speed. If a peripheral "works in debug, fails in release", check the mapping attributes before anything else.

NORMAL MEMORY MAPPED AS DEVICE

The reverse mistake is less dramatic but very costly: correctness is fine, performance collapses. No caching, no speculation, no merging — every access goes to DRAM individually. A memcpy through a Device-mapped buffer can be 50–100× slower than through Normal memory. This is a common cause of "why is my DMA buffer so slow to fill?".

```
/* Two more subtle ones: */

/* 1. Exclusives on Device memory are UNDEFINED. */
        /* A spinlock in a Device-mapped shared region will not
           work — LDXR/STXR need a coherent monitor that Device
           memory does not provide. Shared memory between a CPU
           and a device must be Normal Non-cacheable, not Device,
           if you intend to use atomics on it.                */

/* 2. Unaligned access to Device memory ALWAYS faults, even
      though the same code works on Normal memory.           */
        uint8_t *regs = ioremap(...);
        uint32_t v = *(uint32_t *)(regs + 1);   /* alignment fault */
        /* Which is why casting a byte pointer into a register
           block is a bug, and why register structs must have
           explicit padding (C chapter 07).                   */
```

How the OS picks the type for you

```
/* Linux — the mapping function encodes the choice: */

ioremap(phys, size)          /* Device-nGnRE — the default for MMIO */
ioremap_np(phys, size)       /* Device-nGnRnE — "non-posted", for
                                registers needing strict ordering  */
ioremap_wc(phys, size)       /* Normal Non-cacheable, write-combining
                                — framebuffers, large windows      */
ioremap_cache(phys, size)    /* Normal cacheable — ROM, or memory
                                the device does not modify         */
memremap(phys, size, MEMREMAP_WB)   /* explicitly Normal WB        */

/* And the accessors that go with them: */
readl(addr) / writel(v, addr)               /* with barriers        */
readl_relaxed() / writel_relaxed()          /* without              */

/* Windows */
MmMapIoSpaceEx(pa, len, PAGE_READWRITE | PAGE_NOCACHE);
MmMapIoSpaceEx(pa, len, PAGE_READWRITE | PAGE_WRITECOMBINE);
READ_REGISTER_ULONG / WRITE_REGISTER_ULONG

/* In device tree, a region's cacheability for DMA is expressed
   separately, by whether the device is coherent:              */
        dma-coherent;         /* device participates in coherency  */
        no-map;               /* kernel must not map this at all   */
```

DMA buffers: the third case

```
A DMA buffer is Normal memory, but WHICH Normal memory depends on
whether the device is coherent:

  COHERENT DEVICE (ACE-Lite, CHI, `dma-coherent` in DT)
     map Normal Write-Back Inner-Shareable
     the interconnect keeps the device's view coherent with the
     CPU's caches — no maintenance needed
     dma_alloc_coherent() gives you cacheable memory

  NON-COHERENT DEVICE
     option A: Normal Non-cacheable
               simple and correct, but CPU access is slow
               dma_alloc_coherent() gives you this on such systems
     option B: Normal Write-Back plus explicit cache maintenance
               fast CPU access, but you must clean before the
               device reads and invalidate before you read what
               it wrote (chapter 18, CA chapter 15)
               dma_map_single() / dma_sync_* do this for you

  NEVER map a DMA buffer as Device. It is memory, not registers:
  you want the CPU side to be fast, and the device does not care.

  This is precisely why you must use the DMA API rather than
  choosing yourself — it knows which case applies.
```

Checking what you got

```
# Linux: what is mapped where, and how
cat /proc/vmallocinfo | grep ioremap
#   0xffff800010000000-0xffff800010002000  8192 ioremap ...

cat /proc/iomem                    # physical resource map
cat /proc/self/pagemap             # per-page details (needs root)

# read MAIR and a descriptor from a debug module
pr_info("MAIR_EL1 = %016llx\n", read_sysreg(mair_el1));

# in gdb on a bare-metal target
p/x $MAIR_EL1
p/x $TCR_EL1
# then walk the tables by hand for the address in question
# (chapter 13) and read AttrIndx from bits [4:2]

# the AT instruction asks the hardware to translate for you
        at      s1e1r, x0          // stage-1, EL1, read
        mrs     x1, par_el1        // result: PA plus attributes
/* PAR_EL1 reports the memory attributes actually applied —
   the definitive answer to "how is this address mapped?"   */
```

> **`AT` PLUS `PAR_EL1` IS THE DEBUGGING TOOL TO REMEMBER**
>
> Rather than walking page tables by hand, ask the MMU. `AT S1E1R, Xn` performs a translation and writes the resulting physical address *and its memory attributes* into `PAR_EL1`. If the translation fails, `PAR_EL1.F` is set and the fault status tells you why. This answers "is this address mapped, and as what?" in two instructions, from a live system.

How it shows up

Symptom	Cause
Device works under a debugger, fails at speed	MMIO mapped cacheable; stepping hid it
Status register value never changes	Cached read hitting a stale line
Register writes have no effect	Sitting in a dirty cache line
FIFO loses data with no code reading it	Speculative reads on a cacheable MMIO mapping
Device sees an illegal register sequence	Writes merged or reordered — needs a stricter Device sub-type
Alignment fault on one region only	Device memory always faults on unaligned access
Spinlock in shared memory never acquires	Exclusives on Device memory are undefined
DMA buffer is extremely slow to fill	Mapped Device or Non-cacheable when it need not be
Framebuffer updates are slow	Mapped Device-nGnRnE instead of write-combining
Works on one SoC, fails on another	One is DMA-coherent, the other is not

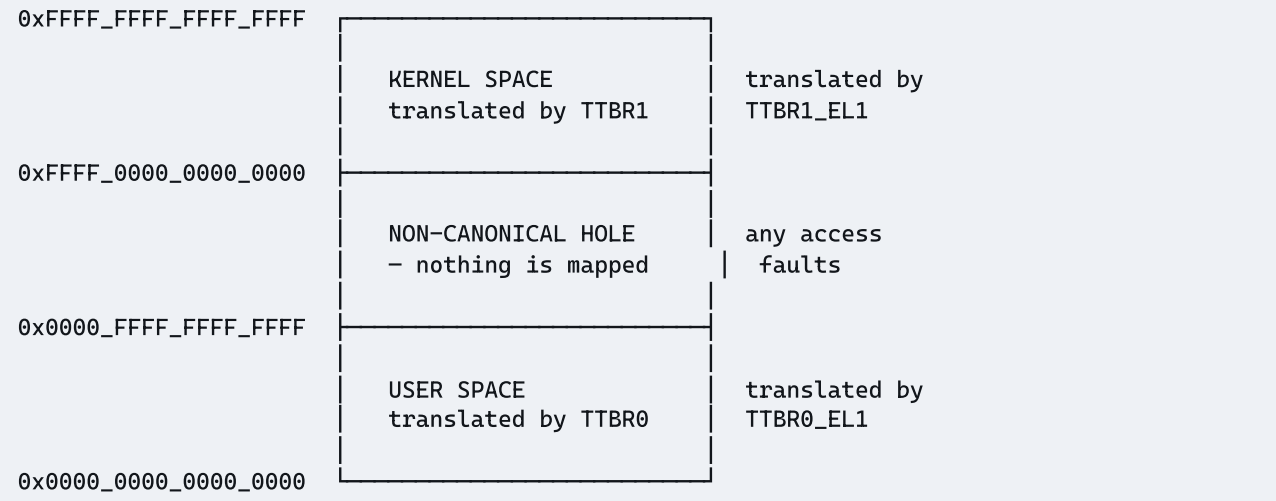
Check yourself

- ✓ Give three optimisations that are correct for RAM and wrong for a peripheral.
- ✓ List five differences between Normal and Device memory.
- ✓ Decode `Device-nGnRE` letter by letter and say when you would use it.
- ✓ Explain what MAIR indirection buys and how a descriptor names a type.
- ✓ Give the four failure modes of mapping MMIO as cacheable.
- ✓ Explain why unaligned access faults on Device memory but not Normal.
- ✓ Say why a DMA buffer should never be Device memory, and what determines its cacheability.
- ✓ Explain how `AT` and `PAR_EL1` answer "how is this address mapped?".

The MMU and the address space split

CA chapter 16 covered virtual memory as an idea. This chapter covers ARM's specific realisation of it: two translation table base registers, a split address space, and the configuration register that ties them together.

Two base registers, one address space




```
The hardware selects the table by the TOP BITS of the address:
   VA[55] (or VA[63], depending on TCR) == 0  →  TTBR0
                                        == 1  →  TTBR1

Bits above the configured VA size must be all-zeros or all-ones
("canonical"). Anything else is a translation fault — which is
why a corrupted pointer with garbage in the top bits produces a
clean fault rather than a wild access.
```

WHY TWO REGISTERS IS THE ELEGANT PART

The kernel mapping lives in `TTBR1_EL1` and never changes. A context switch only rewrites `TTBR0_EL1`. So the kernel stays mapped across every switch, a system call needs no table change at all, and the kernel's TLB entries survive — marked Global so they match under any ASID (chapter 15).

x86-64 has one table root and must place the kernel inside every process's page tables, which is exactly what made Meltdown mitigation (KPTI) so expensive there. ARM's split makes the same isolation nearly free.

TCR_EL1: the configuration register

```
TCR_EL1 controls almost everything about translation. The fields
you will actually set:

   T0SZ  [5:0]     size of the TTBR0 region: VA bits = 64 - T0SZ
                     T0SZ = 16  →  48-bit VA  →  256 TB
                     T0SZ = 25  →  39-bit VA  →  512 GB (3-level)
   T1SZ  [21:16]   the same, for TTBR1

   TG0   [15:14]   TTBR0 granule:  00 = 4KB, 01 = 64KB, 10 = 16KB
   TG1   [31:30]   TTBR1 granule:  10 = 4KB, 11 = 64KB, 01 = 16KB
                     ← note the encodings DIFFER between TG0 and TG1.
                       A classic boot bug.

   IRGN0 [9:8]     cacheability of TTBR0 table WALKS, inner
   ORGN0 [11:10]   ...outer
   SH0   [13:12]   shareability of table walks
   IRGN1/ORGN1/SH1 the same for TTBR1

   IPS   [34:32]   intermediate physical address size
                     000 = 32 bits, 101 = 48 bits, 110 = 52 bits

   TBI0  [37]      Top Byte Ignore — the top 8 bits of a VA are
   TBI1  [38]      not used for translation. This is what makes
                   pointer tagging, HWASAN and MTE possible.

   A1    [22]      which TTBR holds the ASID (0 = TTBR0)
   EPD0  [7]       disable TTBR0 walks entirely
   HA/HD [39:40]   hardware management of Access and Dirty bits
```

```
/* Why walk cacheability matters: a 4-level walk is up to four
   memory accesses. If the tables themselves are uncached, every
   TLB miss costs four DRAM round trips instead of four cache
   hits. Setting IRGN/ORGN to write-back is one of the highest-
   impact single settings in early boot — and forgetting it
   produces a system that boots but is inexplicably slow.    */

#define TCR_T0SZ(x)     ((64 - (x)) << 0)    /* x = VA bits */
#define TCR_T1SZ(x)     ((64 - (x)) << 16)
#define TCR_IRGN_WBWA   ((1UL << 8)  | (1UL << 24))
#define TCR_ORGN_WBWA   ((1UL << 10) | (1UL << 26))
#define TCR_SH_INNER    ((3UL << 12) | (3UL << 28))
#define TCR_TG0_4K      (0UL << 14)
#define TCR_TG1_4K      (2UL << 30)          /* NOT the same value */
#define TCR_IPS_48      (5UL << 32)

        uint64_t tcr = TCR_T0SZ(48) | TCR_T1SZ(48)
                     | TCR_IRGN_WBWA | TCR_ORGN_WBWA | TCR_SH_INNER
                     | TCR_TG0_4K | TCR_TG1_4K | TCR_IPS_48;
        asm volatile("msr tcr_el1, %0" :: "r"(tcr));
```

Granule size

Granule	Levels for 48-bit VA	Block sizes	Notes
4 KB	4 (L0–L3)	1 GB (L1), 2 MB (L2), 4 KB (L3)	Universal. What Linux uses by default.
16 KB	4 (L0–L3)	32 MB (L2), 16 KB (L3)	Apple platforms; good TLB reach with modest waste

Granule	Levels for 48-bit VA	Block sizes	Notes
64 KB	3 (L1–L3)	512 MB (L2), 64 KB (L3)	Fewer levels, best TLB reach, most internal fragmentation

Granule support is optional per implementation — query `ID_AA64MMFR0_EL1` before selecting one. A kernel configured for 64 KB pages will not boot on a core that only implements 4 KB.

Choosing the VA size

```
/* Reducing the VA size removes a whole LEVEL of the page walk. */

   T0SZ = 16  →  48-bit VA  →  L0, L1, L2, L3   (4 levels)
   T0SZ = 25  →  39-bit VA  →  L1, L2, L3       (3 levels)
   T0SZ = 34  →  30-bit VA  →  L2, L3           (2 levels)

/* Trade-off:
     fewer levels  → cheaper TLB miss (fewer memory accesses),
                     smaller tables
     smaller VA    → less address space per process

   Linux offers CONFIG_ARM64_VA_BITS of 39, 42, 47, 48 and 52.
   39 bits (512 GB) is common on embedded systems and saves a
   level; 48 is the desktop/server default.

   Note the kernel and user sizes are configured INDEPENDENTLY
   (T0SZ vs T1SZ), so you can have a large kernel space and a
   small user space, or vice versa.                          */
```

Turning the MMU on

> **THE MOST DELICATE SEQUENCE IN BOOT CODE**
>
> The instruction that enables the MMU is fetched with the MMU *off*, and the instruction after it may be fetched with the MMU *on*. If the currently executing code is not mapped at the same address in the new tables, execution falls off a cliff with no diagnostic whatsoever.
>
> The standard solution is an **identity map**: a temporary mapping where VA equals PA covering the enabling code itself. Enable the MMU, then branch to the final virtual address, then discard the identity map.

```
/* The sequence, with every step justified */

        /* 1. the tables must be VISIBLE to the table walker before
              it is told to use them. The walker reads memory, and
              our writes may still be in a store buffer.        */
        dsb     ish

        /* 2. install the roots and the configuration */
        msr     ttbr0_el1, x0           /* identity map          */
        msr     ttbr1_el1, x1           /* kernel map            */
        msr     tcr_el1, x2
        msr     mair_el1, x3

        /* 3. these must take effect before SCTLR is written    */
        isb

        /* 4. invalidate stale TLB entries from before the MMU
              was configured — there may be entries from a
              previous boot stage                               */
        tlbi    vmalle1
        dsb     nsh
        isb

        /* 5. enable: M (MMU), C (data cache), I (instruction cache) */
        mrs     x4, sctlr_el1
        orr     x4, x4, #(1 << 0)       /* M */
        orr     x4, x4, #(1 << 2)       /* C */
        orr     x4, x4, #(1 << 12)      /* I */
        msr     sctlr_el1, x4

        /* 6. the pipeline must be flushed so subsequent fetches
              are translated                                    */
        isb

        /* 7. now jump to the FINAL virtual address. Until this
              branch, we are running on the identity map.       */
        ldr     x5, =virtual_entry
        br      x5

virtual_entry:
        /* the identity map can now be removed */
```

Top Byte Ignore and pointer tagging

```
/* With TCR.TBI set, bits [63:56] of a virtual address are IGNORED
   for translation. Software may store anything there.       */

   0xFF00_1234_5678_9ABC
     ^^ ignored by the MMU

/* This enables several important features:                  */

/* HWASAN — hardware-assisted AddressSanitizer. A tag in the
   pointer is compared against a shadow-memory tag on each
   access. Far cheaper than classic ASan's redzones.        */

/* MTE — Memory Tagging Extension (ARMv8.5). The comparison moves
   into HARDWARE: each 16-byte granule of memory has a 4-bit tag,
   and the CPU checks it against the pointer's tag on every
   access. Use-after-free and buffer overflow detection at a few
   percent overhead — cheap enough for production. Android
   ships it.                                                */

/* Pointer authentication also lives in these bits when TBI is
   enabled (chapter 09).                                    */

/* The catch: any code that COMPARES pointers or uses them as
   hash keys must strip the tag first, or logically identical
   pointers compare unequal. This is why introducing tagging into
   an existing codebase surfaces latent pointer-arithmetic bugs. */
```

Where the ASID lives

```
/* The ASID is stored in the TOP 16 BITS of TTBR0_EL1 (or TTBR1,
   if TCR.A1 is set — but nobody does that).                */

        ttbr0 = page_table_pa | ((uint64_t)asid << 48);
        msr   ttbr0_el1, x0

/* So a context switch is ONE register write that changes both
   the translation tables and the address-space identity. The
   TLB keeps entries from both processes, tagged by ASID, and
   nothing needs flushing (chapter 15).                     */

/* ASID width is 8 or 16 bits, reported by ID_AA64MMFR0_EL1.ASIDBits.
   Linux uses a generation-rollover scheme: when ASIDs are
   exhausted it bumps a generation counter and flushes, which is
   rare but shows up as a periodic latency blip on systems with
   very many processes.                                     */
```

Translation regimes

```
Each exception level has its OWN translation configuration.
These are entirely separate sets of registers:

   EL1&0 regime     TTBR0_EL1, TTBR1_EL1, TCR_EL1, MAIR_EL1
                    used by EL0 and EL1 — one regime covering both,
                    which is why the split by top bit works

   EL2 regime       TTBR0_EL2, TCR_EL2, MAIR_EL2
                    the hypervisor's own mappings.
                    With VHE (ARMv8.1), EL2 gains TTBR1_EL2 and
                    behaves like EL1&0 — which is what lets a host
                    kernel run at EL2 unmodified.

   EL2 stage 2      VTTBR_EL2, VTCR_EL2
                    translates guest IPAs to PAs (chapter 16)

   EL3 regime       TTBR0_EL3, TCR_EL3, MAIR_EL3
                    single-level, no TTBR1 — the monitor's map

   Secure and Non-secure EL1&0 are also SEPARATE regimes, with
   independent register banks selected by SCR_EL3.NS. So the
   secure world has its own page tables entirely.
```

Inspecting translation

```
/* The AT instruction: ask the hardware to translate an address
   and report the result. The single most useful MMU debugging
   tool.                                                        */

        at      s1e1r, x0       /* stage 1, EL1, read        */
        at      s1e1w, x0       /* stage 1, EL1, write       */
        at      s1e0r, x0       /* as if from EL0            */
        at      s12e1r, x0      /* both stages               */
        isb
        mrs     x1, par_el1

/* PAR_EL1 on SUCCESS:
        bit 0    F = 0
        [51:12]  physical address
        [63:56]  memory attributes (the MAIR value applied)
        [9:8]    shareability

   PAR_EL1 on FAILURE:
        bit 0    F = 1
        [6:1]    FST — fault status, same encoding as ESR.DFSC
        bit 8    stage: 0 = stage 1, 1 = stage 2              */

static void dump_translation(unsigned long va)
{
        unsigned long par;
        asm volatile("at s1e1r, %0\n\tisb" :: "r"(va));
        asm volatile("mrs %0, par_el1" : "=r"(par));

        if (par & 1)
                pr_err("VA %lx: FAULT, FST=%lx\n", va, (par >> 1) & 0x3F);
        else
                pr_info("VA %lx → PA %lx, attrs %02lx\n",
                        va, (par & 0xFFFFFFFFF000UL) | (va & 0xFFF),
                        par >> 56);
}
```

```
# Linux side
cat /proc/self/maps                    # the VMAs (OS chapter 12)
cat /proc/kpageflags                   # per-physical-page state
cat /sys/kernel/debug/kernel_page_tables   # needs CONFIG_PTDUMP_DEBUGFS
#   ---[ Linear Mapping ]---
#   0xffff000000000000-0xffff000040000000  1G  PMD  RW NX SHD AF ...
#   — the definitive dump of what the kernel actually mapped

dmesg | grep -i 'Virtual kernel memory layout'
```

How it shows up

Symptom	Cause
Hangs immediately on enabling the MMU	No identity map covering the enabling code
Boots but everything is slow	Table walks configured non-cacheable in TCR
Kernel boots on one core, hangs on another	Missing `ISB` after the `SCTLR` write
Wrong granule accepted at build, faults at run	TG0 and TG1 use different encodings for the same size

Symptom	Cause
Kernel will not boot on a particular SoC	Granule size not implemented — check `ID_AA64MMFR0_EL1`
Pointer comparisons fail after enabling tagging	Tags not stripped before comparison
Access to a high address faults unexpectedly	Non-canonical — bits above the VA size are not all-ones
Periodic latency blip with many processes	ASID rollover forcing a full TLB flush
Guest translation works, host cannot see the mapping	Different translation regimes — separate register banks

Check yourself

- ✓ Explain how the hardware chooses between TTBR0 and TTBR1.
- ✓ Give two advantages of the two-register split over a single root.
- ✓ Name five `TCR_EL1` fields and what each controls.
- ✓ Explain why walk cacheability matters and what happens if you get it wrong.
- ✓ Say how reducing the VA size changes the page walk.
- ✓ Explain why enabling the MMU needs an identity map.
- ✓ Explain what Top Byte Ignore enables and one bug it exposes.
- ✓ Describe what `AT` plus `PAR_EL1` tells you, in both success and failure cases.

Translation tables and the page walk

The single most important mechanism in system software. Trace it by hand once and address translation stops being mysterious — page faults, TLB maintenance, hypervisors and the SMMU all become variations on this one walk.

Decomposing the address

```
48-bit VA, 4 KB granule — the configuration you will meet most

 63       48 47    39 38    30 29    21 20    12 11         0
┌───────────┬────────┬────────┬────────┬────────┬────────────┐
│ sign ext. │ L0 idx │ L1 idx │ L2 idx │ L3 idx │   offset   │
└───────────┴────────┴────────┴────────┴────────┴────────────┘
              9 bits   9 bits   9 bits   9 bits    12 bits

  9 bits = 512 entries per table.
  Each table is 512 × 8 bytes = 4096 bytes = exactly one page.
  That is not a coincidence: the table size is chosen to equal
  the granule so tables can be allocated from the page allocator.

  Coverage per entry:
     L0 entry → 512 GB      L2 entry →   2 MB
     L1 entry →   1 GB      L3 entry →   4 KB

  So one L1 entry can be a 1 GB BLOCK, one L2 entry a 2 MB block,
  or either can point to the next table. That choice is what
  makes huge pages possible.
```

The walk, step by step

THE 0B01 / 0B11 INVERSION AT L3

At levels 0–2, `0b11` means "table" and `0b01` means "block". At level 3 there is no next level, so `0b11` means "page" and `0b01` is **invalid**. Hand-written table code that uses a single "valid leaf" constant across levels produces a translation fault only at L3 — which looks like a random subset of addresses failing. It is one of the two or three most common bugs in bare-metal MMU setup.

A worked example

```
/* Translate VA = 0x0000_0000_4020_1000, 4 KB granule, 48-bit.

   Binary of the relevant bits:
     0x40201000 = 0100 0000 0010 0000 0001 0000 0000 0000

   Extract the indices:
     VA[47:39] = 0            → L0 index 0
     VA[38:30] = 1            → L1 index 1     (0x40000000 / 1 GB)
     VA[29:21] = 1            → L2 index 1     (0x00200000 / 2 MB)
     VA[20:12] = 1            → L3 index 1     (0x00001000 / 4 KB)
     VA[11:0]  = 0            → offset 0

   Walk, with TTBR0 = 0x80000000:

     L0: read  0x80000000 + 0*8  = 0x80000000
         value 0x0000000080001003
               ↑ next table at 0x80001000, bits[1:0]=11 = table

     L1: read  0x80001000 + 1*8  = 0x80001008
         value 0x0000000080002003   → table at 0x80002000

     L2: read  0x80002000 + 1*8  = 0x80002008
         value 0x0000000080003003   → table at 0x80003000

     L3: read  0x80003000 + 1*8  = 0x80003008
         value 0x00600000_9A00_0743
               bits[1:0] = 11 = valid page
               output address bits[47:12] = 0x9A000
               → PA = 0x9A000000 | 0x000 = 0x9A000000

   Do this once with a real dump from your own system. It converts
   the whole subject from abstract to obvious.                 */
```

Blocks: huge pages

```
A BLOCK descriptor terminates the walk early, mapping a large
contiguous region with ONE entry.

   L1 block   1 GB    used for the kernel linear map, and for
                      hugetlbfs 1 GB pages
   L2 block   2 MB    transparent huge pages, most large mappings
   L3 page    4 KB    the normal case

  Requirements: both the VA and the PA must be aligned to the
  block size, and the whole region must have identical
  permissions and attributes.

  BENEFIT — TLB reach. One entry covers 2 MB instead of 4 KB, so
  1024 TLB entries cover 2 GB rather than 4 MB (CA chapter 16).
  Also a shorter walk: 3 levels instead of 4.

  COST — internal fragmentation, and splitting a block later
  (to change permissions on part of it) requires break-before-make
  (below).

  CONTIGUOUS HINT: a descriptor bit saying "this entry and its
  15 neighbours form one contiguous, aligned region with identical
  attributes". The TLB may then use ONE entry for all 16 — a
  64 KB mapping from 4 KB pages, without needing a real block.
  Setting it when the claim is FALSE is a serious error: the TLB
  will happily use one entry for regions that differ.
```

Building tables in code

```
a minimal but correct table builder
```

```
#define PTE_VALID       (1UL << 0)
#define PTE_TABLE       (1UL << 1)   /* L0-L2: points to next table */
#define PTE_PAGE        (1UL << 1)   /* L3:    a 4 KB page          */
#define PTE_BLOCK       (0UL << 1)   /* L1-L2: a block              */
#define PTE_AF          (1UL << 10)  /* access flag - MANDATORY     */
#define PTE_SH_INNER    (3UL << 8)
#define PTE_AP_RW_EL1   (0UL << 6)
#define PTE_AP_RW_ALL   (1UL << 6)
#define PTE_AP_RO_EL1   (2UL << 6)
#define PTE_UXN         (1UL << 54)
#define PTE_PXN         (1UL << 53)
#define PTE_ATTR(i)     ((uint64_t)(i) << 2)

#define IDX(va, level)  (((va) >> (39 - 9 * (level))) & 0x1FF)

static uint64_t *alloc_table(void);   /* returns a zeroed 4 KB page */

static void map_page(uint64_t *l0, uint64_t va, uint64_t pa,
                     uint64_t attrs)
{
        uint64_t *tbl = l0;

        /* walk levels 0..2, creating tables as needed */
        for (int level = 0; level < 3; level++) {
                uint64_t *e = &tbl[IDX(va, level)];

                if (!(*e & PTE_VALID)) {
                        uint64_t *next = alloc_table();
                        /* the ZEROED table must be visible before
                           the descriptor that points at it        */
                        __asm__ volatile("dsb ishst" ::: "memory");
                        *e = (uint64_t)next | PTE_VALID | PTE_TABLE;
                } else if (!(*e & PTE_TABLE)) {
                        return;         /* a block is already here */
                }
                tbl = (uint64_t *)(*e & 0x0000FFFFFFFFF000UL);
        }

        /* level 3: the leaf */
        tbl[IDX(va, 3)] = (pa & 0x0000FFFFFFFFF000UL)
                        | PTE_VALID | PTE_PAGE | PTE_AF | attrs;

        __asm__ volatile("dsb ishst" ::: "memory");
}

/* Note the two DSBs. The table walker is a separate hardware
   observer that reads memory directly; your stores must be
   visible to it before the descriptor referencing them is
   written. Omitting these gives faults that appear only under
   load or only on certain cores.                            */
```

Break-before-make

> **YOU MAY NOT CHANGE A LIVE MAPPING IN PLACE**
> If a TLB holds an entry for a VA and you overwrite the descriptor with a different output address, size or attributes, the architecture permits the TLB to hold *both* old and new translations simultaneously. The result is UNPREDICTABLE — including a possible TLB conflict abort, or accesses going to either address non-deterministically.
>
> The required sequence is **break-before-make**:
>
> ```
> /* 1. BREAK: make the entry invalid */
> *pte = 0;
> dsb     ishst
>
> /* 2. invalidate the TLB entry, everywhere */
> tlbi    vaae1is, x_va
> dsb     ish
> isb
>
> /* 3. MAKE: write the new descriptor */
> *pte = new_descriptor;
> dsb     ishst
> isb
> ```
>
> This applies when changing the output address, the block size (splitting a 2 MB block into 4 KB pages), or the memory type. It is *not* required for a permission-only change from more permissive to less, or for setting the access/dirty bits — the architecture permits those to be atomic.

Hardware access and dirty bit management

```
/* Without hardware management (ARMv8.0):
     · AF starts clear; the first access takes an ACCESS FLAG FAULT
       and software sets it — this is how the OS tracks page usage
     · the dirty state is emulated by mapping the page read-only,
       taking a permission fault on the first write, and recording
       "dirty" in software before making it writable

   Both cost a fault per page. With HA/HD (ARMv8.1), the hardware
   sets AF and clears the DBM-controlled read-only bit itself, and
   the faults disappear.                                          */

        TCR_EL1.HA  = 1        /* hardware Access flag update */
        TCR_EL1.HD  = 1        /* hardware Dirty bit  update  */
        descriptor DBM bit     /* marks a page as dirty-managed */

/* Query support: ID_AA64MMFR1_EL1.HAFDBS
   Linux enables it when present — worth knowing because it
   changes the fault pattern you will see when profiling.    */
```

What can go wrong during a walk

Fault	DFSC	Meaning
Address size fault	`0b0000LL`	Output address exceeds the configured PA size

Fault	DFSC	Meaning
Translation fault	`0b0001LL`	Descriptor invalid at level LL — **not mapped**
Access flag fault	`0b0010LL`	AF was clear
Permission fault	`0b0011LL`	Mapped, but AP/XN forbids this access
Synchronous external abort	`0b010000`	The bus refused it — power, clock, or a firewall
External abort on a table walk	`0b0101LL`	The table memory itself was unreadable
TLB conflict abort	`0b110000`	Overlapping TLB entries — a break-before-make violation

The `LL` field gives the level, which is the most useful diagnostic in the whole set: a level-0 translation fault means the top-level table is wrong or `TTBR` is garbage; a level-3 fault means only the final page is missing. Chapter 29 covers decoding these in full.

Walking tables from a live system

```
# Linux: dump every mapping the kernel installed
mount -t debugfs none /sys/kernel/debug
cat /sys/kernel/debug/kernel_page_tables
#   ---[ Linear Mapping start ]---
#   0xffff000000000000-0xffff000040000000  1G PMD  RW NX SHD AF UXN MEM/NORMAL
#   ---[ vmalloc() area ]---
#   0xffff800008000000-0xffff800008001000  4K PTE  ro x  SHD AF UXN MEM/NORMAL
#         ^ size ^ level ^ perms ^ attrs

# per-process
cat /sys/kernel/debug/page_tables/current   # some configurations

# ask the hardware instead (chapter 12)
        at   s1e1r, x0
        mrs  x1, par_el1

# in a JTAG debugger, most tools have a built-in walker:
#   OpenOCD:  aarch64 mmu / mem2array
#   Trace32:  MMU.List, MMU.DUMP PageTable
#   gdb:      write a small python walker — worth doing once
```

How it shows up

Symptom	Cause
Some addresses translate, others fault at level 3	Used `0b01` for L3 leaves instead of `0b11`
Every access faults at level 0	`TTBR` wrong, or the table is not physically where you think
Access flag faults on every first touch	Normal without HA; set AF yourself in static tables
Mapping works, then fails after a change	Missing TLB invalidate, or missing break-before-make
TLB conflict abort	Overlapping entries from an in-place block split
Faults only under load or only on some cores	Missing `DSB` before the descriptor became visible
External abort during a walk	Table memory itself unmapped, or in a denied region
Huge page mapping silently ignored	VA or PA not aligned to the block size
Random accesses go to the wrong page	Contiguous hint set on a region that is not actually contiguous

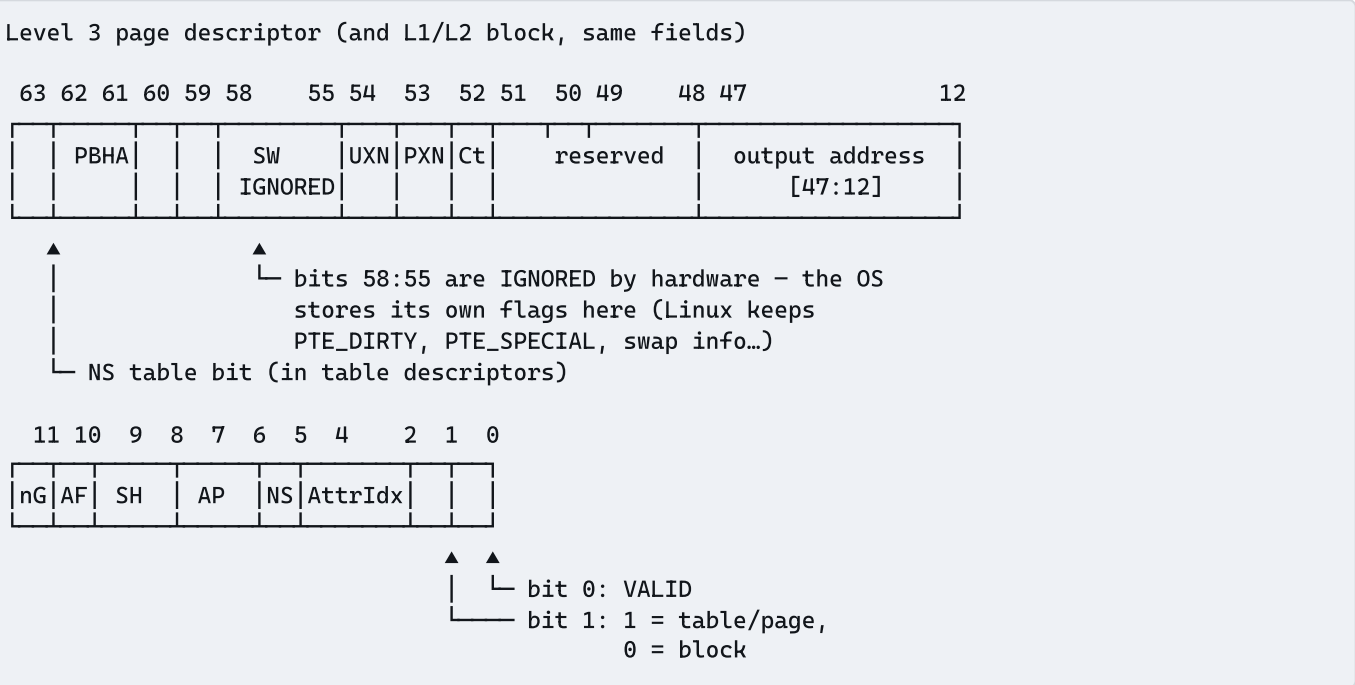
Check yourself

- ✓ Split a 48-bit VA into its four indices and offset, and say how much each level covers.
- ✓ Explain why a table is exactly one page in size.
- ✓ State what `0b01` and `0b11` mean at L2 and at L3, and the bug the difference causes.
- ✓ Walk a real address by hand from a `TTBR` value and four descriptors.
- ✓ Explain what a block descriptor buys and what it costs.
- ✓ State the break-before-make sequence and when it is required.
- ✓ Explain why building tables needs `DSB` instructions.
- ✓ Given a fault with DFSC `0b000101`, say exactly what happened.

Translation table descriptors, decoded

Sixty-four bits per mapped page, and almost every bit does something you can observe. This is the reference chapter — learn to read a descriptor and you can diagnose any permission, attribute or protection question from a raw hex value.

The leaf descriptor



Bits [1:0] — the type

Value	At L0–L2	At L3
`0b00`	Invalid	Invalid
`0b10`	Invalid	Invalid
`0b01`	**Block** — a 1 GB or 2 MB mapping	**Invalid**
`0b11`	**Table** — points to the next level	**Page** — a 4 KB mapping

The L3 inversion is repeated here because it is worth learning twice: it is the most common bare-metal MMU bug (chapter 13).

AP[2:1] — access permissions

```
   AP    EL1 (and above)    EL0        typical use
   ──────────────────────────────────────────────────────
   00    read/write          none       kernel data
   01    read/write          read/write shared with userspace -
                                        rare, and a red flag
   10    read only           none       kernel rodata, code
   11    read only           read only  shared constants, vDSO

  Note what is NOT expressible: "EL0 read/write, EL1 no access".
  Privilege is strictly nested — EL1 can always do at least what
  EL0 can. That is a property of the architecture, not a gap.

  So how does PAN stop the kernel touching user pages? Not through
  AP. PAN is a PSTATE bit that makes any EL1 access to an
  EL0-ACCESSIBLE page fault (chapter 05). It works precisely
  BECAUSE AP cannot express the restriction.
```

Execute-never

```
   UXN  bit 54    Unprivileged eXecute Never — EL0 may not execute
   PXN  bit 53    Privileged   eXecute Never — EL1+ may not execute

/* W^X, the fundamental exploit mitigation, is these two bits: */

   kernel code       AP=10 (RO), PXN=0, UXN=1
   kernel data       AP=00 (RW), PXN=1, UXN=1
   kernel rodata     AP=10 (RO), PXN=1, UXN=1
   user code         AP=11 (RO), PXN=1, UXN=0
   user data         AP=01 (RW), PXN=1, UXN=1

/* Note user data has PXN=1. Without it, an attacker who redirects
   kernel execution into a user-controlled buffer wins — the
   classic ret2usr attack. PXN on all user pages closes it, and
   is the ARM equivalent of x86's SMEP.                       */

/* SCTLR_EL1.WXN makes this automatic: any writable page becomes
   execute-never regardless of the descriptor. A cheap global
   hardening switch.                                          */
```

AttrIndx[2:0] — the memory type

```
/* Three bits selecting one of eight MAIR_EL1 slots
   (chapter 11).                                          */

        pte |= PTE_ATTR(IDX_NORMAL_WB);      /* bits [4:2] */

/* Which slot means what is entirely a software convention —
   whatever you put in MAIR. Linux's arm64 convention:

     0  Device-nGnRnE
     1  Device-nGnRE
     2  Normal Non-cacheable
     3  Normal Write-Back
     4  Normal Write-Through
     5  Normal Tagged (MTE)

   Reading a raw descriptor therefore requires reading MAIR too.
   A descriptor alone does not tell you the memory type.     */
```

SH[1:0] — shareability

SH	Domain	Use for
`00`	Non-shareable	Memory only this core touches; no coherency maintained
`10`	Outer Shareable	Shared with masters beyond the CPU cluster — some DMA
`11`	Inner Shareable	**The normal choice** for all SMP kernel and user memory

Shareability only has meaning for Normal cacheable memory — it is ignored for Device and for Non-cacheable. It determines how far coherency is maintained, and therefore how far a `DMB ISH` or a `TLBI ...IS` propagates (chapter 19). Marking SMP memory Non-shareable produces exactly the coherency bugs of CA chapter 15.

AF — the access flag

THE BIT EVERYONE FORGETS

If `AF` is 0, any access takes an **Access Flag fault** — even though the mapping is otherwise perfect. In hand-written boot tables this presents as "the MMU does not work at all", with a fault code that beginners misread as a translation fault.

Its purpose is page-usage tracking: the OS clears AF, and a later fault tells it the page was touched. That is how LRU approximation works (OS chapter 12). With `TCR.HA` the hardware sets it for you and the fault disappears. In static tables that never move, just set it and forget it.

nG — not global

```
   nG = 0    GLOBAL: the TLB entry matches under ANY ASID
   nG = 1    non-global: tagged with the current ASID

/* Kernel mappings (TTBR1) are Global — they are identical in
   every process, so a context switch need not disturb them.
   User mappings (TTBR0) are non-Global, so several processes'
   entries coexist in the TLB (chapter 15).

   Getting this backwards is a memorable bug: mark a user page
   Global and process B sees process A's memory. Mark a kernel
   page non-Global and it is flushed on every switch, costing
   performance but not correctness.                          */
```

NS — the security bit

```
   NS = 0    this mapping generates SECURE accesses
   NS = 1    this mapping generates NON-SECURE accesses

/* Only meaningful in the Secure translation regime. Secure
   software can create a mapping that issues Non-secure accesses
   — which is how a trusted OS reads a buffer shared with the
   normal world.

   Non-secure software's descriptors have no NS bit that matters:
   its accesses are always NS=1, enforced by hardware.

   In TABLE descriptors there is a separate NSTable bit (bit 63)
   which forces every entry below it to be Non-secure regardless
   of what those entries say — a way to delegate a whole subtree
   safely. Security track, chapter 08.                       */
```

The contiguous hint

```
   Ct  bit 52

/* Asserts: "this entry and its 15 aligned neighbours map a
   contiguous, aligned physical region with identical attributes."
   The TLB may then allocate ONE entry covering all 16 — a 64 KB
   mapping built from 4 KB pages, without needing a block.

   Benefit: 16x the TLB reach for the same table structure.

   Danger: if the claim is FALSE, the TLB uses one translation for
   16 different pages. Accesses silently go to the wrong memory.
   There is no fault. This is the worst kind of bug — silent,
   data-dependent corruption.

   Requirements: all 16 entries valid, physically contiguous,
   16-page aligned in both VA and PA, identical attributes and
   permissions. Changing any one of them requires clearing the
   hint on all 16 first, with break-before-make.            */
```

DBM and hardware dirty tracking

```
   DBM  bit 51    Dirty Bit Modifier

/* With TCR.HD enabled, a descriptor with DBM=1 and AP=read-only
   is upgraded to writable BY HARDWARE on the first write, and
   the hardware records the page as dirty by clearing AP[2].

   Without it, the OS must map the page read-only, take a
   permission fault on the first write, record "dirty" in
   software, and remap writable. One fault per page.

   Reading a descriptor: if DBM is set and AP[2] is CLEAR, the
   page has been written. If AP[2] is SET, it is clean.
   Linux's pte_dirty() checks exactly this.                  */
```

Software-defined bits

```
/* Bits 58:55 (and bits 62:59 in some configurations) are
   IGNORED by hardware. The OS owns them.                    */

/* Linux arm64 uses them for: */
#define PTE_DIRTY       (_AT(pteval_t, 1) << 55)
#define PTE_SPECIAL     (_AT(pteval_t, 1) << 56)
#define PTE_DEVMAP      (_AT(pteval_t, 1) << 57)
#define PTE_PROT_NONE   (_AT(pteval_t, 1) << 58)

/* And for SWAP ENTRIES: when a page is swapped out, the
   descriptor is invalid (bit 0 = 0), so the hardware ignores the
   whole thing — and the OS reuses the other 63 bits to store the
   swap device and offset. That is how the page table doubles as
   the swap map with no extra storage.                       */
```

Decoding a real descriptor

```
/* Given: 0x0060000098765743                                    */

   0x0060000098765743
   = 0000 0000 0110 0000 0000 ... 1001 1000 0111 0110 0101 0111 0100 0011

   bit  0     = 1        VALID
   bit  1     = 1        → at L3: PAGE   (at L1/L2: TABLE)
   bits [4:2] = 0b000    AttrIndx = 0 → MAIR slot 0
   bit  5     = 1        NS — non-secure
   bits [7:6] = 0b01     AP = 01 → RW at EL1 and EL0
   bits [9:8] = 0b11     SH = Inner Shareable
   bit  10    = 1        AF set — accessed
   bit  11    = 0        nG = 0 → GLOBAL
   bits[47:12]= 0x98765  output address → PA base 0x98765000
   bit  53    = 1        PXN — kernel may not execute here
   bit  54    = 1        UXN — user may not execute here

   Conclusion: a global, inner-shareable, read/write,
   non-executable page at PA 0x98765000, memory type = whatever
   MAIR slot 0 holds. Global + user-writable together is unusual
   and worth questioning.

/* A decoder worth keeping in your toolbox: */
static void decode_pte(uint64_t pte, int level)
{
        if (!(pte & 1)) { pr_info("INVALID (%016llx)\n", pte); return; }

        pr_info("PA=%09llx %s %s AP=%llu SH=%llu %s%s%s%s AttrIdx=%llu\n",
                pte & 0x0000FFFFFFFFF000ULL,
                (pte & 2) ? (level == 3 ? "page" : "table") : "block",
                (pte & (1UL << 10)) ? "AF"  : "!AF",
                (pte >> 6) & 3, (pte >> 8) & 3,
                (pte & (1UL << 11)) ? "nG "  : "G ",
                (pte & (1UL << 53)) ? "PXN " : "",
                (pte & (1UL << 54)) ? "UXN " : "",
                (pte & (1UL << 52)) ? "CONT " : "",
                (pte >> 2) & 7);
}
```

Stage-2 descriptors differ

```
Stage-2 (guest IPA → PA, chapter 16) uses a DIFFERENT layout:

   S2AP[7:6]     00 none, 01 read-only, 10 write-only, 11 read/write
                 — absolute permissions, NOT split by EL, because
                   stage 2 applies to the whole guest

   MemAttr[5:2]  the memory type DIRECTLY encoded, with no MAIR
                 indirection — because the guest owns MAIR and
                 the hypervisor cannot rely on it

   XN[54:53]     execute-never, two bits in newer revisions

  The final memory type is the COMBINATION of stage 1 and stage 2,
  and the rule is "the more restrictive wins": if either stage
  says Device, the access is Device; if either says read-only,
  it is read-only. A hypervisor can therefore force a guest's
  mapping to be non-cacheable regardless of what the guest asked
  for — which is how it emulates devices safely.
```

How Linux names these bits

```
/* arch/arm64/include/asm/pgtable-hwdef.h — worth reading once */

#define PTE_VALID       (_AT(pteval_t, 1) << 0)
#define PTE_TYPE_PAGE   (_AT(pteval_t, 3) << 0)
#define PTE_TABLE_BIT   (_AT(pteval_t, 1) << 1)
#define PTE_USER        (_AT(pteval_t, 1) << 6)   /* AP[1] */
#define PTE_RDONLY      (_AT(pteval_t, 1) << 7)   /* AP[2] */
#define PTE_SHARED      (_AT(pteval_t, 3) << 8)   /* SH[1:0] */
#define PTE_AF          (_AT(pteval_t, 1) << 10)
#define PTE_NG          (_AT(pteval_t, 1) << 11)
#define PTE_CONT        (_AT(pteval_t, 1) << 52)
#define PTE_PXN         (_AT(pteval_t, 1) << 53)
#define PTE_UXN         (_AT(pteval_t, 1) << 54)

/* and the composed protections */
#define PAGE_KERNEL       __pgprot(PROT_NORMAL)
#define PAGE_KERNEL_RO    __pgprot(PROT_NORMAL | PTE_RDONLY | PTE_PXN)
#define PAGE_KERNEL_EXEC  __pgprot(PROT_NORMAL & ~PTE_PXN)

/* Reading these definitions is the fastest way to connect the
   architecture to a real kernel's use of it.              */
```

How it shows up

Symptom	Cause
Every access faults despite a correct-looking mapping	`AF` not set
Faults only at level 3	Used `0b01` where L3 needs `0b11`
Kernel executes user-supplied code	`PXN` not set on user pages — ret2usr
Process sees another process's memory	User mapping marked Global instead of non-Global
Performance drops after a context switch	Kernel mapping marked non-Global; flushed every switch
Silent corruption, no fault, data-dependent	Contiguous hint set on a non-contiguous region

Symptom	Cause
Coherency bugs only on multi-core	Shareability set to Non-shareable
Cannot express "user-writable, kernel-inaccessible"	Correct — use PAN, not AP
Guest gets cacheable access to an emulated device	Stage-2 MemAttr too permissive; the stricter stage wins

Check yourself

- ✓ Give the four values of bits [1:0] and their meaning at L2 and at L3.
- ✓ Give the four AP encodings and explain why "EL0-only" cannot be expressed.
- ✓ Explain how PAN achieves what AP cannot.
- ✓ Give the descriptor settings for kernel code, kernel data, user code and user data.
- ✓ Explain why user pages must be PXN.
- ✓ Explain what `AF` is for and why forgetting it looks like a broken MMU.
- ✓ Explain the contiguous hint and why a false claim is worse than a fault.
- ✓ Decode a raw 64-bit descriptor into English.
- ✓ Say how stage-2 descriptors differ and how the two stages combine.

TLBs, ASIDs and invalidation

The TLB is a cache with no coherence protocol. Hardware will never notice that you changed a page table, and the invalidation sequence you must write yourself is exact, order-sensitive, and the source of some of the most confusing bugs in system software.

Why the TLB exists

```
A four-level walk is up to FOUR memory accesses per translation.
Every load and store needs one. Without caching, memory access
would be five times slower.

   L1 TLB      32–128 entries, fully associative, ~1 cycle
               usually SPLIT: separate iTLB and dTLB
   L2 TLB      1024–2048 entries, set-associative, ~10 cycles
               unified, shared between instruction and data
   miss        → hardware table walker → up to 4 memory accesses,
                 each of which may itself hit or miss in the caches

   WALK CACHES are a third structure worth knowing: they cache
   INTERMEDIATE table descriptors (L0/L1/L2), so a miss on a
   nearby address only needs the final level. This is why
   sequential access has far cheaper TLB misses than random.

TLB REACH = entries × page size
     1024 × 4 KB  =    4 MB     ← easily exceeded
     1024 × 2 MB  =    2 GB     ← why huge pages help so much

A TLB miss on a large random working set can cost more than the
cache miss that follows it.
```

What tags an entry

Tag	Distinguishes	Set by
VA	Which address	The access
ASID	Which address space (process)	`TTBR0_EL1[63:48]`
VMID	Which virtual machine	`VTTBR_EL2[63:48]`
Exception level / regime	EL0&1 vs EL2 vs EL3	The current EL
Security state	Secure vs Non-secure	`SCR_EL3.NS`
`nG`	Whether the ASID applies at all	The descriptor (ch. 14)

WHY ASIDS MATTER SO MUCH

Without them, every context switch would have to flush the entire TLB, and the incoming process would take hundreds of misses warming it back up. With them, entries from many processes coexist and a switch is a single `TTBR0` write.

Kernel mappings are marked **Global** (`nG=0`) so they match under any ASID — which is why a system call disturbs the TLB not at all.

```
/* ASID width is 8 or 16 bits — query it, do not assume: */
        ID_AA64MMFR0_EL1.ASIDBits    /* 0 = 8 bits, 2 = 16 bits */
        TCR_EL1.AS                    /* select 8 or 16 bit ASID */

/* A context switch: */
        ttbr0 = next_mm->pgd_pa | ((uint64_t)next_mm->asid << 48);
        asm volatile("msr ttbr0_el1, %0" :: "r"(ttbr0));
        asm volatile("isb");
/* That is the whole switch. No flush.                          */

/* ASID EXHAUSTION: with 16 bits there are 65536; with 8 bits only
   256. When they run out, Linux bumps a GENERATION counter and
   flushes everything, then reassigns. On a system with very many
   short-lived processes this shows up as a periodic latency
   blip — rare, but real, and worth recognising.              */
```

The TLB has no coherence protocol

THE RULE

Caches are kept coherent by hardware (CA ch. 15). **TLBs are not.** If you modify a translation table entry, every TLB in the system may continue using the old translation indefinitely. Nothing will tell you.

Worse, on a multi-core system you must invalidate on *every* core that might hold the entry — which is what the Inner Shareable variants of `TLBI` do, broadcasting the invalidation across the coherency domain.

The TLBI instruction

```
   TLBI <type><level><scope>{, Xt}

   TYPE — what to match
     ALL      everything in this regime
     VMALL    all entries for the current VMID (a whole VM)
     ASID     all entries matching the ASID in Xt
     VA       one VA, matching the ASID in Xt
     VAA      one VA, ALL ASIDs ("VA, All ASID")
     IPAS2    one IPA at stage 2
     RVA      a RANGE of VAs (ARMv8.4+)

   LEVEL — which regime
     E1       EL1&0 regime
     E2       EL2 regime
     E3       EL3 regime
     E1OS     ...outer shareable

   SCOPE
     (none)   this PE only
     IS       Inner Shareable — BROADCAST to all cores in the
              inner domain. This is what SMP needs.
     OS       Outer Shareable — wider still

   Examples:
     tlbi vae1is, x0      one VA, EL1&0, current ASID, broadcast
     tlbi vaae1is, x0     one VA, ALL ASIDs, broadcast
     tlbi aside1is, x0    all entries for one ASID, broadcast
     tlbi vmalle1is       everything in EL1&0, broadcast
     tlbi alle1           everything, this core only
     tlbi vmalls12e1is    both stages, whole VM, broadcast
```

```
/* The Xt operand for VA-based operations is NOT a plain address.
   It packs the ASID and the VA, shifted:                      */

        Xt[63:48] = ASID
        Xt[43:0]  = VA[55:12]        /* the page number */

/* So: */
        addr = (va >> 12) | ((uint64_t)asid << 48);
        asm volatile("tlbi vae1is, %0" :: "r"(addr));

/* Passing a raw byte address is a common error. It invalidates
   the wrong entry, silently, and the stale translation stays.  */
```

The invalidation sequence

> **WRITE → DSB → TLBI → DSB → ISB**
> Every part is load-bearing. Omit any one and you get a bug that appears on one core, under load, or only after an unrelated change.

```
        /* 1. update the descriptor */
        *pte = new_value;

        /* 2. DSB ISHST — ensure the WRITE is visible to the table
              walkers of every core BEFORE we invalidate. Without
              this, a core could refill its TLB from the OLD value
              after our TLBI but before our store lands.       */
        dsb     ishst

        /* 3. TLBI — invalidate, broadcast to the inner domain */
        tlbi    vae1is, x0

        /* 4. DSB ISH — wait for the invalidation to COMPLETE on
              every core. TLBI is issued asynchronously; without
              this, another core may still be using the old entry
              when we proceed.                                 */
        dsb     ish

        /* 5. ISB — flush THIS core's pipeline, so no instruction
              already in flight uses a stale translation.      */
        isb
```

```
/* Linux wraps this. Reading the wrappers is instructive: */

flush_tlb_all()                 /* everything, everywhere      */
flush_tlb_mm(mm)                /* one address space (by ASID) */
flush_tlb_range(vma, start, end) /* a range                    */
flush_tlb_page(vma, addr)       /* one page                    */
flush_tlb_kernel_range(s, e)    /* kernel mappings             */

/* And there is a threshold: below ~32 pages Linux issues
   individual TLBI VA operations; above it, one TLBI ASID, because
   many individual broadcasts cost more than one bulk flush.
   ARMv8.4's TLBI RVA (range) instructions remove that trade-off
   where available.                                            */
```

TLB shootdown

```
Invalidating on OTHER cores is the expensive part.

  ARM: TLBI ...IS BROADCASTS in hardware. The instruction issues,
       the interconnect distributes it, and DSB waits for
       completion. No interrupts, no software involvement.

  x86: NO broadcast instruction. The OS must send an
       inter-processor INTERRUPT to every core that might hold the
       entry, and wait for each to acknowledge. Far more expensive.

  This is a genuine architectural advantage for ARM, and it is why
  munmap-heavy and page-migration-heavy workloads scale better
  there.

  But it is not free. On a large system, a broadcast TLBI must
  reach every core in the domain, and the DSB blocks until it has.
  Workloads that constantly change mappings (JITs, garbage
  collectors, heavy mmap/munmap) can spend measurable time here.

  Mitigations:
    · batch invalidations (Linux's mmu_gather does this: it
      collects the pages freed during an unmap and issues ONE
      flush at the end)
    · use TLBI ASID rather than many TLBI VA
    · TLBI RVA range operations (ARMv8.4)
    · avoid unnecessary permission changes on hot mappings
```

When invalidation is required

Change	Invalidation needed?
Making an invalid entry valid	**No** — nothing could have cached a negative result… *but see below*
Making a valid entry invalid	**Yes**
Changing the output address	**Yes**, with break-before-make
Changing memory attributes	**Yes**, with break-before-make
Making permissions *more* restrictive	**Yes**
Making permissions *less* restrictive	Not architecturally required — the fault handler will re-walk. Linux does it anyway on some paths.
Setting AF or the dirty bit	No
Splitting a block into pages	**Yes**, with break-before-make

> **THE "NO INVALIDATION NEEDED" CASE HAS AN EXCEPTION**
>
> Making an invalid entry valid appears safe — how could a TLB cache "not present"? But some implementations *do* cache negative entries, and more importantly, the walk caches may hold intermediate table descriptors. If you allocate a new L3 table and link it in, a core with a cached L2 entry pointing at the *old* (invalid) state may not see it. Linux issues an invalidation on these paths for exactly this reason. When in doubt, invalidate — it is cheap relative to a stale-translation bug.

Break-before-make, restated

```
/* Changing a LIVE mapping's address, size or attributes in place
   permits the TLB to hold BOTH translations simultaneously.
   The architecture calls the result UNPREDICTABLE, and real
   hardware may raise a TLB conflict abort or use either
   translation non-deterministically.                          */

        /* BREAK */
        *pte = 0;
        dsb     ishst
        tlbi    vaae1is, x_addr
        dsb     ish
        isb

        /* MAKE */
        *pte = new_descriptor;
        dsb     ishst
        isb

/* This is why splitting a 2 MB block into 4 KB pages — which
   Linux does when a partial mprotect() hits a THP — is not a
   simple table edit. It must break the block, invalidate, build
   the L3 table, and only then install the new L2 pointer.   */
```

Stage-2 and VMID

```
/* A guest's TLB entries are tagged with the VMID from
   VTTBR_EL2[63:48], so several VMs coexist in the TLB just as
   several processes do.                                       */

/* Invalidating guest translations from EL2: */
        tlbi    vmalle1is        /* all stage-1 entries, current VMID */
        tlbi    vmalls12e1is     /* both stages, current VMID         */
        tlbi    ipas2e1is, x0    /* one IPA at stage 2                */

/* Note the ordering rule when changing a stage-2 mapping: you
   must invalidate stage 2 BEFORE stage 1, because a stage-1
   entry may have been formed using the old stage-2 translation:

        tlbi    ipas2e1is, x_ipa
        dsb     ish
        tlbi    vmalle1is
        dsb     ish
        isb

   Getting this backwards leaves combined TLB entries built from
   a mixture of old and new — a hypervisor bug class that is
   very hard to reproduce.                                     */
```

Measuring TLB behaviour

```
# is the TLB the bottleneck?
perf stat -e dTLB-load-misses,iTLB-load-misses,\
dtlb_walk,itlb_walk,l1d_tlb_refill,l2d_tlb_refill ./prog

#   dtlb_walk counts actual PAGE WALKS — the expensive event.
#   A high ratio of walks to loads means poor TLB reach.

# does the working set fit?
#   time a pointer chase over increasing sizes; the step changes
#   land at the L1 TLB reach, the L2 TLB reach, and the cache
#   capacities.

# try huge pages and measure the difference
echo always > /sys/kernel/mm/transparent_hugepage/enabled
perf stat -e dtlb_walk ./prog        # before and after

# per-process THP usage
grep -E 'AnonHugePages|ShmemPmdMapped' /proc/PID/smaps_rollup

# TLB shootdown cost, system-wide
perf stat -e tlb:tlb_flush -a sleep 10
cat /proc/interrupts | grep -i tlb    # x86 only; ARM broadcasts
```

How it shows up

Symptom	Cause
Page table change has no effect	Missing `TLBI` entirely
Works on one core, fails on another	Used a non-broadcast `TLBI` instead of the `IS` variant
Works, then fails intermittently under load	Missing `DSB` before or after the `TLBI`
Change visible to other cores but not this one	Missing final `ISB`
TLB conflict abort	Break-before-make violation
Invalidation appears to target the wrong page	Passed a byte address instead of `(va >> 12) | asid<<48`
One process sees another's memory	Mapping marked Global instead of non-Global
Periodic latency blip with many processes	ASID rollover forcing a full flush
Huge pages give a large speedup	TLB reach was the bottleneck
Guest memory corruption after a hypervisor remap	Stage-2 invalidated after stage 1 instead of before
munmap-heavy workload scales badly	Shootdown cost; batch with `mmu_gather`

Check yourself

- ✓ Explain what a TLB caches and why walk caches exist as a separate structure.
- ✓ Name the five things that tag a TLB entry.
- ✓ Explain why ASIDs exist and what Global mappings buy.
- ✓ State the full invalidation sequence and justify each of the five steps.
- ✓ Explain the format of the `TLBI` address operand.
- ✓ Explain what `IS` does and why ARM's approach beats x86's.

- ✓ Say when break-before-make is required and what happens without it.
- ✓ Give the correct ordering for invalidating stage 2 and stage 1, and why.

Two-stage translation

Run the whole translation machinery twice: once under the guest's control, once under the hypervisor's. This is what makes virtualisation efficient rather than emulated — and it is the exact same structure the SMMU uses for devices.

The problem it solves

```
A guest OS believes it owns physical memory starting at 0. So
does every other guest. Three approaches, historically:

  1. PARAVIRTUALISATION — modify the guest to know it is a guest
     and to use real physical addresses. Works, but requires
     changing every OS.

  2. SHADOW PAGE TABLES — the hypervisor maintains its own tables
     mapping guest-VA directly to host-PA, and traps every guest
     page table write to keep them in sync. Correct, and very
     expensive: every guest mmap, fork and page fault becomes a
     trap plus a shadow table update.

  3. TWO-STAGE TRANSLATION — hardware does both mappings.
     The guest edits its own tables freely, with NO traps.

  ARM (and Intel EPT, AMD NPT) all took the third path. It is the
  single feature that made hardware virtualisation practical.
```

The two stages

```
   guest virtual address (VA)
        │
        │  STAGE 1 — the GUEST's page tables
        │  TTBR0_EL1 / TTBR1_EL1, TCR_EL1, MAIR_EL1
        │  the guest writes these; the hypervisor does not touch them
        ▼
   intermediate physical address (IPA)
        │  "what the guest BELIEVES is physical"
        │
        │  STAGE 2 — the HYPERVISOR's tables
        │  VTTBR_EL2, VTCR_EL2
        │  the guest cannot read or write these
        ▼
   physical address (PA)

  Two independent sets of tables, two independent permission
  checks. The guest cannot escape stage 2 because it has no
  access to VTTBR_EL2 — that register is only reachable from EL2.

  HCR_EL2.VM = 1 enables stage 2. With it clear, EL1 accesses go
  straight to PA and the guest is not confined at all.
```

The cost: walks multiply

EVERY STAGE-1 TABLE ACCESS ITSELF NEEDS STAGE-2 TRANSLATION

The stage-1 walker reads descriptors from memory using *IPAs*. Each of those reads must be translated by stage 2. With four levels at each stage:

```
  stage-1 L0 descriptor read → needs a stage-2 walk (4 accesses)
  stage-1 L1 descriptor read → needs a stage-2 walk (4 accesses)
  stage-1 L2 descriptor read → needs a stage-2 walk (4 accesses)
  stage-1 L3 descriptor read → needs a stage-2 walk (4 accesses)
  the final IPA              → needs a stage-2 walk (4 accesses)
  plus the 4 stage-1 descriptor reads themselves       +4
                                                      ──────────

  worst case:  (4 + 1) × 4 + 4  =
24 MEMORY ACCESSES
```

Versus four for a native walk. This is why TLB behaviour matters so much more under virtualisation, why walk caches are critical, and why huge pages at *both* stages are a standard tuning step for VM workloads.

```
/* Mitigations, in order of impact:
     · combined TLB entries — the TLB caches VA → PA directly,
       tagged with both ASID and VMID, so a hit skips both stages
     · walk caches for intermediate descriptors at both stages
     · huge pages at stage 2 (a 1 GB block for the guest's RAM
       is extremely common — one entry covers everything)
     · huge pages inside the guest as well                    */
```

Stage-2 configuration

```
VTTBR_EL2    stage-2 table base, plus the VMID in [63:48]
VTCR_EL2     stage-2 translation control:

   T0SZ    [5:0]    IPA size:  IPA bits = 64 - T0SZ
   SL0     [7:6]    STARTING LEVEL — see below
   IRGN0   [9:8]    walk cacheability
   ORGN0   [11:10]
   SH0     [13:12]
   TG0     [15:14]  granule
   PS      [18:16]  physical address size
   VS      [19]     VMID size: 8 or 16 bits

SL0 is the field unique to stage 2. Because the IPA space is
usually much smaller than 48 bits (a guest with 4 GB of RAM
needs only 32), stage 2 can START at L1 or L2 rather than L0,
removing levels from every walk.

   IPA 40 bits, 4 KB granule → start at L1 (SL0 = 01)
   IPA 32 bits               → start at L2 (SL0 = 00)

CONCATENATED TABLES: stage 2 may also use up to 16 contiguous
tables at the starting level, effectively widening the first
index. A 40-bit IPA can then be covered starting at L1 with
two concatenated tables instead of needing an L0.

Both tricks exist for the same reason: shorten the walk, since
it is already being done up to five times.
```

Stage-2 descriptors

```
/* Stage 2 uses a DIFFERENT descriptor layout from stage 1
   (chapter 14). The important differences:                    */

   S2AP[7:6]      00  no access
                  01  read-only
                  10  write-only
                  11  read/write
      → ABSOLUTE permissions, not split by exception level.
        Stage 2 applies to the whole guest, so "EL0 vs EL1" is
        meaningless here — that distinction belongs to stage 1.

   MemAttr[5:2]   the memory type DIRECTLY encoded, with NO MAIR
                  indirection. The hypervisor cannot use MAIR_EL1,
                  because the guest owns it.

   XN[54:53]      execute-never, two bits in newer revisions

/* COMBINING the two stages — the rule is "more restrictive wins":

     permissions:  the intersection. Read-only at either stage
                   means read-only overall.
     memory type:  the more restrictive. Device at either stage
                   means Device overall. Non-cacheable at either
                   means non-cacheable.
     XN:           execute-never at either stage means no execute.

   So a hypervisor can force a guest's mapping to be non-cacheable
   regardless of what the guest asked for — which is exactly how
   it makes emulated device regions behave correctly even though
   the guest mapped them as ordinary memory.                   */
```

Stage-2 faults

```
A stage-2 fault is taken to EL2, NOT to the guest's EL1. The
guest never sees it — which is the whole point.

   ESR_EL2.EC = 0x24    data abort from a lower EL
   ESR_EL2.EC = 0x20    instruction abort from a lower EL

   HPFAR_EL2            the faulting IPA (bits [47:12] << 4)
   FAR_EL2              the faulting guest VA, when available

   ESR_EL2.ISS for a stage-2 abort:
      DFSC   [5:0]     fault status, with the LEVEL
      WnR    [6]       write (1) or read (0)
      S1PTW  [7]       the fault occurred while the stage-1 WALKER
                       was reading a descriptor — i.e. the guest's
                       PAGE TABLES are in an unmapped IPA
      ISV    [24]      the remaining fields are valid
      SAS    [23:22]   access size: 0=byte, 1=half, 2=word, 3=dword
      SRT    [20:16]   the guest register number involved
      SF     [15]      64-bit register

   ISV/SAS/SRT are the key to MMIO emulation: they tell the
   hypervisor exactly what the guest was trying to do, so it can
   emulate the access without decoding the instruction itself.
```

Emulating a device

```
/* The complete flow — this is how QEMU/KVM presents virtual
   hardware, and it is worth following once end to end.       */

/* GUEST, at EL1: */
        writel(1, uart_base + UART_CR);

/* 1. stage-1 translation: guest VA → IPA. Succeeds — the guest
      mapped its "UART" perfectly well.
   2. stage-2 lookup of that IPA in VTTBR_EL2: the hypervisor
      deliberately left it UNMAPPED.
   3. stage-2 data abort → exception to EL2.                  */

/* HYPERVISOR, at EL2: */
static int handle_mmio(struct kvm_vcpu *v)
{
        u64 esr  = read_sysreg(esr_el2);
        u64 ipa  = (read_sysreg(hpfar_el2) & ~0xFUL) << 8;

        if (!(esr & ESR_ISV))
                return decode_instruction_manually(v);  /* rare, slow */

        bool is_write = esr & ESR_WNR;
        int  size     = 1 << ((esr >> 22) & 3);
        int  reg      = (esr >> 16) & 0x1F;

        if (is_write)
                emulate_device_write(ipa, v->regs[reg], size);
        else
                v->regs[reg] = emulate_device_read(ipa, size);

        /* advance past the faulting instruction */
        write_sysreg(read_sysreg(elr_el2) + 4, elr_el2);
        return 0;               /* then ERET back into the guest */
}

/* The guest resumes with no idea anything happened. Note the
   cost: a full world switch per register access, which is why
   emulated I/O is slow and virtio exists (OS chapter 20).   */
```

VMID and TLB tagging

```
/* Stage-2 TLB entries are tagged with the VMID from
   VTTBR_EL2[63:48], exactly as stage-1 entries are tagged with
   the ASID. Several VMs coexist in the TLB.                  */

        vttbr = stage2_pgd_pa | ((uint64_t)vmid << 48);
        write_sysreg(vttbr, vttbr_el2);
        isb

/* VMID width is 8 or 16 bits (VTCR_EL2.VS). Like ASIDs, they
   are a limited resource with a rollover-and-flush scheme.

/* INVALIDATION ORDER MATTERS (chapter 15): when changing a
   stage-2 mapping, invalidate stage 2 FIRST, because combined
   TLB entries may have been built using the old stage-2
   translation:                                               */
        tlbi    ipas2e1is, x_ipa
        dsb     ish
        tlbi    vmalle1is
        dsb     ish
        isb

/* Reversing these leaves entries built from a mix of old and new
   - a hypervisor bug that is extremely hard to reproduce.   */
```

VHE: running a host kernel at EL2

```
The original design assumed a hypervisor was a small, separate
program at EL2 and the host OS ran at EL1. That is wrong for KVM,
where the "hypervisor" IS the Linux kernel - which would have to
run split across two exception levels, with constant switching.

VHE (Virtualization Host Extensions, ARMv8.1) fixes it:

  HCR_EL2.E2H = 1  makes the EL2 regime behave like EL1&0:
       · EL2 gains TTBR1_EL2 and a two-region address space
       · EL0 can run directly under EL2 (host userspace)
       · the _EL1 system register names, when accessed at EL2,
         are REDIRECTED to their _EL2 equivalents
       · _EL12 accessors let EL2 reach the real guest EL1 state

  Result: an unmodified Linux kernel runs at EL2, host userspace
  runs at EL0 under it, and guests run at EL1. No split, and VM
  entry/exit is far cheaper because the host does not have to
  save and restore its own EL1 state.

  Without VHE, KVM used a small "hyp stub" at EL2 and paid a
  world switch on every exit. With VHE the host never leaves EL2.
  Check: ID_AA64MMFR1_EL1.VH
```

The same structure, for devices

> **THIS IS EXACTLY WHAT THE SMMU DOES**
> An IOMMU applies the identical two-stage scheme to DMA: stage 1 owned by the guest (or the OS) translating device IOVAs to IPAs, stage 2 owned by the hypervisor translating IPAs to PAs. Same descriptor format, same combining rules, same VMID tagging.
>
> That is what makes **device passthrough** safe: a device assigned to a guest is programmed with guest-physical addresses, and stage 2 confines its DMA to that guest's memory exactly as it confines the guest's CPU accesses. Without it, passthrough would let a guest scribble anywhere in host memory. The security track covers this in full.

Debugging two-stage translation

```
/* The AT instruction understands both stages: */
        at      s1e1r,  x0       /* stage 1 only → gives the IPA    */
        at      s12e1r, x0       /* both stages  → gives the PA     */
        at      s1e2r,  x0       /* the EL2 regime's own mapping    */
        isb
        mrs     x1, par_el1

/* PAR_EL1.F = 1 on failure, and bit 8 says WHICH STAGE faulted.
   Running both forms tells you immediately whether the guest's
   tables or the hypervisor's are at fault — the single most
   useful diagnostic here.                                        */

# KVM statistics
perf stat -e kvm:kvm_exit,kvm:kvm_entry -a sleep 10
cat /sys/kernel/debug/kvm/*/vcpu0/*         # per-vCPU exit counts

# which exit reasons dominate?
perf record -e kvm:kvm_exit -a && perf report
#   a high count of MMIO exits means you should be using virtio
#   or passthrough instead of emulation

# stage-2 fault tracing
echo 1 > /sys/kernel/debug/tracing/events/kvm/kvm_mmio/enable
cat /sys/kernel/debug/tracing/trace_pipe
```

Where this goes next

Virtualisation chapter 5 takes stage 2 further: `SL0` and concatenated tables, the descriptor format, the permission and memory-type combining rules, and the invalidation sequence that catches everyone. Chapter 6 covers what happens when a stage-2 walk fails, which is how demand paging, MMIO emulation and live migration all work.

How it shows up

Symptom	Cause
Guest performance far below native on memory-heavy work	Walk multiplication; enable huge pages at both stages
Guest I/O is very slow	Full device emulation — one exit per register access

Symptom	Cause
Guest can access host memory	`HCR_EL2.VM` not set, or stage 2 maps too much
Guest crashes on an address the guest mapped correctly	Stage-2 fault; check `HPFAR_EL2`
Fault with `S1PTW` set	The guest's *page tables* are in an unmapped IPA
Guest memory corruption after a hypervisor remap	Invalidated stage 1 before stage 2
Emulation needs slow instruction decoding	`ESR.ISV` clear — some accesses cannot be decoded from the syndrome
Guest sees cacheable access to an emulated device	Stage-2 `MemAttr` too permissive
VM exits are unexpectedly expensive	No VHE — the host is saving and restoring EL1 state per exit
Passthrough device writes host memory	No IOMMU stage 2 — the exact reason it is mandatory

Check yourself

- ✓ Explain what an IPA is and who owns each stage.
- ✓ Compare shadow page tables with two-stage translation.
- ✓ Compute the worst-case number of memory accesses for a two-stage walk, and explain why.
- ✓ Give three mitigations for walk multiplication.
- ✓ Say how stage-2 descriptors differ from stage-1 and state the combining rule.
- ✓ Explain how `ESR.ISV` / `SAS` / `SRT` make MMIO emulation possible.
- ✓ Give the correct stage-2 then stage-1 invalidation order and why.
- ✓ Explain what VHE changes and why KVM needed it.
- ✓ Explain why device passthrough is unsafe without an IOMMU.

The cache hierarchy

CA chapter 14 covered cache theory. This chapter covers what ARM specifically architects, what it leaves to the implementation, and the terminology — points of coherency and unification — that every cache maintenance operation is defined in terms of.

What the architecture actually guarantees

VERY LITTLE, DELIBERATELY

Cache sizes, associativity, replacement policy, the number of levels, whether they are inclusive — all are **IMPLEMENTATION DEFINED**. The architecture specifies only:

- That caches are *transparent* for Normal memory — correct programs need no maintenance for CPU-to-CPU sharing within a shareability domain.
- The **points** (PoU, PoC, PoP) that maintenance operations target.
- The maintenance instructions themselves and their semantics.
- Discovery registers so software can find out what it is running on.

This is the architecture/microarchitecture split from chapter 01 in its purest form: portable correctness, implementation-specific performance.

Discovering the hierarchy

```
/* Never hardcode a line size. Read it. */

        mrs     x0, ctr_el0            /* Cache Type Register,
                                          readable from EL0        */
/*  CTR_EL0:
       [3:0]    IminLine  log2 of the smallest ICACHE line, in words
       [19:16]  DminLine  log2 of the smallest DCACHE line, in words
       [27:24]  CWG       cache writeback granule
       [23:20]  ERG       exclusives reservation granule
       [31]     DIC       instruction cache is coherent with data
       [29]     IDC       data cache clean not needed for I-fetch  */

static inline unsigned dcache_line_size(void)
{
        uint64_t ctr;
        asm volatile("mrs %0, ctr_el0" : "=r"(ctr));
        return 4U << ((ctr >> 16) & 0xF);
}

/* Enumerating the levels: */
        mrs     x0, clidr_el1          /* how many levels, and their
                                          types (I, D, unified)    */
        msr     csselr_el1, x1         /* select level and type    */
        isb
        mrs     x2, ccsidr_el1         /* size, associativity, sets */

/* CTR_EL0.DIC and .IDC are worth checking: on cores where they
   are set, some cache maintenance around self-modifying code
   becomes unnecessary. Linux uses them to skip work.        */
```

```
# from userspace
lscpu -C
#  NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL SETS
#  L1d       64K     512K    4 Data            1  256
#  L1i      128K       1M    4 Instruction     1  512
#  L2         4M      32M    8 Unified         2 8192
#  L3        16M      16M   16 Unified         3 16384

cat /sys/devices/system/cpu/cpu0/cache/index*/{level,type,size,\
ways_of_associativity,coherency_line_size,shared_cpu_list}

getconf LEVEL1_DCACHE_LINESIZE
```

The points: PoU, PoC, PoP

```
Every cache maintenance operation names a POINT — how far down
the hierarchy the operation must reach. These are the terms the
whole instruction set is defined in.

   PoU — Point of Unification
   ┌──────────────────────────────────────────────────┐
   │ The point where THIS CORE's instruction cache,   │
   │ data cache and table walker see the same copy.   │
   │ Typically the L2 or the unified cache level.     │
   │                                                  │
   │ Used for: making written INSTRUCTIONS visible to │
   │ instruction fetch — JIT, module loading,         │
   │ self-modifying boot code.                        │
   └──────────────────────────────────────────────────┘

   PoC — Point of Coherency
   ┌──────────────────────────────────────────────────┐
   │ The point where ALL observers that can access    │
   │ memory see the same copy — every core, and every │
   │ DMA master. Usually main memory, or a            │
   │ system-level cache that DMA also sees.           │
   │                                                  │
   │ Used for: DMA to and from a non-coherent device. │
   └──────────────────────────────────────────────────┘

   PoP — Point of Persistence   (ARMv8.2, DC CVAP)
   ┌──────────────────────────────────────────────────┐
   │ The point at which data survives power loss —    │
   │ relevant only for persistent memory.             │
   └──────────────────────────────────────────────────┘

   PoU is CLOSER than PoC. Cleaning to PoU is cheaper and is
   sufficient for instruction visibility on the same core;
   cleaning to PoC is required for anything a device will read.

   Choosing PoU where PoC was needed is a classic DMA bug: it
   works on a system where the L2 happens to be coherent with
   the device, and fails on one where it is not.
```

CPU-to-CPU coherency is automatic

```
/* Within a shareability domain, hardware keeps caches coherent
   (MESI/MOESI over ACE or CHI — CA chapter 15). You do NOT need
   cache maintenance between coherent cores.                  */

/* What you DO need is ORDERING — barriers (chapter 21). These are
   different things and confusing them is common:

     COHERENCY  guarantees all cores eventually see the same value
                for one location. Automatic.
     ORDERING   guarantees the RELATIVE order of accesses to
                DIFFERENT locations. NOT automatic — you must ask.

   A missing DMB is not fixed by a cache flush, and a stale DMA
   buffer is not fixed by a barrier. Diagnose which one you have.  */

/* The shareability attribute in the descriptor (chapter 14)
   determines the domain. Marking SMP memory Non-shareable takes
   it OUT of the coherency domain — and then hardware coherency
   genuinely does not apply, which produces exactly the bugs
   people wrongly attribute to missing barriers.             */
```

Where maintenance is genuinely required

Situation	Operation	Point
CPU wrote a buffer, a non-coherent device will read it	**Clean**	PoC
A non-coherent device wrote a buffer, the CPU will read it	**Invalidate**	PoC
Bidirectional DMA buffer	Clean *and* invalidate	PoC
Wrote instructions to memory, about to execute them	Clean D-cache, invalidate I-cache	PoU
Changed page tables	Clean, if the walker is not coherent	PoC or PoU
Powering down a core or a cluster	Clean and invalidate everything	PoC
Before enabling or disabling the MMU	Clean and invalidate	PoC
Writing to persistent memory	Clean	PoP

The instructions themselves are chapter 18. What matters here is recognising *which* of these situations you are in — that determines both the operation and the point.

Inclusion policy

```
  INCLUSIVE      everything in L1 is also in L2
                 + coherency is simple: snoop L2 and you know
                   what the cores hold
                 - wastes capacity: L2 duplicates all of L1

  EXCLUSIVE      a line lives at exactly one level
                 + maximum effective capacity (L1 + L2)
                 - more bookkeeping for coherency

  NON-INCLUSIVE  neither guaranteed
                 the common compromise on ARM designs

Software cannot generally observe which policy is in use, and
should not depend on it. It matters for two things:

1. Reasoning about effective capacity when tuning working-set
   sizes.
2. The "clean and invalidate all levels" sequence used at power
   down, which must walk every level by set and way precisely
   because inclusion cannot be assumed.
```

Set/way operations and why you should not use them

> **`DC CISW` IS ALMOST ALWAYS WRONG**
>
> The set/way operations (`DC ISW`, `DC CSW`, `DC CISW`) iterate over cache *geometry* rather than addresses. They are:
>
> - **Not broadcast.** They affect only the executing core's caches. On SMP they do not do what you want.
> - **Not architecturally reliable for coherency.** A line can migrate between levels or cores while you iterate, and be missed.
> - **Implementation-dependent** in which levels they reach.
>
> Their *only* legitimate use is powering down a core or cluster, where you must flush everything and no other agent is running. For every other purpose — DMA, self-modifying code, MMU changes — use the address-based (`VA`) operations, which *are* broadcast within the shareability domain.
>
> This mistake is common in bare-metal and bootloader code copied from older ARMv7 examples.

```
/* The legitimate set/way use: flush the whole cache before power
   down. Note this is genuinely intricate, which is another
   reason to avoid it unless you must.                          */

flush_all_levels:
        mrs     x0, clidr_el1
        and     w3, w0, #0x07000000     /* LoC: level of coherency */
        lsr     w3, w3, #23
        cbz     w3, finished
        mov     w10, #0                 /* level*2                 */
        mov     w8, #1
loop_level:
        add     w2, w10, w10, lsr #1
        lsr     w1, w0, w2
        and     w1, w1, #7              /* cache type at this level */
        cmp     w1, #2
        b.lt    next_level              /* no data cache here       */

        msr     csselr_el1, x10         /* select this level        */
        isb
        mrs     x1, ccsidr_el1
        /* ... extract line size, ways, sets; iterate DC CISW ... */
next_level:
        add     w10, w10, #2
        cmp     w3, w10
        b.gt    loop_level
finished:
        ret

/* Linux does this in __flush_dcache_all(), used only from CPU
   hotplug and suspend paths. Nowhere else.                     */
```

System-level caches

```
Beyond the CPU cluster, many SoCs have a SYSTEM LEVEL CACHE
shared by the CPU, GPU, NPU and display controller.

   ┌────┐ ┌────┐ ┌────┐  ┌─────┐ ┌─────┐
   │CPU0│ │CPU1│ │CPU2│  │ GPU │ │ NPU │
   └──┬─┘ └──┬─┘ └──┬─┘  └──┬──┘ └──┬──┘
      └──────┼──────┘       │       │
         cluster L2         │       │
             │              │       │
   ┌─────────┴──────────────┴───────┴──────┐
   │   System Level Cache / Last Level     │  ← the PoC may be HERE,
   └──────────────────┬────────────────────┘    not at DRAM
                      │
                   ┌──┴──┐
                   │DRAM │
                   └─────┘

  Consequences worth knowing:
   · the Point of Coherency may be the SLC rather than memory,
     which makes DMA to a device behind the SLC coherent without
     any maintenance
   · a device NOT behind the SLC still needs maintenance to PoC
   · whether a given master is coherent is a SoC integration
     property, which is exactly what `dma-coherent` in the device
     tree records
   · MPAM (ARMv8.4) can partition the SLC between agents, which
     is how mixed-criticality systems bound interference
     (OS chapter 19)
```

Performance characteristics

Level	Typical size	Latency	Shared by
L1 I / L1 D	32–128 KB each	3–5 cycles	One core
L2	256 KB – 2 MB	10–20 cycles	One core, or a cluster
L3 / cluster cache	4–32 MB	30–60 cycles	The cluster
SLC	4–64 MB	60–120 cycles	The whole SoC
DRAM	GBs	150–300 cycles	Everything

```
# measure your own machine's steps empirically
perf stat -e l1d_cache,l1d_cache_refill,\
l2d_cache,l2d_cache_refill,\
ll_cache_rd,ll_cache_miss_rd ./prog

# the ratio refill/access at each level is the miss rate there
# a pointer-chase benchmark over increasing working-set sizes
# reveals the capacity of each level as a step in the latency

# non-temporal access, to avoid polluting the hierarchy for a
# streaming copy (CA chapter 14)
        ldnp    q0, q1, [x1], #32
        stnp    q0, q1, [x0], #32
```

How it shows up

Symptom	Cause
Hardcoded 64-byte line size fails on some SoC	Line size is implementation defined — read `CTR_EL0`
DMA works on one board, corrupts on another	One is coherent, the other is not; maintenance missing
Cleaned to PoU, device still reads stale data	Needed PoC — the device is beyond the point of unification
JIT code executes garbage	Missing D-cache clean plus I-cache invalidate to PoU
Set/way flush does not fix an SMP problem	Set/way is not broadcast; use VA operations
Bootloader code from an ARMv7 example misbehaves	Set/way semantics differ; use address-based operations
Coherency bug only on multi-core	Memory marked Non-shareable, so it is outside the domain
Barrier added, stale DMA data persists	An ordering fix applied to a coherency problem
Large copy destroys unrelated performance	Cache pollution; use non-temporal stores

Check yourself

- ✓ Say what the architecture guarantees about caches and what it leaves implementation defined.
- ✓ Explain how to discover the cache line size correctly.
- ✓ Define PoU and PoC, and give a case that needs each.
- ✓ Explain why choosing PoU where PoC was needed is a portable-looking bug.
- ✓ State the difference between coherency and ordering, and give a symptom of each.
- ✓ Give three reasons set/way operations are the wrong tool for DMA.
- ✓ Explain what a system-level cache changes about the Point of Coherency.
- ✓ List the situations that genuinely require cache maintenance.

Cache maintenance operations

Three verbs — clean, invalidate, and both — applied to a point in the hierarchy. Get the verb or the point wrong and you get silent data corruption that appears on one board and not another. This chapter is the operational reference.

The three verbs

CLEAN, INVALIDATE, CLEAN-AND-INVALIDATE

- **Clean** — write dirty lines out to the target point. The line stays in the cache, now clean. Memory now matches the cache. *Use before a device reads your buffer.*
- **Invalidate** — discard the lines without writing them back. Any dirty data is **lost**. The next access re-fetches from memory. *Use before you read what a device wrote.*
- **Clean and invalidate** — write back, then discard. Safe in both directions, slightly more expensive. *Use when unsure, or for bidirectional buffers.*

The asymmetry matters: cleaning is always safe; **invalidating can destroy data** if the line was dirty and you did not mean to discard it.

The instructions

Instruction	Operation	Point	Use for
`DC CVAC, Xn`	Clean by VA	PoC	Before a non-coherent device reads
`DC IVAC, Xn`	Invalidate by VA	PoC	Before reading what a device wrote (EL1+ only)
`DC CIVAC, Xn`	Clean and invalidate	PoC	Bidirectional; the safe default
`DC CVAU, Xn`	Clean by VA	**PoU**	Before executing written instructions
`DC CVAP, Xn`	Clean by VA	PoP	Persistent memory (ARMv8.2)
`DC ZVA, Xn`	Zero a cache line	—	Fast `memset(0)`
`IC IVAU, Xn`	Invalidate I-cache by VA	PoU	After writing instructions
`IC IALLU`	Invalidate all I-cache, this core	PoU	Bulk code modification
`IC IALLUIS`	…broadcast to the inner domain	PoU	SMP code modification
`DC ISW/CSW/CISW`	By set and way	Level	**Power down only** — see chapter 17

```
Decoding the mnemonics — they are systematic:

   DC = Data Cache          IC = Instruction Cache

   C   Clean                I   Invalidate       CI  Clean+Invalidate
   VA  by Virtual Address   SW  by Set/Way       ALL everything
   C   to Point of Coherency
   U   to Point of Unification
   P   to Point of Persistence

   DC CVAC  = Data Cache, Clean by VA to PoC
   DC CIVAC = Data Cache, Clean and Invalidate by VA to PoC
   IC IVAU  = Instruction Cache, Invalidate by VA to PoU

Once you see the pattern the whole set is memorable.
```

VA operations broadcast; set/way do not

```
/* This is the property that makes VA operations the right choice:

   DC CVAC and friends are BROADCAST to every core in the
   shareability domain of the address. One instruction cleans the
   line wherever it lives.

   DC CISW affects only THIS core's caches. On SMP it silently
   does not do what you want.

   So: use VA-based operations for everything except the
   power-down flush.                                          */
```

Writing a maintenance loop

> **OPERATE ON CACHE LINES, NOT ON YOUR BUFFER'S BOUNDS**
>
> Maintenance operates on whole lines. If your buffer starts mid-line, cleaning "from the start of the buffer" still writes back the *entire* line — including the neighbouring data that shares it. Worse, an **invalidate** on a partial line discards the neighbour's dirty data.
>
> This is exactly why you must never DMA to or from a stack buffer, and why the DMA API requires cache-line-aligned allocations.

```
static void dcache_clean_range(void *start, size_t size)
{
        uint64_t ctr;
        asm volatile("mrs %0, ctr_el0" : "=r"(ctr));
        size_t line = 4UL << ((ctr >> 16) & 0xF);   /* never hardcode */

        uintptr_t addr = (uintptr_t)start & ~(line - 1);   /* round DOWN */
        uintptr_t end  = (uintptr_t)start + size;

        for (; addr < end; addr += line)
                asm volatile("dc cvac, %0" :: "r"(addr) : "memory");

        asm volatile("dsb ish" ::: "memory");   /* wait for completion */
}

/* Three things to notice:
     1. the line size comes from CTR_EL0
     2. the start is rounded DOWN to a line boundary
     3. a DSB is REQUIRED afterwards - maintenance operations are
        issued asynchronously, and without the barrier you may
        start the DMA before the writeback has finished     */
```

The DMA sequences

```
CPU → DEVICE  (the device will READ your buffer)

   1. write the data
   2. DC CVAC over the buffer          clean: push it to PoC
   3. DSB ISH                          wait for completion
   4. start the DMA (write a register)

   Without step 2, the device reads DRAM while your data sits
   dirty in L1. It gets garbage - usually the previous contents
   of that buffer, which makes the bug look like stale data
   rather than a coherency problem.


DEVICE → CPU  (the device WROTE your buffer)

   1. start the DMA, wait for completion
   2. DC IVAC over the buffer          invalidate: discard stale
   3. DSB ISH
   4. read the data

   Without step 2, the CPU may still hold a clean copy from
   before the transfer and return it. The symptom is a buffer
   that "never updates" - often mistaken for the device not
   working.

   SUBTLETY: you must ALSO invalidate BEFORE starting the
   transfer, not only after. A speculative read during the DMA
   could pull a stale line into the cache, which then survives
   your post-transfer invalidate window. Linux's dma_map_single
   handles both edges; hand-rolled code frequently does not.


BIDIRECTIONAL

   DC CIVAC before, DC IVAC after. Or just use the DMA API.
```

```
/* Which is why you should never hand-roll this in an OS driver: */

dma_addr_t d = dma_map_single(dev, buf, len, DMA_TO_DEVICE);
/* ...device owns the buffer... */
dma_unmap_single(dev, d, len, DMA_TO_DEVICE);

/* The API knows whether this device is coherent (from
   `dma-coherent` in the device tree), and emits the right
   maintenance — or correctly emits NONE. Hand-written
   maintenance on a coherent system is wasted work; omitted
   maintenance on a non-coherent one is corruption.        */

dma_sync_single_for_cpu(dev, d, len, DMA_FROM_DEVICE);
inspect(buf);
dma_sync_single_for_device(dev, d, len, DMA_FROM_DEVICE);
```

Making written instructions executable

```
The I-cache and D-cache are SEPARATE and NOT coherent with each
other. Writing bytes through the data path does not make them
visible to instruction fetch.

Required sequence, for JITs, module loaders, trampolines and
self-modifying boot code:

   1. write the instructions          (through the D-cache)
   2. DC CVAU  over the range         clean D-cache to PoU
   3. DSB ISH                         wait
   4. IC IVAU  over the range         invalidate I-cache to PoU
   5. DSB ISH                         wait
   6. ISB                             flush THIS core's pipeline

Every step matters:
   · without 2, the instructions are still dirty in the D-cache
     and the I-side cannot see them
   · without 4, the I-cache may hold the OLD bytes at that address
   · without 6, this core may already have PREFETCHED the old
     instructions into the pipeline

On SMP, other cores need the ISB too. Linux sends an IPI
(kick_all_cpus_sync) to force it, because there is no way to
broadcast an ISB.

Shortcut: CTR_EL0.DIC and .IDC tell you when steps can be
skipped on this implementation. Do not assume — read them.
```

```
static void flush_icache_range(void *start, size_t size)
{
        uintptr_t addr, end = (uintptr_t)start + size;
        size_t iline, dline;
        uint64_t ctr;

        asm volatile("mrs %0, ctr_el0" : "=r"(ctr));
        dline = 4UL << ((ctr >> 16) & 0xF);
        iline = 4UL << (ctr & 0xF);          /* I and D can DIFFER */

        if (!(ctr & (1UL << 28))) {          /* IDC clear: need clean */
                for (addr = (uintptr_t)start & ~(dline - 1);
                     addr < end; addr += dline)
                        asm volatile("dc cvau, %0" :: "r"(addr) : "memory");
                asm volatile("dsb ish" ::: "memory");
        }

        if (!(ctr & (1UL << 29))) {          /* DIC clear: need invalidate */
                for (addr = (uintptr_t)start & ~(iline - 1);
                     addr < end; addr += iline)
                        asm volatile("ic ivau, %0" :: "r"(addr) : "memory");
                asm volatile("dsb ish" ::: "memory");
        }

        asm volatile("isb" ::: "memory");
}

/* In userspace, do not write this - call the builtin, which
   emits whatever the platform requires:                     */
        __builtin___clear_cache(start, end);
```

DC ZVA: fast zeroing

```
/* Zeroes a whole cache line without READING it first - avoiding
   the read-for-ownership that a normal store sequence causes.
   This is what memset(0) uses for large blocks.             */

        mrs     x1, dczid_el0
        tbnz    x1, #4, no_dczva        /* DZP: prohibited?        */
        and     w1, w1, #0xF            /* log2 of the block size
                                           in WORDS               */
        mov     x2, #4
        lsl     x2, x2, x1              /* block size in bytes     */

1:      dc      zva, x0                 /* zero one block          */
        add     x0, x0, x2
        subs    x3, x3, x2
        b.gt    1b

/* The block size is NOT necessarily the cache line size - read
   DCZID_EL0. And DZP may be set, meaning the instruction is
   prohibited (it is trapped at EL0 on some configurations).

   Note: DC ZVA on Device memory is UNPREDICTABLE. It is a
   cache operation, and Device memory is not cached.       */
```

Maintenance needs barriers

> **CACHE OPERATIONS COMPLETE ASYNCHRONOUSLY**
> A `DC CVAC` instruction *issues* the operation; it does not wait for the writeback to reach memory. Without a following `DSB`, you can start a DMA transfer before your data has arrived.
>
> The general rule: **maintenance, then `DSB`, then rely on it.** And for instruction cache work, add an `ISB` because the pipeline must also be flushed.

```
/* Choosing the DSB domain:
     dsb ish     inner shareable — SMP within the cluster.
                 The usual choice.
     dsb osh     outer shareable — when a device outside the inner
                 domain is involved
     dsb sy      full system — the safest, and the slowest
     dsb nsh     non-shareable — this core only; used for local
                 TLB and cache work in boot code

   Use the narrowest domain that is correct. `dsb sy` in a hot
   path is a measurable cost.                                  */
```

Maintenance with the MMU off

```
/* Before the MMU is enabled, all memory is treated as
   Device-nGnRnE and the data cache is effectively bypassed. Two
   consequences for boot code:                                  */

/* 1. Data written with the MMU OFF goes straight to memory. Data
      written with the MMU ON may sit in a cache. So if a boot
      stage writes a structure with the MMU off, then enables the
      MMU and reads it, all is well — but the reverse can leave
      stale data. Clean before disabling the MMU.            */

/* 2. Before enabling the MMU, INVALIDATE the caches: they may
      contain lines from a previous boot stage with different
      attributes.                                             */

        /* early boot, before enabling the MMU */
        bl      invalidate_dcache_all       /* set/way — legitimate
                                               here: single core,
                                               nothing else running */
        ic      iallu
        dsb     nsh
        isb
        /* now build tables and enable the MMU (chapter 12) */

/* 3. And when handing off to another stage that will run with
      the MMU off — such as a kernel entered per the boot
      contract (OS chapter 18) — you must CLEAN the image you
      loaded to PoC, or it will not see your writes.        */
```

Decision procedure

```
Is the other observer a COHERENT CPU in the same domain?
     → NO maintenance needed. You may need a BARRIER
       (chapter 21) — a different problem.

Is it a NON-COHERENT DEVICE?
     device will READ  → DC CVAC  + DSB     (clean to PoC)
     device WROTE      → DC IVAC  + DSB     (invalidate to PoC)
                         and invalidate BEFORE the transfer too
     both              → DC CIVAC + DSB
     → or better: use the DMA API and let it decide

Am I about to EXECUTE bytes I just wrote?
     → DC CVAU, DSB, IC IVAU, DSB, ISB      (to PoU)
     → or: __builtin___clear_cache()

Am I powering down a core or cluster?
     → the set/way flush of everything

Am I changing page tables?
     → TLB maintenance (chapter 15), plus cache maintenance
       only if the table walker is not coherent

Anything else?
     → you probably do not need cache maintenance. Check
       whether you actually have an ORDERING problem.
```

How it shows up

Symptom	Cause
Device reads garbage or previous contents	Missing clean before the transfer
Buffer never appears to update after DMA	Missing invalidate after the transfer
DMA result correct only sometimes	Missing invalidate *before* the transfer; speculation pulled in a stale line
Neighbouring data corrupted by an invalidate	Partial cache line — the buffer was not line-aligned
JIT or module code executes garbage	Missing `DC CVAU` / `IC IVAU` / `ISB`
Self-modifying code works on one core, not another	Missing `ISB` on the other core
Maintenance loop misses the last line	Start not rounded down to a line boundary
Hardcoded 64-byte stride fails on some SoC	Line size differs — read `CTR_EL0`
Set/way flush does not fix an SMP issue	Not broadcast — use VA operations
Corruption starting DMA immediately after cleaning	Missing `DSB`; the writeback had not completed
Next boot stage runs stale code	Image not cleaned to PoC before handing over with the MMU off

Check yourself

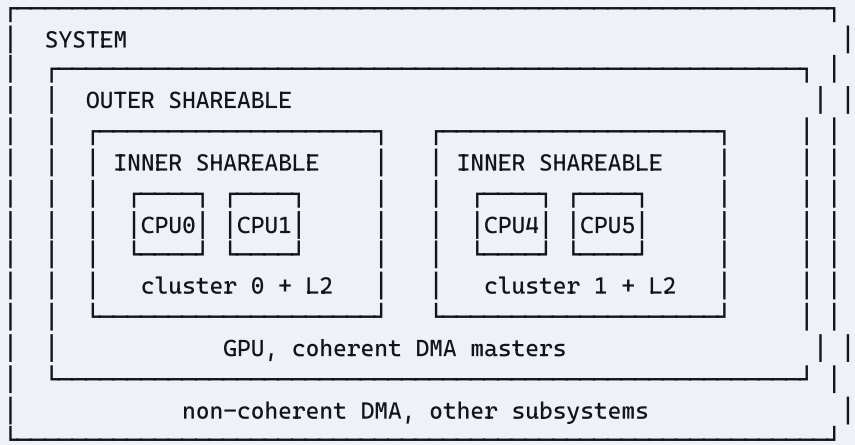
- ✓ Define clean, invalidate and clean-and-invalidate, and say which can lose data.
- ✓ Decode `DC CIVAC` and `IC IVAU` from the mnemonic.
- ✓ Say why VA operations are preferred over set/way on SMP.
- ✓ Write a maintenance loop correctly, including line size and alignment.

- ✓ Give both DMA sequences, and explain why an invalidate is also needed before a device-to-CPU transfer.
- ✓ Give the full six-step sequence for making written instructions executable.
- ✓ Explain why a `DSB` is required after maintenance.
- ✓ Explain what goes wrong when a DMA buffer is not cache-line aligned.

Coherency and shareability domains

Shareability decides which observers hardware keeps coherent, and how far a barrier or a maintenance operation propagates. It is set per page, costs nothing to get right, and produces impossible-looking multi-core bugs when got wrong.

The domains




```
The BOUNDARIES are IMPLEMENTATION DEFINED. On a typical SoC:
   Inner Shareable  = one CPU cluster, kept coherent by the
                      cluster interconnect
   Outer Shareable  = all clusters plus coherent DMA masters,
                      via the system interconnect (CHI/ACE)
   Non-shareable    = this core only; NO coherency maintained

Software cannot query the boundaries directly. It relies on the
SoC's documentation and, in Linux, on the device tree telling it
which masters are coherent.
```

WHAT SHAREABILITY ACTUALLY CONTROLS

Two things, and they are the same mechanism seen from two sides:

1. **Which observers hardware keeps coherent** for this page. Inside the domain, a write by one observer is automatically visible to the others — no cache maintenance needed.
2. **How far a barrier or maintenance operation propagates.** `DMB ISH` orders accesses within the inner domain; `TLBI ...IS` broadcasts to the inner domain. Going wider costs more.

Shareability applies only to **Normal cacheable** memory. It is ignored for Device memory and for Normal Non-cacheable, both of which are coherent by construction because nothing caches them.

Setting it

```
/* In a page table descriptor, SH[9:8] (chapter 14): */
   00   Non-shareable
   10   Outer Shareable
   11   Inner Shareable
   01   reserved

/* Linux: */
#define PTE_SHARED  (_AT(pteval_t, 3) << 8)     /* Inner Shareable */
/* — and PROT_NORMAL always includes it. Every normal Linux
   mapping is Inner Shareable, because SMP requires it.       */

/* In barriers and maintenance, the domain is a suffix: */
        dmb     ish          /* inner shareable  — the usual   */
        dmb     osh          /* outer shareable                */
        dmb     sy           /* full system — widest, slowest  */
        dmb     nsh          /* non-shareable — this core only */

        tlbi    vae1is       /* IS = broadcast to inner domain */
        tlbi    vae1os       /* OS = outer                     */
        tlbi    vae1         /* no suffix = this core only     */
```

The rule that matters

NON-SHAREABLE MEMORY IS NOT KEPT COHERENT — AT ALL

Marking SMP-shared memory Non-shareable takes it *out* of the coherency domain. Hardware then does not snoop it, and two cores can hold different values indefinitely. No barrier fixes this, because it is not an ordering problem: the cores genuinely disagree about the contents of memory.

The symptom is a variable that "does not update" on one core, or a lock that both cores appear to hold. It is easy to misdiagnose as a missing barrier and spend days adding `DMB`s that cannot help.

```
/* WRONG for anything SMP */
pte = pa | PTE_VALID | PTE_AF | PTE_ATTR(NORMAL_WB);   /* SH = 00 */

/* RIGHT */
pte = pa | PTE_VALID | PTE_AF | PTE_ATTR(NORMAL_WB) | PTE_SH_INNER;
```

Mismatched attributes

```
If two observers map the SAME physical memory with DIFFERENT
memory types or shareability, the architecture says the result is
UNPREDICTABLE. Real consequences:

  · one observer's writes may sit in a cache the other does not
    snoop
  · a Non-cacheable read may bypass a cache holding dirty data
  · loss of atomicity — exclusives may not work

COMMON WAYS THIS HAPPENS ACCIDENTALLY:

  1. The kernel's LINEAR MAP covers all of RAM as Normal
     Write-Back Inner-Shareable. If a driver then ioremaps some
     of that same RAM as Device or Non-cacheable, the SAME
     PHYSICAL PAGES now have two conflicting mappings.
     → this is why Linux refuses to ioremap normal RAM, and why
       reserved DMA regions are marked `no-map` in the device
       tree so the linear map excludes them.

  2. A guest and a hypervisor mapping the same page with
     different stage-1 and stage-2 attributes. The combining
     rule (chapter 16) makes the stricter one win, which is
     safe — but a guest expecting cacheable access to something
     the hypervisor forced Non-cacheable will be very slow.

  3. Firmware and OS disagreeing about a shared region across a
     handover — one running with the MMU off (everything is
     Device) and one with it on.

The fix is always the same: exactly ONE mapping per physical
page, or identical attributes on every mapping of it.
```

Coherent and non-coherent masters

```
/* Whether a DMA master participates in coherency is a property of
   how the SoC integrator wired it up:                          */

   ACE / CHI      fully coherent: the master can snoop CPU caches
                  AND be snooped
   ACE-Lite       I/O coherent: the master can snoop CPU caches,
                  but has no cache of its own to be snooped.
                  This is the common case for DMA engines.
   AXI            not coherent at all: software must do cache
                  maintenance

/* Software learns which from the device tree: */
        dma_engine@2c000000 {
                compatible = "vendor,dma";
                reg = <...>;
                dma-coherent;          /* ← this property */
        };

/* And the DMA API acts on it: */
        dev->dma_coherent = of_dma_is_coherent(node);
/* dma_map_single() then emits cache maintenance, or does not.

   The single most important consequence: the SAME driver source
   is correct on both kinds of SoC, PROVIDED it uses the DMA API.
   Hand-rolled cache maintenance is either wasted work or a
   correctness bug, depending on the platform.               */
```

Choosing the barrier domain

Domain	Orders accesses visible to	Use for
`nsh`	This PE only	Local TLB and cache maintenance in boot code
`ish`	The inner shareable domain	**Almost everything in SMP kernel code**
`osh`	The outer shareable domain	Synchronising with a coherent DMA master outside the cluster
`sy`	Everything, including Device memory and non-coherent masters	MMIO ordering, DMA handoff, the conservative choice

```
/* Cost increases with width. `dsb sy` may have to wait for
   transactions to reach the far side of the interconnect, which
   on a large SoC is hundreds of cycles.

   Use the narrowest correct domain. In practice:
     · CPU-to-CPU synchronisation        → ish
     · after TLBI ...IS                  → ish
     · before starting DMA on a
       non-coherent device               → sy
     · MMIO write that must land         → sy
     · single-core boot code             → nsh                */

/* Linux's naming maps directly:
        smp_mb()   → dmb ish
        mb()       → dsb sy         (used for MMIO / DMA)
        dma_wmb()  → dmb oshst
        __iowmb()  → dsb st, before an MMIO write            */
```

Exclusives and the shareability domain

```
/* LDXR/STXR rely on an EXCLUSIVE MONITOR. There are two:

     LOCAL monitor    per-PE, tracks one address
     GLOBAL monitor   in the interconnect, tracks addresses for
                      the coherency domain

   For Non-shareable memory, only the local monitor is used -
   which means exclusives between two cores DO NOT WORK. A
   spinlock in Non-shareable memory will let both cores in.

   For Inner Shareable memory, the global monitor participates
   and exclusives are correct across the domain.

   This is another reason the shareability attribute is not
   optional for shared data - and it is why a lock that
   "sometimes lets two threads in" is often a page-attribute
   bug rather than a logic bug.                              */

/* The reservation granule (CTR_EL0.ERG) is the size the monitor
   tracks - often 64 or 128 bytes. Two locks within one granule
   cause FALSE SHARING of the reservation: each STXR clears the
   other's, producing livelock-like retry storms. Pad locks to
   the granule, exactly as you pad against cache-line false
   sharing (CA chapter 15).                                  */
```

Diagnosing shareability problems

```
# what did the kernel actually map?
cat /sys/kernel/debug/kernel_page_tables | head -40
#   ...  RW NX SHD AF UXN MEM/NORMAL
#              ^^^ SHD = shareable. Its ABSENCE on SMP memory is
#                  the thing to look for.

# ask the hardware for one address (chapter 12)
        at   s1e1r, x0
        mrs  x1, par_el1
        /* PAR_EL1[9:8] = shareability actually applied
           PAR_EL1[63:56] = memory attributes            */

# is this device coherent?
grep -l dma-coherent /proc/device-tree/*/*/
cat /sys/bus/platform/devices/*/of_node/dma-coherent 2>/dev/null

# in the kernel
pr_info("coherent=%d\n", dev_is_dma_coherent(dev));
```

DISTINGUISHING THE THREE FAILURE MODES

When shared data misbehaves on multi-core, decide which of these you have before touching anything:

- **Ordering** — both cores eventually agree, but saw the writes in the wrong sequence. Fix with barriers or acquire/release (chapter 21).
- **Coherency** — the cores durably disagree about a value. Check shareability, and check for mismatched mappings.
- **Atomicity** — an update was lost. Check exclusives, and check that the memory is shareable so the global monitor applies.

Adding barriers to a coherency problem is the most common wasted debugging effort in multi-core work.

How it shows up

Symptom	Cause
A variable never updates on the other core	Memory marked Non-shareable
Adding barriers does not help	It is a coherency problem, not an ordering one
A spinlock lets two cores in	Exclusives on Non-shareable memory — no global monitor
Two locks in one structure cause retry storms	Both inside one exclusives reservation granule
Works single-core, fails on SMP	Shareability, or a genuinely missing barrier
Corruption after `ioremap` of normal RAM	Mismatched attributes with the linear map
Reserved DMA region behaves erratically	Not marked `no-map`; the linear map also covers it
Driver correct on one SoC, corrupts on another	One master is coherent and the other is not
`dsb sy` in a hot path is slow	Widest domain; use `ish` where it suffices

Check yourself

- ✓ Name the shareability domains and say what determines their boundaries.
- ✓ State the two things shareability controls.
- ✓ Explain why shareability is ignored for Device memory.
- ✓ Say what happens to SMP data marked Non-shareable, and why barriers do not fix it.
- ✓ Give two ways mismatched attributes arise accidentally, and the fix.
- ✓ Explain the difference between ACE, ACE-Lite and plain AXI for a DMA master.
- ✓ Choose the right barrier domain for four different situations.
- ✓ Explain why exclusives fail on Non-shareable memory.
- ✓ Distinguish ordering, coherency and atomicity failures by symptom.

The weak memory model

ARM permits your memory accesses to become visible to other cores in almost any order. This is not a bug or an implementation detail — it is the specification, and it is why lock-free code that has worked on x86 for a decade can fail the first time it runs on ARM.

What "weak" means

THE GUARANTEE

ARM guarantees that a single thread observes its own accesses in program order. It guarantees *nothing* about the order in which *other* observers see them, unless you use a barrier or an ordered access.

Accesses to the *same* location are always coherent and ordered — that is a separate guarantee (CA ch. 15). The weakness is entirely about accesses to *different* locations.

Reordering	x86-64 (TSO)	ARM (weak)
Load → Load	Not allowed	**Allowed**
Load → Store	Not allowed	**Allowed**
Store → Store	Not allowed	**Allowed**
Store → Load	**Allowed**	Allowed

```
Why ARM chose this:

  Enforcing order costs hardware. A store buffer that must drain
  in order, load queues that must not reorder, and snoop
  responses that must be sequenced all consume area, power and
  latency. Relaxing them lets the core:

     · retire stores into a buffer and continue
     · issue loads as soon as their address is known
     · let the interconnect route messages by any path
     · merge and reorder writes for bandwidth

  The cost is moved to SOFTWARE, which must ask for ordering
  where it genuinely needs it — which is a small fraction of the
  time. Given that, the trade is clearly correct for a
  power-sensitive architecture.

  x86 made the opposite choice for compatibility reasons, and
  pays for it in hardware complexity.
```

The canonical failure

```
/* Both variables start at 0. */

/* Core 0 — producer */          /* Core 1 — consumer */
data = 42;                       while (flag == 0)
flag = 1;                                ;
                                 use(data);        /* may read 0! */
```

```
Three independent things can produce the failure:

  1. COMPILER reordering — nothing in the source forbids swapping
     the two stores, since they are independent single-threaded.

  2. STORE BUFFER — the store to `data` retires into a buffer and
     has not yet reached the coherency point when `flag`'s store
     does. Other cores see flag=1 first.

  3. LOAD reordering on the consumer — the load of `data` can
     issue BEFORE the load of `flag`, because its address is known
     and there is no dependency between them.

  Point 3 is the one that surprises people most: even if the
  producer's stores are ordered, the CONSUMER can still break it.
  Both sides need ordering.
```

Dependencies that *do* order

```
/* Not everything requires an explicit barrier. Some orderings
   come free from the dependency structure.                    */

/* ADDRESS DEPENDENCY — guaranteed ordered */
        ptr = atomic_load(&head);
        val = ptr->field;          /* the second load CANNOT be
                                      reordered before the first —
                                      its address depends on it   */
/* This is what makes RCU readers free on ARM (OS chapter 09):
   rcu_dereference() relies on exactly this.                   */

/* DATA DEPENDENCY on a STORE — guaranteed ordered */
        x = load(&a);
        store(&b, x);              /* ordered: b's value needs a  */

/* CONTROL DEPENDENCY on a STORE — ordered, but fragile */
        if (load(&a))
                store(&b, 1);      /* the store cannot be
                                      speculated before the load */

/* CONTROL DEPENDENCY on a LOAD — NOT ordered */
        if (load(&a))
                y = load(&b);      /* the second load CAN be
                                      speculated. No ordering.   */
```

> **DO NOT RELY ON CONTROL DEPENDENCIES**
> They order stores but not loads, they are not expressible in the C or C++ memory model, and the compiler is entitled to remove them — by turning a branch into a conditional select (`CSEL`), or by hoisting a store that occurs on both paths. The Linux kernel documents this at length in `Documentation/memory-barriers.txt` and the advice is: use `smp_load_acquire()` or an explicit barrier instead. Address dependencies are safe; control dependencies are a trap.

Fixing it

```
/* Option 1: explicit barriers */
/* Core 0 */                        /* Core 1 */
data = 42;                          while (flag == 0)
DMB ISHST;                                  ;
flag = 1;                           DMB ISHLD;
                                    use(data);

/* Option 2: acquire/release — cheaper, and the modern idiom */
/* Core 0 */                        /* Core 1 */
data = 42;                          while (LDAR(flag) == 0)
STLR(flag, 1);                              ;
                                    use(data);

/*  STLR — store-release:  no EARLIER access may be reordered
                           after it
    LDAR — load-acquire:   no LATER access may be reordered
                           before it

   Together they form a RELEASE-ACQUIRE PAIR: everything core 0
   did before the release is visible to core 1 after the acquire.

   Cheaper than a standalone barrier because the ordering is
   attached to an access the core was performing anyway — the
   core can keep other work in flight.                          */
```

The C and C++ view

```
#include <stdatomic.h>

_Atomic int flag = 0;
int data = 0;                       /* a plain int is fine now */

/* producer */
data = 42;
atomic_store_explicit(&flag, 1, memory_order_release);

/* consumer */
while (atomic_load_explicit(&flag, memory_order_acquire) == 0)
        ;
use(data);                          /* guaranteed to see 42 */
```

`memory_order`	Guarantee	AArch64 instruction
`relaxed`	Atomicity only	Plain `LDR` / `STR`, or a plain atomic
`consume`	Address dependencies (deprecated in practice)	Plain load — ARM gives it for free
`acquire`	Later accesses cannot move before	`LDAR`
`release`	Earlier accesses cannot move after	`STLR`
`acq_rel`	Both, for RMW	`LDAXR` / `STLXR`, or `CASAL`
`seq_cst`	Plus one global total order	`LDAR` / `STLR` (plus `DMB` in some cases)

```
/* Linux kernel spellings of the same concepts: */
smp_store_release(&flag, 1);
val = smp_load_acquire(&flag);
smp_mb();   smp_rmb();   smp_wmb();      /* full / read / write */
READ_ONCE(x);   WRITE_ONCE(x, v);        /* stop the COMPILER only */
rcu_dereference(p);                       /* address dependency     */

/* READ_ONCE/WRITE_ONCE are worth understanding: they prevent the
   COMPILER from tearing, fusing or inventing accesses, but emit
   NO hardware barrier. They are the minimum for any shared
   variable, even one you think is "just a flag".            */
```

Litmus tests

```
The standard vocabulary for reasoning about memory models. Worth
knowing by name, because papers, kernel documentation and tooling
all use them.

MESSAGE PASSING (MP) — the flag/data case above
   P0: x = 1; y = 1        P1: r1 = y; r2 = x
   Can r1 == 1 && r2 == 0?     ARM: YES.  x86: no.

STORE BUFFERING (SB)
   P0: x = 1; r1 = y       P1: y = 1; r2 = x
   Can r1 == 0 && r2 == 0?     ARM: YES.  x86: YES.
   → the one reordering even TSO allows. Requires a full barrier
     on BOTH sides to prevent; acquire/release is not enough.

LOAD BUFFERING (LB)
   P0: r1 = x; y = 1       P1: r2 = y; x = 1
   Can r1 == 1 && r2 == 1?     ARM: YES.  x86: no.

INDEPENDENT READS OF INDEPENDENT WRITES (IRIW)
   P0: x = 1    P1: y = 1
   P2: r1 = x; r2 = y      P3: r3 = y; r4 = x
   Can P2 and P3 disagree about the ORDER of the two writes?
   ARM: NO — ARMv8 is MULTI-COPY ATOMIC. A write becomes visible
   to all observers at the same time. (ARMv7 was not, which made
   reasoning far harder.)

  Multi-copy atomicity is the single most important simplification
  ARMv8 made over ARMv7, and it is why the ARMv8 model is
  formally tractable.
```

```
# tools that check these mechanically — worth knowing they exist
#   herd7 / diy7        the ARM/Inria memory model simulator
#   litmus7             runs litmus tests on real hardware
#   linux/tools/memory-model/   the kernel's formal LKMM, with
#                       its own herd7 model and documentation

# run a litmus test on your own machine and watch it fail:
litmus7 MP.litmus
#   Histogram (4 states)
#   ...
#   1     :>0:X0=1; 0:X1=0;   ← the reordering, observed
```

Device memory is different

```
/* Device memory (chapter 11) has its OWN ordering rules, and they
   are much stronger:

     Device-nGnRnE   no gathering, no reordering, no early ack —
                     accesses arrive in program order
     Device-nGnRE    reordering still forbidden between Device
                     accesses to the same peripheral

   But note what is NOT guaranteed: ordering between a Device
   access and a NORMAL memory access. That combination needs an
   explicit barrier, and it is exactly the DMA handoff case:  */

        /* fill the DMA buffer — Normal memory */
        memcpy(buf, data, len);

        /* the buffer writes must be VISIBLE before the device
           is told to start                                    */
        dsb     sy                    /* or dma_wmb() in Linux */

        /* start the transfer — Device memory */
        writel(START, regs + CTRL);

/* Linux's writel() includes a barrier for exactly this reason;
   writel_relaxed() does not, and is for when you have already
   ordered things yourself or genuinely do not care.        */
```

Porting from x86

THE CODE WAS ALWAYS WRONG; X86 HID IT

x86's TSO forbids three of the four reorderings, which happens to be enough for most naive lock-free code to work by accident. On ARM those reorderings are permitted and do occur.

Worse, the failure is **rare and load-dependent**: it needs the store buffer to be occupied, or two cores to interleave in a particular way. It typically passes every test and appears in production, once, on a customer's machine.

Practical steps when porting:

- Search for every shared variable accessed without a lock. Each one is a candidate.
- Replace hand-rolled flags with `_Atomic` and explicit memory orders — the compiler then emits correct code for both architectures.
- Run under ThreadSanitizer, which models the C++ memory model rather than any one CPU.
- Do not "fix" it by adding `volatile`. It constrains the compiler and does nothing to the hardware (C ch. 10).

Practical rules

Situation	Use
Publishing data then a flag	`store-release` on the flag

Situation	Use
Reading a flag then the data	`load-acquire` on the flag
Following a published pointer	Address dependency — free on ARM; use `rcu_dereference`
A counter nobody synchronises on	`relaxed`
Store-buffering pattern (SB)	A full barrier on *both* sides; acquire/release is insufficient
Any shared variable at all	At minimum `READ_ONCE` / `WRITE_ONCE`
DMA handoff	`dsb sy` / `dma_wmb()` before the doorbell
Unsure	`seq_cst` — correct, and you can relax it later with measurement

How it shows up

Symptom	Cause
Works on x86, fails on ARM	Missing ordering that TSO happened to provide
Consumer sees the flag but stale data	Missing release/acquire pairing
Fails only under load, on many cores	The reordering window widened; the bug was always there
Adding a `printf` makes it disappear	Timing changed; the race remains
`volatile` did not fix it	`volatile` is not a barrier
Works with one compiler, fails with another	Relied on a control dependency the optimiser removed
Both cores see zero in an SB pattern	Store buffering — needs a full barrier both sides
DMA starts before the buffer is visible	Missing barrier between Normal and Device accesses
Lock-free queue corrupts occasionally	Insufficient memory order on the head/tail updates

Check yourself

- ✓ State exactly what ARM guarantees about a single thread and about other observers.
- ✓ Give the four reorderings and which x86 permits.
- ✓ Explain the three separate mechanisms that break the flag/data example.
- ✓ Say which dependencies order accesses for free, and which do not.
- ✓ Explain why control dependencies are unsafe to rely on.
- ✓ Give the release-acquire pairing and what it guarantees.
- ✓ Name three litmus tests and say what each demonstrates.
- ✓ Explain multi-copy atomicity and why it simplified ARMv8.
- ✓ Explain why an SB pattern needs more than acquire/release.

Barriers: DMB, DSB and ISB

Three instructions that constrain what the hardware may reorder. They look interchangeable and are not: each solves a different problem, and choosing the wrong one gives you code that is either incorrect or needlessly slow.

The three barriers

	`DMB`	`DSB`	`ISB`
Full name	Data Memory Barrier	Data Synchronization Barrier	Instruction Synchronization Barrier
Guarantees	Memory accesses before it are *observed* before those after it	All prior memory accesses **and maintenance operations** have *completed*	The pipeline is flushed; later instructions are re-fetched
Blocks execution?	No — non-memory instructions continue	**Yes** — execution stalls until everything completes	Yes
Cost	Low	High	Moderate
Use for	Ordering two data accesses	After `TLBI` / `DC` / `IC`; before DMA; before `WFI`	After a system register write; after modifying code

THE DISTINCTION IN ONE LINE EACH

DMB orders. **DSB** waits. **ISB** re-fetches.

DMB says "these accesses must become visible in this order" but does not stall the pipeline. DSB says "nothing proceeds until all of this has actually finished". ISB says "throw away everything already fetched and start again", which is what makes a configuration change take effect.

Domain and direction

```
   DMB / DSB  <domain><direction>

   DOMAIN — how far the ordering propagates (chapter 19)
      sy    full system — widest, includes Device memory and
            non-coherent masters
      ish   inner shareable — the SMP default
      osh   outer shareable
      nsh   non-shareable — this PE only

   DIRECTION — which accesses are constrained
      (none)  both loads and stores
      ld      loads only — orders prior LOADS against later
              loads and stores
      st      stores only — orders prior STORES against later
              stores

   Combined:
      dmb ishst    prior stores visible before later stores
                   → the PRODUCER side of a message-passing pattern
      dmb ishld    prior loads complete before later accesses
                   → the CONSUMER side
      dmb ish      full ordering within the inner domain
      dsb sy       everything, everywhere, completed
      dsb nshst    local stores completed — boot code

   Narrower is cheaper. `dmb ishst` is materially faster than
   `dmb sy`, and in a hot path the difference is measurable.
```

DMB in practice

```
/* The message-passing pattern (chapter 20), fixed with DMB: */

/* producer */                      /* consumer */
buffer[0] = 42;                     while (flag == 0)
dmb ishst;   /* stores ordered */           ;
flag = 1;                           dmb ishld;   /* loads ordered */
                                    use(buffer[0]);

/* Note both sides need one. Ordering the producer's stores does
   not stop the consumer's loads being reordered.            */

/* What DMB does NOT do:
     · it does not FLUSH anything — it constrains ORDER only
     · it does not wait for completion; a store may still be in
       a buffer, just guaranteed to leave before the next one
     · it does not affect instruction fetch
     · it does nothing for Device-memory-to-DMA handoff, where
       you need completion — use DSB                        */
```

Prefer acquire/release

```
/* Ordering attached to the ACCESS is cheaper than a standalone
   barrier, because the core can keep other work in flight. */

/* with barriers */                 /* with ordered accesses */
str  w1, [x0]                       str  w1, [x0]
dmb  ishst                          stlr w2, [x3]      ← one instruction
str  w2, [x3]

ldr  w1, [x0]                       ldar w1, [x0]      ← one instruction
dmb  ishld
ldr  w2, [x3]                       ldr  w2, [x3]

/* LDAR — load-acquire:  no LATER access may move before it
   STLR — store-release: no EARLIER access may move after it

   These are what C11's memory_order_acquire/release compile to,
   and what Linux's smp_load_acquire/smp_store_release emit.

   Rule of thumb: use acquire/release for synchronisation between
   threads; reserve explicit DMB for cases the ordered accesses
   cannot express, and DSB for completion.                  */
```

DSB: when you need completion, not order

```
/* DSB is required whenever a LATER action depends on a prior
   operation having actually FINISHED — not merely being ordered. */

/* 1. After cache or TLB maintenance (chapters 15, 18) */
        dc      cvac, x0
        dsb     ish             /* the writeback has COMPLETED */
        /* only now is it safe to start the DMA */

        tlbi    vae1is, x0
        dsb     ish             /* the invalidate has completed
                                   on every core */
        isb

/* 2. Before starting DMA */
        /* fill the buffer... */
        dsb     sy              /* the data has reached the point
                                   the device will read from */
        writel(START, regs);

/* 3. Before WFI / WFE */
        dsb     sy              /* outstanding transactions must
                                   complete before we sleep, or
                                   they may never do             */
        wfi

/* 4. Before writing SCTLR to enable the MMU (chapter 12) */
        dsb     ish             /* the page tables must be visible
                                   to the table walker */
        msr     sctlr_el1, x0
        isb

/* 5. Before a reset or power-down */
        dsb     sy
```

DSB IS EXPENSIVE

It stalls the core until every outstanding memory transaction has completed — which on a large SoC means waiting for the far side of the interconnect. Hundreds of cycles is normal. A `dsb sy` in a hot loop is a serious performance bug, and using `dsb` where `dmb` would do is the most common form of that mistake.

ISB: making changes take effect

```
/* ISB flushes the pipeline. Instructions after it are re-fetched
   and re-decoded under the CURRENT context.                  */

/* Required after: */

/* 1. Any system register write affecting fetch, translation or
      permissions (chapter 06) */
        msr     sctlr_el1, x0
        isb

        msr     ttbr0_el1, x0
        isb

        msr     vbar_el1, x0
        isb

/* 2. Modifying instructions (chapter 18) */
        dc      cvau, x0
        dsb     ish
        ic      ivau, x0
        dsb     ish
        isb                       /* THIS core may have prefetched
                                     the old bytes */

/* 3. After changing an exception level's configuration and before
      relying on it */

/* ISB does NOT order memory accesses. It is not a substitute for
   DMB, and DMB is not a substitute for it. They constrain
   different machinery: DMB the memory system, ISB the
   instruction pipeline.                                      */
```

The compiler barrier

```
/* Hardware barriers do nothing about COMPILER reordering. A
   compiler that hoists a load out of a loop has already broken
   your code before any instruction executes.               */

#define barrier()  asm volatile("" ::: "memory")

/* The "memory" clobber tells GCC: this statement may read or
   write any memory, so do not cache values in registers across
   it and do not reorder memory accesses past it. It emits NO
   instruction.                                              */

/* Linux's READ_ONCE and WRITE_ONCE are built on this idea: */
#define READ_ONCE(x)      (*(const volatile __typeof__(x) *)&(x))
#define WRITE_ONCE(x, v)  (*(volatile __typeof__(x) *)&(x) = (v))

/* They prevent the compiler from:
     · tearing a single access into several
     · fusing several accesses into one
     · inventing accesses that were not in the source
     · caching the value in a register across a loop

   They emit NO hardware barrier. Every shared variable needs at
   least this, even one you consider "just a flag".         */

/* Every real barrier macro includes the compiler barrier: */
#define smp_mb()   asm volatile("dmb ish" ::: "memory")
/*                                        ^^^^^^^^ essential */
```

The Linux barrier vocabulary

Macro	arm64 expansion	Use for
`barrier()`	compiler only	Stop reordering by the compiler
`smp_mb()`	`dmb ish`	CPU-to-CPU, both directions
`smp_rmb()`	`dmb ishld`	CPU-to-CPU, loads
`smp_wmb()`	`dmb ishst`	CPU-to-CPU, stores
`mb()`	`dsb sy`	Including MMIO and DMA
`dma_wmb()`	`dmb oshst`	Before a DMA descriptor is published
`dma_rmb()`	`dmb oshld`	After reading a DMA completion flag
`smp_store_release()`	`stlr`	Publish
`smp_load_acquire()`	`ldar`	Consume
`isb()`	`isb`	After a system register change

```
/* On a uniprocessor build the smp_* variants compile to just a
   compiler barrier — which is why you use smp_mb() for CPU-to-CPU
   and mb() for device interaction. Using mb() where smp_mb()
   would do costs a dsb sy on every call.                    */
```

MMIO ordering

```
/* Device memory accesses are ordered relative to EACH OTHER
   (chapter 11), but NOT relative to Normal memory accesses.
   The DMA handoff is exactly that combination:              */

        memcpy(dma_buf, data, len);     /* Normal memory */
        dsb     sy                       /* MUST complete first */
        writel(START, regs + CTRL);      /* Device memory */

/* Linux's accessors handle this for you: */
        writel(v, addr);          /* includes a barrier before the
                                     write so prior Normal writes
                                     are visible                  */
        readl(addr);              /* includes a barrier after      */

        writel_relaxed(v, addr);  /* NO barrier — use when you have
                                     already ordered things, e.g.
                                     several register writes in a
                                     row where only the last needs
                                     ordering                     */

/* The common optimisation: use _relaxed for a burst of register
   writes, then a single full writel() or an explicit barrier at
   the point where ordering actually matters. This can be a
   significant win in a hot driver path.                     */

/* And the POSTED WRITE trap: a write to Device-nGnRE may be
   acknowledged before it reaches the device. To be certain it
   has landed — before releasing a reset, say — you must READ
   BACK from the same peripheral:                            */
        writel(v, regs + CTRL);
        (void)readl(regs + CTRL);        /* forces completion */
```

Cost, and choosing

```
Approximate, and highly implementation dependent — but the
ORDERING is reliable:

   barrier()            0 cycles      (no instruction)
   ldar / stlr          few cycles    (attached to the access)
   dmb ishst/ishld      ~5–20         (one direction, narrow)
   dmb ish              ~10–30
   dmb sy               ~20–50
   dsb ish              ~20–100       (waits for completion)
   dsb sy               ~50–500+      (waits for the whole system)
   isb                  ~10–30        (pipeline flush)

DECISION PROCEDURE:

   Ordering two data accesses between threads?
        → acquire/release, or the narrowest DMB

   Need something to have COMPLETED before proceeding?
        → DSB, narrowest domain that is correct

   Changed a system register or wrote instructions?
        → ISB

   Only worried about the compiler?
        → barrier(), READ_ONCE, WRITE_ONCE

   Handing a buffer to a device?
        → DSB, and use the DMA API which does it for you
```

How it shows up

Symptom	Cause
Consumer sees the flag but stale data	Missing barrier on one or both sides
System register change has no effect	Missing `ISB`
TLB or cache maintenance seems not to happen	Missing `DSB` after it
DMA reads a partially filled buffer	No `DSB` before starting the transfer
Device does not see a register write	Posted write; read back to force completion
Hot loop unexpectedly slow	`dsb sy` where `dmb ishst` would do
Adding `volatile` did not fix a race	`volatile` is not a barrier
Works at `-O0`, fails at `-O2`	Compiler reordering; missing `"memory"` clobber
Core hangs in `WFI`	No `DSB` before it; a transaction never completed
Barrier added, coherency bug persists	It is a shareability problem, not ordering (ch. 19)

Check yourself

- ✓ Give the one-line distinction between DMB, DSB and ISB.
- ✓ Explain the domain and direction suffixes and give three useful combinations.
- ✓ Show a message-passing pattern with barriers, and say why both sides need one.

- ✓ Explain why acquire/release is cheaper than a standalone DMB.
- ✓ Give four situations that specifically require DSB.
- ✓ Give three situations that require ISB.
- ✓ Explain what the `"memory"` clobber does and why every barrier macro includes it.
- ✓ Explain posted writes and how to force a register write to land.

Atomics: exclusives and LSE

Because ARM is load/store, atomicity needs dedicated instructions. Two generations exist — the original retry-loop exclusives and the newer single-instruction atomics — and the difference between them is the difference between a system that scales to 64 cores and one that collapses at eight.

Why dedicated instructions

```
/* From chapter 03: read-modify-write is three instructions. */
        ldr     w1, [x0]
        add     w1, w1, #1
        str     w1, [x0]
/* Any other core can interleave. One increment is lost.      */

/* You cannot fix this with barriers — a barrier orders accesses,
   it does not make a sequence indivisible. You need hardware
   that can detect interference.                              */
```

Load-exclusive / store-exclusive

```
The mechanism: an EXCLUSIVE MONITOR watches an address.

   LDXR  marks the address as exclusively reserved by this PE
   STXR  succeeds ONLY if the reservation is still valid, and
         reports success or failure in a register

   The reservation is CLEARED by:
      · any store to the same reservation granule by any observer
      · a STXR from this PE (success or failure)
      · a CLREX instruction
      · an exception entry or return
      · a context switch
      · sometimes, spuriously — the architecture permits it

So the pattern is always a RETRY LOOP:
```

```
/* atomic increment */
1:      ldaxr   w1, [x0]        /* load-acquire exclusive          */
        add     w1, w1, #1
        stlxr   w2, w1, [x0]    /* store-release exclusive:
                                   w2 = 0 success, 1 failure       */
        cbnz    w2, 1b          /* retry on failure                */

/* compare and swap, built from the same primitive */
1:      ldaxr   w1, [x0]
        cmp     w1, w_expected
        b.ne    2f
        stlxr   w2, w_desired, [x0]
        cbnz    w2, 1b
2:      clrex                   /* release the reservation if we
                                   are abandoning the sequence     */

/* The ordered variants (LDAXR/STLXR) combine exclusivity with
   acquire/release semantics - one instruction instead of an
   exclusive plus a barrier (chapter 21).                    */
```

RULES FOR EXCLUSIVE LOOPS

- **Keep the body tiny.** Only register operations between LDXR and STXR. Any memory access, function call or exception can clear the monitor and cause an infinite retry.
- **Never take an exception inside one.** Exception entry clears the reservation, so a loop that is interrupted every iteration never completes — a livelock that looks like a hang.
- **Always `CLREX` when abandoning** a sequence, or a stale reservation can make an unrelated later STXR succeed when it should not.
- **They do not work on Device memory.** Undefined behaviour — the monitor requires cacheable memory (chapter 11).
- **They need shareable memory** for cross-core correctness — the global monitor only participates for Inner or Outer Shareable pages (chapter 19).

The reservation granule

```
/* The monitor tracks a GRANULE, not a single address. Its size is
   in CTR_EL0.ERG — typically 64 or 128 bytes.               */

static unsigned exclusives_granule(void)
{
        uint64_t ctr;
        asm volatile("mrs %0, ctr_el0" : "=r"(ctr));
        return 4U << ((ctr >> 20) & 0xF);      /* ERG field */
}

/* CONSEQUENCE: two independent locks inside one granule interfere.
   Each STXR clears the other's reservation, so both loops retry
   endlessly under contention — a livelock-like collapse that
   looks like terrible lock performance.                     */

struct bad {
        spinlock_t lock_a;      /* same granule */
        spinlock_t lock_b;      /* → they fight */
};

struct good {
        spinlock_t lock_a __attribute__((aligned(128)));
        spinlock_t lock_b __attribute__((aligned(128)));
};

/* This is the same class of problem as cache-line false sharing
   (CA chapter 15), one level down. Pad to the LARGER of the
   cache line size and the reservation granule.              */
```

Large System Extensions

```
/* ARMv8.1 added single-instruction atomics. No loop, no monitor,
   no retries.                                                */

        ldadd   w1, w2, [x0]     /* atomic add; w2 = OLD value    */
        ldclr   w1, w2, [x0]     /* atomic AND NOT (bit clear)    */
        ldeor   w1, w2, [x0]     /* atomic XOR                    */
        ldset   w1, w2, [x0]     /* atomic OR (bit set)           */
        ldsmax, ldsmin, ldumax, ldumin
        swp     w1, w2, [x0]     /* atomic exchange               */
        cas     w1, w2, [x0]     /* compare and swap              */
        casp    x0, x1, x2, x3, [x4]   /* 128-bit CAS             */

/* Each has ordering variants: */
        ldadda   /* acquire */    ldaddl   /* release */
        ldaddal  /* both    */    ldadd    /* relaxed */
        casal, swpal, ...

/* And a store-only form when the old value is not needed —
   cheaper, because no result must be returned:              */
        stadd   w1, [x0]         /* == ldadd with wzr as dest    */
```

WHY LSE SCALES AND EXCLUSIVES DO NOT

An exclusive loop requires the line to be brought into *this* core's cache in an exclusive state. Under contention, N cores fight over one line: it ping-pongs, most STXRs fail, and throughput *decreases* as cores are added.

An LSE atomic can be executed **near memory** — at the point of coherency, in the interconnect or a system cache. The line never moves. Contention becomes a queue at one place rather than a fight between N caches. Measured difference on a heavily contended counter at 64 cores can be an order of magnitude.

Detecting and using LSE

```
/* Query ID_AA64ISAR0_EL1.Atomic (bits [23:20]) >= 2 */

static bool has_lse(void)
{
        uint64_t isar0;
        asm volatile("mrs %0, id_aa64isar0_el1" : "=r"(isar0));
        return ((isar0 >> 20) & 0xF) >= 2;
}

/* Linux PATCHES ITSELF at boot: the same kernel image contains
   both an exclusive loop and an LSE instruction, and the
   alternatives framework replaces one with the other based on
   what the CPU reports. One binary, optimal everywhere.     */

/* Building for LSE explicitly: */
gcc -march=armv8.1-a           /* or +lse */
gcc -moutline-atomics          /* the DEFAULT since GCC 10:
                                  emits calls to runtime-dispatched
                                  helper functions, so one binary
                                  works on both, with a small
                                  call overhead                  */

# check what your distro built with
readelf -a /lib/aarch64-linux-gnu/libc.so.6 | grep -i lse
grep -o 'atomics' /proc/cpuinfo | head -1     # "atomics" = LSE present
```

From C

```
#include <stdatomic.h>

atomic_int counter = 0;

atomic_fetch_add(&counter, 1);                    /* → ldaddal */
atomic_fetch_add_explicit(&counter, 1,
                          memory_order_relaxed);   /* → ldadd   */

int expected = old;
atomic_compare_exchange_strong(&v, &expected, desired);   /* → casal */

/* _weak may fail spuriously and is cheaper inside a loop —
   it maps directly onto the exclusive retry pattern:      */
do {
        old = atomic_load_explicit(&v, memory_order_relaxed);
        new = f(old);
} while (!atomic_compare_exchange_weak_explicit(
                &v, &old, new,
                memory_order_release, memory_order_relaxed));

/* Choose the WEAKEST correct memory order. seq_cst is the default
   and the most expensive; relaxed is free for a statistics
   counter nobody synchronises on (chapter 20).            */

/* Is it actually lock-free? Check — a large atomic may be
   implemented with a hidden mutex, which is a correctness
   problem in a signal handler or an interrupt context:    */
atomic_is_lock_free(&obj);
ATOMIC_LLONG_LOCK_FREE          /* compile-time: 2 = always */
```

128-bit atomics

```
/* Needed for lock-free structures that must update a pointer and
   a tag together — the standard ABA defence (OS chapter 10).  */

        casp    x0, x1, x2, x3, [x4]      /* LSE, ARMv8.1        */
        ldxp    x0, x1, [x2]              /* exclusive pair       */
        stxp    w3, x4, x5, [x2]

/* In C: */
struct alignas(16) tagged_ptr {
        void    *ptr;
        uint64_t tag;
};
_Atomic struct tagged_ptr head;
/* atomic_compare_exchange on this compiles to CASP if LSE is
   present, LDXP/STXP otherwise. The 16-byte ALIGNMENT is
   mandatory — an unaligned 128-bit atomic is not atomic.   */

/* ARMv8.4 adds LSE2, which guarantees that ordinary aligned
   128-bit loads and stores are atomic — removing the need for
   the pair instructions in some cases.                     */
```

Building a spinlock

```
/* A minimal correct spinlock, showing acquire/release placement */

void spin_lock(uint32_t *lock)
{
        uint32_t tmp;
        asm volatile(
        "       sevl\n"                 /* prime the event register */
        "1:     wfe\n"                  /* sleep until an event     */
        "2:     ldaxr   %w0, [%1]\n"    /* load-ACQUIRE exclusive   */
        "       cbnz    %w0, 1b\n"      /* held? go back to sleep   */
        "       stxr    %w0, %w2, [%1]\n"
        "       cbnz    %w0, 2b\n"      /* lost the race, retry     */
        : "=&r"(tmp)
        : "r"(lock), "r"(1)
        : "memory", "cc");
}

void spin_unlock(uint32_t *lock)
{
        asm volatile("stlr wzr, [%0]"    /* store-RELEASE            */
                     :: "r"(lock) : "memory");
        /* STLR on the exclusive monitor's granule generates an
           EVENT, waking the WFE sleepers. No explicit SEV needed. */
}

/* Note WFE: spinners sleep in a low-power state instead of
   burning cycles. This is why ARM spinlocks are far more power-
   efficient than a naive busy loop (chapter 07).

   Real kernels use a QUEUED spinlock (qspinlock) instead: each
   waiter spins on its OWN cache line, so the coherence traffic
   does not grow with the number of waiters - the same insight
   as MCS locks (OS chapter 10).                              */
```

Measuring atomic performance

```
# uncontended vs contended is the whole story
perf stat -e instructions,cycles ./bench

# on ARM, the relevant PMU events
perf stat -e ldrex_spec,strex_spec,strex_fail_spec ./prog
#   a high strex_FAIL ratio means heavy contention and exclusive
#   retry storms - the signal to switch to LSE, or to reduce
#   sharing entirely

perf stat -e mem_access,bus_access,l2d_cache_wb ./prog
perf c2c record ./prog          # which lines are contended

# a quick empirical check: plot throughput against thread count.
#   flat or rising  -> fine
#   FALLING         -> the line is ping-ponging; look for false
#                      sharing, granule collisions, or exclusives
#                      under contention
```

THE ORDERING OF REMEDIES

When atomic operations are the bottleneck, in order of effectiveness: **(1) stop sharing** — per-CPU or per-thread counters aggregated later; **(2) reduce frequency** — batch updates; **(3) weaken the memory order** — a statistics counter needs only `relaxed`; **(4) use LSE**; **(5) use a better lock** — queued rather than test-and-set. Most real wins come from (1), not from micro-optimising the atomic itself.

How it shows up

Symptom	Cause
Counter loses increments	Plain read-modify-write instead of an atomic
Exclusive loop never completes	An exception or memory access inside the loop clearing the monitor
Two adjacent locks perform terribly together	Same reservation granule — pad them apart
Throughput falls as cores are added	Exclusive contention; try LSE and reduce sharing
Atomics do not work on a shared region	Memory is Device, or Non-shareable
Lock lets two threads in	Non-shareable memory — no global monitor
128-bit atomic is not atomic	Not 16-byte aligned
Same binary slow on a newer CPU	Built without LSE and without `-moutline-atomics`
Lock-free structure corrupts under load	ABA — needs a tagged pointer or safe reclamation
Atomic in a signal handler deadlocks	Not lock-free; check `atomic_is_lock_free`

Check yourself

- ✓ Explain why atomicity needs dedicated instructions on a load/store architecture.
- ✓ Describe the exclusive monitor and list four things that clear a reservation.
- ✓ Give three rules for writing an exclusive retry loop.
- ✓ Explain the reservation granule and the bug two adjacent locks can cause.
- ✓ Explain why LSE scales better than exclusives under contention.
- ✓ Say how Linux ships one binary that uses LSE only where available.
- ✓ Explain `compare_exchange_weak` versus `strong` and which maps to what.
- ✓ Explain the role of `WFE` in a spinlock and why unlock needs no `SEV`.

The GIC: architecture

A separate hardware block that collects interrupt signals from every peripheral, prioritises them, decides which core should handle each, and enforces the secure/non-secure split. Understanding its structure is what makes interrupt routing, affinity and virtualisation comprehensible.

Why a dedicated block

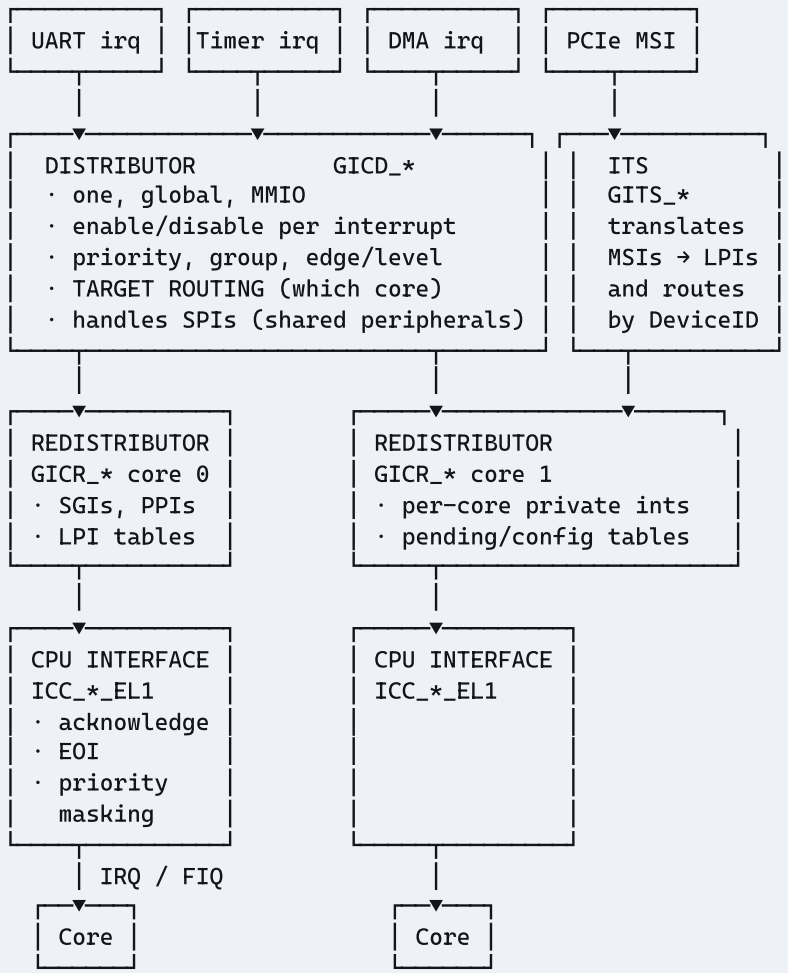
```
A CPU core has ONE interrupt input (two, counting FIQ). A SoC has
hundreds of interrupt sources and several cores. Something must:

   · multiplex hundreds of sources onto a few core inputs
   · PRIORITISE — a timer tick must not delay a critical fault
   · ROUTE — decide which core handles which interrupt
   · MASK selectively, per source and per priority
   · handle EDGE versus LEVEL signalling per source
   · enforce SECURITY — keep secure interrupts away from the
     normal world (security track)
   · support VIRTUALISATION — inject interrupts into a guest
     without a hypervisor trap on every one

That block is the Generic Interrupt Controller. It is architected
by Arm, so the programming model is the same across every SoC —
which is why one Linux driver serves them all.
```

Versions

Version	Brought	CPU interface
GICv2	Up to 8 cores, basic virtualisation	MMIO (`GICC_*`)
GICv3	Affinity routing, LPIs and the ITS, large core counts, four security groups	**System registers** (`ICC_*_EL1`)
GICv4	Direct injection of virtual LPIs — a guest receives an MSI with no hypervisor exit	System registers

The move from MMIO to system registers in GICv3 matters: acknowledging an interrupt becomes an `MRS` rather than a device read, which is dramatically faster and removes the barriers that MMIO access requires (chapter 11).

The components



Interrupt ID ranges

Range	Type	Scope	Used for
0–15	**SGI** — Software Generated	Per core	Core-to-core IPIs: TLB shootdown, rescheduling, function calls, stop-CPU
16–31	**PPI** — Private Peripheral	Per core	Per-core devices: the generic timer, the PMU, the virtual maintenance interrupt
32–1019	**SPI** — Shared Peripheral	Global	Ordinary devices: UART, DMA, Ethernet
1020–1023	Special	—	1023 = spurious, returned when there is nothing to handle
8192+	**LPI** — Locality-specific Peripheral	Global	Message-signalled interrupts (MSI/MSI-X). Effectively unlimited.

```
# see the mapping on a live system
cat /proc/interrupts
#            CPU0    CPU1    CPU2    CPU3
#   3:      102394   98211  101002   99887   GICv3  30 Level  arch_timer
#                                               ^^ PPI 30 — per core,
#                                                  hence counts on all
#  24:       15221       0       0       0   GICv3  56 Level  uart-pl011
#                                               ^^ SPI — routed to CPU0
#  IPI0:      8842    9021    8877    8990   Rescheduling interrupts
#  IPI1:       221     198     205     211   Function call interrupts
#                                           ^^^^ these are SGIs
```

Affinity routing

```
/* GICv3 identifies cores by their MPIDR affinity hierarchy rather
   than by a bitmask — which is what allows more than 8 cores. */

        mrs     x0, mpidr_el1
/*  MPIDR_EL1:
       Aff0 [7:0]    core within a cluster (or thread)
       Aff1 [15:8]   cluster
       Aff2 [23:16]  group of clusters
       Aff3 [39:32]  higher level still                       */

/* An SPI's target is written to GICD_IROUTER<n>: */
        GICD_IROUTER(intid) = (aff3 << 32) | (aff2 << 16) |
                              (aff1 << 8)  |  aff0;
/* or with bit 31 set (Interrupt_Routing_Mode = 1) it means
   "any participating core" — 1-of-N delivery, which lets the
   GIC pick an idle core.                                     */

/* Sending an SGI is a SYSTEM REGISTER write, not MMIO: */
        msr     ICC_SGI1R_EL1, x0
/*  x0 encodes: the target affinity, a 16-bit target list within
    the cluster, and the SGI number. That is how Linux sends a
    reschedule IPI or a TLB shootdown request.               */
```

```
# control affinity from userspace
cat /proc/irq/24/smp_affinity_list
echo 2 > /proc/irq/24/smp_affinity_list        # pin to CPU2
echo 3 > /proc/irq/24/smp_affinity             # bitmask form

# irqbalance moves them automatically — and is usually wrong for
# latency-sensitive work. Disable and pin by hand:
systemctl stop irqbalance

# keep interrupts entirely off the real-time cores
# kernel cmdline: irqaffinity=0,1 isolcpus=2,3
```

Security groups

```
Every interrupt belongs to a GROUP, and the group determines
which exception level receives it and as which signal:

   Group 0          → FIQ, always taken to EL3
                      Secure firmware's interrupts

   Secure Group 1   → IRQ when in the secure world,
                      FIQ when in the non-secure world
                      Trusted OS interrupts

   Non-secure Group 1 → IRQ
                        Ordinary OS interrupts

  THE CONSEQUENCE that matters: a Non-secure OS clearing PSTATE.I
  cannot mask secure interrupts, because they arrive as FIQ and
  are routed to EL3. The secure world can always preempt the
  normal world — which is what makes a TEE's timing guarantees
  meaningful (security track).

  Configured in GICD_IGROUPR / GICD_IGRPMODR, and only writable
  from the secure world.
```

Priority and preemption

```
/* 8-bit priority values, and LOWER NUMBER = HIGHER PRIORITY.
   (This trips people up constantly.)                          */

        GICD_IPRIORITYR(intid) = 0x80;     /* mid priority */

/* Not all 8 bits are necessarily implemented - write 0xFF and
   read back to discover how many are: */
static unsigned priority_bits(void)
{
        writeb(0xFF, GICD_IPRIORITYR(0));
        return 8 - __builtin_ctz(readb(GICD_IPRIORITYR(0)));
        /* commonly 4 or 5 bits → 16 or 32 usable levels */
}

/* THREE registers control what gets through: */

   ICC_PMR_EL1      Priority Mask: interrupts with priority
                    NUMERICALLY >= this value are blocked.
                    0xFF = allow everything, 0x00 = block all.

   ICC_BPR1_EL1     Binary Point: splits the priority field into
                    a GROUP part and a SUB-priority part. Only the
                    group part participates in PREEMPTION, so this
                    controls how much nesting you allow.

   ICC_RPR_EL1      Running Priority: the priority of the
                    interrupt currently being handled. A new
                    interrupt preempts only if its group priority
                    is higher.

/* Linux uses PMR for "pseudo-NMI": rather than masking with
   PSTATE.I (which blocks everything), it raises PMR so that
   ordinary interrupts are blocked but a high-priority NMI still
   gets through. That is how arm64 gets NMIs without hardware
   NMI support.                                                 */
```

The ITS and MSIs

```
A PCIe device signals an interrupt by WRITING to an address —
there is no wire. The GIC must turn that write into an interrupt
for a specific core.

   device writes (DeviceID, EventID) to GITS_TRANSLATER
        |
   ┌────▼──────────────────────────────────────────┐
   |  ITS — Interrupt Translation Service          |
   |                                               |
   |  Device Table:  DeviceID → an ITT             |
   |  ITT:           EventID  → (LPI ID, Collection) |
   |  Collection Table: Collection → Redistributor|
   └────┬──────────────────────────────────────────┘
        ▼
   the LPI is made pending in the target redistributor's
   PENDING TABLE (in memory), and the core is signalled

  Two levels of indirection, both table-driven, because there can
  be tens of thousands of LPIs and they must be redirected when
  a VM migrates between cores.

  Software configures it with COMMANDS in a queue, exactly like
  the SMMU (security track):
     MAPD    map a DeviceID to an ITT
     MAPTI   map an EventID to an LPI and collection
     MOVI    move an LPI to a different collection (core)
     INV     invalidate cached configuration
     SYNC    wait for completion

  GICv4 adds direct injection: a VM's LPI is delivered to the
  guest WITHOUT a hypervisor exit at all.
```

Virtualisation support

```
/* Without hardware help, every guest interrupt would need a
   hypervisor trap: take the interrupt at EL2, work out that it
   belongs to a guest, and inject it. The GIC provides a LIST
   REGISTER mechanism to avoid that.                          */

   ICH_LR<n>_EL2      List Registers — the hypervisor writes a
                      virtual interrupt here, and the GIC delivers
                      it to the guest DIRECTLY. The guest
                      acknowledges and EOIs it with its own
                      ICC_* registers, with NO exit.

   ICH_HCR_EL2        hypervisor control: enable virtual
                      interfaces, request maintenance interrupts

   ICH_VMCR_EL2       the guest's virtual control state

   ICV_*_EL1          the guest's view — when HCR_EL2.IMO is set,
                      the guest's ICC_* accesses are REDIRECTED
                      to these virtual registers

/* Cost: the hypervisor must still populate the list registers on
   entry and read back state on exit, and there are only a few
   (typically 4–16). A guest with many pending interrupts causes
   MAINTENANCE INTERRUPTS to refill them.

   GICv4's direct injection removes even that for LPIs.      */
```

Initialising the GIC

```
/* Distributor — once, globally */
        GICD_CTLR = 0;                          /* disable while
                                                   configuring */
        for each SPI:
                GICD_ICENABLER = all;           /* disable      */
                GICD_ICPENDR   = all;           /* clear pending */
                GICD_IPRIORITYR = 0xA0;         /* default prio  */
                GICD_ICFGR     = level or edge;
                GICD_IGROUPR   = 1;             /* NS Group 1    */
                GICD_IROUTER   = target affinity;
        GICD_CTLR = ARE_NS | EnableGrp1NS;      /* affinity routing
                                                   + enable       */

/* Redistributor — per core */
        GICR_WAKER &= ~ProcessorSleep;          /* wake it        */
        wait until (GICR_WAKER & ChildrenAsleep) == 0;
        configure SGIs and PPIs in GICR_SGI_base

/* CPU interface — per core, SYSTEM REGISTERS */
        msr     ICC_SRE_EL1, #1        /* enable system register
                                          access — MUST be first  */
        isb
        msr     ICC_PMR_EL1, #0xFF     /* unmask all priorities   */
        msr     ICC_BPR1_EL1, #0
        msr     ICC_IGRPEN1_EL1, #1    /* enable Group 1          */
        isb

/* The GICR_WAKER dance is a common bring-up trap: a
   redistributor comes out of reset ASLEEP and delivers nothing
   until woken, with no error to indicate why.               */
```

Device tree description

```
gic: interrupt-controller@2f000000 {
        compatible = "arm,gic-v3";
        #interrupt-cells = <3>;
        interrupt-controller;
        reg = <0x0 0x2f000000 0x0 0x10000>,     /* GICD  */
              <0x0 0x2f100000 0x0 0x200000>;    /* GICR  */
        interrupts = <GIC_PPI 9 IRQ_TYPE_LEVEL_HIGH>;  /* maintenance */

        its: msi-controller@2f020000 {
                compatible = "arm,gic-v3-its";
                msi-controller;
                reg = <0x0 0x2f020000 0x0 0x20000>;
        };
};

/* And a device referencing it: */
        interrupts = <GIC_SPI 56 IRQ_TYPE_LEVEL_HIGH>;
/*                    ^type    ^number  ^trigger
   Note the SPI number here is RELATIVE to 32: SPI 56 is INTID 88.
   Mixing these up is a very common device tree error, and the
   symptom is an interrupt that never fires.              */
```

How it shows up

Symptom	Cause
Interrupt count stays at zero	Wrong INTID (SPI numbering offset), not enabled, or wrong trigger type
Nothing works on secondary cores	`GICR_WAKER` not cleared for those redistributors
All interrupts land on CPU0	No affinity spreading; `GICD_IROUTER` or `smp_affinity`
Latency spikes on real-time cores	Interrupts not steered away from them
Interrupt fires once then never again	Not EOI'd, or an edge interrupt lost while masked
Spurious INTID 1023 returned	Acknowledged when nothing was pending — a race or a stale read
Secure OS cannot preempt the normal world	Interrupt in the wrong group
MSI never arrives	ITS tables not mapped, or `MAPD` / `MAPTI` not issued
Guest misses interrupts under load	List registers exhausted; maintenance interrupts thrashing
Higher-priority interrupt does not preempt	Binary point grouping them at the same preemption level

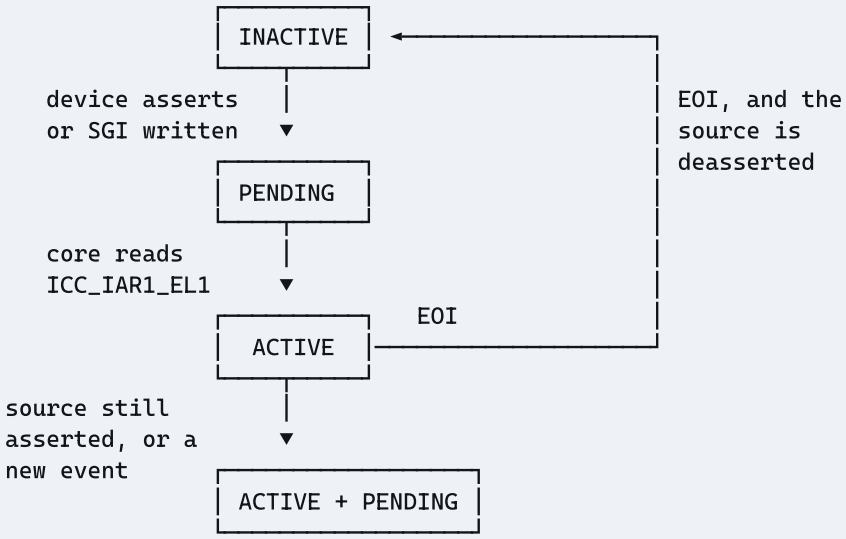
Check yourself

- ✓ Give seven jobs the GIC performs that a bare interrupt line cannot.
- ✓ Name the GIC components and what each is responsible for.
- ✓ Give the interrupt ID ranges and an example of each type.
- ✓ Explain affinity routing and why GICv2's bitmask did not scale.
- ✓ Explain the security groups and why a Non-secure OS cannot mask secure interrupts.
- ✓ State the priority convention and explain what PMR, BPR and RPR each do.
- ✓ Explain how Linux implements pseudo-NMIs using PMR.
- ✓ Describe the ITS's two levels of translation and why both are needed.
- ✓ Explain how list registers let a guest handle an interrupt without a hypervisor exit.

The GIC: interrupt lifecycle

An interrupt moves through a defined state machine from assertion to completion. Every interrupt bug — storms, lost edges, spurious IDs, priority failures — is a specific wrong transition in that machine.

The state machine



ACTIVE means "a core is handling it". While active, the GIC will NOT signal the same interrupt again — which is what prevents an interrupt from re-entering its own handler.

ACTIVE+PENDING is the interesting state: the interrupt fired again while being handled. After EOI it goes straight back to PENDING and fires immediately.

The complete flow

```
/* 1. The device asserts its interrupt line, or writes an MSI
      doorbell. */

/* 2. The distributor (or redistributor, for PPIs/SGIs) marks the
      INTID PENDING, checks it is enabled, and compares its
      priority against what the target core is running. */

/* 3. If it wins, the GIC asserts IRQ or FIQ to that core. */

/* 4. The core takes the exception — if PSTATE.I permits, and if
      the priority passes ICC_PMR_EL1 (chapter 28 for the entry
      sequence). */

/* 5. ACKNOWLEDGE: the handler reads the acknowledge register.
      This returns the INTID and moves it PENDING → ACTIVE. */
        mrs     x0, ICC_IAR1_EL1
        /* x0 = INTID, or 1023 (spurious) if nothing was pending */

/* 6. SERVICE the device — and crucially, CLEAR THE INTERRUPT
      CONDITION AT THE DEVICE, not just at the GIC. */
        writel(status, regs + IRQ_STATUS);      /* W1C at source */

/* 7. END OF INTERRUPT: write the INTID back. This drops the
      running priority, allowing lower-priority interrupts. */
        msr     ICC_EOIR1_EL1, x0

/* 8. ERET back to the interrupted code. */
```

> **STEP 6 BEFORE STEP 7, ALWAYS**
>
> If you EOI without clearing the condition at the device, a level-triggered line is still asserted. The GIC immediately re-signals it, the core re-enters the handler, and nothing else ever runs. The system appears hung with 100% CPU in the interrupt handler.
>
> Linux's spurious-interrupt detector eventually notices and prints `"irq N: nobody cared (try booting with the "irqpoll" option)"`, then disables the line. That message is a diagnosis, not a fix.

EOI modes

```
/* ICC_CTLR_EL1.EOImode selects one of two behaviours: */

/* EOImode = 0 — the simple case, and the default */
        msr     ICC_EOIR1_EL1, x0
/*      does BOTH: drops the running priority AND deactivates
        the interrupt. One write, done.                      */

/* EOImode = 1 — priority drop and deactivation are SEPARATE */
        msr     ICC_EOIR1_EL1, x0     /* priority drop only    */
        ...
        msr     ICC_DIR_EL1, x0       /* deactivate            */

/* Why the split exists: it lets you re-enable higher-priority
   interrupts (by dropping priority) while the current interrupt
   remains ACTIVE, so it cannot re-fire. This is essential for:

     · THREADED interrupt handlers — Linux drops priority when
       the hard handler returns, but does not deactivate until
       the threaded handler completes, preventing a storm from
       a level-triggered device whose condition is cleared late

     · VIRTUALISATION — the hypervisor drops priority, injects
       the interrupt into a guest, and deactivates only when the
       guest EOIs it                                          */

/* Linux uses EOImode=1 when running as a host hypervisor,
   EOImode=0 otherwise. You can see which in dmesg:          */
//   GICv3: Using split EOI/Deactivate mode
```

Edge and level

	Level-triggered	**Edge-triggered**
Pending while	The source is asserted	Latched on the transition
Cleared by	Deasserting at the *device*	Acknowledge, or writing `GICD_ICPENDR`
Sharing	**Possible** — each handler checks and clears its own	Unsafe — two simultaneous edges register as one
Can be missed?	No — the line stays asserted	**Yes**, if masked at the wrong moment
Failure mode	Interrupt storm	Silent hang, waiting for an event that already happened
Configured in	`GICD_ICFGR<n>`, two bits per interrupt	

```
/* The edge-triggered race, and the standard defence: */

static irqreturn_t isr(int irq, void *dev)
{
        u32 status;

        do {
                status = readl(regs + STATUS);
                if (!status)
                        break;
                writel(status, regs + STATUS);   /* clear */
                handle(status);
        } while (readl(regs + STATUS));          /* RE-CHECK */

        return IRQ_HANDLED;
}

/* Without the loop, an event arriving between handle() and the
   return produces no new edge — the device has already raised
   its line and lowered it. The interrupt is lost forever and the
   device waits for a response that never comes.

   The symptom: the device works fine at low rates and hangs
   under load. Extremely common, and easy to misattribute.  */
```

Shared interrupt lines

```
/* Several devices on one level-triggered SPI. Every handler is
   called; each must determine whether it was theirs.       */

static irqreturn_t my_isr(int irq, void *dev_id)
{
        struct my_dev *d = dev_id;
        u32 status = readl(d->regs + STATUS);

        if (!(status & MY_IRQ_PENDING))
                return IRQ_NONE;        /* NOT OURS — essential */

        writel(status, d->regs + STATUS);
        /* ... handle ... */
        return IRQ_HANDLED;
}

/* Returning IRQ_HANDLED when it was not yours breaks the kernel's
   spurious-interrupt detection: it can no longer tell that
   nobody serviced the line, so a genuinely stuck interrupt is
   never caught and the machine simply hangs.

   Registering with IRQF_SHARED requires a valid dev_id, because
   that is how free_irq identifies which handler to remove.  */
        request_irq(irq, my_isr, IRQF_SHARED, name, d);
```

Masking, at four levels

```
There are four independent places an interrupt can be blocked.
Diagnosing "my interrupt never fires" means checking all four,
from the source outward:

  1. AT THE DEVICE       the peripheral's own interrupt enable
                         → readl(regs + IRQ_ENABLE)

  2. AT THE DISTRIBUTOR  GICD_ISENABLER / GICD_ICENABLER
                         → per-INTID enable
                         → Linux: /proc/interrupts shows it only
                           if a handler is registered

  3. AT THE CPU INTERFACE  ICC_PMR_EL1 priority mask, and
                           ICC_IGRPEN1_EL1 group enable

  4. AT THE CORE         PSTATE.I (IRQ) or PSTATE.F (FIQ)
                         → local_irq_disable() in Linux

  A fifth, easily forgotten: the interrupt may be routed to a
  DIFFERENT CORE than the one you are watching. Check
  /proc/interrupts across all CPUs, not just CPU0.
```

Software-generated interrupts

```
/* SGIs are how cores talk to each other. Sending one is a system
   register write.                                              */

        msr     ICC_SGI1R_EL1, x0

/*  x0 encodes:
       [55:48] Aff3      [39:32] Aff2      [23:16] Aff1
       [27:24] SGI ID (0-15)
       [15:0]  TargetList - a bitmask of cores WITHIN that Aff1
                            cluster
       [40]    IRM - 1 means "all cores except self"          */

/* Linux's IPI usage - visible in /proc/interrupts: */
   IPI0  Rescheduling         wake a core to run the scheduler
   IPI1  Function call        run a function on another core
   IPI2  CPU stop             halt for panic or reboot
   IPI3  CPU crash stop
   IPI4  Timer broadcast      wake a core in deep idle for a timer
   IPI5  IRQ work             deferred work from NMI context
   IPI6  CPU wakeup

/* Note what is NOT an IPI on ARM: TLB shootdown. TLBI ...IS
   broadcasts in hardware (chapter 15), so no interrupt is needed
   - a genuine architectural advantage over x86, which must send
   an IPI and wait for acknowledgement.                      */
```

Interrupt latency, decomposed

```
 device asserts
      │  ① propagation to the GIC, arbitration              ~10s ns
      │  ② the core has interrupts DISABLED somewhere    ◄── DOMINANT
      │     (a spinlock, a critical section)                and VARIABLE
      │  ③ exception entry: ELR/SPSR/ESR, vector fetch       ~10s cycles
      │  ④ the handler stub saves 30 registers               ~30 cycles
      │  ⑤ read ICC_IAR1_EL1, dispatch to the handler        ~10s cycles
      ▼
 your ISR's first instruction
      │  ⑥ ISR duration
      ▼
 bottom half / thread wakeup
      │  ⑦ PREEMPTION disabled somewhere                 ◄── the other
      │  ⑧ scheduler decision + context switch               big term
      ▼
 your task runs

② and ⑦ are the terms that ruin guarantees, because they depend
on the LONGEST such region anywhere in the kernel — including
in a driver you did not write.

PREEMPT_RT attacks exactly these: most spinlocks become sleeping
mutexes and handlers become threads, bounding both (OS ch. 19).

MEASURING it:
   cyclictest -t4 -p 90 -i 200 -m -D 3600     ← report the MAX
   ftrace irqsoff tracer → finds the longest disabled region
                           AND its stack trace
```

The Linux view

```
/* Linux abstracts the GIC behind IRQ DOMAINS, which map a
   hardware interrupt number to a Linux virtual IRQ number.  */

        hwirq (GIC INTID 88)  ──irq_domain──►  virq (Linux irq 24)

/* Why: an SoC may have several cascaded controllers (a GPIO
   controller behind an SPI, for instance). Each gets a domain,
   and drivers use virq numbers without caring which controller
   is underneath.                                              */

        irq = platform_get_irq(pdev, 0);      /* resolves the DT
                                                 specifier → virq */
        devm_request_threaded_irq(dev, irq, hard, thread,
                                  IRQF_ONESHOT, name, data);

/* The chip operations the GIC driver implements: */
struct irq_chip {
        void (*irq_mask)(struct irq_data *);
        void (*irq_unmask)(struct irq_data *);
        void (*irq_eoi)(struct irq_data *);
        int  (*irq_set_affinity)(struct irq_data *, const cpumask_t *, bool);
        int  (*irq_set_type)(struct irq_data *, unsigned int flow);
};

/* Return values, and why they matter: */
   IRQ_HANDLED      it was mine and I dealt with it
   IRQ_NONE         NOT mine — required on shared lines
   IRQ_WAKE_THREAD  wake the threaded handler
```

Debugging interrupts

```
# is it firing at all, and on which core?
watch -n1 'cat /proc/interrupts | grep -E "CPU|uart"'

# what does the kernel think the configuration is?
cat /sys/kernel/debug/irq/irqs/24
#   handler:  handle_fasteoi_irq
#   status:   0x00000000
#   istate:   0x00000000
#   ddepth:   0
#   wdepth:   0
#   dstate:   0x02400200
#   node:     0
#   affinity: 0-3
#   effectiv: 0
#   domain:  :interrupt-controller@2f000000
#    hwirq:   0x38            ← the raw GIC INTID
#    chip:    GICv3

cat /sys/kernel/debug/irq/domains/*

# trace them
echo 1 > /sys/kernel/debug/tracing/events/irq/enable
cat /sys/kernel/debug/tracing/trace_pipe

# how long do handlers take?
perf record -e irq:irq_handler_entry -e irq:irq_handler_exit -a
sudo bpftrace -e 'tracepoint:irq:irq_handler_entry {
        @start[cpu] = nsecs; }
    tracepoint:irq:irq_handler_exit /@start[cpu]/ {
        @us = hist((nsecs - @start[cpu]) / 1000); }'

# find long interrupts-disabled regions
echo irqsoff > /sys/kernel/debug/tracing/current_tracer
echo 0 > /sys/kernel/debug/tracing/tracing_max_latency
cat /sys/kernel/debug/tracing/trace
```

THE FIVE-QUESTION CHECKLIST FOR "MY INTERRUPT NEVER FIRES"

1. **Is the device generating it?** Read its status register directly, in a loop.
2. **Is the INTID right?** Device tree SPI numbers are relative to 32.
3. **Is the trigger type right?** An edge configuration on a level source fires once, then never.
4. **Is it enabled at all four levels?** Device, distributor, CPU interface, PSTATE.
5. **Is it routed to the core you are watching?** Check every CPU column.

In practice, the answer is (2) or (3) most of the time, and both are device tree errors rather than driver bugs.

How it shows up

Symptom	Cause
System hangs at 100% CPU in the handler	EOI'd without clearing at the device — a storm
`"irq N: nobody cared"`	A shared handler returned `IRQ_HANDLED` without handling it, or nothing owns the line

Symptom	Cause
Device works at low rate, hangs under load	Lost edge — add the re-check loop
Interrupt fires exactly once	Not EOI'd, or configured edge when it is level
Count stays zero	Wrong INTID, wrong trigger, not enabled, or routed elsewhere
Spurious INTID 1023	Acknowledged with nothing pending — a race, or a stale read
All interrupts on CPU0, poor scaling	No affinity spreading
Latency spikes on isolated cores	Interrupts not steered away
High-priority interrupt does not preempt	Binary point grouping both at one preemption level
Threaded handler storms on a level interrupt	Needs `IRQF_ONESHOT` and split EOI mode

Check yourself

- ✓ Draw the interrupt state machine and explain what ACTIVE prevents.
- ✓ Give the eight steps from device assertion to `ERET`.
- ✓ Explain why clearing at the device must precede EOI.
- ✓ Explain the two EOI modes and give two uses for the split form.
- ✓ Compare edge and level on sharing and on lost interrupts.
- ✓ Write an edge-safe handler and explain the re-check loop.
- ✓ Say why returning `IRQ_HANDLED` incorrectly breaks spurious detection.
- ✓ Name the four masking levels and the fifth thing to check.
- ✓ Explain why ARM needs no IPI for TLB shootdown.

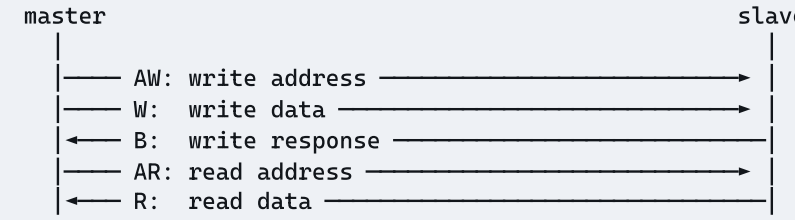
AMBA: APB, AHB, AXI, ACE and CHI

The interconnect is where a load instruction finally becomes electrical activity. Which protocol a block speaks tells you its performance class — and the sideband signals a transaction carries are the foundation of everything in the security track.

The protocol family

Protocol	Year	Character	Typical use
APB	1996	Simple, no bursts, one transfer at a time, two clock cycles minimum	Slow peripheral registers: UART, I2C, GPIO, timers
AHB	1999	Single shared bus, pipelined address/data, bursts	Cortex-M systems, simpler SoCs
AXI	2003	**Five independent channels**, out-of-order via IDs, multiple outstanding transactions	The workhorse: memory controllers, DMA, GPU
ACE	2011	AXI plus coherency: snoop channels	Coherent CPU clusters
ACE-Lite	2011	Can snoop others, has no cache to be snooped	I/O-coherent DMA masters
CHI	2013	Packet-based, credit flow control, scalable mesh	Large multi-core and server SoCs

AXI's five channels




```
Each channel is INDEPENDENT, with its own VALID/READY handshake.
Two consequences that matter enormously:

1. OUTSTANDING TRANSACTIONS. Because address and data are
   separate channels, a master can issue many addresses before
   any data returns. This is how a memory system hides 200-cycle
   DRAM latency — keep dozens of requests in flight rather than
   waiting for each. Little's Law again (CA chapter 18).

2. OUT-OF-ORDER COMPLETION. Each transaction carries an ID
   (AWID/ARID), and responses may return in any order. A slow
   DRAM read does not block a fast SRAM read behind it.

The write response channel (B) is why a write can be ACKNOWLEDGED
before it reaches the endpoint — the "early ack" in
Device-nGnRE (chapter 11), and the reason a posted write needs
a read-back to force completion.
```

What a transaction carries

> **THIS LIST IS THE FOUNDATION OF THE ENTIRE SECURITY TRACK**
> An AXI transaction is far more than an address and some data:

Signal	Carries	Used by
`AxADDR`	The physical address	Everything
`AxID`	Transaction ID — from which the SoC derives a **master ID** or **StreamID**: *who is asking*	SMMU, XPU
`AxPROT[1]`	The **NS bit**: Secure or Non-secure	**TrustZone — this one wire is the entire bus-level enforcement**
`AxPROT[0]`	Privileged or unprivileged	Peripheral protection
`AxPROT[2]`	Instruction or data fetch	Execute-never enforcement
`AxCACHE`	Cacheability and bufferability, derived from the page table's `AttrIndx`	Caches, write buffers
`AxLEN`, `AxSIZE`, `AxBURST`	Burst length, width and type	Memory controller efficiency
`AxLOCK`	Exclusive access	The global exclusive monitor (ch. 22)
`AxUSER`	Implementation-defined sideband	Vendor-specific: often carries a StreamID or QoS tag
`AxQOS`	Quality-of-service priority	Arbitration under contention

> **PAGE TABLE BITS BECOME WIRES**
> The `NS` and `AttrIndx` fields you set in a translation descriptor (chapter 14) are not abstractions. They are converted into `AxPROT[1]` and `AxCACHE` on the bus, and hardware downstream — the SMMU, the TZASC, an XPU — makes allow-or-deny decisions from them.
>
> Setting a bit in a page table entry is *setting a signal on the interconnect*. Once you see that, the whole security track stops being a separate subject.

ACE and coherency

```
ACE adds three SNOOP channels to AXI:

   AC   snoop address    the interconnect asks a master
                         "do you have this line?"
   CR   snoop response   "yes, modified" / "no" / "yes, clean"
   CD   snoop data       the master supplies the line

  This is how MESI is implemented in hardware (CA chapter 15).
  When core 0 wants a line that core 1 holds modified, the
  interconnect snoops core 1, gets the data, and forwards it -
  without going to DRAM at all.

  ACE          full coherency: can snoop AND be snooped.
               For CPU clusters with caches.
  ACE-Lite     can snoop others, but has no cache of its own.
               For DMA masters that need to see CPU-cached data.
               This is what `dma-coherent` in the device tree
               actually means (chapter 19).
  AXI          no coherency at all. Software must do cache
               maintenance (chapter 18).

  The DISTRIBUTED VIRTUAL MEMORY (DVM) messages ride on the same
  channels: TLB invalidations, instruction cache invalidations and
  synchronisation. That is what makes TLBI ...IS broadcast in
  hardware rather than needing an IPI (chapter 15).
```

CHI: scaling past a bus

```
Broadcast snooping does not scale. With 64 cores, every coherent
access would snoop 63 caches.

CHI (Coherent Hub Interface) replaces the bus model entirely:

   · PACKET-BASED, not signal-based. Requests, responses, snoops
     and data are messages on a mesh network.
   · CREDIT-BASED flow control - no combinatorial ready/valid
     paths spanning the chip.
   · A HOME NODE per address range keeps a DIRECTORY of which
     caches hold which lines, so invalidations are sent
     point-to-point rather than broadcast.
   · Scales to hundreds of nodes on a 2-D mesh.

            ┌─────┐ ┌─────┐ ┌─────┐ ┌─────┐
            │ CPU├─┤ CPU├─┤ HN ├─┤ MC │      HN = home node
            └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘      MC = memory controller
            ┌──┴──┐ ┌──┴──┐ ┌──┴──┐ ┌──┴──┐      SN = slave node
            │ CPU├─┤ CPU├─┤ SN ├─┤ IO │
            └─────┘ └─────┘ └─────┘ └─────┘

  Implementations: Arm CMN-600/650/700, and every Neoverse-based
  server SoC.

  The practical consequence for software: latency now depends on
  DISTANCE across the mesh. Two cores adjacent to each other
  communicate faster than two on opposite corners, and memory
  attached to a near controller is faster than a far one. This is
  NUMA appearing inside a single chip, and it is why scheduler
  and allocator locality matters even on "one socket".
```

APB: still everywhere

```
/* The simplest protocol, and the one most peripheral registers
   actually sit behind.                                        */

   PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA, PREADY, PSLVERR

/* A transfer takes a minimum of TWO cycles (setup then access),
   with no pipelining and no bursts. That is why:

     · a register read behind an APB bridge can cost 20–100 CPU
       cycles even though the peripheral is "on chip"
     · reading a status register in a tight polling loop is
       genuinely expensive
     · the APB clock is often much slower than the CPU clock,
       multiplying the cost further

   Practical consequence: minimise register accesses in hot
   paths. Cache values in software where the hardware permits,
   batch writes, and prefer a single wide read over several
   narrow ones. A driver that reads six status registers per
   interrupt is doing something measurably expensive.       */
```

Interconnect topology

```
   SHARED BUS        one set of wires, arbitration decides who
                     drives. Simple, cheap, does not scale.
                     → AHB, APB

   CROSSBAR          every master to every slave. Full parallelism
                     between disjoint pairs; area grows as N×M.
                     → small AXI interconnects (NIC-400)

   NETWORK ON CHIP   routers and links; packets find their way.
                     Scales to dozens of masters, at the cost of
                     variable latency.
                     → CMN, and most large SoC fabrics

   POINT TO POINT    dedicated links, switched.
                     → PCIe, and CHI links

A typical SoC has SEVERAL of these, layered:

   CPU cluster ──CHI──► coherent mesh ──► memory controllers
                              │
                         ──AXI──► GPU, DMA, video
                              │
                         ──AXI──► APB bridge ──► UART, I2C, GPIO
                                                 SPI, timers

  Reading a UART register therefore crosses a mesh, an AXI
  interconnect and an APB bridge. That is the real reason MMIO
  is slow — not the peripheral, the journey.
```

Quality of service and bandwidth

```
/* AxQOS gives each transaction a 4-bit priority. Interconnect
   arbiters use it to decide who wins under contention.      */

/* Why it matters in real systems: a display controller MUST get
   its data before the scan-out deadline or the screen tears.
   A CPU running a batch job can wait. Without QoS, the CPU's
   bandwidth-hungry memcpy starves the display.

   Typical assignment on a phone SoC:
      display / camera   highest — hard real-time deadlines
      GPU                high
      CPU                medium
      background DMA     low

   Configured in the interconnect's registers, usually by
   firmware, and often exposed to Linux via the interconnect
   framework:                                                */

        &display {
                interconnects = <&mmss_noc MASTER_MDP &mc_virt SLAVE_EBI1>;
                interconnect-names = "mdp0-mem";
        };

/* And in the driver: */
        path = of_icc_get(dev, "mdp0-mem");
        icc_set_bw(path, avg_bw, peak_bw);

/* MPAM (ARMv8.4) extends this to cache and memory bandwidth
   PARTITIONING, which is what mixed-criticality systems need to
   bound interference between a real-time and a best-effort
   guest (OS chapter 19).                                    */
```

Bus errors

```
/* A slave can respond with an error, and the response reaches
   the core as an ABORT.                                      */

   AXI RRESP / BRESP:
      OKAY        normal completion
      EXOKAY      exclusive access succeeded
      SLVERR      the slave signalled an error — unclocked
                  peripheral, illegal register, denied access
      DECERR      no slave at that address — a DECODE error

/* How software sees them:

   SYNCHRONOUS — for a read, or a non-posted write, the core is
   still waiting, so the abort is precise:
      ESR_EL1.EC   = 0x24/0x25 (data abort)
      ESR_EL1.DFSC = 0b010000  (synchronous external abort)
      FAR_EL1      = the address

   ASYNCHRONOUS — for a POSTED write, the core moved on long ago.
   The error arrives as an SError, and the reported PC has
   NOTHING to do with the cause. This is the hardest fault class
   to attribute, and it is exactly what an XPU denial of a posted
   write produces (security track, chapter 20).

   Mitigation: ESB (Error Synchronization Barrier) at known
   points confines an SError to a bounded region of code, making
   attribution possible. The RAS extension adds error records
   describing what failed.                                    */
```

Observing the interconnect

```
# the memory map — what lives where
cat /proc/iomem

# interconnect bandwidth requests (where the framework is used)
cat /sys/kernel/debug/interconnect/interconnect_summary

# PMU events for bus traffic
perf stat -e bus_access,bus_access_rd,bus_access_wr,\
bus_cycles,mem_access ./prog

# many SoCs expose interconnect PMUs — CMN, DSU, memory controller
perf list | grep -iE 'cmn|arm_dsu|imc'
perf stat -e arm_cmn/hnf_cache_miss/ ./prog

# bandwidth saturation is the usual finding: if bus_access is
# high and IPC is low with few cache misses, you are waiting on
# the interconnect, not on DRAM latency.

# in a JTAG debugger, most vendors provide interconnect trace —
# Arm CoreSight can capture bus transactions directly, which is
# the definitive way to see what actually reached a peripheral.
```

How it shows up

Symptom	Cause
Register access is far slower than expected	APB bridge with a slow clock; batch or cache accesses
External abort reading a peripheral	DECERR — nothing at that address; or SLVERR — unclocked
SError with an unrelated PC	A posted write was rejected long after it was issued
Write appears to succeed but has no effect	Posted write; read back to force completion
Display tearing under CPU load	QoS not configured; CPU starving the display of bandwidth
DMA works but is not coherent	Master is on plain AXI, not ACE-Lite
Latency varies with which cores communicate	Mesh distance — NUMA within one chip
Exclusives fail on a shared region	`AxLOCK` not supported by that slave, or Device memory
Access denied with no obvious reason	An interconnect firewall checking master ID and NS bit
Bandwidth-bound despite low miss rates	Interconnect saturation, not DRAM

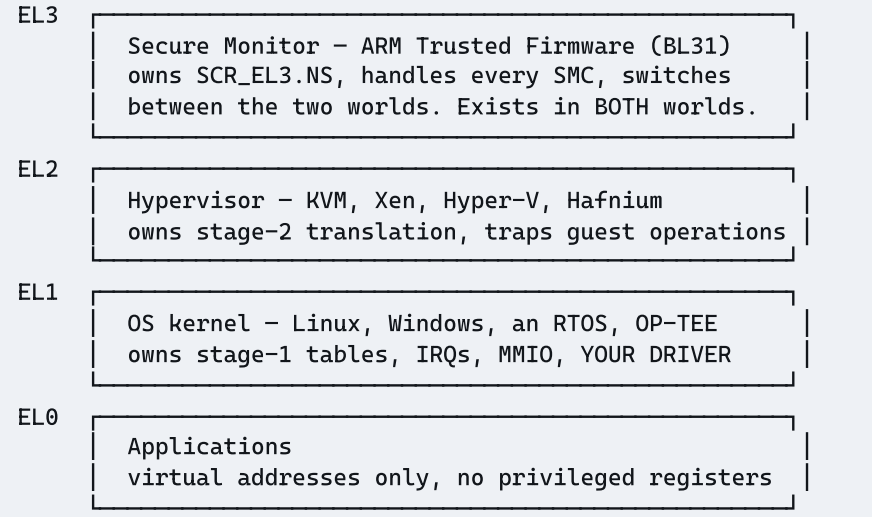
Check yourself

- ✓ Name the AMBA protocols and say what class of block uses each.
- ✓ Explain AXI's five channels and the two properties they enable.
- ✓ List what a transaction carries besides address and data, and why security depends on it.
- ✓ Explain how a page table bit becomes a bus signal.
- ✓ Distinguish ACE, ACE-Lite and plain AXI for a DMA master.
- ✓ Explain why CHI replaced broadcast snooping and what that means for software locality.
- ✓ Say why an APB register read can cost a hundred cycles.
- ✓ Distinguish SLVERR from DECERR, and synchronous from asynchronous aborts.
- ✓ Explain what QoS is for and give an example where its absence is visible.

Exception levels

ARMv8 replaced ARMv7's processor modes with four numbered privilege levels. Every piece of system software on the machine is defined by which one it runs at, and the vocabulary of exception levels is shared by kernels, hypervisors and secure firmware alike.

The four levels




```
Higher number = more privilege, and privilege is strictly
NESTED: EL3 can do everything EL2 can, EL2 everything EL1 can.
There is no "EL0-only" capability — which is why AP bits cannot
express one (chapter 14) and PAN exists instead.

Orthogonal to all of this is the SECURITY STATE (Secure or
Non-secure), giving S-EL0, S-EL1 and S-EL2 as separate
execution contexts with their own register banks.
```

What each level uniquely controls

Level	Owns	Cannot
EL0	Its own general registers, and whatever memory EL1 mapped for it	Access system registers, execute privileged instructions, disable interrupts, touch MMIO
EL1	`TTBR0/1_EL1`, `SCTLR_EL1`, `VBAR_EL1`, interrupt masks, cache and TLB maintenance, device access	See or change stage-2 translation; escape a hypervisor
EL2	`VTTBR_EL2` (stage 2), `HCR_EL2` (traps), virtual interrupts, guest timers	Change the security state
EL3	`SCR_EL3` — the NS bit itself — interrupt routing, SMC handling	—

Which levels exist is a design choice

```
/* EL2 and EL3 are OPTIONAL. Never assume — query. */

        mrs     x0, id_aa64pfr0_el1
/*  ID_AA64PFR0_EL1:
       [3:0]    EL0 support   [7:4]   EL1
       [11:8]   EL2 support   [15:12] EL3
       0 = not implemented, 1 = AArch64 only, 2 = both states  */

static bool has_el2(void)
{
        uint64_t pfr0;
        asm volatile("mrs %0, id_aa64pfr0_el1" : "=r"(pfr0));
        return ((pfr0 >> 8) & 0xF) != 0;
}

/* Typical configurations:
     simple embedded SoC     EL0, EL1 only
     phone / tablet          all four
     server                  all four, plus S-EL2
     Cortex-M                none of this — a different model
                             entirely (chapter 02)             */

/* And note: a level EXISTING is not the same as being USED.
   Linux boots at EL2 on most platforms and immediately drops to
   EL1 unless KVM is enabled. Which level you are at right now:  */
        mrs     x0, CurrentEL        /* bits [3:2] hold the EL */
        lsr     x0, x0, #2
```

Moving between levels

> **UP ONLY BY EXCEPTION, DOWN ONLY BY `ERET`**
>
> You can never simply *jump* to a higher exception level. The only way up is to take an exception — and the entry point is the *higher* level's vector table, at an address *it* chose. That is what makes privilege enforceable: the caller supplies a number, never an address.

Direction	Mechanism
EL0 → EL1	`SVC`, an interrupt, or a fault
EL1 → EL2	`HVC`, a trapped operation, a stage-2 fault, or a routed interrupt
Any → EL3	`SMC`, a Secure interrupt, or an external abort routed to EL3
Downward	`ERET` — target EL from `SPSR_ELx.M`, target PC from `ELR_ELx`
Sideways (world switch)	EL3 changes `SCR_EL3.NS`, then `ERET`s

```
/* Dropping from EL3 to EL1 — what firmware does at the end of
   boot (OS chapter 18):                                      */

        /* configure the world we are about to enter */
        mov     x0, #(1 << 10)          /* RW=1: EL1 below is AArch64 */
        orr     x0, x0, #(1 << 0)       /* NS=1: Non-secure           */
        msr     scr_el3, x0

        /* the PSTATE to install on ERET */
        mov     x0, #0x3C5              /* M=0b0101 (EL1h), DAIF masked */
        msr     spsr_el3, x0

        ldr     x0, =kernel_entry
        msr     elr_el3, x0

        eret                            /* now at EL1, Non-secure */

/* SPSR.M[3:0] encodes the target:
      0b0000 EL0t     0b0100 EL1t     0b1000 EL2t     0b1100 EL3t
                      0b0101 EL1h     0b1001 EL2h     0b1101 EL3h

   The t/h suffix selects SP_EL0 (thread) or SP_ELx (handler)
   as the stack pointer — chapter 04.                         */
```

Where an exception is taken

```
An exception does NOT always go to the next level up. Routing is
configurable, and this is the mechanism virtualisation is built
from.

   SYNCHRONOUS exceptions (faults, SVC)
        → normally the current EL, or EL1 if from EL0
        → HCR_EL2 trap bits can redirect specific ones to EL2
        → HCR_EL2.TGE routes ALL EL0 exceptions to EL2
          (used when the host runs at EL2 with VHE)

   IRQ / FIQ / SError
        → SCR_EL3.{IRQ,FIQ,EA}   route to EL3
        → HCR_EL2.{IMO,FMO,AMO}  route to EL2 instead of EL1

   The typical configuration on a virtualised system:
        IRQ  → EL2   the hypervisor decides which guest gets it,
                     and injects a VIRTUAL interrupt via the GIC
                     list registers (chapter 23)
        FIQ  → EL3   secure interrupts, unmaskable by the guest
        SMC  → EL3   always

   And on a non-virtualised system:
        IRQ  → EL1   straight to the kernel
        FIQ  → EL3   if a secure world exists

  An exception can never be taken to a LOWER level than the one
  that was executing. So EL2 code's faults go to EL2 or EL3,
  never to EL1.
```

Per-level register banks

```
/* Each level has its OWN copy of the exception-handling
   registers. These are genuinely separate registers, not views: */

   ELR_EL1, ELR_EL2, ELR_EL3          return address
   SPSR_EL1, SPSR_EL2, SPSR_EL3       saved PSTATE
   ESR_EL1, ESR_EL2, ESR_EL3          syndrome
   FAR_EL1, FAR_EL2, FAR_EL3          fault address
   VBAR_EL1, VBAR_EL2, VBAR_EL3       vector base
   SP_EL0, SP_EL1, SP_EL2, SP_EL3     stack pointers
   TTBR0_EL1/EL2/EL3, TCR_*, MAIR_*, SCTLR_*

/* Banking these is what allows an exception at EL2 to occur while
   EL1's ELR still holds a pending return — the two do not
   collide.

   But note what is NOT banked: X0–X30. There are no banked
   general-purpose registers in AArch64 (chapter 04), which is
   why every handler must save them in software.            */

/* And Secure/Non-secure banking: the EL1 register set is banked
   AGAIN by security state, so the secure OS and the normal OS
   each have their own SCTLR_EL1, TTBR0_EL1 and so on, selected
   by SCR_EL3.NS. EL3 sees whichever the current state selects,
   which is why the monitor must save and restore them
   explicitly on a world switch.                            */
```

What runs where, in practice

Level	Non-secure	Secure
EL0	Applications, libraries	Trusted applications (keymaster, DRM, biometrics)
EL1	Linux, Windows, an RTOS	Trusted OS: OP-TEE, Trusty, QSEE
EL2	KVM, Xen, Hyper-V, pKVM	Secure partition manager: Hafnium (ARMv8.4+)
EL3	ARM Trusted Firmware — one monitor serving both worlds	

VHE: the host at EL2

```
/* The original design assumed a hypervisor was a small separate
   program at EL2 with the host OS at EL1. That is wrong for KVM,
   where the "hypervisor" IS Linux.                          */

/* Without VHE: Linux runs at EL1 with a small stub at EL2, and
   every VM entry/exit crosses between them — saving and
   restoring the host's EL1 state each time.

   With VHE (HCR_EL2.E2H = 1, ARMv8.1):
     · EL2 gains TTBR1_EL2 and behaves like the EL1&0 regime
     · EL0 can run directly under EL2 (host userspace)
     · accessing the _EL1 register names AT EL2 is REDIRECTED to
       the _EL2 equivalents
     · the _EL12 accessors reach the real guest EL1 state

   Result: an unmodified Linux kernel runs at EL2, host
   userspace at EL0, guests at EL1. The host never leaves EL2,
   so VM exits are far cheaper.                              */

        mrs     x0, id_aa64mmfr1_el1
        ubfx    x0, x0, #8, #4          /* VH field */
        cbz     x0, no_vhe

/* You can see which mode a system is in: */
//   dmesg | grep -i 'hyp mode'
//   kvm [1]: Hyp mode initialized successfully        (no VHE)
//   kvm [1]: VHE mode initialized successfully        (VHE)
```

Determining your level at runtime

```
static inline unsigned current_el(void)
{
        uint64_t el;
        asm volatile("mrs %0, CurrentEL" : "=r"(el));
        return (el >> 2) & 3;
}

/* Boot code commonly branches on this, because firmware may
   hand off at EL2 or EL1 depending on the platform:        */

        mrs     x0, CurrentEL
        cmp     x0, #(2 << 2)
        b.eq    at_el2
        cmp     x0, #(1 << 2)
        b.eq    at_el1
        b       unsupported

at_el2: /* configure EL2, then drop to EL1 - or stay if we are
           going to be a hypervisor */
        bl      init_el2
        b       drop_to_el1

at_el1: /* already where we want to be */
        bl      init_el1

/* Linux's head.S does exactly this, and the "Booted at EL2" /
   "Booted at EL1" distinction determines whether KVM can be
   used at all.                                              */
//   dmesg | grep -i 'CPU: All CPU(s) started at EL'
```

Why EL3 is special

```
EL3 is the only level that exists in BOTH security states
simultaneously. It is not "the most privileged part of the secure
world" - it is the gatekeeper between the worlds.

   Its unique responsibilities:
     · owns SCR_EL3.NS - the bit that selects which world the
       levels below are in
     · saves and restores the ENTIRE state of one world before
       switching to the other, including scrubbing registers so
       nothing leaks
     · dispatches every SMC to the right service (PSCI, a trusted
       OS, a SiP-specific function)
     · routes secure interrupts

   Consequences:
     · EL3 code is the most security-critical on the chip. A bug
       there compromises everything, in both worlds.
     · it should be SMALL. ARM Trusted Firmware's BL31 is tens of
       thousands of lines, deliberately.
     · it stays RESIDENT for the machine's whole life, unlike the
       earlier boot stages.

  The security track covers the world switch, SMC calling
  convention and the TEE stack in detail.
```

How it shows up

Symptom	Cause
KVM will not initialise	Booted at EL1, not EL2 — check firmware and `dmesg`
Undefined instruction on `MRS`	Register requires a higher exception level
`HVC` causes an undefined instruction	EL2 not implemented, or `HCR_EL2.HCD` disables it
`SMC` causes an undefined instruction	EL3 not implemented, or `SCR_EL3.SMD` set
Guest hangs on a system register write	A trap is configured and the hypervisor is not emulating it
Interrupts never reach the kernel	Routed to EL2 or EL3 by `HCR` / `SCR`
Secure interrupt unmaskable by Linux	Delivered as FIQ, routed to EL3 — by design
VM exits unexpectedly expensive	No VHE; host EL1 state saved and restored every exit
Firmware works on one SoC, not another	Assumed EL2 or EL3 exists without querying `ID_AA64PFR0_EL1`

Check yourself

- ✓ Name the four exception levels and what runs at each, in both security states.
- ✓ Explain why privilege being strictly nested means AP bits cannot express "EL0 only".
- ✓ State the rule for moving up and down between levels, and why it makes privilege enforceable.
- ✓ Decode `SPSR.M = 0b0101`.
- ✓ Explain how `HCR_EL2` and `SCR_EL3` change where an interrupt is taken.
- ✓ Say which registers are banked per level and which are not.
- ✓ Explain what VHE changes and why KVM needed it.
- ✓ Explain why EL3 is different in kind from the other levels.

Exception types and the vector table

Sixteen entries of 128 bytes each, indexed by where the exception came from and what kind it was. The layout looks arbitrary until you see what it is optimising for — and then it explains why kernel entry code is structured the way it is.

What counts as an exception

```
ARM uses "exception" as the umbrella term for anything that
diverts execution to a handler. Two families:

  SYNCHRONOUS — caused by the current instruction, reported
                PRECISELY. You know exactly which instruction.

     SVC / HVC / SMC        deliberate calls to EL1 / EL2 / EL3
     Data abort             a load or store failed translation
                            or permission checks
     Instruction abort      the FETCH itself failed
     Undefined instruction  bad encoding, or disabled at this EL
     Alignment fault        unaligned Device access, or SCTLR.A
     Trapped register access a higher EL configured a trap
     PC alignment fault     the PC was not 4-byte aligned
     SP alignment fault     SP was not 16-byte aligned
     Breakpoint / watchpoint / software step

  ASYNCHRONOUS — caused by something outside the instruction
                 stream, taken BETWEEN instructions.

     IRQ      normal interrupt.  Masked by PSTATE.I
     FIQ      fast interrupt.    Masked by PSTATE.F
     SError   System Error — typically an IMPRECISE external
              abort: an ECC failure, or a bus error from a write
              posted long ago. Masked by PSTATE.A.

  SError is the awkward one: the reported PC has NOTHING to do
  with the cause, because the failing transaction retired long
  before the error came back (chapter 25).
```

The vector table

```
VBAR_ELx holds the base address. 16 entries, 128 bytes each,
2 KB total.

VBAR + offset    source of the exception              type

  0x000   Synchronous  ┐
  0x080   IRQ / vIRQ   │  CURRENT EL, using SP_EL0
  0x100   FIQ / vFIQ   │  (SPSel = 0, "EL1t")
  0x180   SError       ┘

  0x200   Synchronous  ┐
  0x280   IRQ / vIRQ   │  CURRENT EL, using SP_ELx
  0x300   FIQ / vFIQ   │  (SPSel = 1, "EL1h")   ← the normal
  0x380   SError       ┘                           kernel case

  0x400   Synchronous  ┐
  0x480   IRQ / vIRQ   │  LOWER EL, AArch64
  0x500   FIQ / vFIQ   │  ← syscalls and user faults land here
  0x580   SError       ┘

  0x600   Synchronous  ┐
  0x680   IRQ / vIRQ   │  LOWER EL, AArch32
  0x700   FIQ / vFIQ   │  (32-bit compatibility)
  0x780   SError       ┘
```

WHY SPLIT BY SOURCE

The handler needs to know immediately whether it must switch stacks and save user state, or whether this is a nested exception already on the kernel stack. Encoding that in the *vector address* means the answer costs zero instructions — no test, no branch, no register read. On a path taken millions of times a second, that matters.

The AArch32 split exists for the same reason: a 32-bit process's register layout differs, so it needs a different save path.

Each entry is code, not an address

```
/* Unlike Cortex-M, where the vector table holds ADDRESSES, on
   A-profile each entry is 128 bytes of actual CODE — 32
   instructions. You either handle it there or branch out.  */

.macro VECTOR_ENTRY label
        .align 7                        /* 128 bytes = 2^7 */
        b       \label
.endm

        .align 11                       /* the table must be 2 KB
                                           aligned */
        .global vector_table
vector_table:
        /* current EL, SP_EL0 — should never happen in a normal
           kernel, so these are error handlers */
        VECTOR_ENTRY  invalid_sync_el1t
        VECTOR_ENTRY  invalid_irq_el1t
        VECTOR_ENTRY  invalid_fiq_el1t
        VECTOR_ENTRY  invalid_err_el1t

        /* current EL, SP_ELx — kernel-mode exceptions */
        VECTOR_ENTRY  el1_sync
        VECTOR_ENTRY  el1_irq
        VECTOR_ENTRY  el1_fiq
        VECTOR_ENTRY  el1_error

        /* lower EL, AArch64 — user-mode exceptions and syscalls */
        VECTOR_ENTRY  el0_sync
        VECTOR_ENTRY  el0_irq
        VECTOR_ENTRY  el0_fiq
        VECTOR_ENTRY  el0_error

        /* lower EL, AArch32 */
        VECTOR_ENTRY  el0_sync_compat
        VECTOR_ENTRY  el0_irq_compat
        VECTOR_ENTRY  el0_fiq_compat
        VECTOR_ENTRY  el0_error_compat

/* Installing it: */
        adr     x0, vector_table
        msr     vbar_el1, x0
        isb                             /* REQUIRED — chapter 06 */
```

ALIGNMENT IS NOT ADVISORY

`VBAR_ELx` requires **2 KB alignment** — the low 11 bits are RES0 and writing a misaligned address silently ignores them. Each entry must be exactly 128 bytes apart. Get either wrong and exceptions dispatch to the wrong handler, or into the middle of an instruction.

The symptom is spectacular and unhelpful: the machine takes an exception and immediately takes another, usually ending in an unrecoverable loop before anything can be printed. Use `.align 11` for the table and `.align 7` per entry, and verify with `readelf -s | grep vector`.

Why 128 bytes matters

```
/* 32 instructions is enough to do real work before branching,
   which several important techniques exploit:               */

/* 1. Linux's KPTI trampoline. When kernel page-table isolation
      is active, the entry code must switch TTBR1 before it can
      touch kernel memory — and that switch has to happen in the
      vector itself, using only registers.                   */

/* 2. Spectre-BHB mitigations. Some cores need a loop of branches
      or a specific instruction sequence at the very start of the
      vector to flush the branch history buffer before any
      speculation on kernel addresses can occur. Linux patches
      the vectors at boot to insert exactly the sequence that
      core requires.                                          */

/* 3. Fast paths. A hypervisor can handle a trivial trap
      entirely within the vector entry and ERET without ever
      building a full context save.                          */

/* Which is why Linux's vectors are generated by a macro with
   several variants, and why `objdump -d` of the vector region
   shows more than a single branch on modern kernels.       */
```

Virtual interrupts

```
/* When HCR_EL2.{IMO,FMO,AMO} are set, physical IRQ/FIQ/SError go
   to EL2, and the entries at EL1 receive VIRTUAL interrupts
   instead — injected by the hypervisor through the GIC list
   registers (chapter 23).                                   */

   HCR_EL2.IMO = 1   physical IRQ    → EL2
                     virtual vIRQ    → EL1
   HCR_EL2.VI  = 1   assert a virtual IRQ to the guest

/* From the guest's point of view nothing is different: it takes
   an exception at VBAR_EL1 + 0x480 exactly as it would on bare
   metal. It cannot tell the interrupt was injected.

   This is what makes guest interrupt handling fast: no
   hypervisor trap is needed for the guest to acknowledge and
   EOI, because the GIC's virtual CPU interface handles it.  */
```

What hardware does on entry

```
Before your first instruction runs, the hardware has ALREADY:

   1. ELR_ELx   ← the address to return to
   2. SPSR_ELx  ← the entire PSTATE at the moment of the exception
   3. ESR_ELx   ← the syndrome: exception class and details
   4. FAR_ELx   ← the faulting address (aborts and watchpoints)
   5. PSTATE    → EL raised, SPSel = 1, and D, A, I, F ALL MASKED
   6. PC        ← VBAR_ELx + the vector offset

  Step 5's masking is not optional and not configurable. The
  reason: ELR_ELx and SPSR_ELx are single registers, not a stack.
  A second exception before your handler saved them would
  overwrite the return address, and you could never get back.

  Your handler saves them to the kernel stack FIRST, and only
  then may unmask to allow nesting. Chapter 28 covers the full
  sequence.

  Note what is NOT saved: X0–X30. Nothing is banked in AArch64
  (chapter 04). Software saves everything.
```

The "impossible" entries

```
/* The four "current EL with SP_EL0" entries should never be taken
   in a correctly configured kernel, which runs with SPSel = 1.
   Reaching one means something is badly wrong — and that is
   exactly why they are worth implementing properly.        */

invalid_sync_el1t:
        /* We cannot trust SP. Switch to a known-good emergency
           stack before doing anything.                       */
        adr_this_cpu x0, emergency_stack, x1
        mov     sp, x0
        bl      panic_bad_mode

/* Common causes of landing here:
     · SPSel accidentally cleared
     · a corrupted SP that faulted during the save sequence
     · a fault in early boot before SP_EL1 was established
     · stack overflow — the guard page faulted, and the fault
       handler could not push a frame

   Linux handles the stack-overflow case specially with an
   OVERFLOW STACK per CPU, so a kernel stack overflow produces a
   readable oops instead of an infinite fault loop. That is worth
   copying in any serious bare-metal system.              */
```

Contrast: Cortex-M

	A-profile	**M-profile**
Table contains	**Code** — 128 bytes per entry	**Addresses** — 4 bytes per entry
Entry 0	A handler	The initial **stack pointer**
Register saving	Entirely software	Hardware stacks R0–R3, R12, LR, PC, xPSR

	A-profile	M-profile
ISR form	Assembly stub, then C	**A plain C function**
Entries	16, by source and type	One per interrupt, hundreds
Base register	`VBAR_ELx`	`VTOR`

The M-profile design costs a fixed ~12 cycles of hardware stacking on every interrupt but removes the software prologue entirely, which is why M-profile interrupt latency is both lower and far more deterministic. A-profile trades that for flexibility and for not paying the cost on exceptions that do not need it.

Inspecting the vectors

```
# where is the table?
sudo cat /proc/kallsyms | grep vectors
#   ffff800008011000 T vectors

# in gdb on a kernel or bare-metal target
p/x $VBAR_EL1
x/16i $VBAR_EL1              # the first entry
x/4i  $VBAR_EL1 + 0x400      # lower EL, synchronous — the syscall path

# check alignment
readelf -s vmlinux | grep -w vectors
#   the address must end in 0x?800 or 0x?000 — 2 KB aligned

# see what Linux patched in
sudo objdump -d --start-address=0x... vmlinux | head -60
#   on a mitigated kernel you will see BHB-flush sequences
#   before the branch
```

> **DEBUGGING A MACHINE THAT RESETS IMMEDIATELY**
> A misaligned or uninstalled vector table produces a system that appears to die instantly with no output. The diagnostic sequence: halt with JTAG at reset, single-step to the point where `VBAR` is written, read it back, and confirm the low 11 bits are zero. Then set a hardware breakpoint on `VBAR + 0x200` and `VBAR + 0x400` — whichever is hit first tells you whether the fault came from kernel or user context, which usually identifies the bug immediately.

How it shows up

Symptom	Cause
Machine resets immediately, no output	Vector table misaligned or `VBAR` never written
Exceptions dispatch to the wrong handler	Entries not exactly 128 bytes apart
Syscalls work but interrupts crash	Only some vector entries implemented
Kernel-mode faults land in the user-mode handler	Confused the 0x200 and 0x400 groups
Landing in an "invalid mode" handler	`SPSel` wrong, or a corrupted SP
Infinite fault loop on stack overflow	No separate overflow stack; the handler cannot push a frame
32-bit process crashes, 64-bit works	AArch32 vector entries not implemented
Guest interrupts never arrive	`HCR_EL2.IMO` set but no virtual interrupt injected

Symptom	Cause
Configuration change has no effect	Missing `ISB` after writing `VBAR`

Check yourself

- ✓ List the synchronous and asynchronous exception types.
- ✓ Explain why SError is harder to attribute than a data abort.
- ✓ Draw the 16-entry table and say what each group of four is for.
- ✓ Explain why the table is split by exception *source*.
- ✓ State the two alignment requirements and the symptom of getting them wrong.
- ✓ Give two things that exploit having 128 bytes per entry.
- ✓ List the six things hardware does before your handler runs.
- ✓ Explain why DAIF is masked on entry and when you may unmask.
- ✓ Give three differences between A-profile and M-profile vectoring.

Taking and returning from an exception

The single most important flow in system software. Six things happen in hardware, atomically, before your first instruction runs — and everything else is software's responsibility. Get this sequence wrong and you corrupt the program you interrupted, in ways that surface far from the cause.

What hardware does

```
  EL0 executing:  svc #0
       │          (or an IRQ arrives, or a load faults)
       ▼
═══════ HARDWARE, ATOMICALLY — nothing you wrote runs here ═══════

  1. ELR_EL1   ← the address to return to
                 · for SVC:      the instruction AFTER the svc
                 · for a fault:  the FAULTING instruction, so it
                                 can be retried after the OS
                                 fixes the mapping
                 · for an IRQ:   the next instruction to execute

  2. SPSR_EL1  ← the entire PSTATE: NZCV, DAIF, the source EL,
                 SPSel, nRW, PAN, and the rest

  3. ESR_EL1   ← the syndrome: exception class + details
                 (chapter 29)

  4. FAR_EL1   ← the faulting virtual address
                 (aborts and watchpoints only; stale otherwise)

  5. PSTATE    → EL = 1
                 SPSel = 1        (switch to SP_EL1)
                 D, A, I, F ALL MASKED
                 PAN set, if SCTLR_EL1.SPAN says so

  6. PC        ← VBAR_EL1 + the vector offset for
                 (source EL, exception type)

═══════════════════════════════════════════════════════════════════
       │
       ▼
  your handler's first instruction
```

NOTICE WHAT IS NOT SAVED

`X0` – `X30` are untouched. AArch64 has **no banked general-purpose registers** (chapter 04). Your handler begins executing with the interrupted code's register values still live, and every one of them must be preserved by software before you can use it.

That saved set is `struct pt_regs` in Linux and the trap frame in Windows, and building it correctly is the entire job of the vector stub.

The entry stub

```
the shape every AArch64 OS has, simplified from Linux's entry.S
```

```
el0_sync:
        /* SP is now SP_EL1 — the kernel stack for this thread.
           The hardware switched it for us because SPSel became 1.
           This is exactly why per-EL stack pointers exist: a
           corrupt user SP cannot take the kernel down.        */

        sub     sp, sp, #S_FRAME_SIZE       /* room for pt_regs   */

        /* save the general-purpose registers, in pairs */
        stp     x0,  x1,  [sp, #16 * 0]
        stp     x2,  x3,  [sp, #16 * 1]
        stp     x4,  x5,  [sp, #16 * 2]
        /* ... x6 through x27 ... */
        stp     x28, x29, [sp, #16 * 14]

        /* now capture the exception state — BEFORE unmasking
           anything, because a nested exception would overwrite
           these single registers                              */
        mrs     x21, sp_el0                 /* the USER stack ptr */
        mrs     x22, elr_el1                /* where to return    */
        mrs     x23, spsr_el1               /* PSTATE to restore  */

        stp     x30, x21, [sp, #16 * 15]
        stp     x22, x23, [sp, #16 * 16]

        /* only NOW is it safe to enable interrupts, if we want
           nesting */
        msr     daifclr, #2                 /* unmask IRQ         */

        /* dispatch on the syndrome */
        mrs     x25, esr_el1
        lsr     x24, x25, #26               /* EC = ESR[31:26]    */
        cmp     x24, #0x15                  /* SVC from AArch64   */
        b.eq    el0_svc
        cmp     x24, #0x24                  /* data abort, lower EL */
        b.eq    el0_da
        cmp     x24, #0x20                  /* instruction abort   */
        b.eq    el0_ia
        b       el0_inv

el0_svc:
        mov     x0, sp                      /* pass pt_regs to C  */
        bl      do_el0_svc
        b       ret_to_user
```

THE ORDERING RULE

Save `ELR` and `SPSR` before unmasking interrupts. They are single registers, not a stack. A nested exception arriving between entry and the save overwrites the return address, and the interrupted program is unrecoverable.

This is why hardware masks DAIF on entry, and why the mask is cleared only after the save is complete.

The return path

```
ret_to_user:
        /* mask interrupts again — the restore must be atomic with
           respect to a new exception                            */
        msr     daifset, #2

        /* work that must happen before returning to userspace */
        ldr     x1, [tsk, #TSK_TI_FLAGS]
        and     x2, x1, #_TIF_WORK_MASK
        cbnz    x2, work_pending            /* signals? resched?  */

finish_ret_to_user:
        /* restore the exception state */
        ldp     x21, x22, [sp, #16 * 16]
        msr     elr_el1,  x21
        msr     spsr_el1, x22

        ldp     x30, x23, [sp, #16 * 15]
        msr     sp_el0,   x23               /* the user stack     */

        /* restore the general registers */
        ldp     x0,  x1,  [sp, #16 * 0]
        /* ... through x28, x29 ... */

        add     sp, sp, #S_FRAME_SIZE

        eret                                /* back to EL0        */

/* ERET restores PC from ELR_EL1 and PSTATE from SPSR_EL1,
   ATOMICALLY. That atomicity is the point: there is no window in
   which the privilege has changed but the PC has not, or vice
   versa. Any such window would be exploitable.               */
```

The ELR subtlety

Exception	`ELR` points at	Because
`SVC`, `HVC`, `SMC`	The instruction **after**	The call completed; resume past it
Data abort, instruction abort	**The faulting instruction**	The OS fixes the mapping and *retries* it
IRQ, FIQ	The next instruction to execute	Nothing failed; resume normally
Undefined instruction	The undefined instruction	So an emulator can inspect and emulate it
Trapped register access	The trapped instruction	The hypervisor emulates it

```
/* CONSEQUENCE: if you EMULATE an instruction, you must advance
   ELR yourself, or you will re-execute it forever.          */

        mrs     x0, elr_el2
        add     x0, x0, #4              /* A64 is fixed-width */
        msr     elr_el2, x0
        eret

/* This is the single most common hypervisor and emulation bug —
   an infinite loop that looks like a hang, with the guest
   apparently frozen on one instruction. Chapter 31.

   For AArch32 the step is 2 or 4 depending on Thumb, which is
   why compat emulation code reads SPSR to decide.          */
```

Nesting

```
Interrupts CAN nest, once the handler has saved ELR and SPSR:

   EL0 running
        | IRQ (priority 0x80)
        ▼
   el0_irq: save regs, save ELR/SPSR
            msr daifclr, #2          ← now nesting is permitted
            handle interrupt A
        | higher-priority IRQ arrives (priority 0x40)
        ▼
   el1_irq: save regs, save ELR_EL1/SPSR_EL1 to THIS stack frame
            handle interrupt B
            restore, eret
        |
        ▼ back in interrupt A's handler
            finish, restore, eret
        |
        ▼ back to EL0

  Note the second entry used the "current EL, SP_ELx" vector at
  0x280, not the "lower EL" one at 0x480 — which is exactly why
  the table is split by source (chapter 27). The second handler
  does NOT need to switch stacks or save SP_EL0.

  The GIC's priority mechanism decides whether B preempts A
  (chapter 24). Linux by default does NOT nest ordinary
  interrupts — it keeps them masked and relies on short handlers
  plus threaded IRQs instead. Nesting is used for pseudo-NMIs.
```

Dropping to a lower level

```
/* The same ERET mechanism is used to LAUNCH software at a lower
   level — which is how every boot handover works (OS ch. 18). */

/* EL3 firmware handing off to a kernel at EL1: */
        mov     x0, #(1 << 10)          /* SCR.RW = AArch64 below  */
        orr     x0, x0, #(1 << 0)       /* SCR.NS = Non-secure     */
        msr     scr_el3, x0

        mov     x0, #0x3C5              /* SPSR.M = 0b0101 = EL1h  */
                                        /* DAIF all masked         */
        msr     spsr_el3, x0

        ldr     x0, =kernel_entry
        msr     elr_el3, x0

        /* the kernel's boot contract: x0 = DTB, x1-x3 = 0 */
        ldr     x0, =dtb_address
        mov     x1, xzr
        mov     x2, xzr
        mov     x3, xzr

        eret

/* Note there is no "jump to EL1" instruction. ERET is the only
   way down, and it takes its target from SPSR and ELR. Which
   means firmware must construct a plausible "saved state" for a
   context that never actually ran.                          */
```

The cost

```
   hardware entry (steps 1–6)          ~10–20 cycles
   save 31 registers + 3 system regs   ~35 stores, ~20–40 cycles
   dispatch on ESR                     ~5–10 cycles
   ──────────────────────────────────────────────
   entry total                         ~50–100 cycles

   ...handler...

   restore + ERET                      ~50–100 cycles

   Plus the INDIRECT costs, which usually dominate:
     · pipeline flush on entry and on ERET
     · I-cache and branch predictor now hold handler state
     · with KPTI, TWO address-space switches per syscall
     · Spectre-BHB mitigation sequences in the vector

   Which is why a "null" syscall costs 50–500 ns rather than the
   ~30 cycles the register saving suggests, and why io_uring and
   the vDSO exist (OS chapter 03).

  OPTIMISATIONS real kernels use:
     · save only what is needed on the fast path (Linux saves
       everything, for simplicity and for ptrace)
     · a dedicated fast path for SVC that skips fault-handling
       setup
     · the vDSO, removing the exception entirely for
       clock_gettime and similar
```

Stack handling

```
/* Three stack pointers are in play on a user-to-kernel entry:

     SP_EL0   the user stack. Saved into pt_regs, and RESTORED on
              return. The kernel never runs on it.
     SP_EL1   the kernel stack for THIS THREAD. The hardware
              switches to it automatically (SPSel = 1).
     the IRQ stack (Linux)  a separate per-CPU stack that deep
              interrupt handling switches to, so a deeply nested
              interrupt cannot overflow the thread's 16 KB stack.

   And the OVERFLOW stack: Linux reserves a per-CPU stack used
   ONLY when a kernel stack overflow is detected via the guard
   page. Without it, the overflow fault handler would itself fail
   to push a frame, producing an infinite fault loop with no
   diagnostic. With it, you get a readable oops.           */

/* The kernel stack pointer for a thread is established at
   context switch time, from the task structure. `current` is
   read from SP_EL0 while at EL1 — the register is otherwise
   unused there, so Linux repurposes it as a per-CPU pointer to
   the current task (OS chapter 04).                       */
```

Debugging entry and exit

```
# Linux prints the saved state in an oops
dmesg | grep -A25 'Internal error'
#   pc : do_something+0x48/0x120
#   lr : caller+0xb4/0x1f0
#   sp : ffff800009abc000
#   x29: ...  x28: ...  x27: ...
#   Call trace:
#     do_something+0x48/0x120
#     ...
#     el0_svc_common+0x88/0x1a0      ← arrived via a syscall
#   pstate: 60400009 (nZCv daif +PAN -UAO)   ← chapter 05

# in gdb on a kernel target
p/x $ELR_EL1
p/x $SPSR_EL1
p/x $ESR_EL1
p/x $SP_EL0

# breakpoint on the vector itself
b *($VBAR_EL1 + 0x400)      # lower EL, synchronous

# trace syscall entry and exit
echo 1 > /sys/kernel/debug/tracing/events/raw_syscalls/enable
cat /sys/kernel/debug/tracing/trace_pipe

# measure the cost
perf bench syscall basic
perf stat -e raw_syscalls:sys_enter ./prog
```

READING WHICH VECTOR WAS TAKEN

In a crash dump, `SPSR`'s low bits tell you the source context — `0b0000` is EL0t (a user-mode exception, so the 0x400 group), `0b0101` is EL1h (a kernel-mode one, the 0x200 group). Combined with `ESR`'s exception class this pins down exactly which of the sixteen entries ran, which usually identifies the bug immediately.

How it shows up

Symptom	Cause
Interrupted program corrupted after an interrupt	Vector stub failed to save all general registers
Corrupted return address in a nested exception	Unmasked interrupts before saving `ELR` / `SPSR`
Emulated instruction executes forever	`ELR` not advanced past it
Page fault loops on the same address	Correct — until the OS fixes the mapping; if it never does, the OS bug is elsewhere
Returns to the wrong exception level	Wrong `M[3:0]` in `SPSR`
User stack corrupted after a syscall	`SP_EL0` not saved and restored
Infinite fault loop with no output	Stack overflow with no separate overflow stack
Signals never delivered	No work-pending check on the return path
Syscalls unexpectedly slow after a kernel update	KPTI or Spectre mitigations in the entry path
TLS broken after a hand-written context switch	`TPIDR_EL0` not restored

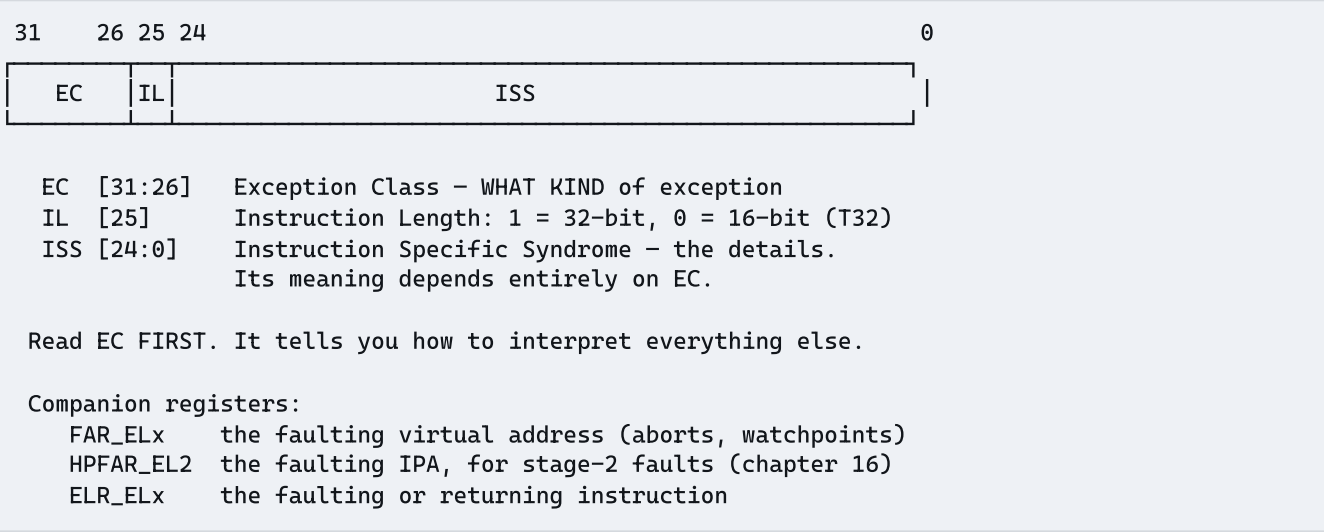
Check yourself

- ✓ List, in order, the six things hardware does on exception entry.
- ✓ Explain why general-purpose registers are not saved and what software must do.
- ✓ State the ordering rule about `ELR` / `SPSR` and unmasking, and why.
- ✓ Say where `ELR` points for an `SVC`, a data abort and an emulated instruction.
- ✓ Explain why `ERET` must be atomic.
- ✓ Explain how nesting works and which vector entry the second exception uses.
- ✓ Show how firmware uses `ERET` to launch a kernel at a lower level.
- ✓ Explain why an overflow stack exists and what happens without one.

ESR: decoding any fault

One register tells you exactly why the last exception happened. Learn its layout and you can read any crash — a kernel oops, a bare-metal hang, a hypervisor trap — and know within seconds what went wrong and where to look next.

The register



Exception classes

EC	Meaning	Usual cause
`0x00`	Unknown reason	A genuinely undefined instruction, or an instruction disabled at this EL
`0x01`	Trapped `WFI` / `WFE`	A hypervisor trapping guest idling
`0x07`	Trapped FP/SIMD access	Lazy FPU switching — `CPACR_EL1.FPEN`
`0x0E`	Illegal execution state	Bad `SPSR` on `ERET`; `PSTATE.IL` set
`0x15`	**`SVC` from AArch64**	**A system call**
`0x16`	`HVC` from AArch64	Hypervisor call
`0x17`	`SMC` from AArch64	Secure monitor call
`0x18`	Trapped `MSR` / `MRS`	A higher EL intercepted a system register access
`0x19`	Trapped SVE access	Lazy SVE state management
`0x20` / `0x21`	Instruction abort, lower / same EL	Fetch failed: unmapped, or execute-never
`0x22`	PC alignment fault	PC not 4-byte aligned — usually a corrupted function pointer
`0x24` / `0x25`	**Data abort, lower / same EL**	**The most common fault**
`0x26`	SP alignment fault	SP not 16-byte aligned (ch. 09)
`0x2C`	Floating-point exception	A trapped FP condition
`0x2F`	SError	Imprecise external abort — see below
`0x30` – `0x35`	Breakpoint, software step, watchpoint	Debug

EC	Meaning	Usual cause
`0x3C`	`BRK` instruction	A software breakpoint — `BUG()` in Linux

> **THE PAIRING RULE**
>
> Notice `0x20` / `0x21` and `0x24` / `0x25`. The lower value means the exception came from a *lower* exception level; the higher means it came from the *same* level. So `EC = 0x24` in a Linux oops means userspace faulted; `0x25` means the kernel did. That single bit tells you immediately whether you are looking at a user bug or a kernel bug.

Data abort ISS

```
EC = 0x24 or 0x25. The ISS layout:

 24 23 22 21 20 19    16 15 14 13 12 11 10  9  8  7  6 5      0
┌──┬──┬──┬──┬──┬────────┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬────────┐
│ISV│SAS │SSE│SRT      │SF│AR│VNCR│  │ │EA│CM│S1PTW│WnR│ DFSC │
└──┴──┴──┴──┴──┴────────┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴────────┘

   DFSC  [5:0]   Fault Status Code — THE key field
   WnR   [6]     0 = read, 1 = write
   S1PTW [7]     the fault occurred on a stage-1 PAGE TABLE WALK
   CM    [8]     the fault was on a cache maintenance instruction
   EA    [9]     external abort type
   FnV   [10]    FAR is NOT valid
   SET   [12:11] synchronous error type
   AR    [14]    the access was acquire/release
   SF    [15]    64-bit register width
   SRT   [20:16] the destination REGISTER number
   SSE   [21]    sign-extended
   SAS   [23:22] access size: 0=byte 1=half 2=word 3=dword
   ISV   [24]    SAS/SRT/SF/SSE are VALID
```

Fault status codes

DFSC	Meaning	What to check
`0b0000LL`	Address size fault, level LL	Output address exceeds the configured PA size (`TCR.IPS`)
`0b0001LL`	**Translation fault, level LL**	**Not mapped.** Null pointer, missing `ioremap`, or a wrong `TTBR`
`0b0010LL`	Access flag fault, level LL	`AF` clear in the descriptor (ch. 14)
`0b0011LL`	**Permission fault, level LL**	**Mapped, but not for this.** AP bits, XN, or PAN
`0b010000`	Synchronous external abort	**The bus refused it.** Clock, power, or a firewall (ch. 25)
`0b0101LL`	External abort on a table walk	The page table memory itself is unreachable
`0b011000`	Synchronous parity/ECC error	Memory corruption detected by hardware
`0b100001`	Alignment fault	Unaligned Device access, or `SCTLR.A`
`0b110000`	TLB conflict abort	Break-before-make violation (ch. 15)
`0b111101` / `0b111110`	Section/page domain fault	AArch32 only

THE FOUR-WAY TRIAGE

- **Translation fault** — the address is simply not mapped. Null pointer, use-after-free, a missing `ioremap`, or a stale pointer.
- **Permission fault** — it *is* mapped, just not for this operation. Writing to read-only, executing no-execute, EL0 touching an EL1 page, or PAN blocking a kernel access to user memory.
- **Access flag fault** — the `AF` bit was clear. Normal in an OS that uses AF for page aging; a bug in hand-written table code.
- **External abort** — translation *succeeded* and the transaction went out on the bus, and something out there refused it. This is a hardware or platform problem, not a page table one: an unclocked peripheral, a powered-down domain, a wrong address, or an access-control block denying you.

That one distinction — page tables versus the bus — sends you to completely different parts of the system, and it is why reading DFSC first saves hours.

A decoder worth keeping

```
drop this into any bare-metal or kernel fault handler
```

```
static const char *dfsc_name(unsigned dfsc)
{
        switch (dfsc & 0x3C) {
        case 0x00: return "address size fault";
        case 0x04: return "TRANSLATION fault (not mapped)";
        case 0x08: return "access flag fault (AF clear)";
        case 0x0C: return "PERMISSION fault (mapped, wrong perms)";
        }
        switch (dfsc) {
        case 0x10: return "EXTERNAL ABORT (bus refused it)";
        case 0x18: return "parity/ECC error";
        case 0x21: return "ALIGNMENT fault";
        case 0x30: return "TLB conflict (break-before-make bug)";
        default:   return "unknown";
        }
}

void dump_fault(void)
{
        uint64_t esr, far, elr;
        asm volatile("mrs %0, esr_el1" : "=r"(esr));
        asm volatile("mrs %0, far_el1" : "=r"(far));
        asm volatile("mrs %0, elr_el1" : "=r"(elr));

        unsigned ec  = (esr >> 26) & 0x3F;
        unsigned iss =  esr & 0x1FFFFFF;

        printf("ESR=%016llx  EC=0x%02x  FAR=%016llx  ELR=%016llx\n",
               esr, ec, far, elr);

        switch (ec) {
        case 0x15:
                printf("  SVC #%u — a system call\n", iss & 0xFFFF);
                break;

        case 0x20: case 0x21:
                printf("  INSTRUCTION abort at %s EL: %s, level %u\n",
                       ec == 0x20 ? "lower" : "same",
                       dfsc_name(iss & 0x3F), (iss & 3));
                printf("  → fetching from %016llx failed\n", far);
                break;

        case 0x24: case 0x25: {
                unsigned dfsc = iss & 0x3F;
                printf("  DATA abort at %s EL on %s of %016llx\n",
                       ec == 0x24 ? "lower" : "same",
                       (iss & (1u << 6)) ? "WRITE" : "READ", far);
                printf("  %s, level %u\n", dfsc_name(dfsc), dfsc & 3);

                if (iss & (1u << 7))
                        printf("  S1PTW: the PAGE TABLE WALK itself faulted\n");
                if (iss & (1u << 10))
                        printf("  FnV: FAR is NOT valid\n");
                if (iss & (1u << 24))
                        printf("  size=%u bytes, dest reg x%u\n",
                               1u << ((iss >> 22) & 3), (iss >> 16) & 0x1F);
                break;
        }

        case 0x18:
                printf("  trapped MSR/MRS — a higher EL is watching\n");
                break;
```

```
        case 0x26:
                printf("  SP ALIGNMENT fault — SP was not 16-byte aligned\n");
                break;

        case 0x2F:
                printf("  SError — IMPRECISE. ELR is NOT the cause.\n");
                break;

        default:
                printf("  EC 0x%02x — look it up in the ARM ARM, D17.2.37\n", ec);
        }
}
```

Worked examples

```
/* ESR = 0x96000004, FAR = 0x0000000000000000 */
   EC   = 0x96000004 >> 26 = 0x25   data abort, SAME EL (kernel)
   ISS  = 0x000004
   DFSC = 0x04 = 0b000100          translation fault, level 0
   WnR  = 0                        a READ
   FAR  = 0                        address zero
   → A KERNEL NULL POINTER DEREFERENCE, on a read.
     Level 0 because nothing at all is mapped at address 0.

/* ESR = 0x9200004F, FAR = 0x0000ffff8a3c1000 */
   EC   = 0x24                     data abort, LOWER EL (user)
   DFSC = 0x0F = 0b001111          permission fault, level 3
   WnR  = 1                        a WRITE
   → Userspace wrote to a read-only page. A .rodata write, or a
     copy-on-write page that has not yet been copied (OS ch. 13).

/* ESR = 0x96000010, FAR = 0x0000000040001000 */
   EC   = 0x25                     kernel
   DFSC = 0x10                     EXTERNAL ABORT
   → Translation SUCCEEDED. The bus refused it. Check: is the
     peripheral clocked? Powered? Is an XPU denying access?
     This is NOT a page table problem.

/* ESR = 0x86000007, FAR = 0x0000000000401234 */
   EC   = 0x21                     INSTRUCTION abort, same EL
   DFSC = 0x07 = 0b000111          translation fault, level 3
   → Tried to FETCH from an unmapped page. A corrupted function
     pointer, or a jump into freed memory.

/* ESR = 0x56000000 */
   EC   = 0x15                     SVC from AArch64
   ISS  = 0                        svc #0
   → A normal system call. Not an error at all.
```

SError: the hard one

```
/* EC = 0x2F. The ISS layout is different again: */
   IDS  [24]     IMPLEMENTATION DEFINED syndrome
   AET  [12:10]  Asynchronous Error Type
                   0 = uncontainable
                   1 = unrecoverable
                   2 = restartable
                   3 = recoverable
                   6 = corrected
   EA   [9]      external abort
   DFSC [5:0]    0x00 = uncategorised, 0x11 = asynchronous SError

/* The problem: an SError is reported at an arbitrary later
   instruction. ELR points at whatever was executing when the
   error was finally taken — which has NOTHING to do with the
   transaction that failed.

   Typical causes:
     · a POSTED WRITE rejected by an access-control block long
       after the store retired (chapter 25, security ch. 20)
     · an ECC error discovered on a writeback
     · a device signalling an error asynchronously

   How to narrow it down:
     1. ESB (Error Synchronization Barrier) at known points
        confines an SError to a bounded region, so you can
        bisect where it originates:
              esb
        Place them around suspect code and see which region
        produces it.
     2. The RAS extension adds ERR<n>STATUS records describing
        exactly which node reported the error.
     3. Vendor-specific error registers in the interconnect —
        an XPU usually LATCHES the offending address and master
        ID even when the CPU cannot attribute it.

   Linux prints these when it can:
        dmesg | grep -i 'SError\|RAS\|EDAC'                */
```

Reading a Linux oops

```
Unable to handle kernel NULL pointer dereference at virtual address 0000000000000010
Mem abort info:
  ESR = 0x0000000096000006
  EC = 0x25: DABT (current EL), IL = 32 bits
  SET = 0, FnV = 0
  EA = 0, S1PTW = 0
  FSC = 0x06: level 2 translation fault
Data abort info:
  ISV = 0, ISS = 0x00000006
  CM = 0, WnR = 0
CPU: 2 PID: 1834 Comm: cat Tainted: G           O
pstate: 60400009 (nZCv daif +PAN -UAO)
pc : my_read+0x48/0x120 [mydriver]
lr : vfs_read+0xb4/0x1f0
sp : ffff800009abc000
Call trace:
 my_read+0x48/0x120 [mydriver]
 vfs_read+0xb4/0x1f0
 ksys_read+0x74/0x110
 __arm64_sys_read+0x24/0x30
 el0_svc_common+0x88/0x1a0

/* Reading it in order:
     EC 0x25   → the KERNEL faulted, not userspace
     FSC 0x06  → translation fault at level 2
     WnR 0     → a READ
     FAR 0x10  → address 16 — a NULL pointer plus a 16-byte
                 field offset. So: a null struct pointer, and
                 the field is at offset 0x10.
     pstate +PAN → PAN was on, so this was NOT an attempt to
                 read user memory
     call trace → arrived via a read() syscall from `cat`

   Conclusion: my_read() dereferenced a null pointer and read the
   member at offset 16. Find it: */

scripts/faddr2line vmlinux my_read+0x48
pahole -C my_device mydriver.ko | grep -A2 'offset:.*16'
```

Stage-2 faults

```
/* Taken to EL2, with ESR_EL2. Two extra things to read: */

   HPFAR_EL2      the faulting IPA:  (HPFAR & ~0xF) << 8
   FAR_EL2        the guest VA, when available
   ESR_EL2.S1PTW  set → the GUEST'S PAGE TABLES are in an
                  unmapped IPA. The guest did nothing wrong; the
                  hypervisor has not mapped the memory holding
                  its tables.

/* And ISV/SAS/SRT are what make MMIO emulation possible without
   decoding the instruction (chapter 16):                    */

        if (esr & ESR_ISV) {
                int size = 1 << ((esr >> 22) & 3);
                int reg  = (esr >> 16) & 0x1F;
                bool wr  = esr & (1 << 6);
                emulate(ipa, reg, size, wr);
        } else {
                decode_instruction_slowly();    /* rare */
        }
```

How it shows up

ESR pattern	Diagnosis
EC 0x25, DFSC 0b0001xx, FAR near 0	Kernel null pointer dereference
EC 0x24, DFSC 0b0001xx	Userspace segfault — normal, handled by the OS
EC 0x25, DFSC 0b0011xx, WnR=1	Kernel wrote to read-only memory
EC 0x25, DFSC 0b0011xx, PAN set	Kernel dereferenced a user pointer directly
DFSC 0b010000	External abort — clock, power, or a bus firewall
S1PTW set	The page tables themselves are unreachable
EC 0x21, DFSC translation	Jumped to unmapped memory — corrupted function pointer
EC 0x26	SP alignment — hand-written assembly, or stack corruption
EC 0x22	PC alignment — a corrupted branch target
DFSC 0b110000	TLB conflict — break-before-make violation
EC 0x2F	SError — the reported PC is meaningless; use ESB to bisect
EC 0x18	A trapped system register — a hypervisor is watching
EC 0x0E	Illegal execution state — usually a bad `SPSR` on `ERET`

Check yourself

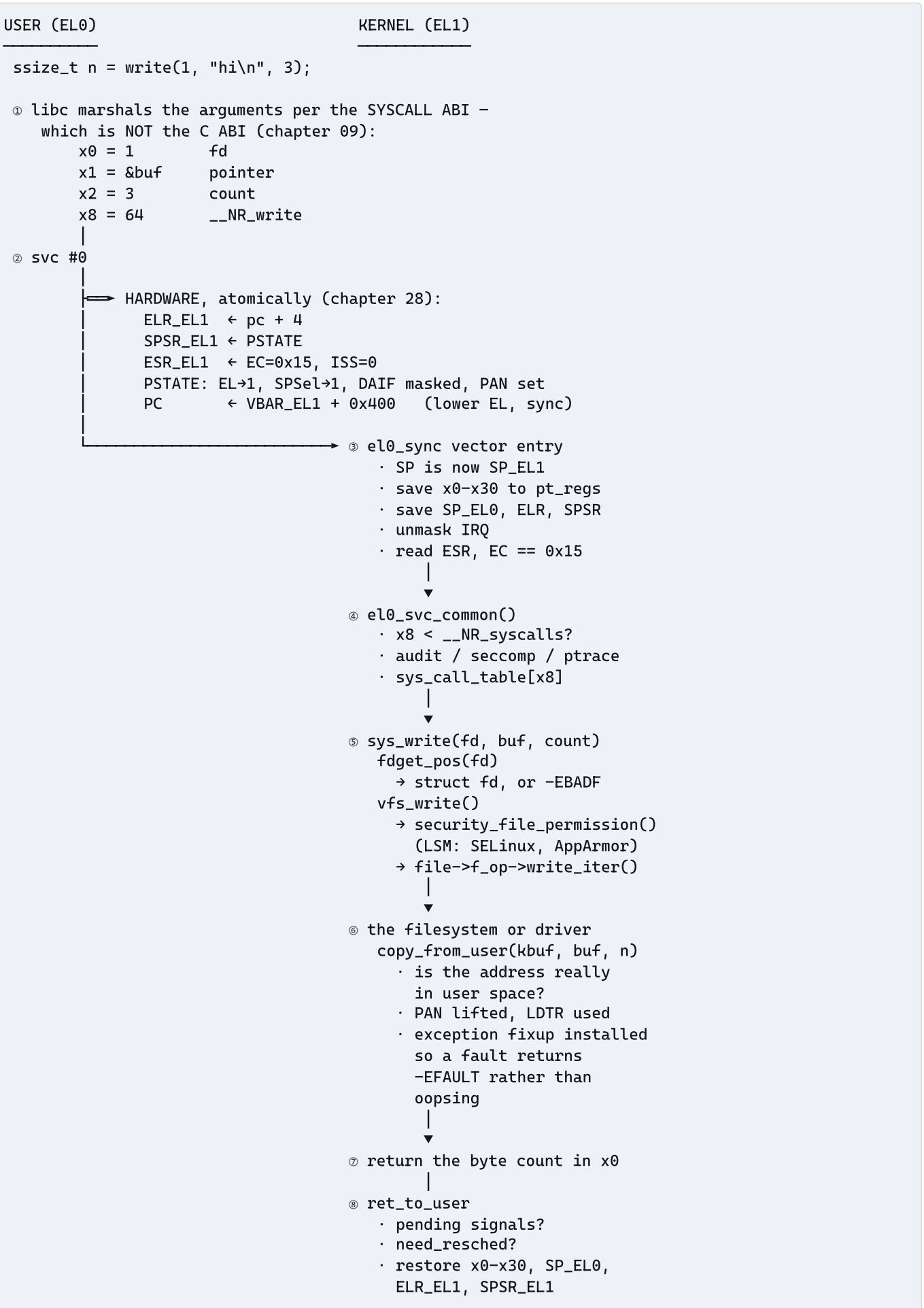
- ✓ Give the three fields of ESR and say which one to read first.
- ✓ Explain the difference between EC 0x24 and 0x25, and why it matters immediately.
- ✓ Give the four main DFSC categories and what each sends you to investigate.
- ✓ Explain why an external abort is not a page table problem.
- ✓ Decode `ESR = 0x96000004, FAR = 0` into plain English.

- ✓ Explain what `S1PTW` means and what it implies about the tables.
- ✓ Explain why SError cannot be attributed from `ELR`, and two ways to narrow it down.
- ✓ Explain how `ISV` / `SAS` / `SRT` enable MMIO emulation.

A system call, end to end

Everything in this track converging on one path. Follow `write(fd, buf, len)` from a C call to the return value, naming every register, every check and every layer — and the whole architecture stops being a collection of separate topics.

The complete path



```
                                          |
        ┌<──────────────────────── ⑨ eret
        |
        ▼
 ⑩ libc: is x0 negative and >= -4095?
        yes → errno = -x0; return -1
        no  → return x0
        |
        ▼
 your code continues
```

Step 1: the syscall ABI

```
/* The syscall ABI differs from the C ABI in two ways that matter: */

   C ABI:       arguments in x0–x7, return in x0
   syscall ABI: arguments in x0–x5 ONLY, syscall number in x8,
                return in x0

/* Why x8 rather than x7? Because it keeps the argument registers
   contiguous and leaves x8 — the indirect result register, which
   a syscall never needs — free for the number.

   Why only six arguments? Because no Linux syscall needs more.
   Those that would (mmap, clone3) pass a POINTER TO A STRUCT
   instead, which is also how they stay extensible (OS ch. 03). */

/* A raw syscall in assembly: */
        mov     x0, #1                  /* fd = stdout */
        adr     x1, msg
        mov     x2, #3
        mov     x8, #64                 /* __NR_write  */
        svc     #0
        /* x0 now holds the result, or a negative errno */

/* From C, without libc: */
static inline long syscall3(long n, long a, long b, long c)
{
        register long x8 asm("x8") = n;
        register long x0 asm("x0") = a;
        register long x1 asm("x1") = b;
        register long x2 asm("x2") = c;
        asm volatile("svc #0"
                     : "+r"(x0)
                     : "r"(x8), "r"(x1), "r"(x2)
                     : "memory", "cc");
        return x0;
}
/* Note "memory": the kernel may modify anything the pointers
   reach, so the compiler must not cache values across it
   (C chapter 18).                                          */
```

Step 4: dispatch

```
/* The syscall table is an array of function pointers — exactly
   the dispatch table pattern from C chapter 12.            */

const syscall_fn_t sys_call_table[__NR_syscalls] = {
        [0 ... __NR_syscalls - 1] = __arm64_sys_ni_syscall,
#include <asm/unistd.h>
};

/* The bounds check is not optional, and on a speculating core it
   must survive speculation (CA chapter 12):                */

        if (scno < __NR_syscalls) {
                scno = array_index_nospec(scno, __NR_syscalls);
                /* → CMP + CSEL + CSDB: a MASK, not a branch, so
                     speculation cannot bypass it            */
                syscall_fn = sys_call_table[scno];
                ret = syscall_fn(regs);
        } else {
                ret = do_ni_syscall(regs, scno);
        }

/* Before dispatch, several hooks may intervene:
     ptrace       a debugger can inspect or change the arguments,
                  or skip the call entirely
     seccomp      a BPF filter can allow, deny, kill or trap
     audit        logging
   Each costs cycles, which is why a heavily-sandboxed process
   has measurably slower syscalls.                          */
```

Step 6: crossing the boundary

> **EVERY BYTE FROM USER SPACE IS HOSTILE**
>
> A system call argument is attacker-controlled input arriving at the highest privilege level. The kernel must assume the caller is malicious, not merely buggy — a local user exploiting a validation gap gets kernel code execution.

```
/* NEVER dereference a __user pointer directly. */

long sys_example(void __user *ubuf, size_t len)
{
        char kbuf[256];

        if (len > sizeof kbuf)
                return -EINVAL;                  /* bound FIRST */
        if (copy_from_user(kbuf, ubuf, len))
                return -EFAULT;                  /* bytes NOT copied */
        ...
}

/* copy_from_user does three things you cannot skip:

   1. RANGE CHECK — is the address really below the user/kernel
      boundary? On arm64 this is a single comparison against
      TASK_SIZE, made speculation-safe.

   2. EXCEPTION FIXUP — an entry in __ex_table maps the faulting
      instruction to a recovery address. If the user page is
      unmapped, the data abort handler finds the fixup and
      returns -EFAULT instead of oopsing. This is what makes
      passing a bad pointer an ERROR rather than a crash.

   3. PAN / LDTR — with PAN enabled (chapter 05), ordinary kernel
      loads from user pages FAULT. The copy routines either lift
      PAN briefly or use the unprivileged LDTR/STTR instructions
      (chapter 08), which behave as EL0 accesses. Either way, a
      pointer aimed at KERNEL memory fails the permission check
      rather than succeeding.                              */

/* The five validation rules (OS chapter 03):
     1. bound every length BEFORE using it
     2. COPY before validating — never validate in place (TOCTOU)
     3. check every index and enum against its range
     4. zero output structures fully — padding leaks stack data
     5. reject unknown flags rather than ignoring them        */
```

Steps 8–10: returning

```
/* ret_to_user is not just a restore. Two things must be checked
   at the boundary, because this is the only safe point to do
   them:                                                        */

   TIF_SIGPENDING     a signal is queued → build a signal frame
                      on the USER stack, and return into the
                      handler instead of the original code
   TIF_NEED_RESCHED   a higher-priority task became runnable →
                      call schedule() before returning

/* This is why signals are delivered "on return to userspace" and
   not at arbitrary points — the kernel needs a known-good state
   to rewrite the return context.                              */

/* The errno convention, at step 10: */
        if ((unsigned long)ret >= (unsigned long)-4095) {
                errno = -ret;
                return -1;
        }
        return ret;

/* The kernel returns a NEGATIVE ERRNO directly (-EINVAL = -22),
   never sets a global. libc translates. This is strictly better
   than errno (C chapter 16): the error travels in the return
   value, so it cannot be clobbered or forgotten.

   The -4095 boundary is why no syscall may return a legitimate
   value in the range -1 to -4095 — mmap returning a high
   address is why it is checked this way rather than just
   testing for negative.                                       */
```

Avoiding the syscall entirely

```
/* Some "syscalls" need no privilege at all. The vDSO is a small
   shared object the kernel maps into every process, containing
   implementations that run in USER MODE.                      */

clock_gettime(CLOCK_MONOTONIC, &ts);
/*  · reads a kernel-maintained page of timekeeping data
    · reads CNTVCT_EL0 — the counter, readable from EL0
      (chapter 32)
    · does the arithmetic
    · returns. NO exception, no privilege change.
    ~25 ns instead of ~300 ns.                                */

cat /proc/self/maps | grep vdso
LD_SHOW_AUXV=1 /bin/true | grep VDSO

/* Also served by the vDSO: gettimeofday, clock_getres, time,
   getcpu, and on some architectures rt_sigreturn.

/* The general principle: the fastest system call is the one you
   do not make. Hence io_uring (shared rings, often zero
   syscalls in steady state), mmap instead of read, and eBPF
   (run the logic kernel-side). OS chapter 03.                */
```

Where the time goes

```
   svc → first kernel instruction        ~15–30 cycles
   save 31 registers + 3 sysregs         ~20–40 cycles
   seccomp / audit / ptrace checks       0 to hundreds
   table lookup + indirect call          ~10 cycles
   ── the actual work ──
   signal / resched checks               ~10 cycles
   restore + eret                        ~30–50 cycles

   Plus, on affected systems:
     KPTI                                two address-space
                                         switches, +100–300 cycles
     Spectre-BHB mitigation              a flush sequence in the
                                         vector, +tens of cycles

   A "null" syscall (getppid) measures roughly:
      no mitigations       ~60–100 ns
      with KPTI + BHB      ~150–400 ns

   Measure your own:
      perf bench syscall basic
      strace -c ./program           ← count them first
```

HVC and SMC follow the same shape

	`SVC`	`HVC`	`SMC`
Target	EL1	EL2	EL3
ESR EC	0x15	0x16	0x17
Vector	`VBAR_EL1` + 0x400	`VBAR_EL2` + 0x400	`VBAR_EL3` + 0x400
Number in	`x8`	The immediate, or `x0`	`x0` (SMCCC)
Used for	System calls	Hypercalls, KVM	PSCI, TEE, SiP services
Disabled by	—	`HCR_EL2.HCD`	`SCR_EL3.SMD`

```
/* The structure is identical: a deliberate synchronous exception
   to a higher level, arguments in registers, a number selecting
   the service, and ERET to return. Once you understand SVC, you
   understand all three.

   The differences are in what the handler does — and for SMC,
   the monitor must also save and restore the ENTIRE state of one
   world before switching (security track).               */

/* A PSCI call, for example — this is how a kernel starts a
   secondary CPU (chapter 33):                             */
        ldr     x0, =0xC4000003         /* PSCI_CPU_ON, SMC64 */
        mov     x1, target_mpidr
        ldr     x2, =secondary_entry
        mov     x3, #0
        smc     #0
        /* x0 = 0 on success, or a negative PSCI error */
```

Watching it happen

```
# count and time every syscall — ALWAYS do this first
strace -c ./program
#   % time     seconds  usecs/call     calls    errors syscall
#   ------ ----------- ----------- --------- --------- --------
#    89.12    0.451203           0   1204811           read
#     → 1.2 MILLION reads. That is the bug, whatever else is true.

strace -f -e trace=openat,read,write ./program
strace -p 1234                  # attach

# lower overhead, system-wide
perf trace ./program
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
        @[comm, args->id] = count(); }'

# the entry path itself
echo 1 > /sys/kernel/debug/tracing/events/raw_syscalls/enable
cat /sys/kernel/debug/tracing/trace_pipe

# the table
ausyscall --dump | head -20
cat /usr/include/asm-generic/unistd.h | grep __NR_write

# see the vDSO in action — no syscall for clock_gettime
strace -e trace=clock_gettime ./prog     # produces nothing
```

> **`STRACE -C` BEFORE ANYTHING ELSE**
>
> When a program is slow with low CPU usage, this one command usually finds it: a million one-byte reads, thousands of failed `stat()` calls probing for a file, or a `poll()` loop with a zero timeout. All three are extremely common and all three are obvious in the summary. Reach for a profiler only after this comes back clean.

How it shows up

Symptom	Cause
Slow program, low CPU	Syscall-bound — check `strace -c`
Byte-at-a-time I/O 100× slower	One syscall per byte; buffer it
Performance dropped after a kernel update	KPTI or Spectre mitigations on the entry path
Kernel oops on a pointer from userspace	Direct dereference instead of `copy_from_user`
Vulnerability from a caller-supplied length	Length used before being bounded
Exploit changes a value after validation	TOCTOU — validated in place instead of after copying
Kernel stack bytes appear in user output	Struct padding not zeroed
Syscall returns a huge positive number instead of an error	Return value not checked against the -4095 boundary
Sandboxed process is measurably slower	seccomp filter evaluated on every call
`clock_gettime` is suspiciously cheap	Served by the vDSO — no exception at all

Check yourself

- ✓ Give the syscall ABI registers and two ways it differs from the C ABI.
- ✓ List the six things hardware does at `svc #0`.
- ✓ Explain why the syscall number bounds check must be speculation-safe.
- ✓ Give the three things `copy_from_user` does that a `memcpy` does not.
- ✓ Explain how PAN and `LDTR` enforce the boundary in hardware.
- ✓ Say why signals are delivered on return to userspace rather than at arbitrary points.
- ✓ Explain the negative-errno convention and the -4095 boundary.
- ✓ Explain what the vDSO is and why `clock_gettime` needs no exception.
- ✓ Give the three deliberate-exception instructions and what distinguishes them.

Traps and virtualisation

A higher exception level can make specific operations at a lower level raise an exception instead of executing. That single mechanism — configurable trapping — is the whole basis of virtualisation, lazy state management and several security features.

The mechanism

TRAP, EMULATE, RESUME

Set a bit in a control register at EL2 or EL3, and a chosen operation at a lower level stops executing and becomes a synchronous exception instead. The handler reads `ESR` to learn exactly what was attempted, does something equivalent, advances `ELR` past the instruction, and returns. The lower level never knows.

This is how a guest can believe it owns hardware it does not have, and how a kernel can defer expensive state management until it is actually needed.

The trap control registers

Register	Owner	Notable bits
`HCR_EL2`	Hypervisor	`VM` enable stage 2 · `TGE` route all EL0 exceptions to EL2 · `IMO/FMO/AMO` virtualise IRQ/FIQ/SError · `TVM` trap virtual-memory register writes · `TWI/TWE` trap `WFI` / `WFE` · `TID0–3` trap ID register reads · `E2H` VHE
`CPTR_EL2` / `CPACR_EL1`	Hypervisor / kernel	`FPEN`, `ZEN`, `SMEN` — trap FP, SVE and SME access
`MDCR_EL2` / `MDCR_EL3`	Hypervisor / monitor	Trap debug and performance-monitor access
`SCR_EL3`	Secure monitor	`NS` security state · `IRQ/FIQ/EA` routing · `SMD` disable `SMC` · `HCE` enable `HVC`
`HFGRTR_EL2` / `HFGWTR_EL2`	Hypervisor	Fine-grained traps (ARMv8.6): individual registers rather than whole groups

Trapping a system register

```
/* Enable the trap at EL2: */
        mrs     x0, hcr_el2
        orr     x0, x0, #(1 << 26)      /* TVM: trap virtual memory
                                           control register writes */
        msr     hcr_el2, x0
        isb

/* Now a guest at EL1 executing this: */
        msr     ttbr0_el1, x5

/* ...takes a synchronous exception to EL2 with ESR_EL2.EC = 0x18
   (trapped MSR/MRS). The ISS encodes exactly which register and
   which general register were involved:                    */

  ESR_EL2.ISS for EC = 0x18:
     [24:22] Op0      [20:17] Op2      [16:14] Op1
     [13:10] CRn      [9:5]   Rt   <- the general register
     [4:1]   CRm      [0]     Direction: 0 = write (MSR),
                                         1 = read  (MRS)

/* The handler: */
static void handle_sysreg_trap(struct vcpu *v)
{
        u64 esr = read_sysreg(esr_el2);
        u32 iss = esr & 0x1FFFFFF;

        int rt        = (iss >> 5) & 0x1F;
        bool is_read  = iss & 1;
        u32 reg_id    = iss & ~0x3E;    /* the encoding, minus Rt */

        if (is_read)
                v->regs[rt] = emulate_sysreg_read(v, reg_id);
        else
                emulate_sysreg_write(v, reg_id, v->regs[rt]);

        /* ADVANCE past the instruction, or we re-trap forever */
        write_sysreg(read_sysreg(elr_el2) + 4, elr_el2);
}

/* That last step is the single most common bug in this area: the
   guest appears frozen on one instruction, forever. Chapter 28. */
```

Trapping MMIO

```
Device emulation does not use a trap BIT — it uses an absent
stage-2 mapping (chapter 16).

   guest writes to what it believes is a UART register
        |
   ① stage 1 (guest's own tables): VA → IPA. SUCCEEDS.
   ② stage 2 (hypervisor's): that IPA is deliberately UNMAPPED
   ③ stage-2 data abort → EL2
        |
   ④ the hypervisor reads:
        HPFAR_EL2    the faulting IPA → which device register
        ESR_EL2.ISV  are the details valid?
             SAS     access size
             SRT     which guest register
             WnR     read or write
        |
   ⑤ emulate the device, update the model
   ⑥ ELR_EL2 += 4
   ⑦ ERET

  ISV is what makes this fast: without it the hypervisor would
  have to FETCH and DECODE the guest's instruction to work out
  what it was doing. Some access types (load/store pair, some
  SIMD forms) do not set ISV, and those take the slow path.

  COST: a full world switch per register access. A guest driver
  doing ten register writes to send a packet takes ten exits.
  This is exactly why virtio exists (OS chapter 20).
```

Lazy FP and SVE state

```
/* FP/SIMD state is large — 32 x 128-bit V registers, or far more
   with SVE. Saving it on every context switch would dominate the
   cost, and most threads never touch it.                    */

/* The optimisation:
     1. on switch, DISABLE FP access for the incoming thread
     2. if it never uses FP, nothing is saved or restored — free
     3. if it executes an FP instruction, a TRAP fires        */

        mrs     x0, cpacr_el1
        bic     x0, x0, #(3 << 20)      /* FPEN = 0: trap FP     */
        msr     cpacr_el1, x0
        isb

/* The trap handler (ESR.EC = 0x07): */
static void fpsimd_trap(void)
{
        save_fpsimd_state(previous_owner);
        restore_fpsimd_state(current);
        enable_fpsimd();                /* CPACR_EL1.FPEN = 3    */
        /* do NOT advance ELR — we want the instruction RETRIED */
}

/* Note the contrast with the sysreg trap: here we do NOT advance
   ELR, because we want the original instruction to execute now
   that FP is enabled. Whether to advance depends entirely on
   whether you EMULATED the instruction or ENABLED it.      */

/* Which is also why kernel code may not use FP freely: its state
   is not saved on kernel entry. Code that needs it must bracket
   it explicitly (C chapter 16):                             */
        kernel_neon_begin();
        /* SIMD crypto or checksum */
        kernel_neon_end();
```

Trapping WFI and WFE

```
/* A guest executing WFI would halt the PHYSICAL core, which the
   hypervisor cannot allow — it wants to schedule another vCPU.  */

        HCR_EL2.TWI = 1     /* trap WFI → ESR.EC = 0x01 */
        HCR_EL2.TWE = 1     /* trap WFE                 */

/* The handler treats it as "this vCPU is idle": */
static void handle_wfx_trap(struct vcpu *v, bool is_wfe)
{
        if (is_wfe)
                kvm_vcpu_on_spin(v);      /* likely spinning on a
                                             lock — yield         */
        else
                kvm_vcpu_block(v);        /* genuinely idle — block
                                             until an interrupt    */
        advance_elr(v);
}

/* The WFE case is interesting: a guest spinning on a lock held by
   a DESCHEDULED vCPU wastes its entire timeslice. Trapping WFE
   lets the hypervisor detect that and yield to the lock holder —
   the "lock holder preemption" problem, and one of the reasons
   paravirtualised spinlocks exist.                        */
```

Trapping identification

```
/* HCR_EL2.TID0–TID3 trap reads of the ID registers, so the
   hypervisor can present a guest with a DIFFERENT feature set
   than the hardware actually has.                          */

/* Why this matters:
     · LIVE MIGRATION. A VM migrating between hosts must see the
       same features throughout, so the hypervisor advertises the
       intersection of the cluster's capabilities.
     · big.LITTLE. Cores in one system genuinely differ. A guest
       migrating between them would see the feature set change
       under it, which no OS expects. The hypervisor normalises.
     · SECURITY. Hiding a feature with a known erratum.

   The cost: every ID register read becomes an exit. Guests read
   them mostly at boot, so this is acceptable — but a guest that
   polls one in a loop will be very slow.                  */
```

Fine-grained traps

```
/* Original trap bits are coarse: HCR_EL2.TVM traps ALL
   virtual-memory register writes, even ones the hypervisor does
   not care about. Every unnecessary trap is a world switch.

   ARMv8.6 adds FINE-GRAINED TRAPS: individual bits for
   individual registers.                                    */

        HFGRTR_EL2      fine-grained READ traps
        HFGWTR_EL2      fine-grained WRITE traps
        HFGITR_EL2      fine-grained INSTRUCTION traps
        HDFGRTR_EL2 / HDFGWTR_EL2   debug registers

/* So a hypervisor can trap exactly TTBR0_EL1 writes without
   trapping TCR_EL1, VBAR_EL1 and the rest. Measurably fewer
   exits on a memory-management-heavy guest.

   The general principle: every trap is a cost, so the
   architecture keeps adding ways to trap LESS.            */
```

Nested virtualisation

```
Running a hypervisor INSIDE a guest. ARMv8.3 (NV) and 8.4 (NV2)
make it practical.

   L0 hypervisor at EL2 (the real one)
        |
   L1 hypervisor - believes it is at EL2, actually runs at EL1
        |
   L2 guest at EL1

  The problem: L1 executes EL2 instructions and accesses EL2
  registers, but it is at EL1. Without help, EVERY one of those
  traps to L0, which must emulate it - including L1's own stage-2
  table management, which L0 must SHADOW into real stage-2 tables.
  Enormously slow.

  NV (ARMv8.3): HCR_EL2.NV makes EL2 register accesses from EL1
  trap cleanly and defines the semantics, so emulation is
  correct rather than ad hoc.

  NV2 (ARMv8.4): HCR_EL2.NV2 REDIRECTS L1's EL2 register accesses
  to MEMORY - a VNCR page - instead of trapping. L1 writes what
  it thinks is VTTBR_EL2 and it lands in a memory structure L0
  reads later. No exit at all for the common case.

  Used for: nested containers-in-VMs, developing hypervisors
  inside a VM, and confidential computing stacks.
```

TGE and the host

```
/* HCR_EL2.TGE — "Trap General Exceptions" — routes ALL EL0
   exceptions to EL2 instead of EL1.                        */

/* Why: when a HOST kernel runs at EL2 with VHE (chapter 26), its
   userspace runs at EL0 and its exceptions must reach EL2, not
   the (nonexistent) EL1. TGE does that.

   TGE also implicitly:
     · disables stage-2 translation for EL1&0
     · makes EL1 accesses behave as if no guest is present

   So a VHE host runs with TGE=1 normally, and CLEARS it when
   entering a guest. That one bit is the boundary between "host
   context" and "guest context", and toggling it is a large part
   of what a VM entry does.                                 */

/* You can see the transition in KVM's world-switch code:
   __activate_traps / __deactivate_traps in
   arch/arm64/kvm/hyp/                                      */
```

The cost of a trap

```
   guest instruction
        │  exception entry to EL2          ~20–50 cycles
        │  save guest state (registers,
        │    system registers)             ~100–500 cycles
        │  decode ESR, dispatch            ~20 cycles
        │  EMULATE                         varies
        │  restore guest state             ~100–500 cycles
        │  ERET                            ~20 cycles
        ▼
   ~500–2000 cycles per exit, and more without VHE

  Which is why the whole design direction is TRAP LESS:
     stage-2 translation      instead of shadow page tables
     GIC list registers       instead of trapping every EOI
     virtio                   instead of emulating a real NIC
     device passthrough       instead of any emulation at all
     fine-grained traps       instead of coarse ones
     NV2 memory redirection   instead of nested traps

  MEASURING exits is the first step in any VM performance work:
     perf stat -e kvm:kvm_exit -a sleep 10
     perf record -e kvm:kvm_exit -a && perf report
        → group by exit reason. MMIO dominating means you should
          be using virtio or passthrough.
```

Debugging traps

```
# which exits, and how many?
perf stat -e 'kvm:*' -a sleep 10
perf record -e kvm:kvm_exit -a && perf report --sort=dso,symbol

# KVM's own statistics
ls /sys/kernel/debug/kvm/*/
cat /sys/kernel/debug/kvm/*/vcpu0/*
#   mmio_exits, wfi_exits, wfe_exits, hvc_exits, sys64_exits...

# trace them
echo 1 > /sys/kernel/debug/tracing/events/kvm/enable
cat /sys/kernel/debug/tracing/trace_pipe
#   kvm_exit: ESR_EC: 0x24 (DABT_LOW) ... ipa 0x9000000

# from inside a guest, confirm what it thinks it has
cat /proc/cpuinfo | grep Features
#   compare with the host's — differences mean ID traps are
#   normalising the feature set

# in a hypervisor you are writing: log every unhandled EC once
        pr_warn_once("unhandled trap EC=0x%02x ISS=0x%07x\n", ec, iss);
```

THE TWO QUESTIONS FOR ANY TRAP HANDLER

Did I emulate the instruction, or did I enable something so it can run? If you emulated it, advance `ELR`. If you enabled something, do *not* — the instruction must be retried.

Is this trap on a hot path? If a workload takes millions of a particular exit, the answer is architectural: use a mechanism that does not trap at all. Emulating faster is rarely the right fix.

Where this goes next

Traps are the mechanism; a hypervisor is what you build out of them. The virtualisation track picks up here — HCR_EL2 bit by bit, stage-2 translation, the vGIC, virtio and KVM's implementation of all of it, ending with a working hypervisor in two hundred lines.

How it shows up

Symptom	Cause
Guest frozen on one instruction	`ELR` not advanced after emulation
FP instruction faults repeatedly	`ELR` advanced when it should have been retried
Guest I/O extremely slow	MMIO emulation — one exit per register access
Guest hangs on a system register write	Trap configured but not handled
Guest sees different CPU features than the host	ID register traps normalising for migration
Guest spinlocks perform terribly	Lock holder preemption; needs WFE trapping and yield
Nested VM is unusably slow	No NV2 — every L1 EL2 access traps to L0
Kernel oops after using SIMD in a driver	FP state not saved; needs `kernel_neon_begin()`
Undefined instruction on `HVC`	`HCR_EL2.HCD` set, or EL2 not implemented

Symptom	Cause
Exits dominated by one reason	Use a non-trapping mechanism for that operation

Check yourself

- ✓ Explain the trap-emulate-resume cycle and which register configures it.
- ✓ Decode the ISS of a trapped `MSR` and say what a handler does with it.
- ✓ Explain why MMIO emulation uses an unmapped stage-2 page rather than a trap bit.
- ✓ Say what `ISV` saves the hypervisor from doing.
- ✓ Explain lazy FP switching and why its handler does *not* advance `ELR`.
- ✓ Give two reasons a hypervisor traps ID register reads.
- ✓ Explain what fine-grained traps improve.
- ✓ Explain what NV2 changes about nested virtualisation.
- ✓ Explain what `HCR_EL2.TGE` does and when a VHE host sets it.

The generic timer and counters

One architected timer, present on every ARMv8 core, with a counter readable from user space and comparators for each exception level. It is why `clock_gettime` needs no system call, and why timekeeping code is portable across every ARM SoC.

The system counter

ONE COUNTER, SHARED BY EVERYTHING

A single free-running 64-bit counter in an always-on power domain, incrementing at a fixed frequency, distributed to every core. It never stops, never wraps in any practical timeframe, and is identical on every core — which is exactly what a clocksource needs (OS chapter 17).

```
        mrs     x0, cntpct_el0          /* PHYSICAL count  */
        mrs     x0, cntvct_el0          /* VIRTUAL count   */
        mrs     x0, cntfrq_el0          /* frequency in Hz */

/* Readable from EL0 — no system call, no trap. This is the whole
   reason the vDSO can implement clock_gettime in user mode
   (chapter 30).                                            */

/* Frequency is IMPLEMENTATION DEFINED, commonly 1–100 MHz.
   Typical: 24 MHz, 25 MHz, 50 MHz, 19.2 MHz.

   CNTFRQ_EL0 is NOT set by hardware — FIRMWARE must program it
   at boot, on every core. If firmware forgets, software reads
   zero or garbage and all timekeeping is wrong. This is a
   classic bring-up bug, and Linux warns about it:          */
//   arch_timer: WARNING: Invalid trigger for IRQ
//   arch_timer: frequency not available

/* At 24 MHz, a 64-bit counter wraps after ~24,000 years. Treat
   it as never wrapping — but do use time_after()-style signed
   comparison anyway (OS chapter 17).                       */
```

Physical versus virtual counters

```
 CNTPCT_EL0    the raw physical count
 CNTVCT_EL0    the physical count MINUS an offset

      CNTVCT = CNTPCT - CNTVOFF_EL2

CNTVOFF_EL2 is per-VM, written by the hypervisor. Why:

 · a guest suspended for 10 seconds and resumed should not see
   10 seconds of elapsed CPU time it did not get
 · a guest MIGRATED to another host must see a continuous
   timeline, even though the new host's counter differs
 · a guest booted later should still see its own time starting
   near zero

So the hypervisor adjusts CNTVOFF and the guest's view stays
coherent, with no traps and no emulation.

CONSEQUENCE for software:
   · a guest OS uses the VIRTUAL counter and timer
   · a hypervisor, and firmware at EL2/EL3, uses the PHYSICAL one
   · Linux at EL1 uses virtual; Linux at EL2 (VHE) uses physical
   · reading the wrong one inside a VM gives host time, which
     breaks after any suspend or migration
```

The timers

Timer	Registers	Interrupt	Used by
EL1 physical	`CNTP_*_EL0`	PPI 30	A kernel with no hypervisor; firmware
EL1 virtual	`CNTV_*_EL0`	PPI 27	**A guest OS, and Linux at EL1**
EL2 physical	`CNTHP_*_EL2`	PPI 26	The hypervisor
EL2 virtual	`CNTHV_*_EL2`	PPI 28	A VHE host
EL3 physical	`CNTPS_*_EL1`	PPI 29	Secure firmware
Secure EL1	`CNTHPS_*`, `CNTHVS_*`	PPI 20, 19	A trusted OS (ARMv8.4+)

Each level gets its own comparator so they do not have to coordinate — a hypervisor's timer cannot be disturbed by its guest, and secure firmware's cannot be disturbed by the normal world.

Programming a timer

```
/* Each timer has three registers: */

   CNTV_CTL_EL0    [0] ENABLE
                   [1] IMASK — mask the interrupt output
                   [2] ISTATUS — read-only: has it fired?
   CNTV_CVAL_EL0   64-bit ABSOLUTE compare value
   CNTV_TVAL_EL0   32-bit RELATIVE value, a convenience:
                     writing it sets CVAL = CNTVCT + TVAL
                     reading it gives CVAL − CNTVCT (may go
                     negative once expired)

/* The timer fires when CNTVCT >= CVAL. It is a COMPARATOR, not a
   countdown — which is what makes it immune to drift.      */

/* Absolute deadline — the correct way for periodic work: */
static void timer_set_deadline(uint64_t deadline)
{
        asm volatile("msr cntv_cval_el0, %0" :: "r"(deadline));
        asm volatile("msr cntv_ctl_el0, %0" :: "r"(1UL));  /* enable */
        asm volatile("isb");
}

static void timer_periodic(uint64_t period_ticks)
{
        static uint64_t next;
        uint64_t now;
        asm volatile("mrs %0, cntvct_el0" : "=r"(now));

        if (!next) next = now;
        next += period_ticks;              /* ABSOLUTE, so no drift */
        timer_set_deadline(next);
}

/* Contrast with the relative form: */
        asm volatile("msr cntv_tval_el0, %0" :: "r"(period));
/* Each period restarts from NOW, so the handler's own execution
   time accumulates into permanent drift. Over an hour at 1 kHz
   with a 10 µs handler, that is 36 seconds of drift.
   Use CVAL and absolute deadlines. (OS chapter 17.)       */
```

Delays and timeouts

```
/* A portable microsecond delay, with no calibration loop: */

static void udelay(unsigned us)
{
        uint64_t freq, start, ticks;

        asm volatile("mrs %0, cntfrq_el0" : "=r"(freq));
        ticks = (freq * us) / 1000000;

        asm volatile("isb");                       /* order the read */
        asm volatile("mrs %0, cntvct_el0" : "=r"(start));

        for (;;) {
                uint64_t now;
                asm volatile("isb");
                asm volatile("mrs %0, cntvct_el0" : "=r"(now));
                if (now - start >= ticks)
                        break;
        }
}

/* The ISB matters: without it the counter read can be speculated
   or reordered, and a short delay can complete early. Linux's
   arch_counter_get_cntvct() includes it for exactly this
   reason.

   And note the subtraction: `now - start >= ticks` is correct
   across a wrap; `now >= start + ticks` is not.           */

/* A bounded wait — always prefer this to an unbounded loop: */
static bool wait_for_bit(volatile uint32_t *reg, uint32_t mask,
                         unsigned timeout_us)
{
        uint64_t freq, start, limit, now;

        asm volatile("mrs %0, cntfrq_el0" : "=r"(freq));
        asm volatile("mrs %0, cntvct_el0" : "=r"(start));
        limit = (freq * timeout_us) / 1000000;

        do {
                if (*reg & mask)
                        return true;
                asm volatile("mrs %0, cntvct_el0" : "=r"(now));
        } while (now - start < limit);

        return false;                      /* TIMED OUT — say so */
}
```

Why the vDSO works

```
clock_gettime(CLOCK_MONOTONIC, &ts) with no system call:

   1. read a kernel-maintained page mapped read-only into every
      process. It holds:
         · a base time value
         · the counter value at that base
         · a multiplier and shift for the conversion
         · a sequence counter (a seqlock — OS chapter 09)

   2. read CNTVCT_EL0                      ← EL0-readable
   3. delta = cntvct - base_cycles
   4. ns = base_ns + ((delta * mult) >> shift)
   5. re-check the sequence counter; retry if the kernel updated
      the page mid-read

   ~25 ns, entirely in user mode. Versus ~300 ns for a syscall.

  Two architectural properties make this possible:
     · the counter is readable from EL0 (CNTKCTL_EL1 permits it)
     · it is architected, so ONE vDSO implementation works on
       every ARM SoC

  CNTKCTL_EL1.EL0VCTEN gates EL0 access. A kernel could clear it
  to hide timing from userspace — some hardened configurations do,
  as a side-channel mitigation, at the cost of losing the vDSO
  fast path.
```

The event stream

```
/* WFE waits for an event (chapter 22). If the event never comes,
   the core waits forever — which is a problem for a spinlock
   whose holder was descheduled.

   The generic timer can generate a PERIODIC EVENT STREAM as a
   safety net:                                              */

   CNTKCTL_EL1.EVNTEN    enable the event stream
   CNTKCTL_EL1.EVNTI     which counter bit transition triggers it
   CNTKCTL_EL1.EVNTDIR   rising or falling edge

/* Typically configured for a ~100 µs period. A WFE therefore
   never sleeps longer than that, guaranteeing forward progress
   even if a SEV is missed.

   Linux enables it, and you can see the configuration:     */
//   dmesg | grep -i 'arch_timer.*event stream'
//   arch_timer: Architected timer HYP event stream enabled

/* This is also what makes WFE safe to use in a guest: the
   hypervisor may trap WFE (chapter 31), but the event stream
   ensures the guest wakes anyway.                          */
```

Firmware responsibilities

WHAT MUST BE SET UP BEFORE LINUX BOOTS

- `CNTFRQ_EL0` **on every core**, with the actual counter frequency. Hardware does not set it. If firmware forgets, all timekeeping is wrong or the kernel refuses to boot.
- `CNTVOFF_EL2` **= 0** when handing off to a kernel at EL1 with no hypervisor. Firmware sometimes leaves it non-zero, and the guest's virtual time then starts at an arbitrary offset.
- `CNTHCTL_EL2` configured so EL1 may access the physical counter and timer, if that is intended.
- **The counter must be enabled and running** in the always-on domain before any core reads it.

Linux checks and complains about several of these; the messages are worth recognising during bring-up.

```
/* The device tree describes it: */
timer {
        compatible = "arm,armv8-timer";
        interrupts = <GIC_PPI 13 IRQ_TYPE_LEVEL_LOW>,   /* secure   */
                     <GIC_PPI 14 IRQ_TYPE_LEVEL_LOW>,   /* physical */
                     <GIC_PPI 11 IRQ_TYPE_LEVEL_LOW>,   /* virtual  */
                     <GIC_PPI 10 IRQ_TYPE_LEVEL_LOW>;   /* hyp      */
        clock-frequency = <24000000>;   /* ONLY if firmware failed
                                           to set CNTFRQ - a
                                           documented workaround,
                                           not the normal path    */
        always-on;                       /* survives CPU idle      */
};

/* The PPI numbers here are relative to 16, so PPI 13 is INTID 29
   (chapter 23). Getting this wrong is a common device tree error
   and the symptom is a kernel that boots then hangs with no
   timer interrupts.                                            */
```

The memory-mapped timer

```
/* Besides the per-core system-register timers, the architecture
   defines an optional MEMORY-MAPPED timer block (CNTControlBase,
   CNTReadBase, CNTBaseN).                                      */

/* Why it exists: devices other than CPUs may need the counter -
   a DMA engine timestamping packets, a power controller, a
   coprocessor. They cannot execute MRS.

   Software rarely touches it directly. It appears in device
   trees as:                                                    */
        compatible = "arm,armv7-timer-mem";

/* Its one common use in Linux: providing a clocksource on
   systems where the per-core timer stops in deep idle, since
   the memory-mapped block is in the always-on domain.     */
```

The Linux view

```
/* The generic timer is Linux's clocksource AND clockevent
   (OS chapter 17):                                        */

cat /sys/devices/system/clocksource/clocksource0/available_clocksource
#   arch_sys_counter                    ← the generic timer
cat /sys/devices/system/clocksource/clocksource0/current_clocksource

cat /proc/interrupts | grep arch_timer
#    3:  102394  98211  101002  99887  GICv3  27 Level  arch_timer
#                                              ^^ PPI, so counts on
#                                                 every core

/* Kernel APIs built on it: */
ktime_get()                    /* CLOCK_MONOTONIC, ns          */
ktime_get_real()               /* CLOCK_REALTIME               */
ktime_get_boottime()           /* includes suspend             */
get_cycles()                   /* the raw counter              */
arch_timer_read_counter()

udelay(50);                    /* busy-wait, atomic-safe       */
usleep_range(100, 200);        /* sleeps; allows BATCHING      */
hrtimer_start(&t, ms_to_ktime(10), HRTIMER_MODE_REL);

/* And the tickless machinery (OS chapter 17) programs CVAL for
   the next actual deadline rather than a fixed tick, which is
   what lets an idle core sleep for seconds.               */
```

How it shows up

Symptom	Cause
Kernel hangs after "Calibrating delay loop"	`CNTFRQ_EL0` not set by firmware
All delays are wildly wrong	Wrong `CNTFRQ_EL0`, or a `clock-frequency` DT override that disagrees with the hardware
No timer interrupts; system boots then freezes	Wrong PPI number in the device tree
Guest time jumps after migration	Used the physical counter instead of the virtual one
Guest sees host uptime	`CNTVOFF_EL2` not set
Periodic task drifts over hours	Used `TVAL` (relative) instead of `CVAL` (absolute)
Short delays complete early	Missing `ISB` around the counter read
`clock_gettime` unexpectedly slow	vDSO unavailable, or `CNTKCTL.EL0VCTEN` cleared
Core stuck in `WFE`	Event stream not enabled, and the `SEV` was missed
Timer stops during deep idle	Per-core timer not in the always-on domain; needs a broadcast timer

Check yourself

- ✓ Explain what makes the system counter suitable as a clocksource.
- ✓ Say who must set `CNTFRQ_EL0` and what happens if they do not.
- ✓ Explain the difference between the physical and virtual counters and who uses each.

- ✓ Explain why `CNTVOFF_EL2` exists, with two scenarios.
- ✓ Name the timers per exception level and why each has its own.
- ✓ Explain why `CVAL` avoids drift and `TVAL` does not.
- ✓ Explain the two architectural properties that make the vDSO's `clock_gettime` possible.
- ✓ Say what the event stream is for and what problem it prevents.

Power states, WFI/WFE and PSCI

Two instructions to stop a core, and a standard firmware interface to turn one off entirely. Between them they cover the whole range from an idle loop to booting a secondary CPU — and PSCI is why one kernel image works across every ARM SoC.

WFI and WFE

	`WFI`	`WFE`
Waits for	An **interrupt** (or another wake-up event)	An **event**: `SEV`, an exclusive-monitor release, the event stream, or an interrupt
Wakes on a MASKED interrupt?	**Yes** — even with `PSTATE.I` set	Yes
Event register	Not used	Consumes it if already set, and does not sleep
Used for	The idle loop	Spinlocks, polling loops

THE PROPERTY THAT MAKES `WFI` USABLE

It wakes on an interrupt *even if that interrupt is masked*. So the idle loop can mask interrupts, check whether there is work, and `WFI` — without a race where an interrupt arrives between the check and the sleep. Unmask after waking and the interrupt is then taken normally.

Without that property, every idle loop would have a lost-wakeup window.

```
/* The correct idle loop */
static void cpu_idle(void)
{
        for (;;) {
                local_irq_disable();            /* mask */
                if (need_resched()) {
                        local_irq_enable();
                        schedule();
                        continue;
                }
                dsb(sy);                        /* outstanding
                                                   transactions must
                                                   complete first  */
                wfi();                          /* sleep           */
                local_irq_enable();             /* now take it     */
        }
}

/* The DSB matters: a memory transaction still in flight when the
   core enters a low-power state may never complete, hanging the
   interconnect. Chapter 21.                                    */

/* WFE, in a spinlock (chapter 22) */
1:      ldaxr   w1, [x0]
        cbnz    w1, 2f                  /* held? wait              */
        stxr    w2, w3, [x0]
        cbnz    w2, 1b
        b       3f
2:      wfe                             /* sleep in low power      */
        b       1b
3:      /* acquired */

/* The unlock's STLR to the monitored address generates the event
   automatically — no explicit SEV is needed. And the timer's
   EVENT STREAM (chapter 32) guarantees a wake within ~100 µs
   even if the event is missed entirely.                        */
```

Both are hints

> **NEITHER INSTRUCTION GUARANTEES ANYTHING ABOUT POWER**
>
> `WFI` and `WFE` are *architectural hints*. A conforming implementation may treat them as `NOP`. What actually happens — clock gating, retention, or nothing at all — is entirely implementation defined.
>
> What *is* architected is the wake-up condition. So the correctness of your idle loop does not depend on the power behaviour, which is the right split: software expresses intent, hardware decides the mechanism. Deeper states than the core can enter by itself require firmware, which is what PSCI is for.

The state hierarchy

```
   RUNNING
     │
     │ WFI — the core requests a low-power state
     ▼
   STANDBY / retention            wake: ~ns to µs
     · clocks gated, state PRESERVED in registers and caches
     · the core resumes exactly where it stopped
     · entered directly by WFI; no firmware needed
     │
     │ PSCI CPU_SUSPEND with a deeper state ID
     ▼
   POWER DOWN                     wake: ~10s–100s of µs
     · the core's power domain is turned OFF
     · ALL state is lost: registers, caches, MMU config, GIC
       CPU interface
     · software must SAVE everything before, and RESTORE after
     · the core restarts at a firmware entry point and works its
       way back
     │
     │ all cores in a cluster down
     ▼
   CLUSTER POWER DOWN
     · the shared L2 must be cleaned and invalidated first
     · the cluster's GIC redistributor state saved
     │
     ▼
   SYSTEM SUSPEND                 wake: ms
     · everything except an always-on domain
     · DRAM in self-refresh
     · wake sources configured in the always-on power controller

  Each level saves more power and costs more to leave. The
  BREAK-EVEN calculation from CA chapter 20 governs which one an
  OS governor picks.
```

PSCI

> **POWER STATE COORDINATION INTERFACE**
>
> A standard SMC-based API implemented by EL3 firmware. It abstracts every SoC-specific power controller behind one interface, so a single kernel image can boot secondary cores, idle, suspend and reboot on any platform. Before PSCI, every SoC needed its own `smp_ops` in the kernel — which is why old ARM32 Linux had dozens of platform-specific power drivers and ARM64 has none.

Function	ID (SMC64)	Does
`PSCI_VERSION`	`0x84000000`	Returns the implemented version
`CPU_SUSPEND`	`0xC4000001`	Enter a low-power state, with a resume entry point
`CPU_OFF`	`0x84000002`	Power this core down; does not return
`CPU_ON`	`0xC4000003`	**Start another core** at a given address
`AFFINITY_INFO`	`0xC4000004`	Is that core on, off, or transitioning?
`SYSTEM_OFF`	`0x84000008`	Shut down
`SYSTEM_RESET`	`0x84000009`	Reboot

Function	ID (SMC64)	Does
`SYSTEM_SUSPEND`	`0xC400000E`	Suspend to RAM
`FEATURES`	`0x8400000A`	Query support for a function

```
/* Starting a secondary CPU — this is how every ARM64 SMP kernel
   brings up its cores (OS chapter 18):                    */

#include <linux/arm-smccc.h>

static int cpu_on(u64 mpidr, u64 entry_point, u64 context_id)
{
        struct arm_smccc_res res;

        arm_smccc_smc(0xC4000003,       /* PSCI_CPU_ON, SMC64 */
                      mpidr,            /* which core         */
                      entry_point,      /* PHYSICAL address   */
                      context_id,       /* passed back in x0  */
                      0, 0, 0, 0, &res);

        return (int)res.a0;             /* 0 = success        */
}

/* The target core comes out of reset at entry_point with:
     · the MMU OFF
     · caches OFF
     · at the same EL the caller was at
     · x0 = context_id

   In other words, exactly the same contract as the primary CPU's
   boot entry (OS chapter 18) — which is why the secondary path
   in head.S shares most of its code with the primary path. */

/* PSCI error codes worth recognising: */
   0   SUCCESS
  -1   NOT_SUPPORTED
  -2   INVALID_PARAMETERS
  -3   DENIED
  -4   ALREADY_ON          ← a common bring-up result
  -5   ON_PENDING
  -6   INTERNAL_FAILURE
  -7   NOT_PRESENT
  -8   DISABLED
  -9   INVALID_ADDRESS
```

CPU_SUSPEND and the power state ID

```
/* CPU_SUSPEND takes a platform-specific state ID describing HOW
   deep to go, and where to resume.                          */

        arm_smccc_smc(0xC4000001,       /* CPU_SUSPEND        */
                      power_state,      /* the state ID       */
                      resume_entry,     /* physical address   */
                      context_id,
                      0, 0, 0, 0, &res);

/* The power_state encoding (extended format, PSCI 1.0+):
     [3:0]   state ID — platform specific
     [16]    0 = standby (state preserved)
             1 = POWERDOWN (state LOST)
     [25:24] affinity level: 0 = core, 1 = cluster, 2 = system

   The state IDs come from the DEVICE TREE, so the kernel does
   not need platform knowledge:                              */

cpus {
        idle-states {
                entry-method = "psci";

                cpu_sleep_0: cpu-sleep-0 {
                        compatible = "arm,idle-state";
                        arm,psci-suspend-param = <0x0010000>;
                        entry-latency-us = <300>;
                        exit-latency-us = <1200>;
                        min-residency-us = <2000>;
                        local-timer-stop;    /* ← the timer STOPS  */
                };
        };
        cpu@0 {
                cpu-idle-states = <&cpu_sleep_0>;
        };
};

/* The kernel's cpuidle governor uses entry/exit latency and
   min-residency to decide whether the expected idle duration
   justifies this state — the break-even calculation from
   CA chapter 20, driven entirely by device tree data.     */
```

> **`LOCAL-TIMER-STOP` MATTERS**
>
> In deep states the per-core generic timer loses power and stops. If the kernel programmed it to wake the core, that wake-up never happens. The fix is a **broadcast timer** in the always-on domain: the kernel switches to it before entering such a state, and one core stays awake (or the always-on timer fires) to wake the others. Getting this wrong produces a core that goes idle and never returns — a hang that only appears when the system is quiet.

What must be saved before power-down

```
A POWERDOWN state loses everything. The full sequence:

  1. save all system registers that are not restored by firmware:
       SCTLR_EL1, TCR_EL1, TTBR0/1_EL1, MAIR_EL1, VBAR_EL1,
       TPIDR_EL0/EL1, CPACR_EL1, and the debug registers

  2. save the GIC CPU interface and redistributor state
       ICC_PMR_EL1, ICC_CTLR_EL1, ICC_IGRPEN1_EL1,
       pending SGIs, the redistributor's enable and priority
       state (chapter 23)

  3. CLEAN AND INVALIDATE the caches to the Point of Coherency
       · this is the ONE legitimate use of the set/way
         operations (chapter 18)
       · for a cluster power-down, the shared L2 too
       · and it must be the LAST thing, because anything after
         it may dirty a line again

  4. DSB, then issue PSCI CPU_SUSPEND

  On resume, the core enters at resume_entry with the MMU off and
  must rebuild everything: page tables, MMU, caches, GIC, timers,
  then restore the saved state and return into the idle loop as
  though WFI had simply returned.

  Linux does this in cpu_suspend() / cpu_resume(), and the
  saved-register set is arch/arm64/mm/proc.S's cpu_do_suspend.

  A missing register in that list gives a system that resumes and
  then behaves subtly wrongly — the hardest class of power bug.
```

CPU hotplug

```
# Offlining a core uses PSCI CPU_OFF underneath
echo 0 > /sys/devices/system/cpu/cpu3/online
echo 1 > /sys/devices/system/cpu/cpu3/online

cat /sys/devices/system/cpu/online
cat /sys/devices/system/cpu/offline

/* What the kernel does on offline:
     1. migrate all tasks off that core
     2. migrate all interrupts (set_affinity) off it
     3. migrate timers and hrtimers
     4. run CPU_DYING notifiers in every subsystem
     5. call PSCI CPU_OFF — which does not return

   And on online:
     1. PSCI CPU_ON with the secondary entry point
     2. the core runs head.S's secondary path: MMU, caches, GIC
        redistributor, timer
     3. CPU_STARTING notifiers
     4. mark it online; the scheduler starts using it

   Used for: thermal management, power saving, taking a core out
   of service after a fault, and testing SMP code paths.   */
```

The Linux idle path

```
/* cpuidle chooses a state, then PSCI enters it: */

cat /sys/devices/system/cpu/cpu0/cpuidle/state*/name
#   WFI
#   cpu-sleep
#   cluster-sleep

cat /sys/devices/system/cpu/cpu0/cpuidle/state1/{latency,residency,usage,time}
#   latency    exit latency in µs
#   residency  minimum time that justifies entering
#   usage      how many times it was entered
#   time       total µs spent in it

cpupower monitor
cpupower idle-info

/* If a state's `usage` is zero, the governor never considers it
   worthwhile — usually because the workload wakes too often
   (OS chapter 17: count WAKEUPS, not CPU percentage).

   powertop shows what is causing the wakeups.            */
sudo powertop
```

The spin-table alternative

```
/* Before PSCI, and still on some simple platforms, secondaries
   are released with a SPIN TABLE: each parked core polls a
   memory location until the primary writes an address.     */

cpu@1 {
        device_type = "cpu";
        compatible = "arm,cortex-a53";
        reg = <0x1>;
        enable-method = "spin-table";
        cpu-release-addr = <0x0 0x8000fff8>;
};

/* The parked core runs, from firmware:
        1:      ldr     x0, [release_addr]
                cbz     x0, 1b
                wfe                     ← so it does not burn power
                br      x0

   The primary writes the entry point and issues SEV.

   Why PSCI is better:
     · it also handles CPU_OFF, SUSPEND and SYSTEM_RESET, which
       a spin table cannot
     · the parked core consumes power and holds memory
     · the release address must be reserved from the kernel's
       memory map, which is error-prone
     · no way to power the core down at all

   Use PSCI unless the platform genuinely has no EL3 firmware. */
```

How it shows up

Symptom	Cause
Only one CPU comes online	PSCI not working, wrong MPIDR in the device tree, or firmware returning an error
Secondary CPU hangs on start	Entry point not physically addressed, or MMU/cache state assumed
Core goes idle and never wakes	Timer stopped in a deep state with no broadcast timer
System resumes but behaves subtly wrongly	A system register not in the save/restore list
Interrupts lost after resume	GIC redistributor state not restored
Corruption after cluster power-down	L2 not cleaned to PoC before power was removed
Idle states show zero usage	Wakeups too frequent to justify entry — find them with `powertop`
Core hangs entering `WFI`	No `DSB`; an outstanding transaction never completed
Spinlock spins burn power	No `WFE`, or the event stream is disabled
Reboot does not work	`SYSTEM_RESET` unimplemented; falls back to a watchdog

Check yourself

- ✓ Give the difference between `WFI` and `WFE` and what each is used for.
- ✓ Explain why `WFI` wakes on a masked interrupt, and the race that prevents.
- ✓ Explain why both are hints, and what *is* architected about them.
- ✓ Name the power state levels and what is lost at each.
- ✓ Say what PSCI is and why it removed platform code from the kernel.
- ✓ Show how a secondary CPU is started, and state the entry contract.
- ✓ List the three things that must be saved before a power-down state.
- ✓ Explain `local-timer-stop` and the broadcast timer it requires.
- ✓ Give two reasons PSCI is preferable to a spin table.

Performance monitoring and the PMU

Every ARM core contains counters that measure what the microarchitecture is actually doing. This is ground truth rather than inference — and knowing which events to count turns performance work from guessing into measurement.

What the PMU provides

```
  PMCCNTR_EL0        a dedicated 64-bit CYCLE counter
  PMEVCNTR0-30_EL0   general counters, 32-bit (64-bit in
                     ARMv8.5), each programmable to count one
                     event
  PMEVTYPER0-30_EL0  which event each counter counts, plus
                     filters (count at EL0? EL1? EL2? secure?)

  PMCR_EL0           enable, reset, and how many counters exist
  PMCNTENSET_EL0     enable individual counters
  PMOVSSET_EL0       overflow status — the interrupt source
  PMINTENSET_EL1     enable overflow interrupts
  PMUSERENR_EL0      may EL0 access these directly?

The NUMBER of counters is implementation defined — typically
4 to 6 on a small core, 6 to 8 on a large one. Read it:

      mrs     x0, pmcr_el0
      ubfx    x0, x0, #11, #5        /* N field */

This is why perf multiplexes: ask for more events than there
are counters and it time-slices them, scaling the results.
The scaling factor appears in perf's output, and heavily
multiplexed measurements are correspondingly noisy.
```

The architected events

Event	Number	Counts
`CPU_CYCLES`	0x11	Cycles — the denominator for everything
`INST_RETIRED`	0x08	Instructions architecturally executed
`INST_SPEC`	0x1B	Instructions *speculatively* executed
`L1D_CACHE`	0x04	L1 data cache accesses
`L1D_CACHE_REFILL`	0x03	L1 data cache misses
`L1I_CACHE_REFILL`	0x01	L1 instruction cache misses
`L2D_CACHE_REFILL`	0x17	L2 misses
`LL_CACHE_MISS_RD`	0x37	Last-level read misses — i.e. DRAM accesses
`L1D_TLB_REFILL`	0x05	Data TLB misses
`DTLB_WALK`	0x34	Actual page-table walks
`BR_RETIRED`	0x21	Branches executed
`BR_MIS_PRED_RETIRED`	0x22	Branch mispredicts

Event	Number	Counts
`STALL_FRONTEND`	0x23	Cycles with no instruction issued, front end starved
`STALL_BACKEND`	0x24	Cycles with no instruction issued, back end busy
`MEM_ACCESS`	0x13	All memory accesses
`BUS_ACCESS`	0x19	Accesses that reached the interconnect
`EXC_TAKEN`	0x09	Exceptions taken
`LDREX_SPEC` / `STREX_FAIL_SPEC`	0x6C / 0x6E	Exclusive attempts and failures (ch. 22)

Some events are architected (the same everywhere), most are IMPLEMENTATION DEFINED. Always check the core's TRM for the ones that matter — `perf list` shows what the kernel knows about.

Reading the numbers

```
START with IPC. It tells you which world you are in:

   IPC = INST_RETIRED / CPU_CYCLES

     < 0.5   STALLED. Find out on what.
     0.5–1.5 mixed
     > 2.0   compute bound — further gains need a better
             ALGORITHM, not micro-optimisation

THEN split the stalls:

   STALL_FRONTEND high  → INSTRUCTION supply
        · I-cache misses, ITLB misses, branch mispredicts
        · fixes: code layout, PGO, fewer/more predictable
          branches, smaller hot code

   STALL_BACKEND high   → DATA supply or execution
        · D-cache misses, dependency chains, divides
        · fixes: data layout, blocking, more ILP (CA ch. 19)

THEN the specific rates:

   L1D miss rate    = L1D_CACHE_REFILL / L1D_CACHE
   LLC miss rate    = LL_CACHE_MISS_RD / LL_CACHE_RD
   branch miss rate = BR_MIS_PRED_RETIRED / BR_RETIRED
                      > 2% is worth investigating
   TLB walks/insn   = DTLB_WALK / INST_RETIRED
                      high → try huge pages

AND the speculation waste:

   INST_SPEC vs INST_RETIRED
     a large gap means work was executed and thrown away —
     heavy misprediction (CA chapter 12)
```

Using perf

```
# the standard first look
perf stat ./program
#   12,405,281,113  cycles                    #  2.98 GHz
#   18,204,113,782  instructions              #  1.47  insn per cycle
#    3,201,554,231  branches
#       62,119,882  branch-misses             #  1.94% of all branches

# ARM-specific, by architected name
perf stat -e cycles,instructions,\
l1d_cache,l1d_cache_refill,\
l2d_cache_refill,ll_cache_miss_rd,\
dtlb_walk,itlb_walk,\
br_retired,br_mis_pred_retired,\
stall_frontend,stall_backend ./program

# by raw event number, when the kernel does not know the name
perf stat -e r0011,r0008 ./program        # cycles, instructions

# WHICH function, and which instruction?
perf record -g ./program && perf report
perf record -e l1d_cache_refill ./program && perf annotate

# system-wide
perf stat -a sleep 10
perf top

# what does this kernel expose?
perf list | grep -i arm
ls /sys/bus/event_source/devices/
```

Sampling versus counting

```
COUNTING       program a counter, run, read the total.
               Exact. No overhead beyond the counter itself.
               → perf stat

SAMPLING       program the counter to OVERFLOW after N events,
               and take an interrupt. The handler records the PC.
               → perf record

  Sampling gives you attribution — WHICH code caused the misses —
  at the cost of overhead and of SKID: the PC recorded at the
  overflow interrupt is not exactly the instruction that caused
  the event, because the pipeline had moved on.

  Skid is why perf annotate sometimes blames the instruction
  AFTER the real culprit. Two mitigations:

    SPE — Statistical Profiling Extension (ARMv8.2). Hardware
          samples individual OPERATIONS and records the PC, the
          data address, the latency and the source of the data,
          into a memory buffer. No skid, and vastly richer.

          perf record -e arm_spe// -- ./program
          perf report --sort=symbol,srcline

    BRBE — Branch Record Buffer Extension (ARMv9). A hardware
           branch history, giving exact taken/not-taken paths.

  If your core has SPE, use it. It answers "which load missed,
  and where did the data come from" directly, which counting
  never can.
```

Access control

```
/* By default EL0 cannot touch the PMU. PMUSERENR_EL0 gates it: */

   PMUSERENR_EL0[0]  EN     allow EL0 access to all PMU registers
                 [1]  SW     allow EL0 writes to PMSWINC
                 [2]  CR     allow EL0 to read the CYCLE counter
                 [3]  ER     allow EL0 to read event counters

/* Enabling direct EL0 access removes the syscall from the
   measurement path — essential for microbenchmarks: */
        msr     pmuserenr_el0, #0xF

/* Linux exposes this as a sysctl: */
//   /proc/sys/kernel/perf_user_access   (arm64, 5.17+)

/* Why it is off by default: a cycle counter readable from
   userspace is a high-resolution TIMING SIDE CHANNEL, useful for
   cache-timing attacks. The same reasoning applies to
   CNTVCT_EL0, which hardened configurations sometimes also
   disable (chapter 32).                                    */

/* And the general perf gate: */
//   /proc/sys/kernel/perf_event_paranoid
//     -1  everything
//      0  no CPU events for unprivileged users
//      1  no tracepoints            (common default)
//      2  no kernel profiling       (hardened default)
//      3  no perf at all
```

Reading counters directly

```
/* For a tight microbenchmark, the perf syscall overhead can
   exceed what you are measuring. Read the counter yourself: */

static inline uint64_t read_cycles(void)
{
        uint64_t v;
        asm volatile("isb");                    /* order it */
        asm volatile("mrs %0, pmccntr_el0" : "=r"(v));
        return v;
}

/* Setup, from EL1: */
static void pmu_enable(void)
{
        /* enable the PMU and reset the cycle counter */
        write_sysreg(BIT(0) | BIT(2), pmcr_el0);   /* E | C  */
        /* enable the cycle counter */
        write_sysreg(BIT(31), pmcntenset_el0);
        /* do not divide by 64 */
        write_sysreg(read_sysreg(pmcr_el0) & ~BIT(3), pmcr_el0);
        /* allow EL0 access */
        write_sysreg(0xF, pmuserenr_el0);
        isb();
}

/* Programming an event counter: */
static void pmu_count_event(int counter, uint32_t event)
{
        write_sysreg(counter, pmselr_el0);      /* select        */
        isb();
        write_sysreg(event, pmxevtyper_el0);    /* which event   */
        write_sysreg(BIT(counter), pmcntenset_el0);
        isb();
}

/* The ISBs matter: without them the counter read can be
   reordered relative to the code being measured, giving results
   that vary with unrelated changes (chapter 21).          */
```

MICROBENCHMARKING TRAPS

- **Frequency scaling.** Cycles are a stable unit; seconds are not. Either pin the governor (`cpupower frequency-set -g performance`) or measure cycles.
- **Core migration.** On big.LITTLE the same code runs at very different speeds. Pin with `taskset`.
- **The counter is per-core.** If the thread migrates mid-measurement, the delta is meaningless.
- **Cold caches.** The first iteration measures the miss, not the code. Warm up, then measure.
- **Dead code elimination.** If the result is unused the compiler deletes the work (C ch. 01). Consume it.

The PMU under virtualisation

```
/* MDCR_EL2 controls guest PMU access: */
   MDCR_EL2.TPM    trap all PMU register access to EL2
   MDCR_EL2.TPMCR  trap PMCR_EL0 only
   MDCR_EL2.HPMN   how many counters the guest may use — the
                   rest are reserved for the hypervisor

/* So a hypervisor can:
     · give the guest a virtual PMU (KVM emulates it)
     · partition the counters between host and guest
     · deny access entirely, which is common in cloud
       environments because a cycle counter is a side channel

   Which is why `perf` inside a VM often reports nothing, or
   reports host-visible events that do not correspond to the
   guest's own execution. Check:                            */
//   dmesg | grep -i 'kvm.*pmu'
//   perf stat ./prog     → "not supported" means the PMU is
//                            not exposed to this guest
```

Beyond the core

```
/* Many SoCs expose additional PMUs outside the CPU: */

   DSU PMU        the DynamIQ Shared Unit — the cluster's L3
   CMN PMU        the coherent mesh interconnect (chapter 25)
   SMMU PMU       translation and TLB behaviour of DMA masters
   DDR / memory controller PMU
   GPU, NPU, and other accelerator counters

perf list | grep -iE 'arm_dsu|arm_cmn|smmu|ddr'
perf stat -e arm_cmn_0/hnf_cache_miss/ -a sleep 10
perf stat -e arm_dsu_0/bus_access/ ./prog

/* These answer questions the core PMU cannot:
     · is the interconnect saturated, or is DRAM the limit?
     · which agent is consuming memory bandwidth?
     · are SMMU TLB misses hurting a DMA-heavy workload?

   On a system where several masters share bandwidth (OS
   chapter 19), these are the only way to attribute the
   contention.                                              */
```

A workflow

```
1. MEASURE the thing you actually care about, end to end.
   If it is not repeatable, stop and fix that first.

2. perf stat — get IPC.
      low  → continue to 3
      high → the algorithm is the problem; stop measuring
             microarchitecture

3. Split the stalls: frontend or backend?

4. Drill into the specific rate — cache, TLB, branch.

5. perf record on THAT event → which function, which line.

6. Fix the largest contributor, and ONLY that one.

7. Re-measure. Did the predicted gain appear? If not, the
   model was wrong — that is information.

8. Re-profile. The bottleneck has moved.

Never skip step 1, and never optimise anything perf has not
shown you. CA chapter 18 has the arithmetic that makes this
worth doing in this order.
```

How it shows up

Symptom	Cause
`perf stat` reports `<not supported>`	PMU not exposed — a VM, or `perf_event_paranoid`
Counts scale oddly between runs	Event multiplexing — more events requested than counters exist
`perf annotate` blames the wrong instruction	Sampling skid; use SPE if available
Cycle counts vary 3× between runs	Frequency scaling, or migration between big and LITTLE
Direct counter reads give inconsistent results	Missing `ISB`, or the thread migrated
IPC high but the program is slow	Executing too many instructions — an algorithmic problem
IPC very low, few cache misses	Dependency chains, or frontend bound
Large `INST_SPEC` / `INST_RETIRED` gap	Heavy branch misprediction
High `strex_fail_spec`	Exclusive contention — switch to LSE or reduce sharing
Core counters look fine, throughput is poor	Interconnect or memory controller saturation — use the uncore PMUs

Check yourself

- ✓ Say what the PMU provides and why the counter count matters for `perf`.
- ✓ Explain what IPC tells you and what each range implies for your next step.
- ✓ Distinguish `STALL_FRONTEND` from `STALL_BACKEND` and the fixes for each.
- ✓ Explain the difference between counting and sampling, and what skid is.
- ✓ Say what SPE adds over sampling.
- ✓ Explain why EL0 PMU access is disabled by default.

- ✓ List four microbenchmarking traps and how to avoid each.
- ✓ Explain why `perf` often reports nothing inside a VM.
- ✓ Give a case where core counters are insufficient and an uncore PMU is needed.

Putting it together: one load instruction

Everything in this track converging on a single instruction. Follow `ldr x1, [x0]` from fetch to retirement, naming every structure it touches and every way it can fail — and the whole architecture becomes one connected picture.

The complete journey

```
       ldr  x1, [x0]        with x0 = 0xFFFF_0000_C0DE_1234

① FETCH                                              chapters 10, 12
     PC → I-TLB → L1 I-cache → the instruction word
     the branch predictor has already guessed the next PC
     FAILS: instruction abort — unmapped, or execute-never

② DECODE                                            chapters 03, 07
     fixed 32-bit encoding, decoded in parallel with neighbours
     recognised: 64-bit load, base x0, offset 0
     FAILS: undefined instruction, or trapped by a higher EL

③ RENAME / ISSUE                                          chapter 10
     x1 mapped to a fresh physical register
     the instruction waits until x0 is ready

④ ADDRESS GENERATION                                      chapter 08
     effective address = x0 + 0
     FAILS: nothing yet

⑤ ALIGNMENT CHECK                                         chapter 11
     Device memory → ANY unaligned access faults
     Normal memory → allowed unless SCTLR.A
     FAILS: alignment fault, ESR.DFSC = 0b100001

⑥ TRANSLATION                                    chapters 12, 13, 15
     VA[55] == 1 → TTBR1_EL1
     D-TLB lookup, tagged by ASID
       HIT  → PA and attributes in ~1 cycle
       MISS → hardware walks L0 → L1 → L2 → L3
              up to 4 memory accesses, each cacheable
     FAILS: translation fault, DFSC = 0b0001LL
            the LEVEL tells you which table was wrong

⑦ PERMISSION CHECK                                        chapter 14
     AP[2:1] vs the current EL and the access type
     UXN / PXN for instruction fetches
     PAN: may EL1 touch this EL0-accessible page right now?
     FAILS: permission fault, DFSC = 0b0011LL

⑧ STAGE 2                                                 chapter 16
     only if a hypervisor is active
     IPA → PA via VTTBR_EL2, second permission check
     FAILS: exception to EL2, not EL1. HPFAR_EL2 holds the IPA.

⑨ MEMORY TYPE                                             chapter 11
     AttrIndx → MAIR_EL1 slot → Normal-WB / Device-nGnRE / …
     shareability from SH[1:0]
     determines: cacheable? speculatable? reorderable?

⑩ CACHE                                              chapters 17, 19
     L1 D-cache probe (VIPT — indexed with the VA while the TLB
     produced the tag)
       miss → L2 → cluster L3 → system cache
     coherency maintained across the shareability domain

⑪ INTERCONNECT                                            chapter 25
     an AXI/CHI read carrying:
       ADDR = PA
       ID   = the master ID
       PROT[1] = the NS bit          ← TrustZone
```

```
        CACHE   = the attributes
      → an SMMU may translate it (non-CPU masters)
      → an XPU may REFUSE it
      FAILS: external abort, DFSC = 0b010000
             — translation SUCCEEDED; the bus said no

⑫ MEMORY CONTROLLER → DRAM

⑬ RETURN
      a 64-byte line fills L2 and L1
      x1 receives the requested 8 bytes

⑭ RETIRE                                                  chapter 10
      x1 architecturally updated, IN PROGRAM ORDER
      only NOW is the instruction irrevocable
```

Every failure, mapped

Step	Failure	ESR	First thing to check
①	Instruction abort	EC 0x20/0x21	Is the code mapped? Is the page XN?
②	Undefined instruction	EC 0x00	Optional feature without an ID check (ch. 01)
②	Trapped operation	EC 0x18, 0x07	A hypervisor or lazy-FP trap (ch. 31)
⑤	Alignment fault	DFSC 0b100001	Is this region Device memory?
⑥	Translation fault	DFSC 0b0001LL	Null pointer, missing `ioremap`, wrong TTBR
⑥	Access flag fault	DFSC 0b0010LL	`AF` not set in the descriptor
⑦	Permission fault	DFSC 0b0011LL	AP bits, XN, or PAN
⑥/⑦	Stale mapping used	—	Missing `TLBI` (ch. 15)
⑧	Stage-2 fault	EC 0x24 at EL2	`HPFAR_EL2`; is it an emulated device?
⑨	Wrong memory type	No fault	Cached MMIO, or Device DMA buffer (ch. 11)
⑩	Stale data	No fault	Missing cache maintenance (ch. 18)
⑩	Cross-core disagreement	No fault	Non-shareable mapping (ch. 19)
⑪	External abort	DFSC 0b010000	**Clock, power, or an access-control block**
⑪	Posted-write rejection	SError, EC 0x2F	Imprecise — use ESB to bisect
⑭	Wrong order observed	No fault	Missing barrier (ch. 21)

THE DISTINCTION THAT SAVES THE MOST TIME

Translation fault versus **external abort**. A translation fault means the page tables are wrong — a software problem you can fix by reading descriptors. An external abort means translation *succeeded*, a real bus transaction went out, and something out there refused it — a platform problem: an unclocked peripheral, a powered-down domain, or a firewall.

They send you to completely different parts of the system, and one field in ESR tells you which.

What each step costs

```
   TLB hit + L1 hit                     ~4 cycles
   TLB hit + L2 hit                     ~15 cycles
   TLB hit + LLC hit                    ~40 cycles
   TLB hit + DRAM                       ~200 cycles
   TLB MISS + walk cache hit            +10–20 cycles
   TLB MISS + full 4-level walk         +4 memory accesses
   TLB miss + TWO-STAGE walk            up to 24 accesses  (ch. 16)
   plus a page fault to the OS          ~1–5 µs           (minor)
   plus a page fault to DISK            ~100 µs – 10 ms   (major)

  Which is why the whole architecture exists as it does:
     · TLBs, because walks are expensive
     · walk caches, because even a partial walk is expensive
     · huge pages, because TLB reach is the limit
     · the two-register split, because context switches must not
       flush the TLB
     · ASIDs, for the same reason
     · caches at three levels, because DRAM is 50x a register
```

The parallel picture

```
One load, and a second core doing the same thing:

   Core 0                          Core 1
   ldr x1, [x0]                    str x2, [x0]
      │                               │
      │ both translate the same VA    │
      │ (same ASID, or a Global page) │
      ▼                               ▼
   L1 D-cache                      L1 D-cache
      │                               │
      └──────── COHERENCY ────────────┘        chapters 17, 19
                MESI over ACE/CHI
                automatic WITHIN the shareability domain
                     │
                     ▼
              same physical line

  Coherency is free. ORDERING is not:                chapters 20, 21
     · these two accesses to the SAME address are coherent and
       ordered — that is guaranteed
     · accesses to DIFFERENT addresses may be observed in any
       order unless you use a barrier or an ordered access

  And ATOMICITY is a third thing again:                  chapter 22
     · read-modify-write needs exclusives or LSE
     · which need SHAREABLE, cacheable memory to work at all

  Three separate guarantees, three separate mechanisms, three
  distinct failure modes. Diagnose which one you have before
  reaching for a fix.
```

The isolation stack

```
The same load, seen as a security question — "who is allowed to
touch this?"

   ┌──────────────────────────────────────────────────────┐
   │ EL0 application                                      │
   │   constrained by: stage-1 AP bits, UXN               │
   ├──────────────────────────────────────────────────────┤
   │ EL1 kernel                          chapters 14, 26  │
   │   constrained by: PXN, PAN, WXN, and its own tables  │
   ├──────────────────────────────────────────────────────┤
   │ EL2 hypervisor                      chapters 16, 31  │
   │   constrains EL1 via stage 2 and traps               │
   ├──────────────────────────────────────────────────────┤
   │ EL3 secure monitor                                   │
   │   owns SCR_EL3.NS — the security state itself        │
   ├──────────────────────────────────────────────────────┤
   │ MMU / SMMU              translation + permissions    │
   ├──────────────────────────────────────────────────────┤
   │ INTERCONNECT FIREWALL                    chapter 25  │
   │   the last checkpoint: master ID + address + NS bit  │
   ├──────────────────────────────────────────────────────┤
   │ DRAM · peripherals · fuses                           │
   └──────────────────────────────────────────────────────┘

  Every access is checked at least twice: once by the page tables
  (is this VA mapped, and readable?) and once by the interconnect
  (is this MASTER allowed here, in this security state?).

  Fail either and you get a fault — but a DIFFERENT fault, which
  is exactly the distinction above.

  The security track takes this stack apart in detail.
```

The diagnostic procedure

```
Given any memory-related failure on ARM:

  1. READ ESR FIRST.
       EC tells you the kind. DFSC tells you the cause.
       0x24 vs 0x25 tells you user vs kernel.        chapter 29

  2. Was there a FAULT at all?
       yes → continue at 3
       no  → this is a coherency, ordering or atomicity problem.
             Which one?                          chapters 19–22
               cores durably disagree  → shareability
               wrong order observed    → barriers
               update lost             → atomics

  3. TRANSLATION fault?
       → the address is not mapped. Check the page tables,
         and check the TLB was invalidated.       chapters 13, 15
       → use AT + PAR_EL1 to ask the hardware.        chapter 12

  4. PERMISSION fault?
       → it IS mapped. Check AP, XN, and PAN.         chapter 14

  5. EXTERNAL abort?
       → translation succeeded. This is a PLATFORM problem:
           · is the peripheral clocked and powered?
           · is the address right?
           · is a firewall denying this master?
         Check clocks FIRST — it is the most common cause and
         the cheapest to rule out.                    chapter 25

  6. SError?
       → imprecise. ELR is meaningless. Use ESB to bisect, and
         read the interconnect's latched error registers.

  7. No fault, wrong DATA?
       → memory type (cached MMIO?), or cache maintenance
         (DMA?).                                 chapters 11, 18
```

The tools, in one place

```
# ask the hardware to translate an address           chapter 12
        at      s1e1r, x0
        mrs     x1, par_el1
        /* success: PA + attributes.  failure: F=1 + fault code */

# what did the kernel map?
cat /sys/kernel/debug/kernel_page_tables
cat /proc/self/maps
cat /proc/iomem

# decode a fault                                     chapter 29
dmesg | grep -A25 'Internal error'
scripts/faddr2line vmlinux my_func+0x48

# is it firing?                                      chapter 24
cat /proc/interrupts
cat /sys/kernel/debug/irq/irqs/24

# what is the microarchitecture doing?               chapter 34
perf stat -e cycles,instructions,l1d_cache_refill,\
dtlb_walk,br_mis_pred_retired,stall_frontend,stall_backend ./prog

# coherency and sharing
perf c2c record ./prog && perf c2c report

# is this device coherent?                           chapter 19
grep -l dma-coherent /proc/device-tree/*/*/

# what does this CPU actually implement?             chapter 01
cat /proc/cpuinfo
lscpu -C
```

How the chapters connect

If you are debugging…	Read
A crash with an ESR value	29, then 13 or 14
MMIO that behaves strangely	11, 21, 25
DMA corruption	18, 19
A multi-core race	20, 21, 22
An interrupt that never fires, or never stops	24, then 23
A page table change with no effect	15
Slow code	34, 10, CA 18
A guest behaving oddly	16, 31
Boot that dies silently	12, 27, OS 18
A core that will not start or wake	33
Hand-written assembly corrupting things	04, 09

Check yourself

✓ Name the fourteen steps a load passes through, and which chapter covers each.

- ✓ Give the failure mode at every step and the ESR field that identifies it.
- ✓ Explain why translation fault and external abort send you to different subsystems.
- ✓ Give the cost of a TLB hit, a TLB miss, and a two-stage walk.
- ✓ Distinguish coherency, ordering and atomicity by mechanism and by symptom.
- ✓ Draw the isolation stack and say what checks an access twice.
- ✓ Walk the seven-step diagnostic procedure from an unknown failure to a subsystem.

TRACK COMPLETE

That closes the ARM track: the architecture and its licensing model, the register file and instruction set, the procedure call standard, the pipeline, memory types, the MMU and page walk, descriptors, TLBs, two-stage translation, caches and maintenance, shareability, the memory model, barriers, atomics, the GIC, AMBA, exception levels, vectors, entry and return, ESR decoding, the syscall path, traps, timers, power and PSCI, and the PMU.

Next is isolation and platform security — the SMMU, TrustZone and interconnect firewalls — which is built entirely on the mechanisms in this track: the NS bit on the bus, the descriptors' security attributes, stage-2 translation, and the exception model.

TRACK

Isolation & platform security

SMMU, TrustZone and interconnect firewalls — how a SoC keeps one piece of software away from another's memory.

20 chapters

The threat model of a modern SoC

Before any mechanism makes sense you need to know what it defends against. This chapter sets out who the adversaries are, what they can reach, and why a chip needs four independent isolation layers rather than one good one.

What is worth protecting

Asset	Loss means	Typical location
Device identity keys	The device can be cloned; attestation is worthless	Fuses, secure storage
Disk encryption keys	All user data at rest	Derived in the secure world from a hardware key
Firmware key hashes	An attacker can run arbitrary firmware — total compromise	One-time-programmable fuses
Biometric templates	Irreplaceable — a user cannot change their fingerprint	Secure world only
DRM content keys	Commercial loss; loss of certification	Trusted application
Payment credentials	Direct financial loss	Secure element or TEE
Anti-rollback counters	Old, vulnerable firmware can be reinstalled	Fuses
Integrity of running code	Everything else	Everywhere

Who you are defending against

```
Ordered by capability. Each level can do everything below it.

  1. REMOTE ATTACKER
       sends network traffic, malicious files, malformed input
       defeated by: memory safety, input validation, sandboxing

  2. LOCAL UNPRIVILEGED CODE
       an app on the device, or a compromised process
       defeated by: the MMU, process isolation, seccomp, SELinux

  3. LOCAL PRIVILEGED CODE  (root / EL1 compromise)
       full control of the OS kernel
       THIS is where the security track begins. Everything up to
       here is the OS's job; from here on the OS is ASSUMED
       COMPROMISED.
       defeated by: TrustZone, the SMMU, interconnect firewalls

  4. MALICIOUS OR BUGGY PERIPHERAL
       a DMA-capable device programmed to read anywhere
       defeated by: the SMMU / IOMMU

  5. PHYSICAL ATTACKER, non-invasive
       JTAG probe, bus probing, cold-boot RAM extraction,
       glitching the clock or voltage
       defeated by: fused-off debug, memory encryption, glitch
       detectors, redundant checks

  6. PHYSICAL ATTACKER, invasive
       decapping the die, microprobing, focused ion beam
       defeated by: shields, sensors, and ultimately economics.
       You raise the cost; you do not eliminate the threat.

MOST embedded security work targets levels 3 and 4. Those are
where a mechanism can give a hard guarantee.
```

THE ASSUMPTION THAT DEFINES THIS TRACK

Assume the rich OS will be compromised. Linux is tens of millions of lines with a continuous stream of CVEs. Designing as though a kernel compromise is impossible is designing for a world that does not exist.

So the question becomes: *what still holds when EL1 is fully attacker-controlled?* Everything in this track answers that question, and it is why the mechanisms live in *hardware*, below the kernel, rather than in it.

Four independent layers

```
   +-----------------------------------------------------+
   | 1. MMU — per-process isolation                      |
   |    constrains: CPU accesses                         |
   |    configured by: the OS kernel                     |
   |    defeated by: a kernel compromise                 |
   |    see: ARM chapters 12-16                          |
   +-----------------------------------------------------+
   | 2. SMMU / IOMMU — per-device isolation              |
   |    constrains: DMA from peripherals                 |
   |    configured by: the OS, or a hypervisor           |
   |    defeated by: whoever owns the tables             |
   |    see: chapters 02-07                              |
   +-----------------------------------------------------+
   | 3. TrustZone — world isolation                      |
   |    constrains: EVERYTHING, by security state        |
   |    configured by: EL3 firmware, before the OS runs  |
   |    survives: a full kernel compromise               |
   |    see: chapters 08-16                              |
   +-----------------------------------------------------+
   | 4. XPU / interconnect firewall                      |
   |    constrains: every transaction on the bus         |
   |    configured by: firmware, then LOCKED until reset |
   |    survives: even a compromised secure world        |
   |    see: chapters 17-20                              |
   +-----------------------------------------------------+

WHY FOUR? Each has a different failure mode:

   · the MMU is configured by software you may not trust
   · the SMMU only sees transactions that pass through it, and
     some masters have no SMMU at all
   · TrustZone splits into two worlds, but everything inside one
     world is mutually trusting
   · the XPU is the last checkpoint and can be locked, but it is
     coarse-grained

No single layer covers everything. That is not redundancy — it is
coverage of disjoint threats.
```

Enumerate every master

> **THE CPU IS ONE MASTER AMONG MANY**
>
> A modern SoC has a dozen or more agents that issue memory transactions independently. Every one is a potential path to your assets, and the ones people forget are the dangerous ones.

```
  +------+ +------+ +------+ +------+ +------+ +------+
  | CPU  | | GPU  | | DSP  | | NPU  | | DMA  | | USB  |
  +------+ +------+ +------+ +------+ +------+ +------+
  +------+ +------+ +------+ +------+ +------+ +------+
  |Video | |Camera| |Crypto| | PCIe | | WiFi | |Sensor|
  |codec | |  ISP | |engine| |      | |modem | | hub  |
  +------+ +------+ +------+ +------+ +------+ +------+
  +----------------+ +-------------+ +------------------+
  | Power mgmt MCU | | Debug / JTAG| | Test / DFT ports |
  +----------------+ +-------------+ +------------------+
    ^^^ THE ONES PEOPLE FORGET — often with no SMMU in front
        of them, and full access to memory.

For each master, ask three questions:
   1. What can it address?
   2. Who programs it, and from which security state?
   3. Is there an SMMU or firewall between it and memory?

A single "no" to question 3, on a master a compromised kernel
can program, bypasses everything above it.
```

Attack paths worth knowing

Path	How	Defence
DMA attack	Program a device to read or write arbitrary physical memory	SMMU (ch. 02)
Confused deputy	Trick privileged code into using its authority for you — pass it a pointer into memory you cannot reach	Validate every incoming address (ch. 16)
TOCTOU	Change a value between the check and the use	Copy into private memory, *then* validate
Downgrade	Install older, correctly signed but vulnerable firmware	Anti-rollback fuse counters (ch. 14)
Debug port	Attach JTAG on a production device	Fuse-disabled secure debug (ch. 10)
Cold boot	Freeze DRAM, move it, read the keys	Memory encryption, key scrubbing on suspend
Side channel	Infer secrets from cache timing, power draw or EM emissions	Constant-time code (crypto 19)
Speculation	Spectre-class leaks across a privilege boundary	`CSDB`, index masking (CA 12)
Fault injection	Glitch voltage or clock to skip a signature check	Redundant checks, randomised delays, detectors
Supply chain	Compromise the build, the signing key, or a vendor blob	Reproducible builds, HSM-held keys, review

The trusted computing base

> **TCB: EVERYTHING THAT MUST BE CORRECT FOR SECURITY TO HOLD**
>
> If any component has a bug, the guarantee fails. So the discipline is simple to state and hard to follow: **make the TCB as small as possible**, and be honest about what is in it.

```
A typical TCB for "the disk encryption key stays secret":

   boot ROM                   ~10 KB, immutable, unauditable
   BL2 trusted boot firmware  ~100 KB
   BL31 EL3 monitor (TF-A)    ~50 KB, RESIDENT forever
   trusted OS (OP-TEE)        ~300 KB
   the trusted application    ~10 KB
   the hardware               fuses, crypto engine, TZASC,
                              the SMMU, the interconnect
   ------------------------------------------------------
   TOTAL                      ~500 KB of code

  NOT in the TCB: Linux, Android, every app, every driver.
  Tens of millions of lines that can be fully compromised
  without exposing the key.

  That ratio — 500 KB versus 30 M lines — IS the value of
  TrustZone. And it is why the design principle is:

     EVERY BYTE added to the secure world enlarges the attack
     surface of the most privileged code on the chip. A
     vulnerability in a trusted application is WORSE than one in
     the kernel.

  So a TEE should do as little as possible: hold keys, perform
  one operation, return a result. Not parse untrusted formats.
  Not implement a filesystem. Not run a network stack.
```

Design principles that actually help

Principle	In practice
Least privilege	Every master gets the minimum access it needs; every trusted app gets one capability
Fail closed	The default rule denies. Anything you forgot to configure is refused, not permitted.
Complete mediation	Every access checked, every time. No cached "already allowed" decisions.
Defence in depth	The four layers above. Assume any one will be bypassed.
Economy of mechanism	A small TCB you can audit beats a large one you cannot
Open design	Security from the key, not from secrecy of the mechanism
Separation of duties	The code that verifies is not the code that executes
Lock what will not change	Configuration set before the OS starts should be immutable afterwards

Device lifecycle states

```
Silicon moves through fuse-controlled states, each disabling more
capability. This is why "it works on my dev board" is not
evidence that it works on a production part.

   BLANK / VIRGIN
      no keys fused, debug fully open, secure boot OFF
      for silicon bring-up

   DEVELOPMENT
      test keys fused, debug open, secure boot may run with
      development keys

   SECURED / PRODUCTION
      production key hash fused, secure boot ENFORCED,
      SECURE DEBUG DISABLED, test ports closed
      this is what ships

   RMA / RETURN
      a controlled path to re-enable debug for failure analysis,
      usually requiring a SIGNED unlock token and often
      destroying the device keys first

  Transitions are ONE-WAY, enforced by one-time-programmable
  fuses. You cannot go back.

  CONSEQUENCES you will meet:
     · JTAG stops working on production silicon
     · a peripheral readable on the dev board aborts in the field
     · timing changes because a debug path was removed
     · firmware that worked now fails signature checks

  Always test on a part in the FINAL lifecycle state before
  shipping.
```

What these mechanisms do not do

> **BE HONEST ABOUT THE LIMITS**
>
> - **TrustZone does not stop physical attacks.** An attacker with the die and a probe station reads the bus directly.
> - **A TEE is not a secure element.** It shares the CPU, caches and DRAM with the normal world, so it inherits every side channel. A separate secure element with its own die is a stronger — and slower, and more expensive — guarantee.
> - **Secure boot does not protect a running system.** It verifies what loads; it says nothing about what happens afterwards. Runtime attestation is a separate mechanism.
> - **An SMMU does not help if it is not configured.** Devices left in bypass are unprotected, and this is a very common misconfiguration.
> - **None of this fixes bugs.** A buffer overflow in a trusted application is still a buffer overflow — with more privilege.
>
> Stating the limits precisely is not pessimism; it is what lets you pick the right mechanism for the actual threat.

How this track is organised

Chapters	Cover	Defend against
02–07	The SMMU: why DMA breaks isolation, StreamIDs, stream tables, stages, faults, and the DMA API	Adversary 4 — peripherals
08–16	TrustZone: the NS bit, the two worlds, hardware enforcement, SMC, the TEE stack, secure boot, TrustZone-M, attacks	Adversary 3 — a compromised OS
17–20	Interconnect firewalls: concepts, programming, policy design, debugging	Everything, as the last checkpoint

Every mechanism here is built on the ARM track: the NS bit is `AxPROT[1]` on the bus (ARM 25), stage-2 translation is ARM 16, descriptor security attributes are ARM 14, and the world switch uses the exception model of ARM 28. If those are not solid, read them first.

How it shows up

Symptom	Underlying issue
Works on the dev board, fails in the field	Lifecycle state changed; fuses tightened the configuration
JTAG cannot attach to production silicon	Secure debug fused off — by design
A peripheral reads as zeros only on shipping units	A firewall or TZPC assignment the dev board lacked
A device reads memory it should not	No SMMU, or the SMMU left in bypass
A TEE vulnerability gives full device compromise	Too much code in the secure world
Firmware downgrade reinstates a fixed bug	No anti-rollback counter
Keys recovered from a powered-off device	Cold boot; no memory encryption or key scrubbing
Secure boot passes but the system is compromised at runtime	Boot verification is not runtime integrity

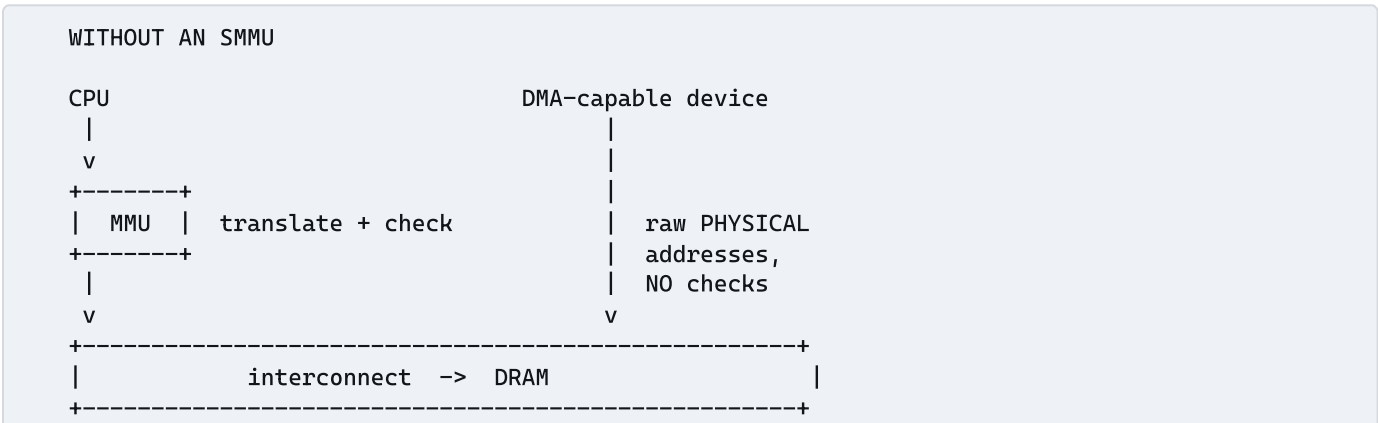
Check yourself

- ✓ Name six adversary classes in order of capability and say where this track begins.
- ✓ State the assumption that motivates hardware-based isolation.
- ✓ Give the four isolation layers, what each constrains, and what defeats each.
- ✓ List five bus masters people forget, and the three questions to ask about each.
- ✓ Define the TCB and explain why its size is the central design concern.
- ✓ Explain the confused deputy and TOCTOU patterns.
- ✓ Name the device lifecycle states and three things that change in the secured state.
- ✓ Give four things these mechanisms explicitly do not protect against.

Why DMA breaks memory protection

The MMU sits between a CPU core and the interconnect. A DMA engine does not go through a CPU core. Every isolation guarantee built out of page tables therefore has a hole in it, and closing that hole is what the SMMU exists to do.

The gap



WHAT THIS MEANS CONCRETELY

An attacker who can program a DMA engine — which on most systems means an attacker who has compromised the kernel, or who can plug in a Thunderbolt device — can:

- Read kernel memory, including disk encryption keys held in RAM.
- Overwrite kernel code or the page tables, escalating from device access to full control.
- Overwrite another VM's memory, escaping the hypervisor.
- Do all of this *with the CPU's MMU perfectly configured*, because the MMU was never involved.

Real attack classes

Attack	Vector	Note
Thunderbolt / PCIe hotplug	Plug a malicious device into a running machine and DMA to arbitrary memory	The "evil maid" attack. Thunderclap (2019) demonstrated it against every major OS.
FireWire	The original — FireWire allowed unmediated DMA by design	Used for years to unlock laptops by patching the password check in RAM
Compromised device firmware	A NIC, GPU or storage controller running attacker code, DMA-ing anywhere	Firmware is rarely audited and often updatable

Attack	Vector	Note
Buggy device	Not malicious — a hardware or firmware bug writing outside its buffer	Far more common than deliberate attacks, and just as destructive
Driver bug	A driver programs a device with an attacker-influenced address	Turns a driver bug into arbitrary physical write
VM escape	A guest with a passed-through device programs it with guest-physical addresses	Without stage-2 IOMMU translation, those are treated as host-physical

Four problems, one solution

```
The SMMU is usually described as a security feature. It solves
FOUR distinct problems, and only one is security:

  1. ISOLATION (security)
       constrain a device to the memory it was given
       -> the whole of this chapter

  2. VIRTUALISATION
       a guest programs a device with GUEST-physical addresses.
       Without translation, the device uses them as HOST-physical
       and scribbles over the host or another guest.
       -> stage 2 makes device passthrough possible at all

  3. SCATTER-GATHER
       a device wanting one large contiguous buffer forces the OS
       to find contiguous PHYSICAL memory, which fragments and
       eventually fails (OS chapter 11).
       -> with translation, physically scattered pages appear
          contiguous to the device

  4. ADDRESS WIDTH
       a legacy 32-bit device cannot reach memory above 4 GB.
       Without translation you need slow BOUNCE BUFFERS: copy the
       data down below 4 GB, DMA, copy back.
       -> with translation, any physical page can be mapped into
          the device's reachable range

  Problems 3 and 4 are why IOMMUs existed long before anyone
  worried about DMA attacks. The security benefit came later,
  and is now the main reason they are mandatory.
```

Putting an MMU in front of the device

```
   WITH AN SMMU

   CPU                              DMA-capable device
    |                                     |
    v                                     v
   +-------+                         +-----------+
   |  MMU  |                         |   SMMU    |
   +-------+                         +-----------+
    |   translate + check              |   translate + check,
    |                                  |   BY STREAMID
    v                                  v
   +-------------------------------------------------+
   |            interconnect  ->  DRAM                |
   +-------------------------------------------------+

  Same idea, same descriptor format, same permission bits. The
  difference is that the SMMU serves MANY devices through one
  block, so it needs the transaction to say WHO is asking.

  That identifier is the STREAMID, and it is the subject of the
  next chapter.

  Terminology: ARM calls it the SMMU (System MMU). The generic
  term is IOMMU. Intel's is VT-d, AMD's is AMD-Vi, and they
  differ in programming model but not in concept.
```

Why the MMU alone is not enough

```
/* A natural question: why not just have the device use virtual
   addresses through the CPU's MMU?

   Three reasons it cannot work:

   1. WHICH page tables? The CPU's MMU uses TTBR0/TTBR1 for the
      CURRENTLY RUNNING process. A device operates asynchronously
      - the process that started the transfer may not be running,
      may have exited, or may be on another core.

   2. The MMU is INSIDE the core. Physically, a device's
      transactions do not pass through it. Routing them there
      would mean routing all DMA through a CPU, which defeats the
      point of DMA entirely.

   3. Devices need their own POLICY. A device should be confined
      to the buffers it was given - not to whatever the process
      that programmed it can see. Those are different sets.

   Hence a separate translation unit, on the device side of the
   interconnect, with its own tables and its own notion of
   identity.                                                  */
```

A second, separate problem

> **ISOLATION AND COHERENCY ARE DIFFERENT ISSUES**
>
> DMA creates two problems and they are often confused:
>
> - **Isolation** — the device can reach memory it should not. Solved by the SMMU.
> - **Coherency** — the device and the CPU disagree about the *contents* of memory it legitimately reached. Solved by cache maintenance or by hardware coherency (ARM 18, ARM 19).
>
> An SMMU does not make DMA coherent, and cache maintenance does not make it isolated. A driver needs to get both right, which is exactly why the DMA API exists to handle them together (chapter 07).

Where the SMMU sits

```
Not every master goes through an SMMU. This is a SoC
INTEGRATION decision, and it is the first thing to establish
about any platform:

   +------+  +------+  +------+  +------+  +------+
   | GPU  |  | NPU  |  | DMA  |  | PCIe |  | PMU  |
   +--+---+  +--+---+  +--+---+  +--+---+  |micro-|
      |         |         |         |      |cntlr |
   +--v---------v---------v---------v--+   +--+---+
   |            SMMU                   |      |
   +-----------------+-----------------+      | NO SMMU
                     |                        | at all
   +-----------------v------------------------v------------+
   |                    interconnect                        |
   +--------------------------------------------------------+

  The power-management microcontroller, a sensor hub, or a
  debug/test port may have direct memory access with nothing in
  front of it. Those masters are outside the SMMU's protection
  entirely, and can only be constrained by an interconnect
  FIREWALL (chapters 17-20).

  So the layered design of chapter 01 is not theoretical: some
  masters are covered by layer 2, some only by layer 4, and you
  must know which is which for your SoC.
```

```
# what does Linux think is behind an IOMMU?
ls /sys/kernel/iommu_groups/
ls -l /sys/kernel/iommu_groups/*/devices/

# is a specific device translated?
ls /sys/bus/pci/devices/0000:01:00.0/iommu_group
ls /sys/bus/platform/devices/*/iommu_group

# a device with NO iommu_group symlink is NOT behind an SMMU
for d in /sys/bus/platform/devices/*; do
  [ -e "$d/iommu_group" ] || echo "no IOMMU: $(basename $d)"
done
```

IOMMU groups

```
The SMMU can only distinguish devices whose transactions carry
DIFFERENT StreamIDs. Sometimes it cannot.

  A PCIe bridge that does not support Access Control Services
  may allow peer-to-peer traffic between devices behind it -
  traffic that never reaches the SMMU at all. Two such devices
  can DMA to each other regardless of what the SMMU says.

  Linux therefore groups devices that CANNOT be isolated from
  each other into one IOMMU GROUP, and treats the group as the
  smallest unit of assignment:

     /sys/kernel/iommu_groups/12/devices/
         0000:01:00.0      <- a GPU
         0000:01:00.1      <- its audio function

  CONSEQUENCE: you cannot pass one device of a group to a VM and
  keep the other on the host. VFIO refuses, because doing so
  would be a false isolation guarantee.

  This is a very common source of "why can't I pass through my
  GPU" - the answer is usually that its group contains something
  else, and the fix is a different slot, ACS support, or (with
  full understanding of the risk) an ACS override.
```

Bypass: the most common misconfiguration

> **AN SMMU THAT IS NOT CONFIGURED PROTECTS NOTHING**
>
> A device whose StreamID has no valid Stream Table Entry can be handled in two ways: **abort** every transaction, or **bypass** translation entirely. Bypass means the device operates exactly as though no SMMU existed.
>
> Bypass is convenient during bring-up and catastrophic in production. Several real systems have shipped with it enabled because a device did not work otherwise and someone turned it on to unblock testing.

```
# Linux kernel command line options that matter here
iommu.passthrough=1              # ALL devices bypass - debug only
arm-smmu-v3.disable_bypass=1     # abort unmatched streams (default
                                 # since 4.20, and the correct setting)
iommu.strict=1                   # invalidate TLBs synchronously on
                                 # unmap. strict=0 is faster and
                                 # leaves a window where a freed page
                                 # is still reachable by the device.

# check what is actually in force
cat /proc/cmdline
dmesg | grep -iE 'smmu|iommu'
#   arm-smmu-v3 2b400000.iommu: ias 48-bit, oas 48-bit (features 0x...)
#   arm-smmu-v3 2b400000.iommu: allocated 65536 entry cmdq
#   iommu: Default domain type: Translated

# "Default domain type: Passthrough" means devices are NOT
# being translated. On a production system that is a finding.
```

What it costs

Cost	Magnitude	Mitigation
Translation latency on a TLB miss	A page-table walk per miss, up to 24 accesses with two stages (ARM 16)	SMMU TLBs, walk caches, huge pages
Map/unmap cost per transfer	Page table edits plus invalidation commands	Persistent mappings, larger buffers, deferred invalidation
Invalidation latency	A command queue round trip plus `CMD_SYNC`	`iommu.strict=0` — faster, and less safe
Memory for tables	Stream tables scale with the StreamID space	Two-level stream tables
Throughput on small transfers	Can be 10–30% on high-rate networking	Larger buffers, or ATS (ch. 05)

These costs are why bypass is tempting and why `iommu.strict=0` exists. Both trade a real security property for measurable performance, and both should be a deliberate, documented decision rather than an accident.

How it shows up

Symptom	Cause
A device writes outside its buffer and corrupts the kernel	No SMMU, or the device is in bypass
Plugging in a Thunderbolt device compromises the machine	Unmediated DMA — the reason IOMMUs became mandatory on laptops
Guest with a passed-through device corrupts the host	No stage-2 IOMMU translation
Large DMA allocation fails on a long-running system	Physical fragmentation; scatter-gather via the SMMU avoids it
32-bit device transfers are slow	Bounce buffers; an SMMU removes the need
Cannot pass a GPU to a VM	Its IOMMU group contains other devices
`Default domain type: Passthrough` in dmesg	Translation disabled — a production finding
Networking throughput drops ~20% after enabling the IOMMU	Map/unmap and invalidation cost on small packets
DMA reads stale data even with the SMMU configured	A coherency problem, not an isolation one

Check yourself

- ✓ Explain in one sentence why the CPU's MMU does not constrain DMA.
- ✓ Give three things an attacker with DMA access can do.
- ✓ Name the four problems an IOMMU solves and say which two predate the security concern.
- ✓ Give three reasons a device cannot simply use the CPU's page tables.
- ✓ Distinguish the isolation problem from the coherency problem, with a symptom of each.
- ✓ Explain why some masters have no SMMU and what constrains them instead.
- ✓ Explain what an IOMMU group is and why VFIO refuses to split one.
- ✓ Say what bypass mode is and why it is a production finding.

SMMU: StreamID and identity

The CPU's MMU serves one core and knows implicitly whose translation to use. The SMMU serves dozens of devices through one block, so every transaction must carry an identifier. Getting that identifier right is the single most common SMMU bring-up problem.

The identity problem

```
  CPU MMU                          SMMU
  -------                          ----
  one core                         MANY devices
  context is IMPLICIT:             context must be EXPLICIT:
    "this core, right now"           "whichever device sent this"
  selected by TTBR0/TTBR1          selected by STREAMID
    plus the ASID                    plus the SubstreamID

So the SMMU's first job on every transaction is:
   look up WHO sent this, and find their translation context.

StreamID is that "who". It arrives as sideband signalling on
the interconnect alongside the address (ARM chapter 25).
```

Where a StreamID comes from

Bus	StreamID derived from	Assigned by
PCIe	The Requester ID: bus:device:function, 16 bits	PCIe enumeration, plus a per-root-complex base offset
On-SoC master	Hard-wired by the SoC integrator, often via `AxUSER` or a dedicated sideband	The chip designer — documented in the TRM
Behind a bridge	The bridge may re-ID transactions, or pass them through	The bridge's configuration
Multi-queue device	Often one StreamID per queue or per virtual function	The device, within a range the SoC allocated

THE NUMBER ONE SMMU BUG

The device tree says StreamID `0x800`; the hardware actually emits `0x801`. The SMMU finds no valid Stream Table Entry, aborts every transaction, and the device does not work.

The fault record tells you the StreamID the SMMU *saw*. Compare that against what you configured — that single comparison resolves most SMMU bring-up problems immediately.

```
arm-smmu-v3 2b400000.iommu: event 0x10 received:
arm-smmu-v3 2b400000.iommu: 0x0000080100000010
                              ^^^^ StreamID 0x801 — but the DT
                                   said 0x800
```

Telling software the StreamID

```
/* Device tree — the embedded case */

smmu: iommu@2b400000 {
        compatible = "arm,smmu-v3";
        reg = <0x0 0x2b400000 0x0 0x100000>;
        interrupts = <GIC_SPI 74 IRQ_TYPE_EDGE_RISING>,   /* eventq   */
                     <GIC_SPI 75 IRQ_TYPE_EDGE_RISING>,   /* gerror   */
                     <GIC_SPI 77 IRQ_TYPE_EDGE_RISING>;   /* cmdq-sync */
        interrupt-names = "eventq", "gerror", "cmdq-sync";
        #iommu-cells = <1>;              /* one cell: the StreamID */
};

/* A device with ONE StreamID: */
dma_engine@2c000000 {
        compatible = "vendor,dma-engine";
        reg = <0x0 0x2c000000 0x0 0x1000>;
        iommus = <&smmu 0x42>;           /* StreamID 0x42 */
        dma-coherent;
};

/* A device with SEVERAL — one per queue, for instance: */
        iommus = <&smmu 0x40>, <&smmu 0x41>,
                 <&smmu 0x42>, <&smmu 0x43>;

/* A whole bus, with a MAPPING rather than a list — used for
   PCIe, where devices appear at enumeration time:          */
pcie@40000000 {
        /* Requester ID -> StreamID:
              <rid-base &smmu stream-base count>              */
        iommu-map = <0x0 &smmu 0x0 0x10000>;
        iommu-map-mask = <0xffff>;
};

/* And a bus whose masters are behind a bridge that re-IDs: */
        iommu-map = <0x100 &smmu 0x800 0x100>;
/*                   ^RID base    ^SID base ^count            */
```

```
/* ACPI — the server and Windows-on-ARM case.

   The IORT (I/O Remapping Table) describes the topology as a
   graph of nodes, each mapping ID ranges to the next node:

      Named component / PCIe root complex
              |  ID mapping: input range -> output base
              v
      SMMUv3 node
              |
              v
      ITS group  (for MSIs — see below)

   Same information, different encoding. Linux parses both and
   presents an identical model to drivers.                   */

# inspect it
sudo cat /sys/firmware/acpi/tables/IORT | xxd | head
sudo iasl -d /sys/firmware/acpi/tables/IORT && less IORT.dsl
```

SubstreamID

```
One device, many contexts.

   StreamID       WHICH DEVICE          "the GPU"
   SubstreamID    WHICH CONTEXT         "process 4271's GPU work"

  Called PASID on PCIe (Process Address Space ID), SubstreamID in
  ARM's terminology. Up to 20 bits.

  WHY it exists: a GPU or accelerator serves many processes at
  once. Without SubstreamIDs, all of them would share one
  translation context, so any process's GPU work could reach any
  other's buffers — no isolation at all between GPU clients.

  With them, each process gets its own Context Descriptor, and
  therefore its own page tables:

     StreamID 0x42 (the GPU)
        +-- SubstreamID 0   -> kernel context
        +-- SubstreamID 1   -> process A's page tables
        +-- SubstreamID 2   -> process B's page tables
        +-- ...

  This is also the mechanism behind SVA — Shared Virtual
  Addressing — where the device's Context Descriptor points at a
  process's OWN page tables, so the device can dereference
  exactly the pointers the application uses, with no mapping
  step at all (chapter 05).
```

How the SMMU uses it

```
 incoming transaction
      StreamID, SubstreamID, address, R/W, NS, privilege
           |
           v
 +--------------------------+
 | STREAM TABLE             |   base = SMMU_STRTAB_BASE
 |   indexed by StreamID    |   entry = STE, 64 bytes
 +--------------------------+
           |
           v
 +--------------------------+
 | STREAM TABLE ENTRY (STE) |
 |   V         valid?       |
 |   Config    bypass/abort/|
 |             S1/S2/both   |
 |   S1ContextPtr -> CD table|
 |   S2TTB     stage-2 root |
 |   VMID, attributes       |
 +--------------------------+
           |
           v
 +--------------------------+
 | CONTEXT DESCRIPTOR       |   indexed by SubstreamID
 |   TTB0/TTB1, TCR, MAIR,  |   - the SAME fields as the CPU's
 |   ASID                   |     translation context
 +--------------------------+
           |
           v
 translation tables - IDENTICAL descriptor format to the CPU's
 (ARM chapter 14)

The next chapter covers these structures in detail. The point
here is the SHAPE: StreamID selects a device, SubstreamID
selects a context within it, and from there everything is the
ordinary ARM translation machinery.
```

StreamID space and table size

```
/* A flat stream table needs 2^StreamIDSize entries of 64 bytes.
   With 16-bit StreamIDs that is 65536 * 64 = 4 MB. With 20-bit
   it is 64 MB. Allocating that when only a dozen devices exist
   is wasteful.                                              */

/* So SMMUv3 supports a TWO-LEVEL stream table: */

   SMMU_STRTAB_BASE_CFG.FMT = 1        /* two-level */

   StreamID[hi] -> L1 descriptor -> a span of L2 entries
   StreamID[lo] -> index within that span

/* Only the spans actually used are allocated. Exactly the same
   sparseness argument as multi-level page tables (ARM ch. 13). */

/* How wide is the StreamID space on this SMMU? */
        SMMU_IDR1.SIDSIZE       /* bits of StreamID supported */
        SMMU_IDR1.SSIDSIZE      /* bits of SubstreamID        */

# Linux reports it at probe:
dmesg | grep smmu
#   arm-smmu-v3 2b400000.iommu: ias 48-bit, oas 48-bit (features 0x00013fbf)
#   arm-smmu-v3 2b400000.iommu: 2-level strtab only covers 0..0xffff
```

MSIs and the StreamID

> **AN MSI IS A MEMORY WRITE, SO IT IS ALSO TRANSLATED**
>
> A device signals an interrupt by writing to an address (ARM 23). That write goes through the SMMU like any other, which means the ITS doorbell address must be *mapped* for that device — otherwise the interrupt silently never arrives, with no fault that obviously points at interrupts.
>
> Linux handles this by reserving an MSI region per device (`iommu_dma_get_resv_regions`) and mapping the ITS translater into every device's address space. When you see a device that transfers data correctly but never raises an interrupt, this is one of the first things to check.
>
> ```
> cat /sys/kernel/iommu_groups/*/reserved_regions
> #   0x0000000008080000 0x000000000808ffff msi
> #                                        ^ the ITS doorbell
> ```

When identity is ambiguous

```
Several situations make StreamIDs less trustworthy than they
look, and each is a real isolation weakness:

  ALIASING — two devices emit the SAME StreamID
       Some SoCs wire several masters to one ID, either by
       oversight or to save sideband wires. The SMMU cannot then
       distinguish them, and they must share one translation
       context. Whatever one can reach, the other can.

  PCIe WITHOUT ACS
       Peer-to-peer traffic between devices under a bridge may
       never reach the SMMU. Linux puts such devices in one
       IOMMU GROUP (chapter 02) and refuses to isolate them
       from each other.

  LEGACY PCI BRIDGES
       Devices behind a PCI-to-PCIe bridge all appear with the
       BRIDGE's Requester ID. Same consequence.

  DMA ALIASES / QUIRKS
       Some devices emit a Requester ID that is not their own —
       a documented hardware bug. Linux keeps a quirk table
       (pci_for_each_dma_alias) mapping the real device to every
       ID it might use.

  In every case the honest response is the same: treat the whole
  ambiguous set as ONE security domain, and do not claim
  isolation you do not have.
```

Verifying the StreamID

```
# 1. what does software think?
cat /proc/device-tree/soc/dma@2c000000/iommus | xxd
#   the second cell is the StreamID

# or from the driver's point of view
ls -l /sys/bus/platform/devices/2c000000.dma/iommu_group
cat /sys/kernel/iommu_groups/*/devices/* 2>/dev/null

# 2. what does the HARDWARE emit? Trigger a transaction and read
#    the fault record — the definitive answer.
dmesg | grep -A4 'event.*received'

# 3. trace map/unmap to confirm the domain is being used at all
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe
#   map: IOVA=0x... paddr=0x... size=...

# 4. SMMU driver internals
ls /sys/kernel/debug/iommu/          # where available
cat /sys/class/iommu/smmu3.0.auto/devices/
```

THE BRING-UP SEQUENCE THAT WORKS

1. Boot with `arm-smmu-v3.disable_bypass=0` temporarily, so unmatched streams bypass rather than abort. If the device now works, the problem is definitely StreamID configuration, not the driver.
2. Re-enable `disable_bypass=1` and read the fault record to learn the StreamID the hardware actually emits.
3. Fix the device tree or IORT to match.
4. Confirm with the `iommu` tracepoints that mappings are being created for the right device.

Never ship with bypass enabled — but it is the fastest way to bisect the problem during bring-up.

How it shows up

Symptom	Cause
`C_BAD_STREAMID` on every transfer	Device tree or IORT StreamID does not match the hardware
Device works with `iommu.passthrough=1` only	Confirms a StreamID or mapping problem
DMA works but interrupts never arrive	The MSI doorbell address is not mapped for that device
Two devices interfere with each other's memory	StreamID aliasing — they share one context
Cannot pass one device of a pair to a VM	They are in the same IOMMU group
Multi-queue device works on queue 0 only	Only one of its StreamIDs is described
Works on one board revision, fails on another	The integrator changed the StreamID assignment
Stream table allocation fails at boot	Flat table for a large StreamID space; needs two-level
GPU clients can see each other's buffers	No SubstreamID support, or not used

Check yourself

- ✓ Explain why the SMMU needs an explicit identifier and the CPU's MMU does not.
- ✓ Say where a StreamID comes from for a PCIe device and for an on-SoC master.
- ✓ Describe the most common SMMU bring-up failure and how the fault record resolves it.
- ✓ Write the device tree for a device with two StreamIDs, and for a PCIe root complex.
- ✓ Explain what a SubstreamID adds and give a concrete case that needs one.
- ✓ Explain why a two-level stream table exists.
- ✓ Explain why an MSI doorbell must be mapped, and the symptom when it is not.
- ✓ Give three situations where StreamID identity is ambiguous, and the correct response.

SMMU: stream tables and context descriptors

SMMUv3 is entirely table-driven: software builds structures in memory and tells the hardware where they are. There is no per-device register bank to program. This chapter walks the lookup chain from a StreamID to a page table.

The lookup chain

The Stream Table Entry

`Config[2:0]`	Behaviour	Use for
`0b000`	**Abort** — every transaction from this stream is terminated	Unconfigured devices. **The correct default.**

`Config[2:0]`	Behaviour	Use for
`0b100`	**Bypass** — no translation, no checks	Debug only (ch. 02)
`0b101`	Stage 1 only	A bare-metal OS isolating its devices
`0b110`	Stage 2 only	Device passthrough to a VM with no guest IOMMU
`0b111`	Both stages	Nested: guest IOMMU plus hypervisor confinement

```
/* Building an STE for stage-1-only translation. Field names
   follow the SMMUv3 spec; the shape is what matters.       */

struct ste {
        uint64_t w[8];          /* 64 bytes */
};

static void ste_make_s1(struct ste *e, uint64_t cd_table_pa,
                        unsigned ssid_bits)
{
        memset(e, 0, sizeof *e);

        /* word 0: valid, config, CD table pointer and format */
        e->w[0] = STE_VALID
                | FIELD_PREP(STE_CONFIG, 0b101)         /* S1 only  */
                | FIELD_PREP(STE_S1FMT, ssid_bits ? 1 : 0)
                | (cd_table_pa & STE_S1CTXPTR_MASK)
                | FIELD_PREP(STE_S1CDMAX, ssid_bits);

        /* word 1: fault behaviour and attribute handling */
        e->w[1] = FIELD_PREP(STE_S1DSS, 0b01)   /* SSID 0 for no-SSID
                                                   transactions      */
                | FIELD_PREP(STE_S1CIR, 0b01)   /* cacheable CD fetch */
                | FIELD_PREP(STE_S1COR, 0b01)
                | FIELD_PREP(STE_S1CSH, 0b11)   /* inner shareable    */
                | FIELD_PREP(STE_STRW, 0b00)    /* EL1&0 regime       */
                | STE_S1STALLD;                  /* terminate rather
                                                    than stall        */

        /* the ZEROED and populated entry must be visible to the
           SMMU before it is marked valid - the SMMU is a separate
           observer reading memory directly (ARM chapter 13)     */
        __asm__ volatile("dsb ishst" ::: "memory");
}

/* And a stage-2 STE, for passthrough to a VM: */
static void ste_make_s2(struct ste *e, uint64_t s2_pgd_pa,
                        unsigned vmid)
{
        memset(e, 0, sizeof *e);
        e->w[0] = STE_VALID | FIELD_PREP(STE_CONFIG, 0b110);
        e->w[2] = FIELD_PREP(STE_S2VMID, vmid)
                | FIELD_PREP(STE_S2T0SZ, 64 - ipa_bits)
                | FIELD_PREP(STE_S2SL0, start_level)
                | FIELD_PREP(STE_S2TG, granule)
                | STE_S2AA64 | STE_S2R;         /* record faults */
        e->w[3] = s2_pgd_pa & STE_S2TTB_MASK;
        __asm__ volatile("dsb ishst" ::: "memory");
}
```

STE AND CD MEMORY HAS ATTRIBUTE REQUIREMENTS

The SMMU fetches these structures itself, as a separate bus master. The memory holding them must be accessible to it and must have *consistent* attributes: if the CPU writes them as cacheable Inner Shareable while the SMMU is configured to fetch them non-cacheable, the SMMU reads stale data. That is a mismatched-attributes bug (ARM 19) with a particularly confusing symptom — a table that looks correct in a memory dump but does not work.

The `S1CIR` / `S1COR` / `S1CSH` fields in the STE, and `SMMU_CR1`'s equivalents for the stream table itself, must match how software writes them.

Two-level stream tables

```
A flat table needs 2^SIDSIZE entries of 64 bytes:

   16-bit StreamID -> 65536 * 64 =  4 MB
   20-bit          -> 1M    * 64 = 64 MB

Wasteful when a dozen devices exist. So:

   SMMU_STRTAB_BASE_CFG.FMT = 1        two-level

   StreamID[hi]  -> L1 descriptor (8 bytes)
                       V, SPAN, L2 table address
   StreamID[lo]  -> index into that L2 table of STEs

   +-------------+
   | L1 desc 0   |--> [ STE ][ STE ][ STE ] ... (SPAN entries)
   | L1 desc 1   |    invalid - no L2 table allocated
   | L1 desc 2   |--> [ STE ][ STE ] ...
   +-------------+

  Only the spans actually used are allocated. Exactly the
  sparseness argument of multi-level page tables (ARM ch. 13).

  SPAN encodes log2(entries + 1), so a span of 0 means the L1
  descriptor is invalid - a common off-by-one when building
  these by hand.
```

The Context Descriptor

```
/* The CD is the SMMU's version of a CPU translation context.
   Compare field by field with ARM chapter 12:              */

   CD field        CPU equivalent
   --------        --------------
   TTB0, TTB1      TTBR0_EL1, TTBR1_EL1
   T0SZ, T1SZ      TCR_EL1.T0SZ / T1SZ
   TG0, TG1        TCR_EL1.TG0 / TG1
   IPS             TCR_EL1.IPS
   IRGN, ORGN, SH  TCR_EL1 walk cacheability and shareability
   EPD0, EPD1      TCR_EL1.EPD0 / EPD1  (disable a TTBR)
   MAIR            MAIR_EL1
   ASID (A)        TTBR0_EL1[63:48]
   AA64            execution state
   nSCFG, ASET     SMMU-specific: NS handling, ASID scope

/* Which is the whole point: the SMMU can be pointed at the
   SAME translation tables a process is using, with the SAME
   ASID, and the device then dereferences exactly the pointers
   the application uses. That is SVA (chapter 05).         */

static void cd_make(struct cd *c, uint64_t ttbr0, unsigned asid)
{
        memset(c, 0, sizeof *c);
        c->w[0] = CD_VALID | CD_AA64
                | FIELD_PREP(CD_T0SZ, 64 - va_bits)
                | FIELD_PREP(CD_TG0, granule_code)
                | FIELD_PREP(CD_IRGN0, 0b01)    /* WB cacheable walks */
                | FIELD_PREP(CD_ORGN0, 0b01)
                | FIELD_PREP(CD_SH0, 0b11)      /* inner shareable    */
                | CD_EPD1                        /* no TTBR1 for a
                                                    device context    */
                | FIELD_PREP(CD_ASID, asid);
        c->w[1] = ttbr0 & CD_TTB0_MASK;
        c->w[3] = mair_value;
        __asm__ volatile("dsb ishst" ::: "memory");
}
```

CD tables

```
Like stream tables, CD tables can be linear or two-level,
depending on how many SubstreamIDs a device uses:

   LINEAR    STE.S1Fmt = 0
             one contiguous array of 2^S1CDMax entries
             fine for a handful of contexts

   TWO-LEVEL STE.S1Fmt = 1 or 2
             for a device with thousands of contexts – a GPU
             serving every process on the system

  And the degenerate case: a device with NO SubstreamIDs uses a
  single CD, with STE.S1CDMax = 0. The CD pointer then points
  directly at one 64-byte descriptor rather than a table.

  STE.S1DSS controls what happens to a transaction that arrives
  WITHOUT a SubstreamID on a device that has them:
       0b00  terminate
       0b01  use SubstreamID 0
       0b10  bypass stage 1

  Getting S1DSS wrong gives a device whose SubstreamID-tagged
  traffic works while its untagged traffic aborts – a confusing
  partial failure.
```

The few registers that exist

Register	Holds
`SMMU_IDR0-5`	Read-only capability: StreamID width, supported granules, stage support, ATS, PRI, coherency of table walks
`SMMU_CR0`	Global enable: `SMMUEN`, `EVENTQEN`, `CMDQEN`, `PRIQEN`
`SMMU_CR1`	Table and queue memory attributes — must match how software writes them
`SMMU_CR2`	Private TLB behaviour, `RECINVSID` (record invalid-StreamID events)
`SMMU_STRTAB_BASE`	Physical address of the stream table
`SMMU_STRTAB_BASE_CFG`	Format (linear or 2-level), `LOG2SIZE`, `SPLIT`
`SMMU_CMDQ_BASE/PROD/CONS`	The command queue
`SMMU_EVENTQ_BASE/PROD/CONS`	The event queue
`SMMU_PRIQ_*`	The page-request queue
`SMMU_GERROR/GERRORN`	Global errors — a malformed command, a queue overflow

That is nearly the whole MMIO interface. Everything else — every device's configuration — lives in memory structures. This is a deliberate contrast with SMMUv2, which had a bank of registers per "context bank" and therefore a hard limit on how many contexts could exist.

Bringing an SMMU up

```
/* The order matters, and several steps are easy to omit. */

/* 1. read capabilities BEFORE assuming anything */
        idr0 = readl(base + SMMU_IDR0);
        idr1 = readl(base + SMMU_IDR1);
        sid_bits   = FIELD_GET(IDR1_SIDSIZE, idr1);
        ssid_bits  = FIELD_GET(IDR1_SSIDSIZE, idr1);
        has_s1     = idr0 & IDR0_S1P;
        has_s2     = idr0 & IDR0_S2P;
        coherent   = idr0 & IDR0_COHACC;    /* are table walks
                                               coherent with the CPU? */

/* 2. DISABLE the SMMU while configuring */
        writel(0, base + SMMU_CR0);
        wait_for_ack(SMMU_CR0ACK, 0);

/* 3. allocate and ZERO the stream table. Every STE must be
      valid-bit-clear or explicitly set to ABORT — never left as
      whatever memory happened to contain.                    */
        strtab = alloc_zeroed(strtab_size);
        for (i = 0; i < nr_entries; i++)
                ste_make_abort(&strtab[i]);

/* 4. allocate the command, event and PRI queues */

/* 5. program the bases and attributes */
        writeq(strtab_pa | fmt, base + SMMU_STRTAB_BASE);
        writel(cfg,               base + SMMU_STRTAB_BASE_CFG);
        writel(queue_attrs,       base + SMMU_CR1);

/* 6. invalidate EVERYTHING cached from a previous boot stage */
        cmdq_issue(CMD_CFGI_ALL);
        cmdq_issue(CMD_TLBI_NSNH_ALL);
        cmdq_issue(CMD_SYNC);

/* 7. enable the queues, THEN the SMMU */
        writel(CR0_CMDQEN | CR0_EVTQEN, base + SMMU_CR0);
        wait_for_ack(...);
        writel(CR0_CMDQEN | CR0_EVTQEN | CR0_SMMUEN, base + SMMU_CR0);
        wait_for_ack(...);

/* Step 3 is the security-critical one: if unconfigured entries
   are left with the valid bit SET from stale memory, unknown
   devices get whatever translation that garbage describes. Zero
   the table, and configure the default as ABORT.          */
```

Secure and non-secure interfaces

```
SMMUv3 has TWO complete programming interfaces, at different
address offsets:

   +0x0000   Non-secure  SMMU_*
   +0x8000   Secure      SMMU_S_*        (present if IDR1.SECURE_IMPL)

  Each has its OWN stream table, command queue, event queue and
  enable bit. They are independent.

  WHY: a device operating in the secure world must be configured
  by secure firmware. If the Non-secure OS could program that
  device's translations, it could redirect a secure DMA engine's
  transfers into normal-world memory - a complete bypass of
  TrustZone.

  And the STE carries security attributes too: STE.NSCFG and
  CD.NSCFG determine whether the resulting transaction is issued
  as Secure or Non-secure, exactly as the descriptor NS bit does
  for the CPU (ARM chapter 14).

  So the SMMU is not merely an isolation device for the normal
  world - it is part of the TrustZone enforcement boundary
  (chapters 08-10).
```

What Linux does

```
/* drivers/iommu/arm/arm-smmu-v3/ - worth reading, and about
   6000 lines. The structures map directly onto the above:  */

struct arm_smmu_device {
        void __iomem            *base;
        struct arm_smmu_strtab_cfg strtab_cfg;   /* stream table   */
        struct arm_smmu_cmdq     cmdq;
        struct arm_smmu_evtq     evtq;
        struct arm_smmu_priq     priq;
        u32                      features;       /* from IDR0-5    */
};

struct arm_smmu_domain {                 /* one per address space  */
        struct arm_smmu_device  *smmu;
        enum arm_smmu_domain_stage stage;         /* S1 or S2      */
        struct io_pgtable_ops   *pgtbl_ops;       /* the page tables */
        struct arm_smmu_ctx_desc cd;              /* the CD         */
};

/* And the generic IOMMU layer above it: */
        iommu_domain_alloc()      create an address space
        iommu_attach_device()     point a StreamID at it
        iommu_map(domain, iova, paddr, size, prot)
        iommu_unmap(domain, iova, size)

/* The io-pgtable library builds the actual tables, and is SHARED
   with the CPU's page table code - because the format is
   identical.                                                  */
```

Inspecting the tables

```
# the SMMU's own view
dmesg | grep -i smmu
#   arm-smmu-v3 2b400000.iommu: ias 48-bit, oas 48-bit (features 0x00013fbf)
#   arm-smmu-v3 2b400000.iommu: allocated 65536 entry cmdq
#   arm-smmu-v3 2b400000.iommu: 2-level strtab only covers 0..0xffff

# domains and groups
ls /sys/kernel/iommu_groups/*/devices/
cat /sys/kernel/iommu_groups/*/type          # DMA, DMA-FQ, identity

# mappings, live
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe
#   map:   IOVA=0x... paddr=0x... size=0x1000
#   unmap: IOVA=0x... size=0x1000

# from a JTAG debugger, walk the tables by hand:
#   1. read SMMU_STRTAB_BASE
#   2. index by StreamID (allowing for the two-level format)
#   3. read the STE's 8 words; check V and Config
#   4. follow S1ContextPtr to the CD
#   5. follow the CD's TTB0 into the page tables, which you can
#      then walk exactly as in ARM chapter 13

# Doing this once by hand makes every later SMMU problem
# tractable.
```

How it shows up

Symptom	Cause
`C_BAD_STE` events	Malformed STE, or the STE memory has attributes the SMMU cannot read coherently
`C_BAD_CD`	Bad TCR-equivalent fields, or a misaligned CD table
Tables look right in a dump but do not work	Mismatched memory attributes between CPU writes and SMMU fetches
Unknown devices get working translations	Stream table not zeroed; stale valid entries
SubstreamID traffic works, untagged traffic aborts	`STE.S1DSS` misconfigured
Stream table allocation fails at boot	Flat format with a large StreamID space; use two-level
Secure device redirected by the normal-world OS	Secure interface not used for secure streams
Changes to an STE have no effect	Missing `CMD_CFGI_STE` and `CMD_SYNC` (ch. 05)
SMMU enable never acknowledges	Queues not enabled first, or a base address the SMMU cannot reach

Check yourself

- ✓ Draw the lookup chain from StreamID to a page table entry.
- ✓ Give the five STE `Config` values and when each is correct.
- ✓ Explain why the default for an unconfigured stream must be abort, not bypass.
- ✓ Map five CD fields onto their CPU system-register equivalents.

- ✓ Explain why the CD's similarity to the CPU's context is the point.
- ✓ Explain when a two-level stream table or CD table is needed.
- ✓ Explain why STE and CD memory attributes must match how software writes them.
- ✓ Say why the SMMU has separate secure and non-secure programming interfaces.

SMMU: stages, queues and invalidation

Two translation stages with two different owners, three queues connecting software to hardware, and an invalidation discipline that mirrors the CPU's exactly. Plus the advanced features — ATS, PRI and SVA — that let a device share a process's address space directly.

Two stages, two owners

	Stage 1	Stage 2
Translates	Device address (IOVA) → IPA	IPA → PA
Owned by	The OS, or a guest OS	**The hypervisor**
Root held in	The Context Descriptor's `TTB0`	The STE's `S2TTB`
Tagged by	ASID	VMID
Purpose	Scatter-gather, per-process isolation, confining a device to its buffers	Confining a whole device to one VM's memory
Guest can change it?	Yes — it is the guest's own	**No** — that is the point

```
  BARE-METAL OS, stage 1 only

    device --> IOVA --[ stage 1, OS tables ]--> PA

  DEVICE PASSED THROUGH TO A VM, stage 2 only

    device --> IPA ---[ stage 2, hypervisor ]--> PA
               ^ the guest programs the device with its own
                 "physical" addresses, and cannot escape

  NESTED — a guest with its own IOMMU driver, both stages

    device --> IOVA --[ stage 1, GUEST ]--> IPA
                      --[ stage 2, HYPERVISOR ]--> PA

Exactly the structure of CPU two-stage translation
(ARM chapter 16), with the same walk-multiplication cost: each
stage-1 table access itself needs a stage-2 translation, so a
full nested walk can be up to 24 memory accesses.

Mitigations are the same too: SMMU TLBs caching the combined
IOVA -> PA translation, walk caches for intermediate
descriptors, and huge pages at stage 2 (a 1 GB block for the
guest's RAM is standard).
```

The three queues

```
Software and the SMMU communicate through circular buffers in
memory, each with producer and consumer indices in registers.

   COMMAND QUEUE          software  ->  SMMU
      CMDQ_BASE, CMDQ_PROD (software writes), CMDQ_CONS (SMMU)
      invalidate a cached STE, CD or TLB entry; synchronise;
      prefetch; respond to a page request

   EVENT QUEUE            SMMU  ->  software
      EVENTQ_BASE, EVENTQ_PROD (SMMU writes), EVENTQ_CONS
      fault records: bad StreamID, translation fault, permission
      fault, external abort. An interrupt signals arrival.

   PRI QUEUE              SMMU  ->  software
      PRIQ_BASE, PRIQ_PROD, PRIQ_CONS
      page requests from a device that supports PRI — "please
      map this address for me". Optional.

  Same design as the GIC's ITS command queue (ARM chapter 23),
  and as virtio and io_uring (OS chapter 08): a shared ring plus
  a doorbell. Recognise the pattern.

  Queue memory attributes must match SMMU_CR1's configuration,
  for the same reason STE memory does (chapter 04).
```

Commands worth knowing

Command	Invalidates	Issue after
`CMD_CFGI_STE`	One cached Stream Table Entry	Modifying an STE
`CMD_CFGI_ALL`	Every cached configuration	Initialisation
`CMD_CFGI_CD`	One cached Context Descriptor	Modifying a CD
`CMD_CFGI_CD_ALL`	All CDs for one StreamID	Tearing down a device's contexts
`CMD_TLBI_NH_VA`	One stage-1 address, by ASID	Unmapping a page
`CMD_TLBI_NH_ASID`	All stage-1 entries for an ASID	Tearing down an address space
`CMD_TLBI_S2_IPA`	One stage-2 IPA	Changing a stage-2 mapping
`CMD_TLBI_S12_VMALL`	Both stages for one VMID	Tearing down a VM
`CMD_TLBI_NSNH_ALL`	Everything non-secure	Initialisation
`CMD_ATC_INV`	A device's own translation cache	**Unmapping, when ATS is in use**
`CMD_PRI_RESP`	—	Answering a page request
`CMD_SYNC`	—	**Wait for everything before it to complete**

> **THE INVALIDATION RULE, EXACTLY AS FOR THE CPU**
>
> Anything you change in memory — an STE, a CD, a page table entry — is invisible to the SMMU until you issue the matching invalidate command *and* a `CMD_SYNC`. This mirrors the CPU's write → DSB → TLBI → DSB → ISB sequence (ARM 15), and omitting it produces exactly the same class of "works sometimes" bug.

```
/* Unmapping a page, correctly: */

        /* 1. clear the descriptor */
        *pte = 0;

        /* 2. make the write visible to the SMMU, which is a
              separate observer reading memory directly       */
        dsb(ishst);

        /* 3. invalidate the SMMU's TLB entry */
        cmdq_issue(CMD_TLBI_NH_VA, asid, iova);

        /* 4. if ATS is in use, invalidate the DEVICE's cache too,
              and WAIT for the device to acknowledge            */
        cmdq_issue(CMD_ATC_INV, sid, ssid, iova, size);

        /* 5. wait for all of it to complete */
        cmdq_issue(CMD_SYNC);
        wait_for_sync_completion();

        /* 6. ONLY NOW is it safe to free or reuse the page */
        free_page(page);

/* Skipping step 5 and freeing immediately leaves a window in
   which the device can still write to a page that now belongs
   to something else. That is a use-after-free with DMA, and it
   is precisely what iommu.strict=0 trades away for speed.  */
```

Strict versus deferred invalidation

```
   iommu.strict=1   (the default)
        every unmap issues an invalidate and waits for CMD_SYNC
        before the IOVA is reused or the page freed.
        SAFE. Costs a command-queue round trip per unmap, which
        on a high-rate NIC is a measurable fraction of the
        packet budget.

   iommu.strict=0   ("lazy" / DMA-FQ)
        unmapped IOVAs go on a flush queue and are invalidated
        in BATCHES, typically every 10 ms or when the queue
        fills. Far fewer syncs.
        UNSAFE WINDOW: between unmap and the batched invalidate,
        the device can still translate and access the page. If
        the page has been freed and reallocated, a malicious or
        buggy device writes to whatever now lives there.

   Which to choose:
      · trusted, on-SoC devices, throughput-critical -> strict=0
        may be a reasonable, documented decision
      · anything hot-pluggable, or any device you do not
        control -> strict=1, always

   cat /sys/kernel/iommu_groups/*/type
   #   DMA      strict
   #   DMA-FQ   deferred (flush queue)
   #   identity bypass
```

ATS: caching translations in the device

```
A PCIe device can ask the SMMU to translate an address in
advance and cache the result in its own ADDRESS TRANSLATION
CACHE (ATC), then issue PRE-TRANSLATED transactions.

   device --> ATS Translation Request --> SMMU
          <-- translated address       <--
   device caches it in the ATC
   device --> transaction marked "already translated" --> memory
                                       (SMMU not consulted)

  BENEFIT: removes the SMMU from the data path entirely. On a
  device doing millions of transfers per second this is a large
  win.

  COST 1 - a new correctness obligation. When software unmaps a
  page it must send CMD_ATC_INV and WAIT for the device's
  response. If the device is slow, buggy, or has been
  hot-unplugged, that wait can stall. Linux has a timeout and
  will report:
       arm-smmu-v3: ATC invalidation timed out

  COST 2 - a TRUST problem. A pre-translated transaction bypasses
  the SMMU's checks entirely. You are trusting the device to
    (a) only use translations it was legitimately given, and
    (b) honour invalidations.
  A malicious device that ignores both has unrestricted access.

  Hence PCIe ACS and the "translation blocking" controls, and
  hence: enable ATS only for devices you trust. Linux will not
  enable it for a device without the appropriate ACS
  capabilities upstream.
```

PRI: page faults for devices

```
/* Normally, a device touching an unmapped address gets an abort.
   With PRI, it can instead REQUEST that the page be mapped:  */

   device --> Page Request (via the PRI queue) --> software
   software pages the memory in, maps it
   software --> CMD_PRI_RESP (success) --> SMMU --> device
   device retries the access

/* This requires STALL mode: STE.S1STALLD = 0, so a faulting
   transaction is HELD rather than terminated. The device's
   pipeline stalls until the response arrives.

   What it enables: DEMAND PAGING for devices. A GPU can be
   given a pointer to memory that is swapped out, and it will
   fault it in exactly as a CPU would.

   What it costs: a stalled transaction occupies device
   resources. A storm of page requests can deadlock a device
   whose queues fill. And it only works if the device supports
   it - most do not.                                        */

/* Check support: */
        SMMU_IDR0.PRI          /* SMMU supports PRI          */
        SMMU_IDR0.STALL_MODEL  /* can it stall, or only
                                  terminate?                 */
```

SVA: sharing a process's address space

> **THE PAYOFF OF THE IDENTICAL DESCRIPTOR FORMAT**
>
> Because the SMMU uses exactly the CPU's translation table format (ARM 14), the OS can point a device's Context Descriptor at a *user process's own page tables*, with the same ASID. The device then dereferences precisely the pointers the application uses — no `dma_map`, no IOVA allocation, no copying.

```
   WITHOUT SVA                       WITH SVA

   app allocates a buffer            app allocates a buffer
   dma_map_single()                  (nothing)
     -> allocate an IOVA
     -> build SMMU page tables
     -> invalidate
   pass the dma_addr_t to the        pass the ORDINARY POINTER
     device                            to the device
   device DMAs                       device DMAs, translating
   dma_unmap_single()                  through the process's own
     -> tear it all down               page tables
                                     (nothing)

  Requirements, all three:
     1. the device supports ATS and PRI, because process memory
        can be swapped out or moved at any time
     2. the SMMU supports SubstreamIDs, so each process gets its
        own CD
     3. the OS wires the process's mm to a CD and keeps the ASID
        in sync, including on TLB invalidation

  Linux API:
        handle = iommu_sva_bind_device(dev, mm);
        pasid  = iommu_sva_get_pasid(handle);
        /* pass pasid to the device; it now sees the process's
           address space */
        iommu_sva_unbind_device(handle);

  Used by: GPU compute, smart NICs, accelerators — anywhere the
  marshalling cost of dma_map dominates the actual work.
```

Performance

Cost	Mitigation
Translation on every transaction	SMMU TLBs; huge pages at both stages
Walk on a TLB miss (up to 24 accesses nested)	Walk caches; cacheable table walks in the CD/STE
Map/unmap page-table edits	Persistent mappings; larger buffers; scatter-gather in one call
Command queue round trip per invalidate	Batching; `iommu.strict=0` (with the trade-off above)
IOVA allocation	Per-CPU IOVA caches (`iova_rcache` in Linux)
SMMU entirely in the data path	ATS — at the cost of trusting the device

```
# is the SMMU the bottleneck?
perf list | grep -i smmu
perf stat -e 'arm_smmu_v3_pmcg_*/cycles/,arm_smmu_v3_pmcg_*/transaction/' \
          -a sleep 10

# many SMMUs have a Performance Monitoring Counter Group (PMCG)
# exposing translation counts, TLB misses and walk counts —
# the same idea as the CPU PMU (ARM chapter 34)

# map/unmap rate
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe | head -100
# a very high map/unmap rate on a networking workload means
# per-packet mapping — consider persistent buffers or SVA
```

The Linux picture

```
/* Almost no driver programs an SMMU directly. The layering: */

   your driver
        |  dma_map_single(dev, buf, size, DMA_TO_DEVICE)
        v
   DMA API   (kernel/dma/mapping.c)
        |  is there an IOMMU for this device?
        +-- no  --> dma_direct: dma_addr == phys (+ offset),
        |           plus cache maintenance if non-coherent
        |
        +-- yes --> dma-iommu layer
                      allocate an IOVA from this device's
                      iova_domain (an allocator, like a small
                      address-space malloc)
                        |
                        v
                    iommu_map(domain, iova, phys, size, prot)
                        |
                        v
                    arm-smmu-v3 driver
                      build page table entries (io-pgtable)
                      queue CMD_TLBI + CMD_SYNC
                        |
                        v
                    returns dma_addr_t = the IOVA
        |
        v
   program the device with that dma_addr_t
   - NEVER a virtual address, NEVER a raw physical address

/* Chapter 07 covers the driver-facing API in full. */
```

How it shows up

Symptom	Cause
STE or page table change has no effect	Missing `CFGI` / `TLBI` and `CMD_SYNC`
Device writes to a freed page	Freed before `CMD_SYNC` completed, or `strict=0`
`ATC invalidation timed out`	Device slow, unplugged, or not honouring invalidations
Device stalls indefinitely	PRI stall mode with no software responding to page requests
Guest translation works, host sees nothing	Stage 1 configured, stage 2 not — or vice versa
Nested guest DMA is very slow	Walk multiplication; use huge pages at stage 2
Networking throughput drops with the IOMMU on	Per-packet map/unmap; batch, or use persistent buffers
Command queue overflows	Software not consuming, or issuing faster than the SMMU drains
SVA bind fails	Device lacks ATS or PRI, or the SMMU lacks SubstreamIDs
Stage-2 change corrupts guest memory	Invalidated stage 1 before stage 2 (ARM 15)

Check yourself

- ✓ Say who owns each translation stage and why the guest cannot change stage 2.
- ✓ Name the three queues and what flows through each.

- ✓ Give the correct unmap sequence, including `CMD_SYNC`, and say what each step prevents.
- ✓ Explain the `iommu.strict=0` trade-off and when it is defensible.
- ✓ Explain what ATS buys and the two costs it introduces.
- ✓ Explain PRI and what stall mode requires of the device.
- ✓ Explain SVA and its three prerequisites.
- ✓ Explain why walk multiplication is worse under nesting and how to mitigate it.

SMMU: faults and debugging

SMMU faults are asynchronous, reported through a queue rather than as an exception, and easy to misread. This chapter is the practical one: decode an event record, work out which of six things went wrong, and fix it.

How a fault is reported

```
Unlike a CPU fault, which is a synchronous exception with ESR
and FAR (ARM chapter 29), an SMMU fault is:

   1. detected while translating a DEVICE transaction
   2. written as a 32-byte EVENT RECORD into the event queue
   3. signalled by an interrupt (or MSI)
   4. read and logged by software, LATER

  So the fault is decoupled from anything the CPU was doing.
  There is no register to read at the moment it happens — the
  event record IS the evidence.

  The transaction itself is either TERMINATED (an abort returned
  to the device) or STALLED, depending on STE.S1STALLD and
  whether the device supports stalling.

  CONSEQUENCE: a device that faults typically reports a bus
  error, a timeout, or simply produces wrong data — and the
  SMMU's event record in dmesg is the only thing that says why.
```

Reading an event record

```
arm-smmu-v3 2b400000.iommu: event 0x10 received:
arm-smmu-v3 2b400000.iommu: 0x0000004200000010
arm-smmu-v3 2b400000.iommu: 0x0000000000000000
arm-smmu-v3 2b400000.iommu: 0x00000000fffff000
arm-smmu-v3 2b400000.iommu: 0x0000000000000000

/* Four 64-bit words. The layout, generally: */

   word 0  [7:0]    event type
           [63:32]  StreamID
   word 1  [31:0]   SubstreamID (if SSV set)
           [32]     SSV - SubstreamID valid
           [33]     PnU - privileged or unprivileged
           [34]     InD - instruction or data
           [35]     RnW - read or write
           [36]     S2  - the fault was at stage 2
           [37]     Class
   word 2  [63:12]  the faulting INPUT ADDRESS (IOVA or IPA)
   word 3           the faulting IPA, for stage-2 faults

/* Decoding the example above:
      type      = 0x10  = C_BAD_STREAMID
      StreamID  = 0x42
      address   = 0xfffff000

   Meaning: a device emitting StreamID 0x42 issued a transaction,
   and there is no valid Stream Table Entry for 0x42.

   The address is almost irrelevant here - the problem is the
   StreamID, not what it tried to touch.                    */
```

The event types

Code	Name	Means	Check
`0x01`	`F_UUT`	Unsupported upstream transaction	The device is issuing something the SMMU cannot handle — an unsupported size or attribute
`0x02`	`C_BAD_STREAMID`	**No valid STE for this StreamID**	**Device tree / IORT StreamID mismatch.** The most common failure.
`0x04`	`F_STE_FETCH`	Could not read the STE	Stream table address wrong, or the memory is unreachable by the SMMU
`0x05`	`C_BAD_STE`	The STE is malformed	Driver bug, reserved bits set, or mismatched memory attributes
`0x06`	`F_BAD_ATS_TREQ`	ATS request from a stream where ATS is not enabled	`STE.EATS`
`0x08`	`F_STREAM_DISABLED`	The stream is configured to abort	Deliberate — an unconfigured device, or `disable_bypass`
`0x0A`	`C_BAD_SUBSTREAMID`	SubstreamID out of range for this STE	`STE.S1CDMax`
`0x0B`	`F_CD_FETCH`	Could not read the CD	`S1ContextPtr`, or CD table memory
`0x0C`	`C_BAD_CD`	The CD is malformed	TCR-equivalent fields, alignment

Code	Name	Means	Check
`0x0D`	`F_WALK_EABT`	External abort during a table walk	The page tables themselves are in unreadable or denied memory
`0x10`	`F_TRANSLATION`	**The address is not mapped**	**Wrong address programmed into the device, or the mapping was torn down early**
`0x11`	`F_ADDR_SIZE`	Address exceeds the configured output size	`dma_set_mask()` versus the SMMU's OAS
`0x12`	`F_ACCESS`	Access flag not set	The `AF` bit in the descriptor
`0x13`	`F_PERMISSION`	**Mapped, but the wrong permission**	**DMA direction mismatch** — mapped `TO_DEVICE` and the device is writing
`0x20`	`F_TLB_CONFLICT`	Overlapping TLB entries	Break-before-make violation
`0x21`	`F_CFG_CONFLICT`	Conflicting cached configuration	An STE or CD changed without invalidation

THE THREE-WAY SPLIT

Group the event types by what they tell you:

- **Configuration faults** (`C_BAD_*`, `F_*_FETCH`) — the SMMU could not find or parse its own tables. A software or integration bug; the device did nothing wrong.
- **Translation faults** (`F_TRANSLATION`, `F_ACCESS`, `F_PERMISSION`) — the tables were fine; the address or the access type was wrong. Almost always a driver bug.
- **Stream faults** (`C_BAD_STREAMID`, `F_STREAM_DISABLED`) — identity. Device tree, IORT, or an unconfigured device.

Triage

```
Device does not work, or corrupts memory
     |
     v
Is a fault reported AT ALL?
   NO  -> the SMMU may be in BYPASS. Check:
            dmesg | grep 'Default domain type'
            cat /proc/cmdline  (iommu.passthrough?)
          A device that "works" with no SMMU configured is a
          SECURITY BUG, not a success.
   YES |
       v
What is the EVENT TYPE?
   |
   +-- C_BAD_STREAMID -----> the StreamID in the record is
   |                         what the HARDWARE emits. Compare
   |                         with the device tree. Fix the DT.
   |
   +-- C_BAD_STE / C_BAD_CD -> driver or integration bug.
   |                         Check memory attributes for the
   |                         table memory (chapter 04).
   |
   +-- F_TRANSLATION ------> the address is not mapped. Look
   |                         at the ADDRESS in word 2:
   |                           looks physical  -> driver passed
   |                                              phys instead of
   |                                              dma_addr_t
   |                           looks like a VA -> driver passed a
   |                                              kernel virtual
   |                                              address
   |                           a valid IOVA    -> unmapped too
   |                                              early, or DMA
   |                                              outlived the
   |                                              mapping
   |
   +-- F_PERMISSION ------> mapped, wrong direction. Check the
   |                        DMA_TO_DEVICE / DMA_FROM_DEVICE
   |                        argument.
   |
   +-- F_ADDR_SIZE -------> dma_set_mask() claims more bits
   |                        than the SMMU's output supports.
   |
   +-- F_WALK_EABT -------> the page tables are in memory the
                            SMMU cannot read. Check the table
                            allocation and any firewall
                            (chapters 17-20).
```

Reading the faulting address

```
/* The address in word 2 is the single most informative field
   after the event type. Learn to recognise the shapes:    */

   0x0000_0000_8xxx_xxxx    looks like a PHYSICAL address on a
                            typical SoC (DRAM at 0x80000000)
                            -> the driver passed phys_to_dma
                               incorrectly, or used virt_to_phys

   0xffff_8xxx_xxxx_xxxx    a KERNEL VIRTUAL address
                            -> the driver passed a kmalloc pointer
                               straight to the device

   0x0000_0000_0000_0000    NULL
                            -> uninitialised descriptor, or a
                               failed allocation not checked

   0x0000_00ff_xxxx_xxxx    a plausible IOVA in the allocated
                            range
                            -> the mapping existed and was torn
                               down. Look for: dma_unmap before
                               the device finished, remove()
                               without quiescing the device, or
                               a driver freeing a buffer early.

   0x0000_0000_08xx_xxxx    the ITS doorbell region
                            -> an MSI write that was not mapped
                               (chapter 03)

/* Cross-check against what was actually mapped: */
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe | grep -i map
#   map: IOVA=0x00000000ff000000 paddr=0x0000000082340000 size=4096
```

The bisection that works

ONE EXPERIMENT HALVES THE SEARCH SPACE

Boot once with `iommu.passthrough=1`.

- **The problem disappears** → it is a mapping or StreamID issue. The device programming is fine; the SMMU configuration is not.
- **The problem persists** → the bug is in how the device is programmed. The SMMU was just the messenger reporting it.

Never ship with passthrough enabled — but during bring-up this single reboot saves hours of speculation.

```
# the sequence that resolves most SMMU bring-up problems

# 1. is the SMMU even active?
dmesg | grep -iE 'smmu|iommu'
#   arm-smmu-v3 ...: ias 48-bit, oas 48-bit (features 0x...)
#   iommu: Default domain type: Translated     <- good
#   iommu: Default domain type: Passthrough    <- NOT translated

# 2. is this device behind it?
ls -l /sys/bus/platform/devices/2c000000.dma/iommu_group
cat /sys/kernel/iommu_groups/*/devices/*

# 3. trigger a transfer and read the fault
dmesg -w & ./trigger_dma

# 4. compare the StreamID in the record with the device tree
hexdump -C /proc/device-tree/soc/dma@2c000000/iommus

# 5. watch the mappings being created
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe

# 6. bisect with passthrough
# add iommu.passthrough=1 to the command line, reboot, retest
```

The driver bugs behind most faults

Bug	Event	Fix
Passed a kernel virtual address to the device	`F_TRANSLATION`, VA-shaped address	Use the `dma_addr_t` from `dma_map_*`
Passed a physical address	`F_TRANSLATION`, PA-shaped address	Same
Unmapped while the transfer was in flight	`F_TRANSLATION`, valid IOVA	Wait for completion before unmapping
Did not quiesce the device in `remove()`	`F_TRANSLATION` after unbind	Stop the hardware *first* (OS 16)
Wrong DMA direction	`F_PERMISSION`	Match `DMA_TO_DEVICE` / `FROM_DEVICE` to reality
DMA to a stack buffer	Varies	Never — the stack is not DMA-able
DMA to `vmalloc` memory	`F_TRANSLATION`	Not physically contiguous; use `kmalloc` or scatter-gather
`dma_set_mask` too optimistic	`F_ADDR_SIZE`	Match the device's real addressing capability
Ignored `dma_mapping_error()`	Varies	Always check
Descriptor ring not `dma_alloc_coherent`	Varies	Rings must be coherent (ch. 07)

Stall mode and hangs

```
/* If STE.S1STALLD = 0 and the device supports stalling, a
   faulting transaction is HELD rather than terminated. The
   device's pipeline stops until software responds.

   Consequences:
     · the device appears HUNG rather than reporting an error
     · a storm of stalled transactions can exhaust the device's
       internal queues and deadlock it
     · software MUST respond with CMD_RESUME, or the device
       never recovers

   Which is why Linux sets S1STALLD = 1 (terminate) unless PRI
   is in use — an unhandled stall is worse than a clean abort. */

/* If a device hangs with no fault reported at all, check: */
        dmesg | grep -i 'stall\|resume'
        cat /sys/kernel/debug/tracing/events/iommu/dev_fault/enable

/* And check the GLOBAL ERROR register — a malformed command or
   a queue overflow is reported there, not in the event queue: */
        SMMU_GERROR vs SMMU_GERRORN
//   arm-smmu-v3: GERROR MSI write aborted
//   arm-smmu-v3: CMD_SYNC timeout at 0x...
//   arm-smmu-v3: unexpected global error reported
```

Faults under virtualisation

```
/* With stage 2 in use, a fault may be at either stage — word 1
   bit 36 (S2) says which, and word 3 holds the IPA.        */

   S2 = 0   a stage-1 fault. The guest's own IOMMU tables are
            wrong, or the guest programmed the device with a bad
            IOVA. The guest's problem.

   S2 = 1   a stage-2 fault. The hypervisor has not mapped that
            IPA for the device. The HOST's problem — usually
            memory the guest owns but the host has not pinned
            and mapped for DMA.

/* A very common case: a guest passed a device a buffer in memory
   the hypervisor had not mapped into the device's stage-2
   tables. VFIO pins and maps all guest memory for exactly this
   reason, which is why passthrough VMs cannot use ballooning
   or overcommit without extra machinery.                   */

# on the host
dmesg | grep -A4 'event.*received'
cat /sys/kernel/debug/tracing/events/iommu/enable
```

The full toolkit

```
# fault records
dmesg -w | grep -i smmu

# mappings, live
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe

# per-event tracepoints
ls /sys/kernel/debug/tracing/events/iommu/
#   add_device_to_group  attach_device_to_domain  map  unmap
#   io_page_fault  dev_fault  dev_page_response

# groups, domains and reserved regions
ls -l /sys/kernel/iommu_groups/*/devices/
cat /sys/kernel/iommu_groups/*/type
cat /sys/kernel/iommu_groups/*/reserved_regions

# SMMU performance counters, where present
perf list | grep -i smmu
perf stat -e arm_smmu_v3_pmcg_*/transaction/ -a sleep 5

# from a JTAG debugger: read SMMU_STRTAB_BASE, index by the
# StreamID from the fault record, and dump the STE. If it is
# zero or invalid, the configuration never reached the hardware.

# bpftrace on the fault path
sudo bpftrace -e 'tracepoint:iommu:io_page_fault {
        printf("%s dev=%s addr=%llx\n", comm, str(args->device),
               args->iova); }'
```

How it shows up

Symptom	Cause
`C_BAD_STREAMID` on every transfer	Device tree or IORT StreamID does not match the hardware
`F_TRANSLATION` with a physical-looking address	Driver passed a physical address instead of `dma_addr_t`
`F_TRANSLATION` with a valid IOVA	Unmapped while DMA was in flight, or on remove without quiescing
`F_PERMISSION`	DMA direction mismatch
`F_ADDR_SIZE`	`dma_set_mask` exceeds the SMMU's output address size
`F_WALK_EABT`	Page tables in memory the SMMU cannot read — check firewalls
Device hangs with no fault logged	Stall mode with no responder; check `GERROR` too
No faults and no isolation	Bypass or passthrough enabled
DMA works, interrupts never arrive	The MSI doorbell is not mapped for that device
Faults only under a hypervisor	Stage-2 mapping missing — check the S2 bit in the record
Faults appear after enabling `strict=0`	Deferred invalidation window; a freed page was still reachable

Check yourself

- ✓ Explain why an SMMU fault is reported differently from a CPU fault.
- ✓ Decode the four words of an event record.

- ✓ Give the three-way grouping of event types and what each group implicates.
- ✓ Given `F_TRANSLATION`, name four address shapes and what each indicates.
- ✓ Describe the passthrough bisection and what each outcome tells you.
- ✓ List five driver bugs that produce SMMU faults, with the event each gives.
- ✓ Explain stall mode and why Linux usually disables it.
- ✓ Say how to tell a stage-1 fault from a stage-2 fault, and whose problem each is.

DMA APIs and coherency in practice

The driver-facing side of everything in the previous five chapters. One API hides whether there is an IOMMU, whether the device is coherent, and whether cache maintenance is needed — which is exactly why hand-rolling any of it is a bug.

What the DMA API hides

```
dma_map_single(dev, buf, size, DMA_TO_DEVICE)
     |
     v
Is there an IOMMU for this device?
   |
   +-- NO  --> dma_direct
   |             dma_addr = phys_addr + a per-device offset
   |             (some SoCs place DRAM at a different address
   |              on the device side of the bus)
   |
   +-- YES --> dma-iommu
                 allocate an IOVA from this device's
                 iova_domain, build SMMU page tables,
                 invalidate, return the IOVA
     |
     v
Is this device COHERENT?
   |
   +-- YES --> nothing more to do
   |
   +-- NO  --> emit cache maintenance:
                 clean before the device reads
                 invalidate before the CPU reads
                 (ARM chapter 18)
     |
     v
returns dma_addr_t — the address the DEVICE must use

FOUR platform differences, one API. This is why the same
driver source is correct on a coherent server with an SMMU and
on a non-coherent MCU with neither.
```

`DMA_ADDR_T` IS NOT A PHYSICAL ADDRESS

It is the address the *device* must be programmed with. With an IOMMU it is an IOVA; without one it is usually the physical address plus a bus offset; on some SoCs it is neither. Treating it as a physical address works on your development board and fails on the next one.

The two kinds of mapping

	Coherent (consistent)	Streaming
Allocate with	`dma_alloc_coherent()`	Any allocator, then `dma_map_*()`
Lifetime	Long-lived — allocated once	One transfer
Cache maintenance	None needed — the API guarantees coherency	Handled by map/unmap/sync

	Coherent (consistent)	Streaming
Cost	May be uncached, so CPU access is slow	Map/unmap cost per transfer
Use for	**Descriptor rings**, doorbell structures, anything both sides poll	**Data buffers**

```
/* COHERENT — for the ring the device and CPU both poll */
struct desc *ring;
dma_addr_t ring_dma;

ring = dma_alloc_coherent(dev, RING_BYTES, &ring_dma, GFP_KERNEL);
if (!ring)
        return -ENOMEM;
/* both sides may access it at any time, with no sync calls.
   On a non-coherent system this memory is UNCACHED, so CPU
   access is slow — keep rings small and do not put bulk data
   here.                                                    */
writel(lower_32_bits(ring_dma), regs + RING_BASE_LO);
writel(upper_32_bits(ring_dma), regs + RING_BASE_HI);
...
dma_free_coherent(dev, RING_BYTES, ring, ring_dma);


/* STREAMING — for the data itself */
dma_addr_t d = dma_map_single(dev, buf, len, DMA_TO_DEVICE);
if (dma_mapping_error(dev, d))          /* ALWAYS check */
        return -ENOMEM;

writel(lower_32_bits(d), regs + DMA_ADDR_LO);
writel(upper_32_bits(d), regs + DMA_ADDR_HI);
writel(len,              regs + DMA_LEN);
writel(DMA_GO,           regs + DMA_CTRL);

/* ... wait for the completion interrupt ... */

dma_unmap_single(dev, d, len, DMA_TO_DEVICE);
/* only NOW may the CPU touch buf again */
```

Buffer ownership

> **BETWEEN MAP AND UNMAP, THE BUFFER BELONGS TO THE DEVICE**
> Touching it from the CPU in that window gives stale or corrupted data on a non-coherent system, and races even on a coherent one. The API models this as an ownership transfer, and the `sync` functions are how you temporarily take it back.

```
/* Peeking mid-transfer, correctly: */
dma_sync_single_for_cpu(dev, d, len, DMA_FROM_DEVICE);
inspect(buf);                                  /* CPU owns it   */
dma_sync_single_for_device(dev, d, len, DMA_FROM_DEVICE);
/* device owns it again                                       */

/* And a partial sync, when only part of a large buffer matters: */
dma_sync_single_range_for_cpu(dev, d, offset, size, DMA_FROM_DEVICE);

/* What the direction argument actually controls: */

   DMA_TO_DEVICE        CPU wrote it, device will read
                        -> map does a CLEAN
                        -> the device must not write it
   DMA_FROM_DEVICE      device will write, CPU will read
                        -> map does an INVALIDATE (and unmap
                           does another)
                        -> the buffer's prior contents are LOST
   DMA_BIDIRECTIONAL    both
                        -> clean and invalidate; more expensive
   DMA_NONE             debugging only

/* The direction is not documentation — it selects the cache
   maintenance and the SMMU permissions. Getting it wrong gives
   either stale data or an F_PERMISSION fault (chapter 06).  */
```

The four rules

Rule	Why	Failure
1. Program the device with `dma_addr_t`	It is neither a virtual nor necessarily a physical address	`F_TRANSLATION`, or DMA to the wrong memory
2. Never DMA to stack memory	Stack buffers are not cache-line aligned, so an invalidate destroys neighbouring stack data	Corruption of unrelated locals; hard to attribute
3. Never DMA to `vmalloc` memory	Virtually contiguous only — the physical pages are scattered	Data lands in the wrong pages
4. Respect ownership	See above	Stale or corrupted data

```
/* Rule 2, made concrete - this is subtler than it looks: */

void bad(struct device *dev)
{
        char buf[64];                   /* on the stack */
        dma_addr_t d = dma_map_single(dev, buf, 64, DMA_FROM_DEVICE);
        /* ... */
}
/* On a NON-COHERENT system, dma_unmap invalidates the cache
   lines covering buf. If buf shares a 64-byte line with other
   locals - a return address, a saved register, a loop counter -
   those are invalidated too, and their dirty values LOST.

   The corruption appears in unrelated variables, after the
   function returns. Almost impossible to trace back.

   CONFIG_DMA_API_DEBUG catches it:
       DMA-API: device driver maps memory from stack           */

/* The fix: heap-allocate, or embed the buffer in a
   cache-line-aligned structure: */
struct my_dev {
        ...
        char buf[64] ____cacheline_aligned;
};
```

Scatter-gather

```
/* For transfers spanning several physically discontiguous pages.
   The SMMU can make them contiguous in the device's view; without
   one, the device must handle a list.                         */

struct scatterlist sgl[N];
sg_init_table(sgl, N);
for (i = 0; i < N; i++)
        sg_set_page(&sgl[i], pages[i], PAGE_SIZE, 0);

int nents = dma_map_sg(dev, sgl, N, DMA_TO_DEVICE);
if (!nents)
        return -ENOMEM;

/* NOTE: nents may be LESS than N. With an IOMMU, adjacent
   physical pages are COALESCED into one device-visible extent.
   Always iterate over the RETURNED count, using the accessors: */
struct scatterlist *sg;
for_each_sg(sgl, sg, nents, i)
        program_descriptor(sg_dma_address(sg), sg_dma_len(sg));
/*                          ^^^^^^^^^^^^^^^^ NOT sg_phys()      */

dma_unmap_sg(dev, sgl, N, DMA_TO_DEVICE);   /* the ORIGINAL N */

/* Using sg->length instead of sg_dma_len(sg), or iterating to N
   instead of nents, is a classic bug that only manifests when
   an IOMMU is present and coalescing occurs.               */
```

Addressing capability

```
/* Tell the core what the device can actually address, BEFORE
   any mapping.                                                */

ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(48));
if (ret) {
        /* fall back - do not just fail */
        ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(32));
        if (ret)
                return ret;
}

/* What it controls:
     · without an IOMMU: which physical memory may be used.
       A 32-bit device gets memory below 4 GB, or BOUNCE
       BUFFERS (copy down, DMA, copy back) via SWIOTLB.
     · with an IOMMU: which IOVA range is allocated from.

   Claiming more than the device supports gives silent
   corruption (the top bits are dropped) or F_ADDR_SIZE
   (chapter 06). Claiming less costs bounce buffers.

   And note the SEPARATE coherent mask: some devices can DMA
   64-bit for data but need 32-bit for descriptor rings.  */
dma_set_mask(dev, DMA_BIT_MASK(64));
dma_set_coherent_mask(dev, DMA_BIT_MASK(32));
```

Coherent or not

```
/* Whether a device participates in cache coherency is a SoC
   integration property (ARM chapter 19), described in the
   device tree:                                              */

        dma_engine@2c000000 {
                compatible = "vendor,dma";
                reg = <...>;
                dma-coherent;          /* ACE-Lite or CHI      */
                iommus = <&smmu 0x42>;
        };

/* The kernel reads it: */
        dev_is_dma_coherent(dev)

/* And it changes what dma_alloc_coherent returns:
     coherent device     -> ordinary cacheable memory, fast
     non-coherent device -> UNCACHED memory, slow for the CPU

   Which is why a descriptor ring that is fine on one SoC is a
   performance problem on another. If profiling shows a driver
   spending time reading its own ring, check coherency.    */

# is it coherent?
grep -l dma-coherent /proc/device-tree/*/*/ 2>/dev/null
cat /sys/bus/platform/devices/*/of_node/dma-coherent 2>/dev/null

# ACPI equivalent: the _CCA method
```

Reserved and restricted regions

```
/* Some devices must use specific memory: a fixed carveout, a
   region visible to a coprocessor, or contiguous memory for a
   camera or codec.                                          */

/* Reserved memory in the device tree: */
reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;

        codec_pool: codec@90000000 {
                compatible = "shared-dma-pool";
                reg = <0x0 0x90000000 0x0 0x2000000>;
                no-map;            /* keep it OUT of the linear map,
                                      so attributes cannot conflict
                                      (ARM chapter 19)             */
        };

        cma_pool: linux,cma {
                compatible = "shared-dma-pool";
                reusable;          /* CMA: normally used for movable
                                      pages, reclaimed on demand    */
                size = <0x0 0x8000000>;
                linux,cma-default;
        };
};

video_codec@... {
        memory-region = <&codec_pool>;
};

/* `no-map` versus `reusable` matters:
     no-map    the kernel never maps it at all. Required when
               the region needs different memory ATTRIBUTES than
               the linear map would give it.
     reusable  CMA — the kernel uses it for movable pages and
               reclaims it when a large contiguous allocation is
               needed (OS chapter 11).                       */
```

Debugging DMA

```
# CONFIG_DMA_API_DEBUG — the single most valuable option here.
# It tracks every mapping and catches:
#   · unmapping with the wrong size or direction
#   · CPU access to a device-owned buffer
#   · mapping stack or vmalloc memory
#   · leaked mappings on driver unload
#   · double unmap
CONFIG_DMA_API_DEBUG=y
CONFIG_DMA_API_DEBUG_SG=y

dmesg | grep 'DMA-API'
#   DMA-API: device driver frees DMA memory with different size
#   DMA-API: device driver maps memory from stack
#   DMA-API: cacheline tracking EEXIST, overlapping mappings

echo 1 > /sys/kernel/debug/dma-api/dump      # every live mapping

# SMMU-side view
echo 1 > /sys/kernel/debug/tracing/events/iommu/enable
cat /sys/kernel/debug/tracing/trace_pipe

# and the SWIOTLB, if bounce buffers are in play
dmesg | grep -i swiotlb
cat /sys/kernel/debug/swiotlb/io_tlb_used
```

DIAGNOSING "DMA GIVES WRONG DATA"

Three distinct causes, and the fix for one does nothing for the others:

1. **All zeros, or the previous contents** → the device never wrote there. Check the address you programmed and look for an SMMU fault.
2. **Stale but plausible data** → coherency. Missing `dma_sync`, or the CPU touched the buffer while the device owned it.
3. **Correct data with corruption nearby** → a partial cache line. The buffer is not cache-line aligned — very often it is on the stack.

The Windows equivalent

```
/* Same concepts, different names (Windows track, chapter 18): */

   WDF_DMA_ENABLER_CONFIG_INIT(&cfg, WdfDmaProfileScatterGather64,
                               MAX_LEN);
   WdfDmaEnablerCreate(device, &cfg, ...);

   WdfDmaTransactionCreate(...);
   WdfDmaTransactionInitializeUsingRequest(..., WdfDmaDirectionWriteToDevice);
   WdfDmaTransactionExecute(...);

   /* the callback receives a scatter-gather list */
   for (i = 0; i < SgList->NumberOfElements; i++)
           program(SgList->Elements[i].Address.QuadPart,   /* LOGICAL */
                   SgList->Elements[i].Length);

/* Windows calls dma_addr_t a LOGICAL ADDRESS. Same meaning:
   the address the device must use, which with an IOMMU is an
   IOVA. The three rules above apply identically.          */
```

How it shows up

Symptom	Cause
Device reads garbage or previous contents	Wrong address programmed, or missing clean on a non-coherent system
Buffer never appears to update	Missing invalidate; or the CPU read a cached copy
Unrelated local variables corrupted	DMA to a stack buffer — partial cache line invalidated
Works on one SoC, corrupts on another	One device is coherent, the other is not
Works without an IOMMU, fails with one	Passed a physical address, or ignored scatter-gather coalescing
Only the first scatter-gather entry transfers	Iterated to `N` instead of the returned `nents`
Silent truncation on large addresses	`dma_set_mask` claims more bits than the device has
Transfers unexpectedly slow	SWIOTLB bounce buffers, from a too-small mask
Driver spends time reading its own descriptor ring	Non-coherent device, so the ring is uncached
Large contiguous allocation fails after uptime	Fragmentation; needs CMA or an SMMU with scatter-gather
`DMA-API` warnings in dmesg	Read them — each one names a specific rule violation

Check yourself

- ✓ Name the four platform differences the DMA API hides.
- ✓ Explain what `dma_addr_t` actually is, in the three possible cases.
- ✓ Give the difference between coherent and streaming mappings and what each is for.
- ✓ State the four rules and the failure each prevents.
- ✓ Explain precisely why DMA to a stack buffer corrupts unrelated data.
- ✓ Explain what the direction argument controls, beyond documentation.
- ✓ Say why `dma_map_sg` may return fewer entries than it was given.
- ✓ Explain what `dma_set_mask` controls with and without an IOMMU.
- ✓ Give the three causes of "DMA gives wrong data" and how to tell them apart.

TrustZone: the NS bit

Everything about TrustZone follows from one design decision: add a single extra bit to every bus transaction saying which world it came from, then teach every block in the SoC to respect it. One wire, system-wide.

The whole idea

> **THE RULE**
>
> A **Non-secure master cannot access Secure resources.** A **Secure master can access both.**
>
> This is enforced in hardware — at the memory controller, at each peripheral, at the interrupt controller, at the SMMU and on the interconnect itself. Not by software checks that a compromised kernel could skip.

```
The bit is AxPROT[1] on AXI (ARM chapter 25), universally called
the NS bit:

   NS = 0   this access is SECURE
   NS = 1   this access is NON-SECURE

   CPU core                        peripheral / memory
      |                                    ^
      |  ADDR                              |
      |  DATA                              |
      |  AxPROT[1] = NS  -----------------+  checked HERE
      |  AxID = master id
      |  AxCACHE = attributes

  Where NS comes from, for a CPU access:
     · at EL3, from SCR_EL3.NS
     · at EL0/EL1/EL2, from the current SECURITY STATE, which
       EL3 set via SCR_EL3.NS before dropping down
     · plus the page descriptor's NS bit, for Secure software
       creating a mapping that issues Non-secure accesses
       (ARM chapter 14)

  Where it comes from for a DEVICE access:
     · hard-wired per master, or
     · from the SMMU's STE/CD NSCFG fields (chapter 04), or
     · assigned dynamically by a TZPC (chapter 10)
```

Why this differs from a hypervisor

```
   HYPERVISOR ISOLATION            TRUSTZONE ISOLATION
   --------------------            -------------------
   software isolating software     SILICON isolating silicon
   the hypervisor is code that     the NS bit is a WIRE; the
     can have bugs                   memory controller physically
   a guest escape gives you the      refuses the transaction
     hypervisor's privilege        a compromised EL1 with
   stage-2 tables are configured     arbitrary code execution
     by that software                still cannot read Secure DRAM
                                     - the data never comes back

  Both are useful, and they compose: a system can run a
  hypervisor in the Normal world AND have a Secure world beneath
  both. They defend against different things.

  The practical distinction: a hypervisor protects guests from
  each other, assuming the hypervisor is correct. TrustZone
  protects secrets from the ENTIRE normal-world software stack,
  assuming only that EL3 firmware and the hardware are correct.
```

Security state versus exception level

```
/* These are ORTHOGONAL. A context is identified by BOTH. */

                   Non-secure          Secure
   EL0             applications        trusted applications
   EL1             Linux, Windows      trusted OS (OP-TEE)
   EL2             hypervisor          secure partition mgr (v8.4+)
   EL3             ------- the monitor, in BOTH -------

/* Which security state am I in? There is no register that says
   so directly at EL1 - by design. Secure software knows because
   it was loaded into the secure world; Non-secure software
   cannot tell and should not need to.

   At EL3: */
        mrs     x0, scr_el3
        tst     x0, #1                  /* SCR_EL3.NS */
        /* NS=0: the world below is Secure
           NS=1: the world below is Non-secure */

/* SCR_EL3.NS is the ONE bit that selects which world exists
   below EL3. Changing it and executing ERET is the world
   switch (chapter 11).                                    */
```

Banked state

```
Because the two worlds must not see each other's configuration,
much of the EL1 system register file is BANKED by security
state:

   SCTLR_EL1        one for Secure, one for Non-secure
   TTBR0_EL1        ditto — the two worlds have entirely
   TTBR1_EL1        separate page tables
   TCR_EL1
   MAIR_EL1
   VBAR_EL1         separate vector tables
   ESR_EL1, FAR_EL1, ELR_EL1, SPSR_EL1
   TPIDR_EL0/EL1
   ...

  SCR_EL3.NS selects which bank EL3 sees. So when the monitor
  switches worlds it does NOT have to save and restore these —
  the hardware provides two copies.

  What it DOES have to save and restore: the general-purpose
  registers X0–X30, the FP/SIMD state, and any registers that
  are NOT banked. That is the bulk of the world-switch cost
  (chapter 11).

  NOT banked: the caches and TLBs, which instead TAG their
  entries with the security state so entries from the two
  worlds coexist without leaking (chapter 10).
```

The NS bit in page tables

```
/* Secure software can create a mapping that issues NON-SECURE
   accesses. That is how a trusted OS reads a buffer shared with
   the normal world.                                          */

   descriptor bit 5   NS
        0   accesses through this mapping are SECURE
        1   accesses through this mapping are NON-SECURE

/* Only meaningful in the SECURE translation regime. Non-secure
   software's accesses are always NS=1 regardless of what its
   descriptors say — enforced by hardware, not by the tables. */

/* And in TABLE descriptors there is a separate NSTable bit
   (bit 63): it forces every entry BELOW it to be Non-secure,
   whatever those entries claim. A way to delegate an entire
   subtree safely — useful when secure software builds a
   mapping region for shared buffers.                       */

/* The practical rule for secure software:
     · map your own memory NS=0
     · map shared buffers NS=1
     · NEVER map normal-world memory NS=0, or a normal-world
       pointer becomes a secure access                       */
```

Partitioning memory

```
Physical memory as a TZASC (chapter 10) is typically configured
at boot:

  0x8000_0000  +---------------------------------+
               | Non-secure DRAM                 |  NS=1 permitted
               |   Linux kernel + userspace      |  NS=0 also permitted
               |   ~1.9 GB                       |  (Secure can read it)
  0xFC00_0000  +---------------------------------+
               | SHARED buffer region            |  both worlds
               |   TEE message passing           |
  0xFE00_0000  +---------------------------------+
               | Secure DRAM                     |  NS=1 -> REFUSED
               |   OP-TEE OS + TA heap           |  invisible to Linux
  0xFF00_0000  +---------------------------------+
               | EL3 firmware (BL31)             |  Secure only, and
               |                                 |  often LOCKED so even
  0xFF10_0000  +---------------------------------+  S-EL1 cannot write it

  What "refused" means concretely: the memory controller returns
  an error response, which becomes an external abort at the CPU
  (ARM chapter 29, DFSC 0b010000) — or, on some configurations,
  returns zeros silently. The latter is far harder to debug and
  is why chapter 20 exists.

  Note the LOCKED EL3 region: even the secure world's own OS
  cannot modify the monitor. Privilege inside a world is still
  layered.
```

Everything checks it

Block	Uses the NS bit to
CPU core	Emit it on every fetch and data access, from the current security state
MMU	Separate translation regimes per state; the descriptor `NS` bit
Caches	Tag lines with the security state, so a Non-secure lookup cannot hit a Secure line
TLBs	Same — entries are tagged
TZASC	Per-region DRAM permission by NS state
TZPC	Mark whole peripherals Secure or Non-secure
GIC	Interrupt groups: Group 0 Secure (FIQ), Group 1 per state
SMMU	Separate secure and non-secure programming interfaces and stream tables
eFuses / OTP	Key material readable only by Secure masters
Debug	`SPIDEN` / `SPNIDEN` gate whether JTAG may see the secure world
Interconnect / XPU	Final per-master, per-region check including NS state

> **THE CONSEQUENCE WORTH INTERNALISING**
>
> A single bit is cheap to add to a bus, so it could be added *everywhere*. That is why TrustZone is a system-wide property rather than a CPU feature, and why an SoC that implements it partially — a peripheral that ignores the NS bit, a bus segment that drops it — has a complete bypass. Verifying that every block honours it is a real part of SoC security review.

The symptom this produces

> **"THE PERIPHERAL READS BACK ALL ZEROS"**
>
> If a peripheral is marked Secure in the TZPC, a Linux driver that `ioremap()`s its registers gets an error or zeros — because the access carries NS=1 and the peripheral refuses it. The address is right, the clock is on, the driver is correct, and nothing works.
>
> This is the single most confusing TrustZone symptom for driver authors, because nothing in the normal-world software stack knows the secure world exists. When a peripheral behaves as though it is not there on production hardware but works on a development board, suspect the security configuration before suspecting your driver.

```
/* What you see from Linux: */
        val = readl(base + REG_ID);
        /* val == 0, or 0xFFFFFFFF, or an external abort */

/* dmesg, if it aborts: */
//   Unhandled fault: synchronous external abort (0x96000010)
//                                                    ^^ DFSC
//                                                    external abort

/* How to confirm it is a security issue rather than a clock or
   address problem:
     1. check the clock and power domain first — same symptom
     2. read the SoC TRM for the peripheral's default TZPC
        assignment
     3. if firmware exposes it, query via an SMC
     4. compare against a development board, where the fuses
        and firmware configuration usually differ            */
```

What one bit does not give you

Not protected against	Why	What does help
Physical bus probing	The NS bit is a wire; an attacker with a probe reads the data lines	Memory encryption, on-die placement
Cold-boot DRAM extraction	DRAM contents survive briefly after power loss	Memory encryption, key scrubbing
Side channels	The two worlds share caches, branch predictors and DRAM	Constant-time code, cache partitioning, flushing on switch
Speculation across the boundary	Spectre-class leaks predate the check	`CSDB`, `SB`, index masking
Bugs in secure software	The NS bit does not validate anything	A small, audited TCB (ch. 16)
A block that ignores NS	Enforcement is only as complete as the implementation	SoC security review

Confirming the hardware honours it

```
/* During SoC bring-up or security review, test the NEGATIVE
   cases explicitly. A policy never exercised is a policy you do
   not know works.                                          */

/* From the Normal world, attempt to read Secure DRAM: */
        void __iomem *p = ioremap(SECURE_DRAM_BASE, 4);
        u32 v = readl(p);
        /* EXPECTED: an external abort, or a latched fault in the
           TZASC. If it returns real data, the region is not
           actually protected.                                  */

/* From the Normal world, attempt to program a Secure device: */
        writel(1, secure_dev_base + CTRL);
        /* EXPECTED: refused. Verify by reading back, and by
           checking the peripheral did not act.                 */

/* And from the Secure world, verify you CAN reach both: */
        secure read of Non-secure DRAM   -> should succeed
        secure read of Secure DRAM       -> should succeed

/* Write these as a boot-time self-test on development
   firmware. Chapter 19 covers building the full matrix.  */
```

How it shows up

Symptom	Cause
Peripheral reads all zeros or aborts on production hardware	Assigned to the secure world by the TZPC
External abort with DFSC `0b010000`	The bus refused the access — often a security check
Works on the dev board, fails on shipping units	Different fuse or firmware security configuration
Linux cannot see a memory region present in the TRM	Carved out as Secure DRAM by the TZASC
A secure driver can read normal-world memory	Correct — Secure masters may access both
Normal-world code reads secure memory successfully	**A security bug.** The region is not protected.
Secure interrupt cannot be masked by Linux	Group 0, delivered as FIQ to EL3 — by design
JTAG cannot see secure state	Secure debug fused off
A shared buffer works from Linux, faults from the TEE	Secure mapping created with NS=0 instead of NS=1

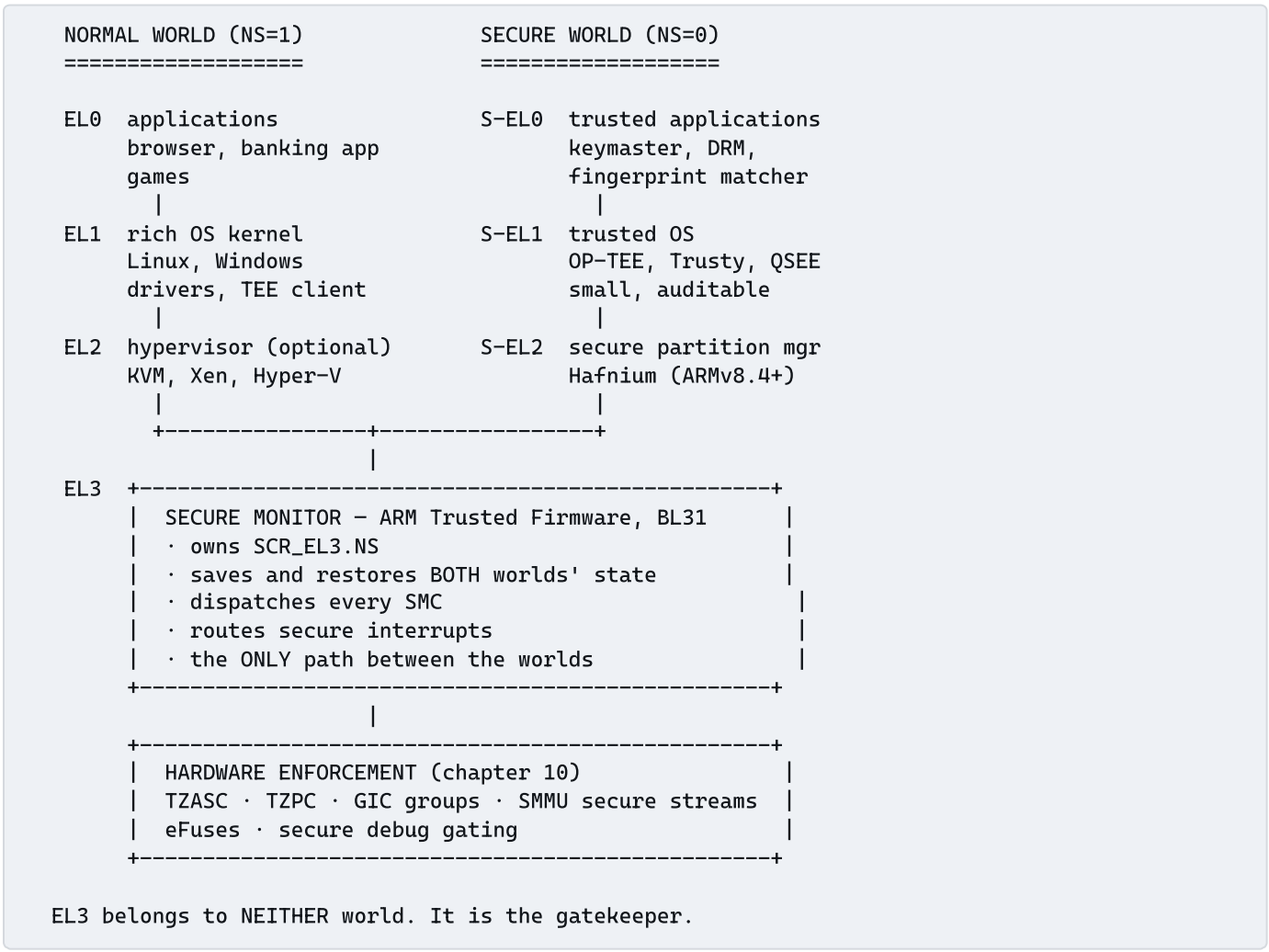
Check yourself

- ✓ State the TrustZone rule in one sentence and name the signal that carries it.
- ✓ Explain why silicon-level isolation differs from hypervisor isolation, and what each assumes.
- ✓ Explain how security state and exception level are orthogonal.
- ✓ Say which EL1 registers are banked by security state and why.
- ✓ Explain the descriptor `NS` bit and the `NSTable` bit.
- ✓ Name six blocks that check the NS bit and what each does with it.
- ✓ Explain the "peripheral reads all zeros" symptom and the order in which to rule out causes.
- ✓ Give four things the NS bit does not protect against.

TrustZone: the two worlds and EL3

One bit splits the machine into two parallel software stacks, each with its own exception levels, page tables and interrupts. Between them sits EL3 — the only code that exists in both, and the most security-critical software on the chip.

The full picture



Why put anything in the secure world

Reason	Concretely
The rich OS is too big to trust	30 M lines with a continuous CVE stream. Assume compromise; keep the secrets elsewhere.
Some secrets must survive kernel compromise	Attestation keys, disk encryption keys, DRM keys, biometric templates, payment credentials
Some operations need a trusted path	"Confirm this payment" must display on a screen the normal world cannot fake and read input it cannot inject
Boot integrity needs an anchor	Secure boot verification runs before the normal world exists at all
Certification requirement	Payment, DRM and eSIM schemes mandate a certified TEE or secure element

What EL3 actually does

```
/* BL31 — the resident EL3 firmware — has four jobs and should
   have no others.                                              */

/* 1. WORLD SWITCHING. Save one world's state, flip SCR_EL3.NS,
      restore the other's, ERET. Chapter 11.                    */

/* 2. SMC DISPATCH. Decode the function ID and route it:
        PSCI            power management (ARM chapter 33)
        SiP             silicon-vendor-specific services
        Trusted OS      forward to S-EL1
        Standard        Arm-defined services (SDEI, RAS, TRNG)  */

/* 3. SECURE INTERRUPT ROUTING. SCR_EL3.{IRQ,FIQ,EA} decide
      which interrupts come to EL3 rather than the current
      world.                                                    */

/* 4. PLATFORM INITIALISATION that must happen once, at the
      highest privilege: TZASC regions, TZPC assignments, SMMU
      secure configuration — then LOCK them.                    */

/* What EL3 must NOT do:
     · parse untrusted input
     · implement a filesystem, a network stack, or crypto
       protocols beyond what boot verification needs
     · grow. TF-A's BL31 is tens of thousands of lines and that
       is a deliberate ceiling.

   Every line at EL3 can compromise BOTH worlds. It is the one
   place where "keep it small" is not a style preference.   */
```

SCR_EL3

Bit	Name	Effect
0	`NS`	**The security state of the world below.** 0 = Secure, 1 = Non-secure.
1	`IRQ`	Route IRQ to EL3
2	`FIQ`	Route FIQ to EL3 — normally set, so secure interrupts reach the monitor
3	`EA`	Route External Aborts and SError to EL3
7	`SMD`	Disable `SMC` entirely — it becomes undefined
8	`HCE`	Enable `HVC`. Cleared means no hypervisor calls.
10	`RW`	Execution state below: 1 = AArch64
11	`ST`	Allow secure EL1 access to the secure timer
12/13	`TWI` / `TWE`	Trap `WFI` / `WFE` to EL3
18	`SIF`	**Secure Instruction Fetch**: forbid the secure world fetching instructions from Non-secure memory
19	`EEL2`	Enable Secure EL2 (ARMv8.4+)

> **`SCR_EL3.SIF` IS A DEFENCE WORTH KNOWING**
> Without it, secure software tricked into branching to a Non-secure address would execute normal-world code *with secure privilege* — total compromise from one corrupted function pointer. Setting `SIF` makes such a fetch fault. It is the secure world's equivalent of `PXN` on user pages (ARM 14), and it should always be set.

The software stack, end to end

```
NORMAL WORLD                              SECURE WORLD
------------                              ------------
 Client Application (EL0)
   TEEC_InitializeContext()
   TEEC_OpenSession(uuid)
   TEEC_InvokeCommand()
        |  GlobalPlatform TEE Client API
        v
 libteec.so
        |  ioctl() on /dev/tee0
        v
 tee_core + optee driver (EL1)
   builds an SMC message,
   allocates SHARED memory
        |
        |  smc #0
        v
====== EL3 · TF-A · OP-TEE dispatcher ==================
        |
        v
                                     OP-TEE OS (S-EL1)
                                       find/load the TA by UUID
                                       VERIFY its signature
                                       set up its S-EL0 space
                                            |
                                            v
                                     Trusted Application (S-EL0)
                                       TA_InvokeCommandEntryPoint()
                                       uses the crypto engine,
                                       secure storage, the
                                       hardware unique key
                                            |
                                            v
                                       result -> shared memory
        |<---------------------------- smc return
        v
 back through the driver, libteec, to the application
```

Who implements what

Component	Runs at	Notes
TF-A (Trusted Firmware-A)	EL3, BL31	The open-source reference monitor. Also provides PSCI. Almost universal.
OP-TEE	S-EL1	Open source, GlobalPlatform-compliant. **Runs on QEMU** — the best way to learn.
Trusty	S-EL1	Google's, used in Android
QSEE / TEEGRIS / Kinibi	S-EL1	Qualcomm, Samsung, Trustonic. Closed source.
Hafnium	S-EL2	Secure Partition Manager — isolates secure partitions from each other

Component	Runs at	Notes
SPMC / FF-A	S-EL1 or S-EL2	Arm's Firmware Framework: a standard for secure partitions, replacing ad-hoc TEE interfaces

Secure EL2 and partitions

```
The original design had ONE secure OS. That is a problem when
several vendors each want a trusted component:

   a payment applet from a bank
   a DRM component from a content provider
   a biometric matcher from the OEM
   an eSIM applet from a carrier

  With one S-EL1 OS, all four are mutually trusting. A bug in
  any one compromises the others — and the whole secure world.

  ARMv8.4's SECURE EL2 fixes this: a Secure Partition Manager at
  S-EL2 gives each component its own stage-2 translation, so
  they are isolated from each other exactly as VMs are.

   S-EL0   TA          TA          TA
   S-EL1   [partition A] [partition B] [partition C]
   S-EL2   Secure Partition Manager (Hafnium)
                stage-2 isolation between partitions
   EL3     monitor

  FF-A standardises the messaging between partitions and the
  normal world, replacing the vendor-specific SMC interfaces
  that came before.

  Practically: it is how a modern phone runs several vendors'
  trusted code without any of them reading the others' keys.
```

Shared memory: the interesting part

```
/* The two worlds must exchange data. The pattern is always the
   same, and getting it wrong is the classic TEE vulnerability. */

/* 1. The NORMAL world allocates a buffer in NON-SECURE memory */
        shm = tee_shm_alloc(ctx, size, TEE_SHM_MAPPED | TEE_SHM_DMA_BUF);

/* 2. It passes the PHYSICAL ADDRESS in the SMC arguments      */

/* 3. The SECURE world maps it with NS=1 in its own page tables
      (chapter 08) and copies data in and out.                 */
```

SECURE CODE MUST NEVER TRUST A NORMAL-WORLD POINTER

Before dereferencing an address supplied by the normal world, secure software must verify that the *entire* range lies inside non-secure memory. Otherwise the normal world hands the TEE a pointer to *secure* memory and has the TEE read or write it on its behalf.

That is a **confused deputy** attack, and it is behind a substantial number of real TEE CVEs. The check must cover the whole range and must handle integer overflow in `base + length`.

```
static bool ns_range_ok(paddr_t base, size_t len)
{
        paddr_t end;

        if (len == 0)                       return false;
        if (ADD_OVERFLOW(base, len, &end)) return false;  /* wrap! */
        if (!core_pbuf_is(CORE_MEM_NON_SEC, base, len))
                return false;               /* the WHOLE range */
        return true;
}
```

```
/* And the TOCTOU rule: COPY FIRST, then validate the copy.

   WRONG — validating shared memory in place: */
        if (req->len > MAX)  return TEE_ERROR_BAD_PARAMETERS;
        process(req->buf, req->len);
        /* the normal world can change req->len between the two
           reads, because it still has the memory mapped       */

/* RIGHT — one copy into secure memory, then validate: */
        struct req local;
        memcpy(&local, ns_req, sizeof local);   /* from NS memory */
        if (local.len > MAX)  return TEE_ERROR_BAD_PARAMETERS;
        process(local.buf, local.len);          /* the COPY       */
```

What a world switch costs

```
   SMC instruction                        ~20 cycles
   exception entry to EL3                 ~30 cycles
   save the outgoing world's state        ~200-1000 cycles
     X0-X30, FP/SIMD if used, any
     non-banked system registers
   dispatch the function ID               ~50 cycles
   restore the incoming world's state     ~200-1000 cycles
   ERET                                   ~20 cycles
   ------------------------------------------------
   ONE WAY                                ~500-2000 cycles

   A round trip is 1,000-10,000 cycles, and on most designs the
   calling core is unavailable for the whole period.

  CONSEQUENCES for design:
     · never put an SMC in a hot path
     · BATCH work into one call rather than many small ones
     · never call from an interrupt handler or with a spinlock
       held
     · a TEE operation needed at high frequency is probably in
       the wrong place

  Compare: a system call is ~50-500 ns; a VM exit ~500-2000
  cycles. A world switch is the most expensive boundary
  crossing on the chip.
```

Interrupts across the boundary

```
/* The GIC assigns every interrupt to a GROUP (ARM chapter 23),
   and the group plus SCR_EL3 routing decides where it lands.  */

   Group 0            -> FIQ, always to EL3
                         secure firmware's own interrupts
   Secure Group 1     -> IRQ in the secure world,
                         FIQ in the non-secure world
                         the trusted OS's interrupts
   Non-secure Group 1 -> IRQ
                         the normal OS's interrupts

/* THE KEY PROPERTY: a Non-secure OS clearing PSTATE.I cannot
   mask secure interrupts, because they arrive as FIQ and route
   to EL3. The secure world can ALWAYS preempt the normal world.

   That is what makes a TEE's timing guarantees meaningful.  */

/* The reverse case: a normal-world interrupt arriving while the
   SECURE world is running. The monitor must either
     · hold it until the secure operation completes — simple,
       but adds latency to the normal world, or
     · return to the normal world with an "operation preempted,
       call again" status, so long secure operations do not
       block the OS.

   OP-TEE does the latter, via RPC returns. That is why a TEE
   call can return a retry status the client must handle.   */
```

Running the whole stack on QEMU

```
# A complete TrustZone system — TF-A at EL3, OP-TEE at S-EL1,
# Linux at EL1 — on your laptop, no hardware needed. The single
# best way to make this chapter concrete.

mkdir optee && cd optee
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
repo sync -j8
cd build
make toolchains -j4
make run -j$(nproc)

# TWO consoles appear: the normal world (Linux) and the secure
# world (OP-TEE's log). Watch both.

# in the Linux console:
xtest                          # the OP-TEE conformance suite
optee_example_hello_world
optee_example_secure_storage
optee_example_acipher

# Watch the SECURE console while running these: the TA loading,
# the command dispatch, and the return, live.
```

READ `OPTEE_EXAMPLES/HELLO_WORLD/`

About 150 lines split between a normal-world client (TEE Client API) and a Trusted Application (with `TA_InvokeCommandEntryPoint`). Tracing one command through both halves — client, `ioctl`, driver, SMC, monitor, trusted OS, TA, and back — teaches more than any amount of reading.

How it shows up

Symptom	Cause
`SMC` causes an undefined instruction	EL3 not implemented, or `SCR_EL3.SMD` set
TEE calls are very slow	World switch cost — batch the work
System stalls during a TEE operation	No preemption path for normal-world interrupts
One TA can read another's secrets	No S-EL2 partitioning — all TAs are mutually trusting
Normal world crashes the TEE with a bad pointer	Missing non-secure range validation — confused deputy
A validated length changes before use	TOCTOU — validated in shared memory instead of after copying
Secure world executes normal-world code	`SCR_EL3.SIF` not set
Linux cannot mask an interrupt	It is a secure Group 0 or Secure Group 1 interrupt
TEE client receives a retry status	The operation was preempted to service a normal-world interrupt — retry is correct

Check yourself

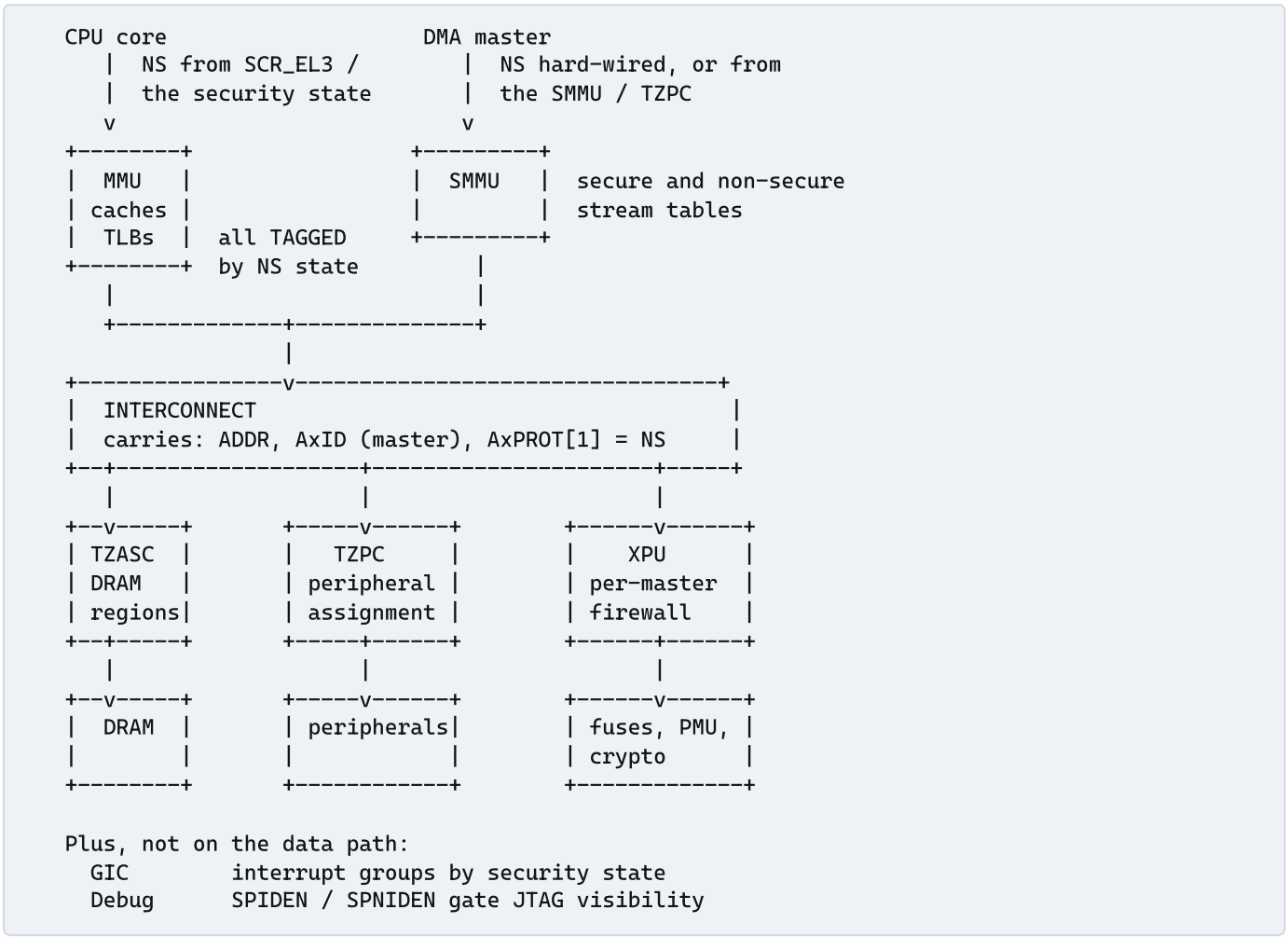
- ✓ Draw the two worlds with their exception levels and say what runs at each.
- ✓ Give EL3's four jobs and three things it must not do.

- ✓ Explain what `SCR_EL3.NS`, `SMD` and `SIF` each control.
- ✓ Explain why `SIF` exists and what it prevents.
- ✓ Trace a TEE call from client application to trusted application and back.
- ✓ Explain what Secure EL2 adds and the problem it solves.
- ✓ Give the confused deputy pattern and write a correct range check, including the overflow case.
- ✓ Explain the TOCTOU rule for shared memory.
- ✓ Say why a secure interrupt cannot be masked by the normal OS.

TrustZone: hardware enforcement blocks

The NS bit is only a promise until something checks it. This chapter covers the blocks that actually do the checking — where they sit, what they can express, and what a partial implementation leaves open.

Where the checks happen



TZASC — memory regions

```
/* The TrustZone Address Space Controller sits in front of the
   memory controller and applies a REGION TABLE.             */

   Per region:
      base address, size (power of two, or a range)
      security setting: Secure only / Non-secure only / both
      per-master read and write permission bitmap
      subregion disable mask (split a region into 8 parts)
      LOCK bit - write-once until reset

   A Non-secure access to a Secure region gets an ERROR
   RESPONSE, which becomes an external abort at the CPU
   (ARM chapter 29, DFSC 0b010000) - or, on some
   configurations, silently returns zeros.

/* Arm's TZC-400 is the common licensed implementation; vendors
   also build their own. Typical: 8 or 9 regions, one of which
   (region 0) is the DEFAULT covering everything not otherwise
   described.                                                */

/* Configuring it - firmware only, and once: */
static void tzc_configure_region(int region, uint64_t base,
                                 uint64_t top, unsigned sec_attr,
                                 unsigned ns_master_bitmap)
{
        tzc_write_region_base(region, base);
        tzc_write_region_top(region, top);
        tzc_write_region_attributes(region,
                (sec_attr << TZC_ATTR_SEC_SHIFT) | TZC_ATTR_REGION_EN);
        tzc_write_region_id_access(region, ns_master_bitmap);
}

/* And the default region must be RESTRICTIVE: */
        tzc_set_action(TZC_ACTION_ERR);          /* fail loudly   */
        tzc_configure_region0(TZC_REGION_S_RDWR, /* secure only   */
                              0);                /* no NS masters */
        /* then open up the specific ranges the normal world needs */

/* Getting region 0 wrong - leaving it permissive - means every
   address you did not explicitly describe is wide open. This is
   the fail-closed principle from chapter 01, applied.      */
```

TZPC — peripheral assignment

```
/* The TrustZone Protection Controller marks whole peripherals
   Secure or Non-secure, often at RUNTIME.                    */

   TZPCDECPROT<n>SET     mark a peripheral NON-SECURE
   TZPCDECPROT<n>CLR     mark it SECURE
   TZPCR0SIZE            how much on-chip SRAM is secure

/* Each bit corresponds to one peripheral, per the SoC's TRM.
   A Non-secure access to a Secure-marked peripheral is refused
   at the peripheral's bus interface.                         */

/* Why RUNTIME assignment is useful:
     · a fingerprint sensor is Secure while enrolling or
       matching, Non-secure the rest of the time
     · a display controller becomes Secure for a trusted-UI
       prompt, then returns
     · a crypto engine is claimed by the TEE for one operation

   The secure world flips the bit, does the work, and flips it
   back. The normal world's driver simply finds the device
   unresponsive in the meantime — which is exactly the
   "reads all zeros" symptom of chapter 08, appearing and
   disappearing.                                              */

/* Which makes for a genuinely confusing bug report: "the sensor
   works, except sometimes". Check whether a TEE operation is
   claiming it.                                               */
```

Caches and TLBs

```
Neither is "banked" — instead every entry is TAGGED with the
security state that created it.

   cache line tag:   address + NS state
   TLB entry tag:    VA + ASID + VMID + NS state + regime

  So a Non-secure lookup cannot HIT a Secure line, even at the
  same address. The two worlds' data coexists in one cache
  without leaking.

  CONSEQUENCES:

  · no cache flush is needed on a world switch for CORRECTNESS,
    which is a large part of why the switch is as fast as it is

  · but a flush may be needed for SECURITY: the two worlds share
    the cache as a RESOURCE, so timing differences leak
    information (chapter 16). Hardened monitors flush or
    partition on switch, at a real performance cost.

  · TLB maintenance from the secure world must specify the right
    regime. TLBI ALLE3 does not invalidate EL1 entries, and
    Secure EL1 entries are distinct from Non-secure EL1 entries
    even at the same VA and ASID.

  · a cache line's NS tag comes from the ACCESS, not the
    address. Secure software creating an NS=1 mapping to shared
    memory (chapter 08) produces Non-secure lines — which is
    correct, and is how sharing works without aliasing.
```

GIC — interrupt groups

Group	Signalled as	Taken to	Used for
Group 0	FIQ	Always EL3	Secure firmware's own interrupts
Secure Group 1	IRQ in the secure world; FIQ in the non-secure world	S-EL1, or EL3	Trusted OS interrupts
Non-secure Group 1	IRQ	EL1 (or EL2)	The normal OS

```
/* Configured in GICD_IGROUPR and GICD_IGRPMODR, writable only
   from the secure world.                                      */

/* The property that matters: a Non-secure OS clearing PSTATE.I
   masks only IRQ. A secure interrupt arrives as FIQ and is
   routed to EL3 by SCR_EL3.FIQ. The normal world CANNOT
   suppress it.

   Without this, a compromised kernel could simply disable
   interrupts and prevent the secure world from ever running —
   a denial of service against the TEE, and a way to defeat any
   secure watchdog.                                            */

/* And the reverse: GICD registers themselves are usually
   Secure-only for the group configuration, so the normal world
   cannot reassign an interrupt to its own group.

   On GICv3 the redistributor and CPU interface state are also
   banked by security state.                                   */
```

SMMU secure interface

```
/* SMMUv3 has TWO complete programming interfaces (chapter 04): */

   +0x0000   Non-secure   SMMU_*
   +0x8000   Secure       SMMU_S_*

/* Each with its own stream table, command queue, event queue and
   enable bit. Independent.

   WHY: a device operating in the secure world must be configured
   by SECURE firmware. If the Non-secure OS could program that
   device's translations, it could point a secure DMA engine at
   normal-world memory — reading secure data out through a
   device the normal world controls. A complete TrustZone
   bypass.

   And the STE and CD carry NSCFG fields determining whether the
   resulting transaction is issued Secure or Non-secure, exactly
   as the page descriptor NS bit does for the CPU.

   So the SMMU is not merely an isolation device for the normal
   world — it is part of the TrustZone boundary itself.     */

/* A common integration gap: an SoC implements the SMMU but not
   its secure interface (SMMU_IDR1.SECURE_IMPL = 0). Secure DMA
   masters are then unprotected, and must be constrained by an
   XPU instead (chapters 17–20).                               */
```

Fuses and one-time-programmable storage

Holds	Read by	Note
Root of Trust public key hash	Boot ROM only, or Secure masters	The anchor for secure boot (ch. 14)
Hardware Unique Key (HUK)	Often only the crypto engine, never software	Per-device; the basis for secure storage
Anti-rollback counters	Secure firmware	Monotonic; prevents downgrade
Lifecycle state	Boot ROM and firmware	Blank / development / secured / RMA

Holds	Read by	Note
Debug enable	The debug authentication logic	Gates `SPIDEN` / `SPNIDEN`
Feature disable	Various	Vendor SKU differentiation

> **THE BEST-DESIGNED HUK IS NEVER VISIBLE TO SOFTWARE**
>
> On a well-built SoC, the hardware unique key is wired directly into the crypto engine's key input. Software asks the engine to "derive a key using the HUK" and receives a *derived* key; it never sees the HUK itself. Even a compromised trusted OS cannot extract it.
>
> A design where software reads the HUK into a register is much weaker — a single memory-disclosure bug in the secure world then reveals the root of all device storage encryption, permanently and unfixably.

Debug authentication

```
Four signals gate what a debugger may see:

   DBGEN     invasive debug (halt, single-step) in the NORMAL world
   NIDEN     non-invasive (trace, PMU) in the normal world
   SPIDEN    invasive debug of the SECURE world
   SPNIDEN   non-invasive of the secure world

  Driven by fuses, by a debug authentication module, or by a
  signed unlock token.

  Typical configurations:
     development part   all four asserted
     production part    DBGEN/NIDEN maybe, SPIDEN/SPNIDEN OFF
     locked-down        all four off

  CONSEQUENCES you will meet:
     · JTAG attaches but cannot see secure state — you can halt
       the core in the normal world and not in the secure one
     · the PMU counts normal-world events only, so a profile
       silently omits secure execution
     · CoreSight trace stops at the world boundary
     · a TEE bug that reproduces only on production silicon is
       very hard to debug — which is why development firmware
       should log aggressively

  RMA UNLOCK: a controlled path back, requiring a token signed
  by the vendor and usually destroying the device keys first so
  that unlocking cannot expose user data.
```

The rest

Block	Enforces
Secure boot ROM	Immutable first stage; verifies everything after it
Crypto engine	Key slots accessible only to Secure masters; some keys never leave the block
True RNG	Entropy source, often Secure-only or with a Secure-only health-test interface
Secure timer / watchdog	A watchdog the normal world cannot disable — so a hung TEE is recoverable
Secure SRAM	On-chip memory, sometimes with no external bus path at all — used for keys in use

Block	Enforces
Memory encryption	Encrypts DRAM contents so bus probing and cold boot recover only ciphertext
Boot-mode straps	Latched at reset; often Secure-readable only, so the normal world cannot learn or change the boot source

Where implementations fall short

ENFORCEMENT IS ONLY AS COMPLETE AS THE SILICON

TrustZone is a system-wide contract, and a partial implementation is a bypass. Real gaps that have shipped:

- **A peripheral that ignores the NS bit.** Its registers are reachable from both worlds regardless of the TZPC.
- **A bus segment that drops or hard-codes NS.** Everything downstream sees a fixed value.
- **A DMA master with no SMMU and no XPU.** Full access to all memory, from whichever world can program it.
- **A TZASC default region left permissive.** Anything not explicitly protected is open.
- **Debug signals not fused off in production.** The most common finding in device security audits.
- **A configuration register left unlocked.** A compromised secure world — or in bad cases the normal world — can widen the policy at runtime.

Verifying each of these is a real part of SoC security review, and it is done by *testing the negative cases*, not by reading the datasheet.

Verifying the configuration

```
/* Write these as a boot-time self-test in development firmware.
   A policy never exercised is a policy you do not know works. */

static void security_selftest(void)
{
        /* 1. Non-secure access to Secure DRAM must FAIL */
        expect_abort(read32_ns(SECURE_DRAM_BASE));

        /* 2. Non-secure access to a Secure peripheral must FAIL */
        expect_abort(read32_ns(SECURE_CRYPTO_BASE));

        /* 3. Secure access to Non-secure DRAM must SUCCEED */
        expect_ok(read32_s(NS_DRAM_BASE));

        /* 4. the TZASC default region must be restrictive */
        assert(tzc_region0_attr() == TZC_REGION_S_RDWR);

        /* 5. every region that should be locked, is */
        for (i = 0; i < nr_regions; i++)
                assert(tzc_region_locked(i) == expected_lock[i]);

        /* 6. debug signals match the lifecycle state */
        if (lifecycle_state() == LC_SECURED)
                assert(!spiden_asserted() && !spniden_asserted());

        /* 7. SCR_EL3 has the expected bits */
        assert(read_scr_el3() & SCR_SIF);
        assert(read_scr_el3() & SCR_FIQ);
        assert(read_scr_el3() & SCR_EA);
}

/* Run it in development builds, on a part in the FINAL
   lifecycle state. Chapter 19 covers building the complete
   access matrix systematically.                          */
```

How it shows up

Symptom	Cause
Peripheral reads zeros or aborts on production units	TZPC marks it Secure
A device works "except sometimes"	Runtime TZPC reassignment during a TEE operation
Linux cannot see a DRAM region in the TRM	Carved out as Secure by the TZASC
External abort, DFSC `0b010000`	TZASC, TZPC or XPU refused the access
Access silently returns zeros instead of aborting	The controller's action is "return zeros" — much harder to debug (ch. 20)
Normal world reads secure memory successfully	**A security bug** — a permissive default region, or a block ignoring NS
JTAG halts in Linux but not in the TEE	`SPIDEN` deasserted — by design
Profile omits secure execution	`SPNIDEN` deasserted
Secure DMA master can be redirected by Linux	SMMU secure interface not implemented or not used
Policy changes at runtime	Configuration registers not locked

Check yourself

- ✓ Name the blocks that check the NS bit and where each sits.
- ✓ Explain what a TZASC region entry contains and why the default must be restrictive.
- ✓ Give a use for runtime TZPC reassignment and the confusing symptom it produces.
- ✓ Explain why caches need no flush on a world switch for correctness, but may for security.
- ✓ Explain why a secure interrupt cannot be masked by the normal OS, and what that prevents.
- ✓ Say why the SMMU needs a separate secure programming interface.
- ✓ Explain why the best HUK designs never expose the key to software.
- ✓ Name the four debug authentication signals and what each gates.
- ✓ List five ways a partial implementation creates a bypass.

The SMC calling convention and the world switch

One instruction crosses the security boundary. This chapter follows it: the register convention, what EL3 does on the way through, how a long secure operation is interrupted, and why the whole design is shaped by the fact that only one world runs at a time.

The instruction

```
        SMC     #imm16          /* Secure Monitor Call */

/* Behaves exactly like SVC and HVC (ARM chapter 30), one level
   up:

     SVC  from EL0  -> EL1     the syscall
     HVC  from EL1  -> EL2     the hypercall
     SMC  from EL1  -> EL3     the secure monitor call

   On execution:
     ELR_EL3   <- address of the instruction AFTER the SMC
     SPSR_EL3  <- PSTATE, including the security state
     ESR_EL3   <- EC 0x17 (AArch64 SMC), ISS = the imm16
     PC        <- VBAR_EL3 + 0x400  (sync, lower EL, AArch64)

   The imm16 is CAPTURED but not used by any real convention -
   everybody passes the function ID in X0 instead, because that
   gives 32 bits instead of 16 and can be computed at runtime. */
```

SMC CAN BE DISABLED, OR STOLEN

`SCR_EL3.SMD = 1` makes `SMC` undefined at EL1 and EL2. And separately, `HCR_EL2.TSC = 1` makes an EL1 `SMC` trap to *EL2* instead of EL3 — which is how a hypervisor filters or virtualises the calls its guests make. A guest that thinks it is talking to firmware may be talking to the hypervisor.

SMCCC — the calling convention

```
Arm's SMC Calling Convention makes the function ID in X0
self-describing:

  bit 31     call type      0 = Yielding, 1 = Fast
  bit 30     calling conv   0 = SMC32,    1 = SMC64
  bits 29:24 owning entity
  bits 23:16 reserved / hint
  bits 15:0  function number

  OWNING ENTITY ranges:
     0x00  Arm architecture calls (SMCCC_VERSION, workarounds)
     0x01  CPU service        (PSCI lives here)
     0x02  SiP service        (vendor firmware)
     0x03  OEM service        (board integrator)
     0x04  Standard Secure    (PSCI, SDEI, FF-A, TRNG)
     0x05  Standard Hypervisor
     0x06  Vendor Hypervisor
     0x30-0x31  Trusted Application calls
     0x32-0x3F  Trusted OS calls

  FAST vs YIELDING:
     Fast     atomic-ish, must not be preempted, returns quickly.
              PSCI, version queries, register reads.
     Yielding may block, may be interrupted by a normal-world
              interrupt, may need to be resumed. TEE requests.

  SMC32 vs SMC64: whether arguments are 32-bit values in W
  registers or 64-bit in X registers. A 32-bit normal world can
  only issue SMC32.
```

```
/* Registers, SMC64: */

   X0        function ID          -> return value / status
   X1..X6    arguments            -> return values X1..X3
   X7        hint / session       (SMCCC 1.2: also an argument)
   X8..X17   volatile             the callee may clobber
   X18       platform register    preserved
   X19..X29  callee-saved         PRESERVED across the SMC
   X30, SP   preserved

/* SMCCC 1.2 extends the argument and result set to X0..X17 for
   SMC64, which is what FF-A uses.

   Standard return codes:
        0   SUCCESS
       -1   NOT_SUPPORTED          (also: unknown function ID)
       -2   INVALID_PARAMETER
       -3   DENIED
   Anything else is service-specific.                          */

/* The unknown-ID rule matters: an SMC with a function ID nobody
   claims must return -1 in X0, not hang and not panic. That is
   how a caller DISCOVERS what firmware supports - it asks, and
   a clean "no" is a valid answer.                             */
```

What EL3 actually does

THE MONITOR IS A CONTEXT SWITCHER, NOT A SERVICE

Almost everything EL3 does is save-and-restore. The interesting work happens in the trusted OS at S-EL1; EL3's job is to make sure that when it starts running, none of the normal world's state is visible to it and vice versa. This is why EL3 code is measured in tens of kilobytes, and why the design goal is to keep it that way (chapter 09).

What the switch costs

Step	Rough cost	Why
SMC exception entry	~30–60 cycles	Pipeline flush, same as any exception
Save NS context	~200–500 cycles	~40 system registers; `MSR` / `MRS` are not cheap and some serialise
Restore secure context	~200–500 cycles	Same again, the other way
`ERET`	~20–40 cycles	Another pipeline flush
FP/SIMD save and restore	+~100 cycles	Skipped if `CPACR` shows it unused; SVE is far worse
Cold caches and TLBs in the other world	Highly variable	The real cost, and the one nobody measures

```
/* Round trip: roughly 1000–4000 cycles for a trivial call, plus
   whatever the trusted OS does, plus the cache effects on both
   sides.

   At 1.5 GHz that is a few microseconds of pure overhead. Fine
   at 100 calls per second. Ruinous at 100,000.

   DESIGN CONSEQUENCE: batch. A TEE interface that asks the
   secure world to encrypt one 16-byte block per SMC will be
   dominated entirely by switching. Pass a buffer of a megabyte
   and the overhead disappears into the noise.

   The same shape of argument as syscall batching (OS chapter 3)
   and as MMIO round trips (ARM chapter 25) — crossing a
   boundary is expensive, so cross it with as much work as you
   can carry.                                                  */
```

Interrupting a long secure call

```
Problem: the trusted OS is halfway through a 50 ms operation and
a normal-world timer interrupt fires. Only one world runs at a
time, so the normal world is FROZEN. Miss enough ticks and Linux
notices - RCU stalls, softlockup warnings, dropped audio.

Solution: the Non-secure interrupt is routed to EL3
(SCR_EL3.IRQ = 1 while in the secure world), which returns to
the normal world with a "please call me back" status.

   Linux                EL3             Trusted OS
   -----                ---             ----------
   SMC(op) ---------->  switch  ------->  starts work
                                             ...
                       <-- NS IRQ arrives, taken to EL3
   <-- X0 = INTERRUPTED, with a resume token
   handles the IRQ
   scheduler runs, other tasks run
   SMC(RESUME, token) ->  switch ----->  continues where it
                                         left off
   <-- X0 = SUCCESS ------------------- done

  In OP-TEE this is OPTEE_SMC_RETURN_RPC_* and RETURN_FROM_RPC.
  In FF-A it is FFA_INTERRUPT and FFA_RUN.

  The secure world must therefore be written so it CAN be
  suspended mid-operation: no unbounded critical sections, and
  state kept in the session rather than on a stack it assumes
  will still be there.

  This is also why YIELDING calls exist as a category. A Fast
  call promises it is short enough that this machinery is
  unnecessary.
```

RPC — the secure world calling back

```
/* The trusted OS has no drivers. It cannot read a file, allocate
   from the kernel, or talk to storage. When it needs any of
   that it returns to the normal world with a REQUEST.       */

   Trusted OS needs to load a TA from disk
        |
        v
   returns X0 = RPC_LOAD_TA, X1 = uuid...
        |
        v
   EL3 switches to the normal world
        |
        v
   tee-supplicant (a userspace daemon) reads the file
        |
        v
   places it in shared memory, SMC(RETURN_FROM_RPC)
        |
        v
   Trusted OS resumes, now with the data

/* Common RPC types: allocate shared memory, free it, load a TA,
   read or write secure storage, get the wall-clock time,
   perform a network request to a provisioning server.       */

/* SECURITY CONSEQUENCE, and it is a big one:

   Everything obtained by RPC comes from an untrusted source.
   The normal world can:
        - return a DIFFERENT TA binary  -> VERIFY THE SIGNATURE
        - return stale secure storage   -> ANTI-ROLLBACK COUNTER
        - lie about the time            -> never trust it for expiry
        - refuse to answer at all       -> handle the failure
        - answer differently each read  -> COPY IN, then validate

   The trusted OS depends on the normal world for AVAILABILITY -
   which is fine, since a normal world that refuses to cooperate
   has only denied itself service. It must never depend on it
   for INTEGRITY.                                            */
```

PSCI: the SMC interface everyone uses

```
/* Most SMCs on a running Linux system never reach a trusted OS at
   all. They are PSCI - Power State Coordination Interface -
   handled entirely inside EL3.                                  */

   PSCI_VERSION            0x84000000
   CPU_SUSPEND             0xC4000001
   CPU_OFF                 0x84000002
   CPU_ON                  0xC4000003
   AFFINITY_INFO           0xC4000004
   SYSTEM_OFF              0x84000008
   SYSTEM_RESET            0x84000009
   PSCI_FEATURES           0x8400000A
   SYSTEM_RESET2           0xC4000012

/* Linux brings up a secondary core: */

static int psci_cpu_on(unsigned long cpuid, unsigned long entry)
{
        return invoke_psci_fn(PSCI_FN_NATIVE(0_2, CPU_ON),
                              cpuid, entry, 0);
}

/* invoke_psci_fn is arm_smccc_smc() or arm_smccc_hvc() depending
   on the CONDUIT, which is read from the device tree:

        psci {
                compatible = "arm,psci-1.0";
                method     = "smc";
        };

   Under a hypervisor the method is "hvc" instead, and the
   hypervisor either handles the call or forwards it. A wrong
   conduit is a classic bring-up failure: SMC from a guest traps
   to EL2 (HCR_EL2.TSC) or is simply undefined, and the
   secondary CPUs never come up.                                 */

/* Which is why "only CPU0 is online" is almost always a PSCI
   problem, not a scheduler problem (ARM chapter 33).        */
```

Issuing an SMC from Linux

```
#include <linux/arm-smccc.h>

static int my_sip_call(u32 arg)
{
        struct arm_smccc_res res;

        arm_smccc_smc(MY_SIP_FUNC_ID,   /* X0 */
                      arg,              /* X1 */
                      0, 0, 0, 0, 0, /* X2..X7 */
                      &res);

        if ((long)res.a0 < 0)
                return -EINVAL;

        return res.a1;                  /* X1 came back here */
}

/* res holds a0..a3 = X0..X3.

   arm_smccc_smc() emits SMC; arm_smccc_hvc() emits HVC. Use
   arm_smccc_1_1_invoke() to pick automatically based on the
   detected conduit - that is what generic code should do.   */

/* CONSTRAINTS on the caller:
     - the call may take a long time; do not hold a spinlock
     - it does not sleep in the Linux sense, but it does BLOCK
       the CPU, so a Yielding call has no place in a truly
       atomic context
     - arguments passed by pointer must be PHYSICAL addresses in
       memory the secure world can reach - a kmalloc'd buffer's
       virtual address is meaningless there
     - that memory must be non-secure and inside the region the
       secure world will accept (chapter 08)
     - and it must be flushed to the point of coherency if the
       two worlds' mappings differ in cacheability           */
```

> **WATCHING SMC TRAFFIC**
> Build TF-A with `DEBUG=1 LOG_LEVEL=40` and it prints every call it handles on the UART. On a QEMU virt machine with OP-TEE this is the fastest way to see the real sequence: PSCI during boot, then the OP-TEE calls once `tee-supplicant` starts, then the RPC round trips inside each one.

FF-A: the newer shape

```
/* The Firmware Framework for Arm A-profile replaces "one trusted
   OS reached by vendor-specific SMCs" with a message-passing
   model between PARTITIONS.                                    */

   FFA_VERSION               negotiate
   FFA_ID_GET                who am I
   FFA_PARTITION_INFO_GET    who else is here
   FFA_MSG_SEND_DIRECT_REQ   send a message, block for the reply
   FFA_MSG_SEND_DIRECT_RESP  the reply
   FFA_MEM_SHARE             share pages with a partition
   FFA_MEM_LEND / DONATE     transfer them
   FFA_MEM_RECLAIM           take them back
   FFA_RUN                   resume a partition
   FFA_INTERRUPT             it was preempted

/* WHY IT MATTERS:
     - several mutually distrusting secure partitions can coexist
       under a Secure EL2 hypervisor (chapter 09), instead of one
       monolithic trusted OS
     - memory sharing is EXPLICIT and revocable, with the manager
       tracking who has what - not an ad hoc agreed region
     - the same interface works for normal-world VMs, so a
       hypervisor can broker access rather than passing SMCs
       through blind

   OP-TEE can run as an FF-A secure partition, and new designs
   generally should. But the older direct-SMC interface is what
   you will meet on most shipping hardware.                     */
```

Designing an SMC interface

RULES THAT KEEP THE BOUNDARY SAFE

- **Validate every argument, every time.** The caller is the attacker. Ranges, sizes, alignment, handles, enum values.
- **Copy shared-memory arguments in before validating.** The other world can rewrite them between your check and your use (chapter 09).
- **Never return a secure pointer.** Return an opaque handle indexed into a table you own.
- **Make calls idempotent or explicitly sequenced.** The normal world can retry, replay, or call from two CPUs at once.
- **Assume concurrency.** Every core can be inside the same call simultaneously; the trusted OS needs real locking.
- **Bound the work.** No operation may run for an unbounded time, because it stalls the normal world.
- **Return `NOT_SUPPORTED` for unknown IDs** — discovery depends on it.
- **Version the interface** from the first release, and negotiate.
- **Keep the surface small.** Every function ID is an entry point into the most privileged code on the SoC.

How it shows up

Symptom	Cause
`SMC` is an undefined instruction	`SCR_EL3.SMD = 1`, or no EL3 implemented
SMC lands in the hypervisor instead of firmware	`HCR_EL2.TSC = 1`
Returns `-1` for a call you implemented	Function ID mismatch, or the SMC32/SMC64 bit is wrong
Only CPU0 comes online	PSCI conduit wrong in DT, or `CPU_ON` failing — check the TF-A log
Secure world reads garbage from your buffer	Virtual address passed instead of physical, or cache not flushed
Works once, fails on retry	Non-idempotent call, or session state not reset
RCU stalls and softlockups during TEE use	A long secure operation that never yields to normal-world interrupts
Random corruption under load	Trusted OS not locking against concurrent calls from multiple cores
Hang inside `arm_smccc_smc`	Firmware never returned — a secure-world crash or deadlock
Callee-saved registers corrupted after the call	Firmware violating the SMCCC — a real firmware bug

Check yourself

- ✓ Place `SMC` alongside `SVC` and `HVC`, and say what each targets.
- ✓ Decode an SMCCC function ID: type, convention, owning entity, number.
- ✓ Explain the difference between a Fast and a Yielding call.
- ✓ List what EL3 saves and restores on a world switch, and estimate the cost.
- ✓ Explain why a TEE interface should batch work.
- ✓ Describe how a long secure operation is interrupted and resumed.
- ✓ Explain what RPC is and why nothing returned by it may be trusted for integrity.
- ✓ Name five PSCI functions and say why a wrong conduit stops secondary CPUs booting.
- ✓ Write a Linux SMC call and state the constraints on any pointer you pass.
- ✓ Give five rules for designing a safe SMC interface.

The TEE software stack and OP-TEE

Six pieces of software cooperate to run one trusted operation. This chapter names every one of them, follows a single request through all six, and gets OP-TEE running on QEMU so you can watch it happen.

The whole stack

```
NORMAL WORLD                          SECURE WORLD
============                          ============

your application                      Trusted Application
     |  libteec                           |  TA framework
     |  (GlobalPlatform Client API)       |  (GP Internal API)
     v                                    ^
+----------------+                  +----+-------------+
| tee-supplicant |                  |   OP-TEE OS      |  S-EL1
| (daemon, EL0)  |                  |   (trusted OS)   |
+--------+-------+                  +----+-------------+
         |  RPC                           ^
         v                                  |
+----------------------+                    |
| optee driver         |  EL1               |
| drivers/tee/optee    |                    |
+----------+-----------+                    |
           |  SMC                           |
           v                                |
+-----------------------------------------+------------+
|  TF-A  BL31 - the secure monitor              EL3     |
+-------------------------------------------------------+

Six pieces, each with one job:

1. your app        wants a secret operation performed
2. libteec         turns API calls into ioctls on /dev/tee0
3. optee driver    turns ioctls into SMCs, manages shared memory
4. TF-A BL31       switches worlds
5. OP-TEE OS       schedules and isolates trusted applications
6. the TA          does the actual work

plus tee-supplicant, off to the side, servicing RPC requests
the secure world cannot perform for itself (chapter 11).
```

The GlobalPlatform APIs

```
/* Two standard APIs, one on each side of the boundary. Learn
   these and your knowledge transfers between OP-TEE, Trusty,
   QSEE and Kinibi.                                          */

/* --- CLIENT API, normal world, libteec ------------------- */

   TEEC_InitializeContext()   connect to the TEE
   TEEC_OpenSession()         start talking to a TA (by UUID)
   TEEC_InvokeCommand()       ask it to do something
   TEEC_CloseSession()
   TEEC_FinalizeContext()
   TEEC_RegisterSharedMemory()  register your own buffer
   TEEC_AllocateSharedMemory()  or let the TEE allocate one

/* --- INTERNAL API, secure world, inside the TA ----------- */

   TA_CreateEntryPoint()          once, when the TA is loaded
   TA_OpenSessionEntryPoint()     per client session
   TA_InvokeCommandEntryPoint()   per command  <- the real work
   TA_CloseSessionEntryPoint()
   TA_DestroyEntryPoint()

   TEE_AllocateOperation()        crypto
   TEE_CipherInit/Update/DoFinal()
   TEE_DigestDoFinal()
   TEE_AsymmetricSignDigest()
   TEE_GenerateRandom()
   TEE_DeriveKey()

   TEE_CreatePersistentObject()   secure storage (chapter 13)
   TEE_OpenPersistentObject()
   TEE_ReadObjectData()

   TEE_Malloc/Free()              heap, inside the TA
   TEE_CheckMemoryAccessRights()  IS this pointer safe to touch?
   TEE_Panic()                    give up loudly
```

The parameter model

```
/* Every invocation carries FOUR parameter slots. Each slot is
   either a VALUE (two 32-bit numbers) or a MEMREF (a shared
   buffer), and each has a direction.                          */

   TEEC_NONE
   TEEC_VALUE_INPUT        a.b: two u32 in
   TEEC_VALUE_OUTPUT       two u32 out
   TEEC_VALUE_INOUT
   TEEC_MEMREF_TEMP_INPUT      a buffer, registered for this call
   TEEC_MEMREF_TEMP_OUTPUT
   TEEC_MEMREF_TEMP_INOUT
   TEEC_MEMREF_WHOLE           a pre-registered shared memory
   TEEC_MEMREF_PARTIAL_INPUT   part of a pre-registered one
   TEEC_MEMREF_PARTIAL_OUTPUT
   TEEC_MEMREF_PARTIAL_INOUT

/* Packed into one word: */
   op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
                                    TEEC_MEMREF_TEMP_OUTPUT,
                                    TEEC_NONE,
                                    TEEC_NONE);

/* And the TA MUST check that it got what it expected: */

TEE_Result TA_InvokeCommandEntryPoint(void *sess, uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INPUT,
                                       TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                       TEE_PARAM_TYPE_NONE,
                                       TEE_PARAM_TYPE_NONE);

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;   /* not optional */
        ...
}

/* Skipping that check means the client can pass a VALUE where
   you expect a MEMREF. You then read params[1].memref.buffer -
   which is now an arbitrary 64-bit number the attacker chose,
   dereferenced by the most privileged code in the system.

   This is the single most common OP-TEE TA vulnerability
   class.                                                      */
```

EVERY BUFFER FROM THE CLIENT IS HOSTILE

A MEMREF points into *non-secure shared memory*. The client can change its contents at any moment, including between your bounds check and your read — and on another CPU it does not even need your call to return first.

- Check `params[i].memref.size` before trusting it, and check it against what you will actually write.
- Call `TEE_CheckMemoryAccessRights()` to confirm the buffer is where it claims to be.
- **Copy the input into TA-local memory, then validate the copy, then use only the copy.**
- Write outputs once, at the end. Never use the shared buffer as scratch space.
- Never place a secret in an output buffer before deciding the caller is entitled to it.

One request, end to end

```
app: TEEC_InvokeCommand(&sess, CMD_SIGN, &op, &origin)
  |
  v
libteec: build struct tee_ioctl_invoke_arg, translate the
         parameters, ioctl(fd, TEE_IOC_INVOKE, &arg)
  |
  v
optee driver (optee_invoke_func):
     get a free "call queue" entry
     translate each MEMREF: virtual -> the shared-memory cookie
       and physical address the secure world understands
     fill an optee_msg_arg in the shared memory pool
     arm_smccc_smc(OPTEE_SMC_CALL_WITH_ARG, msg_pa, ...)
  |
  v
TF-A BL31: save NS context, SCR_EL3.NS = 0, restore secure
           context, ERET to S-EL1
  |
  v
OP-TEE OS:
     entry from the monitor, pick the thread for this call
     find the session, find the TA
     is the TA loaded? -> if not, RPC to tee-supplicant to
                          fetch it, VERIFY ITS SIGNATURE, map it
     map the shared buffers into the TA's address space,
       NON-SECURE, non-executable
     drop to S-EL0 and call TA_InvokeCommandEntryPoint
  |
  v
TA: validate ptypes, copy input in, do the crypto with a key
    it derived from the HUK, write the result out, return
  |
  v
OP-TEE OS: unmap the buffers, set the return code, SMC back
  |
  v
TF-A BL31: save secure context, SCR_EL3.NS = 1, restore NS,
           ERET to Linux
  |
  v
optee driver: copy results back into the ioctl arg, return
  |
  v
libteec: fill in op.params, set return origin, return
  |
  v
app: TEEC_SUCCESS

  Two privilege boundaries crossed in each direction
  (S-EL0/S-EL1 and the world switch), plus a possible RPC
  excursion in the middle.
```

Where a TA runs

Type	Runs at	Loaded from	Use for
User-mode TA	S-EL0	Signed `.ta` file, fetched by RPC	Almost everything — isolated, crashable, updatable
Early TA	S-EL0	Linked into the OP-TEE binary	Needed before the filesystem exists

Type	Runs at	Loaded from	Use for
Pseudo TA	S-EL1, inside the kernel	Compiled in	Only when it truly needs kernel privilege
Secure partition	S-EL1 under S-EL2	FF-A manifest	Mutually distrusting secure components

> **A PSEUDO TA HAS NO MEMORY PROTECTION AT ALL**
>
> It runs in the trusted OS's own address space, at S-EL1, with full access to secure memory and every system register. A buffer overflow in a pseudo TA is not a crash — it is total compromise of the secure world. Write a user-mode TA unless you can state exactly which S-EL1-only operation forces your hand.

How OP-TEE isolates TAs

```
/* Each user-mode TA gets:

     its own S-EL0 address space, with its own translation
       tables and its own ASID
     its own heap and stack, sized by its manifest
     no access to another TA's memory
     no access to OP-TEE OS memory
     no MMIO mappings unless explicitly granted
     no way to execute privileged instructions

   So the isolation model inside the secure world mirrors
   user/kernel in Linux (OS chapters 4 and 12) - and for the
   same reason. A trusted application is trusted RELATIVE TO
   THE NORMAL WORLD, not absolutely.                          */

/* A TA declares its needs in a header: */

#define TA_UUID  { 0x8aaaf200, 0x2450, 0x11e4, \
                 { 0xab, 0xe2, 0x00, 0x02, 0xa5, 0xd5, 0xc5, 0x1b } }

#define TA_FLAGS                (TA_FLAG_EXEC_DDR | \
                                 TA_FLAG_SINGLE_INSTANCE | \
                                 TA_FLAG_MULTI_SESSION)
#define TA_STACK_SIZE           (2 * 1024)
#define TA_DATA_SIZE            (32 * 1024)

/* SINGLE_INSTANCE  one copy shared by all clients
   MULTI_SESSION    several concurrent sessions allowed
   INSTANCE_KEEP_ALIVE   stays resident after the last session
   SECURE_DATA_PATH      may touch secure buffers (video, DRM)

   Ask for the SMALLEST stack and heap that work. The secure
   world's memory is a scarce, permanently carved-out resource
   (chapter 10), not Linux's elastic pool.                    */
```

The thread model

```
/* OP-TEE has a FIXED number of threads - CFG_NUM_THREADS,
   typically 2 to 8, often one per CPU.

   Each concurrent secure call occupies one. When they are all
   busy the SMC returns OPTEE_SMC_RETURN_ETHREAD_LIMIT and the
   Linux driver puts the caller to sleep until one frees.

   CONSEQUENCES:
     - the secure world does not scale to hundreds of concurrent
       clients; it is not a server
     - a TA that blocks holds a thread the whole time
     - an RPC excursion also holds its thread - the normal world
       is servicing a request from a thread that is still
       "inside" the secure world
     - deadlock is possible: all threads waiting on an RPC that
       needs a thread to complete. This is why tee-supplicant
       must never itself call into the TEE.                  */

/* Each thread has a fixed stack, typically 8 KB. Deep recursion
   or a large local array in a pseudo TA overflows it, and the
   result is a secure-world crash with a stack dump on the
   console - which on production silicon you cannot see
   (chapter 10).                                             */
```

Hands-on: OP-TEE on QEMU

```
# The complete stack - TF-A, OP-TEE, Linux, buildroot, the test
# suite - built from one manifest. Budget an hour and ~20 GB.

mkdir -p ~/optee && cd ~/optee
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
repo sync -j8
cd build
make toolchains -j4
make run -j$(nproc)

# Three windows open: QEMU's own console, a "Normal World" UART
# and a "Secure World" UART. In the QEMU console:
(qemu) c

# Log in as root in the normal world, then:
xtest                     # the full GlobalPlatform test suite
xtest -l 0                # quick pass
optee_example_hello_world # the smallest complete example

# WATCH THE SECURE CONSOLE while these run. Every TA load,
# every panic and every trace line appears there. That split -
# normal-world output in one window, secure-world in another -
# is exactly the boundary this track is about.
```

> **READ `OPTEE_EXAMPLE_HELLO_WORLD` BEFORE WRITING ANYTHING**
>
> It is about 120 lines across the two sides: `host/main.c` does `InitializeContext / OpenSession / InvokeCommand`, and `ta/hello_world_ta.c` implements the five entry points. Every TA you ever write has that skeleton. Copy the `optee_examples` directory structure rather than inventing a build.

The other implementations

TEE	Where	Notes
OP-TEE	Open source, Linaro	GlobalPlatform-compliant; the one to learn on
Trusty	Android / Google	Own API, not GP; C++ and Rust TAs; used for Keymint and Gatekeeper
QSEE / QTEE	Qualcomm	Proprietary; the biggest deployed base
Kinibi	Trustonic	Proprietary; long history on Samsung Exynos
TEEGRIS	Samsung	Replaced Kinibi on newer Exynos parts
Hafnium	Open source	Not a TEE — an S-EL2 partition manager that hosts them

They differ in API and in TA packaging; the *architecture* — normal-world driver, monitor, trusted OS, isolated TAs, RPC to a supplicant — is the same in every one. Learn it once here.

Where this goes next

This chapter is the map; the OP-TEE track is the territory. It takes each of the six pieces above and gives it a chapter — the boot handover from TF-A, the message protocol carried over SMC, RPC and tee-supplicant, the Linux TEE subsystem, and both GlobalPlatform APIs in depth.

It ends in three projects: a secure key store with its own client, and a firmware TPM built as a trusted application that Linux then drives as `/dev/tpm0`. If you want to actually write a TA rather than read about one, start there.

How it shows up

Symptom	Cause
`/dev/tee0` missing	OP-TEE not running, or the driver did not probe — check `dmesg | grep -i optee`
`TEEC_ERROR_ITEM_NOT_FOUND` on open session	TA not found; `tee-supplicant` not running, or the `.ta` is not in `/lib/optee_armtz`
`TEEC_ERROR_SECURITY` on load	TA signature check failed — signed with the wrong key
`TEEC_ERROR_BAD_PARAMETERS`	`paramTypes` disagree between client and TA
`TEEC_ERROR_OUT_OF_MEMORY`	Secure heap exhausted, or `TA_DATA_SIZE` too small
TA panics with a stack dump on the secure console	Bad pointer, failed assertion, or stack overflow inside the TA
Calls block for a long time under load	All OP-TEE threads busy — `ETHREAD_LIMIT`
Everything hangs after an RPC	Thread-exhaustion deadlock, or `tee-supplicant` died
Works on QEMU, fails on hardware	Secure memory carve-out too small, or the TZASC region does not match
Output buffer contains a partial result on error	The TA wrote into shared memory before it had finished — write once, at the end

Check yourself

- ✓ Name all six pieces of the stack and say what each contributes.
- ✓ Distinguish the GlobalPlatform Client API from the Internal API.
- ✓ List the five TA entry points and when each is called.

- ✓ Explain the parameter model and why checking `paramTypes` is a security control.
- ✓ Give four rules for handling a client-supplied buffer inside a TA.
- ✓ Trace one `TEEC_InvokeCommand` through every layer and back.
- ✓ Explain the difference between a user-mode TA and a pseudo TA, and why you should default to the former.
- ✓ Explain OP-TEE's fixed thread pool and the deadlock it can produce.
- ✓ Get `xtest` passing on QEMU and find the secure-world console.

Trusted applications and secure storage

Now write one. A complete trusted application — client side, TA side, build, sign, install — plus the mechanism that lets it keep a secret across reboots on a device whose filesystem the attacker controls.

What belongs in a TA

```
A good TA is SMALL, does ONE thing, and its interface is narrow.

  Good candidates:
    · key generation, storage and use - the key never leaves
    · device attestation and identity
    · payment authorisation
    · biometric template matching (template never leaves)
    · DRM licence handling and content key management
    · secure firmware update verification
    · a monotonic counter, or a rollback-resistant state blob

  Bad candidates:
    · anything large or complex - it is attack surface at the
      highest privilege in the system
    · anything performance-critical per-operation, because of
      the world-switch cost (chapter 11)
    · anything needing drivers, filesystems or a network stack
    · "make this hard to reverse engineer" - obscurity is not
      what the boundary provides
    · business logic that merely wants to be tamper-resistant

  THE TEST: would you be comfortable if this code were audited
  line by line as security-critical? If not, it does not belong
  in the secure world.
```

Identity: the UUID

```
/* A TA is named by a 128-bit UUID, not a filename. It appears
   in three places and they must agree.                       */

/* 1. include/my_ta.h - shared by both sides */
#define TA_MY_UUID \
        { 0x12345678, 0x5b69, 0x11d4, \
          { 0x9f, 0xee, 0x00, 0xc0, 0x4f, 0x4c, 0x32, 0x01 } }

/* 2. ta/user_ta_header_defines.h */
#define TA_UUID  TA_MY_UUID

/* 3. the installed filename, in canonical text form:
      /lib/optee_armtz/12345678-5b69-11d4-9fee-00c04f4c3201.ta

   tee-supplicant builds that path from the UUID the secure
   world asks for. Get any character wrong and you get
   TEEC_ERROR_ITEM_NOT_FOUND with no further explanation.   */

/* Generate a real one - never copy an example's: */
        uuidgen

/* The UUID is also the TA's IDENTITY for secure storage. Change
   it and the TA loses access to everything it previously
   stored, permanently. Choose it once.                       */
```

The TA

```
/* ta/my_ta.c - a key store: generate a key inside the secure
   world, sign with it, and never let it out.              */

#include <tee_internal_api.h>
#include <tee_internal_api_extensions.h>
#include <string.h>
#include "my_ta.h"

#define KEY_OBJ_ID      "device-signing-key-v1"
#define KEY_BITS        256

/* ---- lifecycle ---------------------------------------- */

TEE_Result TA_CreateEntryPoint(void)
{
        DMSG("my_ta loaded");
        return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void) { }

TEE_Result TA_OpenSessionEntryPoint(uint32_t ptypes,
                                    TEE_Param params[4],
                                    void **sess_ctx)
{
        struct session *s;

        (void)params;
        if (ptypes != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE))
                return TEE_ERROR_BAD_PARAMETERS;

        s = TEE_Malloc(sizeof(*s), TEE_MALLOC_FILL_ZERO);
        if (!s)
                return TEE_ERROR_OUT_OF_MEMORY;

        *sess_ctx = s;                  /* per-client state */
        return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess_ctx)
{
        TEE_Free(sess_ctx);             /* no leaks - the secure
                                           heap is tiny        */
}

/* ---- generate a key that never leaves ----------------- */

static TEE_Result cmd_generate_key(void)
{
        TEE_ObjectHandle key = TEE_HANDLE_NULL;
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        TEE_Result res;

        res = TEE_AllocateTransientObject(TEE_TYPE_ECDSA_KEYPAIR,
                                          KEY_BITS, &key);
        if (res != TEE_SUCCESS)
                return res;
```

```
        res = TEE_GenerateKey(key, KEY_BITS, NULL, 0);
        if (res != TEE_SUCCESS)
                goto out;

        /* persist it - encrypted with a key derived from the
           HUK, so only this TA on this device can read it   */
        res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
                        KEY_OBJ_ID, sizeof(KEY_OBJ_ID) - 1,
                        TEE_DATA_FLAG_ACCESS_WRITE |
                        TEE_DATA_FLAG_OVERWRITE,
                        key,           /* the attributes go in */
                        NULL, 0,       /* no extra data stream */
                        &obj);
        if (res == TEE_SUCCESS)
                TEE_CloseObject(obj);
out:
        TEE_FreeTransientObject(key);
        return res;
}

/* ---- sign, with a client-supplied digest --------------- */

static TEE_Result cmd_sign(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp = TEE_PARAM_TYPES(
                        TEE_PARAM_TYPE_MEMREF_INPUT,   /* digest */
                        TEE_PARAM_TYPE_MEMREF_OUTPUT,  /* sig    */
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE);
        uint8_t digest[32];
        uint8_t sig[128];
        uint32_t siglen = sizeof(sig);
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        TEE_OperationHandle op = TEE_HANDLE_NULL;
        TEE_Result res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        /* the digest must be exactly SHA-256 sized */
        if (params[0].memref.size != sizeof(digest))
                return TEE_ERROR_BAD_PARAMETERS;

        /* COPY IN before doing anything with it (chapter 12) */
        TEE_MemMove(digest, params[0].memref.buffer, sizeof(digest));

        /* is the output buffer big enough? Tell the caller how
           much it needs rather than truncating.             */
        if (params[1].memref.size < sizeof(sig)) {
                params[1].memref.size = sizeof(sig);
                return TEE_ERROR_SHORT_BUFFER;
        }

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                        KEY_OBJ_ID, sizeof(KEY_OBJ_ID) - 1,
                        TEE_DATA_FLAG_ACCESS_READ, &obj);
        if (res != TEE_SUCCESS)
                return res;            /* not provisioned yet */

        res = TEE_AllocateOperation(&op, TEE_ALG_ECDSA_P256,
                                    TEE_MODE_SIGN, KEY_BITS);
        if (res != TEE_SUCCESS)
```

```
                goto out_obj;

        res = TEE_SetOperationKey(op, obj);
        if (res != TEE_SUCCESS)
                goto out_op;

        res = TEE_AsymmetricSignDigest(op, NULL, 0,
                                       digest, sizeof(digest),
                                       sig, &siglen);
        if (res != TEE_SUCCESS)
                goto out_op;

        /* write the result ONCE, at the end, on success only */
        TEE_MemMove(params[1].memref.buffer, sig, siglen);
        params[1].memref.size = siglen;

out_op:
        TEE_FreeOperation(op);
out_obj:
        TEE_CloseObject(obj);
        return res;
}

/* ---- dispatch ---------------------------------------- */

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx, uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        (void)sess_ctx;

        switch (cmd) {
        case TA_CMD_GENERATE_KEY:
                return cmd_generate_key();
        case TA_CMD_SIGN:
                return cmd_sign(ptypes, params);
        default:
                return TEE_ERROR_NOT_SUPPORTED;
        }
}
```

READ THAT DISPATCH FUNCTION AS AN ATTACKER WOULD

The command number, the parameter types, every buffer size, and the contents of every input buffer are all chosen by an untrusted caller. The `ptypes` check, the exact-size check on the digest, the copy-in, the short-buffer path, and writing the output only on success are each there to close a specific hole. Take any one out and there is a real bug.

The client

```
/* host/main.c */

#include <tee_client_api.h>
#include <stdio.h>
#include <string.h>
#include "my_ta.h"

int main(void)
{
        TEEC_Context ctx;
        TEEC_Session sess;
        TEEC_Operation op;
        TEEC_UUID uuid = TA_MY_UUID;
        uint8_t digest[32] = { 0 };
        uint8_t sig[128];
        uint32_t origin;
        TEEC_Result res;

        res = TEEC_InitializeContext(NULL, &ctx);
        if (res != TEEC_SUCCESS)
                errx(1, "InitializeContext: 0x%x", res);

        res = TEEC_OpenSession(&ctx, &sess, &uuid,
                               TEEC_LOGIN_PUBLIC, NULL, NULL,
                               &origin);
        if (res != TEEC_SUCCESS)
                errx(1, "OpenSession: 0x%x origin 0x%x", res, origin);

        /* one-time provisioning */
        memset(&op, 0, sizeof(op));
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_NONE, TEEC_NONE,
                                         TEEC_NONE, TEEC_NONE);
        res = TEEC_InvokeCommand(&sess, TA_CMD_GENERATE_KEY,
                                 &op, &origin);
        if (res != TEEC_SUCCESS)
                errx(1, "GENERATE_KEY: 0x%x", res);

        /* sign something */
        memset(&op, 0, sizeof(op));
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_OUTPUT,
                                         TEEC_NONE, TEEC_NONE);
        op.params[0].tmpref.buffer = digest;
        op.params[0].tmpref.size   = sizeof(digest);
        op.params[1].tmpref.buffer = sig;
        op.params[1].tmpref.size   = sizeof(sig);

        res = TEEC_InvokeCommand(&sess, TA_CMD_SIGN, &op, &origin);
        if (res != TEEC_SUCCESS)
                errx(1, "SIGN: 0x%x origin 0x%x", res, origin);

        printf("signature: %u bytes\n", op.params[1].tmpref.size);

        TEEC_CloseSession(&sess);
        TEEC_FinalizeContext(&ctx);
        return 0;
}
```

`ORIGIN` TELLS YOU WHO FAILED

`TEEC_ORIGIN_API` (libteec), `_COMMS` (the driver or the transport), `_TEE` (the trusted OS) or `_TRUSTED_APP` (your TA's return value). Print it on every failure — the difference between "the TEE could not load your TA" and "your TA returned an error" saves hours.

Building and signing

```
# Layout, matching optee_examples:

my_ta/
  host/     Makefile   main.c
  ta/       Makefile   my_ta.c   user_ta_header_defines.h
            sub.mk     Android.mk
  include/  my_ta.h

# ta/Makefile is three lines - OP-TEE supplies the rest:
BINARY = 12345678-5b69-11d4-9fee-00c04f4c3201
include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

# ta/sub.mk:
global-incdirs-y += ../include
srcs-y += my_ta.c

# Build:
make -C ta CROSS_COMPILE=aarch64-linux-gnu- \
        TA_DEV_KIT_DIR=.../export-ta_arm64
make -C host CROSS_COMPILE=aarch64-linux-gnu- \
        TEEC_EXPORT=.../out/export

# The build SIGNS the TA automatically with the key in
#   optee_os/keys/default_ta.pem
# and OP-TEE checks that signature before loading (chapter 11:
# never trust what RPC hands you).

# Install:
scp ta/*.ta   root@target:/lib/optee_armtz/
scp host/my_ta root@target:/usr/bin/
```

THE DEFAULT SIGNING KEY IS PUBLIC

`default_ta.pem` ships in the OP-TEE source tree. A product built with it will load a TA written by anyone. Generate a per-product key, keep the private half in an HSM, and build with `TA_SIGN_KEY=`. Better still, on parts that support it, use OP-TEE's asymmetric-key provisioning so the public half is itself anchored in the fuses (chapter 14).

Secure storage: the problem

```
The TA has no filesystem. The bytes must end up somewhere the
NORMAL world controls - eMMC, UFS, a file in /data. So an
attacker with root can read, modify, delete, or restore an old
copy of them.

  Four properties are needed:

  CONFIDENTIALITY  attacker cannot read the contents
     -> encrypt with a key derived from the HUK

  INTEGRITY        attacker cannot modify them undetected
     -> authenticated encryption (AES-GCM), a MAC over
        everything

  BINDING          the blob cannot be moved to another device
     -> the HUK is per-device, so a blob from device A is
        undecryptable on device B

  FRESHNESS        attacker cannot restore an OLD copy
     -> this is the hard one. Encryption and MACs do not help:
        the old blob is authentic, just stale.
        Needs a MONOTONIC COUNTER in hardware, or RPMB.
```

The two backends

	REE filesystem	**RPMB**
Stored in	`/data/tee/`, via `tee-supplicant`	The eMMC/UFS Replay Protected Memory Block partition
Confidentiality	Yes — AES-GCM, HUK-derived key	Yes
Integrity	Yes — hash tree	Yes — HMAC on every access
Device binding	Yes	Yes
Rollback protection	**No** — the whole tree can be restored	**Yes** — a write counter inside the eMMC, monotonic, unresettable
Deletion resistance	No — root can `rm -rf` it	No — but the counter proves it happened
Size	Large	Small: typically 128 KB to 4 MB
Speed	Faster	Slower — one HMAC round trip per block

```
/* RPMB works because the eMMC itself enforces the rules: */

   Provisioning (ONCE, irreversibly):
        the secure world writes an RPMB AUTHENTICATION KEY
        into the eMMC. It can never be read back and never
        changed.

   Every write:
        host sends  { data, write_counter, nonce, HMAC }
        eMMC checks the HMAC with its stored key
        eMMC checks the write_counter matches its own
        on success it writes AND increments the counter

   Every read:
        eMMC returns { data, nonce, HMAC }
        the secure world checks the HMAC and the nonce

   So a replayed write is rejected because its counter is stale,
   and a forged read is rejected because the HMAC will not
   verify. The trusted OS holds the key; Linux merely carries
   the frames (chapter 11's RPC, again - the normal world is
   the transport, never the authority).                    */

/* Select it per object: */
        TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE_RPMB, ...)

/* Or build OP-TEE with CFG_RPMB_FS=y and make it the default.
   Use RPMB for the SMALL things where freshness matters -
   counters, key versions, lock state - and the REE filesystem
   for bulk.                                               */
```

Key derivation

```
/* Never use the HUK directly. Derive per-purpose keys from it,
   binding in the TA's identity so one TA cannot read another's
   storage.                                                 */

static TEE_Result derive_key(const char *label,
                             uint8_t *out, size_t outlen)
{
        /* extra_data mixes in the caller's identity; OP-TEE adds
           the TA UUID internally for TEE_STORAGE_PRIVATE      */
        return tee_otp_get_hw_unique_key_derived(label,
                                                 strlen(label),
                                                 out, outlen);
}

/* PROPERTIES you want:
     - per-device      from the HUK
     - per-TA          UUID mixed in (OP-TEE does this)
     - per-purpose     a distinct label per use - storage,
                       attestation, transport - so compromise
                       of one does not give the others
     - versioned       so a key can be rotated after a fix

   Cryptography chapters 12 and 13 cover KDFs properly; the
   embedded-specific point is only that the ROOT is a hardware
   secret, not a constant in the binary.                   */
```

TA security checklist

BEFORE YOU SHIP A TA

- `paramTypes` checked in every command handler.
- Every `memref.size` validated before use; overflow considered on every size arithmetic.
- Inputs copied into TA memory before validation and use — never validated in place.
- Outputs written once, on success, never used as scratch.
- Secrets zeroed with `TEE_MemFill` before `TEE_Free`.
- Every handle closed and every operation freed on *every* path, including errors.
- Session state freed in `TA_CloseSessionEntryPoint`.
- Error returns carry no information about secret data — no distinct code for "wrong PIN, byte 3".
- Comparisons on secrets are constant-time.
- No secret-dependent branch or memory-access pattern (chapter 16).
- Anything with a retry limit is counted in RPMB, not in RAM or REE storage.
- Stack usage measured, not guessed.
- Signed with a product key, not the default.

Where this goes next

Chapter 12 of the OP-TEE track covers the Internal Core API in full, chapter 15 covers REE FS and RPMB storage properly, and chapter 20 is the one that matters most: writing a TA that survives a client actively trying to break it. The key-store project applies all three.

How it shows up

Symptom	Cause
`ITEM_NOT_FOUND` on open session	UUID mismatch between header and installed filename
`TEEC_ERROR_SECURITY` at load	Signed with a key OP-TEE does not trust
`ERROR_SHORT_BUFFER`	Expected — call again with the size the TA returned
`ERROR_ITEM_NOT_FOUND` from `TEE_OpenPersistentObject`	Not provisioned, or storage wiped, or the UUID changed
Storage works, then fails after a reflash	REE storage wiped with `/data`; the key survived, the blob did not
Old state reappears after a downgrade attack	REE storage has no rollback protection — use RPMB
`ERROR_OUT_OF_MEMORY` after many sessions	Session context leaked in `TA_CloseSessionEntryPoint`
TA panics on the third concurrent client	Not `MULTI_SESSION`-safe, or shared state without locking
RPMB writes fail with a counter error	Two writers, or the key was never provisioned
Works on the dev board, storage empty on production	Different HUK — blobs are device-bound by design

Check yourself

- ✓ Decide whether a given feature belongs in a TA, and justify it.
- ✓ Explain where a TA's UUID appears and why changing it destroys its storage.
- ✓ Write a command handler with correct `paramTypes`, size and copy-in handling.
- ✓ Explain the `SHORT_BUFFER` convention.
- ✓ Use `origin` to localise a failure.
- ✓ Explain why shipping with the default signing key is a vulnerability.
- ✓ Name the four properties secure storage needs, and which one encryption alone cannot give.
- ✓ Explain how RPMB achieves rollback protection and why Linux cannot forge a write.
- ✓ Explain why keys are derived per-TA and per-purpose rather than used directly.
- ✓ Walk the TA security checklist against your own code.

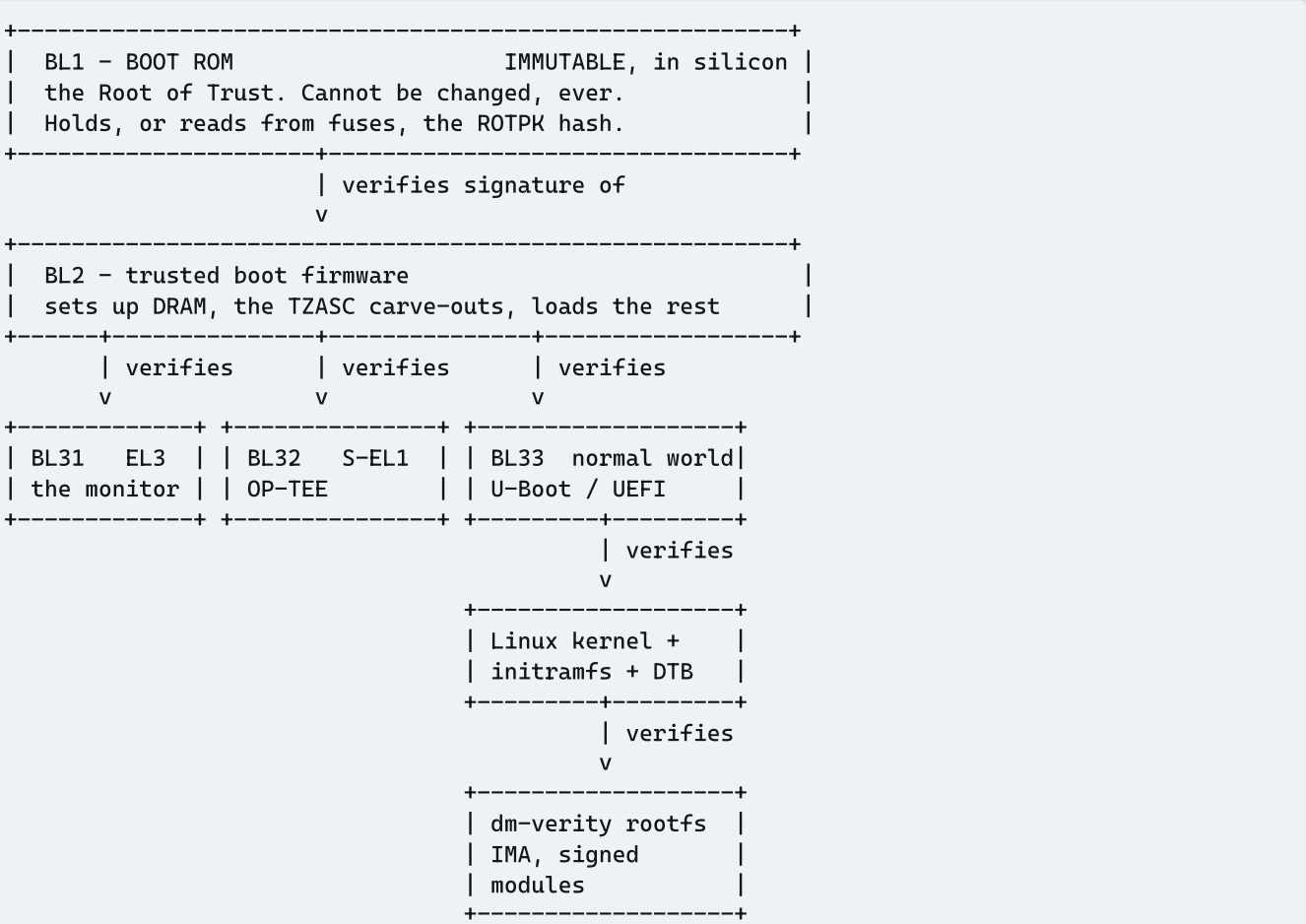
Secure boot and the chain of trust

Every protection in this track assumes the right software is running. Secure boot is what makes that assumption true — a chain from immutable silicon to the kernel, where each link verifies the next before giving it control.

Why it comes first

```
TrustZone, the SMMU, the TZASC, the XPU - every one of them is
CONFIGURED BY SOFTWARE at boot.

  So an attacker who controls the first instruction controls
  everything after it. They can:
        set SCR_EL3.NS = 0 and run their own code as "secure"
        leave the TZASC default region wide open
        skip the SMMU setup entirely
        never lock the configuration registers
        assert the debug signals
        load a trusted OS of their own

  and every mechanism in chapters 01-13 evaporates.

  The chain of trust exists to make "the first instruction is
  the one the vendor intended" a property of the SILICON rather
  than a hope.
```

The chain



> **THE CHAIN IS ONLY AS STRONG AS ITS WEAKEST LINK — AND ITS ANCHOR**
>
> A chain that verifies BL2, BL31 and BL32 perfectly but loads BL33 unverified is not a chain of trust; it is a chain of trust that ends early, and everything the normal world does afterwards is unconstrained. The value comes from it being *unbroken*, from an anchor that cannot be replaced.

The root of trust

```
/* Two things must be immutable, or the whole structure is
   circular:

   1. THE FIRST CODE - the boot ROM. Mask ROM, fixed at
      fabrication. It cannot be patched, which means a bug in
      it is unfixable in the field - which is why boot ROMs are
      tiny and are formally reviewed.

   2. THE ROOT KEY - or rather its hash. Full public keys are
      large; fuses are scarce. So the fuses hold

           ROTPK_HASH = SHA-256(root public key)

      typically 256 bits. At boot:
           read the root public key from the image header
           hash it
           compare with the fuses
           if they differ, STOP
           otherwise use it to verify the next stage's signature

      Storing only the hash means the key itself can be large
      (RSA-4096, or an ECDSA P-384 point) while costing 32 bytes
      of OTP.                                                  */

/* WHY FUSES and not flash: fuses are one-time-programmable.
   Blowing them is physical and irreversible. Flash can be
   rewritten by anything that can write flash - which is
   exactly the attacker you are defending against.          */

/* LIFECYCLE: parts ship BLANK (no ROTPK, secure boot off) so
   they can be developed on, and are moved to SECURED during
   manufacturing by blowing the ROTPK and the enable fuse. That
   transition is one-way. Get the key wrong and the part is
   scrap - which is why key provisioning is done by a small
   number of very careful people.                           */
```

What "verify" means

```
/* Signature verification, at each link: */

   1. parse the image header - WITHOUT trusting its contents
   2. extract the certificate / public key
   3. hash that key, compare against the trusted anchor
        (the ROTPK for BL2; a key delegated by it thereafter)
   4. hash the image payload
   5. verify signature(hash, public key)
   6. check the ROLLBACK COUNTER in the header against the
      fused monotonic counter
   7. only now: map it executable and branch to it

/* STEP 1 IS WHERE BUGS LIVE. The header is attacker-controlled
   data being parsed by the most privileged code in the system,
   before any verification has happened. Every field is
   hostile:

        length fields          -> integer overflow, then a
                                  buffer overflow in the ROM
        offsets                -> read or write out of bounds
        counts                 -> unbounded loops
        algorithm identifiers  -> "algorithm: none"
        key sizes              -> a 1-bit RSA key

   Real boot ROM exploits - across several vendors, on hundreds
   of millions of devices - have almost all been header parsing
   bugs, not cryptographic breaks.                          */

/* And the classic logic bug:

        if (verify_signature(img) == 0)      /* WRONG WAY UP  */
                boot(img);                   /* 0 = failure?  */

   or a verify function that returns success on an error path,
   or one whose result is never checked at all. Fail closed
   (chapter 01): the ONLY path to boot() should be through a
   positive confirmation.                                   */
```

Rollback protection

```
Version 1.0 has a remote-code-execution bug. You ship 1.1.
The attacker installs 1.0 again. It is CORRECTLY SIGNED - your
signature check happily accepts it.

  Signatures prove authenticity, never freshness. You need a
  MONOTONIC COUNTER.

     fuses hold:   nv_counter = 3
     image header: nv_version = 3     -> accept
     image header: nv_version = 2     -> REJECT, too old
     image header: nv_version = 4     -> accept, and BURN the
                                         counter to 4

  Counters are usually implemented as a fuse bank blown one bit
  at a time - 32 fuses give 32 increments. They are FINITE, so:

     · do not increment for every release. Increment only when
       a fixed vulnerability makes the old version dangerous.
     · burn the counter only after the new image has proven it
       BOOTS. Burn first and a failed update bricks the device.
     · a device that skipped several versions must still accept
       the newest; compare >= , never == .

  Same problem, same shape as secure storage freshness
  (chapter 13). Cryptography cannot solve it; monotonic
  hardware state can.
```

Trusted Board Boot in TF-A

```
# Arm's reference implementation. Build with verification on:

make PLAT=qemu \
     TRUSTED_BOARD_BOOT=1 \
     GENERATE_COT=1 \
     ARM_ROTPK_LOCATION=devel_rsa \
     ROT_KEY=plat/arm/board/common/rotpk/arm_rotprivk_rsa.pem \
     BL33=u-boot.bin \
     all fip

# TRUSTED_BOARD_BOOT=1   turn on verification
# GENERATE_COT=1         build the certificate chain
# ARM_ROTPK_LOCATION     where the ROTPK hash comes from
# ROT_KEY                the private root key (an HSM in
#                        production - never a file in the tree)

# The output FIP (Firmware Image Package) contains the images
# AND the certificates:

   fiptool info fip.bin

   Trusted Boot Firmware BL2: offset=0x...
   Trusted key certificate: offset=0x...
   SoC Firmware key certificate: offset=0x...
   SoC Firmware BL31: offset=0x...
   Trusted OS key certificate: offset=0x...
   Trusted OS BL32: offset=0x...
   Non-Trusted Firmware key certificate: offset=0x...
   Non-Trusted Firmware BL33: offset=0x...

# The CHAIN OF TRUST is expressed as X.509 certificates: the
# root key signs a "trusted key certificate" containing two
# delegated keys, those sign per-image key certificates, and
# those sign content certificates carrying the image HASHES.

# WHY THE INDIRECTION: the root key can stay in a vault and be
# used once. Day-to-day signing uses the delegated keys, which
# can be rotated without touching fuses.
```

Handing off to Linux

```
# U-Boot verifies the kernel using FIT images and a public key
# compiled into its own device tree - which was itself verified
# by BL2, which was verified by the ROM.

# 1. describe what to sign:
/dts-v1/;
/ {
    images {
        kernel {
            data = /incbin/("Image.gz");
            type = "kernel";  arch = "arm64";  os = "linux";
            compression = "gzip";
            load = <0x40080000>;  entry = <0x40080000>;
            hash { algo = "sha256"; };
        };
        fdt {
            data = /incbin/("board.dtb");
            type = "flat_dt";  arch = "arm64";
            compression = "none";
            hash { algo = "sha256"; };
        };
    };
    configurations {
        default = "conf";
        conf {
            kernel = "kernel";  fdt = "fdt";
            signature {
                algo = "sha256,rsa2048";
                key-name-hint = "prod";
                sign-images = "kernel", "fdt";
            };
        };
    };
};

# 2. sign, embedding the public key into U-Boot's control DTB:
mkimage -f kernel.its -k keys/ -K u-boot.dtb -r kernel.itb

# 3. U-Boot must be built with CONFIG_FIT_SIGNATURE=y and
#    CONFIG_LEGACY_IMAGE_FORMAT=n - otherwise it will happily
#    boot an unsigned legacy image and the chain ends here.

# NOTE sign-images includes the FDT. An attacker who can change
# the device tree can add "init=/bin/sh" via bootargs, remap
# memory, or disable the SMMU node. The DTB is as security
# critical as the kernel.
```

> **THE CHAIN USUALLY BREAKS AT THE LAST LINK**
> Verified firmware, verified U-Boot, verified kernel — and then an unverified root filesystem, an unsigned kernel module, an attacker-editable `bootargs`, or a debug U-Boot console that accepts `tftpboot`. All of the effort upstream buys nothing if the kernel then loads whatever is on `/dev/mmcblk0p5`.

Extending it into Linux

Mechanism	Protects	How
dm-verity	A read-only rootfs	Merkle tree over the block device; the root hash is on the verified kernel command line. Every block is checked *when read*, so tampering is caught even after boot.
fs-verity	Individual files on a writable fs	Per-file Merkle tree; good for APKs and large read-only assets.
IMA / EVM	Arbitrary files	Measures or appraises on open, against signatures in xattrs.
Module signing	Kernel modules	`CONFIG_MODULE_SIG_FORCE=y`; otherwise a module *is* arbitrary kernel code.
Kernel lockdown	The running kernel	Blocks `/dev/mem`, kexec of unsigned images, some BPF, raw MSR/port access.
Encrypted or authenticated `/data`	Mutable state	dm-crypt with a key from the TEE, so it is device-bound (ch. 13).

```
# dm-verity, put together:

veritysetup format rootfs.img hash.img
  # prints: Root hash: a1b2c3...

# the root hash goes on the KERNEL COMMAND LINE, which is
# signed as part of the FIT configuration - so the hash itself
# is anchored to the ROM:

  dm-mod.create="vroot,,,ro,0 <sectors> verity 1 \
        /dev/mmcblk0p3 /dev/mmcblk0p4 4096 4096 \
        <datablocks> 1 sha256 <roothash> <salt>" \
  root=/dev/dm-0 ro

# Now every block read from the rootfs is verified against the
# tree, whose root is verified by the kernel, which was
# verified by U-Boot, which was verified by BL2, which was
# verified by the ROM against a hash in a fuse.

# THE CHAIN IS COMPLETE. Any single byte changed anywhere on
# that path is detected.
```

Measured boot: the other kind

```
VERIFIED boot   refuses to run software that is not signed.
                Enforcement is local and immediate.

MEASURED boot   runs anything, but RECORDS what it ran into a
                tamper-evident log, so a remote party can decide
                whether to trust the result.

  Each stage extends a hash into a PCR before executing:

        PCR[n] = SHA256( PCR[n] || SHA256(next_stage) )

  Extend-only, so the sequence cannot be rewritten - only
  appended to, and any different software produces a different
  final value.

  Then ATTESTATION: a remote server sends a nonce, the device
  returns the PCR values plus the log, signed by a key that
  only genuine hardware holds. The server decides whether that
  configuration may access the service.

  Stored in a TPM (PCRs 0-23) on PC-class hardware, or in the
  TEE on Arm - which is one of the best reasons to have a TEE
  at all (chapter 13's attestation key, used here).

  Most real products want BOTH: verified boot to stop local
  tampering, measured boot so the back end can tell what it is
  talking to.
```

Living with it during development

KEEP A DEVELOPMENT PATH, AND MAKE IT OBVIOUSLY DIFFERENT

- Use **development keys** on dev boards, production keys only in the factory. Never let a production private key exist outside an HSM.
- Support **unlocking** — a signed token, tied to the device ID, that permits unsigned images. Wipe the device keys when it is used, so unlocking cannot expose user data.
- Make the state **visible**: a boot message, a screen, an API. Android's yellow/orange boot warnings exist for this.
- Never ship a build that *skips* verification on a flag — that flag will be found. Ship a build whose verification uses a different key.
- Test the **negative cases** in CI: flip one bit in each image and assert that boot stops at the right stage. An unexercised check is an unknown check.

How it shows up

Symptom	Cause
Device is silent, no UART output at all	ROM rejected BL2 — signature, ROTPK mismatch, or a corrupt header
`Failed to verify` then reset loop	Wrong signing key, or the image changed after signing
Boots after signing but not after a rebuild	Signing not wired into the build — the FIP contains the old certificate

Symptom	Cause
Boots on one board, not another	Different ROTPK fused; boards from different production runs
Older, valid firmware refuses to boot	Rollback counter already burned — working as designed
Update bricks the device	Counter burned before the new image was known to boot
Firmware verified, but any rootfs boots	Chain ends at the kernel — no dm-verity
Verified boot bypassed via the command line	`bootargs` or the DTB not covered by the signature
Attacker loads a module	`MODULE_SIG_FORCE` not set
Signature check passes on a corrupted image	Return value inverted or unchecked — audit that code path by hand
Part accepts unsigned images in the field	Lifecycle state never advanced to SECURED in manufacturing

Check yourself

- ✓ Explain why secure boot is a precondition for every other mechanism in this track.
- ✓ Draw the chain from boot ROM to root filesystem, naming what verifies what.
- ✓ Explain why the fuses hold a hash of the root key rather than the key.
- ✓ List the steps of one verification and say which one attackers target most.
- ✓ Explain why signatures alone cannot stop a downgrade, and how counters fix it.
- ✓ State the two rules for burning a rollback counter safely.
- ✓ Describe TF-A's certificate chain and why the root key is delegated.
- ✓ Explain why the device tree must be covered by the signature.
- ✓ Set up dm-verity and say where its root hash is anchored.
- ✓ Contrast verified boot with measured boot, and say why products want both.

TrustZone-M for microcontrollers

The same idea on a chip with no MMU, no exception levels, and a 20-cycle interrupt latency budget. TrustZone for Armv8-M drops the monitor entirely and puts the boundary in the address map — which changes everything about how you use it.

What is different

	TrustZone-A	TrustZone-M
Security state determined by	`SCR_EL3.NS` — a register	**The address being executed**
Transition	`SMC` into EL3, which switches contexts	Direct call to a valid entry point — *no monitor*
Transition cost	~1000–4000 cycles	**A few cycles**
Exception levels	EL0–EL3	Thread/Handler mode, privileged/unprivileged, in each state
Address translation	MMU, two stages	MPU only — no translation, just permissions
Region control	TZASC, TZPC, SMMU	SAU and IDAU, plus separate secure/non-secure MPUs
Stacks	Per exception level	**Four** — MSP/PSP in each state
Vector table	One `VBAR` per EL	Two — `VTOR_S` and `VTOR_NS`
Typical software	Linux + OP-TEE	Bare metal or an RTOS on each side

THE CORE IDEA: SECURITY IS A PROPERTY OF THE ADDRESS

On Armv8-M there is no bit in a register that says "we are secure now". The processor is in Secure state *because it is executing from a Secure address*. Fetch an instruction from a Non-secure address and the state changes with it. This is why the transition is nearly free — and why the address map *is* the security policy.

The address map

```
Every address is one of three types:

  SECURE (S)
      accessible only from Secure state.
      code and data for the trusted side.

  NON-SECURE (NS)
      accessible from either state.
      the application side.

  NON-SECURE CALLABLE (NSC)
      SECURE memory, but Non-secure code may BRANCH into it -
      and only to an SG instruction. This is the doorway.

  Two units decide which is which:

    IDAU  Implementation Defined Attribution Unit.
          Fixed by the silicon vendor, coarse, cannot be
          changed. Often splits the whole map by an address bit.

    SAU   Security Attribution Unit.
          8 or so programmable regions, configured by SECURE
          software at boot.

  The result is the MORE SECURE of the two:

    IDAU says NS, SAU says S   -> Secure
    IDAU says S,  SAU says NS  -> Secure
    both say NS                -> Non-secure

  So the SAU can never make something less secure than the
  silicon says it is. Fail-closed, in hardware.

  Typical layout on a part with 1 MB flash / 256 KB RAM:

    0x0C00_0000  +-----------------+
                 | Secure flash    |  S    bootloader, secure lib
    0x0C03_0000  +-----------------+
                 | veneers         |  NSC  the SG entry points
    0x0C03_1000  +-----------------+
    0x0803_1000  | NS flash        |  NS   the application
    0x0810_0000  +-----------------+

    0x3000_0000  +-----------------+
                 | Secure RAM      |  S    keys, secure stacks
    0x3000_8000  +-----------------+
    0x2000_8000  | NS RAM          |  NS   application RAM
    0x2004_0000  +-----------------+

  Note the aliasing: 0x0C00_0000 and 0x0800_0000 are the SAME
  physical flash, seen through a Secure and a Non-secure alias.
  Bit 28 is the security alias bit on many parts. That is the
  IDAU doing its coarse split.
```

Crossing the boundary

```
/* NS -> S: the SG instruction */

   Non-secure code does:   BL   secure_function

   The target must be:
      · in an NSC region, and
      · the FIRST instruction there must be SG

   If either fails -> SecureFault. No exceptions, no
   negotiation. This is what stops Non-secure code jumping into
   the MIDDLE of a secure function to skip its checks.

/* S -> NS: the BXNS / BLXNS instructions */

   BXNS  r0       branch, switching to Non-secure
   BLXNS r0       call Non-secure, and come back

   Secure code must use these EXPLICITLY. A plain BX to a
   Non-secure address faults - so you cannot leave the secure
   state by accident.

/* The compiler writes the glue. Two attributes: */

/* An entry point, callable from Non-secure: */
__attribute__((cmse_nonsecure_entry))
int secure_add(int a, int b)
{
        return a + b;
}
/* generates:
        secure_add:
                SG                      ; the required gateway
                B   secure_add_impl     ; jump to the real body
   and on return, clears the caller-saved registers and issues
   BXNS lr.                                                  */

/* A pointer TO Non-secure code, called from Secure: */
typedef void __attribute__((cmse_nonsecure_call)) (*ns_fn)(void);

   ...  ((ns_fn)ns_reset_handler)();   /* compiles to BLXNS */

/* Build with -mcmse. The linker produces an IMPORT LIBRARY: */

   arm-none-eabi-gcc -mcmse ... \
        -Wl,--cmse-implib -Wl,--out-implib=secure_impl.o

/* secure_impl.o contains only the veneer addresses. Link it
   into the NON-SECURE project and its calls resolve to the
   right NSC addresses. Rebuild the secure side and you must
   re-link the non-secure side too - or ship a stable veneer
   table that never moves.                                   */
```

Validating arguments, without an MMU

```
/* Same problem as chapter 12, harder: there is no separate
   address space to copy from. A pointer from Non-secure code
   is just a number, and it might point at SECURE memory. If
   your secure function dereferences it, it reads or writes
   secure memory ON BEHALF OF the attacker - the classic
   confused deputy.                                          */

#include <arm_cmse.h>

__attribute__((cmse_nonsecure_entry))
int secure_hash(const uint8_t *buf, size_t len, uint8_t *out)
{
        const uint8_t *ok_in;
        uint8_t *ok_out;

        /* does the CALLER have read access to this range? */
        ok_in = cmse_check_address_range((void *)buf, len,
                                         CMSE_NONSECURE | CMSE_MPU_READ);
        if (!ok_in)
                return -1;

        ok_out = cmse_check_address_range(out, 32,
                                          CMSE_NONSECURE | CMSE_MPU_READWRITE);
        if (!ok_out)
                return -1;

        /* now safe - but STILL copy in, because Non-secure code
           on an interrupt, or a DMA engine, can change the
           buffer under you (chapter 09's TOCTOU)            */
        sha256(ok_in, len, ok_out);
        return 0;
}

/* cmse_check_address_range() consults the SAU, IDAU and the
   NON-SECURE MPU, and asks: could the caller have done this
   access itself? If not, refuse - do not do it for them.

   Also available:
     cmse_TT(p)             query the attributes of an address
     cmse_TTA(p)            ...using the alternate domain
     cmse_check_pointed_object(p, flags)   for a single object

/* AND the overflow trap: */
        if (len > MAX_LEN || buf + len < buf)
                return -1;
   cmse_check_address_range handles wraparound itself, but any
   arithmetic YOU do on len does not.                        */
```

THREE WAYS SECURE FUNCTIONS LEAK

- **Unchecked pointers.** The caller passes a secure address and your function reads or writes it. Every pointer argument needs `cmse_check_address_range`.
- **Registers left dirty on return.** The compiler clears caller-saved registers for `cmse_nonsecure_entry` functions — but not your own local buffers on the secure stack, and not the FPU registers unless configured.
- **The secure stack.** Values left there are not readable by Non-secure code directly, but they are the raw material for any subsequent bug. Zero secrets before returning.

Interrupts and stacking

```
Interrupts are assigned per-interrupt to a security state, in
NVIC_ITNS (Interrupt Target Non-Secure). Secure by default -
non-secure only if a bit is set, by secure code.

  AIRCR.PRIS      makes all Non-secure priorities the lower half
                  of the range, so a Non-secure interrupt can
                  never preempt a Secure one.
  AIRCR.BFHFNMINS puts HardFault/NMI in the Non-secure state
                  (usually left Secure).

  THE STACKING PROBLEM:

  Secure code is running. A Non-secure interrupt fires. The
  hardware must save the Secure registers - but it cannot push
  them where Non-secure code can read them.

  So the hardware does it FOR you:

     · pushes the secure context onto the SECURE stack
     · ZEROES the registers before entering the NS handler
     · pushes an integrity signature onto the secure stack
     · on return, checks that signature

  This is automatic and free. You do not write it, and you
  cannot get it wrong - which is a large part of why
  TrustZone-M is usable at all in an interrupt-latency budget.

  FOUR STACKS, and each needs its own limit register:
        MSP_S  PSP_S   MSPLIM_S  PSPLIM_S
        MSP_NS PSP_NS  MSPLIM_NS PSPLIM_NS

  SET THE SECURE STACK LIMITS. On Armv8-M a stack overflow with
  a limit register set is a fault; without one it is silent
  corruption of whatever lies below - which on the secure side
  may be your key material.
```

Boot flow

```
/* Reset always enters the SECURE state, at VTOR_S. Secure code
   sets everything up and then hands over - once, and
   irreversibly for that boot.                              */

void secure_main(void)
{
        /* 1. configure the SAU - which regions are NS and NSC */
        SAU->RNR  = 0;
        SAU->RBAR = (NS_FLASH_START & SAU_RBAR_BADDR_Msk);
        SAU->RLAR = (NS_FLASH_END   & SAU_RLAR_LADDR_Msk)
                  | SAU_RLAR_ENABLE_Msk;

        SAU->RNR  = 1;
        SAU->RBAR = (NSC_START & SAU_RBAR_BADDR_Msk);
        SAU->RLAR = (NSC_END   & SAU_RLAR_LADDR_Msk)
                  | SAU_RLAR_NSC_Msk        /* callable */
                  | SAU_RLAR_ENABLE_Msk;

        /* ... regions for NS RAM and NS peripherals ... */
        SAU->CTRL = SAU_CTRL_ENABLE_Msk;

        /* 2. vendor peripheral security - the M-profile
              equivalent of the TZPC (chapter 10)           */
        vendor_set_peripheral_ns(UART0);
        vendor_set_peripheral_ns(TIMER1);
        /* crypto, RNG, flash controller stay SECURE */

        /* 3. interrupt targets */
        NVIC_SetTargetState(UART0_IRQn);      /* -> NS */
        /* everything else stays Secure       */

        /* 4. the secure MPU, for privilege separation INSIDE
              the secure state                              */
        secure_mpu_init();

        /* 5. stack limits - on both sides */
        __set_MSPLIM(SECURE_STACK_BOTTOM);

        /* 6. init the secure services: keys, storage, crypto */
        secure_services_init();

        /* 7. VERIFY the non-secure image before running it
              (chapter 14 - the chain does not stop here)   */
        if (!verify_ns_image())
                fatal();

        /* 8. set up the NS vector table and jump */
        SCB_NS->VTOR = NS_IMAGE_START;

        __TZ_set_MSP_NS(*(uint32_t *)NS_IMAGE_START);
        ns_fn ns_reset = (ns_fn)(*(uint32_t *)(NS_IMAGE_START + 4));

        ns_reset();          /* BLXNS - never returns */
}

/* AFTER THE HANDOVER the SAU should be locked if the part
   supports it, and the secure code that remains is only what
   is reachable through the NSC veneers.                    */
```

TF-M — the reference implementation

```
/* Trusted Firmware-M is to Cortex-M what TF-A is to Cortex-A.
   You almost certainly want to use it rather than build this
   yourself.                                                  */

   BL2                MCUboot - verified boot (chapter 14)
   SPM                Secure Partition Manager - schedules the
                      secure services, isolates them
   Secure partitions  each service in its own MPU region

   Standard services, all with PSA APIs:
     Crypto           psa_import_key, psa_sign_hash, ...
     Internal Trusted Storage    small, on-chip
     Protected Storage           larger, encrypted, off-chip
     Initial Attestation         signed token about the device
     Firmware Update

/* PSA ISOLATION LEVELS:

   Level 1  Secure / Non-secure split only. One secure blob.
   Level 2  ...plus the secure services are isolated from the
            secure "root of trust" using the secure MPU.
   Level 3  ...plus each secure partition is isolated from
            every other one.

   Level 3 costs context switches and RAM; Level 1 means one
   bug in any secure service is total compromise. Most
   certified products target 2 or 3.                       */

/* PSA Certified is the certification scheme built on this. If
   a datasheet says "PSA Certified Level 2", that is what it is
   referring to.                                              */
```

> **WHEN TRUSTZONE-M EARNS ITS KEEP**
>
> On an MCU there is no second processor and no room for a separate secure MCU. TrustZone-M gives you a place to put a device identity key that survives a bug in 200 KB of application firmware — the firmware that parses BLE packets, runs a filesystem, and was written under schedule pressure. That is the realistic use: *a small, auditable secure side holding the keys and the update path*, and everything else outside it.

How it shows up

Symptom	Cause
SecureFault on the first call into secure code	Target not in an NSC region, or no `SG` at the entry — check the linker script
SecureFault immediately after handover	SAU region for NS flash wrong, or `VTOR_NS` set to the wrong address
Non-secure code hangs at reset	`MSP_NS` not initialised from the NS vector table
Undefined instruction on `BXNS`	Built without `-mcmse`
Link error: undefined reference to a secure function	Import library not linked into the non-secure project
Worked, then broke after rebuilding the secure side	Veneer addresses moved — re-link, or fix the NSC region layout
Peripheral unreachable from the application	Still assigned Secure by the vendor's peripheral protection unit
Non-secure interrupt never fires	`NVIC_ITNS` bit not set — interrupts default to Secure

Symptom	Cause
Secure code corrupts itself under load	Secure stack overflow — `MSPLIM_S` not set
Non-secure code reads secure data through a service	Missing `cmse_check_address_range` — a real vulnerability
Return from a secure call faults	Secure stack integrity signature check failed — the secure stack was corrupted

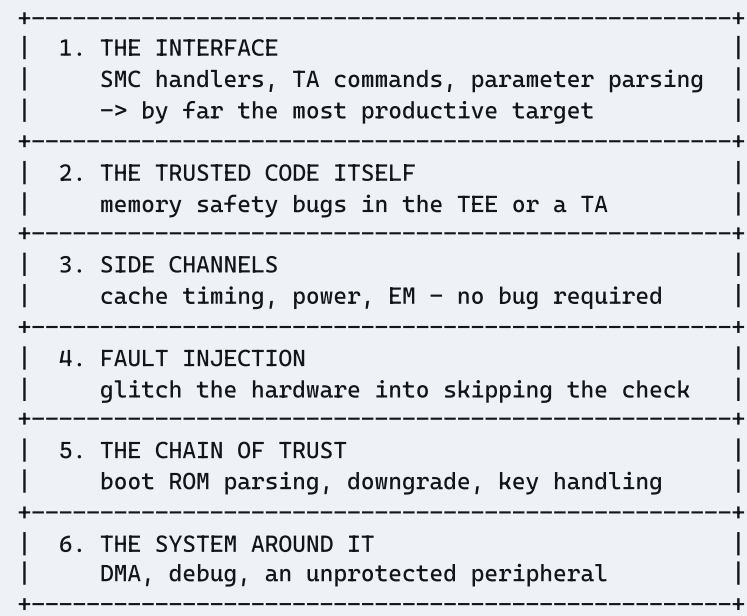
Check yourself

- ✓ State the fundamental difference between TrustZone-A and TrustZone-M.
- ✓ Explain S, NS and NSC memory, and how the SAU and IDAU combine.
- ✓ Explain why `SG` must be the first instruction at an entry point.
- ✓ Write a secure entry point that validates a caller-supplied buffer.
- ✓ Explain why checking the pointer is still not enough on its own.
- ✓ Describe what the hardware does when a Non-secure interrupt preempts secure code.
- ✓ Name the four stacks and say why the secure limit registers matter.
- ✓ Write the boot sequence from reset to the non-secure reset handler.
- ✓ Explain the three PSA isolation levels and their trade-offs.
- ✓ Say what belongs on the secure side of an MCU and what does not.

TEE attacks and mitigations

Every mechanism in this track has been broken on real hardware. Knowing how is not pessimism — it is the only way to build something that holds, and to judge honestly what a TEE is and is not worth.

Where attacks land



Note the ordering. Almost every publicly disclosed TEE compromise has been in layer 1 or 2 - ordinary software bugs in code that happened to be very privileged. The exotic attacks get the conference talks; the boring ones get the CVEs.

1. Interface attacks

```
/* THE CONFUSED DEPUTY - the defining TEE vulnerability. */

/* VULNERABLE: */
static TEE_Result cmd_read_key(uint32_t ptypes, TEE_Param p[4])
{
        /* attacker sets p[0].memref.buffer to a SECURE address */
        memcpy(p[0].memref.buffer, secret_key, 32);
        return TEE_SUCCESS;
}
/* The TA has the privilege to write anywhere. The attacker
   supplies the destination. Result: an arbitrary write
   primitive inside the secure world, driven from userspace. */

/* FIXED: */
        if (!tee_buffer_is_non_secure(p[0].memref.buffer,
                                      p[0].memref.size))
                return TEE_ERROR_ACCESS_DENIED;

/* --- MISSING paramTypes CHECK ------------------------------ */
/* If the TA does not verify ptypes, the client passes a VALUE
   where a MEMREF is expected. params[0].memref.buffer is then
   whatever 64-bit number the attacker put in params[0].value.a.
   Same primitive, reached differently.                       */

/* --- INTEGER OVERFLOW IN A SIZE CHECK ---------------------- */
        if (offset + len > buffer_size)     /* WRAPS */
                return TEE_ERROR_BAD_PARAMETERS;
        memcpy(dst + offset, src, len);
/* offset = 0xFFFFFFF0, len = 0x20 -> sum wraps to 0x10, check
   passes, memcpy writes far out of bounds.                   */
        if (offset > buffer_size || len > buffer_size - offset)
                return TEE_ERROR_BAD_PARAMETERS;   /* no wrap */

/* --- TOCTOU ------------------------------------------------ */
        if (validate(shm->cmd))         /* read 1 */
                execute(shm->cmd);      /* read 2 - DIFFERENT */
/* Another CPU rewrote it in between. Copy in ONCE, then
   validate and use only the copy.                            */

/* --- STATE CONFUSION --------------------------------------- */
/* Call sequence: OPEN, then USE. What if the attacker calls
   USE twice? USE after CLOSE? OPEN twice? Two USEs on two CPUs
   at once? A session state machine that assumes the documented
   order is a use-after-free waiting to happen.               */
```

THE INTERFACE RULES, RESTATED AS ATTACKS

Rule	What it stops
Check `paramTypes`	Type confusion into an arbitrary pointer
Validate every buffer address and size	Confused deputy
Do size arithmetic without wrapping	Overflow into a heap or stack overflow
Copy in before validating	TOCTOU from another CPU or a DMA engine
Write outputs once, on success	Partial-result and error-path leaks
Enforce the session state machine	Use-after-free, double-free, replay
Lock shared state	Races between concurrent calls
Return coarse errors	Oracles that leak secret-dependent detail

2. Memory safety in trusted code

```
/* The secure world is C. It has all of C's problems (see the C
   track, chapter 15) with none of the mitigations you get for
   free in Linux userspace.                                    */

   What is usually MISSING in a TEE:
        ASLR             often none, or very coarse
        stack canaries   sometimes; check your build flags
        NX / W^X         usually yes, but verify
        guard pages      often not - the heap is a fixed array
        a hardened allocator   rarely
        CFI              rarely
        MTE / PAC        only on newer parts, and only if enabled

   And the payoff for a bug is total: S-EL1 code owns the keys,
   the fuses, and the ability to lie to the normal world about
   attestation.

/* MITIGATIONS THAT ARE WORTH THE EFFORT: */

   -fstack-protector-strong    cheap, catches real bugs
   -D_FORTIFY_SOURCE=2
   -Wall -Wextra -Werror       and actually fix the warnings
   -fsanitize=... in a HOST harness, not on target

   CFG_TA_ASLR=y               OP-TEE: randomise TA layout
   CFG_CORE_SANITIZE_KADDRESS  debug builds only
   PAC and BTI on Armv8.3+     enable them

   Fuzz the SMC and TA interfaces on the host, with the same
   code compiled natively. Every TEE interface is a parser, and
   parsers are what fuzzers are good at.

   And: KEEP IT SMALL. A 20,000-line trusted OS can be audited.
   A 500,000-line one cannot, no matter how good the process. */
```

3. Side channels

```
The two worlds SHARE the microarchitecture: caches, branch
predictors, TLBs, the memory controller, the power rail. Sharing
a resource means measuring one another.

  CACHE TIMING (Prime+Probe, Flush+Reload)
     Normal-world code fills a cache set, lets the secure world
     run, then times its own re-access. Slow = the secure world
     evicted it = it touched that set. Repeat, and the pattern
     of accesses reveals the key - because table-driven AES and
     square-and-multiply RSA access memory in key-dependent
     ways.
     -> TruSpy, ARMageddon, and many since.

  BRANCH PREDICTOR
     Same idea, different shared structure. Secret-dependent
     branches leave traces.

  POWER and EM
     Physical access, an oscilloscope and a probe. DPA on a
     naive AES implementation needs a few thousand traces and
     an afternoon.

  MICROARCHITECTURAL SPECULATION
     Spectre-style: train a predictor, make secure code
     speculatively access secret-dependent addresses, read the
     cache. Applies across the world boundary on affected
     cores.

  MITIGATIONS:
     · CONSTANT-TIME code for anything touching a secret:
          no branch on secret data
          no memory INDEX derived from secret data
          no early exit from a comparison
     · bitsliced or table-free crypto, or a hardware engine
     · masking / blinding - randomise the intermediate values
     · flush or partition caches on world switch (costly)
     · disable SMT on the secure side, or do not use SMT
     · use the crypto accelerator: it is not sharing the cache

  ACCEPT THE LIMIT: a TEE does not defend against an attacker
  with physical access, an oscilloscope and time. It defends
  against a compromised NORMAL WORLD. Say so in your threat
  model rather than pretending otherwise.
```

```
/* Constant-time comparison - the smallest example, and the one
   people still get wrong.                                      */

/* LEAKS: returns as soon as it finds a difference, so the time
   reveals how many bytes matched.                              */
int bad_compare(const uint8_t *a, const uint8_t *b, size_t n)
{
        for (size_t i = 0; i < n; i++)
                if (a[i] != b[i])
                        return -1;
        return 0;
}

/* CONSTANT TIME: always touches every byte, no branch on data */
int ct_compare(const uint8_t *a, const uint8_t *b, size_t n)
{
        uint8_t diff = 0;

        for (size_t i = 0; i < n; i++)
                diff |= a[i] ^ b[i];

        return diff;            /* 0 iff equal */
}

/* And the same discipline everywhere: */
        if (secret_flag) x = a; else x = b;     /* leaks       */
        mask = -(uint32_t)!!secret_flag;
        x = (a & mask) | (b & ~mask);           /* does not    */

/* Beware the compiler. It can reintroduce a branch. Check the
   generated assembly for anything genuinely critical, and
   consider a barrier or volatile to pin it down.            */
```

4. Fault injection

```
Do not break the crypto - break the CHECK.

     if (verify_signature(img) != OK)
             fail();            <-- glitch the branch, skip this
     boot(img);

  Techniques, roughly by cost:
     VOLTAGE GLITCH   drop Vcc for nanoseconds. ~$50 of kit.
     CLOCK GLITCH     an extra or shortened clock edge.
     EM PULSE         a coil over the die. No decapsulation.
     LASER            decapsulated die, single-bit precision.
                      Six figures, and a lab.
     ROWHAMMER        software-only DRAM bit flips - a fault
                      attack with no hardware at all.

  Effects: an instruction skipped, a register corrupted, a
  comparison inverted, a loop counter changed.

  SOFTWARE COUNTERMEASURES - none is sufficient alone, but
  together they raise the cost a great deal:

     · DOUBLE-CHECK critical decisions, with different code
     · use non-trivial constants: a "success" value of
       0x5A3C96C3, never 0 or 1, so a cleared register is not
       automatically a pass
     · random delays before sensitive operations
     · redundant loop counters, verified after the loop
     · verify the result of the operation you just did
     · a "flow counter" incremented at each checkpoint and
       validated at the end - so a skipped stage is detectable

  HARDWARE COUNTERMEASURES: voltage and clock glitch detectors,
  light sensors, redundant logic, error-correcting registers.
  If your threat model includes physical attackers, choose
  silicon that has them - you cannot add them later.
```

```
/* Fault-hardened verification, as it is actually written in
   boot ROMs.                                                */

#define FLOW_OK    0x5A3C96C3u
#define FLOW_FAIL  0xA5C36939u

static uint32_t verify_hardened(const image_t *img)
{
        uint32_t r1, r2;
        volatile uint32_t counter = 0;

        r1 = verify_signature(img);
        counter++;

        random_delay();

        r2 = verify_signature(img);     /* again, independently */
        counter++;

        if (r1 != FLOW_OK)  return FLOW_FAIL;
        if (r2 != FLOW_OK)  return FLOW_FAIL;
        if (r1 != r2)       return FLOW_FAIL;
        if (counter != 2)   return FLOW_FAIL;   /* skipped step? */

        return FLOW_OK;
}

        if (verify_hardened(img) == FLOW_OK) {
                if (verify_hardened(img) != FLOW_OK)  /* again */
                        panic();
                boot(img);
        } else {
                panic();
        }

/* Ugly, redundant, and deliberately so. A single glitch must
   not produce a boot. Reviewers should be able to point at
   each redundant check and name the fault it catches.      */
```

Real compromises, and what they teach

Class	What happened	Lesson
TA buffer overflow	Widevine and other TAs on several vendors: a length field from the client overflowed a fixed buffer, giving code execution at S-EL0, then S-EL1 through a second bug	TAs are attack surface; keep them minimal and fuzz them
Confused deputy	Multiple CVEs across vendors where a TA wrote a result to a client-supplied pointer without checking it was non-secure	Validate *every* pointer that crosses the boundary
Downgrade	Old, signed, vulnerable TAs reloaded on patched devices — the signature was still valid	Rollback protection must cover TAs, not just firmware
Boot ROM parsing	Unpatchable ROM bugs on very large device populations, exploited before any signature check ran	Parse nothing you have not verified; keep the ROM tiny
Cache side channel	Academic extraction of AES and RSA keys from a TEE by a normal-world process	Constant-time code, or a hardware engine
Fault injection	Voltage glitching past secure boot on shipped consumer devices with inexpensive equipment	Redundant checks; glitch detectors in silicon

Class	What happened	Lesson
Debug left enabled	Production units with secure debug signals never fused off	Test the production lifecycle state, not the dev board
Unprotected DMA master	A peripheral with neither SMMU nor XPU used to read secure DRAM	Enumerate *every* bus master (ch. 17)

What a TEE is actually worth

> **AN HONEST THREAT MODEL**
>
> **A TEE defends well against:** a compromised normal-world OS, malicious applications, an attacker with root, offline extraction of keys from storage, and cloning a device by copying its filesystem.
>
> **A TEE defends poorly or not at all against:** an attacker with physical access, time and laboratory equipment; side channels, unless the code was written for it; bugs in the trusted code itself; and anything the silicon integration left unprotected.
>
> That is still a great deal of value — it is the difference between "root compromises every device forever" and "root compromises this device's session". Just do not let a datasheet's security bullet points substitute for stating which of the two you are buying.

How it shows up

Symptom	Cause
Secure world crashes on a malformed request	Missing input validation — and a crash means the parser is reachable
A TA works but fails a fuzz run in minutes	Unvalidated length or offset fields
Timing varies with a secret	Non-constant-time comparison or table lookup
Different error codes for different failure reasons	An oracle — collapse them into one
Device boots unsigned images intermittently	Glitch attack, or an unhardened single check
Old firmware still accepted	Rollback counter never burned
Secure memory readable via a peripheral	A bus master outside the SMMU/XPU policy
Attestation passes on a rooted device	Measurement not covering the compromised component
A fix in the TEE does not take effect	The old, signed TA is still loadable

Check yourself

- ✓ Name the six layers where TEE attacks land, and say which produce most real CVEs.
- ✓ Explain the confused deputy and write the check that stops it.
- ✓ Show a size check that overflows, and rewrite it so it cannot.
- ✓ Explain TOCTOU across the world boundary and the copy-in fix.
- ✓ List the exploit mitigations usually missing from a TEE and the build flags that add some back.
- ✓ Explain a cache timing attack and the properties constant-time code must have.
- ✓ Write a constant-time comparison and a branchless select.
- ✓ Explain why fault injection targets the check rather than the crypto, and list four software countermeasures.

- ✓ State plainly what a TEE does and does not defend against.

Access control: XPU concepts

The MMU protects a CPU from itself. The SMMU protects memory from devices that have one. XPUs are the last layer: a firewall in the interconnect that decides, per transaction, whether *this master* may touch *this address* — and it is the one nobody teaches.

The gap XPUs fill

```
Three protection mechanisms, three different questions:

  MMU     "may this VIRTUAL ADDRESS be accessed by the code
           currently running, at its privilege level?"
          Per-CPU. Translates AND checks.

  SMMU    "may this DEVICE's virtual/IO address be translated,
           and to what?"
          Per-StreamID. Translates AND checks.
          Requires the device to be behind an SMMU.

  XPU     "may THIS MASTER touch THIS PHYSICAL ADDRESS, in
           THIS security state, for READ or WRITE?"
          No translation at all. Pure permission check,
          on physical addresses, in the interconnect.

  WHY THE THIRD ONE IS NECESSARY:

  1. Not every master is behind an SMMU. Small DMA engines,
     sensor hubs, audio DSPs, power-management microcontrollers,
     debug blocks, display pipelines - many are wired straight
     to the interconnect.

  2. The SMMU protects DRAM well. It does nothing about
     PERIPHERAL registers. Which master may write the clock
     controller? The fuse controller? Another CPU's mailbox?

  3. TZASC and TZPC (chapter 10) are two-valued: secure or
     non-secure. Real SoCs have more than two principals - a
     modem, an application processor, a video DSP, a sensor
     hub, all mutually distrusting and all NON-SECURE.

  4. Some masters must be constrained BEFORE the SMMU is
     configured - during boot, when a firmware core is running
     and Linux is not.
```

THE NAME

"XPU" is a family, not a standard. Qualcomm calls them XPUs (with variants MPU, PPU, VMIDMT). NXP i.MX has the RDC (Resource Domain Controller), the CSU, and TrustZone-aware AIPSTZ. TI has firewalls. Renesas has the Access Window. Xilinx/AMD has XMPU and XPPU. Arm's own is the TZC family. The *mechanism* is the same everywhere; only the register names and the marketing differ, so learn the model rather than one vendor's manual.

The model

```
 MASTER                            SLAVE
 ------                            -----
 CPU cluster    ---+
 GPU            ---+
 display        ---+          +--> DRAM
 video codec    ---+---> XPU -+--> SRAM
 modem DSP      ---+          +--> peripheral registers
 audio DSP      ---+          +--> fuses, crypto, PMU
 sensor hub     ---+
 DMA engine     ---+

Each transaction on the bus carries:

   ADDRESS        where
   MASTER ID      who   (AxID / AxUSER / a dedicated sideband)
   NS             which world (AxPROT[1])
   PRIVILEGED     AxPROT[0]
   READ / WRITE   direction
   sometimes: a VMID, so one master can be split by which
              virtual machine issued the access

The XPU holds a table of REGIONS:

   +-------------------------------------------------------+
   | region 3                                              |
   |   base   0x8000_0000   size 64 MB                     |
   |   masters allowed: CPU(rw) GPU(r--) DISPLAY(r--)       |
   |   security: non-secure only                           |
   |   locked: yes                                         |
   +-------------------------------------------------------+

Transaction arrives -> find the matching region -> look up
this master's permission -> ALLOW or DENY.

And critically: what happens when NOTHING matches. That is
the DEFAULT, and it should be DENY.
```

Where they sit

Placement	Protects	Trade-off
At the master (a "master-side" or ingress firewall)	Constrains one specific device wherever it goes	One instance per master; a master with several ports needs several
At the slave (memory controller, peripheral bridge)	Everything reaching that slave, from every master	Fewer instances; but the master ID must survive the whole path to get there
In the interconnect fabric	Whole subsystems from one another	Coarse granularity, but catches paths you did not think of
Inside a peripheral (per-register or per-channel)	Fine-grained sharing of one device	Only where the vendor implemented it

> **THE MASTER ID IS ONLY AS TRUSTWORTHY AS THE WIRING**
> An XPU's entire decision rests on the master ID arriving intact and unforgeable. Real integration problems: IDs remapped by a bridge so the XPU sees the bridge's ID and not the originator's; several masters sharing one ID because the sideband was too narrow; an ID that a master can *program*. Any of these collapses the policy to "some device did this", which is not a policy. It is exactly the StreamID integrity question from chapter 03, and it has the same answer: verify it empirically, per SoC.

Granularity and its limits

```
/* Every XPU has hard limits, and they shape your design more
   than any principle does.                                    */

   NUMBER OF REGIONS      typically 8, 16, or 32. Rarely more.
   MINIMUM REGION SIZE    4 KB, 64 KB, or 1 MB depending on
                          the block. Sometimes power-of-two
                          and naturally aligned, sometimes an
                          arbitrary base/limit pair.
   NUMBER OF MASTERS      a bitmap of 8-32 entries, or a list
                          of ID + mask pairs.
   PERMISSION GRANULARITY read/write, sometimes execute,
                          sometimes secure/non-secure
                          independently.

/* CONSEQUENCES you will hit:

   · you cannot give every master a private region - there are
     not enough regions. You will group masters into DOMAINS.

   · buffers must be laid out to MATCH the alignment the XPU
     can express. A 1 MB minimum region means a 4 KB secure
     buffer wastes 1 MB, or worse, cannot be isolated at all
     without moving everything around it.

   · this is a MEMORY MAP DESIGN constraint, decided very early
     in a project. Retrofitting it after the address map is
     frozen is painful and is usually done by giving up on
     some isolation.

   · ID masking lets one entry cover a group:
          match: id & 0xF0 == 0x20
     which is efficient, and also means an accidental grouping
     if the masks are chosen carelessly.                    */
```

What happens on a denial

```
Three possible behaviours, and which one you get changes how
hard debugging is by an order of magnitude:

  1. ERROR RESPONSE
        the bus returns SLVERR or DECERR.
        the master sees an abort. For a CPU that is a
        synchronous or asynchronous external abort
        (ARM ch. 29). For a DMA engine, a channel error.
        BEST: something visibly fails, close to the cause.

  2. RAZ/WI - read as zero, write ignored
        the transaction "succeeds" and does nothing.
        WORST: no error anywhere. A driver reads 0x00000000
        from its ID register and reports "device not present".
        Days are lost to this (chapter 20).

  3. INTERRUPT + LOG, transaction dropped or errored
        the XPU records address, master ID, security state and
        direction in fault registers and raises an interrupt to
        a monitor.
        BEST FOR DEBUGGING, if someone wired the interrupt up
        and someone else wrote the handler.

  ALWAYS configure error responses and the fault interrupt in
  development builds, even if production chooses to be quiet.
  A silent policy is an untestable policy.
```

XPU versus SMMU: choosing

	SMMU	XPU
Translates addresses	Yes — scatter-gather, virtual DMA	No
Per-context (per-process, per-VM)	Yes — thousands of contexts	No — a handful of static regions
Dynamic remapping	Yes, per DMA mapping	Rarely, and expensively
Protects peripheral registers	Not usually	Yes
Works before the OS boots	Needs setup and tables	Yes — a few register writes
Cost in silicon	Large: TLBs, page walkers, queues	Small: comparators
Latency	TLB hit is fast; a walk is not	Fixed, small
Fault reporting	Rich — event queue with full context	Basic — often one fault register

They are complements, not alternatives. A well-built SoC uses the SMMU for masters that need scatter-gather DMA into user or guest memory, and XPUs for the static, coarse questions: which subsystem owns which DRAM carve-out, and who may write the clock and fuse controllers. Where both cover a path, **both** must allow the transaction — and that intersection is itself a defence, since a mistake in one is caught by the other.

Domains: the usable abstraction

```
/* With 16 regions and 20 masters you cannot enumerate pairs.
   So group masters into DOMAINS and write the policy in terms
   of domains.                                                */

   DOMAIN_SECURE     EL3 firmware, the trusted OS, the crypto
                     engine when claimed
   DOMAIN_AP         the application processor and everything
                     Linux drives directly
   DOMAIN_MODEM      the cellular subsystem: its DSP, its own
                     DMA, its dedicated peripherals
   DOMAIN_MEDIA      GPU, display, video codec, camera
   DOMAIN_SENSOR     the always-on sensor hub
   DOMAIN_DEBUG      CoreSight, the debug access port

/* Then the policy is a MATRIX - domains against resources: */

                  secure  AP     modem  media  sensor
   secure DRAM      rw     -      -      -      -
   AP DRAM          rw     rw     -      r      -
   modem DRAM       rw     -      rw     -      -
   shared AP/modem  rw     rw     rw     -      -
   framebuffer      rw     rw     -      rw     -
   fuses            r      -      -      -      -
   clock controller rw     rw     -      -      -
   sensor SRAM      rw     r      -      -      rw

/* This matrix IS the security architecture of the SoC. It
   belongs in a design document, reviewed by people who did not
   write it, and it should be MECHANICALLY DERIVED into the
   register settings rather than hand-written twice
   (chapter 19).

   Note what the matrix makes obvious that prose does not: the
   modem cannot see the framebuffer; the sensor hub cannot see
   AP memory; nothing but secure code reads the fuses. Those
   are decisions. Write them down and they can be argued with. */
```

Why this matters beyond security

> **THE BEST ARGUMENT FOR XPUS IS NOT SECURITY — IT IS DEBUGGING**
>
> A rogue DMA write into another subsystem's memory produces a crash *somewhere else, much later*, in code that is entirely innocent. It is the hardest class of bug on an SoC: no stack trace points at the culprit, and the symptom moves whenever anything is recompiled.
>
> An XPU converts that into an immediate fault at the moment of the bad access, with the offending master's ID in a register. That turns a two-week bug into a two-hour one, and it is why teams that adopt XPU policies for security keep them enabled for reasons that have nothing to do with attackers.

How it shows up

Symptom	Cause
Driver reads all-zero registers, reports "no device"	XPU denying with RAZ/WI

Symptom	Cause
External abort on an MMIO access that used to work	Firmware tightened the policy; this master is no longer allowed
DMA completes but the buffer is unchanged	Writes ignored by the XPU
Works when firmware is old, fails with new firmware	Policy changed — the two must be versioned together
One master works, an identical one does not	Different master IDs; the policy covers only one
Fails only for large transfers	The buffer crosses a region boundary
Fails only sometimes	Buffer address varies; a boundary is crossed only for some allocations
Everything is allowed	Default region permissive, or the XPU never enabled
A subsystem can reach memory it should not	Master ID remapped by a bridge, or shared with another master
Random corruption with no attribution	The exact bug an XPU exists to catch — turn one on

Check yourself

- ✓ State the different question each of the MMU, SMMU and XPU answers.
- ✓ Give four reasons an SoC needs XPUs even though it has an SMMU.
- ✓ List the fields of a bus transaction an XPU decides on.
- ✓ Explain why the master ID's integrity is the foundation of the whole mechanism.
- ✓ Describe how region count and alignment constrain the memory map.
- ✓ Name the three denial behaviours and rank them for debuggability.
- ✓ Say when to use an SMMU, when an XPU, and why overlap is good.
- ✓ Draw a domain-versus-resource matrix for a device you know.
- ✓ Explain the debugging argument for enabling XPUs.

Access control: programming and locking

Now configure one. The register model, the ordering rules that decide whether your policy is ever actually in force, and the locking that stops later code from undoing it.

The register model

```
/* Vendor names differ; the shape does not. A representative
   XPU region block:                                          */

   +0x000  XPU_CTRL          global enable, default action
   +0x004  XPU_STATUS        enabled, locked, error pending
   +0x008  XPU_ERR_STATUS    valid, direction, security, type
   +0x00C  XPU_ERR_ADDR_LO   the address that faulted
   +0x010  XPU_ERR_ADDR_HI
   +0x014  XPU_ERR_MASTER    which master ID
   +0x018  XPU_ERR_CLEAR     write 1 to clear and re-arm
   +0x01C  XPU_IRQ_ENABLE

   /* then N region blocks, 0x20 bytes apart: */
   +0x100  XPU_R0_BASE_LO    region base address
   +0x104  XPU_R0_BASE_HI
   +0x108  XPU_R0_LIMIT_LO   inclusive end, or a size encoding
   +0x10C  XPU_R0_LIMIT_HI
   +0x110  XPU_R0_RD_ALLOW   bitmap: which masters may read
   +0x114  XPU_R0_WR_ALLOW   bitmap: which masters may write
   +0x118  XPU_R0_CONFIG     enable, security, subregion mask
   +0x11C  XPU_R0_LOCK       write 1: frozen until reset

/* THE VARIATIONS you will meet:

   base+limit vs base+size          - and if size, usually a
                                      power of two, naturally
                                      aligned
   master BITMAP vs ID+MASK pairs   - bitmaps are simpler;
                                      ID+mask scales further
   one permission field vs separate secure/non-secure ones
   subregion disable masks          - split a region into 8
   region PRIORITY                  - overlapping regions:
                                      highest index wins, or
                                      lowest, or it is
                                      UNPREDICTABLE. READ THE
                                      TRM. This one bites.  */
```

The ordering that matters

```
Getting the sequence wrong leaves a WINDOW in which the system
is unprotected, or bricks the boot because you locked yourself
out. The safe order:

  1. Program the DEFAULT to DENY, but do NOT enable yet.
  2. Program every specific region: base, limit, permissions.
  3. Verify by reading back every register you wrote.
  4. Add the region containing YOUR OWN code and stack, with
     permission for the master you are running on.
     <-- forget this and step 5 halts the CPU
  5. Enable the XPU.
  6. Test the policy: negative cases, not just positive.
  7. LOCK it.

  AND THE INVERSE ORDER for reconfiguration, which is why you
  should avoid needing it: an XPU cannot usually be "paused",
  so changing a region while masters are active means either
  a brief window of wrong policy, or quiescing everything
  first.

  A DIFFERENT WINDOW to think about: between reset and step 5,
  NOTHING is enforced. Masters that are released from reset
  before the XPU is configured can do anything. So:

     configure the XPU BEFORE releasing other masters from
     reset, or hold them in reset until it is done.

  That single sentence is the most commonly violated rule in
  this whole chapter.
```

Configuring one

```
/* Firmware, running at EL3 during BL2/BL31 (chapter 14).    */

struct xpu_region {
        uint64_t base;
        uint64_t limit;         /* inclusive */
        uint32_t rd_masters;    /* bitmap */
        uint32_t wr_masters;
        uint32_t flags;         /* SECURE_ONLY, NS_ONLY, BOTH */
        bool     lock;
};

static const struct xpu_region policy[] = {
        /* secure firmware DRAM - only the secure domain */
        { .base = 0x0E000000, .limit = 0x0E3FFFFF,
          .rd_masters = M_CPU,
          .wr_masters = M_CPU,
          .flags = XPU_SECURE_ONLY, .lock = true },

        /* the trusted OS carve-out */
        { .base = 0x0E400000, .limit = 0x0EFFFFFF,
          .rd_masters = M_CPU | M_CRYPTO,
          .wr_masters = M_CPU | M_CRYPTO,
          .flags = XPU_SECURE_ONLY, .lock = true },

        /* the normal world's DRAM */
        { .base = 0x80000000, .limit = 0xBFFFFFFF,
          .rd_masters = M_CPU | M_GPU | M_DISPLAY | M_VIDEO | M_DMA,
          .wr_masters = M_CPU | M_GPU | M_VIDEO | M_DMA,
          .flags = XPU_NS_ONLY, .lock = true },
          /* note: DISPLAY reads only. A display controller has
             no business writing DRAM, and a compromised one
             then cannot corrupt the kernel.                 */

        /* modem's private memory - the AP cannot see it */
        { .base = 0xC0000000, .limit = 0xC7FFFFFF,
          .rd_masters = M_MODEM,
          .wr_masters = M_MODEM,
          .flags = XPU_NS_ONLY, .lock = true },

        /* the AP/modem shared buffer - the ONLY overlap */
        { .base = 0xC8000000, .limit = 0xC80FFFFF,
          .rd_masters = M_CPU | M_MODEM,
          .wr_masters = M_CPU | M_MODEM,
          .flags = XPU_NS_ONLY, .lock = true },

        /* fuse controller - secure reads only, no writes at
           runtime by anyone                                  */
        { .base = 0x01790000, .limit = 0x01790FFF,
          .rd_masters = M_CPU,
          .wr_masters = 0,
          .flags = XPU_SECURE_ONLY, .lock = true },
};

int xpu_apply(uintptr_t xpu, const struct xpu_region *r, size_t n)
{
        size_t i;

        if (n > XPU_NR_REGIONS)
                return -EINVAL;          /* fail, do not truncate */
```

```
        /* 1. default DENY, error responses on */
        mmio_write32(xpu + XPU_CTRL,
                     XPU_CTRL_DEFAULT_DENY | XPU_CTRL_ERR_RESP);

        /* 2. program regions */
        for (i = 0; i < n; i++) {
                uintptr_t reg = xpu + XPU_REGION(i);

                mmio_write32(reg + XPU_R_BASE_LO,  (uint32_t)r[i].base);
                mmio_write32(reg + XPU_R_BASE_HI,  r[i].base >> 32);
                mmio_write32(reg + XPU_R_LIMIT_LO, (uint32_t)r[i].limit);
                mmio_write32(reg + XPU_R_LIMIT_HI, r[i].limit >> 32);
                mmio_write32(reg + XPU_R_RD_ALLOW, r[i].rd_masters);
                mmio_write32(reg + XPU_R_WR_ALLOW, r[i].wr_masters);
                mmio_write32(reg + XPU_R_CONFIG,
                             r[i].flags | XPU_R_ENABLE);
        }

        /* disable every region we did NOT program - stale
           entries from a previous boot stage are a real
           hazard                                          */
        for (; i < XPU_NR_REGIONS; i++)
                mmio_write32(xpu + XPU_REGION(i) + XPU_R_CONFIG, 0);

        /* 3. read back and verify (see below) */
        if (xpu_verify(xpu, r, n))
                return -EIO;

        /* 4. enable */
        mmio_write32(xpu + XPU_CTRL,
                     XPU_CTRL_DEFAULT_DENY | XPU_CTRL_ERR_RESP |
                     XPU_CTRL_IRQ_EN | XPU_CTRL_ENABLE);

        dsb(sy);                /* make it visible before we
                                   depend on it (ARM ch. 21) */
        return 0;
}
```

ALWAYS DISABLE THE REGIONS YOU DID NOT PROGRAM

An earlier boot stage — the ROM, or a previous firmware version, or a warm reset path — may have left regions enabled. They persist. A region you never wrote about is not "absent"; it is whatever the last writer left there, and it may be a wide-open range someone added temporarily for bring-up.

Read back, always

```
/* Write-then-verify is not paranoia here. Real reasons a write
   does not take:

     · the region is already LOCKED from an earlier stage
     · the address is not aligned to the granularity, and the
       low bits were silently dropped - so your region is not
       where you think
     · the size is below the minimum and was rounded up
     · you are not allowed to program this XPU from this
       master or this security state
     · the block is not clocked or not out of reset yet
     · a reserved bit rejected the whole write               */

static int xpu_verify(uintptr_t xpu, const struct xpu_region *r,
                      size_t n)
{
        for (size_t i = 0; i < n; i++) {
                uintptr_t reg = xpu + XPU_REGION(i);
                uint64_t base, limit;

                base  = (uint64_t)mmio_read32(reg + XPU_R_BASE_HI) << 32;
                base |= mmio_read32(reg + XPU_R_BASE_LO);
                limit = (uint64_t)mmio_read32(reg + XPU_R_LIMIT_HI) << 32;
                limit |= mmio_read32(reg + XPU_R_LIMIT_LO);

                if (base != r[i].base || limit != r[i].limit) {
                        ERROR("xpu: region %zu base/limit mismatch: "
                              "wanted %llx-%llx got %llx-%llx\n",
                              i, r[i].base, r[i].limit, base, limit);
                        return -1;
                }
                if (mmio_read32(reg + XPU_R_RD_ALLOW) != r[i].rd_masters ||
                    mmio_read32(reg + XPU_R_WR_ALLOW) != r[i].wr_masters) {
                        ERROR("xpu: region %zu permissions mismatch\n", i);
                        return -1;
                }
        }
        return 0;
}

/* And FAIL THE BOOT if verification fails. A system whose
   access-control policy did not apply is not a system that
   should continue booting. "Warn and carry on" here is how
   devices ship unprotected.                                */
```

Locking

```
A policy that can be changed by later code is not a policy - it
is a suggestion. Three mechanisms, usually combined:

  1. PER-REGION LOCK BITS
        write 1 to XPU_Rn_LOCK. Sticky until reset.
        Now that region cannot be moved, resized, or widened.

  2. GLOBAL LOCK
        locks the whole block, including the enable bit and the
        default action. Some blocks also make the lock bit
        itself sticky, so it cannot be cleared.

  3. THE XPU'S OWN REGISTERS PROTECTED BY AN XPU
        the configuration interface is itself a slave. Make it
        reachable only from the secure domain, so even before
        locking, only EL3 firmware could change it.

  ORDER: lock LAST, after verification and after testing. Once
  locked, your only way out is a reset.

  WHAT LOCKING BUYS: an attacker who compromises the normal
  world - or even the trusted OS - cannot widen the policy. The
  configuration window closed in early firmware, in code that
  is much smaller and much more reviewed.

  WHAT TO LEAVE UNLOCKED, deliberately and with a written
  reason: regions that genuinely must change at runtime, such
  as a framebuffer switched to secure for a trusted UI. Those
  become the attack surface, so make the code that changes them
  as small as the SMC handler it lives behind (chapter 11).
```

Dynamic reassignment, when you must

```
/* Use case: a trusted UI. The display buffer must become
   SECURE while a PIN is entered, then return.

   This is a state machine, and every transition is a place to
   get it wrong.                                              */

TEE_Result secure_display_acquire(void)
{
        int ret;

        if (state != DISPLAY_NORMAL)
                return TEE_ERROR_BUSY;      /* no re-entry */

        /* 1. stop the normal world's access FIRST. Order
              matters: if we made it secure while NS DMA was in
              flight, we would get faults, not safety.       */
        ret = display_quiesce_ns();
        if (ret)
                return TEE_ERROR_BUSY;

        /* 2. WIPE the buffer. Whatever the normal world left
              there must not be shown as if it were ours, and
              our content must not leak back later.          */
        memset_secure(fb_va, 0, fb_size);
        cache_clean_range(fb_va, fb_size);

        /* 3. reassign: secure only, CPU and display only */
        ret = xpu_set_region(FB_REGION,
                             M_CPU | M_DISPLAY,   /* read  */
                             M_CPU,               /* write */
                             XPU_SECURE_ONLY);
        if (ret)
                goto out_resume;

        /* 4. verify it took */
        if (xpu_verify_region(FB_REGION, XPU_SECURE_ONLY))
                goto out_resume;

        state = DISPLAY_SECURE;
        return TEE_SUCCESS;

out_resume:
        display_resume_ns();
        return TEE_ERROR_GENERIC;
}

TEE_Result secure_display_release(void)
{
        if (state != DISPLAY_SECURE)
                return TEE_ERROR_BAD_STATE;

        /* WIPE BEFORE HANDING BACK - the PIN pad is still on
           screen. This is the step people forget, and it
           hands the attacker exactly what the trusted UI was
           protecting.                                       */
        memset_secure(fb_va, 0, fb_size);
        cache_clean_range(fb_va, fb_size);

        xpu_set_region(FB_REGION,
                       M_CPU | M_DISPLAY | M_GPU,
```

```
                        M_CPU | M_GPU,
                        XPU_NS_ONLY);

        state = DISPLAY_NORMAL;
        display_resume_ns();
        return TEE_SUCCESS;
}

/* THE RULES for any dynamic reassignment:
     · only secure code may request it
     · quiesce the losing side before the switch
     · WIPE on both transitions
     · verify the change applied
     · make it re-entrant-safe: two CPUs, or a crash mid-way
       does not permanently take the display                */
```

What Linux sees

```
/* Linux is usually a SUBJECT of the policy, not its author -
   the XPU was configured and locked before the kernel started.
   But it must know the shape of it.                          */

/* 1. Memory the kernel must not touch is removed from the
      memory map, in the device tree: */

        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

                optee@e400000 {
                        reg = <0x00 0x0e400000 0x00 0x00c00000>;
                        no-map;          /* do not even MAP it */
                };

                modem_mem: modem@c0000000 {
                        reg = <0x00 0xc0000000 0x00 0x08000000>;
                        no-map;
                };

                shared_mem: shared@c8000000 {
                        reg = <0x00 0xc8000000 0x00 0x00100000>;
                        /* no no-map: the kernel MAY map this,
                           it is the shared window            */
                };
        };

/* "no-map" matters: without it the kernel creates a linear
   mapping, and a speculative access through that mapping can
   fault - producing an SError at a completely unrelated
   moment (ARM chapter 29). With no-map, the memory is never
   mapped and never touched.                                  */

/* 2. A driver claims its share: */

        my_device: dma-thing@1a000000 {
                compatible = "vendor,dma-thing";
                reg = <0x00 0x1a000000 0x00 0x1000>;
                memory-region = <&shared_mem>;
        };

        /* and in probe: */
        ret = of_reserved_mem_device_init(dev);
        /* now dma_alloc_coherent() on this device allocates
           from that region - which is the region the XPU
           policy allows this master to touch. The two
           descriptions must agree, and nothing checks that
           they do except your testing.                       */
```

TWO DESCRIPTIONS OF THE SAME TRUTH

The XPU policy lives in firmware. The reserved-memory nodes live in the device tree. If they disagree, you get either a security hole (the kernel can touch what it should not) or a bug that appears late and far from its cause (the kernel gets memory the hardware will refuse). Generate both from one source, or at minimum test that they agree. Chapter 19 is about making that systematic.

How it shows up

Symptom	Cause
Register writes have no effect	Region already locked, or the block is not clocked/out of reset
Region is not where you programmed it	Base not aligned to the granularity; low bits dropped
System halts the moment you enable the XPU	You did not allow your own code's region
Policy applies but an old, wide region is still active	Unused regions not explicitly disabled
Behaviour differs between cold and warm boot	Warm reset does not clear the block; stale configuration
Overlapping regions behave unpredictably	Priority rules differ per block — check the TRM
A master misbehaves before firmware finishes	Released from reset before the XPU was configured
Trusted UI leaks the previous screen	Buffer not wiped on acquire
PIN pad visible to the normal world afterwards	Buffer not wiped on release
SError at a random later instruction	Kernel speculatively touched a carve-out — add `no-map`
`dma_alloc_coherent` memory rejected by hardware	DT reserved-memory and the XPU policy disagree

Check yourself

- ✓ Sketch an XPU's register map and name what each group does.
- ✓ Give the safe configuration order and say what each step prevents.
- ✓ Explain why other masters must be held in reset until the policy is applied.
- ✓ Write a region table for a device with secure, AP and modem domains.
- ✓ List five reasons a register write silently does not take, and write the read-back check.
- ✓ Explain the three locking mechanisms and when to apply them.
- ✓ Write a safe dynamic reassignment, including both wipes.
- ✓ Explain what `no-map` does and the failure it prevents.
- ✓ Explain why the DT and the firmware policy must be generated from one source.

Access control: designing a policy

Knowing how to program an XPU is the easy half. The hard half is deciding what the policy should *be* — and doing it as engineering rather than as a guess, on a chip with sixty bus masters and a memory map nobody has ever written down in one place.

The method

```
 1. ENUMERATE every bus master. All of them.
 2. ENUMERATE every protected resource.
 3. Build the ACCESS MATRIX: who needs what, and why.
 4. Group masters into DOMAINS to fit the hardware's limits.
 5. Lay out the MEMORY MAP to match the region granularity.
 6. Express the policy as DATA, in one place.
 7. GENERATE the firmware tables and the device tree from it.
 8. TEST the negative cases.
 9. REVIEW, with people who did not write it.

Steps 1 and 2 take a week and feel like clerical work. They
are the whole job. Every failure of a real policy traces back
to a master nobody listed or a resource nobody classified.
```

Step 1: enumerate the masters

```
/* "Every block that can issue a bus transaction." Far more
   than people expect. On a mid-range application SoC:      */

   CPU cluster(s)          often several, with different IDs
                           per core or per cluster
   GPU                     usually several masters: shader,
                           command processor, ...
   display controller      one per pipe
   camera ISP              plus its own DMA
   video encoder/decoder
   audio DSP               and its DMA
   sensor hub / always-on MCU
   modem subsystem         its DSP, its DMA, its own CPU
   Wi-Fi / Bluetooth       often bus masters over PCIe or a
                           dedicated port
   PCIe root complex       and EVERY endpoint behind it
   USB host and device
   SDIO / eMMC / UFS controllers
   Ethernet MAC
   generic DMA engines     several, with many channels
   crypto engine
   power-management controller / SCP
   CoreSight / debug access port     <-- a master, and the
                                         one people forget
   trace funnel writing to memory (ETR)
   a "system cache" or interconnect maintenance block

/* FOR EACH ONE, record:

     what is its master ID, or ID range?
     is the ID fixed, or programmable by that master?
     is it behind an SMMU? which one? which StreamIDs?
     is it behind an XPU? which one?
     what security state does it issue in? fixed or dynamic?
     who controls its firmware, and how is that firmware
       verified?
     when is it released from reset, relative to the XPU
       configuration?

/* SOURCES: the SoC TRM's master ID table, the interconnect
   configuration, the SMMU StreamID mapping, the DT, and -
   this one matters - ASKING THE SoC INTEGRATION TEAM. The
   documentation is routinely incomplete. Blocks get added
   late.

/* AND THEN VERIFY EMPIRICALLY: program a deny-all XPU, run the
   system, and collect the faults. Every master that faults is
   one you now know about. Masters that never appear are either
   idle or not covered - and telling those two apart is the
   point of the exercise.                                   */
```

> **THE MASTER YOU DID NOT LIST IS YOUR VULNERABILITY**
>
> A debug block that can write DRAM. A power-management microcontroller running unsigned vendor firmware with full bus access. A PCIe endpoint that arrives with the user's plug-in card. An audio DSP whose firmware is loaded from the normal-world filesystem. Each of these has been the path around an otherwise sound TrustZone design on shipping hardware. The policy is defined by its *completeness*, not by how carefully you wrote the entries you thought of.

Step 2: enumerate the resources

Class	Examples	Typical rule
Secure code and data	BL31, trusted OS, secure heap, secure stacks	Secure domain only. Locked.
Key material	Fuse controller, key slots, RNG, crypto engine	Secure domain, minimum permissions. Often no write at runtime at all.
Subsystem private memory	Modem heap, DSP firmware and scratch	That subsystem only. Not even the AP.
Shared buffers	AP↔modem IPC, mailboxes, ring buffers	Exactly the two parties. The *only* intended overlaps.
Normal-world DRAM	The kernel and everything it manages	AP domain plus the media masters that need it.
Protected media	DRM decode buffers, framebuffer during trusted UI	Secure or a "protected" domain; write-only for some masters.
System control	Clocks, resets, power, PLLs, thermal	Whoever owns power management. Usually secure or AP, not both.
The access-control blocks themselves	XPU, SMMU secure interface, TZASC config	Secure domain only, then locked.
Debug	CoreSight, ETR buffers, the DAP	Gated by the lifecycle state (ch. 10).

Step 3: the access matrix

```
# One row per resource, one column per domain, and a REASON
# for every non-empty cell. The reason column is what makes
# this a review artifact rather than a table.

resource            secure  ap    modem media sensor debug  reason
------------------------------------------------------------------
bl31_dram            rw      -     -     -     -      -     EL3 only
optee_dram           rw      -     -     -     -      -     TEE only
optee_shm            rw      rw    -     -     -      -     TEE ABI (ch.12)
ns_dram              rw      rw    -     rw    -      -     kernel + media
modem_dram           rw      -     rw    -     -      -     modem private
ap_modem_shm         rw      rw    rw    -     -      -     IPC ring
sensor_sram          rw      r     -     -     rw     -     AP reads results
framebuffer          rw      rw    -     rw    -      -     display path
protected_video      rw      -     -     w     -      -     write-only: the
                                                            codec fills it,
                                                            nothing reads it
                                                            back to the AP
fuse_ctrl            r       -     -     -     -      -     provisioning only
crypto_engine        rw      -     -     -     -      -     secure keys
rng                  rw      rw    -     -     -      -     both need entropy
clock_ctrl           rw      rw    -     -     -      -     Linux CCF drives it
xpu_config           rw      -     -     -     -      -     self-protection
coresight            rw      -     -     -     -      rw    lifecycle gated

# READ THE EMPTY CELLS. They are the security properties:
#   modem cannot see the framebuffer
#   media cannot see modem memory
#   nothing but secure code reads the fuses
#   the AP cannot read protected video back
#
# Each empty cell should survive the question "what breaks if
# we fill this in?" - and if the answer is "nothing", it is a
# property worth keeping.
```

THREE PRINCIPLES, APPLIED CONCRETELY

- **Least privilege.** Start from all-deny and add only what a demonstrated failure requires. Never start from all-allow and subtract.
- **Directionality.** Read and write are separate questions. A display controller reads; it never writes. A video encoder writes a protected buffer; nothing reads it back. Half the value of a good matrix is in the cells that say `r` or `w` rather than `rw`.
- **Justify every allow.** A cell without a reason is a cell nobody can safely remove later, which means it will still be there in five years' worth of derived products.

Step 5: laying out memory to fit

```
The XPU has 16 regions and a 1 MB granule. Your matrix has 24
distinct resource classes. Something has to give - and the
right thing to give is the ADDRESS MAP, decided early.

  BAD LAYOUT - interleaved, needs a region per buffer:

     0x8000_0000  kernel
     0x8010_0000  modem shared
     0x8020_0000  kernel
     0x8030_0000  sensor
     0x8040_0000  kernel
     ...                       -> 20 regions, 4 KB apart,
                                  impossible to express

  GOOD LAYOUT - grouped by DOMAIN, aligned to the granule:

     0x8000_0000  +----------------------+
                  | AP DRAM              |  1 region
                  | 768 MB               |
     0xB000_0000  +----------------------+
                  | AP/modem shared 4 MB |  1 region
     0xB040_0000  +----------------------+
                  | AP/sensor shared 1 MB|  1 region
     0xB050_0000  +----------------------+
                  | (gap - denied)       |
     0xC000_0000  +----------------------+
                  | modem private 128 MB |  1 region
     0xC800_0000  +----------------------+
                  | protected media 64 MB|  1 region
     0xCC00_0000  +----------------------+
                  | secure world 32 MB   |  1 region
     0xCE00_0000  +----------------------+

  Six regions instead of twenty. And note the GAPS: address
  space that matches no region and is therefore denied to
  everyone. Gaps are free - use them as guard bands between
  domains, so an off-by-a-little DMA lands in a fault rather
  than in the neighbour's memory.

  THE RULE: group by WHO MAY TOUCH IT, not by what it is for.
  All AP memory contiguous, all modem memory contiguous, each
  sharing window its own aligned block.

  This decision is made at SoC memory-map definition time,
  which is roughly 18 months before anyone writes the XPU
  driver. Bring the security requirements to that meeting or
  you will spend the project working around its output.
```

Steps 6 and 7: one source of truth

```
# The same policy must appear in at least four places:
#   the firmware's XPU tables
#   the firmware's TZASC/TZC configuration
#   the device tree's reserved-memory nodes
#   the SMMU stream table setup
# and often a fifth: the modem or DSP's own linker script.
#
# Hand-maintaining four copies guarantees they will drift. So
# write the policy ONCE, as data, and generate.

# platform_policy.yaml
domains:
  secure:  { masters: [cpu_s, crypto] }
  ap:      { masters: [cpu_ns, dma0, dma1, usb, ufs] }
  modem:   { masters: [mdsp, mdma] }
  media:   { masters: [gpu, display, venc, vdec] }
  sensor:  { masters: [shub] }

regions:
  - name: ns_dram
    base: 0x80000000
    size: 0x30000000
    access: { ap: rw, media: rw, secure: rw }
    dt_node: false                # the kernel owns it
  - name: ap_modem_shm
    base: 0xB0000000
    size: 0x00400000
    access: { ap: rw, modem: rw, secure: rw }
    dt_node: { name: modem_shm, no_map: false }
  - name: modem_priv
    base: 0xC0000000
    size: 0x08000000
    access: { modem: rw, secure: rw }
    dt_node: { name: modem_mem, no_map: true }
  - name: secure_dram
    base: 0xCE000000
    size: 0x02000000
    access: { secure: rw }
    dt_node: { name: secure_mem, no_map: true }
    lock: true

# Then one generator emits:
#   xpu_policy.c        the firmware table
#   tzc_regions.c       the TZASC configuration
#   reserved-memory.dtsi  the DT nodes
#   policy_test.c       the negative-test list
#   policy.md           a human-readable matrix for review
#
# Now a change is made in ONE place, and the four artifacts
# cannot disagree. This is the single highest-value thing in
# this chapter, and it is a two-hundred-line Python script.
```

Step 8: test the negatives

```
/* Positive tests - "the driver works" - happen anyway. They
   tell you nothing about the policy, because a policy of
   "allow everything" passes all of them.

   NEGATIVE tests are the only ones that measure isolation. */

struct denial_test {
        const char *name;
        uint32_t    master;
        uint64_t    addr;
        bool        write;
};

static const struct denial_test must_fail[] = {
  { "AP reads secure DRAM",        M_CPU_NS, SECURE_DRAM,   false },
  { "AP writes secure DRAM",       M_CPU_NS, SECURE_DRAM,   true  },
  { "AP reads modem private",      M_CPU_NS, MODEM_PRIV,    false },
  { "modem reads AP DRAM",         M_MODEM,  NS_DRAM,       false },
  { "modem reads framebuffer",     M_MODEM,  FRAMEBUFFER,   false },
  { "media writes secure DRAM",    M_GPU,    SECURE_DRAM,   true  },
  { "AP reads protected video",    M_CPU_NS, PROT_VIDEO,    false },
  { "AP writes fuse controller",   M_CPU_NS, FUSE_CTRL,     true  },
  { "AP writes XPU config",        M_CPU_NS, XPU_BASE,      true  },
  { "sensor reads AP DRAM",        M_SHUB,   NS_DRAM,       false },
  { "display writes DRAM",         M_DISPLAY,NS_DRAM,       true  },
  { "unlisted addr, any master",   M_CPU_NS, UNMAPPED_GAP,  false },
};

/* HOW to issue a transaction as a non-CPU master: program that
   master's own DMA to do a small transfer, and check for the
   error. Most subsystems have a test or loopback mode; if not,
   the smallest real operation will do. This is genuine work to
   build, and it is the part that separates a policy you HAVE
   from a policy you KNOW.                                      */

/* RUN THESE:
     · in CI, on real silicon, every night
     · on the FINAL lifecycle state, not a dev part
     · after every firmware change
     · after every DT change
     · and specifically after any "temporary" bring-up widening

   A denial test that starts passing when it should fail is the
   highest-priority bug in the build. Wire it to fail the
   pipeline.                                                  */
```

Step 9: review

> **WHAT A REVIEWER SHOULD BE ABLE TO ASK**
>
> - "Show me the master list, and how you know it is complete."
> - "Which masters are not behind any SMMU or XPU, and why is that acceptable?"
> - "Point at the reason for this `rw`. What breaks if it becomes `r`?"
> - "Which regions are unlocked at runtime, who can change them, and what stops the normal world asking?"
> - "Where is the window between reset and enforcement, and which masters run inside it?"
> - "Show me the negative test for this empty cell, and the CI run where it passed."
> - "Which of these four artifacts is generated, and which is hand-written?"
>
> If any of those questions has no answer, that is the finding — not a comment on the code.

Living with a policy

```
/* Policies rot. Specifically:

   · BRING-UP WIDENING. Someone opens a region to make a new
     block work at 2 a.m., intending to narrow it later.
     -> require every widening to carry an expiry and a ticket;
        fail the build after the date.

   · DERIVED PRODUCTS. A cheaper variant drops the modem, and
     nobody removes its domain - leaving an allow for a master
     that no longer exists, or worse, whose ID has been reused
     by a different block.
     -> regenerate from the policy file per product; never copy
        the generated C.

   · NEW MASTERS mid-project. A block is added; the policy is
     not updated; it works because the default was never
     tightened.
     -> that is why the default is DENY. A new master that
        nobody told you about fails loudly on day one.

   · DEBUG BUILDS diverging. The dev configuration is permissive
     and the production one is not, so the negative tests only
     ever run against the permissive one.
     -> run the denial suite against the PRODUCTION policy on
        production silicon.                                  */
```

How it shows up

Symptom	Cause
A new subsystem faults on everything	Its master ID is not in any region — correct behaviour of a deny-by-default policy
Policy needs more regions than the hardware has	Memory map not grouped by domain; regroup, do not compromise the policy
Buffers fail only at certain sizes or addresses	Crossing a region boundary; alignment does not match the granule
Firmware and DT disagree after a change	Hand-maintained copies — generate them

Symptom	Cause
Isolation works on the dev board, not on production	Different policy build, or lifecycle state differences
An allow nobody can explain	Bring-up widening that was never removed
A master reaches memory the matrix forbids	Master ID shared or remapped — the matrix is describing something the hardware cannot distinguish
Negative tests all pass, but so does everything	The XPU was never enabled — verify `XPU_CTRL`
Corruption during early boot only	A master released from reset before enforcement began

Check yourself

- ✓ List the nine steps and say which two are usually skipped.
- ✓ Enumerate the bus masters of a device you have worked on — all of them.
- ✓ Explain why an unlisted master defeats the policy regardless of its quality.
- ✓ Build an access matrix with a reason in every cell, and read its empty cells aloud as security properties.
- ✓ Explain why read and write must be separate decisions, with an example of each.
- ✓ Redesign an interleaved memory map so it fits sixteen regions.
- ✓ Explain why gaps between domains are worth keeping.
- ✓ Name the four artifacts that must agree, and how to make them agree by construction.
- ✓ Write ten negative tests for a system you know, and say how you would issue each one.
- ✓ List four ways a policy rots and the control that stops each.

Debugging access-control failures

The last chapter of the track, and the one you will actually use. Something is being refused, or worse, silently ignored. Here is how to find out what, who, and why — starting from the two symptoms that look nothing like an access-control problem.

The two faces of a denial

```
LOUD: an abort.

   Linux:  "Synchronous External Abort" / "SError Interrupt"
   TF-A:   an EL3 crash dump
   DMA:    a channel error, a device timeout
   SMMU:   an event in the event queue (chapter 06)

   Annoying, but honest. You get an address and often a
   master.

SILENT: reads return 0, writes vanish.

   "The device isn't present."
   "The ID register reads 0x00000000."
   "DMA completes but the buffer is unchanged."
   "It works on the dev board."

   THIS is the one that costs a week. There is no error
   anywhere in the system, and every layer reports success.
   The driver author reasonably concludes the hardware is
   broken or the address is wrong.

THE TELL for the silent case:

   an MMIO read returning EXACTLY 0x00000000 or 0xFFFFFFFF
   from a register that should have a fixed non-zero value -
   an ID, a version, a revision - is an access problem until
   proven otherwise. Real hardware that is present and
   clocked almost never reads back all-zeros from its ID
   register.

   0x00000000  usually: RAZ/WI denial, or the block is in
               reset, or unclocked
   0xFFFFFFFF  usually: nothing decodes this address at all,
               or the bus returned an error that got
               swallowed
```

Triage: five questions in order

```
/* Work down this list. Each step is cheap and eliminates a
   large class.                                              */

1. IS THE BLOCK POWERED, CLOCKED, AND OUT OF RESET?
      By far the most common cause of all-zero reads, and it is
      not an access-control problem at all.
      -> check the clock and reset drivers, /sys/kernel/debug/clk,
         and the power domain state. Enable the clock by hand
         and read again.

2. IS THE ADDRESS RIGHT?
      Compare the DT reg property against the TRM. Check you
      are reading the right instance - SoCs have four UARTs and
      three of them are wired to nothing.
      -> devmem2 / busybox devmem from userspace, on a
         neighbouring register you know the value of.

3. IS THIS ACCESS DENIED?
      Now the interesting question. Read the XPU fault
      registers, the SMMU event queue, the TZASC fail status.
      -> details below.

4. WHICH MASTER, AND WHICH REGION?
      A fault register usually gives you both. If not,
      bisect - see "when there is no fault register".

5. IS THE POLICY WRONG, OR IS THE CODE WRONG?
      A denial is often CORRECT: the driver really is touching
      something it should not, using the wrong buffer, or
      running on the wrong master. Fix the code before you
      widen the policy. Widening is the last resort, not the
      first.                                                 */
```

> **THE INSTINCT TO RESIST**
>
> When a denial appears, the fastest way to make it go away is to open the region. That is almost always the wrong fix, and it is how policies rot (chapter 19). Ask first: *why is this master touching this address?* Half the time the honest answer reveals a bug — a stale pointer, a buffer allocated from the wrong pool, a DMA descriptor never updated — and the XPU has just done you a large favour.

Decoding a CPU abort

```
Unhandled fault: synchronous external abort (0x96000010)
  at 0xffff800008a01000
[0000000000000000] ESR = 0x96000010

/* ESR_EL1 = 0x96000010, decoded (ARM chapter 29):

     EC   = 0b100101 (0x25)  Data Abort, same EL
     ISV  = 0                no instruction syndrome
     DFSC = 0b010000 (0x10)  SYNCHRONOUS EXTERNAL ABORT,
                             not on a translation table walk

   DFSC 0x10 means: the MMU translated it fine, the transaction
   went out to the bus, AND SOMETHING OUT THERE SAID NO.

   That "something" is an XPU, a TZASC, a peripheral that does
   not decode the address, or a slave in reset. The CPU cannot
   tell you which - it only knows the bus said no.

   FAR_EL1 holds the virtual address. Convert it to physical:
        virt_to_phys(), or read the translation with AT S1E1R
        and TTBR-walk it, or just look at the ioremap that
        produced it.                                          */

/* SYNCHRONOUS vs ASYNCHRONOUS - and this matters a lot:

   Synchronous external abort (DFSC 0x10)
        the faulting instruction is in ELR. Device memory
        (nGnRnE/nGnRE) accesses fault synchronously, so MMIO
        denials point straight at the offending line.

   SError / asynchronous
        a WRITE to Normal memory was posted and completed long
        ago from the CPU's point of view. The abort arrives
        whenever the write finally reached the denier - which
        may be thousands of instructions later, in unrelated
        code, on a different task.
        -> the reported PC IS NOT THE CULPRIT. Do not spend a
           day analysing it.

   TAMING AN SError: insert `dsb sy` after the suspect writes to
   force them to complete, and the abort becomes synchronous
   enough to localise. Or bisect by disabling one master at a
   time.                                                      */
```

Reading XPU fault registers

```
/* Every XPU worth using latches the FIRST fault. Read it. */

void xpu_dump_fault(uintptr_t xpu)
{
        uint32_t st = mmio_read32(xpu + XPU_ERR_STATUS);
        uint64_t addr;

        if (!(st & XPU_ERR_VALID)) {
                INFO("xpu: no fault latched\n");
                return;
        }

        addr  = (uint64_t)mmio_read32(xpu + XPU_ERR_ADDR_HI) << 32;
        addr |= mmio_read32(xpu + XPU_ERR_ADDR_LO);

        ERROR("XPU FAULT\n"
              "  address   : 0x%016llx\n"
              "  master ID : 0x%02x (%s)\n"
              "  direction : %s\n"
              "  security  : %s\n"
              "  privileged: %s\n"
              "  multiple  : %s\n",
              addr,
              mmio_read32(xpu + XPU_ERR_MASTER),
              master_name(mmio_read32(xpu + XPU_ERR_MASTER)),
              (st & XPU_ERR_WRITE)  ? "write" : "read",
              (st & XPU_ERR_NS)     ? "non-secure" : "secure",
              (st & XPU_ERR_PRIV)   ? "privileged" : "unprivileged",
              (st & XPU_ERR_OVF)    ? "yes - more faults occurred"
                                    : "no");

        mmio_write32(xpu + XPU_ERR_CLEAR, 1);   /* re-arm */
}

/* THE OVERFLOW BIT MATTERS. Only the first fault is latched;
   later ones set overflow and are lost. So:

     · clear the fault as soon as you have logged it
     · if overflow is set, you are seeing one of many - fix
       that one, re-run, and look again
     · a repeating fault at 60 Hz is a display or DMA loop;
       one fault at boot is a probe.

/* WIRE THE INTERRUPT UP. An XPU fault interrupt handled in EL3
   that prints this block is worth more than any amount of
   after-the-fact analysis. Do it once, at the start of a
   project, and every subsequent access bug takes minutes.  */
```

The other reporters

Block	Where the evidence is
SMMU	The event queue: StreamID, SubstreamID, IOVA, fault reason. See chapter 06 — richest source on the SoC.
TZC-400	`FAIL_ADDRESS_LOW/HIGH`, `FAIL_CONTROL` (direction, NS, privilege), `FAIL_ID` (master), per filter unit.
Interconnect	Some NoCs have error probes logging the initiator, target, opcode and a timestamp — check whether yours does before building a bisect harness.

Block	Where the evidence is
TF-A	Its exception handlers print an EL3 crash dump with all the ESR/FAR context.
Linux	`dmesg`: external aborts, SErrors, `arm-smmu` fault lines, `rcu` stalls if the secure world hung.
The DMA engine	Per-channel error status, often with the descriptor address that failed — which tells you the buffer, not just the address.

When there is no fault register

```
/* RAZ/WI denials with no logging. You have no evidence, so
   MANUFACTURE some.                                        */

/* TECHNIQUE 1: switch to error responses.
   Most blocks can be told to abort instead of ignoring, even
   if production ships quiet. One build flag turns a
   week-long mystery into a stack trace.                    */
        mmio_write32(xpu + XPU_CTRL, ctrl | XPU_CTRL_ERR_RESP);

/* TECHNIQUE 2: probe from a known-good master.
   Read the same address from EL3 firmware, which is usually
   allowed everything. If EL3 sees the right value and Linux
   sees zeros, it is a policy question, not a hardware one. */

/* TECHNIQUE 3: bisect the policy.
   Open regions one at a time until it works. The one that
   fixes it names the resource; the failing master follows from
   which driver you were exercising. Crude, always works.   */

/* TECHNIQUE 4: bisect the masters.
   Hold subsystems in reset, or disable their DT nodes, one at
   a time. When the corruption stops, the last one you disabled
   is the culprit. This is the standard way to find a rogue
   DMA writer with no XPU at all.                           */

/* TECHNIQUE 5: a canary region.
   Fill a page with a pattern, deny everyone, and scan it
   periodically. If it changes, something is writing where it
   must not - and now you know it exists, which is most of the
   battle.                                                  */

/* TECHNIQUE 6: read a neighbour.
   If register 0x00 reads zero, read 0x04, 0x08, an adjacent
   block's ID register. All zero across a whole peripheral ->
   clock, reset or a whole-block denial. Only one register
   zero -> that register really is zero, and this was never an
   access problem.                                          */
```

A worked example

```
# SYMPTOM
#   New video decoder driver. probe() succeeds, but every
#   decode returns an empty frame. No errors anywhere.

# 1. clocked and out of reset?
   cat /sys/kernel/debug/clk/clk_summary | grep vdec
     vdec_core   1  1  0  300000000        -> yes, enabled

# 2. address right? read the ID register from userspace:
   devmem 0x1c000000
     0x00000000                            -> suspicious.
   devmem 0x1c000004
     0x00000000
   devmem 0x1c000008
     0x00000000                            -> the whole block.
#  TRM says ID should read 0x56444543. So: not present, not
#  clocked (but clk says it is), or DENIED.

# 3. any faults latched? From the TF-A debug console:
   tfa> xpu_dump 0
     XPU FAULT
       address   : 0x000000001c000000
       master ID : 0x04 (CPU_NS)
       direction : read
       security  : non-secure
     -> DENIED. The CPU, non-secure, may not read the decoder's
        registers.

# 4. why? Check the policy:
   grep -A4 vdec platform_policy.yaml
     - name: vdec_regs
       base: 0x1C000000
       access: { secure: rw }
     -> the decoder was classified as a SECURE resource,
        because an earlier product used it for DRM.

# 5. policy wrong, or code wrong?
#  Talk to the team. Conclusion: this product decodes
#  non-protected content on the AP, so the AP legitimately
#  needs the registers - but the PROTECTED OUTPUT BUFFER must
#  still be unreadable by the AP. So the fix is to split the
#  resource, not to open it:

     - name: vdec_regs
       base: 0x1C000000
       access: { ap: rw, secure: rw }       # control plane
     - name: vdec_protected_out
       base: 0xC8000000
       access: { media: w, secure: rw }     # data plane:
                                            # codec writes,
                                            # AP never reads

# 6. regenerate, rebuild firmware, re-run.
#    AND add the negative test:
     { "AP reads protected video output", M_CPU_NS,
       PROT_VIDEO, false },
#    so the day someone widens this by accident, CI says so.
```

> **NOTICE WHAT STEP 5 DID**
> The lazy fix — `access: { ap: rw }` on the whole subsystem — would have worked, shipped, and quietly handed the application processor read access to protected video. Splitting control plane from data plane took ten extra minutes and preserved the property the policy existed for. That distinction, between "make the error go away" and "understand what the error was protecting", is the whole skill.

The toolkit

Tool	Use
`devmem` / `devmem2`	Poke physical addresses from userspace. First thing to reach for.
`dmesg`, `/sys/kernel/debug/clk/clk_summary`	Aborts, SMMU faults, clock and reset state.
TF-A debug console	Read XPU/TZC registers from EL3, where everything is allowed.
JTAG	Halt and inspect — but remember `SPIDEN` gating (ch. 10).
Bus trace / NoC probes	Ground truth on who issued what, if your SoC has them.
A logic analyser on the reset and clock lines	When the software's view of power state is not believable.
The negative test suite	Not a debugging tool — a tool for knowing the bug never comes back.

How it shows up

Symptom	Likely cause	First check
Whole block reads 0x00000000	Unclocked, in reset, or wholly denied	Clock summary, then XPU fault register
Whole block reads 0xFFFFFFFF	Address decodes to nothing	DT `reg` against the TRM
Synchronous external abort, DFSC 0x10	Bus-level denial on MMIO	FAR, then the XPU/TZC fail address
SError at an unrelated instruction	A posted write denied long after issue	`dsb sy` to localise, then bisect masters
DMA "succeeds", buffer unchanged	Writes ignored	Switch the block to error responses
Works from EL3, not from Linux	Policy denies the non-secure master	The access matrix
Works for small buffers only	Larger buffer crosses a region boundary	Region base and limit vs the allocation
Intermittent, allocation-dependent	Same, but only some allocations cross	Print the physical address on failure
Fault overflow bit set	Many faults; you are seeing one	Fix, clear, re-run, look again
Random corruption, no attribution	A master with no XPU coverage at all	Canary region, then master bisection
Fails only on production silicon	Lifecycle state; debug gating; a tighter production policy	Compare the two policy builds directly

Check yourself

- ✓ Name the two faces of a denial and say which costs more time.
- ✓ Explain what an all-zero ID register read tells you, and what it does not.
- ✓ Run the five triage questions in order on a real symptom.
- ✓ Decode `ESR = 0x96000010` and say what it does and does not identify.

- ✓ Explain why an SError's reported PC is not the culprit, and how to localise it.
- ✓ Write a fault-register dump, and explain why the overflow bit matters.
- ✓ Give three techniques for finding a denial with no fault register.
- ✓ Work the video decoder example and explain why splitting the resource beat widening it.
- ✓ Say what you would add to a new project on day one to make all of this take minutes.

END OF THE ISOLATION AND SECURITY TRACK

You have gone from the threat model of an SoC, through the SMMU, TrustZone, the TEE stack, secure boot, real attacks, and access control, to debugging the whole thing. Next comes Linux kernel and drivers, where much of this becomes concrete: DMA APIs against a real SMMU, reserved memory against a real policy, and a driver you write yourself.

TRACK

Virtualisation & hypervisors

EL2, stage-2 translation, the vGIC and virtio — how one machine convincingly pretends to be several, and what that costs.

21 chapters

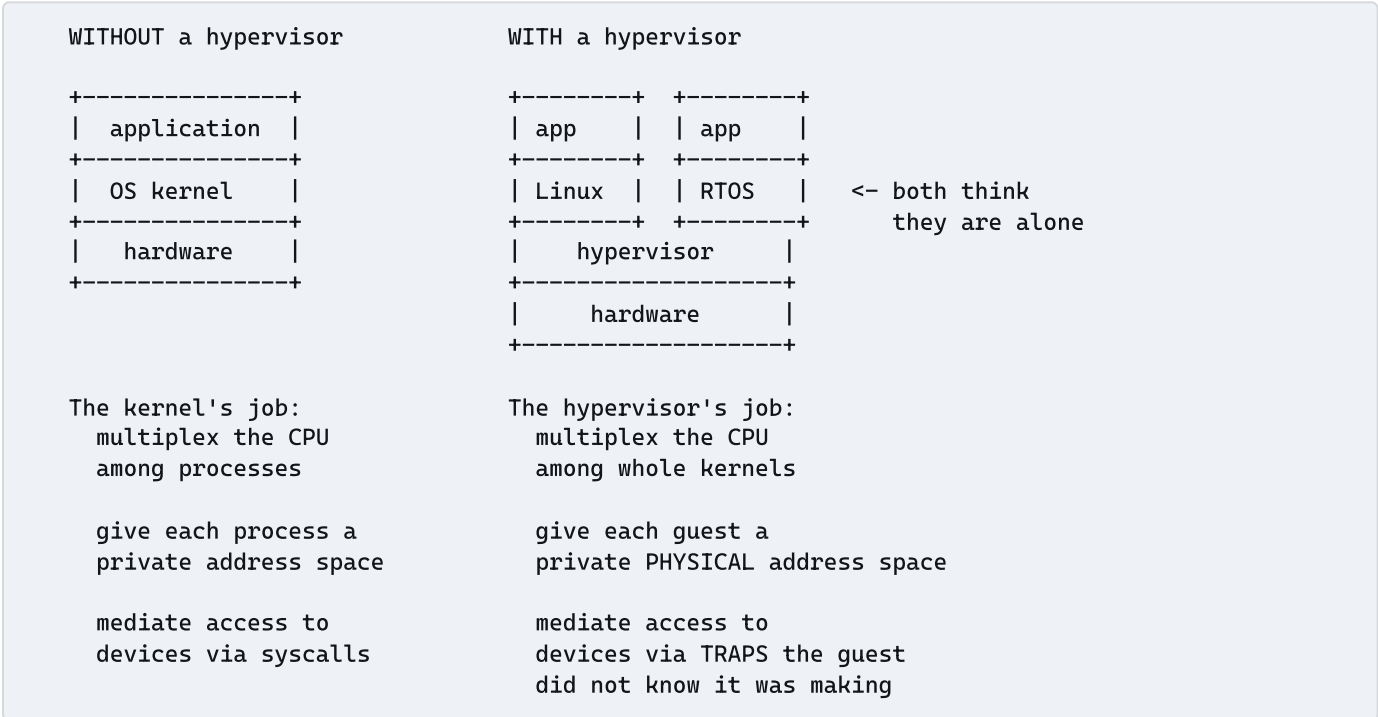
What a hypervisor actually is

A hypervisor is an operating system whose processes are operating systems. Everything else in this track — EL2, stage-2 translation, the vGIC, virtio — is a consequence of that one sentence, because a process that believes it owns the machine needs lying to in ways an ordinary process does not.

The problem it solves

An operating system is written on the assumption that it is alone. It sets up page tables at a physical address it chose. It programs the interrupt controller. It reads a timer and believes the value. It writes to a UART at `0x0900_0000` because the manual says the UART is there.

Now run two of them on one chip. Both want to be alone. Both chose physical addresses. Both programmed the interrupt controller. Neither will tolerate the other, and neither can be modified — that is the whole point, because if you were willing to rewrite the guest you would not need virtualisation, you would need a library.

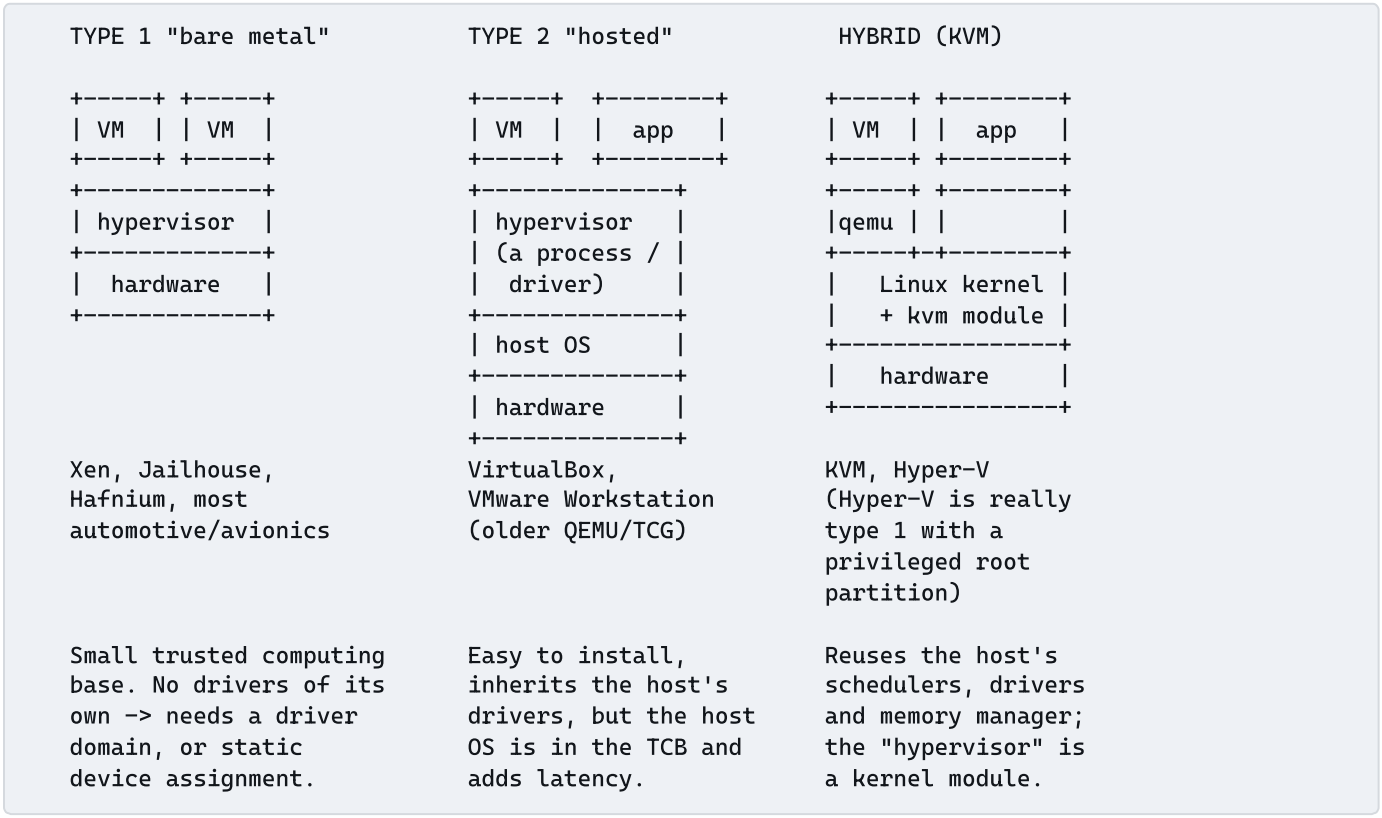


That last row is the interesting one. A process asks for a service by executing `SVC`: it knows it is asking. A guest kernel writes to what it believes is a hardware register: it does not know it is asking. The hypervisor must intercept an operation the guest thought was a plain memory store, work out what the guest meant by it, do something equivalent, and resume the guest as if the store had simply happened.

> **THE DEFINITION WORTH MEMORISING**
>
> A hypervisor gives each guest the *illusion of a whole machine*: its own physical address space, its own CPU, its own interrupt controller, its own timer, and its own devices. Every chapter in this track is one of those illusions, and how the hardware helps you build it.

Type 1 and type 2

The textbook split is between hypervisors that run on bare metal and hypervisors that run inside a host OS. It is a useful first cut and a poor final one, because the interesting systems sit in between.



KVM is the case that breaks the taxonomy, and it is the one you will meet most. The Linux kernel gains the ability to enter EL2 and run a guest; a user-space process (QEMU, crosvm, cloud-hypervisor) supplies the device models and the policy. The scheduler scheduling your vCPU is the ordinary Linux scheduler, and the vCPU is an ordinary thread. That design decision — *a vCPU is a thread* — explains most of KVM's behaviour, good and bad.

Three ways to virtualise

Technique	How	Cost	Where you see it
Trap and emulate	Run the guest directly; privileged operations trap to the hypervisor, which emulates them	One trap per privileged operation — cheap if rare, ruinous if hot	The base mechanism everywhere
Binary translation	Rewrite the guest's instruction stream before running it, replacing awkward instructions	Translation time, cache pressure, enormous complexity	VMware on pre-2006 x86; QEMU's TCG today
Paravirtualisation	Modify the guest to call the hypervisor deliberately	You must be able to change the guest	virtio, Xen PV, PSCI, pvclock, KVM's hypercalls

Real systems use all three at once. An arm64 KVM guest runs natively (trap-and-emulate) for ordinary code, is trapped for GIC and timer access, and deliberately calls the hypervisor for power management via PSCI and for I/O via virtio. Paravirtualisation is not a compromise you make when the hardware is weak — it is the right answer whenever the interface you are emulating is a bad interface. Emulating a 1990s IDE controller register-by-register is possible and pointless; a virtqueue is better in every way.

> **WHY VIRTIO EXISTS AT ALL**
> Emulating a real device means one trap per register access, and real device drivers touch a lot of registers per operation. A virtio device is designed so that the guest can queue many requests into shared memory and then trap *once* to say "there is work". The unit of expensive interaction becomes the batch, not the register. Hold that thought — it recurs in chapter 13.

Why this matters in embedded, not just in the cloud

Server virtualisation is about consolidation and billing. Embedded virtualisation is about *isolation with mixed criticality*, and the shape of the problem is different.

```
A typical automotive or industrial SoC today:

+---------------------+  +---------------+  +--------------+
|  Infotainment       |  |  Instrument   |  | Safety /     |
|  Android or Linux   |  |  cluster      |  | control      |
|  huge, updated often|  |  Linux, must  |  | RTOS, ASIL-B |
|  low trust          |  |  never blank  |  | hard realtime|
+---------------------+  +---------------+  +--------------+
+-------------------------------------------------------------+
|  hypervisor - and its job is NOT consolidation, it is:      |
|    * the RTOS must meet its deadline even when Android melts |
|    * a compromise in Android must not reach the CAN bus      |
|    * the cluster must keep rendering during an infotainment  |
|      restart                                                 |
+-------------------------------------------------------------+

Consequences you will not meet in a cloud hypervisor:
  - static partitioning is often preferred over scheduling
  - device PASSTHROUGH is the norm, not the exception
  - the hypervisor must be small enough to CERTIFY
  - "overcommit" may be forbidden outright
```

This is why the track covers Jailhouse and static partitioning alongside KVM. In a data centre a hypervisor that cannot overcommit memory is useless; in a car a hypervisor that *can* is a hazard.

What virtualisation actually costs

The honest answer is "almost nothing, except at the boundaries, where it is enormous". Guest instructions that do not trap run at native speed — there is no interpreter and no overhead. The cost is concentrated entirely in exits.

```
Rough order-of-magnitude figures on modern arm64 server silicon.
Treat them as ratios, not measurements: measure your own platform
(chapter 21).

  a normal guest instruction ..............  same as native
  a stage-2 TLB hit .......................  same as native
  a stage-2 page-table walk ...............  up to 24 memory accesses
                                             (4 stage-1 levels x
                                              4 stage-2 levels + leaf)
  a trap to EL2 and back (VHE, no work) ....  ~1000-2000 cycles
  an MMIO exit handled inside KVM ..........  ~2-5 us
  an MMIO exit handled by QEMU in user space  ~10-30 us
  a virtio notification + vhost handling ....  ~5-15 us

The lesson: an exit costs roughly a thousand instructions you did
not write. A driver that polls a status register in a tight loop
runs at native speed on hardware and at a few thousand operations
per second in a VM.
```

> **THE CLASSIC EMBEDDED SURPRISE**
>
> Code that "works fine on the board" and is unusably slow in the VM is almost always doing register polling. On hardware each read is ~100 ns; under a hypervisor each read is a trap. The fix is not a faster hypervisor — it is to stop polling, or to pass the device through (chapter 16).

What the rest of this track builds

Illusion	Hardware that provides it	Chapters
A private physical address space	Stage-2 translation	05, 06, 07
A private CPU	EL2, banked registers, the world switch	03, 04, 08, 09
A private interrupt controller	The GIC virtual CPU interface, list registers	10, 11
A private timer	`CNTVOFF_EL2` and the virtual counter	12
Devices	MMIO traps, virtio, the SMMU for passthrough	13, 14, 15, 16
All of it, assembled	KVM, Xen, Jailhouse, Hafnium	17, 18, 19, 20

You need exception levels, translation tables and the GIC before this track will land. If any of those three is shaky, read them first; virtualisation is those three ideas applied a second time, one level up.

How it shows up

Symptom	Cause
Guest boots but is 100x slower than the host	Emulation instead of hardware virtualisation — you are on TCG, not KVM. Check for `-accel kvm` and `/dev/kvm`
A driver that works on hardware crawls in the VM	Register polling; every read is an exit
Guest reports the wrong number of CPUs or wrong memory size	The guest read the real hardware description instead of the one the hypervisor generated — device tree or ACPI table not overridden
Guest hangs immediately after "Starting kernel"	No console: the UART the guest expects is not emulated at the address its device tree claims

Symptom	Cause
Two guests both work alone, neither works together	Overlapping passthrough or a shared device neither expects to share
RTOS misses deadlines only when Linux is busy	The hypervisor is time-slicing where it should be partitioning (chapter 18)
`KVM: entry failed, hardware error`	The CPU refused to enter the guest — usually an invalid stage-2 configuration or an unsupported feature bit

Check yourself

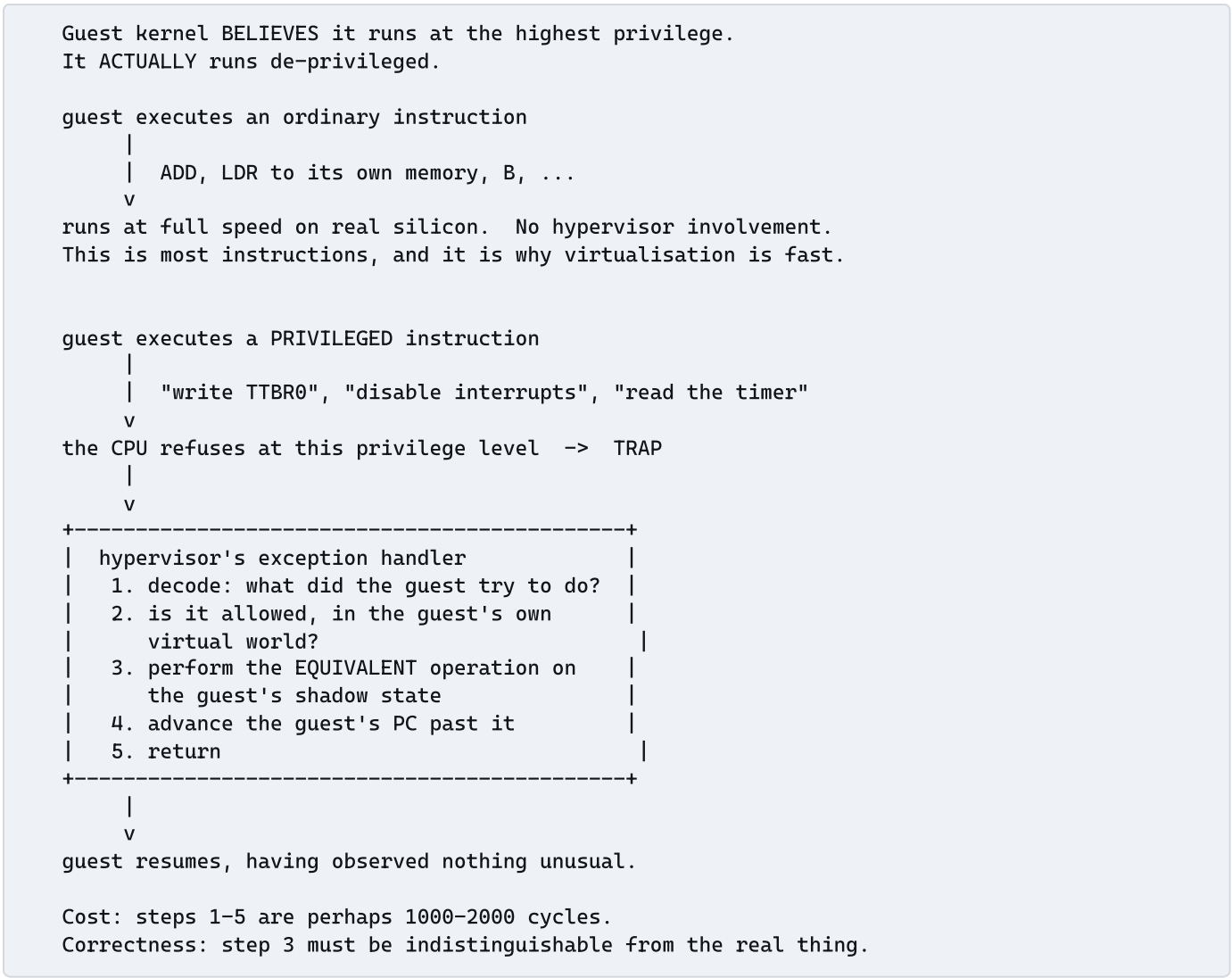
- ✓ Explain in one sentence what a hypervisor provides that an OS kernel does not.
- ✓ Give the difference between a process asking for a service and a guest asking for one.
- ✓ Place KVM, Xen, Jailhouse and VirtualBox on the type-1/type-2 spectrum, and say why KVM is awkward to place.
- ✓ Name the three virtualisation techniques and give a live example of each on an arm64 Linux guest.
- ✓ Explain why paravirtualised I/O is usually better than faithful device emulation, in terms of exits.
- ✓ State two requirements an automotive hypervisor has that a cloud hypervisor does not.
- ✓ Explain why a polling driver is fast on hardware and slow in a VM.
- ✓ List the five illusions a hypervisor must maintain and name the hardware feature behind each.

Trap-and-emulate and the Popek-Goldberg conditions

In 1974 two researchers wrote down the exact property an instruction set must have to be virtualisable. For thirty years most popular architectures failed the test, and the industry built increasingly baroque workarounds. Understanding the test tells you precisely what EL2 was added to fix.

The mechanism, stripped down

Classical virtualisation is one idea: **run the guest at a privilege level lower than it thinks it has, and catch it when it notices.**



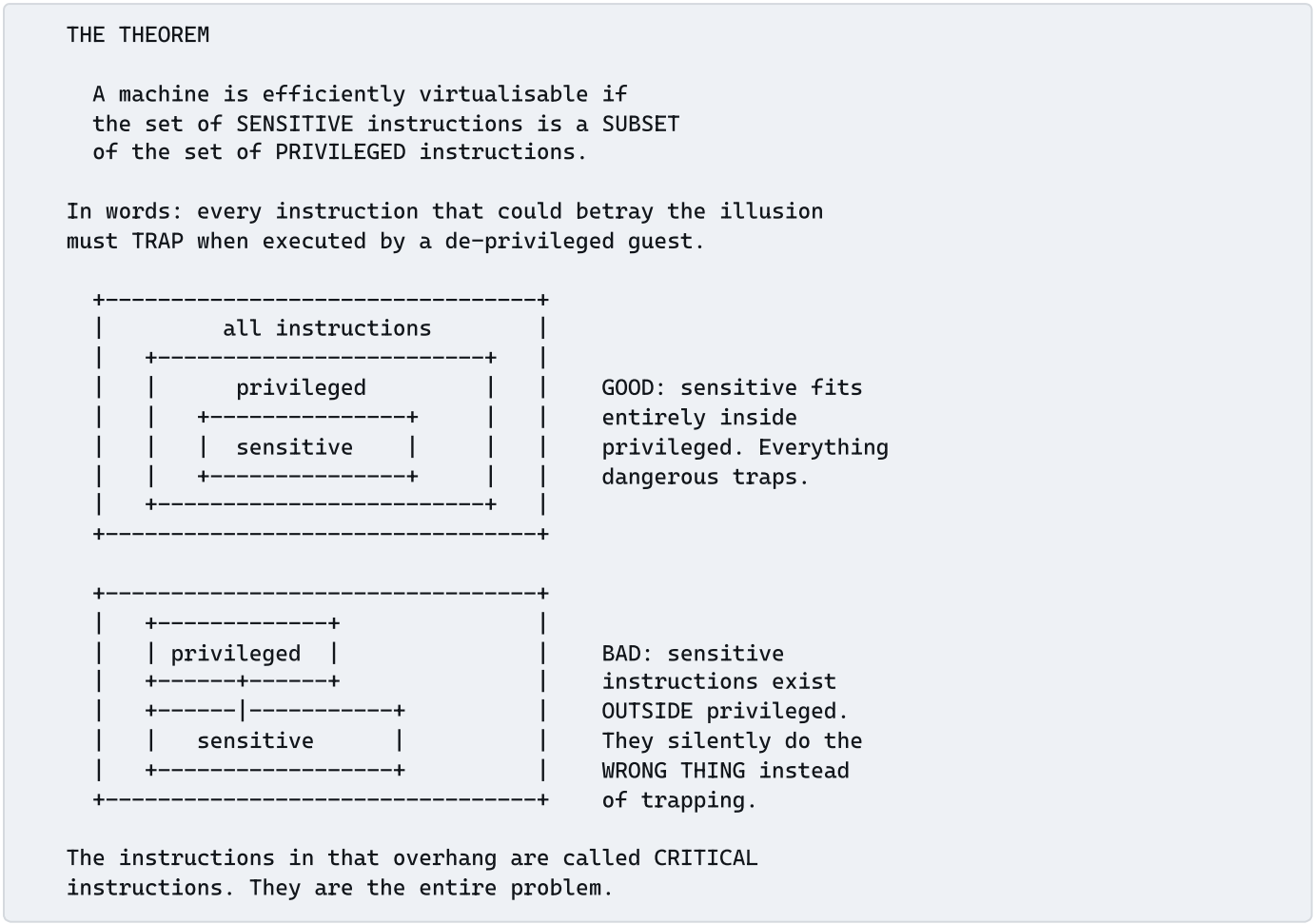
THE INVARIANT

The guest must not be able to tell. Not "must not be able to escape" — *must not be able to tell*. Any observable difference between running natively and running virtualised is a bug, and historically most of them were also security holes.

Popek and Goldberg's instruction classes

The 1974 paper divides every instruction into categories based on two questions: does it touch privileged state, and does its behaviour depend on privileged state?

Class	Definition	AArch64 examples
Privileged	Traps when executed in user mode, works in supervisor mode	`MSR TTBR0_EL1`, `TLBI`, `AT`, `ERET`
Sensitive — control	Changes the configuration of resources: memory mapping, privilege, interrupt state	`MSR SCTLR_EL1`, `MSR DAIFSet`, `MSR VBAR_EL1`
Sensitive — behaviour	Its *result* depends on the privilege level or on real resource configuration	`MRS CurrentEL`, `MRS MIDR_EL1`, reading a physical counter
Innocuous	Everything else	`ADD`, `LDR`, `B`, floating point



Where architectures failed

x86 before VT-x had 17 critical instructions. The canonical one is `POPF`, which pops the flags register. In kernel mode it can change the interrupt-enable flag; in user mode it silently ignores that bit rather than faulting. A de-privileged guest kernel executing `POPF` to disable interrupts would get *no trap and no effect*. The hypervisor never learns the guest wanted interrupts off, and the guest never learns it failed.

Classic ARM (ARMv7-A without the Virtualization Extensions) had its own version of the problem, and it is worth knowing because you meet it on old parts:

```
/* ARMv7-A, no VE. The guest kernel is de-privileged to USR mode. */

    MRS  r0, CPSR        /* reads the mode field. In USR mode the
                            mode bits read back as USR - so a guest
                            that inspects its own mode discovers it
                            is not in SVC. It can TELL. */

    MSR  CPSR_c, #0xD3   /* attempt to switch to SVC and mask IRQs.
                            In USR mode: silently ignored. No trap.
                            The guest believes interrupts are off.
                            They are not. */

    LDM  sp, {r0-r14}^   /* the "^" form accesses USR-mode banked
                            registers - meaningless and unfaulting
                            when you are already in USR mode. */

/* Three sensitive-but-not-privileged instructions, and therefore
   three ways for the illusion to break silently. This is why Xen
   on ARMv7 required PARAVIRTUALISATION: you had to patch the guest
   kernel to replace every critical instruction with a hypercall.  */
```

The three historical workarounds

Approach	How it dodges critical instructions	Price
Binary translation	Scan the guest's instruction stream ahead of execution and rewrite critical instructions into traps or inline emulation	A translator and a translation cache in the trusted computing base; self-modifying guest code is a nightmare
Paravirtualisation	Change the guest source so it never emits a critical instruction — it calls the hypervisor instead	Every guest OS must be ported and maintained; closed guests cannot run at all
Hardware extension	Add a new mode in which the awkward instructions trap	None, once the silicon exists. This is what won

> **WHY "SHADOW PAGE TABLES" WERE THE REAL PAIN**
>
> Even with the instruction problem solved, memory remained hard. The guest builds page tables mapping guest-virtual to *guest-physical* addresses, but the MMU consumes host-physical. Pre-hardware solutions had the hypervisor maintain a hidden *shadow* table composing the two, write-protect the guest's tables to detect edits, and rebuild on every change. It worked, and it cost a fortune on fork-heavy workloads. Stage-2 translation (chapter 5) is the hardware answer, and it is the single biggest reason modern virtualisation is cheap.

How ARMv8-A passes the test

ARM did not try to make EL1 instructions trap by de-privileging the guest into EL0. That was the ARMv7 mistake. Instead it added a *higher* level, EL2, and made trapping **configurable**: the guest genuinely runs at EL1, with real EL1 registers, and the hypervisor selects which EL1 operations should divert to EL2.

```
   ARMv7 approach (broken)        ARMv8 approach (works)

   EL0  app                       EL0  app
        guest kernel  <- squashed
                         in here  EL1  guest kernel, genuinely at EL1
   SVC  hypervisor                     with its own SCTLR_EL1, TTBR0_EL1,
                                       VBAR_EL1 ... all real registers

                                  EL2  hypervisor, and it holds the
                                       CONTROL: HCR_EL2 says which
                                       EL1 operations trap up to here

   Consequence: the guest is NOT de-privileged, so it cannot
   detect de-privileging. Sensitive instructions do not need to
   be privileged - they need to be TRAPPABLE, and HCR_EL2 makes
   almost all of them trappable on demand.

   MRS CurrentEL in a guest reads "EL1" - which is TRUE.
   The lie is not about privilege. The lie is about being alone.
```

This is the key architectural insight of the ARM virtualisation extensions, and it is why HCR_EL2 is the register this whole track revolves around. Popek and Goldberg asked for sensitive ⊆ privileged. ARM instead offered *sensitive ⊆ configurably trappable*, which is strictly more useful: the hypervisor traps what it must emulate and lets the rest run at full speed.

Selective trapping in practice

```
/* Not everything should trap. The art is choosing.        */

  TRAP IT when the guest must not see real hardware state, or
  when you must maintain shadow state:

    guest writes to the GIC distributor      -> trap, emulate
    guest reads the physical counter         -> trap or offset
    guest executes WFI                       -> trap, so you can
                                                schedule another vCPU
    guest reads ID registers (MIDR, ID_AA64*)-> trap, present a
                                                sanitised view
    guest does SMC / HVC                     -> always to EL2/EL3

  DO NOT TRAP when the hardware already isolates it, because a
  trap you did not need is pure cost:

    guest writes TTBR0_EL1, TCR_EL1          -> let it. Stage-2
                                                already constrains
                                                the result.
    guest issues TLBI VAE1                   -> let it. VMID tags
                                                the entries.
    guest takes its own EL0 syscall          -> let it. Never
                                                involves EL2.
    guest sets DAIF                          -> let it, unless you
                                                are virtualising
                                                interrupts strictly.

/* A well-configured arm64 KVM guest running a compile job may
   execute BILLIONS of instructions between exits. A badly
   configured one exits thousands of times a second doing the
   same work.                                                 */
```

How it shows up

Symptom	Cause
Guest works alone, corrupts state when a second guest runs	A sensitive operation is not being trapped — the guest is touching real shared state
Guest detects it is virtualised despite your efforts	An ID register, timer or cache-topology value was not sanitised; the guest read the truth
Guest disables interrupts and still gets interrupted	Interrupt masking is being emulated and the emulation lost a state change — classic critical-instruction bug
Enormous exit rate for an idle guest	`WFI` / `WFE` trapping combined with a guest that spins, or an untrapped-but-should-be timer read
Guest hangs when it re-programs its own page tables	Under shadow paging, a missed write-protect; under stage-2, a stale TLB — see chapter 6
Emulated instruction executes twice	The handler forgot to advance the guest PC past the trapping instruction
Works on one core, breaks on SMP	Shadow state updated without the lock the real hardware provided implicitly

Check yourself

- ✓ State the Popek-Goldberg condition in one sentence, using the words sensitive and privileged.
- ✓ Define a critical instruction and explain why it is worse than an instruction that faults.
- ✓ Give one x86 and one ARMv7 critical instruction and say exactly what goes wrong.
- ✓ Name the three workarounds for a non-virtualisable ISA and one real product that used each.
- ✓ Explain why shadow page tables were expensive and what replaced them.
- ✓ Explain how ARMv8 sidesteps the theorem rather than satisfying it directly.
- ✓ Give two operations a hypervisor should trap and two it should deliberately not, with reasons.
- ✓ List the five steps of a trap-and-emulate handler, and say what breaks if you skip step 4.

EL2 and the ARM virtualisation extensions

EL2 is not simply "one more privilege level". It is a level with its own translation regime, its own vector table, its own timer, its own stack pointer and — crucially — a control register that decides which EL1 operations are allowed to complete without asking permission. This chapter is the anatomy.

Where EL2 sits

```
              NON-SECURE                    SECURE

EL0    application                   trusted application (S-EL0)
         |                                  |
EL1    guest kernel                   trusted OS, e.g. OP-TEE (S-EL1)
         |                                  |
EL2    HYPERVISOR                     S-EL2 partition manager
         |                            (ARMv8.4+, e.g. Hafnium)
         |                                  |
EL3    +-----------------------------------------+
       |  secure monitor - TF-A BL31              |
       |  the only code that switches WORLDS       |
       +-----------------------------------------+
                        |
                    hardware

Two orthogonal axes, and confusing them is the commonest
beginner error on this page:

  EXCEPTION LEVEL  = how much privilege   (EL0 < EL1 < EL2 < EL3)
  SECURITY STATE   = which world          (Non-secure | Secure | Realm)

EL2 is a level. TrustZone is a state. A hypervisor at EL2 and
a trusted OS at S-EL1 are not competing for the same seat;
they are in different worlds, and EL3 arbitrates.
```

> **REALMS, BRIEFLY**
> ARMv9 CCA adds a third security state (Realm) with its own R-EL2 running the Realm Management Monitor. The idea is a guest that even the hypervisor cannot read. It does not change anything in this chapter; it adds a column to the diagram. Chapter 19 covers the secure and realm sides together.

What EL2 actually gives you

Facility	Register(s)	Why it exists
Configurable trapping of EL1/EL0	`HCR_EL2`, `CPTR_EL2`, `HSTR_EL2`, `MDCR_EL2`	Choose what you emulate and what runs natively
A second stage of address translation	`VTTBR_EL2`, `VTCR_EL2`	Give the guest a private "physical" address space
A VM identity for the TLB	`VTTBR_EL2.VMID`	Tag TLB entries so switching guests does not require a full invalidate
Virtual interrupt injection	`HCR_EL2.{VI,VF,VSE}`, GIC list registers	Make an interrupt appear to the guest that no wire produced

Facility	Register(s)	Why it exists
Its own translation regime	`TTBR0_EL2`, `TCR_EL2`, `SCTLR_EL2`	The hypervisor's own mappings, invisible to the guest
Its own exceptions	`VBAR_EL2`, `ELR_EL2`, `SPSR_EL2`, `ESR_EL2`, `FAR_EL2`, `HPFAR_EL2`	Traps land here without disturbing the guest's own EL1 exception state
A timer of its own	`CNTHCTL_EL2`, `CNTHP_*_EL2`, `CNTVOFF_EL2`	Preempt a guest, and shift the guest's view of time
Guest-visible ID control	`VPIDR_EL2`, `VMPIDR_EL2`	Tell the guest a CPU ID and affinity of your choosing
A deliberate call gate	`HVC` instruction	Paravirtualised calls from EL1 into EL2

Register banking: the part that confuses everyone

System registers are *banked* per exception level. `TTBR0_EL1` and `TTBR0_EL2` are two different physical registers. The guest at EL1 writes `TTBR0_EL1` and the hypervisor's own `TTBR0_EL2` is untouched. This is why a guest can run with real EL1 registers without endangering EL2.

```
What is banked (separate copy per level):

  SCTLR_EL1  SCTLR_EL2      system control
  TTBR0_EL1  TTBR0_EL2      translation base
  TCR_EL1    TCR_EL2        translation control
  VBAR_EL1   VBAR_EL2       vector base
  ESR_EL1    ESR_EL2        syndrome
  FAR_EL1    FAR_EL2        fault address
  ELR_EL1    ELR_EL2        return address
  SPSR_EL1   SPSR_EL2       saved state
  SP_EL1     SP_EL2         stack pointer
  MAIR_EL1   MAIR_EL2       memory attributes

What is NOT banked and MUST be saved/restored by hand
on every world switch:

  X0–X30, SP_EL0, PC (via ELR)
  the FP/SIMD register file  (lazily, if you are careful)
  PMU counters, debug/breakpoint registers
  TPIDR_EL0, TPIDRRO_EL0, TPIDR_EL1

THIS asymmetry is exactly why the vCPU context switch
(chapter 8) has the shape it does: the banked half is free,
the unbanked half is a long list of MRS/MSR pairs.
```

> **WHY BANKING IS THE PERFORMANCE STORY**
>
> Because EL1 state is banked away from EL2 state, entering the hypervisor does not destroy the guest's configuration. A trap that the hypervisor can service without touching guest registers — reading `ESR_EL2`, updating a shadow value, returning — costs a few hundred cycles. A full vCPU switch costs thousands, because the unbanked list above must move. Minimising exits matters; minimising *full switches* matters more.

Getting into EL2 in the first place

You do not enter EL2 by asking. On a cold boot the CPU comes up at the highest implemented level (EL3 if TrustZone is implemented), and firmware *drops* into EL2 deliberately. If your bootloader dropped you straight to EL1, no hypervisor can run on that system, and no amount of software will fix it.

```
/* TF-A BL31 preparing to hand off. Simplified. */

    /* Decide the level the next image runs at. */
    scr  = SCR_EL3_RW          /* next level is AArch64      */
         | SCR_EL3_NS          /* next level is Non-secure   */
         | SCR_EL3_HCE;        /* HVC is ENABLED - without
                                  this, HVC is UNDEFINED and
                                  no guest can call EL2      */
    write_scr_el3(scr);

    spsr = SPSR_MODE_EL2h      /* <-- land at EL2, using SP_EL2 */
         | DAIF_ALL_MASKED;
    write_spsr_el3(spsr);
    write_elr_el3(bl33_entry);
    eret;                      /* now executing at EL2         */


/* Linux's own head.S then decides whether to STAY at EL2. */

    mrs     x0, CurrentEL
    cmp     x0, #CurrentEL_EL2
    b.ne    1f                 /* booted at EL1: no KVM, ever  */

    /* We are at EL2. Two choices:
         - stay here  -> VHE, host kernel runs AT EL2 (ch. 9)
         - drop to EL1-> nVHE, only a small stub stays at EL2 */
1:
```

```
# On a running Linux system, ask what happened:

$ dmesg | grep -iE 'kvm|hyp|EL2'
[    0.000000] CPU: All CPU(s) started at EL2
[    0.512430] kvm [1]: Hyp mode initialized successfully
[    0.512461] kvm [1]: VHE mode initialized successfully

# The bad version, and the commonest reason KVM "does not work":
[    0.000000] CPU: All CPU(s) started at EL1
[    0.498112] kvm [1]: HYP mode not available

# EL1 means the firmware or bootloader consumed EL2 and dropped
# you. On many boards this is a U-Boot or vendor-firmware setting;
# on some it is fused. No workaround exists in the kernel.

$ ls -l /dev/kvm            # exists only if EL2 was available
$ grep -c . /sys/module/kvm/parameters/* 2>/dev/null
```

> **THE SINGLE MOST COMMON BRING-UP FAILURE**
>
> `All CPU(s) started at EL1`. Every secondary core must also arrive at EL2, not just the boot core — a firmware that starts CPU0 at EL2 and the rest at EL1 produces a kernel that initialises KVM and then fails on the first vCPU pinned elsewhere. Check `CurrentEL` on every core, not one.

Discovering what the silicon supports

```
/* ID_AA64PFR0_EL1 - the feature register that matters here. */

  bits [11:8]  EL2 support
                 0b0000  EL2 not implemented -> no virtualisation
                 0b0001  EL2 implemented, AArch64 only
                 0b0010  EL2 implemented, AArch64 + AArch32

  bits [15:12] EL3 support (same encoding)

/* ID_AA64MMFR1_EL1 */

  bits [3:0]   VHE          0b0001 = ARMv8.1 VHE present (ch. 9)
  bits [7:4]   VMIDBits     0b0010 = 16-bit VMIDs, else 8-bit
  bits [15:12] HPDS, [19:16] HAFDBS ...

/* ID_AA64MMFR2_EL1 */
  bits [3:0]   CnP, [15:12] IESB, [55:52] E0PD ...

/* From Linux user space, no assembly needed: */
```

```
$ cat /proc/cpuinfo | grep -m1 Features
Features : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics
           fphp asimdhp cpuid asimdrdm lrcpc dcpop asimddp

$ ls /sys/devices/system/cpu/cpu0/regs/identification/
midr_el1  revidr_el1

# The VMID width decides how many VMs can be resident without
# TLB flushing. 8 bits = 255 usable; 16 bits = 65535.
# On a phone this never matters. On a 128-core server running
# hundreds of containers-in-VMs it very much does.
```

HVC: the deliberate call

`SVC` goes EL0→EL1. `HVC` goes EL1→EL2. `SMC` goes to EL3. They are three doors at three levels, and the calling conventions rhyme.

```
/* A guest asking its hypervisor for something. This is how PSCI,
   KVM's PV features and Xen's hypercalls all work.            */

    mov     x0, #0x84000001        /* PSCI CPU_ON, SMC32 range   */
    mov     x1, target_cpu_mpidr
    mov     x2, entry_point
    mov     x3, context_id
    hvc     #0                     /* trap to EL2                */
    /* x0 now holds the return value, per SMCCC                  */

/* Which door a PSCI call goes through depends on the guest:
     bare-metal Linux at EL1, TF-A at EL3   -> SMC
     Linux guest under KVM                  -> HVC, and KVM
                                               either handles it
                                               or forwards to EL3

   KVM tells the guest which to use through the device tree:

     psci {
         compatible = "arm,psci-1.0";
         method = "hvc";        <-- or "smc" on bare metal
     };

   Getting this wrong gives you a guest that hangs on secondary
   CPU bring-up: it calls SMC, the call sails past EL2 to EL3,
   and firmware tries to power on a PHYSICAL core for a VIRTUAL
   request.                                                    */
```

How it shows up

Symptom	Cause
`kvm: HYP mode not available`	Booted at EL1 — firmware consumed EL2. Not fixable from the kernel
KVM initialises, then fails on some vCPUs	Secondary cores were started at EL1 while the boot core reached EL2
`HVC` takes an undefined-instruction exception	`SCR_EL3.HCE` is clear — EL3 firmware never enabled the door
Guest hangs bringing up secondary CPUs	PSCI conduit mismatch: guest device tree says `smc` where it should say `hvc`
Random guest corruption after a vCPU migrates cores	An unbanked register (FP state, TPIDR, PMU) not saved or restored in the switch
Only a handful of VMs before performance collapses	8-bit VMIDs and rollover forcing global TLB invalidation
Hypervisor faults with `ESR_EL2` pointing at its own code	EL2's own translation regime (`TTBR0_EL2`) is misconfigured — not a guest bug at all
Guest reads the host's real MIDR and mis-detects the CPU	`VPIDR_EL2` / `VMPIDR_EL2` not programmed for the vCPU

Check yourself

- ✓ Draw the four exception levels against the security states, and place a hypervisor and a trusted OS correctly.
- ✓ Name six facilities EL2 provides that EL1 cannot.
- ✓ Explain register banking and list four registers that are banked and four that are not.
- ✓ Explain why banking makes a lightweight trap much cheaper than a full vCPU switch.
- ✓ Describe how a CPU comes to be executing at EL2, and what `SCR_EL3.HCE` controls.

- ✓ Given `All CPU(s) started at EL1`, say what is wrong and where the fix must go.
- ✓ Read `ID_AA64PFR0_EL1` and determine whether EL2 is implemented.
- ✓ Explain what VMID width affects in practice.
- ✓ Compare `SVC`, `HVC` and `SMC`, and explain the PSCI conduit property.

HCR_EL2: choosing what traps

One 64-bit register decides, bit by bit, which operations a guest may perform for itself and which must be brought to the hypervisor for adjudication. Every design decision about isolation, fidelity and performance in an ARM hypervisor is ultimately expressed here.

The idea

```
guest at EL1 executes:  MSR SCTLR_EL1, x0

     HCR_EL2.TVM == 0                HCR_EL2.TVM == 1
     ----------------                ----------------
     the write happens.              EXCEPTION to EL2.
     No exception. Full speed.       ESR_EL2.EC = 0x18
                                     (trapped MSR/MRS)
                                     ISS encodes the register,
                                     the direction and Rt.

                                     hypervisor decides what
                                     "writing SCTLR_EL1" should
                                     mean for THIS guest, then
                                     ERETs.

That is the entire mechanism. HCR_EL2 has about forty such
bits; each one moves a class of operation from "free" to
"mediated". Your hypervisor's personality is the bit pattern
you choose.
```

The bits that matter

Bit	Name	Set it to…
VM	Virtualisation enable	Turn on stage-2 translation. Without this, guest "physical" addresses are real physical addresses. The master switch
RW	Register width	1 = EL1 is AArch64, 0 = AArch32 guest
IMO / FMO / AMO	IRQ / FIQ / SError mask override	Route physical IRQ/FIQ/SError to EL2 instead of EL1, and enable the corresponding virtual interrupt
VI / VF / VSE	Virtual IRQ / FIQ / SError pending	Assert a virtual interrupt into the guest. Software writes the bit; the guest sees an interrupt
TWI / TWE	Trap WFI / WFE	Catch an idling guest so you can schedule someone else
TSC	Trap SMC	Intercept the guest's secure-monitor calls so EL2 can filter or forward them
TVM / TRVM	Trap virtual-memory register writes / reads	Catch `SCTLR_EL1`, `TTBR*_EL1`, `TCR_EL1`, `MAIR_EL1`… Needed for shadowing tricks; expensive, usually off
TTLB	Trap TLB maintenance	Catch guest `TLBI`. Usually off: VMID tagging makes guest TLBIs safe
TID0-TID3	Trap ID register groups	Present a synthetic CPU. TID3 is the one that hides features you have not implemented
TGE	Trap general exceptions	With VHE: EL0 belongs to the host, not a guest. Routes EL0 exceptions to EL2

Bit	Name	Set it to...
E2H	EL2 host	The VHE switch — remaps `*_EL1` accesses at EL2 onto EL2 registers. See chapter 9
PTW	Protected table walk	A stage-1 walk that hits device memory at stage 2 becomes a fault rather than undefined behaviour
FB	Force broadcast	Guest TLBI/cache ops become inner-shareable broadcasts, so the guest need not know it is on SMP
DC	Default cacheable	Force stage-1 to look cacheable while the guest's MMU is off — used during guest boot
TACR / TIDCP / TSW / TPC / TPU	Trap aux control, implementation-defined, cache ops	Cache maintenance and vendor registers; TSW in particular matters for set/way ops

What a real hypervisor sets

```
/* Close to what KVM/arm64 programs for a normal 64-bit guest.
   Compare arch/arm64/include/asm/kvm_arm.h.                  */

  HCR_GUEST_FLAGS =
        HCR_TSC          /* trap SMC: EL2 sees and filters them   */
      | HCR_TSW          /* trap set/way cache ops - a guest doing
                            "clean by set/way" would operate on
                            the WHOLE cache, including other VMs  */
      | HCR_TWE          /* trap WFE  -> yield the pCPU           */
      | HCR_TWI          /* trap WFI  -> block the vCPU thread    */
      | HCR_VM           /* stage-2 ON                            */
      | HCR_PTW          /* protected table walk                  */
      | HCR_TID3         /* trap ID regs: show a sanitised CPU    */
      | HCR_FMO | HCR_IMO /* physical FIQ/IRQ come to EL2, and
                             virtual FIQ/IRQ become available     */
      | HCR_AMO          /* same for SError                       */
      | HCR_RW           /* AArch64 guest                         */
      | HCR_FB           /* broadcast the guest's TLBI/cache ops  */
      | HCR_BSU_IS       /* barriers upgrade to inner-shareable   */
      | HCR_TACR         /* trap ACTLR_EL1                        */
      | HCR_TIDCP;       /* trap impl-defined registers           */

/* Notice what is DELIBERATELY ABSENT:

     no TVM   - the guest may write TTBR0_EL1/SCTLR_EL1 freely.
                Stage-2 makes that safe, and trapping every page
                table change would destroy performance.

     no TTLB  - the guest may invalidate its own TLB entries.
                VMID tagging means it can only hit its own.

     no TRVM  - reads of VM registers are harmless.

   Each absent bit is a decision that the HARDWARE already
   provides the isolation, so software should stay out of the
   way. That is the whole art.                                */
```

> **THE RULE OF THUMB**
>
> Trap when you must maintain state the guest cannot be allowed to own, or when the guest would otherwise observe the truth. Do not trap when stage-2 or VMID tagging already contains the damage. Every unnecessary trap bit is a permanent tax on every guest, for ever.

A worked example: why TSW exists

```
/* AArch64 cache maintenance comes in two flavours:      */

    DC CIVAC, x0     /* clean+invalidate by VIRTUAL ADDRESS.
                        Goes through stage 1 and stage 2, so it
                        can only ever touch memory this guest
                        actually owns.  SAFE - do not trap.   */

    DC CISW,  x0     /* clean+invalidate by SET AND WAY.
                        Addresses a physical cache location by
                        geometry. There is no address, so there
                        is nothing for stage 2 to constrain.

                        A guest looping over every set and way -
                        which is exactly what Linux does when it
                        turns its MMU off during boot or kexec -
                        would clean and invalidate lines
                        belonging to OTHER VMs and to the
                        hypervisor.                           */

/* Hence HCR_EL2.TSW. The guest traps, and the hypervisor does
   something equivalent-but-safe. KVM's historical answer was to
   flush the guest's own stage-2 mappings instead; modern KVM
   mostly just makes the operation a no-op after ensuring the
   guest's memory is coherent, because a guest with stage-2
   enabled has no legitimate need for set/way operations.

   This is the general shape of every trap handler you will
   write: the guest asked for something that means one thing on
   bare metal and something dangerous under virtualisation, and
   you must supply the SPIRIT of the request, not the letter. */
```

Decoding a trap when it arrives

```
/* Every trap to EL2 leaves the reason in ESR_EL2. */

    ESR_EL2[31:26]  EC   exception class
    ESR_EL2[25]     IL   instruction length (1 = 32-bit)
    ESR_EL2[24:0]   ISS  class-specific syndrome

  EC values you will meet constantly:

    0x01  trapped WFI/WFE                (HCR.TWI/TWE)
    0x03  trapped MCR/MRC cp15           (AArch32 guest)
    0x07  trapped SVE/FP/SIMD            (CPTR_EL2)
    0x16  HVC from AArch64               (deliberate hypercall)
    0x17  SMC from AArch64               (HCR.TSC)
    0x18  trapped MSR/MRS/system insn    (HCR.TVM, TID*, TACR...)
    0x20  instruction abort, lower EL    (stage-2 fetch fault)
    0x24  data abort, lower EL           (stage-2 data fault)

/* For EC=0x18 the ISS names the register precisely: */

    ISS[21:20] Op0   ISS[19:17] Op2   ISS[16:14] Op1
    ISS[13:10] CRn   ISS[9:5]   Rt    ISS[4:1]   CRm
    ISS[0]     Direction: 0 = write (MSR), 1 = read (MRS)

/* So a handler looks like: */

static int handle_sysreg_trap(struct vcpu *v)
{
        u64 esr  = read_sysreg(esr_el2);
        u32 iss  = esr & 0x1ffffff;
        int rt   = (iss >> 5) & 0x1f;
        bool is_read = iss & 1;

        const struct sysreg_desc *d = find_desc(iss);
        if (!d)
                return inject_undefined(v);   /* guest gets UNDEF */

        if (is_read)
                vcpu_set_reg(v, rt, d->get(v));
        else
                d->set(v, vcpu_get_reg(v, rt));

        /* ALWAYS advance past the trapping instruction, or you
           will execute it again, for ever.                    */
        vcpu_skip_instr(v);
        return 1;
}
```

> **FORGETTING `VCPU_SKIP_INSTR`**
> The guest re-executes the same instruction, traps again, and the VM appears to hang while burning 100% of a physical CPU. It is the single most common bug when hand-writing a trap handler, and it looks like a livelock rather than a crash. If a guest is stuck, check the exit counter — a constant, enormous exit rate with no progress is this bug.

The other trap registers

Register	Controls	Typical use
`CPTR_EL2`	FP/SIMD, SVE, SME, trace	Trap the first FP use so you can save/restore the register file lazily — a large win, since many guests never touch SVE
`MDCR_EL2`	Debug, PMU, trace buffers	Hide or partition the PMU; stop a guest from single-stepping the host
`HSTR_EL2`	AArch32 CP15 access by CRn	Only for 32-bit guests; a bitmask of coprocessor register groups
`HFGRTR_EL2` / `HFGWTR_EL2` etc.	Fine-grained traps (ARMv8.6)	Trap one individual register rather than a whole group — much cheaper than TVM when you need exactly one
`HCRX_EL2`	Extended HCR bits (ARMv8.7)	Ran out of bits in `HCR_EL2`; newer controls live here

Watching traps on a live system

```
# Every exit KVM takes is counted and typed.

$ sudo perf stat -e 'kvm:*' -a sleep 5
        12,043  kvm:kvm_entry
        12,043  kvm:kvm_exit
         8,911  kvm:kvm_mmio
           520  kvm:kvm_wfx_arm64
            17  kvm:kvm_hvc_arm64

$ sudo perf record -e kvm:kvm_exit -a -- sleep 5
$ sudo perf script | head
# the ESR_EL2 value is in the event, so you can bucket exits by EC

# Or read the per-VM counters directly:
$ sudo cat /sys/kernel/debug/kvm/*/vcpu0/*exits* 2>/dev/null

# A healthy compile-in-a-VM: a few thousand exits per second,
# nearly all MMIO to virtio queues.
# A sick one: hundreds of thousands, dominated by kvm_wfx or by
# MMIO to an emulated legacy device.
```

How it shows up

Symptom	Cause
Guest hangs, one pCPU pinned at 100%, huge exit count	Trap handler not advancing the guest PC — the same instruction traps for ever
Guest takes an unexpected undefined-instruction exception	A trapped system register with no handler; the hypervisor injected UNDEF
Guest detects features the host cannot provide, then crashes	`TID3` not set, or ID registers not sanitised — the guest read the real feature bits
Host memory corruption when a guest boots or kexecs	`TSW` clear: guest set/way cache maintenance reached lines outside the VM
Idle guest still consumes a full pCPU	`TWI` clear, so `WFI` parks the physical core instead of yielding the vCPU thread

Symptom	Cause
Guest's secure-monitor calls reach EL3 unfiltered	`TSC` clear — SMCs pass straight through EL2
Terrible performance on fork-heavy workloads	`TVM` set unnecessarily: every page table register write is an exit
Guest FP state corrupted across a vCPU switch	`CPTR_EL2` lazy-FP logic wrong — trap taken but state not saved before reuse
Guest PMU counts host activity	`MDCR_EL2` not partitioning the PMU

Check yourself

- ✓ Explain what `HCR_EL2.VM` turns on and what happens without it.
- ✓ Explain the IMO/FMO pair: what they route, and what they simultaneously enable.
- ✓ Give three bits KVM sets and three it deliberately leaves clear, with the reasoning for each.
- ✓ Explain why `DC CIVAC` is safe to leave untrapped but `DC CISW` is not.
- ✓ Decode an `ESR_EL2` with `EC=0x18` down to the specific register and direction.
- ✓ Say what happens if a trap handler returns without advancing the guest PC, and how it presents.
- ✓ Name the register that controls FP/SVE trapping and explain lazy FP save.
- ✓ Explain what fine-grained traps buy you over `TVM`.
- ✓ Measure the exit rate of a running guest and classify the exits by cause.

Stage-2 translation

The guest builds page tables and believes their output is a physical address. It is not — it is an *intermediate* physical address, and a second, entirely separate MMU translation that the guest cannot see or configure turns it into the real thing. This is the mechanism that makes modern virtualisation cheap.

Three address spaces, not two

```
BARE METAL                      UNDER A HYPERVISOR

VA  virtual address             VA   virtual address
 |   stage 1                     |    stage 1  - the GUEST'S tables,
 |   TTBR0/1_EL1                 |    TTBR0/1_EL1, guest-controlled
 v                               v
PA  physical address            IPA  INTERMEDIATE physical address
 |                               |      "what the guest calls physical"
 v                               |    stage 2 - the HYPERVISOR'S tables,
memory                           |    VTTBR_EL2, guest cannot see them
                                 v
                                PA   real physical address
                                 |
                                 v
                                memory

The guest's entire world model - "RAM starts at 0x4000_0000,
the UART is at 0x0900_0000" - is a statement about IPAs.
It can be completely false about PAs and the guest will never
know.

Crucially: the guest configures stage 1 FREELY. It writes
TTBR0_EL1 and TCR_EL1 without trapping (HCR_EL2.TVM is clear).
Whatever it produces is confined by stage 2 regardless.
```

WHY THIS REPLACED SHADOW PAGE TABLES

Under shadow paging the hypervisor had to intercept every guest page-table edit and rebuild a composed table. Under stage 2 the hardware composes on the fly. The guest edits its own tables at full speed; the hypervisor only touches stage 2 when the *memory map* changes, which is rare. Fork-heavy workloads went from crippling to free.

The registers

```
/* VTTBR_EL2 - Virtualisation Translation Table Base Register */

   bits [47:1]   BADDR   physical base of the stage-2 table
   bits [63:48]  VMID    the VM identity (or [63:56] if 8-bit)
   bit  [0]      CnP     common-not-private

/* One register carries BOTH the table pointer and the VM tag.
   Switching VMs is therefore literally one MSR. That is not an
   accident - it is what makes a vCPU switch affordable.       */


/* VTCR_EL2 - Virtualisation Translation Control Register     */

   T0SZ   [5:0]    input size: IPA space is (64 - T0SZ) bits
   SL0    [7:6]    STARTING LEVEL of the stage-2 walk
   IRGN0  [9:8]    inner cacheability of table walks
   ORGN0  [11:10]  outer cacheability
   SH0    [13:12]  shareability of table walks
   TG0    [15:14]  granule: 00=4K, 01=64K, 10=16K
   PS     [18:16]  physical address size
   VS     [19]     VMID size: 0 = 8-bit, 1 = 16-bit
   HA/HD  [21:22]  hardware access flag / dirty bit management
   NSW/NSA         secure-world controls

/* Example: 40-bit IPA space, 4K granule, 16-bit VMIDs */

   T0SZ = 64 - 40 = 24
   TG0  = 0b00
   VS   = 1
```

SL0: the part people get wrong

Stage 1 always starts its walk at a level implied by `TCR_EL1.T0SZ`. Stage 2 makes the starting level *explicit*, because the IPA space is usually smaller than a full 48-bit space and starting at level 0 would waste an entire table on one used entry. Getting `SL0` inconsistent with `T0SZ` is the classic silent stage-2 misconfiguration.

```
4K granule. Each level consumes 9 bits, leaf offset is 12.

IPA bits   levels needed         SL0 encoding   concat?
--------   --------------------  ------------   -------
<= 30      2:  L2 -> L3          0b00 (start 2)  no
31..39     3:  L1 -> L2 -> L3     0b01 (start 1)  maybe
40..48     4:  L0 -> L1 ->L2 ->L3 0b10 (start 0)  maybe

The 40-bit case, worked:

  IPA[39:30] -> level 1 index   (10 bits, not 9!)
  IPA[29:21] -> level 2 index   (9 bits)
  IPA[20:12] -> level 3 index   (9 bits)
  IPA[11:0]  -> page offset

Ten bits at the top means 1024 entries, but a 4K table holds
only 512. Stage 2 solves this with CONCATENATED TABLES: you
allocate 2 contiguous 4K tables (8K, 8K-aligned) and the
hardware treats them as one 1024-entry table.

Stage 1 has no such feature. It is unique to stage 2, it
exists purely to avoid a wasteful extra level, and up to 16
tables may be concatenated.
```

THE CONCATENATION TRAP

If your IPA size requires concatenation and you allocate one table instead of two — or allocate two that are not *contiguous and aligned to the total size* — the walk reads garbage for the upper half of the address space. Low addresses work, so the guest boots; the failure appears only when it touches memory above the halfway mark. Diagnose it by checking that `VTTBR_EL2.BADDR` is aligned to `table_size * count`, not merely to 4K.

Stage-2 descriptors differ from stage-1

```
Stage-2 block/page descriptor, lower attributes:

 63    55 54 53 52  51        12 11 10 9  8  7 6 5 4    2 1 0
+--------+--+--+--+-------------+--+--+---+--+---+------+-+-+
| ignored|XN|  |CT| output addr |nG|AF| SH |S2AP| MemAttr |1|V|
+--------+--+--+--+-------------+--+--+---+--+---+------+-+-+

The important differences from stage 1:

S2AP [7:6]   access permissions - and they are ABSOLUTE, not
             relative to a privilege level:
               00  no access
               01  read only
               10  write only   (yes, really)
               11  read/write
             There is no "EL0 vs EL1" split here. Stage 2 does
             not care which privilege level the guest was in;
             that is stage 1's business.

MemAttr[5:2] memory attributes ENCODED DIRECTLY - there is no
             MAIR indirection at stage 2:
               0b0000  Device-nGnRnE
               0b0001  Device-nGnRE
               0b0010  Device-nGRE
               0b0011  Device-GRE
               0b1111  Normal, inner+outer write-back cacheable
               0b0101  Normal, outer WT, inner non-cacheable
               ...

XN [54]      execute never. On ARMv8.2 this widens to XN[54:53]
             so you can forbid execution at EL1 but allow it
             at EL0, or vice versa.

AF [10]      access flag - set by hardware if VTCR_EL2.HA, else
             a fault the hypervisor must service.
```

How the two stages combine

```
/* PERMISSIONS: the strictest of the two wins. Always. */

   stage 1 says RW, stage 2 says RO   -> read only
   stage 1 says RO, stage 2 says RW   -> read only
   stage 1 says X,  stage 2 says XN   -> no execute

/* There is no way for a guest to widen a permission stage 2
   denied. That is the isolation guarantee, in one line.     */


/* MEMORY TYPE: the more restrictive/stronger type wins.      */

   stage 1 Normal WB  + stage 2 Normal WB     -> Normal WB
   stage 1 Normal WB  + stage 2 Device        -> DEVICE
   stage 1 Device     + stage 2 Normal        -> Device
   stage 1 Normal NC  + stage 2 Normal WB     -> Normal NC

/* "Device beats Normal, non-cacheable beats cacheable."

   This has a consequence people trip over constantly: if the
   hypervisor maps guest RAM as Device at stage 2 (perhaps by
   accident, perhaps because it came from a PCI BAR), then the
   guest's careful Normal-cacheable mapping is DOWNGRADED. The
   guest's unaligned accesses now fault, its exclusives may
   fail, and its memcpy throughput collapses. The guest is
   blameless and its page tables look perfect.               */

/* HCR_EL2.DC and HCR_EL2.FWB modify this:

   DC  = 1 : while the guest's stage-1 MMU is OFF, treat its
             accesses as Normal cacheable rather than Device.
             Used during guest boot so early code is coherent.

   FWB = 1 : (ARMv8.4) "forced write-back" - stage 2 OVERRIDES
             stage 1 rather than combining with it. Simplifies
             the hypervisor enormously: you stop caring what the
             guest chose. Modern KVM uses it where available.  */
```

The full walk, counted

```
A single guest load, worst case, 4K granule, 48-bit VA,
40-bit IPA, cold TLB:

stage 1 walk needs 4 table reads (L0,L1,L2,L3).
BUT each of those reads produces an IPA, which must itself
be translated by stage 2 - and stage 2 is a 3-level walk.

  s1 L0 descriptor address (an IPA) -> 3 stage-2 reads
  s1 L1 descriptor address (an IPA) -> 3 stage-2 reads
  s1 L2 descriptor address (an IPA) -> 3 stage-2 reads
  s1 L3 descriptor address (an IPA) -> 3 stage-2 reads
  final IPA from stage 1            -> 3 stage-2 reads
  ------------------------------------------------------
                                      15 stage-2 reads
                                    +  4 stage-1 reads
                                    = 19 memory accesses
                                      for ONE load.

With a 4-level stage 2 it reaches 24. This is why:

  * TLBs matter more under virtualisation, not less
  * huge pages / blocks at stage 2 are a big win (they cut
    stage-2 levels AND increase reach)
  * the walk cache (intermediate descriptors) is critical
  * VMID tagging is essential - flushing on every VM switch
    would mean paying this cost constantly
```

Building a stage-2 table

```
/* Minimal, illustrative. 4K granule, 40-bit IPA, so a 3-level
   walk starting at level 1 with 2 concatenated L1 tables.    */

#define S2_VALID      (1UL << 0)
#define S2_TABLE      (3UL << 0)   /* valid + table            */
#define S2_BLOCK      (1UL << 0)   /* valid + block            */
#define S2_PAGE       (3UL << 0)   /* valid + page (level 3)   */
#define S2_AF         (1UL << 10)
#define S2_SH_IS      (3UL << 8)
#define S2_AP_RW      (3UL << 6)
#define S2_AP_RO      (1UL << 6)
#define S2_MEM_NORMAL (0xfUL << 2)
#define S2_MEM_DEVICE (0x1UL << 2)  /* Device-nGnRE            */
#define S2_XN         (1UL << 54)

static u64 *l1;    /* 2 contiguous 4K pages, 8K-aligned        */

void s2_map_block_2m(u64 ipa, u64 pa, bool device, bool exec)
{
        u64 *l2 = get_or_alloc_l2(l1, ipa);
        u64  attr = S2_BLOCK | S2_AF | S2_SH_IS | S2_AP_RW
                  | (device ? S2_MEM_DEVICE : S2_MEM_NORMAL)
                  | (exec ? 0 : S2_XN);

        l2[(ipa >> 21) & 0x1ff] = (pa & ~0x1fffffUL) | attr;

        /* Publish before any walker can see it. */
        dsb(ishst);
}

void s2_enable(u64 vttbr_base, u16 vmid)
{
        u64 vtcr = (24UL << 0)       /* T0SZ: 40-bit IPA        */
                 | (1UL  << 6)       /* SL0 = 1: start level 1  */
                 | (3UL  << 8)       /* IRGN0 = WBWA            */
                 | (3UL  << 10)      /* ORGN0 = WBWA            */
                 | (3UL  << 12)      /* SH0 = inner shareable   */
                 | (0UL  << 14)      /* TG0 = 4K                */
                 | (2UL  << 16)      /* PS = 40-bit PA          */
                 | (1UL  << 19);     /* VS = 16-bit VMID        */

        write_sysreg(vtcr, vtcr_el2);
        write_sysreg(((u64)vmid << 48) | vttbr_base, vttbr_el2);
        isb();

        /* Only now turn stage 2 on. */
        write_sysreg(read_sysreg(hcr_el2) | HCR_VM, hcr_el2);
        isb();
}
```

Invalidating stage-2 entries

```
/* Stage-2 has its own TLBI instructions, and they take IPAs. */

    TLBI IPAS2E1IS,  x0    /* by IPA, current VMID, inner-shareable
                              x0 = IPA >> 12                    */
    TLBI IPAS2LE1IS, x0    /* ...last level only                */
    TLBI VMALLS12E1IS      /* all stage-1 AND stage-2 entries for
                              the current VMID                   */
    TLBI ALLE1IS           /* all EL1 entries, all VMIDs         */
    TLBI VMALLE1IS         /* stage-1 only, current VMID         */

/* The sequence that actually works when you unmap a page: */

    write the descriptor to 0
    dsb(ishst);                 /* descriptor is visible         */
    tlbi ipas2e1is, ipa >> 12;
    dsb(ish);                   /* invalidation has completed    */
    tlbi vmalle1is;             /* stage-1 entries CACHED THE
                                   COMBINED translation - they
                                   must go too!                  */
    dsb(ish);
    isb();

/* That second TLBI surprises people. A stage-1 TLB entry holds
   a VA -> PA mapping that already folded stage 2 in. Changing
   stage 2 therefore invalidates stage-1 entries, even though
   the guest's stage-1 tables did not change.                  */
```

How it shows up

Symptom	Cause
Guest boots, then dies once it touches high memory	Concatenated tables not allocated contiguously or not aligned to the combined size
Stage-2 walk reads nonsense immediately	`SL0` inconsistent with `T0SZ` — the walk starts at the wrong level
Guest memcpy is 10x slow; unaligned accesses fault	Guest RAM mapped as Device at stage 2; the combine rule downgraded the guest's Normal mapping
Guest exclusives (`LDXR` / `STXR`) never succeed	Same cause — exclusives require Normal memory
Guest sees stale data after the hypervisor remaps memory	Missing `TLBI VMALLE1IS` after the stage-2 invalidate
Second VM sees the first VM's translations	VMID not changed, or VMID rollover without a global invalidate
Guest cannot execute from a page it can read	Stage-2 `XN` set — common when memory was mapped through a device path
Performance collapses under memory pressure	Stage 2 built entirely from 4K pages; no blocks, so every walk is maximal
Works with the guest MMU off, breaks when it turns it on	`HCR_EL2.DC` was papering over a stage-2 attribute mistake that becomes visible once stage 1 is live

Check yourself

- ✓ Define VA, IPA and PA and say which software owns the translation between each pair.
- ✓ Explain why stage 2 replaced shadow page tables and what it bought.
- ✓ Explain what `VTTBR_EL2` holds besides a table address, and why that matters for switch cost.

- ✓ Given a 40-bit IPA and a 4K granule, work out `T0SZ`, `SL0` and whether concatenation is needed.
- ✓ Name three ways a stage-2 descriptor differs from a stage-1 one.
- ✓ State the permission-combining rule and the memory-type-combining rule.
- ✓ Explain how mapping guest RAM as Device at stage 2 breaks a guest that looks correct.
- ✓ Count the memory accesses in a worst-case two-stage walk and say what reduces it.
- ✓ Write the correct invalidation sequence for unmapping a stage-2 page, and justify the second TLBI.

Stage-2 faults and how to decode them

A stage-2 fault is not an error. It is the hypervisor's primary input channel: it is how demand paging, MMIO emulation, dirty tracking, ballooning and copy-on-write all get their work. Learning to read one exactly is the difference between guessing and knowing.

A fault is a request

```
The guest does something perfectly ordinary:

    LDR x0, [x1]        where x1 -> 0x0900_0000 (its UART)
                      or x1 -> 0x4100_0000 (its RAM)

Stage 1 translates: VA -> IPA. Fine, the guest's own tables.
Stage 2 translates: IPA -> PA. And here it may say NO.

    +-------------------------------------------------+
    |  stage-2 descriptor invalid, or permission      |
    |  insufficient  ->  DATA ABORT to EL2            |
    +-------------------------------------------------+
                      |
                      v
    hypervisor asks: WHY did I leave that unmapped?

      "because there is no RAM there, but there IS a
       device model"        -> emulate the access (ch. 13)

      "because I have not allocated backing memory yet"
                           -> allocate, map, retry

      "because I am tracking which pages the guest writes"
                           -> record it, make writable, retry

      "because I swapped it out"
                           -> page it back in, map, retry

      "because the guest is genuinely misbehaving"
                           -> inject an abort into the guest

FIVE different meanings, one hardware event. Everything
depends on decoding it correctly.
```

The three registers that describe a fault

Register	Holds	Caveat
`ESR_EL2`	Exception class and syndrome: what kind of access, what size, which register, why it faulted	The syndrome is only *partly* valid — check `ISV`
`FAR_EL2`	The faulting **virtual** address	Meaningless to the hypervisor on its own — it is a guest VA, in the guest's address space
`HPFAR_EL2`	The faulting **IPA**, bits [47:12], shifted right by 8	This is the one you actually want. It exists *because* `FAR_EL2` is useless

```
/* Reconstructing the faulting IPA */

    u64 hpfar = read_sysreg(hpfar_el2);
    u64 far   = read_sysreg(far_el2);

    u64 ipa = (hpfar & GENMASK(47, 4)) << 8;   /* page-aligned  */
    ipa |= far & 0xfff;                        /* offset in page */

/* HPFAR gives you the PAGE. The low 12 bits of the byte offset
   come from FAR - and that is legitimate, because stage 1 does
   not change the low 12 bits of an address.                   */
```

Decoding ESR_EL2

```
   EC = 0x24  Data Abort from a lower EL   (stage-2 data fault)
   EC = 0x20  Instruction Abort, lower EL   (stage-2 fetch fault)

   ISS layout for a data abort:

    24  23 22 21 20 19  16 15 14 13 12 11 10  9  8  7 6 5     0
   +---+---+---+--+---+-----+--+--+--+--+---+--+--+--+---------+
   |ISV| SAS |SSE| SRT  |SF|AR| VNCR|SET|FnV|EA|CM|S1PTW|WnR|DFSC|
   +---+---+---+--+---+-----+--+--+--+--+---+--+--+--+---------+

   ISV   [24]     Instruction Syndrome Valid. If 0, the fields
                  below are NOT populated and you must decode the
                  guest instruction yourself.
   SAS   [23:22]  access size: 00=byte 01=half 10=word 11=double
   SSE   [21]     sign extend
   SRT   [20:16]  the guest register number involved
   SF    [15]     64-bit register
   AR    [14]     acquire/release semantics
   S1PTW [7]      the fault happened while walking a STAGE-1
                  table - i.e. the guest's own page table lives
                  in a page you have not mapped
   WnR   [6]      1 = write, 0 = read
   DFSC  [5:0]    the actual fault status code
```

DFSC	Meaning	What the hypervisor usually does
`0b000100` – `0b000111`	Translation fault, level 0–3	Nothing was mapped. Allocate and map, or route to a device model
`0b001000` – `0b001011`	Access flag fault, level 0–3	Set AF; used to build a page-age estimate for reclaim
`0b001100` – `0b001111`	Permission fault, level 0–3	Mapped but not with these rights. Dirty tracking, CoW, or a real guest violation
`0b010000`	External abort, not on a table walk	The bus said no. Real hardware error — usually fatal
`0b010100` – `0b010111`	External abort on a table walk	Your stage-2 tables are in unbacked or faulty memory
`0b100001`	Alignment fault	Guest did an unaligned access to Device memory — see the attribute-downgrade trap in chapter 5
`0b110000`	TLB conflict abort	Overlapping TLB entries. Invalidate everything and retry

The ISV=0 problem

When `ISV` is 1 the hardware hands you the access size, direction and register number, and MMIO emulation is trivial. When it is 0 you get none of that, and you must fetch and decode the guest's instruction yourself — from guest memory, at a guest VA, which may itself be unmapped.

```
/* ISV is 0 for, among others:

     - any instruction abort (there is no data operand)
     - load/store multiple  (LDP/STP is fine; LDM on AArch32 is not)
     - loads/stores with writeback in some cases
     - DC ZVA and other cache operations
     - accesses to a page where S1PTW is set
     - anything the implementation simply chose not to report

   Consequence: a hypervisor CANNOT rely on ISV. KVM's answer is
   blunt and correct: if ISV is 0 on an MMIO access, refuse.   */

  if (!(esr & ESR_ELx_ISV)) {
          kvm_pr_unimpl("Data abort outside memslots with no valid syndrome\n");
          return -ENOSYS;      /* the VM dies                   */
  }

/* Which is why a guest driver that does this:

       ldp x0, x1, [x2]        // load pair from an MMIO region

   can kill the VM outright while the equivalent

       ldr x0, [x2]
       ldr x1, [x2, #8]

   works perfectly. It is not a bug in your driver on hardware.
   It is a rule you only discover under virtualisation:

     NEVER use load/store-multiple, DC ZVA, or unaligned access
     on memory-mapped device registers.

   Linux enforces this by making you go through readl/writel,
   which compile to single loads and stores for exactly this
   reason.                                                    */
```

> **THE `MEMCPY`-TO-MMIO BUG**
>
> `memcpy()` into a device region compiles to `LDP` / `STP` and possibly `DC ZVA`. On hardware it usually works. Under a hypervisor it produces `ISV=0` and the VM is killed with "no valid syndrome". Use `memcpy_toio()` / `__iowrite32_copy()`, or explicit `writel()` loops.

S1PTW: the nested fault

```
The guest's stage-1 page table is itself in guest memory.
That memory has an IPA. That IPA needs stage-2 translation.

  guest executes    LDR x0, [x1]
     |
     v
  stage-1 walk needs to read the L2 descriptor
     |  descriptor lives at guest-physical 0x4020_3000
     v
  stage 2 asked to translate IPA 0x4020_3000
     |
     v
  NOT MAPPED  ->  data abort to EL2, with S1PTW = 1

Meaning: "I faulted while fetching the guest's own page
table, not while performing the guest's actual access."

Why you must care:
  * WnR is meaningless-ish: the walk may be a READ even
    though the guest's access was a WRITE. Do not use WnR to
    decide writability here. (With hardware dirty-bit
    management the walk may genuinely write, which is why
    modern code checks both.)
  * ISV is 0.
  * You must map the page and RETRY. Do not try to emulate.
  * If the IPA has no memory behind it at all, the guest has
    put its page tables in a device region or in a hole. That
    is a guest bug and deserves an injected abort.
```

A fault handler, end to end

```
int handle_stage2_abort(struct vcpu *v)
{
        u64  esr   = read_sysreg(esr_el2);
        u32  dfsc  = esr & 0x3f;
        bool write = esr & ESR_ELx_WNR;
        bool s1ptw = esr & ESR_ELx_S1PTW;
        u64  ipa   = (read_sysreg(hpfar_el2) & GENMASK(47,4)) << 8;

        ipa |= read_sysreg(far_el2) & 0xfff;

        /* 1. Is this address backed by guest RAM at all? */
        struct memslot *slot = find_memslot(v->vm, ipa);

        if (!slot) {
                /* No RAM. Either a device we model, or nowhere. */
                if (s1ptw)
                        return inject_abort(v);   /* page tables
                                                     in a hole  */
                if (!(esr & ESR_ELx_ISV))
                        return -ENOSYS;           /* cannot emulate */
                return emulate_mmio(v, ipa, esr); /* chapter 13  */
        }

        /* 2. Backed by RAM. Why did it fault? */
        switch (dfsc & ~3) {                /* mask off the level */
        case FSC_TRANSLATION:               /* nothing mapped     */
                return map_page(v, ipa, slot, write);

        case FSC_ACCESS_FLAG:               /* aging              */
                return set_access_flag(v, ipa);

        case FSC_PERMISSION:
                if (write && dirty_logging_enabled(v->vm)) {
                        mark_page_dirty(v->vm, ipa);
                        return make_writable(v, ipa);
                }
                if (write && is_cow(slot, ipa))
                        return do_copy_on_write(v, ipa);
                return inject_abort(v);     /* genuine violation  */

        case FSC_EXTERNAL:
                return inject_serror(v);    /* the bus said no    */

        default:
                return inject_abort(v);
        }
        /* NOTE: no vcpu_skip_instr anywhere. After mapping a page
           we RETRY the guest instruction; it must run again. This
           is the opposite of a system-register trap.           */
}
```

> **RETRY VERSUS SKIP**
>
> A system-register trap is *emulated*, so you advance the guest PC past it. A stage-2 translation fault is *fixed*, so you leave the PC alone and let the instruction re-execute. Skipping after a translation fault silently drops the guest's load or store; retrying after an emulated MMIO write performs it twice. Both are ugly, intermittent bugs.

Dirty tracking, which is just permission faults on purpose

```
/* Live migration needs to know which guest pages changed while
   it was copying. There is no hardware "dirty log" for stage 2
   on most parts, so KVM builds one out of faults:            */

  1. write-protect every stage-2 mapping        (S2AP = read only)
  2. copy all guest memory to the destination
  3. guest writes a page -> PERMISSION FAULT at stage 2
     handler: record the page number in a bitmap,
              set S2AP = RW, TLBI, retry
  4. re-copy the pages in the bitmap
  5. repeat 3-4 until the dirty set is small
  6. pause the guest, copy the remainder, switch over

/* Cost: one exit per page, per round. A guest writing memory
   faster than you can copy it never converges - that is the
   classic "migration never completes" failure, and the fix is
   either to throttle the guest or to give up and pause it.

   ARMv8.1 hardware dirty-bit management (VTCR_EL2.HD) lets the
   PTW set the dirty bit itself, removing the exit. Where it
   exists, KVM uses it and migration gets dramatically cheaper. */
```

Seeing faults on a live system

```
$ sudo perf stat -e 'kvm:kvm_*' -a sleep 5
       142,551  kvm:kvm_exit
        98,204  kvm:kvm_mmio                # device emulation
        31,880  kvm:kvm_guest_fault         # stage-2 faults
           440  kvm:kvm_set_spte_hva

# Per-fault detail, including the IPA and the ESR:
$ sudo perf record -e kvm:kvm_guest_fault -a -- sleep 2
$ sudo perf script | head -3
 CPU 2 ... kvm_guest_fault: ipa 0x40a13000 hsr 0x93000047 hxfar 0xffff800010a13000

#   hsr  = ESR_EL2  -> 0x93000047
#     EC   = 0x24  data abort, lower EL
#     ISV  = 1
#     WnR  = 1     it was a write
#     DFSC = 0x07  translation fault, level 3
#   So: guest wrote to an unmapped page; KVM will allocate one.

# Inside a guest, the other side of the same story:
$ sudo dmesg | grep -i 'unhandled fault\|synchronous external abort'
```

How it shows up

Symptom	Cause
`Data abort outside memslots with no valid syndrome`, VM killed	`ISV=0` on an MMIO access — the guest used `LDP` / `STP`, `memcpy` or `DC ZVA` on a device region
Guest load returns a stale or wrong value once in a while	Handler advanced the PC after fixing a translation fault, dropping the access
Emulated device sees each write twice	Handler retried instead of skipping after emulating an MMIO access

Symptom	Cause
Infinite fault loop on one address	Descriptor written but no `DSB` / `TLBI`, so the walker keeps seeing the old entry
Guest dies during early boot with an abort on its own page tables	`S1PTW` fault the handler treated as MMIO
Live migration never converges	Guest dirties pages faster than they can be copied; no hardware dirty-bit support and no throttling
Alignment faults inside a guest that works on hardware	Stage-2 attributes downgraded the region to Device
Fault reported at a guest VA that makes no sense	Reading `FAR_EL2` instead of `HPFAR_EL2`
Guest sees SError shortly after an abort	External abort turned into an SError injection — usually genuinely broken hardware or a bad DMA target

Check yourself

- ✓ List five different reasons a hypervisor might deliberately leave a stage-2 mapping absent.
- ✓ Explain why `FAR_EL2` is insufficient and what `HPFAR_EL2` provides.
- ✓ Reconstruct a full faulting IPA from `HPFAR_EL2` and `FAR_EL2`.
- ✓ Decode `ESR_EL2 = 0x93000047` completely.
- ✓ Explain `ISV`, name three cases where it is 0, and say what a hypervisor does then.
- ✓ Explain why `memcpy()` to MMIO can kill a VM but works on bare metal.
- ✓ Define `S1PTW` and explain why `WnR` is unreliable when it is set.
- ✓ State the rule for when to retry the guest instruction and when to skip it.
- ✓ Describe how dirty-page tracking is built from permission faults, and what `VTCR_EL2.HD` changes.

Memory attributes and cacheability under virtualisation

Two independent pieces of software now describe the same byte of memory, and the hardware combines their opinions with a rule neither of them chose. Most "impossible" virtualisation bugs — a guest that boots but corrupts data, a device that reads zeroes, an exclusive that never succeeds — live here.

The combining rule

```
For every access, the CPU has TWO opinions about the memory:

  stage 1 attribute   (guest's MAIR_EL1 + descriptor AttrIndx)
  stage 2 attribute   (hypervisor's descriptor MemAttr[3:0])

and it must produce ONE. The rule is "most restrictive wins":

  TYPE           Device beats Normal
  CACHEABILITY   Non-cacheable beats Write-Through
                 Write-Through   beats Write-Back
  SHAREABILITY   Outer beats Inner beats Non-shareable
  DEVICE STRENGTH nGnRnE beats nGnRE beats nGRE beats GRE

Worked cases:

  s1 Normal WB, s2 Normal WB      -> Normal WB       (good)
  s1 Normal WB, s2 Normal NC      -> Normal NC       (slow)
  s1 Normal WB, s2 Device-nGnRE   -> Device-nGnRE    (broken)
  s1 Device,    s2 Normal WB      -> Device          (fine)
  s1 Normal WB, s2 Device-nGnRnE  -> Device-nGnRnE   (very broken)

The GUEST cannot escape upward. It can only make things
stricter than the hypervisor allowed - never looser.

Which means: if the hypervisor gets stage 2 wrong, the guest
has NO WAY to compensate, and its own page tables will look
completely correct while it misbehaves.
```

THE ASYMMETRY THAT MATTERS

Isolation only requires stage 2 to restrict *permissions*. Restricting *memory type* is a side effect of the same mechanism, and it is almost never what you want for ordinary RAM. Map guest RAM as Normal, Inner-Shareable, Write-Back at stage 2 and let the guest decide. Anything else is a bug waiting for a workload.

What Device memory actually costs a guest

Property	Normal memory	Device memory
Unaligned access	Allowed	**Alignment fault**
`LDXR` / `STXR` exclusives	Work	**Constrained unpredictable** — in practice, never succeed
Speculative reads	Allowed	Forbidden
Access merging / reordering	Allowed	Restricted per nGnRE flavour
Caching	Yes	Never
Throughput for a bulk copy	GB/s	Tens of MB/s

Property	Normal memory	Device memory
`DC ZVA`	Works	Faults

An arm64 Linux guest whose RAM is Device at stage 2 will typically boot — early boot code is careful and aligned — and then die the moment the C library uses an optimised `memcpy`, or the moment `futex` takes an exclusive. The symptom looks like memory corruption or a scheduler hang, and nothing in the guest points at the real cause.

FWB: making the guest's opinion irrelevant

ARMv8.4-S2FWB ("stage-2 force write-back") changes the rule from *combine* to *override*. It exists because the combining rule made a whole class of hypervisor bugs unavoidable.

```
/* Without FWB, the hypervisor has a problem it cannot solve:

   The guest boots with its MMU OFF. All its accesses are
   therefore Device-nGnRnE (that is the architectural default
   when stage 1 is disabled). The hypervisor, meanwhile, has
   the page mapped Normal cacheable and has written the kernel
   image into it THROUGH THE CACHE.

   Guest reads with MMU off  -> Device -> bypasses the cache
                             -> reads STALE DRAM
                             -> reads garbage instead of the
                                kernel it was given.

   Historic fixes, both unpleasant:
     a) HCR_EL2.DC = 1: force the guest's stage-1-disabled
        accesses to be treated as Normal cacheable. Works, but
        it lies to a guest that genuinely wanted Device access
        to a real device before enabling its MMU.
     b) The hypervisor cleans the entire guest image to the
        point of coherency before every VM entry where the
        guest MMU is off. Correct and very slow.              */

/* With HCR_EL2.FWB = 1:

     stage 2 MemAttr is no longer combined - it WINS.

     s2 MemAttr = 0b0110  ->  Normal, Write-Back, whatever
                              stage 1 says. Including when
                              stage 1 is disabled.
     s2 MemAttr = 0b0101  ->  Normal, Non-Cacheable, forced
     s2 MemAttr = 0b0001  ->  Device-nGnRE, forced

   The hypervisor now states the truth once, in one place, and
   the guest's stage-1 attributes only affect ordering and
   sharing - not cacheability. KVM enables FWB whenever the
   silicon has it, and a large amount of cache-maintenance code
   becomes dead.                                              */
```

Who cleans the cache, and when

```
Three participants, three different views of memory:

  the GUEST      writes through its cache
  the HYPERVISOR writes through its cache (a different
                 virtual mapping of the same physical page!)
  a DEVICE doing DMA  may or may not be coherent

ALIASING: the hypervisor maps guest RAM into its own address
space to load a kernel image, to emulate a device, or to
migrate the VM. That is a SECOND virtual mapping of the same
physical memory, potentially with different attributes. On
ARM, mismatched attributes for the same physical address are
architecturally UNPREDICTABLE - not merely slow.

The rules that keep you safe:

1. Hypervisor and guest should both use Normal WB Inner-
   Shareable for RAM. Then the coherent interconnect handles
   it and NO maintenance is needed.

2. If the hypervisor writes guest memory while the guest's
   MMU is off (loading a kernel), it must either
     - clean to the Point of Coherency afterwards, or
     - rely on HCR_EL2.DC / FWB.

3. If a NON-COHERENT device will read that memory, someone
   must clean it. Under passthrough this is the GUEST'S
   driver, and it works - the guest's DC CVAC operates on
   the right physical lines because stage 2 translated the
   address.

4. Never map the same page as Device in one place and Normal
   in another. This is the mismatched-attribute rule, and
   the failure is silent and rare enough to reach
   production.
```

> **THE MISMATCHED-ALIAS BUG**
>
> A hypervisor maps a guest's framebuffer as Device (because it came from a PCI BAR) while the guest maps it Normal Non-Cacheable (because that is what framebuffers want). Both mappings are "reasonable"; together they are architecturally unpredictable. Symptoms: tearing, occasional wrong pixels, and behaviour that changes with CPU model. Fix the hypervisor mapping, not the guest.

HCR_EL2.DC and the boot window

```
/* The window: from "hypervisor ERETs into the guest" to
   "guest sets SCTLR_EL1.M = 1". Perhaps a few thousand
   instructions, and every one of them is dangerous.        */

  HCR_EL2.DC = 1   effect while the guest's stage-1 MMU is OFF:

     the guest's accesses are treated as
         Normal, Inner-Shareable, Write-Back Read/Write-Allocate
     instead of
         Device-nGnRnE

     and stage 1 is forced to behave as if disabled anyway.

  WHY YOU WANT IT:
     the hypervisor wrote the kernel image through its cache;
     the guest can now read it coherently without anyone
     performing cache maintenance.

  WHY IT IS NOT FREE:
     a guest that legitimately wants to touch a real device
     before enabling its MMU now does so through the cache.
     For most guests this never happens. For a bare-metal
     payload that pokes a UART before MMU setup, it does -
     and the character never appears.

  KVM's actual policy (simplified):
     if FWB is available          -> use FWB, leave DC clear
     else if guest MMU is off     -> set DC
     else                         -> clear DC
   and it re-evaluates on every trap that could have changed
   SCTLR_EL1.M. That is one reason KVM traps SCTLR_EL1 writes
   on hardware without FWB, and does not on hardware with it. */
```

Checking what a guest really got

```
# From the HOST, dump KVM's stage-2 view. (Requires
# CONFIG_PTDUMP_DEBUGFS and a debug kernel; not all builds.)
$ sudo ls /sys/kernel/debug/kvm/
  1234-5/    # one directory per VM

# From the GUEST, look at what stage 1 asked for:
$ sudo cat /sys/kernel/debug/kernel_page_tables | head -20

# The practical test, from inside the guest - does it BEHAVE
# like Normal memory?

$ cat > attrtest.c <<'EOF'
#include <stdio.h>
#include <stdint.h>
int main(void) {
        static _Alignas(64) uint8_t buf[128];
        /* 1. unaligned access - faults on Device memory */
        volatile uint32_t *p = (uint32_t *)(buf + 1);
        *p = 0xdeadbeef;
        printf("unaligned ok: %08x\n", *p);

        /* 2. exclusives - never succeed on Device memory */
        uint32_t v = 0;
        int ok = __atomic_compare_exchange_n(&v, &(uint32_t){0}, 1,
                        0, __ATOMIC_SEQ_CST, __ATOMIC_SEQ_CST);
        printf("cas ok: %d\n", ok);
        return 0;
}
EOF
$ gcc attrtest.c -o attrtest && ./attrtest
unaligned ok: deadbeef
cas ok: 1

# A SIGBUS on test 1, or cas ok: 0 forever, means your RAM is
# not Normal memory. That is a stage-2 problem, in the host.

# Throughput is the blunt instrument that finds it fastest:
$ dd if=/dev/zero of=/dev/null bs=1M count=2000
# GB/s = Normal.  Tens of MB/s = Device. There is no middle.
```

Attributes for passed-through devices

```
/* When you assign a real device to a guest (chapter 16), its
   MMIO region must be Device at BOTH stages.               */

   stage 2: MemAttr = 0b0001  Device-nGnRE
            S2AP    = 0b11    read/write
            XN      = 1       never execute a device register

/* Getting this wrong in the permissive direction is the
   dangerous one: mapping a device region as Normal cacheable
   at stage 2 means

     - writes sit in the cache and reach the device late,
       out of order, or merged
     - the CPU may SPECULATIVELY READ device registers, which
       for many devices has side effects (clear-on-read status,
       FIFO pop). A speculative read can dequeue data nobody
       asked for.

   That last one is not theoretical. Speculatively reading a
   UART's RX FIFO loses characters that were never delivered to
   anyone. Always XN and always Device for MMIO.             */

/* And the reverse for the DMA buffers that device will use:
   they are ordinary RAM and must be Normal. A common bring-up
   mistake is to map an entire "device" range - registers AND
   the shared descriptor ring - as Device, then wonder why
   throughput is dreadful.                                   */
```

How it shows up

Symptom	Cause
Guest boots, then SIGBUS on unaligned access in libc	Guest RAM is Device at stage 2
Guest hangs in `futex` or spinlocks; exclusives never succeed	Same — exclusives require Normal memory
Guest memory bandwidth is 10–50x below the host's	Stage 2 forced Non-Cacheable or Device
Guest reads garbage instead of its own kernel image at boot	Hypervisor wrote through the cache; guest read with MMU off as Device. Needs `DC` or FWB or a clean
Bare-metal guest's early UART output is missing	`HCR_EL2.DC` set, so the pre-MMU device write went into the cache
Framebuffer tears or shows stale pixels	Mismatched aliases: Device in one mapping, Normal in another
Passed-through UART loses received characters	MMIO mapped Normal at stage 2; speculative reads popped the FIFO
Passed-through device DMA reads stale data	Descriptor ring mapped Device, or non-coherent device with no cache maintenance
Behaviour differs between two CPU models on the same code	Mismatched-attribute aliasing — architecturally unpredictable, so implementations differ
Everything above disappears on a newer SoC	That SoC has S2FWB and KVM stopped combining

Check yourself

- ✓ State the combining rule for type, cacheability and shareability, and apply it to four stage-1/stage-2 pairs.
- ✓ Explain why a guest cannot compensate for a stage-2 attribute mistake.
- ✓ List five things that break when RAM is Device instead of Normal.
- ✓ Explain the boot-window problem and the two historic fixes for it.
- ✓ Say what `HCR_EL2.DC` does, and give a case where setting it is wrong.
- ✓ Explain what S2FWB changes and why it deletes hypervisor code.
- ✓ Define the mismatched-alias rule and give a realistic example.
- ✓ Give the correct stage-2 attributes for a passed-through device's registers and for its DMA buffers.
- ✓ Describe two tests you can run inside a guest to prove its RAM is Normal memory.

The vCPU and the world switch

A vCPU is a data structure plus a promise: that whenever it runs, the physical CPU looks exactly the way that vCPU last left it. Keeping that promise is the world switch, and its length in instructions is the tax every guest pays on every exit.

What a vCPU is

```
struct vcpu – conceptually, three groups of state:

+-----------------------------------------------------------+
|  GUEST CPU STATE – must be identical on resume            |
|    x0–x30, SP_EL0, SP_EL1, PC (in ELR_EL2), PSTATE        |
|    FP/SIMD/SVE register file                              |
|    SCTLR_EL1 TTBR0_EL1 TTBR1_EL1 TCR_EL1 MAIR_EL1         |
|    VBAR_EL1 ESR_EL1 FAR_EL1 ELR_EL1 SPSR_EL1 AMAIR_EL1    |
|    CONTEXTIDR_EL1 TPIDR_EL0 TPIDRRO_EL0 TPIDR_EL1         |
|    CPACR_EL1 PAR_EL1 CSSELR_EL1 ...                       |
+-----------------------------------------------------------+
|  VIRTUAL HARDWARE – state of the devices this vCPU owns   |
|    vGIC CPU interface + list registers  (chapter 10)      |
|    virtual timer: CNTV_CTL, CNTV_CVAL, CNTVOFF (chapter 12)|
|    virtual PMU counters                                   |
|    MPIDR / MIDR the guest should see (VMPIDR/VPIDR_EL2)   |
+-----------------------------------------------------------+
|  HYPERVISOR BOOKKEEPING – never seen by the guest         |
|    pointer to the owning VM (and thus to VTTBR/VMID)      |
|    exit reason, exit counters                             |
|    pending interrupt injections                           |
|    the host thread this vCPU is running on                |
|    run state: runnable / halted (WFI) / stopped           |
+-----------------------------------------------------------+

In KVM this structure lives in the host kernel and is
reachable from user space through a per-vCPU file
descriptor and a shared page (kvm_run).
```

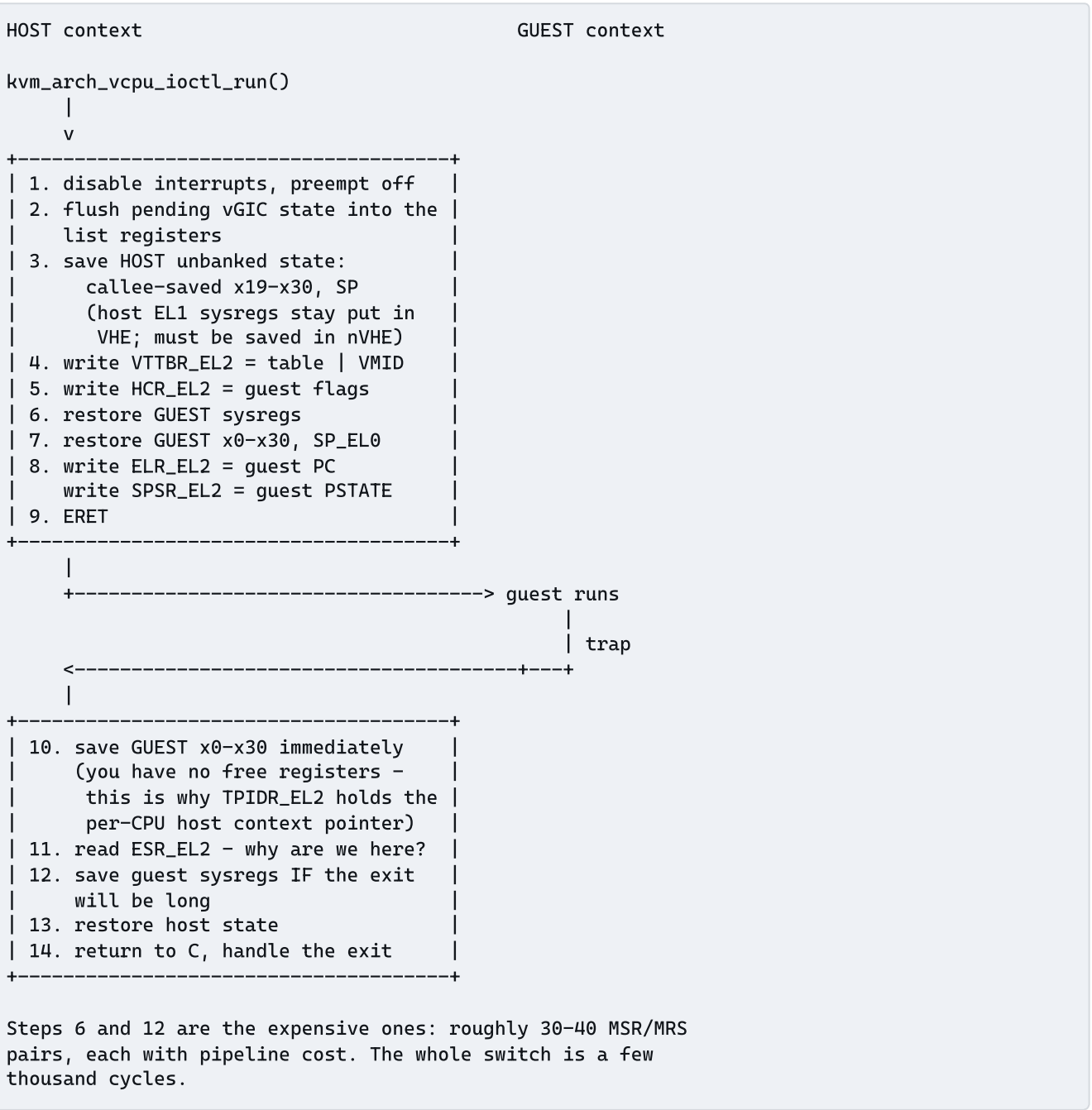
A VCPU IS A THREAD

In KVM, one vCPU is one host thread that spends its life in a loop: `ioctl(vcpu_fd, KVM_RUN)` enters the guest, returns when something needs handling, user space or the kernel handles it, repeat. Every consequence of that follows: vCPUs are scheduled by the ordinary Linux scheduler, they can be pinned with `taskset`, they show up in `top`, and a vCPU blocked in `WFI` is a thread blocked on a wait queue.

The run loop

```
/* User space (QEMU, crosvm, cloud-hypervisor): one thread per vCPU */

void *vcpu_thread(void *arg)
{
        struct vcpu *v = arg;

        for (;;) {
                ioctl(v->fd, KVM_RUN, 0);     /* blocks in the guest */

                switch (v->kvm_run->exit_reason) {

                case KVM_EXIT_MMIO:
                        /* the guest touched an address we model  */
                        mmio_dispatch(v->kvm_run->mmio.phys_addr,
                                      v->kvm_run->mmio.data,
                                      v->kvm_run->mmio.len,
                                      v->kvm_run->mmio.is_write);
                        break;

                case KVM_EXIT_HYPERCALL:
                        handle_hypercall(v);
                        break;

                case KVM_EXIT_SYSTEM_EVENT:   /* PSCI shutdown/reset */
                        return handle_system_event(v);

                case KVM_EXIT_INTR:           /* a host signal       */
                        break;                /* just re-enter       */

                case KVM_EXIT_FAIL_ENTRY:
                        die("entry failed, hw error 0x%llx",
                            v->kvm_run->fail_entry.hardware_entry_failure_reason);
                }
        }
}

/* Note what is ABSENT from that list: stage-2 translation
   faults, timer interrupts, GIC accesses, WFI. Those never
   reach user space - the kernel handles them and re-enters the
   guest without ever returning from the ioctl. That is the
   difference between a "light exit" and a "heavy exit", and it
   is roughly a factor of ten.                                  */
```

The world switch itself



Every optimisation is a form of laziness

Technique	Idea	Saves
Lazy FP/SIMD	Set `CPTR_EL2` to trap the first FP instruction. Only save and restore the register file if the guest actually used it	~1000 cycles per switch for guests that never touch FP; much more with SVE, where the file can be kilobytes
Light exits	Handle the exit in EL2 without restoring host EL1 state at all	Most of steps 12–13. A trapped timer read need not touch guest sysregs
VHE	Host runs at EL2 so its EL1-style registers are already the ones in use	The entire host sysreg save/restore. See chapter 9
VMID tagging	TLB entries carry the VM identity	A full TLB flush per switch — this is enormous
Deferred vGIC sync	Only read back the list registers if the guest could have changed them	Several register reads per exit

Technique	Idea	Saves
Pinned vCPUs	Stop the host scheduler moving a vCPU between physical cores	Cache and TLB warmth, and cross-core IPIs

> **WHY `TPIDR_EL2` MATTERS**
>
> On entry from the guest, every general-purpose register belongs to the guest and must not be clobbered before it is saved. The handler therefore needs one register's worth of scratch and a way to find its own per-CPU data. `TPIDR_EL2` is that anchor: it holds the address of this physical CPU's host context. Read it, stash a register, save the rest. Corrupt it and the hypervisor cannot find itself.

WFI, WFE and the idle guest

```
/* A guest with nothing to do executes WFI. On bare metal that
   parks the core until an interrupt arrives. Under a
   hypervisor that would waste a physical CPU, so HCR_EL2.TWI
   traps it.                                                  */

  guest: WFI
     -> trap to EL2, ESR_EL2.EC = 0x01
     -> KVM: kvm_vcpu_halt()
          the vCPU THREAD blocks on a wait queue.
          the physical CPU runs something else - another vCPU,
          a host process, or genuinely idles.

  later: a virtual interrupt becomes pending for this vCPU
     -> wake the thread
     -> re-enter the guest
     -> the guest's WFI "completes" and it takes the interrupt

/* WFE is different and subtler. A guest spinlock does:

       1: LDAXR w1, [x0]
          CBNZ  w1, 2f
          STXR  w2, w3, [x0]
          CBNZ  w2, 1b
          ... critical section ...
       2: WFE                 // sleep until someone SEVs
          B     1b

   If two vCPUs of the same guest are time-sharing ONE physical
   CPU, and the lock holder is descheduled, the waiter spins
   for its entire time slice waiting for a vCPU that cannot
   run. This is LOCK-HOLDER PREEMPTION, and it is the single
   worst pathology in overcommitted virtualisation.

   Trapping WFE turns the spin into a yield: KVM marks the vCPU
   as yielding and tries to schedule a sibling vCPU of the same
   VM - preferably the one holding the lock. That is
   kvm_vcpu_on_spin(), and it is why HCR_EL2.TWE is set.

   The real fix is not to overcommit vCPUs on a latency-
   sensitive workload. Paravirtualised queued spinlocks help
   on x86; on arm64 the WFE trap plus pinning is the practical
   answer.                                                    */
```

Bringing up a multi-vCPU guest

```
/* The guest cannot power on a CPU - there is no CPU to power
   on. It asks, via PSCI, and the hypervisor creates the
   illusion.                                                  */

  guest CPU0:  HVC  x0=PSCI_CPU_ON, x1=target MPIDR,
                    x2=entry point,  x3=context id
        |
        v
  EL2: find the vCPU whose VMPIDR_EL2 matches x1
       if it is already on          -> return ALREADY_ON
       set its PC = x2, x0 = x3
       set its state = runnable
       wake its host thread
       return SUCCESS
        |
        v
  guest CPU1 begins executing at x2, with x0 = context id,
  MMU off, at EL1.

/* Two details that bite:

   1. VMPIDR_EL2 is what the guest reads as MPIDR_EL1. YOU
      choose the affinity topology. If you hand out MPIDRs that
      imply 4 clusters of 1 CPU, the guest's scheduler builds
      its domains accordingly and may perform badly. Model a
      topology that matches reality, or a flat one.

   2. The PSCI conduit must match. The guest's device tree says
      method = "hvc" (under KVM) or "smc" (bare metal). A
      mismatch means CPU_ON sails past EL2 to EL3, where TF-A
      tries to power on a real core for a virtual request.
      Symptom: guest hangs with "CPU1: failed to boot: -110".  */
```

Measuring the switch

```
# How many exits, and how expensive?

$ sudo perf kvm --host stat live
# live per-exit-reason breakdown, updated continuously

$ sudo perf stat -e kvm:kvm_entry,kvm:kvm_exit -a sleep 10

# Time in the guest vs time in the host, per vCPU thread:
$ ps -T -p $(pgrep -f 'qemu.*mymachine') -o comm,pid,tid,pcpu
COMMAND          PID    TID %CPU
qemu-system-aar 4211   4211  3.1     # main/IO thread
CPU 0/KVM       4211   4219 74.2     # vCPU 0
CPU 1/KVM       4211   4220 71.8     # vCPU 1

# Pin them and watch the exit rate change:
$ taskset -cp 2 4219
$ taskset -cp 3 4220

# The microbenchmark that isolates switch cost: a guest loop
# that does nothing but trap.
#   guest:  1: hvc #0 ; b 1b
# Time N iterations, divide. On modern arm64 with VHE expect
# roughly 1-2 us per round trip if the hypercall is handled in
# the kernel, 10x that if it reaches user space.
```

How it shows up

Symptom	Cause
Guest FP results are wrong, but only sometimes	Lazy FP save/restore lost the register file — the trap fired but the state was not saved before reuse
Guest thread-local storage corrupts across a migration	`TPIDR_EL0` / `TPIDRRO_EL0` not in the save list
Hypervisor crashes with a wild pointer on guest exit	`TPIDR_EL2` clobbered — EL2 lost its per-CPU anchor
Guest is slow only when the host is busy	Lock-holder preemption; vCPUs oversubscribed on physical cores
Idle guest consumes a full physical core	`HCR_EL2.TWI` not set, so guest `WFI` parks real hardware
Guest hangs bringing up CPU1, `failed to boot: -110`	PSCI conduit mismatch or the target MPIDR does not match any vCPU's `VMPIDR_EL2`
Guest scheduler behaves oddly with many vCPUs	Fabricated MPIDR topology implies a cluster layout that does not exist
`KVM_EXIT_FAIL_ENTRY`	The CPU refused `ERET` into the guest — usually an illegal `SPSR_EL2`, or stage-2 not configured
Enormous exit count with a tiny amount of work done	A heavy exit path being taken for something that should be handled in EL2 — check `perf kvm stat`

Check yourself

- ✓ List the three categories of state a vCPU holds and give three examples of each.
- ✓ Explain the statement "a vCPU is a thread" and name three consequences.
- ✓ Walk through the world switch in order and identify the two most expensive steps.

- ✓ Explain why the guest's general-purpose registers must be saved before anything else on exit, and what `TPIDR_EL2` has to do with it.
- ✓ Describe lazy FP save/restore and the register that enables it.
- ✓ Distinguish a light exit from a heavy exit and give an example of each.
- ✓ Explain what trapping `WFI` achieves, and separately what trapping `WFE` achieves.
- ✓ Define lock-holder preemption and give two mitigations.
- ✓ Describe how a guest brings up its secondary CPUs and name two ways it goes wrong.
- ✓ Measure the exit rate of a running VM and identify the dominant exit reason.

VHE: running the host at EL2

The original EL2 design assumed a hypervisor written from scratch. Linux is not that, and KVM paid for the mismatch on every single exit. ARMv8.1's Virtualization Host Extensions fix it by making EL2 look like EL1 to software that runs there — an unusually elegant piece of architecture, and the reason modern arm64 virtualisation is fast.

The problem VHE solves

```
nVHE - the original design

EL0   host processes            guest userspace
EL1   HOST KERNEL  <---------+   guest kernel
                             |
EL2   a small stub of KVM    |
      "the hyp stub"         |
                             |
The host kernel runs at EL1. So does the guest kernel.
They both want SCTLR_EL1, TTBR0_EL1, VBAR_EL1, TPIDR_EL1...
and there is only ONE set of those registers.

Therefore EVERY entry to the guest must:
    save the host's ~25 EL1 system registers
    restore the guest's ~25 EL1 system registers
and every exit must do the reverse.

Worse: KVM's own logic lives at EL1, but only EL2 can do the
switch. So the code is SPLIT - a "hyp" half compiled
separately, with its own tiny address space, that cannot call
ordinary kernel functions, cannot take a lock, cannot print.
Two worlds inside one kernel source tree.
```

```
VHE - ARMv8.1

EL0   host processes            guest userspace
EL1                             guest kernel
EL2   HOST KERNEL + KVM

The host kernel runs AT EL2. The guest kernel keeps EL1 to
itself. Nothing is shared, so nothing must be swapped.

Entering the guest becomes:
    restore the guest's EL1 registers     (they were not in
                                           use by anyone)
    write VTTBR_EL2, HCR_EL2
    ERET

No host save. No host restore. And KVM is ordinary kernel
code running at the level that can do the switch, so the
split disappears.
```

The trick: register redirection

A kernel written for EL1 is full of `msr sctlr_el1, x0` and `mrs x0, ttbr1_el1`. You cannot rewrite all of it, and you should not have to. VHE's insight is to make those instructions *mean something different* when executed at EL2 with `HCR_EL2.E2H` set.

```
With HCR_EL2.E2H = 1, executing AT EL2:

  the instruction        actually accesses
  ----------------       -----------------
  MSR SCTLR_EL1, x0  ->  SCTLR_EL2
  MRS x0, TTBR0_EL1  ->  TTBR0_EL2
  MSR TTBR1_EL1, x0  ->  TTBR1_EL2   (new in VHE - EL2 gained
                                      a second TTBR precisely
                                      so a host kernel with a
                                      high half can live here)
  MSR VBAR_EL1, x0   ->  VBAR_EL2
  MSR TCR_EL1, x0    ->  TCR_EL2     (whose format changed to
                                      match TCR_EL1's)
  MRS x0, CNTV_CTL_EL0 -> CNTHV_CTL_EL2

And when EL2 genuinely needs the real EL1 register - which
KVM does, to load guest state - there is a new suffix:

  MSR SCTLR_EL12, x0 ->  the REAL SCTLR_EL1  (the guest's)
  MSR TTBR0_EL12, x0 ->  the REAL TTBR0_EL1
  MSR VBAR_EL12, x0  ->  the REAL VBAR_EL1

So: an unmodified EL1 kernel runs unmodified at EL2, and the
hypervisor uses the _EL12 forms to touch guest state
explicitly. Two hundred lines of KVM change; the other
twenty million lines of Linux do not.
```

> **WHY `TTBR1_EL2` HAD TO BE INVENTED**
>
> Original EL2 had only `TTBR0_EL2` — one translation base, because a purpose-built hypervisor uses a single flat address space. Linux splits its address space in half: user mappings via `TTBR0`, kernel mappings via `TTBR1`. Without a second base register, Linux simply could not run at EL2. VHE adds it, and reformats `TCR_EL2` to match `TCR_EL1` so the existing kernel code that programs it still works.

TGE: whose EL0 is it?

With the host at EL2, a question appears that did not exist before: when an application traps, does it belong to the host or to a guest? `HCR_EL2.TGE` — Trap General Exceptions — is the answer, and it is flipped on every guest entry and exit.

```
TGE = 1   "EL0 is the HOST's"
          - EL0 exceptions route to EL2
          - EL0 uses the EL2 translation regime
            (TTBR0_EL2, via the E2H remap)
          - stage 2 is effectively disabled for EL0/EL1
          - the virtual timer is not offset

          This is the state while the host is running its
          own processes. An ordinary syscall from an ordinary
          host process arrives at EL2, where the host kernel
          lives, and everything is normal.

TGE = 0   "EL0/EL1 belong to a GUEST"
          - EL0 and EL1 exceptions route to EL1 (the guest
            kernel handles its own syscalls - no exit!)
          - stage 2 applies
          - CNTVOFF_EL2 applies

The sequence around KVM_RUN:

  ... host running, E2H=1, TGE=1 ...
  restore guest EL1 state via _EL12 registers
  HCR_EL2: TGE=0, VM=1, IMO/FMO/AMO=1, ...
  write VTTBR_EL2
  ERET  -> guest at EL1
  ... guest runs; its own syscalls never leave EL1 ...
  trap -> EL2
  HCR_EL2: TGE=1, VM=0
  ... host running again ...

That single bit is the whole context switch, conceptually.
Everything else is bookkeeping.
```

What it is worth

Operation	nVHE	VHE	Why
Save/restore host EL1 sysregs per exit	~25 registers each way	**zero**	Host uses EL2 registers; nothing collides
Trap handled entirely in EL2	Must bounce to EL1 for anything non-trivial	Handled in place	KVM's real code is already at EL2
Round-trip cost (relative)	1.0	~0.4–0.6	Roughly halved on most implementations
KVM code structure	Split into hyp and kernel halves	Ordinary kernel code	No separate address space, no `__hyp_text` restrictions
Debuggability of the hypervisor	Very poor — no printk in hyp	Normal	It is just the kernel

Finding out which mode you are in

```
$ dmesg | grep -i kvm
[    0.512430] kvm [1]: Hyp mode initialized successfully
[    0.512461] kvm [1]: VHE mode initialized successfully      <-- VHE
[    0.512480] kvm [1]: vgic interrupt IRQ9
[    0.512501] kvm [1]: Timer IRQ4

# The other possibilities:
#   "Hyp mode initialized successfully"  alone          -> nVHE
#   "Protected nVHE mode initialized successfully"      -> pKVM
#   "HYP mode not available"                            -> booted at EL1

# Is the hardware capable at all?  ID_AA64MMFR1_EL1.VH != 0
$ sudo dd if=/dev/mem 2>/dev/null   # no - use the kernel's view:
$ dmesg | grep -i 'CPU features'

# Force nVHE on a VHE-capable machine, for comparison:
$ sudo grubby --update-kernel=ALL --args="kvm-arm.mode=nvhe"
# or on the command line:   kvm-arm.mode=nvhe
# Valid values: none, nvhe, protected, nested

# Then measure the same workload both ways. The difference is
# almost entirely in exit-heavy workloads - I/O and interrupts -
# and nearly invisible in a CPU-bound guest.
```

Protected KVM, which needs nVHE

```
   pKVM inverts the trust relationship:

     normal KVM   the host kernel can read all guest memory.
                  It must - it emulates devices, migrates VMs,
                  and swaps guest pages.

     pKVM         a small, auditable hypervisor at EL2 owns
                  stage 2. The HOST is itself constrained:
                  guest memory is unmapped from the host unless
                  the guest explicitly shares it.

   Threat model: protect a guest from a compromised host kernel.
   Used on Android for the Android Virtualization Framework.

   And it deliberately uses the nVHE split, because that split
   is exactly what you want when the EL2 component must be
   small enough to audit:

     EL2   pKVM hypervisor    ~10k lines, no dynamic memory,
                              its own page tables, owns stage 2
     EL1   host Linux         large, untrusted, asks EL2 for
                              permission to do things
     EL1   guest Linux        protected from the host

   So VHE is not universally "better". It is better for the
   common case of a trusted host. When the host is part of the
   threat model, the separation nVHE forces is the point.
```

> **NESTED VIRTUALISATION NEEDS BOTH**
> Running a hypervisor *inside* a guest means emulating EL2 for that guest — including its `E2H` and `TGE` bits, and the register redirection they imply. Chapter 20 covers it; the short version is that VHE makes nested virtualisation harder to implement and much faster when it works.

Writing code that runs at either level

```
/* Linux has to cope with running at EL1 or EL2 from the same
   binary. It does this with alternatives patched at boot.   */

/* arch/arm64/include/asm/kvm_hyp.h, simplified: */

#define read_sysreg_el1(r)   read_sysreg_elx(r, _EL1, _EL12)
#define write_sysreg_el1(v,r) write_sysreg_elx(v, r, _EL1, _EL12)

/* which expands, via an ALTERNATIVE, to either

     mrs x0, sctlr_el1     when running nVHE  (we are at EL2,
                                               EL1 regs are real)
   or
     mrs x0, sctlr_el12    when running VHE   (we are at EL2 with
                                               E2H=1, so _EL1
                                               would hit EL2)

   patched once at boot by the alternatives framework based on
   ARM64_HAS_VIRT_HOST_EXTN.                                    */

/* The lesson for your own EL2 code: NEVER write a bare
   `msr sctlr_el1, x0` in code that might run under VHE. It
   will silently configure the hypervisor instead of the guest,
   and the failure looks like the guest ignoring your writes. */
```

How it shows up

Symptom	Cause
Guest ignores system-register state you set	Wrote `*_EL1` from EL2 under VHE — it went to the EL2 register instead of the guest's
Host kernel corrupts itself when a VM starts	Same bug in reverse: wrote `*_EL12` believing it targeted EL2, or clobbered `TTBR1_EL2`
Host processes stop working while a VM runs	`TGE` not restored to 1 on exit — host EL0 exceptions are routing to the guest's EL1
Exit costs double on new hardware after a kernel change	Fell back to nVHE — check for `kvm-arm.mode=nvhe` on the command line
`VHE mode initialized` absent on capable silicon	Booted at EL1, or firmware left `HCR_EL2.E2H` in a state the kernel rejected
Guest timer runs at the host's rate	`CNTVOFF_EL2` only applies with `TGE=0`; the bit was left set
pKVM refuses to initialise	pKVM requires the nVHE code path plus firmware support; it is not available in every configuration
Hypervisor code faults only under VHE	An `__hyp_text` assumption about a separate EL2 address space that no longer holds

Check yourself

- ✓ Explain the register collision that makes nVHE expensive, and count the registers involved.
- ✓ Explain what `HCR_EL2.E2H` does to an `MSR SCTLR_EL1` executed at EL2.
- ✓ Say why `TTBR1_EL2` had to be added to the architecture.
- ✓ Explain the `_EL12` register suffix and when a hypervisor uses it.
- ✓ Describe what `TGE` selects and where in the run loop it is toggled.
- ✓ Give three things VHE improves besides raw speed.
- ✓ Determine from a running system whether it is in VHE, nVHE or pKVM mode.
- ✓ Explain why pKVM deliberately uses nVHE, and what threat that addresses.
- ✓ Describe the failure mode of writing a bare `msr sctlr_el1` in EL2 code under VHE.

Interrupt virtualisation and the vGIC

A guest must be able to receive an interrupt, acknowledge it, and signal end-of-interrupt — all without trapping, because interrupts are frequent and traps are expensive. The GIC provides a second, virtual CPU interface for exactly this, and a set of list registers through which the hypervisor stages the interrupts it wants the guest to believe in.

The problem

```
An interrupt arrives from real hardware.

1. It must go to the HYPERVISOR first - the guest is not
   allowed to see a wire it does not own, and the interrupt
   may belong to the host, or to a different guest, or to a
   vCPU that is not currently running.

   HCR_EL2.IMO = 1 routes physical IRQ to EL2.

2. The hypervisor decides it belongs to guest G, vCPU 3.

3. If vCPU 3 is running right now: inject it and resume.
   If not: mark it pending; inject when that vCPU next runs.
   (And if the vCPU is halted in WFI: wake its thread.)

4. The guest then does the NORMAL interrupt dance:
     read IAR      to acknowledge and learn the INTID
     run its handler
     write EOIR    to signal completion
     maybe write DIR to deactivate

   Each of those is a register access. If each one trapped,
   a busy guest would exit thousands of times a second just
   to service its own interrupts.

The GIC's answer: give the guest its own CPU interface
registers, which are REAL hardware, and let it drive them
without trapping at all.
```

THE DIVISION OF LABOUR

The *distributor* and *redistributor* (routing, priorities, enables) are emulated in software, because a guest reprogramming them is rare. The *CPU interface* (acknowledge, EOI, priority mask) is virtualised in hardware, because a guest touches it on every single interrupt. Emulate the cold path; virtualise the hot path.

The hardware pieces

Piece	GICv2 name	GICv3 name	Who drives it
Physical CPU interface	`GICC_*`	`ICC_*_EL1`	The hypervisor (and the host)
Virtual CPU interface	`GICV_*`	`ICV_*_EL1`	**The guest**, directly, with no trap
Virtualisation control	`GICH_*`	`ICH_*_EL2`	The hypervisor
List registers	`GICH_LR<n>`	`ICH_LR<n>_EL2`	The hypervisor writes; hardware and guest update

Piece	GICv2 name	GICv3 name	Who drives it
Distributor	`GICD_*`	`GICD_*` + redistributors	Emulated in software

```
GICv3, the essential trick:

The guest executes   MRS x0, ICC_IAR1_EL1
...but HCR_EL2.IMO = 1, so at EL1 that instruction is
REDIRECTED by hardware to ICV_IAR1_EL1 - the VIRTUAL
interface. No trap. The guest reads a virtual INTID out of
real silicon.

Same for ICC_EOIR1_EL1 -> ICV_EOIR1_EL1
         ICC_PMR_EL1   -> ICV_PMR_EL1
         ICC_HPPIR1_EL1-> ICV_HPPIR1_EL1

It is the same redirection idea as VHE's E2H, applied to the
interrupt controller. The guest's driver is unmodified; the
registers it touches are simply different ones.

Meanwhile the hypervisor, at EL2, uses ICH_* to decide what
the virtual interface will report.
```

List registers: the interface between the two

```
ICH_LR<n>_EL2 - typically 4 to 16 of them per CPU.
Each one describes ONE interrupt the guest should see.

 63 62 61 60 59 58 56 55       48 47      32 31          0
+--+--+--+--+-----+-----+---------+---------+------------+
|State|HW|Grp| Pri |     | pINTID            | vINTID     |
+--+--+--+--+-----+-----+---------+---------+------------+

State [63:62]   00 invalid    01 pending
                10 active     11 pending+active
HW    [61]      0 = purely virtual (software generated)
                1 = HARDWARE-BACKED: this virtual interrupt
                    corresponds to physical interrupt pINTID
Group [60]      Group 0 (FIQ) or Group 1 (IRQ)
Priority[55:48]
pINTID[41:32]   the PHYSICAL interrupt this maps to
vINTID[31:0]    what the guest will read from ICV_IAR1_EL1

THE HW BIT IS THE CLEVER PART.

With HW=1, when the guest writes ICV_EOIR1_EL1 with vINTID,
the hardware ALSO deactivates physical interrupt pINTID.
The guest's EOI directly deactivates the real device's
interrupt line - with no trap, no exit, no hypervisor
involvement whatsoever.

That is what makes device PASSTHROUGH performant. Without
it, every EOI would be an exit, and a 100k-interrupts-per-
second NIC would be unusable.
```

One interrupt, end to end

```
/* Case A: an EMULATED device (virtio) wants to interrupt vCPU 2 */

  1. QEMU/vhost decides the virtqueue has data.
  2. It asks KVM to raise virtual INTID 32 on vCPU 2:
        ioctl(vmfd, KVM_IRQ_LINE, {irq: 32, level: 1})
     or, faster, it kicks an irqfd that KVM has pre-bound.
  3. KVM marks INTID 32 pending in its software vGIC model.
  4. If vCPU 2 is running on another physical CPU, KVM sends an
     IPI to force it to exit so the LR can be written.
     If it is halted in WFI, KVM wakes the thread.
  5. On the next guest entry, KVM writes:
        ICH_LR0_EL2 = pending | Group1 | prio | vINTID=32
        HW = 0        (there is no physical interrupt behind it)
  6. ERET. The virtual CPU interface now signals IRQ to EL1.
  7. Guest reads ICV_IAR1_EL1 -> 32.  NO TRAP.
     Guest handles it.
     Guest writes ICV_EOIR1_EL1 = 32. NO TRAP. The LR state
     goes to invalid.
  8. On the next exit, KVM reads back the LRs to learn the
     interrupt completed.

  Exits used: one to inject (maybe zero if the vCPU was already
  going to exit). Zero for the guest's own handling.


/* Case B: a PASSED-THROUGH device (chapter 16)              */

  1. Real device raises physical INTID 137.
  2. HCR_EL2.IMO routes it to EL2. The host's handler runs -
     it does NOT know how to service the device, and must not.
  3. KVM looks up the mapping: physical 137 -> vCPU 2, virtual 137.
  4. It writes:
        ICH_LR0_EL2 = pending | HW=1 | pINTID=137 | vINTID=137
     and does NOT deactivate the physical interrupt.
  5. Guest reads ICV_IAR1_EL1 -> 137, services the real device
     through its passed-through MMIO, writes ICV_EOIR1_EL1.
  6. Because HW=1, the hardware deactivates physical 137 too.

  Exits used: one, for the physical interrupt itself. The
  entire guest-side handling is trap-free.

/* The remaining exit in case B is what GICv4 direct injection
   removes - see chapter 11.                                 */
```

Maintenance interrupts

```
/* The hypervisor has only N list registers (often 4). A guest
   with more than N interrupts pending needs the hypervisor to
   refill them - but the hypervisor is not running.

   ICH_HCR_EL2 configures a MAINTENANCE INTERRUPT that fires
   when the LR state changes in ways you asked about:      */

   ICH_HCR_EL2.UIE   Underflow: fewer than 2 LRs are valid.
                     "I am running out of queued interrupts,
                      give me more."
   ICH_HCR_EL2.LRENPIE  A list register with EOIcount != 0
                     needs attention.
   ICH_HCR_EL2.NPIE  No pending interrupts.
   ICH_HCR_EL2.VGrp0EIE / VGrp1EIE
                     the guest enabled or disabled a group -
                     the hypervisor may want to know.

/* The maintenance interrupt is a normal physical PPI (KVM
   prints it at boot: "vgic interrupt IRQ9"). When it fires:

     -> exit to EL2
     -> drain completed interrupts out of the LRs
     -> load more pending interrupts in
     -> resume

   With enough LRs and a sane workload this is rare. With 4 LRs
   and an interrupt storm it becomes the dominant exit reason,
   and you will see it in perf as a flood of maintenance
   interrupts. That is the signal to reduce interrupt rate
   (coalescing, NAPI) rather than to tune the hypervisor.   */
```

> **THE DISTRIBUTOR IS EMULATED, AND THAT IS A TRAP YOU WILL HIT**
>
> A guest writing `GICD_ISENABLER`, changing a priority, or re-routing an interrupt causes an MMIO exit into the software vGIC model. Guests that reconfigure interrupts constantly — some RTOSes do this per operation — perform terribly under virtualisation for this reason alone. Configure once at init, as Linux does.

GICv2 versus GICv3 for guests

	GICv2	GICv3
Guest CPU interface	MMIO region (`GICV`) mapped over where the guest expects `GICC`	System registers, redirected by `HCR_EL2.IMO`
Max CPUs	8	Effectively unlimited (affinity routing)
Interrupt IDs	SGI 0–15, PPI 16–31, SPI 32–1019	Adds LPIs (8192+) via the ITS
Cost of guest EOI	Uncached MMIO write, but no trap	System register write — cheaper still
KVM device type	`KVM_DEV_TYPE_ARM_VGIC_V2`	`KVM_DEV_TYPE_ARM_VGIC_V3`
QEMU option	`-machine virt,gic-version=2`	`gic-version=3` (or `host` , or `max`)

```
# Which one is in use?

# On the host:
$ dmesg | grep -i vgic
[    0.512480] kvm [1]: vgic interrupt IRQ9
$ dmesg | grep -i 'GICv'
[    0.000000] GICv3: GIC: Using split EOI/Deactivate mode

# Inside the guest:
$ dmesg | grep -i gic
[    0.000000] GICv3: 288 SPIs implemented
[    0.000000] GICv3: CPU0: found redistributor 0 region 0

$ cat /proc/interrupts
           CPU0       CPU1
  3:       1204       1188   GICv3  27 Level  arch_timer
 25:          0          0   GICv3  33 Level  virtio0
 26:      88104          0   GICv3  34 Level  virtio1

# A guest that shows "GICv2" while the host is GICv3 is running
# a legacy-mode vGIC - possible, and slower. Prefer matching.
```

How it shows up

Symptom	Cause
Guest never receives interrupts at all	`HCR_EL2.IMO` clear, so the virtual interface is not enabled and `ICC_*` is not redirected
Guest receives an interrupt once, then never again	Guest's EOI not reflected back into the hypervisor's model — LRs not synced on exit
Passed-through device stops after one interrupt	`HW` bit not set in the list register, so the physical interrupt was never deactivated
Interrupt storm of maintenance interrupts	More pending interrupts than list registers; guest interrupt rate too high
vCPU in `WFI` never wakes	Injection path marks pending but does not kick the blocked thread
Interrupt delivered to the wrong vCPU	Affinity routing mismatch between the emulated distributor and the vCPU's `VMPIDR_EL2`
Guest boots on GICv2 but not GICv3 (or vice versa)	Guest device tree describes a different GIC version than the one configured
Terrible performance in an RTOS guest	The guest reprograms the distributor per operation; every write is an MMIO exit
Timer interrupt lost when a vCPU migrates	Per-CPU (PPI) state not migrated with the vCPU

Check yourself

- ✓ Explain why the CPU interface is virtualised in hardware while the distributor is emulated.
- ✓ Describe what `HCR_EL2.IMO` does to a guest's `ICC_IAR1_EL1` access.
- ✓ Draw a list register and name the state, HW, pINTID and vINTID fields.
- ✓ Explain exactly what the `HW` bit causes to happen on guest EOI, and why passthrough needs it.
- ✓ Trace an emulated-device interrupt from device model to guest handler, counting exits.
- ✓ Trace a passed-through interrupt the same way and say where the remaining exit is.
- ✓ Explain what a maintenance interrupt is and name two conditions that raise one.

- ✓ Give two reasons a guest might be slow that are visible only in the vGIC.
- ✓ Determine from a running host and guest which GIC version each is using.

LPIs, the ITS and direct injection

The vGIC of the previous chapter still costs one exit per physical interrupt. GICv4 removes even that: a device's message-signalled interrupt can be delivered straight into a running guest with the hypervisor asleep. Getting there requires understanding LPIs, the ITS translation tables, and what "doorbell" means.

LPIs: interrupts as messages

```
The three older interrupt kinds are WIRES:

  SGI   0-15     software generated, core to core
  PPI   16-31    private per CPU (timers, PMU)
  SPI   32-1019  shared peripheral - a physical signal
                 routed into the distributor

Their state lives in distributor registers, one bit per
interrupt. That is fine for 1000 interrupts and hopeless for
a NIC with 512 queues on a 128-core machine.

LPI  8192+       Locality-specific Peripheral Interrupt.
                 NOT a wire. A MEMORY WRITE.

A device raises an LPI by writing an ID to a doorbell
address:

    device -> AXI write of value 0x2A to GITS_TRANSLATER
              (an address inside the ITS)

The ITS looks up "which device is this, what does its
EventID 0x2A mean, which CPU should get it" and delivers.

Consequences:
  * state lives in DDR tables, not in registers -> millions
    of interrupts are affordable
  * routing is per-interrupt and reprogrammable
  * this is exactly PCIe MSI/MSI-X, which is also "a device
    writes a value to an address"
```

The ITS and its tables

```
ITS = Interrupt Translation Service. A little engine that
turns (DeviceID, EventID) into (LPI number, target
redistributor).

  device does an MSI write
       |
       |  DeviceID comes from the interconnect - it is the
       |  same identity the SMMU uses as a StreamID
       v
  +-------------------------------------------------+
  |  ITS                                            |
  |                                                 |
  |  Device Table:  DeviceID -> ITT pointer         |
  |         |                                       |
  |         v                                       |
  |  Interrupt Translation Table (per device):      |
  |         EventID -> (LPI ID, Collection ID)      |
  |         |                                       |
  |         v                                       |
  |  Collection Table: Collection -> Redistributor  |
  +-------------------------------------------------+
       |
       v
  redistributor sets the pending bit in the
  LPI Pending Table (in DDR), then signals the CPU

THREE indirections, all through tables in normal memory that
software allocates and the ITS walks. You program them with
COMMANDS, not register writes:

  MAPD    map a DeviceID to an ITT
  MAPTI   map (DeviceID, EventID) -> (LPI, Collection)
  MAPC    map a Collection -> a redistributor
  INV     invalidate a cached entry after changing config
  SYNC    wait for prior commands to complete
  DISCARD unmap an event
  MOVI    move an interrupt to another collection (i.e.
          another CPU) - this is IRQ affinity for LPIs
```

DEVICEID IS STREAMID

The identity the ITS uses to look up a device is the same bus-level identity the SMMU uses to decide which translation tables apply. One identity, two consumers: the SMMU constrains what the device may read and write, the ITS constrains what it may interrupt. On a well-designed SoC they are configured together, and a device that is passed through to a guest gets both a stage-2 translation and an ITS mapping.

Virtualising the ITS: GICv3

```
/* Under GICv3 the ITS is EMULATED. The guest thinks it owns
   one; KVM models it.                                       */

  guest writes an ITS command (MAPTI) into its command queue
     -> MMIO exit into KVM's vgic-its model
     -> KVM records "guest LPI 8192 belongs to vCPU 3"
     -> if a real device is behind it (passthrough), KVM ALSO
        programs the physical ITS so the real device's MSI
        lands on the host

  device raises the MSI
     -> physical LPI delivered to the HOST
     -> host interrupt handler runs
     -> looks up: this physical LPI maps to guest LPI 8192 on vCPU 3
     -> inject into a list register (chapter 10)
     -> if vCPU 3 is running elsewhere, IPI it to force an exit
     -> vCPU 3 re-enters and sees the interrupt

  COST: one host interrupt + one exit + one entry, per device
  interrupt. For a 10GbE NIC at 300k interrupts/sec that is
  300k exits/sec, and it dominates everything.

/* This is why interrupt coalescing matters so much more in a
   VM than on bare metal, and why NAPI polling wins.        */
```

GICv4: direct injection

```
The insight: if the ITS already knows which vCPU an LPI
belongs to, and the redistributor already knows which vCPU is
RESIDENT on this physical CPU, then the hardware can deliver
straight to the guest.

GICv4 adds:

  * VPE (Virtual Processing Element) - the hardware's notion
    of a vCPU. Each has an ID and a pair of tables:
        Virtual LPI Pending Table
        Virtual LPI Configuration Table
    both in normal memory.

  * New ITS commands:
        VMAPP    map a VPE to a redistributor
        VMAPTI   map (DeviceID,EventID) -> virtual LPI on a VPE
        VMOVP    move a VPE to another redistributor
        VINVALL  invalidate cached virtual LPI config
        VSYNC

  * GICR_VPROPBASER / GICR_VPENDBASER - the redistributor
    registers that say "VPE X is currently resident here"

Then:

  device MSI write
       |
       v
  ITS translates to a VIRTUAL LPI on VPE 3
       |
       +--- is VPE 3 resident on some redistributor?
       |
       |    YES -> set the pending bit in the VIRTUAL pending
       |           table and signal the CPU directly.
       |           The guest takes the interrupt.
       |           ZERO EXITS. The hypervisor never runs.
       |
       |    NO  -> the vCPU is not scheduled. Send a
       |           DOORBELL interrupt: a normal physical LPI
       |           to the host saying "VPE 3 has work".
       v
  host wakes the vCPU thread, makes VPE 3 resident,
  and the pending virtual LPI is delivered on entry.

GICv4.1 adds direct virtual SGIs too, so a guest sending an
IPI from vCPU 0 to vCPU 1 need not exit either.
```

> **THE DOORBELL IS THE WHOLE TRICK**
> Direct injection would be unsafe if the target vCPU were descheduled — the interrupt would sit in memory unnoticed. The doorbell converts exactly that case back into a normal host interrupt. So the fast path costs nothing and the slow path costs what it used to. This "fast path in hardware, slow path via a wake-up signal" pattern shows up again in vhost and in virtio notification suppression.

Finding out what you have

```
# Host: is there an ITS at all?
$ dmesg | grep -i its
[    0.000000] GICv3: GIC: Using split EOI/Deactivate mode
[    0.000000] ITS [mem 0x08080000-0x0809ffff]
[    0.000000] ITS@0x0000000008080000: allocated 65536 Devices
               @880000000 (indirect, esz 8, psz 64K, shr 1)
[    0.000000] ITS: using cache flushing for cmd queue
[    0.000000] GICv4 support enabled                        <-- the line you want

# GICv4.1 will say:
[    0.000000] GICv4.1 support enabled

# Is direct injection actually in use for a given VM?
$ sudo cat /sys/kernel/debug/kvm/*/vgic-state 2>/dev/null | head -30

# Per-interrupt counts inside the guest tell you the rate:
$ cat /proc/interrupts | grep -E 'virtio|mlx|ixgbe'

# Host-side, watch for the exits that direct injection removes:
$ sudo perf stat -e kvm:kvm_exit -a sleep 5
# then compare with and without vfio passthrough of the device.

# QEMU: request an ITS in the guest
$ qemu-system-aarch64 -M virt,gic-version=3,its=on ...
# its=off removes it; some guests then fall back to a wired
# interrupt path, and PCIe MSI-X will not work.
```

Memory the ITS demands of you

```
/* The tables are not optional and their sizes surprise people. */

  Device Table        one entry per DeviceID the ITS can see.
                      GITS_TYPER.Devbits says how wide DeviceID
                      is. 16 bits -> 65536 entries. At 8 bytes
                      each that is 512 KB, contiguous, unless
                      you use the INDIRECT (two-level) form.

  ITT (per device)    one entry per EventID that device can
                      raise. A NIC with 2048 MSI-X vectors needs
                      a 2048-entry ITT.

  LPI Config Table    one BYTE per LPI, shared by all
                      redistributors: priority + enable.
                      1M LPIs -> 1 MB.

  LPI Pending Table   one BIT per LPI, PER REDISTRIBUTOR.
                      1M LPIs x 128 CPUs -> 16 MB.

  Virtual equivalents (GICv4) - per VPE, so multiply by the
  number of vCPUs in the system.

/* And they must be:
     - physically contiguous (or indirect)
     - aligned as GITS_BASER demands (often 64 KB)
     - coherent, or explicitly cleaned - GITS_BASER.InnerCache
       tells you whether the ITS snoops. If it does not, every
       table update needs a DC CVAC before the SYNC command.

   "ITS: using cache flushing for cmd queue" in dmesg means
   exactly that: your ITS is not coherent with the CPU caches. */
```

How it shows up

Symptom	Cause
PCIe device in a guest never receives MSI-X	No ITS in the guest (`its=off`), or the guest's ITS command queue was never enabled
Passthrough NIC works but at terrible packet rates	No GICv4, so every device interrupt costs an exit; enable coalescing or accept the cost
Interrupts arrive on the wrong CPU after affinity change	`MOVI` / `VMOVP` issued without a following `SYNC` , or a stale cached ITS entry (missing `INV`)
ITS silently drops interrupts after config changes	Tables updated but not cleaned to memory on a non-coherent ITS
Boot fails allocating ITS tables	Device table too large for contiguous allocation — needs the indirect two-level form
A vCPU misses interrupts while descheduled	Doorbell not configured, so nothing wakes the vCPU thread
Guest sees interrupts belonging to another guest	DeviceID not isolated — two devices behind one DeviceID, or the SMMU StreamID mapping does not match the ITS mapping
`ITS command queue timeout`	The ITS stalled — usually a command referencing an unmapped device or an unaligned table

Check yourself

- ✓ Explain how an LPI differs from an SPI, mechanically.
- ✓ Describe the three table lookups an ITS performs and what each maps.
- ✓ Name five ITS commands and say what each does.
- ✓ Explain the relationship between DeviceID, StreamID and PCIe requester ID.
- ✓ Count the exits per device interrupt for a passed-through device under GICv3.
- ✓ Explain what a VPE is and which two tables belong to it.
- ✓ Describe direct injection and state precisely when it does not apply.
- ✓ Explain what a doorbell interrupt is and why the design needs one.
- ✓ Estimate the memory required for LPI configuration and pending tables on a 128-CPU system with 1M LPIs.
- ✓ Determine from a host's boot log whether GICv4 is available and in use.

Timer virtualisation

Time is the illusion hardest to maintain, because a guest can measure it against the outside world. ARM's answer is a single 64-bit offset register that shifts a guest's view of the counter, plus a dedicated virtual timer per exception level — and a set of compromises about what happens while a vCPU is not running.

One counter, several views

```
There is exactly ONE physical counter in the system: a
free-running, always-on, fixed-frequency counter, shared by
every core. Software reads it through several windows.

CNTPCT_EL0   physical count   = the counter, raw
CNTVCT_EL0   virtual count    = CNTPCT_EL0 - CNTVOFF_EL2

                                        ^^^^^^^^^^^
                          ONE register, per vCPU, 64-bit,
                          writable only at EL2. That is the
                          entire mechanism.

Timers (comparators) come in matching flavours:

  EL1 physical timer   CNTP_CTL_EL0  CNTP_CVAL_EL0  CNTP_TVAL_EL0
  EL1 virtual timer    CNTV_CTL_EL0  CNTV_CVAL_EL0  CNTV_TVAL_EL0
  EL2 hypervisor timer CNTHP_CTL_EL2 CNTHP_CVAL_EL2
  EL2 virtual timer    CNTHV_*_EL2   (VHE)
  EL3 secure timer     CNTPS_*_EL1

Each has its own interrupt (a PPI):
  30  EL1 physical timer
  27  EL1 virtual timer     <-- the one a KVM guest uses
  26  EL2 hypervisor timer
  29  EL3 secure timer

A guest kernel under KVM is told, via its device tree, to use
the VIRTUAL timer. It then reads CNTVCT_EL0 with no trap at
all, and gets a value shifted by CNTVOFF_EL2.
```

WHY THE OFFSET MUST EXIST

A guest that boots two hours after the host expects its counter to start near zero, not at the host's uptime. More importantly, a guest that is suspended and resumed, or migrated to another machine, must not see time jump backwards or leap forwards — its scheduler, its watchdogs and its TCP stack all assume monotonic, roughly-real time. `CNTVOFF_EL2` lets the hypervisor rebase the guest's clock without touching a single guest instruction.

What the hypervisor does with CNTVOFF

```
/* At VM creation: */
    vcpu->cntvoff = read_cntpct();      /* guest time starts at 0 */

/* On every guest entry: */
    write_sysreg(vcpu->cntvoff, cntvoff_el2);

/* All vCPUs of one VM MUST share the same offset, or the guest
   sees different cores disagreeing about the time - which
   breaks its scheduler, its clocksource watchdog and any
   lock-free algorithm using timestamps.

   KVM enforces this: CNTVOFF is per-VM, not per-vCPU.       */

/* On resume from a host suspend: */
    /* The physical counter kept running (or did not - it is
       "always on" but the SoC may have powered it). Either way
       the guest must not observe a jump. Adjust:            */
    vcpu->cntvoff += (now - suspend_started);

/* On live migration: */
    /* The destination machine's counter has a completely
       different value AND possibly a different frequency.
       CNTFRQ_EL0 is not writable per-guest, which is a real
       architectural gap - migration between machines with
       different counter frequencies is not supported. Check
       CNTFRQ on both ends before you promise migration.     */
```

Which timer belongs to whom

Timer	Used by	Trapped?	Notes
EL1 virtual (`CNTV_*`)	The guest	No	The fast path. Guest programs its own comparator; hardware raises PPI 27; the vGIC delivers it
EL1 physical (`CNTP_*`)	Bare-metal EL1 software; a guest that was misconfigured	**Yes**, via `CNTHCTL_EL2.EL1PCEN`	Must be trapped and emulated, because the physical timer's interrupt is not the guest's to own
EL2 hypervisor (`CNTHP_*`)	The hypervisor	n/a	Preemption timer: how KVM takes a vCPU away when its slice expires
Counter reads (`CNTVCT_EL0`)	Guest user space and kernel	No	Offset in hardware. This is why `clock_gettime()` in a guest is as fast as on bare metal
`CNTPCT_EL0`	Anyone	Optionally, via `CNTHCTL_EL2.EL0PCTEN` / `EL1PCTEN`	Trapping it hides the host's real uptime; leaving it open lets a guest detect virtualisation

```
/* CNTHCTL_EL2 - who may touch what, from EL1 and EL0.       */

  Without VHE (or with TGE=0):
    EL1PCTEN  [0]  EL0/EL1 may read CNTPCT_EL0
    EL1PCEN   [1]  EL0/EL1 may use the EL1 PHYSICAL timer
    EVNTEN    [2]  event stream enable
    EVNTDIR   [3]  event stream edge
    EVNTI     [7:4] which counter bit drives the event stream

/* KVM's normal setting for a guest:

     EL1PCTEN = 1   let the guest read CNTPCT - it is only the
                    raw counter, and hiding it costs a trap on
                    a hot path for very little benefit
     EL1PCEN  = 0   TRAP the physical timer. The guest should be
                    using the virtual one; if it insists on the
                    physical one, KVM emulates it on top of the
                    virtual timer.

   A guest device tree that lists the physical timer interrupt
   instead of the virtual one produces a guest that works but
   traps on every timer programming operation. Symptom: high
   exit count with EC=0x18 on CNTP registers.               */
```

The hard part: time while you are not running

```
   A guest sets a timer for 1 ms from now, then its vCPU is
   descheduled for 20 ms because the host is busy.

   The virtual timer fires - in hardware - while the vCPU is
   not resident. Who notices?

   ANSWER: the hypervisor must arrange to notice. KVM does this
   by putting the guest's timer deadline into a HOST timer
   (an hrtimer) whenever the vCPU is descheduled:

     vCPU blocks / is preempted
        |
        v
     read guest CNTV_CVAL, convert to host time
     arm a host hrtimer for that instant
     save and DISABLE the guest's timer in hardware
        |
        ... host runs other things ...
        |
     hrtimer fires -> mark virtual PPI 27 pending for that vCPU
                   -> wake the vCPU thread
        |
        v
     vCPU is scheduled again
        |
        v
     restore guest timer state, inject the pending interrupt

   The guest observes: "my 1 ms timer fired after 20 ms".
   That is LATE, not WRONG - and lateness is unavoidable when
   you overcommit. But it must never be EARLY, and it must
   never be LOST.
```

> **THE RTOS PROBLEM**
> An RTOS guest with a 100 µs deadline cannot tolerate 20 ms of scheduling delay, and no amount of hypervisor cleverness fixes it — the physical CPU was genuinely doing something else. The answer is not a better timer path; it is to stop sharing that CPU: pin the vCPU, isolate the core (`isolcpus`, `nohz_full`), give it a real-time scheduling class, or use a partitioning hypervisor that never time-shares in the first place.

Steal time: telling the guest the truth

```
/* A guest's own accounting is a lie under overcommit: it
   thinks it used 100 ms of CPU when 60 ms of that was spent
   descheduled. Its load average, its CPU-frequency governor
   and its scheduler all draw wrong conclusions.

   The paravirtual fix: the hypervisor publishes, in a shared
   page, how much time this vCPU spent NOT running.        */

  struct pvclock_vcpu_stolen_time {
          __le32 revision;
          __le32 attributes;
          __le64 stolen_time;      /* nanoseconds, monotonic  */
          /* padded to 64 bytes                              */
  };

/* Discovered via SMCCC:
     HV_PV_TIME_FEATURES  -> is it there?
     HV_PV_TIME_ST        -> give me the IPA of my structure   */

/* In the guest: */
$ cat /proc/stat | head -2
cpu  1204 0 388 90112 22 0 8 1877 0 0
                               ^^^^
                               steal time, in USER_HZ

$ vmstat 1
procs -----------memory---------- ---system-- ------cpu-----
 r  b   swpd   free  buff  cache   in   cs us sy id wa st
 2  0      0 743210  1122  88240  912 1840 41  6 22  0 31
                                                     ^^
                                          31% stolen - this
                                          guest is starved

/* st consistently above a few percent means the host is
   oversubscribed. It is the single most useful number for
   diagnosing "the VM is slow" complaints, and it points at the
   host, not the guest.                                    */
```

Checking the timer setup

```
# Guest: which timer is it actually using?
$ cat /proc/interrupts | grep timer
  3:      12045      11982   GICv3  27 Level  arch_timer
                                    ^^ 27 = VIRTUAL timer. Good.
                                    30 would be the physical one.

$ cat /sys/devices/system/clocksource/clocksource0/current_clocksource
arch_sys_counter

$ dmesg | grep -i arch_timer
[    0.000000] arch_timer: cp15 timer(s) running at 54.00MHz (virt).
[    0.000000] clocksource: arch_sys_counter: mask: 0xff...
                                                      ^^^^^^
# "(virt)" is what you want. "(phys)" means the guest is on the
# physical timer and every access traps.

# Host: what frequency does the hardware run at?
$ dmesg | grep -i 'arch_timer.*MHz'
# Must match on both ends of a live migration.

# Prove the offset works - guest uptime should not be host uptime:
$ uptime            # in the guest
$ ssh host uptime   # on the host

# Measure timer accuracy inside the guest:
$ cyclictest -t1 -p 80 -n -i 1000 -l 10000
T: 0 ( 1234) P:80 I:1000 C:  10000 Min:     12 Act:   28 Avg:   35 Max:    9812
                                                              ^^^^^^^
# 9.8 ms worst case = the vCPU was descheduled. Pin it and
# re-run; on an isolated core this should drop to tens of us.
```

How it shows up

Symptom	Cause
Guest time is the host's uptime, not zero-based	`CNTVOFF_EL2` never initialised
Guest cores disagree about the time; scheduler misbehaves	Per-vCPU rather than per-VM `CNTVOFF`
Guest time jumps after host suspend/resume	Offset not adjusted across the suspend window
Huge exit rate with `EC=0x18` on `CNTP_*`	Guest using the physical timer — wrong interrupt in its device tree
Guest timers fire late by whole milliseconds	vCPU descheduled; the host hrtimer path is working as designed. Pin and isolate
Guest timer never fires at all	Descheduled-vCPU hrtimer not armed, or the virtual PPI not injected on wake
Guest clocksource watchdog marks the counter unstable	Time observed going backwards — usually an offset updated on one vCPU only
Live migration leaves the guest's clock wrong	Different `CNTFRQ_EL0` on source and destination
Guest reports high load with idle applications	Steal time not exposed, so the guest attributes stolen cycles to itself
`cyclictest` max latency in milliseconds	Overcommitted host; this is a capacity decision, not a bug

Check yourself

- ✓ Write the relationship between `CNTPCT_EL0`, `CNTVCT_EL0` and `CNTVOFF_EL2`.
- ✓ Name the four timer variants and the PPI number each uses.
- ✓ Explain why `CNTVOFF` must be per-VM rather than per-vCPU.
- ✓ Say which timer a KVM guest should use and how it is told.
- ✓ Explain what `CNTHCTL_EL2.EL1PCEN` controls and why KVM clears it.
- ✓ Describe how a guest timer fires while its vCPU is descheduled.
- ✓ Explain why "late" is acceptable but "early" and "lost" are not.
- ✓ Define steal time, say how a guest learns it, and explain what a high value tells you.
- ✓ Explain why counter frequency blocks some live migrations.
- ✓ Verify from inside a guest that it is using the virtual timer.

Trapping MMIO and emulating a device

A device model is a state machine driven by faults. The guest stores to an address that has no memory behind it, the hypervisor decodes what that store meant, updates some software state, and possibly raises an interrupt. This chapter builds one from nothing, and shows why doing it in the fewest possible traps is the entire design problem.

A device is a hole in the address space

```
Guest's view of its own physical memory:

  0x0000_0000  +--------------------+
               |  (nothing)         |
  0x0800_0000  +--------------------+
               |  GIC distributor   |  <- emulated
  0x0801_0000  +--------------------+
               |  GIC redistributor |  <- emulated
  0x0900_0000  +--------------------+
               |  PL011 UART        |  <- emulated
  0x0900_1000  +--------------------+
               |  RTC               |  <- emulated
  0x0a00_0000  +--------------------+
               |  virtio-mmio x32   |  <- emulated
  0x1000_0000  +--------------------+
               |  PCIe ECAM         |  <- emulated
  0x4000_0000  +--------------------+
               |                    |
               |  RAM               |  <- REAL memory,
               |                    |     mapped at stage 2
               +--------------------+

Stage 2 maps the RAM region and leaves EVERYTHING ELSE
unmapped. Every access to a device region therefore takes a
stage-2 translation fault, which is exactly the notification
the hypervisor wants.

In KVM this is expressed as MEMSLOTS: user space registers
"guest physical range X is backed by this host virtual
address". Anything outside a memslot is, by definition, MMIO.
```

The path of one register write

```
/* Guest driver:                                              */
        writel(0x41, uart_base + UARTDR);   /* print 'A'      */

/* 1. STR w0, [x1] where x1 -> IPA 0x0900_0000                */

/* 2. Stage-2 translation fault. ESR_EL2:
        EC   = 0x24  data abort from lower EL
        ISV  = 1     syndrome valid
        SAS  = 0b10  word access
        SRT  = 0      register x0
        WnR  = 1      it was a write
        DFSC = 0b000111  translation fault, level 3

      HPFAR_EL2 gives IPA 0x0900_0000.                       */

/* 3. KVM: is 0x0900_0000 inside a memslot?  No.
      -> it is MMIO. Is there an in-kernel device claiming it?
         (vGIC, arch timer, PV interfaces)  No.
      -> hand it to user space.                              */

/* 4. KVM_RUN returns. kvm_run->exit_reason = KVM_EXIT_MMIO:  */

        struct {
                __u64 phys_addr;   /* 0x09000000             */
                __u8  data[8];     /* 0x41                   */
                __u32 len;         /* 4                      */
                __u8  is_write;    /* 1                      */
        } mmio;

/* 5. QEMU dispatches to the PL011 model, which appends 'A' to
      the character device. If this were a READ, QEMU would
      instead fill kvm_run->mmio.data before re-entering.    */

/* 6. ioctl(KVM_RUN) again. KVM writes the read data into the
      guest's register x0 if needed, ADVANCES THE GUEST PC past
      the faulting instruction, and ERETs.

      Note: skip, not retry. The access was emulated, not
      fixed - the opposite of a memory fault (chapter 6).    */
```

TWO TIERS OF DEVICE MODEL

Devices that are hot and simple live *in the kernel*: the vGIC, the architected timer, the PV interfaces. Their exits never leave `KVM_RUN`, costing single-digit microseconds. Everything else lives in user space, costing ten times that but gaining the ability to be written in ordinary code with files, threads and sockets. Deciding which tier a device belongs in is the main performance lever you have.

Writing a device model

```
/* A minimal PL011 UART model. This is the shape of every
   device model you will ever write: a struct of state, a read
   function, a write function, and an interrupt line.        */

struct pl011 {
        uint32_t dr;          /* data                        */
        uint32_t fr;          /* flags                       */
        uint32_t imsc;        /* interrupt mask              */
        uint32_t ris;         /* raw interrupt status        */
        uint32_t cr;          /* control                     */
        uint8_t  rxfifo[16];
        unsigned rxhead, rxtail;
        int      irq;         /* which vINTID we raise       */
        int      chr_fd;      /* where output goes           */
};

#define UARTDR    0x000
#define UARTFR    0x018
#define UARTIMSC  0x038
#define UARTRIS   0x03c
#define UARTMIS   0x040
#define UARTCR    0x030
#define UARTICR   0x044

#define FR_RXFE   (1u << 4)   /* receive FIFO empty          */
#define FR_TXFF   (1u << 5)   /* transmit FIFO full          */
#define INT_RX    (1u << 4)
#define INT_TX    (1u << 5)

static uint32_t pl011_read(struct pl011 *u, uint64_t off)
{
        switch (off) {
        case UARTDR: {
                uint32_t c = 0;
                if (u->rxhead != u->rxtail) {
                        c = u->rxfifo[u->rxtail++ % 16];
                        if (u->rxhead == u->rxtail)
                                u->fr |= FR_RXFE;
                }
                /* READING CLEARS STATE. This is why speculative
                   reads of device memory are forbidden - see
                   chapter 7.                                  */
                pl011_update_irq(u);
                return c;
        }
        case UARTFR:   return u->fr;
        case UARTRIS:  return u->ris;
        case UARTMIS:  return u->ris & u->imsc;
        case UARTIMSC: return u->imsc;
        default:       return 0;      /* never fault the guest
                                         for an unknown offset -
                                         real hardware does not */
        }
}

static void pl011_write(struct pl011 *u, uint64_t off, uint32_t v)
{
        switch (off) {
        case UARTDR: {
                uint8_t c = v & 0xff;
```

```
                write(u->chr_fd, &c, 1);
                u->ris |= INT_TX;       /* transmit completed   */
                pl011_update_irq(u);
                break;
        }
        case UARTIMSC: u->imsc = v; pl011_update_irq(u); break;
        case UARTICR:  u->ris &= ~v;  pl011_update_irq(u); break;
        case UARTCR:   u->cr   = v; break;
        }
}

static void pl011_update_irq(struct pl011 *u)
{
        int level = (u->ris & u->imsc) != 0;
        set_irq_line(u->irq, level);     /* KVM_IRQ_LINE ioctl  */
}

/* Rules this small function already demonstrates:

   1. NEVER fault on an unrecognised offset. Return 0 and log.
      Guests probe. Hardware tolerates probing.
   2. Model side effects of READS explicitly (clear-on-read,
      FIFO pop). Getting these wrong causes lost data that
      looks like a driver bug.
   3. Recompute the interrupt line after any state change, and
      drive it as a LEVEL, not an edge. Level semantics are
      self-correcting; edges get lost.
   4. Handle partial-width accesses. A guest may read a byte of
      a 32-bit register.                                      */
```

The cost, and how to escape it

Path	Typical cost	When it applies
In-kernel device (vGIC, timer)	1–3 µs	Hot, simple, no external I/O
User-space device (QEMU)	10–30 µs	Anything needing files, sockets or complex state
`ioeventfd`	~1 µs, and asynchronous	A write to one address whose *value* does not matter — a doorbell
`irqfd`	~1 µs	Raising an interrupt from another thread or from the kernel without an ioctl
vhost (in-kernel datapath)	Comparable to host I/O	virtio net/blk where the whole datapath moves into the kernel
Passthrough	Zero for MMIO	The guest owns the real device (chapter 16)

```
/* ioeventfd: the single most valuable optimisation for a
   device model, and the reason virtio is fast.             */

  Ordinary path for "kick the queue":
     guest writes QueueNotify
       -> stage-2 fault
       -> exit to EL2
       -> exit to host kernel
       -> exit to QEMU
       -> QEMU decodes, finds the queue, processes
       -> re-enter (4 transitions)

  With ioeventfd:
     QEMU tells KVM in advance:
        "a write to 0x0a000050 of any value means: signal
         eventfd 7. Do not tell me the value. Do not return
         from KVM_RUN."

     guest writes QueueNotify
       -> stage-2 fault
       -> KVM writes eventfd 7 and IMMEDIATELY RE-ENTERS
       -> a separate QEMU thread (or vhost kernel thread)
          wakes on eventfd 7 and does the work CONCURRENTLY
          with the guest continuing to run.

  The guest never waits. The exit is a light one. This is the
  difference between "virtio is fine" and "virtio is fast".  */

/* The registration, in QEMU terms: */
   struct kvm_ioeventfd ke = {
            .addr     = 0x0a000050,
            .len      = 4,
            .fd       = evfd,
            .flags    = 0,     /* KVM_IOEVENTFD_FLAG_DATAMATCH
                                  if you need to distinguish
                                  which queue was kicked      */
   };
   ioctl(vmfd, KVM_IOEVENTFD, &ke);
```

Telling the guest the device exists

```
/* A device model is useless unless the guest is told where it
   is. Two mechanisms; on embedded arm64 it is the device tree. */

/ {
        soc {
                #address-cells = <2>;
                #size-cells    = <2>;

                pl011@9000000 {
                        compatible = "arm,pl011", "arm,primecell";
                        reg = <0x0 0x09000000 0x0 0x1000>;
                        interrupts = <0 1 4>;  /* SPI 1, level  */
                        clocks = <&uartclk>;
                        clock-names = "uartclk";
                };

                virtio_mmio@a000000 {
                        compatible = "virtio,mmio";
                        reg = <0x0 0x0a000000 0x0 0x200>;
                        interrupts = <0 16 1>;
                };
        };
};

/* QEMU GENERATES this device tree at runtime from the machine
   model, which is why -M virt "just works" for any memory size
   and CPU count. To see what your guest was actually given: */

$ qemu-system-aarch64 -M virt,dumpdtb=virt.dtb -nographic ...
$ dtc -I dtb -O dts virt.dtb | less

/* And from inside a running guest: */
$ ls /sys/firmware/devicetree/base/
$ dtc -I fs -O dts /sys/firmware/devicetree/base | less

/* A mismatch between the model's address and the device tree's
   reg property produces a device that is never probed, or a
   guest that faults on a region nothing models. The dumpdtb
   trick resolves this in seconds and is the first thing to
   reach for when "my device does not appear".              */
```

> **NEVER FAULT THE GUEST FOR AN UNKNOWN REGISTER**
> It is tempting to signal an abort when a guest touches an offset you did not implement. Do not. Guests probe hardware, read reserved fields, and read whole register blocks in a loop to dump state. Real silicon returns zero or garbage and carries on. A model that aborts turns a harmless probe into a guest kernel panic, and the panic points at the guest driver rather than at your model.

How it shows up

Symptom	Cause
Guest panics probing a device	Model aborts on an unimplemented offset instead of returning zero
Device never appears in the guest	Device tree `reg` does not match the address the model claims, or `compatible` matches no driver

Symptom	Cause
Guest receives the first interrupt only	Interrupt driven as an edge; the line was never lowered, so the next rise is not seen
Characters lost on an emulated UART	Clear-on-read side effects modelled incorrectly, or the RX FIFO overrun path missing
VM killed: "no valid syndrome"	Guest used `memcpy` / `LDP` on the MMIO region; `ISV=0` (chapter 6)
Throughput far below expectations	Doorbell handled as a full user-space exit instead of an `ioeventfd`
Guest hangs on a byte access to a 32-bit register	Model assumes word-width accesses only
Interrupt raised but guest never runs its handler	`irqfd` / `KVM_IRQ_LINE` targeting the wrong vINTID, or the vGIC not told the line is level-triggered
Device works in TCG but not with KVM	The model relies on ordering QEMU's own emulation provided; real guest CPUs reorder

Check yourself

- ✓ Explain how a hypervisor arranges to be notified of a device access, in terms of stage 2 and memslots.
- ✓ Trace a guest `writel()` to an emulated UART through every layer to the host's terminal.
- ✓ Say whether the guest PC is advanced or not after MMIO emulation, and contrast with a memory fault.
- ✓ Give three rules a device model must follow to survive guest probing.
- ✓ Explain why a device's interrupt should be modelled as a level, not an edge.
- ✓ Describe what `ioeventfd` changes and count the transitions it removes.
- ✓ Describe what `irqfd` is for.
- ✓ Explain how a guest learns a device exists, and how you inspect what it was told.
- ✓ Give the cost ratio between an in-kernel device and a user-space one, and the criterion for choosing.

virtio: the transport and the virtqueue

virtio is what you design when you accept that the guest knows it is a guest. Instead of pretending to be an IDE controller, you build the interface you would have wanted all along: a shared-memory ring where the expensive operation — telling the other side there is work — happens once per batch instead of once per register.

The shape of the thing

```
GUEST                                    HOST

virtio driver                            virtio device
(net, blk, console, ...)                 (QEMU, vhost, or a
     |                                    separate process)
     |                                          |
+----+------------------------------------------+----+
|             ONE OR MORE VIRTQUEUES                  |
|    living in GUEST MEMORY, which the host can       |
|    read and write directly - no copying, no         |
|    per-register traps                               |
+-----------------------------------------------------+
     |                                          |
     | NOTIFY: guest -> device                  |
     |   one MMIO write. This is the ONLY       |
     |   trap in the whole data path.           |
     |                                          |
     | INTERRUPT: device -> guest               |
     |   one virtual interrupt when work is done|

Plus a CONFIGURATION space for the slow path: feature
negotiation, queue setup, device-specific config (MAC
address, disk size, screen resolution). Touched at probe
time and then essentially never.

Split the design in your head:
  CONTROL PLANE  = MMIO registers, traps, rare
  DATA PLANE     = shared memory rings, no traps, constant
```

ONE IDEA, THREE TRANSPORTS

The virtqueue is the same everywhere. Only the way you find it and kick it changes: **virtio-mmio** (a flat register block, used on embedded and by QEMU's `virt` machine), **virtio-pci** (BARs and capabilities, used in clouds), and **virtio-ccw** (s390). Learn the queue once; the transports are packaging.

The split virtqueue

```
   Three arrays, all in guest memory, all agreed in advance.

   DESCRIPTOR TABLE - the buffers themselves
   +----+---------------+--------+-------+-------+
   | id |     addr      |  len   | flags |  next |
   +----+---------------+--------+-------+-------+
   |  0 | 0x4123_0000   |   16   | NEXT  |   1   |  request header
   |  1 | 0x4123_1000   | 4096   | NEXT  |   2   |  data
   |  2 | 0x4123_2000   |    1   | WRITE |   -   |  status byte
   +----+---------------+--------+-------+-------+
        flags: NEXT (chained)  WRITE (device writes it)
               INDIRECT (this descriptor points at a whole
                         table of descriptors)

   AVAILABLE RING - guest -> device.  "please process these"
   +-------+-------+--------------------------+
   | flags | idx=7 | ring[0..N]: 0, 5, 9, ...  |
   +-------+-------+--------------------------+
                ^         each entry is the HEAD descriptor
                |         of a chain
                idx only ever increases, and wraps naturally
                at 16 bits

   USED RING - device -> guest.  "these are done"
   +-------+-------+-------------------------------+
   | flags | idx=5 | ring[0..N]: {id=0,len=4096},  |
   +-------+-------+-------------------------------+
                       each entry says which chain completed
                       and how many bytes were written

   THE PROTOCOL:

     guest:  fill descriptors
             write avail.ring[avail.idx % N] = head
             memory barrier
             avail.idx++
             memory barrier
             NOTIFY (one MMIO write)   <-- the only trap

     device: sees avail.idx moved
             reads the chain, does the work
             writes used.ring[used.idx % N] = {head, len}
             memory barrier
             used.idx++
             raises an interrupt (maybe - see suppression)

     guest:  interrupt handler sees used.idx moved
             reclaims all completed chains in one pass
```

The barriers are not optional

```
/* Both sides are ordinary code on weakly-ordered CPUs, writing
   to shared memory with no lock. Every publication needs a
   barrier, and getting one wrong produces corruption that
   appears once an hour under load.                          */

/* GUEST, adding a buffer (linux/drivers/virtio/virtio_ring.c) */

        vq->split.vring.desc[head] = ...;      /* fill descriptors */

        /* The descriptor contents must be visible BEFORE the
           index that publishes them.                          */
        virtio_wmb(vq->weak_barriers);        /* DMB ISHST      */

        vq->split.vring.avail->idx = cpu_to_virtio16(vdev, new);

        /* And the index must be visible before the notify.    */
        virtio_mb(vq->weak_barriers);         /* DMB ISH        */

        if (vring_need_event(...))
                vq->notify(vq);

/* DEVICE, completing a buffer */

        used->ring[i] = (struct vring_used_elem){ head, len };
        smp_wmb();                            /* entry before idx */
        used->idx = new_idx;
        smp_mb();                             /* idx before irq   */
        raise_interrupt();

/* GUEST, reaping */

        if (vq->last_used_idx == virtio16_to_cpu(vdev, used->idx))
                return NULL;                  /* nothing new      */
        virtio_rmb(vq->weak_barriers);        /* idx before entry */
        elem = &used->ring[vq->last_used_idx % vq->num];

/* The "weak_barriers" flag: when the device is a HOST THREAD
   sharing coherent memory, SMP barriers (DMB ISH) suffice.
   When it is a real PCIe device doing DMA, you need full DMA
   barriers. VIRTIO_F_ORDER_PLATFORM tells the driver which.  */
```

Notification suppression: doing nothing, faster

```
/* An interrupt per completed request is as bad as a trap per
   register. Both sides can say "do not bother me".         */

/* Basic form, in the ring flags:

     avail->flags |= VRING_AVAIL_F_NO_INTERRUPT
         the GUEST is saying: I am polling, do not interrupt me.
         Used while NAPI is active.

     used->flags |= VRING_USED_F_NO_NOTIFY
         the DEVICE is saying: I am already processing, do not
         trap on notify. This is what removes the exit
         entirely while a batch is in flight.                */

/* Better form: VIRTIO_F_EVENT_IDX. Instead of on/off, each
   side publishes the index at which it WANTS to be told.  */

     used_event  = the avail.idx value at which the device
                   should notify the guest
     avail_event = the used.idx value at which the guest
                   should notify the device

/* So a busy device processing a stream of requests sets
   avail_event far ahead, and the guest submits hundreds of
   buffers with ZERO exits. When the device drains the queue it
   moves avail_event to "next one", and the following submit
   traps once to wake it.

   Result on a saturated virtio-net link: the exit rate is
   roughly the number of BATCHES per second, not packets. That
   is the difference between 50k and 3M packets per second.  */

/* This is the same fast-path/doorbell pattern as GICv4 direct
   injection (chapter 11). It recurs because it is the only way
   to make an expensive notification cheap: make it
   conditional on the receiver actually being asleep.        */
```

The packed virtqueue

```
virtio 1.1 replaces three arrays with ONE.

Split ring: descriptor table + avail ring + used ring
            -> three cache lines touched per request,
               in three separate places

Packed ring: a single descriptor ring, where ownership is
             encoded in two flag bits and a wrap counter.

+-----------------------------------------------------+
|  addr  |  len  |  id  |  flags: AVAIL | USED       |
+-----------------------------------------------------+

  driver writes a descriptor with AVAIL == wrap_counter
  device sees it, processes, writes back with
         USED == wrap_counter

Each side keeps its own index and wrap counter, and flips
the counter each time it goes round.

Why it is faster:
  * one cache line per descriptor instead of three
  * the device prefetches sequentially - the ring is read in
    order
  * far friendlier to a hardware implementation, which is
    the real motivation: virtio-net offloaded onto a SmartNIC
    or DPU wants a ring a state machine can walk.

Negotiated with VIRTIO_F_RING_PACKED. Both sides must
support it; the split ring remains the universal fallback.
```

virtio-mmio, concretely

```
/* The register block QEMU's virt machine puts at 0x0a000000,
   0x0a000200, 0x0a000400, ... one per device slot.         */

  0x000  MagicValue        0x74726976  ("virt")
  0x004  Version           2 (modern) or 1 (legacy)
  0x008  DeviceID          1=net 2=blk 3=console 4=entropy
                           16=gpu 18=input 19=vsock 25=fs
                           0 = EMPTY SLOT
  0x00c  VendorID
  0x010  DeviceFeatures    read, 32 bits at a time
  0x014  DeviceFeaturesSel which 32-bit word
  0x020  DriverFeatures
  0x024  DriverFeaturesSel
  0x030  QueueSel          which queue the following apply to
  0x034  QueueNumMax       device's maximum ring size
  0x038  QueueNum          driver's chosen size
  0x044  QueueReady
  0x050  QueueNotify       <-- THE DOORBELL. ioeventfd this.
  0x060  InterruptStatus
  0x064  InterruptACK
  0x070  Status            the handshake state machine
  0x080  QueueDescLow      guest-physical address of the
  0x084  QueueDescHigh     descriptor table
  0x090  QueueDriverLow    ... of the avail ring
  0x094  QueueDriverHigh
  0x0a0  QueueDeviceLow    ... of the used ring
  0x0a4  QueueDeviceHigh
  0x100+ Config            device-specific

/* The Status handshake - every virtio driver does exactly
   this, and a device model must enforce the order:        */

    0                          reset
    |= ACKNOWLEDGE   (1)       "I see a virtio device"
    |= DRIVER        (2)       "I have a driver for it"
    ... read DeviceFeatures, write DriverFeatures ...
    |= FEATURES_OK   (8)       "I accept this feature set"
    ... re-read Status: if FEATURES_OK cleared, the device
        rejected the negotiation and the driver must give up
    ... set up queues, write addresses, QueueReady = 1 ...
    |= DRIVER_OK     (4)       "I am live, you may use the
                                queues"

    FAILED           (128)     something went wrong
    DEVICE_NEEDS_RESET (64)    the device gave up

/* A device that starts processing queues before DRIVER_OK, or
   a driver that touches a queue before QueueReady, produces
   the classic "works most of the time" virtio bug.        */
```

> **LEGACY VERSUS MODERN**
> Version 1 (legacy) virtio-mmio uses guest page numbers rather than addresses, has a different endianness rule, and lacks the `QueueReady` handshake. Mixing a modern driver with a legacy device model gives you a device that probes and then silently transfers nothing. Check register `0x004` first — it is two instructions and saves an afternoon.

Watching it work

```
# Guest: which virtio devices exist and what negotiated?
$ ls /sys/bus/virtio/devices/
virtio0  virtio1  virtio2
$ cat /sys/bus/virtio/devices/virtio1/device       # 0x0002 = blk
$ cat /sys/bus/virtio/devices/virtio1/features
10000000000000000000001100001000000000000000000000000000000000000

$ cat /proc/interrupts | grep virtio
 25:      88104          0     GICv3  34 Level     virtio1

# Host: how many exits is the data path costing?
$ sudo perf stat -e kvm:kvm_mmio -a sleep 5
# Then run a big dd inside the guest and compare. With
# ioeventfd + EVENT_IDX the count should barely move.

# Is vhost carrying the datapath, or QEMU?
$ ps -T -p $(pgrep qemu) -o comm | grep vhost
vhost-4211            # kernel thread -> vhost is in use

# QEMU command line for an mmio virtio-blk on the virt machine:
$ qemu-system-aarch64 -M virt -m 1G \
    -drive file=rootfs.ext4,format=raw,if=none,id=hd0 \
    -device virtio-blk-device,drive=hd0 \
    -netdev user,id=n0 -device virtio-net-device,netdev=n0
# "-device virtio-*-device" = mmio transport
# "-device virtio-*-pci"    = PCI transport (needs a PCIe host
#                             bridge, which -M virt provides)
```

How it shows up

Symptom	Cause
Device probes, then no data ever moves	Legacy/modern mismatch, or queue addresses programmed before `QueueReady`
Rare data corruption under load	A missing barrier between descriptor contents and `avail.idx`, or between `used.idx` and the interrupt
Guest hangs waiting for completions	Notification suppression bug: device slept and the guest's notify was suppressed, so nobody woke
Throughput fine, latency terrible	`EVENT_IDX` thresholds too far ahead — batching at the cost of responsiveness
Enormous MMIO exit count on a busy queue	No `ioeventfd` on `QueueNotify`
Guest driver refuses to attach	Feature negotiation failed — `FEATURES_OK` cleared by the device
Works with QEMU, breaks with vhost	Device model and vhost disagree about which features were negotiated
Works on x86, fails on arm64	Barriers that were free on a strongly-ordered CPU and are not here
Ring wedges after ~65536 requests	Index wrap handled with a comparison instead of modular arithmetic

Check yourself

- ✓ Explain the control-plane/data-plane split and say which one costs exits.
- ✓ Name the three arrays of a split virtqueue and say which direction each carries.

- ✓ Walk a request from descriptor allocation to completion reaping, naming every barrier.
- ✓ Explain why the descriptor contents must be published before `avail.idx`.
- ✓ Describe both forms of notification suppression and what `EVENT_IDX` adds.
- ✓ Explain what the packed ring changes and why hardware implementers wanted it.
- ✓ List the virtio status handshake states in order and say what each asserts.
- ✓ Name the doorbell register in virtio-mmio and the optimisation that should be attached to it.
- ✓ Give two symptoms of a legacy/modern transport mismatch.
- ✓ Determine, from inside a guest, which virtio devices exist and what features they negotiated.

virtio device families and vhost

One queue mechanism, thirty device types, and three places the device implementation can live: in the VMM process, in the host kernel, or in a separate process entirely. Which you choose decides your throughput, your latency and your blast radius when the device model has a bug.

The device families

ID	Device	Queues	What it is for
1	`virtio-net`	RX/TX pair per channel, plus a control queue	Networking. The most optimised of the lot; supports multiqueue, checksum and TSO offload
2	`virtio-blk`	1 or more	Block storage. Simplest useful device — a good first one to implement
3	`virtio-console`	2 per port	Serial ports and the hvc console
4	`virtio-rng`	1	Entropy. Small, and matters more than it looks: a guest with no entropy blocks at boot
5	`virtio-balloon`	2–3	Return guest pages to the host under pressure
9	`virtio-9p`	1	File sharing, largely superseded by virtiofs
16	`virtio-gpu`	2	Display and 3D, with venus/virgl for GPU passthrough of API calls
18	`virtio-input`	2	Keyboard, mouse, touch — evdev over a queue
19	`virtio-vsock`	3	Host↔guest sockets with no network stack. The right answer for an agent inside a VM
22	`virtio-snd`	4	Audio
23	`virtio-fs`	2+	FUSE protocol over a queue, with DAX so the guest can map host page cache directly
25	`virtio-iommu`	1	A paravirtual IOMMU — lets a guest manage DMA mappings for its own passed-through devices
27	`virtio-mem`	1	Memory hotplug at sub-DIMM granularity; supersedes ballooning for most uses

WHICH ONES MATTER ON AN EMBEDDED TARGET

`virtio-blk`, `virtio-net`, `virtio-console` and `virtio-rng` give you a fully functional Linux guest on QEMU's `virt` machine with four command-line options. Everything else is specialisation. Start there, and see the recipes in QEMU chapter 15.

Three places a device can live

```
1. IN THE VMM (userspace, e.g. QEMU)

   guest  --notify-->  KVM  --exit-->  QEMU thread
                                         |
                                       host syscall
   Latency:  ~15-30 us per batch
   Pros:     easy to write, debug and sandbox; full access
             to files, sockets, image formats (qcow2)
   Cons:     two context switches per batch

2. IN THE HOST KERNEL (vhost)

   guest  --notify(ioeventfd)-->  KVM  --wake--> vhost-net
                                                 kernel thread
                                                    |
                                                 net stack
   Latency:  close to host-native
   Pros:     no userspace round trip; the kernel already has
             the socket/block layer right there
   Cons:     device model is kernel code - a bug is a host
             kernel bug. Only exists for net, blk (vhost-blk,
             out of tree), scsi, vsock.

   QEMU still does SETUP: feature negotiation, queue
   addresses, and it hands vhost the memory map and the
   eventfds. Only the DATA PATH moves.

3. IN ANOTHER PROCESS (vhost-user / vhost-vdpa)

   guest  --notify-->  KVM  --> a separate daemon
                                (DPDK, SPDK, virtiofsd,
                                 a rust-vmm device)
   Latency:  can be the best of all - a polling DPDK thread
             never sleeps, so there is no wake-up at all
   Pros:     device crash does not take down the VMM or the
             host kernel; can be written in Rust; can be
             updated independently
   Cons:     the daemon must map ALL of guest memory, so it
             is inside the trust boundary for confidentiality

   The daemon learns the guest memory layout over a UNIX
   socket (the vhost-user protocol) and then accesses the
   rings directly.
```

virtio-net in detail

```
/* Feature bits worth knowing, because they decide performance
   far more than the ring implementation does.              */

  VIRTIO_NET_F_CSUM            device handles checksums; the
                               guest can send unchecksummed
                               packets. Saves a pass over data.
  VIRTIO_NET_F_GUEST_CSUM      the guest will accept them.
  VIRTIO_NET_F_HOST_TSO4/6     device will segment; the guest
                               may send 64 KB "packets".
                               THE big win - one descriptor
                               chain instead of forty.
  VIRTIO_NET_F_GUEST_TSO4/6    guest accepts big receives (GRO).
  VIRTIO_NET_F_MRG_RXBUF       merge several receive buffers
                               into one packet - needed for
                               large receives without wasting
                               memory on small ones.
  VIRTIO_NET_F_MQ              multiqueue: one RX/TX pair per
                               vCPU, so no cross-CPU contention.
  VIRTIO_NET_F_CTRL_VQ         a control queue for MAC filters,
                               VLANs, offload changes.
  VIRTIO_NET_F_MTU             device tells the guest the MTU.

/* The header prepended to every packet: */

struct virtio_net_hdr_v1 {
        __u8  flags;            /* NEEDS_CSUM, DATA_VALID       */
        __u8  gso_type;         /* NONE, TCPV4, UDP, TCPV6      */
        __le16 hdr_len;
        __le16 gso_size;        /* MSS for segmentation         */
        __le16 csum_start;      /* where the checksum field is  */
        __le16 csum_offset;
        __le16 num_buffers;     /* MRG_RXBUF: how many buffers
                                   this packet occupies         */
};

/* Descriptor chain for one transmitted packet:

     desc[0]  the header above          (device reads)
     desc[1]  the packet data           (device reads)
     ...      more fragments if scattered

   For a receive:

     desc[0]  a buffer the DEVICE WRITES, big enough for header
              plus data. With MRG_RXBUF, several such buffers
              may be consumed for one large packet, and
              num_buffers says how many.                    */
```

```
# Turn the knobs and watch what happens.

# Multiqueue, 4 pairs, with vhost:
$ qemu-system-aarch64 -M virt -accel kvm \
   -netdev tap,id=n0,vhost=on,queues=4 \
   -device virtio-net-pci,netdev=n0,mq=on,vectors=10

# vectors = 2*queues + 2 (one per queue direction, plus config
# and control). Too few and the guest silently falls back to
# fewer queues.

# In the guest, confirm:
$ ethtool -l eth0
Channels: Combined: 4

# Which offloads did the guest actually get?
$ ethtool -k eth0 | grep -E 'tx-checksum|tso|gro|gso'
tx-checksumming: on
tcp-segmentation-offload: on
generic-receive-offload: on

# Turning TSO off is the single fastest way to prove it matters:
$ ethtool -K eth0 tso off
$ iperf3 -c host      # expect a large drop
```

virtio-blk, the one to implement first

```
/* Three descriptors per request. That is the whole protocol. */

struct virtio_blk_outhdr {
        __le32 type;       /* IN=0 read, OUT=1 write,
                              FLUSH=4, GET_ID=8, DISCARD=11,
                              WRITE_ZEROES=13                */
        __le32 reserved;
        __le64 sector;     /* in 512-byte units, ALWAYS,
                              regardless of the logical block
                              size the device advertises     */
};

  desc[0]  -> struct virtio_blk_outhdr   (device reads)
  desc[1]  -> the data buffer            (read: device writes
                                          write: device reads)
  desc[2]  -> a single status byte       (device writes)
                VIRTIO_BLK_S_OK      0
                VIRTIO_BLK_S_IOERR   1
                VIRTIO_BLK_S_UNSUPP  2

/* A device model's whole hot path: */

static void blk_handle_request(struct blkdev *d, struct chain *c)
{
        struct virtio_blk_outhdr *h = c->desc[0].addr;
        void   *buf  = c->desc[1].addr;
        size_t  len  = c->desc[1].len;
        uint8_t *st  = c->desc[2].addr;
        off_t   off  = (off_t)le64toh(h->sector) * 512;

        switch (le32toh(h->type)) {
        case VIRTIO_BLK_T_IN:
                *st = (pread(d->fd, buf, len, off) == (ssize_t)len)
                    ? VIRTIO_BLK_S_OK : VIRTIO_BLK_S_IOERR;
                break;
        case VIRTIO_BLK_T_OUT:
                *st = (pwrite(d->fd, buf, len, off) == (ssize_t)len)
                    ? VIRTIO_BLK_S_OK : VIRTIO_BLK_S_IOERR;
                break;
        case VIRTIO_BLK_T_FLUSH:
                *st = fdatasync(d->fd) == 0
                    ? VIRTIO_BLK_S_OK : VIRTIO_BLK_S_IOERR;
                break;
        default:
                *st = VIRTIO_BLK_S_UNSUPP;
        }
        vq_complete(c, len);
}

/* THE FLUSH IS NOT OPTIONAL. A device model that ignores
   VIRTIO_BLK_T_FLUSH will pass every functional test and lose
   the guest's filesystem on the first host power cut, because
   the guest believed its journal was durable. If you take one
   thing from this chapter, take that.                       */
```

CACHE MODES ARE A DURABILITY CONTRACT

QEMU's `cache=` option decides who may lie about durability. `cache=writeback` (the default) lets the host page cache absorb writes and relies on the guest issuing flushes — correct, and fast, *if* your flush path works. `cache=unsafe` discards flushes entirely: excellent for a throwaway build VM, catastrophic for anything you care about. `cache=none` uses O_DIRECT. Choose deliberately, and never leave `unsafe` in a script someone else might reuse.

virtio-vsock: the one people forget

```
/* Host-guest communication without a network. No IP, no
   firewall, no DHCP, no risk of the guest routing somewhere.
   Addresses are (CID, port).                                */

  CID 0  hypervisor
  CID 1  local (loopback)
  CID 2  the HOST, always
  CID 3+ guests, assigned by the VMM

# QEMU:
$ qemu-system-aarch64 ... -device vhost-vsock-pci,guest-cid=42

# Guest, a server:
$ socat VSOCK-LISTEN:1234,fork -
# Host, a client:
$ socat - VSOCK-CONNECT:42:1234

/* In C, it is just sockets: */
    int s = socket(AF_VSOCK, SOCK_STREAM, 0);
    struct sockaddr_vm a = {
            .svm_family = AF_VSOCK,
            .svm_cid    = VMADDR_CID_HOST,   /* 2              */
            .svm_port   = 1234,
    };
    connect(s, (struct sockaddr *)&a, sizeof a);

/* Use it for: guest agents, log shipping, test harnesses,
   debugging a guest with no network configured, and anything
   where giving the guest a network interface would be a
   security decision you do not want to make.               */
```

virtiofs and DAX

```
# Sharing a host directory with a guest, properly.

# Host: run the daemon (a vhost-user device in its own process)
$ /usr/libexec/virtiofsd --socket-path=/tmp/vfs.sock \
      -o source=/srv/shared -o cache=auto

# QEMU: connect to it, and give it shared memory
$ qemu-system-aarch64 -M virt,memory-backend=mem \
   -object memory-backend-file,id=mem,size=4G,\
mem-path=/dev/shm,share=on \
   -chardev socket,id=vfs,path=/tmp/vfs.sock \
   -device vhost-user-fs-pci,chardev=vfs,tag=myfs

# Guest:
$ mount -t virtiofs myfs /mnt

/* Why "share=on" and a file-backed memory object are
   mandatory: virtiofsd is a SEPARATE PROCESS. It must be able
   to map guest memory to read and write the queues. Anonymous
   private memory cannot be shared, so the VM will start and
   the mount will hang.

   DAX goes further: with a DAX window, the guest maps host
   PAGE CACHE PAGES directly into its own address space. A
   read() becomes a memory access with no copy and no exit at
   all. That is why virtiofs beats 9p by an order of magnitude
   on metadata-heavy workloads like a source tree.          */
```

How it shows up

Symptom	Cause
Guest filesystem corrupt after a host crash	Device model ignores `VIRTIO_BLK_T_FLUSH`, or `cache=unsafe`
Network throughput a fraction of the host's	TSO/checksum offload not negotiated — every packet is a separate chain
Multiqueue configured but only one queue active	Too few MSI-X vectors, or the guest never sent the control-queue command to enable the pairs
vhost-user device never starts, VM hangs at boot	Guest memory not shared (`share=on` and a file-backed memory object missing)
Guest blocks for a minute early in boot	No `virtio-rng`; the entropy pool is empty
Large receives dropped or truncated	`MRG_RXBUF` negotiated but `num_buffers` not honoured
Guest sees the wrong disk size after resizing	Config-space change not signalled with a configuration-change interrupt
vsock connection refused from the host	Wrong CID — the guest must connect to CID 2, not to its own
virtiofs mount hangs	Daemon not running, socket path mismatch, or DAX window requested without shared memory
Everything is slow only with `-accel tcg`	Not a virtio problem — emulation is the bottleneck (QEMU chapter 2)

Check yourself

- ✓ Name six virtio device types and what each replaces.
- ✓ Compare the three places a device implementation can live, with a latency figure for each.

- ✓ Explain what vhost moves into the kernel and what QEMU still does.
- ✓ Explain why vhost-user requires shared guest memory.
- ✓ List four virtio-net feature bits and say what each saves.
- ✓ Describe the three-descriptor structure of a virtio-blk request.
- ✓ Explain why ignoring `FLUSH` is a data-loss bug and not a performance choice.
- ✓ Give the vsock CID for the host and describe one good use for vsock.
- ✓ Explain what DAX gives virtiofs over 9p.
- ✓ Diagnose "multiqueue configured but only one queue is busy".

Device passthrough, the SMMU and VFIO

Giving a guest a real device removes every emulation cost at once — and hands an untrusted kernel a DMA engine. The SMMU is what makes that survivable, and VFIO is the interface through which Linux lets user space own a device safely. This chapter is where the isolation track and the virtualisation track meet.

Why passthrough, and what it costs you

	Emulated	virtio	Passthrough
MMIO cost	One exit per register	One exit per batch	**Zero** — the guest maps the real registers
Interrupt cost	Exit to inject	Exit to inject	One exit, or zero with GICv4
Guest driver	Any	Must have a virtio driver	The real vendor driver, unmodified
Live migration	Easy	Easy	**Very hard** — device state is opaque (unless it supports the VFIO migration protocol)
Sharing between guests	Trivial	Trivial	Impossible without SR-IOV
Host visibility	Full	Full	None — the host cannot inspect the traffic
Blast radius of a guest bug	Contained	Contained	The device is compromised

Passthrough is the right answer for a GPU, a 100GbE NIC, an accelerator, or an automotive CAN controller that a partition must own outright. It is the wrong answer whenever you need to migrate, oversubscribe, or inspect. Most real systems mix: virtio for the boot disk and management network, passthrough for the one device whose performance actually matters.

The problem passthrough creates

```
The guest programs the real device:

    "DMA 4 KB from your FIFO to address 0x4200_0000"

The guest means its own IPA 0x4200_0000. But the device does
not go through the CPU's MMU - it issues a bus transaction
with that address directly.

WITHOUT an SMMU:

    device writes PHYSICAL 0x4200_0000
         |
         v
    whatever is there. The host kernel. Another guest.
    Firmware. The guest just wrote to it.

A guest with a passed-through device and no SMMU owns the
entire machine. Not "can crash it" - OWNS it. This is not a
subtle bug; it is the reason IOMMUs exist.

WITH an SMMU:

    device writes IPA 0x4200_0000
         |
         |  SMMU looks up the StreamID of this device,
         |  finds the translation tables the hypervisor
         |  installed for it - THE SAME STAGE-2 TABLES the
         |  guest's CPU accesses use
         v
    real physical address, inside the guest's memory. Or a
    fault, if the guest lied.

The elegance: the device and the vCPU now share one notion
of "guest physical". A guest driver written for bare metal
works unmodified, because its DMA addresses mean the same
thing to the device that they mean to its own CPU.
```

> **STAGE 2 DOES DOUBLE DUTY**
> The same translation tables that constrain the guest's CPU also constrain its devices. That is why StreamID and `VTTBR_EL2` appear together in every passthrough setup: the hypervisor points the device's SMMU context descriptor at the VM's stage-2 tables and its VMID, and isolation follows automatically.

VFIO: owning a device from user space

```
VFIO is how Linux hands a device to a non-kernel owner
safely. QEMU uses it; so does DPDK; so can you.

/dev/vfio/vfio          the container - an address space
/dev/vfio/<group>       an IOMMU group - the unit of
                        assignment

+-------------------------------------------------------+
|  CONTAINER  (one IOMMU address space)                 |
|                                                       |
|   +---------------+     +----------------+          |
|   | GROUP 12      |     | GROUP 13       |          |
|   |  device 0000: |     |  device 0000:  |          |
|   |   03:00.0     |     |   04:00.0      |          |
|   |  device 0000: |     |                |          |
|   |   03:00.1     |     |                |          |
|   +---------------+     +----------------+          |
+-------------------------------------------------------+

THE GROUP IS THE UNIT, AND YOU DO NOT CHOOSE IT.

Two devices share a group when the hardware cannot tell them
apart - they sit behind a PCIe bridge that does not do ACS
(Access Control Services), or they share a StreamID, or they
are functions of one multifunction device that alias.

If they cannot be told apart, they cannot be isolated from
each other, so they must be assigned together. A group
containing your NIC and your disk controller means you
cannot pass through the NIC alone. That is a HARDWARE
limitation and no kernel flag legitimately works around it.
(There is an "ACS override" patch. Using it on a machine you
care about is choosing to believe a device that has told you
it cannot be trusted.)
```

```
# The whole workflow, on a PCIe NIC.

# 1. What group is it in, and who else is in there?
$ lspci -nn | grep -i ethernet
03:00.0 Ethernet controller [0200]: Intel X710 [8086:1572]

$ readlink -f /sys/bus/pci/devices/0000:03:00.0/iommu_group
/sys/kernel/iommu_groups/12

$ ls /sys/kernel/iommu_groups/12/devices/
0000:03:00.0  0000:03:00.1        # both ports - both must go

# 2. Unbind from the host driver, bind to vfio-pci
$ echo 0000:03:00.0 | sudo tee /sys/bus/pci/devices/0000:03:00.0/driver/unbind
$ echo 8086 1572 | sudo tee /sys/bus/pci/drivers/vfio-pci/new_id
# (repeat for 03:00.1)

$ ls -l /dev/vfio/
crw------- 1 root root 241, 0 vfio
crw------- 1 root root 511, 0 12          # the group is now ours

# 3. Hand it to QEMU
$ qemu-system-aarch64 -M virt,iommu=smmuv3 -accel kvm \
    -device vfio-pci,host=0000:03:00.0 \
    -device vfio-pci,host=0000:03:00.1 \
    ...

# 4. In the guest, it is simply the real device:
$ lspci
00:01.0 Ethernet controller: Intel X710
$ dmesg | grep i40e
i40e 0000:00:01.0: fw 6.1.42...
```

Why the guest's memory gets pinned

```
/* The moment a device can DMA into guest memory, that memory
   can no longer be swapped, migrated, or reclaimed - because
   the device holds a translation the kernel cannot revoke
   without the device noticing.

   So VFIO PINS it. All of it.                              */

    ioctl(container, VFIO_IOMMU_MAP_DMA, &(struct vfio_iommu_type1_dma_map){
            .vaddr = host_virtual_address_of_guest_ram,
            .iova  = guest_physical_address,
            .size  = whole_guest_ram,
            .flags = VFIO_DMA_MAP_FLAG_READ | VFIO_DMA_MAP_FLAG_WRITE,
    });

/* Consequences you must plan for:

   * A 16 GB guest with passthrough consumes 16 GB of host RAM,
     immediately and permanently. No ballooning, no overcommit,
     no KSM.

   * You will hit RLIMIT_MEMLOCK. The error is
     "VFIO_MAP_DMA failed: Cannot allocate memory" and it is
     nothing to do with free memory.
         $ ulimit -l
         64            # kilobytes. Nowhere near enough.
     Fix in /etc/security/limits.conf or the systemd unit:
         LimitMEMLOCK=infinity

   * Huge pages help a lot: fewer SMMU table levels, fewer
     entries, better TLB reach on the device side too.

   * Live migration of a pinned, passed-through device requires
     the device itself to implement the VFIO migration region
     so its internal state can be saved. Most do not.        */
```

Interrupts for a passed-through device

```
   The device's real interrupt must become the guest's virtual
   interrupt, and the guest's EOI must deactivate the real one.

     device raises MSI / SPI
        |
        v
     physical interrupt -> host (HCR_EL2.IMO routes to EL2)
        |
        v
     VFIO's handler signals an EVENTFD
        |
        v
     KVM has that eventfd bound as an IRQFD to a vINTID:
        ioctl(vmfd, KVM_IRQFD, {fd, gsi})
        |
        v
     KVM writes a list register with
        HW = 1, pINTID = the real one, vINTID = the guest's
        |
        v
     guest reads ICV_IAR1_EL1, services the device through its
     passed-through MMIO, writes ICV_EOIR1_EL1
        |
        v
     hardware deactivates the PHYSICAL interrupt too, because
     HW = 1. (chapter 10)

   Exits: one, for the physical interrupt. GICv4 removes even
   that by delivering the virtual LPI directly (chapter 11).

   The MSI address problem: a device programmed by the guest
   writes its MSI to an address the GUEST chose - an IPA. The
   SMMU must translate that to the real ITS doorbell, so the
   ITS window has to be mapped in the device's stage-2 tables.
   Linux handles this with "reserved regions"; a mismatch
   produces a device that works for MMIO and never interrupts. */
```

```
# The reserved regions the IOMMU insists you avoid or map:
$ cat /sys/kernel/iommu_groups/12/reserved_regions
0x0000000008080000 0x000000000809ffff msi
0x00000000fee00000 0x00000000feefffff msi

# QEMU refuses to place guest RAM over these, and maps the MSI
# window so the device's doorbell writes land on the real ITS.

# Check the SMMU is actually in the path:
$ dmesg | grep -i smmu
[    1.234567] arm-smmu-v3 arm-smmu-v3.0.auto: ias 44-bit, oas 48-bit
[    1.234599] arm-smmu-v3 arm-smmu-v3.0.auto: allocated 65536 entry CMDQ
[    1.234610] arm-smmu-v3 arm-smmu-v3.0.auto: 2-level strtab only

# And that the group is genuinely isolated, not a passthrough
# fallback:
$ cat /sys/kernel/iommu_groups/12/type
DMA          # good.  "identity" means NO TRANSLATION - the
             # device sees physical addresses. Never pass a
             # device through from an identity-mapped group.
```

> **`IOMMU.PASSTHROUGH=1` AND IDENTITY DOMAINS**
> Some distributions boot with the IOMMU in identity (bypass) mode for performance. A device in an identity domain sees real physical addresses. Passing such a device to a guest gives that guest full write access to host memory, silently — everything appears to work. Always check `/sys/kernel/iommu_groups/N/type` before trusting an assignment.

Platform devices, not just PCIe

```
/* Embedded SoCs have few PCIe devices and many platform
   devices described by device tree. VFIO handles those too,
   with more manual work.                                    */

  vfio-platform      generic platform device passthrough
  vfio-amba          for AMBA/PrimeCell devices

# Bind:
$ echo vfio-platform | sudo tee \
    /sys/bus/platform/devices/ff170000.i2c/driver_override
$ echo ff170000.i2c | sudo tee /sys/bus/platform/drivers/vfio-platform/bind

# QEMU:
$ qemu-system-aarch64 -M virt -accel kvm \
    -device vfio-platform,host=ff170000.i2c

/* Extra work platform passthrough demands and PCIe does not:

   1. YOU must write the guest's device tree node for it -
      there is no enumeration. QEMU can generate one for
      simple cases; anything with clocks, resets, regulators
      or pinmux you do yourself.

   2. RESET. PCIe has FLR (function level reset) as a standard.
      A platform device has whatever the vendor invented. VFIO
      requires a reset driver in the host kernel
      (vfio_platform_reset) or it refuses to bind, because
      handing a guest a device you cannot reset means the next
      guest inherits whatever the last one left running -
      including in-flight DMA.

   3. Shared resources. A clock or power domain shared between
      a passed-through device and a host device means the guest
      can switch off something the host is using. Device tree
      will not stop you; only your partitioning discipline
      will.                                                  */
```

How it shows up

Symptom	Cause
`VFIO_MAP_DMA failed: Cannot allocate memory`	`RLIMIT_MEMLOCK` too low — not a shortage of RAM
Cannot bind: "group is not viable"	Another device in the same IOMMU group is still bound to a host driver
Guest sees the device, MMIO works, no interrupts ever	MSI doorbell window not mapped, or `irqfd` not bound to the right vINTID

Symptom	Cause
Device works once, then not after guest reboot	No reset path — the device kept running with stale DMA
Host memory corruption when the guest starts	IOMMU in identity/passthrough mode, or no SMMU in the path at all
Guest boots, device faults on every DMA	Stage-2 tables not shared with the SMMU context, or the guest's DMA addresses are not IPAs it owns
SMMU fault flood in host `dmesg`	Guest driver programming addresses outside its memory — usually a guest bug the SMMU is correctly catching
Cannot migrate the VM	Passed-through device without VFIO migration support. Expected, not a bug
Guest gets far less memory than requested	Pinning failed partway; check for `hugetlb` availability and memlock limits
Two guests both crash when one uses its device	Devices share a StreamID, so the SMMU cannot tell them apart

Check yourself

- ✓ Give three costs of passthrough and three benefits, and name a workload for each side.
- ✓ Explain precisely what a passed-through device can do to a system with no SMMU.
- ✓ Explain how the SMMU and the vCPU come to agree on what a guest-physical address means.
- ✓ Define an IOMMU group and say why you cannot choose its membership.
- ✓ Walk through binding a PCIe device to `vfio-pci` and handing it to QEMU.
- ✓ Explain why guest memory is pinned and list three consequences.
- ✓ Diagnose `VFIO_MAP_DMA failed: Cannot allocate memory`.
- ✓ Trace a passed-through device's interrupt to the guest handler and back, naming the `HW` bit's role.
- ✓ Explain the MSI doorbell translation problem and how reserved regions solve it.
- ✓ Give three things platform passthrough requires that PCIe passthrough does not.
- ✓ Explain why an identity-domain IOMMU group must never be passed through.

KVM/arm64 internals

Everything in this track so far exists in one directory of the Linux source tree. This chapter is the map: where each mechanism lives, what the ioctl interface looks like from the outside, and how to build a working hypervisor in about two hundred lines so that the abstractions stop being abstract.

The source map

```
arch/arm64/kvm/
|
+-- arm.c              VM and vCPU lifecycle, the run loop.
|                      kvm_arch_vcpu_ioctl_run() is the
|                      function everything passes through.
+-- handle_exit.c      the exit dispatch table, keyed on
|                      ESR_EL2.EC. Start reading here.
+-- sys_regs.c         the emulated system register table -
|                      one entry per trapped register, with
|                      its access function and reset value.
|                      Enormous, and mechanical.
+-- mmu.c              stage-2: memslots, user_mem_abort(),
|                      page table management, dirty logging.
+-- pkvm.c             protected-KVM setup.
+-- psci.c             PSCI: CPU_ON/OFF/SUSPEND, SYSTEM_RESET.
+-- pmu.c pmu-emul.c   virtual PMU.
+-- arch_timer.c       CNTVOFF, the timer save/restore and
|                      the hrtimer used while descheduled.
+-- vgic/
|    +-- vgic-init.c   creation, from the KVM device API
|    +-- vgic-mmio*.c  the emulated distributor/redistributor
|    +-- vgic-v3.c     list registers, the hardware side
|    +-- vgic-its.c    the emulated ITS
|    +-- vgic-v4.c     direct injection
+-- hyp/
     +-- nvhe/         code that runs at EL2 in nVHE mode.
     |                 Compiled separately, its own linker
     |                 script, cannot call ordinary kernel
     |                 functions.
     +-- vhe/          the VHE variants
     +-- include/      shared switch code

virt/kvm/              architecture-independent core:
                       kvm_main.c, eventfd.c (irqfd,
                       ioeventfd), the memslot machinery.

READING ORDER that works:
  1. handle_exit.c  - see every exit reason in one table
  2. arm.c          - the loop around it
  3. mmu.c          - user_mem_abort(), the single most
                      important function in the file
  4. sys_regs.c     - pick one register and follow it
  5. vgic/          - only when you need it
```

The ioctl interface, in order

```
/* This is the entire contract between a VMM and KVM. Every
   VMM - QEMU, crosvm, firecracker, cloud-hypervisor - makes
   these calls in this order.                                */

  int kvm = open("/dev/kvm", O_RDWR);

  ioctl(kvm, KVM_GET_API_VERSION);              /* must be 12  */
  ioctl(kvm, KVM_CHECK_EXTENSION, KVM_CAP_ARM_VM_IPA_SIZE);

  int vm = ioctl(kvm, KVM_CREATE_VM, 40);       /* 40-bit IPA  */

  /* Memory: tell KVM which host memory backs which guest
     physical range. Anything not covered is MMIO.            */
  ioctl(vm, KVM_SET_USER_MEMORY_REGION,
        &(struct kvm_userspace_memory_region){
            .slot            = 0,
            .guest_phys_addr = 0x40000000,
            .memory_size     = 1UL << 30,
            .userspace_addr  = (uint64_t)mmap(...),
        });

  /* The interrupt controller is a "KVM device".              */
  int gic = ioctl(vm, KVM_CREATE_DEVICE,
        &(struct kvm_create_device){
            .type = KVM_DEV_TYPE_ARM_VGIC_V3 });
  ioctl(gic, KVM_SET_DEVICE_ATTR, ...);   /* base addresses    */

  /* vCPUs.                                                   */
  int vcpu = ioctl(vm, KVM_CREATE_VCPU, 0);

  struct kvm_vcpu_init init = {};
  ioctl(vm, KVM_ARM_PREFERRED_TARGET, &init);
  init.features[0] |= 1 << KVM_ARM_VCPU_PSCI_0_2;
  ioctl(vcpu, KVM_ARM_VCPU_INIT, &init);

  /* The shared communication page.                           */
  size_t mmap_sz = ioctl(kvm, KVM_GET_VCPU_MMAP_SIZE, 0);
  struct kvm_run *run = mmap(NULL, mmap_sz, PROT_READ|PROT_WRITE,
                             MAP_SHARED, vcpu, 0);

  /* Registers are set through a generic ID-based interface.  */
  ioctl(vcpu, KVM_SET_ONE_REG, &(struct kvm_one_reg){
            .id   = ARM64_CORE_REG(regs.pc),
            .addr = (uint64_t)&entry_point });

  /* And then, forever:                                       */
  for (;;) {
          ioctl(vcpu, KVM_RUN, 0);
          switch (run->exit_reason) { ... }
  }
```

A hypervisor in two hundred lines

```
/* Runs a tiny AArch64 payload that writes to a UART address we
   model, then halts. Build:
       aarch64-linux-gnu-gcc -static -o tinyvmm tinyvmm.c
   Run on an arm64 host with /dev/kvm.                        */

#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/kvm.h>

#define GUEST_RAM   0x40000000UL
#define RAM_SIZE    (2UL * 1024 * 1024)
#define UART_BASE   0x09000000UL

/* The guest program, assembled by hand:

     mov  x1, #0x09000000        // UART base
     mov  w2, #'K'
     str  w2, [x1]
     mov  w2, #'V'
     str  w2, [x1]
     mov  w2, #'M'
     str  w2, [x1]
     mov  w2, #'\n'
     str  w2, [x1]
   1: wfi
     b    1b
*/
static const uint32_t guest_code[] = {
        0xd2a12001,  /* mov x1, #0x09000000                  */
        0x52800962,  /* mov w2, #0x4b  'K'                   */
        0xb9000022,  /* str w2, [x1]                         */
        0x52800ac2,  /* mov w2, #0x56  'V'                   */
        0xb9000022,
        0x528009a2,  /* mov w2, #0x4d  'M'                   */
        0xb9000022,
        0x52800142,  /* mov w2, #0x0a  '\n'                  */
        0xb9000022,
        0xd503207f,  /* wfi                                  */
        0x17ffffff,  /* b .-4                                */
};

#define ARM64_CORE_REG(x) \
        (KVM_REG_ARM64 | KVM_REG_SIZE_U64 | KVM_REG_ARM_CORE | \
         (offsetof(struct kvm_regs, x) / sizeof(__u32)))

int main(void)
{
        int kvmfd = open("/dev/kvm", O_RDWR | O_CLOEXEC);
        if (kvmfd < 0) { perror("/dev/kvm"); return 1; }

        int vmfd = ioctl(kvmfd, KVM_CREATE_VM, 0);

        /* Guest RAM. */
        void *ram = mmap(NULL, RAM_SIZE, PROT_READ | PROT_WRITE,
                         MAP_PRIVATE | MAP_ANONYMOUS | MAP_NORESERVE,
```

```
                -1, 0);
memcpy(ram, guest_code, sizeof guest_code);

struct kvm_userspace_memory_region region = {
        .slot            = 0,
        .guest_phys_addr = GUEST_RAM,
        .memory_size     = RAM_SIZE,
        .userspace_addr  = (uint64_t)ram,
};
ioctl(vmfd, KVM_SET_USER_MEMORY_REGION, &region);

/* One vCPU. */
int vcpufd = ioctl(vmfd, KVM_CREATE_VCPU, 0);

struct kvm_vcpu_init init = {};
ioctl(vmfd, KVM_ARM_PREFERRED_TARGET, &init);
ioctl(vcpufd, KVM_ARM_VCPU_INIT, &init);

size_t run_size = ioctl(kvmfd, KVM_GET_VCPU_MMAP_SIZE, 0);
struct kvm_run *run = mmap(NULL, run_size,
                           PROT_READ | PROT_WRITE,
                           MAP_SHARED, vcpufd, 0);

/* PC = start of our code, PSTATE = EL1h, interrupts
   masked.                                          */
uint64_t pc = GUEST_RAM;
uint64_t pstate = 0x3c5;   /* D,A,I,F masked, EL1h     */
struct kvm_one_reg r;

r = (struct kvm_one_reg){ .id = ARM64_CORE_REG(regs.pc),
                          .addr = (uint64_t)&pc };
ioctl(vcpufd, KVM_SET_ONE_REG, &r);
r = (struct kvm_one_reg){ .id = ARM64_CORE_REG(regs.pstate),
                          .addr = (uint64_t)&pstate };
ioctl(vcpufd, KVM_SET_ONE_REG, &r);

/* The loop. */
for (;;) {
        if (ioctl(vcpufd, KVM_RUN, 0) < 0) {
                perror("KVM_RUN");
                return 1;
        }

        switch (run->exit_reason) {
        case KVM_EXIT_MMIO:
                if (run->mmio.phys_addr == UART_BASE &&
                    run->mmio.is_write) {
                        putchar(run->mmio.data[0]);
                        fflush(stdout);
                }
                break;

        case KVM_EXIT_SYSTEM_EVENT:
                printf("\n[system event %u]\n",
                       run->system_event.type);
                return 0;

        case KVM_EXIT_FAIL_ENTRY:
                fprintf(stderr, "entry failed: 0x%llx\n",
                        run->fail_entry.hardware_entry_failure_reason);
                return 1;

        default:
```

```
                        fprintf(stderr, "exit reason %u\n",
                                run->exit_reason);
                        return 1;
                }
        }
}
```

```
$ aarch64-linux-gnu-gcc -static -o tinyvmm tinyvmm.c
$ ./tinyvmm
KVM

# Then it sits in WFI forever - because we never created a vGIC,
# so nothing can ever wake it. Ctrl-C.

# That single word of output required: a VM, a memslot, a vCPU,
# an architected reset state, a run loop and an MMIO handler.
# Everything else in QEMU is more of the same, plus device
# models, plus a device tree, plus forty years of compatibility.
```

WHAT THIS PROVES

KVM does not "run virtual machines". It provides a CPU in a box and tells you when the box needs something. Every device, every address-space decision, every piece of firmware and every policy is user space's problem. That is why there are half a dozen VMMs with wildly different characters — firecracker boots in 100 ms because it declines to implement almost everything QEMU does.

The VMMs, and what they optimise for

VMM	Language	Optimises for	Notes
QEMU	C	Completeness	Every device, every architecture, TCG as well as KVM. The reference
Firecracker	Rust	Boot time and attack surface	~5 devices, no PCI, no BIOS. Used for AWS Lambda
crosvm	Rust	Sandboxing	Each device in its own process with seccomp; the basis of ChromeOS and Android VMs
cloud-hypervisor	Rust	Modern cloud workloads	Hotplug, live migration, no legacy emulation
rust-vmm	Rust	Being a library	The shared crates the above are built from; the fastest way to write your own
kvmtool	C	Being readable	~10k lines. The best code to learn from after this chapter

Instrumenting KVM

```
# Per-VM statistics (modern kernels expose them per-fd):
$ sudo ls /sys/kernel/debug/kvm/
$ sudo grep . /sys/kernel/debug/kvm/*/vcpu0/stat/* 2>/dev/null

# Tracepoints - the useful ones:
$ sudo ls /sys/kernel/debug/tracing/events/kvm/
kvm_entry  kvm_exit  kvm_guest_fault  kvm_mmio  kvm_set_irq
kvm_wfx_arm64  kvm_hvc_arm64  kvm_arm_setup_debug  ...

$ sudo perf record -e 'kvm:*' -a -- sleep 5
$ sudo perf script | awk '{print $5}' | sort | uniq -c | sort -rn

# Live summary:
$ sudo perf kvm --host stat live

# Function-level, when you are changing KVM itself:
$ sudo trace-cmd record -p function_graph \
      -g kvm_arch_vcpu_ioctl_run -O nofuncgraph-irqs
$ trace-cmd report | head -50

# Building a KVM with your own printk:
#   CONFIG_KVM=y (not =m, if you are changing hyp code)
#   CONFIG_DEBUG_INFO=y
# Note: in nVHE mode, code under arch/arm64/kvm/hyp/nvhe/
# CANNOT printk. Use the trace buffer or a magic register value
# you read after the exit. In VHE mode, ordinary printk works -
# one more reason VHE made KVM development pleasant.
```

How it shows up

Symptom	Cause
`KVM_CREATE_VM: Permission denied`	Not in the `kvm` group; check `ls -l /dev/kvm`
`KVM_ARM_VCPU_INIT: Invalid argument`	Requested a feature the host does not have — call `KVM_ARM_PREFERRED_TARGET` first and start from that
`KVM_RUN` returns `KVM_EXIT_FAIL_ENTRY` immediately	Illegal `PSTATE` in the initial register set, or PC outside any memslot
Guest runs but produces no output	MMIO handler matching the wrong address, or the payload writing to an address in a memslot (so it never traps)
vCPU never wakes from `WFI`	No vGIC created, so no interrupt can ever be delivered
Works as root, fails as a user	`RLIMIT_MEMLOCK`, or `/dev/kvm` permissions
Exit reason 0 or an unknown number	Kernel newer than your headers — rebuild against the running kernel's `linux/kvm.h`
Guest memory writes not visible to the VMM	Memslot registered with `MAP_PRIVATE` where the device or another process needs `MAP_SHARED`
No `printk` output from hyp code	nVHE mode — that code cannot printk at all

Check yourself

- ✓ Name the file in `arch/arm64/kvm/` that handles each of: exit dispatch, stage-2 faults, system-register emulation, the timer.

- ✓ List the ioctls needed to get from `/dev/kvm` to a running vCPU, in order.
- ✓ Explain what a memslot is and what "outside a memslot" implies.
- ✓ Explain what `kvm_run` is and how the VMM gets access to it.
- ✓ Explain why the tiny VMM's guest never wakes from `WFI`.
- ✓ State the division of responsibility between KVM and the VMM in one sentence.
- ✓ Give two VMMs other than QEMU and say what each optimises for.
- ✓ Name three KVM tracepoints and what each tells you.
- ✓ Explain why hyp code cannot `printk` in nVHE mode.
- ✓ Extend the tiny VMM: add a second MMIO address that returns a counter on read.

Static partitioning: Xen, Jailhouse and safety

Everything so far assumed the hypervisor should share resources cleverly. In safety-critical embedded systems that assumption is inverted: sharing is the hazard, and the hypervisor's job is to divide the machine once, at boot, and then get out of the way — ideally while executing no code at all during normal operation.

The inversion

```
CONSOLIDATING hypervisor (KVM, Xen, VMware)

  goal: run N guests on M cores, N > M
  scheduler: yes, and it is the interesting part
  memory: overcommitted, ballooned, swapped
  devices: emulated or shared
  failure mode: a noisy guest degrades its neighbours
  metric: throughput per watt, density, utilisation


PARTITIONING hypervisor (Jailhouse, Xen dom0-less,
                         Bao, PikeOS, QNX Hypervisor)

  goal: run N guests on N disjoint sets of cores
  scheduler: NONE. A core belongs to one guest, always.
  memory: statically carved, never shared, never reclaimed
  devices: statically assigned, one owner each
  failure mode: a guest can only harm itself
  metric: worst-case latency, certifiability, code size

The partitioning hypervisor is not a worse consolidator.
It is a different product, and asking it to time-share is
asking it to stop being what it is.
```

WHY "NO SCHEDULER" IS THE POINT

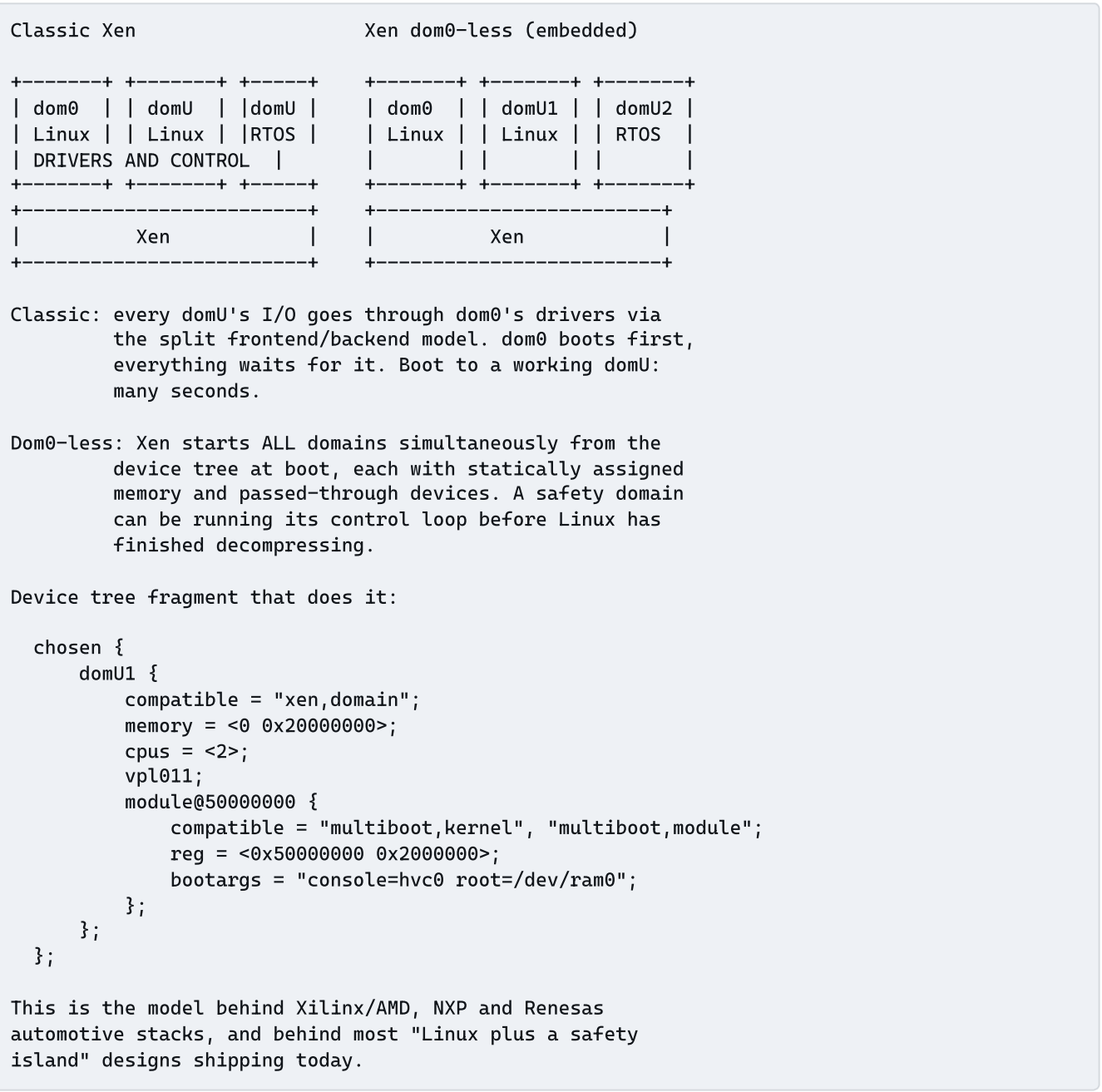
If the hypervisor never preempts a guest, then the guest's worst-case execution time is a property of the guest and the hardware alone. You can analyse it, bound it, and put it in a safety case. Add a scheduler and the worst case becomes a property of every other guest in the system — and that is not analysable in any useful sense. Certification does not reward cleverness; it rewards arguments you can finish.

Jailhouse: the minimal case

```
Jailhouse does not boot the machine. Linux does.
Then Linux hands pieces of itself away.

Step 1: Linux boots normally on all 4 cores.
        You reserve memory with the kernel command line:
           memmap=0x4000000$0x3f000000

Step 2: load the hypervisor
           jailhouse enable system-config.cell
        Jailhouse installs itself at EL2 UNDER the running
        Linux, which becomes the "root cell". Linux does not
        notice; it keeps running, now virtualised.

Step 3: take resources away
           jailhouse cell create rtos.cell
           jailhouse cell load rtos 0 rtos.bin
           jailhouse cell start rtos
        Cores 2 and 3 are removed from Linux (offlined
        first), the reserved memory is unmapped from the
        root cell, and the RTOS starts on them.

AFTER THIS POINT:

  +----------------------+   +----------------------+
  | root cell: Linux     |   | cell 1: RTOS         |
  | cores 0-1            |   | cores 2-3            |
  | RAM 0-1 GB           |   | RAM 1.0-1.06 GB      |
  | eth0, sd, usb        |   | CAN, one timer       |
  +----------------------+   +----------------------+
  +-----------------------------------------------------+
  |  Jailhouse at EL2: ~10k lines, no scheduler,       |
  |  no dynamic memory, no drivers, no interrupts      |
  |  of its own. Configures stage 2, the SMMU and the  |
  |  GIC, then ERETs and is essentially never          |
  |  entered again.                                     |
  +-----------------------------------------------------+

The hypervisor's steady-state CPU consumption is ZERO,
because in steady state it does not run.
```

```
/* A Jailhouse cell configuration is a C struct compiled to a
   binary blob. This is the entire interface, and its
   explicitness is deliberate.                                */

struct { ... } config = {
        .cell = {
                .name = "rtos-cell",
                .flags = JAILHOUSE_CELL_PASSIVE_COMMREG,
                .cpu_set_size = sizeof(config.cpus),
                .num_memory_regions = ARRAY_SIZE(config.mem_regions),
                .num_irqchips = ARRAY_SIZE(config.irqchips),
                .vpci_irq_base = 100,
        },

        .cpus = { 0b1100 },          /* cores 2 and 3          */

        .mem_regions = {
                /* RAM: physical == virtual, exclusively ours  */
                {
                        .phys_start = 0x40000000,
                        .virt_start = 0x40000000,
                        .size       = 0x04000000,
                        .flags = JAILHOUSE_MEM_READ  |
                                 JAILHOUSE_MEM_WRITE |
                                 JAILHOUSE_MEM_EXECUTE |
                                 JAILHOUSE_MEM_LOADABLE,
                },
                /* The CAN controller - ONE owner, and Linux
                   must not have a driver bound to it.        */
                {
                        .phys_start = 0xff060000,
                        .virt_start = 0xff060000,
                        .size       = 0x1000,
                        .flags = JAILHOUSE_MEM_READ |
                                 JAILHOUSE_MEM_WRITE |
                                 JAILHOUSE_MEM_IO,
                },
        },

        .irqchips = {
                {
                        .address = 0xf9010000,   /* GICD       */
                        .pin_base = 32,
                        .pin_bitmap = { 1 << (142 - 32) },
                                       /* SPI 142 = the CAN   */
                },
        },
};

/* Notice what is NOT here: no memory size to grow into, no
   share of CPU time, no priority. Everything is a fixed
   assignment, decided before the system runs, auditable by
   reading one file.                                          */
```

Xen on ARM, and dom0-less



Classic: every domU's I/O goes through dom0's drivers via the split frontend/backend model. dom0 boots first, everything waits for it. Boot to a working domU: many seconds.

Dom0-less: Xen starts ALL domains simultaneously from the device tree at boot, each with statically assigned memory and passed-through devices. A safety domain can be running its control loop before Linux has finished decompressing.

Device tree fragment that does it:

```
  chosen {
      domU1 {
          compatible = "xen,domain";
          memory = <0 0x20000000>;
          cpus = <2>;
          vpl011;
          module@50000000 {
              compatible = "multiboot,kernel", "multiboot,module";
              reg = <0x50000000 0x2000000>;
              bootargs = "console=hvc0 root=/dev/ram0";
          };
      };
  };
```

This is the model behind Xilinx/AMD, NXP and Renesas automotive stacks, and behind most "Linux plus a safety island" designs shipping today.

The part everyone forgets: shared microarchitecture

```
You have partitioned cores, memory and devices. You have
NOT partitioned:

  the last-level cache      a Linux guest streaming data
                            evicts the RTOS's working set

  the memory controller     bandwidth and queueing delay are
                            shared; a DMA-heavy guest adds
                            latency to every other access

  the interconnect          arbitration, and any shared
                            buffering

  shared TLB / prefetchers  on some designs

Consequence: two guests that never share a byte of memory
still interfere in TIME. For a throughput workload this is
noise. For a 100 us control loop it is the difference
between meeting and missing a deadline.

MITIGATIONS, in increasing order of effort:

1. CACHE COLOURING. Allocate each partition only physical
   pages that map to disjoint cache sets. Because the cache
   index comes from physical address bits, choosing pages
   chooses cache regions. Jailhouse and Bao support it;
   you pay in usable memory and in allocator complexity.

2. MPAM (ARMv8.4 Memory System Resource Partitioning and
   Monitoring). Hardware support: tag each request with a
   PARTID, and configure cache-capacity and bandwidth limits
   per PARTID. This is the proper answer, and it is
   appearing in server and automotive silicon now.
     MPAM1_EL1 / MPAM2_EL2 hold the PARTID for each level.

3. MEASURE AND OVER-PROVISION. Run the interference
   benchmark (a deliberately cache-hostile stressor) in the
   Linux partition while measuring the RTOS's worst-case
   latency. Whatever number you get IS your worst case.
   This is what safety cases actually contain.
```

> **INTERFERENCE IS A CERTIFICATION PROBLEM, NOT A BUG**
> ISO 26262 and DO-178C both require you to argue freedom from interference. Partitioning the address space gets you spatial freedom. Temporal freedom needs measurement, and often needs MPAM or colouring. A design review that only shows the memory map has answered half the question.

Choosing

	KVM	Xen dom0-less	Jailhouse	Bao / PikeOS / QNX
Lines of code at EL2	Large (or ~10k for pKVM)	~100k	~10k	~5–50k
Scheduler	Full	Optional (null scheduler)	None	None or partitioned
Boot to first guest	Seconds	Sub-second	Linux first, then instant	Tens of ms

	KVM	Xen dom0-less	Jailhouse	Bao / PikeOS / QNX
Overcommit	Yes	Optional	No	No
Certifiable	Not realistically	Xen has safety-certified variants	Research/ industrial, small enough to try	Yes — that is the product
Best for	Servers, dev boxes, Android VMs	Automotive, mixed criticality	Adding a real-time partition to an existing Linux	Shipping safety systems

Trying it

```
# Jailhouse on QEMU's virt machine is the cheapest way to see
# a partitioning hypervisor work.

$ git clone https://github.com/siemens/jailhouse
$ cd jailhouse
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
       KDIR=/path/to/linux
# Configs for qemu-arm64 ship in configs/arm64/

# Boot Linux under QEMU with reserved memory:
$ qemu-system-aarch64 -M virt,gic-version=3 -cpu cortex-a57 \
    -smp 4 -m 1G -nographic \
    -kernel Image -append "console=ttyAMA0 mem=768M" ...

# Inside the guest Linux:
$ insmod jailhouse.ko
$ jailhouse enable qemu-arm64.cell
Initializing Jailhouse hypervisor v0.12 on CPU 0
Code location: 0x0000ffffc0200000
Page pool usage after early setup: mem 39/993, remap 0/131072
...
The Jailhouse is opening.

$ jailhouse cell create qemu-arm64-inmate-demo.cell
$ jailhouse cell load 1 gic-demo.bin
$ jailhouse cell start 1
$ jailhouse cell list
ID      Name                    State             Assigned CPUs
0       qemu-arm64              running           0-2
1       inmate-demo             running           3

# Core 3 is now gone from Linux entirely:
$ nproc
3

# And the demo inmate is printing timer ticks on the second
# UART with a jitter of a few microseconds, regardless of what
# Linux is doing - which is the entire point.
```

How it shows up

Symptom	Cause
`jailhouse enable` fails with "CPU is not in EL2"	Firmware dropped to EL1 — the same prerequisite as KVM (chapter 3)
Cell creation fails: memory region overlaps	The reserved region was not actually reserved from Linux, or two cells claim it
Linux crashes when a cell starts	A device assigned to the cell is still bound to a Linux driver

Symptom	Cause
Guest partition misses deadlines only when Linux is busy	Cache and memory-bandwidth interference; partitioning was spatial only
Interrupts for a cell's device arrive in the root cell	The GIC `pin_bitmap` does not include that SPI, or affinity routes it elsewhere
Cell runs once and cannot be restarted	No reset path for its assigned device; it resumed with stale state
Everything works until you enable the SMMU	Device StreamIDs not listed in the cell config, so its DMA now faults
Worst-case latency good in the lab, bad in the field	The lab never ran the interference workload; measure with a deliberate stressor

Check yourself

- ✓ State the difference in goal between a consolidating and a partitioning hypervisor.
- ✓ Explain why "no scheduler" makes a safety argument tractable.
- ✓ Describe how Jailhouse installs itself beneath a running Linux, in three steps.
- ✓ Read a cell configuration and say which cores, memory and interrupts a partition owns.
- ✓ Explain what dom0-less Xen changes about boot time and why it matters in a car.
- ✓ Name four microarchitectural resources that partitioning does *not* divide.
- ✓ Explain cache colouring and what it costs.
- ✓ Say what MPAM provides and which registers carry the partition identity.
- ✓ Distinguish spatial from temporal freedom from interference.
- ✓ Choose a hypervisor for: an Android phone, a domain controller in a car, and adding a motor-control loop to an existing Linux product — and justify each.

S-EL2, Hafnium and FF-A

TrustZone gave the secure world one privileged OS and no way to isolate its parts from each other. As secure firmware grew — a TEE, a firmware TPM, a DRM stack, a vendor's key manager, all from different suppliers — that became untenable. S-EL2 adds virtualisation *inside* the secure world, and FF-A is the standard that makes the pieces interoperate.

The problem with one secure world

```
BEFORE S-EL2

S-EL0   TA   TA   TA          isolated from each other by
                              the trusted OS's MMU
S-EL1   OP-TEE                ONE trusted OS. Everything
                              privileged in the secure world
                              lives here.
EL3     TF-A

So: a firmware TPM, a vendor keystore, a DRM widevine
implementation and OP-TEE itself all end up either
  (a) as trusted applications, sharing one trusted OS's
      privilege domain and trusting its kernel completely, or
  (b) bolted INTO the trusted OS as pseudo-TAs, which means
      a bug in a vendor's DRM blob is a bug in the most
      privileged non-EL3 software on the phone.

Neither is acceptable when the components come from
different vendors with different assurance levels.


AFTER S-EL2

S-EL0   TA  TA        app          app
S-EL1   OP-TEE     |  fTPM SP   |  vendor SP
                   |            |
S-EL2   HAFNIUM - the Secure Partition Manager.
        Stage-2 translation for the SECURE world. Each
        secure partition gets its own IPA space, its own
        memory, its own devices, and cannot see the others.
EL3     TF-A - now also the SPM Dispatcher, routing FF-A
        calls between worlds.

Same architecture as the normal world, one world across.
Stage 2, VMIDs, traps - all of it, but with NS=0.
```

THE NAMING

In the normal world you say hypervisor, VM, vCPU. In the secure world you say **SPM** (Secure Partition Manager), **SP** (Secure Partition), and **execution context**. The mechanisms are the same; the vocabulary comes from the FF-A specification, and using the wrong one in a review makes conversations harder than they need to be.

FF-A: the Firmware Framework

Arm's Firmware Framework for A-profile defines how partitions discover each other, send messages, and share memory — across the world boundary and within a world. It replaces a decade of incompatible vendor SMC conventions with one ABI.

```
/* FF-A calls are SMCs with function IDs in a reserved range,
   following SMCCC. The ones you will actually use:        */

  SETUP AND DISCOVERY
    FFA_VERSION                 negotiate the version
    FFA_FEATURES                what does this implementation do?
    FFA_ID_GET                  what is my own partition ID?
    FFA_PARTITION_INFO_GET      who else exists, and what UUID?
    FFA_RXTX_MAP                give the SPM my message buffers
    FFA_RXTX_UNMAP

  MESSAGING
    FFA_MSG_SEND_DIRECT_REQ     synchronous call to a partition
    FFA_MSG_SEND_DIRECT_RESP    its reply
    FFA_MSG_WAIT                "I am idle, wake me on a message"
    FFA_RUN                     schedule a partition's context
    FFA_YIELD
    FFA_INTERRUPT               an interrupt arrived for you

  MEMORY MANAGEMENT - the interesting part
    FFA_MEM_DONATE              transfer ownership permanently
    FFA_MEM_LEND                transfer access, keep ownership
    FFA_MEM_SHARE               both parties may access
    FFA_MEM_RETRIEVE_REQ        receiver claims the transfer
    FFA_MEM_RETRIEVE_RESP
    FFA_MEM_RELINQUISH          receiver gives it back
    FFA_MEM_RECLAIM             sender takes it back

  ERROR / SUCCESS
    FFA_SUCCESS   FFA_ERROR

/* A direct request, from a normal-world driver: */

    x0 = FFA_MSG_SEND_DIRECT_REQ_64   /* 0xC400006F          */
    x1 = (sender_id << 16) | receiver_id
    x2 = 0                            /* flags               */
    x3..x7 = up to five 64-bit values of payload
    smc #0

    /* returns in x0 = FFA_MSG_SEND_DIRECT_RESP, x3..x7 = the
       reply. Synchronous, and the caller's context is
       blocked meanwhile.                                    */
```

Memory transfer, done properly

```
The old way: a normal-world driver passes a physical address
in an SMC and the trusted OS trusts it. Every parameter-
validation bug in every TEE ever, in one sentence.

FF-A instead makes the transfer an explicit, mediated
transaction with THREE distinct semantics:

DONATE   ownership moves. The sender can never get it back.
         Used for: handing a buffer to a partition for its
         own permanent use.

LEND     access moves, ownership stays. The sender loses
         access while the loan is outstanding. Exclusive.
         Used for: "process this buffer and give it back" -
         and because the sender loses access, there is no
         time-of-check-to-time-of-use window.

SHARE    both may access concurrently. Cheapest, and the
         one that reintroduces TOCTOU if you are careless.

The protocol:

  sender:   FFA_MEM_LEND with a descriptor listing
            address ranges, receiver ID, and permissions
               -> SPM validates the sender OWNS those pages
               -> SPM returns a HANDLE
  sender:   passes the handle to the receiver (by message)
  receiver: FFA_MEM_RETRIEVE_REQ(handle)
               -> SPM maps the pages into the RECEIVER'S
                  stage-2 tables with the agreed permissions
               -> and UNMAPS them from the sender if LEND
  ...work happens...
  receiver: FFA_MEM_RELINQUISH(handle)
  sender:   FFA_MEM_RECLAIM(handle)
               -> SPM restores the sender's mapping

The SPM is the only component that edits stage-2 tables, and
it validates every claim of ownership. A partition cannot
lend what it does not have.
```

> **LEND VERSUS SHARE IS A SECURITY DECISION**
>
> With `SHARE`, the sender can modify a buffer after the receiver has validated its contents — the classic double-fetch. With `LEND`, the sender's mapping is removed, so it physically cannot. Use `LEND` for anything where the receiver makes a trust decision based on the buffer's contents, and pay the extra unmap cost.

Hafnium

```
/* Hafnium is Arm's reference SPM. Small, C, no dynamic
   allocation, and it is also usable as a normal-world
   hypervisor (that was its original life, as the Android
   "protected VM" hypervisor before pKVM).                  */

  Runs at:      S-EL2   (as an SPM)
                EL2     (as a plain hypervisor)
  Language:     C, ~20k lines
  Scheduling:   cooperative - partitions are RUN by the normal
                world's scheduler via FFA_RUN. Hafnium does not
                have a scheduler of its own, deliberately.
  Memory:       fully static. Every partition's memory comes
                from its manifest, allocated at boot.

/* A secure partition manifest - device tree, compiled into
   the SP package:                                          */

  sp {
      compatible = "arm,ffa-manifest-1.0";
      ffa-version = <0x00010001>;      /* FF-A v1.1          */
      uuid = <0xb476b8a1 0xdcbf474f 0xa4b7a2a1 0x6f0f4d3c>;
      id = <0x8001>;
      execution-ctx-count = <1>;
      exception-level = <2>;           /* S-EL1              */
      execution-state = <0>;           /* AArch64            */
      load-address = <0x6280000>;
      entrypoint-offset = <0x2000>;
      xlat-granule = <0>;              /* 4 KB               */
      boot-order = <0>;
      messaging-method = <3>;          /* direct req + resp  */

      memory-regions {
          compatible = "arm,ffa-manifest-memory-regions";
          rx-buffer { base-address = <0x0 0x7300000>;
                      pages-count = <1>; attributes = <1>; };
          tx-buffer { base-address = <0x0 0x7301000>;
                      pages-count = <1>; attributes = <3>; };
      };

      device-regions {
          compatible = "arm,ffa-manifest-device-regions";
          crypto-accel {
              base-address = <0x0 0x40010000>;
              pages-count = <16>;
              attributes = <3>;        /* read-write          */
              smmu-id = <0>;
              stream-ids = <0x0 0x1>;  /* SMMU isolation too  */
          };
      };
  };

/* Read it as a contract: this is EXACTLY what partition 0x8001
   may touch. Anything else faults. And because it is static,
   a reviewer can check it without running anything.        */
```

Talking to a secure partition from Linux

```
# Linux has an FF-A bus, like any other bus.

$ ls /sys/bus/ffa/devices/
arm-ffa-8001   arm-ffa-8002

$ cat /sys/bus/ffa/devices/arm-ffa-8001/uuid
b476b8a1-dcbf-474f-a4b7-a2a16f0f4d3c

$ dmesg | grep -i ff-a
[    0.812] ARM FF-A: Initialized with version 1.1
[    0.813] ARM FF-A: Partition ID 0x8001 : 1 exec ctx, props 0x3

/* And a driver binds to it exactly as it would to a PCI or
   platform device:                                          */

static const struct ffa_device_id my_sp_ids[] = {
        { UUID_INIT(0xb476b8a1, 0xdcbf, 0x474f,
                    0xa4, 0xb7, 0xa2, 0xa1,
                    0x6f, 0x0f, 0x4d, 0x3c) },
        {}
};

static int my_sp_probe(struct ffa_device *dev)
{
        const struct ffa_ops *ops = dev->ops;
        struct ffa_send_direct_data d = {
                .data0 = MY_CMD_GET_VERSION,
        };

        ops->msg_ops->sync_send_receive(dev, &d);
        dev_info(&dev->dev, "SP version %llu\n", d.data0);
        return 0;
}

static struct ffa_driver my_sp_driver = {
        .name     = "my_sp",
        .probe    = my_sp_probe,
        .id_table = my_sp_ids,
};
module_ffa_driver(my_sp_driver);

/* This is how the upstream fTPM driver will eventually talk to
   an fTPM secure partition, replacing the OP-TEE-specific path
   we build in the OP-TEE track. The TA logic is the same; the
   transport changes.                                          */
```

Realms, in one page

```
ARMv9 Confidential Compute Architecture adds a FOURTH
security state, for a different threat model again:

  Non-secure   normal world - hypervisor and its VMs
  Secure       TrustZone - TEE, secure partitions
  Realm        confidential VMs
  Root         EL3, and the Granule Protection Table

Threat model: protect a guest from the HYPERVISOR and from
the cloud operator. Not from itself; not from the TEE.

  R-EL2   RMM (Realm Management Monitor) - a small, spec'd
          component that manages realms
  Root    Monitor + GPT: hardware-checked ownership of every
          granule of physical memory. GPC (Granule
          Protection Check) happens on EVERY access, in
          addition to stage 1 and stage 2.

The hypervisor still SCHEDULES realms and still allocates
their memory - but it cannot READ it. Attempts fault with a
Granule Protection Fault.

Where it fits with this track: everything you know about
stage 2 and vCPUs still applies. What changes is that the
hypervisor's stage-2 edits must be ratified by the RMM, and
the guest's memory is not in the hypervisor's address space.
The mental model is "pKVM, enforced by hardware rather than
by trusting a small EL2 component".
```

How it shows up

Symptom	Cause
No FF-A devices in `/sys/bus/ffa/`	No SPM in firmware, or TF-A built without the SPM dispatcher
`FFA_ERROR NOT_SUPPORTED` on a direct request	The partition's manifest declares a different `messaging-method`
Memory retrieve fails with `DENIED`	Sender does not own the pages, or the receiver requested different permissions than were offered
Secure partition faults on its device registers	Device region missing from the manifest, or the StreamID not listed so the SMMU blocks its DMA
Partition never runs	Nothing called `FFA_RUN` — Hafnium has no scheduler of its own
Version negotiation fails	Caller and SPM implement different FF-A versions; check `FFA_VERSION` first and downgrade
Double-fetch vulnerability found in review	`SHARE` used where `LEND` was required
Works on one board, not another	Manifest load addresses are physical and board-specific

Check yourself

- ✓ Explain the problem with a single S-EL1 trusted OS when components come from several vendors.
- ✓ Draw the exception levels and security states with an SPM and two secure partitions.
- ✓ Translate hypervisor/VM/vCPU into the FF-A vocabulary.

- ✓ Name the three FF-A memory-transfer semantics and say when each is correct.
- ✓ Explain why `LEND` closes a TOCTOU window that `SHARE` leaves open.
- ✓ Walk through a lend-retrieve-relinquish-reclaim sequence and say who edits the page tables.
- ✓ Read a secure-partition manifest and list exactly what that partition may touch.
- ✓ Explain why Hafnium deliberately has no scheduler.
- ✓ Describe how a Linux driver binds to a secure partition.
- ✓ State the Realm threat model in one sentence and say what a Granule Protection Fault means.

Nested virtualisation

There is one EL2 and two hypervisors that want it. Resolving that — letting a guest believe it owns the virtualisation extensions while the real owner watches — is the most intricate trick in the architecture, and it is why your cloud VM can itself run Docker Desktop, Android emulators and CI runners that expect KVM.

The names, first

```
L0   the real hypervisor, running at real EL2.
     The only software with genuine EL2 privilege.

L1   a GUEST that is itself a hypervisor. It believes it
     runs at EL2. It actually runs at EL1.

L2   L1's guest. It believes it runs at EL1. It actually
     also runs at EL1 - but with L0 having composed L1's
     intentions into the real hardware.

+-------------------------------------------+
|  L2 guest  (thinks: EL1)     really: EL1  |
+-------------------------------------------+
|  L1 hypervisor (thinks: EL2)  really: EL1 |
|     - has "its own" HCR_EL2, VTTBR_EL2 ...|
|       every one of them a value in memory |
+-------------------------------------------+
|  L0 hypervisor            really: EL2     |
|     - owns the actual system registers    |
|     - owns the actual stage-2 tables      |
+-------------------------------------------+
|  hardware                                 |
+-------------------------------------------+

Real-world instances:
  L0 = your cloud provider's KVM
  L1 = the KVM inside your VM
  L2 = the container-in-a-VM, the Android emulator, the
       CI runner's test VM

Or, on a phone:
  L0 = pKVM
  L1 = the Android guest kernel running its own VMs
  L2 = a protected app VM
```

How L0 fakes EL2 for L1

```
/* L1 executes code that only EL2 may execute. Since L1 is
   really at EL1, that traps to L0. L0 then EMULATES an EL2
   for it.                                                  */

  L1 does:  MSR VTTBR_EL2, x0
     -> trap to L0 (EC = 0x18, or a dedicated nested trap)
     -> L0 records: "L1's virtual VTTBR_EL2 = x0"
        It does NOT program the hardware VTTBR_EL2 with it.

  L1 does:  MSR HCR_EL2, x0
     -> trap. L0 records L1's intended trap configuration and
        RECOMPUTES the real HCR_EL2 as roughly
             real = L0's own requirements
                  UNION L1's requirements
        so that anything either hypervisor wanted to trap
        still traps - to L0, which then decides whether the
        trap belongs to L1.

  L1 does:  ERET into L2
     -> trap. L0 switches to "running L2" mode: it loads the
        shadow stage-2 (below), sets the real HCR_EL2 for an
        L2 guest, and ERETs.

  L2 traps (say, an MMIO access)
     -> arrives at L0, because all traps arrive at real EL2.
     -> L0 asks: did L1 ask for this to trap?
          YES -> synthesise an exception INTO L1: set L1's
                 virtual ESR_EL2, ELR_EL2, SPSR_EL2 and enter
                 L1 at its virtual VBAR_EL2.
          NO  -> handle it myself and resume L2 directly.

/* Every EL2 register L1 touches becomes a field in a struct.
   ARMv8.4-NV and 8.4-NV2 help enormously: NV redirects
   _EL2 register accesses at EL1 to trap, and NV2 redirects
   many of them to a MEMORY PAGE instead of trapping at all -
   so L1 writes "VTTBR_EL2" and the value lands in a VNCR page
   that L0 reads later, with no exception taken.            */
```

> **NV2 IS THE DIFFERENCE BETWEEN "POSSIBLE" AND "USABLE"**
>
> Without `ARMv8.4-NV2`, every single EL2 register access by L1 is a trap, and a hypervisor touches those constantly. NV2 gives L1 a page of memory that the hardware treats as the EL2 register file for most registers. L1's ordinary `MSR`/`MRS` become memory accesses; L0 reads the page when it needs to. Nested virtualisation on hardware without NV2 works and is slow enough that few people use it.

Shadow stage-2: two translations, one MMU

```
The hardware has ONE stage-2. Nesting needs two:

  L2 virtual address
     |  L2's stage 1 (L2's own page tables)
     v
  L2 intermediate physical address
     |  L1's stage 2  - L1 built these tables and believes
     |                  they are in force
     v
  L1 intermediate physical address
     |  L0's stage 2  - the real one
     v
  real physical address

The hardware can do one of those. So L0 COMPOSES them:

  shadow_s2[L2_ipa] = L0_s2[ L1_s2[L2_ipa] ]

and programs the shadow into the real VTTBR_EL2 whenever L2
runs. This is exactly the shadow-page-table technique that
stage 2 was invented to eliminate (chapter 2) - it has
returned, one level up.

Maintaining it:
  * L1 edits its stage-2 tables. Those tables live in L1's
    memory, so L0 can write-protect them and catch edits...
    expensive.
  * Better: L1 must issue a TLBI after any change (the
    architecture requires it). L0 traps the TLBI and
    invalidates the corresponding shadow entries. This is
    why nested virt REQUIRES trapping L1's TLB maintenance
    while ordinary virt deliberately does not.
  * The shadow is built lazily: on an L2 fault, walk L1's
    stage 2 in software, then L0's, install one entry.

COST: an L2 stage-2 fault costs a software walk of L1's
tables plus a shadow install - hundreds of times a plain
stage-2 fault. L2's steady-state execution, once the shadow
is warm, is close to native.
```

Interrupts and timers, nested

```
/* The vGIC problem: L1 wants to write list registers. There is
   one set, and L0 is using it.                              */

  L1 writes ICH_LR0_EL2 to inject into L2
     -> trap (or NV2 memory write)
     -> L0 records L1's intended list-register content
     -> when L0 runs L2, it merges:
           interrupts L0 wants to inject into L1
           interrupts L1 wants to inject into L2
        into the real list registers, with the right virtual
        INTIDs for whichever level is executing.

/* GICv4.1 helps: a VPE can be a NESTED vCPU, so an LPI can be
   delivered directly to L2 without either hypervisor running.
   Support for this is recent and not universal.            */

/* Timers: CNTVOFF_EL2 is singular, and now two hypervisors
   want to offset time.

     L1 sets its virtual CNTVOFF_EL2 = A  (to rebase L2)
     L0 has its own CNTVOFF_EL2      = B  (to rebase L1)

   The value the hardware needs while L2 runs is A + B, and L0
   computes it. ARMv8.6-ECV adds CNTPOFF_EL2 and finer control
   precisely because composing offsets by hand was error-prone.

   Symptom of getting it wrong: L2's clock drifts relative to
   L1's, and the guest's clocksource watchdog eventually
   declares the counter unstable.                           */
```

Turning it on

```
# Hardware requirement: FEAT_NV2 in practice. Check:
$ dmesg | grep -i nested
[    0.512] kvm [1]: Nested virtualization is experimental

# Enable in the host kernel (L0):
#   kvm-arm.mode=nested
# and the VM must be created asking for it:

$ qemu-system-aarch64 -M virt -accel kvm -cpu host,el2=on ...

# Inside L1, confirm it believes it has EL2:
$ dmesg | grep -i 'EL2\|kvm'
[    0.000000] CPU: All CPU(s) started at EL2
[    0.498112] kvm [1]: Hyp mode initialized successfully
$ ls -l /dev/kvm

# Then run L2 from inside L1 exactly as normal.

# x86 equivalent, for comparison - the concept is identical:
$ cat /sys/module/kvm_intel/parameters/nested
Y
$ qemu-system-x86_64 -cpu host,+vmx ...

# EXPECTATIONS. Measure, do not assume:
#   L1 CPU-bound work      ~ native
#   L2 CPU-bound work      ~ native
#   L2 I/O and interrupts  noticeably slower - every exit that
#                          L1 must see is two exits
#   L1 booting L2          slow: the shadow stage-2 is cold and
#                          every page is a software walk
```

> **NESTED IS FOR CAPABILITY, NOT PERFORMANCE**
>
> People enable nesting so that something *works* — a CI job that spawns VMs, a developer's Android emulator, a container runtime that wants a VM per container. If L2's performance is on your critical path, the answer is almost always to flatten the stack rather than to tune the nesting.

Where it actually ships

Stack	L0	L1	L2	Why
Cloud CI	Provider KVM	Your VM's KVM	Test VMs	Build and test images that need a real kernel
Android	pKVM	Android kernel	Protected app VMs	Isolate a payment or biometric workload from Android itself
Windows on Arm	Hyper-V	The "root" Windows	WSL2, containers, VBS	Virtualisation-based security means Windows itself is already a guest
Automotive dev	Type-1 hypervisor	Linux domain	Emulated ECUs	Develop and test partitions without the target hardware
Hypervisor development	KVM	The hypervisor you are writing	Its guests	Debug an EL2 payload with gdb instead of a JTAG probe — see QEMU chapter 11

How it shows up

Symptom	Cause
L1 says `HYP mode not available`	The VM was not created with EL2 (`-cpu host,el2=on`), or L0 does not have nesting enabled
L2 boots then hangs at random points	Shadow stage-2 stale — L1's TLBI not trapped or not translated into shadow invalidation
L2 memory corruption under load	Shadow composition bug: permissions not intersected across both stage-2 levels
L2's clock drifts from L1's	Timer offsets not composed (`A + B`)
L2 never receives interrupts	L1's list-register writes recorded but not merged into the real list registers when L2 runs
Extremely slow, hardware supports nesting	No NV2 — every L1 EL2 register access is a trap
L1 crashes with an unexpected exception	L0 synthesised an exception into L1 with an `ESR` encoding L1 does not handle
Works with one vCPU, breaks with several	Shadow tables shared across vCPUs without the locking the real hardware provided implicitly

Check yourself

- ✓ Define L0, L1 and L2 and state the real exception level each runs at.
- ✓ Explain what happens when L1 executes `MSR VTTBR_EL2`.
- ✓ Explain how L0 computes the real `HCR_EL2` from its own needs and L1's.
- ✓ Describe what happens to an L2 trap and how L0 decides whether L1 should see it.
- ✓ Explain shadow stage-2 and write the composition rule.
- ✓ Say why nested virtualisation must trap L1's `TLBI` when ordinary virtualisation must not.
- ✓ Explain what FEAT_NV2 changes and why it matters so much.
- ✓ Explain how two levels of timer offset are combined.
- ✓ Give three real deployments of nested virtualisation and what each needs it for.
- ✓ Predict which of L2's workloads will be near-native and which will not.

Virtualisation performance and how to measure it

Virtualised code runs at native speed until it does not. All the cost is at boundaries — exits, page walks, interrupts, stolen time — and each has a specific measurement that identifies it. This chapter is the diagnostic procedure, in the order that finds problems fastest.

The cost model

```
Guest execution time =

    instructions that just run          (free)
  + stage-2 page walks                  (bounded, cacheable)
  + exits x cost-per-exit               (the big one)
  + time the vCPU was not scheduled     (steal)
  + interference from other guests      (cache, bandwidth)

Every performance problem you will meet is one of the last
four, and they are distinguishable:

  high exit count          -> the guest is asking too often
  high cost per exit       -> the wrong handler tier
  high steal time          -> the host is oversubscribed
  high cache miss rate,    -> a neighbour is thrashing the
    low exits, low steal      shared cache

Diagnose in that order. It is roughly the order of
likelihood, and each test is cheap.
```

Step 1: is it exits?

```
# The single most informative command:
$ sudo perf kvm --host stat live

Analyze events for all VMs, all VCPUs:
   VM-EXIT   Samples  Samples%  Time%  Min Time  Max Time  Avg time
      MMIO     48112    71.20%  44.1%    1.90us  180.11us   9.44us
       WFx     15003    22.20%  51.2%    0.50us   19.98ms   1.44ms
       HVC      3201     4.74%   1.1%    1.20us    9.02us   2.11us
    SYS64      1250     1.85%   3.6%    2.10us   14.44us   4.02us

# Read it as:
#   MMIO dominating      -> device emulation. Which device?
#   WFx dominating       -> the guest is idle, or spinning on a
#                           lock it cannot get (lock-holder
#                           preemption)
#   SYS64 significant    -> a trapped system register in a hot
#                           path; find which one
#   Huge Max Time on WFx -> normal (it is a sleep), ignore

# Which addresses are the MMIO exits hitting?
$ sudo perf record -e kvm:kvm_mmio -a -- sleep 5
$ sudo perf script | grep -o 'gpa 0x[0-9a-f]*' | sort | uniq -c | sort -rn
  41022 gpa 0x0a000050        # a virtio QueueNotify
   4180 gpa 0x08010000        # the GIC redistributor
    900 gpa 0x09000000        # the UART

# 41k exits on a QueueNotify means no ioeventfd. That is a
# one-line fix (chapter 13) worth an order of magnitude.

# Which system registers?
$ sudo perf record -e kvm:kvm_sys_access -a -- sleep 5
$ sudo perf script | head
```

Dominant exit	Usual cause	Fix
MMIO on a virtio notify address	No `ioeventfd`	Register one; enable `EVENT_IDX`
MMIO on the GIC distributor	Guest reprogramming interrupts constantly	Fix the guest driver; configure once at init
MMIO on a legacy device	Emulating an old NIC or IDE controller	Switch to virtio
MMIO scattered across a device's whole register block	A polling driver	Interrupts, or passthrough
WFx with short durations	Lock-holder preemption	Pin vCPUs; stop oversubscribing
WFx with long durations	Guest is idle	Nothing — this is correct
SYS64 on `CNTP_*`	Guest on the physical timer	Fix the guest device tree (chapter 12)
SYS64 on ID registers	Guest probing features in a loop	Rare; usually a guest bug
Guest faults (stage-2)	Memory being populated, or dirty logging active	Pre-fault with huge pages; expected during migration

Step 2: is it steal time?

```
# In the guest:
$ vmstat 1 5
procs -----------memory---------- ---system-- ------cpu-----
 r  b   swpd   free  buff  cache   in   cs us sy id wa st
 4  0      0 512300  2200 120440 1210 3402 52  8  9  0 31
                                                      ^^
# st = 31%. Nearly a third of this guest's time does not exist.
# No amount of guest-side tuning will recover it.

$ mpstat -P ALL 1
# per-vCPU steal - an uneven distribution points at pinning
# problems rather than global oversubscription

# On the host, confirm:
$ vmstat 1
$ cat /proc/pressure/cpu
some avg10=42.11 avg60=38.90 avg300=31.02 total=...
# PSI "some" above ~20 means real contention

# How oversubscribed?
$ echo "vCPUs: $(pgrep -c 'CPU .*KVM')  pCPUs: $(nproc)"

# Fixes, in order of effectiveness:
#  1. reduce vCPU count or VM count (yes, really)
#  2. pin vCPUs to distinct pCPUs
#       $ taskset -cp 4 $(pgrep -f 'CPU 0/KVM')
#     or in libvirt:  <vcpupin vcpu='0' cpuset='4'/>
#  3. isolate the cores the latency-sensitive VM uses
#       isolcpus=4-7 nohz_full=4-7 rcu_nocbs=4-7
#  4. give the vCPU threads a real-time policy - carefully;
#     a runaway RT vCPU thread can wedge the host
#       $ chrt -f -p 10 $(pgrep -f 'CPU 0/KVM')
```

Step 3: is it the page tables?

```
# A two-stage walk can be 24 memory accesses (chapter 5). Huge
# pages at BOTH stages cut it dramatically.

# Host: back guest RAM with huge pages
$ echo 2048 | sudo tee /proc/sys/vm/nr_hugepages
$ qemu-system-aarch64 \
    -object memory-backend-file,id=mem,size=4G,\
mem-path=/dev/hugepages,share=on,prealloc=on \
    -M virt,memory-backend=mem ...

# Or transparent huge pages, which KVM will happily use at
# stage 2 if the host allocation is contiguous:
$ cat /sys/kernel/mm/transparent_hugepage/enabled
[always] madvise never

# Verify KVM actually built large stage-2 blocks:
$ sudo grep -r . /sys/kernel/debug/kvm/*/  2>/dev/null | grep -i huge

# Guest: measure the TLB pressure directly
$ sudo perf stat -e \
    dTLB-load-misses,iTLB-load-misses,instructions,cycles \
    -- ./workload

# A guest with a much worse dTLB-miss rate than the same
# workload on the host is paying for two-stage walks. Huge
# pages are the fix; there is no software workaround.

# prealloc=on matters more than it looks: without it, the guest
# faults every page in on first touch, and a 4 GB guest spends
# its first seconds taking a million stage-2 faults.
```

Step 4: is it a neighbour?

```
# Symptom: low exits, low steal, and still slow. Something
# else on the machine is evicting your cache lines or
# saturating memory bandwidth.

# Measure the shared resources:
$ sudo perf stat -e \
    LLC-loads,LLC-load-misses,cache-misses,cache-references \
    -a -- sleep 10

# arm64 PMU events for memory traffic (names vary by part):
$ sudo perf list | grep -i -E 'bus|l2d|ll_cache'
$ sudo perf stat -e armv8_pmuv3/ll_cache_rd/,armv8_pmuv3/ll_cache_miss_rd/ \
    -a -- sleep 5

# Prove it by removing the neighbour:
#   run your benchmark alone   -> baseline
#   run it with the suspect VM -> the difference IS the
#                                 interference
# That difference is your worst case, and it is what belongs in
# a capacity plan or a safety case (chapter 18).

# If the silicon has MPAM, partition the cache properly:
$ ls /sys/fs/resctrl/ 2>/dev/null
$ cat /sys/fs/resctrl/info/L3/num_closids
# then assign the latency-sensitive VM its own cache portion.

# If it does not, your options are cache colouring (chapter 18)
# or not co-locating those workloads.
```

> **THE MEASUREMENT DISCIPLINE**
> Always measure the same workload three ways: on bare metal, in the VM alone, and in the VM with realistic neighbours. The first two differences tell you the virtualisation cost; the third tells you the co-location cost. Reporting only "the VM is 20% slower" without saying which of those you measured makes the number useless.

A tuning checklist that actually helps

Change	Typical effect	When
`-accel kvm` instead of TCG	10–100x	Always, if the host architecture matches
`-cpu host`	Enables the guest to use every CPU feature	Always, unless you need migration compatibility
virtio + vhost instead of emulated devices	5–50x on I/O	Always
`ioeventfd` on queue notifies	2–10x on I/O	Always (QEMU does it by default; check your own VMM)
Huge pages, preallocated	5–30% on memory-heavy work; large boot-time win	Whenever the guest is big or TLB-sensitive
vCPU pinning	Removes jitter; small mean improvement	Latency-sensitive guests
`isolcpus` / `nohz_full` on the host	Large tail-latency improvement	Real-time guests

Change	Typical effect	When
Multiqueue virtio-net	Scales with vCPUs	Network-heavy guests with several vCPUs
Passthrough	Removes I/O cost entirely	When you can give up migration and sharing
VHE (do not force nVHE)	~2x on exit cost	Never set `kvm-arm.mode=nvhe` without a reason
Fewer vCPUs	Often the largest single win	Whenever steal time is non-trivial

Benchmarks worth running

```
# Exit cost, isolated. A guest loop that only traps:
#    1: hvc #0
#       b 1b
# Time N iterations. This is your floor.

# Memory, in the guest and on the host:
$ sysbench memory --memory-block-size=1M --memory-total-size=20G run
$ stream_c.exe            # if you have STREAM built

# Block I/O - and use O_DIRECT or you measure the page cache:
$ fio --name=r --rw=randread --bs=4k --iodepth=32 --direct=1 \
      --ioengine=libaio --runtime=30 --filename=/dev/vdb

# Network:
$ iperf3 -c host -P 4 -t 30           # throughput
$ netperf -H host -t TCP_RR           # latency, the harder one

# Latency and jitter - the number that matters for embedded:
$ cyclictest -S -p 90 -i 200 -d 0 -m -D 60
T: 0 (  911) P:90 I:200 C: 300000 Min: 6 Act: 11 Avg: 12 Max: 89

# Run cyclictest in the guest WHILE running a stressor in a
# neighbouring guest. The Max column with interference is the
# only latency figure worth quoting.
$ stress-ng --cpu 4 --vm 2 --vm-bytes 1G --cache 4 --timeout 60s
```

How it shows up

Symptom	Cause
Guest 50–100x slower than the host at everything	TCG, not KVM. Check `/dev/kvm` and `-accel`
CPU work fast, I/O terrible	Emulated devices, or missing `ioeventfd`/vhost
Everything fine except one driver	That driver polls a register; every read is an exit
Steal time high, host looks idle	vCPU threads pinned to the same pCPU, or a cgroup CPU quota
Big guest slow for the first minute, then fine	Memory faulted in on demand; use `prealloc=on`
Worse dTLB miss rate than bare metal	Two-stage walks without huge pages
Latency spikes correlate with a neighbour's activity	Cache or memory-bandwidth interference, not a hypervisor problem
Idle guest burning a core	`WFI` not trapped, or a guest spinning
Exit rate fine, cost per exit terrible	Exits reaching user space that should be handled in the kernel

Symptom	Cause
Performance regressed after a kernel update	Check for `kvm-arm.mode=` on the command line and for a mitigation that changed the exit path

Check yourself

- ✓ Write the cost model and name the four terms that are not free.
- ✓ Give the order in which you would test for each, and why that order.
- ✓ Interpret a `perf kvm stat live` table dominated by MMIO, and by WFx.
- ✓ Find which guest-physical addresses are causing the most exits.
- ✓ Explain what steal time measures and name three fixes.
- ✓ Explain why huge pages matter more under virtualisation than on bare metal.
- ✓ Distinguish interference from steal time by measurement.
- ✓ List the five highest-value tuning changes and say when each applies.
- ✓ Describe the three-way measurement discipline and why a single number is not enough.
- ✓ Design an experiment that produces a defensible worst-case latency figure for a guest.

TRACK

QEMU & emulation

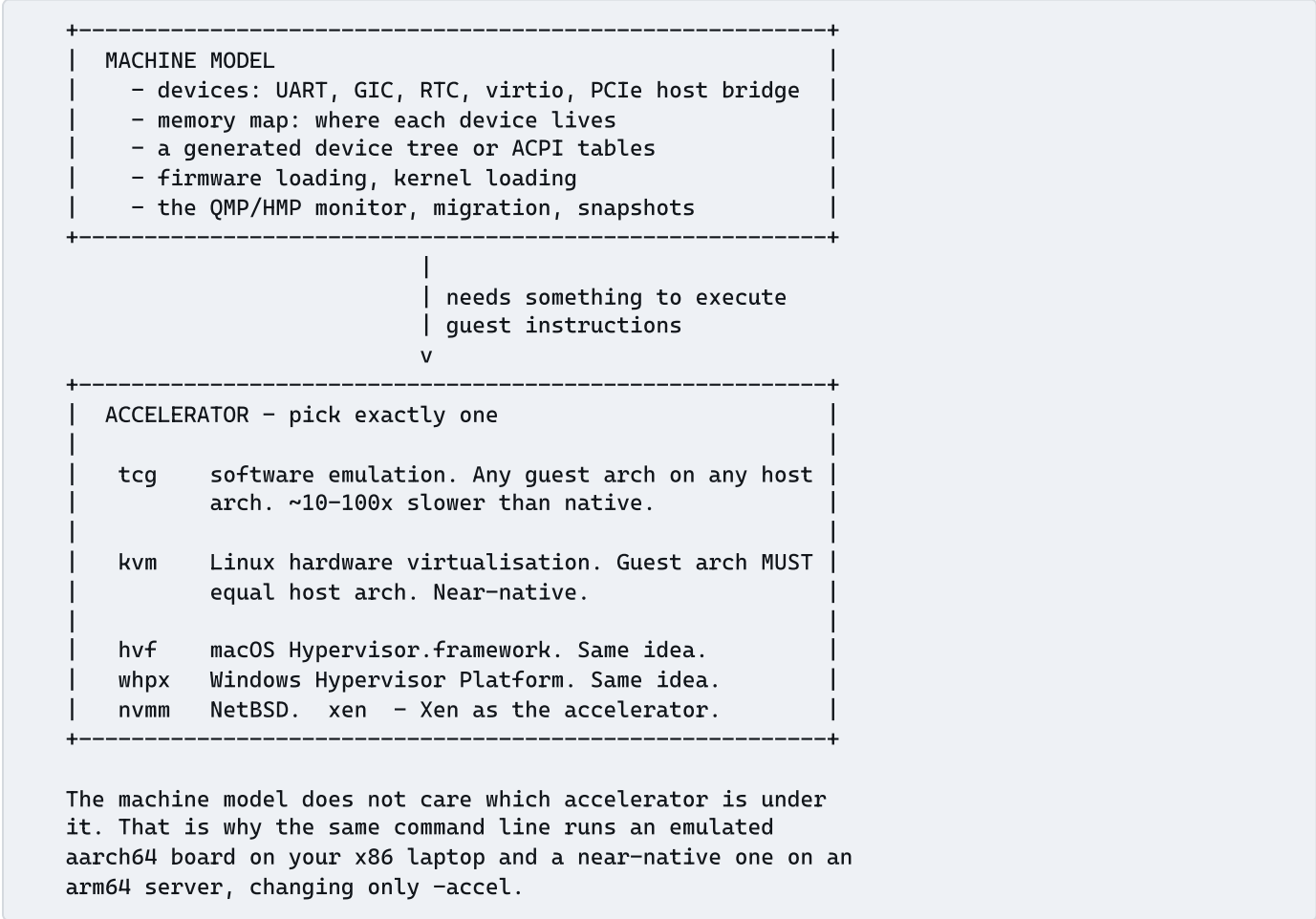
The machine you develop on when the hardware does not exist yet. Building it, booting it, wiring devices into it and debugging through it.

15 chapters

What QEMU is and its four modes

QEMU is two separable things that ship in one binary: a CPU emulator that can execute any supported architecture on any other, and a machine model — devices, buses, firmware, a device tree — that turns a CPU into a computer. Which of those you are using at a given moment decides everything about your speed and your debugging options.

The two halves



The machine model does not care which accelerator is under it. That is why the same command line runs an emulated aarch64 board on your x86 laptop and a near-native one on an arm64 server, changing only `-accel`.

The four ways to run it

Mode	Binary	What it emulates	Use it for
System emulation	`qemu-system-aarch64`	A whole machine: CPU, memory, devices, firmware	Kernels, drivers, bootloaders, OP-TEE, hypervisors — everything in this site
User-mode emulation	`qemu-aarch64`	One Linux process; syscalls are translated to host syscalls	Running a cross-compiled binary or test suite without a VM
Accelerator only	`qemu-system-*` with `-accel kvm`	Nothing — the hardware runs the guest	Production virtualisation
Tools	`qemu-img`, `qemu-nbd`, `qemu-storage-daemon`	Disk images, no CPU involved	Creating, converting and mounting images

```
# System emulation: a whole aarch64 machine, on any host
$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 1G \
    -kernel Image -append "console=ttyAMA0" -nographic

# User-mode: run ONE aarch64 binary on an x86 host
$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./my-arm-binary
# Or transparently, via binfmt_misc:
$ sudo apt install qemu-user-static binfmt-support
$ ./my-arm-binary          # the kernel invokes qemu-aarch64 for you
# This is how "docker run --platform linux/arm64" works on an
# x86 machine, and it is invaluable for running a cross-compiled
# test suite in CI.

# Tools:
$ qemu-img create -f qcow2 disk.qcow2 8G
$ qemu-img convert -O raw disk.qcow2 disk.raw
$ sudo qemu-nbd --connect=/dev/nbd0 disk.qcow2   # mount it
```

WHY THIS TRACK EXISTS

Every project on this site — a Linux driver, a Windows driver, an OP-TEE trusted application, a bare-metal EL2 payload — can be built and debugged on QEMU without buying hardware. More than that: QEMU gives you things hardware cannot, including a stopped CPU with a GDB stub attached before the first instruction, and a complete trace of every memory access. Learn it properly once and every other track gets cheaper.

What to expect from each accelerator

```
   Same aarch64 guest, same workload, three ways:

   TCG on x86 host          ~10–50x slower than native arm64.
                            Kernel boot: 20–60 seconds.
                            Perfectly usable for driver work.
                            Timing is meaningless.

   TCG on arm64 host        Same. TCG does not shortcut when
                            the architectures match — use KVM.

   KVM on arm64 host        ~1.0–1.05x native for CPU work.
                            Kernel boot: under a second.
                            Timing is roughly meaningful.

   The corollary that catches people out:

     A race condition that reproduces on hardware may NEVER
     reproduce under TCG, because TCG executes one guest
     instruction stream at a time per vCPU and its memory
     ordering is far stronger than real ARM's.

     And a race that reproduces under TCG is real, because TCG
     is a valid (if unusual) execution.

   So: use TCG to find logic bugs and KVM to find timing bugs,
   and never conclude "the lock is fine" from a TCG run.
```

MTTCG and the ordering trap

```
/* Multi-Threaded TCG runs each guest vCPU in its own host
   thread. It is the default where the guest and host memory
   models are compatible.                                    */

$ qemu-system-aarch64 -M virt -smp 4 -accel tcg,thread=multi ...
$ qemu-system-aarch64 -M virt -smp 4 -accel tcg,thread=single ...

/* When emulating a WEAKLY ordered guest (arm64) on a STRONGLY
   ordered host (x86), MTTCG is safe: the host provides more
   ordering than the guest requires, so a guest that forgot a
   barrier still works.

   THAT IS THE PROBLEM. Your code appears correct.

   Emulating a strongly ordered guest on a weakly ordered host
   is the dangerous direction, and QEMU inserts explicit
   barriers to be safe.                                      */

/* Practical rules:

   1. Missing barriers in guest code are INVISIBLE under TCG on
      x86. Test on real arm64 hardware, or under KVM on arm64,
      before believing your lock-free code.

   2. -icount gives you deterministic, reproducible timing:
        -icount shift=5,align=off,sleep=off
      Each guest instruction takes 2^5 virtual ns. The run is
      REPRODUCIBLE - the same input gives the same interleaving
      every time. Invaluable for debugging a heisenbug, and it
      also makes record/replay possible:
        -icount shift=auto,rr=record,rrfile=replay.bin
        -icount shift=auto,rr=replay,rrfile=replay.bin

   3. -singlestep (now -accel tcg,one-insn-per-tb=on) forces
      one instruction per translation block. Very slow, and it
      makes tracing and breakpoints exact.                   */
```

Checking your setup

```
$ qemu-system-aarch64 --version
QEMU emulator version 8.2.2

# Which accelerators were compiled in and are usable here?
$ qemu-system-aarch64 -accel help
Accelerators supported in QEMU binary:
tcg
kvm

# Is KVM actually available?
$ ls -l /dev/kvm
crw-rw---- 1 root kvm 10, 232 /dev/kvm
$ groups | grep -q kvm && echo "you can use it" || \
    echo "run: sudo usermod -aG kvm $USER, then log out and in"

# Which machines does this binary know?
$ qemu-system-aarch64 -M help | head -20
virt                 QEMU 8.2 ARM Virtual Machine
raspi3b              Raspberry Pi 3B
sbsa-ref             QEMU SBSA Reference Platform
vexpress-a15         ARM Versatile Express for Cortex-A15
xlnx-zcu102          Xilinx ZynqMP ZCU102

# Which CPUs?
$ qemu-system-aarch64 -cpu help | head
cortex-a53  cortex-a57  cortex-a72  cortex-a76
neoverse-n1 neoverse-v1
max                  # every feature this QEMU implements
host                 # KVM only: pass the host CPU through

# The two you will use:
#   -cpu max   under TCG - gives you SVE, pointer auth, MTE,
#              every architectural feature, so you can develop
#              against hardware you do not own
#   -cpu host  under KVM - no translation, real features
```

> **`-CPU MAX` IS A SUPERPOWER**
>
> It implements architectural features that shipping silicon may not have: SVE2, MTE, pointer authentication, FEAT_NV2 for nested virtualisation, S-EL2. You can write and debug code for a CPU that does not exist yet. The catch is that `max` tracks the QEMU version, so pin your QEMU version in CI or your feature set moves under you.

How it shows up

Symptom	Cause
Guest boots but takes minutes	TCG. Expected. Use KVM if the architectures match
`Could not access KVM kernel module: Permission denied`	Not in the `kvm` group
`-accel kvm` fails on an x86 host with an arm64 guest	KVM cannot cross architectures. This is not a bug
Lock-free code passes under QEMU, fails on hardware	MTTCG on a strongly ordered host hid a missing barrier
A race never reproduces	Try `-icount` with different shifts, or single-thread TCG

Symptom	Cause
Timing measurements make no sense	TCG time is not real time; use `-icount` for determinism or KVM for realism
Feature present under `-cpu max`, absent on the board	Exactly what `max` is for — check `ID_AA64*` registers on the real part
Machine type not found	Wrong binary — `qemu-system-arm` is 32-bit, `qemu-system-aarch64` is 64-bit

Check yourself

- ✓ Name the two separable halves of QEMU and say which one `-accel` selects.
- ✓ List the four ways to run QEMU and give a use for each.
- ✓ Explain why `-accel kvm` requires matching architectures and TCG does not.
- ✓ Give the approximate slowdown of TCG and say what it does and does not invalidate.
- ✓ Explain why a missing memory barrier can be invisible under TCG on an x86 host.
- ✓ Say what `-icount` buys you and name one thing it enables.
- ✓ Explain the difference between `-cpu max` and `-cpu host`.
- ✓ Determine, on your own machine, which accelerators and machine types are available.
- ✓ Explain how `binfmt_misc` plus user-mode QEMU lets you run foreign binaries transparently.

TCG: how emulation actually works

QEMU does not interpret instructions one at a time — that would be a hundred times slower still. It translates blocks of guest code into host code, caches them, and chains them together so that straight-line guest execution never returns to the translator. Understanding that machinery explains every performance surprise and every debugging limitation you will meet.

Translate, cache, chain

```
guest PC = 0x40080000, nothing cached
     |
     v
+-------------------------------------------------+
| 1. FIND: is there a translation block for this  |
|    (PC, CPU flags, MMU mode)?  No.              |
+-------------------------------------------------+
     |
     v
+-------------------------------------------------+
| 2. TRANSLATE. Decode guest instructions from the |
|    PC until a branch, a system instruction, or a |
|    page boundary. Emit TCG IR - a small, target- |
|    independent RISC-like intermediate language.  |
+-------------------------------------------------+
     |
     v
+-------------------------------------------------+
| 3. OPTIMISE the IR: dead code, constant folding, |
|    liveness. Cheap passes only - translation time|
|    is on the critical path.                      |
+-------------------------------------------------+
     |
     v
+-------------------------------------------------+
| 4. EMIT host machine code into the code cache.   |
+-------------------------------------------------+
     |
     v
+-------------------------------------------------+
| 5. EXECUTE it. Guest registers live in a struct  |
|    (CPUARMState); the emitted code loads and     |
|    stores them.                                  |
+-------------------------------------------------+
     |
     v
+-------------------------------------------------+
| 6. CHAIN. If the block ends in a direct branch   |
|    to a block already translated, PATCH the jump |
|    so it goes straight there next time, without  |
|    returning to the dispatcher at all.           |
+-------------------------------------------------+

A hot loop therefore becomes host code jumping directly to
host code. The translator runs once. That is why TCG is
10-50x slower than native and not 1000x.
```

The intermediate representation

```
# See it for yourself. This is the single most useful QEMU
# debugging flag nobody knows about.

$ qemu-system-aarch64 -M virt -cpu cortex-a57 -nographic \
    -kernel Image -d in_asm,op,out_asm 2>trace.log

# Or, more usefully, on a single small binary:
$ qemu-aarch64 -d in_asm,op,out_asm ./hello 2>trace.log

----------------
IN: main                        <-- the GUEST instructions
0x00400594:  d10083ff  sub sp, sp, #0x20
0x00400598:  a9017bfd  stp x29, x30, [sp, #16]
0x0040059c:  910043fd  add x29, sp, #0x10
0x004005a0:  94000010  bl  #+0x40

OP:                             <-- the TCG IR
 ld_i64 tmp0,env,$0xfffffffffffffd8
 movi_i64 tmp1,$0x20
 sub_i64 tmp0,tmp0,tmp1
 st_i64 tmp0,env,$0xfffffffffffffd8
 ...
 goto_tb $0x0
 exit_tb $0x7f2c14000043

OUT: [size=124]                 <-- the HOST instructions
0x7f2c14000000:  48 8b 6d f0    mov -0x10(%rbp),%rbp
0x7f2c14000004:  48 83 ed 20    sub $0x20,%rbp
...

# The IR is deliberately tiny - about 150 operations. Adding a
# new guest architecture means writing a FRONT END that emits
# this IR; adding a new host means writing a BACK END that
# consumes it. That N+M rather than N*M structure is why QEMU
# supports so many combinations.
```

Flag	Shows	Volume
`-d in_asm`	Guest instructions as they are translated	One line per instruction, once per block
`-d op`	TCG IR before optimisation	Large
`-d op_opt`	IR after optimisation	Large
`-d out_asm`	Emitted host code	Large
`-d exec`	Every block *execution*	Enormous — this is the one that traces control flow
`-d cpu`	Full register dump at each block	Enormous
`-d int`	Every exception and interrupt taken	Modest — extremely useful
`-d mmu`	MMU faults and translation	Modest
`-d guest_errors`	Guest doing something invalid	Small — always turn this on
`-d unimp`	Unimplemented device or CPU features	Small — always turn this on

Helpers: the escape hatch

```
/* Some guest instructions are too complex to express in 150 IR
   operations. For those, TCG emits a CALL to a C function
   called a HELPER.                                           */

/* target/arm/helper.c - a real one, simplified: */

void HELPER(msr_i_daifset)(CPUARMState *env, uint32_t imm)
{
        env->daif |= (imm << 6) & PSTATE_DAIF;
        arm_rebuild_hflags(env);
}

/* And in the translator, target/arm/translate-a64.c: */

    gen_helper_msr_i_daifset(cpu_env, tcg_constant_i32(imm));

/* What becomes a helper:
     - system register access
     - MMU translation (the slow path; the fast path is the
       software TLB, below)
     - floating point and SIMD, mostly
     - cryptographic instructions
     - anything touching CPU mode or exception state

   What stays in IR:
     - arithmetic, logic, shifts
     - loads and stores (fast path)
     - branches

   A helper call costs a real host function call, so an
   instruction that becomes a helper is far slower than one
   that does not. This is why FP-heavy code is
   disproportionately slow under TCG, and why crypto benchmarks
   inside QEMU are meaningless.                               */
```

The software TLB

```
Every guest load and store needs a guest-virtual to
host-virtual translation. Doing a full page-table walk in
software per access would be catastrophic, so TCG keeps a
direct-mapped software TLB and inlines the lookup.

Emitted code for a guest LDR, conceptually:

  index = (guest_addr >> 12) & (CPU_TLB_SIZE - 1)
  entry = env->tlb_table[mmu_idx][index]

  if (entry.addr_read == (guest_addr & TARGET_PAGE_MASK)) {
          /* FAST PATH: a few host instructions            */
          host_addr = guest_addr + entry.addend;
          value = *(uint64_t *)host_addr;
  } else {
          /* SLOW PATH: call a helper that walks the guest
             page tables, fills the TLB entry, and may
             raise a guest fault - or dispatches to a
             DEVICE MODEL if the address is MMIO.          */
          value = helper_ldq_mmu(env, guest_addr, oi, ra);
  }

Consequences you can feel:

* A guest that thrashes its own TLB thrashes QEMU's too, and
  the slow path is a C function call plus a walk.
* MMIO is ALWAYS the slow path, by construction - that is
  how device models get called.
* mmu_idx encodes the exception level and translation
  regime, so a guest that switches EL frequently invalidates
  less than you would fear but still pays.
* Flushing: any guest TLBI causes QEMU to flush its software
  TLB. A guest doing aggressive TLB maintenance is slow
  under TCG for a reason that does not exist on hardware.
```

Self-modifying code and page protection

```
/* The cached translation is only valid while the guest code it
   came from is unchanged. If the guest rewrites its own
   instructions - JIT compilers, bootloaders relocating
   themselves, module loaders - QEMU must notice.

   Mechanism: any guest page that has been translated is
   marked, and writes to it are detected. The affected
   translation blocks are invalidated and retranslated.     */

  Symptoms of a guest that fights this:

    * A JIT (a JVM, V8, or the kernel's BPF JIT) inside a TCG
      guest is dramatically slower than the rest of the guest,
      because it repeatedly invalidates translations.

    * A guest that writes data into the same PAGE as code -
      even not overlapping the code - triggers invalidation.
      Literal pools next to code, or a bootloader keeping
      variables in its text section, do this.

  Tuning the code cache, which matters for large guests:

$ qemu-system-aarch64 -accel tcg,tb-size=512 ...
      # 512 MB of translation cache. Default is smaller and is
      # sized from RAM. A large kernel plus a large userspace
      # can thrash a small cache, and the symptom is a guest
      # that gets SLOWER the longer it runs.

$ qemu-system-aarch64 -accel tcg,one-insn-per-tb=on ...
      # one guest instruction per block. Very slow, but exact:
      # every instruction boundary is a block boundary, so
      # breakpoints, tracing and -d exec become precise.
```

WHAT TCG DOES NOT MODEL

Caches, pipelines, branch prediction, memory latency, DVFS, and real interrupt timing. A cycle count from TCG is not merely inaccurate — it is meaningless. Use TCG to prove that logic is correct and to observe what the guest *did*; use hardware or KVM to find out how long it takes.

TCG plugins

```
# Since QEMU 4.2 you can load a shared library that gets called
# on translation and execution events. This is the closest
# thing to a hardware trace probe that costs nothing.

$ ls /usr/lib/qemu/         # or build them from tests/plugin/
libinsn.so  libmem.so  libhotblocks.so  libhotpages.so
libcache.so libhowvec.so

# Count every instruction executed:
$ qemu-aarch64 -plugin /usr/lib/qemu/libinsn.so -d plugin ./workload
insns: 48120311

# Find the hottest basic blocks:
$ qemu-system-aarch64 -M virt -kernel Image \
    -plugin /usr/lib/qemu/libhotblocks.so -d plugin ...
collected 25011 entries
pc, hits, insns
0x40081234, 1204551, 12
0x40095678,  883012,  8

# Model a cache and count misses - a real one, not a guess:
$ qemu-system-aarch64 ... \
    -plugin /usr/lib/qemu/libcache.so,dcachesize=32768,\
dassoc=4,dblksize=64

# Trace every memory access:
$ qemu-aarch64 -plugin /usr/lib/qemu/libmem.so,inline=on ./prog

# Writing your own is ~100 lines: register callbacks in
# qemu_plugin_install(), then use
#   qemu_plugin_register_vcpu_insn_exec_cb()
#   qemu_plugin_register_vcpu_mem_cb()
# This is how you answer "how many times does my driver touch
# that register?" without changing the driver.
```

How it shows up

Symptom	Cause
Guest gets slower the longer it runs	Code cache thrashing — raise `tb-size`
A JIT inside the guest is pathologically slow	Self-modifying-code invalidation; unavoidable under TCG
FP or crypto benchmarks absurdly slow	Those instructions become helper calls
Breakpoint fires at the wrong instruction	Block-granular execution; use `one-insn-per-tb=on`
`-d exec` produces gigabytes	Working as intended. Filter with `-D file` and a narrow window
Guest hits an instruction QEMU does not implement	`-d unimp` will say so — always enable it during bring-up
Device access silently returns zero	No model at that address; `-d guest_errors` reports it
Timing-dependent code behaves differently every run	Use `-icount` to make runs deterministic
Guest TLB-heavy workload disproportionately slow	Every guest `TLBI` flushes QEMU's software TLB

Check yourself

✓ Describe the six steps from an untranslated guest PC to executing host code.

- ✓ Explain block chaining and why it is the reason TCG is not catastrophically slow.
- ✓ Explain the N+M structure of front ends and back ends around the IR.
- ✓ Define a helper and name three instruction classes that become one.
- ✓ Describe the software TLB fast path and say what always takes the slow path.
- ✓ Explain why a guest JIT is slow under TCG.
- ✓ Name five `-d` flags and say what each is for.
- ✓ List four things TCG does not model, and say what that invalidates.
- ✓ Use a TCG plugin to count instructions or find hot blocks.
- ✓ Explain what `one-insn-per-tb=on` changes and when you would pay for it.

Building QEMU from source

Distribution packages lag, omit machine types, and are built without the debug information you need the first time a device model misbehaves. Building QEMU takes about ten minutes and gives you a tree you can patch — which you will, the first time you add a device model in chapter 13.

When the packaged one is not enough

You need	Because
A newer version	Machine types, CPU features (FEAT_NV2, MTE, S-EL2) and virtio devices land in QEMU long before distributions ship them
Debug symbols and assertions	When a device model asserts, you want a stack trace with line numbers, not `??`
To add or modify a device	Chapter 13
TCG plugins	Often not built or shipped by distributions
A reproducible CI pin	`-cpu max` changes meaning between versions; pin the version
A specific board	Vendor forks (Xilinx, NXP) carry machine models not upstream

Dependencies

```
# Debian / Ubuntu
$ sudo apt update
$ sudo apt install -y \
    git build-essential ninja-build meson pkg-config python3-venv \
    libglib2.0-dev libpixman-1-dev libfdt-dev zlib1g-dev \
    libslirp-dev libcap-ng-dev libattr1-dev \
    libaio-dev libbpf-dev libcurl4-gnutls-dev \
    libgnutls28-dev libnfs-dev libssh-dev \
    libncurses-dev libsdl2-dev libgtk-3-dev \
    libspice-server-dev libusb-1.0-0-dev libusbredirhost-dev \
    libvirglrenderer-dev libepoxy-dev \
    flex bison

# Fedora / RHEL
$ sudo dnf install -y \
    git gcc make ninja-build meson glib2-devel pixman-devel \
    libfdt-devel zlib-devel libslirp-devel libcap-ng-devel \
    libaio-devel gnutls-devel ncurses-devel SDL2-devel \
    gtk3-devel libusbx-devel spice-server-devel flex bison

# Arch
$ sudo pacman -S base-devel git ninja meson glib2 pixman dtc \
    libslirp libcap-ng libaio gnutls sdl2 gtk3 libusb

# macOS (Homebrew) - hvf accelerator, useful on Apple silicon
$ brew install ninja meson glib pixman dtc pkg-config libslirp

# The three that actually matter and are most often missing:
#   libglib2.0-dev   - QEMU will not configure without it
#   libpixman-1-dev  - graphics; required even headless
#   libslirp-dev     - user-mode networking (-netdev user)
#                      Without it, "-netdev user" silently
#                      disappears from the help output and you
#                      will spend an hour wondering why.
```

The build

```
$ git clone https://gitlab.com/qemu-project/qemu.git
$ cd qemu

# Pin a release. Building master is fine for development but a
# terrible idea for anything you want to reproduce later.
$ git checkout v8.2.2
$ git submodule update --init --recursive

# Out-of-tree build. Always.
$ mkdir build && cd build

# Minimal, fast build: only the targets you need.
$ ../configure \
    --target-list=aarch64-softmmu,arm-softmmu,aarch64-linux-user \
    --enable-slirp \
    --enable-debug \
    --enable-debug-info \
    --enable-plugins \
    --prefix=$HOME/opt/qemu

$ ninja -j$(nproc)
$ ninja install

# Building ALL targets takes 20+ minutes and produces 40
# binaries you will never run. --target-list is the single
# most valuable configure flag.

$ ~/opt/qemu/bin/qemu-system-aarch64 --version
QEMU emulator version 8.2.2

# Put it on your PATH ahead of the packaged one:
$ export PATH=$HOME/opt/qemu/bin:$PATH
```

configure flag	Effect
`--target-list=`	Which guest architectures to build. `*-softmmu` = system emulation, `*-linux-user` = user mode
`--enable-debug`	No optimisation, assertions on. Roughly 2x slower; worth it while developing a device
`--enable-debug-info`	Symbols without disabling optimisation — the middle ground
`--enable-plugins`	TCG plugin support (chapter 12)
`--enable-trace-backends=log,ftrace`	Built-in tracepoints
`--enable-slirp`	User-mode networking
`--enable-sanitizers`	ASan/UBSan. Invaluable when your own device model corrupts memory
`--disable-werror`	Needed when building an older QEMU with a newer compiler
`--static`	A binary you can copy to a machine with different libraries
`--enable-kvm`	Usually autodetected; force it to catch a missing header

TWO BUILDS, SIDE BY SIDE

Keep an optimised build for daily use and a `--enable-debug --enable-sanitizers` build for when something breaks. Meson supports several build directories against one source tree, so this costs disk space and nothing else:

```
$ mkdir build-opt build-dbg
$ (cd build-opt && ../configure --target-list=aarch64-softmmu \
      --prefix=$HOME/opt/qemu && ninja)
$ (cd build-dbg && ../configure --target-list=aarch64-softmmu \
      --enable-debug --enable-sanitizers && ninja)

# Run from the build directory without installing - the
# binaries work in place:
$ ./build-dbg/qemu-system-aarch64 -M virt -nographic ...
```

Firmware you will also need

```
# QEMU ships some firmware blobs, but for arm64 work you build
# your own. These are the ones this site's tracks use.

# 1. U-Boot - for a realistic boot chain
$ git clone https://source.denx.de/u-boot/u-boot.git
$ cd u-boot && make qemu_arm64_defconfig
$ make CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc)
# produces u-boot.bin -> qemu -bios u-boot.bin

# 2. EDK2 / UEFI - for an ACPI-based boot, as on servers
$ sudo apt install qemu-efi-aarch64
$ ls /usr/share/AAVMF/AAVMF_CODE.fd
# -drive if=pflash,format=raw,readonly=on,file=AAVMF_CODE.fd

# 3. TF-A - EL3 firmware, needed for the OP-TEE track
$ git clone https://github.com/ARM-software/arm-trusted-firmware
$ cd arm-trusted-firmware
$ make CROSS_COMPILE=aarch64-none-elf- PLAT=qemu \
       BL33=/path/to/u-boot.bin all fip
# produces bl1.bin and fip.bin -> loaded with -bios and -device
# loader; the OP-TEE track's chapter 4 does this properly.

# 4. A cross toolchain, if you do not have one
$ sudo apt install gcc-aarch64-linux-gnu       # Linux userspace
$ sudo apt install gcc-aarch64-none-elf        # bare metal
# or fetch the Arm GNU toolchain:
#   https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads
```

Verifying the build

```
# 1. Does it run at all?
$ qemu-system-aarch64 -M virt -nographic -kernel /dev/null 2>&1 | head -2

# 2. Are the accelerators there?
$ qemu-system-aarch64 -accel help

# 3. Is slirp present? (the silent-failure one)
$ qemu-system-aarch64 -netdev help | grep -q user && \
    echo "user networking OK" || echo "MISSING libslirp"

# 4. Are plugins enabled?
$ ls build/tests/plugin/*.so

# 5. Run the smoke tests - fast, and they catch a broken build:
$ cd build && ninja check-qtest-aarch64

# 6. The full suite, if you have changed something:
$ ninja check           # unit tests
$ ninja check-qtest     # device-level tests
$ ninja check-tcg       # requires cross compilers

# 7. Boot something real. The definitive test:
$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -append "console=ttyAMA0 rdinit=/bin/sh" \
    -initrd initramfs.cpio.gz
```

How it shows up

Symptom	Cause
`ERROR: glib-2.56 gthread-2.0 is required`	`libglib2.0-dev` missing — the runtime library alone is not enough
`-netdev user` not accepted	Built without libslirp. Reconfigure with `--enable-slirp`
`Unsupported machine type`	Wrong target list, or wrong binary (`qemu-system-arm` vs `-aarch64`)
Build fails with new-compiler warnings	Old QEMU, new GCC. Add `--disable-werror`
Meson errors about a stale build directory	`rm -rf build` and reconfigure; meson caches aggressively
Submodule errors during configure	`git submodule update --init --recursive`
Enormous build time	No `--target-list` — you are building every architecture
Binary works locally, not on another machine	Shared-library mismatch; build `--static`
Device model bug with no useful backtrace	Built without `--enable-debug`

Check yourself

- ✓ Give three reasons to build QEMU rather than install it.
- ✓ Name the three most commonly missing build dependencies and what each breaks.
- ✓ Explain what `--target-list` controls and why it matters.
- ✓ Distinguish `--enable-debug` from `--enable-debug-info`.
- ✓ Explain the value of keeping two build directories.
- ✓ Name the four firmware components an arm64 development setup may need.

- ✓ Verify from the command line that user networking and plugins are available.
- ✓ Say why pinning a QEMU version matters for CI.
- ✓ Run the qtest suite for aarch64 and interpret a failure.

The AArch64 virt machine

There is no such board. `virt` is a machine designed to be virtualised: a memory map chosen for convenience, only devices worth emulating, and a device tree generated at runtime to describe whatever you asked for. It is the target for every arm64 exercise on this site, and knowing its map by heart saves an enormous amount of time.

The memory map

```
0x0000_0000  +---------------------------+
             |  flash 0 (secure)  64 MB  |  -bios / pflash
0x0400_0000  +---------------------------+
             |  flash 1           64 MB  |  UEFI vars
0x0800_0000  +---------------------------+
             |  GIC distributor    64 KB |
0x0801_0000  +---------------------------+
             |  GICv2 CPU iface / GICv3  |
             |  redistributors           |
0x0808_0000  +---------------------------+
             |  GICv3 ITS                |
0x0809_0000  +---------------------------+
             |  GICv2m (MSI for v2)      |
0x0900_0000  +---------------------------+
             |  PL011 UART        4 KB   |  ttyAMA0
0x0901_0000  +---------------------------+
             |  PL031 RTC                |
0x0903_0000  +---------------------------+
             |  PL061 GPIO               |
0x0905_0000  +---------------------------+
             |  ACPI GED / power button  |
0x0a00_0000  +---------------------------+
             |  virtio-mmio x 32         |  0x200 apart
             |    slot 0: 0x0a00_0000    |  IRQ 16
             |    slot 1: 0x0a00_0200    |  IRQ 17
             |    ...                    |  ...
             |    slot31: 0x0a003e00     |  IRQ 47
0x0c00_0000  +---------------------------+
             |  platform bus (dynamic)   |  vfio-platform
0x1000_0000  +---------------------------+
             |  PCIe MMIO (32-bit)       |
0x3eff_0000  +---------------------------+
             |  PCIe I/O port window     |
0x3f00_0000  +---------------------------+
             |  PCIe ECAM config space   |
0x4000_0000  +---------------------------+
             |                           |
             |  RAM  - starts here, size |
             |  from -m, grows upward    |
             |                           |
             +---------------------------+
             |  high PCIe MMIO (64-bit)  |  above RAM
             +---------------------------+

TWO ADDRESSES TO MEMORISE:
  0x0900_0000   the UART. Your first printf.
  0x4000_0000   where RAM starts. Where your kernel goes.

Everything else you can look up - and the way to look it up
is dumpdtb, below.
```

The device tree is generated, and you can read it

```
# QEMU builds a device tree describing exactly the machine you
# asked for - your CPU count, your memory size, your devices -
# and places it in memory for the kernel.

$ qemu-system-aarch64 -M virt,dumpdtb=virt.dtb \
    -cpu cortex-a57 -smp 4 -m 2G -nographic
# exits immediately, having written virt.dtb

$ dtc -I dtb -O dts virt.dtb -o virt.dts
$ less virt.dts

/dts-v1/;
/ {
    interrupt-parent = <0x8002>;
    #size-cells = <0x02>;
    #address-cells = <0x02>;
    compatible = "linux,dummy-virt";

    memory@40000000 {
        reg = <0x00 0x40000000 0x00 0x80000000>;
        device_type = "memory";
    };

    pl011@9000000 {
        clock-names = "uartclk\0apb_pclk";
        reg = <0x00 0x9000000 0x00 0x1000>;
        interrupts = <0x00 0x01 0x04>;
        compatible = "arm,pl011\0arm,primecell";
    };

    intc@8000000 {
        compatible = "arm,gic-v3";
        reg = <0x00 0x8000000 0x00 0x10000    /* distributor  */
               0x00 0x80a0000 0x00 0xf60000>; /* redistribs   */
        #interrupt-cells = <0x03>;
    };

    timer {
        compatible = "arm,armv8-timer";
        interrupts = <0x01 0x0d 0x104  /* secure phys  PPI 13 */
                      0x01 0x0e 0x104  /* non-sec phys PPI 14 */
                      0x01 0x0b 0x104  /* VIRTUAL      PPI 11 */
                      0x01 0x0a 0x104>;/* hyp          PPI 10 */
    };

    psci {
        compatible = "arm,psci-1.0";
        method = "hvc";
    };
};

# THIS IS THE SINGLE MOST USEFUL DEBUGGING TRICK IN THE TRACK.
# "My device does not appear in the guest" is answered in ten
# seconds by dumping the DTB and looking.

# From inside a running guest, the same information:
$ dtc -I fs -O dts /sys/firmware/devicetree/base | less
```

WHY IT IS GENERATED, NOT FIXED

A real board has a device tree committed to a repository, describing hardware that cannot change. `virt` has no fixed hardware: the number of CPUs, the amount of RAM, the GIC version, whether there is an ITS, and which virtio devices exist are all decided by your command line. Generating the tree at startup is the only way to keep it truthful, and it means you never edit a DTS to change your VM's memory.

The machine options that matter

Option	Values	Effect
`gic-version`	`2`, `3`, `4`, `host`, `max`	Which interrupt controller. `host` matches the hardware under KVM; `max` is the newest QEMU supports
`its`	`on` / `off`	The GICv3 ITS. Needed for PCIe MSI-X (virt chapter 11)
`virtualization`	`on` / `off`	Give the guest EL2, so it can run its own hypervisor
`secure`	`on` / `off`	Give the machine EL3 and a secure world. Required for TF-A and OP-TEE
`mte`	`on` / `off`	Memory Tagging Extension
`iommu`	`smmuv3`	An SMMU, for passthrough and DMA isolation experiments
`highmem`	`on` / `off`	Whether to place devices above 4 GB. Turn off for a 32-bit-limited guest
`acpi`	`on` / `off`	Generate ACPI tables as well as a device tree — the server boot path
`dumpdtb=`	filename	Write the generated device tree and exit
`memory-backend=`	object id	Back RAM with hugetlbfs or shared memory — needed for vhost-user and virtiofs

```
# The configuration this site's ARM and security tracks use:
$ qemu-system-aarch64 \
    -M virt,secure=on,virtualization=on,gic-version=3 \
    -cpu max -smp 2 -m 1G -nographic

# Everything on, for exploring:
$ qemu-system-aarch64 \
    -M virt,gic-version=max,its=on,virtualization=on,secure=on,\
mte=on,iommu=smmuv3,acpi=on \
    -cpu max -smp 4 -m 4G -nographic

# Native speed on an arm64 host:
$ qemu-system-aarch64 -M virt,gic-version=host -cpu host \
    -accel kvm -smp 4 -m 4G -nographic
```

`SECURE=ON` AND `-CPU MAX` TOGETHER

Turning on the secure world changes the boot: the CPU now starts at EL3, and QEMU expects firmware there. With `-kernel` QEMU synthesises a minimal EL3 stub that drops to EL1, which is fine for testing — but the moment you supply real firmware with `-bios`, the stub disappears and your image must handle EL3 itself. This trips up nearly everyone on their first OP-TEE build; see OP-TEE chapter 4.

The other arm64 machines

Machine	Models	Use it when
`virt`	Nothing real	Almost always. Fast, flexible, well-maintained
`sbsa-ref`	An Arm ServerReady platform	Testing UEFI/ACPI firmware as a server would boot it
`raspi3b`, `raspi4b`	Raspberry Pi	Reproducing a Pi-specific issue; slow and partially modelled
`xlnx-zcu102`	Xilinx ZynqMP	ZynqMP driver work — has the real memory map and many real IP blocks
`imx8mp-evk`	NXP i.MX8M Plus	i.MX driver work
`vexpress-a15`	ARM Versatile Express	Legacy 32-bit material
`none`	Nothing at all	Loading a bare-metal ELF with `-device loader` and nothing else

```
# List everything this binary knows:
$ qemu-system-aarch64 -M help

# And what a specific machine accepts:
$ qemu-system-aarch64 -M virt -machine help 2>&1 | head
# or, more reliably, ask QMP (chapter 10):
$ qemu-system-aarch64 -M virt -nographic -S \
    -qmp stdio <<'EOF' | python3 -m json.tool
{"execute":"qmp_capabilities"}
{"execute":"qom-list-properties","arguments":{"typename":"virt-8.2-machine"}}
EOF

# Which devices exist on this machine, at runtime:
# (in the monitor, Ctrl-A c)
(qemu) info qtree
(qemu) info mtree          # the memory map, authoritative
(qemu) info qom-tree
```

info mtree: the memory map, from the machine itself

```
(qemu) info mtree
address-space: memory
  0000000000000000-ffffffffffffffff (prio 0, i/o): system
    0000000000000000-000000007fffffff (prio 0, romd): virt.flash0
    0000000008000000-0000000000800fff (prio 0, i/o): gicv3_dist
    0000000008080000-00000000080bffff (prio 0, i/o): gicv3_its
    0000000009000000-0000000009000fff (prio 0, i/o): pl011
    0000000009010000-0000000009010fff (prio 0, i/o): pl031
    000000000a000000-000000000a0001ff (prio 0, i/o): virtio-mmio
    000000000a000200-000000000a0003ff (prio 0, i/o): virtio-mmio
    ...
    0000000040000000-00000000bfffffff (prio 0, ram): mach-virt.ram

# This is the truth. If your guest faults at an address, find
# it here first: either there is no region (nothing modelled)
# or there is one and your driver is misusing it.

# Regions with "romd" are read-mostly flash; "i/o" are device
# models; "ram" is real backing memory.

# Overlaps are resolved by priority, which is how a boot ROM
# can shadow RAM at address 0 and then be removed.
```

How it shows up

Symptom	Cause
No console output at all	Wrong console in `-append`; it is `console=ttyAMA0` on `virt`, not `ttyS0`
Guest cannot find its root device	virtio device added but the guest's device tree slot differs; check `dumpdtb`
PCIe device gets no MSI-X	`its=off`, or `gic-version=2`
Kernel panics with "no matching CPU" or hangs at "Starting kernel"	Loaded at the wrong address — RAM starts at `0x4000_0000`
Guest cannot run its own hypervisor	`virtualization=on` missing
TF-A or OP-TEE will not start	`secure=on` missing
Boots with `-kernel`, fails with `-bios`	The synthesised EL3 stub is gone; your firmware must handle EL3
virtio device present but never probed	Slot used out of order, or the guest kernel lacks `CONFIG_VIRTIO_MMIO`
Memory map differs from the documentation	The map has changed between QEMU versions — always `dumpdtb` rather than trusting a blog

Check yourself

- ✓ State the addresses of the UART and the start of RAM on `virt`, from memory.
- ✓ Explain why the device tree is generated at runtime rather than committed.
- ✓ Dump the device tree for a machine you configured and find the virtio-mmio slots.
- ✓ Name the machine options that add EL2, EL3, an ITS and an SMMU.
- ✓ Explain what changes when you pass `-bios` on a `secure=on` machine.
- ✓ Give the correct `console=` argument for this machine and say why.
- ✓ Use `info mtree` to determine whether an address is modelled.
- ✓ Choose a machine type for: kernel driver development, server firmware testing, and ZynqMP bring-up.
- ✓ Explain the difference between `gic-version=host` and `gic-version=max`.

Booting a kernel directly

`-kernel` lets you skip the bootloader entirely: QEMU loads your image, synthesises the handoff a bootloader would have performed, and jumps to it. It turns a five-minute build-flash-boot cycle into a two-second one, and it is how you should develop everything until the boot chain itself is the thing you are testing.

What `-kernel` actually does

```
Normally, on real hardware:

  ROM -> BL1 -> BL2 -> BL31 (EL3) -> U-Boot (EL2/EL1)
                                        |
                                        | reads a device tree
                                        | loads Image + initrd
                                        | sets x0 = DTB address
                                        | jumps to the kernel
                                        v
                                      Linux

With -kernel, QEMU IS the bootloader:

  1. Parse the image. An arm64 "Image" has a 64-byte header:
         0x00  code0, code1        (branch instructions)
         0x08  text_offset         (where to load, relative
                                    to the start of RAM)
         0x10  image_size
         0x18  flags
         0x38  magic "ARM\x64"
     QEMU reads text_offset and image_size and places the
     image accordingly - usually 0x4008_0000.

  2. Load the initrd, if given, somewhere above the kernel,
     and record its address and size.

  3. GENERATE the device tree (chapter 4), then PATCH it:
         /chosen/bootargs        = your -append string
         /chosen/linux,initrd-start
         /chosen/linux,initrd-end
     and write it into guest RAM.

  4. Write a tiny boot stub at the start of RAM that:
         sets x0 = DTB address
         x1 = x2 = x3 = 0
         branches to the kernel entry point
     For -smp N, secondary CPUs park in a PSCI-compatible
     wait loop.

  5. Release the CPU at the stub.

The kernel cannot tell the difference, which is the point.
```

The smallest thing that boots

```
# You need three things: a kernel, a console, and a root
# filesystem. Start with only the first two.

$ qemu-system-aarch64 \
    -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image \
    -append "console=ttyAMA0 earlycon"

# It boots, prints, and panics with "No working init found",
# because there is no root filesystem. That panic is SUCCESS:
# it means the kernel ran to the end of its own initialisation.

# Now add an initramfs and you have a usable system:
$ qemu-system-aarch64 \
    -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -initrd initramfs.cpio.gz \
    -append "console=ttyAMA0 earlycon rdinit=/bin/sh"

# Exit with Ctrl-A x.  (Ctrl-A c switches to the monitor.)

# The four arguments that matter, and why:
#
#   console=ttyAMA0   the PL011 at 0x09000000. WITHOUT THIS you
#                     see nothing after the kernel takes over
#                     its own console. It is the single most
#                     common "it hangs" cause.
#
#   earlycon          console output BEFORE the driver probes.
#                     On virt, the kernel finds the address in
#                     /chosen/stdout-path, so plain "earlycon"
#                     works with no arguments.
#
#   rdinit=/bin/sh    run a shell from the initramfs instead of
#                     looking for /sbin/init.
#
#   -nographic        no window; serial to your terminal.
#                     Implies -serial mon:stdio.
```

> **ALWAYS ADD `EARLYCON` WHILE BRINGING SOMETHING UP**
>
> Without it, the window between "kernel entered" and "console driver probed" is silent — and that window contains the MMU setup, the memory-map parse and the device-tree parse, which is exactly where early boot failures live. With it, you see `Booting Linux on physical CPU 0x0` almost immediately, and a hang after that is a much smaller problem than a hang before it.

Building a kernel worth booting

```
$ git clone --depth=1 -b v6.8 \
    https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
$ cd linux

$ export ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
$ make defconfig

# defconfig already includes virtio, PL011 and the GIC. Add the
# things you will want for this site's tracks:
$ scripts/config --enable  DEBUG_INFO_DWARF5 \
                 --enable  GDB_SCRIPTS \
                 --enable  DEBUG_KERNEL \
                 --enable  DYNAMIC_DEBUG \
                 --enable  MAGIC_SYSRQ \
                 --enable  KGDB \
                 --enable  FTRACE \
                 --enable  FUNCTION_TRACER \
                 --enable  KPROBES \
                 --enable  TEE \
                 --enable  OPTEE \
                 --enable  BLK_DEV_INITRD \
                 --set-val CONFIG_RANDOMIZE_BASE n
$ make olddefconfig
$ make -j$(nproc) Image modules

# Result: arch/arm64/boot/Image   (uncompressed - what -kernel
#                                  wants)
#         arch/arm64/boot/Image.gz (what a real board uses)
#         vmlinux                  (with symbols - what GDB
#                                   wants, chapter 11)

# RANDOMIZE_BASE off is deliberate: KASLR moves the kernel
# every boot, and then your GDB symbols are at the wrong
# addresses. Turn it off for debugging, on for realism.
# Alternatively pass "nokaslr" in -append.
```

A root filesystem in thirty seconds

```
# Option 1: busybox, statically linked. Smallest and fastest.

$ wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2
$ tar xf busybox-1.36.1.tar.bz2 && cd busybox-1.36.1
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
$ sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc)
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
       CONFIG_PREFIX=../rootfs install

$ cd ../rootfs
$ mkdir -p proc sys dev etc/init.d
$ cat > init <<'EOF'
#!/bin/sh
mount -t proc  none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
echo "=== up ==="
exec /bin/sh
EOF
$ chmod +x init
$ find . | cpio -o -H newc | gzip > ../initramfs.cpio.gz

# Boot it:
$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -initrd initramfs.cpio.gz \
    -append "console=ttyAMA0 earlycon"

# Option 2: use the host's own aarch64 container image
$ docker create --platform linux/arm64 --name x arm64v8/alpine
$ docker export x | (mkdir -p rootfs && cd rootfs && tar xf -)
$ docker rm x
# ...then cpio it as above, or make it a disk image (chapter 6).
```

Supplying your own device tree

```
# Sometimes you want to modify what QEMU generated - to add a
# node for a device model you are writing, or to change a
# property. The workflow is: dump, edit, feed back.

$ qemu-system-aarch64 -M virt,dumpdtb=virt.dtb -m 1G -smp 2
$ dtc -I dtb -O dts virt.dtb -o virt.dts

# Edit virt.dts - add your node:
#     my-device@c000000 {
#         compatible = "acme,widget";
#         reg = <0x0 0x0c000000 0x0 0x1000>;
#         interrupts = <0x0 0x30 0x4>;
#     };

$ dtc -I dts -O dtb virt.dts -o virt-mod.dtb
$ qemu-system-aarch64 -M virt -m 1G -smp 2 -nographic \
    -kernel Image -dtb virt-mod.dtb -append "console=ttyAMA0"

# CAUTION: -dtb REPLACES the generated tree entirely. If you
# then change -m or -smp, the tree no longer matches the
# machine and the guest will see the OLD memory size. Regenerate
# after any machine change. This causes hours of confusion, so
# script it:

$ cat > boot.sh <<'EOF'
#!/bin/sh
set -e
MEM=${MEM:-1G}; SMP=${SMP:-2}
qemu-system-aarch64 -M virt,dumpdtb=/tmp/base.dtb -m $MEM -smp $SMP
dtc -I dtb -O dts /tmp/base.dtb -o /tmp/base.dts 2>/dev/null
python3 add_my_node.py /tmp/base.dts /tmp/final.dts
dtc -I dts -O dtb /tmp/final.dts -o /tmp/final.dtb 2>/dev/null
exec qemu-system-aarch64 -M virt -m $MEM -smp $SMP -nographic \
    -kernel Image -dtb /tmp/final.dtb -append "console=ttyAMA0 earlycon"
EOF
$ chmod +x boot.sh
```

Choosing the exception level you land at

```
# -kernel normally enters at EL1 (or EL2 with virtualization=on).
# For bare-metal work you often want something else.

# Land at EL2, so you can write a hypervisor:
$ qemu-system-aarch64 -M virt,virtualization=on -cpu max \
    -kernel my-el2-payload.bin -nographic
# Confirm from inside: mrs x0, CurrentEL  -> 0x8 means EL2

# Land at EL3, for firmware work:
$ qemu-system-aarch64 -M virt,secure=on -cpu max \
    -bios my-el3-firmware.bin -nographic
# NOTE: -bios, not -kernel. -bios loads at address 0 and starts
# there with no stub, no device tree patching and no PSCI - you
# are the firmware now.

# Load a raw ELF anywhere, with full control:
$ qemu-system-aarch64 -M virt -cpu max -nographic \
    -device loader,file=payload.elf \
    -device loader,addr=0x40100000,cpu-num=0
# The second loader sets the PC of CPU 0.

# Load several blobs at chosen addresses - the pattern the
# OP-TEE track uses for TF-A + OP-TEE + U-Boot:
$ qemu-system-aarch64 -M virt,secure=on -cpu max -nographic \
    -bios bl1.bin \
    -device loader,file=fip.bin,addr=0x40100000 \
    -semihosting-config enable=on,target=native
```

> **`-KERNEL` VERSUS `-BIOS`**
> `-kernel` means "you are an OS; I will act as your bootloader". `-bios` means "you are the firmware; here is a bare machine". With `-bios` you get no device tree, no `-append`, no initrd loading and no PSCI implementation — QEMU stops helping. Mixing them up produces a payload that runs but finds `x0` pointing at nothing.

How it shows up

Symptom	Cause
Completely silent, no output ever	Missing `console=ttyAMA0`, or `-nographic` omitted so output went to a window
Prints "Booting Linux" then stops	Console handed over from earlycon to the real driver and the real console is wrong
Hangs before any output at all	Add `earlycon`; if still silent, the image is loaded at the wrong address or is the wrong format
`No working init found`	No initrd, or no `/init` in it, or it is not executable, or it is dynamically linked with no libraries
Guest sees the wrong memory size	A stale `-dtb` from a different `-m`
Secondary CPUs never come up	PSCI conduit mismatch, or `-bios` used so there is no PSCI at all
GDB symbols are all wrong	KASLR — add `nokaslr` or disable `RANDOMIZE_BASE`
`Image` rejected or garbage executed	Passed `Image.gz` or `vmlinux` to `-kernel` instead of `Image`
Works with `-kernel`, fails with `-bios`	Expected — no boot stub, no DTB, no PSCI

Symptom	Cause
initramfs unpacks but binaries fail with "not found"	Dynamically linked busybox without its loader; build static

Check yourself

- ✓ List the five things QEMU does on your behalf when you pass `-kernel`.
- ✓ Name the arm64 `Image` header fields QEMU reads and what it does with them.
- ✓ Explain what the kernel finds in `x0` at entry and who put it there.
- ✓ Give the four `-append` arguments needed for a first boot and what each does.
- ✓ Explain why `earlycon` is worth adding during bring-up.
- ✓ Build a static busybox initramfs and boot it.
- ✓ Describe the dump-edit-feed-back workflow for a custom device tree, and its main hazard.
- ✓ State the difference between `-kernel` and `-bios` in terms of what QEMU provides.
- ✓ Show how to land a payload at EL2 and at EL3.
- ✓ Diagnose a completely silent boot in three steps.

Building a root filesystem

A kernel without userspace panics in half a second. This chapter covers the four ways to give it one — an initramfs you assemble by hand, Buildroot, a debootstrap image, and a container image reused as a rootfs — and when each is the right tool.

Choosing

Method	Build time	Size	Use it for
Hand-built initramfs	Seconds	2–5 MB	Driver testing, boot debugging, anything where you want to see everything that runs
Buildroot	15–40 min first time, then incremental	10–100 MB	Realistic embedded userspace, cross-compiled packages, reproducible builds. The right default for this site's projects
debootstrap / mmdebstrap	2–5 min	300 MB–1 GB	You want `apt`, a package archive and a normal distribution
Container image export	Seconds	50–500 MB	You already have the userspace you want as a Docker image
Yocto	Hours	Anything	Product builds. Overkill for learning

Method 1: initramfs, by hand

```
#!/bin/sh
# mkinitramfs.sh - a complete, working root filesystem in one
# script. This is the one to keep.
set -e

BB_VER=1.36.1
ARCH=arm64
CROSS=aarch64-linux-gnu-
ROOT=$PWD/rootfs

# --- busybox, statically linked -----------------------------
if [ ! -d busybox-$BB_VER ]; then
    wget -q https://busybox.net/downloads/busybox-$BB_VER.tar.bz2
    tar xf busybox-$BB_VER.tar.bz2
fi
cd busybox-$BB_VER
make ARCH=$ARCH CROSS_COMPILE=$CROSS defconfig >/dev/null
sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
sed -i 's/CONFIG_TC=y/# CONFIG_TC is not set/' .config
make ARCH=$ARCH CROSS_COMPILE=$CROSS -j$(nproc) >/dev/null
make ARCH=$ARCH CROSS_COMPILE=$CROSS CONFIG_PREFIX=$ROOT install
cd ..

# --- the directory skeleton ---------------------------------
mkdir -p $ROOT/{proc,sys,dev,tmp,run,root,etc/init.d,lib/modules}

# --- init ---------------------------------------------------
cat > $ROOT/init <<'EOF'
#!/bin/sh
mount -t proc     proc     /proc
mount -t sysfs    sysfs    /sys
mount -t devtmpfs devtmpfs /dev
mount -t tmpfs    tmpfs    /tmp
mkdir -p /dev/pts && mount -t devpts devpts /dev/pts

# load any modules we shipped
for m in /lib/modules/*.ko; do
        [ -e "$m" ] && insmod "$m" 2>/dev/null
done

# bring up loopback and the virtio NIC if present
ip link set lo up 2>/dev/null
ip link set eth0 up 2>/dev/null && udhcpc -i eth0 -q 2>/dev/null

echo
echo "=== rootfs up: $(uname -srm) ==="
echo

exec /bin/sh
EOF
chmod +x $ROOT/init

# --- minimal /etc -------------------------------------------
echo "root:x:0:0:root:/root:/bin/sh" > $ROOT/etc/passwd
echo "root:x:0:"                     > $ROOT/etc/group
echo "qemu"                          > $ROOT/etc/hostname
cat > $ROOT/etc/fstab <<'EOF'
proc  /proc  proc  defaults 0 0
sysfs /sys   sysfs defaults 0 0
EOF
```

```
# --- pack ------------------------------------------------
( cd $ROOT && find . -print0 \
  | cpio --null -o --format=newc 2>/dev/null ) | gzip -9 \
  > initramfs.cpio.gz

echo "built: initramfs.cpio.gz ($(du -h initramfs.cpio.gz | cut -f1))"
```

```
$ ./mkinitramfs.sh
built: initramfs.cpio.gz (1.2M)

$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -initrd initramfs.cpio.gz \
    -append "console=ttyAMA0 earlycon"
...
=== rootfs up: Linux 6.8.0 aarch64 ===

/ #

# To add YOUR kernel module to the image, drop it in
# rootfs/lib/modules/ before packing. The init script loads
# everything it finds - which is exactly what you want while
# developing a driver (Linux track, chapter 28).
```

WHY STATIC LINKING MATTERS HERE

An initramfs has no dynamic loader unless you put one there, and a dynamically linked busybox fails with `not found` — a message that describes the missing *loader*, not the missing binary, and sends people hunting for the wrong thing. Build static and the problem cannot occur.

Method 2: Buildroot

```
$ git clone --depth=1 -b 2024.02 \
    https://gitlab.com/buildroot.org/buildroot.git
$ cd buildroot

# There is a ready-made config for exactly our target:
$ make qemu_aarch64_virt_defconfig

# Customise:
$ make menuconfig
#   Target packages -> Debugging  -> gdb, strace, ltrace
#   Target packages -> Networking -> openssh, iperf3
#   Target packages -> Hardware   -> optee-client, optee-test
#   Filesystem images -> ext2/3/4 root filesystem
#                     -> cpio the root filesystem (for initramfs)
#   Toolchain -> Build cross gdb for the host
#   Kernel -> (leave off if you build your own)

$ make -j$(nproc)          # 15-40 minutes the first time

$ ls output/images/
Image  rootfs.cpio.gz  rootfs.ext4  start-qemu.sh

# Buildroot even writes the QEMU command line for you:
$ ./output/images/start-qemu.sh
# or run it yourself:
$ qemu-system-aarch64 -M virt -cpu cortex-a53 -m 512M -nographic \
    -kernel output/images/Image \
    -drive file=output/images/rootfs.ext4,if=none,format=raw,id=hd0 \
    -device virtio-blk-device,drive=hd0 \
    -append "console=ttyAMA0 root=/dev/vda rootwait"

# Rebuild just one package after changing it:
$ make optee-client-rebuild
$ make                       # repacks the images

# Buildroot is the right answer for this site's OP-TEE track
# because it already knows how to build optee-client,
# optee-test and the TA development kit, and puts them in the
# right places.
```

Method 3: a real Debian

```
# When you want apt, a package archive, and systemd.

$ sudo apt install debootstrap qemu-user-static
$ mkdir -p rootfs
$ sudo debootstrap --arch=arm64 --foreign bookworm rootfs \
      http://deb.debian.org/debian

# Second stage runs arm64 binaries via binfmt_misc + qemu-user
$ sudo cp /usr/bin/qemu-aarch64-static rootfs/usr/bin/
$ sudo chroot rootfs /debootstrap/debootstrap --second-stage

# Configure it before first boot
$ sudo chroot rootfs /bin/bash <<'EOF'
echo "root:root" | chpasswd
echo qemu > /etc/hostname
apt-get update
apt-get install -y --no-install-recommends \
    systemd-sysv udev iproute2 openssh-server vim less
systemctl enable serial-getty@ttyAMA0.service
cat > /etc/fstab <<'FSTAB'
/dev/vda / ext4 defaults 0 1
FSTAB
EOF

# Turn the directory into a disk image
$ dd if=/dev/zero of=debian.ext4 bs=1M count=4096
$ mkfs.ext4 -F debian.ext4
$ mkdir -p mnt && sudo mount -o loop debian.ext4 mnt
$ sudo cp -a rootfs/. mnt/
$ sudo umount mnt

$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 2G -nographic \
    -kernel Image \
    -drive file=debian.ext4,if=none,format=raw,id=hd0 \
    -device virtio-blk-device,drive=hd0 \
    -netdev user,id=n0 -device virtio-net-device,netdev=n0 \
    -append "console=ttyAMA0 root=/dev/vda rw rootwait"
```

Method 4: steal a container image

```
# The fastest way to a full userspace you already trust.

$ docker pull --platform linux/arm64 debian:bookworm
$ docker create --platform linux/arm64 --name tmp debian:bookworm
$ mkdir -p rootfs && docker export tmp | tar -C rootfs -xf -
$ docker rm tmp

# Container images have no init and no kernel. Add one:
$ sudo tee rootfs/init >/dev/null <<'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
exec /bin/bash
EOF
$ sudo chmod +x rootfs/init

# Then pack as initramfs, or write into an ext4 image as above.
$ ( cd rootfs && sudo find . -print0 \
    | sudo cpio --null -o --format=newc ) | gzip > initramfs.cpio.gz

# Podman works identically and does not need a daemon:
$ podman pull --arch arm64 alpine
$ podman export $(podman create --arch arm64 alpine) | tar -C rootfs -xf -
```

Making a bootable disk image

```
# A single-partition ext4 image, which is all QEMU needs:
$ truncate -s 2G rootfs.ext4
$ mkfs.ext4 -d rootfs -F rootfs.ext4      # -d populates it
                                          # directly - no
                                          # mounting, no root!

# A partitioned image, closer to a real SD card:
$ truncate -s 4G sd.img
$ sfdisk sd.img <<'EOF'
label: dos
start=2048, size=204800, type=c, bootable
start=206848, type=83
EOF
$ LOOP=$(sudo losetup -Pf --show sd.img)
$ sudo mkfs.vfat -F32 ${LOOP}p1
$ sudo mkfs.ext4 -F ${LOOP}p2
$ sudo mount ${LOOP}p2 /mnt && sudo cp -a rootfs/. /mnt/ \
  && sudo umount /mnt
$ sudo losetup -d $LOOP

# Resize later, without recreating:
$ truncate -s 8G rootfs.ext4
$ e2fsck -f rootfs.ext4 && resize2fs rootfs.ext4

# Inspect an image without mounting - useful in CI:
$ debugfs -R "ls -l /" rootfs.ext4
$ debugfs -R "cat /etc/hostname" rootfs.ext4
```

> **`MKFS.EXT4 -D` IS THE TRICK WORTH REMEMBERING**
> It populates a filesystem image from a directory without mounting anything, which means no `sudo`, no loop devices, and it works inside an unprivileged container. Every CI pipeline that builds an image should use it.

Getting files in and out

```
# 1. virtiofs - best, needs a daemon (virt chapter 15)
$ virtiofsd --socket-path=/tmp/vfs.sock -o source=$PWD/share &
$ qemu-system-aarch64 ... \
    -object memory-backend-file,id=mem,size=2G,mem-path=/dev/shm,share=on \
    -M virt,memory-backend=mem \
    -chardev socket,id=vfs,path=/tmp/vfs.sock \
    -device vhost-user-fs-device,chardev=vfs,tag=share
# guest: mount -t virtiofs share /mnt

# 2. 9p - no daemon, slower, works everywhere
$ qemu-system-aarch64 ... \
    -fsdev local,id=fs0,path=$PWD/share,security_model=none \
    -device virtio-9p-device,fsdev=fs0,mount_tag=share
# guest: mount -t 9p -o trans=virtio,version=9p2000.L share /mnt

# 3. A second disk - simplest, no guest config
$ qemu-img create -f raw scratch.img 1G && mkfs.ext4 -F scratch.img
$ qemu-system-aarch64 ... \
    -drive file=scratch.img,if=none,format=raw,id=s0 \
    -device virtio-blk-device,drive=s0
# guest: mount /dev/vdb /mnt

# 4. SSH over user networking (chapter 8)
$ qemu-system-aarch64 ... \
    -netdev user,id=n0,hostfwd=tcp::2222-:22 \
    -device virtio-net-device,netdev=n0
$ scp -P 2222 file root@localhost:/tmp/
```

How it shows up

Symptom	Cause
`No working init found`	No `/init`, not executable, or the archive was built from the wrong directory
`/init: not found` though the file exists	Dynamically linked binary with no loader — build static, or ship `ld-linux-aarch64.so.1`
`VFS: Unable to mount root fs on unknown-block(0,0)`	Wrong `root=`, or no virtio-blk driver, or missing `rootwait`
Boots to a prompt with no working commands	busybox installed but its symlinks were not created — run `busybox --install -s`
No `/dev` entries	`devtmpfs` not mounted, or `CONFIG_DEVTMPFS` off
Debian image hangs after mounting root	No getty on `ttyAMA0` — `systemctl enable serial-getty@ttyAMA0`
Container-derived rootfs has no `/proc`	Container images omit the pseudo-filesystem mount points; create them
Image full immediately	`truncate` made a sparse file; the filesystem inside is still its original size — `resize2fs`

Symptom	Cause
Buildroot rebuild does not pick up a change	Use `make <pkg>-rebuild`, not `make` alone
9p mount fails with "no such device"	Guest kernel missing `CONFIG_NET_9P_VIRTIO`

Check yourself

- ✓ Give the four rootfs methods and the situation each is best for.
- ✓ Explain why busybox must be statically linked in a hand-built initramfs.
- ✓ Write the minimum `/init` that a kernel will accept, and say what each mount is for.
- ✓ Pack a directory into a `newc` cpio archive and say why that format.
- ✓ Explain what `mkfs.ext4 -d` avoids.
- ✓ Build a Buildroot image for `qemu_aarch64_virt` and boot it from a virtio disk.
- ✓ Explain how `debootstrap --foreign` plus `qemu-user-static` builds a foreign rootfs.
- ✓ Give four ways to move a file between host and guest, ranked.
- ✓ Diagnose `Unable to mount root fs on unknown-block(0,0)`.
- ✓ Add your own kernel module to an initramfs so it loads at boot.

Devices, buses and the command line

QEMU's command line looks arbitrary until you see the object model underneath it. There are three layers — a backend that does real work, a frontend the guest sees, and a bus they plug into — and every confusing option is one of those three. Learn the shape and the syntax stops needing memorisation.

Backend, frontend, bus

```
HOST SIDE                GUEST SIDE
"backend"                "frontend" / device

a file on disk    <---->  virtio-blk-device      on virtio-mmio
a tap interface   <---->  virtio-net-pci         on PCIe
your terminal     <---->  pl011 UART             on the SoC
a unix socket     <---->  virtio-serial port     on virtio
/dev/urandom      <---->  virtio-rng-device

The command line names them SEPARATELY and joins them by id:

  -drive   file=disk.img,if=none,format=raw,id=hd0
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  backend
                                     ^^^^  its name

  -device  virtio-blk-device,drive=hd0
           ^^^^^^^^^^^^^^^^^               frontend
                            ^^^^^^^^^      the link

Once you see that, every option follows:

  -netdev   TYPE,id=NAME,...        network backend
  -device   virtio-net-...,netdev=NAME
  -chardev  TYPE,id=NAME,...        character backend
  -device   ...,chardev=NAME
  -blockdev TYPE,node-name=NAME,... modern block backend
  -object   TYPE,id=NAME,...        memory, rng, crypto,
                                    tls credentials

"if=none" means "do not auto-attach this backend to a
default controller - I will attach it myself with -device".
Forgetting it is the most common mistake, and the symptom is
a disk that appears twice or not at all.
```

Which frontend for which bus

Suffix	Bus	Example	Notes
`-device`	virtio-mmio	`virtio-blk-device`	The 32 mmio slots on `virt`. Simple, no PCIe needed. Best for minimal guests
`-pci`	PCIe	`virtio-blk-pci`	Enumerable, hotpluggable, supports MSI-X and multiqueue properly. Best for full guests
`-ccw`	s390 channel	`virtio-blk-ccw`	s390x only

```
# The same disk, three ways:

# 1. virtio-mmio - no PCIe involved
-drive file=d.img,if=none,format=raw,id=hd0 \
-device virtio-blk-device,drive=hd0

# 2. virtio-pci - needs the PCIe host bridge (virt has one)
-drive file=d.img,if=none,format=raw,id=hd0 \
-device virtio-blk-pci,drive=hd0

# 3. The shorthand, which auto-picks a controller.
#    Convenient, and it hides which bus you got.
-drive file=d.img,format=raw,if=virtio

# Prefer the explicit forms. When something does not appear in
# the guest, the explicit form tells you where to look.

# See what you actually built:
$ qemu-system-aarch64 -M virt ... -monitor stdio
(qemu) info qtree
(qemu) info pci
(qemu) info block
```

Character devices: where output goes

```
# Every serial port, console, monitor and debug channel is a
# chardev. This is the layer people fight with most.

# The simple forms:
-nographic                # serial + monitor multiplexed to
                          # stdio. Ctrl-A c switches, Ctrl-A x
                          # quits.
-serial mon:stdio         # exactly what -nographic does
-serial null              # discard
-serial file:boot.log     # write to a file
-serial tcp::4444,server,nowait     # telnet to it
-serial unix:/tmp/sock,server,nowait
-serial pty               # QEMU prints "char device redirected
                          # to /dev/pts/5" - screen or minicom
                          # to it

# Named chardevs, so several devices can each have their own:
-chardev stdio,id=con0,mux=on \
-chardev file,id=log1,path=secure.log \
-chardev socket,id=dbg,path=/tmp/dbg.sock,server=on,wait=off \
-serial chardev:con0 \
-serial chardev:log1

# TWO UARTs, which the OP-TEE track needs (normal world on one,
# secure world on the other):
-serial mon:stdio -serial file:secure-world.log

# The order matters: the first -serial is ttyAMA0, the second
# is ttyAMA1, and so on. On virt only the first is in the
# generated device tree unless the machine adds more.

# Logging everything to a file while still watching it:
$ qemu-system-aarch64 ... -nographic 2>&1 | tee boot.log

# A virtio-serial channel, for a guest agent:
-device virtio-serial-device \
-chardev socket,path=/tmp/agent.sock,server=on,wait=off,id=ag \
-device virtserialport,chardev=ag,name=org.example.agent
# guest sees /dev/virtio-ports/org.example.agent
```

> **CTRL-A IS THE ESCAPE KEY**
> `Ctrl-A c` toggles between the guest console and the QEMU monitor. `Ctrl-A x` quits. `Ctrl-A h` lists the rest. If your terminal multiplexer already uses `Ctrl-A` (screen does), change QEMU's with `-echr 0x05` to use `Ctrl-E` instead.

A tour of devices worth knowing

```
# Storage
-device virtio-blk-device,drive=hd0            # the fast path
-device virtio-scsi-device -device scsi-hd,drive=hd0
-device nvme,drive=hd0,serial=deadbeef         # a real NVMe
                                               # controller model -
                                               # excellent for
                                               # driver work
-device usb-storage,drive=hd0

# Network
-device virtio-net-device,netdev=n0
-device e1000e,netdev=n0                       # Intel NIC model
-device rtl8139,netdev=n0                      # ancient, useful
                                               # for driver
                                               # teaching

# Console and input
-device virtio-serial-device
-device virtio-keyboard-device
-device virtio-tablet-device
-device usb-kbd -device usb-mouse

# Graphics
-device virtio-gpu-device
-device ramfb                                  # a dumb
                                               # framebuffer -
                                               # simplest possible
-device virtio-gpu-gl-device                   # with virgl

# Entropy - add this to every guest
-object rng-random,filename=/dev/urandom,id=rng0 \
-device virtio-rng-device,rng=rng0

# USB
-device qemu-xhci,id=xhci \
-device usb-host,hostbus=1,hostaddr=5          # pass a real
                                               # USB device
                                               # through!

# The one that matters most for this site:
-device loader,file=payload.bin,addr=0x40100000,force-raw=on
-device loader,addr=0x40100000,cpu-num=0       # set the PC
# Loads arbitrary blobs anywhere. This is how TF-A, OP-TEE and
# bare-metal payloads get placed.

# Ask what a device accepts:
$ qemu-system-aarch64 -device virtio-blk-device,help
$ qemu-system-aarch64 -device help | less
```

Hotplug

```
# PCIe devices can be added and removed while the guest runs.
# virtio-mmio devices cannot - the slots are fixed at startup.
# That alone is a reason to prefer -pci for a long-lived guest.

$ qemu-system-aarch64 -M virt -nographic -monitor stdio ...

(qemu) drive_add 0 if=none,file=extra.img,format=raw,id=hd1
(qemu) device_add virtio-blk-pci,drive=hd1,id=blk1
# guest: a new /dev/vdb appears

(qemu) device_del blk1
# guest: the device is unplugged (the guest must cooperate)

(qemu) info block
(qemu) info pci

# The same through QMP, which is what you would automate
# (chapter 10):
{"execute":"device_add",
 "arguments":{"driver":"virtio-blk-pci","drive":"hd1","id":"blk1"}}

# CPU hotplug on arm64 needs the machine to reserve slots:
$ qemu-system-aarch64 -M virt -smp 2,maxcpus=8 ...
(qemu) device_add cortex-a57-arm-cpu,id=cpu2,core-id=2

# Memory hotplug:
$ qemu-system-aarch64 -m 1G,slots=4,maxmem=8G ...
(qemu) object_add memory-backend-ram,id=m1,size=1G
(qemu) device_add pc-dimm,id=d1,memdev=m1
```

Making command lines survivable

```
# A real command line is 20 lines long. Do not retype it.

# 1. A shell script with a variable per concern:
#!/bin/sh
MACHINE="-M virt,gic-version=3,virtualization=on -cpu max -smp 4 -m 2G"
KERNEL="-kernel Image -append 'console=ttyAMA0 earlycon root=/dev/vda'"
DISK="-drive file=rootfs.ext4,if=none,format=raw,id=hd0
      -device virtio-blk-device,drive=hd0"
NET="-netdev user,id=n0,hostfwd=tcp::2222-:22
     -device virtio-net-device,netdev=n0"
DEBUG="-s -S -d guest_errors,unimp -D qemu.log"
exec qemu-system-aarch64 $MACHINE $KERNEL $DISK $NET -nographic "$@"

# 2. -readconfig, a config file QEMU parses itself:
$ cat vm.cfg
[machine]
  type = "virt"
  gic-version = "3"
[memory]
  size = "2G"
[drive "hd0"]
  file = "rootfs.ext4"
  format = "raw"
  if = "none"
[device "blk0"]
  driver = "virtio-blk-device"
  drive = "hd0"

$ qemu-system-aarch64 -readconfig vm.cfg -kernel Image -nographic

# 3. Dump the configuration QEMU derived, to see defaults:
$ qemu-system-aarch64 -M virt -writeconfig - -m 1G 2>/dev/null

# 4. -nodefaults, when you want NOTHING you did not ask for:
$ qemu-system-aarch64 -M virt -nodefaults \
    -serial mon:stdio -device virtio-blk-device,drive=hd0 ...
# Without it QEMU adds a default VGA, a default NIC, a default
# serial and a default parallel port. -nodefaults plus explicit
# devices is how you get a reproducible machine.
```

> **DEFAULTS CHANGE BETWEEN QEMU VERSIONS**
> A machine type is versioned (`virt-8.2`) precisely so that migration and reproducibility work, but `-M virt` means "the newest", which moves. For anything you want to reproduce in six months — CI, a bug report, a test baseline — pin the versioned machine type and use `-nodefaults` .

How it shows up

Symptom	Cause
`Property 'drive' not found`	Frontend name wrong for the bus — `virtio-blk-device` vs `virtio-blk-pci`
Disk appears twice in the guest	Missing `if=none` , so QEMU auto-attached it as well as your `-device`
`Drive 'hd0' is already in use`	Same, or two `-device` lines referencing one drive
Device configured but absent in the guest	Wrong bus for the machine, or the guest kernel lacks the driver. Check `info qtree` then guest `dmesg`

Symptom	Cause
No console output	chardev routed elsewhere; check the `-serial` ordering and `console=`
Cannot escape QEMU	Terminal multiplexer eating `Ctrl-A`; use `-echr`
Hotplug fails on `virt`	Device is on virtio-mmio, which is not hotpluggable — use `-pci`
Guest gets a NIC you did not ask for	Default devices; add `-nodefaults`
Command line works on one machine, not another	Different QEMU version, different defaults
`-device help` lists a device that then fails	Present in the binary but not valid on this machine type

Check yourself

- ✓ Explain the backend/frontend split and how the two are linked on the command line.
- ✓ Say what `if=none` means and what happens without it.
- ✓ Give the three virtio frontend suffixes and the bus each attaches to.
- ✓ Configure two serial ports, one to the terminal and one to a file.
- ✓ Explain how a guest agent gets a private channel with `virtio-serial`.
- ✓ Name the escape key and how to change it.
- ✓ Hotplug a disk from the monitor and explain why it must be PCIe.
- ✓ Explain what `-nodefaults` is for.
- ✓ Say why a versioned machine type matters for CI.
- ✓ Use `info qtree` to determine whether a device was actually created.

Networking

Four backends, each with a different answer to "who can talk to whom". Picking the wrong one is why guests cannot reach the internet, why you cannot SSH in, and why two guests that should see each other do not. This chapter is the decision table plus the exact commands.

The four backends

Backend	Guest → internet	Host → guest	Guest ↔ guest	Root needed
user (SLIRP)	Yes, NAT	Only via `hostfwd`	No	No
tap	Yes, if you route/NAT	Yes	Yes, via a bridge	Yes (or a pre-made tap)
bridge	Yes, as a peer on the LAN	Yes	Yes	Yes (or `qemu-bridge-helper` setuid)
socket / **vde**	No	No	Yes	No
none	No	No	No	No

```
Decide in this order:

Do I just need the guest to reach the internet and to
SSH in?                          -> USER + hostfwd. Done.
                                    No root, no setup, works
                                    on every host including
                                    macOS and Windows.

Do I need the guest to be a real host on my LAN, with its
own IP, reachable by other machines?
                                 -> BRIDGE.

Do I need several guests on a private network, with the host
as a router or observer?
                                 -> TAP + a host bridge.

Do I need two guests to talk to each other and nothing else,
without root?
                                 -> SOCKET.

Am I testing a driver and do not care about connectivity?
                                 -> USER, or none at all.
```

user: the default that is almost always right

```
# SLIRP implements a TCP/IP stack in QEMU's own process. The
# guest gets 10.0.2.15, a gateway at 10.0.2.2 and DNS at
# 10.0.2.3. No configuration on the host at all.

$ qemu-system-aarch64 -M virt ... \
    -netdev user,id=n0 \
    -device virtio-net-device,netdev=n0

# Guest:
$ udhcpc -i eth0          # or: dhclient eth0
$ ip addr show eth0
    inet 10.0.2.15/24
$ ping -c1 10.0.2.2       # the gateway = QEMU itself
$ curl https://example.com   # works, NATted through the host

# SSH IN - this is the option you will use every day:
$ qemu-system-aarch64 ... \
    -netdev user,id=n0,hostfwd=tcp::2222-:22 \
    -device virtio-net-device,netdev=n0
$ ssh -p 2222 root@localhost

# Several forwards at once:
-netdev user,id=n0,hostfwd=tcp::2222-:22,hostfwd=tcp::8080-:80,\
hostfwd=udp::5353-:53

# Bind a forward to one host interface only:
-netdev user,id=n0,hostfwd=tcp:127.0.0.1:2222-:22

# Serve a directory over TFTP, for netbooting the guest:
-netdev user,id=n0,tftp=/srv/tftp,bootfile=/Image

# Give the guest a share over SMB (needs samba on the host):
-netdev user,id=n0,smb=/home/me/share
# guest: mount -t cifs //10.0.2.4/qemu /mnt

# Change the network:
-netdev user,id=n0,net=192.168.76.0/24,host=192.168.76.9,\
dhcpstart=192.168.76.20

# Restrict the guest to only what you forward:
-netdev user,id=n0,restrict=on,hostfwd=tcp::2222-:22
# No internet, no host access except the forward. Good for
# running something you do not trust.
```

> **WHAT USER NETWORKING CANNOT DO**
>
> ICMP from the guest often does not work (SLIRP needs raw sockets or `ping_group_range`), so `ping` failing does not mean the network is broken — test with `curl` instead. Guests cannot see each other. The host cannot initiate a connection except through `hostfwd`. And throughput is limited by SLIRP's userspace stack: expect hundreds of Mbit/s, not tens of Gbit/s.

tap: a real interface on the host

```
# A tap device is a virtual Ethernet interface the kernel
# exposes as a file descriptor. QEMU writes frames into it; the
# host's network stack sees them arriving on an interface.

# 1. Create a persistent tap owned by you (do this once):
$ sudo ip tuntap add dev tap0 mode tap user $USER
$ sudo ip link set tap0 up
$ sudo ip addr add 192.168.100.1/24 dev tap0

# 2. Use it - no root needed now, because you own tap0:
$ qemu-system-aarch64 -M virt ... \
    -netdev tap,id=n0,ifname=tap0,script=no,downscript=no \
    -device virtio-net-device,netdev=n0,mac=52:54:00:12:34:56

# 3. Guest:
$ ip addr add 192.168.100.2/24 dev eth0
$ ip link set eth0 up
$ ip route add default via 192.168.100.1
$ ping 192.168.100.1              # the host. Works.

# 4. Give the guest internet, by NATting on the host:
$ sudo sysctl -w net.ipv4.ip_forward=1
$ sudo iptables -t nat -A POSTROUTING -s 192.168.100.0/24 \
       -o eth0 -j MASQUERADE
$ sudo iptables -A FORWARD -i tap0 -j ACCEPT
$ sudo iptables -A FORWARD -o tap0 -m state \
       --state RELATED,ESTABLISHED -j ACCEPT

# 5. vhost, for near-native throughput (virt chapter 15):
-netdev tap,id=n0,ifname=tap0,script=no,downscript=no,vhost=on

# ALWAYS set an explicit MAC. QEMU's default is the same for
# every guest, so two guests on one bridge collide, and the
# symptom is intermittent packet loss that looks like a driver
# bug.
```

bridge: the guest as a peer on your LAN

```
# The guest gets an address from your real DHCP server and is
# reachable from any machine on the network.

# 1. Build a bridge and move your physical NIC onto it.
#    WARNING: this drops your connection momentarily. Do not do
#    it over SSH without a console.
$ sudo ip link add br0 type bridge
$ sudo ip link set eth0 master br0
$ sudo ip addr flush dev eth0
$ sudo ip link set br0 up
$ sudo dhclient br0

# NetworkManager equivalent, which survives reboots:
$ nmcli con add type bridge ifname br0 con-name br0
$ nmcli con add type ethernet ifname eth0 master br0
$ nmcli con up br0

# 2. Allow QEMU's helper to use it:
$ echo "allow br0" | sudo tee /etc/qemu/bridge.conf
$ sudo chmod u+s /usr/lib/qemu/qemu-bridge-helper

# 3. Run:
$ qemu-system-aarch64 -M virt ... \
    -netdev bridge,id=n0,br=br0 \
    -device virtio-net-device,netdev=n0,mac=52:54:00:aa:bb:cc

# Guest gets a LAN address by DHCP. Colleagues can reach it.
# So can anything else on that network - treat it accordingly.
```

socket: guests talking to each other, no root

```
# A pure userspace virtual switch. Ideal for testing a network
# protocol between two guests on a laptop.

# Guest A listens:
$ qemu-system-aarch64 ... \
    -netdev socket,id=n0,listen=:1234 \
    -device virtio-net-device,netdev=n0,mac=52:54:00:00:00:01

# Guest B connects:
$ qemu-system-aarch64 ... \
    -netdev socket,id=n0,connect=127.0.0.1:1234 \
    -device virtio-net-device,netdev=n0,mac=52:54:00:00:00:02

# More than two, using multicast as a shared bus:
$ qemu-system-aarch64 ... \
    -netdev socket,id=n0,mcast=230.0.0.1:1234 \
    -device virtio-net-device,netdev=n0,mac=52:54:00:00:00:0N

# Then inside each guest, configure static addresses on the
# same subnet. Nothing leaves the host.

# Two NICs in one guest - one for management, one for the test
# network. This is the standard shape for a router or firewall
# under test:
$ qemu-system-aarch64 ... \
    -netdev user,id=mgmt,hostfwd=tcp::2222-:22 \
    -device virtio-net-device,netdev=mgmt,mac=52:54:00:00:00:10 \
    -netdev socket,id=lab,mcast=230.0.0.1:1234 \
    -device virtio-net-device,netdev=lab,mac=52:54:00:00:00:11
```

Capturing traffic

```
# QEMU can write a pcap of everything on a netdev. No tcpdump
# in the guest, no host interface to find - and it captures
# frames the guest never successfully received, which is
# exactly what you want when debugging a NIC driver.

$ qemu-system-aarch64 ... \
    -netdev user,id=n0 \
    -object filter-dump,id=f0,netdev=n0,file=capture.pcap

$ wireshark capture.pcap
$ tcpdump -r capture.pcap -nn

# Attach a dump filter at runtime, from the monitor:
(qemu) object_add filter-dump,id=f1,netdev=n0,file=/tmp/live.pcap
(qemu) object_del f1

# Other filters worth knowing:
-object filter-buffer,id=f1,netdev=n0,interval=1000
      # delay all traffic by 1 ms - test timeout handling
-object filter-mirror,id=f2,netdev=n0,outdev=chr0
      # mirror to a chardev, for a live analyser
-object filter-redirector,...
      # send traffic somewhere else entirely

# On a tap, the host interface is real, so ordinary tools work:
$ sudo tcpdump -i tap0 -nn
$ sudo tcpdump -i br0 -nn 'ether host 52:54:00:12:34:56'
```

Throughput, and what limits it

```
# Measure honestly: run iperf3 in the guest against the host.
$ iperf3 -s                        # host
$ iperf3 -c 10.0.2.2 -t 20         # guest

# Rough expectations on a modern machine:
#   user (SLIRP), TCG guest        50-200 Mbit/s
#   user (SLIRP), KVM guest        0.5-2 Gbit/s   (SLIRP is the
#                                                  bottleneck)
#   tap, no vhost, KVM             2-5 Gbit/s
#   tap + vhost=on, KVM            10-40 Gbit/s
#   tap + vhost + multiqueue       scales with vCPUs
#   passthrough VF (SR-IOV)        line rate

# The three changes that matter, in order:
#  1. -accel kvm            (if the architectures match)
#  2. vhost=on              (tap only; SLIRP cannot use it)
#  3. multiqueue:
-netdev tap,id=n0,ifname=tap0,script=no,downscript=no,\
vhost=on,queues=4 \
-device virtio-net-pci,netdev=n0,mq=on,vectors=10
# vectors = 2*queues + 2. Too few silently reduces the queue
# count.

# Guest side, confirm what you got:
$ ethtool -l eth0
$ ethtool -k eth0 | grep -E 'tso|gso|gro|checksum'
```

How it shows up

Symptom	Cause
Guest has no interface at all	No `-device` for the netdev, or the guest kernel lacks `CONFIG_VIRTIO_NET`
Interface exists, no address	No DHCP client ran; with user networking run `udhcpc` / `dhclient`
`ping` fails but `curl` works	SLIRP and ICMP. Not a problem
Cannot SSH in	No `hostfwd`, or sshd not running, or the guest firewall
Two guests cannot see each other	Both on `user`, which is isolated by design. Use socket, tap or bridge
Intermittent loss with two guests on a bridge	Both using QEMU's default MAC — set them explicitly
`could not open /dev/net/tun`	Tap not created, not owned by you, or no `CAP_NET_ADMIN`
`bridge helper failed`	Bridge not listed in `/etc/qemu/bridge.conf`, or the helper is not setuid
Host loses network when creating a bridge	Expected while the NIC moves; reconfigure `br0`
Throughput far below expectation	SLIRP, or `vhost=on` missing, or TCG
Multiqueue configured, one queue used	`vectors` too small

Check yourself

- ✓ Give the four backends and fill in the connectivity table from memory.
- ✓ Write the `user` networking options that let you SSH into a guest on port 2222.
- ✓ Explain what `restrict=on` does and when you would want it.
- ✓ Create a tap device you own and attach a guest to it without root.
- ✓ Give the host commands that NAT a tap network to the internet.
- ✓ Explain why every guest needs an explicit MAC address on a shared network.
- ✓ Connect two guests with `socket` and say why no root is needed.
- ✓ Capture guest traffic to a pcap without running anything in the guest.
- ✓ List the three changes that most improve throughput, in order.
- ✓ Diagnose "the guest has an interface but no connectivity" in three steps.

Storage and block devices

QEMU's block layer is a stack of filters — format drivers, protocol drivers, caches, throttling, snapshots — that you compose. The command line hides this behind `-drive`, which is convenient until you need a snapshot chain or an overlay, at which point knowing the real structure saves you.

The block stack

```
guest driver  (virtio-blk, nvme, ahci ...)
     |
     v
+-------------------------------------------+
|  FRONTEND     -device virtio-blk-device   |
+-------------------------------------------+
     |
     v
+-------------------------------------------+
|  FILTERS (optional, stackable)            |
|    throttle    - IOPS/bandwidth limits    |
|    copy-on-read                           |
|    blkverify, blkdebug - fault injection  |
+-------------------------------------------+
     |
     v
+-------------------------------------------+
|  FORMAT driver                            |
|    raw    - nothing, byte for byte        |
|    qcow2  - snapshots, compression, backing |
|             files, sparse allocation      |
|    vmdk / vhdx / vdi / vpc - other people's |
|    luks   - encryption                    |
+-------------------------------------------+
     |
     v
+-------------------------------------------+
|  PROTOCOL driver                          |
|    file   - a local file or block device  |
|    nbd    - network block device          |
|    iscsi / rbd / ssh / http               |
+-------------------------------------------+
     |
     v
host storage

-drive collapses all of this into one option. -blockdev
exposes each layer as a named node you can wire together,
which is what libvirt and any serious automation uses.
```

raw versus qcow2

	raw	qcow2
Overhead	None	~5–10% on random writes
Sparse	Only if the host filesystem is	Always
Snapshots	No	Internal and external

	raw	qcow2
Backing files	No	Yes — the basis of overlays
Compression	No	Yes, read-only after write
Encryption	Via the `luks` layer	Built in (LUKS)
Mount on the host	`mount -o loop`	Needs `qemu-nbd`
Use it for	Rootfs images you rebuild often; anything you want to inspect with ordinary tools	Golden images, overlays, snapshot workflows

```
$ qemu-img create -f qcow2 disk.qcow2 20G
Formatting 'disk.qcow2', fmt=qcow2 size=21474836480 ...

$ qemu-img create -f raw disk.raw 20G
$ ls -lh disk.raw          # 20G apparent
$ du -h  disk.raw          # 0 actual - sparse, on ext4/xfs/btrfs

$ qemu-img info disk.qcow2
image: disk.qcow2
file format: qcow2
virtual size: 20 GiB (21474836480 bytes)
disk size: 196 KiB
cluster_size: 65536

$ qemu-img convert -p -O raw disk.qcow2 disk.raw
$ qemu-img convert -p -O qcow2 -c disk.raw compressed.qcow2

$ qemu-img resize disk.qcow2 +10G
# then, in the guest: growpart /dev/vda 1 && resize2fs /dev/vda1

$ qemu-img check disk.qcow2
$ qemu-img map disk.qcow2         # which ranges are allocated
```

Overlays: the workflow that saves the most time

```
# Build a golden image once. Never boot it directly again.
# Every experiment gets a disposable overlay on top.

$ qemu-img create -f qcow2 -b golden.qcow2 -F qcow2 work.qcow2
Formatting 'work.qcow2', fmt=qcow2 backing_file=golden.qcow2 ...

$ ls -lh
-rw-r--r-- 2.1G golden.qcow2
-rw-r--r--  196K work.qcow2        # writes go here only

$ qemu-system-aarch64 ... -drive file=work.qcow2,if=none,id=hd0 \
    -device virtio-blk-device,drive=hd0

# Broke it? Throw the overlay away, make a new one. Two seconds.
$ rm work.qcow2 && qemu-img create -f qcow2 -b golden.qcow2 \
    -F qcow2 work.qcow2

# Even quicker, for a single throwaway run - no overlay file at
# all, all writes go to a temporary file QEMU deletes on exit:
$ qemu-system-aarch64 ... -snapshot -drive file=golden.qcow2,...

# Commit an overlay back into its backing file, when the change
# was one you wanted to keep:
$ qemu-img commit work.qcow2

# Chains are allowed, and useful:
#   base.qcow2  <- installed.qcow2  <- configured.qcow2  <- run.qcow2
$ qemu-img info --backing-chain run.qcow2

# Flatten a chain into one image:
$ qemu-img convert -O qcow2 run.qcow2 flattened.qcow2
```

> **THE SINGLE HABIT WORTH FORMING**
>
> Keep one read-only golden image per project and always boot an overlay. Experiments become free, "it worked yesterday" becomes recoverable, and you stop being afraid of `rm -rf` inside a guest. The whole trick is one `qemu-img create -b` command.

Cache modes: a correctness setting

```
   Three caches sit between the guest and the platter:

     guest page cache -- guest's own, always present
     HOST page cache  -- controlled by cache=
     disk write cache -- controlled by whether flushes reach it

   cache=writeback   (default)
       host page cache USED. Guest flushes are honoured
       (fdatasync). Fast, and SAFE - provided the guest issues
       flushes and your device model implements them.

   cache=none
       O_DIRECT: bypass the host page cache. The guest's own
       cache is the only one. Predictable, avoids double
       caching, required for some migration setups. Needs
       aligned I/O, so it fails on some filesystems (tmpfs).

   cache=writethrough
       every write goes to stable storage before completing.
       Very slow. Safe even for a guest that never flushes.

   cache=unsafe
       FLUSHES ARE IGNORED. Fastest by far. A host crash loses
       arbitrary amounts of guest data, including data the
       guest was told was durable.

   cache=directsync
       O_DIRECT plus writethrough.

   Choose:
     building/throwaway images, CI        -> unsafe   (10x on
                                             installs)
     day-to-day development               -> writeback
     anything holding work you care about -> writeback, and
                                             verify flushes
     production, shared storage           -> none
```

```
# The full modern form. aio=native needs cache=none or
# directsync; aio=io_uring is better where available.
-drive file=disk.qcow2,if=none,id=hd0,format=qcow2,\
cache=none,aio=io_uring,discard=unmap,detect-zeroes=unmap \
-device virtio-blk-device,drive=hd0

#   discard=unmap        guest TRIM shrinks the host image
#   detect-zeroes=unmap  writes of zeroes become holes
#   Together these keep a long-lived image from growing without
#   bound. Add them to every qcow2 you keep.

# Throttling, to simulate slow storage - genuinely useful for
# finding timeout bugs in a driver:
-drive file=disk.img,if=none,id=hd0,\
throttling.iops-total=100,throttling.bps-total=1048576

# Fault injection, to test error paths:
-drive file=blkdebug::disk.img,if=none,id=hd0
# with a config file that fails the Nth read, or corrupts a
# write, or returns EIO after a delay.
```

Reading and writing images from the host

```
# raw, single filesystem - simplest:
$ sudo mount -o loop rootfs.ext4 /mnt

# raw, partitioned:
$ LOOP=$(sudo losetup -Pf --show sd.img)
$ sudo mount ${LOOP}p2 /mnt
$ ... ; sudo umount /mnt ; sudo losetup -d $LOOP

# qcow2 - via NBD:
$ sudo modprobe nbd max_part=8
$ sudo qemu-nbd --connect=/dev/nbd0 disk.qcow2
$ sudo fdisk -l /dev/nbd0
$ sudo mount /dev/nbd0p1 /mnt
$ ... ; sudo umount /mnt ; sudo qemu-nbd --disconnect /dev/nbd0

# WITHOUT ROOT, and without mounting - libguestfs. This is the
# right tool for scripting:
$ sudo apt install libguestfs-tools
$ virt-ls -a disk.qcow2 /etc
$ virt-cat -a disk.qcow2 /etc/hostname
$ virt-copy-out -a disk.qcow2 /var/log/messages ./
$ virt-copy-in -a disk.qcow2 mydriver.ko /lib/modules/
$ virt-customize -a disk.qcow2 \
      --root-password password:secret \
      --install openssh-server \
      --run-command 'systemctl enable serial-getty@ttyAMA0'
$ guestfish -a disk.qcow2 -i        # an interactive shell

# NEVER mount an image read-write while a guest is using it.
# Two writers on one filesystem corrupts it immediately and
# the corruption may not show for hours.
```

Snapshots

```
# INTERNAL snapshots - stored inside a qcow2, include RAM if
# taken from the monitor. This is save-state/load-state.

(qemu) savevm before-experiment
(qemu) info snapshots
ID   TAG                 VM SIZE  DATE                VM CLOCK
1    before-experiment     512MiB 2026-08-22 10:14   00:02:31
(qemu) loadvm before-experiment      # instant time travel
(qemu) delvm before-experiment

# Offline:
$ qemu-img snapshot -l disk.qcow2
$ qemu-img snapshot -c mytag disk.qcow2
$ qemu-img snapshot -a mytag disk.qcow2
$ qemu-img snapshot -d mytag disk.qcow2

# EXTERNAL snapshots - a new overlay, taken live via QMP. This
# is how backups are done without pausing the guest:
{"execute":"blockdev-snapshot-sync",
 "arguments":{"device":"hd0","snapshot-file":"snap1.qcow2",
              "format":"qcow2"}}

# savevm/loadvm is the most underused debugging tool in QEMU.
# Reproducing a bug that takes ten minutes of setup? Snapshot
# just before it, then loadvm as many times as you need. It
# turns a ten-minute iteration into a one-second one.
# (It requires all devices to support migration state; a
# passed-through device will refuse.)
```

How it shows up

Symptom	Cause
Guest filesystem corrupt after a host crash	`cache=unsafe`, or a device model that ignores `FLUSH`
Image grows until the host disk fills	No `discard=unmap`; deleted guest data still occupies clusters
`Could not open backing file`	Overlay moved away from its backing file — the path is stored inside the overlay. Use `qemu-img rebase`
Guest sees a smaller disk than the file	Image resized but the guest partition and filesystem were not
`cache=none` fails to open	Filesystem does not support O_DIRECT (tmpfs, some network filesystems)
`aio=native` rejected	Requires `cache=none` or `cache=directsync`
Both host and guest wrote to an image; it is wrecked	Two writers. There is no recovery; restore from the golden image
`savevm` refuses	A device without migration support — commonly a passed-through device or a non-migratable model
Guest boots from the wrong disk	Ordering; set `bootindex=` on the device
Terrible install performance	Use `cache=unsafe` for the install, then switch back

Check yourself

✓ Name the four layers of the block stack and give an example driver at each.

- ✓ Give three things qcow2 does that raw cannot, and one thing raw does better.
- ✓ Create an overlay on a golden image and explain why this should be your default workflow.
- ✓ Explain what `-snapshot` does without creating a file.
- ✓ Describe each cache mode and say which is a correctness hazard.
- ✓ Explain what `discard=unmap` and `detect-zeroes=unmap` prevent.
- ✓ Mount a qcow2 image on the host, and give a way to read one without mounting.
- ✓ State the rule about writing to an image a guest is using.
- ✓ Use `savevm`/`loadvm` to shorten a debugging loop, and name one thing that blocks it.
- ✓ Explain what `qemu-img rebase` fixes.

The monitor, QMP and snapshots

QEMU has a control channel into a running machine that can inspect memory, dump registers, add and remove devices, and freeze the whole system into a file. Two interfaces expose it: one for humans and one for programs. The human one is the fastest debugging tool in this track.

HMP and QMP

```
HMP - Human Monitor Protocol
    terse, text, tab-completion, stable enough for daily use
    but NOT a stable API.
    Reach it with:  -monitor stdio
                    Ctrl-A c   (with -nographic)
                    -monitor telnet::4444,server,nowait

QMP - QEMU Machine Protocol
    JSON, versioned, introspectable, the actual API. libvirt
    speaks this. Scripts should too.
    Reach it with:  -qmp unix:/tmp/qmp.sock,server,nowait
                    -qmp tcp::4444,server,nowait
                    -qmp stdio

You can have both at once, and usually should:

  -monitor unix:/tmp/hmp.sock,server,nowait \
  -qmp     unix:/tmp/qmp.sock,server,nowait

And you can run an HMP command through QMP when a QMP
equivalent does not exist:

  {"execute":"human-monitor-command",
   "arguments":{"command-line":"info mtree"}}
```

The HMP commands that matter

Command	Shows
`info registers`	Every CPU register, including system registers on arm64. The first thing to run when a guest hangs
`info registers -a`	All CPUs
`info mtree`	The complete memory map — what is modelled where
`info qtree`	The device tree QEMU built, with every property
`info block`	Drives, their backing files, cache modes
`info network`	netdevs and the devices attached
`info pci`	The PCIe topology
`info irq` / `info pic`	Interrupt counts and controller state
`info cpus`	vCPU list, thread ids, halted state
`info roms`	Everything loaded and where
`info snapshots`	Saved VM states

Command	Shows
`x/<fmt> <addr>`	Examine *virtual* memory
`xp/<fmt> <addr>`	Examine *physical* memory
`gva2gpa` / `gpa2hva`	Translate an address through the stages
`stop` / `cont`	Freeze and resume the machine
`system_reset` / `system_powerdown`	Reset; send an ACPI power button
`savevm` / `loadvm` / `delvm`	Snapshot the whole machine
`migrate`	Move or dump the machine state
`dump-guest-memory`	Write an ELF core of guest RAM for crash analysis
`sendkey`	Inject keystrokes, e.g. `sendkey ctrl-alt-f2`
`nmi`	Poke the guest into its panic handler

```
(qemu) info registers
 PC=ffff800008a41234 X00=0000000000000000 X01=ffff0000c1234000
 X02=0000000000001000 ...
 PSTATE=204003c5 --C- EL1h
 SP_EL0=ffff0000c0a1bf00  SP_EL1=ffff800008003e70
 ELR_EL1=ffff800008a41230 ESR_EL1=96000047
 SCTLR_EL1=0000000034d4d91d  TTBR0_EL1=00000000c1122000
 TTBR1_EL1=0000000041c50000  VBAR_EL1=ffff800008010000

# A hung guest, decoded in one command: PSTATE says EL1h,
# ESR_EL1 says data abort (EC=0x25), and ELR_EL1 is the
# faulting instruction. Feed that address to addr2line.

(qemu) x/8i $pc              # disassemble at the PC
(qemu) x/16xg 0xffff0000c1234000
(qemu) xp/32xb 0x09000000    # the UART's registers, physically
(qemu) gva2gpa 0xffff800008a41234
gpa: 0x41234234

# Dump a region of guest memory to a file:
(qemu) pmemsave 0x40000000 0x100000 /tmp/first-mb.bin
(qemu) memsave 0xffff800008000000 0x1000 /tmp/kernel-page.bin

# A full guest core, readable by crash(8) or gdb:
(qemu) dump-guest-memory /tmp/guest.core
$ crash vmlinux /tmp/guest.core
```

> **`INFO REGISTERS` ON A HUNG GUEST**
>
> A guest that stops responding is not a black box. Press `Ctrl-A c`, type `info registers`, and you have the PC, the exception syndrome and the exception link register — which is more than a kernel oops would have given you, and you did not need the guest to be healthy enough to print it.

QMP, properly

```
$ qemu-system-aarch64 -M virt ... \
    -qmp unix:/tmp/qmp.sock,server=on,wait=off

# Talk to it. Every session starts with a greeting, then you
# must leave capabilities negotiation before anything works:
$ socat - UNIX-CONNECT:/tmp/qmp.sock
{"QMP": {"version": {"qemu": {"major": 8, "minor": 2}}, ...}}
{"execute":"qmp_capabilities"}
{"return": {}}
{"execute":"query-status"}
{"return": {"status": "running", "singlestep": false, "running": true}}

# From Python, which is how you should automate it. QEMU ships
# the client library:
$ pip install qemu.qmp
```

```
#!/usr/bin/env python3
"""Wait for a guest to boot, snapshot it, and pull stats."""
import asyncio
from qemu.qmp import QMPClient

async def main():
    qmp = QMPClient('vm')
    await qmp.connect('/tmp/qmp.sock')

    # What can this QEMU do?
    cmds = await qmp.execute('query-commands')
    print(len(cmds), 'commands available')

    # Machine state
    print(await qmp.execute('query-status'))
    print(await qmp.execute('query-cpus-fast'))
    print(await qmp.execute('query-block'))

    # Wait for the guest to signal readiness. Events are the
    # right way to synchronise - never sleep and hope.
    await qmp.execute('cont')
    async for event in qmp.events:
        print('EVENT', event['event'])
        if event['event'] == 'RESET':
            break

    # Snapshot the whole machine
    await qmp.execute('human-monitor-command',
                      {'command-line': 'savevm booted'})

    # Add a disk while it runs
    await qmp.execute('blockdev-add', {
        'node-name': 'extra',
        'driver': 'qcow2',
        'file': {'driver': 'file', 'filename': 'extra.qcow2'},
    })
    await qmp.execute('device_add', {
        'driver': 'virtio-blk-pci', 'drive': 'extra', 'id': 'blk1',
    })

    await qmp.disconnect()

asyncio.run(main())
```

Events: the synchronisation primitive

```
# QMP pushes events asynchronously. Waiting for one is how a
# test harness knows when to act.

{"event": "RESET",          "data": {"guest": true}, ...}
{"event": "SHUTDOWN",       "data": {"guest": true, "reason": "guest-shutdown"}}
{"event": "STOP"}
{"event": "RESUME"}
{"event": "POWERDOWN"}
{"event": "GUEST_PANICKED",  "data": {"action": "pause"}}
{"event": "DEVICE_TRAY_MOVED"}
{"event": "BLOCK_IO_ERROR",  "data": {"device": "hd0", "operation": "write"}}
{"event": "MIGRATION",       "data": {"status": "completed"}}
{"event": "VSERPORT_CHANGE"}

# A CI harness in eight lines of shell + socat:
$ (echo '{"execute":"qmp_capabilities"}'; cat) \
  | socat - UNIX-CONNECT:/tmp/qmp.sock \
  | while read -r line; do
        case "$line" in
          *GUEST_PANICKED*) echo "FAIL: guest panicked"; exit 1;;
          *SHUTDOWN*)       echo "PASS: clean shutdown";  exit 0;;
        esac
    done

# Make a panic detectable rather than a hang:
$ qemu-system-aarch64 ... -no-reboot \
    -device pvpanic-pci
# The guest's panic handler writes to the pvpanic device, QEMU
# emits GUEST_PANICKED, and your harness fails fast instead of
# waiting for a timeout.
```

Migration, and its offline uses

```
# Migration serialises the entire machine. That is useful even
# when you are not moving anything anywhere.

# 1. Save a machine to a file and restore it later:
(qemu) migrate "exec:gzip -c > state.gz"
# start a NEW qemu with identical arguments plus -incoming:
$ qemu-system-aarch64 ...same args... \
    -incoming "exec:gzip -dc state.gz"

# 2. Move a running guest to another machine:
#    destination:
$ qemu-system-aarch64 ...same args... -incoming tcp:0:4444
#    source:
(qemu) migrate -d tcp:dest-host:4444
(qemu) info migrate
capabilities: xbzrle: off  postcopy-ram: off
Migration status: completed
total time: 4213 ms
downtime: 87 ms

# 3. Requirements people trip over:
#    - IDENTICAL command line on both ends, including the
#      versioned machine type (virt-8.2, not virt)
#    - the same QEMU version, or a compatible one
#    - shared or replicated storage, unless you use
#      --copy-storage-all
#    - NO passed-through devices (virt chapter 16)
#    - the same CPU model; -cpu host will refuse across
#      different silicon

# 4. Tuning, when it will not converge:
(qemu) migrate_set_parameter max-bandwidth 1G
(qemu) migrate_set_parameter downtime-limit 300
(qemu) migrate_set_capability auto-converge on   # throttle the
                                                 # guest until
                                                 # it converges
(qemu) migrate_set_capability postcopy-ram on    # switch over
                                                 # first, fault
                                                 # pages after
```

> **SNAPSHOT BEFORE THE HARD PART**
> Reproducing a bug that needs ten minutes of setup is the most wasteful thing in embedded debugging. `savevm` just before the interesting moment turns every subsequent attempt into `loadvm` plus one second. Combine it with `-icount` (chapter 2) and the replay is deterministic too.

How it shows up

Symptom	Cause
QMP commands all return an error	`qmp_capabilities` not sent first
Cannot reach the monitor with `-nographic`	Use `Ctrl-A c`; the monitor is multiplexed onto the same terminal
`savevm` fails: "state blocked by non-migratable device"	A passed-through or non-migratable device is present

Symptom	Cause
`loadvm` restores but the guest immediately breaks	Different command line — the device model must match exactly
Migration never completes	Guest dirties memory faster than the link copies it; enable `auto-converge` or postcopy
Migration completes, guest is broken	Unversioned machine type, so the two ends built different machines
A guest panic looks like a hang in CI	No `pvpanic` device and no `-no-reboot`
`x` shows garbage	Used `x` (virtual) where you wanted `xp` (physical), or the MMU is off
HMP output changed and broke a script	HMP is not an API. Use QMP

Check yourself

- ✓ Distinguish HMP from QMP and say which a script should use.
- ✓ Show how to run an HMP command through QMP.
- ✓ Given a hung guest, use the monitor to obtain the PC, the fault syndrome and the exception level.
- ✓ Explain the difference between `x` and `xp`, and when each is meaningful.
- ✓ Translate a guest virtual address to a guest physical one from the monitor.
- ✓ Write the QMP handshake and one query command.
- ✓ Name four QMP events and say what a test harness would do with each.
- ✓ Explain what `pvpanic` plus `-no-reboot` buys a CI pipeline.
- ✓ List four requirements for a successful migration.
- ✓ Describe how `savevm` shortens a debugging loop and one thing that prevents it.

Debugging through QEMU with GDB

QEMU has a built-in GDB stub, which means you can stop a machine before its first instruction, single-step through a bootloader, set a watchpoint on a hardware register, and inspect an EL3 exception handler — with no JTAG probe, no cost, and none of the ways a real debug port can interfere with the system it is debugging.

The two flags

```
  -s        shorthand for -gdb tcp::1234
  -S        do NOT start the CPU; wait for the debugger

$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 1G -nographic \
    -kernel Image -append "console=ttyAMA0 nokaslr" \
    -s -S

# In another terminal:
$ gdb-multiarch vmlinux
(gdb) target remote :1234
Remote debugging using :1234
0x0000000040000000 in ?? ()

# You are now stopped at the very first instruction the machine
# will execute, before any firmware, before the kernel header,
# before anything. No real debug hardware gives you this.

(gdb) break start_kernel
(gdb) continue
Breakpoint 1, start_kernel () at init/main.c:936

# Other ports/transports:
-gdb tcp::4321                   # different port
-gdb unix:/tmp/gdb.sock,server=on
-gdb stdio                       # pipe it

# The right GDB:
#   gdb-multiarch                (Debian/Ubuntu)
#   aarch64-linux-gnu-gdb        (from the cross toolchain)
#   aarch64-none-elf-gdb         (bare metal)
#   lldb also speaks the protocol: (lldb) gdb-remote 1234
#
# Plain x86 gdb will connect and then produce nonsense, because
# it does not know the target architecture. If registers look
# absurd, this is why. Force it:
(gdb) set architecture aarch64
```

Debugging a Linux kernel

```
# Build with symbols and without KASLR (chapter 5), then:

$ gdb-multiarch vmlinux
(gdb) target remote :1234

# Linux ships GDB scripts. They are excellent and nobody uses
# them.
(gdb) source /path/to/linux/vmlinux-gdb.py
(gdb) lx-symbols               # load module symbols too
(gdb) lx-dmesg                 # the kernel ring buffer, even
                               # if the console is broken
(gdb) lx-ps                    # every task
(gdb) lx-lsmod
(gdb) lx-device-tree
(gdb) lx-mounts

# You may need:
$ echo "add-auto-load-safe-path /path/to/linux" >> ~/.gdbinit

# Breakpoints in a MODULE. The module is not loaded yet when
# you connect, so:
(gdb) lx-symbols /path/to/module/build/dir
# after insmod in the guest, run lx-symbols again - it finds
# the load address from the kernel's module list and relocates.

(gdb) break my_driver_probe
(gdb) continue
# in the guest: insmod mydriver.ko
Breakpoint 2, my_driver_probe (pdev=0xffff0000c1234000) ...
(gdb) bt
(gdb) p *pdev
(gdb) p pdev->dev.of_node->name

# Per-CPU variables and the current task:
(gdb) p $lx_current().comm
(gdb) p $lx_per_cpu("runqueues", 1)

# Physical memory, when the MMU is off or you do not trust it:
(gdb) monitor xp/16xg 0x40000000
# "monitor" passes the command to QEMU's HMP - so every command
# from chapter 10 is available from inside GDB.
```

> **`MONITOR` IS THE BRIDGE**
>
> Anything you can do in the QEMU monitor, you can do from the GDB prompt by prefixing it with `monitor`. `monitor info registers`, `monitor info mtree`, `monitor savevm x`. That combination — a source-level debugger plus full machine introspection — is strictly more than a JTAG probe offers.

Exception levels and the multi-image problem

```
A full arm64 boot has four separate binaries at three
exception levels, and GDB only knows about symbols you have
told it about.

  bl1.bin / bl2 / bl31.elf     EL3   TF-A
  tee.elf / tee-pager.elf      S-EL1 OP-TEE
  u-boot                       EL2/1 bootloader
  vmlinux                      EL1   Linux

Load them all, at their link addresses:

  (gdb) file bl31.elf
  (gdb) add-symbol-file tee.elf   0x0e100000
  (gdb) add-symbol-file u-boot    0x60000000
  (gdb) add-symbol-file vmlinux

Now a backtrace resolves whichever image the PC is in.

Which EL are we in right now?

  (gdb) p $cpsr
  (gdb) monitor info registers | grep PSTATE
  PSTATE=204003c9 --C- EL3h
                      ^^^^ there it is

Breakpoints across an EL switch: QEMU's stub uses SOFTWARE
breakpoints by default, which need writable memory. Code in
ROM or in a region the current stage cannot write needs a
HARDWARE breakpoint:

  (gdb) hbreak *0x0e100000
  (gdb) watch *(int *)0x09000000       # software watchpoint
  (gdb) rwatch *(int *)0x09000018      # break on READ of a
                                       # device register -
                                       # this is the killer
                                       # feature for driver
                                       # work
```

Bare-metal and early boot

```
# A payload with no OS. This is the OP-TEE and TF-A workflow.

$ aarch64-none-elf-gcc -g -O0 -nostdlib -Wl,-Ttext=0x40100000 \
      -o payload.elf start.S main.c

$ qemu-system-aarch64 -M virt -cpu max -nographic \
    -kernel payload.elf -s -S

$ aarch64-none-elf-gdb payload.elf
(gdb) target remote :1234
(gdb) break _start
(gdb) continue
(gdb) layout asm            # a disassembly window
(gdb) layout regs           # registers alongside
(gdb) si                    # step one instruction
(gdb) info registers sctlr_el1 ttbr0_el1 vbar_el1

# System registers ARE exposed by QEMU's stub on aarch64:
(gdb) p $sctlr_el1
(gdb) p/x $esr_el1
(gdb) p $currentel

# Stepping through MMU enable - the moment that breaks most
# bare-metal code:
(gdb) break *enable_mmu
(gdb) si
(gdb) p/x $sctlr_el1        # watch bit 0 flip
(gdb) x/4i $pc              # are we still executing sane code?

# If the CPU vanishes after enabling the MMU, the identity
# mapping for the code itself is missing. GDB shows you exactly
# which instruction it died on, which a hardware debugger often
# cannot because it loses the connection at the same moment.
```

Multiple CPUs

```
# Each vCPU is a GDB "thread".

(gdb) info threads
  Id   Target Id      Frame
* 1    Thread 1 (CPU#0 [running])  cpu_do_idle () at ...
  2    Thread 2 (CPU#1 [halted ])  0x0000000000000000 in ?? ()

(gdb) thread 2
(gdb) bt

# By default GDB stops ALL CPUs when one hits a breakpoint,
# which is what you want. Non-stop mode lets the others run:
(gdb) set non-stop on          # before connecting

# Break only on a specific CPU:
(gdb) break my_func if $_thread == 2

# A race between two CPUs is best caught with -icount (chapter
# 2), which makes the interleaving deterministic and therefore
# repeatable:
$ qemu-system-aarch64 ... -icount shift=5,align=off,sleep=off -s -S
```

Techniques worth stealing

```
# 1. Script the whole session. Stop retyping.
$ cat > debug.gdb <<'EOF'
target remote :1234
source /work/linux/vmlinux-gdb.py
lx-symbols
break panic
break die
break my_driver_probe
commands 3
  bt
  p *pdev
  continue
end
continue
EOF
$ gdb-multiarch vmlinux -x debug.gdb

# 2. Catch a fault before it becomes an oops:
(gdb) break do_mem_abort
(gdb) break __do_kernel_fault

# 3. Watch a hardware register be touched, and by whom:
(gdb) awatch *(volatile unsigned int *)0x09000018
Hardware access (read/write) watchpoint 1
Value = 0x90
0xffff800008a41234 in pl011_console_write ()
# THIS is how you find the code that is banging on a register
# it should not.

# 4. Change guest state to test an error path you cannot
#    otherwise reach:
(gdb) p $x0 = -12            # make an allocation "fail"
(gdb) return -ENOMEM
(gdb) set var some_flag = 1

# 5. Combine with savevm: snapshot at the interesting point,
#    then loadvm and try a different intervention each time.
(gdb) monitor savevm probe-entry
(gdb) monitor loadvm probe-entry

# 6. Reverse debugging, with record/replay:
$ qemu-system-aarch64 ... -icount shift=auto,rr=record,rrfile=rr.bin -s -S
$ qemu-system-aarch64 ... -icount shift=auto,rr=replay,rrfile=rr.bin -s -S
(gdb) reverse-continue
(gdb) reverse-step
# Stepping BACKWARDS from a crash to its cause. It has
# limitations (no external input during replay) but when it
# works it is unmatched.
```

KASLR WILL WASTE YOUR AFTERNOON

If breakpoints do not hit and symbols point at nonsense, the kernel relocated itself. Add `nokaslr` to `-append` or turn off `CONFIG_RANDOMIZE_BASE`. The same applies to modules — always re-run `lx-symbols` after `insmod`, because until then GDB has no idea where the module landed.

How it shows up

Symptom	Cause
GDB connects, registers are nonsense	Wrong GDB architecture — use `gdb-multiarch` or `set architecture aarch64`
Breakpoints never hit	KASLR, or symbols from a different build than the running image
Breakpoint in a module never hits	`lx-symbols` not re-run after `insmod`
`Cannot insert breakpoint`	Target memory is read-only (ROM, or another EL's region) — use `hbreak`
Guest runs away immediately	Forgot `-S`
Connection refused	QEMU not started, wrong port, or `-s` missing
Everything freezes when you step	Normal — the whole machine is stopped, including its timers
Guest network connections time out while debugging	Also normal; the guest's clock stopped but the world's did not
Symbols resolve for the kernel but not for firmware	Need `add-symbol-file` at the firmware's link address
Race disappears under GDB	Use `-icount` for determinism, or you are observing a genuine timing dependency

Check yourself

- ✓ Explain what `-s` and `-S` do, separately.
- ✓ Name the correct GDB binary for an aarch64 target and what happens if you use the wrong one.
- ✓ Load the Linux GDB scripts and name four `lx-` commands.
- ✓ Explain how to get a breakpoint into a kernel module.
- ✓ Explain what `monitor` does at the GDB prompt and give two uses.
- ✓ Load symbols for four images at three exception levels and determine the current EL.
- ✓ Say when you need `hbreak` instead of `break`.
- ✓ Set a watchpoint that fires when a device register is *read*.
- ✓ Describe how vCPUs appear to GDB and how to inspect a specific one.
- ✓ Explain how to make a race reproducible under the debugger.

Tracing, logging and TCG plugins

Three separate observation systems ship in QEMU and they answer different questions: `-d` tells you what the guest did, `--trace` tells you what QEMU did, and plugins let you write your own instrumentation over every instruction and memory access. Together they give you visibility no real board can match.

Which one answers which question

Question	Tool
What instructions did the guest execute?	`-d in_asm,exec`
What exceptions did it take?	`-d int`
Why did that access fault?	`-d mmu,guest_errors`
Is the guest touching something I have not modelled?	`-d unimp,guest_errors`
What did my device model receive?	`--trace 'mydev_*'`
Which virtqueue operations happened?	`--trace 'virtio_*'`
How many times is that register read?	a TCG plugin
What is the guest's cache miss rate?	`libcache.so` plugin
Which basic blocks are hot?	`libhotblocks.so` plugin

-d: guest-side logging

```
# Always write to a file. -d output to a terminal is unusable.
$ qemu-system-aarch64 -M virt -nographic -kernel Image \
    -d guest_errors,unimp,int -D qemu.log

# The four to enable by default during bring-up. They are cheap
# and they catch real problems:
#   guest_errors  the guest did something invalid
#   unimp         you hit an unimplemented feature or register
#   int           every exception and interrupt taken
#   mmu           translation faults

$ qemu-system-aarch64 -M virt -nographic -kernel Image \
    -d int -D int.log
$ head -20 int.log
Taking exception 5 [IRQ]
...from EL1 to EL1
...with ESR 0x0/0x0
...with ELR 0xffff800008a41234
...to EL1 PC 0xffff800008010480 PSTATE 0x3c5

Taking exception 3 [Prefetch Abort]
...from EL1 to EL1
...with ESR 0x21/0x86000004
...with FAR 0x0000000000000000

# That last one is a NULL instruction fetch - the guest jumped
# through a null function pointer, and you have the ELR to find
# out from where. -d int is the highest-value flag in QEMU and
# almost nobody enables it.

# The heavyweight ones - use with a narrow window:
$ qemu-system-aarch64 ... -d exec,cpu -D trace.log
# every block execution, plus a full register dump. Gigabytes
# per second. Combine with a breakpoint:
#   run under gdb, break where you care, then
#   (gdb) monitor log exec,cpu
#   (gdb) stepi 1000
#   (gdb) monitor log none

# Toggle logging at runtime, which is how you keep it usable:
(qemu) log int,mmu
(qemu) log none
(qemu) logfile /tmp/window.log

# List everything available:
$ qemu-system-aarch64 -d help
```

> **`-D INT` IS THE ONE TO REMEMBER**
>
> A guest that hangs, resets in a loop, or dies silently is almost always taking an exception it cannot handle. `-d int` prints every exception with its ESR, ELR and FAR — the same three registers you would decode by hand — for the entire boot, at negligible cost. Reach for it before anything else.

--trace: QEMU-side tracepoints

```
# QEMU's own code is instrumented with thousands of static
# tracepoints. They are compiled in but disabled by default.

$ qemu-system-aarch64 -M virt --trace help 2>&1 | wc -l
4127

$ qemu-system-aarch64 -M virt --trace 'virtio_*' -D trace.log ...
virtio_queue_notify   vdev 0x5555 n 0 vq 0x5556
virtio_notify_irqfd   vdev 0x5555 vq 0x5556
virtblk_handle_write  req 0x5557 sector 2048 nsectors 8

$ qemu-system-aarch64 -M virt \
    --trace 'pl011_*' --trace 'gicv3_*' -D trace.log ...
pl011_read      addr 0x18 value 0x90
pl011_write     addr 0x00 value 0x4b
gicv3_icc_iar1_read  cpu 0 value 0x21
gicv3_icc_eoir_write cpu 0 value 0x21

# Many patterns at once, from a file:
$ cat > events.txt <<'EOF'
virtio_*
virtqueue_*
kvm_run_exit
memory_region_ops_read
memory_region_ops_write
EOF
$ qemu-system-aarch64 ... --trace events=events.txt -D trace.log

# THE TWO MOST USEFUL, and they work for ANY device including
# one you just wrote:
--trace 'memory_region_ops_read'
--trace 'memory_region_ops_write'
# Every MMIO access QEMU dispatched, with the region name,
# offset, value and size. "Is my device being accessed at all?"
# answered definitively.

# Toggle at runtime:
(qemu) trace-event virtio_queue_notify on
(qemu) trace-event pl011_* off
(qemu) info trace-events | head

# Into the host's ftrace, so guest and host events interleave
# on one timeline (needs --enable-trace-backends=ftrace):
$ echo 1 | sudo tee /sys/kernel/debug/tracing/tracing_on
$ qemu-system-aarch64 ... --trace 'virtio_*'
$ sudo cat /sys/kernel/debug/tracing/trace
```

Adding a tracepoint to your own device

```
# If you are writing a device model (chapter 13), instrument it
# the way QEMU does rather than with printf.

# 1. hw/misc/trace-events
mydev_read(uint64_t offset, uint64_t value, unsigned size) \
    "offset 0x%" PRIx64 " value 0x%" PRIx64 " size %u"
mydev_write(uint64_t offset, uint64_t value, unsigned size) \
    "offset 0x%" PRIx64 " value 0x%" PRIx64 " size %u"
mydev_irq(int level) "level %d"
mydev_reset(void) ""

# 2. In your source:
#include "trace.h"

static uint64_t mydev_read(void *opaque, hwaddr off, unsigned sz)
{
        MyDevState *s = opaque;
        uint64_t v = do_read(s, off);
        trace_mydev_read(off, v, sz);
        return v;
}

# 3. Use it:
$ qemu-system-aarch64 ... --trace 'mydev_*' -D mydev.log

# Zero cost when disabled (a branch on a flag), full formatting
# when enabled, works with every backend, and does not clutter
# stdout. There is no reason to use printf in a device model.
```

TCG plugins: instrumentation you write

```
# Plugins are shared libraries called back on translation and
# execution events. They work under TCG only (not KVM), and
# they see EVERYTHING.

$ ls build/tests/plugin/
libbb.so  libempty.so  libinsn.so  libmem.so  libhotblocks.so
libhotpages.so  libhowvec.so  libcache.so  libsyscall.so

# Count instructions - a deterministic "cycle" count for
# comparing two implementations:
$ qemu-system-aarch64 ... -plugin build/tests/plugin/libinsn.so \
    -d plugin -D insn.log
cpu 0 insns: 148291043

# Hot basic blocks:
$ qemu-system-aarch64 ... \
    -plugin build/tests/plugin/libhotblocks.so,count=20 -d plugin
collected 154321 entries in the hash table
pc, hits, insns
0x0000ffff80081234, 4092211, 14
0x0000ffff80095678, 2881100,  6

# Hot pages - shows the guest's memory access distribution:
$ qemu-system-aarch64 ... -plugin build/tests/plugin/libhotpages.so \
    -d plugin

# A real cache model - miss rates for a configuration you
# choose, on real guest execution:
$ qemu-system-aarch64 ... -plugin build/tests/plugin/libcache.so,\
dcachesize=32768,dassoc=4,dblksize=64,icachesize=32768 -d plugin
core #, data accesses, data misses, dmiss rate, insn accesses, ...
0       12043221       401223      3.33%       48120311  ...
```

```
/* Your own plugin. ~80 lines for something genuinely useful:
   count every access to one MMIO address.                  */

#include <qemu-plugin.h>
#include <stdio.h>
#include <inttypes.h>

QEMU_PLUGIN_EXPORT int qemu_plugin_version = QEMU_PLUGIN_VERSION;

static uint64_t watch_addr = 0;
static uint64_t reads = 0, writes = 0;

static void mem_cb(unsigned int vcpu, qemu_plugin_meminfo_t info,
                   uint64_t vaddr, void *udata)
{
        if (vaddr < watch_addr || vaddr >= watch_addr + 0x1000)
                return;
        if (qemu_plugin_mem_is_store(info))
                writes++;
        else
                reads++;
}

static void vcpu_tb_trans(qemu_plugin_id_t id,
                          struct qemu_plugin_tb *tb)
{
        size_t n = qemu_plugin_tb_n_insns(tb);
        for (size_t i = 0; i < n; i++) {
                struct qemu_plugin_insn *insn =
                        qemu_plugin_tb_get_insn(tb, i);
                qemu_plugin_register_vcpu_mem_cb(
                        insn, mem_cb, QEMU_PLUGIN_CB_NO_REGS,
                        QEMU_PLUGIN_MEM_RW, NULL);
        }
}

static void plugin_exit(qemu_plugin_id_t id, void *p)
{
        g_autoptr(GString) out = g_string_new(NULL);
        g_string_printf(out,
                "region 0x%" PRIx64 ": %" PRIu64 " reads, "
                "%" PRIu64 " writes\n", watch_addr, reads, writes);
        qemu_plugin_outs(out->str);
}

QEMU_PLUGIN_EXPORT int qemu_plugin_install(
        qemu_plugin_id_t id, const qemu_info_t *info,
        int argc, char **argv)
{
        for (int i = 0; i < argc; i++)
                if (g_str_has_prefix(argv[i], "addr="))
                        watch_addr = strtoull(argv[i] + 5, NULL, 0);

        qemu_plugin_register_vcpu_tb_trans_cb(id, vcpu_tb_trans);
        qemu_plugin_register_atexit_cb(id, plugin_exit, NULL);
        return 0;
}
```

```
$ gcc -shared -fPIC -o watch.so watch.c \
      -I/path/to/qemu/include/qemu $(pkg-config --cflags glib-2.0)

$ qemu-system-aarch64 ... -plugin ./watch.so,addr=0x09000000 \
    -d plugin
region 0x9000000: 41022 reads, 8813 writes

# "My driver is slow - how much is it touching the hardware?"
# answered exactly, without modifying the driver, without a
# logic analyser, and with a number you can put in a commit
# message.
```

How it shows up

Symptom	Cause
`-d` produces nothing	Output went to stderr and was swallowed — add `-D file`
`-d exec` fills the disk	Working as intended. Narrow the window with the monitor's `log` command
Plugin fails to load	QEMU built without `--enable-plugins`, or an ABI mismatch between plugin and binary
Plugin produces no output	`-d plugin` missing, or the plugin writes with `qemu_plugin_outs` which needs it
Plugin sees nothing under `-accel kvm`	Plugins are a TCG feature. There is no instruction stream to instrument under KVM
`--trace` pattern matches nothing	Check the exact name with `--trace help`; patterns are glob, not regex
Tracepoint added but never fires	`trace-events` not in the directory's `meson.build`, or `trace.h` not included
Guest behaves differently with tracing on	Logging slows the guest enough to change timing — expected, and sometimes informative
Instruction counts differ between runs	Non-determinism from I/O; add `-icount`

Check yourself

- ✓ Say which of the three systems answers "what did the guest execute", "what did QEMU do", and "how many times".
- ✓ Name the four `-d` flags worth enabling by default and what each catches.
- ✓ Read a `-d int` entry and extract the exception type, ESR, ELR and FAR.
- ✓ Narrow `-d exec` to a small window using the monitor.
- ✓ Name the two `--trace` events that work for any device model.
- ✓ Add a tracepoint to a device model and say why it beats `printf`.
- ✓ Use a stock plugin to find the hottest basic blocks in a guest.
- ✓ Explain why plugins do not work under KVM.
- ✓ Write a plugin that counts accesses to one address range.
- ✓ Explain how to make instruction counts reproducible between runs.

Writing a QEMU device model

When the silicon does not exist yet, you write the device first and the driver against it. This chapter builds a complete, working QEMU device — registers, interrupt, device tree node, migration state — and then a Linux driver for it, so you have the full loop that hardware bring-up teams actually use.

Why you would do this

Situation	What the model buys you
Silicon is six months away	The driver is written, reviewed and tested before tape-out. This is the normal way large SoC drivers are developed
The board is on someone else's desk	Every engineer gets a target
You need to test an error path	Make the model return the error. No fault injector on real hardware can do this as precisely
CI must run driver tests	No board farm required
You are learning	Writing the device teaches you what the driver's assumptions actually are

QOM: QEMU's object model, in one page

```
Every device is a QOM object. Three things to declare:

1. A STATE struct - your instance data, beginning with the
   parent's state struct.

     struct MyDevState {
             SysBusDevice parent_obj;   <- MUST be first
             MemoryRegion iomem;
             qemu_irq     irq;
             uint32_t     regs[8];
     };

2. A TYPE registration - name, parent, sizes, class init.

     static const TypeInfo mydev_info = {
             .name          = TYPE_MYDEV,
             .parent        = TYPE_SYS_BUS_DEVICE,
             .instance_size = sizeof(MyDevState),
             .instance_init = mydev_init,
             .class_init    = mydev_class_init,
     };

3. LIFECYCLE callbacks:

     instance_init  object created. Set defaults only.
     realize        properties are set; NOW create memory
                    regions, allocate, connect IRQs. This is
                    where the real work goes.
     reset          system reset. Restore register defaults.
     unrealize      teardown.

Parent classes you will use:
     TYPE_SYS_BUS_DEVICE   memory-mapped, on the system bus
     TYPE_PCI_DEVICE       a PCIe function
     TYPE_VIRTIO_DEVICE    a virtio device
     TYPE_DEVICE           anything else
```

A complete device

```
/* hw/misc/acme-widget.c
   A device with an ID register, a scratch register, a counter
   that increments on read, and an interrupt you can trigger. */

#include "qemu/osdep.h"
#include "hw/sysbus.h"
#include "hw/irq.h"
#include "hw/qdev-properties.h"
#include "migration/vmstate.h"
#include "qemu/log.h"
#include "qom/object.h"
#include "trace.h"

#define TYPE_ACME_WIDGET "acme-widget"
OBJECT_DECLARE_SIMPLE_TYPE(AcmeWidgetState, ACME_WIDGET)

/* Register map - document it here, it is the contract. */
#define R_ID        0x00   /* RO  magic 0xACE00001            */
#define R_CTRL      0x04   /* RW  bit0 enable, bit1 irq enable */
#define R_STATUS    0x08   /* RO  bit0 irq pending             */
#define R_SCRATCH   0x0c   /* RW  anything                     */
#define R_COUNTER   0x10   /* RO  increments on each read      */
#define R_IRQ_SET   0x14   /* WO  write 1 to raise the irq     */
#define R_IRQ_CLR   0x18   /* WO  write 1 to clear it          */
#define R_MAX       0x1c

#define CTRL_ENABLE   (1u << 0)
#define CTRL_IRQ_EN   (1u << 1)
#define STATUS_IRQ    (1u << 0)

struct AcmeWidgetState {
        SysBusDevice parent_obj;

        MemoryRegion iomem;
        qemu_irq     irq;

        uint32_t ctrl;
        uint32_t status;
        uint32_t scratch;
        uint32_t counter;
        uint32_t id;          /* a property, so it is settable */
};

static void acme_update_irq(AcmeWidgetState *s)
{
        int level = !!(s->status & STATUS_IRQ) &&
                    !!(s->ctrl & CTRL_IRQ_EN);
        trace_acme_widget_irq(level);
        qemu_set_irq(s->irq, level);      /* LEVEL, not pulse */
}

static uint64_t acme_read(void *opaque, hwaddr off, unsigned size)
{
        AcmeWidgetState *s = opaque;
        uint64_t v = 0;

        switch (off) {
        case R_ID:      v = s->id;                  break;
        case R_CTRL:    v = s->ctrl;                break;
        case R_STATUS:  v = s->status;              break;
```

```
        case R_SCRATCH: v = s->scratch;             break;
        case R_COUNTER: v = s->counter++;           break;
        default:
                /* NEVER fault. Log and return zero, like real
                   silicon. */
                qemu_log_mask(LOG_GUEST_ERROR,
                        "acme-widget: read of unknown offset 0x%"
                        HWADDR_PRIx "\n", off);
                break;
        }
        trace_acme_widget_read(off, v, size);
        return v;
}

static void acme_write(void *opaque, hwaddr off, uint64_t val,
                       unsigned size)
{
        AcmeWidgetState *s = opaque;

        trace_acme_widget_write(off, val, size);

        switch (off) {
        case R_CTRL:
                s->ctrl = val & (CTRL_ENABLE | CTRL_IRQ_EN);
                acme_update_irq(s);
                break;
        case R_SCRATCH:
                s->scratch = val;
                break;
        case R_IRQ_SET:
                if (val & 1) {
                        s->status |= STATUS_IRQ;
                        acme_update_irq(s);
                }
                break;
        case R_IRQ_CLR:
                if (val & 1) {
                        s->status &= ~STATUS_IRQ;
                        acme_update_irq(s);
                }
                break;
        case R_ID:
        case R_STATUS:
        case R_COUNTER:
                qemu_log_mask(LOG_GUEST_ERROR,
                        "acme-widget: write to RO offset 0x%"
                        HWADDR_PRIx "\n", off);
                break;
        default:
                qemu_log_mask(LOG_GUEST_ERROR,
                        "acme-widget: write to unknown offset 0x%"
                        HWADDR_PRIx "\n", off);
                break;
        }
}

static const MemoryRegionOps acme_ops = {
        .read       = acme_read,
        .write      = acme_write,
        .endianness = DEVICE_LITTLE_ENDIAN,
        .valid = {                  /* what the GUEST may do    */
                .min_access_size = 4,
                .max_access_size = 4,
```

```
                .unaligned      = false,
        },
        .impl = {                  /* what WE implement; QEMU
                                      splits/combines to match */
                .min_access_size = 4,
                .max_access_size = 4,
        },
};

static void acme_reset(DeviceState *dev)
{
        AcmeWidgetState *s = ACME_WIDGET(dev);

        s->ctrl    = 0;
        s->status  = 0;
        s->scratch = 0;
        s->counter = 0;
        acme_update_irq(s);
}

static void acme_realize(DeviceState *dev, Error **errp)
{
        AcmeWidgetState *s = ACME_WIDGET(dev);
        SysBusDevice   *sbd = SYS_BUS_DEVICE(dev);

        memory_region_init_io(&s->iomem, OBJECT(s), &acme_ops, s,
                              TYPE_ACME_WIDGET, 0x1000);
        sysbus_init_mmio(sbd, &s->iomem);
        sysbus_init_irq(sbd, &s->irq);
}

/* Migration state. Without this, savevm/loadvm and migration
   silently lose your device's registers.                     */
static const VMStateDescription acme_vmstate = {
        .name    = TYPE_ACME_WIDGET,
        .version_id = 1,
        .minimum_version_id = 1,
        .fields = (const VMStateField[]) {
                VMSTATE_UINT32(ctrl,    AcmeWidgetState),
                VMSTATE_UINT32(status,  AcmeWidgetState),
                VMSTATE_UINT32(scratch, AcmeWidgetState),
                VMSTATE_UINT32(counter, AcmeWidgetState),
                VMSTATE_END_OF_LIST()
        },
};

static const Property acme_properties[] = {
        DEFINE_PROP_UINT32("id", AcmeWidgetState, id, 0xACE00001),
};

static void acme_class_init(ObjectClass *klass, void *data)
{
        DeviceClass *dc = DEVICE_CLASS(klass);

        dc->realize  = acme_realize;
        device_class_set_legacy_reset(dc, acme_reset);
        dc->vmsd     = &acme_vmstate;
        dc->desc     = "Acme Widget";
        device_class_set_props(dc, acme_properties);
        set_bit(DEVICE_CATEGORY_MISC, dc->categories);
}

static const TypeInfo acme_info = {
```

```
        .name          = TYPE_ACME_WIDGET,
        .parent        = TYPE_SYS_BUS_DEVICE,
        .instance_size = sizeof(AcmeWidgetState),
        .class_init    = acme_class_init,
};

static void acme_register_types(void)
{
        type_register_static(&acme_info);
}
type_init(acme_register_types)
```

Wiring it into the build and the machine

```
# 1. hw/misc/trace-events - add your tracepoints
acme_widget_read(uint64_t off, uint64_t val, unsigned sz) \
    "off 0x%" PRIx64 " val 0x%" PRIx64 " size %u"
acme_widget_write(uint64_t off, uint64_t val, unsigned sz) \
    "off 0x%" PRIx64 " val 0x%" PRIx64 " size %u"
acme_widget_irq(int level) "level %d"

# 2. hw/misc/Kconfig
config ACME_WIDGET
    bool
    default y if TEST_DEVICES

# 3. hw/misc/meson.build
system_ss.add(when: 'CONFIG_ACME_WIDGET',
              if_true: files('acme-widget.c'))

# 4. configs/devices/aarch64-softmmu/default.mak
CONFIG_ACME_WIDGET=y

# 5. Rebuild
$ ninja -C build

# 6. Instantiate it. The dynamic sysbus lets you do this from
#    the command line without touching the machine code:
$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -append "console=ttyAMA0" \
    -device acme-widget \
    --trace 'acme_widget_*' -D acme.log

# Confirm it exists:
(qemu) info qtree
    dev: acme-widget, id ""
      id = 2900361217 (0xace00001)
      mmio 000000000c000000/0000000000001000
(qemu) info mtree | grep acme
      000000000c000000-000000000c000fff (prio 0, i/o): acme-widget

# Poke it from the monitor, before writing any driver at all:
(qemu) xp/1xw 0x0c000000
000000000c000000: 0xace00001
(qemu) xp/1xw 0x0c000010
000000000c000010: 0x00000000
(qemu) xp/1xw 0x0c000010
000000000c000010: 0x00000001      # the counter increments
```

TEST THE MODEL BEFORE WRITING THE DRIVER

`xp` from the monitor exercises every register without a single line of guest code. If the ID register does not read back, the problem is in your model; if it does, every later failure is in the driver. That separation saves hours, and it is only available because you are on QEMU.

The device tree node

```
/* A dynamic sysbus device gets an auto-generated node with a
   generic compatible string, which no driver will match. For a
   real device you add it to the machine.                     */

/* hw/arm/virt.c, in the machine's device creation: */

static void create_acme_widget(const VirtMachineState *vms)
{
        hwaddr base = 0x0c000000;
        int    irq  = 48;
        DeviceState *dev;
        MachineState *ms = MACHINE(vms);
        char *nodename;

        dev = qdev_new(TYPE_ACME_WIDGET);
        sysbus_realize_and_unref(SYS_BUS_DEVICE(dev), &error_fatal);
        sysbus_mmio_map(SYS_BUS_DEVICE(dev), 0, base);
        sysbus_connect_irq(SYS_BUS_DEVICE(dev), 0,
                           qdev_get_gpio_in(vms->gic,
                                            irq));

        nodename = g_strdup_printf("/acme-widget@%" PRIx64, base);
        qemu_fdt_add_subnode(ms->fdt, nodename);
        qemu_fdt_setprop_string(ms->fdt, nodename,
                                "compatible", "acme,widget-1.0");
        qemu_fdt_setprop_sized_cells(ms->fdt, nodename, "reg",
                                     2, base, 2, 0x1000);
        qemu_fdt_setprop_cells(ms->fdt, nodename, "interrupts",
                               GIC_FDT_IRQ_TYPE_SPI, irq,
                               GIC_FDT_IRQ_FLAGS_LEVEL_HI);
        g_free(nodename);
}

/* Verify it landed: */
$ qemu-system-aarch64 -M virt,dumpdtb=v.dtb ... && \
  dtc -I dtb -O dts v.dtb | grep -A5 acme
```

The driver, for the loop to close

```
// acme_widget.c - a Linux platform driver for the model above.
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/of.h>
#include <linux/io.h>
#include <linux/interrupt.h>

#define R_ID 0x00
#define R_CTRL 0x04
#define R_STATUS 0x08
#define R_SCRATCH 0x0c
#define R_COUNTER 0x10
#define R_IRQ_SET 0x14
#define R_IRQ_CLR 0x18

struct acme {
        void __iomem *base;
        int irq;
};

static irqreturn_t acme_isr(int irq, void *dev_id)
{
        struct acme *a = dev_id;

        writel(1, a->base + R_IRQ_CLR);     /* level: must clear */
        pr_info("acme: interrupt\n");
        return IRQ_HANDLED;
}

static int acme_probe(struct platform_device *pdev)
{
        struct acme *a;
        u32 id;

        a = devm_kzalloc(&pdev->dev, sizeof(*a), GFP_KERNEL);
        if (!a)
                return -ENOMEM;

        a->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(a->base))
                return PTR_ERR(a->base);

        id = readl(a->base + R_ID);
        if (id != 0xACE00001) {
                dev_err(&pdev->dev, "bad id 0x%08x\n", id);
                return -ENODEV;
        }

        a->irq = platform_get_irq(pdev, 0);
        if (a->irq < 0)
                return a->irq;

        if (devm_request_irq(&pdev->dev, a->irq, acme_isr, 0,
                             "acme-widget", a))
                return -EBUSY;

        writel(3, a->base + R_CTRL);        /* enable + irq en   */
        platform_set_drvdata(pdev, a);

        dev_info(&pdev->dev, "probed, id 0x%08x, counter %u\n",
```

```
                  id, readl(a->base + R_COUNTER));

        writel(1, a->base + R_IRQ_SET);     /* fire one          */
        return 0;
}

static const struct of_device_id acme_of_match[] = {
        { .compatible = "acme,widget-1.0" },
        { }
};
MODULE_DEVICE_TABLE(of, acme_of_match);

static struct platform_driver acme_driver = {
        .probe  = acme_probe,
        .driver = {
                .name           = "acme-widget",
                .of_match_table = acme_of_match,
        },
};
module_platform_driver(acme_driver);
MODULE_LICENSE("GPL");
```

```
# Build it against your kernel, put it in the initramfs
# (chapter 6), and run:

$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512M -nographic \
    -kernel Image -initrd initramfs.cpio.gz \
    -append "console=ttyAMA0" \
    --trace 'acme_widget_*' -D acme.log

/ # insmod /lib/modules/acme_widget.ko
acme-widget c000000.acme-widget: probed, id 0xace00001, counter 0
acme: interrupt

$ cat acme.log
acme_widget_read  off 0x0 val 0xace00001 size 4
acme_widget_read  off 0x10 val 0x0 size 4
acme_widget_write off 0x4 val 0x3 size 4
acme_widget_irq   level 0
acme_widget_write off 0x14 val 0x1 size 4
acme_widget_irq   level 1
acme_widget_write off 0x18 val 0x1 size 4
acme_widget_irq   level 0

# Both sides of the boundary, in one log. This is the loop:
# change the model, rebuild, rerun, see exactly what the driver
# did. No hardware exists and none is needed.
```

LEVEL, NOT PULSE

Model the interrupt as a level that you recompute from state (`qemu_set_irq(irq, level)`), never as a pulse. A pulse can be lost, and worse, it hides driver bugs: a driver that forgets to clear the interrupt source works fine against a pulse model and locks the machine up on real hardware. The model should be as unforgiving as the silicon.

How it shows up

Symptom	Cause
Device does not appear in `info qtree`	Not in `meson.build`, or `CONFIG_` not enabled, or not rebuilt
Driver never probes	`compatible` string mismatch — check the dumped DTB
Reads return zero	Wrong base address, or the region size is smaller than the offset you are reading
Guest hangs on the first access	`valid.min_access_size` excludes the access the driver made
Interrupt storm	Level modelled but the driver's clear path does not actually lower it
Interrupt fires once, never again	Modelled as a pulse, or `acme_update_irq` not called after a state change
Registers wrong after `loadvm`	No `VMStateDescription`, or a field missing from it
Byte access from the guest behaves oddly	`impl` and `valid` access sizes inconsistent
Guest kernel panics probing your device	Model faulting on an unknown offset instead of logging and returning zero
Works in `-device` form, not on the machine	The dynamic sysbus node has a generic `compatible`; add a proper node in the machine

Check yourself

- ✓ Give three situations where writing the device model comes before the driver.
- ✓ Name the three things a QOM type declaration needs and the four lifecycle callbacks.
- ✓ Explain why the parent state struct must be the first member.
- ✓ Say what belongs in `instance_init` and what belongs in `realize`.
- ✓ Explain the difference between `valid` and `impl` access sizes.
- ✓ Explain why an unknown register offset must not fault.
- ✓ Explain why the interrupt must be modelled as a level, and what bug a pulse model hides.
- ✓ Say what `VMStateDescription` is for and what breaks without it.
- ✓ Test a device model from the QEMU monitor before writing any driver.
- ✓ Add a device tree node so a real driver's `compatible` string matches.

QEMU with KVM on real hardware

Switching from TCG to KVM changes one flag and everything else: speed goes up a hundredfold, timing becomes meaningful, and every debugging tool that depended on QEMU seeing each instruction stops working. This chapter covers the switch, what you gain, what you lose, and how to set up a machine that serves both.

Making the switch

```
# The requirement: guest architecture == host architecture, and
# the CPU must have reached EL2 at boot (virt chapter 3).

$ ls -l /dev/kvm
crw-rw---- 1 root kvm 10, 232 /dev/kvm
$ sudo usermod -aG kvm $USER      # then log out and back in

$ dmesg | grep -i kvm
[    0.512430] kvm [1]: Hyp mode initialized successfully
[    0.512461] kvm [1]: VHE mode initialized successfully

# TCG, for reference:
$ qemu-system-aarch64 -M virt -cpu cortex-a57 -m 2G -nographic \
    -kernel Image -append "console=ttyAMA0"

# KVM. Three changes:
$ qemu-system-aarch64 -M virt,gic-version=host -cpu host \
    -accel kvm -smp 4 -m 2G -nographic \
    -kernel Image -append "console=ttyAMA0"

#   -accel kvm            use the hardware
#   -cpu host             pass the real CPU through. Any other
#                         -cpu value under KVM either fails or
#                         silently gives you the host CPU anyway
#                         - KVM cannot emulate a different core.
#   gic-version=host      match the real interrupt controller.
#                         A mismatch fails at startup.

# Fall back automatically, for a script that must work on both:
-accel kvm -accel tcg
# QEMU tries them in order and uses the first that initialises.
```

What you gain and lose

	TCG	KVM
Speed	10–50x slower than native	Near native
Cross-architecture	Yes	No
`-d in_asm`, `-d exec`, `-d cpu`	Works	**Nothing** — QEMU never sees the instructions
TCG plugins	Works	**Unavailable**
`-icount`, record/replay, reverse debugging	Works	**Unavailable**

	TCG	KVM
GDB stub	Full	Works, but breakpoints use hardware debug registers and are limited in number
Watchpoints	Unlimited, software	Limited to the CPU's watchpoint registers
`-d int`, `-d mmu`	Full detail	Only what KVM traps
`--trace` on device models	Works	Works — device models still run in QEMU
Monitor `info registers`, `x`, `savevm`	Works	Works
Timing realism	Meaningless	Roughly meaningful
Race reproduction	Poor — ordering is too strong	Real
Emulated CPU features	`-cpu max` gives you anything	Only what the silicon has

USE BOTH, DELIBERATELY

Develop under TCG on your laptop, where `-d exec`, plugins and reverse debugging exist and where you can pretend to have an SoC that does not. Validate under KVM on real arm64 hardware, where timing is real and races reproduce. A bug found only under KVM is a timing bug; a bug found only under TCG is usually a logic bug you got lucky about on hardware. Both are worth finding.

Setting up an arm64 host

```
# Options, cheapest first:
#   Raspberry Pi 5 / Orange Pi 5 - works, slow-ish, ~$80
#   Apple silicon Mac + Asahi Linux or a Linux VM under
#      Virtualization.framework (nested, see below)
#   An Ampere/Graviton cloud instance - the easiest path;
#      c7g/t4g on AWS, or Oracle's free-tier Ampere
#   Any arm64 server board

# 1. Confirm EL2. THIS IS THE ONE THAT FAILS.
$ dmesg | grep -i 'CPU.*started at'
[    0.000000] CPU: All CPU(s) started at EL2       # good
[    0.000000] CPU: All CPU(s) started at EL1       # no KVM,
                                                     # ever
# On a Raspberry Pi, EL2 requires:
$ grep -q '^arm_64bit=1' /boot/firmware/config.txt
$ echo 'arm_control=0x200' | sudo tee -a /boot/firmware/config.txt
# (older firmwares; newer ones start at EL2 by default)

# 2. Packages
$ sudo apt install qemu-system-arm qemu-utils \
      bridge-utils libvirt-daemon-system virtinst

# 3. Permissions
$ sudo usermod -aG kvm,libvirt $USER

# 4. Hugepages, worth doing (virt chapter 21)
$ echo 2048 | sudo tee /proc/sys/vm/nr_hugepages
$ echo 'vm.nr_hugepages=2048' | sudo tee -a /etc/sysctl.conf

# 5. Memlock limit, needed for passthrough
$ sudo tee /etc/security/limits.d/kvm.conf <<'EOF'
@kvm  soft  memlock  unlimited
@kvm  hard  memlock  unlimited
EOF

# 6. Verify end to end
$ qemu-system-aarch64 -M virt,gic-version=host -cpu host \
    -accel kvm -smp 2 -m 1G -nographic \
    -kernel Image -append "console=ttyAMA0"
# Kernel boot in under a second = KVM is working. Twenty
# seconds = you are still on TCG.
```

Nesting, so you can develop hypervisors

```
# To run KVM INSIDE a guest (virt chapter 20), the guest needs
# EL2:

$ qemu-system-aarch64 -M virt,virtualization=on \
    -cpu host,el2=on -accel kvm -smp 4 -m 4G ...

# Inside that guest:
$ dmesg | grep -i 'started at'
[    0.000000] CPU: All CPU(s) started at EL2
$ ls /dev/kvm
# ...and you can run a third-level VM.

# On the HOST, nested support must be enabled:
#   kernel command line: kvm-arm.mode=nested
# and the hardware needs FEAT_NV2 for it to be usable rather
# than merely possible.

# Under TCG there is no such restriction - -cpu max always has
# EL2, and nested virtualisation "works" at TCG speed. That is
# often the right way to develop a hypervisor: correctness
# under TCG with full tracing, performance under KVM later.

$ qemu-system-aarch64 -M virt,virtualization=on,secure=on \
    -cpu max -accel tcg -smp 2 -m 2G -nographic \
    -kernel my-hypervisor.elf -s -S
# EL3, EL2, S-EL2, single-steppable, fully traceable, on a
# laptop. No hardware in the world gives you that.
```

libvirt, when you have more than a couple of VMs

```
# Hand-written command lines stop scaling around the third VM.
# libvirt gives you persistence, autostart, networking and a
# stable API - and it is just generating the same QEMU command
# line underneath.

$ virt-install \
    --name test-arm64 \
    --memory 2048 --vcpus 4 \
    --arch aarch64 --machine virt \
    --cpu host-passthrough \
    --boot uefi \
    --disk path=/var/lib/libvirt/images/test.qcow2,size=20 \
    --network network=default,model=virtio \
    --graphics none --console pty,target_type=serial \
    --os-variant debian12 \
    --location http://deb.debian.org/debian/dists/bookworm/main/installer-arm64/

$ virsh list --all
$ virsh start test-arm64
$ virsh console test-arm64          # Ctrl-] to leave
$ virsh edit test-arm64             # the XML
$ virsh dumpxml test-arm64

# See the command line libvirt actually built - useful when you
# need to reproduce something by hand:
$ virsh domxml-to-native qemu-argv --domain test-arm64

# Pin vCPUs, for latency work:
$ virsh vcpupin test-arm64 0 4
$ virsh emulatorpin test-arm64 8-11

# Hugepages, in the XML:
#   <memoryBacking><hugepages/></memoryBacking>
```

Confirming you are actually getting KVM speed

```
# The three-second check: boot time.
$ time qemu-system-aarch64 -M virt,gic-version=host -cpu host \
    -accel kvm -smp 4 -m 2G -nographic -kernel Image \
    -append "console=ttyAMA0 panic=1" -no-reboot
# under a second to userspace = KVM

# Confirm from inside the guest:
$ dmesg | grep -i 'paravirt\|hypervisor\|KVM'
$ lscpu | grep -i hypervisor
Hypervisor vendor:  KVM

# Exit rate - the number that predicts performance (virt
# chapter 21):
$ sudo perf kvm --host stat live

# Compare like for like:
$ sysbench cpu --threads=4 run          # in the guest
$ sysbench cpu --threads=4 run          # on the host
# Within 5% = KVM working properly. 50x slower = TCG.

# The most common cause of "KVM is slow": it is not KVM.
$ ps aux | grep qemu | grep -o 'accel [a-z]*'
```

MIGRATION AND `-CPU HOST`

`-cpu host` exposes the exact silicon, so a VM started on one CPU model cannot migrate to another. If migration matters, choose a named CPU model that both machines support and accept the lost features. If it does not — which is the case for all development work — use `host` and get everything.

How it shows up

Symptom	Cause
`Could not access KVM kernel module: Permission denied`	Not in the `kvm` group, or you did not log out after being added
`KVM not supported for this target`	Guest and host architectures differ
`kvm [1]: HYP mode not available`	Booted at EL1; firmware consumed EL2
Startup fails with a GIC error	`gic-version` does not match the host — use `host`
Guest is slow despite `-accel kvm`	Check it actually initialised; QEMU may have fallen back to TCG if you listed both
`-d exec` produces nothing	Expected under KVM. Switch to TCG for that investigation
Plugins fail to load	Also expected under KVM
Too many breakpoints	KVM uses hardware debug registers; there are only a handful
Guest cannot see a CPU feature	Under KVM you get the real silicon only; use TCG with `-cpu max`
Migration refused	`-cpu host` across dissimilar machines, or a passed-through device
Nested KVM unavailable in a guest	`virtualization=on` and `el2=on` missing, or host nesting not enabled

Check yourself

- ✓ Give the three command-line changes needed to move from TCG to KVM.
- ✓ Explain why `-cpu host` is effectively mandatory under KVM.
- ✓ List five debugging capabilities you lose under KVM.
- ✓ Give a rule for which accelerator to use for which class of bug.
- ✓ Determine whether an arm64 machine can run KVM at all, and say what to check first.
- ✓ Explain what `-accel kvm -accel tcg` does.
- ✓ Configure a guest that can itself run KVM.
- ✓ Explain why developing a hypervisor under TCG can be better than under KVM.
- ✓ Verify in three ways that a running guest is actually accelerated.
- ✓ Explain the trade-off `-cpu host` imposes on migration.

Recipes: one command line per track

A reference card. Each recipe is a complete, working command line for one kind of work on this site, with the reason for every non-obvious option. Copy, adjust the paths, run.

ASSUMPTIONS

All recipes assume `qemu-system-aarch64` 8.x or newer, a cross toolchain, and an `Image` plus `initramfs.cpio.gz` built as in chapter 5 and chapter 6. Add `-accel kvm -cpu host -M virt,gic-version=host` on an arm64 host to any of them.

ARM architecture track: exception levels and system registers

```
#!/bin/sh
# Bare-metal payload, all exception levels available, stopped
# for the debugger.
qemu-system-aarch64 \
  -M virt,secure=on,virtualization=on,gic-version=3 \
  -cpu max \
  -smp 1 -m 512M -nographic \
  -kernel payload.elf \
  -d int,mmu,guest_errors,unimp -D qemu.log \
  -s -S

#   secure=on         EL3 exists -> you can write monitor code
#   virtualization=on EL2 exists -> you can write hypervisor code
#   -cpu max          every architectural feature: SVE, MTE,
#                     pointer auth, FEAT_NV2, S-EL2
#   -d int            EVERY exception, with ESR/ELR/FAR. This is
#                     the flag that makes this track teachable.
#   -s -S             stopped, GDB on :1234

# then:
$ aarch64-none-elf-gdb payload.elf \
    -ex 'target remote :1234' -ex 'layout regs' -ex 'break _start'
```

Linux kernel and driver track

```
#!/bin/sh
# Kernel plus initramfs, module loading, GDB, and a shared
# directory so you can rebuild a module without repacking.
KDIR=${KDIR:-$HOME/work/linux}

qemu-system-aarch64 \
  -M virt -cpu cortex-a57 -smp 2 -m 1G -nographic \
  -kernel $KDIR/arch/arm64/boot/Image \
  -initrd initramfs.cpio.gz \
  -append "console=ttyAMA0 earlycon nokaslr root=/dev/ram0 rw" \
  -fsdev local,id=fs0,path=$PWD/share,security_model=none \
  -device virtio-9p-device,fsdev=fs0,mount_tag=share \
  -netdev user,id=n0,hostfwd=tcp::2222-:22 \
  -device virtio-net-device,netdev=n0 \
  -object rng-random,filename=/dev/urandom,id=rng0 \
  -device virtio-rng-device,rng=rng0 \
  -d guest_errors,unimp -D qemu.log \
  -s

#   nokaslr           so GDB symbols match (chapter 11)
#   9p share          rebuild a module on the host, insmod it in
#                     the guest, no repacking
#   virtio-rng        without it, early boot blocks on entropy

# In the guest:
#   mount -t 9p -o trans=virtio,version=9p2000.L share /mnt
#   insmod /mnt/mydriver.ko
# On the host:
#   gdb-multiarch $KDIR/vmlinux -ex 'target remote :1234' \
#     -ex "source $KDIR/vmlinux-gdb.py" -ex 'lx-symbols'
```

Virtualisation track: nested, with everything visible

```
#!/bin/sh
# A guest that can itself run KVM, under TCG so that every
# instruction is traceable.
qemu-system-aarch64 \
  -M virt,virtualization=on,gic-version=3,its=on \
  -cpu max \
  -smp 4 -m 4G -nographic \
  -kernel Image \
  -append "console=ttyAMA0 earlycon kvm-arm.mode=nested" \
  -drive file=rootfs.ext4,if=none,format=raw,id=hd0 \
  -device virtio-blk-device,drive=hd0 \
  -netdev user,id=n0,hostfwd=tcp::2222-:22 \
  -device virtio-net-device,netdev=n0 \
  --trace 'kvm_*' --trace 'virtio_*' -D trace.log

#   -cpu max provides FEAT_NV2 even though no laptop has it
#   its=on   needed for PCIe MSI-X inside the guest

# On an arm64 host, the fast version:
#   -M virt,virtualization=on,gic-version=host \
#   -cpu host,el2=on -accel kvm
```

OP-TEE and TrustZone track

```
#!/bin/sh
# TF-A at EL3, OP-TEE at S-EL1, Linux at EL1, with the secure
# world's console on a second UART.
qemu-system-aarch64 \
  -M virt,secure=on,virtualization=on,gic-version=3 \
  -cpu max -smp 2 -m 1G -nographic \
  -bios bl1.bin \
  -device loader,file=fip.bin,addr=0x40100000 \
  -semihosting-config enable=on,target=native \
  -serial mon:stdio \
  -serial file:secure-world.log \
  -netdev user,id=n0 -device virtio-net-device,netdev=n0

#   -bios bl1.bin     TF-A's first stage. NOT -kernel: we are
#                     the firmware now (chapter 5)
#   fip.bin           the Firmware Image Package: BL2, BL31,
#                     BL32 (OP-TEE) and BL33 (U-Boot/Linux)
#   semihosting       TF-A on QEMU uses it to load the FIP
#   two -serial       first = ttyAMA0 = normal world
#                     second = the SECURE world's console. Every
#                     TA panic and trace line lands there.

# The OP-TEE build system generates this for you:
#   cd optee/build && make run
# and the two consoles appear in separate windows. See the
# OP-TEE track's chapter 4 for the full build.
```

Device-model development

```
#!/bin/sh
# Your own QEMU build, your own device, full tracing on both
# sides of the boundary.
QEMU=$HOME/opt/qemu/bin/qemu-system-aarch64

$QEMU \
  -M virt -cpu cortex-a57 -m 512M -nographic \
  -kernel Image -initrd initramfs.cpio.gz \
  -append "console=ttyAMA0 earlycon" \
  -device acme-widget \
  --trace 'acme_widget_*' \
  --trace 'memory_region_ops_read' \
  --trace 'memory_region_ops_write' \
  -d guest_errors,unimp -D dev.log \
  -s

#   memory_region_ops_* traces EVERY MMIO access QEMU
#   dispatched, for every device - so "is my device being
#   touched at all" is answered without any device-specific
#   instrumentation (chapter 12).
```

Fast iteration: overlay plus snapshot

```
#!/bin/sh
# Golden image untouched; every run disposable; snapshot before
# the expensive setup.
[ -f work.qcow2 ] || \
  qemu-img create -f qcow2 -b golden.qcow2 -F qcow2 work.qcow2

qemu-system-aarch64 \
  -M virt -cpu cortex-a57 -smp 4 -m 2G -nographic \
  -kernel Image \
  -drive file=work.qcow2,if=none,id=hd0,cache=unsafe \
  -device virtio-blk-device,drive=hd0 \
  -append "console=ttyAMA0 root=/dev/vda rw" \
  -monitor unix:/tmp/hmp.sock,server=on,wait=off \
  -qmp unix:/tmp/qmp.sock,server=on,wait=off

#   cache=unsafe   10x faster and safe HERE because the overlay
#                  is disposable. Never on anything you keep.

# Reset to pristine:  rm work.qcow2 && rerun
# Snapshot:  echo savevm ready | socat - UNIX-CONNECT:/tmp/hmp.sock
# Restore:   echo loadvm ready | socat - UNIX-CONNECT:/tmp/hmp.sock
```

CI: boot, run a test, exit with a status

```
#!/bin/sh
# Headless, bounded, detects panics, returns a real exit code.
set -e
timeout 300 qemu-system-aarch64 \
  -M virt -cpu cortex-a57 -smp 2 -m 1G \
  -nographic -no-reboot \
  -kernel Image -initrd test-initramfs.cpio.gz \
  -append "console=ttyAMA0 panic=-1 rdinit=/run-tests.sh" \
  -device pvpanic-pci \
  -serial mon:stdio \
  | tee test.log

grep -q "ALL TESTS PASSED" test.log || { echo "FAIL"; exit 1; }
grep -q "Kernel panic"     test.log && { echo "PANIC"; exit 1; }
echo OK

#   -no-reboot      a reset ends the run instead of looping
#   panic=-1        panic reboots immediately -> with -no-reboot,
#                   QEMU exits. A panic becomes a fast failure
#                   rather than a 300-second timeout.
#   pvpanic-pci     QEMU emits a GUEST_PANICKED QMP event too
#   timeout 300     the outer bound. Always have one.

# The test script inside the initramfs ends with:
#   echo "ALL TESTS PASSED"
#   poweroff -f
```

User-mode: run one cross-compiled binary

```
# No VM, no kernel, no rootfs. The fastest way to run an
# aarch64 test suite on an x86 machine.

$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./my-test

# Transparently, via binfmt:
$ sudo apt install qemu-user-static binfmt-support
$ ./my-test              # the kernel routes it to qemu-aarch64

# Under GDB:
$ qemu-aarch64 -g 1234 -L /usr/aarch64-linux-gnu ./my-test &
$ gdb-multiarch ./my-test -ex 'target remote :1234'

# With instruction counting, to compare two implementations
# deterministically:
$ qemu-aarch64 -plugin libinsn.so -d plugin ./my-test

# Emulate a CPU with features your host lacks:
$ qemu-aarch64 -cpu max ./sve-test
```

The flag cheat sheet

Flag	Reach for it when
`-nographic`	Always, for headless work. Ctrl-A c for the monitor, Ctrl-A x to quit
`-s -S`	You want to debug from instruction zero
`-d int`	The guest hangs, resets, or dies silently. *Start here*
`-d guest_errors,unimp`	Always, during bring-up
`-D file`	Every time you use `-d`
`--trace 'memory_region_ops_*'`	"Is my device being accessed?"
`-M virt,dumpdtb=x.dtb`	"Why is my device not probing?"
`-monitor stdio` / Ctrl-A c	Anything is wrong and you want `info registers`
`-snapshot`	You are about to break the disk image
`-icount shift=5`	A race that will not reproduce
`-no-reboot -no-shutdown`	You need to inspect the machine after it dies
`-nodefaults`	You want exactly the machine you specified and nothing more
`-accel kvm -cpu host`	Same architecture and you want speed
`-cpu max`	You need a feature no hardware you own has
`-semihosting-config enable=on`	Bare-metal code that wants host file or console access

How it shows up

Symptom	First thing to try
No output at all	Add `earlycon`; check `console=ttyAMA0` and `-nographic`
Hangs somewhere in boot	`-d int -D log` and read the last exception

Symptom	First thing to try
Device not probing	`dumpdtb` and check the `compatible` and `reg`
Cannot quit QEMU	Ctrl-A x; if the terminal eats Ctrl-A, add `-echr 0x05`
Guest dies and takes the evidence with it	`-no-reboot -no-shutdown`, then `info registers`
Breakpoints never hit	`nokaslr`, and check the symbols match the running build
Painfully slow	You are on TCG. Expected on a foreign architecture
Works today, not tomorrow	Unversioned machine type or an unpinned QEMU; pin both

Check yourself

- ✓ Write, from memory, the command line for a Linux kernel plus initramfs with GDB attached.
- ✓ Give the two machine options that provide EL3 and EL2, and say which track needs each.
- ✓ Explain why the OP-TEE recipe uses `-bios` rather than `-kernel`.
- ✓ Explain what the second `-serial` is for in the OP-TEE recipe.
- ✓ Give the three flags that turn a hung CI job into a fast failure.
- ✓ Explain why `cache=unsafe` is correct in the overlay recipe and nowhere else.
- ✓ Name the first flag to reach for when a guest hangs, and why.
- ✓ Set up a 9p share and use it to iterate on a kernel module without repacking.
- ✓ Run a cross-compiled binary under user-mode emulation, with a debugger.

TRACK

OP-TEE & trusted applications

The open trusted OS, end to end: boot flow, core internals, the two GlobalPlatform APIs, and three projects ending in a firmware TPM with its own client.

25 chapters

What OP-TEE is and what it is for

OP-TEE is a small operating system that runs in the ARM secure world and hosts isolated applications. Its purpose is narrow and worth stating precisely: to keep a handful of operations correct even when the main operating system running beside it has been completely compromised.

The threat model, stated properly

```
WHAT OP-TEE DEFENDS AGAINST

  a compromised Linux kernel
  a compromised bootloader (after the point of measurement)
  a malicious or buggy application with root
  an attacker with a debugger on the normal world
  someone reading the eMMC out of a dead phone

WHAT IT DOES NOT DEFEND AGAINST

  physical attacks on the SoC - decapping, laser fault
    injection, microprobing
  a compromised EL3 (TF-A). EL3 is above OP-TEE.
  a compromised boot ROM or fused-in keys
  side channels it has not been hardened against -
    cache timing, power analysis, Spectre-class leaks
  a bug in your own trusted application. A TA is
    PRIVILEGED code; a bug there is worse than a bug in
    Linux, not better.

The last one is the important one. Moving code into the
secure world does not make it secure. It makes it
PRIVILEGED. If the code is wrong, you have moved the
vulnerability somewhere harder to patch and more valuable
to attack.
```

THE RULE FOR WHAT BELONGS IN A TA

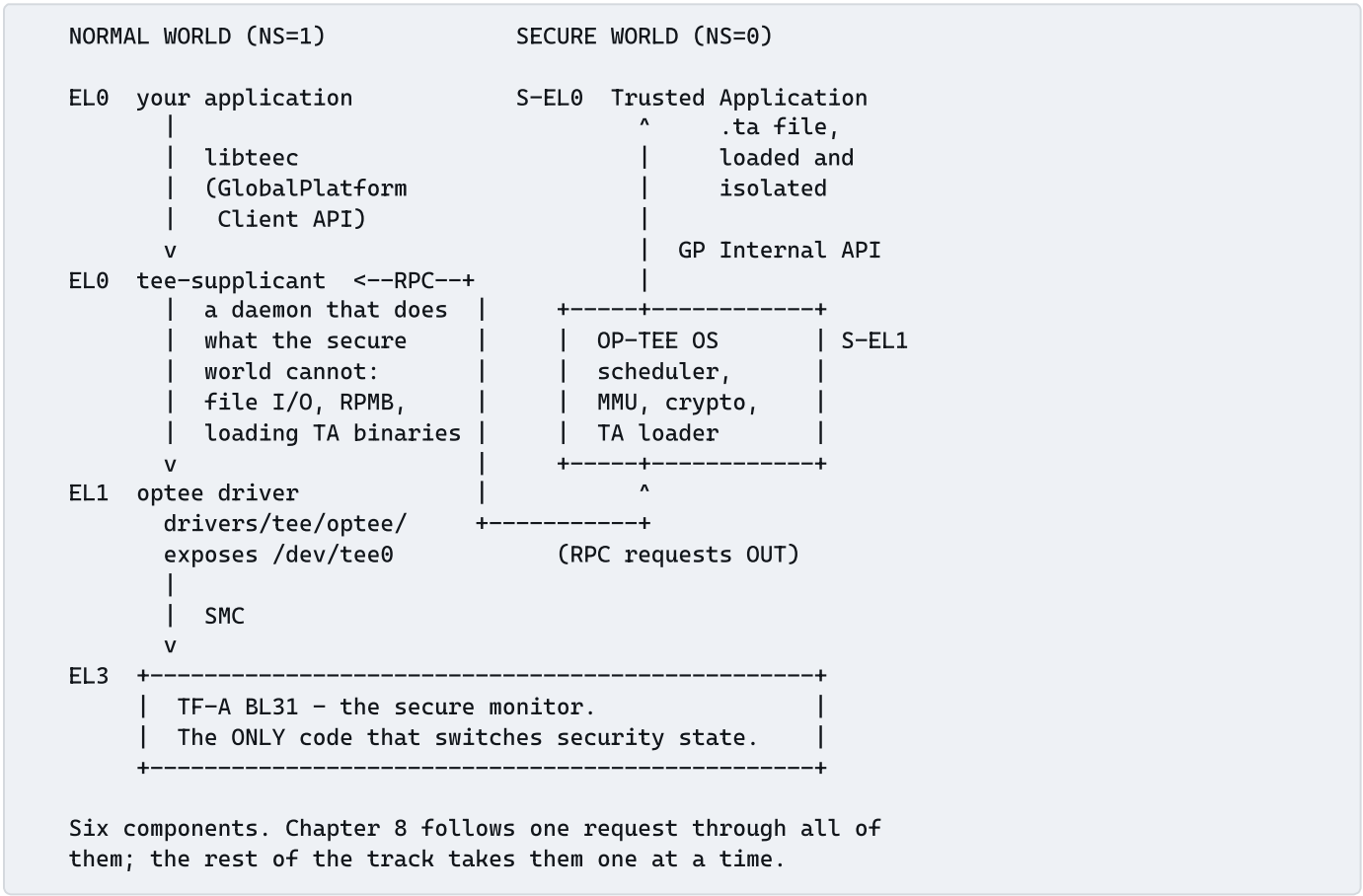
Put in the secure world only what must be protected from the normal world, and as little of it as possible. A key that must never leave: yes. The policy that decides when to use it: yes, if the policy is security-relevant. The parsing of an untrusted network protocol: absolutely not — parse in Linux and pass a validated, fixed-size structure across.

What it actually does for you

Use	What the secure world holds	Real product
Device key storage	A private key that never exists in normal-world memory	Android Keymint, device attestation
Secure boot continuation	Verification keys and the measurement chain	Every phone
Firmware TPM	A full TPM 2.0 in software, with its own NV storage	Windows on ARM, Microsoft's fTPM — chapter 22
Payment and biometrics	The fingerprint template and the matching decision	Samsung Pay, Android BiometricPrompt

Use	What the secure world holds	Real product
DRM	Content keys and the decrypt path	Widevine L1, PlayReady SL3000
Secure display and input	A framebuffer and touch panel Linux cannot see	Transaction confirmation screens
Anti-rollback	Monotonic counters in RPMB	Firmware version enforcement
Automotive / industrial	Key material for CAN authentication, immobilisers	SHE/HSM replacements

The stack, named



OP-TEE among its peers

TEE	Origin	API	Notes
OP-TEE	Linaro, open source	GlobalPlatform	The one to learn on. Upstream Linux driver, runs on QEMU, well documented
Trusty	Google / Android	Its own	C++ and Rust TAs. Keymint and Gatekeeper on Pixel
QSEE / QTEE	Qualcomm	Proprietary	The largest deployed base by volume
TEEGRIS	Samsung	Proprietary	Exynos, replacing Kinibi
Kinibi	Trustonic	Proprietary	Long history on Samsung
Hafnium	Arm, open source	FF-A	Not a TEE — an S-EL2 partition manager that *hosts* TEEs (virt chapter 19)

They differ in API and TA packaging. The *architecture* — a normal-world driver, a monitor, a trusted OS, isolated applications, and RPC back to a supplicant — is identical in all of them. Learn it once here and the proprietary ones become readable.

What this track builds

```
Chapters 2–4    get it RUNNING. Boot flow, build system,
                OP-TEE on QEMU with xtest passing.

Chapters 5–10   how it WORKS. Core architecture, secure
                memory, shared memory, the SMC ABI, RPC,
                the Linux driver.

Chapters 11–14  how you PROGRAM it. Both GlobalPlatform
                APIs, TA lifecycle, building and signing.

Chapters 15–20  what makes it USEFUL and SAFE. Secure
                storage, crypto, rollback protection,
                pseudo TAs, debugging, and how to write a
                TA that survives a hostile client.

Chapters 21–23  THREE PROJECTS:
                  21  a secure key store – a client and a
                      TA you write from nothing
                  22  a firmware TPM as a trusted
                      application – architecture and build
                  23  driving that fTPM from Linux, with
                      PCRs, sealing and attestation

Chapter 24      porting OP-TEE to hardware that is not QEMU.

Prerequisites: TrustZone from the isolation and platform
security track, chapters 8–13, and enough QEMU to
run a two-console machine (QEMU chapter 4).
```

> **READ `SEC-08` TO `SEC-13` FIRST IF TRUSTZONE IS NEW**
>
> This track assumes you know what the NS bit is, what EL3 does, and roughly what an SMC is. The isolation and platform security track covers that ground; chapters 8 through 13 there are the exact prerequisite, and chapter 12 is a compressed version of the stack diagram above.

Four things people get wrong

Belief	Reality
"Code in a TA is secure"	Code in a TA is *privileged*. A buffer overflow there is exploitable at S-EL0 with access to key material
"The TEE protects against physical attack"	Only as far as the SoC's own countermeasures go. TrustZone is not a secure element
"TAs are fast because they are close to the hardware"	Every invocation is a world switch and an SMC. TAs are for infrequent, valuable operations
"OP-TEE can be added to a product later"	It needs a secure memory carve-out, a TZASC configuration, a signing key infrastructure and a boot chain. It is a platform decision, not a feature

How it shows up

Symptom	Cause
A design review asks "why is this in the TEE?"	Usually the right question. If the answer is not "it must be protected from a compromised kernel", it belongs in Linux

Symptom	Cause
The TA is enormous	Something that should have been parsed or filtered in the normal world was moved wholesale
Throughput requirement cannot be met	Per-invocation world-switch cost; batch the work or move the bulk data path out
Vendor TEE has no documentation	Expected. The architecture is the same — map it onto the OP-TEE model in this chapter
Security team rejects the TA	Usually parameter validation (chapter 20) or a rollback gap (chapter 17)

Check yourself

- ✓ State what OP-TEE defends against and name three things it does not.
- ✓ Explain why "in the secure world" and "secure" are not the same claim.
- ✓ Give the rule for deciding whether code belongs in a TA.
- ✓ Name the six components of the stack and what each contributes.
- ✓ Give three shipping products built on a TEE and say what each protects.
- ✓ Name three TEEs other than OP-TEE and say what they have in common architecturally.
- ✓ Explain why Hafnium is not a TEE.
- ✓ Say why adding a TEE to an existing product is a platform decision.

The boot flow: TF-A, BL32 and the handover

OP-TEE does not boot itself. It is loaded, verified and started by ARM Trusted Firmware as one stage of a chain that begins in mask ROM and ends in Linux. Understanding which stage owns what — and which one you are debugging — is the difference between a productive afternoon and a wasted one.

The chain

```
POWER ON
   |
   v
+-----------------------------------------------------+
| BL1   boot ROM, or TF-A's BL1 stage.        EL3     |
|       In mask ROM on real silicon - IMMUTABLE.      |
|       Contains the root of trust: a hash of the public  |
|       key that signs everything after it.           |
|       Loads and VERIFIES BL2.                       |
+-----------------------------------------------------+
   |
   v
+-----------------------------------------------------+
| BL2   trusted boot firmware.                EL1 (S)   |
|       Initialises DRAM. Parses the FIP. Loads and       |
|       verifies BL31, BL32 and BL33 into their places.   |
|       Builds the parameter structures each will need.   |
|       Then hands control to BL31.                   |
+-----------------------------------------------------+
   |
   v
+-----------------------------------------------------+
| BL31  EL3 RUNTIME FIRMWARE - the secure monitor.  EL3   |
|       This one NEVER EXITS. It stays resident for the   |
|       life of the system, and every world switch, every |
|       PSCI call and every SMC goes through it.          |
|                                                         |
|       Its boot job: initialise the GIC, install the     |
|       SMC vector table, then run each "runtime service" |
|       initialiser - including the OP-TEE dispatcher     |
|       (opteed), which starts BL32.                      |
+-----------------------------------------------------+
   |
   +--> BL32 = OP-TEE OS                        S-EL1
   |         initialises: MMU, secure memory pools, the
   |         crypto library, the TA loader, its own
   |         console. Registers its entry vectors with
   |         BL31. Returns "I am ready".
   |
   v
+-----------------------------------------------------+
| BL33  NON-SECURE bootloader - U-Boot or UEFI.  EL2/EL1 |
|       The normal world starts here. From this moment    |
|       the secure world only runs when called.           |
+-----------------------------------------------------+
   |
   v
Linux -> optee driver probes -> /dev/tee0 appears
      -> tee-supplicant starts
      -> your client can open a session
```

> **BL31 IS THE ONE THAT STAYS**
> BL1 and BL2 run once and are gone. BL33 hands off to an OS. **BL31 and BL32 remain resident for ever.** Every SMC your client issues, for the rest of the machine's uptime, enters BL31 and is routed by it. When something goes wrong at the world boundary, BL31 is where it goes wrong.

The FIP: how the images travel together

```
# A Firmware Image Package is a simple container: a header, a
# table of contents keyed by UUID, and the images.

$ fiptool info fip.bin
Trusted Boot Firmware BL2: offset=0xB0, size=0x54D8, cmdline="--tb-fw"
EL3 Runtime Firmware BL31: offset=0x5588, size=0xC1C8, cmdline="--soc-fw"
Secure Payload BL32 (Trusted OS): offset=0x11750, size=0x3E000, cmdline="--tos-fw"
Secure Payload BL32 Extra1: offset=0x4F750, size=0x5A000, cmdline="--tos-fw-extra1"
Secure Payload BL32 Extra2: offset=0xA9750, size=0x21000, cmdline="--tos-fw-extra2"
Non-Trusted Firmware BL33: offset=0xCA750, size=0x9B240, cmdline="--nt-fw"

# The three BL32 entries are OP-TEE's three parts:
#   tos-fw         tee-header_v2.bin   - the header
#   tos-fw-extra1  tee-pager_v2.bin    - the pageable part
#   tos-fw-extra2  tee-pageable_v2.bin - the rest
# (see chapter 5 for why OP-TEE is split this way)

# Building one:
$ fiptool create \
    --tb-fw         bl2.bin \
    --soc-fw        bl31.bin \
    --tos-fw        tee-header_v2.bin \
    --tos-fw-extra1 tee-pager_v2.bin \
    --tos-fw-extra2 tee-pageable_v2.bin \
    --nt-fw         u-boot.bin \
    fip.bin

# Unpack one you were given, to see what a vendor shipped:
$ fiptool unpack fip.bin --out ./extracted/

# TF-A's own build does this for you:
$ make -C arm-trusted-firmware PLAT=qemu \
       BL32=$OPTEE/out/arm-plat-vexpress/core/tee-header_v2.bin \
       BL32_EXTRA1=.../tee-pager_v2.bin \
       BL32_EXTRA2=.../tee-pageable_v2.bin \
       BL33=$UBOOT/u-boot.bin \
       SPD=opteed all fip
```

SPD: how BL31 learns about OP-TEE

```
/* SPD = Secure Payload Dispatcher. A TF-A runtime service that
   knows how to talk to a particular trusted OS. Build TF-A
   with SPD=opteed and you get the OP-TEE one.               */

  services/spd/opteed/
      opteed_main.c     initialisation and the SMC handler
      opteed_common.c   context save/restore
      teesmc_opteed.h   the private ABI between TF-A and OP-TEE

/* At boot: */

  1. BL31 runs its runtime service initialisers.
  2. opteed_setup() finds BL32's entry point from the
     parameters BL2 left behind.
  3. It builds a secure execution context and ERETs into
     OP-TEE's entry point, at S-EL1.
  4. OP-TEE initialises, then calls back:
         SMC(TEESMC_OPTEED_RETURN_ENTRY_DONE, vectors_addr)
     handing BL31 a table of entry points:
         fast_smc_entry     std_smc_entry
         cpu_on_entry       cpu_off_entry
         cpu_resume_entry   cpu_suspend_entry
         fiq_entry          system_off_entry  system_reset_entry
  5. opteed records them and returns to BL2's caller, which
     proceeds to BL33.

/* At RUNTIME, every OP-TEE SMC from the normal world: */

     normal world executes SMC
        -> EL3 vector table
        -> BL31 decodes the function ID
        -> it is in the Trusted OS range -> opteed
        -> opteed SAVES the non-secure context
        -> restores the secure context
        -> ERETs to std_smc_entry in OP-TEE
        ... OP-TEE does the work ...
        -> OP-TEE executes SMC(TEESMC_OPTEED_RETURN_CALL_DONE)
        -> opteed restores the non-secure context
        -> ERET back to the normal world

/* Two SMCs and two full context switches per invocation. That
   is the floor on TA call latency, and it is why TAs are for
   valuable operations rather than hot loops.               */
```

Verified boot: what "chain of trust" means concretely

```
Each stage verifies the next BEFORE executing it.

BL1  holds ROTPK_HASH in ROM or eFuses  (immutable)
  |    reads the Trusted Key certificate from the FIP
  |    checks: SHA256(public key in cert) == ROTPK_HASH
  |    checks: cert signature verifies with that key
  |    -> now the trusted world key is trusted
  v
BL2  its hash is in a certificate signed by that key
  |    BL2 in turn verifies BL31, BL32 and BL33 the same way
  v
BL31, BL32, BL33

Turn it on:  make ... TRUSTED_BOARD_BOOT=1 GENERATE_COT=1 \
                      ARM_ROTPK_LOCATION=devel_rsa \
                      ROT_KEY=plat/arm/board/common/rotpk/arm_rotprivk_rsa.pem

MEASURED boot is different and complementary:
  each stage HASHES the next and EXTENDS the hash into a
  PCR (a TPM register, or OP-TEE's own measurement store)
  before running it. Nothing is blocked; instead the record
  of what ran can later be attested.

  Verified boot  : refuses to run unexpected software
  Measured boot  : runs it, but records what it was

Chapter 22's fTPM is where measured boot lands: the PCRs
that record the boot chain live inside a trusted
application.
```

> **DEVELOPMENT KEYS ARE DEVELOPMENT KEYS**
> Every OP-TEE and TF-A tree ships a test signing key, and every tutorial uses it — including this one. It is public. A device shipped with the default key can have arbitrary trusted applications loaded onto it by anyone. Key management is covered in chapter 14; the short version is that the private key belongs in an HSM and never in a repository.

Watching the boot

```
$ qemu-system-aarch64 -M virt,secure=on,virtualization=on \
    -cpu max -smp 2 -m 1057 -nographic \
    -bios bl1.bin \
    -semihosting-config enable=on,target=native \
    -serial mon:stdio -serial file:secure.log

NOTICE:  Booting Trusted Firmware
NOTICE:  BL1: v2.10.0(release)
NOTICE:  BL1: Booting BL2
NOTICE:  BL2: v2.10.0(release)
NOTICE:  BL1: Booting BL31
NOTICE:  BL31: v2.10.0(release)
I/TC: Reserved shared memory is enabled
I/TC: Dynamic shared memory is enabled
I/TC: Normal World virtualization support is disabled
I/TC: Asynchronous notifications are disabled
I/TC: OP-TEE version: 4.1.0
I/TC: Primary CPU switching to normal world boot
U-Boot 2024.01
...
[    2.114] optee: probing for conduit method.
[    2.115] optee: revision 4.1 (0e3e9c9d)
[    2.118] optee: dynamic shared memory is enabled
[    2.130] optee: initialized driver

# The "I/TC:" lines are OP-TEE's own console - Trusted Core.
# With two -serial they go to secure.log; with one they
# interleave with everything else, which is confusing but
# occasionally useful for ordering.

# Prefixes you will see all track long:
#   I/TC:  info,  trusted core (OP-TEE OS)
#   E/TC:  error, trusted core
#   D/TC:  debug, trusted core
#   I/TA:  from inside a trusted application
#   F/TC:  flow tracing (very verbose)
```

How it shows up

Symptom	Cause
Stops after `BL1: Booting BL2`	BL2 missing from the FIP, or DRAM init failed, or a verification failure with `TRUSTED_BOARD_BOOT`
No `I/TC:` lines at all	TF-A built without `SPD=opteed`, or BL32 not in the FIP
`BL31: Error initializing runtime service opteed_fast`	BL32 entry point wrong, or OP-TEE built for a different platform
OP-TEE prints then the machine hangs	Secure memory carve-out overlaps normal-world RAM (chapter 6)
Linux boots but `/dev/tee0` is absent	`CONFIG_OPTEE` off, or no `optee` node in the device tree, or OP-TEE never registered
`optee: api uid mismatch`	Linux driver and OP-TEE versions incompatible
Verified boot fails on a rebuilt image	The certificate chain was not regenerated — `GENERATE_COT=1`
Works with `-kernel`, not with `-bios`	Different worlds entirely; with `-bios` you supply everything (QEMU chapter 5)

Symptom	Cause
Secondary CPUs hang	PSCI handled by BL31, but OP-TEE's `cpu_on_entry` not registered

Check yourself

- ✓ Name the five boot stages, the exception level each runs at, and which two stay resident.
- ✓ Say what BL2 does that BL31 does not.
- ✓ Explain what a FIP is and list the entries an OP-TEE build puts in one.
- ✓ Explain what an SPD is and how BL31 learns OP-TEE's entry points.
- ✓ Count the SMCs and context switches in one TA invocation.
- ✓ Distinguish verified boot from measured boot, and say which one blocks.
- ✓ Explain the risk of shipping with the default signing key.
- ✓ Identify from a boot log whether OP-TEE initialised, and where its console output goes.
- ✓ Name three reasons `/dev/tee0` might be missing after a successful OP-TEE boot.

The source tree and the build system

An OP-TEE build is nine git repositories, three toolchains and a makefile that coordinates them. The `repo` manifest exists because assembling that by hand is unreasonable — but you should know what it assembled, because the day something breaks you will be building one component alone.

The nine repositories

Repository	What it is	Runs at
`optee_os`	The trusted OS itself. The core of everything in this track	S-EL1
`optee_client`	`libteec` and `tee-supplicant`	Normal world EL0
`optee_test`	`xtest` — the GlobalPlatform conformance suite plus OP-TEE's own tests	Both worlds
`optee_examples`	Small, complete CA+TA pairs. Copy these to start a project	Both worlds
`trusted-firmware-a`	BL1, BL2, BL31 — the monitor	EL3
`u-boot` or `edk2`	BL33, the normal-world bootloader	EL2/EL1
`linux`	The kernel, with `drivers/tee/optee/`	EL1
`buildroot`	Builds the normal-world root filesystem and packages the client tools	host
`build`	The makefiles that tie all of the above together, one `.mk` per platform	host

Getting the tree

```
$ sudo apt install -y \
    android-tools-adb android-tools-fastboot autoconf automake \
    bc bison build-essential ccache cscope curl device-tree-compiler \
    expect flex ftp-upload gdisk git libattr1-dev libcap-ng-dev \
    libfdt-dev libftdi-dev libglib2.0-dev libgmp3-dev libhidapi-dev \
    libmpc-dev libncurses5-dev libpixman-1-dev libslirp-dev \
    libssl-dev libtool make mtools netcat ninja-build python3-cryptography \
    python3-pip python3-pyelftools python3-serial python-is-python3 \
    rsync unzip uuid-dev xdg-utils xterm xz-utils zlib1g-dev

$ pip3 install --user pycryptodome pyelftools cryptography

# repo, Google's multi-repository tool
$ mkdir -p ~/bin
$ curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
$ chmod a+x ~/bin/repo && export PATH=~/bin:$PATH

$ mkdir -p ~/optee && cd ~/optee
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
$ repo sync -j$(nproc) --no-clone-bundle

# Other manifests, for other targets:
#   qemu_v8.xml     QEMU AArch64 virt   <-- this track
#   qemu.xml        QEMU AArch32
#   rpi3.xml        Raspberry Pi 3
#   hikey.xml  hikey960.xml  imx.xml  stm32mp1.xml  rockpi4.xml
#   default.xml     everything

# Pin a release, so the tree does not move under you:
$ repo init -u https://github.com/OP-TEE/manifest.git \
       -m qemu_v8.xml -b 4.1.0
$ repo sync -j$(nproc)

$ repo forall -c 'echo "$REPO_PATH $(git describe --always)"'
optee_os        4.1.0
optee_client    4.1.0
trusted-firmware-a  v2.10.0
linux           v6.6
...
```

Building

```
$ cd ~/optee/build

# 1. Toolchains - downloads aarch64 and arm cross compilers
$ make toolchains -j2

# 2. Everything. First build: 20-60 minutes.
$ make -j$(nproc)

# 3. Run it. Opens three windows.
$ make run
# or, if you have no X display:
$ make run-only CFG_TEE_CORE_LOG_LEVEL=2

# In the QEMU console that appears:
(qemu) c

# In the "Normal World" window, log in as root, then:
# ls /dev/tee*
/dev/tee0  /dev/teepriv0
# xtest -t regression 1001
# optee_example_hello_world
```

Make target	Does
`make`	Build everything and assemble the images
`make run`	Build if needed, then launch QEMU with three consoles
`make run-only`	Launch without rebuilding
`make optee-os`	Just the trusted OS
`make optee-client`	Just libteec and tee-supplicant
`make optee-examples`	Just the example CA/TA pairs
`make buildroot`	Repack the root filesystem — needed after changing anything that lands in it
`make arm-tf`	Just TF-A
`make linux`	Just the kernel
`make optee-os-clean`	Clean one component
`make -n run`	Print the QEMU command line without running it — *very* useful

> **THE REBUILD TRAP THAT COSTS EVERYONE AN HOUR**
>
> Changing a TA or a client and running `make optee-examples` is not enough. The binaries live in the root filesystem image, which must be repacked:

```
# The correct incremental cycle after editing a TA or CA:
$ make optee-examples
$ make buildroot          # <-- repacks rootfs.cpio.gz
$ make run-only

# Or in one line, which is what you should alias:
$ make optee-examples buildroot run-only

# Same for the trusted OS itself:
$ make optee-os
$ make run-only           # OP-TEE is in the FIP, not the
                          # rootfs, so no buildroot needed

# Prove which binary is actually running - the timestamp is in
# the boot log:
I/TC: OP-TEE version: 4.1.0-12-gabc1234 (gcc version 12.2.0)
      #1 Fri Aug 22 10:14:02 UTC 2026 aarch64
```

The configuration flags that matter

```
# OP-TEE is configured with CFG_ variables, passed on the make
# command line or set in the platform's conf.mk.

$ make CFG_TEE_CORE_LOG_LEVEL=3 \
       CFG_TEE_TA_LOG_LEVEL=4 \
       CFG_CORE_ASLR=n \
       CFG_UNWIND=y \
       DEBUG=1

#   CFG_TEE_CORE_LOG_LEVEL   0 none 1 error 2 info 3 debug 4 flow
#   CFG_TEE_TA_LOG_LEVEL     the same, for TAs
#   CFG_CORE_ASLR=n          turn off ASLR so addresses in a
#                            panic dump match your ELF
#   CFG_UNWIND=y             stack traces on panic
#   DEBUG=1                  -O0 and assertions
#   CFG_CORE_SANITIZE_UNDEFINED=y   UBSan inside OP-TEE
#   CFG_CORE_SANITIZE_KADDRESS=y    KASAN inside OP-TEE

# Feature flags you will meet:
#   CFG_RPMB_FS=y            secure storage in RPMB (ch. 15)
#   CFG_RPMB_TESTKEY=y       use a test RPMB key - DEV ONLY
#   CFG_REE_FS=y             secure storage via the normal world
#   CFG_WITH_PAGER=y         demand-paged OP-TEE (ch. 5)
#   CFG_WITH_LPAE=y          large physical addresses
#   CFG_NS_VIRTUALIZATION=y  OP-TEE aware of normal-world VMs
#   CFG_SECURE_DATA_PATH=y   secure buffers for media
#   CFG_TA_ASLR=y            ASLR inside TAs
#   CFG_ATTESTATION_PTA=y    remote attestation pseudo-TA

# See everything a platform defines:
$ make -C optee_os PLATFORM=vexpress-qemu_armv8a \
       CFG_TEE_CORE_LOG_LEVEL=2 -n | head -40
$ cat optee_os/core/arch/arm/plat-vexpress/conf.mk
```

Where things end up

```
~/optee/
  build/                          the makefiles you invoke
  optee_os/
    core/                         the trusted OS
      arch/arm/plat-vexpress/     the QEMU platform
      arch/arm/kernel/            entry, exceptions, boot
      arch/arm/mm/                the MMU and memory maps
      kernel/                     scheduler, TA loading, threads
      drivers/                    OP-TEE's own device drivers
      pta/                        pseudo TAs (ch. 18)
      crypto/                     the crypto layer
      tee/                        the GP Internal API
    ldelf/                        the TA loader/linker
    ta/                           the TA development kit
    lib/libutee/                  the GP Internal API library
    out/arm-plat-vexpress/
      core/tee.elf                <-- the OP-TEE binary, with
                                      symbols. Point GDB here.
      core/tee-header_v2.bin      the FIP pieces
      core/tee-pager_v2.bin
      core/tee-pageable_v2.bin
      export-ta_arm64/            <-- THE TA DEV KIT. Your own
                                      TA builds against this.
  optee_client/out/export/
      usr/lib/libteec.so
      usr/sbin/tee-supplicant
      usr/include/tee_client_api.h
  optee_examples/                 copy one of these to start
  out/                            assembled images
      bin/                        Image, rootfs.cpio.gz
  out-br/                         buildroot's tree
      target/lib/optee_armtz/     <-- WHERE .ta FILES GO
                                      <uuid>.ta, loaded on
                                      demand by tee-supplicant

# The two paths to remember:
#   optee_os/out/arm-plat-vexpress/export-ta_arm64/
#       the dev kit your TA Makefile includes
#   /lib/optee_armtz/ on the TARGET
#       where a signed .ta must be installed to be found
```

Building one component alone

```
# When you are iterating on the trusted OS, the top-level make
# is slow. Go direct:

$ cd ~/optee/optee_os
$ make -j$(nproc) \
    PLATFORM=vexpress-qemu_armv8a \
    CFG_ARM64_core=y \
    CROSS_COMPILE=aarch64-linux-gnu- \
    CROSS_COMPILE_core=aarch64-linux-gnu- \
    CROSS_COMPILE_ta_arm64=aarch64-linux-gnu- \
    CFG_TEE_CORE_LOG_LEVEL=3 \
    DEBUG=1

# The client library alone:
$ cd ~/optee/optee_client
$ cmake -B build -DCMAKE_INSTALL_PREFIX=./out/export \
        -DCMAKE_C_COMPILER=aarch64-linux-gnu-gcc
$ cmake --build build && cmake --install build

# A TA on its own, against the dev kit (ch. 14):
$ export TA_DEV_KIT_DIR=~/optee/optee_os/out/arm-plat-vexpress/export-ta_arm64
$ export CROSS_COMPILE=aarch64-linux-gnu-
$ make -C my_ta

# Find every CFG_ that affects the build:
$ grep -rn "CFG_MY_FEATURE" optee_os/ | head
```

How it shows up

Symptom	Cause
`repo sync` fails	Network, or a manifest branch that no longer exists. Pin a release tag
Build fails on a Python module	`pycryptodome`, `pyelftools` or `cryptography` missing from `pip3`
Changes to a TA have no effect	`make buildroot` not run — the old `.ta` is still in the image
Changes to OP-TEE core have no effect	Stale FIP; run the top-level `make` or rebuild the FIP
`make run` opens nothing	No X display; use `make run-only` and read `make -n run` for the command line
TA build cannot find headers	`TA_DEV_KIT_DIR` unset or pointing at the wrong architecture
Two toolchains, wrong one used	Three separate `CROSS_COMPILE*` variables — core, TA arm32, TA arm64
Panic addresses do not match the ELF	ASLR — rebuild with `CFG_CORE_ASLR=n`
Out of disk	A full tree with objects is 20–30 GB

Check yourself

- ✓ Name the nine repositories and what each contributes.
- ✓ Explain what `repo` does and why a manifest is pinned to a tag.
- ✓ Give the correct incremental build cycle after editing a TA, and say why.
- ✓ Give the cycle after editing the trusted OS, and say why it differs.
- ✓ Name five `CFG_` flags and what each changes.

- ✓ Say where a signed `.ta` must be installed on the target.
- ✓ Say what `TA_DEV_KIT_DIR` points at and why a TA build needs it.
- ✓ Build `optee_os` alone with debug logging enabled.
- ✓ Determine from a boot log exactly which build is running.

Bringing OP-TEE up on QEMU

This is the chapter to get right before any other. A working QEMU setup with two consoles — one per world — and a passing `xtest` is the foundation for everything that follows, and every later failure is much easier to diagnose when you have seen the system work once.

The fast path

```
$ cd ~/optee/build
$ make -j$(nproc)
$ make run

# Three windows appear:
#   1. QEMU's own monitor
#   2. "Normal World" - Linux
#   3. "Secure World" - OP-TEE's console
#
# In window 1:
(qemu) c

# In window 2, after Linux boots (login: root, no password):
buildroot login: root
# ls /dev/tee*
/dev/tee0  /dev/teepriv0
# ps | grep supplicant
  123 root     tee-supplicant
# optee_example_hello_world
Invoking TA to increment 42
TA incremented value to 43

# WATCH WINDOW 3 while that runs. You will see the TA load and
# print. That split - one console per world - is the single
# most useful property of this setup, and the reason to invest
# in getting it working.

# The full conformance suite:
# xtest
Test ID: ...
...
23807 subtests of which 0 failed
78 test cases of which 0 failed
0 test case was skipped
TEE test application done!

# A quick subset, when you only want a sanity check:
# xtest -l 0
# xtest -t regression 1001 1004 1013
```

Without a graphical display

```
# make run needs X for the three xterms. On a headless machine
# or over SSH, drive QEMU yourself.

$ cd ~/optee/build
$ make -n run-only | tr ' ' '\n' | grep -v '^$' | head -40
# ...prints the exact qemu invocation. Adapt it:

$ cd ~/optee
$ qemu-system-aarch64 \
    -nographic \
    -serial mon:stdio \
    -serial file:secure.log \
    -smp 2 \
    -machine virt,secure=on,mte=off,gic-version=3,virtualization=false \
    -cpu max,sve=off \
    -d unimp -semihosting-config enable=on,target=native \
    -m 1057 \
    -bios bl1.bin \
    -initrd rootfs.cpio.gz \
    -kernel Image -no-acpi \
    -append 'console=ttyAMA0,38400 keep_bootcon root=/dev/vda2'

# The two -serial lines are the point:
#   first  -> ttyAMA0 -> the NORMAL world, on your terminal
#   second -> the SECURE world's console -> secure.log
#
# Watch the secure console live in another terminal:
$ tail -f secure.log

# To have both on one screen without X, use tmux:
$ tmux new-session -d 'cd ~/optee/build && make run-only'
$ tmux split-window -h 'tail -f ~/optee/secure.log'
$ tmux attach
```

> **TWO CONSOLES OR NOTHING**
>
> OP-TEE reports TA panics, parameter errors, allocation failures and its own assertions on the secure console — never on the normal-world one. A client that returns `TEEC_ERROR_GENERIC` tells you nothing; the secure console tells you exactly which assertion failed and where. If you set up only one thing from this chapter, set up the second serial port.

Reading the boot log

```
# Secure world (secure.log):
I/TC:
I/TC: Non-secure external DT found
I/TC: Reserved shared memory is enabled
I/TC: Dynamic shared memory is enabled
I/TC: Normal World virtualization support is disabled
I/TC: Asynchronous notifications are disabled
I/TC: OP-TEE version: 4.1.0 (gcc version 12.3.0) #1 ... aarch64
I/TC: WARNING: This OP-TEE configuration might be insecure!
I/TC: WARNING: Please check https://optee.readthedocs.io/...
I/TC: Primary CPU initializing
I/TC: Primary CPU switching to normal world boot

# That WARNING is not noise: it means the build has development
# keys, test RPMB keys, or debug features enabled. Every
# tutorial build shows it. A product build must not.

# Normal world:
[    1.883] optee: probing for conduit method.
[    1.884] optee: revision 4.1 (0e3e9c9d)
[    1.891] optee: dynamic shared memory is enabled
[    1.904] optee: initialized driver

# Four lines, four things confirmed:
#   "probing for conduit method"  the DT has an optee node
#   "revision 4.1"                the SMC handshake worked -
#                                 Linux talked to OP-TEE
#   "dynamic shared memory"       buffer registration will work
#   "initialized driver"          /dev/tee0 now exists

# If any of the four is missing, stop and fix it before writing
# a client.
```

The device tree node

```
/* Linux finds OP-TEE through the device tree. On QEMU the node
   is added by TF-A/QEMU; on real hardware you write it.     */

    firmware {
        optee {
            compatible = "linaro,optee-tz";
            method = "smc";
        };
    };

# Confirm it is there, from inside the guest:
$ ls /proc/device-tree/firmware/optee/
compatible  method  name
$ cat /proc/device-tree/firmware/optee/method
smc

# method = "smc"  bare metal - the driver issues SMC directly
# method = "hvc"  running under a hypervisor that forwards

# Kernel config required:
#   CONFIG_TEE=y
#   CONFIG_OPTEE=y
#   CONFIG_ARM_PSCI_FW=y
$ zcat /proc/config.gz | grep -E 'CONFIG_(TEE|OPTEE)='
CONFIG_TEE=y
CONFIG_OPTEE=y
```

tee-supplicant

```
# Without it, sessions fail with TEEC_ERROR_ITEM_NOT_FOUND,
# because nothing can load the .ta file from the filesystem.
# It is a normal-world daemon (chapter 9 explains why it must
# exist at all).

# Is it running?
$ ps aux | grep [t]ee-supplicant
root  123  tee-supplicant /dev/teepriv0

# Start it by hand:
$ tee-supplicant -d          # daemonise
$ tee-supplicant -f          # foreground, so you see its errors

# Where it looks for trusted applications:
$ ls /lib/optee_armtz/
8aaaf200-2450-11e4-abe2-0002a5d5c51b.ta   # hello_world
f4e750bb-1437-4fbf-8785-8d3580c34994.ta   # aes
...

# The filename IS the UUID. A TA whose UUID does not match its
# filename will never be found.

# systemd unit, on a real distribution:
$ systemctl status tee-supplicant
$ systemctl enable --now tee-supplicant
```

Your first client, from nothing

```
// hello.c - the smallest complete client. Links against
// libteec, talks to the hello_world TA that ships with
// optee_examples.
#include <stdio.h>
#include <string.h>
#include <tee_client_api.h>

#define TA_HELLO_WORLD_UUID \
    { 0x8aaaf200, 0x2450, 0x11e4, \
      { 0xab, 0xe2, 0x00, 0x02, 0xa5, 0xd5, 0xc5, 0x1b} }
#define TA_HELLO_WORLD_CMD_INC_VALUE 0

int main(void)
{
        TEEC_Context   ctx;
        TEEC_Session   sess;
        TEEC_Operation op;
        TEEC_UUID      uuid = TA_HELLO_WORLD_UUID;
        uint32_t       err_origin;
        TEEC_Result    res;

        res = TEEC_InitializeContext(NULL, &ctx);
        if (res != TEEC_SUCCESS) {
                printf("InitializeContext failed 0x%x\n", res);
                return 1;
        }

        res = TEEC_OpenSession(&ctx, &sess, &uuid,
                               TEEC_LOGIN_PUBLIC, NULL, NULL,
                               &err_origin);
        if (res != TEEC_SUCCESS) {
                printf("OpenSession failed 0x%x origin 0x%x\n",
                       res, err_origin);
                TEEC_FinalizeContext(&ctx);
                return 1;
        }

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INOUT,
                                         TEEC_NONE,
                                         TEEC_NONE,
                                         TEEC_NONE);
        op.params[0].value.a = 42;

        res = TEEC_InvokeCommand(&sess, TA_HELLO_WORLD_CMD_INC_VALUE,
                                 &op, &err_origin);
        if (res != TEEC_SUCCESS)
                printf("InvokeCommand failed 0x%x origin 0x%x\n",
                       res, err_origin);
        else
                printf("TA returned %u\n", op.params[0].value.a);

        TEEC_CloseSession(&sess);
        TEEC_FinalizeContext(&ctx);
        return 0;
}
```

```
$ aarch64-linux-gnu-gcc -o hello hello.c \
    -I ~/optee/optee_client/out/export/usr/include \
    -L ~/optee/optee_client/out/export/usr/lib \
    -lteec

# Get it into the guest. Easiest: put it in the rootfs and
# repack (chapter 3), or use the shared 9p mount:
$ cp hello ~/optee/out-br/target/root/
$ cd ~/optee/build && make buildroot run-only

# In the guest:
# ./hello
TA returned 43

# And the secure console shows:
D/TC:0 0 tee_ta_init_user_ta_session:1035 Lookup user TA ELF
       8aaaf200-2450-11e4-abe2-0002a5d5c51b (Secure Storage TA)
I/TA: has been called
I/TA: Got value: 42 from NW
I/TA: Increase value to: 43
```

The bring-up checklist

```
Work through these in order. Each one depends on the last.

[ ] 1. TF-A prints "BL31: v2.x"
       -> if not: FIP or build problem (ch. 2)

[ ] 2. Secure console shows "I/TC: OP-TEE version:"
       -> if not: SPD=opteed missing, or BL32 not in the FIP

[ ] 3. Secure console shows "switching to normal world boot"
       -> if not: OP-TEE init failed. Read the lines above it.

[ ] 4. Linux boots to a shell
       -> ordinary kernel debugging (QEMU ch. 5)

[ ] 5. dmesg | grep optee shows all four driver lines
       -> if not: device tree node, or CONFIG_OPTEE

[ ] 6. /dev/tee0 exists
       -> follows from 5

[ ] 7. tee-supplicant is running
       -> if not: start it, check /lib/optee_armtz exists

[ ] 8. optee_example_hello_world works
       -> if not: the .ta is missing, or signed with the
          wrong key

[ ] 9. xtest -l 0 passes
       -> broad sanity

[ ] 10. xtest passes completely
       -> you have a working system. STOP HERE and commit
          the exact versions you used.

Do not start writing your own TA before step 10 passes. A
failure at step 11 with steps 1-10 unverified is
unattributable, and you will spend the afternoon looking in
the wrong place.
```

How it shows up

Symptom	Cause
No secure-world output anywhere	Only one `-serial`; add a second
`make run` does nothing	No X display — use `make run-only` and `make -n run`
QEMU sits doing nothing	It starts paused; type `c` in the monitor
`/dev/tee0` missing	Device tree node absent, `CONFIG_OPTEE` off, or OP-TEE never initialised
`TEEC_ERROR_ITEM_NOT_FOUND` (0xFFFF0008)	TA not in `/lib/optee_armtz`, or `tee-supplicant` not running
`TEEC_ERROR_SECURITY` (0xFFFF000F) on load	TA signed with a key OP-TEE does not trust
`TEEC_ERROR_COMMUNICATION`	`/dev/tee0` exists but the SMC path is broken — check the secure console
Client links but fails at runtime with a missing library	`libteec.so` not in the rootfs; static-link or copy it

Symptom	Cause
Everything works, then stops after a rebuild	`make buildroot` not run — old binaries in the image
`WARNING: This OP-TEE configuration might be insecure`	Expected in development. Never acceptable in a product

Check yourself

- ✓ Get `xtest` passing on QEMU and record the exact versions.
- ✓ Explain what each of the three `make run` windows shows.
- ✓ Set up two serial ports without X and explain why the second one matters.
- ✓ Name the four `optee:` lines in `dmesg` and what each confirms.
- ✓ Write the device tree node OP-TEE needs and say what `method` selects.
- ✓ Explain what `tee-supplicant` does and what fails without it.
- ✓ Say where `.ta` files live and how their filename relates to the UUID.
- ✓ Write and run a client that opens a session and invokes one command.
- ✓ Work through the ten-step bring-up checklist and say why the order matters.
- ✓ Explain what the "might be insecure" warning is telling you.

OP-TEE OS architecture

OP-TEE is a small preemptible kernel with a fixed pool of threads, its own MMU regime, a demand pager, and a strict rule that it never runs unless the normal world asked it to. Those four properties explain almost every behaviour you will observe, including the deadlock in the last section.

The shape of it

```
S-EL0    TA        TA        TA
         |         |         |
         |  SVC (the GP Internal API is a syscall
         |       interface, exactly like Linux's)
         v
+-----------------------------------------------------+
|  OP-TEE CORE                                S-EL1   |
|                                                     |
|   thread pool     N threads, N fixed at build time  |
|   TA manager      load, verify, map, session state  |
|   MMU             one address space per TA          |
|   pager           optional demand paging of core    |
|   crypto          libtomcrypt / mbedTLS, plus       |
|                   hardware accelerator drivers      |
|   drivers         GIC, timer, RNG, RPMB, i2c...     |
|   pseudo TAs      services built into the core      |
|   RPC             asking the normal world for help  |
+-----------------------------------------------------+
         |
         |  SMC
         v
EL3      TF-A BL31

Things OP-TEE deliberately does NOT have:

  no scheduler of its own in the usual sense - the NORMAL
    world schedules it, by choosing when to make an SMC
  no interrupts it services on its own behalf, except its
    own timer and secure interrupts
  no dynamic memory beyond fixed pools
  no filesystem - it asks the normal world (chapter 9)
  no network, no USB, no display, unless a platform adds one

It is closer to a very careful library with an MMU than to
an operating system, and that is deliberate: a small
trusted computing base is the entire product.
```

The thread pool

```
/* OP-TEE has a FIXED number of threads, set at build time:   */

    CFG_NUM_THREADS = 8      /* typical; sometimes 2 or 4      */

/* One thread is occupied for the ENTIRE duration of a normal-
   world call. From the SMC that enters OP-TEE until the SMC
   that returns, that thread belongs to that caller.          */

  normal world calls in       -> a thread is allocated
     TA runs
     TA does secure storage   -> RPC out to tee-supplicant.
                                 THE THREAD IS STILL HELD - it
                                 is suspended waiting for the
                                 normal world to answer.
     TA finishes
  return to normal world      -> thread freed

/* Consequences:

   1. At most CFG_NUM_THREADS concurrent TA invocations,
      system-wide. The 9th caller gets
      TEEC_ERROR_BUSY / OPTEE_SMC_RETURN_ETHREAD_LIMIT and
      libteec retries.

   2. A TA that blocks holds a thread. A TA that blocks
      FOR EVER holds it for ever.

   3. Thread stacks are statically allocated:
        CFG_STACK_THREAD_SIZE   default 8192 bytes
        CFG_STACK_TMP_SIZE      the temporary/abort stack
      8 threads x 8 KB is 64 KB of your secure memory budget,
      before any TA runs.

   4. TA stack size is separate and per-TA:
        TA_STACK_SIZE in the TA's user_ta_header_defines.h    */
```

THE THREAD-EXHAUSTION DEADLOCK

All `CFG_NUM_THREADS` threads are inside OP-TEE, each waiting on an RPC to `tee-supplicant`. `tee-supplicant` is a normal-world process; to service any of them it must be scheduled by Linux. If `tee-supplicant` itself needs to enter OP-TEE — or if the Linux thread that would run it is itself blocked in an OP-TEE call — nothing can make progress. The system hangs with no error anywhere.

Avoid it by: never calling into the TEE from a thread that services the TEE; keeping RPC use out of hot paths; and raising `CFG_NUM_THREADS` so transient bursts do not saturate the pool. It is a design constraint, not a bug to be fixed later.

How a call enters

```
/* core/arch/arm/kernel/thread_spmc.c and entry_a64.S       */

  BL31 ERETs into OP-TEE's vector table at one of:

    fast_smc_entry        a "fast" call - runs to completion
                          with interrupts masked, on the
                          temporary stack, no thread allocated.
                          Used for: version queries, memory
                          config, cache maintenance requests.

    std_smc_entry         a "standard" call - allocates a
                          THREAD, can be preempted by a
                          non-secure interrupt, can make RPCs.
                          Used for: everything that runs a TA.

    fiq_entry             a secure interrupt arrived while the
                          normal world was running

    cpu_on/off/suspend/resume_entry
                          PSCI events forwarded by BL31 so
                          OP-TEE can save and restore per-CPU
                          state

/* The distinction matters. A fast call cannot:
     - allocate memory
     - call a TA
     - make an RPC
     - be interrupted
   It must be short and self-contained. Trying to do real work
   in a fast call is a classic porting mistake and produces
   hangs with interrupts masked.                            */

/* Foreign (non-secure) interrupt handling: while a standard
   call is running, an interrupt destined for the NORMAL world
   causes OP-TEE to save the thread's state and return to the
   normal world with OPTEE_SMC_RETURN_RPC_FOREIGN_INTR. Linux
   services the interrupt, then RESUMES the same thread. The TA
   never notices. This is why a long-running TA does not block
   Linux's timekeeping.                                      */
```

Memory layout

```
TEE_RAM  - OP-TEE's own memory. Secure, and never mapped
           into any TA.
   TEE_RAM_RX   code, read+execute
   TEE_RAM_RW   data, heap, thread stacks
   TEE_RAM_RO

TA_RAM   - the pool TA memory is allocated from. Each TA
           gets its own mapping; TAs cannot see each other.

SHM      - non-secure shared memory. The ONLY region both
           worlds can touch. Chapter 7.

+-----------------------------------------------+
|  TEE_RAM      core code + data      SECURE     |
+-----------------------------------------------+
|  TA_RAM       TA code + data        SECURE     |
+-----------------------------------------------+
|  SHM          shared buffers        NON-SECURE |
+-----------------------------------------------+

Each TA runs at S-EL0 with its own translation tables. A TA
cannot:
  read another TA's memory
  read TEE_RAM
  read normal-world memory except a buffer explicitly
    passed to it
  execute anything it did not bring

The isolation between TAs is exactly the isolation between
Linux processes, implemented by the same hardware. That is
the point of running TAs at S-EL0 rather than linking them
into the core.
```

The pager

```
/* CFG_WITH_PAGER=y makes OP-TEE demand-page its own code.   */

  Why: secure on-chip SRAM is small - often 128-256 KB - and
  OP-TEE plus crypto plus drivers does not fit. But external
  DRAM is not trustworthy: an attacker with bus access could
  read or modify it.

  Solution: keep a small resident part in SRAM, and page the
  rest in from DRAM, where it is stored ENCRYPTED and
  AUTHENTICATED.

     +--------------------------+
     |  SRAM (secure, on-chip)  |
     |    unpaged core          |
     |    the pager itself      |
     |    page tables           |
     |    a pool of page frames |
     +--------------------------+
              ^  page fault
              |  decrypt + verify + map
              v
     +--------------------------+
     |  DRAM                    |
     |    encrypted pages, each |
     |    with an authentication|
     |    tag                   |
     +--------------------------+

  Cost: a page fault inside OP-TEE now involves AES and a MAC
  check. Measurable, and worth it when the alternative is not
  fitting.

  On QEMU there is plenty of secure DRAM, so the pager is
  usually off. On real SoCs with small secure SRAM it is
  essential. tee-pager_v2.bin and tee-pageable_v2.bin in the
  FIP (chapter 2) are exactly this split.                   */
```

TAs, pseudo TAs and early TAs

	User-mode TA	Pseudo TA	Early TA
Runs at	S-EL0	**S-EL1, inside the core**	S-EL0
Isolated from the core	Yes, by the MMU	**No**	Yes
Where it lives	`/lib/optee_armtz/<uuid>.ta`	Linked into `tee.elf`	Linked into `tee.elf`, extracted at boot
Loaded by	`tee-supplicant`, on demand	Nothing — always there	The core, at boot
Needs the normal world to load	Yes	No	No
Can be updated separately	Yes	No — rebuild OP-TEE	No
A bug means	A crashed TA	**A compromised trusted OS**	A crashed TA

	User-mode TA	Pseudo TA	Early TA
Use when	Almost always	You need core services or must run before the normal world can help	You must work before `tee-supplicant` exists

Default to a user-mode TA. A pseudo TA runs with the trusted OS's full privilege and no memory isolation; the only good reasons are needing a core API that is not exposed to S-EL0, or needing to run before the normal world can load anything. Chapter 18 covers the cases where it is genuinely right.

Reading the source

```
# The files that repay reading, in order:

optee_os/core/arch/arm/kernel/
    entry_a64.S            the vector table. Where every call
                           arrives. Read this first.
    thread.c               the thread pool, suspend/resume
    thread_spmc.c          the SMC-side protocol
    boot.c                 initialisation order

optee_os/core/kernel/
    tee_ta_manager.c       sessions, TA lifecycle, the locking
    ree_fs_ta.c            loading a TA from the normal world
    pseudo_ta.c
    scall.c                the S-EL0 syscall dispatch - the
                           other side of the GP Internal API

optee_os/core/mm/
    core_mmu.c  core_mmu_lpae.c   the core's own mappings
    vm.c                   per-TA address spaces
    pgt_cache.c            page table allocation
    tee_pager.c            the pager

optee_os/core/tee/
    tee_svc*.c             the syscall implementations
    tee_fs*.c              secure storage
    tee_rpmb_fs.c          RPMB

optee_os/ldelf/            the loader that maps a TA - it runs
                           in the TA's own address space,
                           which is an unusual and elegant
                           design

# Find where a GP function is implemented:
$ grep -rn "TEE_AllocateOperation" optee_os/lib/libutee/
$ grep -rn "syscall_cryp_obj_alloc" optee_os/core/
```

How it shows up

Symptom	Cause
Calls fail with `TEEC_ERROR_BUSY` under load	Thread pool exhausted; raise `CFG_NUM_THREADS` or reduce concurrency
Whole system hangs, no error	Thread-exhaustion deadlock — every thread waiting on an RPC that cannot be serviced
TA crashes with a stack dump on the secure console	`TA_STACK_SIZE` too small, or a genuine bug

Symptom	Cause
Core panics with a stack overflow	`CFG_STACK_THREAD_SIZE` too small for deep crypto call chains
Everything hangs with interrupts masked	Real work attempted in a fast SMC
Out of secure memory as TAs are added	Fixed pools; see chapter 6
A pseudo TA bug takes down the whole TEE	By design — it runs at S-EL1 with no isolation
A TA works, then fails only when another TA is busy	Shared thread pool
Paged build is slow	Every fault decrypts and authenticates a page

Check yourself

- ✓ Name the components of the OP-TEE core and three things it deliberately lacks.
- ✓ Explain the fixed thread pool and what happens to the ninth concurrent caller.
- ✓ Explain precisely how the thread-exhaustion deadlock occurs and how to avoid it.
- ✓ Distinguish a fast SMC from a standard one and list what a fast call cannot do.
- ✓ Explain what happens when a normal-world interrupt arrives during a TA invocation.
- ✓ Name the three memory regions and say who can access each.
- ✓ Explain why the pager exists and what it costs.
- ✓ Compare user-mode TAs, pseudo TAs and early TAs across privilege, isolation and update.
- ✓ Give two legitimate reasons to write a pseudo TA.
- ✓ Locate, in the source tree, where an SMC first arrives and where a GP syscall is dispatched.

Secure memory: carve-outs, TZASC and pools

The secure world's memory is secure because a hardware filter in the interconnect refuses non-secure transactions to it. Configuring that filter correctly — and making sure the normal world never believes it owns the same addresses — is the single most common cause of a board that boots on QEMU and dies on real silicon.

What makes memory secure

```
Every bus transaction on an ARM SoC carries an extra signal:
AxPROT[1], the NS bit. 0 = secure, 1 = non-secure.

  CPU at S-EL1  -> issues transactions with NS=0
  CPU at EL1    -> NS=1
  a DMA master  -> whatever the SMMU/interconnect assigns it

A filter in the path enforces a region policy:

  TZASC / TZC-400   for DRAM
  TZPC              for peripherals
  vendor XPU / RPU  the same idea, different name
                    (see the isolation track, ch. 17-20)

+--------+     +---------------------+     +-----------+
|  CPU   |---->|  TZC-400            |---->|   DRAM    |
| NS bit |     |  region 0: 0-1GB    |     |           |
+--------+     |    NS read/write OK |     +-----------+
               |  region 1: 1GB-1.5GB|
               |    SECURE ONLY      |
               |    NS access ->     |
               |    DECERR or RAZ/WI |
               +---------------------+

A non-secure read of a secure region returns zeros or an
external abort, depending on how the filter is configured.
It NEVER returns the data.

Crucially: this is enforced in the INTERCONNECT, not in the
CPU. So it also stops a DMA-capable peripheral that the
normal world programmed. That is the whole reason it exists
rather than relying on the MMU.
```

WHO PROGRAMS THE FILTER

Early secure firmware — BL2 or BL31 — configures the TZASC before any non-secure code runs, and then *locks* it. If the normal world could reconfigure it, the protection would be worthless. On a real port, the TZASC setup lives in your platform's TF-A code and is one of the first things to get right.

The memory map, concretely

```
/* optee_os/core/arch/arm/plat-vexpress/platform_config.h    */

#define TZDRAM_BASE   0x0e100000     /* secure DRAM starts     */
#define TZDRAM_SIZE   0x03000000     /* 48 MB                  */

#define TEE_RAM_START      TZDRAM_BASE
#define TEE_RAM_VA_SIZE    (4 * 1024 * 1024)    /* core: 4 MB  */

#define TA_RAM_START  (TEE_RAM_START + TEE_RAM_VA_SIZE)
#define TA_RAM_SIZE   (TZDRAM_SIZE - TEE_RAM_VA_SIZE)
                                    /* TAs: the remaining
                                       44 MB                  */

#define TEE_SHMEM_START  0x42000000  /* NON-secure shared      */
#define TEE_SHMEM_SIZE   0x00200000  /* 2 MB                   */

/* Which produces:

   0x40000000  +--------------------------+
               |  normal-world DRAM       |  Linux lives here
   0x0e100000  +--------------------------+  <- must NOT
               |  TEE_RAM   4 MB SECURE   |     overlap!
   0x0e500000  +--------------------------+
               |  TA_RAM   44 MB SECURE   |
   0x11100000  +--------------------------+
   ...
   0x42000000  +--------------------------+
               |  SHM  2 MB NON-SECURE    |  both worlds
   0x42200000  +--------------------------+

   THE INVARIANT: the normal world's device tree must describe
   its RAM as excluding every secure region, and the TZASC must
   protect exactly the regions OP-TEE claims. Three descriptions
   of the same memory - platform_config.h, the TZASC
   programming, and the non-secure device tree - and all three
   must agree.

   Disagreement produces:
     DT includes secure RAM -> Linux allocates there -> reads
        return zeros -> random corruption, unattributable
     TZASC protects less than OP-TEE claims -> the secure world
        is NOT actually protected, and everything appears to
        work perfectly
     TZASC protects more -> OP-TEE or Linux takes an external
        abort on a legitimate access                          */
```

```
# Confirm the normal world's view excludes secure regions:
$ cat /proc/iomem | head
40000000-41ffffff : System RAM
42200000-7fffffff : System RAM
# note the gap at 0x42000000-0x421fffff - that is the shared
# memory carve-out, correctly excluded.

$ dtc -I fs -O dts /sys/firmware/devicetree/base | grep -A3 'memory@'
    memory@40000000 {
        reg = <0x00 0x40000000 0x00 0x40000000>;
        device_type = "memory";
    };

# And reserved regions:
$ ls /sys/firmware/devicetree/base/reserved-memory/
$ cat /proc/device-tree/reserved-memory/*/reg | xxd

# The definitive test on real hardware: from Linux, try to read
# an address OP-TEE claims. It must return zeros or fault, not
# data.
$ sudo devmem2 0x0e100000 w
# /dev/mem access to a secure address should not show you
# OP-TEE's code. If it does, the TZASC is not configured.
```

The allocation pools

```
/* OP-TEE has no general heap. It has a small number of pools,
   each sized at build time, each for one purpose.           */

  TEE_RAM        the core's own .text/.data/.bss plus
                 - thread stacks: CFG_NUM_THREADS x
                                  CFG_STACK_THREAD_SIZE
                 - the core heap: CFG_CORE_HEAP_SIZE
                                  (default 65536 bytes)
                   used by malloc() inside the core

  TA_RAM         one region per loaded TA, containing its
                 code, data, stack and heap. Sized per TA by
                 its own header:
                     TA_STACK_SIZE
                     TA_DATA_SIZE
                 A TA is mapped here when loaded and unmapped
                 when the last session closes.

  SHM            the non-secure shared memory pool. Chapter 7.

  SDP            secure data path buffers, if
                 CFG_SECURE_DATA_PATH=y - for media pipelines
                 where a decoder must write into memory the
                 normal world cannot read.

/* Budgeting, worked:

     TZDRAM_SIZE                          48 MB
   - TEE_RAM (core code+data+heap+stacks)  4 MB
   = TA_RAM                               44 MB

     one TA with TA_DATA_SIZE = 1 MB, TA_STACK_SIZE = 32 KB,
     plus its code, mapped with 4 KB granularity
       -> roughly 1.2 MB resident while a session is open

     so ~35 concurrent TA instances before TA_RAM is exhausted.

   In practice you hit CFG_NUM_THREADS (8 concurrent calls)
   long before you hit TA_RAM - but a TA that leaks sessions
   will find this limit, and the error is
   TEE_ERROR_OUT_OF_MEMORY at session open.                  */
```

```
# Watch memory usage. Requires CFG_WITH_STATS=y.
$ xtest --stats

TEE stats:
  Pool: TEE_RAM
    Allocated: 1048576, Max allocated: 1187840, Size: 4194304
  Pool: TA_RAM
    Allocated: 2097152, Max allocated: 4194304, Size: 46137344

# "Max allocated" is the high-water mark - the number that
# tells you whether you are close to the limit. Watch it
# during a stress run, not at idle.

# From the secure console, with debug logging:
D/TC:0   tee_mm_alloc:...  0x0e500000 size 0x120000
D/TC:0   tee_mm_free:...
```

> **SIZING IS A PLATFORM DECISION MADE ONCE**
> `TZDRAM_SIZE` must be chosen before the TZASC is programmed and before the non-secure device tree is written. Growing it later means changing firmware, the device tree and possibly a signed boot image in lockstep. Size it generously at the start of a project — secure DRAM is cheap; a field update to three components is not.

Secure data path, briefly

```
/* The problem: a DRM video pipeline decodes protected content.
   The decoded frames must reach the display without the normal
   world ever being able to read them - but the decoder and the
   display controller are normal-world-programmed hardware.

   The answer: buffers in SECURE memory, with those two
   specific DMA masters permitted to access them by the SMMU
   and the TZASC, while the CPU in the normal world is not.

     CFG_SECURE_DATA_PATH=y
     CFG_TEE_SDP_MEM_BASE / CFG_TEE_SDP_MEM_SIZE

   Linux allocates from a dmabuf heap backed by that region;
   the buffers can be passed to the decoder and the display but
   never mapped by an application.

   $ ls /dev/dma_heap/
   linux,cma  system  sdp

   This is the mechanism behind Widevine L1 and PlayReady
   SL3000, and it is the reason "secure playback" requires
   platform support rather than just a TA.                  */
```

How it shows up

Symptom	Cause
Boots on QEMU, random corruption on hardware	Non-secure device tree includes secure RAM; Linux allocated in it
Linux reads zeros from a region it thinks it owns	Same — the TZASC is doing its job
Everything works but a security review fails	TZASC not configured at all; the "secure" region is readable from Linux
External abort during OP-TEE boot	TZASC protects a region OP-TEE needs to access as non-secure, or vice versa
`TEE_ERROR_OUT_OF_MEMORY` opening a session	TA_RAM exhausted — leaked sessions, or `TZDRAM_SIZE` too small
Core panics allocating in a deep call chain	`CFG_CORE_HEAP_SIZE` too small
TA panics with a stack overflow under load	`TA_STACK_SIZE` too small
Works with one TA, fails when a second is loaded	TA_RAM budget
Secure playback shows a black screen	SDP region not configured, or the display controller's StreamID not permitted
TZASC settings lost after suspend/resume	Not restored on resume — the filter is per-boot state that PSCI resume must re-establish

Check yourself

- ✓ Explain what physically makes a region of DRAM secure, and where the enforcement happens.
- ✓ Say why enforcement in the interconnect matters more than enforcement in the MMU.
- ✓ Name who programs the TZASC and why it must be locked.
- ✓ Name the three descriptions of memory that must agree, and give a failure for each disagreement.
- ✓ Draw the QEMU memory map with TEE_RAM, TA_RAM and SHM.
- ✓ Name the four allocation pools and what each holds.
- ✓ Budget TA_RAM for a given `TZDRAM_SIZE` and TA footprint.
- ✓ Explain why "max allocated" is the number to watch.
- ✓ Verify from Linux that a secure region is genuinely inaccessible.
- ✓ Explain what a secure data path is for and why it needs platform support.

Shared memory across the boundary

Every byte that crosses between worlds crosses through non-secure shared memory, because the normal world cannot be given a pointer into secure memory and the secure world will not trust a pointer the normal world chose. How that buffer is allocated, registered and validated is the security-critical seam of the whole design.

Why it must be non-secure

```
Consider the two alternatives and why both fail.

1. The TA writes directly into normal-world memory at an
   address the client supplied.

   -> The client controls the address. It can name kernel
      memory, another process's memory, a device register,
      or OP-TEE's own memory. The TA, running with secure
      privilege, would perform the write.
   -> This is the single most dangerous class of TEE bug,
      and it is why every address must be VALIDATED, not
      merely used.

2. The client reads from secure memory the TA allocated.

   -> Impossible. The TZASC refuses (chapter 6).

So: a third region, non-secure, agreed by both sides.

  +--------------------+
  |  SHM               |  NS=1, so:
  |  non-secure        |    normal world: read/write
  |  shared memory     |    secure world: read/write
  +--------------------+

The secure world can read and write non-secure memory - it
is more privileged, not less. The restriction is one-way.

And therefore: ANYTHING IN SHM IS ATTACKER-CONTROLLED. The
client can change it at any moment, including between the
TA's validation of a value and the TA's use of it.
```

THE DOUBLE-FETCH RULE

Shared memory can change under you. A TA that reads a length field, checks it, and then reads it again to use it has a time-of-check-to-time-of-use bug: the client can change the value in between, from another thread, and defeat the check. **Copy everything you will act on into secure memory first, then validate the copy, then use only the copy.** This is elaborated in chapter 20 and it is the rule that matters most in this entire track.

Two kinds of shared memory

	Static / reserved SHM	Dynamic SHM
Where it comes from	A carve-out declared at build time; OP-TEE tells Linux the physical range	Ordinary kernel pages, registered with OP-TEE on demand
Contiguity	Physically contiguous	Scattered pages, described by a page list
Size limit	Fixed at build time (often 2 MB)	Whatever the kernel can pin
Registration cost	None	An SMC per registration
Enabled by	Always	`CFG_CORE_DYN_SHM=y` and driver support
Best for	Small, frequent transfers	Large buffers, or buffers the application already has

```
# Which do you have? The driver says so at probe:
$ dmesg | grep optee
optee: dynamic shared memory is enabled

# Static-only systems say nothing about dynamic, and
# TEEC_RegisterSharedMemory on an arbitrary pointer will fail.
```

Three ways a client provides a buffer

```
/* 1. ALLOCATE - let the TEE choose the memory. Simplest,
      always works, always contiguous.                      */

    TEEC_SharedMemory shm = {
            .size  = 4096,
            .flags = TEEC_MEM_INPUT | TEEC_MEM_OUTPUT,
    };
    res = TEEC_AllocateSharedMemory(&ctx, &shm);
    memcpy(shm.buffer, data, len);

    op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_WHOLE,
                                     TEEC_NONE, TEEC_NONE,
                                     TEEC_NONE);
    op.params[0].memref.parent = &shm;
    /* ... invoke ... */
    TEEC_ReleaseSharedMemory(&shm);


/* 2. REGISTER - hand the TEE memory you already have. Avoids
      a copy; needs dynamic SHM.                            */

    void *mybuf = malloc(65536);          /* or mmap, or a
                                             dmabuf           */
    TEEC_SharedMemory shm = {
            .buffer = mybuf,
            .size   = 65536,
            .flags  = TEEC_MEM_INPUT,
    };
    res = TEEC_RegisterSharedMemory(&ctx, &shm);
    /* the kernel pins the pages and sends OP-TEE a page list */


/* 3. TEMPORARY - a pointer passed inline; libteec allocates,
      copies in, invokes, copies out, frees. Convenient and it
      COPIES TWICE.                                         */

    op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                     TEEC_MEMREF_TEMP_OUTPUT,
                                     TEEC_NONE, TEEC_NONE);
    op.params[0].tmpref.buffer = input;
    op.params[0].tmpref.size   = inlen;
    op.params[1].tmpref.buffer = output;
    op.params[1].tmpref.size   = sizeof output;

/* Use TEMP for small parameters and control messages. Use
   REGISTER for anything large or repeated - a 1 MB buffer
   copied twice per call, at 60 calls a second, is 120 MB/s of
   pure waste.                                              */
```

The parameter model

```
/* Four slots per invocation. Each is a VALUE (two 32-bit
   numbers) or a MEMREF (a buffer), each with a direction.  */

    TEEC_NONE
    TEEC_VALUE_INPUT           a, b: two uint32_t in
    TEEC_VALUE_OUTPUT          two uint32_t out
    TEEC_VALUE_INOUT
    TEEC_MEMREF_TEMP_INPUT     an inline buffer
    TEEC_MEMREF_TEMP_OUTPUT
    TEEC_MEMREF_TEMP_INOUT
    TEEC_MEMREF_WHOLE          all of a registered shm
    TEEC_MEMREF_PARTIAL_INPUT  part of a registered shm
    TEEC_MEMREF_PARTIAL_OUTPUT
    TEEC_MEMREF_PARTIAL_INOUT

    op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INPUT,
                                     TEEC_MEMREF_TEMP_OUTPUT,
                                     TEEC_NONE,
                                     TEEC_NONE);

/* On the TA side the same four slots arrive, and the TA MUST
   check that it got what it expected: */

TEE_Result TA_InvokeCommandEntryPoint(void *sess, uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INPUT,
                                       TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                       TEE_PARAM_TYPE_NONE,
                                       TEE_PARAM_TYPE_NONE);
        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;   /* MANDATORY */
        ...
}

/* Skip that check and a client can pass a VALUE where you
   expect a MEMREF. You then read params[1].memref.buffer -
   which is now an arbitrary 64-bit integer the attacker chose,
   dereferenced by code running with secure privilege.

   This is the most common OP-TEE TA vulnerability class, and
   it is prevented by four lines.                            */
```

What the TA must do with a buffer

```
/* The four rules, in order of importance.                    */

/* RULE 1: check the parameter types. Above. */

/* RULE 2: check that the memory is really where it claims. */

    res = TEE_CheckMemoryAccessRights(
                TEE_MEMORY_ACCESS_READ |
                TEE_MEMORY_ACCESS_ANY_OWNER,
                params[1].memref.buffer,
                params[1].memref.size);
    if (res != TEE_SUCCESS)
            return res;

/* This asks OP-TEE: is this pointer, for this length, actually
   a buffer this session was given? It catches a client that
   passed a length larger than the buffer it registered.    */

/* RULE 3: copy before you use. */

    if (params[0].memref.size > MAX_INPUT)
            return TEE_ERROR_SHORT_BUFFER;

    uint8_t local[MAX_INPUT];
    /* ONE read of the shared memory. */
    memcpy(local, params[0].memref.buffer, params[0].memref.size);

    /* validate the COPY, act on the COPY */
    if (!looks_sane(local, params[0].memref.size))
            return TEE_ERROR_BAD_PARAMETERS;
    do_work(local, params[0].memref.size);

/* RULE 4: write output once, at the end, and only on success. */

    if (params[1].memref.size < needed) {
            params[1].memref.size = needed;   /* tell them how
                                                 much          */
            return TEE_ERROR_SHORT_BUFFER;
    }
    memcpy(params[1].memref.buffer, result, needed);
    params[1].memref.size = needed;
    return TEE_SUCCESS;

/* Writing partial results into the output buffer as you go
   leaks intermediate state to the client on the error path -
   and for a crypto TA, intermediate state is often the
   secret.                                                    */
```

THE SHORT-BUFFER PROTOCOL

GlobalPlatform's convention: if the output buffer is too small, set `memref.size` to the size you need and return `TEE_ERROR_SHORT_BUFFER`. The client sees the required size, reallocates and retries. Clients rely on this, so implement it — and note that it means writing to `memref.size` even on the error path, which is the one output write that *is* allowed before success.

Cache and coherency

```
/* Shared memory is Normal, cacheable memory on both sides, and
   the interconnect is coherent on any SoC that runs OP-TEE. So
   ordinarily NO cache maintenance is required.

   Where it does bite:

   1. CFG_CORE_UNMAP_CORE_AT_EL0 / different mapping
      attributes on the two sides. If OP-TEE maps SHM
      non-cacheable and Linux maps it cacheable, you have a
      mismatched-attribute alias (virtualisation chapter 7) and
      the behaviour is architecturally unpredictable.

   2. A non-coherent DMA master reading the buffer. Then
      whoever set it up must clean/invalidate:
          TEE_CacheClean(buf, len);
          TEE_CacheInvalidate(buf, len);
          TEE_CacheFlush(buf, len);
      These are OP-TEE extensions, available only if
      CFG_CACHE_API=y - they are not standard GlobalPlatform.

   3. Suspend/resume, where a cache may be powered down with
      dirty lines.

   The normal case needs nothing. If you find yourself adding
   cache maintenance to make a bug go away, you probably have
   an attribute mismatch instead - fix that.               */
```

How it shows up

Symptom	Cause
`TEEC_ERROR_BAD_PARAMETERS`	`paramTypes` disagree between client and TA — the check is working
`TEEC_ERROR_OUT_OF_MEMORY` allocating shared memory	Static SHM pool exhausted; use dynamic SHM or a smaller buffer
`TEEC_RegisterSharedMemory` fails	Dynamic SHM not enabled on this platform
TA panics dereferencing a pointer	Missing `paramTypes` or `TEE_CheckMemoryAccessRights` check — this is the vulnerability, caught late
Intermittent wrong results under concurrency	Double-fetch: the TA read shared memory twice and the client changed it in between
Client sees partial output after an error	TA wrote into the output buffer before it succeeded
Client's retry loop never terminates	Short-buffer protocol not implemented — the TA did not set `memref.size`
Throughput far below expectations	`TEMP` memrefs copying large buffers twice per call
Data corruption only with a DMA-capable accelerator	Non-coherent master with no cache maintenance
Works on QEMU, corrupts on hardware	Mismatched cacheability attributes between the two worlds' mappings

Check yourself

- ✓ Explain why the shared region must be non-secure, and why the restriction is one-way.
- ✓ State the double-fetch rule and the three-step pattern that avoids it.

- ✓ Compare static and dynamic shared memory across contiguity, size and cost.
- ✓ Give the three ways a client supplies a buffer and say when each is right.
- ✓ List the parameter types and explain what `TEEC_PARAM_TYPES` packs.
- ✓ Write the four lines that check `paramTypes` in a TA and explain what they prevent.
- ✓ Explain what `TEE_CheckMemoryAccessRights` verifies.
- ✓ State the four rules for handling a client buffer, in order.
- ✓ Implement the short-buffer protocol and say why it is the one early output write allowed.
- ✓ Say when cache maintenance is genuinely needed and what a need for it usually indicates instead.

The SMC ABI and the OP-TEE message protocol

Everything the two worlds say to each other travels as an `SMC` instruction with eight register arguments, plus a message structure in shared memory. This chapter is the wire format — enough to read a trace, debug a handshake, or implement the normal-world side yourself.

SMCCC: the calling convention

```
SMC #0 with a function ID in w0. The ID is structured:

 31  30      29 28        24 23    16 15                  0
+---+----------+-------------+--------+--------------------+
|FST| SMC64    |  reserved   | OWNER  |   function number  |
+---+----------+-------------+--------+--------------------+

bit 31    0 = yielding (may be preempted, may make RPCs)
          1 = fast (runs to completion, interrupts masked)
bit 30    0 = SMC32 (32-bit arguments)
          1 = SMC64 (64-bit arguments)
[29:24]   must be zero
[23:16]   OWNER - who handles this call:
             0x00  Arm architecture calls
             0x01  CPU service calls
             0x02  SiP (silicon partner) calls
             0x03  OEM calls
             0x04  Standard Secure Service - PSCI lives here
             0x05  Standard Hypervisor Service
             0x06  Vendor Hypervisor Service
             0x30-0x31  TRUSTED APPLICATION calls
             0x32-0x3F  TRUSTED OS calls  <-- OP-TEE
[15:0]    the function itself

Arguments in x1-x7 (or w1-w7 for SMC32), results in x0-x3.

Examples:
  0x84000000  PSCI_VERSION      fast, SMC32, owner 4
  0xC4000003  PSCI_CPU_ON       fast, SMC64, owner 4
  0xB2000000  OPTEE_SMC_CALLS_UID
  0x32000004  OPTEE_SMC_CALL_WITH_ARG   yielding, owner 0x32
```

The handshake at probe

```
/* Before it will use OP-TEE, the Linux driver asks four
   questions. Reading this sequence tells you exactly where a
   broken bring-up failed.                                    */

  1. "Who are you?"
       SMC(OPTEE_SMC_CALLS_UID)              = 0xBF00FF01
     returns four words - a UUID. Must equal
       384fb3e0-e7f8-11e3-af63-0002a5d5c51b
     Anything else: not OP-TEE, or not a TEE at all. The
     driver gives up quietly.

  2. "What revision?"
       SMC(OPTEE_SMC_CALLS_REVISION)         = 0xBF00FF03
     returns major, minor. The driver checks compatibility.
       -> dmesg: "optee: revision 4.1"

  3. "What memory can we share?"
       SMC(OPTEE_SMC_GET_SHM_CONFIG)         = 0xB2000007
     returns the physical base, size and settings of the
     static shared memory pool.

  4. "What do you support?"
       SMC(OPTEE_SMC_EXCHANGE_CAPABILITIES)  = 0xB2000009
     the driver sends what IT can do:
       OPTEE_SMC_NSEC_CAP_UNIPROCESSOR
     OP-TEE replies with what it can do:
       OPTEE_SMC_SEC_CAP_HAVE_RESERVED_SHM
       OPTEE_SMC_SEC_CAP_DYNAMIC_SHM
       OPTEE_SMC_SEC_CAP_MEMREF_NULL
       OPTEE_SMC_SEC_CAP_ASYNC_NOTIF
       OPTEE_SMC_SEC_CAP_VIRTUALIZATION
       -> dmesg: "optee: dynamic shared memory is enabled"

  Then, if all four succeed:
       -> "optee: initialized driver", /dev/tee0 appears.

/* drivers/tee/optee/smc_abi.c - optee_probe(). Worth reading
   once; it is about 200 lines and it is the whole
   negotiation.                                               */
```

The message structure

```
/* Registers carry only the call type and a pointer. The real
   request lives in shared memory as an optee_msg_arg.       */

struct optee_msg_arg {
        uint32_t cmd;           /* OPEN_SESSION, INVOKE_COMMAND ... */
        uint32_t func;          /* the TA's command ID              */
        uint32_t session;       /* which session                    */
        uint32_t cancel_id;
        uint32_t pad;
        uint32_t ret;           /* TEE_Result out                   */
        uint32_t ret_origin;    /* who failed: API, comms, TEE, TA  */
        uint32_t num_params;
        struct optee_msg_param params[];   /* flexible array        */
};

struct optee_msg_param {
        uint64_t attr;          /* type + direction                 */
        union {
                struct { uint64_t a, b, c; } value;
                struct { uint64_t offs, size; uint64_t shm_ref; } rmem;
                struct { uint64_t buf_ptr, size, shm_ref; } tmem;
        } u;
};

/* attr values: */
#define OPTEE_MSG_ATTR_TYPE_NONE            0x0
#define OPTEE_MSG_ATTR_TYPE_VALUE_INPUT     0x1
#define OPTEE_MSG_ATTR_TYPE_VALUE_OUTPUT    0x2
#define OPTEE_MSG_ATTR_TYPE_VALUE_INOUT     0x3
#define OPTEE_MSG_ATTR_TYPE_RMEM_INPUT      0x5  /* registered  */
#define OPTEE_MSG_ATTR_TYPE_RMEM_OUTPUT     0x6
#define OPTEE_MSG_ATTR_TYPE_RMEM_INOUT      0x7
#define OPTEE_MSG_ATTR_TYPE_TMEM_INPUT      0x9  /* temporary   */
#define OPTEE_MSG_ATTR_TYPE_TMEM_OUTPUT     0xa
#define OPTEE_MSG_ATTR_TYPE_TMEM_INOUT      0xb

/* Commands: */
#define OPTEE_MSG_CMD_OPEN_SESSION          0
#define OPTEE_MSG_CMD_INVOKE_COMMAND        1
#define OPTEE_MSG_CMD_CLOSE_SESSION         2
#define OPTEE_MSG_CMD_CANCEL                3
#define OPTEE_MSG_CMD_REGISTER_SHM          4
#define OPTEE_MSG_CMD_UNREGISTER_SHM        5
#define OPTEE_MSG_CMD_DO_BOTTOM_HALF        6
#define OPTEE_MSG_CMD_STOP_ASYNC_NOTIF      7
```

One invocation, register by register

```
Client calls TEEC_InvokeCommand(sess, 5, &op, &origin).

1. libteec fills a struct tee_ioctl_invoke_arg and calls
     ioctl(fd, TEE_IOC_INVOKE, &arg)

2. The kernel driver allocates an optee_msg_arg in shared
   memory and fills it:
     cmd        = OPTEE_MSG_CMD_INVOKE_COMMAND
     func       = 5
     session    = the session id
     num_params = 4
     params[0].attr = OPTEE_MSG_ATTR_TYPE_VALUE_INPUT
     params[0].u.value.a = ...
     params[1].attr = OPTEE_MSG_ATTR_TYPE_TMEM_OUTPUT
     params[1].u.tmem.buf_ptr = physical address
     params[1].u.tmem.size    = length
     params[1].u.tmem.shm_ref = cookie

3. The driver issues:
     x0 = OPTEE_SMC_CALL_WITH_ARG   (0x32000004)
     x1 = upper 32 bits of the msg_arg physical address
     x2 = lower 32 bits
     x3-x7 = 0
     SMC #0

4. EL3 (opteed) saves the non-secure context, restores the
   secure one, ERETs into OP-TEE's std_smc_entry.

5. OP-TEE allocates a thread, reads the msg_arg from shared
   memory, translates the parameters, finds the session,
   and calls the TA's TA_InvokeCommandEntryPoint.

6. The TA runs. If it needs the normal world (to read a
   file, to get the time, to load another TA) it makes an
   RPC - which RETURNS through this same path with a
   "please do this for me" code, and the call is later
   resumed. Chapter 9.

7. The TA returns. OP-TEE writes ret and ret_origin into
   the msg_arg, frees the thread, and executes
     SMC(TEESMC_OPTEED_RETURN_CALL_DONE)

8. EL3 restores the non-secure context. The driver's SMC
   returns with:
     x0 = OPTEE_SMC_RETURN_OK      (0)
   or
     x0 = OPTEE_SMC_RETURN_ETHREAD_LIMIT  - retry later
     x0 = OPTEE_SMC_RETURN_RPC_*          - do this first

9. The driver reads ret/ret_origin out of the msg_arg and
   returns them up through the ioctl to libteec.
```

Return origin: who actually failed

```
/* Every failure carries an ORIGIN, and it is the most
   under-used debugging aid in the whole API.              */

  TEEC_ORIGIN_API          0x1  libteec itself rejected it -
                                bad arguments in your client
  TEEC_ORIGIN_COMMS        0x2  the transport failed - driver,
                                SMC, or the message never
                                arrived
  TEEC_ORIGIN_TEE          0x3  OP-TEE core rejected it - no
                                such session, TA not found,
                                out of memory, bad params
  TEEC_ORIGIN_TRUSTED_APP  0x4  YOUR TA returned this. The
                                value is whatever your TA
                                returned.

/* So:
     res = 0xFFFF0006, origin = 3  -> OP-TEE says bad params.
                                      Your paramTypes disagree.
     res = 0xFFFF0006, origin = 4  -> your TA returned
                                      TEE_ERROR_BAD_PARAMETERS.
                                      Different bug entirely.
     res = 0x00000001, origin = 4  -> your TA's own error code.
                                      Nothing standard about it.

   Always print both. A client that prints only the result
   throws away half the information.                       */

  printf("failed: res=0x%08x origin=%u\n", res, origin);
```

```
/* The standard result codes worth recognising on sight: */

#define TEEC_SUCCESS                    0x00000000
#define TEEC_ERROR_GENERIC              0xFFFF0000
#define TEEC_ERROR_ACCESS_DENIED        0xFFFF0001
#define TEEC_ERROR_CANCEL               0xFFFF0002
#define TEEC_ERROR_ACCESS_CONFLICT      0xFFFF0003
#define TEEC_ERROR_EXCESS_DATA          0xFFFF0004
#define TEEC_ERROR_BAD_FORMAT           0xFFFF0005
#define TEEC_ERROR_BAD_PARAMETERS       0xFFFF0006
#define TEEC_ERROR_BAD_STATE            0xFFFF0007
#define TEEC_ERROR_ITEM_NOT_FOUND       0xFFFF0008  /* TA not
                                                       found   */
#define TEEC_ERROR_NOT_IMPLEMENTED      0xFFFF0009
#define TEEC_ERROR_NOT_SUPPORTED        0xFFFF000A
#define TEEC_ERROR_NO_DATA              0xFFFF000B
#define TEEC_ERROR_OUT_OF_MEMORY        0xFFFF000C
#define TEEC_ERROR_BUSY                 0xFFFF000D
#define TEEC_ERROR_COMMUNICATION        0xFFFF000E
#define TEEC_ERROR_SECURITY             0xFFFF000F  /* signature
                                                       check
                                                       failed  */
#define TEEC_ERROR_SHORT_BUFFER         0xFFFF0010
#define TEEC_ERROR_TARGET_DEAD          0xFFFF3024  /* the TA
                                                       PANICKED */
```

> **`TEEC_ERROR_TARGET_DEAD` MEANS READ THE SECURE CONSOLE**
> 0xFFFF3024 is OP-TEE telling you the TA panicked — dereferenced a bad pointer, failed an assertion, or called `TEE_Panic()`. The client cannot know why. The secure console printed a stack dump at the moment it happened. This is the single strongest argument for the two-console setup in chapter 4.

Watching the traffic

```
# The Linux driver has tracepoints and dynamic debug:
$ echo 'module optee +p' | sudo tee /sys/kernel/debug/dynamic_debug/control
$ dmesg -w

# Under QEMU, trace every SMC that reaches EL3 by building TF-A
# with debug:
$ make -C arm-trusted-firmware PLAT=qemu DEBUG=1 LOG_LEVEL=50 ...

# OP-TEE's own view, at flow level:
$ make CFG_TEE_CORE_LOG_LEVEL=4
# secure console then shows every entry and exit:
F/TC:0 0 thread_alloc_and_run:...
F/TC:0 0 tee_ta_open_session:...
F/TC:0 0 tee_ta_invoke_command:...

# And from the guest, the ioctl layer:
$ strace -e trace=ioctl ./my_client
ioctl(3, TEE_IOC_VERSION, ...)     = 0
ioctl(3, TEE_IOC_OPEN_SESSION, ...) = 0
ioctl(3, TEE_IOC_INVOKE, ...)      = 0
ioctl(3, TEE_IOC_CLOSE_SESSION, ...) = 0

# The ioctl numbers, if you ever implement a client without
# libteec:
#   TEE_IOC_VERSION        get driver capabilities
#   TEE_IOC_SHM_ALLOC      allocate shared memory, returns a fd
#   TEE_IOC_SHM_REGISTER   register your own memory
#   TEE_IOC_OPEN_SESSION
#   TEE_IOC_INVOKE
#   TEE_IOC_CANCEL
#   TEE_IOC_CLOSE_SESSION
#   TEE_IOC_SUPPL_RECV     tee-supplicant only
#   TEE_IOC_SUPPL_SEND
```

How it shows up

Symptom	Cause
Driver does not probe, no error	The UID call returned something else — OP-TEE is not there
`optee: api uid mismatch`	Something answered the SMC but is not OP-TEE
`optee: unsupported revision`	Kernel driver and OP-TEE versions too far apart
`TEEC_ERROR_COMMUNICATION`	The SMC path failed — check EL3 built with `SPD=opteed`
`TEEC_ERROR_TARGET_DEAD`	The TA panicked. The reason is on the secure console
`TEEC_ERROR_BUSY` under load	`ETHREAD_LIMIT` — the thread pool is full (chapter 5)
Same error code, two different bugs	You are ignoring the origin. Print it
Client hangs forever in an invoke	The TA is blocked, or a thread-exhaustion deadlock

Symptom	Cause
Works with small buffers, fails with large	Static SHM pool size, or a temporary memref larger than the pool
Registered memory rejected	`DYNAMIC_SHM` not in the exchanged capabilities

Check yourself

- ✓ Decode an SMC function ID into fast/yielding, 32/64-bit and owner.
- ✓ Name the four questions the Linux driver asks at probe and what each confirms.
- ✓ Describe `optee_msg_arg` and say where it lives.
- ✓ Explain how a buffer parameter is expressed in `optee_msg_param`.
- ✓ Trace one `TEEC_InvokeCommand` from ioctl to TA entry point and back, naming both SMCs.
- ✓ Explain `OPTEE_SMC_RETURN_ETHREAD_LIMIT` and what the driver does about it.
- ✓ Name the four return origins and give a case where the same result code means two different things.
- ✓ Say what `TEEC_ERROR_TARGET_DEAD` means and where to look next.
- ✓ Recognise `0xFFFF0008`, `0xFFFF000F` and `0xFFFF0010` on sight.
- ✓ Enable driver, TF-A and OP-TEE tracing simultaneously and correlate one call across all three.

RPC and tee-supplicant

The secure world has no filesystem, no network and no way to allocate normal-world memory. So when it needs those things it asks — by returning from the very SMC that invoked it, with "not finished, do this first". That inversion is elegant, it is why OP-TEE stays small, and it is the source of the deadlock in chapter 5.

Returning in order to ask

```
The secure world cannot CALL the normal world. There is no
such instruction: SMC goes up in privilege, never down.

So OP-TEE uses the only channel it has - the return value of
the call it is currently servicing.

  normal world:   SMC(CALL_WITH_ARG, msg)
                       |
                       v
  OP-TEE:         thread allocated, TA runs
                  TA calls TEE_OpenPersistentObject()
                       |
                       |  "I need a file. I cannot open
                       |   files."
                       v
                  SAVE the thread's full state
                  write an RPC request into shared memory
                  return to the normal world with
                     x0 = OPTEE_SMC_RETURN_RPC_CMD
                       |
                       v
  driver:         "this is not a result, it is a request"
                  hand it to tee-supplicant
                       |
                       v
  tee-supplicant: open("/data/tee/...")
                  read the data into shared memory
                  reply
                       |
                       v
  driver:         SMC(CALL_RETURN_FROM_RPC, thread_id, ...)
                       |
                       v
  OP-TEE:         find the saved thread, RESTORE it,
                  TEE_OpenPersistentObject() returns
                  TA continues as if nothing happened
                       |
                       ... eventually ...
                       v
                  return OPTEE_SMC_RETURN_OK

The TA sees an ordinary blocking function call. Underneath,
control left the secure world entirely and came back.

THE THREAD IS HELD THROUGHOUT. That is the whole design
constraint (chapter 5).
```

The RPC requests

Return code	Meaning	Serviced by
`RPC_ALLOC`	Allocate shared memory for me	The kernel driver
`RPC_FREE`	Free it	The kernel driver
`RPC_FOREIGN_INTR`	A normal-world interrupt is pending; go and service it	The kernel, implicitly
`RPC_CMD`	A real request — see the command table below	Mostly `tee-supplicant`

```
/* The RPC_CMD sub-commands: */

#define OPTEE_RPC_CMD_LOAD_TA         0  /* read a .ta file    */
#define OPTEE_RPC_CMD_RPMB            1  /* an RPMB frame      */
#define OPTEE_RPC_CMD_FS              2  /* REE filesystem op  */
#define OPTEE_RPC_CMD_GET_TIME        3  /* wall-clock time    */
#define OPTEE_RPC_CMD_NOTIFICATION    4  /* wait/wake a thread */
#define OPTEE_RPC_CMD_SUSPEND         5  /* sleep              */
#define OPTEE_RPC_CMD_SHM_ALLOC       6
#define OPTEE_RPC_CMD_SHM_FREE        7
#define OPTEE_RPC_CMD_SQL_FS          8
#define OPTEE_RPC_CMD_GPROF           9
#define OPTEE_RPC_CMD_SOCKET         10  /* network!            */
#define OPTEE_RPC_CMD_BENCH_REG      20
#define OPTEE_RPC_CMD_FTRACE         21
#define OPTEE_RPC_CMD_I2C_TRANSFER   22

/* Two of these deserve a second look.

   LOAD_TA - the secure world asks the normal world to give it
   a trusted application. The normal world is untrusted. So
   OP-TEE VERIFIES THE SIGNATURE of whatever it receives
   (chapter 14). An attacker who substitutes a different .ta
   gets TEEC_ERROR_SECURITY, not code execution. This is the
   design pattern: accept data from an untrusted source, then
   verify it yourself.

   SOCKET - yes, a TA can open a TCP connection, tunnelled
   through tee-supplicant. Whether it SHOULD is another
   question: the normal world sees and can modify every byte
   unless you run TLS end-to-end inside the TA.            */
```

> **THE SECURITY PATTERN THAT MAKES THIS SAFE**
>
> Every RPC hands work to an untrusted party. The rule that keeps it sound: **the secure world never trusts the result**. TA binaries are signature-checked. Secure storage is encrypted and authenticated, so a modified file fails its MAC. RPMB frames are MACed with a key the normal world does not have. Time is explicitly labelled untrusted. Where verification is impossible — as with wall-clock time — the API says so, and chapter 17 covers what to do instead.

tee-supplicant

```
/* A normal user-space daemon, ~2000 lines. Its whole life:  */

int main(void)
{
        int fd = open("/dev/teepriv0", O_RDWR);

        for (;;) {
                struct tee_ioctl_supp_recv_arg arg;

                /* Blocks until OP-TEE needs something. */
                ioctl(fd, TEE_IOC_SUPPL_RECV, &arg);

                switch (arg.func) {
                case OPTEE_RPC_CMD_LOAD_TA:
                        load_ta(&arg);      /* read
                                               /lib/optee_armtz/
                                               <uuid>.ta      */
                        break;
                case OPTEE_RPC_CMD_FS:
                        tee_supp_fs(&arg);  /* open/read/write/
                                               unlink under
                                               /data/tee/      */
                        break;
                case OPTEE_RPC_CMD_RPMB:
                        rpmb_process(&arg); /* ioctl to the
                                               eMMC             */
                        break;
                case OPTEE_RPC_CMD_GET_TIME:
                        get_time(&arg);
                        break;
                case OPTEE_RPC_CMD_I2C_TRANSFER:
                        i2c_transfer(&arg);
                        break;
                ...
                }

                ioctl(fd, TEE_IOC_SUPPL_SEND, &arg);
        }
}

/* Where it keeps things: */
    /lib/optee_armtz/<uuid>.ta    trusted applications
    /data/tee/                    secure storage files
                                  (encrypted; the normal world
                                   can delete them but not read
                                   or forge them)

/* Running it: */
$ tee-supplicant -d              # daemonise
$ tee-supplicant -f              # foreground, prints errors
$ tee-supplicant -f /dev/teepriv0
$ systemctl status tee-supplicant

/* WITHOUT IT:
     TEEC_OpenSession -> TEEC_ERROR_ITEM_NOT_FOUND, because
     nothing can load the TA binary.
   And a TA already loaded that tries to use secure storage
   will BLOCK, holding a thread, until the supplicant returns
   or the call is cancelled.                                  */
```

Asynchronous notification

```
/* A problem the RPC model does not solve: the secure world
   needs to tell the normal world something when it is NOT
   servicing a call. A secure interrupt arrived; a background
   task finished.

   With plain RPC there is no channel - OP-TEE only runs when
   called.

   CFG_CORE_ASYNC_NOTIF=y adds one:

     OP-TEE raises a dedicated non-secure interrupt.
       |
       v
     Linux's optee driver takes it, and issues
       SMC(OPTEE_SMC_GET_ASYNC_NOTIF_VALUE)
       to ask what happened.
       |
       v
     OP-TEE returns a notification value.
       |
       v
     The driver wakes whatever was waiting, or schedules
       OPTEE_MSG_CMD_DO_BOTTOM_HALF so OP-TEE can run
       deferred work.

   Confirm it:
     $ dmesg | grep optee
     optee: Asynchronous notifications enabled

   Without it, a TA that wants to wait for a secure interrupt
   must poll, holding a thread - which is exactly the pattern
   that exhausts the pool.                                  */
```

The deadlock, in full

```
CFG_NUM_THREADS = 4. Four application threads each invoke a
TA that writes to secure storage.

  thread 1  in OP-TEE, blocked on RPC_CMD_FS
  thread 2  in OP-TEE, blocked on RPC_CMD_FS
  thread 3  in OP-TEE, blocked on RPC_CMD_FS
  thread 4  in OP-TEE, blocked on RPC_CMD_FS

All four OP-TEE threads are held. Every one is waiting for
tee-supplicant.

tee-supplicant CAN service them - one at a time - and
normally does, and everything is merely slow.

It deadlocks when tee-supplicant itself cannot run:

  * tee-supplicant's own thread tries to invoke a TA
    (never do this)
  * the storage it must write to is on a filesystem that is
    itself waiting on something blocked in the TEE
  * an out-of-memory condition stops it being scheduled
  * a single-threaded supplicant is servicing a request that
    blocks on a device that is unresponsive

Symptom: total silence. No error, no timeout, no log line.
Every client hangs in ioctl(). The secure console shows
nothing because OP-TEE is not running.

DIAGNOSING IT:
  $ cat /proc/<client-pid>/stack        # blocked in optee
  $ cat /proc/<supplicant-pid>/stack    # what is IT waiting
                                          for?
  $ echo t > /proc/sysrq-trigger        # all task stacks

AVOIDING IT:
  * never call into the TEE from a process that services the
    TEE
  * set CFG_NUM_THREADS comfortably above your peak
    concurrency
  * keep RPC-using operations (secure storage) off hot paths
  * give tee-supplicant a favourable scheduling class if it
    is on a loaded system
  * cancel long operations with TEEC_RequestCancellation
    from a watchdog thread
```

How it shows up

Symptom	Cause
`TEEC_ERROR_ITEM_NOT_FOUND` on every session	`tee-supplicant` not running
Everything hangs with no error	Thread-exhaustion deadlock; check the supplicant's own stack
Secure storage operations extremely slow	Every one is an RPC round trip through a user-space daemon. Expected — batch them
TA works, then fails after the supplicant restarts	Sessions holding state the supplicant was mid-way through servicing

Symptom	Cause
`TEEC_ERROR_SECURITY` loading a TA	The supplicant delivered a binary whose signature did not verify — working as designed
Secure storage file deleted, TA still works	Deletion is possible from the normal world; confidentiality and integrity are protected, availability is not
Time from a TA is wrong or moves backwards	`TEE_GetREETime` comes from the normal world and is not trustworthy
A TA cannot wait for a secure interrupt without spinning	Async notification not enabled
Deadlock only under load	`CFG_NUM_THREADS` too small for peak concurrency

Check yourself

- ✓ Explain why the secure world cannot call the normal world, and what it does instead.
- ✓ Trace a `TEE_OpenPersistentObject` call out to `tee-supplicant` and back.
- ✓ Say what happens to the OP-TEE thread during an RPC.
- ✓ Name six RPC commands and who services each.
- ✓ Explain why loading a TA from an untrusted source is safe, and what makes it so.
- ✓ State the general pattern that makes every RPC safe, and name one case where it cannot apply.
- ✓ Describe what `tee-supplicant` does and the two directories it owns.
- ✓ Explain what asynchronous notification adds and what it replaces.
- ✓ Describe the thread-exhaustion deadlock precisely and give four ways to avoid it.
- ✓ Diagnose a total hang: name the two `/proc` files you would read first.

The Linux TEE subsystem

Linux has a generic TEE framework — `drivers/tee/` — with OP-TEE as one back end, AMD-TEE and Trusty as others. It gives user space a stable ioctl interface and gives kernel code a way to use a TA directly, which is how the fTPM driver in chapter 23 works.

The subsystem

```
drivers/tee/
  tee_core.c        the generic layer: device nodes, the
                    ioctl interface, contexts, sessions
  tee_shm.c         shared memory objects, dmabuf export
  tee_shm_pool.c    the allocators
  optee/
    core.c          probe, device registration
    smc_abi.c       the SMC transport (chapter 8)
    ffa_abi.c       the FF-A transport - for OP-TEE running
                    as a secure partition under an SPM
                    (virt chapter 19)
    call.c          building and issuing optee_msg_arg
    rpc.c           servicing RPC returns
    supp.c          the tee-supplicant queue
    device.c        enumerating TAs as kernel devices
  amdtee/  tstee/   other back ends

Two device nodes per TEE:

  /dev/tee0       for ordinary clients
  /dev/teepriv0   privileged - tee-supplicant uses this one

Kernel config:
  CONFIG_TEE=y
  CONFIG_OPTEE=y
  CONFIG_OPTEE_INSECURE_LOAD_IMAGE  (n; for loading OP-TEE
                                     itself from Linux - a
                                     development feature)
```

The user-space interface

```
/* include/uapi/linux/tee.h - this is the whole ABI, and
   libteec is a thin wrapper over it.                        */

  TEE_IOC_VERSION          what implementation, what caps
  TEE_IOC_SHM_ALLOC        allocate shared memory; returns a
                           file descriptor you mmap
  TEE_IOC_SHM_REGISTER     register memory you already have
  TEE_IOC_OPEN_SESSION     open a session to a TA by UUID
  TEE_IOC_INVOKE           invoke a command
  TEE_IOC_CANCEL           cancel an in-flight invocation
  TEE_IOC_CLOSE_SESSION
  TEE_IOC_SUPPL_RECV       (privileged) wait for an RPC
  TEE_IOC_SUPPL_SEND       (privileged) answer it

/* A client without libteec, to show there is no magic: */

    int fd = open("/dev/tee0", O_RDWR);

    struct tee_ioctl_version_data ver;
    ioctl(fd, TEE_IOC_VERSION, &ver);
    /* ver.impl_id  = TEE_IMPL_ID_OPTEE
       ver.gen_caps = TEE_GEN_CAP_GP | TEE_GEN_CAP_REG_MEM   */

    /* Shared memory is a file descriptor you mmap. */
    struct tee_ioctl_shm_alloc_data shm = { .size = 4096 };
    int shm_fd = ioctl(fd, TEE_IOC_SHM_ALLOC, &shm);
    void *buf = mmap(NULL, shm.size, PROT_READ | PROT_WRITE,
                     MAP_SHARED, shm_fd, 0);

    /* Open a session. */
    struct {
            struct tee_ioctl_open_session_arg arg;
            struct tee_ioctl_param params[4];
    } req = {0};
    memcpy(req.arg.uuid, uuid, 16);
    req.arg.clnt_login = TEE_IOCTL_LOGIN_PUBLIC;
    req.arg.num_params = 4;

    struct tee_ioctl_buf_data bd = {
            .buf_ptr = (uintptr_t)&req,
            .buf_len = sizeof(req),
    };
    ioctl(fd, TEE_IOC_OPEN_SESSION, &bd);
    /* req.arg.session is now valid; req.arg.ret and
       req.arg.ret_origin carry the result (chapter 8).      */

/* The shm-as-a-file-descriptor design is deliberate: it makes
   shared memory a first-class object that can be passed
   between processes with SCM_RIGHTS, exported as a dmabuf, and
   reclaimed automatically when the last reference goes.     */
```

Using a TA from kernel code

```
/* The interesting capability, and the one the fTPM depends on.
   A kernel driver can be a TEE client.                       */

#include <linux/tee_drv.h>

static const uuid_t my_ta_uuid =
        UUID_INIT(0x12345678, 0x1234, 0x1234,
                  0x12, 0x34, 0x12, 0x34, 0x12, 0x34, 0x12, 0x34);

/* Used to find an OP-TEE context among possibly several TEEs. */
static int optee_match(struct tee_ioctl_version_data *ver,
                       const void *data)
{
        return ver->impl_id == TEE_IMPL_ID_OPTEE;
}

struct my_private {
        struct tee_context *ctx;
        u32                 session;
        struct tee_shm     *shm;
};

static int my_tee_init(struct device *dev, struct my_private *p)
{
        struct tee_ioctl_open_session_arg sess_arg = {0};
        int rc;

        /* 1. Get a context. */
        p->ctx = tee_client_open_context(NULL, optee_match,
                                         NULL, NULL);
        if (IS_ERR(p->ctx))
                return PTR_ERR(p->ctx);   /* -ENOENT if no TEE  */

        /* 2. Open a session. */
        export_uuid(sess_arg.uuid, &my_ta_uuid);
        sess_arg.clnt_login = TEE_IOCTL_LOGIN_REE_KERNEL;
        sess_arg.num_params = 0;

        rc = tee_client_open_session(p->ctx, &sess_arg, NULL);
        if (rc < 0 || sess_arg.ret != 0) {
                dev_err(dev, "open_session failed %d 0x%x\n",
                        rc, sess_arg.ret);
                tee_client_close_context(p->ctx);
                return -EINVAL;
        }
        p->session = sess_arg.session;

        /* 3. Allocate shared memory once, reuse it. */
        p->shm = tee_shm_alloc_kernel_buf(p->ctx, 4096);
        if (IS_ERR(p->shm)) {
                tee_client_close_session(p->ctx, p->session);
                tee_client_close_context(p->ctx);
                return PTR_ERR(p->shm);
        }
        return 0;
}

static int my_tee_call(struct my_private *p, u32 cmd,
                       const void *in, size_t inlen,
                       void *out, size_t *outlen)
```

```
{
        struct tee_ioctl_invoke_arg arg = {
                .func       = cmd,
                .session    = p->session,
                .num_params = 4,
        };
        struct tee_param param[4] = {0};
        void *buf = tee_shm_get_va(p->shm, 0);
        int rc;

        if (IS_ERR(buf))
                return PTR_ERR(buf);

        memcpy(buf, in, inlen);

        param[0].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INOUT;
        param[0].u.memref.shm      = p->shm;
        param[0].u.memref.size     = inlen;
        param[0].u.memref.shm_offs = 0;

        rc = tee_client_invoke_func(p->ctx, &arg, param);
        if (rc < 0 || arg.ret != 0)
                return -EIO;

        *outlen = param[0].u.memref.size;
        memcpy(out, buf, *outlen);
        return 0;
}

static void my_tee_exit(struct my_private *p)
{
        tee_shm_free(p->shm);
        tee_client_close_session(p->ctx, p->session);
        tee_client_close_context(p->ctx);
}

/* This is, almost exactly, drivers/char/tpm/tpm_ftpm_tee.c.
   Chapter 23 builds on it.                                  */
```

LOGIN TYPES MATTER

`TEE_IOCTL_LOGIN_REE_KERNEL` tells the TA that the caller is the kernel, not a user-space process. A TA can act on that — refusing to serve user space, or applying different policy. Other logins are `PUBLIC` (anyone), `USER` and `GROUP` (the client's uid/gid is hashed into the login identity, so a TA can bind stored data to a specific caller). This is how per-application secure storage works; see chapter 15.

Enumerating TAs as devices

```
/* drivers/tee/optee/device.c asks OP-TEE for the list of TAs
   that want a kernel driver, and registers a device for each.
   A kernel driver then binds by UUID, exactly like PCI.     */

$ ls /sys/bus/tee/devices/
optee-ta-be2dc9a0-02b4-4b33-ba81-9cce8a6c1e1c   # fTPM
optee-ta-e33b0e0e-6e2a-4f3b-8f2f-3e8a5a1a2b3c

$ ls /sys/bus/tee/drivers/
optee-ta-be2dc9a0-02b4-4b33-ba81-9cce8a6c1e1c

/* And a driver: */

static const struct tee_client_device_id my_id_table[] = {
        { UUID_INIT(0xbe2dc9a0, 0x02b4, 0x4b33,
                    0xba, 0x81, 0x9c, 0xce,
                    0x8a, 0x6c, 0x1e, 0x1c) },
        { }
};
MODULE_DEVICE_TABLE(tee, my_id_table);

static struct tee_client_driver my_driver = {
        .id_table = my_id_table,
        .driver = {
                .name  = "my-ta-driver",
                .bus   = &tee_bus_type,
                .probe = my_probe,
                .remove = my_remove,
        },
};
module_driver(my_driver, driver_register, driver_unregister);

/* The TA must declare that it wants this, by setting
   TA_FLAG_DEVICE_ENUM in its properties (chapter 13). Without
   that flag it exists but no device node appears, and a kernel
   driver has nothing to bind to. That is a common and
   confusing failure.                                        */
```

Debugging the driver

```
# Dynamic debug across the whole subsystem:
$ echo 'module optee +p'    | sudo tee /sys/kernel/debug/dynamic_debug/control
$ echo 'module tee +p'      | sudo tee /sys/kernel/debug/dynamic_debug/control
$ dmesg -w

# What did probe conclude?
$ dmesg | grep -E 'optee|tee:'
optee: probing for conduit method.
optee: revision 4.1 (0e3e9c9d)
optee: dynamic shared memory is enabled
optee: initialized driver

# Who has the TEE open?
$ sudo lsof /dev/tee0 /dev/teepriv0
$ sudo fuser -v /dev/tee0

# Where is a hung client blocked?
$ sudo cat /proc/$(pgrep my_client)/stack
[<0>] optee_do_call_with_arg+0x...
[<0>] optee_invoke_func+0x...
[<0>] tee_ioctl+0x...
# In optee_do_call_with_arg = waiting on the secure world.
# Now check tee-supplicant's stack (chapter 9).

# Trace the ioctls a client makes:
$ strace -e trace=ioctl,openat ./my_client

# ftrace the driver's own functions:
$ cd /sys/kernel/debug/tracing
$ echo 'optee_*' | sudo tee set_ftrace_filter
$ echo function | sudo tee current_tracer
$ sudo cat trace_pipe
```

How it shows up

Symptom	Cause
No `/dev/tee0`	`CONFIG_OPTEE` off, device tree node missing, or the probe handshake failed (chapter 8)
`tee_client_open_context` returns `-ENOENT`	No TEE registered yet — the driver probed after yours. Defer with `-EPROBE_DEFER`
No device under `/sys/bus/tee/devices/`	TA missing `TA_FLAG_DEVICE_ENUM`
Kernel driver's session opens, user space's fails	The TA is checking the login type and refusing user space — possibly deliberately
Client blocked in `optee_do_call_with_arg`	Waiting on the secure world; look at `tee-supplicant` next
`tee_shm_alloc` fails	Static SHM pool exhausted; free shared memory or enable dynamic SHM
Works in user space, fails from the kernel	Kernel buffers must come from `tee_shm_alloc_kernel_buf`, not `kmalloc`
Session leaks after a driver unload	Missing `tee_client_close_session` / `close_context` in the remove path
Different behaviour under a hypervisor	The FF-A transport is in use rather than the SMC one

Check yourself

- ✓ Name the generic layer and three back ends, and say what the generic layer provides.
- ✓ Explain the difference between `/dev/tee0` and `/dev/teepriv0`.
- ✓ List the ioctls and say which two are privileged.
- ✓ Explain why shared memory is exposed as a file descriptor.
- ✓ Write the kernel-side sequence to open a context, open a session and invoke a command.
- ✓ Explain what login types exist and what a TA can do with them.
- ✓ Say what `TA_FLAG_DEVICE_ENUM` enables and what fails without it.
- ✓ Bind a kernel driver to a TA by UUID.
- ✓ Determine, for a hung client, whether it is blocked in the secure world.
- ✓ Explain why a kernel client must not pass a `kmalloc` buffer directly.

The Client API in depth

Eleven functions, defined by GlobalPlatform, implemented by `libteec`, and identical across every compliant TEE. Learn them here and your client code moves to Kinibi or QSEE unchanged. This chapter is the complete surface, with the lifecycle rules that the specification states and everyone violates anyway.

The whole API

Function	Does
`TEEC_InitializeContext`	Connect to a TEE. Opens `/dev/tee0`
`TEEC_FinalizeContext`	Disconnect
`TEEC_OpenSession`	Start talking to a TA, identified by UUID
`TEEC_CloseSession`	Stop
`TEEC_InvokeCommand`	Ask the TA to do something
`TEEC_RequestCancellation`	Ask for an in-flight operation to stop
`TEEC_RegisterSharedMemory`	Give the TEE memory you already own
`TEEC_AllocateSharedMemory`	Have the TEE allocate it
`TEEC_ReleaseSharedMemory`	Give it back
`TEEC_RegisterSharedMemoryFileDescriptor`	OP-TEE extension: share a dmabuf
`TEEC_PARAM_TYPES`	A macro that packs four parameter types into one word

The lifecycle, and what it costs

```
   TEEC_InitializeContext
      opens /dev/tee0, queries the driver
      COST: an open() and an ioctl. Cheap-ish.
      |
      v
   TEEC_OpenSession
      -> SMC -> OP-TEE looks up the UUID
      -> if the TA is not resident: RPC to tee-supplicant to
         read the .ta file, then VERIFY ITS SIGNATURE, then
         allocate TA_RAM, map it, run TA_CreateEntryPoint
      -> then TA_OpenSessionEntryPoint
      COST: THE EXPENSIVE ONE. Several milliseconds the first
      time - signature verification of a 100 KB binary is real
      work.
      |
      v
   TEEC_InvokeCommand    (many times)
      -> SMC -> TA_InvokeCommandEntryPoint
      COST: two world switches plus your TA's work. Tens of
      microseconds of overhead.
      |
      v
   TEEC_CloseSession
      -> TA_CloseSessionEntryPoint
      -> if this was the LAST session: TA_DestroyEntryPoint,
         and the TA is unloaded from TA_RAM (unless it is a
         singleton that keeps its instance alive)
      |
      v
   TEEC_FinalizeContext

   THE PERFORMANCE RULE THAT FOLLOWS:

     open the session ONCE, at start-up, and keep it.

   A client that opens and closes a session around every
   operation pays the load-and-verify cost every time. This is
   the single most common OP-TEE performance mistake, and it
   can be a 100x difference.
```

A complete, correct client

```
/* keystore_client.c - the error handling and cleanup are the
   point. Copy this shape.                                    */

#include <err.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <tee_client_api.h>

#define TA_UUID { 0x8aaaf200, 0x2450, 0x11e4, \
        { 0xab, 0xe2, 0x00, 0x02, 0xa5, 0xd5, 0xc5, 0x1b } }

#define CMD_SIGN   0

struct tee_ctx {
        TEEC_Context      ctx;
        TEEC_Session      sess;
        TEEC_SharedMemory io;
        int               have_ctx, have_sess, have_shm;
};

static const char *origin_str(uint32_t o)
{
        switch (o) {
        case TEEC_ORIGIN_API:         return "API";
        case TEEC_ORIGIN_COMMS:       return "COMMS";
        case TEEC_ORIGIN_TEE:         return "TEE";
        case TEEC_ORIGIN_TRUSTED_APP: return "TA";
        default:                      return "?";
        }
}

static void tee_fini(struct tee_ctx *t)
{
        if (t->have_shm)  TEEC_ReleaseSharedMemory(&t->io);
        if (t->have_sess) TEEC_CloseSession(&t->sess);
        if (t->have_ctx)  TEEC_FinalizeContext(&t->ctx);
        memset(t, 0, sizeof *t);
}

static int tee_init(struct tee_ctx *t, size_t bufsz)
{
        TEEC_UUID   uuid = TA_UUID;
        uint32_t    origin;
        TEEC_Result res;

        memset(t, 0, sizeof *t);

        res = TEEC_InitializeContext(NULL, &t->ctx);
        if (res != TEEC_SUCCESS) {
                warnx("InitializeContext: 0x%08x", res);
                return -1;
        }
        t->have_ctx = 1;

        res = TEEC_OpenSession(&t->ctx, &t->sess, &uuid,
                               TEEC_LOGIN_PUBLIC, NULL, NULL,
                               &origin);
        if (res != TEEC_SUCCESS) {
                warnx("OpenSession: 0x%08x origin %s",
```

```
                        res, origin_str(origin));
                goto fail;
        }
        t->have_sess = 1;

        /* Allocate ONCE and reuse - see the note below. */
        t->io.size  = bufsz;
        t->io.flags = TEEC_MEM_INPUT | TEEC_MEM_OUTPUT;
        res = TEEC_AllocateSharedMemory(&t->ctx, &t->io);
        if (res != TEEC_SUCCESS) {
                warnx("AllocateSharedMemory: 0x%08x", res);
                goto fail;
        }
        t->have_shm = 1;
        return 0;
fail:
        tee_fini(t);
        return -1;
}

static int tee_sign(struct tee_ctx *t, const void *msg, size_t len,
                    void *sig, size_t *siglen)
{
        TEEC_Operation op;
        uint32_t       origin;
        TEEC_Result    res;

        if (len > t->io.size)
                return -1;
        memcpy(t->io.buffer, msg, len);

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_PARTIAL_INPUT,
                                         TEEC_MEMREF_PARTIAL_OUTPUT,
                                         TEEC_NONE, TEEC_NONE);
        op.params[0].memref.parent = &t->io;
        op.params[0].memref.offset = 0;
        op.params[0].memref.size   = len;
        op.params[1].memref.parent = &t->io;
        op.params[1].memref.offset = 2048;      /* second half   */
        op.params[1].memref.size   = *siglen;

        res = TEEC_InvokeCommand(&t->sess, CMD_SIGN, &op, &origin);

        /* THE SHORT-BUFFER RETRY. Every real client needs it. */
        if (res == TEEC_ERROR_SHORT_BUFFER) {
                warnx("need %u bytes, had %zu",
                      op.params[1].memref.size, *siglen);
                *siglen = op.params[1].memref.size;
                return -2;      /* caller reallocates, retries  */
        }
        if (res != TEEC_SUCCESS) {
                warnx("InvokeCommand: 0x%08x origin %s",
                      res, origin_str(origin));
                return -1;
        }

        *siglen = op.params[1].memref.size;
        memcpy(sig, (char *)t->io.buffer + 2048, *siglen);
        return 0;
}

int main(void)
```

```
{
        struct tee_ctx t;
        uint8_t sig[512];
        size_t  siglen = sizeof sig;

        if (tee_init(&t, 4096))
                return 1;

        /* Many invocations, ONE session. */
        for (int i = 0; i < 1000; i++) {
                if (tee_sign(&t, "hello", 5, sig, &siglen) < 0)
                        break;
                siglen = sizeof sig;
        }

        tee_fini(&t);
        return 0;
}
```

> **ALLOCATE SHARED MEMORY ONCE**
> `TEEC_AllocateSharedMemory` is not free — it goes to the kernel and possibly to OP-TEE. Allocating a buffer per invocation, in a loop, doubles your overhead for no benefit. Allocate one buffer at start-up, big enough for your largest transfer, and use `TEEC_MEMREF_PARTIAL_INPUT` / `OUTPUT` with offsets to carve it up.

Login types and connection data

```
/* The fifth argument to TEEC_OpenSession says WHO is calling.
   The TA can see it, and OP-TEE derives per-caller storage
   keys from it (chapter 15).                                  */

  TEEC_LOGIN_PUBLIC        no identity claimed. Anyone.
  TEEC_LOGIN_USER          the caller's uid, hashed in by the
                           kernel driver - the client CANNOT
                           forge it
  TEEC_LOGIN_GROUP         a gid, supplied in connectionData
  TEEC_LOGIN_APPLICATION   an application identity
  TEEC_LOGIN_USER_APPLICATION
  TEEC_LOGIN_GROUP_APPLICATION
  TEEC_LOGIN_REE_KERNEL    the caller is the kernel (ch. 10)

    uint32_t gid = 1000;
    res = TEEC_OpenSession(&ctx, &sess, &uuid,
                           TEEC_LOGIN_GROUP, &gid,
                           NULL, &origin);

/* Why it matters: with LOGIN_USER, two different users opening
   sessions to the same TA get DIFFERENT secure storage. The
   TA writes to "my key" and each caller gets their own. That
   separation is enforced by OP-TEE, using an identity the
   client cannot choose.

   With LOGIN_PUBLIC there is one shared storage space for
   everyone. For a multi-user or multi-app system that is
   almost certainly wrong.                                     */
```

Cancellation

```
/* A long-running TA operation can be asked to stop. It is
   COOPERATIVE - the TA must check.                          */

/* Client, from another thread: */
    TEEC_RequestCancellation(&op);

/* TA side: */
    if (TEE_GetCancellationFlag()) {
            cleanup();
            return TEE_ERROR_CANCEL;
    }

    /* A TA can also mask cancellation while in a critical
       section, and must restore it: */
    bool old = TEE_MaskCancellation();
    do_something_uninterruptible();
    if (old) TEE_MaskCancellation(); else TEE_UnmaskCancellation();

/* Uses:
     a watchdog thread that cancels after a timeout
     a UI "cancel" button on a slow key generation
     shutdown

   Limits:
     the TA must poll the flag. A TA blocked in an RPC will
     not see it until the RPC returns.
     There is NO way to forcibly kill a TA invocation. If your
     TA can loop for ever, you have created an unkillable
     thread (chapter 5).                                    */
```

Thread safety

```
/* The GlobalPlatform rules, which OP-TEE follows:

   TEEC_Context   may be shared between threads.
   TEEC_Session   may be shared, BUT concurrent
                  TEEC_InvokeCommand calls on ONE session are
                  serialised by the TA's own instance/session
                  flags (chapter 13). Whether they run
                  concurrently depends on TA_FLAG_*.
   TEEC_Operation must NOT be shared. One per in-flight call.
   TEEC_SharedMemory may be shared, but you are responsible
                  for not having two calls write the same
                  region.

   The practical pattern for a multithreaded client:

     one TEEC_Context           process-wide
     one TEEC_Session           process-wide, if the TA is
                                multi-session capable
     one TEEC_Operation         per call, on the stack
     one TEEC_SharedMemory      per thread, allocated once

   If the TA is TA_FLAG_SINGLE_INSTANCE without
   TA_FLAG_MULTI_SESSION, your threads will serialise inside
   OP-TEE no matter what the client does. Know which your TA
   is.                                                      */
```

How it shows up

Symptom	Cause
Every operation takes milliseconds	Session opened and closed per call — the TA is being loaded and verified each time
`TEEC_ERROR_ITEM_NOT_FOUND` on `OpenSession`	TA not installed, or `tee-supplicant` not running
`TEEC_ERROR_BUSY` intermittently	Thread pool exhausted; libteec retries but concurrency is capped
Corruption with multiple threads	A `TEEC_Operation` or a shared buffer used by two calls at once
Client cannot cancel a stuck TA	Cancellation is cooperative; the TA is not polling the flag
Two users see each other's stored data	`TEEC_LOGIN_PUBLIC` where `LOGIN_USER` was needed
Retry loop on `SHORT_BUFFER` never ends	The TA does not set `memref.size` to the required length
Memory grows over time	Shared memory or sessions not released on the error path
Works alone, fails under concurrency	TA flags serialise sessions; see chapter 13

Check yourself

- ✓ Name the eleven Client API functions and what each does.
- ✓ Describe the lifecycle and identify the expensive step.
- ✓ State the performance rule about sessions and quantify why.
- ✓ Explain why shared memory should be allocated once and carved with partial memrefs.
- ✓ Write a client with correct cleanup on every error path.
- ✓ Implement the short-buffer retry.
- ✓ Name the login types and explain what changes with `LOGIN_USER`.
- ✓ Explain why the client cannot forge its uid.
- ✓ Describe cancellation, what the TA must do, and its two limits.
- ✓ State which Client API objects may be shared between threads and which may not.

The TA Internal Core API in depth

Inside a trusted application there is no libc, no filesystem, no sockets and no `printf` — there is one specified API covering memory, cryptography, storage, time and arithmetic. It is deliberately small and deliberately awkward in places; this chapter is the whole of it, organised by what you actually reach for.

The five entry points

```
/* Every TA implements exactly these. They are the TA's whole
   interface to the world.                                    */

TEE_Result TA_CreateEntryPoint(void)
{
        /* Called ONCE, when the TA is loaded into memory,
           before any session exists. Initialise global state.
           Returning an error prevents the TA loading at all. */
        DMSG("has been called");
        return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
        /* Called once, when the last session closes and the TA
           is unloaded. Free global state. Cannot fail.       */
}

TEE_Result TA_OpenSessionEntryPoint(uint32_t param_types,
                                    TEE_Param params[4],
                                    void **sess_ctx)
{
        /* Once per client session. Allocate per-session state
           and store the pointer in *sess_ctx - it comes back
           on every later call for this session.

           params[] are whatever the client passed to
           TEEC_OpenSession. Check param_types here too.      */
        struct my_session *s = TEE_Malloc(sizeof *s, 0);
        if (!s)
                return TEE_ERROR_OUT_OF_MEMORY;
        *sess_ctx = s;
        return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess_ctx)
{
        TEE_Free(sess_ctx);
}

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx,
                                      uint32_t cmd_id,
                                      uint32_t param_types,
                                      TEE_Param params[4])
{
        /* THE REAL WORK. One switch on cmd_id.              */
        switch (cmd_id) {
        case CMD_SIGN:   return do_sign(sess_ctx, param_types, params);
        case CMD_VERIFY: return do_verify(sess_ctx, param_types, params);
        default:         return TEE_ERROR_NOT_SUPPORTED;
        }
}
```

Memory

```
void *TEE_Malloc(size_t size, uint32_t hint);
void *TEE_Realloc(void *buffer, uint32_t newSize);
void  TEE_Free(void *buffer);

/* hint:
     TEE_MALLOC_FILL_ZERO   (0)  zeroed. The default, and what
                                 you almost always want.
     TEE_USER_MEM_HINT_NO_FILL_ZERO   uninitialised, faster   */

void TEE_MemMove(void *dest, const void *src, uint32_t size);
int32_t TEE_MemCompare(const void *b1, const void *b2, uint32_t size);
void TEE_MemFill(void *buf, uint32_t x, uint32_t size);

/* TEE_MemCompare is NOT constant time. For comparing a MAC or
   a password against a stored value, that is a timing oracle.
   Write your own:                                            */

static bool ct_equal(const uint8_t *a, const uint8_t *b, size_t n)
{
        uint8_t diff = 0;
        for (size_t i = 0; i < n; i++)
                diff |= a[i] ^ b[i];
        return diff == 0;
}

/* Validating a pointer the client gave you: */
TEE_Result TEE_CheckMemoryAccessRights(uint32_t accessFlags,
                                       void *buffer, uint32_t size);
    TEE_MEMORY_ACCESS_READ
    TEE_MEMORY_ACCESS_WRITE
    TEE_MEMORY_ACCESS_ANY_OWNER    /* it may be shared memory  */
    TEE_MEMORY_ACCESS_SECURE       /* it must be secure memory */

/* Giving up: */
void TEE_Panic(TEE_Result panicCode);   /* never returns. The
                                           session dies, the
                                           client gets
                                           TEEC_ERROR_TARGET_DEAD,
                                           and a stack dump
                                           appears on the secure
                                           console.            */
```

Cryptography

```
/* The pattern is always the same: allocate an OPERATION,
   allocate and populate an OBJECT for the key, attach one to
   the other, then init/update/final.                        */

/* ----- 1. an operation ----- */
TEE_Result TEE_AllocateOperation(TEE_OperationHandle *op,
                                 uint32_t algorithm,
                                 uint32_t mode,
                                 uint32_t maxKeySize);
void TEE_FreeOperation(TEE_OperationHandle op);
void TEE_ResetOperation(TEE_OperationHandle op);

/* mode: TEE_MODE_ENCRYPT DECRYPT SIGN VERIFY MAC DIGEST DERIVE */

/* ----- 2. a key object ----- */
TEE_Result TEE_AllocateTransientObject(uint32_t objectType,
                                       uint32_t maxKeySize,
                                       TEE_ObjectHandle *object);
TEE_Result TEE_PopulateTransientObject(TEE_ObjectHandle object,
                                       const TEE_Attribute *attrs,
                                       uint32_t attrCount);
TEE_Result TEE_GenerateKey(TEE_ObjectHandle object,
                           uint32_t keySize,
                           const TEE_Attribute *params,
                           uint32_t paramCount);
void TEE_FreeTransientObject(TEE_ObjectHandle object);

TEE_Result TEE_SetOperationKey(TEE_OperationHandle op,
                               TEE_ObjectHandle key);

/* ----- 3. use it ----- */
void TEE_CipherInit(TEE_OperationHandle op, const void *IV,
                    uint32_t IVLen);
TEE_Result TEE_CipherUpdate(TEE_OperationHandle op,
                            const void *src, uint32_t srcLen,
                            void *dest, uint32_t *destLen);
TEE_Result TEE_CipherDoFinal(...);

TEE_Result TEE_DigestUpdate(...);
TEE_Result TEE_DigestDoFinal(...);

TEE_Result TEE_MACInit(...);   TEE_MACUpdate(...);
TEE_Result TEE_MACComputeFinal(...);
TEE_Result TEE_MACCompareFinal(...);   /* constant time - USE
                                          THIS rather than
                                          comparing yourself  */

TEE_Result TEE_AEInit(...);    TEE_AEUpdateAAD(...);
TEE_Result TEE_AEEncryptFinal(...);  TEE_AEDecryptFinal(...);

TEE_Result TEE_AsymmetricSignDigest(TEE_OperationHandle op,
                                    const TEE_Attribute *params,
                                    uint32_t paramCount,
                                    const void *digest, uint32_t digestLen,
                                    void *signature, uint32_t *signatureLen);
TEE_Result TEE_AsymmetricVerifyDigest(...);
TEE_Result TEE_AsymmetricEncrypt(...);  TEE_AsymmetricDecrypt(...);

TEE_Result TEE_DeriveKey(TEE_OperationHandle op,
                         const TEE_Attribute *params,
```

```
                        uint32_t paramCount,
                        TEE_ObjectHandle derivedKey);

void TEE_GenerateRandom(void *randomBuffer, uint32_t randomBufferLen);
```

```
/* A complete AES-GCM encrypt, showing the shape end to end. */

static TEE_Result encrypt(const uint8_t *key, size_t keylen,
                          const uint8_t *nonce, size_t noncelen,
                          const uint8_t *pt, size_t ptlen,
                          uint8_t *ct, size_t *ctlen,
                          uint8_t *tag, size_t *taglen)
{
        TEE_OperationHandle op = TEE_HANDLE_NULL;
        TEE_ObjectHandle    ko = TEE_HANDLE_NULL;
        TEE_Attribute       attr;
        TEE_Result          res;

        res = TEE_AllocateOperation(&op, TEE_ALG_AES_GCM,
                                    TEE_MODE_ENCRYPT, keylen * 8);
        if (res) goto out;

        res = TEE_AllocateTransientObject(TEE_TYPE_AES,
                                          keylen * 8, &ko);
        if (res) goto out;

        TEE_InitRefAttribute(&attr, TEE_ATTR_SECRET_VALUE,
                             key, keylen);
        res = TEE_PopulateTransientObject(ko, &attr, 1);
        if (res) goto out;

        res = TEE_SetOperationKey(op, ko);
        if (res) goto out;

        res = TEE_AEInit(op, nonce, noncelen, *taglen * 8, 0, ptlen);
        if (res) goto out;

        res = TEE_AEEncryptFinal(op, pt, ptlen, ct, ctlen,
                                 tag, taglen);
out:
        /* Both are safe to call on TEE_HANDLE_NULL. */
        TEE_FreeOperation(op);
        TEE_FreeTransientObject(ko);
        return res;
}

/* Note TEE_FreeTransientObject on the key: the key material is
   wiped when the object is freed. Never keep raw key bytes in
   a local array longer than necessary, and TEE_MemFill(0) any
   you do keep.                                                 */
```

HANDLES ARE OPAQUE AND MUST BE FREED ON EVERY PATH

`TEE_OperationHandle` and `TEE_ObjectHandle` are allocations inside OP-TEE. A TA that leaks them exhausts its own heap and eventually fails with `TEE_ERROR_OUT_OF_MEMORY` — after running correctly for hours. Initialise every handle to `TEE_HANDLE_NULL` and use a single `goto out` cleanup path, as above; the free functions accept NULL precisely so this pattern works.

Persistent storage

```
TEE_Result TEE_CreatePersistentObject(uint32_t storageID,
                                      const void *objectID,
                                      uint32_t objectIDLen,
                                      uint32_t flags,
                                      TEE_ObjectHandle attributes,
                                      const void *initialData,
                                      uint32_t initialDataLen,
                                      TEE_ObjectHandle *object);
TEE_Result TEE_OpenPersistentObject(...);
void       TEE_CloseObject(TEE_ObjectHandle object);
TEE_Result TEE_CloseAndDeletePersistentObject1(TEE_ObjectHandle object);

TEE_Result TEE_ReadObjectData(TEE_ObjectHandle object, void *buffer,
                              uint32_t size, uint32_t *count);
TEE_Result TEE_WriteObjectData(TEE_ObjectHandle object,
                               const void *buffer, uint32_t size);
TEE_Result TEE_SeekObjectData(TEE_ObjectHandle object,
                              int32_t offset, TEE_Whence whence);
TEE_Result TEE_TruncateObjectData(TEE_ObjectHandle object,
                                  uint32_t size);
TEE_Result TEE_GetObjectInfo1(TEE_ObjectHandle object,
                              TEE_ObjectInfo *objectInfo);

/* storageID:
     TEE_STORAGE_PRIVATE        the default; REE FS or RPMB
                                depending on build config
     TEE_STORAGE_PRIVATE_REE    explicitly the normal-world FS
     TEE_STORAGE_PRIVATE_RPMB   explicitly RPMB

   flags:
     TEE_DATA_FLAG_ACCESS_READ
     TEE_DATA_FLAG_ACCESS_WRITE
     TEE_DATA_FLAG_ACCESS_WRITE_META    needed to delete/rename
     TEE_DATA_FLAG_SHARE_READ / SHARE_WRITE
     TEE_DATA_FLAG_OVERWRITE            create even if it exists

   The object ID is an arbitrary byte string YOU choose - it is
   not a filename. Storage is per-TA and, with LOGIN_USER, per
   caller (chapter 15).                                        */
```

Time, properties and arithmetic

```
/* TIME - three sources, three trust levels. */

void TEE_GetSystemTime(TEE_Time *time);
    /* monotonic since boot, from the secure world. TRUSTED
       for measuring intervals. Not wall-clock.              */

TEE_Result TEE_GetREETime(TEE_Time *time);
    /* wall-clock, FROM THE NORMAL WORLD. An attacker can set
       it to anything. NEVER use for expiry or replay
       protection (chapter 17).                              */

TEE_Result TEE_GetTAPersistentTime(TEE_Time *time);
TEE_Result TEE_SetTAPersistentTime(const TEE_Time *time);
    /* a per-TA time that survives reboot, stored securely.
       Returns TEE_ERROR_TIME_NOT_SET / TIME_NEEDS_RESET so
       you can tell whether to trust it.                     */

TEE_Result TEE_Wait(uint32_t timeout);   /* an RPC - holds a
                                            thread! (ch. 9)   */

/* PROPERTIES - configuration, declared in the TA's header or
   provided by the implementation.                           */

TEE_Result TEE_GetPropertyAsString(TEE_PropSetHandle propsetOrEnum,
                                   const char *name,
                                   char *valueBuffer,
                                   uint32_t *valueBufferLen);
TEE_Result TEE_GetPropertyAsBool(...);
TEE_Result TEE_GetPropertyAsU32(...);
TEE_Result TEE_GetPropertyAsBinaryBlock(...);
TEE_Result TEE_GetPropertyAsUUID(...);
TEE_Result TEE_GetPropertyAsIdentity(...);

    TEE_PROPSET_TEE_IMPLEMENTATION   about the TEE
    TEE_PROPSET_CURRENT_TA           about this TA
    TEE_PROPSET_CURRENT_CLIENT       about the caller -
                                     including its identity,
                                     which is how a TA
                                     authorises a client

/* Reading the client's identity: */
    TEE_Identity id;
    TEE_GetPropertyAsIdentity(TEE_PROPSET_CURRENT_CLIENT,
                              "gpd.client.identity", &id);
    /* id.login is the login type; id.uuid is derived from the
       uid/gid, and the client cannot forge it.              */

/* BIG NUMBERS - because a TA may need arithmetic the crypto
   API does not cover. Rarely needed, and slow.              */

void TEE_BigIntInit(TEE_BigInt *bigInt, uint32_t len);
void TEE_BigIntAdd/Sub/Mul/Div(...);
void TEE_BigIntExpMod(...);
TEE_Result TEE_BigIntConvertFromOctetString(...);
```

Tracing from inside a TA

```
/* There is no printf. There are these, from
   lib/libutee/include/trace.h - they go to the SECURE console. */

    EMSG("error: %d", x);      /* always shown                 */
    IMSG("info: %s", s);       /* CFG_TEE_TA_LOG_LEVEL >= 2    */
    DMSG("debug: %p", p);      /* >= 3                          */
    FMSG("flow");              /* >= 4                          */

    EMSG_RAW("no prefix\n");
    DHEXDUMP(buf, len);        /* a hex dump - very useful     */

/* Output:
     E/TA: do_sign:112 error: -1
     I/TA: something happened
   with the function name and line number added for you.

   NEVER log key material. DHEXDUMP of a buffer that might be a
   key is a habit that ships. If you need it during
   development, guard it:
     #ifdef DEBUG_SECRETS
     DHEXDUMP(key, keylen);
     #endif
   and make sure DEBUG_SECRETS cannot be defined in a release
   build.                                                    */
```

How it shows up

Symptom	Cause
TA works for hours then fails with out-of-memory	Leaked operation or object handles on an error path
`TEEC_ERROR_TARGET_DEAD`	`TEE_Panic`, a bad pointer, or an assertion. Read the secure console
`TEE_ERROR_BAD_PARAMETERS` from a crypto call	Algorithm and key type mismatch, or key size outside the operation's `maxKeySize`
`TEE_ERROR_NOT_SUPPORTED`	That algorithm is not compiled in — check the `CFG_CRYPTO_*` flags
MAC verification is a timing oracle	Used `TEE_MemCompare` instead of `TEE_MACCompareFinal`
Expiry check defeated trivially	Used `TEE_GetREETime`, which the attacker controls
Storage works for one user, collides for another	Login type and storage scoping — see chapter 15
Key material found in a memory dump	Raw key bytes left in a local buffer; wipe with `TEE_MemFill`
A TA hangs the system	`TEE_Wait` or a long loop holding a thread
Secrets in the log	`DHEXDUMP` of key material left in a release build

Check yourself

- ✓ Name the five entry points and say exactly when each is called.
- ✓ Explain what `sess_ctx` is for and where it is allocated and freed.
- ✓ Write the allocate-populate-attach-use pattern for a symmetric cipher.
- ✓ Explain why every handle must be freed on every path, and give the pattern that ensures it.
- ✓ Say why `TEE_MemCompare` is unsafe for MAC verification and what to use instead.

- ✓ Name the three time sources and rank them by trustworthiness.
- ✓ Explain how a TA discovers who its client is, and why that identity cannot be forged.
- ✓ Give the four trace macros and the log level each needs.
- ✓ Explain what `TEE_Panic` does to the session and what the client sees.
- ✓ List the persistent-storage flags and say which one is needed to delete an object.

TA lifecycle, properties and flags

Four flags in a header file decide whether your TA can serve several clients at once, whether it keeps state between them, and whether the kernel can see it. Choosing them wrongly produces bugs that only appear under concurrency, which is the worst kind to find late.

The header that configures everything

```
/* ta/include/user_ta_header_defines.h - every TA has one.   */

#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#include "my_ta.h"      /* where TA_MY_UUID is defined        */

#define TA_UUID  TA_MY_UUID

#define TA_FLAGS            (TA_FLAG_SINGLE_INSTANCE | \
                             TA_FLAG_MULTI_SESSION | \
                             TA_FLAG_INSTANCE_KEEP_ALIVE)

#define TA_STACK_SIZE       (2 * 1024)      /* per session     */
#define TA_DATA_SIZE        (32 * 1024)     /* the TA's heap   */

#define TA_CURRENT_TA_EXT_PROPERTIES \
    { "gp.ta.description", USER_TA_PROP_TYPE_STRING, \
      "My example TA" }, \
    { "gp.ta.version", USER_TA_PROP_TYPE_U32, &(const uint32_t){ 0x0100 } }

#endif
```

The flags

Flag	Meaning	Consequence
`TA_FLAG_SINGLE_INSTANCE`	One copy of the TA serves all clients	Global state is shared between sessions. Without it, each client gets its own instance and its own globals
`TA_FLAG_MULTI_SESSION`	The instance accepts more than one session at a time	Without it, a second client waits until the first closes
`TA_FLAG_INSTANCE_KEEP_ALIVE`	The instance survives when the last session closes	Global state and `TA_CreateEntryPoint` work persist. Costs TA_RAM permanently
`TA_FLAG_DEVICE_ENUM`	Advertise this TA to the Linux TEE bus	Required for a kernel driver to bind (chapter 10)
`TA_FLAG_SECURE_DATA_PATH`	May receive secure-memory buffers	For media pipelines
`TA_FLAG_CACHE_MAINTENANCE`	May call `TEE_Cache*`	Only with `CFG_CACHE_API=y`

```
The four combinations that matter, and what each is for.

1. SINGLE_INSTANCE | MULTI_SESSION | KEEP_ALIVE
   One instance, always resident, many concurrent clients,
   shared global state.
   -> A SERVICE. The fTPM is exactly this: one TPM, whose
      PCRs and NV storage are global, serving anyone.
   -> You are now responsible for thread safety of your own
      globals. OP-TEE does NOT serialise for you.

2. SINGLE_INSTANCE | MULTI_SESSION       (no keep-alive)
   Same, but unloaded when idle. Saves TA_RAM; pays the
   load-and-verify cost on the next first session.
   -> A frequently-but-not-constantly used service.

3. SINGLE_INSTANCE only
   One instance, ONE session at a time. Clients serialise.
   -> The safe default when you have shared state and do
      not want to think about locking. Simple and slow.

4. neither SINGLE_INSTANCE nor MULTI_SESSION
   A fresh instance per client, each with its own globals
   and its own TA_RAM allocation.
   -> Perfect isolation between clients, at the cost of
      memory. Good for a per-application key handler.

THE TRAP: combination 1 with global variables and no
locking. Two clients invoke concurrently, both touch a
global, and you have a data race inside the secure world.
It will not show up in testing with one client.
```

> **`MULTI_SESSION` MEANS YOU MUST LOCK**
>
> With `SINGLE_INSTANCE | MULTI_SESSION`, OP-TEE will run two `TA_InvokeCommandEntryPoint` calls concurrently on different threads, in the same address space, over the same globals. There is no implicit mutex. If your TA has mutable global state — a PCR bank, a key cache, a counter — you must protect it yourself. If you would rather not, drop `MULTI_SESSION` and accept serialisation.

Where state lives

```
THREE scopes, three lifetimes.

GLOBAL VARIABLES
    lifetime: from TA_CreateEntryPoint to
              TA_DestroyEntryPoint
    shared:   between all sessions, IF SINGLE_INSTANCE
    lost on:  TA unload (unless KEEP_ALIVE), reboot
    use for:  a service's shared state; caches

SESSION CONTEXT (*sess_ctx)
    lifetime: TA_OpenSessionEntryPoint to
              TA_CloseSessionEntryPoint
    shared:   only within that session
    lost on:  session close, TA panic, client death
    use for:  per-client keys, cursors, in-progress
              operations

PERSISTENT OBJECTS
    lifetime: until explicitly deleted
    shared:   per-TA, and per-client if the login type
              scopes it (chapter 15)
    lost on:  explicit delete, factory reset, or a normal
              world that deletes the backing file
    use for:  keys, counters, anything that must survive
              reboot

A common design error: keeping a key in a global "for
performance" in a SINGLE_INSTANCE TA, so that client B can
use the key client A loaded. Put per-client material in the
session context.
```

A TA with all three

```
#include <tee_internal_api.h>
#include <tee_internal_api_extensions.h>
#include <string.h>
#include "my_ta.h"

/* --- GLOBAL: shared, needs locking under MULTI_SESSION --- */
static uint32_t g_invocations;
static bool     g_initialised;

/* --- SESSION: per client --- */
struct session {
        TEE_ObjectHandle key;     /* this client's key         */
        uint32_t         counter;
        uint8_t          nonce[12];
};

TEE_Result TA_CreateEntryPoint(void)
{
        DMSG("TA loaded");
        g_invocations = 0;
        g_initialised = true;
        return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
        DMSG("TA unloaded after %u invocations", g_invocations);
        g_initialised = false;
}

TEE_Result TA_OpenSessionEntryPoint(uint32_t ptypes,
                                    TEE_Param params[4],
                                    void **sess_ctx)
{
        struct session *s;

        /* Even here, check the types. */
        if (ptypes != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE))
                return TEE_ERROR_BAD_PARAMETERS;

        s = TEE_Malloc(sizeof *s, TEE_MALLOC_FILL_ZERO);
        if (!s)
                return TEE_ERROR_OUT_OF_MEMORY;

        s->key = TEE_HANDLE_NULL;
        TEE_GenerateRandom(s->nonce, sizeof s->nonce);

        /* Who is calling? */
        {
                TEE_Identity id;
                TEE_Result r = TEE_GetPropertyAsIdentity(
                        TEE_PROPSET_CURRENT_CLIENT,
                        "gpd.client.identity", &id);
                if (r == TEE_SUCCESS)
                        DMSG("client login type 0x%08x", id.login);
        }
```

```
        *sess_ctx = s;
        return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess_ctx)
{
        struct session *s = sess_ctx;

        if (!s)
                return;
        if (s->key != TEE_HANDLE_NULL)
                TEE_FreeTransientObject(s->key);
        /* Wipe before freeing - the heap is reused. */
        TEE_MemFill(s, 0, sizeof *s);
        TEE_Free(s);
}

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx,
                                      uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        struct session *s = sess_ctx;

        /* Global mutation. Under MULTI_SESSION this is a race.
           OP-TEE has no user-space mutex API for TAs, so
           either drop MULTI_SESSION, or make the update
           genuinely atomic, or keep the counter per session. */
        g_invocations++;

        s->counter++;

        switch (cmd) {
        case TA_CMD_LOAD_KEY:   return load_key(s, ptypes, params);
        case TA_CMD_SIGN:       return sign(s, ptypes, params);
        case TA_CMD_STATS:
                if (ptypes != TEE_PARAM_TYPES(
                                TEE_PARAM_TYPE_VALUE_OUTPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE))
                        return TEE_ERROR_BAD_PARAMETERS;
                params[0].value.a = s->counter;   /* per session */
                params[0].value.b = g_invocations;/* global      */
                return TEE_SUCCESS;
        default:
                return TEE_ERROR_NOT_SUPPORTED;
        }
}
```

Sizing stack and heap

```
#define TA_STACK_SIZE  (2 * 1024)    /* PER SESSION            */
#define TA_DATA_SIZE   (32 * 1024)   /* the TA's whole heap    */

/* TA_STACK_SIZE: the C stack for TA code. 2 KB is the common
   default and it is small. Things that overflow it:
       - a large local buffer  (uint8_t buf[4096];)
       - deep recursion
       - a crypto call chain with its own locals
   Symptom: a panic with a stack dump, often in an unrelated
   function, and often only for large inputs.

   Do not put big buffers on the stack. TEE_Malloc them, or
   raise TA_STACK_SIZE deliberately.                          */

/* TA_DATA_SIZE: everything TEE_Malloc returns, plus the
   internal allocations OP-TEE makes on the TA's behalf -
   operation handles, object handles, key material.
   Symptom of running out: TEE_ERROR_OUT_OF_MEMORY, usually
   after a leak rather than a genuine shortage.              */

/* Measuring: build with CFG_TA_STATS=y and ask.             */
$ xtest --stats
TA stats:
  <uuid>
    Heap: allocated 8192, max allocated 20480, size 32768
    Stack: max used 1152, size 2048

/* "max used" is the number that matters. 1152 of 2048 is
   comfortable; 1900 of 2048 is a crash waiting for a longer
   input.                                                     */
```

Properties

```
/* Declared in the header, readable by the TA and, for some,
   by the client.                                          */

#define TA_CURRENT_TA_EXT_PROPERTIES \
    { "gp.ta.description", USER_TA_PROP_TYPE_STRING, \
      "Secure key store" }, \
    { "gp.ta.version", USER_TA_PROP_TYPE_U32, \
      &(const uint32_t){ 0x0102 } }, \
    { "com.example.max-keys", USER_TA_PROP_TYPE_U32, \
      &(const uint32_t){ 16 } }

/* Read them: */
    uint32_t maxkeys;
    TEE_GetPropertyAsU32(TEE_PROPSET_CURRENT_TA,
                         "com.example.max-keys", &maxkeys);

/* Standard properties every TA has: */
    gpd.ta.appID            this TA's UUID
    gpd.ta.singleInstance   bool, from the flags
    gpd.ta.multiSession     bool
    gpd.ta.instanceKeepAlive bool
    gpd.ta.dataSize         TA_DATA_SIZE
    gpd.ta.stackSize        TA_STACK_SIZE

/* About the TEE itself: */
    gpd.tee.apiversion
    gpd.tee.internalCore.version
    gpd.tee.trustedos.implementation.version
    gpd.tee.deviceID        a per-device identifier derived
                            from the hardware unique key -
                            useful, and privacy-sensitive

/* About the caller: */
    gpd.client.identity     login type + uuid (chapter 12)
```

How it shows up

Symptom	Cause
Works with one client, corrupts with two	`MULTI_SESSION` with unprotected global state
Second client blocks until the first finishes	`MULTI_SESSION` not set — this may be what you wanted
Global state lost between clients	No `INSTANCE_KEEP_ALIVE`; the TA unloaded when the last session closed
Each client sees its own globals	`SINGLE_INSTANCE` not set
No device under `/sys/bus/tee/devices/`	`TA_FLAG_DEVICE_ENUM` missing
Panic with a stack dump on large inputs	`TA_STACK_SIZE` too small — a big local buffer
`TEE_ERROR_OUT_OF_MEMORY` after long uptime	Leaked handles or allocations; `TA_DATA_SIZE` exhausted
One client can use another's key	Key held in a global rather than the session context
Session context is NULL in a callback	`*sess_ctx` not set, or set after an early error return
Secrets recovered from TA heap	Session structure freed without being wiped

Check yourself

- ✓ Name the TA flags and say what each changes.
- ✓ Give the four useful flag combinations and a use case for each.
- ✓ Explain what `SINGLE_INSTANCE | MULTI_SESSION` obliges you to do.
- ✓ Name the three state scopes, their lifetimes, and what belongs in each.
- ✓ Explain why a per-client key must not live in a global.
- ✓ Say what `TA_STACK_SIZE` and `TA_DATA_SIZE` control and how to measure usage.
- ✓ Give two things that overflow a 2 KB TA stack.
- ✓ Declare a custom property and read it from the TA.
- ✓ Explain why session state should be wiped before it is freed.
- ✓ Say which flag a TA needs for a kernel driver to bind to it.

Building, signing and packaging a TA

A trusted application is delivered as a signed binary sitting in the normal world's filesystem, where an attacker can replace it at will. The signature is the only thing that makes that acceptable, and understanding the format, the build and the key management is what separates a demo from something shippable.

The project layout

```
   Copy optee_examples/hello_world and rename. Every OP-TEE
   project has this shape and there is no reason to invent
   another.

   my_project/
     Makefile              builds both halves
     host/
       Makefile
       main.c              the client (chapter 11)
     ta/
       Makefile            2 lines - includes the dev kit
       sub.mk              lists the source files
       user_ta_header_defines.h    flags, sizes, UUID (ch. 13)
       my_ta.c             the five entry points (ch. 12)
       include/
         my_ta.h           SHARED between host and TA:
                           the UUID and the command IDs
     README.md

   my_ta.h is the interface contract. Both sides include it, so
   a command ID can never drift between them:

     #define TA_MY_UUID \
        { 0x12345678, 0x1234, 0x1234, \
          { 0x12, 0x34, 0x12, 0x34, 0x12, 0x34, 0x12, 0x34 } }

     #define TA_CMD_SIGN     0
     #define TA_CMD_VERIFY   1
     #define TA_CMD_GEN_KEY  2
```

```
# Generate a UUID. Never invent one by hand, and never reuse
# an example's - two TAs with the same UUID cannot coexist.
$ uuidgen
7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b

# In the C form OP-TEE wants:
$ python3 - <<'EOF'
import uuid
u = uuid.uuid4()
b = u.bytes
print(u)
print("#define TA_MY_UUID \\")
print("\t{ 0x%08x, 0x%04x, 0x%04x, \\" % (u.time_low, u.time_mid,
      u.time_hi_version))
print("\t  { " + ", ".join("0x%02x" % c for c in b[8:]) + " } }")
EOF
```

The makefiles

```
# ta/Makefile - genuinely this short. All the work is in the
# dev kit.
CFG_TEE_TA_LOG_LEVEL ?= 3
CPPFLAGS += -DCFG_TEE_TA_LOG_LEVEL=$(CFG_TEE_TA_LOG_LEVEL)

BINARY = 7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b

-include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

ifeq ($(wildcard $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk), )
clean:
        @echo 'TA_DEV_KIT_DIR is not correctly defined' && false
endif

# BINARY must be the UUID. The output is $(BINARY).ta and
# OP-TEE looks it up by that name. A mismatch between BINARY
# and TA_UUID in the header means the file exists and is never
# found.


# ta/sub.mk
global-incdirs-y += include
srcs-y += my_ta.c
srcs-y += crypto_helpers.c
# cflags-my_ta.c-y += -Wno-unused-parameter


# host/Makefile
CC      ?= $(CROSS_COMPILE)gcc
CFLAGS  += -Wall -I../ta/include -I$(TEEC_EXPORT)/include
LDADD   += -lteec -L$(TEEC_EXPORT)/lib

BINARY = my_client
OBJS = main.o

.PHONY: all clean
all: $(BINARY)

$(BINARY): $(OBJS)
        $(CC) -o $@ $< $(LDADD)

%.o: %.c
        $(CC) $(CFLAGS) -c $< -o $@

clean:
        rm -f $(OBJS) $(BINARY)


# Top-level Makefile
export V ?= 0
OUTPUT_DIR := $(CURDIR)/out

.PHONY: all
all: ta host

.PHONY: ta
ta:
        $(MAKE) -C ta CROSS_COMPILE=$(TA_CROSS_COMPILE) \
                LDFLAGS=""
```

```
.PHONY: host
host:
	$(MAKE) -C host CROSS_COMPILE=$(HOST_CROSS_COMPILE) \
		--no-builtin-variables

.PHONY: clean
clean:
	$(MAKE) -C ta clean
	$(MAKE) -C host clean
```

```
# Build it.
$ export TA_DEV_KIT_DIR=$HOME/optee/optee_os/out/arm-plat-vexpress/export-ta_arm64
$ export TEEC_EXPORT=$HOME/optee/optee_client/out/export/usr
$ export TA_CROSS_COMPILE=aarch64-linux-gnu-
$ export HOST_CROSS_COMPILE=aarch64-linux-gnu-
$ make

$ ls ta/*.ta host/my_client
ta/7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b.ta
host/my_client

# Install into the image:
$ cp ta/*.ta   ~/optee/out-br/target/lib/optee_armtz/
$ cp host/my_client ~/optee/out-br/target/usr/bin/
$ cd ~/optee/build && make buildroot run-only

# Or, better, add it to the OP-TEE build as a buildroot
# package so it is rebuilt automatically. See
# optee_examples/ for the pattern.
```

What a .ta file is

```
+-----------------------------------------------------------+
|  shdr - the signed header                                 |
|    magic       0x4f545348  "OTSH"                         |
|    img_type    SHDR_TA / SHDR_BOOTSTRAP_TA /              |
|                SHDR_ENCRYPTED_TA                          |
|    img_size                                               |
|    algo        e.g. RSASSA_PKCS1_PSS_MGF1_SHA256          |
|    hash_size                                              |
|    sig_size                                               |
+-----------------------------------------------------------+
|  hash          SHA-256 of everything below                |
+-----------------------------------------------------------+
|  signature     RSA/ECDSA over that hash                   |
+-----------------------------------------------------------+
|  (bootstrap TA only) the UUID and TA version              |
+-----------------------------------------------------------+
|  the ELF - the actual TA, linked against the dev kit      |
+-----------------------------------------------------------+

VERIFICATION, inside OP-TEE, before a single instruction of
the TA runs:

  1. tee-supplicant delivers the file (chapter 9).
  2. OP-TEE reads shdr and checks magic and sizes.
  3. It hashes the ELF and compares with hash.
  4. It verifies the signature over that hash, using a
     public key COMPILED INTO OP-TEE.
  5. For a bootstrap TA, it checks the UUID inside matches
     the UUID that was requested - so a valid TA cannot be
     substituted for a different valid TA.
  6. Only then is the ELF loaded and mapped.

Failure at any step: TEEC_ERROR_SECURITY (0xFFFF000F).

NOTE WHAT IS NOT PROTECTED: confidentiality. A plain .ta is
readable by anyone with the filesystem. If the TA's code is
itself sensitive, you need SHDR_ENCRYPTED_TA.
```

Signing, and the key problem

```
# The default: every OP-TEE tree ships one key, and it signs
# every TA built in that tree.
$ ls -l optee_os/keys/default_ta.pem
-rw-r--r-- 1 user user 1704 default_ta.pem

# THIS KEY IS PUBLIC. It is in the git repository. A device
# whose OP-TEE trusts it will load any TA anyone builds.

# Use your own:
$ openssl genrsa -out my_ta_key.pem 2048
$ openssl rsa -in my_ta_key.pem -pubout -out my_ta_pub.pem

$ make -C optee_os TA_SIGN_KEY=$PWD/my_ta_key.pem ...
# OP-TEE embeds the PUBLIC half; the private half signs TAs.

# Sign an existing unsigned ELF by hand:
$ python3 optee_os/scripts/sign_encrypt.py sign \
    --key my_ta_key.pem \
    --uuid 7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b \
    --in ta.elf --out ta.ta

# Inspect one you were given:
$ python3 optee_os/scripts/sign_encrypt.py display --in some.ta

# THE PRODUCTION PATTERN: the private key never leaves an HSM.
# sign_encrypt.py supports a two-step flow for exactly this:
#
#   1. produce the digest to be signed
$ python3 scripts/sign_encrypt.py digest \
    --key my_ta_pub.pem --uuid $UUID --in ta.elf --dig ta.dig
#   2. sign the digest elsewhere - HSM, signing service, an
#      air-gapped machine
$ openssl pkeyutl -sign -inkey my_ta_key.pem \
    -in ta.dig -out ta.sig \
    -pkeyopt digest:sha256 -pkeyopt rsa_padding_mode:pss \
    -pkeyopt rsa_pss_saltlen:digest
#   3. staple the signature on
$ python3 scripts/sign_encrypt.py stitch \
    --key my_ta_pub.pem --uuid $UUID \
    --in ta.elf --sig ta.sig --out ta.ta

# The build machine never holds the private key. This is the
# difference between a signing story and a signing key sitting
# in CI's environment variables.
```

> **THREE WAYS TA SIGNING GOES WRONG IN THE FIELD**
>
> **Shipping with the default key** — anyone can load a TA. **Key in the repository** — same, once the repository leaks. **No revocation plan** — the public key is compiled into OP-TEE, which is in a signed boot image, so rotating it means a firmware update. Decide the rotation story before the first device ships, not after a key is compromised.

Encrypted TAs

```
# When the TA's own code is the secret - a proprietary
# algorithm, an anti-tamper routine, an obfuscated licence
# check.

$ make -C optee_os \
    CFG_ENCRYPT_TA=y \
    TA_ENC_KEY=0123456789abcdef0123456789abcdef

$ python3 scripts/sign_encrypt.py sign \
    --key my_ta_key.pem --enc-key $TA_ENC_KEY \
    --enc-key-type SHDR_ENC_KEY_DEV_SPECIFIC \
    --uuid $UUID --in ta.elf --out ta.ta

# Key types:
#   SHDR_ENC_KEY_DEV_SPECIFIC   derived from the device's
#                               hardware unique key. The .ta is
#                               then bound to ONE device and
#                               cannot be moved.
#   SHDR_ENC_KEY_CLASS_WIDE     a key shared by a product
#                               class. Convenient; one
#                               extracted key breaks the whole
#                               class.

# The TA is encrypted at rest and decrypted inside OP-TEE
# during load. It is still in cleartext in TA_RAM while
# running, so this defends against extraction from the
# filesystem, not against an attacker who has already
# compromised the secure world.

# Weigh it honestly: encryption complicates your build and
# your field-update story, and it protects only against
# reverse engineering. Most TAs do not need it. Signing is
# mandatory; encryption is a business decision.
```

Confirming it worked

```
# On the target:
$ ls -l /lib/optee_armtz/
-rw-r--r-- 1 root root 24576 7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b.ta

# Filename MUST equal the UUID. Check:
$ python3 sign_encrypt.py display --in /path/to/ta.ta
uuid: 7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b
img_type: SHDR_BOOTSTRAP_TA
algo: TEE_ALG_RSASSA_PKCS1_PSS_MGF1_SHA256
size: 22016

# Run the client and watch the secure console:
D/TC:0 0 ree_fs_ta_open:...  TA 7f2e9c1a-... loaded
D/TC:0 0 shdr_verify_signature:...  OK
I/TA:  my TA started

# A signature failure looks like:
E/TC:0 0 shdr_verify_signature:...  Signature verification failed
E/TC:0 0 ree_fs_ta_open:...  TEE_ERROR_SECURITY
# and the client sees 0xFFFF000F.
```

How it shows up

Symptom	Cause
`TEEC_ERROR_ITEM_NOT_FOUND`	Filename does not match the UUID, wrong directory, or `tee-supplicant` not running
`TEEC_ERROR_SECURITY`	Signed with a key OP-TEE does not trust — usually a rebuilt OP-TEE with a different `TA_SIGN_KEY`
TA builds but the file is empty or tiny	`BINARY` in `ta/Makefile` does not match, or `sub.mk` lists no sources
`TA_DEV_KIT_DIR is not correctly defined`	Dev kit path wrong, or built for a different architecture (`arm32` vs `arm64`)
Changes have no effect	Old `.ta` still in the image — `make buildroot`
Two TAs conflict	Copied an example without changing the UUID
Client links but cannot find `libteec.so`	Library not in the target rootfs
Works on your build, not on a colleague's	Different `TA_SIGN_KEY`; the TA must be signed by the key that *their* OP-TEE embeds
Cannot rotate a compromised key	The public key is inside a signed boot image; this needed planning
Encrypted TA fails to load on a second device	`DEV_SPECIFIC` key — bound to one device by design

Check yourself

- ✓ Lay out a TA project and say what belongs in the shared header.
- ✓ Explain why `BINARY` in the TA Makefile must be the UUID.
- ✓ Generate a UUID and put it in the C form OP-TEE expects.
- ✓ Describe the `.ta` file format and the six verification steps.
- ✓ Explain what a bootstrap TA's embedded UUID prevents.
- ✓ Say what signing does *not* protect.
- ✓ Explain the risk of the default signing key.
- ✓ Describe the digest-sign-stitch flow and why production needs it.
- ✓ Explain when an encrypted TA is worth the cost, and what it does not defend against.
- ✓ Diagnose `TEEC_ERROR_SECURITY` and `TEEC_ERROR_ITEM_NOT_FOUND` apart.

Secure storage: REE FS and RPMB

The secure world has no flash of its own. It stores data by encrypting it and handing it to the normal world, or by writing it to a partition of the eMMC that authenticates every access. The two approaches protect different things, and knowing which property you are missing is the whole point of this chapter.

Two back ends, different guarantees

Property	REE FS	RPMB
Confidentiality	**Yes** — AES-GCM with a device key	**Yes**
Integrity	**Yes** — authenticated encryption	**Yes** — HMAC in hardware
Rollback protection	**No** — an old ciphertext still verifies	**Yes** — a monotonic write counter
Availability	No — the normal world can delete the file	No — but it cannot silently substitute
Needs hardware	No	eMMC or UFS with an RPMB partition
Typical size	Whatever the filesystem has	128 KB – 16 MB
Speed	RPC per operation; moderate	RPC plus an eMMC transaction; slow
Build flag	`CFG_REE_FS=y`	`CFG_RPMB_FS=y`

```
THE ROLLBACK GAP, which is the whole reason RPMB exists.

REE FS:
  TA writes "failed_attempts = 3"
    -> encrypted, authenticated, written to /data/tee/xyz
  Attacker copies that file.
  TA writes "failed_attempts = 4", then 5, then locks out.
  Attacker restores the copied file.
    -> it decrypts correctly
    -> it authenticates correctly
    -> the TA reads "failed_attempts = 3"

  Nothing was forged. An OLD, GENUINE state was replayed,
  and encryption cannot detect that.

RPMB:
  Every write carries a monotonically increasing counter,
  and the eMMC itself enforces it. A replayed frame has a
  stale counter and the eMMC rejects it. The counter lives
  in the flash controller, not in software, and cannot be
  rewound.

So: use REE FS for things where replay does not matter (a
key, a certificate - old is as good as new). Use RPMB for
anything COUNTED or REVOKED: attempt counters, firmware
anti-rollback versions, licence state, revocation lists.
```

Using it

```
/* The API is the same regardless of back end - only the
   storageID changes.                                          */

#define OBJ_ID  "my-signing-key-v1"

static TEE_Result store_key(const uint8_t *key, size_t len)
{
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        TEE_Result res;
        uint32_t flags = TEE_DATA_FLAG_ACCESS_READ  |
                         TEE_DATA_FLAG_ACCESS_WRITE |
                         TEE_DATA_FLAG_ACCESS_WRITE_META |
                         TEE_DATA_FLAG_OVERWRITE;

        res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
                                         OBJ_ID, sizeof(OBJ_ID) - 1,
                                         flags,
                                         TEE_HANDLE_NULL,   /* no
                                            attributes - raw data */
                                         key, len,
                                         &obj);
        if (res != TEE_SUCCESS) {
                EMSG("create failed 0x%08x", res);
                return res;
        }
        TEE_CloseObject(obj);
        return TEE_SUCCESS;
}

static TEE_Result load_key(uint8_t *key, size_t *len)
{
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        TEE_ObjectInfo   info;
        uint32_t         count;
        TEE_Result       res;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       OBJ_ID, sizeof(OBJ_ID) - 1,
                                       TEE_DATA_FLAG_ACCESS_READ,
                                       &obj);
        if (res != TEE_SUCCESS)
                return res;      /* TEE_ERROR_ITEM_NOT_FOUND if
                                    it was never written - a
                                    NORMAL first-run case, handle
                                    it, do not treat it as an
                                    error                       */

        res = TEE_GetObjectInfo1(obj, &info);
        if (res != TEE_SUCCESS)
                goto out;

        if (info.dataSize > *len) {
                *len = info.dataSize;
                res = TEE_ERROR_SHORT_BUFFER;
                goto out;
        }

        res = TEE_ReadObjectData(obj, key, info.dataSize, &count);
        if (res == TEE_SUCCESS)
                *len = count;
```

```
out:
        TEE_CloseObject(obj);
        return res;
}

static TEE_Result delete_key(void)
{
        TEE_ObjectHandle obj;
        TEE_Result res;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       OBJ_ID, sizeof(OBJ_ID) - 1,
                                       TEE_DATA_FLAG_ACCESS_READ |
                                       TEE_DATA_FLAG_ACCESS_WRITE_META,
                                       &obj);
        if (res != TEE_SUCCESS)
                return res;
        /* This CLOSES the handle as well - do not close it
           again.                                              */
        return TEE_CloseAndDeletePersistentObject1(obj);
}
```

STORING A KEY OBJECT, NOT RAW BYTES

`TEE_CreatePersistentObject`'s `attributes` parameter takes a `TEE_ObjectHandle`. Pass a populated key object there and OP-TEE stores the key *as a key* — you can reopen it and pass it straight to `TEE_SetOperationKey` without the raw material ever appearing in a buffer you manage. For key storage that is strictly better than storing bytes and re-populating.

```
/* The better pattern for keys: */

    TEE_ObjectHandle key = TEE_HANDLE_NULL, persistent = TEE_HANDLE_NULL;

    TEE_AllocateTransientObject(TEE_TYPE_AES, 256, &key);
    TEE_GenerateKey(key, 256, NULL, 0);   /* never leaves the TEE */

    TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
                               OBJ_ID, sizeof(OBJ_ID) - 1,
                               flags,
                               key,          /* <-- the object   */
                               NULL, 0,      /* no extra data     */
                               &persistent);

    /* Later, in another session or after a reboot: */
    TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                             OBJ_ID, sizeof(OBJ_ID) - 1,
                             TEE_DATA_FLAG_ACCESS_READ, &persistent);
    TEE_SetOperationKey(op, persistent);    /* use it directly    */

    /* The raw key bytes existed only inside OP-TEE, ever. */
```

The key hierarchy

```
HUK - the Hardware Unique Key
  Burned into fuses or derived from a PUF. Readable only by
  secure-world code, often only through a hardware key
  derivation block that never exposes it. Different on every
  device.
    |
    |  KDF, with a fixed string
    v
SSK - Secure Storage Key
  huk_derive(HUK, "onlinesecurestorage")
    |
    |  KDF, with the TA's UUID
    v
TSK - TA Storage Key
  hmac(SSK, TA_UUID)
    |
    |  and, if the login type scopes it, the client identity
    v
FEK - File Encryption Key
  A FRESH RANDOM key per file, encrypted with the TSK and
  stored in the file's own header.
    |
    v
the file contents, AES-GCM under the FEK

What each level buys:

  HUK per device      one device's storage cannot be read on
                      another, and a stolen image is useless
  TSK per TA          TA A cannot read TA B's objects. This
                      is enforced by key derivation, not by
                      access control - even a compromised
                      TA A does not have TA B's key.
  per-client scoping  with LOGIN_USER, two users of the same
                      TA get different storage
  FEK per file        one file's key does not compromise
                      another; and rewriting a file uses a
                      new FEK

On QEMU there are no fuses, so the HUK is a hardcoded test
value and every "device" has the same one. That is fine for
development and is exactly what the "might be insecure"
warning is about (chapter 4).
```

RPMB in practice

```
# RPMB is a special eMMC partition. Every read and write is
# authenticated with an HMAC-SHA256 key that is PROGRAMMED ONCE
# and can never be read back or changed.

$ ls /dev/mmcblk0*
/dev/mmcblk0  /dev/mmcblk0boot0  /dev/mmcblk0boot1  /dev/mmcblk0rpmb

$ sudo mmc rpmb read-counter /dev/mmcblk0rpmb
Counter value: 0x00001a2f

# Build OP-TEE for it:
$ make CFG_RPMB_FS=y CFG_RPMB_WRITE_KEY=y CFG_RPMB_TESTKEY=n

#   CFG_RPMB_FS=y         use RPMB for secure storage
#   CFG_RPMB_WRITE_KEY=y  allow OP-TEE to PROGRAM the key if it
#                         is not yet set. ONE-WAY. Enable
#                         deliberately, in manufacturing.
#   CFG_RPMB_TESTKEY=y    use a fixed test key. Development
#                         only, and it makes RPMB pointless.

# Under QEMU there is no real eMMC, so tee-supplicant emulates
# one in a file:
$ tee-supplicant -r          # emulated RPMB
$ ls -l /var/lib/optee/rpmb_emu*
# Useful for developing the logic; provides no real rollback
# protection, because the emulation file can be restored.

# From a TA, force RPMB explicitly:
    TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE_RPMB, ...)
```

> **THE RPMB KEY IS WRITTEN ONCE, FOR EVER**
>
> Programming the RPMB authentication key is irreversible. Get it wrong in manufacturing — wrong key, key not recorded, key derived from something you later change — and those devices can never use RPMB again. There is no reset, no factory tool, no vendor override. Decide the key derivation before the first production run and test the whole flow on sacrificial hardware.

What the normal world can still do

```
# REE FS storage is just files:
$ ls -la /data/tee/
drwx------ 2 root root  4096 .
-rw------- 1 root root  4096 0
-rw------- 1 root root   512 1
-rw------- 1 root root  4096 dirf.db

$ xxd /data/tee/0 | head -3
00000000: a3f1 8c2e 44b9 7710 ...   # ciphertext. Unreadable.

# The normal world CAN:
#   delete these files      -> the TA's data is GONE
#   restore an old copy     -> rollback, unless RPMB
#   fill the filesystem     -> writes fail
#   corrupt them            -> reads fail authentication

# The normal world CANNOT:
#   read the plaintext
#   forge a valid file
#   move the data to another device and use it

# So: AVAILABILITY IS NOT PROTECTED, on either back end. A TA
# must handle its storage vanishing:
#
#   res = TEE_OpenPersistentObject(...);
#   if (res == TEE_ERROR_ITEM_NOT_FOUND) {
#           /* First run, OR someone deleted it. You usually
#              cannot tell the difference. Decide, at design
#              time, which of these your TA does:
#                - regenerate and carry on   (a cache)
#                - refuse to operate         (a licence)
#                - re-provision from a server (a credential)
#              "Regenerate silently" for something that was
#              supposed to be counted is how rollback
#              protection gets lost.                        */
#   }
```

How it shows up

Symptom	Cause
`TEE_ERROR_ITEM_NOT_FOUND` on first run	Normal. Handle it as a create-if-absent case
All stored data lost after a filesystem wipe	REE FS lives in `/data/tee`; nothing protects availability
Attempt counter can be reset	REE FS has no rollback protection — this needs RPMB
`TEE_ERROR_MAC_INVALID` or `CORRUPT_OBJECT`	File modified or truncated by the normal world — authentication working
Storage works on one device, not another	Per-device HUK. This is the intended behaviour
Storage lost after a firmware update	HUK derivation changed, or the TA's UUID changed, so the TSK differs
RPMB operations fail with `TEE_ERROR_TIME_NOT_SET`-like errors	RPMB key not programmed, or the wrong key
RPMB writes fail after a while	Partition full — RPMB is small; store counters, not blobs
Storage is very slow	Every operation is an RPC plus an eMMC transaction. Cache in the session context and write rarely

Symptom	Cause
Two users share stored data	`TEEC_LOGIN_PUBLIC`; use `LOGIN_USER` (chapter 11)

Check yourself

- ✓ Compare REE FS and RPMB across confidentiality, integrity, rollback and availability.
- ✓ Explain the rollback attack against REE FS in concrete steps.
- ✓ Say which kinds of data require RPMB and give three examples.
- ✓ Write a create-read-delete cycle for a persistent object with correct error handling.
- ✓ Explain why storing a key object beats storing raw bytes.
- ✓ Draw the key hierarchy from HUK to file, and say what each level isolates.
- ✓ Explain how one TA is prevented from reading another's storage.
- ✓ Say what the RPMB write counter protects and where it lives.
- ✓ Explain why programming the RPMB key is a manufacturing decision.
- ✓ List what the normal world can and cannot do to secure storage, and say how a TA should handle its data disappearing.

Cryptography inside a TA

The whole reason a TA exists is usually to hold a key. This chapter covers the algorithms OP-TEE offers, how keys are generated and derived so that raw material never crosses the boundary, where the hardware accelerator fits, and the handful of mistakes that turn a correct algorithm into a broken system.

The rule that shapes every design

```
A key that crosses into the normal world is not protected.
Design so it never has to.

BAD                          GOOD

client: generate key         client: "generate a key for me"
client -> TA: here is the      TA: TEE_GenerateKey()
    key, please sign               store it persistently
TA: sign with it                   return only a HANDLE/ID
TA -> client: signature       client: "sign this, with key 7"
                               TA: load key 7, sign, return
                                   the signature

In the second design the key material exists only inside
OP-TEE, from generation to destruction. There is no moment
at which a compromised kernel could read it.

The client's interface becomes:
    create_key()  -> key_id
    sign(key_id, digest) -> signature
    get_public_key(key_id) -> public key
    delete_key(key_id)

That is exactly the shape of Android Keymint, of a PKCS#11
token, and of the project in chapter 21. It is not a
coincidence: it is the only shape that works.
```

What OP-TEE offers

Class	Algorithms	Notes
Hashes	MD5, SHA-1, SHA-224/256/384/512, SHA3-*, SM3	MD5 and SHA-1 exist for legacy interop only
Symmetric	AES (ECB, CBC, CTR, CTS, XTS, CBC-MAC), DES, 3DES, SM4	Do not use ECB. Ever. It is there for building other things
AEAD	AES-GCM, AES-CCM	The default choice for encrypting anything
MAC	HMAC-*, AES-CMAC, AES-CBC-MAC	Verify with `TEE_MACCompareFinal`
Asymmetric	RSA (PKCS#1 v1.5, PSS, OAEP), DSA, ECDSA, Ed25519, SM2	PSS over v1.5 for new designs
Key agreement	DH, ECDH, X25519	Followed by a KDF — never use the raw shared secret as a key
Derivation	HKDF, PBKDF2, Concat KDF	Availability depends on `CFG_CRYPTO_*`
Random	`TEE_GenerateRandom`	Backed by a hardware TRNG where the platform has one

```
# Which are actually compiled in? They are individually
# selectable, and a missing one gives TEE_ERROR_NOT_SUPPORTED
# at runtime rather than a build error.

$ grep -E '^CFG_CRYPTO' optee_os/core/crypto/sub.mk | head -20
CFG_CRYPTO_AES ?= y
CFG_CRYPTO_GCM ?= y
CFG_CRYPTO_ECC ?= y
CFG_CRYPTO_RSA ?= y
CFG_CRYPTO_SHA512 ?= y
CFG_CRYPTO_HKDF ?= y
CFG_CRYPTO_SM2_PKE ?= n
...

# Turn off what you do not need - it is code in your trusted
# computing base:
$ make CFG_CRYPTO_MD5=n CFG_CRYPTO_DES=n CFG_CRYPTO_DSA=n

# And confirm from a TA, at runtime, rather than assuming:
    res = TEE_AllocateOperation(&op, TEE_ALG_AES_GCM,
                                TEE_MODE_ENCRYPT, 256);
    if (res == TEE_ERROR_NOT_SUPPORTED)
            /* this build does not have GCM */
```

Generating and keeping a key

```
/* An ECDSA P-256 signing key that is generated inside the TEE,
   stored persistently, and never exported.                   */

static TEE_Result create_signing_key(const char *id, size_t idlen)
{
        TEE_ObjectHandle key  = TEE_HANDLE_NULL;
        TEE_ObjectHandle pobj = TEE_HANDLE_NULL;
        TEE_Attribute    attr;
        TEE_Result       res;

        res = TEE_AllocateTransientObject(TEE_TYPE_ECDSA_KEYPAIR,
                                          256, &key);
        if (res) goto out;

        /* The curve must be given as a generation parameter. */
        TEE_InitValueAttribute(&attr, TEE_ATTR_ECC_CURVE,
                               TEE_ECC_CURVE_NIST_P256, 0);

        res = TEE_GenerateKey(key, 256, &attr, 1);
        if (res) goto out;

        /* Persist it AS AN OBJECT. */
        res = TEE_CreatePersistentObject(
                TEE_STORAGE_PRIVATE, id, idlen,
                TEE_DATA_FLAG_ACCESS_READ |
                TEE_DATA_FLAG_ACCESS_WRITE_META |
                TEE_DATA_FLAG_OVERWRITE,
                key, NULL, 0, &pobj);
out:
        TEE_FreeTransientObject(key);
        if (pobj != TEE_HANDLE_NULL)
                TEE_CloseObject(pobj);
        return res;
}

/* Exporting ONLY the public half - which is safe and
   necessary, because someone must verify the signatures.   */

static TEE_Result export_public(const char *id, size_t idlen,
                                uint8_t *x, size_t *xlen,
                                uint8_t *y, size_t *ylen)
{
        TEE_ObjectHandle pobj = TEE_HANDLE_NULL;
        TEE_Result res;
        uint32_t l;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       id, idlen,
                                       TEE_DATA_FLAG_ACCESS_READ,
                                       &pobj);
        if (res) return res;

        l = *xlen;
        res = TEE_GetObjectBufferAttribute(pobj,
                        TEE_ATTR_ECC_PUBLIC_VALUE_X, x, &l);
        if (res) goto out;
        *xlen = l;

        l = *ylen;
        res = TEE_GetObjectBufferAttribute(pobj,
```

```
                        TEE_ATTR_ECC_PUBLIC_VALUE_Y, y, &l);
        if (res == TEE_SUCCESS)
                *ylen = l;
out:
        TEE_CloseObject(pobj);
        return res;
}

/* Asking for TEE_ATTR_ECC_PRIVATE_VALUE on a generated key
   returns TEE_ERROR_ACCESS_DENIED. That refusal is the
   guarantee: the private half is structurally unexportable,
   not merely undocumented.                                  */
```

DERIVE, DO NOT STORE

A TA that needs many keys does not have to store many keys. Derive them from one root plus a label: `key_i = HKDF(root, "purpose" || i)`. Storage holds one secret; the derived keys exist only while in use. Fewer persistent objects, no per-key storage failure, and rotating the root rotates everything at once.

Key derivation

```
/* HKDF: the right tool for turning one secret into many. */

static TEE_Result derive(TEE_ObjectHandle root,
                         const char *label, size_t labellen,
                         TEE_ObjectHandle out)
{
        TEE_OperationHandle op = TEE_HANDLE_NULL;
        TEE_Attribute params[2];
        TEE_Result res;

        res = TEE_AllocateOperation(&op,
                        TEE_ALG_HKDF_SHA256_DERIVE_KEY,
                        TEE_MODE_DERIVE, 256);
        if (res) goto out;

        res = TEE_SetOperationKey(op, root);
        if (res) goto out;

        TEE_InitRefAttribute(&params[0], TEE_ATTR_HKDF_INFO,
                             label, labellen);
        TEE_InitRefAttribute(&params[1], TEE_ATTR_HKDF_SALT,
                             SALT, sizeof SALT);

        TEE_DeriveKey(op, params, 2, out);
out:
        TEE_FreeOperation(op);
        return res;
}

/* ECDH, and the mistake everyone makes: */

    /* WRONG: use the shared secret directly as an AES key.
       The X coordinate of an ECDH result is NOT uniformly
       distributed and has algebraic structure. */

    /* RIGHT: */
    TEE_AllocateOperation(&op, TEE_ALG_ECDH_DERIVE_SHARED_SECRET,
                          TEE_MODE_DERIVE, 256);
    TEE_SetOperationKey(op, my_keypair);
    TEE_InitRefAttribute(&p[0], TEE_ATTR_ECC_PUBLIC_VALUE_X,
                         peer_x, sizeof peer_x);
    TEE_InitRefAttribute(&p[1], TEE_ATTR_ECC_PUBLIC_VALUE_Y,
                         peer_y, sizeof peer_y);
    TEE_DeriveKey(op, p, 2, shared);       /* a GENERIC_SECRET */
    /* ...then run HKDF over `shared` to get the actual AES key */
```

The hardware unique key

```
/* Every device has a different HUK (chapter 15). A TA can
   derive from it, which gives you a secret that:
     - is different on every device
     - never existed anywhere else
     - needs no storage at all                              */

#include <tee_internal_api_extensions.h>

    uint8_t key[32];
    TEE_Result res = TEE_GetHUKDerivedKey(
                        (const uint8_t *)"my-ta-purpose-v1", 16,
                        key, sizeof key);
    /* NOT standard GlobalPlatform - an OP-TEE extension,
       available when CFG_HUK_SUBKEY is enabled.            */

/* Uses:
     - encrypting something without a persistent key object
     - a device-binding value, so data cannot move devices
     - a per-device seed for a deterministic key

   Cautions:
     - the derivation label is part of the contract. Change it
       and every previously derived key is gone. Version it.
     - on QEMU the HUK is a fixed test value, so every
       "device" derives the same key. Do not conclude from a
       QEMU test that device binding works.
     - wipe the buffer when you are done:
           TEE_MemFill(key, 0, sizeof key);                 */
```

Hardware acceleration

```
/* Most SoCs have a crypto accelerator. OP-TEE abstracts it
   behind the same API, so a TA does not change - but the
   performance difference is large.                          */

  core/drivers/crypto/
    caam/       NXP i.MX
    se050/      NXP secure element
    stm32/      ST
    hisilicon/  HiSilicon
    versal/     AMD/Xilinx
    aspeed/     ASPEED

  Enabled per platform, e.g.:
    CFG_NXP_CAAM=y
    CFG_NXP_CAAM_SHA_DRV=y
    CFG_NXP_CAAM_CIPHER_DRV=y

/* Rough figures - measure your own part:
     AES-256-GCM, software (libtomcrypt)   ~20-60 MB/s
     AES-256-GCM, CAAM/SAHARA             ~200-800 MB/s
     RSA-2048 sign, software              ~10-40 ms
     RSA-2048 sign, accelerator           ~1-5 ms
     ECDSA P-256 sign, software           ~2-10 ms

   The fixed cost of a TA invocation (chapter 5) is tens of
   microseconds, so for small operations the world switch
   dominates and the accelerator barely shows. For bulk data
   it is the whole story.

   The design consequence: BATCH. One invocation that signs 100
   digests beats 100 invocations that sign one each, by a large
   factor, regardless of the accelerator.                    */
```

The mistakes that matter

Mistake	Why it is fatal	Do this
Reusing a GCM nonce with the same key	Recovers the authentication key; forgeries become trivial	Counter, or 96 bits of `TEE_GenerateRandom`, and never repeat
Comparing MACs with `TEE_MemCompare`	Timing oracle	`TEE_MACCompareFinal`, or a constant-time loop
Using an ECDH shared secret as a key	Not uniform; has structure	Run it through HKDF
AES-ECB	Identical plaintext blocks give identical ciphertext	GCM
RSA PKCS#1 v1.5 encryption	Padding-oracle attacks	OAEP, or better, don't use RSA encryption
Decrypt-then-check	Acting on unauthenticated plaintext	AEAD, which does it in the right order for you
Key material in a local buffer	Left on the TA heap or stack after use	Key objects, and `TEE_MemFill(0)` anything raw
Trusting `TEE_GetREETime` for expiry	The attacker sets the clock	Chapter 17
Logging a key with `DHEXDUMP`	It ships	Never; guard it out of release builds
Rolling your own primitive	You will get it wrong	Use the API; it is there

How it shows up

Symptom	Cause
`TEE_ERROR_NOT_SUPPORTED`	Algorithm not compiled in — check the `CFG_CRYPTO_*` flags
`TEE_ERROR_BAD_PARAMETERS` on `AllocateOperation`	Key size outside the declared maximum, or algorithm/mode mismatch
`TEE_ERROR_ACCESS_DENIED` reading a private attribute	Working as designed — private key material is not exportable
`TEE_ERROR_MAC_INVALID` on decrypt	Wrong key, wrong nonce, or genuinely tampered data
Works, then fails after hours	Leaked operation or object handles
Signature verifies in the TA, fails in OpenSSL	Format mismatch — ECDSA raw r‖s versus DER, or a digest applied twice
Slow despite a hardware accelerator	Per-invocation overhead dominates; batch
Different results on QEMU and hardware	Accelerator driver differences, or the HUK being a fixed test value on QEMU
Key lost after a firmware update	HUK derivation label or TA UUID changed

Check yourself

- ✓ State the rule about keys crossing the boundary and give the client interface it implies.
- ✓ Generate an ECDSA keypair inside a TA and store it as an object.
- ✓ Explain what happens when you ask for the private attribute, and why that is the guarantee.
- ✓ Explain the derive-do-not-store pattern and its advantages.
- ✓ Say why an ECDH shared secret must not be used directly as a key.
- ✓ Describe `TEE_GetHUKDerivedKey`, two uses, and two cautions.
- ✓ Explain why batching matters more than hardware acceleration for small operations.
- ✓ Name five cryptographic mistakes from the table and the correct alternative for each.
- ✓ Explain what GCM nonce reuse costs you.
- ✓ Determine at runtime whether an algorithm is available in the current build.

Time, counters and rollback protection

Two things the secure world cannot get for itself: what time it is, and whether the state it just read is the newest one. Both are supplied by an adversary. Everything in this chapter is about designing around that, because no amount of cryptography fixes a system that believes an old, correctly signed answer.

Three clocks, three trust levels

Source	What it is	Trust
`TEE_GetSystemTime`	Monotonic since boot, from the secure timer	**Trusted for intervals.** Resets on reboot; not wall clock
`TEE_GetREETime`	Wall-clock time from the normal world	**Untrusted.** The attacker sets it to whatever they like
`TEE_GetTAPersistentTime`	A per-TA time that survives reboot, kept in secure storage	**Conditionally trusted** — and it tells you which condition applies

```
/* Persistent time is the interesting one, because it reports
   its own reliability.                                       */

    TEE_Time t;
    TEE_Result res = TEE_GetTAPersistentTime(&t);

    switch (res) {
    case TEE_SUCCESS:
            /* Set, and OP-TEE believes it has not been
               tampered with. Usable.                        */
            break;
    case TEE_ERROR_TIME_NOT_SET:
            /* Never initialised on this device. First run.  */
            break;
    case TEE_ERROR_TIME_NEEDS_RESET:
            /* It was set, but something happened - a battery
               loss, a storage inconsistency. DO NOT TRUST IT.*/
            break;
    case TEE_ERROR_OVERFLOW:
            /* Ran past its representable range. */
            break;
    }

    /* Setting it - the value comes from somewhere you trust,
       which usually means a signed timestamp from a server,
       not from TEE_GetREETime.                              */
    TEE_SetTAPersistentTime(&t);

/* THE ANTI-PATTERN, and it is everywhere:

       TEE_Time now;
       TEE_GetREETime(&now);
       if (now.seconds > licence_expiry)
               return TEE_ERROR_ACCESS_DENIED;

   Defeated with `date -s 2020-01-01`. The check is not weak;
   it is absent, because the attacker supplies the input.  */
```

> **THE GENERAL PRINCIPLE**
> Anything an attacker supplies is not a security input, however carefully you compare it. To bound *something* in time you need a counter you control (below), a signed timestamp from a party you trust, or an online check. If none of those is available, the honest answer is that you cannot enforce expiry offline — and it is better to say so at design time than to ship a check that looks like one.

The rollback problem

```
Encryption and authentication answer: "is this data
genuine?"

They do not answer: "is this the LATEST genuine data?"

Attack, step by step:

  1. TA writes state v1: failed_attempts = 0
     Encrypted, MACed, stored.
  2. Attacker COPIES the file.  (It is in /data/tee - the
     normal world owns that filesystem.)
  3. Legitimate use: 3 failed attempts. TA writes v2, v3, v4.
     After 5 the TA locks out for ever.
  4. Attacker restores their copy of v1.
  5. TA opens it. Decrypts: correct. MAC: correct. Reads
     failed_attempts = 0.

Nothing was forged. The TA has no way to know that a newer
version ever existed - it has no memory outside the storage
it just read.

AFFECTED BY THIS: anything counted, revoked, or versioned.
    PIN attempt counters
    firmware anti-rollback versions
    one-time-use tokens
    licence consumption
    revoked certificate lists
    monotonic nonces

NOT AFFECTED: things where an old value is as good as a new
one.
    a long-lived signing key
    a device certificate
    a configuration blob that never changes
```

Monotonic counters

```
/* The fix: a counter that hardware guarantees can only go up.
   Then every stored blob carries the counter value it was
   written at, and a replayed blob is detectable.            */

/* --- Source 1: RPMB (chapter 15) --- */
/* The eMMC maintains a write counter it will never decrement,
   and authenticates every access with a key the normal world
   does not have. Restore an old RPMB frame and the device
   rejects it: the counter is stale.

   With CFG_RPMB_FS=y, storing to TEE_STORAGE_PRIVATE_RPMB
   gives you rollback protection for free - this is the
   simplest correct answer where the hardware exists.       */

/* --- Source 2: TA persistent time --- */
/* TEE_GetTAPersistentTime is itself backed by secure storage,
   so it inherits whatever protection that storage has. On an
   RPMB-backed system it is monotonic; on REE FS it is not.  */

/* --- Source 3: a fuse or OTP counter --- */
/* Some SoCs have burnable counters. Genuinely monotonic,
   genuinely limited - typically tens of increments in total.
   Reserve them for irreversible events: "firmware version 3
   has been installed, never accept version 2 again."       */

/* --- Source 4: an external secure element --- */
/* An SE05x, an ATECC, a TPM. A separate chip with its own
   monotonic counters and its own tamper resistance. This is
   what you use when the SoC has nothing and the requirement
   is real.                                                 */

/* --- Source 5: an online server --- */
/* The counter lives elsewhere. Correct and simple, and it
   requires connectivity, which is usually why it is
   rejected.                                                */
```

```
/* Using a counter, concretely: */

struct protected_state {
        uint32_t counter;          /* the value it was written
                                      at                       */
        uint32_t failed_attempts;
        uint8_t  data[64];
};

static TEE_Result read_state(struct protected_state *st)
{
        uint32_t hw_counter;
        TEE_Result res;

        res = read_monotonic_counter(&hw_counter);   /* RPMB, SE */
        if (res) return res;

        res = read_object("state", st, sizeof *st);
        if (res) return res;

        /* THE CHECK. A restored old blob has a lower counter. */
        if (st->counter != hw_counter) {
                EMSG("rollback detected: blob %u, hw %u",
                     st->counter, hw_counter);
                return TEE_ERROR_SECURITY;
        }
        return TEE_SUCCESS;
}

static TEE_Result write_state(struct protected_state *st)
{
        uint32_t hw_counter;
        TEE_Result res;

        /* Increment FIRST, then write. If we crash between the
           two, the counter is ahead of the blob and the next
           read fails safe - it refuses rather than accepting
           an old state.                                       */
        res = increment_monotonic_counter(&hw_counter);
        if (res) return res;

        st->counter = hw_counter;
        return write_object("state", st, sizeof *st);
}

/* The ordering matters and the failure mode must be chosen
   deliberately. "Increment then write" fails CLOSED: a crash
   makes the state unreadable, which for a PIN counter is the
   safe direction. "Write then increment" fails OPEN: a crash
   leaves a valid old state that the attacker can trigger on
   purpose.                                                  */
```

COUNTERS ARE A FINITE RESOURCE

RPMB's write counter is 32 bits and the partition wears out. Fuse counters have tens of increments, total. Do not increment on every operation — increment when the *protected state* changes, and cache reads in the session context. A PIN counter that increments on every successful login as well as every failure burns through its budget for no security benefit.

Firmware anti-rollback

```
The most important use, and the one with a standard shape.

Problem: version 1.0 has a known vulnerability. 1.1 fixes
it. An attacker installs 1.0 again and exploits it.
The 1.0 image is genuine and correctly signed, so signature
verification does not help at all.

Solution: a version counter in fuses or RPMB.

  each image carries a MONOTONIC VERSION NUMBER, distinct
  from its marketing version:
      v1.0 -> nv_counter 1
      v1.1 -> nv_counter 2      (security fix)
      v1.2 -> nv_counter 2      (features only - no bump)
      v2.0 -> nv_counter 3      (another security fix)

  at boot, BL2 checks:
      if (image.nv_counter < fuse.nv_counter) REFUSE
  after a successful boot of a newer image:
      fuse.nv_counter = image.nv_counter    (irreversible)

Bump the counter only for security-relevant fixes: every
bump permanently burns a fuse and permanently forecloses a
downgrade path, including a downgrade you might one day
want because a release was bad.

In TF-A this is TRUSTED_BOARD_BOOT with the
trusted/non-trusted NV counters, and the platform hooks
plat_get_nv_ctr() / plat_set_nv_ctr().
```

Replay of messages, not just storage

```
/* The same problem appears in protocols. A client replays a
   previously valid request; the TA cannot tell.            */

/* WRONG: */
    /* client sends {command, signature} */
    if (verify(sig, command)) do_it();
    /* replayable for ever */

/* RIGHT - a challenge the TA chooses: */

    /* 1. client asks for a challenge */
    case CMD_GET_CHALLENGE:
            TEE_GenerateRandom(sess->challenge,
                               sizeof sess->challenge);
            sess->challenge_valid = true;
            memcpy(out, sess->challenge, sizeof sess->challenge);
            return TEE_SUCCESS;

    /* 2. client signs {challenge || command} */
    case CMD_EXECUTE:
            if (!sess->challenge_valid)
                    return TEE_ERROR_BAD_STATE;
            sess->challenge_valid = false;   /* ONE USE. Clear
                                                it BEFORE doing
                                                the work.      */
            if (!verify_over(sess->challenge, cmd, sig))
                    return TEE_ERROR_SECURITY;
            return do_it();

/* The challenge is per-session state (chapter 13), so it dies
   with the session and cannot be replayed across one. Clearing
   it before the work - not after - means an error path cannot
   leave it usable.                                          */
```

How it shows up

Symptom	Cause
Attempt counter resettable by restoring a file	REE FS with no monotonic counter
Licence expiry bypassed by changing the date	`TEE_GetREETime` used as a security input
Old vulnerable firmware reinstallable	No NV counter check in the boot chain
`TEE_ERROR_TIME_NEEDS_RESET`	Persistent time was tampered with or lost — treat it as unusable, do not paper over it
RPMB writes start failing after long service	Counter or wear budget exhausted — you are incrementing too often
State unreadable after a power cut	Counter incremented but the write did not complete — the fail-closed design working
Same request accepted twice	No challenge, or the challenge not cleared after use
Rollback protection works on hardware, not on QEMU	Emulated RPMB is a file the normal world can restore
Cannot downgrade a bad release	NV counter bumped for a non-security change. This is permanent

Check yourself

- ✓ Name the three time sources and rank them by trustworthiness.
- ✓ Explain the four return values of `TEE_GetTAPersistentTime`.
- ✓ Show why an expiry check against `TEE_GetREETime` is not a check at all.
- ✓ Describe the rollback attack in five steps and say what cryptography does not prevent.
- ✓ Classify five kinds of data by whether rollback matters.
- ✓ Name five sources of monotonic counters and their limitations.
- ✓ Write a read/write pair that detects rollback, and justify the ordering.
- ✓ Explain the difference between failing closed and failing open here.
- ✓ Describe firmware anti-rollback and say when to bump the counter.
- ✓ Design a challenge-response that prevents request replay, and say when to clear the challenge.

Pseudo TAs and core drivers

A pseudo TA is a service compiled into OP-TEE itself, running at S-EL1 with the trusted OS's full privilege and none of its isolation. Sometimes that is the only way to do the job. Usually it is not, and the difference is worth being precise about before you write one.

Where a pseudo TA sits

```
S-EL0   +----------+  +----------+
        | user TA  |  | user TA  |   isolated from each
        |          |  |          |   other and from the core
        +----------+  +----------+   by the MMU
              |             |
              |  SVC        |
              v             v
S-EL1   +-----------------------------------------+
        |  OP-TEE CORE                            |
        |   scheduler, MMU, crypto, drivers       |
        |                                         |
        |   +-----------------------------------+ |
        |   |  PSEUDO TA                        | |
        |   |  same privilege as the core.      | |
        |   |  same address space as the core.  | |
        |   |  can call ANY core function.      | |
        |   |  a bug here is a bug in the       | |
        |   |  trusted OS.                       |  |
        |   +-----------------------------------+ |
        +-----------------------------------------+

A user TA that dereferences a wild pointer: that TA panics,
its session dies, the system continues.

A pseudo TA that dereferences a wild pointer: the trusted OS
panics, or worse, silently corrupts another TA's memory or
its own key material.

The privilege is the same as writing kernel code. Treat it
that way.
```

When it is actually right

Reason	Valid?
You need a core API not exposed to S-EL0 — direct hardware access, a core driver, internal state	**Yes.** This is the main reason
You must work before the normal world can load anything — early boot, before `tee-supplicant`	**Yes** — though an *early TA* may serve, and keeps S-EL0 isolation
You are implementing a service other TAs will call	**Sometimes.** A user TA can also be called by another TA
You need to service an interrupt in the secure world	**Yes** — as a core driver, not really as a TA
"It is faster"	**No.** You save one privilege transition, tens of nanoseconds, against a call that costs tens of microseconds
"It is simpler than packaging a .ta"	**No.** That is an hour of build setup against a permanent expansion of the trusted computing base

Reason	Valid?
"It avoids depending on tee-supplicant"	**Use an early TA instead** — same benefit, keeps isolation

Writing one

```
/* core/pta/my_service.c */

#include <kernel/pseudo_ta.h>
#include <kernel/user_access.h>
#include <mm/core_memprot.h>
#include <string.h>
#include <trace.h>

#define PTA_MY_SERVICE_UUID \
        { 0x7c1a9e30, 0x4d6b, 0x4f21, \
          { 0x9a, 0x3c, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66 } }

#define TA_NAME  "my_service.pta"

#define PTA_CMD_READ_REG    0
#define PTA_CMD_WRITE_REG   1

static TEE_Result read_reg(uint32_t ptypes, TEE_Param params[4])
{
        uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INPUT,
                                       TEE_PARAM_TYPE_VALUE_OUTPUT,
                                       TEE_PARAM_TYPE_NONE,
                                       TEE_PARAM_TYPE_NONE);
        vaddr_t base;
        uint32_t off;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        off = params[0].value.a;

        /* WHITELIST. A pseudo TA can read ANY address in the
           system. The client must never choose one. */
        if (off >= MY_DEVICE_SIZE || (off & 3))
                return TEE_ERROR_BAD_PARAMETERS;

        base = (vaddr_t)phys_to_virt_io(MY_DEVICE_BASE,
                                        MY_DEVICE_SIZE);
        if (!base)
                return TEE_ERROR_GENERIC;

        params[1].value.a = io_read32(base + off);
        return TEE_SUCCESS;
}

static TEE_Result invoke_command(void *sess __unused,
                                 uint32_t cmd,
                                 uint32_t ptypes,
                                 TEE_Param params[4])
{
        switch (cmd) {
        case PTA_CMD_READ_REG:  return read_reg(ptypes, params);
        case PTA_CMD_WRITE_REG: return write_reg(ptypes, params);
        default:                return TEE_ERROR_NOT_SUPPORTED;
        }
}

static TEE_Result open_session(uint32_t ptypes __unused,
                               TEE_Param params[4] __unused,
                               void **sess __unused)
```

```
{
        /* WHO may call this? A pseudo TA can and should
           restrict its callers. */
        struct ts_session *s = ts_get_calling_session();

        if (!s || !is_user_ta_ctx(s->ctx)) {
                /* called from the normal world directly */
                return TEE_ERROR_ACCESS_DENIED;
        }
        return TEE_SUCCESS;
}

pseudo_ta_register(.uuid  = PTA_MY_SERVICE_UUID,
                   .name  = TA_NAME,
                   .flags = PTA_DEFAULT_FLAGS,
                   .open_session_entry_point = open_session,
                   .invoke_command_entry_point = invoke_command);

/* Flags:
     TA_FLAG_SINGLE_INSTANCE     always, for a PTA
     TA_FLAG_MULTI_SESSION
     TA_FLAG_INSTANCE_KEEP_ALIVE
     TA_FLAG_CONCURRENT          allow concurrent invocations -
                                 you must then be thread safe
     PTA_DEFAULT_FLAGS = SINGLE_INSTANCE | MULTI_SESSION |
                         INSTANCE_KEEP_ALIVE                */
```

```
# core/pta/sub.mk
srcs-$(CFG_MY_SERVICE_PTA) += my_service.c

# core/pta/pta.mk  (or the platform's conf.mk)
CFG_MY_SERVICE_PTA ?= y

$ make CFG_MY_SERVICE_PTA=y
# It is now inside tee.elf. No .ta file, no signing, no
# tee-supplicant, and no way to update it without replacing
# OP-TEE.
```

TWO RULES THAT ARE NOT OPTIONAL IN A PSEUDO TA

Validate every address. A user TA that dereferences a client-supplied pointer faults harmlessly at S-EL0. A pseudo TA reads or writes whatever is there — another TA's key, the core's own state, a device register. Whitelist offsets against a known region; never accept an address.

Copy client data before using it. The double-fetch rule from chapter 7 applies with much worse consequences here. Use `copy_from_user` / `copy_to_user` when the caller is a user TA, and never act on shared memory in place.

The pseudo TAs that ship

PTA	Provides
`device.pta`	Enumerates TAs for the Linux TEE bus (chapter 10)
`system.pta`	Dynamic shared memory, TA-to-TA calls, entropy
`attestation.pta`	Measures a TA's binary and signs the measurement — remote attestation
`stats.pta`	The memory and TA statistics `xtest --stats` reads

PTA	Provides
`rtc.pta`, `scmi.pta`, `gprof.pta`	Real-time clock, SCMI, profiling
`secstor_ta_mgmt.pta`	Installing TAs into secure storage
`apply_patch.pta`	Live patching of the core

```
# Read these before writing your own - they are the reference
# for style and for how much validation is expected:
$ ls optee_os/core/pta/
attestation.c  benchmark.c  device.c  gprof.c  invoke_tests.c
rtc.c  scmi.c  secstor_ta_mgmt.c  stats.c  system.c ...

# attestation.c is the most instructive: it is short, it
# handles untrusted input carefully, and it does something a
# user TA genuinely cannot.
```

Early TAs: the middle option

```
/* An early TA is an ORDINARY user TA, at S-EL0, with full MMU
   isolation - but its binary is linked into tee.elf rather
   than loaded from the normal world.

   You get:
     + available before tee-supplicant, before Linux
     + no dependency on the normal-world filesystem
     + full S-EL0 isolation - a bug is contained
     + no separate signing/installation step
   You lose:
     - it cannot be updated without replacing OP-TEE
     - it occupies TEE_RAM permanently

   This is almost always the right answer when the reason you
   were reaching for a pseudo TA was "it must work early" or
   "I do not want to depend on the supplicant".            */

$ make CFG_EARLY_TA=y \
       EARLY_TA_PATHS="path/to/7f2e9c1a-....ta"

/* The TA is still signed, still verified, and still runs at
   S-EL0 - it is simply carried inside the OP-TEE image
   instead of fetched at runtime.                          */
```

TA-to-TA calls

```
/* A user TA can open a session to another TA. Often this is
   what you actually wanted when you considered a pseudo TA -
   a shared service, without giving it core privilege.      */

    TEE_TASessionHandle sess = TEE_HANDLE_NULL;
    TEE_UUID other = OTHER_TA_UUID;
    uint32_t origin;
    TEE_Param params[4] = {0};
    TEE_Result res;

    res = TEE_OpenTASession(&other, TEE_TIMEOUT_INFINITE,
                            0, params, &sess, &origin);
    if (res) return res;

    res = TEE_InvokeTACommand(sess, TEE_TIMEOUT_INFINITE,
                              CMD_DO_THING,
                              TEE_PARAM_TYPES(
                                  TEE_PARAM_TYPE_MEMREF_INPUT,
                                  TEE_PARAM_TYPE_NONE,
                                  TEE_PARAM_TYPE_NONE,
                                  TEE_PARAM_TYPE_NONE),
                              params, &origin);

    TEE_CloseTASession(sess);

/* The callee sees the caller's identity as
   TEE_LOGIN_TRUSTED_APP, with the caller's UUID - so it can
   authorise. Two S-EL0 TAs, isolated from each other, with an
   authenticated channel between them.

   CAUTION: the calling TA's thread is held for the duration
   (chapter 5), so a chain of TA-to-TA calls consumes one
   thread, not several - but a CYCLE deadlocks immediately.
   Keep the call graph acyclic and shallow.                */
```

How it shows up

Symptom	Cause
The whole TEE panics when a client misbehaves	A pseudo TA dereferencing a client-supplied address
One TA's data appears in another	A pseudo TA writing outside its intended region — no isolation to stop it
Pseudo TA works, cannot be updated in the field	By design: it is inside a signed OP-TEE image
Intermittent corruption under concurrency	`TA_FLAG_CONCURRENT` set without making the code thread safe
Normal-world client can reach an internal service	No caller check in `open_session`
TA-to-TA call hangs for ever	A cycle in the call graph, or the callee blocked
Service needed before `tee-supplicant`	Use an early TA rather than a pseudo TA
Security review rejects a pseudo TA	Usually correctly. Justify the privilege or move it to S-EL0

Check yourself

- ✓ State the privilege and isolation difference between a user TA and a pseudo TA.
- ✓ Give two valid reasons to write a pseudo TA and three invalid ones.

- ✓ Explain why "it is faster" does not justify one.
- ✓ Write a pseudo TA that exposes one register, with correct validation.
- ✓ Explain why address whitelisting is mandatory here and merely advisable in a user TA.
- ✓ Say how a pseudo TA restricts who may call it.
- ✓ Name four built-in pseudo TAs and what each provides.
- ✓ Describe an early TA and say which pseudo-TA use cases it replaces.
- ✓ Explain TA-to-TA calls, what identity the callee sees, and the deadlock hazard.
- ✓ Given a proposed pseudo TA, decide whether a user TA or an early TA would do instead.

Debugging OP-TEE and trusted applications

Debugging the secure world is harder than debugging Linux and easier than debugging firmware, and the tools are unfamiliar rather than absent. This chapter is the toolkit, ordered by how often you will need each thing, starting with the one that answers most questions.

Reading a panic

```
# A TA panic looks like this on the SECURE console. It is
# dense and it contains everything you need.

E/TC:0 0 tee_ta_verify_param:...  invalid memory access
E/TC:0 0 Status of TA 7f2e9c1a-3b4d-4e5f-8a9b-0c1d2e3f4a5b
E/TC:0 0  arch: aarch64  load address: 0x40016000
E/LD:  arch: aarch64
E/LD:  region  0: va 0x40016000 pa 0x0e56d000 size 0x002000 flags r-x
E/LD:  region  1: va 0x40018000 pa 0x0e56f000 size 0x001000 flags rw-
E/LD:  region  2: va 0x40019000 pa 0x0e570000 size 0x008000 flags rw- (stack)
E/TC:0 0 Core data-abort at address 0x0 (translation fault)
E/TC:0 0  esr 0x92000006  ttbr0 0x...  cidr 0x0
E/TC:0 0  cpu #0          cpsr 0x60000100
E/TC:0 0  x0 0000000000000000 x1 0000000040018210
E/TC:0 0  x2 0000000000000020 x3 0000000000000000
...
E/TC:0 0  elr 0x40016a3c
E/TC:0 0 Call stack:
E/TC:0 0  0x40016a3c
E/TC:0 0  0x40016b10
E/TC:0 0  0x400171d4

# THE PROCEDURE:
#
# 1. Note the LOAD ADDRESS: 0x40016000
# 2. Subtract it from each call-stack address:
#      0x40016a3c - 0x40016000 = 0xa3c
#      0x40016b10 - 0x40016000 = 0xb10
#      0x400171d4 - 0x40016000 = 0x11d4
# 3. Look those offsets up in the TA's ELF:

$ aarch64-linux-gnu-addr2line -f -p -e ta/my_ta.elf 0xa3c 0xb10 0x11d4
do_sign at /work/my_project/ta/my_ta.c:142
handle_command at /work/my_project/ta/my_ta.c:201
TA_InvokeCommandEntryPoint at /work/my_project/ta/my_ta.c:230

# 4. Decode the ESR: 0x92000006
#      EC = 0x24  data abort from a lower EL
#      DFSC = 0x06  translation fault, level 2
#      WnR = 1     it was a WRITE
#    So: a write to an unmapped address - and FAR says 0x0.
#    A NULL pointer write at my_ta.c:142.

# OP-TEE ships a script that does steps 1-3 for you:
$ optee_os/scripts/symbolize.py -d ta/ -d optee_os/out/.../core/ \
    < secure-console.log
# Pipe the whole console through it and every address becomes
# a file and line. Set this up once and never do the
# arithmetic again.
```

BUILD FOR SYMBOLISATION BEFORE YOU NEED IT

Two flags make panics readable: `CFG_UNWIND=y` for the call stack, and `CFG_CORE_ASLR=n` plus `CFG_TA_ASLR=n` so addresses are stable between runs. Keep the unstripped `.elf` next to every `.ta` you ship to a test device — without it, a panic log is a list of numbers.

Logging

```
# Levels, set at build time:
$ make CFG_TEE_CORE_LOG_LEVEL=3 CFG_TEE_TA_LOG_LEVEL=4
#   0 none  1 error  2 info  3 debug  4 flow

# In a TA:
    EMSG("failed: 0x%08x", res);    /* always              */
    IMSG("state now %u", n);        /* level 2             */
    DMSG("entering with %p", p);    /* level 3             */
    FMSG("tick");                   /* level 4             */
    DHEXDUMP(buf, len);             /* a hex dump          */

# Output carries the function and line automatically:
E/TA: do_sign:142 failed: 0xffff0006

# In the CORE, the same macros plus:
    MSG_RAW(...)      no prefix
    EMSG_RAW(...)
    panic("message"); /* takes the whole TEE down          */

# Per-TA level, without rebuilding everything - set it in the
# TA's own Makefile:
    CFG_TEE_TA_LOG_LEVEL ?= 4

# Level 4 on the core is extremely verbose and genuinely
# useful when you are lost:
F/TC:0 0 thread_alloc_and_run:...  thread 0
F/TC:0 0 tee_ta_open_session:...   uuid 7f2e9c1a-...
F/TC:0 0 ldelf_load_ta:...
F/TC:0 0 tee_ta_invoke_command:... cmd 0
# You can see exactly how far a call got before it failed.
```

GDB on the secure world

```
# QEMU's stub debugs every exception level (QEMU chapter 11).
# The secure world is not special.

$ cd ~/optee/build
$ make run-only GDBSERVER=y
# or add -s -S to the QEMU command line yourself

$ aarch64-linux-gnu-gdb
(gdb) target remote :1234

# Load symbols for every image, at its link address:
(gdb) add-symbol-file ~/optee/optee_os/out/arm-plat-vexpress/core/tee.elf \
        0x0e100000
(gdb) add-symbol-file ~/optee/trusted-firmware-a/build/qemu/release/bl31/bl31.elf
(gdb) add-symbol-file ~/optee/linux/vmlinux

# Break inside OP-TEE:
(gdb) break tee_ta_open_session
(gdb) break tee_entry_std
(gdb) break panic
(gdb) continue

# Which world are we in?
(gdb) monitor info registers | grep -E 'PSTATE|SCR'
# PSTATE ... EL1h  with SCR_EL3.NS == 0  ->  secure world

# A TA is loaded at a runtime address, so its symbols must be
# added AFTER it loads:
(gdb) break tee_ta_invoke_command
(gdb) continue
(gdb) p ctx->ts_ctx           # find the load address
(gdb) add-symbol-file ta/my_ta.elf 0x40016000
(gdb) break do_sign

# OP-TEE ships GDB helpers:
(gdb) source ~/optee/optee_os/scripts/gdb/optee.py
(gdb) optee_ta_list
(gdb) optee_mem_map
(gdb) optee_threads

# On real hardware you need a JTAG probe that can see the
# secure world - and many cannot, or are disabled by fuses in
# production. That asymmetry is why so much OP-TEE development
# happens on QEMU.
```

Function tracing inside a TA

```
# OP-TEE can instrument a TA's every function entry and exit,
# and dump the trace through tee-supplicant.

$ make CFG_TA_GPROF_SUPPORT=y CFG_FTRACE_SUPPORT=y \
       CFG_SYSCALL_FTRACE=y

# Build the TA with instrumentation:
#   in ta/Makefile:
    CFG_FTRACE_SUPPORT=y
#   or per-file:
    cflags-my_ta.c-y += -pg

$ tee-supplicant -d
# Run the client, then:
$ ls /tmp/ftrace-*.out
$ optee_os/scripts/ftrace_format.py /tmp/ftrace-7f2e9c1a-....out

  0.000 us | TA_InvokeCommandEntryPoint() {
  1.200 us |   do_sign() {
  0.400 us |     load_key() {
 12.100 us |       TEE_OpenPersistentObject();
             }   /* load_key: 13.900 us */
 45.300 us |     TEE_AsymmetricSignDigest();
             }   /* do_sign: 60.100 us */
             } /* TA_InvokeCommandEntryPoint: 61.500 us */

# This answers "where is the time going" definitively, and the
# answer is usually "the one call that does an RPC". Above,
# TEE_OpenPersistentObject costs 12 us of a 61 us call -
# cache it in the session context and the call halves.

# gprof-style profiling, for the same question aggregated:
$ make CFG_TA_GPROF_SUPPORT=y
$ ls /tmp/gmon-*.out
$ aarch64-linux-gnu-gprof ta/my_ta.elf /tmp/gmon-....out
```

Sanitizers

```
# OP-TEE can build itself and TAs with ASan and UBSan. On
# QEMU, where you have memory to spare, leave them on during
# development.

$ make CFG_CORE_SANITIZE_UNDEFINED=y \
       CFG_CORE_SANITIZE_KADDRESS=y \
       CFG_TA_SANITIZE_UNDEFINED=y

# A UBSan hit on the secure console:
E/TC:0 0 Undefined behavior at my_ta.c:88
E/TC:0 0   left shift of 1 by 31 places cannot be represented
           in type 'int'

# An ASan hit:
E/TC:0 0 KASAN: heap-buffer-overflow at 0x40018210
E/TC:0 0   write of size 4 at offset 68 of a 64-byte region

# These find, before shipping, exactly the class of bug that a
# security researcher would find after. The cost is memory and
# speed, both of which you have in development and neither of
# which matters in a test.

# Also worth running, and cheap:
$ xtest                       # the full GP conformance suite
$ xtest -t regression
$ xtest --stats               # memory high-water marks (ch. 13)
$ xtest -t benchmark
```

A procedure for "it does not work"

```
Work down. Each step is cheap and eliminates a layer.

1. Is there output on the SECURE console at all?
     no  -> the second serial port is not set up (ch. 4).
            Fix that before anything else.

2. What is the CLIENT's result and ORIGIN? (chapter 8)
     origin 1 (API)   -> your client's arguments
     origin 2 (COMMS) -> transport: /dev/tee0, the driver
     origin 3 (TEE)   -> OP-TEE core rejected it. The
                         secure console says why.
     origin 4 (TA)    -> your TA returned this deliberately

3. 0xFFFF0008 ITEM_NOT_FOUND?
     -> the .ta is not in /lib/optee_armtz, or the filename
        does not match the UUID, or tee-supplicant is not
        running.

4. 0xFFFF000F SECURITY?
     -> signed with the wrong key (chapter 14).

5. 0xFFFF3024 TARGET_DEAD?
     -> the TA panicked. Symbolize the log. Go to the top
        of this chapter.

6. 0xFFFF0006 BAD_PARAMETERS with origin 4?
     -> your paramTypes check is failing. Print what you
        got and what you expected:
          EMSG("ptypes 0x%08x expected 0x%08x", ptypes, exp);
        This one line resolves it in seconds and it is
        worth leaving in permanently.

7. It hangs?
     -> thread exhaustion or a supplicant deadlock (ch. 9).
        cat /proc/<client>/stack and
        cat /proc/<tee-supplicant>/stack

8. It works, then stops after a while?
     -> a leak. xtest --stats, and check handle cleanup on
        every error path (chapter 12).

9. Still lost?
     -> CFG_TEE_CORE_LOG_LEVEL=4 and read the flow trace.
        It shows exactly how far the call got.
```

How it shows up

Symptom	Cause
Panic log is just hex addresses	No `.elf` kept, or ASLR on. Rebuild with `CFG_TA_ASLR=n` and keep the ELF
Call stack is empty	`CFG_UNWIND=n`
`addr2line` gives nonsense	Forgot to subtract the load address, or used the `.ta` instead of the `.elf`
No output from `DMSG`	`CFG_TEE_TA_LOG_LEVEL` below 3
GDB breakpoint in a TA never hits	Symbols added before the TA was loaded, or at the wrong address
ftrace produces nothing	`tee-supplicant` not running, or the TA not built with `-pg`

Symptom	Cause
Sanitizer build runs out of memory	ASan needs shadow memory; raise `TA_DATA_SIZE` and `TZDRAM_SIZE`
Cannot attach a debugger on real hardware	Secure debug disabled by fuses. Expected in production
Bug disappears with logging enabled	A race — logging changed the timing. Look for shared state under `MULTI_SESSION`

Check yourself

- ✓ Given a panic log, recover the source file and line for every frame.
- ✓ Decode an ESR from a TA panic into fault type and direction.
- ✓ Name the two build flags that make panics readable.
- ✓ Give the four trace macros and the level each requires.
- ✓ Attach GDB to the secure world and break inside OP-TEE.
- ✓ Explain how to get symbols for a TA that is loaded at runtime.
- ✓ Use ftrace to find where a TA invocation spends its time.
- ✓ Name the sanitizer build flags and what each catches.
- ✓ Walk the nine-step procedure and say what each step eliminates.
- ✓ Explain why the return origin is the second thing to check, after the secure console.

Writing a TA that resists its own client

The client is the attacker. Not "might be" — the entire reason your TA exists is that the normal world may be compromised, so you must write every entry point assuming the caller is hostile, competent and patient. This chapter is the checklist, ordered by how often each mistake appears in real audits.

The mental shift

```
   ORDINARY LIBRARY                TRUSTED APPLICATION

   caller is your own code         caller is an attacker
   arguments are well-formed       arguments are chosen to break
                                     you
   a crash is a bug                a crash is a denial of
                                     service, and a controlled
                                     crash may be an exploit
   memory is stable                shared memory CHANGES WHILE
                                     YOU READ IT
   an error message helps          an error message may leak the
                                     secret
   timing does not matter          timing IS a channel

   Every parameter, every length, every index, every pointer.
   There is no trusted input.
```

1. Check parameter types. Always.

```
TEE_Result TA_InvokeCommandEntryPoint(void *sess, uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);

        if (ptypes != exp) {
                EMSG("ptypes 0x%08x expected 0x%08x", ptypes, exp);
                return TEE_ERROR_BAD_PARAMETERS;
        }
        ...
}

/* WITHOUT this, a client passes VALUE where you expect MEMREF.
   You then read params[0].memref.buffer - which is now an
   arbitrary 64-bit number the attacker chose - and
   dereference it with secure privilege.

   This is the number one OP-TEE TA vulnerability class. It is
   four lines. Check EVERY command, and check it in
   TA_OpenSessionEntryPoint too.                              */
```

2. Never read shared memory twice

```
/* WRONG - the classic double fetch. */

static TEE_Result bad(TEE_Param params[4])
{
        struct hdr *h = params[0].memref.buffer;

        if (h->len > MAX)                /* fetch 1: check    */
                return TEE_ERROR_BAD_PARAMETERS;

        /* the client's other thread changes h->len HERE */

        memcpy(dst, h->data, h->len);    /* fetch 2: use      */
        /* h->len is now 0xFFFFFFFF. Heap overflow, in the
           secure world, with attacker-controlled data.      */
}

/* RIGHT - copy once, then work only on the copy. */

static TEE_Result good(TEE_Param params[4])
{
        struct hdr local;
        uint8_t    buf[MAX];

        if (params[0].memref.size < sizeof local)
                return TEE_ERROR_BAD_PARAMETERS;

        /* ONE read of the shared region. */
        TEE_MemMove(&local, params[0].memref.buffer, sizeof local);

        /* Validate the COPY. It cannot change now. */
        if (local.len > MAX ||
            local.len > params[0].memref.size - sizeof local)
                return TEE_ERROR_BAD_PARAMETERS;

        TEE_MemMove(buf,
                    (uint8_t *)params[0].memref.buffer + sizeof local,
                    local.len);

        return process(buf, local.len);
}

/* The rule, stated once: SHARED MEMORY IS A NETWORK
   CONNECTION. You read from it once, into your own storage,
   and everything after that operates on your storage.     */
```

> **THIS IS THE BUG THAT GETS CVES**
>
> Search any TEE vendor's advisory list and the double fetch appears again and again, because the code looks correct — the check is right there, immediately above the use. The window is a few instructions wide and an attacker with another thread will hit it. There is no defence except never fetching twice.

3. Validate every length and index

```
/* Integer overflow in a bounds check is the second-commonest
   finding.                                                    */

    /* WRONG: offset + len can wrap */
    if (offset + len > buffer_size)
            return TEE_ERROR_BAD_PARAMETERS;
    /* offset = 0xFFFFFFF0, len = 0x20  ->  sum wraps to 0x10,
       check passes, and you read far outside the buffer.  */

    /* RIGHT: rearrange so nothing can overflow */
    if (offset > buffer_size || len > buffer_size - offset)
            return TEE_ERROR_BAD_PARAMETERS;

    /* Also wrong: */
    uint32_t total = count * sizeof(struct item);   /* wraps  */
    /* Right: */
    if (count > UINT32_MAX / sizeof(struct item))
            return TEE_ERROR_OVERFLOW;

    /* Signedness: */
    int idx = params[0].value.a;      /* a uint32_t from the
                                         client              */
    if (idx < n)                      /* -1 < n is TRUE       */
            table[idx] = ...;         /* writes before the
                                         table               */
    /* Use unsigned types for anything that indexes. */

    /* And check ranges explicitly, not implicitly: */
    if (params[0].value.a >= ARRAY_SIZE(g_slots))
            return TEE_ERROR_BAD_PARAMETERS;
```

4. Validate pointers with the API

```
    res = TEE_CheckMemoryAccessRights(
                TEE_MEMORY_ACCESS_READ |
                TEE_MEMORY_ACCESS_ANY_OWNER,
                params[0].memref.buffer,
                params[0].memref.size);
    if (res != TEE_SUCCESS)
            return res;

/* This asks OP-TEE whether that pointer, for that length, is
   really memory this session was given. It catches a client
   that registered a 4 KB buffer and then claimed 4 MB.

   It does NOT catch a double fetch - the memory is legitimate,
   it just changes. Both checks are needed.               */

/* When your TA is called BY ANOTHER TA (chapter 18), the
   buffer may be secure memory:
      TEE_MEMORY_ACCESS_SECURE      it must be secure
      TEE_MEMORY_ACCESS_NONSECURE   it must not be
   Being explicit about which you expect stops a confused
   deputy.                                                 */
```

5. Write output once, on success only

```
/* WRONG - leaks intermediate state on the error path. */
    memcpy(out, partial_result, n);      /* client can see this */
    if (verify_something() != OK)
            return TEE_ERROR_SECURITY;   /* too late            */

/* RIGHT */
    uint8_t local_out[MAX];

    res = compute(local_out, &n);
    if (res != TEE_SUCCESS)
            return res;                  /* client sees nothing */

    if (params[1].memref.size < n) {
            params[1].memref.size = n;   /* the ONE allowed
                                            early write         */
            return TEE_ERROR_SHORT_BUFFER;
    }
    TEE_MemMove(params[1].memref.buffer, local_out, n);
    params[1].memref.size = n;
    return TEE_SUCCESS;

/* For a crypto TA the intermediate state often IS the secret -
   a partially decrypted block, a key schedule, an unverified
   plaintext. Compute in private memory, publish at the end. */
```

6. Constant time where it matters

```
/* Comparison, of anything secret: */

    /* WRONG - returns as soon as bytes differ */
    if (TEE_MemCompare(mac, expected, 32) == 0) ...
    if (memcmp(pin, stored, 4) == 0) ...

    /* RIGHT */
    static bool ct_equal(const uint8_t *a, const uint8_t *b, size_t n)
    {
            uint8_t d = 0;
            for (size_t i = 0; i < n; i++)
                    d |= a[i] ^ b[i];
            return d == 0;
    }
    /* or, for MACs, the API does it: */
    TEE_MACCompareFinal(op, msg, len, mac, maclen);

/* Branching on secrets: */

    /* WRONG - the branch is observable */
    if (key[0] & 1)
            do_a();
    else
            do_b();

    /* Table lookups indexed by secrets are also observable,
       through the cache. This is how AES T-table
       implementations were broken. Use the hardware
       accelerator, or an implementation written to be
       constant time - do not write your own.              */

/* Error messages that distinguish cases: */

    /* WRONG - an oracle */
    if (!user_exists(u)) return TEE_ERROR_ITEM_NOT_FOUND;
    if (!pin_matches(p)) return TEE_ERROR_ACCESS_DENIED;

    /* RIGHT - one answer, one timing */
    bool ok = user_exists(u);
    ok &= pin_matches(p);          /* NOT &&: always evaluate */
    return ok ? TEE_SUCCESS : TEE_ERROR_ACCESS_DENIED;
```

7. Fail closed, and rate limit

```
    /* Default to refusal, not permission. */
    TEE_Result res = TEE_ERROR_ACCESS_DENIED;   /* not SUCCESS */
    ...
    if (everything_checked_out)
            res = TEE_SUCCESS;
    return res;

    /* A missing case in a switch must deny: */
    switch (cmd) {
    case CMD_A: return do_a(...);
    case CMD_B: return do_b(...);
    default:    return TEE_ERROR_NOT_SUPPORTED;   /* never
                                                     fall
                                                     through */
    }

    /* Anything guessable needs a counter, and the counter
       needs rollback protection (chapter 17): */
    if (state.failures >= MAX_FAILURES)
            return TEE_ERROR_ACCESS_DENIED;

    state.failures++;
    write_protected_state(&state);      /* BEFORE checking -
                                           so a power cut
                                           during the attempt
                                           counts against the
                                           attacker           */
    if (check_pin(pin)) {
            state.failures = 0;
            write_protected_state(&state);
            return TEE_SUCCESS;
    }
    return TEE_ERROR_ACCESS_DENIED;
```

8. Clean up on every path

```
static TEE_Result do_work(TEE_Param params[4])
{
        TEE_OperationHandle op  = TEE_HANDLE_NULL;
        TEE_ObjectHandle    key = TEE_HANDLE_NULL;
        uint8_t             secret[32] = {0};
        TEE_Result          res;

        res = TEE_AllocateOperation(&op, ...);
        if (res) goto out;

        res = TEE_AllocateTransientObject(&key, ...);
        if (res) goto out;

        res = actually_do_it(op, key, secret);
out:
        /* Wipe secrets BEFORE freeing - the heap is reused
           and OP-TEE does not zero on free.               */
        TEE_MemFill(secret, 0, sizeof secret);
        TEE_FreeOperation(op);          /* NULL-safe          */
        TEE_FreeTransientObject(key);   /* NULL-safe          */
        return res;
}

/* A leak here is not merely untidy: TA_DATA_SIZE is small
   (chapter 13), so a leak on an error path that an attacker
   can trigger repeatedly is a denial of service against the
   whole TA.                                                  */
```

The review checklist

#	Check	Look for
1	`paramTypes` checked in every entry point	A command with no `exp` comparison
2	No value read twice from shared memory	Any `params[i].memref.buffer` dereferenced more than once
3	Arithmetic cannot overflow	`a + b > limit`, `n * size`, signed indices
4	`TEE_CheckMemoryAccessRights` on client buffers	Missing before a large copy
5	Output written once, at the end	A write to `memref.buffer` before the final `return TEE_SUCCESS`
6	Constant-time comparison of secrets	`memcmp`, `TEE_MemCompare` on a MAC or PIN
7	Fails closed	A `switch` without `default`; a result variable initialised to success
8	Every handle freed, every secret wiped	An early `return` that skips cleanup
9	Rate limiting with rollback protection	A counter in REE FS (chapter 17)
10	No secrets in logs	`DHEXDUMP`, `%s` of key material
11	Session state is per session	A key in a global under `SINGLE_INSTANCE`
12	Thread safety under `MULTI_SESSION`	Any mutable global (chapter 13)
13	Attack surface is minimal	Commands that exist "for debugging"
14	No untrusted parsing	ASN.1, protobuf, JSON, X.509 inside the TA

> **ROW 14 DESERVES EMPHASIS**
> A parser for a complex, attacker-controlled format is the largest single source of memory-safety bugs in any codebase, and putting one inside the secure world concentrates the risk exactly where it hurts most. Parse in Linux; pass the TA a fixed-size, fully validated struct. If the TA genuinely must parse — a certificate, a TPM command — keep the parser small, total, and fuzzed.

Testing hostilely

```
/* Write a client whose job is to break your TA. Run it in CI. */

    /* wrong parameter types for every command */
    for (uint32_t pt = 0; pt < 0x10000; pt++) {
            op.paramTypes = pt;
            TEEC_InvokeCommand(&sess, cmd, &op, &origin);
            /* must never panic the TA */
    }

    /* boundary sizes */
    size_t sizes[] = {0, 1, MAX-1, MAX, MAX+1, SIZE_MAX};

    /* concurrent mutation - the double-fetch test */
    pthread_create(&t, NULL, mutate_shared_buffer_forever, shm);
    for (int i = 0; i < 100000; i++)
            TEEC_InvokeCommand(&sess, cmd, &op, &origin);

    /* every command in every order, including before the
       session is properly set up */

    /* resource exhaustion: open sessions until it fails, then
       check the TA still works after they close */

/* Fuzzing the parsing code natively is cheaper and finds more:
   compile the TA's pure logic for the host with
   -fsanitize=fuzzer,address and run libFuzzer against it. The
   GP API calls need stubbing, which is a good exercise in
   itself - it forces the security-relevant logic to be
   separable from the plumbing.                              */
```

How it shows up

Symptom	Cause
TA panics when a client sends odd arguments	Missing `paramTypes` check — and this is the mild version of that bug
Heap corruption only under concurrency	Double fetch
Read or write far outside a buffer	Integer overflow in a bounds check
Client observes partial results on failure	Output written before success
PIN recoverable in far fewer attempts than expected	Timing oracle, or distinguishable error codes
Attempt counter resettable	Rate limiting without rollback protection
TA stops working after repeated failed calls	A leak on an error path an attacker can trigger at will
Two clients interfere	Shared global state under `MULTI_SESSION`
Security review finds the whole design wrong	Untrusted parsing inside the TA

Symptom	Cause
Keys in a log from a shipped device	`DHEXDUMP` in a release build

Check yourself

- ✓ State the assumption you must make about every client.
- ✓ Write the `paramTypes` check and explain the attack it prevents.
- ✓ Explain the double fetch with a concrete example and give the rule that prevents it.
- ✓ Rewrite `offset + len > size` so it cannot overflow.
- ✓ Say what `TEE_CheckMemoryAccessRights` catches and what it does not.
- ✓ Explain why output must be written once, at the end.
- ✓ Write a constant-time comparison and say why `TEE_MemCompare` is unsuitable.
- ✓ Give two ways an error path can become an oracle.
- ✓ Explain why a leak on an error path is a security bug in a TA.
- ✓ Explain why untrusted parsing does not belong in the secure world.
- ✓ Design a hostile test client covering at least four of the checklist rows.

Project: a secure key store, client and TA

A complete, working pair: a trusted application that generates ECDSA keys which can never leave the secure world, and a command-line client that drives it. Everything from the previous twenty chapters appears here at once, and the result is a genuinely useful thing — a miniature Keymint.

What we are building

```
   ks create <name>        generate a P-256 keypair inside the
                           TEE. The private half never exists
                           outside it.
   ks pubkey <name>        print the public key
   ks sign   <name> <file> sign a file's SHA-256 digest
   ks verify <name> <file> <sig>
   ks list                 list stored key names
   ks delete <name>

   PROPERTIES WE ARE ENFORCING:

     * the private key is generated by TEE_GenerateKey and
       stored as a persistent OBJECT (chapter 15). It is never
       serialised into a buffer.
     * asking for the private attribute returns
       TEE_ERROR_ACCESS_DENIED - structurally, not by policy.
     * keys are scoped to the calling user (LOGIN_USER), so two
       Linux users cannot see each other's keys.
     * the client passes a NAME, never a pointer or an index
       into TA state.
     * every rule from chapter 20 is applied.

   Verification is done with OpenSSL on the host, to prove the
   signatures are real ECDSA and not something only this TA
   understands.
```

Project layout

```
$ mkdir -p keystore/{host,ta/include}
$ cd keystore
$ uuidgen
a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d
```

```
/* ta/include/keystore_ta.h - SHARED between both halves.
   This file is the interface contract.                     */

#ifndef KEYSTORE_TA_H
#define KEYSTORE_TA_H

#define KEYSTORE_TA_UUID \
        { 0xa1b2c3d4, 0xe5f6, 0x4a7b, \
          { 0x8c, 0x9d, 0x0e, 0x1f, 0x2a, 0x3b, 0x4c, 0x5d } }

/* Commands */
#define KS_CMD_CREATE       0   /* in: name        out: -      */
#define KS_CMD_GET_PUBKEY   1   /* in: name        out: x||y   */
#define KS_CMD_SIGN         2   /* in: name, digest out: sig   */
#define KS_CMD_VERIFY       3   /* in: name, digest, sig       */
#define KS_CMD_DELETE       4   /* in: name                    */
#define KS_CMD_LIST         5   /* out: NUL-separated names    */

/* Limits - the TA enforces these, the client respects them. */
#define KS_MAX_NAME_LEN     64
#define KS_DIGEST_LEN       32          /* SHA-256             */
#define KS_PUBKEY_LEN       64          /* P-256: 32 + 32      */
#define KS_MAX_SIG_LEN      64          /* P-256: r || s       */
#define KS_MAX_KEYS         16

#endif /* KEYSTORE_TA_H */
```

The trusted application

```
/* ta/user_ta_header_defines.h */

#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#include "keystore_ta.h"

#define TA_UUID         KEYSTORE_TA_UUID

/* No MULTI_SESSION: this TA has no mutable global state and we
   would rather serialise than reason about locking
   (chapter 13). KEEP_ALIVE so we do not pay the load cost on
   every first session.                                       */
#define TA_FLAGS        (TA_FLAG_SINGLE_INSTANCE | \
                         TA_FLAG_INSTANCE_KEEP_ALIVE)

#define TA_STACK_SIZE   (4 * 1024)
#define TA_DATA_SIZE    (32 * 1024)

#define TA_CURRENT_TA_EXT_PROPERTIES \
    { "gp.ta.description", USER_TA_PROP_TYPE_STRING, \
      "Secure key store" }, \
    { "gp.ta.version", USER_TA_PROP_TYPE_U32, \
      &(const uint32_t){ 0x0100 } }

#endif
```

```
/* ta/keystore_ta.c */

#include <tee_internal_api.h>
#include <tee_internal_api_extensions.h>
#include <string.h>
#include "keystore_ta.h"

/* ------------------------------------------------------------
 * Helpers
 * ------------------------------------------------------------ */

/* Copy a client-supplied name into TA memory and validate it.
 * ONE read of shared memory (chapter 20, rule 2).
 */
static TEE_Result copy_name(const TEE_Param *p, char *out,
                            size_t outsz, size_t *outlen)
{
        size_t len = p->memref.size;

        if (!p->memref.buffer || len == 0 || len >= outsz)
                return TEE_ERROR_BAD_PARAMETERS;

        if (TEE_CheckMemoryAccessRights(
                    TEE_MEMORY_ACCESS_READ |
                    TEE_MEMORY_ACCESS_ANY_OWNER,
                    p->memref.buffer, len) != TEE_SUCCESS)
                return TEE_ERROR_ACCESS_DENIED;

        TEE_MemMove(out, p->memref.buffer, len);
        out[len] = '\0';

        /* Validate the COPY. Only printable, no separators -
         * the name becomes an object ID and appears in the
         * NUL-separated list output.
         */
        for (size_t i = 0; i < len; i++) {
                char c = out[i];
                bool ok = (c >= 'a' && c <= 'z') ||
                          (c >= 'A' && c <= 'Z') ||
                          (c >= '0' && c <= '9') ||
                          c == '-' || c == '_' || c == '.';
                if (!ok)
                        return TEE_ERROR_BAD_PARAMETERS;
        }

        *outlen = len;
        return TEE_SUCCESS;
}

static TEE_Result open_key(const char *name, size_t namelen,
                           uint32_t flags, TEE_ObjectHandle *obj)
{
        return TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                        name, namelen,
                                        flags, obj);
}

/* ------------------------------------------------------------
 * KS_CMD_CREATE
 * ------------------------------------------------------------ */
static TEE_Result cmd_create(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
```

```
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);
        char             name[KS_MAX_NAME_LEN];
        size_t           namelen;
        TEE_ObjectHandle key  = TEE_HANDLE_NULL;
        TEE_ObjectHandle pobj = TEE_HANDLE_NULL;
        TEE_Attribute    attr;
        TEE_Result       res;

        if (ptypes != exp) {
                EMSG("ptypes 0x%08x expected 0x%08x", ptypes, exp);
                return TEE_ERROR_BAD_PARAMETERS;
        }

        res = copy_name(&params[0], name, sizeof name, &namelen);
        if (res != TEE_SUCCESS)
                return res;

        /* Refuse to overwrite silently - a client that wants a
         * new key must delete the old one deliberately.
         */
        res = open_key(name, namelen, TEE_DATA_FLAG_ACCESS_READ,
                       &pobj);
        if (res == TEE_SUCCESS) {
                TEE_CloseObject(pobj);
                return TEE_ERROR_ACCESS_CONFLICT;
        }
        pobj = TEE_HANDLE_NULL;

        res = TEE_AllocateTransientObject(TEE_TYPE_ECDSA_KEYPAIR,
                                          256, &key);
        if (res != TEE_SUCCESS)
                goto out;

        TEE_InitValueAttribute(&attr, TEE_ATTR_ECC_CURVE,
                               TEE_ECC_CURVE_NIST_P256, 0);

        /* The private key is generated HERE, inside the TEE,
         * and never exists anywhere else.
         */
        res = TEE_GenerateKey(key, 256, &attr, 1);
        if (res != TEE_SUCCESS)
                goto out;

        /* Persist the OBJECT, not bytes (chapter 15). */
        res = TEE_CreatePersistentObject(
                TEE_STORAGE_PRIVATE, name, namelen,
                TEE_DATA_FLAG_ACCESS_READ |
                TEE_DATA_FLAG_ACCESS_WRITE_META,
                key, NULL, 0, &pobj);
        if (res == TEE_SUCCESS)
                IMSG("created key '%s'", name);
out:
        TEE_FreeTransientObject(key);
        if (pobj != TEE_HANDLE_NULL)
                TEE_CloseObject(pobj);
        return res;
}

/* ------------------------------------------------------------
 * KS_CMD_GET_PUBKEY
```

```
 * ------------------------------------------------------------ */
static TEE_Result cmd_get_pubkey(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);
        char             name[KS_MAX_NAME_LEN];
        size_t           namelen;
        TEE_ObjectHandle pobj = TEE_HANDLE_NULL;
        uint8_t          pub[KS_PUBKEY_LEN];
        uint32_t         xlen = 32, ylen = 32;
        TEE_Result       res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        res = copy_name(&params[0], name, sizeof name, &namelen);
        if (res != TEE_SUCCESS)
                return res;

        res = open_key(name, namelen, TEE_DATA_FLAG_ACCESS_READ,
                       &pobj);
        if (res != TEE_SUCCESS)
                return res;      /* ITEM_NOT_FOUND if absent    */

        /* Both coordinates are 32 bytes for P-256, but
         * TEE_GetObjectBufferAttribute may return fewer if the
         * value has leading zeros. Zero-pad on the LEFT so the
         * output is always fixed width.
         */
        TEE_MemFill(pub, 0, sizeof pub);

        res = TEE_GetObjectBufferAttribute(pobj,
                        TEE_ATTR_ECC_PUBLIC_VALUE_X,
                        pub + (32 - xlen), &xlen);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_GetObjectBufferAttribute(pobj,
                        TEE_ATTR_ECC_PUBLIC_VALUE_Y,
                        pub + 32 + (32 - ylen), &ylen);
        if (res != TEE_SUCCESS)
                goto out;

        /* Short-buffer protocol (chapter 7). */
        if (params[1].memref.size < sizeof pub) {
                params[1].memref.size = sizeof pub;
                res = TEE_ERROR_SHORT_BUFFER;
                goto out;
        }

        /* Write output ONCE, on success (chapter 20, rule 5). */
        TEE_MemMove(params[1].memref.buffer, pub, sizeof pub);
        params[1].memref.size = sizeof pub;
out:
        TEE_CloseObject(pobj);
        return res;
}

/* ------------------------------------------------------------
 * KS_CMD_SIGN
```

```
 * ------------------------------------------------------------ */
static TEE_Result cmd_sign(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                TEE_PARAM_TYPE_NONE);
        char                name[KS_MAX_NAME_LEN];
        size_t              namelen;
        uint8_t             digest[KS_DIGEST_LEN];
        uint8_t             sig[KS_MAX_SIG_LEN];
        uint32_t            siglen = sizeof sig;
        TEE_ObjectHandle    pobj = TEE_HANDLE_NULL;
        TEE_OperationHandle op   = TEE_HANDLE_NULL;
        TEE_Result          res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        res = copy_name(&params[0], name, sizeof name, &namelen);
        if (res != TEE_SUCCESS)
                return res;

        /* Fixed-size digest. Exactly 32 bytes or nothing -
         * no length arithmetic to get wrong.
         */
        if (params[1].memref.size != KS_DIGEST_LEN)
                return TEE_ERROR_BAD_PARAMETERS;
        if (TEE_CheckMemoryAccessRights(
                    TEE_MEMORY_ACCESS_READ |
                    TEE_MEMORY_ACCESS_ANY_OWNER,
                    params[1].memref.buffer,
                    KS_DIGEST_LEN) != TEE_SUCCESS)
                return TEE_ERROR_ACCESS_DENIED;

        TEE_MemMove(digest, params[1].memref.buffer,
                    KS_DIGEST_LEN);

        res = open_key(name, namelen, TEE_DATA_FLAG_ACCESS_READ,
                       &pobj);
        if (res != TEE_SUCCESS)
                return res;

        res = TEE_AllocateOperation(&op, TEE_ALG_ECDSA_P256,
                                    TEE_MODE_SIGN, 256);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_SetOperationKey(op, pobj);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_AsymmetricSignDigest(op, NULL, 0,
                                       digest, KS_DIGEST_LEN,
                                       sig, &siglen);
        if (res != TEE_SUCCESS)
                goto out;

        if (params[2].memref.size < siglen) {
                params[2].memref.size = siglen;
                res = TEE_ERROR_SHORT_BUFFER;
                goto out;
```

```
        }

        TEE_MemMove(params[2].memref.buffer, sig, siglen);
        params[2].memref.size = siglen;
out:
        TEE_FreeOperation(op);
        TEE_CloseObject(pobj);
        TEE_MemFill(digest, 0, sizeof digest);
        return res;
}

/* ------------------------------------------------------------
 * KS_CMD_VERIFY
 * ------------------------------------------------------------ */
static TEE_Result cmd_verify(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_NONE);
        char                name[KS_MAX_NAME_LEN];
        size_t              namelen;
        uint8_t             digest[KS_DIGEST_LEN];
        uint8_t             sig[KS_MAX_SIG_LEN];
        size_t              siglen;
        TEE_ObjectHandle    pobj = TEE_HANDLE_NULL;
        TEE_OperationHandle op   = TEE_HANDLE_NULL;
        TEE_Result          res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        res = copy_name(&params[0], name, sizeof name, &namelen);
        if (res != TEE_SUCCESS)
                return res;

        if (params[1].memref.size != KS_DIGEST_LEN)
                return TEE_ERROR_BAD_PARAMETERS;

        siglen = params[2].memref.size;
        if (siglen == 0 || siglen > sizeof sig)
                return TEE_ERROR_BAD_PARAMETERS;

        TEE_MemMove(digest, params[1].memref.buffer, KS_DIGEST_LEN);
        TEE_MemMove(sig, params[2].memref.buffer, siglen);

        res = open_key(name, namelen, TEE_DATA_FLAG_ACCESS_READ,
                       &pobj);
        if (res != TEE_SUCCESS)
                return res;

        res = TEE_AllocateOperation(&op, TEE_ALG_ECDSA_P256,
                                    TEE_MODE_VERIFY, 256);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_SetOperationKey(op, pobj);
        if (res != TEE_SUCCESS)
                goto out;

        /* Returns TEE_ERROR_SIGNATURE_INVALID on mismatch -
         * pass it straight back; there is nothing secret about
```

```
         * "this signature is wrong".
         */
        res = TEE_AsymmetricVerifyDigest(op, NULL, 0,
                                         digest, KS_DIGEST_LEN,
                                         sig, siglen);
out:
        TEE_FreeOperation(op);
        TEE_CloseObject(pobj);
        return res;
}

/* ------------------------------------------------------------
 * KS_CMD_DELETE
 * ------------------------------------------------------------ */
static TEE_Result cmd_delete(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);
        char             name[KS_MAX_NAME_LEN];
        size_t           namelen;
        TEE_ObjectHandle pobj = TEE_HANDLE_NULL;
        TEE_Result       res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        res = copy_name(&params[0], name, sizeof name, &namelen);
        if (res != TEE_SUCCESS)
                return res;

        res = open_key(name, namelen,
                       TEE_DATA_FLAG_ACCESS_READ |
                       TEE_DATA_FLAG_ACCESS_WRITE_META, &pobj);
        if (res != TEE_SUCCESS)
                return res;

        /* Closes the handle as well - do not close it again. */
        return TEE_CloseAndDeletePersistentObject1(pobj);
}

/* ------------------------------------------------------------
 * KS_CMD_LIST
 * ------------------------------------------------------------ */
static TEE_Result cmd_list(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);
        TEE_ObjectEnumHandle e = TEE_HANDLE_NULL;
        uint8_t  out[KS_MAX_KEYS * (KS_MAX_NAME_LEN + 1)];
        size_t   used = 0;
        TEE_Result res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        res = TEE_AllocatePersistentObjectEnumerator(&e);
        if (res != TEE_SUCCESS)
```

```
                return res;

        res = TEE_StartPersistentObjectEnumerator(
                        e, TEE_STORAGE_PRIVATE);
        if (res == TEE_ERROR_ITEM_NOT_FOUND) {
                res = TEE_SUCCESS;      /* no keys yet - not an
                                           error                */
                goto emit;
        }
        if (res != TEE_SUCCESS)
                goto out;

        for (;;) {
                TEE_ObjectInfo info;
                uint8_t        id[KS_MAX_NAME_LEN];
                uint32_t       idlen = sizeof id;

                res = TEE_GetNextPersistentObject(e, &info,
                                                  id, &idlen);
                if (res == TEE_ERROR_ITEM_NOT_FOUND) {
                        res = TEE_SUCCESS;
                        break;
                }
                if (res != TEE_SUCCESS)
                        goto out;

                if (used + idlen + 1 > sizeof out) {
                        res = TEE_ERROR_SHORT_BUFFER;
                        goto out;
                }
                TEE_MemMove(out + used, id, idlen);
                used += idlen;
                out[used++] = '\0';
        }

emit:
        if (params[0].memref.size < used) {
                params[0].memref.size = used;
                res = TEE_ERROR_SHORT_BUFFER;
                goto out;
        }
        TEE_MemMove(params[0].memref.buffer, out, used);
        params[0].memref.size = used;
out:
        if (e != TEE_HANDLE_NULL)
                TEE_FreePersistentObjectEnumerator(e);
        return res;
}

/* ------------------------------------------------------------
 * Entry points
 * ------------------------------------------------------------ */

TEE_Result TA_CreateEntryPoint(void)
{
        IMSG("keystore TA loaded");
        return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
        IMSG("keystore TA unloaded");
}
```

```
TEE_Result TA_OpenSessionEntryPoint(uint32_t ptypes,
                                    TEE_Param params[4] __unused,
                                    void **sess_ctx)
{
        TEE_Identity id;
        TEE_Result   res;

        if (ptypes != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE))
                return TEE_ERROR_BAD_PARAMETERS;

        /* Refuse anonymous callers. With LOGIN_USER, OP-TEE
         * scopes secure storage per uid, so each user gets a
         * separate key namespace (chapter 11).
         */
        res = TEE_GetPropertyAsIdentity(TEE_PROPSET_CURRENT_CLIENT,
                                        "gpd.client.identity", &id);
        if (res != TEE_SUCCESS)
                return res;

        if (id.login != TEE_LOGIN_USER &&
            id.login != TEE_LOGIN_REE_KERNEL) {
                EMSG("refusing login type 0x%08x", id.login);
                return TEE_ERROR_ACCESS_DENIED;
        }

        *sess_ctx = NULL;       /* no per-session state needed  */
        return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess_ctx __unused)
{
}

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx __unused,
                                      uint32_t cmd,
                                      uint32_t ptypes,
                                      TEE_Param params[4])
{
        switch (cmd) {
        case KS_CMD_CREATE:     return cmd_create(ptypes, params);
        case KS_CMD_GET_PUBKEY: return cmd_get_pubkey(ptypes, params);
        case KS_CMD_SIGN:       return cmd_sign(ptypes, params);
        case KS_CMD_VERIFY:     return cmd_verify(ptypes, params);
        case KS_CMD_DELETE:     return cmd_delete(ptypes, params);
        case KS_CMD_LIST:       return cmd_list(ptypes, params);
        default:                return TEE_ERROR_NOT_SUPPORTED;
        }
}
```

The client

```
/* host/main.c */

#include <err.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <openssl/sha.h>
#include <tee_client_api.h>
#include "keystore_ta.h"

static TEEC_Context ctx;
static TEEC_Session sess;

static const char *origin_str(uint32_t o)
{
        switch (o) {
        case TEEC_ORIGIN_API:         return "API";
        case TEEC_ORIGIN_COMMS:       return "COMMS";
        case TEEC_ORIGIN_TEE:         return "TEE";
        case TEEC_ORIGIN_TRUSTED_APP: return "TA";
        default:                      return "?";
        }
}

static void ks_open(void)
{
        TEEC_UUID uuid = KEYSTORE_TA_UUID;
        uint32_t  origin;
        TEEC_Result res;

        res = TEEC_InitializeContext(NULL, &ctx);
        if (res != TEEC_SUCCESS)
                errx(1, "InitializeContext: 0x%08x", res);

        /* LOGIN_USER: the kernel supplies our uid; the TA uses
         * it to scope storage. We cannot forge it.
         */
        res = TEEC_OpenSession(&ctx, &sess, &uuid,
                               TEEC_LOGIN_USER, NULL, NULL,
                               &origin);
        if (res != TEEC_SUCCESS)
                errx(1, "OpenSession: 0x%08x origin %s",
                     res, origin_str(origin));
}

static void ks_close(void)
{
        TEEC_CloseSession(&sess);
        TEEC_FinalizeContext(&ctx);
}

static TEEC_Result invoke(uint32_t cmd, TEEC_Operation *op)
{
        uint32_t origin;
        TEEC_Result res = TEEC_InvokeCommand(&sess, cmd, op,
                                             &origin);
        if (res != TEEC_SUCCESS && res != TEEC_ERROR_SHORT_BUFFER)
                warnx("cmd %u: 0x%08x origin %s", cmd, res,
                      origin_str(origin));
        return res;
```

```
}

/* ---- SHA-256 of a file, computed in the NORMAL world.
 *      The TA signs a digest, not a file: hashing is not
 *      security-sensitive here and keeping bulk data out of
 *      the TEE is the whole point (chapter 1).            */
static int sha256_file(const char *path, uint8_t out[32])
{
        SHA256_CTX c;
        uint8_t    buf[65536];
        size_t     n;
        FILE      *f = fopen(path, "rb");

        if (!f) { warn("%s", path); return -1; }
        SHA256_Init(&c);
        while ((n = fread(buf, 1, sizeof buf, f)) > 0)
                SHA256_Update(&c, buf, n);
        fclose(f);
        SHA256_Final(out, &c);
        return 0;
}

static void hexdump(const uint8_t *p, size_t n)
{
        for (size_t i = 0; i < n; i++)
                printf("%02x", p[i]);
        printf("\n");
}

/* ---- commands ---------------------------------------- */

static int do_create(const char *name)
{
        TEEC_Operation op;

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_NONE, TEEC_NONE,
                                         TEEC_NONE);
        op.params[0].tmpref.buffer = (void *)name;
        op.params[0].tmpref.size   = strlen(name);

        if (invoke(KS_CMD_CREATE, &op) != TEEC_SUCCESS)
                return 1;
        printf("created '%s'\n", name);
        return 0;
}

static int do_pubkey(const char *name)
{
        TEEC_Operation op;
        uint8_t pub[KS_PUBKEY_LEN];

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_OUTPUT,
                                         TEEC_NONE, TEEC_NONE);
        op.params[0].tmpref.buffer = (void *)name;
        op.params[0].tmpref.size   = strlen(name);
        op.params[1].tmpref.buffer = pub;
        op.params[1].tmpref.size   = sizeof pub;

        if (invoke(KS_CMD_GET_PUBKEY, &op) != TEEC_SUCCESS)
```

```
                return 1;

        /* Uncompressed SEC1 point, so OpenSSL can read it. */
        printf("04");
        hexdump(pub, op.params[1].tmpref.size);
        return 0;
}

static int do_sign(const char *name, const char *path)
{
        TEEC_Operation op;
        uint8_t digest[KS_DIGEST_LEN], sig[KS_MAX_SIG_LEN];

        if (sha256_file(path, digest))
                return 1;

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_OUTPUT,
                                         TEEC_NONE);
        op.params[0].tmpref.buffer = (void *)name;
        op.params[0].tmpref.size   = strlen(name);
        op.params[1].tmpref.buffer = digest;
        op.params[1].tmpref.size   = sizeof digest;
        op.params[2].tmpref.buffer = sig;
        op.params[2].tmpref.size   = sizeof sig;

        if (invoke(KS_CMD_SIGN, &op) != TEEC_SUCCESS)
                return 1;

        hexdump(sig, op.params[2].tmpref.size);
        return 0;
}

static int hex2bin(const char *hex, uint8_t *out, size_t outsz,
                   size_t *outlen)
{
        size_t n = strlen(hex);

        if (n % 2 || n / 2 > outsz)
                return -1;
        for (size_t i = 0; i < n / 2; i++)
                if (sscanf(hex + 2 * i, "%2hhx", &out[i]) != 1)
                        return -1;
        *outlen = n / 2;
        return 0;
}

static int do_verify(const char *name, const char *path,
                     const char *sighex)
{
        TEEC_Operation op;
        uint8_t digest[KS_DIGEST_LEN], sig[KS_MAX_SIG_LEN];
        size_t  siglen;
        TEEC_Result res;

        if (sha256_file(path, digest))
                return 1;
        if (hex2bin(sighex, sig, sizeof sig, &siglen)) {
                warnx("bad signature hex");
                return 1;
        }
```

```
        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_NONE);
        op.params[0].tmpref.buffer = (void *)name;
        op.params[0].tmpref.size   = strlen(name);
        op.params[1].tmpref.buffer = digest;
        op.params[1].tmpref.size   = sizeof digest;
        op.params[2].tmpref.buffer = sig;
        op.params[2].tmpref.size   = siglen;

        res = invoke(KS_CMD_VERIFY, &op);
        printf("%s\n", res == TEEC_SUCCESS ? "VALID" : "INVALID");
        return res == TEEC_SUCCESS ? 0 : 1;
}

static int do_list(void)
{
        TEEC_Operation op;
        char buf[KS_MAX_KEYS * (KS_MAX_NAME_LEN + 1)];
        size_t off = 0, n;

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_OUTPUT,
                                         TEEC_NONE, TEEC_NONE,
                                         TEEC_NONE);
        op.params[0].tmpref.buffer = buf;
        op.params[0].tmpref.size   = sizeof buf;

        if (invoke(KS_CMD_LIST, &op) != TEEC_SUCCESS)
                return 1;

        n = op.params[0].tmpref.size;
        while (off < n) {
                printf("%s\n", buf + off);
                off += strlen(buf + off) + 1;
        }
        return 0;
}

static int do_delete(const char *name)
{
        TEEC_Operation op;

        memset(&op, 0, sizeof op);
        op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                         TEEC_NONE, TEEC_NONE,
                                         TEEC_NONE);
        op.params[0].tmpref.buffer = (void *)name;
        op.params[0].tmpref.size   = strlen(name);

        if (invoke(KS_CMD_DELETE, &op) != TEEC_SUCCESS)
                return 1;
        printf("deleted '%s'\n", name);
        return 0;
}

int main(int argc, char **argv)
{
        int rc;
```

```
        if (argc < 2) {
                fprintf(stderr,
                        "usage: ks create|pubkey|delete <name>\n"
                        "       ks sign   <name> <file>\n"
                        "       ks verify <name> <file> <sighex>\n"
                        "       ks list\n");
                return 2;
        }

        ks_open();

        if      (!strcmp(argv[1], "create") && argc == 3)
                rc = do_create(argv[2]);
        else if (!strcmp(argv[1], "pubkey") && argc == 3)
                rc = do_pubkey(argv[2]);
        else if (!strcmp(argv[1], "delete") && argc == 3)
                rc = do_delete(argv[2]);
        else if (!strcmp(argv[1], "sign")   && argc == 4)
                rc = do_sign(argv[2], argv[3]);
        else if (!strcmp(argv[1], "verify") && argc == 5)
                rc = do_verify(argv[2], argv[3], argv[4]);
        else if (!strcmp(argv[1], "list")   && argc == 2)
                rc = do_list();
        else {
                warnx("bad usage");
                rc = 2;
        }

        ks_close();
        return rc;
}
```

Building and installing

```
# ta/Makefile
CFG_TEE_TA_LOG_LEVEL ?= 3
CPPFLAGS += -DCFG_TEE_TA_LOG_LEVEL=$(CFG_TEE_TA_LOG_LEVEL)
BINARY = a1b2c3d4-e5f6-4a7b-8c9d-0e1f2a3b4c5d
-include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

# ta/sub.mk
global-incdirs-y += include
srcs-y += keystore_ta.c

# host/Makefile
CC ?= $(CROSS_COMPILE)gcc
CFLAGS += -Wall -Wextra -I../ta/include -I$(TEEC_EXPORT)/include
LDADD  += -lteec -lcrypto -L$(TEEC_EXPORT)/lib
BINARY = ks
all: $(BINARY)
$(BINARY): main.o
        $(CC) -o $@ $< $(LDADD)
clean:
        rm -f main.o $(BINARY)
```

```
$ export TA_DEV_KIT_DIR=~/optee/optee_os/out/arm-plat-vexpress/export-ta_arm64
$ export TEEC_EXPORT=~/optee/optee_client/out/export/usr
$ export CROSS_COMPILE=aarch64-linux-gnu-

$ make -C ta   CROSS_COMPILE=$CROSS_COMPILE
$ make -C host CROSS_COMPILE=$CROSS_COMPILE

# openssl for aarch64 is needed for the client's SHA-256. If
# your rootfs lacks it, either build it, or replace sha256_file
# with a small public-domain SHA-256 - the TA is unaffected.

$ cp ta/*.ta  ~/optee/out-br/target/lib/optee_armtz/
$ cp host/ks  ~/optee/out-br/target/usr/bin/
$ cd ~/optee/build && make buildroot run-only
```

Running it

```
# In the guest:
# ks create device-key
created 'device-key'

# Secure console shows:
#   I/TA: keystore TA loaded
#   I/TA: cmd_create:118 created key 'device-key'

# ks list
device-key

# ks pubkey device-key
04a1e5...c2  (130 hex chars: 04 || X || Y)

# echo "hello world" > /tmp/msg
# ks sign device-key /tmp/msg
3f8c1a...9e   (128 hex chars: r || s)

# ks verify device-key /tmp/msg 3f8c1a...9e
VALID

# Tamper with the file:
# echo "hello world!" > /tmp/msg
# ks verify device-key /tmp/msg 3f8c1a...9e
INVALID

# Survives a reboot - the key is in secure storage:
# reboot
# ks verify device-key /tmp/msg2 <sig>
VALID
```

Proving the signature is real

```
# The most satisfying part: verify with OpenSSL, on the host,
# with no OP-TEE involved at all. If this works, the TA is
# producing genuine ECDSA P-256 signatures.

# 1. Get the public key and turn it into a PEM.
$ PUB=$(ssh guest ks pubkey device-key)
$ echo $PUB | xxd -r -p > pub.raw          # 65 bytes: 04||X||Y

# Wrap it in the SubjectPublicKeyInfo DER prefix for
# prime256v1:
$ printf '3059301306072a8648ce3d020106082a8648ce3d030107034200' \
    | xxd -r -p > spki.der
$ cat spki.der pub.raw > pubkey.der
$ openssl ec -pubin -inform DER -in pubkey.der -out pubkey.pem
read EC key
writing EC key

$ openssl ec -pubin -in pubkey.pem -text -noout
Public-Key: (256 bit)
pub: 04:a1:e5:...
ASN1 OID: prime256v1
NIST CURVE: P-256

# 2. The TA emits raw r||s; OpenSSL wants DER. Convert.
$ SIG=$(ssh guest ks sign device-key /tmp/msg)
$ python3 - "$SIG" <<'EOF' > sig.der
import sys, binascii
raw = binascii.unhexlify(sys.argv[1])
r, s = raw[:32], raw[32:]
def der_int(b):
    b = b.lstrip(b'\x00') or b'\x00'
    if b[0] & 0x80:
        b = b'\x00' + b
    return b'\x02' + bytes([len(b)]) + b
body = der_int(r) + der_int(s)
sys.stdout.buffer.write(b'\x30' + bytes([len(body)]) + body)
EOF

# 3. Verify.
$ printf 'hello world\n' > msg
$ openssl dgst -sha256 -verify pubkey.pem -signature sig.der msg
Verified OK

# THAT is the point of the project. A key that was generated
# inside the secure world, that has never existed in normal-
# world memory, that survives reboots, and whose signatures any
# standard tool can verify.
```

> **RAW R||S VERSUS DER**
>
> GlobalPlatform's `TEE_AsymmetricSignDigest` returns the two ECDSA components concatenated and fixed-width. Almost every other ecosystem — OpenSSL, X.509, JWS with ES256 uses raw, but TLS and CMS use DER — expects an ASN.1 SEQUENCE of two INTEGERs. Converting is trivial and forgetting to is the commonest reason a TA's signatures "do not verify" when they are perfectly correct.

Extensions worth doing

Exercise	What it teaches
Return DER-encoded signatures from the TA	ASN.1 encoding inside a TA, and why you keep the encoder small
Add an attempt counter that locks a key after N failed verifies, stored in RPMB	Rollback protection (chapter 17)
Add `KS_CMD_WRAP`: export a key encrypted under another key	Key wrapping, and the policy questions it raises
Add RSA-2048 as an alternative key type	Handling more than one algorithm behind one interface
Make the TA `MULTI_SESSION` and stress it with concurrent clients	Everything in chapter 13, the hard way
Write the hostile client from chapter 20 against this TA	Whether your validation is real
Put a kernel driver in the middle — see chapter 25	Why the kernel would want a TA at all, and the ioctl ABI that follows
Bind a key to the device with `TEE_GetHUKDerivedKey`	Device binding (chapter 16)
Produce a self-signed X.509 certificate for the key	How device attestation actually ships

How it shows up

Symptom	Cause
`OpenSession: 0xFFFF0008`	`.ta` not installed, filename does not match the UUID, or `tee-supplicant` not running
`OpenSession: 0xFFFF0001` origin TA	The TA refused the login type — the client used `LOGIN_PUBLIC`
`cmd 0: 0xFFFF0006` origin TA	`paramTypes` mismatch; the TA's `EMSG` prints both values
`0xFFFF0003` on create	Key already exists — delete it first, by design
Signatures verify in the TA, fail in OpenSSL	Raw r‖s not converted to DER
Public key rejected by OpenSSL	Missing the `04` uncompressed-point prefix, or coordinates not left-padded to 32 bytes
Keys disappear after reflashing the image	Secure storage lives in `/data/tee`, which the rootfs image replaced
Two users see the same keys	Client used `LOGIN_PUBLIC`; storage is not scoped
Works on QEMU, keys unreadable on a second board	Correct — the HUK differs per device (chapter 15)

Check yourself

- ✓ Explain why the client sends a digest rather than the file.
- ✓ Say where the private key exists, from generation to use, and why it cannot be exported.
- ✓ Point to the `paramTypes` check, the single-fetch copy and the short-buffer protocol in the TA source.
- ✓ Explain why `copy_name` validates the copy rather than the shared buffer.
- ✓ Explain what the login-type check in `TA_OpenSessionEntryPoint` achieves.
- ✓ Explain why `TA_FLAG_MULTI_SESSION` was deliberately not set.
- ✓ Convert a raw r‖s signature to DER and verify it with OpenSSL.
- ✓ Say what happens to the keys if the normal world deletes `/data/tee`, and how you would defend against it.

- ✓ Extend the TA with one command and add the matching client subcommand.
- ✓ Run the hostile client from chapter 20 against this TA and fix anything it finds.

Project: a firmware TPM as a trusted application

A TPM is a small state machine with keys, counters and measurement registers, spoken to over a byte-stream protocol. That description does not require a chip — it requires isolation, and the secure world provides it. Microsoft's fTPM does exactly this, ships on real products, and is the most instructive TA in existence.

Why a TPM in software is a reasonable idea

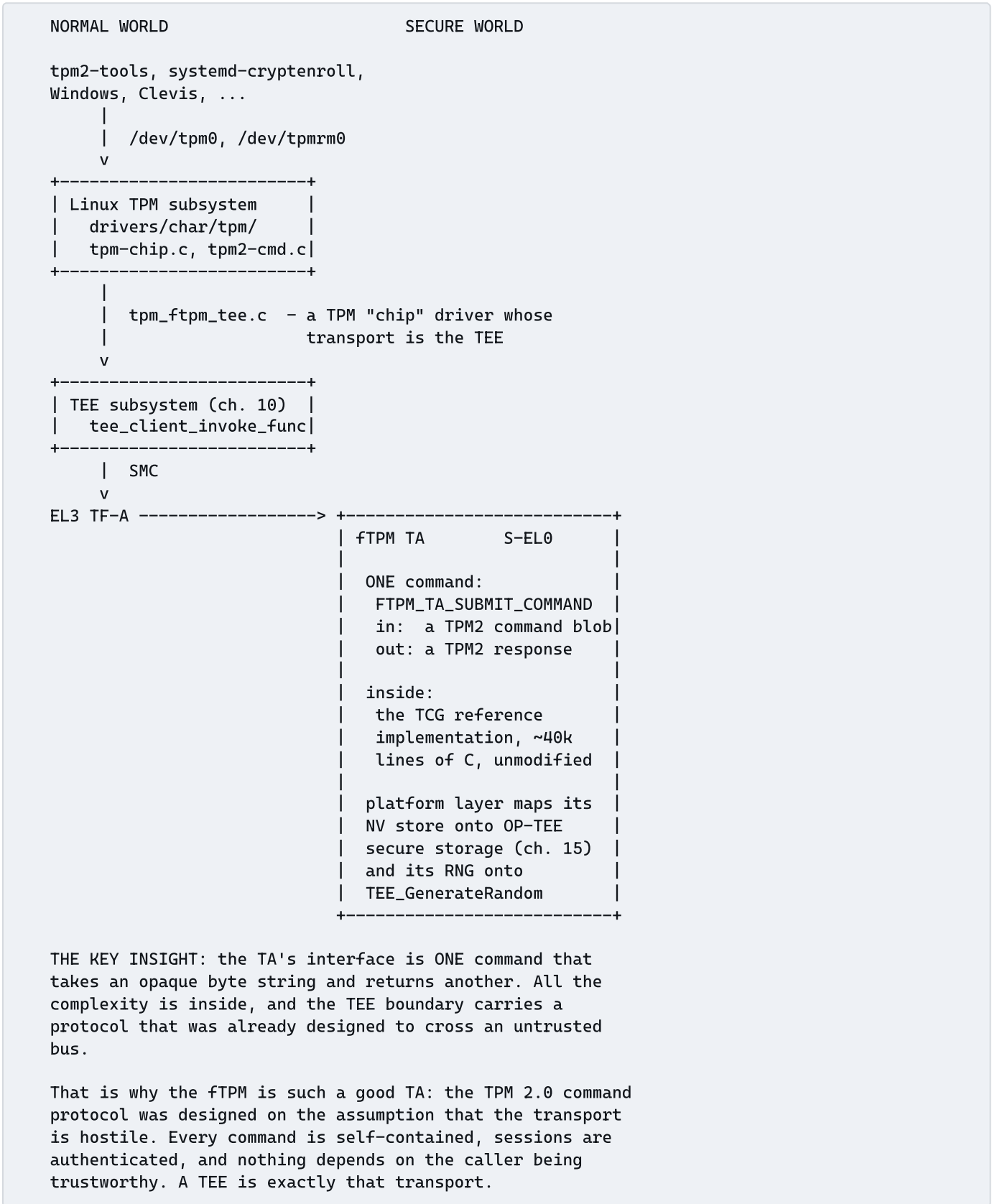
	Discrete TPM (dTPM)	Firmware TPM (fTPM)
Where it runs	A separate chip on the SPI or LPC bus	A trusted application in the secure world
Bill of materials	A part, a footprint, routing	Nothing
Bus interception	**Possible** — the SPI bus is on the board and can be probed	Not applicable; there is no bus
Physical attack on the part	Hard — the chip is hardened, often certified	As hard as attacking the SoC, which is usually less hard
Shares fate with the CPU	No	**Yes** — an SoC compromise is a TPM compromise
Certification	Common Criteria, FIPS	Possible but harder
Updatable	Rarely	Yes, with the firmware
Performance	Slow bus, slow part	Fast — it is the application processor

The honest summary: an fTPM removes bus-probing attacks and costs nothing, at the price of sharing its fate with the SoC. For a phone or a laptop where the SoC is already the root of trust, that trade is clearly right — which is why Windows on ARM, most Android devices and AMD's fTPM all take it. For a system where the threat model includes an attacker with unlimited physical access and the CPU is not hardened, a discrete part still wins.

What a TPM actually is

```
Strip away the specification and a TPM 2.0 is four things:

1. PCRs - Platform Configuration Registers
     24 registers, each holding a hash. You cannot WRITE
     them; you can only EXTEND them:

         PCR[i] = SHA256( PCR[i] || new_measurement )

     One-way and order-dependent. Reset to zero only on
     reboot. So the value of PCR[4] after boot is a
     fingerprint of exactly which bootloader ran, in which
     order - and no software can fake a value it did not
     actually reach.

2. A KEY HIERARCHY
     Primary keys derived from persistent SEEDS:
       Endorsement Seed  -> the device's identity, set at
                            manufacture, never changes
       Storage Seed      -> the owner's key hierarchy
       Platform Seed     -> firmware's own hierarchy
     Child keys are created under a parent and stored
     OUTSIDE the TPM as encrypted blobs, loaded back on
     demand. That is how a TPM with 64 KB of storage
     manages thousands of keys.

3. NV STORAGE
     A few kilobytes of indexed, attribute-controlled
     space. Certificates, counters, policy data.
     Monotonic counters live here.

4. POLICY
     A session-based language for "this key may be used
     only if...". Conditions include PCR values, a
     password, a signature, a time, or a counter.
     This is what makes SEALING work.

SEALING, the flagship use, in one line:

  encrypt a secret so that it can only be decrypted when
  PCR[0..7] have exactly the values they had at seal time

which means: "decrypt only if the machine booted the same
software it did when I sealed this". That is BitLocker,
and it is the reason PCRs exist.
```

The fTPM stack



THE KEY INSIGHT: the TA's interface is ONE command that takes an opaque byte string and returns another. All the complexity is inside, and the TEE boundary carries a protocol that was already designed to cross an untrusted bus.

That is why the fTPM is such a good TA: the TPM 2.0 command protocol was designed on the assumption that the transport is hostile. Every command is self-contained, sessions are authenticated, and nothing depends on the caller being trustworthy. A TEE is exactly that transport.

THE DESIGN LESSON WORTH TAKING AWAY

A good TA interface is *narrow and self-authenticating*. The fTPM has one command and the payload carries its own authorisation. Compare a TA with forty commands, each with pointer parameters and implicit state — every one is an attack surface needing its own validation (chapter 20). If you can express your service as a protocol rather than an API, do it.

Getting the source

```
$ git clone https://github.com/microsoft/ms-tpm-20-ref.git
$ cd ms-tpm-20-ref

# Two things live here:
#   TPMCmd/           the TCG reference implementation. This
#                     is the actual TPM 2.0 specification,
#                     rendered as C, maintained by Microsoft.
#   Samples/ARM32-FirmwareTPM/optee_ta/
#                     the OP-TEE glue: entry points, the
#                     platform layer, the build.

$ ls Samples/ARM32-FirmwareTPM/optee_ta/
fTPM/          the TA itself
Makefile
README.md

$ ls Samples/ARM32-FirmwareTPM/optee_ta/fTPM/
Android.mk  Makefile  sub.mk
include/    reference/    platform/
user_ta_header_defines.h

# The UUID, which you will meet again in chapter 23:
$ grep -r "TA_FTPM_UUID" --include=*.h | head -2
#define TA_FTPM_UUID { 0xbc50d971, 0xd4c9, 0x42c4, \
        { 0x82, 0xcb, 0x34, 0x3f, 0xb7, 0xf3, 0x78, 0x96 } }
```

The TA interface, in full

```
/* fTPM/ftpm_ta.c - the entire command surface.             */

#define TA_FTPM_SUBMIT_COMMAND   (0)
#define TA_FTPM_EMULATE_PPI      (1)   /* physical presence   */

TEE_Result TA_InvokeCommandEntryPoint(void *sess_ctx __unused,
                                      uint32_t cmd_id,
                                      uint32_t param_types,
                                      TEE_Param params[4])
{
        switch (cmd_id) {
        case TA_FTPM_SUBMIT_COMMAND:
                return ftpm_submit_command(param_types, params);
        case TA_FTPM_EMULATE_PPI:
                return ftpm_emulate_ppi(param_types, params);
        default:
                return TEE_ERROR_BAD_PARAMETERS;
        }
}

static TEE_Result ftpm_submit_command(uint32_t param_types,
                                      TEE_Param params[4])
{
        uint32_t exp_param_types =
                TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                TEE_PARAM_TYPE_MEMREF_INOUT,
                                TEE_PARAM_TYPE_NONE,
                                TEE_PARAM_TYPE_NONE);
        uint8_t  *cmd_buf;
        uint32_t  cmd_len;
        uint8_t  *resp_buf;
        uint32_t  resp_len;

        if (param_types != exp_param_types)
                return TEE_ERROR_BAD_PARAMETERS;

        cmd_len  = params[0].memref.size;
        cmd_buf  = params[0].memref.buffer;
        resp_len = params[1].memref.size;
        resp_buf = params[1].memref.buffer;

        if (cmd_len > MAX_COMMAND_SIZE ||
            resp_len < MAX_RESPONSE_SIZE)
                return TEE_ERROR_BAD_PARAMETERS;

        /* Copy the command into TA memory - one fetch
           (chapter 20). The reference implementation then
           works entirely on its own buffers.               */
        memcpy(pCommandBuffer, cmd_buf, cmd_len);

        /* THE WHOLE TPM. One call into the reference code. */
        _plat__RunCommand(cmd_len, pCommandBuffer,
                          &respLen, &respBuf);

        if (respLen > resp_len)
                return TEE_ERROR_SHORT_BUFFER;

        memcpy(resp_buf, respBuf, respLen);
        params[1].memref.size = respLen;
        return TEE_SUCCESS;
```

```
}

/* That is it. One entry point, one memcpy in, one call, one
   memcpy out. Forty thousand lines of TPM behind a twenty-line
   function.                                                  */
```

The platform layer: where the porting work is

```
/* The reference implementation calls out to a platform layer
   for everything the specification leaves to the
   implementer. Porting a TPM to a new environment IS writing
   this layer, and OP-TEE makes it short.                   */

/* --- NV STORAGE: the TPM's persistent state ------------- */
/* platform/NVMem.c                                         */

int _plat__NvCommit(void)
{
        /* The TPM's entire NV image is one blob, written back
           to OP-TEE secure storage (chapter 15).           */
        return NvFileCommit();   /* -> TEE_WriteObjectData   */
}

void _plat__NvMemoryRead(unsigned int startOffset,
                         unsigned int size, void *data)
{
        memcpy(data, &s_NV[startOffset], size);
}

/* The whole NV area is held in RAM and flushed on commit.
   Consequences:
     - a power cut between a state change and the commit loses
       that change. The TPM specification anticipates this and
       defines which operations must be committed
       synchronously.
     - the NV area is a few tens of kilobytes, so this is
       affordable.
     - REE FS gives no rollback protection. An fTPM that
       matters should be built with CFG_RPMB_FS=y, or its NV
       counters can be rewound (chapter 17). THIS IS THE
       SINGLE MOST IMPORTANT DEPLOYMENT DECISION HERE.     */

/* --- ENTROPY -------------------------------------------- */
int32_t _plat__GetEntropy(unsigned char *entropy, uint32_t amount)
{
        TEE_GenerateRandom(entropy, amount);
        return (int32_t)amount;
        /* Backed by the platform's TRNG where one exists.
           On QEMU it is not a real TRNG, which is one more
           reason the QEMU build is for development only. */
}

/* --- TIME ----------------------------------------------- */
uint64_t _plat__TimerRead(void)
{
        TEE_Time t;
        TEE_GetSystemTime(&t);      /* MONOTONIC, secure -
                                       not TEE_GetREETime.
                                       The TPM's clock must
                                       not be attacker-
                                       controlled (ch. 17). */
        return (uint64_t)t.seconds * 1000 + t.millis;
}

/* --- UNIQUE VALUES: where the seeds come from ----------- */
/* The Endorsement Primary Seed is the device's identity. It
   must be:
     - different on every device
```

```
     - stable for the life of the device
     - never derivable from outside the secure world

   Options, best first:
     1. TEE_GetHUKDerivedKey with a fixed label - derived from
        fuses, per-device, needs no storage (chapter 16).
     2. generated once with TEE_GenerateRandom and persisted
        to RPMB.
     3. provisioned at manufacture from an HSM, along with an
        EK certificate signed by your CA.

   Option 3 is what a product does, because an Endorsement Key
   is only useful for attestation if a relying party can chain
   it to a manufacturer they trust.

   The QEMU/development build uses a FIXED seed. Every
   "device" then has the same EK, which is fine for testing
   and completely useless for attestation.               */

/* --- PHYSICAL PRESENCE ---------------------------------- */
/* On a PC this is a person pressing a key during POST. In a
   TA there is nobody to ask, so the platform layer decides -
   usually by asserting presence for operations the platform
   owner is allowed to perform, and refusing otherwise.
   Getting this wrong lets the normal world clear the TPM.  */
```

Building it

```
$ cd ms-tpm-20-ref/Samples/ARM32-FirmwareTPM/optee_ta

$ export TA_DEV_KIT_DIR=~/optee/optee_os/out/arm-plat-vexpress/export-ta_arm64
$ export CROSS_COMPILE=aarch64-linux-gnu-
$ export TA_CPU=cortex-a53

$ make CFG_FTPM_USE_WOLF=y \
       CFG_TA_DEBUG=y \
       CFG_TEE_TA_LOG_LEVEL=3

#   CFG_FTPM_USE_WOLF=y   use wolfSSL for crypto. The
#                         alternative is OpenSSL, which is
#                         larger. Neither uses OP-TEE's own
#                         crypto API - the reference
#                         implementation brings its own, and
#                         that is a deliberate choice by the
#                         TCG so the TPM's crypto is
#                         auditable independently.

$ ls out/
bc50d971-d4c9-42c4-82cb-343fb7f37896.ta        # ~400 KB
bc50d971-d4c9-42c4-82cb-343fb7f37896.elf       # keep this for
                                               # panics (ch. 19)

# It is large. Check it fits:
#   TA_DATA_SIZE  must accommodate the NV image plus wolfSSL's
#                 working set
#   TA_RAM        must accommodate the whole mapped TA
# In fTPM/user_ta_header_defines.h:
#   #define TA_DATA_SIZE   (2 * 1024 * 1024)
# which is why TZDRAM_SIZE (chapter 6) needs to be generous.

$ cp out/*.ta ~/optee/out-br/target/lib/optee_armtz/
$ cd ~/optee/build && make buildroot run-only
```

> **SIZE IS THE FIRST THING THAT WILL BITE YOU**
> The fTPM is by far the largest TA most people will deploy: a 400 KB binary with a 2 MB heap. On QEMU with 48 MB of secure DRAM that is comfortable. On a real SoC with 4 MB of secure SRAM and a pager (chapter 5) it is a genuine engineering problem, and it is why the paged build exists. Budget the memory before you promise the feature.

A minimal TPM, if you want to understand rather than integrate

```
/* Building the real fTPM teaches you integration. Writing a
   tiny one teaches you what a TPM IS. This implements four
   commands in about 200 lines, and it is a genuinely useful
   exercise before chapter 23.                              */

#define TPM_ST_NO_SESSIONS   0x8001
#define TPM_RC_SUCCESS       0x0000
#define TPM_RC_COMMAND_CODE  0x0143
#define TPM_CC_PCR_Extend    0x00000182
#define TPM_CC_PCR_Read      0x0000017E
#define TPM_CC_GetRandom     0x0000017B
#define TPM_CC_Startup       0x00000144

#define NUM_PCR   24
#define PCR_LEN   32

static uint8_t g_pcr[NUM_PCR][PCR_LEN];   /* zeroed at reset */

/* Big-endian helpers - TPM structures are all big-endian. */
static uint32_t be32(const uint8_t *p)
{
        return ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) |
               ((uint32_t)p[2] << 8)  |  (uint32_t)p[3];
}
static void put_be32(uint8_t *p, uint32_t v)
{
        p[0] = v >> 24; p[1] = v >> 16; p[2] = v >> 8; p[3] = v;
}
static void put_be16(uint8_t *p, uint16_t v)
{
        p[0] = v >> 8; p[1] = v;
}

/* PCR_Extend: PCR[i] = SHA256(PCR[i] || digest)           */
static TEE_Result pcr_extend(uint32_t idx, const uint8_t *digest)
{
        TEE_OperationHandle op = TEE_HANDLE_NULL;
        uint8_t  out[PCR_LEN];
        uint32_t outlen = sizeof out;
        TEE_Result res;

        if (idx >= NUM_PCR)
                return TEE_ERROR_BAD_PARAMETERS;

        res = TEE_AllocateOperation(&op, TEE_ALG_SHA256,
                                    TEE_MODE_DIGEST, 0);
        if (res != TEE_SUCCESS)
                return res;

        TEE_DigestUpdate(op, g_pcr[idx], PCR_LEN);
        res = TEE_DigestDoFinal(op, digest, PCR_LEN,
                                out, &outlen);
        if (res == TEE_SUCCESS)
                TEE_MemMove(g_pcr[idx], out, PCR_LEN);

        TEE_FreeOperation(op);
        return res;
}

/* The command dispatcher. Every TPM command has this header:
```

```
     uint16 tag | uint32 commandSize | uint32 commandCode
   and every response has:
     uint16 tag | uint32 responseSize | uint32 responseCode  */

static uint32_t tpm_dispatch(const uint8_t *cmd, uint32_t cmdlen,
                             uint8_t *rsp, uint32_t *rsplen)
{
        uint32_t cc, size;
        uint32_t rc = TPM_RC_SUCCESS;
        uint32_t body = 0;      /* bytes of response payload  */

        if (cmdlen < 10)
                return TPM_RC_COMMAND_CODE;

        size = be32(cmd + 2);
        cc   = be32(cmd + 6);

        /* Never trust the caller's length. */
        if (size != cmdlen)
                return TPM_RC_COMMAND_CODE;

        switch (cc) {
        case TPM_CC_Startup:
                break;                          /* no payload  */

        case TPM_CC_GetRandom: {
                uint16_t want;
                if (cmdlen < 12) { rc = TPM_RC_COMMAND_CODE; break; }
                want = (cmd[10] << 8) | cmd[11];
                if (want > 64) want = 64;
                put_be16(rsp + 10, want);
                TEE_GenerateRandom(rsp + 12, want);
                body = 2 + want;
                break;
        }

        case TPM_CC_PCR_Extend: {
                uint32_t idx;
                if (cmdlen < 10 + 4 + PCR_LEN) {
                        rc = TPM_RC_COMMAND_CODE; break;
                }
                idx = be32(cmd + 10);
                if (pcr_extend(idx, cmd + 14) != TEE_SUCCESS)
                        rc = TPM_RC_COMMAND_CODE;
                break;
        }

        case TPM_CC_PCR_Read: {
                uint32_t idx;
                if (cmdlen < 14) { rc = TPM_RC_COMMAND_CODE; break; }
                idx = be32(cmd + 10);
                if (idx >= NUM_PCR) {
                        rc = TPM_RC_COMMAND_CODE; break;
                }
                TEE_MemMove(rsp + 10, g_pcr[idx], PCR_LEN);
                body = PCR_LEN;
                break;
        }

        default:
                rc = TPM_RC_COMMAND_CODE;
        }
```

```
        put_be16(rsp, TPM_ST_NO_SESSIONS);
        put_be32(rsp + 2, 10 + body);
        put_be32(rsp + 6, rc);
        *rsplen = 10 + body;
        return rc;
}

/* Wire it to the single SUBMIT_COMMAND entry point exactly as
   the real fTPM does, and you have a TPM you can drive from
   Linux. It will not pass the TCG test suite, but it will
   extend and read PCRs, which is enough to understand what
   chapter 23 is doing.                                    */
```

How it shows up

Symptom	Cause
TA will not load, out of memory	`TA_DATA_SIZE` or `TZDRAM_SIZE` too small for a 2 MB heap
TPM state lost on every reboot	NV commit not reaching secure storage, or `tee-supplicant` not running when it tried
PCR values reset without a reboot	TA unloaded — needs `TA_FLAG_INSTANCE_KEEP_ALIVE`
Every device has the same EK	Fixed development seed. Correct for QEMU, fatal in a product
NV counters can be rewound	REE FS without RPMB (chapter 17)
Sealed data unsealable after a firmware update	Working as designed — the PCRs changed. This is the BitLocker recovery-key experience
Build fails on wolfSSL	Submodules not initialised in `ms-tpm-20-ref`
TPM commands time out	Response buffer smaller than `MAX_RESPONSE_SIZE`
Attestation rejected by a relying party	No EK certificate chaining to a trusted manufacturer

Check yourself

- ✓ Compare a discrete TPM and an fTPM, and say which threat each addresses that the other does not.
- ✓ Define a PCR and write the extend operation. Explain why it cannot be forged.
- ✓ Explain sealing in terms of PCRs, and say what happens after a firmware update.
- ✓ Explain why the fTPM TA has only one real command, and why that is good design.
- ✓ Name the four things the platform layer must provide and how OP-TEE supplies each.
- ✓ Say why the TPM's clock must come from `TEE_GetSystemTime`.
- ✓ Explain why the NV store should be in RPMB and what is lost if it is not.
- ✓ Give three ways to produce an Endorsement Seed and say which a product uses.
- ✓ Explain why the fTPM's size is an engineering constraint on a real SoC.
- ✓ Implement `PCR_Extend` and `PCR_Read` in a minimal TA and drive them from a client.

Project: driving the fTPM from Linux

The trusted application from the previous chapter is useless until something speaks to it. This chapter closes the loop: the kernel driver that presents a TA as `/dev/tpm0`, the userspace tools that then work unmodified, and three real exercises — extending PCRs, sealing a secret to boot state, and attesting to a remote party.

The driver that makes a TA look like a chip

```
The whole trick is that Linux's TPM subsystem already
abstracts the transport. A TPM driver's job is to move
bytes; where they go is its own business.

  tpm_tis.c      -> bytes over memory-mapped registers
  tpm_i2c_*.c    -> bytes over I2C
  tpm_vtpm_proxy -> bytes to a userspace process
  tpm_ftpm_tee.c -> bytes to a TRUSTED APPLICATION

All four register a struct tpm_chip and implement send()
and recv(). Everything above - the TPM 2.0 command layer,
/dev/tpm0, the resource manager, the integrity subsystem -
is shared and unmodified.

So tpm2-tools, systemd-cryptenroll, Clevis, IMA and
everything else work against an fTPM with no changes at
all. That is the payoff of a narrow, protocol-shaped TA
interface (chapter 22).
```

```
/* drivers/char/tpm/tpm_ftpm_tee.c - upstream, ~350 lines.
   This is the annotated core of it.                       */

#define TA_FTPM_UUID UUID_INIT(0xbc50d971, 0xd4c9, 0x42c4, \
                               0x82, 0xcb, 0x34, 0x3f, \
                               0xb7, 0xf3, 0x78, 0x96)

#define FTPM_OPTEE_TA_SUBMIT_COMMAND  (0)
#define MAX_COMMAND_SIZE   4096
#define MAX_RESPONSE_SIZE  4096

struct ftpm_tee_private {
        struct tpm_chip    *chip;
        struct tee_context *ctx;
        u32                 session;
        struct tee_shm     *shm;
};

/* --- send: hand a TPM command to the TA ----------------- */
static int ftpm_tee_tpm_op_send(struct tpm_chip *chip,
                                u8 *buf, size_t len)
{
        struct ftpm_tee_private *pvt = dev_get_drvdata(&chip->dev);
        struct tee_ioctl_invoke_arg transceive_args = {
                .func       = FTPM_OPTEE_TA_SUBMIT_COMMAND,
                .session    = pvt->session,
                .num_params = 4,
        };
        struct tee_param command_params[4] = {0};
        size_t resp_len;
        u8    *temp_buf;
        int    rc;

        if (len > MAX_COMMAND_SIZE)
                return -EIO;

        temp_buf = tee_shm_get_va(pvt->shm, 0);
        if (IS_ERR(temp_buf))
                return PTR_ERR(temp_buf);

        /* Command in at offset 0, response out at offset
           MAX_COMMAND_SIZE. One shared buffer, carved with
           partial memrefs (chapter 11).                    */
        memset(temp_buf, 0, MAX_COMMAND_SIZE);
        memcpy(temp_buf, buf, len);

        command_params[0] = (struct tee_param) {
                .attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INPUT,
                .u.memref = { .shm = pvt->shm,
                              .size = len, .shm_offs = 0 },
        };
        command_params[1] = (struct tee_param) {
                .attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INOUT,
                .u.memref = { .shm = pvt->shm,
                              .size = MAX_RESPONSE_SIZE,
                              .shm_offs = MAX_COMMAND_SIZE },
        };

        rc = tee_client_invoke_func(pvt->ctx, &transceive_args,
                                    command_params);
        if (rc < 0 || transceive_args.ret != 0) {
                dev_err(&chip->dev, "invoke failed %d 0x%x\n",
                        rc, transceive_args.ret);
```

```
                return rc ? rc : -EIO;
        }

        resp_len = command_params[1].u.memref.size;

        /* The TPM response header carries its own length -
           sanity-check the TA's claim against it. */
        if (resp_len > MAX_RESPONSE_SIZE)
                return -EIO;

        temp_buf = tee_shm_get_va(pvt->shm, MAX_COMMAND_SIZE);
        if (IS_ERR(temp_buf))
                return PTR_ERR(temp_buf);

        memcpy(buf, temp_buf, resp_len);
        return 0;
}

/* --- recv: the response is already in buf --------------- */
static int ftpm_tee_tpm_op_recv(struct tpm_chip *chip,
                                u8 *buf, size_t count)
{
        /* send() was synchronous and left the response in
           place, so there is nothing to do. This is why the
           TEE transport is simpler than a real bus: there is
           no polling, no interrupt, no status register.    */
        return 0;
}

static const struct tpm_class_ops ftpm_tee_tpm_ops = {
        .flags     = TPM_OPS_AUTO_STARTUP,
        .recv      = ftpm_tee_tpm_op_recv,
        .send      = ftpm_tee_tpm_op_send,
        .cancel    = ftpm_tee_tpm_op_cancel,
        .status    = ftpm_tee_tpm_op_status,
        .req_complete_mask = 0,
        .req_complete_val  = 0,
        .req_canceled      = ftpm_tee_tpm_req_canceled,
};

/* --- probe: open the session, register the chip --------- */
static int ftpm_tee_probe(struct device *dev)
{
        struct ftpm_tee_private *pvt;
        struct tee_ioctl_open_session_arg sess_arg = {0};
        struct tpm_chip *chip;
        int rc;

        pvt = devm_kzalloc(dev, sizeof(*pvt), GFP_KERNEL);
        if (!pvt)
                return -ENOMEM;
        dev_set_drvdata(dev, pvt);

        /* 1. A TEE context (chapter 10). */
        pvt->ctx = tee_client_open_context(NULL, ftpm_tee_match,
                                           NULL, NULL);
        if (IS_ERR(pvt->ctx))
                return PTR_ERR(pvt->ctx) == -ENOENT
                        ? -EPROBE_DEFER : PTR_ERR(pvt->ctx);

        /* 2. A session to the fTPM TA. */
        export_uuid(sess_arg.uuid, &ftpm_ta_uuid);
        sess_arg.clnt_login = TEE_IOCTL_LOGIN_REE_KERNEL;
```

```
        rc = tee_client_open_session(pvt->ctx, &sess_arg, NULL);
        if (rc < 0 || sess_arg.ret != 0)
                goto out_ctx;
        pvt->session = sess_arg.session;

        /* 3. One shared buffer, for the life of the driver. */
        pvt->shm = tee_shm_alloc_kernel_buf(pvt->ctx,
                        MAX_COMMAND_SIZE + MAX_RESPONSE_SIZE);
        if (IS_ERR(pvt->shm)) {
                rc = -ENOMEM;
                goto out_sess;
        }

        /* 4. Register as a TPM chip. From here on, the rest of
              Linux does not know or care that this is a TA. */
        chip = tpm_chip_alloc(dev, &ftpm_tee_tpm_ops);
        if (IS_ERR(chip)) {
                rc = PTR_ERR(chip);
                goto out_shm;
        }
        chip->flags |= TPM_CHIP_FLAG_TPM2;
        pvt->chip = chip;
        dev_set_drvdata(&chip->dev, pvt);

        rc = tpm_chip_register(chip);
        if (rc)
                goto out_shm;
        return 0;

out_shm:  tee_shm_free(pvt->shm);
out_sess: tee_client_close_session(pvt->ctx, pvt->session);
out_ctx:  tee_client_close_context(pvt->ctx);
        return rc;
}

/* Bound by UUID on the TEE bus (chapter 10) - so the TA must
   have TA_FLAG_DEVICE_ENUM set, or no device appears and this
   driver never probes.                                      */
static const struct tee_client_device_id ftpm_tee_id_table[] = {
        { TA_FTPM_UUID },
        { }
};
MODULE_DEVICE_TABLE(tee, ftpm_tee_id_table);
```

Enabling it

```
# Kernel config:
CONFIG_TEE=y
CONFIG_OPTEE=y
CONFIG_TCG_TPM=y
CONFIG_TCG_FTPM_TEE=y          # the driver above
CONFIG_HW_RANDOM_TPM=y
CONFIG_SECURITYFS=y

# Optional but useful:
CONFIG_IMA=y                   # measure files into PCR 10
CONFIG_IMA_MEASURE_PCR_IDX=10
CONFIG_TCG_TPM2_HMAC=y         # encrypted TPM sessions

$ scripts/config --enable TCG_TPM --enable TCG_FTPM_TEE \
                 --enable HW_RANDOM_TPM
$ make -j$(nproc) Image modules

# On the target:
$ dmesg | grep -iE 'tpm|optee'
optee: probing for conduit method.
optee: revision 4.1
optee: dynamic shared memory is enabled
optee: initialized driver
tpm_ftpm_tee optee-ta-bc50d971-d4c9-42c4-82cb-343fb7f37896: \
    ftpm_tee_probe: TPM chip registered

$ ls -l /dev/tpm*
crw-rw---- 1 tss root  10, 224 /dev/tpm0      # raw
crw-rw---- 1 tss tss  253,  65 /dev/tpmrm0    # resource manager

$ ls /sys/class/tpm/tpm0/
tpm_version_major  pcr-sha256/  ppi/  ...
$ cat /sys/class/tpm/tpm0/tpm_version_major
2
```

> **USE `/DEV/TPMRM0`, NOT `/DEV/TPM0`**
>
> A TPM has room for only a few loaded objects and sessions at a time — three, typically. The resource manager virtualises that: it swaps objects in and out so several processes can use the TPM concurrently without colliding. `/dev/tpm0` is exclusive and unmanaged. Every tool should target `tpmrm0`; if two things fight over the TPM, this is why.

Exercise 1: PCRs and measurement

```
$ apt install tpm2-tools     # or: buildroot -> tpm2-tools
$ export TPM2TOOLS_TCTI=device:/dev/tpmrm0

# What are the PCRs now?
$ tpm2_pcrread sha256
sha256:
  0 : 0x0000000000000000000000000000000000000000000000000000000000000000
  1 : 0x0000...
  ...
# All zero, because on QEMU nothing measured the boot chain
# into them. On a real platform with measured boot, PCRs 0-7
# are populated by firmware before Linux starts.

# Extend one by hand and watch it change:
$ echo -n "hello" | sha256sum
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

$ tpm2_pcrextend 23:sha256=2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
$ tpm2_pcrread sha256:23
  23: 0xF5A5FD42D16A20302798EF6ED309979B43003D2320D9F0E8EA9831A92759FB4B

# Verify the extend rule yourself - PCR_new = SHA256(0^32 || d)
$ python3 -c "
import hashlib, binascii
old = b'\x00'*32
d = binascii.unhexlify('2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824')
print(hashlib.sha256(old+d).hexdigest())
"
f5a5fd42d16a20302798ef6ed309979b43003d2320d9f0e8ea9831a92759fb4b
# Matches. The TA really is computing the extend correctly.

# Extend again with the SAME value - a different result,
# because extend is order- and history-dependent:
$ tpm2_pcrextend 23:sha256=2cf24dba...
$ tpm2_pcrread sha256:23
  23: 0x9B54...      # different again

# Only a reboot resets it:
$ tpm2_pcrreset 23         # allowed for PCR 16 and 23 only;
                           # 0-15 are reset-protected, which is
                           # what makes them trustworthy.

# Let the kernel measure files into PCR 10:
$ mount -t securityfs securityfs /sys/kernel/security
$ cat /sys/kernel/security/ima/ascii_runtime_measurements | head -3
10 8f4a... ima-ng sha256:e3b0... boot_aggregate
10 c1d2... ima-ng sha256:9f86... /init
$ tpm2_pcrread sha256:10
```

Exercise 2: sealing a secret to boot state

```
# This is BitLocker, in miniature: a secret that can only be
# recovered if the machine boots the same software.

# 1. Establish a known state.
$ tpm2_pcrreset 23
$ tpm2_pcrextend 23:sha256=$(echo -n "my-boot-state" | sha256sum | cut -d' ' -f1)
$ tpm2_pcrread sha256:23
  23: 0x4EF7...

# 2. Create a primary key in the owner hierarchy.
$ tpm2_createprimary -C o -g sha256 -G ecc -c primary.ctx

# 3. Build a policy: "only when PCR 23 has its current value".
$ tpm2_startauthsession -S session.ctx
$ tpm2_policypcr -S session.ctx -l sha256:23 -L policy.digest
$ tpm2_flushcontext session.ctx
$ xxd policy.digest

# 4. Seal a secret under that policy.
$ echo -n "the launch codes" > secret.txt
$ tpm2_create -C primary.ctx -i secret.txt \
      -u sealed.pub -r sealed.priv -L policy.digest \
      -a "fixedtpm|fixedparent"
$ tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx

#   fixedtpm    this object can never be duplicated to another
#               TPM. The secret is bound to THIS device.
#   fixedparent it cannot be rewrapped under a different parent.

# 5. Unseal - with the policy satisfied.
$ tpm2_startauthsession --policy-session -S session.ctx
$ tpm2_policypcr -S session.ctx -l sha256:23
$ tpm2_unseal -p session:session.ctx -c sealed.ctx
the launch codes
$ tpm2_flushcontext session.ctx

# 6. NOW CHANGE THE BOOT STATE and try again.
$ tpm2_pcrextend 23:sha256=$(echo -n "tampered" | sha256sum | cut -d' ' -f1)

$ tpm2_startauthsession --policy-session -S session.ctx
$ tpm2_policypcr -S session.ctx -l sha256:23
$ tpm2_unseal -p session:session.ctx -c sealed.ctx
ERROR: Esys_Unseal(0x99D) - tpm:session(1):a policy check failed
$ tpm2_flushcontext session.ctx

# The sealed blob is on the normal-world filesystem, readable
# by anyone. It is useless to them: the decryption key lives in
# the TPM and the TPM will not use it unless the PCRs match.
#
# And on this system the "TPM" is a trusted application. The
# whole guarantee rests on the secure world's isolation - which
# is exactly the claim chapter 1 started with.
```

SEALING TO PCRS IS A SUPPORT BURDEN

Every firmware update changes PCRs 0–7 and every sealed secret becomes unrecoverable. That is not a bug; it is the entire point. Real systems handle it by sealing to a policy signed by the vendor (`tpm2_policyauthorize`) so a new signed PCR set can be authorised after an update, and by always keeping a recovery path. Seal to raw PCR values only for experiments.

Exercise 3: attestation

```
# Prove to a remote party what this machine booted - without
# trusting anything the machine says about itself.

# --- On the device ---
# 1. An attestation key, restricted to signing TPM-internal
#    data. Restricted means it CANNOT sign arbitrary bytes,
#    so a quote cannot be forged by asking the TPM to sign a
#    fabricated one.
$ tpm2_createprimary -C e -g sha256 -G ecc -c ek.ctx
$ tpm2_create -C ek.ctx -G ecc -u ak.pub -r ak.priv \
      -a "fixedtpm|fixedparent|sensitivedataorigin|\
userwithauth|restricted|sign"
$ tpm2_load -C ek.ctx -u ak.pub -r ak.priv -c ak.ctx
$ tpm2_readpublic -c ak.ctx -o ak.pem -f pem

# 2. The verifier sends a random nonce. Freshness matters:
#    without it, a quote can be replayed (chapter 17).
$ NONCE=$(openssl rand -hex 20)

# 3. Quote: the TPM signs {PCR digest, nonce, clock, firmware
#    version} with the AK.
$ tpm2_quote -c ak.ctx -l sha256:0,1,2,3,4,5,6,7,10 \
      -q $NONCE -m quote.msg -s quote.sig -o pcrs.out -g sha256

# --- Send ak.pem, quote.msg, quote.sig, pcrs.out to the
#     verifier ---

# --- On the verifier ---
$ tpm2_checkquote -u ak.pem -m quote.msg -s quote.sig \
      -f pcrs.out -g sha256 -q $NONCE
pcrs:
  sha256:
    0 : 0x0000...
    10: 0x3F5A...
sig: signature is valid

# The verifier now knows:
#   - a genuine TPM produced this (if the AK chains to an EK
#     certificate from a manufacturer it trusts)
#   - these were the PCR values at quote time
#   - the quote is fresh (the nonce matches)
#
# It does NOT know the PCRs are "good" - that requires a
# reference set of expected values. Attestation proves WHAT
# ran; deciding whether that is acceptable is policy, and it
# is the hard part of every attestation system.

# THE MISSING PIECE ON QEMU: the EK has no certificate, and
# every emulated device shares a seed (chapter 22). So this
# proves the mechanism works, not that the device is genuine.
# A product provisions an EK certificate at manufacture.
```

What this is actually for

Use	Command
Unlock a disk if the boot chain is unchanged	`systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=0+2+7`

Use	Command
Same, on other distributions	`clevis luks bind -d /dev/sda2 tpm2 '{"pcr_ids":"7"}'`
SSH keys the machine cannot export	`tpm2-pkcs11`, then `ssh-keygen -D /usr/lib/libtpm2_pkcs11.so`
TLS client certificates bound to hardware	`tpm2-openssl` or `tpm2-tss-engine`
Runtime file integrity measurement	IMA, into PCR 10
Hardware entropy for the kernel	`CONFIG_HW_RANDOM_TPM`
Remote device health attestation	`tpm2_quote` plus a verifier
Anti-rollback for application state	TPM NV counters — `tpm2_nvdefine -a "nt=counter"`

```
# The one worth trying end to end, because it is a real
# workflow people actually deploy:

$ systemd-cryptenroll /dev/sda2 --tpm2-device=/dev/tpmrm0 \
      --tpm2-pcrs=7
$ reboot
# The disk unlocks with no password - unless PCR 7 changed,
# in which case it asks for the recovery passphrase.
#
# On this system, "the TPM" is a trusted application running
# beside Linux in the secure world. Nothing above it knows.
```

How it shows up

Symptom	Cause
No `/dev/tpm0`	`CONFIG_TCG_FTPM_TEE` off, TA not installed, or the TA lacks `TA_FLAG_DEVICE_ENUM`
Driver probes then fails	Session open failed — check the secure console; usually the TA is missing or unsigned
`tpm2_*` hangs or errors with a TCTI failure	Pointing at `/dev/tpm0` while something else holds it. Use `tpmrm0`
Permission denied	Not in the `tss` group
`out of memory for object contexts`	Too many loaded objects — use `tpmrm0`, or `tpm2_flushcontext -t`
PCRs all zero on a real board	Firmware is not doing measured boot; the fTPM is fine
PCRs reset without a reboot	The TA unloaded — needs `TA_FLAG_INSTANCE_KEEP_ALIVE`
Unseal fails after an update	PCRs changed. Correct behaviour; use a signed policy
TPM state lost across reboot	NV commit not persisting (chapter 22)
`tpm2_checkquote` fails	Nonce mismatch, wrong PCR selection, or the AK is not restricted
Attestation rejected by a real verifier	No EK certificate — expected on QEMU

Check yourself

- ✓ Explain how one driver makes a TA indistinguishable from a TPM chip, and why nothing above it changes.
- ✓ Point to the `send()` path and say how command and response share one buffer.
- ✓ Explain why `recv()` has nothing to do.

- ✓ Say what the resource manager does and why tools should use `/dev/tpmrm0`.
- ✓ Verify the PCR extend rule by hand against the TPM's output.
- ✓ Explain why PCRs 0–15 cannot be reset and what that buys.
- ✓ Seal a secret to a PCR value and demonstrate that changing the PCR prevents unsealing.
- ✓ Explain what `fixedtpm` and `fixedparent` guarantee.
- ✓ Explain why an attestation key must be *restricted*, and what the nonce prevents.
- ✓ State what a quote proves and what it does not.
- ✓ Enrol a disk with `systemd-cryptenroll` against the fTPM and explain what happens after a firmware update.

Porting OP-TEE to new hardware

Everything so far ran on QEMU, where the platform port already existed and the secure memory was imaginary. Bringing OP-TEE up on real silicon is a different kind of work: a few hundred lines of platform code, one console, one memory carve-out that must be enforced by hardware, and a long list of questions only the SoC's security manual can answer.

What a port actually is

```
OP-TEE is mostly platform-independent. A port is a
directory:

  core/arch/arm/plat-<yours>/
    conf.mk            what the platform has and wants
    main.c             console, early init, hardware setup
    platform_config.h  addresses and sizes
    sub.mk             which files to build
    kern.ld.S          only if your layout is unusual

Plus decisions made OUTSIDE OP-TEE, which are the hard part:

  TF-A            must load OP-TEE as BL32 and pass a
                  correct boot argument (chapter 2)
  TZASC / XPU     must actually FENCE the secure DRAM
                  (chapter 6) - firmware's job, not
                  OP-TEE's
  device tree     Linux must be told OP-TEE is there
  fuses / keys    the HUK, the TA signing key, secure boot

Rough effort on a well-supported SoC whose vendor already
ships TF-A:

  console up ................. half a day
  first TA runs .............. two days
  xtest passing .............. one to two weeks
  RPMB, HUK, secure boot .... weeks, and mostly paperwork

The middle number is small and the last one is not. Budget
accordingly: the port is easy, the PLATFORM SECURITY is
not.
```

THE QUESTION THAT DECIDES EVERYTHING

Can the hardware make a region of DRAM inaccessible to the non-secure world, and can that configuration be locked before non-secure code runs? If yes, a port is worth doing. If no — no TZASC, no XPU, no lockable firewall — then OP-TEE will run and provide *no isolation at all*, because normal-world Linux can read secure DRAM directly. Answer this before writing a line of code.

The information you need first

Question	Where the answer lives	Why it matters
Is there a TZASC/TZC-400, or a vendor XPU?	SoC security manual	Without it there is no secure memory (chapter 6)

Question	Where the answer lives	Why it matters
Which DRAM range can be made secure, at what granularity?	Memory map + TZC region docs	Sets `TZDRAM_BASE` / `SIZE` and their alignment
Is there secure on-chip SRAM, and how much?	Memory map	Decides whether you can run without a DRAM carve-out, and where `TEE_SHMEM` lives
Which UART can be made secure-only?	Peripheral firewall docs	The secure console. Sharing one with Linux works but interleaves badly
GICv2 or GICv3? Which interrupts are Group 0?	GIC integration section	OP-TEE's notification and its own timer
Is the CPU count and cluster topology known at build time?	TRM	`CFG_TEE_CORE_NB_CORE`
Does the vendor ship TF-A, and which version?	Vendor BSP	Determines the handover mechanism you must match
Is there an eMMC with RPMB, and is the RPMB key programmed?	Board schematic + manufacturing	Rollback protection (chapter 17)
Where does the HUK come from?	Security manual: fuses, key ladder, or crypto accelerator	All of secure storage depends on it
Is there a hardware TRNG?	Peripheral list	Otherwise entropy is a real problem

The four files

```
# core/arch/arm/plat-acme/conf.mk

PLATFORM_FLAVOR ?= acme_evk

include core/arch/arm/cpu/cortex-armv8-0.mk

$(call force,CFG_TEE_CORE_NB_CORE,4)
$(call force,CFG_ARM64_core,y)          # 64-bit TEE core
$(call force,CFG_GIC,y)
$(call force,CFG_ARM_GICV3,y)
$(call force,CFG_PL011,y)               # console driver
$(call force,CFG_SECURE_TIME_SOURCE_CNTPCT,y)
$(call force,CFG_WITH_ARM_TRUSTED_FW,y) # TF-A loads us as BL32

CFG_TZDRAM_START  ?= 0x0e100000
CFG_TZDRAM_SIZE   ?= 0x00f00000         # 15 MB
CFG_SHMEM_START   ?= 0x0f000000
CFG_SHMEM_SIZE    ?= 0x00200000         # 2 MB

CFG_DRAM0_BASE    ?= 0x00000000
CFG_DRAM0_SIZE    ?= 0x80000000

CFG_CORE_DYN_SHM      ?= y   # dynamic shared memory (ch. 7)
CFG_CORE_ASLR         ?= y
CFG_TEE_CORE_LOG_LEVEL ?= 2

# Turn these on only once the basics work:
# CFG_RPMB_FS         ?= y
# CFG_RPMB_WRITE_KEY  ?= n     # NEVER y in production images
# CFG_WITH_PAGER      ?= y     # if secure RAM is tight
# CFG_CORE_SEL2_SPMC  ?= n
```

```
/* core/arch/arm/plat-acme/platform_config.h */

#ifndef PLATFORM_CONFIG_H
#define PLATFORM_CONFIG_H

#include <mm/generic_ram_layout.h>

/* Cache line size - wrong value here corrupts data silently
   through partial cache maintenance.                       */
#define CACHELINE_LEN           64
#define STACK_ALIGNMENT         64

/* GIC */
#define GICD_BASE               0x2f000000
#define GICR_BASE               0x2f100000

/* Console: the UART OP-TEE owns. Must be firewalled to the
   secure world, or normal-world Linux will also drive it and
   the output interleaves into nonsense.                    */
#define CONSOLE_UART_BASE       0x1c090000
#define CONSOLE_BAUDRATE        115200
#define CONSOLE_UART_CLK_IN_HZ  24000000

#endif /* PLATFORM_CONFIG_H */
```

```
/* core/arch/arm/plat-acme/main.c - the whole platform port. */

#include <console.h>
#include <drivers/gic.h>
#include <drivers/pl011.h>
#include <kernel/boot.h>
#include <kernel/panic.h>
#include <mm/core_memprot.h>
#include <platform_config.h>

static struct pl011_data console_data;

/* Tell the memory manager these physical ranges exist and how
   they may be mapped. MISSING ONE OF THESE IS THE COMMONEST
   PORTING BUG: the driver writes to an address that was never
   mapped and the core panics with a translation fault.     */
register_phys_mem_pgdir(MEM_AREA_IO_SEC,
                        CONSOLE_UART_BASE, PL011_REG_SIZE);
register_phys_mem_pgdir(MEM_AREA_IO_SEC, GICD_BASE, GIC_DIST_REG_SIZE);
register_phys_mem_pgdir(MEM_AREA_IO_SEC, GICR_BASE, GIC_RDIST_REG_SIZE);

/* Normal-world DRAM: OP-TEE must know which memory is NOT
   secure, so it can refuse a TA that hands it a pointer into
   secure RAM (chapter 20).                                 */
register_ddr(CFG_DRAM0_BASE, CFG_DRAM0_SIZE);

void console_init(void)
{
        pl011_init(&console_data, CONSOLE_UART_BASE,
                   CONSOLE_UART_CLK_IN_HZ, CONSOLE_BAUDRATE);
        register_serial_console(&console_data.chip);
}

void boot_primary_init_intc(void)
{
        gic_init(GICD_BASE, GICR_BASE);
}

void boot_secondary_init_intc(void)
{
        gic_init_per_cpu();
}

/* Optional: platform-specific early setup - clocks, a crypto
   accelerator, the fuse controller that supplies the HUK.  */
static TEE_Result acme_late_init(void)
{
        IMSG("ACME platform init");
        return TEE_SUCCESS;
}
service_init(acme_late_init);
```

```
# core/arch/arm/plat-acme/sub.mk
global-incdirs-y += .
srcs-y += main.c

# And register the platform so `make PLATFORM=acme` works -
# the directory name is the platform name; no central list to
# edit.

$ make PLATFORM=acme CROSS_COMPILE64=aarch64-linux-gnu- \
       CFG_ARM64_core=y -j$(nproc)
$ ls out/arm-plat-acme/core/
tee.bin  tee.elf  tee-header_v2.bin  tee-pager_v2.bin
tee-pageable_v2.bin
```

The TF-A side

```
/* OP-TEE cannot load itself. TF-A's BL2 loads it as BL32 and
   BL31 hands control over. Two things must agree.        */

/* 1. TF-A must be built to know about it: */
$ make -C arm-trusted-firmware \
      PLAT=acme \
      SPD=opteed \                    # the OP-TEE dispatcher
      BL32=~/optee_os/out/arm-plat-acme/core/tee-header_v2.bin \
      BL32_EXTRA1=.../tee-pager_v2.bin \
      BL32_EXTRA2=.../tee-pageable_v2.bin \
      BL33=u-boot.bin \
      ARM_TSP_RAM_LOCATION=dram \
      all fip

/* 2. The boot argument. TF-A passes OP-TEE a device tree
   pointer (or a legacy argument) in x0/x2 at entry. Getting
   this wrong gives you a hang with no output at all, which
   is the single most demoralising failure in this chapter.

   Debug it by putting a character out of the UART as the
   VERY FIRST instruction of OP-TEE's entry point:

       core/arch/arm/kernel/entry_a64.S, at _start:

           mov  x20, x0            // save the real arg first!
           ldr  x1, =0x1c090000    // UART DR
           mov  w2, #'0'
           str  w2, [x1]

   If '0' appears, TF-A reached you and the problem is later.
   If it does not, the problem is the load address, the
   entry point, or the TZASC fencing you off from your own
   code.                                                  */
```

The part that is not software

```
/* The carve-out must be enforced by HARDWARE, before any
   non-secure code runs, and the configuration must be
   LOCKED. This is TF-A's BL2/BL31 job.                    */

/* TZC-400 example, from a platform's BL2:                  */

void plat_acme_security_setup(void)
{
        tzc400_init(TZC400_BASE);
        tzc400_disable_filters();

        /* Region 0 is the background: everything not covered
           by another region. Make it NON-SECURE by default,
           so a region you forget to configure is at least not
           silently secure.                                  */
        tzc400_configure_region0(TZC_REGION_S_NS_RDWR, 0);

        /* Region 1: the OP-TEE carve-out. SECURE ONLY.
           No non-secure master - CPU or DMA - may touch it. */
        tzc400_configure_region(TZC_FILTERS_ALL, 1,
                TZDRAM_BASE,
                TZDRAM_BASE + TZDRAM_SIZE - 1,
                TZC_REGION_S_RDWR,      /* secure only       */
                0);                     /* no NSAID access   */

        tzc400_set_action(TZC_ACTION_ERR);  /* fault, do not
                                               silently drop */
        tzc400_enable_filters();

        /* And lock it, if the hardware supports locking.
           An unlocked TZASC can be reprogrammed by the
           normal world later, which defeats the whole
           exercise.                                         */
}

/* VERIFY IT. Do not assume. From Linux, as root:          */

$ devmem2 0x0e100000 w
/dev/mem opened.
Memory mapped at address 0xffff...
Bus error                       <-- CORRECT. The fence works.

# If instead you see:
Value at address 0x0E100000: 0x14000000
# then you are reading OP-TEE's own code from the normal
# world, and there is no isolation. Everything else in this
# track is theatre until this reads "Bus error".

# Also check DMA, which is the harder half: program a DMA
# engine to read the same address. The TZASC must block the
# device too, and on some SoCs devices are governed by a
# DIFFERENT firewall (an XPU) that you must configure
# separately.
```

> **THE TWO MISTAKES THAT VOID THE WHOLE PORT**
> First: configuring the TZASC but never locking it, so normal-world firmware can widen the window later. Second: fencing the CPU but not the DMA masters — the SMMU and the peripheral firewall are separate blocks with separate configuration, and a passthrough-capable DMA engine that is not covered can read secure DRAM on the guest's behalf. Test both paths explicitly; a passing `xtest` proves neither.

Telling Linux

```
/ {
        firmware {
                optee {
                        compatible = "linaro,optee-tz";
                        method = "smc";
                };
        };

        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

                /* The carve-out, so Linux never allocates
                   here. The TZASC would fault it anyway, but
                   a fault at a random later moment is far
                   worse than never touching it.           */
                optee@0e100000 {
                        reg = <0x0 0x0e100000 0x0 0x00f00000>;
                        no-map;
                };
        };
};

# Then in the guest:
$ dmesg | grep -i optee
optee: probing for conduit method.
optee: revision 4.1 (4a5b6c7d)
optee: dynamic shared memory is enabled
optee: initialized driver

$ ls -l /dev/tee*
crw------- 1 root root 234, 0 /dev/tee0
crw------- 1 root root 234, 16 /dev/teepriv0
```

Bring-up order that works

Step	Success looks like	If it fails
1. TF-A boots to BL33 *without* OP-TEE	U-Boot prompt	Not an OP-TEE problem yet
2. Add BL32, get one character out	Your debug character on the secure UART	Load address, entry point, or the carve-out is fencing OP-TEE from itself
3. OP-TEE banner	`OP-TEE version: 4.1`	Console clock or base address wrong
4. Secondary cores	`Secondary CPU 1 initializing`	PSCI or GIC per-CPU init

Step	Success looks like	If it fails
5. Linux probes the driver	`optee: initialized driver`	Device tree node, or the SMC conduit
6. `tee-supplicant` runs	No errors in its log	Missing `/data/tee`, or permissions
7. `optee_example_hello_world`	An incrementing value	TA not installed, or signature key mismatch
8. `xtest -l 0`	All pass	Read the secure console; failures are usually crypto or storage
9. `xtest` full	All pass	Now it is a real port
10. `devmem2` on the carve-out	**Bus error**	You have no isolation. Stop and fix the firewall
11. RPMB, HUK, secure boot, signing keys	Product-ready	Mostly manufacturing process

What separates a port from a product

```
/* A port that passes xtest is a development port. Shipping
   requires all of the following, and none of them is code you
   write in OP-TEE.                                          */

  1. A REAL HUK.
     The default is often zeros or a build-time constant. Then
     every device derives the same storage key and secure
     storage protects nothing (chapter 16). Wire
     tee_otp_get_hw_unique_key() to fuses or a key ladder,
     and prove two boards produce different values.

  2. A REAL TA SIGNING KEY.
     The default key ships in the source tree. Anyone can sign
     a TA for your device. Generate one, keep the private half
     in an HSM, and pass TA_SIGN_KEY= at build time
     (chapter 14).

  3. SECURE BOOT, enabled and fused.
     A verified boot chain that is not enforced by fuses is a
     boot chain an attacker replaces.

  4. RPMB, with the key programmed at manufacture.
     CFG_RPMB_WRITE_KEY=y is a development convenience that
     programs a one-time-writable key. Shipping it lets the
     first attacker claim the RPMB.

  5. LOGGING TURNED DOWN.
     CFG_TEE_CORE_LOG_LEVEL=1, CFG_TEE_TA_LOG_LEVEL=0. Debug
     output from the secure world is an information leak, and
     DMSG in a hot path is a timing side channel.

  6. CFG_CORE_ASLR=y, and no CFG_TEE_CORE_DEBUG.

  7. A DOCUMENTED THREAT MODEL.
     Which attacks is this defending against? Bus probing?
     A malicious app? A compromised kernel? Physical
     decapping? The answer decides whether the port is
     adequate, and it is the question an auditor asks first.
```

How it shows up

Symptom	Cause
Total silence after TF-A hands over	Wrong load address or entry point, or the TZASC is fencing OP-TEE out of its own memory
Garbage on the secure console	`CONSOLE_UART_CLK_IN_HZ` wrong
Interleaved secure and normal output	Both worlds driving one UART; firewall it or accept the mess in development only
Panic with a translation fault in a driver	Missing `register_phys_mem_pgdir()` for that MMIO range
Boots on one core, hangs on SMP	`CFG_TEE_CORE_NB_CORE` wrong, or per-CPU GIC init missing
Random corruption under load	`CACHELINE_LEN` wrong
Linux never probes the driver	Device tree `firmware/optee` node missing or `method` wrong
Everything works and `devmem2` reads the carve-out	No hardware fencing. The port provides zero isolation
Secure storage readable on another board	HUK is a constant
A TA signed by someone else loads	Default signing key still in use
Works on the EVK, fails on production silicon	Different fuse configuration, or a firewall the EVK left open

Check yourself

- ✓ Name the four files in a platform port and what each contributes.
- ✓ State the one hardware question that decides whether a port is worth doing.
- ✓ Explain what `register_phys_mem_pgdir()` does and the failure when it is missing.
- ✓ Explain why `register_ddr()` matters for TA parameter validation.
- ✓ Describe how TF-A loads OP-TEE and name the build variables involved.
- ✓ Give the first-character debug technique and say what each outcome tells you.
- ✓ Write the TZASC configuration for a secure carve-out and say why region 0 defaults to non-secure.
- ✓ Prove from Linux that the carve-out is genuinely fenced, and say what a successful read means.
- ✓ Explain why fencing the CPU is not sufficient, and what else must be configured.
- ✓ List the bring-up steps in order and give the success signal for each.
- ✓ Name five things that separate a working port from a shippable one.

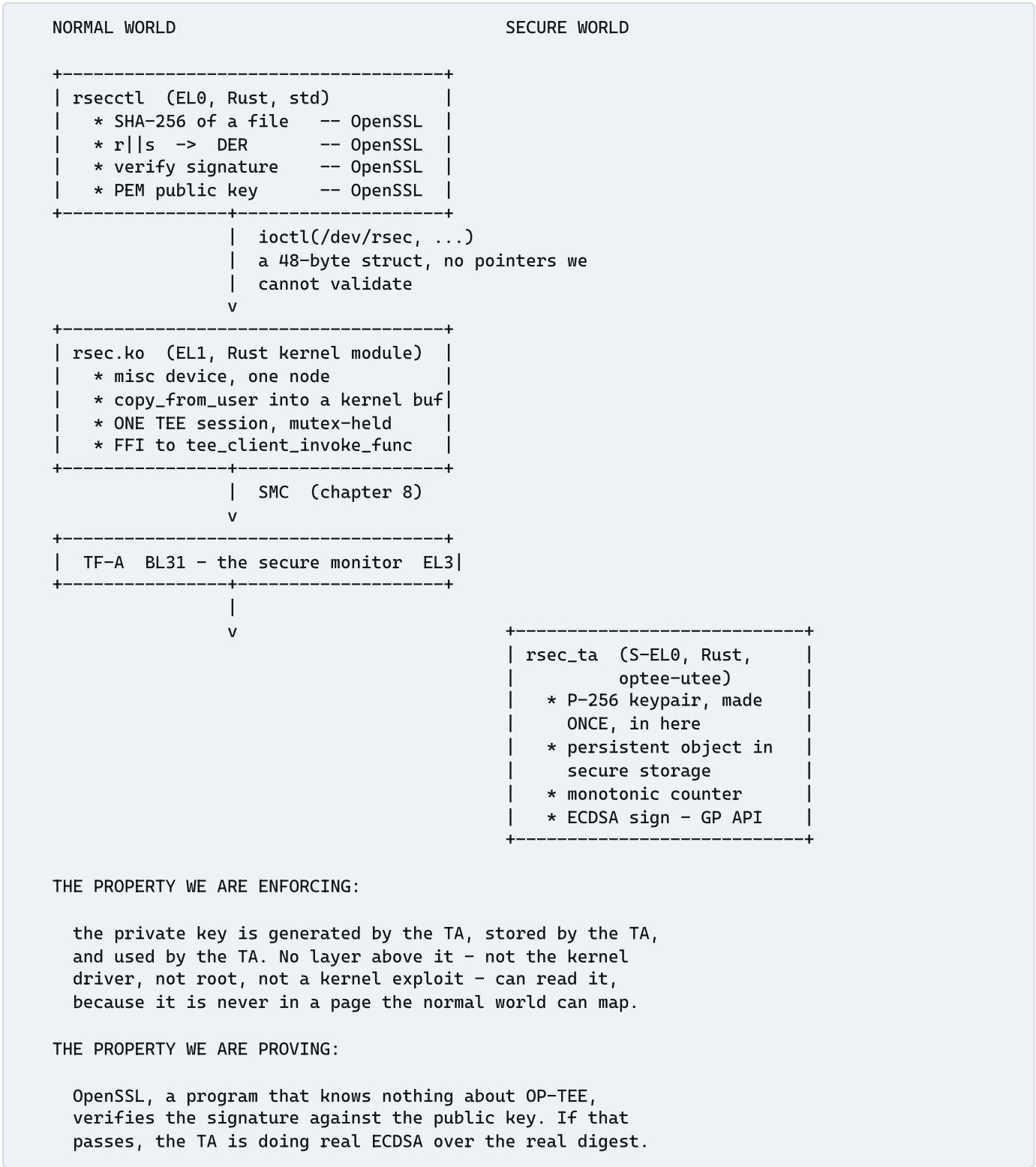
Project: a full-stack secure signing service in Rust

One request travels through four privilege domains — a user-mode driver, a kernel driver, the secure monitor, and a trusted application — and comes back with a signature made by a key that never existed in normal-world memory. Every line of it is Rust. OpenSSL, in the normal world, then verifies that signature, which is how you know the whole chain is real and not merely self-consistent.

WHERE THIS SITS

Harder than the key store, which talked to its TA directly from user space. Easier than the fTPM, which integrated forty thousand lines of someone else's C. The new idea here is the **kernel driver in the middle**, and the language is the other new idea.

What we are building



Why put a kernel driver in the middle at all?

The key store project called `libteec` straight from user space and worked perfectly. Adding a kernel layer is not free — it is a module to maintain, an ioctl ABI to freeze, and a new place for bugs. It earns its place for four reasons, and if none of them apply to your problem you should skip it.

Reason	What it buys
The kernel itself is a consumer	dm-crypt wants a key, IMA wants to extend a measurement, the RNG wants entropy. Kernel code cannot call `libteec` — it must use the in-kernel client API
Policy and serialisation	One TA session shared by every caller, with a mutex, instead of every process opening its own. OP-TEE has a fixed thread pool (chapter 5); uncontrolled session creation exhausts it
A standard interface	Present the service as a device, an `hwrng`, a `crypto_alg` or a TPM, and existing tools work unmodified — exactly what the fTPM driver does
The data is kernel-owned	A measurement taken from a device the kernel owns must not be laundered through user space on its way to the TEE, or the value proves nothing

> **THE COST, STATED HONESTLY**
>
> Every request now crosses user→kernel *and* normal→secure. On QEMU expect tens of microseconds per round trip; the extra syscall is noise next to the world switch. What the kernel layer really costs you is *an ABI you can never change*, which is why the ioctl design below gets a section to itself.

Where the cryptography lives

The brief says "cryptographic operations done with OpenSSL". That is right, and it belongs in the normal world — not inside the TA. This is worth being explicit about because it is a design decision, not a limitation I am working around.

```
OPENSSL, in rsecctl, normal world:

  SHA-256 over the file             bulk data. Megabytes.
                                    Hashing is not secret and
                                    moving megabytes through
                                    shared memory into a TA
                                    would be slow and pointless.

  raw r||s  ->  DER SEQUENCE        a format conversion.
                                    No secrets involved.

  ECDSA VERIFY                      uses the PUBLIC key. There
                                    is nothing to protect.

  SubjectPublicKeyInfo / PEM        packaging, for the rest of
                                    the world to consume.

THE TA, secure world, GP Internal API:

  key generation                    the private key must come
                                    into existence inside the
                                    boundary or the whole
                                    exercise is theatre.

  key storage                       secure storage, sealed to
                                    the HUK (chapter 15).

  ECDSA SIGN over a 32-byte digest  the only operation that
                                    touches the private key.

THE RULE, which generalises far beyond this project:

  put in the TEE exactly what must not leak, and nothing
  else. Every extra byte and every extra line inside the
  boundary is attack surface (chapter 20) and secure RAM
  you do not have (chapter 6).

And note what is NOT possible: linking OpenSSL into a TA.
A TA is S-EL0 with no OS, no sockets, no files, a few
hundred KB of heap and a libc subset. The fTPM manages it
with wolfSSL and a 2 MB heap (chapter 22) and that is a
serious undertaking. Use the GP crypto API inside; use
OpenSSL outside.
```

The ABI: the part you cannot change later

```
// crates/rsec-abi/src/lib.rs
//
// Shared by all three layers. The kernel module and the user
// program both depend on this crate, so a change that breaks
// one fails to compile the other. That is the entire point of
// putting it in a crate instead of writing the numbers twice.

#![cfg_attr(not(feature = "std"), no_std)]

pub const RSEC_ABI_VERSION: u32 = 1;

/// ioctl "type" byte. Anything unclaimed in
/// Documentation/userspace-api/ioctl/ioctl-number.rst will do.
pub const RSEC_IOC_MAGIC: u32 = 0xE7;

// ------------------------------------------------------------------
// ioctl encoding, asm-generic (arm64, x86, riscv - not sparc/ppc)
//
//   bits 31..30  direction   1 = write, 2 = read, 3 = both
//   bits 29..16  size of the argument struct
//   bits 15..8   type
//   bits  7..0   number
//
// We compute these by hand rather than pulling in a macro crate,
// because the kernel side and the user side must agree exactly
// and there is value in seeing the arithmetic once.
// ------------------------------------------------------------------
const DIR_WRITE: u32 = 1;
const DIR_READ:  u32 = 2;

const fn ioc(dir: u32, nr: u32, size: usize) -> u32 {
    (dir << 30) | ((size as u32) << 16) | (RSEC_IOC_MAGIC << 8) | nr
}
const fn iowr(nr: u32, size: usize) -> u32 {
    ioc(DIR_READ | DIR_WRITE, nr, size)
}

/// Fetch the P-256 public key as 64 raw bytes: X || Y.
#[repr(C)]
#[derive(Debug, Clone, Copy, Default)]
pub struct RsecPubkey {
    /// Caller writes RSEC_ABI_VERSION. Kernel rejects anything else.
    pub version: u32,
    /// Caller writes the capacity of `buf`; kernel writes back
    /// the number of bytes produced.
    pub len:     u32,
    /// User pointer, as a u64 so the struct is the same size for
    /// 32-bit and 64-bit user space. See the note below.
    pub buf:     u64,
}

/// Sign one SHA-256 digest. Returns raw r || s, 64 bytes.
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct RsecSign {
    pub version: u32,
    /// in: capacity of `sig`; out: bytes written
    pub sig_len: u32,
    /// The digest travels INLINE. It is 32 bytes; a pointer plus
    /// a length would be more code and more to validate.
```

```
    pub digest:  [u8; 32],
    pub sig:     u64,
}

/// Read, and optionally increment, the monotonic counter.
#[repr(C)]
#[derive(Debug, Clone, Copy, Default)]
pub struct RsecCounter {
    pub version:   u32,
    /// 1 = increment before reading, 0 = read only
    pub increment: u32,
    /// out
    pub value:     u64,
}

pub const RSEC_IOC_PUBKEY:  u32 = iowr(1, core::mem::size_of::<RsecPubkey>());
pub const RSEC_IOC_SIGN:    u32 = iowr(2, core::mem::size_of::<RsecSign>());
pub const RSEC_IOC_COUNTER: u32 = iowr(3, core::mem::size_of::<RsecCounter>());

// Command ids on the TEE side. Deliberately separate from the
// ioctl numbers: the kernel is a translator, not a pass-through,
// and the two ABIs are allowed to evolve independently.
pub const TA_CMD_GET_PUBKEY: u32 = 0;
pub const TA_CMD_SIGN:       u32 = 1;
pub const TA_CMD_COUNTER:    u32 = 2;

pub const RSEC_TA_UUID: &str = "7f2e5c10-9a41-4b8d-b3c6-1e0d5a72f491";

pub const PUBKEY_LEN: usize = 64;   // P-256: 32-byte X, 32-byte Y
pub const SIG_LEN:    usize = 64;   // P-256: 32-byte r, 32-byte s
pub const DIGEST_LEN: usize = 32;   // SHA-256
```

> **FOUR ABI RULES, LEARNED THE HARD WAY BY EVERYONE**
>
> **1.** Pointers in ioctl structs are `u64`, never `usize` or a real pointer type — otherwise a 32-bit process on a 64-bit kernel sends a differently-sized struct and you need a whole `compat_ioctl` path. **2.** Every field is naturally aligned and the struct has no implicit padding, so there are no uninitialised bytes to leak back to user space. **3.** A version field is first, always, and the kernel rejects a mismatch. **4.** The size is baked into the ioctl number, so a struct that changes size produces `ENOTTY` rather than silent corruption.

The trusted application, in Rust

TAs in Rust are built with the Apache Teaclave TrustZone SDK, which wraps the GlobalPlatform Internal Core API in the `optee-utee` crate and emits an ordinary signed `.ta` file. The five entry points of chapter 12 become five attribute macros.

```
# ta/Cargo.toml
[package]
name    = "rsec-ta"
version = "0.1.0"
edition = "2021"

[dependencies]
optee-utee       = { git = "https://github.com/apache/incubator-teaclave-trustzone-sdk.git" }
optee-utee-sys   = { git = "https://github.com/apache/incubator-teaclave-trustzone-sdk.git" }
rsec-abi         = { path = "../crates/rsec-abi", default-features = false }

[build-dependencies]
optee-utee-build = { git = "https://github.com/apache/incubator-teaclave-trustzone-sdk.git" }

[profile.release]
panic = "abort"      # a TA has nowhere to unwind to
lto   = true
opt-level = "z"      # secure RAM is the scarce resource
```

```
// ta/src/main.rs
#![no_main]

use optee_utee::{
    ta_close_session, ta_create, ta_destroy, ta_invoke_command,
    ta_open_session, trace_println,
};
use optee_utee::{Error, ErrorKind, Parameters, Result};
use optee_utee::{
    Asymmetric, AlgorithmId, OperationMode,
    TransientObject, TransientObjectType,
    PersistentObject, ObjectStorageConstants, DataFlag,
    Attribute, AttributeId,
};

use rsec_abi::{
    TA_CMD_GET_PUBKEY, TA_CMD_SIGN, TA_CMD_COUNTER,
    PUBKEY_LEN, SIG_LEN, DIGEST_LEN,
};

/// Object ids in this TA's private secure storage. Scoped to the
/// TA's UUID by OP-TEE, so no other TA can name them.
const KEY_OBJECT_ID:     &[u8] = b"rsec.signing-key.v1";
const COUNTER_OBJECT_ID: &[u8] = b"rsec.counter.v1";

const P256_BITS: u32 = 256;

// ---------------------------------------------------------------------
// Entry points
// ---------------------------------------------------------------------

#[ta_create]
fn create() -> Result<()> {
    trace_println!("[rsec-ta] loaded");
    Ok(())
}

#[ta_open_session]
fn open_session(_params: &mut Parameters) -> Result<()> {
    // The only client is our kernel module, which logs in as
    // TEE_LOGIN_REE_KERNEL. We deliberately do NOT check the login
    // type here: doing so would make the TA unusable from a plain
    // libteec client during bring-up, and the kernel module is not
    // a security boundary we are relying on anyway - the key never
    // leaves this side regardless of who asks.
    Ok(())
}

#[ta_close_session]
fn close_session() {}

#[ta_destroy]
fn destroy() {
    trace_println!("[rsec-ta] unloaded");
}

#[ta_invoke_command]
fn invoke_command(cmd_id: u32, params: &mut Parameters) -> Result<()> {
    match cmd_id {
        TA_CMD_GET_PUBKEY => cmd_get_pubkey(params),
        TA_CMD_SIGN       => cmd_sign(params),
        TA_CMD_COUNTER    => cmd_counter(params),
        _ => Err(Error::new(ErrorKind::NotSupported)),
```

```
    }
}

// ---------------------------------------------------------------------
// The signing key
// ---------------------------------------------------------------------

/// Open the persistent keypair, creating it on first use.
///
/// This is the heart of the project. `TransientObject::generate_key`
/// runs inside S-EL0; the private scalar is written into secure
/// heap and then into a persistent object that OP-TEE encrypts
/// with a key derived from the hardware unique key (chapter 15).
/// At no point does it exist in a page the normal world can map.
fn open_or_create_key() -> Result<PersistentObject> {
    // Fast path: it already exists.
    match PersistentObject::open(
        ObjectStorageConstants::Private,
        KEY_OBJECT_ID,
        DataFlag::ACCESS_READ,
    ) {
        Ok(obj) => return Ok(obj),
        Err(e) if e.kind() == ErrorKind::ItemNotFound => {}
        Err(e) => return Err(e),
    }

    trace_println!("[rsec-ta] generating a new P-256 keypair");

    let mut key = TransientObject::allocate(
        TransientObjectType::EcdsaKeypair,
        P256_BITS as usize,
    )?;

    // The curve is an ATTRIBUTE of the generation, not of the type.
    // Forget this and you get TEE_ERROR_NOT_SUPPORTED with no clue
    // as to why.
    let curve = Attribute::from_value(
        AttributeId::EccCurve,
        optee_utee_sys::TEE_ECC_CURVE_NIST_P256,
        0,
    );
    key.generate_key(P256_BITS as usize, &[curve])?;

    // Persist the OBJECT. Note we do not serialise the private
    // scalar into a buffer and write bytes - that would put it in
    // TA memory in the clear, briefly, for no reason.
    let obj = PersistentObject::create(
        ObjectStorageConstants::Private,
        KEY_OBJECT_ID,
        DataFlag::ACCESS_READ | DataFlag::ACCESS_WRITE_META,
        Some(&mut key),
        &[],
    )?;

    Ok(obj)
}

// ---------------------------------------------------------------------
// TA_CMD_GET_PUBKEY   params: [0] = memref out, 64 bytes
// ---------------------------------------------------------------------
fn cmd_get_pubkey(params: &mut Parameters) -> Result<()> {
    let mut p0 = unsafe { params.0.as_memref()? };
    let out = p0.buffer();
```

```
    if out.len() < PUBKEY_LEN {
        return Err(Error::new(ErrorKind::ShortBuffer));
    }

    let obj = open_or_create_key()?;

    // X and Y may come back SHORTER than 32 bytes when the value
    // has leading zeros - roughly one time in 256 per coordinate.
    // Left-pad into a fixed 64-byte field so the caller always
    // sees the same layout. Getting this wrong produces a key
    // that works 99% of the time, which is the worst kind of bug.
    let mut x = [0u8; 32];
    let mut y = [0u8; 32];

    let xlen = obj.ref_attribute(AttributeId::EccPublicValueX, &mut x)?;
    let ylen = obj.ref_attribute(AttributeId::EccPublicValueY, &mut y)?;

    let mut pub_raw = [0u8; PUBKEY_LEN];
    pub_raw[32 - xlen..32].copy_from_slice(&x[..xlen]);
    pub_raw[64 - ylen..64].copy_from_slice(&y[..ylen]);

    out[..PUBKEY_LEN].copy_from_slice(&pub_raw);
    p0.set_updated_size(PUBKEY_LEN);
    Ok(())
}

// ---------------------------------------------------------------------
// TA_CMD_SIGN   params: [0] = memref in  (32-byte digest)
//                       [1] = memref out (64-byte signature)
// ---------------------------------------------------------------------
fn cmd_sign(params: &mut Parameters) -> Result<()> {
    let mut p0 = unsafe { params.0.as_memref()? };
    let mut p1 = unsafe { params.1.as_memref()? };

    let digest_in = p0.buffer();
    let out       = p1.buffer();

    // Exactly 32 bytes or nothing. A fixed size means there is no
    // length arithmetic anywhere in this function to get wrong.
    if digest_in.len() != DIGEST_LEN {
        return Err(Error::new(ErrorKind::BadParameters));
    }
    if out.len() < SIG_LEN {
        return Err(Error::new(ErrorKind::ShortBuffer));
    }

    // ONE fetch out of shared memory into TA memory, then work on
    // the copy (chapter 20). The normal world can rewrite the
    // shared buffer at any instant, including between two reads
    // of the same byte.
    let mut digest = [0u8; DIGEST_LEN];
    digest.copy_from_slice(digest_in);

    let key = open_or_create_key()?;

    let mut op = Asymmetric::allocate(
        AlgorithmId::EcdsaP256,
        OperationMode::Sign,
        P256_BITS as usize,
    )?;
    op.set_key(&key)?;
```

```
    let mut sig = [0u8; SIG_LEN];
    let n = op.sign_digest(&[], &digest, &mut sig)?;

    // Write the output ONCE, at the end, on success. A TA that
    // writes partial results into shared memory and then fails
    // has handed the caller something that looks like an answer.
    out[..n].copy_from_slice(&sig[..n]);
    p1.set_updated_size(n);
    Ok(())
}

// ---------------------------------------------------------------
// TA_CMD_COUNTER   params: [0] = value in/out
//                            a = 1 to increment first
//                            b = (out) the counter value
// ---------------------------------------------------------------
fn cmd_counter(params: &mut Parameters) -> Result<()> {
    let mut p0 = unsafe { params.0.as_value()? };
    let do_increment = p0.a() == 1;

    let mut value: u64 = 0;

    match PersistentObject::open(
        ObjectStorageConstants::Private,
        COUNTER_OBJECT_ID,
        DataFlag::ACCESS_READ | DataFlag::ACCESS_WRITE,
    ) {
        Ok(mut obj) => {
            let mut buf = [0u8; 8];
            let n = obj.read(&mut buf)?;
            if n as usize == buf.len() {
                value = u64::from_le_bytes(buf);
            }
            if do_increment {
                value = value.saturating_add(1);
                obj.seek(0, optee_utee::Whence::DataSeekSet)?;
                obj.write(&value.to_le_bytes())?;
            }
        }
        Err(e) if e.kind() == ErrorKind::ItemNotFound => {
            if do_increment {
                value = 1;
            }
            PersistentObject::create(
                ObjectStorageConstants::Private,
                COUNTER_OBJECT_ID,
                DataFlag::ACCESS_READ
                    | DataFlag::ACCESS_WRITE
                    | DataFlag::ACCESS_WRITE_META,
                None,
                &value.to_le_bytes(),
            )?;
        }
        Err(e) => return Err(e),
    }

    // Two 32-bit halves, because a GP value parameter is a pair of
    // u32 and we want a u64 (chapter 12).
    p0.set_a((value & 0xffff_ffff) as u32);
    p0.set_b((value >> 32) as u32);
    Ok(())
}
```

```
// ------------------------------------------------------------------
// The TA header. These constants become the .ta file's metadata.
// ------------------------------------------------------------------
const TA_FLAGS: u32 = optee_utee_sys::TA_FLAG_SINGLE_INSTANCE
    | optee_utee_sys::TA_FLAG_INSTANCE_KEEP_ALIVE
    | optee_utee_sys::TA_FLAG_DEVICE_ENUM;   // <-- see below
const TA_STACK_SIZE: u32 = 4 * 1024;
const TA_DATA_SIZE:  u32 = 32 * 1024;
const TA_VERSION:     &[u8] = b"0.1\0";
const TA_DESCRIPTION: &[u8] = b"rsec device-bound signing service\0";
const EXT_PROP_VALUE_1: &[u8] = b"rsec TA\0";
const EXT_PROP_VALUE_2: u32 = 0x0010;
const TRACE_LEVEL: i32 = 4;
const TRACE_EXT_PREFIX: &[u8] = b"TA\0";
const TA_FRAMEWORK_STACK_SIZE: u32 = 2048;

include!(concat!(env!("OUT_DIR"), "/user_ta_header.rs"));
```

> **`TA_FLAG_DEVICE_ENUM` IS NOT OPTIONAL HERE**
> Without it the TA is invisible to the Linux TEE bus, no device appears under `/sys/bus/tee/devices/`, and the kernel module's `probe` is never called — silently. The module loads, prints nothing, and creates no device node. This is the single most common way this project fails, and it fails in the one layer where there is no error message to read. See chapter 13.

```
// ta/build.rs
fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Generates user_ta_header.rs from the constants above and
    // emits the linker script the TA dev kit needs.
    optee_utee_build::TaConfig::new_default_with_cargo_manifest()?
        .build()?;
    Ok(())
}
```

The kernel driver, in Rust

Rust for Linux gives you a misc device, a mutex and safe user-copy helpers. It does **not** give you the TEE client API — there is no `kernel::tee` module upstream. So the first thing this driver does is declare the C functions itself.

```
// rsec/src/tee_ffi.rs
//
// Hand-written bindings to drivers/tee's in-kernel client API.
// Everything here is EXPORT_SYMBOL_GPL, so a GPL module may call
// it. See include/linux/tee_drv.h for the authority.
//
// The types come from bindgen. For that to work you must add
//
//     #include <linux/tee_drv.h>
//
// to rust/bindings/bindings_helper.h in your kernel tree, or these
// `bindings::` paths will not resolve.

use core::ffi::c_void;
use kernel::bindings;

pub type MatchFn = unsafe extern "C" fn(
    *mut bindings::tee_ioctl_version_data,
    *const c_void,
) -> core::ffi::c_int;

extern "C" {
    pub fn tee_client_open_context(
        start: *mut bindings::tee_context,
        match_fn: MatchFn,
        data: *const c_void,
        vers: *mut bindings::tee_ioctl_version_data,
    ) -> *mut bindings::tee_context;

    pub fn tee_client_close_context(ctx: *mut bindings::tee_context);

    pub fn tee_client_open_session(
        ctx: *mut bindings::tee_context,
        arg: *mut bindings::tee_ioctl_open_session_arg,
        param: *mut bindings::tee_param,
    ) -> core::ffi::c_int;

    pub fn tee_client_close_session(
        ctx: *mut bindings::tee_context,
        session: u32,
    ) -> core::ffi::c_int;

    pub fn tee_client_invoke_func(
        ctx: *mut bindings::tee_context,
        arg: *mut bindings::tee_ioctl_invoke_arg,
        param: *mut bindings::tee_param,
    ) -> core::ffi::c_int;

    pub fn tee_shm_alloc_kernel_buf(
        ctx: *mut bindings::tee_context,
        size: usize,
    ) -> *mut bindings::tee_shm;

    pub fn tee_shm_free(shm: *mut bindings::tee_shm);

    pub fn tee_shm_get_va(
        shm: *mut bindings::tee_shm,
        offs: usize,
    ) -> *mut c_void;
}

/// Accept only OP-TEE, not AMD-TEE or any other back end.
///
```

```
/// # Safety
/// Called by the TEE core with a valid `ver` pointer.
pub unsafe extern "C" fn optee_match(
    ver: *mut bindings::tee_ioctl_version_data,
    _data: *const c_void,
) -> core::ffi::c_int {
    // SAFETY: the caller guarantees `ver` points at a live struct.
    let impl_id = unsafe { (*ver).impl_id };
    (impl_id == bindings::TEE_IMPL_ID_OPTEE) as core::ffi::c_int
}

/// Map a GlobalPlatform TEE_Result onto an errno.
///
/// Do this once, in one place. A driver that returns -EINVAL for
/// every TA failure is a driver nobody can debug from user space.
pub fn tee_result_to_errno(res: u32) -> kernel::error::Error {
    use kernel::error::code::*;
    match res {
        0x0000_0000 => EINVAL,        // caller should not be here
        0xFFFF_0004 => ENOSYS,        // NOT_SUPPORTED
        0xFFFF_0006 => EINVAL,        // BAD_PARAMETERS
        0xFFFF_000C => ENOSPC,        // SHORT_BUFFER
        0xFFFF_0008 => ENOENT,        // ITEM_NOT_FOUND
        0xFFFF_000F => EACCES,        // ACCESS_DENIED
        0xFFFF_3024 => ENOMEM,        // OUT_OF_MEMORY
        0xFFFF_3110 => ETIME,         // TARGET_DEAD (TA panicked)
        _           => EIO,
    }
}
```

```
// rsec/src/lib.rs
// SPDX-License-Identifier: GPL-2.0
//! A device-bound signing service backed by an OP-TEE trusted
//! application, exposed as /dev/rsec.

use core::ffi::c_void;
use kernel::prelude::*;
use kernel::{
    bindings,
    device::Device,
    miscdevice::{MiscDevice, MiscDeviceOptions, MiscDeviceRegistration},
    new_mutex,
    sync::Mutex,
    types::ARef,
    uaccess::{UserSlice, UserSliceReader, UserSliceWriter},
};

mod tee_ffi;
use tee_ffi::*;

use rsec_abi::*;

module! {
    type:        RsecModule,
    name:        "rsec",
    author:      "Embedded Learning Hub",
    description: "OP-TEE backed signing service",
    license:     "GPL",
}

/// Size of the one shared buffer we allocate and reuse.
/// 4 KB is one page and comfortably larger than 64 bytes; there is
/// no reason to be clever here.
const SHM_SIZE: usize = 4096;

// ---------------------------------------------------------------------
// The TEE session
//
// ONE session for the whole system, created at probe, guarded by a
// mutex. Every ioctl serialises through it.
//
// Why not a session per open() file? Because OP-TEE has a fixed
// number of threads (chapter 5). A hundred processes each holding a
// session will exhaust them, and the failure mode is the entire
// secure world wedging, not an error return.
// ---------------------------------------------------------------------
struct TeeSession {
    ctx:     *mut bindings::tee_context,
    shm:     *mut bindings::tee_shm,
    shm_va:  *mut u8,
    session: u32,
}

// SAFETY: the raw pointers are only ever touched while the enclosing
// Mutex is held, and they remain valid from probe() to remove().
unsafe impl Send for TeeSession {}

impl TeeSession {
    fn new(uuid: &[u8; 16]) -> Result<Self> {
        // SAFETY: passing a null `start` asks for the first matching
        // TEE; `optee_match` is a valid callback.
        let ctx = unsafe {
            tee_client_open_context(
```

```
                core::ptr::null_mut(),
                optee_match,
                core::ptr::null(),
                core::ptr::null_mut(),
            )
        };
        // The TEE core returns an ERR_PTR, not null, on failure.
        if kernel::error::from_err_ptr(ctx).is_err() {
            // -ENOENT here means the TEE is not up YET, which during
            // boot is normal. Returning EPROBE_DEFER lets the driver
            // core retry us later.
            pr_info!("rsec: no OP-TEE context yet, deferring\n");
            return Err(EPROBE_DEFER);
        }

        let mut arg = bindings::tee_ioctl_open_session_arg::default();
        arg.uuid.copy_from_slice(uuid);
        arg.clnt_login = bindings::TEE_IOCTL_LOGIN_REE_KERNEL;
        arg.num_params = 0;

        // SAFETY: ctx is valid; no parameters, so a null param array.
        let rc = unsafe {
            tee_client_open_session(ctx, &mut arg, core::ptr::null_mut())
        };
        if rc < 0 || arg.ret != 0 {
            pr_err!("rsec: open_session rc={} ret=0x{:08x}\n", rc, arg.ret);
            // SAFETY: ctx is valid and we are giving it up.
            unsafe { tee_client_close_context(ctx) };
            return Err(EINVAL);
        }

        // SAFETY: ctx is valid.
        let shm = unsafe { tee_shm_alloc_kernel_buf(ctx, SHM_SIZE) };
        if kernel::error::from_err_ptr(shm).is_err() {
            // SAFETY: both handles are valid and we are releasing them.
            unsafe {
                tee_client_close_session(ctx, arg.session);
                tee_client_close_context(ctx);
            }
            return Err(ENOMEM);
        }

        // SAFETY: shm is valid, offset 0 is in range.
        let shm_va = unsafe { tee_shm_get_va(shm, 0) } as *mut u8;

        Ok(Self { ctx, shm, shm_va, session: arg.session })
    }

    /// Build a memref parameter pointing into our shared buffer.
    fn memref(&self, attr: u32, offs: usize, size: usize)
        -> bindings::tee_param
    {
        let mut p = bindings::tee_param::default();
        p.attr = attr as u64;
        // SAFETY: writing the memref arm of the union. `tee_param`
        // is a plain C union of POD types; every arm is valid to
        // write and the TEE core reads the arm named by `attr`.
        unsafe {
            p.u.memref.shm      = self.shm;
            p.u.memref.shm_offs = offs as u64;
            p.u.memref.size     = size as u64;
        }
        p
```

```
    }

    fn invoke(&self, func: u32, params: &mut [bindings::tee_param])
        -> Result<()>
    {
        let mut arg = bindings::tee_ioctl_invoke_arg::default();
        arg.func       = func;
        arg.session    = self.session;
        arg.num_params = params.len() as u32;

        // SAFETY: ctx and session are live for the driver's lifetime;
        // `params` is a valid array of `num_params` entries.
        let rc = unsafe {
            tee_client_invoke_func(self.ctx, &mut arg, params.as_mut_ptr())
        };
        if rc < 0 {
            return Err(kernel::error::Error::from_errno(rc));
        }
        if arg.ret != 0 {
            pr_warn!("rsec: TA cmd {} returned 0x{:08x}\n", func, arg.ret);
            return Err(tee_result_to_errno(arg.ret));
        }
        Ok(())
    }
}

impl Drop for TeeSession {
    fn drop(&mut self) {
        // Order matters and is the reverse of construction: buffers,
        // then session, then context. Freeing the context first
        // leaves the TA holding a reference to shared memory that
        // has gone away.
        // SAFETY: all three handles were created in `new` and are
        // still live; this runs once, at remove().
        unsafe {
            tee_shm_free(self.shm);
            tee_client_close_session(self.ctx, self.session);
            tee_client_close_context(self.ctx);
        }
    }
}

// ---------------------------------------------------------------
// The misc device
// ---------------------------------------------------------------

#[pin_data]
struct RsecDevice {
    #[pin]
    tee: Mutex<TeeSession>,
}

#[vtable]
impl MiscDevice for RsecDevice {
    type Ptr = Arc<RsecDevice>;

    fn open(_file: &kernel::fs::File, misc: &MiscDeviceRegistration<Self>)
        -> Result<Arc<RsecDevice>>
    {
        // Every open() hands back a reference to the SAME device, and
        // therefore the same TEE session. There is no per-open state.
        Ok(misc.device_data().clone())
    }
```

```
    fn ioctl(dev: ArcBorrow<'_, RsecDevice>, _file: &kernel::fs::File,
             cmd: u32, arg: usize) -> Result<isize>
    {
        match cmd {
            RSEC_IOC_PUBKEY  => dev.ioctl_pubkey(arg),
            RSEC_IOC_SIGN    => dev.ioctl_sign(arg),
            RSEC_IOC_COUNTER => dev.ioctl_counter(arg),
            // ENOTTY, not EINVAL: it is what user space expects from
            // an ioctl it does not recognise, and what `strace`
            // prints intelligibly.
            _ => Err(ENOTTY),
        }
    }
}

impl RsecDevice {
    // ---------------------------------------------------------------
    // RSEC_IOC_PUBKEY
    // ---------------------------------------------------------------
    fn ioctl_pubkey(&self, arg: usize) -> Result<isize> {
        // 1. Copy the request struct in. UserSlice does the
        //    access_ok + copy_from_user, and will not let us
        //    accidentally dereference a user pointer.
        let mut req = RsecPubkey::default();
        let mut reader = UserSlice::new(
            arg as *mut c_void,
            core::mem::size_of::<RsecPubkey>(),
        ).reader();
        reader.read_raw(unsafe {
            core::slice::from_raw_parts_mut(
                &mut req as *mut _ as *mut u8,
                core::mem::size_of::<RsecPubkey>(),
            )
        })?;

        // 2. Validate BEFORE doing anything. Version first.
        if req.version != RSEC_ABI_VERSION {
            return Err(EINVAL);
        }
        if (req.len as usize) < PUBKEY_LEN {
            return Err(ENOSPC);
        }

        // 3. Talk to the TA under the lock.
        let mut key = [0u8; PUBKEY_LEN];
        {
            let tee = self.tee.lock();
            let mut params = [
                tee.memref(
                    bindings::TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_OUTPUT,
                    0, PUBKEY_LEN),
                bindings::tee_param::default(),
                bindings::tee_param::default(),
                bindings::tee_param::default(),
            ];
            tee.invoke(TA_CMD_GET_PUBKEY, &mut params)?;

            // SAFETY: the TA wrote at most PUBKEY_LEN bytes at
            // offset 0 of a buffer we allocated with SHM_SIZE >
            // PUBKEY_LEN, and shm_va is a valid kernel mapping.
            unsafe {
                core::ptr::copy_nonoverlapping(
```

```
                    tee.shm_va, key.as_mut_ptr(), PUBKEY_LEN);
            }
        }   // lock released here, before we touch user memory

        // 4. Copy out. NOTE we are no longer holding the mutex:
        //    copy_to_user can fault and sleep, and sleeping with a
        //    lock held that every other caller needs is how you turn
        //    a slow disk into a system-wide stall.
        let mut writer = UserSlice::new(
            req.buf as *mut c_void, PUBKEY_LEN).writer();
        writer.write_slice(&key)?;

        Ok(0)
    }

    // ------------------------------------------------------------
    // RSEC_IOC_SIGN
    // ------------------------------------------------------------
    fn ioctl_sign(&self, arg: usize) -> Result<isize> {
        let mut req = RsecSign {
            version: 0, sig_len: 0, digest: [0u8; 32], sig: 0,
        };
        let mut reader = UserSlice::new(
            arg as *mut c_void,
            core::mem::size_of::<RsecSign>(),
        ).reader();
        reader.read_raw(unsafe {
            core::slice::from_raw_parts_mut(
                &mut req as *mut _ as *mut u8,
                core::mem::size_of::<RsecSign>(),
            )
        })?;

        if req.version != RSEC_ABI_VERSION {
            return Err(EINVAL);
        }
        if (req.sig_len as usize) < SIG_LEN {
            return Err(ENOSPC);
        }

        let mut sig = [0u8; SIG_LEN];
        {
            let tee = self.tee.lock();

            // Digest into shared memory at offset 0,
            // signature comes back at offset 64. One buffer, two
            // regions, addressed with partial memrefs.
            // SAFETY: both regions are inside the SHM_SIZE buffer.
            unsafe {
                core::ptr::copy_nonoverlapping(
                    req.digest.as_ptr(), tee.shm_va, DIGEST_LEN);
            }

            let mut params = [
                tee.memref(
                    bindings::TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INPUT,
                    0, DIGEST_LEN),
                tee.memref(
                    bindings::TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_OUTPUT,
                    64, SIG_LEN),
                bindings::tee_param::default(),
                bindings::tee_param::default(),
            ];
```

```
            tee.invoke(TA_CMD_SIGN, &mut params)?;

            // SAFETY: offset 64 + SIG_LEN is within SHM_SIZE.
            unsafe {
                core::ptr::copy_nonoverlapping(
                    tee.shm_va.add(64), sig.as_mut_ptr(), SIG_LEN);
            }
        }

        let mut writer = UserSlice::new(
            req.sig as *mut c_void, SIG_LEN).writer();
        writer.write_slice(&sig)?;

        Ok(0)
    }

    // ---------------------------------------------------------------
    // RSEC_IOC_COUNTER
    // ---------------------------------------------------------------
    fn ioctl_counter(&self, arg: usize) -> Result<isize> {
        let mut req = RsecCounter::default();
        let slice = UserSlice::new(
            arg as *mut c_void,
            core::mem::size_of::<RsecCounter>(),
        );
        let mut reader = slice.reader();
        reader.read_raw(unsafe {
            core::slice::from_raw_parts_mut(
                &mut req as *mut _ as *mut u8,
                core::mem::size_of::<RsecCounter>(),
            )
        })?;

        if req.version != RSEC_ABI_VERSION {
            return Err(EINVAL);
        }

        let value: u64;
        {
            let tee = self.tee.lock();
            let mut p0 = bindings::tee_param::default();
            p0.attr = bindings::TEE_IOCTL_PARAM_ATTR_TYPE_VALUE_INOUT as u64;
            // SAFETY: writing the value arm of a POD union.
            unsafe {
                p0.u.value.a = if req.increment != 0 { 1 } else { 0 };
                p0.u.value.b = 0;
            }

            let mut params = [
                p0,
                bindings::tee_param::default(),
                bindings::tee_param::default(),
                bindings::tee_param::default(),
            ];
            tee.invoke(TA_CMD_COUNTER, &mut params)?;

            // SAFETY: the TA wrote the value arm, which is what we
            // asked for with VALUE_INOUT.
            let (lo, hi) = unsafe {
                (params[0].u.value.a, params[0].u.value.b)
            };
            value = (lo as u64) | ((hi as u64) << 32);
        }
```

```
        req.value = value;
        let mut writer = UserSlice::new(
            arg as *mut c_void,
            core::mem::size_of::<RsecCounter>(),
        ).writer();
        writer.write_slice(unsafe {
            core::slice::from_raw_parts(
                &req as *const _ as *const u8,
                core::mem::size_of::<RsecCounter>(),
            )
        })?;

        Ok(0)
    }
}

// ---------------------------------------------------------------------
// Module init / exit
// ---------------------------------------------------------------------

struct RsecModule {
    _misc: Pin<KBox<MiscDeviceRegistration<RsecDevice>>>,
}

/// Parse the UUID string from the shared crate into the 16 bytes
/// the TEE core wants, in big-endian "export_uuid" order.
fn parse_uuid(s: &str) -> Result<[u8; 16]> {
    let mut out = [0u8; 16];
    let mut i = 0;
    let bytes = s.as_bytes();
    let mut k = 0;
    while k < bytes.len() {
        if bytes[k] == b'-' { k += 1; continue; }
        if i >= 16 || k + 1 >= bytes.len() { return Err(EINVAL); }
        let hi = (bytes[k] as char).to_digit(16).ok_or(EINVAL)? as u8;
        let lo = (bytes[k + 1] as char).to_digit(16).ok_or(EINVAL)? as u8;
        out[i] = (hi << 4) | lo;
        i += 1;
        k += 2;
    }
    if i != 16 { return Err(EINVAL); }
    Ok(out)
}

impl kernel::Module for RsecModule {
    fn init(_module: &'static ThisModule) -> Result<Self> {
        pr_info!("rsec: initialising\n");

        let uuid = parse_uuid(RSEC_TA_UUID)?;
        let session = TeeSession::new(&uuid)?;

        let dev = Arc::pin_init(
            try_pin_init!(RsecDevice {
                tee <- new_mutex!(session, "rsec::tee"),
            }),
            GFP_KERNEL,
        )?;

        let options = MiscDeviceOptions { name: c_str!("rsec") };
        let misc = KBox::pin_init(
            MiscDeviceRegistration::register(options, dev),
            GFP_KERNEL,
```

```
        )?;

        pr_info!("rsec: /dev/rsec ready\n");
        Ok(Self { _misc: misc })
    }
}
```

> **RUST-FOR-LINUX APIS MOVE**
> `MiscDevice`, `UserSlice` and the `pin_init` machinery have all changed shape between 6.8 and 6.13, and will change again. The *structure* above — one session, one mutex, copy in, validate, invoke, copy out with the lock dropped — is stable; the exact trait signatures are not. Check `rust/kernel/miscdevice.rs` in the tree you are building against and adjust. This is the current price of admission for Rust in the kernel, and it is worth saying out loud rather than discovering.

The user-mode driver, in Rust with OpenSSL

```
# host/Cargo.toml
[package]
name    = "rsecctl"
version = "0.1.0"
edition = "2021"

[dependencies]
libc     = "0.2"
openssl  = "0.10"
rsec-abi = { path = "../crates/rsec-abi", features = ["std"] }

# Cross-compiling openssl: either point at buildroot's copy
#   export OPENSSL_DIR=$HOME/optee/out-br/host/aarch64-buildroot-linux-gnu/sysroot/usr
#   export PKG_CONFIG_ALLOW_CROSS=1
# or build it from source into the binary:
#   openssl = { version = "0.10", features = ["vendored"] }
```

```
// host/src/lib.rs  -  the user-mode driver proper
//
// This is the layer that turns "an ioctl on a file descriptor"
// into an API a program would want to use. Keeping it separate
// from the CLI is what makes it a driver rather than a script.

use std::fs::{File, OpenOptions};
use std::io;
use std::os::unix::io::AsRawFd;

use rsec_abi::*;

pub struct Rsec {
    f: File,
}

impl Rsec {
    pub fn open() -> io::Result<Self> {
        let f = OpenOptions::new()
            .read(true)
            .write(true)
            .open("/dev/rsec")?;
        Ok(Self { f })
    }

    /// # Safety contract we uphold for every ioctl below
    /// `arg` points at a live, correctly-typed struct for `req`,
    /// and the kernel only reads/writes within its declared size.
    unsafe fn ioctl(&self, req: u32, arg: *mut libc::c_void)
        -> io::Result<()>
    {
        let rc = unsafe {
            libc::ioctl(self.f.as_raw_fd(), req as libc::c_ulong, arg)
        };
        if rc < 0 {
            return Err(io::Error::last_os_error());
        }
        Ok(())
    }

    pub fn public_key(&self) -> io::Result<[u8; PUBKEY_LEN]> {
        let mut out = [0u8; PUBKEY_LEN];
        let mut req = RsecPubkey {
            version: RSEC_ABI_VERSION,
            len:     PUBKEY_LEN as u32,
            buf:     out.as_mut_ptr() as u64,
        };
        unsafe {
            self.ioctl(RSEC_IOC_PUBKEY,
                       &mut req as *mut _ as *mut libc::c_void)?
        };
        Ok(out)
    }

    pub fn sign(&self, digest: &[u8; DIGEST_LEN])
        -> io::Result<[u8; SIG_LEN]>
    {
        let mut sig = [0u8; SIG_LEN];
        let mut req = RsecSign {
            version: RSEC_ABI_VERSION,
            sig_len: SIG_LEN as u32,
            digest:  *digest,
            sig:     sig.as_mut_ptr() as u64,
```

```
        };
        unsafe {
            self.ioctl(RSEC_IOC_SIGN,
                       &mut req as *mut _ as *mut libc::c_void)?
        };
        Ok(sig)
    }

    pub fn counter(&self, increment: bool) -> io::Result<u64> {
        let mut req = RsecCounter {
            version:   RSEC_ABI_VERSION,
            increment: increment as u32,
            value:     0,
        };
        unsafe {
            self.ioctl(RSEC_IOC_COUNTER,
                       &mut req as *mut _ as *mut libc::c_void)?
        };
        Ok(req.value)
    }
}
```

```
// host/src/crypto.rs  -  everything OpenSSL does
//
// Note what is here: hashing, format conversion, verification.
// All public-key or public-data operations. Nothing secret ever
// reaches this file.

use openssl::bn::{BigNum, BigNumContext};
use openssl::ec::{EcGroup, EcKey, EcPoint};
use openssl::ecdsa::EcdsaSig;
use openssl::error::ErrorStack;
use openssl::nid::Nid;
use openssl::pkey::Public;
use openssl::sha::Sha256;

use std::fs::File;
use std::io::Read;

/// SHA-256 a file in 64 KB chunks. Bulk data stays in the normal
/// world - only the 32-byte result crosses into the TEE.
pub fn sha256_file(path: &str) -> std::io::Result<[u8; 32]> {
    let mut f = File::open(path)?;
    let mut h = Sha256::new();
    let mut buf = vec![0u8; 64 * 1024];
    loop {
        let n = f.read(&mut buf)?;
        if n == 0 { break; }
        h.update(&buf[..n]);
    }
    Ok(h.finish())
}

/// Turn the TA's raw 64-byte X||Y into an OpenSSL EC public key.
///
/// The 0x04 prefix marks an UNCOMPRESSED SEC1 point. Forget it and
/// `EcPoint::from_bytes` fails with an error message that does not
/// mention prefixes.
pub fn public_key_from_raw(raw: &[u8; 64])
    -> Result<EcKey<Public>, ErrorStack>
{
    let group = EcGroup::from_curve_name(Nid::X9_62_PRIME256V1)?;
    let mut ctx = BigNumContext::new()?;

    let mut sec1 = Vec::with_capacity(65);
    sec1.push(0x04);
    sec1.extend_from_slice(raw);

    let point = EcPoint::from_bytes(&group, &sec1, &mut ctx)?;
    EcKey::from_public_key(&group, &point)
}

/// The TA emits raw r||s, fixed width. OpenSSL, X.509, TLS and CMS
/// all want an ASN.1 SEQUENCE of two INTEGERs. Converting is four
/// lines; forgetting to is the commonest reason a perfectly correct
/// TA signature "does not verify".
pub fn sig_from_raw(raw: &[u8; 64]) -> Result<EcdsaSig, ErrorStack> {
    let r = BigNum::from_slice(&raw[..32])?;
    let s = BigNum::from_slice(&raw[32..])?;
    // Badly named: it builds a signature from its (r, s) components.
    EcdsaSig::from_private_components(r, s)
}

pub fn verify(digest: &[u8; 32], raw_sig: &[u8; 64],
              key: &EcKey<Public>) -> Result<bool, ErrorStack>
```

```
{
    sig_from_raw(raw_sig)?.verify(digest, key)
}

pub fn sig_to_der(raw: &[u8; 64]) -> Result<Vec<u8>, ErrorStack> {
    sig_from_raw(raw)?.to_der()
}

pub fn key_to_pem(key: &EcKey<Public>) -> Result<Vec<u8>, ErrorStack> {
    key.public_key_to_pem()
}
```

```
// host/src/main.rs  -  the CLI
use std::process::ExitCode;

mod crypto;
use rsecctl::Rsec;          // src/lib.rs
use crypto::*;

fn hex(b: &[u8]) -> String {
    b.iter().map(|x| format!("{:02x}", x)).collect()
}

fn unhex(s: &str) -> Option<Vec<u8>> {
    if s.len() % 2 != 0 { return None; }
    (0..s.len() / 2)
        .map(|i| u8::from_str_radix(&s[2 * i..2 * i + 2], 16).ok())
        .collect()
}

fn run() -> Result<(), Box<dyn std::error::Error>> {
    let args: Vec<String> = std::env::args().collect();
    let dev = Rsec::open()?;

    match args.get(1).map(String::as_str) {
        Some("pubkey") => {
            let raw = dev.public_key()?;
            println!("raw : 04{}", hex(&raw));
            let key = public_key_from_raw(&raw)?;
            print!("{}", String::from_utf8_lossy(&key_to_pem(&key)?));
        }

        Some("sign") if args.len() == 3 => {
            let d = sha256_file(&args[2])?;
            let sig = dev.sign(&d)?;
            println!("{}", hex(&sig));
        }

        Some("verify") if args.len() == 4 => {
            let d = sha256_file(&args[2])?;
            let raw = unhex(&args[3]).ok_or("bad signature hex")?;
            if raw.len() != 64 { return Err("signature must be 64 bytes".into()); }
            let mut sig = [0u8; 64];
            sig.copy_from_slice(&raw);

            let key = public_key_from_raw(&dev.public_key()?)?;
            // THE PROOF: OpenSSL, which knows nothing about OP-TEE,
            // checks a signature the secure world produced.
            println!("{}", if verify(&d, &sig, &key)? { "VALID" } else { "INVALID" });
        }

        Some("der") if args.len() == 3 => {
            let d = sha256_file(&args[2])?;
            let sig = dev.sign(&d)?;
            std::io::Write::write_all(
                &mut std::io::stdout(), &sig_to_der(&sig)?)?;
        }

        Some("counter") => {
            let inc = args.get(2).map(String::as_str) == Some("inc");
            println!("{}", dev.counter(inc)?);
        }

        _ => {
            eprintln!("usage: rsecctl pubkey");
```

```
            eprintln!("       rsecctl sign   <file>");
            eprintln!("       rsecctl verify <file> <sighex>");
            eprintln!("       rsecctl der    <file>   # DER to stdout");
            eprintln!("       rsecctl counter [inc]");
            return Err("bad usage".into());
        }
    }
    Ok(())
}

fn main() -> ExitCode {
    match run() {
        Ok(()) => ExitCode::SUCCESS,
        Err(e) => { eprintln!("rsecctl: {e}"); ExitCode::FAILURE }
    }
}
```

Building all three

```
# 0. The OP-TEE QEMU tree, as in chapter 4.
$ mkdir -p ~/optee && cd ~/optee
$ repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
$ repo sync -j8
$ cd build && make toolchains -j4

# 1. Rust toolchain for both targets.
$ rustup target add aarch64-unknown-linux-gnu     # host binary
$ rustup toolchain install nightly                # the TA SDK wants it
$ rustup component add rust-src --toolchain nightly

# 2. The TA.
$ git clone https://github.com/apache/incubator-teaclave-trustzone-sdk.git
$ cd incubator-teaclave-trustzone-sdk
$ ./setup.sh
$ source environment          # sets OPTEE_DIR, TA_DEV_KIT_DIR, ...

$ cd ~/rsec/ta
$ make TARGET_TA=aarch64-unknown-linux-gnu
$ ls target/*/release/*.ta
7f2e5c10-9a41-4b8d-b3c6-1e0d5a72f491.ta

# 3. The kernel module. This is the fiddly one.
#    Rust modules need a kernel built with CONFIG_RUST=y, which the
#    OP-TEE manifest's kernel is usually NOT. Check first:
$ cd ~/optee/linux
$ make LLVM=1 ARCH=arm64 rustavailable
Rust is available!            # if not, it tells you exactly what is missing

$ ./scripts/config --enable RUST --enable SAMPLES_RUST
$ ./scripts/config --enable TEE  --enable OPTEE
$ make LLVM=1 ARCH=arm64 olddefconfig
$ make LLVM=1 ARCH=arm64 -j$(nproc) Image modules

#    Add linux/tee_drv.h to the bindings so `bindings::tee_context`
#    and friends exist on the Rust side:
$ grep -q tee_drv rust/bindings/bindings_helper.h || \
    sed -i '1i #include <linux/tee_drv.h>' rust/bindings/bindings_helper.h

#    Then build the module in-tree (out-of-tree Rust modules work but
#    are still rough):
$ cp -r ~/rsec/rsec drivers/misc/rsec
$ echo 'obj-m += rsec/' >> drivers/misc/Makefile
$ make LLVM=1 ARCH=arm64 M=drivers/misc/rsec modules

# 4. The user-mode driver.
$ cd ~/rsec/host
$ export OPENSSL_DIR=~/optee/out-br/host/aarch64-buildroot-linux-gnu/sysroot/usr
$ export PKG_CONFIG_ALLOW_CROSS=1
$ cargo build --release --target aarch64-unknown-linux-gnu

# 5. Stage everything into the image.
$ cp ~/rsec/ta/target/*/release/*.ta   ~/optee/out-br/target/lib/optee_armtz/
$ cp ~/optee/linux/drivers/misc/rsec/rsec.ko ~/optee/out-br/target/root/
$ cp ~/rsec/host/target/aarch64-unknown-linux-gnu/release/rsecctl \
     ~/optee/out-br/target/usr/bin/
$ cd ~/optee/build && make buildroot run-only
```

Running it on QEMU

```
# Three windows appear: the QEMU monitor, "Normal World" and
# "Secure World". In the monitor:
(qemu) c

# Normal world, log in as root:
# insmod rsec.ko
[  12.481203] rsec: initialising
[  12.494771] rsec: /dev/rsec ready

# Secure world console, at the same moment:
D/TC:? ldelf_init_with_ldelf:... loading TA 7f2e5c10-...
I/TA: [rsec-ta] loaded

# ls -l /dev/rsec
crw-------    1 root     root       10, 122 /dev/rsec

# First public key request generates the keypair:
# rsecctl pubkey
raw : 04a91f...c3
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEqR8...
-----END PUBLIC KEY-----

# Secure world:
I/TA: [rsec-ta] generating a new P-256 keypair

# Sign something:
# echo "hello from the normal world" > /tmp/msg
# rsecctl sign /tmp/msg
4b7d2e...91

# Verify it, through OpenSSL:
# rsecctl verify /tmp/msg 4b7d2e...91
VALID

# Tamper and re-verify:
# echo "hello from the normal world!" > /tmp/msg
# rsecctl verify /tmp/msg 4b7d2e...91
INVALID

# The counter, which proves persistent storage works:
# rsecctl counter
0
# rsecctl counter inc
1
# rsecctl counter inc
2
# reboot, then:
# rsecctl counter
2                       # survived, because it is in secure storage
```

Proving it independently

```
# Everything above trusted our own code to verify our own code.
# Take the artefacts off the machine and check them with stock
# OpenSSL, which has never heard of OP-TEE.

# On the guest:
# rsecctl pubkey | sed -n '/BEGIN/,/END/p' > /tmp/pub.pem
# rsecctl der /tmp/msg > /tmp/sig.der

# Copy /tmp/pub.pem, /tmp/sig.der and /tmp/msg to the host, then:
$ openssl ec -pubin -in pub.pem -text -noout
Public-Key: (256 bit)
pub:
    04:a9:1f:...
ASN1 OID: prime256v1
NIST CURVE: P-256

$ openssl dgst -sha256 -verify pub.pem -signature sig.der msg
Verified OK

# THAT is the result the whole project exists to produce.
#
# A signature that verifies under a public key, over a digest of a
# file, where the corresponding private key was created inside the
# secure world, has never been present in any page the normal world
# could map, and cannot be extracted by root, by the kernel module,
# or by a kernel exploit.
#
# And the request travelled: Rust user program -> Rust kernel module
# -> SMC -> TF-A -> OP-TEE -> Rust trusted application, and back.
```

Checking the claim rather than assuming it

```
# The claim is "the private key is not reachable from the normal
# world". Test it, do not trust it.

# 1. It is not in the module's memory - the module never sees it.
#    Everything the driver copies is 32 or 64 bytes of digest or
#    signature. Grep the source for a private key: there is none.

# 2. Secure storage is opaque from the normal world.
# ls -l /data/tee/
-rw-------    1 root  root  128  0
# xxd /data/tee/0 | head -2
00000000: 8f3a 2c91 e5d7 ...     # AES-GCM ciphertext, keyed from
00000010: 1b44 90ff 7c22 ...     # the HUK (chapter 15)

# 3. The secure DRAM carve-out is fenced. On QEMU:
# devmem2 0x0e100000 w
Bus error                        # correct - see chapter 24

# 4. The TA is single-instance and keep-alive, so its heap - where
#    the loaded key object lives - is never handed back to the
#    normal world's allocator.
# cat /sys/kernel/debug/tee/... 2>/dev/null

# What this does NOT prove, and you should be clear about it:
#   * on QEMU the HUK is a fixed test value, so secure storage is
#     not device-bound (chapter 15). On real silicon it is.
#   * a compromised OP-TEE core, or a TA with a memory bug, still
#     reaches the key. The boundary is only as good as what runs
#     inside it (chapter 20).
#   * physical attacks are out of scope entirely.
```

If you cannot enable CONFIG_RUST

Rust kernel modules need a recent kernel, a matching `rustc`, `bindgen`, and patience. If the kernel you must ship on does not have it, the ABI in `rsec-abi` is language-neutral — a C module implementing the same three ioctls keeps the Rust TA and the Rust user-mode driver working unchanged.

```
/* rsec_shim.c - the same driver in C, if you need it.
   Abridged: the ioctl bodies mirror the Rust ones exactly.     */

static long rsec_ioctl(struct file *f, unsigned int cmd,
                       unsigned long arg)
{
        struct rsec_dev *d = f->private_data;
        int rc;

        switch (cmd) {
        case RSEC_IOC_SIGN: {
                struct rsec_sign req;
                u8 sig[64];

                if (copy_from_user(&req, (void __user *)arg,
                                   sizeof req))
                        return -EFAULT;
                if (req.version != RSEC_ABI_VERSION)
                        return -EINVAL;
                if (req.sig_len < 64)
                        return -ENOSPC;

                mutex_lock(&d->lock);
                memcpy(d->shm_va, req.digest, 32);
                rc = rsec_invoke(d, TA_CMD_SIGN, 32, 64);
                if (!rc)
                        memcpy(sig, d->shm_va + 64, 64);
                mutex_unlock(&d->lock);
                if (rc)
                        return rc;

                if (copy_to_user((void __user *)req.sig, sig, 64))
                        return -EFAULT;
                return 0;
        }
        ...
        default:
                return -ENOTTY;
        }
}

/* The point worth taking: because the ABI lives in a crate that
   generates the same numbers a C header would, swapping the middle
   layer's language changes nothing above or below it. That is what
   a frozen ABI is FOR.                                          */
```

Extensions worth doing

Exercise	What it teaches
Return DER from the TA instead of raw r‖s	ASN.1 inside a TA, and why you keep the encoder tiny
Register the driver as a Linux `hwrng` fed by `TEE_GenerateRandom`	Why the kernel wants a TEE at all — `/dev/hwrng` then works with no new user-space code
Add a `crypto_alg` registration so `dm-crypt` can use the key	The kernel-as-consumer case in full
Sign the counter value along with the digest	Replay resistance (chapter 17)
Emit a self-signed X.509 certificate with OpenSSL over the TA's key	How device attestation actually ships

Exercise	What it teaches
Add `compat_ioctl` and run a 32-bit ARM binary against it	Why the ABI uses `u64` for pointers
Hammer it with 64 concurrent `rsecctl sign` processes	The mutex, and OP-TEE's thread limit (chapter 5)
Run the hostile client from chapter 20 against the TA	Whether the validation is real
Remove the module while a signature is in flight	Reference counting, and why `Drop` order matters

How it shows up

Symptom	Cause
`insmod` succeeds, no `/dev/rsec`, no messages	TA missing `TA_FLAG_DEVICE_ENUM`, so no TEE device appeared and `probe` never ran
`rsec: no OP-TEE context yet, deferring` for ever	OP-TEE driver not loaded, or `CONFIG_OPTEE` off — check `dmesg | grep optee`
`open_session ret=0xFFFF0008`	`ITEM_NOT_FOUND`: the `.ta` is not in `/lib/optee_armtz`, or its filename does not match the UUID
`open_session` hangs	`tee-supplicant` not running; loading a TA needs it (chapter 9)
`ioctl: Inappropriate ioctl for device` (ENOTTY)	Struct size changed on one side only — the size is encoded in the ioctl number, which is the design working
`ioctl: Invalid argument` immediately	Version mismatch, or the TA's `paramTypes` disagree with what the module sent
Signature verifies in `rsecctl`, fails in `openssl dgst`	Raw r‖s not converted to DER
Public key rejected by OpenSSL	Missing the `0x04` prefix, or X/Y not left-padded to 32 bytes
Verification fails roughly 1 signature in 128	Exactly the left-padding bug — a coordinate with a leading zero byte
System stalls under concurrent load	Mutex held across `copy_to_user`, which can fault and sleep
Kernel oops on `rmmod`	`Drop` order — shared memory freed after the context, or a session still in use
TA panics, then every call returns `ETIME`	`TEE_ERROR_TARGET_DEAD`; the session must be torn down and reopened
Key changes after reflashing the rootfs	Secure storage lives in `/data/tee`, which the new image replaced (chapter 15)
Works on QEMU, key unreadable on a second board	Correct — the HUK differs per device

Check yourself

- ✓ Draw the request path through all four domains and name the mechanism that crosses each boundary.
- ✓ Give four reasons to put a kernel driver between user space and a TA, and one reason not to.
- ✓ Explain why OpenSSL does the hashing and verification while the TA does only the signing.
- ✓ Explain why a TA cannot realistically link OpenSSL.
- ✓ State the four ioctl ABI rules and the failure each one prevents.
- ✓ Explain why pointers in the ioctl structs are `u64`.
- ✓ Say what `TA_FLAG_DEVICE_ENUM` enables, and describe exactly how it fails without it.
- ✓ Point to the single-fetch copy, the `paramTypes` check and the fixed-size digest in the TA source.

- ✓ Explain the left-padding of X, Y, r and s, and the bug you get without it.
- ✓ Explain why the mutex is dropped before `copy_to_user`.
- ✓ Justify one shared TEE session rather than one per `open()`.
- ✓ Give the correct teardown order in `Drop` and say what breaks if you reverse it.
- ✓ Verify a TA-produced signature with stock `openssl dgst` and explain what that proves — and what it does not.

TRACK

Cryptography

From entropy to TLS to constant-time code — the cryptography a firmware and platform engineer is expected to know.

21 chapters

What cryptography guarantees

Four properties, and the precise threat model each defends. Most cryptographic failures in real systems come not from broken algorithms but from expecting a property the primitive never provided — so this chapter is about being exact.

The four goals

```
CONFIDENTIALITY   an adversary cannot learn the content
                  -> encryption

INTEGRITY         an adversary cannot modify the message
                  undetected
                  -> MACs, signatures

AUTHENTICITY      the message came from who it claims
                  -> MACs (shared key), signatures
                     (public key)

NON-REPUDIATION   the sender cannot later deny it
                  -> signatures ONLY, never MACs

AND THE ONES CRYPTOGRAPHY DOES NOT PROVIDE:
  availability      - a jammer, a flood, a cut cable
  traffic analysis  - who talked to whom, when, how much
  endpoint security - a compromised device signs whatever
                      it is told to
  correctness of the plaintext
  protection against a user who chooses to leak it

THE DISTINCTION THAT MATTERS MOST IN PRACTICE:

  A MAC proves the message came from someone holding the
  shared key. Since BOTH parties hold it, EITHER could
  have produced it - so a MAC cannot prove to a third
  party who sent it.

  A SIGNATURE proves it came from the holder of the
  private key, and only they hold it - so a third party
  can verify it, and the signer cannot deny it.

Choosing a MAC where non-repudiation was needed, or a
signature where a MAC would have been faster, is a design
error that no amount of correct implementation fixes.
```

Encryption alone is not enough

```
/* THE SINGLE MOST IMPORTANT RULE IN THIS TRACK:

     ENCRYPTION WITHOUT AUTHENTICATION IS BROKEN.

   Confidentiality does not imply integrity. An attacker who
   cannot READ your ciphertext can very often MODIFY it in
   ways that produce a predictable change in the plaintext.  */

/* --- THE BIT-FLIP ATTACK on a stream cipher --- */
        plaintext:   "TRANSFER 0100 TO ACCOUNT 12345"
        keystream:   k0 k1 k2 ...
        ciphertext:  p XOR k

/* The attacker does not know the key. But XOR is linear, so
   flipping bit i of the ciphertext flips bit i of the
   plaintext. They change "0100" to "9100" by XORing the
   right positions with a known delta - WITHOUT DECRYPTING
   ANYTHING.

   This applies to every stream cipher, and to CTR, OFB and
   CFB modes (chapter 05).

/* --- THE PADDING ORACLE on CBC --- */
/* If the receiver distinguishes "bad padding" from "bad
   MAC" - by an error message, a different response, or
   merely a TIMING DIFFERENCE - an attacker can decrypt the
   entire message by submitting modified ciphertexts and
   observing which are accepted. About 128 queries per byte.

   This broke TLS (Lucky 13), ASP.NET, XML encryption, and
   many devices. It is why the answer is not "fix the error
   messages" but "use authenticated encryption".

/* THE FIX, and it is the same in every case:

     USE AN AEAD (chapter 09): AES-GCM, ChaCha20-Poly1305,
     AES-CCM. They encrypt and authenticate together, in a
     construction that has been analysed as a whole.

   NEVER: encrypt then hope
   NEVER: encrypt-and-MAC or MAC-then-encrypt hand-rolled
   ALWAYS: an AEAD, or if you must compose, ENCRYPT-THEN-MAC
           over the ciphertext AND the IV AND the associated
           data.                                          */
```

> **THE RULE THAT PREVENTS MOST FAILURES**
>
> **Do not design cryptography. Do not implement primitives. Use a reviewed library, at the highest level of abstraction it offers.**
>
> Every chapter in this track explains how something works so you can reason about it, choose correctly, and read a security review. None of it is an invitation to write your own AES. The gap between "understanding the algorithm" and "implementing it without a side channel" is where the failures are (chapter 19).

Kerckhoffs's principle

```
/* THE SYSTEM MUST BE SECURE EVEN IF EVERYTHING ABOUT IT IS
   PUBLIC EXCEPT THE KEY.

   Stated in 1883, and still the property that separates
   cryptography from wishful thinking.

/* WHY: algorithms leak. They are reverse-engineered from
   firmware, extracted from silicon, leaked by employees,
   published by researchers. A system whose security depends
   on the algorithm staying secret has a single point of
   failure that cannot be rotated.

/* THE CASE STUDIES, all of them real:
     · A5/1 (GSM)         - reverse-engineered, then broken
     · CSS (DVD)          - reverse-engineered, 40-bit key
     · MIFARE Classic     - proprietary CRYPTO1, broken in
                            minutes
     · KeeLoq (car remotes) - broken once published
     · numerous vendor "encryption" schemes in IoT devices

   In every case the algorithm was weak, and secrecy delayed
   the discovery rather than preventing it.

/* THE CORRECT POSTURE:
     · use PUBLISHED, ANALYSED algorithms - AES, SHA-2,
       SHA-3, ChaCha20, Ed25519, ML-KEM
     · assume the attacker has your firmware and your
       schematics
     · put the secret in the KEY, and protect the key
       (chapter 18)
     · design so a compromised device does not compromise the
       fleet - per-device keys, not one key everywhere

/* AND THE COROLLARY that people miss: "we obfuscated it" is
   not a security control. It raises the cost of the first
   break and does nothing thereafter, because the result is
   published.                                              */
```

Threat models

```
/* A guarantee is meaningless without saying against WHOM.
   The standard adversary models, in increasing strength: */

   CIPHERTEXT-ONLY       sees ciphertexts
   KNOWN-PLAINTEXT       sees some plaintext/ciphertext pairs
   CHOSEN-PLAINTEXT      can get any plaintext encrypted
                         (CPA)
   CHOSEN-CIPHERTEXT     can also get chosen ciphertexts
                         DECRYPTED (CCA)
   ADAPTIVE CHOSEN-      can choose based on previous
   CIPHERTEXT (CCA2)     answers

/* MODERN SCHEMES TARGET CCA2, because that is what a network
   attacker actually has: they can send whatever they like to
   your server and observe the response.

   IND-CPA   "indistinguishable under chosen plaintext" - an
             attacker cannot tell which of two chosen
             plaintexts was encrypted. Requires
             RANDOMISATION: the same plaintext must not
             produce the same ciphertext, which is why every
             mode needs an IV or nonce.
   IND-CCA2  the same, with a decryption oracle available.
             AEADs provide this; unauthenticated modes do
             not.

/* AND THE PHYSICAL MODELS, which matter for embedded: */
   PASSIVE REMOTE        network observation only
   ACTIVE REMOTE         can inject and modify
   LOCAL SOFTWARE        code on the same machine - cache
                         timing (chapter 19)
   PHYSICAL, NON-        power and EM measurement
   INVASIVE
   PHYSICAL, INVASIVE    decapsulation, probing, laser fault
                         injection

/* THE EMBEDDED QUESTION: your threat model must state where
   on that scale you stop. "Resists a remote attacker" and
   "resists an attacker holding the device with an
   oscilloscope" are entirely different products
   (isolation chapter 16), and pretending otherwise is how
   security reviews go badly.                              */
```

Key sizes and security levels

Security level	Symmetric	RSA / DH	Elliptic curve	Hash
80 bits — **broken**	—	1024	160	SHA-1
112 bits — legacy only	3DES	2048	224	SHA-224
128 bits — the baseline	**AES-128**	3072	**256**	**SHA-256**
192 bits	AES-192	7680	384	SHA-384
256 bits	AES-256	15360	521	SHA-512

```
/* READ THE TABLE ROW-WISE: those entries are EQUIVALENT.
   AES-128, RSA-3072 and P-256 all give about 128 bits of
   security. Pairing AES-256 with RSA-2048 wastes the AES
   strength; the system is as strong as its weakest link.

/* WHAT "128 BITS" MEANS: roughly 2^128 operations to break.
   That is not merely large - it is beyond any conceivable
   physical computation. Counting to 2^128 at one operation
   per Planck time, using the entire energy output of the
   sun, does not finish.

/* SO KEY SEARCH IS NOT THE THREAT. Real breaks come from:
     · implementation bugs
     · side channels (chapter 19)
     · key management failures (chapter 18)
     · protocol errors
     · bad randomness (chapter 02)
     · using a broken primitive after it was broken

/* THE ASYMMETRIC SIZES ARE LARGER because the attacks are
   better than brute force: the number field sieve for
   RSA/DH factoring and discrete log, Pollard's rho for
   elliptic curves. ECC needs only 2n bits for n bits of
   security, which is why it dominates on embedded
   (chapter 14).

/* AND THE QUANTUM CAVEAT (chapter 20):
     Grover's algorithm halves symmetric security - AES-128
     becomes ~64 bits of quantum security, which is why
     AES-256 is recommended for long-term secrets.
     Shor's algorithm BREAKS RSA, DH and ECC entirely. The
     replacements are standardised (ML-KEM, ML-DSA) and
     deployment has begun.                                */
```

The primitives, and what each is for

Need	Use	Never
Encrypt a message	AES-GCM, ChaCha20-Poly1305, AES-CCM	ECB, unauthenticated CBC/CTR
Hash data	SHA-256, SHA-3, BLAKE2/3	MD5, SHA-1
Authenticate with a shared key	HMAC-SHA256, or the AEAD's tag	A hash of key‖message
Sign	Ed25519, ECDSA P-256, RSA-PSS	RSA PKCS#1 v1.5 for new designs
Key agreement	X25519, ECDH P-256	Unauthenticated DH
Derive keys from a key	HKDF	Truncating a hash
Hash a password	Argon2id, scrypt, bcrypt	Any plain hash, however salted
Random numbers	The OS CSPRNG, or a hardware TRNG	`rand()`, `time()`, a PRNG
Post-quantum KEM	ML-KEM (Kyber), hybrid with X25519	Anything unstandardised

```
/* THE LIBRARIES worth using, by environment:

   EMBEDDED / no OS
     mbedTLS          the standard for microcontrollers
     wolfSSL          small, certifiable, commercial
     TinyCrypt        minimal, NIST primitives only
     Monocypher       tiny, public domain, modern primitives
     RustCrypto       pure Rust, no_std, audited

   LINUX / POSIX
     libsodium        THE recommendation - a small, safe API
                      that is hard to misuse
     OpenSSL / BoringSSL   ubiquitous, large, easy to misuse
     the kernel crypto API   for in-kernel use

   HARDWARE
     the SoC's crypto accelerator, through the vendor driver
     a TPM or secure element, for key storage
     the TEE (isolation chapters 12-13)

/* PREFER THE HIGHEST-LEVEL API AVAILABLE:
     libsodium's crypto_secretbox()  rather than assembling
     AES-GCM from parts. The high-level API chose the nonce
     handling, the tag length and the composition for you,
     and those are exactly the choices that go wrong.     */
```

How it shows up

Symptom	Cause
Ciphertext modified undetectably	No authentication — use an AEAD
Plaintext recovered from error responses	A padding oracle — the same fix
Same plaintext gives the same ciphertext	No IV, reused IV, or ECB mode
Scheme broken once the firmware leaked	Security through obscurity — Kerckhoffs
One device compromised, whole fleet exposed	A shared key instead of per-device keys
AES-256 with RSA-1024	Mismatched security levels
A third party cannot verify a message's origin	A MAC was used where a signature was needed
Keys predictable across devices	Bad randomness at provisioning (ch. 02)
Passwords cracked from a stolen database	A plain hash instead of a password KDF
Key extracted with an oscilloscope	No side-channel countermeasures (ch. 19)
"We wrote our own so it is unique"	The most reliable predictor of a break

Check yourself

- ✓ Name the four goals and the primitive that provides each.
- ✓ Explain precisely why a MAC cannot give non-repudiation.
- ✓ List four things cryptography does not provide.
- ✓ Explain the bit-flip attack and why encryption alone is insufficient.
- ✓ Explain a padding oracle and why the fix is AEAD rather than better error messages.
- ✓ State Kerckhoffs's principle and give three systems that violated it.

- ✓ Name the adversary models and say which modern schemes target.
- ✓ Explain why every mode needs an IV or nonce.
- ✓ Read the equivalent-strength table and spot a mismatched pairing.
- ✓ Explain why key search is not the realistic threat.
- ✓ Choose the right primitive for nine common needs, and name what never to use.

Randomness and entropy

Every key, nonce, IV and salt depends on unpredictable numbers, and on an embedded device at first boot there is almost nothing unpredictable available. This is the most common single cause of real cryptographic failure in shipped products.

Three kinds of random

```
PRNG          deterministic from a seed. Fast, repeatable,
(non-crypto)  and PREDICTABLE. rand(), Mersenne Twister,
              xorshift, LCG.
              -> simulations, games, test data.
              -> NEVER for anything security-relevant.

CSPRNG        deterministic from a seed, but
              computationally infeasible to predict
              without it. ChaCha20-based, HMAC-DRBG,
              CTR-DRBG, AES-CTR.
              -> ALL cryptographic use.
              -> still needs a good seed.

TRNG          a physical entropy source: thermal noise, a
              ring oscillator's jitter, avalanche noise,
              radioactive decay.
              -> seeding the CSPRNG, and nothing else -
                 they are slow, and their raw output is
                 usually biased.

THE STANDARD ARCHITECTURE, and the one you should use:

     physical noise source (TRNG)
           |
     health tests, de-biasing (von Neumann, or a hash)
           |
     SEED (256 bits of real entropy)
           |
     CSPRNG  --> unlimited output, fast
           ^
           |
     periodic RESEEDING from the noise source

THE CSPRNG DOES THE WORK; the TRNG only ever provides the
seed. Reading a TRNG directly for every byte is slow and,
because raw output is biased, often worse.
```

How this fails in practice

```
/* --- 1. DEBIAN OpenSSL, 2006-2008 --- */
/* A developer removed a line that Valgrind complained about.
   It was the one mixing uninitialised memory into the pool.
   The seed collapsed to the process ID - 32,768 possible
   values. Every SSH and TLS key generated on Debian for two
   years was one of ~32,768 keys, enumerable in minutes.

/* --- 2. EMBEDDED DEVICES AT FIRST BOOT --- */
/* The 2012 "Mining Ps and Qs" study scanned the internet and
   found that 0.5% of TLS hosts and 1% of SSH hosts shared
   keys, or had keys sharing a prime factor - making them
   trivially factorable. The cause: routers and embedded
   devices generating keys at first boot, before any entropy
   existed.

   THE CONDITIONS THAT CAUSED IT, and every one of them is
   still common:
     · no hardware RNG
     · no persistent state from a previous boot
     · no user input
     · no network traffic yet
     · a real-time clock reading the same value on every
       device
     · the boot sequence identical every time

/* --- 3. THE PLAYSTATION 3 ECDSA KEY, 2010 --- */
/* Sony used the SAME nonce k for every signature. ECDSA
   leaks the private key immediately if k repeats
   (chapter 15). The master signing key was recovered from
   two signatures.

/* --- 4. ANDROID SecureRandom, 2013 --- */
/* An initialisation bug caused repeated nonces in Bitcoin
   wallet apps. The same ECDSA property applied. Funds were
   stolen.

/* THE PATTERN: in every case the ALGORITHM was fine. The
   randomness was not. That is where the failures are.   */
```

> **THE PROVISIONING RULE**
>
> An embedded device must **not** generate its long-term keys at first boot. Inject them during manufacturing, from a source with real entropy — an HSM on the production line — and burn the device-unique secret into fuses (isolation chapter 10). A device that generates its own identity before it has entropy is a device whose identity is predictable, and no amount of later reseeding fixes a key that has already been created.

Entropy sources

```
/* --- ON A HOSTED SYSTEM: use the OS. It is not optional to
       do better; it is impossible.                        */

    Linux    getrandom(buf, len, 0)     /* THE right call */
             /dev/urandom               /* the same pool  */
             /dev/random                /* identical since
                                           Linux 5.6 - the
                                           old blocking
                                           behaviour is
                                           gone           */
    Windows  BCryptGenRandom()
    macOS    getentropy(), SecRandomCopyBytes()
    POSIX    arc4random_buf()

/* getrandom() BLOCKS UNTIL THE POOL IS INITIALISED, which is
   exactly the behaviour you want at boot - and the reason
   some embedded systems hang at startup. The fix is to give
   the kernel entropy, not to stop blocking.

/* THE KERNEL'S SOURCES:
     interrupt timing jitter          the main one
     RDRAND / RNDR (ARMv8.5)          if available and
                                      trusted
     device-tree rng-seed             passed by the
                                      bootloader
     a stored seed file across reboots
     hardware RNG drivers (hwrng)

/* --- ON A MICROCONTROLLER --- */

    1. A HARDWARE TRNG, if the part has one. Most modern MCUs
       do: STM32 RNG, nRF RNG, ESP32 RNG, RP2040 ROSC.
       READ ITS DOCUMENTATION - some are only entropic when
       specific peripherals are active, and the ESP32's is
       only a true RNG when the radio is on.

    2. AN ADC ON A FLOATING INPUT, sampling the low bits. Some
       entropy, poorly characterised - use it only as one
       input among several, hashed.

    3. RING OSCILLATOR JITTER, if you have two independent
       oscillators. Sample one with the other.

    4. TIMING JITTER between an independent clock and the CPU.

    5. SRAM POWER-UP STATE. The uninitialised pattern is
       partly device-unique and partly random - useful as a
       PUF for device identity, weak as an entropy source.

/* AND ALWAYS: CONDITION THE OUTPUT. Never use a raw source
   directly: */
        uint8_t pool[64];
        collect_raw(pool, sizeof pool);   /* biased */
        sha256(pool, sizeof pool, seed);  /* 256 bits of
                                             conditioned
                                             output       */
        chacha20_seed(seed);              /* now use the
                                             CSPRNG       */
```

Using a CSPRNG

```
/* A ChaCha20-based CSPRNG - the modern standard construction,
   and what Linux, BSD and libsodium all use.             */

    struct Csprng {
        uint8_t key[32];
        uint64_t counter;
    };

    void csprng_seed(struct Csprng *r, const uint8_t seed[32])
    {
        memcpy(r->key, seed, 32);
        r->counter = 0;
    }

    void csprng_bytes(struct Csprng *r, uint8_t *out, size_t n)
    {
        uint8_t block[64];

        while (n) {
            chacha20_block(r->key, r->counter++, block);
            size_t take = n < 64 ? n : 64;
            memcpy(out, block, take);
            out += take; n -= take;
        }
        /* FAST KEY ERASURE: replace the key with fresh
           output, so a later compromise cannot recover
           earlier output. FORWARD SECRECY for the RNG.  */
        chacha20_block(r->key, r->counter++, block);
        memcpy(r->key, block, 32);
        secure_zero(block, sizeof block);
    }

/* THE FAST KEY ERASURE STEP is what makes this construction
   better than a plain counter mode: an attacker who reads
   the state cannot compute what you generated before.

/* THE OTHER STANDARD CONSTRUCTIONS (NIST SP 800-90A):
     HMAC-DRBG     conservative, easy to implement, slow
     CTR-DRBG      AES-CTR based; fast with an AES
                   accelerator
     Hash-DRBG
     (and Dual_EC_DRBG, which was withdrawn after evidence
      of a deliberate back door - a useful reminder that
      "standardised" is not the same as "trustworthy", and
      that public analysis is what matters)

/* RESEEDING: mix in fresh entropy periodically, and always
   after a fork, a restore from snapshot, or a VM clone -
   because all three duplicate the state, and a duplicated
   CSPRNG produces duplicated keys.                       */
```

Getting it right in code

```
/* --- ALWAYS CHECK THE RETURN VALUE --- */
    uint8_t key[32];
    if (getrandom(key, sizeof key, 0) != sizeof key)
            abort();            /* do NOT continue with a
                                   partly-filled buffer  */

/* A short read, a failed hardware RNG, or an uninitialised
   pool must be FATAL. Generating a key from a zero buffer is
   worse than not starting.

/* --- NEVER THESE --- */
    srand(time(NULL)); rand();          /* predictable */
    key = hash(serial_number);          /* not secret */
    key = hash(mac_address);            /* enumerable */
    key = hash(time());                 /* one second of
                                           entropy */
    for (i = 0; i < 32; i++)
            key[i] = adc_read() & 0xFF; /* biased, unconditioned */
    memcpy(key, uninitialised_buffer, 32);  /* UB, and not
                                               random */

/* --- NONCES AND IVs (chapter 05) --- */
/* THE REQUIREMENT DIFFERS BY MODE, and confusing them is a
   real bug:
     CBC     the IV must be UNPREDICTABLE (random)
     CTR/GCM the nonce must be UNIQUE, and need not be secret
             or random - a COUNTER is correct and often
             better, because a counter cannot collide
     A REPEATED GCM NONCE with the same key is catastrophic:
     it reveals the XOR of the plaintexts AND allows forgery
     of arbitrary messages.

/* SO FOR GCM, prefer a counter with persistent storage: */
    struct { uint32_t device_id; uint64_t counter; } nonce;
    nonce.counter = load_and_increment_persistent();
/* and if the counter can ever reset - a device that is
   reflashed, a state restore - rotate the key instead.

/* --- WIPE KEYS AFTER USE (compilers chapter 06) --- */
    explicit_bzero(key, sizeof key);        /* glibc, BSD */
    SecureZeroMemory(key, sizeof key);      /* Windows */
    sodium_memzero(key, sizeof key);
    /* plain memset is DELETED by the optimiser, because
       nothing reads the buffer afterwards */

/* --- AND FOR PASSWORDS, NEVER a plain hash (chapter 10) */
    argon2id(password, salt, ...);
```

Testing randomness

```
# --- STATISTICAL TEST SUITES ---
    dieharder -a -g 201 -f random.bin
    ent random.bin              # entropy, chi-square, mean
    NIST STS                    # SP 800-22
    PractRand                   # the most sensitive; catches
                                # what dieharder misses
    TestU01 BigCrush

# --- WHAT THEY CAN AND CANNOT TELL YOU ---
#   THEY DETECT: bias, short periods, correlations, stuck
#                bits, obviously broken sources
#   THEY DO NOT DETECT: a perfectly uniform but PREDICTABLE
#                sequence. AES-CTR with a key the attacker
#                knows passes every test.
#
#   SO: passing is necessary, not sufficient. It tells you
#   the source is not obviously broken; it says nothing about
#   whether it is secret.

# --- THE HEALTH TESTS THAT BELONG IN THE PRODUCT ---
    · REPETITION COUNT: fail if the same value repeats too
      often (SP 800-90B)
    · ADAPTIVE PROPORTION: fail if one value dominates a
      window
    · continuous self-test on every read from the hardware
      RNG
    · a startup test at boot, before generating anything
    · FAIL CLOSED: if the RNG fails its health test, refuse
      to operate. Do not fall back to a weaker source.

# --- ESTIMATING ENTROPY ---
#   min-entropy H∞ = -log2(max probability)
#   This is the conservative measure and the one that matters
#   for key generation - Shannon entropy overestimates when
#   the distribution is skewed.
#   A source producing 0 with probability 0.6 has 0.97 bits
#   of Shannon entropy and only 0.74 bits of min-entropy.
#   Collect enough raw bits that the CONDITIONED output has a
#   full 256 bits of min-entropy - typically several times
#   as many raw bits as you need.

# --- ON LINUX, CHECK THE POOL ---
    cat /proc/sys/kernel/random/entropy_avail
    dmesg | grep -i random
      random: crng init done      # this line is what you
                                  # need to see before
                                  # generating keys
```

The embedded checklist

Question	Right answer
Does the part have a TRNG?	Check the TRM, and check its *conditions* — some need a peripheral active
Where do long-term keys come from?	**Injected at manufacture**, not generated on device
Is there a device-unique secret in fuses?	There should be — it is the root of everything else
Is the RNG health-tested at boot?	Yes, and it must fail closed

Question	Right answer
Is a seed persisted across reboots?	Yes, in secure storage, and mixed with fresh entropy
Are GCM nonces counters or random?	Counters, persisted, with key rotation on reset
Are keys wiped after use?	With `explicit_bzero`, not `memset`
What happens if the RNG fails?	Refuse to operate — never a weaker fallback
Is every device's key different?	Yes — one compromise must not be fleet-wide
Has the source been characterised?	Min-entropy measured on real hardware, across temperature and voltage

How it shows up

Symptom	Cause
Two devices have the same key	Keys generated on device with no entropy
Keys share a prime factor	The same, during RSA generation
Boot hangs in `getrandom`	The pool is uninitialised — supply entropy, do not bypass it
Private key recovered from two signatures	A repeated ECDSA nonce
Ciphertexts reveal plaintext relationships	A repeated nonce in a stream cipher or CTR mode
GCM forgeries accepted	Nonce reuse — the authentication key is recoverable
Random values repeat after a VM clone	Duplicated CSPRNG state — reseed after cloning
Keys recoverable from a core dump	Not wiped, or wiped with `memset`
RNG passes dieharder but keys are guessable	Uniform but predictable — the tests cannot see it
Entropy quality varies with temperature	The source was characterised at one operating point

Check yourself

- ✓ Distinguish PRNG, CSPRNG and TRNG, and say what each is for.
- ✓ Draw the standard seed-and-expand architecture and explain why the TRNG only seeds.
- ✓ Describe two real failures and identify the common cause.
- ✓ List the conditions that make first-boot key generation predictable.
- ✓ State the provisioning rule and why later reseeding does not fix it.
- ✓ Name five entropy sources on a microcontroller and their weaknesses.
- ✓ Explain fast key erasure and what it protects.
- ✓ State the different requirements for a CBC IV and a GCM nonce.
- ✓ Explain why `memset` does not wipe a key.
- ✓ Say what statistical tests can and cannot detect.
- ✓ Explain min-entropy and why it is the conservative measure.
- ✓ Work the embedded checklist against a product you know.

The one-time pad and perfect secrecy

The only cipher that is provably unbreakable, why it is almost never usable, and what happens when the "one-time" part is violated. Understanding this fixes the intuition for every stream cipher that follows.

The construction

```
KEY: a truly random sequence, AS LONG AS THE MESSAGE,
     used exactly ONCE, known only to both parties.

ENCRYPT:  c = p XOR k
DECRYPT:  p = c XOR k

because  (p XOR k) XOR k = p

plaintext   0 1 1 0  1 0 0 1
key         1 1 0 1  0 1 1 0
ciphertext  1 0 1 1  1 1 1 1

THE CLAIM, proved by Shannon in 1949:

  For any ciphertext c and any candidate plaintext p' of
  the same length, there EXISTS a key k' = c XOR p' that
  produces it.

  Since every key is equally likely, every plaintext of
  that length is EQUALLY LIKELY given the ciphertext.

  The ciphertext therefore conveys ZERO information about
  the plaintext beyond its length.

PERFECT SECRECY: P(plaintext | ciphertext)
               = P(plaintext)

This holds against an adversary with UNLIMITED
computation — no future computer, no quantum algorithm,
no mathematical advance changes it. It is the only cipher
with that property.

AND THE COST OF IT: Shannon also proved that perfect
secrecy REQUIRES a key at least as long as the message.
That is not an artefact of this construction; it is a
lower bound. Which is why every practical cipher settles
for COMPUTATIONAL security instead.
```

Why it is unusable

```
/* THE FOUR REQUIREMENTS, and each one is a practical
   problem: */

   1. THE KEY MUST BE AS LONG AS THE MESSAGE.
      To send a gigabyte you need a gigabyte of key. But you
      must already have shared that key securely - and if you
      have a secure channel for a gigabyte of key, you could
      have sent the message on it.

   2. THE KEY MUST BE TRULY RANDOM.
      Not a PRNG (chapter 02). Real physical entropy, in the
      quantity above. Generating and storing that is itself a
      significant engineering problem.

   3. THE KEY MUST BE USED EXACTLY ONCE.
      See below - reuse is catastrophic, not merely
      weakening.

   4. THE KEY MUST BE DESTROYED AFTER USE.
      A stored key is a stored plaintext.

/* AND WHAT IT DOES NOT PROVIDE, which is the point people
   miss:

     NO INTEGRITY. XOR is linear (chapter 01), so flipping a
     ciphertext bit flips the plaintext bit. An attacker who
     knows the plaintext format can make arbitrary
     predictable changes without knowing the key at all.

     Perfect CONFIDENTIALITY, zero AUTHENTICITY. A one-time
     pad still needs a MAC.

/* WHERE IT IS ACTUALLY USED:
     · diplomatic and intelligence traffic, historically -
       the Moscow-Washington hotline used them
     · numbers stations broadcasting to agents with printed
       pads
     · quantum key distribution, where the physics provides
       the key material
   In every case the key distribution problem was solved by
   physical means - couriers, printed booklets, a fibre - and
   the volume was tiny.                                  */
```

> **THE LESSON THAT GENERALISES**
>
> Every stream cipher — RC4, ChaCha20, AES-CTR — is a one-time pad in which the long random key is replaced by a keystream generated from a short key. That substitution is what makes it practical, and it means every one of them inherits the one-time pad's properties: **keystream reuse is catastrophic**, and **there is no integrity**. Both facts follow directly from this chapter, and both are responsible for real breaks (chapter 06).

The two-time pad

```
/* Use the same key twice and perfect secrecy collapses
   completely. */

        c1 = p1 XOR k
        c2 = p2 XOR k

        c1 XOR c2 = (p1 XOR k) XOR (p2 XOR k)
                  = p1 XOR p2          /* THE KEY CANCELS */

/* The attacker now has the XOR of two plaintexts, with no
   key involved. That is enough:

     · if either plaintext is known, the other falls out
       immediately
     · if both are English text, CRIB DRAGGING recovers them:
       guess a common word at some offset, XOR it in, and see
       whether the result at that offset in the other message
       is plausible text. Iterate.
     · with several messages under the same keystream, it is
       nearly trivial - the statistical structure of natural
       language does the work
     · with structured data - a protocol with fixed headers -
       it is immediate

/* THE HISTORICAL CASE: the VENONA project. Soviet one-time
   pads were duplicated during wartime production pressure.
   US cryptanalysts exploited the reuse to read thousands of
   messages over four decades. The cipher was perfect; the
   procedure was not.

/* AND THE MODERN CASES, all the same mathematics:
     · WEP reused IVs after 2^24 packets, and often much
       sooner
     · MS-PPTP reused the RC4 keystream in both directions
     · TLS implementations that reused a GCM nonce
     · a device with a fixed IV compiled into the firmware -
       common in IoT

/* SO THE RULE, which applies to every stream cipher and
   every counter mode:

     NEVER REUSE A KEY-AND-NONCE PAIR. The nonce need not be
     secret or random, but it must be UNIQUE for the lifetime
     of the key (chapter 02).                              */
```

Computational security

```
/* Since perfect secrecy demands an impractical key, every
   real cipher settles for something weaker but sufficient. */

   PERFECT SECRECY
     an adversary with UNLIMITED computation learns nothing.
     Requires |key| >= |message|.

   COMPUTATIONAL SECURITY
     an adversary with BOUNDED computation - say 2^128
     operations - learns nothing useful.
     Allows a short key and an arbitrarily long message.

/* THE TRADE IS OVERWHELMINGLY WORTH IT: 2^128 operations is
   not merely impractical but physically impossible
   (chapter 01), and a 256-bit key encrypts a terabyte.

/* WHAT COMPUTATIONAL SECURITY RESTS ON:
     · that certain problems are hard - factoring, discrete
       log, or that AES behaves like a random permutation
     · NONE of these are PROVEN hard. They are believed hard
       because they have resisted concerted attack for
       decades.
     · P versus NP is unresolved; if P = NP, most of
       cryptography fails.

/* SO THE HONEST STATEMENT: modern cryptography is secure
   because nobody has found a way to break it, not because
   nobody can. That is a weaker claim than the one-time pad's
   and it is the best available for anything practical.

/* THE SEMANTIC SECURITY DEFINITION, which is the modern
   formalisation:

     An adversary chooses two plaintexts of equal length. One
     is encrypted at random. The adversary cannot do better
     than guessing which.

   This is IND-CPA (chapter 01), and it has a direct
   consequence: encryption MUST BE RANDOMISED. A
   deterministic cipher fails immediately, because the
   adversary encrypts one candidate themselves and compares.
   That is why every mode needs an IV or nonce, and why ECB
   (chapter 05) is broken.                                  */
```

Related "perfect" properties

```
/* Two other properties use the word, and neither is perfect
   secrecy. */

/* --- FORWARD SECRECY (sometimes "perfect forward
       secrecy") --- */
/* Compromising a long-term key does NOT reveal past session
   traffic, because each session used an EPHEMERAL key that
   was discarded.

     RSA key transport      NO forward secrecy - recover the
                            server's private key and every
                            recorded session decrypts
     ECDHE                  YES - the ephemeral keys are
                            discarded after the handshake

   THIS IS WHY TLS 1.3 REMOVED static RSA key exchange
   entirely (chapter 17). "Harvest now, decrypt later" is a
   real adversary model, and forward secrecy is the defence.

/* --- POST-COMPROMISE SECURITY (self-healing) --- */
/* After a compromise ENDS, security is RESTORED - because
   the parties ratchet to new keys. The Signal protocol's
   double ratchet provides it; TLS does not.

/* NEITHER IS PERFECT SECRECY. Both are computational
   properties about WHICH keys are exposed by a compromise,
   not about an unbounded adversary.

/* AND THE RELATED CONSTRUCTION worth knowing - SECRET
   SHARING, which does achieve information-theoretic
   security: */
        split a secret s into n shares
        any k shares reconstruct s
        any k-1 shares reveal NOTHING - information-
        theoretically

/* Shamir's scheme: pick a random degree-(k-1) polynomial
   with s as the constant term; the shares are points on it.
   k points determine the polynomial; k-1 leave every
   constant term equally likely - the same argument as the
   one-time pad.

   USED FOR: splitting a root key across several HSMs or
   people, so no one person can use it. Genuinely practical,
   unlike the one-time pad, because the secret is small.  */
```

How it shows up

Symptom	Cause
Two ciphertexts XOR to readable text	Keystream reuse — the two-time pad
Known plaintext reveals another message	The same
A fixed IV in the firmware	Every message uses the same keystream
Ciphertext modified undetectably	No integrity — XOR is linear (ch. 01)
Same plaintext gives same ciphertext	Deterministic encryption — fails IND-CPA

Symptom	Cause
Recorded traffic decrypted after a key leak	No forward secrecy — static key exchange
"We use a one-time pad" in a product	Almost certainly a stream cipher with a reused key
Nonce counter resets after a reflash	Reuse across the reset — rotate the key instead

Check yourself

- ✓ State the one-time pad construction and Shannon's perfect secrecy result.
- ✓ Explain why perfect secrecy requires a key as long as the message.
- ✓ List the four requirements and why each is impractical.
- ✓ Explain why the one-time pad provides no integrity.
- ✓ Derive the two-time pad attack and explain crib dragging.
- ✓ Name three modern systems broken by keystream reuse.
- ✓ State the nonce rule that follows, in the form that applies to AES-GCM.
- ✓ Contrast perfect with computational security and say what the latter rests on.
- ✓ Explain why semantic security requires randomised encryption.
- ✓ Distinguish forward secrecy and post-compromise security from perfect secrecy.
- ✓ Explain Shamir secret sharing and why it *is* information-theoretically secure.

Block ciphers and AES

A keyed permutation on fixed-size blocks — the workhorse primitive of symmetric cryptography. This chapter is how AES works, why its structure resists the known attacks, and what a hardware or software implementation actually costs.

What a block cipher is

```
E : {0,1}^k × {0,1}^n -> {0,1}^n

For each key K, E_K is a PERMUTATION of the n-bit blocks -
a bijection, so decryption exists and is unique.

THE SECURITY GOAL: E_K should be indistinguishable from a
RANDOM PERMUTATION chosen uniformly from all 2^n!
permutations, to an adversary who does not know K.

That is a strong requirement, and it is what the design
effort goes into. A block cipher is not "scrambling"; it
is an attempt to build something that behaves like a
randomly chosen lookup table of size 2^128, using a few
kilobytes of logic.

THE PARAMETERS THAT MATTER:

  BLOCK SIZE n     AES: 128 bits.
                   Too small is dangerous: with a 64-bit
                   block, a birthday collision appears
                   after ~2^32 blocks (32 GB), which is
                   the SWEET32 attack that killed 3DES
                   and Blowfish in TLS.
  KEY SIZE k       AES: 128, 192 or 256.

AND THE CRITICAL POINT: a block cipher encrypts ONE BLOCK.
Encrypting a message requires a MODE OF OPERATION
(chapter 05), and the mode is where most of the security
properties - and most of the mistakes - live.
```

Design principles

```
/* Shannon, 1949, and still the framework: */

   CONFUSION   the relationship between key and ciphertext
               must be complex.
               -> achieved by SUBSTITUTION (S-boxes),
                  which are non-linear.

   DIFFUSION   changing one plaintext bit must change about
               half the ciphertext bits.
               -> achieved by PERMUTATION and MIXING, which
                  spread local changes globally.

   A cipher applies both, alternately, for many ROUNDS. Each
   round is weak; the composition is not.

/* THE AVALANCHE CRITERION: flip one input bit, and each
   output bit should flip with probability 1/2. AES achieves
   full avalanche after two rounds - and then does eight more.

/* THE TWO STRUCTURES:

   SUBSTITUTION-PERMUTATION NETWORK (SPN) - AES
       every round transforms the WHOLE block:
           substitute -> permute -> mix -> add key
       decryption needs the INVERSE of each step.

   FEISTEL NETWORK - DES, Blowfish, Camellia
       split the block in half:
           L' = R
           R' = L XOR F(R, K)
       decryption is the SAME circuit with the round keys
       reversed - so F need not be invertible, which was
       valuable when gates were expensive.

/* THE ROUND COUNT is chosen with a SECURITY MARGIN: the best
   known attacks reach some number of rounds, and the cipher
   uses substantially more. AES-128 uses 10 rounds; the best
   published attack on reduced-round AES reaches 7. That
   margin is the design's insurance against future
   cryptanalysis.                                          */
```

AES

```
The state is a 4x4 matrix of BYTES - 128 bits - filled
COLUMN-WISE:

     b0  b4  b8  b12
     b1  b5  b9  b13
     b2  b6  b10 b14
     b3  b7  b11 b15

ROUNDS: 10 for AES-128, 12 for AES-192, 14 for AES-256.

ENCRYPTION:
     AddRoundKey(state, K0)
     for r = 1 .. Nr-1:
             SubBytes(state)
             ShiftRows(state)
             MixColumns(state)
             AddRoundKey(state, Kr)
     // the final round OMITS MixColumns
     SubBytes(state)
     ShiftRows(state)
     AddRoundKey(state, KNr)

THE FOUR OPERATIONS:

SubBytes     each byte replaced via a fixed 256-entry
             S-BOX. THE ONLY NON-LINEAR STEP, and
             therefore the whole source of confusion.
             Constructed as the multiplicative inverse in
             GF(2^8) followed by an affine map - chosen for
             provably good resistance to differential and
             linear cryptanalysis, not by trial and error.

ShiftRows    row 0 unchanged, row 1 left by 1, row 2 by 2,
             row 3 by 3. Spreads bytes ACROSS columns.

MixColumns   each column multiplied by a fixed matrix in
             GF(2^8). Every output byte of a column depends
             on all four inputs. Spreads WITHIN columns.
             ShiftRows and MixColumns together give full
             diffusion in two rounds.

AddRoundKey  XOR the round key. The only step involving
             the key.

THE KEY SCHEDULE expands the key into Nr+1 round keys,
using the S-box, a byte rotation and round constants -
deliberately non-linear so that recovering one round key
does not immediately give the others.

DECRYPTION uses InvSubBytes, InvShiftRows, InvMixColumns
with the round keys reversed - or, in the "equivalent
inverse cipher" form, the same structure with modified
keys, which is what implementations use.
```

WHY AES IS THE ANSWER

Selected in 2001 after a five-year open competition with fifteen candidates and worldwide public cryptanalysis. Twenty-five years later, the best attack on full AES-128 is a biclique attack costing about 2^{126} operations — a factor of four better than brute force, and entirely academic. Meanwhile every general-purpose CPU and most microcontrollers have hardware instructions for it. There is no reason to use anything else for block encryption, and choosing something else needs a specific justification.

Implementing it

```
/* --- 1. HARDWARE INSTRUCTIONS: always, if available --- */

    /* ARMv8 Cryptography Extension */
        aese  v0.16b, v1.16b        /* SubBytes + ShiftRows
                                       + AddRoundKey */
        aesmc v0.16b, v0.16b        /* MixColumns */
    /* two instructions per round; a full AES-128 block in
       about 10 cycles, and ~1 cycle/byte with pipelining
       across blocks.

    /* x86: AES-NI - aesenc, aesenclast, aeskeygenassist

    /* DETECT IT: */
        getauxval(AT_HWCAP) & HWCAP_AES         /* Linux ARM */
        __builtin_cpu_supports("aes")           /* x86 */
        ID_AA64ISAR0_EL1.AES                    /* bare metal */

/* HARDWARE AES IS ALSO CONSTANT TIME by construction - no
   table lookups, so no cache-timing side channel
   (chapter 19). That is at least as important as the speed.

/* --- 2. THE T-TABLE IMPLEMENTATION: fast, and UNSAFE --- */
    /* Precompute four 1 KB tables merging SubBytes,
       ShiftRows and MixColumns. One round becomes 16 table
       lookups and 16 XORs - very fast in software.       */
        s0 = T0[b0] ^ T1[b5] ^ T2[b10] ^ T3[b15] ^ rk[0];

/* AND IT LEAKS THE KEY. The table indices depend on the key,
   so an attacker sharing the cache observes which lines were
   touched and recovers the key - demonstrated repeatedly,
   including across VMs and from JavaScript. NEVER USE
   T-TABLES for anything an attacker can co-locate with.

/* --- 3. BITSLICED: constant time in software --- */
    /* Represent the state across registers so that one
       "bit" of each of N blocks lives in one register bit,
       and compute the S-box as a Boolean circuit - no
       lookups at all. Constant time by construction,
       and fast when processing several blocks in parallel
       (which CTR and GCM modes do).

    /* This is what a good software AES uses when the
       hardware instructions are absent: BearSSL's
       aes_ct64, or the RustCrypto soft backend.

/* --- 4. THE NAIVE S-BOX TABLE: small, and still leaky --- */
        state[i] = sbox[state[i]];
/* A single 256-byte table is four cache lines - fewer to
   distinguish, but still key-dependent. Acceptable ONLY on a
   microcontroller with no cache and no co-located attacker,
   and that assumption should be written down.

/* --- USING A LIBRARY, which is the actual advice --- */
    #include <mbedtls/aes.h>

    mbedtls_aes_context ctx;
    mbedtls_aes_init(&ctx);
    mbedtls_aes_setkey_enc(&ctx, key, 256);
    mbedtls_aes_crypt_ecb(&ctx, MBEDTLS_AES_ENCRYPT, in, out);
    mbedtls_aes_free(&ctx);
```

```
/* And note that raw ECB is exposed only because modes are
   built on it - you should be calling the AEAD interface
   (chapter 09), not this.                                */
```

The cryptanalysis

```
/* Understanding what these attacks look for explains why AES
   is shaped as it is. */

   DIFFERENTIAL CRYPTANALYSIS
     Track how a chosen INPUT DIFFERENCE propagates to an
     output difference. If some difference pair occurs with
     probability much higher than 2^-n, that bias reveals key
     bits.
     -> AES's S-box has a maximum differential probability of
        2^-6, which is the best achievable for an 8-bit
        S-box, and the wide-trail design bounds how few
        S-boxes an attack can involve.

   LINEAR CRYPTANALYSIS
     Find a linear approximation relating plaintext,
     ciphertext and key bits that holds slightly more often
     than half the time.
     -> AES's S-box has maximum linear bias 2^-3, again
        optimal.

   RELATED-KEY ATTACKS
     Exploit relationships between keys. AES-256's key
     schedule is weaker here than AES-128's - there is a
     related-key attack on full AES-256 at 2^99. It requires
     the attacker to obtain encryptions under keys with a
     chosen relationship, which a sane protocol never
     provides, so it is not a practical concern - but it is
     why "AES-256 is stronger than AES-128" is not
     unqualifiedly true.

   BICLIQUE ATTACK (2011)
     The best known attack on FULL AES: 2^126.1 for AES-128.
     A factor of four better than exhaustive search, and
     entirely theoretical.

   THE PRACTICAL BREAKS ARE ALL IMPLEMENTATION:
     cache timing on T-tables
     power analysis (DPA) on unprotected hardware
     fault injection (isolation chapter 16)
     nonce and IV misuse (chapter 05)
     key management failures (chapter 18)

/* WHICH IS THE THEME OF THIS WHOLE TRACK: the mathematics
   holds; the engineering is where things break.         */
```

The other block ciphers

Cipher	Block / Key	Status
AES	128 / 128-256	**The default.** Hardware everywhere
DES	64 / 56	**Broken** — 56-bit key, brute-forced since 1998
3DES	64 / 112	Deprecated — 64-bit block, SWEET32

Cipher	Block / Key	Status
Blowfish	64 / 32-448	Legacy — 64-bit block. `bcrypt` uses its key schedule
Twofish, Serpent	128 / 128-256	AES finalists, sound, unaccelerated
Camellia	128 / 128-256	Sound; standardised in Japan and in TLS
ARIA, SM4	128 / 128	National standards (Korea, China)
PRESENT, GIFT	64 / 80-128	Lightweight — tiny hardware, 64-bit block
ASCON	—	**NIST lightweight standard (2023)** — a permutation-based AEAD, not a block cipher
SPECK, SIMON	various	NSA-designed; rejected by ISO amid distrust

```
/* THE PRACTICAL SELECTION:

   Have AES hardware?                  -> AES-GCM
   No AES hardware, software only?     -> ChaCha20-Poly1305
                                          (chapter 06) - it
                                          is faster than
                                          software AES and
                                          constant time by
                                          design
   Extremely constrained hardware?     -> ASCON, the NIST
                                          lightweight
                                          standard
   Regulatory requirement for a
   national standard?                  -> whatever it
                                          mandates

   AND NEVER: DES, 3DES, RC4, or anything a vendor invented.
   The 64-bit block ciphers are excluded not because the
   ciphers are broken but because 64 bits is too small
   (SWEET32) - a reminder that block size is a security
   parameter, not just a performance one.                */
```

How it shows up

Symptom	Cause
Identical plaintext blocks give identical ciphertext	ECB mode — the cipher is working, the mode is wrong (ch. 05)
Key recovered by a co-located process	T-table cache timing — use hardware AES or bitsliced
Key recovered with an oscilloscope	Power analysis (ch. 19)
Collisions after ~32 GB	A 64-bit block cipher — SWEET32
AES much slower than expected	Hardware instructions not detected or not used
Encryption works, decryption gives garbage	Key schedule direction, or endianness of the block
Interoperability failure with another implementation	Byte order — AES fills the state column-wise from a byte string
"We use military-grade AES-256"	Says nothing about the mode, the nonce handling, or the key management

Check yourself

- ✓ Define a block cipher and state its security goal.
- ✓ Explain why block size is a security parameter and what SWEET32 exploits.
- ✓ Explain confusion and diffusion and which AES step provides each.

- ✓ Contrast an SPN with a Feistel network and say why Feistel was historically attractive.
- ✓ Name AES's four round operations and what each does.
- ✓ Say why the final round omits MixColumns.
- ✓ Explain what a security margin is and give AES's.
- ✓ Give four AES implementation strategies and their side-channel properties.
- ✓ Explain why T-tables leak and what to use instead.
- ✓ Describe differential and linear cryptanalysis and how AES resists them.
- ✓ Choose a cipher for four different constraints, and name what never to use.

Modes of operation

A block cipher encrypts 16 bytes. A mode turns it into something that encrypts a message — and the choice of mode determines almost every security property that matters. This is where most real encryption failures live.

ECB — never

```
ELECTRONIC CODEBOOK: encrypt each block independently.

     C_i = E_K(P_i)

THE FAILURE IS IMMEDIATE: identical plaintext blocks give
identical ciphertext blocks. The ciphertext preserves the
structure of the plaintext.

THE CANONICAL DEMONSTRATION is the "ECB penguin" - a
bitmap encrypted with ECB in which the image is still
plainly visible, because regions of uniform colour become
regions of uniform ciphertext.

WHAT THIS LEAKS IN PRACTICE:
  · repeated records in a database
  · structure in a protocol with fixed fields
  · whether two encrypted values are equal - which is
    enough to break a great many things
  · block boundaries and message structure

AND IT FAILS IND-CPA (chapter 03): encryption is
deterministic, so an adversary distinguishes two chosen
plaintexts trivially.

ECB IS ALSO MALLEABLE: an attacker can reorder, duplicate
or delete whole blocks, and the result decrypts.

THE ONLY LEGITIMATE USE: encrypting a single block of
random data - a key being wrapped, for instance - and even
there, use a proper key-wrap construction (RFC 3394)
instead.

IF YOU SEE ECB IN A CODEBASE, it is a finding.
```

CBC

```
/* CIPHER BLOCK CHAINING: XOR each plaintext block with the
   previous ciphertext block before encrypting.           */

        C_0 = E_K(P_0 XOR IV)
        C_i = E_K(P_i XOR C_{i-1})

        P_0 = D_K(C_0) XOR IV
        P_i = D_K(C_i) XOR C_{i-1}

/* PROPERTIES:
     + identical plaintext gives different ciphertext,
       provided the IV differs
     + a ciphertext error affects only two plaintext blocks
     - ENCRYPTION IS SEQUENTIAL - no parallelism
     + decryption IS parallelisable
     - requires PADDING to a block multiple
     - NO AUTHENTICATION

/* THE IV REQUIREMENT: it must be UNPREDICTABLE, not merely
   unique. A predictable IV enables a chosen-plaintext attack
   in which the adversary confirms a guessed plaintext block
   - which is exactly the BEAST attack on TLS 1.0, where the
   IV for each record was the previous record's last
   ciphertext block.

   SO: generate the IV with a CSPRNG (chapter 02), and
   transmit it in the clear alongside the ciphertext.

/* PADDING - PKCS#7, and it is where CBC goes wrong: */
        add N bytes each of value N, where 1 <= N <= 16
        a full block gets a WHOLE EXTRA BLOCK of padding

        "HELLO" -> 48 45 4C 4C 4F 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B

/* THE PADDING ORACLE (chapter 01): if the receiver
   distinguishes "bad padding" from "bad MAC" - through an
   error code, a response, or a TIMING DIFFERENCE - the
   attacker decrypts the whole message with about 128
   queries per byte. This broke TLS (Lucky 13), ASP.NET,
   Ruby on Rails, XML encryption and countless devices.

   THE FIX IS NOT constant-time padding checks - people
   tried, and Lucky 13 defeated them through timing anyway.
   THE FIX IS AEAD (chapter 09).

/* IF YOU MUST USE CBC: encrypt-then-MAC, with the MAC
   covering the IV and the ciphertext, and verify the MAC
   BEFORE attempting to decrypt or unpad. Then a forged
   message never reaches the padding check at all.       */
```

> **ENCRYPT-THEN-MAC, AND ONLY THAT ORDER**
>
> **Encrypt-then-MAC** — MAC over the ciphertext and IV. Provably secure, and a forgery is rejected before any decryption happens. **MAC-then-encrypt** (what TLS 1.0–1.2 did) requires decrypting before you can check, which is exactly what enables padding oracles. **Encrypt-and-MAC** (SSH) can leak plaintext information through the MAC. Use an AEAD and the question does not arise.

CTR

```
/* COUNTER MODE: turn the block cipher into a stream cipher
   by encrypting a counter and XORing the result.         */

        C_i = P_i XOR E_K(nonce ‖ counter_i)

/* PROPERTIES:
     + FULLY PARALLELISABLE, both directions
     + RANDOM ACCESS - decrypt block 1000 without the
       preceding blocks. Essential for disk and flash
       encryption.
     + NO PADDING - the keystream is truncated to the message
       length
     + only the ENCRYPT direction of the cipher is needed,
       which halves the code and table size on a
       microcontroller
     - NO AUTHENTICATION
     - CATASTROPHIC on nonce reuse

/* THE COUNTER BLOCK, typically:
        [ 96-bit nonce ][ 32-bit counter ]
   giving 2^32 blocks = 64 GB per nonce, which is the limit
   before the counter wraps into the next nonce's space.

/* THE NONCE REQUIREMENT: UNIQUE, not secret and not random.
   A counter is the correct choice, and better than random -
   because random nonces collide by the birthday bound
   (2^48 messages for a 96-bit nonce is uncomfortably close
   for a long-lived key), while a counter cannot collide
   until it wraps.

/* NONCE REUSE IS THE TWO-TIME PAD (chapter 03):
        C1 XOR C2 = P1 XOR P2
   and the key cancels. Everything in chapter 03 applies.

/* AND CTR IS MALLEABLE, exactly like any stream cipher: flip
   a ciphertext bit, flip the plaintext bit. Which is why CTR
   alone is never correct, and why GCM = CTR + a MAC
   (chapter 09).                                          */
```

The rest

Mode	Parallel	Padding	Auth	Use
ECB	Yes	Yes	No	**Never**
CBC	Dec only	Yes	No	Legacy; only with encrypt-then-MAC
CTR	Yes	No	No	The basis of GCM; alone only inside a construction
CFB	Dec only	No	No	Legacy
OFB	No	No	No	Legacy
GCM	Yes	No	**Yes**	**The default** with AES hardware
CCM	Partly	No	**Yes**	Constrained devices; only needs encryption
OCB	Yes	No	Yes	Fast, formerly patent-encumbered
SIV / GCM-SIV	Yes	No	**Yes**	**Nonce-misuse resistant** — use where nonce uniqueness is not guaranteed
XTS	Yes	No	No	**Disk encryption** — the specific case below

Mode	Parallel	Padding	Auth	Use
KW / KWP	No	—	Yes	Key wrapping (RFC 3394)

```
/* --- XTS, and why disk encryption is different --- */
/* A disk sector must encrypt to exactly one sector - there
   is no room for an IV, a nonce or a tag. So XTS uses the
   SECTOR NUMBER as a tweak:                              */

        T = E_K2(sector) * α^j          /* in GF(2^128) */
        C_j = E_K1(P_j XOR T) XOR T

/* PROPERTIES: length-preserving, random access, and identical
   plaintext in DIFFERENT sectors encrypts differently.

   WHAT IT DOES NOT PROVIDE, and this is important:
     · NO AUTHENTICATION. An attacker with write access can
       modify sectors; the result decrypts to garbage, but
       predictable garbage in some cases.
     · NO FRESHNESS. An old sector can be replayed - the
       "watermarking" and rollback attacks on full-disk
       encryption.
     · identical plaintext in the SAME sector at different
       times gives identical ciphertext, so an observer sees
       when a sector did not change.

   XTS IS THE RIGHT ANSWER FOR DISKS anyway, because the
   alternatives require storing a tag per sector - which
   dm-integrity does, at a real cost. Know what you are not
   getting.

/* --- SIV and AES-GCM-SIV: nonce misuse resistance --- */
/* The tag is computed FIRST, over the plaintext, and used as
   the nonce for the encryption. So a repeated nonce with
   different plaintexts still produces different ciphertexts;
   the only leak is that identical (nonce, plaintext) pairs
   are visible as identical ciphertexts.

   USE IT when you cannot guarantee nonce uniqueness - a
   device that may be restored from backup, a stateless
   protocol, a system where the counter might reset
   (chapter 02). The cost is two passes over the plaintext,
   so it cannot stream.                                   */
```

Choosing

```
Do you need encryption?
     |
     +-- with authentication (you do)
     |     |
     |     +-- can you guarantee unique nonces?
     |     |     |
     |     |     +-- YES, and you have AES hardware
     |     |     |      -> AES-GCM
     |     |     +-- YES, no AES hardware
     |     |     |      -> ChaCha20-Poly1305
     |     |     +-- YES, very constrained device
     |     |     |      -> AES-CCM, or ASCON
     |     |     +-- NO / not sure
     |     |            -> AES-GCM-SIV
     |     |
     |     +-- and the API you should call is the AEAD
     |         one, not a mode plus a MAC you compose
     |         yourself (chapter 09)
     |
     +-- length-preserving, random access (a disk)
     |     -> AES-XTS, and understand what it omits
     |
     +-- wrapping a key
           -> AES-KW / KWP

WHAT NEVER TO CHOOSE:
  ECB, for anything
  CBC or CTR without a MAC
  a mode you assembled from primitives yourself
  a mode because it was in the example code you copied
```

Getting it right

```
/* --- THE MESSAGE FORMAT --- */
    [ nonce/IV ][ ciphertext ][ tag ]
/* The nonce is NOT secret and must be transmitted. The tag
   is typically 16 bytes. Put them in a fixed layout, and
   include the format version so it can change later.

/* --- NONCE MANAGEMENT, which is where products fail --- */
    /* GOOD: a persistent counter */
    struct Nonce { uint32_t device_id; uint64_t counter; };
    /* 96 bits total, unique per device, monotonic */

    /* AND WHAT HAPPENS WHEN IT COULD RESET:
         a device reflashed, a config restored, a clone
       -> ROTATE THE KEY. Do not hope. A repeated
          (key, nonce) pair with GCM does not merely leak the
          plaintexts - it leaks the AUTHENTICATION KEY, and
          the attacker can then forge arbitrary messages.  */

    /* ACCEPTABLE: random 96-bit nonces, if the number of
       messages per key stays well below 2^32 - the birthday
       bound gives a collision probability of about 2^-33 at
       that point.

    /* NEVER: a fixed nonce, a nonce derived from the
       plaintext, or a nonce that restarts at zero.

/* --- KEY AND MESSAGE LIMITS --- */
    AES-GCM     max 2^32 - 2 blocks (64 GB) per (key, nonce)
                and rekey well before 2^32 messages
    ChaCha20    2^32 blocks (256 GB) per nonce
    AES-XTS     one key per volume, sector number as tweak

/* --- USING A LIBRARY: the whole point --- */
    /* libsodium - the API to prefer */
    unsigned char nonce[crypto_aead_xchacha20poly1305_ietf_NPUBBYTES];
    randombytes_buf(nonce, sizeof nonce);
    crypto_aead_xchacha20poly1305_ietf_encrypt(
        ct, &ct_len, pt, pt_len,
        ad, ad_len, NULL, nonce, key);

/* XChaCha20 uses a 192-BIT nonce, which is large enough that
   random nonces are safe indefinitely - removing the entire
   nonce-management problem. Where it is available, it is the
   easiest correct choice.

    /* mbedTLS, on embedded */
    mbedtls_gcm_crypt_and_tag(&ctx, MBEDTLS_GCM_ENCRYPT,
                              len, iv, 12, ad, ad_len,
                              input, output, 16, tag);

/* AND ON DECRYPT, CHECK THE TAG FIRST AND IN CONSTANT TIME:
   */
    if (mbedtls_gcm_auth_decrypt(...) != 0) {
            /* DISCARD EVERYTHING. Do not use the plaintext,
               do not report which part failed, do not log
               the ciphertext. */
            return ERROR;
    }
```

How it shows up

Symptom	Cause
Structure visible in the ciphertext	ECB
Identical messages encrypt identically	ECB, a fixed IV, or a repeated nonce
Plaintext recovered from error responses	A padding oracle in CBC
Plaintext recovered from response timing	Lucky 13 — the same problem, through timing
Attacker confirms guessed plaintext	Predictable CBC IV — BEAST
Two ciphertexts XOR to plaintext	Nonce reuse in CTR or GCM
Forged messages accepted	GCM nonce reuse leaked the authentication key
Ciphertext modified undetectably	No authentication — the mode is unauthenticated
Disk sectors replayed successfully	XTS provides no freshness — expected
Decryption fails after 64 GB	CTR counter wrapped into the next nonce
Interop failure on the tag	Different tag lengths, or the AAD is not identical on both sides

Check yourself

- ✓ Explain why ECB fails, in three distinct ways.
- ✓ Write out CBC encryption and decryption and say why the IV must be unpredictable.
- ✓ Explain PKCS#7 padding and the padding oracle attack.
- ✓ State the three MAC composition orders and why only one is correct.
- ✓ Explain CTR mode and why a counter beats a random nonce.
- ✓ Explain why GCM nonce reuse is worse than CTR nonce reuse.
- ✓ Say what XTS provides and the three things it does not.
- ✓ Explain nonce-misuse resistance and when to use SIV.
- ✓ Choose a mode for five situations using the decision tree.
- ✓ Design a message format and a nonce scheme for a device that may be reflashed.
- ✓ State the message limits for AES-GCM and ChaCha20.

Stream ciphers

A pseudorandom keystream XORed with the plaintext — the one-time pad made practical. Fast, simple, constant-time by construction, and unforgiving of nonce reuse. ChaCha20 is the one to use.

The model

```
key, nonce ---> KEYSTREAM GENERATOR ---> k0 k1 k2 k3 ...
                                          |
plaintext  p0 p1 p2 p3 ...  ---------- XOR |
                                          v
ciphertext c0 c1 c2 c3 ...

ENCRYPTION AND DECRYPTION ARE THE SAME OPERATION - one
implementation serves both, which halves the code on a
constrained device.

PROPERTIES INHERITED FROM THE ONE-TIME PAD (chapter 03):
  + no padding - encrypt any length
  + no block alignment
  + can start encrypting before the length is known
  + a bit error in transit corrupts exactly one bit
  - NONCE REUSE IS CATASTROPHIC: c1 XOR c2 = p1 XOR p2
  - NO INTEGRITY: flipping a ciphertext bit flips the
    plaintext bit

AND THE TWO FAMILIES:

  SYNCHRONOUS       the keystream depends only on the key
                    and nonce, not on the data. ChaCha20,
                    RC4, AES-CTR.
  SELF-SYNCHRONISING the keystream depends on previous
                    ciphertext. CFB mode. Recovers from
                    insertions and deletions, at the cost
                    of error propagation. Rarely used.

NOTE THAT CTR MODE (chapter 05) IS A STREAM CIPHER built
from a block cipher. The distinction between "stream
cipher" and "block cipher in a streaming mode" is about
construction, not about the properties you get.
```

ChaCha20

```
/* Daniel Bernstein, 2008. A refinement of Salsa20, and the
   modern default for software encryption.                */

/* THE STATE: 16 words of 32 bits = 512 bits, 4x4 */

        c c c c          c = constants "expand 32-byte k"
        k k k k          k = the 256-bit key
        k k k k
        b n n n          b = 32-bit block counter
                         n = 96-bit nonce

/* THE QUARTER-ROUND, the entire nonlinear core: */
    #define QR(a, b, c, d)                    \
        a += b;  d ^= a;  d = rotl32(d, 16);   \
        c += d;  b ^= c;  b = rotl32(b, 12);   \
        a += b;  d ^= a;  d = rotl32(d,  8);   \
        c += d;  b ^= c;  b = rotl32(b,  7);

/* ADD, ROTATE, XOR - an "ARX" cipher. No S-boxes, no tables,
   NO DATA-DEPENDENT MEMORY ACCESS, and no data-dependent
   branches. CONSTANT TIME BY CONSTRUCTION, on every
   platform, with no special effort (chapter 19).

/* THE BLOCK FUNCTION: 20 rounds, alternating column and
   diagonal quarter-rounds, then add the original state. */

    void chacha20_block(const uint32_t in[16], uint32_t out[16])
    {
        uint32_t x[16];
        memcpy(x, in, sizeof x);

        for (int i = 0; i < 10; i++) {      /* 10 double
                                               rounds = 20  */
            /* column rounds */
            QR(x[0], x[4], x[ 8], x[12]);
            QR(x[1], x[5], x[ 9], x[13]);
            QR(x[2], x[6], x[10], x[14]);
            QR(x[3], x[7], x[11], x[15]);
            /* diagonal rounds */
            QR(x[0], x[5], x[10], x[15]);
            QR(x[1], x[6], x[11], x[12]);
            QR(x[2], x[7], x[ 8], x[13]);
            QR(x[3], x[4], x[ 9], x[14]);
        }

        for (int i = 0; i < 16; i++)
            out[i] = x[i] + in[i];      /* the feed-forward
                                           makes it
                                           non-invertible  */
    }

/* THE FINAL ADDITION MATTERS: without it the block function
   would be an invertible permutation and an attacker could
   run it backwards. Adding the input makes it one-way.

/* WHY IT IS THE DEFAULT NOW:
     · 3-4x faster than software AES on a CPU without AES-NI
     · constant time with no effort - which software AES
       requires bitslicing to achieve (chapter 04)
     · a 20-round design with a large margin; the best attack
```

```
       reaches 7 rounds
     · small: a few hundred bytes of code, 64 bytes of state
     · vectorises well - SSE, AVX2 and NEON implementations
       process several blocks at once
     · no patents, a public-domain reference implementation

/* XCHACHA20: extends the nonce to 192 bits by hashing the
   key and the first 128 nonce bits through the same
   permutation (HChaCha20) to derive a subkey. A 192-bit
   nonce is large enough that RANDOM nonces never collide -
   removing the nonce management problem entirely
   (chapter 05). Prefer XChaCha20-Poly1305 wherever it is
   available.                                              */
```

CHACHA20 OR AES?

Hardware AES available (any modern Arm application core, any x86): AES-GCM is faster — around 1 cycle/byte versus 3-4. **No hardware AES** (most microcontrollers, older cores): ChaCha20-Poly1305 wins on speed, code size and side-channel resistance simultaneously. TLS 1.3 negotiates between exactly these two for exactly this reason, and mobile clients without AES acceleration end up on ChaCha20.

RC4 — the cautionary tale

```
/* Designed 1987, leaked 1994, ubiquitous for two decades,
   comprehensively broken. Worth understanding because every
   one of its failures illustrates a general principle. */

    /* the key schedule */
    for (i = 0; i < 256; i++) S[i] = i;
    for (i = j = 0; i < 256; i++) {
        j = (j + S[i] + key[i % keylen]) & 0xFF;
        swap(S[i], S[j]);
    }
    /* the keystream */
    i = j = 0;
    while (n--) {
        i = (i + 1) & 0xFF;
        j = (j + S[i]) & 0xFF;
        swap(S[i], S[j]);
        *out++ = S[(S[i] + S[j]) & 0xFF];
    }

/* Beautifully small - which is why it spread. And:

   1. THE SECOND OUTPUT BYTE is 0 with probability 2/256
      instead of 1/256. There are biases throughout the first
      few thousand bytes.
      -> the RC4 NOMORE and Bar-Mitzvah attacks recovered
         plaintext from TLS and WPA-TKIP.

   2. THE KEY SCHEDULE IS WEAK. Related keys - a per-message
      IV concatenated with a fixed key - leak the key.
      -> THIS IS WEP. Concatenating a 24-bit IV with the
         shared key produced related keys, and WEP was broken
         to the point of key recovery in minutes.

   3. IT USES A KEY-DEPENDENT LOOKUP TABLE, so software
      implementations have a cache-timing side channel
      (chapter 19).

   PROHIBITED in TLS since RFC 7465. Removed from every
   modern library.

/* THE GENERAL LESSONS:
     · a cipher can be "unbroken" for years and then fall to
       accumulated statistical work
     · KEY SCHEDULES MATTER as much as the round function
     · concatenating an IV with a key is not a nonce
       mechanism - use a construction designed for it
     · small and elegant is not the same as secure         */
```

The others

Cipher	Status
ChaCha20	**The default.** TLS 1.3, WireGuard, SSH, the Linux CSPRNG
Salsa20	ChaCha20's predecessor. Sound; ChaCha has better diffusion
AES-CTR	A block cipher in stream mode — fine, inside an AEAD
RC4	**Broken.** Prohibited

Cipher	Status
A5/1, A5/2	**Broken.** GSM; A5/2 deliberately weakened
E0	**Weak.** Bluetooth classic
Trivium, Grain	eSTREAM hardware profile — tiny, sound, niche
Rabbit, HC-128, SOSEMANUK	eSTREAM software profile — sound, little-used
ASCON	**NIST lightweight standard (2023).** A sponge-based AEAD for constrained devices
ZUC, SNOW 3G	3GPP standards for LTE/5G air-interface encryption

```
/* THE PATTERN IN THE BROKEN ONES: A5/1, A5/2, E0 and CSS
   were all designed in secret, for constrained hardware, by
   committees under export-control pressure, and published
   only after deployment. Every one fell quickly once
   examined (chapter 01's Kerckhoffs point).

   THE eSTREAM PORTFOLIO (2008) and the NIST LIGHTWEIGHT
   COMPETITION (2019–2023) were run the opposite way - open
   submission, years of public cryptanalysis - and their
   outputs have held.                                    */
```

Using one correctly

```
/* --- THE ONLY CORRECT WAY: use the AEAD, not the raw
       cipher --- */

    /* libsodium, and this is the recommendation */
    #include <sodium.h>

    unsigned char nonce[crypto_aead_xchacha20poly1305_ietf_NPUBBYTES];
    randombytes_buf(nonce, sizeof nonce);   /* 192 bits -
                                               random is safe */

    crypto_aead_xchacha20poly1305_ietf_encrypt(
            ciphertext, &ct_len,
            plaintext, pt_len,
            aad, aad_len,
            NULL, nonce, key);

    /* decryption FAILS CLOSED if the tag does not verify */
    if (crypto_aead_xchacha20poly1305_ietf_decrypt(
            decrypted, &pt_len, NULL,
            ciphertext, ct_len, aad, aad_len,
            nonce, key) != 0) {
            /* FORGED OR CORRUPTED - discard everything */
            return -1;
    }

/* --- THE RAW CIPHER, only inside a construction you are
       building deliberately --- */
    crypto_stream_chacha20_ietf_xor(out, in, len, nonce, key);
/* No authentication. If you call this directly you are
   responsible for the MAC, and you should probably not be.

/* --- THE NONCE RULES, restated because they are what
       fails --- */
    ChaCha20 (IETF)   96-bit nonce, 32-bit counter
                      -> 256 GB max per (key, nonce)
                      -> random nonces collide at ~2^32
                         messages: USE A COUNTER
    XChaCha20         192-bit nonce
                      -> RANDOM NONCES ARE SAFE, always.
                         This is why it is preferable.

/* --- SEEKING WITHIN A STREAM, which stream ciphers make
       easy --- */
    crypto_stream_chacha20_ietf_xor_ic(out, in, len,
                                       nonce, block_index,
                                       key);
/* Set the counter to the block containing your offset and
   decrypt from there. Useful for a firmware image being
   decrypted in chunks, or a file being seeked - and one of
   the genuine advantages over CBC.

/* --- ON A MICROCONTROLLER --- */
    /* Monocypher: ~2 KB of code for the whole AEAD */
    crypto_aead_lock(ciphertext, mac, key, nonce,
                     aad, aad_len, plaintext, pt_len);

    if (crypto_aead_unlock(plaintext, mac, key, nonce,
                           aad, aad_len, ciphertext, ct_len))
            return ERROR;       /* non-zero = FORGED */
```

```
/* Monocypher is public domain, single-file, constant time,
   and implements exactly the modern primitives - ChaCha20,
   Poly1305, BLAKE2b, X25519, EdDSA, Argon2. For a
   microcontroller with no crypto accelerator it is often the
   right answer over mbedTLS, because it is a tenth the
   size.                                                  */
```

How it shows up

Symptom	Cause
Two ciphertexts XOR to readable text	Nonce reuse — the two-time pad (ch. 03)
Attacker flips bits in the plaintext	No authentication — use the AEAD
Key recovered from many WEP frames	RC4's key schedule with a concatenated IV
Plaintext recovered from many TLS sessions	RC4 keystream biases
Nonce counter resets after a reflash	Reuse across the reset — rotate the key
Random 96-bit nonces on a long-lived key	Birthday collision at ~2^{32} messages — use a counter or XChaCha20
Decryption fails after 256 GB	The 32-bit block counter wrapped
Key recovered by a co-located process	A table-based cipher — ChaCha20 has no tables
Interop failure between implementations	Original ChaCha20 (64-bit nonce) versus IETF (96-bit)
Decrypted plaintext used before the tag check	Never do this — verify first, then release

Check yourself

- ✓ Draw the stream cipher model and list the properties inherited from the one-time pad.
- ✓ Explain why encryption and decryption are the same operation.
- ✓ Describe ChaCha20's state and the quarter-round.
- ✓ Explain why an ARX design is constant time without effort.
- ✓ Say why the final addition in the block function matters.
- ✓ Give four reasons ChaCha20 is preferred where AES hardware is absent.
- ✓ Explain what XChaCha20 changes and why it removes a whole class of bug.
- ✓ Give three distinct reasons RC4 is broken, and the general lesson from each.
- ✓ Explain what WEP did wrong with its IV.
- ✓ Use an AEAD correctly, including the fail-closed decryption path.
- ✓ State the nonce and message limits for IETF ChaCha20 and XChaCha20.

Hash functions

A fixed-size fingerprint of arbitrary data, with three security properties that are routinely confused. Hashes underpin signatures, MACs, key derivation, secure boot and blockchains — and the property you need determines which is safe to use.

The three properties

```
H : {0,1}* -> {0,1}^n

PREIMAGE RESISTANCE
  given h, infeasible to find m with H(m) = h
  "one-way"
  cost to break: 2^n

SECOND PREIMAGE RESISTANCE
  given m1, infeasible to find m2 != m1 with
  H(m2) = H(m1)
  cost to break: 2^n

COLLISION RESISTANCE
  infeasible to find ANY m1 != m2 with H(m1) = H(m2)
  cost to break: 2^(n/2)     <-- THE BIRTHDAY BOUND

THE FACTOR OF TWO IS THE POINT. For SHA-256:
  preimage      2^256   - unreachable
  collision     2^128   - unreachable
For SHA-1 (160 bits):
  preimage      2^160   - still unreachable
  collision     2^80    - REACHED IN 2017 (SHAttered)

WHICH IS WHY SHA-1 IS BROKEN FOR SIGNATURES AND
CERTIFICATES but not yet for HMAC: signatures need
collision resistance (a forger picks BOTH messages), while
HMAC's security rests on the key and needs only much
weaker properties. The distinction matters when you are
deciding whether a legacy system must be changed
immediately.

AND THE BIRTHDAY PARADOX, since it is the whole reason:
among 23 people two share a birthday with probability
> 1/2, because there are 253 PAIRS. Collisions come from
pairs, so n/2 bits of work suffices - and any protocol
whose security depends on collisions needs twice the
output length you might expect.
```

What hashes are for

```
/* --- LEGITIMATE --- */
    integrity checking       compare H(file) with a known
                             value
    digital signatures       sign H(m), not m (chapter 15)
    MACs                     HMAC (chapter 08)
    key derivation           HKDF (chapter 10)
    commitment schemes       publish H(x), reveal x later
    Merkle trees             efficient verification of a
                             large set
    deduplication            content-addressed storage
    proof of work            find m with H(m) < target
    secure boot              measure each stage before
                             executing it
                             (isolation chapter 14)

/* --- NOT LEGITIMATE, and each is a real mistake --- */

    PASSWORD STORAGE with a plain hash
        Hashes are FAST - billions per second on a GPU. That
        is the wrong property here. Use Argon2id
        (chapter 10).

    A MAC AS H(key ‖ message)
        LENGTH EXTENSION (below) lets an attacker append data
        and produce a valid tag WITHOUT the key. Use HMAC.

    ENCRYPTION
        A hash is not reversible. "Hashing" data does not
        protect it if the input space is small - an attacker
        enumerates. H(email) is not anonymisation; H(phone
        number) is trivially reversible.

    RANDOM NUMBER GENERATION from a low-entropy input
        H(timestamp) has as much entropy as the timestamp
        (chapter 02).

    COMPARING SECRETS with memcmp
        Early-exit comparison leaks how many bytes matched
        (chapter 19). Use a constant-time compare.        */
```

How they are built

```
MERKLE-DAMGARD - MD5, SHA-1, SHA-2

  message -> pad -> split into blocks
  IV -> [f] -> [f] -> [f] -> ... -> output
         ^       ^       ^
         m0      m1      m2

  A compression function f applied iteratively to a
  chaining value. If f is collision resistant, so is the
  whole hash - which is the construction's theorem and the
  reason it was used.

  AND ITS FLAW: the output IS the final chaining value.
  So an attacker who knows H(m) can CONTINUE HASHING from
  that state:

     H(m ‖ padding ‖ extra)

  computed without knowing m - only its length. That is
  LENGTH EXTENSION, and it is why H(key ‖ message) is not
  a MAC.

SPONGE - SHA-3, ASCON, BLAKE3's tree mode

  a large state, split into RATE and CAPACITY:

    ABSORB:  XOR message blocks into the rate, permute
    SQUEEZE: read output from the rate, permute

    state  [ rate | capacity ]
              ^        ^
           message   never touched directly by
           goes in   the message or the output

  THE CAPACITY IS NEVER EXPOSED, so length extension is
  impossible. Output can be ANY LENGTH - which gives
  SHAKE128/256, the extendable-output functions used by
  the post-quantum schemes (chapter 20).

AND THE PRACTICAL CONSEQUENCE: SHA-3, BLAKE2 and BLAKE3
are all immune to length extension by construction, and
SHA-512/256 is immune because it truncates. SHA-256 is
not.
```

Which to use

Hash	Output	Status
MD5	128	**Broken** — collisions in seconds. Never
SHA-1	160	**Broken** for collisions (2017). Never for new use
SHA-256	256	**The default.** Hardware acceleration common
SHA-384 / SHA-512	384 / 512	Sound. **Faster than SHA-256 on 64-bit**
SHA-512/256	256	SHA-512 truncated — 256-bit output, no length extension, fast on 64-bit
SHA3-256 / SHAKE	256 / any	Sound, sponge-based. Slower in software, good in hardware
BLAKE2b / BLAKE2s	any ≤ 512	Faster than SHA-256, keyed mode built in

Hash	Output	Status
BLAKE3	any	Very fast, parallel, tree-structured
CRC32 / CRC16	32 / 16	**Not cryptographic.** Error detection only
SipHash	64 / 128	Keyed, for hash tables — resists collision-flooding DoS

```
/* CHOOSING:

   Interoperability required (TLS, signatures, standards)
        -> SHA-256, or SHA-384 where the standard says so

   64-bit CPU, performance matters
        -> SHA-512/256 or BLAKE2b - both faster than SHA-256
           because they work on 64-bit words

   Microcontroller with a SHA accelerator
        -> whatever it accelerates, usually SHA-256

   Microcontroller without one, code size matters
        -> BLAKE2s (~1 KB) or SHA-256 (~2 KB)

   Hashing very large amounts of data
        -> BLAKE3, which parallelises across cores and SIMD

   Need arbitrary-length output
        -> SHAKE128/256, or BLAKE2X, or BLAKE3

   Need a keyed hash
        -> BLAKE2's keyed mode, or HMAC (chapter 08)

   Hash table keys, attacker-controlled
        -> SipHash with a per-process random key

/* AND CRC IS NOT A HASH. It is linear, so an attacker can
   compute exactly what to change to preserve it. It detects
   accidental corruption, which is a completely different
   job. Using CRC32 for integrity against an adversary is a
   finding, and it appears in shipped firmware regularly. */
```

> **SHA-256 IS NOT ALWAYS THE FASTEST CHOICE**
>
> SHA-256 uses 32-bit operations; SHA-512 uses 64-bit ones and does fewer rounds per byte. On any 64-bit CPU without SHA hardware extensions, **SHA-512/256 is roughly 1.5x faster than SHA-256 and gives the same 256-bit output** — plus immunity to length extension. It is under-used simply because the name is unfamiliar.

Length extension, concretely

```
/* THE VULNERABLE CONSTRUCTION: */
        tag = SHA256(secret_key ‖ message)

/* THE ATTACK: the attacker knows `message` and `tag`, and
   the LENGTH of the key - not the key itself.

   1. SHA-256's output IS its internal state after the final
      block.
   2. Load that state into a fresh SHA-256 context.
   3. Continue hashing arbitrary `extra` data.
   4. The result is a valid tag for:

        secret_key ‖ message ‖ padding ‖ extra

   The attacker has forged a tag for a message they chose,
   without ever learning the key.

/* A REAL EXAMPLE, and this shape appears constantly: */
        /* the server checks: */
        tag == SHA256(key ‖ "user=alice&admin=false")

        /* the attacker produces a valid tag for: */
        "user=alice&admin=false" ‖ 0x80 00 ... len ‖
                                        "&admin=true"

   If the parser takes the LAST occurrence of a duplicated
   parameter - which many do - the attacker is now an
   administrator. This has broken Flickr's API, several
   payment gateways, and a long list of others.

/* THE FIXES, in order of preference:
     1. USE HMAC (chapter 08). It is designed for this and
        the nesting defeats the attack.
     2. Use a hash that is immune: SHA-3, BLAKE2/3,
        SHA-512/256.
     3. Use an AEAD, which authenticates as part of
        encryption (chapter 09).

   H(message ‖ key) is also weak - it reduces the forgery to
   finding a collision in H. HMAC exists precisely so that
   nobody has to reason about this.                       */
```

Using them

```
/* --- INCREMENTAL HASHING, which every API supports --- */
    #include <mbedtls/sha256.h>

    mbedtls_sha256_context ctx;
    uint8_t digest[32];

    mbedtls_sha256_init(&ctx);
    mbedtls_sha256_starts(&ctx, 0);         /* 0 = SHA-256 */
    while ((n = read_chunk(buf, sizeof buf)) > 0)
            mbedtls_sha256_update(&ctx, buf, n);
    mbedtls_sha256_finish(&ctx, digest);
    mbedtls_sha256_free(&ctx);

/* Streaming means you never need the whole message in
   memory - essential for verifying a firmware image larger
   than your RAM (isolation chapter 14).

/* --- COMPARING DIGESTS: constant time, always --- */
    /* WRONG: leaks how many bytes matched */
    if (memcmp(computed, expected, 32) == 0) accept();

    /* RIGHT: */
    static int ct_equal(const uint8_t *a, const uint8_t *b,
                        size_t n)
    {
            uint8_t diff = 0;
            for (size_t i = 0; i < n; i++)
                    diff |= a[i] ^ b[i];
            return diff == 0;
    }
/* Or the library's version: mbedtls_ct_memcmp,
   sodium_memcmp, CRYPTO_memcmp. This matters whenever the
   attacker can submit candidate digests and measure the
   response (chapter 19).

/* --- DOMAIN SEPARATION: hash a purpose, not just data --- */
    /* WRONG - the same hash used for two purposes: */
    key1 = SHA256(master ‖ "a");
    key2 = SHA256(master ‖ "ab");   /* and "a" ‖ "b" collides
                                       with "ab" ‖ "" */
    /* RIGHT - unambiguous encoding, or a proper KDF: */
    key1 = HKDF(master, info="encryption-key-v1");
    key2 = HKDF(master, info="mac-key-v1");
/* Concatenating variable-length fields into a hash input is
   a recipe for ambiguity. Include lengths, or use a KDF with
   an info string (chapter 10).

/* --- MERKLE TREES, for verifying a large image in
       pieces --- */
             root
            /    \
          h01     h23
         /  \    /  \
        h0  h1  h2  h3
        |   |   |   |
        b0  b1  b2  b3

/* Verify block 2 with h3, h01 and the root - three hashes
   instead of the whole image. Used by dm-verity
```

```
    (Linux chapter 14 / isolation chapter 14), by every
    package manager, and by BLAKE3's internal structure.  */
```

How it shows up

Symptom	Cause
Forged tag accepted for an appended message	Length extension on `H(key ‖ msg)` — use HMAC
Two different files with the same hash	MD5 or SHA-1 — a chosen-prefix collision
Passwords cracked at billions per second	A plain hash instead of a password KDF (ch. 10)
"Anonymised" identifiers reversed	Small input space — hashing is not anonymisation
Digest comparison leaks through timing	`memcmp` — use a constant-time compare
Integrity check passes on modified data	CRC used as if it were a hash
Two different inputs hash the same by construction	Ambiguous concatenation — add domain separation
Hash table performance collapses under load	Collision flooding — use SipHash with a random key
Certificate accepted that should not be	SHA-1 signature — collision attack
Hashing is the bottleneck	Try SHA-512/256, BLAKE2 or BLAKE3; check for hardware support

Check yourself

- ✓ State the three security properties and the cost of breaking each.
- ✓ Explain the birthday bound and why collision resistance is only n/2 bits.
- ✓ Explain why SHA-1 is broken for certificates but not for HMAC.
- ✓ List five legitimate uses and five misuses.
- ✓ Contrast Merkle–Damgård with the sponge construction.
- ✓ Explain length extension concretely and give three fixes.
- ✓ Say why `H(key ‖ msg)` and `H(msg ‖ key)` are both wrong.
- ✓ Choose a hash for six different situations.
- ✓ Explain why CRC is not a cryptographic hash.
- ✓ Write a constant-time digest comparison and say when it matters.
- ✓ Explain domain separation and the ambiguity it prevents.

Message authentication codes

A keyed tag proving a message came from someone holding the key and has not been modified. This is the integrity half of chapter 01 — and the one place where a hand-rolled construction is almost guaranteed to be broken.

The model

```
MAC : key × message -> tag

SENDER:    t = MAC(k, m);  send (m, t)
RECEIVER:  t' = MAC(k, m); accept iff t' == t
                                    (in CONSTANT TIME)

THE SECURITY GOAL: EXISTENTIAL UNFORGEABILITY under a
CHOSEN-MESSAGE ATTACK (EUF-CMA).

  An adversary who can obtain tags for any messages they
  choose still cannot produce a valid (m, t) pair for any
  message they have not seen.

WHAT A MAC PROVIDES:
  · INTEGRITY   - the message was not modified
  · AUTHENTICITY - it came from a holder of k

WHAT IT DOES NOT:
  · CONFIDENTIALITY - the message is in the clear
  · NON-REPUDIATION - BOTH parties hold k, so either could
    have produced the tag. A third party learns nothing
    about which. That needs a signature (chapter 15).
  · FRESHNESS - a valid (m, t) pair replays forever. Add a
    counter, a timestamp or a nonce, and authenticate it.

TAG LENGTH: 128 bits is standard. A truncated tag of t
bits gives an attacker a 2^-t chance per forgery attempt -
so 64 bits is acceptable only where the number of attempts
is rate-limited, and 32 bits is a bad idea in almost every
case.
```

HMAC

```
/* THE CONSTRUCTION, RFC 2104: */

    HMAC(K, m) = H( (K' XOR opad) ‖ H( (K' XOR ipad) ‖ m ) )

        K'    the key, padded to the hash's BLOCK SIZE
              (64 bytes for SHA-256), or H(K) if longer
        ipad  0x36 repeated
        opad  0x5C repeated

/* WHY THE NESTING: it defeats length extension (chapter 07).
   The inner hash's output is fed into an outer hash keyed
   differently, so an attacker who knows the tag cannot
   continue the computation - the outer hash's state is
   unknown.

/* WHY TWO DIFFERENT PADS: ipad and opad differ in every bit
   position of every byte pair, which makes the two derived
   keys effectively independent.

/* THE SECURITY PROOF is remarkable: HMAC is secure if the
   compression function is a pseudorandom function - a much
   weaker assumption than collision resistance. Which is why
   HMAC-SHA1 REMAINS SECURE despite SHA-1's collision breaks
   (chapter 07). It should still be replaced, but it is not
   urgent in the way an SHA-1 signature is.

/* IMPLEMENTATION: */
    #include <mbedtls/md.h>

    uint8_t tag[32];
    const mbedtls_md_info_t *info =
        mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);

    mbedtls_md_hmac(info, key, key_len, msg, msg_len, tag);

    /* streaming, for a large message */
    mbedtls_md_context_t ctx;
    mbedtls_md_setup(&ctx, info, 1);        /* 1 = HMAC */
    mbedtls_md_hmac_starts(&ctx, key, key_len);
    while ((n = read(buf, sizeof buf)) > 0)
        mbedtls_md_hmac_update(&ctx, buf, n);
    mbedtls_md_hmac_finish(&ctx, tag);
    mbedtls_md_free(&ctx);

/* VERIFICATION, and this is where implementations fail: */
    /* WRONG - early exit leaks how many bytes matched */
    if (memcmp(tag, received, 32) == 0) accept();

    /* RIGHT */
    if (mbedtls_ct_memcmp(tag, received, 32) == 0) accept();
    /* or sodium_memcmp, CRYPTO_memcmp, or the loop from
       chapter 07                                          */

/* THE TIMING ATTACK IS REAL AND PRACTICAL: an attacker
   submits candidate tags and measures the response. Each
   additional matching byte takes measurably longer, so they
   recover the tag one byte at a time - 256 x 16 attempts
   instead of 2^128. This has broken real APIs, including
   over a network where the difference was nanoseconds -
```

```
   statistical averaging over many requests extracts it
   (chapter 19).                                          */
```

> **NEVER BUILD A MAC YOURSELF**
>
> `H(k ‖ m)` falls to length extension. `H(m ‖ k)` reduces to finding a hash collision. `H(k ‖ m ‖ k)` has weaknesses too. Encrypting a hash, XORing a key in, truncating cleverly — every ad-hoc construction has been tried and broken. HMAC exists precisely so nobody has to reason about this again, and it is one function call.

Poly1305 and universal hashing

```
/* A different construction entirely: a ONE-TIME MAC based on
   polynomial evaluation in a prime field. Much faster than
   HMAC, and the authenticator inside ChaCha20-Poly1305 and
   (in a related form) AES-GCM.                             */

    Poly1305(r, s, m):
        split m into 16-byte chunks c1..cn, each with a
        high bit set
        evaluate  ((c1*r^n + c2*r^(n-1) + ... + cn*r) mod
                   (2^130 - 5))  +  s   mod 2^128

/* r and s are derived from the key and the nonce, and r has
   some bits CLAMPED to make the arithmetic efficient.

/* THE CRITICAL PROPERTY - AND THE CRITICAL WARNING:

     POLY1305 IS A ONE-TIME MAC. The (r, s) pair must NEVER
     be reused for two messages.

   With two tags under the same key, the attacker gets two
   polynomial equations in the unknown r, and can SOLVE FOR
   THE AUTHENTICATION KEY. They can then forge arbitrary
   messages.

   This is exactly why GCM nonce reuse is worse than CTR
   nonce reuse (chapter 05): CTR reuse leaks plaintexts;
   GCM reuse additionally hands over the ability to forge.

/* SO IT IS ALWAYS USED WITH A NONCE-DERIVED KEY: */
    ChaCha20-Poly1305:
        (r, s) = first 32 bytes of ChaCha20(key, nonce,
                                            counter=0)
        ciphertext = ChaCha20(key, nonce, counter=1) XOR
                     plaintext
        tag = Poly1305(r, s, aad ‖ ciphertext ‖ lengths)

/* THE LENGTH FIELDS AT THE END MATTER. Without them, an
   attacker could move bytes between the AAD and the
   ciphertext while keeping the concatenation identical -
   producing a valid tag for a different (aad, ciphertext)
   split. Any construction concatenating variable-length
   fields into an authenticator needs unambiguous encoding.

/* GHASH, used by GCM, is the same idea over GF(2^128)
   instead of a prime field - faster in hardware (the
   PMULL/CLMUL instructions), and with the same one-time
   requirement.                                             */
```

CMAC and the block-cipher MACs

```
/* When you have an AES accelerator and no hash accelerator,
   a block-cipher MAC costs nothing extra.                 */

/* --- CBC-MAC: the naive version, and why it is broken --- */
       t = last block of CBC-encrypt(m) with IV = 0

/* SECURE ONLY for FIXED-LENGTH messages. With variable
   lengths:
        given  MAC(m1) = t1  and  MAC(m2) = t2
        then   MAC(m1 ‖ (m2[0] XOR t1) ‖ m2[1..]) = t2
   A forgery from two known tags, with no key.

/* --- CMAC (RFC 4493): CBC-MAC done correctly --- */
    Derive two subkeys K1, K2 from K.
    XOR the final block with K1 (if the message is a whole
    number of blocks) or pad and XOR with K2 (if not).
    That distinguishes padded from unpadded messages and
    closes the extension attack.

    /* mbedTLS */
    mbedtls_cipher_cmac(cipher_info, key, 128,
                        msg, msg_len, tag);

/* CMAC IS THE RIGHT CHOICE when:
     · you have AES hardware and no SHA hardware
     · you are already doing AES and want one primitive
     · a standard requires it (many do - it is NIST SP
       800-38B)

/* --- GMAC: GCM with no plaintext --- */
    AES-GCM with an empty plaintext and the data as AAD is a
    MAC. Very fast with PMULL/CLMUL. Inherits GCM's nonce
    requirement absolutely.

/* --- KMAC: the SHA-3 keyed mode --- */
    A sponge is naturally keyed - just absorb the key first.
    No HMAC-style nesting needed, because there is no length
    extension (chapter 07). Same for BLAKE2's keyed mode and
    BLAKE3's keyed_hash.                                    */
```

Choosing

MAC	Based on	Use when
HMAC-SHA256	Any hash	**The safe default.** Standard everywhere, no nonce needed
HMAC-SHA512	SHA-512	Faster on 64-bit CPUs
Poly1305	Polynomial	Inside ChaCha20-Poly1305. **Never standalone with a reused key**
GMAC / GHASH	GF(2^{128})	Inside GCM; fast with PMULL
CMAC	AES	AES hardware available, no hash hardware
KMAC	SHA-3	Where SHA-3 is already used
BLAKE2 keyed / BLAKE3	Their own	Fast software, no separate HMAC needed
SipHash	ARX	Short inputs, hash-table DoS resistance — *not* a general MAC
CBC-MAC	Any cipher	**Never** — use CMAC

```
/* THE DECISION, in practice:

   Encrypting anyway?           -> use an AEAD (chapter 09).
                                   The tag comes free and the
                                   composition is already
                                   correct.
   Authenticating only?         -> HMAC-SHA256
   AES hardware, no SHA?        -> CMAC
   Very constrained?            -> CMAC with the AES engine,
                                   or ASCON
   Already using BLAKE2/3?      -> its keyed mode

   AND ALMOST ALWAYS THE FIRST: if you find yourself choosing
   a MAC separately from a cipher, check whether you should
   be using an AEAD instead. Composing them by hand is where
   the mistakes are.                                        */
```

Using a MAC in a protocol

```
/* --- 1. AUTHENTICATE EVERYTHING THAT MATTERS --- */
    /* WRONG - the header is unprotected */
    tag = HMAC(k, payload);
    send(header, payload, tag);
    /* an attacker changes the header freely: the message
       type, the destination, the length */

    /* RIGHT */
    tag = HMAC(k, header ‖ payload);

/* --- 2. INCLUDE A COUNTER, OR REPLAY IS TRIVIAL --- */
    struct Message {
        uint32_t seq;           /* monotonic, per direction */
        uint8_t  type;
        uint16_t len;
        uint8_t  payload[];
    };
    tag = HMAC(k, seq ‖ type ‖ len ‖ payload);

    /* the receiver: */
    if (msg.seq <= last_seen_seq) reject();     /* REPLAY */
    if (!ct_equal(tag, computed, 16)) reject();
    last_seen_seq = msg.seq;

/* AND CHECK THE TAG BEFORE THE SEQUENCE, or an attacker can
   probe your sequence-number state with forged messages.
   Actually check BOTH, and reject on either - but never act
   on unauthenticated data.

/* --- 3. USE SEPARATE KEYS FOR EACH DIRECTION AND
       PURPOSE --- */
    k_c2s_enc = HKDF(master, "client-to-server encryption");
    k_c2s_mac = HKDF(master, "client-to-server mac");
    k_s2c_enc = HKDF(master, "server-to-client encryption");
    k_s2c_mac = HKDF(master, "server-to-client mac");

/* ONE KEY FOR BOTH DIRECTIONS allows REFLECTION: the
   attacker replays a client message back to the client, and
   it authenticates. Real protocols have fallen to this.
   Key derivation is cheap (chapter 10); use distinct keys.

/* --- 4. UNAMBIGUOUS ENCODING --- */
    /* WRONG: */
    tag = HMAC(k, user ‖ action);
    /* ("alice", "delete") and ("alic", "edelete") produce the
       same input */
    /* RIGHT: */
    tag = HMAC(k, len(user) ‖ user ‖ len(action) ‖ action);

/* --- 5. AND THE VERIFICATION DISCIPLINE --- */
    · constant-time comparison, always
    · compare the FULL tag, never a prefix
    · on failure: reject the whole message, log nothing
      attacker-controlled, and return an INDISTINGUISHABLE
      error - "authentication failed", never "bad sequence
      number" or "unknown message type"
    · rate-limit failures, so an attacker cannot make 2^32
      attempts                                              */
```

How it shows up

Symptom	Cause
Forged tag accepted for an extended message	Length extension on `H(k ‖ m)` — use HMAC
Tag recovered one byte at a time	Non-constant-time comparison
Forgery after two messages	Poly1305 or GHASH key reused — nonce reuse
Forgery from two known tags	Raw CBC-MAC with variable-length messages
Old messages replayed successfully	No sequence number or timestamp in the authenticated data
A client's own message accepted back	The same key in both directions — reflection
Header modified without detection	Only the payload was authenticated
Two different messages produce the same input	Ambiguous concatenation — add length prefixes
Attacker learns why a message was rejected	Distinguishable error responses
A third party cannot prove who sent a message	A MAC does not provide non-repudiation — by design

Check yourself

- ✓ State the MAC security goal and the three things a MAC does not provide.
- ✓ Write out the HMAC construction and explain why it is nested.
- ✓ Explain why HMAC-SHA1 remains secure while SHA-1 signatures are not.
- ✓ Explain the timing attack on tag comparison and why it works over a network.
- ✓ Explain why Poly1305 is a one-time MAC and what reuse hands to an attacker.
- ✓ Say why the length fields at the end of the Poly1305 input matter.
- ✓ Explain why raw CBC-MAC is broken and what CMAC changes.
- ✓ Choose a MAC for five situations, and say when the answer is really "use an AEAD".
- ✓ List the five protocol rules for using a MAC correctly.
- ✓ Explain reflection attacks and the key-separation fix.

Authenticated encryption

Confidentiality and integrity in one primitive, analysed as a whole. This is the answer to almost every "how do I encrypt this?" question, and using anything else needs a specific justification.

The interface

```
ENCRYPT(key, nonce, plaintext, associated_data)
     -> ciphertext, tag

DECRYPT(key, nonce, ciphertext, associated_data, tag)
     -> plaintext, or FAILURE

THE FOUR INPUTS:

  KEY               secret, 128 or 256 bits
  NONCE             not secret; MUST BE UNIQUE per key
  PLAINTEXT         encrypted AND authenticated
  ASSOCIATED DATA   authenticated but NOT encrypted

THE ASSOCIATED DATA IS THE UNDER-USED PART. It is for
anything that must be bound to the message but has to stay
readable:

  · a packet header the router must see
  · a message type or version field
  · a sequence number
  · the sender and recipient identifiers
  · the record layer header in TLS

Bind them, and an attacker cannot take a valid ciphertext
and replay it with a different header - which is a real
attack whenever the header carries routing or type
information.

THE FAILURE MODE IS THE POINT: decryption returns FAILURE,
not plaintext. There is no partial result, no "decrypt
then check", no window in which unauthenticated plaintext
exists. That is what makes an AEAD safe where a composed
construction is not (chapter 05's padding oracle).
```

AES-GCM

```
/* THE CONSTRUCTION: AES-CTR for confidentiality, GHASH for
   authentication, sharing one key.                          */

    H = AES_K(0^128)                    /* the GHASH key */
    J0 = nonce ‖ 0^31 ‖ 1               /* for a 96-bit
                                           nonce */
    ciphertext = AES-CTR_K(J0+1, plaintext)
    tag = GHASH_H(aad ‖ ciphertext ‖ lengths) XOR AES_K(J0)

/* GHASH is polynomial evaluation in GF(2^128) - the same
   universal-hashing idea as Poly1305 (chapter 08), and with
   the same one-time-key requirement.

/* WHY IT WON:
     · CTR mode is fully parallelisable
     · GHASH maps onto the PMULL (ARM) and PCLMULQDQ (x86)
       carry-less multiply instructions
     · with AES-NI or the ARM crypto extension, roughly
       1 cycle/byte
     · it is in every standard: TLS, IPsec, SSH, 802.11ac,
       NVMe, storage

/* THE NONCE RULE, and it is absolute:

     NEVER REUSE A (key, nonce) PAIR.

   Reuse does not merely leak the XOR of the plaintexts, as
   it would with plain CTR. It reveals H, THE
   AUTHENTICATION KEY - because two tags give two
   polynomial equations in H. The attacker can then FORGE
   ARBITRARY MESSAGES under that key.

   This is the single most damaging misuse in modern
   cryptography, and it has happened in shipped TLS stacks,
   VPNs and IoT devices.

/* THE LIMITS:
     96-bit nonce  the only size that avoids an extra GHASH
                   step; always use exactly 12 bytes
     2^32 - 2 blocks (about 64 GB) per (key, nonce)
     2^32 messages per key before the birthday bound on
       random nonces becomes uncomfortable - use a counter

/* TRUNCATED TAGS: GCM allows 96, 104, 112, 120 or 128 bits.
   USE 128. Short tags weaken the forgery bound far more than
   the arithmetic suggests, because a successful forgery with
   a short tag can leak information about H.              */
```

ChaCha20-Poly1305

```
/* RFC 8439. ChaCha20 for confidentiality (chapter 06),
   Poly1305 for authentication (chapter 08).              */

    one-time key (r, s) = ChaCha20(key, nonce, counter = 0)
                          [first 32 bytes]
    ciphertext          = ChaCha20(key, nonce, counter = 1)
                          XOR plaintext
    tag = Poly1305(r, s,
                   aad ‖ pad16 ‖ ciphertext ‖ pad16 ‖
                   len(aad) ‖ len(ciphertext))

/* THE PADDING AND LENGTH FIELDS make the encoding
   unambiguous - without them an attacker could shift bytes
   between the AAD and the ciphertext (chapter 08).

/* THE ONE-TIME KEY IS DERIVED FROM THE NONCE, which is what
   makes Poly1305's one-time requirement satisfiable.

/* WHY USE IT:
     · 3-4x faster than SOFTWARE AES-GCM
     · CONSTANT TIME with no effort - ARX, no tables
       (chapter 19)
     · small: a few kilobytes of code total
     · no timing variability, so no cache-attack surface

/* AND XCHACHA20-POLY1305, which is the version to prefer:
     192-bit nonce, derived via HChaCha20.
     RANDOM NONCES ARE SAFE INDEFINITELY - a 192-bit random
     nonce will not collide before the heat death of
     anything. That removes the entire nonce-management
     problem, which is where most AEAD misuse comes from.

/* THE PRACTICAL SPLIT: TLS 1.3 offers both AES-GCM and
   ChaCha20-Poly1305 and negotiates. A server with AES-NI
   picks GCM; a phone without AES acceleration picks
   ChaCha20. Both are correct choices for their hardware. */
```

THE DECISION, IN FOUR LINES

```
hardware AES available          -> AES-GCM (or AES-GCM-SIV)
software only                   -> XChaCha20-Poly1305
nonce uniqueness not guaranteed -> AES-GCM-SIV or XChaCha20
very constrained hardware       -> AES-CCM, or ASCON
```

And in all four cases: call the AEAD API, not a cipher plus a MAC you compose yourself.

The others

AEAD	Notes
AES-GCM	The standard. Fast with hardware. Absolutely nonce-critical
ChaCha20-Poly1305	Fast in software, constant time. 96-bit nonce
XChaCha20-Poly1305	192-bit nonce — random nonces are safe. **The easiest correct choice**

AEAD	Notes
AES-GCM-SIV	**Nonce-misuse resistant.** Two passes, so it cannot stream
AES-CCM	CTR + CBC-MAC. Only needs the encrypt direction — good on tiny hardware. 802.15.4, Zigbee, Bluetooth LE, WPA2
AES-OCB	One pass, very fast. Patents expired
AES-EAX	CTR + CMAC. Simple, streaming, no patents
ASCON	**NIST lightweight standard (2023).** Sponge-based, tiny hardware
AEGIS, MORUS	AES-round-based, extremely fast where AES-NI exists
Deoxys-II	CAESAR winner for misuse resistance

```
/* --- AES-CCM, worth understanding for embedded --- */
    CBC-MAC over the nonce, AAD and plaintext -> tag
    CTR mode to encrypt the plaintext AND the tag

/* IT NEEDS ONLY THE AES ENCRYPT DIRECTION, so the
   implementation is half the size - which matters when the
   whole crypto library must fit in 8 KB. And it is what
   every 802.15.4-derived protocol uses, so if you are doing
   Zigbee, Thread, Bluetooth LE or Matter, you are using CCM
   whether you chose it or not.

   THE COSTS: two passes over the data (so the length must be
   known in advance), not parallelisable, and it is
   MAC-then-encrypt in structure - which is provably fine
   here because it was analysed as a whole, but is a reminder
   that "use a standard construction" beats "apply a rule of
   thumb".

/* --- AES-GCM-SIV, and when to reach for it --- */
    tag = POLYVAL(auth_key, aad ‖ plaintext ‖ lengths)
    nonce_for_ctr = tag
    ciphertext = AES-CTR(enc_key, tag, plaintext)

/* The tag is computed FIRST, over the plaintext, and then
   used as the CTR nonce. So repeating a nonce with
   DIFFERENT plaintexts still gives different ciphertexts;
   the only leak is that identical (nonce, plaintext) pairs
   produce identical ciphertexts.

   USE IT WHEN NONCE UNIQUENESS CANNOT BE GUARANTEED:
     · a device that may be restored from a backup
     · a stateless server with no shared counter
     · firmware whose counter resets on reflash (chapter 02)
     · any design where "the counter is monotonic" is a hope
       rather than a mechanism

   THE COST: two passes, so no streaming, and the plaintext
   must be buffered.                                    */
```

Using one correctly

```
/* --- THE HIGH-LEVEL API, which is the recommendation --- */
    #include <sodium.h>

    /* KEY: from a KDF or the CSPRNG, never a password
       directly (chapter 10) */
    unsigned char key[crypto_aead_xchacha20poly1305_ietf_KEYBYTES];
    crypto_aead_xchacha20poly1305_ietf_keygen(key);

    unsigned char nonce[crypto_aead_xchacha20poly1305_ietf_NPUBBYTES];
    randombytes_buf(nonce, sizeof nonce);   /* 192 bits:
                                               random is safe */

    unsigned long long ct_len;
    crypto_aead_xchacha20poly1305_ietf_encrypt(
            ciphertext, &ct_len,
            plaintext, plaintext_len,
            header, header_len,             /* the AAD */
            NULL, nonce, key);

    /* transmit: nonce ‖ header ‖ ciphertext (tag included) */

    /* DECRYPT - and this is the whole error handling: */
    if (crypto_aead_xchacha20poly1305_ietf_decrypt(
            decrypted, &pt_len, NULL,
            ciphertext, ct_len,
            header, header_len,
            nonce, key) != 0) {
            /* FORGED, CORRUPTED, OR WRONG KEY.
               DISCARD EVERYTHING. Do not use any bytes of
               `decrypted`. Do not report which check failed.
               Do not log the ciphertext. */
            return -1;
    }

/* --- ON EMBEDDED, mbedTLS --- */
    mbedtls_gcm_context ctx;
    mbedtls_gcm_init(&ctx);
    mbedtls_gcm_setkey(&ctx, MBEDTLS_CIPHER_ID_AES, key, 256);

    mbedtls_gcm_crypt_and_tag(&ctx, MBEDTLS_GCM_ENCRYPT,
                              len, nonce, 12,
                              aad, aad_len,
                              input, output,
                              16, tag);      /* FULL 16-byte
                                                tag */

    /* and decryption, which checks the tag internally: */
    if (mbedtls_gcm_auth_decrypt(&ctx, len, nonce, 12,
                                 aad, aad_len, tag, 16,
                                 input, output) != 0)
            return ERROR;                   /* FAIL CLOSED */

    mbedtls_gcm_free(&ctx);

/* --- THE RULES, restated because each is a real failure --- */

  1. NEVER REUSE A (key, nonce) PAIR. Use a persistent
     counter, or a 192-bit random nonce.

  2. VERIFY BEFORE USING ANY PLAINTEXT. Not "decrypt then
```

```
   check" - a correct API does not give you the plaintext
   until the tag verified. If yours does, treat the buffer
   as untrusted until the check returns.

3. AUTHENTICATE THE HEADERS via the AAD. Anything that
   changes the meaning of the message must be bound to it.

4. USE THE FULL TAG LENGTH.

5. INCLUDE A SEQUENCE NUMBER, in the AAD, and reject
   replays (chapter 08). An AEAD gives no freshness.

6. ROTATE KEYS before the message or byte limits.

7. ON FAILURE, RETURN ONE INDISTINGUISHABLE ERROR, and
   rate-limit.                                          */
```

Where it still goes wrong

```
/* An AEAD removes most of the classic failures. These
   remain: */

   1. NONCE REUSE - by far the most common, and the most
      damaging. Sources: a hardcoded IV, a counter in
      volatile RAM, a device cloned from an image, a random
      nonce that is too short, a counter that resets.

   2. NOT AUTHENTICATING THE AAD YOU MEANT TO. Sending a
      header outside the AAD makes it forgeable.

   3. USING THE PLAINTEXT BEFORE VERIFICATION. Some
      streaming APIs release plaintext incrementally. If you
      act on it before the final tag check, you have
      re-created the padding-oracle situation.

   4. TREATING THE TAG AS OPTIONAL - truncating it, or
      skipping verification "for performance".

   5. NO REPLAY PROTECTION. A valid message replays forever.

   6. KEY REUSE ACROSS CONTEXTS - the same key for both
      directions, or for two protocols. Derive separate keys
      (chapter 10).

   7. EXCEEDING THE LIMITS - 64 GB per GCM nonce, and the
      message-count bounds.

   8. ENCRYPTING THE LENGTH AWAY BUT NOT THE PATTERN. An
      AEAD hides content, not size or timing. A protocol
      whose message lengths reveal the content - a keyboard
      remote, a voice codec - needs padding as well.

   9. AND THE ONE PEOPLE FORGET: AN AEAD DOES NOT
      AUTHENTICATE THE SENDER'S IDENTITY, only that they
      hold the key. If two parties share a key, either could
      have sent any message (chapter 01). For a real identity
      claim you need a signature or a properly authenticated
      key exchange (chapters 13 and 15).                */
```

How it shows up

Symptom	Cause
Forged messages accepted	GCM nonce reuse leaked the authentication key
Two ciphertexts XOR to plaintext	Nonce reuse
Header modified without detection	Not included in the AAD
Old messages replayed	No sequence number — AEADs give no freshness
Decryption fails after 64 GB	GCM block limit for one nonce
Interop failure on the tag	Different tag lengths, or AAD not byte-identical
Works one way, fails the other	Same key both directions, or nonce spaces overlapping
Plaintext acted on before verification	A streaming API used incorrectly
Attacker infers content from message sizes	An AEAD hides content, not length — pad
Nonce repeats after a factory reset	Counter in volatile storage — use GCM-SIV or XChaCha20

Check yourself

- ✓ State the AEAD interface and what associated data is for.
- ✓ Explain why the failure mode — no partial plaintext — is the security property.
- ✓ Describe AES-GCM's construction and why GCM nonce reuse is worse than CTR nonce reuse.
- ✓ State GCM's nonce size, tag size and message limits.
- ✓ Describe ChaCha20-Poly1305 and say why the length fields matter.
- ✓ Explain what XChaCha20 changes and why it removes a class of bug.
- ✓ Say when AES-CCM is the right choice and what it costs.
- ✓ Explain nonce-misuse resistance and four situations that call for it.
- ✓ Write a correct encrypt and a correct fail-closed decrypt.
- ✓ List the seven usage rules and the nine remaining failure modes.
- ✓ Explain what an AEAD does *not* authenticate.

Key derivation and password hashing

Turning one secret into many, and turning a low-entropy password into something that survives an offline attack. Two problems that look similar and need completely different tools.

Two different problems

```
PROBLEM 1: KEY DERIVATION (KDF)
  input:  a HIGH-ENTROPY secret
          - a Diffie-Hellman shared secret
          - a master key from a fuse or an OTP region
          - a random 256-bit key
  want:   several independent keys, each bound to a
          purpose
  cost:   SHOULD BE FAST. There is no offline guessing
          attack on 256 bits of entropy, so slowing down
          derivation buys nothing and costs you boot time.
  tool:   HKDF

PROBLEM 2: PASSWORD HASHING (PBKDF)
  input:  a LOW-ENTROPY secret
          - "hunter2", a 6-digit PIN, a passphrase
          - maybe 20-40 bits of real entropy
  want:   a key, and resistance to an attacker who has
          stolen the stored value and can guess offline
  cost:   MUST BE DELIBERATELY EXPENSIVE. That is the
          entire mechanism. If verification takes 100 ms
          on your hardware, an attacker's GPU farm gets
          far fewer guesses per second.
  tool:   Argon2id, scrypt, or PBKDF2

USING THE WRONG ONE:
  HKDF on a password   -> a GPU tries billions of guesses
                          per second; the password falls
                          in minutes
  Argon2 on a DH secret -> 100 ms and 64 MB of RAM wasted
                          per handshake, for no security
                          gain at all

AND THE WORST OPTION, still common in shipped firmware:
  SHA256(password) -> a rainbow table, or a GPU doing
  10^10 guesses/second. Not a password hash. Never.
```

HKDF

```
/* RFC 5869. Two steps, and the separation is deliberate. */

/* --- EXTRACT: concentrate whatever entropy exists into a
       uniform pseudorandom key --- */
    PRK = HMAC(salt, input_key_material)

    /* The salt is NOT secret and MAY be absent (a string of
       zeros). Its job is to make the extraction independent
       across contexts, not to hide anything.

       This step matters when the input is not uniform - a
       raw Diffie-Hellman output, for instance, is a group
       element with structure, not 256 uniform bits.       */

/* --- EXPAND: stretch the PRK into as much keying material
       as needed, each piece bound to a label --- */
    T(1) = HMAC(PRK, info ‖ 0x01)
    T(2) = HMAC(PRK, T(1) ‖ info ‖ 0x02)
    ...
    OKM  = first L bytes of T(1) ‖ T(2) ‖ ...

/* THE `info` PARAMETER IS THE IMPORTANT ONE and the one
   people leave empty. It is a CONTEXT STRING that makes each
   derived key independent. Compromise of one key tells you
   nothing about the others.                                 */

    #include <mbedtls/hkdf.h>

    uint8_t k_enc[32], k_mac[32], k_iv[12];
    const mbedtls_md_info_t *sha256 =
        mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);

    mbedtls_hkdf(sha256,
                 salt, salt_len,
                 shared_secret, secret_len,
                 (const uint8_t *)"fw-update v1 encryption", 23,
                 k_enc, sizeof k_enc);

    mbedtls_hkdf(sha256, salt, salt_len,
                 shared_secret, secret_len,
                 (const uint8_t *)"fw-update v1 mac", 16,
                 k_mac, sizeof k_mac);

/* OR, MORE EFFICIENTLY, ONE EXTRACT AND SEVERAL EXPANDS: */
    uint8_t prk[32];
    mbedtls_hkdf_extract(sha256, salt, salt_len,
                         ikm, ikm_len, prk);

    mbedtls_hkdf_expand(sha256, prk, 32,
                        (const uint8_t *)"c2s enc", 7,
                        k_c2s_enc, 32);
    mbedtls_hkdf_expand(sha256, prk, 32,
                        (const uint8_t *)"s2c enc", 7,
                        k_s2c_enc, 32);
    /* ... and so on for each direction and purpose      */

/* GOOD CONTEXT STRINGS INCLUDE:
     · the protocol name and VERSION
     · the purpose ("encryption", "mac", "iv")
     · the DIRECTION ("client-to-server")
```

```
     · the session or device identifier, where it exists

   "myapp v2 device-4711 client-to-server encryption"

   INCLUDING THE VERSION MEANS a protocol upgrade derives
   entirely different keys, so a downgrade attack cannot
   reuse material across versions.                        */
```

ONE MASTER KEY, MANY DERIVED KEYS

A device with a single hardware root key should never use that key directly for anything. Derive: one key for firmware decryption, one for attestation, one for storage, one per peer session. Each has a distinct `info` string. Compromise of the storage key then tells an attacker nothing about the firmware key, and the root key never leaves the key-derivation step — which is exactly what a hardware key store enforces for you (chapter 18).

Password hashing

```
/* THE THREAT: your database or flash image is stolen. The
   attacker now guesses offline, as fast as their hardware
   allows. Your only defence is making each guess expensive.

/* --- ARGON2id: the current recommendation --- */
    Winner of the Password Hashing Competition, 2015.
    THREE cost parameters:

      m  MEMORY, in KiB    - the important one
      t  TIME (iterations)
      p  PARALLELISM (lanes)

    MEMORY-HARDNESS IS THE POINT. A GPU has thousands of
    cores but limited memory bandwidth and per-core memory.
    Requiring 64 MiB per guess reduces a GPU's advantage from
    ~1000x to single digits, and makes a dedicated ASIC
    expensive rather than trivial.

    THE THREE VARIANTS:
      Argon2d   data-dependent addressing - fastest, but
                side-channel exposed (chapter 19)
      Argon2i   data-independent - side-channel resistant,
                weaker against time-memory tradeoffs
      Argon2id  HYBRID: first pass independent, rest
                dependent. USE THIS ONE.

    /* libsodium */
    #include <sodium.h>

    char hashed[crypto_pwhash_STRBYTES];
    if (crypto_pwhash_str(
            hashed, password, password_len,
            crypto_pwhash_OPSLIMIT_MODERATE,   /* t */
            crypto_pwhash_MEMLIMIT_MODERATE)   /* 256 MiB */
        != 0)
            return OUT_OF_MEMORY;

    /* `hashed` is a self-describing string containing the
       algorithm, the parameters and the SALT. Store it as
       is. */

    /* verify - constant time internally */
    if (crypto_pwhash_str_verify(hashed, password,
                                 password_len) != 0)
            return AUTH_FAILED;

    /* AND: derive a KEY from a password */
    uint8_t key[32];
    crypto_pwhash(key, sizeof key, password, password_len,
                  salt,                    /* 16 random bytes,
                                              STORED */
                  opslimit, memlimit,
                  crypto_pwhash_ALG_ARGON2ID13);

/* --- SCRYPT: memory-hard, older, widely available --- */
    N  cost (a power of two; 2^15 = 32768 is reasonable)
    r  block size (8)
    p  parallelism (1)
    memory used ~= 128 * N * r bytes
```

```
/* --- PBKDF2: only when a standard requires it --- */
    HMAC repeated a large number of times. NOT memory-hard,
    so GPUs and ASICs get their full advantage. Still
    infinitely better than a bare hash.

    mbedtls_pkcs5_pbkdf2_hmac_ext(
            MBEDTLS_MD_SHA256,
            password, password_len,
            salt, 16,
            600000,              /* OWASP 2023 minimum for
                                    SHA-256 */
            32, key);

    USE IT WHEN: FIPS compliance demands it, a file format
    specifies it, or the device genuinely cannot spare the
    RAM for Argon2.                                       */
```

Choosing parameters

Setting	Argon2id	PBKDF2-SHA256
Server login	m=64 MiB, t=3, p=4	600 000 iterations
Disk encryption	m=1 GiB, t=4, p=4	—
Phone / tablet	m=64 MiB, t=3, p=2	210 000
Linux-class embedded (256 MB RAM)	m=32 MiB, t=4, p=1	310 000
MCU (256 KB RAM)	m=16 KiB, t=32 (weak)	100 000, or a rate limit instead

```
/* THE METHOD, not the table: MEASURE ON YOUR HARDWARE and
   pick the largest parameters that fit your latency budget.

     interactive login        ~100 ms  is the usual target
     disk unlock at boot      ~1 s     is acceptable
     a background operation   several seconds

/* AND FOR CONSTRAINED DEVICES, THE HONEST ANSWER: a 4-digit
   PIN has 13 bits of entropy. NO KDF SAVES IT. 10,000
   guesses is nothing, whatever the per-guess cost. The
   defence must be elsewhere:

     · A HARDWARE-BACKED ATTEMPT COUNTER that wipes the key
       after N failures - which is exactly what a secure
       element or TPM provides, and what phone secure
       enclaves do (chapter 18)
     · A DEVICE-UNIQUE SECRET mixed into the derivation, so
       the attacker must have the physical device:
           key = Argon2id(PIN ‖ device_secret, salt, ...)
       where device_secret comes from fuses or a PUF and
       never leaves the secure world (the isolation track)
     · RATE LIMITING with exponential backoff

   THE KDF BUYS TIME. IT DOES NOT CREATE ENTROPY.        */
```

Salts, peppers, and upgrades

```
/* --- SALT: unique per password, stored alongside it --- */
    · 16 random bytes, from the CSPRNG (chapter 02)
    · NOT SECRET
    · UNIQUE PER USER, generated fresh each time the
      password is set

    WHY: without a salt, identical passwords hash
    identically. An attacker sees which users share a
    password, and precomputes ONE table that breaks every
    account at once - the rainbow table. With per-user salts,
    each password must be attacked separately.

/* --- PEPPER: a site-wide secret, NOT in the database --- */
    stored_hash = Argon2id(password ‖ pepper, salt, ...)

    Keep the pepper in an HSM, a secure element, or an
    environment variable outside the database. A stolen
    DATABASE alone is then useless. It defends against
    exactly one thing - database-only compromise - and it is
    cheap, so it is usually worth having.

/* --- REHASHING ON LOGIN, the migration pattern --- */
    if (verify(stored, password) == OK) {
            if (needs_rehash(stored, current_params)) {
                    /* we have the plaintext password RIGHT
                       NOW and only right now */
                    new_hash = hash(password, current_params);
                    store(new_hash);
            }
            grant_access();
    }

    This is how you raise cost parameters over time, or
    migrate from PBKDF2 to Argon2id, without asking anyone to
    reset a password.

/* --- AND THE VERIFICATION DISCIPLINE --- */
    · compare in constant time (the library does this)
    · take the SAME TIME for an unknown username as a known
      one - otherwise you have a user-enumeration oracle.
      Hash a dummy value when the user does not exist.
    · never log the password, and never put it in an error
      message or a crash dump
    · zero the plaintext buffer as soon as it is consumed
      (chapter 19 - and note the compiler may remove a
      plain memset; use explicit_bzero or
      mbedtls_platform_zeroize)                          */
```

How it shows up

Symptom	Cause
Stolen database cracked in hours	A bare hash, or an unsalted one, instead of a password KDF
Identical hashes for different users	No salt, or a constant salt
Handshake takes 100 ms per connection	Argon2 used where HKDF belongs
Compromising one key exposes others	No `info` context separation in HKDF

Symptom	Cause
Login timing reveals valid usernames	No dummy hash on the unknown-user path
Argon2 fails on the device	Memory parameter exceeds available RAM
PIN brute-forced despite a strong KDF	Low entropy — needs a hardware attempt counter
Password recoverable from a core dump	Buffer not zeroed, or the memset optimised away
Cannot raise cost parameters	No rehash-on-login path
Downgrade attack reuses old session keys	Protocol version not in the KDF context string

Check yourself

- ✓ State the two problems and why one tool must be fast and the other slow.
- ✓ Explain HKDF's extract and expand steps and what each is for.
- ✓ Write a good `info` string and say what each component prevents.
- ✓ Explain why memory-hardness defeats GPUs, and pick between Argon2i, 2d and 2id.
- ✓ Choose parameters for a server, a phone and a 256 KB MCU, and justify the method.
- ✓ Explain why no KDF can save a 4-digit PIN, and name three things that can.
- ✓ Explain what a salt prevents and what a pepper adds.
- ✓ Describe rehash-on-login and why it is the only practical migration path.
- ✓ Explain the user-enumeration timing oracle and its fix.
- ✓ Say why a plain `memset` may not clear a password buffer.

Number theory for public-key cryptography

The minimum mathematics needed to understand what RSA, Diffie–Hellman and elliptic curves actually do — and, more importantly, what makes them hard to break and easy to implement wrongly.

Modular arithmetic

```
a mod n  is the remainder of a divided by n, always in
the range [0, n-1].

ARITHMETIC MODULO n WRAPS:
     (a + b) mod n
     (a * b) mod n
     (a ^ e) mod n

All of public-key cryptography lives inside one of these
wrapped worlds, because they are FINITE and because some
operations in them are EASY FORWARDS AND HARD BACKWARDS.

CONGRUENCE:  a ≡ b (mod n)  means n divides (a − b).

THE KEY STRUCTURES:

  Z_n        the integers 0..n-1 with + and *
  Z_n*       the elements of Z_n that have a MULTIPLICATIVE
             INVERSE — exactly those coprime to n

WHEN n IS PRIME p:
  every non-zero element is invertible, so Z_p is a FIELD.
  You can add, subtract, multiply AND DIVIDE. This is what
  makes prime moduli so useful.

WHEN n = p*q (RSA):
  the elements sharing a factor with n have no inverse —
  and finding one of them means factoring n, which is the
  hard problem RSA rests on.

THE PRACTICAL POINT FOR IMPLEMENTERS: these numbers are
2048 to 4096 bits wide. There is no CPU instruction for
them. Every operation is a loop over machine words, and
the TIMING of that loop is where the secrets leak
(chapter 19).
```

Euclid, inverses and the extended algorithm

```
/* --- GCD: the oldest algorithm still in daily use --- */
    gcd(a, b):
        while b != 0:
            a, b = b, a mod b
        return a

    gcd(252, 198) = gcd(198, 54) = gcd(54, 36)
                  = gcd(36, 18) = gcd(18, 0) = 18

/* --- EXTENDED EUCLID: also finds x, y with
       a*x + b*y = gcd(a, b)  --- */

    ext_gcd(a, b):
        if b == 0: return (a, 1, 0)
        (g, x, y) = ext_gcd(b, a mod b)
        return (g, y, x - (a / b) * y)

/* WHY THIS MATTERS: THE MODULAR INVERSE.

       a * a^-1 ≡ 1  (mod n)

   Set b = n. If gcd(a, n) = 1, then a*x + n*y = 1, so
   a*x ≡ 1 (mod n), and x is the inverse.

     inverse(a, n):
         (g, x, _) = ext_gcd(a, n)
         if g != 1: FAIL - no inverse exists
         return x mod n

/* USED FOR:
     · computing the RSA private exponent d from e
       (chapter 12)
     · converting projective to affine coordinates on an
       elliptic curve (chapter 14)
     · the s value in ECDSA signing (chapter 15)
     · Montgomery arithmetic setup

/* THE SIDE-CHANNEL WARNING: the loop above runs a
   DATA-DEPENDENT NUMBER OF ITERATIONS. Used on a secret, it
   leaks. Real libraries use a constant-time binary GCD, or
   Fermat's little theorem (a^(p-2) mod p) when the modulus
   is prime. Never write your own for secret inputs.      */
```

Fermat, Euler, and why exponents wrap

```
/* --- FERMAT'S LITTLE THEOREM --- */
    For prime p and a not divisible by p:

        a^(p-1) ≡ 1  (mod p)

    Consequences:
        a^p     ≡ a       (mod p)
        a^(p-2) ≡ a^-1    (mod p)   /* a constant-time
                                       inverse! */

/* --- EULER'S TOTIENT ϕ(n) --- */
    ϕ(n) = how many integers in [1, n] are coprime to n

        ϕ(p)     = p - 1              /* p prime */
        ϕ(p*q)   = (p-1)(q-1)         /* p, q distinct
                                         primes */
        ϕ(p^k)   = p^k - p^(k-1)

/* --- EULER'S THEOREM: the generalisation --- */
        a^ϕ(n) ≡ 1  (mod n)   when gcd(a, n) = 1

/* THIS IS EXACTLY WHY RSA WORKS.

   Choose e and d with  e*d ≡ 1 (mod ϕ(n)),
   so  e*d = 1 + k*ϕ(n)  for some k.

   Then:
        (m^e)^d = m^(e*d) = m^(1 + k*ϕ(n))
                = m * (m^ϕ(n))^k
                ≡ m * 1^k
                ≡ m   (mod n)

   Encryption and decryption are inverse operations because
   the exponents wrap modulo ϕ(n). That is the whole of RSA's
   correctness argument.

/* --- CARMICHAEL'S λ(n) --- */
    The SMALLEST exponent that works:
        λ(p*q) = lcm(p-1, q-1)
    It divides ϕ(n), so d computed mod λ(n) is smaller and
    decryption is faster. Modern RSA key generation uses λ.

/* --- THE CHINESE REMAINDER THEOREM --- */
    Knowing x mod p and x mod q, with p, q coprime, uniquely
    determines x mod (p*q).

    RSA-CRT makes private-key operations ~4x FASTER:
        m_p = c^(d mod p-1) mod p
        m_q = c^(d mod q-1) mod q
        m   = recombine(m_p, m_q)

    Two exponentiations on half-size numbers beat one on the
    full size, because modular exponentiation is roughly
    cubic in the bit length.

    AND THE FAMOUS DANGER: if a FAULT corrupts exactly one of
    m_p or m_q, the attacker computes
        gcd(m^e - c, n)
    and RECOVERS A PRIME FACTOR from a single faulty
    signature. This is the Bellcore attack. Real
```

```
   implementations VERIFY the signature before releasing it,
   which is why you sometimes see a public-key operation
   inside a signing routine.                              */
```

TWO HARD PROBLEMS HOLD UP EVERYTHING

```
FACTORING:  given n = p*q, find p and q.
            -> RSA (chapter 12)

DISCRETE LOG: given g and g^x mod p, find x.
            -> Diffie-Hellman (13), DSA, ElGamal
            and the same problem on an elliptic
            curve group -> ECC (14), ECDSA, Ed25519

Neither is PROVEN hard. Both have resisted centuries and
decades respectively of effort. Both FALL COMPLETELY to a
sufficiently large quantum computer (chapter 20).
```

Groups, generators and orders

```
/* A GROUP is a set with an operation that is:
     closed, associative, has an identity, and has inverses.

   For cryptography, always FINITE and usually ABELIAN
   (commutative).

/* --- THE MULTIPLICATIVE GROUP MOD p --- */
    Z_p* = {1, 2, ..., p-1} under multiplication mod p
    ORDER (size) = p - 1

    A GENERATOR g produces every element by repeated
    multiplication:
        g^1, g^2, ..., g^(p-1) = all of Z_p*
    Such a g is also called a PRIMITIVE ROOT.

    Example, p = 7:  3^1=3, 3^2=2, 3^3=6, 3^4=4, 3^5=5,
                     3^6=1
    3 generates all six non-zero elements, so 3 is a
    generator mod 7.

/* --- THE ORDER OF AN ELEMENT --- */
    The smallest k with g^k = 1. LAGRANGE'S THEOREM says it
    always divides the group order.

/* WHY THIS MATTERS IN PRACTICE:

   1. SAFE PRIMES. For Diffie-Hellman, choose p = 2q + 1
      with q also prime. Then the only subgroup orders are
      1, 2, q and 2q. An attacker cannot trap you in a small
      subgroup.

   2. SMALL-SUBGROUP CONFINEMENT ATTACKS. If p-1 has many
      small factors and you accept an arbitrary peer key, the
      attacker sends an element of small order. Your shared
      secret then lies in a tiny set, and they learn your
      private exponent modulo that small order. Repeat with
      several small factors, apply CRT, and they recover the
      key.

      THE DEFENCE: use a safe prime, VALIDATE the peer's
      public key (1 < y < p-1 and y^q ≡ 1), or use a curve
      designed so the check is unnecessary - X25519 clamps
      and has a cofactor of 8 (chapter 14).

   3. THE DISCRETE LOG PROBLEM. Given g, p and g^x mod p,
      find x. Best classical algorithms:
        · baby-step giant-step / Pollard rho: O(sqrt(order))
          -> so the SUBGROUP ORDER must be at least 2^256 for
             128-bit security
        · index calculus / number field sieve: SUBEXPONENTIAL
          in p, and this is why finite-field DH needs a
          3072-bit p for 128-bit security while an elliptic
          curve needs only 256 bits - index calculus does not
          apply to elliptic curve groups.              */
```

Generating primes

```
/* RSA and finite-field DH need large random primes.
   Deterministic primality proving is slow; probabilistic
   testing is what everyone uses.                          */

/* --- MILLER-RABIN --- */
    Write n - 1 = 2^s * d with d odd.
    Pick a random base a.
        compute a^d mod n
        if it is 1 or n-1: n MIGHT be prime
        else square up to s-1 times looking for n-1
        if never found: n is DEFINITELY COMPOSITE

    Each round has at most a 1/4 chance of being fooled, so
    k rounds give an error probability below 4^-k. For random
    candidates the true error rate is far lower; 40-64 rounds
    is standard, and FIPS 186-5 tabulates the requirement by
    key size.

/* --- THE GENERATION LOOP --- */
    do {
        p = random_odd_number(bits);
        p |= (1 << (bits-1));      /* force the top bit, so
                                      the modulus has the
                                      full length */
        p |= 1;                     /* force odd */
    } while (divisible_by_small_prime(p) ||
             !miller_rabin(p, rounds));

    /* the small-prime sieve first: trial division by the
       primes below ~2000 rejects ~90% of candidates for
       almost no cost, and Miller-Rabin is expensive */

/* --- WHAT GOES WRONG, and it has gone wrong at scale --- */

   1. A WEAK RNG. In 2012 researchers found that a
      meaningful fraction of TLS and SSH keys on the public
      internet shared a prime factor with another key -
      because embedded devices generated keys at first boot
      with almost no entropy (chapter 02). Computing
      gcd(n1, n2) over the whole set factored them all.
      THE FIX: never generate a key before the entropy pool
      is ready.

   2. THE PRIMES TOO CLOSE TOGETHER. If |p - q| is small,
      FERMAT FACTORISATION finds them immediately. Enforce a
      minimum difference.

   3. p-1 OR p+1 SMOOTH (having only small factors). Pollard
      p-1 and Williams p+1 then factor n cheaply. "Strong
      primes" avoid this; with random 1024-bit primes the
      probability is negligible, so modern standards no
      longer require the check.

   4. REUSED PRIMES ACROSS KEYS - same failure as (1), and
      the same gcd attack.

/* AND THE ADVICE: DO NOT IMPLEMENT KEY GENERATION. Use the
   library's, seeded from a real entropy source, and check
   that it is seeded before you call it.                   */
```

How it shows up

Symptom	Cause
Two devices' keys share a factor	Key generated before the entropy pool was seeded
A single faulty signature reveals a prime	RSA-CRT without verify-after-sign — the Bellcore attack
Private exponent recovered over many handshakes	Small-subgroup confinement — no key validation, unsafe prime
Modular inverse returns garbage	`gcd(a, n) ≠ 1`, so no inverse exists
Key operation timing correlates with the key	Data-dependent GCD or exponentiation loop
Modulus is shorter than requested	Top bit not forced during prime generation
n factored instantly	p and q too close — Fermat factorisation
Key generation takes minutes on an MCU	No small-prime sieve before Miller–Rabin
3072-bit DH needed where 256-bit ECC suffices	Index calculus applies to finite fields, not curves

Check yourself

- ✓ Explain why `Z_p` is a field and `Z_n` for composite `n` is not.
- ✓ Use extended Euclid to find a modular inverse, and say when none exists.
- ✓ Derive RSA correctness from Euler's theorem.
- ✓ Explain what CRT buys RSA and what fault attack it opens.
- ✓ Define a generator and the order of an element, and state Lagrange's theorem.
- ✓ Explain a small-subgroup confinement attack and three defences.
- ✓ Say why finite-field DH needs 3072 bits where a curve needs 256.
- ✓ Describe Miller–Rabin and why a small-prime sieve comes first.
- ✓ Explain the 2012 shared-factor result and what caused it.
- ✓ Name the two hard problems and which primitives rest on each.

RSA

The first practical public-key system, still everywhere in firmware signing and PKI, and the one whose textbook description is dangerously incomplete. The padding is not a detail — it is most of the security.

The core operation

```
KEY GENERATION
  1. pick two large random primes p, q  (1024 bits each
     for a 2048-bit key)
  2. n = p * q                    the MODULUS, public
  3. λ(n) = lcm(p-1, q-1)
  4. e = 65537                    the PUBLIC EXPONENT
  5. d = e^-1 mod λ(n)            the PRIVATE EXPONENT

  PUBLIC KEY  = (n, e)
  PRIVATE KEY = (n, d)  plus p, q and the CRT values
  AND p AND q MUST BE DESTROYED or protected exactly as
  hard as d - either one factors n instantly.

THE OPERATIONS
  "encrypt" / verify:   c = m^e mod n
  "decrypt" / sign:     m = c^d mod n

  They are inverses because e*d ≡ 1 (mod λ(n)) - the
  derivation is in chapter 11.

WHY e = 65537 = 2^16 + 1
  · only two set bits, so verification is 17 squarings and
    one multiply - very fast, which matters because
    verification happens on every boot
  · large enough to avoid the small-exponent attacks that
    killed e = 3
  · TINY e MAKES VERIFICATION FAST AND SIGNING SLOW, which
    is the right trade: a device verifies constantly and
    signs never.

THE ASYMMETRY IN COST
  verify (e=65537)    ~0.1 ms      2048-bit
  sign   (d, w/ CRT)  ~2 ms
  A 4096-bit key is roughly 6-8x slower to sign with, and
  ~3x slower to verify.
```

Why textbook RSA is broken

```
/* "Just compute m^e mod n" fails in at least five distinct
   ways. Every one of these has broken a real system.     */

/* 1. IT IS DETERMINISTIC.
      Same message -> same ciphertext. An attacker who
      suspects the plaintext just encrypts the guess and
      compares. For a small message space - a command byte,
      a PIN, a yes/no - this breaks it completely.        */

/* 2. SMALL MESSAGES DO NOT WRAP.
      With e = 3 and a short m, m^3 < n, so no modular
      reduction happens. The attacker takes an ORDINARY CUBE
      ROOT over the integers and recovers m. No key needed. */

/* 3. IT IS MULTIPLICATIVELY HOMOMORPHIC.
          E(m1) * E(m2) = (m1*m2)^e = E(m1 * m2)  mod n

      So an attacker can BLIND a ciphertext: send c * r^e to
      a decryption oracle, get back m * r, divide by r,
      recover m. This is a total break of any system that
      decrypts attacker-supplied ciphertexts.

      The same property lets an attacker forge signatures:
      given signatures on m1 and m2, they have one on
      m1 * m2.                                            */

/* 4. NO STRUCTURE MEANS NO INTEGRITY.
      Any n-bit value is a "valid" ciphertext, decrypting to
      garbage the receiver cannot distinguish from a real
      message. Padding provides the redundancy that makes
      forgery detectable.                                 */

/* 5. RELATED MESSAGES LEAK.
      Håstad's attack: the same m encrypted to three
      recipients with e = 3 is recoverable by CRT plus a cube
      root. Franklin-Reiter: two messages with a known linear
      relation are both recoverable.                      */

/* THE RULE THAT FOLLOWS:

     NEVER CALL A RAW MODEXP ON A MESSAGE.
     ALWAYS USE A PADDING SCHEME - OAEP or PSS.

   And when you see mbedtls_rsa_public()/mbedtls_rsa_private()
   in code, that is the RAW primitive. The functions you want
   are mbedtls_rsa_rsaes_oaep_encrypt() and
   mbedtls_rsa_rsassa_pss_sign().                      */
```

OAEP: encryption padding

```
/* PKCS#1 v2, RFC 8017. Optimal Asymmetric Encryption
   Padding.                                              */

    EM = 0x00 ‖ maskedSeed ‖ maskedDB

    DB         = lHash ‖ PS(zeros) ‖ 0x01 ‖ message
    seed       = random, hLen bytes
    dbMask     = MGF1(seed, k - hLen - 1)
    maskedDB   = DB XOR dbMask
    seedMask   = MGF1(maskedDB, hLen)
    maskedSeed = seed XOR seedMask

    then  c = EM^e mod n

/* WHAT EACH PART DOES:
     the RANDOM SEED makes encryption probabilistic, killing
       problem (1)
     the two-way MASKING is a Feistel structure: every
       ciphertext bit depends on every plaintext bit
     lHash and the 0x01 SEPARATOR give the redundancy that
       makes an invalid ciphertext detectable, killing (4)
     the FULL-WIDTH ENCODED MESSAGE means m is always close
       to n, killing (2)

/* CAPACITY: k - 2*hLen - 2 bytes.
     2048-bit key, SHA-256:  256 - 64 - 2 = 190 bytes.

   SO YOU CANNOT ENCRYPT A LARGE MESSAGE WITH RSA. Nobody
   does. The pattern is HYBRID ENCRYPTION:

       k_session = random 32 bytes
       c1 = RSA-OAEP(pubkey, k_session)
       c2 = AES-GCM(k_session, plaintext)     /* chapter 09 */
       send c1 ‖ c2

/* --- PKCS#1 v1.5 ENCRYPTION PADDING: DO NOT USE --- */
    EM = 0x00 ‖ 0x02 ‖ random_nonzero_padding ‖ 0x00 ‖ m

    BLEICHENBACHER'S ATTACK (1998): if the receiver reveals -
    by an error message, a timing difference, or a connection
    reset - whether the 0x00 0x02 prefix was correct, the
    attacker decrypts any ciphertext with about a million
    adaptive queries. It has been rediscovered repeatedly:
    ROBOT (2017) found it live in load balancers from several
    major vendors, nineteen years later.

    THE ONLY SAFE HANDLING is to make the failure path
    completely indistinguishable - which is so hard to get
    right that TLS 1.3 REMOVED RSA KEY TRANSPORT ENTIRELY.
    Use OAEP, or better, use ECDH (chapter 13).          */
```

PSS: signature padding

```
/* RSASSA-PSS, and this is the more important half for
   embedded work, because firmware verification is where RSA
   still lives.                                            */

    mHash  = H(message)
    salt   = random, sLen bytes
    M'     = 0x00 * 8 ‖ mHash ‖ salt
    H2     = H(M')
    DB     = PS(zeros) ‖ 0x01 ‖ salt
    EM     = maskedDB ‖ H2 ‖ 0xBC

    signature = EM^d mod n

/* THE RANDOM SALT makes signatures PROBABILISTIC: signing
   the same message twice gives different signatures. That
   is not a bug - it gives PSS a tight security proof that
   v1.5 does not have.

/* --- PKCS#1 v1.5 SIGNATURES: legacy but everywhere --- */
    EM = 0x00 ‖ 0x01 ‖ 0xFF...0xFF ‖ 0x00 ‖ DigestInfo ‖ H(m)

    Deterministic. NOT KNOWN TO BE BROKEN when implemented
    correctly, and mandated by so many existing formats and
    boot ROMs that it is not going away. USE IT WHEN A FORMAT
    REQUIRES IT; use PSS for anything new.

/* AND THE IMPLEMENTATION BUG THAT KEEPS RECURRING -
   BLEICHENBACHER'S SIGNATURE FORGERY, "BERserk":

   A LAZY VERIFIER parses the structure left to right,
   finds the hash, compares it, and IGNORES TRAILING BYTES.
   With e = 3, an attacker can construct a value whose cube
   root has the right prefix and arbitrary garbage after -
   and forge a signature WITH NO PRIVATE KEY.

   THE CORRECT VERIFICATION:
     · RECONSTRUCT the full expected EM from the hash
     · compare the ENTIRE block, byte for byte, in constant
       time
     · never parse-and-extract

   NSS, GnuTLS, Firefox, Chrome and several TLS libraries
   have all shipped this bug. If you are writing a boot ROM
   verifier, this is the bug to look for.               */
```

VERIFY A FIRMWARE SIGNATURE LIKE THIS

```
1. hash the image with SHA-256
2. RSA-verify with PSS (or v1.5 if the format demands)
3. compare the FULL reconstructed encoding
4. check the key against a hash burned in OTP
5. check the version against an anti-rollback counter
6. only then jump to the image
```

Steps 4 and 5 are not part of RSA and are just as important — see the secure boot chapter.

Key sizes and the alternatives

RSA size	Security	Equivalent ECC	Status
1024-bit	~80-bit	P-160	**Broken.** Factorable by a well-funded attacker
2048-bit	~112-bit	P-224	Acceptable to about 2030
3072-bit	~128-bit	P-256	The current recommendation
4096-bit	~140-bit	P-313	Common for long-lived roots
15360-bit	~256-bit	P-521	Impractical — use ECC

```
/* THE SIZE CURVE IS BRUTAL: doubling RSA security multiplies
   the key size by roughly five, because the number field
   sieve is subexponential. ECC scales linearly - 256 bits
   for 128-bit security (chapter 14).

/* SO WHY IS RSA STILL EVERYWHERE IN EMBEDDED?

     · BOOT ROMS ARE MASK-PROGRAMMED and were fixed years
       ago. If the ROM verifies RSA-2048 PKCS#1 v1.5, that
       is what your firmware is signed with, permanently.
     · VERIFICATION IS FAST with e = 65537 - actually faster
       than ECDSA verification, which matters at boot.
     · UNIVERSAL TOOLING support.

   WHERE ECC WINS:
     · signature size: 64 bytes vs 256-512
     · key size: 32 bytes vs 256-512, which matters when the
       public key lives in OTP fuses
     · SIGNING speed
     · key GENERATION: microseconds vs seconds

   AND FOR NEW DESIGNS: use Ed25519 or ECDSA P-256, and plan
   for a post-quantum signature (chapter 20) if the device
   will live past 2035.                                  */
```

Implementation hazards

```
/* --- 1. TIMING ATTACKS ON THE PRIVATE OPERATION --- */
    A naive square-and-multiply branches on each bit of d:

        for each bit of d:
            square
            if bit: multiply          /* <-- LEAKS d */

    Kocher (1996) recovered private keys this way. Brumley
    and Boneh (2003) did it REMOTELY over a network against
    OpenSSL.

    DEFENCES: a Montgomery ladder (constant operation
    sequence), and BLINDING:
        c' = c * r^e mod n
        m' = c'^d mod n
        m  = m' * r^-1 mod n
    The exponentiation now runs on a randomised value, so its
    timing carries no information about c. Every serious
    library blinds by default; check that yours has it
    enabled.

/* --- 2. THE CRT FAULT ATTACK --- */
    One corrupted half of a CRT signature reveals a prime via
    gcd(s^e - m, n) (chapter 11).

    DEFENCE: VERIFY EVERY SIGNATURE BEFORE RELEASING IT.
        sign(); if (verify() != OK) { wipe(); abort(); }
    Costs one cheap public operation. Non-negotiable on
    anything physically accessible.

/* --- 3. LEAKING THE PRIMES --- */
    p and q are as sensitive as d. Zero them, keep them out
    of swap and core dumps, and never log a key structure.

/* --- 4. SMALL PUBLIC EXPONENTS --- */
    e = 3 is legal and fast and has a long history of
    breaking things. Use 65537. If you must accept e = 3,
    padding must be flawless.

/* --- 5. NOT VALIDATING A RECEIVED PUBLIC KEY --- */
    Before using someone's RSA key:
        · n has the expected bit length
        · n is ODD (an even n is trivially factorable)
        · e is odd and 3 <= e < n
        · e is 65537 if your policy says so
    A hostile "public key" with a smooth n turns your
    encryption into a no-op.

/* --- 6. THE CODE PATTERN, mbedTLS --- */
    mbedtls_rsa_context rsa;
    mbedtls_rsa_init(&rsa);
    mbedtls_rsa_set_padding(&rsa, MBEDTLS_RSA_PKCS_V21,
                            MBEDTLS_MD_SHA256);  /* PSS/OAEP */

    ret = mbedtls_rsa_rsassa_pss_verify(
              &rsa, MBEDTLS_MD_SHA256, 32, hash, sig);
    if (ret != 0) {
            /* INVALID. No detail in the error path. */
            return -1;
    }
```

```
    mbedtls_rsa_free(&rsa);

/* NOTE THE ARGUMENT ORDER AND THE PADDING MODE. Calling
   pkcs1_v15_verify on a PSS signature returns an error;
   calling the RAW primitive returns success on garbage. */
```

How it shows up

Symptom	Cause
Same plaintext gives the same ciphertext	No padding, or v1.5 with a broken RNG
Short message recovered without a key	Small `e`, no padding — an integer cube root
Signature forged with no private key	Lazy v1.5 parsing with `e = 3` — BERserk
Ciphertext decrypted with ~10^6 queries	Bleichenbacher oracle in PKCS#1 v1.5 decryption
Private key extracted over the network	Unblinded exponentiation timing
A prime recovered from one signature	CRT fault, no verify-after-sign
"Message too long" at 200 bytes	OAEP capacity — use hybrid encryption
Verification succeeds on random data	The raw primitive called instead of a padded verify
Signing takes 400 ms	CRT not enabled, or a 4096-bit key on a slow core
Interop failure with a valid signature	PSS vs v1.5 mismatch, or a different salt length

Check yourself

- ✓ Write out key generation and say why `p` and `q` must be destroyed.
- ✓ Explain why `e = 65537` and what the cost asymmetry is good for.
- ✓ List five ways textbook RSA breaks, with a concrete attack for each.
- ✓ Explain what each component of OAEP defends against.
- ✓ Compute OAEP capacity for a 2048-bit key and explain hybrid encryption.
- ✓ Describe the Bleichenbacher oracle and why TLS 1.3 dropped RSA key transport.
- ✓ Explain BERserk and write the correct v1.5 verification procedure.
- ✓ Explain blinding and why it defeats timing attacks.
- ✓ Explain the CRT fault attack and the one-line defence.
- ✓ List the checks to run on a received public key.
- ✓ Say why RSA persists in embedded boot ROMs and where ECC wins.

Diffie–Hellman and key exchange

Two parties who have never met agree on a shared secret over a channel an attacker is reading. The most surprising result in cryptography, and the foundation of every modern secure channel.

The basic exchange

```
PUBLIC PARAMETERS: a large prime p, a generator g.
Both are known to everyone, including the attacker.

  ALICE                                  BOB
  a = random secret                      b = random secret
  A = g^a mod p        ---- A ---->
                       <---- B ----      B = g^b mod p

  s = B^a mod p                          s = A^b mod p
    = (g^b)^a                              = (g^a)^b
    = g^(a*b) mod p    ===== SAME =====    = g^(a*b) mod p

THE EAVESDROPPER SEES p, g, A and B.
To compute s they must find a from g^a - the DISCRETE
LOGARITHM PROBLEM (chapter 11). No efficient classical
algorithm is known.

THE SHARED SECRET IS NOT A KEY YET. g^(a*b) mod p is a
structured group element, not uniform bits. RUN IT THROUGH
A KDF:

    k_enc = HKDF(s, salt, "protocol v1 encryption")
    k_mac = HKDF(s, salt, "protocol v1 mac")

Using the raw DH output as an AES key is a real and common
bug (chapter 10).

WHAT THIS BUYS AND WHAT IT DOES NOT:
  · a shared secret over a public channel        YES
  · secrecy against a PASSIVE eavesdropper       YES
  · FORWARD SECRECY, if a and b are ephemeral    YES
  · protection against an ACTIVE attacker        NO
  · authentication of who you are talking to     NO
  · a usable symmetric key without a KDF         NO
```

The man in the middle

```
/* UNAUTHENTICATED DH IS COMPLETELY BROKEN AGAINST AN ACTIVE
   ATTACKER, and this is not a subtle failure.            */

    ALICE          MALLORY          BOB
      A  ------->   (drops A)
                     M1  ------->
                     <-------  B    (drops B)
      <-------  M2

    Alice and Mallory share s1 = g^(a*m1).
    Mallory and Bob   share s2 = g^(m2*b).
    Mallory decrypts everything from Alice, reads it, and
    re-encrypts it to Bob. NEITHER SIDE SEES ANYTHING WRONG.

/* THE FIX IS ALWAYS AUTHENTICATION - proving that the public
   value you received belongs to the party you intended to
   talk to. The options:

   1. SIGN THE EPHEMERAL PUBLIC VALUES with a long-term key
      whose certificate chains to a trusted root. This is
      TLS's ECDHE + certificate (chapters 16, 17).

   2. STATIC-EPHEMERAL DH: one side's key is long-term and
      already known (e.g. burned into the device at
      manufacture), the other's is ephemeral.

   3. A PASSWORD-AUTHENTICATED KEY EXCHANGE (PAKE) - SPAKE2,
      OPAQUE, or the CPace family. Both sides prove knowledge
      of a shared low-entropy secret WITHOUT it being
      brute-forceable offline. This is how a device pairs
      with a phone using a 6-digit code.

   4. OUT-OF-BAND VERIFICATION: derive a short fingerprint
      from the transcript and have humans compare it. The
      "safety number" in a messaging app, or the SSH host key
      fingerprint you were supposed to check.

   5. TOFU - trust on first use. Remember the peer's key and
      alarm if it ever changes. SSH's known_hosts. Weak on
      the first connection, strong afterwards.

/* FOR EMBEDDED DEVICES the usual answer is a
   FACTORY-PROVISIONED IDENTITY: a device key pair generated
   at manufacture, its private half in a secure element, and
   a certificate signed by the vendor CA. The device then
   authenticates every exchange with it (chapter 18).     */
```

FORWARD SECRECY

Use a *fresh, ephemeral* `a` and `b` for every session and destroy them afterwards. An attacker who records all the traffic today and steals the device's long-term key in five years still cannot decrypt those recordings — the ephemeral secrets are gone. This is the "E" in ECDHE, and it is the single most valuable property a protocol design can have. A protocol that encrypts a session key *to* a long-term key (RSA key transport, chapter 12) has no forward secrecy at all, which is why TLS 1.3 removed it.

ECDH and X25519

```
/* The same protocol, in an elliptic curve group instead of
   the integers mod p (chapter 14). Same maths, far smaller
   numbers.                                                  */

    private:  a random scalar d
    public:   the point  Q = d * G
    shared:   d_A * Q_B = d_A * d_B * G = d_B * Q_A

/* SIZES, and this is the whole reason ECC won:
     finite-field DH, 128-bit security:  3072-bit p
                                         -> 384-byte messages
     ECDH P-256, 128-bit security:       -> 32-64 bytes
     X25519:                             -> 32 bytes exactly

/* --- X25519: the one to use --- */
    #include <sodium.h>

    /* each side */
    unsigned char sk[crypto_kx_SECRETKEYBYTES];
    unsigned char pk[crypto_kx_PUBLICKEYBYTES];
    crypto_kx_keypair(pk, sk);

    /* exchange pk over the wire (32 bytes), then: */
    unsigned char rx[crypto_kx_SESSIONKEYBYTES];
    unsigned char tx[crypto_kx_SESSIONKEYBYTES];
    if (crypto_kx_client_session_keys(rx, tx, pk, sk,
                                      server_pk) != 0)
            return BAD_PEER_KEY;

    /* NOTE: SEPARATE rx AND tx KEYS, derived for you. That
       is the key-separation rule from chapter 08 built into
       the API.                                              */

/* THE LOW-LEVEL FORM, when you need the raw shared value: */
    unsigned char q[crypto_scalarmult_BYTES];
    if (crypto_scalarmult(q, my_sk, their_pk) != 0)
            return BAD_PEER_KEY;    /* all-zero output:
                                       a small-order point */

    /* AND THEN ALWAYS: */
    crypto_generichash_state h;
    crypto_generichash_init(&h, NULL, 32, 32);
    crypto_generichash_update(&h, q, 32);
    crypto_generichash_update(&h, client_pk, 32);
    crypto_generichash_update(&h, server_pk, 32);
    crypto_generichash_final(&h, session_key, 32);

/* HASHING BOTH PUBLIC KEYS ALONGSIDE THE SHARED SECRET binds
   the key to the specific exchange. Without it, some
   protocols admit key-reuse and unknown-key-share attacks.

/* WHY X25519 IS THE DEFAULT RECOMMENDATION:
     · every 32-byte string is a valid public key - no point
       validation code to get wrong
     · the Montgomery ladder is naturally constant time
     · clamping removes the small-subgroup problem
     · fast: ~100 microseconds on a Cortex-M4
     · no patents, no NIST-curve provenance questions
     · one function, no parameter choices to get wrong
```

```
   USE P-256 INSTEAD ONLY WHEN a standard, a certificate
   profile, or a hardware accelerator requires it.       */
```

Where key exchange goes wrong

```
/* --- 1. NO AUTHENTICATION --- */
    Covered above. The most common failure in home-grown
    protocols, and it is total.

/* --- 2. NOT VALIDATING THE PEER'S PUBLIC VALUE --- */
    Finite-field DH: reject y <= 1, y >= p-1, and check
    y^q ≡ 1 mod p for the subgroup order q. Otherwise:
    small-subgroup confinement (chapter 11).

    ECDH on a NIST curve: verify the point is ON THE CURVE.
    An INVALID CURVE ATTACK sends a point on a different,
    weaker curve; the scalar multiplication still "works" and
    leaks the private key a few bits at a time. This broke
    several TLS libraries and a Bluetooth implementation.

    X25519: the output being all zeros means a small-order
    input. Check it, or use a library that does.

/* --- 3. REUSING EPHEMERAL KEYS --- */
    Caching a "temporary" key across sessions - usually done
    for performance on a slow MCU - DESTROYS FORWARD SECRECY
    and enables key-recovery attacks against unvalidated
    implementations. If you must cache, cache for seconds,
    not for the device's lifetime.

/* --- 4. USING THE RAW SHARED SECRET AS A KEY --- */
    Always through a KDF, always with a context string.

/* --- 5. NOT BINDING THE TRANSCRIPT --- */
    Authenticate the ENTIRE HANDSHAKE - every message, in
    order, including the negotiated parameters. Otherwise a
    DOWNGRADE ATTACK works: the attacker strips the strong
    ciphersuites from the client's offer and both sides
    happily negotiate the weak one, each believing the other
    only supported it. LOGJAM downgraded TLS to 512-bit
    export DH this way; FREAK did the same to export RSA.

/* --- 6. WEAK OR SHARED PARAMETERS --- */
    LOGJAM again: a huge fraction of the internet used the
    SAME 1024-bit DH group. Precomputation against one group
    - expensive but feasible for a state actor - then breaks
    every connection using it. Use named, modern groups
    (RFC 7919) or, better, use a curve.

/* --- 7. TIMING LEAKS IN THE SCALAR MULTIPLY --- */
    Chapter 19. Use a constant-time implementation; X25519's
    ladder is one by construction.

/* --- 8. NO REPLAY PROTECTION ON 0-RTT DATA --- */
    TLS 1.3's early data can be replayed by an attacker. Only
    send idempotent requests in it.                       */
```

Real handshake patterns

```
/* --- TLS 1.3, simplified (chapter 17) --- */
    ClientHello:  key_share = ephemeral X25519 or P-256 pk
    ServerHello:  key_share = server's ephemeral pk
      -> both derive the handshake secret immediately, so
         the REST OF THE HANDSHAKE IS ENCRYPTED
    Certificate + CertificateVerify: the server SIGNS the
      transcript, authenticating the exchange
    Finished (both sides): a MAC over the whole transcript,
      closing the downgrade hole
    One round trip. Forward secret. Authenticated.

/* --- NOISE PROTOCOL FRAMEWORK --- */
    A construction kit for DH handshakes with a formal naming
    scheme. Used by WireGuard, WhatsApp, Lightning.

      Noise_NN   neither party authenticated
      Noise_NK   the responder's static key is known to the
                 initiator     <-- typical for a device
                                   talking to a known server
      Noise_XX   both authenticate during the handshake
      Noise_IK   initiator knows the responder's static key
                 and sends its own immediately

    EACH PATTERN IS A SHORT SEQUENCE OF DH OPERATIONS, mixed
    into a running hash. WireGuard's whole handshake is about
    150 lines. FOR AN EMBEDDED PROTOCOL, PICKING A NOISE
    PATTERN IS ALMOST ALWAYS BETTER THAN DESIGNING ONE.

/* --- THE DEVICE PROVISIONING PATTERN --- */
    factory:  generate a device key pair inside the secure
              element; the private half NEVER LEAVES
              (chapter 18)
              vendor CA signs the public half -> device cert
    field:    device does ECDHE with the server
              device signs the transcript with its device key
              server validates the cert chain to the vendor
              root and checks the device is not revoked
              server signs its side; device validates against
              a root PINNED IN FIRMWARE

    NOTE THE PINNING. A device that trusts the general
    public CA list can be attacked by any compromised CA in
    the world. Pin your own root (chapter 16).          */
```

How it shows up

Symptom	Cause
Traffic silently readable by a proxy	Unauthenticated DH — man in the middle
Old recorded sessions decrypted after a key theft	No forward secrecy — static keys or RSA key transport
Private key recovered over many handshakes	Invalid-curve or small-subgroup attack — no peer key validation
Connection negotiates 512-bit DH	Downgrade — transcript not authenticated (Logjam/FREAK)
Many devices share a DH group and all fall together	A common, precomputable 1024-bit group
Shared secret works but the AEAD fails	Raw DH output used as a key without a KDF, or mismatched context strings

Symptom	Cause
Same session key on two connections	Cached "ephemeral" key
X25519 returns all zeros	Small-order peer point — reject
Repeated requests executed twice	0-RTT early data replay
Device trusts a hostile certificate	Public CA list instead of a pinned vendor root

Check yourself

- ✓ Walk through basic Diffie–Hellman and say what the eavesdropper must solve.
- ✓ Explain why the raw shared secret is not a key.
- ✓ Draw the man-in-the-middle attack and list five ways to authenticate the exchange.
- ✓ Define forward secrecy and explain why RSA key transport lacks it.
- ✓ Compare finite-field DH and X25519 sizes at 128-bit security.
- ✓ List five properties that make X25519 hard to misuse.
- ✓ Describe an invalid-curve attack and the validation that stops it.
- ✓ Explain transcript binding and how Logjam exploited its absence.
- ✓ Name the Noise patterns and pick one for a device talking to a known server.
- ✓ Describe the factory-provisioning pattern and why the root must be pinned.

Elliptic curve cryptography

The same discrete-logarithm idea in a group where the best known attacks are much worse, so the keys are eight to twelve times smaller. That size difference is what makes public-key cryptography practical on a microcontroller.

The group

```
A SHORT WEIERSTRASS CURVE over the field F_p:

     y^2 = x^3 + a*x + b   (mod p)

The GROUP ELEMENTS are the (x, y) pairs satisfying it,
plus a POINT AT INFINITY O that acts as the identity.

POINT ADDITION, geometrically:
  · draw the line through P and Q
  · it meets the curve at a third point
  · REFLECT it in the x-axis -> that is P + Q

           |        .-'
           |  P  .-'      the chord through P and Q
           |  x-'         hits the curve at -(P+Q);
           | / \  Q       reflecting gives P+Q
      -----+------x-------------
           |       \
           |        x  P+Q

POINT DOUBLING (P + P) uses the TANGENT at P instead.

SCALAR MULTIPLICATION is repeated addition:
     k * P = P + P + ... + P     (k times)
done by DOUBLE-AND-ADD in O(log k) operations.

THE ELLIPTIC CURVE DISCRETE LOGARITHM PROBLEM (ECDLP):
     given P and Q = k*P, find k.

THE BEST KNOWN ATTACK is Pollard's rho, at O(sqrt(n))
where n is the group order. INDEX CALCULUS - the
subexponential algorithm that forces finite-field DH and
RSA to use thousands of bits - DOES NOT APPLY to elliptic
curve groups.

HENCE:
     256-bit curve  ->  ~128-bit security
     3072-bit RSA   ->  ~128-bit security
     Same strength. Twelve times the key.
```

The arithmetic, in practice

```
/* THE AFFINE FORMULAS, for reference: */

    P + Q, with P != Q:
        s  = (y2 - y1) / (x2 - x1)   mod p
        x3 = s^2 - x1 - x2           mod p
        y3 = s*(x1 - x3) - y1        mod p

    P + P (doubling):
        s  = (3*x1^2 + a) / (2*y1)   mod p
        x3 = s^2 - 2*x1              mod p
        y3 = s*(x1 - x3) - y1        mod p

/* THE DIVISIONS ARE MODULAR INVERSES (chapter 11), and they
   are EXPENSIVE - roughly 100x a multiplication.

/* SO REAL IMPLEMENTATIONS USE PROJECTIVE COORDINATES:
   represent a point as (X, Y, Z) with x = X/Z, y = Y/Z (or
   Jacobian: x = X/Z^2, y = Y/Z^3). Addition and doubling
   then need NO INVERSIONS - one inversion at the very end
   converts back to affine. This is not an optimisation
   detail you can skip; it is the difference between a
   scalar multiply taking 1 ms and 100 ms.

/* --- THE SCALAR MULTIPLY, AND ITS SIDE CHANNEL --- */

    /* NAIVE - LEAKS EVERY BIT OF THE SCALAR */
    R = O;
    for (i = top_bit; i >= 0; i--) {
            R = double(R);
            if (bit_i_of_k)                 /* <-- BRANCH */
                    R = add(R, P);          /* <-- TIMING */
    }
    /* An attacker measuring power or time sees the pattern
       of doublings and additions, and reads k directly. This
       is textbook SPA - simple power analysis - and it works
       on an oscilloscope in an afternoon (chapter 19). */

    /* THE MONTGOMERY LADDER - constant operation sequence */
    R0 = O; R1 = P;
    for (i = top_bit; i >= 0; i--) {
            b  = bit_i_of_k;
            /* CONSTANT-TIME CONDITIONAL SWAP - no branch */
            cswap(b, &R0, &R1);
            R1 = add(R0, R1);
            R0 = double(R0);
            cswap(b, &R0, &R1);
    }
    return R0;
    /* EVERY ITERATION DOES EXACTLY ONE ADD AND ONE DOUBLE,
       whatever the bit. The cswap is a masked exchange:
           mask = -(uint64_t)b;
           t = mask & (R0[j] ^ R1[j]);
           R0[j] ^= t;  R1[j] ^= t;
       No branch, no memory-access difference.            */

/* AND EVEN THAT IS NOT ENOUGH AGAINST DIFFERENTIAL POWER
   ANALYSIS. Add SCALAR BLINDING (use k + r*n for random r;
   it computes the same point since n*P = O) and PROJECTIVE
   COORDINATE RANDOMISATION.                              */
```

The curves that matter

Curve	Bits	Form	Notes
Curve25519 / X25519	255	Montgomery	Key exchange. **The default choice.** Every 32-byte string is a valid key
Ed25519	255	Twisted Edwards	Signatures. Same curve, different form. Deterministic
P-256 (secp256r1)	256	Weierstrass	NIST/FIPS. Ubiquitous in TLS certificates and hardware
P-384, P-521	384/521	Weierstrass	Higher security levels; required by some suites
secp256k1	256	Weierstrass	Bitcoin and Ethereum. Koblitz curve, efficient endomorphism
Curve448 / Ed448	448	Edwards	~224-bit security, for a conservative margin
Brainpool	256+	Weierstrass	Verifiably pseudorandom parameters. Used in EU eID

```
/* WHY THE CURVE FORM MATTERS:

   MONTGOMERY (Curve25519): By^2 = x^3 + Ax^2 + x
     · x-coordinate-only ladder, very fast and naturally
       constant time
     · no point validation needed for X25519
     · perfect for KEY EXCHANGE, awkward for signatures

   TWISTED EDWARDS (Ed25519): -x^2 + y^2 = 1 + d*x^2*y^2
     · a COMPLETE addition formula - ONE formula works for
       every pair of points, including P+P and P+O
     · no special cases means no exceptional-case branches,
       and no exceptional-case bugs
     · perfect for SIGNATURES

   SHORT WEIERSTRASS (P-256): the classical form
     · addition and doubling are DIFFERENT formulas, and
       P + (-P) = O is a special case
     · so implementations have branches, and those branches
       are where the bugs and leaks live
     · but it is what FIPS specifies and what most hardware
       accelerators implement

/* THE NIST-CURVE CONTROVERSY, briefly: the P-curve
   parameters were generated by hashing unexplained seed
   values. Nobody has demonstrated a weakness, but nobody can
   explain the seeds either. Curve25519 and Brainpool were
   both designed with FULLY EXPLAINED, "NOTHING UP MY SLEEVE"
   parameter derivation as an explicit goal.

   THE PRACTICAL POSITION: P-256 is fine and you will use it
   whenever a certificate or an accelerator demands it.
   Choose 25519 when you are free to.                    */
```

WHAT TO PICK

```
key exchange, free choice      -> X25519
signatures, free choice        -> Ed25519
TLS certificates, hardware,
  FIPS, or an existing PKI     -> P-256 (ECDSA/ECDH)
higher margin                  -> Ed448 or P-384
blockchain interop             -> secp256k1
```

Point encoding and validation

```
/* --- ENCODING, for Weierstrass curves (SEC1) --- */
    0x04 ‖ X ‖ Y        UNCOMPRESSED, 65 bytes for P-256
    0x02 ‖ X            COMPRESSED, y even     33 bytes
    0x03 ‖ X            COMPRESSED, y odd

    Compression works because y^2 = x^3 + ax + b has exactly
    two solutions for y; one bit says which. Recovering y
    costs a modular square root - a single exponentiation,
    typically ~1/50 of a scalar multiply. WORTH IT when the
    key lives in OTP fuses or travels over a constrained
    radio link.

/* --- X25519 AND Ed25519 --- */
    Always 32 bytes. X25519 sends the x-coordinate only.
    Ed25519 sends y with the sign of x in the top bit.
    No format byte, no options, no negotiation.

/* --- VALIDATION: MANDATORY FOR WEIERSTRASS CURVES --- */

    int validate_point(const ec_point *Q, const ec_group *g)
    {
        /* 1. NOT the point at infinity */
        if (is_infinity(Q)) return -1;

        /* 2. coordinates are in range [0, p-1] */
        if (cmp(Q->x, g->p) >= 0) return -1;
        if (cmp(Q->y, g->p) >= 0) return -1;

        /* 3. THE POINT IS ACTUALLY ON THE CURVE */
        if (!on_curve(Q, g)) return -1;

        /* 4. it is in the correct SUBGROUP:  n*Q == O
              (skippable when the cofactor is 1, as for the
              NIST prime curves) */
        if (!is_infinity(scalar_mul(g->n, Q))) return -1;

        return 0;
    }

/* SKIPPING STEP 3 IS THE INVALID CURVE ATTACK. The attacker
   sends a point that satisfies a DIFFERENT curve equation -
   one whose group has small order. Your scalar multiply
   ignores `b` entirely (look at the formulas above: b never
   appears), so it happily computes on the attacker's curve
   and the result reveals your private scalar modulo a small
   number. Repeat with several curves, apply CRT, and the
   full key falls in a few thousand queries.

   THIS HAS BROKEN: several TLS libraries, a Bluetooth
   pairing implementation (CVE-2018-5383, affecting
   essentially every major OS), and multiple hardware
   wallets.

/* AND THE TWIST ATTACK is the same idea against
   x-coordinate-only implementations. Curve25519 was designed
   to be TWIST-SECURE - the twist's order also has a large
   prime factor - so X25519 needs no validation at all. That
   is a deliberate design decision to remove a class of
```

```
   implementation bug, and it is why X25519 is the
   recommendation.                                          */
```

Using it

```
/* --- ECDH WITH P-256, mbedTLS --- */
    mbedtls_ecdh_context ctx;
    mbedtls_ecdh_init(&ctx);
    mbedtls_ecdh_setup(&ctx, MBEDTLS_ECP_DP_SECP256R1);

    /* our ephemeral key, and the public part to send */
    mbedtls_ecdh_make_public(&ctx, &olen, buf, sizeof buf,
                             mbedtls_ctr_drbg_random, &drbg);

    /* the peer's - THIS VALIDATES THE POINT internally,
       and returning non-zero here is a security event, not
       a parse error */
    if (mbedtls_ecdh_read_public(&ctx, peer, peer_len) != 0)
            return INVALID_PEER_KEY;

    uint8_t secret[32];
    mbedtls_ecdh_calc_secret(&ctx, &olen, secret,
                             sizeof secret,
                             mbedtls_ctr_drbg_random, &drbg);
    /* PASSING THE RNG HERE ENABLES BLINDING. Do not pass
       NULL to "save time" - that disables the countermeasure
       (chapter 19). */

    /* AND THEN, ALWAYS, A KDF (chapter 10): */
    mbedtls_hkdf(sha256, salt, salt_len, secret, olen,
                 (const uint8_t *)"myproto v1 session", 18,
                 session_key, 32);

    mbedtls_platform_zeroize(secret, sizeof secret);
    mbedtls_ecdh_free(&ctx);

/* --- X25519, libsodium: the whole thing --- */
    unsigned char my_sk[32], my_pk[32], shared[32];
    randombytes_buf(my_sk, 32);
    crypto_scalarmult_base(my_pk, my_sk);
    /* send my_pk ... receive their_pk ... */
    if (crypto_scalarmult(shared, my_sk, their_pk) != 0)
            return BAD_KEY;
    /* then hash, as in chapter 13 */

/* --- PERFORMANCE, Cortex-M4 at 100 MHz, rough --- */
    X25519 scalar multiply       ~ 4 M cycles   ( 40 ms)
    P-256 with a good library    ~ 8 M cycles   ( 80 ms)
    Ed25519 sign                 ~ 5 M cycles   ( 50 ms)
    Ed25519 verify               ~14 M cycles   (140 ms)
    RSA-2048 verify (e=65537)    ~ 3 M cycles   ( 30 ms)
    RSA-2048 sign with CRT       ~90 M cycles   (900 ms)

   THE SHAPE TO REMEMBER: RSA VERIFICATION IS THE FASTEST
   THING HERE, which is why boot ROMs use it. Everything
   else favours ECC by a wide margin, and key generation
   favours it by orders of magnitude.

   AND IF THE PART HAS A PKA/CRYPTOCELL ACCELERATOR, these
   numbers drop by 10-50x - check before you optimise
   software.                                                */
```

How it shows up

Symptom	Cause
Private key recovered in a few thousand handshakes	Invalid-curve attack — the on-curve check was skipped
Key read from a power trace	Branching double-and-add — use a Montgomery ladder
Key recovered despite constant-time code	DPA — needs scalar and coordinate blinding too
Scalar multiply takes 100 ms instead of 1 ms	Affine coordinates — a modular inversion per step
Peer key rejected as malformed	Compressed vs uncompressed encoding mismatch
ECDH result differs between the two sides	Different curve, or one side hashing the point and the other the x-coordinate
Works with X25519, fails with P-256	P-256 needs explicit point validation; X25519 does not
Blinding silently disabled	`NULL` RNG passed to the ECDH/ECDSA call
Certificate rejects the device key	Ed25519 where the PKI profile requires ECDSA P-256
Signature verification slower than RSA	Expected — RSA verify with `e=65537` is very cheap

Check yourself

- ✓ Describe the curve group, its operation and its identity element.
- ✓ State the ECDLP and explain why 256 bits matches RSA-3072.
- ✓ Explain why projective coordinates are used and what they avoid.
- ✓ Write the naive scalar multiply, say what it leaks, and write the ladder.
- ✓ Explain a constant-time conditional swap.
- ✓ Compare Montgomery, Edwards and Weierstrass forms and say what each is good for.
- ✓ Describe point compression and when it is worth the cost.
- ✓ Write the four point-validation checks and explain what skipping each allows.
- ✓ Explain the invalid-curve attack, including why `b` never appearing matters.
- ✓ Say what twist security is and why it removes a whole class of bug.
- ✓ Recall the rough cycle counts and explain why boot ROMs still use RSA.

Digital signatures

Anyone can verify; only the key holder can produce. This is what makes secure boot, firmware updates and certificates possible — and ECDSA is the primitive with the sharpest edge in all of applied cryptography.

Signatures versus MACs

```
SIGN(private_key, message)  -> signature
VERIFY(public_key, message, signature) -> true / false

                  MAC                  SIGNATURE
keys              one shared           key PAIR
who can verify    only key holders     ANYONE
who can produce   any key holder       only the private
                                       key holder
NON-REPUDIATION   NO                   YES
size              16-32 bytes          64-512 bytes
speed             microseconds         milliseconds
key distribution  hard (must be         easy (the public
                  secret)              key is public)

THE ASYMMETRY IS THE POINT. A device can ship with a
VENDOR PUBLIC KEY in read-only memory and verify every
firmware image the vendor ever signs - without the device
ever holding a secret that could be extracted to forge
one. That is impossible with a MAC.

THE SECURITY GOAL, same as for MACs:
  EUF-CMA - existential unforgeability under chosen
  message attack. Even after seeing signatures on
  messages of their choosing, an attacker cannot produce
  a valid signature on a NEW message.

AND SOMETIMES STRONGER: SUF-CMA, which additionally
forbids producing a DIFFERENT valid signature on an
ALREADY-SIGNED message. ECDSA is NOT SUF-CMA - see
malleability below - and that has caused real bugs.
```

ECDSA, and the nonce

```
/* SIGNING with private key d, on curve with generator G and
   order n:

    1. e = leftmost n bits of H(message)
    2. k = A RANDOM NONCE in [1, n-1]        <<< DANGER
    3. (x1, y1) = k * G
    4. r = x1 mod n           (if r == 0, restart)
    5. s = k^-1 * (e + r*d) mod n   (if s == 0, restart)
    signature = (r, s)

   VERIFYING with public key Q:
    1. check 1 <= r, s <= n-1
    2. e = leftmost n bits of H(message)
    3. u1 = e * s^-1 mod n,   u2 = r * s^-1 mod n
    4. (x1, y1) = u1*G + u2*Q
    5. valid iff  r == x1 mod n

/* ======================================================
   THE NONCE k MUST BE UNIQUE AND SECRET AND UNPREDICTABLE.
   ANY FAILURE RECOVERS THE PRIVATE KEY FROM SIGNATURES.
   ====================================================== */

/* --- REUSE: two signatures with the same k --- */
        s1 = k^-1 (e1 + r*d)
        s2 = k^-1 (e2 + r*d)      /* same r, because same k */

    subtract:  s1 - s2 = k^-1 (e1 - e2)
        =>  k = (e1 - e2) / (s1 - s2)   mod n
        =>  d = (s1*k - e1) / r         mod n

    FOUR LINES OF ARITHMETIC AND THE KEY IS YOURS. And note
    that reuse is DETECTABLE FROM THE OUTSIDE: identical r
    values in two signatures are a public giveaway.

    THIS HAPPENED TO:
      · SONY PLAYSTATION 3 (2010) - a FIXED k in the boot
        code. The console signing key was published. Full
        homebrew.
      · ANDROID BITCOIN WALLETS (2013) - SecureRandom
        returned repeated values after a fork(). Coins
        stolen automatically by bots scanning the blockchain
        for repeated r.
      · Numerous hardware wallets and IoT devices since.

/* --- BIAS: k not uniform --- */
    Even a FEW BITS of bias - k always having its top bit
    clear, or k computed as (random mod n) with a slight
    modulo skew - lets LATTICE ATTACKS recover d from a few
    hundred signatures. "Minerva" and "TPM-FAIL" (2019) both
    exploited a handful of leaked bits from timing.

/* --- PARTIAL LEAK: k's top bits visible via timing --- */
    Same lattice attack. This is why the scalar multiply must
    be constant time AND the nonce generation must not depend
    on k's length (chapter 19).

/* ======================================================
   THE FIX: RFC 6979 DETERMINISTIC ECDSA
   ====================================================== */
        k = HMAC_DRBG(private_key, H(message))
```

```
    The nonce is derived deterministically from the key and
    the message. NO RNG INVOLVED.
      · unique, because different messages give different k
      · secret, because it depends on d
      · unbiased, by construction
      · REPRODUCIBLE, so signatures are testable

    USE IT ALWAYS. It removes the entire category. mbedTLS:
    build with MBEDTLS_ECDSA_DETERMINISTIC and call
    mbedtls_ecdsa_sign_det_ext().

    (One caveat: determinism makes FAULT ATTACKS easier -
    sign the same message twice, glitch one, and compare. On
    a physically exposed device, add a small random component
    - "hedged" signing, as Ed25519ph and some newer specs
    do.)                                                  */
```

NEVER GENERATE AN ECDSA NONCE FROM A DEVICE RNG YOU HAVE NOT AUDITED

A hardware RNG that has not finished seeding, a PRNG duplicated across a fleet by a firmware image, or a counter that resets on reboot will all produce repeated `k`. And unlike most cryptographic failures, this one is *publicly detectable*: anyone collecting your signatures can grep for a repeated `r`. Use RFC 6979, or Ed25519.

Ed25519

```
/* EdDSA on the Edwards form of Curve25519 (chapter 14).
   Designed specifically to remove every foot-gun above. */

    KEY GENERATION:
        seed  = 32 random bytes
        h     = SHA-512(seed)
        a     = clamp(h[0..31])       /* the scalar */
        prefix= h[32..63]             /* for nonces */
        A     = a * B                 /* public key,
                                         32 bytes */

    SIGN(message):
        r = SHA-512(prefix ‖ message)  mod L   /* NO RNG */
        R = r * B
        k = SHA-512(R ‖ A ‖ message)   mod L
        S = (r + k*a)                  mod L
        signature = R ‖ S              /* 64 bytes */

    VERIFY:
        k = SHA-512(R ‖ A ‖ message) mod L
        check  S*B == R + k*A

/* WHAT IT FIXES, point by point:
     · THE NONCE IS DETERMINISTIC by construction - not an
       option you can forget to enable
     · COMPLETE ADDITION FORMULAS - no exceptional cases, so
       no branches
     · CONSTANT TIME by design
     · the PUBLIC KEY IS HASHED INTO k, preventing
       key-substitution attacks
     · COLLISION-RESILIENT: forging needs a SHA-512
       collision, not just a hash collision on the message
     · FIXED SIZES: 32-byte keys, 64-byte signatures, no
       DER parsing at all

/* THE CODE: */
    unsigned char pk[crypto_sign_PUBLICKEYBYTES];  /* 32 */
    unsigned char sk[crypto_sign_SECRETKEYBYTES];  /* 64 */
    crypto_sign_keypair(pk, sk);

    unsigned char sig[crypto_sign_BYTES];          /* 64 */
    crypto_sign_detached(sig, NULL, msg, msg_len, sk);

    if (crypto_sign_verify_detached(sig, msg, msg_len, pk)
        != 0)
            return INVALID;

/* THAT IS THE ENTIRE API. No curve parameter, no hash
   choice, no nonce, no encoding options, no padding mode.
   Compare with the ECDSA and RSA sections and the appeal is
   obvious.

/* THE ONE INTEROP WRINKLE: verification strictness varies
   between implementations - cofactored vs cofactorless
   equations, and whether non-canonical encodings are
   rejected. It matters for consensus systems where every
   node must agree; for a device verifying its own vendor's
   signatures it does not. RFC 8032 plus the ZIP-215 rules
   are the strict reference.                              */
```

Choosing

Scheme	Sig size	Pub key	Verdict
Ed25519	64 B	32 B	**The default.** Fast, safe, no parameters
Ed448	114 B	57 B	Higher margin
ECDSA P-256	64 B (72 DER)	33/65 B	Where FIPS, PKI or hardware demand it. **Use RFC 6979**
RSA-PSS 3072	384 B	384 B	Fast verification — boot ROMs
RSA v1.5 2048	256 B	256 B	Legacy formats only. Verify the *whole* block
ML-DSA-65 (Dilithium)	3309 B	1952 B	Post-quantum (ch. 20)
SLH-DSA (SPHINCS+)	7856 B+	32 B	Hash-based, very conservative, huge signatures
LMS / XMSS	~2 KB	32 B	**Stateful** hash-based. NIST-approved for firmware signing

The other ways signatures fail

```
/* --- 1. MALLEABILITY --- */
    ECDSA: if (r, s) is valid, so is (r, n - s). An attacker
    modifies a signature into a different valid one WITHOUT
    the key.

    Harmless if you only care about the message. FATAL if you
    use the signature as an identifier - Bitcoin's original
    transaction IDs included the signature, so an attacker
    could change a transaction's ID after broadcast. That was
    "transaction malleability", and it took a protocol change
    to fix.

    THE RULE: require the canonical low-s form, and NEVER
    treat a signature as a unique identifier for what it
    signs.

/* --- 2. NOT VERIFYING WHAT YOU THINK YOU VERIFIED --- */
    /* WRONG: */
    if (verify(pubkey, hash, sig) == OK)
            install(image);       /* is `hash` the hash of
                                     `image`? */
    /* The classic bug: hash computed over a header, or over
       a buffer that is later replaced. TIME-OF-CHECK TO
       TIME-OF-USE.

    /* RIGHT: hash the exact bytes you are about to use,
       immediately before using them, from memory the
       attacker cannot modify in between. */

/* --- 3. NOT CHECKING WHOSE KEY IT IS --- */
    A valid signature from the WRONG KEY is not an
    authorisation. The verification must be against a key
    that is:
      · pinned (a hash in OTP), or
      · chained to a pinned root (chapter 16)
    "It verified" means nothing without "and the key was
    trusted for this purpose".

/* --- 4. ALGORITHM CONFUSION --- */
    If the signature format carries its own algorithm
    identifier and you trust it, an attacker sets it to
    "none", or to a scheme where they hold a key. JWT's
    alg:none is the famous instance, and the same shape
    appears in firmware container formats.
    THE FIX: THE VERIFIER DECIDES THE ALGORITHM, not the
    message.

/* --- 5. SIGNING WHAT YOU DID NOT MEAN TO --- */
    A signing oracle - anything that will sign
    attacker-supplied bytes - can be tricked into signing
    something meaningful in another context. USE DOMAIN
    SEPARATION: sign H("myapp firmware v1" ‖ data), never
    raw data.

/* --- 6. NO ROLLBACK PROTECTION --- */
    An OLD, GENUINELY SIGNED image with a known
    vulnerability verifies perfectly. Signatures give
    authenticity, not freshness (the same gap as MACs,
    chapter 08). You need a MONOTONIC VERSION COUNTER in
    fuses or a replay-protected memory block
```

```
    (chapter 18).

/* --- 7. DER PARSING BUGS --- */
    ECDSA signatures in X.509 are DER-encoded ASN.1: nested
    length-prefixed structures, with rules about minimal
    encoding and leading zero bytes. This is a rich source
    of parser bugs and signature-forgery CVEs. Ed25519's
    fixed 64 bytes avoid it entirely - one more reason to
    prefer it.                                          */
```

The firmware signing pattern

```
/* THE SIGNING SIDE, in a facility you control: */
    private key lives in an HSM. It NEVER exists as a file.
    the build server calls the HSM to sign; access is
      logged, multi-party, and revocable
    a manifest is signed, not just the image:

      struct manifest {
          uint32_t magic;
          uint32_t format_version;
          uint32_t fw_version;        /* ANTI-ROLLBACK */
          uint32_t image_len;
          uint8_t  image_hash[32];    /* SHA-256 of the
                                         image */
          uint32_t device_class;      /* not the neighbour's
                                         product */
          uint64_t not_before;
      };
      signature = Ed25519(sk, "vendor-fw-v1" || manifest);

/* THE VERIFYING SIDE, on the device: */
    1. read the manifest
    2. check magic and format_version - REJECT UNKNOWN
    3. verify the signature over the WHOLE manifest, with a
       key whose HASH IS IN OTP (or a cert chained to it)
    4. check device_class matches THIS product
    5. check fw_version >= the fuse counter  -- ROLLBACK
    6. hash the image and compare to image_hash IN CONSTANT
       TIME
    7. only now: erase and write, or jump

/* NOTE THE ORDER. Every check that can reject happens
   BEFORE anything irreversible. And note that step 6 hashes
   the image AFTER it is in its final location, so an
   attacker with DMA access cannot swap it in between
   (the isolation track).                                */
```

How it shows up

Symptom	Cause
Private key published after a few signatures	Repeated ECDSA nonce — identical `r` values
Key recovered from a few hundred signatures	Biased nonce — a lattice attack
A signature altered but still valid	ECDSA malleability — require low `s`
Unsigned firmware installed	Algorithm confusion — the verifier trusted the message's alg field
Old vulnerable firmware reinstalled	No anti-rollback counter

Symptom	Cause
Verified image differs from the installed one	TOCTOU between hashing and use
A valid signature from an untrusted key accepted	No key pinning or chain check
Signature forged via a malformed encoding	DER parser bug
The neighbouring product's firmware installs	No device class in the signed manifest
Key extracted by glitching a repeated signature	Deterministic signing plus fault injection — hedge it

Check yourself

- ✓ Contrast MACs and signatures on five axes, and say why a device can ship a public key safely.
- ✓ Write out ECDSA signing and verification.
- ✓ Derive the private key from two signatures sharing a nonce.
- ✓ Explain nonce bias and why a few leaked bits suffice.
- ✓ Explain RFC 6979 and the one situation where pure determinism is a liability.
- ✓ List five design decisions in Ed25519 that remove a class of bug each.
- ✓ Explain malleability and the Bitcoin transaction-ID failure.
- ✓ Explain algorithm confusion and state the fix in one sentence.
- ✓ Explain why "the signature verified" is not an authorisation decision.
- ✓ Write the seven-step firmware verification order and justify the ordering.

Certificates and PKI

A signature proves a message came from a key. A certificate is the machinery that tells you *whose* key it is — and getting that binding wrong is how otherwise-correct cryptography ends up protecting nothing.

The binding problem

```
You verified a signature with public key Q. Excellent.
WHO OWNS Q?

Without an answer, an attacker just generates their own
key pair, signs whatever they like, and hands you their
public key. Everything verifies.

A CERTIFICATE is a signed statement:

     "The key Q belongs to <identity>,
      valid from <date> to <date>,
      and may be used for <purposes>."
                          -- signed, some issuer

THE RECURSION: how do you trust the ISSUER's key?
Answer: another certificate. Until you reach a
SELF-SIGNED ROOT that you trust because it was PUT THERE
OUT OF BAND - shipped in your OS, burned into your OTP
fuses, compiled into your firmware.

THE CHAIN:

  ROOT CA (self-signed, in your trust store)
     |  signs
     v
  INTERMEDIATE CA
     |  signs
     v
  END-ENTITY CERT  ("device-4711", or
                    "api.example.com")

INTERMEDIATES EXIST so the root's private key can stay
OFFLINE, in a safe, powered off. If an intermediate is
compromised, it is revoked and replaced. If the root is
compromised, every device that trusts it must be
physically updated - which for embedded hardware may mean
never.

THE TRUST MODEL IS THE WEAK PART: with the public web PKI
you are trusting ~150 organisations worldwide, ANY OF
WHICH can issue a certificate for ANY name. Several have
been compromised or have misissued. DigiNotar (2011) was
used to intercept Gmail for 300,000 Iranian users, and the
CA went bankrupt.
```

Inside an X.509 certificate

```
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number: 04:1a:...              /* unique per CA;
                                             must be random
                                             (see below) */
    Signature Algorithm: ecdsa-with-SHA256
    Issuer:  C=US, O=Example CA, CN=Example Issuing CA
    Validity
      Not Before: Jun  1 00:00:00 2026 GMT
      Not After : Aug 30 23:59:59 2026 GMT
    Subject: CN=api.example.com
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
        NIST CURVE: P-256
        pub: 04:8f:...
    X509v3 extensions:
      X509v3 Basic Constraints: critical
        CA:FALSE                      <-- THE IMPORTANT ONE
      X509v3 Key Usage: critical
        Digital Signature
      X509v3 Extended Key Usage:
        TLS Web Server Authentication
      X509v3 Subject Alternative Name:
        DNS:api.example.com, DNS:www.example.com
                                      <-- THE NAME THAT
                                          COUNTS
      X509v3 Authority Key Identifier: keyid:...
      X509v3 CRL Distribution Points: URI:http://...
      Authority Information Access:
        OCSP - URI:http://ocsp.example.com
  Signature Algorithm: ecdsa-with-SHA256
  Signature Value: 30:45:02:20:...

/* THE FIELDS THAT ACTUALLY MATTER FOR SECURITY:

   BASIC CONSTRAINTS / CA:FALSE
     Says this certificate may NOT sign other certificates.
     A verifier that ignores it lets ANY certificate holder
     become a CA and issue a cert for any name. This was a
     REAL BUG in Internet Explorer, in the iPhone, and in
     several embedded stacks. It is the single most important
     extension.

   KEY USAGE / EXTENDED KEY USAGE
     What the key may do. A certificate issued for code
     signing should not authenticate a TLS server.

   SUBJECT ALTERNATIVE NAME
     THE NAMES THE CERT IS VALID FOR. The Subject CN is
     LEGACY and modern verifiers ignore it. Wildcards match
     ONE label: *.example.com covers a.example.com but NOT
     a.b.example.com and NOT example.com itself.

   VALIDITY DATES
     Which means the device NEEDS A CORRECT CLOCK - see
     below, because on embedded hardware it usually does
     not have one.

   PATH LENGTH CONSTRAINT
```

```
     How many intermediates may appear below this CA.

/* AND A NOTE ON ENCODING: all of this is DER-encoded ASN.1 -
   nested tag-length-value structures. X.509 PARSERS ARE A
   HISTORIC SOURCE OF MEMORY-SAFETY BUGS, because the parser
   runs on ATTACKER-CONTROLLED INPUT BEFORE any signature has
   been checked. If you are choosing a TLS library for a
   device, the quality of its ASN.1 parser matters more than
   its cipher performance.                                  */
```

Verifying a chain, correctly

```
/* THE FULL PROCEDURE. Skipping any step has broken real
   software.                                              */

  FOR EACH certificate from the leaf up to a trusted root:

   1. SIGNATURE: verify it with the ISSUER's public key.

   2. VALIDITY DATES: now is within [notBefore, notAfter].

   3. ISSUER/SUBJECT CHAINING: this cert's Issuer matches
      the next cert's Subject, byte for byte.

   4. FOR EVERY NON-LEAF: BASIC CONSTRAINTS says CA:TRUE.
      <-- THE STEP PEOPLE SKIP

   5. FOR EVERY NON-LEAF: KEY USAGE includes keyCertSign.

   6. PATH LENGTH within the constraint.

   7. NAME CONSTRAINTS, if the CA has any, are satisfied.

   8. CRITICAL EXTENSIONS you do not understand -> REJECT.
      That is what "critical" means.

   9. REVOCATION: not on a CRL, or OCSP says good.

  AND THEN, SEPARATELY, ON THE LEAF:

  10. NAME MATCHING: the SAN contains the name YOU INTENDED
      to connect to - not the name in the certificate, and
      not the name the server told you.

  11. EXTENDED KEY USAGE matches your purpose.

  12. THE CHAIN ENDS AT A ROOT IN YOUR TRUST STORE.

/* THE FAMOUS FAILURES:
     · Apple's "goto fail" (2014): a duplicated goto
       statement skipped the signature check entirely.
     · GnuTLS (2014): a return-value confusion made invalid
       certificates verify.
     · Countless mobile apps: implementing a custom
       TrustManager that returns "valid" unconditionally, to
       make a self-signed test certificate work - and
       shipping it.
     · Basic-constraints omissions in at least four major
       stacks.

   THE PATTERN: THE FAILURE IS ALWAYS "ACCEPTS WHAT IT
   SHOULD REJECT", AND IT IS INVISIBLE IN TESTING, because
   the happy path still works perfectly.

/* SO: TEST YOUR VERIFIER WITH BAD CERTIFICATES.
     - expired
     - wrong name
     - self-signed
     - signed by a non-CA leaf
     - a valid chain to an untrusted root
     - a revoked certificate
     - a truncated / malformed DER blob
```

```
   If all seven are rejected, you have a verifier. If you
   only ever tested the good certificate, you have nothing.
   openssl s_server plus a small script generates all of
   these in a few minutes, and badssl.com hosts them
   publicly.                                              */
```

NEVER DISABLE CERTIFICATE VERIFICATION "TEMPORARILY"

`curl -k`, `verify=False`, `MBEDTLS_SSL_VERIFY_NONE`, a custom `TrustManager` that always returns success — every one of these has shipped to production in a real product. If a self-signed development certificate is inconvenient, add it to the device's trust store for development builds. Make the insecure path impossible to compile into a release build, not merely discouraged.

Revocation, and why it barely works

```
/* A certificate is valid until it expires. If the private
   key is stolen before then, you need to say so. All the
   mechanisms are unsatisfying.                          */

/* --- CRL: certificate revocation list --- */
    A signed list of revoked serial numbers, published by the
    CA.
    PROBLEMS: it grows without bound (megabytes); it is
    cached, so revocation is delayed by hours or days; and
    fetching it needs network access at exactly the moment a
    connection is being established.

/* --- OCSP: online status protocol --- */
    Ask the CA about one certificate.
    PROBLEMS: a PRIVACY LEAK (the CA learns every site you
    visit); a latency cost on every connection; and the fatal
    one - MOST CLIENTS SOFT-FAIL. If the OCSP responder is
    unreachable, they proceed anyway. An attacker who can
    intercept your traffic can also block OCSP, so revocation
    checking provides no security against the attacker it
    was designed to stop.

/* --- OCSP STAPLING --- */
    The SERVER fetches its own signed OCSP response and
    includes it in the handshake. Fixes the privacy and
    latency problems. Still soft-fails unless the
    certificate carries the "must-staple" extension.

/* --- SHORT-LIVED CERTIFICATES: the practical answer --- */
    Issue for 90 days, or 7, or 1. Revocation becomes
    expiry. This is why ACME/Let's Encrypt automation
    exists, and the industry is moving toward 47-day
    maximum lifetimes.

/* --- FOR EMBEDDED DEVICES, be realistic --- */
    A device may be offline, may have no clock, and may live
    for fifteen years. The web PKI's assumptions do not hold.
    WHAT WORKS INSTEAD:

      · YOUR OWN CA, not the public one. You control issuance
        and you can make the certificates last as long as the
        hardware.
      · PINNING: the device trusts EXACTLY ONE root, whose
        hash is in OTP. No public CA can attack it.
      · A REVOCATION LIST DELIVERED WITH FIRMWARE UPDATES,
        signed, with a monotonic version.
      · A DEVICE IDENTITY THAT CANNOT BE STOLEN: the private
        key generated inside a secure element and never
        exported (chapter 18).
      · AND FOR THE CLOCK PROBLEM: either accept that
        notBefore/notAfter cannot be checked and rely on
        pinning plus revocation, or seed the clock from a
        SIGNED time source before doing any validation.
        Do not let an attacker set your clock to 2035 and
        use an expired certificate.                      */
```

Device PKI in practice

```
/* --- THE HIERARCHY --- */
      Vendor Root CA          offline, in an HSM, in a safe.
        |                     30-year validity. Its PUBLIC
        |                     KEY HASH goes into every
        |                     device's OTP.
        +-- Manufacturing CA  signs device certificates on
        |                     the production line
        +-- Firmware CA       signs firmware manifests
        |                     (chapter 15)
        +-- Service CA        signs the cloud endpoints the
                              device connects to

    SEPARATE INTERMEDIATES PER PURPOSE. A compromise of the
    factory does not let an attacker sign firmware.

/* --- PROVISIONING, on the line --- */
    1. the secure element GENERATES ITS OWN key pair; the
       private half never exists outside it
    2. it emits a CSR (certificate signing request) - the
       public key plus a self-signature proving possession
    3. the factory HSM signs it, embedding a DEVICE SERIAL
       and a PRODUCT CLASS
    4. the certificate is written back to the device (it is
       public; it needs no protection)
    5. the (serial -> certificate) mapping goes into the
       vendor's inventory database

    THE PRIVATE KEY IS NEVER TRANSPORTED, NEVER BACKED UP,
    AND NEVER KNOWN TO ANYONE. A compromised factory database
    leaks nothing secret.

/* --- MUTUAL TLS IN THE FIELD --- */
    device -> cloud:  presents its device certificate,
                      proves possession via the handshake
                      signature
    cloud  -> device: presents a service certificate, which
                      the device validates against its
                      PINNED vendor root

    The cloud additionally checks the device serial against
    its inventory: is this device registered, activated, not
    reported stolen, and running a firmware version we still
    accept?

/* --- WHAT TO PUT IN OTP --- */
    NOT the root certificate - it is too big for fuses.
    THE SHA-256 HASH of the root public key. 32 bytes.
    The certificate itself lives in flash, and the boot code
    hashes it and compares before trusting it. That way the
    fuse cost is fixed and small, and the certificate can be
    re-encoded or re-formatted without burning new fuses.

/* --- AND PLAN FOR ROTATION --- */
    Fifteen-year devices outlive algorithms. Provision
    SEVERAL root key slots in OTP, with a fuse indicating
    which are active, so a root can be retired without
    bricking the fleet. This costs a few hundred fuse bits
    and it is the difference between a recoverable incident
    and a recall.                                          */
```

How it shows up

Symptom	Cause
Any leaf certificate can sign for any name	Basic constraints not checked
Connection succeeds to the wrong host	SAN not matched against the intended name
Self-signed certificates accepted in production	A development bypass left compiled in
A stolen key still works	Revocation soft-fail — the attacker blocks OCSP
All connections fail on a device after a reboot	No RTC — `notBefore` in the future
An expired certificate accepted	Clock set by the attacker, or dates not checked
Crash while parsing a certificate	ASN.1 parser bug — runs before any signature check
One compromised CA anywhere breaks the device	Public CA list instead of a pinned vendor root
A factory database leak compromises devices	Private keys generated outside the secure element
Cannot retire a root algorithm	Single root hash burned in OTP, no spare slots
`*.example.com` fails for `a.b.example.com`	Correct — a wildcard matches one label

Check yourself

- ✓ Explain what a certificate binds and why the chain must terminate out of band.
- ✓ Say why intermediates exist and what a root compromise costs an embedded fleet.
- ✓ Name the five certificate fields that carry real security meaning.
- ✓ List the twelve chain-verification steps and name a real failure for three of them.
- ✓ Explain what "critical extension" obliges a verifier to do.
- ✓ List the seven bad certificates every verifier must be tested against.
- ✓ Explain why OCSP soft-fail provides no security against a network attacker.
- ✓ Give four revocation strategies that actually work for offline devices.
- ✓ Describe factory provisioning and why the private key is never transported.
- ✓ Say what goes in OTP and why it is a hash rather than a certificate.
- ✓ Explain why multiple root slots matter for a fifteen-year product.

TLS

Every primitive so far, assembled into a working secure channel. TLS 1.3 is the reference design for how to combine them — and the thirty years of attacks on its predecessors are the reference list of how not to.

The two layers

```
HANDSHAKE PROTOCOL
  · negotiate the version and the cipher suite
  · exchange keys (ECDHE - chapter 13)
  · authenticate the peer (certificates - chapter 16)
  · derive the traffic keys (HKDF - chapter 10)
  · confirm the whole transcript matched

RECORD PROTOCOL
  · fragment the byte stream into records
  · protect each with an AEAD (chapter 09)
  · number them implicitly, so reordering, deletion and
    replay are all detected

TLS 1.3 HANDSHAKE, one round trip:

CLIENT                                  SERVER
  ClientHello
    + supported_versions
    + key_share (X25519 pubkey)
    + signature_algorithms
    + server_name (SNI)         ------>
                                          ServerHello
                                            + key_share
                        <=== both sides now derive the
                             handshake keys; EVERYTHING
                             BELOW IS ENCRYPTED ===>
                                          {EncryptedExtensions}
                                          {Certificate}
                                          {CertificateVerify}
                                          {Finished}
                                <------
  {Finished}                    ------>
  [Application Data]            <------>  [Application Data]

  {} = handshake-key encrypted   [] = traffic-key encrypted

WHAT EACH MESSAGE IS FOR:
  key_share            ephemeral DH, so FORWARD SECRECY is
                       not optional
  Certificate          the server's identity
  CertificateVerify    a SIGNATURE OVER THE TRANSCRIPT -
                       this is what binds the certificate
                       to this exchange, and stops the
                       man in the middle
  Finished             a MAC over the whole transcript,
                       from both sides - this is what kills
                       downgrade attacks
```

What TLS 1.3 removed

```
/* The most important design work in TLS 1.3 was DELETION.
   Each removal closed a family of attacks.               */

   REMOVED                        ATTACK IT ENABLED
   ------------------------------------------------------
   RSA key transport              no forward secrecy;
                                  Bleichenbacher/ROBOT
                                  (chapter 12)
   Static DH / static ECDH        no forward secrecy
   CBC mode ciphersuites          BEAST, Lucky13, padding
                                  oracles (chapter 05)
   RC4                            biased keystream (ch. 06)
   3DES                           SWEET32 birthday bound
   MD5, SHA-1 signatures          collisions (chapter 07)
   Compression                    CRIME, BREACH
   Renegotiation                  the 2009 renegotiation
                                  injection flaw
   Custom DH groups               Logjam (chapter 13)
   Export ciphersuites            FREAK, Logjam
   Non-AEAD ciphers               everything above

   WHAT REMAINS: FIVE cipher suites, all AEAD, all forward
   secret.
       TLS_AES_128_GCM_SHA256
       TLS_AES_256_GCM_SHA384
       TLS_CHACHA20_POLY1305_SHA256
       TLS_AES_128_CCM_SHA256          <-- for constrained
       TLS_AES_128_CCM_8_SHA256            devices

   THE LESSON GENERALISES: A PROTOCOL'S SECURITY IS BOUNDED
   BY ITS WEAKEST NEGOTIABLE OPTION. Every algorithm you keep
   "for compatibility" is an algorithm an attacker will try
   to force you into. Delete, do not deprecate.

/* AND ON THE KEY SCHEDULE: TLS 1.3 derives every key with
   HKDF, with a distinct label for each purpose:
       "c hs traffic", "s hs traffic",
       "c ap traffic", "s ap traffic",
       "exp master",   "res master"
   Textbook key separation (chapter 10), including separate
   keys per direction (chapter 08).                       */
```

The record layer

```
/* Each record is protected with the negotiated AEAD.     */

    struct {
        ContentType type;          /* always 23
                                      "application_data" on
                                      the wire in 1.3 - the
                                      real type is INSIDE
                                      the encrypted body */
        uint16 legacy_version;     /* always 0x0303 */
        uint16 length;
        opaque encrypted_record[length];
    } TLSCiphertext;

/* THE NONCE IS NOT TRANSMITTED. It is derived:
        nonce = static_iv XOR sequence_number
   The sequence number is implicit and increments per record.

   THIS IS AN ELEGANT PIECE OF DESIGN, and it does three
   things at once:
     · guarantees nonce uniqueness without state on the wire
       (chapter 09's cardinal rule, enforced structurally)
     · makes REPLAY impossible - a replayed record decrypts
       under the wrong nonce and fails the tag check
     · makes REORDERING and DELETION detectable for free

/* RECORD SIZE AND THE KEY UPDATE:
     max 2^14 bytes of plaintext per record.
     KeyUpdate messages rotate the traffic keys mid-session,
     so the AEAD limits (chapter 09) never bite on a
     long-lived connection.

/* PADDING: TLS 1.3 lets you append zero bytes inside the
   encrypted body to hide the true length. Nobody uses it,
   and TRAFFIC ANALYSIS remains a real leak - the size and
   timing of your records still describe what you are doing.
   For a device that sends distinguishable commands, pad to a
   fixed size.                                             */
```

> **0-RTT IS NOT FREE**
> TLS 1.3 lets a resuming client send application data in its first flight. That data has *no forward secrecy* against a compromise of the resumption secret, and it *can be replayed* by an attacker who captured it. Only send idempotent requests in early data, and only enable it if the latency actually matters to your product.

DTLS, and TLS on constrained devices

```
/* --- DTLS 1.3: TLS over UDP --- */
    Needed because TLS assumes a reliable ordered stream, and
    UDP gives neither.
      · explicit sequence numbers in the record header
      · a REPLAY WINDOW (a sliding bitmap) instead of a strict
        counter
      · retransmission and reassembly of handshake messages
      · a HELLORETRYREQUEST COOKIE, so the server does no work
        and holds no state until the client proves it can
        receive at its claimed address - AMPLIFICATION DDOS
        PROTECTION

    USED BY: CoAP over DTLS, WebRTC, QUIC's ancestor, and
    most constrained-device IoT stacks.

/* --- WHAT TLS COSTS ON AN MCU --- */
    mbedTLS, ECDHE-ECDSA-AES128-GCM, Cortex-M4:
      code                 ~60-100 KB flash
      RAM per connection   ~20-40 KB   <-- usually the
                                           binding constraint
      handshake            ~300-800 ms in software
      record throughput    a few hundred KB/s

    THE RAM IS THE PROBLEM. Two 16 KB record buffers plus the
    certificate chain plus the session state. On a part with
    64 KB of SRAM, one TLS connection may be all you get.

/* --- HOW TO MAKE IT FIT --- */
    · MBEDTLS_SSL_MAX_CONTENT_LEN 2048  - the single biggest
      saving. Requires the max_fragment_length extension and
      a server that honours it.
    · MBEDTLS_SSL_KEEP_PEER_CERTIFICATE off - free the chain
      after verification
    · compile out every cipher suite you do not use; the
      default config includes dozens
    · use RAW PUBLIC KEYS (RFC 7250) instead of certificates
      where both ends are yours - no ASN.1, no chain, 32
      bytes
    · or use PSK, below
    · SESSION RESUMPTION to avoid repeated handshakes - one
      handshake per boot instead of one per message

/* --- PSK MODE: when public-key crypto will not fit --- */
    TLS_PSK_WITH_AES_128_CCM_8, or 1.3's psk_ke:
      a symmetric key provisioned per device at manufacture.
      TINY: no certificates, no ECC, a few kilobytes total.

    THE TRADE-OFFS, and they are serious:
      · NO FORWARD SECRECY in pure PSK mode (use psk_dhe_ke
        if you can afford one ECDH)
      · the server must hold every device's key - a database
        breach is a fleet compromise
      · key rotation is hard
      · if the key is not per-device, ONE EXTRACTED DEVICE
        BREAKS EVERYTHING. This is the classic IoT failure:
        a single symmetric key in every unit of a product.

    USE PSK ONLY WITH PER-DEVICE KEYS, stored in a secure
    element (chapter 18), and prefer psk_dhe_ke.
```

```
/* --- THE CLIENT CODE, mbedTLS, with the checks that
       matter --- */
    mbedtls_ssl_config_defaults(&conf,
            MBEDTLS_SSL_IS_CLIENT,
            MBEDTLS_SSL_TRANSPORT_STREAM,
            MBEDTLS_SSL_PRESET_DEFAULT);

    /* 1. VERIFICATION IS MANDATORY, NOT OPTIONAL */
    mbedtls_ssl_conf_authmode(&conf,
            MBEDTLS_SSL_VERIFY_REQUIRED);

    /* 2. OUR PINNED ROOT ONLY - not a system CA bundle */
    mbedtls_ssl_conf_ca_chain(&conf, &vendor_root, NULL);

    /* 3. FLOOR THE VERSION */
    mbedtls_ssl_conf_min_tls_version(&conf,
            MBEDTLS_SSL_VERSION_TLS1_2);

    /* 4. THE HOSTNAME - sets SNI *AND* enables name
          verification. FORGETTING THIS MEANS ANY VALID
          CERTIFICATE FROM ANY SERVER IS ACCEPTED. */
    mbedtls_ssl_set_hostname(&ssl, "api.example.com");

    /* 5. our device certificate, for mutual TLS */
    mbedtls_ssl_conf_own_cert(&conf, &dev_cert, &dev_key);

    ret = mbedtls_ssl_handshake(&ssl);
    if (ret != 0) { /* FAIL CLOSED. No retry without TLS. */ }

    /* 6. AND CHECK THE RESULT EXPLICITLY */
    uint32_t flags = mbedtls_ssl_get_verify_result(&ssl);
    if (flags != 0) { abort_connection(); }

/* STEP 4 IS THE ONE THAT GETS OMITTED, and it silently
   disables hostname verification. It is the embedded
   equivalent of the mobile TrustManager bug from
   chapter 16.                                          */
```

The attack history, and what it teaches

Attack	Year	Mechanism	General lesson
Bleichenbacher	1998	PKCS#1 v1.5 decryption oracle	Error messages are an oracle
BEAST	2011	Predictable CBC IV	IVs must be unpredictable
CRIME	2012	Compression before encryption	Never compress secrets with attacker data
Lucky13	2013	MAC-then-encrypt timing	Encrypt-then-MAC; use an AEAD
Heartbleed	2014	Missing length check	Memory safety *is* crypto security
goto fail	2014	Duplicated statement skipped a check	Test the rejection paths
POODLE	2014	SSLv3 padding, reached by downgrade	Remove old versions entirely
FREAK / Logjam	2015	Downgrade to export grade	Delete weak options, don't deprecate
DROWN	2016	SSLv2 on a server sharing a key	Key reuse across protocols is fatal
SWEET32	2016	64-bit block birthday bound	Block size limits data per key
ROBOT	2017	Bleichenbacher, again	Removed attacks come back if the code stays
Raccoon	2020	DH timing side channel	Constant time, everywhere

```
/* WHAT THE LIST ACTUALLY SAYS:

   · ALMOST NONE OF THESE BROKE A CIPHER. AES is fine. RSA is
     fine. The failures are in COMPOSITION, NEGOTIATION,
     ERROR HANDLING, PARSING and IMPLEMENTATION.

   · Half of them are DOWNGRADE or LEGACY-OPTION attacks. The
     old code you kept for compatibility is the attack
     surface.

   · Several are TIMING. Constant-time code is not paranoia
     (chapter 19).

   · One is a plain buffer over-read. MEMORY SAFETY IS PART
     OF CRYPTOGRAPHY, which is a large part of why Rust is
     appearing in this space (see the Rust track).

   · AND THE META-LESSON: USE A MAINTAINED LIBRARY AND KEEP
     IT UPDATED. Every one of these was fixed centrally, and
     every device that never got an update is still
     vulnerable. YOUR UPDATE MECHANISM IS A SECURITY CONTROL
     (chapter 15).                                        */
```

How it shows up

Symptom	Cause
Any server's valid certificate is accepted	`set_hostname` not called — no name verification
Handshake fails with "unknown ca"	Root not in the trust store, or an incomplete chain from the server
Handshake fails only after a power cycle	No RTC — certificate dates fail
Out of memory during the handshake	16 KB record buffers — set `MAX_CONTENT_LEN`
Connection negotiates an old version	No minimum version floor
Requests executed twice	0-RTT early data replay
One extracted device decrypts the whole fleet	A shared PSK instead of per-device keys
Traffic content inferred without decryption	Record sizes and timing — pad
Connection drops after hours of traffic	AEAD limits reached, KeyUpdate not implemented
DTLS server used for amplification	Cookie exchange disabled
Everything works, and a proxy reads it all	Verification disabled somewhere in the build

Check yourself

- ✓ Draw the TLS 1.3 handshake and say what each message contributes.
- ✓ Explain what CertificateVerify and Finished each prevent.
- ✓ List six things TLS 1.3 removed and the attack each removal closed.
- ✓ Explain how the record nonce is derived and the three properties it gives at once.
- ✓ State the two costs of 0-RTT.
- ✓ Say what DTLS adds over TLS and why the cookie exchange exists.
- ✓ Name the dominant resource constraint for TLS on an MCU and four ways to reduce it.
- ✓ Give the trade-offs of PSK mode and the one rule that makes it survivable.

- ✓ Write the six client-side configuration steps, and say which omission is silent.
- ✓ Explain why almost no TLS attack broke a cipher, and what that implies for your own protocols.

Key management and secure storage

Cryptography converts the problem of protecting a lot of data into the problem of protecting a small key. That is a good trade only if you actually protect the key — and on a device an attacker can hold in their hand, that is the hardest part of the whole system.

The key lifecycle

```
GENERATE -> STORE -> USE -> ROTATE -> DESTROY
     |         |       |        |         |
good entropy   |   never in     |     verifiably
(chapter 02)   |   plaintext    |     unrecoverable
               |   outside the  |
          hardware-backed,      |
          never exported   before the limits
                           (chapter 09)

AT EVERY STAGE, ASK:
  · who can read this key?
  · what happens if the device is stolen and
    decapsulated?
  · what happens if the manufacturing database leaks?
  · what happens when this key must be replaced?
  · how do we know it was destroyed?

THE HIERARCHY, and the whole design follows from it:

  ROOT OF TRUST      in hardware fuses or a PUF.
                     NEVER READABLE BY SOFTWARE.
     |  KDF (chapter 10)
     +-- storage key         encrypts the filesystem
     +-- attestation key     proves device identity
     +-- firmware key        decrypts update images
     +-- per-session keys    ephemeral, from ECDHE

THE ROOT IS USED ONLY AS A KDF INPUT, and ideally only
inside a hardware block that will not reveal it. Every
working key is derived, so every working key is
REPLACEABLE by changing its context string - without
touching the root.
```

Where keys must never be

```
/* --- 1. HARDCODED IN FIRMWARE --- */
    static const uint8_t key[16] = { 0xde, 0xad, ... };

    Anyone with the firmware image has the key. And firmware
    images are on vendor download pages, extracted from SPI
    flash with a $20 clip, or dumped over an unprotected
    JTAG port. `strings` and `binwalk` find them in seconds;
    entropy analysis finds them even when obfuscated.

    AND THE MULTIPLIER: it is the SAME KEY IN EVERY UNIT. One
    extraction compromises the entire product line, past and
    future. This is the single most common serious flaw in
    shipped IoT devices.

/* --- 2. IN A REPOSITORY --- */
    Committed keys live in git history forever. Rotating
    after a leak means rotating on every device that ever
    received the old one. Automated scanners find published
    keys within minutes of a push.

/* --- 3. IN LOGS, TRACES OR CRASH DUMPS --- */
    printf("key=%s", key)  - and it goes to a UART pad
    somebody can solder to, or to a cloud crash reporter, or
    into a support bundle a customer emails you.

/* --- 4. IN SWAP, HIBERNATION IMAGES OR CORE DUMPS --- */
    On Linux: mlock() the buffer, disable core dumps for the
    process, and prefer the kernel keyring or a TEE
    (see the isolation track).

/* --- 5. LEFT IN MEMORY AFTER USE --- */
    void f(void) {
        uint8_t key[32];
        derive(key);
        use(key);
        memset(key, 0, 32);   /* <-- THE COMPILER MAY DELETE
                                 THIS. The buffer is dead
                                 afterwards, so the store
                                 is "unobservable" and
                                 dead-store elimination
                                 removes it. */
    }

    /* USE ONE OF: */
    explicit_bzero(key, sizeof key);           /* BSD, glibc */
    memset_s(key, sizeof key, 0, sizeof key);  /* C11 annex K */
    mbedtls_platform_zeroize(key, sizeof key);
    sodium_memzero(key, sizeof key);
    SecureZeroMemory(key, sizeof key);         /* Windows */

    /* AND REMEMBER THE COPIES: a value in a register, a
       stack spill, a DMA buffer, an unencrypted cache line.
       Zeroing is best-effort; keeping the key in hardware
       and never materialising it is the real answer.     */

/* --- 6. THE SAME KEY FOR TWO PURPOSES --- */
    Derive (chapter 10). Always.

/* --- 7. TRANSPORTED FROM THE FACTORY --- */
    If a key is generated outside the device and injected,
```

```
    then it existed somewhere else, and that somewhere is now
    part of your attack surface for the life of the product.
    GENERATE ON-DEVICE, INSIDE THE SECURE ELEMENT
    (chapter 16).                                          */
```

Hardware key storage

```
/* IN INCREASING ORDER OF ASSURANCE AND COST.             */

/* --- OTP FUSES / eFUSE --- */
    One-time programmable bits in the SoC. Blown at
    manufacture.
      + cannot be changed, so cannot be rolled back
      + readable by the boot ROM before software runs
      - LIMITED SPACE, often a few hundred bits
      - VISIBLE UNDER A MICROSCOPE if not protected;
        decapsulation and optical or e-beam probing recovers
        fuse state on older processes
      - and once burned, WRONG IS FOREVER

    USE FOR: root public key HASHES (chapter 16), device
    serials, anti-rollback counters, lifecycle state.
    PREFER NOT TO USE FOR: raw symmetric secrets, unless the
    fuse block is a "hidden" region readable only by the
    crypto engine.

/* --- A HIDDEN/HW-ONLY KEY REGION --- */
    Many SoCs have fuses wired DIRECTLY INTO THE AES ENGINE
    and readable by nothing else. Software says "encrypt with
    the hardware key"; software never sees the key. This is
    the minimum bar for a device-unique storage key, and it
    is often free on parts you already use - check the TRM.

/* --- PUF: physically unclonable function --- */
    Derives a key from manufacturing variation in SRAM
    start-up values or ring oscillators, with error
    correction and helper data.
      + NO KEY IS STORED AT ALL - it exists only while
        powered
      + resistant to at-rest extraction
      - needs careful error correction; helper data must not
        leak the key
      - ageing and temperature drift must be handled

/* --- SECURE ELEMENT / SE --- */
    A separate chip (ATECC608, SE050, OPTIGA) on I2C or SPI.
      + keys GENERATED INSIDE and never exported
      + tamper detection, active meshes, sensors
      + certified against physical attack (Common Criteria)
      + a monotonic counter and a real RNG
      - an EXTERNAL BUS, so bind it to the host: the bus
        traffic must be authenticated, or an attacker just
        replays "authentication succeeded"

/* --- TPM --- */
    Standardised (TCG). PCRs for measured boot, sealing
    (release a key only if the PCRs match an expected boot
    state), attestation, monotonic counters. Discrete or
    firmware.

/* --- A TEE --- */
    TrustZone and a trusted OS. Keys live in secure world
    memory that the normal world cannot address at all. Much
    more flexible than an SE, and much larger attack surface.
    The isolation track covers this in depth.

/* --- HSM --- */
```

```
    For the SERVER side: the CA and signing keys. FIPS 140-3
    Level 3+, tamper-responding, with multi-party
    authorisation. YOUR FIRMWARE SIGNING KEY BELONGS IN ONE,
    not on a build server (chapter 15).

/* THE PRINCIPLE THAT UNIFIES ALL OF THESE:

     THE BEST KEY STORAGE IS ONE WHERE THE KEY NEVER EXISTS
     IN CPU-ADDRESSABLE MEMORY. You hand data in, you get
     results out, and no software bug can leak what software
     never held.                                          */
```

THE LINUX EQUIVALENTS

```
keyctl / the kernel keyring   keys held in kernel memory,
                              not in your process
trusted keys                  sealed to a TPM or a TEE;
                              userspace sees only a blob
dm-crypt / LUKS2              volume key wrapped by an
                              Argon2id-derived key (ch. 10)
fscrypt                       per-directory file encryption
Op-TEE + a TA                 keys never leave secure world
```

See the Linux track for the driver-side mechanics.

Encrypting data at rest

```
/* --- THE KEY WRAPPING PATTERN, and use it everywhere --- */

    DEK   Data Encryption Key    encrypts the actual data
    KEK   Key Encryption Key     encrypts the DEK

    stored:  wrapped_DEK = AES-KW(KEK, DEK)
             ciphertext  = AES-GCM(DEK, data)

/* WHY THE INDIRECTION:
     · ROTATE THE KEK without re-encrypting terabytes -
       unwrap and rewrap one small blob
     · wrap the SAME DEK under SEVERAL KEKs: a user
       password, a recovery key, a corporate escrow key
     · the DEK can be random and full-strength even when the
       KEK comes from a weak password
     · CRYPTO-ERASE: destroy the wrapped DEK and the data is
       instantly unrecoverable, however many copies exist.
       This is how a "secure erase" completes in
       milliseconds on an SSD.

/* --- FULL-DISK ENCRYPTION AND THE SECTOR PROBLEM --- */
    A disk sector must be encrypted in place: same size in,
    same size out. THERE IS NO ROOM FOR A NONCE OR A TAG.

    So FDE uses XTS-AES: a tweakable block cipher where the
    SECTOR NUMBER is the tweak. Same plaintext in different
    sectors gives different ciphertext.

    AND THE HONEST LIMITATION: XTS PROVIDES NO INTEGRITY. An
    attacker who can write to the disk can flip ciphertext
    bits and corrupt the plaintext in a controlled,
    block-local way. FDE protects a LOST OR STOLEN device; it
    does not protect against an attacker who can modify the
    storage while you keep using it.

    IF YOU NEED INTEGRITY TOO: dm-verity for read-only
    volumes (a Merkle tree, chapter 07), dm-integrity plus
    dm-crypt for read-write, or a filesystem with built-in
    authentication.

/* --- FOR AN MCU WITH EXTERNAL FLASH --- */
    struct record {
        uint32_t version;      /* AAD - anti-rollback */
        uint8_t  nonce[12];    /* from a monotonic counter */
        uint8_t  ciphertext[N];
        uint8_t  tag[16];
    };

    key = KDF(hw_unique_key, "config storage v1");

    /* THE THREE THINGS TO GET RIGHT:
       1. THE NONCE COUNTER MUST SURVIVE RESET. In flash,
          not RAM. If it can ever restart, use
          AES-GCM-SIV or XChaCha20 (chapter 09).
       2. THE VERSION IS AUTHENTICATED, so an attacker
          cannot restore an old config blob - that is a
          rollback attack on your settings, and it is how
          "factory reset the password" attacks work.
       3. THE KEY IS DEVICE-UNIQUE, so a blob copied from
          one unit does not decrypt on another.
```

```
   /* AND ON WEAR LEVELLING: flash controllers move and copy
      blocks. An "overwritten" secret may still be present
      in a remapped block. Never rely on overwriting flash
      to destroy a secret - CRYPTO-ERASE instead: destroy
      the key and let the ciphertext rot.               */
```

Rotation, revocation and destruction

```
/* --- WHY ROTATE --- */
    · the AEAD usage limits (chapter 09)
    · to bound the damage from an undetected compromise
    · because someone left the company
    · because an algorithm aged out

/* --- HOW, WITHOUT AN OUTAGE --- */
    Every stored or transmitted object carries a KEY ID:

      struct blob {
          uint8_t  key_id;      /* which key encrypted this */
          uint8_t  nonce[12];
          ...
      };

    ROLLOUT:
      1. deploy the new key everywhere; keep accepting both
      2. switch WRITERS to the new key
      3. re-encrypt or let old data expire
      4. remove the old key

    A DESIGN WITH NO KEY ID CANNOT BE ROTATED WITHOUT A FLAG
    DAY. Add the field on day one; it costs one byte.

/* --- DESTRUCTION --- */
    · overwrite the RAM copy with an explicit zeroise
    · CRYPTO-ERASE anything on flash or disk
    · blow a fuse to disable a key slot, if the hardware
      supports it
    · and for an HSM, use its documented zeroise procedure -
      and get the certificate of destruction

/* --- LIFECYCLE STATES, which the SoC usually enforces --- */
      BLANK       -> nothing provisioned, debug wide open
      DEVELOPMENT -> keys provisioned, JTAG open, TEST KEYS
                     ONLY
      SECURED     -> production keys, DEBUG DISABLED,
                     secure boot enforced
      RMA         -> a controlled path back to debug, which
                     MUST ERASE ALL SECRETS FIRST
      DECOMMISSIONED -> keys destroyed

    THE TRANSITIONS ARE ONE-WAY AND FUSE-BACKED. THE RMA PATH
    IS THE DANGEROUS ONE: if a returned device can be put
    back into debug WITHOUT erasing its keys, an attacker
    with physical access has a key extraction procedure that
    the vendor documented for them. Verify that your SoC's
    RMA transition erases, and that it cannot be triggered
    without an authenticated command.

/* --- AND FINALLY, THE PART THAT IS NOT TECHNICAL --- */
    Write down, before the product ships:
      · every key in the system, and where it lives
      · who can access each one, and how that is enforced
      · the rotation procedure for each
      · what happens when each one is compromised
      · who is called, and in what order

    A COMPROMISE RESPONSE PLAN WRITTEN DURING AN INCIDENT IS
```

```
    NOT A PLAN. The devices are in the field; the options
    you have are the ones you designed in.              */
```

How it shows up

Symptom	Cause
Every unit of a product shares a key	Hardcoded in firmware — one extraction breaks the line
Key found by `strings` on the image	Key compiled in
Key recovered from a core dump	`memset` optimised away, or no `mlock`
Config blob copied between devices works	Storage key not device-unique
Old settings restored to bypass a lock	Version not authenticated — rollback on stored data
Secret still present after "erase"	Flash wear levelling — needed crypto-erase
Disk contents corrupted in a controlled way	XTS gives confidentiality, not integrity
Secure element bypassed entirely	Unauthenticated I2C bus — responses replayed
Cannot roll a key without a flag day	No key ID in the wire or storage format
Returned device yields production keys	RMA transition re-enables debug without erasing
Nonce repeats after a brownout	Counter kept in RAM instead of flash

Check yourself

- ✓ Draw the key hierarchy and explain why only derived keys are ever used.
- ✓ List seven places a key must never be, with the concrete attack for each.
- ✓ Explain why `memset` may not clear a key and name three portable alternatives.
- ✓ Compare fuses, PUFs, secure elements, TPMs, TEEs and HSMs on assurance and cost.
- ✓ State the principle that unifies all good key storage.
- ✓ Explain the DEK/KEK pattern and four things it enables.
- ✓ Explain why FDE uses XTS and what XTS does not provide.
- ✓ Write the three rules for encrypting a config blob on external flash.
- ✓ Explain crypto-erase and why flash wear levelling makes it necessary.
- ✓ Describe a zero-downtime key rotation and the one field it requires.
- ✓ List the lifecycle states and say why RMA is the dangerous transition.

Side channels and constant-time programming

The algorithm is secure. The implementation runs on real hardware that takes measurable time, draws measurable current and emits measurable radiation. Everything it does with a secret is potentially visible.

What leaks

```
THE CHANNELS, roughly by how easy they are to exploit:

TIMING             How long an operation takes.
                   Measurable REMOTELY, over a network,
                   by averaging many samples. No physical
                   access needed.

CACHE              Which memory lines were touched.
                   Measurable BY ANOTHER PROCESS on the
                   same machine - or another VM on the
                   same host.

BRANCH PREDICTOR   Which way a branch went. Same
                   cross-process reach as cache.

POWER (SPA/DPA)    Instantaneous current draw. Needs a
                   resistor in the power line and an
                   oscilloscope. ~$100 of equipment.

ELECTROMAGNETIC    Near-field emissions. Same information
                   as power, WITHOUT TOUCHING THE BOARD.

FAULT INJECTION    Not observation - INDUCED ERROR.
                   Voltage glitch, clock glitch, laser,
                   EM pulse. Makes the device compute
                   wrong, and wrong answers leak.

ACOUSTIC, THERMAL, PHOTONIC - real, demonstrated,
rarely the easiest route.

THE THREAT MODEL DECIDES WHICH MATTER:

  a server in a datacentre     -> timing, cache, branch
  a phone running your app     -> plus power, and
                                  untrusted co-resident
                                  code
  a device in a customer's
    hands                      -> ALL OF THEM. They own
                                  the hardware, they have
                                  unlimited time, and
                                  ChipWhisperer costs
                                  less than a phone.

AND THE RULE THAT FOLLOWS: IF THE ATTACKER CAN HOLD THE
DEVICE, ASSUME THEY CAN SEE POWER AND INJECT FAULTS.
Design for it, or accept that per-device keys will be
extracted.
```

The constant-time discipline

```
/* THE RULE, and it is short:

     NO BRANCH, NO MEMORY ADDRESS, AND NO VARIABLE-LATENCY
     INSTRUCTION MAY DEPEND ON SECRET DATA.

   Everything below follows from it.                      */

/* --- VIOLATION 1: BRANCHING ON A SECRET --- */
    if (secret_byte == 0x42) do_a(); else do_b();
    /* the branch predictor and the instruction timing both
       reveal which way it went */

/* --- VIOLATION 2: EARLY EXIT --- */
    for (i = 0; i < n; i++)
        if (a[i] != b[i]) return 0;    /* <-- leaks the
                                              match length */

    /* CONSTANT-TIME COMPARISON: */
    int ct_memcmp(const void *a, const void *b, size_t n)
    {
        const volatile uint8_t *pa = a, *pb = b;
        uint8_t diff = 0;
        for (size_t i = 0; i < n; i++)
                diff |= pa[i] ^ pb[i];   /* ALWAYS all n */
        return diff;                     /* 0 == equal */
    }
    /* the `volatile` stops the compiler proving it can
       stop early. Better still, call the library's:
       sodium_memcmp, CRYPTO_memcmp, mbedtls_ct_memcmp. */

/* --- VIOLATION 3: SECRET-DEPENDENT MEMORY ADDRESSES --- */
    value = table[secret_index];
    /* WHICH CACHE LINE was loaded is visible to any process
       sharing the cache. This is exactly the AES T-table
       attack (chapter 04), and it recovers a full AES key
       from another process in seconds. */

    /* CONSTANT-TIME TABLE LOOKUP - touch EVERY entry: */
    uint8_t ct_lookup(const uint8_t *tab, size_t n,
                      size_t secret_idx)
    {
        uint8_t r = 0;
        for (size_t i = 0; i < n; i++) {
                uint8_t m = ct_eq(i, secret_idx);  /* 0xFF or
                                                      0x00 */
                r |= tab[i] & m;
        }
        return r;
    }
    /* O(n) per lookup. For a 256-entry S-box this is
       expensive, which is why AES is BITSLICED instead -
       computing the S-box arithmetically with no table at
       all. */

/* --- VIOLATION 4: VARIABLE-LATENCY INSTRUCTIONS --- */
    Integer DIVIDE and MODULO take data-dependent time on
    many cores, including several Cortex-M and Cortex-A
    parts. So do some multiplies on small cores (the classic
    ARM7TDMI multiplier terminated early on small operands).
    AVOID div/mod ON SECRETS entirely; use masking and
```

```
    shifts.

/* --- THE PRIMITIVES YOU BUILD EVERYTHING FROM --- */

    /* mask: 0xFFFF... if b, else 0 */
    static inline uint32_t ct_mask(uint32_t b)
    {   return -(uint32_t)(b & 1); }

    /* select a or b without branching */
    static inline uint32_t ct_select(uint32_t cond,
                                     uint32_t a, uint32_t b)
    {   uint32_t m = ct_mask(cond);
        return (a & m) | (b & ~m); }

    /* equality -> 1 or 0, no branch */
    static inline uint32_t ct_eq(uint32_t x, uint32_t y)
    {   uint32_t d = x ^ y;
        return 1 ^ ((d | -d) >> 31); }

    /* greater-than, for bignum comparison */
    static inline uint32_t ct_gt(uint32_t x, uint32_t y)
    {   return (((y - x) ^ ((x ^ y) & ((x - y) ^ x)))
                >> 31) & 1; }

    /* conditional swap - the heart of a Montgomery ladder
       (chapter 14) */
    static void ct_cswap(uint32_t cond, uint32_t *a,
                         uint32_t *b, size_t n)
    {
        uint32_t m = ct_mask(cond);
        for (size_t i = 0; i < n; i++) {
                uint32_t t = m & (a[i] ^ b[i]);
                a[i] ^= t;  b[i] ^= t;
        }
    }
```

THE COMPILER IS NOT ON YOUR SIDE

A C compiler is permitted to turn your branchless `ct_select` back into a conditional branch if it decides that is faster, and modern optimisers do exactly that. The mitigations, in increasing order of reliability: use `volatile` on the inputs; use an inline-assembly barrier such as `asm volatile("" : "+r"(x))` to make the value opaque; use the library's tested primitives; or, for the innermost loops, write the assembly yourself. Then *verify* — see the tooling section below. This is not hypothetical: it has silently reintroduced timing leaks into libraries that had them fixed.

Power and fault attacks

```
/* --- SPA: SIMPLE POWER ANALYSIS --- */
    Read the secret from a SINGLE trace by looking at it.
    A square-and-multiply exponentiation shows square and
    multiply operations with visibly different current
    profiles - so the exponent bits are readable off the
    oscilloscope screen (chapters 12 and 14).
    DEFENCE: constant operation sequences. The ladder.

/* --- DPA: DIFFERENTIAL POWER ANALYSIS --- */
    Collect thousands of traces with known inputs.
    Hypothesise one byte of the key. Predict an intermediate
    value's Hamming weight. Correlate the prediction with the
    measured power at every time offset. The correct
    hypothesis shows a correlation spike.

    ONE KEY BYTE AT A TIME: 16 x 256 hypotheses for AES,
    instead of 2^128. A few thousand traces and an afternoon.
    CONSTANT-TIME CODE DOES NOT STOP THIS - the timing is
    identical, but the DATA VALUES still modulate the current.

    DEFENCES:
      MASKING     split every secret value into shares:
                  x = x1 XOR x2, with x1 random per
                  execution. Compute on the shares. The
                  measured power then correlates with random
                  values, not the secret. Costs 2-4x in
                  performance and is HARD TO GET RIGHT - the
                  shares must never recombine, including in a
                  register or a pipeline forwarding path.
      HIDING      random delays, dummy operations, shuffled
                  operation order. Raises the trace count,
                  does not eliminate the leak.
      HARDWARE    dual-rail logic, noise generators,
                  integrated voltage regulators. This is what
                  a certified secure element buys you
                  (chapter 18).

/* --- FAULT INJECTION --- */
    Voltage glitch, clock glitch, EM pulse, laser. Skip an
    instruction, corrupt a register, flip a bit.

    THE CLASSIC TARGET IS A SECURITY DECISION:
        if (signature_valid)     <-- glitch the compare
                jump_to_image();     or the branch

    AND THE CRYPTOGRAPHIC ONES:
      · RSA-CRT: one faulty half reveals a prime (ch. 11, 12)
      · Differential fault analysis on AES: one fault in
        round 9 recovers the key from a pair of correct and
        faulty ciphertexts
      · Glitching a deterministic signature to compare two
        outputs (chapter 15)

    DEFENCES:
      · VERIFY AFTER SIGN, always
      · DOUBLE-CHECK SECURITY DECISIONS with different code,
        separated in time, and use non-trivial constants:

            #define OK   0x5A3C96E1u
            #define FAIL 0xA5C36918u
```

```
            /* not 1 and 0 - a stuck-at fault produces 0 or
               all-ones far more easily than a specific
               32-bit value */

            uint32_t v1 = verify(sig);
            random_delay();
            uint32_t v2 = verify(sig);
            if (v1 != OK || v2 != OK || v1 != v2)
                    lockdown();

      · REDUNDANT LOOP COUNTERS, so a skipped iteration is
        detected
      · SENSORS: voltage, clock, temperature, light. Most
        secure MCUs have them; ENABLE THEM.
      · and a real INFINITE-LOOP-FREE failure path: on
        detection, wipe and halt.

/* THE HONEST ASSESSMENT: full fault resistance is
   specialist work. If your threat model includes a funded
   attacker with the device on a bench, PUT THE KEYS IN A
   CERTIFIED SECURE ELEMENT and let people whose whole job
   this is defend them.                                  */
```

Microarchitectural attacks

```
/* The cache and speculation attacks that reshaped CPU
   security from 2018 onward. Relevant to you if your device
   runs untrusted code - a browser, a container, a customer
   app - or if you are writing a driver or a hypervisor.  */

   FLUSH+RELOAD    flush a shared line, wait, time the
                   reload. Fast and precise; needs shared
                   memory (a shared library, page dedup).
   PRIME+PROBE     fill a cache set, wait, time it again.
                   No sharing needed.
   SPECTRE         mistrain the branch predictor so the CPU
                   speculatively reads out of bounds; the
                   access leaves a cache trace even though
                   the result is discarded.
   MELTDOWN        speculative reads across the
                   user/kernel boundary. Fixed in hardware
                   in current parts.
   ROWHAMMER       repeated DRAM row activation flips bits
                   in ADJACENT rows. Not a leak - a WRITE
                   primitive, from reads alone.

/* WHAT THIS MEANS FOR CRYPTOGRAPHIC CODE:
     · NO SECRET-DEPENDENT MEMORY ACCESS - the same rule as
       above, now with a second reason
     · no secret-dependent branches, because speculation
       executes both paths
     · AND THE HARD PART: an attacker on the same core can
       observe your code even if your code is perfect, via
       SMT/hyperthreading. Disable SMT for
       security-critical workloads, or pin them to a
       dedicated core.

/* ON ARM, THE RELEVANT MITIGATIONS: CSDB (consumption of
   speculative data barrier) and the SB instruction bound
   speculation explicitly; the DIT (Data Independent Timing)
   bit in PSTATE on Armv8.4+ guarantees data-independent
   timing for a defined instruction set - SET IT AROUND
   CRYPTOGRAPHIC CODE where the core supports it. See the
   ARM track for the barrier semantics.                 */
```

Verifying it

```
/* YOU CANNOT EYEBALL CONSTANT TIME. Measure and prove.   */

/* --- dudect: statistical timing test --- */
    Run the function with two input classes - one fixed, one
    random - many times. Apply Welch's t-test to the timing
    distributions. If |t| > 4.5, the timing depends on the
    data.
    Easy to integrate, catches real leaks, and NEEDS NO
    SOURCE ANALYSIS. Run it in CI.

/* --- ctgrind / MemSan-based checking --- */
    Mark the secret as UNINITIALISED memory for Valgrind:

        VALGRIND_MAKE_MEM_UNDEFINED(key, sizeof key);
        crypto_operation(key, ...);

    Valgrind already reports branches and memory addresses
    that depend on undefined data - which is EXACTLY the
    property you want to check. An elegant reuse, and it
    finds violations statically along the executed path.

/* --- ChipWhisperer --- */
    Open-source hardware plus software for power analysis and
    glitching. A few hundred dollars. IF YOU SHIP HARDWARE
    WITH KEYS IN IT, YOU SHOULD OWN ONE - because your
    attackers do, and finding your own DPA leak is much
    cheaper than reading about it.

/* --- OTHERS WORTH KNOWING --- */
    · binsec/rel, ct-verif: formal verification of
      constant-timeness
    · Compiler flags will not save you; there is no
      -fconstant-time
    · Cachegrind and perf counters for cache behaviour

/* --- AND THE PRACTICAL CHECKLIST --- */
    [ ] no branch on secret data
    [ ] no table index derived from secret data
    [ ] no div/mod on secret data
    [ ] all comparisons of secrets are constant-time
    [ ] blinding enabled on every asymmetric operation
    [ ] verify-after-sign on RSA-CRT
    [ ] security decisions double-checked, non-trivial
        constants
    [ ] secrets zeroised with an unremovable primitive
    [ ] dudect or ctgrind in CI
    [ ] and, if the threat model warrants it, the keys are
        not in the application processor at all         */
```

How it shows up

Symptom	Cause
MAC tag recovered byte by byte	Early-exit comparison
AES key recovered by a co-resident process	T-table lookups — cache timing
RSA key recovered over the network	Unblinded exponentiation timing

Symptom	Cause
ECDSA key from a few hundred signatures	Nonce bias from a leaky scalar multiply
Key read off an oscilloscope in one trace	SPA — branching square-and-multiply
Key recovered despite constant-time code	DPA — needs masking, not just timing equality
Signature check bypassed by a voltage pulse	Single unprotected branch on the security decision
A prime recovered from one signature	CRT fault, no verify-after-sign
A fixed leak reappears after a compiler upgrade	The optimiser reintroduced the branch
Timing varies only under load	SMT sibling contention — pin or disable SMT

Check yourself

- ✓ List the side channels and say which apply to a device in a customer's hands.
- ✓ State the constant-time rule in one sentence.
- ✓ Write constant-time equality, select, mask and conditional swap from memory.
- ✓ Explain why a constant-time table lookup is O(n) and what AES does instead.
- ✓ Name two instructions with data-dependent latency.
- ✓ Explain why the compiler can undo your branchless code, and three mitigations.
- ✓ Contrast SPA and DPA, and say why constant time does not stop DPA.
- ✓ Explain masking, and the subtlety that makes it hard to implement.
- ✓ Write a fault-resistant verification check and say why the constants are not 0 and 1.
- ✓ Explain Flush+Reload and why speculation makes secret-dependent branches worse.
- ✓ Describe dudect and ctgrind, and say what each catches that the other does not.

Post-quantum cryptography

A sufficiently large quantum computer breaks RSA and every elliptic curve completely. It does not exist yet. For a product that will still be in the field in 2040, that is not a reason to wait.

What quantum computers actually break

```
SHOR'S ALGORITHM (1994)
  Solves FACTORING and DISCRETE LOGARITHM in polynomial
  time.

  BROKEN COMPLETELY:
    RSA           any key size (chapter 12)
    Diffie-Hellman, finite field          (chapter 13)
    ECDH, ECDSA, Ed25519, all curves      (14, 15)

  "Use a bigger key" does not help. A 15360-bit RSA key
  falls to the same algorithm as a 1024-bit one; it just
  needs a slightly larger machine.

GROVER'S ALGORITHM (1996)
  Searches an unstructured space in O(sqrt(N)) instead of
  O(N).

  WEAKENED, NOT BROKEN:
    AES-128   -> effectively ~64-bit    -> UPGRADE
    AES-256   -> effectively ~128-bit   -> FINE
    SHA-256   -> ~128-bit preimage      -> FINE
    SHA-512, SHA-3                      -> FINE

  AND GROVER'S SPEEDUP IS HARD TO REALISE IN PRACTICE: it
  is inherently sequential, so it does not parallelise the
  way classical search does. The consensus is that
  doubling symmetric key sizes is sufficient and probably
  generous.

SO THE SUMMARY IS SIMPLE:

  SYMMETRIC CRYPTO SURVIVES. Use AES-256, SHA-384/512,
  and 256-bit MACs.

  ALL DEPLOYED PUBLIC-KEY CRYPTO DIES. It must be
  replaced.

WHEN? Nobody knows. Current machines have a few hundred
noisy qubits; breaking RSA-2048 is estimated to need
millions of physical qubits with error correction.
Estimates for a cryptographically relevant machine range
from "the 2030s" to "never". PLAN, DO NOT PANIC.
```

HARVEST NOW, DECRYPT LATER

An adversary can record encrypted traffic today and decrypt it once a quantum computer exists. So the relevant question is not "when will quantum computers arrive?" but "how long must this data stay secret?" If the answer is more than about ten years — medical records, state secrets, long-lived device identities — the migration is *already* late. Signatures are less urgent, since a signature only needs to resist forgery while it is still being verified. But a firmware root key burned into a 2026 device that ships until 2035 and runs until 2050 is exactly the case that needs planning now.

The NIST standards

Standard	Was called	Type	Based on
FIPS 203 — ML-KEM	Kyber	Key encapsulation	Module lattices (MLWE)
FIPS 204 — ML-DSA	Dilithium	Signature	Module lattices
FIPS 205 — SLH-DSA	SPHINCS+	Signature	Hash functions only
FIPS 206 — FN-DSA	Falcon	Signature	NTRU lattices
SP 800-208 — LMS / XMSS	—	**Stateful** signature	Hash functions only
HQC	—	Key encapsulation	Codes — a backup for ML-KEM

```
/* SIZES, AND THIS IS THE PART THAT HURTS EMBEDDED WORK
   (128-bit-class parameter sets)                          */

                    PUBLIC KEY   CIPHERTEXT/SIG   SECRET KEY
   X25519               32 B          32 B           32 B
   ML-KEM-768         1,184 B       1,088 B         2,400 B
                        ^37x          ^34x

   Ed25519              32 B          64 B           64 B
   ML-DSA-65          1,952 B       3,309 B         4,032 B
                        ^61x          ^52x
   FN-DSA-512           897 B         666 B         1,281 B
   SLH-DSA-128s          32 B       7,856 B           64 B
   LMS (h=20)            60 B      ~2,800 B          ~50 B

/* WHAT THAT MEANS IN PRACTICE:

   · A TLS HANDSHAKE grows by several kilobytes. Over a
     constrained radio link - LoRa, BLE, 802.15.4 with
     127-byte frames - that is a fragmentation and
     retransmission problem, not just a bandwidth one.
   · A FIRMWARE MANIFEST with an ML-DSA signature is 3 KB
     instead of 64 bytes. Fine.
   · A PUBLIC KEY IN OTP FUSES IS NOW IMPOSSIBLE at 1,952
     bytes. STORE A HASH (chapter 16) - which you should
     have been doing anyway.
   · RAM: ML-KEM-768 key generation needs a few kilobytes of
     working memory. Manageable on a Cortex-M4, painful on
     an M0+ with 16 KB.

/* SPEED IS ACTUALLY GOOD NEWS. Lattice operations are
   number-theoretic transforms and small-integer arithmetic -
   no big-number modular exponentiation. On a Cortex-M4,
   ML-KEM-768 encapsulation is roughly COMPARABLE TO OR
   FASTER THAN X25519, and dramatically faster than RSA. The
   cost is bandwidth and memory, not cycles.              */
```

Key encapsulation, and why the shape changed

```
/* PQ KEY EXCHANGE IS NOT DIFFIE-HELLMAN. There is no
   "raise the peer's value to my secret". The primitive is a
   KEM: KEY ENCAPSULATION MECHANISM.                      */

    KEYGEN()            -> (pk, sk)
    ENCAPS(pk)          -> (ciphertext, shared_secret)
    DECAPS(sk, ct)      -> shared_secret

/* THE FLOW:
     Alice generates (pk, sk), sends pk.
     Bob runs ENCAPS(pk), gets a random shared secret and a
       ciphertext. Sends the ciphertext.
     Alice runs DECAPS(sk, ct) and gets the same secret.

   ONE ROUND TRIP, same as DH, but ASYMMETRIC IN SHAPE - Bob
   chooses the secret rather than both contributing. That
   means SOME DH-BASED PROTOCOL PATTERNS DO NOT TRANSLATE
   DIRECTLY. Noise's DH-based patterns (chapter 13), for
   instance, needed a redesign; the PQNoise variants use KEMs
   explicitly.

/* --- HYBRID MODE: what everyone is actually deploying --- */

    shared_secret = KDF( X25519_output ‖ ML-KEM_output
                         ‖ transcript )

    THE POINT: THE RESULT IS SECURE IF *EITHER* PRIMITIVE IS
    SECURE.
      · if ML-KEM turns out to have a classical break - and
        it is a young algorithm; SIKE was broken in 2022 by a
        single laptop over a weekend, after reaching the
        final round - X25519 still protects you
      · if a quantum computer arrives, ML-KEM still protects
        you

    THIS IS THE RIGHT DEFAULT AND IT IS ALREADY DEPLOYED:
    X25519MLKEM768 is on by default in Chrome and Firefox,
    supported by OpenSSL 3.5+, and used by a large fraction
    of Cloudflare's traffic. If you are adding PQ to a
    product today, ADD IT AS A HYBRID.

/* --- THE CODE, liboqs --- */
    OQS_KEM *kem = OQS_KEM_new(OQS_KEM_alg_ml_kem_768);

    uint8_t pk[OQS_KEM_ml_kem_768_length_public_key];
    uint8_t sk[OQS_KEM_ml_kem_768_length_secret_key];
    OQS_KEM_keypair(kem, pk, sk);
    /* send pk (1184 bytes) */

    /* the peer: */
    uint8_t ct[OQS_KEM_ml_kem_768_length_ciphertext];
    uint8_t ss[OQS_KEM_ml_kem_768_length_shared_secret];
    OQS_KEM_encaps(kem, ct, ss, pk);
    /* send ct (1088 bytes) */

    /* back at the originator: */
    OQS_KEM_decaps(kem, ss, ct, sk);

    /* THEN HYBRID-COMBINE AND RUN IT THROUGH A KDF - never
       use a KEM output directly as a key (chapter 10): */
```

```
    hkdf(prk, x25519_ss ‖ ss, "myproto v2 hybrid", key, 32);

    OQS_KEM_free(kem);

/* NOTE: DECAPS NEVER FAILS. ML-KEM uses an implicit
   rejection construction - a malformed ciphertext produces a
   deterministic but WRONG shared secret rather than an
   error. That is deliberate: it removes a decryption oracle
   of exactly the Bleichenbacher shape (chapter 12). Your
   protocol finds out later, when the AEAD tag check fails.
   DO NOT ADD YOUR OWN "IS THIS CIPHERTEXT VALID" CHECK - you
   would be re-creating the oracle.                       */
```

Signatures, and the stateful option

```
/* --- ML-DSA (Dilithium): the general-purpose choice --- */
    Lattice-based, Fiat-Shamir with aborts. Fast, moderate
    sizes, no state. USE THIS unless you have a specific
    reason not to.

/* --- FN-DSA (Falcon): smaller, harder to implement --- */
    666-byte signatures - the smallest lattice option. But
    signing requires FLOATING-POINT GAUSSIAN SAMPLING, which
    is a side-channel minefield (chapter 19) and behaves
    differently across FPUs. VERIFICATION is easy and
    integer-only. A reasonable split: sign in the factory
    with a well-tested implementation, verify on-device.

/* --- SLH-DSA (SPHINCS+): the conservative choice --- */
    SECURITY RESTS ONLY ON HASH FUNCTIONS. No lattices, no
    new mathematical assumptions - if SHA-256 survives, this
    survives. Stateless.
    THE COST: 7-30 KB signatures and slow signing.
    USE FOR: a root of trust you cannot ever update. If your
    boot ROM's root key must be trustworthy for thirty years
    and you cannot patch it, the conservative assumption is
    worth the size.

/* --- LMS AND XMSS: STATEFUL, and read this carefully --- */
    Hash-based Merkle signature schemes. Small keys, moderate
    signatures, very conservative security.

    THE CATCH:

      EACH ONE-TIME KEY MAY BE USED EXACTLY ONCE.
      REUSING AN INDEX LEAKS THE PRIVATE KEY.

    So the signer must keep a COUNTER THAT NEVER GOES
    BACKWARDS - across crashes, backups, VM snapshots, and
    redundant signing servers. A restored backup is a key
    compromise. This is why NIST approved them ONLY for
    firmware signing (SP 800-208): a controlled,
    low-frequency, single-signer environment where the state
    can genuinely be managed.

    DO NOT USE THEM for anything with multiple signers or
    uncontrolled infrastructure.

/* --- THE PRACTICAL RECOMMENDATION TABLE ---
     TLS / session keys        hybrid X25519 + ML-KEM-768
     general signatures        ML-DSA-65 (hybrid with
                               Ed25519 during transition)
     firmware signing, updatable   ML-DSA-65 or LMS
     an unpatchable boot ROM root  SLH-DSA, or LMS with
                               rigorous state management
     symmetric everything      AES-256, SHA-384/512,
                               HMAC-SHA-384              */
```

Migrating a product

```
/* --- STEP 1: INVENTORY --- */
    Every place the product uses public-key cryptography:
      TLS to the cloud, firmware signature verification, the
      device identity certificate, secure boot, attestation,
      any licensing or pairing scheme, third-party SDKs.
    YOU CANNOT MIGRATE WHAT YOU HAVE NOT LISTED, and the
    third-party SDKs are the ones people forget.

/* --- STEP 2: CLASSIFY BY LIFETIME --- */
      confidentiality needed for > 10 years   -> URGENT,
                                                 harvest-now
      device identities lasting > 10 years    -> URGENT
      signatures verified only while current  -> can wait
      session data with short value           -> can wait

/* --- STEP 3: BUILD FOR CRYPTOGRAPHIC AGILITY --- */
    THIS IS THE HIGHEST-VALUE WORK, and it is useful even if
    quantum computers never arrive.

      · ALGORITHM IDENTIFIERS in every stored and transmitted
        format
      · KEY IDs, so keys can be rotated (chapter 18)
      · SIZE-AGNOSTIC BUFFERS AND FIELDS - a protocol with a
        fixed 64-byte signature field cannot ever carry a
        3 KB one. LENGTH-PREFIX EVERYTHING.
      · a NEGOTIATION mechanism, with a floor
      · no algorithm assumptions baked into wire formats or
        flash layouts
      · reserve space: OTP slots for multiple root key
        hashes, flash regions sized for larger manifests

/* --- STEP 4: DEPLOY HYBRIDS FIRST --- */
    Never a flag day. Support both, negotiate, measure, then
    deprecate.

/* --- AND THE HARDWARE QUESTION --- */
    Silicon designed today ships for years and lives for
    decades. If you are specifying an SoC now:
      · does the boot ROM support a PQ signature, or at
        least a second-stage loader that can?
      · is there enough OTP for larger key hashes and more
        slots?
      · enough RAM for lattice operations in the boot path?
      · a hash accelerator - SLH-DSA and LMS are almost
        entirely hashing
    A DEVICE WHOSE MASK-ROM ROOT OF TRUST IS RSA-2048 AND
    WHOSE ROM CANNOT BE UPDATED HAS A FIXED EXPIRY DATE.
    That is a business decision as much as a technical one,
    and it should be made deliberately rather than by
    default.                                              */
```

How it shows up

Symptom	Cause
Handshake fails over a constrained radio	Multi-kilobyte PQ messages exceed the frame or MTU budget
Cannot fit the public key in fuses	PQ keys are kilobytes — store a hash

Symptom	Cause
Out of memory during key generation	Lattice working buffers on a small MCU
A protocol cannot carry the new signature	Fixed-size signature field — no agility
Recorded traffic decryptable in 2040	Classical-only key exchange — harvest now, decrypt later
A new algorithm is broken and there is no fallback	PQ-only instead of hybrid
LMS private key compromised	State counter reused after a restore from backup
Falcon signatures leak the key	Floating-point Gaussian sampling side channel
A decryption oracle reappears	A custom "valid ciphertext?" check added around ML-KEM decaps
The product cannot be migrated at all	Unpatchable mask ROM with a classical root of trust

Check yourself

- ✓ Say precisely what Shor's and Grover's algorithms each do to which primitives.
- ✓ Explain harvest-now-decrypt-later and how it changes the urgency calculation.
- ✓ Name the four NIST standards and what each is for.
- ✓ Quote the size ratios versus X25519 and Ed25519, and name three practical consequences.
- ✓ Explain why PQ speed is not the problem but bandwidth and memory are.
- ✓ Define a KEM and say how its shape differs from Diffie–Hellman.
- ✓ Explain hybrid mode and give two independent reasons for it.
- ✓ Explain why ML-KEM decapsulation never fails, and why adding a check is dangerous.
- ✓ Say when SLH-DSA is worth its signature size.
- ✓ Explain the LMS/XMSS state requirement and why NIST restricted its use.
- ✓ List the four migration steps and the four hardware questions.

Cryptography in embedded systems

Everything in this track, applied to a device with 256 KB of RAM, no clock, a physically accessible attacker and a fifteen-year service life. The closing chapter: what to choose, what it costs, and what actually goes wrong in shipped products.

What makes embedded different

```
THE CONSTRAINTS

MEMORY        20–40 KB of RAM for one TLS session; a
              whole MCU may have 64 KB. RAM, not CPU, is
              usually what stops you.
FLASH         a TLS stack is 60–100 KB. On a 256 KB part
              that is a third of your budget.
POWER         a battery device may have a lifetime energy
              budget. Every handshake costs measurable
              charge.
NO CLOCK      no RTC, or one that resets. Certificate
              dates cannot be checked (chapter 16).
NO ENTROPY    at first boot, before any peripheral has
              produced noise (chapter 02).
NO UPDATES    or updates that many units never take.
BANDWIDTH     a 127-byte 802.15.4 frame; a LoRa payload
              measured in tens of bytes.
LIFETIME      15–20 years. Longer than the algorithms.

THE THREAT MODEL

THE ATTACKER OWNS THE HARDWARE. They can:
  · read the external flash with a $20 clip
  · probe the buses
  · glitch the power and clock (chapter 19)
  · decapsulate the die
  · buy a hundred units and destroy ninety-nine
  · and take as long as they like

AND THE ECONOMICS: BREAK ONE, BREAK ALL. If the key is the
same in every unit, one extraction is a product-wide
compromise, permanently, with no way to recall. This is
the single fact that should drive every design decision
in this chapter.

SO: PER-DEVICE KEYS, GENERATED ON-DEVICE, HELD IN
HARDWARE (chapter 18). Everything else is secondary.
```

Picking the algorithms

Need	Choose	Because
Encryption + integrity	AES-256-GCM with hardware; else ChaCha20-Poly1305	AEAD, one primitive, no composition mistakes (ch. 09)
Nonce uniqueness in doubt	AES-GCM-SIV or XChaCha20-Poly1305	Counters reset; misuse resistance is cheap insurance
Tiny AES-only hardware	AES-CCM	Encrypt direction only; what 802.15.4 uses anyway

Need	Choose	Because
Very constrained	ASCON	NIST lightweight standard; tiny in gates and code
Hashing	SHA-256 with hardware; else BLAKE2s	Accelerators are common; BLAKE2s is fast in software (ch. 07)
MAC	HMAC-SHA-256, or CMAC with AES hardware	ch. 08
Key exchange	X25519	32 bytes, constant time, no validation to get wrong (ch. 14)
Signatures (device)	Ed25519	No nonce to leak, no DER to parse (ch. 15)
Signatures (boot ROM)	Whatever the ROM implements	Not your decision — usually RSA-2048/3072 or ECDSA P-256
Password / PIN	Argon2id if RAM allows; else a hardware attempt counter	ch. 10 — and low entropy needs hardware, not maths
Key derivation	HKDF-SHA-256	Cheap, and it gives you key separation for free
Long-lived roots	Plan for ML-DSA or LMS	ch. 20

```
/* AND THE FIRST QUESTION, BEFORE ANY OF THAT:

     WHAT DOES THE SILICON ALREADY HAVE?

   Read the TRM's security chapter before choosing anything.
   Most modern MCUs have more than developers use:

     AES engine (often with GCM/CCM modes)
     SHA accelerator
     a true RNG
     a PKA / ECC accelerator
     a hardware-only key slot wired into the AES engine
     a unique device ID
     OTP fuses
     a tamper or glitch detector
     a monotonic counter

   USING THE ACCELERATOR IS 10-50x FASTER AND USUALLY MORE
   SIDE-CHANNEL RESISTANT than your software implementation.
   And the hardware key slot means the key never enters
   CPU-addressable memory at all - which is worth more than
   any amount of careful C.

   THE INTEGRATION POINT: mbedTLS ALT implementations
   (MBEDTLS_AES_ALT and friends), or the PSA Crypto API,
   which is the modern way - you ask for "an AES-GCM
   operation with key handle 3" and the implementation routes
   it to hardware or software as available. WRITE AGAINST
   PSA CRYPTO if your stack supports it; it makes the
   hardware/software choice a build-time concern instead of
   a code change.                                          */
```

What it costs

```
/* CORTEX-M4 AT 100 MHz, SOFTWARE, ROUGH ORDERS OF MAGNITUDE

   OPERATION                    TIME        FLASH
   AES-128-GCM, 1 KB            ~250 us     ~6 KB
     with hardware AES          ~10 us      ~1 KB
   ChaCha20-Poly1305, 1 KB      ~150 us     ~4 KB
   SHA-256, 1 KB                ~100 us     ~3 KB
     with hardware              ~5 us
   HMAC-SHA-256                 ~200 us     +1 KB
   X25519 scalar multiply       ~40 ms      ~8 KB
   Ed25519 sign                 ~50 ms      ~12 KB
   Ed25519 verify               ~140 ms
   ECDSA P-256 sign             ~80 ms      ~15 KB
   RSA-2048 verify (e=65537)    ~30 ms      ~10 KB
   RSA-2048 sign (CRT)          ~900 ms
   Argon2id, 16 MB              ~2 s        ~5 KB + 16 MB RAM
   ML-KEM-768 encapsulate       ~15 ms      ~15 KB

   A FULL TLS 1.3 HANDSHAKE, ECDHE + ECDSA, in software:
     ~400-800 ms and ~25 KB of RAM.
   With a PKA accelerator: ~50-100 ms.

/* THE ENERGY VIEW, which matters more on a battery:
     one handshake  ~ 50-200 mJ in software
     one AEAD record ~ negligible

   SO THE DESIGN RULE FOR BATTERY DEVICES:
     HANDSHAKE RARELY, ENCRYPT FREELY.
       · session resumption / PSK tickets, so one handshake
         per boot rather than per message
       · long-lived sessions rather than reconnecting
       · batch messages
       · and if the device wakes to send 20 bytes once an
         hour, a full TLS handshake each time may cost more
         energy than the radio. Consider a
         provisioned-symmetric design (chapter 17's PSK
         mode) with per-device keys, or OSCORE/EDHOC for
         CoAP - which were designed for exactly this
         profile.

/* AND THE SIZING SHORTCUT: if the part has hardware AES and
   SHA, symmetric crypto is essentially free and you should
   use it liberally. It is the PUBLIC-KEY operations that
   cost, so the design question is always "how often do we do
   asymmetric work?"                                        */
```

Four patterns worth copying

```
/* --- 1. SECURE BOOT --- */
    ROM (immutable)
      verify bootloader signature against a key hash in OTP
      check the anti-rollback counter
    BOOTLOADER
      verify the application signature
      check version, device class, image length
      measure into a TPM/PCR if present
    APPLICATION
      verify any signed data it loads

    EACH STAGE VERIFIES THE NEXT BEFORE TRANSFERRING
    CONTROL. The chain is only as strong as the ROM, which
    cannot be changed - so ROM bugs are permanent, and this
    is why boot ROM code is short and heavily reviewed.
    (Full treatment in the isolation track.)

/* --- 2. FIRMWARE UPDATE --- */
    a signed MANIFEST containing:
      format version, firmware version, device class,
      image hash, image length, timestamp
    encrypted image, if confidentiality matters
    verification BEFORE installation
    an anti-rollback counter in fuses
    A/B SLOTS AND A ROLLBACK-ON-FAILURE PATH

    THE ORDERING RULE: EVERY CHECK THAT CAN REJECT HAPPENS
    BEFORE ANYTHING IRREVERSIBLE. And hash the image in its
    final location, so DMA cannot swap it after the check
    (chapter 15).

/* --- 3. DEVICE IDENTITY --- */
    key pair generated INSIDE the secure element at
      manufacture; the private half never exists elsewhere
    a certificate signed by the vendor CA, containing the
      device serial and product class
    mutual TLS to the cloud, validated against a PINNED
      vendor root
    the cloud checks the serial against inventory:
      registered? activated? reported stolen? firmware
      version acceptable?
    (chapters 16 and 17)

/* --- 4. SECURE STORAGE --- */
    key = KDF(hardware_unique_key, "storage v1")
    each record: version ‖ nonce ‖ ciphertext ‖ tag
    version authenticated as AAD -> no rollback
    nonce from a counter IN FLASH, not RAM
    crypto-erase to destroy, never overwrite
    (chapter 18)                                          */
```

THE FIVE RULES, IF YOU REMEMBER NOTHING ELSE

1. PER-DEVICE KEYS, generated on-device, in hardware.
2. USE AN AEAD. Never compose a cipher and a MAC yourself.
3. NEVER REUSE A (key, nonce) PAIR.
4. USE A MAINTAINED LIBRARY, AND KEEP IT UPDATED.
   Your update mechanism is a security control.
5. VERIFY BEFORE YOU ACT. Signature, then version, then
   identity, then use — and only then anything
   irreversible.

The mistakes that ship

```
/* Ranked by how often they appear in real teardowns.     */

   1.  THE SAME KEY IN EVERY UNIT. Found by `strings`.
       One extraction, whole product line.

   2.  NO SECURE BOOT. Or secure boot present but disabled
       in production, because the fuse was never blown on
       the line. CHECK YOUR PRODUCTION UNITS, not your dev
       boards.

   3.  DEBUG PORTS OPEN. JTAG/SWD enabled in shipped
       hardware, or re-enabled by a lifecycle state that
       was never advanced.

   4.  CERTIFICATE VALIDATION DISABLED. The development
       bypass that shipped (chapter 16).

   5.  HOSTNAME VERIFICATION NOT ENABLED. A one-line
       omission that silently accepts any valid certificate
       (chapter 17).

   6.  WEAK ENTROPY AT FIRST BOOT. Keys generated before the
       RNG was seeded (chapter 02), producing duplicate or
       factorable keys across a fleet.

   7.  ROLLING YOUR OWN. A custom "encryption" scheme, an
       XOR obfuscation, a hand-built MAC. It is always
       broken, and it is always broken quickly.

   8.  ECB MODE. Because it was the default parameter
       (chapter 05).

   9.  NO ANTI-ROLLBACK. Old vulnerable firmware reinstalls
       with a perfectly valid signature.

   10. NO UPDATE MECHANISM AT ALL. Which converts every
       future vulnerability into a permanent one.

   11. NONCE COUNTERS IN RAM, resetting on every power
       cycle.

   12. UNAUTHENTICATED BUSES to a secure element - the SE
       is correct, and the attacker replays "OK" on the I2C
       lines.

   13. SECRETS IN LOGS on an accessible UART.

   14. NO PLAN FOR COMPROMISE. No key IDs, no rotation
       path, no revocation, no incident procedure.

/* NOTICE HOW FEW OF THESE ARE CRYPTOGRAPHIC. Almost every
   one is a PROVISIONING, CONFIGURATION, LIFECYCLE or
   PROCESS failure. That is the real lesson of this track:
   the algorithms are the easy part, and they are not where
   your product will fail.                                */
```

A pre-ship checklist

```
   KEYS
   [ ] every key is per-device
   [ ] device keys generated on-device, never exported
   [ ] no key material in the firmware image (verify with
       `strings` and an entropy scan on the RELEASE binary)
   [ ] all working keys derived from a root, with context
       strings
   [ ] key IDs in every stored and transmitted format
   [ ] a written rotation and compromise plan

   BOOT AND UPDATE
   [ ] secure boot enabled AND VERIFIED ON A PRODUCTION UNIT
   [ ] root key hash in OTP, with spare slots
   [ ] anti-rollback counter, and it is actually incremented
   [ ] update images signed; manifest includes version and
       device class
   [ ] verify-before-install, with A/B and recovery
   [ ] the update path itself has been tested from a
       corrupted state

   PROTOCOL
   [ ] AEAD everywhere; no hand-composed cipher+MAC
   [ ] nonce uniqueness guaranteed by a mechanism, not a hope
   [ ] certificate validation on, hostname verification on,
       root pinned
   [ ] minimum TLS version floored
   [ ] replay protection (sequence numbers) present
   [ ] tested against expired, wrong-name, self-signed and
       malformed certificates

   IMPLEMENTATION
   [ ] a maintained library, pinned version, CVE monitoring
       subscribed
   [ ] hardware accelerators used where present
   [ ] constant-time comparisons for all secrets
   [ ] secrets zeroised with a non-removable primitive
   [ ] dudect or ctgrind in CI if you wrote any primitive
       yourself

   LIFECYCLE
   [ ] debug ports disabled in the SECURED state
   [ ] the RMA path erases secrets before re-enabling debug
   [ ] no secrets on any log output
   [ ] production units audited, not just development
       boards

/* AND THE LAST ONE, WHICH IS NOT A CHECKBOX:

     HAVE SOMEONE ELSE TRY TO BREAK IT.

   An external review, or at minimum a colleague who did not
   write it, with a logic analyser, a flash clip and a
   ChipWhisperer. Every item above is something a reviewer
   finds in an afternoon and a designer misses for a year.
                                                          */
```

How it shows up

Symptom	Cause
A published exploit affects every unit ever made	Shared key, no update mechanism
Firmware dumped from SPI flash and read directly	No image encryption, no secure boot
Root shell over an unpopulated header	Debug port left enabled in production
Cloud traffic readable through a proxy	Certificate or hostname verification disabled
Duplicate device keys across a batch	RNG unseeded at first boot
Vulnerable firmware reinstalled deliberately	No anti-rollback counter
Handshake drains the battery	Full TLS handshake per wake-up — resume or use PSK
Out of RAM when the connection opens	16 KB TLS record buffers on a 64 KB part
Secure element bypassed	Unauthenticated bus between host and SE
Nothing can be fixed after disclosure	No update path, no key rotation, no revocation
It worked on the dev board	Fuses never blown on the production line

Check yourself

- ✓ List the eight embedded constraints and say which usually binds first.
- ✓ State the "break one, break all" economics and the design rule it forces.
- ✓ Choose algorithms for eight needs and justify each in one line.
- ✓ Say what to look for in a TRM before choosing anything, and why PSA Crypto helps.
- ✓ Recall the rough cost of a software TLS handshake in time, flash, RAM and energy.
- ✓ State the battery design rule and three ways to apply it.
- ✓ Describe the four standard patterns: secure boot, update, identity, storage.
- ✓ Recite the five rules.
- ✓ Name ten of the fourteen shipping mistakes, and say what most of them have in common.
- ✓ Walk the pre-ship checklist and name the item that is not a checkbox.

TRACK

Linux kernel & drivers

Kernel internals and driver development, ending in two complete drivers you build and break yourself.

32 chapters

The kernel source tree and how to navigate it

Forty million lines of C, and you will never read most of it. What you need is the skill of finding the twenty lines that matter in under a minute — which is a learnable, mechanical skill, and the first one to acquire.

Getting it

```
# The mainline tree, full history. ~4 GB, worth having.
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux

# Shallow, if you only want to read one version:
git clone --depth 1 --branch v6.12 \
    https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git

# The trees you will care about in embedded work:
#   torvalds/linux          mainline - where everything lands
#   stable/linux-stable     the -stable branches, v6.12.y etc.
#   next/linux-next         everything queued for the NEXT merge
#                           window; check here before writing
#                           something that already exists
#   soc/soc                 the arm-soc tree
#   <vendor>/<board>        the BSP you actually ship, usually
#                           an old LTS plus 3,000 patches

# THE VERSION SCHEME:
#   6.12        a mainline release, roughly every 9-10 weeks
#   6.12.34     a stable update: fixes only, no features
#   6.12-rc3    a release candidate
#   LTS         one release a year, maintained ~2-6 years.
#               6.1, 6.6, 6.12... - this is what products ship.
```

The layout

```
linux/
  arch/          per-architecture code. arch/arm64/ is yours.
      arm64/boot/dts/       DEVICE TREES live here
      arm64/kernel/         entry.S, setup.c, the exception
                            vectors from ARM chapter 27
      arm64/mm/             page table code
      arm64/include/asm/    architecture headers

  block/         the block layer
  certs/         module signing keys
  crypto/        the crypto API

  Documentation/ genuinely good. Start here more often than
                 you think. Documentation/driver-api/,
                 Documentation/devicetree/bindings/ ...

  drivers/       ~70% of the tree. Where your code goes.
      base/      the DEVICE MODEL - bus, driver, device
      gpio/  i2c/  spi/  pci/  usb/  net/  mmc/  tty/
      char/      character devices
      dma/       DMA engine drivers
      iommu/     the SMMU drivers (isolation ch. 03-07)
      tee/       the OP-TEE driver (isolation ch. 12)
      clk/       the common clock framework
      regulator/ power supplies
      pinctrl/   pin muxing
      soc/       vendor SoC glue
      staging/   not yet good enough. Do not learn style here.

  fs/            filesystems. fs/proc/, fs/sysfs/, fs/ext4/ ...
  include/
      linux/     the headers you include. THE reference.
      uapi/      the USERSPACE ABI. Changing these breaks
                 programs; treat as immutable.
      asm-generic/
  init/          start_kernel() lives in init/main.c
  ipc/           SysV IPC
  kernel/        the core: sched/, locking/, irq/, time/,
                 rcu/, workqueue.c, module/, dma/
  lib/           data structures, string ops, sorting
  mm/            memory management: page_alloc.c, slub.c,
                 vmalloc.c, mmap.c
  net/           the network stack
  samples/       small worked examples. Underused.
  scripts/       build tooling, checkpatch.pl, coccinelle
  security/      LSMs: SELinux, AppArmor, IMA
  sound/         ALSA
  tools/         perf, bpf, selftests, and much else
  virt/          KVM
```

> **THREE DIRECTORIES CARRY MOST OF YOUR WORK**
>
> For embedded driver development it is `drivers/` for the code, `include/linux/` for the API you are calling, and `Documentation/` for why. `arch/arm64/boot/dts/` joins them the moment you touch hardware description. Everything else you visit occasionally.

Finding things — the actual skill

```
# SET UP ONE OF THESE ON DAY ONE. Grepping 40 million lines
# interactively does not scale.

# cscope + ctags, the classic:
make ARCH=arm64 cscope tags
#   vim -t symbol_name        jump to a definition
#   Ctrl-]                    follow
#   :cs find c func           who CALLS this?

# Or the modern option, and the better one:
#   clangd + compile_commands.json
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc)
./scripts/clang-tools/gen_compile_commands.py
# now any LSP editor gives you real go-to-definition,
# find-references, and call hierarchies - CONFIG-aware, which
# grep is not.

# Or the web: https://elixir.bootlin.com/linux/latest/source
# Cross-referenced, version-switchable, no setup. Use it when
# you are away from your build machine. It is excellent.

# git grep is fast enough for a lot:
git grep -n "devm_platform_ioremap_resource" -- drivers/ | head
git grep -n "EXPORT_SYMBOL_GPL(dma_alloc_attrs)"
git grep -W "static int spi_probe"        # -W = whole function

# Restrict by directory, always:
git grep -n "regmap_write" -- drivers/iio/
```

The five questions, and how to answer each

```
# 1. "WHAT DOES THIS FUNCTION DO?"
     git grep -n "^\(static \)\?[a-z_ *]*\bfunc_name\b *(" -- '*.c'
   The DEFINITION is what you want, not the 200 call sites.
   Then read the kernel-doc comment above it - most exported
   functions have one, and it is usually accurate.

# 2. "HOW DO I USE THIS API?"
     Find an EXISTING DRIVER that uses it, and copy its shape.
     git grep -l "devm_request_threaded_irq" -- drivers/ | head
   This is the single most effective kernel technique. The tree
   is its own best documentation, because every user compiles
   and most were reviewed.
     Prefer a RECENT, SIMPLE driver: check `git log --oneline -3`
   on the file. Old drivers use deprecated patterns.

# 3. "WHY IS IT LIKE THIS?"
     git log -p --follow drivers/foo/bar.c
     git log -S "the_exact_string" -- drivers/foo/
     git blame drivers/foo/bar.c
   Kernel commit messages are unusually good. The one that
   introduced the odd-looking line usually explains the bug it
   fixed, with the reproducer.

# 4. "WHO CALLS THIS?"
     LSP find-references, or cscope, or:
     git grep -n "\bfunc_name\b" -- '*.c' '*.h'
   Watch for indirect calls through ops structures - grep finds
   `.probe = my_probe` but not the core code that invokes it.
   For those, search for the STRUCT MEMBER name instead:
     git grep -n "->probe(" -- drivers/base/

# 5. "IS THIS CODE EVEN COMPILED?"
     The most common wasted hour. Find the Makefile:
     cat drivers/foo/Makefile
       obj-$(CONFIG_FOO) += foo.o
     then check your .config:
     grep CONFIG_FOO .config
   And check the #ifdefs. Half the file may be behind
   CONFIG_PM or CONFIG_OF. Reading code that your build never
   compiles is a genuinely common failure mode - which is why
   an LSP with your real compile_commands.json beats grep.
```

> **READ A SUBSYSTEM IN THIS ORDER**
>
> 1. `Documentation/` for the subsystem — the design intent.
> 2. `include/linux/<subsys>.h` — the public API and the key structs. This is the contract.
> 3. The smallest, newest driver in that subsystem — the shape of a real user.
> 4. The core file — only now, and only the paths your driver touches.
>
> Going straight to the core file is the mistake everyone makes once. You will read 4,000 lines of generic machinery before learning what it is for.

Conventions that decode the code

You see	It means
`__init`	Used only at boot; freed afterwards. Calling it later is a bug.
`__exit`	Only in a module unload path; discarded if built in.
`__user`	A user-space pointer. **Never dereference** (ch. 08). Checked by `sparse`.
`__iomem`	An MMIO pointer. Use `readl` / `writel`, not `*p` (ch. 23).
`__percpu`, `__rcu`	Access only through the matching accessors.
`__must_check`	Ignoring the return value is a warning — and usually a bug.
`static`	File-local. Most driver functions are. If it is not static, ask why.
`EXPORT_SYMBOL_GPL`	Available to GPL modules only. The absence of an export is a deliberate boundary.
`devm_*`	Device-managed: freed automatically on unbind (ch. 26). Prefer these.
`__weak`	A default that an architecture or driver may override.
`likely()`, `unlikely()`	Branch hints. `unlikely` almost always marks an error path.
`ERR_PTR`, `IS_ERR`, `PTR_ERR`	An error code smuggled inside a pointer. Check with `IS_ERR`, never against `NULL`.
A trailing `_locked` or `__` prefix	The caller must already hold the relevant lock.

Kconfig: how code is selected

```
# drivers/foo/Kconfig
config FOO_DRIVER
        tristate "Foo device support"      # tristate: y/m/n
        depends on ARCH_MYSOC && OF        # only offered if...
        select REGMAP_MMIO                 # force this on
        default n
        help
          Say Y here if you have a Foo device.

# drivers/foo/Makefile
obj-$(CONFIG_FOO_DRIVER) += foo.o          # y -> built in
                                           # m -> a module
foo-objs := foo-core.o foo-hw.o            # multi-file module

# THE THREE STATES, and they matter:
#   y  compiled into vmlinux. Available at boot. No unload.
#   m  a .ko module. Loadable, unloadable, and therefore
#      exercisable - which makes development far faster.
#   n  not compiled at all.

# depends on   the option is HIDDEN unless the dependency is met
# select       forcibly enables another option - use sparingly,
#              it bypasses that option's own dependencies and is
#              a classic source of unbuildable configs

# FINDING WHY AN OPTION IS UNAVAILABLE:
make ARCH=arm64 menuconfig
#   press '/' and search for the symbol. It shows the full
#   dependency chain and what you must enable first. This one
#   keystroke answers 90% of "why can't I turn this on".
```

Where the real answers are

Resource	Use
elixir.bootlin.com	Cross-referenced source, any version. Best browser.
lore.kernel.org	Every mailing list, searchable. The design discussion behind any patch.
LWN.net	The best writing about kernel development anywhere. Read the merge-window summaries.
`Documentation/process/`	How to submit, how to report bugs, the coding style.
`scripts/get_maintainer.pl`	Who owns this file and which list to ask.
`git log`	Underrated. The answer to "why" is usually in a commit message.

```
# Who maintains the code you are about to change?
./scripts/get_maintainer.pl -f drivers/tee/optee/core.c

# What has changed in this file recently, and by whom?
git log --oneline -20 -- drivers/tee/optee/

# Has someone already proposed what I am about to write?
#   search lore.kernel.org, and check linux-next.
# This takes five minutes and occasionally saves a month.
```

How it shows up

Symptom	Cause
Your edits have no effect	The file is not compiled — check `CONFIG_` and the Makefile
Grep finds no callers of a function	Called indirectly through an ops struct — search the member name
The API you copied does not exist	Copied from a different kernel version; check against *your* tree
A Kconfig option cannot be enabled	An unmet `depends on` — use `/` in menuconfig
Enabling one option silently enables five	`select` chains
Code compiles in your editor, not in the build	Editor not using the real `compile_commands.json`; different `#ifdef`s active
Behaviour differs from the documentation	Your vendor BSP is old; read *its* source, not mainline
A pattern from `staging/` gets rejected in review	`staging/` is where non-conforming code waits — never a style reference

Check yourself

- ✓ Name what lives in `drivers/`, `include/linux/`, `include/uapi/`, `arch/arm64/` and `Documentation/`.
- ✓ Set up cscope/ctags or clangd against a configured tree, and jump to a definition.
- ✓ Answer all five questions for a function you have never seen.
- ✓ Explain why finding an existing user beats reading the core implementation first.
- ✓ Decode `__init`, `__user`, `__iomem`, `devm_` and `ERR_PTR` on sight.
- ✓ Read a Kconfig entry and say what it takes to enable it.
- ✓ Explain the difference between `y` and `m`, and why `m` is better during development.
- ✓ Use `get_maintainer.pl` and find the discussion behind a commit on lore.

Kernel versus user-space programming

You already know C. Kernel C is the same language with a different standard library, no safety net, and a set of rules that are not written in the compiler. This chapter is the list of everything that changes, so the first week costs you hours instead of days.

Everything that is different

	User space	Kernel
Standard library	libc: `printf`, `malloc`, `strcpy`	None. A kernel-specific subset: `pr_info`, `kmalloc`, `strscpy`
Floating point	Free	**Forbidden** without `kernel_fpu_begin()`; on ARM64 essentially never
Stack size	8 MB, grows	**16 KB total**, fixed, shared with interrupts
A bad pointer	SIGSEGV, one process dies	Oops or panic; the machine may go down
Memory leak	Freed at exit	Leaked until reboot
Preemption	Always	Depends on context and config; sometimes forbidden
Sleeping	Anywhere	Only in process context, never holding a spinlock
Concurrency	Threads you created	Every CPU, plus interrupts, plus timers — all at once
Debugging	gdb, valgrind, ASan	printk, ftrace, KASAN, and reading dumps
Errors	`errno`, exceptions	Negative return codes: `-ENOMEM`, `-EINVAL`
ABI stability	Stable for decades	**None internally**; the user-facing ABI is sacred
Compiler	Your choice of flags	`-ffreestanding`, no stdlib, kernel flags, strict warnings

The stack is 16 KB

```
/* THE most common beginner crash. On arm64, THREAD_SIZE is
   16 KB by default - and that is the whole kernel stack for
   the task, shared by every function on the call chain.     */

/* WRONG - 8 KB of stack in one line: */
static int my_ioctl(struct file *f, unsigned int cmd,
                    unsigned long arg)
{
        char buf[8192];                 /* half the stack */
        struct big_thing thing;         /* another 2 KB    */
        ...
}

/* RIGHT - allocate it: */
static int my_ioctl(struct file *f, unsigned int cmd,
                    unsigned long arg)
{
        char *buf;
        int ret;

        buf = kmalloc(8192, GFP_KERNEL);
        if (!buf)
                return -ENOMEM;
        ...
        kfree(buf);
        return ret;
}

/* THE RULE OF THUMB: keep any single frame under ~256 bytes,
   and never put an array larger than about 128 bytes on the
   stack. The compiler will warn above 1024 with
   -Wframe-larger-than, and the kernel sets that. DO NOT
   silence it.

   ALSO ON THE STACK, uninvited:
     · interrupt handlers, unless IRQ_STACKS is on
     · deep call chains through the block or network stack
     · recursion - which is why the kernel has almost none

   OVERFLOW SYMPTOM: not a clean error. You corrupt
   thread_info or the adjacent page and crash later,
   somewhere else, with a nonsensical backtrace. Enable
   CONFIG_VMAP_STACK (default on arm64) and overflow becomes a
   clean fault with a real trace instead.                   */
```

Context: the rule everything else follows

```
Kernel code runs in one of three contexts, and what you may do
depends entirely on which:

  PROCESS CONTEXT (also "task context")
     Entered via a syscall, or in a kernel thread.
     You have a `current` task. You may SLEEP.
     -> syscalls, ioctl, read/write, probe(), workqueues

  SOFTIRQ / TASKLET / BOTTOM-HALF CONTEXT
     Deferred interrupt work. No `current` you can rely on.
     MAY NOT SLEEP.
     -> network receive, some timers

  HARDIRQ CONTEXT (interrupt handler)
     Interrupts possibly disabled, must be FAST.
     MAY NOT SLEEP. May not call anything that might.
     -> your request_irq() top half

  AND A FOURTH STATE, orthogonal: ATOMIC.
     Holding a spinlock, or with preemption or interrupts
     disabled. MAY NOT SLEEP - even in process context.

  "MAY NOT SLEEP" MEANS you may not call:
       kmalloc(GFP_KERNEL)      -- may wait for reclaim
       mutex_lock()             -- may block
       copy_to_user()           -- may take a page fault
       msleep(), schedule()
       anything that says "might sleep" in its docs
       vmalloc(), most of the filesystem API

  and you MAY call:
       kmalloc(GFP_ATOMIC)      -- fails rather than waits
       spin_lock()
       readl()/writel()
       pr_info() (with care - it can be slow)

  HOW TO KNOW WHICH CONTEXT YOU ARE IN:
     read the caller. There is no runtime flag that answers it
     reliably - which is why the kernel gives you a checker:

       CONFIG_DEBUG_ATOMIC_SLEEP=y

     turns every illegal sleep into a loud
       "BUG: sleeping function called from invalid context"
     with a backtrace. TURN IT ON. It is the single most
     valuable debug option for a new kernel developer.
```

> **THE QUESTION TO ASK BEFORE EVERY CALL**
> "What context am I in, and can this function sleep?" Nearly every kernel bug a newcomer writes is an answer to that question they never asked. The kernel-doc for a function says *Context:* explicitly for many APIs; where it does not, the convention is that `GFP_KERNEL`, `mutex` and `_interruptible` in a name all imply sleeping.

The library you get instead

```
/* PRINTING */
        printf("x=%d\n", x);            /* NO */
        pr_info("x=%d\n", x);           /* yes */
        pr_err(), pr_warn(), pr_debug()
        dev_info(dev, "..."), dev_err(dev, "...")   /* BEST -
                       prefixes the device name automatically */

/* format specifiers differ - use the kernel's: */
        %pK    a pointer, hashed unless privileged
        %pa    a phys_addr_t / dma_addr_t
        %pOF   a device_node's full path
        %pe    an ERR_PTR, printed as "-ENOMEM"
        %*ph   a hex dump of n bytes
        size_t is %zu, not %lu

/* MEMORY */
        malloc(n)   ->  kmalloc(n, GFP_KERNEL)
        calloc      ->  kcalloc(n, size, GFP_KERNEL)
        realloc     ->  krealloc()
        free        ->  kfree()
        large       ->  vmalloc()  (virtually contiguous only)
        page-sized  ->  alloc_pages() / __get_free_pages()
        best        ->  devm_kzalloc(dev, n, GFP_KERNEL)
                        - freed automatically (chapter 26)

/* STRINGS - and the safe variants matter */
        strcpy      ->  strscpy(dst, src, sizeof(dst))
                        NOT strncpy, NOT strlcpy - strscpy
                        always NUL-terminates and returns
                        -E2BIG on truncation
        sprintf     ->  scnprintf(buf, size, ...)
                        returns what was ACTUALLY written,
                        unlike snprintf
        memcpy, memset, memcmp   -- these exist, use freely

/* ERRORS - negative errno, always */
        return -ENOMEM;
        return -EINVAL;
        return -EIO;
        return -EPROBE_DEFER;   /* kernel-specific: "try me
                                   again later" (chapter 17) */

/* POINTERS THAT ARE ERRORS */
        p = some_func();                /* returns ERR_PTR */
        if (IS_ERR(p))
                return PTR_ERR(p);
        /* NOT: if (!p) - a valid error pointer is non-NULL */

/* NO FLOATING POINT. None. The kernel does not save FP state
   on entry, so touching it corrupts user space's registers.
   Use fixed-point arithmetic:                                */
        /* instead of: temp_c = raw * 0.0625;              */
        temp_milli_c = (raw * 625) / 10;

/* DIVISION of 64-bit values on 32-bit builds needs helpers: */
        do_div(n64, d32);               /* modifies n64 */
        div_u64(n64, d32);
        div64_u64(n64, d64);
/* A plain `a / b` on two u64s can fail to link on 32-bit. */
```

Everything runs at once

```
/* In user space you know which threads you created. In the
   kernel, your code can be entered:

     · by every CPU simultaneously
     · re-entered by an interrupt on the same CPU
     · by a timer, a workqueue, or an RCU callback
     · by a different process through the same file descriptor
     · while your module is being unloaded

   ALL AT ONCE, from the first line you write.              */

/* WRONG - a shared counter with no protection: */
static int count;
static ssize_t my_write(...)
{
        count++;                /* read-modify-write RACE */
        ...
}

/* RIGHT - pick a mechanism (chapter 11): */
static atomic_t count = ATOMIC_INIT(0);
        atomic_inc(&count);

/* or, for anything more than a counter: */
static DEFINE_MUTEX(my_lock);           /* process context   */
static DEFINE_SPINLOCK(my_slock);       /* any context       */

/* AND THE SUBTLER ONE - your module can be unloaded while a
   callback is still queued: */
static void my_exit(void)
{
        cancel_delayed_work_sync(&work);  /* SYNC: wait for it */
        del_timer_sync(&timer);           /* SYNC */
        free_irq(irq, dev);
        /* only now is it safe for your code to disappear */
}
/* Get this wrong and you get "BUG: unable to handle page
   fault" inside freed module text - a crash with a
   backtrace pointing at addresses that no longer exist.  */
```

The compiler is not the one you know

```
# The kernel builds with, roughly:
    -ffreestanding          no assumptions about libc
    -fno-common
    -fno-strict-aliasing    the kernel type-puns deliberately
    -fno-delete-null-pointer-checks
    -fno-stack-protector    (or a kernel-specific one)
    -Wall -Wundef -Werror=implicit-function-declaration
    -Wframe-larger-than=1024 (or 2048)
    -std=gnu11
    no -O0 - the kernel REQUIRES optimisation to build
             correctly (inline assembly and __builtin
             constant folding depend on it)

# CONSEQUENCES:
#   · no libc headers. <stdio.h> does not exist. Use
#     <linux/kernel.h>, <linux/string.h>, <linux/types.h>.
#   · u8/u16/u32/u64 and s8..s64 instead of stdint types
#     (though stdint-style names exist via <linux/types.h>)
#   · __le32 / __be32 for explicit endianness, with
#     le32_to_cpu() etc. - and `sparse` checks you converted
#   · GNU extensions are used freely: statement expressions,
#     typeof, __attribute__, designated initialisers,
#     case ranges

# RUN SPARSE. It catches __user/__iomem/endian mistakes that
# the compiler cannot:
    make C=1 M=drivers/mydrv/          # check changed files
    make C=2 M=drivers/mydrv/          # check all files
```

Two kinds of ABI, opposite rules

	Internal kernel API	User-space ABI
Stability	**None.** Functions and structs change every release	**Absolute.** "We do not break user space"
Includes	`include/linux/`	`include/uapi/`, syscalls, ioctl numbers, sysfs paths, netlink messages, `/proc` formats
Consequence	Out-of-tree modules break on every upgrade — a reason to upstream	Design your ioctl interface once, carefully (ch. 21)

> **THE ASYMMETRY IS DELIBERATE**
>
> Internal churn is *allowed* precisely so that the kernel can be refactored to keep the external promise. That is why maintainers care so much about what your driver exposes to userspace and comparatively little about how it is structured internally: the first is forever, the second they can fix themselves.

Style, and why it is enforced

```
# Documentation/process/coding-style.rst. Non-negotiable in
# review, and mechanically checkable:

    ./scripts/checkpatch.pl --strict -f drivers/mydrv/mydrv.c

# The rules that trip people up:
#   tabs, 8 wide. Not spaces.
#   ~80 columns (100 tolerated, but aim for 80)
#   opening brace on the same line, EXCEPT for functions
#   no braces on single-statement bodies
#   typedef is discouraged - write `struct foo`
#   one declaration per line, no assignments in conditions
#   goto for error unwinding is CORRECT and idiomatic (ch. 26)
#   comments: /* ... */, not //, for multi-line

/* The idiomatic error path - learn to read and write this: */
static int my_probe(struct platform_device *pdev)
{
        struct my_dev *d;
        int ret;

        d = devm_kzalloc(&pdev->dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;

        d->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(d->base))
                return PTR_ERR(d->base);

        ret = my_setup(d);
        if (ret)
                return ret;            /* devm cleans up */

        platform_set_drvdata(pdev, d);
        return 0;
}
/* Note: `sizeof(*d)` not `sizeof(struct my_dev)` - so it stays
   correct if the type changes. This is house style and
   reviewers will ask for it.                                */
```

How it shows up

Symptom	Cause
Crash with a nonsensical backtrace	Stack overflow — check for large locals; enable `VMAP_STACK`
`BUG: sleeping function called from invalid context`	Slept while atomic — exactly what `DEBUG_ATOMIC_SLEEP` is for
`scheduling while atomic`	Same cause, caught elsewhere
Undefined reference to `__aeabi_uldivmod` or similar	64-bit division — use `div_u64()`
Undefined reference to `printf`, `malloc`	Used libc; there is no libc
Random user-space FP corruption	Floating point in kernel code
Crash inside freed module memory after `rmmod`	Timer, work or IRQ not cancelled synchronously
Works on one CPU, corrupts under load	Missing locking — it was always racy

Symptom	Cause
Checking a returned pointer against NULL still crashes	It was an `ERR_PTR`; use `IS_ERR`
Module loads on your kernel, not on the target's	No internal ABI stability — rebuild against the target's headers
Sparse warnings about `__user` or endianness	Real bugs. Fix them, don't cast them away

Check yourself

- ✓ List ten things that differ between kernel and user-space C.
- ✓ State the kernel stack size and rewrite a function with a large local buffer.
- ✓ Name the three contexts plus "atomic", and say what each forbids.
- ✓ Decide, for five given functions, whether they may sleep.
- ✓ Translate `printf`, `malloc`, `strcpy` and `sprintf` to their kernel equivalents, and say why `strscpy` beats the alternatives.
- ✓ Explain why floating point is forbidden and show a fixed-point replacement.
- ✓ Explain what `ERR_PTR` is for and check a result correctly.
- ✓ List four ways your code can be entered concurrently.
- ✓ Explain the opposite stability rules for internal and user-facing interfaces.
- ✓ Run `checkpatch.pl` and `sparse` on a file and fix what they report.

The ARM64 boot flow

From power-on to the first line of C. This is where the ARM architecture track and the Linux track meet: exception levels, the MMU, the device tree and secure firmware all appear in the first few hundred instructions, and knowing the sequence is what makes bring-up tractable.

The whole sequence

```
POWER ON
   |
   v
BOOT ROM (BL1)            EL3, MMU off, caches off
   |  verifies and loads BL2 (isolation ch. 14)
   v
BL2                       EL3 (or EL1S)
   |  initialises DRAM, loads BL31/BL32/BL33
   v
BL31 - TF-A runtime       EL3, stays resident forever
   |  installs the SMC handlers, PSCI, the vectors
   |  drops to...
   +--> BL32 - OP-TEE     S-EL1  (isolation ch. 12)
   |
   v
BL33 - U-Boot / UEFI      EL2 (usually) or EL1
   |  loads Image + DTB into DRAM
   |  may verify them (FIT signatures)
   v
KERNEL, at its entry point           EL2 or EL1
   |
   +-- head.S:  primary_entry
   |      preserve x0 = DTB physical address
   |      record the EL we were entered at
   |      if EL2: init_kernel_el -> either stay for KVM,
   |              or drop to EL1
   |      __create_page_tables    build the initial mapping
   |      __primary_switch        turn the MMU ON
   |      set up the stack, clear BSS
   v
start_kernel()            the first C function (chapter 04)
   |
   v
... subsystems, initcalls, drivers ...
   |
   v
init process, PID 1
```

The boot protocol

```
/* Documentation/arch/arm64/booting.rst - the CONTRACT between
   the bootloader and the kernel. Violate any of it and you get
   a silent hang with no output at all.                        */

   THE BOOTLOADER MUST:

   1. Load the kernel Image to a 2 MB-aligned address, anywhere
      in the first usable DRAM. The Image header declares where
      it wants to go and how big it is.

   2. Set:
        x0 = physical address of the DEVICE TREE BLOB
        x1 = 0    (reserved)
        x2 = 0
        x3 = 0

   3. Enter at the first byte of the Image, in:
        EL2 (preferred - enables KVM) or EL1 (non-secure)
        AArch64
        MMU OFF
        D-cache OFF          <-- and this one matters
        I-cache on or off
        interrupts (DAIF) masked

   4. Have CLEANED the kernel image and DTB TO THE POINT OF
      COHERENCY. With the MMU off, the kernel's accesses are
      Device/non-cacheable and will NOT snoop dirty lines the
      bootloader left in the cache (ARM chapters 18 and 19).
      This is the single most common cause of "loads, jumps,
      dies instantly".

   5. Ensure all secondary CPUs are either in a WFE holding
      pen, or - much better - parked and brought up via PSCI
      (isolation ch. 11).

   6. Provide a valid DTB, itself in memory the kernel can
      reach, with /memory and /chosen nodes.

/* THE IMAGE HEADER, first 64 bytes of arch/arm64/kernel/head.S: */

        code0           /* executable code, or an MZ magic for
                           EFI so the same file boots as a PE  */
        code1
        text_offset     /* where to load, relative to 2 MB     */
        image_size      /* total size including BSS - use this
                           to know how much room to leave      */
        flags           /* endianness, page size, placement    */
        res2..res4
        magic           /* "ARM\x64"                           */
        res5            /* PE header offset                    */

/* Read it with: */
        head -c 64 arch/arm64/boot/Image | xxd
```

"IT HANGS BEFORE ANY OUTPUT" — THE FOUR CAUSES

1. **Caches not cleaned** before jumping. The kernel reads stale memory.
2. **Entered at the wrong EL**, or in AArch32, or with the MMU on.
3. **A bad DTB** — wrong address in x0, corrupt magic, or a `/memory` node that does not describe real DRAM.
4. **The console is not where you think.** The kernel is running fine and printing to a UART you are not watching. Try `earlycon` before assuming a crash.

head.S, in order

```
/* arch/arm64/kernel/head.S. Read it once; it is only a few
   hundred lines and it demystifies the whole thing.        */

SYM_CODE_START(primary_entry)
        bl      record_mmu_state      /* were we entered with
                                         the MMU on? (EFI)    */
        bl      preserve_boot_args    /* stash x0..x3 - the
                                         DTB pointer!         */
        bl      create_idmap          /* an IDENTITY map: VA
                                         == PA, so we can keep
                                         executing across the
                                         MMU enable            */
        b       __primary_switch
SYM_CODE_END(primary_entry)

/* THE IDENTITY MAP TRICK - the central problem of MMU enable:

   Before:  PC = 0x40080000 (physical), MMU off
   After:   PC must be 0xffff800008080000 (virtual)

   You cannot change both at once. So:
     1. build a page table where VA 0x40080000 maps to
        PA 0x40080000 (the identity map), AND where the real
        kernel VA range maps to the same physical memory
     2. enable the MMU - the next instruction fetch at
        0x40080000 now translates through the identity map and
        finds the same code. Nothing appears to change.
     3. branch to the high virtual address
     4. tear down the identity map

   Same trick every OS uses, and the reason arm64 requires the
   kernel to be position-independent early on.             */

__primary_switch:
        adrp    x1, reserved_pg_dir
        adrp    x2, init_idmap_pg_dir
        bl      __enable_mmu          /* set SCTLR_EL1.M = 1   */
        bl      clear_page_tables
        bl      create_kernel_mapping /* the real mapping */
        ...
        bl      __primary_switched

__primary_switched:
        adr_l   x8, vectors           /* the exception table
                                         (ARM ch. 27)          */
        msr     vbar_el1, x8          /* now faults are
                                         survivable            */
        ...
        str_l   x21, __fdt_pointer, x5   /* save the DTB addr */
        bl      __pi_memset              /* clear BSS */
        ...
        b       start_kernel             /* into C */
```

Which exception level, and why it matters

```
/* The kernel records the EL it was entered at, and behaves
   differently:                                              */

   Entered at EL2:
        · KVM is possible. The kernel installs a hypervisor
          stub at EL2 and runs itself at EL1.
        · With VHE (ARMv8.1, FEAT_VHE), the kernel runs
          DIRECTLY at EL2 with EL2 registers behaving like EL1
          ones - this is the normal case on modern hardware.
        · dmesg says:  "CPU: All CPU(s) started at EL2"

   Entered at EL1:
        · no KVM. Everything else works.
        · dmesg says:  "CPU: All CPU(s) started at EL1"
        · usually means the bootloader dropped to EL1 itself,
          or a hypervisor is already running above you

/* CHECK IT FIRST when virtualisation does not work: */
        dmesg | grep "CPU(s) started at"
        dmesg | grep -i kvm

/* A bootloader that drops to EL1 before entering Linux
   silently disables KVM forever. On U-Boot this is
   CONFIG_ARMV8_SWITCH_TO_EL1 - leave it off.               */
```

The device tree arrives

```
x0 at entry -> __fdt_pointer -> used in three phases:

  1. EARLY, before the MMU is fully set up:
       early_init_dt_scan()
         - /chosen  -> the kernel command line, initrd, stdout
         - /memory  -> how much DRAM, and where
         - reserved-memory -> carve-outs to keep out of the
           allocator (isolation ch. 18)
       These have to work with almost nothing available, so the
       early FDT code is a self-contained flat parser.

  2. UNFLATTENING, once memblock works:
       unflatten_device_tree() converts the flat blob into a
       tree of struct device_node objects with pointers.
       Now of_find_node_by_*() and friends work.

  3. DEVICE CREATION, much later, from an initcall:
       of_platform_populate() walks the tree and creates a
       struct platform_device for each node with a
       `compatible` - which is what your driver eventually
       binds to (chapter 17).

  So the DTB is consumed three times, in three different
  representations. Knowing which phase you are in explains why
  of_* APIs "do not work yet" in early code.
```

Getting output before the console exists

```
# The single most useful bring-up skill. A normal console is
# only registered once the serial DRIVER probes - hundreds of
# initcalls in. Everything before that is invisible by default.

# EARLYCON - a minimal, direct-MMIO UART writer, available
# almost immediately:

    earlycon                    # use /chosen/stdout-path from DT
    earlycon=pl011,0x9000000    # or name it explicitly
    earlycon=uart8250,mmio32,0xff000000
    earlycon=uart,mmio32,0x1c020000,115200n8

# combine with:
    keep_bootcon    # do not switch away when the real console
                    # registers - you see BOTH, and the handover
    ignore_loglevel # print everything regardless of loglevel
    debug           # raise the loglevel
    initcall_debug  # print every initcall and its result -
                    # invaluable for "it hangs during boot"

# A good bring-up command line:
    earlycon keep_bootcon ignore_loglevel initcall_debug \
      console=ttyAMA0,115200 root=/dev/mmcblk0p2 rw

# And in the device tree:
    chosen {
        stdout-path = "serial0:115200n8";
        bootargs = "earlycon console=ttyAMA0,115200";
    };

# IF EARLYCON PRODUCES NOTHING:
#   the address is wrong; the UART is not clocked; the pin mux
#   is not set; or the driver name does not match. Try writing
#   a character directly from the bootloader to confirm the
#   UART works at all before blaming the kernel.

# LAST RESORT: CONFIG_DEBUG_LL / earlyprintk with a hard-coded
# address, compiled in. Ugly, and it works when nothing else
# does.
```

> **`INITCALL_DEBUG` IS THE BOOT-HANG DEBUGGER**
>
> It prints every initcall before and after it runs, with the time it took. A hang shows as a `calling ...` line with no matching `initcall ... returned`. That names the exact function, which names the driver, in one boot. Pair it with `printk.time=1` and you also get a free boot-time profile.

Bringing up the other CPUs

```
/* Only CPU0 runs everything above. The rest are brought up
   much later, from smp_init(), using the method the device
   tree declares:                                          */

        cpus {
                cpu@0 {
                        device_type = "cpu";
                        compatible = "arm,cortex-a53";
                        reg = <0x0>;
                        enable-method = "psci";
                };
                cpu@1 { ... reg = <0x1>; enable-method = "psci"; };
        };

        psci {
                compatible = "arm,psci-1.0";
                method = "smc";          /* or "hvc" */
        };

/* PSCI path (isolation ch. 11):
        Linux issues SMC(CPU_ON, target_mpidr, entry_point)
        EL3 firmware powers the core and starts it at
        secondary_entry, which repeats the MMU setup and joins
        the scheduler.

   The older "spin-table" method has the bootloader park each
   secondary in a WFE loop watching a mailbox address. It works
   and it cannot power cores off - so PSCI is what you want.

/* SYMPTOM: only CPU0 online.
        cat /proc/cpuinfo          # one entry
        dmesg | grep -i psci
        dmesg | grep "CPU. failed to come online"
   Almost always: wrong `method` (smc vs hvc), missing PSCI
   node, wrong `reg` values (they must match MPIDR_EL1
   affinity), or firmware refusing CPU_ON. Check the TF-A
   console.                                                */
```

How the kernel learns about memory

```
        memory@80000000 {
                device_type = "memory";      /* REQUIRED */
                reg = <0x0 0x80000000 0x0 0x40000000>;  /* 1 GB */
        };

/* Consumed by early_init_dt_scan_memory() into MEMBLOCK - a
   simple early allocator that exists before the real page
   allocator (chapter 05).

   Then:
     reserved-memory nodes are removed from it
     the kernel image itself is reserved
     the initrd is reserved
     memblock hands the rest to the buddy allocator at
     mem_init()

   CHECK IT:
        dmesg | grep -i "Memory:"
          Memory: 892340K/1048576K available (...)
        cat /proc/iomem
        dmesg | grep -i memblock       (with memblock=debug)

   COMMON PROBLEMS:
     · device_type = "memory" omitted -> node ignored -> the
       kernel finds no RAM and hangs immediately
     · #address-cells/#size-cells mismatch -> the reg values
       are parsed wrong -> wildly wrong size, or an address of
       zero
     · memory described that does not physically exist -> a
       crash the first time the allocator hands out that range,
       long after boot
     · a secure carve-out NOT removed -> the kernel maps memory
       the TZASC denies -> an SError at an unpredictable moment
       (isolation ch. 18)                                  */
```

How it shows up

Symptom	Cause
Absolutely no output	Caches not cleaned; wrong EL; bad DTB pointer; or wrong console — try `earlycon` first
`earlycon` works, then output stops	Real console mismatch — add `keep_bootcon` to see the handover
Hangs after "Starting kernel..."	Load address, image alignment, or the DTB
Hangs at a specific driver	`initcall_debug` names it in one boot
"CPU(s) started at EL1", KVM unavailable	Bootloader dropped to EL1
Only CPU0 online	PSCI conduit or `enable-method` wrong; check TF-A
Kernel reports the wrong amount of RAM	`#address-cells` / `#size-cells` mismatch in the memory node
Immediate crash with no RAM found	`device_type = "memory"` missing
SError at a random later instruction	Mapped a carve-out the hardware denies — add `no-map`
Boots from one loader, not another	Boot protocol violated — check register state and cache maintenance

Symptom	Cause
Works cold, fails on warm reset	State left behind by the previous boot; firmware not reinitialising

Check yourself

- ✓ Draw the sequence from power-on to `start_kernel`, naming each stage.
- ✓ State the six requirements of the arm64 boot protocol.
- ✓ Explain why the caches must be cleaned before entering the kernel.
- ✓ Explain the identity-map trick and why the MMU cannot simply be switched on.
- ✓ Say what changes if the kernel is entered at EL1 instead of EL2, and how to check.
- ✓ Name the three phases in which the device tree is consumed.
- ✓ Build an `earlycon` command line for a board and say what to check if it is silent.
- ✓ Use `initcall_debug` to locate a boot hang.
- ✓ Explain how secondary CPUs start, and the three usual reasons they do not.
- ✓ Write a `/memory` node and list four ways it goes wrong.

`start_kernel` and initcalls

One function, about a hundred calls, and by the end the kernel is fully alive. Understanding its order is what lets you answer the two questions that dominate bring-up: *why is my driver probing too early*, and *where exactly did it hang*.

The order, and why it is that order

```
init/main.c :: start_kernel()

  set_task_stack_end_magic()   a canary on the init stack
  smp_setup_processor_id()     which CPU am I
  boot_cpu_init()              mark this CPU online
  page_address_init()
  setup_arch(&command_line)    <-- the big one, per-arch
     |  arch/arm64/kernel/setup.c
     |    setup_machine_fdt()     parse the DTB
     |    parse_early_param()     command line, early options
     |    arm64_memblock_init()   memory map, reserved regions
     |    paging_init()           REAL page tables
     |    bootmem_init()          zones, sparsemem, NUMA
     |    smp_init_cpus()         count CPUs from DT
     |    hyp_mode_check()        EL2 or EL1?
     v
  setup_boot_config()
  setup_command_line()
  setup_nr_cpu_ids()
  setup_per_cpu_areas()        per-CPU variables now usable
  boot_cpu_hotplug_init()

  build_all_zonelists()        the page allocator's structure
  page_alloc_init()
  parse_early_param() / parse_args()   the FULL command line
  jump_label_init()
  setup_log_buf()              printk's ring buffer resized
  vfs_caches_early_init()
  sort_main_extable()
  trap_init()
  mm_core_init()               <-- buddy allocator LIVE.
                                   alloc_pages() works now.
  poking_init()
  ftrace_init()
  early_irq_init()
  init_IRQ()                   the GIC driver runs here
                               (ARM ch. 23-24)
  tick_init()
  init_timers()
  hrtimers_init()
  softirq_init()
  timekeeping_init()
  time_init()                  the arch timer starts
                               (ARM ch. 32)
  local_irq_enable()           <-- INTERRUPTS ON. The system
                                   is now preemptible-ish and
                                   can actually respond.
  kmem_cache_init_late()       slab fully up: kmalloc works
  console_init()               <-- REAL consoles register.
                                   Everything before this was
                                   buffered, or earlycon.
  lockdep_init()
  ...
  rcu_init()
  ...
  fork_init()
  proc_caches_init()
  buffer_init()
  vfs_caches_init()
  pagecache_init()
  signals_init()
```

```
cgroup_init()
...
arch_call_rest_init()
      |
      v
   rest_init()
      kernel_thread(kernel_init)   -> becomes PID 1
      kernel_thread(kthreadd)      -> PID 2
      cpu_startup_entry(CPUHP_ONLINE)  -> CPU0 becomes the
                                          idle task, forever

THE INVARIANT that explains the whole ordering:
each step may only use facilities set up above it. That is
why memory comes before interrupts, interrupts before timers,
timers before scheduling, and drivers after all of it.
```

THREE LINES TO REMEMBER

- `mm_core_init()` — before this, no `kmalloc`. Early code uses memblock instead.
- `local_irq_enable()` — before this, nothing is asynchronous. No timers, no interrupts, no preemption.
- `console_init()` — before this, `printk` output goes into a ring buffer and is only flushed once a console exists. A crash before it, without `earlycon`, is silent.

Initcalls: how everything else runs

```
/* The kernel has thousands of subsystems and drivers to
   initialise. Rather than a giant list of calls, each one
   REGISTERS itself into a section of the binary, and the
   kernel walks those sections in order.                    */

static int __init my_init(void)
{
        pr_info("hello\n");
        return 0;               /* 0 = success. Non-zero is
                                   logged but boot continues. */
}
subsys_initcall(my_init);

/* The macro puts a POINTER to my_init into a section named
   .initcall4.init. The linker script gathers all such
   sections in order, and do_initcalls() walks them.        */

/* THE LEVELS, in execution order: */

  early_initcall          before most things; very restricted
  pure_initcall           0    no dependencies at all
  core_initcall           1    core subsystems
  postcore_initcall       2
  arch_initcall           3    architecture-specific
  subsys_initcall         4    BUSES register here
                               (platform, i2c, spi, pci)
  fs_initcall             5    filesystems
  rootfs_initcall         rootfs
  device_initcall         6    <-- MOST DRIVERS. module_init()
                                   becomes this when built in.
  late_initcall           7    things needing everything else

  and each has a _sync variant that waits for outstanding
  asynchronous work before the next level begins.

/* WHY THE LEVELS EXIST: a driver at device_initcall can rely
   on its bus existing, because buses registered at
   subsys_initcall. Ordering WITHIN a level is link order -
   i.e. Makefile order - which you must never depend on.

/* module_init() is level 6 when built in, and simply "when
   insmod runs" when built as a module. That difference is why
   a driver can work as a module and fail built-in, or the
   reverse.                                                 */

/* DO NOT solve ordering problems by changing your initcall
   level. That is the classic wrong fix - it works on your
   board and breaks on the next one. Use -EPROBE_DEFER
   instead (chapter 17).                                    */
```

Watching it happen

```
# initcall_debug on the command line:

    calling  gic_of_init+0x0/0x1a4 @ 1
    initcall gic_of_init+0x0/0x1a4 returned 0 after 1953 usecs
    calling  my_driver_init+0x0/0x28 @ 1
    initcall my_driver_init+0x0/0x28 returned -517 after 12 usecs
                                              ^^^^
                                              -EPROBE_DEFER

# READS AS:
#   "calling" with no matching "initcall ... returned"  -> IT
#     HUNG HERE. You just found your boot hang, by name.
#   returned -517  -> EPROBE_DEFER: a dependency was missing;
#     it will be retried later. Normal, not an error.
#   returned -19   -> ENODEV: no such device. Usually fine.
#   a huge "after N usecs" -> your boot-time regression.

# Sort the slow ones:
    dmesg | grep initcall | \
      sed 's/.*after \([0-9]*\) usecs/\1 &/' | sort -rn | head -20

# And the graphical version:
    systemd-analyze plot > boot.svg          # userspace
    cat /sys/kernel/debug/bootgraph          # with
                                             # CONFIG_BOOT_PRINTK

# printk timestamps, essential for any of this:
    printk.time=1        # or CONFIG_PRINTK_TIME=y
```

Handing over to user space

```
/* kernel_init() runs as PID 1, still in kernel mode: */

static int __ref kernel_init(void *unused)
{
        kernel_init_freeable();
             /* wait for async work
                do_basic_setup():
                   driver_init()          the device model
                   init_irq_proc()
                   do_ctors()
                   do_initcalls()      <-- ALL levels run here
                prepare_namespace()       mount the root fs */

        free_initmem();      /* __init code and data FREED.
                                "Freeing unused kernel memory:
                                 4096K" in dmesg. This is why
                                calling an __init function
                                later crashes.               */
        mark_readonly();

        /* Then try, in order: */
        if (execute_command) {          /* init= on cmdline */
                ret = run_init_process(execute_command);
                if (!ret) return 0;
                panic("Requested init %s failed (error %d).",
                      execute_command, ret);
        }

        if (!try_to_run_init_process("/sbin/init")  ||
            !try_to_run_init_process("/etc/init")   ||
            !try_to_run_init_process("/bin/init")   ||
            !try_to_run_init_process("/bin/sh"))
                return 0;

        panic("No working init found.  Try passing init= ...");
}

/* THE TWO FAMOUS PANICS:

   "Kernel panic - not syncing: VFS: Unable to mount root fs
    on unknown-block(0,0)"
        the ROOT FILESYSTEM was not found. Causes: wrong
        root=, the storage driver not built in (a module in an
        initramfs you did not supply), the filesystem type not
        compiled in, or the partition table not read.
        Look at the "Available partitions:" list the kernel
        prints just above - if it is empty, the storage driver
        never probed.

   "No working init found."
        the root filesystem MOUNTED but has no init. Wrong
        rootfs image, missing /sbin/init, a dynamically linked
        init with no libraries, or the wrong architecture
        binaries.
        Try init=/bin/sh to get a shell and look around.    */
```

> **`INIT=/BIN/SH` IS THE DEBUGGING TRAPDOOR**
> It skips the entire init system and drops you at a shell with the root filesystem mounted. From there you can check what is present, load modules by hand, and read `/proc`. If *that* works, the problem is in userspace, not the kernel — which halves the search space in one boot.

Kernel parameters

```
/* Two mechanisms, easily confused. */

/* 1. early_param - parsed in setup_arch(), very early, for
      things that must be known before memory is set up:   */

static int __init parse_my_option(char *p)
{
        my_value = memparse(p, &p);
        return 0;
}
early_param("my_option", parse_my_option);
        /* command line:  my_option=64M */

/* 2. module_param - a variable exposed as a parameter,
      settable at insmod time or via the command line:     */

static int buffer_size = 4096;
module_param(buffer_size, int, 0644);
MODULE_PARM_DESC(buffer_size, "Ring buffer size in bytes");

        /* as a module:   insmod mydrv.ko buffer_size=8192
           built in:      mydrv.buffer_size=8192
           at runtime:    /sys/module/mydrv/parameters/buffer_size
                          (because the mode is 0644)        */

/* The permission argument is the sysfs mode. Use 0444 for
   read-only visibility, 0644 to allow runtime changes - and
   only if changing it at runtime is actually safe, which for
   a buffer size it usually is not.                         */

/* SEEING what was passed: */
        cat /proc/cmdline
        cat /sys/module/mydrv/parameters/*
        dmesg | grep "Kernel command line"
```

The ordering problem, properly solved

```
/* THE SITUATION: your driver needs a clock, a regulator, a
   GPIO or a DMA channel that another driver provides - and
   that driver has not probed yet.

   THE WRONG FIXES:
     · changing your initcall level          (board-specific)
     · reordering the Makefile               (fragile, silent)
     · a retry loop with msleep()            (blocks boot)
     · initialising in module_init instead of probe (breaks
       hotplug and unbind)

   THE RIGHT FIX - return -EPROBE_DEFER: */

static int my_probe(struct platform_device *pdev)
{
        struct clk *clk;

        clk = devm_clk_get(&pdev->dev, "core");
        if (IS_ERR(clk))
                return dev_err_probe(&pdev->dev, PTR_ERR(clk),
                                     "failed to get core clock\n");
        ...
}

/* dev_err_probe() is the modern idiom: it logs at error level
   for real errors, at DEBUG level for -EPROBE_DEFER (so you
   are not spammed by the normal case), and returns the error.
   Use it for every resource acquisition in probe.

   The driver core keeps a DEFERRED PROBE LIST and retries
   everything on it whenever any new driver successfully
   probes. Eventually the dependency appears and you succeed -
   in any link order, on any board.                          */

/* WHEN IT NEVER RESOLVES: */
        cat /sys/kernel/debug/devices_deferred
   lists every device still waiting, and why. If your device is
   there at the end of boot, the provider driver is missing,
   not merely late - a missing CONFIG, or a DT node that does
   not match anything.                                       */
```

How it shows up

Symptom	Cause
Boot hangs at a named initcall	That function does not return — `initcall_debug` found it for you
Initcall returns `-517`	`-EPROBE_DEFER`. Normal; check `devices_deferred` if it never resolves
Driver works as a module, not built in	An ordering dependency masked by `insmod` timing — use deferred probe
Crash calling a function after boot	It was `__init` and the memory was freed
No output before a certain point	`console_init()` not reached — use `earlycon`
`Unable to mount root fs on unknown-block(0,0)`	Storage driver not built in, wrong `root=`, or missing filesystem support

Symptom	Cause
`No working init found`	Rootfs mounted but wrong contents — try `init=/bin/sh`
Parameter has no effect	Wrong prefix built-in vs module, or parsed too late
Boot is slow	Sort initcall durations; usually one driver with a long timeout
Works on one board, wrong order on another	Depending on link order — the bug deferred probe exists to remove

Check yourself

- ✓ Explain the invariant that determines `start_kernel`'s ordering.
- ✓ Say what becomes available at `mm_core_init`, `local_irq_enable` and `console_init`.
- ✓ Explain how initcalls are implemented, and name the levels in order.
- ✓ Say what `module_init` maps to when built in versus as a module.
- ✓ Read `initcall_debug` output and identify a hang and a deferral.
- ✓ Explain what `free_initmem` does and the bug it creates for `__init` functions.
- ✓ Diagnose both classic root-filesystem panics.
- ✓ Distinguish `early_param` from `module_param`.
- ✓ Explain why `-EPROBE_DEFER` is the correct fix for ordering, and use `dev_err_probe`.

Kernel memory allocators

Five allocators, layered on one another, each solving a problem the one below it cannot. Choosing wrongly costs you memory, performance, or — with DMA — a driver that silently corrupts data.

The stack of allocators

```
+-----------------------------------------------------------+
|  devm_kmalloc()     device-managed. Freed on unbind.      |
|                     USE THIS IN DRIVERS. (chapter 26)     |
+-----------------------------------------------------------+
|  kmalloc() / kmem_cache_alloc()          SLAB / SLUB      |
|     small objects, PHYSICALLY CONTIGUOUS                  |
|     8 bytes .. a few pages                                |
+-----------------------------------------------------------+
|  vmalloc()                                                |
|     large, VIRTUALLY contiguous only                      |
|     builds page tables; physically scattered              |
+-----------------------------------------------------------+
|  alloc_pages() / __get_free_pages()      BUDDY ALLOCATOR  |
|     power-of-two page runs, physically contiguous         |
|     the foundation everything above sits on               |
+-----------------------------------------------------------+
|  memblock                                                 |
|     boot only, before the buddy allocator exists          |
+-----------------------------------------------------------+
|  PHYSICAL MEMORY, from the device tree                    |
+-----------------------------------------------------------+

And, off to the side but important for drivers:

dma_alloc_coherent()   physically contiguous, DMA-mapped,
                       cache-coherent. (chapter 24)
CMA                    a reserved pool that can serve large
                       contiguous allocations at runtime
```

The buddy allocator

```
/* The foundation: allocates 2^order contiguous PAGES.       */

struct page *alloc_pages(gfp_t gfp, unsigned int order);
void         __free_pages(struct page *page, unsigned int order);

unsigned long __get_free_pages(gfp_t gfp, unsigned int order);
void          free_pages(unsigned long addr, unsigned int order);

        order 0  =  1 page   =  4 KB
        order 1  =  2 pages  =  8 KB
        order 2  =  4 pages  = 16 KB
        ...
        order 10 = 1024 pages = 4 MB    (MAX_ORDER limit,
                                         configurable)

/* HOW IT WORKS: free memory is kept in 11 lists, one per
   order. To satisfy an order-2 request with only order-4
   blocks free, it splits an order-4 into two order-3s, takes
   one, splits it into two order-2s, takes one. On free, if
   the adjacent "buddy" block is also free, they merge back.

   CONSEQUENCE - FRAGMENTATION. After hours of uptime, plenty
   of memory can be free while no high-order block exists.
   An order-0 allocation succeeds; an order-4 fails. This is
   why:

     · high-order allocations must ALWAYS be error-checked
     · a driver that needs 4 MB contiguous at runtime should
       use CMA, or allocate at probe time and keep it
     · "it works after reboot, fails after a week" is often
       exactly this

   SEE THE STATE:
        cat /proc/buddyinfo
   Node 0, zone   Normal   1432  882  341  102   43  12  3 0 0 0 0
                           ^order0                        ^order10
   Lots of low-order, nothing high-order = fragmented.   */
```

kmalloc and the slab allocator

```
void *kmalloc(size_t size, gfp_t flags);
void *kzalloc(size_t size, gfp_t flags);        /* zeroed */
void *kcalloc(size_t n, size_t size, gfp_t f);  /* n*size,
                                                   OVERFLOW-
                                                   CHECKED   */
void *krealloc(const void *p, size_t new, gfp_t f);
void  kfree(const void *p);
size_t ksize(const void *p);       /* what you ACTUALLY got */

/* kmalloc serves from a set of fixed-size caches:
        8, 16, 32, 64, 96, 128, 192, 256, 512, 1024,
        2048, 4096, 8192 ... up to KMALLOC_MAX_SIZE

   ROUNDING UP MATTERS: kmalloc(100) consumes 128 bytes.
   kmalloc(129) consumes 192. In a driver allocating one
   object per packet, choosing 128 over 129 is a 33% memory
   saving. Check with ksize().

   PROPERTIES:
     · PHYSICALLY CONTIGUOUS - so it is DMA-able (with the
       right GFP flags), and virt_to_phys() works on it
     · fast: per-CPU freelists, usually no locking
     · above ~2 pages it falls back to the page allocator, and
       becomes subject to fragmentation

   ALWAYS CHECK THE RETURN:
        p = kmalloc(n, GFP_KERNEL);
        if (!p)
                return -ENOMEM;
   Even GFP_KERNEL can fail. "Small allocations never fail" is
   almost true and entirely unsafe to rely on.             */

/* SIZE ARITHMETIC - use the checked helpers: */
        p = kmalloc(n * size, GFP_KERNEL);      /* CAN OVERFLOW */
        p = kmalloc_array(n, size, GFP_KERNEL); /* checked      */
        p = kcalloc(n, size, GFP_KERNEL);       /* checked+zero */
        p = kmalloc(struct_size(obj, items, n), GFP_KERNEL);
                        /* checked, for flexible array members */
```

Your own slab cache

```
/* When you allocate many objects of ONE type, a dedicated
   cache is tighter and faster than kmalloc's rounding.     */

static struct kmem_cache *my_cache;

static int __init my_init(void)
{
        my_cache = kmem_cache_create("my_request",
                                     sizeof(struct my_request),
                                     __alignof__(struct my_request),
                                     SLAB_HWCACHE_ALIGN, /* flags */
                                     NULL);              /* ctor */
        if (!my_cache)
                return -ENOMEM;
        return 0;
}

        obj = kmem_cache_alloc(my_cache, GFP_KERNEL);
        kmem_cache_free(my_cache, obj);

static void __exit my_exit(void)
{
        kmem_cache_destroy(my_cache);   /* WARNS loudly if
                                           objects are still
                                           allocated - which is
                                           a free leak detector */
}

/* USEFUL FLAGS:
     SLAB_HWCACHE_ALIGN   align to a cache line - avoids false
                          sharing between CPUs (CA ch. 15)
     SLAB_POISON          fill freed objects with a pattern
     SLAB_RED_ZONE        guard bytes around each object
     SLAB_ACCOUNT         charge to the memory cgroup

/* WHY BOTHER:
     · exact sizing: 100-byte objects cost 100 bytes, not 128
     · a NAMED line in /proc/slabinfo, so leaks are visible
       and attributable to your driver
     · kmem_cache_destroy() warning is a built-in leak test

   INSPECT:
        sudo cat /proc/slabinfo | grep my_request
        sudo slabtop -s c            # sorted by cache size  */
```

vmalloc

```
void *vmalloc(unsigned long size);
void *vzalloc(unsigned long size);
void  vfree(const void *addr);

/* Allocates SCATTERED physical pages and maps them into a
   contiguous VIRTUAL range by building page tables.

   USE IT FOR: large buffers where only the CPU touches the
   memory. Firmware images, big lookup tables, module text.

   DO NOT USE IT FOR:
     · DMA. The pages are not physically contiguous, and
       virt_to_phys() DOES NOT WORK on vmalloc addresses.
       This is a top-three driver bug: DMA "works" on a
       machine with little fragmentation, then corrupts
       random memory on a busy one.
     · anything in atomic context - vmalloc can sleep.
     · small allocations - the page-table overhead and TLB
       cost dominate.

   COMPARE:
                      kmalloc         vmalloc
     contiguous       physical+virt   virtual only
     max size         ~4 MB           very large
     speed            fast            slow (page tables, TLB)
     DMA-safe         yes             NO
     can sleep        depends on GFP  always
     virt_to_phys     works           BROKEN

   For a vmalloc buffer you must reach page by page:
        struct page *p = vmalloc_to_page(addr);           */
```

> **THE DMA RULE, STATED ONCE**
>
> Memory for DMA must be obtained from `dma_alloc_coherent()`, or from `kmalloc` / `alloc_pages` and then mapped with `dma_map_single()`. Never `vmalloc`. Never a stack buffer (it may not be DMA-able and its physical address is meaningless to a device). Never a module-static array without mapping it. Chapter 24 covers this properly, but the rule is worth learning before you need it.

Choosing

You need	Use
A small struct in a driver, freed on unbind	`devm_kzalloc()`
A small struct with a different lifetime	`kzalloc()` + explicit `kfree()`
Many objects of one type	`kmem_cache_create()`
An array	`kcalloc()` / `kmalloc_array()`
Whole pages	`alloc_pages()`
A large CPU-only buffer	`vmalloc()`
A DMA buffer, long-lived	`dma_alloc_coherent()`
A DMA buffer, streaming	`kmalloc()` + `dma_map_single()`

You need	Use
Many small DMA buffers	`dma_pool_create()`
Large contiguous at runtime	CMA, via `dma_alloc_coherent()`
In an interrupt handler	`kmalloc(..., GFP_ATOMIC)`, and check the result
Something that must not fail	Preallocate at probe time. There is no other answer.

CMA — contiguous memory at runtime

```
/* THE PROBLEM: a camera or video codec needs 32 MB physically
   contiguous, at runtime, after hours of uptime. The buddy
   allocator cannot deliver that - fragmentation guarantees it.

   THE SOLUTION: reserve a region at boot, but let MOVABLE
   pages (page cache, anonymous memory) use it meanwhile. When
   a contiguous allocation arrives, migrate them out.       */

        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

                cma_pool: linux,cma {
                        compatible = "shared-dma-pool";
                        reusable;          /* <-- makes it CMA */
                        size = <0x0 0x8000000>;   /* 128 MB */
                        alignment = <0x0 0x400000>;
                        linux,cma-default;
                };
        };

        my_codec: codec@1c000000 {
                compatible = "vendor,codec";
                reg = <0x0 0x1c000000 0x0 0x10000>;
                memory-region = <&cma_pool>;
        };

/* Then in the driver, dma_alloc_coherent() transparently comes
   from CMA:                                                  */
        of_reserved_mem_device_init(dev);
        buf = dma_alloc_coherent(dev, SZ_32M, &dma, GFP_KERNEL);

/* CAVEATS worth knowing before you rely on it:
     · migration can FAIL if a page is pinned - so CMA
       allocation can still fail, just much less often
     · migration takes time; a large allocation can stall for
       tens of milliseconds
     · `reusable` is what allows the memory to be borrowed. A
       carve-out without it is simply removed from the kernel's
       view entirely - which is what you want for a security
       boundary (isolation ch. 18), and not what you want for
       a codec buffer.

   MONITOR:
        cat /proc/meminfo | grep Cma
        CmaTotal:  131072 kB
        CmaFree:    98304 kB                               */
```

Finding leaks and overruns

```
# KASAN - the one that finds real bugs. Enable it in
# development and leave it on.
    CONFIG_KASAN=y
    CONFIG_KASAN_INLINE=y
# Catches: use-after-free, out-of-bounds read and write,
# double-free, use-after-scope. Reports with two backtraces -
# where it was allocated and where it was freed. ~2-3x slower
# and ~1/8 more memory, and worth every bit of it.

# KFENCE - a sampling version, cheap enough for production:
    CONFIG_KFENCE=y
    CONFIG_KFENCE_SAMPLE_INTERVAL=100
# Catches a small fraction of the same bugs at near-zero cost.

# kmemleak - finds objects nothing points to any more:
    CONFIG_DEBUG_KMEMLEAK=y
    echo scan > /sys/kernel/debug/kmemleak
    cat /sys/kernel/debug/kmemleak
# Reports false positives; read them, do not just believe them.

# Slab poisoning and red zones, without full KASAN:
    slub_debug=FZPU              # on the command line
    slub_debug=FZPU,my_request   # or just your cache

# Where memory went:
    cat /proc/meminfo
    sudo slabtop -s c
    cat /proc/vmallocinfo
    cat /proc/buddyinfo
    cat /proc/pagetypeinfo
```

> **THE ALLOCATION-FAILURE MESSAGE IS A DIAGNOSIS, NOT JUST AN ERROR**
>
> When an allocation fails the kernel dumps the order requested, the GFP flags, the zone state and a full backtrace. The *order* tells you whether you hit real exhaustion (order 0 failing) or fragmentation (order 0 fine, order 4 failing) — and those have completely different fixes. Read the dump before concluding you are out of memory.

How it shows up

Symptom	Cause
DMA works sometimes, corrupts memory otherwise	`vmalloc` or stack memory used for DMA
Large allocation fails after long uptime	Fragmentation — check `/proc/buddyinfo`; use CMA
Memory shrinks steadily	A leak — `slabtop` to find the cache, then kmemleak
Crash long after a `kfree`	Use-after-free — enable KASAN
Corruption in an unrelated structure	Buffer overrun into the neighbouring object — KASAN or `slub_debug`
`kmem_cache_destroy` warns on unload	Objects still allocated — a leak your driver owns
Allocation returns NULL in an interrupt	`GFP_ATOMIC` pool exhausted — preallocate instead

Symptom	Cause
`BUG: sleeping function called from invalid context` at kmalloc	`GFP_KERNEL` in atomic context (ch. 06)
Memory use far above what you allocated	kmalloc rounding — check `ksize()`, consider a cache
CMA allocation fails	Pages pinned and unmigratable, or the pool is too small
`virt_to_phys` returns nonsense	Called on a vmalloc address

Check yourself

- ✓ Draw the allocator stack and say what each layer adds.
- ✓ Explain buddy fragmentation and read `/proc/buddyinfo`.
- ✓ Explain kmalloc size rounding and its cost in a hot path.
- ✓ Create a slab cache and say what it buys over `kmalloc`.
- ✓ List three things `vmalloc` must never be used for.
- ✓ Choose the right allocator for eight concrete situations.
- ✓ Explain what CMA does, and the difference `reusable` makes.
- ✓ Turn on KASAN and read one of its reports.
- ✓ Read an allocation-failure dump and say whether it is exhaustion or fragmentation.

GFP flags and allocation context

The second argument to every allocation answers one question: *what may the kernel do to satisfy this request?* Get it wrong and you either deadlock the machine or crash with a message you will learn to recognise instantly.

Why a flag at all

```
When memory is short, the allocator has options:

   · hand back a free page immediately          (always fine)
   · reclaim: drop clean page cache             (fast, safe)
   · write back dirty pages to disk             (SLOW, and it
                                                 needs the I/O
                                                 path - which
                                                 may be YOUR
                                                 driver)
   · swap out anonymous memory                  (slower still)
   · compact memory to make a contiguous run    (slow)
   · invoke the OOM killer                      (drastic)
   · give up and return NULL                    (honest)

  Which of those are ALLOWED depends entirely on who is
  asking. Consider:

     your block driver's interrupt handler calls
     kmalloc(GFP_KERNEL), memory is short, so the allocator
     starts writing dirty pages back to disk - through your
     driver - which is currently inside an interrupt handler
     waiting for that same allocation.

     DEADLOCK. The machine stops.

  The GFP flags exist to make that impossible to express by
  accident.
```

The three you will use

Flag	May sleep	Use in	Behaviour when memory is short
`GFP_KERNEL`	**Yes**	Process context: syscalls, ioctl, probe, workqueues	Reclaims, writes back, swaps, may OOM-kill. Rarely fails.
`GFP_ATOMIC`	No	Interrupt handlers, softirqs, holding a spinlock	Uses emergency reserves, then **fails immediately**.
`GFP_NOWAIT`	No	Anywhere you can handle failure	No reclaim, no reserves. Fails readily. Politer than `GFP_ATOMIC`.

```
/* THE RULE, and it is the whole chapter:

     GFP_KERNEL  if you may sleep
     GFP_ATOMIC  if you may not, and failure is a real problem
     GFP_NOWAIT  if you may not, and failure is fine

   "May I sleep?" is answered by your CONTEXT (chapter 02):

     process context, no locks held    -> GFP_KERNEL
     process context, spinlock held    -> GFP_ATOMIC/NOWAIT
     softirq / tasklet / timer         -> GFP_ATOMIC/NOWAIT
     hardirq handler                   -> GFP_ATOMIC/NOWAIT
     RCU read-side critical section    -> GFP_ATOMIC/NOWAIT
     with preemption or IRQs disabled  -> GFP_ATOMIC/NOWAIT  */

/* GFP_ATOMIC IS NOT FREE. It dips into a small emergency
   reserve that exists precisely so that the network stack can
   allocate an skb during an interrupt storm. Draining it
   affects the whole system.

   So: do not reach for GFP_ATOMIC because it "always works".
   It works less often than GFP_KERNEL, not more - it just
   fails instead of waiting. If you find yourself needing large
   or frequent GFP_ATOMIC allocations, the fix is to
   PREALLOCATE at probe time, or to move the work to a
   workqueue where you can sleep (chapter 13).             */
```

What they are made of

```
/* The named flags are compositions of modifiers: */

   GFP_ATOMIC   = __GFP_HIGH | __GFP_KSWAPD_RECLAIM
   GFP_KERNEL   = __GFP_RECLAIM | __GFP_IO | __GFP_FS
   GFP_NOWAIT   = __GFP_KSWAPD_RECLAIM
   GFP_NOIO     = __GFP_RECLAIM
   GFP_NOFS     = __GFP_RECLAIM | __GFP_IO
   GFP_USER     = GFP_KERNEL | __GFP_HARDWALL
   GFP_DMA32    = __GFP_DMA32

/* THE ACTION MODIFIERS:
     __GFP_RECLAIM         may reclaim (implies it can sleep)
     __GFP_IO              may start disk I/O
     __GFP_FS              may call into filesystems
     __GFP_HIGH            may use emergency reserves
     __GFP_NOFAIL          retry forever. Never returns NULL.
                           Discouraged, and for good reason.
     __GFP_NORETRY         try once, give up quickly
     __GFP_NOWARN          do not print the failure dump
     __GFP_ZERO            zero the memory (= kzalloc)
     __GFP_COMP            compound page, for higher orders
     __GFP_MOVABLE         may be migrated (CMA-friendly)

/* ZONE MODIFIERS - which physical memory: */
     __GFP_DMA             the low DMA zone (ISA-era, 16 MB)
     __GFP_DMA32           below 4 GB
     (none)                anywhere in the normal zone

/* On arm64 you rarely need the zone modifiers directly - the
   DMA API handles addressing limits properly via the device's
   dma_mask (chapter 24). Setting GFP_DMA32 by hand is usually
   a sign someone worked around a missing dma_set_mask().  */
```

GFP_NOIO and GFP_NOFS: the recursion flags

```
These exist for one reason: to stop the allocator calling back
into the subsystem that is currently allocating.

  GFP_NOFS   "reclaim, but do not call into filesystems"
       Used BY filesystem code. Without it:
         ext4 allocates -> reclaim -> writes back a dirty page
         -> calls ext4 -> which is already holding a lock
         -> deadlock.

  GFP_NOIO   "reclaim, but do not start any I/O at all"
       Used BY block and storage drivers, and in any code on
       the writeback path. Without it:
         your MMC driver allocates -> reclaim -> writes back
         -> submits a request to your MMC driver -> which is
         mid-allocation -> deadlock.

  WHEN YOU NEED THEM: if your driver is in the storage or
  filesystem path, or is called during suspend/resume while
  the block layer is frozen.

  THE MODERN WAY - scope, not per-call:

     unsigned int flags = memalloc_noio_save();
     ... everything in here, including code you call, behaves
         as if GFP_NOIO ...
     memalloc_noio_restore(flags);

  This is better than passing GFP_NOIO down through every
  function, because it also covers allocations made by library
  code you did not write. The block and MMC layers do this for
  you around resume, which is why you may see GFP_KERNEL
  working in a resume path that "should" need GFP_NOIO.
```

Patterns

```
/* --- 1. probe: the easy case ---------------------------- */
static int my_probe(struct platform_device *pdev)
{
        struct my_dev *d;

        /* process context, no locks -> GFP_KERNEL */
        d = devm_kzalloc(&pdev->dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;
        ...
}

/* --- 2. interrupt handler: preallocate ------------------ */
/* WRONG - allocating in an IRQ, where it can fail: */
static irqreturn_t my_isr(int irq, void *data)
{
        struct my_event *e = kmalloc(sizeof(*e), GFP_ATOMIC);
        if (!e)
                return IRQ_HANDLED;     /* EVENT LOST */
        ...
}

/* RIGHT - a preallocated pool, filled at probe: */
struct my_dev {
        struct my_event  events[MY_NR_EVENTS];  /* fixed pool */
        DECLARE_KFIFO(fifo, struct my_event, MY_NR_EVENTS);
};
static irqreturn_t my_isr(int irq, void *data)
{
        struct my_dev *d = data;
        struct my_event e = { .ts = ktime_get(), ... };

        if (!kfifo_put(&d->fifo, e))
                d->overruns++;          /* counted, not lost
                                           silently */
        return IRQ_HANDLED;
}
/* An overrun counter is a far better failure mode than an
   allocation that quietly returns NULL.                   */

/* --- 3. spinlock held: allocate outside it -------------- */
/* WRONG: */
        spin_lock(&d->lock);
        p = kmalloc(size, GFP_KERNEL);   /* BUG: may sleep */
        list_add(&p->node, &d->list);
        spin_unlock(&d->lock);

/* RIGHT - allocate first, then take the lock: */
        p = kmalloc(size, GFP_KERNEL);
        if (!p)
                return -ENOMEM;

        spin_lock(&d->lock);
        list_add(&p->node, &d->list);
        spin_unlock(&d->lock);

/* This restructuring - allocate outside, insert inside - is
   the standard fix, and it is almost always possible.     */

/* --- 4. a large, optional allocation -------------------- */
```

```
        buf = kmalloc(SZ_1M, GFP_KERNEL | __GFP_NOWARN |
                             __GFP_NORETRY);
        if (!buf)
                buf = vmalloc(SZ_1M);   /* fall back */
/* kvmalloc() does exactly this for you: */
        buf = kvmalloc(SZ_1M, GFP_KERNEL);
        ...
        kvfree(buf);
/* Use kvmalloc when the buffer is CPU-only and you do not
   care whether it is physically contiguous. Never for DMA. */
```

THE DESIGN LESSON UNDERNEATH

Almost every `GFP_ATOMIC` in a driver can be removed by allocating earlier. Fixed pools sized at probe time, kfifos, preallocated descriptor rings and mempools all trade a little memory for the guarantee that the hot path cannot fail. That trade is nearly always right in embedded work, where the memory budget is known and an allocation failure at 3 a.m. is not recoverable.

Mempools: when failure is not an option

```
/* A mempool keeps a guaranteed reserve. Allocation tries the
   normal allocator first; if that fails, it takes from the
   reserve; if the reserve is empty AND you may sleep, it waits
   for something to be returned.                              */

        mempool_t *pool;

        pool = mempool_create_kmalloc_pool(MIN_NR, sizeof(struct req));
        /* or with your own alloc/free callbacks:            */
        pool = mempool_create(MIN_NR, mempool_alloc_slab,
                              mempool_free_slab, my_cache);

        p = mempool_alloc(pool, GFP_NOIO);
        ...
        mempool_free(p, pool);

        mempool_destroy(pool);

/* USED BY: the block layer, dm, md, and anything on the
   writeback path - places where returning NULL would mean
   losing data rather than failing a request.

   THE GUARANTEE holds only if you RETURN objects promptly and
   the reserve is sized for your maximum in-flight count. A
   mempool with MIN_NR=4 and 8 concurrent requests will still
   block.                                                     */
```

Catching mistakes

```
# TURN THESE ON. They convert silent, rare, unreproducible
# corruption into a loud message with a backtrace.

    CONFIG_DEBUG_ATOMIC_SLEEP=y     # THE important one
    CONFIG_PROVE_LOCKING=y          # lockdep
    CONFIG_DEBUG_SPINLOCK=y
    CONFIG_KASAN=y

# What DEBUG_ATOMIC_SLEEP prints:

  BUG: sleeping function called from invalid context at mm/slab.h:494
  in_atomic(): 1, irqs_disabled(): 0, non_block: 0, pid: 431
  Call trace:
    dump_backtrace+0x0/0x1a0
    show_stack+0x18/0x24
    ___might_sleep+0x120/0x130
    __might_sleep+0x4c/0x88
    kmem_cache_alloc+0x40/0x2d0     <-- the allocation
    my_driver_handler+0x38/0xc0     <-- YOUR CODE
    ...

# Reads as: my_driver_handler called kmem_cache_alloc in a
# context where sleeping is illegal. Either the GFP flag is
# wrong, or the allocation is in the wrong place.

# FAULT INJECTION - test your error paths, which are otherwise
# never exercised:
    CONFIG_FAILSLAB=y
    CONFIG_FAIL_PAGE_ALLOC=y
    echo 10 > /sys/kernel/debug/failslab/probability
    echo 100 > /sys/kernel/debug/failslab/times
# Now 10% of allocations fail. Run your driver. Anything that
# crashes is an unchecked return value - and you will find
# some.
```

How it shows up

Symptom	Cause
`BUG: sleeping function called from invalid context`	`GFP_KERNEL` in atomic context
System deadlocks under memory pressure	Storage or filesystem driver allocating with `GFP_KERNEL` — needs `GFP_NOIO` / `NOFS`
Deadlock only during suspend/resume	Allocating on the frozen block path — use `memalloc_noio_save()`
Allocations fail in interrupts under load	`GFP_ATOMIC` reserves exhausted — preallocate
Events dropped during interrupt storms	Same; use a fixed pool and count overruns
Works with plenty of RAM, fails on a small board	Unchecked allocation return values — use fault injection to find them
Rare crash after an allocation failure	An error path that was never tested
Sluggish under memory pressure	`__GFP_NOFAIL` or heavy `GFP_KERNEL` use in a hot path
Allocation failure dump with order 0	Genuine exhaustion, not fragmentation

Check yourself

- ✓ Explain the deadlock GFP flags exist to prevent.
- ✓ Choose between `GFP_KERNEL`, `GFP_ATOMIC` and `GFP_NOWAIT` for six contexts.
- ✓ Explain why `GFP_ATOMIC` is less reliable than `GFP_KERNEL`, not more.
- ✓ Say when `GFP_NOIO` and `GFP_NOFS` are needed, and what `memalloc_noio_save()` adds.
- ✓ Rewrite an allocation under a spinlock so it is legal.
- ✓ Replace an interrupt-context allocation with a preallocated pool.
- ✓ Explain what a mempool guarantees and what breaks the guarantee.
- ✓ Read a `DEBUG_ATOMIC_SLEEP` backtrace and name the two possible fixes.
- ✓ Use fault injection to find an unchecked allocation in your own code.

The kernel address space

Where the kernel lives in virtual memory on arm64, which addresses can be translated to physical ones and which cannot, and why `virt_to_phys()` is right about half the pointers you will hand it.

The split

```
arm64 uses TTBR0 for user and TTBR1 for kernel (ARM ch. 12) -
so the split is architectural, not a software convention.

  With a 48-bit VA (CONFIG_ARM64_VA_BITS=48), the default:

  0x0000_0000_0000_0000  +--------------------------+
                         |  USER SPACE              |  TTBR0_EL1
                         |  256 TB                  |  per-process
  0x0000_ffff_ffff_ffff  +--------------------------+
                         |                          |
                         |   NON-CANONICAL          |  faults
                         |   (the "hole")           |
                         |                          |
  0xffff_0000_0000_0000  +--------------------------+  TTBR1_EL1
                         |  KERNEL SPACE            |  shared by
                         |  256 TB                  |  all tasks
  0xffff_ffff_ffff_ffff  +--------------------------+

  The top bits select the table, so the "hole" is addresses
  that are neither - and touching one is an immediate
  translation fault, which is a useful property: a corrupted
  pointer usually lands there rather than somewhere plausible.

  INSIDE THE KERNEL HALF (layout varies by version; this is
  the shape, check Documentation/arch/arm64/memory.rst for
  yours):

   +------------------------------+
   |  KASAN shadow                |  if enabled - a quarter
   |                              |  of the space
   +------------------------------+
   |  modules            128 MB   |  must be within branch
   |                              |  range of the kernel text
   +------------------------------+
   |  vmalloc / ioremap           |  large. vmalloc(), and
   |                              |  every ioremap() mapping
   +------------------------------+
   |  fixmap, PCI I/O             |
   +------------------------------+
   |  vmemmap                     |  the struct page array
   +------------------------------+
   |  LINEAR MAP                  |  <-- ALL of physical
   |  (the "direct map")          |      memory, mapped
   |                              |      contiguously
   +------------------------------+
```

The linear map is the key idea

```
/* Every byte of physical RAM appears at a fixed virtual
   address, in order, at a constant offset. So:            */

        virt = phys + PAGE_OFFSET_ADJUSTMENT

/* which makes translation ARITHMETIC, not a page-table walk: */

        phys_addr_t p = virt_to_phys(v);    /* subtract */
        void       *v = phys_to_virt(p);    /* add      */

        struct page *pg   = virt_to_page(v);
        void        *v2   = page_address(pg);
        unsigned long pfn = page_to_pfn(pg);

/* WHY IT EXISTS: the kernel constantly needs to convert
   between "a page I am tracking" and "a pointer I can
   dereference". Doing that with a page-table walk every time
   would be intolerable.

/* AND THE CRITICAL LIMITATION:

   These functions are ONLY valid for LINEAR MAP addresses -
   memory from kmalloc(), alloc_pages(), or the kernel image
   itself.

   THEY ARE WRONG, silently, for:
        vmalloc() addresses     - scattered physical pages
        ioremap() addresses     - device memory, not RAM
        user-space pointers     - a different address space
        stack addresses on a VMAP_STACK kernel - the kernel
                                 stack is vmalloc'd!

   "Silently wrong" is the problem. virt_to_phys() on a
   vmalloc address does not fault - it returns a number, which
   points at some unrelated physical memory. Hand that to a
   DMA engine and you have a device writing into whatever
   happens to live there.                                  */

/* THE CORRECT TOOLS for the non-linear cases: */
        vmalloc_to_page(addr)         /* vmalloc -> page     */
        virt_to_page(addr)            /* linear only         */
        dma_map_single(dev, ...)      /* the ONLY right way
                                         to get a device
                                         address (ch. 24)    */
```

FOUR KINDS OF POINTER, ONE C TYPE

A `void *` in the kernel may be a linear-map address, a vmalloc address, an `__iomem` cookie, or a `__user` address. They look identical to the compiler and behave completely differently. The annotations exist to make `sparse` tell them apart — which is the entire argument for running `make C=1` on your driver.

ioremap: reaching device registers

```
/* Device registers are NOT in the linear map. RAM is; MMIO is
   not. To touch a peripheral you must create a mapping.    */

        void __iomem *base;

        base = ioremap(0x1c000000, 0x1000);
        if (!base)
                return -ENOMEM;
        ...
        iounmap(base);

/* IN A DRIVER, do not do that. Use the managed, DT-driven
   form, which also RESERVES the region so two drivers cannot
   claim it:                                                */

        base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(base))
                return PTR_ERR(base);

/* WHAT ioremap GIVES YOU:
     · a mapping with Device memory attributes (ARM ch. 11):
       nGnRE - no gathering, no reordering, no early write
       acknowledgement. So accesses reach the device in
       program order and are not merged.
     · a __iomem pointer, which sparse will police

   THE VARIANTS:
     ioremap()        Device nGnRE. The default. Use this.
     ioremap_wc()     write-combining - for framebuffers,
                      where merging writes is a huge win
     ioremap_np()     non-posted, strictly ordered
     ioremap_cache()  Normal cacheable - for RAM-like regions
                      such as a firmware image in a carve-out

   NEVER dereference an __iomem pointer directly:
        val = *base;                    /* WRONG            */
        val = readl(base);              /* right (ch. 23)   */

   On arm64 a direct dereference often appears to work, which
   makes it worse - it compiles to a load with the wrong
   assumptions about reordering and merging, and fails on a
   different core or compiler version.                      */
```

The page tables themselves

```
/* arm64, 4 KB granule, 48-bit VA -> four levels (ARM ch. 13): */

    PGD -> PUD -> PMD -> PTE -> page
    (level 0)(1)   (2)    (3)

/* Kernel types:
        pgd_t, p4d_t, pud_t, pmd_t, pte_t
   and the walk, which you will see in mm/ code:            */

static int walk_one(struct mm_struct *mm, unsigned long addr)
{
        pgd_t *pgd;  p4d_t *p4d;  pud_t *pud;
        pmd_t *pmd;  pte_t *pte;

        pgd = pgd_offset(mm, addr);
        if (pgd_none(*pgd) || pgd_bad(*pgd))  return -EFAULT;

        p4d = p4d_offset(pgd, addr);
        if (p4d_none(*p4d) || p4d_bad(*p4d))  return -EFAULT;

        pud = pud_offset(p4d, addr);
        if (pud_none(*pud) || pud_bad(*pud))  return -EFAULT;
        if (pud_huge(*pud))  { /* 1 GB block */ }

        pmd = pmd_offset(pud, addr);
        if (pmd_none(*pmd) || pmd_bad(*pmd))  return -EFAULT;
        if (pmd_huge(*pmd))  { /* 2 MB block */ }

        pte = pte_offset_map(pmd, addr);
        ...
        pte_unmap(pte);
        return 0;
}

/* You will rarely write this. You WILL read it, in mm/, in
   the DMA code, and in any driver doing something unusual
   with mappings.

   BLOCK MAPPINGS matter for performance: the kernel maps its
   own linear map with 2 MB or 1 GB blocks where it can, which
   collapses four TLB entries into one (ARM ch. 15). That is
   also why CONFIG_RODATA_FULL_DEFAULT_ENABLED, which splits
   blocks to apply finer permissions, costs a measurable
   amount of performance.                                   */
```

Looking at it

```
# The full kernel page tables, level by level:
    CONFIG_PTDUMP_DEBUGFS=y
    sudo cat /sys/kernel/debug/kernel_page_tables

  ---[ Linear Mapping start ]---
  0xffff000000000000-0xffff000040000000  1G PMD  RW NX SHD AF ...
  0xffff000040000000-0xffff000040200000  2M PTE  ro x  SHD AF ...
                                                 ^^^^ kernel text
  ---[ vmalloc() Area ]---
  0xffff800000000000-0xffff800000001000  4K PTE  RW NX ...

# Read the permissions: kernel TEXT should be `ro x`, RODATA
# `ro NX`, everything else `RW NX`. Anything both writable and
# executable is a finding.

# Physical memory layout:
    cat /proc/iomem

# vmalloc allocations, with the caller:
    sudo cat /proc/vmallocinfo
    0xffff800008000000-0xffff800008002000 8192 my_probe+0x40/0x200 ...

# The kernel's own summary at boot:
    dmesg | grep -A20 "Virtual kernel memory layout"

# Where a symbol lives:
    sudo grep my_symbol /proc/kallsyms
```

> **`/proc/vmallocinfo` ATTRIBUTES EVERY MAPPING TO A CALLER**
>
> It shows the size and the function that allocated it, including `ioremap` mappings. If vmalloc space is filling up, or you suspect a driver is leaking mappings across bind/unbind cycles, this file names the culprit directly.

KASLR and pointer hiding

```
/* Two things that surprise people reading addresses: */

/* 1. KASLR randomises the kernel's base address each boot.
      So an address from one boot means nothing in another,
      and a crash dump must be decoded against that boot's
      offset. Disable it while debugging:                  */
        nokaslr        # on the command line

/* 2. %p HASHES pointers for unprivileged readers, to avoid
      leaking kernel layout: */
        pr_info("ptr=%p\n", p);
                -> ptr=00000000e2f4a1b7      /* hashed */

        /* Real values, for debugging only: */
        pr_info("ptr=%px\n", p);          /* raw - NEVER ship */
        pr_info("ptr=%pK\n", p);          /* hashed unless
                                             privileged */
        no_hash_pointers                  /* cmdline: make all
                                             %p raw. Prints a
                                             warning banner. */

/* If two %p values print identically, they are the same
   pointer - hashing is stable within a boot. That is often
   enough to debug without unhashing at all.               */
```

A note on highmem

```
/* On 32-bit systems, the linear map could only cover ~896 MB,
   so memory above that ("highmem") needed temporary mappings:

        void *v = kmap(page);           /* map it        */
        ... use v ...
        kunmap(page);

   You will see kmap/kmap_atomic/kmap_local_page all over
   older mm code and in generic drivers.

   ON ARM64 THIS DOES NOT APPLY. The 48-bit address space maps
   all physical memory linearly; there is no highmem.
   kmap_local_page() is still the portable idiom and is
   essentially free on arm64 - so generic code uses it and
   arm64-only code need not.

   MENTION IT ONLY because you will read the calls and wonder
   why they exist.                                          */
```

How it shows up

Symptom	Cause
DMA writes land in unrelated memory	`virt_to_phys()` on a vmalloc or stack address
`Unable to handle kernel paging request at 0x0000...`	NULL or small-offset pointer dereference
Fault at an address in the non-canonical hole	Corrupted pointer, or uninitialised memory used as one

Symptom	Cause
Reading a register returns garbage	Direct dereference of an `__iomem` pointer instead of `readl()`
`ioremap` returns NULL	vmalloc space exhausted, or the region overlaps an existing mapping
Two drivers fight over a peripheral	Used bare `ioremap` instead of the resource-requesting form
Framebuffer writes are very slow	Mapped Device instead of write-combining — use `ioremap_wc()`
Addresses differ every boot	KASLR — use `nokaslr` while debugging
`%p` prints meaningless values	Pointer hashing; use `%px` in a debug build only
vmalloc space exhausted over time	Leaked `ioremap` / `vmalloc` — check `/proc/vmallocinfo`
Writable and executable kernel mapping	A real hardening bug — check the ptdump output

Check yourself

- ✓ Explain the TTBR0/TTBR1 split and what the non-canonical hole gives you.
- ✓ Name the major regions of the kernel half.
- ✓ Explain the linear map and why `virt_to_phys()` can be arithmetic.
- ✓ List four kinds of address for which `virt_to_phys()` is silently wrong.
- ✓ Explain what `ioremap` does and why direct dereference is wrong.
- ✓ Choose between `ioremap`, `ioremap_wc` and `ioremap_cache`.
- ✓ Walk a four-level page table in kernel types.
- ✓ Read `/sys/kernel/debug/kernel_page_tables` and spot a bad permission.
- ✓ Use `/proc/vmallocinfo` to attribute a mapping to a driver.
- ✓ Explain `%p` hashing and how to debug around it safely.

Copying to and from user space

The kernel/user boundary is the one place where a hostile caller meets privileged code every single second. Every pointer that crosses it is an attack, and the rules for handling one are short, strict, and universally applicable.

Why you cannot just dereference

```
A user pointer is a number in a DIFFERENT address space, chosen
by an untrusted program. Dereferencing it directly is wrong for
four separate reasons:

  1. IT MAY BE INVALID. Unmapped, freed, or never valid. In
     user space that is a segfault for one process; in the
     kernel it is an oops, and possibly a panic with a lock
     still held.

  2. IT MAY POINT AT KERNEL MEMORY. A caller passing
     0xffff800008000000 to your ioctl is asking your privileged
     code to read or write kernel memory on their behalf. This
     is the confused deputy again - the same shape as the TEE
     problem in isolation chapter 12, at a different boundary.

  3. IT MAY NOT BE RESIDENT. The page may be swapped out or
     not yet faulted in, so touching it requires SLEEPING to
     fault it in - illegal in atomic context.

  4. THE MAPPING MAY CHANGE. Another thread in the same process
     can munmap or remap it while you work. Anything you check
     and then re-read can differ.

  So arm64 also enforces it in hardware: PAN (Privileged Access
  Never, ARMv8.1) faults on ANY EL1 access to a user address
  unless it is explicitly permitted. The accessor functions
  toggle it; your stray dereference does not.

  Which means the "just dereference it" bug does not silently
  work on modern hardware - it faults immediately. That is a
  kindness.
```

The accessors

```
#include <linux/uaccess.h>

/* BULK - the workhorses. Return the number of bytes NOT
   copied. Zero means success.                              */

unsigned long copy_to_user(void __user *to, const void *from,
                           unsigned long n);
unsigned long copy_from_user(void *to, const void __user *from,
                             unsigned long n);

        if (copy_to_user(ubuf, kbuf, len))
                return -EFAULT;         /* the ONLY idiom */

/* NOT `if (copy_to_user(...) != 0)` and definitely not
   `if (copy_to_user(...) < 0)` - the return is UNSIGNED, so a
   `< 0` test is always false and your error handling never
   runs.                                                    */

/* SCALARS - faster, type-checked: */
        get_user(x, uptr);              /* one value in  */
        put_user(x, uptr);              /* one value out */
        /* these return 0 or -EFAULT, so: */
        if (get_user(val, (int __user *)arg))
                return -EFAULT;

/* STRINGS: */
        strncpy_from_user(dst, src, count);   /* returns the
                                                 length, or
                                                 -EFAULT      */
        strnlen_user(src, max);
        /* or, better, let the kernel allocate: */
        char *s = strndup_user(uptr, PATH_MAX);
        if (IS_ERR(s))
                return PTR_ERR(s);
        ...
        kfree(s);

/* CLEARING: */
        clear_user(uptr, n);

/* CHECKING WITHOUT COPYING - rarely what you want: */
        if (!access_ok(uptr, size))
                return -EFAULT;
/* access_ok only checks the address RANGE is plausibly user
   space. It does NOT check the memory is mapped. Between the
   check and the copy anything can change. The copy functions
   do this check themselves, so calling access_ok separately is
   almost always redundant - and relying on it INSTEAD of a
   real copy is a bug.                                      */
```

THE FOUR RULES

1. **Never dereference a `__user` pointer.** Only the accessors touch it.
2. **Always check the return.** A partial copy is a failure; return `-EFAULT`.
3. **Only in process context.** These functions can sleep — never in an interrupt, a softirq, or while holding a spinlock.
4. **Copy once, then use only the copy.** Re-reading user memory after validating it is the TOCTOU bug, and it is exploitable.

The double-fetch bug

```
/* WRONG - reads the same user memory twice: */
static long my_ioctl(struct file *f, unsigned cmd, unsigned long arg)
{
        struct my_req __user *ureq = (void __user *)arg;
        u32 len;

        if (get_user(len, &ureq->len))          /* fetch 1 */
                return -EFAULT;
        if (len > MAX_LEN)
                return -EINVAL;                 /* validated */

        buf = kmalloc(len, GFP_KERNEL);

        /* fetch 2 - another thread changed ureq->len in
           between, and copy_from_user now copies MORE than
           we allocated                                    */
        if (copy_from_user(buf, ureq->data, ureq->len))
                return -EFAULT;
        ...
}

/* RIGHT - one copy, then work entirely from kernel memory: */
static long my_ioctl(struct file *f, unsigned cmd, unsigned long arg)
{
        struct my_req req;
        void *buf;

        if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
                return -EFAULT;         /* ONE fetch */

        if (req.len > MAX_LEN)
                return -EINVAL;         /* validate the COPY */

        buf = kmalloc(req.len, GFP_KERNEL);
        if (!buf)
                return -ENOMEM;

        if (copy_from_user(buf, u64_to_user_ptr(req.data),
                           req.len)) {  /* req.len is OURS now */
                kfree(buf);
                return -EFAULT;
        }
        ...
}

/* This is exactly the "copy in, then validate the copy" rule
   from the TEE chapters. Same boundary problem, same fix,
   different privilege levels.                             */
```

Leaking kernel memory outward

```
/* The other direction is just as dangerous. Copying a struct
   out copies its PADDING too - and padding contains whatever
   was previously on the kernel stack.                      */

struct my_info {
        u8  type;       /* offset 0 */
                        /* 3 BYTES OF PADDING */
        u32 value;      /* offset 4 */
        u16 flags;      /* offset 8 */
                        /* 6 BYTES OF PADDING */
        u64 timestamp;  /* offset 16 */
};                      /* sizeof = 24, 9 bytes uninitialised */

/* WRONG - leaks 9 bytes of stack: */
        struct my_info info;
        info.type = 1;
        info.value = x;
        info.flags = f;
        info.timestamp = t;
        if (copy_to_user(uptr, &info, sizeof(info)))
                return -EFAULT;

/* RIGHT - zero it first: */
        struct my_info info = { };        /* zeroes EVERYTHING,
                                             padding included */
        /* or: memset(&info, 0, sizeof(info)); */

/* THIS IS A REAL CVE CLASS. Kernel stack contents include
   return addresses (defeating KASLR), pointers, and fragments
   of other syscalls' data. Dozens of CVEs, year after year.

   MITIGATIONS to know:
     · `= { }` or memset on EVERY struct you copy out
     · design UAPI structs with NO PADDING: order fields
       largest-first, pad EXPLICITLY with reserved fields, and
       make the size a multiple of 8
     · CONFIG_INIT_STACK_ALL_ZERO=y - zero-initialises all
       stack variables. Cheap on modern compilers. Turn it on.
     · CONFIG_GCC_PLUGIN_STRUCTLEAK

   And the explicit-padding version of the struct above: */

struct my_info {
        __u64 timestamp;        /* 8-byte fields first */
        __u32 value;
        __u16 flags;
        __u8  type;
        __u8  reserved;         /* explicit, and DOCUMENTED as
                                   must-be-zero so it can be
                                   used later               */
};                              /* sizeof = 16, no holes */
```

32-bit callers on a 64-bit kernel

```
/* An arm64 kernel can run arm32 user space. Their struct
   layouts differ, so a struct that works natively silently
   mis-parses from a compat caller.                          */

/* THE PROBLEM: */
struct bad {
        int      n;
        void    *ptr;      /* 4 bytes on arm32, 8 on arm64 */
        long     val;      /* likewise */
};      /* different size AND different offsets */

/* THE FIX - use fixed-width types in ALL uapi structs, and
   __u64 for pointers: */
struct good {
        __u64 ptr;         /* always 8 bytes; convert with
                              u64_to_user_ptr()             */
        __u64 val;
        __u32 n;
        __u32 reserved;
};      /* identical on 32- and 64-bit */

        void __user *p = u64_to_user_ptr(req.ptr);

/* If you cannot change the layout, provide a compat handler: */
static const struct file_operations my_fops = {
        .unlocked_ioctl = my_ioctl,
        .compat_ioctl   = my_compat_ioctl,   /* or
                                compat_ptr_ioctl if the only
                                difference is pointer width */
};

/* compat_ptr_ioctl is the common case and handles it in one
   line - use it whenever your structs are already
   compat-clean.                                             */
```

> **DESIGN THE UAPI STRUCT AS IF IT WERE PERMANENT, BECAUSE IT IS**
>
> Fixed-width types, explicit reserved fields, no holes, no pointers-as-pointers, a size field or a version field. You get one chance: once a userspace program depends on the layout, "we do not break user space" means you are keeping it forever. Chapter 21 covers the interface design; this is the memory-layout half of the same decision.

Sharing instead of copying

```
/* For large or high-rate data, copying is the bottleneck.
   mmap() maps the buffer into user space so there is no copy
   at all.                                                    */

static int my_mmap(struct file *f, struct vm_area_struct *vma)
{
        struct my_dev *d = f->private_data;
        unsigned long size = vma->vm_end - vma->vm_start;

        /* 1. BOUNDS CHECK. The user chose the length. */
        if (size > d->buf_size)
                return -EINVAL;
        if (vma->vm_pgoff)              /* no offset support */
                return -EINVAL;

        /* 2. read-only if they should not write */
        if (vma->vm_flags & VM_WRITE)
                return -EPERM;

        /* 3. map it */
        return dma_mmap_coherent(d->dev, vma, d->cpu_addr,
                                 d->dma_addr, size);
        /* or for ordinary kernel memory:
             remap_pfn_range(vma, vma->vm_start,
                             virt_to_phys(d->buf) >> PAGE_SHIFT,
                             size, vma->vm_page_prot);        */
}

/* THE TRADE-OFF: no copy, but now user space can change the
   buffer AT ANY TIME, including while your driver or a DMA
   engine is reading it. Everything in the shared buffer is
   untrusted, permanently - the same discipline as TEE shared
   memory.

   So: use mmap for BULK DATA, and keep CONTROL - indices,
   lengths, commands - in ioctls where you can copy and
   validate. That split is what chapter 28's project is
   built around.                                              */
```

The boundary checklist

FOR EVERY IOCTL, READ, WRITE AND MMAP HANDLER

- Every user pointer is marked `__user`, and `sparse` is clean.
- Every copy's return value is checked, with `-EFAULT` on failure.
- Each field is fetched **once**; validation runs against the kernel copy.
- Sizes are validated *before* allocation, and the arithmetic cannot overflow.
- Structs copied out are zero-initialised.
- The UAPI layout is compat-clean: fixed-width types, no holes, no native pointers.
- Copies happen only in process context, never under a spinlock.
- Permissions are checked — `capable()`, file mode, or a device-node permission model.
- Error paths free everything they allocated.
- Shared (mmap'd) memory is treated as hostile on every read.

How it shows up

Symptom	Cause
Oops on a user address from kernel code	Direct dereference of a `__user` pointer — PAN caught it
Error handling never triggers	`if (copy_to_user(...) < 0)` — the return is unsigned
`BUG: sleeping function called from invalid context` in a copy	Copying under a spinlock or in an interrupt
Heap overflow reachable from an ioctl	Double fetch — the size changed between validation and use
Userspace sees stale kernel data in unused fields	Struct padding not zeroed — an information leak
Works for 64-bit programs, garbage for 32-bit	Missing `compat_ioctl`, or native types in the UAPI struct
`-EFAULT` from a valid-looking pointer	Address not mapped in that process, or the wrong process's address space
mmap succeeds, then the kernel crashes	Size not bounds-checked against the real buffer
Data corruption only under concurrent access	Trusting mmap'd memory that userspace is changing
Sparse warns "incorrect type in argument"	A `__user`/kernel pointer mix-up — a real bug, not noise

Check yourself

- ✓ Give four reasons a user pointer cannot be dereferenced, and say what PAN does.
- ✓ Write the correct idiom for `copy_to_user` and explain why `< 0` is wrong.
- ✓ Explain the double-fetch bug and rewrite a handler to avoid it.
- ✓ Explain how struct padding leaks kernel memory and give three mitigations.
- ✓ Design a UAPI struct that is compat-clean and hole-free.
- ✓ Say when `compat_ptr_ioctl` is sufficient.
- ✓ Write a bounds-checked `mmap` handler.
- ✓ Explain why mmap'd memory must be treated as hostile, and how to split control from data.

✓ Run the boundary checklist against a driver you have written.

Processes, threads and `task_struct`

Linux has no separate notion of a process and a thread. There is one structure, `task_struct`, and everything else is a question of what two tasks choose to share — which turns out to be a much cleaner model than the one most textbooks describe.

One structure for both

```
A "process" with three threads is really THREE task_structs
that share most of their pointers:

   task_struct (tid 100)      task_struct (tid 101)
   +------------------+       +------------------+
   | pid   = 100      |       | pid   = 100      |  <- SAME
   | tid   = 100      |       | tid   = 101      |  <- differs
   | mm    = ---------+---+---+ mm    =          |
   | files = ---------+-+ |   | files =          |
   | fs    = ...      | | |   | fs    =          |
   | signal= ...      | | |   | signal=          |
   | stack = 0xffff.. | | |   | stack = 0xffff.. |  <- OWN
   +------------------+ | |   +------------------+
                        | |
                        | +--> struct mm_struct   the address
                        |      (page tables, VMAs) space
                        |
                        +----> struct files_struct  the fd table

  WHAT IS ALWAYS PRIVATE:
     the kernel stack, the register state, the tid,
     the scheduler state, pending signals

  WHAT MAY BE SHARED, decided at clone() time:
     CLONE_VM       the address space   -> "threads"
     CLONE_FILES    the fd table
     CLONE_FS       cwd, root, umask
     CLONE_SIGHAND  signal handlers
     CLONE_THREAD   the thread group (same pid)
     CLONE_NEWNS/NEWPID/NEWNET/...  namespaces -> containers

  fork()   = clone(SIGCHLD)                    shares nothing
  pthread  = clone(CLONE_VM|CLONE_FILES|CLONE_FS|
                   CLONE_SIGHAND|CLONE_THREAD|...)
  container= clone(... | CLONE_NEWPID | CLONE_NEWNET | ...)

  So processes, threads and containers are three points on one
  continuum, not three mechanisms.
```

pid, tid, and the naming trap

```
/* The kernel's names are the OPPOSITE of what userspace calls
   them, and this confuses everyone once.                    */

   task->pid       the THREAD id.  Unique per task.
                   Userspace calls this the TID.
   task->tgid      the THREAD GROUP id. Shared by all threads
                   in a process. Userspace calls this the PID.

/* So: */
        getpid()  in userspace  returns  task->tgid
        gettid()  in userspace  returns  task->pid

/* In kernel code: */
        current->pid            /* this THREAD */
        current->tgid           /* the process */
        task_pid_nr(current)    /* pid in the INIT namespace */
        task_tgid_nr(current)
        task_pid_vnr(current)   /* pid as the task's own
                                   namespace sees it - what a
                                   containerised process
                                   believes its pid to be    */

/* WHICH TO USE when identifying a caller in a driver:
     for logging          -> current->pid and current->comm
     for ownership        -> the struct file, NOT the pid.
                             A pid can be reused after exit.
     across a namespace   -> think hard; a pid means nothing
                             outside the namespace that
                             created it                      */
```

current

```
#include <linux/sched.h>

        current             /* the task_struct running NOW */
        current->pid
        current->comm       /* the command name, 16 bytes */
        current->mm         /* NULL for a kernel thread! */
        current->flags

/* On arm64 `current` is SP_EL0, repurposed: the kernel keeps
   the current task_struct pointer there while running at EL1,
   so reading it is a single register read, not a lookup.
   (arch/arm64/include/asm/current.h)                        */

/* USEFUL IN A DRIVER: */
        dev_dbg(dev, "ioctl from %s[%d]\n",
                current->comm, current->pid);

/* AND THE TRAPS:

   1. `current` is only meaningful in PROCESS CONTEXT. In an
      interrupt handler it points at whatever task was
      unluckily interrupted - which has nothing to do with the
      device. Never attribute interrupt work to `current`.

   2. current->mm is NULL for KERNEL THREADS. Code that
      assumes an address space exists will crash when called
      from a kthread or a workqueue.

           if (!current->mm)
                   return -EINVAL;      /* kernel thread */

   3. Do not store `current` and use it later. The task can
      exit. If you need a reference, take one:
           get_task_struct(task);
           ...
           put_task_struct(task);                            */
```

Task states

State	`ps`	Meaning
`TASK_RUNNING`	R	Running, or on a runqueue ready to run
`TASK_INTERRUPTIBLE`	S	Sleeping; **signals wake it**
`TASK_UNINTERRUPTIBLE`	D	Sleeping; signals do **not** wake it
`TASK_KILLABLE`	D	Uninterruptible except by a fatal signal — usually the right choice
`__TASK_STOPPED`	T	Stopped by SIGSTOP
`EXIT_ZOMBIE`	Z	Exited, not yet reaped by its parent
`TASK_IDLE`	I	Uninterruptible but not counted in load average — for kthreads

THE D STATE IS A DRIVER'S FAULT MORE OFTEN THAN NOT

A process stuck in `D` cannot be killed, not even with `SIGKILL`, and it drives the load average up. It is almost always waiting on I/O in a driver that used `wait_event()` — the uninterruptible form — instead of `wait_event_interruptible()`. Use the interruptible or killable variants unless you can state exactly why a signal must not interrupt this wait; and if the hardware genuinely cannot be abandoned mid-operation, use a timeout so the wait is at least bounded.

Sleeping and waking, correctly

```
/* THE PATTERN every blocking driver uses. */

struct my_dev {
        wait_queue_head_t  wq;
        bool               data_ready;
        spinlock_t         lock;
};

        init_waitqueue_head(&d->wq);

/* --- the reader: wait for data --------------------------- */
static ssize_t my_read(struct file *f, char __user *buf,
                       size_t count, loff_t *ppos)
{
        struct my_dev *d = f->private_data;
        int ret;

        /* non-blocking mode must be honoured */
        if (f->f_flags & O_NONBLOCK) {
                if (!d->data_ready)
                        return -EAGAIN;
        } else {
                ret = wait_event_interruptible(d->wq,
                                               d->data_ready);
                if (ret)
                        return -ERESTARTSYS;   /* a signal:
                                                  let libc
                                                  restart it */
        }
        ...
}

/* --- the interrupt handler: wake it ---------------------- */
static irqreturn_t my_isr(int irq, void *dev_id)
{
        struct my_dev *d = dev_id;

        d->data_ready = true;
        wake_up_interruptible(&d->wq);
        return IRQ_HANDLED;
}

/* WHY wait_event_* IS A MACRO AND NOT A FUNCTION:
   it expands to the correct sleep sequence, which is subtle:

        for (;;) {
                prepare_to_wait(&wq, &entry, TASK_INTERRUPTIBLE);
                if (condition)         <-- re-check AFTER
                        break;             being queued
                if (signal_pending(current)) { ret = -ERESTARTSYS;
                                               break; }
                schedule();
        }
        finish_wait(&wq, &entry);

   The ORDER is what matters: queue yourself FIRST, then check
   the condition. Check first and a wakeup arriving in between
   is lost, and you sleep forever. This is the lost-wakeup
   race, and it is why you use the macro rather than writing
   it out.                                                  */
```

```
/* THE VARIANTS: */
        wait_event(wq, cond);                   /* D state  */
        wait_event_interruptible(wq, cond);     /* S state  */
        wait_event_killable(wq, cond);          /* SIGKILL  */
        wait_event_timeout(wq, cond, HZ);       /* bounded  */
        wait_event_interruptible_timeout(wq, cond, HZ);

/* AND THE RETURN VALUES DIFFER - a classic bug: */
        ret = wait_event_interruptible_timeout(wq, cond, HZ);
        if (ret == 0)   return -ETIMEDOUT;   /* timed out    */
        if (ret < 0)    return ret;          /* interrupted  */
        /* ret > 0: condition true, jiffies remaining        */
```

Kernel threads

```
#include <linux/kthread.h>

static int my_thread_fn(void *data)
{
        struct my_dev *d = data;

        while (!kthread_should_stop()) {
                wait_event_interruptible(d->wq,
                        d->work_pending || kthread_should_stop());
                if (kthread_should_stop())
                        break;
                do_the_work(d);
        }
        return 0;
}

        d->task = kthread_run(my_thread_fn, d, "my_worker/%d", id);
        if (IS_ERR(d->task))
                return PTR_ERR(d->task);
        ...
        kthread_stop(d->task);   /* sets the flag, wakes it,
                                    WAITS for it to return   */

/* PROPERTIES:
     · runs in process context - CAN SLEEP
     · current->mm is NULL - no user address space
     · appears in ps in [square brackets]
     · schedulable, so it can be niced and affined
     · kthread_should_stop() must be checked, AND the thread
       must be woken for it to notice - hence the condition
       above includes it

   WHEN TO USE ONE:
     you need a long-running, sleeping loop of your own.

   WHEN NOT TO - which is most of the time:
     for short deferred work, use a WORKQUEUE (chapter 13).
     A workqueue shares worker threads across the whole
     kernel; a dedicated kthread per device costs memory and
     scheduler entries. Reach for a kthread only when you
     genuinely need a private, long-lived, sleeping context -
     a polling loop, or a thread servicing a hardware queue.  */
```

Signals in the kernel

```
/* A driver rarely handles signals, but it must react to them
   correctly when interrupted.                                */

        if (signal_pending(current))
                return -ERESTARTSYS;

/* THE ERROR CODES, and the difference matters to userspace:

     -EINTR         "your syscall was interrupted". Userspace
                    sees EINTR and must retry itself.

     -ERESTARTSYS   "restart the syscall automatically, IF the
                    handler was installed with SA_RESTART".
                    Never reaches userspace as-is; the signal
                    machinery converts it to EINTR or restarts
                    the call. THIS IS USUALLY WHAT YOU WANT
                    from a wait that made no progress.

     -ERESTARTNOINTR  always restart, regardless of SA_RESTART

   THE RULE: if the operation made NO observable progress,
   return -ERESTARTSYS. If it partially completed - you copied
   some bytes - return the count you completed instead, and
   let the caller resume from there. Returning -ERESTARTSYS
   after partial progress causes the work to be done twice. */

        if (copied > 0)
                return copied;          /* partial success */
        if (signal_pending(current))
                return -ERESTARTSYS;    /* nothing done */
```

Looking at tasks

```
# From userspace:
    ps -eLf                 # -L shows THREADS, not just
                            # processes: LWP is the tid
    cat /proc/$PID/status   # Tgid, Pid, Threads, State, ...
    cat /proc/$PID/task/*/  # one directory per thread
    cat /proc/$PID/stack    # kernel backtrace - where is it
                            # blocked? THE tool for a D state
    cat /proc/$PID/wchan    # the function it is sleeping in
    cat /proc/$PID/maps     # the address space

# Finding what a D-state process is stuck on:
    ps -eo pid,stat,comm | awk '$2 ~ /D/'
    sudo cat /proc/1234/stack
      [<0>] my_driver_read+0x120/0x2a0     <-- there it is
      [<0>] vfs_read+0xb4/0x1e0
      ...

# Or the whole system's blocked tasks, from the kernel:
    echo w > /proc/sysrq-trigger
    dmesg                   # "Showing all locks held" and
                            # blocked task backtraces

# The hung task detector will do it for you automatically:
    CONFIG_DETECT_HUNG_TASK=y
    # prints a backtrace for any task in D state for 120s
```

How it shows up

Symptom	Cause
Process stuck in D, unkillable	Uninterruptible wait in a driver — check `/proc/PID/stack`
Load average high with idle CPUs	D-state tasks count toward it
Process sleeps forever despite the event happening	Lost wakeup — condition checked before queuing; use `wait_event_*`
Wakes up but the condition is false	Spurious wakeup — always re-check in a loop, which the macros do
`Ctrl-C` does not interrupt a read	Used `wait_event()` instead of the interruptible form
Data delivered twice after a signal	Returned `-ERESTARTSYS` after partial progress
Crash in an interrupt handler using `current->mm`	`current` is meaningless there
NULL deref on `current->mm`	Called from a kernel thread
`rmmod` hangs	`kthread_stop()` waiting on a thread that never checks `kthread_should_stop()`
PID-based ownership breaks	PIDs are reused; key off the `struct file`

Check yourself

- ✓ Explain how one structure represents processes, threads and containers.
- ✓ Say which of `pid` and `tgid` userspace calls "the PID".
- ✓ List three traps in using `current`.

- ✓ Name the task states and explain why D state is usually a driver bug.
- ✓ Write a correct blocking read with a wakeup from an interrupt.
- ✓ Explain the lost-wakeup race and why `wait_event` is a macro.
- ✓ Give the return-value convention for the timeout variants.
- ✓ Write a kernel thread with correct shutdown, and say when not to use one.
- ✓ Distinguish `-EINTR` from `-ERESTARTSYS`, and the partial-progress rule.
- ✓ Find where a D-state process is blocked, from userspace.

Scheduling in Linux

Five scheduling classes, one runqueue per CPU, and a set of latency properties that decide whether your embedded system meets its deadlines. This chapter is about what a driver author needs: which class to ask for, what preemption model you are running under, and where latency actually comes from.

The classes, in priority order

```
A task is picked from the HIGHEST non-empty class. Lower
classes only run when every higher one is idle.

  1. STOP          the stop-machine task. Not for you.
  2. DEADLINE      SCHED_DEADLINE - EDF with admission control
  3. REAL-TIME     SCHED_FIFO, SCHED_RR - fixed priority 1..99
  4. FAIR          SCHED_OTHER, SCHED_BATCH - CFS/EEVDF.
                   Where 99.9% of everything lives.
  5. IDLE          SCHED_IDLE - only when nothing else wants
                   the CPU

  CONSEQUENCE, and it is the one that bites: a runaway
  SCHED_FIFO task at priority 50 that never blocks will starve
  EVERY normal task on that CPU, forever. Including your shell.
  Including kernel threads that other things wait on.

  Linux ships a safety valve for exactly this:
     /proc/sys/kernel/sched_rt_runtime_us  = 950000
     /proc/sys/kernel/sched_rt_period_us   = 1000000
  RT tasks get at most 950 ms of every second, leaving 50 ms
  for everything else. Turn it off (-1) only on a system you
  have fully analysed.
```

The fair class

```
/* CFS (and EEVDF, which replaced it in 6.6) does not use
   priorities directly. It tracks how much VIRTUAL RUNTIME each
   task has consumed and runs the one that is furthest behind.

   NICE VALUES (-20 .. +19) scale the rate at which vruntime
   accumulates. Each step of 1 is about 1.25x, so:

        nice -20  ~ 88x the CPU share of nice +19
        nice   0  the default
        nice +19  minimal share

   NICE IS A RATIO, NOT A GUARANTEE. Two nice-0 tasks get 50%
   each; a nice-0 and a nice-5 get roughly 75/25. With one
   runnable task, nice is irrelevant - it gets everything.  */

/* THE KNOBS you will actually touch: */

  /sys/kernel/debug/sched/base_slice_ns
        the target time a task runs before being preempted.
        Smaller = lower latency, more context switches.

  /proc/sys/kernel/sched_child_runs_first
  /proc/sys/kernel/sched_autogroup_enabled
        desktop-oriented; usually off in embedded

/* AND THE ONE THAT MATTERS ON MOBILE SoCs: */

  SCHED_UTIL / schedutil governor + EAS (Energy Aware
  Scheduling). On a big.LITTLE system the scheduler uses a
  per-CPU energy model to decide WHERE to place a task, not
  just when. A task placed on a little core that then needs
  performance costs a migration plus a frequency ramp - which
  is why "the benchmark is slow for the first 200 ms" is a
  scheduling story, not a code story. (ARM ch. 33)         */
```

Real-time classes

```
#include <sched.h>      /* userspace */

        struct sched_param p = { .sched_priority = 50 };
        sched_setscheduler(0, SCHED_FIFO, &p);

/* SCHED_FIFO   runs until it blocks, yields, or is preempted
                by a HIGHER priority RT task. No time slice.
   SCHED_RR     same, but round-robins with equal-priority
                peers after a quantum
                (/proc/sys/kernel/sched_rr_timeslice_ms)

   Priorities 1 (lowest) to 99 (highest).

   IN THE KERNEL, for a thread you created: */
        sched_set_fifo(task);           /* a sane default RT */
        sched_set_fifo_low(task);       /* priority 1 */
        sched_set_normal(task, nice);

/* WHEN AN RT THREAD IS RIGHT:
     · a control loop with a hard period
     · servicing a hardware FIFO that overruns
     · audio, motor control, safety interlocks

   THE RULES for writing one:
     · it MUST block regularly. An RT task that spins is a
       system hang.
     · preallocate everything. No allocation in the loop.
     · mlockall() in userspace, so no page faults.
     · never take a lock also held by a non-RT task without
       priority inheritance (rt_mutex) - that is priority
       inversion, and it is how Mars Pathfinder famously
       failed.
     · pin it: sched_setaffinity, and isolate the CPU with
       isolcpus= or cpusets, so nothing else competes.     */

/* SCHED_DEADLINE - the honest one, when you can characterise
   the work: */
        struct sched_attr a = {
                .sched_policy   = SCHED_DEADLINE,
                .sched_runtime  =  1 * 1000 * 1000,  /* 1 ms  */
                .sched_deadline =  5 * 1000 * 1000,  /* 5 ms  */
                .sched_period   = 10 * 1000 * 1000,  /* 10 ms */
        };
        sched_setattr(0, &a, 0);
/* The kernel ADMITS it only if the system can still meet every
   existing deadline - it does the utilisation arithmetic for
   you and returns -EBUSY if not. That admission control is
   what makes it a real guarantee rather than a hope.       */
```

Preemption models

Model	Kernel code is	Typical worst-case latency	Use for
`PREEMPT_NONE`	Never preempted	Tens of ms	Throughput servers
`PREEMPT_VOLUNTARY`	Preempted at explicit points	Several ms	Desktops
`PREEMPT`	Preemptible except in critical sections	Hundreds of µs	**Most embedded**

Model	Kernel code is	Typical worst-case latency	Use for
`PREEMPT_RT`	Nearly fully preemptible	Tens of µs	Hard real-time

```
/* PREEMPT_RT, merged into mainline over 6.12, changes the
   rules your driver must obey:

     · spinlocks become SLEEPING locks (rt_mutex based). Code
       under a spin_lock() can be preempted.
     · interrupt handlers run in THREADS by default, so they
       can be prioritised and preempted.
     · raw_spinlock_t is the only truly non-preemptible lock,
       reserved for the scheduler and low-level arch code.
     · local_irq_disable() sections must be tiny.

   WHAT THIS MEANS FOR YOUR DRIVER:
     · a long spin_lock section that was merely rude on a
       normal kernel becomes a LATENCY BUG on RT
     · you must not assume spin_lock disables preemption
     · GFP_ATOMIC under a spinlock is now questionable, since
       the lock can sleep
     · code that must be atomic uses raw_spinlock_t - and
       every use of that needs justification

   TEST YOUR DRIVER ON PREEMPT_RT even if you do not ship it.
   It surfaces latency and locking assumptions that are
   invisible otherwise.                                    */
```

Where latency actually comes from

```
An interrupt fires. How long until your RT thread runs?

   1. INTERRUPTS DISABLED somewhere        <-- usually the
      the GIC cannot even deliver              biggest source
   2. hardware IRQ latency, GIC to CPU
   3. the top half runs
   4. it wakes your thread
   5. PREEMPTION DISABLED somewhere        <-- second biggest
      the scheduler cannot switch
   6. the scheduler picks your thread
   7. context switch, cache and TLB cold
   8. your code runs

  THE TWO DOMINANT TERMS are (1) and (5) - some other piece of
  code holding interrupts or preemption off. And they are
  caused by things you would not guess:

     · a long spinlock section in an unrelated driver
     · a printk to a slow serial console (milliseconds! and it
       happens with interrupts disabled)
     · TLB shootdowns and cache maintenance over a large range
     · a slow MMIO read in an interrupt handler
     · SMMU invalidation waiting for a sync completion
     · firmware: an SMC that takes a long time (isolation
       ch. 11), or SMI-equivalent secure work
     · CPU frequency changes and idle-state exit latency

  MEASURE, DO NOT GUESS:

     cyclictest -p 80 -t1 -n -i 200 -l 100000 -m
        the standard tool. Reports min/avg/MAX wakeup latency.
        The MAX is the only number that matters.

     ftrace's latency tracers name the culprit:
        echo preemptirqsoff > /sys/kernel/tracing/current_tracer
        echo 0 > /sys/kernel/tracing/tracing_max_latency
        ... run the load ...
        cat /sys/kernel/tracing/trace
     -> a full backtrace of the LONGEST section that had
        interrupts or preemption disabled. This tells you
        exactly which function to fix.
```

> **THE ONE THING TO REMEMBER ABOUT LATENCY**
>
> Your worst-case latency is not determined by your code. It is determined by the *worst* critical section anywhere in the kernel that runs on the same CPU — including drivers you did not write and did not enable deliberately. That is why real-time embedded systems isolate CPUs, strip the config down to what is needed, and measure with `cyclictest` under realistic load rather than reasoning about it.

Placing work on CPUs

```
# ISOLATE a CPU from the general scheduler:
    isolcpus=3 nohz_full=3 rcu_nocbs=3        # kernel cmdline

# isolcpus    no task is placed there unless explicitly affined
# nohz_full   no scheduler tick on that CPU when one task runs
# rcu_nocbs   RCU callbacks are offloaded elsewhere
# Together: CPU 3 becomes nearly quiet, and an RT thread pinned
# there sees dramatically better worst-case latency.

# Then pin your thread:
    taskset -c 3 ./my_control_loop
    # or in code: sched_setaffinity()

# IRQ affinity - route the device's interrupt to the same CPU
# (or deliberately away from it):
    cat /proc/interrupts
    echo 8 > /proc/irq/42/smp_affinity        # CPU 3 = bit 3
    # and stop irqbalance from undoing it:
    systemctl stop irqbalance

# In the kernel, for a threaded IRQ or kthread:
    irq_set_affinity_hint(irq, cpumask_of(3));
    kthread_bind(task, 3);

# WHY PIN AT ALL: migration costs a cold cache and a cold TLB
# (CA ch. 14), and on big.LITTLE it may cost a move to a
# slower core. For a periodic control loop the variance
# matters more than the average.
```

Observing the scheduler

```
# Per-task statistics:
    cat /proc/PID/sched
      se.sum_exec_runtime          : total CPU time
      se.statistics.wait_max       : LONGEST time waiting to
                                     run  <-- latency, per task
      nr_switches, nr_voluntary_switches
      nr_involuntary_switches      : preempted this often

# System-wide:
    cat /proc/schedstat
    cat /sys/kernel/debug/sched/debug     # runqueue contents

# Tracing - the real tool:
    trace-cmd record -e sched_switch -e sched_wakeup \
                     -e irq_handler_entry -e irq_handler_exit
    trace-cmd report
    # or kernelshark for the graphical view

# perf, for where time goes:
    perf sched record -- sleep 10
    perf sched latency --sort max
      # per-task: average and MAXIMUM scheduling delay
    perf sched timehist

# The single most useful one-liner for "why is my thread late":
    perf sched latency --sort max | head -20
```

What this means for a driver

```
/* You will rarely set a scheduling policy from a driver. What
   you WILL do is avoid making everyone else's latency worse: */

   · KEEP INTERRUPT HANDLERS SHORT. Acknowledge, grab the data,
     wake something, return. Everything else goes to a threaded
     handler or a workqueue (chapter 13).

   · KEEP SPINLOCK SECTIONS SHORT. Microseconds. Never do MMIO
     reads, memory allocation, or loops of unknown length under
     one.

   · NEVER busy-wait for hardware without a bound:
        /* WRONG */
        while (readl(base + STATUS) & BUSY)
                ;
        /* RIGHT */
        ret = readl_poll_timeout(base + STATUS, val,
                                 !(val & BUSY),
                                 10,        /* poll every 10us */
                                 1000);     /* give up at 1ms  */
        if (ret)
                return -ETIMEDOUT;
     /* and the _atomic variant if you cannot sleep:
        readl_poll_timeout_atomic()                          */

   · PREFER threaded IRQs, so the work is schedulable:
        devm_request_threaded_irq(dev, irq,
                                  my_hardirq,   /* quick */
                                  my_thread_fn, /* the work */
                                  IRQF_ONESHOT, name, d);

   · DO NOT printk in hot paths. A serial console printk can
     take MILLISECONDS with interrupts disabled. Use trace
     events or dev_dbg (compiled out unless enabled).      */
```

How it shows up

Symptom	Cause
System freezes, only the RT task runs	A `SCHED_FIFO` task that never blocks — RT throttling saves you, if enabled
Latency spikes under load	Some other critical section — find it with the `preemptirqsoff` tracer
Occasional multi-millisecond stall	A `printk` to a serial console, or a long TLB/cache maintenance
Latency fine on the bench, bad in the field	Real load exercises paths your test did not — `cyclictest` under load
An RT thread misses deadlines despite high priority	Priority inversion on a lock; or blocked on a non-RT kthread
Throughput drops when you raise priority	Preemption thrashing — more context switches, colder caches
First iterations slow, then fine	Frequency ramp and cache warming; big.LITTLE placement
`sched_setscheduler` returns `-EPERM`	Needs `CAP_SYS_NICE` or an `RLIMIT_RTPRIO` allowance
`SCHED_DEADLINE` returns `-EBUSY`	Admission control refused — the system cannot meet it
Driver works on `PREEMPT`, breaks on `PREEMPT_RT`	Assumed `spin_lock` disables preemption

Symptom	Cause
Interrupts land on the wrong CPU again	`irqbalance` is running

Check yourself

- ✓ Name the five scheduling classes in priority order.
- ✓ Explain what RT throttling protects against.
- ✓ Explain what a nice value actually controls, and when it does nothing.
- ✓ Distinguish `SCHED_FIFO`, `SCHED_RR` and `SCHED_DEADLINE`.
- ✓ List five rules for writing an RT thread.
- ✓ Name the four preemption models and what `PREEMPT_RT` changes for drivers.
- ✓ List the components of interrupt-to-thread latency and the two that dominate.
- ✓ Use `cyclictest` and the `preemptirqsoff` tracer to find a latency source.
- ✓ Isolate a CPU and pin both a thread and an IRQ to it.
- ✓ List five things a driver should do to avoid harming system latency.

Kernel concurrency: choosing a lock

The kernel gives you a dozen synchronisation primitives. Choosing correctly is nearly mechanical once you can answer two questions — *can this context sleep?* and *who else touches this data?* — and getting it wrong produces the hardest bugs you will ever chase.

The decision, as a flowchart

```
What am I protecting?
    |
    +-- a single integer or flag, no other invariant
    |      -> ATOMIC operations. No lock at all.
    |
    +-- data read constantly, written rarely, and readers
    |   can tolerate seeing the old version briefly
    |      -> RCU (chapter 12)
    |
    +-- anything else: I need a lock.
           |
           v
    Is the data ever touched from an INTERRUPT handler?
           |
    +------+------+
    |             |
   YES           NO
    |             |
    |             +-- Can every accessor SLEEP?
    |             |      (process context, no atomic ctx)
    |             |         |
    |             |    +----+----+
    |             |   YES       NO
    |             |    |         |
    |             |  MUTEX    SPINLOCK
    |             |    |
    |             |    +-- and if readers vastly outnumber
    |             |        writers: rw_semaphore
    |             |
    v
 spin_lock_irqsave() in process context
 spin_lock()        inside the handler itself

A MUTEX MAY NEVER BE TAKEN IN INTERRUPT CONTEXT. Not
"should not" - the machine will deadlock or oops.
```

Mutexes

```
#include <linux/mutex.h>

        DEFINE_MUTEX(my_mutex);            /* static */
        mutex_init(&d->lock);              /* dynamic */

        mutex_lock(&d->lock);
        ... critical section ...
        mutex_unlock(&d->lock);

        /* interruptible - the caller can be Ctrl-C'd while
           waiting. Use this in syscall paths.              */
        if (mutex_lock_interruptible(&d->lock))
                return -ERESTARTSYS;

        /* try, do not wait */
        if (!mutex_trylock(&d->lock))
                return -EBUSY;

/* PROPERTIES:
     · the waiter SLEEPS - so the CPU does other work
     · only in process context
     · you MAY sleep while holding one (kmalloc(GFP_KERNEL),
       copy_to_user, another mutex) - which is the main reason
       to prefer it
     · strictly one owner; must be released by the same task
       that took it
     · optimistically spins briefly if the owner is running on
       another CPU, so short holds are not as expensive as
       "it sleeps" suggests

   USE A MUTEX BY DEFAULT in process-context driver code. It is
   the right answer far more often than a spinlock, and code
   under it is far easier to write correctly.              */

/* THE SCOPED FORM, newer and worth adopting - it releases
   automatically on every exit path, including returns and
   gotos:                                                   */
        scoped_guard(mutex, &d->lock) {
                ...
        }
        /* or for a whole function: */
        guard(mutex)(&d->lock);
```

Spinlocks

```
#include <linux/spinlock.h>

        DEFINE_SPINLOCK(my_lock);
        spin_lock_init(&d->lock);

/* THE THREE FORMS, and choosing between them is the whole
   difficulty:                                              */

/* 1. Data touched ONLY in process context (but in an atomic
      section, so a mutex is not allowed):                  */
        spin_lock(&d->lock);
        ...
        spin_unlock(&d->lock);

/* 2. Data ALSO touched by a SOFTIRQ/tasklet: */
        spin_lock_bh(&d->lock);
        ...
        spin_unlock_bh(&d->lock);

/* 3. Data ALSO touched by a HARDWARE INTERRUPT handler: */
        unsigned long flags;

        spin_lock_irqsave(&d->lock, flags);
        ...
        spin_unlock_irqrestore(&d->lock, flags);

/* WHY THE _irqsave VARIANT EXISTS - the classic self-deadlock:

     process context:  spin_lock(&d->lock);
                       ... interrupt arrives on THIS CPU ...
     handler:          spin_lock(&d->lock);   /* spins    */
                       /* waiting for a lock held by the
                          code it interrupted, which cannot
                          continue until the handler returns */
                       DEADLOCK, this CPU, forever.

   Disabling local interrupts while holding the lock makes it
   impossible. Note it only needs to disable them LOCALLY -
   another CPU's handler spinning is fine, because the holder
   is still making progress.

/* INSIDE the interrupt handler itself, plain spin_lock() is
   correct - interrupts are already off for that line.      */

/* THE RULES while holding a spinlock:
     · MAY NOT SLEEP. Not kmalloc(GFP_KERNEL), not
       copy_to_user, not mutex_lock, not msleep.
     · keep it SHORT - microseconds. Every other CPU wanting
       it is burning cycles, and on PREEMPT_RT your section is
       a latency bound (chapter 10).
     · no unbounded loops, no MMIO polling.

   `flags` MUST be a local variable. Sharing one between
   CPUs is a subtle, devastating bug.                       */
```

> **THE RULE THAT PREVENTS MOST SPINLOCK BUGS**
>
> Ask: *what is the highest-priority context that touches this data?* Then use the form that protects against it, in **every** place the data is touched. Protecting a structure with `spin_lock_irqsave` in one function and plain `spin_lock` in another gives you no protection at all — and it will work perfectly until the interrupt happens to land at the wrong microsecond, in the field, months later.

Atomics: no lock at all

```
#include <linux/atomic.h>

        atomic_t counter = ATOMIC_INIT(0);

        atomic_inc(&counter);
        atomic_dec(&counter);
        atomic_add(5, &counter);
        val = atomic_read(&counter);
        atomic_set(&counter, 0);

        /* the ones that RETURN, and are the useful ones: */
        if (atomic_dec_and_test(&refcount))
                free_it();                  /* hit zero */
        old = atomic_cmpxchg(&v, expected, new);
        old = atomic_xchg(&v, new);
        if (atomic_add_unless(&v, 1, 0))    /* unless it is 0 */
                ...

        atomic64_t for 64-bit.

/* BIT OPERATIONS, also atomic: */
        set_bit(FLAG_BUSY, &d->flags);
        clear_bit(FLAG_BUSY, &d->flags);
        if (test_and_set_bit(FLAG_BUSY, &d->flags))
                return -EBUSY;              /* was already set */
        if (test_bit(FLAG_BUSY, &d->flags))
                ...
        /* __set_bit etc. are the NON-atomic versions - faster,
           and only safe under an existing lock                */

/* REFERENCE COUNTS - use the dedicated type, not atomic_t: */
        refcount_t ref;
        refcount_set(&ref, 1);
        refcount_inc(&ref);
        if (refcount_dec_and_test(&ref))
                free_it();
/* refcount_t detects overflow and use-after-free (increment
   from zero) and WARNs, where atomic_t silently wraps. That
   difference has closed real CVEs.                         */

/* THE LIMIT OF ATOMICS: they protect ONE variable. If your
   invariant spans two - "count matches the number of list
   entries" - atomics give you nothing. You need a lock.   */
```

Reader/writer locks

```
/* rw_semaphore - sleeping, many readers or one writer: */
        DECLARE_RWSEM(my_rwsem);

        down_read(&my_rwsem);   ... up_read(&my_rwsem);
        down_write(&my_rwsem);  ... up_write(&my_rwsem);

/* rwlock_t - the spinning version: */
        read_lock(&lock);   ... read_unlock(&lock);
        write_lock(&lock);  ... write_unlock(&lock);

/* USE THESE SPARINGLY. They are slower than a plain lock in
   the uncontended case, the cacheline still bounces between
   readers, and writers can starve.

   The kernel's general advice: if you think you want a
   reader/writer lock, you probably want RCU (chapter 12) or
   just a plain mutex. rwlock_t in particular is discouraged in
   new code and is being removed from many subsystems.

   THE EXCEPTION where rw_semaphore genuinely fits: readers
   that hold the lock for a LONG time (walking a large
   structure, doing I/O), where the parallelism is real.   */

/* seqlock - writers never block, readers retry: */
        unsigned seq;
        do {
                seq = read_seqbegin(&d->sl);
                a = d->value_a;
                b = d->value_b;
        } while (read_seqretry(&d->sl, seq));

        write_seqlock(&d->sl);
        d->value_a = x;  d->value_b = y;
        write_sequnlock(&d->sl);

/* Perfect for a few values updated by a writer that must not
   be delayed - timekeeping uses it. READERS MUST BE PURE:
   they may run several times and see torn intermediate state,
   so they must not have side effects or dereference pointers
   read inside the section without care.                    */
```

Deadlock, and how the kernel finds it for you

```
/* THE FOUR CLASSIC WAYS:

   1. AB-BA. Thread 1 takes A then B; thread 2 takes B then A.
   2. Self-deadlock. Taking a non-recursive lock twice.
   3. Interrupt inversion. The spin_lock case above.
   4. Sleeping while holding a spinlock, then something else
      needs that lock to make progress.

   THE FIX FOR (1) IS ALWAYS THE SAME: define a global LOCK
   ORDER and document it at the top of the file.            */

/*
 * Locking order:
 *      dev->config_mutex
 *        -> dev->queue_lock
 *          -> chan->lock
 * Never acquire in the reverse direction.
 */

/* AND THEN LET LOCKDEP CHECK IT: */
        CONFIG_PROVE_LOCKING=y
        CONFIG_DEBUG_ATOMIC_SLEEP=y
        CONFIG_DEBUG_SPINLOCK=y
        CONFIG_DEBUG_MUTEXES=y

/* Lockdep builds a graph of every lock ordering it has ever
   observed, and reports a POTENTIAL deadlock the first time it
   sees an inconsistent order - even if the deadlock did not
   actually happen. It also catches:
        · sleeping in atomic context
        · taking an IRQ-unsafe lock from IRQ context
        · a lock taken and never released
        · recursive acquisition

   The report looks like:

     ======================================================
     WARNING: possible circular locking dependency detected
     ------------------------------------------------------
     kworker/0:1/23 is trying to acquire lock:
      (&d->queue_lock){+.+.} at: my_queue_work+0x40
     but task is already holding lock:
      (&chan->lock){+.+.} at: my_chan_start+0x28
     which lock already depends on the new lock.

   It names both locks, both call sites, and the order it
   expected. It is the single most valuable debugging feature
   in the kernel for concurrency, and it finds bugs your
   testing never will. ENABLE IT IN DEVELOPMENT, ALWAYS.  */
```

Not needing a lock

```
/* The best lock is the one you removed. In order of
   preference: */

/* 1. DON'T SHARE. Per-CPU data needs no lock against other
      CPUs - only against interrupts on the same one:      */
        DEFINE_PER_CPU(struct my_stats, my_stats);

        struct my_stats *s = get_cpu_ptr(&my_stats);
        s->count++;
        put_cpu_ptr(&my_stats);         /* re-enables
                                           preemption      */
        /* or, if only ever from one context: */
        this_cpu_inc(my_stats.count);

/* 2. IMMUTABLE DATA needs no lock. Set it up in probe, never
      change it, and readers are free.

/* 3. OWNERSHIP TRANSFER. A single-producer, single-consumer
      ring needs no lock, only barriers - and kfifo does it
      for you:                                             */
        DECLARE_KFIFO(fifo, struct event, 128);
        kfifo_put(&fifo, ev);           /* producer */
        kfifo_get(&fifo, &ev);          /* consumer */

/* 4. A SMALLER CRITICAL SECTION. Compute outside, mutate
      inside:                                              */
        val = expensive_computation(input);     /* no lock */
        spin_lock(&d->lock);
        d->result = val;                        /* tiny    */
        spin_unlock(&d->lock);

/* 5. RCU, for read-mostly data (chapter 12).             */
```

The reference table

Primitive	Sleeps	IRQ context	Use for
`mutex`	Yes	No	**The default** in process context
`spinlock_t`	No	Yes	Short sections; data shared with interrupts
`atomic_t`	No	Yes	A single counter or flag
`refcount_t`	No	Yes	Object lifetimes
`rw_semaphore`	Yes	No	Long reads, rare writes
`seqlock_t`	No (readers)	Careful	A few values, writer must not block
`completion`	Yes	Signal from IRQ	"Wait until this finishes"
`semaphore`	Yes	No	Counting resources. Rare — usually a mutex is meant
RCU	No (readers)	Yes	Read-mostly pointers and lists
per-CPU	—	—	Statistics, caches

```
/* COMPLETION - the "wait for the hardware" primitive, and the
   one drivers use constantly: */

        struct completion done;
        init_completion(&done);

        /* start the operation, then: */
        if (!wait_for_completion_timeout(&done,
                                         msecs_to_jiffies(100)))
                return -ETIMEDOUT;

        /* in the interrupt handler: */
        complete(&done);

/* Always use the _timeout or _interruptible_timeout form.
   wait_for_completion() with no bound is how a driver hangs a
   process in D state forever when the hardware misbehaves
   (chapter 09).                                              */
```

How it shows up

Symptom	Cause
One CPU hangs, others fine	Spinlock self-deadlock — a handler took a lock the interrupted code holds
Whole system freezes	AB-BA deadlock, or a spinlock held while sleeping
`BUG: sleeping function called from invalid context`	Slept while holding a spinlock
Lockdep "possible circular locking dependency"	Inconsistent lock order — a real bug even if it never triggered
Rare corruption under load	Missing lock, or inconsistent variants of the same lock
Counter drifts from reality	Non-atomic read-modify-write
Works on one CPU, fails on SMP	It was always racy; single-CPU just hid it
Use-after-free on an object	Refcount race — use `refcount_t`, which would have warned
Soft lockup, "CPU stuck for 22s"	Spinning too long, or a spinlock never released
Latency spikes on PREEMPT_RT	Long spinlock sections that were merely rude before
Process in D state forever	Unbounded `wait_for_completion()`

Check yourself

- ✓ Walk the decision flowchart for four different data structures.
- ✓ Explain why a mutex may never be taken in interrupt context.
- ✓ Explain the spinlock self-deadlock and why `irqsave` only disables local interrupts.
- ✓ Say why using different spinlock variants on the same data gives no protection.
- ✓ Explain when atomics are enough and when they are not.
- ✓ Say why `refcount_t` is preferred over `atomic_t` for lifetimes.
- ✓ Explain what a seqlock requires of its readers.
- ✓ Name the four classic deadlocks and the universal fix for AB-BA.
- ✓ Enable lockdep and read one of its reports.

✓ Give five ways to avoid needing a lock at all.

Atomics, RCU and lockless techniques

RCU lets readers run with *zero* synchronisation overhead — no atomics, no cache-line bouncing, no waiting. The price is a discipline about how updates are published and when memory can be freed, and it rests entirely on the memory model from ARM chapters 20 and 21.

The idea

```
A pointer to a data structure, read constantly, updated rarely.

  WITH A LOCK: every reader takes and releases the lock. On an
  8-core system that is one cache line ping-ponging between
  all eight caches, thousands of times a second, purely to
  read data that almost never changes.

  WITH RCU: readers do NOTHING. No atomic, no write, no wait.
  The update instead works like this:

     1. the writer makes a NEW copy with the change
     2. it publishes the new one with a single pointer store
     3. new readers see the new version; readers already
        running keep using the old one - which is fine,
        because it is still a consistent, valid version
     4. the writer WAITS until every reader that could still
        hold the old pointer has finished
     5. only then does it free the old copy

  Read - Copy - Update. Step 4 is the whole trick, and it is
  cheap because of a clever definition of "finished".

     time ->
     reader A: [--- reads old ---]
     reader B:      [------ reads old ------]
     writer:     publish new
                       |
                       +-- grace period --+
                                          |
                                       free old
                                    (A and B are done;
                                     anyone starting after
                                     "publish" sees new)
```

Grace periods, and why they are cheap

```
/* THE KEY OBSERVATION: an RCU read-side critical section may
   not sleep or be preempted (in classic RCU). So if a CPU has
   passed through a CONTEXT SWITCH, it cannot possibly still be
   inside one.

   A "quiescent state" = context switch, idle, or userspace.
   A "grace period"    = every CPU has passed through at least
                         one quiescent state since the update.

   So RCU does not track individual readers at all. It waits
   for a system-wide condition that is already being reported
   for other reasons. That is why readers cost NOTHING - there
   is nothing to record.

   Typical grace period: a few milliseconds to tens of ms.
   RCU is therefore for data where UPDATES ARE RARE and the
   latency of reclaiming memory does not matter.           */
```

The API

```
#include <linux/rcupdate.h>

/* --- READER ------------------------------------------------ */
        rcu_read_lock();                /* NOT a lock. It just
                                           marks the section
                                           and disables
                                           preemption.       */
        p = rcu_dereference(gp);        /* the ONLY correct
                                           way to read the
                                           pointer           */
        if (p)
                use(p->field);          /* safe until
                                           rcu_read_unlock  */
        rcu_read_unlock();

/* RULES for the read side:
     · MAY NOT SLEEP inside (unless using SRCU)
     · the pointer is only valid INSIDE the section. Do not
       stash it in a global and use it later.
     · rcu_dereference() carries the dependency barrier that
       makes the ARM memory model safe here - a plain read is
       NOT equivalent (see below).

/* --- WRITER ------------------------------------------------ */
        struct foo *new, *old;

        new = kmalloc(sizeof(*new), GFP_KERNEL);
        *new = *old_contents;
        new->field = updated_value;      /* fully initialise
                                            BEFORE publishing */

        spin_lock(&update_lock);         /* writers still need
                                            mutual exclusion
                                            with EACH OTHER   */
        old = rcu_dereference_protected(gp,
                        lockdep_is_held(&update_lock));
        rcu_assign_pointer(gp, new);     /* PUBLISH */
        spin_unlock(&update_lock);

        synchronize_rcu();               /* wait for the grace
                                            period - SLEEPS  */
        kfree(old);

/* OR, without blocking: */
        call_rcu(&old->rcu_head, my_free_cb);
        /* the callback runs after the grace period */

static void my_free_cb(struct rcu_head *h)
{
        struct foo *p = container_of(h, struct foo, rcu_head);
        kfree(p);
}
/* For a plain kfree there is a shortcut: */
        kfree_rcu(old, rcu_head);
```

WHY `RCU_ASSIGN_POINTER` AND `RCU_DEREFERENCE` ARE NOT OPTIONAL

They are the memory barriers that make this correct on a weakly-ordered machine like ARM (ARM chapters 20 and 21). `rcu_assign_pointer` is a release store: it guarantees the initialisation of the new object is visible *before* the pointer to it. `rcu_dereference` preserves the address dependency so the reader cannot observe the new pointer with stale contents. Write `gp = new;` and `p = gp;` instead and it will work on x86 and fail on ARM, rarely, under load — the worst possible failure mode.

RCU-protected lists

```
/* The common real use: a list of devices, sessions, or rules,
   read on every operation, changed on hotplug.             */

        struct my_entry {
                struct list_head list;
                struct rcu_head  rcu;
                int              id;
                ...
        };

        LIST_HEAD(my_list);
        DEFINE_SPINLOCK(my_list_lock);   /* writers only */

/* --- READER: traverse without any lock ------------------- */
static struct my_entry *find_entry(int id)
{
        struct my_entry *e;

        list_for_each_entry_rcu(e, &my_list, list) {
                if (e->id == id)
                        return e;
        }
        return NULL;
}
        /* called as: */
        rcu_read_lock();
        e = find_entry(id);
        if (e)
                do_something(e->value);
        rcu_read_unlock();

/* --- WRITER: add ----------------------------------------- */
        spin_lock(&my_list_lock);
        list_add_rcu(&new->list, &my_list);
        spin_unlock(&my_list_lock);
        /* no grace period needed to ADD - a reader either
           sees it or does not, and both are correct       */

/* --- WRITER: remove -------------------------------------- */
        spin_lock(&my_list_lock);
        list_del_rcu(&e->list);
        spin_unlock(&my_list_lock);

        synchronize_rcu();       /* now no reader holds e */
        kfree(e);
        /* or: kfree_rcu(e, rcu); */

/* NOTE list_del_rcu leaves the FORWARD pointer intact, so a
   reader already traversing the deleted node can still walk
   onward to the rest of the list. That is deliberate, and it
   is why you must use the _rcu variants rather than the
   ordinary list functions.                                 */
```

The flavours

Flavour	Readers may sleep	Use
`rcu_read_lock()`	No	The default. Preemption disabled in the section.

Flavour	Readers may sleep	Use
`rcu_read_lock_bh()`	No	Data also touched from softirqs (networking).
`rcu_read_lock_sched()`	No	Preempt-disable regions.
`srcu_read_lock()`	**Yes**	Sleepable RCU — readers may block. Per-domain, so one slow reader does not stall everyone.
RCU Tasks	—	For trampolines and tracing; waits for voluntary context switches.

```
/* SRCU, when a reader must sleep - e.g. it does I/O or copies
   to userspace: */

        struct srcu_struct my_srcu;
        init_srcu_struct(&my_srcu);

        int idx = srcu_read_lock(&my_srcu);
        p = srcu_dereference(gp, &my_srcu);
        ... may sleep here ...
        srcu_read_unlock(&my_srcu, idx);

        synchronize_srcu(&my_srcu);     /* the writer side */

/* SRCU grace periods can be MUCH longer, because a sleeping
   reader may sleep for a long time. Do not use it where you
   need memory back promptly.                              */
```

Barriers, briefly

```
/* ARM is weakly ordered (ARM ch. 20-21). Without barriers,
   both the compiler and the CPU may reorder your accesses. */

        smp_mb()        full barrier
        smp_rmb()       reads before / reads after
        smp_wmb()       writes before / writes after
        smp_store_release(&p, v)    /* release store */
        v = smp_load_acquire(&p)    /* acquire load  */
        barrier()       compiler only, no CPU instruction

/* THE CANONICAL PAIR - publish a structure safely: */
        /* producer */
        obj->data = 42;
        smp_store_release(&shared_ptr, obj);

        /* consumer */
        p = smp_load_acquire(&shared_ptr);
        if (p)
                use(p->data);          /* guaranteed to be 42 */

/* WITHOUT the release/acquire pair, the consumer can observe
   shared_ptr as non-NULL while p->data is still stale. On
   arm64 this is not theoretical - it happens.

/* AND FOR DEVICE MEMORY, a DIFFERENT set (chapter 23):
        wmb()/rmb()/mb()       full barriers including devices
        dma_wmb()/dma_rmb()    ordering against a DMA engine
        writel()/readl()       already carry the ordering you
                               need for MMIO in most cases

   Do not use smp_* barriers for DMA ordering: the smp_
   variants compile to nothing on a UP kernel, and a DMA
   engine is a second agent regardless of how many CPUs you
   have. That substitution is a real and subtle bug.       */
```

> **THE HONEST ADVICE ABOUT LOCKLESS CODE**
>
> Use RCU where the pattern fits — a read-mostly pointer or list — because it is a well-tested framework with lockdep support and clear rules. **Do not invent your own lockless algorithms.** Hand-rolled lock-free code on a weakly-ordered architecture is beyond what code review reliably catches, and the bugs appear as one-in-a-billion corruption under load. If a mutex is fast enough, use the mutex.

Debugging RCU

```
# Enable these while developing anything using RCU:
    CONFIG_PROVE_RCU=y              # lockdep for RCU
    CONFIG_DEBUG_OBJECTS_RCU_HEAD=y # double call_rcu, etc.
    CONFIG_RCU_EQS_DEBUG=y

# PROVE_RCU catches the big one - dereferencing outside a
# read-side section:

  WARNING: suspicious RCU usage
  drivers/mydrv/mydrv.c:142 suspicious rcu_dereference_check()
  usage!
  other info that might help us debug this:
  rcu_scheduler_active = 2, debug_locks = 1
  no locks held by mytask/431.

# "no locks held" = you called rcu_dereference() without
# rcu_read_lock(). A real bug, caught statically at runtime.

# If you legitimately hold a lock instead, say so:
    p = rcu_dereference_protected(gp, lockdep_is_held(&my_lock));

# RCU STALLS - a reader that never finishes:
  INFO: rcu_sched self-detected stall on CPU
# Means a CPU spent tens of seconds without a quiescent state.
# Causes: an infinite loop with preemption disabled, a very
# long rcu_read_lock section, or a spinlock deadlock.

# Watch grace periods and callbacks:
    cat /sys/kernel/debug/rcu/rcu_preempt/rcugp
    cat /proc/sched_debug | grep -i rcu
```

When RCU is the answer

Situation	RCU?
A list of registered devices, read on every I/O	**Yes** — the textbook case
A configuration structure, changed via sysfs	**Yes**
A routing or filter table	**Yes**
A counter	No — use `atomic_t` or per-CPU
A queue with frequent adds and removes	No — grace periods will dominate
Data written as often as it is read	No — use a mutex
Memory that must be freed promptly	No — grace periods delay reclamation
Readers that need a consistent view across two structures	No — RCU gives per-pointer consistency only

How it shows up

Symptom	Cause
Use-after-free in a reader	Freed without a grace period, or the pointer used after `rcu_read_unlock`
Reader sees a partially initialised object	Plain store instead of `rcu_assign_pointer`

Symptom	Cause
Works on x86, corrupts on arm64	Missing barriers — the memory model is weaker
`suspicious rcu_dereference_check() usage`	Dereference outside a read-side section
`BUG: sleeping function called from invalid context` inside RCU	Slept in an RCU read section — use SRCU
`rcu_sched self-detected stall`	A CPU not reaching a quiescent state — a loop or a deadlock
Memory usage climbs under a rapid update rate	Callbacks queued faster than grace periods complete
Update latency of tens of milliseconds	`synchronize_rcu()` in a hot path — use `call_rcu` / `kfree_rcu`
Rare corruption with hand-rolled lockless code	Exactly why not to write it

Check yourself

- ✓ Explain the read-copy-update cycle and why readers pay nothing.
- ✓ Define a quiescent state and a grace period, and say why they are cheap to detect.
- ✓ Write a correct RCU reader and writer.
- ✓ Explain what `rcu_assign_pointer` and `rcu_dereference` guarantee on ARM.
- ✓ Convert a spinlock-protected list to RCU.
- ✓ Say when SRCU is needed and what it costs.
- ✓ Pair a release store with an acquire load and say what it guarantees.
- ✓ Explain why `smp_wmb()` is wrong for DMA ordering.
- ✓ Enable `PROVE_RCU` and interpret a warning.
- ✓ Decide, for eight situations, whether RCU is the right tool.

Deferred work: softirqs, tasklets and workqueues

An interrupt handler must be short. Everything it cannot do goes somewhere else — and Linux offers four somewhere-elses, differing in whether they can sleep, how soon they run, and how much they cost. Choosing is easy once you know the trade.

The top half / bottom half split

```
   device raises IRQ
        |
        v
   TOP HALF (hard IRQ handler)
        · interrupts may be disabled
        · MUST BE FAST - microseconds
        · CANNOT SLEEP
        · do only: acknowledge the device, stop it
          re-interrupting, grab the minimum state
        |
        +--> schedule the bottom half
        |
        v
   return IRQ_HANDLED
        ...
        |
        v
   BOTTOM HALF, later
        · the actual work: parse the data, wake readers,
          submit the next transfer, log the error
        · may or may not sleep, depending which kind

  WHY: while your handler runs, that interrupt is masked, and
  on many systems others are too. Every microsecond spent in a
  top half is added directly to the worst-case interrupt
  latency of the WHOLE SYSTEM (chapter 10).
```

The four mechanisms

	Can sleep	Runs on	Latency	Verdict
Softirq	No	The CPU that raised it	Very low	Core subsystems only — you cannot add one
Tasklet	No	The CPU that scheduled it	Low	**Deprecated.** Do not use in new code
Workqueue	**Yes**	Any CPU, a kworker thread	Scheduler-dependent	**The default choice**
Threaded IRQ	**Yes**	A dedicated thread per IRQ	Low, and prioritisable	**Best for device work**

THE SHORT ANSWER

For work arising from an interrupt on your device: use a **threaded IRQ**. For work arising from anywhere else — a timer, a syscall path that must not block, a retry — use a **workqueue**. You will almost never need anything else, and tasklets are actively being removed from the kernel.

Threaded interrupts

```
/* The modern pattern, and the one to reach for first. */

static irqreturn_t my_hardirq(int irq, void *dev_id)
{
        struct my_dev *d = dev_id;
        u32 status;

        status = readl(d->base + REG_STATUS);
        if (!(status & INT_PENDING))
                return IRQ_NONE;        /* not ours - important
                                           on a shared line   */

        /* acknowledge and mask, so it stops asserting */
        writel(status, d->base + REG_STATUS);

        d->pending_status = status;     /* hand it over */

        return IRQ_WAKE_THREAD;         /* run the thread fn */
}

static irqreturn_t my_thread_fn(int irq, void *dev_id)
{
        struct my_dev *d = dev_id;

        /* PROCESS CONTEXT. We may sleep here: */
        mutex_lock(&d->lock);
        process_data(d, d->pending_status);   /* can be slow  */
        mutex_unlock(&d->lock);

        wake_up_interruptible(&d->wq);

        return IRQ_HANDLED;
}

        ret = devm_request_threaded_irq(&pdev->dev, irq,
                                        my_hardirq,
                                        my_thread_fn,
                                        IRQF_ONESHOT,
                                        dev_name(&pdev->dev),
                                        d);

/* IRQF_ONESHOT keeps the interrupt MASKED until the thread
   function finishes. Essential for level-triggered interrupts
   and for slow buses - without it a device that keeps
   asserting will re-interrupt immediately and livelock the
   CPU.

/* THE DEGENERATE CASE, and it is very common - pass NULL for
   the hard handler and the core provides one that just wakes
   the thread: */
        devm_request_threaded_irq(dev, irq, NULL, my_thread_fn,
                                  IRQF_ONESHOT, name, d);
/* This is exactly right for I2C and SPI devices, where you
   cannot even READ the status register without sleeping.  */

/* WHY THREADED IRQs ARE BETTER THAN A WORKQUEUE HERE:
     · one thread per interrupt, so no sharing with unrelated
       work
     · it can be given a real-time priority and pinned
     · on PREEMPT_RT this is the default for ALL interrupts,
```

```
        so your driver already matches the model
      · the masking is handled for you                      */
```

Workqueues

```
#include <linux/workqueue.h>

/* --- simple: use the shared system queue ----------------- */
struct my_dev {
        struct work_struct  work;
        ...
};

static void my_work_fn(struct work_struct *w)
{
        struct my_dev *d = container_of(w, struct my_dev, work);

        /* process context - CAN SLEEP */
        mutex_lock(&d->lock);
        do_slow_thing(d);
        mutex_unlock(&d->lock);
}

        INIT_WORK(&d->work, my_work_fn);
        schedule_work(&d->work);        /* queue it */

/* --- delayed work ---------------------------------------- */
        struct delayed_work poll_work;

        INIT_DELAYED_WORK(&d->poll_work, poll_fn);
        schedule_delayed_work(&d->poll_work,
                              msecs_to_jiffies(100));
        /* and to re-arm from inside poll_fn, for a poller: */
        schedule_delayed_work(&d->poll_work, HZ);

/* --- your own workqueue, when you need control ----------- */
        d->wq = alloc_workqueue("my_dev_wq",
                                WQ_UNBOUND | WQ_MEM_RECLAIM,
                                1);           /* max_active */
        if (!d->wq)
                return -ENOMEM;

        queue_work(d->wq, &d->work);
        ...
        destroy_workqueue(d->wq);    /* drains first */

/* THE FLAGS THAT MATTER:
     WQ_MEM_RECLAIM   guarantees a rescuer thread, so the queue
                      can make progress under memory pressure.
                      REQUIRED if your work is on a memory
                      reclaim path (a storage or swap driver).
     WQ_UNBOUND       not tied to a CPU; the scheduler places
                      it. Better for long-running work.
     WQ_HIGHPRI       runs ahead of normal work
     WQ_FREEZABLE     stops during suspend
     max_active = 1   serialises - only one work item at a
                      time, which often replaces a lock

/* CANCELLING - and this is where drivers leak or crash: */
        cancel_work_sync(&d->work);           /* WAITS */
        cancel_delayed_work_sync(&d->poll_work);
        flush_workqueue(d->wq);

/* In remove(), you MUST use the _sync forms. cancel_work()
   without _sync returns immediately, the work may still be
```

```
   running, and it will touch your freed device structure
   moments later.                                          */
```

THE TEARDOWN ORDERING BUG

```
static void my_remove(struct platform_device *pdev)
{
        struct my_dev *d = platform_get_drvdata(pdev);

        /* 1. STOP THE SOURCE first - no new work can arrive */
        free_irq(d->irq, d);            /* or devm handles it */
        writel(0, d->base + REG_INT_ENABLE);

        /* 2. THEN cancel what is already queued, and WAIT   */
        cancel_delayed_work_sync(&d->poll_work);
        cancel_work_sync(&d->work);
        del_timer_sync(&d->timer);

        /* 3. ONLY NOW is it safe to free anything          */
}
```

Reverse steps 1 and 2 and the interrupt re-queues the work after you cancelled it. Skip the `_sync` and the work runs against freed memory. Both produce crashes minutes later, in unrelated code, and both are found instantly by KASAN.

Timers

```
/* --- timer_list: jiffy-resolution, runs in SOFTIRQ ------- */
#include <linux/timer.h>

static void my_timer_fn(struct timer_list *t)
{
        struct my_dev *d = from_timer(d, t, timer);
        /* SOFTIRQ CONTEXT - cannot sleep */
        schedule_work(&d->work);        /* punt if you must */
}

        timer_setup(&d->timer, my_timer_fn, 0);
        mod_timer(&d->timer, jiffies + msecs_to_jiffies(500));
        ...
        del_timer_sync(&d->timer);      /* _sync, always */

/* Resolution is one jiffy = 1/CONFIG_HZ, typically 1-10 ms,
   and timers fire LATE, never early. Fine for timeouts and
   watchdogs; useless for precise periods.                 */

/* --- hrtimer: nanosecond resolution ---------------------- */
#include <linux/hrtimer.h>

static enum hrtimer_restart my_hr_fn(struct hrtimer *t)
{
        struct my_dev *d = container_of(t, struct my_dev, hrt);

        /* HARD IRQ context by default - be quick */
        do_sample(d);

        hrtimer_forward_now(t, ns_to_ktime(1000000));  /* 1 ms */
        return HRTIMER_RESTART;
}

        hrtimer_init(&d->hrt, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
        d->hrt.function = my_hr_fn;
        hrtimer_start(&d->hrt, ns_to_ktime(1000000),
                      HRTIMER_MODE_REL);
        ...
        hrtimer_cancel(&d->hrt);

/* Use hrtimer_forward_now() to re-arm a PERIODIC timer -
   it advances from the scheduled time, so the period does not
   drift with handler latency. Re-arming from "now" instead
   accumulates error, which is the classic sampling-rate
   bug.                                                    */

/* --- short delays, no timer at all ----------------------- */
        udelay(10);          /* BUSY-WAITS. atomic-safe.
                                Under ~10us only.           */
        ndelay(100);
        mdelay(5);           /* busy-waits MILLISECONDS -
                                almost always wrong          */

        usleep_range(100, 200);   /* SLEEPS. process context.
                                     Give a range so the
                                     kernel can coalesce it.  */
        msleep(20);              /* sleeps, coarse           */

/* NEVER mdelay() in a driver unless a datasheet demands a
```

```
   busy-wait. It burns a CPU for the whole duration with
   preemption implications for everything else.           */
```

Tasklets, and why not

```
/* You will see these in existing drivers: */
        DECLARE_TASKLET(my_tasklet, my_tasklet_fn);
        tasklet_schedule(&my_tasklet);

/* THEY ARE DEPRECATED. Reasons:
     · they run in softirq context, so they cannot sleep -
       yet they are used for work that wants to
     · a tasklet cannot be preempted, so a long one is a
       latency bug that nothing reports
     · the same tasklet never runs concurrently on two CPUs,
       which sounds helpful but hides races that appear the
       moment someone converts it
     · they are being actively removed from the tree

   IF YOU ARE MAINTAINING ONE, convert it:
        tasklet doing work that can sleep -> threaded IRQ
        tasklet doing quick work          -> the hard IRQ
                                             handler itself
        tasklet used for deferral         -> workqueue      */
```

Choosing

```
   Work arises from MY DEVICE'S INTERRUPT?
        |
        +-- and it can sleep, or takes more than ~10 us
        |      -> THREADED IRQ
        |
        +-- and it is a few register writes
               -> do it in the hard handler. Deferring it
                  costs more than doing it.

   Work arises from ELSEWHERE?
        |
        +-- must run periodically, precise timing
        |      -> hrtimer (+ workqueue if it must sleep)
        |
        +-- must run periodically, approximate
        |      -> delayed_work, re-armed
        |
        +-- must run once, soon, and may sleep
        |      -> workqueue
        |
        +-- must run once, after a delay
               -> delayed_work, or timer_list for a timeout

   And on a memory-reclaim path (storage, swap):
        -> a dedicated workqueue with WQ_MEM_RECLAIM
```

Observing

```
# Softirq activity:
    cat /proc/softirqs
    # HI, TIMER, NET_TX, NET_RX, BLOCK, TASKLET, SCHED,
    # HRTIMER, RCU - per CPU

# ksoftirqd busy = softirq load so high it was pushed to a
# thread. On an embedded system that usually means network or
# timer pressure:
    top -H | grep ksoftirq

# Workqueue state:
    ls /sys/bus/workqueue/devices/
    cat /sys/bus/workqueue/devices/my_dev_wq/max_active
    # and with CONFIG_WQ_WATCH:
    dmesg | grep "workqueue.*hogged CPU"

# Tracing - the useful one:
    trace-cmd record -e workqueue -e irq -e timer
    trace-cmd report
    # shows workqueue_queue_work / execute_start / execute_end,
    # so you can measure how long each item took and how long
    # it waited

# Stuck work is reported automatically:
    BUG: workqueue lockup - pool cpus=0 flags=0x0 nice=0
         stuck for 32s!
# Means a work item blocked forever, starving everything else
# on that queue. Common cause: work item A waiting on work
# item B, both on the same single-threaded queue.
```

How it shows up

Symptom	Cause
Crash after `rmmod` or unbind	Work, timer or IRQ not cancelled with a `_sync` variant
Crash in work function touching freed memory	Cancelled the work before stopping the interrupt that re-queues it
Interrupt storm, system livelocks	Level-triggered IRQ not masked — missing `IRQF_ONESHOT`
`BUG: scheduling while atomic` in a timer	`timer_list` callbacks run in softirq — cannot sleep
`workqueue lockup ... stuck for 32s`	A work item blocked, or two items on one queue waiting on each other
Work runs much later than expected	Shared system queue behind other work — use your own
Deadlock under memory pressure	Reclaim-path work without `WQ_MEM_RECLAIM`
Sampling period drifts	Re-armed from "now" instead of `hrtimer_forward_now()`
High CPU with nothing running	`mdelay()` busy-waiting
`ksoftirqd` at high CPU	Softirq load exceeding what can be handled inline
Two work items corrupt shared state	Concurrent execution on different CPUs — use `max_active = 1` or a lock

Check yourself

- ✓ Explain the top-half / bottom-half split and why top halves must be short.
- ✓ Compare the four mechanisms on sleeping, latency and cost.

- ✓ Write a threaded IRQ handler and explain `IRQF_ONESHOT`.
- ✓ Say when to pass NULL as the hard handler.
- ✓ Write a work item, a delayed work item and a private workqueue.
- ✓ Give the correct teardown order and explain both bugs the wrong order causes.
- ✓ Say when `WQ_MEM_RECLAIM` is required.
- ✓ Distinguish `timer_list` from `hrtimer`, and re-arm a periodic timer without drift.
- ✓ Choose between `udelay`, `usleep_range` and `msleep`.
- ✓ Explain why tasklets are deprecated and what replaces them.

Interrupt handling in Linux

From the GIC delivering an exception to your handler being called: the number spaces, the device-tree description, the registration API, and the handful of mistakes that produce interrupt storms and missed events.

The path from wire to handler

```
  device asserts its interrupt line
       |
       v
  GIC distributor -> redistributor -> CPU interface
       |  (ARM ch. 23-24: priority, targeting, grouping)
       v
  CPU takes an IRQ exception -> VBAR_EL1 + 0x280
       |  (ARM ch. 27-28)
       v
  arch/arm64/kernel/entry.S  el1_irq / el0_irq
       |
       v
  handle_arch_irq  ->  gic_handle_irq()
       |
       |  read GICC_IAR / ICC_IAR1_EL1 -> the hardware INTID
       v
  irq_find_mapping(domain, hwirq)  ->  the LINUX IRQ NUMBER
       |
       v
  generic_handle_irq(virq)
       |
       v
  handle_level_irq() / handle_edge_irq()   the FLOW HANDLER
       |    mask, ack, or eoi as the type requires
       v
  for each action on this irq:
       your_handler(irq, dev_id)
       |
       v
  IRQ_HANDLED / IRQ_WAKE_THREAD / IRQ_NONE
       |
       v
  EOI to the GIC, return from exception

TWO NUMBER SPACES, and confusing them wastes hours:

   HWIRQ    the hardware number. SPI 42, or "bit 3 of GPIO
            controller 2". Meaningful only to that
            controller. This is what the DEVICE TREE says.

   VIRQ     the Linux irq number. A global integer allocated
            by the kernel. What you pass to request_irq().
            What /proc/interrupts shows.

The IRQ DOMAIN maps between them, per controller. That is
what makes nested controllers work: a GPIO chip is an
interrupt controller whose own interrupt comes from the GIC,
and each of its pins gets a virq of its own.
```

Describing an interrupt in the device tree

```
/* GIC SPI, the common case: */

        my_device: device@1c000000 {
                compatible = "vendor,my-device";
                reg = <0x0 0x1c000000 0x0 0x1000>;
                interrupts = <GIC_SPI 42 IRQ_TYPE_LEVEL_HIGH>;
                interrupt-parent = <&gic>;   /* often inherited */
        };

/* THREE CELLS for a GICv2/v3:
     cell 1: GIC_SPI (0) or GIC_PPI (1)
     cell 2: the interrupt number WITHIN that type.
             SPI n in DT = INTID (n + 32) in the GIC.
             THIS OFF-BY-32 CATCHES EVERYONE. The TRM says
             "INTID 74"; the device tree says 42.
     cell 3: the trigger type:
                IRQ_TYPE_EDGE_RISING     1
                IRQ_TYPE_EDGE_FALLING    2
                IRQ_TYPE_LEVEL_HIGH      4
                IRQ_TYPE_LEVEL_LOW       8

/* NAMED interrupts, when a device has several - always prefer
   this over relying on index order: */
        interrupts = <GIC_SPI 42 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 43 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "tx", "rx";

/* A GPIO used as an interrupt - a NESTED controller: */
        interrupt-parent = <&gpio2>;
        interrupts = <7 IRQ_TYPE_EDGE_FALLING>;
        /* two cells here, because that is what gpio2's
           #interrupt-cells says. Every controller defines its
           own cell format - check its binding.            */

/* GETTING THE VIRQ IN YOUR DRIVER: */
        irq = platform_get_irq(pdev, 0);           /* by index */
        irq = platform_get_irq_byname(pdev, "rx"); /* by name  */
        if (irq < 0)
                return irq;      /* it already logged the error
                                    and may be -EPROBE_DEFER  */

/* platform_get_irq() does the whole domain translation for
   you. Never hard-code a number.                          */
```

THE SPI OFF-BY-32

GIC interrupt IDs 0–15 are SGIs, 16–31 are PPIs, and SPIs start at 32. The device tree's `GIC_SPI n` means INTID `n + 32`. A datasheet quoting "interrupt 74" almost always means INTID 74, which you write as `GIC_SPI 42`. Getting this wrong gives you a driver whose interrupt never fires, or — worse — one that steals another device's.

Registering a handler

```
/* The managed form. Use it. */

        ret = devm_request_irq(&pdev->dev, irq,
                               my_handler,
                               0,                 /* flags */
                               dev_name(&pdev->dev),
                               d);                /* dev_id */
        if (ret)
                return dev_err_probe(&pdev->dev, ret,
                                     "cannot request irq %d\n",
                                     irq);

/* Or threaded (chapter 13), which is usually what you want: */
        ret = devm_request_threaded_irq(&pdev->dev, irq,
                                        my_hardirq, my_thread,
                                        IRQF_ONESHOT,
                                        dev_name(&pdev->dev), d);

/* THE FLAGS:
     IRQF_SHARED       the line is shared with other devices
     IRQF_ONESHOT      keep masked until the thread finishes
     IRQF_TRIGGER_*    override the DT trigger type. Do not -
                       fix the device tree instead.
     IRQF_NO_SUSPEND   keep enabled across suspend
     IRQF_PERCPU       a per-CPU interrupt (a PPI)
     IRQF_NOBALANCING  exclude from irqbalance

/* dev_id MUST be unique and non-NULL for a shared interrupt -
   it is how free_irq() knows which action to remove, and it
   is what your handler receives. Pass your device struct.

/* THE HANDLER: */
static irqreturn_t my_handler(int irq, void *dev_id)
{
        struct my_dev *d = dev_id;
        u32 status;

        status = readl(d->base + REG_STATUS);

        /* ON A SHARED LINE THIS CHECK IS MANDATORY: */
        if (!(status & d->irq_mask))
                return IRQ_NONE;        /* not mine */

        writel(status, d->base + REG_STATUS);   /* ack */
        ... handle ...

        return IRQ_HANDLED;
}

/* RETURN VALUES:
     IRQ_HANDLED       it was mine and I dealt with it
     IRQ_NONE          not mine. On a shared line, essential.
     IRQ_WAKE_THREAD   run the threaded handler

   IF EVERY handler on a line returns IRQ_NONE repeatedly, the
   kernel disables the interrupt and prints:
        "irq 42: nobody cared (try booting with irqpoll)"
        "Disabling IRQ #42"
   which is the spurious-interrupt protection saving your
```

```
   system from a livelock. It means the interrupt is asserted
   and nobody is clearing it.                              */
```

Edge versus level: the failure modes differ

```
LEVEL-TRIGGERED: the line stays asserted until the device is
serviced.

  · If your handler does not CLEAR the condition in the
    device, the interrupt fires again immediately, forever.
    -> interrupt storm, system livelock, "nobody cared".
  · Cannot be missed: if it is still asserted, it will be
    taken.
  · Safe with sharing.
  · Needs IRQF_ONESHOT with a threaded handler.

EDGE-TRIGGERED: a transition latches a pending bit.

  · CAN BE MISSED. The classic race:

       handler reads STATUS       -> sees event A
       ... device raises event B, edge occurs ...
       handler clears STATUS      -> clears BOTH
       handler returns
       -> event B is lost, and no further edge will come
          because the device is now waiting for you

    THE FIX - loop until the device is genuinely idle:

       do {
               status = readl(d->base + REG_STATUS);
               if (!status)
                       break;
               writel(status, d->base + REG_STATUS); /* ack */
               handle(status);
       } while (1);

    Bound the loop if the hardware might never settle.

  · Also: clear the condition BEFORE re-enabling, or you
    latch your own acknowledgement.

  THE SYMPTOM of getting edge handling wrong is a device that
  works fine and then stops responding forever under load -
  one lost edge and it waits for an event that will never be
  re-signalled.
```

Controlling interrupts

```
        disable_irq(irq);          /* waits for handlers to
                                      finish. NOT from inside
                                      the handler - deadlock. */
        disable_irq_nosync(irq);   /* returns immediately */
        enable_irq(irq);           /* must MATCH the number of
                                      disables - they nest    */

        /* from within the handler, to mask your own line: */
        disable_irq_nosync(irq);

        /* wake-up capability, for suspend (chapter 25): */
        enable_irq_wake(irq);
        disable_irq_wake(irq);

/* LOCAL interrupt control - almost never right in a driver: */
        local_irq_disable();       /* this CPU only */
        local_irq_enable();
        local_irq_save(flags);
        local_irq_restore(flags);

/* Use spin_lock_irqsave() instead (chapter 11). Bare
   local_irq_disable() in a driver is a red flag: it does not
   protect against other CPUs, it hurts system latency, and it
   is almost always a lock in disguise.                    */

/* AFFINITY (chapter 10): */
        irq_set_affinity_hint(irq, cpumask_of(2));
```

Reading /proc/interrupts

```
$ cat /proc/interrupts
            CPU0       CPU1       CPU2       CPU3
   3:      12043      11982          0          0  GICv3  30 Level arch_timer
  42:       8471          0          0          0  GICv3  74 Level 1c000000.my-device
  43:          0          0          0          0  GICv3  75 Edge  1c000000.my-device
  56:     102394          0          0          0  GICv3  88 Level eth0
 IPI0:      4821       5102       4993       4877  Rescheduling interrupts

# COLUMN BY COLUMN:
#   42        the LINUX irq (virq)
#   counts    per CPU. This is the useful part.
#   GICv3     the controller
#   74        the HWIRQ - note 42 in DT became INTID 74
#   Level     the trigger type ACTUALLY configured
#   name      what you passed to request_irq

# WHAT TO LOOK FOR:
#   count is ZERO         -> the interrupt never fires. Wrong
#                            number, wrong trigger type, device
#                            not enabled, or pin not muxed.
#   count climbs wildly   -> a storm; the condition is not
#                            being cleared.
#   count on one CPU only -> normal unless you expected
#                            balancing.
#   your device missing   -> request_irq never succeeded.
#   trigger type differs from your DT -> something overrode it
#                            with IRQF_TRIGGER_*.

# Watch it live:
    watch -d -n1 'grep my-device /proc/interrupts'
```

When the interrupt does not fire

```
# Work down this list. It is almost always one of these.

# 1. Does the count in /proc/interrupts move at all?
#    No -> the interrupt is not reaching the CPU.

# 2. Is the device actually enabled?
    devmem 0x1c000004        # read the interrupt-enable reg
#    Many devices need BOTH a global enable and a per-source
#    unmask, and drivers commonly set only one.

# 3. Is the pin muxed as an interrupt?
    cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep ...
#    A GPIO-based interrupt with the pin in the wrong function
#    is silent.

# 4. Is the number right?
    cat /sys/kernel/debug/irq/domains/*        # the mappings
    cat /sys/kernel/debug/irq/irqs/42          # this one
#    Compare hwirq against the TRM, remembering the +32.

# 5. Is the trigger type right?
#    A level-high device described as edge-rising fires once
#    at most. Check the "Level"/"Edge" column.

# 6. Is it masked at the GIC?
    cat /sys/kernel/debug/irq/irqs/42
      handler:  handle_level_irq
      status:   0x00000000
      istate:   0x00000000
      ddepth:   1                <-- DISABLED. Someone called
                                     disable_irq and did not
                                     re-enable.
      wdepth:   0

# 7. Is the clock to the device running? An unclocked
#    peripheral cannot assert anything.

# TRACING, once it does fire:
    echo 1 > /sys/kernel/tracing/events/irq/enable
    cat /sys/kernel/tracing/trace_pipe
      irq_handler_entry: irq=42 name=1c000000.my-device
      irq_handler_exit:  irq=42 ret=handled
```

How it shows up

Symptom	Cause
Count stays at 0	Wrong number, wrong trigger, device not enabled, pin not muxed, or clock off
Device not in `/proc/interrupts`	`request_irq` failed — check the probe return
`nobody cared, disabling IRQ`	Condition never cleared, or the wrong device is being blamed on a shared line
System livelocks on the first interrupt	Level-triggered and not acknowledged; missing `IRQF_ONESHOT`
Fires once, then never again	Level source described as edge, or the ack sequence is wrong
Works, then stops under load	Lost edge — handler does not loop until the device is idle
`request_irq` returns `-EBUSY`	Line already claimed without `IRQF_SHARED`, or trigger types conflict

Symptom	Cause
Deadlock in `disable_irq`	Called from within the handler — use `_nosync`
Interrupt stays disabled	Unbalanced `disable_irq` / `enable_irq` — check `ddepth`
Handler crashes on `dev_id`	Passed NULL, or the device struct was already freed
Interrupt does not wake the system	`enable_irq_wake()` not called

Check yourself

- ✓ Trace an interrupt from the GIC to your handler.
- ✓ Explain hwirq versus virq and what an IRQ domain does.
- ✓ Write an `interrupts` property and explain the SPI off-by-32.
- ✓ Use named interrupts and `platform_get_irq_byname`.
- ✓ Write a handler that behaves correctly on a shared line.
- ✓ Explain what `IRQ_NONE` is for and what "nobody cared" means.
- ✓ Contrast the failure modes of edge and level triggering.
- ✓ Write an edge handler that cannot lose an event.
- ✓ Read every column of `/proc/interrupts`.
- ✓ Work the seven-step checklist for an interrupt that never fires.

Kernel modules

A module is kernel code you can load and unload while the system runs. That single property changes development from "reboot to test" into "reload in a second", which is worth more to your productivity than any other kernel skill.

The smallest complete module

```
/* hello.c */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init hello_init(void)
{
        pr_info("hello: loaded\n");
        return 0;              /* non-zero aborts the load */
}

static void __exit hello_exit(void)
{
        pr_info("hello: unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");         /* REQUIRED. See below.      */
MODULE_AUTHOR("Your Name");
MODULE_DESCRIPTION("A minimal example module");
MODULE_VERSION("1.0");
```

```
# Makefile
obj-m += hello.o

KDIR ?= /lib/modules/$(shell uname -r)/build

all:
        $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
        $(MAKE) -C $(KDIR) M=$(PWD) clean

# CROSS-COMPILING for an ARM64 target - what you will
# actually do:
#   make -C /path/to/kernel/source M=$(PWD) \
#        ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- modules
#
# KDIR must point at a CONFIGURED kernel tree that matches the
# target EXACTLY - same version, same .config. A module built
# against a different configuration will refuse to load, or
# load and corrupt memory.
```

```
# Using it:
    sudo insmod hello.ko              # load this exact file
    lsmod | grep hello                # is it there?
    dmesg | tail                      # what did it print?
    sudo rmmod hello                  # unload
    modinfo hello.ko                  # metadata, parameters,
                                      # dependencies, vermagic

# modprobe, the better tool - resolves DEPENDENCIES:
    sudo cp hello.ko /lib/modules/$(uname -r)/extra/
    sudo depmod -a                    # rebuild the dep map
    sudo modprobe hello               # loads what it needs
    sudo modprobe -r hello            # unload, with deps
```

MODULE_LICENSE is not paperwork

```
/* It has a mechanical effect: symbols exported with
   EXPORT_SYMBOL_GPL are only linkable from a module whose
   license string is GPL-compatible.                        */

   "GPL"                  GPL v2 or later
   "GPL v2"
   "GPL and additional rights"
   "Dual BSD/GPL"
   "Dual MIT/GPL"
   "Proprietary"          <-- TAINTS the kernel, and cannot
                              use GPL-only symbols

/* Omit it entirely and you get:

     hello: module license 'unspecified' taints kernel.
     Disabling lock debugging due to kernel taint

   A tainted kernel is reported in every oops, and many kernel
   developers will decline to debug one. More practically:
   most of the API you need for a real driver is
   EXPORT_SYMBOL_GPL, so a "Proprietary" module cannot call
   it. This is a deliberate design decision, not an
   oversight.                                               */

/* CHECK the taint state: */
        cat /proc/sys/kernel/tainted
        # or read the flags in an oops header:
        #   "CPU: 0 PID: 431 Tainted: G           O"
        #    O = an out-of-tree module is loaded
        #    P = a proprietary module
        #    E = an unsigned module
```

Module parameters

```
static int buffer_size = 4096;
module_param(buffer_size, int, 0644);
MODULE_PARM_DESC(buffer_size, "Ring buffer size in bytes");

static char *device_name = "mydev";
module_param(device_name, charp, 0444);

static bool verbose;
module_param(verbose, bool, 0644);

static int channels[4];
static int nr_channels;
module_param_array(channels, int, &nr_channels, 0444);

/* SET THEM: */
        insmod mydrv.ko buffer_size=8192 verbose=1 \
               channels=1,2,3
        modprobe mydrv buffer_size=8192
        # built into the kernel, on the command line:
        mydrv.buffer_size=8192
        # persistently:
        echo "options mydrv buffer_size=8192" \
            > /etc/modprobe.d/mydrv.conf

/* AND READ OR CHANGE AT RUNTIME, if the mode allows: */
        cat /sys/module/mydrv/parameters/buffer_size
        echo 1 > /sys/module/mydrv/parameters/verbose

/* THE PERMISSION ARGUMENT is the sysfs mode:
        0        not visible in sysfs at all
        0444     read-only
        0644     writable by root

   Only use 0644 if changing the value at runtime is actually
   SAFE. A buffer size that is read once at probe time should
   be 0444 - otherwise you invite someone to change it and
   wonder why nothing happened.

/* AND THE BIGGER POINT: module parameters are a poor
   configuration mechanism for a real driver. They are global,
   not per-device. For anything describing HARDWARE, use the
   device tree (chapter 18). Parameters are for debug knobs
   and for behaviour that genuinely applies to the whole
   driver.                                                  */
```

Exporting and using symbols

```
/* In one module: */
int my_shared_function(int arg)
{
        ...
}
EXPORT_SYMBOL_GPL(my_shared_function);

/* In another - just call it, but the FIRST module must be
   loaded first. modprobe works this out from the symbol
   table; insmod does not.                                  */

        depmod -a
        modprobe consumer          # loads provider first

/* SEEING what a module needs and provides: */
        modinfo mydrv.ko | grep depends
        cat /proc/kallsyms | grep my_shared_function
        nm -u mydrv.ko             # undefined = required
        nm -g --defined-only mydrv.ko

/* WHEN A SYMBOL IS MISSING: */
        insmod: ERROR: could not insert module: Unknown symbol
                in module
        dmesg:
          mydrv: Unknown symbol my_shared_function (err -2)
          mydrv: Unknown symbol dma_alloc_attrs (err 0)
                                                 ^^^^^^
        # err -2  = the symbol does not exist at all
        # err 0   = it exists but is GPL-only and your license
        #           string does not qualify

/* VERSION MAGIC - the other common load failure: */
        insmod: ERROR: could not insert module: Invalid module
                format
        dmesg:
          mydrv: version magic '6.6.0 SMP mod_unload aarch64'
          should be '6.6.0-vendor SMP preempt mod_unload aarch64'

   Every difference matters: version, SMP, PREEMPT, and the
   ABI-relevant config options. Rebuild against the target's
   exact tree. With CONFIG_MODVERSIONS the CRC of each
   exported function's signature is checked too, so a changed
   struct definition is caught rather than silently
   miscompiled.                                             */
```

Making a module safely unloadable

> **UNLOAD IS WHERE MODULES CRASH**
>
> Loading is easy. Unloading requires that *nothing* can still call into your code after `module_exit` returns, because the memory holding your functions is freed. Everything you registered must be unregistered, and everything asynchronous must be waited for.

```
static void __exit my_exit(void)
{
        /* IN REVERSE ORDER OF INITIALISATION, and stopping
           sources BEFORE cancelling consumers:            */

        /* 1. stop new work arriving */
        platform_driver_unregister(&my_driver);
        misc_deregister(&my_misc);
        unregister_chrdev_region(devno, 1);

        /* 2. stop the hardware */
        writel(0, base + REG_INT_ENABLE);
        free_irq(irq, d);

        /* 3. WAIT for anything already in flight */
        cancel_delayed_work_sync(&d->poll_work);
        cancel_work_sync(&d->work);
        del_timer_sync(&d->timer);
        hrtimer_cancel(&d->hrt);
        kthread_stop(d->task);
        synchronize_rcu();              /* if you used RCU */

        /* 4. only now, free */
        kfree(d->buffer);
        iounmap(base);
}

/* THE _sync SUFFIX IS NOT OPTIONAL. cancel_work() returns
   immediately; cancel_work_sync() waits for a running
   instance to finish. The first one gives you a crash a few
   microseconds after rmmod, in code that no longer exists.

/* REFERENCE COUNTING - preventing unload while in use:

   For file operations, .owner = THIS_MODULE in the
   file_operations struct does it automatically: the module
   count is incremented while any fd is open, and rmmod
   returns -EBUSY.

        static const struct file_operations my_fops = {
                .owner = THIS_MODULE,     <-- do not omit
                .open  = my_open,
                ...
        };

   For anything else:
        try_module_get(THIS_MODULE);
        module_put(THIS_MODULE);

   CHECK who is using you:
        lsmod
        Module      Size  Used by
        mydrv      16384  2 othermod,thirdmod
                          ^ refcount   ^ by whom

   "Used by 0" and rmmod still fails -> something took a
   reference without registering a name, or a device is still
   bound.                                                  */
```

Module versus built-in

	`=m` (module)	`=y` (built in)
Development speed	**Fast** — reload in a second	Slow — rebuild and reboot
Available at	Whenever loaded	Boot, at its initcall level
Needed for the root filesystem	Requires an initramfs	Works directly
Unloadable	Yes — exercises your teardown path	No
Memory	Only when loaded	Always
Ordering	By load order and dependencies	By initcall level, then link order

```
/* THE ADVICE: develop as a module, ship however you need.

   Building as a module during development is not just about
   iteration speed. It forces you to write a correct remove()
   path - and a driver whose teardown is correct is a driver
   that also survives unbind, suspend, and probe failure.

   TEST THE CYCLE, repeatedly:
        for i in $(seq 100); do
                insmod mydrv.ko && rmmod mydrv || break
        done
   with KASAN enabled. Anything that leaks or races shows up
   within a few iterations, and this is by far the cheapest
   bug-finding exercise available to you.

   ALSO TEST UNBIND/BIND, which exercises probe and remove
   without touching the module:
        echo 1c000000.my-device > \
             /sys/bus/platform/drivers/my-driver/unbind
        echo 1c000000.my-device > \
             /sys/bus/platform/drivers/my-driver/bind      */
```

Module signing

```
# A hardened kernel refuses unsigned modules (isolation
# ch. 14 - the chain of trust does not stop at the kernel):

    CONFIG_MODULE_SIG=y
    CONFIG_MODULE_SIG_FORCE=y       # REFUSE unsigned
    CONFIG_MODULE_SIG_ALL=y         # sign during the build
    CONFIG_MODULE_SIG_HASH="sha256"

# The build signs with certs/signing_key.pem. For a product,
# supply your own key and keep the private half in an HSM:
    CONFIG_MODULE_SIG_KEY="/path/to/key.pem"

# Sign an out-of-tree module by hand:
    scripts/sign-file sha256 signing_key.pem \
                      signing_key.x509 mydrv.ko

# SYMPTOM of an unsigned module on an enforcing kernel:
    insmod: ERROR: could not insert module: Key was rejected
            by service
    dmesg: Loading of unsigned module is rejected

# Without SIG_FORCE it loads but taints with 'E'.
```

How it shows up

Symptom	Cause
`Invalid module format`	Version magic mismatch — rebuild against the target's exact tree
`Unknown symbol ... (err -2)`	Provider module not loaded, or the function does not exist in this version
`Unknown symbol ... (err 0)`	GPL-only symbol, non-GPL `MODULE_LICENSE`
`module license 'unspecified' taints kernel`	Missing `MODULE_LICENSE`
`Key was rejected by service`	Unsigned module, `MODULE_SIG_FORCE` enabled
`rmmod: Device or resource busy`	Refcount non-zero — check `lsmod`'s "Used by"
Crash seconds after `rmmod`	Work, timer or IRQ not cancelled synchronously
Crash in freed module text	A callback still registered somewhere
Memory grows across load/unload cycles	A leak in the teardown path — the 100-cycle test finds it
Works as a module, fails built in	An ordering dependency — use deferred probe (ch. 04)
Parameter change has no effect	Read once at probe; the sysfs write only changed the variable
`modprobe` works, `insmod` fails	Unresolved dependencies — that is the difference between them

Check yourself

- ✓ Write a minimal module and its Makefile, and cross-compile it for arm64.
- ✓ Explain the mechanical effect of `MODULE_LICENSE`.
- ✓ Add parameters of three types and choose their sysfs modes correctly.
- ✓ Say why the device tree is preferable to parameters for hardware description.

- ✓ Diagnose `Unknown symbol` with `err -2` versus `err 0`.
- ✓ Explain version magic and what it protects against.
- ✓ Write a correct `__exit` function, in the right order, with `_sync` everywhere.
- ✓ Explain what `.owner = THIS_MODULE` does.
- ✓ Run a 100-iteration load/unload test under KASAN, and an unbind/bind cycle.
- ✓ Sign a module and explain where it fits in the chain of trust.

Building and booting a custom kernel

Configure, cross-compile, boot it under QEMU, and get to a shell — with a development loop fast enough that you will actually use it. This is the workbench everything else in the track runs on.

The toolchain

```
# Debian/Ubuntu:
sudo apt install gcc-aarch64-linux-gnu build-essential \
     bc bison flex libssl-dev libelf-dev libncurses-dev \
     qemu-system-arm qemu-user-static debootstrap \
     device-tree-compiler cpio

# Verify:
aarch64-linux-gnu-gcc --version
qemu-system-aarch64 --version

# EVERY kernel command needs these two, so export them once:
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-

# (Or pass them on each make line. Forgetting them and
# building a host x86 kernel by accident is the single most
# common wasted 20 minutes in embedded work.)
```

Configuring

```
# Start from a sensible base:
make defconfig              # arm64 generic - boots on QEMU
                            # virt and most server-class parts
make ARCH=arm64 vendor_defconfig      # a board's own config
                                      # in arch/arm64/configs/

# Then adjust:
make menuconfig             # ncurses. '/' SEARCHES - use it.
make nconfig                # nicer ncurses
make xconfig                # Qt
make savedefconfig          # writes a MINIMAL defconfig
                            # containing only non-default
                            # options - what you commit

# Scripted, which is what CI should do:
./scripts/config --enable  CONFIG_DEBUG_INFO_DWARF5
./scripts/config --enable  CONFIG_KASAN
./scripts/config --enable  CONFIG_PROVE_LOCKING
./scripts/config --module  CONFIG_MY_DRIVER
./scripts/config --disable CONFIG_MODULE_SIG_FORCE
./scripts/config --set-val CONFIG_LOG_BUF_SHIFT 18
make olddefconfig           # resolve the consequences
                            # WITHOUT prompting

# Other useful targets:
make oldconfig              # prompts for NEW options only -
                            # use after updating the tree
make listnewconfig          # what changed, without asking
make localmodconfig         # keep only modules currently
                            # loaded. Shrinks a distro config
                            # enormously; be careful, it
                            # drops anything not plugged in
                            # right now.
```

THE DEVELOPMENT CONFIG

Turn these on while writing drivers. They cost performance and find bugs your testing never will:

```
CONFIG_DEBUG_KERNEL=y
CONFIG_DEBUG_INFO_DWARF5=y     # for gdb and pahole
CONFIG_KASAN=y                 # memory errors
CONFIG_KASAN_INLINE=y
CONFIG_PROVE_LOCKING=y         # lockdep
CONFIG_DEBUG_ATOMIC_SLEEP=y    # sleeping in atomic context
CONFIG_DEBUG_SPINLOCK=y
CONFIG_DEBUG_MUTEXES=y
CONFIG_DEBUG_OBJECTS=y
CONFIG_DEBUG_LIST=y            # corrupted list_head
CONFIG_DEBUG_KMEMLEAK=y
CONFIG_DETECT_HUNG_TASK=y
CONFIG_PANIC_ON_OOPS=y         # fail loudly, do not limp on
CONFIG_FTRACE=y
CONFIG_FUNCTION_TRACER=y
CONFIG_DYNAMIC_DEBUG=y
CONFIG_MAGIC_SYSRQ=y
CONFIG_GDB_SCRIPTS=y
```

Building

```
# The whole thing:
make -j$(nproc) Image dtbs modules

# Targets that matter:
#   Image        the raw kernel (arm64 does not use zImage)
#   Image.gz     compressed - most bootloaders accept it
#   dtbs         all device trees for this arch
#   modules      the .ko files
#   vmlinux      the ELF with symbols - NOT bootable, but this
#                is what gdb and addr2line need. Keep it.

# Just one file, while iterating on a driver:
make drivers/mydrv/mydrv.ko

# Just one device tree:
make arch/arm64/boot/dts/vendor/myboard.dtb

# Install modules to a staging root:
make INSTALL_MOD_PATH=/path/to/rootfs modules_install

# SPEED, which matters more than anything else here:
    ccache      export CROSS_COMPILE="ccache aarch64-linux-gnu-"
                a rebuild after a config change goes from
                20 minutes to 2
    make -j$(nproc)
    -j with distcc or icecc if you have a build farm

# Verify what you produced:
    file arch/arm64/boot/Image
      Linux kernel ARM64 boot executable Image, little-endian
    ls -lh arch/arm64/boot/Image
```

Booting it under QEMU

```
# The fastest development loop available, and it needs no
# hardware. Start with an initramfs so there is no filesystem
# to manage.

# 1. Build a tiny userspace with busybox:
    wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2
    tar xf busybox-1.36.1.tar.bz2 && cd busybox-1.36.1
    make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
         defconfig
    ./scripts/config --enable CONFIG_STATIC   # no libc needed
    make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
         -j$(nproc) install          # -> ./_install

# 2. Build the initramfs:
    cd _install
    mkdir -p proc sys dev
    cat > init <<'EOF'
#!/bin/sh
mount -t proc  none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
echo "=== userspace is up ==="
exec /bin/sh
EOF
    chmod +x init
    find . | cpio -o -H newc | gzip > ../initramfs.cpio.gz

# 3. Boot:
    qemu-system-aarch64 \
      -M virt -cpu cortex-a57 -smp 4 -m 1024 \
      -kernel arch/arm64/boot/Image \
      -initrd initramfs.cpio.gz \
      -append "console=ttyAMA0 earlycon rdinit=/init" \
      -nographic

# Ctrl-A then X quits. Ctrl-A then C is the QEMU monitor.

# THE ITERATION LOOP is now:
#   edit -> make -j -> qemu -> test        ~30 seconds
# versus flashing hardware, which is minutes. Do as much as
# possible here and move to real hardware only for what
# genuinely needs it.
```

Useful QEMU options

```
# Attach a virtual disk:
    -drive file=rootfs.ext4,format=raw,if=none,id=hd0 \
    -device virtio-blk-device,drive=hd0 \
    -append "root=/dev/vda rw console=ttyAMA0"

# Networking with a port forward to ssh:
    -netdev user,id=net0,hostfwd=tcp::5555-:22 \
    -device virtio-net-device,netdev=net0

# SHARE A HOST DIRECTORY - the best way to get a freshly
# built module into the guest without rebuilding anything:
    -virtfs local,path=/host/dir,mount_tag=host,\
security_model=none,id=host
    # in the guest:
    mount -t 9p -o trans=virtio host /mnt

# Dump the device tree QEMU generated, to read or modify:
    -machine dumpdtb=virt.dtb
    dtc -I dtb -O dts virt.dtb > virt.dts

# Emulate TrustZone with secure firmware (isolation ch. 11):
    -machine virt,secure=on -bios bl1.bin

# GDB - halt at the first instruction and wait:
    -s -S
    # then, in another terminal:
    aarch64-linux-gnu-gdb vmlinux
    (gdb) target remote :1234
    (gdb) hbreak start_kernel
    (gdb) continue
```

> **QEMU'S `VIRT` MACHINE IS A REAL, WELL-DOCUMENTED PLATFORM**
>
> It has a GIC, a PL011 UART, virtio devices, an SMMU (`-machine virt,iommu=smmuv3`), PSCI, and optional EL3 and EL2. Nearly everything in this track can be exercised on it, including SMMU faults, secure-world calls and CPU hotplug. Use hardware for timing, real peripherals, and power — and QEMU for everything else.

A real root filesystem

```
# Option 1: Debian, via debootstrap. Big, but complete -
# apt, gdb, perf, python all available.
    sudo debootstrap --arch=arm64 --foreign bookworm \
         ./rootfs http://deb.debian.org/debian
    sudo cp /usr/bin/qemu-aarch64-static ./rootfs/usr/bin/
    sudo chroot ./rootfs /debootstrap/debootstrap --second-stage
    sudo chroot ./rootfs passwd root

    dd if=/dev/zero of=rootfs.ext4 bs=1M count=2048
    mkfs.ext4 rootfs.ext4
    sudo mount -o loop rootfs.ext4 /mnt
    sudo cp -a rootfs/* /mnt/
    sudo umount /mnt

# Option 2: Buildroot. Small, fast, reproducible - what most
# embedded products actually use.
    git clone https://git.buildroot.net/buildroot
    cd buildroot
    make qemu_aarch64_virt_defconfig
    make menuconfig      # add packages, set the kernel
    make -j$(nproc)
    # produces output/images/{Image,rootfs.ext4,start-qemu.sh}

# Option 3: Yocto. Heaviest, most configurable, and the one
# you will meet at a vendor. Layers, recipes, bitbake.
# Worth learning if you ship products; overkill for study.

# INSTALL YOUR MODULES into whichever you chose:
    make INSTALL_MOD_PATH=./rootfs modules_install
    # then depmod against the TARGET's module tree:
    depmod -b ./rootfs $(make kernelrelease)
```

Onto real hardware

```
# The parts you now need that QEMU did not:

# 1. A SERIAL CONSOLE. Non-negotiable. A USB-TTL adapter and
#    three wires. Without it you are debugging blind.
    picocom -b 115200 /dev/ttyUSB0
    # or: screen /dev/ttyUSB0 115200

# 2. A device tree for YOUR board, not the generic one.
    make arch/arm64/boot/dts/vendor/myboard.dtb

# 3. A way to load a kernel without reflashing every time.
#    U-Boot over TFTP is the standard answer:
    # on the host:
    sudo apt install tftpd-hpa
    sudo cp Image myboard.dtb /srv/tftp/

    # in U-Boot:
    setenv serverip 192.168.1.10
    setenv ipaddr 192.168.1.20
    tftpboot ${kernel_addr_r} Image
    tftpboot ${fdt_addr_r} myboard.dtb
    setenv bootargs "console=ttyS0,115200 earlycon \
                     root=/dev/nfs nfsroot=192.168.1.10:/srv/nfs \
                     ip=dhcp rw"
    booti ${kernel_addr_r} - ${fdt_addr_r}

# 4. NFS ROOT, so userspace changes need no reflash either.
#    Kernel over TFTP + rootfs over NFS = a development loop
#    on real hardware that is nearly as fast as QEMU. Set this
#    up on day one of any board bring-up; it pays for itself
#    within a day.

# 5. A recovery path. Know how to get back to a working image
#    before you break the one that boots - a second boot slot,
#    an SD card, or a USB recovery mode.
```

Keeping the loop fast

Technique	Saves
`ccache`	Most of a rebuild after a config change
Build the driver as `=m`	The whole kernel rebuild, and the reboot
QEMU instead of hardware	Minutes per iteration
TFTP + NFS root	Reflashing, on real hardware
9p share into QEMU	Rebuilding the initramfs to test one module
`make M=drivers/mydrv`	Scanning the whole tree
Unbind/bind instead of reboot	Re-running probe without unloading
A scripted config	Re-deriving your options after every tree update

How it shows up

Symptom	Cause
Built an x86 kernel by accident	`ARCH` / `CROSS_COMPILE` not set
Config option keeps disappearing	Unmet dependency — `make menuconfig`, then `/` to search
`make olddefconfig` changes things you set	A `select` or dependency overrode you
Module will not load on the target	Version magic — built against the wrong tree or config
Kernel boots on QEMU, not on hardware	Wrong device tree, or a missing driver for the real console
No output on real hardware	Console mismatch — try `earlycon` (ch. 03)
Hangs mounting root	Storage driver is a module with no initramfs — build it in
Builds are slow	No ccache, no `-j`, or rebuilding everything for one file
QEMU exits immediately	Wrong `-M` / `-cpu`, or the Image is not arm64
Cannot type in QEMU	Missing `console=ttyAMA0` in `-append`
gdb symbols do not match	`vmlinux` is from a different build than the running Image

Check yourself

- ✓ Set up a cross toolchain and build an arm64 `Image` and `dtbs`.
- ✓ Configure with `scripts/config` and `olddefconfig`, and produce a minimal defconfig.
- ✓ List eight debug options worth enabling during development, and what each catches.
- ✓ Build a busybox initramfs and boot it under QEMU.
- ✓ Attach gdb to a QEMU-hosted kernel and break in `start_kernel`.
- ✓ Share a host directory into the guest to test a module without rebuilding the initramfs.
- ✓ Dump and read QEMU's generated device tree.
- ✓ Set up TFTP plus NFS root on real hardware.
- ✓ Explain why `vmlinux` must be kept even though it is not bootable.
- ✓ Get your edit-build-test loop under a minute, and say what dominates it.

The Linux device model

Buses, devices and drivers, and the matching process that brings them together. Once you can see this structure, every driver in the tree has the same shape — and the answer to "why does my probe never run" becomes a three-command investigation.

Three objects, one relationship

```
struct bus_type            "platform", "i2c", "spi", "pci"
     |     owns the MATCH function and the probe/remove
     |     protocol for everything on it
     |
     +-- struct device_driver     code that can drive
     |         · a name                something
     |         · a match table (of_match_table, id_table)
     |         · .probe / .remove
     |
     +-- struct device            a thing that exists
               · a parent (the topology)
               · a bus
               · resources: reg, irq, clocks, DMA config
               · a driver, once bound

THE CORE LOOP, in drivers/base/:

  a DEVICE is registered   -> try every driver on its bus
  a DRIVER is registered   -> try every unbound device on
                              its bus

  for each candidate:  bus->match(dev, drv)
                              |
                       matched? -> drv->probe(dev)
                                          |
                                    0     -> BOUND
                                    -EPROBE_DEFER -> retry later
                                    other -> not bound

That is the entire mechanism. Everything else is detail.

YOU SUBCLASS the generic structures:

  struct platform_device  { struct device dev; ... }
  struct i2c_client       { struct device dev; ... }
  struct spi_device       { struct device dev; ... }

and get back to yours with container_of(), or with the
helpers each bus provides (to_platform_device(dev)).
```

Where devices come from

Source	Bus	How
Device tree	platform, i2c, spi	`of_platform_populate()` walks the tree at boot. **The embedded default.**
ACPI	platform, pci	Firmware tables. Arm servers, some laptops.
Enumeration	PCI, USB	The bus itself reports what is plugged in, with vendor/product IDs.

Source	Bus	How
Parent driver	Any	An MFD or a controller registers its children.
Board file	platform	Static C tables. Legacy; replaced by DT on arm64.
Userspace	Any	Device tree overlays, or a bus-specific `new_device` sysfs file.

Matching

```
/* Each bus decides what "match" means. For platform devices
   (drivers/base/platform.c), it tries in order:              */

   1. OF: does dev->of_node's `compatible` appear in the
          driver's of_match_table?          <-- what you use
   2. ACPI match table
   3. the platform id_table
   4. plain strcmp of driver->name against device->name

/* THE OF WAY, which is what an embedded driver writes: */

static const struct of_device_id my_of_match[] = {
        { .compatible = "vendor,my-device-v2",
          .data = &v2_config },          /* per-variant data */
        { .compatible = "vendor,my-device",
          .data = &v1_config },
        { }                              /* SENTINEL - the
                                            empty entry is
                                            REQUIRED         */
};
MODULE_DEVICE_TABLE(of, my_of_match);   /* so modprobe can
                                           autoload this
                                           module for this
                                           compatible        */

static struct platform_driver my_driver = {
        .driver = {
                .name           = "my-driver",
                .of_match_table = my_of_match,
                .pm             = &my_pm_ops,
        },
        .probe  = my_probe,
        .remove = my_remove,
};
module_platform_driver(my_driver);       /* generates
                                            module_init and
                                            module_exit      */

/* USING THE MATCH DATA to support several variants with one
   driver - the standard pattern: */
static int my_probe(struct platform_device *pdev)
{
        const struct my_config *cfg;

        cfg = of_device_get_match_data(&pdev->dev);
        if (!cfg)
                return -EINVAL;

        d->nr_channels = cfg->nr_channels;
        d->has_dma     = cfg->has_dma;
        ...
}

/* THE SENTINEL MATTERS: the match loop runs until it finds an
   entry with an empty compatible string. Omit it and the
   kernel reads past the array - which manifests as a random
   crash during boot, or a driver that matches something
   absurd.                                                   */
```

A probe function, annotated

```
static int my_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        struct my_dev *d;
        int ret;

        /* 1. allocate the driver's per-device state, managed
              so it is freed automatically (chapter 26)      */
        d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;
        d->dev = dev;

        /* 2. MMIO - maps AND reserves the region */
        d->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(d->base))
                return PTR_ERR(d->base);

        /* 3. clocks. May defer if the clock driver is late. */
        d->clk = devm_clk_get_enabled(dev, "core");
        if (IS_ERR(d->clk))
                return dev_err_probe(dev, PTR_ERR(d->clk),
                                     "no core clock\n");

        /* 4. regulators, resets, GPIOs, phys - same shape */
        d->vdd = devm_regulator_get(dev, "vdd");
        if (IS_ERR(d->vdd))
                return dev_err_probe(dev, PTR_ERR(d->vdd),
                                     "no vdd supply\n");
        ret = regulator_enable(d->vdd);
        if (ret)
                return ret;

        /* 5. DT properties specific to this device */
        ret = device_property_read_u32(dev, "vendor,fifo-depth",
                                       &d->fifo_depth);
        if (ret)
                d->fifo_depth = 16;      /* a sane default */

        /* 6. per-device locks and structures */
        mutex_init(&d->lock);
        init_waitqueue_head(&d->wq);
        INIT_WORK(&d->work, my_work_fn);

        /* 7. the interrupt - LAST among resources, because
              from here on the hardware can call into us     */
        d->irq = platform_get_irq(pdev, 0);
        if (d->irq < 0)
                return d->irq;

        ret = devm_request_threaded_irq(dev, d->irq, NULL,
                                        my_thread_fn,
                                        IRQF_ONESHOT,
                                        dev_name(dev), d);
        if (ret)
                return dev_err_probe(dev, ret, "irq %d\n",
                                     d->irq);

        /* 8. stash it so remove() and the fops can find it */
        platform_set_drvdata(pdev, d);
```

```
        /* 9. initialise the hardware */
        ret = my_hw_init(d);
        if (ret)
                return ret;

        /* 10. LAST: expose it to userspace. Nothing before
               this point could be used by anyone, so there is
               no window in which a half-set-up device is
               reachable.                                      */
        ret = misc_register(&d->misc);
        if (ret)
                return ret;

        dev_info(dev, "ready, %u channels\n", d->nr_channels);
        return 0;
}

/* THE TWO ORDERING RULES that matter:
     · acquire resources before enabling anything that can
       generate events
     · register the userspace interface LAST, and unregister
       it FIRST in remove()                                  */
```

`DEV_ERR_PROBE()` IN ONE LINE

It logs the error *with the device name*, records the deferral reason where `/sys/kernel/debug/devices_deferred` can show it, downgrades `-EPROBE_DEFER` to a debug message so your logs are not flooded during normal boot, and returns the error code so you can `return` it directly. Use it for every resource acquisition in probe; there is no reason not to.

Deferred probe, properly understood

```
/* The problem: drivers probe in an order nobody controls, and
   your device needs a clock from a driver that has not loaded.

   The mechanism:

     probe returns -EPROBE_DEFER
        -> the device goes on the DEFERRED LIST
        -> every time ANY driver probes successfully, the
           whole deferred list is retried
        -> eventually the provider appears and you succeed

   So ordering becomes self-organising. No initcall levels, no
   Makefile ordering, no retry loops.                        */

/* WHICH APIs RETURN IT: essentially every resource getter -
   clk_get, regulator_get, gpiod_get, reset_control_get,
   dma_request_chan, phy_get, of_find_i2c_device_by_node...

   JUST PROPAGATE IT. Never swallow it:                     */
        if (IS_ERR(clk))
                return PTR_ERR(clk);     /* right */

        if (IS_ERR(clk)) {
                dev_err(dev, "no clock\n");
                return -ENODEV;          /* WRONG - destroys
                                            the deferral, and
                                            the device never
                                            probes           */
        }

/* WHEN IT NEVER RESOLVES: */
        cat /sys/kernel/debug/devices_deferred
        1c000000.my-device      platform: wait for supplier
                                /soc/clock-controller@1b000000

/* That names both the waiting device and what it waits for.
   If the supplier never appears:
     · its driver is not enabled in the config
     · its DT node is missing or has a typo in `compatible`
     · its own probe is failing - look further up the log

   AND THE END-OF-BOOT REPORT:
        deferred_probe_timeout=10        # cmdline
   After the timeout, remaining deferrals are converted to
   real failures and logged - which turns a silent
   never-probes into a visible error.                       */
```

The device model in sysfs

```
# Every device, by bus:
    ls /sys/bus/platform/devices/
    ls /sys/bus/i2c/devices/

# Every driver, and what it has bound:
    ls /sys/bus/platform/drivers/my-driver/
      bind  unbind  uevent  1c000000.my-device -> ...

# The physical topology:
    ls /sys/devices/platform/soc/1c000000.my-device/
      driver -> ../../../bus/platform/drivers/my-driver
      of_node -> ...
      power/
      uevent
      modalias

# MANUAL BIND AND UNBIND - the most useful debugging tool in
# this chapter:
    echo 1c000000.my-device > \
         /sys/bus/platform/drivers/my-driver/unbind
    echo 1c000000.my-device > \
         /sys/bus/platform/drivers/my-driver/bind

# This re-runs remove() and probe() without unloading the
# module or rebooting. It exercises exactly the paths that
# leak and race, and it is how you test error handling: make
# probe fail, unbind, fix, bind.

# What device tree node produced a device:
    cat /sys/devices/platform/.../of_node/compatible

# Which driver would autoload:
    cat /sys/devices/platform/.../modalias
      of:Nmy-deviceT(null)Cvendor,my-device
# udev matches this against MODULE_DEVICE_TABLE entries.
```

Classes and the userspace view

```
/* A device's place in /sys/devices is its PHYSICAL topology.
   A CLASS gives it a FUNCTIONAL identity:                  */

        /sys/class/net/eth0
        /sys/class/tty/ttyS0
        /sys/class/gpio/
        /sys/class/thermal/thermal_zone0
        /sys/class/misc/my-device

/* Registering with the right class is what makes generic
   userspace tools work with your device. A driver that
   registers as an IIO device gets libiio, a hwmon device gets
   `sensors`, an input device gets evdev - for free.

   THE ADVICE: before writing a character device with a custom
   ioctl interface (chapter 21), check whether a SUBSYSTEM
   already exists for your device class:

        IIO        sensors, ADCs, DACs
        hwmon      temperature, voltage, fan
        input      buttons, touch, keyboards
        rtc        real-time clocks
        watchdog   watchdogs
        pwm, led, gpio, regulator, clk, thermal
        v4l2       video capture
        ALSA       audio
        MTD        raw flash
        crypto     accelerators
        tee        trusted execution (isolation ch. 12)

   Using the right subsystem means: a stable, documented
   userspace ABI you did not have to design; existing tools;
   power management and buffering handled for you; and a much
   easier upstreaming path (chapter 30). Writing a bespoke
   char device for a temperature sensor is the most common
   avoidable mistake in embedded driver work.             */
```

Remove, the mirror image

```
static void my_remove(struct platform_device *pdev)
{
        struct my_dev *d = platform_get_drvdata(pdev);

        /* EXACT REVERSE of probe: */

        /* 1. userspace interface first - no new users */
        misc_deregister(&d->misc);

        /* 2. stop the hardware generating events */
        writel(0, d->base + REG_INT_ENABLE);

        /* 3. wait for anything in flight (chapter 13) */
        cancel_work_sync(&d->work);

        /* 4. quiesce the device */
        my_hw_shutdown(d);

        /* 5. devm_ handles the irq, ioremap, clk and the
              allocation itself - in reverse order,
              automatically                                */

        /* 6. what devm does NOT handle: anything you enabled
              or initialised by hand                       */
        regulator_disable(d->vdd);
}

/* NOTE remove() returns void in modern kernels (6.11+). It
   used to return int, and the return value was ignored
   anyway - because there is no meaningful way to refuse to
   remove a device that is going away.                    */
```

How it shows up

Symptom	Cause
Probe never runs	No matching `compatible`; check the DT node and `of_match_table`
Device in `/sys/bus/platform/devices` with no `driver` link	No driver matched — compare strings character by character
Device not in sysfs at all	DT node has `status = "disabled"`, or is not under a populated bus
Probe runs, returns `-517` forever	Supplier never appears — check `devices_deferred`
Device never probes and nothing is logged	Deferral swallowed and converted to another error
Random crash during boot enumeration	Missing sentinel in the match table
Module does not autoload	Missing `MODULE_DEVICE_TABLE`
Crash on unbind	Teardown order wrong, or missing `_sync` cancellations
Leak across bind/unbind cycles	Something acquired by hand and never released
Userspace sees a half-initialised device	Registered the interface before finishing setup
Wrong variant behaviour on a similar board	Match data not used; hard-coded per-SoC values

Check yourself

- ✓ Describe the bus/device/driver relationship and the core matching loop.
- ✓ Name six sources of devices and which dominates on embedded arm64.
- ✓ Write an `of_match_table` with variant data, and explain the sentinel.
- ✓ Explain what `MODULE_DEVICE_TABLE` enables.
- ✓ Write a probe function in the correct order and justify each step's position.
- ✓ Explain everything `dev_err_probe` does.
- ✓ Explain deferred probe and why swallowing `-EPROBE_DEFER` is fatal.
- ✓ Use `devices_deferred` to diagnose a device that never probes.
- ✓ Use unbind/bind to exercise probe and remove without rebooting.
- ✓ Name six kernel subsystems and say why using one beats a custom char device.

Device tree

A data structure describing hardware that cannot discover itself. It is how one kernel binary runs on a thousand different boards, and writing one correctly is half of embedded Linux bring-up.

Why it exists

```
PCI and USB devices announce themselves: vendor ID, device ID,
BARs, interrupts — all readable from the bus.

An SoC's peripherals do not. There is no way to ask a chip
"is there a UART at 0x1c000000, and which interrupt does it
use?" The answer exists only in the datasheet.

  BEFORE DEVICE TREE: "board files" — C code in
  arch/arm/mach-*/ with static tables per board. By 2011 this
  was tens of thousands of lines of near-duplicate data, one
  kernel per board, and it was famously unsustainable.

  WITH DEVICE TREE: the hardware description is DATA, passed
  to the kernel at boot. One arm64 kernel binary boots
  hundreds of boards — the difference is which .dtb the
  bootloader hands it.

  WHAT BELONGS IN IT:
     what hardware exists, and where
     addresses, interrupts, clocks, resets, regulators
     how blocks connect to one another
     board-specific wiring: which GPIO is the reset line

  WHAT DOES NOT:
     policy or configuration ("enable debug mode")
     anything discoverable at runtime
     driver implementation details
     Linux-specific concepts

  The last point is a real rule with real teeth: device tree
  bindings are an ABI shared with U-Boot, BSD, and other
  kernels. "It is what my driver happens to need" is not a
  justification for a property.
```

Syntax

```
/dts-v1/;

#include <dt-bindings/interrupt-controller/arm-gic.h>
#include <dt-bindings/gpio/gpio.h>

/ {                                   /* the ROOT node */
        model = "Vendor MyBoard v2";
        compatible = "vendor,myboard", "vendor,mysoc";
        #address-cells = <2>;         /* addresses use 2 cells */
        #size-cells = <2>;            /* sizes use 2 cells    */

        chosen {
                stdout-path = "serial0:115200n8";
                bootargs = "console=ttyAMA0 earlycon";
        };

        aliases {
                serial0 = &uart0;
        };

        memory@80000000 {
                device_type = "memory";
                reg = <0x0 0x80000000 0x0 0x40000000>;
        };

        soc {
                compatible = "simple-bus";
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;               /* child addresses ==
                                         parent addresses    */

                uart0: serial@1c000000 {
                        compatible = "arm,pl011", "arm,primecell";
                        reg = <0x0 0x1c000000 0x0 0x1000>;
                        interrupts = <GIC_SPI 5 IRQ_TYPE_LEVEL_HIGH>;
                        clocks = <&uart_clk>, <&apb_clk>;
                        clock-names = "uartclk", "apb_pclk";
                        status = "okay";
                };
        };
};

/* THE PIECES:
     label:            uart0 - a handle for &references
     node@address      the @ part MUST match the first reg
                       entry, or dtc warns
     property = value;
     <>   cells: 32-bit big-endian integers
     ""   strings
     []   raw bytes
     &lbl a phandle - a reference to another node
     ;    every property ends with one                    */
```

#address-cells and #size-cells

```
The single most common source of silent breakage.

  A PARENT declares how many 32-bit cells its CHILDREN use in
  their `reg` properties:

     #address-cells = <1>;  -> a 32-bit address
     #address-cells = <2>;  -> a 64-bit address (two cells)
     #size-cells    = <1>;  -> a 32-bit size
     #size-cells    = <2>;  -> a 64-bit size

  So with #address-cells=2, #size-cells=2:

     reg = <0x0 0x1c000000 0x0 0x1000>;
            \_____________/ \________/
             address, 64-bit  size, 64-bit
             = 0x1c000000     = 0x1000

  and with #address-cells=1, #size-cells=1:

     reg = <0x1c000000 0x1000>;

  GET IT WRONG and the values are parsed as completely
  different numbers. Symptoms: the kernel reports absurd
  memory sizes, ioremap maps the wrong place, or a device gets
  a zero-length resource. NOTHING WARNS at runtime - the data
  is syntactically valid, just meaningless.

  THE RULE: the cells properties on the PARENT govern the
  child's reg. An i2c controller declares
  #address-cells = <1>; #size-cells = <0>; because its
  children have a bus address and no size:

     eeprom@50 { reg = <0x50>; };

  ALWAYS run `make dtbs_check` - it catches most of these.
```

The standard properties

Property	Meaning
`compatible`	What this is, most specific first. The match key.
`reg`	Address and size, per the parent's cells
`interrupts`	Per the interrupt parent's `#interrupt-cells`
`interrupt-parent`	Which controller; inherited if absent
`clocks` / `clock-names`	Clock inputs, by phandle and name
`resets` / `reset-names`	Reset lines
`*-supply`	A regulator, e.g. `vdd-supply = <&reg_1v8>`
`*-gpios`	GPIOs, e.g. `reset-gpios`, with flags
`dmas` / `dma-names`	DMA channels
`status`	`"okay"` or `"disabled"` — disabled nodes create no device
`ranges`	Address translation between a bus and its parent

Property	Meaning
`iommus`	The SMMU and StreamID (isolation ch. 03)
`dma-coherent`	This device's DMA is coherent with the CPU caches
`dma-ranges`	What physical range the device can address
`memory-region`	A reserved-memory carve-out

```
/* COMPATIBLE, most specific first - this is what lets one
   driver serve many parts: */

        compatible = "vendor,mysoc-uart", "arm,pl011",
                     "arm,primecell";

/* The kernel tries each in order. If nobody claims the
   SoC-specific string, the generic pl011 driver matches -
   which is exactly right when the block really is a pl011.

/* NAMING RULES for a new compatible:
        "manufacturer,model"
        lowercase, no underscores
        the manufacturer prefix must be in
        Documentation/devicetree/bindings/vendor-prefixes.yaml
        include a version or SoC name if variants differ:
             "vendor,mysoc-v2-uart"                          */
```

How real device trees are organised

```
# arch/arm64/boot/dts/vendor/

  mysoc.dtsi          the SoC: every peripheral, addresses,
                      interrupts, clocks. status = "disabled"
                      for anything optional.
  mysoc-pinctrl.dtsi  pin configurations
  myboard-common.dtsi shared across a board family
  myboard-v1.dts      one board: which peripherals are wired,
                      which regulators, which GPIOs
  myboard-v2.dts

# A board file INCLUDES the SoC and then OVERRIDES:

/dts-v1/;
#include "mysoc.dtsi"

/ {
        model = "Vendor MyBoard v2";
        compatible = "vendor,myboard-v2", "vendor,mysoc";
};

&uart0 {
        status = "okay";                /* enable it */
        pinctrl-0 = <&uart0_pins>;
        pinctrl-names = "default";
};

&i2c1 {
        status = "okay";
        clock-frequency = <400000>;

        /* a device that only exists on THIS board */
        sensor@48 {
                compatible = "vendor,my-sensor";
                reg = <0x48>;
                interrupt-parent = <&gpio2>;
                interrupts = <7 IRQ_TYPE_EDGE_FALLING>;
                vdd-supply = <&reg_3v3>;
                reset-gpios = <&gpio1 12 GPIO_ACTIVE_LOW>;
                vendor,sample-rate-hz = <100>;
        };
};

# THE PATTERN: the SoC .dtsi describes what the SILICON has,
# all disabled. Each board .dts enables what is actually wired
# and adds what is on the board. That separation is why one
# .dtsi supports dozens of products.
```

Reading properties in a driver

```
/* Prefer the FIRMWARE-AGNOSTIC device_property_* API - it
   works for DT and ACPI alike: */

        u32 val;
        ret = device_property_read_u32(dev, "vendor,fifo-depth",
                                       &val);
        if (ret)
                val = 16;               /* default */

        u32 array[4];
        device_property_read_u32_array(dev, "vendor,thresholds",
                                       array, 4);

        const char *mode;
        device_property_read_string(dev, "vendor,mode", &mode);

        if (device_property_read_bool(dev, "vendor,invert"))
                d->invert = true;
        /* a BOOLEAN in DT is PRESENCE, not a value. Writing
           `vendor,invert = <0>;` still reads as TRUE.      */

        int n = device_property_count_u32(dev, "vendor,list");

/* THE OF-SPECIFIC API, when you need node relationships: */
        struct device_node *np = dev->of_node;
        struct device_node *child;

        for_each_available_child_of_node(np, child) {
                /* iterate sub-nodes - channels, ports, ... */
                of_property_read_u32(child, "reg", &idx);
                ...
                /* of_node_put(child) is handled by the macro
                   on normal exit, but you MUST call it if you
                   break out early:                          */
                if (error) {
                        of_node_put(child);
                        break;
                }
        }

        struct device_node *other;
        other = of_parse_phandle(np, "vendor,partner", 0);
        ...
        of_node_put(other);             /* REFCOUNTED - leaking
                                           these is a real bug */

/* CUSTOM PROPERTY NAMING: always prefix with your vendor
   string. "fifo-depth" may collide with a future standard
   property; "vendor,fifo-depth" cannot.                    */
```

Bindings: the contract

```
# Documentation/devicetree/bindings/.../vendor,my-device.yaml
# A new binding MUST come with the driver, in YAML, and it is
# machine-checked.

%YAML 1.2
---
$id: http://devicetree.org/schemas/misc/vendor,my-device.yaml#
$schema: http://devicetree.org/meta-schemas/core.yaml#

title: Vendor My Device

maintainers:
  - Your Name <you@example.com>

properties:
  compatible:
    enum:
      - vendor,my-device
      - vendor,my-device-v2

  reg:
    maxItems: 1

  interrupts:
    maxItems: 1

  clocks:
    maxItems: 1

  vendor,fifo-depth:
    $ref: /schemas/types.yaml#/definitions/uint32
    enum: [8, 16, 32, 64]
    default: 16
    description: Depth of the hardware FIFO in entries.

required:
  - compatible
  - reg
  - interrupts
  - clocks

additionalProperties: false

examples:
  - |
    device@1c000000 {
        compatible = "vendor,my-device";
        reg = <0x1c000000 0x1000>;
        interrupts = <0 42 4>;
        clocks = <&clk 3>;
        vendor,fifo-depth = <32>;
    };

# CHECK IT:
    make dt_binding_check DT_SCHEMA_FILES=vendor,my-device.yaml
    make dtbs_check                # every dtb against every
                                   # schema - run this
    pip install dtschema           # needed for the above
```

> **A BINDING IS A PERMANENT ABI**
> Once a device tree using your property ships in a product or a bootloader, you cannot change its meaning. Old device trees must keep working with new kernels — that is why `dtbs_check` exists and why maintainers review bindings more carefully than driver code. Design the properties before you write the driver, not after.

Inspecting and debugging

```
# The live tree, as the running kernel sees it:
    ls /proc/device-tree/
    cat /proc/device-tree/model
    hexdump -C /proc/device-tree/soc/serial@1c000000/reg
    # or, much better:
    dtc -I fs -O dts /proc/device-tree > live.dts

# Compile and decompile:
    dtc -I dts -O dtb -o my.dtb my.dts
    dtc -I dtb -O dts my.dtb > readable.dts   # what ACTUALLY
                                              # got built,
                                              # after includes
# ALWAYS decompile after building. The result of #include and
# override chains is frequently not what you intended, and
# reading the flattened output takes ten seconds.

# Preprocess only, to debug an include problem:
    cpp -nostdinc -I include -undef -x assembler-with-cpp \
        my.dts | less

# Which devices exist and what bound:
    ls /sys/firmware/devicetree/base/
    ls /sys/bus/platform/devices/
    cat /sys/kernel/debug/devices_deferred

# OVERLAYS - patch the tree at runtime, for add-on boards:
    /dts-v1/;
    /plugin/;
    &{/soc/i2c@1c001000} {
            sensor@48 { compatible = "vendor,my-sensor";
                        reg = <0x48>; };
    };
    dtc -@ -I dts -O dtb -o overlay.dtbo overlay.dts
    # applied by the bootloader, or via configfs where
    # CONFIG_OF_OVERLAY and a suitable interface exist
```

How it shows up

Symptom	Cause
Device does not appear at all	`status = "disabled"`, or the node is not under a populated bus
Driver never probes	`compatible` string mismatch — compare exactly
Absurd memory size, or ioremap of the wrong address	`#address-cells` / `#size-cells` mismatch
Resource has size 0	Same cause, in the `reg` parse
Interrupt never fires	Wrong cell count, wrong trigger, or the SPI off-by-32 (ch. 14)
Boolean property has no effect	Presence is truth — `= <0>` is still true

Symptom	Cause
Your override did not apply	Label out of scope, or the include order puts the SoC file after it — decompile and look
Node refcount leak warnings	Missing `of_node_put()` on an early `break`
Works on one board revision, not another	Board-specific wiring in the shared `.dtsi`
`dtbs_check` failures	The binding and the DT disagree — one of them is wrong
New kernel breaks an old board	A binding was changed incompatibly

Check yourself

- ✓ Explain why device tree exists and what belongs in it versus what does not.
- ✓ Write a node with `compatible`, `reg`, `interrupts` and `clocks`.
- ✓ Explain `#address-cells`/`#size-cells` and decode a `reg` under each setting.
- ✓ Explain why `compatible` lists several strings and how the fallback works.
- ✓ Describe the SoC `.dtsi` / board `.dts` split and why it scales.
- ✓ Read four kinds of property in a driver and handle a missing one.
- ✓ Explain the boolean-property trap.
- ✓ Write a YAML binding and validate it with `dt_binding_check`.
- ✓ Decompile a built dtb and confirm your overrides applied.
- ✓ Say why a binding is an ABI and what that constrains.

Writing a platform driver

Everything from the last two chapters, assembled into one complete, correct driver. This is the skeleton you will start from every time — read it once carefully and you have the shape of most of `drivers/`.

What a platform device is

```
/* A device that is NOT on a discoverable bus. It exists
   because the device tree says so.

     · memory-mapped registers at a fixed address
     · one or more interrupts
     · clocks, resets, regulators, GPIOs, DMA channels
     · no vendor/product ID to enumerate

   That describes most of an SoC: UARTs, timers, watchdogs,
   crypto engines, DMA controllers, display pipelines, custom
   accelerators.

   IF YOUR DEVICE IS ON I2C, SPI OR PCI, use that bus's driver
   type instead - i2c_driver, spi_driver, pci_driver. The
   shape is identical; only the resource acquisition and the
   register access differ.                                  */
```

The complete driver

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Vendor MyDevice driver
 *
 * Locking:
 *      d->lock protects d->state and the register block.
 *      Never held across a sleep other than the regmap I/O
 *      it wraps.
 */

#include <linux/clk.h>
#include <linux/interrupt.h>
#include <linux/io.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/property.h>
#include <linux/regulator/consumer.h>
#include <linux/reset.h>

#define MYDEV_REG_ID            0x00
#define MYDEV_REG_CTRL          0x04
#define  MYDEV_CTRL_ENABLE      BIT(0)
#define  MYDEV_CTRL_RESET       BIT(1)
#define MYDEV_REG_STATUS        0x08
#define  MYDEV_STATUS_IRQ       BIT(0)
#define  MYDEV_STATUS_BUSY      BIT(1)
#define MYDEV_REG_DATA          0x0c
#define MYDEV_REG_INT_EN        0x10

#define MYDEV_EXPECTED_ID       0x4d594445

/* Per-variant differences, selected by compatible string */
struct mydev_config {
        unsigned int nr_channels;
        unsigned int fifo_depth;
        bool         has_dma;
};

struct mydev {
        struct device           *dev;
        void __iomem            *base;
        struct clk              *clk;
        struct reset_control    *rst;
        struct regulator        *vdd;
        int                      irq;

        const struct mydev_config *cfg;
        u32                      fifo_depth;

        struct mutex             lock;      /* see header */
        wait_queue_head_t        wq;
        bool                     done;
        u32                      last_status;
};

/* ---- register helpers ---------------------------------- */

static inline u32 mydev_rd(struct mydev *d, u32 reg)
```

```
{
        return readl(d->base + reg);
}

static inline void mydev_wr(struct mydev *d, u32 reg, u32 val)
{
        writel(val, d->base + reg);
}

/* ---- interrupt ----------------------------------------- */

static irqreturn_t mydev_isr(int irq, void *dev_id)
{
        struct mydev *d = dev_id;
        u32 status;

        status = mydev_rd(d, MYDEV_REG_STATUS);
        if (!(status & MYDEV_STATUS_IRQ))
                return IRQ_NONE;        /* shared line */

        mydev_wr(d, MYDEV_REG_STATUS, MYDEV_STATUS_IRQ);  /* ack */

        d->last_status = status;
        d->done = true;
        wake_up_interruptible(&d->wq);

        return IRQ_HANDLED;
}

/* ---- hardware ------------------------------------------ */

static int mydev_hw_init(struct mydev *d)
{
        u32 id;

        /* IDENTITY CHECK - catches a wrong address, an
           unclocked block, or an access-control denial
           (isolation ch. 20) immediately, at probe, instead
           of as mysterious behaviour later.               */
        id = mydev_rd(d, MYDEV_REG_ID);
        if (id != MYDEV_EXPECTED_ID) {
                dev_err(d->dev, "bad ID 0x%08x (expected 0x%08x)\n",
                        id, MYDEV_EXPECTED_ID);
                return -ENODEV;
        }

        /* reset, then wait for it to clear */
        mydev_wr(d, MYDEV_REG_CTRL, MYDEV_CTRL_RESET);
        udelay(10);
        mydev_wr(d, MYDEV_REG_CTRL, 0);

        mydev_wr(d, MYDEV_REG_INT_EN, MYDEV_STATUS_IRQ);
        mydev_wr(d, MYDEV_REG_CTRL, MYDEV_CTRL_ENABLE);

        return 0;
}

static void mydev_hw_shutdown(struct mydev *d)
{
        mydev_wr(d, MYDEV_REG_INT_EN, 0);
        mydev_wr(d, MYDEV_REG_CTRL, 0);
}
```

```
/* ---- probe ------------------------------------------- */

static int mydev_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        struct mydev *d;
        int ret;

        d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;

        d->dev = dev;
        mutex_init(&d->lock);
        init_waitqueue_head(&d->wq);

        d->cfg = of_device_get_match_data(dev);
        if (!d->cfg)
                return -EINVAL;

        /* --- resources --- */

        d->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(d->base))
                return PTR_ERR(d->base);

        d->vdd = devm_regulator_get(dev, "vdd");
        if (IS_ERR(d->vdd))
                return dev_err_probe(dev, PTR_ERR(d->vdd),
                                     "no vdd supply\n");

        d->clk = devm_clk_get(dev, NULL);
        if (IS_ERR(d->clk))
                return dev_err_probe(dev, PTR_ERR(d->clk),
                                     "no clock\n");

        d->rst = devm_reset_control_get_optional_exclusive(dev,
                                                           NULL);
        if (IS_ERR(d->rst))
                return dev_err_probe(dev, PTR_ERR(d->rst),
                                     "no reset control\n");

        d->irq = platform_get_irq(pdev, 0);
        if (d->irq < 0)
                return d->irq;

        /* --- optional DT properties --- */

        ret = device_property_read_u32(dev, "vendor,fifo-depth",
                                       &d->fifo_depth);
        if (ret)
                d->fifo_depth = d->cfg->fifo_depth;

        /* --- power on, in order --- */

        ret = regulator_enable(d->vdd);
        if (ret)
                return dev_err_probe(dev, ret,
                                     "cannot enable vdd\n");

        ret = clk_prepare_enable(d->clk);
        if (ret) {
                dev_err(dev, "cannot enable clock: %d\n", ret);
```

```
                goto err_reg;
        }

        ret = reset_control_deassert(d->rst);
        if (ret) {
                dev_err(dev, "cannot deassert reset: %d\n", ret);
                goto err_clk;
        }

        ret = mydev_hw_init(d);
        if (ret)
                goto err_rst;

        /* --- interrupt: after the hardware is quiet --- */

        ret = devm_request_irq(dev, d->irq, mydev_isr,
                               IRQF_SHARED, dev_name(dev), d);
        if (ret) {
                dev_err(dev, "cannot request irq %d: %d\n",
                        d->irq, ret);
                goto err_hw;
        }

        platform_set_drvdata(pdev, d);

        dev_info(dev, "ready: %u channels, fifo %u\n",
                 d->cfg->nr_channels, d->fifo_depth);
        return 0;

err_hw:
        mydev_hw_shutdown(d);
err_rst:
        reset_control_assert(d->rst);
err_clk:
        clk_disable_unprepare(d->clk);
err_reg:
        regulator_disable(d->vdd);
        return ret;
}

/* ---- remove: the exact mirror -------------------------- */

static void mydev_remove(struct platform_device *pdev)
{
        struct mydev *d = platform_get_drvdata(pdev);

        mydev_hw_shutdown(d);           /* stop interrupts
                                           BEFORE devm frees
                                           the handler       */
        reset_control_assert(d->rst);
        clk_disable_unprepare(d->clk);
        regulator_disable(d->vdd);
}

/* ---- match --------------------------------------------- */

static const struct mydev_config mydev_v1_cfg = {
        .nr_channels = 4,
        .fifo_depth  = 16,
        .has_dma     = false,
};

static const struct mydev_config mydev_v2_cfg = {
```

```
        .nr_channels = 8,
        .fifo_depth  = 64,
        .has_dma     = true,
};

static const struct of_device_id mydev_of_match[] = {
        { .compatible = "vendor,my-device-v2",
          .data = &mydev_v2_cfg },
        { .compatible = "vendor,my-device",
          .data = &mydev_v1_cfg },
        { }
};
MODULE_DEVICE_TABLE(of, mydev_of_match);

static struct platform_driver mydev_driver = {
        .driver = {
                .name           = "my-device",
                .of_match_table = mydev_of_match,
                .pm             = pm_ptr(&mydev_pm_ops),
        },
        .probe  = mydev_probe,
        .remove = mydev_remove,
};
module_platform_driver(mydev_driver);

MODULE_DESCRIPTION("Vendor MyDevice driver");
MODULE_AUTHOR("Your Name <you@example.com>");
MODULE_LICENSE("GPL");
```

READ THE ERROR PATH

The `goto` ladder unwinds in exactly the reverse order of setup, each label undoing one step. This is idiomatic kernel C, not a code smell — it is the only structure that stays correct when you add a step in the middle. Chapter 26 shows how `devm_` removes most of it, and why some of it always remains.

Acquiring the common resources

```
/* --- MMIO ----------------------------------------------- */
        d->base = devm_platform_ioremap_resource(pdev, 0);
        /* by name, better when there are several: */
        d->base = devm_platform_ioremap_resource_byname(pdev,
                                                        "ctrl");
        /* the long form, if you need the resource itself: */
        struct resource *res;
        res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
        d->base = devm_ioremap_resource(dev, res);
        d->phys = res->start;

/* --- CLOCKS --------------------------------------------- */
        d->clk = devm_clk_get(dev, "core");     /* by name */
        d->clk = devm_clk_get(dev, NULL);       /* the only one */
        clk_prepare_enable(d->clk);
        clk_disable_unprepare(d->clk);

        /* the modern one-liner - gets AND enables, and
           disables automatically on unbind: */
        d->clk = devm_clk_get_enabled(dev, "core");

        /* several at once: */
        struct clk_bulk_data clks[] = {
                { .id = "core" }, { .id = "bus" },
        };
        devm_clk_bulk_get(dev, ARRAY_SIZE(clks), clks);
        clk_bulk_prepare_enable(ARRAY_SIZE(clks), clks);

        rate = clk_get_rate(d->clk);
        clk_set_rate(d->clk, 100000000);

/* --- OPTIONAL resources: use the _optional variants ------ */
        d->rst = devm_reset_control_get_optional_exclusive(dev,
                                                           NULL);
        /* returns NULL, not an error, if absent - and the
           reset_control_* calls accept NULL harmlessly. So
           optional resources need no `if` at every use.    */

        d->gpio = devm_gpiod_get_optional(dev, "reset",
                                          GPIOD_OUT_HIGH);
        gpiod_set_value_cansleep(d->gpio, 0);

/* --- REGULATORS ----------------------------------------- */
        d->vdd = devm_regulator_get(dev, "vdd");
        regulator_enable(d->vdd);
        /* several: */
        devm_regulator_bulk_get(dev, n, supplies);

/* --- DMA channels (chapter 24) -------------------------- */
        d->rxchan = dma_request_chan(dev, "rx");
        if (IS_ERR(d->rxchan))
                return dev_err_probe(dev, PTR_ERR(d->rxchan),
                                     "no rx dma\n");
```

Power-on ordering

```
Hardware has a sequence, and the datasheet's timing diagram is
the specification. The usual shape:

   1. REGULATORS on          (the device has power)
   2. wait the specified settling time
   3. CLOCKS enabled         (it has a clock to run on)
   4. RESET deasserted       (it starts)
   5. wait for it to be ready
   6. verify the ID register
   7. configure
   8. enable interrupts LAST

  AND EXACTLY THE REVERSE ON THE WAY DOWN.

  GETTING IT WRONG produces some of the most confusing
  symptoms in embedded work:

    reset released before the clock runs
        -> the block latches garbage; often works at room
           temperature and fails cold
    register written before power is stable
        -> the write is lost, silently
    clock enabled before the regulator
        -> in-rush current, a brownout, or a latch-up
    interrupts enabled before the handler is registered
        -> "nobody cared", or a crash in an unregistered
           handler
    no settling delay
        -> intermittent, temperature- and part-dependent

  READ THE POWER-UP SEQUENCE in the datasheet and implement it
  literally, including the delays. This is one of the few
  places in software where "it works on my board" is
  genuinely not evidence of correctness.
```

Testing it

```
# 1. Does it bind?
    dmesg | grep my-device
    ls -l /sys/bus/platform/drivers/my-device/

# 2. Are the registers reachable?
    dmesg      # the ID check tells you immediately

# 3. Does probe fail cleanly?
#    Force each failure and confirm no leak and no crash:
#      - remove the clock from the DT
#      - point reg at a wrong address
#      - make the ID check fail
#    Each should log one clear error and unwind fully.

# 4. Unbind and rebind, repeatedly, under KASAN:
    for i in $(seq 50); do
      echo 1c000000.my-device > .../my-device/unbind
      echo 1c000000.my-device > .../my-device/bind
    done
    dmesg | grep -i -e kasan -e warn -e bug

# 5. Load and unload as a module, 100 times (chapter 15).

# 6. Check for leaked resources after unbind:
    cat /proc/iomem | grep 1c000000     # should be gone
    cat /proc/interrupts | grep my-device
    cat /proc/vmallocinfo | grep my_probe

# ALL FIVE of these find real bugs in code that "works".
```

How it shows up

Symptom	Cause
Probe never called	Compatible mismatch, or `status = "disabled"`
ID register reads 0 or 0xFFFFFFFF	Clock off, in reset, wrong address, or an access-control denial
`devm_platform_ioremap_resource` fails with `-EBUSY`	Another driver already claimed the region — check `/proc/iomem`
Probe returns `-517` repeatedly	Deferred on a supplier — check `devices_deferred`
Works cold, fails warm (or the reverse)	Missing settling delay in the power-up sequence
Interrupt storm at probe	Interrupts enabled before the handler was registered
Crash on unbind	Hardware not quiesced before `devm` freed the IRQ
Resources leak across bind cycles	Something enabled by hand and not disabled in remove
Second probe fails after a successful unbind	State left in the hardware, or a regulator refcount imbalance
Works on v1 silicon, wrong on v2	Match data not used — values hard-coded
Optional resource treated as fatal	Used the non-`_optional` getter

Check yourself

- ✓ Say what makes a device a platform device, and when to use another bus type.
- ✓ Write a complete driver from memory: match table, probe, remove, module boilerplate.

- ✓ Explain why the error path is a `goto` ladder in reverse order.
- ✓ Acquire MMIO, clocks, resets, regulators and GPIOs, using the optional variants correctly.
- ✓ Explain what `devm_clk_get_enabled` saves you.
- ✓ Give the power-on ordering and four symptoms of getting it wrong.
- ✓ Explain why an ID-register check belongs in probe.
- ✓ Explain why interrupts are requested last and quiesced first.
- ✓ Use match data to support two silicon variants with one driver.
- ✓ Run all five tests and fix what they find.

Character devices

How a driver becomes a file in `/dev`. Major and minor numbers, the `file_operations` table, and the per-open state that makes concurrent users work — plus the much simpler misc-device shortcut that is right most of the time.

Major, minor, and the node

```
 $ ls -l /dev/mydev
 crw-rw----  1 root root  241, 0  Jan  1 00:00 /dev/mydev
 ^                         ^^^  ^
 |                         major minor
 character device

MAJOR   which driver
MINOR   which instance of it

A driver claims a range of (major, minor) pairs; the kernel
routes open() on a matching node to that driver.

dev_t packs both into one 32-bit value:
   MAJOR(dev), MINOR(dev), MKDEV(major, minor)

THREE WAYS TO GET THE NODE INTO /dev:

 1. udev/devtmpfs, automatically. The kernel creates the
    node when you call device_create(). THIS IS WHAT YOU
    WANT - no mknod, correct permissions, and it disappears
    on unbind.

 2. mknod by hand - for a minimal initramfs with no udev:
       mknod /dev/mydev c 241 0

 3. A misc device: major 10, a dynamically assigned minor,
    node created for you. Simplest of all.
```

The misc device: start here

```
/* If you need ONE device node and no special numbering, this
   is 15 lines instead of 60, and it is what most simple
   drivers should use.                                        */

#include <linux/miscdevice.h>

static const struct file_operations mydev_fops = {
        .owner          = THIS_MODULE,
        .open           = mydev_open,
        .release        = mydev_release,
        .read           = mydev_read,
        .write          = mydev_write,
        .unlocked_ioctl = mydev_ioctl,
        .compat_ioctl   = compat_ptr_ioctl,
        .poll           = mydev_poll,
        .llseek         = no_llseek,
};

/* in probe: */
        d->misc.minor  = MISC_DYNAMIC_MINOR;
        d->misc.name   = "mydev";
        d->misc.fops   = &mydev_fops;
        d->misc.parent = dev;          /* links it under the
                                          real device in sysfs */

        ret = misc_register(&d->misc);
        if (ret)
                return ret;
        /* -> /dev/mydev now exists */

/* in remove: */
        misc_deregister(&d->misc);

/* GETTING BACK TO YOUR DEVICE from a file operation:
   misc_open() sets file->private_data to the miscdevice, so
   container_of() recovers yours.                            */
static int mydev_open(struct inode *inode, struct file *file)
{
        struct miscdevice *m = file->private_data;
        struct mydev *d = container_of(m, struct mydev, misc);
        ...
}
```

The full cdev interface

```
/* Use this when you need SEVERAL minors - one node per
   channel, per port, per instance.                          */

#include <linux/cdev.h>
#include <linux/fs.h>
#include <linux/device.h>

#define MYDEV_NR_DEVS  4

static dev_t          mydev_devt;
static struct class  *mydev_class;

struct mydev {
        struct cdev     cdev;
        struct device  *cdev_dev;
        ...
};

static int __init mydev_init(void)
{
        int ret;

        /* 1. allocate a major and a range of minors */
        ret = alloc_chrdev_region(&mydev_devt, 0,
                                  MYDEV_NR_DEVS, "mydev");
        if (ret)
                return ret;

        /* 2. a class, so udev creates the nodes */
        mydev_class = class_create("mydev");
        if (IS_ERR(mydev_class)) {
                ret = PTR_ERR(mydev_class);
                goto err_region;
        }

        ret = platform_driver_register(&mydev_driver);
        if (ret)
                goto err_class;

        return 0;

err_class:
        class_destroy(mydev_class);
err_region:
        unregister_chrdev_region(mydev_devt, MYDEV_NR_DEVS);
        return ret;
}

/* per device, in probe: */
static int mydev_add_cdev(struct mydev *d, int idx)
{
        dev_t devt = MKDEV(MAJOR(mydev_devt),
                           MINOR(mydev_devt) + idx);
        int ret;

        cdev_init(&d->cdev, &mydev_fops);
        d->cdev.owner = THIS_MODULE;

        /* cdev_add PUBLISHES it - from this instant, open()
           can arrive. Everything else must already be
```

```
           initialised.                                          */
        ret = cdev_add(&d->cdev, devt, 1);
        if (ret)
                return ret;

        d->cdev_dev = device_create(mydev_class, d->dev, devt,
                                    d, "mydev%d", idx);
        if (IS_ERR(d->cdev_dev)) {
                cdev_del(&d->cdev);
                return PTR_ERR(d->cdev_dev);
        }
        return 0;
}

/* teardown, exact reverse: */
        device_destroy(mydev_class, devt);
        cdev_del(&d->cdev);
        ...
        class_destroy(mydev_class);
        unregister_chrdev_region(mydev_devt, MYDEV_NR_DEVS);

/* THE ORDERING RULE: cdev_add() is the point of no return.
   After it, a concurrent open() can run. Initialise
   everything the file operations touch BEFORE calling it, and
   call cdev_del() FIRST when tearing down.                 */
```

The file operations

```
static int mydev_open(struct inode *inode, struct file *file)
{
        struct mydev *d;
        struct mydev_session *s;

        /* recover the device from the inode - cdev is embedded
           in your struct                                      */
        d = container_of(inode->i_cdev, struct mydev, cdev);

        /* PER-OPEN state - one per file descriptor, so two
           processes do not share a position or a mode        */
        s = kzalloc(sizeof(*s), GFP_KERNEL);
        if (!s)
                return -ENOMEM;

        s->dev = d;
        file->private_data = s;

        /* exclusive access, if the hardware demands it: */
        if (test_and_set_bit(0, &d->in_use)) {
                kfree(s);
                return -EBUSY;
        }

        return 0;
}

static int mydev_release(struct inode *inode, struct file *file)
{
        struct mydev_session *s = file->private_data;

        clear_bit(0, &s->dev->in_use);
        kfree(s);
        return 0;
}

static ssize_t mydev_read(struct file *file, char __user *buf,
                          size_t count, loff_t *ppos)
{
        struct mydev_session *s = file->private_data;
        struct mydev *d = s->dev;
        u32 sample;
        int ret;

        if (count < sizeof(sample))
                return -EINVAL;

        if (file->f_flags & O_NONBLOCK) {
                if (!d->done)
                        return -EAGAIN;
        } else {
                ret = wait_event_interruptible(d->wq, d->done);
                if (ret)
                        return -ERESTARTSYS;
        }

        mutex_lock(&d->lock);
        sample = d->last_sample;
        d->done = false;
        mutex_unlock(&d->lock);
```

```
        if (copy_to_user(buf, &sample, sizeof(sample)))
                return -EFAULT;

        return sizeof(sample);          /* BYTES transferred */
}

static ssize_t mydev_write(struct file *file,
                           const char __user *buf,
                           size_t count, loff_t *ppos)
{
        struct mydev_session *s = file->private_data;
        u32 val;

        if (count != sizeof(val))
                return -EINVAL;

        if (copy_from_user(&val, buf, sizeof(val)))
                return -EFAULT;

        mydev_wr(s->dev, MYDEV_REG_DATA, val);
        return count;                   /* what you CONSUMED */
}

/* RETURN CONVENTIONS - getting these wrong breaks userspace
   in ways that look like your hardware is faulty:

     > 0     bytes transferred. May be LESS than requested;
             a correct caller loops.
     0       end of file. For a device this usually means
             "no more data, ever" - do not return it for
             "nothing right now" (that is -EAGAIN).
     -EAGAIN nothing available, and O_NONBLOCK was set
     -EINTR / -ERESTARTSYS   interrupted before any progress
     -EFAULT bad user pointer
     -EINVAL bad arguments

   AND: never return more than `count`. Userspace sized the
   buffer.                                                  */
```

poll: making select() work

```
static __poll_t mydev_poll(struct file *file,
                           struct poll_table_struct *wait)
{
        struct mydev_session *s = file->private_data;
        struct mydev *d = s->dev;
        __poll_t mask = 0;

        /* register interest - does NOT sleep. It adds this
           task to the wait queue so a later wake_up() will
           notify poll().                                    */
        poll_wait(file, &d->wq, wait);

        if (d->done)
                mask |= EPOLLIN | EPOLLRDNORM;   /* readable */

        if (mydev_space_available(d))
                mask |= EPOLLOUT | EPOLLWRNORM;  /* writable */

        if (d->error)
                mask |= EPOLLERR;

        return mask;
}

/* THE CONTRACT:
     · poll_wait() first, ALWAYS, before checking anything.
       Checking first and registering after loses events - the
       same lost-wakeup race as chapter 09.
     · poll() must not block.
     · every wake_up() that changes readability must be on the
       queue you passed to poll_wait().

   WITHOUT .poll, select(), poll() and epoll() report your fd
   as always-ready, and a userspace event loop spins at 100%
   CPU. This is not optional for any device a program will
   wait on.                                                  */
```

PER-DEVICE STATE VERSUS PER-OPEN STATE

The `struct mydev` is the hardware — one per physical device, shared by every user. The session in `file->private_data` is per file descriptor: read position, subscribed events, requested mode. Putting a per-user value in the device struct means two programs silently corrupt each other's view, and it is the most common design bug in character drivers.

Permissions and access control

```
/* By default your node is root-only. Real products need
   something better than "run everything as root".          */

/* 1. A udev rule - the normal answer:
     /etc/udev/rules.d/99-mydev.rules                       */
        KERNEL=="mydev", MODE="0660", GROUP="mydev"
        SUBSYSTEM=="mydev", TAG+="uaccess"   /* the logged-in
                                                user          */

/* 2. Set the mode from the driver, via the class: */
static char *mydev_devnode(const struct device *dev,
                           umode_t *mode)
{
        if (mode)
                *mode = 0660;
        return NULL;
}
        mydev_class->devnode = mydev_devnode;

/* 3. Capability checks inside a specific operation: */
        if (!capable(CAP_SYS_RAWIO))
                return -EPERM;
/* Use the most specific capability that fits, not
   CAP_SYS_ADMIN, which is effectively root.                */

/* 4. Check the open mode for write operations: */
        if (!(file->f_mode & FMODE_WRITE))
                return -EBADF;
/* Important for ioctls: a file opened O_RDONLY can still
   issue any ioctl unless you check. A "set configuration"
   ioctl reachable from a read-only fd is a real privilege
   bug.                                                     */
```

Testing from userspace

```
#include <fcntl.h>
#include <poll.h>
#include <stdint.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
        struct pollfd pfd;
        uint32_t sample;
        int fd;

        fd = open("/dev/mydev", O_RDWR);
        if (fd < 0) { perror("open"); return 1; }

        pfd.fd = fd;
        pfd.events = POLLIN;

        for (;;) {
                int ret = poll(&pfd, 1, 1000);
                if (ret < 0)  { perror("poll"); break; }
                if (ret == 0) { printf("timeout\n"); continue; }

                if (read(fd, &sample, sizeof(sample)) < 0) {
                        perror("read");
                        break;
                }
                printf("sample: %u\n", sample);
        }

        close(fd);
        return 0;
}
```

```
# Quick checks without writing a program:
    cat /dev/mydev | hexdump -C
    echo -ne '\x01\x00\x00\x00' > /dev/mydev
    dd if=/dev/mydev bs=4 count=10 | xxd

# Does poll work, or does it spin?
    strace -e poll,read ./mytest
    # A correct driver blocks in poll. A broken one returns
    # immediately, forever.

# Confirm the node and its numbers:
    ls -l /dev/mydev
    cat /proc/devices | grep mydev
    ls /sys/class/mydev/
```

How it shows up

Symptom	Cause
`/dev/mydev` does not appear	`device_create` not called, or no devtmpfs/udev
`open` returns `-ENODEV`	Node exists with numbers no driver claims — stale `mknod`

Symptom	Cause
`open` returns `-EPERM`	Node permissions — add a udev rule
Crash on the first `open`	`cdev_add` called before the state it uses was initialised
Two programs corrupt each other	Per-open state kept in the device struct
`select` / `poll` spins at 100% CPU	No `.poll` handler, or it does not call `poll_wait`
Events missed under load	`poll_wait` called after checking the condition
`read` returns 0 and the program exits	Returned EOF instead of `-EAGAIN`
Program hangs, unkillable	Uninterruptible wait (ch. 09)
`rmmod` says busy while an fd is open	Correct — `.owner = THIS_MODULE` working
Crash after `rmmod` with an fd open	`.owner` not set
Read-only fd can change configuration	`f_mode` not checked in the ioctl

Check yourself

- ✓ Explain major and minor numbers and the three ways a node appears in `/dev`.
- ✓ Register a misc device and recover your device struct from a file operation.
- ✓ Use the full cdev interface for four minors, with correct init and teardown order.
- ✓ Explain why `cdev_add` is the point of no return.
- ✓ Write `open`, `release`, `read` and `write` with correct return values.
- ✓ Distinguish per-device from per-open state and say what breaks if you confuse them.
- ✓ Write a `poll` handler and explain the ordering requirement.
- ✓ Handle `O_NONBLOCK` correctly.
- ✓ Set node permissions three different ways.
- ✓ Explain why an ioctl must check `f_mode`.

Designing an ioctl interface

read() and write() move bytes. An ioctl is everything else — configure, query, control — and because it becomes part of the user-facing ABI the moment it ships, it deserves more care than any other part of a driver.

Encoding a request number

```
#include <linux/ioctl.h>

/* NEVER a bare integer. The encoding macros pack DIRECTION,
   SIZE and TYPE into the number, which is what lets the
   kernel validate calls generically and what lets tools like
   strace decode them without your driver's headers.        */

#define MYDEV_MAGIC  'k'                /* pick something
                                           unused - see
                                           Documentation/
                                           userspace-api/
                                           ioctl/ioctl-number.rst */

struct mydev_config {
        __u32  sample_rate_hz;
        __u32  channel_mask;
        __u8   gain;
        __u8   _pad[3];                 /* EXPLICIT padding -
                                           see below           */
};

#define MYDEV_GET_CONFIG   _IOR(MYDEV_MAGIC, 1, struct mydev_config)
#define MYDEV_SET_CONFIG   _IOW(MYDEV_MAGIC, 2, struct mydev_config)
#define MYDEV_RESET        _IO (MYDEV_MAGIC, 3)
#define MYDEV_XFER         _IOWR(MYDEV_MAGIC, 4, struct mydev_xfer)

/* _IO    no data transferred
   _IOR   kernel -> user  (a "get")
   _IOW   user -> kernel  (a "set")
   _IOWR  both

   These are NAMES for humans; nothing enforces that _IOR
   actually only reads. But get it right anyway - strace and
   generic ioctl tracers use it to know which direction to
   dump.                                                    */
```

THE STRUCT IS A PERMANENT ABI

Once a binary in the field calls `MYDEV_SET_CONFIG` with this struct layout, that layout is frozen forever. Pad explicitly — never rely on the compiler's implicit padding, which can differ between 32-bit and 64-bit builds of the *same* struct. Use fixed-width types (`__u32`, not `int` or `unsigned long`) so the size does not change with the architecture. And leave room: reserved fields cost nothing today and save you from ever needing `MYDEV_SET_CONFIG_V2`.

The handler

```
static long mydev_ioctl(struct file *file, unsigned int cmd,
                        unsigned long arg)
{
        struct mydev_session *s = file->private_data;
        struct mydev *d = s->dev;
        void __user *uarg = (void __user *)arg;
        struct mydev_config cfg;
        int ret;

        switch (cmd) {
        case MYDEV_GET_CONFIG:
                mutex_lock(&d->lock);
                cfg = d->config;
                mutex_unlock(&d->lock);

                if (copy_to_user(uarg, &cfg, sizeof(cfg)))
                        return -EFAULT;
                return 0;

        case MYDEV_SET_CONFIG:
                if (!(file->f_mode & FMODE_WRITE))
                        return -EBADF;    /* a read-only fd may
                                             not configure it   */

                if (copy_from_user(&cfg, uarg, sizeof(cfg)))
                        return -EFAULT;

                /* VALIDATE EVERYTHING. This struct came from
                   userspace and userspace is the attacker.   */
                if (cfg.sample_rate_hz == 0 ||
                    cfg.sample_rate_hz > MYDEV_MAX_RATE)
                        return -EINVAL;
                if (cfg.channel_mask & ~MYDEV_VALID_CHANNELS)
                        return -EINVAL;
                if (cfg.gain > MYDEV_MAX_GAIN)
                        return -EINVAL;

                mutex_lock(&d->lock);
                ret = mydev_apply_config(d, &cfg);
                if (!ret)
                        d->config = cfg;
                mutex_unlock(&d->lock);
                return ret;

        case MYDEV_RESET:
                if (!(file->f_mode & FMODE_WRITE))
                        return -EBADF;
                return mydev_hw_reset(d);

        default:
                return -ENOTTY;          /* THE conventional
                                            "no such ioctl",
                                            not -EINVAL       */
        }
}
```

FIVE RULES, IN ORDER OF HOW OFTEN THEY ARE VIOLATED

1. **Validate every field.** Range, mask, size — the struct crossed a trust boundary.
2. **Check** `f_mode` for anything that changes state.
3. **Use** `copy_to_user` **/** `copy_from_user`**, never a direct dereference** of a user pointer (chapter 08).
4. **Return** `-ENOTTY` **for an unknown command**, so a program can probe for support.
5. **Never trust a size or count field to bound a copy** without an upper limit check first — that is a remotely-triggered `kmalloc(attacker_controlled)`.

Variable-length transfers

```
/* A common and commonly wrong pattern: a struct that carries a
   pointer and a count, for a buffer of user-chosen size.    */

struct mydev_xfer {
        __u64  buf;              /* NOT a pointer type - see
                                    below */
        __u32  len;
        __u32  flags;
};

static long mydev_xfer_ioctl(struct mydev *d, unsigned long arg)
{
        struct mydev_xfer x;
        void __user *ubuf;
        void *kbuf;
        int ret;

        if (copy_from_user(&x, (void __user *)arg, sizeof(x)))
                return -EFAULT;

        /* BOUND IT before allocating. Without this, a
           malicious len of 0xFFFFFFFF either fails an
           allocation harmlessly, or - on a kernel without
           kmalloc's internal limits - is a resource
           exhaustion vector. Always cap explicitly.        */
        if (x.len == 0 || x.len > MYDEV_MAX_XFER)
                return -EINVAL;

        kbuf = kmalloc(x.len, GFP_KERNEL);
        if (!kbuf)
                return -ENOMEM;

        ubuf = u64_to_user_ptr(x.buf);   /* the CORRECT cast -
                                            see below          */

        if (x.flags & MYDEV_XFER_WRITE) {
                if (copy_from_user(kbuf, ubuf, x.len)) {
                        ret = -EFAULT;
                        goto out;
                }
                ret = mydev_hw_write(d, kbuf, x.len);
        } else {
                ret = mydev_hw_read(d, kbuf, x.len);
                if (!ret && copy_to_user(ubuf, kbuf, x.len))
                        ret = -EFAULT;
        }

out:
        kfree(kbuf);
        return ret;
}

/* WHY __u64 buf, not a pointer field:

   A struct with a real pointer member has a DIFFERENT SIZE
   and LAYOUT on a 32-bit process running on a 64-bit kernel
   (compat mode) versus a native 64-bit process. __u64 is the
   same size everywhere, and u64_to_user_ptr() does the
   sign-correct, warning-free cast:
```

```
        #define u64_to_user_ptr(x) ( \
                (void __user *)(uintptr_t)(x) )

   This single choice is what saves you from writing a
   .compat_ioctl handler at all (see below).               */
```

32-bit compatibility

```
/* If a 32-bit binary can ever run on this kernel - and on
   arm64 that is common - a struct containing a real pointer
   or a `long` has a DIFFERENT layout when called from that
   32-bit process. Without a compat handler, SET_CONFIG from a
   32-bit program corrupts the kernel's read of the struct.  */

static const struct file_operations mydev_fops = {
        .unlocked_ioctl = mydev_ioctl,
        .compat_ioctl   = compat_ptr_ioctl,   /* the escape
                                                 hatch, below */
};

/* compat_ptr_ioctl() is a generic helper that works AS LONG
   AS your structs contain no raw pointers or `long` fields -
   exactly the discipline the __u64/__u32-only design above
   already gives you. It converts the compat pointer and calls
   your normal handler. No second implementation needed.

/* If you cannot avoid a pointer-shaped field (rare, and
   usually a sign to redesign), you need a REAL compat
   handler with a parallel struct and explicit translation.
   Avoid that path; it doubles your maintenance burden and is
   a frequent source of security bugs in real drivers.       */
```

Consider not using ioctl at all

Need	Better than a custom ioctl
A simple scalar setting	A sysfs attribute (chapter 22) — discoverable, self-documenting, one `cat` / `echo`
Streaming samples	`read()`, or a subsystem with its own buffering (IIO, V4L2)
Asynchronous events	`poll()` plus `read()`, or `uevent`
A sensor, ADC, DAC	The IIO subsystem — a whole ABI already exists
Temperature, voltage, fan	hwmon
A structured, extensible protocol	Netlink — versioned, introspectable, used by real subsystems
Something genuinely device-specific and complex	ioctl — the right tool, sometimes

An ioctl interface is invisible to generic tools, undiscoverable without your header, and — once shipped — permanent. Every one of the alternatives above is inspectable with tools that already exist. Reach for ioctl last, after checking whether a subsystem already solves your problem (chapter 17).

Shipping the header

```
/* include/uapi/linux/mydev.h - the UAPI header, shared
   VERBATIM between the kernel and any userspace program.   */

#ifndef _UAPI_LINUX_MYDEV_H
#define _UAPI_LINUX_MYDEV_H

#include <linux/types.h>
#include <linux/ioctl.h>

struct mydev_config {
        __u32 sample_rate_hz;
        __u32 channel_mask;
        __u8  gain;
        __u8  reserved[3];       /* MUST be zeroed by callers;
                                    reserved for future use   */
};

#define MYDEV_MAGIC        'k'
#define MYDEV_GET_CONFIG   _IOR(MYDEV_MAGIC, 1, struct mydev_config)
#define MYDEV_SET_CONFIG   _IOW(MYDEV_MAGIC, 2, struct mydev_config)
#define MYDEV_RESET        _IO (MYDEV_MAGIC, 3)

#endif

/* RULES for this file specifically:
     · __u8/__u16/__u32/__u64 only - never int, long, size_t
     · no kernel-only types (struct list_head, spinlock_t...)
     · no #ifdef __KERNEL__ tricks to hide fields - one struct,
       one meaning, for both sides
     · never remove or renumber an ioctl or a struct field
     · a NEW, incompatible need gets a NEW number - never
       redefine an existing one

   This header is literally installed into
   /usr/include/linux/mydev.h on end systems and copied into
   the musl/glibc UAPI headers. Programs compile against it
   for years after you have forgotten it exists.             */
```

Testing an ioctl

```
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <linux/mydev.h>

int main(void)
{
        int fd = open("/dev/mydev", O_RDWR);
        struct mydev_config cfg;

        if (ioctl(fd, MYDEV_GET_CONFIG, &cfg) < 0) {
                perror("GET_CONFIG");
                return 1;
        }
        printf("rate=%u mask=0x%x gain=%u\n",
               cfg.sample_rate_hz, cfg.channel_mask, cfg.gain);

        cfg.sample_rate_hz = 48000;
        if (ioctl(fd, MYDEV_SET_CONFIG, &cfg) < 0)
                perror("SET_CONFIG");        /* expect -EINVAL
                                                cases to work
                                                too - test them*/

        close(fd);
        return 0;
}
```

```
# strace decodes the direction/size/type automatically once it
# has your header or a matching ioctl.rst entry:
    strace -e ioctl ./mytest
      ioctl(3, MYDEV_GET_CONFIG or _IOC(_IOC_READ, 0x6b, 1, 8), ...) = 0

# NEGATIVE TESTS - the ones that actually find bugs:
    · call with a NULL pointer                -> -EFAULT
    · call with an unmapped pointer           -> -EFAULT
    · call an unknown command number          -> -ENOTTY
    · SET_CONFIG on an O_RDONLY fd             -> -EBADF
    · out-of-range fields                     -> -EINVAL
    · a huge `len` in a variable transfer     -> -EINVAL,
                                                  not a hang
    · call from a 32-bit binary, if supported  -> matches
                                                  64-bit result

# Fuzz it - genuinely effective on ioctl handlers:
    syzkaller, or a simple harness with
    CONFIG_KCOV + CONFIG_KASAN, feeding random cmd/arg pairs
    at your fd. This finds the missing validation checks that
    manual testing never will.                          */
```

How it shows up

Symptom	Cause
Works on x86 host, corrupts on the arm64 target	Implicit struct padding differs — pad explicitly

Symptom	Cause
Works as a 64-bit binary, garbage from a 32-bit one	Missing `.compat_ioctl`, or a pointer/`long` field in the struct
Kernel crash from a userspace ioctl call	Unvalidated length used to size an allocation or copy — a real vulnerability
`-EFAULT` on every call	Direct pointer dereference instead of `copy_to/from_user`
Read-only fd can reconfigure the device	`f_mode` not checked
Unknown ioctl returns `-EINVAL`	Should be `-ENOTTY` — breaks capability probing
A future kernel breaks an old binary	A struct field was reused or renumbered instead of extended
`strace` shows garbage for the ioctl name	Number not registered in `ioctl-number.rst`, or magic collides with another driver
Huge allocation from a small request	Length field trusted without an upper bound

Check yourself

- ✓ Encode a request number with `_IOR` / `_IOW` / `_IOWR` and explain each field.
- ✓ Explain why the struct is a permanent ABI and what explicit padding buys you.
- ✓ Write an ioctl handler that validates every field of an incoming struct.
- ✓ Explain why an unknown command returns `-ENOTTY`, not `-EINVAL`.
- ✓ Design a variable-length transfer struct that needs no `.compat_ioctl` of its own.
- ✓ Explain why a pointer field breaks 32-bit compatibility and how `__u64` avoids it.
- ✓ List five subsystems to check before writing a custom ioctl interface.
- ✓ Write a correct UAPI header and state its permanence rules.
- ✓ List six negative tests every ioctl handler should pass.

sysfs, debugfs and procfs

Three filesystems that expose kernel state as files. They look interchangeable and are not: one is a permanent ABI, one is a debugging free-for-all, and one you should not add to at all.

Which is which

	sysfs	debugfs	procfs
Mounted at	`/sys`	`/sys/kernel/debug`	`/proc`
Stability	**Permanent ABI**	**None** — change freely	Permanent for what exists
Rule	**One value per file**	Anything you like	Historical mess
Available in production	Yes	Usually not mounted	Yes
Documented in	`Documentation/ABI/`	Nowhere required	—
Use for	Configuration and status userspace depends on	Development, diagnostics, dumps	**Do not add new files**

THE DECISION

If a program will depend on it, it goes in **sysfs**, one value per file, with an entry in `Documentation/ABI/`. If it is for you and your colleagues while debugging, it goes in **debugfs**, where you can restructure it tomorrow. Nothing new goes in **procfs** — it predates both and is reserved for process information.

sysfs attributes

```
#include <linux/sysfs.h>
#include <linux/device.h>

/* --- a read-only attribute ------------------------------- */
static ssize_t temperature_show(struct device *dev,
                                struct device_attribute *attr,
                                char *buf)
{
        struct mydev *d = dev_get_drvdata(dev);
        int milli_c;

        milli_c = mydev_read_temp(d);

        /* sysfs_emit() is the correct helper: it knows the
           buffer is one PAGE and bounds the write for you.
           Use it, not sprintf.                              */
        return sysfs_emit(buf, "%d\n", milli_c);
}
static DEVICE_ATTR_RO(temperature);
        /* creates: struct device_attribute dev_attr_temperature
           with .show = temperature_show, mode 0444          */

/* --- a read/write attribute ------------------------------ */
static ssize_t gain_show(struct device *dev,
                         struct device_attribute *attr, char *buf)
{
        struct mydev *d = dev_get_drvdata(dev);

        return sysfs_emit(buf, "%u\n", d->gain);
}

static ssize_t gain_store(struct device *dev,
                          struct device_attribute *attr,
                          const char *buf, size_t count)
{
        struct mydev *d = dev_get_drvdata(dev);
        unsigned int val;
        int ret;

        ret = kstrtouint(buf, 0, &val);   /* base 0: accepts
                                             decimal AND 0x   */
        if (ret)
                return ret;

        if (val > MYDEV_MAX_GAIN)
                return -EINVAL;

        mutex_lock(&d->lock);
        ret = mydev_set_gain(d, val);
        mutex_unlock(&d->lock);
        if (ret)
                return ret;

        return count;         /* RETURN count ON SUCCESS.
                                 Returning 0 makes the writing
                                 program loop forever, because
                                 it thinks nothing was
                                 consumed.                  */
}
static DEVICE_ATTR_RW(gain);
```

```
/* --- grouping and registering ---------------------------- */
static struct attribute *mydev_attrs[] = {
        &dev_attr_temperature.attr,
        &dev_attr_gain.attr,
        NULL,                           /* sentinel */
};
ATTRIBUTE_GROUPS(mydev);        /* -> mydev_groups */

static struct platform_driver mydev_driver = {
        .driver = {
                .name = "my-device",
                .dev_groups = mydev_groups,   /* <-- the core
                                                 creates and
                                                 removes them
                                                 around probe */
                ...
        },
        ...
};

/* USING dev_groups is much better than device_create_file()
   in probe: the attributes exist BEFORE probe returns and are
   removed AFTER remove, so there is no window where userspace
   can see a file backed by a half-initialised device - and no
   cleanup for you to forget.                                  */
```

The sysfs rules

```
/* These are enforced in review, and for good reasons: */

   1. ONE VALUE PER FILE.
        good:  /sys/.../temperature       "42000\n"
        bad:   /sys/.../status            "temp=42 gain=3 ok"
      A multi-value file cannot be parsed reliably, cannot be
      extended without breaking parsers, and cannot be watched
      with poll() per-value.

   2. TEXT, human readable, with a trailing newline.

   3. USE THE STANDARD UNITS. The sysfs convention is
      milli-units: millivolts, millidegrees, milliamps. Not
      degrees, not raw ADC counts. Userspace tools assume it.

   4. NO ACTIONS WITH SIDE EFFECTS beyond the obvious. A file
      called `gain` sets the gain. A file that reprograms the
      firmware when you write "1" to it is a surprise and
      belongs elsewhere.

   5. DOCUMENT IT in Documentation/ABI/testing/sysfs-...
      before it becomes stable:

        What:           /sys/bus/platform/devices/*/gain
        Date:           March 2026
        KernelVersion:  6.14
        Contact:        you@example.com
        Description:
                        Amplifier gain, 0-15. Writing a value
                        outside that range returns -EINVAL.

   6. RETURN count FROM store(), or a negative error. Never 0.

   7. THE BUFFER IN show() IS ONE PAGE. sysfs_emit() enforces
      it; sprintf does not.                                  */
```

Making a sysfs file pollable

```
/* An attribute can notify userspace of a change, so a program
   can wait on it rather than polling in a loop.            */

        /* when the value changes, in your driver: */
        sysfs_notify(&dev->kobj, NULL, "alarm");

/* userspace: */
        fd = open("/sys/.../alarm", O_RDONLY);
        read(fd, buf, sizeof(buf));      /* read it once FIRST */

        pfd.fd = fd;
        pfd.events = POLLPRI | POLLERR;  /* NOT POLLIN - sysfs
                                            uses POLLPRI      */
        poll(&pfd, 1, -1);

        lseek(fd, 0, SEEK_SET);          /* rewind */
        read(fd, buf, sizeof(buf));      /* the new value */

/* THE THREE THINGS PEOPLE GET WRONG:
     · waiting on POLLIN instead of POLLPRI
     · not reading once before the first poll (the initial
       read arms the notification)
     · not lseek()ing back to 0 before re-reading           */
```

debugfs

```
#include <linux/debugfs.h>

/* --- the one-liners, for simple values ---------------------- */
static int __init mydev_debugfs_init(struct mydev *d)
{
        d->dbg = debugfs_create_dir("mydev", NULL);
        /* NOTE: debugfs_create_dir NEVER returns NULL on
           failure - it returns an ERR_PTR, and every other
           debugfs function accepts an ERR_PTR harmlessly. So
           you do NOT need to check it. This is deliberate:
           debugfs failures must never break a driver.      */

        debugfs_create_u32("irq_count",  0444, d->dbg,
                           &d->irq_count);
        debugfs_create_u32("error_count", 0444, d->dbg,
                           &d->error_count);
        debugfs_create_bool("verbose", 0644, d->dbg,
                            &d->verbose);
        debugfs_create_x32("last_status", 0444, d->dbg,
                           &d->last_status);   /* hex */

        /* a raw memory blob */
        d->blob.data = d->fw_image;
        d->blob.size = d->fw_size;
        debugfs_create_blob("firmware", 0444, d->dbg, &d->blob);

        return 0;
}
        debugfs_remove_recursive(d->dbg);      /* in remove */

/* --- a custom file, for anything structured ----------------- */
static int mydev_regs_show(struct seq_file *s, void *unused)
{
        struct mydev *d = s->private;
        int i;

        seq_printf(s, "%-16s %s\n", "REGISTER", "VALUE");
        for (i = 0; i < ARRAY_SIZE(mydev_reg_names); i++)
                seq_printf(s, "%-16s 0x%08x\n",
                           mydev_reg_names[i],
                           mydev_rd(d, i * 4));
        return 0;
}
DEFINE_SHOW_ATTRIBUTE(mydev_regs);
        /* generates the whole file_operations for you */

        debugfs_create_file("registers", 0444, d->dbg, d,
                            &mydev_regs_fops);

/* seq_file handles pagination for you - output larger than a
   page just works, unlike sysfs. That is why register dumps,
   tables and statistics belong here.                       */

/* MOUNT IT: */
        mount -t debugfs none /sys/kernel/debug
        cat /sys/kernel/debug/mydev/registers
```

> **DEBUGFS IS ROOT-ONLY AND OFTEN ABSENT**
> Production kernels frequently do not enable `CONFIG_DEBUG_FS`, and hardened ones do not mount it. So a driver must **work correctly when debugfs does not exist** — never put required functionality there, and never make a debugfs failure fatal. The API is designed so that ignoring every return value is the correct thing to do.

Dynamic debug

```
/* The best debug-printing mechanism the kernel has: messages
   compiled in but disabled, switchable at runtime with zero
   cost when off (they are patched out with jump labels).   */

        CONFIG_DYNAMIC_DEBUG=y

/* In your driver, just use these: */
        pr_debug("value is %d\n", val);
        dev_dbg(dev, "channel %u ready\n", ch);

/* Then at runtime: */
    # everything in one file
    echo 'file mydev.c +p' > /sys/kernel/debug/dynamic_debug/control

    # one function
    echo 'func mydev_probe +p' > .../control

    # one module
    echo 'module mydev +p' > .../control

    # a line range
    echo 'file mydev.c line 100-200 +p' > .../control

    # turn it off again
    echo 'module mydev -p' > .../control

    # add the module, function, line and thread id to each
    # message:
    echo 'module mydev +pmfl' > .../control

    # see what is available and what is enabled:
    cat /sys/kernel/debug/dynamic_debug/control | grep mydev

/* AND AT BOOT, before debugfs is even mounted: */
        dyndbg="module mydev +p"          # kernel command line
        mydev.dyndbg="+p"                 # per-module form

/* THIS REPLACES the #ifdef DEBUG / custom-verbosity-flag
   pattern entirely. Use dev_dbg() liberally in your driver -
   it costs nothing when off, and it means a field problem can
   be diagnosed without a rebuild.                          */
```

Trace events

```
/* For anything in a hot path, a trace event beats a printk:
   it is nearly free when disabled, has a binary format, and
   integrates with ftrace, perf and trace-cmd.             */

/* include/trace/events/mydev.h */
#undef TRACE_SYSTEM
#define TRACE_SYSTEM mydev

#if !defined(_TRACE_MYDEV_H) || defined(TRACE_HEADER_MULTI_READ)
#define _TRACE_MYDEV_H

#include <linux/tracepoint.h>

TRACE_EVENT(mydev_xfer,
        TP_PROTO(unsigned int chan, unsigned int len, int ret),
        TP_ARGS(chan, len, ret),
        TP_STRUCT__entry(
                __field(unsigned int, chan)
                __field(unsigned int, len)
                __field(int,          ret)
        ),
        TP_fast_assign(
                __entry->chan = chan;
                __entry->len  = len;
                __entry->ret  = ret;
        ),
        TP_printk("chan=%u len=%u ret=%d",
                  __entry->chan, __entry->len, __entry->ret)
);

#endif
#include <trace/define_trace.h>

/* in the driver: */
        trace_mydev_xfer(chan, len, ret);

/* use it: */
    echo 1 > /sys/kernel/tracing/events/mydev/mydev_xfer/enable
    cat /sys/kernel/tracing/trace_pipe
    # or:
    trace-cmd record -e mydev
    trace-cmd report
```

How it shows up

Symptom	Cause
Writing to a sysfs file hangs the program	`store()` returned 0 instead of `count`
sysfs output truncated	Used `sprintf` past one page — use `sysfs_emit`
Attribute exists before the device works	Created in probe instead of via `dev_groups`
Attribute left behind after unbind	Manual `device_create_file` without a matching remove
`poll()` on a sysfs file never fires	Waiting on POLLIN instead of POLLPRI, or no initial read
Userspace tool cannot parse your value	Multiple values in one file, or non-standard units
debugfs files missing in production	`CONFIG_DEBUG_FS` off or not mounted — as expected

Symptom	Cause
Driver fails to probe when debugfs is absent	Checked a debugfs return value and treated it as fatal
Large debugfs output truncated	Not using `seq_file`
`dev_dbg` prints nothing	Dynamic debug not enabled for that file/module
Kernel upgrade breaks a userspace tool	A sysfs attribute was renamed or its format changed — it was an ABI

Check yourself

- ✓ Say which of the three filesystems a given piece of information belongs in, and why.
- ✓ Write a read-only and a read/write sysfs attribute correctly.
- ✓ Explain why `store()` must return `count`.
- ✓ Explain the advantage of `dev_groups` over creating files in probe.
- ✓ List the seven sysfs rules.
- ✓ Make an attribute pollable and write the userspace side correctly.
- ✓ Add debugfs counters and a `seq_file` register dump.
- ✓ Explain why debugfs return values need not be checked.
- ✓ Use dynamic debug to enable one function's messages at runtime and at boot.
- ✓ Add a trace event and say when to prefer it over `dev_dbg`.

MMIO access and regmap

Touching hardware registers correctly: the accessor functions, the ordering guarantees they carry, why a plain pointer dereference is wrong, and the framework that removes most of the boilerplate along with a class of bugs.

The accessors

```
#include <linux/io.h>

        u8  readb(const volatile void __iomem *addr);
        u16 readw(...);         /* 16-bit */
        u32 readl(...);         /* 32-bit - the common one */
        u64 readq(...);         /* 64-bit */

        void writeb(u8  value, volatile void __iomem *addr);
        void writew(u16 value, ...);
        void writel(u32 value, ...);
        void writeq(u64 value, ...);

/* NOTE THE ARGUMENT ORDER: writel(VALUE, ADDRESS). It is the
   opposite of what most people expect and of memcpy. Getting
   it backwards compiles fine and writes an address into a
   register.                                                  */

/* WHAT readl/writel GUARANTEE:
     · exactly one access of exactly that width. The compiler
       cannot split, merge, reorder, or elide it.
     · ordering with respect to OTHER readl/writel calls -
       they carry the barriers needed so accesses reach the
       device in program order.
     · correct byte order: they assume LITTLE-ENDIAN device
       registers and convert if the CPU is big-endian.

/* WHY A PLAIN DEREFERENCE IS WRONG: */
        val = *(volatile u32 *)base;      /* NO */
   volatile stops the compiler reordering, but does NOTHING
   about the CPU's memory ordering, nothing about access
   width (the compiler may use a different instruction), and
   nothing about endianness. On arm64 it often appears to
   work, which is what makes it dangerous - it fails on
   another core, another compiler, or under load.

/* THE RELAXED VARIANTS - no ordering barriers: */
        readl_relaxed(), writel_relaxed()

   Faster, because they omit the barrier. Use them ONLY when
   you know ordering does not matter - typically a burst of
   writes to a FIFO where only the last one must be ordered:

        for (i = 0; i < n; i++)
                writel_relaxed(data[i], base + FIFO);
        writel(GO, base + CTRL);    /* ordered: this one
                                       carries the barrier   */

   THE RULE: use readl/writel by default. Reach for _relaxed
   only in a measured hot path, and then make sure the last
   access in the sequence is ordered.                       */
```

Ordering, DMA and the barriers that matter

```
Three different ordering questions, three different tools:

  1. CPU-to-DEVICE ordering
        Does the device see my writes in the order I made
        them?
     -> readl()/writel() handle this. Device memory
        (nGnRE, ARM ch. 11) is not reordered by the CPU,
        and the accessors add the compiler barriers.

  2. MEMORY-to-DEVICE ordering
        I filled a DMA buffer in RAM, then wrote the "GO"
        register. Will the device see the DATA before it
        acts on GO?
     -> NOT AUTOMATIC. Normal memory writes and Device
        writes are ordered differently. You need:

           fill_dma_buffer(buf);
           dma_wmb();                /* buffer visible to
                                        the device FIRST   */
           writel(GO, base + CTRL);

        And on the receive side:

           if (readl(base + STATUS) & DONE) {
                   dma_rmb();        /* do not read the
                                        buffer before the
                                        status read        */
                   consume(buf);
           }

  3. CPU-to-CPU ordering
        -> smp_wmb(), smp_rmb(), smp_mb() (chapter 12).
        NEVER substitute these for the DMA ones: smp_* compile
        to nothing on a uniprocessor kernel, and a DMA engine
        is a second agent regardless of CPU count.

  THE MISTAKE that costs the most time: using smp_wmb() where
  dma_wmb() is needed. It works on your 4-core test board and
  silently corrupts on a single-core variant, or under a
  different compiler.
```

Waiting for hardware

```
#include <linux/iopoll.h>

/* NEVER an unbounded spin: */
        while (readl(base + STATUS) & BUSY)     /* NO */
                ;
/* If the device wedges - and it will - this hangs a CPU
   forever with no diagnostic.                              */

/* SLEEPING VERSION - process context: */
        u32 val;
        int ret;

        ret = readl_poll_timeout(base + REG_STATUS,  /* addr   */
                                 val,                /* var    */
                                 !(val & BUSY),      /* cond   */
                                 10,          /* poll every 10us*/
                                 1000);       /* give up at 1ms */
        if (ret)
                return dev_err_probe(dev, ret, "timed out\n");

/* ATOMIC VERSION - interrupt context, holding a spinlock: */
        ret = readl_poll_timeout_atomic(base + REG_STATUS, val,
                                        !(val & BUSY), 1, 100);

/* PICKING THE NUMBERS:
     sleep_us   how long between polls. Too small and you burn
                CPU; too large and you add latency. Roughly
                one tenth of the expected completion time.
     timeout_us the datasheet's worst case, times a safety
                factor of two or three. NEVER "a number that
                works on my board".

   AND ALWAYS handle the timeout. A driver that logs
   "timed out" and returns an error is debuggable; one that
   ignores it produces corrupt data with no clue why.       */
```

regmap: the framework

```
/* regmap abstracts "a set of registers" over MMIO, I2C, SPI or
   anything else, and adds caching, locking and debugfs for
   free. Use it unless you have a reason not to.            */

#include <linux/regmap.h>

static const struct regmap_config mydev_regmap_config = {
        .reg_bits      = 32,       /* register address width */
        .val_bits      = 32,       /* register value width   */
        .reg_stride    = 4,        /* registers are 4 apart  */
        .max_register  = 0xff,

        .cache_type    = REGCACHE_MAPLE,   /* or RBTREE, FLAT,
                                              or NONE         */
        .volatile_reg  = mydev_volatile_reg,
        .writeable_reg = mydev_writeable_reg,
        .readable_reg  = mydev_readable_reg,
        .precious_reg  = mydev_precious_reg,
};

/* WHICH REGISTERS ARE WHICH - this is the important part: */

static bool mydev_volatile_reg(struct device *dev, unsigned int reg)
{
        /* VOLATILE = the hardware can change it by itself.
           Never cache these; always read the hardware.     */
        switch (reg) {
        case MYDEV_REG_STATUS:
        case MYDEV_REG_FIFO_DATA:
        case MYDEV_REG_IRQ_STATUS:
                return true;
        default:
                return false;
        }
}

static bool mydev_precious_reg(struct device *dev, unsigned int reg)
{
        /* PRECIOUS = reading it has a SIDE EFFECT, such as
           clearing the bits or popping a FIFO. regmap must
           never read these speculatively - not for a cache
           fill, not for the debugfs dump.                  */
        return reg == MYDEV_REG_IRQ_STATUS ||
               reg == MYDEV_REG_FIFO_DATA;
}

/* in probe: */
        d->regmap = devm_regmap_init_mmio(dev, d->base,
                                          &mydev_regmap_config);
        if (IS_ERR(d->regmap))
                return PTR_ERR(d->regmap);

/* USING IT: */
        unsigned int val;

        ret = regmap_read(d->regmap, MYDEV_REG_STATUS, &val);
        ret = regmap_write(d->regmap, MYDEV_REG_CTRL, 0x1);

        /* read-modify-write, ATOMICALLY with respect to other
           regmap users - this is the big win over hand-rolled
```

```
   MMIO, where the RMW race is easy to write        */
ret = regmap_update_bits(d->regmap, MYDEV_REG_CTRL,
                         MYDEV_CTRL_ENABLE,   /* mask */
                         MYDEV_CTRL_ENABLE);  /* val  */

/* bulk, for FIFOs and contiguous blocks */
ret = regmap_bulk_read(d->regmap, MYDEV_REG_DATA,
                       buf, count);

/* poll, the regmap equivalent of readl_poll_timeout */
ret = regmap_read_poll_timeout(d->regmap,
                               MYDEV_REG_STATUS, val,
                               !(val & BUSY), 10, 1000);

/* fields, when you work with one bitfield repeatedly */
static const struct reg_field gain_field =
        REG_FIELD(MYDEV_REG_CTRL, 4, 7);
d->gain_rf = devm_regmap_field_alloc(dev, d->regmap,
                                     gain_field);
regmap_field_write(d->gain_rf, 3);   /* shifts and
                                        masks for you */
```

WHAT REGMAP BUYS YOU

- **Atomic read-modify-write** — the single most common MMIO race, eliminated.
- **Locking** against concurrent access, without you writing it.
- **A register cache**, so a chatty I2C or SPI device is not re-read constantly, and so suspend/resume can restore state with `regcache_sync()`.
- **Bus independence** — the same driver logic over MMIO, I2C and SPI variants of one chip.
- **A free debugfs register dump** at `/sys/kernel/debug/regmap/<dev>/registers`.
- **Endianness and width handling** declared once instead of at every call site.

Cached registers and suspend

```
/* The cache is what makes regmap worth using on a slow bus,
   and it is also what makes resume simple.                */

static int mydev_suspend(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);

        /* stop touching the hardware, and remember everything
           by marking the cache dirty                       */
        regcache_cache_only(d->regmap, true);
        regcache_mark_dirty(d->regmap);

        clk_disable_unprepare(d->clk);
        return 0;
}

static int mydev_resume(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);
        int ret;

        ret = clk_prepare_enable(d->clk);
        if (ret)
                return ret;

        regcache_cache_only(d->regmap, false);

        /* write every cached value back to hardware that just
           came up from reset - the whole register state,
           restored in one call                             */
        ret = regcache_sync(d->regmap);
        if (ret)
                dev_err(dev, "cache sync failed: %d\n", ret);

        return ret;
}

/* Without regmap you would write, and maintain, a shadow copy
   of every register plus a restore function - and get it
   subtly wrong when someone adds a register.

/* THE CACHE TYPES:
     REGCACHE_NONE    no caching. Correct for MMIO where reads
                      are cheap and you want the truth.
     REGCACHE_FLAT    an array. Fast, uses max_register words
                      of memory. Good for small maps.
     REGCACHE_MAPLE   the modern sparse cache. Good default.
     REGCACHE_RBTREE  the older sparse cache.

   For a plain MMIO device, NONE is often right - the cache
   exists to avoid slow bus traffic, and MMIO is not slow.
   But even with NONE you keep the locking, the atomic RMW and
   the debugfs dump.                                        */
```

Register definition style

```
/* Define registers and their fields together, with the bit
   positions expressed structurally rather than as magic
   numbers.                                                  */

#include <linux/bitfield.h>
#include <linux/bits.h>

#define MYDEV_REG_CTRL          0x04
#define  MYDEV_CTRL_ENABLE      BIT(0)
#define  MYDEV_CTRL_RESET       BIT(1)
#define  MYDEV_CTRL_MODE        GENMASK(5, 4)   /* bits 5:4 */
#define  MYDEV_CTRL_GAIN        GENMASK(11, 8)  /* bits 11:8 */

/* FIELD_PREP shifts a value INTO position;
   FIELD_GET extracts one OUT. Both check at compile time
   that the value fits the mask.                             */

        u32 ctrl = MYDEV_CTRL_ENABLE
                 | FIELD_PREP(MYDEV_CTRL_MODE, MODE_BURST)
                 | FIELD_PREP(MYDEV_CTRL_GAIN, 7);
        writel(ctrl, base + MYDEV_REG_CTRL);

        u32 gain = FIELD_GET(MYDEV_CTRL_GAIN,
                             readl(base + MYDEV_REG_CTRL));

/* versus the hand-rolled version, which has two bugs waiting
   to happen (a wrong shift, and an unmasked value that
   overflows into the neighbouring field):                   */
        ctrl = 1 | (MODE_BURST << 4) | (7 << 8);   /* avoid */

/* A NAMING CONVENTION that pays for itself:
     MYDEV_REG_xxx     a register offset
     MYDEV_xxx_yyy     a field within register xxx
   and indent the field defines under their register, as
   above, so the structure is visible at a glance.         */
```

Debugging register access

```
# regmap's free debugfs interface:
    ls /sys/kernel/debug/regmap/1c000000.my-device/
      access  name  range  registers  rbtree
    cat /sys/kernel/debug/regmap/*/registers
      00: 4d594445
      04: 00000701
      08: 00000000

# From userspace, without any driver support:
    devmem 0x1c000004                # read
    devmem 0x1c000004 32 0x1         # write
    busybox devmem 0x1c000004

# Trace every regmap access - extremely useful:
    echo 1 > /sys/kernel/tracing/events/regmap/enable
    cat /sys/kernel/tracing/trace_pipe
      regmap_reg_write: 1c000000.my-device reg=4 val=1
      regmap_reg_read:  1c000000.my-device reg=8 val=0

# This is the regmap feature that most often justifies using
# it: a complete, timestamped log of every register access,
# with no code changes. Reproducing a bug once with this
# enabled usually shows you the wrong write immediately.
```

How it shows up

Symptom	Cause
Register writes have no effect	Arguments reversed — `writel(value, addr)`
Reads return 0 or 0xFFFFFFFF	Clock off, in reset, wrong address, or access denied (isolation ch. 20)
Works on one compiler, breaks on another	Direct dereference instead of `readl` / `writel`
DMA reads stale data	Missing `dma_wmb()` before the doorbell write
Corruption on a single-core build only	`smp_wmb()` used where `dma_wmb()` was needed
CPU hangs waiting for the device	Unbounded polling loop — use `readl_poll_timeout`
Two threads clobber a shared register	Non-atomic read-modify-write — use `regmap_update_bits`
A FIFO empties by itself	Regmap read it speculatively — mark it `precious`
Status register never changes	Cached — mark it `volatile`
Device is misconfigured after resume	No `regcache_sync()`, or the cache was not marked dirty
A field write corrupts the neighbouring field	Hand-rolled shift without masking — use `FIELD_PREP`

Check yourself

- ✓ Write the accessors with the correct argument order and say what they guarantee.
- ✓ Explain why `volatile` plus a cast is not equivalent.
- ✓ Say when `_relaxed` is appropriate and what must still be ordered.
- ✓ Distinguish the three ordering questions and pick the right barrier for each.
- ✓ Explain why `smp_wmb()` is wrong for DMA ordering.

- ✓ Replace a spin loop with `readl_poll_timeout` and justify the timings.
- ✓ Write a `regmap_config` with correct volatile and precious predicates.
- ✓ Explain what `precious` prevents and give an example register.
- ✓ Use the register cache to implement suspend and resume.
- ✓ Define a register with `BIT` / `GENMASK` and use `FIELD_PREP` / `FIELD_GET`.

The DMA API

Giving a device direct access to memory, correctly. Three address spaces, two allocation styles, cache coherency, and the SMMU underneath it all — this is where the isolation track's theory becomes code you write.

Three kinds of address

```
 VIRTUAL ADDRESS      what the CPU uses. void *.
      |               Only meaningful to the kernel.
      |
 PHYSICAL ADDRESS     what is on the memory bus.
      |               phys_addr_t.
      |
 DMA ADDRESS          what the DEVICE must be programmed
                      with. dma_addr_t.

THEY ARE NOT THE SAME NUMBER, and assuming they are is the
defining DMA bug.

The DMA address differs from the physical address when:
   · an SMMU/IOMMU is translating (isolation ch. 03-05) -
     then the DMA address is an IOVA, an entirely made-up
     number that exists only in the SMMU's page tables
   · a bus applies an offset (dma-ranges in the DT)
   · bounce buffering (SWIOTLB) is in play because the
     device cannot address the memory you allocated
   · the device sits behind a bridge that remaps

SO: NEVER program a device with virt_to_phys(). Always with
the dma_addr_t the DMA API gave you. The API is what knows
about the SMMU, the offsets and the bounce buffers - and it
is what SETS UP the SMMU mapping, so a hand-computed
physical address is not merely wrong, it is unmapped and
will fault (isolation ch. 06).
```

Step one: declare what the device can address

```
/* Before any allocation or mapping, in probe: */

        ret = dma_set_mask_and_coherent(dev, DMA_BIT_MASK(40));
        if (ret) {
                dev_err(dev, "no suitable DMA mask\n");
                return ret;
        }

/* WHAT THIS DECLARES: "this device's address register is 40
   bits wide". The kernel then:
     · allocates only from memory the device can reach
     · sets up bounce buffers if you later map something above
       that limit
     · configures the SMMU's IOVA range accordingly

   GET IT WRONG — claim 64 bits on a device with a 32-bit
   address register — and the top bits are silently dropped by
   the hardware. The DMA lands somewhere else in memory. This
   is one of the nastiest bugs in the field: it works on a
   board with 2 GB of RAM and corrupts memory on the 8 GB
   variant, because only then do allocations land above 4 GB.

   THE DEFAULT is 32 bits. If your device really is 32-bit,
   you can omit the call — but state it explicitly anyway, so
   the next reader knows it was considered.                */
```

Coherent (consistent) mappings

```
/* For memory the CPU and device both touch REPEATEDLY, for
   the lifetime of the driver: descriptor rings, command
   queues, status blocks.                                    */

        void *cpu_addr;
        dma_addr_t dma_handle;

        cpu_addr = dma_alloc_coherent(dev, size, &dma_handle,
                                      GFP_KERNEL);
        if (!cpu_addr)
                return -ENOMEM;

        /* program the device with dma_handle */
        writel(lower_32_bits(dma_handle), base + REG_DESC_LO);
        writel(upper_32_bits(dma_handle), base + REG_DESC_HI);

        /* the CPU uses cpu_addr normally - NO cache
           maintenance needed, ever                          */
        ring[0].addr = ...;
        ring[0].len  = ...;

        ...
        dma_free_coherent(dev, size, cpu_addr, dma_handle);

/* WHAT "COHERENT" MEANS: the CPU and device see the same
   data without explicit cache maintenance. How that is
   achieved depends on the hardware:

     · on a system with hardware coherency (dma-coherent in
       the DT, ARM ch. 19), it is ordinary cached memory and
       the interconnect keeps it consistent
     · without it, the kernel maps the memory NON-CACHED

   THAT SECOND CASE IS THE COST. Non-cached CPU access is
   slow - every read goes to DRAM. So use coherent mappings
   for SMALL, control-plane structures, and streaming mappings
   for BULK data.

/* AND EVEN COHERENT MEMORY NEEDS ORDERING BARRIERS
   (chapter 23) - coherency is about caches, not about the
   order in which writes become visible: */

        ring[idx].flags = DESC_OWN;
        dma_wmb();                      /* descriptor visible
                                           BEFORE the doorbell */
        writel(idx, base + REG_DOORBELL);

/* A POOL, for many small identical buffers - avoids wasting a
   page each: */
        pool = dma_pool_create("mydev", dev, 64, /* size  */
                                             64, /* align */
                                             0); /* boundary */
        p = dma_pool_alloc(pool, GFP_KERNEL, &dma);
        dma_pool_free(pool, p, dma);
        dma_pool_destroy(pool);
```

Streaming mappings

```
/* For buffers used for ONE transfer, that the CPU already
   owns: a network packet, a block of user data, a frame.   */

        dma_addr_t dma;

        dma = dma_map_single(dev, buf, len, DMA_TO_DEVICE);
        if (dma_mapping_error(dev, dma))     /* CHECK IT */
                return -ENOMEM;

        /* the buffer now belongs to the DEVICE. The CPU MUST
           NOT touch it until it is unmapped.               */
        writel(lower_32_bits(dma), base + REG_ADDR);
        writel(len, base + REG_LEN);
        writel(GO, base + REG_CTRL);

        wait_for_completion(&d->done);

        dma_unmap_single(dev, dma, len, DMA_TO_DEVICE);
        /* now the CPU owns it again */

/* THE DIRECTIONS, and they are not decorative: */
        DMA_TO_DEVICE       CPU wrote it, device reads it
        DMA_FROM_DEVICE     device writes it, CPU reads it
        DMA_BIDIRECTIONAL   both. More cache maintenance.
        DMA_NONE            debugging only

/* WHAT map/unmap ACTUALLY DO on a non-coherent system:
     map,   TO_DEVICE:    CLEAN the cache - push the CPU's
                           writes out to memory so the device
                           sees them
     unmap, FROM_DEVICE:  INVALIDATE the cache - discard stale
                           lines so the CPU reads what the
                           device wrote
   plus, always: set up (or tear down) the SMMU mapping for
   this buffer, allocating an IOVA.

   THE OWNERSHIP RULE follows directly: touching the buffer
   while the device owns it either loses your write (it gets
   invalidated) or reads stale data. It is not a convention;
   it is what the cache maintenance implies.              */

/* TOUCHING IT MID-TRANSFER, when you genuinely must: */
        dma_sync_single_for_cpu(dev, dma, len, DMA_FROM_DEVICE);
        inspect(buf);                   /* CPU owns it now */
        dma_sync_single_for_device(dev, dma, len, DMA_FROM_DEVICE);
        /* device owns it again */
```

WHAT YOU MUST NEVER DMA TO

- **Stack memory.** On a `VMAP_STACK` kernel the stack is vmalloc'd, so it is not physically contiguous and `virt_to_phys` is meaningless. Even without that, it may share a cache line with other stack data, which cache maintenance would corrupt.
- `vmalloc` **memory.** Physically scattered (chapter 05).
- **Module static data.** May be in the module's vmalloc area.
- **Anything you did not map.** With an SMMU present, an unmapped address is not "the wrong data" — it is a fault.

Allocate with `kmalloc` and map it, or use `dma_alloc_coherent`. Every other route is a bug that hides until the machine configuration changes.

Scatter-gather

```
/* Large buffers are rarely physically contiguous. Rather than
   forcing a contiguous allocation, describe the pieces.    */

        struct sg_table sgt;
        struct scatterlist *sg;
        int i, nents;

        /* build the list - or get one from a caller */
        sg_alloc_table(&sgt, nr_pages, GFP_KERNEL);
        for_each_sgtable_sg(&sgt, sg, i)
                sg_set_page(sg, pages[i], PAGE_SIZE, 0);

        /* map it ALL at once */
        nents = dma_map_sgtable(dev, &sgt, DMA_FROM_DEVICE, 0);
        if (nents)
                return nents;

        /* CRITICAL: iterate the MAPPED list, not the original.
           The IOMMU may have COALESCED several physical
           segments into one contiguous IOVA range, so the
           mapped list can be SHORTER.                       */
        for_each_sgtable_dma_sg(&sgt, sg, i) {
                program_descriptor(d, i,
                                   sg_dma_address(sg),
                                   sg_dma_len(sg));
        }

        ... transfer ...

        dma_unmap_sgtable(dev, &sgt, DMA_FROM_DEVICE, 0);
        sg_free_table(&sgt);

/* THE COALESCING POINT IS THE COMMON BUG: code that iterates
   with for_each_sg() using the ORIGINAL nents, and reads
   sg->length instead of sg_dma_len(sg), programs the wrong
   descriptors. It works without an SMMU (where no coalescing
   happens) and breaks the moment one is enabled - which is
   exactly when someone turns on isolation.              */
```

Using a DMA engine

```
/* Many SoC peripherals do not master the bus themselves; a
   shared DMA controller moves data for them. The dmaengine
   API drives it.                                          */

#include <linux/dmaengine.h>

/* device tree: */
        dmas = <&dmac 12>, <&dmac 13>;
        dma-names = "tx", "rx";

/* probe: */
        d->rxchan = dma_request_chan(dev, "rx");
        if (IS_ERR(d->rxchan))
                return dev_err_probe(dev, PTR_ERR(d->rxchan),
                                     "no rx channel\n");

        struct dma_slave_config cfg = {
                .direction    = DMA_DEV_TO_MEM,
                .src_addr     = d->phys_base + REG_FIFO,
                .src_addr_width = DMA_SLAVE_BUSWIDTH_4_BYTES,
                .src_maxburst = 8,
        };
        dmaengine_slave_config(d->rxchan, &cfg);

/* one transfer: */
        struct dma_async_tx_descriptor *desc;
        dma_cookie_t cookie;

        desc = dmaengine_prep_slave_sg(d->rxchan, sgt.sgl,
                                       sgt.nents,
                                       DMA_DEV_TO_MEM,
                                       DMA_PREP_INTERRUPT |
                                       DMA_CTRL_ACK);
        if (!desc)
                return -EINVAL;

        desc->callback       = mydev_dma_done;
        desc->callback_param = d;

        cookie = dmaengine_submit(desc);
        if (dma_submit_error(cookie))
                return -EIO;

        dma_async_issue_pending(d->rxchan);   /* GO */

        ... the callback fires ...

        dmaengine_terminate_sync(d->rxchan);  /* in remove -
                                                 _sync waits  */
        dma_release_channel(d->rxchan);

/* CYCLIC transfers, for continuous capture - audio, ADCs:  */
        desc = dmaengine_prep_dma_cyclic(chan, dma_addr,
                                         buf_len, period_len,
                                         DMA_DEV_TO_MEM,
                                         DMA_PREP_INTERRUPT);
/* the callback fires once per PERIOD and the transfer never
   ends, which is how you build a ring without gaps.       */
```

The SMMU, from the driver's side

```
/* Everything the isolation track described (chapters 03-07)
   happens inside the DMA API. From the driver you see it in
   three places:                                            */

/* 1. THE DEVICE TREE declares the relationship: */
        my_device: dma-thing@1c000000 {
                compatible = "vendor,dma-thing";
                reg = <0x0 0x1c000000 0x0 0x1000>;
                iommus = <&smmu 0x42>;    /* StreamID 0x42 */
                dma-coherent;             /* hardware coherency */
        };

/* 2. dma_map_* SETS UP THE TRANSLATION and returns an IOVA,
      not a physical address. dma_unmap_* tears it down and
      invalidates the SMMU TLB.

      This is why a "clever" driver that caches a dma_addr_t
      and reuses it after unmapping gets an SMMU fault: the
      mapping is gone.

/* 3. FAULTS APPEAR IN dmesg with the StreamID: */
      arm-smmu-v3 : event 0x10 received:
      arm-smmu-v3 : 0x0000004200000010
      arm-smmu-v3 : 0x0000000000000000
        -> C_BAD_STREAMID, or F_TRANSLATION at an IOVA

      Read them with isolation chapter 06. The usual causes,
      from a driver author's point of view:
        · programmed the device with a physical address
        · used a dma_addr_t after unmapping
        · the device is issuing a longer transfer than you
          mapped
        · wrong StreamID in the DT, so the SMMU has no
          context for this device at all

/* AND THE DEBUGGING SWITCH worth knowing: */
        iommu.passthrough=1        # kernel cmdline: bypass
                                   # translation entirely
   If the problem disappears with passthrough, it is a mapping
   problem, not a hardware one. Never ship with it.        */
```

Debugging DMA

```
# THE DMA DEBUG INFRASTRUCTURE - turn it on while developing:
    CONFIG_DMA_API_DEBUG=y

# It tracks every mapping and reports:
#   · unmapping with the wrong size or direction
#   · freeing memory that is still mapped
#   · CPU access to a buffer the device owns
#   · mapping the same buffer twice
#   · a dma_addr_t used after unmap
#   · DMA to a stack or vmalloc address
#
# The report names both the map and the misuse site. This
# option finds more real DMA bugs than any amount of testing.

    DMA-API: device driver tries to free DMA memory it has
             not allocated [device address=0x00000000abcd0000]
    DMA-API: device driver maps memory from stack

# See the mappings:
    cat /sys/kernel/debug/dma-api/dump
    cat /sys/kernel/debug/dma-api/error_count

# Force bounce buffering, to test that path:
    swiotlb=force                # kernel cmdline

# Is the SMMU actually translating for this device?
    ls /sys/kernel/iommu_groups/*/devices/
    dmesg | grep -i smmu
```

How it shows up

Symptom	Cause
DMA writes land in unrelated memory	Programmed with a physical or virtual address instead of the `dma_addr_t`
SMMU translation fault at a plausible address	Same cause — the physical address is not a valid IOVA
Works on 2 GB board, corrupts on 8 GB	DMA mask too wide for the hardware; top address bits dropped
Device reads stale data	Missing `dma_wmb()`, or the CPU wrote after mapping
CPU reads stale data	Read before `dma_unmap` or `dma_sync_for_cpu`
Random corruption near a buffer	DMA to a stack buffer sharing a cache line
Works without an SMMU, faults with one	Iterating the unmapped scatterlist, or using `sg->length` instead of `sg_dma_len`
Fault after a successful transfer	`dma_addr_t` reused after unmapping
Only the first N bytes transfer	Device programmed with a length longer than the mapping, truncated at the fault
Everything works with `iommu.passthrough=1`	Confirms a mapping bug, not a hardware one
`dma_alloc_coherent` fails for a large size	Fragmentation — use CMA (ch. 05)
CPU access to a coherent buffer is very slow	Non-coherent system: it is mapped uncached. Use streaming for bulk data

Check yourself

- ✓ Name the three address spaces and four reasons a DMA address differs from a physical one.
- ✓ Set a DMA mask and explain the failure a too-wide mask causes.
- ✓ Choose between coherent and streaming mappings, with the reason.
- ✓ Explain what `dma_map_single` does on a non-coherent system, per direction.
- ✓ State the ownership rule and why it follows from cache maintenance.
- ✓ List four kinds of memory you must never DMA to.
- ✓ Map a scatterlist and explain why you must iterate the mapped list.
- ✓ Drive a transfer through the dmaengine API, including teardown.
- ✓ Explain what the DMA API does with the SMMU, and read a fault.
- ✓ Enable `DMA_API_DEBUG` and name five things it catches.

Power management and runtime PM

On a battery-powered device, a driver that never lets its hardware sleep is a bug — even if it works perfectly. Two independent mechanisms handle it: system suspend, for the whole machine, and runtime PM, for one idle device.

Two mechanisms, one set of callbacks

```
SYSTEM SUSPEND                RUNTIME PM
--------------                ----------
the whole machine sleeps      ONE device sleeps while the
user or policy initiated      system runs
everything goes down in       reference counted: sleeps when
  dependency order            nobody is using it
.suspend / .resume            .runtime_suspend /
                                .runtime_resume

THEY COMPOSE. A device may be runtime-suspended when system
suspend arrives, and must handle that. The core helps:

   pm_runtime_force_suspend()
   pm_runtime_force_resume()

used as the system callbacks, do the right thing in both
cases.

THE STATES a device moves through:

   RPM_ACTIVE      powered, usable
      |  usage_count drops to 0, autosuspend delay expires
      v
   RPM_SUSPENDING  .runtime_suspend running
      |
      v
   RPM_SUSPENDED   powered down
      |  someone calls pm_runtime_get()
      v
   RPM_RESUMING    .runtime_resume running
      |
      v
   RPM_ACTIVE
```

Runtime PM

```
#include <linux/pm_runtime.h>

/* --- probe: opt in ---------------------------------------- */
static int mydev_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        ...

        /* the hardware IS on at this point, so tell the core */
        pm_runtime_set_active(dev);

        /* wait this long after the last use before suspending -
           avoids thrashing on a bursty workload             */
        pm_runtime_set_autosuspend_delay(dev, 1000);   /* ms  */
        pm_runtime_use_autosuspend(dev);

        /* from here, the core may call our callbacks */
        pm_runtime_enable(dev);

        /* devm form, which handles disable on unbind: */
        ret = devm_pm_runtime_enable(dev);
        if (ret)
                return ret;

        ...
        /* let it go idle once probe is done */
        pm_runtime_put_autosuspend(dev);
        return 0;
}

/* --- the callbacks ---------------------------------------- */
static int mydev_runtime_suspend(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);

        /* save anything the hardware will forget */
        regcache_cache_only(d->regmap, true);
        regcache_mark_dirty(d->regmap);

        clk_disable_unprepare(d->clk);
        regulator_disable(d->vdd);
        return 0;
}

static int mydev_runtime_resume(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);
        int ret;

        ret = regulator_enable(d->vdd);
        if (ret)
                return ret;

        ret = clk_prepare_enable(d->clk);
        if (ret)
                goto err_reg;

        /* the hardware came up from reset - restore it
           (chapter 23)                                      */
        regcache_cache_only(d->regmap, false);
```

```
        ret = regcache_sync(d->regmap);
        if (ret)
                goto err_clk;

        return 0;

err_clk:
        clk_disable_unprepare(d->clk);
err_reg:
        regulator_disable(d->vdd);
        return ret;
}

/* --- wrapping every hardware access ---------------------- */
static ssize_t mydev_read(struct file *f, char __user *buf,
                          size_t count, loff_t *ppos)
{
        struct mydev *d = ...;
        int ret;

        /* resume if needed, and BLOCK until it is up */
        ret = pm_runtime_resume_and_get(d->dev);
        if (ret < 0)
                return ret;

        ... touch the hardware ...

        pm_runtime_mark_last_busy(d->dev);
        pm_runtime_put_autosuspend(d->dev);
        return ret;
}

/* USE pm_runtime_resume_and_get(), not the older
   pm_runtime_get_sync(). The older one leaves the usage
   count INCREMENTED on failure, so an error path that
   returns without a put() pins the device awake forever.
   resume_and_get() drops it for you on error. This is a real
   and very common leak in older drivers.                   */
```

THE REFERENCE-COUNTING DISCIPLINE

Every path that touches hardware must be bracketed by a get and a put — *including every error path*. An unbalanced get pins the device permanently awake, which looks like "power management does not work" and is invisible in testing. An unbalanced put lets the device suspend under an active user, which looks like random hardware failure. Both are found immediately by watching `/sys/.../power/runtime_status` while exercising the driver.

System suspend

```
static int mydev_suspend(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);

        /* refuse if we are mid-operation and cannot stop */
        if (mydev_busy(d))
                return -EBUSY;      /* aborts the suspend */

        /* stop new work, quiesce the hardware */
        disable_irq(d->irq);
        mydev_stop(d);

        /* if this device can wake the system, arm it */
        if (device_may_wakeup(dev))
                enable_irq_wake(d->irq);

        return 0;
}

static int mydev_resume(struct device *dev)
{
        struct mydev *d = dev_get_drvdata(dev);

        if (device_may_wakeup(dev))
                disable_irq_wake(d->irq);

        mydev_hw_init(d);          /* it came up from reset */
        enable_irq(d->irq);
        return 0;
}

/* --- declaring them ------------------------------------- */
static const struct dev_pm_ops mydev_pm_ops = {
        SYSTEM_SLEEP_PM_OPS(mydev_suspend, mydev_resume)
        RUNTIME_PM_OPS(mydev_runtime_suspend,
                       mydev_runtime_resume, NULL)
};

/* or, when the runtime callbacks do the whole job: */
static const struct dev_pm_ops mydev_pm_ops = {
        SET_SYSTEM_SLEEP_PM_OPS(pm_runtime_force_suspend,
                                pm_runtime_force_resume)
        SET_RUNTIME_PM_OPS(mydev_runtime_suspend,
                           mydev_runtime_resume, NULL)
};

static struct platform_driver mydev_driver = {
        .driver = {
                .name = "my-device",
                .pm   = pm_ptr(&mydev_pm_ops),
                ...
        },
        ...
};
/* pm_ptr() compiles the reference away when CONFIG_PM is off,
   so the ops struct is discarded rather than kept as dead
   weight. Pair it with __maybe_unused or the
   DEFINE_*_DEV_PM_OPS macros on the callbacks.            */
```

Suspend phases

```
System suspend runs in stages, and which one you need depends
on what your callback must do:

   prepare()          may sleep, may allocate. Last chance to
                      refuse.
   suspend()          the normal one. Interrupts enabled,
                      may sleep. USE THIS.
   suspend_late()     after other devices; still can sleep
   suspend_noirq()    interrupts DISABLED. Only for interrupt
                      controllers and things that must be last.
                      Cannot sleep on anything that needs an
                      interrupt - which includes most I2C and
                      SPI transfers.

   ... the system sleeps ...

   resume_noirq()
   resume_early()
   resume()           the normal one
   complete()

  THE ORDERING GUARANTEE: children suspend before parents, and
  resume after them. So an I2C client suspends before its bus
  controller - which is what makes it safe to talk to the chip
  in .suspend but not in .suspend_noirq.

  IF YOU NEED I2C/SPI IN A CALLBACK, it must be in
  .suspend/.resume, not the _noirq variants. Putting it later
  produces a hang at suspend that is very hard to diagnose,
  because the console is usually gone by then.
```

Wakeup sources

```
/* A device that can wake the system - a button, an RTC alarm,
   a network packet, a modem interrupt.                     */

/* in probe: */
        device_init_wakeup(dev, true);      /* capable */
        /* or, respecting the DT: */
        if (device_property_read_bool(dev, "wakeup-source"))
                device_init_wakeup(dev, true);

/* in suspend: */
        if (device_may_wakeup(dev))
                enable_irq_wake(d->irq);

/* USERSPACE CONTROLS whether it is ENABLED: */
        cat /sys/devices/.../power/wakeup       # enabled/disabled
        echo enabled > /sys/devices/.../power/wakeup

/* device_init_wakeup() says "this CAN wake"; the sysfs file
   says "this SHOULD". Respect device_may_wakeup(), which
   combines them - a driver that always arms its wakeup
   ignores a user decision and drains the battery.        */

/* SEE WHAT WOKE THE SYSTEM: */
        cat /sys/kernel/debug/wakeup_sources
        dmesg | grep -i wakeup
        cat /sys/power/pm_wakeup_irq
```

Watching it work

```
# Per-device runtime PM state - the file to watch:
    cat /sys/devices/platform/.../power/runtime_status
      active | suspended | suspending | resuming
    cat .../power/runtime_active_time      # ms awake
    cat .../power/runtime_suspended_time   # ms asleep
    cat .../power/autosuspend_delay_ms
    cat .../power/control                  # auto | on

# FORCE a device to stay awake, to test whether PM is the
# cause of a bug:
    echo on > /sys/devices/.../power/control
    # and back:
    echo auto > /sys/devices/.../power/control

# Test system suspend:
    echo mem > /sys/power/state          # suspend-to-RAM
    echo freeze > /sys/power/state       # s2idle - lighter
    cat /sys/power/state                 # what is supported

# Suspend for a fixed time using the RTC, so you get back:
    rtcwake -m mem -s 10

# WHEN SUSPEND FAILS, this names the device:
    echo 1 > /sys/power/pm_debug_messages
    dmesg | grep -i "PM:"
      PM: suspend of devices aborted after ...
      my-device: PM: failed to suspend: error -16

# Where the time goes during suspend/resume:
    echo 1 > /sys/power/pm_print_times
    dmesg | grep "PM: "
      call my-device+ returned 0 after 12043 usecs
# A device taking hundreds of milliseconds is your resume
# latency, and users notice it.

# Power usage attribution, on hardware that supports it:
    cat /sys/kernel/debug/pm_genpd/pm_genpd_summary
    powertop
```

Power domains

```
/* Several devices often share one power rail or one clock
   domain in silicon. The genpd framework models that: the
   domain powers down only when EVERY device in it is idle. */

        /* device tree: */
        my_device: device@1c000000 {
                compatible = "vendor,my-device";
                power-domains = <&pd_periph>;
        };

/* You usually need no driver code at all - the core attaches
   the device to the domain and the domain's own driver
   handles the rail. Your runtime_suspend runs first, then the
   domain powers off if nobody else is holding it.

   THE CONSEQUENCE worth understanding: your device may not
   actually lose power when it suspends, because a sibling is
   still active. So resume must be correct whether or not the
   hardware was reset - which is another argument for
   regcache_sync() rather than assuming a known state.     */

/* SEE THE DOMAINS: */
        cat /sys/kernel/debug/pm_genpd/pm_genpd_summary
        domain          status    children   performance
        pd_periph       on                   0
            /device:1c000000.my-device  active
            /device:1c001000.other      suspended
```

A PM review checklist

Check	Why
Every hardware access bracketed by get/put	Otherwise you touch a powered-down device
Every error path drops its reference	Otherwise the device never sleeps
`pm_runtime_resume_and_get`, not `get_sync`	Avoids the failure-path count leak
Resume restores *all* register state	The hardware may have been reset
Resume works even if it was *not* reset	A shared power domain may have stayed on
Suspend refuses (`-EBUSY`) if mid-operation	Better than corrupting a transfer
No I2C/SPI in `_noirq` phases	The bus is already suspended
Wakeup respects `device_may_wakeup()`	The user's choice, not yours
Interrupts disabled before suspending the hardware	Avoids a handler on a dead device
Tested with `rtcwake` in a loop	Suspend bugs are often intermittent

How it shows up

Symptom	Cause
Device never runtime-suspends	Unbalanced `get`, often on an error path
`runtime_status` stuck at `active`	Same, or `power/control` is `on`
Crash or timeout on the first access after idle	Hardware access without a `get`

Symptom	Cause
Device misbehaves after resume	Register state not restored — use `regcache_sync()`
Works after system resume, not after runtime resume	Resume assumes a full reset that did not happen
Suspend hangs the machine	I2C/SPI access in a `_noirq` callback
Suspend aborts with `-EBUSY`	Your `.suspend` refused — usually correct
System wakes immediately	A wakeup source armed that should not be — check `pm_wakeup_irq`
Device cannot wake the system	`enable_irq_wake()` not called, or wakeup disabled in sysfs
Resume takes hundreds of milliseconds	`pm_print_times` names the device
Battery drains with the screen off	Some device never suspending — check every `runtime_status`

Check yourself

- ✓ Distinguish system suspend from runtime PM and say how they compose.
- ✓ Enable runtime PM in probe with autosuspend, and explain each call.
- ✓ Write runtime suspend and resume callbacks with correct error unwinding.
- ✓ Bracket a hardware access correctly, including the error path.
- ✓ Explain why `pm_runtime_resume_and_get` replaced `get_sync`.
- ✓ Name the suspend phases and say which may talk to an I2C device.
- ✓ Explain the parent/child ordering guarantee.
- ✓ Implement a wakeup source that respects the user's setting.
- ✓ Use `rtcwake`, `pm_print_times` and `runtime_status` to verify behaviour.
- ✓ Explain why resume must work whether or not the hardware was reset.

Error handling patterns and `devm`

The kernel has no exceptions and no destructors, so unwinding is manual — and error paths are the least-tested code in any driver. Two disciplines make them reliable: the `goto` ladder, and letting the device model free things for you.

The goto ladder

```
/* This is idiomatic kernel C, not a code smell. Every label
   undoes exactly one step, in reverse order.               */

static int mydev_probe(struct platform_device *pdev)
{
        struct mydev *d;
        int ret;

        d = kzalloc(sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;

        ret = regulator_enable(d->vdd);
        if (ret)
                goto err_free;

        ret = clk_prepare_enable(d->clk);
        if (ret)
                goto err_reg;

        ret = mydev_hw_init(d);
        if (ret)
                goto err_clk;

        ret = request_irq(d->irq, mydev_isr, 0, "mydev", d);
        if (ret)
                goto err_hw;

        return 0;

err_hw:
        mydev_hw_shutdown(d);
err_clk:
        clk_disable_unprepare(d->clk);
err_reg:
        regulator_disable(d->vdd);
err_free:
        kfree(d);
        return ret;
}

/* WHY IT BEATS THE ALTERNATIVES:

   · nested ifs get unreadable past three levels, and the
     cleanup is duplicated at each depth
   · a single "cleanup everything" function must guess how far
     you got, which means tracking state - more code, more
     bugs
   · early returns duplicate the unwind at every exit

   THE RULE THAT MAKES IT WORK: label names say WHAT THEY
   UNDO, not where they came from. `err_clk` disables the
   clock. Add a step in the middle and you insert one label
   and one goto - nothing else changes.

   NAME THEM AFTER THE THING BEING UNDONE, never err1, err2,
   err3 - those force a renumber every time the order
   changes, and renumbering is where the mistakes creep in. */
```

devm: managed resources

```
/* The device model can free things automatically when the
   device unbinds - which deletes most of the ladder.       */

static int mydev_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        struct mydev *d;

        d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;

        d->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(d->base))
                return PTR_ERR(d->base);

        d->clk = devm_clk_get_enabled(dev, NULL);
        if (IS_ERR(d->clk))
                return PTR_ERR(d->clk);

        d->irq = platform_get_irq(pdev, 0);
        if (d->irq < 0)
                return d->irq;

        ret = devm_request_irq(dev, d->irq, mydev_isr, 0,
                               "mydev", d);
        if (ret)
                return ret;

        return 0;      /* no labels, no unwind, no leaks */
}

/* HOW IT WORKS: each devm_ call appends an entry to a list on
   the struct device. When the device unbinds - whether
   because probe failed, remove ran, or the module unloaded -
   the core walks that list IN REVERSE and calls each release
   function.

   So the unwinding order is automatically correct, and it is
   correct on the failure path too, without you writing it.

/* THE COMMON ONES: */
        devm_kzalloc, devm_kcalloc, devm_kstrdup
        devm_ioremap, devm_platform_ioremap_resource
        devm_clk_get, devm_clk_get_enabled, devm_clk_bulk_get
        devm_regulator_get, devm_regulator_bulk_get
        devm_reset_control_get
        devm_gpiod_get, devm_gpiod_get_optional
        devm_request_irq, devm_request_threaded_irq
        devm_regmap_init_mmio
        devm_pm_runtime_enable
        devm_add_action_or_reset   /* <-- the general one */
```

WHAT `DEVM` DOES NOT DO

`devm_clk_get()` releases the clock reference — it does **not** disable the clock. `devm_regulator_get()` does not disable the regulator. `devm_gpiod_get()` does not restore the pin state. Anything you *enabled* by hand, you must disable by hand in remove. Use `devm_clk_get_enabled()` and the bulk enable helpers where they exist; for the rest, the pattern below closes the gap.

devm_add_action_or_reset

```
/* Register YOUR OWN cleanup with the device, so anything can
   participate in managed unwinding.                          */

static void mydev_disable_vdd(void *data)
{
        regulator_disable(data);
}

        ret = regulator_enable(d->vdd);
        if (ret)
                return ret;

        ret = devm_add_action_or_reset(dev, mydev_disable_vdd,
                                       d->vdd);
        if (ret)
                return ret;
        /* from here on, the regulator is disabled
           automatically on unbind - and if the registration
           itself failed, _or_reset already called the action,
           so there is nothing to unwind.                     */

/* THE "_or_reset" SUFFIX IS THE POINT: plain
   devm_add_action() leaves you holding the resource if the
   registration fails, so you would need an error path for the
   error path. _or_reset calls the action immediately on
   failure, so `return ret;` is always correct.

   ALWAYS USE _or_reset. There is no good reason for the
   plain form in new code.                                    */

/* THIS PATTERN GENERALISES to anything: */
static void mydev_hw_off(void *data)
{
        mydev_hw_shutdown(data);
}
        ret = mydev_hw_init(d);
        if (ret)
                return ret;
        ret = devm_add_action_or_reset(dev, mydev_hw_off, d);
        if (ret)
                return ret;

/* Now even hardware shutdown is managed, and remove() can be
   empty - which is the goal: a driver whose teardown cannot
   be wrong because there is no teardown code to get wrong. */
```

ERR_PTR and error returns

```
/* Kernel convention: functions returning a pointer encode
   errors IN the pointer, using the top page of the address
   space, which no valid pointer occupies.                  */

        void *p = some_function();

        if (IS_ERR(p))
                return PTR_ERR(p);      /* extract the errno */

/* NOT: */
        if (!p)                          /* WRONG - an error
                                            pointer is NOT
                                            NULL              */

/* WRITING one: */
struct mydev *mydev_find(int id)
{
        if (id < 0)
                return ERR_PTR(-EINVAL);
        if (!exists(id))
                return ERR_PTR(-ENODEV);
        return &devices[id];
}

/* THE THREE-WAY CASE - can return NULL, an error, or a valid
   pointer (optional resources do this): */
        if (IS_ERR_OR_NULL(p))
                ...
        /* or, more precisely: */
        if (IS_ERR(p))
                return PTR_ERR(p);      /* a real error */
        if (!p)
                /* legitimately absent - the optional case */

/* PRINTING one: */
        dev_err(dev, "failed: %pe\n", p);
                -> "failed: -ENOMEM"

/* AND THE ERRNO CODES that matter in drivers: */
        -ENOMEM         out of memory
        -EINVAL         bad argument from a caller
        -ENODEV         no such device / not present
        -EIO            hardware failed
        -EBUSY          in use, try later
        -ETIMEDOUT      hardware did not respond
        -EPROBE_DEFER   a supplier is not ready yet
        -ENOTSUPP       this hardware cannot do that
        -ENOTTY         no such ioctl
        -EAGAIN         nothing available right now
        -ERESTARTSYS    interrupted, please restart

/* PICK THE SPECIFIC ONE. -EINVAL for everything makes a
   userspace program's diagnostics useless, and yours too. */
```

dev_err_probe

```
/* Every resource acquisition in probe should use it: */

        d->clk = devm_clk_get(dev, "core");
        if (IS_ERR(d->clk))
                return dev_err_probe(dev, PTR_ERR(d->clk),
                                     "failed to get core clock\n");

/* IT DOES FOUR THINGS:
     1. logs with the device name prefixed
     2. for -EPROBE_DEFER, logs at DEBUG level instead of
        ERROR - so normal boot is not flooded with messages
        about deferrals that resolve
     3. records the deferral reason, visible in
        /sys/kernel/debug/devices_deferred
     4. returns the error code, so `return` composes

   THE ALTERNATIVE people write: */
        if (IS_ERR(d->clk)) {
                dev_err(dev, "no clock\n");      /* spams on
                                                    every retry */
                return PTR_ERR(d->clk);
        }
/* which works but floods the log during normal deferred
   probe, and loses the deferral reason.                    */
```

Scope-based cleanup

```
/* Newer kernels have __cleanup__-based helpers, borrowed from
   C++ RAII. They release on EVERY exit from the scope,
   including returns and gotos.                              */

#include <linux/cleanup.h>

/* LOCKS: */
static int mydev_do_thing(struct mydev *d)
{
        guard(mutex)(&d->lock);       /* held until return */

        if (!d->ready)
                return -EAGAIN;        /* unlocked correctly */

        return mydev_thing(d);         /* and here too */
}

/* a narrower scope: */
        scoped_guard(spinlock_irqsave, &d->lock) {
                d->count++;
        }

/* MEMORY: */
        char *buf __free(kfree) = kmalloc(len, GFP_KERNEL);
        if (!buf)
                return -ENOMEM;
        ...
        return 0;              /* buf freed automatically */

/* REFERENCE COUNTS: */
        struct device_node *np __free(device_node) =
                of_parse_phandle(dev->of_node, "partner", 0);

/* WHERE THIS HELPS MOST: functions with many early returns
   under a lock, which is exactly where forgotten unlocks
   live. It does not replace the goto ladder for probe -
   ordered hardware teardown still needs explicit sequencing -
   but it removes a whole class of lock leaks.

   Check whether your target kernel has it before relying on
   it; it landed in 6.4 and later.                           */
```

Testing error paths

```
# Error paths are the least-executed and least-tested code in
# any driver. Force them.

# 1. FAULT INJECTION on allocations:
    CONFIG_FAILSLAB=y
    CONFIG_FAULT_INJECTION_DEBUG_FS=y
    echo 10   > /sys/kernel/debug/failslab/probability
    echo 100  > /sys/kernel/debug/failslab/times
    echo 1    > /sys/kernel/debug/failslab/verbose
    # then load your driver repeatedly. 10% of allocations
    # fail; anything that crashes is an unchecked return.

# 2. REMOVE RESOURCES from the device tree, one at a time:
#      delete the clocks property   -> probe must fail cleanly
#      delete the interrupt         -> same
#      point reg at a bad address   -> the ID check must catch it

# 3. BIND/UNBIND in a loop, under KASAN:
    for i in $(seq 100); do
      echo 1c000000.my-device > .../unbind
      echo 1c000000.my-device > .../bind
    done
    dmesg | grep -i -e kasan -e "warn" -e "bug"

# 4. CHECK FOR LEAKS after a failed probe:
    cat /proc/iomem      | grep 1c000000    # should be gone
    cat /proc/interrupts | grep my-device   # should be gone
    cat /sys/kernel/debug/kmemleak

# EVERY ONE of these finds real bugs in drivers that pass
# their functional tests. The fault-injection run in
# particular usually finds two or three unchecked returns the
# first time it is used on a driver.
```

> **THE RULE THAT PREVENTS MOST OF IT**
>
> Whenever you write a line that acquires something, write the line that releases it *in the same edit* — the `goto` label, the `devm_add_action_or_reset`, or the `__free` annotation. Deferring "I will add the cleanup later" is how leaks are born, because later never has the full context of what order things were acquired in.

How it shows up

Symptom	Cause
Crash when a NULL check "passed"	Checked for NULL where `IS_ERR` was needed
Resource leak after a failed probe	Missing unwind step, or an early return past the ladder
Region still in `/proc/iomem` after unbind	Bare `ioremap` instead of the devm/resource form
Clock stays on after unbind	`devm_clk_get` does not disable — use `devm_clk_get_enabled`
Regulator refcount imbalance warnings	`regulator_enable` without a matching disable path
Second bind fails after a successful unbind	State left behind — something not released
Log flooded with "failed to get clock" at boot	`dev_err` instead of `dev_err_probe` on a deferral
Deadlock on an error path	Early return under a lock — `guard()` prevents this class

Symptom	Cause
Crash only under memory pressure	Unchecked allocation — fault injection finds it in minutes
Unwind order wrong, hardware left in a bad state	Manual cleanup function that guessed how far setup got

Check yourself

- ✓ Write a `goto` ladder and explain the label-naming rule.
- ✓ Explain how `devm` unwinding works and in what order.
- ✓ List three things `devm` does *not* release for you.
- ✓ Use `devm_add_action_or_reset` and explain what the suffix guarantees.
- ✓ Distinguish `IS_ERR`, NULL and `IS_ERR_OR_NULL`, and print an error pointer.
- ✓ Choose the right errno for six different failures.
- ✓ Explain all four things `dev_err_probe` does.
- ✓ Use `guard()` and `__free()` and say which problem each removes.
- ✓ Run fault injection against your driver and fix what it finds.
- ✓ Verify no resources leak after a deliberately failed probe.

Debugging the kernel

Reading an oops, tracing with ftrace, attaching a debugger, and the config options that turn silent corruption into a loud message with a backtrace. The single highest-leverage skill in kernel work is extracting the maximum from one crash.

Reading an oops

```
Unable to handle kernel NULL pointer dereference at virtual
        address 0000000000000018
Mem abort info:
  ESR = 0x0000000096000004
  EC = 0x25: DABT (current EL), IL = 32 bits
  SET = 0, FnV = 0
  EA = 0, S1PTW = 0
  FSC = 0x04: level 0 translation fault
Data abort info:
  ISV = 0, ISS = 0x00000004
  CM = 0, WnR = 0
CPU: 2 PID: 431 Comm: mytest Tainted: G           O
                                     6.12.0 #1
Hardware name: ARM Juno development board (DT)
pstate: 60000005 (nZCv daif -PAN -UAO)
pc : mydev_read+0x48/0x1c0 [mydev]
lr : vfs_read+0xb4/0x1e0
sp : ffff800012345678
x29: ffff800012345678 x28: ffff0000c1234000
x27: 0000000000000000 x26: 0000000000000004
...
Call trace:
 mydev_read+0x48/0x1c0 [mydev]
 vfs_read+0xb4/0x1e0
 ksys_read+0x70/0x100
 __arm64_sys_read+0x24/0x30
 invoke_syscall+0x50/0x120
 el0_svc_common+0x4c/0x100
 do_el0_svc+0x30/0xa0
 el0_svc+0x2c/0x90
 el0t_64_sync_handler+0xb8/0xc0
 el0t_64_sync+0x1a0/0x1a4
Code: aa0003f3 f9400400 91006000 f9400c00 (f9400c01)
---[ end trace 0000000000000000 ]---
```

```
READ IT IN THIS ORDER:

  1. THE FIRST LINE says what happened.
       "NULL pointer dereference at 0000000000000018"
       -> a pointer was NULL and something read offset 0x18
          from it. Look for a struct whose member is at 0x18.

  2. ESR/FSC decode the fault (ARM ch. 29):
       EC 0x25 = data abort, same EL
       FSC 0x04 = level 0 translation fault (nothing mapped)
       WnR = 0  = it was a READ

  3. pc : mydev_read+0x48/0x1c0 [mydev]
              ^^^^^^^^^^ ^^^^ ^^^^^  ^^^^^
              function   offset size  module
     -> THIS IS THE INSTRUCTION THAT FAULTED. Everything
        else is context.

  4. THE CALL TRACE, read bottom-up, is how you got there:
     userspace read() -> syscall -> vfs_read -> your function.

  5. "Tainted: G O" - O means an out-of-tree module is
     loaded. Other letters matter too (chapter 15).

  6. Code: shows the instructions around the fault, with the
     faulting one in (parentheses).

  THE ONE NUMBER THAT MATTERS: +0x48 in mydev_read. Turn it
  into a source line:
```

```
# Function + offset to file and line:
    aarch64-linux-gnu-addr2line -e mydev.ko -f -i 0x48
      mydev_read
      /home/you/mydev/mydev.c:142

# For a built-in function, use vmlinux and the absolute
# address (with nokaslr, or after adjusting for the offset):
    aarch64-linux-gnu-addr2line -e vmlinux -f -i ffff800008123456

# THE BEST TOOL - decodes a whole oops at once:
    ./scripts/decode_stacktrace.sh vmlinux . < oops.txt
# It annotates every frame with file:line. Paste an oops in,
# get a readable trace out. Learn this one command.

# Or disassemble around the offset:
    aarch64-linux-gnu-objdump -dS --start-address=0x30 \
        --stop-address=0x60 mydev.ko

# KEEP vmlinux AND THE UNSTRIPPED .ko from the exact build
# that is running. Without them an oops is just numbers.
```

NULL-PLUS-OFFSET TELLS YOU WHICH MEMBER

A fault at `0x18` means the code did `ptr->something` where `something` is 24 bytes into the struct and `ptr` was NULL. Use `pahole -C mydev mydev.ko` to print the struct layout with offsets, and the member is named directly. That usually identifies the bug faster than reading the source.

The other crash messages

Message	Means	First thing to check
`NULL pointer dereference`	Unchecked return, or use-after-free that poisoned the pointer	The offset, via `pahole`
`Unable to handle paging request`	A wild pointer	Whether the address looks like a poison pattern
`BUG: sleeping function called from invalid context`	Slept while atomic	The backtrace's caller (ch. 06)
`BUG: scheduling while atomic`	Same class	Locks held at the time
`WARNING: at ...`	A `WARN_ON` fired — a bug the kernel survived	The condition at that line
`soft lockup - CPU stuck for 22s`	A loop with preemption off	The looping function in the trace
`hard LOCKUP`	Interrupts disabled too long	Same
`INFO: task blocked for more than 120 seconds`	D state (ch. 09)	`/proc/PID/stack`
`rcu_sched self-detected stall`	No quiescent state	A loop or deadlock (ch. 12)
`possible circular locking dependency`	Lockdep found an AB-BA	Both lock sites it prints
`Kernel panic - not syncing`	Unrecoverable	The line above the panic

```
/* POISON PATTERNS - a faulting address that is one of these
   tells you immediately what kind of bug it is: */

   0x6b6b6b6b6b6b6b6b   SLAB_POISON: freed memory, reused
   0xa5a5a5a5a5a5a5a5   uninitialised slab memory
   0x5a5a5a5a5a5a5a5a   uninitialised
   0xdead000000000100   LIST_POISON1 - a list_head that was
                        deleted, then used
   0xdead000000000122   LIST_POISON2
   0xffffffffffffffff   often an ERR_PTR treated as a pointer

/* "Unable to handle kernel paging request at dead000000000100"
   means: you used a list entry after list_del(). That is a
   complete diagnosis from the address alone.              */
```

The config options that do the work

```
# A bug that KASAN catches at the moment it happens is a
# ten-minute fix. The same bug found by its downstream crash
# is a week. Enable these in every development build.

CONFIG_KASAN=y                  # use-after-free, OOB access
CONFIG_KASAN_INLINE=y           # faster instrumentation
CONFIG_KFENCE=y                 # cheap sampling version,
                                # usable in production
CONFIG_UBSAN=y                  # shifts, overflow, alignment
CONFIG_DEBUG_KMEMLEAK=y         # unreferenced allocations
CONFIG_PROVE_LOCKING=y          # lockdep
CONFIG_DEBUG_ATOMIC_SLEEP=y     # sleeping in atomic
CONFIG_DEBUG_LIST=y             # corrupted list_head
CONFIG_DEBUG_OBJECTS=y          # misused timers, work items
CONFIG_DEBUG_OBJECTS_TIMERS=y
CONFIG_DMA_API_DEBUG=y          # DMA misuse (chapter 24)
CONFIG_DEBUG_SG=y               # scatterlist misuse
CONFIG_DETECT_HUNG_TASK=y
CONFIG_PANIC_ON_OOPS=y          # fail loudly, do not limp on
CONFIG_DEBUG_INFO_DWARF5=y      # symbols for gdb/addr2line
CONFIG_GDB_SCRIPTS=y
CONFIG_MAGIC_SYSRQ=y
CONFIG_FTRACE=y
CONFIG_FUNCTION_GRAPH_TRACER=y
CONFIG_DYNAMIC_DEBUG=y

# A KASAN report is the best diagnostic in the kernel: it
# gives you the bad access, the allocation site, AND the free
# site, each with a full backtrace.

  BUG: KASAN: use-after-free in mydev_read+0x120/0x2a0 [mydev]
  Read of size 4 at addr ffff0000c1234018 by task mytest/431
  Allocated by task 430:
   mydev_probe+0x88/0x400 [mydev]
  Freed by task 431:
   mydev_release+0x40/0x80 [mydev]
   ^^^ there is your bug, in three lines
```

ftrace

```
# Mounted at /sys/kernel/tracing (or debug/tracing).

# --- WHICH FUNCTIONS RUN, and how long ---
    cd /sys/kernel/tracing
    echo function_graph > current_tracer
    echo 'mydev_*'      > set_ftrace_filter
    echo 1 > tracing_on
    ... run the thing ...
    echo 0 > tracing_on
    cat trace

  2)               |  mydev_read() {
  2)   0.520 us    |    mydev_rd();
  2) + 15.240 us   |    wait_event_interruptible();
  2)   1.100 us    |    copy_to_user();
  2) + 18.140 us   |  }

# The durations are real. This is a profiler you did not have
# to instrument.

# --- WHO CALLS THIS FUNCTION ---
    echo 1 > options/func_stack_trace
    echo mydev_reset > set_ftrace_filter
    echo function > current_tracer
    cat trace         # every call, with a full backtrace

# --- EVENTS, cheaper and structured ---
    echo 1 > events/irq/enable
    echo 1 > events/sched/sched_switch/enable
    echo 1 > events/regmap/enable
    cat trace_pipe                    # live stream

# --- LATENCY: find the longest critical section ---
    echo preemptirqsoff > current_tracer
    echo 0 > tracing_max_latency
    ... run the load ...
    cat trace     # a backtrace of the WORST offender

# --- KPROBES: trace a function without recompiling ---
    echo 'p:myprobe mydev_read dev=%x0 count=%x2' \
         > kprobe_events
    echo 1 > events/kprobes/myprobe/enable
    cat trace_pipe

# --- trace-cmd, which is much nicer than the raw files ---
    trace-cmd record -p function_graph -l 'mydev_*' -- sleep 5
    trace-cmd report
    kernelshark                       # graphical

# --- A ONE-SHOT SNAPSHOT from inside your driver ---
    trace_printk("state=%d count=%d\n", state, count);
# Writes into the ftrace ring buffer, orders of magnitude
# faster than printk, and never blocks. Ideal in a hot path or
# an interrupt handler. Remove them before submitting - the
# kernel prints a banner warning if any remain.
```

A real debugger

```
# --- QEMU: the easy case (chapter 16) ---
    qemu-system-aarch64 ... -s -S
    # in another terminal:
    aarch64-linux-gnu-gdb vmlinux
    (gdb) target remote :1234
    (gdb) hbreak start_kernel
    (gdb) continue

# --- Debugging a MODULE: gdb needs to know where it landed ---
    # on the target:
    cat /sys/module/mydev/sections/.text
      0xffff800008a00000
    # in gdb:
    (gdb) add-symbol-file mydev.ko 0xffff800008a00000
    (gdb) break mydev_read
    (gdb) continue

# --- The kernel's own gdb helpers (CONFIG_GDB_SCRIPTS) ---
    (gdb) lx-dmesg              # the kernel log
    (gdb) lx-lsmod              # loaded modules
    (gdb) lx-ps                 # every task
    (gdb) lx-symbols            # AUTO-loads module symbols
    (gdb) lx-device-tree        # the DT
    (gdb) lx-iomem
# lx-symbols alone justifies enabling the option.

# --- KGDB on real hardware, over serial ---
    # kernel cmdline:
    kgdboc=ttyS0,115200 kgdbwait
    # the kernel waits for a debugger at boot
    (gdb) target remote /dev/ttyUSB0
    # or break in at any time:
    echo g > /proc/sysrq-trigger

# --- crash, for post-mortem on a dump ---
    crash vmlinux vmcore
    crash> bt                   # backtrace
    crash> ps
    crash> log
    crash> struct mydev ffff0000c1234000

# CAPTURING a dump needs kdump/kexec configured in advance -
# set it up before you need it, not after.
```

SysRq: when the system is nearly dead

```
# Over serial: send a BREAK, then the letter.
# From a shell: echo <letter> > /proc/sysrq-trigger

    l   backtrace of ALL CPUs         <-- for a hang
    w   backtrace of all BLOCKED tasks <-- for D states
    t   backtrace of every task
    m   memory statistics
    d   show all locks held            <-- for a deadlock
    p   registers
    c   crash deliberately (to trigger kdump)
    g   enter kgdb
    9   set loglevel to maximum
    s   sync disks
    u   remount read-only
    b   reboot immediately

# THE SEQUENCE FOR A HANG YOU CANNOT LOG INTO:
    <break>l      # what is every CPU doing?
    <break>w      # what is blocked?
    <break>d      # who holds what?
# Those three usually identify the culprit without any other
# tooling, and they work when nothing else does.

# Enable it: CONFIG_MAGIC_SYSRQ=y and
    echo 1 > /proc/sys/kernel/sysrq
```

A method

```
   1. MAKE IT REPRODUCE. A bug you can trigger on demand is
      most of the way solved. Script it.

   2. TURN ON THE DEBUG OPTIONS and reproduce again. KASAN,
      lockdep and DMA_API_DEBUG frequently name the bug
      outright, and you have just skipped steps 3-6.

   3. READ THE WHOLE MESSAGE. The first line, the ESR, the
      offset, the taint flags, the poison pattern. Most oopses
      contain more diagnosis than people extract.

   4. LOCALISE IN TIME. Was it always broken, or did it start?
      git bisect on the kernel, or on your driver.

   5. LOCALISE IN CODE. ftrace function_graph on your module
      shows exactly which function last ran.

   6. INSTRUMENT. trace_printk or a trace event around the
      suspect region. Cheap, fast, and safe in atomic context.

   7. CHANGE ONE THING. Then check. Never two.

   8. UNDERSTAND WHY IT WORKED. A fix you cannot explain is a
      timing change that moved the bug somewhere else - which
      is the most expensive kind of "fix" there is.
```

How it shows up

Symptom	Approach
Reproducible oops	`decode_stacktrace.sh`, then read the offset
Crash far from the cause	KASAN — it reports at the moment of the bad access
Memory grows over time	kmemleak, `slabtop`
Intermittent corruption	KASAN plus `slub_debug=FZPU`
Hang with no output	SysRq `l`, `w`, `d`
Process stuck in D	`/proc/PID/stack`
Deadlock	lockdep — it usually reports before the deadlock happens
Latency spike	`preemptirqsoff` tracer
Works, but slowly	`function_graph` with durations, or `perf`
Boot hang	`initcall_debug`, `earlycon` (ch. 03)
Regression after an upgrade	`git bisect` on the kernel
Only on production hardware	kdump, or serial logging with `keep_bootcon`

Check yourself

- ✓ Read an oops and name the faulting function, offset and call path.
- ✓ Turn a function+offset into a source line, three different ways.
- ✓ Use a NULL-plus-offset fault and `pahole` to name the struct member.
- ✓ Recognise five poison patterns and say what each implies.
- ✓ List ten debug config options and what each catches.
- ✓ Read a KASAN report and identify the allocation and free sites.
- ✓ Use `function_graph` to profile your driver's entry points.
- ✓ Add a kprobe to a function you did not compile in.
- ✓ Attach gdb to a module and set a breakpoint in it.
- ✓ Diagnose a hang using only SysRq.

Project: the ring-buffer character driver

A complete, working Linux driver you build, load and break yourself. No hardware needed — it runs on QEMU or any Linux machine. It exercises character devices, blocking I/O, poll, ioctl, locking, sysfs and correct teardown: nearly everything in the last twenty chapters.

What you are building

```
   /dev/ringbuf         a character device

   write(fd, data, n)   -> push bytes into a circular buffer
   read(fd, buf, n)     -> pop bytes out; BLOCKS if empty
   poll(fd, POLLIN)     -> wakes when data arrives
   ioctl(RB_GET_STATS)  -> how many bytes, drops, wakeups
   ioctl(RB_RESET)      -> empty it
   /sys/class/ringbuf/ringbuf/size     -> capacity, read-only
   /sys/class/ringbuf/ringbuf/used     -> bytes in it now

  THE POINT is not the ring buffer - it is everything around
  it: the concurrency between a reader that sleeps and a writer
  that wakes it, the teardown that must not race, the ioctl
  ABI, the error paths. That is what a real driver is made of;
  the hardware part is usually the easy half.
```

The files

```
ringbuf/
  ringbuf.c          the driver
  ringbuf.h          the UAPI header, shared with userspace
  Makefile
  test/
    rbtest.c         a userspace exerciser
    Makefile
```

ringbuf.h — the ABI

```
/* SPDX-License-Identifier: GPL-2.0 */
#ifndef _UAPI_RINGBUF_H
#define _UAPI_RINGBUF_H

#include <linux/types.h>
#include <linux/ioctl.h>

struct rb_stats {
        __u32 capacity;         /* total bytes                */
        __u32 used;             /* bytes currently held       */
        __u64 bytes_written;    /* since load                 */
        __u64 bytes_read;
        __u64 drops;            /* bytes discarded when full  */
        __u64 wakeups;          /* reader wakeups             */
        __u32 reserved[8];      /* MUST be zero (chapter 21)  */
};

#define RB_IOC_MAGIC    'R'
#define RB_GET_STATS    _IOR(RB_IOC_MAGIC, 1, struct rb_stats)
#define RB_RESET        _IO (RB_IOC_MAGIC, 2)

#endif /* _UAPI_RINGBUF_H */
```

ringbuf.c

```
// SPDX-License-Identifier: GPL-2.0
/*
 * ringbuf - a teaching character driver.
 *
 * Locking: rb->lock (spinlock, irqsave) protects head, tail
 * and every counter. It is never held across a copy to or
 * from user space - see rb_read()/rb_write() for how the
 * copy is staged outside the lock.
 */

#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/module.h>
#include <linux/poll.h>
#include <linux/slab.h>
#include <linux/uaccess.h>

#include "ringbuf.h"

#define RB_DEFAULT_SIZE  4096
#define RB_CHUNK         1024        /* staging buffer size */

static unsigned int bufsize = RB_DEFAULT_SIZE;
module_param(bufsize, uint, 0444);
MODULE_PARM_DESC(bufsize, "Ring capacity in bytes (power of 2)");

static bool overwrite;
module_param(overwrite, bool, 0644);
MODULE_PARM_DESC(overwrite,
                 "Drop oldest data instead of blocking when full");

struct rb_dev {
        struct cdev        cdev;
        struct device     *dev;
        dev_t              devt;

        u8                *buf;
        u32                capacity;     /* power of two   */
        u32                head;          /* write index    */
        u32                tail;          /* read index     */

        spinlock_t         lock;
        wait_queue_head_t  readq;
        wait_queue_head_t  writeq;

        u64                bytes_written;
        u64                bytes_read;
        u64                drops;
        u64                wakeups;
};

static struct rb_dev  *rb_device;
static struct class   *rb_class;

/* ---- buffer maths, callers hold rb->lock ---------------- */

static inline u32 rb_used(struct rb_dev *rb)
{
        return rb->head - rb->tail;      /* unsigned wrap is
```

```
                                          well-defined and
                                          is the trick that
                                          makes this simple */
}

static inline u32 rb_space(struct rb_dev *rb)
{
        return rb->capacity - rb_used(rb);
}

static inline bool rb_empty(struct rb_dev *rb)
{
        return rb->head == rb->tail;
}

/* Note: head and tail are FREE-RUNNING counters, masked only
   when indexing. That removes the classic "is it full or
   empty?" ambiguity of a wrapped-index ring entirely. */

/* ---- copy in, caller holds the lock ---------------------- */

static u32 rb_push(struct rb_dev *rb, const u8 *src, u32 len)
{
        u32 n, first;

        if (overwrite) {
                if (len > rb->capacity) {       /* keep the tail */
                        src += len - rb->capacity;
                        rb->drops += len - rb->capacity;
                        len = rb->capacity;
                }
                if (len > rb_space(rb)) {
                        u32 discard = len - rb_space(rb);

                        rb->tail += discard;    /* drop oldest */
                        rb->drops += discard;
                }
                n = len;
        } else {
                n = min(len, rb_space(rb));
        }

        /* the ring wraps: up to two memcpys */
        first = min(n, rb->capacity - (rb->head & (rb->capacity - 1)));
        memcpy(rb->buf + (rb->head & (rb->capacity - 1)), src, first);
        if (n > first)
                memcpy(rb->buf, src + first, n - first);

        rb->head += n;
        rb->bytes_written += n;
        return n;
}

static u32 rb_pop(struct rb_dev *rb, u8 *dst, u32 len)
{
        u32 n, first;

        n = min(len, rb_used(rb));

        first = min(n, rb->capacity - (rb->tail & (rb->capacity - 1)));
        memcpy(dst, rb->buf + (rb->tail & (rb->capacity - 1)), first);
        if (n > first)
                memcpy(dst + first, rb->buf, n - first);
```

```
        rb->tail += n;
        rb->bytes_read += n;
        return n;
}

/* ---- file operations ---------------------------------- */

static int rb_open(struct inode *inode, struct file *file)
{
        struct rb_dev *rb = container_of(inode->i_cdev,
                                         struct rb_dev, cdev);

        file->private_data = rb;
        return 0;
}

static ssize_t rb_read(struct file *file, char __user *ubuf,
                       size_t count, loff_t *ppos)
{
        struct rb_dev *rb = file->private_data;
        u8 *stage;
        size_t done = 0;
        unsigned long flags;
        int ret = 0;

        if (!count)
                return 0;

        stage = kmalloc(RB_CHUNK, GFP_KERNEL);
        if (!stage)
                return -ENOMEM;

        while (done < count) {
                u32 n;

                spin_lock_irqsave(&rb->lock, flags);
                n = rb_pop(rb, stage, min_t(size_t,
                                            count - done, RB_CHUNK));
                spin_unlock_irqrestore(&rb->lock, flags);

                if (n) {
                        /* copy OUTSIDE the lock - copy_to_user
                           can sleep on a page fault, which is
                           illegal under a spinlock (ch. 06) */
                        if (copy_to_user(ubuf + done, stage, n)) {
                                ret = -EFAULT;
                                goto out;
                        }
                        done += n;
                        wake_up_interruptible(&rb->writeq);
                        continue;
                }

                /* nothing available */
                if (done)
                        break;          /* return what we have */

                if (file->f_flags & O_NONBLOCK) {
                        ret = -EAGAIN;
                        goto out;
                }
```

```
                ret = wait_event_interruptible(rb->readq,
                                               !rb_empty(rb));
                if (ret) {
                        ret = -ERESTARTSYS;   /* no progress yet */
                        goto out;
                }
        }

        ret = done;
out:
        kfree(stage);
        return ret;
}

static ssize_t rb_write(struct file *file, const char __user *ubuf,
                        size_t count, loff_t *ppos)
{
        struct rb_dev *rb = file->private_data;
        u8 *stage;
        size_t done = 0;
        unsigned long flags;
        int ret = 0;

        if (!count)
                return 0;

        stage = kmalloc(RB_CHUNK, GFP_KERNEL);
        if (!stage)
                return -ENOMEM;

        while (done < count) {
                size_t chunk = min_t(size_t, count - done, RB_CHUNK);
                u32 n;

                if (copy_from_user(stage, ubuf + done, chunk)) {
                        ret = -EFAULT;
                        goto out;
                }

                spin_lock_irqsave(&rb->lock, flags);
                n = rb_push(rb, stage, chunk);
                if (n)
                        rb->wakeups++;
                spin_unlock_irqrestore(&rb->lock, flags);

                if (n) {
                        done += n;
                        wake_up_interruptible(&rb->readq);
                        continue;
                }

                /* full, and not overwriting */
                if (done)
                        break;

                if (file->f_flags & O_NONBLOCK) {
                        ret = -EAGAIN;
                        goto out;
                }

                ret = wait_event_interruptible(rb->writeq,
                                               rb_space(rb) > 0);
                if (ret) {
```

```
                        ret = -ERESTARTSYS;
                        goto out;
                }
        }

        ret = done;
out:
        kfree(stage);
        return ret;
}

static __poll_t rb_poll(struct file *file, poll_table *wait)
{
        struct rb_dev *rb = file->private_data;
        __poll_t mask = 0;
        unsigned long flags;

        /* register on BOTH queues BEFORE checking (ch. 20) */
        poll_wait(file, &rb->readq,  wait);
        poll_wait(file, &rb->writeq, wait);

        spin_lock_irqsave(&rb->lock, flags);
        if (!rb_empty(rb))
                mask |= EPOLLIN | EPOLLRDNORM;
        if (rb_space(rb) > 0)
                mask |= EPOLLOUT | EPOLLWRNORM;
        spin_unlock_irqrestore(&rb->lock, flags);

        return mask;
}

static long rb_ioctl(struct file *file, unsigned int cmd,
                     unsigned long arg)
{
        struct rb_dev *rb = file->private_data;
        void __user *uarg = (void __user *)arg;
        struct rb_stats st;
        unsigned long flags;

        switch (cmd) {
        case RB_GET_STATS:
                memset(&st, 0, sizeof(st));   /* zero the
                                                 reserved
                                                 fields too   */
                spin_lock_irqsave(&rb->lock, flags);
                st.capacity      = rb->capacity;
                st.used          = rb_used(rb);
                st.bytes_written = rb->bytes_written;
                st.bytes_read    = rb->bytes_read;
                st.drops         = rb->drops;
                st.wakeups       = rb->wakeups;
                spin_unlock_irqrestore(&rb->lock, flags);

                if (copy_to_user(uarg, &st, sizeof(st)))
                        return -EFAULT;
                return 0;

        case RB_RESET:
                if (!(file->f_mode & FMODE_WRITE))
                        return -EBADF;      /* chapter 21 */

                spin_lock_irqsave(&rb->lock, flags);
                rb->head = rb->tail = 0;
```

```
                spin_unlock_irqrestore(&rb->lock, flags);

                wake_up_interruptible(&rb->writeq);
                return 0;

        default:
                return -ENOTTY;
        }
}

static const struct file_operations rb_fops = {
        .owner          = THIS_MODULE,
        .open           = rb_open,
        .read           = rb_read,
        .write          = rb_write,
        .poll           = rb_poll,
        .unlocked_ioctl = rb_ioctl,
        .compat_ioctl   = compat_ptr_ioctl,
        .llseek         = no_llseek,
};

/* ---- sysfs ------------------------------------------------ */

static ssize_t size_show(struct device *dev,
                         struct device_attribute *attr, char *buf)
{
        struct rb_dev *rb = dev_get_drvdata(dev);

        return sysfs_emit(buf, "%u\n", rb->capacity);
}
static DEVICE_ATTR_RO(size);

static ssize_t used_show(struct device *dev,
                         struct device_attribute *attr, char *buf)
{
        struct rb_dev *rb = dev_get_drvdata(dev);
        unsigned long flags;
        u32 used;

        spin_lock_irqsave(&rb->lock, flags);
        used = rb_used(rb);
        spin_unlock_irqrestore(&rb->lock, flags);

        return sysfs_emit(buf, "%u\n", used);
}
static DEVICE_ATTR_RO(used);

static struct attribute *rb_attrs[] = {
        &dev_attr_size.attr,
        &dev_attr_used.attr,
        NULL,
};
ATTRIBUTE_GROUPS(rb);

/* ---- module init / exit ----------------------------------- */

static int __init rb_init(void)
{
        struct rb_dev *rb;
        int ret;

        if (!is_power_of_2(bufsize) || bufsize < 64) {
                pr_err("ringbuf: bufsize must be a power of 2, >= 64\n");
```

```
                return -EINVAL;
        }

        rb = kzalloc(sizeof(*rb), GFP_KERNEL);
        if (!rb)
                return -ENOMEM;

        rb->capacity = bufsize;
        rb->buf = kzalloc(bufsize, GFP_KERNEL);
        if (!rb->buf) {
                ret = -ENOMEM;
                goto err_free_dev;
        }

        spin_lock_init(&rb->lock);
        init_waitqueue_head(&rb->readq);
        init_waitqueue_head(&rb->writeq);

        ret = alloc_chrdev_region(&rb->devt, 0, 1, "ringbuf");
        if (ret)
                goto err_free_buf;

        rb_class = class_create("ringbuf");
        if (IS_ERR(rb_class)) {
                ret = PTR_ERR(rb_class);
                goto err_region;
        }
        rb_class->dev_groups = rb_groups;

        cdev_init(&rb->cdev, &rb_fops);
        rb->cdev.owner = THIS_MODULE;

        /* everything above must be ready: cdev_add publishes
           it and open() can arrive immediately (chapter 20) */
        ret = cdev_add(&rb->cdev, rb->devt, 1);
        if (ret)
                goto err_class;

        rb->dev = device_create(rb_class, NULL, rb->devt, rb,
                                "ringbuf");
        if (IS_ERR(rb->dev)) {
                ret = PTR_ERR(rb->dev);
                goto err_cdev;
        }

        rb_device = rb;
        pr_info("ringbuf: ready, %u bytes, major %d\n",
                rb->capacity, MAJOR(rb->devt));
        return 0;

err_cdev:
        cdev_del(&rb->cdev);
err_class:
        class_destroy(rb_class);
err_region:
        unregister_chrdev_region(rb->devt, 1);
err_free_buf:
        kfree(rb->buf);
err_free_dev:
        kfree(rb);
        return ret;
}
```

```
static void __exit rb_exit(void)
{
        struct rb_dev *rb = rb_device;

        /* exact reverse. cdev_del first: no new opens.
           .owner = THIS_MODULE means existing opens already
           hold a module reference, so rmmod cannot even reach
           here while an fd is open (chapter 15).           */
        device_destroy(rb_class, rb->devt);
        cdev_del(&rb->cdev);
        class_destroy(rb_class);
        unregister_chrdev_region(rb->devt, 1);
        kfree(rb->buf);
        kfree(rb);

        pr_info("ringbuf: unloaded\n");
}

module_init(rb_init);
module_exit(rb_exit);

MODULE_DESCRIPTION("Teaching ring-buffer character device");
MODULE_AUTHOR("Embedded Learning Hub");
MODULE_LICENSE("GPL");
```

Makefile

```
obj-m += ringbuf.o

KDIR ?= /lib/modules/$(shell uname -r)/build

all:
        $(MAKE) -C $(KDIR) M=$(PWD) modules

# cross-compile for an arm64 target:
arm64:
        $(MAKE) -C $(KDIR) M=$(PWD) ARCH=arm64 \
                CROSS_COMPILE=aarch64-linux-gnu- modules

clean:
        $(MAKE) -C $(KDIR) M=$(PWD) clean

check:
        $(KDIR)/scripts/checkpatch.pl --no-tree -f ringbuf.c

.PHONY: all arm64 clean check
```

The exerciser

```
/* test/rbtest.c - build with: gcc -O2 -o rbtest rbtest.c -lpthread */
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include "../ringbuf.h"

#define DEV "/dev/ringbuf"

static void show_stats(int fd)
{
        struct rb_stats st;

        if (ioctl(fd, RB_GET_STATS, &st) < 0) {
                perror("RB_GET_STATS");
                return;
        }
        printf("cap=%u used=%u written=%llu read=%llu drops=%llu\n",
               st.capacity, st.used,
               (unsigned long long)st.bytes_written,
               (unsigned long long)st.bytes_read,
               (unsigned long long)st.drops);
}

/* --- test 1: round trip ----------------------------------- */
static void test_roundtrip(void)
{
        char out[256], in[256];
        int fd = open(DEV, O_RDWR);
        ssize_t n;

        memset(out, 'A', sizeof(out));
        n = write(fd, out, sizeof(out));
        printf("wrote %zd\n", n);

        n = read(fd, in, sizeof(in));
        printf("read  %zd, match=%s\n", n,
               memcmp(out, in, n) == 0 ? "yes" : "NO");

        show_stats(fd);
        close(fd);
}

/* --- test 2: non-blocking on empty ------------------------ */
static void test_nonblock(void)
{
        char buf[16];
        int fd = open(DEV, O_RDWR | O_NONBLOCK);
        ssize_t n = read(fd, buf, sizeof(buf));

        printf("nonblock read: %zd errno=%d (%s) - expect EAGAIN\n",
               n, errno, strerror(errno));
        close(fd);
}
```

```
/* --- test 3: producer/consumer with poll ---------------- */
static void *producer(void *arg)
{
        int fd = open(DEV, O_WRONLY);
        char buf[64];
        int i;

        for (i = 0; i < 100; i++) {
                int len = snprintf(buf, sizeof(buf),
                                   "message %d\n", i);
                if (write(fd, buf, len) < 0) {
                        perror("write");
                        break;
                }
                usleep(1000);
        }
        close(fd);
        return NULL;
}

static void test_poll(void)
{
        pthread_t th;
        struct pollfd pfd;
        char buf[128];
        int fd = open(DEV, O_RDONLY);
        int total = 0;

        pthread_create(&th, NULL, producer, NULL);

        pfd.fd = fd;
        pfd.events = POLLIN;

        for (;;) {
                int ret = poll(&pfd, 1, 500);

                if (ret <= 0)
                        break;                  /* timeout: done */

                ret = read(fd, buf, sizeof(buf));
                if (ret <= 0)
                        break;
                total += ret;
        }
        printf("poll test: received %d bytes\n", total);

        pthread_join(th, NULL);
        close(fd);
}

/* --- test 4: bad ioctl ---------------------------------- */
static void test_bad_ioctl(void)
{
        int fd = open(DEV, O_RDWR);
        int ret = ioctl(fd, _IO('Z', 99));

        printf("bad ioctl: %d errno=%d (%s) - expect ENOTTY\n",
               ret, errno, strerror(errno));

        /* RESET on a read-only fd must be refused */
        close(fd);
        fd = open(DEV, O_RDONLY);
        ret = ioctl(fd, RB_RESET);
```

```
        printf("reset on O_RDONLY: %d errno=%d - expect EBADF\n",
               ret, errno);
        close(fd);
}

int main(void)
{
        printf("--- roundtrip ---\n");   test_roundtrip();
        printf("--- nonblock ---\n");    test_nonblock();
        printf("--- poll ---\n");        test_poll();
        printf("--- bad ioctl ---\n");   test_bad_ioctl();
        return 0;
}
```

Building and running it

```
# On the machine you will run it on (or cross-compile):
    cd ringbuf
    make
    sudo insmod ringbuf.ko
    dmesg | tail -2
      ringbuf: ready, 4096 bytes, major 241

    ls -l /dev/ringbuf
    cat /sys/class/ringbuf/ringbuf/size
      4096

# The shell can drive it directly:
    sudo sh -c 'echo hello > /dev/ringbuf'
    cat /sys/class/ringbuf/ringbuf/used
      6
    sudo cat /dev/ringbuf        # blocks after "hello" - Ctrl-C

# Run the exerciser:
    cd test && make && sudo ./rbtest

# Try the parameters:
    sudo rmmod ringbuf
    sudo insmod ringbuf.ko bufsize=65536
    echo 1 | sudo tee /sys/module/ringbuf/parameters/overwrite
```

Exercises — where the learning is

DO THESE IN ORDER. EACH ONE BREAKS SOMETHING YOU MUST THEN DIAGNOSE.

1. **Prove the lock matters.** Delete the `spin_lock_irqsave` pairs. Run two writers and a reader concurrently. Watch the counters drift from reality. Then put it back.
2. **Break the copy-outside-the-lock rule.** Move `copy_to_user` inside the spinlock. Build with `CONFIG_DEBUG_ATOMIC_SLEEP=y` and read the BUG message.
3. **Create a lost wakeup.** Replace `wait_event_interruptible` with a manual `set_current_state` / `schedule` that checks the condition *before* queueing. Reproduce the hang.
4. **Remove `.poll`.** Run `rbtest` and watch it spin at 100% CPU.
5. **Remove `poll_wait` on `writeq`.** Fill the buffer, then poll for `POLLOUT`. Observe that it never fires.
6. **Leak the buffer.** Delete `kfree(rb->buf)` from `rb_exit`. Load and unload 50 times with `CONFIG_DEBUG_KMEMLEAK=y`, then `echo scan > /sys/kernel/debug/kmemleak`.
7. **Use after free.** Move `kfree(rb)` before `cdev_del`. Hold an fd open, unload, and read. With KASAN you get a report naming both sites.
8. **Drop the `f_mode` check** in `RB_RESET` and confirm a read-only fd can now clear the buffer — a real privilege bug, in four lines.
9. **Add a feature:** a `RB_SET_SIZE` ioctl that reallocates the buffer. Think carefully about what must be locked, what happens to data in flight, and what a concurrent blocked reader sees. This is harder than it looks, which is the point.
10. **Add a second minor** so `/dev/ringbuf0` and `/dev/ringbuf1` are independent buffers. This converts the module-global design into a per-device one — the shape every real driver needs.

Reviewing your own work

```
# Run all of these before you consider it finished:

# 1. Style
    make check                      # checkpatch.pl

# 2. Sparse - __user annotations, endianness
    make C=1 M=$PWD

# 3. Load/unload 100 times under KASAN
    for i in $(seq 100); do
      sudo insmod ringbuf.ko && sudo rmmod ringbuf || break
    done
    dmesg | grep -i -e kasan -e bug -e warn

# 4. Concurrency: several writers and readers at once
    for i in 1 2 3 4; do
      (sudo dd if=/dev/urandom of=/dev/ringbuf bs=512 \
            count=1000 2>/dev/null) &
    done
    sudo dd if=/dev/ringbuf of=/dev/null bs=512 count=4000

# 5. Fault injection on the allocations
    echo 10  | sudo tee /sys/kernel/debug/failslab/probability
    echo 100 | sudo tee /sys/kernel/debug/failslab/times
    # load repeatedly; nothing should crash

# 6. rmmod while an fd is open - must return EBUSY
    exec 3</dev/ringbuf
    sudo rmmod ringbuf            # expect: Device or resource busy
    exec 3<&-

# THE ONES THAT USUALLY FIND SOMETHING are 3, 4 and 5.
```

Check yourself

- ✓ Explain why free-running head and tail counters avoid the full/empty ambiguity.
- ✓ Say why the user copy happens outside the spinlock, and what happens if it does not.
- ✓ Explain why `poll` registers on both wait queues before checking anything.
- ✓ Explain the partial-read rule: why `done > 0` returns rather than blocking again.
- ✓ Justify the order of every step in `rb_init` and `rb_exit`.
- ✓ Say what `.owner = THIS_MODULE` prevents, and demonstrate it.
- ✓ Complete exercises 1 through 8 and describe the symptom each produced.
- ✓ Implement exercise 9 or 10 and defend your locking.
- ✓ Run the full six-step review with no findings.

> **NEXT**
> Chapter 29 takes the same driver and attaches it to real hardware description — a device tree node, MMIO registers, an interrupt and DMA — turning it from a software toy into the shape of a shipping driver.

Project: the device-tree platform driver

The second project: a driver bound to a real device-tree node, driving MMIO registers, handling an interrupt, and exposing samples through a character device. Built and tested on QEMU against a device that actually exists — so everything you learn transfers directly to silicon.

The target device

```
We drive the PL031 real-time clock on QEMU's `virt` machine.
It is a real ARM PrimeCell, documented in a public TRM, present
on countless boards - and simple enough to fit in one chapter
while still having every element that matters:

     memory-mapped registers
     an interrupt
     a device-tree binding
     clocks
     a userspace interface

  REGISTER MAP (base 0x09010000 on qemu virt):

     0x00  RTCDR    data      - the current count (RO)
     0x04  RTCMR    match     - fire an interrupt at this value
     0x08  RTCLR    load      - set the counter
     0x0C  RTCCR    control   - bit 0: enable
     0x10  RTCIMSC  interrupt mask set/clear
     0x14  RTCRIS   raw interrupt status
     0x18  RTCMIS   masked interrupt status
     0x1C  RTCICR   interrupt clear (write 1)
     0xFE0 PeriphID0..3  - 0x31, 0x10, 0x14, 0x00
     0xFF0 PCellID0..3   - 0x0D, 0xF0, 0x05, 0xB1

  WHAT WE BUILD:  /dev/rtcalarm

     read()   blocks until the alarm fires, then returns the
              timestamp
     poll()   wakes on the alarm
     ioctl    set an alarm N seconds ahead; query state
     sysfs    current time, alarm time, count of alarms fired

  Linux already has a PL031 driver, so we bind to a DIFFERENT
  compatible string on a node we add ourselves. That keeps the
  real driver out of the way and teaches the binding mechanics
  properly.
```

Step 1: the device tree

```
# Get QEMU's own device tree, add our node, and feed it back.

# 1. dump what QEMU generates:
    qemu-system-aarch64 -M virt,dumpdtb=virt.dtb \
        -cpu cortex-a57 -nographic
    dtc -I dtb -O dts virt.dtb > virt.dts

# 2. find the existing pl031 node in virt.dts:
    pl031@9010000 {
        clock-names = "apb_pclk";
        clocks = <0x8000>;
        interrupts = <0x00 0x02 0x04>;
        reg = <0x00 0x9010000 0x00 0x1000>;
        compatible = "arm,pl031", "arm,primecell";
    };

# 3. CHANGE the compatible so OUR driver claims it, and add a
#    custom property to read:
    pl031@9010000 {
        clock-names = "apb_pclk";
        clocks = <0x8000>;
        interrupts = <0x00 0x02 0x04>;
        reg = <0x00 0x9010000 0x00 0x1000>;
        compatible = "hub,rtcalarm";           /* <-- ours */
        hub,default-alarm-seconds = <5>;       /* <-- custom */
    };

# 4. rebuild and boot with it:
    dtc -I dts -O dtb -o virt-mod.dtb virt.dts
    qemu-system-aarch64 -M virt -cpu cortex-a57 -m 512 \
      -dtb virt-mod.dtb \
      -kernel Image -initrd initramfs.cpio.gz \
      -append "console=ttyAMA0 earlycon rdinit=/init" \
      -nographic

# 5. confirm the device exists but nothing has claimed it:
    ls /sys/bus/platform/devices/ | grep 9010000
    ls -l /sys/bus/platform/devices/9010000.pl031/driver
      # No such file - no driver bound. Good.

# NOTE the interrupt: <0x00 0x02 0x04> = GIC_SPI 2, level high
# -> INTID 34 (chapter 14's off-by-32). You will see it in
# /proc/interrupts as hwirq 34.
```

Step 2: the driver

```
// SPDX-License-Identifier: GPL-2.0
/*
 * rtcalarm - a teaching platform driver for the PL031.
 *
 * Locking: d->lock (spinlock, irqsave) protects alarm_set,
 * alarm_time, fired and the counters. Taken from both process
 * context and the interrupt handler, hence irqsave.
 */

#include <linux/bitops.h>
#include <linux/clk.h>
#include <linux/interrupt.h>
#include <linux/io.h>
#include <linux/miscdevice.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/poll.h>
#include <linux/property.h>
#include <linux/uaccess.h>

#include "rtcalarm.h"

#define RTC_DR          0x00
#define RTC_MR          0x04
#define RTC_LR          0x08
#define RTC_CR          0x0c
#define  RTC_CR_EN      BIT(0)
#define RTC_IMSC        0x10
#define RTC_RIS         0x14
#define RTC_MIS         0x18
#define RTC_ICR         0x1c
#define RTC_PERIPH_ID0  0xfe0

#define PL031_PERIPH_ID0  0x31

struct rtcalarm {
        struct device      *dev;
        void __iomem       *base;
        struct clk         *clk;
        int                 irq;
        struct miscdevice   misc;

        spinlock_t          lock;
        wait_queue_head_t   wq;

        u32                 default_alarm_s;
        u32                 alarm_time;
        bool                alarm_set;
        bool                fired;
        u32                 fired_time;
        u64                 alarm_count;
};

static inline u32 ra_rd(struct rtcalarm *d, u32 reg)
{
        return readl(d->base + reg);
}

static inline void ra_wr(struct rtcalarm *d, u32 reg, u32 val)
```

```
{
        writel(val, d->base + reg);
}

/* ---- interrupt -------------------------------------------- */

static irqreturn_t rtcalarm_isr(int irq, void *dev_id)
{
        struct rtcalarm *d = dev_id;
        unsigned long flags;

        if (!(ra_rd(d, RTC_MIS) & 1))
                return IRQ_NONE;        /* not ours (ch. 14) */

        ra_wr(d, RTC_ICR, 1);           /* acknowledge FIRST */

        spin_lock_irqsave(&d->lock, flags);
        d->fired      = true;
        d->fired_time = ra_rd(d, RTC_DR);
        d->alarm_set  = false;
        d->alarm_count++;
        spin_unlock_irqrestore(&d->lock, flags);

        ra_wr(d, RTC_IMSC, 0);          /* mask until re-armed */
        wake_up_interruptible(&d->wq);

        return IRQ_HANDLED;
}

/* ---- arming ----------------------------------------------- */

static int rtcalarm_arm(struct rtcalarm *d, u32 seconds)
{
        unsigned long flags;
        u32 now, target;

        if (seconds == 0 || seconds > 86400)
                return -EINVAL;

        now = ra_rd(d, RTC_DR);
        target = now + seconds;

        spin_lock_irqsave(&d->lock, flags);
        d->alarm_time = target;
        d->alarm_set  = true;
        d->fired      = false;
        spin_unlock_irqrestore(&d->lock, flags);

        ra_wr(d, RTC_MR, target);
        ra_wr(d, RTC_ICR, 1);           /* clear any stale one */
        ra_wr(d, RTC_IMSC, 1);          /* unmask */

        dev_dbg(d->dev, "armed: now=%u target=%u (+%us)\n",
                now, target, seconds);
        return 0;
}

static void rtcalarm_disarm(struct rtcalarm *d)
{
        unsigned long flags;

        ra_wr(d, RTC_IMSC, 0);
        ra_wr(d, RTC_ICR, 1);
```

```
        spin_lock_irqsave(&d->lock, flags);
        d->alarm_set = false;
        spin_unlock_irqrestore(&d->lock, flags);
}

/* ---- file operations -------------------------------- */

static struct rtcalarm *to_rtcalarm(struct file *file)
{
        struct miscdevice *m = file->private_data;

        return container_of(m, struct rtcalarm, misc);
}

static ssize_t rtcalarm_read(struct file *file, char __user *ubuf,
                             size_t count, loff_t *ppos)
{
        struct rtcalarm *d = to_rtcalarm(file);
        struct ra_event ev;
        unsigned long flags;
        int ret;

        if (count < sizeof(ev))
                return -EINVAL;

        if (file->f_flags & O_NONBLOCK) {
                if (!READ_ONCE(d->fired))
                        return -EAGAIN;
        } else {
                ret = wait_event_interruptible(d->wq, d->fired);
                if (ret)
                        return -ERESTARTSYS;
        }

        memset(&ev, 0, sizeof(ev));
        spin_lock_irqsave(&d->lock, flags);
        ev.timestamp = d->fired_time;
        ev.count     = d->alarm_count;
        d->fired     = false;           /* consume it */
        spin_unlock_irqrestore(&d->lock, flags);

        if (copy_to_user(ubuf, &ev, sizeof(ev)))
                return -EFAULT;

        return sizeof(ev);
}

static __poll_t rtcalarm_poll(struct file *file, poll_table *wait)
{
        struct rtcalarm *d = to_rtcalarm(file);
        __poll_t mask = 0;

        poll_wait(file, &d->wq, wait);

        if (READ_ONCE(d->fired))
                mask |= EPOLLIN | EPOLLRDNORM;

        return mask;
}

static long rtcalarm_ioctl(struct file *file, unsigned int cmd,
                           unsigned long arg)
```

```
{
        struct rtcalarm *d = to_rtcalarm(file);
        void __user *uarg = (void __user *)arg;
        struct ra_state st;
        unsigned long flags;
        u32 seconds;

        switch (cmd) {
        case RA_SET_ALARM:
                if (!(file->f_mode & FMODE_WRITE))
                        return -EBADF;
                if (copy_from_user(&seconds, uarg, sizeof(seconds)))
                        return -EFAULT;
                return rtcalarm_arm(d, seconds);

        case RA_CANCEL_ALARM:
                if (!(file->f_mode & FMODE_WRITE))
                        return -EBADF;
                rtcalarm_disarm(d);
                return 0;

        case RA_GET_STATE:
                memset(&st, 0, sizeof(st));
                st.now = ra_rd(d, RTC_DR);

                spin_lock_irqsave(&d->lock, flags);
                st.alarm_time  = d->alarm_time;
                st.alarm_set   = d->alarm_set;
                st.fired       = d->fired;
                st.alarm_count = d->alarm_count;
                spin_unlock_irqrestore(&d->lock, flags);

                if (copy_to_user(uarg, &st, sizeof(st)))
                        return -EFAULT;
                return 0;

        default:
                return -ENOTTY;
        }
}

static const struct file_operations rtcalarm_fops = {
        .owner          = THIS_MODULE,
        .read           = rtcalarm_read,
        .poll           = rtcalarm_poll,
        .unlocked_ioctl = rtcalarm_ioctl,
        .compat_ioctl   = compat_ptr_ioctl,
        .llseek         = no_llseek,
};

/* ---- sysfs ------------------------------------------------ */

static ssize_t now_show(struct device *dev,
                        struct device_attribute *attr, char *buf)
{
        struct rtcalarm *d = dev_get_drvdata(dev);

        return sysfs_emit(buf, "%u\n", ra_rd(d, RTC_DR));
}
static DEVICE_ATTR_RO(now);

static ssize_t alarm_count_show(struct device *dev,
                                struct device_attribute *attr,
```

```
                                char *buf)
{
        struct rtcalarm *d = dev_get_drvdata(dev);
        unsigned long flags;
        u64 n;

        spin_lock_irqsave(&d->lock, flags);
        n = d->alarm_count;
        spin_unlock_irqrestore(&d->lock, flags);

        return sysfs_emit(buf, "%llu\n", n);
}
static DEVICE_ATTR_RO(alarm_count);

static ssize_t alarm_in_store(struct device *dev,
                              struct device_attribute *attr,
                              const char *buf, size_t count)
{
        struct rtcalarm *d = dev_get_drvdata(dev);
        unsigned int seconds;
        int ret;

        ret = kstrtouint(buf, 0, &seconds);
        if (ret)
                return ret;

        ret = rtcalarm_arm(d, seconds);
        if (ret)
                return ret;

        return count;           /* chapter 22 */
}
static DEVICE_ATTR_WO(alarm_in);

static struct attribute *rtcalarm_attrs[] = {
        &dev_attr_now.attr,
        &dev_attr_alarm_count.attr,
        &dev_attr_alarm_in.attr,
        NULL,
};
ATTRIBUTE_GROUPS(rtcalarm);

/* ---- probe ------------------------------------------------ */

static int rtcalarm_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        struct rtcalarm *d;
        u32 id;
        int ret;

        d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;

        d->dev = dev;
        spin_lock_init(&d->lock);
        init_waitqueue_head(&d->wq);

        d->base = devm_platform_ioremap_resource(pdev, 0);
        if (IS_ERR(d->base))
                return PTR_ERR(d->base);
```

```
        d->clk = devm_clk_get_enabled(dev, "apb_pclk");
        if (IS_ERR(d->clk))
                return dev_err_probe(dev, PTR_ERR(d->clk),
                                     "no apb_pclk\n");

        /* IDENTITY CHECK - catches a wrong address, an
           unclocked block or a denied access, at probe time
           rather than as mysterious behaviour later
           (chapter 19)                                      */
        id = ra_rd(d, RTC_PERIPH_ID0) & 0xff;
        if (id != PL031_PERIPH_ID0) {
                dev_err(dev, "unexpected periph id 0x%02x\n", id);
                return -ENODEV;
        }

        ret = device_property_read_u32(dev,
                        "hub,default-alarm-seconds",
                        &d->default_alarm_s);
        if (ret)
                d->default_alarm_s = 10;    /* a sane default */

        /* quiesce before requesting the interrupt */
        ra_wr(d, RTC_IMSC, 0);
        ra_wr(d, RTC_ICR, 1);
        ra_wr(d, RTC_CR, RTC_CR_EN);

        d->irq = platform_get_irq(pdev, 0);
        if (d->irq < 0)
                return d->irq;

        ret = devm_request_irq(dev, d->irq, rtcalarm_isr,
                               IRQF_SHARED, dev_name(dev), d);
        if (ret)
                return dev_err_probe(dev, ret, "irq %d\n", d->irq);

        platform_set_drvdata(pdev, d);

        /* userspace interface LAST (chapter 17) */
        d->misc.minor  = MISC_DYNAMIC_MINOR;
        d->misc.name   = "rtcalarm";
        d->misc.fops   = &rtcalarm_fops;
        d->misc.parent = dev;

        ret = misc_register(&d->misc);
        if (ret)
                return ret;

        dev_info(dev, "ready: count=%u default alarm %us\n",
                 ra_rd(d, RTC_DR), d->default_alarm_s);
        return 0;
}

static void rtcalarm_remove(struct platform_device *pdev)
{
        struct rtcalarm *d = platform_get_drvdata(pdev);

        /* exact reverse of probe (chapter 19) */
        misc_deregister(&d->misc);     /* no new users */
        ra_wr(d, RTC_IMSC, 0);         /* stop interrupts
                                          BEFORE devm frees
                                          the handler       */
        ra_wr(d, RTC_ICR, 1);
        /* devm releases the irq, the clock and the mapping */
```

```
}

/* ---- power management (chapter 25) ---------------------- */

static int rtcalarm_suspend(struct device *dev)
{
        struct rtcalarm *d = dev_get_drvdata(dev);

        if (d->alarm_set && device_may_wakeup(dev))
                enable_irq_wake(d->irq);
        return 0;
}

static int rtcalarm_resume(struct device *dev)
{
        struct rtcalarm *d = dev_get_drvdata(dev);

        if (device_may_wakeup(dev))
                disable_irq_wake(d->irq);
        return 0;
}

static DEFINE_SIMPLE_DEV_PM_OPS(rtcalarm_pm_ops,
                                rtcalarm_suspend, rtcalarm_resume);

/* ---- match ---------------------------------------------- */

static const struct of_device_id rtcalarm_of_match[] = {
        { .compatible = "hub,rtcalarm" },
        { }
};
MODULE_DEVICE_TABLE(of, rtcalarm_of_match);

static struct platform_driver rtcalarm_driver = {
        .driver = {
                .name           = "rtcalarm",
                .of_match_table = rtcalarm_of_match,
                .dev_groups     = rtcalarm_groups,
                .pm             = pm_ptr(&rtcalarm_pm_ops),
        },
        .probe  = rtcalarm_probe,
        .remove = rtcalarm_remove,
};
module_platform_driver(rtcalarm_driver);

MODULE_DESCRIPTION("Teaching PL031 alarm platform driver");
MODULE_AUTHOR("Embedded Learning Hub");
MODULE_LICENSE("GPL");
```

rtcalarm.h

```
/* SPDX-License-Identifier: GPL-2.0 */
#ifndef _UAPI_RTCALARM_H
#define _UAPI_RTCALARM_H

#include <linux/types.h>
#include <linux/ioctl.h>

struct ra_event {
        __u32 timestamp;
        __u32 _pad;
        __u64 count;
};

struct ra_state {
        __u32 now;
        __u32 alarm_time;
        __u8  alarm_set;
        __u8  fired;
        __u8  _pad[6];
        __u64 alarm_count;
        __u32 reserved[4];
};

#define RA_IOC_MAGIC     'A'
#define RA_SET_ALARM     _IOW(RA_IOC_MAGIC, 1, __u32)
#define RA_CANCEL_ALARM  _IO (RA_IOC_MAGIC, 2)
#define RA_GET_STATE     _IOR(RA_IOC_MAGIC, 3, struct ra_state)

#endif
```

Step 3: run it

```
# Build for the target, install into the initramfs, and boot
# with the modified DTB from step 1.

    make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
         -C $KDIR M=$PWD modules

# In the guest:
    insmod rtcalarm.ko
    dmesg | tail -1
      rtcalarm 9010000.pl031: ready: count=1718000000 \
                default alarm 5s

# It bound - confirm through the device model (chapter 17):
    ls -l /sys/bus/platform/devices/9010000.pl031/driver
      -> ../../../bus/platform/drivers/rtcalarm

    ls /sys/bus/platform/devices/9010000.pl031/
      alarm_count  alarm_in  now  driver  of_node  ...

# Drive it entirely from the shell:
    cat /sys/bus/platform/devices/9010000.pl031/now
      1718000042
    echo 3 > /sys/bus/platform/devices/9010000.pl031/alarm_in
    sleep 4
    cat /sys/bus/platform/devices/9010000.pl031/alarm_count
      1

# Watch the interrupt actually arrive (chapter 14):
    grep rtcalarm /proc/interrupts
     34:   1   0   0   0  GICv3  34 Level  9010000.pl031
           ^ it fired once, on CPU0, hwirq 34

# Blocking read:
    cat /dev/rtcalarm | hexdump -C &
    echo 2 > /sys/bus/platform/devices/9010000.pl031/alarm_in
    # two seconds later, 16 bytes appear

# Unbind and rebind - exercises probe and remove (ch. 17):
    echo 9010000.pl031 > /sys/bus/platform/drivers/rtcalarm/unbind
    echo 9010000.pl031 > /sys/bus/platform/drivers/rtcalarm/bind
```

The userspace client

```
/* ratest.c */
#include <fcntl.h>
#include <poll.h>
#include <stdio.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include "rtcalarm.h"

int main(int argc, char **argv)
{
        unsigned int seconds = (argc > 1) ? atoi(argv[1]) : 3;
        struct ra_state st;
        struct ra_event ev;
        struct pollfd pfd;
        int fd;

        fd = open("/dev/rtcalarm", O_RDWR);
        if (fd < 0) { perror("open"); return 1; }

        if (ioctl(fd, RA_GET_STATE, &st) < 0) {
                perror("RA_GET_STATE");
                return 1;
        }
        printf("now=%u fired=%u count=%llu\n", st.now, st.fired,
               (unsigned long long)st.alarm_count);

        if (ioctl(fd, RA_SET_ALARM, &seconds) < 0) {
                perror("RA_SET_ALARM");
                return 1;
        }
        printf("armed for %u seconds; waiting...\n", seconds);

        pfd.fd = fd;
        pfd.events = POLLIN;
        if (poll(&pfd, 1, (seconds + 5) * 1000) <= 0) {
                fprintf(stderr, "timed out - did the interrupt fire?\n");
                return 1;
        }

        if (read(fd, &ev, sizeof(ev)) != sizeof(ev)) {
                perror("read");
                return 1;
        }
        printf("ALARM at %u (alarm #%llu)\n", ev.timestamp,
               (unsigned long long)ev.count);

        close(fd);
        return 0;
}
```

Exercises

BREAK IT DELIBERATELY, THEN DIAGNOSE FROM THE SYMPTOM ALONE

1. **Wrong compatible.** Change it to `"hub,rtcalarm2"` in the driver only. Load it. Confirm probe never runs and the device has no `driver` link (ch. 17).
2. **Wrong address.** Change the `reg` in the DTS to `0x09011000`. The ID check fires — read the error and understand why an ID check is worth the five lines.
3. **Remove the clock.** Delete the `clocks` property. Observe `-EPROBE_DEFER` or a clean failure, and check `/sys/kernel/debug/devices_deferred`.
4. **Forget the acknowledge.** Delete `ra_wr(d, RTC_ICR, 1)` from the ISR. Arm an alarm. Watch the interrupt storm and the kernel's `nobody cared` protection (ch. 14).
5. **Return `IRQ_HANDLED` unconditionally**, dropping the `MIS` check. Nothing breaks here — then explain why it would on a genuinely shared line.
6. **Break the teardown order.** Move `misc_deregister` to the end of `remove`. Hold an fd open, unbind, and use it.
7. **Leave interrupts enabled in remove.** Unbind with an alarm armed. Watch what happens when it fires against a freed handler — with KASAN on, you get a report (ch. 27).
8. **Drop the lock** in the ISR and in `read`. Arm alarms in a tight loop from several processes and watch `alarm_count` lose increments.
9. **Add a feature:** a periodic mode that re-arms automatically in the ISR. Decide whether the re-arm belongs in the hard handler or a threaded one, and defend the choice (ch. 13).
10. **Add a proper binding.** Write `hub,rtcalarm.yaml` and validate it with `make dt_binding_check` (ch. 18).
11. **Convert to regmap** (ch. 23), marking `RTC_DR`, `RTC_RIS` and `RTC_MIS` volatile and `RTC_ICR` appropriately. Then enable the regmap trace events and watch every access.
12. **Add runtime PM** (ch. 25): suspend the block when no alarm is armed, resume on arm. Verify with `runtime_status`.

The review pass

```
# Before you call it done:

    make check                          # checkpatch
    make C=1 M=$PWD                     # sparse

# probe/remove cycles under KASAN:
    for i in $(seq 100); do
      echo 9010000.pl031 > /sys/bus/platform/drivers/rtcalarm/unbind
      echo 9010000.pl031 > /sys/bus/platform/drivers/rtcalarm/bind
    done
    dmesg | grep -i -e kasan -e warn -e bug

# no leaked resources after unbind:
    grep 9010000 /proc/iomem              # should be gone
    grep rtcalarm /proc/interrupts        # should be gone

# module load/unload cycles:
    for i in $(seq 100); do
      insmod rtcalarm.ko && rmmod rtcalarm || break
    done

# fault injection on probe:
    echo 20 > /sys/kernel/debug/failslab/probability
    # then bind repeatedly - no crashes, clean failures only

# the interrupt really fires, every time:
    for i in $(seq 20); do ./ratest 1; done
```

What transfers to real hardware

Everything structural. Swap the PL031 for your SoC's block and the changes are: a different register map, a different compatible string, more clocks and regulators, and probably DMA (chapter 24) instead of a single interrupt. The probe order, the teardown order, the locking discipline, the ioctl ABI rules, the sysfs conventions and the review checklist are identical.

That is the argument for building this on QEMU first: the parts you get wrong on real silicon are almost never the hardware parts. They are the ordering, the concurrency and the error paths — and you can practise all three without owning a board.

Check yourself

- ✓ Modify a device tree, rebuild it, and boot QEMU with your node.
- ✓ Explain how the `compatible` string binds your driver to that node.
- ✓ Justify the order of every step in probe and remove.
- ✓ Explain why the ID check belongs in probe and what three problems it catches.
- ✓ Explain why the ISR acknowledges before doing anything else.
- ✓ Say why `IRQF_SHARED` requires the `MIS` check.
- ✓ Read a custom DT property with a sensible fallback.
- ✓ Drive the device entirely from the shell via sysfs.
- ✓ Confirm the interrupt fires by reading `/proc/interrupts`.

- ✓ Complete exercises 1 through 8 and describe each symptom.
- ✓ Complete at least one of exercises 9 through 12.
- ✓ Pass the whole review with no findings.

Upstreaming and kernel culture

Getting your driver into mainline: why it is worth the effort, the mechanics of submitting a patch series, and what reviewers are actually looking for. The last chapter of the Linux track, and the one that decides whether your work outlives the product.

Why bother

```
THE OUT-OF-TREE COST, which compounds:

   the internal kernel API has no stability guarantee
   (chapter 02), so every kernel upgrade breaks your driver
      -> someone must fix it, forever
      -> that someone eventually leaves
      -> the product is stuck on a 2019 kernel with known CVEs

   THE VENDOR BSP PATTERN: a 4.19 kernel with 4,000 patches,
   which cannot take security fixes without a merge that
   nobody has budget for. This is the normal state of shipped
   embedded Linux, and it is a direct consequence of staying
   out of tree.

THE UPSTREAM BENEFIT:

   · other people fix your driver during API refactors, for
     free, forever
   · you get code review from people who have seen a thousand
     drivers
   · your users can run any kernel
   · security fixes flow automatically
   · new SoC variants are cheaper - the framework is already
     there
   · it is a portable, public record of your work

  THE HONEST COST: the first patch series takes weeks and
  several review rounds. The second takes days. It is a fixed
  learning cost, not a recurring one.
```

Before you send anything

```
# 1. IS IT ALREADY THERE? Search first.
    git log --oneline --all -- drivers/whatever/
    # and search lore.kernel.org for the device name, the
    # compatible string, and the vendor.
    # Finding an existing driver, or someone else's series
    # in flight, saves weeks.

# 2. USE THE RIGHT SUBSYSTEM (chapter 17). A temperature
#    sensor submitted as a custom char device will be asked
#    to become a hwmon driver. Find that out now, not after
#    three review rounds.

# 3. BASE ON THE RIGHT TREE.
    git remote add next \
      https://git.kernel.org/pub/scm/linux/kernel/git/next/linux-next.git
    # develop against linux-next or the relevant maintainer
    # tree, not against your vendor BSP.

# 4. CHECKPATCH, CLEAN.
    ./scripts/checkpatch.pl --strict -g HEAD~5..HEAD
    # --strict is what maintainers run. Fix everything it
    # reports, or be ready to explain why not.

# 5. SPARSE, COCCINELLE, THE BUILD BOTS' CHECKS.
    make C=1 M=drivers/mydrv/
    make W=1 M=drivers/mydrv/
    make coccicheck M=drivers/mydrv/

# 6. BUILD FOR OTHER ARCHITECTURES. The 0-day bot will do
#    this and email you if it breaks. Beat it to it:
    make ARCH=x86_64 allmodconfig && make M=drivers/mydrv/
    make ARCH=arm allmodconfig    && make M=drivers/mydrv/
    # a 32-bit build catches the u64-division and pointer-size
    # bugs that arm64-only testing never will.

# 7. THE DT BINDING VALIDATES (chapter 18).
    make dt_binding_check DT_SCHEMA_FILES=vendor,my-device.yaml
    make dtbs_check
```

Splitting the series

```
# ONE LOGICAL CHANGE PER PATCH. A typical new-driver series:

  [PATCH 0/4] Add support for the Vendor MyDevice
  [PATCH 1/4] dt-bindings: misc: add vendor,my-device
  [PATCH 2/4] misc: mydev: add Vendor MyDevice driver
  [PATCH 3/4] arm64: dts: vendor: add mydev node to mysoc
  [PATCH 4/4] MAINTAINERS: add entry for Vendor MyDevice

# WHY THIS ORDER: the binding comes first because it defines
# the interface the driver implements, and because it goes to
# a different maintainer who reviews it independently.
# The DTS patch comes after the driver, and often goes through
# the SoC tree rather than the subsystem tree - which is why
# it must be a SEPARATE patch.

# EACH PATCH MUST:
#   · compile on its own
#   · not break anything on its own (bisectability)
#   · do one thing, described in its subject line

# THE SUBJECT LINE FORMAT matters and is checked:
    subsystem: driver: short imperative description
# Get the prefix right by looking at history:
    git log --oneline -10 -- drivers/misc/
```

Writing the commit message

```
misc: mydev: add Vendor MyDevice driver

The MyDevice block is a data acquisition controller found on
Vendor MySoC parts. It provides four sampling channels with a
configurable hardware FIFO, and raises an interrupt when the
FIFO reaches a programmable threshold.

Add a driver exposing the device through a character node,
with blocking reads, poll() support and per-channel
configuration via ioctl. The v2 variant has eight channels and
a deeper FIFO; both are supported through match data.

The device is documented in the MySoC TRM revision 1.4,
section 12.

Signed-off-by: Your Name <you@example.com>

/* WHAT MAKES THIS ONE ACCEPTABLE:

   · says WHAT the hardware is, for a reviewer who has never
     heard of it
   · says WHAT the patch adds, in the imperative mood
     ("Add a driver", not "Added" or "This adds")
   · mentions the variants and how they are handled
   · cites the documentation
   · wrapped at 72 columns
   · has a Signed-off-by

   WHAT TO AVOID:
     · "misc: mydev: fixes" - says nothing
     · a message that only restates the diff
     · internal ticket numbers with no public meaning
     · "as discussed offline"                          */
```

The trailer tags

Tag	Meaning	Who adds it
`Signed-off-by`	The Developer's Certificate of Origin — a legal statement that you may contribute this	You, and everyone in the chain
`Reviewed-by`	"I read this carefully and it is correct"	A reviewer
`Acked-by`	"I am fine with this touching my area"	A maintainer
`Tested-by`	"I ran it"	Anyone who did
`Reported-by`	Who found the bug	You, crediting them
`Closes:`	A URL to the bug report	You
`Fixes:`	`<12-char sha> ("subject")` — enables backporting	You, on any bug fix
`Cc: stable@vger.kernel.org`	Backport this to stable kernels	You, for real fixes
`Suggested-by`	Who proposed the approach	You

```
# Signed-off-by is NOT optional and is NOT a formality. It
# asserts the DCO (Documentation/process/submitting-patches.rst):
# that you wrote it or have the right to submit it. Patches
# without it are rejected automatically.

    git commit -s              # adds it for you
    git config format.signOff true

# Fixes: format, exactly:
    Fixes: a1b2c3d4e5f6 ("misc: mydev: add error handling")

    git config core.abbrev 12
    git config alias.fixes "show -s --format='Fixes: %h (\"%s\")'"
    git fixes a1b2c3d
```

Sending it

```
# 1. WHO TO SEND TO:
    ./scripts/get_maintainer.pl -f drivers/misc/mydev.c
    ./scripts/get_maintainer.pl 0001-*.patch
      Greg Kroah-Hartman <gregkh@...> (maintainer:CHAR AND MISC)
      linux-kernel@vger.kernel.org (open list)

# Maintainers in To:, lists in Cc:.

# 2. GENERATE:
    git format-patch -4 --cover-letter -o outgoing/ \
        --subject-prefix="PATCH v2" origin/master

# Then EDIT the cover letter. A cover letter that still says
# "*** SUBJECT HERE ***" is a guaranteed re-send.

# 3. CONFIGURE git send-email once:
    git config sendemail.smtpserver smtp.example.com
    git config sendemail.smtpuser you@example.com
    git config sendemail.smtpencryption tls
    git config sendemail.smtpserverport 587

# 4. TEST IT ON YOURSELF FIRST. Always.
    git send-email --to=you@example.com outgoing/*.patch
# Then check the received mail: no HTML, no wrapped lines, no
# base64, no signature separator eating the diff. Apply it:
    git am received.mbox
# If it does not apply cleanly to your own tree, it will not
# apply to anyone's.

# 5. SEND:
    git send-email \
      --to="Maintainer <maint@example.com>" \
      --cc="linux-kernel@vger.kernel.org" \
      --cc="linux-arm-kernel@lists.infradead.org" \
      outgoing/*.patch

# 6. OR use b4, which handles most of this:
    b4 prep --new my-driver
    b4 send --dry-run
    b4 send
```

> **YOUR MAIL CLIENT WILL MANGLE THE PATCH**
> Gmail's web interface, Outlook, and any client that sends HTML or re-wraps lines will corrupt a patch beyond recovery, and the reply you get will be "your patch is whitespace-damaged, please resend". Use `git send-email` or `b4`. There is no third option that reliably works, and `Documentation/process/email-clients.rst` exists because so many people have learned this the hard way.

The cover letter

```
Subject: [PATCH v3 0/4] Add support for the Vendor MyDevice

This series adds support for the MyDevice data acquisition
block found on Vendor MySoC parts.

The device provides four (v1) or eight (v2) sampling channels
feeding a hardware FIFO, with a threshold interrupt. Userspace
reads samples through a character device and configures
channels with ioctl.

Tested on the Vendor EVK board with both v1 and v2 silicon,
and under QEMU with a stub model.

Changes in v3:
 - use dev_err_probe() throughout, per Krzysztof's review
 - drop the custom "vendor,irq-threshold" property and derive
   it from the FIFO depth instead
 - fix the 32-bit build (u64 division), reported by the 0-day
   bot
 - add the MAINTAINERS entry that was missing in v2

Changes in v2:
 - convert the binding to YAML schema
 - use devm_clk_get_enabled() instead of manual enable/disable
 - drop the module parameter; the DT property covers it

Your Name (4):
  dt-bindings: misc: add vendor,my-device
  misc: mydev: add Vendor MyDevice driver
  arm64: dts: vendor: add mydev node to mysoc
  MAINTAINERS: add entry for Vendor MyDevice

 Documentation/devicetree/bindings/misc/vendor,my-device.yaml |  62 ++
 MAINTAINERS                                                  |   7 +
 arch/arm64/boot/dts/vendor/mysoc.dtsi                        |  11 +
 drivers/misc/Kconfig                                         |  10 +
 drivers/misc/Makefile                                        |   1 +
 drivers/misc/mydev.c                                         | 480 +++++++
 6 files changed, 571 insertions(+)

/* THE CHANGELOG IS THE MOST IMPORTANT PART for a v2 or
   later. Reviewers who commented on v1 read it first to see
   whether you addressed them. Omitting it means they must
   re-derive your changes by diffing, and they will not.  */
```

Handling review

WHAT REVIEWERS ARE ACTUALLY CHECKING

- **The userspace ABI** — the one thing that cannot be fixed later (ch. 21). Expect the most scrutiny here.
- **The DT binding** — also permanent. Are the properties describing *hardware*, or your driver's preferences?
- **Is there already a subsystem for this?** The most common substantive review outcome for a new driver.
- **Error paths** — every one, including the unwinding order (ch. 26).
- **Locking** — is it documented, is it consistent, is it necessary.
- **Modern APIs** — `devm_`, `dev_err_probe`, `of_device_get_match_data` rather than the patterns of ten years ago.
- **Does it build everywhere** — 32-bit, other architectures, `allmodconfig`.
- **Style** — cheaply checked, so failing it signals you did not run checkpatch.

```
# HOW TO RESPOND:

# · Reply to every comment. Even "will fix" - silence reads as
#   disagreement or absence.
# · Reply inline, below the quoted text, trimming the rest.
#   Top-posting makes threads unreadable and will be noted.
# · Plain text only.
# · If you disagree, say why, technically. Reviewers change
#   their minds when given a reason.
# · Do not argue about style. Just change it.
# · WAIT a week before resending. Maintainers batch review;
#   a v2 sent four hours after v1 wastes everyone's time.
# · Send v2 as a NEW thread, with In-Reply-To pointing at the
#   v1 cover letter, and a changelog.

# THE TONE: reviews are blunt and about the code. "This is
# wrong because X" is a technical statement, not an insult.
# The kernel has a Code of Conduct
# (Documentation/process/code-of-conduct.rst) and it is
# enforced - but expect directness, and give directness back.

# HOW LONG IT TAKES:
#   first response:   days to two weeks
#   a merge:          two to six versions, over one to three
#                     months, is entirely normal for a new
#                     driver
#   NOTHING AT ALL:   ping politely after two weeks, in-thread.
#                     Patches do get lost.
```

Tracking your patch

```
# Is it applied?
    git log --oneline --all --grep="MyDevice"

# The public archive of your thread:
    https://lore.kernel.org/all/?q=MyDevice
    b4 mbox <message-id>        # fetch the whole thread

# Automated feedback arrives within a day or two:
#   the 0-day bot: build failures on architectures you did not
#                  try, sparse warnings, coccinelle findings
#   the DT bot:    binding schema errors
#   syzbot:        if it finds a crash later
# These emails are automated but the findings are real. Fix
# them before a human has to point them out.

# WHEN IT IS ACCEPTED, a maintainer replies "Applied, thanks"
# and it lands in their tree -> linux-next -> the next merge
# window -> a release. That is typically 2-10 weeks after
# acceptance.

# THEN: you are the maintainer. Add yourself to MAINTAINERS,
# and expect bug reports.
```

The recurring mistakes

Mistake	Consequence
A new custom char device where a subsystem exists	Asked to rewrite against IIO/hwmon/input
DT properties describing driver policy, not hardware	Binding rejected
No YAML binding, or an unvalidated one	Bounced by the DT maintainers immediately
Sent from an HTML mail client	"Whitespace damaged, please resend"
One giant patch	Asked to split; nobody reviews a 3,000-line diff
No changelog on v2	Reviewers cannot tell what you changed
Resending within hours	Reviewers stop reading the series
Ignoring the 0-day bot	A human eventually says the same thing, less patiently
Arguing about coding style	Goodwill spent on nothing
Silence on review comments	The patch is dropped
Module parameters for hardware description	Asked to use DT
Missing `Signed-off-by`	Cannot be applied at all

What to read

Document	Why
`Documentation/process/submitting-patches.rst`	The definitive guide. Read it once, fully.
`Documentation/process/coding-style.rst`	Short, opinionated, and worth reading for the reasoning
`Documentation/process/email-clients.rst`	Saves you one embarrassing round
`Documentation/process/submit-checklist.rst`	Run through it before every send

Document	Why
`Documentation/process/stable-kernel-rules.rst`	What qualifies for a backport
`Documentation/driver-api/`	Subsystem-specific expectations
lore.kernel.org	Read a few accepted series in your subsystem before writing yours
LWN.net	Context for why things are the way they are

> **THE BEST PREPARATION COSTS AN AFTERNOON**
>
> Find three recently-merged driver series in your target subsystem on lore, and read every message in each thread — the patches, the review comments, and the revisions. You will learn what that specific maintainer cares about, what the house patterns are, and what gets pushed back on, far faster than from any documentation. Then write yours to match.

How it shows up

Symptom	Cause
No response at all after two weeks	Wrong recipients, mangled patch, or genuinely lost — ping in-thread
"Please resend, whitespace damaged"	Mail client — use `git send-email`
"Please split this up"	One patch doing several things
"Use the IIO subsystem"	Reinvented an existing ABI
"This property is not hardware description"	Driver policy in the DT
0-day bot reports a build failure	An architecture or config you did not build
Applied, then reverted	Broke something in linux-next — fix and resend quickly
Maintainer asks for a v2 you already sent	Sent as a new thread with no `In-Reply-To`
Merged, then bug reports arrive	Working as intended — you are the maintainer now

Check yourself

- ✓ Explain the compounding cost of staying out of tree.
- ✓ Run the seven pre-submission checks on your own driver.
- ✓ Split a new driver into a properly ordered patch series.
- ✓ Write a commit message that a stranger could act on.
- ✓ Explain what `Signed-off-by` asserts and why it is required.
- ✓ Use `get_maintainer.pl` and send a patch to yourself successfully.
- ✓ Write a cover letter with a real changelog.
- ✓ Name eight things reviewers check, in order of how permanent they are.
- ✓ Describe how to respond to review, including disagreement.
- ✓ Read three merged series in your subsystem and list what that maintainer asks for.

END OF THE LINUX TRACK

Thirty chapters, from the source tree to upstream, with two working drivers in between. Next: Windows kernel and drivers, which solves the same problems with a completely different architecture — and the comparison is what makes both clearer.

Project: a C2PA capture-attestation driver

Content Credentials are only as trustworthy as the moment the shutter closed. This project puts that moment inside the kernel and the signing key inside the secure world, so that a provenance claim is a statement about pixels the hardware actually produced — not about a file someone handed the signer. This chapter builds the capture side; chapter 32 builds the trusted application and the manifest.

What C2PA actually is

C2PA — the Coalition for Content Provenance and Authenticity — standardises a signed metadata block travelling with a media file. Strip away the vocabulary and it is four things.

```
A C2PA MANIFEST, from the inside out

1. ASSERTIONS - statements about the asset
     c2pa.hash.data   a hash over the asset's bytes, with
                      exclusions for the manifest itself.
                      THE HARD BINDING. Everything rests here.
     c2pa.actions     "created", "edited", "colour adjusted"
     stds.exif        camera, lens, exposure
     c2pa.thumbnail   a small preview
     (and ingredients, referencing parent assets)

2. CLAIM - a CBOR map that lists the assertions by URI and
     hash, names the generator, and picks the signature
     algorithm. The claim is what gets signed.

3. CLAIM SIGNATURE - a COSE_Sign1 structure over the claim,
     with an X.509 certificate chain in the protected header.

4. MANIFEST STORE - all of the above packed into JUMBF boxes
     and embedded in the file (an APP11 segment in JPEG, an
     iTXt/eXIf-adjacent box in PNG, a uuid box in BMFF).

THE CHAIN OF REASONING A VERIFIER FOLLOWS:

  the certificate chains to a trusted root
    -> so I know who signed
  the COSE_Sign1 signature is valid over the claim
    -> so the claim is what the signer said
  each assertion's hash matches the assertion bytes
    -> so the assertion list is intact
  c2pa.hash.data matches the asset's actual bytes
    -> so THIS FILE is the file that was signed

Break any link and the credential is invalid. Notice what
the chain does NOT establish: that the pixels depict
reality. C2PA proves PROVENANCE, not TRUTH.
```

The problem hardware solves

```
THE NAIVE CAMERA

sensor -> kernel -> user space -> signing library -> file
                        ^
                        |
                  an attacker with code execution here
                  substitutes any image they like, and the
                  signature over it is PERFECTLY VALID.

The credential now says "this device produced this image",
with a real device key, and it is a lie. The signature is
not forged; it is genuine, over forged content.

THIS IS THE ENTIRE THREAT. A device signing key protects
against someone ELSE claiming to be your camera. It does
nothing about your camera being told to sign a picture it
never took.

WHAT ACTUALLY HELPS

sensor -> kernel ----------------+
            |  hash the frame    |
            |  HERE, before any  |
            |  user process can  |
            |  see it            |
            v                    |
     +--------------+            |
     | attest(hash) |            |
     +------+-------+            |
            | SMC                | frame bytes
            v                    v
     +-------------------+   user space assembles the
     | TA remembers the  |   manifest, and asks the TA
     | hashes it has     |   to sign the claim
     | witnessed         |
     |                   |<-- sign(claim)
     | REFUSES to sign a |
     | claim whose hard  |    the TA parses out the
     | binding is not on |    c2pa.hash.data value and
     | that list         |    checks it against its list
     +-------------------+

Now user space can still lie about EXIF, about the title,
about anything soft. It cannot get a signature over an image
the kernel did not hash.
```

THE DESIGN IDEA WORTH TAKING AWAY

A signing service that signs whatever it is given proves only that the key was available. Binding the signature to something the *lower* layer witnessed is what turns a key into an attestation. The same pattern is a TPM quote (OP-TEE chapter 23), a measured-boot PCR, and a secure-element camera credential.

WHAT THIS DOES *NOT* DEFEND AGAINST

A compromised **kernel** can attest whatever it likes — the kernel is inside the trust boundary by construction, which is why real camera implementations put the hash even lower, in the ISP or the sensor package. A compromised **TA** is game over. And nothing here stops someone photographing a screen: C2PA never claimed to. State the boundary honestly whenever you describe such a system, because the marketing around Content Credentials frequently does not.

The ABI

```
/* include/uapi/linux/c2pa_capture.h
 *
 * Shared by the driver, the user-space tool and (via the command
 * ids at the bottom) the trusted application.
 */

#ifndef _UAPI_LINUX_C2PA_CAPTURE_H
#define _UAPI_LINUX_C2PA_CAPTURE_H

#include <linux/types.h>
#include <linux/ioctl.h>

#define C2PA_ABI_VERSION   1
#define C2PA_HASH_LEN      32      /* SHA-256                    */
#define C2PA_SIG_LEN       64      /* ES256: raw r || s          */
#define C2PA_PUBKEY_LEN    64      /* P-256: X || Y              */
#define C2PA_MAX_SIGSTRUCT 4096    /* a COSE Sig_structure       */

/* One capture. The driver fills this in; user space never gets
 * to choose any of it except by asking for another capture.
 */
struct c2pa_capture {
        __u32 version;          /* in:  C2PA_ABI_VERSION            */
        __u32 frame_len;        /* in:  capacity   out: bytes       */
        __u64 frame;            /* in:  user pointer to the buffer  */

        __u64 capture_id;       /* out: TA monotonic counter value  */
        __u64 timestamp_ns;     /* out: CLOCK_MONOTONIC at capture  */
        __u32 width;            /* out                              */
        __u32 height;           /* out                              */
        __u8  hash[C2PA_HASH_LEN];  /* out: SHA-256 of frame bytes  */
};

/* Ask the TA to sign a COSE Sig_structure. The TA will refuse
 * unless the claim it contains binds to a hash it witnessed.
 */
struct c2pa_sign {
        __u32 version;
        __u32 sigstruct_len;
        __u64 sigstruct;        /* in:  user pointer                */
        __u8  binding[C2PA_HASH_LEN];  /* in: the c2pa.hash.data
                                        *     value in this claim   */
        __u8  signature[C2PA_SIG_LEN]; /* out: raw r || s           */
};

struct c2pa_pubkey {
        __u32 version;
        __u32 reserved;
        __u8  pubkey[C2PA_PUBKEY_LEN];   /* out: X || Y            */
};

#define C2PA_IOC_MAGIC   'C'
#define C2PA_IOC_CAPTURE _IOWR(C2PA_IOC_MAGIC, 1, struct c2pa_capture)
#define C2PA_IOC_SIGN    _IOWR(C2PA_IOC_MAGIC, 2, struct c2pa_sign)
#define C2PA_IOC_PUBKEY  _IOWR(C2PA_IOC_MAGIC, 3, struct c2pa_pubkey)

/* Commands on the far side of the SMC. Separate numbering from
 * the ioctls on purpose: the driver translates, and the two
 * interfaces version independently.
 */
```

```
#define TA_CMD_GET_PUBKEY     0
#define TA_CMD_ATTEST_CAPTURE 1
#define TA_CMD_SIGN_CLAIM     2

#endif /* _UAPI_LINUX_C2PA_CAPTURE_H */
```

> **WHY `BINDING` IS A SEPARATE FIELD**
>
> The TA could parse the CBOR claim out of the Sig_structure and find `c2pa.hash.data` itself. It deliberately does not: a CBOR parser inside a TA is a large, attacker-reachable piece of code (OP-TEE chapter 20). Instead user space *declares* the binding, the TA checks that declaration against its witnessed list, and — the part that makes it sound — chapter 32 shows the TA also verifying that the declared binding actually appears in the bytes it is about to sign, with a fixed-offset memcmp rather than a parser.

The driver

```
// SPDX-License-Identifier: GPL-2.0
/*
 * c2pa_capture.c - a C2PA capture-attestation test driver.
 *
 * Produces synthetic frames, hashes them in the kernel, and has
 * an OP-TEE trusted application witness each hash before user
 * space ever sees the pixels.
 *
 * On real hardware the frame source is a V4L2 subdev or an ISP
 * DMA completion; everything else is unchanged.
 */

#include <linux/module.h>
#include <linux/miscdevice.h>
#include <linux/fs.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/mutex.h>
#include <linux/tee_drv.h>
#include <linux/uuid.h>
#include <linux/crypto.h>
#include <crypto/hash.h>
#include <linux/ktime.h>
#include <uapi/linux/c2pa_capture.h>

#define DRV_NAME "c2pa-capture"

/* 8b1e4d72-3f5a-4c26-9d80-6a7b2e14f903 */
#define C2PA_TA_UUID UUID_INIT(0x8b1e4d72, 0x3f5a, 0x4c26, \
                               0x9d, 0x80, 0x6a, 0x7b, \
                               0x2e, 0x14, 0xf9, 0x03)

#define FRAME_W   640
#define FRAME_H   480
#define FRAME_LEN (FRAME_W * FRAME_H * 3)      /* RGB888        */
#define SHM_SIZE  8192

struct c2pa_dev {
        struct miscdevice       misc;
        struct mutex            lock;           /* one request at a time */

        /* TEE plumbing */
        struct tee_context      *ctx;
        u32                     session;
        struct tee_shm          *shm;
        void                    *shm_va;

        /* crypto */
        struct crypto_shash     *tfm;

        /* frame source */
        u8                      *frame;
        u32                     frame_no;
};

static struct c2pa_dev *g_dev;

/* ------------------------------------------------------------------
 * TEE glue
 * ------------------------------------------------------------------ */
```

```
static int optee_match(struct tee_ioctl_version_data *ver, const void *data)
{
        return ver->impl_id == TEE_IMPL_ID_OPTEE;
}

static int c2pa_tee_init(struct c2pa_dev *d)
{
        struct tee_ioctl_open_session_arg arg = { };
        uuid_t uuid = C2PA_TA_UUID;
        int rc;

        d->ctx = tee_client_open_context(NULL, optee_match, NULL, NULL);
        if (IS_ERR(d->ctx)) {
                /* -ENOENT during boot means OP-TEE is not up yet. */
                rc = PTR_ERR(d->ctx);
                d->ctx = NULL;
                return rc == -ENOENT ? -EPROBE_DEFER : rc;
        }

        export_uuid(arg.uuid, &uuid);
        /*
         * REE_KERNEL, not PUBLIC. The TA uses the login type to
         * decide who may ATTEST: only the kernel may say "I saw this
         * frame". A user-space process opening its own session gets
         * LOGIN_USER and is refused (chapter 32).
         */
        arg.clnt_login = TEE_IOCTL_LOGIN_REE_KERNEL;
        arg.num_params = 0;

        rc = tee_client_open_session(d->ctx, &arg, NULL);
        if (rc < 0 || arg.ret != 0) {
                pr_err(DRV_NAME ": open_session rc=%d ret=0x%08x\n",
                       rc, arg.ret);
                tee_client_close_context(d->ctx);
                d->ctx = NULL;
                return rc < 0 ? rc : -EINVAL;
        }
        d->session = arg.session;

        d->shm = tee_shm_alloc_kernel_buf(d->ctx, SHM_SIZE);
        if (IS_ERR(d->shm)) {
                rc = PTR_ERR(d->shm);
                d->shm = NULL;
                tee_client_close_session(d->ctx, d->session);
                tee_client_close_context(d->ctx);
                d->ctx = NULL;
                return rc;
        }
        d->shm_va = tee_shm_get_va(d->shm, 0);
        if (IS_ERR(d->shm_va))
                return PTR_ERR(d->shm_va);

        return 0;
}

static void c2pa_tee_exit(struct c2pa_dev *d)
{
        /* Reverse order of init, always. */
        if (d->shm)
                tee_shm_free(d->shm);
        if (d->ctx) {
                tee_client_close_session(d->ctx, d->session);
```

```
                tee_client_close_context(d->ctx);
        }
}

/* GlobalPlatform TEE_Result -> errno, in one place. */
static int tee_result_to_errno(u32 res)
{
        switch (res) {
        case 0xFFFF0004: return -EOPNOTSUPP;    /* NOT_SUPPORTED    */
        case 0xFFFF0006: return -EINVAL;        /* BAD_PARAMETERS   */
        case 0xFFFF0008: return -ENOENT;        /* ITEM_NOT_FOUND   */
        case 0xFFFF000C: return -ENOSPC;        /* SHORT_BUFFER     */
        case 0xFFFF000F: return -EACCES;        /* ACCESS_DENIED    */
        case 0xFFFF3024: return -ENOMEM;        /* OUT_OF_MEMORY    */
        case 0xFFFF3110: return -ENODEV;        /* TARGET_DEAD      */
        default:         return -EIO;
        }
}

/* ------------------------------------------------------------------
 * The frame source
 *
 * A moving test pattern, so successive captures differ and you can
 * see the hash change. Replace with a V4L2 buffer on real hardware.
 * ------------------------------------------------------------------ */
static void c2pa_generate_frame(struct c2pa_dev *d)
{
        u32 x, y;
        u8 *p = d->frame;

        for (y = 0; y < FRAME_H; y++) {
                for (x = 0; x < FRAME_W; x++) {
                        *p++ = (u8)(x + d->frame_no);           /* R */
                        *p++ = (u8)(y + d->frame_no * 2);       /* G */
                        *p++ = (u8)((x ^ y) + d->frame_no);     /* B */
                }
        }
        d->frame_no++;
}

/* ------------------------------------------------------------------
 * Hashing, with the kernel crypto API
 * ------------------------------------------------------------------ */
static int c2pa_hash_frame(struct c2pa_dev *d, u8 out[C2PA_HASH_LEN])
{
        SHASH_DESC_ON_STACK(desc, d->tfm);

        desc->tfm = d->tfm;
        return crypto_shash_digest(desc, d->frame, FRAME_LEN, out);
        /*
         * The frame is hashed HERE, from the kernel's own buffer,
         * before copy_to_user. That ordering is the whole security
         * argument: there is no window in which user space could
         * substitute pixels between the hash and the attestation.
         */
}

/* ------------------------------------------------------------------
 * C2PA_IOC_CAPTURE
 * ------------------------------------------------------------------ */
static long c2pa_do_capture(struct c2pa_dev *d, unsigned long arg)
{
        struct c2pa_capture req;
```

```
        struct tee_ioctl_invoke_arg inv = { };
        struct tee_param param[4] = { };
        u8 hash[C2PA_HASH_LEN];
        u64 timestamp;
        int rc;

        if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
                return -EFAULT;
        if (req.version != C2PA_ABI_VERSION)
                return -EINVAL;
        if (req.frame_len < FRAME_LEN)
                return -ENOSPC;

        mutex_lock(&d->lock);

        c2pa_generate_frame(d);

        rc = c2pa_hash_frame(d, hash);
        if (rc)
                goto out;

        timestamp = ktime_get_ns();

        /*
         * Attest: hand the hash to the TA. It records it and returns
         * its monotonic counter as a capture id.
         *
         * param[0] memref in  : the 32-byte hash
         * param[1] value out  : a = counter low, b = counter high
         */
        memcpy(d->shm_va, hash, C2PA_HASH_LEN);

        param[0].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INPUT;
        param[0].u.memref.shm      = d->shm;
        param[0].u.memref.shm_offs = 0;
        param[0].u.memref.size     = C2PA_HASH_LEN;
        param[1].attr = TEE_IOCTL_PARAM_ATTR_TYPE_VALUE_OUTPUT;

        inv.func       = TA_CMD_ATTEST_CAPTURE;
        inv.session    = d->session;
        inv.num_params = 4;

        rc = tee_client_invoke_func(d->ctx, &inv, param);
        if (rc < 0)
                goto out;
        if (inv.ret != 0) {
                pr_warn(DRV_NAME ": attest returned 0x%08x\n", inv.ret);
                rc = tee_result_to_errno(inv.ret);
                goto out;
        }

        req.capture_id   = (u64)param[1].u.value.a |
                           ((u64)param[1].u.value.b << 32);
        req.timestamp_ns = timestamp;
        req.width        = FRAME_W;
        req.height       = FRAME_H;
        req.frame_len    = FRAME_LEN;
        memcpy(req.hash, hash, C2PA_HASH_LEN);

        /*
         * Only NOW does user space get the pixels - after the TA has
         * already committed to their hash.
         */
```

```
        if (copy_to_user((void __user *)req.frame, d->frame, FRAME_LEN)) {
                rc = -EFAULT;
                goto out;
        }
        if (copy_to_user((void __user *)arg, &req, sizeof(req)))
                rc = -EFAULT;
out:
        mutex_unlock(&d->lock);
        return rc;
}

/* ------------------------------------------------------------------
 * C2PA_IOC_SIGN
 * ------------------------------------------------------------------ */
static long c2pa_do_sign(struct c2pa_dev *d, unsigned long arg)
{
        struct c2pa_sign req;
        struct tee_ioctl_invoke_arg inv = { };
        struct tee_param param[4] = { };
        void *sigstruct = NULL;
        long rc;

        if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
                return -EFAULT;
        if (req.version != C2PA_ABI_VERSION)
                return -EINVAL;
        if (req.sigstruct_len == 0 ||
            req.sigstruct_len > C2PA_MAX_SIGSTRUCT)
                return -EINVAL;
        /* Leave room for the binding at a fixed offset in the SHM. */
        if (req.sigstruct_len + C2PA_HASH_LEN > SHM_SIZE)
                return -EINVAL;

        sigstruct = memdup_user((void __user *)req.sigstruct,
                                req.sigstruct_len);
        if (IS_ERR(sigstruct))
                return PTR_ERR(sigstruct);

        mutex_lock(&d->lock);

        /*
         * Shared memory layout for this call:
         *   [0                .. sigstruct_len)  the Sig_structure
         *   [sigstruct_len    .. +32)            the declared binding
         * Two partial memrefs over one buffer (OP-TEE chapter 7).
         */
        memcpy(d->shm_va, sigstruct, req.sigstruct_len);
        memcpy((u8 *)d->shm_va + req.sigstruct_len,
               req.binding, C2PA_HASH_LEN);

        param[0].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INPUT;
        param[0].u.memref.shm      = d->shm;
        param[0].u.memref.shm_offs = 0;
        param[0].u.memref.size     = req.sigstruct_len;

        param[1].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_INPUT;
        param[1].u.memref.shm      = d->shm;
        param[1].u.memref.shm_offs = req.sigstruct_len;
        param[1].u.memref.size     = C2PA_HASH_LEN;

        param[2].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_OUTPUT;
        param[2].u.memref.shm      = d->shm;
        param[2].u.memref.shm_offs = SHM_SIZE - C2PA_SIG_LEN;
```

```
        param[2].u.memref.size     = C2PA_SIG_LEN;

        inv.func       = TA_CMD_SIGN_CLAIM;
        inv.session    = d->session;
        inv.num_params = 4;

        rc = tee_client_invoke_func(d->ctx, &inv, param);
        if (rc < 0)
                goto out;
        if (inv.ret != 0) {
                /*
                 * ACCESS_DENIED here is the interesting one: it means
                 * the TA refused because the declared binding is not a
                 * hash it witnessed. That is the system working.
                 */
                pr_warn(DRV_NAME ": sign returned 0x%08x\n", inv.ret);
                rc = tee_result_to_errno(inv.ret);
                goto out;
        }

        memcpy(req.signature, (u8 *)d->shm_va + SHM_SIZE - C2PA_SIG_LEN,
               C2PA_SIG_LEN);
        rc = copy_to_user((void __user *)arg, &req, sizeof(req)) ? -EFAULT : 0;
out:
        mutex_unlock(&d->lock);
        kfree(sigstruct);
        return rc;
}

/* ------------------------------------------------------------------
 * C2PA_IOC_PUBKEY
 * ------------------------------------------------------------------ */
static long c2pa_do_pubkey(struct c2pa_dev *d, unsigned long arg)
{
        struct c2pa_pubkey req;
        struct tee_ioctl_invoke_arg inv = { };
        struct tee_param param[4] = { };
        long rc;

        if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
                return -EFAULT;
        if (req.version != C2PA_ABI_VERSION)
                return -EINVAL;

        mutex_lock(&d->lock);

        param[0].attr = TEE_IOCTL_PARAM_ATTR_TYPE_MEMREF_OUTPUT;
        param[0].u.memref.shm      = d->shm;
        param[0].u.memref.shm_offs = 0;
        param[0].u.memref.size     = C2PA_PUBKEY_LEN;

        inv.func       = TA_CMD_GET_PUBKEY;
        inv.session    = d->session;
        inv.num_params = 4;

        rc = tee_client_invoke_func(d->ctx, &inv, param);
        if (rc < 0)
                goto out;
        if (inv.ret != 0) {
                rc = tee_result_to_errno(inv.ret);
                goto out;
        }
```

```
        memcpy(req.pubkey, d->shm_va, C2PA_PUBKEY_LEN);
        rc = copy_to_user((void __user *)arg, &req, sizeof(req)) ? -EFAULT : 0;
out:
        mutex_unlock(&d->lock);
        return rc;
}

/* ------------------------------------------------------------------
 * File operations
 * ------------------------------------------------------------------ */
static long c2pa_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
        struct c2pa_dev *d = g_dev;

        switch (cmd) {
        case C2PA_IOC_CAPTURE: return c2pa_do_capture(d, arg);
        case C2PA_IOC_SIGN:    return c2pa_do_sign(d, arg);
        case C2PA_IOC_PUBKEY:  return c2pa_do_pubkey(d, arg);
        default:               return -ENOTTY;
        }
}

static const struct file_operations c2pa_fops = {
        .owner          = THIS_MODULE,
        .unlocked_ioctl = c2pa_ioctl,
        .compat_ioctl   = compat_ptr_ioctl,   /* the structs use __u64
                                               * for pointers, so 32-bit
                                               * user space needs nothing
                                               * more than this           */
        .llseek         = no_llseek,
};

/* ------------------------------------------------------------------
 * Module init / exit
 * ------------------------------------------------------------------ */
static int __init c2pa_init(void)
{
        struct c2pa_dev *d;
        int rc;

        d = kzalloc(sizeof(*d), GFP_KERNEL);
        if (!d)
                return -ENOMEM;
        mutex_init(&d->lock);

        d->frame = vmalloc(FRAME_LEN);
        if (!d->frame) {
                rc = -ENOMEM;
                goto err_free;
        }

        d->tfm = crypto_alloc_shash("sha256", 0, 0);
        if (IS_ERR(d->tfm)) {
                rc = PTR_ERR(d->tfm);
                pr_err(DRV_NAME ": no sha256 shash: %d\n", rc);
                goto err_frame;
        }

        rc = c2pa_tee_init(d);
        if (rc)
                goto err_tfm;

        d->misc.minor = MISC_DYNAMIC_MINOR;
```

```
        d->misc.name  = "c2pa0";
        d->misc.fops  = &c2pa_fops;
        d->misc.mode  = 0600;      /* root only: attestation is not
                                    * something every process should be
                                    * able to consume                 */

        rc = misc_register(&d->misc);
        if (rc)
                goto err_tee;

        g_dev = d;
        pr_info(DRV_NAME ": /dev/c2pa0 ready\n");
        return 0;

err_tee:
        c2pa_tee_exit(d);
err_tfm:
        crypto_free_shash(d->tfm);
err_frame:
        vfree(d->frame);
err_free:
        kfree(d);
        return rc;
}

static void __exit c2pa_exit(void)
{
        struct c2pa_dev *d = g_dev;

        misc_deregister(&d->misc);
        c2pa_tee_exit(d);
        crypto_free_shash(d->tfm);
        vfree(d->frame);
        kfree(d);
        g_dev = NULL;
}

module_init(c2pa_init);
module_exit(c2pa_exit);

MODULE_DESCRIPTION("C2PA capture attestation, backed by OP-TEE");
MODULE_LICENSE("GPL");
```

THE ORDERING THAT CARRIES THE WHOLE ARGUMENT

In `c2pa_do_capture`: generate, hash, attest, *then* `copy_to_user`. Move the copy earlier — which looks harmless, and which a later refactor will be tempted to do — and a second thread sharing the buffer could in principle observe or race the pixels before the TA has committed to their hash. Comment the ordering in the code, not just in the design document, because that is where the next person edits.

Building and loading

```
# Kbuild
obj-m += c2pa_capture.o
ccflags-y += -I$(src)/include

# Build against the OP-TEE QEMU kernel (OP-TEE chapter 4).
$ cd ~/optee/linux
$ ./scripts/config --enable TEE --enable OPTEE \
                   --enable CRYPTO_SHA256 --enable CRYPTO_USER_API_HASH
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- olddefconfig
$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc) Image modules

$ cd ~/c2pa/driver
$ make -C ~/optee/linux M=$PWD ARCH=arm64 \
       CROSS_COMPILE=aarch64-linux-gnu- modules

$ cp c2pa_capture.ko ~/optee/out-br/target/root/
$ cd ~/optee/build && make buildroot run-only
```

```
# In the guest:
# insmod c2pa_capture.ko
[   14.221408] c2pa-capture: /dev/c2pa0 ready

# The secure console, at the same moment:
I/TA: c2pa TA loaded

# If instead you see this, the TA is not installed:
[   14.220110] c2pa-capture: open_session rc=0 ret=0xffff0008
insmod: ERROR: could not insert module: No such file or directory

# And this means OP-TEE itself is not up:
[   14.219980] c2pa-capture: probing deferred
$ dmesg | grep optee
```

A smoke test before the manifest work

```
/* tools/c2pa-smoke.c - proves the driver and TA agree before you
 * go anywhere near CBOR. Build it first, always.
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include "c2pa_capture.h"

static void hex(const char *label, const unsigned char *p, size_t n)
{
        printf("%-12s", label);
        for (size_t i = 0; i < n; i++)
                printf("%02x", p[i]);
        printf("\n");
}

int main(void)
{
        int fd = open("/dev/c2pa0", O_RDWR);
        if (fd < 0) { perror("/dev/c2pa0"); return 1; }

        /* 1. Who is the camera? */
        struct c2pa_pubkey pk = { .version = C2PA_ABI_VERSION };
        if (ioctl(fd, C2PA_IOC_PUBKEY, &pk)) { perror("PUBKEY"); return 1; }
        hex("pubkey", pk.pubkey, sizeof(pk.pubkey));

        /* 2. Capture. */
        unsigned char *frame = malloc(640 * 480 * 3);
        struct c2pa_capture cap = {
                .version   = C2PA_ABI_VERSION,
                .frame_len = 640 * 480 * 3,
                .frame     = (unsigned long long)(uintptr_t)frame,
        };
        if (ioctl(fd, C2PA_IOC_CAPTURE, &cap)) { perror("CAPTURE"); return 1; }
        printf("capture id  %llu  %ux%u\n",
               (unsigned long long)cap.capture_id, cap.width, cap.height);
        hex("frame hash", cap.hash, sizeof(cap.hash));

        /* 3. Sign something that DOES bind to that capture. */
        unsigned char fake_sigstruct[64];
        memset(fake_sigstruct, 0xAB, sizeof(fake_sigstruct));

        struct c2pa_sign s = {
                .version       = C2PA_ABI_VERSION,
                .sigstruct_len = sizeof(fake_sigstruct),
                .sigstruct     = (unsigned long long)(uintptr_t)fake_sigstruct,
        };
        memcpy(s.binding, cap.hash, C2PA_HASH_LEN);

        if (ioctl(fd, C2PA_IOC_SIGN, &s)) { perror("SIGN (valid)"); return 1; }
        hex("signature", s.signature, sizeof(s.signature));

        /* 4. And now something that does NOT. This MUST fail. */
        s.binding[0] ^= 0xff;
        if (ioctl(fd, C2PA_IOC_SIGN, &s) == 0) {
                fprintf(stderr, "FAIL: TA signed an unwitnessed binding!\n");
                return 1;
```

```
        }
        printf("refused an unwitnessed binding, as it should\n");

        close(fd);
        return 0;
}
```

```
# ./c2pa-smoke
pubkey      a91f3c...e7b2
capture id  1  640x480
frame hash  4d8e2a...7c
signature   3f81b9...04
refused an unwitnessed binding, as it should

# That last line is the project. Everything in chapter 32 is
# packaging around it.
```

How it shows up

Symptom	Cause
`insmod` fails, `ret=0xffff0008`	`ITEM_NOT_FOUND`: the `.ta` is not in `/lib/optee_armtz`, or its filename does not match the UUID
Module load deferred for ever	OP-TEE driver absent — check `CONFIG_OPTEE` and `dmesg | grep optee`
`open_session` hangs	`tee-supplicant` not running; loading a TA needs it (OP-TEE chapter 9)
`crypto_alloc_shash` returns `-ENOENT`	`CONFIG_CRYPTO_SHA256` not enabled
Signing returns `-EACCES` for a real capture	The TA's witnessed list was cleared — it is bounded, and an old capture ages out (chapter 32)
Signing succeeds for a tampered binding	**The security property is broken.** The TA is not checking, or the driver is passing the wrong buffer
Every capture has the same hash	The test pattern is not advancing — `frame_no` not incrementing
Sporadic `-EFAULT`	User buffer smaller than `frame_len` claims, or freed while the ioctl runs
Deadlock under concurrent use	An error path returning without `mutex_unlock`
`rmmod` oopses	Teardown order — shared memory freed after the context, or a file still open

Check yourself

- ✓ Name the four layers of a C2PA manifest and say which one is the hard binding.
- ✓ State what a C2PA credential proves and what it explicitly does not.
- ✓ Explain the attack a device signing key alone does not prevent.
- ✓ Explain why the frame is hashed in the kernel rather than in user space.
- ✓ Say what the TA checks before signing, and why that turns a key into an attestation.
- ✓ Name two things this design still does not defend against.
- ✓ Explain why the driver logs in as `REE_KERNEL` rather than `PUBLIC`.
- ✓ Explain why the binding is declared by user space rather than parsed by the TA.
- ✓ Point to the ordering in `c2pa_do_capture` that the security argument depends on.

- ✓ Describe the shared-memory layout used by the sign command and count the memrefs.
- ✓ Run the smoke test and explain what its final line demonstrates.

Project: the C2PA trusted application and manifest

The secure half of the capture project: a trusted application that will only sign claims about images it witnessed, the COSE structure that carries the signature, and a real Content Credential that `c2patool` validates. Plus a clear list of what still stands between this and something you could ship.

COSE_Sign1, and a pleasant surprise

```
A C2PA claim signature is a COSE_Sign1 (RFC 9052):

  COSE_Sign1 = [
     protected   : bstr,   ; a serialised CBOR map
     unprotected : map,
     payload     : bstr,   ; the claim CBOR
     signature   : bstr
  ]

The protected header, for C2PA with ES256:

  {
     1  : -7,              ; alg = ES256 (ECDSA P-256 + SHA-256)
     33 : [ bstr, ... ]    ; x5chain - the certificate chain,
  }                        ;   RFC 9360, leaf first

WHAT IS ACTUALLY SIGNED is not the COSE_Sign1 and not the
payload. It is the Sig_structure:

  Sig_structure = [
     "Signature1",         ; context
     protected,            ; bstr, exactly as it appears above
     external_aad,         ; bstr, empty for C2PA
     payload               ; bstr, the claim
  ]

Serialise that array to CBOR, SHA-256 it, sign the digest.

THE PLEASANT PART:

  RFC 9053 s2.1 - "The signature value is the concatenation
  of the R and S values", fixed width, big-endian.

  So COSE wants RAW r || s. Exactly what
  TEE_AsymmetricSignDigest produces. No DER anywhere.

  Compare:
    COSE / JOSE / CNG   -> raw r || s      (64 bytes for P-256)
    OpenSSL / X.509 /   -> DER SEQUENCE of two INTEGERs
      TLS / CMS

  The Rust project needed a converter; this one does not.
  The TA's output drops straight into the signature slot.
```

The trusted application

```
/* ta/include/c2pa_ta.h - shared with the driver. */

#ifndef C2PA_TA_H
#define C2PA_TA_H

#define C2PA_TA_UUID \
        { 0x8b1e4d72, 0x3f5a, 0x4c26, \
          { 0x9d, 0x80, 0x6a, 0x7b, 0x2e, 0x14, 0xf9, 0x03 } }

#define TA_CMD_GET_PUBKEY     0
#define TA_CMD_ATTEST_CAPTURE 1
#define TA_CMD_SIGN_CLAIM     2

#define C2PA_HASH_LEN   32
#define C2PA_SIG_LEN    64
#define C2PA_PUBKEY_LEN 64

/* How many recent captures the TA remembers. Small on purpose:
 * see the note on the witness window below.
 */
#define C2PA_WITNESS_SLOTS 8

#endif
```

```
/* ta/user_ta_header_defines.h */
#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#include "c2pa_ta.h"
#define TA_UUID C2PA_TA_UUID

/*
 * SINGLE_INSTANCE + KEEP_ALIVE is not a performance choice here,
 * it is a correctness requirement: the witness ring lives in TA
 * globals. Without KEEP_ALIVE the TA is unloaded when the last
 * session closes and every witnessed capture is forgotten
 * (OP-TEE chapter 13).
 */
#define TA_FLAGS (TA_FLAG_SINGLE_INSTANCE | TA_FLAG_INSTANCE_KEEP_ALIVE)

#define TA_STACK_SIZE (8 * 1024)
#define TA_DATA_SIZE  (64 * 1024)

#define TA_CURRENT_TA_EXT_PROPERTIES \
        { "gp.ta.description", USER_TA_PROP_TYPE_STRING, \
          "C2PA capture attestation" }, \
        { "gp.ta.version", USER_TA_PROP_TYPE_U32, \
          &(const uint32_t){ 0x0100 } }

#endif
```

```
/* ta/c2pa_ta.c */

#include <tee_internal_api.h>
#include <tee_internal_api_extensions.h>
#include <string.h>
#include "c2pa_ta.h"

#define KEY_OBJECT_ID     "c2pa.device-key.v1"
#define COUNTER_OBJECT_ID "c2pa.counter.v1"

/* ------------------------------------------------------------------
 * The witness ring
 *
 * A small circular buffer of hashes the KERNEL told us it produced.
 * Deliberately bounded: an unbounded list is a memory-exhaustion
 * target reachable by anyone who can call ATTEST, and secure heap
 * is measured in tens of kilobytes (OP-TEE chapter 6).
 *
 * The bound has a real consequence, and it is a design decision
 * rather than a limitation: a claim must be signed within
 * C2PA_WITNESS_SLOTS captures of the frame it describes. A camera
 * signs immediately, so eight is generous. If your pipeline
 * batches, raise it - and understand that you are widening the
 * window in which a compromised user space may pick which
 * witnessed frame to lie about.
 * ------------------------------------------------------------------ */
struct witness {
        uint8_t  hash[C2PA_HASH_LEN];
        uint64_t capture_id;
        bool     used;          /* single-use: one signature per capture */
        bool     valid;
};

static struct witness g_witness[C2PA_WITNESS_SLOTS];
static unsigned int   g_next;
static uint64_t       g_counter;
static bool           g_counter_loaded;

/* ------------------------------------------------------------------
 * Monotonic counter, persisted
 * ------------------------------------------------------------------ */
static TEE_Result counter_load(void)
{
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        uint32_t got = 0;
        TEE_Result res;

        if (g_counter_loaded)
                return TEE_SUCCESS;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       COUNTER_OBJECT_ID,
                                       sizeof(COUNTER_OBJECT_ID) - 1,
                                       TEE_DATA_FLAG_ACCESS_READ, &obj);
        if (res == TEE_ERROR_ITEM_NOT_FOUND) {
                g_counter = 0;
                g_counter_loaded = true;
                return TEE_SUCCESS;
        }
        if (res != TEE_SUCCESS)
                return res;

        res = TEE_ReadObjectData(obj, &g_counter, sizeof(g_counter), &got);
```

```
        TEE_CloseObject(obj);
        if (res != TEE_SUCCESS)
                return res;
        if (got != sizeof(g_counter))
                g_counter = 0;

        g_counter_loaded = true;
        return TEE_SUCCESS;
}

static TEE_Result counter_store(void)
{
        TEE_ObjectHandle obj = TEE_HANDLE_NULL;
        TEE_Result res;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       COUNTER_OBJECT_ID,
                                       sizeof(COUNTER_OBJECT_ID) - 1,
                                       TEE_DATA_FLAG_ACCESS_WRITE, &obj);
        if (res == TEE_SUCCESS) {
                res = TEE_SeekObjectData(obj, 0, TEE_DATA_SEEK_SET);
                if (res == TEE_SUCCESS)
                        res = TEE_WriteObjectData(obj, &g_counter,
                                                  sizeof(g_counter));
                TEE_CloseObject(obj);
                return res;
        }
        if (res != TEE_ERROR_ITEM_NOT_FOUND)
                return res;

        res = TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
                                         COUNTER_OBJECT_ID,
                                         sizeof(COUNTER_OBJECT_ID) - 1,
                                         TEE_DATA_FLAG_ACCESS_READ |
                                         TEE_DATA_FLAG_ACCESS_WRITE |
                                         TEE_DATA_FLAG_ACCESS_WRITE_META,
                                         TEE_HANDLE_NULL,
                                         &g_counter, sizeof(g_counter), &obj);
        if (res == TEE_SUCCESS)
                TEE_CloseObject(obj);
        return res;
}

/* ------------------------------------------------------------------
 * The device key
 * ------------------------------------------------------------------ */
static TEE_Result open_or_create_key(TEE_ObjectHandle *out)
{
        TEE_ObjectHandle key = TEE_HANDLE_NULL;
        TEE_Attribute curve;
        TEE_Result res;

        res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                       KEY_OBJECT_ID,
                                       sizeof(KEY_OBJECT_ID) - 1,
                                       TEE_DATA_FLAG_ACCESS_READ, out);
        if (res == TEE_SUCCESS)
                return TEE_SUCCESS;
        if (res != TEE_ERROR_ITEM_NOT_FOUND)
                return res;

        IMSG("generating device key");
```

```
        res = TEE_AllocateTransientObject(TEE_TYPE_ECDSA_KEYPAIR, 256, &key);
        if (res != TEE_SUCCESS)
                return res;

        TEE_InitValueAttribute(&curve, TEE_ATTR_ECC_CURVE,
                               TEE_ECC_CURVE_NIST_P256, 0);

        res = TEE_GenerateKey(key, 256, &curve, 1);
        if (res == TEE_SUCCESS)
                res = TEE_CreatePersistentObject(
                        TEE_STORAGE_PRIVATE, KEY_OBJECT_ID,
                        sizeof(KEY_OBJECT_ID) - 1,
                        TEE_DATA_FLAG_ACCESS_READ |
                        TEE_DATA_FLAG_ACCESS_WRITE_META,
                        key, NULL, 0, out);

        TEE_FreeTransientObject(key);
        return res;
}

/* Left-pad an ECC coordinate to a fixed 32 bytes. The minimal
 * encoding drops leading zero bytes, so about one coordinate in
 * 256 comes back short. Writing it flush-left produces a key that
 * works 99.6% of the time.
 */
static TEE_Result get_coord(TEE_ObjectHandle obj, uint32_t attr,
                            uint8_t out[32])
{
        uint8_t tmp[32];
        uint32_t len = sizeof(tmp);
        TEE_Result res;

        res = TEE_GetObjectBufferAttribute(obj, attr, tmp, &len);
        if (res != TEE_SUCCESS)
                return res;
        if (len > 32)
                return TEE_ERROR_BAD_STATE;

        TEE_MemFill(out, 0, 32);
        TEE_MemMove(out + (32 - len), tmp, len);
        return TEE_SUCCESS;
}

/* ------------------------------------------------------------------
 * Who is calling?
 *
 * ATTEST may only be issued by the KERNEL. If a user process could
 * attest, it could witness a hash of any image it liked and the
 * entire binding argument collapses.
 * ------------------------------------------------------------------ */
static bool caller_is_kernel(void)
{
        TEE_Identity id;

        if (TEE_GetPropertyAsIdentity(TEE_PROPSET_CURRENT_CLIENT,
                                      "gpd.client.identity",
                                      &id) != TEE_SUCCESS)
                return false;

        return id.login == TEE_LOGIN_REE_KERNEL;
}

/* ------------------------------------------------------------------
```

```
 * TA_CMD_GET_PUBKEY   [0] memref out, 64 bytes
 * ------------------------------------------------------------------ */
static TEE_Result cmd_get_pubkey(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                             TEE_PARAM_TYPE_NONE,
                                             TEE_PARAM_TYPE_NONE,
                                             TEE_PARAM_TYPE_NONE);
        TEE_ObjectHandle key = TEE_HANDLE_NULL;
        uint8_t pub[C2PA_PUBKEY_LEN];
        TEE_Result res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;
        if (params[0].memref.size < C2PA_PUBKEY_LEN) {
                params[0].memref.size = C2PA_PUBKEY_LEN;
                return TEE_ERROR_SHORT_BUFFER;
        }

        res = open_or_create_key(&key);
        if (res != TEE_SUCCESS)
                return res;

        res = get_coord(key, TEE_ATTR_ECC_PUBLIC_VALUE_X, pub);
        if (res == TEE_SUCCESS)
                res = get_coord(key, TEE_ATTR_ECC_PUBLIC_VALUE_Y, pub + 32);

        TEE_CloseObject(key);
        if (res != TEE_SUCCESS)
                return res;

        TEE_MemMove(params[0].memref.buffer, pub, C2PA_PUBKEY_LEN);
        params[0].memref.size = C2PA_PUBKEY_LEN;
        return TEE_SUCCESS;
}

/* ------------------------------------------------------------------
 * TA_CMD_ATTEST_CAPTURE  [0] memref in, 32-byte hash
 *                        [1] value out: capture id
 * ------------------------------------------------------------------ */
static TEE_Result cmd_attest(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                             TEE_PARAM_TYPE_VALUE_OUTPUT,
                                             TEE_PARAM_TYPE_NONE,
                                             TEE_PARAM_TYPE_NONE);
        struct witness *w;
        TEE_Result res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;

        /* THE CHECK THAT MATTERS. */
        if (!caller_is_kernel()) {
                EMSG("attest refused: caller is not the kernel");
                return TEE_ERROR_ACCESS_DENIED;
        }

        if (params[0].memref.size != C2PA_HASH_LEN)
                return TEE_ERROR_BAD_PARAMETERS;

        if (TEE_CheckMemoryAccessRights(TEE_MEMORY_ACCESS_READ |
                                        TEE_MEMORY_ACCESS_ANY_OWNER,
```

```
                                        params[0].memref.buffer,
                                        C2PA_HASH_LEN) != TEE_SUCCESS)
                return TEE_ERROR_ACCESS_DENIED;

        res = counter_load();
        if (res != TEE_SUCCESS)
                return res;

        g_counter++;

        w = &g_witness[g_next];
        g_next = (g_next + 1) % C2PA_WITNESS_SLOTS;

        /* ONE fetch out of shared memory, into TA memory. */
        TEE_MemMove(w->hash, params[0].memref.buffer, C2PA_HASH_LEN);
        w->capture_id = g_counter;
        w->used  = false;
        w->valid = true;

        /*
         * Persist the counter on every capture. This is the expensive
         * line in the whole TA - a secure-storage write per frame - and
         * on a real camera you would batch it or accept a bounded
         * rollback. Left synchronous here because correctness is
         * easier to teach than the compromise.
         */
        res = counter_store();
        if (res != TEE_SUCCESS)
                return res;

        params[1].value.a = (uint32_t)(g_counter & 0xffffffffu);
        params[1].value.b = (uint32_t)(g_counter >> 32);
        return TEE_SUCCESS;
}

/* ------------------------------------------------------------------
 * TA_CMD_SIGN_CLAIM  [0] memref in  the COSE Sig_structure
 *                    [1] memref in  the declared binding (32 bytes)
 *                    [2] memref out the signature (64 bytes)
 * ------------------------------------------------------------------ */
static TEE_Result cmd_sign_claim(uint32_t ptypes, TEE_Param params[4])
{
        const uint32_t exp = TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                                             TEE_PARAM_TYPE_MEMREF_INPUT,
                                             TEE_PARAM_TYPE_MEMREF_OUTPUT,
                                             TEE_PARAM_TYPE_NONE);
        TEE_ObjectHandle    key = TEE_HANDLE_NULL;
        TEE_OperationHandle op  = TEE_HANDLE_NULL;
        uint8_t  binding[C2PA_HASH_LEN];
        uint8_t  digest[C2PA_HASH_LEN];
        uint8_t  sig[C2PA_SIG_LEN];
        uint32_t siglen = sizeof(sig);
        uint32_t dlen = sizeof(digest);
        uint8_t *sigstruct = NULL;
        uint32_t sslen;
        struct witness *match = NULL;
        unsigned int i;
        bool found_inline = false;
        TEE_Result res;

        if (ptypes != exp)
                return TEE_ERROR_BAD_PARAMETERS;
        if (params[1].memref.size != C2PA_HASH_LEN)
```

```
        return TEE_ERROR_BAD_PARAMETERS;
    if (params[2].memref.size < C2PA_SIG_LEN) {
        params[2].memref.size = C2PA_SIG_LEN;
        return TEE_ERROR_SHORT_BUFFER;
    }

    sslen = params[0].memref.size;
    if (sslen == 0 || sslen > 8192)
        return TEE_ERROR_BAD_PARAMETERS;

    /* Copy BOTH inputs out of shared memory before looking at
     * either. The normal world can rewrite that buffer between
     * any two reads (OP-TEE chapter 20).
     */
    sigstruct = TEE_Malloc(sslen, 0);
    if (!sigstruct)
        return TEE_ERROR_OUT_OF_MEMORY;

    TEE_MemMove(sigstruct, params[0].memref.buffer, sslen);
    TEE_MemMove(binding, params[1].memref.buffer, C2PA_HASH_LEN);

    /* ---- CHECK 1: did we witness this image? ---- */
    for (i = 0; i < C2PA_WITNESS_SLOTS; i++) {
        if (!g_witness[i].valid || g_witness[i].used)
            continue;
        if (TEE_MemCompare(g_witness[i].hash, binding,
                           C2PA_HASH_LEN) == 0) {
            match = &g_witness[i];
            break;
        }
    }
    if (!match) {
        EMSG("refusing to sign: binding not witnessed");
        res = TEE_ERROR_ACCESS_DENIED;
        goto out;
    }

    /* ---- CHECK 2: does the binding actually appear in what we
     * are about to sign?
     *
     * Without this, user space could declare a witnessed hash and
     * hand us a Sig_structure describing a completely different
     * image - and we would sign it.
     *
     * We do NOT parse CBOR to find it. A parser here is a large
     * attacker-reachable surface. A substring search over a
     * bounded buffer is a few lines and cannot go wrong.
     *
     * The cost of this simplicity: we prove the 32 bytes are
     * PRESENT, not that they occupy the c2pa.hash.data field.
     * A hostile user space could place them elsewhere in the
     * claim. Closing that gap properly means either parsing in
     * the TA, or having the TA build the claim itself - and the
     * latter is what a production camera does.
     */
    if (sslen >= C2PA_HASH_LEN) {
        for (i = 0; i + C2PA_HASH_LEN <= sslen; i++) {
            if (TEE_MemCompare(sigstruct + i, binding,
                               C2PA_HASH_LEN) == 0) {
                found_inline = true;
                break;
            }
        }
```

```
        }
        if (!found_inline) {
                EMSG("refusing to sign: binding absent from the claim");
                res = TEE_ERROR_ACCESS_DENIED;
                goto out;
        }

        /* ---- Hash the Sig_structure, then sign the digest ---- */
        res = TEE_AllocateOperation(&op, TEE_ALG_SHA256,
                                    TEE_MODE_DIGEST, 0);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_DigestDoFinal(op, sigstruct, sslen, digest, &dlen);
        TEE_FreeOperation(op);
        op = TEE_HANDLE_NULL;
        if (res != TEE_SUCCESS)
                goto out;

        res = open_or_create_key(&key);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_AllocateOperation(&op, TEE_ALG_ECDSA_P256,
                                    TEE_MODE_SIGN, 256);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_SetOperationKey(op, key);
        if (res != TEE_SUCCESS)
                goto out;

        res = TEE_AsymmetricSignDigest(op, NULL, 0,
                                       digest, C2PA_HASH_LEN,
                                       sig, &siglen);
        if (res != TEE_SUCCESS)
                goto out;

        /* Single use. A witnessed capture buys exactly one signature,
         * so a leaked Sig_structure cannot be re-signed later.
         */
        match->used = true;

        /* Write the output once, at the end, on success only. */
        TEE_MemMove(params[2].memref.buffer, sig, siglen);
        params[2].memref.size = siglen;
out:
        if (op != TEE_HANDLE_NULL)
                TEE_FreeOperation(op);
        if (key != TEE_HANDLE_NULL)
                TEE_CloseObject(key);
        if (sigstruct) {
                TEE_MemFill(sigstruct, 0, sslen);
                TEE_Free(sigstruct);
        }
        return res;
}

/* ------------------------------------------------------------------
 * Entry points
 * ------------------------------------------------------------------ */
TEE_Result TA_CreateEntryPoint(void)
{
```

```
        TEE_MemFill(g_witness, 0, sizeof(g_witness));
        g_next = 0;
        IMSG("c2pa TA loaded");
        return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
        IMSG("c2pa TA unloaded");
}

TEE_Result TA_OpenSessionEntryPoint(uint32_t ptypes,
                                    TEE_Param params[4] __unused,
                                    void **sess __unused)
{
        if (ptypes != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE,
                                      TEE_PARAM_TYPE_NONE))
                return TEE_ERROR_BAD_PARAMETERS;
        return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void *sess __unused) { }

TEE_Result TA_InvokeCommandEntryPoint(void *sess __unused, uint32_t cmd,
                                      uint32_t ptypes, TEE_Param params[4])
{
        switch (cmd) {
        case TA_CMD_GET_PUBKEY:     return cmd_get_pubkey(ptypes, params);
        case TA_CMD_ATTEST_CAPTURE: return cmd_attest(ptypes, params);
        case TA_CMD_SIGN_CLAIM:     return cmd_sign_claim(ptypes, params);
        default:                    return TEE_ERROR_NOT_SUPPORTED;
        }
}
```

BE PRECISE ABOUT WHAT CHECK 2 BUYS

The substring search proves the witnessed hash is *somewhere* in the bytes being signed. It does not prove it is the `c2pa.hash.data` value rather than, say, a thumbnail hash the caller planted. That residual gap is stated in the code comment on purpose. The honest ways to close it are a CBOR parser inside the TA — more attack surface — or having the TA construct the claim itself, which is what a production camera does and what the exercises at the end suggest you try.

Building the claim in user space

```
/* tools/c2pa-sign.c - a minimal CBOR/COSE encoder, and the
 * assembly of one real Content Credential.
 *
 * Nothing here is secret, so it all lives in the normal world.
 */

#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* ---- a 120-line CBOR writer, which is all C2PA needs ---- */
struct cbor {
        uint8_t *buf;
        size_t   len, cap;
};

static void cb_raw(struct cbor *c, const void *p, size_t n)
{
        if (c->len + n > c->cap) {
                c->cap = (c->len + n) * 2;
                c->buf = realloc(c->buf, c->cap);
        }
        memcpy(c->buf + c->len, p, n);
        c->len += n;
}

static void cb_byte(struct cbor *c, uint8_t b) { cb_raw(c, &b, 1); }

/* major type in the top 3 bits, then the argument */
static void cb_head(struct cbor *c, uint8_t major, uint64_t val)
{
        major <<= 5;
        if (val < 24) {
                cb_byte(c, major | (uint8_t)val);
        } else if (val < 0x100) {
                cb_byte(c, major | 24); cb_byte(c, (uint8_t)val);
        } else if (val < 0x10000) {
                cb_byte(c, major | 25);
                cb_byte(c, (uint8_t)(val >> 8)); cb_byte(c, (uint8_t)val);
        } else if (val < 0x100000000ULL) {
                cb_byte(c, major | 26);
                for (int i = 3; i >= 0; i--)
                        cb_byte(c, (uint8_t)(val >> (i * 8)));
        } else {
                cb_byte(c, major | 27);
                for (int i = 7; i >= 0; i--)
                        cb_byte(c, (uint8_t)(val >> (i * 8)));
        }
}

static void cb_uint(struct cbor *c, uint64_t v)  { cb_head(c, 0, v); }
static void cb_nint(struct cbor *c, uint64_t v)  { cb_head(c, 1, v); }
static void cb_bstr(struct cbor *c, const void *p, size_t n)
{ cb_head(c, 2, n); cb_raw(c, p, n); }
static void cb_tstr(struct cbor *c, const char *s)
{ cb_head(c, 3, strlen(s)); cb_raw(c, s, strlen(s)); }
static void cb_array(struct cbor *c, size_t n)   { cb_head(c, 4, n); }
static void cb_map(struct cbor *c, size_t n)     { cb_head(c, 5, n); }
```

```
/* ---- the protected header ----
 *   { 1: -7, 33: [ cert ] }
 * CBOR negative integers encode as -1 - n, so -7 is nint(6).
 */
static void build_protected(struct cbor *out,
                            const uint8_t *cert, size_t certlen)
{
        cb_map(out, 2);
        cb_uint(out, 1);   cb_nint(out, 6);        /* alg = ES256    */
        cb_uint(out, 33);                          /* x5chain        */
        cb_array(out, 1);  cb_bstr(out, cert, certlen);
}

/* ---- the Sig_structure ----
 *   [ "Signature1", protected, external_aad, payload ]
 * THIS is what gets hashed and signed. Not the claim alone.
 */
static void build_sigstruct(struct cbor *out,
                            const uint8_t *prot, size_t protlen,
                            const uint8_t *claim, size_t claimlen)
{
        cb_array(out, 4);
        cb_tstr(out, "Signature1");
        cb_bstr(out, prot, protlen);
        cb_bstr(out, "", 0);              /* external_aad: empty     */
        cb_bstr(out, claim, claimlen);
}

/* ---- the claim ----
 * A minimal but structurally correct C2PA claim. A real generator
 * emits more assertions; the fields below are the ones a verifier
 * requires.
 */
static void build_claim(struct cbor *out,
                        const uint8_t *binding,
                        const uint8_t *assertion_hash)
{
        cb_map(out, 6);

        cb_tstr(out, "dc:title");
        cb_tstr(out, "capture.jpg");

        cb_tstr(out, "dc:format");
        cb_tstr(out, "image/jpeg");

        cb_tstr(out, "instanceID");
        cb_tstr(out, "xmp:iid:6b3f0e12-1c44-4a9e-9f77-0c2b8d4e51a7");

        cb_tstr(out, "claim_generator");
        cb_tstr(out, "elh-c2pa-capture/0.1");

        cb_tstr(out, "alg");
        cb_tstr(out, "sha256");

        /* Each assertion is referenced by a hashed URI: where it is,
         * and what it hashes to. Tamper with an assertion and this
         * hash stops matching.
         */
        cb_tstr(out, "assertions");
        cb_array(out, 1);
          cb_map(out, 3);
          cb_tstr(out, "url");
          cb_tstr(out, "self#jumbf=c2pa.assertions/c2pa.hash.data");
```

```
          cb_tstr(out, "alg");  cb_tstr(out, "sha256");
          cb_tstr(out, "hash"); cb_bstr(out, assertion_hash, 32);

        (void)binding;   /* it lives inside the assertion, below */
}

/* ---- the c2pa.hash.data assertion ----
 * The hard binding. `exclusions` names the byte ranges that are
 * NOT covered - which must include the manifest itself, or the
 * hash would have to contain its own value.
 */
static void build_hash_assertion(struct cbor *out,
                                 uint64_t excl_start, uint64_t excl_len,
                                 const uint8_t *binding)
{
        cb_map(out, 4);

        cb_tstr(out, "exclusions");
        cb_array(out, 1);
          cb_map(out, 2);
          cb_tstr(out, "start");  cb_uint(out, excl_start);
          cb_tstr(out, "length"); cb_uint(out, excl_len);

        cb_tstr(out, "name"); cb_tstr(out, "jumbf manifest");
        cb_tstr(out, "alg");  cb_tstr(out, "sha256");
        cb_tstr(out, "hash"); cb_bstr(out, binding, 32);
}

/* ---- COSE_Sign1 ----
 *   [ protected, {}, payload, signature ]
 * The signature slot takes the TA's raw r||s unchanged.
 */
static void build_cose_sign1(struct cbor *out,
                             const uint8_t *prot, size_t protlen,
                             const uint8_t *claim, size_t claimlen,
                             const uint8_t sig[64])
{
        cb_array(out, 4);
        cb_bstr(out, prot, protlen);
        cb_map(out, 0);                    /* unprotected: empty     */
        cb_bstr(out, claim, claimlen);
        cb_bstr(out, sig, 64);
}
```

```
/* tools/c2pa-sign.c, continued - the flow */

#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <openssl/sha.h>
#include "c2pa_capture.h"

int main(void)
{
        int fd = open("/dev/c2pa0", O_RDWR);
        if (fd < 0) { perror("/dev/c2pa0"); return 1; }

        /* 1. Capture. The kernel hashes and the TA witnesses, before
         *    we see a single pixel.
         */
        uint8_t *frame = malloc(640 * 480 * 3);
        struct c2pa_capture cap = {
                .version   = C2PA_ABI_VERSION,
                .frame_len = 640 * 480 * 3,
                .frame     = (uint64_t)(uintptr_t)frame,
        };
        if (ioctl(fd, C2PA_IOC_CAPTURE, &cap)) { perror("CAPTURE"); return 1; }

        /* 2. Build the assertion that carries the binding, and hash it.
         *
         *    NOTE the simplification: the hard binding here is the hash
         *    of the RAW FRAME, which is what the kernel witnessed. A
         *    real generator hashes the ENCODED FILE with the manifest
         *    range excluded - so a real pipeline must either encode in
         *    the kernel too, or add a c2pa.actions assertion recording
         *    the encode step and accept that the encoder is trusted.
         *    This is the single biggest gap between this project and a
         *    shipping camera, and it is worth understanding before you
         *    quote the design to anyone.
         */
        struct cbor assertion = {0};
        build_hash_assertion(&assertion, 0, 0, cap.hash);

        uint8_t assertion_hash[32];
        SHA256(assertion.buf, assertion.len, assertion_hash);

        /* 3. The claim, referencing that assertion. */
        struct cbor claim = {0};
        build_claim(&claim, cap.hash, assertion_hash);

        /* 4. The certificate. On QEMU this is self-signed - see the
         *    trust-list note below.
         */
        size_t certlen;
        uint8_t *cert = load_file("device-cert.der", &certlen);

        struct cbor prot = {0};
        build_protected(&prot, cert, certlen);

        /* 5. The Sig_structure - what actually gets signed. */
        struct cbor ss = {0};
        build_sigstruct(&ss, prot.buf, prot.len, claim.buf, claim.len);

        /* 6. Sign it, in the secure world. We DECLARE the binding; the
         *    TA checks it against what the kernel witnessed and against
         *    the bytes we are asking it to sign.
         */
```

```
        struct c2pa_sign sr = {
                .version       = C2PA_ABI_VERSION,
                .sigstruct_len = (uint32_t)ss.len,
                .sigstruct     = (uint64_t)(uintptr_t)ss.buf,
        };
        memcpy(sr.binding, cap.hash, C2PA_HASH_LEN);

        if (ioctl(fd, C2PA_IOC_SIGN, &sr)) {
                perror("SIGN");
                fprintf(stderr, "EACCES here means the TA refused - "
                                "either the binding was not witnessed, or "
                                "it is absent from the claim\n");
                return 1;
        }

        /* 7. Assemble COSE_Sign1 with the raw r||s straight from the
         *    TA. No DER conversion: COSE wants exactly this form.
         */
        struct cbor cose = {0};
        build_cose_sign1(&cose, prot.buf, prot.len,
                         claim.buf, claim.len, sr.signature);

        write_file("claim.cbor",      claim.buf,     claim.len);
        write_file("assertion.cbor",  assertion.buf, assertion.len);
        write_file("signature.cose",  cose.buf,      cose.len);
        write_file("capture.raw",     frame,         cap.frame_len);

        printf("capture id %llu, %u bytes of COSE_Sign1\n",
               (unsigned long long)cap.capture_id, (unsigned)cose.len);
        close(fd);
        return 0;
}
```

The manifest container

```
The three CBOR blobs above must be packed into JUMBF boxes
and embedded in the file:

JPEG file
 +-- SOI
 +-- APP11  "JP"  <- JUMBF superbox, split across segments
 |    +-- c2pa   (manifest store)
 |         +-- <manifest label>
 |              +-- c2pa.assertions
 |              |    +-- c2pa.hash.data   <- assertion.cbor
 |              +-- c2pa.claim            <- claim.cbor
 |              +-- c2pa.signature        <- signature.cose
 +-- ... the actual image data ...
 +-- EOI

Hand-rolling this is possible and tedious: box headers, UUIDs
per box type, label strings, and the APP11 segmentation rules
for payloads over 64 KB.

DO NOT hand-roll it for anything real. Use c2pa-rs, which is
the reference implementation, and plug the TEE in as a SIGNER
rather than reimplementing the container:

  c2pa-rs exposes a `Signer` trait whose whole job is

      fn sign(&self, data: &[u8]) -> Result<Vec<u8>>

  where `data` is precisely the Sig_structure our TA already
  signs. Implement it over /dev/c2pa0 and the SDK does the
  claim, the assertions, the JUMBF and the embedding.

That is the integration this project is really for: the SDK
handles the format, and the TEE handles the only part that
must not be forgeable.
```

Verifying

```
# Build and install everything (OP-TEE chapter 4 for the tree).
$ cd ~/c2pa/ta && make CROSS_COMPILE=aarch64-linux-gnu- \
      TA_DEV_KIT_DIR=~/optee/optee_os/out/arm-plat-vexpress/export-ta_arm64
$ cp *.ta ~/optee/out-br/target/lib/optee_armtz/
$ cp ~/c2pa/driver/c2pa_capture.ko ~/c2pa/tools/c2pa-sign \
     ~/optee/out-br/target/root/
$ cd ~/optee/build && make buildroot run-only

# In the guest:
# insmod c2pa_capture.ko
[   14.221408] c2pa-capture: /dev/c2pa0 ready
# ./c2pa-sign
capture id 1, 412 bytes of COSE_Sign1

# Secure console:
I/TA: c2pa TA loaded
I/TA: generating device key

# --- Verify the COSE signature by hand, on the host ---------------
# Reconstruct the Sig_structure, hash it, check the signature
# against the device public key. This is the whole cryptographic
# claim, checked without any C2PA tooling at all.

$ python3 <<'EOF'
import hashlib, binascii
from cryptography.hazmat.primitives.asymmetric import ec, utils
from cryptography.hazmat.primitives import hashes

pub = bytes.fromhex(open("pubkey.hex").read().strip())
key = ec.EllipticCurvePublicNumbers(
        int.from_bytes(pub[:32], "big"),
        int.from_bytes(pub[32:], "big"),
        ec.SECP256R1()).public_key()

ss  = open("sigstruct.cbor", "rb").read()
raw = open("signature.raw", "rb").read()      # 64 bytes, r||s
der = utils.encode_dss_signature(
        int.from_bytes(raw[:32], "big"),
        int.from_bytes(raw[32:], "big"))

key.verify(der, ss, ec.ECDSA(hashes.SHA256()))
print("COSE signature verifies")
EOF
COSE signature verifies

# Note the conversion on the Python side: `cryptography`, like
# OpenSSL, wants DER. COSE and the TA speak raw r||s. Same
# mathematics, two conventions.

# --- And with the reference tooling -------------------------------
$ cargo install c2patool
$ c2patool capture.jpg
{
  "manifests": {
    "urn:uuid:...": {
      "claim_generator": "elh-c2pa-capture/0.1",
      "format": "image/jpeg",
      "assertions": [ { "label": "c2pa.hash.data" } ],
      "signature_info": {
        "alg": "Es256",
```

```
        "issuer": "elh test device",
        "cert_serial_number": "..."
      }
    }
  },
  "validation_status": [
    {
      "code": "signingCredential.untrusted",
      "explanation": "signing certificate is not on the trust list"
    }
  ]
}

# signingCredential.untrusted is EXPECTED and CORRECT here. The
# signature is valid; the certificate is self-signed and chains
# to nothing anyone trusts. See below.
```

WHAT STILL STANDS BETWEEN THIS AND A SHIPPABLE CAMERA

1. A real certificate. C2PA validators check the leaf against a trust list of accepted CAs, and require specific extended key usages. A self-signed cert produces `signingCredential.untrusted` for ever. Getting on the list is a commercial and audit process, not a code change. **2. A trusted timestamp.** Without an RFC 3161 `sigTst`, a credential stops validating when the signing certificate expires. The TA's monotonic counter is *not* a timestamp — it proves ordering, not time. **3. Binding to the encoded file, not the raw frame** — see the comment in `c2pa-sign.c`. **4. A hardened key.** On QEMU the HUK is a fixed test value, so secure storage is not device-bound (OP-TEE chapter 15).

Testing the property, not the happy path

```
# A working demo proves nothing. These four tests are the ones
# that matter, and all four must behave as shown.

# 1. Sign a claim about an image the kernel never produced.
#    Craft a Sig_structure containing a hash of your own choosing.
# ./c2pa-attack --forge-binding
SIGN: Permission denied
#    TA log: E/TA: refusing to sign: binding not witnessed

# 2. Declare a witnessed binding, but sign a DIFFERENT claim.
# ./c2pa-attack --swap-claim
SIGN: Permission denied
#    TA log: E/TA: refusing to sign: binding absent from the claim

# 3. Replay: sign the same capture twice.
# ./c2pa-attack --replay
first  signature ok
second SIGN: Permission denied
#    Single-use witness slots.

# 4. Attest from user space, bypassing the kernel.
#    Open a session directly with libteec and call ATTEST.
# ./c2pa-attack --user-attest
TEEC_InvokeCommand: 0xffff000f
#    ACCESS_DENIED - the TA saw LOGIN_USER, not LOGIN_REE_KERNEL.

# If any of these SUCCEEDS, the project is decoration. Write
# them before you write the manifest code, not after.
```

Extensions worth doing

Exercise	What it teaches
Implement the `c2pa-rs` `Signer` trait over `/dev/c2pa0`	The real integration path, and how little of the SDK you have to touch
Move JPEG encoding into the kernel and bind to the encoded bytes	Closes the biggest gap in the design
Have the TA build the claim CBOR itself	Removes the residual gap in check 2, at the cost of CBOR in the TA
Add an RFC 3161 timestamp to the COSE unprotected header	Why signatures outlive certificates
Add `c2pa.actions` and an ingredient referencing a parent asset	How edit chains work, which is most of what C2PA is for
Replace the test pattern with a V4L2 `vivid` device	A realistic capture path (chapter 29)
Store the witness ring in secure storage instead of TA globals	Survives TA unload; costs a write per capture
Run the hostile client from OP-TEE chapter 20 against the TA	Whether the parameter validation is real
Measure the per-frame cost of the synchronous counter write	Why real cameras batch, and what they give up

How it shows up

Symptom	Cause
Signing always returns `-EACCES`	The declared binding does not match any witnessed hash — check that `CAPTURE` ran first and the hash was not modified
`-EACCES` with the right binding, after many frames	The witness slot aged out of the ring; sign sooner or enlarge `C2PA_WITNESS_SLOTS`
Second signature over one capture fails	Correct — witness slots are single-use
All witnessed captures vanish between runs	`TA_FLAG_INSTANCE_KEEP_ALIVE` missing
User-space attest succeeds	**The property is broken.** The login check is absent or the driver is not using `LOGIN_REE_KERNEL`
`c2patool`: `signingCredential.untrusted`	Expected on a self-signed certificate
`c2patool`: `claimSignature.mismatch`	The Sig_structure you signed is not the one you embedded — most often the protected header was re-encoded between signing and assembly
`c2patool`: `assertion.hashedURI.mismatch`	The assertion bytes changed after its hash went into the claim
`c2patool`: `assertion.dataHash.mismatch`	The hard binding does not match the file — the exclusion range is wrong, or you bound to the raw frame and embedded a JPEG
Signature verifies in Python, rejected by `c2patool`	DER versus raw r‖s confusion in the COSE signature slot
Verification fails about 1 time in 128	ECC coordinates not left-padded to 32 bytes
Counter resets on reboot	Secure storage on a volatile rootfs, or no RPMB (OP-TEE chapter 17)

Check yourself

✓ Write out a `Sig_structure` and say which of its four elements is hashed and signed.

- ✓ State which signature encoding COSE uses and contrast it with OpenSSL's.
- ✓ Explain why `ATTEST` is restricted to `LOGIN_REE_KERNEL`.
- ✓ Give the two checks the TA performs before signing, and what each one alone would fail to prevent.
- ✓ State precisely what check 2 does *not* prove, and two ways to close that gap.
- ✓ Explain why the witness ring is bounded and what the bound costs.
- ✓ Explain why witness slots are single-use.
- ✓ Describe the difference between binding to the raw frame and binding to the encoded file, and why it matters.
- ✓ Explain what `signingCredential.untrusted` means and why it is expected here.
- ✓ Say why a monotonic counter is not a substitute for an RFC 3161 timestamp.
- ✓ Run all four attack tests and confirm each is refused for the stated reason.
- ✓ Describe how you would plug this TA into `c2pa-rs` instead of hand-rolling JUMBF.

TRACK

Windows kernel & drivers

NT internals and KMDF driver development, ending in a complete signed driver and its user-mode client.

27 chapters

NT architecture

Windows solves the same problems as Linux with a different structure, different names, and a different philosophy about stability. This chapter is the map — and because you have just finished the Linux track, the comparisons will do most of the teaching.

The layers

```
 USER MODE
 +-----------------------------------------------------+
 | applications                                        |
 +-----------------------------------------------------+
 | subsystem DLLs:  kernel32.dll  user32.dll  advapi32 |
 +-----------------------------------------------------+
 | ntdll.dll        the syscall stubs. NtCreateFile etc.  |
 +--------------------------+--------------------------+
                            |  syscall (SVC on arm64)
 ===========================================================
 KERNEL MODE                |
 +--------------------------v--------------------------+
 |  ntoskrnl.exe  -  THE EXECUTIVE                      |
 |                                                      |
 |   Ob  object manager     Io  I/O manager             |
 |   Mm  memory manager     Ps  process/thread manager  |
 |   Cc  cache manager      Se  security reference monitor|
 |   Cm  configuration      Ex  executive support       |
 |       (registry)         Po  power manager           |
 |   Pnp plug and play      Rtl runtime library         |
 +------------------------------------------------------+
 |  KERNEL (Ke)  -  scheduling, interrupts, spinlocks,  |
 |                  DPCs, IRQL. The lowest software layer.|
 +------------------------------------------------------+
 |  HAL  -  hardware abstraction layer                  |
 |          interrupt controller, timers, bus access    |
 +------------------------------------------------------+
 |  DRIVERS  -  loaded into the same address space,     |
 |              at the same privilege, as everything above|
 +------------------------------------------------------+

THE PREFIXES ARE THE MAP. Every kernel function begins with
the two- or three-letter tag of the component that owns it:

   ObReferenceObjectByHandle    object manager
   IoCreateDevice               I/O manager
   MmGetSystemAddressForMdlSafe memory manager
   KeAcquireSpinLock            kernel
   ExAllocatePool2              executive
   PsGetCurrentProcessId        process manager
   RtlCopyMemory                runtime library
   ZwCreateFile                 the kernel-mode syscall entry

Learn the prefixes and you can guess where to look for
almost anything - which is the Windows equivalent of knowing
the Linux source tree layout (Linux ch. 01).
```

Against Linux

	Linux	Windows
Driver ABI	**Unstable** — rebuild every version	**Stable** — a 2015 driver often still loads
Source	Open; upstream or perish	Closed; drivers ship as binaries
Driver model	One: `struct device_driver`	Several: WDM, KMDF, UMDF
Hardware description	Device tree	ACPI and the registry
Installation	Copy a `.ko`, `modprobe`	An INF file, driver store, PnP
Signing	Optional (ch. 15)	**Mandatory** on x64 and arm64
Priority discipline	Context (process/softirq/hardirq)	**IRQL**, a numeric level
Sleeping rule	"Am I in atomic context?"	"Is IRQL <= PASSIVE_LEVEL?"
Request object	`struct bio`, `struct urb`, ad hoc	**The IRP**, universally
Layering	Subsystem-specific	**The device stack**, universal
Allocation	`kmalloc(size, GFP_*)`	`ExAllocatePool2(flags, size, tag)`
Debugging	printk, ftrace, KASAN	WinDbg, Driver Verifier, WPP

THE ONE DIFFERENCE THAT SHAPES EVERYTHING ELSE

Windows guarantees a **stable kernel driver ABI**. That single decision explains the closed driver ecosystem, the INF/driver-store installation model, mandatory signing, the WDF frameworks, and why Microsoft invests so heavily in verification tools: they cannot fix your driver during a refactor, because they never see it. Linux made the opposite trade — churn the API freely, and fix every in-tree driver when you do.

Neither is wrong. They are consistent answers to "who is responsible for keeping drivers working".

A syscall, end to end

```
application:  ReadFile(hFile, buf, len, &read, NULL);
      |
      v
kernel32.dll: parameter marshalling, then
      |
      v
ntdll.dll:    NtReadFile
                mov  x8, #0x06        ; the syscall number
                svc  #0               ; (arm64)
      |
      v  =============== kernel mode ===============
      |
KiSystemService -> the SSDT -> NtReadFile in ntoskrnl
      |
      v
I/O MANAGER (Io):
      · validate the handle    -> ObReferenceObjectByHandle
      · find the FILE_OBJECT and its DEVICE_OBJECT
      · BUILD AN IRP: IRP_MJ_READ, with the buffer, length
        and offset
      · IoCallDriver(deviceObject, irp)
      |
      v
THE TOP DRIVER in the stack
      · may complete it, or pass it down
      |
      v
... each lower driver in turn ...
      |
      v
THE FUNCTION DRIVER (yours)
      · does the work, or queues the IRP and returns
        STATUS_PENDING
      |
      v
IoCompleteRequest(irp, IO_NO_INCREMENT)
      · completion routines run, bottom-up
      · the I/O manager copies results back
      · the waiting thread is woken
      |
      v
ReadFile returns TRUE

COMPARE Linux: read() -> vfs_read -> f_op->read on your
file_operations. One indirection, one driver.

Windows routes EVERYTHING through a self-contained REQUEST
OBJECT (the IRP) that travels down a STACK of drivers, and
may complete asynchronously at any level. That is more
machinery, and it is what makes filter drivers, layered
storage, and asynchronous I/O uniform rather than
per-subsystem.
```

Everything is an object

```
/* The object manager (Ob) is the closest Windows analogue to
   Linux's VFS-plus-everything, and it is more general.     */

   \                        the root
   \Device\                 device objects
   \Device\HarddiskVolume1
   \Driver\                 loaded drivers
   \DosDevices\  (\??\)     symbolic links: C: -> \Device\...
   \BaseNamedObjects\       named events, mutexes, sections
   \Windows\                windowing
   \REGISTRY\               the registry, as objects
   \KernelObjects\

/* EVERY object has:
     a header with a type, a NAME (optionally), a reference
     count, a handle count, and a security descriptor
     a body specific to its type

   TYPES: Process, Thread, File, Device, Driver, Event,
   Mutant, Semaphore, Timer, Section, Key, Token, Job, ...

   HANDLES are per-process indices into a handle table, with
   an access mask. A handle is NOT a pointer, and converting
   one is an explicit, checked operation:                   */

        status = ObReferenceObjectByHandle(handle,
                                           FILE_READ_DATA,
                                           *IoFileObjectType,
                                           UserMode,   /* CHECK
                                                          ACCESS */
                                           &fileObject,
                                           NULL);
        ...
        ObDereferenceObject(fileObject);

/* THE `UserMode` ARGUMENT IS A SECURITY CONTROL: it tells the
   object manager to enforce the access mask as if a user-mode
   caller asked. Passing KernelMode skips those checks - and
   passing it while handling a user request is a classic
   privilege-escalation bug (chapter 03).                   */

/* SEE IT: the WinObj tool from Sysinternals browses the whole
   namespace. Run it once; it makes the model concrete.   */
```

The registry, not the device tree

```
; Where a driver's configuration lives:

HKLM\SYSTEM\CurrentControlSet\Services\MyDriver
    ImagePath     = \SystemRoot\System32\drivers\mydriver.sys
    Type          = 1          ; SERVICE_KERNEL_DRIVER
    Start         = 3          ; 0=boot 1=system 2=auto
                               ; 3=demand 4=disabled
    ErrorControl  = 1
    Group         = Extended Base

HKLM\SYSTEM\CurrentControlSet\Enum\PCI\VEN_1234&DEV_5678\...
    Service       = MyDriver   ; which driver claims it
    ClassGUID     = {...}
    HardwareID    = PCI\VEN_1234&DEV_5678
    Driver        = {class}\0001

; Per-device settings, the closest thing to DT properties:
HKLM\SYSTEM\CurrentControlSet\Enum\...\Device Parameters
    FifoDepth     = 32
    SampleRate    = 48000

/* READING them from a driver, via WDF: */
        WDFKEY key;
        ULONG  value;
        DECLARE_CONST_UNICODE_STRING(name, L"FifoDepth");

        status = WdfDeviceOpenRegistryKey(device,
                        PLUGPLAY_REGKEY_DEVICE,
                        KEY_READ, WDF_NO_OBJECT_ATTRIBUTES,
                        &key);
        if (NT_SUCCESS(status)) {
                status = WdfRegistryQueryULong(key, &name,
                                               &value);
                WdfRegistryClose(key);
        }
        if (!NT_SUCCESS(status))
                value = DEFAULT_FIFO_DEPTH;   /* always have a
                                                 default      */

/* WHERE THE VALUES COME FROM: the INF file writes them at
   install time (chapter 19). So the INF plays the role the
   device tree plays on Linux - describing this particular
   device - but it is applied once, into the registry, rather
   than parsed at every boot.

   ON ARM64 WINDOWS, hardware enumeration comes from ACPI
   rather than a device tree: the ACPI namespace describes the
   devices, and the PnP manager creates a device object for
   each.                                                    */
```

NTSTATUS

```
/* Windows kernel functions return NTSTATUS, not a negative
   errno.                                                    */

        NTSTATUS status;

        status = SomeFunction();
        if (!NT_SUCCESS(status))
                return status;

/* THE ENCODING - bits 31:30 are the severity: */
        0x0.......   SUCCESS
        0x4.......   INFORMATIONAL
        0x8.......   WARNING
        0xC.......   ERROR

        NT_SUCCESS(s)       s >= 0     (severity 0 or 1)
        NT_INFORMATION(s)
        NT_WARNING(s)
        NT_ERROR(s)

/* NOTE NT_SUCCESS IS TRUE FOR WARNINGS. A function returning
   STATUS_BUFFER_OVERFLOW (0x80000005) passes NT_SUCCESS().
   Checking `status == STATUS_SUCCESS` and checking
   `NT_SUCCESS(status)` are DIFFERENT tests, and choosing the
   wrong one is a real bug. Be deliberate.                */

/* THE ONES YOU WILL SEE CONSTANTLY: */
        STATUS_SUCCESS
        STATUS_PENDING                /* async, will complete
                                         later - not an error */
        STATUS_INVALID_PARAMETER
        STATUS_INSUFFICIENT_RESOURCES  /* = -ENOMEM */
        STATUS_BUFFER_TOO_SMALL
        STATUS_INVALID_DEVICE_REQUEST  /* = -ENOTTY */
        STATUS_DEVICE_NOT_READY
        STATUS_ACCESS_DENIED
        STATUS_NOT_SUPPORTED
        STATUS_CANCELLED
        STATUS_TIMEOUT
        STATUS_UNSUCCESSFUL           /* the last resort */

/* Win32 error codes are DIFFERENT again; user mode sees those
   via GetLastError(). The I/O manager translates. So one
   failure has two names, and the WinDbg command
   `!error <value>` decodes either.                       */
```

UNICODE_STRING

```
/* Kernel strings are counted UTF-16, not NUL-terminated
   char*. This catches every newcomer once.                */

typedef struct _UNICODE_STRING {
        USHORT Length;           /* in BYTES, not characters */
        USHORT MaximumLength;    /* in BYTES */
        PWSTR  Buffer;           /* NOT necessarily
                                    NUL-terminated!          */
} UNICODE_STRING, *PUNICODE_STRING;

/* CREATING one: */
        UNICODE_STRING name;
        RtlInitUnicodeString(&name, L"\\Device\\MyDevice");

        /* or, for a constant, with no runtime cost: */
        DECLARE_CONST_UNICODE_STRING(name,
                                     L"\\Device\\MyDevice");

/* WORKING with them: */
        RtlCopyUnicodeString(&dst, &src);        /* truncates */
        RtlAppendUnicodeStringToString(&dst, &src);
        RtlCompareUnicodeString(&a, &b, TRUE);   /* case-insens */
        RtlEqualUnicodeString(&a, &b, TRUE);
        RtlUnicodeStringToAnsiString(&ansi, &uni, TRUE);

/* PRINTING one - the %wZ specifier takes a POINTER: */
        DbgPrint("name: %wZ\n", &name);

/* THE TRAPS:
     · Length is in BYTES. A 5-character string has
       Length == 10. Off-by-two-times is common.
     · the buffer may not be NUL-terminated, so wcslen() and
       friends can run off the end. Use Length.
     · a UNICODE_STRING from user mode has an attacker-chosen
       Length AND Buffer. Validate both, and probe the buffer
       (chapter 07).                                          */
```

Getting set up

```
# WHAT YOU NEED:
#   Visual Studio 2022 (Community is fine)
#     + "Desktop development with C++"
#     + the Windows SDK
#   The Windows Driver Kit (WDK), matching the SDK version
#   The WDK's Visual Studio extension
#
# https://learn.microsoft.com/windows-hardware/drivers/download-the-wdk

# NEW PROJECT -> "Kernel Mode Driver, Empty (KMDF)"
# That template gives you a .sys, a .inf, and the signing and
# packaging steps already wired up.

# A TEST MACHINE IS NOT OPTIONAL. A driver bug bugchecks the
# machine, and you do not want that to be your development
# box. Use a VM:
#   Hyper-V, VMware or VirtualBox
#   with a serial or network kernel-debugging connection to
#   the host (chapter 22)

# ENABLE TEST SIGNING on the target, so unsigned-by-Microsoft
# builds will load (chapter 20):
    bcdedit /set testsigning on
    bcdedit /set nointegritychecks off
    shutdown /r /t 0
# A "Test Mode" watermark appears on the desktop. That is
# expected.

# AND SET UP KERNEL DEBUGGING on the target now, before you
# need it:
    bcdedit /debug on
    bcdedit /dbgsettings net hostip:192.168.1.10 port:50000 \
            key:1.2.3.4
```

How it shows up

Symptom	Cause
Driver will not load, "digitally signed" error	Test signing not enabled (ch. 20)
Bugcheck immediately on load	Almost always IRQL or a bad pointer (ch. 02)
String prints as garbage	Used `%s` instead of `%wZ`, or a non-terminated buffer
Buffer overrun in string handling	`Length` is bytes, not characters
Code treats a warning as success	`NT_SUCCESS` is true for `0x8...` — check the specific status
Access check bypassed	Passed `KernelMode` where `UserMode` was required
Device never appears	INF did not match the hardware ID (ch. 19)
Registry value not found	The INF did not write it; always have a default
Debugging the dev box, box is dead	Use a VM — this is the first lesson everyone learns twice

Check yourself

✓ Name the executive components and their function prefixes.

- ✓ List eight differences between Linux and Windows driver architecture.
- ✓ Explain why a stable driver ABI leads to signing, INFs and WDF.
- ✓ Trace `ReadFile` from the application to a driver and back.
- ✓ Explain what an IRP is and how it differs from Linux's per-subsystem request types.
- ✓ Describe the object manager namespace and what a handle is.
- ✓ Explain why `UserMode` versus `KernelMode` matters in an object reference.
- ✓ Say where a driver's per-device configuration lives and how it got there.
- ✓ Explain the NTSTATUS severity encoding and the `NT_SUCCESS` trap.
- ✓ Use `UNICODE_STRING` correctly and name its three traps.
- ✓ Set up a VM with test signing and kernel debugging enabled.

IRQL

Interrupt Request Level is the single most important concept in Windows kernel programming. It is a per-CPU number that determines what your code is allowed to do, and violating its rules is the cause of most driver bugchecks.

The levels

```
 HIGH_LEVEL        (15)  bugcheck, everything masked
 ...
 CLOCK_LEVEL       (13)  the clock interrupt
 IPI_LEVEL         (14)  inter-processor interrupts
 ...
 DIRQL             (3-12) DEVICE interrupt levels - your ISR
                          runs here
 DISPATCH_LEVEL     (2)  the scheduler and DPCs
 APC_LEVEL          (1)  asynchronous procedure calls
 PASSIVE_LEVEL      (0)  normal thread execution
                                ^ lowest

THE RULE THAT DEFINES EVERYTHING:

   Code running at IRQL N can only be interrupted by code at
   IRQL > N, on the same CPU. Code at IRQL N cannot be
   preempted by anything at or below N.

So raising IRQL MASKS everything at or below it. At
DISPATCH_LEVEL the scheduler itself cannot run - which is
why you cannot block, wait, or page fault there.

THE CRITICAL BOUNDARY IS BETWEEN 1 AND 2:

   <= APC_LEVEL       the scheduler still works.
                      You may block, wait, and touch
                      paged memory.

   >= DISPATCH_LEVEL  the scheduler is masked.
                      NO blocking. NO waiting. NO paged
                      memory. NO page faults.

Everything else in this chapter follows from that line.
```

THE LINUX COMPARISON

IRQL does the job that "context" does in Linux (chapter 02), but as an explicit, readable, *settable* number rather than an implicit property of how you were called. Roughly: `PASSIVE_LEVEL` is process context; `DISPATCH_LEVEL` is softirq context or holding a spinlock; `DIRQL` is hardirq context. The difference is that Windows lets you *query* it (`KeGetCurrentIrql()`) and *raise* it deliberately, and every API's documentation states the IRQL it requires.

What you may do at each level

	PASSIVE (0)	APC (1)	DISPATCH (2)	DIRQL (3+)
Block / wait with a timeout	Yes	Yes	**No**	**No**

	PASSIVE (0)	APC (1)	DISPATCH (2)	DIRQL (3+)
Touch paged memory	Yes	Yes	**No**	**No**
Allocate paged pool	Yes	Yes	**No**	**No**
Allocate non-paged pool	Yes	Yes	Yes	Yes
Acquire a fast mutex	Yes	Yes	**No**	**No**
Acquire a spinlock	Yes (raises to DISPATCH)	Yes	Yes	Only at that DIRQL
Access user memory	Yes, in the right process	Careful	**No**	**No**
Call most Zw* / Io* APIs	Yes	Some	Few	Almost none
Queue a DPC	Yes	Yes	Yes	Yes
`DbgPrint`	Yes	Yes	Yes	Careful

```
/* THE PAGED MEMORY RULE IS THE SUBTLE ONE.

   Touching paged-out memory takes a page fault. Servicing a
   page fault requires disk I/O, which requires the scheduler
   to block the thread. At DISPATCH_LEVEL the scheduler cannot
   run.

   RESULT: bugcheck 0xA, IRQL_NOT_LESS_OR_EQUAL.

   And it is a LATENT bug: the memory is usually resident, so
   the code works for weeks. Then the machine is under memory
   pressure, the page is evicted, and the driver crashes -
   apparently at random.

   THIS IS WHY Driver Verifier has a "force IRQL checking"
   option that makes every pageable page non-resident, turning
   the latent bug into a reliable one (chapter 21). Use it.  */
```

Querying and changing IRQL

```
        KIRQL irql = KeGetCurrentIrql();

        if (irql > DISPATCH_LEVEL)
                return STATUS_INVALID_DEVICE_STATE;

/* RAISING and LOWERING - always paired, always LIFO: */
        KIRQL oldIrql;

        KeRaiseIrql(DISPATCH_LEVEL, &oldIrql);
        /* ... the scheduler cannot run here ... */
        KeLowerIrql(oldIrql);

/* You rarely do this explicitly. Acquiring a spinlock does it
   for you: */
        KIRQL oldIrql;

        KeAcquireSpinLock(&lock, &oldIrql);   /* raises to
                                                 DISPATCH     */
        /* ... critical section ... */
        KeReleaseSpinLock(&lock, oldIrql);    /* restores */

/* Or, when you KNOW you are already at DISPATCH_LEVEL: */
        KeAcquireSpinLockAtDpcLevel(&lock);
        ...
        KeReleaseSpinLockFromDpcLevel(&lock);
/* Cheaper - no IRQL change. Calling it below DISPATCH_LEVEL
   is a bug that Driver Verifier catches.                   */

/* ASSERT IT, in every function whose IRQL matters: */
        NT_ASSERT(KeGetCurrentIrql() == PASSIVE_LEVEL);
        NT_ASSERT(KeGetCurrentIrql() <= DISPATCH_LEVEL);

/* And DOCUMENT it in the function header, the way Microsoft's
   own headers do:

     IRQL_requires_max_(DISPATCH_LEVEL)
     NTSTATUS MyFunction(...);

   Those SAL annotations are checked by static analysis
   (chapter 21) - so an annotated codebase catches IRQL
   violations at BUILD time rather than at run time. That is
   the highest-value habit in this chapter.               */
```

SAL annotations

```
/* Windows drivers are annotated with SAL - Microsoft's source
   annotation language. Code Analysis reads them and finds
   real bugs statically.                                      */

_IRQL_requires_max_(DISPATCH_LEVEL)
_IRQL_requires_min_(PASSIVE_LEVEL)
_IRQL_requires_(PASSIVE_LEVEL)
_IRQL_raises_(DISPATCH_LEVEL)
_IRQL_saves_                    /* on an out KIRQL parameter */
_IRQL_restores_

_Must_inspect_result_
_Check_return_

_In_            /* input, must be valid */
_Out_           /* output, written by the callee */
_Inout_
_In_opt_        /* may be NULL */
_In_reads_bytes_(n)
_Out_writes_bytes_(n)

_Requires_lock_held_(lock)
_Acquires_lock_(lock)
_Releases_lock_(lock)

/* A realistic annotated declaration: */
_IRQL_requires_max_(DISPATCH_LEVEL)
_Must_inspect_result_
NTSTATUS
MyQueueRequest(
        _In_ PMY_DEVICE Device,
        _In_reads_bytes_(Length) PVOID Buffer,
        _In_ SIZE_T Length
        );

/* WHY BOTHER: Code Analysis will then tell you, at compile
   time, that you called this from a DIRQL ISR, or ignored its
   return value, or passed a buffer shorter than Length. Those
   are exactly the bugs that otherwise become bugchecks on a
   customer machine.

   ANNOTATE AS YOU WRITE. Retrofitting SAL to an existing
   driver is tedious; adding it to a new function costs
   seconds.                                                   */
```

The bugchecks IRQL causes

```
/* --- 0x0A  IRQL_NOT_LESS_OR_EQUAL --------------------------
   The most common driver bugcheck by a wide margin.

   MEANS: memory was touched at too high an IRQL. Either the
   memory is paged out, or the address is garbage.

   Parameters:
     1  the address referenced
     2  the IRQL at the time
     3  0 = read, 1 = write, 8 = execute
     4  the instruction address

   PARAMETER 2 IS THE DIAGNOSIS. If it is 2 or higher and the
   address looks plausible, you touched paged memory at
   DISPATCH_LEVEL. If the address is near zero, it is a NULL
   dereference that happens to have occurred at high IRQL. */

/* --- 0xD1  DRIVER_IRQL_NOT_LESS_OR_EQUAL ------------------
   The same thing, but the faulting code was identified as a
   driver. Same parameters. Parameter 4 names YOUR function.

/* --- 0xC4  DRIVER_VERIFIER_DETECTED_VIOLATION -------------
   Verifier caught an IRQL rule violation before it became a
   crash. The subcode says which rule (chapter 21).

/* --- 0x1E  KMODE_EXCEPTION_NOT_HANDLED --------------------
   An unhandled exception - often an access violation at high
   IRQL.

/* --- 0x7E  SYSTEM_THREAD_EXCEPTION_NOT_HANDLED ------------
   Same, in a system thread.

/* --- 0x133 DPC_WATCHDOG_VIOLATION -------------------------
   A DPC ran for more than ~20 microseconds too long, or the
   system stayed at DISPATCH_LEVEL for more than ~10 seconds.
   MEANS: your DPC is doing far too much work
   (chapter 10).                                          */

/* READING ONE in WinDbg (chapter 22): */
        !analyze -v
        .bugcheck
        kb                    /* the stack */
        !irql                 /* the IRQL at crash time */
```

Patterns that keep you correct

```
/* 1. KNOW WHICH IRQL YOUR CALLBACK RUNS AT. Every framework
      callback documents it. Memorise the common ones:      */

        DriverEntry                     PASSIVE_LEVEL
        EvtDriverDeviceAdd              PASSIVE_LEVEL
        EvtDevicePrepareHardware        PASSIVE_LEVEL
        EvtDeviceD0Entry                PASSIVE_LEVEL
        EvtIoRead / EvtIoWrite          <= DISPATCH_LEVEL
        EvtIoDeviceControl              <= DISPATCH_LEVEL
        EvtInterruptIsr                 DIRQL
        EvtInterruptDpc                 DISPATCH_LEVEL
        EvtWorkItem                     PASSIVE_LEVEL
        EvtTimerFunc                    DISPATCH_LEVEL
        EvtCleanupCallback              <= DISPATCH_LEVEL

/* 2. IF YOU NEED PASSIVE_LEVEL FROM A HIGHER ONE, queue a
      WORK ITEM (chapter 10). You cannot simply lower IRQL to
      escape - lowering below where you were called is a
      serious bug.

/* 3. MARK CODE PAGEABLE ONLY IF IT RUNS AT PASSIVE_LEVEL: */

        #pragma alloc_text(INIT,  DriverEntry)
        #pragma alloc_text(PAGE,  MyDeviceAdd)
        #pragma alloc_text(PAGE,  MyPrepareHardware)

        NTSTATUS MyDeviceAdd(...)
        {
                PAGED_CODE();     /* ASSERTS PASSIVE_LEVEL -
                                     put this at the top of
                                     every PAGE function     */
                ...
        }

   INIT   discarded after boot (like Linux's __init)
   PAGE   pageable
   .text  always resident - the default

/* PAGED_CODE() costs nothing in a release build and catches
   the mistake immediately in a checked one. Use it in every
   pageable function without exception.

/* 4. NON-PAGED DATA for anything touched above PASSIVE: */
        ExAllocatePool2(POOL_FLAG_NON_PAGED, size, 'GATd');
   and never a global marked with #pragma data_seg("PAGE").

/* 5. THE ONE-LINE HABIT that prevents most of this: before
      writing any line that touches memory or calls an API,
      ask "what IRQL am I at?" If you cannot answer instantly,
      look at the callback that got you here.             */
```

IRQL and spinlocks

```
/* A spinlock protects data; raising IRQL protects against
   PREEMPTION. Windows combines them because you almost always
   need both.                                                  */

/* Data shared between PASSIVE code and a DPC: */
        KeAcquireSpinLock(&d->Lock, &oldIrql);
        /* now at DISPATCH_LEVEL: the DPC cannot run on this
           CPU, and the spinlock excludes other CPUs        */
        ...
        KeReleaseSpinLock(&d->Lock, oldIrql);

/* Data shared with an ISR - the spinlock must be acquired at
   the DEVICE's IRQL, or the ISR can interrupt you while you
   hold it and deadlock (exactly the Linux spinlock
   self-deadlock, chapter 11):                              */
        KeAcquireInterruptSpinLock(interrupt, &oldIrql);
        ...
        KeReleaseInterruptSpinLock(interrupt, oldIrql);

/* In WDF this is: */
        WdfInterruptAcquireLock(interrupt);
        ...
        WdfInterruptReleaseLock(interrupt);

/* THE RULES:
     · a spinlock section runs at DISPATCH_LEVEL or higher, so
       everything forbidden at DISPATCH_LEVEL is forbidden
       inside it - no paged memory, no blocking, no
       paged-pool allocation
     · keep it SHORT. Other CPUs spin.
     · never acquire a lower-IRQL lock while holding a
       higher-IRQL one
     · the `oldIrql` variable must be LOCAL. Sharing one
       between CPUs corrupts the IRQL restore.            */
```

How it shows up

Symptom	Cause
Bugcheck 0xA or 0xD1, parameter 2 >= 2	Paged memory touched at DISPATCH_LEVEL or above
Works for days, then crashes under memory pressure	The same bug, latent — Verifier's IRQL checking exposes it
Bugcheck 0x133 DPC_WATCHDOG	A DPC doing far too much work
Hang with one CPU spinning	Spinlock acquired at the wrong IRQL, then interrupted by the ISR
Random corruption after a spinlock release	`oldIrql` shared instead of local
Verifier reports an IRQL violation on a working driver	A latent bug it made deterministic
Crash only on a multi-processor machine	Missing spinlock; IRQL alone does not exclude other CPUs
`ExAllocatePool2` fails or crashes in a DPC	Requested paged pool above PASSIVE_LEVEL
Code Analysis warns about IRQL	A real bug found at build time — fix it, do not suppress
Crash in a pageable function	Called above PASSIVE_LEVEL — `PAGED_CODE()` would have asserted

Check yourself

- ✓ List the IRQL levels in order and say what runs at each.
- ✓ State the masking rule and why DISPATCH_LEVEL is the critical boundary.
- ✓ Map IRQL onto Linux's execution contexts.
- ✓ Explain the paged-memory rule and why the resulting bug is latent.
- ✓ Raise and lower IRQL correctly, and say why you rarely should.
- ✓ Annotate a function with SAL and say what Code Analysis will then check.
- ✓ Decode bugcheck 0xD1 from its four parameters.
- ✓ State the IRQL of eight common framework callbacks.
- ✓ Use `#pragma alloc_text` and `PAGED_CODE()` correctly.
- ✓ Explain why data shared with an ISR needs an interrupt spinlock.

The object manager and handles

Windows unifies processes, files, devices, events and the registry into one object system with uniform naming, reference counting and security. Understanding it explains handle leaks, access checks, and a whole class of privilege-escalation bugs.

Anatomy of an object

```
 +-----------------------------------+
 |  OBJECT HEADER                    |
 |    PointerCount   kernel refs     |  <-- ObReference/
 |    HandleCount    open handles    |      ObDereference
 |    Type           -> OBJECT_TYPE  |
 |    SecurityDescriptor             |
 |    Name / Directory  (optional)   |
 |    Flags: permanent, exclusive... |
 +-----------------------------------+
 |  OBJECT BODY                      |
 |    type-specific data:            |
 |      FILE_OBJECT, DEVICE_OBJECT,  |
 |      EPROCESS, ETHREAD, KEVENT... |
 +-----------------------------------+

A POINTER to an object points at the BODY. The header is at
a negative offset, which is why you never touch it directly.

TWO INDEPENDENT COUNTS:
   HandleCount    how many open handles exist
   PointerCount   how many kernel references exist
                  (includes one per handle)

The object is destroyed when BOTH reach zero. That is what
makes it safe for a driver to keep using an object after
userspace has closed its handle - as long as it took a
reference.

THE TYPE OBJECT holds the procedures for that type:
   OpenProcedure, CloseProcedure, DeleteProcedure,
   ParseProcedure, SecurityProcedure, QueryNameProcedure

The ParseProcedure is how \Device\HarddiskVolume1\foo\bar
works: the object manager resolves as far as the device
object, then hands the REST OF THE PATH to the file system,
which parses it. That single mechanism is why the object
namespace and file system namespaces compose.
```

The namespace

```
\                          root
\Device\                   device objects created by drivers
    \Device\HarddiskVolume1
    \Device\MyDevice           <-- yours
\GLOBAL??\  (was \DosDevices\)  symbolic links
    \GLOBAL??\C:      -> \Device\HarddiskVolume3
    \GLOBAL??\MyDev   -> \Device\MyDevice
\Driver\                   loaded driver objects
\FileSystem\
\BaseNamedObjects\         named events, mutexes, sections
                           created by user mode
\Sessions\<n>\             per-session namespace
\KernelObjects\
\REGISTRY\MACHINE\...      the registry, as objects
\ObjectTypes\              the type objects themselves

/* WHY \GLOBAL?? MATTERS TO YOU: user mode cannot open
   \Device\MyDevice directly with CreateFile - it resolves
   names relative to \GLOBAL?? (and the per-session \??).
   So a driver that wants to be reachable creates a symbolic
   link:                                                  */

        DECLARE_CONST_UNICODE_STRING(devName,
                                     L"\\Device\\MyDevice");
        DECLARE_CONST_UNICODE_STRING(symLink,
                                     L"\\DosDevices\\MyDevice");

        status = IoCreateSymbolicLink(&symLink, &devName);
        ...
        IoDeleteSymbolicLink(&symLink);    /* in unload! */

/* then user mode opens: */
        h = CreateFileW(L"\\\\.\\MyDevice", ...);
              /* \\.\ maps to \GLOBAL?? */

/* BROWSE IT with Sysinternals WinObj. Do this once - seeing
   the real namespace makes the model concrete in a way no
   description does.                                      */
```

Handles

```
/* A handle is a per-process index into a handle table, not a
   pointer. Each entry holds:
        a pointer to the object
        a GRANTED ACCESS MASK
        flags: inheritable, protect-from-close, audit        */

/* THE ACCESS MASK is fixed at OPEN time. The object manager
   checks the requested access against the object's security
   descriptor once, then records what was granted. Every later
   operation is checked against the RECORDED mask - which is
   why a handle opened for read cannot later be used to
   write, even if the security descriptor changes.        */

        GENERIC_READ / GENERIC_WRITE / GENERIC_ALL
        FILE_READ_DATA / FILE_WRITE_DATA
        SYNCHRONIZE
        DELETE / READ_CONTROL / WRITE_DAC / WRITE_OWNER

/* CONVERTING A HANDLE TO AN OBJECT, in a driver: */
        PFILE_OBJECT fileObject;
        NTSTATUS status;

        status = ObReferenceObjectByHandle(
                        handle,
                        FILE_READ_DATA,     /* required access */
                        *IoFileObjectType,  /* expected TYPE   */
                        UserMode,           /* <-- CRITICAL    */
                        (PVOID *)&fileObject,
                        NULL);
        if (!NT_SUCCESS(status))
                return status;

        /* ... use fileObject ... */

        ObDereferenceObject(fileObject);    /* ALWAYS */
```

THE `ACCESSMODE` PARAMETER IS A SECURITY CONTROL, NOT A FORMALITY

`UserMode` tells the object manager to validate the handle as if a user-mode caller presented it: check that it is a user handle, that the type matches, and that the granted access covers what you asked for. `KernelMode` skips all of that.

Passing `KernelMode` while handling a request that *came from user mode* lets an unprivileged caller hand you a kernel handle value, or a handle without the access you require, and have your driver operate on it with full privilege. This is a textbook elevation-of-privilege bug and it has shipped in real drivers repeatedly. When servicing a user request, pass `Irp->RequestorMode` or `UserMode` — never a hard-coded `KernelMode`.

Reference counting

```
/* THE RULE: every ObReferenceObject* must be matched by
   exactly one ObDereferenceObject, on EVERY path including
   errors.                                                  */

        ObReferenceObject(object);              /* +1 */
        ObDereferenceObject(object);            /* -1 */

        ObReferenceObjectByPointer(object,
                                   DesiredAccess,
                                   ObjectType,
                                   AccessMode);

/* WHY A DRIVER TAKES A REFERENCE: you want to keep using an
   object after the operation that gave it to you returns -
   for example, storing a FILE_OBJECT to complete a request
   later. Without a reference, the object can be freed the
   moment the handle is closed, and you have a use-after-free
   in a place you cannot reproduce.                         */

/* THE FAILURE MODES:
     · leaked reference -> the object never dies. A device that
       cannot be removed, a process stuck in "Terminated"
       state, a driver that cannot be unloaded. Diagnosed with
       !object and !handle in WinDbg (chapter 22).
     · extra dereference -> premature free, then a
       use-after-free crash somewhere unrelated. Verifier's
       special pool catches it.

/* HANDLES YOU OPEN must be closed: */
        ZwClose(handle);

/* AND ALWAYS ASK: does this function return a referenced
   object? The documentation says so explicitly, and the
   answer determines whether you owe a dereference. Examples:

     IoGetCurrentProcess()            NO reference
     PsGetCurrentProcess()            NO reference
     ObReferenceObjectByHandle()      YES - dereference it
     IoGetDeviceObjectPointer()       YES - two things to
                                      release, actually      */
```

Creating a device object

```
/* The classic WDM way. WDF wraps this (chapter 14), but you
   should see it once because it explains what WDF is doing. */

NTSTATUS
DriverEntry(_In_ PDRIVER_OBJECT DriverObject,
            _In_ PUNICODE_STRING RegistryPath)
{
        UNICODE_STRING devName, symLink;
        PDEVICE_OBJECT deviceObject;
        NTSTATUS status;

        RtlInitUnicodeString(&devName, L"\\Device\\MyDevice");

        status = IoCreateDevice(
                        DriverObject,
                        sizeof(MY_DEVICE_EXTENSION),  /* your
                                                         state */
                        &devName,
                        FILE_DEVICE_UNKNOWN,
                        FILE_DEVICE_SECURE_OPEN,      /* apply
                                                         ACLs to
                                                         opens  */
                        FALSE,          /* not exclusive */
                        &deviceObject);
        if (!NT_SUCCESS(status))
                return status;

        /* the extension is YOUR per-device data - the Windows
           equivalent of drvdata (Linux ch. 17)             */
        PMY_DEVICE_EXTENSION ext = deviceObject->DeviceExtension;
        RtlZeroMemory(ext, sizeof(*ext));
        KeInitializeSpinLock(&ext->Lock);

        RtlInitUnicodeString(&symLink, L"\\DosDevices\\MyDevice");
        status = IoCreateSymbolicLink(&symLink, &devName);
        if (!NT_SUCCESS(status)) {
                IoDeleteDevice(deviceObject);
                return status;
        }

        DriverObject->MajorFunction[IRP_MJ_CREATE] = MyCreate;
        DriverObject->MajorFunction[IRP_MJ_CLOSE]  = MyClose;
        DriverObject->MajorFunction[IRP_MJ_READ]   = MyRead;
        DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] =
                                                  MyIoctl;
        DriverObject->DriverUnload = MyUnload;

        /* THE LAST STEP - until this clears, the device
           reports itself as still initialising and I/O is
           refused: */
        deviceObject->Flags &= ~DO_DEVICE_INITIALIZING;

        return STATUS_SUCCESS;
}

/* FILE_DEVICE_SECURE_OPEN IS IMPORTANT. Without it, a caller
   can open \Device\MyDevice\anything and BYPASS the device's
   ACL, because the security check is only applied to the
```

```
   exact device name. Set it unless you have a namespace of
   your own to parse.                                        */
```

Securing a device

```
/* By default, a device object created by a driver may be
   openable by any user. Decide deliberately who may open it. */

/* 1. AN SDDL STRING in the INF (chapter 19) - the normal way: */
        [MyDevice_AddReg]
        HKR,,DeviceCharacteristics,0x10001,0x0100  ; SECURE_OPEN
        HKR,,Security,,"D:P(A;;GA;;;SY)(A;;GA;;;BA)"

/*    D:P                     a protected DACL
      (A;;GA;;;SY)            Allow, Generic All, SYSTEM
      (A;;GA;;;BA)            Allow, Generic All, Builtin Admins
   Add (A;;GRGW;;;WD) to allow read+write for Everyone - and
   think hard before you do.

/* 2. IoCreateDeviceSecure, in code: */
        DECLARE_CONST_UNICODE_STRING(sddl,
                L"D:P(A;;GA;;;SY)(A;;GA;;;BA)(A;;GRGW;;;WD)");

        status = IoCreateDeviceSecure(DriverObject,
                        sizeof(MY_DEVICE_EXTENSION),
                        &devName, FILE_DEVICE_UNKNOWN,
                        FILE_DEVICE_SECURE_OPEN, FALSE,
                        &sddl, NULL, &deviceObject);

/* 3. IN WDF (chapter 14): */
        WdfDeviceInitAssignSDDLString(deviceInit, &sddl);

/* THE COMMON SDDL ACRONYMS:
        SY  Local System        BA  Builtin Administrators
        WD  Everyone            AU  Authenticated Users
        IU  Interactive Users   RC  Restricted Code
        GA  Generic All         GR  Generic Read
        GW  Generic Write       GX  Generic Execute

/* AND A CHECK INSIDE AN OPERATION, when a specific action
   needs more privilege than opening does: */
        if (!SeSinglePrivilegeCheck(SeLoadDriverPrivilege,
                                    Irp->RequestorMode))
                return STATUS_PRIVILEGE_NOT_HELD;

/* THE DESIGN QUESTION to answer explicitly: if any user can
   open this device, then every IOCTL you expose is reachable
   by every user, including from a sandboxed process. Size the
   ACL to the least privilege your device genuinely needs -
   this is the Windows form of the least-privilege argument
   from isolation chapter 19.                                */
```

Inspecting objects

```
# --- Tools on the running machine ---
    WinObj              (Sysinternals) browse the namespace
    Process Explorer    per-process handle list, lower pane
    handle.exe          command line: who has this file open?
        handle.exe MyDevice

# --- In WinDbg (chapter 22) ---
    !object \Device                 list a directory
    !object \Device\MyDevice        one object's header
    !object ffff8a0c12345678

      Object: ffff8a0c12345678  Type: (ffff8a0c00112233) Device
        ObjectHeader: ffff8a0c12345650
        HandleCount: 2  PointerCount: 5
        Directory Object: ffffc00012340000  Name: MyDevice

    !handle                         all handles, this process
    !handle 0x1c4 f                 full detail on one
    !devobj ffff8a0c12345678        device object detail
    !drvobj \Driver\MyDriver 7      the driver and its
                                    dispatch table
    !object \ObjectTypes            every type, with counts

# --- FINDING A HANDLE LEAK ---
    !htrace -enable                 (before reproducing)
    ... run the workload ...
    !htrace -diff
# Shows every handle opened since the enable, with the stack
# that opened it. This is the tool for "handles keep climbing".

# Or from user mode:
    Process Explorer -> the process -> lower pane -> Handles
    and watch the count climb.
```

How it shows up

Symptom	Cause
`CreateFile` fails with ERROR_FILE_NOT_FOUND	No symbolic link, or the wrong name — check WinObj
`CreateFile` fails with ACCESS_DENIED	The device's ACL excludes the caller
Any user can open a privileged device	No SDDL set — the default is permissive
ACL bypassed by opening a sub-path	`FILE_DEVICE_SECURE_OPEN` not set
Unprivileged caller operates on a kernel object	`KernelMode` passed to `ObReferenceObjectByHandle` — a real EoP bug
Handle count climbs forever	Missing `ZwClose` — use `!htrace`
Driver cannot be unloaded	Leaked object reference — `!object` shows PointerCount
Use-after-free on a FILE_OBJECT	Kept a pointer without taking a reference
Crash on the extra dereference	Unbalanced `ObDereferenceObject`
I/O refused right after `IoCreateDevice`	`DO_DEVICE_INITIALIZING` never cleared
Symbolic link left behind after unload	No `IoDeleteSymbolicLink` in the unload path

Check yourself

- ✓ Describe an object's header, body and type, and the two reference counts.
- ✓ Explain how a parse procedure lets file paths extend the object namespace.
- ✓ Explain why a driver needs a symbolic link for user mode to reach it.
- ✓ Explain what a handle is and when its access mask is checked.
- ✓ State exactly what `UserMode` versus `KernelMode` changes, and the bug the wrong choice creates.
- ✓ Explain when a driver must take a reference, and both failure modes of getting it wrong.
- ✓ Create a device object with a device extension and a symbolic link, with correct cleanup.
- ✓ Explain what `FILE_DEVICE_SECURE_OPEN` prevents.
- ✓ Write an SDDL string granting access to SYSTEM and Administrators only.
- ✓ Find a handle leak with `!htrace`.

The I/O manager and IRPs

Every I/O operation in Windows is an I/O Request Packet travelling down a stack of drivers. One data structure carries reads, writes, IOCTLs, power transitions and PnP events alike — and understanding its lifecycle is understanding Windows I/O.

What an IRP is

```
 +-----------------------------------------+
 |  IRP HEADER                             |
 |    MdlAddress        for DIRECT I/O     |
 |    AssociatedIrp.SystemBuffer  BUFFERED |
 |    UserBuffer        METHOD_NEITHER     |
 |    IoStatus.Status   the result         |
 |    IoStatus.Information  bytes transferred|
 |    RequestorMode     UserMode / KernelMode|
 |    Cancel, CancelRoutine                |
 |    CurrentLocation                      |
 +-----------------------------------------+
 |  IO_STACK_LOCATION  [n-1]  bottom driver  |
 |  IO_STACK_LOCATION  [n-2]               |
 |  ...                                    |
 |  IO_STACK_LOCATION  [0]    top driver     |
 +-----------------------------------------+

ONE STACK LOCATION PER DRIVER in the device stack. Each
holds THAT driver's view of the request:

   MajorFunction     IRP_MJ_READ, IRP_MJ_DEVICE_CONTROL, ...
   MinorFunction     for PnP and power
   Parameters        a union: Read, Write, DeviceIoControl,
                     QueryDeviceRelations, Power, ...
   DeviceObject      which device
   FileObject        which open instance
   CompletionRoutine to run when a lower driver finishes

YOU GET YOURS WITH:
      PIO_STACK_LOCATION sp = IoGetCurrentIrpStackLocation(irp);

AND THE ONE BELOW YOU WITH:
      IoGetNextIrpStackLocation(irp)

THE DESIGN IDEA: the IRP is a self-contained request that
can be queued, cancelled, passed between drivers, and
completed asynchronously from any context. Linux has no
single equivalent - it has bio for block, urb for USB, skb
for network, and ad-hoc structures elsewhere. Windows pays
more machinery for one uniform mechanism.
```

The major function codes

Code	From	Notes
`IRP_MJ_CREATE`	`CreateFile`	An open. Your chance to refuse.
`IRP_MJ_CLOSE`	Last handle closed	Tear down per-open state

Code	From	Notes
`IRP_MJ_CLEANUP`	Last handle in a process closed	**Cancel pending I/O here**
`IRP_MJ_READ` / `_WRITE`	`ReadFile` / `WriteFile`	The data path
`IRP_MJ_DEVICE_CONTROL`	`DeviceIoControl`	IOCTLs (ch. 16)
`IRP_MJ_INTERNAL_DEVICE_CONTROL`	Kernel callers only	Driver-to-driver
`IRP_MJ_PNP`	The PnP manager	Start, stop, remove, query (ch. 05)
`IRP_MJ_POWER`	The power manager	D-states, S-states
`IRP_MJ_SYSTEM_CONTROL`	WMI	Instrumentation
`IRP_MJ_FLUSH_BUFFERS`	`FlushFileBuffers`	
`IRP_MJ_QUERY_INFORMATION`	File size etc.	

> **`CLEANUP` VERSUS `CLOSE` — THE DISTINCTION THAT MATTERS**
>
> `IRP_MJ_CLEANUP` arrives when the *last handle in a process* is closed, while the file object still exists. `IRP_MJ_CLOSE` arrives when the *last reference* to the file object goes away, which may be much later. Cancel and complete pending requests in **CLEANUP**: if you wait for CLOSE, a process that exits with I/O outstanding hangs in termination, because the file object cannot be released while your driver still holds its IRPs.

A dispatch routine

```
NTSTATUS
MyRead(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_DEVICE_EXTENSION ext = DeviceObject->DeviceExtension;
        PIO_STACK_LOCATION sp = IoGetCurrentIrpStackLocation(Irp);
        ULONG length = sp->Parameters.Read.Length;
        PVOID buffer;
        NTSTATUS status;
        ULONG_PTR info = 0;

        /* For METHOD_BUFFERED the I/O manager already copied
           the user buffer somewhere safe (chapter 07):     */
        buffer = Irp->AssociatedIrp.SystemBuffer;

        if (length < sizeof(ULONG)) {
                status = STATUS_BUFFER_TOO_SMALL;
                goto complete;
        }

        status = MyReadData(ext, buffer, length, &info);

complete:
        Irp->IoStatus.Status = status;
        Irp->IoStatus.Information = info;   /* BYTES
                                               TRANSFERRED  */
        IoCompleteRequest(Irp, IO_NO_INCREMENT);
        return status;
}

/* THREE RULES, and each has a specific consequence:

   1. SET BOTH IoStatus FIELDS. `Information` is what
      ReadFile reports as bytes read. Leaving it uninitialised
      makes userspace read uninitialised stack contents as a
      length - and act on it.

   2. THE RETURN VALUE MUST MATCH IoStatus.Status, except that
      you return STATUS_PENDING when you did not complete it.

   3. ONCE YOU CALL IoCompleteRequest, THE IRP IS GONE.
      Touching it afterwards is a use-after-free. Not "may
      be" - the I/O manager may have freed it before the call
      returns.                                              */

/* IO_NO_INCREMENT vs a priority boost: the second argument
   boosts the waiting thread's priority on completion. Use
   IO_NO_INCREMENT unless the request took long enough that
   the thread deserves to run promptly (IO_DISK_INCREMENT and
   friends). Getting it wrong is a performance issue, not a
   correctness one.                                         */
```

Asynchronous completion

```
/* The interesting case: you cannot complete now, so you queue
   the IRP and return.                                          */

NTSTATUS
MyRead(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_DEVICE_EXTENSION ext = DeviceObject->DeviceExtension;

        /* 1. MARK IT PENDING BEFORE QUEUEING. If you queue it
              first, another thread can complete it before you
              mark it - and marking a completed IRP is a
              use-after-free.                                   */
        IoMarkIrpPending(Irp);

        /* 2. set a cancel routine, so the request can be
              abandoned (below)                                 */
        IoSetCancelRoutine(Irp, MyCancelRoutine);

        /* 3. queue it */
        ExInterlockedInsertTailList(&ext->PendingIrps,
                                    &Irp->Tail.Overlay.ListEntry,
                                    &ext->QueueLock);

        /* 4. return PENDING. The IRP now belongs to your
              queue; do NOT touch it after this point without
              the queue lock.                                   */
        return STATUS_PENDING;
}

/* LATER - in the DPC when the hardware finishes: */
        PIRP irp = DequeueNextIrp(ext);

        if (irp) {
                irp->IoStatus.Status = STATUS_SUCCESS;
                irp->IoStatus.Information = bytesRead;
                IoCompleteRequest(irp, IO_NO_INCREMENT);
        }

/* AND THE USER SEES: ReadFile returns FALSE with
   GetLastError() == ERROR_IO_PENDING if the handle was opened
   with FILE_FLAG_OVERLAPPED, and the OVERLAPPED event is
   signalled at completion. Without FILE_FLAG_OVERLAPPED, the
   I/O manager blocks the thread for them and ReadFile appears
   synchronous.

   THIS IS ALL THE MACHINERY WDF QUEUES REMOVE (chapter 15).
   You should understand it once; you should not write it. */
```

Cancellation

```
/* A queued IRP can be cancelled at any moment - the thread
   called CancelIo, the process is exiting, the handle closed.
   Getting this right in raw WDM is genuinely hard.         */

VOID
MyCancelRoutine(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_DEVICE_EXTENSION ext = DeviceObject->DeviceExtension;
        KIRQL oldIrql;

        /* The I/O manager called us holding the GLOBAL cancel
           spinlock, at DISPATCH_LEVEL. RELEASE IT FIRST -
           holding it while taking our own lock is a lock
           ordering violation and a system-wide bottleneck. */
        IoReleaseCancelSpinLock(Irp->CancelIrql);

        KeAcquireSpinLock(&ext->QueueLock, &oldIrql);
        RemoveEntryList(&Irp->Tail.Overlay.ListEntry);
        KeReleaseSpinLock(&ext->QueueLock, oldIrql);

        Irp->IoStatus.Status = STATUS_CANCELLED;
        Irp->IoStatus.Information = 0;
        IoCompleteRequest(Irp, IO_NO_INCREMENT);
}

/* AND THE RACE, which is the whole difficulty:

     the cancel routine can fire at ANY time after
     IoSetCancelRoutine, including WHILE you are dequeuing the
     same IRP to complete it normally.

   THE PROTOCOL: whoever successfully clears the cancel
   routine owns the IRP.

        /* dequeuing normally: */
        KeAcquireSpinLock(&ext->QueueLock, &oldIrql);
        irp = DequeueOne(ext);
        if (irp && IoSetCancelRoutine(irp, NULL) == NULL) {
                /* the cancel routine ALREADY fired and owns
                   this IRP - do not touch it */
                irp = NULL;
        }
        KeReleaseSpinLock(&ext->QueueLock, oldIrql);

   Every raw-WDM driver must get this exactly right, on every
   queue, and most of the historical cancellation bugs in
   Windows drivers are here.                                */
```

> **THIS IS THE STRONGEST ARGUMENT FOR WDF**
>
> A WDF queue (chapter 15) handles pending, cancellation, the cancel-routine race, power-state stalling, and per-file-object cleanup — correctly, once, for everyone. The code above exists in this chapter so you can read and debug legacy drivers, and so that you understand what the framework is doing on your behalf. **Write new drivers in KMDF.**

Passing an IRP down

```
/* A filter driver, or any driver with something below it. */

/* --- pass it down and forget about it -------------------- */
NTSTATUS
MyPassThrough(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_DEVICE_EXTENSION ext = DeviceObject->DeviceExtension;

        IoSkipCurrentIrpStackLocation(Irp);   /* reuse OUR slot -
                                                 we want no
                                                 completion
                                                 callback      */
        return IoCallDriver(ext->LowerDevice, Irp);
}

/* --- pass it down and see the result --------------------- */
NTSTATUS
MyFilter(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_DEVICE_EXTENSION ext = DeviceObject->DeviceExtension;

        IoCopyCurrentIrpStackLocationToNext(Irp);
        IoSetCompletionRoutine(Irp, MyCompletion, ext,
                               TRUE,    /* on success */
                               TRUE,    /* on error   */
                               TRUE);   /* on cancel  */
        return IoCallDriver(ext->LowerDevice, Irp);
}

NTSTATUS
MyCompletion(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp,
             _In_ PVOID Context)
{
        /* Runs at <= DISPATCH_LEVEL, in an ARBITRARY thread.
           Do not assume you are in the requesting process. */

        if (Irp->PendingReturned)
                IoMarkIrpPending(Irp);   /* propagate it up -
                                            REQUIRED         */

        /* inspect or modify Irp->IoStatus here */

        return STATUS_CONTINUE_COMPLETION;
        /* or STATUS_MORE_PROCESSING_REQUIRED to STOP the
           completion and take ownership of the IRP - then you
           must complete it yourself later.               */
}

/* THE SKIP vs COPY DISTINCTION:
     IoSkipCurrentIrpStackLocation   reuses your slot for the
                                     next driver. Cheaper. No
                                     completion routine
                                     possible.
     IoCopyCurrentIrpStackLocationToNext  gives the next
                                     driver its own copy, and
                                     leaves yours free for the
                                     completion routine.

   Using Skip and then setting a completion routine corrupts
```

```
   the stack - a classic filter-driver bug that bugchecks
   randomly.                                                  */
```

The full lifecycle

```
   ReadFile()
       |
   I/O manager: IoAllocateIrp / IoBuildSynchronousFsdRequest
       |  fills IoStatus, RequestorMode, the buffer fields
       |  sets up stack location [0] for the top driver
       v
   IoCallDriver(top)
       |
   +---v-------------------+
   | top driver            |  IoSkip/IoCopy, IoCallDriver
   +---+-------------------+
       |
   +---v-------------------+
   | function driver (you) |  complete now, or
   |                       |  IoMarkIrpPending + queue
   +---+-------------------+       |
       |                           | later, from a DPC
       v                           v
   IoCompleteRequest(irp, boost)
       |
       |  walk UP the stack, running each completion routine
       |  bottom-first
       v
   the I/O manager:
       · copies SystemBuffer back to the user buffer
         (METHOD_BUFFERED)
       · unlocks and frees the MDL (DIRECT)
       · sets the OVERLAPPED / event
       · wakes the waiting thread
       · FREES THE IRP
       v
   ReadFile returns

  THE INVARIANT: an IRP is completed EXACTLY ONCE, by exactly
  one driver. Completing twice is a bugcheck; never completing
  it hangs the caller forever.
```

Debugging IRPs

```
# In WinDbg (chapter 22):

    !irp ffff8a0c12345678
      Irp is active with 3 stacks, 2 is current
       cmd flg cl Device  File   Completion-Context
       [IRP_MJ_READ(3), N/A(0)]
         0  0  ffff8a0c...  ffff8a0c...   00000000-00000000
                \Driver\MyDriver
                Args: 00001000 00000000 00000000 00000000

    !irpfind                    every IRP in the system
    !irpfind -v 0 0 device ffff8a0c12345678
                                IRPs for one device
    !devstack ffff8a0c12345678  the whole device stack
    !devobj ffff8a0c12345678    one device object

# WHY !irpfind IS THE TOOL FOR A HANG: a thread stuck in
# ReadFile has an IRP that nobody completed. Find it, see
# which driver holds the current stack location, and you have
# named the culprit.

# LIVE TRACING: the I/O verifier option in Driver Verifier
# (chapter 21) checks IRP handling rules aggressively - double
# completion, touching a completed IRP, wrong IRQL at
# completion, missing PendingReturned propagation. Turn it on;
# these are exactly the bugs that are otherwise
# unreproducible.
```

How it shows up

Symptom	Cause
Bugcheck 0x44 MULTIPLE_IRP_COMPLETE_REQUESTS	Completed the same IRP twice
Application hangs in `ReadFile` forever	IRP queued and never completed — find it with `!irpfind`
Process will not terminate	Pending I/O not cancelled in `IRP_MJ_CLEANUP`
Crash after completing a request	Touched the IRP after `IoCompleteRequest`
`ReadFile` reports a wrong byte count	`IoStatus.Information` not set
Userspace acts on garbage length	Same — uninitialised `Information`
Random bugcheck in a filter driver	`IoSkipCurrentIrpStackLocation` plus a completion routine
Asynchronous request never signals	`PendingReturned` not propagated in the completion routine
Deadlock during cancellation	Cancel spinlock not released before taking your own
Cancelled IRP completed twice	Lost the cancel-routine race — the `IoSetCancelRoutine(NULL)` protocol
Verifier reports an IRP rule violation	A real bug it made deterministic

Check yourself

- ✓ Describe an IRP's header and stack locations, and how you get yours.
- ✓ Name eight major function codes and what triggers each.

- ✓ Explain the difference between CLEANUP and CLOSE, and which cancels pending I/O.
- ✓ Write a synchronous dispatch routine and state the three completion rules.
- ✓ Explain why `IoMarkIrpPending` must precede queueing.
- ✓ Write a cancel routine and explain the ownership protocol.
- ✓ Distinguish Skip from Copy and say what breaks if you mix them up.
- ✓ Explain why a completion routine must propagate `PendingReturned`.
- ✓ Trace an IRP's full lifecycle and state the completed-exactly-once invariant.
- ✓ Use `!irpfind` to diagnose a hung application.

The device stack and Plug and Play

Windows layers several drivers over one piece of hardware, and the PnP manager orchestrates their lifecycle. Understanding the stack explains filter drivers, why devices appear with a yellow bang, and what WDF is doing for you when it "just works".

One device, several drivers

```
For a single physical device:

+-----------------------------------+
|  UPPER FILTER   (optional, several)|  encryption, tracing,
|     e.g. a monitoring filter       |  policy
+-----------------------------------+
|  FUNCTION DRIVER (FDO)             |  <-- YOURS. Knows
|     the driver that knows the      |      how the hardware
|     hardware                       |      works.
+-----------------------------------+
|  LOWER FILTER   (optional, several)|
+-----------------------------------+
|  BUS DRIVER (PDO)                  |  pci.sys, acpi.sys,
|     enumerated this device,        |  usbhub.sys
|     owns its resources             |
+-----------------------------------+

NAMES YOU MUST KNOW:

  PDO   Physical Device Object. Created by the BUS driver
        to say "a device exists here". One per child.
  FDO   Functional Device Object. Created by YOUR driver
        and attached above the PDO.
  FiDO  Filter Device Object.

An IRP enters at the TOP and travels down (chapter 04).
Each layer may handle it, modify it, or pass it on.

WHY THIS EXISTS: it makes filtering universal. A disk
encryption filter, a keyboard logger, a USB analyser and a
power policy owner all attach the same way, to any device,
without the function driver knowing. Linux has no direct
equivalent - filtering there is per-subsystem or done with
stacked block devices.

SEE A REAL ONE:
     WinDbg:  !devstack ffff8a0c12345678
     Device Manager -> device -> Details -> "Upper filters"
```

The PnP state machine

```
BUS DRIVER enumerates hardware
     |  reports a new child in
     |  IRP_MN_QUERY_DEVICE_RELATIONS (BusRelations)
     v
PnP MANAGER creates a PDO, reads its HARDWARE IDs
     |     PCI\VEN_1234&DEV_5678&SUBSYS_...&REV_01
     |     ACPI\VEN0001
     v
matches against installed INFs (chapter 19)
     |
     v
loads the driver, calls DriverEntry (once per driver)
     |
     v
calls EvtDriverDeviceAdd / AddDevice (once PER DEVICE)
     |     you create the FDO and attach it to the stack
     v
IRP_MN_QUERY_RESOURCE_REQUIREMENTS
     |     "what does this device need?"
     v
PnP manager ASSIGNS resources (memory ranges, interrupts,
DMA channels) - arbitrating between all devices
     |
     v
IRP_MN_START_DEVICE                <-- the big one
     |     WDF: EvtDevicePrepareHardware  (map registers)
     |          EvtDeviceD0Entry          (power it up)
     |          interrupts connected
     v
===== DEVICE IS RUNNING; I/O flows =====
     |
     v
IRP_MN_QUERY_REMOVE_DEVICE     "may I remove it?"
     |     you may VETO this if you are busy
     v
IRP_MN_REMOVE_DEVICE
     |     WDF: EvtDeviceD0Exit
     |          EvtDeviceReleaseHardware
     |          EvtDeviceSelfManagedIoCleanup
     v
device object deleted

OTHER TRANSITIONS you must handle:
  IRP_MN_CANCEL_REMOVE_DEVICE   the remove was vetoed
  IRP_MN_QUERY_STOP / STOP      resource rebalance
  IRP_MN_SURPRISE_REMOVAL       the user yanked it out
  IRP_MN_QUERY_CAPABILITIES
  IRP_MN_QUERY_ID / QUERY_DEVICE_TEXT
```

> **SURPRISE REMOVAL IS NOT AN EDGE CASE**
> For anything on USB, PCIe hot-plug, or a removable bus, `SURPRISE_REMOVAL` is the *normal* path — the hardware is already gone when you are told. Every register read now returns `0xFFFFFFFF`, every pending request must be completed with an error, and any code that waits for the hardware to respond will hang forever. Write the driver so that "the device vanished mid-operation" is a handled case from the first day, not something retrofitted after a bug report.

Doing it in raw WDM

```
/* Shown so you can read legacy drivers and understand what
   WDF replaces. Do not write new code this way.           */

NTSTATUS
MyAddDevice(_In_ PDRIVER_OBJECT DriverObject,
            _In_ PDEVICE_OBJECT PhysicalDeviceObject)
{
        PDEVICE_OBJECT fdo;
        PMY_EXT ext;
        NTSTATUS status;

        status = IoCreateDevice(DriverObject, sizeof(MY_EXT),
                                NULL,      /* unnamed - PnP
                                              devices use
                                              interfaces     */
                                FILE_DEVICE_UNKNOWN,
                                FILE_DEVICE_SECURE_OPEN,
                                FALSE, &fdo);
        if (!NT_SUCCESS(status))
                return status;

        ext = fdo->DeviceExtension;
        RtlZeroMemory(ext, sizeof(*ext));
        ext->Self = fdo;
        ext->Pdo  = PhysicalDeviceObject;

        /* ATTACH to the stack - this returns the device
           DIRECTLY BELOW us, which may be a lower filter, not
           the PDO.                                          */
        ext->LowerDevice = IoAttachDeviceToDeviceStack(
                                fdo, PhysicalDeviceObject);
        if (!ext->LowerDevice) {
                IoDeleteDevice(fdo);
                return STATUS_DEVICE_REMOVED;
        }

        /* inherit the buffering method and power flags from
           below - getting this wrong breaks I/O in ways that
           are hard to trace (chapter 07)                    */
        fdo->Flags |= ext->LowerDevice->Flags &
                      (DO_BUFFERED_IO | DO_DIRECT_IO |
                       DO_POWER_PAGABLE);

        fdo->Flags &= ~DO_DEVICE_INITIALIZING;   /* LAST */

        return STATUS_SUCCESS;
}

/* AND THE PNP DISPATCH, which is where the real work is: */
NTSTATUS
MyPnp(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PMY_EXT ext = DeviceObject->DeviceExtension;
        PIO_STACK_LOCATION sp = IoGetCurrentIrpStackLocation(Irp);

        switch (sp->MinorFunction) {
        case IRP_MN_START_DEVICE:
                /* MUST send it down FIRST and WAIT: the lower
                   drivers must start before we touch the
                   hardware. This synchronous-forward pattern
```

```
                   is 40 lines of event and completion routine
                   in real code.                              */
                status = MyForwardAndWait(ext, Irp);
                if (NT_SUCCESS(status))
                        status = MyStartDevice(ext,
                                sp->Parameters.StartDevice.
                                   AllocatedResourcesTranslated);
                Irp->IoStatus.Status = status;
                IoCompleteRequest(Irp, IO_NO_INCREMENT);
                return status;

        case IRP_MN_REMOVE_DEVICE:
                MyStopDevice(ext);
                IoSkipCurrentIrpStackLocation(Irp);
                status = IoCallDriver(ext->LowerDevice, Irp);
                IoDetachDevice(ext->LowerDevice);
                IoDeleteDevice(DeviceObject);
                return status;
                /* NOTE: for REMOVE you forward it down and do
                   NOT complete it, then delete yourself. For
                   START you complete it. The rules differ per
                   minor function, and there are a dozen.   */

        default:
                IoSkipCurrentIrpStackLocation(Irp);
                return IoCallDriver(ext->LowerDevice, Irp);
        }
}

/* THERE ARE ROUGHLY 25 PNP MINOR FUNCTIONS, each with its own
   forwarding and completion rules, plus a state machine with
   about 15 states that you must implement correctly to pass
   certification. WDF implements all of it (chapter 14).

   THAT is why KMDF exists, and why a KMDF driver is typically
   a third the size of the WDM equivalent.               */
```

The same thing in KMDF

```
NTSTATUS
MyEvtDeviceAdd(_In_ WDFDRIVER Driver,
               _Inout_ PWDFDEVICE_INIT DeviceInit)
{
        WDF_PNPPOWER_EVENT_CALLBACKS pnpPower;
        WDF_OBJECT_ATTRIBUTES attributes;
        WDFDEVICE device;
        PMY_CONTEXT ctx;
        NTSTATUS status;

        UNREFERENCED_PARAMETER(Driver);
        PAGED_CODE();

        WDF_PNPPOWER_EVENT_CALLBACKS_INIT(&pnpPower);
        pnpPower.EvtDevicePrepareHardware = MyPrepareHardware;
        pnpPower.EvtDeviceReleaseHardware = MyReleaseHardware;
        pnpPower.EvtDeviceD0Entry         = MyD0Entry;
        pnpPower.EvtDeviceD0Exit          = MyD0Exit;
        WdfDeviceInitSetPnpPowerEventCallbacks(DeviceInit,
                                               &pnpPower);

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attributes,
                                                MY_CONTEXT);

        status = WdfDeviceCreate(&DeviceInit, &attributes,
                                 &device);
        if (!NT_SUCCESS(status))
                return status;

        ctx = MyGetContext(device);
        ctx->Device = device;

        /* publish a device interface, so user mode can find
           us without a hard-coded name (below)             */
        status = WdfDeviceCreateDeviceInterface(device,
                        &GUID_DEVINTERFACE_MYDEVICE, NULL);
        if (!NT_SUCCESS(status))
                return status;

        return MyQueueInitialize(device);
}

/* THAT IS THE WHOLE PNP IMPLEMENTATION. The framework:
     · creates the FDO and attaches it
     · implements every PnP minor function correctly
     · runs the state machine, including surprise removal,
       query-remove veto, and resource rebalance
     · calls your callbacks at the right time, at the right
       IRQL, in the right order
     · stops and restarts your queues around power transitions

   YOUR JOB reduces to four callbacks (chapter 14).      */
```

The four callbacks that matter

Callback	When	Do	IRQL
`EvtDevicePrepareHardware`	Resources assigned	Map registers, get the interrupt's resource descriptor	PASSIVE

Callback	When	Do	IRQL
`EvtDeviceD0Entry`	Powering up	Initialise the hardware, restore state	PASSIVE
`EvtDeviceD0Exit`	Powering down	Quiesce, save state	PASSIVE
`EvtDeviceReleaseHardware`	Resources revoked	Unmap. **Do not touch the hardware** — it may be gone	PASSIVE

```
NTSTATUS
MyPrepareHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesRaw,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        ULONG i;

        PAGED_CODE();
        UNREFERENCED_PARAMETER(ResourcesRaw);

        /* Walk the TRANSLATED list - the raw one holds bus
           addresses, the translated one holds addresses the
           CPU can actually use (chapter 17).              */
        for (i = 0; i < WdfCmResourceListGetCount(ResourcesTranslated);
             i++) {
                PCM_PARTIAL_RESOURCE_DESCRIPTOR desc =
                        WdfCmResourceListGetDescriptor(
                                ResourcesTranslated, i);

                switch (desc->Type) {
                case CmResourceTypeMemory:
                        ctx->RegBase = MmMapIoSpaceEx(
                                desc->u.Memory.Start,
                                desc->u.Memory.Length,
                                PAGE_READWRITE | PAGE_NOCACHE);
                        if (!ctx->RegBase)
                                return STATUS_INSUFFICIENT_RESOURCES;
                        ctx->RegLength = desc->u.Memory.Length;
                        break;

                case CmResourceTypeInterrupt:
                        /* WDF connects it for you if you
                           created a WDFINTERRUPT object
                           (chapter 17) - nothing to do here  */
                        break;
                }
        }

        if (!ctx->RegBase)
                return STATUS_DEVICE_CONFIGURATION_ERROR;

        return STATUS_SUCCESS;
}

NTSTATUS
MyReleaseHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        PAGED_CODE();
        UNREFERENCED_PARAMETER(ResourcesTranslated);

        /* DO NOT touch the hardware here. On surprise removal
           it is physically gone, and a register write may
           bugcheck or hang the bus. Quiesce in D0Exit, which
           runs while the device is still present.         */

        if (ctx->RegBase) {
                MmUnmapIoSpace(ctx->RegBase, ctx->RegLength);
                ctx->RegBase = NULL;
        }
```

```
        return STATUS_SUCCESS;
}
```

Device interfaces

```
/* A PnP device should NOT have a fixed name and symbolic link
   (chapter 03) - there may be several of them, and the name
   would collide. Instead it publishes a device INTERFACE: a
   GUID that says "I provide this kind of service".        */

/* 1. define a GUID, in a header shared with user mode: */
        // {A1B2C3D4-1234-5678-9ABC-DEF012345678}
        DEFINE_GUID(GUID_DEVINTERFACE_MYDEVICE,
                0xa1b2c3d4, 0x1234, 0x5678, 0x9a, 0xbc,
                0xde, 0xf0, 0x12, 0x34, 0x56, 0x78);

/* 2. publish it, in EvtDeviceAdd: */
        WdfDeviceCreateDeviceInterface(device,
                        &GUID_DEVINTERFACE_MYDEVICE,
                        NULL);   /* an optional reference
                                    string, to distinguish
                                    several interfaces on one
                                    device                  */

/* 3. USER MODE FINDS IT by GUID, not by name: */

        HDEVINFO info;
        SP_DEVICE_INTERFACE_DATA ifData = { sizeof(ifData) };
        PSP_DEVICE_INTERFACE_DETAIL_DATA_W detail;
        DWORD needed;
        HANDLE h;

        info = SetupDiGetClassDevsW(&GUID_DEVINTERFACE_MYDEVICE,
                        NULL, NULL,
                        DIGCF_PRESENT | DIGCF_DEVICEINTERFACE);

        SetupDiEnumDeviceInterfaces(info, NULL,
                        &GUID_DEVINTERFACE_MYDEVICE, 0, &ifData);

        SetupDiGetDeviceInterfaceDetailW(info, &ifData, NULL, 0,
                                         &needed, NULL);
        detail = malloc(needed);
        detail->cbSize = sizeof(*detail);
        SetupDiGetDeviceInterfaceDetailW(info, &ifData, detail,
                                         needed, &needed, NULL);

        h = CreateFileW(detail->DevicePath, GENERIC_READ |
                        GENERIC_WRITE, 0, NULL, OPEN_EXISTING,
                        FILE_ATTRIBUTE_NORMAL, NULL);

/* THE PATH looks like:
     \\?\pci#ven_1234&dev_5678#4&1a2b3c4d&0&00e0#{a1b2c3d4-...}

   WHY THIS IS BETTER than a symbolic link:
     · several identical devices coexist, each with its own
       path
     · user mode can enumerate ALL of them
     · arrival and removal can be watched with
       CM_Register_Notification / RegisterDeviceNotification,
       so an application reacts to hot-plug
     · no name collisions between vendors            */
```

Debugging the stack

```
# --- Device Manager, first ---
#   View -> Show hidden devices
#   The device's Properties -> Details tab:
#       Hardware Ids           what the INF must match
#       Device Instance Path
#       Upper/Lower filters
#       "Problem code"         <-- the yellow bang's meaning
#
#   COMMON PROBLEM CODES:
#     Code 10  the driver failed IRP_MN_START_DEVICE
#     Code 12  resource conflict - PnP could not assign
#     Code 18  reinstall required
#     Code 28  no driver installed
#     Code 31  the driver failed to load
#     Code 39  driver corrupt or missing
#     Code 43  the driver reported a failure
#     Code 52  signature verification failed (chapter 20)

# --- WinDbg ---
    !devnode 0 1                    the whole PnP tree
    !devnode 0 1 mydriver           just yours
    !devstack ffff8a0c12345678      one device's stack
    !devobj  ffff8a0c12345678
    !drvobj \Driver\MyDriver 7

# --- The setup log, for install-time problems ---
    C:\Windows\INF\setupapi.dev.log
# It records every INF match attempt, every rank, and exactly
# why a driver was or was not selected. This is the file to
# read when a device shows Code 28.

# --- Live PnP tracing ---
    tracelog -start pnp -guid #B3A7...  -f pnp.etl
    # or use Windows Performance Recorder with the
    # "Device I/O" profile
```

How it shows up

Symptom	Cause
Yellow bang, Code 28	No matching INF — check the hardware IDs against the INF
Code 10, "device cannot start"	`START_DEVICE` failed — usually `PrepareHardware` or `D0Entry` returned an error
Code 12	Resource conflict; the device asked for something unavailable
Code 52	Signing (ch. 20)
Bugcheck on device removal	Touched the hardware in `ReleaseHardware` after surprise removal
Hang when unplugging	Waiting for hardware that is gone; no timeout
Registers read `0xFFFFFFFF`	The device was surprise-removed — detect and fail fast
Device stops working after sleep	State not restored in `D0Entry`
Two devices, only one works	Used a fixed symbolic link instead of a device interface
Buffering behaves oddly in a filter	Flags not inherited from the lower device (ch. 07)
Random bugcheck in a filter driver	PnP IRP forwarded with the wrong rules for that minor function

Check yourself

- ✓ Draw a device stack and name PDO, FDO and filter DOs.
- ✓ Explain what filters make possible and why Linux has no direct equivalent.
- ✓ Walk the PnP sequence from enumeration to `START_DEVICE`.
- ✓ Explain why surprise removal is a normal path, not an edge case.
- ✓ Say what WDM's PnP dispatch has to do and why WDF replaces it.
- ✓ Name the four PnP/power callbacks and what belongs in each.
- ✓ Explain why you must not touch hardware in `ReleaseHardware`.
- ✓ Distinguish raw from translated resources.
- ✓ Publish a device interface and open it from user mode by GUID.
- ✓ Decode five Device Manager problem codes and know which log explains Code 28.

Memory management for drivers

Two pools, one crucial distinction, and a tagging scheme that turns "the system is leaking memory" from a week of guesswork into a single command. Plus the lookaside lists that make per-request allocation cheap.

Paged and non-paged pool

```
 NON-PAGED POOL
    · always resident in physical memory
    · usable at ANY IRQL
    · a finite, system-wide resource
    · REQUIRED for anything touched at DISPATCH_LEVEL or
      above: DPC data, ISR data, spinlock-protected state,
      DMA buffers, anything in a completion path

 PAGED POOL
    · may be written to the page file
    · usable ONLY at IRQL <= APC_LEVEL
    · much larger
    · use for: configuration, per-open state touched only
      at PASSIVE_LEVEL, large infrequently-used buffers

THE RULE, and it is chapter 02 restated:
   if the memory can be touched at DISPATCH_LEVEL or above,
   it MUST be non-paged. No exceptions.

THE FAILURE if you get it wrong: bugcheck 0xA,
IRQL_NOT_LESS_OR_EQUAL - and it is LATENT, because the page
is usually resident. It fires weeks later, under memory
pressure, on a customer machine.

WHEN IN DOUBT, USE NON-PAGED. Then measure. A driver holding
a few hundred kilobytes of unnecessary non-paged pool is a
minor inefficiency; a driver bugchecking under load is a
recall.

COMPARE LINUX: kmalloc is always resident, vmalloc is too -
Linux does not page kernel memory at all. So this whole
distinction has no Linux counterpart, and it is the thing
Linux developers most often get wrong on their first Windows
driver.
```

Allocating

```
#include <ntddk.h>

/* THE MODERN API - use this. ExAllocatePoolWithTag is
   deprecated and ExAllocatePool is long gone.             */

        PVOID p = ExAllocatePool2(POOL_FLAG_NON_PAGED,
                                  size,
                                  'DYMx');   /* the TAG */
        if (p == NULL)
                return STATUS_INSUFFICIENT_RESOURCES;

        /* ExAllocatePool2 ZEROES the memory by default -
           unlike the old ExAllocatePoolWithTag, which did
           not. That change closed a large class of
           information-disclosure bugs.                    */

        ExFreePoolWithTag(p, 'DYMx');   /* tag must MATCH */

/* THE FLAGS: */
        POOL_FLAG_NON_PAGED          resident, any IRQL
        POOL_FLAG_PAGED              pageable, PASSIVE only
        POOL_FLAG_NON_PAGED_EXECUTE  rarely justified
        POOL_FLAG_UNINITIALIZED      skip the zeroing, if you
                                     will fill it all anyway
        POOL_FLAG_CACHE_ALIGNED      start on a cache line
        POOL_FLAG_RAISE_ON_FAILURE   raise an exception rather
                                     than returning NULL

/* THE TAG is four characters, written REVERSED because it is
   stored little-endian and displayed as a string: */
        'DYMx'   displays as  "xMYD"
        ' yMD'   displays as  "DMy "

   /* So pick it so the DISPLAY is readable: */
        #define MY_POOL_TAG  'DYMx'    /* shows as xMYD */

/* USE A DISTINCT TAG PER ALLOCATION SITE, or at least per
   purpose. This is the single most valuable habit in this
   chapter - see "finding leaks" below.                    */

/* CONTIGUOUS PHYSICAL MEMORY, for DMA (chapter 18): */
        PHYSICAL_ADDRESS low = {0}, high, boundary = {0};
        high.QuadPart = 0xFFFFFFFF;      /* 32-bit device */

        p = MmAllocateContiguousMemorySpecifyCacheNode(
                size, low, high, boundary, MmCached,
                MM_ANY_NODE_OK);
        ...
        MmFreeContiguousMemory(p);

/* But prefer WDF's DMA common buffers (chapter 18), which do
   the mapping as well as the allocation.                  */
```

ALWAYS CHECK THE RETURN

`ExAllocatePool2` returns NULL on failure and does not raise. An unchecked allocation is a NULL dereference at whatever IRQL you happen to be at, which is bugcheck 0xA with a confusing address near zero. Every allocation, every time — Driver Verifier's low-resource simulation exists specifically to find the ones you missed (chapter 21).

Lookaside lists

```
/* For many same-sized objects - one per request, one per
   packet. The Windows equivalent of a Linux slab cache
   (Linux ch. 05).                                          */

        LOOKASIDE_LIST_EX Lookaside;

        status = ExInitializeLookasideListEx(
                        &ctx->Lookaside,
                        NULL,          /* custom allocate  */
                        NULL,          /* custom free      */
                        NonPagedPoolNx,
                        0,             /* flags            */
                        sizeof(MY_REQUEST),
                        MY_POOL_TAG,
                        0);            /* depth: 0 = auto  */
        if (!NT_SUCCESS(status))
                return status;

        PMY_REQUEST req = ExAllocateFromLookasideListEx(
                                &ctx->Lookaside);
        if (req == NULL)
                return STATUS_INSUFFICIENT_RESOURCES;
        ...
        ExFreeToLookasideListEx(&ctx->Lookaside, req);

        ExDeleteLookasideListEx(&ctx->Lookaside);   /* unload */

/* WHY: the list keeps a small per-CPU cache of freed objects,
   so the common case is a pop off a list rather than a pool
   allocation with its locking. The system TUNES THE DEPTH
   automatically based on your hit rate.

   USE ONE when you allocate the same size repeatedly in a hot
   path. Do not bother for a handful of allocations at
   startup.

   NOTE NonPagedPoolNx - the NX (no-execute) variant. Plain
   NonPagedPool is executable and deprecated for that reason;
   always use the Nx form.                                  */
```

Finding pool leaks

```
# THIS IS WHY YOU TAG EVERY ALLOCATION.

# --- On the live machine ---
    poolmon.exe                    # from the WDK
    # sort by bytes with 'b', by allocation count with 'a'
    # Watch YOUR tag over time. A tag whose "Diff" column
    # climbs and never falls is your leak, named precisely.

     Tag  Type     Allocs      Frees      Diff   Bytes
     xMYD Nonp     104321     100002      4319   1105664  <--
     xMYB Nonp        512        512         0         0

# --- Enable pool tagging if it is off ---
    gflags /r +ptg
    # (on by default on checked builds and modern Windows)

# --- In WinDbg ---
    !poolused 2                    # sorted by non-paged usage
    !poolused /t 10 2              # top 10
    !poolfind xMYD                 # every allocation with
                                   # this tag
    !pool ffff8a0c12345678         # what is this address?
      Pool page ffff8a0c12345678 region is Nonpaged pool
       ffff8a0c12345670 size:   40 previous size:    0
                                   (Allocated) xMYD

# --- Track allocation STACKS, not just counts ---
    gflags /i mydriver.sys +ust    # user stack trace
    # or Driver Verifier's pool tracking (chapter 21), then:
    !verifier 3 mydriver.sys
    # shows the stack for every outstanding allocation

# THE WORKFLOW for "the system is running out of non-paged
# pool":
#   1. poolmon, sort by Diff, find the tag
#   2. the tag names the driver and the allocation site
#   3. verifier pool tracking gives you the stack
#   4. fix the missing free
# Steps 1 and 2 take about thirty seconds IF the tags are
# distinct. With every allocation using the same tag, or the
# default, you learn only "some driver is leaking".
```

Pool corruption

```
/* Writing past the end of an allocation corrupts the pool
   header of the NEXT block. The crash happens later, in an
   unrelated allocation, in someone else's driver.         */

/* BUGCHECKS THAT MEAN POOL CORRUPTION:
     0xC2  BAD_POOL_CALLER
             parameter 1 says what:
               0x07  freeing already-freed memory
               0x0D  freeing with the wrong tag
               0x99  freeing an invalid address
     0x19  BAD_POOL_HEADER
     0x1A  MEMORY_MANAGEMENT
     0xCD  DRIVER_PAGE_FAULT_BEYOND_END_OF_ALLOCATION
     0xC1  SPECIAL_POOL_DETECTED_MEMORY_CORRUPTION

/* SPECIAL POOL is the tool. It puts each allocation on its
   own page, with a GUARD PAGE immediately after (or before),
   so an overrun faults AT THE MOMENT IT HAPPENS rather than
   corrupting a neighbour.                                  */

         verifier /flags 0x1 /driver mydriver.sys
         # or, targeted at one tag, without full verifier:
         gflags /r +spp
         # then set the tag in
         # HKLM\SYSTEM\CCS\Control\Session Manager\Memory
         # Management\PoolTag

/* Special pool is the Windows equivalent of KASAN
   (Linux ch. 27): it converts a delayed, misattributed crash
   into an immediate, correctly-attributed one. Enable it
   whenever you are chasing corruption, and keep it on during
   development.

   COST: each allocation consumes a full page, so the system
   uses far more memory and runs slower. Target it at your
   driver, not the whole system.                           */
```

Touching user memory safely

```
/* A user-mode address is only valid:
     · in the context of THAT process
     · at IRQL <= APC_LEVEL (it can page-fault)
     · until the user changes it, which they can do at any
       moment from another thread                          */

/* THE MANDATORY PATTERN when you handle a raw user pointer -
   METHOD_NEITHER, or a pointer embedded in a buffer
   (chapter 07):                                            */

        __try {
                if (Irp->RequestorMode != KernelMode) {
                        ProbeForRead(userBuffer, length,
                                     __alignof(ULONG));
                        /* or ProbeForWrite for output */
                }

                /* COPY IT IN. Do not validate in place - the
                   user can change it between your check and
                   your use, from another thread. Same TOCTOU
                   argument as isolation chapter 09.       */
                RtlCopyMemory(localCopy, userBuffer, length);

        } __except (EXCEPTION_EXECUTE_HANDLER) {
                return GetExceptionCode();
        }

        /* now validate localCopy, and use ONLY localCopy */

/* WHAT ProbeForRead/Write CHECK:
     · the range lies entirely in user address space
     · the alignment is correct
     · nothing about whether it is mapped - a fault can still
       occur, which is why the __try is also required

/* THE THREE MISTAKES:
     1. no __try. A bad pointer becomes a bugcheck instead of
        an error return.
     2. probing but then using the ORIGINAL pointer. The probe
        proved nothing about what is there a microsecond
        later.
     3. probing at DISPATCH_LEVEL. Illegal - user memory is
        pageable.

/* THE GOOD NEWS: with METHOD_BUFFERED or METHOD_*_DIRECT
   (chapter 07), the I/O manager does all of this for you, and
   you never see a user pointer. That is why METHOD_NEITHER is
   discouraged.                                             */
```

A first look at MDLs

```
/* A Memory Descriptor List describes a buffer's PHYSICAL
   pages. Covered fully in chapter 08; here is why it exists.

   The problem: a user buffer is virtual, in one process's
   address space, and possibly paged out. A DMA engine needs
   physical addresses, and a DPC needs an address valid in any
   context. An MDL solves both: */

        PMDL mdl = IoAllocateMdl(buffer, length, FALSE, FALSE,
                                 NULL);
        if (!mdl)
                return STATUS_INSUFFICIENT_RESOURCES;

        __try {
                MmProbeAndLockPages(mdl, UserMode,
                                    IoWriteAccess);
        } __except (EXCEPTION_EXECUTE_HANDLER) {
                IoFreeMdl(mdl);
                return GetExceptionCode();
        }

        /* now: the pages are LOCKED IN MEMORY and cannot be
           paged out or freed. Get a kernel address valid in
           any context: */
        PVOID kva = MmGetSystemAddressForMdlSafe(mdl,
                        NormalPagePriority | MdlMappingNoExecute);
        if (!kva) {
                MmUnlockPages(mdl);
                IoFreeMdl(mdl);
                return STATUS_INSUFFICIENT_RESOURCES;
        }

        ... use kva at any IRQL, in any process context ...

        MmUnlockPages(mdl);
        IoFreeMdl(mdl);

/* USE MmGetSystemAddressForMdlSafe, NEVER the older
   MmGetSystemAddressForMdl - the latter raises a bugcheck on
   failure instead of returning NULL, which turns a resource
   shortage into a crash.                                    */
```

Pool limits and quotas

```
/* Non-paged pool is finite and shared. A driver that
   allocates without bound in response to user requests is a
   denial-of-service vector - any user can exhaust system
   memory.                                                  */

/* CHARGE IT TO THE CALLER'S PROCESS, so their quota limits
   them rather than the system: */
        p = ExAllocatePool2(POOL_FLAG_NON_PAGED |
                            POOL_FLAG_USE_QUOTA, size, TAG);

/* OR, better, BOUND IT YOURSELF: */
        if (requestedSize > MY_MAX_BUFFER)
                return STATUS_INVALID_PARAMETER;
        if (ctx->OutstandingRequests >= MY_MAX_REQUESTS)
                return STATUS_INSUFFICIENT_RESOURCES;

/* SEE THE LIMITS: */
        WinDbg:  !vm
                 Non-paged pool usage / maximum
        Perfmon: Memory \ Pool Nonpaged Bytes
        Task Manager -> Performance -> Memory ->
                        Non-paged pool

/* THE DESIGN RULE, and it is the same as the Linux one
   (ch. 06): preallocate what you need at start time, bound
   what you allocate per request, and never let an untrusted
   caller decide how much kernel memory to consume.        */
```

How it shows up

Symptom	Cause
Bugcheck 0xA under memory pressure only	Paged pool touched at DISPATCH_LEVEL — latent until the page is evicted
Bugcheck 0xC2, parameter 1 = 0x07	Double free
Bugcheck 0xC2, parameter 1 = 0x0D	Freed with the wrong tag
Bugcheck 0x19 or 0xCD	Pool overrun — enable special pool
Crash in an unrelated driver	You corrupted the pool; special pool will attribute it correctly
Non-paged pool climbs forever	Leak — `poolmon`, sort by Diff, read the tag
Leak found but the site is unknown	Same tag used everywhere — use distinct tags
Bugcheck at an address near zero	Unchecked allocation return
Bugcheck touching a user pointer	No `__try`, or probed at the wrong IRQL
User changes data after your validation	Validated in place instead of copying in
A user program exhausts system memory	Unbounded allocation driven by request parameters
Crash when `MmGetSystemAddressForMdl` fails	Use the `Safe` variant

Check yourself

- ✓ State the rule for choosing paged versus non-paged pool, and why the wrong choice is latent.
- ✓ Explain why this distinction has no Linux equivalent.

- ✓ Allocate and free with `ExAllocatePool2`, and explain the tag byte order.
- ✓ Explain why distinct tags per site are worth the effort.
- ✓ Use a lookaside list and say when it is justified.
- ✓ Find a leak with `poolmon` and `!poolused`.
- ✓ Explain what special pool does and what it corresponds to on Linux.
- ✓ Write the full safe pattern for touching a user pointer, and name the three mistakes.
- ✓ Explain what an MDL gives you and why the `Safe` variant matters.
- ✓ Explain why unbounded allocation from a request is a security bug.

User buffers: buffered, direct and neither

Three ways the I/O manager can hand you a user's buffer, with completely different performance, safety and access rules. Choosing correctly is a design decision made once per interface — and choosing `METHOD_NEITHER` by accident is how drivers get CVEs.

The three methods

	METHOD_BUFFERED	METHOD_IN/OUT_DIRECT	METHOD_NEITHER
What you get	A kernel copy	An MDL	The raw user pointer
Field	`Irp->AssociatedIrp.SystemBuffer`	`Irp->MdlAddress`	`Irp->UserBuffer`
Copy cost	Two copies	**Zero**	Zero
Safe at any IRQL	**Yes**	Yes, once mapped	**No**
Valid in any process	**Yes**	Yes	**No**
Probing done for you	Yes	Yes	**No**
Good for	Small transfers, control	Large transfers, DMA	Almost nothing
Typical size	< 4 KB	> 4 KB	—

THE DECISION, IN ONE LINE

Use **METHOD_BUFFERED** for control operations and small data. Use **METHOD_OUT_DIRECT** / **METHOD_IN_DIRECT** for bulk data and anything you will DMA. Use **METHOD_NEITHER** only when you have a specific, defensible reason — and then write the probing code with the care of someone who knows it is the most commonly botched thing in Windows drivers.

METHOD_BUFFERED

```
 WRITE:
   user buffer --copy--> SystemBuffer (non-paged pool)
                              |
                              v
                         your driver reads it

 READ:
   your driver writes SystemBuffer
                              |
   user buffer <--copy-- SystemBuffer
   (the I/O manager copies back AT COMPLETION)

ONE buffer serves BOTH directions in an IOCTL. It is sized
max(InputBufferLength, OutputBufferLength), so:

   · your input is available when you start
   · you overwrite it with output
   · the I/O manager copies OutputBufferLength bytes back

READ YOUR INPUT BEFORE WRITING YOUR OUTPUT. Writing output
first destroys the input you have not read yet - a bug that
works whenever input and output happen not to overlap, and
fails when they do.

THE SAFETY: the I/O manager probed the user buffer, copied
it in kernel space in the CALLER'S context at PASSIVE_LEVEL,
and the copy is in non-paged pool. So SystemBuffer is safe
to touch at any IRQL, in any thread, at any time - which is
exactly what you need for asynchronous completion from a
DPC.

THE COST: two copies. For a 4-byte control value that is
irrelevant. For a 4 MB frame it is not.
```

```
NTSTATUS
MyDeviceControl(_In_ PDEVICE_OBJECT DeviceObject, _In_ PIRP Irp)
{
        PIO_STACK_LOCATION sp = IoGetCurrentIrpStackLocation(Irp);
        ULONG inLen  = sp->Parameters.DeviceIoControl.
                           InputBufferLength;
        ULONG outLen = sp->Parameters.DeviceIoControl.
                           OutputBufferLength;
        PVOID buffer = Irp->AssociatedIrp.SystemBuffer;
        NTSTATUS status;
        ULONG_PTR info = 0;

        switch (sp->Parameters.DeviceIoControl.IoControlCode) {
        case IOCTL_MY_SET_CONFIG: {
                MY_CONFIG cfg;

                if (inLen < sizeof(MY_CONFIG)) {
                        status = STATUS_BUFFER_TOO_SMALL;
                        break;
                }
                /* SystemBuffer is a kernel copy already, but
                   copy again into a local if you will write
                   output into the same buffer               */
                RtlCopyMemory(&cfg, buffer, sizeof(cfg));

                status = MyValidateAndApply(&cfg);
                break;
        }

        case IOCTL_MY_GET_STATS: {
                MY_STATS stats;

                if (outLen < sizeof(MY_STATS)) {
                        status = STATUS_BUFFER_TOO_SMALL;
                        Irp->IoStatus.Information = sizeof(MY_STATS);
                        break;          /* tell them the size */
                }
                MyGatherStats(&stats);
                RtlCopyMemory(buffer, &stats, sizeof(stats));
                info = sizeof(stats);
                status = STATUS_SUCCESS;
                break;
        }

        default:
                status = STATUS_INVALID_DEVICE_REQUEST;
                break;
        }

        Irp->IoStatus.Status = status;
        Irp->IoStatus.Information = info;
        IoCompleteRequest(Irp, IO_NO_INCREMENT);
        return status;
}
```

METHOD_DIRECT

```
The I/O manager:
  1. probes the user buffer
  2. builds an MDL describing its physical pages
  3. LOCKS those pages in memory
  4. hands you Irp->MdlAddress

NO COPY happens. You either:
  · map the MDL to a kernel virtual address and touch it,
    or
  · hand the MDL straight to a DMA engine (chapter 18)

METHOD_IN_DIRECT   the buffer is INPUT to the driver
                   (despite "IN" - the user WRITES it, the
                   driver READS it)
METHOD_OUT_DIRECT  the buffer is OUTPUT from the driver

THE NAMING IS GENUINELY CONFUSING. Remember it from the
USER's point of view: OUT_DIRECT means data comes OUT of
the driver to the user, so the driver writes it. That is
the common case for a read.

AND FOR IOCTLs: only the OUTPUT buffer is described by the
MDL. The INPUT buffer of a DIRECT ioctl is still buffered,
in SystemBuffer. So a DIRECT ioctl gives you BOTH:

     input  -> Irp->AssociatedIrp.SystemBuffer
     output -> Irp->MdlAddress

That asymmetry catches everyone once.
```

```
        PVOID kva;
        ULONG length;

        if (Irp->MdlAddress == NULL) {
                status = STATUS_INVALID_PARAMETER;
                goto complete;
        }

        length = MmGetMdlByteCount(Irp->MdlAddress);

        /* map it to a kernel address valid in ANY context */
        kva = MmGetSystemAddressForMdlSafe(Irp->MdlAddress,
                        NormalPagePriority | MdlMappingNoExecute);
        if (kva == NULL) {
                status = STATUS_INSUFFICIENT_RESOURCES;
                goto complete;
        }

        RtlCopyMemory(kva, source, min(length, sourceLength));
        info = min(length, sourceLength);
        status = STATUS_SUCCESS;

/* NOTES:
     · MmGetSystemAddressForMdlSafe can FAIL under memory
       pressure - check it. The non-Safe variant bugchecks
       instead, which is why it is never the right call.
     · MdlMappingNoExecute is required on modern Windows;
       omitting it fails the mapping outright.
     · the I/O manager unlocks and frees the MDL at
       completion. Do NOT unlock it yourself.
     · for DMA you do not map at all - you pass the MDL to the
       DMA adapter (chapter 18). That is the real payoff:
       user pages go straight to the hardware with no copy and
       no kernel mapping.                                  */
```

METHOD_NEITHER

```
/* You get the RAW USER POINTERS:
        input  -> sp->Parameters.DeviceIoControl.Type3InputBuffer
        output -> Irp->UserBuffer

   NOTHING has been probed, nothing copied, nothing locked.
   These addresses are valid ONLY:
        · in the calling process's context
        · at IRQL <= APC_LEVEL
        · until the user changes the memory, which another
          thread can do at any instant                      */

/* THE ONLY CORRECT WAY TO USE IT: */

        PVOID userIn  = sp->Parameters.DeviceIoControl.
                            Type3InputBuffer;
        PVOID userOut = Irp->UserBuffer;
        MY_CONFIG local;

        if (inLen < sizeof(MY_CONFIG))
                return STATUS_BUFFER_TOO_SMALL;

        __try {
                if (Irp->RequestorMode != KernelMode) {
                        ProbeForRead(userIn, inLen,
                                     __alignof(MY_CONFIG));
                        ProbeForWrite(userOut, outLen,
                                      __alignof(ULONG));
                }

                /* COPY IN. Never validate in place. */
                RtlCopyMemory(&local, userIn, sizeof(local));

        } __except (EXCEPTION_EXECUTE_HANDLER) {
                return GetExceptionCode();
        }

        /* validate the COPY, use only the COPY */
        if (local.Rate == 0 || local.Rate > MAX_RATE)
                return STATUS_INVALID_PARAMETER;

        status = MyApply(&local);

        __try {
                RtlCopyMemory(userOut, &result, sizeof(result));
        } __except (EXCEPTION_EXECUTE_HANDLER) {
                return GetExceptionCode();
        }
```

WHY METHOD_NEITHER PRODUCES CVES

- **No `__try`.** A bad pointer becomes a bugcheck — a denial of service any user can trigger.
- **Probed, then used the original pointer.** Another thread rewrites the buffer between the probe and the use. Classic TOCTOU (the same argument as isolation chapter 09).
- **Validated in place.** Same problem: the value you checked is not the value you use.
- **Forgot `RequestorMode`.** Probing is skipped for kernel callers — but if you never check, a user-mode caller's kernel-looking address is used unprobed.
- **Touched the pointer from a DPC or another thread.** The address means nothing outside the caller's process context.
- **Trusted a length field inside the user buffer** to bound a copy.

Every one of these has shipped in real drivers. If you cannot state why `METHOD_BUFFERED` or `METHOD_DIRECT` is unacceptable for your case, do not use `METHOD_NEITHER`.

Choosing it, per interface

```
/* FOR READ/WRITE: the DEVICE OBJECT's flags decide, and it
   applies to every read and write on that device.        */

        /* WDM: */
        deviceObject->Flags |= DO_BUFFERED_IO;
        /* or */
        deviceObject->Flags |= DO_DIRECT_IO;
        /* neither flag set = METHOD_NEITHER */

        /* WDF: */
        WdfDeviceInitSetIoType(DeviceInit,
                               WdfDeviceIoBuffered);
        WdfDeviceInitSetIoType(DeviceInit,
                               WdfDeviceIoDirect);
        /* and per-queue, in newer versions: */
        WdfDeviceInitSetIoTypeEx(DeviceInit, &ioTypeConfig);

/* FOR IOCTLs: encoded in the CONTROL CODE itself
   (chapter 16), so it is per-operation: */

        #define IOCTL_MY_SET_CONFIG \
            CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, \
                     METHOD_BUFFERED, FILE_WRITE_ACCESS)

        #define IOCTL_MY_READ_BULK \
            CTL_CODE(FILE_DEVICE_UNKNOWN, 0x801, \
                     METHOD_OUT_DIRECT, FILE_READ_ACCESS)

/* SO A SINGLE DRIVER can use BUFFERED for its control IOCTLs
   and DIRECT for its bulk transfer IOCTL, which is exactly
   the right design: cheap safety where the data is small,
   zero copy where it is large.                           */

/* AND IN A FILTER DRIVER: inherit the method from the device
   below you, or reads and writes arrive in a form your lower
   driver does not expect:                                */
        fdo->Flags |= lowerDevice->Flags &
                      (DO_BUFFERED_IO | DO_DIRECT_IO);
```

In WDF, it is simpler

```
/* WDF hides the method behind one API. You ask for a buffer
   and the framework gives you a safe kernel pointer,
   whichever transfer method was used.                    */

VOID
MyEvtIoDeviceControl(_In_ WDFQUEUE Queue,
                     _In_ WDFREQUEST Request,
                     _In_ size_t OutputBufferLength,
                     _In_ size_t InputBufferLength,
                     _In_ ULONG IoControlCode)
{
        PVOID inBuf, outBuf;
        size_t inLen, outLen;
        NTSTATUS status;

        status = WdfRequestRetrieveInputBuffer(Request,
                        sizeof(MY_CONFIG),   /* MINIMUM size -
                                                checked FOR you */
                        &inBuf, &inLen);
        if (!NT_SUCCESS(status)) {
                WdfRequestComplete(Request, status);
                return;
        }

        status = WdfRequestRetrieveOutputBuffer(Request,
                        sizeof(MY_STATS), &outBuf, &outLen);
        ...

        WdfRequestCompleteWithInformation(Request, status,
                                          bytesWritten);
}

/* WdfRequestRetrieve*Buffer:
     · works for BUFFERED and DIRECT identically
     · CHECKS THE MINIMUM LENGTH and returns
       STATUS_BUFFER_TOO_SMALL if it is short - so the most
       common validation is done for you
     · for DIRECT, maps the MDL and handles the failure
     · refuses outright for METHOD_NEITHER unless you opted
       in

   AND for DMA, ask for the MDL instead:               */
        status = WdfRequestRetrieveOutputWdmMdl(Request, &mdl);

/* THIS IS THE ARGUMENT FOR WDF in miniature: the same code
   handles both transfer methods, the length check is not
   optional-and-forgotten, and METHOD_NEITHER is opt-in rather
   than the default you fall into.                        */
```

How it shows up

Symptom	Cause
Buffer is NULL	Wrong field for the method — `SystemBuffer` vs `MdlAddress` vs `UserBuffer`
Input data is garbage after writing output	METHOD_BUFFERED shares one buffer — read input first
Bugcheck touching the buffer in a DPC	METHOD_NEITHER pointer used outside the caller's context

Symptom	Cause
Bugcheck on a bad user pointer	No `__try` / `__except`
Validation passes, bad data used	Validated in place instead of copying in
Works for small buffers, fails for large	DIRECT MDL not mapped, or length taken from the wrong place
`MmGetSystemAddressForMdlSafe` returns NULL	Memory pressure — handle it, do not use the unsafe variant
DIRECT ioctl input buffer is NULL	Input is still in `SystemBuffer`, not the MDL
Filter driver corrupts data	Buffering flags not inherited from the lower device
Output not returned to the user	`IoStatus.Information` not set
Security review flags the driver	METHOD_NEITHER without justification

Check yourself

- ✓ Name the three methods, the field each uses, and their trade-offs.
- ✓ Explain why METHOD_BUFFERED is safe at any IRQL and in any process.
- ✓ Explain the shared input/output buffer and the ordering rule it implies.
- ✓ Explain what METHOD_DIRECT gives you and why the input buffer of a DIRECT ioctl is still buffered.
- ✓ Say why `MmGetSystemAddressForMdlSafe` is the only correct choice.
- ✓ Write the complete safe pattern for METHOD_NEITHER.
- ✓ List six ways METHOD_NEITHER is got wrong.
- ✓ Set the method for read/write and per-IOCTL, in both WDM and WDF.
- ✓ Explain what `WdfRequestRetrieveInputBuffer` does for you.
- ✓ Choose the right method for four different interfaces and justify each.

MDLs and locked memory

A Memory Descriptor List describes where a virtual buffer really lives in physical memory. It is how Windows hands user pages to a DMA engine, how a driver reaches a buffer from a DPC, and how one buffer gets two mappings — and it has a strict ownership protocol you must not violate.

What an MDL describes

```
A virtual buffer:

  0x00007ff6'12345678, length 12288 (3 pages)

is scattered in physical memory:

  +-------------+   page 1  -> PFN 0x1A2B3
  +-------------+   page 2  -> PFN 0x0C4D5   (not adjacent)
  +-------------+   page 3  -> PFN 0x39E10

THE MDL captures exactly that:

+---------------------------------------+
|  MDL HEADER                           |
|    Next          chain to another MDL |
|    Size                               |
|    MdlFlags      locked? mapped? ...  |
|    Process       whose address space  |
|    MappedSystemVa  the kernel mapping |
|    StartVa       page-aligned base    |
|    ByteCount     length               |
|    ByteOffset    offset into the first|
|                  page                 |
+---------------------------------------+
|  PFN ARRAY:  0x1A2B3 0x0C4D5 0x39E10  |
+---------------------------------------+

THE ACCESSORS - use these, never the fields directly:

  MmGetMdlVirtualAddress(mdl)   the original VA
  MmGetMdlByteCount(mdl)        length
  MmGetMdlByteOffset(mdl)       offset in the first page
  MmGetMdlPfnArray(mdl)         the page frame numbers
  MmGetSystemAddressForMdlSafe(mdl, priority)
                                a KERNEL VA, any context

NOTE ByteOffset: a buffer rarely starts on a page boundary,
so the first page is partial. Code that assumes
page-aligned data - a DMA descriptor builder, for example -
must account for it, and forgetting is a classic
off-by-a-few-bytes corruption.
```

Why they exist

Problem	What the MDL solves
A DMA engine needs physical addresses	The PFN array *is* the physical description
User pages can be paged out mid-transfer	`MmProbeAndLockPages` pins them

Problem	What the MDL solves
A user VA is meaningless in a DPC	Map it to a system VA valid in any context
A user VA is meaningless in another process	Same
Copying large buffers is expensive	No copy at all — describe, do not duplicate
The buffer is not physically contiguous	Scatter-gather, straight from the PFN array

The Linux analogue is roughly a `struct scatterlist` plus `get_user_pages()` (chapter 24): the same two jobs — pin the pages, describe them physically — combined into one object.

The ownership protocol

```
/* The steps, and every one of them must be undone. */

/* 1. ALLOCATE - describes the buffer, does nothing yet */
        PMDL mdl = IoAllocateMdl(buffer,        /* VA        */
                                 length,
                                 FALSE,         /* secondary */
                                 FALSE,         /* charge
                                                   quota     */
                                 NULL);         /* no IRP    */
        if (mdl == NULL)
                return STATUS_INSUFFICIENT_RESOURCES;

/* 2. PROBE AND LOCK - pins the pages, fills the PFN array.
      CAN RAISE. Must be at IRQL <= APC_LEVEL, in the
      process that owns the buffer.                        */
        __try {
                MmProbeAndLockPages(mdl,
                                    UserMode,   /* or KernelMode */
                                    IoWriteAccess);
                                    /* IoReadAccess, IoWriteAccess,
                                       IoModifyAccess       */
        } __except (EXCEPTION_EXECUTE_HANDLER) {
                IoFreeMdl(mdl);
                return GetExceptionCode();
        }

/* 3. MAP, if the CPU will touch it */
        PVOID kva = MmGetSystemAddressForMdlSafe(mdl,
                        NormalPagePriority | MdlMappingNoExecute);
        if (kva == NULL) {
                MmUnlockPages(mdl);
                IoFreeMdl(mdl);
                return STATUS_INSUFFICIENT_RESOURCES;
        }

/* 4. USE IT - kva is valid at ANY IRQL, in ANY process */
        RtlCopyMemory(kva, source, length);

/* 5. UNWIND, in exactly reverse order */
        MmUnlockPages(mdl);     /* unmaps too, if mapped */
        IoFreeMdl(mdl);

/* THE ACCESS ARGUMENT IS A SECURITY CONTROL. IoWriteAccess
   means "the driver will WRITE this buffer", and the memory
   manager verifies the caller has write permission to it.
   Requesting IoReadAccess and then writing corrupts a
   read-only mapping - possibly a shared code page.       */

/* AND `UserMode` versus `KernelMode`, exactly as in
   chapter 03: pass UserMode when the buffer came from a user
   request, so the access check is enforced.             */
```

> **LOCKED PAGES ARE A SYSTEM-WIDE RESOURCE**
>
> `MmProbeAndLockPages` removes pages from the pageable pool for as long as the MDL is locked. A driver that locks unbounded amounts, or fails to unlock on an error path, degrades the whole machine and eventually causes `STATUS_WORKING_SET_QUOTA` failures for everyone. Bound how much you lock, unlock on *every* path, and never hold a lock across an operation with no timeout.

MDLs the I/O manager gives you

```
/* With METHOD_DIRECT (chapter 07), the I/O manager has
   already done steps 1 and 2 for you.                     */

        PMDL mdl = Irp->MdlAddress;

        if (mdl == NULL)
                return STATUS_INVALID_PARAMETER;

        PVOID kva = MmGetSystemAddressForMdlSafe(mdl,
                        NormalPagePriority | MdlMappingNoExecute);
        if (kva == NULL)
                return STATUS_INSUFFICIENT_RESOURCES;

        ULONG len = MmGetMdlByteCount(mdl);

        ... use it ...

/* DO NOT UNLOCK OR FREE IT. IoCompleteRequest does that.
   Calling MmUnlockPages on an IRP's MDL is a double-unlock
   and bugchecks.

   THE RULE: whoever allocated the MDL frees it. If the I/O
   manager built it, the I/O manager tears it down.       */

/* IN WDF you rarely see the MDL at all: */
        status = WdfRequestRetrieveOutputBuffer(Request, minLen,
                                                &buffer, &len);
        /* the framework maps it and checks the length */

        /* unless you want the MDL, for DMA: */
        status = WdfRequestRetrieveOutputWdmMdl(Request, &mdl);
```

Double mapping

```
/* An MDL can describe pages that are ALSO mapped somewhere
   else. That gives you a second view of the same physical
   memory - and it is how a driver shares a buffer with user
   mode.                                                    */

/* --- kernel buffer, mapped into a user process ---------- */

        PVOID kernelBuf;
        PMDL  mdl;
        PVOID userVa = NULL;

        kernelBuf = ExAllocatePool2(POOL_FLAG_NON_PAGED,
                                    size, TAG);
        if (!kernelBuf)
                return STATUS_INSUFFICIENT_RESOURCES;

        mdl = IoAllocateMdl(kernelBuf, size, FALSE, FALSE, NULL);
        if (!mdl) { ... }

        MmBuildMdlForNonPagedPool(mdl);   /* non-paged pool is
                                             already resident -
                                             no probe needed  */

        __try {
                userVa = MmMapLockedPagesSpecifyCache(
                                mdl,
                                UserMode,
                                MmCached,
                                NULL,       /* let it choose */
                                FALSE,
                                NormalPagePriority);
        } __except (EXCEPTION_EXECUTE_HANDLER) {
                status = GetExceptionCode();
                goto err;
        }

        /* userVa is now valid IN THE CURRENT PROCESS ONLY.
           Return it to the caller; they see the same physical
           memory the driver writes.                        */

        /* teardown, IN THE SAME PROCESS: */
        MmUnmapLockedPages(userVa, mdl);
        IoFreeMdl(mdl);
        ExFreePoolWithTag(kernelBuf, TAG);

/* THE RULES for a shared mapping, and they are strict:
     · map and unmap in the SAME process context. Doing it
       from a system thread maps into the wrong address space.
     · unmap BEFORE the process exits, or in IRP_MJ_CLEANUP
       (chapter 04) - which is exactly what CLEANUP is for.
     · the user can write anything into it at any time. Treat
       every byte as hostile, exactly as with a shared TEE
       buffer (isolation chapter 09).
     · never share memory containing anything the user should
       not see - the whole page is exposed, not just your
       struct.

   THIS IS A REAL SECURITY BOUNDARY, and shared-memory
   interfaces are where driver security reviews concentrate.
```

```
   Prefer IOCTLs unless the throughput genuinely demands
   sharing.                                                */
```

Partial MDLs

```
/* Describe a SUB-RANGE of an existing MDL, without locking
   anything again. Used to split a large transfer into
   hardware-sized chunks.                                  */

        PMDL partial = IoAllocateMdl(
                        (PUCHAR)MmGetMdlVirtualAddress(sourceMdl)
                                + offset,
                        chunkLength,
                        FALSE, FALSE, NULL);
        if (!partial)
                return STATUS_INSUFFICIENT_RESOURCES;

        IoBuildPartialMdl(sourceMdl,        /* the parent   */
                          partial,
                          (PUCHAR)MmGetMdlVirtualAddress(sourceMdl)
                                  + offset,
                          chunkLength);

        ... use `partial` for one hardware transfer ...

        IoFreeMdl(partial);      /* free ONLY the partial -
                                    do NOT unlock; the parent
                                    owns the lock            */

/* THE OWNERSHIP RULE AGAIN: the partial MDL borrows the
   parent's locked pages. Unlocking through the partial
   corrupts the parent's state and bugchecks later.

   AND: the parent must outlive every partial derived from it.
   Completing the parent IRP while a partial is still in
   flight is a use-after-free with a DMA engine writing into
   it - about the worst failure mode available.           */
```

MDL chains

```
/* MDLs link through ->Next, describing a discontiguous
   logical buffer - used heavily by the network stack, where
   a packet is a header MDL plus a payload MDL.            */

        PMDL mdl;
        ULONG total = 0;

        for (mdl = firstMdl; mdl != NULL; mdl = mdl->Next) {
                PVOID va = MmGetSystemAddressForMdlSafe(mdl,
                                LowPagePriority |
                                MdlMappingNoExecute);
                if (va == NULL)
                        return STATUS_INSUFFICIENT_RESOURCES;

                ProcessChunk(va, MmGetMdlByteCount(mdl));
                total += MmGetMdlByteCount(mdl);
        }

/* ALWAYS WALK THE WHOLE CHAIN. Code that handles only the
   first MDL works perfectly for small transfers and silently
   truncates large ones - a data-loss bug that appears only
   under load, which is the hardest kind to attribute.    */
```

Debugging MDLs

```
# In WinDbg (chapter 22):

    dt nt!_MDL ffff8a0c12345678
       +0x000 Next             : (null)
       +0x008 Size             : 0n56
       +0x00a MdlFlags         : 0x2004
       +0x010 Process          : 0xffff8a0c`11223344 _EPROCESS
       +0x018 MappedSystemVa   : 0xffffcb01`23456000
       +0x020 StartVa          : 0x00007ff6`12345000
       +0x028 ByteCount        : 0x3000
       +0x02c ByteOffset       : 0x678

# DECODE MdlFlags - this tells you the MDL's state:
    0x0001  MDL_MAPPED_TO_SYSTEM_VA
    0x0002  MDL_PAGES_LOCKED
    0x0004  MDL_SOURCE_IS_NONPAGED_POOL
    0x0008  MDL_ALLOCATED_FIXED_SIZE
    0x0010  MDL_PARTIAL
    0x0020  MDL_PARTIAL_HAS_BEEN_MAPPED
    0x0040  MDL_IO_PAGE_READ
    0x0080  MDL_WRITE_OPERATION
    0x1000  MDL_IO_SPACE

# So 0x2004 = SOURCE_IS_NONPAGED_POOL | (0x2000 internal).
# No PAGES_LOCKED bit -> it was never probed and locked, which
# is a bug if you expected it to be.

    !pte 0xffff8a0c12345678         # is this VA mapped?
    !pfn 0x1a2b3                    # one physical page's state
    !process 0 7                    # locked pages per process

# DRIVER VERIFIER (chapter 21) checks MDL rules aggressively:
#   unlocking an MDL you did not lock
#   freeing a mapped MDL
#   using an MDL after completion
#   mapping in the wrong process
# Every one of these is otherwise an intermittent crash.
```

How it shows up

Symptom	Cause
Bugcheck in `MmUnlockPages`	Unlocked an MDL you did not lock, or unlocked twice
Bugcheck completing an IRP	Freed or unlocked the IRP's MDL yourself
`MmGetSystemAddressForMdlSafe` returns NULL	Memory pressure — a real case to handle
Mapping fails on modern Windows	Missing `MdlMappingNoExecute`
Data corrupt by a few bytes at the start	`ByteOffset` ignored — the first page is partial
Large transfers truncated	Only the first MDL of a chain processed
User mapping is invalid	Mapped in a different process than the one using it
Crash when the user process exits	Mapping not torn down in `IRP_MJ_CLEANUP`
DMA writes into freed memory	Parent MDL completed while a partial was still in flight
System degrades, working-set errors	Unbounded or leaked locked pages

Symptom	Cause
Read-only page corrupted	Locked with `IoReadAccess`, then written

Check yourself

- ✓ Describe an MDL's contents and name the accessor for each field.
- ✓ Explain why `ByteOffset` exists and what ignoring it breaks.
- ✓ List the six problems MDLs solve.
- ✓ Write the full allocate-lock-map-unmap-unlock-free sequence with correct error unwinding.
- ✓ Explain the access argument as a security control.
- ✓ Say who owns an IRP's MDL and what happens if you free it.
- ✓ Map a kernel buffer into user space and list the four rules for doing it safely.
- ✓ Explain partial MDLs and the ownership rule they impose.
- ✓ Walk an MDL chain and say what breaks if you do not.
- ✓ Decode `MdlFlags` in WinDbg and say what a missing `PAGES_LOCKED` bit means.

Synchronisation primitives

Windows offers a dozen locks, and choosing between them is decided almost entirely by one question: what IRQL will touch this data? Get that right and the rest follows mechanically.

The decision

```
 What is the HIGHEST IRQL at which this data is touched?
      |
      +-- DIRQL (an ISR touches it)
      |      -> the INTERRUPT SPINLOCK
      |         WdfInterruptAcquireLock / KeAcquireInterruptSpinLock
      |
      +-- DISPATCH_LEVEL (a DPC or timer touches it)
      |      -> SPINLOCK
      |         KeAcquireSpinLock / WdfSpinLockAcquire
      |
      +-- PASSIVE / APC only
             |
             +-- do I need to WAIT for something?
             |      -> KEVENT
             |
             +-- many readers, rare writers, long holds?
             |      -> ERESOURCE
             |
             +-- just mutual exclusion, short?
             |      -> FAST MUTEX
             |
             +-- a single counter or flag?
                    -> INTERLOCKED operations, no lock at all

AND THE OVERRIDING RULE: everything forbidden at
DISPATCH_LEVEL (chapter 02) is forbidden INSIDE A SPINLOCK,
because acquiring one raises you to DISPATCH_LEVEL. No
paged memory, no blocking, no paged-pool allocation, no
waiting.

This is the same shape as the Linux decision
(Linux ch. 11), with IRQL replacing "context".
```

Spinlocks

```
#include <ntddk.h>

        KSPIN_LOCK Lock;
        KeInitializeSpinLock(&Lock);

/* From PASSIVE or APC level - raises IRQL and saves the old */
        KIRQL oldIrql;

        KeAcquireSpinLock(&Lock, &oldIrql);
        /* ... at DISPATCH_LEVEL now ... */
        KeReleaseSpinLock(&Lock, oldIrql);

/* When you KNOW you are already at DISPATCH_LEVEL - in a DPC,
   a timer callback, or nested inside another spinlock: */
        KeAcquireSpinLockAtDpcLevel(&Lock);
        ...
        KeReleaseSpinLockFromDpcLevel(&Lock);
/* Cheaper: no IRQL change. Calling it below DISPATCH_LEVEL is
   a bug Driver Verifier catches immediately.              */

/* IN-STACK QUEUED - fairer under contention, and preferred
   on many-core systems because it avoids the cacheline
   stampede of a plain spinlock: */
        KLOCK_QUEUE_HANDLE handle;

        KeAcquireInStackQueuedSpinLock(&Lock, &handle);
        ...
        KeReleaseInStackQueuedSpinLock(&handle);

/* THE RULES:
     · SHORT. Microseconds. Other CPUs are spinning.
     · `oldIrql` must be a LOCAL variable. Sharing one between
       CPUs corrupts the IRQL restore - a subtle, devastating
       bug.
     · never acquire a lower-IRQL lock while holding a
       higher-IRQL one
     · define and document a lock ORDER, exactly as on Linux
     · a spinlock protects against OTHER CPUs; raising IRQL
       protects against PREEMPTION on THIS one. You almost
       always need both, which is why Windows combines them. */

/* IN WDF, an object with proper lifetime management: */
        WDFSPINLOCK lock;
        WDF_OBJECT_ATTRIBUTES attr;

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ParentObject = device;      /* freed with it */
        status = WdfSpinLockCreate(&attr, &lock);

        WdfSpinLockAcquire(lock);
        ...
        WdfSpinLockRelease(lock);
```

Interrupt spinlocks

```
/* Data shared with an ISR needs a lock acquired at the
   DEVICE's IRQL - otherwise the ISR can interrupt you while
   you hold it, and spin forever waiting for code that cannot
   resume. Exactly the Linux self-deadlock (ch. 11).       */

/* WDM: */
        KIRQL oldIrql;

        KeAcquireInterruptSpinLock(interruptObject, &oldIrql);
        /* now at DIRQL - the ISR cannot run on this CPU */
        ...
        KeReleaseInterruptSpinLock(interruptObject, oldIrql);

/* WDF - simpler, and it knows the right IRQL: */
        WdfInterruptAcquireLock(interrupt);
        ...
        WdfInterruptReleaseLock(interrupt);

/* INSIDE the ISR itself, you already hold it - WDF acquires
   it before calling EvtInterruptIsr. Do not re-acquire.

/* OR, AVOID THE PROBLEM: use
   WdfInterruptSynchronize to run a small function AT DIRQL
   with the lock held, rather than holding it around a longer
   section:                                                 */

static BOOLEAN
MySyncRoutine(_In_ WDFINTERRUPT Interrupt, _In_ WDFCONTEXT Ctx)
{
        PMY_CONTEXT c = (PMY_CONTEXT)Ctx;

        /* runs at DIRQL with the interrupt lock held */
        c->Enabled = TRUE;
        WRITE_REGISTER_ULONG(c->Base + REG_INT_EN, 1);
        return TRUE;
}

        WdfInterruptSynchronize(interrupt, MySyncRoutine, ctx);

/* This is the idiomatic way to touch ISR-shared state from a
   normal path: a tiny function, at DIRQL, for the shortest
   possible time.                                           */
```

Events

```
/* KEVENT is the "wait for something to happen" primitive -
   the analogue of a Linux completion or wait queue.       */

        KEVENT Event;

        KeInitializeEvent(&Event,
                          NotificationEvent,  /* or Synchronization */
                          FALSE);             /* initial state */

/* NotificationEvent    stays signalled until manually reset.
                        ALL waiters wake. ("manual reset")
   SynchronizationEvent auto-resets when ONE waiter wakes.
                        ("auto reset")                       */

/* WAITING - at PASSIVE_LEVEL only: */
        LARGE_INTEGER timeout;
        NTSTATUS status;

        timeout.QuadPart = -10LL * 1000 * 1000 * 5;  /* 5 s */
        /* NEGATIVE = relative. POSITIVE = an absolute time in
           1601-epoch 100ns units. Getting the sign wrong
           gives you either an instant return or a wait until
           the far future - both common bugs.            */

        status = KeWaitForSingleObject(&Event,
                                       Executive,
                                       KernelMode,
                                       FALSE,     /* alertable */
                                       &timeout);
        if (status == STATUS_TIMEOUT)
                return STATUS_IO_TIMEOUT;

/* SIGNALLING - legal at any IRQL <= DISPATCH_LEVEL, which is
   why a DPC can wake a waiting thread: */
        KeSetEvent(&Event, IO_NO_INCREMENT, FALSE);
        KeClearEvent(&Event);
        KeResetEvent(&Event);        /* clear, return old state */

/* WAITING ON SEVERAL: */
        PVOID objects[2] = { &EventA, &EventB };

        status = KeWaitForMultipleObjects(2, objects,
                        WaitAny,        /* or WaitAll */
                        Executive, KernelMode, FALSE,
                        &timeout, NULL);
        /* returns STATUS_WAIT_0 or STATUS_WAIT_1 for WaitAny */

/* ALWAYS USE A TIMEOUT in a driver. An infinite wait on
   hardware that has stopped responding hangs the thread
   permanently, and if it is a system thread, it hangs
   shutdown. Same argument as the Linux
   wait_for_completion_timeout rule (ch. 11).             */
```

Fast mutexes and ERESOURCE

```
/* FAST MUTEX - simple exclusion at PASSIVE_LEVEL: */
        FAST_MUTEX Mutex;

        ExInitializeFastMutex(&Mutex);

        ExAcquireFastMutex(&Mutex);
        /* raises IRQL to APC_LEVEL - so APCs are blocked,
           which is what makes it "fast"                    */
        ...
        ExReleaseFastMutex(&Mutex);

        if (ExTryToAcquireFastMutex(&Mutex)) { ... }

/* NOT RECURSIVE. Acquiring twice on one thread deadlocks.
   NOT usable above APC_LEVEL.
   You MAY sleep while holding one - which is the whole point
   compared to a spinlock.                                  */

/* ERESOURCE - reader/writer, recursive, PASSIVE only: */
        ERESOURCE Resource;

        ExInitializeResourceLite(&Resource);

        KeEnterCriticalRegion();     /* disable normal APCs -
                                        REQUIRED, and forgetting
                                        it is the classic bug */
        ExAcquireResourceSharedLite(&Resource, TRUE);
        /* ... many readers concurrently ... */
        ExReleaseResourceLite(&Resource);
        KeLeaveCriticalRegion();

        KeEnterCriticalRegion();
        ExAcquireResourceExclusiveLite(&Resource, TRUE);
        /* ... one writer ... */
        ExReleaseResourceLite(&Resource);
        KeLeaveCriticalRegion();

        ExDeleteResourceLite(&Resource);

/* The TRUE argument means "wait". FALSE means "try", and
   returns FALSE if it would block.

   THE CRITICAL REGION requirement exists because a normal
   kernel APC could suspend the thread while it holds the
   resource, blocking every other waiter indefinitely. The
   Ex*ResourceLite functions ASSERT you are in one on checked
   builds.

   USE ERESOURCE when reads genuinely dominate AND the hold
   time is long. Otherwise a fast mutex is cheaper - the same
   advice as Linux's on rw_semaphores (ch. 11).          */

/* WDF WAIT LOCK - the framework's PASSIVE-level lock, and the
   one to reach for in WDF code: */
        WDFWAITLOCK lock;

        WdfWaitLockCreate(&attr, &lock);
        WdfWaitLockAcquire(lock, NULL);   /* NULL = wait
                                             forever; or a
                                             timeout          */
```

```
        ...
        WdfWaitLockRelease(lock);
```

Interlocked operations

```
/* No lock at all, for a single value. */

        LONG counter = 0;

        InterlockedIncrement(&counter);
        InterlockedDecrement(&counter);
        InterlockedExchange(&counter, 5);
        InterlockedExchangeAdd(&counter, 10);
        old = InterlockedCompareExchange(&counter, newVal,
                                         expected);

        /* 64-bit and pointer-sized variants: */
        InterlockedIncrement64(&counter64);
        InterlockedExchangePointer(&ptr, newPtr);
        InterlockedCompareExchangePointer(&ptr, new, expected);

        /* bit operations: */
        InterlockedBitTestAndSet(&flags, 3);
        InterlockedOr(&flags, MY_FLAG);
        InterlockedAnd(&flags, ~MY_FLAG);

/* INTERLOCKED LISTS - a lock-free singly linked stack, used
   heavily for free lists: */
        SLIST_HEADER ListHead;

        ExInitializeSListHead(&ListHead);
        ExInterlockedPushEntrySList(&ListHead, &entry->Link,
                                    NULL);
        entry = ExInterlockedPopEntrySList(&ListHead, NULL);

/* USABLE AT ANY IRQL. No lock ordering to worry about.

   THE LIMIT, same as Linux atomics: they protect ONE
   variable. If your invariant spans two - "the count matches
   the list length" - interlocked operations give you nothing.
   You need a lock.                                          */

/* REFERENCE COUNTING done right: */
        if (InterlockedDecrement(&obj->RefCount) == 0)
                FreeObject(obj);
/* And the acquire side must never increment from zero -
   that is a use-after-free. Structure the code so a
   reference can only be taken from something that already
   holds one.                                               */
```

THE COMPARISON TABLE

Primitive	Max IRQL	Can wait	Recursive	Use for
Interlocked	Any	—	—	One counter or flag
Interrupt spinlock	DIRQL	No	No	Data shared with an ISR
Spinlock	DISPATCH	No	No	Data shared with a DPC
Fast mutex	APC	Yes	**No**	General PASSIVE exclusion
WDFWAITLOCK	PASSIVE	Yes	No	The WDF default
ERESOURCE	PASSIVE	Yes	**Yes**	Long reads, rare writes
KEVENT	Wait: PASSIVE Set: DISPATCH	Yes	—	"Wait until this happens"
KSEMAPHORE	Same	Yes	—	Counted resources
Rundown protection	DISPATCH	Yes	—	"Wait for all users to leave"

Rundown protection

```
/* The problem: you are tearing down an object, and other
   threads may be using it right now. You must wait for them
   all to finish, and prevent new ones from starting.
   Doing this with a refcount and an event is subtly racy;
   EX_RUNDOWN_REF does it correctly.                        */

        EX_RUNDOWN_REF RunRef;

        ExInitializeRundownProtection(&RunRef);

/* Every user, before touching the object: */
        if (!ExAcquireRundownProtection(&RunRef))
                return STATUS_DEVICE_REMOVED;   /* rundown has
                                                   started    */
        ... use the object ...
        ExReleaseRundownProtection(&RunRef);

/* The teardown path, once: */
        ExWaitForRundownProtectionRelease(&RunRef);
        /* returns when every acquirer has released, AND all
           future acquires fail. NOW it is safe to free.    */

        ... free the object ...

/* THIS IS EXACTLY the problem Linux solves with module
   refcounts and _sync cancellation (ch. 13/15). Windows gives
   it a name and a correct implementation.

   USE IT for anything with a lifetime shorter than the
   driver: a per-file context, a session, a queued operation
   the user can cancel.                                     */
```

Deadlock and how it is found

```
/* THE FOUR CLASSIC WAYS, same as Linux (ch. 11):
     1. AB-BA lock ordering
     2. self-deadlock on a non-recursive lock
     3. an ISR taking a lock its interrupted code holds
     4. waiting at DISPATCH_LEVEL

/* THE FIX FOR (1) IS ALWAYS THE SAME: a documented order. */

/*
 * Lock order:
 *      device->ConfigLock  (WDFWAITLOCK)
 *        -> device->QueueLock  (WDFSPINLOCK)
 *          -> interrupt lock
 * Never acquire in the reverse direction.
 */

/* AND LET DRIVER VERIFIER CHECK IT: */
        verifier /flags 0x20 /driver mydriver.sys
        /* 0x20 = deadlock detection */

/* Verifier's deadlock detector builds a lock ORDER GRAPH at
   runtime and bugchecks with 0xC4 (subcode 0x1000-series) the
   first time it sees an inconsistent acquisition - even if no
   deadlock actually occurred. This is the direct equivalent
   of Linux's lockdep (ch. 11), and it finds the same class of
   latent bug.

   ENABLE IT DURING DEVELOPMENT, ALWAYS. It is the single most
   valuable verifier flag for concurrency.               */

/* DEBUGGING A HANG in WinDbg (chapter 22): */
        !locks                  /* every ERESOURCE and its
                                   owner                    */
        !deadlock               /* verifier's graph */
        !stacks 2               /* every thread's stack */
        !thread ffff8a0c...     /* what is this one waiting on */
        !process 0 7            /* everything, verbose */
```

How it shows up

Symptom	Cause
Bugcheck 0xA in a spinlock section	Touched paged memory at DISPATCH_LEVEL
One CPU spins forever	ISR took a lock held by the code it interrupted — use the interrupt spinlock
Whole system hangs	AB-BA deadlock, or waiting while holding a spinlock
Bugcheck 0xC4 with a deadlock subcode	Verifier found an inconsistent lock order — a real bug
Random corruption after a lock release	`oldIrql` shared rather than local
Thread waits forever	Infinite timeout on hardware that stopped responding
Wait returns immediately	Timeout sign wrong — negative is relative
Deadlock on a fast mutex	Acquired recursively; it is not recursive
ERESOURCE assertion on a checked build	Missing `KeEnterCriticalRegion`
Counter drifts under load	Non-interlocked read-modify-write

Symptom	Cause
Use-after-free during teardown	No rundown protection — a user was still inside
Works on one core, fails on many	It was always racy; low core count hid it

Check yourself

- ✓ Walk the decision tree for four different pieces of driver data.
- ✓ Explain why acquiring a spinlock imposes every DISPATCH_LEVEL restriction.
- ✓ Say why data shared with an ISR needs an interrupt spinlock, and what `WdfInterruptSynchronize` is for.
- ✓ Distinguish notification from synchronisation events.
- ✓ Explain the sign convention for timeouts and why every driver wait needs one.
- ✓ Say why a fast mutex must not be acquired recursively.
- ✓ Explain what `KeEnterCriticalRegion` protects against.
- ✓ Use interlocked operations correctly, and state their limit.
- ✓ Explain what rundown protection solves and when to use it.
- ✓ Enable Verifier's deadlock detection and explain what it corresponds to on Linux.

ISRs, DPCs and work items

Windows splits interrupt handling into three tiers by IRQL: a tiny ISR at device level, a DPC at dispatch level, and a work item at passive level. Knowing which tier each piece of work belongs to is the core skill of interrupt-driven driver design.

The three tiers

```
  device raises its interrupt
       |
       v
  ISR                              at DIRQL
    · acknowledge the hardware, stop it re-asserting
    · read the minimum state you need
    · queue a DPC
    · RETURN. Target: under a few microseconds.
    · CANNOT: touch paged memory, allocate paged pool,
      wait, block, call most APIs
       |
       v
  DPC                              at DISPATCH_LEVEL
    · the real work: parse data, complete requests, start
      the next transfer
    · CAN: complete IRPs, use spinlocks, allocate non-paged
      pool, queue work items
    · CANNOT: touch paged memory, wait, block
    · Target: tens of microseconds. Over ~100us and you are
      hurting system latency; the DPC watchdog fires at
      around 20ms.
       |
       v
  WORK ITEM                        at PASSIVE_LEVEL
    · anything that must sleep: registry access, file I/O,
      waiting on an event, sending a request to another
      driver, allocating paged pool
    · runs on a system worker thread, so it may be delayed
       |
       v
  done

COMPARE LINUX (ch. 13):
   ISR        = the hard IRQ handler
   DPC        = a tasklet or softirq
   work item  = a workqueue item
   and WDF's passive-level interrupts = a threaded IRQ

THE SAME DESIGN PRESSURE produces the same three tiers in
both systems: something must be fast and constrained,
something must be deferred but prompt, and something must be
able to sleep.
```

The ISR

```
BOOLEAN
MyEvtInterruptIsr(_In_ WDFINTERRUPT Interrupt,
                  _In_ ULONG MessageID)
{
        PMY_CONTEXT ctx = MyInterruptGetContext(Interrupt);
        ULONG status;

        UNREFERENCED_PARAMETER(MessageID);

        /* WDF has ALREADY acquired the interrupt spinlock
           before calling us (chapter 09).                  */

        status = READ_REGISTER_ULONG(
                        (PULONG)(ctx->RegBase + REG_INT_STATUS));

        /* IS IT OURS? On a shared line this is mandatory -
           returning TRUE for someone else's interrupt breaks
           their driver.                                     */
        if ((status & INT_PENDING) == 0)
                return FALSE;

        /* ACKNOWLEDGE FIRST, so the device stops asserting */
        WRITE_REGISTER_ULONG(
                (PULONG)(ctx->RegBase + REG_INT_CLEAR), status);

        /* hand the minimum state to the DPC */
        ctx->LastStatus = status;

        /* queue the DPC and return */
        WdfInterruptQueueDpcForIsr(Interrupt);

        return TRUE;
}

/* THE RETURN VALUE:
     TRUE   this interrupt was mine and I handled it
     FALSE  not mine - let the next handler on the shared
            line try

   IF EVERY HANDLER RETURNS FALSE repeatedly, Windows
   eventually declares the interrupt unhandled and the system
   may bugcheck with 0x7A or hang in an interrupt storm - the
   equivalent of Linux's "nobody cared" (ch. 14).

/* WHAT NOT TO DO IN AN ISR:
     · a slow MMIO read (a PCIe config read can take
       microseconds - do it in the DPC)
     · a loop of unknown length
     · DbgPrint on a serial console (milliseconds!)
     · anything touching paged memory
     · acquiring any lock other than the interrupt lock     */
```

The DPC

```
VOID
MyEvtInterruptDpc(_In_ WDFINTERRUPT Interrupt,
                  _In_ WDFOBJECT AssociatedObject)
{
        PMY_CONTEXT ctx = MyInterruptGetContext(Interrupt);
        WDFREQUEST request;
        ULONG status;
        NTSTATUS s;

        UNREFERENCED_PARAMETER(AssociatedObject);

        /* read the state the ISR captured, under the
           interrupt lock (chapter 09) */
        WdfInterruptAcquireLock(Interrupt);
        status = ctx->LastStatus;
        ctx->LastStatus = 0;
        WdfInterruptReleaseLock(Interrupt);

        if (status & INT_ERROR) {
                ctx->Errors++;
                MyHandleError(ctx, status);
        }

        if (status & INT_DATA_READY) {
                /* complete a pending request (chapter 15) */
                s = WdfIoQueueRetrieveNextRequest(ctx->ReadQueue,
                                                  &request);
                if (NT_SUCCESS(s)) {
                        MyFillRequest(ctx, request);
                        WdfRequestCompleteWithInformation(
                                request, STATUS_SUCCESS, len);
                }
        }

        /* need PASSIVE_LEVEL? queue a work item */
        if (status & INT_NEEDS_RESET)
                WdfWorkItemEnqueue(ctx->ResetWorkItem);
}

/* DPC PROPERTIES worth knowing:

   · ONE DPC OBJECT runs at most once at a time, system-wide.
     Queueing it again while it is queued is a no-op - so a
     burst of interrupts produces ONE DPC run, not N. Your DPC
     must therefore handle "several things happened" in one
     pass, not assume a one-to-one mapping with interrupts.

   · It runs on the CPU that queued it, by default.
     WdfDpcCreate lets you request otherwise; on a
     multi-queue device, spreading DPCs across CPUs is how you
     scale.

   · It can be interrupted by any DIRQL interrupt, including
     your own ISR - so shared state still needs the interrupt
     lock.

   · DPCs from all drivers share one per-CPU queue. A long DPC
     delays everyone's. This is the single biggest source of
     system-wide latency spikes on Windows, and it is the
     direct analogue of the Linux argument for keeping
```

```
     softirqs short (ch. 10).                                */

/* A STANDALONE DPC, not tied to an interrupt: */
        WDF_DPC_CONFIG dpcConfig;
        WDF_OBJECT_ATTRIBUTES attr;

        WDF_DPC_CONFIG_INIT(&dpcConfig, MyDpcFunc);
        dpcConfig.AutomaticSerialization = TRUE;

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ParentObject = device;

        status = WdfDpcCreate(&dpcConfig, &attr, &ctx->Dpc);
        ...
        WdfDpcEnqueue(ctx->Dpc);

        /* teardown - WAIT for it, like Linux's _sync forms */
        WdfDpcCancel(ctx->Dpc, TRUE);   /* TRUE = wait */
```

THE DPC WATCHDOG

Bugcheck `0x133 DPC_WATCHDOG_VIOLATION` fires when a single DPC runs for roughly 20 ms, or when the system stays at DISPATCH_LEVEL for around 10 seconds. It exists because a long DPC is invisible in normal testing but destroys interactive latency in the field. If you hit it, the fix is never to raise the timeout — it is to move work into a work item or split it across several DPC invocations.

Work items

```
/* For anything needing PASSIVE_LEVEL. */

VOID
MyEvtWorkItem(_In_ WDFWORKITEM WorkItem)
{
        PMY_WORKITEM_CONTEXT wc = MyWorkItemGetContext(WorkItem);
        PMY_CONTEXT ctx = wc->DeviceContext;

        PAGED_CODE();     /* asserts PASSIVE_LEVEL */

        /* we may sleep here: registry, files, waiting */
        MyResetHardwareSlowly(ctx);
        MyReadConfigFromRegistry(ctx);
}

/* creating one, in EvtDeviceAdd: */
        WDF_WORKITEM_CONFIG workConfig;
        WDF_OBJECT_ATTRIBUTES attr;

        WDF_WORKITEM_CONFIG_INIT(&workConfig, MyEvtWorkItem);

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attr,
                        MY_WORKITEM_CONTEXT);
        attr.ParentObject = device;

        status = WdfWorkItemCreate(&workConfig, &attr,
                                   &ctx->WorkItem);

        wc = MyWorkItemGetContext(ctx->WorkItem);
        wc->DeviceContext = ctx;

/* enqueue from a DPC or anywhere: */
        WdfWorkItemEnqueue(ctx->WorkItem);

/* teardown: */
        WdfWorkItemFlush(ctx->WorkItem);   /* WAIT for it */

/* THE SAME COALESCING RULE as DPCs: enqueueing an already
   queued work item is a no-op. If each event needs its own
   handling, either create several work items, or maintain
   your own queue of pending events and have one work item
   drain it.

   THAT SECOND PATTERN is usually the right one:
        DPC: push an event onto a list, enqueue the work item
        work item: drain the list until empty
   It handles bursts correctly and needs only one work item. */

/* SYSTEM WORKER THREADS are a shared, limited resource. A
   work item that blocks for a long time occupies one. Do not
   use work items for long-running loops - create a dedicated
   system thread instead (chapter 11).                    */
```

Passive-level interrupts

```
/* Some devices are on a slow bus - I2C, SPI, or behind a
   USB transfer - where you CANNOT read the interrupt status
   register without sleeping. A DIRQL ISR cannot do that.

   WDF's answer: a passive-level interrupt.                */

        WDF_INTERRUPT_CONFIG config;

        WDF_INTERRUPT_CONFIG_INIT(&config,
                                  MyEvtInterruptIsr,
                                  MyEvtInterruptDpc);

        config.PassiveHandling = TRUE;    /* <-- the ISR runs
                                             at PASSIVE_LEVEL */
        config.AutomaticSerialization = FALSE;   /* required
                                                    with
                                                    passive   */

        status = WdfInterruptCreate(device, &config, &attr,
                                    &ctx->Interrupt);

/* NOW EvtInterruptIsr runs at PASSIVE_LEVEL, on a dedicated
   thread, and may sleep - so it can do an I2C read to find
   out what happened.

   This is EXACTLY Linux's threaded IRQ (ch. 13), including
   the reason it exists.

   CONSEQUENCES:
     · higher latency - a thread must be scheduled
     · the interrupt stays masked until your handler returns,
       so it is inherently one-shot
     · use WdfInterruptAcquireLock, which becomes a
       passive-level lock, not a spinlock
     · you may not need a DPC at all - do the work in the ISR

   USE IT for I2C/SPI/GPIO-connected devices, which on modern
   ARM64 Windows systems is most of them.                  */
```

Timers

```
/* WDFTIMER - the callback runs at DISPATCH_LEVEL by default. */

        WDF_TIMER_CONFIG timerConfig;

        WDF_TIMER_CONFIG_INIT_PERIODIC(&timerConfig,
                                       MyEvtTimer,
                                       100);   /* ms period */
        /* or one-shot: */
        WDF_TIMER_CONFIG_INIT(&timerConfig, MyEvtTimer);

        timerConfig.AutomaticSerialization = TRUE;
        timerConfig.TolerableDelay = 50;   /* ms - lets the
                                              system coalesce
                                              timers and save
                                              power           */

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ParentObject = device;

        status = WdfTimerCreate(&timerConfig, &attr,
                                &ctx->Timer);

        WdfTimerStart(ctx->Timer,
                      WDF_REL_TIMEOUT_IN_MS(100));

        WdfTimerStop(ctx->Timer, TRUE);   /* TRUE = WAIT for
                                             any running
                                             callback         */

VOID
MyEvtTimer(_In_ WDFTIMER Timer)
{
        WDFDEVICE device = WdfTimerGetParentObject(Timer);
        PMY_CONTEXT ctx = MyGetContext(device);

        /* DISPATCH_LEVEL - cannot sleep, cannot touch paged
           memory                                            */
        MyPollHardware(ctx);
}

/* FOR A PASSIVE-LEVEL TIMER CALLBACK: */
        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ExecutionLevel = WdfExecutionLevelPassive;

/* SHORT DELAYS without a timer: */
        KeStallExecutionProcessor(50);    /* BUSY-WAITS, in
                                             microseconds. Any
                                             IRQL. Keep it
                                             under ~50us.    */

        LARGE_INTEGER interval;
        interval.QuadPart = -10LL * 1000 * 20;   /* 20 ms */
        KeDelayExecutionThread(KernelMode, FALSE, &interval);
        /* SLEEPS. PASSIVE_LEVEL only. */

/* KeStallExecutionProcessor is Linux's udelay(); the delay
   thread is msleep(). The same advice applies: never busy-wait
   for milliseconds.                                          */
```

Teardown ordering

```
/* THE SAME RULE AS LINUX (ch. 13): stop the source first,
   then wait for what is already in flight.               */

NTSTATUS
MyD0Exit(_In_ WDFDEVICE Device,
         _In_ WDF_POWER_DEVICE_STATE TargetState)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        /* 1. stop the hardware generating interrupts.
              WDF disconnects the interrupt for us AFTER this
              callback, but the device must stop asserting
              first or we take a spurious one.            */
        WRITE_REGISTER_ULONG(
                (PULONG)(ctx->RegBase + REG_INT_ENABLE), 0);

        /* 2. wait for anything already queued */
        WdfDpcCancel(ctx->Dpc, TRUE);        /* TRUE = wait */
        WdfWorkItemFlush(ctx->WorkItem);
        WdfTimerStop(ctx->Timer, TRUE);      /* TRUE = wait */

        /* 3. now nothing can touch our state */
        return STATUS_SUCCESS;
}

/* THE "TRUE" ARGUMENTS ARE NOT OPTIONAL. Without them these
   calls return immediately, the callback may still be
   running, and it will touch your context after
   EvtDeviceReleaseHardware has unmapped the registers -
   producing a bugcheck at an address that no longer maps.

   This is the exact Windows counterpart of forgetting
   cancel_work_sync() on Linux, with the same symptom and the
   same fix.                                             */
```

How it shows up

Symptom	Cause
Bugcheck 0x133 DPC_WATCHDOG_VIOLATION	A DPC doing far too much — move work to a work item
Bugcheck 0xA inside a DPC	Paged memory or paged pool at DISPATCH_LEVEL
Interrupt storm, system unresponsive	ISR not acknowledging the device
Another device stops working	Your ISR returned TRUE for an interrupt that was not yours
Bugcheck 0x7A or a hang	Nobody claimed the interrupt
Events lost during a burst	Assumed one DPC per interrupt — DPCs coalesce
Work items lost during a burst	Same coalescing rule — use a list plus one drainer
Bugcheck on device removal or sleep	DPC, timer or work item not cancelled with wait
System-wide latency spikes	A long DPC delaying everyone's
Cannot read the status register in the ISR	Slow bus — use `PassiveHandling`
Work item never runs	Enqueued while already queued, or the worker pool is exhausted
High CPU with nothing happening	`KeStallExecutionProcessor` for too long

Check yourself

- ✓ Name the three tiers, their IRQLs, and what each may and may not do.
- ✓ Map them onto Linux's equivalents.
- ✓ Write an ISR and explain both return values.
- ✓ Explain what happens if every handler on a shared line returns FALSE.
- ✓ Explain DPC coalescing and how to write a DPC that handles bursts.
- ✓ Say what bugcheck 0x133 means and what the correct fix is.
- ✓ Explain why a long DPC harms the whole system.
- ✓ Use a work item, and give the pattern for handling many events with one.
- ✓ Say when a passive-level interrupt is needed and what it costs.
- ✓ Write the teardown sequence and explain why the wait arguments are mandatory.

Threads and scheduling

System threads, thread priorities, APCs, and the context rules that decide whether a user-mode address means anything to your code. Shorter than the Linux equivalent, because most drivers should not create threads at all — but you need to know when they should.

Priorities

```
31  ┐
...  │  REAL-TIME range (16-31)
16  ┘  Fixed. NEVER boosted, never decayed.
       A runaway thread here starves everything below.

15  ┐
...  │  DYNAMIC range (1-15)
 1  ┘  Boosted on I/O completion, decayed over time.
 0     the zero-page thread only

THREAD PRIORITY = process priority class + thread relative
priority, for user threads. Kernel threads set an absolute
value.

FOR DRIVER THREADS the useful constants:

  LOW_REALTIME_PRIORITY     16
  HIGH_PRIORITY             31
  LOW_PRIORITY               0
  THREAD_PRIORITY_NORMAL     8   (dynamic)

A driver's worker thread usually wants LOW_REALTIME_PRIORITY
(16) if it services hardware with timing requirements, or
simply the default 8 if it does background work.

THE RULE: raising priority is almost never the fix for a
latency problem. If a driver thread is not running promptly
the usual causes are a long DPC somewhere (chapter 10), a
lock it is waiting on, or too much work in the wrong tier.
Raising priority moves the problem onto someone else.
```

System threads

```
#include <ntddk.h>

VOID
MyThreadRoutine(_In_ PVOID Context)
{
        PMY_CONTEXT ctx = (PMY_CONTEXT)Context;
        NTSTATUS status;

        /* PASSIVE_LEVEL. May sleep. No user address space -
           we are in the SYSTEM process.                    */

        KeSetPriorityThread(KeGetCurrentThread(),
                            LOW_REALTIME_PRIORITY);

        for (;;) {
                status = KeWaitForSingleObject(&ctx->WorkEvent,
                                Executive, KernelMode, FALSE,
                                NULL);   /* or a timeout */

                if (ctx->Stopping)
                        break;

                MyDoLongRunningWork(ctx);
        }

        PsTerminateSystemThread(STATUS_SUCCESS);
}

/* CREATING one: */
        HANDLE threadHandle;
        NTSTATUS status;

        status = PsCreateSystemThread(&threadHandle,
                                      THREAD_ALL_ACCESS,
                                      NULL,     /* attributes */
                                      NULL,     /* process:
                                                   NULL = the
                                                   system one  */
                                      NULL,     /* client id  */
                                      MyThreadRoutine,
                                      ctx);
        if (!NT_SUCCESS(status))
                return status;

        /* Take a REFERENCE so we can wait for it later, then
           close the handle - we do not need it. */
        status = ObReferenceObjectByHandle(threadHandle,
                        THREAD_ALL_ACCESS, *PsThreadType,
                        KernelMode, &ctx->ThreadObject, NULL);
        ZwClose(threadHandle);

/* STOPPING it - and this must be exactly right: */
        ctx->Stopping = TRUE;
        KeSetEvent(&ctx->WorkEvent, IO_NO_INCREMENT, FALSE);

        KeWaitForSingleObject(ctx->ThreadObject, Executive,
                              KernelMode, FALSE, NULL);
        /* the thread has now RETURNED - it is safe to free its
           context and to unload                              */
```

```
        ObDereferenceObject(ctx->ThreadObject);

/* WITHOUT THAT WAIT, the driver can unload while the thread
   is still executing its code, and the next instruction it
   runs is in freed memory. Exactly the Linux kthread_stop()
   requirement (ch. 09).                                    */
```

> **WHEN TO CREATE A THREAD — RARELY**
>
> Use a **work item** (chapter 10) for short deferred work; it shares the system worker pool and costs nothing when idle. Create a dedicated system thread only when you need a long-lived loop that blocks — a polling loop, a thread servicing a hardware queue, a watchdog. A thread per device that mostly sleeps is wasted memory and a scheduler entry; a work item that blocks for seconds starves the shared pool. Match the tool to the duration.

Thread and process context

```
/* WHICH PROCESS AM I IN? It determines whether a user-mode
   address means anything.                                    */

        PEPROCESS process = IoGetCurrentProcess();
        PETHREAD  thread  = PsGetCurrentThread();
        HANDLE    pid     = PsGetCurrentProcessId();
        HANDLE    tid     = PsGetCurrentThreadId();

/* THE THREE CONTEXTS a driver routine can run in:

   1. THE CALLER'S CONTEXT
        A dispatch routine called synchronously from a
        syscall. `IoGetCurrentProcess()` is the requesting
        process, and user-mode addresses are valid.
        -> only reliable for synchronous, non-queued requests

   2. AN ARBITRARY THREAD CONTEXT
        A DPC, a completion routine, a timer callback. You are
        on whatever thread happened to be interrupted. User
        addresses are MEANINGLESS.
        -> this is why METHOD_BUFFERED and MDLs exist
           (chapter 07)

   3. A SYSTEM THREAD CONTEXT
        Your own thread, or a work item. In the System
        process. No user address space at all.

/* THE ONE-LINE RULE: unless you are certain you are in case
   1, never touch a user-mode address. Use SystemBuffer or a
   mapped MDL, which are valid everywhere.

/* IF YOU MUST reach a specific process's address space: */
        KAPC_STATE apcState;

        KeStackAttachProcess(targetProcess, &apcState);
        /* now user addresses in targetProcess are valid */
        __try {
                ProbeForRead(userVa, len, 1);
                RtlCopyMemory(local, userVa, len);
        } __except (EXCEPTION_EXECUTE_HANDLER) {
                status = GetExceptionCode();
        }
        KeUnstackDetachProcess(&apcState);

/* ATTACHING IS EXPENSIVE and constrained: PASSIVE_LEVEL only,
   no nesting across other attaches, and you must detach on
   every path. Treat it as a last resort - the existence of
   MDLs means you almost never need it.                   */
```

APCs

```
/* An Asynchronous Procedure Call runs a function IN THE
   CONTEXT OF A SPECIFIC THREAD. It is the mechanism behind
   asynchronous I/O completion, thread suspension, and
   termination.                                          */

   KERNEL APCs      run at APC_LEVEL, in the target thread.
                    Used by the I/O manager to copy results
                    back into the caller's buffer at
                    completion - which is why METHOD_BUFFERED
                    output lands in the right process without
                    you doing anything.

   SPECIAL KERNEL APCs  a higher-priority variant, used for
                    I/O completion and thread termination.
                    Cannot be disabled by a critical region.

   USER APCs        run at PASSIVE_LEVEL in user mode, when
                    the thread enters an ALERTABLE wait.
                    QueueUserAPC, and the completion routine
                    of ReadFileEx.

/* WHY A DRIVER CARES:

   1. KeEnterCriticalRegion() disables normal kernel APCs.
      Required around ERESOURCE (chapter 09), because an APC
      could otherwise suspend a thread holding the resource.

   2. An ALERTABLE wait can be interrupted by a user APC:
        KeWaitForSingleObject(..., TRUE, ...)   /* alertable */
      returns STATUS_USER_APC. Handle it, or use FALSE.

   3. Thread termination is delivered as a special kernel APC.
      A thread that never enters an alertable wait can be hard
      to terminate - which is why driver threads should use an
      explicit stop flag plus an event, as above, rather than
      relying on termination.

/* QUEUEING ONE yourself is possible but rarely right - the
   APC structures are undocumented in detail and the lifetime
   rules are unforgiving. If you think you need an APC, you
   probably want a work item.                            */
```

Cancellation and thread exit

```
/* When a thread exits with I/O outstanding, the I/O manager
   cancels its pending IRPs. Your driver must cooperate or the
   process hangs in termination.                              */

/* THE OBLIGATIONS:
     1. every queued request must have a cancel path
        (chapter 04, or WDF queues in chapter 15)
     2. IRP_MJ_CLEANUP must cancel everything belonging to
        that file object
     3. no wait without a timeout

/* WDF DOES ALL OF THIS if you use queues:
     · WdfIoQueuePurgeSynchronously cancels and waits
     · the framework cancels per-file-object requests on
       cleanup automatically
     · EvtIoCanceledOnQueue lets you clean up device state

/* SEEING A HUNG TERMINATION in WinDbg (chapter 22): */
        !process 0 7               /* find the process */
        !thread ffff8a0c...        /* what is it waiting on */
        !irpfind -v 0 0 device ...  /* which IRP is stuck */

/* THE PATTERN: a process stuck in "Terminated" state with a
   non-zero thread count almost always means a driver holds an
   IRP it never completes and never cancels. The Linux
   equivalent is a task stuck in D state (ch. 09), and it is
   diagnosed the same way: find the request, find the driver
   holding it.                                                */
```

Observing

```
# --- On the machine ---
    Process Explorer           per-thread CPU, priority,
                               start address, stack
    Process Monitor            I/O by thread
    Performance Monitor        Thread \ Priority Current
    xperf / WPA                scheduling traces, wait
                               analysis - the best tool for
                               "why was my thread not running"

# --- In WinDbg ---
    !process 0 0               every process, brief
    !process ffff8a0c... 7     one process, all threads,
                               all stacks
    !thread ffff8a0c...        one thread: priority, state,
                               wait reason, stack
    !ready                     threads ready to run, by
                               priority
    !running                   what each CPU is running now
    !stacks 2                  every thread's stack, filtered
    ~*k                        all stacks in the current
                               process

# WAIT REASONS worth recognising in !thread output:
    Executive        waiting on a dispatcher object
    UserRequest      a user-mode wait
    WrQueue          waiting on a work queue
    Suspended
    WrLpcReceive     an ALPC call

# THE QUESTION !thread ANSWERS: "what is this thread blocked
# on, and who owns it?" That is usually the whole diagnosis
# for a hang.
```

How it shows up

Symptom	Cause
Bugcheck after driver unload	Thread not waited for — `KeWaitForSingleObject` on the thread object is mandatory
Driver will not unload	Thread never exits; the stop flag is not checked, or the event is never signalled
Process stuck in "Terminated"	A driver holds an IRP it never completes or cancels
User buffer is garbage in a DPC	Arbitrary thread context — the address means nothing there
Works synchronously, fails when queued	Same cause; the request completed in a different context
Wait returns `STATUS_USER_APC`	An alertable wait was interrupted — handle it or pass FALSE
System-wide starvation	A real-time-priority driver thread that never blocks
ERESOURCE assertion	Missing `KeEnterCriticalRegion` (ch. 09)
Worker pool exhausted	Work items blocking for a long time — use a dedicated thread
Crash in `KeUnstackDetachProcess`	Attach/detach not paired on every path

Check yourself

✓ Explain the real-time and dynamic priority ranges and why raising priority rarely fixes latency.

- ✓ Create a system thread and stop it correctly, including the reference and the wait.
- ✓ Say when a work item is right and when a dedicated thread is.
- ✓ Name the three contexts a driver routine can run in and which allows user addresses.
- ✓ Explain why `KeStackAttachProcess` is a last resort.
- ✓ Explain what APCs are used for and the three reasons a driver cares.
- ✓ Explain why `IRP_MJ_CLEANUP` matters to process termination.
- ✓ Use `!thread` to determine what a blocked thread is waiting on.
- ✓ Map each of these onto its Linux equivalent.

Driver models: WDM, KMDF and UMDF

Three ways to write a Windows driver, with a clear answer about which to choose. This chapter explains what each is, what WDF actually does for you, and the narrow cases where the older model is still required.

The three models

	WDM	KMDF	UMDF
Runs in	Kernel	Kernel	**User mode**
Introduced	1999	2005	2006 (v2 in 2013)
A bug means	Bugcheck	Bugcheck	**A crashed process**
Lines for a simple driver	~2000	~600	~600
PnP/power state machine	**You write it**	Framework	Framework
Request cancellation	**You write it**	Framework	Framework
Direct hardware access	Yes	Yes	Limited
Interrupts	Yes	Yes	Yes (passive only)
DMA	Yes	Yes	**No**
Debuggable with	WinDbg (kernel)	WinDbg (kernel)	**Visual Studio**
Language	C	C	C++ (COM-based)

THE DECISION, IN ORDER

1. **Can it be UMDF?** If your device needs no DMA, no DIRQL interrupt handling, and no kernel-only API, write it in user mode. A bug becomes a restarted service, not a blue screen — and you debug it in Visual Studio with a normal debugger.
2. **Otherwise, KMDF.** This is the answer for essentially every kernel driver written today.
3. **WDM only if forced.** Some device classes have their own port/miniport models that are WDM-based, and some filter positions are not expressible in WDF. Those are the real exceptions; "we have always used WDM" is not one.

What WDF actually gives you

```
THE WDM DRIVER MUST IMPLEMENT:

  · ~25 PnP minor functions, each with its own forwarding
    and completion rules (chapter 05)
  · a PnP state machine with roughly 15 states, including
    surprise removal, query-remove veto, and resource
    rebalance
  · power IRP handling, correctly serialised, including
    the notorious rule that power IRPs must not be
    completed out of order
  · queue management for pending requests
  · cancellation, with the cancel-routine ownership race
    (chapter 04)
  · request stalling across power transitions
  · per-file-object request tracking, for cleanup
  · reference counting for safe removal
  · synchronisation between all of the above

ALL OF THAT IS THE FRAMEWORK'S JOB IN WDF. What remains
yours:

  · EvtDriverDeviceAdd            create the device
  · EvtDevicePrepareHardware      map registers
  · EvtDeviceD0Entry / D0Exit     power the device
  · EvtDeviceReleaseHardware      unmap
  · EvtIoRead / Write / DeviceControl   the actual work
  · EvtInterruptIsr / Dpc         the interrupt

That is why a KMDF driver is roughly a third the size, and
why the parts that shrink are exactly the parts that were
historically buggy. The framework is not hiding the
hardware from you - it is hiding the OS protocol.
```

The WDF object model in outline

```
/* Everything is a WDF object, with a handle, a parent, an
   optional context, and cleanup callbacks. Covered fully in
   chapter 13; the shape:                                    */

   WDFDRIVER          one per driver
     WDFDEVICE        one per device
       WDFQUEUE       request queues
       WDFINTERRUPT
       WDFDMAENABLER
       WDFTIMER, WDFDPC, WDFWORKITEM
       WDFSPINLOCK, WDFWAITLOCK
       WDFMEMORY
       WDFREQUEST     one per in-flight request
       WDFFILEOBJECT  one per open handle

/* PARENTING IS THE KEY IDEA: destroying a parent destroys its
   children, in order. Set ParentObject and most of your
   cleanup disappears - the Windows counterpart of Linux's
   devm_ (ch. 26).                                           */

        WDF_OBJECT_ATTRIBUTES attr;

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ParentObject = device;      /* dies with it */
        WdfSpinLockCreate(&attr, &lock);
```

A KMDF driver, complete

```
#include <ntddk.h>
#include <wdf.h>

typedef struct _MY_CONTEXT {
        WDFDEVICE     Device;
        WDFQUEUE      DefaultQueue;
        WDFINTERRUPT  Interrupt;
        PUCHAR        RegBase;
        ULONG         RegLength;
        ULONG         LastStatus;
} MY_CONTEXT, *PMY_CONTEXT;

WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(MY_CONTEXT, MyGetContext)
/* generates the accessor MyGetContext(WDFDEVICE) */

DRIVER_INITIALIZE DriverEntry;
EVT_WDF_DRIVER_DEVICE_ADD MyEvtDeviceAdd;
EVT_WDF_DEVICE_PREPARE_HARDWARE MyPrepareHardware;
EVT_WDF_DEVICE_RELEASE_HARDWARE MyReleaseHardware;
EVT_WDF_DEVICE_D0_ENTRY MyD0Entry;
EVT_WDF_DEVICE_D0_EXIT MyD0Exit;
EVT_WDF_IO_QUEUE_IO_READ MyEvtIoRead;
EVT_WDF_IO_QUEUE_IO_DEVICE_CONTROL MyEvtIoDeviceControl;

#pragma alloc_text(INIT, DriverEntry)
#pragma alloc_text(PAGE, MyEvtDeviceAdd)
#pragma alloc_text(PAGE, MyPrepareHardware)
#pragma alloc_text(PAGE, MyReleaseHardware)

/* ---- DriverEntry ------------------------------------- */

NTSTATUS
DriverEntry(_In_ PDRIVER_OBJECT DriverObject,
            _In_ PUNICODE_STRING RegistryPath)
{
        WDF_DRIVER_CONFIG config;
        NTSTATUS status;

        WDF_DRIVER_CONFIG_INIT(&config, MyEvtDeviceAdd);

        status = WdfDriverCreate(DriverObject, RegistryPath,
                                 WDF_NO_OBJECT_ATTRIBUTES,
                                 &config,
                                 WDF_NO_HANDLE);
        if (!NT_SUCCESS(status))
                KdPrint(("WdfDriverCreate failed 0x%x\n", status));

        return status;
}

/* THAT IS THE WHOLE DriverEntry. Compare the WDM version:
   creating device objects, filling a 28-entry MajorFunction
   table, setting DriverUnload, AddDevice, and unwinding all
   of it on failure.                                       */

/* ---- EvtDeviceAdd: once per device ------------------- */

NTSTATUS
MyEvtDeviceAdd(_In_ WDFDRIVER Driver,
               _Inout_ PWDFDEVICE_INIT DeviceInit)
```

```
{
        WDF_PNPPOWER_EVENT_CALLBACKS pnpPower;
        WDF_OBJECT_ATTRIBUTES attributes;
        WDF_IO_QUEUE_CONFIG queueConfig;
        WDF_INTERRUPT_CONFIG interruptConfig;
        WDFDEVICE device;
        PMY_CONTEXT ctx;
        NTSTATUS status;

        UNREFERENCED_PARAMETER(Driver);
        PAGED_CODE();

        /* 1. PnP/power callbacks */
        WDF_PNPPOWER_EVENT_CALLBACKS_INIT(&pnpPower);
        pnpPower.EvtDevicePrepareHardware = MyPrepareHardware;
        pnpPower.EvtDeviceReleaseHardware = MyReleaseHardware;
        pnpPower.EvtDeviceD0Entry         = MyD0Entry;
        pnpPower.EvtDeviceD0Exit          = MyD0Exit;
        WdfDeviceInitSetPnpPowerEventCallbacks(DeviceInit,
                                               &pnpPower);

        /* 2. how user buffers arrive (chapter 07) */
        WdfDeviceInitSetIoType(DeviceInit, WdfDeviceIoBuffered);

        /* 3. who may open it (chapter 03) */
        DECLARE_CONST_UNICODE_STRING(sddl,
                L"D:P(A;;GA;;;SY)(A;;GA;;;BA)");
        WdfDeviceInitAssignSDDLString(DeviceInit, &sddl);

        /* 4. create the device, with our context */
        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attributes,
                                                MY_CONTEXT);
        status = WdfDeviceCreate(&DeviceInit, &attributes,
                                 &device);
        if (!NT_SUCCESS(status))
                return status;

        ctx = MyGetContext(device);
        ctx->Device = device;

        /* 5. publish an interface so user mode can find us */
        status = WdfDeviceCreateDeviceInterface(device,
                        &GUID_DEVINTERFACE_MYDEVICE, NULL);
        if (!NT_SUCCESS(status))
                return status;

        /* 6. a default queue for requests (chapter 15) */
        WDF_IO_QUEUE_CONFIG_INIT_DEFAULT_QUEUE(&queueConfig,
                        WdfIoQueueDispatchParallel);
        queueConfig.EvtIoRead          = MyEvtIoRead;
        queueConfig.EvtIoDeviceControl = MyEvtIoDeviceControl;

        status = WdfIoQueueCreate(device, &queueConfig,
                                  WDF_NO_OBJECT_ATTRIBUTES,
                                  &ctx->DefaultQueue);
        if (!NT_SUCCESS(status))
                return status;

        /* 7. the interrupt (chapter 17) */
        WDF_INTERRUPT_CONFIG_INIT(&interruptConfig,
                                  MyEvtInterruptIsr,
                                  MyEvtInterruptDpc);
        status = WdfInterruptCreate(device, &interruptConfig,
```

```
                                    WDF_NO_OBJECT_ATTRIBUTES,
                                    &ctx->Interrupt);
        return status;
}

/* NOTE: NO CLEANUP CODE ANYWHERE. Every object is parented to
   the device; when the device goes away, they all go with it,
   in reverse order. If EvtDeviceAdd returns a failure, the
   framework destroys the partially-built device for you.

   That is the same guarantee Linux's devm_ gives (ch. 26),
   and it removes the same class of bug.                  */
```

UMDF

```
/* Same object model, same callback names, C++ and COM instead
   of C. A UMDF v2 driver's EvtDeviceAdd looks almost
   identical: */

NTSTATUS
MyEvtDeviceAdd(_In_ WDFDRIVER Driver,
               _Inout_ PWDFDEVICE_INIT DeviceInit)
{
        WDFDEVICE device;
        NTSTATUS status;

        status = WdfDeviceCreate(&DeviceInit,
                                 WDF_NO_OBJECT_ATTRIBUTES,
                                 &device);
        ...
}

/* WHAT UMDF CAN DO:
     · USB devices (through WinUSB)
     · HID devices
     · serial, sensors, biometrics
     · anything reachable through another driver's interface
     · MMIO, on modern versions, with restrictions
     · interrupts, at passive level only

   WHAT IT CANNOT:
     · DMA
     · DIRQL interrupt handling
     · kernel-only APIs
     · being anything other than a leaf or near-leaf in the
       stack

   HOW IT RUNS: the driver is a DLL hosted in
   WUDFHost.exe, a service process. The reflector - a small
   kernel component - forwards IRPs to it.

   THE PAYOFF is large and worth taking seriously:
     · a bug crashes WUDFHost, which RESTARTS. No blue screen,
       no lost user work.
     · debug it in Visual Studio, with breakpoints, watch
       windows and edit-and-continue - not WinDbg on a second
       machine.
     · it runs with restricted privileges, so a compromised
       driver is far less valuable to an attacker.
     · the whole class of IRQL and paged-pool bugs simply does
       not exist.

   IF YOUR DEVICE QUALIFIES, USE IT. The reflexive assumption
   that "real drivers are kernel drivers" costs teams months
   of debugging that user-mode development would not have
   required.                                                */
```

Framework versions

```
; The framework ships as a library BOUND INTO your driver's
; INF, not as a fixed OS component. You target a version.

    [MyDriver_Install.Wdf]
    KmdfService = MyDriver, MyDriver_wdfsect

    [MyDriver_wdfsect]
    KmdfLibraryVersion = 1.15

; VERSION -> MINIMUM WINDOWS:
;   1.9   Windows 7
;   1.11  Windows 8
;   1.13  Windows 8.1
;   1.15  Windows 10 1507
;   1.17  Windows 10 1607
;   1.21  Windows 10 1809
;   1.31  Windows 10 2004
;   1.33  Windows 11

; PICK THE LOWEST VERSION that has the features you need -
; it maximises the range of systems your driver installs on.
; The build knows, and Code Analysis will tell you if you call
; an API newer than your declared version:

;   in the project properties, or the .vcxproj:
;     <KMDF_VERSION_MAJOR>1</KMDF_VERSION_MAJOR>
;     <KMDF_VERSION_MINOR>15</KMDF_VERSION_MINOR>

/* AND IN CODE, for a feature that may not be present: */
        if (WdfDriverIsVersionAvailable(driver, &versionInfo)) {
                ...
        }
```

Reading a WDM driver

WDM	KMDF
`IoCreateDevice` + `IoAttachDeviceToDeviceStack`	`WdfDeviceCreate`
`DeviceExtension`	A context, via `WDF_DECLARE_CONTEXT_TYPE`
`MajorFunction[IRP_MJ_READ]`	`EvtIoRead` on a queue
`IRP_MN_START_DEVICE`	`EvtDevicePrepareHardware` + `EvtDeviceD0Entry`
`IRP_MN_REMOVE_DEVICE`	`EvtDeviceD0Exit` + `EvtDeviceReleaseHardware`
Hand-written queue + cancel routine	`WDFQUEUE`
`IoCompleteRequest`	`WdfRequestComplete`
`IoConnectInterrupt`	`WdfInterruptCreate`
`KeInitializeDpc` + `KeInsertQueueDpc`	`WdfDpcCreate` + `WdfDpcEnqueue`
`IoAllocateMdl` + `MmProbeAndLockPages`	`WdfRequestRetrieveOutputBuffer`
`KSPIN_LOCK`	`WDFSPINLOCK`
Manual cleanup on every path	Object parenting

How it shows up

Symptom	Cause
Driver installs but never starts	KMDF version in the INF is newer than the OS supports
`WdfDriverCreate` fails	Called twice, or the co-installer/INF section is wrong
Bugcheck on removal in a WDM driver	PnP state machine implemented incorrectly — the reason WDF exists
Requests never complete after resume	Queue not restarted — WDF does this for you
Cancellation races	Hand-written queue instead of a WDFQUEUE
Context accessor returns garbage	`WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE` not used at creation
Objects leak across device removal	`ParentObject` not set
UMDF driver cannot do DMA	By design — it needs to be KMDF
Code Analysis flags an unavailable API	Calling something newer than your declared framework version

Check yourself

- ✓ Compare the three models across ten dimensions.
- ✓ State the decision order and the real reasons to choose WDM.
- ✓ List nine things WDF implements that WDM leaves to you.
- ✓ Write a complete `DriverEntry` and `EvtDeviceAdd` from memory.
- ✓ Explain object parenting and its Linux counterpart.
- ✓ Say what UMDF can and cannot do, and list four benefits of using it.
- ✓ Explain how framework versioning works and how to choose a version.
- ✓ Translate ten WDM constructs into their KMDF equivalents.

The WDF object model

Handles, contexts, parenting and cleanup callbacks. Once you understand these four ideas, every WDF API follows the same shape — and most of your cleanup code disappears.

Objects and handles

```
/* A WDF object is an opaque HANDLE, not a pointer: */

        WDFDEVICE     device;
        WDFQUEUE      queue;
        WDFREQUEST    request;
        WDFMEMORY     memory;
        WDFSPINLOCK   lock;
        WDFTIMER      timer;
        WDFINTERRUPT  interrupt;
        WDFDMAENABLER dmaEnabler;
        WDFCHILDLIST  childList;
        WDFFILEOBJECT fileObject;
        WDFCOLLECTION collection;
        WDFKEY        registryKey;
        WDFSTRING     string;
        WDFIOTARGET   ioTarget;

/* EVERY CREATION FUNCTION HAS THE SAME SHAPE: */

        NTSTATUS Wdf<Type>Create(
                <parent or config>,
                <type-specific config>,
                PWDF_OBJECT_ATTRIBUTES Attributes,
                <Type> *Handle);

/* AND EVERY OBJECT HAS: */
        a TYPE
        a PARENT              <-- the important one
        an optional CONTEXT   your data, attached to it
        a reference count
        optional EvtCleanupCallback and EvtDestroyCallback

/* YOU NEVER FREE AN OBJECT DIRECTLY in normal code. It is
   destroyed when its parent is. WdfObjectDelete exists but
   is needed only for objects you deliberately parented to
   something long-lived.                                    */
```

Parenting

```
WDFDRIVER
   |
   +-- WDFDEVICE                (parent: the driver)
          |
          +-- WDFQUEUE          (parent: the device)
          |      |
          |      +-- WDFREQUEST (parent: the queue)
          |             |
          |             +-- WDFMEMORY
          |
          +-- WDFINTERRUPT
          +-- WDFDMAENABLER
          |      +-- WDFDMATRANSACTION
          +-- WDFTIMER
          +-- WDFWORKITEM
          +-- WDFSPINLOCK
          +-- WDFFILEOBJECT     (one per open handle)

DESTROYING A PARENT DESTROYS ITS CHILDREN, depth-first,
in reverse creation order.

SO: parent everything to your WDFDEVICE and device removal
cleans up the entire tree, in the correct order, on every
path - including a failed EvtDeviceAdd.

THE DEFAULT PARENT is usually sensible: WdfSpinLockCreate
with WDF_NO_OBJECT_ATTRIBUTES parents to the DRIVER, which
is almost never what you want. SET IT EXPLICITLY:

     WDF_OBJECT_ATTRIBUTES_INIT(&attr);
     attr.ParentObject = device;
     WdfSpinLockCreate(&attr, &lock);

THIS IS EXACTLY Linux's devm_ (ch. 26) with a tree instead
of a flat list - and the same payoff: correct unwinding
that you did not have to write, on the error path you did
not test.
```

Contexts

```
/* A context is YOUR structure attached to a WDF object. It
   replaces WDM's DeviceExtension, and unlike it, you can
   attach one to ANY object.                                  */

/* 1. DECLARE the type and its accessor: */

typedef struct _MY_DEVICE_CONTEXT {
        WDFDEVICE     Device;
        WDFQUEUE      ReadQueue;
        WDFINTERRUPT  Interrupt;
        PUCHAR        RegBase;
        ULONG         RegLength;
        WDFSPINLOCK   StateLock;
        ULONG         SampleRate;
        LONG64        InterruptCount;
} MY_DEVICE_CONTEXT, *PMY_DEVICE_CONTEXT;

WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(MY_DEVICE_CONTEXT,
                                   MyGetDeviceContext)
/* generates:
     PMY_DEVICE_CONTEXT MyGetDeviceContext(WDFOBJECT);
   plus the type information WDF needs to size the
   allocation.                                                */

/* 2. REQUEST it at creation: */
        WDF_OBJECT_ATTRIBUTES attributes;

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attributes,
                                        MY_DEVICE_CONTEXT);
        status = WdfDeviceCreate(&DeviceInit, &attributes,
                                 &device);

/* 3. USE it: */
        PMY_DEVICE_CONTEXT ctx = MyGetDeviceContext(device);
        ctx->SampleRate = 48000;

/* SEVERAL CONTEXT TYPES ON ONE OBJECT is allowed, with
   different accessors - useful when a shared library and your
   driver both need private data on the same device.       */

/* PER-REQUEST CONTEXT, the common second use: */
typedef struct _MY_REQUEST_CONTEXT {
        WDFMEMORY  Memory;
        ULONG      Channel;
        LARGE_INTEGER StartTime;
} MY_REQUEST_CONTEXT, *PMY_REQUEST_CONTEXT;

WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(MY_REQUEST_CONTEXT,
                                   MyGetRequestContext)

        /* attach it to requests arriving on a queue: */
        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attr,
                                        MY_REQUEST_CONTEXT);
        queueConfig.  /* ... */;
        WdfIoQueueCreate(device, &queueConfig, &attr, &queue);
        /* now every request on this queue has your context */

/* THE CONTEXT MEMORY IS NON-PAGED and ZEROED at creation, so
```

```
    you do not allocate or free it - it lives inside the
    object's own allocation.                              */
```

> **THE MOST COMMON WDF MISTAKE**
>
> Calling `MyGetDeviceContext()` on an object created *without* `WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE`. The accessor returns garbage or NULL, and you get a crash with no obvious cause. If a context pointer looks wrong, check the creation call first — the attributes must name the same context type the accessor expects.

Cleanup and destroy callbacks

```
/* Two callbacks, at different points in an object's death. */

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attr,
                                        MY_DEVICE_CONTEXT);
        attr.EvtCleanupCallback = MyEvtDeviceCleanup;
        attr.EvtDestroyCallback = MyEvtDeviceDestroy;
        attr.ParentObject       = parent;

VOID
MyEvtDeviceCleanup(_In_ WDFOBJECT Object)
{
        PMY_DEVICE_CONTEXT ctx = MyGetDeviceContext(Object);

        /* CLEANUP: the object is being deleted, but its
           children still exist and its context is still
           valid. This is where you release things you
           acquired yourself.

           IRQL: <= DISPATCH_LEVEL by default, or PASSIVE if
           you asked for WdfExecutionLevelPassive.        */

        if (ctx->MyOwnAllocation) {
                ExFreePoolWithTag(ctx->MyOwnAllocation, TAG);
                ctx->MyOwnAllocation = NULL;
        }
}

VOID
MyEvtDeviceDestroy(_In_ WDFOBJECT Object)
{
        /* DESTROY: the last reference is gone, all children
           are destroyed, and the memory is about to be
           freed. Very little belongs here.

           IRQL: <= DISPATCH_LEVEL. Do not touch children -
           they no longer exist.                          */
}

/* THE ORDER on deletion:
     1. EvtCleanupCallback on the object
     2. children are deleted (each with its own cleanup)
     3. EvtDestroyCallback on the object
     4. memory freed

/* WHICH TO USE: almost always Cleanup. Destroy is for the
   rare case where you must know the object is truly gone -
   for example, signalling that a device's memory can now be
   reused.

/* AND THE POINT: if everything you allocate is either a WDF
   object parented correctly, or a WDFMEMORY, you need
   NEITHER callback. Reach for them only for raw resources
   the framework does not know about.                     */
```

WDFMEMORY

```
/* A WDF object wrapping a buffer, so allocation participates
   in parenting and lifetime management.                    */

        WDFMEMORY memory;
        PVOID buffer;
        WDF_OBJECT_ATTRIBUTES attr;

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.ParentObject = request;    /* dies with the
                                           request           */

        status = WdfMemoryCreate(&attr,
                                 NonPagedPoolNx,
                                 MY_POOL_TAG,
                                 size,
                                 &memory,
                                 &buffer);   /* optional out */
        if (!NT_SUCCESS(status))
                return status;

        /* no free needed - the request's completion destroys
           it                                                 */

/* GETTING AT IT LATER: */
        size_t length;
        PVOID p = WdfMemoryGetBuffer(memory, &length);

/* WRAPPING A BUFFER YOU ALREADY HAVE, without copying: */
        status = WdfMemoryCreatePreallocated(&attr,
                         existingBuffer, size, &memory);
        /* WDF does NOT free the buffer in this case - you
           still own it. Use this to hand an existing buffer
           to an API that wants a WDFMEMORY.               */

/* COPYING, with bounds checking done for you: */
        WdfMemoryCopyFromBuffer(memory, offset, src, length);
        WdfMemoryCopyToBuffer(memory, offset, dst, length);
/* These return STATUS_BUFFER_TOO_SMALL rather than
   overrunning - which is the argument for using them instead
   of RtlCopyMemory on a raw pointer.                      */
```

Synchronisation scope

```
/* WDF can AUTOMATICALLY serialise your callbacks, so you may
   not need locks at all.                                      */

        WDF_OBJECT_ATTRIBUTES attr;

        WDF_OBJECT_ATTRIBUTES_INIT(&attr);
        attr.SynchronizationScope = WdfSynchronizationScopeDevice;
        attr.ExecutionLevel       = WdfExecutionLevelPassive;

/* SYNCHRONIZATION SCOPE:

   WdfSynchronizationScopeDevice
        ALL callbacks for this device, and its queues, timers
        and DPCs, are serialised against one another with a
        single device lock. Simplest to reason about; limits
        parallelism.

   WdfSynchronizationScopeQueue
        callbacks for one queue are serialised with each
        other, but different queues run in parallel.

   WdfSynchronizationScopeNone
        no automatic serialisation. YOU do the locking.
        The default for a parallel queue, and the right choice
        for a high-throughput device.

   WdfSynchronizationScopeInheritFromParent
        the default for most objects.

/* EXECUTION LEVEL:

   WdfExecutionLevelPassive
        every callback runs at PASSIVE_LEVEL, so it may touch
        paged memory and sleep. The framework uses work items
        to arrange it.
        NOTE: cannot be combined with a queue that must run at
        DISPATCH_LEVEL, and it adds latency.

   WdfExecutionLevelDispatch
        callbacks may run at DISPATCH_LEVEL.

   WdfExecutionLevelInheritFromParent   the default.

/* THE TRADE, stated plainly:
     ScopeDevice + LevelPassive is the EASIEST model to write
     correctly - one lock, everything sleepable, no IRQL
     puzzles. It is also the slowest, because every callback
     serialises against every other.

     ScopeNone + parallel dispatch is the FASTEST and requires
     you to get the locking right yourself (chapter 09).

   START with the simple model. Move to the parallel one when
   you have measured a reason to.                            */
```

Collections and child lists

```
/* WDFCOLLECTION - an ordered set of WDF objects: */
        WDFCOLLECTION collection;

        WdfCollectionCreate(&attr, &collection);
        WdfCollectionAdd(collection, someObject);

        ULONG count = WdfCollectionGetCount(collection);
        for (ULONG i = 0; i < count; i++) {
                WDFOBJECT obj = WdfCollectionGetItem(collection, i);
                ...
        }
        WdfCollectionRemove(collection, someObject);

/* Holds a REFERENCE on each item, so an object in a
   collection outlives its parent's deletion attempt until
   removed. Convenient, and a source of "the device will not
   go away" bugs if you forget to empty it.               */

/* WDFCHILDLIST - for a BUS DRIVER that enumerates children: */
        WDF_CHILD_LIST_CONFIG config;

        WDF_CHILD_LIST_CONFIG_INIT(&config,
                        sizeof(MY_CHILD_ID),
                        MyEvtChildListCreateDevice);
        WdfFdoInitSetDefaultChildListConfig(DeviceInit,
                        &config, WDF_NO_OBJECT_ATTRIBUTES);

        /* when you discover a child: */
        WdfChildListBeginScan(list);
        WdfChildListAddOrUpdateChildDescriptionAsPresent(
                        list, &id.Header, NULL);
        WdfChildListEndScan(list);
        /* anything not re-reported during the scan is
           automatically reported as REMOVED - which handles
           hot-unplug without you tracking it            */

/* This is how a driver for a bus, a multi-function device, or
   a device with dynamically appearing endpoints reports its
   children to PnP (chapter 05).                          */
```

Debugging WDF objects

```
# The WDF debugger extension - load it first:
    .load wdfkd.dll
    !wdfkd.wdfhelp

# THE ESSENTIAL COMMANDS:
    !wdfdriverinfo mydriver         everything about the driver
    !wdfdevicequeues <WDFDEVICE>    every queue and its
                                    contents
    !wdfobjectdump                  every WDF object
    !wdfhandle <handle>             one object: type, parent,
                                    context, refcount
    !wdfobject <handle>
    !wdfqueue <WDFQUEUE>            requests in a queue
    !wdfrequest <WDFREQUEST>        one request's state
    !wdflogdump mydriver            THE FRAMEWORK'S OWN LOG

# !wdflogdump IS THE ONE TO REMEMBER. WDF keeps an internal
# trace of every state transition, every request, and every
# error it detected. On a crash, it usually tells you what the
# framework thought was happening - which is often the whole
# answer.

# ENABLE VERBOSE FRAMEWORK LOGGING during development:
#   in the registry, under the driver's Parameters\Wdf key:
    VerboseOn      = 1
    VerifierOn     = 1     ; WDF VERIFIER - see below
    DbgBreakOnError = 1

# Or via the WDF verifier settings in the WDK's
# "WDF Verifier Control Application" (wdfverifier.exe).

# WDF VERIFIER checks framework-specific rules: object
# lifetime, handle validity, IRQL for each API, correct
# request completion. It is separate from Driver Verifier
# (chapter 21) and catches a different, WDF-specific class of
# bug. Turn BOTH on.
```

How it shows up

Symptom	Cause
Context accessor returns NULL or garbage	Object created without the matching context type
Objects leak across device removal	`ParentObject` not set — they parented to the driver
Crash in a cleanup callback	Touched a child that was already destroyed — use Cleanup, not Destroy
Device will not go away	A reference held by a collection, or an object parented too high
Callbacks race with each other	`SynchronizationScopeNone` without your own locking
Callback bugchecks touching paged memory	Execution level is Dispatch, not Passive
Buffer freed while still in use	WDFMEMORY parented to something shorter-lived than its use
Preallocated memory double-freed	`WdfMemoryCreatePreallocated` does not take ownership
Child devices never removed	Not re-scanned — WDFCHILDLIST reports absence by omission
WDF verifier bugcheck	A framework rule violation it made deterministic

Check yourself

- ✓ Name ten WDF object types and the common shape of their creation functions.
- ✓ Draw the parenting tree and explain what destroying a device does.
- ✓ Explain why the default parent is often wrong and what to set instead.
- ✓ Declare a context type, request it at creation, and use the accessor.
- ✓ Explain the most common context mistake and how to spot it.
- ✓ Distinguish Cleanup from Destroy and say which to use.
- ✓ Use WDFMEMORY, including the preallocated form and its ownership rule.
- ✓ Explain synchronisation scope and execution level, and the trade between them.
- ✓ Explain how a WDFCHILDLIST reports removal.
- ✓ Use `!wdflogdump` and say why it is the first command to try.

The WDF device lifecycle

Which callback runs when, what belongs in each, and what the framework guarantees about the order. Get this table right and PnP, power management and surprise removal all work without you writing a state machine.

The order

```
DriverEntry                        once, per driver load
     |
     v
EvtDriverDeviceAdd                 once PER DEVICE
     |   create the WDFDEVICE, queues, interrupt objects
     |   NO HARDWARE ACCESS - resources are not assigned
     v
EvtDevicePrepareHardware           resources assigned
     |   map registers, find the interrupt's descriptor
     |   PASSIVE_LEVEL
     v
EvtDeviceD0Entry                   powering up
     |   initialise the hardware, restore state
     |   PASSIVE_LEVEL
     v
EvtDeviceD0EntryPostInterruptsEnabled
     |   interrupts are now live; safe to start the device
     v
===== RUNNING: queues dispatch requests, ISR/DPC fire =====
     |
     v
EvtDeviceD0ExitPreInterruptsDisabled
     |   stop the device generating interrupts
     v
EvtDeviceD0Exit                    powering down
     |   quiesce, save state. THE DEVICE IS STILL THERE.
     v
EvtDeviceReleaseHardware           resources revoked
     |   unmap. DO NOT TOUCH THE HARDWARE.
     v
EvtCleanupCallback / EvtDestroyCallback
     |
     v
DriverUnload / EvtDriverUnload     once

AND THE POWER CYCLE, which happens repeatedly WITHOUT
PrepareHardware/ReleaseHardware:

     D0Entry -> running -> D0Exit -> (sleep) -> D0Entry ...

THAT IS THE KEY ASYMMETRY: PrepareHardware runs once per
resource assignment; D0Entry runs on every power-up. So
MAPPING goes in PrepareHardware and INITIALISATION goes in
D0Entry - and putting initialisation in PrepareHardware
produces a device that works until the first sleep.
```

What belongs where

Callback	Do	Do not
`DriverEntry`	`WdfDriverCreate`, global state	Anything per-device
`EvtDriverDeviceAdd`	Create device, queues, interrupts, locks, timers; publish the interface	Touch hardware — there is none yet
`EvtDevicePrepareHardware`	Map MMIO, read resource descriptors, allocate buffers sized from resources	Program the device
`EvtDeviceD0Entry`	Reset, configure, restore registers, verify the ID	Enable interrupts (use the Post callback)
`EvtDeviceD0Exit`	Stop the device, save state, flush	Unmap
`EvtDeviceReleaseHardware`	Unmap, free what PrepareHardware allocated	**Touch the hardware**
`EvtDeviceSelfManagedIoInit`	Start self-managed I/O — timers, polling threads	
`EvtDeviceSelfManagedIoSuspend`	Stop them, and **wait**	
`EvtDeviceSelfManagedIoCleanup`	Final teardown of self-managed I/O	

> **THE RULE THAT PREVENTS THE WORST BUG**
>
> `EvtDeviceReleaseHardware` can run *after the device has physically been removed*. A register read then returns `0xFFFFFFFF`; a write may hang the bus or bugcheck. All hardware quiescing belongs in `EvtDeviceD0Exit`, which is guaranteed to run while the device is still present in the normal path — and which the framework *skips the hardware-touching parts of* on surprise removal.

The callbacks, implemented

```
NTSTATUS
MyPrepareHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesRaw,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        ULONG i, count;
        BOOLEAN foundMemory = FALSE;

        PAGED_CODE();
        UNREFERENCED_PARAMETER(ResourcesRaw);

        count = WdfCmResourceListGetCount(ResourcesTranslated);

        for (i = 0; i < count; i++) {
                PCM_PARTIAL_RESOURCE_DESCRIPTOR desc =
                        WdfCmResourceListGetDescriptor(
                                ResourcesTranslated, i);

                switch (desc->Type) {
                case CmResourceTypeMemory:
                        if (foundMemory)
                                break;    /* only the first */

                        ctx->RegLength = desc->u.Memory.Length;
                        ctx->RegBase = (PUCHAR)MmMapIoSpaceEx(
                                desc->u.Memory.Start,
                                desc->u.Memory.Length,
                                PAGE_READWRITE | PAGE_NOCACHE);
                        if (ctx->RegBase == NULL)
                                return STATUS_INSUFFICIENT_RESOURCES;
                        foundMemory = TRUE;
                        break;

                case CmResourceTypeInterrupt:
                        /* WdfInterruptCreate in DeviceAdd
                           already claimed it (chapter 17) */
                        break;
                }
        }

        if (!foundMemory)
                return STATUS_DEVICE_CONFIGURATION_ERROR;

        return STATUS_SUCCESS;
}

NTSTATUS
MyD0Entry(_In_ WDFDEVICE Device,
          _In_ WDF_POWER_DEVICE_STATE PreviousState)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        ULONG id;

        UNREFERENCED_PARAMETER(PreviousState);

        /* The device may have lost all state. Verify it is
           there, then configure it from scratch.          */

        id = READ_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_ID));
```

```
        if (id != MY_EXPECTED_ID) {
                /* 0xFFFFFFFF means it is gone */
                return STATUS_DEVICE_NOT_CONNECTED;
        }

        /* reset */
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_CTRL),
                             CTRL_RESET);
        KeStallExecutionProcessor(10);
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_CTRL), 0);

        /* restore our configuration */
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_RATE),
                             ctx->SampleRate);

        return STATUS_SUCCESS;
}

NTSTATUS
MyD0EntryPostInterruptsEnabled(_In_ WDFDEVICE Device,
                       _In_ WDF_POWER_DEVICE_STATE PreviousState)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        UNREFERENCED_PARAMETER(PreviousState);

        /* interrupts are connected NOW - safe to let the
           device raise them                                */
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_INT_EN),
                             INT_ALL);
        return STATUS_SUCCESS;
}

NTSTATUS
MyD0ExitPreInterruptsDisabled(_In_ WDFDEVICE Device,
                       _In_ WDF_POWER_DEVICE_STATE TargetState)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        UNREFERENCED_PARAMETER(TargetState);

        /* stop the device asserting BEFORE the framework
           disconnects the interrupt - otherwise we take one
           with no handler                                  */
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_INT_EN),
                             0);
        return STATUS_SUCCESS;
}

NTSTATUS
MyD0Exit(_In_ WDFDEVICE Device,
         _In_ WDF_POWER_DEVICE_STATE TargetState)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        /* quiesce. The device is still present here. */
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + REG_CTRL), 0);

        /* wait for our own deferred work (chapter 10) */
        WdfDpcCancel(ctx->Dpc, TRUE);
        WdfTimerStop(ctx->Timer, TRUE);

        if (TargetState == WdfPowerDeviceD3Final) {
```

```
                /* the device is going away for good */
        }

        return STATUS_SUCCESS;
}

NTSTATUS
MyReleaseHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        PAGED_CODE();
        UNREFERENCED_PARAMETER(ResourcesTranslated);

        /* NO HARDWARE ACCESS HERE. */

        if (ctx->RegBase) {
                MmUnmapIoSpace(ctx->RegBase, ctx->RegLength);
                ctx->RegBase = NULL;
        }
        return STATUS_SUCCESS;
}
```

Power states

```
/* DEVICE states (D-states): */
        D0   fully on
        D1   light sleep, context preserved (rare)
        D2   deeper (rare)
        D3hot powered down, still in the slot
        D3cold power removed entirely

/* WDF's enumeration adds the reason: */
        WdfPowerDeviceD0
        WdfPowerDeviceD1 / D2 / D3
        WdfPowerDeviceD3Final     <-- the device is being
                                      REMOVED, not just slept
        WdfPowerDevicePrepareForHibernation
        WdfPowerDeviceMaximum

/* CHECK FOR D3Final in D0Exit: it distinguishes "going to
   sleep, we will be back" from "goodbye". Freeing things in
   the first case leaves you broken on resume.            */

/* SYSTEM states (S-states): S0 working, S3 sleep, S4
   hibernate, S5 off. The power manager maps a system
   transition onto D-state transitions for each device.

/* IDLE POWER MANAGEMENT - let the device sleep when unused: */

        WDF_DEVICE_POWER_POLICY_IDLE_SETTINGS idleSettings;

        WDF_DEVICE_POWER_POLICY_IDLE_SETTINGS_INIT(
                &idleSettings, IdleCanWakeFromS0);
        idleSettings.IdleTimeout   = 5000;    /* ms */
        idleSettings.UserControlOfIdleSettings =
                        IdleAllowUserControl;
        idleSettings.Enabled = WdfTrue;

        status = WdfDeviceAssignS0IdlePolicy(device,
                                             &idleSettings);

/* Then the framework calls D0Exit after 5 seconds of no
   requests, and D0Entry when the next one arrives - stalling
   the request until the device is up. You write nothing else.

   This is the direct counterpart of Linux's runtime PM
   (ch. 25), and it is considerably less code because the
   framework does the reference counting through the queues
   rather than making you bracket every access.           */

/* WAKE FROM SLEEP: */
        WDF_DEVICE_POWER_POLICY_WAKE_SETTINGS wakeSettings;

        WDF_DEVICE_POWER_POLICY_WAKE_SETTINGS_INIT(
                &wakeSettings);
        status = WdfDeviceAssignSxWakeSettings(device,
                                               &wakeSettings);
```

Self-managed I/O

```
/* Queues are stopped and started by the framework around
   power transitions. But work you started YOURSELF - a
   polling timer, a system thread, a continuous DMA - is not
   known to the framework. Self-managed I/O callbacks are
   where you handle it.                                    */

        WDF_PNPPOWER_EVENT_CALLBACKS pnpPower;

        WDF_PNPPOWER_EVENT_CALLBACKS_INIT(&pnpPower);
        pnpPower.EvtDeviceSelfManagedIoInit    = MySmIoInit;
        pnpPower.EvtDeviceSelfManagedIoSuspend = MySmIoSuspend;
        pnpPower.EvtDeviceSelfManagedIoRestart = MySmIoRestart;
        pnpPower.EvtDeviceSelfManagedIoCleanup = MySmIoCleanup;

NTSTATUS
MySmIoInit(_In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        /* runs ONCE, after the first D0Entry */
        WdfTimerStart(ctx->PollTimer,
                      WDF_REL_TIMEOUT_IN_MS(100));
        return STATUS_SUCCESS;
}

NTSTATUS
MySmIoSuspend(_In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        /* STOP AND WAIT. The framework will not proceed with
           the power transition until this returns, so
           anything still running afterwards is a bug you
           introduced.                                     */
        WdfTimerStop(ctx->PollTimer, TRUE);   /* TRUE = wait */
        WdfWorkItemFlush(ctx->WorkItem);
        return STATUS_SUCCESS;
}

NTSTATUS
MySmIoRestart(_In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        WdfTimerStart(ctx->PollTimer,
                      WDF_REL_TIMEOUT_IN_MS(100));
        return STATUS_SUCCESS;
}

VOID
MySmIoCleanup(_In_ WDFDEVICE Device)
{
        /* final teardown - the device is going away */
}

/* THE SEQUENCE around a sleep:
     SelfManagedIoSuspend -> D0ExitPre -> D0Exit
     ... sleep ...
     D0Entry -> D0EntryPost -> SelfManagedIoRestart
```

```
   IF YOU HAVE NO TIMERS OR THREADS OF YOUR OWN, you do not
   need these callbacks at all. That is the common case - and
   another reason to prefer queues over self-started work. */
```

Surprise removal

```
/* The user unplugged it. The hardware is GONE.

   WHAT THE FRAMEWORK DOES:
     · fails all new requests with STATUS_DEVICE_REMOVED
     · calls EvtDeviceSurpriseRemoval if you registered one
     · SKIPS D0Exit's hardware access expectations - but it
       still CALLS D0Exit, so your code must tolerate a
       missing device
     · calls ReleaseHardware
     · destroys the device

   WHAT YOU MUST DO:
     · detect the "all ones" read and stop rather than
       retrying forever
     · never wait for hardware without a timeout
     · complete every pending request                    */

static BOOLEAN
MyDeviceIsPresent(PMY_CONTEXT ctx)
{
        ULONG id = READ_REGISTER_ULONG(
                        (PULONG)(ctx->RegBase + REG_ID));

        return (id != 0xFFFFFFFF) && (id == MY_EXPECTED_ID);
}

        /* use it before any operation that would otherwise
           poll: */
        if (!MyDeviceIsPresent(ctx)) {
                WdfRequestComplete(request,
                                   STATUS_DEVICE_REMOVED);
                return;
        }

/* TESTING IT: in Device Manager, disable the device while I/O
   is in flight. Or, for USB, unplug it during a transfer. Or
   use the WDK's "Device Fundamentals" tests, which do exactly
   this automatically - and which you must pass for
   certification anyway (chapter 21).                  */
```

How it shows up

Symptom	Cause
Works until the first sleep, then dead	Initialisation in `PrepareHardware` instead of `D0Entry`
Bugcheck on unplug	Hardware touched in `ReleaseHardware`
Hang on unplug	Polling for hardware that is gone, with no timeout
Spurious interrupt during power-down	Device not silenced in `D0ExitPreInterruptsDisabled`
Crash after resume in a timer callback	Timer not stopped-and-waited in self-managed I/O suspend
Device never idles	No S0 idle policy assigned

Symptom	Cause
Device sleeps mid-operation	Work started outside a queue, so the framework does not know it is busy
State lost on resume	`D0Entry` does not restore configuration
Freed something needed after resume	Did not check for `D3Final` in `D0Exit`
Device Manager shows Code 10	`PrepareHardware` or `D0Entry` returned an error
Registers read `0xFFFFFFFF`	Surprise removal — detect and fail fast

Check yourself

- ✓ Write out the callback order from `DriverEntry` to unload.
- ✓ Explain the asymmetry between `PrepareHardware` and `D0Entry`.
- ✓ Say what belongs in each of the nine callbacks, and what does not.
- ✓ Explain why hardware access is forbidden in `ReleaseHardware`.
- ✓ Explain the purpose of the Pre/Post interrupt callbacks.
- ✓ Distinguish `D3` from `D3Final` and say why it matters.
- ✓ Assign an S0 idle policy and compare it with Linux runtime PM.
- ✓ Say when self-managed I/O callbacks are needed and what they must guarantee.
- ✓ Write a device-present check and explain the all-ones read.
- ✓ Test surprise removal deliberately.

WDF request queues

Queues are where WDF earns its keep. They handle dispatch, cancellation, power stalling and per-handle cleanup — the four things that made raw WDM I/O so error-prone — and they reduce your driver to a handful of callbacks.

Dispatch types

```
SEQUENTIAL
   One request at a time. The next is not delivered until
   the current one COMPLETES.
   -> simplest to write; no concurrency to reason about
   -> correct for hardware that can only do one thing
   -> the default choice for a first driver

PARALLEL
   Requests delivered as they arrive, from any CPU,
   concurrently.
   -> you must handle concurrency yourself
   -> correct for hardware with multiple channels, or when
      you queue internally

MANUAL
   Nothing is delivered. You call
   WdfIoQueueRetrieveNextRequest when you are ready.
   -> a holding pen for pending requests
   -> THE standard pattern for asynchronous I/O: the
      default queue receives, hands to a manual queue, and
      the DPC retrieves and completes

THE COMMON ARCHITECTURE, and the one to learn:

   default queue (parallel)   receives every request
         |
         +-- a read arrives, no data yet
         |         -> WdfRequestForwardToIoQueue(manual)
         |
         v
   manual queue               holds pending reads
         ^
         |  interrupt -> DPC
         |         -> WdfIoQueueRetrieveNextRequest
         |         -> fill it, complete it

That structure gives you asynchronous, cancellable,
power-aware I/O in about forty lines - versus several
hundred in WDM (chapter 04).
```

Creating queues

```
/* THE DEFAULT QUEUE - receives anything not routed elsewhere */

        WDF_IO_QUEUE_CONFIG queueConfig;
        NTSTATUS status;

        WDF_IO_QUEUE_CONFIG_INIT_DEFAULT_QUEUE(&queueConfig,
                        WdfIoQueueDispatchParallel);

        queueConfig.EvtIoRead          = MyEvtIoRead;
        queueConfig.EvtIoWrite         = MyEvtIoWrite;
        queueConfig.EvtIoDeviceControl = MyEvtIoDeviceControl;
        queueConfig.EvtIoStop          = MyEvtIoStop;

        status = WdfIoQueueCreate(device, &queueConfig,
                                  WDF_NO_OBJECT_ATTRIBUTES,
                                  &ctx->DefaultQueue);

/* A MANUAL QUEUE, for holding pending requests: */

        WDF_IO_QUEUE_CONFIG_INIT(&queueConfig,
                                 WdfIoQueueDispatchManual);
        queueConfig.PowerManaged = WdfFalse;   /* see below */

        status = WdfIoQueueCreate(device, &queueConfig,
                                  WDF_NO_OBJECT_ATTRIBUTES,
                                  &ctx->PendingReadQueue);

/* ROUTING a specific request type to its own queue: */

        status = WdfDeviceConfigureRequestDispatching(device,
                        ctx->WriteQueue,
                        WdfRequestTypeWrite);
/* now every IRP_MJ_WRITE goes to WriteQueue instead of the
   default one - useful when reads and writes have completely
   different handling and concurrency needs.              */

/* POWER MANAGED (the default for the default queue):
     the framework STOPS the queue when the device leaves D0
     and RESTARTS it on return. Requests wait rather than
     failing. This is what makes S0 idle transparent to
     userspace (chapter 14).

   NOT POWER MANAGED:
     the queue keeps delivering regardless of power state.
     Required for a manual queue used from a DPC, and for any
     queue that must work while powering down.

   GETTING THIS WRONG produces "requests hang after the device
   idles" (power-managed manual queue that nobody restarts) or
   "requests arrive while the device is off" (not power
   managed when it should be).                            */
```

The callbacks

```
VOID
MyEvtIoRead(_In_ WDFQUEUE Queue, _In_ WDFREQUEST Request,
            _In_ size_t Length)
{
        WDFDEVICE device = WdfIoQueueGetDevice(Queue);
        PMY_CONTEXT ctx = MyGetContext(device);
        PVOID buffer;
        size_t bufLen;
        NTSTATUS status;

        if (Length < sizeof(MY_SAMPLE)) {
                WdfRequestComplete(Request,
                                   STATUS_BUFFER_TOO_SMALL);
                return;
        }

        /* data available right now? */
        WdfSpinLockAcquire(ctx->StateLock);
        if (ctx->DataReady) {
                ctx->DataReady = FALSE;
                WdfSpinLockRelease(ctx->StateLock);

                status = WdfRequestRetrieveOutputBuffer(Request,
                                sizeof(MY_SAMPLE),
                                &buffer, &bufLen);
                if (!NT_SUCCESS(status)) {
                        WdfRequestComplete(Request, status);
                        return;
                }

                RtlCopyMemory(buffer, &ctx->LastSample,
                              sizeof(MY_SAMPLE));

                WdfRequestCompleteWithInformation(Request,
                                STATUS_SUCCESS,
                                sizeof(MY_SAMPLE));
                return;
        }
        WdfSpinLockRelease(ctx->StateLock);

        /* no data - park it on the manual queue */
        status = WdfRequestForwardToIoQueue(Request,
                                ctx->PendingReadQueue);
        if (!NT_SUCCESS(status))
                WdfRequestComplete(Request, status);

        /* THAT IS ALL. The framework now owns cancellation,
           power stalling and cleanup for this request.    */
}

/* AND IN THE DPC (chapter 10): */
VOID
MyEvtInterruptDpc(_In_ WDFINTERRUPT Interrupt,
                  _In_ WDFOBJECT AssociatedObject)
{
        PMY_CONTEXT ctx = MyGetContext(
                        WdfInterruptGetDevice(Interrupt));
        WDFREQUEST request;
        PVOID buffer;
        size_t bufLen;
```

```
        NTSTATUS status;

        UNREFERENCED_PARAMETER(AssociatedObject);

        status = WdfIoQueueRetrieveNextRequest(
                        ctx->PendingReadQueue, &request);
        if (!NT_SUCCESS(status))
                return;      /* nobody waiting - fine */

        /* From here, the request is OURS. It can no longer be
           cancelled out from under us - retrieving it removed
           it from the queue and took ownership.          */

        status = WdfRequestRetrieveOutputBuffer(request,
                        sizeof(MY_SAMPLE), &buffer, &bufLen);
        if (NT_SUCCESS(status)) {
                RtlCopyMemory(buffer, &ctx->LastSample,
                              sizeof(MY_SAMPLE));
                WdfRequestCompleteWithInformation(request,
                                STATUS_SUCCESS,
                                sizeof(MY_SAMPLE));
        } else {
                WdfRequestComplete(request, status);
        }
}
```

THE OWNERSHIP RULE

A request belongs to exactly one place at a time: the queue, or you. `WdfIoQueueRetrieveNextRequest` transfers ownership to you and guarantees it cannot be cancelled behind your back. Once you own it, you **must** complete it, forward it, or requeue it — dropping it hangs the caller forever. This single guarantee is what removes the cancel-routine race that dominates WDM queue code (chapter 04).

Cancellation

```
/* A request sitting on a queue is cancellable automatically.
   The framework completes it with STATUS_CANCELLED and you
   write nothing.

   IF YOU NEED TO KNOW - to abort a hardware operation, for
   example - mark it cancellable explicitly after taking
   ownership: */

        status = WdfRequestMarkCancelableEx(request,
                                            MyEvtRequestCancel);
        if (status == STATUS_CANCELLED) {
                /* it was ALREADY cancelled - complete it and
                   do not use it                              */
                WdfRequestComplete(request, STATUS_CANCELLED);
                return;
        }

VOID
MyEvtRequestCancel(_In_ WDFREQUEST Request)
{
        WDFDEVICE device = WdfIoQueueGetDevice(
                        WdfRequestGetIoQueue(Request));
        PMY_CONTEXT ctx = MyGetContext(device);

        /* abort the hardware operation */
        MyAbortTransfer(ctx);

        WdfRequestComplete(Request, STATUS_CANCELLED);
}

/* AND BEFORE COMPLETING IT NORMALLY, take it back: */
        status = WdfRequestUnmarkCancelable(request);
        if (status == STATUS_CANCELLED) {
                /* the cancel routine is running or has run -
                   it owns the request. DO NOT complete it. */
                return;
        }
        WdfRequestComplete(request, STATUS_SUCCESS);

/* THIS IS THE SAME OWNERSHIP PROTOCOL as WDM's
   IoSetCancelRoutine(NULL) dance (chapter 04) - but it is
   three lines with a clear return code, rather than a
   spinlock-protected sequence you must get exactly right on
   every path.                                              */

/* PURGING a queue - cancel everything on it: */
        WdfIoQueuePurgeSynchronously(queue);
        /* returns when every request is completed */

        WdfIoQueueStopSynchronously(queue);   /* pause */
        WdfIoQueueStart(queue);               /* resume */
```

EvtIoStop

```
/* Called when the framework must stop the queue - a power
   transition, a purge, or removal - and a request is still
   IN FLIGHT (you own it, not the queue).                   */

VOID
MyEvtIoStop(_In_ WDFQUEUE Queue, _In_ WDFREQUEST Request,
            _In_ ULONG ActionFlags)
{
        if (ActionFlags & WdfRequestStopActionSuspend) {
                /* "can you pause this?" - acknowledge that we
                   will not complete it soon, and let the
                   framework proceed                        */
                WdfRequestStopAcknowledge(Request, FALSE);

        } else if (ActionFlags & WdfRequestStopActionPurge) {
                /* "cancel it" */
                WdfRequestCancelSentRequest(Request);
        }
}

/* WHAT HAPPENS IF YOU DO NOT IMPLEMENT IT: the framework
   waits for the request to complete before allowing the power
   transition. For a request waiting on hardware that will not
   respond until the device is powered - which is exactly the
   case during a suspend - that is a DEADLOCK, and it appears
   as "the machine will not sleep".

   IMPLEMENT EvtIoStop for any queue that can hold a request
   for an unbounded time. That is most manual queues.     */
```

Getting at a request

```
/* BUFFERS - method-independent (chapter 07): */
        status = WdfRequestRetrieveInputBuffer(request,
                        minLength, &buffer, &actualLength);
        status = WdfRequestRetrieveOutputBuffer(request,
                        minLength, &buffer, &actualLength);
/* both CHECK the minimum length for you and return
   STATUS_BUFFER_TOO_SMALL - so the most commonly forgotten
   validation is not optional.                              */

/* AS WDFMEMORY, if you want to pass it on: */
        status = WdfRequestRetrieveInputMemory(request, &memory);

/* AS AN MDL, for DMA (chapter 18): */
        status = WdfRequestRetrieveOutputWdmMdl(request, &mdl);

/* THE PARAMETERS - what kind of request is this: */
        WDF_REQUEST_PARAMETERS params;

        WDF_REQUEST_PARAMETERS_INIT(&params);
        WdfRequestGetParameters(request, &params);

        params.Type                  /* WdfRequestTypeRead ... */
        params.Parameters.Read.Length
        params.Parameters.DeviceIoControl.IoControlCode
        params.Parameters.DeviceIoControl.InputBufferLength
        params.Parameters.DeviceIoControl.OutputBufferLength

/* THE FILE OBJECT - which handle issued this: */
        WDFFILEOBJECT file = WdfRequestGetFileObject(request);
        PMY_FILE_CONTEXT fc = MyGetFileContext(file);
/* This is how you keep PER-HANDLE state, the Windows
   equivalent of file->private_data (Linux ch. 20).       */

/* COMPLETING: */
        WdfRequestComplete(request, status);
        WdfRequestCompleteWithInformation(request, status,
                                          bytesTransferred);
        WdfRequestCompleteWithPriorityBoost(request, status,
                                            IO_DISK_INCREMENT);

/* SET THE INFORMATION. It is what ReadFile reports as bytes
   transferred; omitting it means userspace sees zero, or
   worse, an uninitialised value.                          */
```

Per-handle state

```
/* Get callbacks when a handle is opened and closed: */

        WDF_FILEOBJECT_CONFIG fileConfig;
        WDF_OBJECT_ATTRIBUTES fileAttr;

        WDF_FILEOBJECT_CONFIG_INIT(&fileConfig,
                        MyEvtDeviceFileCreate,
                        MyEvtFileClose,
                        MyEvtFileCleanup);

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&fileAttr,
                        MY_FILE_CONTEXT);

        WdfDeviceInitSetFileObjectConfig(DeviceInit,
                        &fileConfig, &fileAttr);

VOID
MyEvtDeviceFileCreate(_In_ WDFDEVICE Device,
                      _In_ WDFREQUEST Request,
                      _In_ WDFFILEOBJECT FileObject)
{
        PMY_FILE_CONTEXT fc = MyGetFileContext(FileObject);

        fc->OpenTime = KeQueryInterruptTime();
        fc->Subscribed = FALSE;

        WdfRequestComplete(Request, STATUS_SUCCESS);
        /* complete it to ACCEPT the open; complete with an
           error to REFUSE it                                */
}

VOID
MyEvtFileCleanup(_In_ WDFFILEOBJECT FileObject)
{
        /* Last handle in the process closed. CANCEL this
           handle's pending I/O here - the framework already
           purges requests owned by this file object from its
           queues, but anything YOU are holding is your
           responsibility.                                   */
}

/* THE FRAMEWORK AUTOMATICALLY completes requests belonging to
   a closing file object with STATUS_CANCELLED - which is the
   IRP_MJ_CLEANUP obligation from chapter 04, handled for
   you.                                                      */
```

Debugging queues

```
# The single most useful command for "requests are stuck":
    !wdfkd.wdfdevicequeues <WDFDEVICE>

      Queue: 0x0000... State: Accepting Requests, Dispatching
      Type: Parallel, PowerManaged: TRUE
      Requests: 0

      Queue: 0x0000... State: Accepting Requests, NOT Dispatching
      Type: Manual, PowerManaged: FALSE
      Requests: 3               <-- three pending reads
        WDFREQUEST 0x0000...
        WDFREQUEST 0x0000...

# READ THE STATE:
#   "NOT Dispatching" on a power-managed queue -> the device
#     is not in D0. Expected while idle; a bug if it never
#     resumes.
#   Requests climbing on a manual queue -> nothing is
#     retrieving them. Your DPC is not running, or the
#     hardware stopped.
#   Requests on the default queue -> your callback is not
#     completing or forwarding them.

    !wdfqueue <WDFQUEUE>            one queue in detail
    !wdfrequest <WDFREQUEST>        one request's state,
                                    owner, and IRP
    !wdflogdump mydriver            the framework's own trace

# AND FROM USER MODE, the symptom to correlate: a thread stuck
# in ReadFile. Find its IRP with !irpfind (chapter 04), then
# !wdfrequest on the corresponding WDFREQUEST.
```

How it shows up

Symptom	Cause
Application hangs in `ReadFile`	Request retrieved and never completed — you own it
Requests pile up on a manual queue	Nothing retrieving them; the DPC is not firing
Requests hang after the device idles	Power-managed manual queue with nothing to restart it
Requests arrive while the device is off	Queue not power-managed when it should be
Machine will not sleep	No `EvtIoStop`; the framework waits for an in-flight request
Bugcheck completing a cancelled request	Ignored the return of `WdfRequestUnmarkCancelable`
Corruption with a parallel queue	Concurrency not handled — use sequential or add locking
Byte count is zero in userspace	`Information` not set — use `CompleteWithInformation`
Two handles interfere with each other	Per-handle state kept in the device context
Requests leak on handle close	Holding a request outside a queue without cleanup handling
`WdfRequestRetrieveOutputBuffer` fails	Buffer shorter than your minimum — that check is the point

Check yourself

- ✓ Compare the three dispatch types and say when each is right.
- ✓ Draw the default-plus-manual-queue architecture and explain what it buys.
- ✓ Explain what "power managed" changes, and both failure modes of choosing wrong.
- ✓ State the request ownership rule and its consequence.
- ✓ Write a read callback that completes immediately or parks the request.
- ✓ Retrieve and complete a request from a DPC.
- ✓ Use the cancellable protocol correctly, including both return codes.
- ✓ Explain what `EvtIoStop` prevents and when it is required.
- ✓ Keep per-handle state and say what the framework does on cleanup.
- ✓ Diagnose stuck requests with `!wdfdevicequeues`.

IOCTLs and `DeviceIoControl`

The control interface between your driver and user mode. The control code encodes the device type, the access required and the buffering method — and because it is a permanent ABI reachable by whoever can open your device, it is where driver security is won or lost.

The control code

```
#define CTL_CODE(DeviceType, Function, Method, Access) ( \
        ((DeviceType) << 16) | ((Access) << 14) |         \
        ((Function) << 2)    | (Method) )

/* 32 bits, four fields:

     31..16  DeviceType   FILE_DEVICE_UNKNOWN (0x22) for a
                          custom device
     15..14  Access       FILE_ANY_ACCESS
                          FILE_READ_ACCESS
                          FILE_WRITE_ACCESS
                          FILE_READ_ACCESS|FILE_WRITE_ACCESS
     13..2   Function     YOURS. 0x800-0xFFF is the range
                          reserved for third parties.
                          Below 0x800 is Microsoft's.
      1..0   Method       METHOD_BUFFERED     0
                          METHOD_IN_DIRECT    1
                          METHOD_OUT_DIRECT   2
                          METHOD_NEITHER      3   */

/* A realistic set: */

#define MY_DEVICE_TYPE  0x8000    /* >= 0x8000 for vendors */

#define IOCTL_MY_GET_VERSION \
    CTL_CODE(MY_DEVICE_TYPE, 0x800, METHOD_BUFFERED, \
             FILE_ANY_ACCESS)

#define IOCTL_MY_GET_CONFIG \
    CTL_CODE(MY_DEVICE_TYPE, 0x801, METHOD_BUFFERED, \
             FILE_READ_ACCESS)

#define IOCTL_MY_SET_CONFIG \
    CTL_CODE(MY_DEVICE_TYPE, 0x802, METHOD_BUFFERED, \
             FILE_WRITE_ACCESS)

#define IOCTL_MY_READ_BULK \
    CTL_CODE(MY_DEVICE_TYPE, 0x803, METHOD_OUT_DIRECT, \
             FILE_READ_ACCESS)

#define IOCTL_MY_RESET \
    CTL_CODE(MY_DEVICE_TYPE, 0x804, METHOD_BUFFERED, \
             FILE_WRITE_ACCESS)
```

THE ACCESS FIELD IS ENFORCED BY THE I/O MANAGER

An IOCTL declared `FILE_WRITE_ACCESS` is **rejected before your driver sees it** if the handle was not opened with write access. This is free, automatic, correct access control — and it is why `FILE_ANY_ACCESS` should be reserved for genuinely harmless queries. Marking a "reconfigure the device" IOCTL as `FILE_ANY_ACCESS` means any process that can open the device read-only can reconfigure it, which is precisely the bug Linux chapter 21 warns about — except here the platform would have prevented it for free.

The handler

```
VOID
MyEvtIoDeviceControl(_In_ WDFQUEUE Queue,
                     _In_ WDFREQUEST Request,
                     _In_ size_t OutputBufferLength,
                     _In_ size_t InputBufferLength,
                     _In_ ULONG IoControlCode)
{
        WDFDEVICE device = WdfIoQueueGetDevice(Queue);
        PMY_CONTEXT ctx = MyGetContext(device);
        NTSTATUS status = STATUS_INVALID_DEVICE_REQUEST;
        size_t info = 0;
        PVOID inBuf, outBuf;
        size_t inLen, outLen;

        switch (IoControlCode) {

        case IOCTL_MY_GET_VERSION: {
                PMY_VERSION ver;

                status = WdfRequestRetrieveOutputBuffer(Request,
                                sizeof(MY_VERSION),
                                (PVOID *)&ver, &outLen);
                if (!NT_SUCCESS(status))
                        break;

                ver->Major = MY_VERSION_MAJOR;
                ver->Minor = MY_VERSION_MINOR;
                info = sizeof(MY_VERSION);
                status = STATUS_SUCCESS;
                break;
        }

        case IOCTL_MY_SET_CONFIG: {
                PMY_CONFIG cfg;
                MY_CONFIG local;

                status = WdfRequestRetrieveInputBuffer(Request,
                                sizeof(MY_CONFIG),
                                (PVOID *)&cfg, &inLen);
                if (!NT_SUCCESS(status))
                        break;

                /* COPY IN before validating. With
                   METHOD_BUFFERED this is already a kernel
                   copy, but copying to a local also protects
                   against the shared input/output buffer
                   overwriting your input (chapter 07).    */
                RtlCopyMemory(&local, cfg, sizeof(local));

                /* VALIDATE EVERY FIELD */
                if (local.SampleRate == 0 ||
                    local.SampleRate > MY_MAX_RATE) {
                        status = STATUS_INVALID_PARAMETER;
                        break;
                }
                if (local.ChannelMask & ~MY_VALID_CHANNELS) {
                        status = STATUS_INVALID_PARAMETER;
                        break;
                }
                if (local.Gain > MY_MAX_GAIN) {
```

```
                        status = STATUS_INVALID_PARAMETER;
                        break;
                }

                status = MyApplyConfig(ctx, &local);
                break;
        }

        case IOCTL_MY_READ_BULK: {
                PMDL mdl;

                status = WdfRequestRetrieveOutputWdmMdl(Request,
                                                        &mdl);
                if (!NT_SUCCESS(status))
                        break;

                /* start a DMA transfer into it (chapter 18),
                   and complete the request in the DPC.
                   Forward to the manual queue and return. */
                status = WdfRequestForwardToIoQueue(Request,
                                ctx->PendingQueue);
                if (NT_SUCCESS(status))
                        return;      /* do NOT complete here */
                break;
        }

        default:
                status = STATUS_INVALID_DEVICE_REQUEST;
                break;
        }

        WdfRequestCompleteWithInformation(Request, status, info);
}

/* NOTE the early `return` in the bulk case: a request that
   has been forwarded is no longer yours to complete
   (chapter 15). Falling through to the completion call would
   be a double completion and a bugcheck.               */
```

Internal IOCTLs

```
/* IRP_MJ_INTERNAL_DEVICE_CONTROL is reachable ONLY from
   kernel mode. Use it for driver-to-driver interfaces.    */

        queueConfig.EvtIoInternalDeviceControl =
                        MyEvtIoInternalDeviceControl;

/* WHY IT MATTERS FOR SECURITY: an internal IOCTL cannot be
   invoked by a user-mode program at all - the I/O manager
   refuses. So a privileged operation exposed only as an
   internal IOCTL has no user-mode attack surface.

   USE IT for anything a peer driver needs but userspace must
   never reach: raw register access, firmware update entry
   points, diagnostic modes.

/* AND THE TRAP: an internal IOCTL's buffers come from a
   kernel caller, so RequestorMode is KernelMode and probing
   is skipped. That is correct - but it means you are
   trusting the calling driver completely. Do not expose an
   internal IOCTL that a user-mode-reachable driver simply
   forwards, because you have then re-opened the surface you
   were closing.                                            */
```

The user-mode side

```
#include <windows.h>
#include <setupapi.h>
#include <cfgmgr32.h>
#include "mydevice.h"      /* the shared header */

/* --- find the device by interface GUID (chapter 05) ------ */
static HANDLE OpenMyDevice(void)
{
        HDEVINFO info;
        SP_DEVICE_INTERFACE_DATA ifData = { sizeof(ifData) };
        PSP_DEVICE_INTERFACE_DETAIL_DATA_W detail = NULL;
        DWORD needed = 0;
        HANDLE h = INVALID_HANDLE_VALUE;

        info = SetupDiGetClassDevsW(&GUID_DEVINTERFACE_MYDEVICE,
                        NULL, NULL,
                        DIGCF_PRESENT | DIGCF_DEVICEINTERFACE);
        if (info == INVALID_HANDLE_VALUE)
                return INVALID_HANDLE_VALUE;

        if (SetupDiEnumDeviceInterfaces(info, NULL,
                        &GUID_DEVINTERFACE_MYDEVICE, 0,
                        &ifData)) {

                SetupDiGetDeviceInterfaceDetailW(info, &ifData,
                                NULL, 0, &needed, NULL);
                detail = malloc(needed);
                if (detail) {
                        detail->cbSize = sizeof(*detail);
                        if (SetupDiGetDeviceInterfaceDetailW(
                                info, &ifData, detail, needed,
                                &needed, NULL)) {
                                h = CreateFileW(detail->DevicePath,
                                        GENERIC_READ | GENERIC_WRITE,
                                        0, NULL, OPEN_EXISTING,
                                        FILE_ATTRIBUTE_NORMAL, NULL);
                        }
                        free(detail);
                }
        }

        SetupDiDestroyDeviceInfoList(info);
        return h;
}

/* --- a synchronous IOCTL --------------------------------- */
        MY_CONFIG cfg = { .SampleRate = 48000,
                          .ChannelMask = 0x0F, .Gain = 3 };
        DWORD returned;

        if (!DeviceIoControl(h, IOCTL_MY_SET_CONFIG,
                             &cfg, sizeof(cfg),     /* in  */
                             NULL, 0,               /* out */
                             &returned, NULL)) {
                printf("SET_CONFIG failed: %lu\n",
                       GetLastError());
        }

/* --- an asynchronous one --------------------------------- */
        HANDLE h = CreateFileW(path, GENERIC_READ | GENERIC_WRITE,
```

```
                        0, NULL, OPEN_EXISTING,
                        FILE_FLAG_OVERLAPPED,    /* <-- async */
                        NULL);

        OVERLAPPED ov = {0};
        ov.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
        BYTE buffer[4096];

        if (!DeviceIoControl(h, IOCTL_MY_READ_BULK,
                             NULL, 0,
                             buffer, sizeof(buffer),
                             NULL, &ov)) {
                if (GetLastError() != ERROR_IO_PENDING) {
                        /* a real error */
                }
        }

        /* do other work, then: */
        WaitForSingleObject(ov.hEvent, INFINITE);

        DWORD transferred;
        if (GetOverlappedResult(h, &ov, &transferred, FALSE))
                printf("read %lu bytes\n", transferred);

        CloseHandle(ov.hEvent);

/* THE PENDING CONVENTION: DeviceIoControl returns FALSE with
   GetLastError() == ERROR_IO_PENDING when the driver returned
   STATUS_PENDING. That is SUCCESS, not failure - a mistake
   almost every first async client makes.                */
```

Designing the interface

RULES, IN ORDER OF HOW MUCH DAMAGE BREAKING THEM CAUSES

1. **Validate every field of every input.** The caller is untrusted, always, even if today only your own application calls it.
2. **Use the Access field.** Free enforcement; use it rather than checking by hand.
3. **Fixed-width types, explicit padding.** `UINT32`, not `ULONG` or `int`; pad so the layout is identical for 32- and 64-bit callers.
4. **Version the interface** from day one, with a `GET_VERSION` IOCTL that never changes.
5. **Reserved fields, required to be zero.** They cost nothing and are the only way to extend a struct later.
6. **Never embed a pointer** in a buffer. It breaks 32-on-64 and it is a confused-deputy invitation.
7. **Bound every length.** A size field that drives an allocation is a denial-of-service vector.
8. **Return** `STATUS_INVALID_DEVICE_REQUEST` for unknown codes, so callers can probe for support.
9. **Keep the surface small.** Every IOCTL is an entry point into kernel mode from whoever can open the device.

```
/* THE 32-ON-64 PROBLEM, and why fixed-width types matter: */

/* WRONG - different layout for a 32-bit caller: */
typedef struct {
        ULONG   Count;
        PVOID   Buffer;     /* 4 bytes on Win32, 8 on Win64 */
        SIZE_T  Length;     /* same problem                 */
} BAD_REQUEST;

/* RIGHT: */
typedef struct {
        UINT32  Count;
        UINT32  Reserved;   /* explicit padding */
        UINT64  BufferOffset;   /* an OFFSET, not a pointer */
        UINT32  Length;
        UINT32  Reserved2[3];
} GOOD_REQUEST;

/* AND IF YOU MUST support both layouts, detect the caller: */
        if (IoIs32bitProcess(NULL)) {
                /* the request came from a 32-bit process -
                   parse the 32-bit layout                  */
        }

/* THE BETTER ANSWER: design so there is only one layout.
   Fixed-width types and no pointers achieves it, exactly as
   in Linux chapter 21.                                     */
```

Testing

```
# NEGATIVE TESTS - these find the bugs:
    · an unknown control code       -> STATUS_INVALID_DEVICE_REQUEST
    · a zero-length input buffer     -> STATUS_BUFFER_TOO_SMALL
    · an input buffer one byte short -> same
    · a NULL output buffer           -> handled, not a crash
    · out-of-range field values      -> STATUS_INVALID_PARAMETER
    · a huge length field            -> refused, not a hang
    · SET_CONFIG on a read-only handle -> refused by the I/O
                                          manager
    · concurrent calls from many threads -> no corruption
    · a 32-bit client, if you support one

# TOOLS:
    devcon.exe                  install, enable, disable
    Device Fundamentals tests   in the WDK - includes
                                fuzzing of your IOCTL
                                interface. REQUIRED for
                                certification anyway, so run
                                them early (chapter 21).
    IoSpy / IoAttack            the WDK's IOCTL fuzzer

# IoAttack sends malformed buffers, wrong lengths and random
# control codes at your device. With Driver Verifier enabled
# it finds unvalidated input in minutes. Run it before anyone
# else does.
```

How it shows up

Symptom	Cause
`DeviceIoControl` fails with ERROR_ACCESS_DENIED	Handle lacks the access the control code requires — usually correct
Fails with ERROR_INVALID_FUNCTION	Control code mismatch between client and driver headers
Fails with ERROR_IO_PENDING	Not a failure — asynchronous completion
Output buffer unchanged	`Information` not set on completion
Input corrupted mid-handler	METHOD_BUFFERED shares one buffer — copy input to a local first
Bugcheck from a user-mode call	Unvalidated field — a security bug, not just a crash
Works from a 64-bit client, garbage from 32-bit	Pointer or `SIZE_T` in the struct
Double-completion bugcheck	Completed a request already forwarded to another queue
Read-only handle reconfigures the device	`FILE_ANY_ACCESS` on a privileged operation
Kernel memory exhausted by a user program	Unbounded length field
Cannot extend the struct in v2	No reserved fields in v1

Check yourself

- ✓ Decode a control code into its four fields.
- ✓ Explain what the Access field enforces and who enforces it.
- ✓ Write a handler with correct validation, completion and forwarding.
- ✓ Explain why a forwarded request must not be completed.
- ✓ Say what internal IOCTLs are for and the trap they carry.
- ✓ Open a device by interface GUID from user mode.
- ✓ Issue a synchronous and an asynchronous IOCTL, handling `ERROR_IO_PENDING` correctly.
- ✓ List the nine interface design rules.
- ✓ Explain the 32-on-64 layout problem and design a struct that avoids it.
- ✓ Run the negative test list and the WDK's IOCTL fuzzer.

Hardware resources and interrupts in WDF

How your driver learns where its registers are and which interrupt is its own, how to map and access them safely, and the WDF interrupt object that connects the ISR and DPC from chapter 10 to real silicon.

The resource list

```
/* PnP assigns resources and hands you two lists in
   EvtDevicePrepareHardware (chapter 14):

     ResourcesRaw         bus-relative addresses, as the
                          device sees them
     ResourcesTranslated  system addresses the CPU can use

   USE THE TRANSLATED LIST for mapping. Use the RAW list only
   when you must program a bus-relative value into the device
   itself - which is rare on modern systems.              */

NTSTATUS
MyPrepareHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesRaw,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        ULONG i, count;

        PAGED_CODE();
        UNREFERENCED_PARAMETER(ResourcesRaw);

        count = WdfCmResourceListGetCount(ResourcesTranslated);

        for (i = 0; i < count; i++) {
                PCM_PARTIAL_RESOURCE_DESCRIPTOR desc =
                        WdfCmResourceListGetDescriptor(
                                ResourcesTranslated, i);

                switch (desc->Type) {

                case CmResourceTypeMemory:
                case CmResourceTypeMemoryLarge:
                        /* desc->u.Memory.Start   PHYSICAL_ADDRESS
                           desc->u.Memory.Length  ULONG          */
                        if (ctx->RegBase == NULL) {
                                ctx->RegLength =
                                        desc->u.Memory.Length;
                                ctx->RegBase = (PUCHAR)
                                        MmMapIoSpaceEx(
                                          desc->u.Memory.Start,
                                          desc->u.Memory.Length,
                                          PAGE_READWRITE |
                                          PAGE_NOCACHE);
                                if (!ctx->RegBase)
                                        return STATUS_INSUFFICIENT_RESOURCES;
                        }
                        break;

                case CmResourceTypePort:
                        /* legacy I/O ports - x86 only */
                        ctx->PortBase =
                                (PUCHAR)(ULONG_PTR)
                                desc->u.Port.Start.QuadPart;
                        break;

                case CmResourceTypeInterrupt:
                        /* the WDFINTERRUPT created in
                           EvtDeviceAdd is matched to this
                           automatically - nothing to do   */
```

```
                        ctx->InterruptVector =
                                desc->u.Interrupt.Vector;
                        break;

                case CmResourceTypeDma:
                        break;      /* chapter 18 */

                case CmResourceTypeConnection:
                        /* a GPIO, I2C or SPI connection, on
                           ACPI-enumerated ARM64 systems.
                           u.Connection.IdLowPart /
                           IdHighPart form a resource hub ID
                           you open with a target
                           (chapter 12's WDFIOTARGET).      */
                        ctx->ConnectionId.LowPart =
                                desc->u.Connection.IdLowPart;
                        ctx->ConnectionId.HighPart =
                                desc->u.Connection.IdHighPart;
                        break;
                }
        }

        if (ctx->RegBase == NULL)
                return STATUS_DEVICE_CONFIGURATION_ERROR;

        return STATUS_SUCCESS;
}

/* NEVER ASSUME THE ORDER OR COUNT of descriptors. A device
   with two memory ranges may report them in either order on
   different firmware. Match on what you find, and fail
   cleanly if something expected is absent - that is what
   turns a mysterious later crash into a clear Code 10 at
   install time.                                            */
```

Register access

```
/* Use the READ_/WRITE_REGISTER_* macros, not raw pointer
   dereferences - the same argument as Linux's readl/writel
   (ch. 23): they guarantee width, ordering and no compiler
   reordering.                                              */

        UCHAR  b = READ_REGISTER_UCHAR (ctx->RegBase + 0x00);
        USHORT w = READ_REGISTER_USHORT((PUSHORT)(ctx->RegBase + 0x02));
        ULONG  l = READ_REGISTER_ULONG ((PULONG)(ctx->RegBase + 0x04));
        ULONG64 q= READ_REGISTER_ULONG64((PULONG64)(ctx->RegBase + 0x08));

        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + 0x04), value);

        /* bulk, for FIFOs: */
        READ_REGISTER_BUFFER_ULONG((PULONG)(ctx->RegBase + FIFO),
                                   buffer, count);
        WRITE_REGISTER_BUFFER_ULONG((PULONG)(ctx->RegBase + FIFO),
                                    buffer, count);

/* FOR LEGACY I/O PORTS: */
        READ_PORT_ULONG(), WRITE_PORT_ULONG(), and the
        _BUFFER_ variants.

/* MEMORY BARRIERS, when ordering matters beyond a single
   access: */
        KeMemoryBarrier();           /* full */
        MemoryBarrier();
        _ReadWriteBarrier();         /* compiler only */

/* AND THE DMA CASE: after filling a buffer and before the
   doorbell write, you need the data visible to the device
   first. On Windows the DMA API's flush routines handle this
   (chapter 18); for a common buffer, KeMemoryBarrier() before
   the doorbell is the portable form.                       */

/* A HELPER PAIR worth writing, because it makes every call
   site readable: */
static FORCEINLINE ULONG
MyRd(PMY_CONTEXT ctx, ULONG reg)
{
        return READ_REGISTER_ULONG((PULONG)(ctx->RegBase + reg));
}

static FORCEINLINE VOID
MyWr(PMY_CONTEXT ctx, ULONG reg, ULONG val)
{
        WRITE_REGISTER_ULONG((PULONG)(ctx->RegBase + reg), val);
}
```

Creating the interrupt object

```
/* In EvtDeviceAdd - BEFORE resources are assigned. WDF
   matches it to the interrupt descriptor later.           */

        WDF_INTERRUPT_CONFIG interruptConfig;
        WDF_OBJECT_ATTRIBUTES attributes;

        WDF_INTERRUPT_CONFIG_INIT(&interruptConfig,
                                  MyEvtInterruptIsr,
                                  MyEvtInterruptDpc);

        /* optional enable/disable callbacks, called around
           power transitions (chapter 14) */
        interruptConfig.EvtInterruptEnable  = MyIntEnable;
        interruptConfig.EvtInterruptDisable = MyIntDisable;

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attributes,
                                        MY_INTERRUPT_CONTEXT);

        status = WdfInterruptCreate(device, &interruptConfig,
                                    &attributes,
                                    &ctx->Interrupt);

/* THE ENABLE/DISABLE CALLBACKS run at DIRQL with the
   interrupt lock held, and are the right place to touch the
   device's interrupt-enable register: */

NTSTATUS
MyIntEnable(_In_ WDFINTERRUPT Interrupt, _In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        MyWr(ctx, REG_INT_ENABLE, INT_ALL);
        return STATUS_SUCCESS;
}

NTSTATUS
MyIntDisable(_In_ WDFINTERRUPT Interrupt, _In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);

        MyWr(ctx, REG_INT_ENABLE, 0);
        return STATUS_SUCCESS;
}

/* WDF calls Enable after connecting the interrupt and Disable
   before disconnecting it - which is exactly the ordering
   that avoids a spurious interrupt with no handler
   (chapter 14).                                            */
```

Message-signalled interrupts

```
/* MSI/MSI-X: a PCIe device signals by WRITING to a special
   address rather than asserting a line.

   ADVANTAGES:
     · no sharing, so no "is it mine?" check needed
     · several messages per device, each with its own ISR -
       one per queue, one per channel
     · inherently ordered with respect to the DMA writes that
       preceded them, which removes a whole class of
       "interrupt arrived before the data" races           */

/* THE INF requests them: */
        [MyDevice_AddReg]
        HKR,"Interrupt Management",,0x00000010
        HKR,"Interrupt Management\MessageSignaledInterruptProperties",,0x00000010
        HKR,"Interrupt Management\MessageSignaledInterruptProperties",\
            MSISupported,0x00010001,1
        HKR,"Interrupt Management\MessageSignaledInterruptProperties",\
            MessageNumberLimit,0x00010001,4

/* AND THE DRIVER creates one WDFINTERRUPT PER MESSAGE: */
        for (i = 0; i < MY_NUM_MESSAGES; i++) {
                WDF_INTERRUPT_CONFIG_INIT(&interruptConfig,
                                MyEvtInterruptIsr,
                                MyEvtInterruptDpc);

                status = WdfInterruptCreate(device,
                                &interruptConfig, &attributes,
                                &ctx->Interrupts[i]);
                if (!NT_SUCCESS(status))
                        return status;

                ictx = MyGetInterruptContext(ctx->Interrupts[i]);
                ictx->MessageIndex = i;
        }

/* The ISR receives the message id: */
BOOLEAN
MyEvtInterruptIsr(_In_ WDFINTERRUPT Interrupt,
                  _In_ ULONG MessageID)
{
        /* With MSI, MessageID identifies WHICH message fired -
           so you often need no status-register read at all,
           which makes the ISR dramatically cheaper.       */
        ...
}

/* IF FEWER MESSAGES ARE GRANTED than you asked for, the
   remaining ones map onto message 0. Handle that: check
   WdfInterruptGetInfo() for what you actually got, and do not
   assume the count.                                       */

        WDF_INTERRUPT_INFO info;

        WDF_INTERRUPT_INFO_INIT(&info);
        WdfInterruptGetInfo(ctx->Interrupts[0], &info);
        /* info.MessageSignaled, info.MessageNumber,
           info.Vector, info.Irql, info.TargetProcessorSet  */
```

MSI REMOVES THE SHARED-LINE PROBLEM

A line-based interrupt may be shared, so every ISR must read a status register to ask "is this mine?" — and that read can cost microseconds on PCIe. An MSI is by definition yours, and the message number tells you what happened. On any device where you control the hardware design or the interrupt count, MSI-X with one message per queue is both faster and simpler.

Synchronising with the ISR

```
/* Data shared between the ISR and everything else needs the
   interrupt lock (chapter 09).                              */

/* Option 1: acquire it around a section */
        WdfInterruptAcquireLock(ctx->Interrupt);
        ctx->SharedState = newValue;
        MyWr(ctx, REG_CONFIG, newValue);
        WdfInterruptReleaseLock(ctx->Interrupt);

/* Option 2 - PREFERRED: run a small function at DIRQL with
   the lock held: */
static BOOLEAN
MySetEnableSync(_In_ WDFINTERRUPT Interrupt,
                _In_ WDFCONTEXT Context)
{
        PMY_CONTEXT ctx = (PMY_CONTEXT)Context;

        ctx->Enabled = TRUE;
        MyWr(ctx, REG_INT_ENABLE, INT_ALL);
        return TRUE;
}

        WdfInterruptSynchronize(ctx->Interrupt,
                                MySetEnableSync, ctx);

/* WHY OPTION 2 IS BETTER: the critical section is a named
   function whose entire body is visible, so it is obvious it
   is short and does nothing forbidden at DIRQL. An
   Acquire/Release pair invites someone to add a line in the
   middle later.                                              */

/* NOTE: with a PASSIVE-LEVEL interrupt (chapter 10), the
   interrupt "lock" is a passive-level lock and you MAY sleep
   inside it. WdfInterruptSynchronize still works and still
   serialises correctly.                                      */
```

Interrupt affinity and policy

```
/* Which CPU handles the interrupt, and at what priority.
   Set from the INF, not from code - it is a system policy
   decision, and the administrator may override it.        */

        [MyDevice_AddReg]
        HKR,"Interrupt Management\Affinity Policy",\
            DevicePolicy,0x00010001,5    ; IrqPolicySpreadMessagesAcrossAllProcessors
        HKR,"Interrupt Management\Affinity Policy",\
            DevicePriority,0x00010001,3  ; IrqPriorityHigh

/* THE POLICIES:
     0  MachineDefault
     1  AllCloseProcessors
     2  OneCloseProcessor
     3  AllProcessorsInMachine
     4  SpecifiedProcessors
     5  SpreadMessagesAcrossAllProcessors   <-- for MSI-X

/* SPREADING MSI-X MESSAGES across CPUs is what lets a
   multi-queue device scale: each message's DPC runs on a
   different core, so DPC processing is parallel rather than
   serialised on one CPU (chapter 10).

/* FROM CODE, if you must: */
        WdfInterruptSetPolicy(ctx->Interrupt,
                        WdfIrqPolicySpecifiedProcessors,
                        WdfIrqPriorityNormal,
                        1 << 2);        /* CPU 2 */

/* And read what you got: */
        WdfInterruptGetInfo(interrupt, &info);
        /* info.TargetProcessorSet */
```

Debugging resources and interrupts

```
# --- Device Manager ---
#   Properties -> Resources tab: exactly what was assigned.
#   If a range or IRQ you expect is missing, the problem is in
#   the firmware or the INF, not your driver.

# --- WinDbg ---
    !devnode 0 1 mydriver           the PnP node
    !devobj ffff8a0c...             the device object
    !arbiter 4                      the resource arbiters -
                                    who owns which ranges
    !idt                            the interrupt dispatch
                                    table: which vector goes
                                    where
    !wdfkd.wdfdriverinfo mydriver 0x10   WDF's view of the
                                    interrupt objects

# --- Is my interrupt firing at all? ---
#   Add a counter in the ISR and expose it in a debug IOCTL or
#   read it with WinDbg. If it never increments:
#     · the device is not enabled - check REG_INT_ENABLE
#     · the interrupt was never connected - check the
#       WdfInterruptCreate return
#     · the resource descriptor was absent - check
#       PrepareHardware
#     · the device is not clocked or not out of reset
#     · MSI was requested but not granted, and you assumed it
#
#   THE SAME CHECKLIST as Linux chapter 14, in a different
#   dialect.

# --- ETW tracing of interrupts and DPCs ---
    wpr -start GeneralProfile -start DeviceIO
    ... reproduce ...
    wpr -stop trace.etl
    wpa trace.etl
# The "DPC/ISR" graph shows every ISR and DPC with durations.
# It is the tool for "why is this system's latency bad", and
# it will show your DPC's cost next to everyone else's.
```

How it shows up

Symptom	Cause
Code 10 at start	`PrepareHardware` returned an error — usually a missing expected resource
Registers read `0xFFFFFFFF`	Wrong range mapped, device not powered, or surprise-removed
Mapped the wrong range	Assumed descriptor order — match on type and index deliberately
Interrupt never fires	Device's interrupt enable not set, or the resource was absent
Interrupt storm	ISR not acknowledging the device
Spurious interrupt during power-down	Device not silenced before disconnect — use `EvtInterruptDisable`
Only message 0 ever fires	Fewer MSI messages granted than requested
All DPCs on one CPU	No affinity policy — MSI-X messages not spread
Corruption between ISR and normal code	Interrupt lock not used

Symptom	Cause
Bugcheck 0xA in the ISR	Paged memory or a paged-pool allocation at DIRQL
Works on one machine, Code 12 on another	Resource conflict — the arbiter could not satisfy the request

Check yourself

- ✓ Distinguish raw from translated resources and say which to map.
- ✓ Walk a resource list and handle five descriptor types.
- ✓ Explain why you must not assume descriptor order or count.
- ✓ Use the register access macros and say why a raw dereference is wrong.
- ✓ Create a WDF interrupt object with enable and disable callbacks, and explain the ordering they give.
- ✓ Explain what MSI changes about ISR design.
- ✓ Request MSI-X from an INF and handle getting fewer messages than asked.
- ✓ Use `WdfInterruptSynchronize` and say why it beats an acquire/release pair.
- ✓ Set an interrupt affinity policy and explain why spreading MSI-X matters.
- ✓ Work the checklist for an interrupt that never fires.

DMA in WDF

Giving a Windows device direct memory access: the DMA enabler, transactions, common buffers and scatter-gather. The same problem as Linux chapter 24, with a framework that manages the state machine for you.

The model

```
WDFDMAENABLER          created once, in PrepareHardware.
     |                 describes the device's DMA
     |                 capabilities: direction, maximum
     |                 transfer length, alignment,
     |                 address width
     |
     +-- WDFDMATRANSACTION   one per logical transfer
     |        |              May be split into several
     |        |              hardware transfers if the
     |        |              buffer exceeds what the device
     |        |              can do in one go.
     |        |
     |        +-- SCATTER/GATHER LIST
     |               the physical fragments, ready to
     |               program into the device
     |
     +-- WDFCOMMONBUFFER    memory the CPU and device both
                            access continuously - rings,
                            descriptors, status blocks

THE THREE ADDRESS SPACES are the same as Linux's
(ch. 24): virtual, physical, and LOGICAL (Windows' name
for the device-visible address). The DMA enabler is what
knows the mapping, including any IOMMU translation - so you
program the device with the LOGICAL address the framework
gives you, never a physical one you computed.
```

Creating the enabler

```
NTSTATUS
MyPrepareHardware(_In_ WDFDEVICE Device,
                  _In_ WDFCMRESLIST ResourcesRaw,
                  _In_ WDFCMRESLIST ResourcesTranslated)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        WDF_DMA_ENABLER_CONFIG dmaConfig;
        NTSTATUS status;

        ... map registers (chapter 17) ...

        /* 1. DECLARE THE ADDRESS WIDTH. This is the single
              most consequential call: get it wrong and the
              hardware silently drops the high address bits.
              Same failure as a wrong dma_set_mask on Linux -
              works on a small-memory machine, corrupts
              random memory on a large one.               */
        status = WdfDeviceSetAlignmentRequirement(Device,
                        FILE_LONG_ALIGNMENT);   /* 4-byte */
        if (!NT_SUCCESS(status))
                return status;

        WDF_DMA_ENABLER_CONFIG_INIT(&dmaConfig,
                        WdfDmaProfileScatterGather64,
                        MY_MAX_TRANSFER_LENGTH);

        dmaConfig.WdmDmaVersionOverride = 3;

        status = WdfDmaEnablerCreate(Device, &dmaConfig,
                        WDF_NO_OBJECT_ATTRIBUTES,
                        &ctx->DmaEnabler);
        if (!NT_SUCCESS(status))
                return status;

        return STATUS_SUCCESS;
}

/* THE PROFILES:

   WdfDmaProfilePacket           single contiguous buffer,
                                 32-bit addresses
   WdfDmaProfileScatterGather    scattered, 32-bit
   WdfDmaProfilePacket64         contiguous, 64-bit
   WdfDmaProfileScatterGather64  scattered, 64-bit   <-- the
                                 usual modern choice
   WdfDmaProfileSystem           uses a system DMA controller
                                 rather than bus mastering
   WdfDmaProfileSystemDuplex

   PICK 64-BIT ONLY IF THE DEVICE REALLY IS. Declaring 64 on a
   32-bit device is the classic silent corruption; declaring
   32 on a 64-bit device merely costs you bounce buffering,
   which is safe but slow.

/* AND THE MAXIMUM LENGTH: if a request exceeds it, the
   framework splits the transaction into several transfers
   automatically - which is why you must handle
   EvtProgramDma being called more than once per
   transaction.                                          */
```

A transaction

```
/* The common flow: an IOCTL or write arrives with a buffer,
   and you DMA it to the device.                          */

VOID
MyEvtIoWrite(_In_ WDFQUEUE Queue, _In_ WDFREQUEST Request,
             _In_ size_t Length)
{
        WDFDEVICE device = WdfIoQueueGetDevice(Queue);
        PMY_CONTEXT ctx = MyGetContext(device);
        WDFDMATRANSACTION transaction;
        NTSTATUS status;

        if (Length == 0 || Length > MY_MAX_TRANSFER_LENGTH) {
                WdfRequestComplete(Request,
                                   STATUS_INVALID_PARAMETER);
                return;
        }

        /* 1. create a transaction */
        status = WdfDmaTransactionCreate(ctx->DmaEnabler,
                        WDF_NO_OBJECT_ATTRIBUTES,
                        &transaction);
        if (!NT_SUCCESS(status)) {
                WdfRequestComplete(Request, status);
                return;
        }

        /* 2. bind it to the request's buffer. The framework
              gets the MDL, ensures the pages are locked, and
              builds the scatter-gather list.             */
        status = WdfDmaTransactionInitializeUsingRequest(
                        transaction, Request,
                        MyEvtProgramDma,
                        WdfDmaDirectionWriteToDevice);
        if (!NT_SUCCESS(status)) {
                WdfObjectDelete(transaction);
                WdfRequestComplete(Request, status);
                return;
        }

        /* remember it so the DPC can finish it */
        ctx->CurrentTransaction = transaction;

        /* 3. GO. This calls MyEvtProgramDma. */
        status = WdfDmaTransactionExecute(transaction,
                                          WDF_NO_CONTEXT);
        if (!NT_SUCCESS(status)) {
                WdfObjectDelete(transaction);
                WdfRequestComplete(Request, status);
                return;
        }

        /* the request completes later, in the DPC */
}

/* 4. PROGRAM THE HARDWARE - called once per hardware
      transfer, possibly SEVERAL times per transaction.  */
BOOLEAN
MyEvtProgramDma(_In_ WDFDMATRANSACTION Transaction,
                _In_ WDFDEVICE Device,
```

```
                _In_ WDFCONTEXT Context,
                _In_ WDF_DMA_DIRECTION Direction,
                _In_ PSCATTER_GATHER_LIST SgList)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        ULONG i;

        UNREFERENCED_PARAMETER(Context);
        UNREFERENCED_PARAMETER(Direction);

        /* Runs at DISPATCH_LEVEL. Program the descriptors and
           start the device. Do NOT block.                */

        for (i = 0; i < SgList->NumberOfElements; i++) {
                /* THE LOGICAL ADDRESS - what the device must
                   be programmed with.                      */
                MyWr(ctx, REG_DESC_LO(i),
                     SgList->Elements[i].Address.LowPart);
                MyWr(ctx, REG_DESC_HI(i),
                     SgList->Elements[i].Address.HighPart);
                MyWr(ctx, REG_DESC_LEN(i),
                     SgList->Elements[i].Length);
        }

        KeMemoryBarrier();      /* descriptors visible before
                                   the doorbell (chapter 17) */
        MyWr(ctx, REG_DMA_COUNT, SgList->NumberOfElements);
        MyWr(ctx, REG_DMA_START, 1);

        return TRUE;    /* FALSE aborts the transaction */
}

/* 5. IN THE DPC, when the hardware signals completion: */
VOID
MyEvtInterruptDpc(_In_ WDFINTERRUPT Interrupt,
                  _In_ WDFOBJECT AssociatedObject)
{
        PMY_CONTEXT ctx = MyGetContext(
                        WdfInterruptGetDevice(Interrupt));
        WDFDMATRANSACTION transaction = ctx->CurrentTransaction;
        WDFREQUEST request;
        NTSTATUS status;
        size_t transferred;
        BOOLEAN more;

        UNREFERENCED_PARAMETER(AssociatedObject);

        if (transaction == NULL)
                return;

        /* IS THERE MORE? A long buffer is split into several
           hardware transfers. DmaCompleted returns TRUE if
           the whole transaction is finished, FALSE if it
           queued the next transfer - in which case
           EvtProgramDma is called again and we must NOT
           complete the request.                          */
        more = !WdfDmaTransactionDmaCompleted(transaction,
                                              &status);
        if (more)
                return;

        transferred = WdfDmaTransactionGetBytesTransferred(
                                transaction);
```

```
        request = WdfDmaTransactionGetRequest(transaction);

        ctx->CurrentTransaction = NULL;
        WdfObjectDelete(transaction);   /* releases the SG list
                                           and unmaps        */

        if (request)
                WdfRequestCompleteWithInformation(request,
                                status, transferred);
}

/* THE MULTI-TRANSFER RULE IS THE ONE PEOPLE MISS. A driver
   that completes the request on the first DPC works
   perfectly for small buffers and truncates large ones -
   exactly the Linux scatter-gather coalescing trap
   (ch. 24), in a different form.                       */
```

DELETING THE TRANSACTION IS NOT OPTIONAL

`WdfObjectDelete` on the transaction is what releases the scatter-gather list, unmaps the buffer and returns the map registers to the system pool. Map registers are a finite, shared resource; leaking them degrades DMA for every device on the machine and eventually makes `WdfDmaTransactionExecute` fail with `STATUS_INSUFFICIENT_RESOURCES` — for someone else's driver.

Common buffers

```
/* For memory the CPU and device both touch continuously:
   descriptor rings, command queues, status blocks. The
   Windows equivalent of dma_alloc_coherent (Linux ch. 24). */

        WDFCOMMONBUFFER commonBuffer;
        PVOID va;
        PHYSICAL_ADDRESS la;

        status = WdfCommonBufferCreate(ctx->DmaEnabler,
                        MY_RING_SIZE,
                        WDF_NO_OBJECT_ATTRIBUTES,
                        &commonBuffer);
        if (!NT_SUCCESS(status))
                return status;

        va = WdfCommonBufferGetAlignedVirtualAddress(
                        commonBuffer);
        la = WdfCommonBufferGetAlignedLogicalAddress(
                        commonBuffer);

        RtlZeroMemory(va, MY_RING_SIZE);

        /* program the device with the LOGICAL address */
        MyWr(ctx, REG_RING_BASE_LO, la.LowPart);
        MyWr(ctx, REG_RING_BASE_HI, la.HighPart);

        /* the CPU uses `va` normally, at any IRQL */
        ring = (PMY_DESCRIPTOR)va;
        ring[0].Address = dataLa.QuadPart;
        ring[0].Length  = len;
        ring[0].Flags   = DESC_OWN;

        KeMemoryBarrier();
        MyWr(ctx, REG_DOORBELL, 0);

/* WITH SPECIFIC ALIGNMENT: */
        status = WdfCommonBufferCreateWithConfig(
                        ctx->DmaEnabler, size, &config,
                        &attributes, &commonBuffer);
        /* config.AlignmentRequirement = 4096 - 1; */

/* PROPERTIES:
     · physically contiguous
     · cache-coherent with the device, or mapped uncached if
       the platform is not coherent - so no explicit flushing
     · allocated at PrepareHardware time, so no runtime
       fragmentation risk
     · freed automatically when its parent dies

   USE IT FOR SMALL CONTROL STRUCTURES. On a non-coherent
   platform it is uncached, so CPU access is slow - bulk data
   belongs in a transaction, not a common buffer. Exactly the
   Linux coherent-versus-streaming trade (ch. 24).       */
```

Direct-programmed DMA

```
/* Some devices need the driver to manage the buffer
   lifecycle itself - a continuous capture ring, for example,
   where there is no per-request transaction.             */

        /* build a transaction from an MDL you own */
        status = WdfDmaTransactionInitialize(transaction,
                        MyEvtProgramDma,
                        WdfDmaDirectionReadFromDevice,
                        mdl,
                        virtualAddress,
                        length);

        /* or, for a buffer you will reuse indefinitely,
           allocate a common buffer and manage it yourself */

/* AND FOR "I just need the logical address of this buffer": */
        status = WdfDmaTransactionGetTransferInfo(transaction,
                        &mapRegisters, &sgElements);

/* THE HONEST ADVICE: if the framework's transaction model
   fits your device, use it - it handles splitting, map
   register allocation, cache flushing and cleanup. Reach for
   direct programming only when the device's model genuinely
   does not match request-oriented DMA, which in practice
   means continuous streaming devices: audio, video capture,
   and network receive rings.                             */
```

The IOMMU

```
/* Modern Windows enables DMA remapping (VT-d / SMMU) for
   kernel DMA protection. The consequences for your driver are
   the same as the Linux ones (isolation ch. 03-07):

   · the LOGICAL address the framework gives you is an IOVA,
     not a physical address. Programming a physical address
     you computed yourself will FAULT.

   · a transfer longer than what you mapped faults, rather
     than reading a neighbour's memory. That is a feature: it
     turns silent corruption into an attributable error.

   · using a logical address after deleting the transaction
     faults, because the mapping is gone.

   · the device must be correctly identified to the IOMMU by
     firmware. On a machine where it is not, DMA either
     bypasses protection entirely or fails outright.

/* SEEING IT: */
        WinDbg:  !iommu                (on supported builds)
        Event Viewer -> Microsoft-Windows-Kernel-IoTrace
        System event log: DMA remapping faults are logged

/* KERNEL DMA PROTECTION also affects HOT-PLUGGABLE buses
   (Thunderbolt): a device may be blocked from DMA entirely
   until the user authorises it. A driver must handle
   STATUS_DEVICE_NOT_READY gracefully rather than assuming
   DMA will always be permitted.                          */
```

Debugging DMA

```
# DRIVER VERIFIER's DMA checking (chapter 21) is the primary
# tool - the Windows counterpart of CONFIG_DMA_API_DEBUG:
    verifier /flags 0x10 /driver mydriver.sys
# It validates:
#   · map register accounting
#   · buffer alignment against the declared requirement
#   · transfers exceeding the declared maximum
#   · use after completion
#   · flushing rules
#   · adapter object misuse
# and bugchecks with 0xE6 DRIVER_VERIFIER_DMA_VIOLATION,
# naming the rule.

# --- WinDbg ---
    !wdfkd.wdfdriverinfo mydriver 0x20    DMA enabler state
    !dma                                  adapter objects and
                                          map register usage
    dt nt!_SCATTER_GATHER_LIST <address>

# --- The classic symptoms ---
#   data lands in the wrong place  -> you programmed a
#                                     physical rather than a
#                                     logical address
#   large transfers truncated      -> completed the request on
#                                     the first DPC
#   works on 4 GB, fails on 32 GB  -> declared 64-bit DMA on a
#                                     32-bit device
#   intermittent under load        -> missing memory barrier
#                                     before the doorbell
#   DMA fails for other drivers    -> you are leaking map
#                                     registers
```

How it shows up

Symptom	Cause
Data written to the wrong memory	Programmed a physical address instead of the logical one
IOMMU fault logged	Same — the physical address is not a valid IOVA
Works with 4 GB RAM, corrupts with more	Declared a 64-bit profile on a 32-bit device
Large transfers truncated	Request completed on the first DPC — check `DmaCompleted`
Device reads stale descriptors	Missing `KeMemoryBarrier()` before the doorbell
Bugcheck 0xE6	Verifier caught a DMA rule violation
`WdfDmaTransactionExecute` fails with insufficient resources	Map registers exhausted — usually your own leak
Other drivers' DMA starts failing	Leaked transactions — delete them
Alignment fault or corruption at buffer start	Alignment requirement not declared or not honoured
Common buffer access is very slow	Non-coherent platform maps it uncached — use it only for control structures
DMA blocked on a Thunderbolt device	Kernel DMA protection pending user authorisation

Check yourself

- ✓ Name the WDF DMA objects and what each is for.
- ✓ Choose a DMA profile and explain the consequence of declaring the wrong address width.
- ✓ Write the full transaction flow: create, initialise, execute, program, complete.
- ✓ Explain why `EvtProgramDma` may be called several times per transaction.
- ✓ Explain what `WdfDmaTransactionDmaCompleted` returns and why it matters.
- ✓ Say what deleting the transaction releases, and what leaking it costs the system.
- ✓ Use a common buffer and say when it is the right choice.
- ✓ Explain what the IOMMU changes and why a computed physical address now faults.
- ✓ Enable Verifier's DMA checking and name five rules it enforces.
- ✓ Map each concept onto its Linux counterpart.

INF files and installation

The INF tells Windows which hardware your driver serves, what to copy, and what registry state to create. It plays the role the device tree plays on Linux — describing this particular device — but applied once at install time rather than parsed at every boot.

Anatomy of an INF

```
;
; mydriver.inf - Vendor MyDevice
;

[Version]
Signature   = "$WINDOWS NT$"
Class       = System                ; or a custom class
ClassGuid   = {4d36e97d-e325-11ce-bfc1-08002be10318}
Provider    = %ManufacturerName%
CatalogFile = mydriver.cat          ; the signature catalog
DriverVer   = 03/15/2026,1.0.0.0    ; DATE,VERSION
PnpLockdown = 1                     ; REQUIRED on modern
                                    ; Windows - protects the
                                    ; driver's files

[DestinationDirs]
DefaultDestDir = 13                 ; 13 = the DRIVER STORE.
                                    ; Not System32\drivers.

[SourceDisksNames]
1 = %DiskName%,,,""

[SourceDisksFiles]
mydriver.sys = 1,,

; ---- which hardware, on which architectures ----

[Manufacturer]
%ManufacturerName% = Standard,NTamd64,NTarm64

[Standard.NTamd64]
%MyDevice.DeviceDesc% = MyDevice_Install, PCI\VEN_1234&DEV_5678

[Standard.NTarm64]
%MyDevice.DeviceDesc% = MyDevice_Install, ACPI\VEN0001

; ---- what to install ----

[MyDevice_Install.NT]
CopyFiles = MyDevice_CopyFiles

[MyDevice_CopyFiles]
mydriver.sys

[MyDevice_Install.NT.Services]
AddService = mydriver, 0x00000002, MyDevice_Service
             ;         ^ SPSVCINST_ASSOCSERVICE - this is the
             ;           FUNCTION driver for the device

[MyDevice_Service]
DisplayName   = %MyDevice.SvcDesc%
ServiceType   = 1                   ; SERVICE_KERNEL_DRIVER
StartType     = 3                   ; SERVICE_DEMAND_START
ErrorControl  = 1                   ; SERVICE_ERROR_NORMAL
ServiceBinary = %13%\mydriver.sys

; ---- KMDF version binding (chapter 12) ----

[MyDevice_Install.NT.Wdf]
KmdfService = mydriver, MyDevice_wdfsect
```

```
[MyDevice_wdfsect]
KmdfLibraryVersion = 1.15

; ---- registry: device configuration ----

[MyDevice_Install.NT.HW]
AddReg = MyDevice_AddReg

[MyDevice_AddReg]
; per-device parameters your driver reads (chapter 01)
HKR,,FifoDepth,0x00010001,32          ; REG_DWORD 32
HKR,,SampleRate,0x00010001,48000
HKR,,DeviceMode,,"normal"             ; REG_SZ

; SECURITY: who may open this device (chapter 03)
HKR,,DeviceCharacteristics,0x10001,0x0100   ; SECURE_OPEN
HKR,,Security,,"D:P(A;;GA;;;SY)(A;;GA;;;BA)"

; MSI support (chapter 17)
HKR,"Interrupt Management",,0x00000010
HKR,"Interrupt Management\MessageSignaledInterruptProperties",,0x00000010
HKR,"Interrupt Management\MessageSignaledInterruptProperties",\
    MSISupported,0x00010001,1
HKR,"Interrupt Management\MessageSignaledInterruptProperties",\
    MessageNumberLimit,0x00010001,4

; ---- strings, localisable ----

[Strings]
ManufacturerName    = "Vendor Corporation"
DiskName            = "MyDevice Installation Disk"
MyDevice.DeviceDesc = "Vendor MyDevice Controller"
MyDevice.SvcDesc    = "Vendor MyDevice Driver"
```

DESTINATION 13 IS NOT OPTIONAL

`DefaultDestDir = 13` means the driver store, and it is required for a modern, universal INF. Copying to `%windir%\System32\drivers` (destination 12) directly is the legacy pattern; it breaks driver store integrity, prevents proper rollback, and fails certification. Let Windows manage the files.

Hardware IDs and ranking

```
The PnP manager asks the bus driver for a list of IDs, most
specific first, and picks the BEST-RANKED matching INF.

  For a PCI device, the bus reports:

    HARDWARE IDs, most specific first:
      PCI\VEN_1234&DEV_5678&SUBSYS_9ABCDEF0&REV_01
      PCI\VEN_1234&DEV_5678&SUBSYS_9ABCDEF0
      PCI\VEN_1234&DEV_5678&REV_01
      PCI\VEN_1234&DEV_5678
    COMPATIBLE IDs, less specific:
      PCI\VEN_1234&DEV_5678&CC_010601
      PCI\VEN_1234&CC_0106
      PCI\CC_010601

  RANK, lower is better:
    0x0000  a signed INF matching a HARDWARE id
    0x1000  a signed INF matching a COMPATIBLE id
    0x8000+ unsigned equivalents

  So a driver matching the full subsystem ID beats a generic
  class driver - which is how a vendor driver replaces
  Microsoft's inbox one, and how a generic driver still works
  when no vendor driver is installed.

  THE PRACTICAL RULE: match the MOST SPECIFIC id your driver
  genuinely supports. Matching PCI\VEN_1234&DEV_5678 claims
  every board with that chip, including ones with different
  wiring - and if yours behaves differently, use the SUBSYS
  form.

  FIND THE IDs: Device Manager -> Properties -> Details ->
  "Hardware Ids". Copy them exactly; a single wrong character
  means Code 28 and no explanation.
```

Universal INFs

```
; A "universal" (or "declarative") INF is required for
; Windows 10+ certification and for anything shipping on
; Windows on ARM, HoloLens or IoT Core. The rules:

; MUST:
;   · DefaultDestDir = 13 (the driver store)
;   · use only DECLARATIVE directives - no co-installers, no
;     RunOnce, no custom setup code
;   · AddReg only under HKR (relative) - no absolute HKLM
;     paths
;   · no [DDInstall.CoInstallers] section
;   · no calls out to external programs

; MUST NOT:
;   · RegisterDlls, UnregisterDlls
;   · AddService for a user-mode service that runs setup code
;   · absolute file paths
;   · legacy WDF co-installers (WdfCoInstaller*.dll) -
;     modern KMDF is inbox and needs none

; CHECK IT:
    InfVerif.exe /u mydriver.inf
    ; /u = universal check. Also:
    InfVerif.exe /v /w mydriver.inf

; The Windows Driver Kit's build already runs InfVerif in
; recent versions. Failures here block certification, so fix
; them as you write rather than at submission time.

; IF YOU NEED SETUP LOGIC - to install a user-mode service or
; a control panel applet - ship it as a separate DCH-compliant
; component package or a Windows app, not as a co-installer.
; That separation is the whole point of the universal model:
; the driver package is pure data.
```

Installing and testing

```
# --- Prepare a test machine (chapter 01) ---
    bcdedit /set testsigning on
    shutdown /r /t 0

# --- Install ---
    pnputil /add-driver mydriver.inf /install
    ; /install also updates matching devices

    ; or, the older tool, still useful for testing:
    devcon install mydriver.inf "PCI\VEN_1234&DEV_5678"

    ; for a device with no physical hardware - a software
    ; device or a test device:
    devcon install mydriver.inf "root\mydevice"

# --- Inspect ---
    pnputil /enum-drivers                ; every third-party
                                         ; driver in the store
    pnputil /enum-devices /connected
    devcon status "PCI\VEN_1234*"
    devcon driverfiles "PCI\VEN_1234*"

# --- Update and remove ---
    pnputil /add-driver mydriver.inf /install    ; upgrade
    devcon update mydriver.inf "PCI\VEN_1234&DEV_5678"

    pnputil /delete-driver oem12.inf /uninstall /force
    devcon remove "PCI\VEN_1234&DEV_5678"
    devcon rescan

# --- The development loop ---
    devcon disable "PCI\VEN_1234*"
    pnputil /delete-driver oem12.inf /uninstall
    ; rebuild
    pnputil /add-driver mydriver.inf /install
    devcon enable "PCI\VEN_1234*"

# Script this. You will run it several hundred times.
```

setupapi.dev.log

```
# THE FILE THAT ANSWERS "why did my driver not install":
    C:\Windows\INF\setupapi.dev.log

# It records every INF considered, its rank, and the decision:

  >>>  [Device Install (Hardware initiated) - PCI\VEN_1234&DEV_5678\...]
  >>>  Section start 2026/03/15 10:23:45.123
       dvi: Searching for hardware ID(s):
       dvi:     pci\ven_1234&dev_5678&subsys_9abcdef0&rev_01
       dvi:     pci\ven_1234&dev_5678&subsys_9abcdef0
       dvi:     pci\ven_1234&dev_5678&rev_01
       dvi:     pci\ven_1234&dev_5678
       dvi: Selected driver installs from section
            [MyDevice_Install] in 'c:\windows\system32\
            driverstore\filerepository\mydriver.inf_amd64_.../
            mydriver.inf'
       dvi: Class GUID of device changed to: {4d36e97d-...}
       dvi: Set selected driver complete.
       ...
       !!!  dvi: Device not started: Device has problem: 0x0a
            (CM_PROB_FAILED_START)

# READ IT TOP-DOWN:
#   "Searching for hardware ID(s)"  -> what the bus reported.
#      Compare against your INF character by character.
#   "Selected driver installs from" -> which INF won.
#      If it names someone else's, your rank was worse.
#   "No matching drivers found"     -> Code 28. Your IDs do
#      not match.
#   "Device has problem: 0x0a"      -> the INF matched and the
#      driver loaded, but START_DEVICE failed. Now it is a
#      driver bug, not an installation one (chapter 14).

# THIS DISTINCTION - install problem versus start problem - is
# the first thing to establish, and this log settles it in
# thirty seconds.
```

Reading your settings

```
/* What the INF's HKR entries wrote, read back at probe time
   (chapter 01):                                            */

NTSTATUS
MyReadDeviceParameters(_In_ WDFDEVICE Device)
{
        PMY_CONTEXT ctx = MyGetContext(Device);
        WDFKEY key;
        ULONG value;
        NTSTATUS status;

        DECLARE_CONST_UNICODE_STRING(fifoName, L"FifoDepth");
        DECLARE_CONST_UNICODE_STRING(rateName, L"SampleRate");

        PAGED_CODE();

        status = WdfDeviceOpenRegistryKey(Device,
                        PLUGPLAY_REGKEY_DEVICE,
                        KEY_READ,
                        WDF_NO_OBJECT_ATTRIBUTES,
                        &key);
        if (!NT_SUCCESS(status)) {
                /* NOT FATAL - use defaults */
                ctx->FifoDepth  = MY_DEFAULT_FIFO;
                ctx->SampleRate = MY_DEFAULT_RATE;
                return STATUS_SUCCESS;
        }

        status = WdfRegistryQueryULong(key, &fifoName, &value);
        ctx->FifoDepth = NT_SUCCESS(status) ? value
                                            : MY_DEFAULT_FIFO;

        status = WdfRegistryQueryULong(key, &rateName, &value);
        ctx->SampleRate = NT_SUCCESS(status) ? value
                                             : MY_DEFAULT_RATE;

        WdfRegistryClose(key);

        /* VALIDATE - the registry is user-writable by an
           administrator, so treat the values as input       */
        if (ctx->FifoDepth == 0 || ctx->FifoDepth > MY_MAX_FIFO)
                ctx->FifoDepth = MY_DEFAULT_FIFO;

        return STATUS_SUCCESS;
}

/* THE TWO KEYS:
     PLUGPLAY_REGKEY_DEVICE  the device's "Device Parameters"
                             subkey - per-instance settings.
                             THIS is where HKR from the .HW
                             section lands.
     PLUGPLAY_REGKEY_DRIVER  the driver's software key -
                             per-driver settings.

/* ALWAYS HAVE A DEFAULT. A missing value must not be fatal -
   the driver may be loaded on a device whose INF section did
   not run, or the value may have been deleted.          */
```

How it shows up

Symptom	Cause
Code 28, "drivers not installed"	Hardware ID mismatch — compare against setupapi.dev.log
A different driver was chosen	Your rank was worse; match a more specific ID
Code 52, signature	Test signing off, or the catalog is missing (ch. 20)
Code 10 after a successful install	Installation worked; `START_DEVICE` failed — a driver bug
Code 39, driver corrupt	File not copied, or the service entry is wrong
Driver installs but no service is created	Missing `SPSVCINST_ASSOCSERVICE` (0x2) on `AddService`
Registry settings absent	`AddReg` in the wrong section — device settings go under `.NT.HW`
InfVerif fails	Non-universal directives — fix before certification, not after
Works on x64, no match on ARM64	Missing `NTarm64` in `[Manufacturer]`
Old driver keeps coming back	Still in the driver store — `pnputil /delete-driver`
Device opens for any user	No SDDL in the INF (ch. 03)
KMDF version error at load	`KmdfLibraryVersion` newer than the OS supports

Check yourself

- ✓ Write a complete INF for a PCI and an ACPI device.
- ✓ Explain what destination 13 means and why it is required.
- ✓ Explain hardware versus compatible IDs, and how ranking picks a driver.
- ✓ List the rules a universal INF must follow, and run `InfVerif`.
- ✓ Install, update and remove a driver with `pnputil` and `devcon`.
- ✓ Use setupapi.dev.log to distinguish an install failure from a start failure.
- ✓ Write device settings from the INF and read them back with validation.
- ✓ Explain why every setting needs a default.
- ✓ Set the device's security descriptor from the INF.
- ✓ Script the full uninstall-rebuild-install loop.

Driver signing

On 64-bit Windows, an unsigned kernel driver does not load. This chapter covers why, the difference between test signing and production signing, and the attestation and HLK paths to a driver that installs on someone else's machine.

Why it is mandatory

```
A kernel driver runs at ring 0 with full access to memory,
hardware and every other driver. There is no sandbox, no
privilege boundary, and no way to contain a malicious one
after it loads.

  SO WINDOWS ENFORCES CODE INTEGRITY at load time:

     x64 and ARM64        signing REQUIRED, always
     Secure Boot on       the boot chain is verified too
     HVCI enabled         additional requirements: no
                          W+X memory, no dynamic code
     Windows 10 1607+     new kernel drivers must be signed by
                          MICROSOFT, not merely by a
                          certificate you bought

  THAT LAST POINT IS THE ONE THAT SURPRISES PEOPLE. Buying an
  EV code-signing certificate is no longer sufficient by
  itself for a new kernel-mode driver on a clean, Secure-Boot
  system - the driver must carry a Microsoft signature
  obtained through the Partner Center.

  THIS IS THE SAME ARGUMENT as isolation chapter 14's chain of
  trust: the boot ROM verifies firmware, firmware verifies the
  kernel, and the kernel verifies drivers. Break any link and
  the ones above it are decoration.
```

Test signing

```
# For development, you self-sign and tell the machine to
# accept it.

# --- 1. Enable test mode on the TARGET machine ---
    bcdedit /set testsigning on
    shutdown /r /t 0
# A "Test Mode" watermark appears on the desktop. Expected.

# On a Secure Boot machine you must first DISABLE Secure Boot
# in firmware, or testsigning is refused.

# --- 2. Create a test certificate ---
    makecert -r -pe -ss PrivateCertStore -n "CN=MyTestCert" \
             mytestcert.cer
# The WDK's Visual Studio template does this for you and
# stores it as "WDKTestCert".

# --- 3. Sign the driver and the catalog ---
    inf2cat /driver:. /os:10_x64,10_ARM64
    ; produces mydriver.cat from the INF

    signtool sign /v /s PrivateCertStore /n MyTestCert \
             /t http://timestamp.digicert.com mydriver.cat
    signtool sign /v /s PrivateCertStore /n MyTestCert \
             /t http://timestamp.digicert.com mydriver.sys

# --- 4. Install the certificate on the target ---
    certmgr /add mytestcert.cer /s /r localMachine root
    certmgr /add mytestcert.cer /s /r localMachine \
            trustedpublisher

# --- 5. Verify ---
    signtool verify /v /kp mydriver.sys
    signtool verify /v /c mydriver.cat mydriver.sys

# THE VISUAL STUDIO PROJECT does steps 2, 3 and part of 4
# automatically when "Sign Mode" is set to "Test Sign".
# Configuration Properties -> Driver Signing.

# --- The alternative for quick experiments ---
    bcdedit /set nointegritychecks on
# Disables enforcement entirely. Works, but it also disables
# it for malware, and it masks signature problems you will hit
# later. Prefer test signing.
```

> **NEVER SHIP A MACHINE IN TEST MODE**
>
> Test signing disables the guarantee the whole mechanism exists to provide. It is a development convenience on a machine you control. A product that requires `testsigning on` or `nointegritychecks` to run is a product with no code integrity, and it will fail every security review it meets.

Production signing

```
THE MODERN PATH, for a new kernel driver:

1. GET AN EV CODE SIGNING CERTIFICATE
     from a CA Microsoft recognises (DigiCert, Sectigo,
     GlobalSign...). "EV" means extended validation: a
     hardware token, and identity verification of your
     organisation. Weeks, and a real cost.

2. REGISTER ON THE MICROSOFT PARTNER CENTER
     for the Hardware Program. The EV certificate is what
     establishes your identity there.

3. SIGN YOUR PACKAGE with the EV certificate.

4. SUBMIT IT to the Partner Center as a driver package
     (a .cab or .hlkx).

5. MICROSOFT SIGNS IT and returns the signed package.
     THAT signature is what makes it load.

6. OPTIONALLY, distribute via Windows Update.

TWO SUBMISSION ROUTES:

  ATTESTATION SIGNING
     · you attest that the driver is safe
     · NO test results required
     · fast: often under an hour
     · limited to Windows 10/11, and NOT for drivers that
       need to run on older releases
     · NOT eligible for Windows Update distribution of
       some classes
     -> the right route for most in-house and niche
        hardware

  HLK / WHQL CERTIFICATION
     · run the Hardware Lab Kit test suite
     · produces an .hlkx package with results
     · required for the "Certified for Windows" logo, for
       Windows Update distribution, and for some device
       classes
     · days to weeks of test running and fixing
     -> required for consumer hardware sold at scale
```

The HLK

```
# The Hardware Lab Kit is the certification test suite. Even
# if you use attestation signing, RUN THE RELEVANT TESTS -
# they find real bugs.

# SETUP:
#   an HLK Controller machine (with SQL Server)
#   an HLK Studio (the UI)
#   one or more TEST SYSTEMS running the HLK Client, with your
#   driver installed
#
# The controller drives the clients, runs tests, and collects
# results into a submission package.

# THE TESTS THAT MATTER MOST for a typical driver:

    Device Fundamentals
      · DF - PNP Disable and Enable
      · DF - PNP Remove and Restart
      · DF - PNP Surprise Remove       <-- chapter 14
      · DF - Sleep and PNP (with IO Before and After)
      · DF - Reboot Restart
      · DF - Concurrent Hardware and Operating System
      · DF - Fuzz tests (IoSpy/IoAttack)  <-- chapter 16
      · DF - Driver Verifier tests        <-- chapter 21

    Static Tools Logo
      · Code Analysis with driver-specific rules
      · Static Driver Verifier              <-- chapter 21
      Both must pass with zero defects.

# RUN THESE EARLY. A driver that fails "PNP Surprise Remove"
# at certification time usually needs architectural changes -
# whereas the same test on day three costs an afternoon.

# THE DEVICE FUNDAMENTALS TESTS ALONE, run with Driver
# Verifier enabled, catch the majority of the bugs this whole
# track has been warning about: IRQL violations, leaked
# resources, unhandled removal, unvalidated IOCTL input.
# They are, in effect, a free test suite for your driver -
# use them as one.
```

HVCI and additional requirements

```
/* Hypervisor-protected Code Integrity runs the code integrity
   check inside a VM-isolated environment, so a compromised
   kernel cannot disable it. It is DEFAULT ON for many
   Windows 11 configurations.

   HVCI IMPOSES EXTRA RULES on drivers: */

   1. NO WRITABLE AND EXECUTABLE MEMORY. A driver may not
      allocate memory that is both.
        -> use POOL_FLAG_NON_PAGED, never
           POOL_FLAG_NON_PAGED_EXECUTE unless truly required
        -> no self-modifying code, no dynamically generated
           trampolines

   2. NO SECTION ALIGNMENT SMALLER THAN 4 KB.
        the linker flag /ALIGN:4096, which the WDK templates
        already set

   3. NO EXECUTABLE SECTIONS THAT ARE ALSO WRITABLE.

   4. IMPORT TABLES MUST BE VALID and resolvable.

/* CHECK YOUR DRIVER: */
        Device Guard / HVCI readiness test in the HLK
        DGReadiness.ps1 (the Device Guard readiness tool)

/* THE SYMPTOM if you fail: the driver silently does not load
   on an HVCI system, with an event log entry about code
   integrity. It works perfectly on your development machine
   with HVCI off - which is why testing on an HVCI-enabled
   system is a required part of the matrix, not an
   afterthought.                                          */

/* AND THE DEEPER POINT, which connects to the isolation
   track: HVCI is Windows applying the same reasoning as
   TrustZone (isolation ch. 09) - move the enforcement to a
   more privileged, smaller, separately-verified component,
   so that compromising the large component does not
   compromise the check.                                  */
```

Verifying a signature

```
# --- Is this file signed, and by whom? ---
    signtool verify /v /kp mydriver.sys
      Verifying: mydriver.sys
      Signature Index: 0 (Primary Signature)
      Hash of file (sha256): 3A2B...
      Signing Certificate Chain:
          Issued to: Microsoft Windows Third Party Component CA
          ...
      Successfully verified: mydriver.sys

    ; /kp = check against KERNEL POLICY, which is stricter
    ; than the default. Use it - a driver can pass plain
    ; verification and still be refused at load.

# --- Against the catalog ---
    signtool verify /v /c mydriver.cat mydriver.sys

# --- What is actually loaded ---
    WinDbg:  lm                 ; loaded modules
             !chkimg mydriver   ; image integrity
             .imgscan

# --- WHY DID IT NOT LOAD? ---
    Event Viewer ->
      Applications and Services Logs ->
        Microsoft -> Windows -> CodeIntegrity -> Operational

# Event 3004: "Windows is unable to verify the image integrity
# of the file ... because file hash could not be found on the
# system."  -> the catalog is missing or does not cover the
# file.
# Event 3033: "... did not meet the Microsoft signing level
# requirements."  -> signed, but not by an authority this
# policy accepts.

# THESE TWO EVENTS distinguish "no signature" from "wrong
# signature", which is the first thing to establish.
```

A signing pipeline

```
# What a real build should do, in order:

# 1. BUILD
    msbuild mydriver.sln /p:Configuration=Release \
            /p:Platform=x64

# 2. STATIC ANALYSIS - must be clean before signing
    msbuild /p:RunCodeAnalysis=true
    ; and Static Driver Verifier (chapter 21)

# 3. BUILD THE CATALOG
    inf2cat /driver:.\output /os:10_x64,10_ARM64 /verbose

# 4. VERIFY THE INF
    InfVerif /u .\output\mydriver.inf

# 5. SIGN, with a timestamp
    signtool sign /v /fd sha256 /a \
             /tr http://timestamp.digicert.com /td sha256 \
             .\output\mydriver.cat
    signtool sign /v /fd sha256 /a \
             /tr http://timestamp.digicert.com /td sha256 \
             .\output\mydriver.sys

# THE TIMESTAMP MATTERS: without it, the signature becomes
# invalid when the certificate expires - typically in one to
# three years. With it, the signature remains valid because
# the timestamp proves it was signed while the certificate
# was live. Never sign without /tr.

# 6. VERIFY
    signtool verify /v /kp .\output\mydriver.sys

# 7. PACKAGE for submission
    makecab /f driver.ddf     ; -> mydriver.cab
    ; sign the cab with the EV certificate, then submit

# KEEP THE EV PRIVATE KEY IN ITS HARDWARE TOKEN. It is the
# identity of your organisation to Microsoft; a stolen one
# signs malware in your name, and revoking it invalidates
# every driver you have shipped.
```

How it shows up

Symptom	Cause
Code 52, "cannot verify the digital signature"	Unsigned, or test signing not enabled
CodeIntegrity event 3004	Catalog missing, or it does not cover the file's hash
CodeIntegrity event 3033	Signed, but not to the required level — needs a Microsoft signature
Works on your machine, not on a clean one	Your test certificate is installed locally only
Works, then stops after a rebuild	The catalog was not regenerated — hashes no longer match
Loads with HVCI off, not with it on	Writable-and-executable memory, or bad section alignment
Signature invalid after a year	Signed without a timestamp
`bcdedit /set testsigning on` refused	Secure Boot is enabled — disable it in firmware first

Symptom	Cause
Attestation submission rejected	The driver class is not eligible; HLK certification is required
HLK Surprise Remove test fails	A real removal-handling bug (ch. 14)
Static Tools Logo fails	Code Analysis or SDV findings — fix them (ch. 21)

Check yourself

- ✓ Explain why kernel driver signing is mandatory and what it protects.
- ✓ Set up test signing end to end, including the certificate on the target.
- ✓ Explain why an EV certificate alone is no longer sufficient.
- ✓ Compare attestation signing with HLK certification and say when each applies.
- ✓ Name six Device Fundamentals tests and the bug class each finds.
- ✓ List HVCI's four requirements and the symptom of failing them.
- ✓ Verify a signature against kernel policy, and read the CodeIntegrity event log.
- ✓ Distinguish event 3004 from 3033.
- ✓ Write a signing pipeline and explain why the timestamp is required.
- ✓ Relate this chapter to the chain of trust in the isolation track.

Static analysis and Driver Verifier

Windows ships the best driver verification tooling of any operating system, and most of it is free and automatic. Static analysis finds bugs at build time; Driver Verifier turns latent runtime bugs into immediate, attributable bugchecks. Use both, from day one.

Why these matter more here

```
A driver bug on Linux is found by the community, because the
source is in the tree and everyone builds it (Linux ch. 30).

A Windows driver ships as a signed binary that Microsoft never
sees the source of. So the verification burden moves to
TOOLING, and Microsoft has invested accordingly:

   BUILD TIME              Code Analysis (PREfast)
                           Static Driver Verifier (SDV)
                           SAL annotations (chapter 02)
                           InfVerif (chapter 19)

   RUN TIME                Driver Verifier
                           WDF Verifier (chapter 13)
                           Application Verifier (user mode)
                           Special pool (chapter 06)

   CERTIFICATION           HLK Device Fundamentals
                           Static Tools Logo test

  THE PAYOFF: these tools find, mechanically, most of the bug
  classes this entire track has described - IRQL violations,
  leaked resources, unvalidated input, incorrect IRP handling,
  DMA misuse, lock ordering. A driver that passes all of them
  clean is in far better shape than one that merely works.
```

Code Analysis

```
# PREfast for Drivers - runs during the build, understands
# SAL annotations (chapter 02) and driver-specific rules.

    msbuild mydriver.sln /p:RunCodeAnalysis=true
    ; or: Visual Studio -> Analyze -> Run Code Analysis

# In the project: use the "Recommended driver rules" ruleset,
# which adds the driver-specific checks to the general C/C++
# ones.

# WHAT IT CATCHES:
#   · calling a function at the wrong IRQL, per its
#     _IRQL_requires_max_ annotation
#   · ignoring a _Must_inspect_result_ return
#   · buffer sizes inconsistent with _In_reads_bytes_
#   · using an uninitialised variable
#   · leaking a resource on an error path
#   · dereferencing a possibly-NULL pointer
#   · a lock acquired and not released on some path
#   · incorrect WDF object usage

# A TYPICAL FINDING:
    warning C28118: The current IRQL was too high. Prior
    function calls indicate IRQL was DISPATCH_LEVEL, but the
    function MmMapIoSpaceEx requires PASSIVE_LEVEL.

# FIX IT. Do not suppress it. If a warning is genuinely a
# false positive, suppress it NARROWLY and with a comment:
    #pragma warning(suppress: 28118)  // IRQL is checked by
                                      // the caller; see the
                                      // header contract

# THE HIGHEST-VALUE HABIT in Windows driver development is
# annotating your own functions with SAL as you write them.
# The analyser then checks your CALLERS against your
# contract - which is the difference between finding an IRQL
# bug at build time and finding it in a customer's crash
# dump.
```

Static Driver Verifier

```
# SDV is different from Code Analysis: it does symbolic model
# checking against a set of RULES that describe how the driver
# framework must be used. It explores paths, not just
# expressions.

    msbuild /p:Configuration="Win8.1 Release" \
            /p:Platform=x64 /t:sdv /p:Inputs="/check:*"
    ; or: Visual Studio -> Driver -> Launch Static Driver
    ;     Verifier

# THE PROCESS:
#   1. scan - identify entry points and roles
#   2. verify - run each rule against the model
#   3. produce a report, with a COUNTEREXAMPLE TRACE for each
#      violation

# EXAMPLE RULES:
    IrqlKeApcLte1        KeXxx APC functions at the right IRQL
    IrqlIoPassive1       Io functions requiring PASSIVE_LEVEL
    SpinLockRelease      every acquire has a release
    LowerDriverReturn    completion routines return correctly
    PnpIrpCompletion     PnP IRPs completed per the rules
    RequestCompleted     every WDF request is completed exactly
                         once
    MarkIrpPending2      IoMarkIrpPending used correctly
    WdfSpinlock          WDF spinlock usage
    DoubleCompletion     no IRP completed twice

# READING A DEFECT: SDV shows the exact path through your code
# that violates the rule, step by step. It is a proof, not a
# heuristic - so a reported defect on a reachable path is a
# real bug.

# CAVEATS, stated honestly:
#   · SDV can time out on complex drivers. Increase the limit,
#     or simplify the function it struggles with.
#   · it can report defects on paths that cannot actually
#     occur, because it does not know a hardware invariant you
#     rely on. Those need review, not blind fixing.
#   · it must be run on the whole driver, not one file.

# BUT: the Static Tools Logo certification test REQUIRES a
# clean SDV run and a clean Code Analysis run. So this is not
# optional if you intend to certify - run it early, when the
# fixes are cheap.
```

> **RUN BOTH BEFORE YOU EVER LOAD THE DRIVER**
>
> A bug caught by Code Analysis costs a minute. The same bug caught by Driver Verifier costs an hour of crash-dump analysis. The same bug found by a customer costs a support case, a hotfix, and a re-certification. The ordering of these tools is also the ordering of their cost.

Driver Verifier

```
# The runtime tool. It instruments your driver's interaction
# with the kernel and BUGCHECKS at the moment a rule is
# violated, rather than letting the consequence appear later
# somewhere else.

# --- Enable it ---
    verifier /standard /driver mydriver.sys
    ; or a specific flag set:
    verifier /flags 0x209BB /driver mydriver.sys
    ; then reboot

    verifier /querysettings        ; what is enabled
    verifier /volatile /flags 0x1 /adddriver mydriver.sys
                                   ; some flags, no reboot
    verifier /reset                ; turn it all off
    ; or the GUI: verifier.exe

# --- THE FLAGS, and what each finds ---

    0x00001  Special pool
             each allocation on its own page with a guard
             page. Catches overruns AT THE MOMENT they happen.
             The equivalent of Linux's KASAN (ch. 27).

    0x00002  Force IRQL checking
             makes pageable code and data non-resident, so
             touching it at DISPATCH_LEVEL faults IMMEDIATELY
             instead of weeks later (chapter 02). THE most
             valuable flag.

    0x00004  Low resources simulation
             randomly fails allocations, so untested error
             paths execute. Linux's fault injection (ch. 26).

    0x00008  Pool tracking
             detects allocations still outstanding at unload,
             with the stack that made them (chapter 06).

    0x00010  I/O verification
             IRP rule checking: double completion, use after
             completion, wrong IRQL, missing PendingReturned
             (chapter 04).

    0x00020  Deadlock detection
             builds a lock-order graph and reports
             inconsistent acquisition. Linux's lockdep
             (ch. 09).

    0x00080  DMA verification
             map register accounting, alignment, overruns
             (chapter 18).

    0x00100  Security checks
             insecure object creation, missing access checks

    0x00800  Miscellaneous checks
             freeing an object with references, incorrect
             string usage

    0x02000  Invariant MDL checking (stack)
```

```
    0x04000  Invariant MDL checking (driver)

    0x20000  DDI compliance checking
             the KMDF framework rules

# --- START HERE ---
    verifier /flags 0x209BB /driver mydriver.sys
# = special pool, IRQL checking, pool tracking, I/O
# verification, deadlock detection, DMA, security, misc, DDI
# compliance. This is the standard development set.

# --- AND FOR A HARDER PASS, add low resources ---
    verifier /flags 0x209BF /driver mydriver.sys
# Expect to find unchecked allocations. That is the point.
```

The cost, and when to enable what

Flag	Cost	When
Special pool	High memory, moderate speed	Development, and any corruption hunt
Force IRQL checking	Moderate	**Always** during development
Low resources	Causes failures by design	A dedicated error-path testing pass
Pool tracking	Low	Always
I/O verification	Moderate	Always
Deadlock detection	Moderate	Always
DMA verification	Low	Any driver doing DMA
DDI compliance	Low	Always, for WDF drivers
Everything at once	Very slow	An overnight soak run

```
/* THE PRACTICAL SCHEDULE:

   EVERY DEVELOPMENT BUILD
        the standard set, always on. You will catch bugs the
        day you write them.

   BEFORE EACH MILESTONE
        add low resources simulation and run the full
        functional test suite. Every crash is an unchecked
        allocation.

   BEFORE SUBMISSION
        the full flag set, plus the HLK Device Fundamentals
        tests (chapter 20), running overnight.

   ON A TEST MACHINE, NOT YOUR DEVELOPMENT BOX. Verifier
   bugchecks, by design.                                  */
```

Reading a Verifier bugcheck

```
DRIVER_VERIFIER_DETECTED_VIOLATION (c4)
A device driver attempting to corrupt the system has been
caught.
Arguments:
Arg1: 0000000000000062, A driver has forgotten to free its
                        pool allocations prior to unloading.
Arg2: ffff8a0c11223344, name of the driver having the issue
Arg3: 0000000000000004, number of allocations leaked
Arg4: 0000000000000000

/* THE FIRST ARGUMENT IS THE RULE. Common ones:

     0x62  pool allocations not freed at unload
     0x81  freeing memory that contains active timers
     0xE6  DMA violation (also bugcheck 0xE6 itself)
     0xF6  a driver referenced a freed object
     0x91  a thread holding a spinlock at exit
     0x1000-series  deadlock detection

/* AND THE INVESTIGATION, in WinDbg (chapter 22): */

        !analyze -v
        !verifier 3 mydriver.sys
          /* every outstanding allocation, WITH THE STACK that
             made it - which names the leak directly */
        !verifier 0x80 mydriver.sys
          /* IRP log */
        !verifier 1
          /* the enabled flags */

/* THE OTHER VERIFIER BUGCHECKS:
     0xC4  DRIVER_VERIFIER_DETECTED_VIOLATION
     0xC6  DRIVER_CAUGHT_MODIFYING_FREED_POOL
     0xC9  DRIVER_VERIFIER_IOMANAGER_VIOLATION
     0xE6  DRIVER_VERIFIER_DMA_VIOLATION
     0xC1  SPECIAL_POOL_DETECTED_MEMORY_CORRUPTION

/* EACH ONE NAMES YOUR DRIVER AND THE RULE. That is the whole
   value: the crash is at the point of the mistake, attributed
   to the code that made it, rather than a random later
   consequence in someone else's driver.                  */
```

WDF Verifier

```
# Separate from Driver Verifier, and it checks a different
# thing: framework contract violations.

# Enable via the WDK's tool:
    wdfverifier.exe
# or in the registry, under the driver's Parameters\Wdf key:
    VerifierOn        = 1
    VerboseOn         = 1
    DbgBreakOnError   = 1
    TrackHandles      = 1     ; find handle leaks
    VerifyDownLevel   = 1

# WHAT IT CATCHES:
#   · using a WDF handle after deletion
#   · completing a request twice
#   · completing a request you forwarded
#   · calling a WDF API at the wrong IRQL
#   · object hierarchy violations (chapter 13)
#   · lock misuse
#   · queue state violations (chapter 15)

# WHEN IT FIRES, with DbgBreakOnError it breaks into the
# debugger with a message naming the rule:
    *** WDFVERIFIER: WdfRequestComplete called on a request
        that is not owned by the driver.

# THEN:
    !wdflogdump mydriver
# gives the framework's own trace of what led there.

# TURN BOTH VERIFIERS ON. They overlap very little.
```

The workflow

```
# A realistic development cycle:

# EVERY BUILD
    msbuild /p:RunCodeAnalysis=true
    ; zero warnings, or a documented suppression

# EVERY TEST RUN, on the test machine
    verifier /flags 0x209BB /driver mydriver.sys
    wdfverifier: VerifierOn=1
    ; both enabled, always

# WEEKLY
    SDV full run
    low resources simulation pass
    100 install/uninstall cycles
    100 disable/enable cycles
    sleep/resume cycles with I/O in flight

# BEFORE SUBMISSION
    full verifier flags, overnight soak
    HLK Device Fundamentals (chapter 20)
    HLK Static Tools Logo (Code Analysis + SDV, clean)
    IoAttack fuzzing of the IOCTL interface (chapter 16)

# THE THINGS THAT MOST OFTEN FIND REAL BUGS, in order:
#   1. Force IRQL checking - the paged-memory-at-DISPATCH bug
#   2. Low resources simulation - unchecked allocations
#   3. PNP Surprise Remove - unhandled removal
#   4. Deadlock detection - latent lock ordering
#   5. IoAttack - unvalidated IOCTL input
#
# All five are free, automated, and take an afternoon to set
# up once.
```

How it shows up

Symptom	Cause
Bugcheck 0xC4 Arg1 = 0x62	Pool leaked at unload — `!verifier 3` names the site
Bugcheck 0xC1	Special pool caught an overrun at the moment it happened
Bugcheck 0xC6	Wrote to freed pool
Bugcheck 0xC9	I/O manager rule — usually double completion
Bugcheck 0xE6	DMA rule (ch. 18)
Driver crashes only with Verifier on	A real latent bug Verifier made deterministic — not a false alarm
Crashes with low resources simulation	Unchecked allocation return
WDFVERIFIER message about request ownership	Completed a forwarded request (ch. 15)
SDV reports an unreachable defect	A path it cannot rule out — review, annotate, or accept
SDV times out	Function too complex; simplify or raise the limit
Static Tools Logo fails	Code Analysis or SDV not clean — required for certification
Everything is very slow	Too many Verifier flags for routine use — trim to the standard set

Check yourself

- ✓ Explain why Windows invests more in tooling than Linux does, and what that follows from.
- ✓ Run Code Analysis and fix a finding rather than suppressing it.
- ✓ Explain what SAL annotations let the analyser check about your callers.
- ✓ Explain how SDV differs from Code Analysis and what a counterexample trace is.
- ✓ Name eight Driver Verifier flags and the bug class each finds.
- ✓ Map five of them onto their Linux equivalents.
- ✓ Choose a flag set for daily use and one for a milestone pass.
- ✓ Read a 0xC4 bugcheck and use `!verifier 3` to locate the leak.
- ✓ Enable WDF Verifier and say what it catches that Driver Verifier does not.
- ✓ Write the weekly and pre-submission verification schedule.

WinDbg for driver developers

Setting up kernel debugging, reading a crash dump, and the two dozen commands that cover almost everything you will need. Unlike Linux, Windows expects you to use a real debugger — and the tooling is correspondingly good.

Setting up a connection

```
# You need TWO machines: the HOST running WinDbg, and the
# TARGET running your driver. A VM is ideal.

# --- On the TARGET: choose a transport ---

# NETWORK (fastest, and the modern default):
    bcdedit /debug on
    bcdedit /dbgsettings net hostip:192.168.1.10 \
            port:50000 key:1.2.3.4.5.6.7.8
    ; the key can be anything; note it for the host
    shutdown /r /t 0

# SERIAL (works on anything, including physical boards):
    bcdedit /debug on
    bcdedit /dbgsettings serial debugport:1 baudrate:115200

# For a Hyper-V VM, the simplest option:
    bcdedit /dbgsettings serial debugport:1 baudrate:115200
    ; and configure the VM's COM1 as a named pipe:
    ;   \\.\pipe\mydebug

# BREAK INTO THE DEBUGGER AT BOOT, to catch DriverEntry:
    bcdedit /set bootdebug on

# --- On the HOST: connect ---
    windbg -k net:port=50000,key=1.2.3.4.5.6.7.8
    windbg -k com:pipe,port=\\.\pipe\mydebug,resets=0,reconnect

# --- SYMBOLS - do this first, always ---
    .sympath srv*C:\symbols*https://msdl.microsoft.com/download/symbols
    .sympath+ C:\myproject\x64\Debug
    .reload /f

    ; or set _NT_SYMBOL_PATH as an environment variable so
    ; every tool picks it up:
    setx _NT_SYMBOL_PATH "srv*C:\symbols*https://msdl.microsoft.com/download/symbols"

# WITHOUT SYMBOLS, every command prints hex addresses and you
# learn nothing. This is the single most common reason a
# newcomer finds WinDbg useless.

# --- Source-level debugging ---
    .srcpath C:\myproject\src
    l+t          ; source line mode
    .lines       ; enable line number information
```

Reading a crash dump

```
# CONFIGURE DUMPS on the target, before you need them:
#   System Properties -> Advanced -> Startup and Recovery
#   "Kernel memory dump" is the right choice for driver work -
#   a minidump often lacks the pool data you need.
#   -> C:\Windows\MEMORY.DMP

# Open it:
    windbg -z C:\Windows\MEMORY.DMP

# THE FIRST COMMAND, ALWAYS:
    !analyze -v

# It gives you:
#   the bugcheck code and its four parameters, decoded
#   the faulting module and function
#   the stack trace
#   often, a plain-English "PROCESS_NAME" and
#   "FAILURE_BUCKET_ID" that names the bug class
#   and a "MODULE_NAME" - which, if it is yours, is the answer

# THEN, IN ORDER:
    .bugcheck                 ; the code and parameters again
    kb                        ; the stack with arguments
    kv                        ; verbose - includes frame
                              ; pointers and trap frames
    kP                        ; with full parameter names
    lm                        ; loaded modules
    lmvm mydriver             ; details of yours - version,
                              ; timestamp, symbol status
    !irql                     ; the IRQL at the time
    !process                  ; the current process
    !thread                   ; the current thread
    r                         ; registers
    .trap <address>           ; switch to a trap frame's
                              ; context, from the kv output

# IF THE STACK IS CORRUPT:
    dps @rsp L100             ; dump the stack as symbols -
                              ; often reveals return addresses
                              ; the unwinder missed
    !stackusage
    .thread /r /p <ethread>   ; switch thread context
```

> **`!ANALYZE -V` IS RIGHT MORE OFTEN THAN YOU EXPECT**
>
> It applies a large body of heuristics about known bug patterns and frequently names the exact module and reason. Read its whole output before doing anything else — including the "STACK_COMMAND" and "FOLLOWUP_NAME" lines, which tell you what it thinks and why. Then verify its conclusion rather than trusting it blindly; it is a very good first hypothesis, not a proof.

The commands worth memorising

Command	Does
`!analyze -v`	Automated crash analysis
`kb` / `kv` / `kP`	Stack trace, increasingly verbose

Command	Does
`lm` / `lmvm mod`	Loaded modules; details of one
`x mydriver!*`	Symbols matching a pattern
`dt nt!_IRP addr`	Display a typed structure
`dt -r mydriver!_MY_CONTEXT addr`	Recursively, your own type
`db` / `dw` / `dd` / `dq addr`	Dump memory, by width
`dps addr`	Dump pointers with symbol names
`u addr`	Disassemble
`bp mydriver!MyRead`	Breakpoint
`ba w4 addr`	Break on write to an address — invaluable for corruption
`g` / `p` / `t`	Go, step over, step into
`!process 0 0`	All processes, brief
`!thread addr`	One thread: state, wait reason, stack
`!irp addr` / `!irpfind`	One IRP; find all IRPs
`!devobj` / `!devstack` / `!drvobj`	Device and driver objects
`!devnode 0 1`	The PnP tree
`!pool addr` / `!poolused 2`	Pool allocation; usage by tag
`!poolfind Tag`	Every allocation with a tag
`!verifier 3 mydriver`	Verifier's outstanding allocations, with stacks
`!locks` / `!deadlock`	Held resources; the deadlock graph
`!stacks 2`	Every thread's stack
`!vm`	Memory usage summary
`!error 0xc0000001`	Decode an NTSTATUS or Win32 error
`.reload /f`	Force symbol reload

The WDF extension

```
    .load wdfkd.dll
    !wdfkd.wdfhelp                  ; the full list

# THE ONES THAT MATTER:
    !wdfdriverinfo mydriver         ; everything: version,
                                    ; devices, queues
    !wdfdriverinfo mydriver 0xff    ; maximum verbosity
    !wdfdevicequeues <WDFDEVICE>    ; every queue and its
                                    ; pending requests
                                    ; (chapter 15)
    !wdfhandle <handle>             ; type, parent, context,
                                    ; refcount (chapter 13)
    !wdfobject <handle>
    !wdfrequest <WDFREQUEST>        ; state, owner, IRP
    !wdfqueue <WDFQUEUE>
    !wdfobjectdump                  ; the whole object tree

# AND THE MOST VALUABLE ONE:
    !wdflogdump mydriver
    !wdflogdump mydriver -d         ; include the IFR (in-flight
                                    ; recorder) detail

# The framework keeps a circular trace of every state
# transition, request, and error it detected - IN MEMORY, at
# all times, with no configuration. After a crash it usually
# tells you what the framework thought was happening, which is
# frequently the entire diagnosis.

# YOUR OWN TRACES appear there too, if you use WPP or
# TraceEvents - see below.
```

Tracing from your driver

```
/* THREE OPTIONS, in increasing order of quality: */

/* 1. DbgPrint / KdPrint - simplest, slowest */
        KdPrint(("MyRead: length %lu\n", length));
        /* compiled out in release builds */

        DbgPrintEx(DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL,
                   "MyRead: length %lu\n", length);

        /* enable output on the target: */
        ; registry: HKLM\SYSTEM\CCS\Control\Session Manager\Debug
        ;           Print Filter -> IHVDRIVER = 0xF
        ; or in WinDbg:
        ed nt!Kd_IHVDRIVER_Mask 0xF

/* DbgPrint IS SLOW - it serialises to the debugger connection
   and can take milliseconds with interrupts disabled. Never
   in a hot path or an ISR (chapter 10).                    */

/* 2. WPP TRACING - the standard for production drivers.
      Compiled to a binary format with a message GUID; the
      strings live in a PDB, not in the driver.             */

        /* trace.h */
        #define WPP_CONTROL_GUIDS \
            WPP_DEFINE_CONTROL_GUID( \
                MyDriverTraceGuid, \
                (A1B2C3D4,1234,5678,9ABC,DEF012345678), \
                WPP_DEFINE_BIT(TRACE_DEVICE)  \
                WPP_DEFINE_BIT(TRACE_QUEUE)   \
                WPP_DEFINE_BIT(TRACE_DPC)     \
                )

        /* in the driver: */
        WPP_INIT_TRACING(DriverObject, RegistryPath);
        ...
        TraceEvents(TRACE_LEVEL_INFORMATION, TRACE_QUEUE,
                    "Read request, length %lu", length);
        ...
        WPP_CLEANUP(DriverObject);

/* WPP IS FAST - a few dozen cycles per event, safe at any
   IRQL. And the messages land in the WDF log, so
   !wdflogdump shows them interleaved with the framework's
   own trace, which is exactly the correlation you want.  */

        /* capture live: */
        tracelog -start MyTrace -guid #A1B2C3D4-1234-5678-9ABC-DEF012345678 \
                 -f mytrace.etl -flag 0xF -level 5
        tracelog -stop MyTrace
        tracefmt -o output.txt mytrace.etl
        ; or traceview.exe for a GUI

/* 3. ETW TRACE EVENTS - for anything you want correlated with
      system-wide traces in WPA. More setup, best integration.
      Use it when latency or performance is the question
      rather than correctness.                              */
```

Live debugging techniques

```
# --- Break in whenever you like ---
    Ctrl+Break in WinDbg, or on the target:
    ; SysRq equivalent: the debugger breaks on demand over the
    ; connection

# --- Break on your driver loading ---
    sxe ld mydriver.sys
    g
    ; stops the moment it loads, before DriverEntry

# --- Breakpoint on a function, with a condition ---
    bp mydriver!MyEvtIoRead ".if (@rdx == 0) {} .else {gc}"
    ; break only when the second argument is zero

# --- Breakpoint that just logs and continues ---
    bp mydriver!MyEvtIoRead "dt mydriver!_MY_CONTEXT @rcx; gc"

# --- BREAK ON MEMORY WRITE - the corruption finder ---
    ba w8 ffff8a0c12345678
    ; breaks when 8 bytes at that address are written.
    ; THE tool for "who is corrupting my structure?" - set it
    ; on the field that gets clobbered and the culprit
    ; identifies itself.
    ; Limited to four hardware breakpoints.

# --- Watch a value change over time ---
    .while (1) { dd mydriver!g_Counter L1; .sleep 1000; g }

# --- Call a function in the target ---
    .call mydriver!MyDumpState(0)
    g
    ; requires private symbols and a compatible calling
    ; convention. Powerful, and occasionally the fastest way
    ; to interrogate driver state.

# --- Scripting ---
    $$>a< C:\scripts\myscript.txt
    dx @$curprocess.Threads     ; the NatVis/dx data model
    dx -g @$curprocess.Threads.Where(t => t.Id > 100)
```

Diagnosing the common failures

```
# --- "My driver bugchecked" ---
    !analyze -v
    ; then verify its conclusion with kb and the bugcheck
    ; parameters (chapter 02 for 0xA/0xD1)

# --- "The system hangs, no bugcheck" ---
    Ctrl+Break
    !running                  ; what is each CPU doing?
    !stacks 2                 ; every thread
    !locks                    ; who holds what
    !deadlock                 ; if verifier's flag is on
    !irql                     ; are we stuck at high IRQL?

# --- "An application hangs on my device" ---
    !process 0 7              ; find the process and thread
    !thread <ethread>         ; what is it waiting on
    !irpfind -v 0 0 device <devobj>     ; which IRP is stuck
    !wdfdevicequeues <wdfdevice>        ; and which queue
                                        ; holds it

# --- "Memory is leaking" ---
    !poolused 2               ; sorted by non-paged usage
    !poolfind xMYD            ; your tag (chapter 06)
    !verifier 3 mydriver.sys  ; with stacks, if pool tracking
                              ; is enabled

# --- "Something is corrupting my structure" ---
    ba w8 <address of the field>
    ; or enable special pool and let it fault at the moment of
    ; the overrun (chapter 21)

# --- "The driver will not load" ---
    !devnode 0 1 mydriver
    ; and read setupapi.dev.log (chapter 19) and the
    ; CodeIntegrity event log (chapter 20) on the target

# --- "It works with the debugger attached" ---
    ; a timing bug. The debugger changes timing enormously.
    ; Use WPP tracing instead of breakpoints, and enable
    ; Driver Verifier - which perturbs timing differently and
    ; often makes the race deterministic.
```

How it shows up

Symptom	Cause
Every command prints hex, no names	Symbols not configured — fix `.sympath` first
Your driver's symbols missing	PDB not in the path, or a different build than the one running
`lmvm` shows "export symbols only"	Same — you need the private PDB
Stack trace makes no sense	Corrupt stack — try `dps @rsp`, or a trap frame with `.trap`
Minidump lacks the data you need	Configure a kernel memory dump instead
Cannot connect over the network	Firewall, wrong key, or the NIC is not debug-capable
Bug disappears under the debugger	A timing-dependent race — use tracing and Verifier
`DbgPrint` output missing	Print filter mask not set for your component

Symptom	Cause
Driver is slow only in debug builds	`DbgPrint` in a hot path — switch to WPP
Four breakpoints, the fifth fails	Hardware breakpoint limit

Check yourself

- ✓ Set up a network or serial kernel debugging connection to a VM.
- ✓ Configure symbols so both Microsoft's and your own resolve.
- ✓ Read a crash dump with `!analyze -v` and verify its conclusion.
- ✓ Use fifteen of the core commands from memory.
- ✓ Use the WDF extension to inspect queues, requests and the framework log.
- ✓ Add WPP tracing and explain why it beats `DbgPrint`.
- ✓ Set a conditional breakpoint and a data breakpoint.
- ✓ Diagnose a hang using only `!running`, `!stacks` and `!locks`.
- ✓ Find a hung application's stuck IRP and the queue holding it.
- ✓ Locate a pool leak with `!poolused` and `!verifier 3`.

Project: the KMDF ring-buffer driver

A complete, working Windows driver you build, sign, install and break yourself. It needs no hardware — it installs as a software device — and it exercises queues, IOCTLs, asynchronous completion, timers, locking and the full PnP lifecycle. Deliberately the same problem as the Linux project in chapter 28, so the comparison teaches as much as the code.

What you are building

```
   \\.\RingBuf                    a software device

   WriteFile()                    push bytes into a ring
   ReadFile()                     pop bytes; BLOCKS if empty,
                                  completing asynchronously
   IOCTL_RB_GET_STATS             counters
   IOCTL_RB_RESET                 empty it
   IOCTL_RB_SET_CONFIG            capacity policy

   A periodic timer generates synthetic data, standing in for
   an interrupt - so the asynchronous completion path is real
   even without hardware.

  WHAT IT EXERCISES:
     WDF object model and contexts       (chapter 13)
     the device lifecycle                (chapter 14)
     default and manual queues           (chapter 15)
     asynchronous completion from a DPC  (chapter 10)
     IOCTL design and validation         (chapter 16)
     spinlocks and IRQL discipline       (chapters 02, 09)
     an INF and installation             (chapter 19)
     Driver Verifier and WinDbg          (chapters 21, 22)

  That is most of the track, in about 600 lines.
```

The files

```
ringbuf\
  ringbuf.h          shared with user mode - the ABI
  driver.h           internal definitions
  driver.c           DriverEntry, EvtDeviceAdd
  queue.c            the I/O callbacks
  ring.c             the buffer itself
  ringbuf.inf
  ringbuf.vcxproj
  test\
    rbtest.c         the client (chapter 24)
```

ringbuf.h — the shared ABI

```
#pragma once

#include <initguid.h>

// {8F3A2B1C-4D5E-6F70-8192-A3B4C5D6E7F8}
DEFINE_GUID(GUID_DEVINTERFACE_RINGBUF,
    0x8f3a2b1c, 0x4d5e, 0x6f70, 0x81, 0x92,
    0xa3, 0xb4, 0xc5, 0xd6, 0xe7, 0xf8);

#define RB_DEVICE_TYPE   0x8123

typedef struct _RB_STATS {
        UINT32 Capacity;
        UINT32 Used;
        UINT64 BytesWritten;
        UINT64 BytesRead;
        UINT64 Drops;
        UINT64 TimerTicks;
        UINT32 PendingReads;
        UINT32 Reserved[7];        /* MUST be zero */
} RB_STATS, *PRB_STATS;

typedef struct _RB_CONFIG {
        UINT32 Overwrite;          /* 0 or 1 */
        UINT32 TimerPeriodMs;      /* 0 disables the generator */
        UINT32 Reserved[6];
} RB_CONFIG, *PRB_CONFIG;

#define IOCTL_RB_GET_STATS \
    CTL_CODE(RB_DEVICE_TYPE, 0x800, METHOD_BUFFERED, \
             FILE_READ_ACCESS)

#define IOCTL_RB_RESET \
    CTL_CODE(RB_DEVICE_TYPE, 0x801, METHOD_BUFFERED, \
             FILE_WRITE_ACCESS)

#define IOCTL_RB_SET_CONFIG \
    CTL_CODE(RB_DEVICE_TYPE, 0x802, METHOD_BUFFERED, \
             FILE_WRITE_ACCESS)

#define IOCTL_RB_GET_CONFIG \
    CTL_CODE(RB_DEVICE_TYPE, 0x803, METHOD_BUFFERED, \
             FILE_READ_ACCESS)

/* NOTE the Access fields: GET_* need only read access,
   SET_* and RESET require write. The I/O manager enforces
   this before the driver sees the request (chapter 16) -
   free, correct access control.                          */
```

driver.h

```
#pragma once

#include <ntddk.h>
#include <wdf.h>
#include "ringbuf.h"

#define RB_POOL_TAG        'FBgR'      /* shows as "RgBF" */
#define RB_DEFAULT_CAPACITY 4096
#define RB_MAX_CAPACITY     (1 * 1024 * 1024)

typedef struct _RB_CONTEXT {
        WDFDEVICE       Device;
        WDFQUEUE        DefaultQueue;
        WDFQUEUE        PendingReadQueue;   /* manual */
        WDFTIMER        Timer;
        WDFSPINLOCK     Lock;

        PUCHAR          Buffer;
        ULONG           Capacity;           /* power of two */
        ULONG           Head;               /* free-running */
        ULONG           Tail;

        BOOLEAN         Overwrite;
        ULONG           TimerPeriodMs;

        ULONG64         BytesWritten;
        ULONG64         BytesRead;
        ULONG64         Drops;
        ULONG64         TimerTicks;
        ULONG           PendingReads;
} RB_CONTEXT, *PRB_CONTEXT;

WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(RB_CONTEXT, RbGetContext)

DRIVER_INITIALIZE                       DriverEntry;
EVT_WDF_DRIVER_DEVICE_ADD               RbEvtDeviceAdd;
EVT_WDF_OBJECT_CONTEXT_CLEANUP          RbEvtDeviceCleanup;
EVT_WDF_IO_QUEUE_IO_READ                RbEvtIoRead;
EVT_WDF_IO_QUEUE_IO_WRITE               RbEvtIoWrite;
EVT_WDF_IO_QUEUE_IO_DEVICE_CONTROL      RbEvtIoDeviceControl;
EVT_WDF_TIMER                           RbEvtTimer;

/* ring.c - all callers must hold ctx->Lock */
ULONG RbUsed(_In_ PRB_CONTEXT Ctx);
ULONG RbSpace(_In_ PRB_CONTEXT Ctx);
ULONG RbPush(_In_ PRB_CONTEXT Ctx, _In_reads_bytes_(Len)
             const UCHAR *Src, _In_ ULONG Len);
ULONG RbPop(_In_ PRB_CONTEXT Ctx, _Out_writes_bytes_(Len)
            UCHAR *Dst, _In_ ULONG Len);
VOID  RbCompletePendingReads(_In_ PRB_CONTEXT Ctx);
```

driver.c

```
#include "driver.h"

#ifdef ALLOC_PRAGMA
#pragma alloc_text (INIT, DriverEntry)
#pragma alloc_text (PAGE, RbEvtDeviceAdd)
#endif

NTSTATUS
DriverEntry(_In_ PDRIVER_OBJECT DriverObject,
            _In_ PUNICODE_STRING RegistryPath)
{
        WDF_DRIVER_CONFIG config;
        NTSTATUS status;

        WDF_DRIVER_CONFIG_INIT(&config, RbEvtDeviceAdd);

        status = WdfDriverCreate(DriverObject, RegistryPath,
                                 WDF_NO_OBJECT_ATTRIBUTES,
                                 &config, WDF_NO_HANDLE);
        if (!NT_SUCCESS(status))
                KdPrint(("ringbuf: WdfDriverCreate 0x%x\n",
                         status));

        return status;
}

NTSTATUS
RbEvtDeviceAdd(_In_ WDFDRIVER Driver,
               _Inout_ PWDFDEVICE_INIT DeviceInit)
{
        WDF_OBJECT_ATTRIBUTES attributes;
        WDF_IO_QUEUE_CONFIG queueConfig;
        WDF_TIMER_CONFIG timerConfig;
        WDFDEVICE device;
        PRB_CONTEXT ctx;
        NTSTATUS status;

        UNREFERENCED_PARAMETER(Driver);
        PAGED_CODE();

        /* buffered I/O: the framework copies user buffers for
           us, safe at any IRQL (chapter 07)                */
        WdfDeviceInitSetIoType(DeviceInit, WdfDeviceIoBuffered);

        /* who may open it (chapter 03). Administrators and
           SYSTEM only - tighten or loosen deliberately.    */
        {
                DECLARE_CONST_UNICODE_STRING(sddl,
                        L"D:P(A;;GA;;;SY)(A;;GA;;;BA)");
                status = WdfDeviceInitAssignSDDLString(
                                DeviceInit, &sddl);
                if (!NT_SUCCESS(status))
                        return status;
        }

        WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attributes,
                                                RB_CONTEXT);
        attributes.EvtCleanupCallback = RbEvtDeviceCleanup;

        status = WdfDeviceCreate(&DeviceInit, &attributes,
```

```
                            &device);
    if (!NT_SUCCESS(status))
            return status;

    ctx = RbGetContext(device);
    ctx->Device        = device;
    ctx->Capacity      = RB_DEFAULT_CAPACITY;
    ctx->TimerPeriodMs = 100;
    ctx->Overwrite     = TRUE;

    /* the ring itself - NON-PAGED, because the timer DPC
       touches it at DISPATCH_LEVEL (chapter 06)        */
    ctx->Buffer = (PUCHAR)ExAllocatePool2(
                    POOL_FLAG_NON_PAGED,
                    ctx->Capacity, RB_POOL_TAG);
    if (ctx->Buffer == NULL)
            return STATUS_INSUFFICIENT_RESOURCES;

    /* a lock, parented to the device (chapter 13) */
    WDF_OBJECT_ATTRIBUTES_INIT(&attributes);
    attributes.ParentObject = device;
    status = WdfSpinLockCreate(&attributes, &ctx->Lock);
    if (!NT_SUCCESS(status))
            return status;

    /* the default queue - parallel dispatch */
    WDF_IO_QUEUE_CONFIG_INIT_DEFAULT_QUEUE(&queueConfig,
                    WdfIoQueueDispatchParallel);
    queueConfig.EvtIoRead          = RbEvtIoRead;
    queueConfig.EvtIoWrite         = RbEvtIoWrite;
    queueConfig.EvtIoDeviceControl = RbEvtIoDeviceControl;

    status = WdfIoQueueCreate(device, &queueConfig,
                    WDF_NO_OBJECT_ATTRIBUTES,
                    &ctx->DefaultQueue);
    if (!NT_SUCCESS(status))
            return status;

    /* the manual queue for pending reads. NOT power
       managed, because the timer DPC retrieves from it
       (chapter 15).                                    */
    WDF_IO_QUEUE_CONFIG_INIT(&queueConfig,
                    WdfIoQueueDispatchManual);
    queueConfig.PowerManaged = WdfFalse;

    status = WdfIoQueueCreate(device, &queueConfig,
                    WDF_NO_OBJECT_ATTRIBUTES,
                    &ctx->PendingReadQueue);
    if (!NT_SUCCESS(status))
            return status;

    /* the data generator, standing in for an interrupt */
    WDF_TIMER_CONFIG_INIT_PERIODIC(&timerConfig,
                                   RbEvtTimer,
                                   ctx->TimerPeriodMs);
    WDF_OBJECT_ATTRIBUTES_INIT(&attributes);
    attributes.ParentObject = device;

    status = WdfTimerCreate(&timerConfig, &attributes,
                            &ctx->Timer);
    if (!NT_SUCCESS(status))
            return status;
```

```
        /* publish the interface LAST (chapter 17) */
        status = WdfDeviceCreateDeviceInterface(device,
                        &GUID_DEVINTERFACE_RINGBUF, NULL);
        if (!NT_SUCCESS(status))
                return status;

        WdfTimerStart(ctx->Timer,
                      WDF_REL_TIMEOUT_IN_MS(ctx->TimerPeriodMs));

        KdPrint(("ringbuf: ready, %lu bytes\n", ctx->Capacity));
        return STATUS_SUCCESS;
}

VOID
RbEvtDeviceCleanup(_In_ WDFOBJECT Object)
{
        PRB_CONTEXT ctx = RbGetContext(Object);

        /* stop the timer AND WAIT (chapter 10) - otherwise it
           can fire against the buffer we are about to free */
        if (ctx->Timer)
                WdfTimerStop(ctx->Timer, TRUE);

        if (ctx->Buffer) {
                ExFreePoolWithTag(ctx->Buffer, RB_POOL_TAG);
                ctx->Buffer = NULL;
        }

        /* every WDF object is parented to the device and dies
           with it - no other cleanup needed (chapter 13)   */
}
```

ring.c

```
#include "driver.h"

/* Free-running Head and Tail; unsigned wraparound removes the
   full-versus-empty ambiguity entirely. Same trick as the
   Linux version (chapter 28).                              */

ULONG RbUsed(_In_ PRB_CONTEXT Ctx)
{
        return Ctx->Head - Ctx->Tail;
}

ULONG RbSpace(_In_ PRB_CONTEXT Ctx)
{
        return Ctx->Capacity - RbUsed(Ctx);
}

ULONG
RbPush(_In_ PRB_CONTEXT Ctx,
       _In_reads_bytes_(Len) const UCHAR *Src, _In_ ULONG Len)
{
        ULONG n, first, mask = Ctx->Capacity - 1;

        if (Ctx->Overwrite) {
                if (Len > Ctx->Capacity) {
                        Src += Len - Ctx->Capacity;
                        Ctx->Drops += Len - Ctx->Capacity;
                        Len = Ctx->Capacity;
                }
                if (Len > RbSpace(Ctx)) {
                        ULONG discard = Len - RbSpace(Ctx);

                        Ctx->Tail  += discard;
                        Ctx->Drops += discard;
                }
                n = Len;
        } else {
                n = min(Len, RbSpace(Ctx));
        }

        first = min(n, Ctx->Capacity - (Ctx->Head & mask));
        RtlCopyMemory(Ctx->Buffer + (Ctx->Head & mask),
                      Src, first);
        if (n > first)
                RtlCopyMemory(Ctx->Buffer, Src + first,
                              n - first);

        Ctx->Head         += n;
        Ctx->BytesWritten += n;
        return n;
}

ULONG
RbPop(_In_ PRB_CONTEXT Ctx,
      _Out_writes_bytes_(Len) UCHAR *Dst, _In_ ULONG Len)
{
        ULONG n, first, mask = Ctx->Capacity - 1;

        n = min(Len, RbUsed(Ctx));

        first = min(n, Ctx->Capacity - (Ctx->Tail & mask));
```

```
        RtlCopyMemory(Dst, Ctx->Buffer + (Ctx->Tail & mask),
                      first);
        if (n > first)
                RtlCopyMemory(Dst + first, Ctx->Buffer,
                              n - first);

        Ctx->Tail      += n;
        Ctx->BytesRead += n;
        return n;
}

/* Called from the timer DPC and from write, WITHOUT the lock
   held - it takes the lock itself around each pop.        */
VOID
RbCompletePendingReads(_In_ PRB_CONTEXT Ctx)
{
        WDFREQUEST request;
        NTSTATUS status;

        for (;;) {
                PVOID buffer;
                size_t bufLen;
                ULONG got;

                status = WdfIoQueueRetrieveNextRequest(
                                Ctx->PendingReadQueue,
                                &request);
                if (!NT_SUCCESS(status))
                        break;      /* nobody waiting */

                /* WE NOW OWN THIS REQUEST. It cannot be
                   cancelled behind us (chapter 15) - so it
                   MUST be completed on every path below.  */

                status = WdfRequestRetrieveOutputBuffer(request,
                                1, &buffer, &bufLen);
                if (!NT_SUCCESS(status)) {
                        WdfRequestComplete(request, status);
                        continue;
                }

                WdfSpinLockAcquire(Ctx->Lock);
                got = RbPop(Ctx, (PUCHAR)buffer,
                            (ULONG)min(bufLen, MAXULONG));
                if (got)
                        Ctx->PendingReads--;
                WdfSpinLockRelease(Ctx->Lock);

                if (got == 0) {
                        /* raced - nothing left. Put it back
                           rather than completing it with
                           zero, which userspace would read as
                           end of file (chapter 20 of the
                           Linux track).                    */
                        status = WdfRequestForwardToIoQueue(
                                request, Ctx->PendingReadQueue);
                        if (!NT_SUCCESS(status))
                                WdfRequestComplete(request,
                                                   status);
                        break;
                }

                WdfRequestCompleteWithInformation(request,
```

```
                                    STATUS_SUCCESS, got);
        }
}
```

queue.c

```
#include "driver.h"

VOID
RbEvtIoRead(_In_ WDFQUEUE Queue, _In_ WDFREQUEST Request,
            _In_ size_t Length)
{
        PRB_CONTEXT ctx = RbGetContext(
                        WdfIoQueueGetDevice(Queue));
        PVOID buffer;
        size_t bufLen;
        ULONG got;
        NTSTATUS status;

        if (Length == 0) {
                WdfRequestComplete(Request, STATUS_SUCCESS);
                return;
        }

        status = WdfRequestRetrieveOutputBuffer(Request, 1,
                                                &buffer, &bufLen);
        if (!NT_SUCCESS(status)) {
                WdfRequestComplete(Request, status);
                return;
        }

        WdfSpinLockAcquire(ctx->Lock);
        got = RbPop(ctx, (PUCHAR)buffer,
                    (ULONG)min(bufLen, MAXULONG));
        if (got == 0)
                ctx->PendingReads++;
        WdfSpinLockRelease(ctx->Lock);

        if (got) {
                WdfRequestCompleteWithInformation(Request,
                                STATUS_SUCCESS, got);
                return;
        }

        /* nothing available - park it. The framework now owns
           cancellation and cleanup for this request.      */
        status = WdfRequestForwardToIoQueue(Request,
                                ctx->PendingReadQueue);
        if (!NT_SUCCESS(status)) {
                WdfSpinLockAcquire(ctx->Lock);
                ctx->PendingReads--;
                WdfSpinLockRelease(ctx->Lock);
                WdfRequestComplete(Request, status);
        }
}

VOID
RbEvtIoWrite(_In_ WDFQUEUE Queue, _In_ WDFREQUEST Request,
             _In_ size_t Length)
{
        PRB_CONTEXT ctx = RbGetContext(
                        WdfIoQueueGetDevice(Queue));
        PVOID buffer;
        size_t bufLen;
        ULONG pushed;
        NTSTATUS status;
```

```
        if (Length == 0) {
                WdfRequestComplete(Request, STATUS_SUCCESS);
                return;
        }

        status = WdfRequestRetrieveInputBuffer(Request, 1,
                                               &buffer, &bufLen);
        if (!NT_SUCCESS(status)) {
                WdfRequestComplete(Request, status);
                return;
        }

        WdfSpinLockAcquire(ctx->Lock);
        pushed = RbPush(ctx, (const UCHAR *)buffer,
                        (ULONG)min(bufLen, MAXULONG));
        WdfSpinLockRelease(ctx->Lock);

        /* wake anyone waiting - OUTSIDE the lock, because
           completing a request can be slow                */
        if (pushed)
                RbCompletePendingReads(ctx);

        WdfRequestCompleteWithInformation(Request,
                        STATUS_SUCCESS, pushed);
}

VOID
RbEvtTimer(_In_ WDFTIMER Timer)
{
        PRB_CONTEXT ctx = RbGetContext(
                        WdfTimerGetParentObject(Timer));
        UCHAR sample[16];
        ULONG i;

        /* DISPATCH_LEVEL. No paged memory, no blocking
           (chapter 02).                                 */

        WdfSpinLockAcquire(ctx->Lock);
        ctx->TimerTicks++;
        for (i = 0; i < sizeof(sample); i++)
                sample[i] = (UCHAR)(ctx->TimerTicks + i);
        RbPush(ctx, sample, sizeof(sample));
        WdfSpinLockRelease(ctx->Lock);

        RbCompletePendingReads(ctx);
}

VOID
RbEvtIoDeviceControl(_In_ WDFQUEUE Queue,
                     _In_ WDFREQUEST Request,
                     _In_ size_t OutputBufferLength,
                     _In_ size_t InputBufferLength,
                     _In_ ULONG IoControlCode)
{
        PRB_CONTEXT ctx = RbGetContext(
                        WdfIoQueueGetDevice(Queue));
        NTSTATUS status = STATUS_INVALID_DEVICE_REQUEST;
        size_t info = 0;

        UNREFERENCED_PARAMETER(OutputBufferLength);
        UNREFERENCED_PARAMETER(InputBufferLength);
```

```
        switch (IoControlCode) {

        case IOCTL_RB_GET_STATS: {
                PRB_STATS st;

                status = WdfRequestRetrieveOutputBuffer(Request,
                                sizeof(RB_STATS),
                                (PVOID *)&st, NULL);
                if (!NT_SUCCESS(status))
                        break;

                RtlZeroMemory(st, sizeof(*st));

                WdfSpinLockAcquire(ctx->Lock);
                st->Capacity     = ctx->Capacity;
                st->Used         = RbUsed(ctx);
                st->BytesWritten = ctx->BytesWritten;
                st->BytesRead    = ctx->BytesRead;
                st->Drops        = ctx->Drops;
                st->TimerTicks   = ctx->TimerTicks;
                st->PendingReads = ctx->PendingReads;
                WdfSpinLockRelease(ctx->Lock);

                info = sizeof(RB_STATS);
                status = STATUS_SUCCESS;
                break;
        }

        case IOCTL_RB_RESET:
                WdfSpinLockAcquire(ctx->Lock);
                ctx->Head = ctx->Tail = 0;
                WdfSpinLockRelease(ctx->Lock);
                status = STATUS_SUCCESS;
                break;

        case IOCTL_RB_SET_CONFIG: {
                PRB_CONFIG cfg;
                RB_CONFIG local;

                status = WdfRequestRetrieveInputBuffer(Request,
                                sizeof(RB_CONFIG),
                                (PVOID *)&cfg, NULL);
                if (!NT_SUCCESS(status))
                        break;

                /* COPY IN, then validate the copy
                   (chapter 16)                          */
                RtlCopyMemory(&local, cfg, sizeof(local));

                if (local.Overwrite > 1) {
                        status = STATUS_INVALID_PARAMETER;
                        break;
                }
                if (local.TimerPeriodMs > 10000) {
                        status = STATUS_INVALID_PARAMETER;
                        break;
                }

                WdfSpinLockAcquire(ctx->Lock);
                ctx->Overwrite     = (BOOLEAN)local.Overwrite;
                ctx->TimerPeriodMs = local.TimerPeriodMs;
                WdfSpinLockRelease(ctx->Lock);
```

```
                WdfTimerStop(ctx->Timer, FALSE);
                if (local.TimerPeriodMs)
                        WdfTimerStart(ctx->Timer,
                                WDF_REL_TIMEOUT_IN_MS(
                                        local.TimerPeriodMs));

                status = STATUS_SUCCESS;
                break;
        }

        case IOCTL_RB_GET_CONFIG: {
                PRB_CONFIG cfg;

                status = WdfRequestRetrieveOutputBuffer(Request,
                                sizeof(RB_CONFIG),
                                (PVOID *)&cfg, NULL);
                if (!NT_SUCCESS(status))
                        break;

                RtlZeroMemory(cfg, sizeof(*cfg));
                cfg->Overwrite     = ctx->Overwrite;
                cfg->TimerPeriodMs = ctx->TimerPeriodMs;
                info = sizeof(RB_CONFIG);
                status = STATUS_SUCCESS;
                break;
        }

        default:
                break;
        }

        WdfRequestCompleteWithInformation(Request, status, info);
}
```

ringbuf.inf

```
[Version]
Signature   = "$WINDOWS NT$"
Class       = System
ClassGuid   = {4d36e97d-e325-11ce-bfc1-08002be10318}
Provider    = %ProviderName%
CatalogFile = ringbuf.cat
DriverVer   = 03/15/2026,1.0.0.0
PnpLockdown = 1

[DestinationDirs]
DefaultDestDir = 13

[SourceDisksNames]
1 = %DiskName%

[SourceDisksFiles]
ringbuf.sys = 1

[Manufacturer]
%ProviderName% = Standard,NTamd64,NTarm64

; a ROOT-enumerated software device - no hardware needed
[Standard.NTamd64]
%DeviceDesc% = RingBuf_Install, root\ringbuf

[Standard.NTarm64]
%DeviceDesc% = RingBuf_Install, root\ringbuf

[RingBuf_Install.NT]
CopyFiles = RingBuf_CopyFiles

[RingBuf_CopyFiles]
ringbuf.sys

[RingBuf_Install.NT.Services]
AddService = ringbuf, 0x00000002, RingBuf_Service

[RingBuf_Service]
DisplayName   = %ServiceDesc%
ServiceType   = 1
StartType     = 3
ErrorControl  = 1
ServiceBinary = %13%\ringbuf.sys

[RingBuf_Install.NT.Wdf]
KmdfService = ringbuf, RingBuf_wdfsect

[RingBuf_wdfsect]
KmdfLibraryVersion = 1.15

[Strings]
ProviderName = "Embedded Learning Hub"
DiskName     = "RingBuf Installation Disk"
DeviceDesc   = "RingBuf Test Device"
ServiceDesc  = "RingBuf Driver"
```

Building and installing

```
# --- On the DEVELOPMENT machine ---
    msbuild ringbuf.vcxproj /p:Configuration=Debug \
            /p:Platform=x64
    ; the WDK template signs it with a test certificate

# --- On the TEST machine (a VM), once ---
    bcdedit /set testsigning on
    bcdedit /debug on
    bcdedit /dbgsettings serial debugport:1 baudrate:115200
    shutdown /r /t 0

# --- Install ---
    certmgr /add ringbuf.cer /s /r localMachine root
    certmgr /add ringbuf.cer /s /r localMachine trustedpublisher

    pnputil /add-driver ringbuf.inf /install
    devcon install ringbuf.inf root\ringbuf

# --- Verify it started ---
    devcon status root\ringbuf
      root\ringbuf\0000
          Name: RingBuf Test Device
          Driver is running.

    ; and in Device Manager, under System devices

# --- The development loop, scripted ---
    devcon remove root\ringbuf
    pnputil /delete-driver oem12.inf /uninstall /force
    ; rebuild, copy over
    pnputil /add-driver ringbuf.inf /install
    devcon install ringbuf.inf root\ringbuf

# --- Turn the verifiers on immediately (chapter 21) ---
    verifier /flags 0x209BB /driver ringbuf.sys
    shutdown /r /t 0
```

Exercises

BREAK IT DELIBERATELY. EACH OF THESE PRODUCES A SPECIFIC, DIAGNOSABLE FAILURE.

1. **Remove the spinlock** from `RbEvtIoWrite`. Run several writers and a reader. Watch the counters drift. Then re-add it.
2. **Allocate the buffer from paged pool.** Enable Verifier's force-IRQL flag. The timer DPC now bugchecks with 0xA immediately, rather than in three weeks (chapter 02).
3. **Complete a forwarded request.** In `RbEvtIoRead`, call `WdfRequestComplete` after `WdfRequestForwardToIoQueue`. WDF Verifier catches it (chapter 13).
4. **Make the pending queue power-managed.** Let the device idle, then read. The request never completes — and `!wdfdevicequeues` shows the queue not dispatching (chapter 15).
5. **Drop** `WdfTimerStop(..., TRUE)` from cleanup. Uninstall while the timer is running. With special pool on, you get a use-after-free bugcheck naming your driver.
6. **Skip the** `RtlZeroMemory` in `GET_STATS`. The reserved fields now leak whatever was in the buffer — an information disclosure, in one deleted line.
7. **Remove the validation** from `SET_CONFIG` and pass a huge `TimerPeriodMs`. Then run the WDK's IoAttack fuzzer against it and see what else it finds.
8. **Change** `IOCTL_RB_RESET` **to** `FILE_ANY_ACCESS` and reset the buffer from a read-only handle. Observe that the platform was preventing this for free (chapter 16).
9. **Leak pool:** remove the `ExFreePoolWithTag`. Install and uninstall ten times with pool tracking on. Bugcheck 0xC4, Arg1 = 0x62, with the allocation stack.
10. **Add a feature:** support several concurrent readers, each with an independent read position. This requires per-file-object state (chapter 15) and changes the ring's semantics — think carefully about what "consumed" means with several readers.
11. **Add an** `EvtIoStop` **handler** and verify the machine can now sleep with a read pending.
12. **Convert to a real interrupt.** If you have any PCIe device with a documented register map, replace the timer with a `WDFINTERRUPT` (chapter 17). The queue and completion logic does not change at all — which is the point.

The review pass

```
# Before you call it finished:

# 1. Static analysis, clean (chapter 21)
    msbuild /p:RunCodeAnalysis=true
    ; and Static Driver Verifier

# 2. Driver Verifier, standard flags, through a full test run

# 3. Low resources simulation
    verifier /flags 0x209BF /driver ringbuf.sys
    ; expect to find unchecked allocations; there is at least
    ; one obvious candidate in EvtDeviceAdd

# 4. 50 install/uninstall cycles
# 5. 50 disable/enable cycles
    for /L %i in (1,1,50) do devcon disable root\ringbuf & \
                             devcon enable root\ringbuf

# 6. Sleep and resume with a read pending
    pwrtest /sleep /c:5

# 7. The HLK Device Fundamentals tests (chapter 20)

# 8. IoAttack against the IOCTL interface (chapter 16)

# STEPS 3, 5 AND 8 find the most. Step 6 finds the
# EvtIoStop omission specifically.
```

Check yourself

- ✓ Explain why the ring buffer must be in non-paged pool.
- ✓ Justify the choice of buffered I/O and the SDDL string.
- ✓ Explain why the pending queue is not power-managed.
- ✓ State the request ownership rule and find every place the code honours it.
- ✓ Explain why `RbCompletePendingReads` is called outside the lock.
- ✓ Explain the requeue-on-zero path and what bug it prevents.
- ✓ Justify the Access field on each IOCTL.
- ✓ Explain why `RtlZeroMemory` on the output struct is a security control.
- ✓ Complete exercises 1 through 9 and describe each symptom.
- ✓ Complete at least one of exercises 10 to 12.
- ✓ Pass the eight-step review with no findings.

> **NEXT**
> Chapter 24 builds the user-mode client: device discovery by interface GUID, synchronous and overlapped I/O, and a benchmark that shows what the asynchronous path actually buys.

Project: the user-mode client

The other half of the Windows project: finding the device by interface GUID, driving it synchronously and with overlapped I/O, watching for arrival and removal, and measuring what the asynchronous path is actually worth.

Finding the device

```
/* rbtest.c - build with:
     cl /W4 /Zi rbtest.c setupapi.lib cfgmgr32.lib   */

#include <windows.h>
#include <setupapi.h>
#include <cfgmgr32.h>
#include <stdio.h>
#include <stdlib.h>
#include "../ringbuf.h"

/* Open by INTERFACE GUID, not by a hard-coded name
   (chapter 05). This works for any number of instances and
   survives the device moving.                              */
static HANDLE RbOpen(BOOL overlapped)
{
        HDEVINFO info;
        SP_DEVICE_INTERFACE_DATA ifData = { sizeof(ifData) };
        PSP_DEVICE_INTERFACE_DETAIL_DATA_W detail = NULL;
        DWORD needed = 0;
        HANDLE h = INVALID_HANDLE_VALUE;
        DWORD flags = FILE_ATTRIBUTE_NORMAL;

        if (overlapped)
                flags |= FILE_FLAG_OVERLAPPED;

        info = SetupDiGetClassDevsW(&GUID_DEVINTERFACE_RINGBUF,
                        NULL, NULL,
                        DIGCF_PRESENT | DIGCF_DEVICEINTERFACE);
        if (info == INVALID_HANDLE_VALUE) {
                fprintf(stderr, "SetupDiGetClassDevs: %lu\n",
                        GetLastError());
                return INVALID_HANDLE_VALUE;
        }

        if (!SetupDiEnumDeviceInterfaces(info, NULL,
                        &GUID_DEVINTERFACE_RINGBUF, 0, &ifData)) {
                fprintf(stderr, "no device present (%lu)\n",
                        GetLastError());
                goto out;
        }

        /* two-call idiom: ask the size, then fetch */
        SetupDiGetDeviceInterfaceDetailW(info, &ifData, NULL, 0,
                                         &needed, NULL);
        detail = (PSP_DEVICE_INTERFACE_DETAIL_DATA_W)
                        malloc(needed);
        if (!detail)
                goto out;

        detail->cbSize =
                sizeof(SP_DEVICE_INTERFACE_DETAIL_DATA_W);

        if (!SetupDiGetDeviceInterfaceDetailW(info, &ifData,
                        detail, needed, &needed, NULL)) {
                fprintf(stderr, "GetDetail: %lu\n",
                        GetLastError());
                goto out;
        }

        wprintf(L"opening %s\n", detail->DevicePath);
```

```
        h = CreateFileW(detail->DevicePath,
                        GENERIC_READ | GENERIC_WRITE,
                        0, NULL, OPEN_EXISTING, flags, NULL);
        if (h == INVALID_HANDLE_VALUE)
                fprintf(stderr, "CreateFile: %lu\n",
                        GetLastError());

out:
        free(detail);
        SetupDiDestroyDeviceInfoList(info);
        return h;
}

/* THE TWO-CALL IDIOM appears throughout the Windows API: call
   once with a NULL buffer to learn the size, allocate, call
   again. It returns ERROR_INSUFFICIENT_BUFFER on the first
   call, which is expected, not a failure.                */
```

Synchronous use

```
static void ShowStats(HANDLE h)
{
        RB_STATS st;
        DWORD returned;

        if (!DeviceIoControl(h, IOCTL_RB_GET_STATS,
                             NULL, 0,
                             &st, sizeof(st),
                             &returned, NULL)) {
                fprintf(stderr, "GET_STATS: %lu\n",
                        GetLastError());
                return;
        }

        printf("cap=%u used=%u written=%llu read=%llu "
               "drops=%llu ticks=%llu pending=%u\n",
               st.Capacity, st.Used,
               (unsigned long long)st.BytesWritten,
               (unsigned long long)st.BytesRead,
               (unsigned long long)st.Drops,
               (unsigned long long)st.TimerTicks,
               st.PendingReads);
}

static void TestRoundTrip(void)
{
        HANDLE h = RbOpen(FALSE);
        BYTE out[256], in[256];
        DWORD written, read;

        if (h == INVALID_HANDLE_VALUE)
                return;

        memset(out, 'A', sizeof(out));

        if (!WriteFile(h, out, sizeof(out), &written, NULL))
                fprintf(stderr, "WriteFile: %lu\n",
                        GetLastError());
        else
                printf("wrote %lu\n", written);

        if (!ReadFile(h, in, sizeof(in), &read, NULL))
                fprintf(stderr, "ReadFile: %lu\n",
                        GetLastError());
        else
                printf("read %lu\n", read);

        ShowStats(h);
        CloseHandle(h);
}

static void TestConfig(void)
{
        HANDLE h = RbOpen(FALSE);
        RB_CONFIG cfg;
        DWORD returned;

        if (h == INVALID_HANDLE_VALUE)
                return;
```

```
        memset(&cfg, 0, sizeof(cfg));   /* zero the reserved
                                           fields - the ABI
                                           requires it       */
        cfg.Overwrite     = 1;
        cfg.TimerPeriodMs = 50;

        if (!DeviceIoControl(h, IOCTL_RB_SET_CONFIG,
                             &cfg, sizeof(cfg), NULL, 0,
                             &returned, NULL))
                fprintf(stderr, "SET_CONFIG: %lu\n",
                        GetLastError());

        /* NEGATIVE TEST: an out-of-range value must be
           refused (chapter 16)                          */
        cfg.TimerPeriodMs = 99999;
        if (DeviceIoControl(h, IOCTL_RB_SET_CONFIG,
                            &cfg, sizeof(cfg), NULL, 0,
                            &returned, NULL))
                fprintf(stderr, "BUG: bad config accepted\n");
        else
                printf("bad config refused: %lu (expect 87)\n",
                       GetLastError());

        CloseHandle(h);
}
```

WIN32 ERRORS, NOT NTSTATUS

User mode sees Win32 error codes; the I/O manager translates. `STATUS_INVALID_PARAMETER` becomes `ERROR_INVALID_PARAMETER` (87), `STATUS_BUFFER_TOO_SMALL` becomes `ERROR_INSUFFICIENT_BUFFER` (122), `STATUS_INVALID_DEVICE_REQUEST` becomes `ERROR_INVALID_FUNCTION` (1). When a test asserts on an error code, assert on the Win32 one — and use `!error` in WinDbg to translate either direction.

Overlapped I/O

```
static void TestOverlapped(void)
{
        HANDLE h = RbOpen(TRUE);        /* FILE_FLAG_OVERLAPPED */
        OVERLAPPED ov = { 0 };
        BYTE buf[64];
        DWORD transferred;

        if (h == INVALID_HANDLE_VALUE)
                return;

        ov.hEvent = CreateEventW(NULL, TRUE, FALSE, NULL);

        printf("issuing an async read...\n");

        if (!ReadFile(h, buf, sizeof(buf), NULL, &ov)) {
                DWORD err = GetLastError();

                if (err != ERROR_IO_PENDING) {
                        fprintf(stderr, "ReadFile: %lu\n", err);
                        goto out;
                }
                printf("pending - doing other work\n");
        }

        /* THIS IS THE POINT: we are free while the driver
           waits for data. In the synchronous version the
           thread would be blocked here.                    */
        Sleep(50);
        printf("...other work done, now waiting\n");

        if (WaitForSingleObject(ov.hEvent, 5000) != WAIT_OBJECT_0) {
                fprintf(stderr, "timed out\n");
                CancelIoEx(h, &ov);     /* exercises the
                                           driver's
                                           cancellation path */
                goto out;
        }

        if (GetOverlappedResult(h, &ov, &transferred, FALSE))
                printf("async read completed: %lu bytes\n",
                       transferred);
        else
                fprintf(stderr, "GetOverlappedResult: %lu\n",
                        GetLastError());

out:
        CloseHandle(ov.hEvent);
        CloseHandle(h);
}

/* THE ERROR_IO_PENDING CONVENTION catches everyone once:
   ReadFile returns FALSE and GetLastError() is
   ERROR_IO_PENDING. That is SUCCESS. Treating FALSE as
   failure means your first async client never works.     */
```

An I/O completion port

```
/* The scalable pattern: several outstanding requests, one
   thread servicing all completions.                        */

#define RB_DEPTH  8

typedef struct _RB_OP {
        OVERLAPPED Overlapped;      /* MUST be first */
        BYTE       Buffer[64];
        int        Index;
} RB_OP;

static void TestCompletionPort(int seconds)
{
        HANDLE h = RbOpen(TRUE);
        HANDLE iocp;
        RB_OP ops[RB_DEPTH];
        ULONGLONG deadline;
        ULONG64 total = 0, count = 0;
        int i;

        if (h == INVALID_HANDLE_VALUE)
                return;

        iocp = CreateIoCompletionPort(h, NULL, 0, 1);
        if (!iocp) {
                fprintf(stderr, "CreateIoCompletionPort: %lu\n",
                        GetLastError());
                CloseHandle(h);
                return;
        }

        /* keep RB_DEPTH reads outstanding at all times */
        for (i = 0; i < RB_DEPTH; i++) {
                memset(&ops[i], 0, sizeof(ops[i]));
                ops[i].Index = i;
                if (!ReadFile(h, ops[i].Buffer,
                              sizeof(ops[i].Buffer), NULL,
                              &ops[i].Overlapped) &&
                    GetLastError() != ERROR_IO_PENDING) {
                        fprintf(stderr, "ReadFile %d: %lu\n", i,
                                GetLastError());
                        goto out;
                }
        }

        deadline = GetTickCount64() + (ULONGLONG)seconds * 1000;

        while (GetTickCount64() < deadline) {
                DWORD transferred;
                ULONG_PTR key;
                LPOVERLAPPED pov;
                RB_OP *op;

                if (!GetQueuedCompletionStatus(iocp,
                                &transferred, &key, &pov, 1000)) {
                        if (pov == NULL)
                                continue;      /* timeout */
                        fprintf(stderr, "completion error %lu\n",
                                GetLastError());
                        break;
```

```
                }

                op = CONTAINING_RECORD(pov, RB_OP, Overlapped);
                total += transferred;
                count++;

                /* re-issue immediately */
                memset(&op->Overlapped, 0,
                       sizeof(op->Overlapped));
                if (!ReadFile(h, op->Buffer, sizeof(op->Buffer),
                              NULL, &op->Overlapped) &&
                    GetLastError() != ERROR_IO_PENDING)
                        break;
        }

        printf("%llu completions, %llu bytes in %d s "
               "(%.1f completions/s)\n",
               (unsigned long long)count,
               (unsigned long long)total, seconds,
               (double)count / seconds);

out:
        CancelIoEx(h, NULL);        /* cancel everything
                                       outstanding          */
        CloseHandle(iocp);
        CloseHandle(h);
}

/* CONTAINING_RECORD recovers your structure from the
   OVERLAPPED pointer - the Windows equivalent of
   container_of. That is why OVERLAPPED must be the first
   member, or at a known offset.                          */

/* AND THE CANCELIOEX AT THE END is not tidiness: it makes
   the driver's cancellation path run on every test, which is
   how you find the bugs in it.                           */
```

Watching for arrival and removal

```
/* React when the device appears or disappears, rather than
   polling for it.                                          */

static DWORD CALLBACK
RbNotifyCallback(HCMNOTIFICATION notify, PVOID context,
                 CM_NOTIFY_ACTION action,
                 PCM_NOTIFY_EVENT_DATA data, DWORD size)
{
        UNREFERENCED_PARAMETER(notify);
        UNREFERENCED_PARAMETER(context);
        UNREFERENCED_PARAMETER(size);

        switch (action) {
        case CM_NOTIFY_ACTION_DEVICEINTERFACEARRIVAL:
                wprintf(L"device arrived: %s\n",
                        data->u.DeviceInterface.SymbolicLink);
                break;
        case CM_NOTIFY_ACTION_DEVICEINTERFACEREMOVAL:
                wprintf(L"device removed\n");
                break;
        case CM_NOTIFY_ACTION_DEVICEQUERYREMOVE:
                wprintf(L"query remove - close your handles\n");
                /* RETURN ERROR_SUCCESS to allow it. Returning
                   an error VETOES the removal, which the user
                   sees as "the device is in use". Only veto if
                   you genuinely cannot stop.              */
                break;
        case CM_NOTIFY_ACTION_DEVICEREMOVECOMPLETE:
                wprintf(L"remove complete\n");
                break;
        }
        return ERROR_SUCCESS;
}

static void WatchDevice(void)
{
        CM_NOTIFY_FILTER filter = { 0 };
        HCMNOTIFICATION notify = NULL;
        CONFIGRET cr;

        filter.cbSize = sizeof(filter);
        filter.FilterType =
                CM_NOTIFY_FILTER_TYPE_DEVICEINTERFACE;
        filter.u.DeviceInterface.ClassGuid =
                GUID_DEVINTERFACE_RINGBUF;

        cr = CM_Register_Notification(&filter, NULL,
                                      RbNotifyCallback, &notify);
        if (cr != CR_SUCCESS) {
                fprintf(stderr, "CM_Register_Notification: %d\n",
                        cr);
                return;
        }

        printf("watching - disable/enable the device in Device "
               "Manager. Enter to stop.\n");
        getchar();

        CM_Unregister_Notification(notify);
}
```

```
/* THIS IS HOW A REAL APPLICATION handles hot-plug: it
   registers for the interface class, opens the device on
   arrival, and closes its handles on query-remove. An
   application that keeps a handle open through a removal
   attempt is what makes "Windows cannot stop this device"
   dialogs appear.                                       */
```

Measuring the difference

```
static void Benchmark(void)
{
        HANDLE h;
        BYTE buf[64];
        DWORD read;
        LARGE_INTEGER freq, start, end;
        int i;
        const int N = 2000;

        QueryPerformanceFrequency(&freq);

        /* --- synchronous --- */
        h = RbOpen(FALSE);
        if (h == INVALID_HANDLE_VALUE)
                return;

        QueryPerformanceCounter(&start);
        for (i = 0; i < N; i++) {
                if (!ReadFile(h, buf, sizeof(buf), &read, NULL))
                        break;
        }
        QueryPerformanceCounter(&end);
        CloseHandle(h);

        printf("sync:  %d reads in %.1f ms (%.1f us each)\n",
               i,
               (end.QuadPart - start.QuadPart) * 1000.0
                       / freq.QuadPart,
               (end.QuadPart - start.QuadPart) * 1e6
                       / freq.QuadPart / (i ? i : 1));

        /* --- the completion-port version, for comparison --- */
        TestCompletionPort(2);
}

/* WHAT YOU WILL SEE: with the driver's 100 ms timer, the
   synchronous loop is limited to roughly one read per timer
   tick, because each read blocks until data arrives. The
   completion-port version keeps eight requests outstanding
   and drains a whole tick's data at once.

   THE LESSON, and it is the reason asynchronous I/O exists:
   the bottleneck is not CPU time, it is the round trip. Depth
   hides latency. Exactly the argument for batching across any
   expensive boundary - the same one as SMC calls in isolation
   chapter 11.                                                */

int main(int argc, char **argv)
{
        const char *what = (argc > 1) ? argv[1] : "all";

        if (!strcmp(what, "all") || !strcmp(what, "sync")) {
                printf("--- round trip ---\n");  TestRoundTrip();
                printf("--- config ---\n");      TestConfig();
        }
        if (!strcmp(what, "all") || !strcmp(what, "async")) {
                printf("--- overlapped ---\n");  TestOverlapped();
                printf("--- iocp ---\n");        TestCompletionPort(3);
        }
        if (!strcmp(what, "all") || !strcmp(what, "bench")) {
```

```
                printf("--- benchmark ---\n");   Benchmark();
        }
        if (!strcmp(what, "watch"))
                WatchDevice();

        return 0;
}
```

Running it

```
# Build (from a Developer Command Prompt):
    cl /W4 /Zi rbtest.c setupapi.lib cfgmgr32.lib

# With the driver installed (chapter 23):
    rbtest.exe
      opening \\?\root#system#0000#{8f3a2b1c-...}
      --- round trip ---
      wrote 256
      read 256
      cap=4096 used=0 written=256 read=256 drops=0 ticks=41 pending=0
      --- config ---
      bad config refused: 87 (expect 87)
      --- overlapped ---
      issuing an async read...
      pending - doing other work
      ...other work done, now waiting
      async read completed: 64 bytes
      --- iocp ---
      152 completions, 9728 bytes in 3 s (50.7 completions/s)

# NOTE the completion rate tracks the driver's timer period -
# 50 ms in the config test means ~20 ticks/second, times the
# bytes per tick divided by the read size. If your numbers do
# not line up, work out why: that arithmetic is a good check
# that you understand what the driver is doing.

# Hot-plug behaviour:
    rbtest.exe watch
    ; then, in another window:
    devcon disable root\ringbuf
    devcon enable root\ringbuf

# ACCESS CONTROL - confirm the SDDL works (chapter 23):
    ; as a standard user, not an administrator:
    rbtest.exe
      CreateFile: 5           ; ERROR_ACCESS_DENIED - correct
```

Exercises

CLIENT-SIDE EXERCISES

1. **Handle removal properly.** Extend `WatchDevice` to close its handle on query-remove and reopen on arrival. Verify the device can then be disabled without a "device in use" dialog.
2. **Vary the queue depth** in the completion-port test from 1 to 32 and plot completions per second. Find where it stops improving, and explain why.
3. **Cancel deliberately.** Issue ten reads, then `CancelIoEx` them all. Confirm each completes with `ERROR_OPERATION_ABORTED`, and check the driver's stats for consistency.
4. **Kill the process** with reads outstanding. Confirm the process terminates promptly — this exercises the driver's cleanup path (chapter 04).
5. **Two processes at once.** Run two clients reading concurrently. Observe how the ring's single read position makes them compete — then decide whether that is the semantics you want, which is exercise 10 of chapter 23.
6. **Fuzz it yourself.** Send random control codes, random buffer sizes, and NULL pointers. With Driver Verifier on, anything that bugchecks is a driver bug.
7. **Add a 32-bit build** of the client and run it against the 64-bit driver. Confirm the structs still work — that is what the fixed-width types in `ringbuf.h` bought you (chapter 16).

Linux and Windows, side by side

Task	Linux (ch. 28)	Windows (ch. 23)
Find the device	Fixed path `/dev/ringbuf`	Enumerate by interface GUID
Open	`open()`	`CreateFileW`
Blocking read	`wait_event_interruptible`	Forward to a manual queue
Wake the reader	`wake_up_interruptible`	`WdfIoQueueRetrieveNextRequest` + complete
Readiness notification	`poll()` / `.poll`	Overlapped I/O + completion port
Control interface	`ioctl` with `_IOR` / `_IOW`	`DeviceIoControl` with `CTL_CODE`
Access control	udev rule on the node	SDDL in the INF + the IOCTL Access field
Per-open state	`file->private_data`	WDFFILEOBJECT context
Cancellation	Signal interrupts the wait	`CancelIoEx`, handled by the queue
Cleanup on exit	`release()`	`IRP_MJ_CLEANUP`, automatic in WDF
Install	`insmod`	INF + `pnputil` + signing
Verification	KASAN, lockdep, sparse	Driver Verifier, SDV, Code Analysis

The two drivers solve an identical problem, and the differences are almost entirely about *who is responsible for what*: Linux gives you primitives and expects the driver to compose them; Windows gives you a framework that owns the protocol and calls you at the right moments. Neither is better in the abstract — but knowing both makes the trade-offs in each visible, which is exactly what the next chapter is about.

Check yourself

- ✓ Open a device by interface GUID, using the two-call idiom correctly.
- ✓ Explain why Win32 error codes differ from NTSTATUS and how to translate.
- ✓ Issue an overlapped read and handle `ERROR_IO_PENDING` correctly.
- ✓ Build a completion-port loop that keeps N requests outstanding.
- ✓ Explain `CONTAINING_RECORD` and why `OVERLAPPED` comes first.
- ✓ Register for device arrival and removal, and respond to query-remove correctly.
- ✓ Measure synchronous versus asynchronous throughput and explain the gap.
- ✓ Verify the driver's access control from a non-administrator account.
- ✓ Complete at least four of the seven exercises.
- ✓ Fill in the Linux/Windows comparison table from memory.

Linux and Windows drivers compared

Two operating systems solving identical problems with opposite philosophies. Having built a driver in each, the comparison is now concrete — and it is the fastest way to see what each design actually costs and buys.

The root difference

```
ONE DECISION EXPLAINS ALMOST EVERYTHING ELSE:

LINUX: NO STABLE INTERNAL ABI.
   The kernel may refactor any interface at any time,
   because it can fix every in-tree driver in the same
   commit.

   -> drivers must be UPSTREAM to survive
   -> therefore the source is public
   -> therefore review is by humans reading code
   -> therefore no signing requirement, no certification
   -> therefore no framework is needed: the core can be
      changed to suit the drivers
   -> and an out-of-tree driver is a permanent maintenance
      liability (Linux ch. 30)

WINDOWS: A STABLE DRIVER ABI, FOR DECADES.
   A driver from 2015 often still loads.

   -> drivers ship as closed binaries
   -> therefore Microsoft never sees the source
   -> therefore review must be MECHANICAL: static
      analysis, Driver Verifier, HLK
   -> therefore signing is mandatory - identity is the only
      accountability available
   -> therefore a FRAMEWORK (WDF) is needed, because the
      protocol cannot be simplified by changing the core
   -> and installation needs a declarative package (the
      INF) rather than "copy the file"

NEITHER IS WRONG. They are consistent answers to: who is
responsible for keeping drivers working?
  Linux says: the kernel community, for code it can see.
  Windows says: the vendor, for code it cannot.
```

Concept map

Concept	Linux	Windows
Execution constraint	Context (process / softirq / hardirq / atomic)	IRQL, an explicit number
"Can I sleep?"	Am I in atomic context?	Is IRQL ≤ APC_LEVEL?
Deferred work tiers	hardirq → softirq/tasklet → workqueue	ISR → DPC → work item
Threaded interrupt	`request_threaded_irq`	`PassiveHandling`
Allocation	`kmalloc(size, GFP_*)`	`ExAllocatePool2(flags, size, tag)`
Pageable kernel memory	**None**	Paged pool — the extra rule
Object cache	`kmem_cache`	Lookaside list

Concept	Linux	Windows
Managed cleanup	`devm_*`	WDF object parenting
Per-device data	`drvdata`	Device context / DeviceExtension
Per-open data	`file->private_data`	WDFFILEOBJECT context
Request object	Per-subsystem (bio, urb, skb)	The IRP, universally
Layering	Per-subsystem	The device stack, universally
Hardware description	Device tree	ACPI + registry, written by the INF
Binding	`compatible` string	Hardware ID + INF rank
User interface	char dev, sysfs, or a subsystem	Device interface GUID + IOCTLs
Readiness	`poll()`	Overlapped I/O
Memory safety tool	KASAN	Special pool
Lock checking	lockdep	Verifier deadlock detection
Fault injection	`CONFIG_FAILSLAB`	Low resources simulation
Sleeping-in-atomic check	`DEBUG_ATOMIC_SLEEP`	Force IRQL checking
Static analysis	sparse, coccinelle, smatch	Code Analysis, SDV
Debugger	printk, ftrace, kgdb (rarely)	WinDbg (routinely)
Tracing	ftrace, trace events	WPP, ETW

Where each is genuinely easier

Task	Easier on	Why
Getting a first driver running	**Linux**	`insmod`, no signing, no INF, no reboot
The edit–build–test loop	**Linux**	Module reload in a second; QEMU for free
Reading how something works	**Linux**	The source of every driver in the tree is a worked example
Asking for help	**Linux**	Public archives of every design discussion
PnP, hot-plug, surprise removal	**Windows**	WDF implements the whole state machine
Request cancellation	**Windows**	Queues own it; the WDM-era race is gone
Power management	**Windows**	S0 idle policy is declarative
Layering and filtering	**Windows**	Universal, not per-subsystem
Finding your own bugs	**Windows**	Verifier + SDV + HLK are unmatched
Post-mortem debugging	**Windows**	WinDbg on a real dump beats reading a printk log
Keeping a driver working for ten years	**Windows** if out-of-tree, **Linux** if upstream	The ABI decision, from both directions
Shipping to a closed hardware ecosystem	**Windows**	The model was designed for exactly that
Shipping an embedded product you control	**Linux**	No signing gate, full source control

> **THE HONEST SUMMARY**
> Linux is easier to *start* with and easier to *learn from*, because everything is readable and the loop is fast. Windows is better at *keeping you correct*, because the framework owns the hard protocols and the verification tooling finds the bugs mechanically. A developer who has done both tends to want Linux's iteration speed and Windows' verifiers — and, in practice, to bring habits from each into the other: annotating invariants like SAL, and running the negative-test discipline that the HLK enforces.

What is the same everywhere

```
Strip away the APIs and the same problems remain, because
they come from the HARDWARE, not the OS:

  · an interrupt handler must be short, because it blocks
    everything else
  · deferred work needs a tier that can sleep and one that
    cannot
  · shared data needs a lock chosen by the highest-priority
    context that touches it
  · a lock held too long is a latency bug for the whole system
  · DMA needs the device's address, not the CPU's, and the
    pages must be pinned
  · caches must be maintained, or coherent hardware must do it
  · the power-up sequence in the datasheet is a specification
  · resources must be acquired in order and released in
    reverse
  · teardown must stop the source before waiting for what is
    in flight
  · every input crossing a trust boundary must be validated,
    copied in, and bounded
  · hardware that stops responding must produce a timeout,
    never an infinite wait
  · error paths are the least-tested code and need mechanical
    exercise

  THAT LIST is what this whole track has actually been about.
  The two dialects are how you say it; the list is what you
  say.
```

Porting between them

```
/* If you must support both - which is common for a chip
   vendor - structure the driver so the shared part is real. */

   +-----------------------------------------------+
   |  HARDWARE LOGIC - shared, portable C          |
   |    register definitions                       |
   |    the state machine                          |
   |    protocol encoding and decoding             |
   |    NO OS calls at all                         |
   +-----------------------------------------------+
              |                       |
   +----------v---------+   +---------v-----------+
   |  LINUX SHIM        |   |  WINDOWS SHIM       |
   |   readl/writel     |   |   READ_REGISTER_*   |
   |   spin_lock        |   |   WdfSpinLockAcquire|
   |   kmalloc          |   |   ExAllocatePool2   |
   |   dev_err          |   |   TraceEvents       |
   |   probe/remove     |   |   EvtDeviceAdd/...  |
   +--------------------+   +---------------------+

/* DEFINE A THIN OS ABSTRACTION and implement it twice: */

         osal_reg_read32(dev, offset)
         osal_reg_write32(dev, offset, value)
         osal_lock_acquire(lock)
         osal_alloc(size)
         osal_delay_us(n)
         osal_log(level, fmt, ...)

/* WHAT PORTS WELL:  register access, hardware sequences,
   protocol logic, data formats, error decoding.

   WHAT DOES NOT:    everything about the OS - PnP, power,
   queueing, buffering, cancellation, installation, security.
   Do NOT try to abstract those; the models are too different
   and the abstraction becomes worse than either.

/* THE REALISTIC SPLIT on a typical device driver: 30-40% of
   the code is hardware logic and shareable; the rest is OS
   integration, written twice. That is still a large saving,
   and more importantly it means the hardware behaviour is
   defined in ONE place, so a silicon erratum workaround is
   applied once rather than diverging.                    */
```

Which to learn

```
EMBEDDED LINUX dominates:
  · phones, tablets, TVs, set-top boxes
  · automotive infotainment and increasingly ADAS
  · networking equipment, from home routers to core
  · industrial control, robotics, drones
  · almost everything running on an ARM SoC that is not a
    microcontroller
  -> if "embedded" is your field, Linux is the default and
     this is where the jobs are

WINDOWS DRIVERS dominate:
  · PC peripherals of every kind
  · enterprise storage and networking hardware
  · specialised industrial and medical equipment on
    Windows
  · anything sold into a Windows-first ecosystem
  -> a smaller field, with fewer people who know it well,
     and correspondingly good demand

LEARNING BOTH is worth more than the sum: each makes the
other's design decisions visible. You cannot really see
why Linux has no stable ABI until you have felt what a
stable one costs, and you cannot appreciate what WDF does
until you have hand-written a cancellation path.

AND THE HARDWARE KNOWLEDGE - the ARM architecture,
isolation, memory ordering, DMA, caches, interrupts, power -
TRANSFERS COMPLETELY. That is the durable part, and it is
why the tracks before these two are the foundation rather
than the preamble.
```

A driver review checklist, either OS

Check	Linux	Windows
Every allocation checked	Fault injection	Low resources simulation
No sleeping in atomic context	`DEBUG_ATOMIC_SLEEP`	Force IRQL checking
No unbounded waits	`wait_event_*_timeout`	Timeouts on every wait
Locks consistent and ordered	lockdep	Verifier deadlock detection
All input validated	Review + fuzzing	IoAttack + Code Analysis
Teardown stops the source first	`_sync` everywhere	`TRUE` wait arguments
Removal handled	unbind/bind loop	HLK Surprise Remove
No resource leaks	kmemleak, 100 cycles	Pool tracking, 50 cycles
Memory errors	KASAN	Special pool
DMA correct	`DMA_API_DEBUG`	Verifier DMA checking
Power cycle survives	suspend/resume loop	`pwrtest /sleep`
Static analysis clean	sparse, `W=1`, checkpatch	Code Analysis, SDV

Every row is the same requirement expressed twice. A driver that passes the whole column on either system is a genuinely solid driver — and the tools that verify it are free on both.

Check yourself

- ✓ State the root difference between the two systems and derive four consequences from it.
- ✓ Translate fifteen concepts from one dialect to the other.
- ✓ Say which system is easier for six specific tasks, with the reason.
- ✓ List twelve problems that are identical on both, and explain why.
- ✓ Design a portable driver split and say what should not be abstracted.
- ✓ Estimate what fraction of a real driver is shareable.
- ✓ Say which system dominates in which markets, and what transfers between them.
- ✓ Run the twelve-row review checklist against a driver you have written.

END OF THE WINDOWS TRACK

Twenty-five chapters, from NT architecture to a signed, verified driver and its client. With the Linux track before it, you have now built the same device twice on two operating systems — which is the point at which driver development stops being a set of recipes and becomes a set of principles. Next: compilers and toolchains, which explains what has been turning all of this code into instructions.

Project: a TrustZone-backed crypto driver in C++

A Windows KMDF driver on ARM64 that hashes with CNG in the normal world, sends only the digest across a transport into the secure world, and returns a signature made by a key it has never seen. This chapter builds the Windows half; chapter 27 builds the trusted application and runs the whole thing on QEMU.

Read this before you start

Windows on ARM does not talk to OP-TEE. That is not a gap in your knowledge; it is the actual state of the world, and the project is designed around it rather than pretending otherwise.

Claim	Reality
"Windows uses OP-TEE for TrustZone"	**No.** Windows reaches the secure world through **TrEE** (Trusted Execution Environment), a class-extension driver model. The secure-side implementation is the silicon vendor's — QSEE/QTEE on Qualcomm parts, not OP-TEE
"A KMDF driver can issue an SMC"	**No.** There is no documented SMC API for ordinary Windows drivers. The world switch belongs to the TrEE provider, which is firmware-adjacent and vendor-supplied
"Windows and OP-TEE can run together under TF-A"	In principle. In practice no public TrEE provider speaks the OP-TEE SMC ABI, so nothing bridges them
"Windows ARM64 runs under QEMU"	**Yes**, via UEFI, slowly but usably. That part is real

SO WHAT ARE WE ACTUALLY BUILDING?

A driver whose *transport is a seam*. Above the seam — the IOCTL surface, the CNG hashing, the command ABI, the trusted application, the user-mode client — everything is real and production-shaped. Below it there are two implementations: **TrEE**, which is what you would ship on hardware, and a **shared-memory mailbox**, which is what actually runs on QEMU. The driver does not know or care which one it has.

```
WINDOWS ARM64 GUEST                          THE SECURE SIDE

+--------------------------------------+
| secteectl.exe   (user mode, C++)     |
|   CreateFile on the device interface |
|   DeviceIoControl                     |
+------------------+-------------------+
                   |  IOCTL_SECTEE_*
                   v
+--------------------------------------+
| sectee.sys   (KMDF, C++)             |
|                                      |
|  * SHA-256 with CNG (ksecdd)  <--- the "crypto driver"
|  * ECDSA verify with CNG           part: normal-world
|  * marshals a command              crypto stays here
|  * ITeeTransport::Invoke()         |
+------------------+-------------------+
                   |
     +-------------+--------------+
     |                            |
ON HARDWARE                   ON QEMU
TreeTransport                 MailboxTransport
TrEE class extension          ivshmem BAR2 + doorbell
     |                            |
     v                            v
TrustZone, S-EL1              a second VM running
vendor trusted OS             OP-TEE, bridged by a
                              host daemon (chapter 27)
                                    |
                                    v
                         +--------------------------+
                         | sectee_ta  (S-EL0, C++)  |
                         |   P-256 key, made and    |
                         |   kept inside            |
                         |   ECDSA sign             |
                         +--------------------------+

WHAT IS HONEST ABOUT THE QEMU PATH:
  the isolation you get is VM isolation between two QEMU
  instances, NOT TrustZone isolation. It exercises every
  line of software and proves the protocol; it does not
  prove the security property. Chapter 27 says so again,
  louder.
```

C++ in a Windows kernel driver

The WDK compiles C++ perfectly well. What it does not give you is a C++ *runtime*: there is no CRT startup to run your global constructors, no exception unwinder, and no heap behind `new`. You get the language, and you supply the plumbing.

```
// SecTee/Kernel/CppSupport.h
//
// Everything C++ needs from us before the language works in
// kernel mode. About forty lines, written once, reused for ever.

#pragma once
#include <ntddk.h>

#define SECTEE_POOL_TAG 'eeTS'   // "STee" reversed, as WinDbg
                                 // displays tags little-endian

// ---------------------------------------------------------------
// operator new / delete
//
// We deliberately do NOT define a global parameterless `new`.
// Every allocation must name its pool, because getting NonPaged
// vs Paged wrong is a bugcheck at DISPATCH_LEVEL, and a default
// makes it easy to not think about it.
// ---------------------------------------------------------------
_IRQL_requires_max_(DISPATCH_LEVEL)
inline void* __cdecl operator new(size_t size, POOL_FLAGS flags,
                                  ULONG tag)
{
    if (size == 0) size = 1;
    // ExAllocatePool2 zeroes by default and is the modern spelling;
    // ExAllocatePoolWithTag is deprecated and fails WDK analysis.
    return ExAllocatePool2(flags, size, tag);
}

inline void __cdecl operator delete(void* p, size_t)
{
    if (p) ExFreePoolWithTag(p, SECTEE_POOL_TAG);
}
inline void __cdecl operator delete(void* p)
{
    if (p) ExFreePoolWithTag(p, SECTEE_POOL_TAG);
}

// Placement new, for constructing into memory WDF already owns
// (a WDFDEVICE context, for instance).
inline void* __cdecl operator new(size_t, void* where) { return where; }
inline void  __cdecl operator delete(void*, void*) {}

// ---------------------------------------------------------------
// The compiler emits a reference to this for any file-scope object
// with a destructor. We never rely on static destruction, but the
// symbol must exist to link.
// ---------------------------------------------------------------
extern "C" int __cdecl atexit(void (__cdecl*)(void)) { return 0; }
extern "C" void* __dso_handle = nullptr;

// ---------------------------------------------------------------
// A scope guard. This is most of the reason to use C++ here: it
// replaces the goto-cleanup ladder that every C driver grows.
// ---------------------------------------------------------------
template <typename F>
class ScopeExit
{
public:
    explicit ScopeExit(F f) : m_f(f), m_armed(true) {}
    ~ScopeExit() { if (m_armed) m_f(); }
    void Dismiss() { m_armed = false; }
```

```
    ScopeExit(const ScopeExit&)            = delete;
    ScopeExit& operator=(const ScopeExit&) = delete;
private:
    F    m_f;
    bool m_armed;
};

template <typename F>
ScopeExit<F> MakeScopeExit(F f) { return ScopeExit<F>(f); }

#define SECTEE_CONCAT2(a, b) a##b
#define SECTEE_CONCAT(a, b)  SECTEE_CONCAT2(a, b)
#define SECTEE_ON_EXIT(code) \
    auto SECTEE_CONCAT(guard_, __LINE__) = MakeScopeExit([&]() { code; })
```

THE FOUR PROJECT SETTINGS THAT MAKE THIS LEGAL

In the `.vcxproj`: `<CompileAs>CompileAsCpp</CompileAs>`, `<RuntimeTypeInfo>false</RuntimeTypeInfo>` (`/GR-`), `<ExceptionHandling>false</ExceptionHandling>` (`/EHs-c-`), and the `/kernel` switch, which makes the compiler *reject* constructs that need a runtime rather than letting them link and fail at load. Turn `/kernel` on first, not last — it tells you immediately which C++ you are not allowed to have.

```
   WHAT YOU MAY USE          WHAT YOU MAY NOT

   classes, members          exceptions (try/catch/throw)
   constructors/destructors  RTTI (dynamic_cast, typeid)
   RAII and scope guards     the STL - all of it
   templates                 std::string, std::vector, ...
   references                global objects with constructors
   overloading               new/delete without a pool
   constexpr, enum class     virtual inheritance games
   lambdas (no captures      thread_local
     that allocate)          floating point (without
   nullptr, auto               KeSaveFloatingPointState)

   The STL exclusion is not snobbery: every container allocates
   from a heap that does not exist here, and throws on failure
   in a context with no unwinder. Write the two or three small
   containers you need, or use WDF's collections.
```

The ABI

```
// SecTee/Common/SecTeeAbi.h
//
// Shared verbatim by the driver, the user-mode client and (via a
// C-compatible subset) the trusted application. One file, three
// consumers, no numbers written twice.

#pragma once

#ifdef _KERNEL_MODE
#include <ntddk.h>
#else
#include <windows.h>
#endif

#include <initguid.h>

// The device interface the client opens. A GUID, not a fixed
// symbolic-link name, so several instances can coexist and
// SetupDi can enumerate them (chapter 19).
// {6F1A2B3C-4D5E-4F60-9A1B-2C3D4E5F6071}
DEFINE_GUID(GUID_DEVINTERFACE_SECTEE,
    0x6f1a2b3c, 0x4d5e, 0x4f60, 0x9a, 0x1b,
    0x2c, 0x3d, 0x4e, 0x5f, 0x60, 0x71);

#define SECTEE_ABI_VERSION 1u

#define SECTEE_DEVICE_TYPE 0x8000u   // vendor range starts here

#define IOCTL_SECTEE_GET_PUBKEY  CTL_CODE(SECTEE_DEVICE_TYPE, 0x800, \
                                 METHOD_BUFFERED, FILE_READ_DATA)
#define IOCTL_SECTEE_SIGN        CTL_CODE(SECTEE_DEVICE_TYPE, 0x801, \
                                 METHOD_BUFFERED, \
                                 FILE_READ_DATA | FILE_WRITE_DATA)
#define IOCTL_SECTEE_VERIFY      CTL_CODE(SECTEE_DEVICE_TYPE, 0x802, \
                                 METHOD_BUFFERED, FILE_READ_DATA)
#define IOCTL_SECTEE_COUNTER     CTL_CODE(SECTEE_DEVICE_TYPE, 0x803, \
                                 METHOD_BUFFERED, \
                                 FILE_READ_DATA | FILE_WRITE_DATA)

// METHOD_BUFFERED: the I/O manager copies in and out for us, into
// a single system buffer. For payloads of a few hundred bytes it
// is the right choice - no MDLs, no probe-and-lock, no user
// pointer ever dereferenced in the driver. METHOD_OUT_DIRECT would
// only start paying off in the tens of kilobytes.

#define SECTEE_PUBKEY_LEN 64u   // P-256, X || Y, fixed width
#define SECTEE_SIG_LEN    64u   // P-256, r || s, fixed width
#define SECTEE_DIGEST_LEN 32u   // SHA-256
#define SECTEE_MAX_INPUT  (64u * 1024u)

#include <pshpack8.h>

typedef struct _SECTEE_HEADER {
    ULONG Version;     // caller sets SECTEE_ABI_VERSION
    ULONG Reserved;    // must be zero; gives us room and keeps
                       // the following members 8-byte aligned
} SECTEE_HEADER;

typedef struct _SECTEE_PUBKEY_OUT {
    SECTEE_HEADER Header;
```

```
    UCHAR         PublicKey[SECTEE_PUBKEY_LEN];
} SECTEE_PUBKEY_OUT;

// The driver hashes, so the client sends DATA, not a digest.
// That is the point of putting crypto in the driver.
typedef struct _SECTEE_SIGN_IN {
    SECTEE_HEADER Header;
    ULONG         DataLength;
    ULONG         Reserved2;
    UCHAR         Data[ANYSIZE_ARRAY];
} SECTEE_SIGN_IN;

typedef struct _SECTEE_SIGN_OUT {
    SECTEE_HEADER Header;
    UCHAR         Digest[SECTEE_DIGEST_LEN];   // what we signed
    UCHAR         Signature[SECTEE_SIG_LEN];   // raw r || s
} SECTEE_SIGN_OUT;

typedef struct _SECTEE_VERIFY_IN {
    SECTEE_HEADER Header;
    UCHAR         Digest[SECTEE_DIGEST_LEN];
    UCHAR         Signature[SECTEE_SIG_LEN];
} SECTEE_VERIFY_IN;

typedef struct _SECTEE_COUNTER_INOUT {
    SECTEE_HEADER Header;
    ULONG         Increment;   // 1 = bump before reading
    ULONG         Reserved2;
    ULONGLONG     Value;       // out
} SECTEE_COUNTER_INOUT;

#include <poppack.h>

// Commands on the far side of the transport. Deliberately NOT the
// same numbers as the IOCTLs: the driver is a translator, and the
// two interfaces are allowed to version independently.
#define TA_CMD_GET_PUBKEY 0u
#define TA_CMD_SIGN       1u
#define TA_CMD_COUNTER    2u
```

WHY THE DRIVER HASHES AND THE TA ONLY SIGNS

The same split as the Rust project, for the same reason: bulk data is not secret, and pushing 64 KB through a transport into a secure world with a few hundred kilobytes of heap is slow and pointless. Only the 32-byte digest crosses. The private key is the one thing that must not leak, so it is the one thing that lives on the far side.

The transport seam

```
// SecTee/Kernel/ITeeTransport.h
#pragma once
#include "CppSupport.h"
#include "..\Common\SecTeeAbi.h"

// A pure interface. This is where C++ earns its place a second
// time: two wildly different mechanisms behind one vtable, chosen
// at PrepareHardware, with no #ifdefs sprayed through the driver.
class ITeeTransport
{
public:
    virtual ~ITeeTransport() {}

    // Called at PASSIVE_LEVEL from EvtDevicePrepareHardware.
    virtual NTSTATUS Start(WDFDEVICE Device,
                           WDFCMRESLIST Raw,
                           WDFCMRESLIST Translated) = 0;

    virtual void Stop() = 0;

    // One request into the secure world and back.
    //   Command  - TA_CMD_*
    //   In/InLen - marshalled input, may be null
    //   Out      - buffer for the reply
    //   InOutLen - in: capacity of Out; out: bytes produced
    //
    // PASSIVE_LEVEL only. Every implementation may block.
    _IRQL_requires_(PASSIVE_LEVEL)
    virtual NTSTATUS Invoke(ULONG Command,
                            const void* In,  ULONG InLen,
                            void* Out, ULONG* InOutLen) = 0;
};

// Factories. Exactly one is compiled in, selected by the INF's
// hardware ID match - see the two INF sections at the end.
NTSTATUS CreateMailboxTransport(ITeeTransport** Out);   // QEMU
NTSTATUS CreateTreeTransport(ITeeTransport** Out);      // hardware
```

The production transport: TrEE

```
// SecTee/Kernel/TreeTransport.cpp
//
// The shape of the production path. Windows exposes the secure
// world to drivers through the Trusted Execution Environment class
// extension: your driver is a "TrEE service driver", it links
// against the TrEE ClassExtension, and it addresses a trusted
// application by GUID.
//
// HONEST CAVEAT: the exact class-extension entry points and the
// structures they take differ between WDK releases, and the
// interface is thinly documented compared with the rest of WDF.
// The sequence below is correct in SHAPE - open a session to a TA
// by GUID, marshal parameters, send, read the reply, close - and
// you should take the precise spellings from the
// TrustedRuntime headers in the WDK you are building against.
//
// Everything above this file is unaffected either way, which is
// the whole argument for the ITeeTransport seam.

#include "ITeeTransport.h"

class TreeTransport : public ITeeTransport
{
public:
    NTSTATUS Start(WDFDEVICE Device, WDFCMRESLIST, WDFCMRESLIST) override
    {
        m_Device = Device;

        // 1. Attach to the TrEE class extension for this device.
        //    The platform's TrEE provider - vendor firmware glue -
        //    is what actually performs the world switch.
        //
        // 2. Open a session to our trusted application, addressed
        //    by its GUID. On Windows a TA is identified by a GUID
        //    rather than the 16-byte UUID OP-TEE uses; they are
        //    the same 128 bits in a different textual convention.
        //
        // 3. Cache the session handle for the driver's lifetime,
        //    exactly as the mailbox transport caches its mapping.

        return STATUS_NOT_IMPLEMENTED;   // see caveat above
    }

    void Stop() override
    {
        // Close the session, then detach from the class extension.
        // Reverse order of Start, always.
    }

    NTSTATUS Invoke(ULONG Command,
                    const void* In, ULONG InLen,
                    void* Out, ULONG* InOutLen) override
    {
        UNREFERENCED_PARAMETER(Command);
        UNREFERENCED_PARAMETER(In);
        UNREFERENCED_PARAMETER(InLen);
        UNREFERENCED_PARAMETER(Out);
        UNREFERENCED_PARAMETER(InOutLen);

        // Marshal into the TrEE parameter list - the same
        // four-slot value/memref model GlobalPlatform uses
```

```
        // (chapter 12), because both descend from the same
        // TEE Client API specification.
        //
        // Then issue the command and map the returned status
        // onto an NTSTATUS with the table below.
        return STATUS_NOT_IMPLEMENTED;
    }

private:
    WDFDEVICE m_Device = nullptr;
};

NTSTATUS CreateTreeTransport(ITeeTransport** Out)
{
    auto* t = new (POOL_FLAG_NON_PAGED, SECTEE_POOL_TAG) TreeTransport();
    if (!t) return STATUS_INSUFFICIENT_RESOURCES;
    *Out = t;
    return STATUS_SUCCESS;
}
```

The transport that runs: a shared-memory mailbox

```
// SecTee/Kernel/MailboxTransport.cpp
//
// Runs on QEMU against an ivshmem device, which gives us a PCI
// function with:
//   BAR0  256 bytes of registers, including a doorbell
//   BAR2  a shared memory window, visible to another VM
//
// The far end is a daemon in a second VM that owns the same
// memory and calls into OP-TEE. Chapter 27 builds it.
//
// No QEMU patching is required: ivshmem is a stock device.

#include "ITeeTransport.h"

// ---- the ivshmem register block, BAR0 -------------------------
#define IVSHMEM_REG_INTRMASK   0x00
#define IVSHMEM_REG_INTRSTATUS 0x04
#define IVSHMEM_REG_IVPOSITION 0x08   // our peer id, read-only
#define IVSHMEM_REG_DOORBELL   0x0c   // (peer << 16) | vector

// ---- our protocol, laid out in BAR2 ---------------------------
#include <pshpack8.h>
struct MailboxHeader
{
    ULONG    Magic;        // 'TEEM'
    ULONG    Version;
    volatile LONG Sequence;   // bumped by us on submit
    volatile LONG Ack;        // bumped by the peer on completion
    ULONG    Command;
    ULONG    InLength;
    ULONG    OutLength;
    LONG     Result;       // the peer's TEE_Result, 0 = success
};
#include <poppack.h>

#define MAILBOX_MAGIC     0x4D454554u   // "TEEM" little-endian
#define MAILBOX_VERSION   1u
#define MAILBOX_DATA_OFF  0x100u
#define MAILBOX_DATA_MAX  0x1000u
#define MAILBOX_TIMEOUT_MS 5000

class MailboxTransport : public ITeeTransport
{
public:
    NTSTATUS Start(WDFDEVICE Device,
                   WDFCMRESLIST /*Raw*/,
                   WDFCMRESLIST Translated) override
    {
        PAGED_CODE();
        m_Device = Device;

        // Walk the translated resource list. On ARM64 the PCI
        // driver has already assigned these; we take the first
        // two memory descriptors in BAR order.
        const ULONG count = WdfCmResourceListGetCount(Translated);
        ULONG memIndex = 0;

        for (ULONG i = 0; i < count; i++)
        {
            PCM_PARTIAL_RESOURCE_DESCRIPTOR d =
                WdfCmResourceListGetDescriptor(Translated, i);
```

```
        if (!d) continue;

        if (d->Type == CmResourceTypeMemory ||
            d->Type == CmResourceTypeMemoryLarge)
        {
            // MmNonCached: this is device memory shared with
            // another virtual machine. Caching it would let
            // us read a stale Ack for ever - the exact
            // attribute mistake chapter 7 of the
            // virtualisation track describes.
            PVOID va = MmMapIoSpaceEx(d->u.Memory.Start,
                                      d->u.Memory.Length,
                                      PAGE_READWRITE | PAGE_NOCACHE);
            if (!va) return STATUS_INSUFFICIENT_RESOURCES;

            if (memIndex == 0) {
                m_Regs    = static_cast<PUCHAR>(va);
                m_RegsLen = d->u.Memory.Length;
            } else if (memIndex == 1) {
                m_Shm     = static_cast<PUCHAR>(va);
                m_ShmLen  = d->u.Memory.Length;
            } else {
                MmUnmapIoSpace(va, d->u.Memory.Length);
            }
            memIndex++;
        }
    }

    if (!m_Regs || !m_Shm || m_ShmLen < MAILBOX_DATA_OFF + MAILBOX_DATA_MAX)
        return STATUS_DEVICE_CONFIGURATION_ERROR;

    // Our peer id, so the far end knows who to answer.
    m_PeerId = READ_REGISTER_ULONG(
        reinterpret_cast<PULONG>(m_Regs + IVSHMEM_REG_IVPOSITION));

    auto* hdr = Header();

    // The daemon writes the magic when it is ready. If it is
    // absent we fail PrepareHardware rather than pretending to
    // work - a driver that starts with no far end produces
    // timeouts nobody can diagnose.
    if (hdr->Magic != MAILBOX_MAGIC)
    {
        TraceError("mailbox magic 0x%08x, expected 0x%08x - "
                   "is secteed running?", hdr->Magic, MAILBOX_MAGIC);
        return STATUS_DEVICE_NOT_READY;
    }
    if (hdr->Version != MAILBOX_VERSION)
        return STATUS_REVISION_MISMATCH;

    m_LastAck = hdr->Ack;
    return STATUS_SUCCESS;
}

void Stop() override
{
    if (m_Shm)  { MmUnmapIoSpace(m_Shm,  m_ShmLen);  m_Shm  = nullptr; }
    if (m_Regs) { MmUnmapIoSpace(m_Regs, m_RegsLen); m_Regs = nullptr; }
}

_IRQL_requires_(PASSIVE_LEVEL)
NTSTATUS Invoke(ULONG Command,
                const void* In, ULONG InLen,
```

```
                void* Out, ULONG* InOutLen) override
{
    PAGED_CODE();

    if (InLen > MAILBOX_DATA_MAX) return STATUS_INVALID_BUFFER_SIZE;
    if (!InOutLen)                return STATUS_INVALID_PARAMETER;

    // One request at a time. The mailbox is a single slot, so
    // this is not merely a nicety - two concurrent submits
    // would interleave in shared memory.
    NTSTATUS status = KeWaitForSingleObject(&m_Lock, Executive,
                                            KernelMode, FALSE, nullptr);
    if (!NT_SUCCESS(status)) return status;
    SECTEE_ON_EXIT(KeReleaseMutex(&m_Lock, FALSE));

    auto* hdr  = Header();
    auto* data = m_Shm + MAILBOX_DATA_OFF;

    if (In && InLen)
        RtlCopyMemory(data, In, InLen);

    hdr->Command   = Command;
    hdr->InLength  = InLen;
    hdr->OutLength = 0;
    hdr->Result    = 0;

    // Publish the payload BEFORE the sequence number that
    // announces it. Same rule as a virtqueue: the reader must
    // never see the index move ahead of the data.
    KeMemoryBarrier();
    const LONG seq = InterlockedIncrement(&hdr->Sequence);
    KeMemoryBarrier();

    Ring();

    status = WaitForAck(seq);
    if (!NT_SUCCESS(status)) return status;

    KeMemoryBarrier();   // Ack observed before we read the reply

    if (hdr->Result != 0)
    {
        TraceError("TA command %u returned 0x%08x",
                   Command, hdr->Result);
        return TeeResultToNtStatus(static_cast<ULONG>(hdr->Result));
    }

    const ULONG produced = hdr->OutLength;
    if (produced > MAILBOX_DATA_MAX)   return STATUS_DATA_ERROR;
    if (produced > *InOutLen)          { *InOutLen = produced;
                                         return STATUS_BUFFER_TOO_SMALL; }

    if (Out && produced)
        RtlCopyMemory(Out, data, produced);
    *InOutLen = produced;
    return STATUS_SUCCESS;
}

MailboxTransport()
{
    KeInitializeMutex(&m_Lock, 0);
}
```

```
private:
    MailboxHeader* Header() const
    {
        return reinterpret_cast<MailboxHeader*>(m_Shm);
    }

    void Ring() const
    {
        // Peer 0 is the daemon by convention; vector 0.
        WRITE_REGISTER_ULONG(
            reinterpret_cast<PULONG>(m_Regs + IVSHMEM_REG_DOORBELL),
            (0u << 16) | 0u);
    }

    // Poll, because this transport has no interrupt wired up and
    // the whole point is to keep it simple. A real driver would
    // take the ivshmem MSI-X vector and complete the request
    // asynchronously from the ISR's DPC - see the exercise at the
    // end of chapter 27.
    NTSTATUS WaitForAck(LONG expect)
    {
        LARGE_INTEGER interval;
        interval.QuadPart = -1 * 10 * 1000;   // 1 ms, relative

        for (ULONG waited = 0; waited < MAILBOX_TIMEOUT_MS; waited++)
        {
            if (Header()->Ack == expect) return STATUS_SUCCESS;
            KeDelayExecutionThread(KernelMode, FALSE, &interval);
        }
        TraceError("mailbox timeout waiting for ack %d", expect);
        return STATUS_IO_TIMEOUT;
    }

    // GlobalPlatform TEE_Result -> NTSTATUS. Doing this in one
    // place is the difference between a driver you can debug from
    // user mode and one that returns STATUS_UNSUCCESSFUL for
    // everything.
    static NTSTATUS TeeResultToNtStatus(ULONG r)
    {
        switch (r)
        {
        case 0xFFFF0004: return STATUS_NOT_SUPPORTED;        // NOT_SUPPORTED
        case 0xFFFF0006: return STATUS_INVALID_PARAMETER;    // BAD_PARAMETERS
        case 0xFFFF0008: return STATUS_OBJECT_NAME_NOT_FOUND;// ITEM_NOT_FOUND
        case 0xFFFF000C: return STATUS_BUFFER_TOO_SMALL;     // SHORT_BUFFER
        case 0xFFFF000F: return STATUS_ACCESS_DENIED;        // ACCESS_DENIED
        case 0xFFFF3024: return STATUS_INSUFFICIENT_RESOURCES;
        case 0xFFFF3110: return STATUS_DEVICE_NOT_CONNECTED; // TARGET_DEAD
        default:         return STATUS_UNSUCCESSFUL;
        }
    }

    WDFDEVICE m_Device  = nullptr;
    PUCHAR    m_Regs    = nullptr;
    PUCHAR    m_Shm     = nullptr;
    SIZE_T    m_RegsLen = 0;
    SIZE_T    m_ShmLen  = 0;
    ULONG     m_PeerId  = 0;
    LONG      m_LastAck = 0;
    KMUTEX    m_Lock;
};

NTSTATUS CreateMailboxTransport(ITeeTransport** Out)
```

```
{
    auto* t = new (POOL_FLAG_NON_PAGED, SECTEE_POOL_TAG) MailboxTransport();
    if (!t) return STATUS_INSUFFICIENT_RESOURCES;
    *Out = t;
    return STATUS_SUCCESS;
}
```

The crypto half: CNG in kernel mode

This is the part that makes it a *crypto driver* rather than a pipe. Windows exposes CNG to kernel drivers through `ksecdd.sys`; the API is the same `BCrypt*` surface user mode gets, with two extra rules.

```
// SecTee/Kernel/Crypto.cpp
//
// RULE 1: pass BCRYPT_PROV_DISPATCH when opening an algorithm
//         provider in kernel mode. Without it the call fails.
// RULE 2: PASSIVE_LEVEL only. CNG allocates and may page.
//
// Link against ksecdd.lib.

#include "CppSupport.h"
#include <bcrypt.h>
#include "..\Common\SecTeeAbi.h"

// A tiny RAII wrapper per handle type. Four small classes remove
// every goto in this file.
class BcryptAlg
{
public:
    ~BcryptAlg() { if (m_h) BCryptCloseAlgorithmProvider(m_h, 0); }
    NTSTATUS Open(LPCWSTR id)
    {
        return BCryptOpenAlgorithmProvider(&m_h, id, nullptr,
                                           BCRYPT_PROV_DISPATCH);
    }
    operator BCRYPT_ALG_HANDLE() const { return m_h; }
private:
    BCRYPT_ALG_HANDLE m_h = nullptr;
};

class BcryptHash
{
public:
    ~BcryptHash() { if (m_h) BCryptDestroyHash(m_h); }
    NTSTATUS Create(BCRYPT_ALG_HANDLE alg)
    {
        return BCryptCreateHash(alg, &m_h, nullptr, 0,
                                nullptr, 0, 0);
    }
    operator BCRYPT_HASH_HANDLE() const { return m_h; }
private:
    BCRYPT_HASH_HANDLE m_h = nullptr;
};

class BcryptKey
{
public:
    ~BcryptKey() { if (m_h) BCryptDestroyKey(m_h); }
    BCRYPT_KEY_HANDLE* Put() { return &m_h; }
    operator BCRYPT_KEY_HANDLE() const { return m_h; }
private:
    BCRYPT_KEY_HANDLE m_h = nullptr;
};

_IRQL_requires_(PASSIVE_LEVEL)
NTSTATUS SecTeeSha256(const void* data, ULONG length,
                      UCHAR digest[SECTEE_DIGEST_LEN])
{
    PAGED_CODE();

    BcryptAlg alg;
    NTSTATUS status = alg.Open(BCRYPT_SHA256_ALGORITHM);
    if (!NT_SUCCESS(status)) return status;

    BcryptHash hash;
```

```
    status = hash.Create(alg);
    if (!NT_SUCCESS(status)) return status;

    // BCryptHashData takes a non-const pointer. It does not write
    // through it; the const_cast is the API's fault, not ours.
    status = BCryptHashData(hash,
                            const_cast<PUCHAR>(
                                static_cast<const UCHAR*>(data)),
                            length, 0);
    if (!NT_SUCCESS(status)) return status;

    return BCryptFinishHash(hash, digest, SECTEE_DIGEST_LEN, 0);
    // Both handles close here, in reverse construction order,
    // on every path including the early returns above.
}

// Verify a signature the SECURE WORLD produced, using the public
// key it gave us. Nothing here is secret - this is a check the
// driver performs on its own output, and it is how you catch a
// transport that is corrupting bytes.
_IRQL_requires_(PASSIVE_LEVEL)
NTSTATUS SecTeeVerifyP256(const UCHAR publicKey[SECTEE_PUBKEY_LEN],
                          const UCHAR digest[SECTEE_DIGEST_LEN],
                          const UCHAR signature[SECTEE_SIG_LEN])
{
    PAGED_CODE();

    BcryptAlg alg;
    NTSTATUS status = alg.Open(BCRYPT_ECDSA_P256_ALGORITHM);
    if (!NT_SUCCESS(status)) return status;

    // CNG's public-key blob: a BCRYPT_ECCKEY_BLOB header followed
    // by X then Y, each cbKey bytes, big-endian, fixed width.
    // Note there is NO 0x04 uncompressed-point prefix here -
    // that is a SEC1/OpenSSL convention. CNG does not want it.
    struct {
        BCRYPT_ECCKEY_BLOB Header;
        UCHAR              XY[SECTEE_PUBKEY_LEN];
    } blob;

    blob.Header.dwMagic = BCRYPT_ECDSA_PUBLIC_P256_MAGIC;
    blob.Header.cbKey   = 32;
    RtlCopyMemory(blob.XY, publicKey, SECTEE_PUBKEY_LEN);

    BcryptKey key;
    status = BCryptImportKeyPair(alg, nullptr, BCRYPT_ECCPUBLIC_BLOB,
                                 key.Put(),
                                 reinterpret_cast<PUCHAR>(&blob),
                                 sizeof(blob), 0);
    if (!NT_SUCCESS(status)) return status;

    // And CNG wants the signature as raw r || s, fixed width -
    // NOT the DER SEQUENCE that OpenSSL insists on. Our TA emits
    // exactly this, so it goes straight in.
    return BCryptVerifySignature(key, nullptr,
                                 const_cast<PUCHAR>(digest),
                                 SECTEE_DIGEST_LEN,
                                 const_cast<PUCHAR>(signature),
                                 SECTEE_SIG_LEN, 0);
    // STATUS_INVALID_SIGNATURE on mismatch, STATUS_SUCCESS on match.
}
```

TWO ECOSYSTEMS, TWO SIGNATURE ENCODINGS

The TA produces raw `r || s`, 64 fixed bytes. **CNG takes that directly. OpenSSL does not** — it wants an ASN.1 `SEQUENCE` of two `INTEGER`s, which is why the Rust project needed a conversion step and this one does not. Neither is more correct; they are different conventions with the same mathematics underneath, and knowing which side wants which saves an afternoon every single time.

The driver itself

```
// SecTee/Kernel/Driver.cpp
#include "CppSupport.h"
#include <wdf.h>
#include "ITeeTransport.h"

extern "C" {
DRIVER_INITIALIZE                DriverEntry;
EVT_WDF_DRIVER_DEVICE_ADD        SecTeeEvtDeviceAdd;
EVT_WDF_DEVICE_PREPARE_HARDWARE  SecTeeEvtPrepareHardware;
EVT_WDF_DEVICE_RELEASE_HARDWARE  SecTeeEvtReleaseHardware;
EVT_WDF_IO_QUEUE_IO_DEVICE_CONTROL SecTeeEvtIoDeviceControl;
}

NTSTATUS SecTeeSha256(const void*, ULONG, UCHAR[SECTEE_DIGEST_LEN]);
NTSTATUS SecTeeVerifyP256(const UCHAR[SECTEE_PUBKEY_LEN],
                          const UCHAR[SECTEE_DIGEST_LEN],
                          const UCHAR[SECTEE_SIG_LEN]);

// Per-device context. WDF allocates it; we placement-new the C++
// members into it, because WDF does not know about constructors.
struct DEVICE_CONTEXT
{
    ITeeTransport* Transport;
};
WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(DEVICE_CONTEXT, DeviceGetContext)

extern "C"
NTSTATUS DriverEntry(_In_ PDRIVER_OBJECT DriverObject,
                     _In_ PUNICODE_STRING RegistryPath)
{
    WDF_DRIVER_CONFIG config;
    WDF_DRIVER_CONFIG_INIT(&config, SecTeeEvtDeviceAdd);

    return WdfDriverCreate(DriverObject, RegistryPath,
                           WDF_NO_OBJECT_ATTRIBUTES,
                           &config, WDF_NO_HANDLE);
}

NTSTATUS SecTeeEvtDeviceAdd(_In_ WDFDRIVER, _Inout_ PWDFDEVICE_INIT DeviceInit)
{
    PAGED_CODE();

    // We own hardware, so we need the PnP callbacks.
    WDF_PNPPOWER_EVENT_CALLBACKS pnp;
    WDF_PNPPOWER_EVENT_CALLBACKS_INIT(&pnp);
    pnp.EvtDevicePrepareHardware = SecTeeEvtPrepareHardware;
    pnp.EvtDeviceReleaseHardware = SecTeeEvtReleaseHardware;
    WdfDeviceInitSetPnpPowerEventCallbacks(DeviceInit, &pnp);

    // Exclusive: one client at a time. The mailbox is a single
    // slot and the TA is single-instance; letting two processes
    // in would only move the queueing somewhere less obvious.
    WdfDeviceInitSetExclusive(DeviceInit, TRUE);

    WDF_OBJECT_ATTRIBUTES attrs;
    WDF_OBJECT_ATTRIBUTES_INIT_CONTEXT_TYPE(&attrs, DEVICE_CONTEXT);

    WDFDEVICE device;
    NTSTATUS status = WdfDeviceCreate(&DeviceInit, &attrs, &device);
    if (!NT_SUCCESS(status)) return status;
```

```
    RtlZeroMemory(DeviceGetContext(device), sizeof(DEVICE_CONTEXT));

    // The interface the user-mode client looks for.
    status = WdfDeviceCreateDeviceInterface(device,
                                            &GUID_DEVINTERFACE_SECTEE,
                                            nullptr);
    if (!NT_SUCCESS(status)) return status;

    // A sequential queue: WDF delivers one request at a time, so
    // the IOCTL handler never re-enters. Combined with the
    // transport's own mutex this is belt and braces, and it costs
    // nothing.
    WDF_IO_QUEUE_CONFIG queueConfig;
    WDF_IO_QUEUE_CONFIG_INIT_DEFAULT_QUEUE(&queueConfig,
                                           WdfIoQueueDispatchSequential);
    queueConfig.EvtIoDeviceControl = SecTeeEvtIoDeviceControl;

    // The handler calls CNG and blocks on the mailbox, so it must
    // run at PASSIVE_LEVEL. This is the line that guarantees it.
    WDF_OBJECT_ATTRIBUTES queueAttrs;
    WDF_OBJECT_ATTRIBUTES_INIT(&queueAttrs);
    queueAttrs.ExecutionLevel = WdfExecutionLevelPassive;

    WDFQUEUE queue;
    return WdfIoQueueCreate(device, &queueConfig, &queueAttrs, &queue);
}

NTSTATUS SecTeeEvtPrepareHardware(_In_ WDFDEVICE Device,
                                  _In_ WDFCMRESLIST Raw,
                                  _In_ WDFCMRESLIST Translated)
{
    PAGED_CODE();
    auto* ctx = DeviceGetContext(Device);

    NTSTATUS status = CreateMailboxTransport(&ctx->Transport);
    if (!NT_SUCCESS(status)) return status;

    // If Start fails we must not leak the object - and this is
    // exactly the case a C driver gets wrong on the third edit.
    auto guard = MakeScopeExit([&]() {
        delete ctx->Transport;
        ctx->Transport = nullptr;
    });

    status = ctx->Transport->Start(Device, Raw, Translated);
    if (!NT_SUCCESS(status)) return status;

    guard.Dismiss();
    return STATUS_SUCCESS;
}

NTSTATUS SecTeeEvtReleaseHardware(_In_ WDFDEVICE Device, _In_ WDFCMRESLIST)
{
    PAGED_CODE();
    auto* ctx = DeviceGetContext(Device);
    if (ctx->Transport)
    {
        ctx->Transport->Stop();
        delete ctx->Transport;      // virtual destructor runs
        ctx->Transport = nullptr;
    }
    return STATUS_SUCCESS;
```

```
}

// ------------------------------------------------------------
// The IOCTL handler
// ------------------------------------------------------------
void SecTeeEvtIoDeviceControl(_In_ WDFQUEUE Queue,
                              _In_ WDFREQUEST Request,
                              _In_ size_t OutputBufferLength,
                              _In_ size_t InputBufferLength,
                              _In_ ULONG IoControlCode)
{
    PAGED_CODE();

    WDFDEVICE device = WdfIoQueueGetDevice(Queue);
    auto*     ctx    = DeviceGetContext(device);
    NTSTATUS  status = STATUS_INVALID_DEVICE_REQUEST;
    size_t    written = 0;

    // METHOD_BUFFERED, so both of these are the same system
    // buffer and neither is a user pointer. WdfRequestRetrieve*
    // also validates the length for us.
    PVOID  inBuf  = nullptr;
    PVOID  outBuf = nullptr;

    if (InputBufferLength)
    {
        status = WdfRequestRetrieveInputBuffer(Request,
                    sizeof(SECTEE_HEADER), &inBuf, nullptr);
        if (!NT_SUCCESS(status)) goto complete;

        // Version check before anything else, always.
        if (static_cast<SECTEE_HEADER*>(inBuf)->Version != SECTEE_ABI_VERSION)
        {
            status = STATUS_REVISION_MISMATCH;
            goto complete;
        }
    }
    if (OutputBufferLength)
    {
        status = WdfRequestRetrieveOutputBuffer(Request,
                    sizeof(SECTEE_HEADER), &outBuf, nullptr);
        if (!NT_SUCCESS(status)) goto complete;
    }

    switch (IoControlCode)
    {
    // --------------------------------------------------------
    case IOCTL_SECTEE_GET_PUBKEY:
    {
        if (OutputBufferLength < sizeof(SECTEE_PUBKEY_OUT))
        { status = STATUS_BUFFER_TOO_SMALL; break; }

        auto* out = static_cast<SECTEE_PUBKEY_OUT*>(outBuf);
        ULONG len = SECTEE_PUBKEY_LEN;

        status = ctx->Transport->Invoke(TA_CMD_GET_PUBKEY,
                                        nullptr, 0,
                                        out->PublicKey, &len);
        if (!NT_SUCCESS(status)) break;
        if (len != SECTEE_PUBKEY_LEN) { status = STATUS_DATA_ERROR; break; }

        out->Header.Version  = SECTEE_ABI_VERSION;
        out->Header.Reserved = 0;
```

```
        written = sizeof(SECTEE_PUBKEY_OUT);
        break;
    }

    // ------------------------------------------------------------
    case IOCTL_SECTEE_SIGN:
    {
        if (InputBufferLength  < sizeof(SECTEE_SIGN_IN) ||
            OutputBufferLength < sizeof(SECTEE_SIGN_OUT))
        { status = STATUS_BUFFER_TOO_SMALL; break; }

        auto* in  = static_cast<SECTEE_SIGN_IN*>(inBuf);
        auto* out = static_cast<SECTEE_SIGN_OUT*>(outBuf);

        // Validate the length the caller CLAIMS against the length
        // the I/O manager actually gave us. Trusting DataLength
        // alone is the classic Windows driver overread.
        const size_t headerBytes =
            FIELD_OFFSET(SECTEE_SIGN_IN, Data);
        if (in->DataLength > SECTEE_MAX_INPUT ||
            in->DataLength > InputBufferLength - headerBytes)
        { status = STATUS_INVALID_BUFFER_SIZE; break; }

        // 1. Hash HERE, in the normal world, with CNG.
        UCHAR digest[SECTEE_DIGEST_LEN];
        status = SecTeeSha256(in->Data, in->DataLength, digest);
        if (!NT_SUCCESS(status)) break;

        // 2. Only the 32-byte digest crosses the boundary.
        ULONG sigLen = SECTEE_SIG_LEN;
        status = ctx->Transport->Invoke(TA_CMD_SIGN,
                                        digest, SECTEE_DIGEST_LEN,
                                        out->Signature, &sigLen);
        if (!NT_SUCCESS(status)) break;
        if (sigLen != SECTEE_SIG_LEN) { status = STATUS_DATA_ERROR; break; }

        RtlCopyMemory(out->Digest, digest, SECTEE_DIGEST_LEN);
        out->Header.Version  = SECTEE_ABI_VERSION;
        out->Header.Reserved = 0;
        written = sizeof(SECTEE_SIGN_OUT);
        break;
    }

    // ------------------------------------------------------------
    case IOCTL_SECTEE_VERIFY:
    {
        if (InputBufferLength < sizeof(SECTEE_VERIFY_IN))
        { status = STATUS_BUFFER_TOO_SMALL; break; }

        auto* in = static_cast<SECTEE_VERIFY_IN*>(inBuf);

        UCHAR pub[SECTEE_PUBKEY_LEN];
        ULONG len = SECTEE_PUBKEY_LEN;
        status = ctx->Transport->Invoke(TA_CMD_GET_PUBKEY,
                                        nullptr, 0, pub, &len);
        if (!NT_SUCCESS(status)) break;

        // Verified with CNG, in the driver, against the public key
        // the secure world published. No secret is involved.
        status = SecTeeVerifyP256(pub, in->Digest, in->Signature);
        break;
    }
```

```
    // ------------------------------------------------------------
    case IOCTL_SECTEE_COUNTER:
    {
        if (InputBufferLength  < sizeof(SECTEE_COUNTER_INOUT) ||
            OutputBufferLength < sizeof(SECTEE_COUNTER_INOUT))
        { status = STATUS_BUFFER_TOO_SMALL; break; }

        auto* in  = static_cast<SECTEE_COUNTER_INOUT*>(inBuf);
        auto* out = static_cast<SECTEE_COUNTER_INOUT*>(outBuf);

        ULONG increment = in->Increment ? 1u : 0u;
        ULONGLONG value = 0;
        ULONG len = sizeof(value);

        status = ctx->Transport->Invoke(TA_CMD_COUNTER,
                                        &increment, sizeof(increment),
                                        &value, &len);
        if (!NT_SUCCESS(status)) break;
        if (len != sizeof(value)) { status = STATUS_DATA_ERROR; break; }

        out->Header.Version = SECTEE_ABI_VERSION;
        out->Header.Reserved = 0;
        out->Increment = increment;
        out->Reserved2 = 0;
        out->Value     = value;
        written = sizeof(SECTEE_COUNTER_INOUT);
        break;
    }

    default:
        status = STATUS_INVALID_DEVICE_REQUEST;
        break;
    }

complete:
    WdfRequestCompleteWithInformation(Request, status, written);
}
```

THE OVERREAD THIS CODE IS WRITTEN TO AVOID

`SECTEE_SIGN_IN::DataLength` is a number the caller chose. The buffer size is a number the I/O manager measured. Hashing `DataLength` bytes without checking it against `InputBufferLength - FIELD_OFFSET(...)` reads past the system buffer into whatever pool follows — a kernel information disclosure straight back to the caller, since the digest goes in the output. Check the claim against the measurement, every time, for every variable-length field.

The user-mode client

```
// SecTee/Client/secteectl.cpp
//
// Finds the device by interface GUID rather than by a hard-coded
// name, which is what lets the driver be installed more than once
// and is what SetupDi exists for (chapter 24).

#include <windows.h>
#include <setupapi.h>
#include <cfgmgr32.h>
#include <stdio.h>
#include <string>
#include <vector>
#include <memory>
#include "..\Common\SecTeeAbi.h"

#pragma comment(lib, "setupapi.lib")

namespace {

// RAII for a HANDLE. Here in user mode we DO have the STL and
// exceptions, and there is no reason not to use them.
struct HandleDeleter
{
    using pointer = HANDLE;
    void operator()(HANDLE h) const
    { if (h && h != INVALID_HANDLE_VALUE) CloseHandle(h); }
};
using UniqueHandle = std::unique_ptr<HANDLE, HandleDeleter>;

UniqueHandle OpenSecTee()
{
    HDEVINFO set = SetupDiGetClassDevsW(&GUID_DEVINTERFACE_SECTEE,
                       nullptr, nullptr,
                       DIGCF_PRESENT | DIGCF_DEVICEINTERFACE);
    if (set == INVALID_HANDLE_VALUE)
        throw std::runtime_error("SetupDiGetClassDevs failed");

    SP_DEVICE_INTERFACE_DATA ifData{};
    ifData.cbSize = sizeof(ifData);

    if (!SetupDiEnumDeviceInterfaces(set, nullptr,
                                     &GUID_DEVINTERFACE_SECTEE,
                                     0, &ifData))
    {
        SetupDiDestroyDeviceInfoList(set);
        throw std::runtime_error(
            "no SecTee device present - is the driver installed "
            "and started?");
    }

    DWORD needed = 0;
    SetupDiGetDeviceInterfaceDetailW(set, &ifData, nullptr, 0,
                                     &needed, nullptr);

    std::vector<BYTE> buf(needed);
    auto* detail = reinterpret_cast<PSP_DEVICE_INTERFACE_DETAIL_DATA_W>(
                       buf.data());
    detail->cbSize = sizeof(SP_DEVICE_INTERFACE_DETAIL_DATA_W);

    if (!SetupDiGetDeviceInterfaceDetailW(set, &ifData, detail,
```

```
                                        needed, nullptr, nullptr))
    {
        SetupDiDestroyDeviceInfoList(set);
        throw std::runtime_error("SetupDiGetDeviceInterfaceDetail failed");
    }

    UniqueHandle h(CreateFileW(detail->DevicePath,
                               GENERIC_READ | GENERIC_WRITE,
                               0, nullptr, OPEN_EXISTING, 0, nullptr));
    SetupDiDestroyDeviceInfoList(set);

    if (h.get() == INVALID_HANDLE_VALUE)
        throw std::runtime_error("CreateFile failed, error " +
                                 std::to_string(GetLastError()));
    return h;
}

void Ioctl(HANDLE h, DWORD code,
           void* in, DWORD inLen, void* out, DWORD outLen)
{
    DWORD returned = 0;
    if (!DeviceIoControl(h, code, in, inLen, out, outLen,
                         &returned, nullptr))
        throw std::runtime_error("ioctl 0x" + std::to_string(code) +
                                 " failed, error " +
                                 std::to_string(GetLastError()));
}

std::string Hex(const UCHAR* p, size_t n)
{
    static const char* d = "0123456789abcdef";
    std::string s;
    s.reserve(n * 2);
    for (size_t i = 0; i < n; i++)
    { s += d[p[i] >> 4]; s += d[p[i] & 0xf]; }
    return s;
}

std::vector<BYTE> ReadWholeFile(const wchar_t* path)
{
    UniqueHandle f(CreateFileW(path, GENERIC_READ, FILE_SHARE_READ,
                               nullptr, OPEN_EXISTING, 0, nullptr));
    if (f.get() == INVALID_HANDLE_VALUE)
        throw std::runtime_error("cannot open input file");

    LARGE_INTEGER size{};
    GetFileSizeEx(f.get(), &size);
    if (size.QuadPart > SECTEE_MAX_INPUT)
        throw std::runtime_error("file too large for one request");

    std::vector<BYTE> data(static_cast<size_t>(size.QuadPart));
    DWORD read = 0;
    if (!data.empty())
        ReadFile(f.get(), data.data(),
                 static_cast<DWORD>(data.size()), &read, nullptr);
    data.resize(read);
    return data;
}

} // namespace

int wmain(int argc, wchar_t** argv)
{
```

```
    try
    {
        auto dev = OpenSecTee();

        if (argc >= 2 && !wcscmp(argv[1], L"pubkey"))
        {
            SECTEE_PUBKEY_OUT out{};
            Ioctl(dev.get(), IOCTL_SECTEE_GET_PUBKEY,
                  nullptr, 0, &out, sizeof(out));
            printf("04%s\n", Hex(out.PublicKey, SECTEE_PUBKEY_LEN).c_str());
            //     ^^ the SEC1 uncompressed prefix, added here so the
            //        output can be pasted into OpenSSL. CNG did not
            //        want it; OpenSSL does.
        }
        else if (argc >= 3 && !wcscmp(argv[1], L"sign"))
        {
            auto data = ReadWholeFile(argv[2]);

            const size_t inSize =
                FIELD_OFFSET(SECTEE_SIGN_IN, Data) + data.size();
            std::vector<BYTE> inBuf(inSize);
            auto* in = reinterpret_cast<SECTEE_SIGN_IN*>(inBuf.data());
            in->Header.Version = SECTEE_ABI_VERSION;
            in->Header.Reserved = 0;
            in->DataLength = static_cast<ULONG>(data.size());
            in->Reserved2  = 0;
            if (!data.empty())
                memcpy(in->Data, data.data(), data.size());

            SECTEE_SIGN_OUT out{};
            Ioctl(dev.get(), IOCTL_SECTEE_SIGN,
                  inBuf.data(), static_cast<DWORD>(inSize),
                  &out, sizeof(out));

            printf("digest %s\n", Hex(out.Digest, SECTEE_DIGEST_LEN).c_str());
            printf("sig    %s\n", Hex(out.Signature, SECTEE_SIG_LEN).c_str());
        }
        else if (argc >= 3 && !wcscmp(argv[1], L"verify"))
        {
            auto data = ReadWholeFile(argv[2]);

            // Sign, then ask the driver to verify its own result.
            // A round trip that exercises CNG on both sides.
            const size_t inSize =
                FIELD_OFFSET(SECTEE_SIGN_IN, Data) + data.size();
            std::vector<BYTE> inBuf(inSize);
            auto* si = reinterpret_cast<SECTEE_SIGN_IN*>(inBuf.data());
            si->Header.Version = SECTEE_ABI_VERSION;
            si->DataLength = static_cast<ULONG>(data.size());
            memcpy(si->Data, data.data(), data.size());

            SECTEE_SIGN_OUT so{};
            Ioctl(dev.get(), IOCTL_SECTEE_SIGN,
                  inBuf.data(), static_cast<DWORD>(inSize),
                  &so, sizeof(so));

            SECTEE_VERIFY_IN vi{};
            vi.Header.Version = SECTEE_ABI_VERSION;
            memcpy(vi.Digest,    so.Digest,    SECTEE_DIGEST_LEN);
            memcpy(vi.Signature, so.Signature, SECTEE_SIG_LEN);

            Ioctl(dev.get(), IOCTL_SECTEE_VERIFY,
                  &vi, sizeof(vi), nullptr, 0);
```

```
            printf("VALID\n");
        }
        else if (argc >= 2 && !wcscmp(argv[1], L"counter"))
        {
            SECTEE_COUNTER_INOUT io{};
            io.Header.Version = SECTEE_ABI_VERSION;
            io.Increment = (argc >= 3 && !wcscmp(argv[2], L"inc")) ? 1 : 0;
            Ioctl(dev.get(), IOCTL_SECTEE_COUNTER,
                  &io, sizeof(io), &io, sizeof(io));
            printf("%llu\n", io.Value);
        }
        else
        {
            fwprintf(stderr, L"usage: secteectl pubkey\n"
                             L"       secteectl sign   <file>\n"
                             L"       secteectl verify <file>\n"
                             L"       secteectl counter [inc]\n");
            return 2;
        }
        return 0;
    }
    catch (const std::exception& e)
    {
        fprintf(stderr, "secteectl: %s\n", e.what());
        return 1;
    }
}
```

The INF

```
; SecTee.inf - binds to the ivshmem PCI function on QEMU.
;
; On real hardware the same driver would be an ACPI-enumerated
; device with a TrEE-capable hardware ID; only this section and
; the transport factory in PrepareHardware change.

[Version]
Signature   = "$WINDOWS NT$"
Class       = System
ClassGuid   = {4d36e97d-e325-11ce-bfc1-08002be10318}
Provider    = %ManufacturerName%
CatalogFile = SecTee.cat
DriverVer   =
PnpLockdown = 1

[DestinationDirs]
DefaultDestDir = 13          ; the driver store package directory

[SourceDisksNames]
1 = %DiskName%

[SourceDisksFiles]
SecTee.sys = 1

[Manufacturer]
%ManufacturerName% = Standard, NTARM64

; ivshmem: Red Hat vendor 1AF4, device 1110.
[Standard.NTARM64]
%SecTee.DeviceDesc% = SecTee_Install, PCI\VEN_1AF4&DEV_1110

[SecTee_Install.NT]
CopyFiles = SecTee_CopyFiles

[SecTee_CopyFiles]
SecTee.sys

[SecTee_Install.NT.Services]
AddService = SecTee, 0x00000002, SecTee_Service

[SecTee_Service]
DisplayName   = %SecTee.SvcDesc%
ServiceType   = 1            ; SERVICE_KERNEL_DRIVER
StartType     = 3            ; SERVICE_DEMAND_START
ErrorControl  = 1
ServiceBinary = %13%\SecTee.sys

[SecTee_Install.NT.Wdf]
KmdfService = SecTee, SecTee_wdfsect

[SecTee_wdfsect]
KmdfLibraryVersion = 1.33

[Strings]
ManufacturerName   = "Embedded Learning Hub"
DiskName           = "SecTee Installation Disk"
SecTee.DeviceDesc  = "TrustZone-backed crypto device"
SecTee.SvcDesc     = "SecTee Service"
```

```
REM Building. From an ARM64 Developer Command Prompt for the WDK.

msbuild SecTee.sln /p:Configuration=Release /p:Platform=ARM64

REM Test-sign it, because an unsigned driver will not load
REM (chapter 20). On the TARGET machine, once:
bcdedit /set testsigning on
shutdown /r /t 0

REM Then, on the target, as Administrator:
certmgr /add SecTee.cer /s /r localMachine root
certmgr /add SecTee.cer /s /r localMachine trustedpublisher
pnputil /add-driver SecTee.inf /install

REM Confirm:
pnputil /enum-devices /class System /connected
sc query SecTee
```

How it shows up

Symptom	Cause
Link error: unresolved `operator new`	`CppSupport.h` not included, or a parameterless `new` used somewhere
Link error: `_purecall`, `__CxxFrameHandler`, `atexit`	Exceptions or RTTI still enabled, or a global object with a constructor
Driver loads, then bugchecks on first IOCTL	CNG called above PASSIVE_LEVEL — set `ExecutionLevel = WdfExecutionLevelPassive` on the queue
`BCryptOpenAlgorithmProvider` returns `STATUS_INVALID_PARAMETER`	`BCRYPT_PROV_DISPATCH` not passed
Unresolved `BCrypt*` at link	`ksecdd.lib` not in the linker inputs
`STATUS_DEVICE_NOT_READY` from PrepareHardware	Mailbox magic absent — the far-end daemon is not running (chapter 27)
Every request times out after 5 s	Doorbell going to the wrong peer, or the daemon is not watching the doorbell
Ack never observed although the daemon says it replied	Shared memory mapped cached — must be `PAGE_NOCACHE`
`STATUS_REVISION_MISMATCH`	Client and driver built against different `SecTeeAbi.h`
Signature verifies with CNG, fails with OpenSSL	Expected — OpenSSL needs DER, CNG needs raw r‖s
Public key rejected by CNG	The `0x04` SEC1 prefix was included; CNG's `BCRYPT_ECCKEY_BLOB` does not take it
Driver will not install: "third-party INF does not contain digital signature information"	Not test-signed, or test signing is off on the target
Verifier flags a pool leak on unload	`ReleaseHardware` not deleting the transport, or an early-return path in `PrepareHardware` without the scope guard

Check yourself

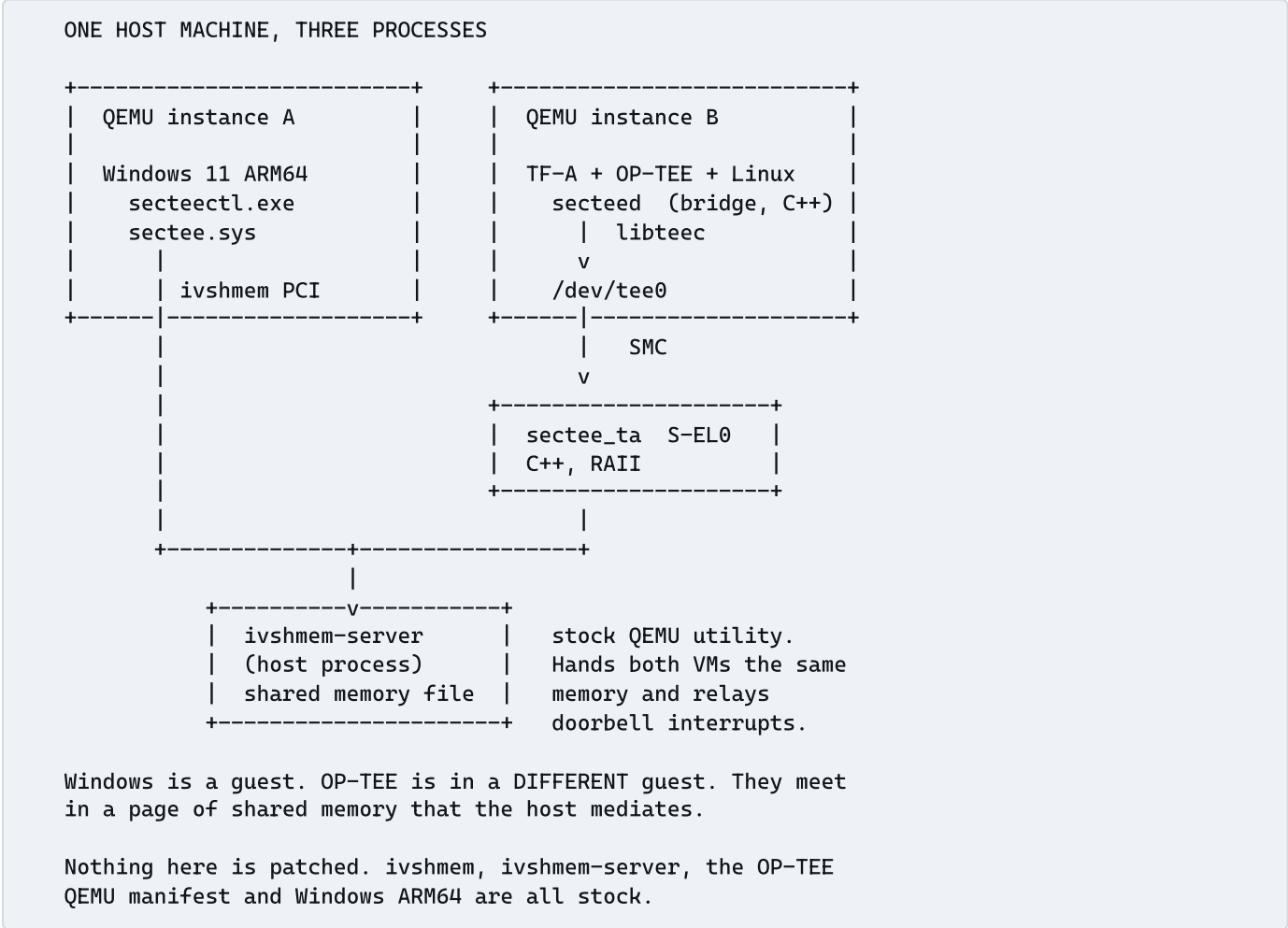
- ✓ State what Windows actually uses to reach the secure world on ARM, and why this project needs a transport seam.
- ✓ Say precisely which security property the QEMU configuration does *not* demonstrate.

- ✓ List four things C++ gives you in a kernel driver and four it forbids, with the reason for each prohibition.
- ✓ Explain why `operator new` here takes a pool type instead of defaulting.
- ✓ Justify `METHOD_BUFFERED` for this ABI and say when you would switch.
- ✓ Explain why `DataLength` must be checked against `InputBufferLength`, and what leaks if it is not.
- ✓ Give the two extra rules for calling CNG from kernel mode.
- ✓ Explain why the driver hashes but the TA signs.
- ✓ State the difference between CNG's and OpenSSL's ECDSA signature encodings, and which one the TA emits.
- ✓ Explain why the shared memory must be mapped non-cached.
- ✓ Point to the memory barrier before the sequence increment and say what it prevents.
- ✓ Trace one `secteectl sign` from `DeviceIoControl` to the transport and back, naming every buffer the data occupies.

Project: the trusted application, and running it on QEMU

The secure half of the project: an OP-TEE trusted application written in C++, a bridge daemon that connects it to the Windows guest, and the two-VM QEMU topology that lets you run the whole thing on a laptop. Plus an honest accounting of which security properties this configuration demonstrates and which it only simulates.

The topology



WHAT THIS CONFIGURATION DOES AND DOES NOT PROVE

Does: that the driver, the ABI, the marshalling, the trusted application and the client are all correct; that a key generated inside OP-TEE signs digests the Windows guest computes; that CNG in the Windows kernel accepts those signatures. Every line of software is exercised for real.

Does not: demonstrate TrustZone isolation. The Windows guest is separated from the TA by *VM boundaries*, not by the NS bit. On production hardware the same driver reaches the same TA through TrEE and the isolation is real; here it is a development harness. Never quote a QEMU run as evidence of a security property.

Why write a TA in C++

Not for templates or the STL — neither is available. For one thing only, and it is worth the whole chapter: **destructors**. Compare the C key-store TA of OP-TEE chapter 21 with what follows.

```
   THE C SHAPE, from chapter 21          THE C++ SHAPE

   TEE_ObjectHandle  obj = NULL;         {
   TEE_OperationHandle op = NULL;            PersistentKey key(id);
   res = open(&obj);                         if (!key) return key.Error();
   if (res) goto out;
   res = TEE_AllocateOperation(&op...);      SignOperation op(key);
   if (res) goto out;                        if (!op) return op.Error();
   res = TEE_SetOperationKey(op, obj);
   if (res) goto out;                        return op.Sign(digest, sig);
   res = TEE_AsymmetricSignDigest(...);  }   // both freed here, on
   out:                                      // EVERY path, in reverse
       TEE_FreeOperation(op);                // order, including the
       TEE_CloseObject(obj);                 // early returns
       return res;

   A leaked TEE_OperationHandle is not a tidiness problem. The
   secure heap is a few tens of kilobytes (chapter 6); leak one
   per request and the TA panics after a few thousand
   operations, in production, weeks later, under load.

   The goto ladder works. It also breaks the moment someone adds
   a fourth resource in the middle and forgets one branch. RAII
   makes that class of bug unrepresentable.
```

```
# ta/Makefile
CFG_TEE_TA_LOG_LEVEL ?= 3
BINARY = 3c8d1f60-5a72-4e91-b4d3-7e2a0f6c9b18
-include $(TA_DEV_KIT_DIR)/mk/ta_dev_kit.mk

# ta/sub.mk
global-incdirs-y += include
srcs-y += sectee_ta.cpp

# C++ in a TA needs the runtime switched off. There is no
# unwinder, no typeinfo and no __cxa_atexit behind you.
cxxflags-y += -fno-exceptions
cxxflags-y += -fno-rtti
cxxflags-y += -fno-use-cxa-atexit
cxxflags-y += -fno-threadsafe-statics
cxxflags-y += -std=c++17

# And -fno-threadsafe-statics matters specifically: a function-
# local static with a constructor would otherwise emit a call to
# __cxa_guard_acquire, which does not exist here.
```

The trusted application

```
// ta/include/sectee_ta.h  -  shared with the bridge daemon.
// Plain C so it can be included from either language.

#ifndef SECTEE_TA_H
#define SECTEE_TA_H

#define SECTEE_TA_UUID \
    { 0x3c8d1f60, 0x5a72, 0x4e91, \
      { 0xb4, 0xd3, 0x7e, 0x2a, 0x0f, 0x6c, 0x9b, 0x18 } }

#define TA_CMD_GET_PUBKEY 0
#define TA_CMD_SIGN       1
#define TA_CMD_COUNTER    2

#define SECTEE_PUBKEY_LEN 64
#define SECTEE_SIG_LEN    64
#define SECTEE_DIGEST_LEN 32

#endif
```

```
// ta/user_ta_header_defines.h
#ifndef USER_TA_HEADER_DEFINES_H
#define USER_TA_HEADER_DEFINES_H

#include "sectee_ta.h"

#define TA_UUID SECTEE_TA_UUID

// SINGLE_INSTANCE + KEEP_ALIVE: one copy, never unloaded, so the
// counter and the loaded key survive between sessions. Without
// KEEP_ALIVE the TA is torn down when the last session closes and
// the next request pays a full load (chapter 13).
#define TA_FLAGS (TA_FLAG_SINGLE_INSTANCE | TA_FLAG_INSTANCE_KEEP_ALIVE)

#define TA_STACK_SIZE (4 * 1024)
#define TA_DATA_SIZE  (32 * 1024)

#define TA_CURRENT_TA_EXT_PROPERTIES \
    { "gp.ta.description", USER_TA_PROP_TYPE_STRING, \
      "SecTee signing service" }, \
    { "gp.ta.version", USER_TA_PROP_TYPE_U32, \
      &(const uint32_t){ 0x0100 } }

#endif
```

```
// ta/sectee_ta.cpp

extern "C" {
#include <tee_internal_api.h>
#include <tee_internal_api_extensions.h>
#include <string.h>
}
#include "sectee_ta.h"

namespace {

constexpr const char KEY_ID[]     = "sectee.key.v1";
constexpr const char COUNTER_ID[] = "sectee.counter.v1";
constexpr uint32_t   P256_BITS    = 256;

// ---------------------------------------------------------------
// RAII wrappers. Each owns exactly one TEE handle, is
// non-copyable, and frees on destruction. This is the entire
// argument for C++ here.
// ---------------------------------------------------------------

class Operation
{
public:
    Operation(uint32_t alg, uint32_t mode, uint32_t maxKeyBits)
    {
        m_res = TEE_AllocateOperation(&m_h, alg, mode, maxKeyBits);
        if (m_res != TEE_SUCCESS) m_h = TEE_HANDLE_NULL;
    }
    ~Operation()
    {
        if (m_h != TEE_HANDLE_NULL) TEE_FreeOperation(m_h);
    }

    Operation(const Operation&)            = delete;
    Operation& operator=(const Operation&) = delete;

    explicit operator bool() const { return m_res == TEE_SUCCESS; }
    TEE_Result Error() const       { return m_res; }
    TEE_OperationHandle Get() const { return m_h; }

private:
    TEE_OperationHandle m_h   = TEE_HANDLE_NULL;
    TEE_Result          m_res = TEE_SUCCESS;
};

class TransientObject
{
public:
    TransientObject(uint32_t type, uint32_t maxKeyBits)
    {
        m_res = TEE_AllocateTransientObject(type, maxKeyBits, &m_h);
        if (m_res != TEE_SUCCESS) m_h = TEE_HANDLE_NULL;
    }
    ~TransientObject()
    {
        if (m_h != TEE_HANDLE_NULL) TEE_FreeTransientObject(m_h);
    }

    TransientObject(const TransientObject&)            = delete;
    TransientObject& operator=(const TransientObject&) = delete;

    explicit operator bool() const { return m_res == TEE_SUCCESS; }
```

```
    TEE_Result Error() const        { return m_res; }
    TEE_ObjectHandle Get() const    { return m_h; }
    TEE_ObjectHandle* Put()         { return &m_h; }

private:
    TEE_ObjectHandle m_h   = TEE_HANDLE_NULL;
    TEE_Result       m_res = TEE_SUCCESS;
};

class PersistentObject
{
public:
    PersistentObject() = default;

    ~PersistentObject()
    {
        if (m_h != TEE_HANDLE_NULL) TEE_CloseObject(m_h);
    }

    PersistentObject(const PersistentObject&)            = delete;
    PersistentObject& operator=(const PersistentObject&) = delete;

    TEE_Result Open(const char* id, size_t idLen, uint32_t flags)
    {
        return TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,
                                        id, idLen, flags, &m_h);
    }

    TEE_Result Create(const char* id, size_t idLen, uint32_t flags,
                      TEE_ObjectHandle attrs,
                      const void* initial, size_t initialLen)
    {
        return TEE_CreatePersistentObject(TEE_STORAGE_PRIVATE,
                                          id, idLen, flags, attrs,
                                          initial, initialLen, &m_h);
    }

    // Release ownership without closing - used when we hand the
    // handle to TEE_CloseAndDeletePersistentObject1, which closes
    // it for us. Double-closing a TEE handle panics the TA.
    TEE_ObjectHandle Release()
    {
        TEE_ObjectHandle h = m_h;
        m_h = TEE_HANDLE_NULL;
        return h;
    }

    bool Valid() const           { return m_h != TEE_HANDLE_NULL; }
    TEE_ObjectHandle Get() const { return m_h; }

private:
    TEE_ObjectHandle m_h = TEE_HANDLE_NULL;
};

// ---------------------------------------------------------------
// Helpers
// ---------------------------------------------------------------

// Open the signing key, generating it on first use.
//
// The private scalar is created by TEE_GenerateKey inside S-EL0 and
// written into a persistent object that OP-TEE encrypts with a key
// derived from the hardware unique key (OP-TEE chapter 15). It is
```

```
// never serialised into a buffer, so it never exists anywhere the
// normal world could observe even in principle.
TEE_Result OpenOrCreateKey(PersistentObject& out)
{
    TEE_Result res = out.Open(KEY_ID, sizeof(KEY_ID) - 1,
                              TEE_DATA_FLAG_ACCESS_READ);
    if (res == TEE_SUCCESS) return TEE_SUCCESS;
    if (res != TEE_ERROR_ITEM_NOT_FOUND) return res;

    IMSG("generating a new P-256 keypair");

    TransientObject key(TEE_TYPE_ECDSA_KEYPAIR, P256_BITS);
    if (!key) return key.Error();

    TEE_Attribute curve;
    TEE_InitValueAttribute(&curve, TEE_ATTR_ECC_CURVE,
                           TEE_ECC_CURVE_NIST_P256, 0);

    res = TEE_GenerateKey(key.Get(), P256_BITS, &curve, 1);
    if (res != TEE_SUCCESS) return res;

    // Persist the OBJECT, not bytes.
    return out.Create(KEY_ID, sizeof(KEY_ID) - 1,
                      TEE_DATA_FLAG_ACCESS_READ |
                      TEE_DATA_FLAG_ACCESS_WRITE_META,
                      key.Get(), nullptr, 0);
    // `key` is freed here by its destructor, on every path above.
}

// Copy an ECC coordinate out, LEFT-PADDED to a fixed 32 bytes.
//
// TEE_GetObjectBufferAttribute returns the minimal encoding, so a
// coordinate with a leading zero byte comes back 31 bytes long -
// about one time in 256. Writing it flush-left produces a key that
// works 99.6% of the time, which is the worst possible failure
// rate: too rare to catch in testing, common enough to ship.
TEE_Result GetCoordinate(TEE_ObjectHandle obj, uint32_t attrId,
                         uint8_t out[32])
{
    uint8_t  tmp[32];
    uint32_t len = sizeof(tmp);

    TEE_Result res = TEE_GetObjectBufferAttribute(obj, attrId,
                                                  tmp, &len);
    if (res != TEE_SUCCESS) return res;
    if (len > 32) return TEE_ERROR_BAD_STATE;

    TEE_MemFill(out, 0, 32);
    TEE_MemMove(out + (32 - len), tmp, len);
    return TEE_SUCCESS;
}

// ------------------------------------------------------------------
// TA_CMD_GET_PUBKEY   [0] memref out, 64 bytes
// ------------------------------------------------------------------
TEE_Result CmdGetPubkey(uint32_t ptypes, TEE_Param params[4])
{
    const uint32_t expected =
        TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_OUTPUT,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE);
    if (ptypes != expected)
```

```
    {
        EMSG("ptypes 0x%08x expected 0x%08x", ptypes, expected);
        return TEE_ERROR_BAD_PARAMETERS;
    }
    if (params[0].memref.size < SECTEE_PUBKEY_LEN)
    {
        params[0].memref.size = SECTEE_PUBKEY_LEN;
        return TEE_ERROR_SHORT_BUFFER;
    }

    PersistentObject key;
    TEE_Result res = OpenOrCreateKey(key);
    if (res != TEE_SUCCESS) return res;

    uint8_t pub[SECTEE_PUBKEY_LEN];
    res = GetCoordinate(key.Get(), TEE_ATTR_ECC_PUBLIC_VALUE_X, pub);
    if (res != TEE_SUCCESS) return res;
    res = GetCoordinate(key.Get(), TEE_ATTR_ECC_PUBLIC_VALUE_Y, pub + 32);
    if (res != TEE_SUCCESS) return res;

    // Write the output ONCE, at the end, on success only.
    TEE_MemMove(params[0].memref.buffer, pub, SECTEE_PUBKEY_LEN);
    params[0].memref.size = SECTEE_PUBKEY_LEN;
    return TEE_SUCCESS;
}

// ---------------------------------------------------------------
// TA_CMD_SIGN   [0] memref in  (32-byte digest)
//               [1] memref out (64-byte signature)
// ---------------------------------------------------------------
TEE_Result CmdSign(uint32_t ptypes, TEE_Param params[4])
{
    const uint32_t expected =
        TEE_PARAM_TYPES(TEE_PARAM_TYPE_MEMREF_INPUT,
                        TEE_PARAM_TYPE_MEMREF_OUTPUT,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE);
    if (ptypes != expected) return TEE_ERROR_BAD_PARAMETERS;

    // Exactly 32 bytes or nothing. A fixed size means there is no
    // length arithmetic in this function to get wrong.
    if (params[0].memref.size != SECTEE_DIGEST_LEN)
        return TEE_ERROR_BAD_PARAMETERS;
    if (params[1].memref.size < SECTEE_SIG_LEN)
    {
        params[1].memref.size = SECTEE_SIG_LEN;
        return TEE_ERROR_SHORT_BUFFER;
    }

    if (TEE_CheckMemoryAccessRights(
            TEE_MEMORY_ACCESS_READ | TEE_MEMORY_ACCESS_ANY_OWNER,
            params[0].memref.buffer,
            SECTEE_DIGEST_LEN) != TEE_SUCCESS)
        return TEE_ERROR_ACCESS_DENIED;

    // ONE fetch out of shared memory, then work on the copy. The
    // normal world can rewrite that buffer at any instant,
    // including between two reads of the same byte
    // (OP-TEE chapter 20).
    uint8_t digest[SECTEE_DIGEST_LEN];
    TEE_MemMove(digest, params[0].memref.buffer, SECTEE_DIGEST_LEN);

    PersistentObject key;
```

```
    TEE_Result res = OpenOrCreateKey(key);
    if (res != TEE_SUCCESS) return res;

    Operation op(TEE_ALG_ECDSA_P256, TEE_MODE_SIGN, P256_BITS);
    if (!op) return op.Error();

    res = TEE_SetOperationKey(op.Get(), key.Get());
    if (res != TEE_SUCCESS) return res;

    uint8_t  sig[SECTEE_SIG_LEN];
    uint32_t sigLen = sizeof(sig);

    res = TEE_AsymmetricSignDigest(op.Get(), nullptr, 0,
                                   digest, SECTEE_DIGEST_LEN,
                                   sig, &sigLen);
    if (res != TEE_SUCCESS) return res;

    TEE_MemMove(params[1].memref.buffer, sig, sigLen);
    params[1].memref.size = sigLen;

    TEE_MemFill(digest, 0, sizeof(digest));
    return TEE_SUCCESS;
    // op and key are released here, in reverse order, on this path
    // and on all eight early returns above.
}

// ---------------------------------------------------------------
// TA_CMD_COUNTER   [0] value inout: a = increment?, b unused
//                                   out: a = low32, b = high32
// ---------------------------------------------------------------
TEE_Result CmdCounter(uint32_t ptypes, TEE_Param params[4])
{
    const uint32_t expected =
        TEE_PARAM_TYPES(TEE_PARAM_TYPE_VALUE_INOUT,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE,
                        TEE_PARAM_TYPE_NONE);
    if (ptypes != expected) return TEE_ERROR_BAD_PARAMETERS;

    const bool increment = (params[0].value.a == 1);
    uint64_t   value     = 0;

    PersistentObject obj;
    TEE_Result res = obj.Open(COUNTER_ID, sizeof(COUNTER_ID) - 1,
                              TEE_DATA_FLAG_ACCESS_READ |
                              TEE_DATA_FLAG_ACCESS_WRITE);

    if (res == TEE_SUCCESS)
    {
        uint32_t got = 0;
        res = TEE_ReadObjectData(obj.Get(), &value, sizeof(value), &got);
        if (res != TEE_SUCCESS) return res;
        if (got != sizeof(value)) value = 0;

        if (increment)
        {
            value++;
            res = TEE_SeekObjectData(obj.Get(), 0, TEE_DATA_SEEK_SET);
            if (res != TEE_SUCCESS) return res;
            res = TEE_WriteObjectData(obj.Get(), &value, sizeof(value));
            if (res != TEE_SUCCESS) return res;
        }
    }
```

```
    else if (res == TEE_ERROR_ITEM_NOT_FOUND)
    {
        value = increment ? 1 : 0;
        res = obj.Create(COUNTER_ID, sizeof(COUNTER_ID) - 1,
                         TEE_DATA_FLAG_ACCESS_READ |
                         TEE_DATA_FLAG_ACCESS_WRITE |
                         TEE_DATA_FLAG_ACCESS_WRITE_META,
                         TEE_HANDLE_NULL, &value, sizeof(value));
        if (res != TEE_SUCCESS) return res;
    }
    else
    {
        return res;
    }

    // A GP value parameter is a pair of u32, so a u64 goes across
    // in two halves (OP-TEE chapter 12).
    params[0].value.a = static_cast<uint32_t>(value & 0xffffffffu);
    params[0].value.b = static_cast<uint32_t>(value >> 32);
    return TEE_SUCCESS;
}

} // anonymous namespace

// ---------------------------------------------------------------
// The five entry points. extern "C" because OP-TEE's framework
// looks them up by C linkage name.
// ---------------------------------------------------------------
extern "C" {

TEE_Result TA_CreateEntryPoint(void)
{
    IMSG("sectee TA loaded");
    return TEE_SUCCESS;
}

void TA_DestroyEntryPoint(void)
{
    IMSG("sectee TA unloaded");
}

TEE_Result TA_OpenSessionEntryPoint(uint32_t ptypes,
                                    TEE_Param params[4],
                                    void** sessCtx)
{
    (void)params;
    if (ptypes != TEE_PARAM_TYPES(TEE_PARAM_TYPE_NONE,
                                  TEE_PARAM_TYPE_NONE,
                                  TEE_PARAM_TYPE_NONE,
                                  TEE_PARAM_TYPE_NONE))
        return TEE_ERROR_BAD_PARAMETERS;

    *sessCtx = nullptr;   // no per-session state
    return TEE_SUCCESS;
}

void TA_CloseSessionEntryPoint(void* sessCtx)
{
    (void)sessCtx;
}

TEE_Result TA_InvokeCommandEntryPoint(void* sessCtx, uint32_t cmd,
                                      uint32_t ptypes,
```

```
                                    TEE_Param params[4])
{
    (void)sessCtx;
    switch (cmd)
    {
    case TA_CMD_GET_PUBKEY: return CmdGetPubkey(ptypes, params);
    case TA_CMD_SIGN:       return CmdSign(ptypes, params);
    case TA_CMD_COUNTER:    return CmdCounter(ptypes, params);
    default:                return TEE_ERROR_NOT_SUPPORTED;
    }
}

} // extern "C"
```

THREE C++ TRAPS SPECIFIC TO A TA

Globals with constructors. There is no CRT to run them. Anything at file scope must be `constexpr` or a POD, which is why the constants above are `constexpr` and every stateful object is a local. **Function-local statics.** They emit `__cxa_guard_acquire` unless you pass `-fno-threadsafe-statics`. **Double-free.** `TEE_CloseAndDeletePersistentObject1` closes the handle itself, so an RAII wrapper must `Release()` first — closing a TEE handle twice panics the TA, and the panic appears on the secure console with a stack you will have to decode by hand (OP-TEE chapter 19).

The bridge daemon

```
// bridge/secteed.cpp
//
// Runs in the OP-TEE Linux guest. Owns the same shared memory the
// Windows driver mapped, watches the sequence number, and turns
// each request into a TEEC_InvokeCommand.
//
// Build:
//   aarch64-linux-gnu-g++ -std=c++17 -O2 -o secteed secteed.cpp \
//       -I$TEEC_EXPORT/include -L$TEEC_EXPORT/lib -lteec

#include <atomic>
#include <cerrno>
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <stdexcept>
#include <string>

#include <fcntl.h>
#include <poll.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

extern "C" {
#include <tee_client_api.h>
}
#include "../ta/include/sectee_ta.h"

namespace {

constexpr uint32_t MAILBOX_MAGIC    = 0x4D454554u;  // "TEEM"
constexpr uint32_t MAILBOX_VERSION  = 1u;
constexpr size_t   MAILBOX_DATA_OFF = 0x100;
constexpr size_t   MAILBOX_DATA_MAX = 0x1000;

// Byte-for-byte the layout the Windows driver declares. If these
// two structs ever disagree the symptom is silent nonsense, so
// keep them in one place if you can and assert the size if you
// cannot.
#pragma pack(push, 8)
struct MailboxHeader
{
    uint32_t Magic;
    uint32_t Version;
    volatile int32_t Sequence;   // written by Windows
    volatile int32_t Ack;        // written by us
    uint32_t Command;
    uint32_t InLength;
    uint32_t OutLength;
    int32_t  Result;
};
#pragma pack(pop)
static_assert(sizeof(MailboxHeader) == 32, "layout drift");

// RAII again, on this side too.
class TeeSession
{
public:
    TeeSession()
```

```
    {
        TEEC_Result res = TEEC_InitializeContext(nullptr, &m_ctx);
        if (res != TEEC_SUCCESS)
            throw std::runtime_error("TEEC_InitializeContext: " +
                                     std::to_string(res));
        m_haveCtx = true;

        TEEC_UUID uuid = SECTEE_TA_UUID;
        uint32_t origin = 0;
        res = TEEC_OpenSession(&m_ctx, &m_sess, &uuid,
                               TEEC_LOGIN_PUBLIC, nullptr,
                               nullptr, &origin);
        if (res != TEEC_SUCCESS)
            throw std::runtime_error("TEEC_OpenSession: " +
                                     std::to_string(res) +
                                     " origin " + std::to_string(origin));
        m_haveSess = true;
    }

    ~TeeSession()
    {
        if (m_haveSess) TEEC_CloseSession(&m_sess);
        if (m_haveCtx)  TEEC_FinalizeContext(&m_ctx);
    }

    TeeSession(const TeeSession&)            = delete;
    TeeSession& operator=(const TeeSession&) = delete;

    // Returns the TEE_Result; fills out/outLen on success.
    uint32_t Invoke(uint32_t cmd,
                    const uint8_t* in, uint32_t inLen,
                    uint8_t* out, uint32_t* outLen)
    {
        TEEC_Operation op{};
        uint32_t origin = 0;

        switch (cmd)
        {
        case TA_CMD_GET_PUBKEY:
            op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_OUTPUT,
                                             TEEC_NONE, TEEC_NONE,
                                             TEEC_NONE);
            op.params[0].tmpref.buffer = out;
            op.params[0].tmpref.size   = *outLen;
            break;

        case TA_CMD_SIGN:
            if (inLen != SECTEE_DIGEST_LEN) return TEEC_ERROR_BAD_PARAMETERS;
            op.paramTypes = TEEC_PARAM_TYPES(TEEC_MEMREF_TEMP_INPUT,
                                             TEEC_MEMREF_TEMP_OUTPUT,
                                             TEEC_NONE, TEEC_NONE);
            op.params[0].tmpref.buffer = const_cast<uint8_t*>(in);
            op.params[0].tmpref.size   = inLen;
            op.params[1].tmpref.buffer = out;
            op.params[1].tmpref.size   = *outLen;
            break;

        case TA_CMD_COUNTER:
        {
            if (inLen != sizeof(uint32_t)) return TEEC_ERROR_BAD_PARAMETERS;
            uint32_t increment;
            std::memcpy(&increment, in, sizeof(increment));
```

```
            op.paramTypes = TEEC_PARAM_TYPES(TEEC_VALUE_INOUT,
                                             TEEC_NONE, TEEC_NONE,
                                             TEEC_NONE);
            op.params[0].value.a = increment;
            op.params[0].value.b = 0;

            uint32_t res = TEEC_InvokeCommand(&m_sess, cmd, &op, &origin);
            if (res != TEEC_SUCCESS) return res;

            uint64_t value = static_cast<uint64_t>(op.params[0].value.a) |
                             (static_cast<uint64_t>(op.params[0].value.b) << 32);
            if (*outLen < sizeof(value)) return TEEC_ERROR_SHORT_BUFFER;
            std::memcpy(out, &value, sizeof(value));
            *outLen = sizeof(value);
            return TEEC_SUCCESS;
        }

        default:
            return TEEC_ERROR_NOT_SUPPORTED;
        }

        uint32_t res = TEEC_InvokeCommand(&m_sess, cmd, &op, &origin);
        if (res != TEEC_SUCCESS)
        {
            std::fprintf(stderr, "cmd %u -> 0x%08x origin %u\n",
                         cmd, res, origin);
            return res;
        }

        *outLen = (cmd == TA_CMD_GET_PUBKEY)
                ? op.params[0].tmpref.size
                : op.params[1].tmpref.size;
        return TEEC_SUCCESS;
    }

private:
    TEEC_Context m_ctx{};
    TEEC_Session m_sess{};
    bool m_haveCtx  = false;
    bool m_haveSess = false;
};

} // namespace

int main(int argc, char** argv)
{
    const char* uio = (argc > 1) ? argv[1] : "/dev/uio0";

    try
    {
        // ivshmem in the Linux guest shows up as a uio device.
        // map1 is BAR2, the shared memory.
        int fd = open(uio, O_RDWR);
        if (fd < 0) throw std::runtime_error(std::string("open ") + uio);

        // Size comes from sysfs; 1 MB is what we ask ivshmem-server for.
        constexpr size_t SHM_SIZE = 1024 * 1024;

        void* shm = mmap(nullptr, SHM_SIZE, PROT_READ | PROT_WRITE,
                         MAP_SHARED, fd, 1 * getpagesize());
        if (shm == MAP_FAILED) throw std::runtime_error("mmap BAR2");

        auto* hdr  = static_cast<MailboxHeader*>(shm);
```

```
        auto* data = static_cast<uint8_t*>(shm) + MAILBOX_DATA_OFF;

        TeeSession tee;   // opens the TA, and closes it on any throw

        // Publish the magic LAST, after everything is ready. The
        // Windows driver refuses to start until it sees this, so
        // it doubles as a readiness handshake.
        hdr->Version  = MAILBOX_VERSION;
        hdr->Ack      = hdr->Sequence;
        std::atomic_thread_fence(std::memory_order_release);
        hdr->Magic    = MAILBOX_MAGIC;

        std::printf("secteed: ready, waiting for the Windows guest\n");

        int32_t lastSeq = hdr->Sequence;

        for (;;)
        {
            // Wait for the doorbell. Reading the uio fd blocks
            // until an interrupt arrives; the 32-bit value is the
            // interrupt count.
            uint32_t count = 0;
            struct pollfd pfd { fd, POLLIN, 0 };

            if (poll(&pfd, 1, 1000) > 0)
                (void)!read(fd, &count, sizeof(count));

            const int32_t seq = hdr->Sequence;
            if (seq == lastSeq) continue;      // spurious wake

            // Observe the sequence number before the payload it
            // announces. Mirrors the barrier on the Windows side.
            std::atomic_thread_fence(std::memory_order_acquire);

            const uint32_t cmd   = hdr->Command;
            const uint32_t inLen = hdr->InLength;

            uint32_t outLen = MAILBOX_DATA_MAX;
            uint32_t result = TEEC_ERROR_GENERIC;

            if (inLen <= MAILBOX_DATA_MAX)
            {
                // Copy out of shared memory before use: the peer
                // could rewrite it while we work. Same discipline
                // the TA applies to ITS shared buffer.
                static uint8_t inCopy[MAILBOX_DATA_MAX];
                std::memcpy(inCopy, data, inLen);

                result = tee.Invoke(cmd, inCopy, inLen, data, &outLen);
            }
            else
            {
                outLen = 0;
                result = TEEC_ERROR_BAD_PARAMETERS;
            }

            hdr->OutLength = (result == TEEC_SUCCESS) ? outLen : 0;
            hdr->Result    = static_cast<int32_t>(result);

            // Payload before Ack, always.
            std::atomic_thread_fence(std::memory_order_release);
            hdr->Ack = seq;
```

```
            lastSeq = seq;
        }
    }
    catch (const std::exception& e)
    {
        std::fprintf(stderr, "secteed: %s\n", e.what());
        return 1;
    }
}
```

Running it

```
# ------------------------------------------------------------------
# 1. The shared-memory server. Start it first; both VMs connect.
# ------------------------------------------------------------------
$ ivshmem-server -F -l 1M -n 2 -S /tmp/ivshmem.sock -M sectee-shm
#   -l 1M   one megabyte, matching SHM_SIZE
#   -n 2    two interrupt vectors
#   -F      foreground, so you can see peers connect

# ------------------------------------------------------------------
# 2. QEMU instance B: OP-TEE + Linux, with the TA and the bridge.
#    Start from the standard OP-TEE QEMU v8 build (OP-TEE ch. 4).
# ------------------------------------------------------------------
$ cd ~/optee
$ cp sectee/ta/*.ta out-br/target/lib/optee_armtz/
$ cp sectee/bridge/secteed out-br/target/usr/bin/
$ cd build && make buildroot

# Add the ivshmem device to the OP-TEE QEMU command line. The
# OP-TEE build system generates it, so append via QEMU_EXTRA_ARGS:
$ make run-only QEMU_EXTRA_ARGS="\
    -chardev socket,path=/tmp/ivshmem.sock,id=ivsh \
    -device ivshmem-doorbell,chardev=ivsh,vectors=2"

# In the OP-TEE guest, once it boots:
# modprobe uio_ivshmem 2>/dev/null || modprobe uio_pci_generic
# ls /dev/uio*
/dev/uio0
# secteed /dev/uio0 &
secteed: ready, waiting for the Windows guest

# ------------------------------------------------------------------
# 3. QEMU instance A: Windows 11 ARM64.
# ------------------------------------------------------------------
$ qemu-system-aarch64 \
    -M virt,gic-version=3 -cpu max -smp 4 -m 4G \
    -bios QEMU_EFI.fd \
    -device virtio-gpu-pci \
    -device qemu-xhci -device usb-kbd -device usb-tablet \
    -drive file=windows-arm64.qcow2,if=none,id=hd0,format=qcow2 \
    -device virtio-blk-pci,drive=hd0 \
    -netdev user,id=n0 -device virtio-net-pci,netdev=n0 \
    -chardev socket,path=/tmp/ivshmem.sock,id=ivsh \
    -device ivshmem-doorbell,chardev=ivsh,vectors=2

# Windows is slow under TCG. On an ARM64 host add -accel hvf
# (macOS) or -accel kvm (Linux) and it becomes usable.

# ------------------------------------------------------------------
# 4. In the Windows guest.
# ------------------------------------------------------------------
C:\> bcdedit /set testsigning on   && shutdown /r /t 0

C:\> pnputil /add-driver SecTee.inf /install
Microsoft PnP Utility
Adding driver package:  SecTee.inf
Driver package added successfully.
Published Name:         oem3.inf

C:\> secteectl.exe pubkey
04a91f3c...e7b2
```

```
# And on instance B's SECURE console, at that moment:
I/TA: sectee TA loaded
I/TA: generating a new P-256 keypair

C:\> echo hello from windows > C:\msg.txt
C:\> secteectl.exe sign C:\msg.txt
digest 9f2a7c...41
sig    4b7d2e...91

C:\> secteectl.exe verify C:\msg.txt
VALID

C:\> secteectl.exe counter
0
C:\> secteectl.exe counter inc
1
C:\> secteectl.exe counter inc
2
# Reboot instance B (not Windows) and the counter is still 2:
# it lives in OP-TEE secure storage, not in the Windows guest.
```

Proving it independently

```
# Everything above trusted our own code to check our own code.
# Take the artefacts out and verify with stock OpenSSL, which has
# never heard of OP-TEE, ivshmem or Windows.

# In Windows, capture the public key and a signature:
C:\> secteectl.exe pubkey > C:\pub.hex
C:\> secteectl.exe sign C:\msg.txt > C:\sig.txt

# Copy them out, then on any Linux box:
$ xxd -r -p pub.hex > pub.raw          # 65 bytes: 04 || X || Y
$ printf '3059301306072a8648ce3d020106082a8648ce3d030107034200' \
    | xxd -r -p > spki.der
$ cat spki.der pub.raw > pubkey.der
$ openssl ec -pubin -inform DER -in pubkey.der -out pubkey.pem
read EC key
writing EC key

# The TA emits raw r||s; OpenSSL wants DER. (CNG in the Windows
# driver took the raw form directly - chapter 26.)
$ python3 - "$(cat sig.txt)" <<'EOF' > sig.der
import sys, binascii
raw = binascii.unhexlify(sys.argv[1].strip())
r, s = raw[:32], raw[32:]
def der_int(b):
    b = b.lstrip(b'\x00') or b'\x00'
    if b[0] & 0x80: b = b'\x00' + b
    return b'\x02' + bytes([len(b)]) + b
body = der_int(r) + der_int(s)
sys.stdout.buffer.write(b'\x30' + bytes([len(body)]) + body)
EOF

$ printf 'hello from windows\r\n' > msg.txt   # note CRLF - Windows echo
$ openssl dgst -sha256 -verify pubkey.pem -signature sig.der msg.txt
Verified OK

# THAT is the result. A signature over a digest computed by CNG in
# a Windows ARM64 kernel driver, made by a key generated inside an
# OP-TEE trusted application written in C++, verified by an
# unrelated tool on a third machine.
```

> **THE CRLF TRAP**
>
> `echo hello from windows > C:\msg.txt` writes `hello from windows\r\n`, and `echo` also keeps any trailing space before the `>`. If OpenSSL says `Verification Failure`, compare the digest the driver printed with `sha256sum msg.txt` before suspecting the crypto. Nine times out of ten the bytes differ, not the mathematics.

Extensions worth doing

Exercise	What it teaches
Take the ivshmem MSI-X vector and complete requests from a DPC instead of polling	Interrupts and asynchronous request completion in WDF (chapter 17)
Make the queue parallel and give the mailbox four slots	Concurrency against a fixed-size resource, and OP-TEE's thread limit

Exercise	What it teaches
Add `IOCTL_SECTEE_SIGN_FILE` using `METHOD_OUT_DIRECT` and an MDL	Why buffered I/O stops paying at scale (chapter 18)
Run Driver Verifier with pool tracking and IRQL checking against the driver	Every latent lifetime bug at once (chapter 21)
Register the driver as a CNG key-storage provider (KSP)	How a TEE-backed key becomes usable by CertEnroll, TLS and Windows Hello rather than only by your own client
Have the TA sign the counter alongside the digest	Replay resistance (OP-TEE chapter 17)
Point the driver at a real TrEE provider on Qualcomm silicon	Whether the transport seam was honestly abstract
Run the hostile client from OP-TEE chapter 20 against the TA	Whether the validation is real
Kill `secteed` mid-request and see what the driver does	Timeout handling, and why `STATUS_IO_TIMEOUT` beats hanging for ever

How it shows up

Symptom	Cause
TA will not build: `undefined reference to __cxa_guard_acquire`	A function-local static with a constructor; add `-fno-threadsafe-statics`
TA will not build: `__gxx_personality_v0`	Exceptions still enabled — add `-fno-exceptions`
TA loads then panics immediately	A global object with a constructor that never ran; make it `constexpr` or move it into a function
TA panics on the second request	Double-close — an RAII wrapper and `TEE_CloseAndDeletePersistentObject1` both released the same handle
`TEEC_OpenSession` returns `0xFFFF0008`	The `.ta` is not in `/lib/optee_armtz`, or its filename does not match the UUID
`secteed`: cannot open `/dev/uio0`	No uio driver bound to the ivshmem PCI function; check `lspci -k`
Windows driver: `STATUS_DEVICE_NOT_READY`	`secteed` not started, so the magic was never written
Both sides running, every request times out	The two VMs connected to different `ivshmem-server` sockets, or `-n` too small for the vectors requested
Ack seen but the reply is garbage	`MailboxHeader` layout differs between the two sources — check the `static_assert`
Works once, then never again	Sequence and Ack out of step after an error path that did not write Ack
Signature verifies in CNG, fails in OpenSSL	Raw r‖s not converted to DER
Verification fails about 1 time in 128	ECC coordinates not left-padded to 32 bytes
Counter resets when the OP-TEE guest reboots	Secure storage in `/data/tee` on a volatile rootfs — persist it, or enable RPMB
Everything is unbearably slow	Windows under TCG. Use `-accel kvm` or `-accel hvf` on an ARM64 host

Check yourself

- ✓ Draw the two-VM topology and name what crosses each boundary.
- ✓ State plainly which security property this configuration demonstrates and which it only simulates.

- ✓ Give the single strongest reason to write a TA in C++, with a concrete failure it prevents.
- ✓ Name three C++ features that need a runtime a TA does not have, and the flag that disables each.
- ✓ Explain why the RAII wrapper needs a `Release()` method.
- ✓ Point to the single-fetch copy and the `paramTypes` check in the TA source.
- ✓ Explain the left-padding of ECC coordinates and the failure rate without it.
- ✓ Explain why the bridge publishes the magic value last.
- ✓ Identify both memory barriers in the mailbox protocol and say what each orders.
- ✓ Explain why the bridge copies the input out of shared memory before using it.
- ✓ Verify a signature end to end with stock OpenSSL and say what it proves about the driver, the transport and the TA.
- ✓ Describe what you would change to move this driver onto real Qualcomm silicon.

TRACK

GPU architecture & programming

How a processor built for throughput rather than latency actually works, the graphics and compute pipelines that ride on it, and DX12 in enough depth to draw a triangle and know why every line is there.

20 chapters

What a GPU is and why it exists

A CPU is built to finish one instruction stream as fast as possible. A GPU is built to have so much work outstanding that it never has to wait. Almost everything that seems strange about GPU programming follows from that one difference in objective.

Latency versus throughput

```
THE SAME PROBLEM: memory is ~300 cycles away.

A CPU SOLVES IT BY NOT WAITING
  huge caches                    keep data close
  out-of-order execution         run later instructions now
  branch prediction + speculation
  prefetchers
  deep pipelines, high clocks

  -> most of the transistor budget is spent on CONTROL and
     CACHE. Perhaps 1-5% is arithmetic.

A GPU SOLVES IT BY HAVING SOMETHING ELSE TO DO
  thousands of threads resident at once
  a scheduler that switches between them EVERY CYCLE,
    at zero cost, because every thread's registers stay in
    the register file
  small caches, enormous register files
  in-order, simple cores, modest clocks

  -> most of the budget is ARITHMETIC. Latency is not
     reduced at all; it is HIDDEN behind other work.

THE CONSEQUENCE, and it is the whole chapter:

  a CPU is fast at ONE thing.
  a GPU is fast only if you give it TEN THOUSAND things.

  A GPU running one thread is far slower than a CPU running
  one thread. There is no configuration in which a GPU wins
  on a serial problem.
```

	Modern CPU core	Modern GPU
Cores	8–128	Thousands of ALUs, in dozens of SMs / CUs
Threads in flight	1–2 per core (SMT)	Tens of thousands
Clock	3–5 GHz	1–2.5 GHz
Registers per core	~180 physical	65,536 ×*32-bit* per SM
L1 per core	32–64 KB	~128 KB, shared by many threads
Context switch	~1–5 µs	**Free**, every cycle
Memory bandwidth	50–200 GB/s	500 GB/s – 3 TB/s (HBM)
Branch cost	~15 cycles if mispredicted	Divergence — *both* paths execute (chapter 3)
FP32 throughput	~1 TFLOP/s	20–100 TFLOP/s

THE REGISTER FILE IS THE SURPRISING NUMBER

A GPU SM has *more register storage than L1 cache* — commonly 256 KB of registers against 128 KB of L1. That inversion is the whole latency-hiding trick: every resident thread keeps its full register state live, so switching between them costs nothing. It also explains why "using too many registers" is a real performance problem (chapter 5): registers are the budget that decides how many threads fit.

How it became a general-purpose processor

```
1990s   FIXED FUNCTION
          transform, light, rasterise, texture. All in
          silicon, configured by a few registers. You did
          not program it; you set state.

2001    PROGRAMMABLE SHADERS
          vertex and pixel shaders. Tiny assembly programs,
          no branches, no loops, 128 instructions.

2004    people notice the pixel shader is a parallel
          floating-point machine, and start smuggling
          physics and linear algebra through it as
          textures. "GPGPU" - genuinely by abusing the
          graphics API.

2006    UNIFIED SHADERS. One pool of cores runs vertex,
          pixel and geometry work. The hardware stops
          caring which is which.

2007    CUDA. You may now write C. The graphics API is no
          longer in the way.

2010s   compute shaders in every graphics API; OpenCL;
          GPUs enter supercomputers.

2017    TENSOR CORES. Fixed-function matrix-multiply units
          bolted back on - because the general machine was
          not fast enough for deep learning.

2018    RT CORES. Fixed-function ray-triangle intersection,
          for the same reason.

NOTE THE ARC: fixed function -> programmable -> general ->
fixed function again, for the workloads that matter enough.
That cycle is the most reliable pattern in hardware, and it
is worth expecting the next turn of it.
```

The execution model, in one picture

```
You write a KERNEL: one function, describing what ONE
thread does. You then launch millions of them.

  __global__ void add(float *a, float *b, float *c, int n)
  {
          int i = blockIdx.x * blockDim.x + threadIdx.x;
          if (i < n) c[i] = a[i] + b[i];
  }
  add<<<(n + 255) / 256, 256>>>(a, b, c, n);

THE HIERARCHY (CUDA names / portable names)

  thread          one lane. Has its own registers.
    |
  warp / wavefront    32 (NVIDIA) or 32-64 (AMD) threads
    |                 that execute IN LOCKSTEP. The real
    |                 unit of execution - chapter 2.
    |
  block / workgroup   up to 1024 threads. Share fast
    |                 on-chip memory and can barrier with
    |                 each other. Runs entirely on ONE SM.
    |
  grid / NDRange      all the blocks of one launch. Blocks
                      may run in ANY order, on any SM, and
                      cannot synchronise with each other.

THE RULE THAT SHAPES EVERYTHING:
  threads within a block can cooperate.
  blocks cannot. If your algorithm needs global
  synchronisation, you need a SECOND KERNEL LAUNCH.
```

What it costs to use one

```
/* The numbers that decide whether a GPU is worth it. */

  kernel launch overhead ..............   5-20 us
  PCIe transfer, host to device ........   ~25 GB/s (PCIe 4 x16)
  PCIe latency, one way ................   1-2 us
  device memory bandwidth ..............   500-3000 GB/s
  device memory latency ................   400-800 cycles
  shared memory latency ................   ~20-30 cycles

/* THE ARITHMETIC THAT MATTERS

   Copying 1 MB to the GPU:      ~40 us
   Adding 1 MB of floats there:  ~2 us

   -> the transfer costs twenty times the compute. A kernel
      that is only worth running if the data is ALREADY on the
      device is the normal case, not the exception.

   THE COMPUTE-TO-COMMUNICATION RATIO is the first thing to
   estimate. Arithmetic intensity - FLOPs per byte moved -
   tells you whether you are compute-bound or bandwidth-bound
   before you write a line:

        vector add       0.08 FLOP/byte   hopelessly
                                          bandwidth-bound
        SGEMM (large)    ~50 FLOP/byte    compute-bound
        LLM inference,
          batch size 1   ~2 FLOP/byte     bandwidth-bound,
                                          which is why it is
                                          slow (AI chapter 25)

   Integrated GPUs (phones, Apple Silicon, consoles) share
   DRAM with the CPU, so there is no copy at all - but also no
   HBM, so bandwidth is much lower. Different trade, same
   analysis.                                                  */
```

> **NOT EVERYTHING BELONGS ON A GPU**
>
> Good fits are *uniform* (every element does the same work), *independent* (no ordering between elements), *arithmetic-heavy* relative to bytes moved, and *large* (millions of items, not thousands). Bad fits: pointer chasing, branch-heavy logic, small problems where launch overhead dominates, anything sequentially dependent, and anything needing frequent CPU coordination. A "GPU rewrite" of an unsuitable algorithm typically lands between two and ten times *slower*.

The landscape

	NVIDIA	AMD	ARM Mali / Adreno	Apple
Compute unit	SM	CU / WGP	Shader core	GPU core
Lockstep group	Warp (32)	Wavefront (32 or 64)	Warp (4–16)	SIMD group (32)
Compute API	CUDA	HIP / ROCm	OpenCL, Vulkan	Metal
Graphics API	DX12, Vulkan		Vulkan, GLES	Metal
Memory	Discrete, GDDR/HBM		Unified with CPU	Unified

	NVIDIA	AMD	ARM Mali / Adreno	Apple
Architecture	Immediate mode		Tile-based (chapter 10)	

This track uses **DX12** as the worked API from chapter 11 onward, because it pairs with the Windows track and because Vulkan and Metal are close enough that the concepts transfer directly. Where they differ meaningfully, the chapters say so.

How it shows up

Symptom	Cause
GPU version slower than the CPU one	Transfer cost, launch overhead, or a problem too small
GPU sits at 5% utilisation	Not enough parallelism resident — too few threads
Speed-up far below the FLOPs ratio	Bandwidth-bound; compute throughput was never the limit
Scales to 1000 elements, then stops improving	Launch overhead dominates below a few tens of thousands
Fast on the desktop, slow on the phone	Tile-based architecture, and much lower bandwidth
Profile shows most time in `memcpy`	Data is crossing PCIe every frame; keep it resident
A branch-heavy kernel is 10× slower than expected	Divergence — both sides of the branch run
Works with 256 threads, fails with 1024	Register or shared-memory budget exceeded per block

Check yourself

- ✓ State the difference in objective between a CPU and a GPU in one sentence.
- ✓ Explain how a GPU hides memory latency without reducing it.
- ✓ Say why a GPU SM has more register storage than L1, and what that enables.
- ✓ Trace the arc from fixed-function to programmable and back, with an example of the return.
- ✓ Name the four levels of the execution hierarchy and what can synchronise at each.
- ✓ Explain why global synchronisation requires a second kernel launch.
- ✓ Compare the cost of copying 1 MB to a GPU against computing on it.
- ✓ Define arithmetic intensity and classify vector-add, SGEMM and batch-1 LLM inference.
- ✓ Give four properties of a workload that suits a GPU and four that do not.
- ✓ Name the vendor terms for a compute unit and a lockstep group across three vendors.

SIMT: how a GPU really executes

You write code for one thread. The hardware runs thirty-two of them from a single instruction pointer. That gap between the programming model and the execution model is where GPU performance lives, and pretending it is not there is why most first kernels are slow.

SIMD wearing a disguise

```
SIMD — what the hardware does
  one instruction, one register file of N lanes, N results.
  The programmer sees vectors and writes vector code.

     vaddps  ymm0, ymm1, ymm2      ; 8 floats at once

SIMT — what the GPU shows you
  one instruction, N THREADS each with their own registers
  and their own view of memory. The programmer writes
  SCALAR code for one thread.

     c[i] = a[i] + b[i];           // "one thread"

UNDERNEATH THEY ARE THE SAME MACHINE. The GPU groups your
threads into WARPS of 32 and issues one instruction for the
whole warp, every cycle.

  warp = 32 threads
         ONE program counter
         32 sets of registers
         a 32-bit ACTIVE MASK saying which lanes are live

WHAT SIMT ADDS OVER SIMD

  1. per-lane addressing. Each thread computes its own
     address, so gather and scatter are free at the
     programming level (though not at the memory level —
     chapter 4).
  2. per-lane control flow, via the mask. You may write an
     `if` and it will behave correctly — at a cost
     (chapter 3).

WHAT IT DOES NOT CHANGE

  the 32 threads still share ONE program counter. They are
  not independent. Anything that makes them want to be in
  different places costs you.
```

THE WARP IS THE UNIT OF EVERYTHING

Scheduling, instruction issue, memory coalescing, divergence and most synchronisation all operate on warps, not threads. "How many threads" is almost never the useful question; "how many warps, and are their lanes doing the same thing on adjacent data" is. A block of 100 threads occupies four warps and wastes 28 lanes of the fourth — which is why block sizes are always multiples of 32.

Inside an SM

```
One Streaming Multiprocessor, roughly (NVIDIA Ampere-ish):

+-----------------------------------------------------------+
|  INSTRUCTION CACHE                                        |
+-----------------------------------------------------------+
|  WARP SCHEDULERS  x4                                      |
|    each holds up to 16 resident warps and picks ONE       |
|    ready warp per cycle to issue                          |
+-----------------------------------------------------------+
|  REGISTER FILE   65,536 x 32-bit  =  256 KB               |
|    partitioned among resident warps. THIS is the budget   |
|    that decides how many fit (chapter 5).                 |
+-----------------------------------------------------------+
|  EXECUTION UNITS                                          |
|    32x INT32/FP32 lanes    (the main array)               |
|    16x FP64                (few, on consumer parts)       |
|     4x SFU     sin, cos, rsqrt - slow, low throughput     |
|     4x LD/ST   address generation                         |
|     4x TENSOR  matrix-multiply-accumulate                 |
+-----------------------------------------------------------+
|  128 KB, split between L1 CACHE and SHARED MEMORY         |
|    the split is configurable per kernel                   |
+-----------------------------------------------------------+

THE KEY OBSERVATION: there are 32 FP32 lanes and up to 64
RESIDENT WARPS. The hardware can only execute a few warps
per cycle, but it KEEPS dozens ready. When one stalls on
memory, the scheduler issues from another - in the same
cycle, at no cost, because every warp's registers are
already live in the file.

That is latency hiding, and it is why occupancy matters.
```

One cycle in the life of a scheduler

```
/* A warp scheduler holds N resident warps. Every cycle it
   picks one that is READY and issues its next instruction. */

  cycle 1:  warp 3  -> LDG   R4, [R2]     ; global load, ~500
                                            cycles. warp 3 is
                                            now STALLED.
  cycle 2:  warp 7  -> FMA   R1, R2, R3   ; ready, issue it
  cycle 3:  warp 12 -> LDG   R8, [R6]     ; stalls too
  cycle 4:  warp 7  -> FMA   R5, R1, R4   ; still ready
  cycle 5:  warp 19 -> IADD  R0, R0, 1
  ...
  cycle 500: warp 3's load returns -> warp 3 is READY again

/* WITH ENOUGH WARPS the ALUs never idle, and the 500-cycle
   memory latency is completely invisible.

   WITH TOO FEW WARPS - say 4 resident - all four issue a load
   and then everyone waits. The SM sits idle for hundreds of
   cycles with nothing to do.

   THE ARITHMETIC:
     to hide 500 cycles of latency with an SM that issues 4
     instructions per cycle, you need roughly
         500 x 4 / (instructions between loads)
     warps in flight. For memory-heavy code that is dozens.

   THIS IS WHY "more parallelism" is the answer to almost
   every GPU performance question, and why a kernel with
   perfect arithmetic and no occupancy is slow.            */
```

Independent thread scheduling

```
/* PRE-VOLTA (before 2017): one PC per warp. Threads within a
   warp could NOT make independent progress. This was legal:  */

        /* the "warp-synchronous" idiom - once ubiquitous */
        shared[tid] = value;
        /* no __syncthreads() needed if tid < 32, because the
           warp is in lockstep by construction */
        if (tid < 16) shared[tid] += shared[tid + 16];

/* VOLTA AND LATER: each thread has its own PC and call stack.
   Threads in a warp may diverge and RECONVERGE independently,
   and the hardware may interleave them.

   That broke every warp-synchronous program in existence,
   silently and intermittently.

   THE FIX: state your assumption explicitly. */

        __syncwarp();                    /* barrier the warp    */
        val = __shfl_down_sync(0xffffffff, val, 16);
        /*                     ^^^^^^^^^^ the mask of lanes you
                               are synchronising with. The _sync
                               suffix is mandatory now; the old
                               unsuffixed __shfl is removed.   */

/* WHY THE CHANGE WAS WORTH IT: it makes this correct, which
   it was not before: */

        while (!atomicCAS(&lock, 0, 1)) ;   /* lane 0 wins ...  */
        /* critical section */
        atomicExch(&lock, 0);
/* Pre-Volta, the winning lane could not proceed to release
   the lock while the losing lanes spun - the whole warp
   deadlocked. Per-thread PCs fixed a genuine footgun.       */
```

NEVER ASSUME IMPLICIT LOCKSTEP

If your code relies on threads in a warp being at the same instruction, it is wrong on modern hardware — even though it will usually appear to work. Every intra-warp operation now takes an explicit mask: `__shfl_sync`, `__ballot_sync`, `__any_sync`, `__syncwarp`. In HLSL and Vulkan the equivalents are the wave intrinsics (`WaveActiveSum`, `WaveReadLaneFirst`), which were designed after this lesson and require no mask.

Warp-level operations

```
/* Because a warp shares an instruction stream, the hardware
   can offer cross-lane operations that cost almost nothing -
   no shared memory, no barrier, just wires in the register
   file. They are the single best optimisation available. */

/* CUDA */
  __shfl_sync(mask, var, srcLane)      /* read another lane   */
  __shfl_down_sync(mask, var, delta)   /* shift within warp   */
  __shfl_xor_sync(mask, var, lanemask) /* butterfly exchange  */
  __ballot_sync(mask, pred)            /* 32-bit vote bitmap  */
  __all_sync / __any_sync(mask, pred)  /* reductions on bools */
  __activemask()                       /* who is live now     */
  __reduce_add_sync(mask, val)         /* Ampere+: one insn   */

/* HLSL / DX12 wave intrinsics (SM 6.0+) */
  WaveActiveSum(v)      WaveActiveMax(v)
  WaveActiveBallot(p)   WaveActiveAllTrue(p)
  WaveGetLaneIndex()    WaveReadLaneFirst(v)
  WavePrefixSum(v)

/* A WARP REDUCTION - 5 instructions, no memory, no barrier: */
__device__ float warp_sum(float v)
{
        for (int off = 16; off > 0; off /= 2)
                v += __shfl_down_sync(0xffffffff, v, off);
        return v;      /* lane 0 holds the total              */
}

/* Compare with the shared-memory version: allocate shared
   memory (which costs occupancy), write, __syncthreads(),
   tree-reduce with more barriers, read back. The warp version
   is several times faster and uses no shared memory at all.

   THE PATTERN FOR A FULL BLOCK REDUCTION:
     1. warp_sum within each warp        (no barrier)
     2. lane 0 of each warp writes to shared    (32 floats max)
     3. __syncthreads()                          (one barrier)
     4. the first warp reduces those 32 values with warp_sum
   Two barriers total, whatever the block size.             */
```

How it shows up

Symptom	Cause
Kernel uses 3% of peak FLOPs	Too few resident warps to hide memory latency
Block size 100 performs like block size 128	Warps are 32 wide; the remainder wastes lanes
Code works on Pascal, fails intermittently on Volta+	Warp-synchronous programming without explicit sync
`__shfl` does not compile	The unsuffixed forms were removed; use `_sync`
Deadlock in an intra-warp spinlock on old hardware	Single PC per warp — the winner could not proceed
Reduction is slower than expected	Using shared memory where warp shuffles would do
Occupancy is low despite small blocks	Register pressure — the file is the limit (chapter 5)
Adding one variable halved performance	It pushed register count over a threshold, evicting warps
Wave intrinsics return odd results on AMD	Wavefront is 32 or 64 depending on the part; do not hardcode 32

Check yourself

- ✓ Distinguish SIMD from SIMT and say what SIMT adds.
- ✓ State what a warp shares and what each thread in it owns privately.
- ✓ Explain why block sizes should be multiples of 32.
- ✓ Draw an SM and name the resource that limits how many warps are resident.
- ✓ Walk through several cycles of warp scheduling and explain how latency is hidden.
- ✓ Estimate how many warps are needed to hide a 500-cycle load.
- ✓ Explain what independent thread scheduling changed and which idiom it broke.
- ✓ Say why every intra-warp intrinsic now takes a mask.
- ✓ Write a warp-level sum reduction and say why it beats the shared-memory version.
- ✓ Give the four steps of an efficient block-wide reduction and count its barriers.

Warps, wavefronts and divergence

Thirty-two threads share one program counter. When an `if` sends them different ways the hardware does not pick one — it runs *both* sides, masking off the lanes that should not be there. Understanding exactly when that costs you, and when it does not, is the difference between a fast kernel and a correct one.

Execution masking

```
if (threadIdx.x % 2 == 0)  even_work();
else                       odd_work();

ONE warp, 32 lanes. 16 want each branch.

  mask 11111111111111111111111111111111   all active
     |
     +-- evaluate the condition -> a 32-bit predicate
     |
  mask 10101010101010101010101010101010   even lanes
     run even_work()          <-- 16 lanes idle, burning
                                  the same cycles
     |
  mask 01010101010101010101010101010101   odd lanes
     run odd_work()           <-- the other 16 idle
     |
  mask 11111111111111111111111111111111   reconverge

TIME TAKEN = time(even_work) + time(odd_work)
WORK DONE  = half of it

The branch was not mispredicted. It was not slow. BOTH
SIDES SIMPLY RAN, and half your lanes were switched off for
each. That is divergence.

THE COST IS THE SUM OF ALL TAKEN PATHS. A switch with 8
cases hit by different lanes of one warp costs 8x.
```

DIVERGENCE IS PER-WARP, NOT PER-THREAD

If *every* lane of a warp takes the same branch, there is **no cost at all** — the hardware skips the untaken side entirely. A branch on `blockIdx.x`, or on data that is uniform across 32 consecutive elements, is free. It is only divergence *within* a warp that hurts. So the question is never "does my kernel branch" but "do adjacent threads branch differently".

Divergent and uniform, side by side

```
/* DIVERGENT - alternate lanes disagree. Worst case. */
        if (threadIdx.x % 2 == 0) { ... }

/* UNIFORM - the whole warp agrees. FREE. */
        if (blockIdx.x == 0)      { ... }
        if (N > 1024)             { ... }    /* same for all   */

/* UNIFORM BY CONSTRUCTION - the standard trick: branch on
   something that changes only every 32 threads. */
        if ((threadIdx.x / 32) % 2 == 0) { ... }   /* per-warp  */

/* THE BOUNDS CHECK - technically divergent, and free anyway,
   because only the LAST warp of the grid diverges. */
        int i = blockIdx.x * blockDim.x + threadIdx.x;
        if (i < n) c[i] = a[i] + b[i];

/* DATA-DEPENDENT - the real problem, because you cannot see
   it in the source. */
        if (particle[i].alive) simulate(i);
/* If alive/dead are scattered randomly, every warp diverges
   and you pay 2x for ever. If particles are SORTED by
   liveness, whole warps agree and it is free. Same code, same
   data, ten times the difference.                          */

/* A LOOP WITH A DATA-DEPENDENT TRIP COUNT - divergence that
   lasts as long as the longest lane. */
        while (x > threshold) { x = step(x); }
/* The warp runs until the SLOWEST lane finishes. 31 lanes
   idle while one iterates 400 more times.                  */
```

Removing divergence

```
/* 1. PREDICATION - for tiny branches, the compiler does it
      for you: compute both, select. No branch at all. */
        y = (x > 0) ? f(x) : g(x);
/* becomes: compute f, compute g, select. Costs both, but no
   branch overhead and no reconvergence. Good when both sides
   are one or two instructions; bad when one side is
   expensive - you now always pay for it.                    */

/* 2. SORT THE DATA so lanes agree. Usually the biggest win
      available. */
        /* before: particles in creation order, alive scattered
           after:  partition alive to the front               */
        int n_alive = partition_by_alive(particles);
        simulate<<<grid(n_alive), 256>>>(particles);
/* Now the whole grid is alive and the branch disappears. The
   sort costs a kernel; the divergence cost every frame.    */

/* 3. STREAM COMPACTION - build a dense index list of the
      items that need work, then launch over that. */
        // pass 1: ballot + prefix sum to compact indices
        // pass 2: kernel over the compacted list
/* This is the general form of (2) and the standard answer for
   ray tracing, sparse grids and particle systems.          */

/* 4. SEPARATE KERNELS per case, when the cases are big and
      known in advance. Two uniform kernels beat one divergent
      one, even with two launches.                            */

/* 5. RESTRUCTURE INDEXING so that consecutive lanes handle
      consecutive similar work. Often as simple as swapping
      which dimension maps to threadIdx.x.                    */

/* 6. PERSISTENT THREADS + a work queue. Each warp pulls the
      next item when it finishes, so a long-running lane does
      not stall a whole block. Complex, but the standard answer
      for highly irregular workloads.                         */
```

> **DO NOT FIGHT DIVERGENCE YOU DO NOT HAVE**
>
> A branch that every lane of a warp takes together is free, and branch-free tricks (bit twiddling, arithmetic selects) cost real instructions. Measure the divergence first — Nsight Compute reports *branch efficiency* and *warp execution efficiency* directly. Rewriting a uniform branch into branch-free arithmetic is a pure loss, and it happens constantly.

Reconvergence, and the deadlock it used to cause

```
The hardware must know where diverged lanes rejoin. It
tracks this with a stack of (mask, reconvergence-PC) pairs.

  push  (else-mask, PC-after-if)
  run   the if-body with the if-mask
  pop   -> restore else-mask, jump to else
  run   the else-body
  pop   -> full mask, continue at PC-after-if

PRE-VOLTA this stack was PER WARP, and reconvergence was
forced at the immediate post-dominator. Consequence:

     if (lane == 0) {
             while (atomicCAS(&lock, 0, 1) != 0) ;   /* spin */
             critical();
             atomicExch(&lock, 0);
     }
Lane 0 acquires the lock. The other 31 lanes are masked off
at the SAME PC, waiting to reconverge. Lane 0 proceeds,
releases - fine. But reverse it:

     while (atomicCAS(&lock, 0, 1) != 0) ;
with several lanes competing: one lane wins and wants to
move on; the losers are still in the loop. The warp cannot
be in two places, so the winner is held back at the loop,
which means it never releases the lock. DEADLOCK, in code
that is obviously correct on a CPU.

VOLTA+ gives each thread its own PC, so the winner really
can proceed. That is why the change was made, and why any
intra-warp locking on old hardware is a trap.

The trade: reconvergence is no longer guaranteed at the
post-dominator, so you must call __syncwarp() where you
need it.
```

Measuring it

```
# Nsight Compute - the two metrics that matter.
$ ncu --metrics \
    smsp__thread_inst_executed_per_inst_executed.ratio,\
    smsp__sass_average_branch_targets_threads_uniform.pct \
    ./app

  smsp__thread_inst_executed_per_inst_executed.ratio   18.4
# "average active threads per issued instruction", out of 32.
# 32.0 = perfect. 18.4 means you are wasting ~43% of every
# instruction issue.

  ...branch_targets_threads_uniform.pct                61.2%
# fraction of branches where the whole warp agreed.

# Source-level attribution, which is what you actually want:
$ ncu --set full --import-source yes -o report ./app
$ ncu-ui report.ncu-rep
# the source view colours each line by divergence.

# AMD:
$ rocprof --stats ./app

# DX12 / HLSL: PIX and RenderDoc both show wave occupancy and
# divergence per draw. Nsight Graphics does the same for
# Vulkan and DX12.

# THE CHEAP ESTIMATE, if you have no profiler: comment out the
# divergent branch (take one side unconditionally, accept the
# wrong answer) and time it. The gap is your divergence cost.
```

Warp width is not always 32

Vendor	Width	Note
NVIDIA	32	Stable since 2006; `warpSize` is still a variable, do not hardcode
AMD GCN	64	Divergence costs twice as much per branch
AMD RDNA	32 or 64	Wave32 by default for graphics, wave64 available; the compiler chooses
Intel Xe	8, 16 or 32	Chosen per shader by the compiler — SIMD8 for register-heavy shaders
ARM Mali	4–16	Much narrower, so divergence is far cheaper; older Mali is effectively scalar
Apple	32	SIMD group

Write portable code with `WaveGetLaneCount()` (HLSL), `gl_SubgroupSize` (Vulkan) or `warpSize` (CUDA), and never assume 32 in a loop bound. A reduction hardcoded to five shuffle steps is silently wrong on a 64-wide wavefront.

How it shows up

Symptom	Cause
Kernel is 2× slower than the instruction count suggests	A two-way branch diverging in every warp
Performance depends on input data ordering	Data-dependent divergence — sort or compact

Symptom	Cause
Warp execution efficiency well below 32	Divergence, or a partially-filled last warp
A loop takes as long as its worst element	Divergent trip count; consider persistent threads
Branch-free rewrite made it slower	The branch was uniform and therefore already free
Deadlock in intra-warp locking	Pre-Volta forced reconvergence
Reduction gives wrong results on AMD	Wavefront is 64; the shuffle loop assumed 32
Fast on mobile, slow on desktop for the same branchy code	Mali's narrow warps make divergence cheap
Ray tracer slows as the scene gets complex	Rays in a warp hit different materials — the canonical divergence problem

Check yourself

- ✓ Explain what the hardware does when lanes of a warp take different branches.
- ✓ State the cost rule for a divergent branch, and for a switch with eight cases.
- ✓ Explain why divergence is per-warp and give two branches that are free.
- ✓ Explain why a bounds check `if (i < n)` costs essentially nothing.
- ✓ Give six techniques for removing divergence, and when each applies.
- ✓ Explain why sorting data can be faster than the divergence it removes.
- ✓ Say when predication is a win and when it is a loss.
- ✓ Describe the pre-Volta intra-warp deadlock and what fixed it.
- ✓ Name the two profiler metrics that quantify divergence and what perfect looks like.
- ✓ Give warp widths for three vendors and say what breaks if you hardcode 32.

The GPU memory hierarchy

Most GPU kernels are limited by memory, not arithmetic. The hierarchy has more levels than a CPU's, several of them are explicitly managed by you, and the single most important rule — coalescing — has no CPU equivalent at all.

The levels

```
                                latency   bandwidth   scope
REGISTERS                        ~1 cyc   ~20 TB/s    thread
  256 KB per SM. Private per thread. The fastest storage in
  the machine, and the budget that caps occupancy.
     |
SHARED MEMORY / LDS             ~20-30    ~10 TB/s    block
  up to 164 KB per SM, split with L1. SOFTWARE MANAGED -
  you decide what lives here. Banked (see below).
     |
L1 / TEXTURE CACHE              ~30       ~10 TB/s    SM
  automatic. Small per thread once you divide by occupancy.
     |
L2 CACHE                        ~200      ~2 TB/s     device
  2-100 MB. Shared by all SMs. Where atomics resolve.
     |
GLOBAL MEMORY (VRAM/HBM)        ~400-800  0.5-3 TB/s  device
  GB. Everything else lives here. THE BOTTLENECK.
     |
HOST MEMORY over PCIe           ~1-2 us   ~25 GB/s    system
  100x less bandwidth than VRAM. Avoid crossing it.

PLUS TWO READ-ONLY PATHS:
  CONSTANT MEMORY   64 KB, broadcast-optimised. One value
                    to all lanes in a cycle; serialised if
                    lanes read different addresses.
  TEXTURE/__ldg     read-only path with its own cache,
                    spatial (2D) locality, free filtering
                    and border handling.

THE RATIO THAT MATTERS: registers are ~40x the bandwidth of
global memory. Every byte you keep in registers or shared
memory instead of re-reading is the highest-value
optimisation available.
```

Coalescing: the rule with no CPU analogue

```
Memory is served in 32-byte SECTORS (four per 128-byte
line). A warp issues 32 addresses AT ONCE, and the hardware
merges them into as few transactions as it can.

PERFECT - consecutive lanes, consecutive floats

  lane:   0  1  2  3  ...           31
  addr: 000 004 008 012 ...        124
                                -> ONE 128-byte transaction
                                   128 bytes requested,
                                   128 used. 100%.

STRIDED BY 2 - c[i*2]

  lane:   0    1    2   ...          31
  addr: 000  008  016  ...          248
                                -> TWO transactions
                                   256 fetched, 128 used.
                                   50% of bandwidth wasted.

STRIDED BY 32 - a column of a row-major matrix

  lane:   0     1     2  ...         31
  addr: 000   128   256  ...       3968
                                -> 32 SEPARATE transactions
                                   4096 fetched, 128 used.
                                   3% efficiency.
                                   THIRTY-TWO TIMES SLOWER
                                   than the perfect case,
                                   for the same code.

RANDOM - a gather

  32 transactions, and no prefetcher can help.

THE RULE: adjacent THREADS should touch adjacent MEMORY.
Not adjacent iterations - adjacent threads. Getting this
backwards is the single most common GPU performance bug.
```

```
/* THE CLASSIC MISTAKE - each thread handles a contiguous
   chunk, "like a CPU would". */

__global__ void bad(float *a, int n, int per_thread)
{
        int t = blockIdx.x * blockDim.x + threadIdx.x;
        int base = t * per_thread;                 /* lane 0 -> 0
                                                      lane 1 -> 64
                                                      lane 2 -> 128 */
        for (int i = 0; i < per_thread; i++)
                a[base + i] *= 2.0f;               /* stride 64
                                                      between
                                                      lanes!    */
}

/* THE FIX - grid-stride loop. Lanes stay adjacent; the WHOLE
   GRID strides. */
__global__ void good(float *a, int n)
{
        int stride = blockDim.x * gridDim.x;
        for (int i = blockIdx.x * blockDim.x + threadIdx.x;
             i < n; i += stride)
                a[i] *= 2.0f;              /* lanes 0..31 touch
                                              adjacent floats,
                                              every iteration   */
}
/* Same work, same instruction count, typically 5-20x faster.
   And the grid-stride form is independent of grid size, so
   the same kernel is correct for any launch configuration. */
```

STRUCTURE OF ARRAYS, NOT ARRAY OF STRUCTURES

An AoS layout (`struct Particle { float x, y, z; }; Particle p[N];`) means lane 0 reads offset 0 and lane 1 reads offset 12 — a stride of 12 bytes, so reading just `x` wastes two-thirds of every transaction. SoA (`float px[N], py[N], pz[N];`) makes each field perfectly coalesced. This is the same transformation that helps SIMD on a CPU, but on a GPU the penalty for getting it wrong is far larger.

Shared memory and bank conflicts

```
/* Shared memory is divided into 32 BANKS, 4 bytes wide,
   interleaved. Bank = (address / 4) % 32.

   The 32 lanes of a warp can each access a DIFFERENT bank in
   one cycle. Two lanes hitting the SAME bank at DIFFERENT
   addresses serialise.                                    */

__shared__ float tile[32][32];

/* CONFLICT-FREE - row access. Lane i reads tile[k][i], which
   is bank i. All 32 banks, one cycle.                      */
        float v = tile[k][threadIdx.x];

/* 32-WAY CONFLICT - column access. Lane i reads tile[i][k],
   at address (i*32 + k)*4, so bank = (i*32 + k) % 32 = k for
   EVERY lane. All 32 lanes hit bank k. 32x serialisation. */
        float v = tile[threadIdx.x][k];

/* THE FIX - PAD THE ROW. One extra column shifts each row by
   one bank. */
__shared__ float tile[32][33];      /* <-- 33, not 32          */
        float v = tile[threadIdx.x][k];
        /* address = (i*33 + k)*4, bank = (i*33 + k) % 32
                                        = (i + k) % 32
           every lane a different bank. Conflict-free.        */

/* Four bytes of waste per row buys a 32x speed-up on the
   transpose. It is the best-known trick in GPU programming
   and it is one character.

   BROADCAST IS FREE: if every lane reads the SAME address,
   the hardware broadcasts in one cycle. Same-bank same-address
   is not a conflict; same-bank different-address is.       */
```

Tiling: the reason shared memory exists

```
/* Naive matrix multiply: every thread reads a whole row and a
   whole column from global memory. Each element of A is read
   N times.                                                    */

__global__ void matmul_naive(float *C, const float *A,
                             const float *B, int N)
{
        int r = blockIdx.y * blockDim.y + threadIdx.y;
        int c = blockIdx.x * blockDim.x + threadIdx.x;
        float sum = 0;
        for (int k = 0; k < N; k++)
                sum += A[r * N + k] * B[k * N + c];   /* B is
                                                         strided! */
        C[r * N + c] = sum;
}
/* Global traffic: 2 * N^3 * 4 bytes. Bandwidth-bound, and B's
   access pattern is a column walk. Perhaps 5% of peak.     */

#define T 32
__global__ void matmul_tiled(float *C, const float *A,
                             const float *B, int N)
{
        __shared__ float As[T][T + 1];      /* +1: no conflicts */
        __shared__ float Bs[T][T + 1];

        int r = blockIdx.y * T + threadIdx.y;
        int c = blockIdx.x * T + threadIdx.x;
        float sum = 0;

        for (int t = 0; t < N / T; t++) {
                /* COALESCED loads: adjacent lanes (threadIdx.x)
                   read adjacent floats. */
                As[threadIdx.y][threadIdx.x] =
                        A[r * N + t * T + threadIdx.x];
                Bs[threadIdx.y][threadIdx.x] =
                        B[(t * T + threadIdx.y) * N + c];

                __syncthreads();            /* tile is loaded   */

                for (int k = 0; k < T; k++)
                        sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];

                __syncthreads();            /* before OVERWRITING
                                               the tile - this
                                               second barrier is
                                               the one people
                                               forget           */
        }
        C[r * N + c] = sum;
}
/* Global traffic: 2 * N^3 / T * 4 bytes - THIRTY-TWO TIMES
   LESS. Each element is loaded once per tile instead of once
   per output. Typically 10-20x faster.

   THE PRINCIPLE: load a tile cooperatively into fast memory,
   reuse it many times, move on. It is the same blocking
   transformation as a cache-oblivious CPU algorithm, except
   you write it explicitly.                                  */
```

BOTH `__syncthreads()` ARE LOAD-BEARING

The first ensures the tile is fully written before anyone reads it. The second ensures everyone has finished reading before the next iteration overwrites it. Omitting the second is a classic bug: it works at small sizes where warps happen to stay in step, and corrupts results once the block spans enough warps that they drift apart. Also note `__syncthreads()` must be reached by *every* thread in the block — putting one inside a divergent branch hangs the block.

How it shows up

Symptom	Cause
Kernel achieves 3% of peak bandwidth	Uncoalesced access — check the stride between adjacent lanes
Transpose is 30× slower than a copy	Bank conflicts; pad the shared tile
Reading one field of a struct is slow	AoS layout; switch to SoA
Matrix multiply far below peak	Not tiled — every element re-read from global
Results wrong only for large blocks	Missing the second `__syncthreads()`
Kernel hangs	`__syncthreads()` inside a divergent branch
Occupancy drops when you add shared memory	Shared memory per block limits resident blocks per SM
Constant memory is slow	Lanes reading different addresses — it serialises, it only broadcasts well
Most of the profile is `cudaMemcpy`	Crossing PCIe every iteration; keep data resident
Atomics are the bottleneck	They serialise at L2; use warp aggregation or a per-block partial

Check yourself

- ✓ List the memory levels with their scope and rough latency.
- ✓ State the coalescing rule and explain why stride 32 is thirty-two times worse.
- ✓ Explain why "each thread takes a contiguous chunk" is wrong on a GPU.
- ✓ Write a grid-stride loop and say why it is launch-configuration independent.
- ✓ Explain AoS versus SoA and why the GPU penalty exceeds the CPU one.
- ✓ Compute the bank for a shared-memory address and construct a 32-way conflict.
- ✓ Explain why `[32][33]` removes the conflict.
- ✓ Say when same-bank access is free.
- ✓ Explain what tiling saves in matrix multiply, quantitatively.
- ✓ Justify both `__syncthreads()` calls and describe the bug from omitting the second.

Occupancy and latency hiding

Occupancy is how many warps are resident on an SM as a fraction of the maximum. It is the knob everyone tunes first and the one most often misunderstood: more is not always better, and 100% occupancy is frequently the wrong target.

Three budgets, and the smallest wins

```
An SM has fixed resources. A block consumes some of each.
The number of resident blocks is the MINIMUM across all
limits.

Typical modern SM:
  max warps resident ....... 64      (2048 threads)
  max blocks resident ...... 16-32
  register file ............ 65,536 x 32-bit
  shared memory ............ up to 164 KB

WORKED EXAMPLE
  block = 256 threads = 8 warps
  registers per thread = 40
  shared memory per block = 8 KB

  by warps:      64 / 8            = 8 blocks
  by registers:  65536 / (256*40)  = 6.4  -> 6 blocks
  by shared mem: 164 KB / 8 KB     = 20 blocks
  by block limit:                    16 blocks

  LIMIT = 6 blocks = 48 warps = 48/64 = 75% occupancy
  and REGISTERS are the binding constraint.

NOW ADD ONE VARIABLE that pushes registers to 48:
  by registers: 65536 / (256*48)   = 5.3  -> 5 blocks
  = 40 warps = 62% occupancy

One local variable cost you 13% of your occupancy. This is
why GPU performance is famously sensitive to trivial code
changes, and why "register pressure" is a phrase you will
hear constantly.
```

```
# Find out which budget binds, before guessing.
$ nvcc -Xptxas -v --generate-line-info kernel.cu
ptxas info : Compiling entry function '_Z6kernelPfS_i'
ptxas info : Function properties for _Z6kernelPfS_i
             0 bytes stack frame, 0 bytes spill stores,
             0 bytes spill loads                <-- WATCH THIS
ptxas info : Used 40 registers, 8192 bytes smem, 380 bytes cmem[0]

# Non-zero spill stores/loads means registers overflowed into
# LOCAL memory - which is global memory with a friendly name,
# 400+ cycles away. A spilling kernel is usually far worse
# than a lower-occupancy one that does not spill.

$ ncu --set full ./app          # reports achieved occupancy,
                                # not just theoretical

# Cap registers to raise occupancy - and measure, because it
# may cause spills that cost more than the occupancy gained:
$ nvcc -maxrregcount=32 kernel.cu
  __launch_bounds__(256, 4)     /* 256 threads, want 4 blocks
                                   per SM - lets the compiler
                                   budget registers itself   */
```

How much is enough

```
Occupancy exists to hide latency. Once latency is hidden,
more occupancy buys NOTHING.

THE LITTLE'S LAW VIEW
  needed parallelism = latency x throughput

  to hide a 500-cycle memory latency on an SM that issues
  4 instructions/cycle, you need ~2000 instructions in
  flight. If each warp has 10 independent instructions
  between dependent loads, that is 200 warp-instructions -
  roughly 25 warps.

TYPICAL SHAPE OF THE CURVE

  performance
     |             ....------------------  flat
     |          ..
     |        .
     |      .
     |    .
     |  .
     +------------------------------------ occupancy
     0%   25%   50%   75%   100%

  The knee is usually between 25% and 50%. Past it you are
  tuning a number that does not affect the outcome.

AND THE CURVE CAN GO DOWN:
  * more warps share the same L1 and L2, so each gets less
    cache. A tiled kernel with a working set that fits at
    50% occupancy may thrash at 100%.
  * fewer registers per thread means less data kept close,
    so more memory traffic.

INSTRUCTION-LEVEL PARALLELISM SUBSTITUTES FOR OCCUPANCY.
Volkov's well-known result: a kernel where each thread
processes 4 independent elements can hit peak at 25%
occupancy, because the 4 loads are in flight simultaneously
from one warp. Fewer, fatter threads often beat more,
thinner ones.
```

> **THE RIGHT QUESTION**
> Not "how do I get to 100% occupancy" but "am I latency-bound, bandwidth-bound or compute-bound?" Occupancy only helps the first. If you are already at 80% of peak memory bandwidth, no amount of extra occupancy matters — you need to move fewer bytes (chapter 4). Nsight's Speed-of-Light section answers this in one screen and should be the first thing you look at.

Trading occupancy for work per thread

```
/* ONE ELEMENT PER THREAD - maximum occupancy, minimum ILP.
   Each thread issues one load and then waits.              */
__global__ void v1(float *out, const float *a, const float *b, int n)
{
        int i = blockIdx.x * blockDim.x + threadIdx.x;
        if (i < n) out[i] = a[i] * b[i];
}

/* FOUR ELEMENTS PER THREAD - fewer warps, but four
   INDEPENDENT loads in flight per thread. The scheduler can
   overlap them without needing another warp.               */
__global__ void v4(float4 *out, const float4 *a, const float4 *b, int n4)
{
        int i = blockIdx.x * blockDim.x + threadIdx.x;
        if (i < n4) {
                float4 x = a[i];        /* ONE 16-byte load -
                                           wider transactions,
                                           fewer of them      */
                float4 y = b[i];
                out[i] = make_float4(x.x*y.x, x.y*y.y,
                                     x.z*y.z, x.w*y.w);
        }
}
/* Two wins at once:
     - vectorised loads (float4 = 16 bytes) use the memory
       system better than four separate 4-byte loads
     - more independent work per thread, so lower occupancy
       still hides the latency

   Requires 16-byte alignment. cudaMalloc guarantees 256-byte
   alignment, so the base is fine; an offset into the middle
   of an array may not be.

   THE GENERAL TECHNIQUE is called THREAD COARSENING, and it
   is usually worth trying 2 or 4 elements per thread before
   any other optimisation.                                  */
```

Choosing a block size

Size	Warps	Notes
32	1	Too small — the block limit (16–32 blocks) caps you at 32 warps before any other resource does
128	4	Good default. Divides evenly into most SM limits, flexible for the scheduler
256	8	The most common choice. Good balance of granularity and overhead
512	16	Fine if register use is low; coarse granularity wastes an SM on the tail
1024	32	Maximum. Only one or two blocks fit, so a single barrier stalls half the SM. Rarely best
Not a multiple of 32	—	Always wrong. The final warp runs with idle lanes

```
/* Do not guess - ask the runtime. */
int block, grid;
cudaOccupancyMaxPotentialBlockSize(&grid, &block, my_kernel, 0, 0);
my_kernel<<<grid, block>>>(...);

/* Or query for a specific choice: */
int blocks_per_sm;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocks_per_sm, my_kernel, 256, 0);

/* AND SIZE THE GRID FOR THE MACHINE, not for the data, when
   using a grid-stride loop (chapter 4): */
int nsm;
cudaDeviceGetAttribute(&nsm, cudaDevAttrMultiProcessorCount, 0);
int grid = nsm * blocks_per_sm;     /* fill the GPU exactly    */

/* THE TAIL EFFECT, worth knowing:
     108 SMs, 110 blocks -> the last 2 blocks run alone while
     106 SMs idle. The kernel takes twice as long as the work
     requires. Either round the grid to a multiple of the SM
     count with a grid-stride loop, or use many small blocks
     so the imbalance is proportionally tiny.               */
```

REGISTER SPILLING BEATS OCCUPANCY EVERY TIME

Forcing `-maxrregcount=32` to reach 100% occupancy, and thereby spilling ten registers to local memory, is a large net loss: each spill is a global-memory round trip in the inner loop. Always check the `ptxas` spill counters after changing register limits. Zero spills at 50% occupancy is almost always better than heavy spilling at 100%.

How it shows up

Symptom	Cause
Low achieved occupancy, low performance	Register or shared-memory pressure — check `-Xptxas -v`
Adding a variable halved throughput	Crossed a register threshold, evicting a resident block
Raising occupancy changed nothing	You were bandwidth- or compute-bound, not latency-bound
Raising occupancy made it slower	Cache thrashing, or spills introduced by the register cap
Achieved occupancy far below theoretical	Load imbalance, or the tail effect — some SMs finish early
Kernel time does not scale with grid size	The tail: a partial final wave of blocks
Block size 1024 is slower than 256	Coarse granularity and barrier cost across 32 warps
Non-zero spill stores in the build log	Registers exhausted — reduce live values or raise the cap
Four elements per thread beat one	ILP substituting for occupancy, plus wider transactions

Check yourself

- ✓ Define occupancy and name the four budgets that limit it.
- ✓ Work an example and identify the binding constraint.
- ✓ Explain why one extra variable can cost 13% occupancy.
- ✓ Use Little's law to estimate the warps needed to hide a 500-cycle latency.

- ✓ Draw the occupancy-versus-performance curve and mark the knee.
- ✓ Give two reasons more occupancy can reduce performance.
- ✓ Explain how ILP substitutes for occupancy, with an example.
- ✓ Say why block sizes must be multiples of 32 and why 1024 is rarely best.
- ✓ Describe the tail effect and two ways to avoid it.
- ✓ Explain why zero spills at 50% occupancy usually beats spilling at 100%.

The graphics pipeline, stage by stage

Triangles in, pixels out. Every graphics API is a way of configuring the same fixed sequence of stages, some programmable and some not — and knowing which is which tells you exactly what you can and cannot change.

The pipeline

```
[P] = programmable    [F] = fixed function, configurable

INPUT ASSEMBLER                                          [F]
  reads vertex + index buffers, assembles primitives.
  Decides what a "triangle" is: list, strip, or fan.
     |  vertices
     v
VERTEX SHADER                                            [P]
  runs ONCE PER VERTEX. Transforms object space -> clip
  space. Must output SV_Position. Whatever else you emit is
  interpolated for the pixel shader.
     |
     v
[ HULL / TESSELLATOR / DOMAIN ]              [P][F][P]
  optional. Subdivide patches into more triangles on the
  GPU. Used for terrain and smooth surfaces; less popular
  now that mesh shaders exist.
     |
[ GEOMETRY SHADER ]                                      [P]
  optional, runs per PRIMITIVE, can emit 0..N primitives.
  Famously slow on most hardware - it serialises. Avoid.
     |
     v
CLIPPING + PERSPECTIVE DIVIDE + VIEWPORT                  [F]
  clip to the frustum, divide xyz by w, map to pixels.
     |
     v
RASTERISER                                               [F]
  triangles -> fragments. Also does back-face culling and
  the early-Z test. Generates 2x2 QUADS, always
  (chapter 8).
     |  fragments
     v
PIXEL / FRAGMENT SHADER                                  [P]
  runs ONCE PER FRAGMENT. Computes a colour. This is where
  nearly all your shading cost lives.
     |
     v
OUTPUT MERGER                                            [F]
  depth test, stencil test, blending, write to the render
  target. Blending is fixed-function because it must be
  ORDER-DEPENDENT and atomic per pixel.
     |
     v
RENDER TARGET

NOTE WHAT IS FIXED: rasterisation, clipping, depth test and
blending. These are fixed because they are done billions of
times per frame and because they need guarantees
(ordering, atomicity) that general code cannot give
cheaply. You configure them; you do not write them.
```

> **THE TWO COUNTS THAT DECIDE YOUR FRAME TIME**
>
> Vertex shaders run per *vertex* — perhaps a million per frame. Pixel shaders run per *fragment* — at 4K with any overdraw, tens of millions. A pixel shader is typically 10–100× more invocations than the vertex shader, which is why "move work to the vertex shader" is the oldest optimisation in real-time graphics, and why overdraw matters so much.

The coordinate spaces

```
/* A vertex is transformed through five spaces. Every graphics
   bug that renders "nothing at all" is a matrix in the wrong
   place in this chain.                                        */

  OBJECT / MODEL SPACE    the mesh as the artist made it,
                          origin at its own centre
        | x World matrix          (position, rotate, scale)
        v
  WORLD SPACE             everything in one shared frame
        | x View matrix           (the inverse of the camera's
        v                          transform)
  VIEW / CAMERA SPACE     camera at the origin, looking down
                          -Z (GL) or +Z (D3D)
        | x Projection matrix     (frustum -> a cube)
        v
  CLIP SPACE              homogeneous. -w <= x,y <= w and
                          0 <= z <= w (D3D) or -w <= z <= w (GL)
        | divide by w             (the PERSPECTIVE DIVIDE)
        v
  NDC                     the canonical cube
        | viewport transform
        v
  SCREEN SPACE            pixels, plus a depth in [0,1]

/* The vertex shader's job is object -> clip. Everything after
   is fixed function.                                          */

float4 VSMain(float3 pos : POSITION) : SV_Position
{
        float4 world = mul(float4(pos, 1.0), World);
        float4 view  = mul(world, View);
        return mul(view, Projection);          /* CLIP space   */
        /* Do NOT divide by w yourself - the hardware does it,
           and it also uses w for perspective-correct
           interpolation.                                       */
}

/* D3D vs GL DIFFERENCES that cause upside-down or clipped
   output when porting:
     clip-space Z range   D3D [0,1]      GL [-1,1]
     texture origin       D3D top-left   GL bottom-left
     matrix convention    D3D row-major, row-vector by default
                          GL  column-major, column-vector
   Vulkan follows D3D on Z and GL on Y, which catches
   everyone exactly once.                                      */
```

Perspective-correct interpolation

```
The rasteriser interpolates your vertex outputs across the
triangle. Doing that LINEARLY IN SCREEN SPACE is wrong for
anything in perspective.

A wall receding into the distance, with a checker texture:

  LINEAR interpolation of u:
    the checks stay evenly spaced -> the wall looks like
    it is BENDING. This is the classic PlayStation 1 look;
    that hardware had no perspective correction.

  PERSPECTIVE-CORRECT:
    interpolate u/w and 1/w linearly, then divide:
          u = (u/w interpolated) / (1/w interpolated)
    checks compress with distance. Correct.

The hardware does this for you, per attribute, which is
why the vertex shader must output w and must NOT divide.

YOU CAN TURN IT OFF, and occasionally should:

  noperspective float2 screenUV : TEXCOORD0;
    for screen-space effects, where you want the value to
    vary linearly in pixels.

  nointerpolation uint materialID : TEXCOORD1;
    "flat" - takes the value from the provoking vertex.
    MANDATORY for integers: interpolating an ID gives you
    fractional nonsense.
```

Hidden surface removal

```
/* Drawing a pixel you cannot see costs the same as one you
   can. Four mechanisms remove that work, at different
   stages.                                                  */

/* 1. BACK-FACE CULLING - rasteriser, free.
      Discards triangles facing away, based on winding order.
      Removes ~50% of a closed mesh for nothing.           */
        rasterizerDesc.CullMode = D3D12_CULL_MODE_BACK;
        rasterizerDesc.FrontCounterClockwise = FALSE;
/* If your model renders inside-out, this is the setting. */

/* 2. FRUSTUM CULLING - your code, on the CPU or in a compute
      shader. Do not submit what is off-screen.            */

/* 3. EARLY-Z - rasteriser, before the pixel shader.
      Tests depth BEFORE shading. The single biggest saving
      in a typical scene.

      IT IS DISABLED, silently, if the pixel shader:
        * writes SV_Depth
        * uses `discard` / `clip()`
        * writes to a UAV
      because in each case the hardware cannot know the final
      depth or whether the pixel survives before running the
      shader.

      So a single alpha-test material can cost far more than
      its pixel count suggests: it forces late-Z for those
      draws. If you must discard, mark it:                 */
        [earlydepthstencil]           /* HLSL: "test anyway,
        void PSMain(...) { ... }         I promise it is safe" */

/* 4. HIERARCHICAL-Z - a low-resolution depth pyramid, so
      whole tiles can be rejected at once. Automatic, but it
      is INVALIDATED by depth writes from the pixel shader
      and by changing the depth comparison direction
      mid-frame. Pick one direction and keep it.           */

/* THE STANDARD CONSEQUENCE: sort opaque geometry FRONT TO
   BACK. Near objects fill the depth buffer first, so far
   ones fail early-Z and never shade. Then draw transparent
   geometry BACK TO FRONT, because blending is order
   dependent - and that is why transparency is expensive and
   why it cannot use early-Z at all.                        */
```

OVERDRAW IS MEASURED IN SHADED FRAGMENTS, NOT PIXELS

A 4K frame is 8.3 million pixels. With an average overdraw of 3 — perfectly normal for a scene with foliage, particles and UI — the pixel shader runs 25 million times. Halving the shader cost saves less than eliminating one layer of overdraw. Every profiler has an overdraw visualisation; it is usually the first place to look when a frame is slow and the geometry count is modest.

Mesh shaders: the pipeline being replaced

```
The front of the pipeline is a 1998 design: fixed input
assembly, one vertex in and one vertex out, optional
tessellation stages nobody enjoys.

MESH SHADERS (DX12 Ultimate, Vulkan, Metal) replace
IA + VS + hull/domain/GS with two compute-like stages:

  AMPLIFICATION / TASK SHADER
    runs first, decides HOW MANY mesh shader groups to
    launch. This is where you do GPU-side culling.
     |
  MESH SHADER
    a workgroup cooperatively produces a MESHLET - a small
    batch of vertices and primitives (typically up to 64
    vertices, 126 triangles) - and writes them straight
    into the rasteriser.

WHY IT IS BETTER
  * you control the data layout; no fixed input assembler
  * culling per meshlet, on the GPU, before any vertex work
  * no serialisation bottleneck like the geometry shader
  * one workgroup can share vertex work across triangles

WHY IT IS NOT UNIVERSAL YET
  * needs newer hardware (Turing+, RDNA2+)
  * the asset pipeline must build meshlets offline
  * the classic path still works fine for most content

Worth knowing exists; the rest of this track uses the
classic pipeline because that is what you will meet.
```

How it shows up

Symptom	Cause
Nothing renders at all	Matrix order wrong, or the object is behind the camera or outside the frustum
Model appears inside-out	Winding order versus cull mode
Textures swim or bend on receding surfaces	Interpolation not perspective-correct — usually because `w` was divided too early
Image is upside down after a port	D3D/GL texture origin or clip-space Y
Geometry clipped at the wrong distance	D3D [0,1] versus GL [-1,1] clip Z
An integer varying comes through as garbage	Needs `nointerpolation`
Frame time collapses when foliage appears	`discard` disabled early-Z for those draws
Depth pre-pass made it slower	The scene was not overdraw-bound; you doubled the geometry cost
Transparent objects render in the wrong order	Blending is order-dependent; sort back to front
Geometry shader is unexpectedly slow	It serialises on most hardware — use instancing or mesh shaders

Check yourself

- ✓ List the pipeline stages in order and mark which are programmable.
- ✓ Say why rasterisation and blending are fixed function.

- ✓ Give the ratio of pixel-shader to vertex-shader invocations in a typical frame.
- ✓ Name the five coordinate spaces and the matrix between each pair.
- ✓ Explain why the vertex shader must not divide by `w`.
- ✓ Explain perspective-correct interpolation and what it looks like when absent.
- ✓ Say when to use `noperspective` and when `nointerpolation` is mandatory.
- ✓ Name three things that disable early-Z and why each does.
- ✓ Explain the sort order for opaque and for transparent geometry, and why they differ.
- ✓ Describe what mesh shaders replace and the main advantage they bring.

Shaders and the programmable stages

A shader is a small program run millions of times a frame with no memory of the last invocation. Learning the language takes an afternoon; learning what it may and may not do — and how it gets from source to silicon — is the part that matters.

The languages, and the IRs beneath them

```
HLSL   ---> DXBC        DX11 and earlier
       ---> DXIL        DX12 Shader Model 6+, LLVM-based
       ---> SPIR-V      via DXC, for Vulkan

GLSL   ---> SPIR-V      OpenGL 4.6, Vulkan
MSL    ---> AIR         Metal
WGSL   ---> whatever    WebGPU
Slang  ---> all of them a newer language that targets
                        everything; increasingly common

THE TWO-STAGE COMPILE, and it explains a lot:

  OFFLINE     your HLSL -> DXIL/SPIR-V.  Portable IR, but
              still not machine code. Ships in your app.
  AT RUNTIME  the DRIVER compiles IR -> the actual ISA of
              the installed GPU.

CONSEQUENCES

  * you cannot ship machine code: the ISA differs per
    vendor, per generation, and is often undocumented.
  * the FIRST use of a shader triggers a driver compile,
    which is why games stutter on new effects and why
    "shader pre-caching" screens exist.
  * a driver update can change your shader's performance
    without you rebuilding anything.
  * two GPUs disagreeing about your shader is usually two
    different back-end compilers, not a spec violation.

PIPELINE STATE OBJECTS (chapter 15) exist largely to give
the driver everything it needs to do this compile ONCE, at
a time you choose, instead of mid-frame.
```

```
// HLSL, the whole language you need for a first shader.

cbuffer PerObject : register(b0)      // constant buffer
{
    float4x4 WorldViewProj;
    float4   Tint;
};

Texture2D    Albedo  : register(t0);  // SRV
SamplerState Linear  : register(s0);  // sampler

struct VSIn {
    float3 pos    : POSITION;         // SEMANTICS bind these
    float3 normal : NORMAL;           // to vertex buffer
    float2 uv     : TEXCOORD0;        // elements
};

struct VSOut {
    float4 pos    : SV_Position;      // SV_ = SYSTEM VALUE,
    float3 normal : NORMAL;           // meaningful to the
    float2 uv     : TEXCOORD0;        // fixed-function stages
};

VSOut VSMain(VSIn i)
{
    VSOut o;
    o.pos    = mul(float4(i.pos, 1.0), WorldViewProj);
    o.normal = i.normal;
    o.uv     = i.uv;
    return o;
}

float4 PSMain(VSOut i) : SV_Target       // -> render target 0
{
    float3 n = normalize(i.normal);      // renormalise: linear
                                         // interpolation of
                                         // unit vectors does
                                         // NOT preserve length
    float  ndl = saturate(dot(n, float3(0, 1, 0)));
    float4 base = Albedo.Sample(Linear, i.uv);
    return base * Tint * (ndl * 0.8 + 0.2);
}
```

> **SEMANTICS ARE THE WIRING**
>
> There are no linkers between shader stages. `TEXCOORD0` out of the vertex shader connects to `TEXCOORD0` in the pixel shader *by name*, and the input layout binds `POSITION` to a byte offset in the vertex buffer *by name*. A mismatched or missing semantic is the commonest reason a mesh renders as garbage, and it usually produces no error — just wrong data.

Types and what is actually fast

Type	Cost	Notes
`float`, `float2/3/4`	1 lane-op	The native type. A `float4` is *not* a SIMD vector on modern GPUs — it is four scalar ops
`half` / `min16float`	Often 2× throughput	Real win on mobile and on tensor-heavy work; on desktop often just converted

Type	Cost	Notes
`double`	1/32 to 1/64 rate on consumer parts	Effectively unusable outside compute-focused hardware
`int` / `uint`	~same as float	Integer division and modulo are *very* slow — use bit ops for powers of two
`rcp`, `rsqrt`, `exp2`, `log2`, `sin`	1 SFU op, ~4× slower issue rate	Still cheap. `rsqrt` is faster than `1.0/sqrt()`
`pow(x, y)`	`exp2(y * log2(x))`	Two SFU ops; and it is *wrong* for negative `x`
`normalize`	`rsqrt` + 3 muls	Cheap. Do it in the pixel shader after interpolation
Branching	See chapter 3	Free if the whole wave agrees
Texture sample	~100s of cycles, latency-hidden	The dependent-sample chain is what actually costs you

```
/* Things that are slower than they look. */

  a / b            -> reciprocal + multiply. Fine.
  a % b   (int)    -> genuinely slow. If b is a power of two,
                      use a & (b-1).
  a / b   (int)    -> same. Use a >> log2(b).
  pow(x, 2)        -> use x * x. The compiler often catches
                      this; do not rely on it.
  length(v) < r    -> compare SQUARED lengths and skip the
                      sqrt entirely: dot(v,v) < r*r
  sin/cos of the
    same angle     -> sincos(x, s, c) computes both at once

/* Things that are faster than they look. */

  saturate(x)      -> FREE. It is an output modifier on the
                      preceding instruction, not an operation.
                      Prefer it to clamp(x, 0, 1).
  mad / fma        -> a*b+c is ONE instruction. Write your
                      arithmetic so it factors that way.
  lerp(a, b, t)    -> one mad.
  swizzles         -> free. v.xyzw, v.wwww, v.xz cost nothing.
  abs and negate   -> free input modifiers.

/* The general rule: the ALU is nearly free; MEMORY and
   DIVERGENCE are what cost. Optimising arithmetic in a shader
   that is texture-bound is wasted effort - measure first. */
```

What a shader may not do

```
NO recursion            the stack is finite and tiny.
                        (Ray-tracing shaders are the one
                        exception, with a bounded depth.)

NO function pointers    everything is inlined. There is no
                        call in the final ISA.

NO dynamic allocation   no malloc, no new. All storage is
                        declared up front.

NO pointers, mostly     buffers are indexed, not chased.
                        (DX12 has descriptor indexing, and
                        newer models add buffer pointers.)

NO cross-invocation
  communication         except within a wave (chapter 2) or
                        a compute workgroup via groupshared.
                        Two pixels cannot talk.

NO guaranteed order     pixel shader invocations complete in
                        any order. Blending is ordered by
                        fixed-function hardware, not by you.

LOOPS must be bounded   or the compiler unrolls until it
                        gives up. An unbounded while loop
                        driven by a texture value will
                        either fail to compile or produce a
                        multi-megabyte shader.

[loop]     forces a real loop (smaller code, more
           divergence)
[unroll]   forces unrolling (bigger code, more registers,
           often much faster)
[branch]   real branch
[flatten]  evaluate both sides and select

These attributes matter: a [unroll] on a 4-iteration loop
is usually a clear win, and a missing [loop] on a long one
can blow your register budget and destroy occupancy
(chapter 5).
```

Derivatives, and why pixels come in quads

```
/* The rasteriser ALWAYS shades 2x2 quads, even for a triangle
   covering one pixel. The other three lanes are "helper
   invocations": they run, they compute, their output is
   discarded.

   WHY: mip level selection needs to know how fast the texture
   coordinate is changing across the screen, and the only way
   to know that is to compare with your neighbour.          */

        float2 dx = ddx(uv);       /* difference with the pixel
                                      to the right             */
        float2 dy = ddy(uv);       /* ... and below            */
        /* Texture.Sample() calls these implicitly to pick a
           mip level.                                          */

/* THREE CONSEQUENCES

   1. SMALL TRIANGLES ARE EXPENSIVE. A triangle covering one
      pixel still costs four pixel-shader invocations - 4x
      the shading for the same coverage. This is why dense
      meshes tank performance long before the vertex cost
      matters, and the main problem meshlets and variable-rate
      shading exist to address.

   2. YOU CANNOT SAMPLE WITH IMPLICIT MIPS IN DIVERGENT FLOW.
      If half the quad took the other branch, its derivative
      is undefined. The compiler will either hoist the sample
      out of the branch or warn you. Use SampleLevel() or
      SampleGrad() to supply the mip yourself:             */
        if (someCondition)
                c = tex.Sample(s, uv);         /* UNDEFINED    */
        c = tex.SampleLevel(s, uv, 0);         /* fine anywhere */

   3. DERIVATIVES ARE PER-QUAD, so ddx of a value that varies
      per-pixel is a 2x2 approximation, not a true gradient.
      Screen-space effects built on ddx are blocky at 2x2
      granularity by construction.                         */
```

> **THE DENSE-MESH TRAP**
>
> Quad overshading means a mesh tessellated to sub-pixel triangles can cost *four times* the pixel shading of the same silhouette drawn with larger triangles — while every profiler tells you the pixel shader is the bottleneck, which it is, for a reason that has nothing to do with the shader. Check triangle size before optimising shader code on a highly detailed asset.

How it shows up

Symptom	Cause
Mesh renders as garbage, no errors	Semantic mismatch between input layout and vertex shader
Lighting is subtly wrong across a triangle	Normal not renormalised after interpolation
First frame with a new effect stutters	Driver compiling the shader on demand — pre-warm the PSOs
Works on NVIDIA, artefacts on AMD	Different back-end compilers; often uninitialised values or undefined behaviour you got away with

Symptom	Cause
Shader compile takes minutes	An unrolled loop with a large or unbounded trip count
Occupancy collapses after a small edit	Register pressure from unrolling
Blurry or aliased textures in a branch	Implicit-mip sample inside divergent flow
High-poly model far slower than expected	Quad overshading on sub-pixel triangles
`pow` returns NaN	Negative base — `pow` is `exp2(y*log2(x))`
Integer modulo dominates the profile	Use a bitmask if the divisor is a power of two

Check yourself

- ✓ Describe the two-stage shader compile and name three consequences.
- ✓ Explain what semantics do and why a mismatch produces no error.
- ✓ Say why a `float4` is not a SIMD vector on modern GPUs.
- ✓ Name four operations that are slower than they look and four that are free.
- ✓ Explain why `saturate` costs nothing.
- ✓ List five things a shader may not do.
- ✓ Explain what `[unroll]` and `[loop]` trade against each other.
- ✓ Explain why pixels are shaded in 2×2 quads.
- ✓ Give the three consequences of quad shading, including the one about dense meshes.
- ✓ Say why `Sample` is undefined in divergent flow and what to use instead.

Rasterisation, depth and blending

The fixed-function stages nobody writes and everybody has to configure. Getting the rules wrong here produces cracks between triangles, z-fighting, transparency in the wrong order, and shadow acne — a catalogue of artefacts that all look like bugs in your shader and are not.

Rasterisation is a coverage test

```
For each pixel, is its CENTRE inside the triangle?

     +-------+-------+-------+
     |       |   *   |       |    * = pixel centre
     |   *   |  / \  |   *   |
     +-------+-/---\-+-------+
     |     */|      \|       |
     |     / |   *   |\  *   |
     +----/--+-------+-\-----+

Not "does the triangle overlap the pixel". The CENTRE.
That single rule gives you everything below.

THE TOP-LEFT RULE
  A pixel centre exactly on an edge belongs to exactly one
  triangle: the one for which that edge is a top or left
  edge. Without a tie-break rule, two triangles sharing an
  edge would either both draw that pixel (double-blending
  -> a visible seam in transparent geometry) or neither
  (a crack -> a one-pixel gap you can see through).

  This is why watertight meshes rasterise watertight. It is
  also why "gaps between adjacent triangles" is almost
  never a rasterisation bug and almost always your vertices
  not being bit-for-bit identical: compute a shared vertex
  ONCE and reuse it, do not recompute it per triangle.

SUB-PIXEL PRECISION
  Vertices snap to a fixed-point grid, typically 1/256 of a
  pixel (8 sub-pixel bits). Very large coordinates lose
  precision and wobble; this is one reason to keep world
  coordinates near the origin, or to use camera-relative
  rendering for large worlds.
```

Multisampling

```
/* The trick: decouple COVERAGE from SHADING.

   Supersampling  = shade every sample.  4x SSAA = 4x cost.
   Multisampling  = shade ONCE per pixel, test coverage and
                    depth per SAMPLE, resolve at the end.
                    4x MSAA costs ~1.2x shading and 4x memory
                    bandwidth on the render target.          */

  1 sample                4 samples (4x MSAA)
   +-------+               +-------+
   |       |               |  x  x |    each sample gets its
   |   *   |               |   *   |    own coverage bit and
   |       |               |  x  x |    depth value; all four
   +-------+               +-------+    share ONE shaded colour

/* WHY IT ONLY FIXES GEOMETRIC ALIASING

   MSAA antialiases EDGES, because that is where coverage
   varies. It does nothing for:
     * shader aliasing - a specular highlight that flickers,
       a high-frequency normal map. The shader ran once.
     * alpha-tested foliage - coverage is decided inside the
       shader by `discard`, after MSAA has done its work.
       ALPHA-TO-COVERAGE converts alpha into a coverage mask
       and restores MSAA's usefulness there.

   THIS IS WHY MSAA FELL OUT OF FASHION with deferred
   rendering: a G-buffer stores per-sample data, so MSAA
   multiplies its already-large memory cost, and the lighting
   pass has to run per sample to benefit. Post-process
   antialiasing (FXAA, SMAA) and temporal methods (TAA, DLSS,
   FSR) took over - they are cheaper and they also catch
   shader aliasing, at the cost of blur and ghosting.       */

/* Per-sample shading, when you do want it: */
        float4 PSMain(VSOut i, uint sample : SV_SampleIndex)
        /* declaring SV_SampleIndex forces the shader to run
           PER SAMPLE - i.e. supersampling. Know that you are
           asking for 4x the shading.                        */
```

Depth: precision, and where the artefacts come from

```
Z-FIGHTING: two surfaces at nearly the same depth flicker
between each other as the camera moves. It is a precision
failure, and the cause is almost always the near plane.

THE PROBLEM: the perspective divide makes depth
NON-LINEAR. With a 24-bit depth buffer, near=0.1,
far=1000:

     distance     depth values available
     0.1 - 0.2    ~8,000,000     (half the buffer!)
     1   - 2      ~800,000
     10  - 20     ~80,000
     100 - 200    ~8,000
     900 - 1000   ~200            <-- z-fighting lives here

PRECISION IS PROPORTIONAL TO near/far. Moving the near
plane from 0.1 to 1.0 gains you TEN TIMES the precision
everywhere else. Pushing the far plane out costs you almost
nothing by comparison.

THE FIXES, in order of effectiveness:

1. RAISE THE NEAR PLANE. Free, and usually enough.

2. REVERSED-Z. Map near->1.0, far->0.0, clear to 0, and
   use GREATER_EQUAL. Floating-point has most of its
   precision near 0, and the perspective divide puts most
   depth range near 1 - so reversing them CANCELS OUT.
   With a 32-bit float depth buffer this is close to
   uniform precision across the whole range, for free.
   It is the modern default and there is no reason not to
   use it.

3. A 32-bit float depth buffer instead of 24-bit unorm.

4. Depth bias / polygon offset, for coplanar geometry like
   decals - but this is a patch, not a fix.
```

```
/* Reversed-Z, in DX12. Three changes, no shader edits. */

  /* 1. Build the projection with near and far swapped. */
     XMMatrixPerspectiveFovLH(fov, aspect, farZ, nearZ);

  /* 2. Clear depth to 0 instead of 1. */
     cmd->ClearDepthStencilView(dsv, D3D12_CLEAR_FLAG_DEPTH,
                                0.0f, 0, 0, nullptr);

  /* 3. Reverse the comparison. */
     dsDesc.DepthFunc = D3D12_COMPARISON_FUNC_GREATER_EQUAL;

/* Use a DXGI_FORMAT_D32_FLOAT buffer to get the full benefit.
   Note hierarchical-Z may need a depth-buffer "resummarise"
   if you change the comparison direction mid-frame - so pick
   one direction and stay with it (chapter 6).             */

/* SHADOW ACNE is the same family of problem: a shadow map
   samples depth at a different resolution than the receiving
   surface, so a surface shadows itself.
     depth bias        - shift the comparison; causes PETER
                         PANNING (shadows detach) if too large
     slope-scaled bias - scale the bias by the surface angle;
                         much better, and what you want
     normal offset     - move the sample position along the
                         normal instead of the depth; best
     front-face culling when rendering the shadow map - cheap
                         and effective for closed meshes    */
```

RAISE THE NEAR PLANE BEFORE ANYTHING ELSE

Depth precision is dominated by the ratio near:far, and the relationship is roughly linear in `near`. A team fighting z-fighting with bias tweaks and 32-bit buffers can often fix it entirely by changing `0.01f` to `0.5f` in one line — assuming nothing legitimately gets that close to the camera. Check that number first, every time.

Blending, and why order matters

```
/* Blending is fixed function because it must be atomic per
   pixel and applied in DRAW ORDER. The general form:      */

        result = src * SrcBlend  BlendOp  dst * DestBlend

/* The ones you actually use: */

  OPAQUE            src=ONE  dst=ZERO
                    (or just disable blending)

  ALPHA BLEND       src=SRC_ALPHA  dst=INV_SRC_ALPHA  op=ADD
                    result = s.rgb*s.a + d.rgb*(1-s.a)
                    ORDER DEPENDENT. Must sort back to front.

  PREMULTIPLIED     src=ONE  dst=INV_SRC_ALPHA
                    the colour already has alpha baked in.
                    Better: it composites correctly, filters
                    correctly, and makes "additive" and
                    "alpha" the same blend state with
                    different data. Prefer it.

  ADDITIVE          src=ONE  dst=ONE
                    ORDER INDEPENDENT - addition commutes.
                    Fire, sparks, glow. No sorting needed.

  MULTIPLY          src=DEST_COLOR  dst=ZERO

/* THE TRANSPARENCY PROBLEM

   Alpha blending needs back-to-front order. But:
     * sorting per object fails when objects intersect
     * sorting per triangle is expensive and still fails for
       a single self-intersecting mesh
     * you cannot use early-Z, so overdraw is fully paid

   MITIGATIONS
     depth PRE-PASS then depth-write-off for transparents:
       stops transparent surfaces occluding each other wrongly
     alpha test instead of blend where the edges allow it:
       restores Z, at the cost of hard edges
     WEIGHTED BLENDED OIT: an order-independent approximation
       that is cheap and looks acceptable for particles
     per-pixel linked lists / ROVs: exact, expensive, rarely
       worth it outside CAD

   Transparency remains genuinely unsolved in real-time
   graphics. Every game ships with a compromise.          */
```

> **ALPHA-BLENDED GEOMETRY MUST NOT WRITE DEPTH**
>
> If a transparent surface writes to the depth buffer, later transparent surfaces behind it fail the depth test and vanish — a very common "my particles disappear behind glass" bug. The standard setup is: depth *test* on, depth *write* off, sorted back to front. Opaque geometry writes depth; transparent geometry only reads it.

How it shows up

Symptom	Cause
One-pixel cracks between adjacent triangles	Shared vertices computed twice and not bit-identical
Dark seams in transparent geometry	Double-blending along a shared edge — usually a duplicated vertex
Surfaces flicker between each other	Z-fighting — raise the near plane, or use reversed-Z
Geometry wobbles far from the origin	Sub-pixel precision loss; use camera-relative coordinates
Shadows have stripes on lit surfaces	Shadow acne — use slope-scaled bias or normal offset
Shadows detached from their casters	Depth bias too large — peter panning
MSAA does nothing for foliage	Alpha test decides coverage after MSAA — use alpha-to-coverage
MSAA does nothing for shimmering highlights	Shader aliasing; MSAA only fixes edges
Particles vanish behind glass	Transparent geometry writing depth
Transparent objects pop as the camera moves	Per-object sorting with intersecting geometry
Everything is dark after switching to premultiplied	Textures not converted — the colour must be multiplied by alpha

Check yourself

- ✓ State the coverage rule and explain what the top-left rule prevents.
- ✓ Explain why cracks between triangles are usually a vertex problem, not a rasteriser one.
- ✓ Distinguish supersampling from multisampling and give the cost of each.
- ✓ Name two kinds of aliasing MSAA does not fix, and what does.
- ✓ Explain why depth precision is concentrated near the camera.
- ✓ Explain reversed-Z and why it works, and list the three changes it needs.
- ✓ Say which single parameter most affects z-fighting.
- ✓ Name three shadow-acne mitigations and rank them.
- ✓ Give the blend factors for alpha, premultiplied and additive, and say which is order-independent.
- ✓ Explain why transparent geometry must not write depth.

Textures, samplers and compression

Textures are the largest thing in memory and the largest source of bandwidth in most frames. The hardware gives you filtering, mip selection and decompression for free — provided you set up half a dozen things correctly, each of which has a characteristic artefact when you do not.

What a sample actually does

```
Texture.Sample(sampler, uv) is not a memory read. It is a
fixed-function unit doing five things:

1. ADDRESS      apply the wrap mode to uv
                (wrap / clamp / mirror / border)

2. MIP SELECT   from ddx(uv), ddy(uv) - the derivatives
                from the 2x2 quad (chapter 7) - compute how
                many texels fall in one pixel, and pick the
                level where that ratio is ~1.

3. FETCH        read 4 texels (bilinear) from level N, and
                4 more from level N+1 if trilinear

4. DECOMPRESS   BC/ASTC blocks are decoded in hardware, for
                free, at full rate

5. FILTER       blend the 4 (or 8) samples

A trilinear anisotropic 16x sample can read 128 texels. It
is one instruction in your shader and a great deal of work
in the texture unit - which is why texture-bound is such a
common state, and why the SAMPLER settings matter as much
as the shader.
```

Mipmaps

```
/* A chain of half-size images. Level 0 is full resolution,
   level 1 is half, and so on to 1x1.

   COST: +33% memory.        1 + 1/4 + 1/16 + ... = 4/3
   BENEFIT: enormous, and in two ways.                    */

/* 1. CORRECTNESS. Without mips, a surface at a distance where
      one pixel covers 16 texels samples ONE of those 16 at
      random. As the camera moves, WHICH one changes -> the
      texture crawls and shimmers. This is the single most
      visible aliasing artefact in real-time graphics.

   2. PERFORMANCE. Without mips, adjacent pixels sample texels
      far apart, so the texture cache misses constantly. WITH
      mips, a distant surface reads a small image that fits in
      cache. Mips often make distant geometry SEVERAL TIMES
      faster, not just prettier.

   MIP SELECTION uses the derivatives, which is why:
     - implicit-mip sampling in divergent flow is undefined
       (chapter 7)
     - a shader that computes uv from a branch may pick the
       wrong level and produce a visible seam
     - SampleLevel() with a hardcoded 0 disables all of this,
       reintroducing shimmer AND the cache misses. Use it only
       when you genuinely want level 0.                    */

/* MIP BIAS: nudge the selection. */
        tex.SampleBias(s, uv, -0.5);   /* sharper, more aliased */
        tex.SampleBias(s, uv, +1.0);   /* blurrier, cheaper     */
/* A slight negative bias is standard when using TAA, because
   temporal accumulation restores the detail while suppressing
   the aliasing.                                              */
```

> **GENERATE MIPS OFFLINE, NOT AT LOAD TIME**
>
> A proper offline chain uses a good filter (Kaiser or similar), can be gamma-correct, and handles normal maps specially — renormalising after downsampling, because averaging unit vectors does not produce a unit vector. Runtime `GenerateMips` is a box filter in linear space applied to whatever you give it, which makes normal maps flatten and sRGB textures darken as they recede. It is fine for a prototype and wrong for a product.

Filtering and anisotropy

Filter	Taps	Use for
Point / nearest	1	Pixel art, index/ID lookups, any texture where interpolating between values is meaningless
Bilinear	4	Within one mip level. The baseline
Trilinear	8	Bilinear on two mip levels, blended. Removes the visible band where the mip changes
Anisotropic *N*×	up to 8*N*	Surfaces seen at a glancing angle — floors, roads, terrain. The single best quality-per-cost setting there is

```
/* WHY ANISOTROPIC EXISTS

   A floor stretching to the horizon: one pixel covers a
   footprint that is 1 texel wide and 16 texels long.

   Isotropic mip selection must pick ONE level. It picks based
   on the LONGEST axis, so it chooses a level 16x too blurry -
   and the floor turns to mush in the distance. Choose by the
   shortest axis instead and it shimmers violently.

   Anisotropic filtering takes several samples ALONG the long
   axis at the sharper mip level. Correct in both directions.

   16x anisotropic typically costs a few percent of frame time
   and is the most visible quality setting in any game. Enable
   it.                                                        */

D3D12_SAMPLER_DESC s = {};
s.Filter         = D3D12_FILTER_ANISOTROPIC;
s.MaxAnisotropy  = 16;
s.AddressU       = D3D12_TEXTURE_ADDRESS_MODE_WRAP;
s.AddressV       = D3D12_TEXTURE_ADDRESS_MODE_WRAP;
s.AddressW       = D3D12_TEXTURE_ADDRESS_MODE_WRAP;
s.MipLODBias     = 0.0f;
s.MinLOD         = 0.0f;
s.MaxLOD         = D3D12_FLOAT32_MAX;    /* NOT 0 - a very
                                            common typo that
                                            disables mips
                                            entirely          */

/* COMPARISON SAMPLERS - for shadow maps. The hardware does
   the depth compare AND the bilinear filter in one go, giving
   free 2x2 percentage-closer filtering: */
s.Filter         = D3D12_FILTER_COMPARISON_MIN_MAG_LINEAR_MIP_POINT;
s.ComparisonFunc = D3D12_COMPARISON_FUNC_LESS_EQUAL;
        /* HLSL: shadowMap.SampleCmpLevelZero(cmpSampler, uv, z) */
```

Block compression

```
Textures are stored compressed and decoded IN THE TEXTURE
UNIT, for free, on every sample. This is not a load-time
decompression - the compressed data stays compressed in
VRAM, so it saves memory AND bandwidth AND cache space, all
at once. It is the highest-leverage decision in your asset
pipeline.

Fixed 4x4 blocks, fixed bits per block -> random access.

DESKTOP (BC / DXT / S3TC)
  BC1   0.5 B/texel  RGB + 1-bit alpha   8:1   albedo
  BC3   1.0 B/texel  RGBA               4:1   albedo+alpha
  BC4   0.5 B/texel  single channel     -     height, mask
  BC5   1.0 B/texel  two channels       -     NORMAL MAPS
  BC6H  1.0 B/texel  HDR RGB (float)    -     env maps
  BC7   1.0 B/texel  high-quality RGBA  4:1   everything
                                              modern

MOBILE
  ETC2  0.5-1 B/texel   the Android baseline
  ASTC  0.9-8 bpp       variable block size, best quality
                        per bit available anywhere

THE RULES

* NORMAL MAPS USE BC5, not BC7 or BC3. BC5 stores two
  channels at high precision; reconstruct z in the shader:
      n.xy = tex.Sample(s, uv).rg * 2 - 1;
      n.z  = sqrt(saturate(1 - dot(n.xy, n.xy)));
  Using a three-channel format for a normal map wastes bits
  on a channel you can derive, and BC1's colour-space
  assumptions produce visible blocking on normals.

* sRGB IS PART OF THE FORMAT. Use BC7_UNORM_SRGB for
  albedo so the hardware converts to linear for free on
  every sample. Doing the conversion in the shader is
  slower and wrong for filtering - the hardware filters in
  linear space, which is correct; shader conversion
  filters in sRGB space, which is not.

* NEVER compress data that is not colour. Masks, IDs,
  lookup tables and anything you index with must be
  uncompressed or BC4 - block compression is lossy and
  interpolates across block boundaries.
```

> **COMPRESSING A NORMAL MAP WITH BC1 OR BC7**
>
> The classic asset-pipeline bug. BC1 and BC7 are designed around perceptual colour and will happily introduce blocky artefacts in the X and Y of a normal — which show up as faceted, shimmering lighting on smooth surfaces that looks like a shader bug. BC5 exists for exactly this. If lighting looks blocky and the geometry is smooth, check the normal map's format before anything else.

Memory and streaming

```
/* THE ARITHMETIC that decides whether your game fits in VRAM. */

  4096 x 4096 RGBA8, uncompressed .... 64 MB   (+33% mips = 85)
  4096 x 4096 BC7 ................... 16 MB   (+mips = 21)
  4096 x 4096 BC1 ................... 8 MB    (+mips = 11)

  200 such textures, uncompressed .... 17 GB   impossible
  200 such textures, BC7 ............. 4.2 GB  plausible
  200 such textures, BC7 + streaming . ~1 GB   what ships

/* STREAMING: keep only the mip levels you need resident.

   A texture on a distant object needs only its 64x64 mip -
   16 KB instead of 21 MB. The standard system:
     1. every frame, work out the desired mip per texture,
        either from a feedback buffer or from distance
     2. asynchronously load missing levels
     3. evict levels nobody wants

   DX12 gives you RESERVED (tiled) RESOURCES for this: a
   texture whose pages are mapped individually, so you can
   have a 16K x 16K texture with only the visible tiles
   resident. Vulkan calls them sparse resources.

   SAMPLER FEEDBACK (DX12 Ultimate) makes the GPU tell you
   which tiles were actually sampled, which removes the
   guesswork.                                            */

/* AND THE PRACTICAL ADVICE: measure with a tool, not by
   adding up your assets. */
$ # PIX on Windows: memory view shows per-resource residency
$ # RenderDoc: resource inspector, and the texture viewer
$ #   shows the actual format and mip chain
$ nvidia-smi                      # total VRAM in use
```

How it shows up

Symptom	Cause
Texture crawls and shimmers in the distance	No mipmaps, or `MaxLOD` left at 0
Distant geometry is unexpectedly slow	Same cause — mip-less sampling thrashes the texture cache
Floors and roads look blurry at a glancing angle	Anisotropic filtering off
Visible band where the mip level changes	Bilinear instead of trilinear
Lighting is blocky on smooth surfaces	Normal map compressed with BC1/BC7 instead of BC5
Normal maps look flat after mip generation	Mips not renormalised — generate them offline
Textures darken as they recede	Mips generated in sRGB space rather than linear
Colours are washed out or too dark overall	sRGB format not used; the conversion is missing or doubled
Seams at texture edges	Wrap mode should be clamp, or the atlas has no padding
Mask or ID lookups return wrong values	A non-colour texture was block-compressed, or a linear filter is interpolating IDs
Out of VRAM	Uncompressed textures, or no streaming

Check yourself

- ✓ List the five things a texture sample does in hardware.
- ✓ Give the memory cost of a mip chain and two distinct benefits.
- ✓ Explain how mip level is selected and why that breaks in divergent flow.
- ✓ Explain why mipless sampling is slow as well as ugly.
- ✓ Explain why anisotropic filtering exists, in terms of the pixel footprint.
- ✓ Name the format for normal maps and how to reconstruct the third channel.
- ✓ Explain why sRGB should be part of the texture format rather than a shader conversion.
- ✓ Say which data must never be block-compressed and why.
- ✓ Compute the memory for a 4K RGBA8 texture with mips, and the same in BC7.
- ✓ Describe how texture streaming decides what to keep resident.

Tile-based rendering on mobile GPUs

Every phone, tablet and Apple Silicon GPU renders in a fundamentally different order from a desktop card. It is not a minor variation — it inverts which operations are cheap, and a renderer tuned for one architecture can be several times slower on the other.

Bandwidth is the whole reason

```
A desktop GPU has 500–3000 GB/s of dedicated GDDR/HBM and
a 300 W power budget.

A phone has ~50 GB/s of LPDDR, SHARED with the CPU, and
about 3 W for the whole SoC.

THE PHYSICS: moving a byte off-chip costs roughly 100x the
energy of an arithmetic operation. On a phone, DRAM traffic
IS the power budget, and power is the thermal budget, and
thermal is your sustained frame rate.

IMMEDIATE MODE (desktop) - what it costs

  for each triangle:
          rasterise
          for each fragment:
                  READ depth from DRAM
                  test, shade
                  WRITE depth to DRAM
                  READ colour, blend, WRITE colour

  Every fragment touches off-chip memory several times.
  Fine at 3 TB/s. Impossible at 50 GB/s.

TILE-BASED DEFERRED RENDERING (mobile) - the fix

  PHASE 1 - BINNING
    run ALL vertex shaders for the whole frame.
    sort the resulting primitives into screen TILES
    (typically 16x16 or 32x32 pixels).
    write the per-tile primitive lists to memory.

  PHASE 2 - for each tile, independently:
    the tile's colour+depth live in ON-CHIP TILE MEMORY -
    a few tens of KB, single-cycle access
    rasterise and shade every primitive in that tile
    ... all depth and blending traffic stays ON CHIP ...
    finally, WRITE THE FINISHED TILE to DRAM. Once.

RESULT: the colour buffer is written exactly once per
frame, and the depth buffer need never be written at all.
Bandwidth drops by an order of magnitude.
```

THE CONSEQUENCE THAT MATTERS MOST

On a tiler, **the entire frame's geometry is processed before any pixel is shaded**. Vertex output for the whole frame must be stored to memory and read back — which means vertex data and the number of draw calls cost *more* on mobile, while overdraw, blending and MSAA cost dramatically *less*. Every performance intuition from desktop needs re-examining.

What changes

	Immediate (desktop)	Tiled (mobile)
Overdraw	Expensive — DRAM traffic per layer	**Cheap** — it is all in tile memory. Some tilers also do hidden-surface removal per tile and shade each pixel once
MSAA 4×	4× render-target memory and bandwidth	**Nearly free** — samples live in tile memory and resolve on the way out
Alpha blending	Read-modify-write to DRAM	**Cheap** — on-chip
Vertex count	Cheap	**Expensive** — all vertex output is written to and read from DRAM
Draw calls	Moderate CPU cost	Higher — more binning work
Render-target switches	Cheap	**Very expensive** — flushes every tile
Reading a target you just wrote	Normal	**Forces a full flush**, unless it is the same tile via a subpass
Deferred shading (G-buffer)	The standard technique	Painful — a fat G-buffer defeats the entire point. Use subpasses or forward+
Depth pre-pass	Usually a win	Usually a **loss** — doubles the geometry cost to save overdraw that was already cheap

Load and store actions: the one thing to get right

```
/* Because a tile is loaded, worked on, and stored, you must
   tell the driver what to do at each boundary. Getting this
   wrong is the single biggest mobile performance mistake. */

  AT TILE LOAD (start of a render pass)
    CLEAR      cheapest. No read from DRAM at all - the tile
               is simply initialised on chip.
    LOAD       read the previous contents from DRAM. Costs
               full bandwidth.
    DONT_CARE  cheapest, when you will overwrite everything.

  AT TILE STORE (end of a render pass)
    STORE      write the tile to DRAM.
    DONT_CARE  discard it. FREE.

/* THE DEPTH BUFFER should almost always be:
       load  = CLEAR
       store = DONT_CARE
   You need depth during the pass and never afterwards, so
   there is no reason to spend bandwidth writing it out.
   Many engines write depth to DRAM every frame for no reason
   and lose 20-30% of their bandwidth budget to it.        */

/* Vulkan makes it explicit: */
   VkAttachmentDescription depth = {
        .loadOp         = VK_ATTACHMENT_LOAD_OP_CLEAR,
        .storeOp        = VK_ATTACHMENT_STORE_OP_DONT_CARE,
        .stencilLoadOp  = VK_ATTACHMENT_LOAD_OP_DONT_CARE,
        .stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE,
   };

/* Metal, the same: */
   pass.depthAttachment.loadAction  = MTLLoadActionClear;
   pass.depthAttachment.storeAction = MTLStoreActionDontCare;

/* GLES, via an extension that is easy to forget: */
   glInvalidateFramebuffer(GL_FRAMEBUFFER, 1,
                           (GLenum[]){GL_DEPTH_ATTACHMENT});

/* AND THE ANTI-PATTERN: a partial clear, or a draw that
   covers only part of the screen, forces a LOAD of the whole
   render target so the untouched parts survive. Clear the
   whole thing, or use DONT_CARE and cover it.             */
```

Keeping data on chip

```
/* Deferred shading writes a G-buffer, then reads it back.
   On a tiler that is a full round trip to DRAM and back -
   exactly what tiling exists to avoid.

   THE FIX: tell the API that the second pass reads the same
   TILE, not the same TEXTURE. Then the data never leaves
   chip.                                                    */

/* Vulkan SUBPASSES with input attachments: */
   /* subpass 0: write G-buffer attachments
      subpass 1: read them as INPUT ATTACHMENTS
      dependency: BY_REGION                                 */
   VkSubpassDependency dep = {
        .srcSubpass    = 0,
        .dstSubpass    = 1,
        .srcStageMask  = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT,
        .dstStageMask  = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT,
        .dependencyFlags = VK_DEPENDENCY_BY_REGION_BIT,  /* <-- */
   };
   /* BY_REGION is the promise that pixel (x,y) in subpass 1
      only reads pixel (x,y) from subpass 0. That is what lets
      the driver keep everything in tile memory.            */

   /* In the shader: */
   layout(input_attachment_index = 0, set = 0, binding = 0)
        uniform subpassInput gbufferAlbedo;
   vec4 albedo = subpassLoad(gbufferAlbedo);

/* Metal: programmable blending - just read the colour
   attachment directly in the fragment shader: */
   fragment float4 lighting(GBuffer gbuf [[color(0)]]) { ... }

/* GLES: GL_EXT_shader_framebuffer_fetch */
   gl_LastFragData[0]

/* This is why "just port the desktop deferred renderer" goes
   badly on mobile, and why mobile engines favour
   FORWARD+ / CLUSTERED FORWARD instead: one pass, no
   G-buffer, lights culled per tile on the CPU or in a
   compute pass.                                            */
```

THE RENDER-TARGET PING-PONG

Post-processing chains that bounce between two full-screen targets — blur horizontal, blur vertical, tonemap, bloom — cost one full flush and reload *per pass* on a tiler. Four post passes at 1080p RGBA8 is roughly 66 MB of traffic per frame, at 60 Hz nearly 4 GB/s, on a device with 50 GB/s total. Merge post passes aggressively, work at reduced resolution, and prefer compute with groupshared memory over repeated full-screen draws.

The landscape, and what "deferred" means here

GPU	Style	Note
Imagination PowerVR	TBDR	Invented it. Full hidden-surface removal per tile — opaque overdraw is essentially free
Apple	TBDR	PowerVR lineage; the same on iPhone and on Apple Silicon Macs

GPU	Style	Note
ARM Mali	TBR	Tiled, plus "forward pixel kill" rather than full HSR
Qualcomm Adreno	Hybrid	"FlexRender" — chooses tiled or immediate mode per render pass based on heuristics
NVIDIA (Maxwell+)	Tiled caching	Immediate-mode API semantics, but bins into tiles internally to save bandwidth. Invisible to you
AMD (Vega+)	DSBR	Same idea, also transparent

Note the vocabulary trap: *tile-based **deferred** rendering* is a hardware architecture — deferring *rasterisation* until the geometry is binned. *Deferred shading* is a software technique — deferring *lighting* until after a G-buffer is filled. They are unrelated, and deferred shading is the technique that fits TBDR hardware *worst*.

How it shows up

Symptom	Cause
Desktop renderer runs at a fraction of expected speed on mobile	Bandwidth — usually depth stores, render-target switches, or a G-buffer
Device gets hot and frame rate halves after a minute	Thermal throttling driven by DRAM traffic
Adding a depth pre-pass made it slower	Doubled geometry cost to save overdraw that was already free
MSAA 4× is nearly free	Correct — enable it, it is the best quality-per-cost on mobile
Frame time jumps when a post-process chain is added	One flush and reload per pass
High-poly models much more expensive than on desktop	All vertex output goes through DRAM
A partial-screen clear is slower than a full one	It forces a tile load of the whole target
Deferred renderer performs badly	G-buffer round trip; use subpasses or switch to forward+
Reading the current render target hangs or corrupts	Needs framebuffer fetch or an input attachment, not a texture read
Profiler shows huge "load/store" cost	`storeOp` left as STORE on depth

Check yourself

- ✓ Explain why mobile GPUs tile, in terms of energy per byte.
- ✓ Describe the two phases of tiled rendering and what memory each touches.
- ✓ Say what becomes cheaper on a tiler and what becomes more expensive, with three of each.
- ✓ Explain why a depth pre-pass is often a loss on mobile.
- ✓ Give the correct load and store actions for a depth buffer, and the cost of getting them wrong.
- ✓ Explain why a partial clear can be slower than a full one.
- ✓ Explain what `BY_REGION` promises and what it enables.
- ✓ Say why deferred shading fits TBDR hardware badly, and what to use instead.
- ✓ Distinguish tile-based deferred *rendering* from deferred *shading*.
- ✓ Estimate the bandwidth cost of a four-pass 1080p post chain at 60 Hz.

Why DX12, Vulkan and Metal exist

The explicit APIs are harder to use than the ones they replaced, and that is the point. They exist because the old drivers were doing enormous amounts of guessing on your behalf — badly, on one thread, at unpredictable moments. This chapter is what they took away and what they gave back.

What the old drivers were doing

```
DX11 / OpenGL: you call a function, and the driver decides
everything.

  ctx->PSSetShaderResources(0, 1, &srv);
  ctx->Draw(3, 0);

Behind that Draw the driver must:

  * check every piece of state you set and work out whether
    a new pipeline compile is needed. If so, COMPILE A
    SHADER, right now, mid-frame. -> a stutter you cannot
    predict or control.
  * track the lifetime of every resource you have ever
    bound, to know when it is safe to free or overwrite.
  * insert synchronisation between GPU operations by
    guessing your intent from the order of calls.
  * VALIDATE everything, every call, in the shipping
    driver.
  * do all of this on ONE THREAD, because the context is
    not thread-safe.

THE RESULT
  * a driver thread that is the bottleneck on a 16-core
    machine
  * frame-time spikes with no visible cause
  * per-vendor "optimisations" that recognise specific
    games and substitute different shaders - genuinely, by
    executable name
  * ~1-2 us of CPU per draw call, capping you at a few
    thousand draws per frame

THE EXPLICIT APIs' ANSWER: you already know your intent.
Say it up front, once, and the driver stops guessing.
```

Job	DX11 / GL	DX12 / Vulkan / Metal
Shader compilation	Driver, whenever it decides	**You**, by creating a PSO at load time
Resource lifetime	Driver refcounts everything	**You**, with fences
Synchronisation	Driver infers it	**You**, with barriers
Memory allocation	Driver	**You**, with heaps and placed resources
Command recording	One thread	**Any number of threads**
Validation	Always on	Debug layer only, off in release
Descriptor management	Driver, per binding call	**You**, in heaps/tables
CPU cost per draw	~1–2 µs	~0.1 µs

> **THE TRADE, STATED HONESTLY**
> You get predictable frame times, multi-threaded submission, and roughly ten times the draw calls. You pay with perhaps five times the code for a first triangle, whole new bug classes (a missed barrier is a race between GPU stages), and the obligation to do your own memory management. For a small project or a tool, DX11 or OpenGL is still the right choice. For anything shipping at scale, the explicit APIs won for good reasons.

The five concepts they all share

```
Learn these once; the three APIs differ mainly in spelling.

1. DEVICE and QUEUES
     the device is the GPU. Queues are where work goes -
     typically GRAPHICS, COMPUTE and COPY, which run
     concurrently on separate hardware engines.

2. COMMAND LISTS / BUFFERS
     you RECORD commands into a list, on any thread, and
     SUBMIT it later. Recording touches no GPU state, so
     eight threads can record eight lists at once.
     (chapter 13)

3. PIPELINE STATE OBJECTS
     every piece of state that could affect shader
     compilation - shaders, blend, depth, rasteriser,
     vertex layout, render-target formats - baked into one
     immutable object, created at load time.
     (chapter 15)

4. DESCRIPTORS and DESCRIPTOR HEAPS
     a descriptor is a small opaque struct describing a
     resource. They live in heaps; shaders index into them.
     Binding becomes "point at an offset" instead of "tell
     the driver about this texture".
     (chapter 14)

5. EXPLICIT SYNCHRONISATION
     FENCES     CPU waits for GPU
     BARRIERS   GPU waits for GPU, and transitions a
                resource between usages
     This is the part that is genuinely hard, and the part
     the debug layer will save you on.
     (chapter 16)
```

The three APIs

	DX12	Vulkan	Metal
Platforms	Windows, Xbox	Windows, Linux, Android, Switch (macOS via MoltenVK)	macOS, iOS
Verbosity to first triangle	~600 lines	~1200 lines	~150 lines
Shader language	HLSL → DXIL	GLSL/HLSL → SPIR-V	MSL
Binding model	Root signature + descriptor heaps	Descriptor set layouts + pools	Argument buffers
Memory	Heaps + placed resources, or committed	Explicit, with memory types	Mostly managed for you

	DX12	Vulkan	Metal
Render passes	Optional (`RenderPass` API added later)	Mandatory, and tiler-aware	Mandatory, tiler-aware
Ergonomics	Middle	Hardest	Easiest by a wide margin

This track uses **DX12** from here on. It is the middle of the three in complexity, it pairs with the Windows track, and its concepts map onto Vulkan and Metal almost one-to-one. Where they diverge in a way that matters, the chapters say so.

What multi-threading actually buys

```
/* THE OLD MODEL - one thread does everything. */

  main thread:  cull -> for each object: set state, Draw
                     -> the DRIVER thread then translates all
                        of that into GPU commands, serially

  On a 16-core machine you are using 2 of them, and the
  driver thread is the wall.

/* THE NEW MODEL - record in parallel, submit once. */

  thread 0: record shadow pass       -> command list A
  thread 1: record opaque, part 1    -> command list B
  thread 2: record opaque, part 2    -> command list C
  thread 3: record transparent + UI  -> command list D

  main:     queue->ExecuteCommandLists(4, {A, B, C, D});

/* THE RULES that make it safe:
     * one COMMAND ALLOCATOR per thread per frame-in-flight.
       Allocators are not thread-safe and cannot be reset
       while the GPU is still reading them.
     * command lists are recorded independently but EXECUTE
       IN THE ORDER YOU SUBMIT THEM.
     * resource state transitions must be consistent across
       list boundaries - the debug layer checks this, and it
       is where multi-threaded recording actually goes wrong.

   THE HONEST CAVEAT: parallel recording helps only if you
   are CPU-bound on draw submission. Many titles are GPU-bound
   and gain nothing. Measure which you are before building
   the machinery - and note that the draw-call ceiling rising
   from ~5,000 to ~50,000 changes what art can do, which is
   often the real benefit rather than frame time.          */
```

> **THE DEBUG LAYER IS NOT OPTIONAL**
> The explicit APIs removed validation from the shipping driver, so an error usually produces a device removal, a hang, or corrupted output with no message at all. The debug layer (DX12), validation layers (Vulkan) or Metal's API validation give you precise diagnostics for every mistake in this track. Develop with them on, always; the performance cost is irrelevant during development and they will save you days.

```
// Turn it on before creating the device - order matters.
#if defined(_DEBUG)
    ComPtr<ID3D12Debug1> dbg;
    if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&dbg)))) {
        dbg->EnableDebugLayer();
        dbg->SetEnableGPUBasedValidation(TRUE);   // slower,
                                                  // catches far
                                                  // more
    }
#endif

// And make errors stop the debugger where they happen:
ComPtr<ID3D12InfoQueue> iq;
if (SUCCEEDED(device.As(&iq))) {
    iq->SetBreakOnSeverity(D3D12_MESSAGE_SEVERITY_CORRUPTION, TRUE);
    iq->SetBreakOnSeverity(D3D12_MESSAGE_SEVERITY_ERROR,      TRUE);
}
```

How it shows up

Symptom	Cause
Frame spikes the first time an effect appears	Shader compiled on demand — pre-create the PSOs (chapter 15)
CPU-bound at a few thousand draws in DX11	Driver thread saturated; this is what DX12 fixes
Ported to DX12 and it got slower	Common. Usually over-synchronisation, or per-frame allocations that DX11's driver was pooling for you
Device removed, no message	Debug layer off. Turn it on
Corruption only in release builds	Validation was hiding a missing barrier
Crash resetting a command allocator	The GPU is still executing from it — needs a fence wait
Multi-threaded recording produces wrong output	Inconsistent resource states across command list boundaries
Works on NVIDIA, hangs on AMD	Almost always a missing barrier that one driver tolerated
Parallel recording gained nothing	You were GPU-bound

Check yourself

- ✓ List five things a DX11 driver does behind a single `Draw` call.
- ✓ Explain why mid-frame shader compilation causes unpredictable stutter.
- ✓ Name the five concepts shared by all three explicit APIs.
- ✓ State what moved from the driver to you, and what you get in return.
- ✓ Give the approximate CPU cost per draw call before and after.
- ✓ Explain the rule about command allocators and threads.
- ✓ Say why parallel command recording sometimes buys nothing.
- ✓ Explain why the debug layer matters more than it did in DX11.
- ✓ Give one reason to choose DX11 or OpenGL for a new project.
- ✓ Name the DX12, Vulkan and Metal terms for the binding model.

DX12: adapters, devices and the debug layer

Everything starts with picking a GPU and creating a device. It is thirty lines, and every one of them is a decision that will bite you later if you copy it from a tutorial without reading it — starting with which adapter you got and whether validation is on.

The object graph

```
IDXGIFactory                enumerates adapters, creates
     |                      swap chains
     +-- IDXGIAdapter       one GPU (or the software one)
          |
          +-- ID3D12Device   THE ROOT. Everything is created
               |             from it. Not tied to a window.
               |
               +-- ID3D12CommandQueue      where work is
               |    |                      submitted
               |    +-- IDXGISwapChain     presentation
               |
               +-- ID3D12CommandAllocator  backing memory
               |    +-- ID3D12GraphicsCommandList
               |
               +-- ID3D12DescriptorHeap    (chapter 14)
               +-- ID3D12Resource          buffers, textures
               +-- ID3D12RootSignature     (chapter 15)
               +-- ID3D12PipelineState     (chapter 15)
               +-- ID3D12Fence             (chapter 16)

NOTE what the device is NOT: it has no state. There is no
"current shader" or "current texture" on it. All state
lives in a command list, and is reset every time you reset
the list. That statelessness is what makes multi-threaded
recording possible.
```

Choosing an adapter

```
#include <d3d12.h>
#include <dxgi1_6.h>
#include <wrl/client.h>
using Microsoft::WRL::ComPtr;

#pragma comment(lib, "d3d12.lib")
#pragma comment(lib, "dxgi.lib")

static void ThrowIfFailed(HRESULT hr, const char *what)
{
    if (FAILED(hr))
        throw std::runtime_error(std::string(what) + " failed: 0x"
                                 + std::to_string((unsigned)hr));
}

ComPtr<IDXGIAdapter1> PickAdapter(IDXGIFactory6 *factory)
{
    ComPtr<IDXGIAdapter1> adapter;

    /* EnumAdapterByGpuPreference sorts for you. Without it you
       get adapter 0, which on a laptop is very often the
       INTEGRATED GPU - and then you spend an afternoon
       wondering why performance is a third of expected. */
    for (UINT i = 0;
         factory->EnumAdapterByGpuPreference(
             i, DXGI_GPU_PREFERENCE_HIGH_PERFORMANCE,
             IID_PPV_ARGS(&adapter)) != DXGI_ERROR_NOT_FOUND;
         i++)
    {
        DXGI_ADAPTER_DESC1 desc;
        adapter->GetDesc1(&desc);

        if (desc.Flags & DXGI_ADAPTER_FLAG_SOFTWARE)
            continue;              /* WARP - useful, but not
                                      what you want by default */

        /* Probe WITHOUT creating: pass nullptr for the device
           and it just tells you whether it would succeed. */
        if (SUCCEEDED(D3D12CreateDevice(adapter.Get(),
                                        D3D_FEATURE_LEVEL_12_0,
                                        _uuidof(ID3D12Device),
                                        nullptr)))
        {
            wprintf(L"using %s (%zu MB)\n", desc.Description,
                    desc.DedicatedVideoMemory / (1024 * 1024));
            return adapter;
        }
    }
    throw std::runtime_error("no D3D12-capable adapter");
}
```

> **FEATURE LEVELS ARE NOT THE WHOLE STORY**
>
> `D3D_FEATURE_LEVEL_12_0` is a coarse baseline. The things you actually care about — resource binding tier, shader model, raytracing, mesh shaders, wave intrinsics — are queried separately with `CheckFeatureSupport`. A device can report feature level 12_1 and still not support the shader model your HLSL was compiled for, which fails at PSO creation with a message that does not mention shader models.

```
/* The capabilities worth checking at startup, before you
   depend on them. */

D3D12_FEATURE_DATA_D3D12_OPTIONS opts = {};
device->CheckFeatureSupport(D3D12_FEATURE_D3D12_OPTIONS,
                            &opts, sizeof opts);
/* opts.ResourceBindingTier decides how many descriptors you
   may use per table - Tier 1 is genuinely restrictive, Tier 3
   is effectively unlimited (chapter 14). */

D3D12_FEATURE_DATA_SHADER_MODEL sm = { D3D_SHADER_MODEL_6_6 };
if (FAILED(device->CheckFeatureSupport(D3D12_FEATURE_SHADER_MODEL,
                                       &sm, sizeof sm)))
        sm.HighestShaderModel = D3D_SHADER_MODEL_5_1;
/* NOTE the in/out convention: you pass the HIGHEST you want,
   and it writes back the highest supported. */

D3D12_FEATURE_DATA_D3D12_OPTIONS5 o5 = {};
device->CheckFeatureSupport(D3D12_FEATURE_D3D12_OPTIONS5,
                            &o5, sizeof o5);
bool raytracing = o5.RaytracingTier >=
                  D3D12_RAYTRACING_TIER_1_0;

D3D12_FEATURE_DATA_D3D12_OPTIONS7 o7 = {};
device->CheckFeatureSupport(D3D12_FEATURE_D3D12_OPTIONS7,
                            &o7, sizeof o7);
bool meshShaders = o7.MeshShaderTier != D3D12_MESH_SHADER_TIER_NOT_SUPPORTED;

/* Descriptor sizes are DEVICE-DEPENDENT and must be queried.
   Hardcoding 32 works on your machine and corrupts on
   someone else's (chapter 14). */
UINT rtvSize = device->GetDescriptorHandleIncrementSize(
                   D3D12_DESCRIPTOR_HEAP_TYPE_RTV);
UINT cbvSize = device->GetDescriptorHandleIncrementSize(
                   D3D12_DESCRIPTOR_HEAP_TYPE_CBV_SRV_UAV);
```

The debug layer, properly configured

```
/* MUST be enabled BEFORE the device is created. Enabling it
   afterwards silently does nothing, which is a common and
   frustrating waste of an afternoon. */

void EnableDebugLayer()
{
#if defined(_DEBUG)
    ComPtr<ID3D12Debug1> debug;
    if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&debug))))
    {
        debug->EnableDebugLayer();

        /* GPU-based validation instruments your SHADERS to
           check descriptor and resource-state correctness at
           execution time. It is 10-100x slower and it finds
           an entire class of bug the CPU-side layer cannot -
           notably reading a descriptor that was never
           written. Run it periodically, not always. */
        debug->SetEnableGPUBasedValidation(TRUE);
    }
#endif
}

/* After device creation: break in the debugger AT the error,
   rather than discovering it later. */
void ConfigureInfoQueue(ID3D12Device *device)
{
#if defined(_DEBUG)
    ComPtr<ID3D12InfoQueue> iq;
    if (FAILED(device->QueryInterface(IID_PPV_ARGS(&iq))))
        return;

    iq->SetBreakOnSeverity(D3D12_MESSAGE_SEVERITY_CORRUPTION, TRUE);
    iq->SetBreakOnSeverity(D3D12_MESSAGE_SEVERITY_ERROR,      TRUE);
    iq->SetBreakOnSeverity(D3D12_MESSAGE_SEVERITY_WARNING,    TRUE);

    /* Silence the handful of warnings that are genuinely
       noise, and NOTHING ELSE. Every suppression you add is a
       bug you have chosen not to see. */
    D3D12_MESSAGE_ID deny[] = {
        D3D12_MESSAGE_ID_CLEARRENDERTARGETVIEW_MISMATCHINGCLEARVALUE,
        D3D12_MESSAGE_ID_CLEARDEPTHSTENCILVIEW_MISMATCHINGCLEARVALUE,
    };
    D3D12_INFO_QUEUE_FILTER filter = {};
    filter.DenyList.NumIDs  = _countof(deny);
    filter.DenyList.pIDList = deny;
    iq->PushStorageFilter(&filter);
#endif
}
```

> **DEVICE REMOVAL, AND HOW TO FIND OUT WHY**
>
> `DXGI_ERROR_DEVICE_REMOVED` is DX12's segfault: the GPU hit something it could not recover from — a page fault, a timeout, a bad command. By itself it tells you nothing. **Enable `ID3D12DeviceRemovedExtendedData` (DRED) before device creation** and you get the last executed command and the auto-breadcrumbs leading to it. Without DRED you are guessing; with it you usually know within a minute.

```
/* DRED - set this up at startup, always, including in
   release builds. It costs almost nothing and it is the
   difference between a diagnosable crash and a mystery. */

ComPtr<ID3D12DeviceRemovedExtendedDataSettings> dred;
if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&dred)))) {
    dred->SetAutoBreadcrumbsEnablement(D3D12_DRED_ENABLEMENT_FORCED_ON);
    dred->SetPageFaultEnablement(D3D12_DRED_ENABLEMENT_FORCED_ON);
}

/* Then, when Present or a fence wait returns DEVICE_REMOVED: */
void ReportDeviceRemoval(ID3D12Device *device)
{
    HRESULT reason = device->GetDeviceRemovedReason();
    /* DXGI_ERROR_DEVICE_HUNG    - your commands hung the GPU
       DXGI_ERROR_DEVICE_RESET   - TDR: took longer than ~2 s
       DXGI_ERROR_DRIVER_INTERNAL_ERROR - driver bug, or yours
       DXGI_ERROR_INVALID_CALL   - you did something illegal */

    ComPtr<ID3D12DeviceRemovedExtendedData> data;
    if (FAILED(device->QueryInterface(IID_PPV_ARGS(&data))))
        return;

    D3D12_DRED_AUTO_BREADCRUMBS_OUTPUT crumbs = {};
    if (SUCCEEDED(data->GetAutoBreadcrumbsOutput(&crumbs)))
        for (auto *n = crumbs.pHeadAutoBreadcrumbNode; n;
             n = n->pNext)
            wprintf(L"cmdlist %s: %u of %u ops completed\n",
                    n->pCommandListDebugNameW,
                    *n->pLastBreadcrumbValue,
                    n->BreadcrumbCount);
            /* The op AFTER the last completed one is the one
               that killed you. */

    D3D12_DRED_PAGE_FAULT_OUTPUT pf = {};
    if (SUCCEEDED(data->GetPageFaultAllocationOutput(&pf)))
        wprintf(L"page fault at GPU VA 0x%llx\n",
                pf.PageFaultVA);
        /* and the existing/freed allocation lists tell you
           WHICH resource - usually one you freed too early */
}
```

Name everything

```
/* Every D3D12 object takes a debug name. Without them, PIX,
   RenderDoc, the debug layer and DRED all report
   "ID3D12Resource" and you have no idea which one. */

resource->SetName(L"GBuffer.Albedo");
pso->SetName(L"PSO.OpaqueLit");
cmdList->SetName(L"CmdList.Shadow.Thread2");

/* A macro so it is never skipped: */
#if defined(_DEBUG)
  #define NAME_OBJ(obj) (obj)->SetName(L#obj)
#else
  #define NAME_OBJ(obj) ((void)0)
#endif

/* And mark up your command lists so a capture is readable: */
PIXBeginEvent(cmdList, PIX_COLOR(0,255,0), "Shadow Pass");
    /* ... draws ... */
PIXEndEvent(cmdList);

/* This takes ten minutes to add and saves hours on every
   single debugging session afterwards. Do it on day one, not
   when you first need it.                                    */
```

How it shows up

Symptom	Cause
Performance a third of expected on a laptop	Got the integrated GPU — use `EnumAdapterByGpuPreference`
Debug layer produces no output	Enabled after device creation
PSO creation fails with an unhelpful message	Shader model higher than the device supports
Corruption on another machine only	Hardcoded descriptor increment size
`DEVICE_REMOVED` with no information	DRED not enabled
`DEVICE_RESET` after a heavy frame	TDR — a single submission took over ~2 seconds
Page fault at a GPU address	A resource freed while still referenced by in-flight work
Everything is named `ID3D12Resource` in PIX	No `SetName` calls
Runs in debug, corrupts in release	GPU-based validation was masking a descriptor error
Feature works on your GPU, missing on another	Capability assumed rather than queried

Check yourself

- ✓ Draw the DX12 object graph from the factory down to a pipeline state.
- ✓ Explain why the device has no state, and what that enables.
- ✓ Say why adapter 0 is often the wrong choice and how to avoid it.
- ✓ Explain what a feature level does and does not tell you.
- ✓ Name four capabilities you must query separately.
- ✓ Say why descriptor increment sizes must be queried.

- ✓ Explain when the debug layer must be enabled and what happens otherwise.
- ✓ Say what GPU-based validation adds and what it costs.
- ✓ Explain what DRED gives you after a device removal.
- ✓ Give four causes of `DEVICE_REMOVED` and how to distinguish them.

DX12: command lists, allocators and queues

Nothing you call executes immediately. You record commands into a list, submit the list to a queue, and the GPU gets to it whenever it gets to it. The gap between "I recorded it" and "it ran" is where every DX12 lifetime bug lives.

Three objects, three jobs

```
COMMAND ALLOCATOR
  the MEMORY the commands are written into. Grows as you
  record; never shrinks. You can only RESET it - which frees
  everything at once - and only when the GPU has finished
  executing every list ever recorded from it.

COMMAND LIST
  the RECORDER. Cheap. Reset it to begin recording, Close
  it when done. A list may be reset and reused as soon as
  it is submitted - it does not own the memory.

COMMAND QUEUE
  the SUBMISSION POINT. Lists execute in submission order
  within a queue. Different queues run CONCURRENTLY.

THE LIFETIME RULE, and it is the whole chapter:

  allocator: reset only after the GPU is DONE with it
  list:      reset any time after Close + submit

So you need ONE ALLOCATOR PER FRAME IN FLIGHT, and a fence
to know when it is safe to reuse.

  frame 0 -> allocator A -> submit -> fence value 1
  frame 1 -> allocator B -> submit -> fence value 2
  frame 2 -> wait for fence >= 1, then reuse allocator A

Reset an allocator the GPU is still reading and you get
corruption or a device removal, usually several frames
later and nowhere near the cause.
```

```
/* Creating the three. */

D3D12_COMMAND_QUEUE_DESC qd = {};
qd.Type  = D3D12_COMMAND_LIST_TYPE_DIRECT;
qd.Flags = D3D12_COMMAND_QUEUE_FLAG_NONE;
ComPtr<ID3D12CommandQueue> queue;
device->CreateCommandQueue(&qd, IID_PPV_ARGS(&queue));

/* One allocator per frame in flight. */
static const UINT kFrames = 3;
ComPtr<ID3D12CommandAllocator> alloc[kFrames];
for (UINT i = 0; i < kFrames; i++)
    device->CreateCommandAllocator(
        D3D12_COMMAND_LIST_TYPE_DIRECT, IID_PPV_ARGS(&alloc[i]));

/* One list is enough if you record single-threaded. */
ComPtr<ID3D12GraphicsCommandList> cmd;
device->CreateCommandList(0, D3D12_COMMAND_LIST_TYPE_DIRECT,
                          alloc[0].Get(), nullptr,
                          IID_PPV_ARGS(&cmd));
cmd->Close();     /* lists are created OPEN. Close it so the
                     frame loop's Reset is symmetrical. */
```

The frame loop

```
UINT64 fenceValue[kFrames] = {};
UINT64 nextFence = 1;
UINT   frame = 0;

void RenderFrame()
{
    /* 1. WAIT until the GPU has finished the frame that last
          used this allocator. THIS is what makes reuse safe. */
    if (fence->GetCompletedValue() < fenceValue[frame]) {
        fence->SetEventOnCompletion(fenceValue[frame], fenceEvent);
        WaitForSingleObject(fenceEvent, INFINITE);
    }

    /* 2. Reset the allocator (frees its memory), then the list
          (begins recording). Order matters: allocator first. */
    alloc[frame]->Reset();
    cmd->Reset(alloc[frame].Get(), initialPSO.Get());

    /* 3. Record. A freshly reset list has NO STATE - no
          descriptor heaps, no root signature, no viewport, no
          render targets. You set all of it, every frame. That
          is not overhead; it is what makes lists independent
          and recordable in parallel. */
    cmd->SetGraphicsRootSignature(rootSig.Get());
    ID3D12DescriptorHeap *heaps[] = { srvHeap.Get() };
    cmd->SetDescriptorHeaps(1, heaps);
    cmd->RSSetViewports(1, &viewport);
    cmd->RSSetScissorRects(1, &scissor);

    Transition(cmd.Get(), backBuffer[frame].Get(),
               D3D12_RESOURCE_STATE_PRESENT,
               D3D12_RESOURCE_STATE_RENDER_TARGET);

    cmd->OMSetRenderTargets(1, &rtv[frame], FALSE, &dsv);
    cmd->ClearRenderTargetView(rtv[frame], clearColour, 0, nullptr);
    /* ... draws ... */

    Transition(cmd.Get(), backBuffer[frame].Get(),
               D3D12_RESOURCE_STATE_RENDER_TARGET,
               D3D12_RESOURCE_STATE_PRESENT);

    /* 4. Close, then submit. A list must be Closed before
          ExecuteCommandLists or the runtime rejects it. */
    cmd->Close();
    ID3D12CommandList *lists[] = { cmd.Get() };
    queue->ExecuteCommandLists(1, lists);

    swapChain->Present(1, 0);

    /* 5. Signal a fence AFTER the submission, so we can later
          tell when this frame's work is done. */
    fenceValue[frame] = nextFence;
    queue->Signal(fence.Get(), nextFence++);

    frame = (frame + 1) % kFrames;
}
```

> **FRAMES IN FLIGHT IS A LATENCY-VERSUS-THROUGHPUT DIAL**
> Two frames buffered keeps the GPU fed while the CPU records the next one. Three smooths hitches further but adds a frame of input latency. One means the CPU and GPU alternate, each idle half the time. Two or three is the normal range, and it must match your number of allocators and your swap-chain buffer count — mismatch them and you either stall or corrupt.

Queue types, and why they run concurrently

Type	Can do	Hardware
`DIRECT`	Everything — draw, dispatch, copy	The main graphics engine
`COMPUTE`	Dispatch and copy, no draws	A separate async compute engine on most GPUs
`COPY`	Copy only	A dedicated DMA engine — runs while the GPU renders, and is the right way to stream data
`BUNDLE`	A small pre-recorded set of draws, replayed into a direct list	Not a queue. Rarely worth it on modern drivers

```
/* ASYNC COMPUTE: overlap work that uses different units.

   A shadow pass is geometry- and depth-bound; SSAO is
   ALU-bound. Run them on separate queues and they fill each
   other's gaps - typically 5-15% of frame time on a GPU that
   supports it well.                                        */

  graphics queue:  [ shadow maps ][ gbuffer ][ lighting ]
  compute queue:         [ SSAO ][ particle sim ]
                         ^                     ^
                         |                     |
                   both running at once, on different units

/* SYNCHRONISING BETWEEN QUEUES is done with fences, on the
   GPU - no CPU involvement: */

  computeQueue->ExecuteCommandLists(1, &ssaoList);
  computeQueue->Signal(sharedFence.Get(), 42);

  /* graphics queue WAITS for that, without stalling the CPU */
  graphicsQueue->Wait(sharedFence.Get(), 42);
  graphicsQueue->ExecuteCommandLists(1, &lightingList);

/* THE COPY QUEUE is the one everybody should use and few do.
   Uploading a texture on the direct queue serialises against
   your rendering. On the copy queue it happens in parallel,
   on a DMA engine that is otherwise idle:                  */

  copyQueue->ExecuteCommandLists(1, &uploadList);
  copyQueue->Signal(uploadFence.Get(), n);
  graphicsQueue->Wait(uploadFence.Get(), n);   /* use it after */

/* THE CAVEAT: resources have state (chapter 16), and the copy
   and compute queues support only a SUBSET of states. A
   resource used on a copy queue must be in COMMON state at
   the boundary. Getting this wrong is a classic multi-queue
   bug, and the debug layer catches it.                     */
```

Recording in parallel

```
/* The rules:
     - one ALLOCATOR per thread per frame in flight
     - one LIST per thread (lists are cheap to create)
     - lists EXECUTE in submission order, whatever order they
       were recorded in                                      */

struct ThreadCtx {
    ComPtr<ID3D12CommandAllocator>    alloc[kFrames];
    ComPtr<ID3D12GraphicsCommandList> list;
};
ThreadCtx ctx[kThreads];

void RecordChunk(int t, int frame, int first, int count)
{
    auto *c = &ctx[t];
    c->alloc[frame]->Reset();
    c->list->Reset(c->alloc[frame].Get(), pso.Get());

    /* every list sets up its own state - nothing is inherited */
    c->list->SetGraphicsRootSignature(rootSig.Get());
    c->list->SetDescriptorHeaps(1, heaps);
    c->list->RSSetViewports(1, &viewport);
    c->list->RSSetScissorRects(1, &scissor);
    c->list->OMSetRenderTargets(1, &rtv[frame], FALSE, &dsv);

    for (int i = first; i < first + count; i++)
        DrawObject(c->list.Get(), i);

    c->list->Close();
}

/* Then, on the main thread: */
  parallel_for(0, kThreads, [&](int t) {
      RecordChunk(t, frame, t * per, per);
  });

  ID3D12CommandList *all[kThreads];
  for (int t = 0; t < kThreads; t++) all[t] = ctx[t].list.Get();
  queue->ExecuteCommandLists(kThreads, all);   /* ONE submit   */

/* SUBMIT IN ONE CALL where you can: each
   ExecuteCommandLists has real driver overhead, so twenty
   calls of one list is measurably worse than one call of
   twenty.

   AND NOTE the barriers: resource transitions must be
   consistent across list boundaries. If list A leaves a
   resource in RENDER_TARGET and list B expects PIXEL_SHADER
   _RESOURCE, one of them must do the transition - and the
   runtime cannot check across lists at record time, only at
   execute time.                                            */
```

THE TWO MISTAKES THAT PRODUCE DELAYED, CONFUSING CORRUPTION

Resetting an allocator the GPU is still using. Symptoms appear frames later, in unrelated draws. **Freeing or overwriting a resource that an in-flight command list still references.** Same delay, same confusion. Both are solved the same way: a fence value per frame, and nothing is reused or released until the fence says that frame completed. Build that discipline into your allocator and upload helpers on day one rather than retrofitting it.

How it shows up

Symptom	Cause
Corruption appearing a few frames after a change	Allocator reset while the GPU was still reading it
`DEVICE_REMOVED` with a page fault	A resource released while in-flight work referenced it
`ExecuteCommandLists` rejects a list	It was not `Close`d
Nothing renders after adding a second list	The new list did not set its own render targets or root signature — no state is inherited
CPU and GPU each idle half the time	Only one frame in flight
Input latency high but frame rate fine	Too many frames buffered
Async compute gives no speed-up	Over-synchronised, or the two passes use the same units
Debug layer error about resource state on the copy queue	Copy queues support only a subset of states — return to COMMON at the boundary
Parallel recording is slower than single-threaded	Too many `ExecuteCommandLists` calls, or lists too small to amortise setup
Random hangs under load	A fence waited on a value that is never signalled

Check yourself

- ✓ Say what each of allocator, list and queue owns.
- ✓ State the reset rule for allocators and for lists, and why they differ.
- ✓ Explain why you need one allocator per frame in flight.
- ✓ Walk through the five steps of the frame loop and say what each guards against.
- ✓ Explain why a freshly reset command list has no state, and why that is a feature.
- ✓ Name the queue types and what each can do.
- ✓ Explain what async compute overlaps and give a realistic gain.
- ✓ Show how two queues synchronise without involving the CPU.
- ✓ Give the rules for multi-threaded recording, including the allocator rule.
- ✓ Explain why submitting twenty lists in one call beats twenty calls.

DX12: resources, heaps and descriptors

DX11 had "create a texture". DX12 splits that into three separate decisions — where the memory lives, what the resource is, and how a shader finds it — and makes you take all three. Understanding the split is most of what makes DX12 feel large.

Memory, resource, descriptor

```
1. A HEAP is memory. It has a TYPE that decides who can
   see it and how fast.

     DEFAULT   GPU-local VRAM. Fast for the GPU, the CPU
               cannot touch it. Where everything you draw
               from lives.
     UPLOAD    CPU-writable, GPU-readable. Write-combined:
               writing is fast, READING IS CATASTROPHIC
               (uncached, ~1 byte at a time). Staging
               buffers and per-frame constants.
     READBACK  GPU-writable, CPU-readable. For getting
               results back - screenshots, queries.
     CUSTOM    you specify the pool and CPU page property.

2. A RESOURCE is a typed view of some of that memory: a
   buffer, a 2D texture, a texture array. It has a format,
   dimensions, and a STATE (chapter 16).

3. A DESCRIPTOR is a small opaque struct - 32 to 64 bytes,
   device-dependent - that tells a shader HOW to read a
   resource. It lives in a DESCRIPTOR HEAP.

THE SAME MEMORY CAN HAVE MANY DESCRIPTORS:
     one texture, viewed as
       SRV  R8G8B8A8_UNORM_SRGB   for sampling
       UAV  R8G8B8A8_UNORM        for compute writes
       RTV  R8G8B8A8_UNORM        as a render target
     Three descriptors, one resource, one allocation.
```

```
/* COMMITTED - a resource with its own implicit heap. Simple,
   and what you use until you have a reason not to. */

D3D12_HEAP_PROPERTIES hp = { D3D12_HEAP_TYPE_DEFAULT };
D3D12_RESOURCE_DESC rd = {};
rd.Dimension        = D3D12_RESOURCE_DIMENSION_TEXTURE2D;
rd.Width            = 1024;
rd.Height           = 1024;
rd.DepthOrArraySize = 1;
rd.MipLevels        = 0;              /* 0 = full chain       */
rd.Format           = DXGI_FORMAT_R8G8B8A8_UNORM;
rd.SampleDesc.Count = 1;
rd.Layout           = D3D12_TEXTURE_LAYOUT_UNKNOWN;
rd.Flags            = D3D12_RESOURCE_FLAG_NONE;

ComPtr<ID3D12Resource> tex;
device->CreateCommittedResource(
    &hp, D3D12_HEAP_FLAG_NONE, &rd,
    D3D12_RESOURCE_STATE_COPY_DEST,   /* initial state        */
    nullptr, IID_PPV_ARGS(&tex));

/* PLACED - you allocate one big heap and sub-allocate. This
   is what a real engine does: allocation is expensive
   (milliseconds, and it can fail), so you make a few large
   heaps at startup and manage offsets yourself. */

ComPtr<ID3D12Heap> heap;
D3D12_HEAP_DESC hd = {};
hd.SizeInBytes = 256ull * 1024 * 1024;
hd.Properties  = { D3D12_HEAP_TYPE_DEFAULT };
hd.Alignment   = D3D12_DEFAULT_RESOURCE_PLACEMENT_ALIGNMENT;
hd.Flags       = D3D12_HEAP_FLAG_ALLOW_ONLY_NON_RT_DS_TEXTURES;
device->CreateHeap(&hd, IID_PPV_ARGS(&heap));

device->CreatePlacedResource(heap.Get(), offset, &rd,
                             D3D12_RESOURCE_STATE_COPY_DEST,
                             nullptr, IID_PPV_ARGS(&tex));
/* Offsets must respect GetResourceAllocationInfo()'s
   alignment - 64 KB usually, 4 MB for MSAA targets. */

/* Unless you enjoy writing allocators, use D3D12MemoryAllocator
   (D3D12MA) - the same library as Vulkan's VMA, and it handles
   placed resources, pools and defragmentation properly. */
```

NEVER READ FROM AN UPLOAD HEAP

Upload memory is write-combined: writes are buffered and flushed efficiently, but reads bypass the cache entirely and go across PCIe one access at a time. A single innocent-looking `+=` on a mapped upload buffer can cost *hundreds of times* what you expect. Build your data in ordinary memory and `memcpy` it in one pass, and never use a mapped pointer as scratch space.

Getting data onto the GPU

```
/* DEFAULT heap memory is not CPU-visible, so everything goes
   via an UPLOAD staging buffer and a copy. */

void UploadBuffer(ID3D12Device *dev, ID3D12GraphicsCommandList *cmd,
                  ID3D12Resource *dst, const void *src, size_t n,
                  ComPtr<ID3D12Resource> &keepAlive)
{
    /* 1. staging buffer in UPLOAD */
    D3D12_HEAP_PROPERTIES up = { D3D12_HEAP_TYPE_UPLOAD };
    D3D12_RESOURCE_DESC bd = {};
    bd.Dimension        = D3D12_RESOURCE_DIMENSION_BUFFER;
    bd.Width            = n;
    bd.Height           = 1;
    bd.DepthOrArraySize = 1;
    bd.MipLevels        = 1;
    bd.Format           = DXGI_FORMAT_UNKNOWN;
    bd.SampleDesc.Count = 1;
    bd.Layout           = D3D12_TEXTURE_LAYOUT_ROW_MAJOR;

    dev->CreateCommittedResource(&up, D3D12_HEAP_FLAG_NONE, &bd,
        D3D12_RESOURCE_STATE_GENERIC_READ, nullptr,
        IID_PPV_ARGS(&keepAlive));
    /* keepAlive matters: the staging buffer must OUTLIVE the
       GPU's execution of the copy. Releasing it at the end of
       this function is a use-after-free the GPU discovers
       later (chapter 13). */

    /* 2. write it - one memcpy, no reads */
    void *p = nullptr;
    D3D12_RANGE none = { 0, 0 };        /* we read nothing */
    keepAlive->Map(0, &none, &p);
    memcpy(p, src, n);
    keepAlive->Unmap(0, nullptr);
    /* Map/Unmap are cheap and may be left mapped permanently
       for per-frame buffers - "persistent mapping". */

    /* 3. copy on the GPU */
    cmd->CopyBufferRegion(dst, 0, keepAlive.Get(), 0, n);

    /* 4. transition to its use state (chapter 16) */
    D3D12_RESOURCE_BARRIER b = {};
    b.Type  = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    b.Transition.pResource   = dst;
    b.Transition.StateBefore = D3D12_RESOURCE_STATE_COPY_DEST;
    b.Transition.StateAfter  =
        D3D12_RESOURCE_STATE_VERTEX_AND_CONSTANT_BUFFER;
    b.Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
    cmd->ResourceBarrier(1, &b);
}

/* TEXTURES are harder: the GPU wants a swizzled, tiled layout
   with per-row alignment, so you cannot memcpy an image
   directly. Use the helper: */
    D3D12_SUBRESOURCE_DATA sub = { pixels, rowPitch, slicePitch };
    UpdateSubresources(cmd, dstTex, staging, 0, 0, 1, &sub);
/* It queries the required footprint - rows padded to 256 bytes
   - copies row by row into the staging buffer, and issues
   CopyTextureRegion. Doing this by hand is possible and
   almost never worth it. */
```

Descriptors and heaps

```
FOUR HEAP TYPES, and only two are shader-visible.

  CBV_SRV_UAV   constant buffers, textures, UAVs.
                SHADER-VISIBLE. Usually one big heap.
  SAMPLER       SHADER-VISIBLE, separate heap, max 2048.
  RTV           render target views. CPU-only.
  DSV           depth stencil views. CPU-only.

YOU MAY ONLY HAVE ONE SHADER-VISIBLE HEAP OF EACH TYPE
BOUND AT A TIME. That is the constraint that shapes every
DX12 binding design: you cannot bind a small heap per
object. You make ONE large heap and put everything in it.

SetDescriptorHeaps is also EXPENSIVE - it can flush the
GPU on some hardware. Call it once per command list, not
per draw.

THE TWO HANDLE TYPES, and confusing them is a classic bug:

  D3D12_CPU_DESCRIPTOR_HANDLE   for CREATING descriptors
                                (CreateShaderResourceView)
  D3D12_GPU_DESCRIPTOR_HANDLE   for BINDING them
                                (SetGraphicsRootDescriptorTable)

A CPU-only heap has no GPU handles at all. A shader-visible
heap has both, and you index them in parallel.

AND THE INCREMENT SIZE IS DEVICE-DEPENDENT:
  UINT n = device->GetDescriptorHandleIncrementSize(type);
  handle.ptr += index * n;
Hardcoding it works on your GPU and corrupts on another.
```

```
/* A minimal descriptor allocator - a bump pointer over one
   big heap. This is close to what real engines do for static
   descriptors. */

class DescriptorHeap {
public:
    void Init(ID3D12Device *dev, D3D12_DESCRIPTOR_HEAP_TYPE type,
              UINT count, bool shaderVisible)
    {
        D3D12_DESCRIPTOR_HEAP_DESC d = {};
        d.Type           = type;
        d.NumDescriptors = count;
        d.Flags = shaderVisible
                ? D3D12_DESCRIPTOR_HEAP_FLAG_SHADER_VISIBLE
                : D3D12_DESCRIPTOR_HEAP_FLAG_NONE;
        dev->CreateDescriptorHeap(&d, IID_PPV_ARGS(&heap_));

        size_    = dev->GetDescriptorHandleIncrementSize(type);
        cpuStart_ = heap_->GetCPUDescriptorHandleForHeapStart();
        if (shaderVisible)
            gpuStart_ = heap_->GetGPUDescriptorHandleForHeapStart();
        capacity_ = count;
    }

    UINT Allocate() {
        if (used_ >= capacity_) throw std::runtime_error("heap full");
        return used_++;
    }

    D3D12_CPU_DESCRIPTOR_HANDLE Cpu(UINT i) const
    { return { cpuStart_.ptr + SIZE_T(i) * size_ }; }

    D3D12_GPU_DESCRIPTOR_HANDLE Gpu(UINT i) const
    { return { gpuStart_.ptr + UINT64(i) * size_ }; }

    ID3D12DescriptorHeap *Get() const { return heap_.Get(); }

private:
    ComPtr<ID3D12DescriptorHeap> heap_;
    D3D12_CPU_DESCRIPTOR_HANDLE  cpuStart_{};
    D3D12_GPU_DESCRIPTOR_HANDLE  gpuStart_{};
    UINT size_ = 0, used_ = 0, capacity_ = 0;
};

/* Creating a descriptor writes into the heap slot: */
UINT slot = srvHeap.Allocate();
D3D12_SHADER_RESOURCE_VIEW_DESC srv = {};
srv.Format                  = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
srv.ViewDimension           = D3D12_SRV_DIMENSION_TEXTURE2D;
srv.Shader4ComponentMapping = D3D12_DEFAULT_SHADER_4_COMPONENT_MAPPING;
srv.Texture2D.MipLevels     = -1;        /* all of them        */
device->CreateShaderResourceView(tex.Get(), &srv,
                                 srvHeap.Cpu(slot));

/* Binding points the shader at a slot: */
cmd->SetGraphicsRootDescriptorTable(1, srvHeap.Gpu(slot));
```

BINDLESS IS WHERE THIS IS GOING

With Resource Binding Tier 2 or 3 you can put every texture in the scene in one heap, pass an *integer index* to the shader, and index the heap directly: `ResourceDescriptorHeap[materialIndex]` in Shader Model 6.6. No per-draw descriptor tables, no rebinding, and the GPU can drive its own draws. It removes most of the binding code in this chapter, and it is what modern renderers do — worth designing toward even if you start with tables.

How it shows up

Symptom	Cause
A CPU loop over mapped memory takes milliseconds	Reading from an UPLOAD heap
Corruption or page fault after an upload	Staging buffer released before the GPU executed the copy
Texture uploads as garbage or skewed	Row pitch not padded to 256 bytes — use `UpdateSubresources`
Works on your GPU, corrupts elsewhere	Hardcoded descriptor increment size
Shader reads the wrong texture	CPU handle used where a GPU handle was needed, or the wrong slot index
Frame time collapses with many draws	`SetDescriptorHeaps` called per draw
Debug layer: heap not shader-visible	Binding a descriptor from an RTV/DSV or non-visible heap
Random blank textures	Descriptor slot overwritten by another allocation — no ownership tracking
Resource creation takes milliseconds	Committed resources per object; use placed resources from a pre-made heap
Out of memory with plenty of VRAM free	Heap fragmentation, or heap flags too restrictive for the resource type

Check yourself

- ✓ Explain the split between heap, resource and descriptor, and why DX12 separates them.
- ✓ Name the four heap types and say who can access each.
- ✓ Explain why reading from an UPLOAD heap is catastrophic.
- ✓ Describe the four steps of uploading a buffer, and the lifetime rule for the staging resource.
- ✓ Say why textures cannot be uploaded with a plain `memcpy`.
- ✓ Name the four descriptor heap types and which are shader-visible.
- ✓ State the one-heap-bound-at-a-time constraint and what it implies for your design.
- ✓ Distinguish CPU and GPU descriptor handles and give a bug from confusing them.
- ✓ Explain why descriptor increment size must be queried.
- ✓ Say what bindless changes, and which tier and shader model it needs.

DX12: root signatures and pipeline state

Two objects that between them replace all of DX11's state setting. The root signature is the function signature of your shaders; the pipeline state object is everything the driver needs to compile a shader ahead of time instead of mid-frame. Getting both right is what removes stutter.

The root signature is a function signature

```
Think of your shader as a function. The root signature
declares its parameters.

  void PSMain(cbuffer b0,            <- root parameter 0
              Texture2D t0[],        <- root parameter 1
              SamplerState s0)       <- static sampler

THREE KINDS OF ROOT PARAMETER, in cost order:

1. ROOT CONSTANTS          cost: 1 DWORD each
     values inlined directly into the command list. No
     indirection at all - the fastest possible way to get
     a handful of numbers to a shader.
     Use for: an object index, a material id, a few floats.

2. ROOT DESCRIPTORS        cost: 2 DWORDs
     a raw GPU virtual address of ONE buffer. One
     indirection. No descriptor heap involved.
     Use for: a per-object constant buffer.
     NOTE: no bounds checking, and only for buffers -
     textures cannot be root descriptors.

3. DESCRIPTOR TABLES       cost: 1 DWORD
     a pointer INTO the bound descriptor heap, plus a
     range. Two indirections.
     Use for: textures, and anything in bulk.

THE BUDGET: 64 DWORDs total. That is the whole thing.

     64 root constants, or
     32 root descriptors, or
     64 descriptor tables, or any mix.

AND SMALLER IS FASTER: the root signature is copied on
every draw. On some hardware the first 16 DWORDs live in
registers and the rest spill to memory. Aim for well under
16 DWORDs.
```

```
/* A practical root signature. */

D3D12_ROOT_PARAMETER1 params[3] = {};

/* 0: root constants - an object index. 1 DWORD. */
params[0].ParameterType = D3D12_ROOT_PARAMETER_TYPE_32BIT_CONSTANTS;
params[0].Constants.ShaderRegister = 0;      /* b0             */
params[0].Constants.Num32BitValues = 1;
params[0].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

/* 1: root CBV - the per-frame constant buffer. 2 DWORDs. */
params[1].ParameterType = D3D12_ROOT_PARAMETER_TYPE_CBV;
params[1].Descriptor.ShaderRegister = 1;     /* b1             */
params[1].ShaderVisibility = D3D12_SHADER_VISIBILITY_ALL;

/* 2: a table of textures. 1 DWORD. */
D3D12_DESCRIPTOR_RANGE1 range = {};
range.RangeType          = D3D12_DESCRIPTOR_RANGE_TYPE_SRV;
range.NumDescriptors     = 8;                /* t0..t7         */
range.BaseShaderRegister = 0;
range.Flags = D3D12_DESCRIPTOR_RANGE_FLAG_DATA_STATIC;
              /* promises the data will not change while in
                 use - lets the driver optimise. Use
                 DATA_VOLATILE if it might. */
params[2].ParameterType =
        D3D12_ROOT_PARAMETER_TYPE_DESCRIPTOR_TABLE;
params[2].DescriptorTable.NumDescriptorRanges = 1;
params[2].DescriptorTable.pDescriptorRanges   = &range;
params[2].ShaderVisibility = D3D12_SHADER_VISIBILITY_PIXEL;
        /* Restricting visibility is FREE performance: the
           driver need not copy this parameter to stages that
           cannot see it. Never leave everything on ALL. */

/* STATIC SAMPLERS cost NOTHING from the 64-DWORD budget and
   are baked into the signature. Almost every sampler you use
   should be static. */
D3D12_STATIC_SAMPLER_DESC samp = {};
samp.Filter           = D3D12_FILTER_ANISOTROPIC;
samp.MaxAnisotropy    = 16;
samp.AddressU = samp.AddressV = samp.AddressW =
        D3D12_TEXTURE_ADDRESS_MODE_WRAP;
samp.MaxLOD           = D3D12_FLOAT32_MAX;
samp.ShaderRegister   = 0;                   /* s0             */
samp.ShaderVisibility = D3D12_SHADER_VISIBILITY_PIXEL;

D3D12_VERSIONED_ROOT_SIGNATURE_DESC desc = {};
desc.Version = D3D_ROOT_SIGNATURE_VERSION_1_1;
desc.Desc_1_1.NumParameters     = 3;
desc.Desc_1_1.pParameters       = params;
desc.Desc_1_1.NumStaticSamplers = 1;
desc.Desc_1_1.pStaticSamplers   = &samp;
desc.Desc_1_1.Flags =
    D3D12_ROOT_SIGNATURE_FLAG_ALLOW_INPUT_ASSEMBLER_INPUT_LAYOUT
  | D3D12_ROOT_SIGNATURE_FLAG_DENY_HULL_SHADER_ROOT_ACCESS
  | D3D12_ROOT_SIGNATURE_FLAG_DENY_DOMAIN_SHADER_ROOT_ACCESS
  | D3D12_ROOT_SIGNATURE_FLAG_DENY_GEOMETRY_SHADER_ROOT_ACCESS;
    /* DENY flags for stages you do not use are also free
       performance. */

ComPtr<ID3DBlob> blob, err;
D3D12SerializeVersionedRootSignature(&desc, &blob, &err);
device->CreateRootSignature(0, blob->GetBufferPointer(),
```

```
                                blob->GetBufferSize(),
                                IID_PPV_ARGS(&rootSig));
```

ORDER ROOT PARAMETERS BY HOW OFTEN THEY CHANGE

Setting a root parameter invalidates everything after it on some hardware. Put the things that change least often first — per-frame, then per-material, then per-object — so a per-object update does not force the driver to re-emit the per-frame bindings. It is a five-minute reordering that costs nothing and is measurable in draw-heavy scenes.

The pipeline state object

```
/* Everything that could affect shader compilation, frozen
   into one immutable object. This is the whole point: the
   driver compiles ONCE, at load time, at a moment you chose. */

D3D12_GRAPHICS_PIPELINE_STATE_DESC pso = {};
pso.pRootSignature = rootSig.Get();
pso.VS = { vsBlob->GetBufferPointer(), vsBlob->GetBufferSize() };
pso.PS = { psBlob->GetBufferPointer(), psBlob->GetBufferSize() };

D3D12_INPUT_ELEMENT_DESC layout[] = {
  { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0,  0,
    D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 },
  { "NORMAL",   0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
    D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 },
  { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT,    0, 24,
    D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 },
};
pso.InputLayout = { layout, _countof(layout) };
/* The semantic names here must match the vertex shader's
   input semantics EXACTLY (chapter 7). */

pso.RasterizerState.FillMode = D3D12_FILL_MODE_SOLID;
pso.RasterizerState.CullMode = D3D12_CULL_MODE_BACK;
pso.RasterizerState.FrontCounterClockwise = FALSE;
pso.RasterizerState.DepthClipEnable       = TRUE;

pso.BlendState.RenderTarget[0].BlendEnable = FALSE;
pso.BlendState.RenderTarget[0].RenderTargetWriteMask =
        D3D12_COLOR_WRITE_ENABLE_ALL;

pso.DepthStencilState.DepthEnable    = TRUE;
pso.DepthStencilState.DepthWriteMask = D3D12_DEPTH_WRITE_MASK_ALL;
pso.DepthStencilState.DepthFunc      =
        D3D12_COMPARISON_FUNC_GREATER_EQUAL;   /* reversed-Z    */

pso.PrimitiveTopologyType = D3D12_PRIMITIVE_TOPOLOGY_TYPE_TRIANGLE;
pso.NumRenderTargets = 1;
pso.RTVFormats[0]    = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
pso.DSVFormat        = DXGI_FORMAT_D32_FLOAT;
pso.SampleMask       = UINT_MAX;      /* forgetting this gives
                                         a black screen - it
                                         defaults to 0, which
                                         masks out every sample */
pso.SampleDesc.Count = 1;

device->CreateGraphicsPipelineState(&pso, IID_PPV_ARGS(&pipeline));

/* WHAT IS *NOT* IN THE PSO - the state you still set per draw:
     viewport, scissor          RSSetViewports/ScissorRects
     render targets             OMSetRenderTargets
     primitive topology         IASetPrimitiveTopology
       (TYPE is in the PSO; LIST vs STRIP is not)
     vertex/index buffers       IASetVertexBuffers
     descriptor heaps           SetDescriptorHeaps
     root arguments             SetGraphicsRoot*
     stencil ref, blend factor  OMSetStencilRef, OMSetBlendFactor  */
```

PSO COUNT IS THE THING THAT SURPRISES TEAMS

A PSO is the *combination* of shaders, blend, depth, formats and layout. Ten shaders × three blend modes × two depth modes × two render-target formats is 120 PSOs, and each takes 1–50 ms to create. Real games have thousands and spend real effort on this — which is why they show a "compiling shaders" screen on first run and ship a pipeline cache. Enumerate the combinations you will actually use; do not create them lazily.

```
/* CACHING - so the second run is fast. */

/* 1. Per-PSO blob: */
ComPtr<ID3DBlob> cached;
pipeline->GetCachedBlob(&cached);
WriteFile("pso_042.bin", cached);
/* next run: */
pso.CachedPSO = { data, size };     /* driver skips the compile */

/* 2. Better - a whole-library cache: */
ComPtr<ID3D12PipelineLibrary> lib;
device->CreatePipelineLibrary(diskBlob, diskSize,
                              IID_PPV_ARGS(&lib));
lib->LoadGraphicsPipeline(L"OpaqueLit", &pso,
                          IID_PPV_ARGS(&pipeline));
if (!pipeline) {
    device->CreateGraphicsPipelineState(&pso, IID_PPV_ARGS(&pipeline));
    lib->StorePipeline(L"OpaqueLit", pipeline.Get());
}
/* At shutdown, serialise the library to disk. */

/* THE CACHE IS INVALIDATED BY: a driver update, a different
   GPU, a different OS build. Always be able to fall back to
   compiling, and never treat a cache miss as an error. */

/* 3. Create them in PARALLEL - PSO creation is thread-safe
      and CPU-bound, so it scales almost linearly: */
   parallel_for(0, psoDescs.size(), [&](size_t i) {
       device->CreateGraphicsPipelineState(&psoDescs[i],
                                           IID_PPV_ARGS(&psos[i]));
   });
/* A load screen that takes 40 s single-threaded takes 4 s on
   a modern CPU. This is worth doing on day one. */
```

Putting it together in a draw

```
cmd->SetPipelineState(pipeline.Get());
cmd->SetGraphicsRootSignature(rootSig.Get());
    /* Setting a root signature RESETS all root arguments.
       Set it first, then the arguments - the other order
       silently loses them. */

ID3D12DescriptorHeap *heaps[] = { srvHeap.Get() };
cmd->SetDescriptorHeaps(1, heaps);          /* once per list */

cmd->SetGraphicsRootConstantBufferView(1, frameCB->GetGPUVirtualAddress());
cmd->SetGraphicsRootDescriptorTable(2, srvHeap.Gpu(0));

cmd->IASetPrimitiveTopology(D3D_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
cmd->IASetVertexBuffers(0, 1, &vbv);
cmd->IASetIndexBuffer(&ibv);

for (uint32_t i = 0; i < objectCount; i++) {
    cmd->SetGraphicsRoot32BitConstant(0, i, 0);   /* 1 DWORD  */
    cmd->DrawIndexedInstanced(indexCount, 1, 0, 0, 0);
}
/* The inner loop is now ONE root constant and one draw -
   which is close to the minimum possible CPU cost per
   object, and why DX12 reaches tens of thousands of draws. */
```

How it shows up

Symptom	Cause
Black screen, no errors	`SampleMask` left at 0
Root arguments appear not to take effect	Root signature set *after* them — it resets them all
PSO creation fails with a vague message	Input layout semantics do not match the vertex shader, or formats do not match the render targets
Debug layer: root signature mismatch	The shader was compiled against a different signature than the PSO uses
Root signature will not serialise	Over the 64-DWORD budget
First appearance of an effect stutters	PSO created lazily — enumerate and pre-create
Load times of tens of seconds	PSOs created serially; parallelise
Cache gives no benefit after a driver update	Expected — the cache is driver-specific
Draw calls cost more than expected	Root signature too large, or parameters ordered so per-object updates invalidate per-frame ones
Geometry renders inside-out or not at all	Cull mode versus winding order in the rasteriser state

Check yourself

- ✓ Explain the root signature as a function signature, and name its three parameter kinds with their costs.
- ✓ State the root signature budget and what happens when you exceed it.
- ✓ Say why static samplers are preferable, and what they cost.

- ✓ Explain why shader visibility and DENY flags are free performance.
- ✓ Give the rule for ordering root parameters and why it matters.
- ✓ List six pieces of state that go in a PSO and five that do not.
- ✓ Explain why `SampleMask` is a classic black-screen bug.
- ✓ Estimate a PSO count from shader and state combinations, and say why it matters.
- ✓ Describe two levels of PSO caching and what invalidates them.
- ✓ Explain why the root signature must be set before its arguments.

DX12: fences, barriers and residency

The hardest part of the explicit APIs, and the one the old drivers were silently doing for you. Fences order the CPU against the GPU; barriers order the GPU against itself. Get either wrong and you get corruption, a hang, or a device removal — usually far from the cause.

Fences: CPU-to-GPU ordering

```
/* A fence is a monotonically increasing 64-bit counter that
   the GPU writes and either side can read or wait on. That
   is the entire abstraction. */

ComPtr<ID3D12Fence> fence;
device->CreateFence(0, D3D12_FENCE_FLAG_NONE, IID_PPV_ARGS(&fence));
HANDLE event = CreateEvent(nullptr, FALSE, FALSE, nullptr);

/* GPU side: "when you reach this point in the queue, set the
   counter to n". Signal is QUEUED - it happens when the GPU
   gets there, not when you call it. */
queue->Signal(fence.Get(), n);

/* CPU side: wait until the counter reaches n. */
if (fence->GetCompletedValue() < n) {
    fence->SetEventOnCompletion(n, event);
    WaitForSingleObject(event, INFINITE);
}

/* GPU side waiting on another queue - no CPU involvement: */
otherQueue->Wait(fence.Get(), n);

/* THE CANONICAL USE - frames in flight (chapter 13): */
UINT64 frameFence[kFrames] = {};
UINT64 next = 1;

/* end of frame: */
    frameFence[i] = next;
    queue->Signal(fence.Get(), next++);

/* start of the frame that reuses slot i, kFrames later: */
    WaitForFence(frameFence[i]);
    alloc[i]->Reset();            /* now provably safe         */

/* THE THREE THINGS FENCES PROTECT

   1. command allocators   - do not reset while executing
   2. upload buffers       - do not overwrite while being read
   3. resource destruction - do not free while referenced

   Every "it corrupts a few frames later" bug is one of these
   three. If you take one habit from this chapter, take this:
   NOTHING IS REUSED OR RELEASED UNTIL A FENCE PROVES THE GPU
   IS DONE WITH IT.                                          */
```

> **DO NOT WAIT ON THE FENCE YOU JUST SIGNALLED**
> `queue->Signal(f, n); WaitForFence(n);` is a full CPU/GPU stall — you have serialised the two processors and thrown away all the pipelining the API exists to provide. It is correct, and it is what "flush" means. Use it at shutdown, on resize, and nowhere in a frame loop. If your frame time equals CPU time plus GPU time rather than the maximum of the two, this is why.

Barriers: GPU-to-GPU ordering

```
A resource is in exactly ONE STATE at a time, and the state
tells the hardware how to lay it out and cache it.

  RENDER_TARGET            being drawn into
  PIXEL_SHADER_RESOURCE    being sampled
  DEPTH_WRITE / DEPTH_READ
  UNORDERED_ACCESS         compute reads and writes
  COPY_SOURCE / COPY_DEST
  PRESENT                  handed to the display
  COMMON                   the neutral state; required at
                           queue boundaries

You cannot sample a texture you are still rendering into.
The TRANSITION between states is a barrier, and it does
three things:

  1. WAITS for prior work touching the resource to finish
  2. FLUSHES/INVALIDATES the relevant caches
  3. DECOMPRESSES or re-lays-out the memory if the two
     states use different layouts

That third one is why barriers are not free: moving a
render target to a shader resource may involve a full
depth or colour decompression pass over the surface.
```

```
/* The three barrier types. */

/* 1. TRANSITION - the common one. */
D3D12_RESOURCE_BARRIER b = {};
b.Type  = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
b.Flags = D3D12_RESOURCE_BARRIER_FLAG_NONE;
b.Transition.pResource   = tex.Get();
b.Transition.StateBefore = D3D12_RESOURCE_STATE_RENDER_TARGET;
b.Transition.StateAfter  = D3D12_RESOURCE_STATE_PIXEL_SHADER_RESOURCE;
b.Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
cmd->ResourceBarrier(1, &b);

/* 2. UAV - no state change, but "finish all writes before any
      reads". Needed between two dispatches where the second
      reads what the first wrote. */
D3D12_RESOURCE_BARRIER u = {};
u.Type          = D3D12_RESOURCE_BARRIER_TYPE_UAV;
u.UAV.pResource = buffer.Get();
cmd->ResourceBarrier(1, &u);

/* 3. ALIASING - when two placed resources share heap memory
      and you switch which one is live. */

/* BATCH THEM. One call with five barriers is much cheaper
   than five calls, because the driver can merge the waits. */
D3D12_RESOURCE_BARRIER batch[3] = { b0, b1, b2 };
cmd->ResourceBarrier(3, batch);

/* SPLIT BARRIERS - start the transition early, finish it
   later, so the GPU can overlap the decompression with other
   work: */
b.Flags = D3D12_RESOURCE_BARRIER_FLAG_BEGIN_ONLY;
cmd->ResourceBarrier(1, &b);
    /* ... unrelated work happens here ... */
b.Flags = D3D12_RESOURCE_BARRIER_FLAG_END_ONLY;
cmd->ResourceBarrier(1, &b);
/* Between BEGIN and END the resource may not be used at all. */
```

```
/* THE TYPICAL FRAME, as a state machine. */

  backbuffer:  PRESENT
        |  transition
        v
      RENDER_TARGET      <- draw the scene
        |  transition
        v
      PRESENT            <- Present()

  shadowMap:   DEPTH_WRITE     <- render shadows
        |  transition
        v
      PIXEL_SHADER_RESOURCE    <- sample during lighting
        |  transition (next frame)
        v
      DEPTH_WRITE

  gbuffer:     RENDER_TARGET   <- geometry pass
        |
        v
      PIXEL_SHADER_RESOURCE    <- lighting pass

/* A helper, because writing the struct out every time is how
   mistakes happen: */
inline void Transition(ID3D12GraphicsCommandList *cmd,
                       ID3D12Resource *res,
                       D3D12_RESOURCE_STATES from,
                       D3D12_RESOURCE_STATES to)
{
    if (from == to) return;                /* no-op, and the
                                              debug layer
                                              complains       */
    D3D12_RESOURCE_BARRIER b = {};
    b.Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    b.Transition.pResource   = res;
    b.Transition.StateBefore = from;
    b.Transition.StateAfter  = to;
    b.Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
    cmd->ResourceBarrier(1, &b);
}

/* AND TRACK THE STATE YOURSELF. The runtime does not tell you
   what state a resource is in; you must know. A small
   wrapper that stores the current state per resource and
   asserts on mismatch pays for itself immediately - the debug
   layer catches errors, but only at execute time, and only
   the ones that actually execute. */
```

ENHANCED BARRIERS ARE THE NEWER, BETTER MODEL

DX12's original barriers conflate three separate things: execution dependency, memory visibility, and layout transition. **Enhanced barriers** (Agility SDK 1.7+) split them, matching Vulkan's model and letting you express "wait for compute, but do not decompress" — which the old model could not say. New code should prefer them; the concepts in this chapter map across directly.

Too many barriers, and too few

```
/* TOO FEW: corruption, hangs, or the classic "works on
   NVIDIA, breaks on AMD" - because one driver happened to
   flush a cache the other did not.

   TOO MANY: every barrier is a potential pipeline drain. A
   barrier between every draw serialises the GPU completely
   and can cost more than the draws.                        */

/* BAD - a barrier per object. */
for (auto &o : objects) {
    Transition(cmd, o.tex, COPY_DEST, PIXEL_SHADER_RESOURCE);
    Draw(o);
}

/* GOOD - transition everything once, then draw. */
std::vector<D3D12_RESOURCE_BARRIER> barriers;
for (auto &o : objects)
    barriers.push_back(MakeTransition(o.tex, COPY_DEST,
                                      PIXEL_SHADER_RESOURCE));
cmd->ResourceBarrier((UINT)barriers.size(), barriers.data());
for (auto &o : objects) Draw(o);

/* THE GENERAL PATTERN: group your frame into PASSES, and put
   all the barriers at pass boundaries. Inside a pass, nothing
   changes state.

   A RENDER GRAPH is the industrial version of this idea: you
   declare which passes read and write which resources, and
   the graph derives the minimal barrier set, the state
   transitions and the aliasing opportunities automatically.
   Every large engine has one, for exactly this reason.    */
```

Residency

```
/* VRAM is not infinite and you are not the only application.
   Windows may EVICT your resources to system memory when
   another app needs the memory. */

DXGI_QUERY_VIDEO_MEMORY_INFO info = {};
adapter->QueryVideoMemoryInfo(0, DXGI_MEMORY_SEGMENT_GROUP_LOCAL,
                              &info);
/* info.Budget       - what the OS says you may use
   info.CurrentUsage - what you are using
   Exceeding the budget does not fail; it makes the OS start
   evicting, and your frame time falls off a cliff. */

/* Explicit control, for streaming systems: */
ID3D12Pageable *res[] = { texture.Get() };
device->Evict(1, res);                /* push to system memory */
device->MakeResident(1, res);         /* pull back - BLOCKING  */

/* The async form, so you can prefetch without stalling: */
device->EnqueueMakeResident(D3D12_RESIDENCY_FLAG_NONE, 1, res,
                            fence.Get(), fenceValue);

/* THE PRACTICAL ADVICE
     * poll the budget each frame and shed quality if you are
       over - drop mip levels, reduce the shadow atlas
     * do not fight the OS: if the budget shrinks because the
       user alt-tabbed to a browser, respect it
     * a frame time that suddenly doubles with no code change
       is very often eviction, not your rendering            */
```

How it shows up

Symptom	Cause
Corruption a few frames after a change	Allocator or upload buffer reused without a fence wait
Page fault after freeing a resource	Released while in-flight work still referenced it
Frame time equals CPU + GPU rather than max	A full flush in the frame loop
Works on NVIDIA, corrupts on AMD	Missing barrier that one driver's cache behaviour hid
Sampling a render target returns garbage	No transition from `RENDER_TARGET` to `PIXEL_SHADER_RESOURCE`
Second dispatch reads stale data	Missing UAV barrier between dispatches
Debug layer: "resource is in state X, expected Y"	Your state tracking and reality have diverged
GPU serialised, poor utilisation	Barriers per draw instead of per pass
Frame time doubles when another app opens	VRAM eviction — check the budget
Hang with a fence never reaching its value	Signalled on a different queue, or the value was never signalled at all
Debug layer complains about a same-state transition	Guard with `if (from == to) return;`

Check yourself

- ✓ Describe a fence in one sentence and say who signals and who waits.
- ✓ Name the three things fences protect and the bug from each.

- ✓ Explain why signalling and immediately waiting is a stall, and when it is legitimate.
- ✓ List six resource states and say what a transition barrier actually does.
- ✓ Explain why a transition is not free.
- ✓ Name the three barrier types and when the UAV one is needed.
- ✓ Explain what a split barrier buys.
- ✓ Give the failure mode for too few barriers and for too many.
- ✓ Explain what a render graph automates.
- ✓ Say what residency is and how a frame-time cliff can come from another application.

Project: a DX12 triangle, every line explained

The smallest complete DX12 program. Roughly 400 lines, and by the end of this chapter every one of them should be a decision you understand rather than a line you copied — which is the entire difference between using DX12 and fighting it.

The order of operations

```
STARTUP
  1.  enable the debug layer      BEFORE the device
  2.  create the DXGI factory
  3.  pick an adapter             not adapter 0
  4.  create the device
  5.  configure the info queue and DRED
  6.  create the command queue
  7.  create the swap chain       needs the QUEUE, not the
                                  device
  8.  create descriptor heaps     RTV heap for backbuffers
  9.  create RTVs for each backbuffer
  10. create command allocators   one per frame in flight
  11. create the command list
  12. create the fence and event
  13. compile shaders
  14. create the root signature
  15. create the PSO
  16. create the vertex buffer + upload it
  17. WAIT for the upload to finish

PER FRAME
  18. wait for this frame slot's fence
  19. reset allocator, then list
  20. transition backbuffer PRESENT -> RENDER_TARGET
  21. set state, clear, draw
  22. transition RENDER_TARGET -> PRESENT
  23. close, execute, present, signal

SHUTDOWN
  24. flush - wait for the GPU to be completely idle
  25. release, in any order (ComPtr does it)

Step 24 is not optional: destroying resources the GPU is
still using is a page fault or a hang, and it happens on
exit where nobody looks.
```

The shader

```
// triangle.hlsl - compile both entry points:
//   dxc -T vs_6_0 -E VSMain -Fo vs.cso triangle.hlsl
//   dxc -T ps_6_0 -E PSMain -Fo ps.cso triangle.hlsl
//
// dxc, not fxc: fxc only emits DXBC (shader model 5.1 and
// below). Anything using wave intrinsics or SM 6 features
// needs dxc.

struct VSIn  { float3 pos : POSITION; float4 col : COLOR; };
struct VSOut { float4 pos : SV_Position; float4 col : COLOR; };

VSOut VSMain(VSIn i)
{
    VSOut o;
    o.pos = float4(i.pos, 1.0);   // already in clip space -
                                  // no matrices in a first
                                  // triangle
    o.col = i.col;
    return o;
}

float4 PSMain(VSOut i) : SV_Target
{
    return i.col;                 // interpolated across the
                                  // triangle by the
                                  // rasteriser, for free
}
```

Initialisation

```
#include <d3d12.h>
#include <dxgi1_6.h>
#include <d3dcompiler.h>
#include <wrl/client.h>
#include <stdexcept>
#include <vector>
using Microsoft::WRL::ComPtr;

#pragma comment(lib, "d3d12.lib")
#pragma comment(lib, "dxgi.lib")

static const UINT kFrames = 2;

struct Vertex { float pos[3]; float col[4]; };

class TriangleApp {
    ComPtr<ID3D12Device>              device_;
    ComPtr<ID3D12CommandQueue>        queue_;
    ComPtr<IDXGISwapChain3>           swap_;
    ComPtr<ID3D12DescriptorHeap>      rtvHeap_;
    ComPtr<ID3D12Resource>            backBuf_[kFrames];
    ComPtr<ID3D12CommandAllocator>    alloc_[kFrames];
    ComPtr<ID3D12GraphicsCommandList> cmd_;
    ComPtr<ID3D12RootSignature>       rootSig_;
    ComPtr<ID3D12PipelineState>       pso_;
    ComPtr<ID3D12Resource>            vb_;
    D3D12_VERTEX_BUFFER_VIEW          vbv_{};
    ComPtr<ID3D12Fence>               fence_;
    HANDLE   fenceEvent_ = nullptr;
    UINT64   fenceVal_[kFrames] = {};
    UINT64   nextFence_ = 1;
    UINT     frame_ = 0, rtvSize_ = 0;
    UINT     width_ = 1280, height_ = 720;

public:
void Init(HWND hwnd)
{
    /* 1. DEBUG LAYER FIRST. After device creation it silently
          does nothing. */
#if defined(_DEBUG)
    ComPtr<ID3D12Debug> dbg;
    if (SUCCEEDED(D3D12GetDebugInterface(IID_PPV_ARGS(&dbg))))
        dbg->EnableDebugLayer();
    UINT factoryFlags = DXGI_CREATE_FACTORY_DEBUG;
#else
    UINT factoryFlags = 0;
#endif

    /* 2-4. factory, adapter, device (chapter 12). */
    ComPtr<IDXGIFactory6> factory;
    CreateDXGIFactory2(factoryFlags, IID_PPV_ARGS(&factory));

    ComPtr<IDXGIAdapter1> adapter;
    factory->EnumAdapterByGpuPreference(
        0, DXGI_GPU_PREFERENCE_HIGH_PERFORMANCE,
        IID_PPV_ARGS(&adapter));

    if (FAILED(D3D12CreateDevice(adapter.Get(),
                                 D3D_FEATURE_LEVEL_11_0,
                                 IID_PPV_ARGS(&device_))))
```

```
        throw std::runtime_error("D3D12CreateDevice");

    /* 6. The queue. The swap chain needs it, not the device -
          because presentation is itself queued work. */
    D3D12_COMMAND_QUEUE_DESC qd = {};
    qd.Type = D3D12_COMMAND_LIST_TYPE_DIRECT;
    device_->CreateCommandQueue(&qd, IID_PPV_ARGS(&queue_));

    /* 7. Swap chain. FLIP_DISCARD is mandatory for DX12 and
          is also the only mode that allows tearing control
          and true independent flip. */
    DXGI_SWAP_CHAIN_DESC1 sd = {};
    sd.BufferCount = kFrames;
    sd.Width       = width_;
    sd.Height      = height_;
    sd.Format      = DXGI_FORMAT_R8G8B8A8_UNORM;
    sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    sd.SwapEffect  = DXGI_SWAP_EFFECT_FLIP_DISCARD;
    sd.SampleDesc.Count = 1;          /* MSAA is NOT allowed on
                                         a flip-model swap
                                         chain - render to your
                                         own MSAA target and
                                         resolve into this   */
    ComPtr<IDXGISwapChain1> sc1;
    factory->CreateSwapChainForHwnd(queue_.Get(), hwnd, &sd,
                                    nullptr, nullptr, &sc1);
    factory->MakeWindowAssociation(hwnd, DXGI_MWA_NO_ALT_ENTER);
    sc1.As(&swap_);
    frame_ = swap_->GetCurrentBackBufferIndex();

    /* 8-9. RTV heap and one view per backbuffer. */
    D3D12_DESCRIPTOR_HEAP_DESC hd = {};
    hd.NumDescriptors = kFrames;
    hd.Type  = D3D12_DESCRIPTOR_HEAP_TYPE_RTV;
    hd.Flags = D3D12_DESCRIPTOR_HEAP_FLAG_NONE;   /* CPU-only */
    device_->CreateDescriptorHeap(&hd, IID_PPV_ARGS(&rtvHeap_));

    /* QUERY the increment - do not assume (chapter 14). */
    rtvSize_ = device_->GetDescriptorHandleIncrementSize(
                   D3D12_DESCRIPTOR_HEAP_TYPE_RTV);

    auto rtv = rtvHeap_->GetCPUDescriptorHandleForHeapStart();
    for (UINT i = 0; i < kFrames; i++) {
        swap_->GetBuffer(i, IID_PPV_ARGS(&backBuf_[i]));
        device_->CreateRenderTargetView(backBuf_[i].Get(),
                                        nullptr, rtv);
        rtv.ptr += rtvSize_;
        device_->CreateCommandAllocator(
            D3D12_COMMAND_LIST_TYPE_DIRECT,
            IID_PPV_ARGS(&alloc_[i]));       /* 10 */
    }

    /* 12. Fence. */
    device_->CreateFence(0, D3D12_FENCE_FLAG_NONE,
                         IID_PPV_ARGS(&fence_));
    fenceEvent_ = CreateEvent(nullptr, FALSE, FALSE, nullptr);

    CreatePipeline();
    CreateVertexBuffer();
}
```

```
void CreatePipeline()
{
    /* 14. An EMPTY root signature - this shader takes no
           parameters at all. Still required: a PSO must have
           one. */
    D3D12_ROOT_SIGNATURE_DESC rs = {};
    rs.Flags = D3D12_ROOT_SIGNATURE_FLAG_ALLOW_INPUT_ASSEMBLER_INPUT_LAYOUT;

    ComPtr<ID3DBlob> sig, err;
    if (FAILED(D3D12SerializeRootSignature(
            &rs, D3D_ROOT_SIGNATURE_VERSION_1, &sig, &err)))
        throw std::runtime_error((char *)err->GetBufferPointer());

    device_->CreateRootSignature(0, sig->GetBufferPointer(),
                                 sig->GetBufferSize(),
                                 IID_PPV_ARGS(&rootSig_));

    ComPtr<ID3DBlob> vs = LoadBlob(L"vs.cso");
    ComPtr<ID3DBlob> ps = LoadBlob(L"ps.cso");

    /* The semantic NAMES must match the HLSL exactly, and the
       OFFSETS must match the C++ struct exactly. Both are
       silent when wrong (chapter 7). */
    D3D12_INPUT_ELEMENT_DESC layout[] = {
      { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT,    0,  0,
        D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 },
      { "COLOR",    0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12,
        D3D12_INPUT_CLASSIFICATION_PER_VERTEX_DATA, 0 },
    };

    D3D12_GRAPHICS_PIPELINE_STATE_DESC p = {};
    p.pRootSignature = rootSig_.Get();
    p.VS = { vs->GetBufferPointer(), vs->GetBufferSize() };
    p.PS = { ps->GetBufferPointer(), ps->GetBufferSize() };
    p.InputLayout = { layout, _countof(layout) };

    p.RasterizerState.FillMode = D3D12_FILL_MODE_SOLID;
    p.RasterizerState.CullMode = D3D12_CULL_MODE_NONE;
        /* NONE for a first triangle: with BACK, a winding
           order you did not think about gives you an empty
           screen and no error. Turn culling on once you can
           see something. */
    p.RasterizerState.DepthClipEnable = TRUE;

    p.BlendState.RenderTarget[0].RenderTargetWriteMask =
        D3D12_COLOR_WRITE_ENABLE_ALL;
        /* zero-initialised means BlendEnable = FALSE, which is
           what we want - but the WRITE MASK also defaults to
           0, which writes nothing. Set it. */

    p.DepthStencilState.DepthEnable   = FALSE;
    p.DepthStencilState.StencilEnable = FALSE;

    p.PrimitiveTopologyType = D3D12_PRIMITIVE_TOPOLOGY_TYPE_TRIANGLE;
    p.NumRenderTargets = 1;
    p.RTVFormats[0]    = DXGI_FORMAT_R8G8B8A8_UNORM;
        /* MUST match the swap chain format exactly, or PSO
           creation fails. */
    p.SampleDesc.Count = 1;
    p.SampleMask       = UINT_MAX;
        /* THE CLASSIC BLACK SCREEN. Defaults to 0, which
           masks out every sample. */
```

```
    if (FAILED(device_->CreateGraphicsPipelineState(
            &p, IID_PPV_ARGS(&pso_))))
        throw std::runtime_error("CreateGraphicsPipelineState");

    /* 11. The list is created OPEN, on allocator 0. Close it
           so the frame loop can Reset symmetrically. */
    device_->CreateCommandList(0, D3D12_COMMAND_LIST_TYPE_DIRECT,
                               alloc_[0].Get(), pso_.Get(),
                               IID_PPV_ARGS(&cmd_));
    cmd_->Close();
}
```

```
void CreateVertexBuffer()
{
    Vertex verts[] = {
        { {  0.0f,  0.75f, 0.0f }, { 1, 0, 0, 1 } },
        { {  0.75f,-0.75f, 0.0f }, { 0, 1, 0, 1 } },
        { { -0.75f,-0.75f, 0.0f }, { 0, 0, 1, 1 } },
    };
    const UINT size = sizeof verts;

    /* SHORTCUT, and it is worth naming as one: this puts the
       vertex buffer in an UPLOAD heap so we can memcpy
       directly, with no staging buffer and no copy command.
       For three vertices that is fine. For real geometry it
       is wrong - upload memory is slower for the GPU to read,
       and you want DEFAULT plus a copy (chapter 14). */
    D3D12_HEAP_PROPERTIES hp = { D3D12_HEAP_TYPE_UPLOAD };
    D3D12_RESOURCE_DESC rd = {};
    rd.Dimension        = D3D12_RESOURCE_DIMENSION_BUFFER;
    rd.Width            = size;
    rd.Height           = 1;
    rd.DepthOrArraySize = 1;
    rd.MipLevels        = 1;
    rd.Format           = DXGI_FORMAT_UNKNOWN;
    rd.SampleDesc.Count = 1;
    rd.Layout           = D3D12_TEXTURE_LAYOUT_ROW_MAJOR;

    device_->CreateCommittedResource(
        &hp, D3D12_HEAP_FLAG_NONE, &rd,
        D3D12_RESOURCE_STATE_GENERIC_READ,
            /* UPLOAD heap resources must be created in
               GENERIC_READ and can never leave it. */
        nullptr, IID_PPV_ARGS(&vb_));

    void *p = nullptr;
    D3D12_RANGE readNothing = { 0, 0 };
    vb_->Map(0, &readNothing, &p);
    memcpy(p, verts, size);          /* WRITE ONLY. Never read
                                        from upload memory. */
    vb_->Unmap(0, nullptr);

    vbv_.BufferLocation = vb_->GetGPUVirtualAddress();
    vbv_.StrideInBytes  = sizeof(Vertex);
    vbv_.SizeInBytes    = size;
        /* Stride must equal the C++ struct size and match the
           input layout offsets. A mismatch renders garbage
           silently. */
}
```

The frame

```
void Render()
{
    /* 18. Do not touch this allocator until the GPU has
           finished the last frame that used it. */
    WaitForFence(fenceVal_[frame_]);

    /* 19. Allocator first (frees its memory), then the list
           (begins recording into it). */
    alloc_[frame_]->Reset();
    cmd_->Reset(alloc_[frame_].Get(), pso_.Get());

    cmd_->SetGraphicsRootSignature(rootSig_.Get());

    D3D12_VIEWPORT vp = { 0, 0, (float)width_, (float)height_,
                          0.0f, 1.0f };
    D3D12_RECT     sc = { 0, 0, (LONG)width_, (LONG)height_ };
    cmd_->RSSetViewports(1, &vp);
    cmd_->RSSetScissorRects(1, &sc);
        /* A scissor of {0,0,0,0} clips everything away. It is
           the second most common blank-screen cause after
           SampleMask, and neither produces an error. */

    /* 20. The backbuffer arrives in PRESENT state. */
    Transition(backBuf_[frame_].Get(),
               D3D12_RESOURCE_STATE_PRESENT,
               D3D12_RESOURCE_STATE_RENDER_TARGET);

    D3D12_CPU_DESCRIPTOR_HANDLE rtv =
        rtvHeap_->GetCPUDescriptorHandleForHeapStart();
    rtv.ptr += SIZE_T(frame_) * rtvSize_;

    cmd_->OMSetRenderTargets(1, &rtv, FALSE, nullptr);

    const float clear[4] = { 0.05f, 0.06f, 0.09f, 1.0f };
    cmd_->ClearRenderTargetView(rtv, clear, 0, nullptr);

    /* 21. Three lines to draw a triangle. */
    cmd_->IASetPrimitiveTopology(D3D_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
        /* The PSO holds the TYPE (triangle); the list holds
           LIST vs STRIP. Both are required. */
    cmd_->IASetVertexBuffers(0, 1, &vbv_);
    cmd_->DrawInstanced(3, 1, 0, 0);

    /* 22. Back to PRESENT, or Present() fails validation. */
    Transition(backBuf_[frame_].Get(),
               D3D12_RESOURCE_STATE_RENDER_TARGET,
               D3D12_RESOURCE_STATE_PRESENT);

    /* 23. Close, submit, present, signal. */
    cmd_->Close();
    ID3D12CommandList *lists[] = { cmd_.Get() };
    queue_->ExecuteCommandLists(1, lists);

    swap_->Present(1, 0);        /* 1 = vsync                  */

    fenceVal_[frame_] = nextFence_;
    queue_->Signal(fence_.Get(), nextFence_++);

    frame_ = swap_->GetCurrentBackBufferIndex();
        /* ASK the swap chain, do not assume (frame+1)%N.
```

```
           With FLIP_DISCARD the order is not guaranteed. */
}

void WaitForFence(UINT64 v)
{
    if (v == 0 || fence_->GetCompletedValue() >= v) return;
    fence_->SetEventOnCompletion(v, fenceEvent_);
    WaitForSingleObject(fenceEvent_, INFINITE);
}

void Transition(ID3D12Resource *r, D3D12_RESOURCE_STATES from,
                D3D12_RESOURCE_STATES to)
{
    D3D12_RESOURCE_BARRIER b = {};
    b.Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
    b.Transition.pResource   = r;
    b.Transition.StateBefore = from;
    b.Transition.StateAfter  = to;
    b.Transition.Subresource = D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
    cmd_->ResourceBarrier(1, &b);
}

/* 24. SHUTDOWN. Wait for everything, then let ComPtr free. */
void Shutdown()
{
    queue_->Signal(fence_.Get(), nextFence_);
    WaitForFence(nextFence_);
    CloseHandle(fenceEvent_);
}
};   /* class TriangleApp */
```

THE FOUR BLANK-SCREEN CAUSES, IN ORDER OF FREQUENCY

1. `SampleMask` left at 0. **2.** Scissor rect not set, so it is {0,0,0,0}. **3.** `RenderTargetWriteMask` left at 0. **4.** Backface culling with the winding order you did not think about. None of the four produces an error message. If you get a clear colour but no triangle, work down that list before anything else.

Where to go from here

Next step	What it teaches
Add a constant buffer with a rotation matrix	Root descriptors, per-frame upload, and the lifetime rule (chapter 15)
Move the vertex buffer to a DEFAULT heap	Staging buffers, copies and the barrier that follows (chapter 14)
Add a depth buffer	DSV heaps, and reversed-Z (chapter 8)
Add a texture	Descriptor heaps, SRVs, `UpdateSubresources` and static samplers
Draw 10,000 instances	`DrawInstanced`, and where the real CPU cost is
Handle window resize	Flush, release the backbuffers, `ResizeBuffers`, recreate RTVs — a surprising amount of care
Record from four threads	Allocator-per-thread and barrier consistency (chapter 13)
Capture it in PIX	What a frame actually looks like to a tool

How it shows up

Symptom	Cause
Clear colour appears, no triangle	One of the four causes above
Nothing at all, not even the clear	Missing barrier to `RENDER_TARGET`, or the RTV handle offset is wrong
Garbage geometry	Vertex stride or input layout offsets do not match the struct
PSO creation fails	RTV format does not match the swap chain, or semantics do not match the shader
`Present` fails validation	Backbuffer not returned to `PRESENT` state
Crash or corruption after a few seconds	Allocator reset without waiting on the fence
Crash on exit	No flush before releasing resources
Swap chain creation fails	MSAA requested on a flip-model swap chain, or the wrong swap effect
Alt-Enter does something strange	`MakeWindowAssociation` not called
Second frame renders into the first buffer	Assumed `(frame+1)%N` instead of asking `GetCurrentBackBufferIndex`

Check yourself

- ✓ List the startup steps in order and say why the debug layer must be first.
- ✓ Explain why the swap chain is created from the queue rather than the device.
- ✓ Say why `SampleMask` and the write mask must be set explicitly.
- ✓ Give the four blank-screen causes and how to distinguish them.
- ✓ Explain why the vertex buffer here is in an UPLOAD heap and why that is a shortcut.
- ✓ Say why the allocator is reset before the list, and what the fence protects.
- ✓ Explain the two places primitive topology is specified and why both are needed.
- ✓ Explain why the backbuffer index is queried rather than computed.
- ✓ Say what must happen at shutdown and what goes wrong without it.
- ✓ Get the triangle on screen, then break it four ways deliberately and recognise each symptom.

Compute shaders and GPGPU

The graphics pipeline with the graphics removed: no vertices, no rasteriser, no render target. Just a grid of threads, groupshared memory and read-write buffers. It is the same hardware as a pixel shader with the fixed-function stages switched off, and it is how most GPU work is written today.

The dispatch model

```
Dispatch(gx, gy, gz)   launches gx*gy*gz THREAD GROUPS.
[numthreads(tx,ty,tz)] says each group has tx*ty*tz threads.

  total threads = gx*gy*gz * tx*ty*tz

THE IDS available in the shader:

  SV_GroupID           which group am I in       (gx,gy,gz)
  SV_GroupThreadID     where in my group         (tx,ty,tz)
  SV_DispatchThreadID  global = GroupID*size + GroupThreadID
  SV_GroupIndex        flattened index within the group

THE HIERARCHY, and it is the same as CUDA's with different
names (chapter 1):

  thread          one lane
  wave/warp       32 (or 64) threads in lockstep
  THREAD GROUP    up to 1024 threads. Shares GROUPSHARED
                  memory and can use GroupMemoryBarrier.
                  Runs entirely on one compute unit.
  dispatch        all groups. NO synchronisation between
                  groups, and no guaranteed order.

THE RULE: cooperation within a group, none between groups.
Need a global barrier? That is a second Dispatch.

GROUP SIZE: use a multiple of the wave size. 64 is the
safe default - one wave on AMD, two on NVIDIA - and 256 is
the usual choice for 1D work. A group of 1 wastes 31 lanes
of every wave.
```

```
// A compute shader, complete.
// dxc -T cs_6_0 -E CSMain -Fo cs.cso blur.hlsl

Texture2D<float4>   Input  : register(t0);
RWTexture2D<float4> Output : register(u0);   // UAV: writable

cbuffer Params : register(b0) { uint2 Size; float Radius; };

[numthreads(8, 8, 1)]           // 64 threads - one wave on AMD
void CSMain(uint3 id : SV_DispatchThreadID)
{
    if (any(id.xy >= Size)) return;      // bounds check: the
                                         // grid is rounded up
    float4 sum = 0;
    int    n   = 0;
    for (int dy = -2; dy <= 2; dy++)
        for (int dx = -2; dx <= 2; dx++) {
            int2 p = clamp(int2(id.xy) + int2(dx, dy),
                           int2(0,0), int2(Size) - 1);
            sum += Input[p];             // [] is a LOAD, not a
            n++;                         // Sample - no filter,
        }                                // no mip, no sampler
    Output[id.xy] = sum / n;
}
```

```
/* Dispatching it from DX12. */

cmd->SetPipelineState(computePSO.Get());
cmd->SetComputeRootSignature(computeRootSig.Get());
    /* NOTE: SetCompute*, not SetGraphics*. Graphics and
       compute root arguments are SEPARATE sets of state on the
       same command list. Using the wrong setter binds nothing
       and produces no error. */

ID3D12DescriptorHeap *heaps[] = { srvUavHeap.Get() };
cmd->SetDescriptorHeaps(1, heaps);
cmd->SetComputeRootDescriptorTable(0, srvUavHeap.Gpu(0));

/* ROUND UP - the last group is partly out of bounds, which is
   why the shader needs its bounds check. */
UINT gx = (width  + 7) / 8;
UINT gy = (height + 7) / 8;
cmd->Dispatch(gx, gy, 1);

/* THEN A UAV BARRIER before anything reads the output. */
D3D12_RESOURCE_BARRIER uav = {};
uav.Type          = D3D12_RESOURCE_BARRIER_TYPE_UAV;
uav.UAV.pResource = outputTex.Get();
cmd->ResourceBarrier(1, &uav);

/* And a transition if a pixel shader will sample it: */
Transition(outputTex.Get(), D3D12_RESOURCE_STATE_UNORDERED_ACCESS,
           D3D12_RESOURCE_STATE_PIXEL_SHADER_RESOURCE);
```

COMPUTE PSOS ARE SEPARATE OBJECTS

`CreateComputePipelineState` takes a much smaller desc — a root signature and a compute shader, nothing else. There is no blend state, no rasteriser, no render-target format, because none of those stages exist. That also means a compute PSO is far cheaper to create than a graphics one, which is why compute-heavy engines have far fewer PSO problems.

Groupshared memory

```
/* The whole reason thread groups exist: a fast scratchpad
   the group shares. Same hardware as CUDA's __shared__, same
   banking rules (chapter 4).                                  */

groupshared float tile[10][10];      // 8x8 plus a 1-pixel
                                     // border

[numthreads(8, 8, 1)]
void CSMain(uint3 gid : SV_GroupID,
            uint3 tid : SV_GroupThreadID,
            uint3 did : SV_DispatchThreadID)
{
    /* COOPERATIVE LOAD: each thread fetches one texel, and a
       few also fetch the border. The 3x3 blur below then
       reads 9 texels per thread from GROUPSHARED memory
       instead of 9 from the texture cache. */
    tile[tid.y + 1][tid.x + 1] = Input[did.xy].r;
    if (tid.x == 0) tile[tid.y+1][0] = Input[did.xy - int2(1,0)].r;
    if (tid.x == 7) tile[tid.y+1][9] = Input[did.xy + int2(1,0)].r;
    /* ... and the same for y, and the corners ... */

    GroupMemoryBarrierWithGroupSync();
    /* The barrier. THREE VARIANTS, and choosing wrong is
       either a bug or wasted time:
         GroupMemoryBarrier()               groupshared only,
                                            no thread sync
         GroupMemoryBarrierWithGroupSync()  groupshared + all
                                            threads arrive
         DeviceMemoryBarrierWithGroupSync() also UAV/texture
                                            writes - slower  */

    float sum = 0;
    for (int dy = 0; dy < 3; dy++)
        for (int dx = 0; dx < 3; dx++)
            sum += tile[tid.y + dy][tid.x + dx];
    Output[did.xy] = sum / 9;
}

/* THE SAVING: without the tile, a 3x3 blur reads each texel
   NINE times. With it, once. On a bandwidth-bound filter
   that is close to a 9x reduction in traffic.

   THE LIMITS:
     * 32 KB per group typically (16 KB on older hardware)
     * more groupshared -> fewer groups resident -> lower
       occupancy (chapter 5). It is the same budget trade as
       registers.
     * ALL THREADS IN THE GROUP MUST REACH THE BARRIER. Putting
       GroupMemoryBarrierWithGroupSync inside a divergent
       branch hangs the group - and on a GPU that means a TDR
       and a device removal.                                  */
```

THE EARLY-RETURN BUG

`if (any(id.xy >= Size)) return;` is correct in the first example and *fatal* in the second: threads that returned never reach the barrier, so the rest of the group waits for ever. When a group uses barriers, every thread must run to the end — clamp the coordinates instead of returning, or guard only the final write. This produces a GPU hang, a two-second TDR and a device removal, which is a confusing way to discover a one-line mistake.

The patterns worth knowing

```
/* PARALLEL REDUCTION - sum a buffer. The canonical example,
   and the structure of half of all compute work. */

groupshared float partial[256];

[numthreads(256, 1, 1)]
void Reduce(uint3 did : SV_DispatchThreadID,
            uint  idx : SV_GroupIndex,
            uint3 gid : SV_GroupID)
{
    partial[idx] = (did.x < Count) ? Input[did.x] : 0.0;
    GroupMemoryBarrierWithGroupSync();

    /* Tree reduction: halve the active threads each step. */
    for (uint s = 128; s > 0; s >>= 1) {
        if (idx < s) partial[idx] += partial[idx + s];
        GroupMemoryBarrierWithGroupSync();
        /* the barrier is OUTSIDE the if - every thread reaches
           it, including the ones with nothing to do */
    }

    if (idx == 0) Output[gid.x] = partial[0];
    /* one partial per group. Dispatch again over those, until
       one value remains. */
}

/* BETTER: use wave intrinsics for the last steps and skip
   most of the barriers entirely (chapter 2): */
    float v = WaveActiveSum(myValue);        // whole wave, no
                                             // memory, no
                                             // barrier
    if (WaveIsFirstLane()) partial[waveId] = v;
    GroupMemoryBarrierWithGroupSync();       // ONE barrier
    if (idx < numWaves) {
        v = WaveActiveSum(partial[idx]);
        if (idx == 0) Output[gid.x] = v;
    }
/* Two barriers instead of eight, and far less groupshared
   traffic. This is the modern shape of a reduction. */

/* ATOMICS - when you need one global result and can tolerate
   contention: */
    InterlockedAdd(Counter[0], 1);
    InterlockedMax(MaxVal[0], asuint(myValue));   // float via
                                                  // bit trick
/* Every thread hitting one address serialises at L2. AGGREGATE
   FIRST - reduce within the wave, then have one lane do the
   atomic. That turns 32 atomics into 1: */
    uint total = WaveActiveCountBits(predicate);
    if (WaveIsFirstLane()) InterlockedAdd(Counter[0], total);

/* APPEND BUFFERS - a GPU-side vector, for culling and
   compaction: */
    AppendStructuredBuffer<Item> Visible : register(u1);
    if (IsVisible(item)) Visible.Append(item);
/* The counter is an atomic underneath, so the same
   aggregation advice applies for hot cases. */

/* INDIRECT DISPATCH - the GPU decides its own work size,
   with no CPU round trip: */
```

```
    cmd->ExecuteIndirect(cmdSig, 1, argBuffer, 0, nullptr, 0);
/* argBuffer was written by an earlier compute pass. This is
   how GPU-driven rendering works: cull on the GPU, write the
   draw arguments, and execute them without the CPU ever
   learning how many objects survived.                    */
```

What compute is used for

Use	Why compute rather than a pixel shader
Post-processing: blur, bloom, tonemap	Groupshared memory removes redundant texture reads; no rasteriser overhead
Particle simulation	Read-write buffers; no vertex/fragment structure to fight
Culling and LOD selection	Feeds `ExecuteIndirect`; keeps the CPU out of the loop
Light binning (clustered/forward+)	Arbitrary data structures, atomics, append buffers
Skinning, cloth, physics	Large uniform data-parallel work
Mipmap and BVH generation	Hierarchical, needs multiple dispatches with barriers
ML inference	What tensor cores are for; usually via DirectML or ONNX rather than by hand
Sorting, scan, histogram	Classic parallel primitives; use a library

> **ASYNC COMPUTE IS WHERE THE FREE PERFORMANCE IS**
>
> A shadow pass is geometry- and depth-bound; a compute post-process is ALU-bound. Run them on separate queues (chapter 13) and they fill each other's idle units — typically 5–15% of frame time for no algorithmic change. It is one of the few optimisations that costs only plumbing, and it is why compute-shader post-processing displaced pixel-shader post-processing.

How it shows up

Symptom	Cause
GPU hang, TDR, device removed	A barrier not reached by every thread — usually an early `return`
Wrong results at tile edges	Cooperative load missing the border, or a missing barrier
Reader sees stale data	No UAV barrier between the writing and reading dispatches
Bindings appear to do nothing	`SetGraphicsRoot*` used instead of `SetComputeRoot*`
Last row or column of the image is wrong	Grid not rounded up, or the bounds check is missing
Occupancy collapses after adding groupshared	Groupshared is a budget like registers
Atomics dominate the profile	Every thread hitting one address — aggregate per wave first
Group size 32 is slower than 64 on AMD	Half a wavefront idle on wave64 hardware
Async compute gives no gain	Over-synchronised, or both passes bound by the same unit
Reduction result is non-deterministic	Floating-point addition is not associative — expected, not a bug

Check yourself

- ✓ Compute the total thread count from a dispatch and a `numthreads` declaration.
- ✓ Name the four system-value IDs and what each gives you.
- ✓ Say what can synchronise within a group and what cannot between groups.

- ✓ Explain why group size should be a multiple of the wave size.
- ✓ Write a cooperative tile load and say what traffic it saves.
- ✓ Name the three barrier intrinsics and when each applies.
- ✓ Explain the early-return bug and its symptom.
- ✓ Write a tree reduction, then rewrite it with wave intrinsics and count the barriers saved.
- ✓ Explain wave aggregation for atomics and the factor it saves.
- ✓ Describe what `ExecuteIndirect` enables that the CPU path cannot.

Profiling and debugging a GPU

You cannot step through a shader with a breakpoint, and the thing you want to time runs asynchronously on another processor. GPU debugging is therefore a different discipline: capture a frame, inspect state, and measure with the right counter — in that order.

Find the bottleneck before optimising

```
FOUR THINGS CAN LIMIT A FRAME. Optimising the wrong one
changes nothing, and this is the most common wasted week in
graphics work.

1. CPU-BOUND
     GPU idle, waiting for commands.
     SIGN: GPU busy % well below 100. Frame time tracks CPU
           time.
     FIX: fewer draw calls, instancing, multi-threaded
          recording, fewer PSO switches.

2. GEOMETRY / VERTEX BOUND
     SIGN: frame time scales with triangle count; the
           vertex shader dominates.
     FIX: LODs, better culling, fewer vertices. On mobile
          this hits far sooner (chapter 10).

3. PIXEL / FILL BOUND
     SIGN: frame time scales with RESOLUTION. Halve the
           render resolution and it nearly halves.
     FIX: reduce overdraw, cheaper pixel shaders, smaller
          render targets, better early-Z.

4. BANDWIDTH BOUND
     SIGN: frame time does NOT change when you simplify the
           shader, but DOES when you shrink texture formats
           or render-target sizes.
     FIX: compression, fewer/smaller targets, tighter
          formats, better locality.

THE TWO-MINUTE TEST THAT IDENTIFIES ALL FOUR

  halve the resolution   -> much faster? PIXEL or BANDWIDTH
  replace the pixel
    shader with
    return float4(1,0,0,1)  -> much faster? PIXEL
                            -> no change?   BANDWIDTH or
                                            GEOMETRY
  draw half the objects  -> much faster? CPU or GEOMETRY
  do the same draws but
    with an empty
    command list         -> still slow? CPU-BOUND

Do this before opening a profiler. It takes minutes and it
stops you optimising the wrong thing.
```

The tools

Tool	For	Notes
PIX on Windows	DX12 frame capture and timing	The primary DX12 tool. Timing captures need hardware counters enabled
RenderDoc	Frame capture, state inspection, shader debugging	Free, cross-API, cross-platform. Best for *correctness*; deliberately not a timing tool
Nsight Graphics	NVIDIA: deep pipeline analysis	Tells you which unit is the limiter, per draw
Nsight Compute	NVIDIA: CUDA and compute kernels	Occupancy, divergence, memory efficiency, source attribution
Radeon GPU Profiler	AMD: wave occupancy timeline	Shows async compute overlap better than anything else
Arm/Mali Streamline, **Adreno GPA**, **Xcode GPU debugger**	Mobile	The only way to see tiler-specific costs
Timestamp queries	Your own in-engine timings	Always ship these — they work on user machines where no tool is installed

RENDERDOC FOR "WHY IS IT WRONG", PIX FOR "WHY IS IT SLOW"

RenderDoc lets you click a draw, see every bound resource, step through a pixel shader for a chosen pixel, and view the history of every write to that pixel. That is the fastest way to answer a correctness question. PIX and the vendor tools have the hardware counters that answer performance questions. Using the wrong one wastes an afternoon.

Timing it yourself

```
/* GPU timestamps: the GPU writes a counter into a buffer at
   points you choose. You read them back a few frames later -
   never immediately, or you stall (chapter 16). */

D3D12_QUERY_HEAP_DESC qd = {};
qd.Type  = D3D12_QUERY_HEAP_TYPE_TIMESTAMP;
qd.Count = 64;                    /* 32 pairs */
device->CreateQueryHeap(&qd, IID_PPV_ARGS(&queryHeap));

/* Results land in a READBACK buffer. */
CreateBuffer(D3D12_HEAP_TYPE_READBACK, 64 * sizeof(UINT64),
             &readback);

/* The frequency is per-QUEUE and can change with clocks - ask
   each frame, do not cache it once. */
UINT64 freq = 0;
queue->GetTimestampFrequency(&freq);

/* --- recording --- */
cmd->EndQuery(queryHeap.Get(), D3D12_QUERY_TYPE_TIMESTAMP, 0);
    /* EndQuery, even for the "begin" - a timestamp is a point,
       not a range. Confusing, and it catches everyone once. */
    DrawShadowPass(cmd);
cmd->EndQuery(queryHeap.Get(), D3D12_QUERY_TYPE_TIMESTAMP, 1);

cmd->ResolveQueryData(queryHeap.Get(),
                      D3D12_QUERY_TYPE_TIMESTAMP,
                      0, 2, readback.Get(), 0);

/* --- reading, N frames later --- */
UINT64 *ts = nullptr;
D3D12_RANGE r = { 0, 2 * sizeof(UINT64) };
readback->Map(0, &r, (void **)&ts);
double ms = double(ts[1] - ts[0]) / double(freq) * 1000.0;
readback->Unmap(0, nullptr);

/* CAVEATS THAT MAKE NAIVE TIMINGS LIE

   1. The GPU pipelines aggressively. Two adjacent "passes"
      may overlap, so their measured times sum to more than
      the frame. Timestamps measure WHEN, not HOW LONG.

   2. Clocks vary. A GPU idling at 300 MHz and boosting to
      2.5 GHz gives you an 8x spread on the same work. Lock
      the clocks when benchmarking:
        nvidia-smi -lgc 1500,1500
        or use the tool's "stable power state" option.

   3. The first frame after a PSO change includes the
      compile. Discard the first several frames, always.

   4. Vsync makes everything look like 16.67 ms. Turn it off
      to measure.                                          */
```

Debugging a shader

```
/* You cannot printf from a shader. Four techniques, roughly
   in order of how often they are the right one. */

/* 1. OUTPUT THE VALUE AS COLOUR. Crude, immediate, and it
      answers most questions. */
        return float4(frac(worldPos.xyz * 0.1), 1);   /* is my
                                                         position
                                                         sane? */
        return float4(normal * 0.5 + 0.5, 1);         /* normals */
        return float4(uv, 0, 1);                      /* UVs     */
        return float4(depth.xxx / 100.0, 1);          /* depth   */
/* Values outside [0,1] clip to white or black, so scale
   deliberately - "everything is white" usually means your
   value is 47, not 0.47. */

/* 2. RENDERDOC PIXEL DEBUGGING. Capture, click the pixel,
      "Debug this pixel", and step through the shader with all
      intermediate values shown. This is the closest thing to
      a real debugger and it is remarkably good. */

/* 3. WRITE TO A UAV and read it back. The GPU's printf. */
        RWStructuredBuffer<float4> Debug : register(u7);
        if (all(id.xy == uint2(512, 384)))    /* one pixel only */
                Debug[0] = float4(computedValue, 0, 0, 1);
/* Guard it to ONE invocation, or thousands of threads race
   for slot 0 and you learn nothing. */

/* 4. WARP/WAVE INSPECTION in Nsight or PIX: pause a dispatch
      and inspect live registers per lane. Heavy, but the only
      way to see divergence directly. */

/* AND THE STRUCTURAL ONE: bisect the shader. Replace the
   second half with a constant, see if the artefact persists,
   and halve again. Four iterations localises almost any
   shader bug, and it needs no tools at all.               */
```

> **TIMING CAPTURES NEED PERMISSION**
>
> PIX and the vendor profilers read hardware performance counters, which are disabled by default because they can leak information between processes. On Windows you need *Developer Mode* plus, for NVIDIA, the "allow access to GPU performance counters" setting in the control panel. The symptom is a capture that opens with every timing showing zero, which looks like a tool bug and is a permissions problem.

Make captures readable

```
/* An unmarked capture is 4,000 anonymous draws. Ten minutes
   of markers turns it into a readable frame. */

#include <pix3.h>

PIXBeginEvent(cmd, PIX_COLOR(255,0,0), "Shadow Pass");
    for (auto &light : lights) {
        PIXBeginEvent(cmd, PIX_COLOR(255,128,0),
                      "Light %d", light.id);
        DrawShadowMap(cmd, light);
        PIXEndEvent(cmd);
    }
PIXEndEvent(cmd);

/* An RAII wrapper so an early return cannot unbalance them: */
struct GpuScope {
    ID3D12GraphicsCommandList *c;
    GpuScope(ID3D12GraphicsCommandList *cl, const char *n,
             UINT col = PIX_COLOR_DEFAULT) : c(cl)
    { PIXBeginEvent(c, col, n); }
    ~GpuScope() { PIXEndEvent(c); }
};
#define GPU_SCOPE(cmd, name) GpuScope _s##__LINE__(cmd, name)

/* Plus SetName on every resource and PSO (chapter 12).
   Together these two habits are the difference between a
   capture you can read and one you cannot. */

/* Vulkan: vkCmdBeginDebugUtilsLabelEXT
   Metal:  pushDebugGroup / popDebugGroup
   All three tools understand all three.                    */
```

Reading the counters that matter

```
# NVIDIA, a compute kernel:
$ ncu --set full -o report ./app

  Speed Of Light
    Compute (SM) Throughput          18.2%
    Memory Throughput                91.4%     <-- the limiter
  Occupancy
    Achieved Occupancy               24.1%
    Theoretical Occupancy            50.0%     <-- and a gap
  Memory Workload Analysis
    L1 Hit Rate                      12.3%
    Global Load Efficiency           23.0%     <-- uncoalesced

# Read it top-down: memory-bound, poor coalescing, and only
# half the theoretical occupancy achieved. Fix the access
# pattern (chapter 4) before touching anything else.

# The gap between achieved and theoretical occupancy is
# usually load imbalance or the tail effect (chapter 5).

# AMD:
$ rgp ./app          # then open in Radeon GPU Profiler
# The wavefront timeline shows async compute overlap directly.

# Mobile:
$ Streamline / Adreno GPA - look for tile load/store cost
# above all else (chapter 10).
```

How it shows up

Symptom	Cause
Optimised the shader, no change	Not pixel-bound — run the four-way test first
Capture shows all timings as zero	Performance counters not permitted; enable Developer Mode
Pass timings sum to more than the frame	Passes overlap; timestamps measure when, not how long
Timings vary 3× run to run	Clock boost — lock the clocks
First frame is 50× slower	PSO compilation; discard warm-up frames
Everything measures 16.67 ms	Vsync
Debug UAV output is garbage	Every thread writing the same slot — guard to one invocation
Debug colour output is entirely white	Value outside [0,1]; scale it
Capture is 4,000 unnamed draws	No debug markers or resource names
Achieved occupancy far below theoretical	Load imbalance or the tail effect
Frame time doubles when another app opens	VRAM eviction (chapter 16)

Check yourself

- ✓ Name the four possible bottlenecks and the sign of each.
- ✓ Give the four-step test that identifies the bottleneck without a profiler.
- ✓ Say which tool to reach for on a correctness question and which on a performance one.

- ✓ Explain why timestamps measure when rather than how long.
- ✓ Give three reasons a naive GPU timing lies.
- ✓ Name four ways to debug a shader and when each applies.
- ✓ Explain why a debug UAV write must be guarded to one invocation.
- ✓ Say why a capture might show all timings as zero.
- ✓ Read a Speed-of-Light report and say what to fix first.
- ✓ Explain what a gap between achieved and theoretical occupancy usually means.

What a GPU driver actually does

Between your `Draw` call and the silicon sits several hundred thousand lines of someone else's code, split across a user-mode library and a kernel driver. Knowing what each half does closes the loop between this track and the Linux and Windows driver tracks — and explains most of the odd behaviour you have met along the way.

The two halves

```
    YOUR APPLICATION
         |  d3d12.dll / libvulkan.so
         v
    +-------------------------------------------------------+
    |  USER-MODE DRIVER  (UMD)                              |
    |    runs IN YOUR PROCESS, at your privilege            |
    |                                                       |
    |    * compiles DXIL/SPIR-V -> the GPU's real ISA       |
    |    * translates API calls into hardware COMMAND       |
    |      PACKETS, written into a buffer                   |
    |    * manages descriptors and their hardware layout    |
    |    * per-application workarounds and shader           |
    |      replacements, keyed on the executable name       |
    |                                                       |
    |    A CRASH HERE CRASHES YOUR APP, and nothing else.   |
    |    This is why most of the driver lives here.         |
    +-------------------------------------------------------+
         |  ioctl / DeviceIoControl  - the ONLY kernel entry
         v
    +-------------------------------------------------------+
    |  KERNEL-MODE DRIVER  (KMD)                            |
    |    ring 0. Shared by every process on the machine.    |
    |                                                       |
    |    * SUBMITS command buffers to the hardware rings    |
    |    * SCHEDULES between processes - your game and the  |
    |      compositor and the browser all want the GPU      |
    |    * manages GPU VIRTUAL MEMORY and page tables       |
    |    * handles interrupts and fence completion          |
    |    * power and clock management                       |
    |    * RESETS the GPU when something hangs (TDR)        |
    |                                                       |
    |    A CRASH HERE IS A BUGCHECK / KERNEL PANIC.         |
    +-------------------------------------------------------+
         |  MMIO, doorbells, DMA
         v
    THE GPU

    THE SPLIT IS DELIBERATE: everything that can be done at
    application privilege is, so that the enormous, fast-moving,
    game-specific part of the driver cannot take down the
    machine. Only the parts that genuinely need ring 0 -
    submission, memory mapping, interrupts, arbitration between
    processes - live in the kernel.
```

> **THIS IS EXACTLY THE SPLIT YOU LEARNED IN THE DRIVER TRACKS**
> A GPU driver is the largest example of the same architecture: a user-mode component doing the complex, untrusted work, and a small kernel component owning the hardware, the address space and the arbitration. Compare Windows' WDM/KMDF/UMDF split and the Linux DRM/KMS design — the reasoning is identical, and GPU drivers are simply where it is pushed hardest.

How a draw actually reaches the GPU

```
/* Follow one Draw all the way down. */

  1. YOUR CODE
        cmd->DrawInstanced(3, 1, 0, 0);

  2. UMD, in your process
        appends a hardware command packet to the command
        allocator's memory. On AMD that is a PM4 packet; on
        NVIDIA a method write. Perhaps 20-100 bytes.
        NO SYSCALL. This is why DX12 draw calls are ~0.1 us.

  3. ExecuteCommandLists
        UMD builds a submission descriptor and calls into the
        KMD. ONE ioctl for the whole list - not one per draw.

  4. KMD
        * validates the buffer belongs to you
        * ensures every referenced resource is RESIDENT and
          mapped in this process's GPU page tables
        * places the submission in a hardware RING BUFFER
        * writes a DOORBELL register - an MMIO write that
          tells the GPU "there is work"

  5. GPU COMMAND PROCESSOR
        fetches from the ring, decodes packets, and feeds the
        front end. The GPU is now executing.

  6. COMPLETION
        the GPU writes a fence value to memory and raises an
        INTERRUPT. The KMD's ISR runs, signals the OS
        synchronisation object, and your WaitForSingleObject
        returns.

  NOTE how few kernel transitions there are: one per SUBMIT,
  not one per draw. That is the single biggest structural
  reason DX12 and Vulkan are faster than DX11 and OpenGL -
  the batching is explicit and under your control.        */
```

GPU virtual memory

```
Modern GPUs have an MMU and per-process page tables, for
the same reasons a CPU does.

  GPU virtual address
       |  GPU page tables, per process
       v
  physical: VRAM, or system RAM over PCIe, or migrated
            between them by the driver

WHAT THIS BUYS
  * isolation: one process cannot read another's textures.
    Before per-process GPU page tables this was genuinely
    not true, and it was a real vulnerability class.
  * OVERSUBSCRIPTION: allocate more than VRAM and let the
    driver page. Slow, but it works instead of failing.
  * a stable address for a resource across residency
    changes - GetGPUVirtualAddress() stays valid even if
    the pages move.

WHAT IT COSTS
  * a GPU page fault is enormously expensive - the whole
    wave stalls, and on older hardware the whole GPU
  * this is what a DRED page-fault report is telling you
    (chapter 12): a shader dereferenced a GPU VA that is
    not mapped, almost always a freed resource or a
    descriptor pointing at nothing

IOMMU / SMMU applies here too: on a system with an IOMMU
the GPU's DMA is itself translated, which is what makes
GPU passthrough to a VM safe (virtualisation chapter 16).
```

Scheduling, preemption and TDR

```
/* THE GPU IS SHARED. Your game, the compositor, the browser
   playing video and Windows' own UI all submit work. The KMD
   arbitrates.                                              */

  PREEMPTION GRANULARITY - how quickly a running job can be
  interrupted:

    DRAW-LEVEL     between draw calls. Old hardware.
    PRIMITIVE      between triangles.
    PIXEL / WAVE   mid-draw. Modern hardware.
    INSTRUCTION    fully preemptible. Newest.

  Coarse preemption is why a single enormous draw or dispatch
  can make the whole desktop stutter: nothing else can run
  until it finishes.

/* TDR - TIMEOUT DETECTION AND RECOVERY

   If a submission does not complete within ~2 seconds,
   Windows assumes the GPU is hung:
     1. the KMD is asked to reset the GPU
     2. every application's device is marked REMOVED
     3. your app gets DXGI_ERROR_DEVICE_REMOVED with
        DXGI_ERROR_DEVICE_RESET (chapter 12)

   CAUSES, in order of likelihood:
     * an infinite loop in a shader
     * a compute dispatch that is genuinely too large - split
       it, or it will TDR on slower GPUs even though it works
       on yours
     * a barrier a group never reaches (chapter 18)
     * an actual driver or hardware fault

   THE REGISTRY KEY people reach for:
     HKLM\System\CurrentControlSet\Control\GraphicsDrivers
        TdrDelay = 60
   Raise it FOR DEBUGGING ONLY. Shipping software that needs
   it has a bug, and users will not have set it.

   Linux equivalents: amdgpu.lockup_timeout, and the
   GPU reset paths in i915/amdgpu. Same idea, same causes. */
```

> **APPLICATION PROFILES ARE REAL, AND THEY EXPLAIN A LOT**
>
> Drivers ship per-executable profiles that change defaults, force settings, and in some cases *substitute entire shaders* with hand-optimised replacements. This is why renaming your executable can change performance, why a driver update can alter behaviour with no change on your side, and why benchmarking against a shipped game tells you about the profile as much as the hardware. When something behaves differently on one vendor for no reason you can find, this is a real candidate.

The open-source stacks

Layer	Linux	Windows
Kernel driver	`drivers/gpu/drm/` — amdgpu, i915/xe, nouveau, panfrost, msm	WDDM miniport (vendor, closed)

Layer	Linux	Windows
Kernel framework	DRM/KMS — mode setting, GEM/TTM memory, `dma-fence`, scheduler	DXGkrnl
User-mode driver	Mesa: RADV, ANV, NVK, turnip, panvk	Vendor UMD (closed)
Shader compiler	NIR + ACO or LLVM	Vendor back end
Interface	libdrm ioctls	D3DKMT ioctls

Mesa is the best code to read if you want to see this for real: `src/amd/vulkan` (RADV) is a complete, readable Vulkan driver, and `drivers/gpu/drm/amdgpu` is a complete kernel driver for the same hardware. Between them they contain every idea in this chapter, in code you can build and instrument — which is a rare thing for hardware this complex.

```
# Look at what the driver is doing on a Linux box.

$ lsmod | grep -E 'amdgpu|i915|nouveau'
$ dmesg | grep -i drm
[drm] amdgpu kernel modesetting enabled.
[drm] Initialized amdgpu 3.57.0 for 0000:03:00.0 on minor 0

# Every GPU client, and what it is using:
$ sudo cat /sys/kernel/debug/dri/0/clients
$ sudo cat /sys/kernel/debug/dri/0/amdgpu_gem_info
$ sudo cat /sys/kernel/debug/dri/0/amdgpu_vm_info    # page tables

# The ring buffers - the actual submission queues:
$ sudo cat /sys/kernel/debug/dri/0/amdgpu_ring_gfx | head

# A GPU hang, after the fact:
$ sudo cat /sys/kernel/debug/dri/0/amdgpu_gpu_recover
$ dmesg | grep -i 'ring gfx timeout\|GPU reset'

# Trace submissions as they happen:
$ sudo perf trace -e 'amdgpu:*' -a
$ sudo cat /sys/kernel/debug/tracing/events/amdgpu/available_filters

# And the disassembly of what your shader really became:
$ RADV_DEBUG=shaders ./app 2>shaders.txt
$ head -50 shaders.txt        # actual GCN/RDNA assembly
```

Where this track sits

You have gone from "a GPU hides latency instead of reducing it" to reading the assembly a driver generated for your shader. The natural continuations are Linux kernel and drivers if you want to work on the KMD side, compilers if the shader back end is the interesting part, concurrency because a GPU is the most parallel machine you will program, and AI and large language models for the workload that now drives most GPU design decisions.

How it shows up

Symptom	Cause
`DEVICE_REMOVED` after a heavy dispatch	TDR — split the work; it will fail on slower GPUs too
Whole desktop stutters during your frame	Coarse preemption and a very long draw or dispatch
DRED reports a page fault	A shader dereferenced an unmapped GPU VA — usually a freed resource or a stale descriptor

Symptom	Cause
Performance changes after renaming the executable	Driver application profile
Behaviour changes after a driver update, no code change	UMD shader compiler or profile change
Works on one vendor, hangs on another	Almost always a missing barrier one driver tolerated
Allocation succeeds but everything is slow	Oversubscribed VRAM; the driver is paging
A GPU crash takes down the whole machine	A KMD bug — rare, and reportable
Raising `TdrDelay` "fixed" it	It did not; you have a shader or dispatch that is too long

Check yourself

- ✓ Draw the UMD/KMD split and say what each half does.
- ✓ Explain why most of a GPU driver lives in user mode.
- ✓ Trace one `Draw` from your call to the GPU executing it, naming each layer.
- ✓ Say how many kernel transitions a thousand draws cost, and why that matters.
- ✓ Give three things GPU virtual memory buys and one it costs.
- ✓ Explain what a DRED page-fault report is telling you.
- ✓ Define preemption granularity and say what coarse granularity causes.
- ✓ Explain TDR, its usual causes, and why raising `TdrDelay` is not a fix.
- ✓ Say what an application profile is and three symptoms of one.
- ✓ Name the Linux equivalents of the UMD and the KMD, and where to read them.

TRACK

Video & image systems

How a picture becomes bytes and back again: colour and pixel formats, the transform-and-predict machinery inside every codec, the fixed-function blocks that run it, and the V4L2 and DRM drivers that feed them.

21 chapters

How an image is represented

An image is a sampled measurement of light, stored as numbers. Every later chapter — colour, compression, drivers — is an argument about which numbers, how many bits each, and in what order in memory. Getting this layer wrong produces green faces, diagonal stripes and off-by-one-row corruption.

A picture is a sampling of a continuous signal

```
The world is continuous. A sensor measures it at a grid of
points, and quantises each measurement to an integer.

  SPATIAL SAMPLING     how many points   -> resolution
  TEMPORAL SAMPLING    how often         -> frame rate
  AMPLITUDE QUANTISING how finely        -> bit depth

EACH HAS ITS OWN ALIASING ARTEFACT, and naming them is
half of image work:

  spatial undersampling  -> MOIRE, jagged edges, texture
                            shimmer. Fix: filter BEFORE
                            sampling (an optical low-pass
                            filter, or mipmaps - GPU
                            chapter 9).

  temporal undersampling -> the WAGON-WHEEL effect, judder,
                            strobing. Fix: motion blur, or
                            a higher frame rate.

  amplitude quantising   -> BANDING in gradients. Fix:
                            more bits, or DITHER - adding
                            noise below the quantisation
                            step converts a visible contour
                            into invisible grain.

Nyquist applies to all three: to represent detail at
frequency f you must sample at more than 2f. Everything a
codec does is a controlled violation of this, chosen so the
error lands where the eye will not look.
```

Bit depth	Levels	Where used
1	2	Fax, e-ink, masks
8	256	The web, SDR video, most sensors' output after processing
10	1024	HDR video (HEVC Main10, AV1), broadcast. The current default for anything serious
12	4096	Cinema, high-end capture
16 integer	65536	Raw sensor data, intermediate processing
16 float (half)	~1000 with huge range	HDR compositing, GPU render targets
32 float	—	Scientific, film scanning, linear-light working space

WHY 8 BITS IS NOT ENOUGH FOR A GRADIENT

A smooth sky spanning 100 pixels between two adjacent 8-bit levels produces a visible step — the eye is extremely good at spotting an edge in an otherwise flat area, far better than at judging absolute brightness. This is *banding*, and it is the single most common visible defect in 8-bit video. Ten bits mostly fixes it; dithering fixes it at 8 bits by trading a contour for noise, which the eye forgives.

Planar, semi-planar and packed

```
The same 4 pixels, three ways of arranging them in memory.

PACKED / INTERLEAVED  - components of one pixel adjacent

  RGBA8:  R0 G0 B0 A0  R1 G1 B1 A1  R2 G2 B2 A2  ...

  + one pointer, one stride
  + perfect for a GPU texture unit reading whole pixels
  - reading only one component wastes bandwidth (GPU
    chapter 4: this is AoS)

PLANAR - each component in its own contiguous plane

  Y  plane:  Y0 Y1 Y2 Y3 Y4 ...
  Cb plane:  U0 U1 ...
  Cr plane:  V0 V1 ...

  + each plane can have a DIFFERENT RESOLUTION - which is
    the whole point, because chroma is subsampled
    (chapter 3)
  + a filter that only needs luma reads one plane
  - three pointers, three strides, three allocations

SEMI-PLANAR - luma separate, chroma interleaved

  Y  plane:  Y0 Y1 Y2 Y3 ...
  UV plane:  U0 V0 U1 V1 ...

  + TWO planes, and the chroma pair is fetched in one
    access - which is what hardware codecs and display
    controllers actually want
  -> this is NV12, and it is the most common video pixel
     format in the world.

THE RULE OF THUMB
  GPUs and image files like PACKED.
  Codecs and display hardware like SEMI-PLANAR.
  Image processing code likes PLANAR.
Conversions between them are a real cost in every pipeline.
```

Stride is not width

```
/* THE SINGLE MOST COMMON BUG IN IMAGE CODE. */

  WIDTH   how many pixels are meaningful in a row
  STRIDE  how many BYTES from the start of one row to the
          start of the next        (also called PITCH)

  stride >= width * bytes_per_pixel,  and often strictly >

  WHY THERE IS PADDING
    * hardware wants each row aligned to 16, 32, 64, 128 or
      256 bytes so a DMA burst or a SIMD load never straddles
    * a 1920-wide NV12 buffer very often has a 1984 or 2048
      byte stride
    * a sub-image (a crop, a tile) has the PARENT's stride

/* WRONG - assumes packed rows. Works on 1920x1080 by luck,
   corrupts on 1922x1080 with a skew that looks like the
   image was sheared. */
for (int i = 0; i < w * h; i++)
        dst[i] = src[i];

/* RIGHT - one row at a time. */
for (int y = 0; y < h; y++)
        memcpy(dst + y * dst_stride,
               src + y * src_stride,
               w * bytes_per_pixel);   /* copy WIDTH, step
                                          STRIDE               */

/* AND FOR CHROMA in a subsampled format, the chroma plane has
   its OWN stride and its own height: */
for (int y = 0; y < h / 2; y++)
        memcpy(dst_uv + y * dst_uv_stride,
               src_uv + y * src_uv_stride,
               w);                     /* NV12: w bytes = w/2
                                          U + w/2 V pairs      */

/* THE DIAGONAL-SKEW SYMPTOM: if an image looks sheared, with
   each row shifted a little further than the last, you have
   used width where stride was needed. The shift per row is
   exactly (stride - width * bpp).                            */
```

NEGATIVE STRIDE IS LEGAL AND MEANS BOTTOM-UP

Windows DIBs, some capture APIs and BMP files store rows bottom-to-top, expressed as a pointer to the *last* row and a negative stride. Code that computes `row * stride` handles this automatically; code that computes `abs(stride)` or assumes the buffer starts at the lowest address produces a vertically flipped image or a buffer underrun. If your image is upside down, check the sign before adding a flip.

Sizing a buffer

```
/* Get this wrong and you get a heap overflow that shows up as
   corruption in whatever was allocated next. */

size_t plane_size(int stride, int height)   { return (size_t)stride * height; }

/* RGBA8, 1920x1080, 256-byte aligned rows */
        stride = align_up(1920 * 4, 256);        /* 7936        */
        size   = 7936 * 1080;                     /* 8,570,880  */

/* NV12, 1920x1080 */
        y_stride  = align_up(1920, 128);          /* 1920       */
        y_size    = 1920 * 1080;                  /* 2,073,600  */
        uv_stride = y_stride;                     /* same!      */
        uv_size   = uv_stride * (1080 / 2);       /* 1,036,800  */
        total     = y_size + uv_size;             /* 3,110,400  */
                                                  /* = w*h*1.5  */

/* P010 (10-bit NV12), 1920x1080 - each sample is 16 bits with
   the value in the TOP 10, bottom 6 zero. */
        y_stride  = 1920 * 2;                     /* 3840       */
        total     = 3840 * 1080 * 3 / 2;          /* 6,220,800  */

/* THE TRAPS

   1. ODD DIMENSIONS. A 4:2:0 format needs even width and
      height, because chroma is half in each direction.
      1921x1080 either rounds up (and the extra column is
      garbage) or is rejected. Most hardware requires
      multiples of 2, and many require 16.

   2. THE LAST ROW. Some hardware reads a full aligned row
      even for the final line, so the buffer must be a
      multiple of the alignment, not just stride*height.
      Allocators for video usually round the whole thing up.

   3. ALIGNMENT OF THE BASE POINTER, not just the stride.
      A DMA engine may require the buffer itself to be 4 KB
      aligned (Linux chapter 24).                          */
```

What comes off a sensor

```
A camera sensor does NOT produce RGB. Each photosite
measures one colour, through a BAYER filter:

     R G R G R G          50% green, because the eye is
     G B G B G B          most sensitive to green
     R G R G R G
     G B G B G B

DEMOSAICING interpolates the two missing components at
every site. It is the first and most quality-critical step
of an image pipeline, and it is why a "12 megapixel" camera
does not have 12 million real RGB measurements.

THE FULL ISP PIPELINE (SoC chapter on the home page):

  Bayer raw
    -> black level, defective pixel correction
    -> lens shading correction
    -> white balance
    -> DEMOSAIC                    -> now RGB
    -> colour correction matrix    -> a known colour space
    -> gamma / tone curve          -> now non-linear
    -> noise reduction, sharpening
    -> RGB to YCbCr                -> chapter 3
    -> chroma subsampling
    -> to the encoder

EVERY STAGE IS LOSSY OR OPINIONATED. "Raw" formats
(DNG, CR2) store the Bayer data so the decisions can be
made later; JPEG and video store the output, with the
decisions baked in.
```

How it shows up

Symptom	Cause
Image sheared diagonally	Used width where stride was needed
Image upside down	Negative stride, or a bottom-up format
Visible steps in a sky or gradient	Banding — 8-bit quantisation; use 10-bit or dither
Moiré on fine patterns	Spatial aliasing — filter before downsampling
Heap corruption after decoding	Buffer sized as `w*h*bpp` instead of `stride*h`, or chroma plane forgotten
Right-hand column is garbage	Reading padding as image data — crop to width
Works at 1920×1080, fails at odd sizes	4:2:0 needs even dimensions; alignment assumptions broke
Colours look plausible but wrong	Component order — RGBA versus BGRA (chapter 4)
Bottom of the frame is corrupt	Last-row alignment; the buffer needed rounding up
DMA fails or produces nothing	Base pointer not aligned to the engine's requirement

Check yourself

- ✓ Name the three sampling axes and the aliasing artefact each produces.
- ✓ Explain banding and give two fixes.

- ✓ Say why dithering makes a gradient look better despite adding noise.
- ✓ Distinguish packed, planar and semi-planar, and say which hardware prefers each.
- ✓ Define stride and give the symptom of confusing it with width.
- ✓ Explain what a negative stride means and what breaks if you take its absolute value.
- ✓ Compute the total buffer size for 1920×1080 NV12 and for P010.
- ✓ Say why 4:2:0 requires even dimensions.
- ✓ Explain what a Bayer filter is and why demosaicing is necessary.
- ✓ List the ISP stages between a sensor and an encoder, and say which are irreversible.

Colour, gamma and colour spaces

The number 128 in a pixel does not mean "half as bright as 255". Almost every colour bug in software comes from that one fact, and from code that treats stored values as if they were light.

The single most important idea

```
PIXEL VALUES ARE NOT LINEAR IN LIGHT.

sRGB, roughly:      light = (value / 255) ^ 2.2

  value    relative light
    0        0.000
   64        0.052        <- a QUARTER of the value gives
  128        0.216           5% of the light
  192        0.528        <- HALF the value gives 22%
  255        1.000

WHY IT IS DONE

  Human vision is roughly logarithmic: we distinguish
  dark tones far more finely than bright ones. Storing
  LINEAR light in 8 bits wastes most codes on highlights
  nobody can tell apart, and leaves visible banding in the
  shadows.

  A gamma curve spends the codes where the eye looks. It
  is perceptual compression, and it predates computers -
  CRT phosphors happened to have a response near 2.2, so
  the convention was free.

THE CONSEQUENCE

  Any arithmetic that models LIGHT - averaging, blending,
  resizing, blurring, alpha compositing, lighting - must
  be done in LINEAR space. Doing it on stored values is
  mathematically wrong, and the error is visible.
```

```
/* THE CLASSIC DEMONSTRATION: average black and white.

   In sRGB values:      (0 + 255) / 2  = 128
   128 in sRGB is 21.6% of the light.

   In linear light:     (0.0 + 1.0) / 2 = 0.5
   0.5 linear encodes back to sRGB 188.

   So the "correct" mid-grey is 188, not 128. Averaging in
   the wrong space makes everything TOO DARK, and it is why
   badly-resized images look muddier than the original.   */

/* THE CORRECT SANDWICH */

float srgb_to_linear(float c)      /* decode */
{
        return (c <= 0.04045f) ? c / 12.92f
                               : powf((c + 0.055f) / 1.055f, 2.4f);
        /* the linear segment near zero avoids an infinite
           slope at 0 - it is why sRGB is "gamma 2.2-ish"
           rather than exactly 2.2                          */
}

float linear_to_srgb(float c)      /* encode */
{
        return (c <= 0.0031308f) ? c * 12.92f
                                 : 1.055f * powf(c, 1.0f / 2.4f) - 0.055f;
}

/* resize, blur, blend, light -> ALL between those two calls. */

/* AND THE FREE VERSION: let the hardware do it.
     GPU:  use a _SRGB texture format and an _SRGB render
           target. The sampler decodes on read and the ROP
           encodes on write, for nothing, and it filters in
           linear space which is also correct (GPU chapter 9).
     Display: a framebuffer declared sRGB gets the same
           treatment in the display controller.

   Doing the conversion in a shader is slower AND filters in
   the wrong space. Use the format.                       */
```

ALPHA BLENDING IN SRGB IS WRONG, AND EVERYONE DOES IT

Compositing 50% of a bright colour over a dark one with plain sRGB arithmetic gives a result that is too dark, with a characteristic dark fringe around anti-aliased edges — most visible on white text over a dark background. Browsers, UI toolkits and many game engines do this, partly for speed and partly because "fixing" it changes the appearance of every existing asset. Know that it is wrong, and know why your text looks thin.

What a colour space actually specifies

```
THREE INDEPENDENT THINGS, and conflating them causes most
colour-management confusion:

1. PRIMARIES - which physical red, green and blue.
     Given as CIE xy coordinates. Two displays both
     "showing RGB(255,0,0)" emit different reds if their
     primaries differ. This defines the GAMUT: the set of
     colours representable at all.

2. TRANSFER FUNCTION - how code values map to light.
     sRGB, BT.1886, PQ, HLG, linear, Log-C...
     This is "gamma", above.

3. WHITE POINT - what RGB(255,255,255) means.
     D65 (6500 K) almost everywhere; D50 in print.

A COLOUR SPACE = primaries + transfer + white point.
Change any one and the same numbers mean a different
colour.

THE ONES YOU WILL MEET

  sRGB / BT.709    same primaries. The web and HD video.
                   Slightly different transfer functions,
                   which is a real and annoying subtlety.
  BT.601           SD video. DIFFERENT primaries, and a
                   different YCbCr matrix (chapter 3).
  DCI-P3           wider gamut. Apple displays, cinema.
  BT.2020          very wide. UHD and HDR.
  ACEScg, Rec.2100 working spaces for grading and HDR.

THE CONVERSION between two RGB spaces is a 3x3 matrix,
applied IN LINEAR LIGHT:

  decode transfer -> matrix (RGB_a -> XYZ -> RGB_b)
    -> gamut map if out of range -> encode transfer

Skipping the decode is the same error as before, one level
up.
```

Untagged data is the real problem

```
/* Pixel values alone are meaningless. They need METADATA
   saying which colour space they are in. When that metadata
   is missing, software GUESSES - and different software
   guesses differently.                                    */

  WHERE THE TAG LIVES
    images   ICC profile, or EXIF ColorSpace, or a PNG cHRM
             + gAMA chunk
    video    the "VUI" in the bitstream: colour_primaries,
             transfer_characteristics, matrix_coefficients,
             video_full_range_flag
    surfaces DXGI_COLOR_SPACE_*, VkColorSpaceKHR,
             drm_color_encoding / drm_color_range properties

/* THE FOUR VIDEO TAGS, and they are separate on purpose: */

  colour_primaries          1 = BT.709, 5 = BT.601 PAL,
                            6 = BT.601 NTSC, 9 = BT.2020
  transfer_characteristics  1 = BT.709, 16 = PQ, 18 = HLG
  matrix_coefficients       1 = BT.709, 6 = BT.601,
                            9 = BT.2020 non-constant
  video_full_range_flag     0 = LIMITED (16-235), 1 = FULL
                            (0-255)

/* THAT LAST ONE causes more visible bugs than the other
   three combined - see below. */

# Read what a file actually claims:
$ ffprobe -v error -select_streams v:0 \
    -show_entries stream=color_primaries,color_transfer,color_space,color_range \
    -of default=nw=1 input.mp4
color_range=tv
color_space=bt709
color_transfer=bt709
color_primaries=bt709

# "unknown" for any of them means the next tool will guess.
# The usual guess is: BT.601 below 720 lines, BT.709 at or
# above. Which is why an SD clip scaled to HD sometimes
# shifts colour.
```

LIMITED RANGE VERSUS FULL RANGE

Broadcast video stores luma in **16–235**, not 0–255, leaving headroom and footroom for filter overshoot. If a decoder outputs limited-range data and something downstream treats it as full range, blacks become dark grey and whites become light grey — a *washed out* image. Do the opposite and blacks clip to zero and highlights blow out — *crushed*. This is the single most common video colour bug, it is entirely a metadata problem, and both directions look like a broken display rather than a tagging error.

A correct pipeline, in order

```
SOURCE (tagged: BT.709, limited range, 8-bit)
     |
     |  1. expand range 16-235 -> 0-1
     |  2. YCbCr -> RGB with the BT.709 matrix (chapter 3)
     v
NON-LINEAR RGB, BT.709 primaries
     |
     |  3. decode the transfer function
     v
LINEAR RGB, BT.709 primaries
     |
     |  4. ALL image maths happens HERE:
     |     resize, blur, blend, composite, colour grade
     |
     |  5. matrix to the DISPLAY's primaries
     v
LINEAR RGB, display primaries
     |
     |  6. encode the display's transfer function
     |  7. dither to the output bit depth
     v
DISPLAY

SKIPPING STEP 3 is the commonest error and makes everything
too dark.
SKIPPING STEP 5 is the second, and makes wide-gamut content
oversaturated on an sRGB screen - or sRGB content look flat
on a P3 one.
SKIPPING STEP 7 puts banding back after you paid for
precision.
```

How it shows up

Symptom	Cause
Blacks are grey, whites are dull	Limited-range data treated as full range
Shadows crushed, highlights clipped	Full-range data treated as limited
Downscaled image looks darker than the original	Resizing in gamma space instead of linear
Dark halo around white text on a dark background	Alpha compositing in gamma space
Colours oversaturated on a wide-gamut display	sRGB content shown without a gamut conversion
Colour shifts when an SD clip is upscaled	BT.601 versus BT.709 matrix guessed from height
Screenshot does not match the app	One of them applied a colour transform the other did not
Banding after colour correction	Processed in 8 bits, or no dither on output
Two players show the same file differently	Untagged stream; each guessed
Blending looks right on GPU, wrong on CPU	The GPU used an sRGB format and decoded automatically

Check yourself

- ✓ Explain why pixel values are not linear in light, and why that encoding was chosen.
- ✓ Compute the correct mid-grey between black and white in sRGB, and say why 128 is wrong.
- ✓ Name four operations that must be done in linear light.

- ✓ Say why using an sRGB texture format beats converting in a shader.
- ✓ Name the three things a colour space specifies and give an example that differs in each.
- ✓ List the four video colour tags and what each controls.
- ✓ Explain limited versus full range and the symptom of each mismatch direction.
- ✓ Give the seven steps of a correct colour pipeline in order.
- ✓ Say what happens when steps 3, 5 and 7 are skipped.
- ✓ Read a file's colour tags with `ffprobe` and say what "unknown" implies.

YCbCr and chroma subsampling

The first and largest compression step in every video pipeline happens before the codec runs: separate brightness from colour, then throw away three quarters of the colour. It halves the data for almost no visible loss — and it is why red text on video looks smeared.

Why separate luma from chroma

```
The eye has ~6 million cones and ~120 million rods, and the
cones are concentrated in the fovea. Spatial acuity for
BRIGHTNESS is several times higher than for COLOUR.

So: transform RGB into one brightness channel and two
colour-difference channels, and then store the colour ones
at lower resolution.

  Y   luma          brightness. Keep at full resolution.
  Cb  blue-difference   B - Y, scaled
  Cr  red-difference    R - Y, scaled

In 4:2:0 this saves 50% of the data before any codec has
done anything, and in a blind test almost nobody sees it on
natural imagery.

IT ALSO DECORRELATES. R, G and B in a photograph are highly
correlated - a bright pixel is bright in all three. Y, Cb
and Cr are much less so, and a transform codec compresses
uncorrelated channels far better (chapter 6).

NOTE THE NAMES, because people use them interchangeably and
they are not the same:
  Y'CbCr   digital, gamma-encoded. What video uses.
           The prime means "non-linear".
  YUV      strictly the analogue PAL/SECAM version.
           Universally misused to mean Y'CbCr. Every API
           you will meet says "YUV" and means Y'CbCr.
  YCoCg    a cheaper, reversible variant used in some
           codecs and GPU texture formats.
  Luma vs LUMINANCE: luma (Y') is computed from
           gamma-encoded RGB; luminance (Y) is a linear
           light quantity. They are not the same thing, and
           conflating them is why some "greyscale"
           conversions look wrong.
```

The matrix, and why there are several

```
/* BT.709 (HD), full range, gamma-encoded R'G'B' in [0,1]. */

    Y' =  0.2126 R' + 0.7152 G' + 0.0722 B'
    Cb = (B' - Y') / 1.8556
    Cr = (R' - Y') / 1.5748

/* Note the coefficients sum to 1 and green dominates - the
   eye is most sensitive there. */

/* BT.601 (SD) uses DIFFERENT coefficients: */
    Y' =  0.299  R' + 0.587  G' + 0.114  B'

/* BT.2020 (UHD): */
    Y' =  0.2627 R' + 0.6780 G' + 0.0593 B'

/* USING THE WRONG ONE produces a colour shift that is subtle
   on most content and obvious on saturated reds and greens -
   faces go slightly wrong, skies shift. It is the classic
   "why does my SD clip look off after upscaling" bug
   (chapter 2).                                            */

/* THE INTEGER FORM, which is what code actually runs.
   8-bit, LIMITED range: Y in [16,235], Cb/Cr in [16,240]
   with 128 as neutral.                                    */

static inline void rgb_to_ycbcr_bt709(uint8_t r, uint8_t g, uint8_t b,
                                      uint8_t *y, uint8_t *cb, uint8_t *cr)
{
        /* fixed point, 16-bit fractions, limited range */
        int Y  = ( 4735 * r + 15934 * g +  1608 * b + 32768) >> 16;
        int Cb = (-2607 * r -  8773 * g + 11380 * b + 32768) >> 16;
        int Cr = (11380 * r - 10336 * g -  1044 * b + 32768) >> 16;

        *y  = (uint8_t)clamp(Y  + 16,  16, 235);
        *cb = (uint8_t)clamp(Cb + 128, 16, 240);
        *cr = (uint8_t)clamp(Cr + 128, 16, 240);
}

/* THE FOUR THINGS THAT MUST AGREE between encoder and
   decoder, or the colour is wrong:
     1. the matrix     (601 / 709 / 2020)
     2. the range      (limited / full)
     3. the transfer   (chapter 2)
     4. the primaries  (chapter 2)
   All four are carried in the bitstream's VUI, and all four
   are routinely missing.                                  */
```

The subsampling notation

```
J : a : b   over a conceptual 4x2 block of pixels

  J = width of the region                    (always 4)
  a = number of chroma samples in the TOP row
  b = number of chroma CHANGES in the BOTTOM row
      (b = 0 means "same as the top row", i.e. vertical
       subsampling too)

4:4:4     no subsampling. Chroma at full resolution.
    Y Y Y Y        C C C C
    Y Y Y Y        C C C C
    -> 24 bits/pixel at 8-bit. Screen capture, graphics,
       professional capture, chroma keying.

4:2:2     horizontal only. Half the chroma columns.
    Y Y Y Y        C   C
    Y Y Y Y        C   C
    -> 16 bits/pixel. Broadcast, editing formats, most
       capture cards. Survives colour correction well.

4:2:0     horizontal AND vertical. A quarter the chroma.
    Y Y Y Y        C   C
    Y Y Y Y        (shared with the row above)
    -> 12 bits/pixel. H.264/HEVC/AV1 baseline, every
       streaming service, every phone camera. THE default.

4:1:1     quarter horizontal, full vertical. DV, NTSC
          legacy. Rare now.

THE SAVING
  4:4:4 -> 4:2:2   33% less data
  4:4:4 -> 4:2:0   50% less data
before the codec has compressed anything at all.
```

> **CHROMA SITING IS A REAL THING AND IT IS WHY FORMATS DISAGREE**
>
> When you keep one chroma sample for four luma samples, *where* does it sit? MPEG-2 and H.264 place it left-aligned, vertically centred between the two rows. JPEG and MPEG-1 place it in the centre of the 2×2 block. Getting it wrong shifts colour half a pixel relative to luma — invisible on photographs, clearly visible as coloured fringes on sharp graphics and text. It is signalled by `chroma_sample_loc_type`, and most converters ignore it.

What subsampling actually costs

```
4:2:0 IS NEARLY FREE ON NATURAL IMAGES and expensive on
everything else.

FINE ON              BAD ON
  photographs          text and UI, especially coloured
  faces, skin          thin saturated lines
  natural scenes       screen recordings
  film                 chroma keying / green screen
                       anything to be colour graded
                       red and blue detail specifically

THE RED TEXT PROBLEM, concretely:
  red is almost entirely in Cr. Subsample Cr by 2 in each
  direction and a one-pixel red line becomes a two-pixel
  smear with dark edges. This is why:
    * a screen recording of a terminal looks soft
    * subtitles burned in before encoding look worse than
      subtitles rendered by the player
    * you record screencasts at 4:4:4 if you can, and
      upscale the resolution if you cannot

THE GENERATION-LOSS PROBLEM:
  decode 4:2:0 -> edit -> re-encode 4:2:0 subsamples
  again, on data that was already interpolated. Colour
  detail degrades every pass. Editing formats use 4:2:2 or
  4:4:4 for exactly this reason, and you only go to 4:2:0
  on the final delivery encode.

CHROMA KEYING on 4:2:0 gives ragged edges, because the key
is computed from chroma that only exists at half
resolution. Broadcast shoots 4:2:2 minimum, often 4:4:4.
```

Converting, correctly

```
# The command lines that actually matter, and the flags people
# leave out.

# Encode 4:4:4 - for screen content, if your decoder supports it
$ ffmpeg -i in.mov -c:v libx264 -profile:v high444 \
         -pix_fmt yuv444p -crf 18 out.mp4
# NOTE: many hardware decoders and most browsers do NOT
# support 4:4:4 H.264. Check your target before shipping it.

# The safe compromise for screen content: 4:2:0 at double
# resolution, downscaled on playback. Ugly, and it works.

# Convert 4:2:0 -> 4:2:2 for editing, with a good chroma
# upsampler rather than nearest-neighbour:
$ ffmpeg -i in.mp4 -vf "scale=in_color_matrix=bt709:\
out_color_matrix=bt709,format=yuv422p10le" -c:v prores_ks out.mov

# TAG IT. An untagged file will be guessed at (chapter 2):
$ ffmpeg -i in.y4m -c:v libx264 \
    -color_primaries bt709 -color_trc bt709 \
    -colorspace bt709 -color_range tv \
    -pix_fmt yuv420p out.mp4

# Inspect what you produced:
$ ffprobe -show_streams -select_streams v:0 out.mp4 | \
    grep -E 'pix_fmt|color_'
pix_fmt=yuv420p
color_range=tv
color_space=bt709
color_transfer=bt709
color_primaries=bt709

# THE CHROMA UPSAMPLER MATTERS. ffmpeg's default swscale
# chroma interpolation is fast and mediocre; -sws_flags
# +full_chroma_int+accurate_rnd is measurably better for
# anything you will look at closely.
```

> **DO NOT COLOUR-CORRECT 4:2:0 FOOTAGE AGGRESSIVELY**
>
> Chroma only exists at quarter resolution, so any operation that amplifies colour — saturation boosts, secondary colour keys, heavy grading — amplifies the interpolation artefacts too, producing blocky colour patches that were invisible in the original. If the material will be graded, capture 4:2:2 or better. This is not a preference; it is the reason professional formats exist.

How it shows up

Symptom	Cause
Faces slightly wrong, skies shifted	BT.601 matrix used on HD content, or vice versa
Red or blue text smeared with dark edges	4:2:0 subsampling of saturated colour
Coloured fringes offset half a pixel	Chroma siting mismatch between encoder and decoder
Screen recordings look soft	4:2:0 on screen content — use 4:4:4 or higher resolution
Green-screen key has ragged edges	Keying from subsampled chroma

Symptom	Cause
Colour degrades a little every edit	Repeated 4:2:0 round trips; edit in 4:2:2
Blocky colour patches after a saturation boost	Amplified chroma interpolation artefacts
File plays but colours are wrong in one player only	Untagged stream; players guess differently
4:4:4 file will not play on a phone or in a browser	Hardware decoders usually support 4:2:0 only
Greyscale conversion looks too dark or too light	Luma versus luminance confusion — the wrong coefficients, or done in the wrong space

Check yourself

- ✓ Explain why luma and chroma are separated, giving both the perceptual and the compression reason.
- ✓ Distinguish Y'CbCr from YUV, and luma from luminance.
- ✓ Give the BT.709 luma coefficients and say why green dominates.
- ✓ Name the four properties that must agree between encoder and decoder for correct colour.
- ✓ Decode the `J:a:b` notation and give the bits per pixel for 4:4:4, 4:2:2 and 4:2:0.
- ✓ Explain chroma siting and the artefact from getting it wrong.
- ✓ Say why red text suffers most from 4:2:0.
- ✓ Explain why editing formats use 4:2:2 or better.
- ✓ Explain why chroma keying needs 4:2:2 minimum.
- ✓ Write an `ffmpeg` command that encodes 4:2:0 with correct colour tags.

Pixel formats, strides and tiling

A FourCC is a promise about byte order, plane count, subsampling and bit packing. Different subsystems make that promise using different names for the same layout, and one of them is almost certainly lying about endianness. This chapter is the decoder ring.

The formats you will actually meet

FourCC	Layout	bpp	Where
NV12	Y plane, then interleaved **UV**, 4:2:0	12	The universal video format. Hardware codecs, display controllers, cameras
NV21	Same but **VU** order	12	Android camera preview. The swap is why previews come out purple
I420 / YU12	Three planes: Y, U, V, 4:2:0	12	Software codecs, libavcodec's native layout, WebRTC
YV12	Three planes: Y, **V, U**	12	Same as I420 with the chroma planes swapped. A classic source of purple output
P010	NV12 with 16-bit samples, value in the **top 10 bits**	24	10-bit HDR. Note the 6 zero bits at the bottom
YUY2 / YUYV	Packed 4:2:2, order Y U Y V	16	USB webcams, capture cards
UYVY	Packed 4:2:2, order U Y V Y	16	Broadcast SDI, macOS
RGBA / BGRA	Packed 8-bit	32	GPUs, UI. The order differs by API and by endianness
RGB565	Packed 16-bit, 5-6-5	16	Embedded displays, low-memory framebuffers
AYUV / Y410	Packed 4:4:4	32	Screen content, professional capture

NV12 IS THE FORMAT TO KNOW

If you learn one layout, learn this one. Every hardware encoder, every hardware decoder, every display controller and almost every camera ISP speaks NV12. A pipeline that stays in NV12 end to end never converts; one that drops into RGB in the middle pays a full-frame conversion twice, and on a 4K60 stream that is gigabytes per second of pointless memory traffic.

```
NV12, 8x4 pixels. Two planes.

Y PLANE, y_stride bytes per row, 4 rows
  +-------------------------------+--------+
  | Y00 Y01 Y02 Y03 Y04 Y05 Y06 Y07| padding|
  | Y10 Y11 ...                   | padding|
  | Y20 ...                       | padding|
  | Y30 ...                       | padding|
  +-------------------------------+--------+

UV PLANE, uv_stride bytes per row, 2 rows (half height)
  +-------------------------------+--------+
  | U00 V00 U01 V01 U02 V02 U03 V03| padding|   covers Y
  | U10 V10 U11 V11 U12 V12 U13 V13| padding|   rows 0-1
  +-------------------------------+--------+   and 2-3

ONE (U,V) PAIR SERVES A 2x2 BLOCK OF LUMA.

Addressing:
  Y at (x,y)     = base   + y * y_stride + x
  U at (x,y)     = uvbase + (y/2) * uv_stride + (x/2)*2
  V at (x,y)     = uvbase + (y/2) * uv_stride + (x/2)*2 + 1

The uv plane usually shares the Y stride, and usually
follows the Y plane in one allocation - but NEITHER IS
GUARANTEED. Always use the offsets and strides the
allocator gave you.
```

```
/* Reading a pixel from NV12, correctly. */

struct nv12 {
        uint8_t *y,  *uv;
        int      y_stride, uv_stride;
        int      width, height;
};

static void nv12_get(const struct nv12 *f, int x, int y,
                     uint8_t *Y, uint8_t *U, uint8_t *V)
{
        *Y = f->y[(size_t)y * f->y_stride + x];

        const uint8_t *uv = f->uv + (size_t)(y / 2) * f->uv_stride
                                  + (x / 2) * 2;
        *U = uv[0];
        *V = uv[1];      /* NV21 swaps these two lines */
}

/* CONVERTING NV12 -> RGB, BT.709 limited range. The integer
   form, because this runs per pixel. */
static void nv12_to_rgb(uint8_t Y, uint8_t U, uint8_t V,
                        uint8_t *r, uint8_t *g, uint8_t *b)
{
        int y = (Y - 16)  * 1192;      /* 255/219 in Q10 */
        int u =  U - 128;
        int v =  V - 128;

        int R = (y + 1836 * v)               >> 10;
        int G = (y -  547 * v -  218 * u)    >> 10;
        int B = (y + 2166 * u)               >> 10;

        *r = clamp_u8(R);  *g = clamp_u8(G);  *b = clamp_u8(B);
        /* CLAMPING IS NOT OPTIONAL: limited-range input can
           legally exceed 0-255 after conversion (that is what
           the headroom is FOR), and without a clamp you wrap
           around and get bright speckles in dark areas. */
}

/* AND IN PRACTICE: do not write this loop. Use libyuv, or
   swscale, or a shader, or the hardware colour-space
   converter in your display controller. All three are
   several times faster and already correct. */
```

10-bit packing, and the trap in it

```
/* P010: 16 bits per sample, value in the TOP 10 bits. */

     bit  15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00
          [------------ 10-bit value -------------] 0  0  0  0  0  0
                                                    ^^^^^^^^^^^^^^^^
                                                    must be zero

/* SO: to read the actual value you SHIFT DOWN, and to write
   one you SHIFT UP.                                          */
        uint16_t sample = plane[i];
        uint16_t value  = sample >> 6;        /* 0..1023      */

        plane[i] = value << 6;                /* writing back */

/* FORGETTING THE SHIFT gives you an image that is 64x too
   bright and clipped to white, or 64x too dark - depending on
   which direction you missed.

   THE OTHER 10-BIT PACKINGS, and they are all different:

     P010      16-bit container, value in top 10.   MSB-aligned
     P016      16-bit container, all 16 used
     yuv420p10le  16-bit container, value in BOTTOM 10.
                  LSB-aligned - the OPPOSITE of P010, and this
                  is ffmpeg's native 10-bit layout
     V210      packed 4:2:2, THREE 10-bit samples per 32-bit
               word with 2 bits wasted. Broadcast SDI.
     10-bit MIPI RAW  four pixels in five bytes, the low bits
               of all four gathered in the fifth. Camera
               sensors.

   Converting between P010 and yuv420p10le is a shift by 6 in
   one direction. Getting the direction wrong is one of the
   most common 10-bit bugs, and it looks like a broken
   exposure rather than a bit-packing error.                  */
```

> **THE ENDIANNESS LIE IN RGBA NAMES**
>
> DirectX's `DXGI_FORMAT_B8G8R8A8_UNORM` lists components in *memory order*. OpenGL's `GL_RGBA` with `GL_UNSIGNED_INT_8_8_8_8` lists them in *most-significant-bit-first order of a 32-bit integer*, which on a little-endian machine is the reverse. So `GL_BGRA` + `GL_UNSIGNED_BYTE` matches DXGI's `B8G8R8A8`, but `GL_RGBA` + `GL_UNSIGNED_INT_8_8_8_8_REV` also does. If red and blue are swapped, this is why, and the fix is one enum rather than a byte-shuffling loop.

Tiled and compressed layouts

```
LINEAR layout: row after row. Simple, and bad for 2D
access - a vertical neighbour is `stride` bytes away, in a
different DRAM page.

TILED layout: the image is cut into small rectangles and
each tile is stored contiguously.

  LINEAR                   TILED (4x4 tiles)
  0  1  2  3  4  5         [0  1  4  5] [2  3  6  7]
  6  7  8  9 10 11         [8  9 12 13] [10 11 14 15]

WHY: a 2D access pattern - a filter kernel, a motion
compensation block, a texture sample - touches a small
square. In a tiled layout that square is one or two DRAM
pages instead of sixteen. The bandwidth saving is large,
and every GPU and video block uses it.

THE PROBLEM: tiling is VENDOR AND HARDWARE SPECIFIC and
usually undocumented. A tiled buffer produced by the
decoder cannot be read by the CPU without either
detiling or knowing the exact swizzle.

PLUS FRAMEBUFFER COMPRESSION (AFBC on Arm, DCC on AMD,
UBWC on Qualcomm, Delta Color Compression on NVIDIA):
lossless block compression applied on the fly to save
bandwidth, with a separate metadata plane. Also opaque.

HOW LINUX EXPRESSES THIS: DRM FORMAT MODIFIERS. A buffer is
described by (fourcc, modifier), where the modifier names
the exact tiling and compression:

  DRM_FORMAT_MOD_LINEAR
  DRM_FORMAT_MOD_ARM_AFBC(...)
  DRM_FORMAT_MOD_QCOM_COMPRESSED
  I915_FORMAT_MOD_Y_TILED

NEGOTIATING A COMMON MODIFIER between the camera, the
decoder, the GPU and the display is what makes zero-copy
possible (chapter 18). Get it wrong and something in the
chain silently inserts a full-frame detiling copy - or
produces a garbled image that looks like scrambled
rectangles.
```

Inspecting a buffer

```
# What format is this device actually offering?
$ v4l2-ctl -d /dev/video0 --list-formats-ext
        [0]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
        [1]: 'MJPG' (Motion-JPEG, compressed)
        [2]: 'NV12' (Y/UV 4:2:0)

# Dump raw frames and look at them:
$ v4l2-ctl -d /dev/video0 --set-fmt-video=width=1920,height=1080,pixelformat=NV12 \
           --stream-mmap --stream-count=1 --stream-to=frame.nv12

$ ffplay -f rawvideo -pixel_format nv12 -video_size 1920x1080 frame.nv12
# If it looks right, your format assumption is correct. If the
# bottom third is noise, your stride or plane offsets are wrong.

# Convert and inspect:
$ ffmpeg -f rawvideo -pix_fmt nv12 -s 1920x1080 -i frame.nv12 frame.png

# Every format ffmpeg knows, with its bit depth and plane count:
$ ffmpeg -pix_fmts | head -20

# Check what a DRM buffer really is:
$ sudo cat /sys/kernel/debug/dri/0/framebuffer
framebuffer[42]:
        allocated by = compositor
        format=XR24 little-endian (0x34325258)
        modifier=0x100000000000002        <-- NOT linear
        size=1920x1080
        pitch[0]=7680
```

How it shows up

Symptom	Cause
Image is purple or orange, luma looks fine	U and V swapped — NV12 versus NV21, or I420 versus YV12
Red and blue swapped	RGBA versus BGRA, or an endianness-naming mismatch
Bottom third of the frame is noise	Chroma plane offset or stride wrong
Image sheared diagonally	Width used as stride (chapter 1)
10-bit content is blown out or nearly black	Missing the shift by 6 between P010 and LSB-aligned
Bright speckles in dark areas	No clamp after limited-range conversion
Image looks like scrambled rectangles	A tiled buffer read as linear
Pipeline works but uses far more bandwidth than expected	A hidden detiling or format-conversion copy from a modifier mismatch
Colours wrong only on one platform	Different default matrix or range for the same FourCC
Zero-copy path silently stops working after an update	The negotiated modifier changed; check the DRM framebuffer debug node

Check yourself

✓ Describe NV12's plane layout and write the addressing for Y, U and V at (x, y).

- ✓ Say what distinguishes NV12 from NV21 and I420 from YV12, and the symptom of confusing them.
- ✓ Explain why staying in NV12 through a pipeline matters.
- ✓ Explain P010's bit packing and how it differs from `yuv420p10le`.
- ✓ Say why a clamp is required after limited-range YCbCr to RGB.
- ✓ Explain the RGBA/BGRA endianness naming trap.
- ✓ Explain why tiling exists and what it costs you.
- ✓ Say what a DRM format modifier describes and why negotiating one matters.
- ✓ Dump a raw frame from a V4L2 device and view it with the correct format.
- ✓ Diagnose "the bottom third of my frame is noise".

HDR and wide colour gamut

HDR changes three things at once — the transfer function, the gamut and the bit depth — and most "HDR looks wrong" bugs come from changing one without the others. It also introduces the first transfer function in history that specifies *absolute* brightness, which breaks a lot of assumptions.

What HDR actually changes

```
SDR                          HDR

peak ~100 nits (cd/m2)       peak 1000–10000 nits
BT.709 gamut                 BT.2020 container, P3 in
                             practice
gamma ~2.2, RELATIVE         PQ (absolute) or HLG (relative)
8-bit                        10-bit minimum, 12 for grading

"Nits" = candela per square metre. Useful anchors:

  0.005    the darkest a good OLED can show
  1        a lit interior at night
  100      SDR reference white – a paper page, a monitor
  203      HDR "diffuse white" reference (ITU-R BT.2408)
  1000     a typical consumer HDR TV's peak
  4000     a mastering monitor
  10000    the PQ maximum
  1600000  the sun

THE MISCONCEPTION worth killing early: HDR is not "brighter
everything". It is a WIDER RANGE. Diffuse white – a shirt,
a wall, a page of text – should still sit near 100–200
nits. What HDR adds is HEADROOM for specular highlights,
the sun, a lightbulb, a reflection – things that were
previously clipped to the same white as the shirt.

If an HDR image looks uniformly blinding, someone has
mapped SDR white to HDR peak. That is the most common
authoring error.
```

PQ and HLG

	PQ (ST 2084)	HLG (ARIB STD-B67)
Encodes	**Absolute** nits — 0.0001 to 10,000	**Relative** scene light, like SDR gamma
Code value 940 (10-bit) means	Exactly 10,000 nits	"Peak of whatever this display does"
SDR-compatible?	No — looks very dark on an SDR screen	**Yes**, roughly. Its lower half is close to a gamma curve
Needs display metadata?	Yes — the display must tone-map from mastering peak to its own	No — it adapts by design
Used by	HDR10, HDR10+, Dolby Vision, streaming, UHD Blu-ray	Broadcast, BBC/NHK, live sport, iPhone video
Designed for	Mastered content shown in a dark room	Live production and mixed SDR/HDR distribution

```
/* PQ, the ST 2084 EOTF. Note it returns ABSOLUTE nits. */

#define M1 (2610.0 / 16384.0)          /* 0.1593017578125 */
#define M2 (2523.0 / 4096.0 * 128.0)   /* 78.84375        */
#define C1 (3424.0 / 4096.0)           /* 0.8359375       */
#define C2 (2413.0 / 4096.0 * 32.0)    /* 18.8515625      */
#define C3 (2392.0 / 4096.0 * 32.0)    /* 18.6875         */

double pq_to_nits(double e)            /* e in [0,1]      */
{
        double p  = pow(e, 1.0 / M2);
        double num = fmax(p - C1, 0.0);
        double den = C2 - C3 * p;
        return 10000.0 * pow(num / den, 1.0 / M1);
}

double nits_to_pq(double nits)
{
        double y = fmin(fmax(nits / 10000.0, 0.0), 1.0);
        double p = pow(y, M1);
        return pow((C1 + C2 * p) / (1.0 + C3 * p), M2);
}

/* THE CONSEQUENCE OF "ABSOLUTE": a PQ value is a promise
   about photons. It does not adapt. Show 10,000-nit content
   on a 600-nit display and the display MUST tone-map, and the
   quality of that tone map is now a property of the TV, not
   of your content. Hence the metadata below.             */

/* HLG, by contrast, is relative and has a system gamma that
   depends on the display's peak - it degrades gracefully and
   needs no metadata, which is why broadcasters chose it. */
```

WHY HDR NEEDS 10 BITS, NOT AS A NICETY

PQ spreads its code values over a range 100× wider than SDR. At 8 bits the steps between adjacent codes become visible as banding in gradients — a sky, a wall lit by a lamp — much sooner than in SDR. Ten bits is the practical minimum and it is why HEVC Main10 and AV1 Main (which is 10-bit capable) became the HDR codecs. An "8-bit HDR" file is a red flag.

The metadata

```
/* PQ content must tell the display what it was mastered on,
   so the display knows how to tone-map. Two levels: */

/* 1. MASTERING DISPLAY COLOUR VOLUME (SMPTE ST 2086)
      static, per title. */
     display_primaries  R, G, B in CIE xy
     white_point        xy
     max_display_mastering_luminance   e.g. 1000 nits
     min_display_mastering_luminance   e.g. 0.005 nits

/* 2. CONTENT LIGHT LEVEL (CTA-861.3)
      also static, and often more useful in practice. */
     MaxCLL   Maximum Content Light Level - the brightest
              single pixel in the whole title
     MaxFALL  Maximum Frame-Average Light Level - the
              brightest frame, averaged. This is what
              protects the panel from being asked to run at
              peak across the whole screen.

/* 3. DYNAMIC METADATA - per scene or per frame.
     HDR10+       SMPTE ST 2094-40, open, royalty-free-ish
     Dolby Vision ST 2094-10, proprietary, licensed, and
                  carries a full colour-volume mapping plus
                  optionally a second enhancement layer

   HDR10 = PQ + BT.2020 + 10-bit + static metadata. It is the
   baseline that everything supports, and the one to target
   unless you have a reason not to.                        */

# Setting it with ffmpeg - the syntax is unpleasant and
# exact. The values are in units of 1/50000 for chromaticity
# and 1/10000 nits for luminance.
$ ffmpeg -i in.mov -c:v libx265 -pix_fmt yuv420p10le \
  -x265-params "hdr-opt=1:repeat-headers=1:\
colorprim=bt2020:transfer=smpte2084:colormatrix=bt2020nc:\
master-display=G(13250,34500)B(7500,3000)R(34000,16000)\
WP(15635,16450)L(10000000,1):max-cll=1000,400" \
  out.mp4

# Verify - and this is worth doing every time, because a
# silently-dropped side-data block is the usual failure:
$ ffprobe -show_frames -read_intervals "%+#1" \
    -show_entries frame=side_data_list out.mp4
[SIDE_DATA]
side_data_type=Mastering display metadata
red_x=34000/50000 ...
max_luminance=10000000/10000
[/SIDE_DATA]
[SIDE_DATA]
side_data_type=Content light level metadata
max_content=1000
max_average=400
[/SIDE_DATA]
```

> **METADATA IS THE PART THAT GETS LOST**
> Remux a file, run it through a filter, transcode it with default settings, or pass it through a player that does not understand side data, and the mastering metadata quietly disappears. The video still decodes, the colour tags may survive, and the display now tone-maps with no idea what it is looking at — producing a washed-out or crushed picture that looks like a decoder bug. When HDR "looks wrong", check the metadata survived *before* suspecting anything else.

Tone mapping and gamut mapping

```
TONE MAPPING: 1000 nits of content, 400 nits of display.

CLIP                everything above 400 becomes 400.
                    Highlights lose all detail; a sunset
                    becomes a white disc. Cheap, and bad.

REINHARD            L' = L / (1 + L). Smooth, and it
                    desaturates and flattens the whole
                    image, including the parts that fitted
                    perfectly well.

ROLL-OFF / BT.2390  linear below a knee, a smooth curve
                    above it. Preserves everything that
                    fits and compresses only the excess.
                    THIS IS THE RIGHT DEFAULT.

ACES / HABLE        filmic S-curves from film emulation.
                    Popular in games; more of a look than
                    a mapping.

DYNAMIC             HDR10+/Dolby Vision drive the curve
                    per scene, so a dark scene and a bright
                    one are mapped differently.

GAMUT MAPPING is the same problem in colour: BT.2020
content on a P3 display has saturated colours that simply
do not exist there.

  CLIP        oversaturated colours all become the same
              boundary colour -> flat patches, lost detail
  COMPRESS    squeeze the whole gamut inward -> everything
              slightly duller, but relationships preserved
  PERCEPTUAL  preserve hue and lightness, sacrifice
              saturation. Usually what you want.

AND THE HARD DIRECTION: HDR -> SDR. You are throwing away
most of the range. A naive conversion looks grey and flat;
a good one applies a roll-off AND a saturation boost,
because desaturation is what makes tone-mapped images look
washed out.
```

```
# Tone map HDR to SDR with something reasonable, in linear
# light (chapter 2 - doing it in PQ space is wrong):
$ ffmpeg -i hdr.mp4 -vf "\
zscale=t=linear:npl=100,\
format=gbrpf32le,\
zscale=p=bt709,\
tonemap=tonemap=bt2390:desat=0,\
zscale=t=bt709:m=bt709:r=tv,\
format=yuv420p" \
  -c:v libx264 -crf 20 sdr.mp4

# The chain, read left to right:
#   zscale t=linear   decode PQ to linear light
#   npl=100           "nominal peak luminance" - the anchor
#   format=gbrpf32le  work in float; 8-bit here would band
#   zscale p=bt709    gamut map BT.2020 -> BT.709
#   tonemap=bt2390    the roll-off curve
#   desat=0           do NOT desaturate; bt2390 handles it
#   zscale t=bt709    encode the SDR transfer function
#   format=yuv420p    back to 8-bit 4:2:0

# Getting the ORDER wrong - tone mapping before linearising,
# or gamut mapping after encoding - produces plausible-looking
# output that is measurably wrong, which is worse than an
# obvious failure.
```

How it shows up

Symptom	Cause
HDR content looks washed out and grey	PQ decoded as if it were gamma, or metadata lost so the display tone-mapped blindly
Everything uniformly blinding	SDR white mapped to HDR peak instead of ~203 nits
Highlights are flat white discs	Clipping instead of a roll-off tone map
Banding in HDR skies	8-bit, or an 8-bit intermediate in the chain
HDR file plays but looks like SDR	Transfer tagged as bt709 instead of smpte2084
Metadata present in the master, gone after transcode	Side data dropped — pass `hdr-opt` / `repeat-headers` and verify
Colours oversaturated and blotchy	BT.2020 clipped to the display gamut rather than mapped
Tone-mapped SDR looks lifeless	No saturation compensation after the roll-off
HLG content too dark on an SDR screen	HLG is only *roughly* compatible; it still wants its system gamma applied
Two HDR TVs show the same file very differently	Expected — with static metadata, tone mapping is the display's choice

Check yourself

- ✓ Name the three things HDR changes relative to SDR.
- ✓ Explain why HDR is not "brighter everything", and where diffuse white should sit.
- ✓ Distinguish PQ from HLG in terms of absolute versus relative, and say who uses each.
- ✓ Explain why HDR needs 10 bits.
- ✓ Say what MaxCLL and MaxFALL each describe and why MaxFALL protects the panel.
- ✓ Name the components of HDR10 and say why it is the baseline.

- ✓ Explain why static metadata means two TVs legitimately differ.
- ✓ Compare clipping, Reinhard and BT.2390 roll-off as tone maps.
- ✓ Explain why tone mapping must happen in linear light.
- ✓ Say why naive HDR-to-SDR conversion looks washed out and what fixes it.

Why compression works

Raw 4K60 video is 12 gigabits per second. A streaming service delivers the same picture in 15 megabits — a ratio of 800:1. That is not clever packing; it is the systematic removal of four different kinds of redundancy, three of which you never notice.

The problem, in numbers

```
/* Uncompressed video is not slightly too big. It is
   impossible.                                          */

  bitrate = width * height * bpp * fps

  1920x1080  4:2:0 8-bit  30 fps
        1920 * 1080 * 1.5 * 8 * 30      =  747 Mbit/s
  3840x2160  4:2:0 10-bit 60 fps
        3840 * 2160 * 1.5 * 10 * 60     =  7.5 Gbit/s
  3840x2160  4:4:4 12-bit 60 fps
        3840 * 2160 * 3   * 12 * 60     = 17.9 Gbit/s

  ONE HOUR of the middle one: 3.4 TERABYTES.
  Delivered at 15 Mbit/s: 6.75 GB. A ratio of about 500:1.

  FOR COMPARISON
    a good LOSSLESS image compressor    ~2:1
    lossless video (FFV1, lossless x264) ~2-3:1

  So lossless gets you 2x and we need 500x. Everything
  interesting in this chapter is about the other 250x, and all
  of it is DISCARDING INFORMATION.
```

The four redundancies

```
1. SPATIAL - neighbouring pixels are similar
     A photograph is mostly smooth. Knowing a pixel tells
     you a lot about the one beside it.
     EXPLOITED BY: intra prediction (predict from
     neighbours) and the transform (concentrate energy into
     a few coefficients).                    Chapters 7, 11

2. TEMPORAL - consecutive frames are nearly identical
     At 30 fps, 33 ms apart, most of the frame did not
     change, and what did change mostly MOVED.
     EXPLOITED BY: motion compensation - describe this
     block as "that block over there, shifted".
     THE BIGGEST WIN BY FAR.                    Chapter 9

3. STATISTICAL - symbols are not equally likely
     After prediction, most residuals are near zero. Give
     the common values short codes.
     EXPLOITED BY: entropy coding.               Chapter 8
     This is the only one that is LOSSLESS.

4. PERCEPTUAL - the eye does not see everything
     Less acuity for colour than brightness. Less for high
     spatial frequencies. Less next to a strong edge, and
     for a moment after a scene cut.
     EXPLOITED BY: chroma subsampling (chapter 3) and
     QUANTISATION (chapter 7).
     THIS IS WHERE THE LOSS HAPPENS, deliberately.

THE STRUCTURE OF EVERY CODEC SINCE 1990

  PREDICT   -> guess the block from what the decoder
               already has (neighbours, or an earlier
               frame)
  TRANSFORM -> convert the prediction ERROR to frequency,
               concentrating its energy
  QUANTISE  -> divide and round. THE LOSSY STEP.
  ENTROPY   -> code the result losslessly

and the decoder runs it backwards. H.261 in 1988 and AV1 in
2018 differ in how well each stage is done, not in the
structure.
```

QUANTISATION IS THE ONLY LOSSY STEP

Prediction, transform and entropy coding are all reversible in principle. It is quantisation — dividing coefficients by a step size and rounding — that throws information away, and it is the *only* place that happens. That is why one number, the quantisation parameter, controls quality and bitrate throughout every codec, and why "QP" is the number you actually tune.

The theoretical floor

```
/* Shannon: you cannot losslessly code a source below its
   ENTROPY.

     H = - sum over symbols of  p(s) * log2 p(s)   bits/symbol

   A source where every one of 256 values is equally likely
   has H = 8 bits. Nothing can compress it. Random data does
   not compress - not because compressors are weak, but
   because there is nothing there.

   A source where 0 occurs 90% of the time:
     H = -(0.9*log2(0.9) + 0.1*log2(0.1/255)*255...) ~ 0.9 bits

   So the whole game is: TRANSFORM THE DATA UNTIL ITS ENTROPY
   IS LOW, then code near that floor.

   Prediction does exactly that. Raw pixel values are roughly
   uniform over their range. Prediction RESIDUALS - the error
   after guessing - cluster hard around zero:

     raw pixel values      residuals after prediction
     .....::::::::::::.           :
     ...::::::::::::::...        :::
     .:::::::::::::::::.        ..:::..
     0            255         -50  0  +50

   Same information, far lower entropy. The entropy coder then
   gets close to that floor (chapter 8).                  */
```

Rate-distortion: the actual decision

```
An encoder is not "compressing". It is repeatedly choosing
between options that trade BITS against ERROR.

  Should this block be intra or inter coded?
  Which of 35 intra directions?
  Split into four sub-blocks, or not?
  Which reference frame? Which motion vector?
  What quantisation step?

Each choice costs some RATE (bits) and causes some
DISTORTION (error). You cannot minimise both.

THE LAGRANGIAN, and this one equation is the heart of every
modern encoder:

     J = D + lambda * R

  D  distortion - usually SSD (sum of squared differences)
                or SATD (transformed absolute differences)
  R  rate - the actual bits this choice costs
  lambda  the exchange rate between them, derived from QP

The encoder tries the options, computes J for each, and
picks the smallest. That is RATE-DISTORTION OPTIMISATION,
and it is why encoding is 10-100x slower than decoding.

     distortion
        |*
        | *
        |  *.
        |    *..              the RD CURVE
        |       *....
        |           *.......
        |                  *.........
        +------------------------------ rate

A BETTER CODEC is one whose curve sits lower and left: the
same quality for fewer bits. "AV1 is 30% better than H.264"
means its curve is 30% left at equal quality - measured
with BD-rate, which integrates the gap between two curves.

AND THE PRESET DIAL: "veryslow" searches more options and
lands closer to the true optimum. "ultrafast" tries a
handful. Same codec, same decoder, 2-3x difference in
bitrate for the same quality - which is why encoder
QUALITY is mostly a search problem, not a format one.
```

Measuring what you lost

Metric	What it measures	Trust it?
PSNR	Log of mean squared error	Poorly. Correlates weakly with what people see, but it is cheap, stable and every paper quotes it
SSIM	Structural similarity — luminance, contrast, structure	Better. Still misses temporal artefacts
MS-SSIM	SSIM at several scales	Better again; a reasonable default
VMAF	Netflix's machine-learned fusion of several metrics, trained on human scores	The best automated proxy in common use. Tuned for streaming content on a TV

Metric	What it measures	Trust it?
BD-rate	Average bitrate difference between two codecs at equal quality	The standard way to compare codecs. Depends entirely on which quality metric feeds it
Human eyes	The actual target	The only ground truth, and the reason every metric above is a compromise

```
# Compute them, on a real pair of files.
$ ffmpeg -i encoded.mp4 -i original.y4m \
    -lavfi "[0:v][1:v]libvmaf=feature='name=psnr|name=float_ssim'" \
    -f null -
VMAF score: 93.412
PSNR y:42.31 u:45.02 v:44.88
SSIM: 0.9871

# ROUGH VMAF ANCHORS, for streaming content:
#   95+   visually indistinguishable from the source
#   90    excellent; the usual target for a top rendition
#   80    good; noticeable on an A/B but fine standalone
#   60    watchable, clearly compressed
#   <50   obviously bad

# THE CAVEATS THAT MATTER
#   * VMAF was trained on 1080p TV viewing. Its numbers mean
#     less for a phone, for screen content, or for 4K.
#   * every metric here compares FRAME TO FRAME. None of them
#     sees flicker, pumping, or a scene that breaks up for
#     three frames after a cut - which is what viewers
#     actually complain about.
#   * optimising a metric directly ("VMAF hacking") produces
#     encoders that score well and look worse. Sharpening
#     filters do this reliably.
#   * ALWAYS LOOK AT THE VIDEO. A metric tells you where to
#     look, not whether it is acceptable.
```

> **COMPRESSION ARTEFACTS ARE NOT RANDOM NOISE**
> Each has a signature that names its cause. **Blocking** — visible square edges — is coarse quantisation plus a block transform. **Ringing / mosquito noise** — shimmer around sharp edges — is quantising away high-frequency coefficients (Gibbs). **Banding** in gradients is quantisation plus insufficient bit depth. **Smearing** is motion compensation failing to find a match. **Pumping** — quality visibly oscillating — is rate control. Recognising the artefact tells you which knob to turn.

How it shows up

Symptom	Cause
Visible square blocks	QP too high; deblocking filter disabled or too weak
Shimmering around text and edges	Ringing — high-frequency coefficients quantised away
Gradients show contours	Banding; needs higher bit depth or dither
Quality visibly pulses	Rate control reacting to scene complexity (chapter 13)
Detail smears during fast motion	Motion search failing, or too few bits for a hard scene
File is huge despite a low bitrate setting	Rate control mode misunderstood — CRF ignores bitrate
Encoding takes hours	A slow preset; the search space grew, not the codec

Symptom	Cause
Metrics look fine, viewers complain	A temporal artefact no frame-wise metric sees
Random or already-compressed data will not compress	Its entropy is already at the floor
Re-encoding degrades every pass	Generation loss — each pass quantises already-quantised data

Check yourself

- ✓ Compute the raw bitrate of 4K60 4:2:0 10-bit and the ratio to a 15 Mbit/s stream.
- ✓ Say roughly what lossless compression achieves on video, and why that is not enough.
- ✓ Name the four redundancies and the codec stage that exploits each.
- ✓ Say which single stage is lossy and why that makes QP the master control.
- ✓ State Shannon's entropy bound and explain why random data does not compress.
- ✓ Explain why prediction lowers entropy without losing information.
- ✓ Write the rate-distortion Lagrangian and say what each term is.
- ✓ Explain what an encoder preset changes, and why it affects bitrate but not the decoder.
- ✓ Compare PSNR, SSIM and VMAF, and give two things none of them measure.
- ✓ Name five compression artefacts and the cause of each.

Transform and quantisation: JPEG end to end

JPEG is thirty years old and still the clearest illustration of how every codec works. Follow one 8×8 block from pixels to bits and back, and you have understood the machinery inside H.264, HEVC and AV1 — they differ in the details, not the shape.

The whole encoder

```
RGB image
  |  1. colour transform  -> Y'CbCr           (chapter 3)
  |  2. chroma subsample  -> 4:2:0 or 4:2:2
  v
planes, split into 8x8 BLOCKS
  |  3. level shift: subtract 128 -> range [-128,127]
  |  4. FORWARD DCT       -> 64 frequency coefficients
  |  5. QUANTISE          -> divide by a table, round
  |                          THE LOSSY STEP
  |  6. ZIGZAG scan       -> 1D, low frequencies first
  |  7. DC differential, AC run-length
  |  8. HUFFMAN code                          (chapter 8)
  v
bitstream

The decoder runs it backwards, and only step 5 cannot be
undone.

NOTE THE ORDER: transform BEFORE quantise. Quantising
pixels directly just posterises the image. Quantising
FREQUENCIES lets you keep the ones the eye cares about and
discard the ones it does not - that is the entire
trick.
```

The DCT, concretely

```
/* The 2D DCT-II. 64 pixels in, 64 coefficients out - no
   compression yet, just a change of basis.               */

  F(u,v) = 1/4 C(u) C(v) *
           sum_{x=0..7} sum_{y=0..7}
               f(x,y) * cos((2x+1) u pi / 16)
                      * cos((2y+1) v pi / 16)

  C(0) = 1/sqrt(2),  C(k) = 1 otherwise

/* WHAT THE 64 OUTPUTS MEAN

     F(0,0)  the DC coefficient: the block's average
             brightness. Typically 10-100x larger than
             anything else.
     F(1,0)  a slow horizontal ramp
     F(0,1)  a slow vertical ramp
     F(7,7)  the finest checkerboard the block can hold

   THE PROPERTY THAT MATTERS - ENERGY COMPACTION. On natural
   images, almost all the energy lands in the top-left few
   coefficients. A typical smooth block:

     -415  -30   -61    27     56   -20    -2     0
      4    -22   -61    10     13    -7    -9     5
     -47    7     77   -25    -29    10     5    -6
     -49   12     34   -15    -10     6     2     2
      12   -7    -13    -4     -2     2    -3     3
      -8    3      2    -6     -2     1     4     2
      -1    0      0    -2     -1    -3     4    -1
       0    0     -1    -4     -1     0     1     2

   Fifty of those 64 numbers are single digits. After
   quantisation nearly all of them become ZERO - and a long
   run of zeros costs almost nothing to code.             */

/* NOBODY COMPUTES THE FORMULA. The separable fast DCT does
   8 rows then 8 columns, and the AAN algorithm needs just
   5 multiplies and 29 adds per 8-point transform. Modern
   codecs go further and use INTEGER transforms - see the
   note below.                                            */
```

WHY MODERN CODECS ABANDONED THE TRUE DCT

The real DCT uses cosines, so an encoder and a decoder on different hardware can round differently — and in a video codec that error *accumulates* across every predicted frame, causing drift between encoder and decoder. H.264 onwards therefore use **integer approximations** of the DCT: exactly specified, bit-exact on every implementation, and computable with adds and shifts alone. JPEG predates that insight, which is why two JPEG decoders can legitimately produce slightly different pixels.

Quantisation: where the loss happens

```
/* Divide each coefficient by a per-position step and round. */

     Fq(u,v) = round( F(u,v) / Q(u,v) )

/* The standard JPEG luminance table at quality 50. Note the
   values GROW toward the bottom right - high frequencies get
   coarser steps, because the eye notices them less.       */

     16  11  10  16  24  40  51  61
     12  12  14  19  26  58  60  55
     14  13  16  24  40  57  69  56
     14  17  22  29  51  87  80  62
     18  22  37  56  68 109 103  77
     24  35  55  64  81 104 113  92
     49  64  78  87 103 121 120 101
     72  92  95  98 112 100 103  99

/* Applying it to the block above: */

     -26  -3  -6   2   2  -1   0   0
       0  -2  -4   1   1   0   0   0
      -3   1   5  -1  -1   0   0   0
      -3   1   2  -1   0   0   0   0
       1   0   0   0   0   0   0   0
       0   0   0   0   0   0   0   0
       0   0   0   0   0   0   0   0
       0   0   0   0   0   0   0   0

   FORTY-FOUR ZEROS out of 64, and they are all in the bottom
   right - which is exactly what the zigzag scan is designed
   to exploit.

   THE QUALITY DIAL scales this table:
     quality 90 -> divide the table by ~5   -> finer steps
     quality 50 -> the table as printed
     quality 10 -> multiply by ~5           -> almost
                   everything becomes zero

   CHROMA gets its own, coarser table - another application of
   the same perceptual argument as subsampling.

   AND THE REASON ARTEFACTS LOOK THE WAY THEY DO:
     quantise the DC too coarsely  -> BLOCKING (each block's
                                      average steps visibly)
     quantise the AC too coarsely  -> RINGING around edges,
                                      because a sharp edge
                                      needs its high
                                      frequencies             */
```

Zigzag and run-length

```
The zeros are clustered bottom-right, so scan in the order
that visits low frequencies first:

   0  1  5  6 14 15 27 28
   2  4  7 13 16 26 29 42
   3  8 12 17 25 30 41 43
   9 11 18 24 31 40 44 53
  10 19 23 32 39 45 52 54
  20 22 33 38 46 51 55 60
  21 34 37 47 50 56 59 61
  35 36 48 49 57 58 62 63

Our block becomes:

  -26, -3, 0, -3, -2, -6, 2, -4, 1, -3, 1, 1, 5, 1, 2,
  -1, 1, -1, 2, 0, 0, 0, 0, 0, -1, -1, 0, 0, ... 0

-> a short burst of values, then a long tail of zeros.

THEN:
  DC is coded DIFFERENTIALLY against the previous block's
     DC - adjacent blocks have similar averages, so the
     difference is small.
  AC is coded as (RUN, SIZE) pairs: "5 zeros, then a value
     needing 3 bits". A single EOB symbol ends the block
     once only zeros remain - which is why 44 trailing
     zeros cost about four bits, not 44 values.

Those symbols then go to the Huffman coder (chapter 8).

MODERN CODECS DO THE SAME THING with better tools: adaptive
scan orders that learn where the coefficients actually are,
context-modelled arithmetic coding instead of Huffman, and
several transform sizes rather than only 8x8.
```

Where JPEG stops and video codecs begin

JPEG	What modern codecs added
Fixed 8×8 blocks	4×4 to 64×64, chosen per region — small blocks for detail, large for smooth areas (chapter 11)
No prediction between blocks except the DC	Full **intra prediction** from neighbouring pixels, in 35+ directions
No prediction between images	**Motion compensation** — the biggest win of all (chapter 9)
True DCT, drift-prone	Integer transforms, bit-exact; plus DST and identity transforms for particular content
Huffman coding	Context-adaptive arithmetic coding, ~10–15% better (chapter 8)
Visible block edges	**In-loop deblocking**, so the filtered frame is what gets referenced (chapter 12)
One quality for the whole image	Per-block QP, driven by rate-distortion optimisation and perceptual masking

```
# Look inside a real JPEG.
$ djpeg -verbose -verbose photo.jpg > /dev/null
Quantization table 0:
    3   2   2   3   4   6   8  10
    2   2   2   3   4   9  10   9
    ...
Huffman table 0x00: ...
Start Of Frame 0xc0: width=1920, height=1080, components=3
    Component 1: 2hx2v q=0        <-- luma, 2x2 sampling
    Component 2: 1hx1v q=1        <-- chroma, subsampled 4:2:0
    Component 3: 1hx1v q=1

# The quantisation tables ARE the quality setting - there is no
# "quality 82" stored in the file, only the tables it produced.
# This is how tools guess a JPEG's original quality, and why
# the guess is approximate.

$ identify -format "%Q\n" photo.jpg      # ImageMagick's guess
82

# Re-encoding LOSSLESSLY, to strip metadata or rotate:
$ jpegtran -copy none -optimize -progressive -outfile out.jpg in.jpg
# jpegtran works on the coefficients directly - no decode, no
# re-quantisation, no generation loss. Anything that decodes
# to pixels and re-encodes loses quality every time.
```

GENERATION LOSS IS NOT SYMMETRICAL

Re-encoding a JPEG at the *same* quality still degrades it: the second encoder's 8×8 grid may not align with the first's, and it quantises coefficients that were already quantised. Re-encoding at a *higher* quality is worse than pointless — it makes the file bigger while preserving the original's artefacts faithfully. If you must edit, work from the original, or use coefficient-domain tools like `jpegtran` that never decode to pixels.

How it shows up

Symptom	Cause
Visible 8×8 squares	DC quantised too coarsely; quality too low
Shimmer or halo around text and edges	Ringing — AC coefficients quantised away
Colour bleeding across sharp edges	Chroma subsampling plus a coarser chroma table
File degrades a little every save	Generation loss from decode–re-encode
Higher quality setting produced a bigger, uglier file	Re-encoding an already-lossy source
Two decoders produce slightly different pixels	JPEG's floating-point DCT is not bit-exact
Photographs compress well, screenshots do not	Sharp edges need high frequencies; the DCT spreads them across many coefficients
Rotating a JPEG made it blurrier	A pixel-domain tool re-encoded it; use `jpegtran`
Progressive JPEG loads oddly but is smaller	Working as intended — same coefficients, different scan order

Check yourself

✓ List the eight JPEG encoding steps in order and name the only lossy one.

- ✓ Explain why the transform comes before quantisation.
- ✓ Say what the DC coefficient represents and why it dominates.
- ✓ Define energy compaction and say why it makes natural images compressible.
- ✓ Explain why the quantisation table's values grow toward the bottom right.
- ✓ Say which artefact comes from coarse DC quantisation and which from coarse AC.
- ✓ Explain what the zigzag scan achieves and how EOB exploits it.
- ✓ Explain why modern codecs use integer transforms rather than a true DCT.
- ✓ Name five things video codecs added that JPEG lacks.
- ✓ Explain why re-encoding at a higher quality is worse than pointless.

Entropy coding: Huffman, arithmetic and CABAC

The last stage of every codec, and the only lossless one. It takes the quantised symbols and writes them in as few bits as their statistics allow — and the gap between "as few as Huffman allows" and "as few as information theory allows" is worth about 15% of your bitrate.

Huffman: one integer number of bits per symbol

```
Build a binary tree, most frequent symbols nearest the
root. Symbol probabilities:

  A 0.45   B 0.25   C 0.15   D 0.10   E 0.05

                (1.0)
               /     \
            A(.45)  (.55)
                   /     \
                B(.25)   (.30)
                         /     \
                      C(.15)   (.15)
                               /     \
                            D(.10)  E(.05)

  A = 0        1 bit
  B = 10       2 bits
  C = 110      3 bits
  D = 1110     4 bits
  E = 1111     4 bits

Average = .45(1) + .25(2) + .15(3) + .10(4) + .05(4) = 2.00
Entropy = 2.04... wait, entropy here is 1.98 bits.

So Huffman costs 2.00 where the floor is 1.98. Close - on
THIS distribution.

THE FUNDAMENTAL LIMIT: a Huffman code must use a WHOLE
NUMBER of bits per symbol. When one symbol is very likely,
that rounding is brutal:

  P(zero) = 0.95  ->  ideal cost = -log2(0.95) = 0.074 bits
                      Huffman must spend      = 1 bit
                      THIRTEEN TIMES too many.

And "mostly zeros" is exactly what a quantised transform
block looks like (chapter 7). This is why JPEG's entropy
coder leaves real compression on the table, and why every
codec after it moved to arithmetic coding.

THE WORKAROUND JPEG USES: code (RUN, SIZE) PAIRS rather
than individual coefficients, so one symbol covers many
zeros. It helps, and it is still not enough.
```

Arithmetic coding: fractional bits

```
/* The whole message becomes ONE number in [0,1). Each symbol
   narrows the interval in proportion to its probability. */

  Alphabet: A=0.6  B=0.3  C=0.1
  Encode "ABA":

  start                 [0.0000, 1.0000)   width 1.0
  'A' -> first 60%      [0.0000, 0.6000)   width 0.6
  'B' -> next 30% of it [0.3600, 0.5400)   width 0.18
  'A' -> first 60%      [0.3600, 0.4680)   width 0.108

  Transmit ANY number in the final interval - say 0.4.
  Bits needed = -log2(0.108) = 3.21 bits.

  Huffman on this alphabet: A=1 bit, B=2, C=2 -> "ABA" = 4 bits.

/* THE KEY PROPERTY: a symbol of probability p costs
   -log2(p) bits, and that number need not be an integer.
   A symbol with p = 0.95 genuinely costs 0.074 bits, because
   it barely narrows the interval at all.

   THE PRACTICAL PROBLEMS, and their solutions:

   1. Infinite precision is impossible.
      -> RENORMALISATION: as the interval narrows, the leading
         bits stop changing. Emit them and rescale. The
         encoder keeps a fixed-width register.

   2. The E3 / straddle condition: the interval sits across
      0.5 so no leading bit is settled.
      -> count the pending bits and emit them once the
         ambiguity resolves.

   3. Division is expensive in hardware.
      -> MULTIPLICATION-FREE variants using lookup tables.
         This is exactly what CABAC's M-coder does.

   4. It was patented until the mid-2000s.
      -> which is why JPEG has an arithmetic mode that
         nobody implemented, and why H.264 shipped CAVLC as
         well as CABAC.                                    */
```

> **CODING IS ONLY HALF OF IT; MODELLING IS THE OTHER HALF**
>
> An arithmetic coder achieves `-log2(p)` bits — but only if your *p* is right. A coder with a perfect model and a mediocre engine beats a perfect engine with a bad model, every time. That is why modern codecs spend enormous effort on **context modelling**: not "how do I write this bit cheaply", but "given everything the decoder already knows, how likely is this bit to be 1?"

CABAC: context-adaptive binary arithmetic coding

```
H.264's entropy coder, and the shape of every one since.
THREE STAGES:

1. BINARISATION
     Every syntax element becomes a string of bits.
     A motion vector difference of 5 becomes, say,
     "1 1 1 1 1 0" in a unary-ish code. Now the coder only
     ever handles BINARY decisions, which makes the
     arithmetic tractable in hardware.

2. CONTEXT MODELLING
     Each bin is coded with a probability chosen from a
     CONTEXT selected by neighbouring data:

       coding "is this block's coefficient non-zero?"
         context depends on how many neighbours were
         non-zero
       coding a motion vector
         context depends on the neighbours' motion vectors
       coding a skip flag
         context depends on whether the blocks above and
         left were skipped

     H.264 has ~460 contexts. Each holds a 6-bit
     probability state and an MPS (most probable symbol)
     flag.

3. BINARY ARITHMETIC CODING
     The M-coder: multiplication-free, table-driven,
     4 x 64 entries. Two paths:
       REGULAR    uses and updates a context
       BYPASS     assumes p = 0.5, no context, much faster
                  - used for sign bits and the tails of
                  long values, which really are random

ADAPTATION: after coding each bin, the context's state
moves toward what actually happened. The model tracks the
content as it changes, with no side information - the
decoder performs the identical update.

THE RESULT: 10-15% smaller than CAVLC at the same quality,
for free at the decoder. AV1's multi-symbol coder and
VVC's CABAC are refinements of the same idea.
```

```
/* The decoder side, simplified, to show the shape. */

typedef struct { uint8_t state; uint8_t mps; } ctx_t;

static int decode_bin(cabac_t *c, ctx_t *ctx)
{
        /* Look up the sub-interval width for this state. */
        int qIdx  = (c->range >> 6) & 3;
        int rLPS  = rangeTabLPS[ctx->state][qIdx];
        c->range -= rLPS;

        int bin;
        if (c->offset >= c->range) {          /* LPS path      */
                c->offset -= c->range;
                c->range   = rLPS;
                bin        = 1 - ctx->mps;
                if (ctx->state == 0)
                        ctx->mps ^= 1;        /* flip MPS      */
                ctx->state = transIdxLPS[ctx->state];
        } else {                              /* MPS path      */
                bin        = ctx->mps;
                ctx->state = transIdxMPS[ctx->state];
        }

        renormalise(c);          /* shift in bits until range
                                    is back in range          */
        return bin;
}

/* NOTE WHAT THIS COSTS: the next decision depends on the
   updated state, which depends on this decision. CABAC is
   INHERENTLY SERIAL - one bin at a time, with a loop-carried
   dependency.

   That single property is why:
     * CABAC decode is the hardest part of a video decoder to
       accelerate, and often the bottleneck at high bitrates
     * hardware decoders have a dedicated CABAC engine, and
       its throughput sets the maximum bitrate the chip
       supports
     * H.264 levels specify a maximum bin rate, not just a
       bitrate
     * SLICES and TILES exist partly so several CABAC engines
       can run in parallel - each slice resets the contexts
       and can be decoded independently. WPP (wavefront
       parallel processing) in HEVC is the refined version.  */
```

What came after

Coder	Used by	Idea
Huffman	JPEG, MPEG-2, MP3	Static tree, integer bits
CAVLC	H.264 baseline	Several VLC tables, switched by context. Cheaper than CABAC, ~10% worse
CABAC	H.264 main+, HEVC, VVC	Binary, adaptive, context-modelled
Boolean coder	VP8, VP9	Similar to CABAC, patent-free by design

Coder	Used by	Idea
Multi-symbol coder	AV1	Codes a symbol from an alphabet of up to 16 directly, instead of binarising. Fewer serial steps for the same compression — an explicit answer to CABAC's throughput problem
ANS / rANS	JPEG XL, Zstandard, LZFSE	Asymmetric numeral systems: arithmetic-coding efficiency with table-driven speed close to Huffman. The most important recent development

```
# See the effect for yourself.
$ ffmpeg -i in.y4m -c:v libx264 -crf 23 -coder 0 cavlc.mp4   # CAVLC
$ ffmpeg -i in.y4m -c:v libx264 -crf 23 -coder 1 cabac.mp4   # CABAC
$ ls -l cavlc.mp4 cabac.mp4
-rw-r--r-- 12481923 cavlc.mp4
-rw-r--r-- 10847216 cabac.mp4        # ~13% smaller, same CRF

# CABAC is on by default in every profile above Baseline. The
# only reasons to disable it:
#   * a decoder that genuinely only supports Baseline
#   * extremely low-power decode
#   * you need the higher bin throughput CAVLC allows

# Where the decode time actually goes:
$ perf record -g ffmpeg -i cabac.mp4 -f null -
$ perf report --sort symbol | head
  31.2%  ff_h264_decode_mb_cabac
  18.4%  h264_idct_add8_8_avx
  12.1%  ff_h264_filter_mb            # deblocking
# At high bitrates the CABAC line grows and everything else
# stays flat - it scales with BITS, not with pixels.
```

> **ENTROPY CODING IS WHERE BITSTREAM CORRUPTION BECOMES CATASTROPHIC**
>
> Because arithmetic coding has no byte alignment and the context state is carried forward, a single flipped bit desynchronises the decoder for the rest of the slice — you do not get one wrong pixel, you get garbage until the next resynchronisation point. That is why streams have start codes, slices and IDR frames, and why a dropped packet on a long GOP produces seconds of smearing rather than one bad frame (chapter 10).

How it shows up

Symptom	Cause
File ~13% larger than expected at the same CRF	CAVLC instead of CABAC
Decode is CPU-bound at high bitrate, fine at low	CABAC — it scales with bits, not pixels
Hardware decoder rejects a valid stream	Bin rate over the level limit, even though the bitrate is within it
Garbage from a corruption point to the end of the frame	Arithmetic decoder desynchronised; no resync until the next slice
Multi-threaded decode gains little	One slice per frame — nothing to parallelise across
Encoder is slow in the entropy stage	RDO is calling the entropy coder to measure real bit costs, repeatedly
Lossless mode still compresses well	Entropy coding alone; no quantisation involved
Already-compressed data does not shrink	Its entropy is already near the floor (chapter 6)

Check yourself

✓ Build a Huffman code for a given distribution and compute its average length.

- ✓ Explain Huffman's fundamental limit and quantify it for a symbol with p = 0.95.
- ✓ Encode a three-symbol message with arithmetic coding and compute the bits used.
- ✓ Name three practical problems arithmetic coding must solve and how each is solved.
- ✓ Explain why modelling matters more than the coder itself.
- ✓ Describe CABAC's three stages and what binarisation buys.
- ✓ Explain what a context is and give two examples of how one is selected.
- ✓ Explain why CABAC is inherently serial and name three consequences.
- ✓ Say what slices and tiles do for entropy decoding.
- ✓ Explain why a single bit error destroys the rest of a slice.

Temporal redundancy and motion compensation

The single largest saving in video compression. Consecutive frames are nearly identical, and what did change mostly *moved* — so instead of coding a block, code a vector pointing at where it used to be. This one idea is the difference between JPEG-per-frame and real video.

The idea

```
FRAME N-1                    FRAME N

+----------------+           +----------------+
|                |           |                |
|    [ball]      |           |       [ball]   |
|                |           |                |
|  static bg     |           |  static bg     |
+----------------+           +----------------+

CODING FRAME N INDEPENDENTLY  (as JPEG would)
  every block from scratch. ~50 KB.

CODING IT AS A DIFFERENCE FROM N-1
  background blocks: identical -> "SKIP". Zero bits beyond
                     the flag.
  ball blocks:       "copy from (x-12, y+3) in frame N-1,
                     plus this small residual"
  -> ~2 KB. TWENTY-FIVE TIMES SMALLER.

THE THREE PIECES

  1. MOTION VECTOR   where to copy from
  2. REFERENCE INDEX which earlier frame
  3. RESIDUAL        the error after copying, transformed
                     and quantised exactly as in chapter 7

A typical P-frame is 5-20% the size of an I-frame of the
same content. On static content it can be 1%.

AND THE FAILURE MODES FALL OUT OF THIS DIRECTLY:
  a scene cut          -> nothing matches -> huge frame
  a fade or flash      -> everything changed slightly ->
                          every block has a residual
  new content entering -> occlusion; no source to copy
  noise / grain        -> genuinely random, cannot be
                          predicted, and it destroys
                          compression efficiency
```

BLOCK MATCHING IS NOT MOTION ESTIMATION

The encoder is not trying to understand the scene. It searches for the block that *minimises cost*, which is usually but not always the physically correct match. On a repeating texture — a brick wall, a picket fence — it will happily pick a neighbouring identical block. That is fine: the decoder reproduces exactly what the encoder chose, so the picture is correct even when the "motion" is physically nonsense. Motion vectors are a compression tool, not a measurement, and using them for computer vision gives poor results for exactly this reason.

Searching

```
/* THE COST FUNCTION. SAD is cheap and standard; SATD
   (Hadamard-transformed) correlates better with what the
   transform will actually cost, and is what good encoders use
   for the final decision.                                    */

static int sad_16x16(const uint8_t *cur, int cs,
                     const uint8_t *ref, int rs)
{
        int sum = 0;
        for (int y = 0; y < 16; y++)
                for (int x = 0; x < 16; x++)
                        sum += abs(cur[y*cs + x] - ref[y*rs + x]);
        return sum;
}

/* FULL SEARCH over +/-64 pixels: 129 x 129 = 16,641 positions,
   each 256 subtractions = 4.3 MILLION operations PER BLOCK.
   At 8160 blocks per 1080p frame and 30 fps that is 10^12
   operations per second. Nobody does this.               */

/* WHAT ENCODERS ACTUALLY DO */

  1. PREDICT THE STARTING POINT. Motion is spatially and
     temporally coherent, so the neighbours' vectors and the
     co-located block's vector are excellent guesses. Start
     there, and you are usually within a pixel or two.

  2. DIAMOND / HEXAGON SEARCH. Test a small pattern around the
     current best, move to the winner, repeat until the centre
     wins. Typically 20-50 positions instead of 16,641.

  3. HIERARCHICAL / PYRAMID SEARCH. Search a 1/8-scale image
     first to find large motion cheaply, then refine at each
     level up. This is how you find fast motion without a
     huge search window.

  4. EARLY TERMINATION. If the cost is already below a
     threshold, stop.

/* AND THE PREDICTED VECTOR IS ALSO WHAT GETS CODED: the
   bitstream carries the DIFFERENCE between the actual vector
   and the predicted one. Coherent motion therefore costs
   almost nothing - a pan where every block moves identically
   codes as a field of zeros.                             */
```

Sub-pixel accuracy

```
Real motion is not a whole number of pixels. Restricting
vectors to integers leaves a residual that the transform
then has to code - expensively.

H.264 onward: QUARTER-PIXEL precision for luma.
AV1 and VVC: EIGHTH-pixel in some modes.

HOW: interpolate the reference frame.

  half-pixel   a 6-tap filter (1,-5,20,20,-5,1)/32 in
               H.264 - not bilinear, because bilinear
               blurs and blurring the reference degrades
               every future frame that predicts from it
  quarter-pixel  average of the two nearest half/full
               positions

THE PAYOFF: typically 20-30% bitrate reduction over
integer-pixel motion. It is one of the largest single wins
in H.264 over MPEG-2.

THE COST: the interpolation filter runs on EVERY predicted
block, in the DECODER. It is one of the most expensive
parts of decoding, and the reason motion compensation is
always a hand-written SIMD routine.

AND A SUBTLETY: the filter must be BIT-EXACT and specified
by the standard. If the encoder and decoder interpolate
even slightly differently, the error feeds into the next
frame's reference and DRIFTS - accumulating until the
picture visibly diverges. This is why every codec after
MPEG-2 specifies integer arithmetic exactly.
```

Beyond one vector per block

```
/* Simple block translation cannot express zoom, rotation, or
   two objects in one block. Codecs added tools for each. */

/* VARIABLE BLOCK SIZE (H.264 onwards)
     16x16 down to 4x4, chosen per region. Large blocks for
     flat backgrounds - one vector covers a lot. Small blocks
     at object boundaries, where two things move differently.
     HEVC goes to 64x64, AV1 to 128x128 (chapter 11).      */

/* BI-PREDICTION
     Average two predictions, from a past AND a future frame:
        pred = (ref0[mv0] + ref1[mv1] + 1) / 2
     Better on occlusion (something hidden in the past may be
     visible in the future) and it AVERAGES AWAY NOISE, which
     often beats either reference alone. Requires B-frames
     (chapter 10).                                          */

/* WEIGHTED PREDICTION
     pred = w * ref[mv] + o
     Handles fades and lighting changes, where every pixel
     changed by a multiplicative or additive amount. Without
     it, a one-second fade costs as much as a scene cut.  */

/* GLOBAL / WARPED MOTION (AV1, and VVC's affine mode)
     One affine or perspective model for the whole frame, or
     per block:
        x' = a*x + b*y + c
        y' = d*x + e*y + f
     Expresses pan, zoom and rotation in six parameters
     instead of thousands of vectors. A slow zoom becomes
     nearly free.                                           */

/* OVERLAPPED BLOCK MC (OBMC)
     Blend predictions from neighbouring blocks' vectors near
     the block edges, so the block boundary is not a
     discontinuity. Reduces blocking at source rather than
     filtering it afterwards.                               */

/* AND WHAT IS STILL HARD
     non-rigid motion  - water, fire, cloth, foliage
     occlusion         - genuinely new pixels must be intra
                         coded
     film grain        - random by construction. AV1's answer
                         is to REMOVE it before encoding and
                         SYNTHESISE it in the decoder from a
                         parametric model - which is far
                         cheaper than coding noise.        */
```

> **GRAIN AND NOISE ARE THE ENEMY OF EVERY CODEC**
>
> Motion compensation predicts, and noise is by definition unpredictable — so every noisy block gets a large residual and consumes bits. A grainy source can cost several times the bitrate of a clean one at the same perceptual quality. This is why encoders ship denoisers, why streaming services denoise before encoding, and why AV1's film grain synthesis exists: strip the grain, code the clean signal cheaply, and add statistically-matched grain back at playback.

Looking at the vectors

```
# Visualise motion vectors over the video:
$ ffplay -flags2 +export_mvs -vf codecview=mv=pf+bf+bb input.mp4
#   pf = P-frames forward, bf = B forward, bb = B backward

# Or render them to a file:
$ ffmpeg -flags2 +export_mvs -i input.mp4 \
    -vf codecview=mv=pf+bf+bb -c:v libx264 -crf 18 mvs.mp4

# WHAT TO LOOK FOR
#   long vectors in a static area  -> the search found a false
#                                     match; usually harmless,
#                                     occasionally a sign of
#                                     noise
#   a field of identical vectors   -> a pan. Should be cheap.
#   chaotic vectors everywhere     -> the encoder is failing to
#                                     predict; expect a big
#                                     frame
#   no vectors at all              -> an I-frame

# Per-frame sizes, which is where prediction success shows:
$ ffprobe -select_streams v -show_entries \
    frame=pict_type,pkt_size -of csv input.mp4 | head -20
frame,I,142883
frame,P,8214
frame,B,3102
frame,B,2894
frame,P,9017

# The I-frame is 15x the P-frames. That ratio IS temporal
# redundancy, measured. If your P-frames are nearly as big as
# your I-frames, prediction is failing - check for noise, a
# scene cut, or a source that was already heavily compressed.
```

How it shows up

Symptom	Cause
P-frames nearly as large as I-frames	Prediction failing — noise, grain, or a constantly-changing scene
Bitrate spikes at every scene cut	Expected; nothing to predict from. Rate control must absorb it (chapter 13)
Fades cost as much as cuts	Weighted prediction disabled
Detail smears during fast motion	Motion search window too small, or the preset too fast
Blocks appear to slide independently	Block size too large for the motion; sub-block partitioning would help
Grainy source needs 3× the bitrate	Noise is unpredictable — denoise, or use grain synthesis
Encoder is very slow	Motion search dominates; a faster preset shrinks the search
Picture drifts wrong over a long GOP	Encoder and decoder interpolating differently — a conformance bug
Motion vectors give poor optical flow	Expected — they minimise bits, not physical error
Slow zoom is expensive on H.264, cheap on AV1	Warped/global motion expresses it in six parameters

Check yourself

✓ Explain why a P-frame is typically 5–20% the size of an I-frame.

- ✓ Name the three pieces of information an inter-coded block carries.
- ✓ Explain why block matching is not the same as measuring motion.
- ✓ Estimate the cost of a full search and name three techniques that avoid it.
- ✓ Explain why motion vectors are coded as differences from a prediction.
- ✓ Say what quarter-pixel motion buys and what it costs the decoder.
- ✓ Explain drift and why interpolation filters must be bit-exact.
- ✓ Give three tools beyond simple block translation and what each handles.
- ✓ Explain why film grain is expensive and how AV1 addresses it.
- ✓ Read a per-frame size listing and say whether prediction is working.

Frame types, GOPs and reference management

Once frames predict from other frames, order matters — and the order they are stored in is not the order they are shown. Every seeking bug, every "the first two seconds are broken" streaming complaint, and the whole design of adaptive streaming comes out of this chapter.

The frame types

```
I-FRAME   Intra. Coded from itself alone, like a JPEG.
          Large - 5 to 20x a P-frame. Decodable with no
          other frame.

P-FRAME   Predicted. References EARLIER frames.
          Small. Useless without its references.

B-FRAME   Bi-predicted. References earlier AND later
          frames, and can average the two.
          Smallest - often half a P-frame.

AND THE ONE THAT MATTERS OPERATIONALLY:

IDR       Instantaneous Decoder Refresh. An I-frame that
          also RESETS the reference list - nothing after
          it may reference anything before it.

A PLAIN I-FRAME IS NOT A SEEK POINT. Later frames may
still reference frames from before it, so starting there
gives you garbage. Only an IDR (or an HEVC CRA/BLA, or an
AV1 keyframe) is a guaranteed entry point.

This distinction is the source of a great many bugs. When
someone says "I set keyint=250 but seeking is still
broken", they emitted I-frames and not IDRs.

  $ ffmpeg ... -x264-params "keyint=48:min-keyint=48:\
    scenecut=0:open-gop=0"
                                     ^^^^^^^^^^^ closed GOP
                                     forces IDR
```

Display order versus decode order

```
A B-frame needs a FUTURE frame, so that future frame must
be DECODED FIRST. The bitstream is reordered.

DISPLAY ORDER (what the viewer sees)
  I0   B1   B2   P3   B4   B5   P6
   |    ^    ^   ^     ^    ^    ^
   |    |    |   |     |    |    |
   +----+----+---+     +----+----+
     B1,B2 need I0 AND P3

DECODE ORDER (what is in the file)
  I0   P3   B1   B2   P6   B4   B5

P3 arrives before the B-frames that need it.

THE CONSEQUENCES

1. TWO TIMESTAMPS PER FRAME
     PTS  Presentation Time Stamp - when to show it
     DTS  Decode Time Stamp       - when to decode it
   For B-frames, PTS != DTS, and DTS can even be negative
   at the start of a stream. Container muxers must carry
   both (chapter 15).

2. LATENCY. The decoder cannot output frame B1 until it
   has decoded P3. With 3 B-frames that is 4 frames of
   structural delay - 133 ms at 30 fps. THIS IS WHY
   VIDEOCONFERENCING DISABLES B-FRAMES ENTIRELY.

3. A REORDER BUFFER is required, sized by
   max_num_reorder_frames in the stream. Getting it wrong
   gives you frames displayed out of order - a stutter
   that looks like dropped frames but is not.

4. Seeking must land on an IDR and decode forward to the
   target, discarding output until the PTS matches.
```

WHY B-FRAMES ARE SO CHEAP

Two reasons, and the second is the underrated one. First, occlusion: something hidden in the past frame may be visible in the future one, so bi-prediction finds a match where forward prediction cannot. Second, *averaging two references halves the noise* — the prediction is closer to the truth than either reference alone, so the residual is smaller. That is why a B-frame is often half the size of a P-frame even on content with simple motion.

GOP structure

```
/* A Group Of Pictures runs from one keyframe to the next. */

  IBBPBBPBBPBBI...           classic MPEG-2, GOP of 12
  IBBBPBBBPBBBP...           H.264, 3 B-frames
  I P P P P P P P...         low latency, no B-frames

/* CLOSED vs OPEN GOP

     CLOSED  no frame references across the boundary. Every
             GOP is independently decodable. Slightly less
             efficient; MANDATORY for adaptive streaming,
             because you must be able to splice segments.

     OPEN    B-frames at the start of a GOP may reference the
             previous GOP's last P-frame. A little more
             efficient, and it breaks clean splicing.

   Streaming: ALWAYS closed, with the GOP length equal to the
   segment length.

/* GOP LENGTH is a three-way trade:

     SHORT (1-2 s)     fast seeking, quick channel change,
                       fast recovery from packet loss, and
                       cheap ABR switching.
                       COSTS BITS - I-frames are expensive.

     LONG (5-10 s)     better compression, 10-20% smaller at
                       the same quality.
                       Slow seek, slow start, and a lost
                       packet corrupts until the next
                       keyframe.

   TYPICAL CHOICES
     streaming VOD        2 s, matching the segment
     live streaming       1-2 s (latency)
     broadcast            0.5-1 s (channel change)
     videoconference      infinite, with periodic refresh
                          instead (see below)
     archival             10 s+, or all-intra for editing

/* SCENE-CUT DETECTION: a good encoder inserts a keyframe at a
   cut anyway, because prediction across a cut is useless.
   Forcing a fixed interval AND allowing scene cuts gives you
   both - but for segmented streaming you must disable scene
   cuts so segment boundaries stay aligned:
     -x264-params "keyint=48:min-keyint=48:scenecut=0"       */
```

Reference management

```
H.264 onward keep a DECODED PICTURE BUFFER of several
frames, and each block chooses which one to predict from.

  DPB, up to 16 frames:
    [ I0 ] [ P3 ] [ P6 ] [ P9 ] ...

  block A -> reference index 0 (the most recent)
  block B -> reference index 2 (three frames back)

WHY MULTIPLE REFERENCES HELP
  * periodic motion - a walking figure, a rotating fan -
    matches a frame several back better than the last one
  * occlusion - something briefly hidden reappears
  * noise - an older, cleaner frame may match better
  * a flash or a dropped frame is skipped over

Typical gain: 5-10% for 4 references over 1. Diminishing
returns beyond that, and the DPB costs MEMORY -
16 references at 4K 10-bit is about 400 MB, which is why
levels cap it.

HIERARCHICAL B-FRAMES / PYRAMID - the modern structure:

  display:  I0  b1  B2  b3  P4
  decode:   I0  P4  B2  b1  b3

  temporal layer 0:  I0 ........... P4     never discarded
  temporal layer 1:        B2             references L0
  temporal layer 2:    b1     b3          references L0,L1
                       ^^     ^^
                 lowercase b = NOT used as a reference

THIS GIVES YOU TEMPORAL SCALABILITY FOR FREE: drop layer 2
and you have a valid 15 fps stream. Drop layer 1 too and
you have 7.5 fps. Used by:
  * videoconferencing, to shed frame rate under congestion
    (SVC), without re-encoding
  * streaming, for trick modes
  * and it compresses BETTER than a flat IBBP structure,
    because the layer-0 frames are coded at higher quality
    and everything else predicts from them.

LONG-TERM REFERENCES: pin a frame in the DPB indefinitely -
a clean background, or the last known-good frame. WebRTC
uses these to recover from loss without a full keyframe.
```

Low latency, and intra refresh

```
/* Videoconferencing cannot afford B-frames (reordering
   latency) or periodic I-frames (a bitrate spike that blows
   the buffer and causes a stall).

   INTRA REFRESH solves the second: instead of one huge
   I-frame, refresh a COLUMN of blocks each frame. After N
   frames the whole picture has been refreshed.             */

     frame 0:  [I][ ][ ][ ][ ]     I = intra column
     frame 1:  [ ][I][ ][ ][ ]
     frame 2:  [ ][ ][I][ ][ ]
     frame 3:  [ ][ ][ ][I][ ]
     frame 4:  [ ][ ][ ][ ][I]     fully refreshed

   Plus a constraint: blocks may not predict from the
   not-yet-refreshed region, or the refresh is undone.

   RESULT: every frame is roughly the same size - no spikes,
   no buffer stalls - and a decoder joining at any point has
   a correct picture after N frames.

$ ffmpeg ... -c:v libx264 -tune zerolatency \
    -x264-params "intra-refresh=1:bframes=0:sliced-threads=1"

/* -tune zerolatency sets, among other things:
     bframes=0            no reorder delay
     sync-lookahead=0     no lookahead buffering
     sliced-threads=1     threads work within a frame, not
                          across frames - frame-level
                          threading adds N frames of latency
     rc-lookahead=0

   THE COST: roughly 10-20% worse compression. That is the
   price of the latency, and it is not negotiable.        */
```

FRAME-THREADING IS A HIDDEN SOURCE OF LATENCY

By default `x264` and `x265` use frame-level threading: thread N works on frame N while thread N+1 works on frame N+1. With 8 threads that is up to 8 frames of encoder delay — a third of a second at 25 fps — before a single byte emerges. Any real-time use needs `sliced-threads` or an explicit thread limit. This surprises people who have already disabled B-frames and cannot work out where the latency is coming from.

Inspecting structure

```
# Frame types and sizes in decode order:
$ ffprobe -select_streams v -show_entries \
    frame=pict_type,key_frame,pkt_pts_time,pkt_dts_time,pkt_size \
    -of csv=p=0 input.mp4 | head
1,I,0.000000,-0.083333,142883      # note DTS < PTS: B-frames
0,P,0.133333,-0.050000,8214        # present
0,B,0.033333,-0.016667,3102
0,B,0.066667,0.016667,2894

# Just the keyframe positions - what you can seek to:
$ ffprobe -select_streams v -skip_frame nokey \
    -show_entries frame=pkt_pts_time -of csv=p=0 input.mp4
0.000000
2.000000
4.000000          # a clean 2-second GOP

# Are they real IDRs? Look at the NAL types:
$ ffmpeg -i input.mp4 -c copy -bsf:v trace_headers -f null - 2>&1 \
    | grep -E 'nal_unit_type' | sort | uniq -c
   1500 nal_unit_type = 1     # non-IDR slice
     50 nal_unit_type = 5     # IDR slice          <-- these
     51 nal_unit_type = 7     # SPS
     51 nal_unit_type = 8     # PPS

# NOTE the SPS/PPS repeated before every IDR - that is
# repeat-headers, and without it a player that tunes in
# mid-stream has no parameter sets and cannot start.

# GOP structure at a glance:
$ ffprobe -select_streams v -show_entries frame=pict_type \
    -of csv=p=0 input.mp4 | head -30 | tr '\n' ' '
I B B P B B P B B P B B P I B B P ...
```

How it shows up

Symptom	Cause
Seeking lands on a broken frame	I-frames but not IDRs — use a closed GOP
First seconds of a stream are garbage	SPS/PPS not repeated before each keyframe
Frames displayed out of order	Reorder buffer too small, or PTS/DTS mishandled
Video and audio drift apart	PTS used where DTS was needed, or vice versa (chapter 15)
133 ms of unexplained latency	B-frames — disable them for real-time
Latency persists after disabling B-frames	Frame-level threading; use `sliced-threads`
Bitrate spikes every GOP, buffer stalls	Periodic I-frames; use intra refresh
ABR switching produces a glitch	GOP boundaries not aligned across renditions, or an open GOP
Packet loss corrupts several seconds	Long GOP — nothing recovers until the next keyframe
4K decode runs out of memory	DPB size — 16 references at 4K is hundreds of megabytes

Check yourself

✓ Distinguish I, P and B frames by what they reference and their typical relative sizes.

- ✓ Explain why an I-frame is not necessarily a seek point, and what is.
- ✓ Write a display order and its corresponding decode order for an IBBP structure.
- ✓ Explain why PTS and DTS differ and where DTS can be negative.
- ✓ Give two reasons B-frames compress so well.
- ✓ Explain closed versus open GOP and why streaming requires closed.
- ✓ State the three-way trade in GOP length with a typical value for two use cases.
- ✓ Explain hierarchical B-frames and the temporal scalability they provide free.
- ✓ Describe intra refresh and the problem it solves.
- ✓ Name two sources of encoder latency beyond B-frames.

Intra prediction, inter prediction and partitioning

Everything a codec codes is a *prediction error*. The tools that generate the prediction — directional extrapolation from neighbours, motion vectors into other frames, and the block splitting that chooses where each applies — are where most of the gain from H.264 to AV1 actually came from.

Intra prediction: guessing from neighbours

```
JPEG codes each 8x8 block independently. That wastes the
fact that the pixels immediately above and to the left are
ALREADY DECODED and are excellent predictors.

  A B C D E F G H     <- reconstructed row above
I . . . .
J . . . .             the 4x4 block to predict
K . . . .
L . . . .
^
reconstructed column left

H.264's 4x4 luma modes, all built from those 13 pixels:

  0 VERTICAL      copy A B C D straight down
  1 HORIZONTAL    copy I J K L across
  2 DC            fill with the average of all of them
  3 DIAGONAL DOWN-LEFT
  4 DIAGONAL DOWN-RIGHT
  5 VERTICAL-RIGHT
  6 HORIZONTAL-DOWN
  7 VERTICAL-LEFT
  8 HORIZONTAL-UP

Nine modes for 4x4, four for 16x16.
HEVC:  35 modes - DC, PLANAR, and 33 angles.
AV1:   56 directional modes plus PAETH, SMOOTH, and
       intra block copy.

PLANAR / SMOOTH deserves a mention: it interpolates a
gradient across the block rather than copying a direction.
Skies and soft shading are extremely common and a
directional copy handles them badly, so planar is one of
the most-selected modes in real content.

THE ENCODER TRIES THEM ALL and picks by rate-distortion
(chapter 6): the mode index costs a few bits, and the
residual costs what it costs.

WHY IT MATTERS: on a clean edge, the right directional mode
makes the residual almost zero. This is the single biggest
reason an H.264 I-frame is roughly HALF the size of an
equivalent-quality JPEG.
```

> **PREDICTION USES *RECONSTRUCTED* NEIGHBOURS, NOT ORIGINAL ONES**
> The encoder must predict from the same pixels the decoder will have — which are the quantised, reconstructed ones, not the source. So every encoder contains a complete decoder inside it, reconstructing each block before coding the next. Get that wrong and encoder and decoder diverge, which is **drift**: an error that compounds through every subsequent prediction until the picture visibly falls apart.

Partitioning: choosing the block size

```
One size cannot fit both a flat wall and a face.

  LARGE blocks   fewer vectors, fewer mode decisions,
                 less overhead. Good on flat or uniformly
                 moving regions.
  SMALL blocks   follow detail and object boundaries.
                 Good at edges. Costs overhead per block.

H.264   macroblock 16x16, split to 16x8, 8x16, 8x8, and
        8x8 further to 8x4, 4x8, 4x4.

HEVC    QUADTREE. A 64x64 Coding Tree Unit recursively
        split into four, down to 8x8:

          +---------------+       +-------+-------+
          |               |       |       |   |   |
          |               |  ->   |       +---+---+
          |               |       |       |   |   |
          +---------------+       +---+---+-------+
                                  | | |   |       |
                                  +-+-+   |       |
                                  | | |   |       |
                                  +---+---+-------+

        Flat sky -> one 64x64 block, a handful of bits.
        Detailed foliage -> many 8x8 blocks.
        This alone is worth a large part of HEVC's gain
        over H.264, and it is why HEVC helps most at high
        resolutions - a 4K frame has far more area that is
        locally flat.

AV1     quadtree PLUS 1:2, 2:1 and 1:4, 4:1 rectangular
        splits, 128x128 down to 4x4. Ten partition types
        per node.

VVC     quadtree + binary + ternary splits, in any
        combination - the most flexible, and the most
        expensive to search.

THE COST IS ENCODER TIME. Each additional partition type
multiplies the search space, and the encoder must
rate-distortion test them all. This is why HEVC encoding is
roughly 5-10x slower than H.264 and AV1 slower again, for
the same picture.
```

Inter prediction, in detail

```
/* The modes an encoder chooses between for each block. */

  SKIP / MERGE
      Take the motion vector from a neighbour, code NO
      residual and NO vector - just a flag.
      On static content this is most of the frame, and it is
      why a still shot costs almost nothing.
      HEVC's MERGE builds a candidate list from spatial and
      temporal neighbours; you code only the INDEX.

  INTER with MVD
      A merge candidate as the predictor, plus a coded motion
      vector DIFFERENCE. Coherent motion -> tiny differences.

  BI-PREDICTION
      Two references, averaged (chapter 9).

  INTRA, inside a P or B frame
      Sometimes prediction from a neighbour beats prediction
      from another frame - new content entering the shot,
      an occlusion boundary, a scene with a lot of noise.
      A "P-frame" is full of intra blocks at a scene cut.

/* THE CANDIDATE LIST IS THE INTERESTING PART. Coding a full
   motion vector costs 10-20 bits. Coding "use candidate 1"
   costs 1-2. So codecs spend real design effort on making the
   candidate list contain the right answer:

     spatial neighbours    left, above, above-right,
                           below-left, above-left
     temporal              the co-located block in a
                           previously coded frame
     history-based (AV1/VVC)  a rolling table of recently
                           used vectors anywhere in the frame
     pairwise average      the mean of two other candidates

   When the list is good, most blocks cost a few bits total.
   That is where modern codecs beat older ones far more than
   in the transform.                                       */
```

The tools that came later

Tool	What it handles	First in
Intra block copy	Predict from elsewhere in the *same* frame — repeated UI elements, text, game HUDs	HEVC SCC, AV1
Palette mode	Blocks with few distinct colours — screen content, graphics	HEVC SCC, AV1
Affine motion	Zoom and rotation, via a 4- or 6-parameter model per block	AV1, VVC
Compound / wedge prediction	Two predictions blended along a diagonal edge — an object boundary inside one block	AV1
OBMC	Blending neighbours' predictions at block edges, reducing blocking at source	AV1
CfL — chroma from luma	Predict chroma from the reconstructed luma of the same block	AV1

Tool	What it handles	First in
Multiple transform sizes and types	DCT, DST and identity transforms chosen per block — identity is right for sharp screen content	HEVC, AV1

```
# Watch the encoder's decisions. x264 will tell you what it
# chose, which is the fastest way to understand a bitrate.

$ x264 --crf 23 --psnr --verbose input.y4m -o out.264 2>&1 | tail -20
x264 [info]: frame I:50    Avg QP:20.11  size: 42883
x264 [info]: frame P:1200  Avg QP:23.44  size:  8214
x264 [info]: frame B:2250  Avg QP:25.87  size:  3102
x264 [info]: consecutive B-frames: 12.5%  8.2% 30.1% 49.2%
x264 [info]: mb I  I16..4: 22.1% 61.3% 16.6%
x264 [info]: mb P  I16..4:  2.1%  4.8%  0.9%  P16..4: 45.2% 18.1% 6.2%  skip:22.7%
x264 [info]: mb B  I16..4:  0.3%  0.6%  0.1%  B16..8: 12.1%  3.2%  direct: 8.9%  skip:74.8%
                                                                          ^^^^^^^^^^

# 74.8% of B-frame macroblocks are SKIPPED - copied with no
# residual and no vector. THAT is where the compression is.
# If that number is low, the content is noisy or the motion
# search is failing (chapter 9).

# Intra modes chosen in an I-frame:
$ x264 --crf 23 --verbose ... 2>&1 | grep 'i16 v,h,dc'
x264 [info]: i16 v,h,dc,p: 31% 24% 27% 18%
x264 [info]: i8 v,h,dc,ddl,ddr,vr,hd,vl,hu: 22% 19% 13% 6% 6% 8% 7% 10% 9%
# A healthy spread. If almost everything is DC, the content is
# flat; if it is all one direction, you may be looking at
# something synthetic.
```

> **MORE TOOLS MEANS SLOWER, NOT AUTOMATICALLY BETTER**
>
> AV1 has roughly ten times H.264's decision space. An encoder that searches it badly — a fast preset, or an immature implementation — can produce *worse* output than a well-tuned H.264 encoder at the same speed. The format sets the ceiling; the encoder's search decides how close you get. This is why "AV1 is 30% better" is only true at slow presets, and why `x264 --preset veryslow` beats `SVT-AV1 --preset 12` at equal encoding time.

How it shows up

Symptom	Cause
Skip percentage very low on static content	Noise or grain defeating prediction (chapter 9)
I-frames much larger than expected	Intra prediction failing — noisy or highly detailed source
HEVC barely beats H.264 on your content	Low resolution — the quadtree gains most where large flat areas exist
AV1 output worse than H.264 at the same encode time	Preset too fast for the search space
Screen content compresses badly	Needs intra block copy, palette mode or an identity transform
Picture degrades progressively through a GOP	Drift — encoder predicting from source rather than reconstructed pixels
Visible seams along object edges	Block partitioning too coarse; OBMC or wedge prediction would help
Slow zoom is expensive	No affine or warped motion model
Encoding time explodes on a newer codec	Partition and mode search space, not the transform

Check yourself

- ✓ Explain what intra prediction predicts from, and why planar mode matters.
- ✓ Say why an H.264 I-frame is smaller than an equivalent JPEG.
- ✓ Explain why the encoder must predict from reconstructed pixels, and what drift is.
- ✓ Describe the HEVC quadtree and say why it helps most at high resolution.
- ✓ Name four inter modes and say which dominates on static content.
- ✓ Explain why the merge candidate list is worth so much design effort.
- ✓ Say why intra blocks appear inside P-frames.
- ✓ Name four tools added after H.264 and what each handles.
- ✓ Read an x264 statistics dump and say what the skip percentage tells you.
- ✓ Explain why a newer codec can produce worse output at the same encoding time.

In-loop filters and post-processing

Block-based coding produces block-shaped artefacts. Filtering them away is worth 5–10% of bitrate — but only if the filter runs *inside* the coding loop, so that the cleaned frame is what future frames predict from. That word "in-loop" is the whole idea.

In-loop versus post-processing

```
POST-PROCESSING - filter on the way to the screen

  decode -> reconstruct -> [reference buffer]
                       \-> FILTER -> display

  The reference is UNFILTERED. Every future frame predicts
  from blocky data, so the artefacts propagate and the
  filter has to remove them again, every frame.
  It is also OPTIONAL, so two decoders show different
  pictures. MPEG-2 worked this way.

IN-LOOP - filter before storing as a reference

  decode -> reconstruct -> FILTER -> [reference buffer]
                                  \-> display

  The reference is CLEAN. Prediction from it is better, so
  residuals are smaller, so you spend fewer bits - 5-10%
  at the same quality.
  And it is NORMATIVE: every decoder must apply exactly
  the same filter, or drift (chapter 11).

THIS IS WHY IN-LOOP FILTERING IS NOT COSMETIC. It changes
what the encoder can predict from, which changes the
bitrate. Turning deblocking off does not just look worse -
it costs bits.
```

The deblocking filter

```
/* Blocking happens because each block is quantised
   independently, so two neighbours' DC levels can differ by a
   step even though the original was smooth. The filter
   smooths ACROSS block edges - and only where the
   discontinuity looks like an artefact rather than real
   content.                                                  */

  FOR EACH EDGE, decide a BOUNDARY STRENGTH (bS):

    bS = 4   one side is intra-coded AND the edge is a
             macroblock boundary        -> strongest filter
    bS = 3   one side is intra-coded
    bS = 2   either side has non-zero transform coefficients
    bS = 1   different reference frames, or motion vectors
             differing by >= 1 pixel
    bS = 0   nothing differs                -> DO NOT FILTER

  THEN a content test, and this is the clever part:

    filter only if
        |p0 - q0| < alpha  AND
        |p1 - p0| < beta   AND
        |q1 - q0| < beta

    where p are pixels left of the edge and q right of it,
    and alpha/beta scale with QP.

  THE LOGIC: a SMALL step across the edge with FLAT sides is
  almost certainly an artefact - smooth it. A LARGE step is
  almost certainly a real edge in the picture - leave it
  alone. That single heuristic is why deblocking does not
  turn everything to mush.

  AND alpha/beta SCALE WITH QP: at low QP the quantisation
  error is small, so only tiny steps get filtered. At high QP
  the filter works much harder. The filter strength follows
  the artefact severity automatically.

/* TUNING, in x264: */
    --deblock -1:-1     sharper, more blocking. Good for
                        high-bitrate, detailed content.
    --deblock 0:0       the default.
    --deblock 1:1       softer, less blocking. Good for
                        low-bitrate or animation.
/* The two numbers are alpha and beta offsets. Film grain and
   detailed live action usually want negative; flat animation
   and low bitrates want positive.                           */
```

> **DEBLOCKING IS A LARGE FRACTION OF DECODE TIME**
>
> It touches every pixel, several times, with data-dependent branches — typically 15–30% of a software decoder's runtime, comparable to motion compensation. It is always hand-written SIMD, and it is one of the first things a hardware decoder implements as a fixed-function block. When profiling a decoder, `ff_h264_filter_mb` or its equivalent near the top of the profile is normal, not a bug.

The filters that came after

Filter	Fixes	In
Deblocking	Block-edge discontinuities	H.264, HEVC, AV1, VVC
SAO — sample adaptive offset	Ringing and banding. Adds a small per-category offset to samples, with the offsets sent in the bitstream	HEVC
CDEF — constrained directional enhancement	Ringing, while *preserving* edges — it detects the dominant direction in each block and filters along it	AV1
Loop restoration	Wiener and self-guided filters that restore detail lost to quantisation, with parameters chosen per unit by the encoder	AV1
ALF — adaptive loop filter	A Wiener filter whose coefficients are computed per frame by the encoder to minimise error against the source	VVC
LMCS — luma mapping with chroma scaling	Redistributes code values to suit the content, particularly HDR	VVC
Film grain synthesis	Not a filter — grain is *removed* before encoding and *re-generated* from parameters at playback	AV1, H.264 SEI, VVC

```
/* SAO, because it shows the general shape of the newer
   filters: the ENCODER measures the error and SENDS a
   correction.                                          */

  BAND OFFSET
    Split the sample range into 32 bands. For four chosen
    bands, send an offset. Applied to every sample in those
    bands.
    -> fixes BANDING in gradients, because it can nudge one
       band's level without touching the others.

  EDGE OFFSET
    Classify each sample against two neighbours in one of four
    directions (0deg, 90deg, 135deg, 45deg):

       category 0:  local minimum      -> add offset[0]
       category 1:  edge, falling      -> add offset[1]
       category 2:  edge, rising       -> add offset[2]
       category 3:  local maximum      -> add offset[3]

    -> fixes RINGING, by pulling overshoots back toward their
       neighbours.

  The offsets are chosen by the ENCODER to minimise error
  against the ORIGINAL frame - so unlike deblocking, which is
  a fixed heuristic, SAO is a measured correction. That is
  why it works on artefacts deblocking cannot see.

/* CDEF in AV1 goes further: it estimates the DOMINANT
   DIRECTION of each 8x8 block, then applies a nonlinear
   filter ALONG that direction (which preserves the edge) and
   a weaker one across it (which removes the ringing). The
   direction search is why it costs more than SAO and why it
   looks better on detailed content.                    */
```

Post-processing, outside the loop

```
# Filters the player or the display applies. NOT normative -
# they change the picture, not the bitstream, and every
# device does something different.

  DEINTERLACING     for legacy interlaced sources. yadif,
                    bwdif, or a motion-compensated filter.
                    Getting it wrong gives combing on motion.
  DEBANDING         adds dither or gradient reconstruction.
                    gradfun, deband.
  UPSCALING         Lanczos, or a learned model (DLSS,
                    FSR, NVIDIA VSR, Apple's neural scaler).
  SHARPENING        popular, and it amplifies compression
                    artefacts as enthusiastically as detail.
  FRAME INTERPOLATION  "motion smoothing" on TVs. Produces the
                    soap-opera effect, and 60-120 ms of
                    latency - which is why every TV has a
                    "game mode" that turns it off.
  TONE MAPPING      HDR to SDR (chapter 5).

# In practice:
$ ffplay -vf "bwdif=mode=send_field,gradfun=1.2:8" input.mkv

# THE ORDER MATTERS, and the general rule is:
#   deinterlace -> denoise/deband -> scale -> sharpen
# Scaling interlaced content before deinterlacing destroys the
# field structure and cannot be undone. Sharpening before
# scaling amplifies what the scaler then smears.

# AND THE HONEST WARNING: post-processing cannot recover
# information. A deband filter is guessing what the gradient
# should have been; an upscaler is inventing detail. Both can
# look better and neither is more correct. If you control the
# encode, fix it there - more bits, higher bit depth, better
# rate control - because that is the only place the
# information still exists.
```

> **DO NOT SHARPEN BEFORE MEASURING QUALITY**
>
> Sharpening reliably raises perceived detail and reliably *lowers* PSNR and SSIM while often raising VMAF — which is why "VMAF hacking" with a sharpening filter is a known way to produce encoders that score well and look worse (chapter 6). If you are comparing encoders, disable every post-process on both sides, or you are measuring the filter.

How it shows up

Symptom	Cause
Visible block edges at moderate bitrate	Deblocking disabled or offsets too negative
Picture looks soft and smeared	Deblocking too strong — try negative offsets
Bitrate rose after disabling deblocking	Expected — references are dirtier, so residuals grow
Two decoders show different pictures	A post-process filter, not an in-loop one — in-loop is normative
Ringing around text that deblocking does not fix	Needs SAO or CDEF; deblocking only sees block edges
Banding in gradients despite enough bits	SAO band offset disabled, or 8-bit throughout

Symptom	Cause
Decoder profile dominated by the filter	Normal — deblocking is 15–30% of decode
Combing artefacts on motion	Interlaced source not deinterlaced, or the wrong field order
Grain looks wrong or absent on AV1	Film grain synthesis not applied by the decoder
TV adds noticeable input lag	Frame interpolation; use game mode

Check yourself

- ✓ Explain the difference between in-loop and post-processing filtering, and why it affects bitrate.
- ✓ Say why in-loop filters must be normative.
- ✓ Describe how boundary strength is decided and what `bS = 0` means.
- ✓ Explain the content test that stops deblocking smoothing real edges.
- ✓ Say why the filter thresholds scale with QP.
- ✓ Explain what SAO band offset and edge offset each fix.
- ✓ Say what makes SAO different in kind from deblocking.
- ✓ Explain what CDEF adds over SAO.
- ✓ Explain AV1 film grain synthesis and why it saves bits.
- ✓ Give the correct order for a post-processing chain and one ordering mistake.

Rate control and buffer models

Deciding the quantisation parameter for every frame, under a constraint you cannot violate. It is the part of encoding that most affects what viewers actually see, and the part most often configured by copying a command line without understanding which mode it selected.

The modes, and which one you want

Mode	You specify	Use for
CQP	A fixed QP	Testing and research only. Bitrate varies wildly with content
CRF	A quality target	**The default for anything stored.** Constant perceptual quality, variable bitrate, one pass
ABR (1-pass)	An average bitrate	Rarely the right answer — it must guess the future, and it guesses badly
2-pass VBR	A target bitrate or file size	When the size is a hard requirement. Pass 1 measures complexity, pass 2 spends accordingly
CBR	A constant bitrate	Broadcast, and links with a fixed capacity. Wastes bits on easy content by padding
Capped CRF	CRF *plus* a maximum bitrate	**The right answer for streaming.** Quality-driven, but never exceeds what the client can pull

```
# CRF - the one to reach for.
$ ffmpeg -i in.mov -c:v libx264 -crf 23 -preset slow out.mp4

#   x264 CRF 18   visually lossless-ish
#         23      the default; good
#         28      noticeably compressed
#   x265 and AV1 use a similar scale but the numbers are NOT
#   comparable - x265 CRF 28 is roughly x264 CRF 23.
#   +6 CRF is about half the bitrate. -6 is about double.

# CAPPED CRF - quality-driven with a ceiling. This is what
# streaming services actually use:
$ ffmpeg -i in.mov -c:v libx264 -crf 23 \
    -maxrate 6M -bufsize 12M -preset slow out.mp4
#   crf      the quality you want
#   maxrate  the ceiling you must not exceed
#   bufsize  how much short-term variation is allowed
# Easy scenes come out well under 6M; hard scenes are capped.

# 2-PASS, when the SIZE is the requirement:
$ ffmpeg -i in.mov -c:v libx264 -b:v 5M -pass 1 -an -f null /dev/null
$ ffmpeg -i in.mov -c:v libx264 -b:v 5M -pass 2 out.mp4
# Pass 1 writes a stats file recording each frame's
# complexity. Pass 2 distributes the bit budget across the
# whole file, giving hard scenes more and easy ones less.

# CBR - genuinely constant, with padding:
$ ffmpeg -i in.mov -c:v libx264 -b:v 5M -minrate 5M -maxrate 5M \
    -bufsize 5M -x264-params "nal-hrd=cbr" out.ts

# THE MISTAKE PEOPLE MAKE: setting -b:v AND -crf together.
# CRF wins and the bitrate is ignored, so the file comes out
# a completely different size than intended and nobody can
# see why.
```

> **CRF IS NOT "CONSTANT QUALITY", IT IS CONSTANT *EFFORT***
>
> CRF adjusts QP per frame using a complexity estimate, deliberately spending fewer bits on frames where the eye will not notice — fast motion, dark scenes, high-detail texture. That is *adaptive quantisation*, and it is why CRF looks better than fixed QP at the same average bitrate. It also means the measured PSNR varies frame to frame by design, which surprises people benchmarking it.

The buffer model

```
A decoder receives bits at some rate and consumes a whole
frame instantly at each display time. If the buffer runs
dry, playback stalls. The HRD (Hypothetical Reference
Decoder) is the model that guarantees it will not.

  buffer
  fullness
     |     /\      /\
     |    /  \    /  \        fills at the bitrate,
full |---/----\--/----\-----  drains by one frame's bits
     |  /      \/      \      at each frame time
     | /                \
empty|/------------------\--  <-- UNDERFLOW = a stall
     +----------------------- time

THE THREE PARAMETERS

  bitrate    the fill rate
  bufsize    the buffer capacity, in bits
  initial    how full before playback starts - this is
             your startup delay

THE RULE: bufsize / bitrate = the seconds of variation
allowed.

  bufsize = bitrate      1 second of slack. Tight; forces
                         near-constant frame sizes; costs
                         quality on hard scenes.
  bufsize = 2 x bitrate  the usual streaming choice.
  bufsize = 5 x bitrate  loose. Better quality, but the
                         client needs a bigger buffer and
                         a longer startup delay.

WHY IT MATTERS
  * a live stream that overflows the client buffer causes
    REBUFFERING - the visible failure viewers actually
    complain about
  * hardware decoders implement the HRD literally, and a
    non-conformant stream can be REJECTED outright even
    though software players accept it
  * broadcast multiplexers allocate a fixed slot; exceeding
    it corrupts the whole multiplex, not just your channel

AND THE FRAME-SIZE CONSEQUENCE: an I-frame is 10-20x a
P-frame (chapter 10). With a tight buffer the encoder must
either shrink the I-frame - hurting the whole GOP that
predicts from it - or spread the cost over the following
frames. Both look worse than a bigger buffer.
```

Adaptive quantisation and lookahead

```
/* WITHIN a frame, not all blocks deserve the same QP. */

  PSYCHOVISUAL MASKING: the eye cannot see quantisation error
  in a busy, high-contrast region, but sees it immediately in
  a smooth one. So:

     flat areas    -> LOWER QP (finer quantisation)
                      because banding and blocking would show
     detailed areas -> HIGHER QP
                      because the detail masks the error

  This is backwards from what "spend bits where the detail is"
  suggests, and it is measurably better.

    --aq-mode 1    variance-based AQ
    --aq-mode 2    auto-variance
    --aq-mode 3    auto-variance with a dark-scene bias -
                   x264's default, because dark areas show
                   banding badly
    --aq-strength  how aggressive, default 1.0

/* LOOKAHEAD: the encoder buffers N frames before deciding. */

    --rc-lookahead 40      x264's default

  WITH LOOKAHEAD THE ENCODER CAN:
    * see a scene cut coming and not waste bits on the frames
      about to be discarded
    * identify frames that many others will predict from and
      spend MORE on them - MB-tree / macroblock tree, which
      propagates a "how much does this block matter to the
      future" weight backwards
    * distribute the bit budget over an upcoming hard scene
      instead of reacting to it late

  MB-TREE IS WORTH 5-15% ON ITS OWN and is why x264 at
  --preset slow beats a naive encoder at the same bitrate.

  THE COST: latency. 40 frames of lookahead is 1.6 seconds at
  25 fps. Real-time encoding sets --rc-lookahead 0 and
  accepts the loss (chapter 10).                            */
```

QUALITY PUMPING AND HOW TO RECOGNISE IT

If quality visibly oscillates — sharp, then soft, then sharp again on a two-second cycle — rate control is overreacting. Usual causes: `bufsize` too small for the content, ABR mode trying to hit an average without lookahead, or a GOP whose I-frame consumes so much of the budget that the following frames are starved. The fix is almost always a larger `bufsize` or switching to capped CRF; it is very rarely a higher bitrate.

Rate control for adaptive streaming

```
# A streaming ladder is several renditions of the same source,
# and the client switches between them. Constraints:

  1. GOP-ALIGNED. Every rendition must have a keyframe at the
     same timestamps, or switching produces a glitch.
       -x264-params "keyint=48:min-keyint=48:scenecut=0"
     with 48 = 2 seconds at 24 fps. Scene cuts MUST be
     disabled or the alignment breaks (chapter 10).

  2. CAPPED CRF per rung, not fixed bitrate. An easy scene at
     1080p may only need 2 Mbit/s; forcing 6 wastes bandwidth
     and gains nothing.

  3. THE LADDER ITSELF should be per-title. Netflix's
     per-title encoding measures each title's complexity and
     builds a custom ladder - an animated series needs far
     less bitrate at 1080p than a grainy film, and a fixed
     ladder over-serves one and under-serves the other.

# A reasonable fixed ladder, for reference:
#   1920x1080  crf 21  maxrate  6000k  bufsize 12000k
#   1280x720   crf 21  maxrate  3000k  bufsize  6000k
#    854x480   crf 22  maxrate  1500k  bufsize  3000k
#    640x360   crf 23  maxrate   800k  bufsize  1600k
#    426x240   crf 24  maxrate   400k  bufsize   800k

# Segment and package:
$ ffmpeg -i in.mp4 \
  -c:v libx264 -crf 21 -maxrate 6000k -bufsize 12000k \
  -x264-params "keyint=48:min-keyint=48:scenecut=0" \
  -f hls -hls_time 2 -hls_playlist_type vod \
  -hls_segment_filename "1080p_%03d.ts" 1080p.m3u8

# VERIFY THE ALIGNMENT - this is the check people skip:
$ for f in 1080p 720p 480p; do
    echo -n "$f: "
    ffprobe -select_streams v -skip_frame nokey \
      -show_entries frame=pkt_pts_time -of csv=p=0 $f.m3u8 \
      | head -5 | tr '\n' ' '
    echo
  done
1080p: 0 2 4 6 8
720p:  0 2 4 6 8
480p:  0 2 4 6 8        # aligned. If these differ, switching
                        # will glitch.
```

How it shows up

Symptom	Cause
File size nothing like the bitrate you set	`-crf` and `-b:v` both given; CRF won
Quality visibly pulses	`bufsize` too small, or ABR without lookahead
Playback rebuffers on a good connection	Peak bitrate exceeds what the buffer model allows — cap it
Hardware decoder rejects a valid-looking stream	HRD non-conformance
ABR switching glitches	GOPs not aligned across renditions, or scene cuts left on

Symptom	Cause
Dark scenes band badly	Adaptive quantisation off, or `aq-mode` without the dark bias
First frames after a cut look terrible	No lookahead, so the encoder reacted late
Real-time encoding has 1.6 s of latency	`rc-lookahead` — set it to 0
Two-pass produced a smaller file than requested	The content was easier than the target; that is correct behaviour
Animation over-served, film under-served by one ladder	Fixed ladder rather than per-title

Check yourself

- ✓ Name the six rate-control modes and say which to use for stored video, for streaming and for a fixed file size.
- ✓ Explain what CRF actually holds constant.
- ✓ Say what happens if you specify both a bitrate and a CRF.
- ✓ Draw the HRD buffer model and say what underflow means to a viewer.
- ✓ Explain what `bufsize / bitrate` represents and give a typical value.
- ✓ Explain why a tight buffer hurts quality across a whole GOP.
- ✓ Explain adaptive quantisation and why flat areas get *lower* QP.
- ✓ Say what lookahead enables, and name the technique worth 5–15%.
- ✓ Give three constraints on rate control for an adaptive streaming ladder.
- ✓ Verify GOP alignment across renditions and say what a mismatch causes.

The codec landscape: H.264 to AV1 and beyond

Choosing a codec is rarely a technical decision. Compression efficiency is the easy part; what actually decides it is hardware decode support, patent licensing, and how much CPU you can afford to spend on encoding. This chapter is the map.

The codecs that matter

Codec	Year	Bitrate vs H.264	HW decode	Licensing
MPEG-2	1995	+150%	Everywhere	Expired
H.264 / AVC	2003	baseline	**Universal**	MPEG LA pool; most patents expiring
VP9	2013	−35%	Wide (2015+)	Royalty-free (Google)
H.265 / HEVC	2013	−40%	Wide (2015+)	**Three pools plus unpooled holders.** The reason it failed on the web
AV1	2018	−50%	Growing (2020+)	Royalty-free (AOMedia)
H.266 / VVC	2020	−65%	Rare	Pools forming; same uncertainty as HEVC
H.267 / ECM	~2028	−80% (target)	None	Unknown

THOSE PERCENTAGES ARE MARKETING UNTIL YOU NAME THE CONDITIONS

"AV1 is 50% better than H.264" means: at a slow preset, on 4K content, measured by a specific metric, at a specific quality point. At fast presets the gap shrinks dramatically; at 480p it is far smaller; and `x264 --preset veryslow` beats `SVT-AV1 --preset 10` at equal *encoding time*. Always ask which encoder, which preset, which resolution, which metric — the format sets a ceiling, the encoder decides how close you get (chapter 11).

Why HEVC lost the web

```
HEVC is technically excellent and commercially poisoned.

H.264's licensing:  ONE pool (MPEG LA), a published rate
                    card, a cap on annual fees, and free
                    for internet video that is free to
                    end users. Everyone knew the cost.

HEVC's licensing:   MPEG LA -> then HEVC Advance appeared
                    -> then Velos Media -> plus companies
                    holding patents outside every pool.
                    Different terms, some demanding
                    CONTENT royalties (a fee per stream,
                    not per device), and no way to know
                    your total exposure.

THE RESULT
  * no browser shipped HEVC for years; Safari eventually
    did because Apple pays anyway
  * Google, Mozilla, Amazon, Netflix, Cisco, Intel, ARM
    and others formed AOMedia and built AV1 specifically
    to escape this
  * HEVC still won in one place - BROADCAST AND PHYSICAL
    MEDIA - where device makers already pay pool fees and
    hardware support was mandated

THE LESSON, and it repeats with VVC: a codec's success is
decided by whether an implementer can determine the cost
in advance. VVC is technically ahead of AV1 and is
repeating HEVC's licensing structure.

AV1's counter-position: AOMedia members grant a royalty-free
licence, and the patent defence is collective. It is not a
guarantee of zero risk - Sisvel formed an AV1 pool in 2019 -
but the practical outcome so far is that AV1 ships.
```

Choosing, in practice

```
/* THE DECISION TREE THAT ACTUALLY APPLIES */

  Must it play EVERYWHERE, including 2012 hardware?
      -> H.264 High profile, 4:2:0 8-bit. No exceptions.
         Still ~80% of internet video for this reason alone.

  Web video, modern browsers, you control the encode?
      -> AV1 for the top rungs, H.264 fallback for the rest.
         This is what YouTube and Netflix do.

  Broadcast, set-top, Blu-ray, or a TV app?
      -> HEVC. Hardware decode is mandated and the licensing
         is already paid by the device maker.

  Apple ecosystem?
      -> HEVC. Hardware encode AND decode on every device
         since 2017, and HDR support is well integrated.

  Real-time conferencing?
      -> H.264 (universal, low latency, cheap hardware
         encode) or AV1 if both ends support it. VP8 still
         appears in WebRTC fallbacks.

  Editing / mastering / archival?
      -> ProRes, DNxHR, or FFV1. Intra-only, 4:2:2 or 4:4:4,
         10-bit+. NOT a delivery codec - you want every frame
         independently decodable and no generation loss
         (chapter 3).

  Screen recording, text-heavy?
      -> H.264 4:4:4 if your decoder supports it, or AV1 with
         palette and intra-block-copy. 4:2:0 destroys text.

/* AND THE ONE THAT CATCHES PEOPLE OUT

   HARDWARE ENCODE IS NOT HARDWARE DECODE. A phone from 2019
   decodes AV1 in software only, but has HEVC decode in
   silicon. A GPU may encode H.264 and HEVC but only decode
   AV1. Check both directions before committing.          */
```

Profiles, levels and tiers

```
/* A codec is a menu. A PROFILE says which tools you may use;
   a LEVEL says how big and fast. A decoder claims support for
   a profile AND a level, and both must be satisfied.       */

  H.264 PROFILES
    Baseline    no B-frames, no CABAC. Videoconferencing.
    Main        + B-frames, CABAC, interlacing.
    High        + 8x8 transform, custom quant matrices.
                THE ONE YOU WANT. Universally supported and
                ~10% better than Main.
    High10, High 4:2:2, High 4:4:4  more bit depth and chroma.
                Poor hardware support - check before using.

  LEVELS cap: resolution, frame rate, bitrate, DPB size, and
  the number of motion vectors per second.

    Level 3.1   720p30       14 Mbit/s
    Level 4.0   1080p30      20 Mbit/s
    Level 4.2   1080p60      50 Mbit/s
    Level 5.1   4K30        240 Mbit/s
    Level 6.2   8K120       800 Mbit/s

  HEVC adds TIERS: Main tier for consumer, High tier for
  professional, with much higher bitrate caps at the same
  level.

/* WHY THIS MATTERS PRACTICALLY: a hardware decoder implements
   a level literally. Exceed the DPB size or the bin rate and
   it REFUSES the stream or produces corruption - even though
   ffmpeg plays it fine on your desktop. A stream that is
   "1080p at 20 Mbit/s" but declares level 4.0 while actually
   using 6 reference frames will fail on devices that took the
   declaration at face value.                               */

# Check what a file declares:
$ ffprobe -v error -select_streams v:0 \
    -show_entries stream=codec_name,profile,level,width,height \
    -of default=nw=1 input.mp4
codec_name=h264
profile=High
level=40

# Constrain the encoder to a target device:
$ ffmpeg -i in.mov -c:v libx264 -profile:v high -level:v 4.0 \
    -maxrate 20M -bufsize 25M out.mp4
# x264 will then refuse settings that would violate the level,
# rather than silently producing an unplayable file.
```

> **THE 80/20 OF CODEC SELECTION**
>
> If you are unsure: **H.264 High profile, level 4.0, 4:2:0 8-bit, CRF 21, closed 2-second GOP** plays on essentially every device made since 2010 and is good enough for almost everything. Add an AV1 rendition when you have measured that the bandwidth saving is worth the encoding cost. Reaching for a newer codec before you have measured is how projects acquire compatibility bugs they cannot reproduce.

The encoders

Encoder	Codec	Character
x264	H.264	Mature, fast, superbly tuned. Still the quality/speed benchmark everything else is measured against
x265	HEVC	Good quality, slow. Presets are not comparable to x264's
SVT-AV1	AV1	The practical AV1 encoder — scalable threading, sane speed/quality curve. Presets 0 (slowest) to 13
libaom	AV1	The reference. Best quality at glacial speed; used for research and for the very top rungs
rav1e	AV1	Rust, focused on correctness and speed
NVENC / QuickSync / AMF / V4L2 M2M	Various	Hardware. 10–50× faster, noticeably worse per bit — fine for live, wasteful for VOD (chapter 16)

```
# A fair comparison, which is harder than it looks.
# Same source, same target quality metric, measure BOTH size
# and time.

$ time ffmpeg -i src.y4m -c:v libx264  -preset veryslow -crf 23 h264.mp4
$ time ffmpeg -i src.y4m -c:v libx265  -preset slow     -crf 28 hevc.mp4
$ time ffmpeg -i src.y4m -c:v libsvtav1 -preset 4 -crf 35        av1.mp4

# THE CRF SCALES ARE DIFFERENT. Roughly equivalent quality:
#   x264 crf 23  ~  x265 crf 28  ~  SVT-AV1 crf 35
# Comparing "crf 23" across all three is the most common
# benchmarking error and makes AV1 look terrible.

$ for f in h264 hevc av1; do
    printf "%-6s %8s bytes  VMAF " $f $(stat -c%s $f.mp4)
    ffmpeg -v error -i $f.mp4 -i src.y4m \
      -lavfi libvmaf -f null - 2>&1 | grep -o 'score:.*'
  done

# AND REPORT THE ENCODING TIME. A 40% bitrate saving that
# costs 20x the CPU is a business decision, not a technical
# win - it may be right for a title streamed a million times
# and clearly wrong for a security camera.
```

How it shows up

Symptom	Cause
AV1 looks worse than H.264 in your test	Comparing the same CRF number, or too fast a preset
File plays on desktop, fails on a TV or phone	Level or profile exceeded — check the DPB and reference count
HEVC will not play in a browser	Licensing, not capability. Safari only, and not always
Hardware decodes the codec but will not encode it	They are separate blocks; check both directions
4:4:4 or 10-bit H.264 will not play	High 4:4:4 / High10 profiles have poor hardware support
Battery drains fast playing AV1 on an older phone	Software decode
Encoding a live stream cannot keep up	Software encoder too slow — use hardware, or a faster preset
Editing a delivery-codec file is painfully slow	Long GOP; transcode to an intra-only editing codec first

Symptom	Cause
Legal review blocks a codec late in a project	Licensing was never checked — do it before you build

Check yourself

- ✓ Give the approximate bitrate saving of VP9, HEVC and AV1 over H.264, and the conditions that qualify those numbers.
- ✓ Explain why HEVC failed on the web despite being technically strong.
- ✓ Say what AOMedia was formed to do and what its licensing position is.
- ✓ Choose a codec for: universal web playback, a TV app, videoconferencing and editing.
- ✓ Explain the difference between hardware encode and hardware decode support.
- ✓ Distinguish a profile from a level, and say what a level caps.
- ✓ Explain why a stream can play in ffmpeg and fail on a hardware decoder.
- ✓ Give the safe default configuration for maximum compatibility.
- ✓ Say why comparing encoders at the same CRF number is wrong.
- ✓ Explain why encoding time belongs in any codec comparison.

Containers, timestamps and A/V sync

A codec produces a stream of compressed frames with no notion of time, language or what audio goes with them. The container supplies all of that — and the reason lip sync drifts, seeking misses, and files play in one player but not another is almost always here rather than in the codec.

What a container does

```
A CONTAINER (or "format", or "wrapper") provides:

  MULTIPLEXING     interleave video, audio, subtitles so a
                   player can read them in one pass without
                   seeking
  TIMING           a timestamp for every packet
  INDEXING         "the keyframe nearest 3:42 is at byte
                   offset N" - what makes seeking possible
  METADATA         title, language, chapters, cover art
  CODEC SETUP      the "extradata" a decoder needs before
                   the first frame: SPS/PPS for H.264,
                   sequence header for AV1

THE CODEC AND CONTAINER ARE INDEPENDENT. "MP4" says nothing
about the video codec inside it. A .mp4 can hold H.264,
HEVC, AV1 or MPEG-4 Part 2; an .mkv can hold anything at
all. "It is an MP4" is not a statement about
compatibility.

AND THE CONFUSION THAT FOLLOWS: a device that "supports
MP4" means it supports SOME codecs in MP4. Nearly every
playback failure reported as "this MP4 does not work" is a
codec, profile or level problem (chapter 14), not a
container problem.
```

Container	Strengths	Use for
MP4 / ISOBMFF	Universal support, good indexing, fragmentable	The default for anything delivered. Also the base of DASH and CMAF
MKV / WebM	Any codec, any number of tracks, extensible, robust to truncation	Archival, anything unusual. WebM is the restricted subset browsers accept
MPEG-TS	188-byte packets, resynchronises after loss, no index needed	Broadcast, HLS segments, anything over a lossy link
fMP4 / CMAF	Fragmented MP4 — a moov header then independent segments	Streaming. One format serving both HLS and DASH
MOV	MP4's ancestor; richer professional metadata	Editing, ProRes
Annex B / raw ES	No container at all — just NAL units with start codes	Debugging, and feeding a hardware decoder directly

Timestamps and timebases

```
/* A timestamp is an INTEGER in a container-defined TIMEBASE.
   It is not seconds.                                          */

    time_in_seconds = timestamp * timebase

  MP4        timebase = 1 / timescale, chosen per track.
             90000 is traditional; 1000 (milliseconds) common.
  MPEG-TS    always 1/90000. PCR at 27 MHz.
  MKV        1/1000000000 by default (nanoseconds).

/* WHY NOT JUST USE SECONDS: 30000/1001 fps (NTSC's 29.97)
   cannot be represented exactly in any decimal. With a
   timescale of 30000 each frame is exactly 1001 ticks, and
   there is no accumulating rounding error. Floating point
   here loses sync over a long file, slowly, in a way that is
   very hard to debug.                                         */

/* TWO TIMESTAMPS PER PACKET (chapter 10): */

    PTS   when to PRESENT it
    DTS   when to DECODE it

  With B-frames, PTS != DTS and DTS may start negative:

    packet:   I     P     B     B     P     B     B
    DTS:     -2    -1     0     1     2     3     4
    PTS:      0     3     1     2     6     4     5

  DTS is always monotonically increasing. PTS is not.
  A muxer that writes PTS where DTS belongs produces a file
  that plays wrong or not at all.

/* EDIT LISTS - MP4's way of saying "start playback at PTS
   0.0666 even though the first sample has PTS 0". Used to
   trim, to compensate for encoder delay, and for that
   negative-DTS offset.

   MANY TOOLS IGNORE THEM, which is why a file can be in sync
   in one player and 66 ms out in another. If audio and video
   are offset by exactly one or two frames, suspect an edit
   list.                                                       */

# Look at the real numbers:
$ ffprobe -select_streams v -show_entries \
    packet=pts,dts,pts_time,dts_time,duration -of csv input.mp4 | head -5
packet,0,-1024,0.000000,-0.021333,512
packet,3072,0,0.064000,0.000000,512
packet,1024,512,0.021333,0.010667,512

$ ffprobe -show_entries stream=time_base,start_time,duration \
    -select_streams v input.mp4
time_base=1/48000
start_time=0.000000
```

> **EVERY SYNC BUG IS A TIMESTAMP BUG**
>
> Audio and video are decoded by independent decoders at independent rates. The *only* thing keeping them together is their timestamps and a common clock. If lip sync drifts, nothing is wrong with either codec — a timestamp is being generated, converted, or ignored incorrectly somewhere in the chain.

How playback actually stays in sync

```
THREE POSSIBLE MASTER CLOCKS:

1. AUDIO MASTER   (what almost everyone uses)
     The audio device consumes samples at a fixed, known
     rate, so its playback position IS a clock. Video is
     presented to match.
     If video falls behind: DROP frames.
     If video runs ahead:   REPEAT or wait.
     WHY: humans notice an audio glitch instantly and a
     dropped video frame barely at all. Audio must never
     stutter, so audio wins.

2. EXTERNAL CLOCK
     The system clock, or a PCR from a broadcast stream.
     Used when both audio and video must follow a third
     party - live broadcast, or synchronised playback
     across devices.

3. VIDEO MASTER
     Rare. Used when frame-accurate video matters more than
     audio continuity - editing, frame-stepping.

THE DRIFT PROBLEM, and it is physical:
  the sound card's crystal runs at 47,999.6 Hz while
  claiming 48,000. That is 8 ppm - about 30 ms per hour.
  Over a two-hour film, a minute of accumulated error if
  nothing corrects it.

  THE FIX: RESAMPLE the audio slightly - 48000 in,
  47999.6 out - so it tracks the master clock. This is
  what every serious player does, and it is inaudible.
  The naive alternative (drop or duplicate a sample block)
  is audible as a click.

ACCEPTABLE ERROR, from ITU-R BT.1359:
  audio LEADING video:   -40 ms is detectable
  audio LAGGING video:   +60 ms is detectable
  Asymmetric because in the real world sound always
  arrives after light, so we tolerate lag better.
  Aim for +/-20 ms.
```

Fragmented MP4 and streaming

```
/* A normal MP4 has one 'moov' box holding the index for the
   entire file. Two consequences:

   1. If moov is at the END (which is where a muxer naturally
      writes it, since it needs the whole file first), a
      player must download the whole file before it can start.
      -> +faststart moves it to the front. THIS IS THE MOST
         COMMON MP4 MISTAKE.                                  */

$ ffmpeg -i in.mov -c copy -movflags +faststart out.mp4

/*  2. It cannot be produced live, because you do not know
       the index until you are finished.
       -> FRAGMENTED MP4: a small moov with no sample table,
          then a sequence of (moof + mdat) fragments, each
          self-describing.                                   */

    [ftyp][moov]  [moof][mdat] [moof][mdat] [moof][mdat] ...
                  \___________/
                   one fragment - independently decodable if
                   it starts with a keyframe

$ ffmpeg -i in.mp4 -c copy \
    -movflags "frag_keyframe+empty_moov+default_base_moof" \
    out_frag.mp4

/* CMAF is fMP4 with a specific set of constraints, so that
   ONE set of segments serves both HLS and DASH. Before CMAF
   you encoded twice - MPEG-TS for HLS, fMP4 for DASH - and
   stored both. Now you store one.

   THE REQUIREMENTS for segmented streaming, all of which
   must hold (chapters 10 and 13):
     * closed GOP, keyframe at every segment boundary
     * segment duration an exact multiple of the GOP
     * identical timestamps across every rendition
     * scene-cut keyframes DISABLED, or alignment breaks   */

# Verify segments really do start with a keyframe:
$ for s in seg_*.m4s; do
    printf "%s " $s
    ffprobe -v error -select_streams v -read_intervals "%+#1" \
      -show_entries frame=key_frame -of csv=p=0 "$s"
  done
seg_001.m4s 1
seg_002.m4s 1
seg_003.m4s 1        # all 1 = correct. Any 0 = a broken
                     # switch point.
```

> **VARIABLE FRAME RATE BREAKS ASSUMPTIONS EVERYWHERE**
>
> Screen recordings, phone cameras and browser capture routinely produce VFR: frames arrive when something changed, so durations vary. Tools that assume constant frame rate — many editors, naive muxers, anything computing `frame_number / fps` — will produce drifting sync. Convert to CFR at ingest with `-vsync cfr -r 30`, or carry the real timestamps end to end. Choosing halfway produces a file that is subtly wrong and very hard to diagnose.

Diagnosing sync

```
# 1. Are the streams offset at the start?
$ ffprobe -show_entries stream=index,codec_type,start_time \
    -of csv input.mp4
stream,0,video,0.000000
stream,1,audio,0.021333        # audio starts 21 ms later

# A small offset is normal - encoder delay. A large one, or
# one that GROWS, is the bug.

# 2. Does the gap grow over time? This distinguishes a fixed
#    offset from drift.
$ ffprobe -select_streams v -show_entries packet=pts_time \
    -of csv=p=0 input.mp4 | tail -1
7200.033333
$ ffprobe -select_streams a -show_entries packet=pts_time \
    -of csv=p=0 input.mp4 | tail -1
7203.115000          # 3 SECONDS out after 2 hours -> drift,
                     # not an offset. Suspect a clock or a
                     # wrong timebase.

# 3. Fix a fixed offset:
$ ffmpeg -i in.mp4 -itsoffset 0.15 -i in.mp4 \
    -map 0:v -map 1:a -c copy fixed.mp4

# 4. Check for VFR:
$ ffprobe -select_streams v -show_entries frame=pkt_duration_time \
    -of csv=p=0 in.mp4 | sort | uniq -c | sort -rn | head
   4021 0.033333
    118 0.041667        # mixed durations = VFR
     37 0.025000

# 5. The empirical test that beats all of the above: encode a
#    clapperboard. A visible flash and a simultaneous click,
#    then step through frame by frame and count.
$ ffmpeg -f lavfi -i "testsrc=size=640x480:rate=30" \
         -f lavfi -i "sine=frequency=1000:duration=10" \
         -t 10 synctest.mp4
```

How it shows up

Symptom	Cause
Video must fully download before playing	`moov` at the end — use `+faststart`
Sync drifts steadily over a long file	Clock drift with no resampling, or a wrong timebase
Fixed offset, constant throughout	Encoder delay or a missing edit list
In sync in one player, out in another	Edit list honoured by one and ignored by the other
Sync fine at first, drifts in an editor	VFR source treated as CFR
File will not play, "moov atom not found"	Truncated upload of a non-faststart MP4
Seeking lands in the wrong place	Missing or wrong index; MPEG-TS has none, so players estimate
ABR switch produces a glitch	Segments not starting on a keyframe, or timestamps not aligned
Audio clicks periodically	Sample dropping instead of resampling to correct drift
Remuxing broke playback	Extradata (SPS/PPS) not carried across

Check yourself

- ✓ Name five things a container provides that a codec does not.
- ✓ Explain why "it is an MP4" says nothing about compatibility.
- ✓ Explain what a timebase is and why 30000/1001 motivates integer timestamps.
- ✓ Distinguish PTS from DTS and say which is monotonic.
- ✓ Explain what an edit list does and the sync symptom of ignoring one.
- ✓ Name the three master-clock choices and say why audio usually wins.
- ✓ Explain clock drift and the correct way to compensate.
- ✓ Give the acceptable A/V sync window and why it is asymmetric.
- ✓ Explain `+faststart` and what fragmented MP4 adds beyond it.
- ✓ Distinguish a fixed offset from drift using timestamps.

Hardware encoders and decoders

A dedicated block that decodes 4K60 at a fraction of a watt, where software would need several CPU cores. It is not a fast CPU — it is a fixed-function pipeline with fixed capabilities, and knowing what it can and cannot do decides whether your product works on battery.

Why the silicon exists

```
DECODING 4K60 HEVC

  software, 8 CPU cores      ~25 W, and the phone is now
                             unusable for anything else
  hardware VPU               ~0.5 W

A 50x power difference. That is the entire argument, and it
is why every SoC has one (see the SoC section on the home
page). Battery life on video playback is almost entirely a
question of whether the hardware path was used.

WHAT THE BLOCK ACTUALLY CONTAINS

  +-------------------------------------------------+
  |  entropy decoder (CABAC)  - the serial bottleneck  |
  |    -> its throughput sets the max bitrate the      |
  |       chip supports (chapter 8)                    |
  |  inverse transform + dequantise                    |
  |  motion compensation - the interpolation filters   |
  |  intra prediction                                  |
  |  in-loop filters: deblock, SAO, CDEF               |
  |  reference frame DMA + a reference cache           |
  +-------------------------------------------------+
                |
          DRAM: the DPB (chapter 10)

IT IS A PIPELINE, not a processor. Every stage is
hard-wired for specific codecs, specific profiles and
specific block sizes. There is no microcode to change - a
chip that decodes H.264 and HEVC will never decode AV1,
ever, at any clock speed.

WHICH IS WHY the codec landscape (chapter 14) moves so
slowly: adopting a new codec means waiting for a hardware
generation, then for the installed base to turn over.
Roughly five to eight years.
```

HARDWARE ENCODE IS WORSE PER BIT THAN SOFTWARE, ALWAYS

A fixed-function encoder has a fixed, shallow search: limited motion search range, few partition candidates, no multi-pass lookahead, simplified rate-distortion decisions. Expect **20–50% more bitrate** than `x264 --preset slow` at the same quality. That is the price of 50× the speed and 1/50th the power. Use hardware for live and for anything power-constrained; use software for VOD you encode once and stream a million times.

The constraints that bite

```
/* A hardware codec has HARD limits, and it fails ungracefully
   when you exceed them - usually a silent fall back to
   software (slow), or an error code with no explanation. */

  CODEC AND PROFILE
      "H.264 decode" means specific profiles. High 4:4:4 or
      High10 is often absent even where High is present
      (chapter 14).

  LEVEL
      Max resolution, frame rate, bitrate, DPB size and BIN
      RATE. A stream can be within the bitrate and still
      exceed the bin rate.

  PIXEL FORMAT
      Almost always NV12 in, NV12 out (chapter 4). Feeding it
      RGB means a conversion pass you did not budget for.

  ALIGNMENT
      Width and height usually to 16 or 32; stride to 128 or
      256; the buffer base to 4 KB. A 1920x1080 surface is
      often physically 1920x1088 - the extra 8 rows are real
      and must not be displayed.

  MEMORY
      Physically contiguous, or IOMMU-mapped. Often a specific
      heap - CMA, or a carve-out. malloc'd memory will not do.

  TILING
      The output may be in a vendor tiled or compressed layout
      (chapter 4). Reading it with the CPU without detiling
      gives you scrambled rectangles.

  CONCURRENCY
      Usually ONE session, or a small fixed number. Two apps
      both wanting hardware decode is a real failure mode on
      phones.

  LATENCY
      The pipeline is deep. Frames go in and come out several
      later - you must feed it continuously and collect
      asynchronously, never call "decode this frame" and wait.

/* THE 1088 PROBLEM, concretely: H.264 codes in 16x16
   macroblocks, so 1080 rounds up to 1088. The bitstream
   carries CROPPING information saying "display 1080 of these
   1088 rows". Ignore it and you get 8 rows of garbage at the
   bottom - one of the most common hardware-decode bugs, and
   it looks like a driver fault.                          */
```

The APIs

API	Platform	Notes
V4L2 M2M	Linux, embedded	The kernel-native interface. Stateful and stateless variants (chapter 17)
VA-API	Linux, Intel and AMD	The desktop standard. Exposes both decode and encode
NVDEC / NVENC	NVIDIA, all platforms	Via the Video Codec SDK, or ffmpeg's `_nvenc` / `_cuvid`

API	Platform	Notes
MediaCodec	Android	The only sanctioned path. Wraps whatever the vendor provides
VideoToolbox	Apple	Clean API, excellent hardware
DXVA2 / D3D11VA / D3D12 Video	Windows	Decode via the graphics stack; integrates with the swap chain
Media Foundation	Windows	Higher-level; wraps DXVA and vendor encoders
VDPAU	Linux, legacy	Decode only, largely superseded by VA-API

```
# What does this machine actually have?

$ vainfo                                  # VA-API
libva info: VA-API version 1.20.0
vainfo: Driver version: Intel iHD driver
      VAProfileH264High               : VAEntrypointVLD
      VAProfileH264High               : VAEntrypointEncSlice
      VAProfileHEVCMain               : VAEntrypointVLD
      VAProfileHEVCMain10             : VAEntrypointVLD
      VAProfileAV1Profile0            : VAEntrypointVLD
#     VLD = decode.  EncSlice = encode.
#     Note AV1 appears for VLD only - decode but not encode.

$ v4l2-ctl --list-devices                 # V4L2
$ ls /dev/video* /dev/media*

$ nvidia-smi --query-gpu=name --format=csv  # NVIDIA
$ ffmpeg -hide_banner -encoders | grep -E 'nvenc|vaapi|qsv|v4l2m2m'
 V....D h264_nvenc    NVIDIA NVENC H.264 encoder
 V....D h264_vaapi    H.264/AVC (VAAPI)
 V....D h264_v4l2m2m  V4L2 mem2mem H.264 encoder

$ ffmpeg -hide_banner -hwaccels
Hardware acceleration methods:
vdpau  cuda  vaapi  qsv  drm  vulkan

# THE TEST THAT MATTERS - did it actually use hardware?
$ ffmpeg -hwaccel vaapi -hwaccel_device /dev/dri/renderD128 \
    -hwaccel_output_format vaapi -i in.mp4 -f null - 2>&1 | tail -3
# then watch the GPU's video engine:
$ sudo intel_gpu_top          # look for the "Video" row
$ nvidia-smi dmon -s u        # the "dec" and "enc" columns
# If those stay at 0%, you are on the software path and
# something silently fell back.
```

Zero-copy, and why it is the whole game

```
THE NAIVE PIPELINE - four copies of every frame:

  decoder -> DRAM -> CPU reads -> converts NV12 to RGB
          -> DRAM -> GPU uploads as a texture
          -> DRAM -> composites -> display

  At 4K60 NV12 that is 12 MB per frame, 750 MB/s per copy.
  Four copies plus a colour conversion is several GB/s of
  memory traffic, and on a phone that is the entire
  bandwidth budget (mobile GPU chapter 10).

THE ZERO-COPY PIPELINE - one buffer, passed by handle:

  decoder writes NV12 into a DMABUF
     |  export as an fd
     v
  GPU imports the SAME memory as an EGLImage / VkImage
     |
     v
  display controller scans out the SAME memory directly,
  converting NV12 to RGB in its own hardware plane
  (chapter 18)

  ZERO copies. The pixels are written once, by the decoder,
  and read once, by the display.

WHAT MAKES IT WORK, and all four must hold:
  1. DMABUF - a kernel object representing a buffer,
     passed between drivers as a file descriptor
  2. an agreed PIXEL FORMAT (NV12) and MODIFIER (the exact
     tiling - chapter 4)
  3. an IOMMU or contiguous memory so every device can
     address it
  4. explicit or implicit FENCING so the GPU does not read
     a frame the decoder is still writing

WHEN ONE OF THOSE FAILS the stack silently inserts a copy,
or a format conversion, or both. Your pipeline still works
and uses three times the power. This is the single most
common performance problem in embedded video, and it is
invisible unless you measure memory bandwidth.
```

> **"IT WORKS" AND "IT IS ON THE FAST PATH" ARE DIFFERENT QUESTIONS**
>
> Every media framework falls back gracefully. Ask for hardware decode into a GPU texture and, if any link in the chain cannot be negotiated, you will get a working pipeline with a hidden `memcpy` and a software colour conversion. Frame rate may even be fine on a desktop. The way to know is to measure: GPU video engine utilisation, memory bandwidth, and power — not whether the picture appears.

How it shows up

Symptom	Cause
8 rows of garbage at the bottom of the picture	Cropping information ignored — 1080 versus 1088
Hardware decode "works" but CPU is at 100%	Silent fallback to software; check the video engine counter
Battery drains during playback	Software decode, or a copy-heavy pipeline

Symptom	Cause
Image is scrambled rectangles	Tiled output read as linear
Decoder rejects a stream ffmpeg plays fine	Profile, level, DPB size or bin rate exceeded
Second app cannot get a decoder	Limited concurrent sessions
Decode stalls waiting for output	Pipeline depth — feed continuously, collect asynchronously
Buffer allocation fails	Needs contiguous or heap-specific memory, not `malloc`
Hardware encode is 40% bigger than x264	Expected — shallow search
Tearing or a frame torn between two images	Missing fence between decoder and consumer

Check yourself

- ✓ Give the rough power ratio between hardware and software 4K decode.
- ✓ Explain why a hardware block cannot gain support for a new codec.
- ✓ Say how much worse hardware encode is per bit, and when that is the right trade.
- ✓ Name six hard constraints a hardware codec imposes.
- ✓ Explain the 1080/1088 cropping problem and its symptom.
- ✓ Name the hardware codec API on Linux embedded, Android, Apple and Windows.
- ✓ Show how to verify that hardware acceleration is actually being used.
- ✓ Explain what DMABUF is and the four conditions zero-copy requires.
- ✓ Estimate the memory traffic of a four-copy 4K60 pipeline.
- ✓ Explain why a working pipeline is not necessarily a fast one.

V4L2: capture and memory-to-memory

Video4Linux2 is how Linux talks to cameras, capture cards, and — through the memory-to-memory extension — hardware codecs and scalers. It is an ioctl-based API with a buffer-queue model, and once you have internalised that model, every V4L2 device works the same way.

The buffer queue model

```
V4L2 is built on TWO QUEUES per stream. You never allocate
and free per frame; you hand buffers to the driver and get
them back.

  userspace                       driver
     |                              |
     |  VIDIOC_QBUF  ------------>  | INPUT QUEUE
     |  "here is an empty buffer"   |   driver fills it
     |                              |     from the sensor
     |                              v
     |                           [ hardware DMA ]
     |                              |
     |  VIDIOC_DQBUF <------------  | DONE QUEUE
     |  "here is a full one"        |
     v
  process it, then QBUF it again

THE LIFECYCLE, and every V4L2 program is these seven steps:

  1. open("/dev/video0")
  2. VIDIOC_QUERYCAP     what can this device do?
  3. VIDIOC_S_FMT        set resolution and pixel format
  4. VIDIOC_REQBUFS      ask the driver to allocate N
                         buffers (or accept yours)
  5. VIDIOC_QUERYBUF + mmap    map them into your address
                         space
  6. VIDIOC_QBUF x N     hand them all over
     VIDIOC_STREAMON     go
  7. loop: poll() -> VIDIOC_DQBUF -> use -> VIDIOC_QBUF

THE THREE MEMORY MODELS

  V4L2_MEMORY_MMAP     the DRIVER allocates; you mmap.
                       The default, and what most devices
                       support.
  V4L2_MEMORY_USERPTR  YOU allocate; you pass a pointer.
                       Needs an IOMMU or contiguous memory,
                       and many drivers refuse it.
  V4L2_MEMORY_DMABUF   you pass an fd for a buffer another
                       driver allocated.
                       THIS IS THE ZERO-COPY ONE
                       (chapter 16), and it is what you
                       want for anything real.
```

```
/* A minimal capture loop. This is the whole API. */

#include <linux/videodev2.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <poll.h>
#include <string.h>
#include <errno.h>

/* EVERY ioctl must be retried on EINTR. A signal — including
   the one your profiler sends — will otherwise look like a
   device failure. */
static int xioctl(int fd, unsigned long req, void *arg)
{
        int r;
        do { r = ioctl(fd, req, arg); }
        while (r == -1 && errno == EINTR);
        return r;
}

#define NBUF 4
struct buffer { void *start; size_t length; };
static struct buffer bufs[NBUF];

int capture_init(const char *dev, int w, int h, uint32_t fourcc)
{
        int fd = open(dev, O_RDWR | O_NONBLOCK);
        if (fd < 0) return -1;

        /* 2. What is this? */
        struct v4l2_capability cap = {0};
        if (xioctl(fd, VIDIOC_QUERYCAP, &cap) < 0) return -1;
        if (!(cap.capabilities & V4L2_CAP_VIDEO_CAPTURE)) return -1;
        if (!(cap.capabilities & V4L2_CAP_STREAMING))     return -1;

        /* 3. Set the format. THE DRIVER MAY CHANGE IT — always
              read back what you actually got. Asking for
              1920x1080 NV12 and getting 1280x720 YUYV without
              noticing is the classic V4L2 mistake. */
        struct v4l2_format fmt = {0};
        fmt.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        fmt.fmt.pix.width       = w;
        fmt.fmt.pix.height      = h;
        fmt.fmt.pix.pixelformat = fourcc;
        fmt.fmt.pix.field       = V4L2_FIELD_NONE;
        if (xioctl(fd, VIDIOC_S_FMT, &fmt) < 0) return -1;

        if (fmt.fmt.pix.pixelformat != fourcc)
                fprintf(stderr, "driver chose a different format\n");
        /* AND KEEP THE STRIDE the driver reports — it is not
           width * bpp (chapter 1). */
        int stride = fmt.fmt.pix.bytesperline;

        /* 4. Buffers. */
        struct v4l2_requestbuffers req = {0};
        req.count  = NBUF;
        req.type   = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        req.memory = V4L2_MEMORY_MMAP;
        if (xioctl(fd, VIDIOC_REQBUFS, &req) < 0) return -1;
        /* req.count may come back SMALLER than you asked. */

        /* 5. Map them. */
```

```
        for (unsigned i = 0; i < req.count; i++) {
                struct v4l2_buffer b = {0};
                b.type   = V4L2_BUF_TYPE_VIDEO_CAPTURE;
                b.memory = V4L2_MEMORY_MMAP;
                b.index  = i;
                if (xioctl(fd, VIDIOC_QUERYBUF, &b) < 0) return -1;

                bufs[i].length = b.length;
                bufs[i].start  = mmap(NULL, b.length,
                                      PROT_READ | PROT_WRITE,
                                      MAP_SHARED, fd, b.m.offset);
                if (bufs[i].start == MAP_FAILED) return -1;
        }

        /* 6. Queue them all, then start. */
        for (unsigned i = 0; i < req.count; i++) {
                struct v4l2_buffer b = {0};
                b.type   = V4L2_BUF_TYPE_VIDEO_CAPTURE;
                b.memory = V4L2_MEMORY_MMAP;
                b.index  = i;
                if (xioctl(fd, VIDIOC_QBUF, &b) < 0) return -1;
        }
        enum v4l2_buf_type t = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        if (xioctl(fd, VIDIOC_STREAMON, &t) < 0) return -1;

        return fd;
}

/* 7. The loop. */
int capture_frame(int fd, void (*use)(void *, size_t, struct timeval))
{
        struct pollfd p = { .fd = fd, .events = POLLIN };
        if (poll(&p, 1, 2000) <= 0) return -1;   /* timeout */

        struct v4l2_buffer b = {0};
        b.type   = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        b.memory = V4L2_MEMORY_MMAP;
        if (xioctl(fd, VIDIOC_DQBUF, &b) < 0) return -1;

        /* b.timestamp is the CAPTURE time, from the driver.
           Use it - do not stamp frames when your code happens
           to run, or you build jitter into every downstream
           timestamp (chapter 15). */
        use(bufs[b.index].start, b.bytesused, b.timestamp);

        /* GIVE IT BACK IMMEDIATELY. Holding buffers starves
           the driver: with 4 buffers, holding 4 means the
           hardware has nowhere to write and DROPS FRAMES. */
        return xioctl(fd, VIDIOC_QBUF, &b);
}
```

BUFFER COUNT IS A LATENCY-VERSUS-DROPS DIAL

Too few and the driver has nowhere to write while you are processing, so frames are dropped. Too many and you add latency — each queued frame is one frame period of delay before yours is displayed. Four to six is the usual range for capture; a real-time pipeline uses the minimum that never drops. Check `v4l2_buffer.sequence` for gaps: a jump means frames were lost, and the driver will not tell you any other way.

Memory-to-memory: codecs and scalers

```
The same queue model, but with TWO streams on ONE device:

  OUTPUT queue   what you send TO the device
                 (confusingly named - "output" is from the
                  APPLICATION's point of view)
                 V4L2_BUF_TYPE_VIDEO_OUTPUT_MPLANE

  CAPTURE queue  what you get back
                 V4L2_BUF_TYPE_VIDEO_CAPTURE_MPLANE

FOR A DECODER
  OUTPUT  = compressed bitstream in
  CAPTURE = decoded NV12 frames out

FOR AN ENCODER
  OUTPUT  = raw NV12 frames in
  CAPTURE = compressed bitstream out

Same device, same ioctls, both queues streaming
simultaneously. You poll for both.

TWO KINDS OF M2M CODEC, and the difference is fundamental:

  STATEFUL   the driver parses the bitstream itself, finds
             the headers, manages the DPB, handles
             resolution changes. You feed it bytes and get
             frames. Most vendor drivers.

  STATELESS  the driver does ONE frame at a time and you
             supply ALL the parsed state - SPS, PPS, slice
             parameters, the reference list - through
             V4L2 request-API controls. The hardware is
             dumber and the kernel driver is far smaller;
             userspace (ffmpeg, GStreamer, Chromium) does
             the parsing.
             Used by Rockchip, Allwinner, RPi and the newer
             mainline drivers. It is where the kernel is
             heading, because it keeps codec-specific
             complexity out of the kernel.
```

```
/* The decoder sequence, stateful. The asymmetry in step 4 is
   the part people get wrong. */

  1. open the m2m device, VIDIOC_QUERYCAP
       -> V4L2_CAP_VIDEO_M2M_MPLANE

  2. S_FMT on OUTPUT: the compressed format
       fmt.type = V4L2_BUF_TYPE_VIDEO_OUTPUT_MPLANE
       fmt.fmt.pix_mp.pixelformat = V4L2_PIX_FMT_H264
       fmt.fmt.pix_mp.plane_fmt[0].sizeimage = 1024*1024
       /* you must size the bitstream buffer yourself */

  3. REQBUFS + QBUF on OUTPUT, STREAMON, and feed the first
     chunk of bitstream - INCLUDING SPS/PPS.

  4. WAIT FOR THE SOURCE CHANGE EVENT.
       VIDIOC_SUBSCRIBE_EVENT V4L2_EVENT_SOURCE_CHANGE
       The driver parses the headers and only THEN knows the
       real resolution. You cannot set the CAPTURE format
       before this - you do not know it yet.

       On the event:
         G_FMT on CAPTURE     -> the real resolution
         G_SELECTION          -> the CROP rectangle
                                 (the 1080-vs-1088 problem,
                                  chapter 16)
         REQBUFS on CAPTURE   -> now you can allocate
         STREAMON on CAPTURE

  5. Now both queues stream. Poll for both:
       POLLOUT -> an OUTPUT buffer is free, feed more bitstream
       POLLIN  -> a CAPTURE buffer holds a decoded frame

  6. Resolution can change MID-STREAM. You get another
     SOURCE_CHANGE event, and must:
       STREAMOFF capture, REQBUFS(0), reallocate, STREAMON.
     Handling this is what separates a demo from a product.

  7. Draining at end of stream:
       VIDIOC_DECODER_CMD with V4L2_DEC_CMD_STOP
       keep dequeuing CAPTURE until you get a buffer with
       V4L2_BUF_FLAG_LAST set.
     Skip this and you lose the last few frames - the pipeline
     is deep (chapter 16).
```

THE MULTIPLANAR API IS NOT OPTIONAL

Formats with separate planes — NV12, I420 — need `V4L2_BUF_TYPE_*_MPLANE` and the `v4l2_pix_format_mplane` structures, where each plane has its own length, stride and DMABUF fd. Most codec drivers accept *only* the multiplanar variants. Code written against the single-planar API works for a YUYV webcam and fails immediately on a hardware codec, with an error that just says `EINVAL`.

The tools

```
# Everything about a device:
$ v4l2-ctl -d /dev/video0 --all
Driver Info:
        Driver name      : uvcvideo
        Card type        : Integrated Camera
        Capabilities     : 0x84a00001
                Video Capture
                Streaming
Format Video Capture:
        Width/Height     : 1920/1080
        Pixel Format     : 'YUYV'
        Bytes per Line   : 3840
        Size Image       : 4147200

# What formats, sizes and rates does it support?
$ v4l2-ctl -d /dev/video0 --list-formats-ext

# Capture frames straight to a file:
$ v4l2-ctl -d /dev/video0 \
    --set-fmt-video=width=1920,height=1080,pixelformat=NV12 \
    --stream-mmap --stream-count=100 --stream-to=out.nv12

# Controls - exposure, gain, focus:
$ v4l2-ctl -d /dev/video0 --list-ctrls
$ v4l2-ctl -d /dev/video0 --set-ctrl=exposure_auto=1
$ v4l2-ctl -d /dev/video0 --set-ctrl=exposure_absolute=300

# THE MEDIA CONTROLLER. On an embedded SoC the camera is not
# one device but a GRAPH: sensor -> CSI receiver -> ISP ->
# several /dev/video nodes. You must configure the LINKS and
# the format at EVERY pad, or streaming fails with EPIPE.
$ media-ctl -d /dev/media0 -p          # print the graph
$ media-ctl -d /dev/media0 \
    -V "'ov5640 1-003c':0 [fmt:UYVY8_2X8/1920x1080]"
$ media-ctl -d /dev/media0 -l "'csi':1 -> 'isp':0 [1]"

# EPIPE from STREAMON on an embedded platform almost always
# means a pad format mismatch somewhere in that graph - the
# formats must agree link by link.

# ffmpeg through V4L2:
$ ffmpeg -f v4l2 -input_format nv12 -video_size 1920x1080 \
    -i /dev/video0 -c:v h264_v4l2m2m -b:v 4M out.mp4

# Kernel-side tracing when nothing works:
$ echo 0x1f | sudo tee /sys/class/video4linux/video0/dev_debug
$ dmesg -w
```

How it shows up

Symptom	Cause
Frames come out at the wrong size or format	Did not read back what `S_FMT` actually set
Image sheared diagonally	`bytesperline` ignored (chapter 1)
Frames dropped under load	Buffers held too long — requeue immediately; check `sequence` for gaps
Latency of several frames	Too many buffers queued

Symptom	Cause
`EINVAL` from every codec ioctl	Single-planar API against a multiplanar device
`EPIPE` from `STREAMON` on embedded	Media-controller pad formats do not match link by link
Decoder produces nothing	Waiting for frames before handling `SOURCE_CHANGE`
Last few frames missing	No drain — send `DEC_CMD_STOP` and wait for `BUF_FLAG_LAST`
8 rows of garbage at the bottom	`G_SELECTION` crop rectangle ignored
Random `EINTR` failures	ioctls not retried
Stream breaks when resolution changes mid-file	Second `SOURCE_CHANGE` not handled

Check yourself

- ✓ Describe the V4L2 queue model and the seven steps of a capture program.
- ✓ Name the three memory models and say which enables zero-copy.
- ✓ Explain why every ioctl must be retried on `EINTR`.
- ✓ Say why you must read back the format after `S_FMT`.
- ✓ Explain the buffer-count trade and how to detect dropped frames.
- ✓ Explain what OUTPUT and CAPTURE mean on an m2m device, for both a decoder and an encoder.
- ✓ Distinguish stateful from stateless codec drivers and say which direction the kernel is taking.
- ✓ Explain why the CAPTURE format cannot be set before `SOURCE_CHANGE`.
- ✓ Describe the drain sequence and what is lost without it.
- ✓ Say what the media controller configures and what `EPIPE` usually means.

DRM/KMS, planes and zero-copy display

The last stage of the pipeline, and the one that decides whether video playback costs half a watt or five. The display controller can scan out NV12 directly, scale it, convert its colour and composite it — if you ask correctly. If you do not, the GPU does all of that instead, at ten times the power.

The KMS object model

```
Kernel Mode Setting exposes the display hardware as five
object types. Every display configuration is a set of
properties on these.

FRAMEBUFFER    a buffer plus its format and modifier
     |         (chapter 4). Wraps a DMABUF or GEM object.
     v
PLANE          a hardware scanout engine. Reads a
     |         framebuffer, optionally scales it, converts
     |         its colour, and positions it.
     |           PRIMARY  the main one, one per CRTC
     |           OVERLAY  extra planes for compositing
     |           CURSOR   small, always on top
     v
CRTC           the scanout engine proper. Owns the mode
     |         (resolution, refresh, timings), blends the
     |         planes, and generates the pixel clock.
     v
ENCODER        converts the pixel stream to a signal -
     |         HDMI, DisplayPort, MIPI DSI, LVDS.
     v
CONNECTOR      the physical port and the attached display.
               Carries EDID: what the monitor supports.

THE CHAIN: framebuffer -> plane -> CRTC -> encoder ->
connector -> the screen.

PLANES ARE THE POINT OF THIS CHAPTER. A hardware plane can
do, for free and in the display controller:
  * YUV to RGB conversion
  * scaling
  * positioning and cropping
  * alpha blending with the other planes
  * (sometimes) rotation and colour management

That is exactly the work a GPU composite would otherwise
do, and the display controller does it while reading the
pixels it has to read anyway - so it costs no extra memory
bandwidth at all.
```

THE VIDEO PLANE IS WHY PHONES CAN PLAY 4K FOR HOURS

Put decoded NV12 on an overlay plane and the pipeline is: decoder writes the frame once, display controller reads it once. The GPU is never involved and can stay powered down. Composite the same frame with the GPU instead and you add a texture read, a colour conversion, a render pass and a write — several hundred megabytes per second at 4K60, and the GPU cannot idle. On mobile that is the difference between a warm phone and a hot one.

Atomic modesetting

```
/* The old API set one thing at a time - mode, then plane,
   then cursor - and each could fail halfway, leaving the
   display in a broken intermediate state. The ATOMIC API
   submits every change as ONE transaction that either applies
   completely or not at all.                                  */

#include <xf86drm.h>
#include <xf86drmMode.h>

int fd = open("/dev/dri/card0", O_RDWR | O_CLOEXEC);

/* You must ask for the modern capabilities explicitly. */
drmSetClientCap(fd, DRM_CLIENT_CAP_UNIVERSAL_PLANES, 1);
drmSetClientCap(fd, DRM_CLIENT_CAP_ATOMIC, 1);

/* Build a request from property changes. */
drmModeAtomicReq *req = drmModeAtomicAlloc();

drmModeAtomicAddProperty(req, plane_id, prop_fb_id,   fb_id);
drmModeAtomicAddProperty(req, plane_id, prop_crtc_id, crtc_id);
/* SRC coordinates are 16.16 FIXED POINT - the source
   rectangle within the framebuffer. */
drmModeAtomicAddProperty(req, plane_id, prop_src_x, 0);
drmModeAtomicAddProperty(req, plane_id, prop_src_y, 0);
drmModeAtomicAddProperty(req, plane_id, prop_src_w, 1920 << 16);
drmModeAtomicAddProperty(req, plane_id, prop_src_h, 1080 << 16);
/* CRTC coordinates are plain integers - where it lands on
   screen. Different units for source and destination, in the
   same call, is a reliable source of bugs. */
drmModeAtomicAddProperty(req, plane_id, prop_crtc_x, 0);
drmModeAtomicAddProperty(req, plane_id, prop_crtc_y, 0);
drmModeAtomicAddProperty(req, plane_id, prop_crtc_w, 3840);
drmModeAtomicAddProperty(req, plane_id, prop_crtc_h, 2160);
/* src != crtc size means the plane SCALES - for free. */

/* TEST_ONLY asks "would this work?" without changing
   anything. This is the single most useful feature of the
   atomic API: probe your configuration, fall back if the
   hardware cannot do it, and never leave the display broken. */
int ret = drmModeAtomicCommit(fd, req,
                              DRM_MODE_ATOMIC_TEST_ONLY, NULL);
if (ret == 0)
        drmModeAtomicCommit(fd, req,
                            DRM_MODE_ATOMIC_NONBLOCK |
                            DRM_MODE_PAGE_FLIP_EVENT, user_data);
else
        fallback_to_gpu_composite();

drmModeAtomicFree(req);

/* PAGE_FLIP_EVENT means the commit happens at the next
   VBLANK and you get an event on the fd when it has. That is
   how you avoid tearing and how you pace your rendering -
   read the event, then draw the next frame. */
```

Getting video onto a plane

```
THE FULL ZERO-COPY PATH (chapter 16):

  V4L2 decoder
    VIDIOC_EXPBUF  ->  dmabuf fd
          |
          v
  drmPrimeFDToHandle(fd, dmabuf_fd, &gem_handle)
          |
          v
  drmModeAddFB2WithModifiers(
          fd, w, h, DRM_FORMAT_NV12,
          handles,      /* the same handle for both planes */
          pitches,      /* per-plane stride                */
          offsets,      /* per-plane offset into the buffer*/
          modifiers,    /* THE TILING - must match!        */
          &fb_id, DRM_MODE_FB_MODIFIERS)
          |
          v
  atomic commit: put fb_id on an overlay plane
          |
          v
  the display controller scans it out, converting NV12 to
  RGB in hardware.

THE FOUR THINGS THAT MUST LINE UP, and any one of them
failing sends you back to a GPU composite:

  1. FORMAT   the plane must advertise NV12 (or P010 for
              10-bit). Query IN_FORMATS on the plane.
  2. MODIFIER the exact tiling the decoder produced must be
              one the plane accepts (chapter 4).
  3. SCALING  the plane's min/max scale factors. Many
              overlay planes cannot scale at all, and some
              cannot scale down below 1:2.
  4. BANDWIDTH the display controller has a fixed budget.
              Two 4K planes may simply exceed it, and the
              commit fails at TEST_ONLY.

AND COLOUR: a YUV plane needs its properties set, or you
get the washed-out or crushed picture of chapter 2:
  COLOR_ENCODING  ITU-R BT.601 / BT.709 / BT.2020
  COLOR_RANGE     YCbCr limited range / YCbCr full range
Defaults are usually BT.601 limited, which is wrong for
almost all modern content.
```

```
# What planes does this hardware have, and what can they do?
$ sudo modetest -M i915 -p
Planes:
id      crtc    fb      CRTC x,y   x,y    gamma size    possible crtcs
31      0       0       0,0        0,0    0               0x00000001
  formats: XR24 AR24 NV12 P010 YUYV
  props:
        6 IN_FORMATS:
                flags: immutable blob
                value blob = 42
                        NV12: LINEAR, I915_FORMAT_MOD_Y_TILED,
                              I915_FORMAT_MOD_4_TILED
        20 type: OVERLAY               <-- an overlay plane
        68 COLOR_ENCODING: ITU-R BT.709
        69 COLOR_RANGE: YCbCr limited range

# The whole display topology:
$ sudo modetest -M i915 -c        # connectors and modes
$ sudo modetest -M i915 -e        # encoders
$ sudo modetest -M i915 -a -s 32@31:1920x1080   # test a mode

# Is the compositor actually using a plane, or falling back?
$ sudo cat /sys/kernel/debug/dri/0/i915_display_info
$ sudo cat /sys/kernel/debug/dri/0/state
plane[31]: plane 1B
        crtc-pos: 3840x2160+0+0
        src-pos: 1920.000000x1080.000000+0.000000+0.000000
        format=NV12 modifier=0x100000000000002
        color-encoding=ITU-R BT.709
#  ^ NV12 on a plane, scaled 1920 -> 3840. This is the fast
#    path working.

# On Wayland, ask the compositor:
$ WAYLAND_DEBUG=1 mpv --gpu-context=wayland video.mp4 2>&1 \
    | grep -i 'dmabuf\|plane'
$ sudo cat /sys/kernel/debug/dri/0/state | grep -c 'format=NV12'
# 0 means every frame is being composited by the GPU.
```

PLANE ASSIGNMENT IS A NEGOTIATION YOU CAN LOSE SILENTLY

A compositor tries to put your video on a hardware plane and falls back to GPU composition when it cannot — because another window overlaps it, because the scaling factor is out of range, because a rounded corner needs alpha, or because bandwidth is exhausted. Nothing tells you. The picture is identical; the power draw triples. On Linux the way to know is `/sys/kernel/debug/dri/0/state`; on Android it is `dumpsys SurfaceFlinger` and the `DEVICE` versus `CLIENT` composition column.

Tearing, vblank and presentation

```
/* TEARING: the scanout reads the framebuffer while you are
   writing it, so the top of the screen shows the old frame
   and the bottom the new one.

   THE FIX: DOUBLE BUFFERING plus a flip at VBLANK - the
   interval between the last line of one frame and the first
   of the next, when nothing is being read.                */

  drmModePageFlip(fd, crtc, new_fb, DRM_MODE_PAGE_FLIP_EVENT,
                  user_data);
  /* returns immediately; the flip happens at the next vblank */

  struct pollfd p = { .fd = drm_fd, .events = POLLIN };
  poll(&p, 1, -1);
  drmHandleEvent(drm_fd, &evctx);    /* your page_flip_handler
                                        runs; now the old
                                        buffer is free to
                                        reuse               */

/* THE RULE: do not touch a buffer until you have been told
   the flip completed. It is exactly the fence discipline of
   GPU chapter 16, in a different API.

/* VARIABLE REFRESH RATE - the display waits for you instead
   of the other way round. FreeSync/G-Sync/HDMI VRR, exposed
   as the CRTC's VRR_ENABLED property. Removes judder when
   the content rate does not divide the refresh rate.

/* AND THE CADENCE PROBLEM, which VRR was invented for:
     24 fps film on a 60 Hz display needs 3:2 PULLDOWN -
     frame A for 3 refreshes, frame B for 2, alternating.
     The uneven timing is visible as JUDDER on slow pans.
     A 120 Hz display divides evenly by both 24 and 30, which
     is why 120 Hz panels matter more for film than for games.

/* MULTIPLE PLANES, ONE COMMIT: the atomic API lets you flip
   the UI plane and the video plane in the same transaction,
   so they change together. Flipping them separately means one
   frame where the subtitle belongs to the previous video
   frame - a subtle artefact that is very hard to attribute.*/
```

How it shows up

Symptom	Cause
Video playback drains the battery	GPU compositing instead of a hardware plane
Washed-out or crushed video	`COLOR_ENCODING` / `COLOR_RANGE` plane properties not set (chapter 2)
Atomic commit fails with `EINVAL`	Format, modifier, scaling range or bandwidth — use `TEST_ONLY` to find out which
Fast path works alone, stops when a UI element overlaps	The compositor fell back; nothing reports it
Horizontal tear across the picture	Writing the scanout buffer without waiting for vblank
Video and subtitles are one frame out of step	Planes flipped in separate commits
Judder on slow pans	3:2 pulldown — 24 fps on a 60 Hz display

Symptom	Cause
Scrambled rectangles on screen	Modifier mismatch between decoder and plane
Second display cannot be enabled at full resolution	Display controller bandwidth exhausted
Works on one SoC, falls back on another	Different plane capabilities — query `IN_FORMATS`, never assume

Check yourself

- ✓ Name the five KMS object types and the chain from buffer to screen.
- ✓ List five things a hardware plane does for free, and say why they cost no extra bandwidth.
- ✓ Explain why atomic modesetting replaced the legacy API.
- ✓ Say what `TEST_ONLY` is for and why it is the most useful part of the API.
- ✓ Explain the units of the source and destination rectangles and the bug from confusing them.
- ✓ Trace a decoded frame from V4L2 to a plane, naming each call.
- ✓ Give the four conditions that must hold for the zero-copy path.
- ✓ Say which colour properties a YUV plane needs and what the defaults usually are.
- ✓ Explain the vblank flip discipline and what must not be touched until the event arrives.
- ✓ Determine from a running Linux system whether video is on a plane or being composited.

Media frameworks and acceleration APIs

Between your application and the hardware sits a stack of abstractions whose job is to hide the differences between vendors — and whose main failure mode is to hide the fact that it silently stopped using the hardware at all.

The stack

```
YOUR APPLICATION
     |
+-------------------------------------------------+
|  PIPELINE FRAMEWORK                             |
|    GStreamer, ffmpeg/libav, MediaFoundation,    |
|    AVFoundation, ExoPlayer                      |
|    - demux, decode, filter, render, sync        |
+-------------------------------------------------+
     |
+-------------------------------------------------+
|  ACCELERATION API                               |
|    VA-API, V4L2 M2M, NVDEC, MediaCodec,         |
|    VideoToolbox, DXVA2 / D3D11VA                |
+-------------------------------------------------+
     |
+-------------------------------------------------+
|  KERNEL DRIVER                                  |
|    V4L2, DRM/KMS, vendor VPU driver             |
+-------------------------------------------------+
     |
   HARDWARE

EACH LAYER CAN FALL BACK. That is the design goal - a
pipeline that always works - and the operational problem,
because "always works" and "works efficiently" are
different properties and only one of them is reported.
```

Framework	Where	Character
ffmpeg / libav*	Everywhere	The universal toolkit. A library and a CLI; every other framework uses it or reimplements it
GStreamer	Linux, embedded	A graph of elements negotiating caps. Powerful, and the negotiation is where the difficulty lives
MediaCodec / ExoPlayer	Android	The only sanctioned path. ExoPlayer handles the container and adaptive streaming above it
AVFoundation / VideoToolbox	Apple	Clean, well documented, hardware paths that work by default
Media Foundation	Windows	Transform-based; wraps DXVA and vendor encoders
libcamera	Linux	Replaces ad-hoc V4L2 camera code — handles the media graph, 3A and per-sensor tuning
Chromium / WebCodecs	Browsers	WebCodecs exposes raw encode and decode to JavaScript, hardware-backed where available

GStreamer, and the negotiation problem

```
# A pipeline is a graph of elements. The pieces negotiate
# CAPS - format, resolution, framerate, and crucially the
# memory type.

$ gst-launch-1.0 filesrc location=in.mp4 ! qtdemux ! h264parse \
    ! avdec_h264 ! videoconvert ! autovideosink
#   ^ software decode, and a videoconvert that will quietly
#     do an NV12 -> RGB pass on the CPU

# The hardware version, with everything staying in GPU memory:
$ gst-launch-1.0 filesrc location=in.mp4 ! qtdemux ! h264parse \
    ! vaapih264dec ! "video/x-raw(memory:VASurface)" \
    ! vaapipostproc ! vaapisink
#              ^^^^^^^^^^^^^^^^^^^ the caps filter that FORCES
#              the frames to stay in VA surfaces. Without it
#              GStreamer may insert a download to system
#              memory and you lose the whole benefit.

# V4L2 on embedded:
$ gst-launch-1.0 v4l2src device=/dev/video0 \
    ! "video/x-raw,format=NV12,width=1920,height=1080" \
    ! v4l2h264enc ! h264parse ! matroskamux ! filesink location=out.mkv

# THE DIAGNOSTIC THAT MATTERS - see what was actually
# negotiated, element by element:
$ GST_DEBUG=GST_CAPS:5 gst-launch-1.0 ... 2>&1 | grep -i 'caps:'

# Or dump the graph and look at it:
$ GST_DEBUG_DUMP_DOT_DIR=/tmp gst-launch-1.0 ...
$ dot -Tpng /tmp/*PLAYING*.dot -o pipeline.png
# Every element and the caps on every link. If you see
# videoconvert or a "system memory" caps where you expected
# a hardware memory type, you found your problem.

# "not-negotiated" is GStreamer's most common error and it
# always means: no common format between two adjacent
# elements. Read the caps on both sides and look for the
# missing overlap - usually a pixel format or a modifier
# (chapter 4).
```

> **HARDWARE MEMORY TYPES ARE THE WHOLE POINT**
> Every acceleration API has a notion of "a frame that lives in hardware memory" — `memory:VASurface`, `memory:DMABuf`, `memory:GLMemory`, `AVHWFramesContext`. Keeping frames in that domain from decoder to display is what zero-copy *is* (chapter 16). The moment a frame is downloaded to system memory the benefit is gone, and every framework will do that download automatically and silently to make an otherwise-impossible link work.

ffmpeg's hardware model

```
/* Two contexts, and understanding the split is most of it. */

  AVHWDeviceContext   the device: a VA display, a CUDA
                      context, a DRM fd
  AVHWFramesContext   a POOL of frames living on that device,
                      with a fixed format and size

/* Decoding into hardware frames: */

  AVBufferRef *hw_device = NULL;
  av_hwdevice_ctx_create(&hw_device, AV_HWDEVICE_TYPE_VAAPI,
                         "/dev/dri/renderD128", NULL, 0);

  codec_ctx->hw_device_ctx = av_buffer_ref(hw_device);
  codec_ctx->get_format    = get_hw_format;   /* see below */

/* THE CALLBACK IS MANDATORY. libavcodec asks which pixel
   format you want from a list; if you do not pick the
   hardware one, it silently gives you software frames and
   everything still works, slowly. */

static enum AVPixelFormat get_hw_format(AVCodecContext *ctx,
                                        const enum AVPixelFormat *fmts)
{
        for (const enum AVPixelFormat *p = fmts;
             *p != AV_PIX_FMT_NONE; p++)
                if (*p == AV_PIX_FMT_VAAPI)
                        return *p;
        fprintf(stderr, "no hardware format offered\n");
        return AV_PIX_FMT_NONE;      /* fail LOUDLY rather
                                        than fall back */
}

/* After decode, frame->format is AV_PIX_FMT_VAAPI and
   frame->data[3] is the VASurfaceID. The pixels are NOT in
   frame->data[0] - reading them requires an explicit
   transfer, which is exactly the copy you are trying to
   avoid: */

  av_hwframe_transfer_data(sw_frame, hw_frame, 0);   /* SLOW  */

/* Or export as DMABUF and hand it to DRM/KMS or the GPU
   (chapter 18): */
  av_hwframe_map(drm_frame, hw_frame, AV_HWFRAME_MAP_DIRECT);

/* THE CLI equivalents: */
$ ffmpeg -hwaccel vaapi -hwaccel_device /dev/dri/renderD128 \
    -hwaccel_output_format vaapi \
    -i in.mp4 -c:v h264_vaapi -b:v 5M out.mp4
#          ^^^^^^^^^^^^^^^^^^^^^^^^^ WITHOUT this the frames
#          are downloaded to system memory after decode and
#          uploaded again before encode - two full copies per
#          frame, and it still "works".
```

Android and the browser

```
// MediaCodec - Android's only sanctioned codec API.
// The same queue model as V4L2 underneath (chapter 17).

MediaCodec codec = MediaCodec.createDecoderByType("video/avc");
MediaFormat fmt = MediaFormat.createVideoFormat("video/avc", 1920, 1080);
fmt.setByteBuffer("csd-0", spsBuffer);      // SPS
fmt.setByteBuffer("csd-1", ppsBuffer);      // PPS

// THE ZERO-COPY DECISION IS THIS ONE LINE:
codec.configure(fmt, surface, null, 0);
//                   ^^^^^^^
//   a Surface  -> frames go straight to the compositor,
//                 never touch the CPU. Fast path.
//   null       -> frames come back as ByteBuffers you must
//                 read, convert and upload. Slow path.
// Passing null "to inspect the frames" is how a video app
// becomes a battery problem.

codec.start();
for (;;) {
    int in = codec.dequeueInputBuffer(10000);
    if (in >= 0) { /* fill and queueInputBuffer */ }

    MediaCodec.BufferInfo info = new MediaCodec.BufferInfo();
    int out = codec.dequeueOutputBuffer(info, 10000);
    if (out >= 0)
        codec.releaseOutputBuffer(out, true);  // true = render
                                               // it to the
                                               // Surface
}

// Check what the fast path is doing:
//   adb shell dumpsys SurfaceFlinger
//   look at the composition column: DEVICE = a hardware
//   plane, CLIENT = GPU composited (chapter 18)
```

```
// WebCodecs - raw codec access in the browser, hardware-backed
// where the platform allows.

const decoder = new VideoDecoder({
  output: (frame) => {
    // A VideoFrame. It may live in GPU memory - draw it
    // straight to a canvas or a WebGL texture and it never
    // touches JS memory.
    ctx.drawImage(frame, 0, 0);
    frame.close();          // MANDATORY. VideoFrames hold
                            // hardware buffers and the pool is
                            // small; forgetting close() stalls
                            // the decoder within a few frames.
  },
  error: (e) => console.error(e),
});

// Ask before you commit - support varies by browser and
// platform, and hardwareAcceleration is a HINT:
const support = await VideoDecoder.isConfigSupported({
  codec: 'avc1.640028',            // High profile, level 4.0
  hardwareAcceleration: 'prefer-hardware',
});

decoder.configure({ codec: 'avc1.640028', codedWidth: 1920,
                    codedHeight: 1080 });
decoder.decode(new EncodedVideoChunk({
  type: 'key', timestamp: 0, data: chunkBytes }));
```

EVERY FRAMEWORK WILL LIE TO YOU BY SUCCEEDING

Ask for hardware decode into a texture. If the modifier does not match, or the format is unsupported, or the surface is the wrong type, the framework inserts a download, a colour conversion and an upload, and returns success. Frame rate on a desktop may be unaffected. The only reliable check is to measure the video engine's utilisation and the memory bandwidth — `intel_gpu_top`, `nvidia-smi dmon`, `dumpsys SurfaceFlinger`, or `/sys/kernel/debug/dri/0/state` — not whether the picture appears.

How it shows up

Symptom	Cause
CPU at 100% despite requesting hardware decode	`get_format` callback missing, or the fallback was taken
Two full-frame copies per frame in the profile	`-hwaccel_output_format` not set
GStreamer: "not-negotiated"	No common caps between adjacent elements — dump the graph
GStreamer works but is slow	A `videoconvert` or a download inserted during negotiation
Android app drains battery playing video	`configure()` called with `null` instead of a Surface
MediaCodec stalls after a few frames	Output buffers not released
WebCodecs decoder stalls	`frame.close()` not called
Works on one machine, falls back on another	Different driver, format or modifier support
Frames arrive but pixels are empty	Hardware frame read as if it were software — `data[0]` is not the picture
Encoder output is much larger than expected	Hardware encoder, shallow search (chapter 16)

Check yourself

- ✓ Name the four layers between an application and video hardware.
- ✓ Explain why every layer falling back gracefully is an operational problem.
- ✓ Say what a hardware memory type is and name three examples.
- ✓ Explain GStreamer caps negotiation and what "not-negotiated" means.
- ✓ Distinguish `AVHWDeviceContext` from `AVHWFramesContext`.
- ✓ Explain why ffmpeg's `get_format` callback is mandatory and what happens without it.
- ✓ Say what `-hwaccel_output_format` prevents.
- ✓ Identify the one line in a MediaCodec setup that decides the fast path.
- ✓ Explain why `VideoFrame.close()` is mandatory in WebCodecs.
- ✓ Give three ways to verify hardware acceleration is genuinely in use.

Debugging a video pipeline

A video bug has a visible signature, and the signature names the layer. Learning to read "green frame", "diagonal skew", "washed out", "judder" as diagnoses rather than symptoms turns a day of guessing into ten minutes of bisection.

Read the artefact, find the layer

What you see	Layer	Look at
Diagonal skew, each row shifted further	Buffer	Stride confused with width (ch 1)
Purple / orange picture, luma correct	Buffer	U and V swapped — NV12 vs NV21 (ch 4)
Scrambled rectangles	Buffer	Tiled buffer read as linear; modifier mismatch
Washed out or **crushed**	Colour	Limited/full range mismatch (ch 2)
Slight colour shift, faces off	Colour	BT.601 vs BT.709 matrix
Bottom rows garbage	Codec	Crop ignored — 1080 vs 1088 (ch 16)
Green frame then correct	Codec	Displaying before the first keyframe, or an uninitialised DPB
Smearing that clears at a keyframe	Transport	Packet loss; references corrupted (ch 10)
Blocking that pulses	Encoder	Rate control (ch 13)
Judder on pans	Display	Cadence — 24 fps on 60 Hz (ch 18)
Tearing, a horizontal seam	Display	No vblank synchronisation
Lip sync drift	Container	Timestamps (ch 15)
Combing on motion	Source	Interlaced, not deinterlaced

BISECT THE PIPELINE, DO NOT STARE AT THE OUTPUT

A pipeline is capture → encode → mux → transmit → demux → decode → convert → display. The fastest way to localise a bug is to *dump the intermediate*: write the raw frames after capture, the elementary stream after encode, the decoded frames before display. Whichever dump is the first to look wrong contains the bug. Ten minutes of dumping beats a day of reasoning about which stage is guilty.

Dumping at every stage

```
# 1. AFTER CAPTURE - raw frames from the device
$ v4l2-ctl -d /dev/video0 \
    --set-fmt-video=width=1920,height=1080,pixelformat=NV12 \
    --stream-mmap --stream-count=30 --stream-to=cap.nv12
$ ffplay -f rawvideo -pixel_format nv12 -video_size 1920x1080 cap.nv12
# Wrong here -> sensor, ISP, or your format assumption.

# 2. AFTER ENCODE - the elementary stream, no container
$ ffmpeg -i input.mp4 -c:v copy -bsf:v h264_mp4toannexb -f h264 es.264
$ ffplay es.264
# Plays -> the codec data is fine and the problem is the
# container or the transport.

# 3. AFTER DECODE - what the decoder actually produced
$ ffmpeg -i input.mp4 -pix_fmt nv12 -f rawvideo dec.nv12
$ ffplay -f rawvideo -pixel_format nv12 -video_size 1920x1080 dec.nv12
# Correct here but wrong on screen -> conversion or display.

# 4. COMPARE two stages numerically rather than by eye
$ ffmpeg -i original.y4m -i decoded.y4m \
    -lavfi "[0:v][1:v]psnr=stats_file=psnr.log" -f null -
$ awk '$0 ~ /psnr_avg/ {print}' psnr.log | sort -t: -k6 -n | head
# The worst frames first. A single terrible frame among good
# ones is a dropped reference; a gradual decline is drift
# (chapter 11).

# 5. VISUAL DIFF, which finds what metrics miss
$ ffmpeg -i a.mp4 -i b.mp4 \
    -lavfi "[0:v][1:v]blend=all_mode=difference,eq=contrast=8" \
    -c:v libx264 -crf 18 diff.mp4
# Black = identical. Anything visible is a real difference,
# amplified 8x.
```

Inspecting the bitstream itself

```
# When the pixels are wrong and you suspect the stream.

# Every NAL unit, its type and size:
$ ffmpeg -i in.mp4 -c copy -bsf:v trace_headers -f null - 2>&1 | head -60
[trace_headers] Sequence Parameter Set
[trace_headers] 0           forbidden_zero_bit          0
[trace_headers] 1           nal_ref_idc                11 = 3
[trace_headers] 3           nal_unit_type            00111 = 7
[trace_headers] 8           profile_idc           01100100 = 100
[trace_headers] 16          level_idc             00101000 = 40
[trace_headers] ...         pic_width_in_mbs_minus1   = 119
[trace_headers] ...         pic_height_in_map_units_minus1 = 67
#                                                      ^^ 68
#   68 * 16 = 1088, and frame_cropping tells you it is really
#   1080. This is where you check the crop (chapter 16).

# Frame types, sizes and timestamps in one view:
$ ffprobe -select_streams v -show_entries \
    frame=pict_type,key_frame,pkt_size,pkt_pts_time \
    -of csv=p=0 in.mp4 | head -20

# Bitrate over time - finds spikes and starved regions:
$ ffprobe -select_streams v -show_entries frame=pkt_size \
    -of csv=p=0 in.mp4 | awk '{s+=$1; n++}
      n%30==0 {printf "%d %.0f kbit/s\n", n/30, s*8/1000; s=0}'

# Is the file structurally valid at all?
$ ffmpeg -v error -i in.mp4 -f null -
# Silence = clean. Any output here is a real decode error.

# Deeper conformance:
$ ffmpeg -v verbose -err_detect explode -i in.mp4 -f null -
# Turns warnings that are normally tolerated into failures.

# For MP4 box structure:
$ MP4Box -info in.mp4
$ mp4dump in.mp4 | head -40
# Missing moov, wrong track duration, an edit list you did not
# expect (chapter 15) - all visible here.
```

> **"IT PLAYS IN VLC" PROVES ALMOST NOTHING**
> VLC and ffmpeg are extraordinarily tolerant — they resynchronise after corruption, guess missing metadata, ignore level violations and work around broken muxers. A hardware decoder does none of that. If a file plays in VLC and fails on a device, the file is very likely wrong and VLC is compensating. Test with `-err_detect explode`, and on the actual target.

Performance, not correctness

```
# THE FIRST QUESTION: is the hardware path in use at all
# (chapters 16, 19)?

$ sudo intel_gpu_top             # "Video" and "VideoEnhance" rows
$ nvidia-smi dmon -s u           # "dec" and "enc" columns
$ adb shell dumpsys SurfaceFlinger | grep -A5 'Composition'
$ sudo cat /sys/kernel/debug/dri/0/state | grep -c 'format=NV12'
# Zero in any of these = you are on the software path.

# THE SECOND: where is the time going?
$ perf top -p $(pidof myplayer)
# Expect to see, in a healthy software decoder:
#   ff_h264_decode_mb_cabac      entropy   (chapter 8)
#   h264_idct_add8_8_avx         transform
#   ff_h264_filter_mb            deblock   (chapter 12)
#   ff_hscale / yuv2rgb          CONVERSION - if this is high,
#                                you have an unnecessary
#                                format conversion

# THE THIRD: memory bandwidth, which is what actually limits
# embedded pipelines (chapter 16).
$ sudo perf stat -e uncore_imc/data_reads/,uncore_imc/data_writes/ \
    -a sleep 5
# Compare against the theoretical (frame size x fps x copies).
# Much higher than expected = hidden copies.

# LATENCY, end to end, measured rather than estimated:
$ ffmpeg -f lavfi -i "testsrc=size=640x480:rate=60" \
    -vf "drawtext=text='%{pts\\:hms}':fontsize=48:x=10:y=10" \
    -f v4l2 /dev/video1
# Point a camera at the screen showing the timestamp, and
# compare. Crude, and it measures the WHOLE chain including
# the display, which is the number that actually matters.

# GStreamer's built-in latency tracer:
$ GST_DEBUG="GST_TRACER:7" GST_TRACERS="latency" \
    gst-launch-1.0 ... 2>&1 | grep latency
```

The method

1. **Name the artefact.** Use the table above. Half of all video bugs are identified by their appearance alone.
2. **Bisect by dumping.** Raw after capture, elementary stream after encode, raw after decode. The first bad dump contains the bug.
3. **Check the metadata before the pixels.** Format, stride, colour tags, crop, timestamps. These are cheap to inspect and are the cause far more often than the codec.
4. **Reproduce with ffmpeg alone.** If `ffmpeg -i in -f null -` is clean, the file is fine and your pipeline is not.
5. **Test on the target.** Desktop tolerance hides level violations, missing headers and alignment problems that a hardware decoder will not accept.

6 **Separate correctness from performance.** They have different tools and different answers, and confusing them wastes the most time.

How it shows up

Symptom	Cause
Plays on desktop, fails on device	Level, profile, alignment or missing SPS/PPS — desktop tolerance
First frames green or grey	Rendering before the first keyframe
Corruption clears at each keyframe	Packet loss corrupting references
Frame rate fine, power high	Software path or hidden copies — measure, do not watch
PSNR fine, viewers complain	A temporal artefact no frame-wise metric sees (ch 6)
Bug disappears under a debugger	Timing-dependent — a buffer race or a missing fence
Works with 4 buffers, drops with 2	Starving the driver (ch 17)
Encoder output valid but unplayable on a TV	Non-conformant HRD or an exceeded level
Latency far above the sum of the stages	Queueing between stages, or encoder lookahead
Reproducible only at a specific resolution	Alignment — try a multiple of 16 in both dimensions

Check yourself

- ✓ Given diagonal skew, purple output and a washed-out picture, name the layer for each.
- ✓ Explain the bisect-by-dumping method and the three points to dump at.
- ✓ Extract an elementary stream from an MP4 and say what it proves if it plays.
- ✓ Use `trace_headers` to find the coded height and the crop.
- ✓ Say why "it plays in VLC" is weak evidence.
- ✓ Give three ways to verify the hardware path is in use.
- ✓ Read a decoder profile and identify an unnecessary format conversion.
- ✓ Describe a way to measure true end-to-end latency.
- ✓ Explain why a frame-wise metric can look fine while viewers complain.
- ✓ List the six steps of the method in order.

Project: a capture-to-encode pipeline

A complete program: capture NV12 from a V4L2 camera, encode it with a hardware codec, and mux it to a file — with the frames never once copied by the CPU. Everything in this track appears here at once, and the zero-copy path is the part that makes it worth building rather than downloading.

What we are building

```
/dev/video0            /dev/video11          out.mp4
CAMERA                 HARDWARE ENCODER
V4L2 capture           V4L2 M2M
     |                      |
     |  VIDIOC_EXPBUF       |
     |  -> dmabuf fd        |
     +--------------------->|  OUTPUT queue
                            |  V4L2_MEMORY_DMABUF
                            |
                            |  the encoder reads the
                            |  CAMERA'S buffer directly.
                            |  ZERO COPIES.
                            |
                            |  CAPTURE queue
                            |  -> H.264 bitstream
                           v
                      userspace: mux to MP4

THE PROPERTY WE ARE ENFORCING
  the CPU never touches a pixel. It moves file descriptors
  and a few hundred bytes of bitstream per frame.

THE MEASUREMENT THAT PROVES IT
  at 1080p30 NV12, one copy is 93 MB/s. A naive pipeline
  does three or four (chapter 16). We will measure memory
  bandwidth and CPU and show both are near zero.

TARGET: a Raspberry Pi 4/5, a Rockchip or i.MX board, or
any x86 machine with a UVC camera and VA-API. The V4L2 code
is identical; only the device node changes.
```

The shared header

```
/* pipeline.h */

#ifndef PIPELINE_H
#define PIPELINE_H

#include <linux/videodev2.h>
#include <stdint.h>
#include <stdbool.h>

#define MAX_BUFS 8

struct buf {
        void    *start;      /* mmap'd, only for MMAP bufs   */
        size_t   length;
        int      dmabuf_fd;  /* -1 if not exported           */
};

struct stream {
        int      fd;
        enum v4l2_buf_type type;
        enum v4l2_memory   memory;
        struct buf bufs[MAX_BUFS];
        unsigned n;
        uint32_t fourcc;
        int      width, height;
        int      stride;      /* NOT width (chapter 1)        */
};

/* Every ioctl retries on EINTR (chapter 17). */
int xioctl(int fd, unsigned long req, void *arg);

#endif
```

The capture side

```
/* capture.c - open the camera, export its buffers as dmabufs.
   The EXPBUF call is what makes zero-copy possible. */

#include "pipeline.h"
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <stdio.h>

int xioctl(int fd, unsigned long req, void *arg)
{
        int r;
        do { r = ioctl(fd, req, arg); }
        while (r == -1 && errno == EINTR);
        return r;
}

int capture_open(struct stream *s, const char *dev,
                 int w, int h, unsigned nbuf)
{
        s->fd = open(dev, O_RDWR | O_NONBLOCK | O_CLOEXEC);
        if (s->fd < 0) { perror(dev); return -1; }

        s->type   = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        s->memory = V4L2_MEMORY_MMAP;   /* the DRIVER
                                           allocates, and we
                                           export what it gave
                                           us                 */

        struct v4l2_capability cap = {0};
        if (xioctl(s->fd, VIDIOC_QUERYCAP, &cap) < 0) return -1;
        if (!(cap.capabilities & V4L2_CAP_STREAMING)) {
                fprintf(stderr, "no streaming support\n");
                return -1;
        }

        struct v4l2_format fmt = {0};
        fmt.type = s->type;
        fmt.fmt.pix.width       = w;
        fmt.fmt.pix.height      = h;
        fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_NV12;
        fmt.fmt.pix.field       = V4L2_FIELD_NONE;
        if (xioctl(s->fd, VIDIOC_S_FMT, &fmt) < 0) return -1;

        /* READ BACK WHAT YOU GOT. The driver may have chosen
           something else entirely (chapter 17). */
        s->width  = fmt.fmt.pix.width;
        s->height = fmt.fmt.pix.height;
        s->stride = fmt.fmt.pix.bytesperline;
        s->fourcc = fmt.fmt.pix.pixelformat;

        if (s->fourcc != V4L2_PIX_FMT_NV12) {
                fprintf(stderr, "camera gave %.4s, not NV12 - "
                        "a conversion will be needed\n",
                        (char *)&s->fourcc);
                /* Do not silently continue. Either accept the
                   conversion cost deliberately, or pick a
```

```
                    camera mode that gives you NV12. */
        }

        struct v4l2_requestbuffers req = {0};
        req.count  = nbuf;
        req.type   = s->type;
        req.memory = s->memory;
        if (xioctl(s->fd, VIDIOC_REQBUFS, &req) < 0) return -1;
        s->n = req.count;         /* may be fewer than asked  */

        for (unsigned i = 0; i < s->n; i++) {
                struct v4l2_buffer b = {0};
                b.type = s->type; b.memory = s->memory; b.index = i;
                if (xioctl(s->fd, VIDIOC_QUERYBUF, &b) < 0) return -1;

                s->bufs[i].length = b.length;
                s->bufs[i].start  = mmap(NULL, b.length,
                                         PROT_READ | PROT_WRITE,
                                         MAP_SHARED, s->fd, b.m.offset);
                /* We mmap only so we CAN look at pixels while
                   debugging. The fast path never dereferences
                   this pointer. */

                /* THE KEY CALL: hand me a file descriptor
                   representing this buffer, so another driver
                   can import the same memory. */
                struct v4l2_exportbuffer exp = {0};
                exp.type  = s->type;
                exp.index = i;
                exp.flags = O_CLOEXEC | O_RDWR;
                if (xioctl(s->fd, VIDIOC_EXPBUF, &exp) < 0) {
                        perror("EXPBUF - no dmabuf export, so "
                               "no zero-copy on this driver");
                        return -1;
                }
                s->bufs[i].dmabuf_fd = exp.fd;
        }
        return 0;
}

int capture_start(struct stream *s)
{
        for (unsigned i = 0; i < s->n; i++) {
                struct v4l2_buffer b = {0};
                b.type = s->type; b.memory = s->memory; b.index = i;
                if (xioctl(s->fd, VIDIOC_QBUF, &b) < 0) return -1;
        }
        return xioctl(s->fd, VIDIOC_STREAMON, &s->type);
}
```

`VIDIOC_EXPBUF` IS THE WHOLE PROJECT

It turns a driver-allocated buffer into a file descriptor that any other driver can import. Without it you must `mmap` the camera buffer, `memcpy` into an encoder buffer, and pay 93 MB/s at 1080p30 — plus a second copy if the encoder cannot take your memory either. A driver that does not implement `EXPBUF` cannot participate in a zero-copy pipeline at all, and that is worth discovering at design time rather than after the code is written.

The encoder side

```
/* encoder.c - a V4L2 M2M encoder whose OUTPUT queue imports
   the camera's dmabufs. */

int encoder_open(struct stream *out, struct stream *cap_out,
                 const struct stream *cam, int bitrate)
{
        int fd = open("/dev/video11", O_RDWR | O_NONBLOCK | O_CLOEXEC);
        if (fd < 0) { perror("encoder"); return -1; }
        out->fd = cap_out->fd = fd;   /* one device, two queues */

        /* --- OUTPUT queue: raw frames IN.
               "OUTPUT" is from the APPLICATION's point of view
               (chapter 17). --- */
        out->type   = V4L2_BUF_TYPE_VIDEO_OUTPUT_MPLANE;
        out->memory = V4L2_MEMORY_DMABUF;      /* <-- import   */

        struct v4l2_format f = {0};
        f.type = out->type;
        f.fmt.pix_mp.width        = cam->width;
        f.fmt.pix_mp.height       = cam->height;
        f.fmt.pix_mp.pixelformat  = V4L2_PIX_FMT_NV12;
        f.fmt.pix_mp.field        = V4L2_FIELD_NONE;
        f.fmt.pix_mp.num_planes   = 1;   /* NV12 is often ONE
                                            plane with the UV
                                            after the Y - not
                                            two. Check
                                            num_planes after
                                            S_FMT.            */
        f.fmt.pix_mp.plane_fmt[0].bytesperline = cam->stride;
        if (xioctl(fd, VIDIOC_S_FMT, &f) < 0) return -1;

        /* --- CAPTURE queue: compressed bitstream OUT --- */
        cap_out->type   = V4L2_BUF_TYPE_VIDEO_CAPTURE_MPLANE;
        cap_out->memory = V4L2_MEMORY_MMAP;

        struct v4l2_format g = {0};
        g.type = cap_out->type;
        g.fmt.pix_mp.width       = cam->width;
        g.fmt.pix_mp.height      = cam->height;
        g.fmt.pix_mp.pixelformat = V4L2_PIX_FMT_H264;
        g.fmt.pix_mp.num_planes  = 1;
        /* YOU size the bitstream buffer. Too small and frames
           are truncated - a keyframe is 10-20x a P-frame
           (chapter 10), so size for the keyframe. */
        g.fmt.pix_mp.plane_fmt[0].sizeimage = 2 * 1024 * 1024;
        if (xioctl(fd, VIDIOC_S_FMT, &g) < 0) return -1;

        /* --- controls: rate control and GOP (chapters 10, 13) */
        struct v4l2_control c;
        c.id = V4L2_CID_MPEG_VIDEO_BITRATE;        c.value = bitrate;
        xioctl(fd, VIDIOC_S_CTRL, &c);
        c.id = V4L2_CID_MPEG_VIDEO_BITRATE_MODE;
        c.value = V4L2_MPEG_VIDEO_BITRATE_MODE_CBR;
        xioctl(fd, VIDIOC_S_CTRL, &c);
        c.id = V4L2_CID_MPEG_VIDEO_GOP_SIZE;       c.value = 30;
        xioctl(fd, VIDIOC_S_CTRL, &c);
        c.id = V4L2_CID_MPEG_VIDEO_H264_PROFILE;
        c.value = V4L2_MPEG_VIDEO_H264_PROFILE_HIGH;
        xioctl(fd, VIDIOC_S_CTRL, &c);
        /* REPEAT THE HEADERS before every keyframe, or a
```

```
           player tuning in mid-stream has no SPS/PPS
           (chapter 10). */
        c.id = V4L2_CID_MPEG_VIDEO_REPEAT_SEQ_HEADER; c.value = 1;
        xioctl(fd, VIDIOC_S_CTRL, &c);
        /* Not every driver implements every control. Check the
           return, and do not assume silence means success. */

        /* --- buffers ---
           OUTPUT: DMABUF, so REQBUFS allocates only the
           bookkeeping, not memory. */
        struct v4l2_requestbuffers r = {0};
        r.count = cam->n; r.type = out->type; r.memory = out->memory;
        if (xioctl(fd, VIDIOC_REQBUFS, &r) < 0) return -1;
        out->n = r.count;

        /* CAPTURE: MMAP, and we DO read these - they are the
           bitstream, a few hundred KB per second, not pixels. */
        struct v4l2_requestbuffers r2 = {0};
        r2.count = 4; r2.type = cap_out->type; r2.memory = cap_out->memory;
        if (xioctl(fd, VIDIOC_REQBUFS, &r2) < 0) return -1;
        cap_out->n = r2.count;

        for (unsigned i = 0; i < cap_out->n; i++) {
                struct v4l2_plane p = {0};
                struct v4l2_buffer b = {0};
                b.type = cap_out->type; b.memory = cap_out->memory;
                b.index = i; b.m.planes = &p; b.length = 1;
                if (xioctl(fd, VIDIOC_QUERYBUF, &b) < 0) return -1;
                cap_out->bufs[i].length = p.length;
                cap_out->bufs[i].start  = mmap(NULL, p.length,
                        PROT_READ | PROT_WRITE, MAP_SHARED,
                        fd, p.m.mem_offset);
                if (xioctl(fd, VIDIOC_QBUF, &b) < 0) return -1;
        }

        if (xioctl(fd, VIDIOC_STREAMON, &out->type) < 0) return -1;
        if (xioctl(fd, VIDIOC_STREAMON, &cap_out->type) < 0) return -1;
        return 0;
}
```

The loop

```
/* main.c - three file descriptors, one poll, no pixel copies. */

int run(struct stream *cam, struct stream *enc_in,
        struct stream *enc_out, FILE *sink)
{
        /* Map camera buffer index -> encoder buffer index.
           They are separate index spaces; the DMABUF fd is
           what actually links them. */
        int cam_of_enc[MAX_BUFS];
        for (unsigned i = 0; i < MAX_BUFS; i++) cam_of_enc[i] = -1;

        unsigned free_enc = enc_in->n;   /* encoder input slots */
        uint64_t frames = 0;

        for (;;) {
                struct pollfd p[2] = {
                        { .fd = cam->fd,     .events = POLLIN },
                        { .fd = enc_out->fd, .events = POLLIN },
                };
                if (poll(p, 2, 1000) <= 0) continue;

                /* ---- a camera frame is ready ---- */
                if ((p[0].revents & POLLIN) && free_enc > 0) {
                        struct v4l2_buffer cb = {0};
                        cb.type = cam->type; cb.memory = cam->memory;
                        if (xioctl(cam->fd, VIDIOC_DQBUF, &cb) < 0)
                                continue;

                        /* Hand the SAME MEMORY to the encoder,
                           by file descriptor. No copy. */
                        unsigned slot = cb.index;   /* 1:1 here */
                        struct v4l2_plane pl = {0};
                        struct v4l2_buffer eb = {0};
                        eb.type      = enc_in->type;
                        eb.memory    = V4L2_MEMORY_DMABUF;
                        eb.index     = slot;
                        eb.m.planes  = &pl;
                        eb.length    = 1;
                        pl.m.fd      = cam->bufs[cb.index].dmabuf_fd;
                        pl.bytesused = cb.bytesused;
                        pl.length    = cam->bufs[cb.index].length;

                        /* CARRY THE CAPTURE TIMESTAMP THROUGH.
                           Do not stamp frames when your code
                           happens to run (chapter 15). */
                        eb.timestamp = cb.timestamp;
                        eb.flags     = V4L2_BUF_FLAG_TIMESTAMP_COPY;

                        if (xioctl(enc_in->fd, VIDIOC_QBUF, &eb) < 0) {
                                /* give the camera buffer back
                                   even on failure, or we leak
                                   it and the camera starves */
                                xioctl(cam->fd, VIDIOC_QBUF, &cb);
                                continue;
                        }
                        cam_of_enc[slot] = cb.index;
                        free_enc--;
                }

                /* ---- an encoded frame is ready ---- */
```

```
                if (p[1].revents & POLLIN) {
                        struct v4l2_plane pl = {0};
                        struct v4l2_buffer ob = {0};
                        ob.type     = enc_out->type;
                        ob.memory   = enc_out->memory;
                        ob.m.planes = &pl;
                        ob.length   = 1;
                        if (xioctl(enc_out->fd, VIDIOC_DQBUF, &ob) == 0) {
                                fwrite(enc_out->bufs[ob.index].start,
                                       1, pl.bytesused, sink);
                                /* pl.bytesused only - the buffer
                                   is 2 MB and the frame is not */
                                xioctl(enc_out->fd, VIDIOC_QBUF, &ob);
                                frames++;
                        }

                        /* Reclaim the OUTPUT buffer the encoder
                           has finished reading, and return the
                           camera buffer it was pointing at. */
                        struct v4l2_plane ipl = {0};
                        struct v4l2_buffer ib = {0};
                        ib.type     = enc_in->type;
                        ib.memory   = V4L2_MEMORY_DMABUF;
                        ib.m.planes = &ipl;
                        ib.length   = 1;
                        if (xioctl(enc_in->fd, VIDIOC_DQBUF, &ib) == 0) {
                                int ci = cam_of_enc[ib.index];
                                if (ci >= 0) {
                                        struct v4l2_buffer cb = {0};
                                        cb.type   = cam->type;
                                        cb.memory = cam->memory;
                                        cb.index  = ci;
                                        xioctl(cam->fd, VIDIOC_QBUF, &cb);
                                        cam_of_enc[ib.index] = -1;
                                }
                                free_enc++;
                        }
                }
        }
}

/* THE BUFFER DISCIPLINE, which is the part that goes wrong:

   A camera buffer is IN USE from the moment the encoder
   accepts it until the encoder returns the OUTPUT buffer.
   Requeue it to the camera before that and the encoder reads
   a frame the sensor is overwriting - producing tearing that
   looks like a decoder bug.

   And if you never requeue, the camera runs out of buffers
   and starts dropping frames silently. Watch
   v4l2_buffer.sequence for gaps (chapter 17).            */
```

THE ELEMENTARY STREAM IS NOT A FILE ANYONE CAN PLAY

`out.h264` is raw Annex B — no container, no timestamps, no duration. It plays in ffmpeg because ffmpeg guesses a frame rate, and it will not play in most players or seek at all. Mux it. The `-r` below is a claim about frame rate, not a measurement; for variable-rate capture you must carry the real timestamps through and mux from those (chapter 15).

Building and running

```
$ gcc -O2 -Wall -o pipeline main.c capture.c encoder.c
$ ./pipeline /dev/video0 /dev/video11 out.h264

# Find the encoder node - it is not always video11:
$ for d in /dev/video*; do
    echo -n "$d: "
    v4l2-ctl -d $d --info | grep -i 'card type' | cut -d: -f2
  done
/dev/video0:  Integrated Camera
/dev/video11: bcm2835-codec-encode      <-- this one

$ v4l2-ctl -d /dev/video11 --list-formats-ext --all | head -30
# Confirm it accepts NV12 in and H264 out, and check the
# DMABUF capability:
$ v4l2-ctl -d /dev/video11 --info | grep -i 'streaming\|m2m'

# Mux the elementary stream:
$ ffmpeg -r 30 -i out.h264 -c:v copy -movflags +faststart out.mp4

# VERIFY IT IS ACTUALLY CORRECT (chapter 20):
$ ffmpeg -v error -i out.mp4 -f null -            # silence = clean
$ ffprobe -select_streams v -show_entries \
    frame=pict_type,pkt_size -of csv=p=0 out.mp4 | head
I,84213
P,3241
P,2988
# I-frame 25x a P-frame. Prediction is working (chapter 9).

# PROVE THE ZERO COPY - this is the point of the exercise.
$ pidstat -p $(pidof pipeline) 1 5
# CPU should be 1-3%. If it is 40%, something is copying.

$ sudo perf stat -e uncore_imc/data_reads/,uncore_imc/data_writes/ \
    -a -- sleep 10
# At 1080p30 NV12 the FLOOR is:
#   camera writes    93 MB/s
#   encoder reads    93 MB/s
#   encoder writes   ~0.5 MB/s (the bitstream)
# ~190 MB/s total. If you measure 400+, there is a hidden
# copy (chapter 16).

$ sudo perf top -p $(pidof pipeline)
# You should see poll and ioctl. If memcpy or a yuv2rgb
# routine appears, the fast path is not being taken.
```

Extensions worth doing

Exercise	What it teaches
Mux to fragmented MP4 in-process rather than writing Annex B	Timestamps, timebases and `+faststart` (ch 15)
Add a preview: import the same dmabuf into a DRM plane	The full zero-copy display path (ch 18)
Carry the camera's real timestamps into the container	Why VFR breaks naive muxers
Handle a resolution change mid-stream	`SOURCE_CHANGE`, and the reallocation dance (ch 17)

Exercise	What it teaches
Implement the drain on Ctrl-C	`ENC_CMD_STOP` and `BUF_FLAG_LAST` — without it you lose the last frames
Force a keyframe on demand	`V4L2_CID_MPEG_VIDEO_FORCE_KEY_FRAME`, and why a streaming server needs it
Compare hardware output with `x264 --preset slow` at equal VMAF	The 20–50% penalty, measured on your own hardware (ch 16)
Deliberately requeue the camera buffer too early	Reproduce the tearing, so you recognise it later
Set the buffer count to 2 and watch `sequence`	Frame drops, and why buffer count is a latency dial

How it shows up

Symptom	Cause
`EXPBUF` returns `ENOTTY`	Driver has no dmabuf export — zero-copy is impossible; you must copy
`QBUF` on the encoder returns `EINVAL`	Single-planar structs against a multiplanar device, or a stride mismatch
Tearing, half of two different frames	Camera buffer requeued before the encoder finished with it
Frames dropped, `sequence` has gaps	Too few buffers, or buffers held too long
Output file is truncated garbage	`sizeimage` too small for a keyframe
Player will not open `out.h264`	Elementary stream — mux it
Stream will not play from the middle	`REPEAT_SEQ_HEADER` not set
CPU at 40% instead of 2%	A copy somewhere — check `perf top` for `memcpy`
Camera gives YUYV, not NV12	The mode does not exist; a conversion is unavoidable — choose it deliberately
Last second of video missing	No drain at exit
Works at 1920×1080, fails at 1280×720	Stride assumption; read `bytesperline` back

Check yourself

- ✓ Draw the pipeline and say where a copy would occur without `EXPBUF`.
- ✓ Explain what `VIDIOC_EXPBUF` produces and why it enables zero-copy.
- ✓ Say why the encoder's OUTPUT queue uses `V4L2_MEMORY_DMABUF` while its CAPTURE queue uses `MMAP`.
- ✓ Explain why the camera buffer must not be requeued until the encoder returns its OUTPUT buffer.
- ✓ Say how to detect dropped frames, and what causes them.
- ✓ Explain why `sizeimage` must be sized for a keyframe.
- ✓ Say why `REPEAT_SEQ_HEADER` matters for a stream someone joins late.
- ✓ Compute the memory bandwidth floor for 1080p30 NV12 and say what a higher figure means.
- ✓ Give two ways to prove the CPU is not touching pixels.
- ✓ Explain why the raw `.h264` is not a playable file.

TRACK

AI & large language models

The mathematics first, then machine learning, then transformers and LLMs — ending with running a model on embedded hardware.

28 chapters

What this track assumes and where it goes

Twenty-eight chapters from "what is a vector" to a working transformer you write yourself. No prior machine learning, and no mathematics beyond school algebra — everything else is built here.

Why an embedded engineer should read this

```
THREE HONEST REASONS:

1. THE WORK IS ARRIVING IN YOUR DOMAIN.
   Keyword spotting on a Cortex-M4. Anomaly detection on
   a motor controller. Vision on a $5 SoC. NPUs in
   application processors you already ship. Someone has
   to write the driver for the accelerator, decide
   whether the model fits in SRAM, and explain why the
   quantised version is 3% less accurate.

2. THE INTUITIONS TRANSFER BOTH WAYS.
   A transformer's forward pass is a fixed sequence of
   matrix multiplications over a known-size working set.
   That is a MEMORY BANDWIDTH problem and a CACHE
   BLOCKING problem - things you already reason about.
   Conversely, understanding numerical precision, fixed
   point and saturation is exactly what quantisation
   needs, and embedded engineers already have it.

3. AND THE PLAIN ONE: these systems are now part of the
   tooling you use every day. Knowing what they are -
   really, mechanically - is the difference between using
   them well and guessing.

WHAT THIS TRACK IS NOT:
  · not a research course. No proofs of convergence, no
    measure theory.
  · not a framework tutorial. You will see PyTorch, but
    the point is what it computes, not its API.
  · not a survey. One path, taken properly, ending at a
    working transformer.
```

What is assumed

```
/* YOU NEED:
     · comfort with C (the C track), or any language with
       loops, arrays and functions
     · school algebra: symbols, functions, exponents
     · the willingness to write out a small example by hand
       when the notation stops making sense

/* YOU DO NOT NEED:
     · university mathematics. Linear algebra, calculus and
       probability are each built from zero here, in the
       amount this subject actually uses - which is much
       less than a full course.
     · Python experience. The code is readable if you know
       C; the small amount of syntax you need is introduced
       where it appears.
     · a GPU. Everything in this track runs on a laptop CPU.
       The final project runs on a Raspberry Pi.

/* THE TOOLS, if you want to run the code:
     python3, numpy, and eventually torch (CPU build)

     $ python3 -m venv venv
     $ source venv/bin/activate     # Windows:
                                    # venv\Scripts\activate
     $ pip install numpy matplotlib
     $ pip install torch --index-url \
           https://download.pytorch.org/whl/cpu

   THE NUMPY-ONLY CHAPTERS COME FIRST DELIBERATELY. Writing
   a matrix multiply as three nested loops before ever
   calling a library is the difference between knowing what
   a framework does and trusting that it does something.  */
```

The shape of the track

```
PART 1 — THE MATHEMATICS (chapters 02–11)
  Exactly what is needed, and no more.

  02 vectors            what a vector IS and what
                        operations mean
  03 matrices           as transformations, not tables
  04 matmul             THE core operation — and where
                        all the compute goes
  05 eigen/SVD          why some directions matter more
  06 derivatives        rate of change; gradients
  07 chain rule         -> BACKPROPAGATION. The chapter
                        everything trains on.
  08 probability        distributions, expectation
  09 Bayes              likelihood, estimation
  10 information theory entropy -> CROSS-ENTROPY, which
                        is the loss function you will
                        use for every model here
  11 optimisation       gradient descent, momentum, Adam

PART 2 — MACHINE LEARNING (12–15)
  12 what ML is         and when it is the wrong tool
  13 regression         the simplest models, done fully
  14 neural networks    the MLP; why non-linearity
                        matters
  15 training           loss, initialisation,
                        regularisation, what goes wrong

PART 3 — LANGUAGE MODELS (16–25)
  16 tokenisation       text -> integers. Where many
                        weird model behaviours originate.
  17 embeddings         integers -> vectors with meaning
  18 sequence models    RNNs and LSTMs, and why they were
                        replaced
  19 attention          THE key idea
  20 the transformer    the full architecture
  21 multi-head + positional encoding
  22 norm, residuals, FFN
  23 pretraining        and scaling laws
  24 fine-tuning        instruction tuning, RLHF, DPO
  25 inference          sampling, the KV cache, context
                        length

PART 4 — MAKING IT RUN (26–28)
  26 quantisation       int8, int4, and what precision
                        costs
  27 edge deployment    NPUs, TFLite Micro, ONNX, memory
                        budgets — THE EMBEDDED CHAPTER
  28 PROJECT            a character-level transformer,
                        written and trained from scratch,
                        in numpy and then in PyTorch

THE DEPENDENCY THAT MATTERS: chapters 04, 07 and 10 are
load-bearing. Matrix multiplication, the chain rule and
cross-entropy appear in every chapter after them. If a
later chapter stops making sense, the gap is almost
always in one of those three.
```

Notation

```
/* Introduced once, used throughout. Refer back here.     */

   SCALARS        lowercase italic:  x, y, alpha
   VECTORS        lowercase bold:    x, w      (column
                                               vectors by
                                               default)
   MATRICES       uppercase bold:    W, X, A
   TENSORS        the general case: an n-dimensional array.
                  A scalar is rank 0, a vector rank 1, a
                  matrix rank 2. "Tensor" in this field just
                  means "n-dimensional array" - the physics
                  meaning is not implied.

   x_i            the i-th element of vector x
   W_ij           row i, column j of W
   x^T            transpose
   ||x||          the norm (length) of x
   x · y          dot product
   A @ B          matrix multiplication (numpy/PyTorch
                  notation, used in code)
   A * B          ELEMENTWISE product (in code) - NOT matrix
                  multiplication. A frequent source of silent
                  bugs.

   SHAPES ARE WRITTEN LIKE THIS:  (batch, seq_len, d_model)
   and you should read every line of model code by tracking
   shapes. MOST BUGS IN THIS FIELD ARE SHAPE BUGS. The
   habit to build now: after every operation, ask "what
   shape is this, and why?"

   f(x)           a function
   df/dx          derivative
   ∇f             gradient - the vector of all partial
                  derivatives
   E[X]           expectation
   P(x)           probability

/* AND THE ONE CONVENTION THAT CONFUSES EVERYONE:

   In mathematics, a linear layer is usually written
        y = W x + b        with x a COLUMN vector
   In code, data comes in ROWS, so it is written
        Y = X @ W.T + b    or often just  X @ W + b
   with W stored already transposed.

   THESE ARE THE SAME OPERATION. When a paper and an
   implementation seem to disagree about which index is
   which, this is usually why.                          */
```

HOW TO READ THIS TRACK

Do the arithmetic by hand at least once per chapter. A 2×2 matrix multiply, one step of gradient descent, one softmax over three numbers. The notation is compressed and it hides the fact that everything here is arithmetic you could do on paper — until you have done it on paper, the compression reads as magic. Every chapter ends with a "check yourself" list written to be answerable without looking.

An honest framing of the subject

```
/* WHAT THESE SYSTEMS ARE:
     Function approximators with an enormous number of
     parameters, fitted to data by gradient descent. A large
     language model is a function from a sequence of tokens
     to a probability distribution over the next token,
     with a few hundred billion adjustable numbers, fitted
     to a large fraction of the written internet.

     THAT DESCRIPTION IS COMPLETE AND MECHANICAL. Everything
     in chapters 16–25 is an elaboration of it.

/* WHAT IS GENUINELY SURPRISING:
     That scaling this simple recipe — more parameters, more
     data, more compute — produces capabilities nobody
     explicitly designed: translation, arithmetic, code,
     in-context learning from examples in the prompt.
     Chapter 23 covers the scaling laws that describe this
     empirically. NOBODY HAS A COMPLETE THEORY OF WHY IT
     WORKS AS WELL AS IT DOES. That is the honest state of
     the field, and any source that tells you otherwise is
     selling something.

/* WHAT IS OFTEN OVERSTATED:
     · these are not databases; they do not retrieve facts
       reliably
     · they have no persistent memory between calls beyond
       what is in the context window (chapter 25)
     · confident output and correct output are not
       correlated as strongly as the fluency suggests
     · "reasoning" is a description of behaviour, not a
       claim about mechanism

/* WHAT THIS MEANS FOR AN ENGINEER:
     Treat model output as UNTRUSTED INPUT. Validate it,
     constrain it, and design the system so a wrong answer
     is recoverable. That is the same discipline you apply
     to a sensor reading or a network packet, and it is the
     right one here.                                    */
```

Check yourself

- ✓ State three reasons this material is relevant to embedded work.
- ✓ List what the track assumes and what it deliberately does not.
- ✓ Name the four parts and the three load-bearing chapters.
- ✓ Explain the difference between `A @ B` and `A * B`.
- ✓ Explain why `y = Wx + b` and `Y = X @ W + b` are the same operation.
- ✓ Give a complete, mechanical, one-sentence definition of a large language model.
- ✓ Say what is genuinely surprising about scaling, and what is commonly overstated.

Linear algebra: vectors and vector spaces

A vector is an array of numbers, and also a point in space, and also a direction with a length. Holding all three pictures at once is most of what "having intuition for linear algebra" means.

Three views of the same object

```
x = [3, 4]

AS AN ARRAY          two numbers in order. This is what
                     the computer stores:
                         float x[2] = {3.0f, 4.0f};

AS A POINT           the location (3, 4) in the plane

AS AN ARROW          from the origin to (3, 4), with a
                     DIRECTION and a LENGTH of 5

      y
      |
    4 +--------* (3,4)
      |       /|
      |      / |
      |     /  |    length = sqrt(3^2 + 4^2) = 5
      |    /   |
      |   /    |
      |  /     |
      | /      |
      +--------+------ x
      0        3

IN MACHINE LEARNING, THE THIRD VIEW IS THE USEFUL ONE:
a vector represents something, its DIRECTION carries the
meaning, and its LENGTH is often incidental. Two word
embeddings pointing the same way mean similar things,
whether one has length 3 and the other 30 (chapter 17).

AND THE DIMENSION IS NOT TWO. A word embedding lives in
768 or 4096 dimensions. YOU CANNOT PICTURE THAT, and you
do not need to: every operation below is defined
element by element and works identically at any
dimension. The 2-D picture is for intuition; the
arithmetic is what runs.
```

The operations

```
import numpy as np

a = np.array([3.0, 4.0])
b = np.array([1.0, 2.0])

# --- ADDITION: elementwise. Geometrically, tip to tail. ---
a + b            # [4., 6.]

# --- SCALAR MULTIPLICATION: scales the length,
#     keeps the direction (or reverses it if negative) ---
2 * a            # [6., 8.]
-1 * a           # [-3., -4.]

# --- IN C, so the loop is visible: ---
#   void vec_add(float *out, const float *x,
#                const float *y, int n) {
#       for (int i = 0; i < n; i++)
#           out[i] = x[i] + y[i];
#   }
#   That is the whole operation, at any dimension.

# --- NORM: the length ---
np.linalg.norm(a)                # 5.0
np.sqrt(np.sum(a**2))            # the same thing

#   L2 norm (Euclidean):  sqrt(sum(x_i^2))     <- the default
#   L1 norm (Manhattan):  sum(|x_i|)
#   L-inf norm:           max(|x_i|)
#   These appear again as REGULARISERS in chapter 15: L2
#   shrinks all weights, L1 drives some to exactly zero.

# --- NORMALISATION: length 1, direction preserved ---
a_hat = a / np.linalg.norm(a)      # [0.6, 0.8]
np.linalg.norm(a_hat)              # 1.0

#   A UNIT VECTOR IS PURE DIRECTION. This is why embeddings
#   are often normalised before comparison - you want to ask
#   "same direction?" without "same magnitude?" muddying it.

# --- DOT PRODUCT: the operation everything is built on ---
np.dot(a, b)                       # 3*1 + 4*2 = 11
a @ b                              # the same

#   int dot(const float *x, const float *y, int n) {
#       float s = 0;
#       for (int i = 0; i < n; i++) s += x[i] * y[i];
#       return s;
#   }

#   MULTIPLY-ACCUMULATE. One line. It is the single most
#   executed operation in all of machine learning, which is
#   why every DSP and NPU has a MAC unit and why chapter 04
#   is about doing many of these at once.
```

What the dot product means

```
   THE GEOMETRIC IDENTITY, and it is worth memorising:

        a · b = ||a|| ||b|| cos(theta)

   where theta is the angle between them.

   SO THE SIGN AND SIZE OF A DOT PRODUCT TELL YOU ABOUT
   ALIGNMENT:

     a · b  LARGE POSITIVE   ->  pointing the same way
     a · b  ZERO             ->  PERPENDICULAR (orthogonal)
     a · b  LARGE NEGATIVE   ->  pointing opposite ways

        b               b            b
        ^               ^             \
        |               |              \
        +---> a         +---> a    a <--+
     theta = 45         theta = 90   theta = 135
     dot > 0            dot = 0      dot < 0

   COSINE SIMILARITY removes the magnitudes:

        cos(theta) = (a · b) / (||a|| ||b||)

        +1  identical direction
         0  unrelated
        -1  opposite

   THIS IS THE STANDARD MEASURE OF SEMANTIC SIMILARITY
   BETWEEN EMBEDDINGS. When a vector database "finds similar
   documents", it is computing this, or its unnormalised
   cousin, against every stored vector.

   AND IT IS ALSO WHAT ATTENTION COMPUTES (chapter 19): a
   query vector is dotted with every key vector, and the
   result says how much each position matters. The whole
   mechanism is "dot product as a relevance score".
```

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) *
                           np.linalg.norm(b))

king  = np.array([0.9, 0.7, 0.1])
queen = np.array([0.8, 0.8, 0.1])
truck = np.array([0.1, 0.2, 0.9])

cosine_similarity(king, queen)   # 0.99  - very similar
cosine_similarity(king, truck)   # 0.36  - not similar

# IF THE VECTORS ARE ALREADY NORMALISED, cosine similarity
# IS JUST THE DOT PRODUCT - which is why production
# retrieval systems normalise once at insert time and then
# do nothing but MACs at query time. A useful optimisation
# to recognise when reading such code.
```

Vector spaces, bases and spans

```
/* A VECTOR SPACE is a set of vectors closed under addition
   and scalar multiplication. For our purposes: R^n, the set
   of all n-element real vectors. Nothing exotic.

/* LINEAR COMBINATION - the central idea: */

        c1*v1 + c2*v2 + ... + ck*vk

   Everything in linear algebra is a statement about which
   vectors can be written as linear combinations of which
   others.

/* SPAN: all the vectors you can reach. */
    span{[1,0]}           = the x-axis (a LINE)
    span{[1,0], [0,1]}    = the whole plane
    span{[1,0], [2,0]}    = still just the x-axis - the
                            second vector adds NOTHING

/* LINEAR INDEPENDENCE: no vector in the set is a
   combination of the others. [1,0] and [2,0] are DEPENDENT.
   [1,0] and [0,1] are INDEPENDENT.

/* BASIS: an independent set that spans the space. A basis
   for R^n always has exactly n vectors.
   THE STANDARD BASIS for R^3:
        e1 = [1,0,0], e2 = [0,1,0], e3 = [0,0,1]
   and [3, 4, 5] = 3*e1 + 4*e2 + 5*e3. The COORDINATES ARE
   THE COEFFICIENTS - that is all a coordinate is.

/* DIMENSION: the number of vectors in a basis.

/* WHY THIS MATTERS FOR MACHINE LEARNING - and it does,
   directly:

   AN EMBEDDING SPACE IS A VECTOR SPACE, and the interesting
   claim is that MEANING IS ENCODED IN DIRECTIONS. The famous
   example:

        vec("king") - vec("man") + vec("woman")
             lands near vec("queen")

   Read that as: there is a DIRECTION in the space that
   corresponds roughly to "gender", and another to "royalty",
   and they combine linearly. THE LINEAR ALGEBRA IS DOING
   SEMANTIC WORK.

   (The real behaviour is messier than the popular version
   suggests - the analogy only works with certain
   normalisations and often the nearest neighbour is one of
   the input words. But the underlying claim, that
   directions carry meaning, holds up and is central to
   chapter 17.)

   THE MODERN VERSION OF THIS IDEA IS THE SUPERPOSITION
   HYPOTHESIS: a model with d dimensions represents far more
   than d features, by placing them in ALMOST-orthogonal
   directions. In high dimensions there are exponentially
   many nearly-perpendicular directions, so this is
```

```
   possible - and it is one reason big models do
   disproportionately better than small ones.           */
```

HIGH DIMENSIONS ARE STRANGE

In 768 dimensions, two random unit vectors are almost always nearly orthogonal — the expected cosine similarity is about zero with a standard deviation near $1/\sqrt{768} \approx 0.036$. So *any* measured similarity above ~0.1 is meaningful, and near-orthogonality is the default rather than a coincidence. Most of the volume of a high-dimensional ball is near its surface, and most of a Gaussian's mass is in a thin shell. Do not trust 2-D intuition about distances in these spaces; trust the arithmetic.

Vectors in practice

```
import numpy as np

# SHAPES. Track them obsessively (chapter 01).
v = np.array([1.0, 2.0, 3.0])
v.shape           # (3,)      a 1-D array - NOT a column
                  #           or a row, just 1-D

col = v.reshape(3, 1)     # (3, 1)  a column
row = v.reshape(1, 3)     # (1, 3)  a row

# NUMPY BROADCASTING - the rule that makes batched code
# short, and the source of many silent bugs:
#   compare shapes from the RIGHT; dimensions must be equal
#   or one of them must be 1; size-1 dimensions are
#   stretched.

batch = np.random.randn(32, 768)     # 32 vectors of dim 768
bias  = np.random.randn(768)         # (768,)
out   = batch + bias                 # (32, 768) - bias
                                     # added to every row

# THE CLASSIC BUG:
a = np.random.randn(32, 1)
b = np.random.randn(1, 32)
c = a + b                            # (32, 32) !!
# You wanted elementwise addition of 32 numbers and got a
# 32x32 outer sum. It runs, it produces numbers, and the
# model trains badly. ASSERT YOUR SHAPES:
assert c.shape == (32, 1), c.shape

# BATCHED OPERATIONS - one vector per row, always:
X = np.random.randn(32, 768)         # a BATCH of 32

norms = np.linalg.norm(X, axis=1, keepdims=True)  # (32, 1)
X_normalised = X / norms                          # (32, 768)

# all pairwise dot products between two batches:
A = np.random.randn(32, 768)
B = np.random.randn(64, 768)
sims = A @ B.T                        # (32, 64)
# THIS IS ATTENTION'S CORE COMPUTATION IN ONE LINE
# (chapter 19), and it is why chapter 04 exists.

# --- AND THE PRECISION NOTE, for the embedded reader ---
# float32 gives ~7 decimal digits; float16 gives ~3.
# Summing 4096 products in float16 accumulates real error,
# so hardware MAC units accumulate in HIGHER PRECISION than
# they multiply in - int8 x int8 -> int32, fp16 x fp16 ->
# fp32. When you meet quantisation in chapter 26, this is
# the reason the accumulator is always wider.
```

How it shows up

Symptom	Cause
A result is (32, 32) where (32, 1) was expected	Broadcasting a column against a row
Cosine similarity above 1.0	Floating-point error, or the norms were computed on the wrong axis

Symptom	Cause
All similarities near zero	Normal in high dimensions — compare against 1/√d, not against 0.5
Division by zero in normalisation	A zero vector — add a small epsilon
Similarity dominated by one long vector	Dot product used where cosine similarity was meant
`ValueError: shapes not aligned`	Missing transpose — `A @ B` instead of `A @ B.T`
Silently wrong results with no error	Broadcasting hid a shape mistake — assert shapes
Accuracy collapses in float16	Accumulation error over long dot products — widen the accumulator

Check yourself

- ✓ Give the three views of a vector and say which one matters most for embeddings.
- ✓ Compute the norm of [3, 4] and normalise it, by hand.
- ✓ Write the dot product as a C loop.
- ✓ State the geometric identity for the dot product and interpret all three sign cases.
- ✓ Define cosine similarity and say why retrieval systems normalise at insert time.
- ✓ Define span, linear independence, basis and dimension.
- ✓ Explain what "meaning is encoded in directions" claims, and one caveat.
- ✓ Explain why two random vectors in 768 dimensions are nearly orthogonal.
- ✓ State the broadcasting rule and give the classic bug it causes.
- ✓ Say why MAC hardware accumulates in higher precision than it multiplies.

Linear algebra: matrices as transformations

A matrix is a grid of numbers. It is more usefully a *function* that takes vectors to vectors. Once you see a matrix as a transformation, every neural network layer becomes readable.

A matrix is a function

```
THE DEFINITION THAT MAKES EVERYTHING CLICK:

  THE COLUMNS OF A MATRIX ARE WHERE THE BASIS VECTORS GO.

Take W = [ 2  0 ]     as columns:  [2]  and  [0]
         [ 0  3 ]                  [0]       [3]

e1 = [1,0] is sent to [2,0]   (the first column)
e2 = [0,1] is sent to [0,3]   (the second column)

And because the transformation is LINEAR, everything else
follows:
    W [3,4] = 3*(where e1 goes) + 4*(where e2 goes)
            = 3*[2,0] + 4*[0,3]
            = [6, 12]

THAT IS THE WHOLE OF MATRIX-VECTOR MULTIPLICATION. You
never have to remember the index formula: just read the
columns as destinations.

SOME TRANSFORMATIONS IN 2-D:

  SCALE          [2 0]      stretch x by 2, y by 3
                 [0 3]

  ROTATION       [cos -sin]  rotate by theta
                 [sin  cos]

  SHEAR          [1 1]      slide x by the amount of y
                 [0 1]

  PROJECTION     [1 0]      flatten onto the x-axis;
                 [0 0]      INFORMATION IS LOST

  IDENTITY       [1 0]      change nothing
                 [0 1]

WHAT "LINEAR" MEANS, precisely:
    W(a + b) = Wa + Wb
    W(c * a) = c * (Wa)
Grid lines stay straight, parallel and evenly spaced, and
the origin does not move.

THAT LAST CLAUSE IS WHY NEURAL NETWORKS NEED A BIAS TERM
AND NON-LINEARITIES. A linear map alone cannot move the
origin, cannot bend anything, and a stack of them
collapses into a single matrix (chapter 14).
```

Shapes, and what they tell you

```
import numpy as np

W = np.array([[1., 2., 3.],
              [4., 5., 6.]])
W.shape        # (2, 3)  -> 2 ROWS, 3 COLUMNS

# A (m, n) MATRIX MAPS R^n -> R^m.
#   n = the INPUT dimension  (columns)
#   m = the OUTPUT dimension (rows)
# Read it right to left: it eats an n-vector, emits an
# m-vector.

x = np.array([1., 1., 1.])     # (3,)
W @ x                          # (2,)  -> [6., 15.]

# TRANSPOSE: flip rows and columns
W.T.shape                      # (3, 2)

# --- THE INDEX FORMULA, for completeness ---
#   (W x)_i = sum_j W_ij * x_j
#   Row i of W, dotted with x. So a matrix-vector product is
#   m DOT PRODUCTS (chapter 02).

# --- IN CODE, so it is concrete: ---
# void matvec(float *y, const float *W, const float *x,
#             int m, int n) {
#     for (int i = 0; i < m; i++) {
#         float s = 0.0f;
#         for (int j = 0; j < n; j++)
#             s += W[i*n + j] * x[j];     /* ROW-MAJOR */
#         y[i] = s;
#     }
# }
# NOTE THE ROW-MAJOR INDEXING W[i*n + j]. C, numpy and
# PyTorch are row-major; Fortran and BLAS conventions are
# column-major. Getting this wrong transposes your model
# silently.

# --- A NEURAL NETWORK LAYER IS EXACTLY THIS ---
#      y = W x + b
#   W: (out_features, in_features)
#   x: (in_features,)
#   b: (out_features,)
#   y: (out_features,)
#
# nn.Linear(768, 3072) holds a (3072, 768) weight matrix.
# It maps a 768-dimensional vector to a 3072-dimensional
# one. THAT IS ALL A LINEAR LAYER IS.

# --- BATCHED, which is how it actually runs ---
X = np.random.randn(32, 768)      # 32 inputs, ROWS
W = np.random.randn(3072, 768)
b = np.random.randn(3072)

Y = X @ W.T + b                   # (32, 3072)
# One matrix multiply does all 32 at once. This is why
# batching matters for throughput - see chapter 04.
```

Special matrices worth recognising

```
/* --- IDENTITY --- */
    I = np.eye(3)
    I @ x == x       for every x

/* --- DIAGONAL: independent scaling per dimension --- */
    D = np.diag([2., 3., 4.])
    D @ x            # [2*x0, 3*x1, 4*x2]
    /* CHEAP: O(n) instead of O(n^2). Whenever a
       transformation is diagonal, the implementation should
       be an elementwise multiply, not a matmul. LayerNorm's
       scale parameter is exactly this (chapter 22). */

/* --- SYMMETRIC: A == A.T --- */
    Covariance matrices, Gram matrices, attention scores
    before masking when queries and keys are the same. They
    have real eigenvalues and orthogonal eigenvectors
    (chapter 05).

/* --- ORTHOGONAL: Q.T @ Q == I --- */
    ROTATIONS AND REFLECTIONS. They PRESERVE LENGTHS AND
    ANGLES:
        ||Q x|| == ||x||
    And the inverse is free: Q^-1 == Q.T.

    WHY THIS MATTERS FOR TRAINING: an orthogonal
    transformation neither amplifies nor shrinks. Orthogonal
    weight initialisation is used in deep networks precisely
    to stop signals exploding or vanishing as they pass
    through many layers (chapter 15).

/* --- TRIANGULAR --- */
    np.tril(np.ones((4,4)))
        [[1 0 0 0]
         [1 1 0 0]
         [1 1 1 0]
         [1 1 1 1]]
    THIS EXACT MATRIX IS THE CAUSAL MASK IN A TRANSFORMER
    (chapter 20): position i may attend to positions 0..i and
    nothing later. When you see tril() in model code, that is
    what it is doing.

/* --- SPARSE --- */
    Mostly zeros. Stored as (index, value) pairs. Pruned
    networks and mixture-of-experts routing produce these,
    and exploiting sparsity is a major topic in inference
    hardware - but only STRUCTURED sparsity (e.g. 2:4)
    reliably translates into speed, because unstructured
    sparsity destroys memory locality.                    */
```

Inverse, rank and singularity

```
/* --- INVERSE: the transformation that undoes it --- */
    A @ A_inv == I

    A_inv = np.linalg.inv(A)

/* NOT EVERY MATRIX HAS ONE. If the transformation LOST
   INFORMATION, it cannot be undone:

        P = [1 0]     flattens the plane onto a line.
            [0 0]     Every point on a vertical line maps to
                      the same output. Which one would the
                      inverse return?

   Such a matrix is SINGULAR, and its DETERMINANT IS ZERO.

/* --- DETERMINANT: the volume scale factor --- */
    np.linalg.det(np.diag([2., 3.]))     # 6.0
    /* the unit square becomes a 2x3 rectangle */

    det == 0   ->  the transformation COLLAPSES space
                   -> singular, no inverse
    det < 0    ->  orientation is FLIPPED (a reflection)

/* --- RANK: the dimension of the output space --- */
    np.linalg.matrix_rank(A)

    A (3, 3) matrix with rank 3 is FULL RANK - invertible.
    Rank 2 means it squashes 3-D into a plane. Rank 1 means
    everything lands on a line.

/* --- WHY THIS SECTION MATTERS, and it does concretely ---

   1. LOW-RANK ADAPTATION (LoRA), the standard way to
      fine-tune a large model (chapter 24), works by adding
      a LOW-RANK UPDATE:

          W_new = W + B @ A       with A: (r, d), B: (d, r)
                                  and r much smaller than d

      Instead of training d*d numbers, you train 2*d*r. For
      d = 4096 and r = 8 that is 16.7 million parameters
      versus 65,536 - a 250x reduction. THE ENTIRE TECHNIQUE
      IS A STATEMENT ABOUT RANK.

   2. YOU ALMOST NEVER COMPUTE AN INVERSE NUMERICALLY.
      To solve Ax = b, use np.linalg.solve(A, b), not
      inv(A) @ b. It is faster and far more numerically
      stable. Explicit inversion is a red flag in numerical
      code.

   3. NEAR-SINGULAR MATRICES are the practical problem. A
      determinant of 1e-15 is not zero, but the inverse will
      be numerically garbage. The CONDITION NUMBER
      (np.linalg.cond) measures this: how much a small input
      error is amplified. A condition number of 1e10 means
      you have lost ten digits of precision.

   4. AND IN DEEP LEARNING, INVERSES BARELY APPEAR AT ALL.
      Training uses gradient descent (chapter 11), not
      solving linear systems. The concepts matter for
```

```
      UNDERSTANDING - especially rank - more than for the
      code you will write.                                */
```

> **COMPOSITION IS MULTIPLICATION**
> Applying transformation `A` and then `B` is the same as applying the single matrix `B @ A`. Note the order: the rightmost matrix acts first, because it is written next to the vector in `B @ (A @ x)`. This is exactly why stacked linear layers with no non-linearity between them collapse into one layer — `W3 @ W2 @ W1` is just some other matrix, and the depth bought you nothing. The whole reason activation functions exist is to break this collapse (chapter 14).

How it shows up

Symptom	Cause
`shapes (32,768) and (3072,768) not aligned`	Missing `.T` — inner dimensions must match
Model trains but outputs are nonsense	Weight matrix transposed — row-major vs column-major
`LinAlgError: singular matrix`	Rank-deficient — the transformation lost information
Results differ wildly between runs	Ill-conditioned matrix — check `np.linalg.cond`
A deep stack performs no better than one layer	No non-linearity — the layers collapsed
Activations explode through the layers	Weight scale — orthogonal initialisation helps
Diagonal operation is unexpectedly slow	Materialised as a full matmul instead of an elementwise multiply
The model attends to future tokens	Causal mask (`tril`) missing or inverted
Pruning gives no speedup	Unstructured sparsity — hardware needs structured patterns

Check yourself

- ✓ State what the columns of a matrix represent, and use it to compute `W [3,4]` by hand.
- ✓ Say what a `(m, n)` matrix does to dimensions, and which is input and which output.
- ✓ Write matrix-vector multiplication as a C loop with row-major indexing.
- ✓ Give the weight shape of `nn.Linear(768, 3072)` and explain it.
- ✓ Explain what makes a matrix orthogonal and why that helps deep networks.
- ✓ Say what `tril` is doing in transformer code.
- ✓ Explain the connection between determinant, rank and invertibility.
- ✓ Explain LoRA as a statement about rank, with the parameter count.
- ✓ Say why `solve` is preferred over `inv`.
- ✓ Explain why stacked linear layers collapse and what prevents it.

Matrix multiplication: the core operation

Well over 90% of the arithmetic in training or running a neural network is matrix multiplication. This chapter is the one an embedded engineer will find most familiar — it is a memory hierarchy problem wearing a mathematics costume.

The definition

```
A is (m, k)     B is (k, n)     C = A @ B is (m, n)

     C_ij = sum over p of  A_ip * B_pj

IN WORDS: element (i, j) of the result is ROW i OF A
dotted with COLUMN j OF B.

THE INNER DIMENSIONS MUST MATCH. That is the only rule,
and it is the rule your shape errors are about:

     (m, k) @ (k, n) -> (m, n)
          ^^^^^^^ these must be equal
     ^^^          ^^^ these survive

A WORKED 2x3 @ 3x2:

  A = [1 2 3]        B = [ 7  8]
      [4 5 6]            [ 9 10]
                         [11 12]

  C_00 = 1*7 + 2*9  + 3*11 = 7 + 18 + 33 = 58
  C_01 = 1*8 + 2*10 + 3*12 = 8 + 20 + 36 = 64
  C_10 = 4*7 + 5*9  + 6*11 = 28 + 45 + 66 = 139
  C_11 = 4*8 + 5*10 + 6*12 = 32 + 50 + 72 = 154

  C = [ 58  64]
      [139 154]

DO THIS ONCE BY HAND. It is the last time you will need
the index formula, and it makes every shape error
afterwards obvious.

PROPERTIES:
  ASSOCIATIVE     (AB)C = A(BC)   <- and the grouping
                                     changes the COST
                                     enormously
  DISTRIBUTIVE    A(B+C) = AB + AC
  NOT COMMUTATIVE AB != BA        <- usually not even the
                                     same shape
  TRANSPOSE       (AB)^T = B^T A^T   <- note the reversal;
                                        this shows up
                                        constantly in
                                        backpropagation
                                        (chapter 07)
```

What it costs

```
/* (m, k) @ (k, n) requires  m * n * k  multiply-accumulates.

   FOR A TRANSFORMER LAYER with d_model = 4096 and a batch
   of 1024 tokens:
     the QKV projection:  1024 x 4096 x 12288
                          = 51.5 GFLOP-pairs, i.e. ~103
                            GFLOPs counting multiply and add

   AND A FULL FORWARD PASS of a 70-billion-parameter model
   on one token is roughly 2 * 70e9 = 140 GFLOPs. THE RULE
   OF THUMB WORTH MEMORISING:

     inference FLOPs  ~=  2 * (number of parameters) * tokens
     training FLOPs   ~=  6 * (number of parameters) * tokens

   The factor of 6 is: 2 for the forward pass, 4 for the
   backward pass (which computes two gradients per weight).
   This single formula is how every training-cost estimate
   in every paper is produced, and it is used again in
   chapter 23.

/* --- THE NAIVE IMPLEMENTATION --- */
    for (i = 0; i < m; i++)
      for (j = 0; j < n; j++) {
        float s = 0;
        for (p = 0; p < k; p++)
            s += A[i*k + p] * B[p*n + j];   /* <-- B IS
                                                   STRIDED */
        C[i*n + j] = s;
      }

/* THIS IS CORRECT AND SLOW, and the reason is entirely
   familiar to an embedded engineer:

   A[i*k + p]   walks CONTIGUOUSLY. Good: one cache miss
                per line, then hits.
   B[p*n + j]   jumps n floats every iteration. EVERY ACCESS
                IS A CACHE MISS on a large matrix.

   With n = 4096 and 64-byte lines, each B access pulls in a
   line and uses 4 bytes of it. YOU ARE RUNNING AT 1/16 OF
   MEMORY BANDWIDTH before you even count latency.

/* --- FIX 1: LOOP REORDERING (ikj) --- */
    for (i = 0; i < m; i++)
      for (p = 0; p < k; p++) {
        float a = A[i*k + p];
        for (j = 0; j < n; j++)
            C[i*n + j] += a * B[p*n + j];  /* BOTH
                                              CONTIGUOUS */
      }
    /* 2-5x faster for a one-line change, purely from access
       pattern. Vectorises cleanly too. */

/* --- FIX 2: BLOCKING / TILING --- */
    Work on TILES that fit in cache. Load a tile of A and a
    tile of B, compute all the products they contribute to,
    then move on.

    for (ii = 0; ii < m; ii += BS)
```

```
      for (pp = 0; pp < k; pp += BS)
        for (jj = 0; jj < n; jj += BS)
          /* multiply the BS x BS blocks */

    Choose BS so three BS x BS tiles fit in L1 or L2. This
    turns an O(m*n*k) memory-traffic problem into
    O(m*n*k / BS), and it is THE reason a tuned BLAS is 10-50x
    faster than the naive loop.

/* --- FIX 3: SIMD --- */
    NEON on ARM, AVX on x86. Eight or sixteen MACs per
    instruction.

    #include <arm_neon.h>
    float32x4_t acc = vdupq_n_f32(0.0f);
    for (p = 0; p < k; p += 4) {
        float32x4_t va = vld1q_f32(&A[i*k + p]);
        float32x4_t vb = vld1q_f32(&B[p*n + j]);
        acc = vfmaq_f32(acc, va, vb);   /* fused
                                           multiply-add */
    }

/* --- FIX 4: DEDICATED HARDWARE --- */
    GPU tensor cores, ARM SME, NPU MAC arrays, systolic
    arrays. All of them are "do a small matrix multiply as
    one instruction". A systolic array flows data through a
    grid of MAC units so each value loaded from memory is
    REUSED by many multipliers - the same tiling idea,
    implemented in silicon.

/* --- AND THE POINT --- */
    NEVER WRITE YOUR OWN MATMUL FOR PRODUCTION. Use BLAS,
    cuBLAS, oneDNN, CMSIS-NN, or whatever your NPU vendor
    ships. Write one once to understand it, then never
    again.

    But DO understand the cost model, because it explains
    every performance decision in the rest of this track.  */
```

Compute bound and memory bound

```
ARITHMETIC INTENSITY = FLOPs performed / bytes moved

FOR A SQUARE (n,n) @ (n,n) MATMUL:
  FLOPs  = 2 n^3
  bytes  = 3 n^2 * 4      (two inputs, one output, fp32)
  intensity = 2n / 12  ~= n / 6

SO INTENSITY GROWS WITH n. Big matmuls are COMPUTE BOUND;
the hardware can keep its multipliers busy.

FOR A MATRIX-VECTOR PRODUCT (n,n) @ (n,1):
  FLOPs  = 2 n^2
  bytes  = n^2 * 4        (the matrix dominates)
  intensity ~= 0.5

CONSTANT, AND TINY. Matrix-vector is MEMORY BOUND: you
read each weight once, use it twice, and throw it away.

THE ROOFLINE:

  performance
     ^
     |          ____________ peak compute
     |         /
     |        /   COMPUTE BOUND
     |       /
     |      /
     |     /  MEMORY BOUND
     |    /   (slope = memory bandwidth)
     |   /
     +--/------------------------> arithmetic intensity
        ^
        the ridge point

WHY THIS EXPLAINS LLM INFERENCE:

  PREFILL (processing the prompt) does matrix-MATRIX
    products over all prompt tokens at once. COMPUTE
    BOUND. Fast per token.

  DECODE (generating one token at a time) does
    matrix-VECTOR products - batch of one. MEMORY BOUND.
    Every weight in the model must be read from memory to
    produce ONE token.

FOR A 70B MODEL IN fp16 THAT IS 140 GB READ PER TOKEN. At
2 TB/s of HBM bandwidth, that is ~70 ms per token, ~14
tokens/second - AND THE ARITHMETIC UNITS ARE MOSTLY IDLE.

THIS SINGLE FACT DRIVES:
  · QUANTISATION (chapter 26) - int4 weights means 1/4 the
    bytes, so ~4x the tokens per second. The speedup comes
    from BANDWIDTH, not from faster arithmetic.
  · BATCHING - serve 32 requests together and the weight
    read is amortised across 32 tokens
  · SPECULATIVE DECODING - a small model proposes several
    tokens, the big model verifies them in ONE
    memory-bound pass
  · KV CACHING (chapter 25) - and why the KV cache itself
    becomes the bandwidth bottleneck at long contexts
```

```
   IF YOU REMEMBER ONE THING FROM THIS TRACK'S SYSTEMS SIDE:
   GENERATING TEXT IS A MEMORY BANDWIDTH PROBLEM.
```

Where matmuls appear

```
import numpy as np

# --- 1. A LINEAR LAYER (chapter 03) ---
X = np.random.randn(32, 768)        # batch, d_in
W = np.random.randn(768, 3072)      # d_in, d_out
Y = X @ W                           # (32, 3072)

# --- 2. ATTENTION SCORES (chapter 19) ---
Q = np.random.randn(32, 128, 64)    # batch, seq, d_head
K = np.random.randn(32, 128, 64)
scores = Q @ K.transpose(0, 2, 1)   # (32, 128, 128)
# BATCHED MATMUL: numpy and torch treat all leading
# dimensions as batch and multiply the last two. This is how
# multi-head attention runs every head in one call
# (chapter 21).

# --- 3. THE OUTPUT PROJECTION TO VOCABULARY ---
h = np.random.randn(32, 4096)          # hidden states
W_out = np.random.randn(4096, 50000)   # vocab
logits = h @ W_out                     # (32, 50000)
# 6.5 GFLOPs for a batch of 32. The vocabulary projection is
# often the single largest matmul in a small model, which is
# why weight tying (sharing it with the embedding table) is
# common (chapter 17).

# --- ASSOCIATIVITY IS A REAL OPTIMISATION ---
A = np.random.randn(1000, 5)
B = np.random.randn(5, 1000)
C = np.random.randn(1000, 20)

# (A @ B) @ C : 1000*1000*5 + 1000*20*1000 = 25,000,000
# A @ (B @ C) : 5*20*1000   + 1000*20*5    =    200,000
#                                            125x FEWER
# Same answer. THIS IS EXACTLY WHY LOW-RANK METHODS ARE
# CHEAP (chapter 03's LoRA), and why linear-attention
# variants reorder Q(K^T V) as (QK^T)V or not depending on
# the sequence length.

# --- CHECKING YOUR SHAPES, always ---
def linear(x, W, b):
    assert x.shape[-1] == W.shape[0], \
        f"{x.shape} @ {W.shape}"
    out = x @ W + b
    assert out.shape[-1] == W.shape[1]
    return out
```

THE DEBUGGING HABIT

When model code does not work, print shapes before printing values. Nine times in ten the bug is a transpose, a missing batch dimension, or a broadcast that silently produced the wrong rank. Shapes are cheap to check and they localise the error immediately; values tell you almost nothing until the shapes are right.

How it shows up

Symptom	Cause
`shapes (a,b) and (c,d) not aligned`	`b != c` — a missing transpose
Result has an extra dimension	Broadcasting over an unintended batch axis
Naive matmul is 30× slower than numpy	Cache-hostile access pattern — reorder and tile
GPU utilisation near zero during generation	Memory-bound decode — expected; batch or quantise
Quantising to int4 gives 4× speedup but not 4× FLOPs	The gain is bandwidth, not arithmetic
Throughput collapses at batch size 1	Matrix-vector, not matrix-matrix
An operation is far slower than the FLOP count suggests	Low arithmetic intensity — check the roofline
Chained matmuls are unexpectedly slow	Wrong association order — regroup
Results differ slightly between CPU and GPU	Different summation order in floating point — expected

Check yourself

- ✓ Multiply a 2×3 by a 3×2 by hand.
- ✓ State the shape rule and predict the output shape of any pair.
- ✓ State the FLOP rules for inference and training, and explain the factor of 6.
- ✓ Explain why `B[p*n + j]` is cache-hostile and what reordering fixes.
- ✓ Explain blocking and how to choose the tile size.
- ✓ Define arithmetic intensity and compute it for matmul and matrix-vector.
- ✓ Explain why prefill is compute bound and decode is memory bound.
- ✓ Compute tokens/second for a 70B fp16 model at 2 TB/s and explain the number.
- ✓ Explain why int4 quantisation speeds up generation.
- ✓ Give the associativity example and say why LoRA is cheap.

Eigenvectors, SVD and dimensionality

Some directions survive a transformation unchanged, and some carry most of the information in a dataset. Finding them is how you compress, denoise, and understand what a matrix is really doing.

Eigenvectors: the directions that survive

```
MOST vectors change direction when a matrix acts on them.
A FEW ONLY GET SCALED:

     A v = lambda * v

v is an EIGENVECTOR, lambda is its EIGENVALUE.

A = [3 1]      try v = [1, 1]:
    [1 3]            A v = [3+1, 1+3] = [4, 4] = 4 * [1,1]
                     EIGENVECTOR, eigenvalue 4

               try v = [1, -1]:
                     A v = [3-1, 1-3] = [2, -2]
                         = 2 * [1,-1]
                     EIGENVECTOR, eigenvalue 2

               try v = [1, 0]:
                     A v = [3, 1]  - a DIFFERENT direction.
                     Not an eigenvector.

GEOMETRICALLY, for this A: the plane is stretched by 4
along the [1,1] diagonal and by 2 along [1,-1]. THOSE TWO
NUMBERS AND DIRECTIONS DESCRIBE THE ENTIRE
TRANSFORMATION.

WHAT THE EIGENVALUES TELL YOU:
  |lambda| > 1   that direction is AMPLIFIED
  |lambda| = 1   preserved
  |lambda| < 1   SHRUNK toward zero
  lambda < 0     flipped
  complex        rotation is involved

AND THE REASON THIS MATTERS FOR NEURAL NETWORKS:

APPLYING A MATRIX REPEATEDLY - which is exactly what a
deep network, or a recurrent network unrolled over time,
does - AMPLIFIES ALONG THE LARGEST EIGENVALUE AND KILLS
EVERYTHING ELSE:

     A^n v  ->  dominated by lambda_max^n

  lambda_max > 1  ->  EXPLODING GRADIENTS
  lambda_max < 1  ->  VANISHING GRADIENTS

That is not a metaphor; it is the literal mechanism
behind why deep RNNs could not be trained (chapter 18)
and why weight initialisation is chosen to keep the
spectrum near 1 (chapter 15).

EIGENDECOMPOSITION, when it exists:

     A = V diag(lambda) V^-1

"change to the eigenbasis, scale each axis, change back."
For SYMMETRIC matrices, V is ORTHOGONAL and this is
always possible with real eigenvalues - which is why
covariance matrices are so pleasant to work with.
```

Singular value decomposition

```
/* EIGENVECTORS ONLY EXIST NICELY FOR SQUARE MATRICES. THE
   SVD WORKS FOR ANY MATRIX AT ALL, AND IT IS THE MORE
   USEFUL TOOL.                                          */

        A  =  U  S  V^T

     A: (m, n)   any matrix
     U: (m, m)   ORTHOGONAL - output directions
     S: (m, n)   DIAGONAL, non-negative, sorted DESCENDING
                 - the SINGULAR VALUES
     V: (n, n)   ORTHOGONAL - input directions

/* THE GEOMETRIC READING, and it is complete:

     EVERY LINEAR TRANSFORMATION IS
         a ROTATION (V^T),
         then a SCALING along axes (S),
         then another ROTATION (U).

   That is all any matrix ever does. The singular values say
   how much stretching happens in each of n perpendicular
   directions.                                           */

import numpy as np

A = np.array([[3., 1.], [1., 3.], [0., 2.]])   # (3, 2)
U, s, Vt = np.linalg.svd(A, full_matrices=False)

U.shape     # (3, 2)
s           # [4.24, 1.87]   <- SORTED, DESCENDING, always
Vt.shape    # (2, 2)

# reconstruct
A_check = U @ np.diag(s) @ Vt
np.allclose(A, A_check)      # True

/* WHAT THE SINGULAR VALUES MEAN:

   · s[0] is the largest stretch factor = the SPECTRAL NORM
   · the number of NON-ZERO singular values is the RANK
     (chapter 03)
   · s[0] / s[-1] is the CONDITION NUMBER - how badly the
     matrix amplifies errors
   · a singular value of 1e-16 is numerically zero, which is
     the reliable way to compute rank

/* --- LOW-RANK APPROXIMATION: THE ECKART-YOUNG THEOREM --- */

    Keep only the k largest singular values:

        A_k = U[:, :k] @ diag(s[:k]) @ Vt[:k, :]

    THIS IS PROVABLY THE BEST RANK-k APPROXIMATION TO A, in
    the least-squares sense. Not a heuristic - optimal.

    STORAGE: k(m + n + 1) numbers instead of m*n.
    For a (4096, 4096) matrix with k = 64:
        16.8 million -> 524 thousand.  32x smaller.

/* AND THIS IS DIRECTLY WHY LoRA WORKS (chapters 03, 24).
```

```
   The empirical observation is that the WEIGHT UPDATE
   during fine-tuning has low intrinsic rank - the
   adaptation lives in a small subspace - so representing it
   as B @ A with small r loses almost nothing.

   IT IS ALSO the basis of several model-compression methods,
   of PCA below, and of classical latent semantic analysis -
   which was doing "embeddings via SVD" decades before
   neural embeddings (chapter 17).                        */
```

PCA: finding the directions that matter

```
/* THE QUESTION: given data in 768 dimensions, which few
   directions capture most of the variation?             */

import numpy as np

X = np.random.randn(1000, 50)      # 1000 samples, 50 dims

# 1. CENTRE THE DATA. Not optional - PCA finds directions of
#    variance ABOUT THE MEAN, and skipping this makes the
#    first component point at the mean instead.
X_centred = X - X.mean(axis=0)

# 2. SVD OF THE CENTRED DATA
#    (More stable than eigendecomposing the covariance
#     matrix, which squares the condition number.)
U, s, Vt = np.linalg.svd(X_centred, full_matrices=False)

# 3. THE PRINCIPAL COMPONENTS ARE THE ROWS OF Vt
components = Vt[:10]               # top 10 directions

# 4. PROJECT ONTO THEM
X_reduced = X_centred @ components.T    # (1000, 10)

# 5. HOW MUCH DID WE KEEP?
variance = s**2 / (len(X) - 1)
explained = variance / variance.sum()
print(explained[:10].sum())        # e.g. 0.85 -> 85%

/* THE INTERPRETATION:
     PC1  the direction of greatest variance in the data
     PC2  the greatest remaining variance, PERPENDICULAR to
          PC1
     ... and so on. Each is orthogonal to all previous ones.

/* WHERE YOU MEET IT:
     · VISUALISATION: project 768-dimensional embeddings to
       2-D to plot them. (t-SNE and UMAP are more popular
       for this and preserve local structure better - but
       they DISTORT global distances, so never read
       cluster-to-cluster distances off a t-SNE plot.)
     · WHITENING: decorrelate and unit-scale features before
       training
     · NOISE REDUCTION: keep the top components, discard the
       tail
     · UNDERSTANDING WHAT A MODEL LEARNED: PCA of a layer's
       activations often reveals interpretable directions

/* AND THE HONEST LIMITATION: PCA IS LINEAR. It finds
   directions, not curves. Data on a spiral or a sphere has
   structure PCA cannot represent, which is precisely the
   gap that non-linear models - autoencoders, and neural
   networks generally (chapter 14) - fill.

/* --- THE POWER METHOD, because it explains a lot --- */
    def power_iteration(A, iters=100):
        v = np.random.randn(A.shape[1])
        for _ in range(iters):
            v = A @ v
            v = v / np.linalg.norm(v)   # RENORMALISE, or it
                                        # overflows
```

```
    return v, v @ A @ v             # eigenvector,
                                    # eigenvalue

Repeatedly multiply and renormalise; converge to the
dominant eigenvector. NOTHING BUT MATRIX-VECTOR PRODUCTS.

THIS IS THE SAME AMPLIFICATION that causes exploding
gradients - here used deliberately. It is also,
essentially, PageRank. And the renormalisation step is
exactly what normalisation layers do inside a network
(chapter 22): stop repeated multiplication from
diverging.                                              */
```

WHERE THIS ACTUALLY APPEARS IN MODERN MODELS

```
LoRA fine-tuning        a low-rank update; ch. 24
Model compression       SVD of weight matrices
Attention analysis      SVD of the QK matrix reveals
                        what a head compares
Embedding visualisation PCA / t-SNE / UMAP; ch. 17
Gradient clipping       bounding the spectral norm
Spectral normalisation  dividing weights by s[0] to
                        stabilise training
Whitening               decorrelating inputs; ch. 15
```

You will rarely call `np.linalg.svd` in a training loop. You will constantly meet the *ideas* — rank, spectrum, dominant direction — in explanations of why something trains or does not.

The numerical reality

```
/* --- DO NOT EIGENDECOMPOSE A COVARIANCE MATRIX --- */
    /* WRONG - squares the condition number */
    C = X.T @ X / n
    vals, vecs = np.linalg.eigh(C)

    /* RIGHT - SVD of the data directly */
    U, s, Vt = np.linalg.svd(X_centred)
    /* The singular values of X are the square roots of the
       eigenvalues of X^T X, so you get the same answer with
       half the numerical damage. */

/* --- SIGNS ARE ARBITRARY --- */
    If v is an eigenvector, so is -v. Different libraries,
    versions and platforms return different signs. NEVER
    WRITE A TEST THAT DEPENDS ON THE SIGN of a component;
    compare absolute values or fix a convention yourself.

/* --- ORDER IS ONLY GUARANTEED FOR SVD --- */
    np.linalg.svd returns singular values DESCENDING.
    np.linalg.eig returns eigenvalues in NO PARTICULAR
    ORDER. np.linalg.eigh (symmetric) returns them
    ASCENDING. Read the docs; assuming the wrong one silently
    selects the least important components.

/* --- COST --- */
    Full SVD of an (m, n) matrix is O(m n min(m,n)).
    For a (4096, 4096) weight matrix that is billions of
    operations - seconds, not microseconds.

    IF YOU ONLY NEED THE TOP k:
        from scipy.sparse.linalg import svds
        U, s, Vt = svds(A, k=10)          # ITERATIVE
    or randomised SVD, which is what scikit-learn's
    TruncatedSVD uses by default. Both are dramatically
    cheaper and are what production code uses.

/* --- AND A WARNING ABOUT INTERPRETATION --- */
    A large singular value means a direction with large
    VARIANCE. That is not the same as a direction that is
    IMPORTANT FOR YOUR TASK. A feature can vary enormously
    and be irrelevant; a subtle direction can be decisive.
    PCA is unsupervised - it does not know what you are
    predicting. When labels are available, methods that use
    them (LDA, or simply training a model) will find better
    directions.                                          */
```

How it shows up

Symptom	Cause
PC1 points at the data mean	Data not centred before PCA
Components flip sign between runs	Eigenvector signs are arbitrary — expected
Selected the least important components	Assumed descending order from `eigh`, which is ascending
Gradients explode through depth	Largest eigenvalue above 1, amplified by repeated multiplication
Gradients vanish through depth	Largest eigenvalue below 1

Symptom	Cause
Numerically garbage principal components	Eigendecomposed the covariance matrix — use SVD of the data
SVD takes minutes	Full decomposition where truncated or randomised SVD suffices
Reduced features perform worse than raw	PCA is unsupervised — high variance is not relevance
t-SNE clusters look far apart but are not	t-SNE does not preserve global distances
`power_iteration` overflows	Missing renormalisation each step

Check yourself

- ✓ Define an eigenvector and verify one by hand for a 2×2 matrix.
- ✓ Explain how eigenvalues cause exploding and vanishing gradients.
- ✓ State the SVD and give the rotate–scale–rotate reading.
- ✓ Say what singular values tell you about rank, norm and conditioning.
- ✓ State the Eckart–Young result and compute the storage saving for a rank-64 approximation of a 4096×4096 matrix.
- ✓ Connect low-rank approximation to LoRA.
- ✓ Write the five steps of PCA and explain why centring is mandatory.
- ✓ Say what PCA cannot represent and what fills that gap.
- ✓ Describe the power method and its two connections elsewhere in the track.
- ✓ Give three numerical pitfalls: covariance eigendecomposition, signs, ordering.

Calculus: derivatives and gradients

A derivative answers one question: if I nudge this input, how much does the output move? Training a neural network is nothing but asking that question about a few billion inputs at once and stepping downhill.

The derivative

```
THE DEFINITION:

     f'(x) = lim       f(x + h) - f(x)
             h->0      ---------------
                              h

IN WORDS: the rate of change. The slope of the tangent
line. "If x increases by a tiny amount, f increases by
f'(x) times that amount."

     f(x) = x^2

      |          .
      |         /
      |       ./  slope at x=3 is 6
      |     ../
      |   ..
      | ..
   ---+------------
      0    3

  f'(3) = 6  means: nudge x from 3 to 3.001 and f goes
  from 9 to about 9.006.

THE ONLY DERIVATIVES YOU NEED HERE:

  f(x) = c            f' = 0
  f(x) = x            f' = 1
  f(x) = x^n          f' = n x^(n-1)
  f(x) = e^x          f' = e^x          <- itself!
  f(x) = ln(x)        f' = 1/x
  f(x) = sin(x)       f' = cos(x)

  sum:      (f + g)' = f' + g'
  scalar:   (c f)'   = c f'
  product:  (f g)'   = f' g + f g'
  quotient: (f/g)'   = (f' g - f g') / g^2
  CHAIN:    f(g(x))' = f'(g(x)) * g'(x)   <- CHAPTER 07.
                                              THE ONE THAT
                                              MATTERS.

THE SIGN IS THE POINT:

  f'(x) > 0   f is INCREASING here -> to decrease f, go
              LEFT
  f'(x) < 0   f is DECREASING here -> to decrease f, go
              RIGHT
  f'(x) = 0   flat: a minimum, a maximum, or a saddle

SO: TO MINIMISE f, MOVE IN THE DIRECTION OPPOSITE TO THE
DERIVATIVE. That single sentence is gradient descent
(chapter 11), and everything else is bookkeeping.
```

Partial derivatives and the gradient

```
/* Real functions have many inputs. A neural network's loss
   is a function of every weight in the model - billions of
   them.

   A PARTIAL DERIVATIVE holds all other inputs fixed:

        f(x, y) = x^2 + 3xy + y^2

        df/dx = 2x + 3y        (treat y as a constant)
        df/dy = 3x + 2y        (treat x as a constant)

   THE GRADIENT collects them all into a vector:

        grad f = [df/dx, df/dy]

   At the point (1, 2):
        grad f = [2*1 + 3*2, 3*1 + 2*2] = [8, 7]

/* THE TWO FACTS ABOUT THE GRADIENT THAT MATTER:

   1. IT POINTS IN THE DIRECTION OF STEEPEST INCREASE.
      So -grad f points steepest DOWNHILL. That is the
      update rule.

   2. ITS MAGNITUDE IS THE STEEPNESS. A large gradient means
      a small step changes the loss a lot; near a minimum
      the gradient goes to zero and progress stalls
      naturally.

/* THE UPDATE RULE, in full: */

        w  <-  w - learning_rate * grad_w L

   Every training run of every model in this track is that
   line, repeated.

/* --- IN CODE --- */
import numpy as np

def f(v):
    x, y = v
    return x**2 + 3*x*y + y**2

def grad_f(v):
    x, y = v
    return np.array([2*x + 3*y, 3*x + 2*y])

# ONE STEP OF GRADIENT DESCENT
v = np.array([1.0, 2.0])
lr = 0.1
for step in range(5):
    g = grad_f(v)
    v = v - lr * g
    print(step, v, f(v))

# Watch f decrease. That is the entire mechanism of
# training, at two parameters instead of seventy billion.

/* --- HIGHER DERIVATIVES, briefly --- */
    The HESSIAN is the matrix of second partial derivatives.
```

```
    It describes CURVATURE:
      · all eigenvalues positive -> a bowl, a true minimum
      · mixed signs               -> a SADDLE POINT
      · near-zero eigenvalues     -> a flat plateau

    SECOND-ORDER METHODS (Newton's method) use it and
    converge in far fewer steps. NOBODY USES THEM FOR LARGE
    NEURAL NETWORKS: the Hessian for n parameters has n^2
    entries. For a 7-billion-parameter model that is 4.9e19
    numbers. Not storable, not computable, not close.

    SO ALL DEEP LEARNING IS FIRST-ORDER. Adam and friends
    (chapter 11) approximate curvature information cheaply
    from the history of gradients, which is the closest
    anyone gets.                                          */
```

The functions that actually appear

```
/* You need the derivatives of these five. They are the
   whole vocabulary of a neural network's forward pass.  */

/* --- ReLU: the default activation --- */
    relu(x)  = max(0, x)
    relu'(x) = 1 if x > 0 else 0

    ONE COMPARISON AND A SELECT. No exponentials, no
    division. That cheapness is a large part of why deep
    learning became practical - and it matters even more on
    an MCU (chapter 27).

    THE PROBLEM: for x <= 0 the gradient is EXACTLY ZERO. A
    unit that always outputs zero never updates. This is a
    DEAD ReLU, and it is a real failure mode.
    THE FIXES: LeakyReLU (slope 0.01 for x < 0), GELU, SiLU.

/* --- SIGMOID --- */
    sigmoid(x)  = 1 / (1 + e^-x)                 range (0,1)
    sigmoid'(x) = sigmoid(x) * (1 - sigmoid(x))

    NOTE THE ELEGANCE: the derivative is expressed in terms
    of the OUTPUT, so a forward pass that saved its output
    gets the backward pass almost free. Every framework
    exploits this.

    THE PROBLEM: the maximum of s(1-s) is 0.25, at x = 0.
    Multiply that through ten layers and the gradient is
    0.25^10 = 1e-6. THIS IS THE VANISHING GRADIENT PROBLEM,
    and it is why sigmoid was abandoned for hidden layers
    (chapter 18). It survives for BINARY OUTPUTS and GATES.

/* --- TANH --- */
    tanh(x)  = (e^x - e^-x)/(e^x + e^-x)        range (-1,1)
    tanh'(x) = 1 - tanh(x)^2                    max 1.0

    Zero-centred, and a maximum gradient of 1 rather than
    0.25 - strictly better than sigmoid for hidden layers,
    still saturating. Used inside LSTMs (chapter 18).

/* --- GELU and SiLU: what transformers use --- */
    gelu(x) = x * Phi(x)      Phi = the Gaussian CDF
    silu(x) = x * sigmoid(x)  (also called swish)

    Smooth, non-monotonic near zero, and empirically better
    than ReLU in transformers. GELU is in GPT and BERT;
    SwiGLU (a gated SiLU variant) is in Llama and most
    recent models (chapter 22).

/* --- SOFTMAX: turning scores into a distribution --- */
    softmax(x)_i = e^(x_i) / sum_j e^(x_j)

    Outputs are positive and sum to 1. This is how a model
    produces "the probability of each next token".

    THE NUMERICAL RULE, AND IT IS NOT OPTIONAL:

        SUBTRACT THE MAXIMUM FIRST.

        e^1000 overflows to inf in float32. But
```

```
        softmax(x) == softmax(x - c) for any constant c, so
        subtracting max(x) makes the largest exponent e^0 = 1
        and nothing overflows.

    def softmax(x):
        x = x - np.max(x, axis=-1, keepdims=True)   # <<<
        e = np.exp(x)
        return e / np.sum(e, axis=-1, keepdims=True)

    THE JACOBIAN is a full matrix (every output depends on
    every input):
        d softmax_i / d x_j = s_i (delta_ij - s_j)

    BUT - and this is the practical mercy - COMBINED WITH
    CROSS-ENTROPY LOSS THE WHOLE THING COLLAPSES TO:

        dL/dx = predicted - target

    A subtraction. That is why every framework has a fused
    `cross_entropy_with_logits` and why you should never
    apply softmax before it. Chapter 10 derives this.     */
```

NEVER APPLY SOFTMAX AND THEN A SEPARATE CROSS-ENTROPY

Doing it in two steps computes logs of small probabilities and loses precision, and it discards the algebraic cancellation that makes the gradient a clean subtraction. Pass raw logits to `F.cross_entropy` (PyTorch) or `tf.nn.softmax_cross_entropy_with_logits`. Applying softmax first is one of the most common beginner bugs, and it usually shows as a model that trains but plateaus early.

How derivatives are actually computed

```
/* THREE OPTIONS, and only one is used.                    */

/* --- 1. SYMBOLIC (what you did in school) --- */
    Manipulate the expression algebraically.
    PROBLEM: EXPRESSION SWELL. The symbolic derivative of a
    deep composition grows exponentially in size. Unusable
    beyond a few layers.

/* --- 2. NUMERICAL (finite differences) --- */
    f'(x) ~= (f(x + h) - f(x - h)) / (2h)

    PROBLEM 1: O(n) function evaluations for n parameters.
      Seven billion parameters means fourteen billion
      forward passes per step. Absurd.
    PROBLEM 2: precision. Too large an h and truncation
      error dominates; too small and floating-point
      cancellation destroys the answer. The sweet spot is
      narrow.

    BUT IT HAS ONE ESSENTIAL USE - CHECKING YOUR ANALYTIC
    GRADIENT:

    def grad_check(f, x, analytic_grad, h=1e-5):
        num = np.zeros_like(x)
        for i in range(x.size):
            xp = x.copy(); xp.flat[i] += h
            xm = x.copy(); xm.flat[i] -= h
            num.flat[i] = (f(xp) - f(xm)) / (2*h)
        rel = (np.abs(num - analytic_grad) /
               (np.abs(num) + np.abs(analytic_grad) + 1e-12))
        return rel.max()          # want < 1e-6 in float64

    IF YOU WRITE A CUSTOM LAYER OR A CUSTOM BACKWARD PASS,
    RUN THIS. It is the unit test of this field, and it
    catches sign errors and transposes that would otherwise
    show up as "the model just does not train well".

/* --- 3. AUTOMATIC DIFFERENTIATION: what is used --- */
    Build a graph of elementary operations during the
    forward pass. Each operation knows its own local
    derivative. Then apply the chain rule mechanically
    backwards through the graph.

      · EXACT to machine precision - not an approximation
      · costs about the SAME as one forward pass, for ALL
        parameters simultaneously
      · works for arbitrary code with loops and branches

    THIS IS THE ALGORITHM THAT MADE DEEP LEARNING POSSIBLE,
    and chapter 07 builds one from scratch.

/* --- SEEING IT WORK --- */
import torch

x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x**2).sum() + 3*x[0]*x[1]
y.backward()
x.grad          # tensor([8., 7.])  - matches our hand
                # calculation above
```

```
/* FORWARD MODE vs REVERSE MODE, since the terms appear:
     FORWARD  cheap for FEW inputs, many outputs
     REVERSE  cheap for MANY inputs, ONE output
   A loss function is many parameters -> one scalar, so deep
   learning always uses REVERSE MODE. That is what
   "backpropagation" names.                              */
```

How it shows up

Symptom	Cause
`nan` in the loss immediately	Softmax overflow — the max was not subtracted
Loss plateaus early despite training	Softmax applied before cross-entropy
Some units always output zero	Dead ReLU — try LeakyReLU or a lower learning rate
Gradients near zero in early layers	Saturating activations — sigmoid in hidden layers
Model trains, but worse than it should	A sign or transpose error in a custom backward — run a gradient check
Gradient check fails at 1e-2	A real bug. Below 1e-6 in float64 is passing; float32 is noisier
Gradient check fails only at ReLU	Expected at exactly zero — the kink is not differentiable there
Finite differences give garbage	`h` too small — floating-point cancellation
Training stalls at a flat region	Saddle point — momentum helps (chapter 11)

Check yourself

- ✓ Define the derivative and interpret its sign in terms of which way to move.
- ✓ Compute the gradient of `x² + 3xy + y²` at (1, 2) by hand.
- ✓ State the two facts about the gradient that make gradient descent work.
- ✓ Write the update rule.
- ✓ Explain why nobody uses the Hessian for large models, with the number.
- ✓ Give the derivatives of ReLU, sigmoid and tanh, and the failure mode of each.
- ✓ Explain the vanishing gradient with the 0.25^{10} calculation.
- ✓ Write a numerically stable softmax and say why the shift is valid.
- ✓ State what softmax + cross-entropy collapses to, and why that matters.
- ✓ Compare symbolic, numerical and automatic differentiation.
- ✓ Write a gradient check and say when you must run one.

The chain rule and backpropagation

One rule from calculus, applied mechanically to a graph of operations. This is how every neural network that has ever been trained was trained, and it is simpler than its reputation.

The chain rule

```
IF  y = f(u)  AND  u = g(x)   THEN

     dy       dy     du
     --   =   --  *  --
     dx       du     dx

"RATES MULTIPLY ALONG A CHAIN."

THE INTUITION THAT MAKES IT OBVIOUS: gears.

  x turns 3x as fast as u,
  u turns 2x as fast as y
  -> x turns 6x as fast as y.

A WORKED EXAMPLE:
     y = (3x + 1)^2

     let u = 3x + 1     du/dx = 3
     then y = u^2       dy/du = 2u

     dy/dx = 2u * 3 = 6(3x + 1)

LONGER CHAINS JUST MULTIPLY MORE TERMS:

     x -> a -> b -> c -> L

     dL     dL   dc   db   da
     --  =  -- * -- * -- * --
     dx     dc   db   da   dx

AND THAT IS BACKPROPAGATION. A neural network IS a long
chain: input -> layer -> activation -> layer -> ... ->
loss. To find how the loss depends on a weight buried
twenty layers deep, multiply the local derivatives along
the path.

TWO CONSEQUENCES THAT EXPLAIN MOST TRAINING PATHOLOGIES:

  each factor < 1  ->  the product SHRINKS toward zero
                       -> VANISHING GRADIENTS
  each factor > 1  ->  the product GROWS
                       -> EXPLODING GRADIENTS

Twenty layers of sigmoid, each contributing at most 0.25:
     0.25^20 = 1e-12.
The first layer receives essentially no signal. That is
the entire reason deep networks were considered
untrainable before ReLU, careful initialisation, and
residual connections (chapters 15 and 22).
```

The computation graph

```
/* A neural network is a DIRECTED ACYCLIC GRAPH of simple
   operations. Backpropagation walks it backwards.

   TAKE  L = (w*x + b - y)^2   -  one neuron, squared error.

   FORWARD:
        x, w  -->  [*] --> u = w*x
        u, b  -->  [+] --> v = u + b
        v, y  -->  [-] --> e = v - y
        e     -->  [^2]--> L = e^2

   BACKWARD - start with dL/dL = 1 and walk back, applying
   each node's LOCAL derivative:

        dL/dL = 1
        dL/de = 2e                 (d(e^2)/de)
        dL/dv = dL/de * 1  = 2e    (d(v-y)/dv = 1)
        dL/db = dL/dv * 1  = 2e    (d(u+b)/db = 1)
        dL/du = dL/dv * 1  = 2e
        dL/dw = dL/du * x  = 2e*x  (d(w*x)/dw = x)

   NOTE THE TWO KINDS OF NODE BEHAVIOUR - this is the whole
   of backprop:

     ADDITION  PASSES THE GRADIENT THROUGH UNCHANGED to both
               inputs. It is a gradient DISTRIBUTOR.
     MULTIPLICATION SWAPS: the gradient to each input is
               multiplied by the OTHER input's forward value.
               Which is why THE FORWARD VALUES MUST BE SAVED
               - and why training uses far more memory than
               inference.

   Every other rule follows from those two plus the local
   derivative of each elementary function (chapter 06).
```

Autodiff from scratch

```
"""A complete reverse-mode automatic differentiation engine.
   About sixty lines. Everything PyTorch does for scalars is
   here; the rest is tensors and performance."""

class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None   # how to push
                                        # gradient to inputs
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) \
                      else Value(other)
        out = Value(self.data + other.data,
                    (self, other), '+')
        def _backward():
            # ADDITION DISTRIBUTES THE GRADIENT
            self.grad  += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) \
                      else Value(other)
        out = Value(self.data * other.data,
                    (self, other), '*')
        def _backward():
            # MULTIPLICATION SWAPS THE FORWARD VALUES
            self.grad  += other.data * out.grad
            other.grad += self.data  * out.grad
        out._backward = _backward
        return out

    def __pow__(self, n):
        out = Value(self.data ** n, (self,), f'**{n}')
        def _backward():
            self.grad += n * (self.data ** (n-1)) * out.grad
        out._backward = _backward
        return out

    def relu(self):
        out = Value(max(0.0, self.data), (self,), 'relu')
        def _backward():
            # GRADIENT FLOWS ONLY WHERE THE OUTPUT WAS
            # POSITIVE
            self.grad += (out.data > 0) * out.grad
        out._backward = _backward
        return out

    def backward(self):
        # 1. TOPOLOGICAL SORT: a node's gradient is only
        #    complete once every consumer has contributed.
        topo, visited = [], set()
        def build(v):
            if v not in visited:
                visited.add(v)
```

```
                for child in v._prev:
                    build(child)
                topo.append(v)
        build(self)

        # 2. SEED and walk backwards
        self.grad = 1.0
        for v in reversed(topo):
            v._backward()

    def __neg__(self):      return self * -1
    def __sub__(self, o):   return self + (-o)
    def __radd__(self, o):  return self + o
    def __rmul__(self, o):  return self * o
    def __repr__(self):
        return f"Value({self.data:.4f}, grad={self.grad:.4f})"


# --- USE IT ---
w = Value(2.0)
x = Value(3.0)
b = Value(1.0)
y = Value(10.0)

pred = w*x + b            # 7.0
loss = (pred - y)**2      # 9.0
loss.backward()

print(w.grad)   # 2*(7-10)*3 = -18.0
print(b.grad)   # 2*(7-10)   = -6.0

# NEGATIVE GRADIENTS MEAN: INCREASING w REDUCES THE LOSS. So
# the update  w -= lr * w.grad  moves w UP. Correct.

# --- ONE GRADIENT DESCENT LOOP ---
for step in range(20):
    pred = w*x + b
    loss = (pred - y)**2
    w.grad = b.grad = 0.0        # <<< ZERO THE GRADIENTS
    loss.backward()
    w.data -= 0.01 * w.grad
    b.data -= 0.01 * b.grad

# THE `+=` IN EVERY _backward IS DELIBERATE: a value used in
# two places receives gradient from BOTH paths, and they must
# ADD (the multivariable chain rule). That accumulation is
# also why you must ZERO GRADIENTS between steps - PyTorch's
# optimizer.zero_grad() does exactly this, and forgetting it
# is one of the most common bugs in the field.
```

THE THREE-LINE SUMMARY OF BACKPROPAGATION

```
1. FORWARD: compute the output, SAVING intermediate
   values.
2. BACKWARD: seed dL/dL = 1 and walk the graph in reverse
   topological order, each node multiplying by its local
   derivative and ACCUMULATING into its inputs.
3. UPDATE: step every parameter against its gradient.
```

The cost of the backward pass is roughly twice the forward pass, and it produces gradients for *every* parameter at once. That efficiency — one backward pass for billions of derivatives — is the entire reason this field exists.

Backprop through a layer

```
/* THE SCALAR CASE GENERALISES TO MATRICES DIRECTLY. These
   three formulas are worth memorising - they are most of
   what a framework computes.                              */

    FORWARD:   Y = X @ W + b
        X: (batch, d_in)
        W: (d_in, d_out)
        b: (d_out,)
        Y: (batch, d_out)

    BACKWARD, given dY (the gradient of the loss w.r.t. Y):

        dX = dY @ W.T           (batch, d_in)
        dW = X.T @ dY           (d_in, d_out)
        db = dY.sum(axis=0)     (d_out,)

/* HOW TO REMEMBER THEM WITHOUT DERIVING: THE SHAPES ONLY
   WORK ONE WAY. dX must have X's shape; the only product of
   dY and W with that shape is dY @ W.T. This is a genuinely
   reliable check - if your shapes work, your formula is
   almost certainly right.

/* AND NOTE THE TRANSPOSES: (AB)^T = B^T A^T from chapter 04
   is exactly why they appear.                             */

import numpy as np

class Linear:
    def __init__(self, d_in, d_out):
        # He initialisation - chapter 15 explains the scale
        self.W = (np.random.randn(d_in, d_out) *
                  np.sqrt(2.0 / d_in))
        self.b = np.zeros(d_out)

    def forward(self, X):
        self.X = X                  # SAVED for backward
        return X @ self.W + self.b

    def backward(self, dY):
        self.dW = self.X.T @ dY
        self.db = dY.sum(axis=0)
        return dY @ self.W.T        # gradient to pass down

class ReLU:
    def forward(self, X):
        self.mask = X > 0           # SAVED
        return X * self.mask
    def backward(self, dY):
        return dY * self.mask       # zero where input was
                                    # negative

# --- A TWO-LAYER NETWORK, COMPLETE ---
layers = [Linear(784, 128), ReLU(), Linear(128, 10)]

def forward(X):
    for layer in layers:
        X = layer.forward(X)
    return X

def backward(dY):
```

```
    for layer in reversed(layers):   # <<< REVERSED
        dY = layer.backward(dY)

# training step
logits = forward(X_batch)
probs  = softmax(logits)
dlogits = (probs - one_hot_targets) / len(X_batch)
                      # ^^ chapter 06's clean gradient
backward(dlogits)

for layer in layers:
    if hasattr(layer, 'W'):
        layer.W -= lr * layer.dW
        layer.b -= lr * layer.db

/* THAT IS A COMPLETE, WORKING NEURAL NETWORK TRAINER. Add a
   data loader and it learns MNIST. Everything after this
   chapter is better architectures, better optimisers and
   more scale - the training mechanism does not change. */
```

What this costs, and what goes wrong

```
/* --- MEMORY: THE REASON TRAINING NEEDS SO MUCH RAM --- */
    The backward pass needs the forward ACTIVATIONS. So
    training stores, per layer, per batch element, the full
    activation tensor.

    For a transformer: roughly
        activations ~ batch * seq_len * d_model * n_layers
                      * (a constant around 10-20)

    This usually exceeds the parameter memory, and it is why
    a model that fits in memory for inference will not train
    on the same card.

    THE MITIGATIONS:
      GRADIENT CHECKPOINTING  save only some activations;
                              RECOMPUTE the rest during the
                              backward pass. ~30% more
                              compute for a large memory
                              saving. Standard practice.
      SMALLER BATCHES, with gradient accumulation to keep the
                              effective batch size
      MIXED PRECISION         fp16/bf16 activations
                              (chapter 26)

/* --- THE BUGS, in order of frequency --- */

   1. FORGETTING TO ZERO GRADIENTS.
      Gradients accumulate by design. Miss zero_grad() and
      each step uses the sum of all previous gradients. The
      model appears to train, badly and erratically.

   2. NOT DETACHING WHEN YOU SHOULD.
      Keeping a reference to a tensor that is part of the
      graph keeps the WHOLE GRAPH alive. In an RNN loop this
      is an unbounded memory leak. Use .detach() or
      .item() for logging - never accumulate a raw loss
      tensor into a running total.

   3. BACKWARD THROUGH THE GRAPH TWICE.
      The graph is freed after backward(). Calling it again
      raises an error; retain_graph=True is usually papering
      over a design mistake.

   4. IN-PLACE OPERATIONS ON SAVED TENSORS.
      x += 1 can corrupt a value the backward pass needs.
      PyTorch usually detects this; numpy will not.

   5. GRADIENT FLOW BLOCKED.
      An argmax, a rounding, a hard threshold, a .detach()
      in the wrong place - all have zero or undefined
      gradient, and everything upstream of them stops
      learning silently. If a parameter's gradient is
      exactly zero every step, this is why.

   6. EXPLODING GRADIENTS.
      Clip them:
          torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)
      Standard in transformer training, not optional.

/* --- THE DIAGNOSTIC HABIT --- */
```

```
    for name, p in model.named_parameters():
        if p.grad is None:
            print(f"{name}: NO GRADIENT")      # detached?
        else:
            print(f"{name}: {p.grad.norm():.3e}")

    Print this once at the start of training. Gradients that
    are None, exactly zero, nan, or wildly different in
    magnitude between layers all point at specific bugs, and
    all are invisible in the loss curve for a while.      */
```

How it shows up

Symptom	Cause
Loss decreases then oscillates wildly	Gradients not zeroed between steps
Memory grows every iteration	Accumulating a graph-attached tensor — detach it
`Trying to backward through the graph a second time`	The graph was freed — restructure rather than retaining
A parameter's gradient is always `None`	It is not on the path from the loss, or was detached
A parameter's gradient is always exactly zero	A non-differentiable operation upstream — argmax, rounding, hard threshold
Early layers barely change	Vanishing gradients — check activations and initialisation
Loss becomes `nan` after a few steps	Exploding gradients — clip the norm
Out of memory in training but not inference	Saved activations — use gradient checkpointing
Custom layer trains worse than expected	Wrong backward formula — run a gradient check (chapter 06)
Shapes fail in the backward pass	A missing transpose — the shapes only work one way

Check yourself

- ✓ State the chain rule and explain it with the gears analogy.
- ✓ Explain vanishing and exploding gradients as products of local derivatives.
- ✓ Backpropagate `L = (wx + b − y)²` by hand.
- ✓ State what addition and multiplication nodes do to gradients.
- ✓ Explain why the forward values must be saved.
- ✓ Write the three backward formulas for a linear layer and check them by shape.
- ✓ Explain why `_backward` uses `+=` and what that implies about `zero_grad`.
- ✓ Explain why a topological sort is needed before the backward walk.
- ✓ Estimate training activation memory and name three ways to reduce it.
- ✓ List six common backprop bugs and their symptoms.
- ✓ Write the gradient-norm diagnostic and say what each anomaly indicates.

Probability: distributions and expectation

A language model does not output text. It outputs a probability distribution over tokens, and something else turns that into text. This chapter is the vocabulary for what that distribution is and what you can do with it.

The pieces

```
RANDOM VARIABLE   a quantity whose value is uncertain.
                  X = the next token.

DISTRIBUTION      the assignment of probability to each
                  possible value.
                  P(X = "the") = 0.15, ...

THE TWO RULES, and everything follows from them:
     P(x) >= 0  for every x
     sum over all x of P(x) = 1

DISCRETE          countable outcomes; a PROBABILITY MASS
                  FUNCTION. Tokens, dice, classes.
CONTINUOUS        a PROBABILITY DENSITY FUNCTION. P(exact
                  value) = 0; only INTERVALS have
                  probability, and the DENSITY CAN EXCEED
                  1. Weights, activations, noise.

JOINT             P(X, Y)      both together
MARGINAL          P(X) = sum over y of P(X, y)
                  - "sum out" what you do not care about
CONDITIONAL       P(X | Y) = P(X, Y) / P(Y)
                  - "given that Y happened"

INDEPENDENCE      P(X, Y) = P(X) P(Y)
                  equivalently P(X | Y) = P(X)

AND THE CHAIN RULE OF PROBABILITY, which is the single
most important identity in this track:

     P(x1, x2, ..., xn)
       = P(x1) P(x2|x1) P(x3|x1,x2) ... P(xn|x1..x(n-1))

A LANGUAGE MODEL IS EXACTLY THIS FACTORISATION. The
probability of a document is the product of the
probability of each token given all preceding tokens, and
the model's only job is to estimate

     P(next token | everything so far)

Everything in chapters 16-25 is machinery for computing
that one conditional distribution well.
```

Expectation and variance

```
/* EXPECTATION: the probability-weighted average. */

        E[X] = sum over x of  x * P(x)

    A fair die:  E[X] = (1+2+3+4+5+6)/6 = 3.5
    Note it need not be an attainable value.

/* PROPERTIES THAT GET USED CONSTANTLY:
        E[aX + b]   = a E[X] + b
        E[X + Y]    = E[X] + E[Y]      ALWAYS - even if X
                                       and Y are dependent
        E[XY]       = E[X] E[Y]        ONLY IF INDEPENDENT

/* VARIANCE: the expected squared deviation. */

        Var[X] = E[(X - E[X])^2] = E[X^2] - E[X]^2

        Var[aX + b] = a^2 Var[X]       <- note the SQUARE,
                                          and that b
                                          vanishes
        Var[X + Y]  = Var[X] + Var[Y]  ONLY IF INDEPENDENT

    STANDARD DEVIATION = sqrt(Var), in the same units as X.

/* WHY THIS MATTERS DIRECTLY - WEIGHT INITIALISATION.

   A linear layer computes  y_i = sum over j of W_ij x_j.
   If the W are independent with variance v and there are
   n_in inputs with variance 1, then

        Var[y] = n_in * v

   To keep the variance at 1 through the layer - so signals
   neither explode nor vanish through depth (chapter 07) -
   you need

        v = 1 / n_in           XAVIER/GLOROT
        v = 2 / n_in           HE (for ReLU, which zeroes
                               half the outputs and so halves
                               the variance)

   THAT IS THE ENTIRE DERIVATION of the initialisation
   scheme every framework uses. It is variance arithmetic,
   nothing more (chapter 15).

/* --- IN CODE --- */
import numpy as np

x = np.random.randn(10000)
x.mean()            # ~0.0
x.var()             # ~1.0
x.std()             # ~1.0

W = np.random.randn(512, 512) * np.sqrt(2.0/512)   # He
h = np.random.randn(1000, 512)
out = h @ W
out.std()           # ~1.0 - the variance was PRESERVED

W_bad = np.random.randn(512, 512)                  # naive
```

```
(h @ W_bad).std()   # ~22.6 - and after ten layers,
                    # 22.6^10 = 1e13. Overflow.
```

The distributions you will meet

```
/* --- BERNOULLI: one binary trial --- */
    P(1) = p,  P(0) = 1-p
    E = p,  Var = p(1-p)
    USE: binary classification output; DROPOUT masks.

/* --- CATEGORICAL: one draw from k outcomes --- */
    P(i) = p_i,  with the p_i summing to 1
    USE: THE OUTPUT OF EVERY LANGUAGE MODEL. Softmax
    produces exactly this over the vocabulary, and sampling
    from it is how text is generated (chapter 25).

    np.random.choice(vocab_size, p=probs)

/* --- BINOMIAL: n independent Bernoulli trials --- */
    E = np,  Var = np(1-p)

/* --- UNIFORM --- */
    Equal density on [a, b]. E = (a+b)/2,
    Var = (b-a)^2 / 12.
    USE: initialisation ranges, random sampling.

/* --- GAUSSIAN (NORMAL): the important one --- */
    p(x) = (1 / sqrt(2 pi sigma^2))
           * exp(-(x - mu)^2 / (2 sigma^2))

    THE 68-95-99.7 RULE: that fraction of mass lies within
    1, 2 and 3 standard deviations.

    WHY IT IS EVERYWHERE:
      · THE CENTRAL LIMIT THEOREM: the sum of many
        independent random contributions tends to a Gaussian,
        WHATEVER their individual distributions. Neural
        network pre-activations are sums of hundreds of
        terms, so they are approximately Gaussian - which is
        why the variance arithmetic above works.
      · maximum entropy for a given mean and variance
        (chapter 10)
      · analytically convenient: sums of Gaussians are
        Gaussian, and the maths stays closed-form

    USE: weight initialisation, noise, VAE latents,
    diffusion models, and the assumption behind
    least-squares loss.

/* --- MULTIVARIATE GAUSSIAN --- */
    Parameterised by a mean VECTOR and a COVARIANCE MATRIX.
    The covariance's eigenvectors are the axes of the
    probability ellipsoid - chapter 05's machinery applied
    directly.

/* --- SOFTMAX AS A DISTRIBUTION-MAKER --- */
    Any vector of real "scores" becomes a categorical
    distribution:

        P(i) = exp(z_i / T) / sum_j exp(z_j / T)

    THE TEMPERATURE T IS THE SAMPLING CONTROL YOU WILL MEET
    IN EVERY LLM API:

        T -> 0     the distribution collapses onto the
```

```
                argmax. DETERMINISTIC, repetitive.
    T = 1       the model's own calibrated distribution.
    T > 1       FLATTENED. More surprising, more
                incoherent.
    T -> inf    uniform. Pure noise.

def softmax_T(z, T=1.0):
    z = z / T
    z = z - z.max()          # stability (chapter 06)
    e = np.exp(z)
    return e / e.sum()

THAT IS THE WHOLE MECHANISM behind the "temperature"
slider. Chapter 25 covers the rest of the sampling
toolkit.                                              */
```

LOG PROBABILITIES, ALWAYS

A 500-token sequence has a probability like 10^{-800}, which is zero in float32 and float64 alike. So everything is computed in **log space**: products become sums, tiny numbers become manageable negatives, and nothing underflows. When you see `log_softmax`, `logsumexp`, or a loss reported in "nats per token", this is why. The rule: *never multiply probabilities; add log probabilities.*

Sampling, and the practical side

```
import numpy as np

# --- SAMPLING FROM A CATEGORICAL DISTRIBUTION ---
probs = np.array([0.1, 0.7, 0.2])
np.random.choice(3, p=probs)

# HOW IT WORKS UNDERNEATH - inverse CDF sampling:
def sample(probs):
    r = np.random.rand()
    cum = 0.0
    for i, p in enumerate(probs):
        cum += p
        if r < cum:
            return i
    return len(probs) - 1
# One uniform random number, one cumulative walk. This is
# what runs in the inner loop of text generation.

# --- THE GUMBEL-MAX TRICK, which appears in real code ---
# Sampling from softmax(z) is equivalent to:
#     argmax_i (z_i + g_i),  g_i ~ Gumbel(0,1)
g = -np.log(-np.log(np.random.rand(len(z))))
sample_idx = np.argmax(z + g)
# NO SOFTMAX NEEDED. Numerically convenient, and the basis
# of the Gumbel-softmax relaxation used when you need a
# DIFFERENTIABLE approximation to sampling.

# --- LOG-SPACE ARITHMETIC ---
def logsumexp(x):
    m = np.max(x)
    return m + np.log(np.sum(np.exp(x - m)))
# The same max-subtraction trick as softmax, and the reason
# it exists: log(sum(exp(x))) computed naively overflows.

def log_softmax(z):
    return z - logsumexp(z)

# SEQUENCE LOG-PROBABILITY - how perplexity is computed:
total_logprob = sum(log_softmax(logits[t])[token[t]]
                    for t in range(T))
avg_nll = -total_logprob / T
perplexity = np.exp(avg_nll)
# PERPLEXITY is "the effective number of tokens the model is
# choosing between". A perplexity of 20 means the model is
# about as uncertain as if it were picking uniformly from 20
# options. Chapter 10 shows why it is exp of the
# cross-entropy.

# --- REPRODUCIBILITY, which you will need ---
np.random.seed(42)
# torch.manual_seed(42)
# AND NOTE: full determinism also needs deterministic GPU
# kernels and fixed data ordering. Seeding alone is not
# enough, and "it changed between runs" is usually this.

# --- ESTIMATING AN EXPECTATION BY SAMPLING ---
# E[f(X)] ~= (1/n) sum f(x_i)
samples = np.random.randn(10000)
np.mean(samples**2)      # ~1.0, since Var = E[X^2] - 0
# MONTE CARLO. The error shrinks as 1/sqrt(n), which is slow
```

```
# - 100x more samples for 10x more accuracy. This is why
# reinforcement learning from sampled rewards (chapter 24)
# is so much noisier than supervised training.
```

How it shows up

Symptom	Cause
Sequence probability is exactly 0.0	Underflow — work in log space
`inf` from `exp`	No max subtraction in softmax or logsumexp
Activations explode through depth	Initialisation variance not scaled by `1/n_in`
Activations vanish through depth	Initialisation variance too small
Generated text is repetitive	Temperature near zero — effectively greedy
Generated text is incoherent	Temperature too high — the distribution is flattened
`probabilities do not sum to 1`	Floating-point drift — renormalise, or sample in log space
Results differ between runs despite a seed	Non-deterministic kernels or data ordering
An RL or sampled estimate is very noisy	Monte Carlo error falls only as 1/√n
Density value greater than 1 looks wrong	Correct — densities are not probabilities

Check yourself

- ✓ State the two rules of a distribution and the difference between mass and density.
- ✓ Write the chain rule of probability and explain what a language model estimates.
- ✓ Give the linearity properties of expectation and say which require independence.
- ✓ Derive the He initialisation scale from variance arithmetic.
- ✓ Name four distributions and where each appears in a model.
- ✓ State the central limit theorem and why it justifies Gaussian assumptions in networks.
- ✓ Write temperature-scaled softmax and describe the behaviour at T→0, 1 and ∞.
- ✓ Explain why everything is done in log space.
- ✓ Write `logsumexp` and say why the max subtraction is needed.
- ✓ Define perplexity in words.
- ✓ Explain the Monte Carlo error rate and its consequence for RL.

Bayes, likelihood and estimation

How to update a belief when evidence arrives, and how to choose the parameters of a model from data. Maximum likelihood is the principle that justifies every loss function in this track.

Bayes' theorem

```
From the definition of conditional probability:

     P(A|B) P(B) = P(A, B) = P(B|A) P(A)

Rearranged:

                  P(B|A) P(A)
     P(A|B)  =  -------------
                     P(B)

THE NAMES, which are worth learning because papers use
them constantly:

     P(A)     PRIOR       what you believed before
     P(B|A)   LIKELIHOOD  how well A explains the
                          evidence B
     P(A|B)   POSTERIOR   what you believe after
     P(B)     EVIDENCE    a normalising constant

AND THE FORM THAT MATTERS IN PRACTICE:

     POSTERIOR  is proportional to  LIKELIHOOD x PRIOR

The denominator only makes it sum to 1, so it is often
ignored until the end.

THE CLASSIC WORKED EXAMPLE - a medical test, because the
result is genuinely surprising:

  disease prevalence          P(D)   = 0.001
  test sensitivity            P(+|D) = 0.99
  false positive rate         P(+|~D)= 0.05

  A patient tests positive. What is P(D|+)?

     P(+) = 0.99 * 0.001  +  0.05 * 0.999
          = 0.00099 + 0.04995 = 0.05094

     P(D|+) = 0.00099 / 0.05094 = 0.0194

  ABOUT 2%. Not 99%.

THE LESSON - and it is the one people get wrong: A RARE
CONDITION MEANS MOST POSITIVES ARE FALSE POSITIVES,
however good the test. The prior dominates.

THE SAME ARITHMETIC APPLIES TO ANY RARE-EVENT CLASSIFIER
YOU BUILD: fault detection on a machine that rarely
fails, intrusion detection, defect screening. A model
with 99% accuracy on a 0.1% base rate is generating
fifty false alarms for every true one, and reporting
"99% accurate" is actively misleading. Report PRECISION
AND RECALL instead (chapter 12).
```

Maximum likelihood

```
/* THE QUESTION: I have data. What parameters make that data
   most probable?

        theta_MLE = argmax  P(data | theta)

/* IN LOG SPACE, always (chapter 08) - products of many
   small numbers underflow, and sums are easier to
   differentiate:

        theta_MLE = argmax  sum_i log P(x_i | theta)

/* AND SINCE OPTIMISERS MINIMISE:

        theta_MLE = argmin  -sum_i log P(x_i | theta)
                             ^^^^^^^^^^^^^^^^^^^^^^^^
                             THE NEGATIVE LOG LIKELIHOOD

   THIS IS THE LOSS FUNCTION. Not an analogy - it is
   literally what "cross-entropy loss" computes
   (chapter 10).

/* --- WORKED: A COIN --- */
    n flips, k heads, parameter p.

        L(p) = p^k (1-p)^(n-k)
        log L = k log p + (n-k) log(1-p)
        d/dp  = k/p - (n-k)/(1-p) = 0
        =>  p = k/n

    The MLE is the observed frequency. Reassuringly obvious -
    but note it derived rigorously from a principle rather
    than being assumed.

/* --- WORKED: A GAUSSIAN, AND WHY MSE EXISTS --- */
    Assume  y_i = f(x_i) + noise,  noise ~ Normal(0, sigma^2)

        log P(y_i | x_i)
            = -(y_i - f(x_i))^2 / (2 sigma^2)
              - log(sigma sqrt(2 pi))

    Maximising this over the parameters of f means
    MINIMISING sum (y_i - f(x_i))^2.

    SO MEAN SQUARED ERROR IS NOT AN ARBITRARY CHOICE. IT IS
    MAXIMUM LIKELIHOOD UNDER AN ASSUMPTION OF GAUSSIAN
    NOISE.

    And that tells you when NOT to use it: if your errors are
    heavy-tailed, MSE's implicit Gaussian assumption makes it
    over-react to outliers. Huber loss or an L1 loss
    corresponds to a heavier-tailed noise model, which is
    exactly why they are more robust.

/* --- WORKED: CLASSIFICATION, AND WHY CROSS-ENTROPY
       EXISTS --- */
    The model outputs P(class | x) via softmax. The
    likelihood of the correct label is that probability, so

        NLL = -sum_i log P(y_i | x_i)
```

```
    which IS cross-entropy loss (chapter 10). One derivation
    produces both of the loss functions you will ever use.

/* --- MAP: MAXIMUM A POSTERIORI --- */
    Include a prior:

        theta_MAP = argmax  P(data|theta) P(theta)
                  = argmin  -log P(data|theta)
                            - log P(theta)
                                    ^^^^^^^^^^^^^
                                    A REGULARISATION TERM

    AND THE TWO STANDARD REGULARISERS FALL OUT DIRECTLY:

      a GAUSSIAN prior on the weights
          -> -log P(theta) proportional to ||theta||^2
          -> L2 REGULARISATION / WEIGHT DECAY

      a LAPLACE prior
          -> proportional to ||theta||_1
          -> L1 REGULARISATION, which drives weights to
             exactly zero (sparsity)

    So weight decay is not a hack. IT IS A STATEMENT THAT
    YOU BELIEVE SMALL WEIGHTS ARE MORE LIKELY A PRIORI
    (chapter 15).                                          */
```

THE SINGLE MOST USEFUL IDEA IN THIS CHAPTER

Every loss function is a negative log likelihood under some assumed noise model, and every regulariser is a negative log prior. If you know what distribution you are assuming, you know which loss to use — and if a loss is behaving strangely, ask what distributional assumption it encodes and whether your data actually satisfies it.

A worked classifier

```
"""NAIVE BAYES - the simplest useful application of Bayes'
   theorem, and still a strong baseline for text
   classification. Worth building once because it makes the
   abstract machinery concrete."""

import numpy as np
from collections import defaultdict

class NaiveBayes:
    """P(class | words) proportional to
       P(class) * product of P(word | class)

       "Naive" = assuming words are INDEPENDENT given the
       class. That assumption is obviously false, and the
       classifier works well anyway - a recurring lesson."""

    def fit(self, docs, labels):
        self.classes = set(labels)
        self.log_prior = {}
        self.log_lik = {}
        vocab = set(w for d in docs for w in d)
        V = len(vocab)

        for c in self.classes:
            c_docs = [d for d, l in zip(docs, labels)
                      if l == c]
            self.log_prior[c] = np.log(len(c_docs) /
                                       len(docs))

            counts = defaultdict(int)
            total = 0
            for d in c_docs:
                for w in d:
                    counts[w] += 1
                    total += 1

            # LAPLACE SMOOTHING: add 1 to every count.
            # WITHOUT IT, a word unseen in this class gives
            # P = 0, and one zero makes the WHOLE PRODUCT
            # zero - a single unseen word vetoes the class.
            # This is the same "never assign zero probability
            # to something possible" principle that appears
            # again in chapter 10.
            self.log_lik[c] = {
                w: np.log((counts[w] + 1) / (total + V))
                for w in vocab
            }
            self.log_lik[c]['<UNK>'] = np.log(1 /
                                              (total + V))

    def predict(self, doc):
        best, best_score = None, -np.inf
        for c in self.classes:
            # SUM OF LOGS, never a product of probabilities
            score = self.log_prior[c]
            for w in doc:
                score += self.log_lik[c].get(
                             w, self.log_lik[c]['<UNK>'])
            if score > best_score:
                best, best_score = c, score
```

```
        return best


docs = [["buy", "now", "cheap"], ["meeting", "at", "three"],
        ["cheap", "offer", "now"], ["project", "meeting"]]
labels = ["spam", "ham", "spam", "ham"]

nb = NaiveBayes()
nb.fit(docs, labels)
nb.predict(["cheap", "meeting"])

/* WHY THIS IS WORTH KNOWING IN 2026:
   · it is a genuine probabilistic model you can read in
     full
   · it trains in one pass and runs in microseconds - which
     makes it a real option on an MCU where a transformer is
     not
   · the smoothing problem it exposes is universal
   · and it is the honest baseline: if a neural network
     cannot beat naive Bayes on your task, the network is
     not the problem, the data is.                      */
```

Uncertainty, and why models are overconfident

```
/* TWO KINDS OF UNCERTAINTY, and the distinction matters:

   ALEATORIC   irreducible randomness in the data itself.
               A coin flip. Sensor noise. MORE DATA DOES NOT
               HELP.
   EPISTEMIC   uncertainty about the model or its
               parameters, from limited data. MORE DATA
               REDUCES IT.

   A model far outside its training distribution has huge
   EPISTEMIC uncertainty - and a standard softmax has NO WAY
   TO EXPRESS IT. It will report 99% confidence on an input
   unlike anything it has ever seen. That is not a bug in
   the training; it is a limitation of the output
   parameterisation.

/* CALIBRATION: does "70% confident" mean right 70% of the
   time?

   Modern neural networks are systematically OVERCONFIDENT.
   The standard measurement is a RELIABILITY DIAGRAM - bin
   predictions by confidence, plot accuracy per bin against
   the bin's confidence, and look for the diagonal.

   THE CHEAP FIX - TEMPERATURE SCALING:
     after training, fit ONE scalar T on a validation set to
     minimise NLL:
         p = softmax(logits / T)
     T > 1 softens overconfident predictions. One parameter,
     no retraining, and it works remarkably well.

/* ESTIMATING EPISTEMIC UNCERTAINTY, in increasing cost:
     MC DROPOUT       keep dropout on at inference, run n
                      forward passes, look at the spread.
                      Cheap and approximate.
     DEEP ENSEMBLES   train k models, compare. Expensive,
                      and the most reliable method in
                      practice.
     BAYESIAN NNs     distributions over weights.
                      Principled, rarely practical at scale.

/* AND FOR LANGUAGE MODELS SPECIFICALLY:
   The token-level probabilities are a poor guide to factual
   correctness. A model can be highly confident in each token
   of a fabricated citation, because each token is a
   plausible continuation of the previous ones - the
   confidence is about FLUENCY, not TRUTH. This is worth
   understanding precisely, because it explains why
   "just check the model's confidence" does not solve
   hallucination.

   WHAT DOES HELP: sampling several answers and checking
   agreement (self-consistency), retrieving evidence and
   requiring citation, and constraining the output format so
   invalid answers are unrepresentable. All engineering, not
   probability.                                          */
```

How it shows up

Symptom	Cause
A 99%-accurate detector floods you with false alarms	Low base rate — most positives are false
Accuracy looks great, the model is useless	Imbalanced classes — report precision and recall
One unseen word forces a class to zero	No smoothing — a zero probability vetoes the product
Probabilities underflow to zero	Multiplying instead of summing logs
MSE dominated by a few outliers	Its Gaussian assumption — use Huber or L1
Model is 99% confident on nonsense input	Softmax cannot express epistemic uncertainty
Confidence does not match accuracy	Miscalibration — apply temperature scaling
Weight decay described as a hack	It is a Gaussian prior — MAP estimation
Confident, fluent, fabricated output	Token confidence measures fluency, not truth

Check yourself

- ✓ State Bayes' theorem and name all four terms.
- ✓ Work the medical-test example and explain why the answer is 2%.
- ✓ Explain what this implies for any rare-event detector you build.
- ✓ Define maximum likelihood and show why it becomes a minimisation.
- ✓ Derive the MLE for a coin.
- ✓ Derive MSE from a Gaussian noise assumption, and say when that assumption fails.
- ✓ Explain how MAP estimation produces L1 and L2 regularisation.
- ✓ Explain Laplace smoothing and the failure it prevents.
- ✓ Distinguish aleatoric and epistemic uncertainty.
- ✓ Describe temperature scaling and what a reliability diagram shows.
- ✓ Explain why token confidence is a poor guide to factual correctness.

Information theory: entropy and cross-entropy

How to measure surprise, and how to measure the gap between two distributions. Cross-entropy is the loss function used to train essentially every model in this track, and it deserves to be understood rather than imported.

Information as surprise

```
THE SELF-INFORMATION of an outcome:

     I(x) = -log P(x)

WHY THAT FORM? Because it is the only one satisfying
three requirements anyone would want:

  · a CERTAIN event carries no information:
       P = 1 -> I = 0
  · a RARER event carries more:
       P smaller -> I larger
  · INDEPENDENT events ADD:
       I(x, y) = I(x) + I(y)
    and log is the function that turns the product
    P(x)P(y) into a sum.

UNITS DEPEND ON THE LOG BASE:
     log2  -> BITS       (the natural unit for storage)
     ln    -> NATS       (the natural unit for maths and
                          for every ML framework)

EXAMPLES:
     a fair coin        -log2(0.5)     = 1 bit
     a fair die         -log2(1/6)     = 2.58 bits
     one of 50,000
       tokens, uniform  -log2(1/50000) = 15.6 bits
     "the" after
       "once upon"      -log2(0.0001)  = 13.3 bits
     "time" after
       "once upon a"    -log2(0.95)    = 0.07 bits

THAT LAST PAIR IS THE WHOLE IDEA OF A LANGUAGE MODEL: a
good model assigns high probability to what actually
comes next, so real text carries LOW surprise under it.
Training minimises average surprise. Nothing more
mysterious than that.

ENTROPY is the AVERAGE surprise of a distribution:

     H(P) = -sum_x P(x) log P(x)  =  E[-log P(x)]

  fair coin           H = 1 bit          (maximum for 2
                                          outcomes)
  biased coin p=0.9   H = 0.469 bits
  certain outcome     H = 0 bits         (no surprise
                                          ever)
  uniform over n      H = log2(n)        (MAXIMUM)

AND THE INTERPRETATION THAT MAKES IT CONCRETE: H(P) IS
THE MINIMUM AVERAGE NUMBER OF BITS NEEDED TO ENCODE
SAMPLES FROM P. That is Shannon's source coding theorem,
and it is why this subject and data compression are the
same subject.

WHICH LEADS DIRECTLY TO A CLAIM WORTH SITTING WITH:
A LANGUAGE MODEL IS A COMPRESSOR. A model that predicts
the next token well can encode text in few bits. The
better the prediction, the better the compression - and
the training objective is literally "minimise the
compressed size of the training data".
```

Cross-entropy and KL divergence

```
/* --- CROSS-ENTROPY: encoding P's data with Q's code --- */

        H(P, Q) = -sum_x P(x) log Q(x)

    P is the TRUE distribution, Q is the MODEL's.

    If Q = P, cross-entropy equals entropy - you cannot do
    better. Any mismatch costs extra bits.

/* --- KL DIVERGENCE: exactly that extra cost --- */

        KL(P || Q) = H(P, Q) - H(P)
                   = sum_x P(x) log( P(x) / Q(x) )

    PROPERTIES:
      KL >= 0, always              (Gibbs' inequality)
      KL = 0 if and only if P = Q
      NOT SYMMETRIC: KL(P||Q) != KL(Q||P)
      NOT a distance - it violates the triangle inequality

    THE ASYMMETRY IS MEANINGFUL AND WORTH KNOWING:

      KL(P || Q)  "FORWARD", MODE-COVERING.
                  Heavily penalises Q(x) being near zero
                  where P(x) is not. Q is forced to spread
                  out and cover everything P does.
                  THIS IS WHAT MAXIMUM LIKELIHOOD MINIMISES
                  (chapter 09) - which is why generative
                  models trained this way produce some
                  low-quality samples: they would rather
                  cover everything than be sharp.

      KL(Q || P)  "REVERSE", MODE-SEEKING.
                  Q can safely ignore parts of P; it just
                  must not put mass where P has none.
                  Produces sharp but narrow results.
                  Used in variational inference and in the
                  RLHF objective (chapter 24).

/* --- WHY CROSS-ENTROPY IS THE LOSS FUNCTION --- */

    For a classification problem the true distribution is
    ONE-HOT: P(correct class) = 1, everything else 0.

    So the sum collapses to a single term:

        H(P, Q) = -log Q(correct class)

    AND THAT IS THE NEGATIVE LOG LIKELIHOOD from chapter 09.
    Cross-entropy loss, negative log likelihood, and
    minimising KL divergence from the data distribution are
    THE SAME OBJECTIVE seen from three angles.

    Also note: since H(P) is fixed by the data,
        minimise cross-entropy  ==  minimise KL(P||Q).
    The entropy of the data is a constant floor you cannot
    get below, which is why a well-trained model's loss
    plateaus at a non-zero value - that floor is the
    irreducible ambiguity of language.                    */
```

Implementing it

```
import numpy as np

def cross_entropy(logits, target_idx):
    """Numerically stable, from raw logits."""
    logits = logits - np.max(logits)          # chapter 06
    log_probs = logits - np.log(np.sum(np.exp(logits)))
    return -log_probs[target_idx]

# EQUIVALENTLY, and this is what frameworks do:
#     loss = logsumexp(logits) - logits[target]
# ONE SUBTRACTION AND ONE STABLE REDUCTION. No softmax, no
# log of a probability, no chance of log(0).

# --- THE GRADIENT, and why it is beautiful ---
#
#   dL/dlogits = softmax(logits) - one_hot(target)
#
# The messy softmax Jacobian (chapter 06) cancels against
# the log in cross-entropy, leaving a SUBTRACTION.
#
#   Interpretation: the gradient is the PREDICTION ERROR.
#   Push down the probability of everything you predicted,
#   push up the probability of what was correct, in
#   proportion to how wrong you were.

def cross_entropy_backward(probs, target_idx):
    grad = probs.copy()
    grad[target_idx] -= 1.0
    return grad

# --- BATCHED, in PyTorch ---
import torch
import torch.nn.functional as F

logits  = torch.randn(32, 50000)     # batch, vocab
targets = torch.randint(0, 50000, (32,))

loss = F.cross_entropy(logits, targets)     # RAW LOGITS

# NEVER:
#   probs = F.softmax(logits, -1)
#   loss = F.nll_loss(torch.log(probs), targets)
# It works, and it is less accurate and slower, and it
# throws away the cancellation above.

# --- LABEL SMOOTHING ---
loss = F.cross_entropy(logits, targets, label_smoothing=0.1)
# Instead of a one-hot target, use 0.9 for the correct class
# and 0.1/(V-1) spread over the rest.
#
# WHY: a one-hot target asks the model to output probability
# exactly 1, which requires an INFINITE logit. The model
# pushes logits ever larger, becomes overconfident
# (chapter 09) and generalises worse. Smoothing gives it a
# finite target to reach. Standard in vision and machine
# translation; used more selectively for LLM pretraining.

# --- PERPLEXITY ---
perplexity = torch.exp(loss)
# The exponential of the cross-entropy in NATS.
```

```
#
# THE INTERPRETATION: "the model is as uncertain as if it
# were choosing uniformly among this many options."
#   perplexity 1      perfect prediction
#   perplexity 20     a good modern LLM on general text
#   perplexity 50000  no better than uniform over the
#                     vocabulary
#
# AND THE CAVEAT THAT INVALIDATES MOST COMPARISONS:
# PERPLEXITY DEPENDS ON THE TOKENISER. A model with a larger
# vocabulary spreads the same text over fewer tokens, so its
# per-token perplexity is not comparable to a model with a
# different vocabulary. Compare BITS PER BYTE instead
# (chapter 16).
```

THE ONE IDENTITY TO REMEMBER

```
minimise cross-entropy
  == maximise likelihood        (chapter 09)
  == minimise KL(data || model)
  == minimise the compressed size of the data
  == minimise the model's average surprise
```

Five descriptions of the same optimisation. Which one you use depends on which makes the current problem obvious.

The other quantities worth knowing

```
/* --- MUTUAL INFORMATION --- */
        I(X; Y) = H(X) - H(X|Y)
                = KL( P(X,Y) || P(X)P(Y) )

    "How many bits does knowing Y save you about X?"
    Zero exactly when X and Y are independent. SYMMETRIC,
    unlike KL.

    WHERE IT APPEARS: feature selection, InfoNCE and
    contrastive learning objectives, analysing what a
    network layer retains about its input, and the
    information-bottleneck view of representation learning.

/* --- CONDITIONAL ENTROPY --- */
        H(X|Y) = H(X, Y) - H(Y)
    "Remaining uncertainty about X once Y is known."
    A LANGUAGE MODEL ESTIMATES exactly
        H(next token | context)
    and reducing it is the whole training objective.

/* --- THE ENTROPY OF ENGLISH --- */
    Shannon estimated ~1.0-1.3 bits per character for
    English text, using human prediction experiments.

    Modern large models achieve roughly 0.6-0.8 BITS PER
    BYTE on general text - BELOW Shannon's human estimate.
    That is a genuinely meaningful benchmark, because it is
    tokeniser-independent and directly comparable across
    models and across decades.

/* --- CHANNEL CAPACITY, for completeness --- */
    Shannon's other great result: a noisy channel has a
    maximum reliable rate, and codes exist that approach it.
    Not directly used in this track, but it is the same
    mathematics, and it is why error-correcting codes and
    machine learning keep borrowing each other's tools.

/* --- ENTROPY AS A DIAGNOSTIC, which is genuinely
       useful --- */

    def output_entropy(probs):
        return -np.sum(probs * np.log(probs + 1e-12))

    · HIGH entropy in the output distribution -> the model is
      uncertain about this token. Often a good place to
      branch, to ask for confirmation, or to spend more
      compute.
    · ENTROPY COLLAPSING TOWARD ZERO during training ->
      overconfidence, or a degenerate solution.
    · ENTROPY OF THE ATTENTION WEIGHTS -> whether a head is
      focused on one position or spread across many
      (chapter 19). A useful interpretability probe.
    · AND FOR DATA: the entropy of your label distribution
      tells you the accuracy a trivial "always predict the
      majority class" model would get. Compare against that,
      not against zero.                                  */
```

How it shows up

Symptom	Cause
`nan` loss	`log(0)` — softmax applied before cross-entropy, or no epsilon
Loss plateaus at a non-zero value	Expected — that floor is the data's own entropy
Loss of exactly `log(vocab_size)` at step 0	Correct — a uniform distribution. A useful sanity check
Loss much higher than `log(V)` at step 0	Initialisation is too large, or labels are misaligned
Model becomes overconfident and generalises worse	One-hot targets push logits to infinity — try label smoothing
Perplexity comparisons disagree with quality	Different tokenisers — use bits per byte
Generative samples are blurry or generic	Forward KL is mode-covering
A model collapses to one output mode	Reverse KL is mode-seeking
Accuracy beats a baseline by little	Compare against the majority-class entropy, not zero

Check yourself

- ✓ Give the three requirements that force `-log P` as the measure of information.
- ✓ Compute the entropy of a fair coin and of a uniform distribution over n outcomes.
- ✓ State the source coding theorem and explain why a language model is a compressor.
- ✓ Define cross-entropy and KL divergence, and state the relationship.
- ✓ Explain the difference between forward and reverse KL and where each is used.
- ✓ Show why cross-entropy collapses to negative log likelihood for a one-hot target.
- ✓ Write a numerically stable cross-entropy from logits.
- ✓ State the gradient of cross-entropy with respect to logits and interpret it.
- ✓ Explain label smoothing and the problem it solves.
- ✓ Define perplexity and say why it is not comparable across tokenisers.
- ✓ Give three diagnostic uses of entropy.

Gradient descent and its variants

The loop that actually trains everything. Ten lines of arithmetic, a handful of hyperparameters, and more practical folklore than any other part of the subject.

The algorithm

```
REPEAT:
   1. compute the loss on some data
   2. compute the gradient of the loss w.r.t. every
      parameter        (chapter 07)
   3. step every parameter AGAINST its gradient

     theta  <-  theta - lr * grad

THAT IS ALL. Everything below is about doing step 3
better.

THE THREE FLAVOURS, by how much data step 1 uses:

  BATCH (full)      the whole dataset per step.
                    Accurate gradient, impossibly slow,
                    and it gets stuck in whatever basin it
                    starts in.
  STOCHASTIC (SGD)  one sample per step.
                    Noisy, fast per step, and the noise
                    helps escape bad regions.
  MINI-BATCH        32-2048 samples. WHAT EVERYONE
                    ACTUALLY USES: a good gradient
                    estimate, and it saturates the
                    hardware's parallelism.

THE LEARNING RATE IS THE MOST IMPORTANT HYPERPARAMETER
IN THE FIELD:

  TOO SMALL     training crawls; may never arrive
  TOO LARGE     the loss oscillates, or diverges to nan
  JUST RIGHT    steady decrease

  loss                    loss                 loss
   |\                      |  /\  /\            |\
   | \____                 | /  \/  \           | \
   |      \____            |/         \         |  \___
   +---------- good        +---------- too     +------
                                       large     too small
                                       (bouncing) (crawling)

AND THE UNCOMFORTABLE TRUTH: THERE IS NO WAY TO KNOW THE
RIGHT VALUE WITHOUT TRYING. The standard method is an
LR RANGE TEST - increase the learning rate exponentially
over a few hundred steps, plot the loss, and pick roughly
an order of magnitude below where it starts rising.
```

Momentum and adaptivity

```
/* --- PLAIN SGD --- */
    theta -= lr * g

    Problems: it oscillates across narrow valleys, crawls
    along flat directions, and treats every parameter
    identically regardless of how large its gradients
    usually are.

/* --- MOMENTUM --- */
    v = beta * v + g              /* beta typically 0.9 */
    theta -= lr * v

    An exponentially weighted average of past gradients.
    THE PHYSICAL PICTURE: a ball rolling downhill has
    inertia. It builds speed in consistent directions and
    averages away oscillation across the valley.

    beta = 0.9 means the effective average covers roughly the
    last 1/(1-beta) = 10 gradients.

/* --- NESTEROV MOMENTUM --- */
    Evaluate the gradient at where momentum is ABOUT to take
    you, rather than where you are. A "look-ahead" correction
    that slightly improves convergence.

/* --- ADAGRAD: per-parameter learning rates --- */
    s += g^2                       /* elementwise */
    theta -= lr * g / (sqrt(s) + eps)

    Parameters with historically large gradients get smaller
    steps. Good for sparse features.
    FATAL FLAW: s only grows, so the effective learning rate
    decays monotonically to zero and training stops.

/* --- RMSPROP: fix the decay --- */
    s = beta2 * s + (1 - beta2) * g^2
    theta -= lr * g / (sqrt(s) + eps)

    An exponential average instead of a running sum, so it
    adapts to the RECENT gradient scale and never dies.

/* --- ADAM: momentum + RMSProp. THE DEFAULT. --- */
    m = beta1 * m + (1 - beta1) * g          /* 1st moment */
    v = beta2 * v + (1 - beta2) * g**2       /* 2nd moment */

    m_hat = m / (1 - beta1**t)               /* BIAS
                                                CORRECTION */
    v_hat = v / (1 - beta2**t)

    theta -= lr * m_hat / (sqrt(v_hat) + eps)

    DEFAULTS: beta1 = 0.9, beta2 = 0.999, eps = 1e-8.
    For transformers, beta2 = 0.95 is common and more
    stable.

    WHY THE BIAS CORRECTION: m and v start at zero, so early
    estimates are biased toward zero. Dividing by
    (1 - beta^t) - which is small early and approaches 1 -
    removes exactly that bias. Without it the first hundred
    steps take almost no movement.
```

```
    THE MEMORY COST: TWO EXTRA FULL-SIZE STATE TENSORS.
    Training a 7-billion-parameter model in fp32 needs
        28 GB weights + 28 GB gradients + 56 GB Adam state
        = 112 GB
    before activations. THIS IS WHY OPTIMISER STATE
    SHARDING (ZeRO, FSDP) exists, and why 8-bit optimisers
    are used.

/* --- ADAMW: use this one --- */
    theta -= lr * (m_hat / (sqrt(v_hat) + eps)
                   + weight_decay * theta)
                   ^^^^^^^^^^^^^^^^^^^^^^^
                   DECOUPLED from the adaptive scaling

    In plain Adam, L2 regularisation added to the gradient
    gets divided by sqrt(v) like everything else - so
    parameters with large gradients receive LESS
    regularisation, which is backwards. AdamW applies decay
    directly to the weights.

    ADAMW IS THE STANDARD FOR TRANSFORMERS. If you are
    choosing an optimiser and have no specific reason
    otherwise, use AdamW with lr around 3e-4 for a small
    model and weight_decay = 0.1.

/* --- THE NEWER ONES, briefly --- */
    LION       sign-based updates, less memory (one state
               tensor), competitive results
    SHAMPOO / SOAP   approximate second-order preconditioning
    ADAFACTOR  factorises the second-moment matrix to save
               memory; used for very large models
    MUON       orthogonalised momentum, strong recent results
               on transformer training

   None of these has displaced AdamW as the safe default.  */
```

Learning rate schedules

```
/* A CONSTANT LEARNING RATE IS ALMOST NEVER OPTIMAL. Large
   early to make progress, small later to settle.        */

import numpy as np

# --- WARMUP: essential for transformers ---
def warmup_cosine(step, warmup=2000, total=100000,
                  peak=3e-4, final=3e-5):
    if step < warmup:
        return peak * step / warmup          # LINEAR RAMP
    p = (step - warmup) / (total - warmup)
    return final + 0.5 * (peak - final) * (1 +
                                        np.cos(np.pi * p))

/* WHY WARMUP IS NOT OPTIONAL FOR ADAM + TRANSFORMERS:
   at step 1, Adam's second-moment estimate is based on a
   single gradient and is unreliable, so the adaptive
   denominator is meaningless and the first few steps can be
   enormous. That destabilises LayerNorm statistics and
   attention patterns before they have formed. Ramping from
   zero over 1-2k steps avoids it. Skipping warmup is one of
   the most common causes of "my transformer will not train".

/* THE SCHEDULES YOU WILL MEET:
     COSINE          smooth decay to near zero. THE DEFAULT
                     for LLM pretraining.
     LINEAR DECAY    simple, works well for fine-tuning
     STEP            drop by 10x at fixed milestones.
                     Classic vision training.
     ONE-CYCLE       up then down, with high peak LR.
                     Fast convergence for smaller models.
     CONSTANT + COOLDOWN   hold, then decay at the end.
                     Convenient when the total number of
                     steps is not known in advance, which is
                     increasingly common.

/* --- GRADIENT CLIPPING: not optional either --- */
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    If the total gradient norm exceeds the threshold, scale
    the WHOLE gradient down to that norm - preserving its
    direction. A single bad batch can otherwise produce a
    gradient large enough to destroy the model in one step,
    and recovery may take thousands of steps or never
    happen.

/* --- THE COMPLETE TRAINING LOOP --- */
import torch

model = MyModel()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4,
                        weight_decay=0.1,
                        betas=(0.9, 0.95))

for step, (x, y) in enumerate(dataloader):
    lr = warmup_cosine(step)
    for g in opt.param_groups:
        g['lr'] = lr

    logits = model(x)
```

```
    loss = F.cross_entropy(logits.view(-1, V), y.view(-1))

    opt.zero_grad(set_to_none=True)   # <<< chapter 07
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()

    if step % 100 == 0:
        print(f"{step} loss {loss.item():.4f} lr {lr:.2e}")

/* THAT LOOP TRAINS ANY MODEL IN THIS TRACK, INCLUDING THE
   TRANSFORMER IN CHAPTER 28. The architecture changes; this
   does not.

/* --- GRADIENT ACCUMULATION, when the batch will not fit --- */
    for i, (x, y) in enumerate(dataloader):
        loss = compute_loss(x, y) / accum_steps
        loss.backward()                    # ACCUMULATES
        if (i + 1) % accum_steps == 0:
            clip_grad_norm_(...)
            opt.step()
            opt.zero_grad(set_to_none=True)

    Effective batch size = batch_size * accum_steps.
    NOTE THE DIVISION BY accum_steps - without it the
    effective learning rate is multiplied by accum_steps.  */
```

DEBUGGING A TRAINING RUN, IN ORDER

```
1. OVERFIT ONE BATCH. Take 8 samples, turn off
   regularisation, and train until the loss is ~0. If it
   cannot, the bug is in the model or the data
   pipeline, not the optimiser. Do this FIRST, always.
2. Check the step-0 loss against the theoretical value
   (log V for a V-way classifier - chapter 10).
3. Print gradient norms per layer (chapter 07).
4. Run an LR range test.
5. Only then tune anything else.
```

What the loss landscape is actually like

```
/* THE PICTURE IN TEXTBOOKS - a smooth bowl with one minimum
   and some traps - is misleading for high-dimensional
   networks.

/* --- LOCAL MINIMA ARE NOT THE PROBLEM --- */
    In n dimensions, a local minimum requires ALL n
    eigenvalues of the Hessian to be positive. For large n
    that is astronomically unlikely at a random critical
    point. SADDLE POINTS - some directions up, some down -
    dominate instead.

    And saddles are escapable: gradient descent with any
    noise eventually finds a descending direction. This is
    part of why SGD's noise is a feature, and why the
    "getting stuck in a local minimum" worry from classical
    optimisation largely does not apply here.

/* --- MOST MINIMA ARE EQUIVALENT --- */
    Empirically, the many minima that gradient descent finds
    in a large network have similar loss. You are not
    searching for THE optimum; you are finding one of
    enormously many adequate ones.

/* --- FLAT MINIMA GENERALISE BETTER --- */
    A wide, flat basin means small parameter perturbations
    barely change the loss - so the solution is robust, and
    it tends to transfer to unseen data. Sharp minima do not.

    THIS EXPLAINS SEVERAL PRACTICES:
      · SMALL BATCHES generalise better than very large ones:
        their gradient noise cannot settle into a narrow
        basin
      · SAM (sharpness-aware minimisation) explicitly seeks
        flat regions
      · weight averaging (SWA, EMA of weights) lands in the
        middle of a basin rather than on its wall, and is
        nearly free

/* --- THE PRACTICAL CONSEQUENCES --- */
    · noise HELPS. Do not try to eliminate it.
    · the batch size and learning rate are COUPLED. The
      common heuristic: scale lr with the square root of the
      batch size for Adam, linearly for SGD.
    · a "bad run" is often just an unlucky initialisation or
      data order. Try a different seed before redesigning.
    · AND WATCH FOR LOSS SPIKES in large-model training.
      They are common, usually recoverable, and the standard
      response is to roll back to a checkpoint and skip the
      offending data batch - which is a real operational
      procedure, not a joke.                              */
```

How it shows up

Symptom	Cause
Loss is `nan` after a few steps	Learning rate too high, no clipping, or no warmup
Loss oscillates without decreasing	Learning rate too high

Symptom	Cause
Loss barely moves	Learning rate too low, or gradients not flowing
Transformer will not train at all	Missing warmup with Adam
Training is unstable early, fine later	Adam's second-moment estimate is unreliable at first — warm up
Cannot overfit 8 samples	A bug in the model or data pipeline, not the optimiser
Out of memory with Adam but not SGD	Two extra state tensors per parameter
Weight decay has no effect	Plain Adam, not AdamW — decay was scaled away
Effective learning rate too high with accumulation	Loss not divided by `accum_steps`
Large-batch training generalises worse	Sharp minima — reduce batch or add noise
A sudden loss spike mid-run	Common at scale — roll back and skip the batch

Check yourself

- ✓ Write the gradient descent update and the three batching flavours.
- ✓ Describe the loss curve for a learning rate that is too high, too low and correct.
- ✓ Explain momentum and what `beta = 0.9` means in terms of averaging.
- ✓ Write Adam from memory, including bias correction, and say why it is needed.
- ✓ Compute the memory cost of AdamW for a 7B model in fp32.
- ✓ Explain what AdamW decouples and why plain Adam's L2 is backwards.
- ✓ Explain why transformers need learning-rate warmup.
- ✓ Explain gradient clipping and what it preserves.
- ✓ Write the complete training loop from memory.
- ✓ Give the five debugging steps in order, starting with overfitting one batch.
- ✓ Explain why saddle points, not local minima, dominate high-dimensional landscapes.
- ✓ Say why flat minima generalise better and name two practices that exploit it.

What machine learning is

Fitting a function to data instead of writing it. That is the whole idea — and knowing when *not* to do it is as valuable as knowing how.

The shift

```
TRADITIONAL PROGRAMMING
     rules + data  ->  [program]  ->  answers

MACHINE LEARNING
     data + answers -> [training] ->  RULES
     then:  new data + rules      ->  answers

YOU SPECIFY WHAT, NOT HOW.

THE FORMAL SETUP:
  · a HYPOTHESIS SPACE - the set of functions the model
    can represent, determined by its architecture and
    parameters
  · a LOSS FUNCTION - how wrong a given function is on
    the data (chapters 09, 10)
  · an OPTIMISER - a search procedure over the hypothesis
    space (chapter 11)

TRAINING = SEARCHING FOR THE FUNCTION IN THAT SPACE THAT
MINIMISES THE LOSS.

THE THREE CLASSICAL SETTINGS:

SUPERVISED       inputs WITH labels.
                 classification (discrete labels),
                 regression (continuous).
                 The overwhelming majority of deployed
                 systems.

UNSUPERVISED     inputs, no labels. Clustering,
                 dimensionality reduction (chapter 05),
                 density estimation.

REINFORCEMENT    an agent acting in an environment,
                 receiving rewards. Games, robotics, and
                 RLHF (chapter 24).

AND THE ONE THAT MATTERS MOST NOW:

SELF-SUPERVISED  labels DERIVED FROM THE DATA ITSELF.
                 "Predict the next token" needs no
                 annotation - the next token IS the
                 label, and every document is millions of
                 training examples.

                 THIS IS WHY LARGE LANGUAGE MODELS WERE
                 POSSIBLE. Labelled data is scarce and
                 expensive; raw text is essentially free
                 and effectively unlimited. The entire
                 scaling story (chapter 23) rests on
                 removing the labelling bottleneck.
```

Generalisation: the actual problem

```
/* MEMORISING THE TRAINING DATA IS TRIVIAL. A lookup table
   achieves zero training error. IT IS ALSO USELESS.

   THE GOAL IS PERFORMANCE ON DATA YOU HAVE NEVER SEEN.

/* --- THE SPLIT, and it is non-negotiable --- */
    TRAIN        60-80%   the model fits this
    VALIDATION   10-20%   you choose hyperparameters using
                          this
    TEST         10-20%   YOU LOOK AT THIS ONCE, AT THE END

    IF YOU TUNE ON THE TEST SET, YOU HAVE NO TEST SET. Each
    decision you make based on test performance leaks a
    little information, and after fifty experiments your
    test score is an optimistic fiction. This is the most
    common methodological failure in applied work, and it is
    invisible until deployment.

/* --- UNDERFITTING AND OVERFITTING --- */

    UNDERFIT       high train error, high test error.
                   The model is too simple, or undertrained.
    GOOD FIT       low train error, low test error.
    OVERFIT        LOW train error, HIGH test error.
                   The model memorised noise.

      error
        |  \                    /   test
        |   \                  /
        |    \        ________/
        |     \______/
        |      \_______________ train
        +---------------------------> model capacity
                    ^
                the sweet spot

/* --- BIAS AND VARIANCE --- */
    BIAS       error from wrong assumptions. A linear model
               fitting a curve. -> UNDERFITTING.
    VARIANCE   sensitivity to the particular training sample.
               -> OVERFITTING.

    Classically these trade off. IN DEEP LEARNING THEY
    LARGELY DO NOT, and this is worth knowing because it
    contradicts the textbooks:

/* --- DOUBLE DESCENT --- */
      test
      error |    /\
            |   /  \        <- classical U-curve
            |  /    \
            | /      \    /-------____
            |/        \__/            ---____
            +-------------------------------------->
                       ^         model capacity
                  INTERPOLATION THRESHOLD
                  (enough capacity to fit the training data
                   exactly)

    Past the point where a model can fit the training data
```

```
    perfectly, test error DROPS AGAIN - often below the
    classical minimum. Bigger models generalise BETTER, not
    worse, which is the opposite of what classical
    statistical learning theory predicts.

    NOBODY HAS A COMPLETE EXPLANATION. The leading intuition
    involves implicit regularisation from SGD and the
    flat-minima argument (chapter 11). This is one of
    several places where the practice ran ahead of the
    theory and the theory has not caught up.

/* --- WHAT ACTUALLY CONTROLS OVERFITTING (chapter 15) --- */
      MORE DATA           the most effective by a wide
                          margin
      data AUGMENTATION   more data, cheaply
      REGULARISATION      weight decay, dropout
      EARLY STOPPING      stop when validation loss rises
      a SMALLER model     the classical answer, and often
                          the wrong one now               */
```

Measuring it properly

```
/* ACCURACY IS USUALLY THE WRONG METRIC, for the reason in
   chapter 09: with a 0.1% positive rate, a model that always
   says "no" is 99.9% accurate and worthless.             */

    THE CONFUSION MATRIX:
                      PREDICTED
                    pos       neg
        ACTUAL pos   TP        FN     (missed)
               neg   FP        TN
                    (false alarm)

    PRECISION = TP / (TP + FP)
        "Of the ones I flagged, how many were real?"
        Matters when a FALSE ALARM IS EXPENSIVE - shutting
        down a production line, blocking a legitimate
        transaction.

    RECALL    = TP / (TP + FN)
        "Of the real ones, how many did I catch?"
        Matters when a MISS IS EXPENSIVE - a missed fault, a
        missed diagnosis.

    F1 = 2 * P * R / (P + R)      the harmonic mean

    ROC-AUC       ranking quality across all thresholds
    PR-AUC        BETTER FOR IMBALANCED DATA than ROC-AUC,
                  which looks deceptively good when
                  negatives dominate

/* THE PRECISION-RECALL TRADE-OFF IS A THRESHOLD CHOICE, not
   a model property. One model gives a whole curve; where you
   sit on it is a PRODUCT decision, and it should be made
   with whoever owns the consequences.

/* --- REGRESSION METRICS --- */
    MSE / RMSE    penalises large errors quadratically;
                  sensitive to outliers (chapter 09)
    MAE           linear penalty, robust
    R^2           fraction of variance explained
    MAPE          percentage error - and it EXPLODES when
                  true values are near zero

/* --- CROSS-VALIDATION, when data is scarce --- */
    Split into k folds; train k times, each time holding out
    one fold. Average the results.
    k = 5 or 10 is standard.

    AND THE TRAPS:
      · TIME SERIES: never shuffle. Use forward-chaining
        splits, or you train on the future and predict the
        past.
      · GROUPED DATA: if the same patient, device or user
        appears in both train and test, you are measuring
        memorisation. Split by GROUP.
      · IMBALANCED CLASSES: stratify, so each fold has the
        same class ratio.

/* --- DATA LEAKAGE: the failure that ruins projects --- */
    Information in the training data that will not exist at
    prediction time. The model learns it, scores brilliantly
```

```
    in evaluation, and fails completely in production.

    REAL EXAMPLES:
      · a feature computed AFTER the outcome (an "account
        closed" flag when predicting churn)
      · normalising using statistics from the FULL dataset
        before splitting — the test set's mean has leaked
        into training
      · duplicate records across the train/test boundary
      · an ID that correlates with the label because of how
        the data was collected
      · IN LLM EVALUATION: the benchmark's test set is in
        the pretraining corpus. This is now a serious and
        widespread measurement problem, and it is why
        contamination checks and held-out private evals
        matter.

    THE SMELL TEST: IF YOUR RESULTS LOOK TOO GOOD, THEY ARE.
    Go looking for the leak; you will usually find it.    */
```

WHEN NOT TO USE MACHINE LEARNING

```
· THE RULES ARE KNOWN. Write them. A deterministic
  algorithm is faster, testable, explainable, and does
  not degrade silently.
· YOU LACK DATA. A few hundred examples rarely
  supports a learned model.
· ERRORS ARE UNACCEPTABLE. ML systems are wrong
  sometimes, by construction.
· YOU NEED AN EXPLANATION. Regulated decisions often
  require a reason, not a probability.
· THE DISTRIBUTION SHIFTS CONSTANTLY. A model trained
  on last month's data may be worse than a simple
  heuristic on this month's.
· A SIMPLE BASELINE ALREADY WORKS. Threshold, moving
  average, lookup table, linear model.
```

Always build the trivial baseline first — majority class, last value, a hand-tuned threshold. If your model does not clearly beat it, it does not justify its complexity, its latency, or its maintenance cost. This is the single most useful habit in applied machine learning.

The real workflow

```
/* THE PROPORTIONS SURPRISE PEOPLE:

     define the problem and the metric      5%
     get and clean the data                40%     <<<
     feature engineering / preprocessing   20%
     model selection and training          10%
     evaluation and error analysis         15%
     deployment and monitoring             10%

   THE MODEL IS THE SMALL PART. Anyone telling you
   otherwise has not shipped one.

/* --- 1. DEFINE THE PROBLEM AND THE METRIC FIRST --- */
    What decision does this output feed?
    What does a false positive cost? A false negative?
    What performance would make this worth deploying?
    AGREE THE METRIC BEFORE YOU START, or you will pick the
    metric that flatters the result you got.

/* --- 2. DATA --- */
    Is it representative of production?
    How is it labelled, and how good are the labels? (Label
      noise sets a ceiling on achievable accuracy.)
    Is it balanced? Biased? Does it encode a historical
      decision you should not replicate?
    HOW MUCH IS THERE? This determines what is possible more
      than any architectural choice.

/* --- 3. BASELINE --- */
    The trivial one. Then a simple model - logistic
    regression, gradient-boosted trees. FOR TABULAR DATA,
    GRADIENT BOOSTING (XGBoost, LightGBM) STILL BEATS NEURAL
    NETWORKS MOST OF THE TIME. Deep learning wins on
    perceptual and sequential data - images, audio, text -
    not on spreadsheets.

/* --- 4. ITERATE ON ERRORS, NOT ON ARCHITECTURE --- */
    LOOK AT THE MISTAKES. Sort by loss, read the worst
    hundred examples by hand. You will find mislabelled
    data, a preprocessing bug, or a systematic gap - and any
    of those is worth more than a hyperparameter sweep.

    This is the highest-value hour in any ML project and it
    is the one most often skipped.

/* --- 5. DEPLOY AND MONITOR --- */
    LATENCY, memory, throughput - the embedded constraints
      (chapter 27)
    INPUT DISTRIBUTION MONITORING: is production data
      drifting from training data?
    OUTPUT MONITORING: is the prediction distribution
      changing?
    A FALLBACK PATH for when the model is unavailable or
      obviously wrong
    AND A RETRAINING PLAN, because the world moves.

    MODELS DEGRADE SILENTLY. Unlike a crashed service,
    a model that has become wrong keeps returning confident
    answers. YOU WILL NOT NOTICE WITHOUT MONITORING.      */
```

How it shows up

Symptom	Cause
99% accuracy, useless in production	Imbalanced classes — use precision and recall
Excellent evaluation, poor deployment	Data leakage, or a train/production distribution gap
Test score drifts up over a project	Tuning on the test set — it is no longer held out
Perfect training loss, poor validation	Overfitting — more data or more regularisation
Both losses high	Underfitting — more capacity or longer training
Time-series model looks perfect	Shuffled split — the model saw the future
Great cross-validation, bad on new users	Same group in train and test — split by group
Benchmark scores far above real-world quality	Test-set contamination in the training corpus
Bigger model generalises better than theory predicts	Double descent — expected past the interpolation threshold
Model quality decays over months	Distribution drift with no monitoring
Neural network loses to gradient boosting	Tabular data — that is the normal result

Check yourself

- ✓ Contrast traditional programming and machine learning in one line each.
- ✓ Name the three components of a learning setup.
- ✓ Explain self-supervision and why it enabled large language models.
- ✓ Explain the three-way split and why tuning on the test set destroys it.
- ✓ Define underfitting, overfitting, bias and variance.
- ✓ Describe double descent and say why it contradicts classical theory.
- ✓ Define precision and recall and give a situation where each dominates.
- ✓ Give four concrete examples of data leakage.
- ✓ List six situations where machine learning is the wrong tool.
- ✓ State the workflow proportions and which step is most often skipped.
- ✓ Say why models degrade silently and what monitoring must cover.

Linear and logistic regression

The two simplest useful models, built end to end. Everything in the rest of this track is a variation on what happens here — and a single logistic regression unit *is* a neuron.

Linear regression

```
THE MODEL:
     y = w1 x1 + w2 x2 + ... + wn xn + b
       = w · x + b

ONE DOT PRODUCT (chapter 02) plus an offset. That is the
entire hypothesis space: all the hyperplanes.

THE LOSS - mean squared error, which chapter 09 derived
from a Gaussian noise assumption:

     L = (1/N) sum_i (y_i - y_hat_i)^2

THE GRADIENTS, derived with the chain rule (chapter 07):

     dL/dw = -(2/N) sum_i (y_i - y_hat_i) x_i
     dL/db = -(2/N) sum_i (y_i - y_hat_i)

IN MATRIX FORM, which is what you implement:

     y_hat = X @ w + b            X: (N, d)
     err   = y_hat - y
     dw    = (2/N) * X.T @ err
     db    = (2/N) * err.sum()

AND THERE IS A CLOSED-FORM SOLUTION - the NORMAL
EQUATIONS:

     w = (X^T X)^-1 X^T y

WHEN TO USE IT: small d (up to a few thousand features),
when you want the exact answer in one step.
WHEN NOT TO: large d - inverting a (d, d) matrix is
O(d^3); when X^T X is singular or ill-conditioned
(chapter 03); when the data does not fit in memory. And
in practice, use np.linalg.lstsq rather than forming the
inverse yourself.

GRADIENT DESCENT SCALES WHERE THE CLOSED FORM DOES NOT,
which is why every model past this chapter uses it.
```

```
import numpy as np

class LinearRegression:
    def fit(self, X, y, lr=0.01, epochs=1000):
        N, d = X.shape
        self.w = np.zeros(d)
        self.b = 0.0
        self.history = []

        for epoch in range(epochs):
            y_hat = X @ self.w + self.b
            err = y_hat - y
            loss = np.mean(err**2)
            self.history.append(loss)

            self.w -= lr * (2/N) * (X.T @ err)
            self.b -= lr * (2/N) * err.sum()
        return self

    def predict(self, X):
        return X @ self.w + self.b


# --- FEATURE SCALING IS NOT OPTIONAL ---
# If x1 is in [0, 1] and x2 is in [0, 100000], the loss
# surface is an extremely elongated valley. Gradient descent
# zig-zags across the narrow direction and crawls along the
# long one - the conditioning problem from chapter 03,
# appearing as a training problem.
X_scaled = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-8)

# COMPUTE THE STATISTICS ON THE TRAINING SET ONLY, and apply
# them to validation and test. Using the full dataset's mean
# is data leakage (chapter 12).

# --- POLYNOMIAL FEATURES: linear in the PARAMETERS ---
X_poly = np.column_stack([x, x**2, x**3])
# The model is still "linear regression" - the linearity that
# matters is in w, not in x. This is how a linear model fits
# a curve, and it is the same trick kernel methods
# generalise.

# --- REGULARISED VARIANTS (chapter 09's MAP view) ---
# RIDGE (L2):  loss + alpha * ||w||^2
#     shrinks all weights; handles correlated features
#     gracefully; has a closed form:
#         w = (X.T @ X + alpha*I)^-1 @ X.T @ y
#     Note the +alpha*I: it also FIXES SINGULARITY, which is
#     why ridge is numerically better behaved than plain
#     least squares.
#
# LASSO (L1):  loss + alpha * ||w||_1
#     drives weights to EXACTLY zero -> automatic feature
#     selection. No closed form; needs iterative solvers.
#
# ELASTIC NET: both.
```

Logistic regression

```
/* CLASSIFICATION, NOT REGRESSION - the name is historical
   and unhelpful.

   THE PROBLEM WITH USING LINEAR REGRESSION FOR
   CLASSIFICATION: it outputs unbounded values. What does
   -3.7 mean when the label is 0 or 1?

   THE FIX: SQUASH IT THROUGH A SIGMOID (chapter 06).

        z = w · x + b                  the LOGIT
        p = sigmoid(z) = 1/(1 + e^-z)  a PROBABILITY in (0,1)

   THE LOSS - binary cross-entropy, which chapter 10 derived
   as the negative log likelihood:

        L = -(1/N) sum_i [ y_i log(p_i)
                         + (1-y_i) log(1-p_i) ]

   Read it: when y = 1 only the first term survives, so the
   loss is -log(p) - pushing p up. When y = 0 only the
   second survives. It is exactly "the negative log
   probability of the correct answer".

/* AND THE GRADIENT, WHICH IS THE SAME AS LINEAR
   REGRESSION'S: */

        dL/dw = (1/N) X.T @ (p - y)
        dL/db = (1/N) sum(p - y)

   THE SIGMOID DERIVATIVE CANCELS AGAINST THE LOG IN THE
   LOSS, leaving PREDICTION MINUS TARGET - the same clean
   form as softmax + cross-entropy (chapter 06). This
   cancellation is not a coincidence: it happens for every
   member of the exponential family paired with its natural
   loss.

/* WHY YOU MUST NOT USE MSE FOR CLASSIFICATION:
     with MSE the gradient carries a factor of
     sigmoid'(z) = p(1-p), which is nearly ZERO when the
     model is confidently WRONG (p near 0 with y = 1). The
     model gets almost no gradient exactly when it most needs
     one. Cross-entropy has no such factor - a confidently
     wrong prediction produces a LARGE gradient. This is one
     of the clearest cases where the right loss function is
     not a matter of taste.                              */

import numpy as np

def sigmoid(z):
    # STABLE: exp of a large positive number overflows
    return np.where(z >= 0,
                    1 / (1 + np.exp(-z)),
                    np.exp(z) / (1 + np.exp(z)))

class LogisticRegression:
    def fit(self, X, y, lr=0.1, epochs=1000, l2=0.0):
        N, d = X.shape
        self.w = np.zeros(d)
        self.b = 0.0
        for _ in range(epochs):
```

```
            p = sigmoid(X @ self.w + self.b)
            grad = p - y                        # <<< that is
                                                #     all of it
            self.w -= lr * ((X.T @ grad) / N + l2 * self.w)
            self.b -= lr * grad.mean()
        return self

    def predict_proba(self, X):
        return sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

/* THE THRESHOLD IS A SEPARATE DECISION FROM THE MODEL. 0.5
   is a default, not a law. Move it to trade precision
   against recall according to what a false positive costs
   (chapter 12).

/* --- MULTI-CLASS: SOFTMAX REGRESSION --- */
    W: (d, K) instead of (d,). Softmax instead of sigmoid.
    Categorical cross-entropy instead of binary.

        logits = X @ W + b          # (N, K)
        probs  = softmax(logits)
        grad   = probs - one_hot(y) # (N, K) - SAME FORM
        dW     = X.T @ grad / N

    AND NOTE WHAT THIS IS: a single linear layer followed by
    softmax. IT IS THE OUTPUT HEAD OF EVERY CLASSIFIER IN
    THIS TRACK, including a language model's projection to
    the vocabulary (chapter 20). You have now written it.  */
```

A LOGISTIC REGRESSION UNIT IS A NEURON

```
weighted sum -> non-linearity -> output
   w · x + b  ->   sigmoid     ->   p
```

Stack many of these in parallel and you have a layer. Stack layers and you have a neural network (chapter 14). The forward pass, the loss, the gradient and the update loop you just wrote do not change — only the number of layers between input and output. That is genuinely the whole conceptual step from here to a transformer.

Interpretation, and the limits

```
/* --- WHAT THE COEFFICIENTS MEAN --- */
    LINEAR:   w_j is the change in y for a one-unit change in
              x_j, holding the others fixed.
    LOGISTIC: w_j is the change in the LOG-ODDS. exp(w_j) is
              the ODDS RATIO - "this feature multiplies the
              odds by exp(w_j)".

    THIS INTERPRETABILITY IS THE MAIN REASON THESE MODELS
    ARE STILL EVERYWHERE in medicine, finance and any
    regulated domain. You can hand a coefficient table to a
    domain expert and they can tell you whether it is
    sensible. You cannot do that with a transformer.

    THE CAVEATS:
      · coefficients are only comparable if the features are
        SCALED
      · CORRELATED FEATURES split their weight
        unpredictably; the individual values become
        meaningless even though predictions are fine
      · AND THE BIG ONE: CORRELATION IS NOT CAUSATION. A
        positive coefficient does not mean intervening on
        that feature will change the outcome. Models trained
        on observational data cannot answer interventional
        questions, and treating them as though they can has
        caused real harm.

/* --- WHAT LINEAR MODELS CANNOT DO --- */
    XOR:
        x1 x2 | y
         0  0 | 0
         0  1 | 1
         1  0 | 1
         1  1 | 0

    NO LINE SEPARATES THESE POINTS. This is Minsky and
    Papert's 1969 result, and it stalled neural network
    research for over a decade.

    THE RESOLUTIONS:
      · engineer a feature: add x1*x2 and it becomes
        separable
      · OR LEARN THE FEATURES - which is exactly what a
        hidden layer does, and why chapter 14 exists.

    The historical lesson worth carrying: the limitation was
    real, the conclusion drawn from it was wrong, and the fix
    (a hidden layer plus a way to train it) had to wait for
    backpropagation to become widely understood.

/* --- WHEN A LINEAR MODEL IS STILL THE RIGHT ANSWER --- */
      · few samples relative to features
      · you need an explanation, not just a prediction
      · the relationship really is roughly linear
      · you need microsecond inference in a few hundred bytes
        of code - which on an MCU is a decisive argument
        (chapter 27)
      · as a BASELINE. Always (chapter 12).

/* --- AND THE HONEST NOTE ON TABULAR DATA --- */
    For structured/tabular problems, the ladder that usually
```

```
    works is:
        linear/logistic  ->  gradient-boosted trees
        (XGBoost, LightGBM)  ->  and only then neural
        networks, which frequently do not win at all.
    Deep learning's advantage is on perceptual and
    sequential data. Knowing this saves months.          */
```

How it shows up

Symptom	Cause
Training diverges immediately	Features unscaled — an ill-conditioned loss surface
Loss decreases painfully slowly	Same cause, or the learning rate is too low
`nan` from `exp`	Unstable sigmoid — use the branch-free stable form
Confidently wrong predictions never improve	MSE used for classification — the gradient vanishes
Coefficients are huge and unstable	Collinear features — use ridge
`Singular matrix` in the normal equations	Rank-deficient `X` — add ridge, or use `lstsq`
Model cannot learn XOR	Correct — linear models cannot. Add a hidden layer
Validation worse than training by a lot	Overfitting — regularise or get more data
Predictions all one class	Imbalanced data at threshold 0.5 — move the threshold
Coefficients contradict domain knowledge	Correlated features splitting weight, or leakage
Test performance drops after scaling	Statistics computed on the full dataset — leakage

Check yourself

- ✓ Write the linear model, its loss and its gradients in matrix form.
- ✓ Give the normal equations and say when not to use them.
- ✓ Explain why feature scaling matters, in terms of the loss surface.
- ✓ Explain how polynomial features keep a model "linear".
- ✓ Contrast ridge and lasso, including what ridge does to singularity.
- ✓ Write binary cross-entropy and read both terms aloud.
- ✓ State the logistic gradient and explain why the sigmoid derivative cancels.
- ✓ Explain why MSE is the wrong loss for classification.
- ✓ Interpret a logistic coefficient as an odds ratio, with the three caveats.
- ✓ Show why XOR is not linearly separable and give two fixes.
- ✓ Say when a linear model is still the right choice, including on an MCU.

Neural networks: the multilayer perceptron

Stack the units from the last chapter, put a non-linearity between the layers, and the model can represent anything. This is the architecture every later one is a specialisation of.

The architecture

```
INPUT      HIDDEN 1     HIDDEN 2      OUTPUT
(784)       (128)        (64)          (10)

  o ---\   /-- o ---\   /-- o ---\   /-- o
  o ----\ /--- o ----\ /--- o ----\ /--- o
  o -----X---- o -----X---- o -----X---- o
  o ----/ \--- o ----/ \--- o ----/ \--- o
  o ---/   \-- o ---/   \-- o ---/   \-- o
           W1,b1       W2,b2        W3,b3
           ReLU        ReLU         softmax

EVERY LAYER IS:
     h = activation(W @ x + b)

THE FORWARD PASS, in full:
     h1 = relu(x  @ W1 + b1)      (784,) -> (128,)
     h2 = relu(h1 @ W2 + b2)      (128,) -> ( 64,)
     y  = softmax(h2 @ W3 + b3)   ( 64,) -> ( 10,)

PARAMETER COUNT:
     784*128 + 128   = 100,480
     128*64  +  64   =   8,256
     64*10   +  10   =     650
                     = 109,386 total

THE ACTIVATION IS THE WHOLE POINT. Without it:
     W3 (W2 (W1 x)) = (W3 W2 W1) x = W x
Three layers collapse into one matrix (chapter 03). THE
DEPTH WOULD BUY NOTHING AT ALL. A non-linearity between
layers is what makes composition meaningful.

THE UNIVERSAL APPROXIMATION THEOREM says a network with
ONE hidden layer can approximate any continuous function
on a bounded domain, to any accuracy, given enough units.

AND THE THEOREM IS ALMOST USELESS IN PRACTICE, which is
worth being clear about:
  · "enough units" may be exponentially many
  · it says nothing about whether TRAINING will FIND those
    weights
  · it says nothing about GENERALISATION

DEPTH IS WHAT ACTUALLY WORKS. Deep networks represent
many functions exponentially more compactly than shallow
ones, and they are the ones gradient descent can
successfully train. That empirical fact, not the theorem,
is why the field is called DEEP learning.
```

A complete implementation

```
"""A full MLP in numpy. Every line here was derived in
   chapters 06, 07, 10 and 11 - this is just assembly."""

import numpy as np

def relu(x):          return np.maximum(0, x)
def relu_grad(x):     return (x > 0).astype(x.dtype)

def softmax(x):
    x = x - x.max(axis=-1, keepdims=True)      # ch. 06
    e = np.exp(x)
    return e / e.sum(axis=-1, keepdims=True)

class MLP:
    def __init__(self, sizes):
        self.W, self.b = [], []
        for i in range(len(sizes) - 1):
            # HE INITIALISATION - the variance argument from
            # chapter 08. Getting this wrong is the most
            # common reason a network refuses to train.
            self.W.append(np.random.randn(sizes[i],
                                          sizes[i+1])
                          * np.sqrt(2.0 / sizes[i]))
            self.b.append(np.zeros(sizes[i+1]))

    def forward(self, X):
        self.z, self.a = [], [X]     # SAVED for backward
        for i in range(len(self.W)):
            z = self.a[-1] @ self.W[i] + self.b[i]
            self.z.append(z)
            # no activation on the OUTPUT layer - the loss
            # takes raw logits (chapter 10)
            self.a.append(relu(z) if i < len(self.W) - 1
                          else z)
        return self.a[-1]

    def backward(self, y_onehot):
        N = y_onehot.shape[0]
        grads_W = [None] * len(self.W)
        grads_b = [None] * len(self.b)

        # THE CLEAN GRADIENT: softmax + cross-entropy
        # collapses to (prediction - target). Chapter 06.
        delta = (softmax(self.a[-1]) - y_onehot) / N

        for i in reversed(range(len(self.W))):
            grads_W[i] = self.a[i].T @ delta     # ch. 07
            grads_b[i] = delta.sum(axis=0)
            if i > 0:
                delta = (delta @ self.W[i].T
                         * relu_grad(self.z[i-1]))
        return grads_W, grads_b

    def step(self, gW, gb, lr):
        for i in range(len(self.W)):
            self.W[i] -= lr * gW[i]
            self.b[i] -= lr * gb[i]


# --- TRAIN IT ---
```

```
model = MLP([784, 128, 64, 10])

for epoch in range(20):
    perm = np.random.permutation(len(X_train))
    for i in range(0, len(X_train), 64):
        idx = perm[i:i+64]
        Xb, yb = X_train[idx], y_train_onehot[idx]

        logits = model.forward(Xb)
        gW, gb = model.backward(yb)
        model.step(gW, gb, lr=0.1)

    logits = model.forward(X_val)
    acc = (logits.argmax(1) == y_val).mean()
    print(f"epoch {epoch}  val acc {acc:.4f}")

# ~98% ON MNIST. About sixty lines, no framework, and every
# line traceable to a chapter in this track.

# --- THE SAME THING IN PYTORCH ---
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 64),  nn.ReLU(),
    nn.Linear(64, 10),
)
# The framework provides autodiff (chapter 07),
# initialisation, optimisers (chapter 11) and GPU
# execution. IT COMPUTES EXACTLY WHAT THE CODE ABOVE
# COMPUTES.
```

Choosing an activation

Function	Formula	Use
ReLU	`max(0, x)`	**The default.** Cheap, no saturation for positive inputs
LeakyReLU	`max(0.01x, x)`	When units are dying
GELU	`x · Φ(x)`	Transformers — GPT, BERT
SiLU / Swish	`x · σ(x)`	Modern LLMs, usually gated as SwiGLU
Sigmoid	$1/(1+e^{-x})$	**Output only** — binary probability, or a gate
Tanh	—	LSTM internals; zero-centred
Softmax	—	Output layer for multi-class
None (identity)	—	Regression output, and every logit layer

```
/* THE DECISION IN PRACTICE:
     hidden layers, general        -> ReLU
     hidden layers, transformer    -> GELU or SwiGLU
     output, binary classification -> sigmoid
     output, multi-class           -> softmax (fused into
                                      the loss)
     output, regression            -> NONE

/* THE MOST COMMON ERROR: PUTTING AN ACTIVATION ON THE
   OUTPUT LAYER OF A CLASSIFIER AND THEN USING A LOSS THAT
   EXPECTS LOGITS. It halves your effective gradient and
   costs accuracy, and it produces no error message
   (chapter 06).

/* --- DEAD ReLUs, concretely --- */
    A unit whose pre-activation is negative for EVERY input
    outputs zero always, and its gradient is zero always, so
    it never recovers. Usually caused by too high a learning
    rate driving a large negative bias.

    DIAGNOSE:
        frac_dead = (activations == 0).all(axis=0).mean()
    A few percent is normal - ReLU sparsity is useful.
    Thirty percent means something is wrong.

    FIX: lower the learning rate, use LeakyReLU or GELU,
    check the initialisation.

/* --- AND FOR THE EMBEDDED READER (chapter 27) --- */
    ReLU is ONE COMPARISON. GELU involves a tanh or an erf
    approximation - roughly 10-20x the cost, and it needs a
    lookup table or a polynomial on a device without an FPU.
    On an MCU, ReLU (or ReLU6, which clamps at 6 and
    quantises cleanly) is almost always the right choice,
    and every embedded inference library optimises for
    it.                                                  */
```

WHAT THE LAYERS ARE DOING

Each hidden layer computes a new *representation* of the input — a set of learned features. The first layer of an image network learns edges; the next, corners and textures; the next, object parts. You did not specify any of those. **Representation learning is the actual contribution of deep learning**: instead of hand-engineering features and then fitting a linear model on top, the network learns the features and the classifier together, end to end. Every architecture after this chapter is a different structural prior about what representations are useful — convolution for spatial locality, attention for relevance across a sequence.

Why depth, and what the priors are

```
/* --- AN MLP HAS NO STRUCTURAL PRIOR --- */
    Every input connects to every unit. The model has no
    notion that pixel (3,4) is next to pixel (3,5), or that
    token 7 came before token 8. It must learn any such
    structure from data - which needs vastly more data than
    building the assumption in.

/* --- CONVOLUTIONAL NETWORKS: the spatial prior --- */
    A small kernel slides over the input, SHARING WEIGHTS
    across positions.

      · TRANSLATION EQUIVARIANCE: a feature detector that
        works at one position works at all of them
      · LOCALITY: nearby pixels are related
      · PARAMETER SHARING: a 3x3 kernel is 9 weights,
        regardless of image size

    A 224x224 image into a 1000-unit MLP layer needs 50
    million weights for one layer. A conv layer needs
    hundreds. THE PRIOR IS THE POINT - and it is why CNNs
    beat MLPs on images by an enormous margin.

    STILL THE RIGHT CHOICE for vision on embedded hardware:
    MobileNet, EfficientNet-Lite and their kin run on
    microcontrollers (chapter 27) where a vision transformer
    does not.

/* --- RECURRENT NETWORKS: the sequential prior --- */
    A hidden state carried from step to step. Chapter 18
    covers them and why they were superseded.

/* --- TRANSFORMERS: the relevance prior --- */
    Attention lets every position look at every other
    position, WEIGHTED BY LEARNED RELEVANCE (chapter 19).
    A weaker prior than convolution - which is why
    transformers need more data, and why, with enough data,
    they win.

    THE PATTERN ACROSS ALL THREE: A STRONGER PRIOR NEEDS
    LESS DATA BUT CAPS PERFORMANCE; A WEAKER PRIOR NEEDS
    MORE DATA AND SCALES FURTHER. That trade-off is the
    single most useful lens for reading architecture papers.

/* --- WHY DEEP RATHER THAN WIDE --- */
    · COMPOSITIONAL efficiency: some functions need
      exponentially more units in a shallow network than in
      a deep one
    · HIERARCHY: features build on features
    · EMPIRICALLY, depth wins - and residual connections
      (chapter 22) made very deep networks trainable, which
      was the unlock.

/* --- THE PROBLEMS DEPTH INTRODUCES, and their fixes --- */
      vanishing/exploding gradients  -> careful init,
                                        residual connections
      internal covariate shift       -> normalisation layers
      overfitting                    -> more data, dropout,
                                        weight decay
      compute cost                   -> the entire hardware
                                        industry
```

```
    Chapters 15 and 22 cover the first three. The fourth is
    someone else's problem.                                  */
```

How it shows up

Symptom	Cause
Deep network performs like a linear model	Missing activation between layers — they collapsed
Loss stuck at `log(n_classes)`	No learning — check initialisation and gradient flow
Many units always output zero	Dead ReLUs — lower the learning rate or switch activation
Accuracy lower than expected, no error	Softmax applied before a logit-expecting loss
Gradients vanish in the first layers	Depth without residuals or proper initialisation
Loss is `nan` from step 1	Initialisation scale far too large
MLP needs enormous data for images	No spatial prior — use convolution
Transformer underperforms a CNN on a small dataset	Weaker prior — expected below a data threshold
Wider network does not help	Depth, not width, is usually the lever
GELU is slow on an MCU	Transcendental function — use ReLU or ReLU6

Check yourself

- ✓ Write the forward pass of a three-layer MLP and count its parameters.
- ✓ Explain why layers collapse without a non-linearity.
- ✓ State the universal approximation theorem and three reasons it is not practically useful.
- ✓ Implement the backward pass for a two-layer MLP from memory.
- ✓ Explain why the output layer has no activation when using cross-entropy.
- ✓ Choose activations for hidden layers, binary output, multi-class output and regression.
- ✓ Diagnose and fix dead ReLUs.
- ✓ Explain representation learning in one paragraph.
- ✓ Give the structural prior of a CNN, an RNN and a transformer.
- ✓ State the prior-strength versus data-requirement trade-off.
- ✓ List four problems depth introduces and their standard fixes.

Training: loss, initialisation and regularisation

The practical craft. Everything that determines whether a correct architecture actually learns anything — and the diagnostic procedure for when it does not.

Initialisation

```
THE PROBLEM: the initial weights determine whether
signals and gradients survive their trip through the
network.

ALL ZEROS       every unit in a layer computes the same
                thing, receives the same gradient, and
                stays identical forever. THE NETWORK HAS
                ONE EFFECTIVE UNIT PER LAYER. Symmetry
                must be broken.

TOO LARGE       activations grow layer by layer;
                gradients explode; loss goes to nan
                within a few steps.

TOO SMALL       activations shrink to zero; gradients
                vanish; nothing learns.

THE RULE, derived from variance arithmetic in chapter 08:
KEEP THE VARIANCE OF THE ACTIVATIONS ROUGHLY CONSTANT
THROUGH DEPTH.

  XAVIER / GLOROT     Var(W) = 1 / n_in
                      (or 2/(n_in+n_out))
                      for tanh and sigmoid

  HE / KAIMING        Var(W) = 2 / n_in
                      for ReLU - the factor of 2
                      compensates for ReLU zeroing half
                      the outputs

  ORTHOGONAL          Q from a QR decomposition.
                      Preserves norms exactly
                      (chapter 03). Good for RNNs and
                      very deep stacks.

  BIASES              zero. Almost always.

AND FOR TRANSFORMERS SPECIFICALLY:
  · a SMALL initialisation (std 0.02) on all weights
  · the RESIDUAL PROJECTIONS scaled down by
    1/sqrt(2*n_layers), so the residual stream does not
    grow with depth (chapter 22)
  · embeddings drawn from N(0, 0.02)

THIS IS NOT FOLKLORE - it is the same variance argument,
applied to a specific topology. GPT-2's initialisation
scheme is three lines of code and it is load-bearing.
```

```
import numpy as np
import torch.nn as nn

# NUMPY
W = np.random.randn(n_in, n_out) * np.sqrt(2.0 / n_in)   # He

# PYTORCH - the defaults are reasonable but not optimal for
# every architecture, so set them explicitly:
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.normal_(m.weight, mean=0.0, std=0.02)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Embedding):
        nn.init.normal_(m.weight, mean=0.0, std=0.02)

model.apply(init_weights)

# THE DIAGNOSTIC - run this ONCE before training. It takes
# thirty seconds and it has saved more debugging time than
# any other habit in this chapter:
def check_activations(model, x):
    acts = {}
    hooks = [m.register_forward_hook(
                lambda mod, i, o, n=n: acts.__setitem__(
                    n, o.detach()))
             for n, m in model.named_modules()]
    model(x)
    for name, a in acts.items():
        print(f"{name:30s} std={a.std():.4f} "
              f"mean={a.mean():+.4f} "
              f"dead={(a == 0).float().mean():.2%}")
    for h in hooks: h.remove()

# WHAT YOU WANT TO SEE: std roughly CONSTANT across layers,
# somewhere near 1 (or near 1 after each normalisation
# layer). A std that shrinks 10x per layer, or grows 10x,
# tells you the problem before you waste an hour on the
# loss curve.
```

Choosing the loss

Task	Loss	Output layer
Binary classification	`BCEWithLogitsLoss`	1 logit, **no sigmoid**
Multi-class	`CrossEntropyLoss`	K logits, **no softmax**
Multi-label	`BCEWithLogitsLoss`	K logits
Regression	`MSELoss`	Linear
Regression with outliers	`HuberLoss` / `SmoothL1`	Linear
Language modelling	`CrossEntropyLoss`	V logits per position
Ranking / retrieval	Contrastive, InfoNCE, triplet	Embeddings
Imbalanced classification	Focal loss, or class weights	Logits

```
/* THE "WithLogits" VARIANTS ARE NOT A CONVENIENCE. They
   fuse the activation and the loss to gain numerical
   stability and the clean gradient (chapter 06). Using the
   separated version is measurably worse and occasionally
   produces nan.

/* --- HANDLING IMBALANCE --- */
    class weights:
        w = torch.tensor([1.0, 10.0])       # 10x on the
                                            # rare class
        F.cross_entropy(logits, y, weight=w)

    FOCAL LOSS down-weights easy examples so training
    focuses on the hard ones:
        FL = -(1 - p_t)^gamma * log(p_t)      gamma = 2
    Designed for extreme imbalance in object detection;
    useful whenever the easy negatives swamp the signal.

    AND THE SIMPLER OPTIONS THAT OFTEN WIN: resample the
    data, or just move the decision threshold after training
    (chapter 13). Try those first.

/* --- AUXILIARY AND COMBINED LOSSES --- */
    loss = main_loss + 0.1 * auxiliary_loss

    THE WEIGHT MATTERS ENORMOUSLY and is a real
    hyperparameter. Check the MAGNITUDES of the terms - if
    one is 100x the other, the small one is decoration. Print
    both separately during training; a combined loss that
    only ever reflects one term is a common silent failure.

/* --- THE SANITY CHECK YOU SHOULD ALWAYS RUN --- */
    At step 0, before any learning:
      · K-way classification -> loss should be ~log(K)
        (10 classes -> 2.303; 50k vocabulary -> 10.82)
      · regression -> loss should be ~the variance of y

    IF THE INITIAL LOSS IS WRONG, THE BUG IS IN THE MODEL,
    THE LABELS OR THE LOSS - not in the training. Finding it
    now costs minutes; finding it after a day of training
    costs a day.                                          */
```

Regularisation

```
/* --- MORE DATA. Always first, always best. --- */
    Nothing on this list comes close to the effect of
    doubling the dataset.

/* --- DATA AUGMENTATION: more data, for free --- */
    images  flips, crops, rotations, colour jitter,
            mixup, cutmix
    audio   time/pitch shift, noise, SpecAugment
    text    back-translation, synonym swap, token dropout -
            all much weaker than the vision equivalents,
            because language is less tolerant of
            perturbation
    sensor  jitter, scaling, window shifting, and SYNTHETIC
            FAULT INJECTION - which for embedded anomaly
            detection is often the only way to get positive
            examples at all

/* --- WEIGHT DECAY (L2) --- */
    opt = torch.optim.AdamW(params, weight_decay=0.1)
    A Gaussian prior on the weights (chapter 09). USE ADAMW,
    not Adam with L2 in the loss (chapter 11).

    AND EXCLUDE BIASES AND NORMALISATION PARAMETERS from
    decay - decaying a LayerNorm gain toward zero actively
    hurts:
        decay    = [p for n,p in model.named_parameters()
                    if p.dim() >= 2]
        no_decay = [p for n,p in model.named_parameters()
                    if p.dim() < 2]

/* --- DROPOUT --- */
    nn.Dropout(0.1)
    Randomly zero a fraction of activations during training;
    scale the rest by 1/(1-p) so the expected value is
    unchanged. At inference it is a no-op.

    THE INTUITION: units cannot rely on any particular other
    unit being present, so they learn redundant, distributed
    representations. Also approximately an ensemble over
    exponentially many sub-networks.

    RATES: 0.5 for old-style MLPs, 0.1 for transformers,
    OFTEN ZERO FOR LARGE-SCALE PRETRAINING - when the data
    is effectively unlimited and each example is seen once,
    there is nothing to overfit, and dropout only slows
    learning. It comes back for fine-tuning on small
    datasets.

    AND THE CLASSIC BUG: FORGETTING model.eval(). Dropout
    stays active, and your validation numbers are noisy and
    pessimistic. The same call switches BatchNorm to its
    running statistics.

/* --- EARLY STOPPING --- */
    best, patience, wait = inf, 10, 0
    for epoch in ...:
        val = validate()
        if val < best:
            best, wait = val, 0
            torch.save(model.state_dict(), "best.pt")
```

```
        else:
            wait += 1
            if wait >= patience:
                break
    Free, effective, and it also gives you the checkpoint.

/* --- LABEL SMOOTHING --- */
    Chapter 10. Prevents the model chasing infinite logits.

/* --- NORMALISATION LAYERS --- */
    BATCHNORM  normalise across the BATCH. Great for CNNs.
               DEPENDS ON BATCH SIZE, behaves differently in
               train and eval, and is awkward for sequences
               and for batch size 1 - which is exactly the
               embedded inference case.
    LAYERNORM  normalise across the FEATURES of each sample
               independently. NO BATCH DEPENDENCE, identical
               in train and eval. THE TRANSFORMER CHOICE
               (chapter 22).
    RMSNORM    LayerNorm without the mean subtraction.
               Cheaper, and what most recent LLMs use.

/* --- AND THE CHEAPEST WIN OF ALL: EMA OF THE WEIGHTS --- */
    ema = 0.999 * ema + 0.001 * weights
    Evaluate with the averaged weights. Costs one extra copy
    of the model and frequently gives a free accuracy
    improvement, because it lands in the middle of a basin
    rather than on its wall (chapter 11).                */
```

THE ORDER TO REACH FOR THESE

```
1. More data.
2. Data augmentation.
3. Early stopping (free, do it anyway).
4. Weight decay.
5. Dropout.
6. A smaller model.
```

And measure after each one. Stacking regularisers blindly is how you end up with a model that underfits and a hyperparameter search that cannot recover.

The debugging procedure

```
/* IN ORDER. Do not skip steps; each one rules out a whole
   class of bug and takes minutes.                          */

/* --- 1. OVERFIT A SINGLE BATCH --- */
    x, y = next(iter(dataloader))
    x, y = x[:8], y[:8]
    for i in range(500):
        loss = criterion(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
        if i % 50 == 0: print(i, loss.item())

    THE LOSS MUST REACH APPROXIMATELY ZERO. If it does not,
    the bug is in the model, the data pipeline or the loss -
    NOT in the hyperparameters. Nothing else you do matters
    until this passes.

/* --- 2. CHECK THE INITIAL LOSS --- */
    ~log(K) for K classes. Wrong value -> wrong labels,
    wrong loss, or a broken output layer.

/* --- 3. VISUALISE THE INPUT AT THE MODEL BOUNDARY --- */
    Print or plot exactly what the model receives, after all
    preprocessing. Wrong normalisation, channels in the wrong
    order, off-by-one label indexing, and a shuffled
    label/feature correspondence are all VISIBLE HERE and
    invisible everywhere else.

/* --- 4. CHECK GRADIENT FLOW --- */
    for n, p in model.named_parameters():
        print(n, None if p.grad is None
                 else p.grad.norm().item())
    None -> disconnected from the loss.
    0.0  -> a non-differentiable operation upstream.
    1e-8 -> vanishing. 1e+8 -> exploding. (chapter 07)

/* --- 5. CHECK ACTIVATION STATISTICS --- */
    The hook from the top of this chapter.

/* --- 6. LR RANGE TEST --- */
    Chapter 11.

/* --- 7. ONLY NOW: TUNE --- */
    And tune ONE THING AT A TIME, with a fixed seed, logging
    everything.

/* ================= THE SYMPTOM TABLE ================= */

   LOSS IS nan
     -> lr too high; log(0); division by zero; bad input
        data; missing gradient clipping; fp16 overflow

   LOSS DOES NOT MOVE AT ALL
     -> lr too low; gradients not flowing; optimizer.step()
        never called; parameters not passed to the optimiser
        (a real and common mistake when a submodule is
        created after the optimiser)

   LOSS DECREASES THEN EXPLODES
     -> lr too high for the later, sharper region; add a
        schedule and clipping
```

```
TRAIN LOSS DOWN, VAL LOSS UP
  -> overfitting. See the list above.

BOTH LOSSES PLATEAU HIGH
  -> underfitting: more capacity, longer training, higher
     lr, or the features genuinely do not contain the
     signal

VAL LOSS LOWER THAN TRAIN LOSS
  -> normal early on with dropout (train is measured with
     regularisation on); if persistent, suspect a data
     split problem

RESULTS VARY WILDLY BETWEEN SEEDS
  -> the model is too small, the data too little, or the
     lr too high. Report a mean over seeds, not your best
     run.

GREAT VALIDATION, POOR PRODUCTION
  -> leakage, or distribution shift (chapter 12)      */
```

How it shows up

Symptom	Cause
All units in a layer identical	Zero initialisation — symmetry never broken
Activation std shrinks 10× per layer	Initialisation scale too small
Loss `nan` at step 1	Initialisation too large, or bad input data
Initial loss not `log(K)`	Wrong labels, wrong loss, or a broken output layer
Cannot overfit 8 samples	A bug in the model, data pipeline or loss
Validation noisy and pessimistic	`model.eval()` not called — dropout still active
Weight decay hurts accuracy	Applied to biases and norm parameters — exclude them
A parameter never changes	Not passed to the optimiser
An auxiliary loss has no effect	Its magnitude is dwarfed by the main term
BatchNorm behaves differently at inference	By design — use LayerNorm for batch size 1
Results depend heavily on the seed	Model or data too small — report means across seeds

Check yourself

- ✓ Explain why zero initialisation fails and what "breaking symmetry" means.
- ✓ Give the Xavier and He variances and say why He has a factor of 2.
- ✓ Describe the transformer initialisation scheme, including the residual scaling.
- ✓ Write the activation-statistics diagnostic and say what good output looks like.
- ✓ Choose the loss and output layer for six task types.
- ✓ Explain why `WithLogits` losses are not merely convenient.
- ✓ Give three approaches to class imbalance and say which to try first.
- ✓ State the step-0 loss sanity check for classification and regression.
- ✓ Explain dropout, its scaling, and why large-scale pretraining often omits it.

- ✓ Contrast BatchNorm, LayerNorm and RMSNorm, and say which suits batch size 1.
- ✓ Recite the seven debugging steps in order.
- ✓ Diagnose: `nan`, flat loss, exploding loss, diverging validation, high plateau.

Tokenisation

Text becomes integers before a model sees any of it. This unglamorous preprocessing step is the origin of a surprising number of the strange behaviours people attribute to the model itself.

The problem

```
A NEURAL NETWORK CONSUMES NUMBERS. Text is a string. Some
mapping is required, and the choice has consequences.

OPTION 1 - CHARACTERS
  "hello" -> [104, 101, 108, 108, 111]
  + tiny vocabulary (~256 with bytes)
  + NO out-of-vocabulary words, ever
  - LONG sequences: attention is quadratic in length
    (chapter 19), so 5x longer input is 25x the attention
    cost
  - the model must learn spelling before it learns
    meaning

OPTION 2 - WORDS
  "hello world" -> [4821, 923]
  + short sequences, semantically meaningful units
  - HUGE vocabulary: 500k+ for English, and the output
    layer is (d_model x vocab) - the largest matrix in a
    small model
  - out-of-vocabulary words become <UNK> and their
    information is DESTROYED
  - "run", "runs", "running", "ran" are four unrelated
    entries
  - and it is not even well-defined for Chinese, Japanese
    or Thai, which do not delimit words with spaces

OPTION 3 - SUBWORDS. WHAT EVERYONE USES.
  "tokenisation" -> ["token", "isation"]
  "unhappiness"  -> ["un", "happiness"]
  "GPT-4"        -> ["G", "PT", "-", "4"]

  · common words stay whole
  · rare words split into meaningful pieces
  · NO out-of-vocabulary case at all - anything
    decomposes to bytes in the worst case
  · vocabulary of 30k-200k: a workable output layer
  · morphology is partially captured for free

THE PRACTICAL RATIO FOR ENGLISH: about 4 CHARACTERS PER
TOKEN, or roughly 0.75 words per token. 1,000 tokens is
about 750 words. This is the arithmetic behind every
context-length and pricing calculation you will do.
```

Byte-pair encoding

```
"""BPE, the dominant algorithm. Originally a 1994
   compression method, repurposed for NLP in 2015.

   TRAINING: start from individual bytes, repeatedly merge
   the most frequent adjacent pair, until the vocabulary
   reaches the target size."""

from collections import Counter

def train_bpe(corpus_words, num_merges):
    # start: every word as a tuple of bytes/characters
    vocab = {tuple(w): c for w, c in
             Counter(corpus_words).items()}
    merges = []

    for _ in range(num_merges):
        # 1. COUNT ADJACENT PAIRS, weighted by word frequency
        pairs = Counter()
        for word, freq in vocab.items():
            for i in range(len(word) - 1):
                pairs[(word[i], word[i+1])] += freq
        if not pairs:
            break

        # 2. MERGE THE MOST FREQUENT PAIR
        best = max(pairs, key=pairs.get)
        merges.append(best)

        new_vocab = {}
        for word, freq in vocab.items():
            w, i = [], 0
            while i < len(word):
                if (i < len(word) - 1 and
                        (word[i], word[i+1]) == best):
                    w.append(word[i] + word[i+1])
                    i += 2
                else:
                    w.append(word[i])
                    i += 1
            new_vocab[tuple(w)] = freq
        vocab = new_vocab

    return merges

def encode(word, merges):
    """Apply the learned merges IN THE ORDER THEY WERE
       LEARNED. That ordering is the model - it is why the
       merge list, not just the vocabulary, must be
       shipped."""
    tokens = list(word)
    for a, b in merges:
        i = 0
        while i < len(tokens) - 1:
            if tokens[i] == a and tokens[i+1] == b:
                tokens[i:i+2] = [a + b]
            else:
                i += 1
    return tokens

# WORKED, on a corpus of "low low lower newest widest":
```

```
#   merge 1: ('e','s') -> 'es'      (in newest, widest)
#   merge 2: ('es','t') -> 'est'
#   merge 3: ('l','o') -> 'lo'
#   ...
# Frequent substrings become single tokens; rare ones stay
# split.

/* --- THE VARIANTS YOU WILL MEET --- */
    BPE            merge by FREQUENCY. GPT-2/3/4, Llama,
                   most models.
    BYTE-LEVEL BPE Operate on RAW BYTES, not Unicode
                   characters. 256 base tokens covers
                   EVERYTHING - every language, every emoji,
                   every binary blob - with no unknown
                   token possible. GPT-2 onwards.
    WORDPIECE      merge by LIKELIHOOD GAIN rather than
                   raw frequency. BERT. Marks continuations
                   with "##".
    UNIGRAM / SentencePiece
                   start LARGE and prune tokens that cost
                   least likelihood. Handles languages
                   without spaces natively, and is
                   REVERSIBLE - it encodes whitespace as a
                   visible marker, so decode(encode(x)) == x
                   exactly. T5, Llama, Gemma.

/* --- USING ONE --- */
from tokenizers import Tokenizer
tok = Tokenizer.from_pretrained("gpt2")

ids = tok.encode("Hello, world!").ids   # [15496, 11, 995, 0]
tok.decode(ids)                          # "Hello, world!"

/* NOTE THE FOUR TOKENS FOR THREE WORDS AND TWO PUNCTUATION
   MARKS. And note that " world" (with the leading space)
   is a DIFFERENT TOKEN from "world". The space belongs to
   the following token in GPT-style tokenisers, which is the
   source of a whole family of surprises below.          */
```

Why models behave strangely

```
/* A large fraction of "the model is bad at X" is really
   "the tokeniser makes X hard", and knowing which is which
   saves a great deal of confusion.                        */

/* --- 1. COUNTING LETTERS --- */
    "How many r's in strawberry?"
    The model sees ["str", "aw", "berry"]. IT NEVER SEES
    INDIVIDUAL LETTERS. Asking it to count characters is
    like asking you to count the pixels in a word you read.
    It can sometimes do it by memorised association; it has
    no direct access.

/* --- 2. ARITHMETIC --- */
    Numbers tokenise inconsistently:
        "1234"  might be one token, or ["12","34"],
                or ["1","234"]
        "1235"  might split completely differently
    Digit-position alignment - which is what column
    arithmetic requires - is destroyed. THE FIX, used by
    several recent models: force digits to tokenise
    individually, or in fixed groups of three. It measurably
    improves arithmetic, which is strong evidence the
    tokeniser was the problem.

/* --- 3. REVERSING STRINGS, RHYMING, WORDPLAY --- */
    All require character-level access. Same cause.

/* --- 4. WHITESPACE SENSITIVITY --- */
    "hello" and " hello" are different tokens with different
    embeddings. A prompt ending in a trailing space puts the
    model in a strange state, because it now expects a
    continuation of a token rather than a new word. THIS IS
    A REAL SOURCE OF DEGRADED OUTPUT and it is invisible
    when you read the prompt.

/* --- 5. NON-ENGLISH TEXT COSTS MORE --- */
    Tokenisers are trained on corpora dominated by English.
        English   ~4 characters per token
        Chinese   ~1-1.5 characters per token
        Thai, Burmese, many African languages: sometimes
        1 token PER BYTE
    So the same document costs 2-5x more tokens, fits in
    less context, and costs more money. This is a genuine
    equity issue in deployed systems, not a curiosity.

/* --- 6. GLITCH TOKENS --- */
    " SolidGoldMagikarp" and a handful of others appeared in
    GPT-2's tokeniser training data (scraped from Reddit
    usernames) but were almost absent from the model's
    training data. Their embeddings were never meaningfully
    trained, so they sit at essentially random positions in
    embedding space, and prompting with them produced
    bizarre, evasive or nonsensical output.

    THE GENERAL LESSON: A TOKEN THAT EXISTS IN THE VOCABULARY
    BUT NOT IN THE TRAINING DATA IS AN UNTRAINED PARAMETER
    SITTING IN YOUR MODEL. Tokeniser and model must be
    trained on consistent data.

/* --- 7. CODE --- */
```

```
    Indentation matters in Python, and a tokeniser trained
    mostly on prose splits whitespace poorly. Code-focused
    models add explicit tokens for runs of 2, 4, 8, 16
    spaces - a small change with a large effect on code
    quality and on token count.                          */
```

THE RULE

The tokeniser and the model are a matched pair. Changing the tokeniser invalidates every embedding and requires retraining from scratch. Vocabulary size, digit handling and whitespace treatment are architectural decisions made *before* pretraining and effectively permanent afterwards — which is why tokeniser choices from 2019 are still shaping model behaviour today.

Practical matters

```
/* --- SPECIAL TOKENS --- */
    <|endoftext|>    document boundary; also used as
                     padding and as BOS in GPT-2
    [CLS] [SEP]      BERT's classification and separator
    <|im_start|>
    <|im_end|>       chat message boundaries in
                     instruction-tuned models
    <PAD>            padding, and it MUST BE MASKED OUT of
                     both attention and loss
    <|fim_prefix|>   fill-in-the-middle, for code models

    THE CHAT TEMPLATE MATTERS. An instruction-tuned model
    was fine-tuned on a specific format, and using a
    different one degrades it substantially:

        tok.apply_chat_template(messages, tokenize=False)

    USE THE LIBRARY FUNCTION rather than assembling the
    string yourself - this is a common and silent source of
    poor results.

/* --- VOCABULARY SIZE: THE TRADE-OFF --- */
    LARGER vocabulary
      + shorter sequences -> cheaper attention (quadratic!)
      + more semantics per token
      - a bigger embedding table and output projection
      - each token is seen less often during training
      - and a slower softmax over the vocabulary

    TYPICAL: 32k (Llama 2), 50k (GPT-2), 100k (GPT-4),
    128k-256k (Llama 3, Gemma). THE TREND IS UPWARD,
    because attention cost dominates and multilingual
    coverage matters.

/* --- COUNTING TOKENS, which you will do constantly --- */
    ids = tok.encode(text).ids
    n = len(ids)

    RULES OF THUMB FOR ENGLISH:
        1 token   ~= 4 characters ~= 0.75 words
        1,000 tokens ~= 750 words ~= 1.5 pages
    ALWAYS MEASURE for other languages and for code.

/* --- BATCHING AND PADDING --- */
    Sequences in a batch must be the same length.
      PAD to the longest in the batch (dynamic padding),
        not to the model maximum - padding to 2048 when your
        longest sample is 100 tokens wastes 95% of the
        compute
      MASK the padding in attention
      IGNORE it in the loss:
          F.cross_entropy(logits, targets,
                          ignore_index=PAD_ID)
      SORT BY LENGTH into buckets to minimise padding
      AND FOR PRETRAINING, don't pad at all: concatenate
        documents with a separator and slice into fixed
        blocks. Zero waste, and it is what every large
        pretraining run does.

/* --- FOR EMBEDDED DEPLOYMENT (chapter 27) --- */
```

```
    The tokeniser must ship with the model and run on the
    device. A 128k-entry vocabulary with its merge list is
    a few megabytes of data and needs a real string
    implementation.

    ON A CONSTRAINED DEVICE, CONSIDER:
      · a SMALL DOMAIN-SPECIFIC VOCABULARY trained on your
        actual data - 1-4k tokens is often plenty for a
        command or intent model, and shrinks the embedding
        table dramatically
      · CHARACTER-LEVEL, if sequences are short. No
        vocabulary file at all.
      · doing tokenisation on the HOST and sending IDs, if
        the device is not the entry point.
    The output projection is proportional to vocabulary
    size, so on an MCU this is frequently the single
    largest layer in the model.                          */
```

How it shows up

Symptom	Cause
Model miscounts letters in a word	It never sees characters — subword tokens
Arithmetic errors on multi-digit numbers	Inconsistent digit tokenisation destroys place alignment
Output degrades with a trailing space in the prompt	Whitespace belongs to the following token
Non-English text costs 3× more tokens	Tokeniser trained mostly on English
A rare string produces bizarre output	Glitch token — an untrained embedding
Instruction model responds poorly	Chat template not applied, or applied wrongly
Loss includes garbage	Padding not excluded via `ignore_index`
Attention attends to padding	No attention mask
Training is 10× slower than expected	Padding to the model maximum instead of dynamic batching
Model outputs nonsense after a tokeniser swap	Embeddings are tied to the tokeniser — retrain
Code model handles indentation badly	No whitespace-run tokens

Check yourself

- ✓ Give the trade-offs of character, word and subword tokenisation.
- ✓ State the characters-per-token and words-per-token rules for English.
- ✓ Describe the BPE training algorithm and why the merge *order* must be preserved.
- ✓ Contrast BPE, byte-level BPE, WordPiece and Unigram.
- ✓ Explain why byte-level tokenisation makes unknown tokens impossible.
- ✓ Explain the strawberry problem and the arithmetic problem in terms of tokens.
- ✓ Explain glitch tokens and the general lesson about untrained parameters.
- ✓ Give the vocabulary-size trade-off in both directions.
- ✓ Explain why non-English text costs more and why that matters.
- ✓ Describe correct padding, masking and loss exclusion — and the pretraining alternative.
- ✓ Give three tokenisation strategies for a constrained device.

Embeddings and vector semantics

Token IDs are arbitrary integers with no meaning. An embedding table turns each one into a learned vector whose *position* carries meaning — and that is the input to every model in the rest of this track.

Why a lookup table

```
TOKEN ID 4821 IS NOT A NUMBER YOU CAN COMPUTE WITH.
ID 4821 is not "more" than ID 4820, and their difference
means nothing.

ONE-HOT ENCODING makes them comparable but is useless:
    "cat"  -> [0,0,...,1,...,0]     50,000 entries
    "dog"  -> [0,0,...,0,...,1]
  · 50,000 dimensions per token
  · EVERY PAIR IS EQUALLY DISSIMILAR - "cat" and "dog"
    are exactly as unrelated as "cat" and "hydraulic"
  · almost all zeros

AN EMBEDDING is a LEARNED DENSE VECTOR:

    "cat"  -> [0.2, -0.5, 0.8, ..., 0.1]   768 numbers
    "dog"  -> [0.3, -0.4, 0.7, ..., 0.2]   NEARBY
    "hydraulic" -> [-0.9, 0.1, -0.3, ...]  FAR AWAY

THE EMBEDDING TABLE is simply a (vocab_size, d_model)
matrix, and "embedding a token" is a ROW LOOKUP:

    E = matrix of shape (50000, 768)
    embed(4821) = E[4821]

AND NOTICE: THIS IS EXACTLY EQUIVALENT to multiplying the
one-hot vector by E. The lookup is just the efficient
implementation of that matrix product - which is why the
gradient of an embedding lookup is "add the incoming
gradient to that row" and nothing else.

THE PARAMETERS ARE LEARNED. Nobody assigns coordinates.
Backpropagation moves each row so that the model predicts
better, and semantic structure EMERGES from that pressure.

SIZE MATTERS: a 128k vocabulary at d_model = 4096 is
524 million parameters IN THE EMBEDDING TABLE ALONE -
often 10-20% of a mid-sized model, and the largest single
tensor in a small one.
```

What the space looks like

```
/* THE DISTRIBUTIONAL HYPOTHESIS (Firth, 1957):
   "You shall know a word by the company it keeps."

   Words appearing in similar contexts get similar vectors,
   because the training objective pushes them there. That is
   the entire mechanism - no semantics were programmed in. */

import numpy as np

def cosine(a, b):                              # chapter 02
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

cosine(E["cat"], E["dog"])        # ~0.7   related
cosine(E["cat"], E["kitten"])     # ~0.8   closely related
cosine(E["cat"], E["hydraulic"])  # ~0.05  unrelated

/* --- THE ANALOGY RESULT --- */
    vec("king") - vec("man") + vec("woman")  ~=  vec("queen")
    vec("Paris") - vec("France") + vec("Italy")
                                           ~=  vec("Rome")

    DIRECTIONS IN THE SPACE CORRESPOND TO RELATIONSHIPS.
    Gender is a direction. Capital-of is a direction. Plural
    is a direction.

    THE HONEST CAVEAT, because this result is over-sold: the
    analogy typically only works when you EXCLUDE the three
    input words from the nearest-neighbour search, and it is
    sensitive to normalisation. The underlying claim -
    that linear structure exists - is solid; the party trick
    is fragile.

/* --- WHAT ELSE IS ENCODED --- */
    · syntax: nouns cluster, verbs cluster, tenses align
    · sentiment: a direction separating positive from
      negative
    · morphology: "run"/"running" offset resembles
      "walk"/"walking"
    · AND SOCIAL BIAS. Embeddings trained on human text
      encode human associations, including harmful ones:
      occupations align with gender directions,
      names align with sentiment by ethnicity. This is not a
      training bug - it is an accurate reflection of the
      corpus, and it propagates into any downstream system
      that uses the embeddings. MEASURE IT (WEAT and similar
      tests) IF YOUR SYSTEM MAKES DECISIONS ABOUT PEOPLE.

/* --- SUPERPOSITION: how so much fits in so few
       dimensions --- */
    A model with 768 dimensions appears to represent far
    more than 768 distinct features. It can, because in high
    dimensions there are exponentially many ALMOST-orthogonal
    directions (chapter 02). Features are stored in
    superposition, with small interference.

    THIS IS THE LEADING EXPLANATION for why bigger models
    are disproportionately better - more dimensions means
    exponentially more room - and it is the basis of the
    sparse-autoencoder work in mechanistic interpretability,
```

```
 which tries to decompose those superposed directions back
 into individual interpretable features.              */
```

Static versus contextual

```
STATIC EMBEDDINGS - word2vec (2013), GloVe, fastText
  ONE VECTOR PER WORD, FIXED.

  "I went to the BANK to deposit money"
  "I sat on the river BANK"
     -> the SAME vector. The model cannot distinguish
        them.

  word2vec's two training objectives:
    SKIP-GRAM   predict context words from the centre
                word
    CBOW        predict the centre word from its context
  plus NEGATIVE SAMPLING, which turns an expensive softmax
  over the whole vocabulary into a handful of binary
  classifications.

  STILL USEFUL: they are tiny, fast, need no GPU, and for
  a keyword or intent classifier on a microcontroller they
  may be the only option that fits (chapter 27).

CONTEXTUAL EMBEDDINGS - ELMo, BERT, GPT, everything since
  THE VECTOR DEPENDS ON THE SURROUNDING TEXT.

  The embedding TABLE still gives a static starting
  vector, but each transformer layer MIXES INFORMATION
  FROM OTHER POSITIONS via attention (chapter 19). By the
  final layer, "bank" in the two sentences has two very
  different representations.

  THE INPUT EMBEDDING IS THE STARTING POINT, NOT THE
  ANSWER. This distinction is the single most important
  thing in this chapter: everything a transformer does is
  progressively refining these vectors using context.

SENTENCE / DOCUMENT EMBEDDINGS
  One vector for a whole passage, for retrieval and
  similarity.
  · mean-pool the token embeddings (simple, decent)
  · use the [CLS] position (BERT-style)
  · OR TRAIN FOR IT: sentence-transformers, E5, BGE, and
    similar models are fine-tuned with a CONTRASTIVE
    objective so that cosine similarity actually
    corresponds to semantic similarity.

  AND THIS MATTERS: raw mean-pooled BERT embeddings are
  notoriously poor at similarity. Use a model TRAINED for
  embedding, not a language model's hidden states.
```

Implementation

```
import torch
import torch.nn as nn

# --- THE LAYER ---
emb = nn.Embedding(num_embeddings=50000, embedding_dim=768)

ids = torch.tensor([[15496, 11, 995]])   # (batch=1, seq=3)
vecs = emb(ids)                           # (1, 3, 768)

# THAT IS THE ENTIRE INPUT PIPELINE OF A TRANSFORMER:
# integers in, vectors out.

# --- WEIGHT TYING: a free improvement ---
model.lm_head.weight = model.embed_tokens.weight
# Share the input embedding table with the output projection
# to the vocabulary. THE INTUITION: both matrices map
# between token space and vector space, in opposite
# directions.
#
#   · SAVES vocab*d_model PARAMETERS - 524M for the example
#     above, which is enormous
#   · usually IMPROVES quality for small and mid-sized
#     models
#   · standard in GPT-2 and many others
#
# Very large models sometimes untie them, having enough
# capacity that the constraint costs more than it saves.

# --- PADDING ---
emb = nn.Embedding(50000, 768, padding_idx=0)
# Row 0 is fixed at zero and receives NO GRADIENT. Without
# this, the pad token accumulates updates and drifts into a
# meaningless but non-zero position.

# --- FROZEN PRETRAINED EMBEDDINGS ---
emb = nn.Embedding.from_pretrained(vectors, freeze=True)
# Useful with small datasets: you inherit semantics learned
# from a large corpus without the data to relearn them.

# --- A RETRIEVAL SYSTEM, end to end ---
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

docs = ["The cat sat on the mat", "Dogs are loyal",
        "Python is a programming language"]
doc_vecs = model.encode(docs, normalize_embeddings=True)

q = model.encode("feline on a rug", normalize_embeddings=True)
scores = doc_vecs @ q          # NORMALISED -> the dot
                               # product IS cosine similarity
best = scores.argmax()         # -> document 0

# THIS IS THE WHOLE OF SEMANTIC SEARCH, and the retrieval
# half of RAG (retrieval-augmented generation). At scale,
# the linear scan is replaced by an APPROXIMATE NEAREST
# NEIGHBOUR index - HNSW or IVF-PQ, in FAISS or a vector
# database - which trades a little recall for logarithmic
# search time.

# --- AND THE THING TO GET RIGHT ---
```

```
# NORMALISE ONCE AT INSERT TIME (chapter 02). Then querying
# is pure matrix-vector multiplication, which is exactly the
# memory-bound operation from chapter 04 - so a vector
# search over a million documents is a BANDWIDTH problem,
# and quantising the stored vectors to int8 makes it 4x
# faster. The same reasoning as LLM inference, applied to a
# database.
```

EMBEDDINGS ARE THE INTERFACE

Every modality reaches a transformer the same way: as a sequence of vectors. Text goes through a token embedding table. Images are split into patches and each patch is linearly projected. Audio becomes spectrogram frames. Once anything is a sequence of `d_model`-dimensional vectors, the architecture in the next chapters does not care where it came from — which is precisely why multimodal models were an incremental change rather than a redesign.

How it shows up

Symptom	Cause
`IndexError` in the embedding layer	A token ID at or above `num_embeddings` — tokeniser/model mismatch
Pad token acquires meaning	`padding_idx` not set
Similarity scores are all near 0.8	Anisotropic space — use a model trained for embedding, or centre the vectors
Semantic search returns poor matches	Raw LM hidden states instead of a contrastively trained encoder
Cosine similarity does not match the dot product	Vectors not normalised
Homonyms confused	Static embeddings — one vector per word by design
Model is 30% embedding parameters	Large vocabulary — consider weight tying
Occupation predictions correlate with gender	Corpus bias encoded in the embeddings — measure it
Analogy arithmetic fails	Input words not excluded from the neighbour search
Vector search is slow at a million documents	Linear scan — use an ANN index, and quantise

Check yourself

- ✓ Explain why one-hot encoding is useless and what an embedding table replaces it with.
- ✓ Say why an embedding lookup is equivalent to a matrix product, and what its gradient is.
- ✓ Compute the parameter count of a 128k × 4096 embedding table.
- ✓ State the distributional hypothesis and how it becomes a training signal.
- ✓ State the analogy result and its honest caveat.
- ✓ Explain superposition and why it favours larger models.
- ✓ Contrast static and contextual embeddings with the "bank" example.
- ✓ Say why a language model's hidden states make poor sentence embeddings.
- ✓ Explain weight tying and quantify what it saves.
- ✓ Write a semantic search from encoding to ranking.
- ✓ Explain why embeddings are the universal interface for multimodal models.

Sequence models before transformers

Recurrent networks dominated sequence modelling for twenty-five years. Understanding what they did well and exactly why they were displaced is the clearest possible motivation for attention — and they are quietly coming back on embedded hardware.

The recurrent neural network

```
THE IDEA: carry a HIDDEN STATE from step to step.

     h_t = tanh(W_hh @ h_{t-1} + W_xh @ x_t + b)
     y_t = W_hy @ h_t

UNROLLED OVER TIME:

  x1      x2      x3      x4
   |       |       |       |
   v       v       v       v
 [RNN]->[RNN]->[RNN]->[RNN]->
   |  h1   |  h2   |  h3   |  h4
   v       v       v       v
  y1      y2      y3      y4

THE SAME WEIGHTS AT EVERY STEP. That parameter sharing is
the sequential prior (chapter 14): the model assumes the
same computation applies at every position.

WHAT IS APPEALING ABOUT THIS:
  · O(1) MEMORY PER STEP - the state is fixed size
    regardless of how much text has been seen
  · O(n) TOTAL COMPUTE for a sequence of length n
  · handles ARBITRARY LENGTH input
  · it is the obvious, natural model of a sequence

WHAT KILLS IT:

1. NO PARALLELISM. h_t depends on h_{t-1}. You cannot
   compute step 5 before step 4. TRAINING IS INHERENTLY
   SEQUENTIAL, which wastes almost all of a GPU. This,
   more than accuracy, is why transformers won: a
   transformer processes an entire sequence in parallel
   during training.

2. VANISHING GRADIENTS. Backpropagation through time
   multiplies by W_hh at every step (chapters 05, 07):

      dL/dh_1 involves  (W_hh)^n

   Largest eigenvalue below 1 -> the gradient vanishes
   exponentially. Above 1 -> it explodes. IN PRACTICE A
   PLAIN RNN CANNOT LEARN DEPENDENCIES BEYOND ABOUT 10
   STEPS.

3. THE INFORMATION BOTTLENECK. Everything about the past
   must fit in one fixed-size vector. For a 1,000-token
   document compressed into 512 numbers, most of it is
   simply gone.
```

LSTM and GRU

```
/* LONG SHORT-TERM MEMORY (Hochreiter and Schmidhuber,
   1997). The fix that made recurrence work, and it held the
   field for two decades.

   THE KEY IDEA: A SEPARATE CELL STATE WITH AN ADDITIVE PATH,
   controlled by learned GATES.                          */

    f_t = sigmoid(W_f @ [h_{t-1}, x_t] + b_f)  /* FORGET  */
    i_t = sigmoid(W_i @ [h_{t-1}, x_t] + b_i)  /* INPUT   */
    o_t = sigmoid(W_o @ [h_{t-1}, x_t] + b_o)  /* OUTPUT  */
    g_t = tanh(   W_g @ [h_{t-1}, x_t] + b_g)  /* CANDIDATE */

    c_t = f_t * c_{t-1} + i_t * g_t     /* THE CELL STATE */
    h_t = o_t * tanh(c_t)               /* the hidden state */

/* WHY THIS SOLVES VANISHING GRADIENTS:

   Look at the cell state update. The path from c_{t-1} to
   c_t is MULTIPLICATION BY f_t AND AN ADDITION. If the
   forget gate is near 1, the gradient flows back
   essentially UNCHANGED - no repeated multiplication by a
   weight matrix.

   This is called the CONSTANT ERROR CARROUSEL, and it is
   the same structural idea as a RESIDUAL CONNECTION
   (chapter 22): give the gradient a path that does not go
   through a transformation.

   THE GATES ARE LEARNED, SOFT SWITCHES:
     forget gate near 0 -> discard the past
     input gate near 1  -> write the new candidate
     output gate        -> decide what to expose

   AND THE ONE PRACTICAL TRICK: INITIALISE THE FORGET GATE
   BIAS TO 1 (or 2). It starts the network in
   "remember everything" mode, and it measurably improves
   training. A one-line change that was a small paper.

/* --- GRU: the simplified version --- */
    z_t = sigmoid(W_z @ [h_{t-1}, x_t])     /* UPDATE gate */
    r_t = sigmoid(W_r @ [h_{t-1}, x_t])     /* RESET  gate */
    h~  = tanh(W @ [r_t * h_{t-1}, x_t])
    h_t = (1 - z_t) * h_{t-1} + z_t * h~

    Two gates instead of three, no separate cell state,
    ~25% fewer parameters. Usually comparable in accuracy
    and faster. WHEN CHOOSING BETWEEN THEM ON A CONSTRAINED
    DEVICE, START WITH GRU.

/* --- IN CODE --- */
import torch.nn as nn

lstm = nn.LSTM(input_size=128, hidden_size=256,
               num_layers=2, batch_first=True,
               dropout=0.1, bidirectional=False)

out, (h_n, c_n) = lstm(x)     # x: (batch, seq, 128)
                              # out: (batch, seq, 256)

/* BIDIRECTIONAL: run one LSTM forwards and one backwards,
```

```
    concatenate. Every position then sees the whole sequence.
    USE FOR CLASSIFICATION AND TAGGING; IMPOSSIBLE FOR
    GENERATION, because you cannot look at the future you
    have not generated yet.                                */
```

Where attention came from

```
/* THE SETTING: SEQUENCE-TO-SEQUENCE TRANSLATION (2014).

   ENCODER LSTM reads the source sentence -> a single
   context vector
   DECODER LSTM generates the target from that vector

      "the cat sat on the mat"
              |
              v
        [ONE VECTOR]        <-- EVERYTHING must fit here
              |
              v
      "le chat s'est assis sur le tapis"

   AND IT WORKED - until sentences got long. Translation
   quality fell off a cliff past about 30 words, because the
   whole source had to be compressed into one fixed vector.

/* BAHDANAU ET AL., 2015: LET THE DECODER LOOK BACK.

   Instead of one context vector, keep ALL the encoder
   states, and at each decoding step compute a WEIGHTED SUM
   of them:

     score_i   = alignment(decoder_state, encoder_state_i)
     alpha     = softmax(scores)          /* weights summing
                                             to 1 */
     context   = sum_i alpha_i * encoder_state_i

   THE WEIGHTS ARE LEARNED AND DYNAMIC. When generating
   "chat", the model attends to "cat". When generating
   "tapis", it attends to "mat". You can PLOT the alignment
   matrix and it looks like a translation dictionary -
   which was the moment the field realised something
   important had happened.

/* THIS IS ATTENTION, and note what it fixed:
     · no fixed-size bottleneck
     · a DIRECT PATH from any output position to any input
       position - so gradients do not have to travel through
       n recurrent steps
     · and it is INTERPRETABLE

/* --- LUONG (2015) simplified the scoring to a dot
       product --- */
        score = decoder_state · encoder_state

   Cheaper, and it is one step from the scaled dot-product
   attention of chapter 19.

/* --- AND THEN THE OBSERVATION THAT CHANGED EVERYTHING --- */

   Vaswani et al., 2017: "ATTENTION IS ALL YOU NEED".

   If attention gives every position direct access to every
   other position, WHAT IS THE RECURRENCE FOR?

   REMOVE IT. Keep only attention, plus a feed-forward
   network and positional information.
```

```
   THE PAYOFF WAS NOT PRIMARILY ACCURACY - IT WAS
   PARALLELISM. Without the sequential dependency, an entire
   sequence trains in ONE parallel pass. That turned
   sequence modelling into a big matrix multiplication
   (chapter 04), which is exactly what GPUs do, and it made
   scaling to hundreds of billions of parameters an
   engineering problem rather than an impossible one.     */
```

THE COMPARISON THAT EXPLAINS THE SWITCH

	RNN/LSTM	TRANSFORMER
training parallelism	NONE	FULL
path between two positions	O(distance)	O(1)
compute per layer	O(n·d²)	O(n²·d + n·d²)
memory at inference	O(1)	O(n) (the KV cache)
max useful context	hundreds	hundreds of thousands

The transformer is *worse* asymptotically in sequence length and *worse* in inference memory. It won because training parallelism and the O(1) path length mattered far more — a reminder that the right complexity measure is the one that matches your hardware.

Where recurrence stands now

```
/* RNNs ARE NOT ONLY OF HISTORICAL INTEREST, and this is the
   part that matters most for an embedded engineer.       */

/* --- THE TRANSFORMER'S WEAKNESS IS INFERENCE MEMORY --- */
    A transformer must keep a KV CACHE that GROWS WITH
    SEQUENCE LENGTH (chapter 25). An RNN keeps a fixed-size
    state, forever.

    FOR STREAMING INFERENCE ON A DEVICE - continuous audio,
    a sensor stream, always-on keyword spotting - a
    fixed-size state is a decisive advantage. The memory
    budget is known at compile time.

/* --- THE MODERN RECURRENT MODELS --- */
    STATE SPACE MODELS (S4, MAMBA)
      A linear recurrence, formulated so that TRAINING can be
      parallelised with a scan (or a convolution) while
      INFERENCE remains a constant-memory recurrence.

      THAT COMBINATION IS THE POINT: it recovers the
      transformer's training parallelism WITHOUT the growing
      inference state. Mamba's selective mechanism makes the
      recurrence input-dependent, which was the missing
      ingredient for language.

    RWKV, RETNET, GRIFFIN, xLSTM
      Various routes to the same goal - "transformer-quality
      training, RNN-quality inference".

    HYBRID ARCHITECTURES
      Most recent efficient models interleave a few attention
      layers among many recurrent or linear ones. Attention
      where precise recall matters; recurrence for
      everything else. This is currently the most practical
      answer.

/* --- AND THE CLASSIC RNN IS STILL DEPLOYED --- */
    On microcontrollers, a small GRU or LSTM remains a
    sensible choice for:
      · keyword spotting ("hey device")
      · sensor time-series classification and anomaly
        detection
      · gesture recognition from an IMU
      · predictive maintenance from vibration data

    WHY: a 2-layer GRU with 64 hidden units is a few tens of
    kilobytes, runs in constant memory, has no attention
    matrix to materialise, and every embedded inference
    library supports it. CMSIS-NN and TFLite Micro both have
    optimised LSTM/GRU kernels. Chapter 27 covers the
    deployment.

    THE ENGINEERING JUDGEMENT: FOR SHORT SEQUENCES AND TIGHT
    MEMORY, RECURRENCE IS OFTEN STILL CORRECT. Do not reach
    for a transformer because it is fashionable; reach for
    it when you need long-range precise recall and you have
    the memory for the cache.                              */
```

How it shows up

Symptom	Cause
RNN cannot learn dependencies past ~10 steps	Vanishing gradients — use an LSTM or GRU
Loss becomes `nan` during BPTT	Exploding gradients — clip the norm
GPU utilisation is low when training an RNN	Inherently sequential — expected
Translation quality collapses on long sentences	The fixed-size context bottleneck — add attention
Bidirectional model cannot generate	It sees the future — use unidirectional for generation
LSTM forgets everything early in training	Forget-gate bias not initialised to 1
Transformer runs out of memory on long streams	KV cache grows with length — consider a recurrent model
Transformer will not fit on an MCU	Attention working set — a GRU may be the right answer
Padded steps corrupt the hidden state	No `pack_padded_sequence` or mask

Check yourself

- ✓ Write the RNN recurrence and say what parameter sharing assumes.
- ✓ Give the three reasons plain RNNs fail, and say which one mattered most commercially.
- ✓ Write the LSTM equations and explain the constant error carrousel.
- ✓ Connect the LSTM cell path to residual connections.
- ✓ Say what the forget-gate bias initialisation does.
- ✓ Contrast LSTM and GRU and pick one for a microcontroller.
- ✓ Explain the seq2seq bottleneck and how Bahdanau attention removed it.
- ✓ State what "attention is all you need" removed and what the real payoff was.
- ✓ Fill in the RNN-versus-transformer comparison table from memory.
- ✓ Explain what state space models recover and why it matters for streaming.
- ✓ Give four embedded tasks where a small GRU is still the right choice.

Attention

Every position looks at every other position and takes a weighted average of what it finds, with the weights computed from the content itself. One equation, and it is the central mechanism of every modern language model.

The idea, before the equation

```
THE PROBLEM: "The animal didn't cross the street because
IT was too tired."

What does "it" refer to? To represent "it" properly, the
model must incorporate information from "animal" -
which is six positions away, and which position varies
from sentence to sentence.

THE MECHANISM - a soft, differentiable dictionary lookup:

  QUERY   what this position is LOOKING FOR
  KEY     what each position OFFERS
  VALUE   what each position CONTRIBUTES if selected

A HARD DICTIONARY: find the key that matches exactly,
                   return its value.
ATTENTION:         compare the query to EVERY key, get a
                   similarity score for each, softmax the
                   scores into weights, and return the
                   WEIGHTED AVERAGE OF ALL VALUES.

IT IS DIFFERENTIABLE, so it can be learned.

FOR THE TOKEN "it":
   query("it") · key("animal")  -> HIGH
   query("it") · key("street")  -> medium
   query("it") · key("because") -> low
   ...
   softmax  ->  [0.6, 0.2, 0.05, ...]
   output   =  0.6*value("animal") + 0.2*value("street")
               + ...

The representation of "it" now CONTAINS information about
"animal". That is what "contextual embedding"
(chapter 17) means, mechanically.

AND THE THREE PROPERTIES THAT MATTER:
  · CONTENT-BASED. The weights depend on what the tokens
    ARE, not on their positions. Attention itself is
    PERMUTATION-INVARIANT - which is why positional
    encoding must be added separately (chapter 21).
  · O(1) PATH LENGTH between any two positions. Contrast
    the RNN's O(distance) (chapter 18).
  · FULLY PARALLEL. Every position's output is computed
    simultaneously - one matrix multiply.
```

Scaled dot-product attention

```
/* THE EQUATION. Memorise it. */

    Attention(Q, K, V) = softmax( Q K^T / sqrt(d_k) ) V

    Q: (n, d_k)     queries   - one per position
    K: (n, d_k)     keys
    V: (n, d_v)     values
    output: (n, d_v)

/* STEP BY STEP, WITH SHAPES - the only way to read
   attention code:

    1. SCORES = Q @ K.T              (n, n)
       Every query dotted with every key. THIS MATRIX IS THE
       QUADRATIC COST.

    2. SCALE by 1/sqrt(d_k)          (n, n)

    3. MASK (optional)               (n, n)
       set forbidden positions to -inf

    4. WEIGHTS = softmax(scores, dim=-1)   (n, n)
       each ROW sums to 1

    5. OUTPUT = WEIGHTS @ V          (n, d_v)
       a weighted average of the values

/* WHY DIVIDE BY sqrt(d_k) - AND THIS IS NOT ARBITRARY:

   If q and k have independent components with mean 0 and
   variance 1, then their dot product over d_k dimensions
   has variance d_k (chapter 08). So for d_k = 64 the scores
   have a standard deviation of 8, and for d_k = 128, about
   11.

   LARGE SCORES SATURATE THE SOFTMAX: it approaches a
   one-hot distribution, the gradient becomes almost zero,
   and the layer stops learning. Dividing by sqrt(d_k)
   returns the variance to 1 and keeps the softmax in its
   useful range.

   SKIP THE SCALING AND DEEP TRANSFORMERS SIMPLY DO NOT
   TRAIN. It is one line and it is load-bearing.

/* --- WHERE Q, K AND V COME FROM --- */
    They are LEARNED LINEAR PROJECTIONS of the same input:

        Q = X @ W_Q
        K = X @ W_K
        V = X @ W_V

    W_Q, W_K, W_V are (d_model, d_k) parameter matrices.

    SELF-ATTENTION:  Q, K, V all from the SAME sequence.
                     "How do these tokens relate to each
                     other?"
    CROSS-ATTENTION: Q from one sequence, K and V from
                     another. Used in encoder-decoder
                     translation, and in multimodal models
                     where text queries attend to image
```

```
                      patches.

/* --- THE IMPLEMENTATION --- */
import torch
import torch.nn.functional as F
import math

def attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0,
                                    float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return weights @ V, weights

/* SEVEN LINES. That is the mechanism that made large
   language models possible.                              */
```

Masking

```
/* --- CAUSAL MASKING: the one that makes generation
       possible --- */

    A language model predicts the next token. If position 3
    could see position 4, it would be reading the answer -
    and the model would learn nothing useful and fail
    completely at inference, when the future does not exist.

    SO POSITION i MAY ATTEND ONLY TO POSITIONS 0..i.

    THE MASK (chapter 03's tril):

        [1 0 0 0]        the score matrix becomes
        [1 1 0 0]        [ s00  -inf -inf -inf ]
        [1 1 1 0]        [ s10   s11 -inf -inf ]
        [1 1 1 1]        [ s20   s21  s22 -inf ]
                         [ s30   s31  s32  s33 ]

    -inf becomes exactly 0 after softmax. NOT a small
    number - exactly zero, because exp(-inf) = 0.

    mask = torch.tril(torch.ones(n, n))
    scores = scores.masked_fill(mask == 0, float('-inf'))

/* WHY THIS IS BRILLIANT FOR TRAINING: with the causal mask,
   ONE forward pass over a sequence of length n produces n
   training examples simultaneously - each position predicts
   its own next token, and none of them can cheat. That is
   the parallelism the RNN could not offer (chapter 18), and
   it is the whole efficiency argument for transformers.

/* --- PADDING MASKS --- */
    Batched sequences are padded (chapter 16). Attending to
    padding pollutes the representation, so pad positions
    must also be masked to -inf.

    THE TWO MASKS COMBINE:
        mask = causal_mask & padding_mask

/* --- THE NUMERICAL TRAP --- */
    A row that is ENTIRELY -inf gives softmax(all -inf) =
    nan. This happens when a sequence is all padding, or
    when a mask is constructed wrongly.

    USE A LARGE FINITE NEGATIVE NUMBER instead of -inf if
    you cannot guarantee otherwise:
        scores.masked_fill(mask == 0, -1e9)
    And in fp16, -1e9 overflows - use -65504 or the dtype's
    minimum. This is a real and frequently-hit bug when
    moving a model to half precision.

/* --- BIDIRECTIONAL: NO CAUSAL MASK --- */
    BERT-style encoders let every position see every other
    position. Better representations for classification and
    retrieval, IMPOSSIBLE FOR GENERATION.

    THE FAMILIES:
      DECODER-ONLY (causal)    GPT, Llama, Mistral, Claude.
                               Generation.
      ENCODER-ONLY (bidir.)    BERT, RoBERTa. Classification,
```

```
                              embeddings.
    ENCODER-DECODER           T5, BART. Translation,
                              summarisation.
  DECODER-ONLY HAS WON for general-purpose models, largely
  because one simple objective plus scale beat
  architectural specialisation.                          */
```

THE COST, AND WHY CONTEXT LENGTH IS EXPENSIVE

```
The (n, n) score matrix means attention is
O(n² · d) in compute and O(n²) in memory.

  n =  1,000    1,000,000 scores      4 MB in fp32
  n = 10,000  100,000,000 scores    400 MB
  n = 100,000  10 BILLION scores      40 GB, per head,
                                      per layer

That is why context length was stuck at 512-2048 for
years, and why every efficiency technique in this
field is ultimately an attack on this quadratic.
```

Making it cheaper

```
/* --- FLASH ATTENTION: the one that actually mattered --- */
    THE INSIGHT: the (n, n) matrix is never needed in full.
    Process it in TILES that fit in on-chip SRAM, computing
    a running softmax with the online-normalisation trick, so
    the full score matrix is NEVER WRITTEN TO MAIN MEMORY.

      · MATHEMATICALLY IDENTICAL output - not an
        approximation
      · MEMORY becomes O(n) instead of O(n^2)
      · 2-4x FASTER, because attention is MEMORY BOUND
        (chapter 04) and this removes most of the traffic

    THIS IS A PURE SYSTEMS OPTIMISATION - cache blocking,
    exactly as in chapter 04's matmul - and it is the single
    largest practical advance in transformer efficiency. It
    is why long contexts became affordable.

    In PyTorch you get it for free:
        F.scaled_dot_product_attention(q, k, v,
                                       is_causal=True)

/* --- SPARSE ATTENTION PATTERNS --- */
    Do not attend to everything.
      SLIDING WINDOW   only the last w tokens. O(n*w).
                       Mistral uses this.
      DILATED          every k-th token, for a wider
                       receptive field
      GLOBAL + LOCAL   a few global tokens everyone sees,
                       plus a local window (Longformer,
                       BigBird)
      BLOCK-SPARSE     structured blocks that map well onto
                       hardware

/* --- LINEAR ATTENTION --- */
    Replace softmax with a kernel so the operation
    reassociates:
        (Q K^T) V   becomes   Q (K^T V)
    O(n^2 d) becomes O(n d^2). This is chapter 04's
    associativity argument, applied to make the sequence
    length linear.
    THE CATCH: quality has consistently lagged. Related to
    the state-space models of chapter 18.

/* --- MULTI-QUERY AND GROUPED-QUERY ATTENTION --- */
    Share K and V across attention heads.
      MHA   every head has its own K, V
      MQA   ALL heads share ONE K, V
      GQA   groups of heads share - the middle ground, and
            what Llama 2/3 and most current models use

    THIS DOES NOT REDUCE TRAINING COMPUTE. It reduces the KV
    CACHE SIZE AT INFERENCE by the sharing factor - often 8x
    - and since generation is memory-bandwidth bound
    (chapters 04, 25), that is a direct throughput win.

/* --- AND THE PRACTICAL SUMMARY --- */
    · use F.scaled_dot_product_attention, always. It selects
      the best available kernel.
    · GQA for inference-heavy deployments
    · sliding-window if you need very long contexts cheaply
```

```
    · and do not implement attention by hand in production -
      write it once, as in this chapter, to understand it,
      then use the fused kernel.                          */
```

What attention learns

```
/* THE WEIGHTS ARE INSPECTABLE, which makes this one of the
   few interpretable parts of a modern model.            */

    out, weights = attention(Q, K, V)
    # weights: (n, n) - row i shows what position i
    # attended to

    import matplotlib.pyplot as plt
    plt.imshow(weights.detach(), cmap='Blues')

/* PATTERNS THAT SHOW UP CONSISTENTLY ACROSS MODELS:

   POSITIONAL HEADS      attend to the previous token, or
                         the one before that. Effectively
                         learned n-gram features.
   SYNTACTIC HEADS       verbs to their subjects, adjectives
                         to their nouns, and matching
                         brackets in code
   COREFERENCE HEADS     pronouns to their antecedents -
                         exactly the "it" example above
   DELIMITER HEADS       attend heavily to the first token
                         or to punctuation. Often an
                         ATTENTION SINK: a place to dump
                         probability mass when the head has
                         nothing useful to do, since softmax
                         forces the row to sum to 1.
   INDUCTION HEADS       the important one. They implement
                         "[A][B] ... [A] -> predict [B]" -
                         copying a pattern seen earlier in
                         the context. They FORM ABRUPTLY
                         during training, and their
                         appearance coincides with the model
                         acquiring IN-CONTEXT LEARNING -
                         the ability to pick up a pattern
                         from examples in the prompt.

   THE INDUCTION HEAD RESULT IS WORTH KNOWING because it is
   one of the clearest cases where a specific circuit was
   identified and connected to a capability. Most
   interpretability is not this clean.

/* AND THE CAVEAT, because attention maps are over-read:
   ATTENTION WEIGHTS ARE NOT AN EXPLANATION. A head can
   attend strongly to a token whose value vector contributes
   almost nothing, and information also flows through the
   residual stream and the feed-forward layers (chapter 22).
   "The model attended to X, therefore it used X" is not a
   valid inference. Attention maps are a useful signal and a
   poor proof.                                           */
```

How it shows up

Symptom	Cause
Deep transformer will not train	Missing `1/√d_k` scaling — the softmax saturates
Loss near zero, generation is garbage	No causal mask — the model read the answer
`nan` in attention weights	A fully masked row — softmax of all `-inf`
`nan` only in fp16	`-1e9` overflows — use the dtype minimum
Out of memory at long sequence length	The (n, n) matrix — use Flash Attention
Model attends to padding	Padding mask missing or not combined with the causal mask
Attention is slower than the FLOPs suggest	Memory bound — use the fused kernel
Generation throughput collapses at long context	KV cache bandwidth — consider GQA
Shuffling input tokens changes nothing	No positional encoding — attention is permutation-invariant
A head attends almost entirely to token 0	An attention sink — usually normal, not a bug

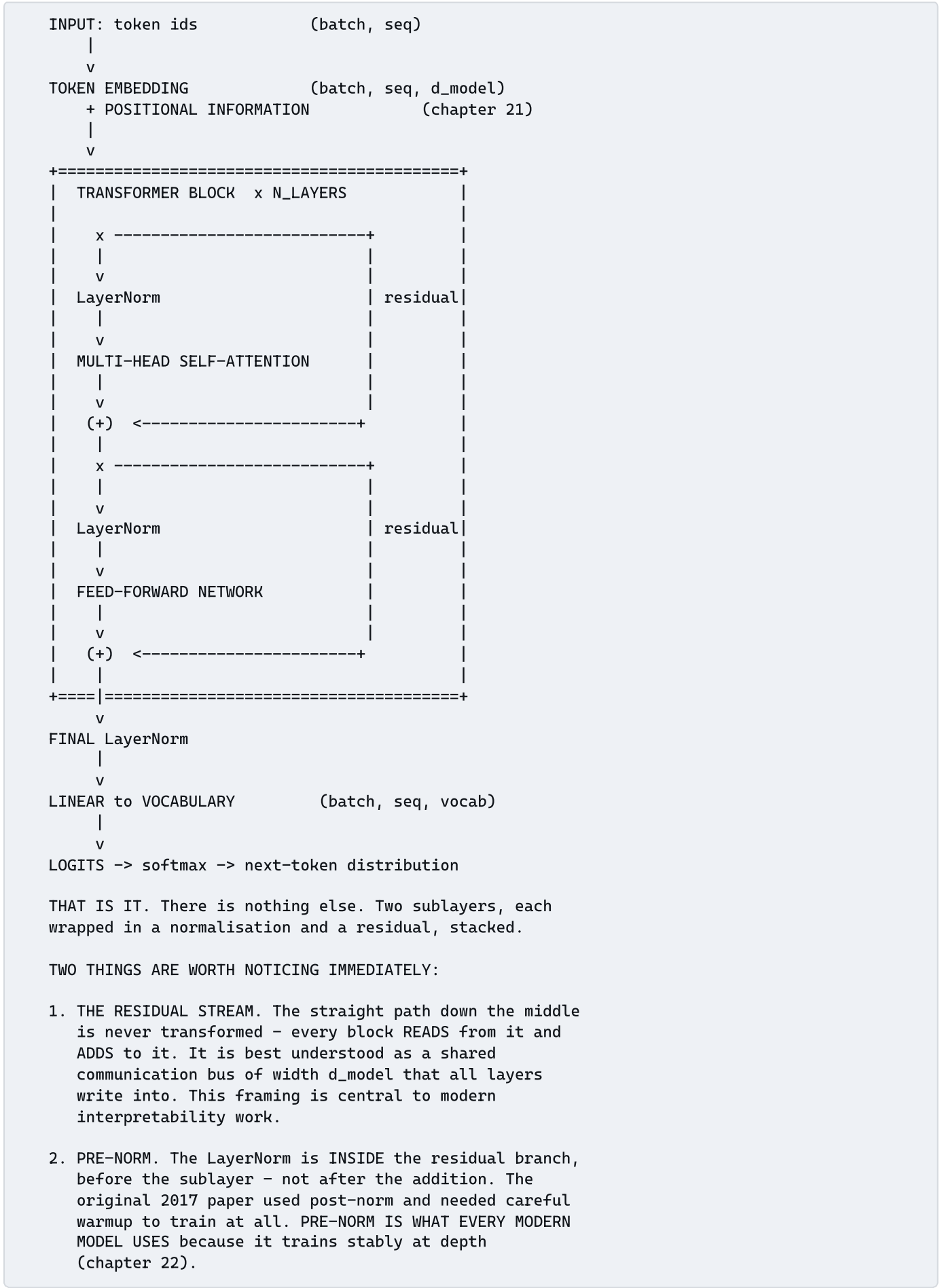
Check yourself

- ✓ Explain query, key and value as a soft dictionary lookup.
- ✓ Write the attention equation from memory with shapes at every step.
- ✓ Derive why the scaling factor is `1/√d_k` and say what happens without it.
- ✓ Distinguish self-attention from cross-attention.
- ✓ Draw the causal mask and explain why `-inf` becomes exactly zero.
- ✓ Explain why causal masking yields n training examples from one forward pass.
- ✓ Explain the fully-masked-row `nan` and the fp16 variant.
- ✓ Compute the attention memory cost at n = 100,000.
- ✓ Explain what Flash Attention does and why it is exact rather than approximate.
- ✓ Explain what GQA reduces and why that helps generation specifically.
- ✓ Name five attention patterns, and say what induction heads do.
- ✓ State why attention weights are not an explanation.

The transformer architecture

Attention, a feed-forward network, normalisation and a residual connection, repeated N times. Every large language model in existence is this diagram with different numbers in it.

The whole thing



THAT IS IT. There is nothing else. Two sublayers, each
wrapped in a normalisation and a residual, stacked.

TWO THINGS ARE WORTH NOTICING IMMEDIATELY:

1. THE RESIDUAL STREAM. The straight path down the middle
   is never transformed - every block READS from it and
   ADDS to it. It is best understood as a shared
   communication bus of width d_model that all layers
   write into. This framing is central to modern
   interpretability work.

2. PRE-NORM. The LayerNorm is INSIDE the residual branch,
   before the sublayer - not after the addition. The
   original 2017 paper used post-norm and needed careful
   warmup to train at all. PRE-NORM IS WHAT EVERY MODERN
   MODEL USES because it trains stably at depth
   (chapter 22).

The numbers

Model	d_model	Layers	Heads	Params
Original (2017)	512	6	8	65 M
BERT-base	768	12	12	110 M
GPT-2 small	768	12	12	124 M
GPT-2 XL	1600	48	25	1.5 B
GPT-3	12288	96	96	175 B
Llama 3 8B	4096	32	32 (8 KV)	8 B
Llama 3 70B	8192	80	64 (8 KV)	70 B

```
/* THE PARAMETER COUNT, per layer, in terms of d = d_model:

    ATTENTION
      W_Q, W_K, W_V, W_O    4 * d * d      = 4 d^2
    FEED-FORWARD (d_ff = 4d, the standard ratio)
      W_1: d x 4d, W_2: 4d x d             = 8 d^2
    LayerNorms                              ~ negligible
                                            ----------
                           PER LAYER       = 12 d^2

    TOTAL ~= 12 * n_layers * d^2  +  vocab * d

    CHECK IT AGAINST GPT-3:
      12 * 96 * 12288^2 = 174 billion.  Correct.

    NOTE THE PROPORTIONS: the FEED-FORWARD NETWORK IS TWO
    THIRDS OF THE PARAMETERS. Attention gets the attention;
    the FFN holds the weights. Chapter 22 discusses what it
    is believed to be doing.

/* SOME CONVENTIONS THAT ARE ALMOST ALWAYS TRUE:
     d_head  = d_model / n_heads      (usually 64 or 128)
     d_ff    = 4 * d_model            (or 8/3 * d for gated
                                       SwiGLU variants,
                                       which keeps the
                                       parameter count
                                       equal)
     depth and width scale together as models grow

/* AND THE MEMORY, which is what you actually plan around:
     WEIGHTS, fp16          2 bytes/param
       7B model  -> 14 GB
       70B model -> 140 GB   (chapter 26 halves or quarters
                              this)
     KV CACHE per token:
       2 * n_layers * n_kv_heads * d_head * 2 bytes
       Llama-3-8B: 2*32*8*128*2 = 131 KB PER TOKEN
       -> 8k context = 1 GB. This is why GQA exists
          (chapter 19).                                  */
```

The complete implementation

```
"""A working GPT-style transformer. This is the real thing -
   about a hundred lines, and it is what chapter 28 trains."""

import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class Config:
    vocab_size = 50257
    n_layer    = 12
    n_head     = 12
    d_model    = 768
    block_size = 1024        # max sequence length
    dropout    = 0.1

class CausalSelfAttention(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        assert cfg.d_model % cfg.n_head == 0
        # Q, K and V in ONE matrix - a single larger matmul
        # is faster than three (chapter 04)
        self.qkv  = nn.Linear(cfg.d_model, 3 * cfg.d_model)
        self.proj = nn.Linear(cfg.d_model, cfg.d_model)
        self.n_head = cfg.n_head
        self.d_head = cfg.d_model // cfg.n_head
        self.dropout = cfg.dropout

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)

        # (B, T, C) -> (B, n_head, T, d_head)
        # so the batched matmul runs every head at once
        q = q.view(B, T, self.n_head,
                   self.d_head).transpose(1, 2)
        k = k.view(B, T, self.n_head,
                   self.d_head).transpose(1, 2)
        v = v.view(B, T, self.n_head,
                   self.d_head).transpose(1, 2)

        # Flash Attention when available (chapter 19)
        y = F.scaled_dot_product_attention(
                q, k, v, is_causal=True,
                dropout_p=self.dropout if self.training
                          else 0.0)

        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(y)

class MLP(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc   = nn.Linear(cfg.d_model, 4 * cfg.d_model)
        self.proj = nn.Linear(4 * cfg.d_model, cfg.d_model)
        self.drop = nn.Dropout(cfg.dropout)
    def forward(self, x):
        return self.drop(self.proj(F.gelu(self.fc(x))))

class Block(nn.Module):
```

```
    def __init__(self, cfg):
        super().__init__()
        self.ln1 = nn.LayerNorm(cfg.d_model)
        self.attn = CausalSelfAttention(cfg)
        self.ln2 = nn.LayerNorm(cfg.d_model)
        self.mlp = MLP(cfg)
    def forward(self, x):
        # PRE-NORM, and note that x is returned unchanged
        # through the additions - that is the residual stream
        x = x + self.attn(self.ln1(x))
        x = x + self.mlp(self.ln2(x))
        return x

class GPT(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
        self.wte = nn.Embedding(cfg.vocab_size, cfg.d_model)
        self.wpe = nn.Embedding(cfg.block_size, cfg.d_model)
        self.drop = nn.Dropout(cfg.dropout)
        self.blocks = nn.ModuleList(
            [Block(cfg) for _ in range(cfg.n_layer)])
        self.ln_f = nn.LayerNorm(cfg.d_model)
        self.head = nn.Linear(cfg.d_model, cfg.vocab_size,
                              bias=False)

        # WEIGHT TYING (chapter 17)
        self.head.weight = self.wte.weight

        self.apply(self._init)
        # scale the residual projections by 1/sqrt(2*n_layer)
        # (chapter 15)
        for name, p in self.named_parameters():
            if name.endswith('proj.weight'):
                nn.init.normal_(p, mean=0.0, std=0.02 /
                                math.sqrt(2 * cfg.n_layer))

    def _init(self, m):
        if isinstance(m, (nn.Linear, nn.Embedding)):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)
            if isinstance(m, nn.Linear) and m.bias is not None:
                nn.init.zeros_(m.bias)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.drop(self.wte(idx) + self.wpe(pos))
        for block in self.blocks:
            x = block(x)
        x = self.ln_f(x)
        logits = self.head(x)              # (B, T, vocab)

        loss = None
        if targets is not None:
            # EVERY POSITION PREDICTS THE NEXT TOKEN -
            # T training examples from one pass (chapter 19)
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)),
                targets.view(-1),
                ignore_index=-1)
        return logits, loss

model = GPT(Config())
```

```
n = sum(p.numel() for p in model.parameters())
print(f"{n/1e6:.1f}M parameters")     # ~124M - GPT-2 small
```

THE WHOLE TRAINING OBJECTIVE

```
input:   [The, cat, sat, on, the]
targets: [cat, sat, on, the, mat]

Shift by one. Cross-entropy at every position. That is
the entire pretraining objective of every large language
model - no labels, no annotation, just text
(chapter 23).
```

The families, and what changed since 2017

```
/* --- THE THREE ARCHITECTURES --- */
    ENCODER-ONLY     bidirectional attention, masked-token
                     objective. BERT, RoBERTa, DeBERTa.
                     Classification, NER, EMBEDDINGS
                     (chapter 17).
    DECODER-ONLY     causal attention, next-token objective.
                     GPT, Llama, Mistral, Claude, Gemma.
                     GENERATION - and, it turned out,
                     everything else too.
    ENCODER-DECODER  cross-attention from decoder to
                     encoder. T5, BART, Whisper.
                     Translation, summarisation,
                     speech-to-text.

    DECODER-ONLY DOMINATES general-purpose models. The
    lesson the field drew: one simple objective plus scale
    outperformed architectural specialisation - the same
    lesson as chapter 14's prior-strength trade-off.

/* --- WHAT ACTUALLY CHANGED BETWEEN 2017 AND NOW --- */

    2017 ORIGINAL              MODERN (Llama 3 and similar)
    ------------------------------------------------------
    post-norm                  PRE-NORM
    LayerNorm                  RMSNORM (cheaper, no mean)
    learned absolute positions  ROPE (rotary, relative)
    ReLU FFN                   SWIGLU (gated)
    MHA                        GQA (smaller KV cache)
    d_ff = 4d                  d_ff = 8/3 d (to keep the
                               parameter count with gating)
    dropout 0.1                often 0 for pretraining
    biases everywhere          MOSTLY NO BIASES
    2k context                 128k+ context

    EVERY ONE OF THESE IS A SMALL CHANGE. There has been no
    architectural revolution since 2017 - the transformer
    block is essentially unchanged, and the gains have come
    from SCALE, DATA and SYSTEMS ENGINEERING. That is worth
    stating plainly, because it is not the impression the
    press gives.

/* --- MIXTURE OF EXPERTS, the one structural change --- */
    Replace the FFN with N expert FFNs and a router that
    sends each token to the top-k of them (usually k = 2).

      · TOTAL parameters go up enormously
      · ACTIVE parameters per token stay small
      · so capacity grows without proportional compute

    Mixtral 8x7B: 47B total, ~13B active per token.
    THE COSTS: all experts must be in memory, load balancing
    across experts needs an auxiliary loss, and distributed
    training becomes considerably harder.

/* --- AND THE MULTIMODAL EXTENSION --- */
    Nothing about the block is text-specific. Feed it image
    patches (ViT), audio frames (Whisper), or a mixture, and
    the same architecture applies. Chapter 17's point
```

```
    exactly: once everything is a sequence of vectors, the
    transformer does not care.                          */
```

How it shows up

Symptom	Cause
Training diverges past ~12 layers	Post-norm without warmup — use pre-norm
Loss drops to near zero, generation fails	No causal mask
Parameter count is far off the expected value	Check the $12 \cdot n_{layer} \cdot d^2$ estimate — usually a wrong `d_ff` or untied weights
Output is identical for shuffled input	Positional embedding missing or not added
Sequence longer than `block_size` crashes	Learned absolute positions have a hard limit
Activations grow with depth	Residual projections not scaled at initialisation
KV cache dominates memory	Full MHA — use GQA
Attention is slower than expected	Manual implementation instead of the fused kernel
MoE model has idle experts	No load-balancing auxiliary loss
Model is huge but the FFN is small	Wrong `d_ff` ratio — it should be two thirds of the parameters

Check yourself

- ✓ Draw the transformer block from memory, including both residuals.
- ✓ Explain the residual stream as a communication bus.
- ✓ Explain pre-norm versus post-norm and which trains at depth.
- ✓ Derive the $12 \cdot d^2$ per-layer parameter count and verify it against GPT-3.
- ✓ Say which sublayer holds two thirds of the parameters.
- ✓ Compute the KV cache per token for Llama-3-8B.
- ✓ Implement causal self-attention including the head reshape.
- ✓ Explain the input/target shift and why one pass gives T examples.
- ✓ Name the three architecture families and what each is used for.
- ✓ List eight changes between the 2017 transformer and a modern one.
- ✓ Explain mixture of experts and its three costs.

Multi-head attention and positional encoding

Two details that the previous chapter glossed over, and both matter more than their size suggests. One gives attention its expressive power; the other is the only thing telling the model that word order exists.

Multi-head attention

```
ONE ATTENTION HEAD PRODUCES ONE WEIGHTED AVERAGE. But a
token often needs several unrelated things at once:

  "The bank by the river was steep."

  · what is the SUBJECT of "was"?          -> "bank"
  · which sense of "bank" is this?         -> "river"
  · what is the previous token?            -> "was"

THREE DIFFERENT QUESTIONS, THREE DIFFERENT WEIGHTINGS.
One softmax-weighted average cannot express all of them -
whatever it emphasises, it de-emphasises the rest.

THE SOLUTION: RUN h ATTENTION OPERATIONS IN PARALLEL,
EACH IN A LOWER-DIMENSIONAL SUBSPACE, AND CONCATENATE.

     d_head = d_model / h

  x --> [W_Q1,W_K1,W_V1] --> attention --> head_1 \
    --> [W_Q2,W_K2,W_V2] --> attention --> head_2  \
    --> ...                                         >-- concat --> W_O --> out
    --> [W_Qh,W_Kh,W_Vh] --> attention --> head_h  /

THE PARAMETER COUNT IS UNCHANGED. Splitting 768
dimensions into 12 heads of 64 costs exactly the same as
one head of 768 - you are reshaping, not adding.

WHAT YOU BUY: h INDEPENDENT ATTENTION PATTERNS instead
of one. Empirically this matters a great deal; ablating
to a single head degrades models badly.

AND THE OUTPUT PROJECTION W_O IS NOT DECORATION. The
concatenated heads are just stacked subspaces; W_O mixes
them back into the residual stream and decides how each
head's contribution is written. Interpretability work
often factors attention as (W_Q W_K^T) for "where to
look" and (W_V W_O) for "what to write".

TYPICAL: d_head is 64 or 128, essentially always. Models
grow by adding HEADS and LAYERS, not by widening heads.
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.1):
        super().__init__()
        assert d_model % n_head == 0
        self.n_head = n_head
        self.d_head = d_model // n_head
        # ONE fused projection for Q, K, V (chapter 04:
        # one big matmul beats three small ones)
        self.qkv  = nn.Linear(d_model, 3 * d_model,
                              bias=False)
        self.proj = nn.Linear(d_model, d_model, bias=False)
        self.dropout = dropout

    def forward(self, x, is_causal=True):
        B, T, C = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)

        # THE RESHAPE THAT MAKES HEADS WORK:
        # (B, T, C) -> (B, n_head, T, d_head)
        # The head dimension becomes a BATCH dimension, so
        # one batched matmul runs every head simultaneously.
        def split(t):
            return t.view(B, T, self.n_head,
                          self.d_head).transpose(1, 2)
        q, k, v = split(q), split(k), split(v)

        y = F.scaled_dot_product_attention(
                q, k, v, is_causal=is_causal,
                dropout_p=self.dropout if self.training
                          else 0.0)

        # (B, n_head, T, d_head) -> (B, T, C)
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(y)

# THE .contiguous() IS NOT OPTIONAL. transpose() returns a
# VIEW with permuted strides; view() requires contiguous
# memory. Omitting it raises an error - and if you use
# reshape() instead, it silently copies, which works but
# hides the cost.

# --- GROUPED-QUERY ATTENTION (chapter 19) ---
class GQA(nn.Module):
    """n_head query heads, n_kv_head key/value heads.
       n_kv_head = n_head    -> standard MHA
       n_kv_head = 1         -> multi-query
       in between            -> GQA, what Llama uses"""
    def __init__(self, d_model, n_head, n_kv_head):
        super().__init__()
        self.n_head, self.n_kv_head = n_head, n_kv_head
        self.d_head = d_model // n_head
        self.q = nn.Linear(d_model, n_head * self.d_head,
                           bias=False)
        self.k = nn.Linear(d_model, n_kv_head * self.d_head,
                           bias=False)
        self.v = nn.Linear(d_model, n_kv_head * self.d_head,
                           bias=False)
        self.proj = nn.Linear(d_model, d_model, bias=False)
```

```
    def forward(self, x):
        B, T, C = x.shape
        q = self.q(x).view(B, T, self.n_head,
                           self.d_head).transpose(1, 2)
        k = self.k(x).view(B, T, self.n_kv_head,
                           self.d_head).transpose(1, 2)
        v = self.v(x).view(B, T, self.n_kv_head,
                           self.d_head).transpose(1, 2)

        # REPEAT the KV heads to match the query heads.
        # No extra parameters, and the CACHE only stores
        # n_kv_head copies - which is the entire point.
        rep = self.n_head // self.n_kv_head
        k = k.repeat_interleave(rep, dim=1)
        v = v.repeat_interleave(rep, dim=1)

        y = F.scaled_dot_product_attention(q, k, v,
                                           is_causal=True)
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(y)
```

Why position must be added

```
/* ATTENTION IS PERMUTATION-EQUIVARIANT.

   Shuffle the input tokens and the outputs shuffle
   identically - the VALUES do not change. Look at the
   equation (chapter 19): it is a set of dot products and a
   weighted sum. NOTHING IN IT REFERS TO POSITION.

   So without positional information:

        "dog bites man"  ==  "man bites dog"

   The model literally cannot distinguish them. POSITION
   MUST BE INJECTED EXPLICITLY, and how you do it turns out
   to determine how far the model can extrapolate beyond its
   training length.                                        */

/* --- 1. LEARNED ABSOLUTE (GPT-2, BERT) --- */
    self.wpe = nn.Embedding(max_len, d_model)
    x = self.wte(idx) + self.wpe(positions)

    A lookup table, one vector per position, learned like
    any other parameter.
      + simple, and it works
      - HARD LIMIT at max_len. Position 1025 has no
        embedding. Cannot extrapolate at all.
      - positions are learned independently, so nothing
        forces position 5 and position 6 to be related

/* --- 2. SINUSOIDAL (the 2017 paper) --- */
    PE(pos, 2i)   = sin(pos / 10000^(2i/d))
    PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

    Different dimensions oscillate at different frequencies -
    a binary-counter-like encoding in continuous form. The
    intended property: PE(pos+k) is a LINEAR FUNCTION of
    PE(pos), so relative offsets are learnable.
      + no parameters, defined for any position
      - extrapolation is poor in practice despite the theory

/* --- 3. ROPE: ROTARY POSITION EMBEDDING. THE STANDARD. --- */

    THE IDEA: do not ADD anything to the embeddings. ROTATE
    the query and key vectors by an angle proportional to
    their position.

      Treat each pair of dimensions as a 2-D vector and
      rotate it by (position * theta_i), with theta_i
      decreasing across dimension pairs.

    WHY IT IS ELEGANT: after rotation, the dot product
    between a query at position m and a key at position n
    depends only on (m - n).

        q_m · k_n  =  f(q, k, m - n)

    RELATIVE POSITION FALLS OUT OF THE ARITHMETIC. No
    relative-position bias table, no extra parameters, and
    it applies to Q and K only - the values and the residual
    stream are untouched.
```

```
    · used by Llama, Mistral, Gemma, Qwen, and essentially
      every recent open model
    · extrapolates better than the alternatives
    · and CONTEXT EXTENSION IS POSSIBLE AFTER TRAINING by
      scaling the frequencies - "NTK-aware" scaling, YaRN,
      position interpolation. This is how an 8k model
      becomes a 128k model with modest fine-tuning, and it
      is the main reason context lengths grew so fast.

def rope(x, positions, base=10000.0):
    """x: (B, n_head, T, d_head), d_head even."""
    B, H, T, D = x.shape
    half = D // 2
    freqs = 1.0 / (base ** (torch.arange(0, half,
                    device=x.device).float() / half))
    angles = positions[:, None].float() * freqs[None, :]
    cos, sin = angles.cos(), angles.sin()      # (T, half)

    x1, x2 = x[..., :half], x[..., half:]
    return torch.cat([x1 * cos - x2 * sin,
                      x1 * sin + x2 * cos], dim=-1)

# APPLY TO Q AND K ONLY, NOT V:
#     q = rope(q, pos);  k = rope(k, pos)

/* --- 4. ALiBi --- */
    Add a linear penalty to the attention scores based on
    distance, with a different slope per head:
        score = q·k - m_head * |i - j|
    No embeddings at all. Extrapolates well; nearer tokens
    are simply favoured. Used by BLOOM and MPT.

/* --- 5. NoPE --- */
    And a genuinely surprising result: DECODER-ONLY models
    with NO POSITIONAL ENCODING AT ALL still learn position,
    because the causal mask itself is positional information
    - position i can see exactly i+1 tokens, and that count
    is recoverable. Performance is competitive at small
    scale. A good reminder that architectural "necessities"
    deserve testing.                                      */
```

WHAT TO USE

```
new model, any size        -> RoPE
need long extrapolation    -> RoPE with frequency
                              scaling, or ALiBi
reproducing GPT-2/BERT     -> learned absolute
encoder for fixed-length
  classification           -> learned absolute is fine
tiny embedded model,
  short sequences          -> learned absolute; it is one
                              small table and the cheapest
                              to run
```

What the heads do

```
/* HEADS SPECIALISE, without being told to. The patterns
   from chapter 19, seen through the multi-head lens:

     · POSITIONAL heads   attend to the previous token, or
                          n back. Early layers.
     · SYNTACTIC heads    subject-verb, adjective-noun,
                          matching brackets
     · COREFERENCE heads  pronoun to antecedent
     · INDUCTION heads    copy a pattern seen earlier in
                          context. Requires TWO heads
                          working together across layers -
                          one to record "what followed this
                          token last time", one to use it.
                          Their formation coincides with
                          in-context learning appearing.
     · SINK heads         dump attention on token 0

/* --- HEAD REDUNDANCY --- */
    A well-known result: MANY HEADS CAN BE PRUNED with
    little loss. Some layers work fine with a single head.
    Others have one critical head whose removal breaks a
    capability entirely.

    THIS IS PRACTICALLY USEFUL: head pruning is a real
    compression technique (chapter 26), and it is structured
    - removing a whole head removes whole rows and columns,
    which actually speeds up the matmul, unlike unstructured
    sparsity (chapter 03).

/* --- INSPECTING THEM --- */
    def get_attention_maps(model, ids):
        maps = []
        def hook(mod, inp, out):
            q, k, _ = mod.qkv(inp[0]).chunk(3, dim=-1)
            # recompute the scores just to look at them
            ...
            maps.append(weights.detach())
        handles = [b.attn.register_forward_hook(hook)
                   for b in model.blocks]
        model(ids)
        for h in handles: h.remove()
        return maps       # (n_layer, B, n_head, T, T)

    AND THE CAVEAT FROM CHAPTER 19 STILL APPLIES: an
    attention map shows where a head LOOKED, not what the
    model USED. Treat it as a hypothesis generator, not
    evidence.                                              */
```

How it shows up

Symptom	Cause
Word order has no effect on output	No positional encoding
Crash at sequence length above the trained maximum	Learned absolute positions — no embedding exists
Quality collapses beyond the training length	Poor extrapolation — use RoPE with scaling or ALiBi
`view size is not compatible`	Missing `.contiguous()` after `transpose`

Symptom	Cause
Heads produce identical patterns	Initialisation or a reshape bug collapsing the head dimension
Output is scrambled across heads	Wrong reshape order — check `(B, T, H, D)` then transpose
RoPE applied to values	It belongs on Q and K only
KV cache far larger than expected	Full MHA rather than GQA
Extended context degrades badly	Frequencies not rescaled for the new length
Pruning heads gives no speedup	Masked rather than physically removed

Check yourself

- ✓ Explain why one attention head is insufficient, with a concrete sentence.
- ✓ Show that multi-head attention costs the same parameters as single-head.
- ✓ Explain what `W_O` does and the (W_Q W_K) / (W_V W_O) factorisation.
- ✓ Write the reshape that turns heads into a batch dimension, and say why `.contiguous()` is needed.
- ✓ Explain GQA's repeat step and what it saves.
- ✓ Explain why attention is permutation-equivariant and what that implies.
- ✓ Compare learned absolute, sinusoidal, RoPE and ALiBi.
- ✓ State RoPE's key property and why it gives relative position for free.
- ✓ Explain how context extension works after training.
- ✓ Say why decoder-only models can work with no positional encoding at all.
- ✓ Name five head specialisations and say what induction heads require.
- ✓ Explain why head pruning is structured sparsity and why that matters.

Normalisation, residuals and the feed-forward block

The unglamorous two thirds of a transformer. Residual connections are what make depth trainable, normalisation is what keeps it stable, and the feed-forward network holds most of the parameters and, probably, most of the knowledge.

Residual connections

```
      x ------------------+
      |                   |
      v                   |
   [ sublayer ]           |
      |                   |
      v                   |
     (+) <----------------+
      |
      v
    output = x + sublayer(x)

THE GRADIENT ARGUMENT, and it is the whole reason:

      d(output)/dx = 1 + d(sublayer)/dx
                     ^
                     THE IDENTITY PATH

Backpropagating through n layers multiplies n such terms.
Without the residual, the product is of n Jacobians and
vanishes or explodes exponentially (chapter 07). WITH IT,
the "1" guarantees a path along which the gradient
arrives essentially UNCHANGED, however deep the stack.

This is the same structural trick as the LSTM's cell
state (chapter 18) - give the gradient a route that does
not pass through a transformation.

BEFORE RESNET (2015), 20 layers was hard. AFTER, 1000+
layers train. IT IS THE SINGLE MOST IMPORTANT
ARCHITECTURAL IDEA OF THE DECADE, and it is one addition.

THE OTHER READING - THE RESIDUAL STREAM:

  Think of x as a SHARED BUS of width d_model that runs
  the full depth of the model. Each sublayer:
     · READS from the bus (through its LayerNorm)
     · COMPUTES something
     · ADDS its result back

  Nothing overwrites. Information written by layer 3
  survives to layer 40 unless something explicitly
  subtracts it. Layers can therefore communicate at a
  distance, and interpretability work treats the residual
  stream as the model's working memory.

  A CONSEQUENCE WORTH KNOWING: THE RESIDUAL STREAM'S NORM
  GROWS WITH DEPTH, because every layer adds to it. That
  is why the residual projections are initialised scaled
  down by 1/sqrt(2*n_layers) (chapter 15), and why the
  final LayerNorm before the output head is necessary.
```

Normalisation

```
/* --- LAYERNORM --- */
    mu    = mean(x, over the FEATURE dimension)
    sigma = std(x,  over the FEATURE dimension)
    y     = gamma * (x - mu) / (sigma + eps) + beta

    gamma and beta are LEARNED per-feature scale and shift,
    so the layer can undo the normalisation if that is
    useful.

    NORMALISED PER SAMPLE, PER POSITION, ACROSS FEATURES.
    No dependence on the batch at all - which is why
    transformers use it (chapter 15).

    nn.LayerNorm(d_model)      # (B, T, d) -> normalises
                               # over d

/* --- RMSNORM: what modern models use --- */
    y = gamma * x / sqrt(mean(x^2) + eps)

    NO MEAN SUBTRACTION, NO BETA. Just rescale to unit root
    mean square.

      · ~10-15% faster (one reduction instead of two)
      · fewer parameters
      · EMPIRICALLY JUST AS GOOD - which suggests the
        re-CENTRING was never the active ingredient; the
        re-SCALING was.

    Llama, Mistral, Gemma, T5 and most recent models use it.

    class RMSNorm(nn.Module):
        def __init__(self, d, eps=1e-6):
            super().__init__()
            self.weight = nn.Parameter(torch.ones(d))
            self.eps = eps
        def forward(self, x):
            rms = x.pow(2).mean(-1,
                                keepdim=True).add(self.eps).rsqrt()
            return self.weight * (x * rms)

/* --- PRE-NORM VERSUS POST-NORM. THIS ONE MATTERS. --- */

    POST-NORM (the 2017 paper):
        x = LayerNorm(x + sublayer(x))

        The normalisation is ON the residual path. So the
        gradient must pass through a LayerNorm at EVERY
        layer, and the clean identity path is destroyed.
        RESULT: deep post-norm transformers are hard to
        train and need careful warmup, and past ~12-18
        layers they frequently diverge.

    PRE-NORM (everything modern):
        x = x + sublayer(LayerNorm(x))

        The normalisation is INSIDE the branch. The residual
        path is a pure addition, so the identity gradient
        survives intact.
        RESULT: stable training at any depth, far less
        sensitivity to warmup and learning rate.
```

```
    THIS IS A ONE-LINE DIFFERENCE THAT DETERMINES WHETHER A
    100-LAYER MODEL TRAINS AT ALL. If you are reading old
    code and it will not converge, check this first.

    (Pre-norm's cost: the residual stream grows unnormalised
    with depth, which is why a FINAL LayerNorm before the
    output head is required. Some models add extra norms -
    "sandwich" or QK-norm - for stability at very large
    scale.)

/* --- WHY NORMALISATION HELPS AT ALL --- */
    The original claim was "reducing internal covariate
    shift". That explanation has not held up well.

    THE BETTER-SUPPORTED VIEW: it SMOOTHS THE LOSS
    LANDSCAPE, so gradients are more predictable and larger
    learning rates become usable. It also makes the model
    invariant to the SCALE of the weights, which decouples
    the effective learning rate from the weight magnitude.

    THE HONEST POSITION: IT WORKS RELIABLY AND THE THEORY IS
    STILL DEBATED. That is true of several things in this
    chapter, and it is better to know it than to repeat a
    tidy explanation that is probably wrong.            */
```

The feed-forward network

```
/* THE SIMPLEST PART OF THE ARCHITECTURE AND TWO THIRDS OF
   THE PARAMETERS (chapter 20).                          */

    FFN(x) = W_2 @ activation(W_1 @ x + b_1) + b_2

        W_1: (d_model, 4*d_model)     EXPAND
        W_2: (4*d_model, d_model)     PROJECT BACK

    APPLIED INDEPENDENTLY AT EVERY POSITION. There is no
    mixing across the sequence here at all - that is
    attention's job. The FFN is a per-token transformation.

    THE DIVISION OF LABOUR IN A TRANSFORMER BLOCK:
        ATTENTION  moves information BETWEEN positions
        FFN        processes information AT a position

/* --- SWIGLU: what modern models use --- */
    SwiGLU(x) = (SiLU(x @ W_gate) * (x @ W_up)) @ W_down

    A GATED linear unit: one branch produces values, the
    other produces a multiplicative gate. Three matrices
    instead of two, so d_ff is reduced from 4d to
    (8/3)d to keep the parameter count equal.

    Consistently a small quality improvement over ReLU or
    GELU FFNs. Nobody has a fully satisfying explanation;
    the gating presumably lets the layer suppress its own
    output conditionally.

    class SwiGLU(nn.Module):
        def __init__(self, d, hidden):
            super().__init__()
            self.gate = nn.Linear(d, hidden, bias=False)
            self.up   = nn.Linear(d, hidden, bias=False)
            self.down = nn.Linear(hidden, d, bias=False)
        def forward(self, x):
            return self.down(F.silu(self.gate(x))
                             * self.up(x))

/* --- WHAT THE FFN IS BELIEVED TO DO --- */

    THE KEY-VALUE MEMORY VIEW, which has real evidence behind
    it:
      · each ROW of W_1 acts as a PATTERN DETECTOR - the
        dot product with x fires when a particular feature is
        present
      · the activation gates it
      · each COLUMN of W_2 is the VALUE written back into the
        residual stream when that pattern fires

      So the FFN is a large associative memory: "when you see
      this pattern, add this to the stream". FACTUAL RECALL
      HAS BEEN LOCALISED TO SPECIFIC FFN NEURONS in several
      studies - the ROME and MEMIT model-editing work
      modifies exactly these weights to change a stored fact.

    THE 4x EXPANSION gives room for many more patterns than
    d_model dimensions - superposition again (chapter 17).

/* --- MIXTURE OF EXPERTS, which is an FFN change --- */
```

```
   Replace the single FFN with N of them plus a router:

       gate   = softmax(x @ W_router)        # (N,)
       top_k  = argtopk(gate, k=2)
       out    = sum over the chosen experts of
                gate_i * Expert_i(x)

     · total parameters up, ACTIVE parameters per token
       unchanged
     · requires a LOAD-BALANCING AUXILIARY LOSS, or the
       router collapses onto a few favourite experts and
       the rest never train
     · all experts must be resident in memory even though
       only k run - so it saves COMPUTE, not MEMORY, which
       is exactly the wrong trade for an embedded device

   NOTE THAT MoE MODIFIES THE FFN, NOT ATTENTION. That is
   not a coincidence: the FFN is where the parameters are.
                                                        */
```

THE BLOCK, IN ONE SENTENCE EACH

```
RESIDUAL     gives the gradient an identity path, so
             depth is trainable.
LAYERNORM    keeps activations in a usable range and
             smooths the landscape.
PRE-NORM     keeps the residual path clean, so very
             deep models train.
ATTENTION    moves information between positions.
FFN          transforms information at each position,
             and stores most of what the model knows.
```

The cost picture

```
/* PER LAYER, per token, in FLOPs (roughly, ignoring
   constants):

     ATTENTION projections   8 * d^2      (Q,K,V,O)
     ATTENTION scores+AV     4 * n * d    (n = sequence
                                           length)
     FFN                    16 * d^2      (4d expansion,
                                           two matmuls)
     norms                   negligible

   SO FOR SHORT SEQUENCES THE FFN DOMINATES COMPUTE, and
   attention's quadratic term only takes over when
   n is comparable to d:

        4*n*d  >  24*d^2   when   n > 6d

   For d = 4096 that is n > 24,576 tokens. BELOW THAT, THE
   MODEL IS MOSTLY DOING FFN MATMULS, not attention. This
   surprises people who assume attention is always the
   bottleneck - it becomes one at long context, and at short
   context the FFN is the cost.

/* --- WHAT THIS MEANS FOR OPTIMISATION --- */
     SHORT CONTEXT   optimise the FFN: quantise it
                     (chapter 26), use MoE, prune it
     LONG CONTEXT    optimise attention: Flash Attention,
                     sliding windows, GQA (chapter 19)
     GENERATION      neither - you are memory-bandwidth
                     bound (chapters 04, 25)

/* --- AND FOR EMBEDDED (chapter 27) --- */
     · LayerNorm needs a mean, a variance and a reciprocal
       square root per token. On a device without an FPU
       that is expensive; RMSNorm halves it, and
       fixed-point implementations need care with the
       variance's dynamic range.
     · SwiGLU needs a sigmoid per hidden unit - 11k
       transcendental evaluations per token at d = 4096.
       ReLU is one comparison. On an MCU, choose ReLU.
     · the FFN's two large matmuls are where a small
       transformer spends its time, so an NPU that only
       accelerates dense matmul still gets you most of the
       win.                                              */
```

How it shows up

Symptom	Cause
Deep model diverges, shallow one trains	Post-norm — switch to pre-norm
Training needs very long warmup	Post-norm, or missing residual scaling at init
Activations grow steadily with depth	Normal for pre-norm — ensure a final LayerNorm exists
Removing the residual breaks training entirely	Expected — the gradient loses its identity path
`nan` in LayerNorm	Zero variance, or `eps` too small in fp16
MoE model uses only two experts	No load-balancing loss — router collapse

Symptom	Cause
MoE saves compute but not memory	By design — all experts must be resident
Parameter count doesn't match after switching to SwiGLU	`d_ff` must drop to 8/3·d for three matrices
Attention optimisation gives little speedup	Short context — the FFN dominates
LayerNorm is slow on an MCU	Two reductions plus rsqrt — use RMSNorm

Check yourself

- ✓ Give the gradient argument for residual connections.
- ✓ Explain the residual stream as a bus, and why its norm grows with depth.
- ✓ Write LayerNorm and RMSNorm, and say what RMSNorm drops.
- ✓ Explain pre-norm versus post-norm and which one enables depth.
- ✓ State the honest position on why normalisation helps.
- ✓ Write the FFN and state the division of labour with attention.
- ✓ Explain SwiGLU and why `d_ff` changes to 8/3·d.
- ✓ Give the key-value memory view of the FFN and the evidence for it.
- ✓ Explain mixture of experts and why it saves compute but not memory.
- ✓ Compute the crossover sequence length where attention overtakes the FFN.
- ✓ Say which component to optimise for short context, long context and generation.

Pretraining and scaling laws

Predict the next token on a very large amount of text. The objective is trivial; the engineering is not, and the empirical laws governing how performance improves with scale are the most practically useful result in the field.

The objective

```
TRAINING DATA:  raw text. No labels. No annotation.

  input   [The, cat, sat, on, the]
  target  [cat, sat, on, the, mat]

SHIFT BY ONE. Cross-entropy at every position
(chapter 10). With causal masking, one forward pass over
a sequence of length n yields n training examples
(chapter 19).

THAT IS THE ENTIRE PRETRAINING OBJECTIVE OF EVERY LARGE
LANGUAGE MODEL.

WHY IT WORKS AT ALL - and this is the genuinely
surprising part:

  "The capital of France is ___"      needs a FACT
  "2 + 2 = ___"                       needs ARITHMETIC
  "def fib(n): return ___"            needs CODE
  "Q: ...  A: ___"                    needs REASONING
  "Translate: le chat -> ___"         needs TRANSLATION
  "... therefore the answer is ___"   needs INFERENCE

PREDICTING THE NEXT TOKEN WELL ENOUGH, ACROSS ENOUGH
TEXT, REQUIRES MODELLING WHATEVER PRODUCED THE TEXT.
Nobody designed for translation or arithmetic; those
capabilities are means to an end for a compression
objective (chapter 10).

THE HONEST FRAMING: this is an EMPIRICAL result, not a
predicted one. In 2018 essentially nobody expected that
scaling this objective would produce in-context learning
or chain-of-thought reasoning. The theory is still
catching up.

AND THE ALTERNATIVE OBJECTIVES, for context:
  MASKED LM (BERT)   hide 15% of tokens, predict them.
                     Bidirectional context, better
                     representations - but you only train
                     on 15% of positions, and it does not
                     generate.
  SPAN CORRUPTION    (T5) mask contiguous spans.
  NEXT-TOKEN won because it trains on EVERY position, it
  generates natively, and it scales.
```

The data

```
/* A MODERN PRETRAINING CORPUS: 2-20 TRILLION TOKENS.
   Roughly the sources, in order of contribution:          */

    WEB CRAWL          Common Crawl, FineWeb, RefinedWeb.
                       The bulk, and the most in need of
                       filtering.
    CODE               GitHub, The Stack. Improves reasoning
                       measurably, even on non-code tasks.
    BOOKS              long-form coherence, narrative
                       structure
    WIKIPEDIA          dense, high-quality facts. Small but
                       heavily upweighted.
    ACADEMIC           arXiv, PubMed
    CURATED / SYNTHETIC increasingly significant: textbook-
                       style generated data, filtered by
                       quality models

/* --- THE PIPELINE, and it matters more than the
       architecture --- */

  1. EXTRACT text from HTML. Boilerplate removal is
     surprisingly hard and surprisingly important.

  2. LANGUAGE IDENTIFICATION and filtering.

  3. QUALITY FILTERING
       · heuristics: mean word length, punctuation ratio,
         proportion of stopwords, lines ending in "..."
       · a CLASSIFIER trained to recognise "reference-like"
         text
       · perplexity filtering against a small reference
         model

  4. DEDUPLICATION - THE HIGHEST-VALUE STEP.
       exact-match, then FUZZY (MinHash / LSH) at document
       and paragraph level.
       Duplicated data causes memorisation without
       generalisation, and it wastes compute. Removing
       duplicates reliably improves models at fixed
       compute - one of the few interventions that is
       almost always positive.

  5. DECONTAMINATION - remove anything matching benchmark
     test sets (chapter 12). Frequently done badly, which
     is why published benchmark numbers should be read
     sceptically.

  6. PII AND TOXICITY FILTERING.

  7. MIXING AND UPWEIGHTING. Wikipedia might be repeated
     several times; low-quality web crawl seen once. The
     mixture weights are a real hyperparameter and are
     usually not published.

/* --- HOW MUCH DATA IS THERE, HONESTLY --- */
    High-quality English text on the public internet is
    estimated at a few tens of trillions of tokens. Frontier
    models already train on a meaningful fraction of it.

    THE RESPONSES: multilingual data, code, multiple epochs
```

```
    (up to ~4 epochs costs little quality), synthetic data,
    and non-public licensed corpora. "Running out of data"
    is a real constraint on the current recipe, and it is
    part of why attention has shifted toward post-training
    (chapter 24) and inference-time compute.               */
```

Scaling laws

```
THE CENTRAL EMPIRICAL RESULT (Kaplan 2020, Hoffmann
2022):

  LOSS FALLS AS A POWER LAW IN MODEL SIZE, DATA AND
  COMPUTE, SMOOTHLY, OVER MANY ORDERS OF MAGNITUDE.

     L(N) = (Nc / N)^alpha        N = parameters
     L(D) = (Dc / D)^beta         D = tokens
     L(C) = (Cc / C)^gamma        C = compute

On a log-log plot these are STRAIGHT LINES:

  log
  loss
    |  \
    |    \
    |      \
    |        \
    |          \
    +------------------ log compute

THE PRACTICAL CONSEQUENCE, and it is why these laws
matter: YOU CAN TRAIN SMALL MODELS AND PREDICT THE LOSS
OF A LARGE ONE BEFORE SPENDING THE MONEY. Frontier runs
are planned this way - a series of small runs fits the
curve, and the big run's loss is forecast in advance and
usually lands close.

THE CHINCHILLA RESULT (2022), which corrected the field:

  For a COMPUTE BUDGET C, the optimal split is
  approximately

     20 TOKENS PER PARAMETER

  GPT-3: 175B parameters, 300B tokens
         -> 1.7 tokens/parameter. SEVERELY
            UNDERTRAINED.
  Chinchilla: 70B parameters, 1.4T tokens
         -> a 70B model BEAT the 175B one, at the same
            compute.

THE LESSON WAS THAT EVERYONE HAD BEEN BUILDING MODELS
TOO BIG AND TRAINING THEM TOO LITTLE.

AND THE MODERN CORRECTION TO THE CORRECTION:

  Chinchilla optimises TRAINING compute. But a deployed
  model is trained ONCE and run BILLIONS of times, so
  INFERENCE cost dominates total cost.

  So current practice deliberately OVERTRAINS SMALL
  MODELS far past Chinchilla-optimal:
    Llama 3 8B: ~15T tokens -> ~1,875 tokens/parameter,
    nearly 100x Chinchilla.

  A smaller model trained much longer costs more to
  train and far less to serve. FOR ANYTHING THAT WILL
  RUN ON A DEVICE (chapter 27), THIS IS THE RIGHT TRADE
  AND IT IS WHY GOOD SMALL MODELS EXIST NOW.
```

```
THE COMPUTE FORMULA (chapter 04):

     C  ~=  6 * N * D     FLOPs

  Llama-3-8B: 6 * 8e9 * 15e12 = 7.2e23 FLOPs.
  On 16,000 H100s at ~400 TFLOP/s effective:
     7.2e23 / (16000 * 4e14) = ~112,000 seconds of pure
     compute, and in practice weeks of wall-clock once
     efficiency, restarts and data loading are counted.
```

The engineering

```
/* NO SINGLE ACCELERATOR HOLDS A LARGE MODEL. Training is a
   distributed systems problem.                              */

/* --- THE PARALLELISM AXES --- */
    DATA PARALLEL       every device holds a FULL COPY of
                        the model, processes a different
                        batch slice, and gradients are
                        ALL-REDUCED. Simple; limited by
                        model memory.

    ZeRO / FSDP         SHARD the optimiser state, then the
                        gradients, then the parameters
                        across devices, gathering each layer
                        only when needed. Removes the
                        Adam-state memory wall (chapter 11)
                        at the cost of extra communication.

    TENSOR PARALLEL     SPLIT INDIVIDUAL MATRICES across
                        devices - each holds part of a
                        weight matrix. Needs very fast
                        interconnect (NVLink), so it stays
                        WITHIN a node.

    PIPELINE PARALLEL   different LAYERS on different
                        devices. Creates a "bubble" of idle
                        time, mitigated by splitting the
                        batch into micro-batches.

    EXPERT PARALLEL     MoE experts on different devices
                        (chapter 22).

    IN PRACTICE, ALL AT ONCE: tensor parallel inside a node,
    pipeline parallel across nodes, data parallel across
    replicas. "3-D parallelism".

/* --- MIXED PRECISION --- */
    bf16 for the forward and backward passes; fp32 for the
    master weights and the optimiser state.

    BF16, NOT FP16. Same exponent range as fp32, fewer
    mantissa bits. It cannot overflow where fp16 does, so
    NO LOSS SCALING IS NEEDED. This alone removed a whole
    class of training instability, and it is why every
    modern accelerator supports it.

/* --- GRADIENT CHECKPOINTING (chapter 07) --- */
    Recompute activations in the backward pass instead of
    storing them. ~30% more compute, a large memory saving.
    Standard.

/* --- WHAT THE OPERATIONS ACTUALLY LOOK LIKE --- */
    · CHECKPOINT FREQUENTLY. Hardware fails; at 16,000 GPUs
      something fails every few hours. Meta reported a
      failure roughly every 3 hours during Llama 3 training.
    · MONITOR THE LOSS CURVE CONTINUOUSLY. LOSS SPIKES ARE
      NORMAL. The standard response is to roll back to a
      checkpoint and skip the offending data batch
      (chapter 11).
    · MFU (model FLOPs utilisation) is the efficiency
      metric: achieved FLOPs / peak FLOPs. 40-55% is good.
```

```
      Below 30% means something is wrong - usually
      communication or data loading.
    · DATA LOADING MUST KEEP UP. Feeding thousands of
      accelerators is itself a serious I/O problem.

/* --- THE SEQUENCE OF A REAL RUN --- */
    1. tokenise and shard the corpus into fixed blocks
       (chapter 16) - no padding, documents concatenated
       with separators
    2. warmup over ~2000 steps (chapter 11)
    3. cosine decay to ~10% of peak
    4. AdamW, beta2 = 0.95, weight decay 0.1, grad clip 1.0
    5. batch size ramped up over training - small batches
       early are more sample-efficient
    6. evaluate on held-out perplexity and a benchmark suite
       every few thousand steps
    7. a final "cooldown" or annealing phase on
       high-quality data, which reliably improves the final
       model                                              */
```

EMERGENCE, AND THE ARGUMENT ABOUT IT

Some capabilities appear abruptly with scale — multi-digit arithmetic, chain-of-thought benefit, in-context learning — showing near-zero performance below a threshold and useful performance above it. **But a well-supported critique** argues much of this is an artefact of *discontinuous metrics*: exact-match accuracy on a 5-digit sum is all-or-nothing, so a smooth improvement in per-token probability looks like a sudden jump. Measured with a continuous metric, many "emergent" curves become smooth. Some genuine phase changes remain — induction-head formation (chapter 19) is a real, abrupt structural transition. Treat "emergence" claims as needing a look at the metric first.

What it costs, and what it produces

```
/* --- ROUGH TRAINING COSTS --- */
    GPT-2 (1.5B, 2019)         ~$50k
    GPT-3 (175B, 2020)         ~$5M
    Llama 3 70B (2024)         ~$30-60M
    Frontier models (2025-26)  hundreds of millions

    NOT INCLUDING: failed runs, data pipeline work,
    salaries, or the evaluation infrastructure - which
    collectively often exceed the compute bill.

/* --- WHAT YOU GET: A BASE MODEL --- */
    A base model COMPLETES TEXT. It does not follow
    instructions, answer questions, or refuse anything.

        prompt: "What is the capital of France?"
        base model: "What is the capital of Germany?
                     What is the capital of Italy?
                     ..."

    It continued the pattern - a list of questions - which
    is exactly what the objective asked for. IT IS NOT
    BROKEN; it is doing precisely what it was trained to do.

    TURNING IT INTO AN ASSISTANT IS A SEPARATE STAGE
    (chapter 24), and that stage costs a tiny fraction of
    pretraining while changing the user-visible behaviour
    almost entirely.

/* --- AND WHAT THIS MEANS IF YOU ARE NOT OPENAI --- */

    YOU WILL NOT PRETRAIN A FRONTIER MODEL. What you will
    do:

      · FINE-TUNE an open base model (chapter 24) -
        hundreds to thousands of dollars
      · PRETRAIN A SMALL DOMAIN MODEL - a 10-100M parameter
        model on your own corpus (sensor logs, protocol
        traces, domain text) is genuinely feasible on one
        GPU in days, and for a narrow embedded task it may
        beat anything general-purpose
      · DISTIL a large model into a small one (chapter 26)
      · or simply USE an existing model well

    THE SCALING LAWS ARE STILL USEFUL AT YOUR SCALE: they
    tell you how to split a fixed budget between model size
    and data, and 20 tokens per parameter remains a sound
    starting point for a from-scratch run. Chapter 28's
    project is exactly this, at the smallest possible size.
                                                          */
```

How it shows up

Symptom	Cause
Base model does not answer questions	Working as designed — it completes text; it needs post-training
Model memorises passages verbatim	Duplicated data — deduplicate
Benchmark scores far above real quality	Test-set contamination — decontaminate

Symptom	Cause
Large model underperforms a smaller one	Undertrained — check tokens per parameter
MFU below 30%	Communication overhead or a starved data loader
Loss spikes mid-run	Normal at scale — roll back and skip the batch
Training unstable in fp16	Use bf16 — no loss scaling needed
Out of memory with a model that should fit	Optimiser state — use ZeRO/FSDP and checkpointing
Serving cost dominates the budget	Chinchilla-optimal sizing — overtrain a smaller model instead
An "emergent" capability vanishes under a different metric	The metric was discontinuous, not the capability

Check yourself

- ✓ State the pretraining objective and why one pass gives n examples.
- ✓ Explain why next-token prediction produces translation and arithmetic.
- ✓ Contrast next-token, masked-LM and span-corruption objectives.
- ✓ List the seven data pipeline stages and say which has the highest value.
- ✓ State the scaling laws and their single most useful practical consequence.
- ✓ State the Chinchilla ratio and why GPT-3 was undertrained.
- ✓ Explain why current practice deliberately overtrains small models.
- ✓ Use `C ≈ 6ND` to estimate a training run.
- ✓ Name five parallelism axes and say which stays within a node.
- ✓ Explain why bf16 is preferred over fp16.
- ✓ Give the critique of emergence and the one clear counterexample.
- ✓ Describe what a base model does with a question, and why.

Fine-tuning, instruction tuning and RLHF

Turning a text completer into something useful. Post-training costs a tiny fraction of pretraining and accounts for almost everything a user perceives about a model's character and usefulness.

The stages

```
1. PRETRAINING          (chapter 23)
     trillions of tokens, months, millions of dollars
     -> a BASE MODEL that completes text

2. SUPERVISED FINE-TUNING (SFT) / INSTRUCTION TUNING
     10k-1M curated (instruction, response) pairs
     hours to days
     -> a model that FOLLOWS INSTRUCTIONS

3. PREFERENCE OPTIMISATION (RLHF, DPO, and relatives)
     10k-1M human preference comparisons
     -> a model that is HELPFUL, HONEST AND HARMLESS
        according to whoever wrote the guidelines

4. (optional) CONTINUED PRETRAINING on domain data,
     inserted between 1 and 2 when you need domain
     knowledge rather than domain style

THE PROPORTIONS ARE STRIKING:
     pretraining      >99% of the compute
     post-training     <1% of the compute
     but post-training is responsible for essentially
     ALL of the difference between "a strange text
     completer" and "a useful assistant".

THE STANDARD INTERPRETATION - the SUPERFICIAL ALIGNMENT
HYPOTHESIS: post-training does not teach new knowledge.
It teaches the model WHICH OF ITS EXISTING CAPABILITIES
TO SURFACE, and in what format. Evidence: a few thousand
high-quality examples get most of the benefit, and
fine-tuning rarely adds facts the base model did not
already contain.

THE PRACTICAL COROLLARY, and it is the most useful thing
in this chapter:

  IF THE BASE MODEL DOES NOT KNOW SOMETHING,
  FINE-TUNING WILL NOT TEACH IT RELIABLY.
  IT WILL TEACH IT TO SOUND CONFIDENT ABOUT IT.

For new facts, use RETRIEVAL (chapter 17) or continued
pretraining on a lot of domain text - not a few thousand
Q&A pairs.
```

Supervised fine-tuning

```
/* MECHANICALLY IDENTICAL TO PRETRAINING - same
   cross-entropy, same optimiser. Only the data changes. */

    TRAINING EXAMPLE:
      <|im_start|>user
      Explain what a mutex is.<|im_end|>
      <|im_start|>assistant
      A mutex is a synchronisation primitive that...<|im_end|>

    AND THE ONE IMPORTANT DETAIL:

      MASK THE LOSS ON THE PROMPT TOKENS.

    You want the model to learn to PRODUCE the response, not
    to predict the user's question. Set the prompt positions
    to ignore_index (chapter 16).

        labels = input_ids.clone()
        labels[:prompt_len] = -100     # ignored by
                                       # cross_entropy

    Omitting this still "works" and produces a measurably
    worse model that sometimes continues by inventing the
    next user turn.

/* --- WHAT MAKES GOOD SFT DATA --- */
    QUALITY BEATS QUANTITY, decisively. The LIMA result:
    1,000 carefully curated examples produced a model
    competitive with one trained on 50,000 mediocre ones.

    WHAT TO CURATE FOR:
      · DIVERSITY of task types, lengths and formats
      · CONSISTENT STYLE - the model learns the average, so
        inconsistent formatting produces inconsistent output
      · CORRECTNESS. A wrong answer in the data teaches the
        model to be wrong confidently.
      · REFUSALS where appropriate, or the model will
        never decline anything
      · and MULTI-TURN conversations, or it will handle only
        single turns well

    SOURCES: human-written (expensive, best), model-generated
    and filtered (Self-Instruct, Evol-Instruct), converted
    from existing datasets, or DISTILLED from a stronger
    model (check the licence).

/* --- LoRA: the technique that made this accessible --- */
    Freeze W. Learn a LOW-RANK UPDATE (chapters 03, 05):

        W_new = W + (alpha/r) * B @ A

        A: (r, d_in)   initialised randomly
        B: (d_out, r)  INITIALISED TO ZERO, so the model
                       starts unchanged

    For d = 4096 and r = 16:
        full fine-tune  16.8M parameters per matrix
        LoRA             131k parameters
        -> 128x fewer trainable parameters
```

```
    WHY IT WORKS: the weight UPDATE during fine-tuning has
    low intrinsic rank. Adaptation lives in a small
    subspace, so representing it compactly loses little.

    THE PRACTICAL BENEFITS:
      · fine-tune a 7B model on one consumer GPU
      · the adapter is a few megabytes - ship many task
        adapters for one base model, swap at runtime
      · NO INFERENCE COST if you merge B@A into W afterwards
      · and QLoRA: quantise the frozen base to 4 bits
        (chapter 26) and train the adapter in bf16. A 70B
        model fine-tunes on a single 48 GB card.

    HYPERPARAMETERS THAT MATTER: r (8-64), alpha (usually
    2r), and WHICH MODULES to adapt - attention projections
    only is the classic recipe, but including the FFN
    generally works better.

    from peft import LoraConfig, get_peft_model
    cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                     target_modules=["q_proj","k_proj",
                                     "v_proj","o_proj",
                                     "gate_proj","up_proj",
                                     "down_proj"])
    model = get_peft_model(base_model, cfg)
    model.print_trainable_parameters()
    # trainable: 0.24% of all parameters

/* --- CATASTROPHIC FORGETTING --- */
    Fine-tuning hard on a narrow task degrades everything
    else. A model tuned only on SQL forgets how to write
    prose.

    MITIGATIONS: a LOW learning rate (1e-5 to 5e-5, an order
    of magnitude below pretraining), FEW epochs (1-3), MIX
    IN general data, or use LoRA - which by constraining the
    update to a low-rank subspace forgets substantially
    less.                                                  */
```

Learning from preferences

```
/* THE PROBLEM SFT CANNOT SOLVE: for most requests there is
   no single correct answer, only better and worse ones. And
   it is far easier for a human to say "B is better than A"
   than to write the ideal A.

/* --- CLASSIC RLHF, THREE STEPS --- */

  1. COLLECT PREFERENCES
       Sample two responses to the same prompt. A human
       picks the better one. Repeat 10k-1M times, against
       written guidelines.

  2. TRAIN A REWARD MODEL
       A copy of the model with a scalar head, trained so
       that preferred responses score higher:

         L = -log sigmoid( r(chosen) - r(rejected) )

       (The Bradley-Terry model - the same logistic form as
       chapter 13.)

  3. OPTIMISE THE POLICY AGAINST THE REWARD, with PPO,
     subject to a KL PENALTY:

         objective = E[ r(x, y) ]
                     - beta * KL(policy || reference)

       THE KL TERM IS ESSENTIAL. Without it the policy
       finds adversarial inputs that the reward model scores
       highly and humans find worthless - REWARD HACKING.
       Classic failure modes: extreme verbosity, excessive
       hedging, opening every response with flattery,
       compulsive list-making. IF YOU HAVE NOTICED MODELS
       DOING THESE THINGS, THIS IS WHERE THEY COME FROM.

       It is the REVERSE KL of chapter 10 - mode-seeking,
       keeping the policy near the reference.

/* --- DPO: what most people use now --- */
    Direct Preference Optimisation removes the reward model
    and the RL loop entirely. The insight: the optimal
    policy under a KL-constrained reward objective has a
    closed form, so the reward can be expressed in terms of
    the policy itself. Substituting gives a plain
    supervised loss over preference pairs:

      L = -log sigmoid( beta * [
              log pi(y_w|x) - log pi_ref(y_w|x)
            - log pi(y_l|x) + log pi_ref(y_l|x) ] )

    NO REWARD MODEL. NO SAMPLING LOOP. NO PPO. Just a loss
    function over (prompt, chosen, rejected) triples, run
    like any other fine-tune.

    · far simpler and more stable
    · usually comparable results
    · the main loss is that you cannot do online exploration
      against a learned reward

    VARIANTS: IPO, KTO (needs only binary good/bad labels,
```

```
    not pairs), ORPO (merges SFT and preference learning
    into one stage), SimPO (reference-free).

/* --- RLAIF AND CONSTITUTIONAL AI --- */
    Replace the human labeller with a model, guided by an
    explicit written set of principles. The model critiques
    and revises its own outputs against the constitution,
    and those revisions become training data.

    · dramatically cheaper and faster to iterate
    · the principles are WRITTEN DOWN and inspectable, which
      is a genuine transparency advantage over implicit
      preferences in a labelling pool
    · and the obvious concern: the judging model's biases
      become the training signal

/* --- RLVR: verifiable rewards --- */
    For maths, code and other checkable domains, skip
    preferences entirely: RUN THE TESTS. Reward = did the
    unit test pass, did the proof check, is the answer
    correct.

    An objective, unhackable reward signal, and the basis of
    the recent reasoning-model results. It only works where
    correctness is mechanically checkable - which is a real
    limit, and also exactly the situation in a great deal of
    engineering work.                                    */
```

CHOOSING YOUR APPROACH

```
NEED A FORMAT OR STYLE          -> SFT, or just prompting
NEED DOMAIN KNOWLEDGE           -> RETRIEVAL first;
                                   continued pretraining
                                   if you have a lot of
                                   text
NEED BETTER JUDGEMENT/TONE      -> DPO on preference pairs
NEED CHECKABLE CORRECTNESS      -> RLVR against tests
LIMITED COMPUTE                 -> LoRA or QLoRA
JUST TRYING SOMETHING           -> PROMPT FIRST
```

Try prompting before fine-tuning. A good system prompt plus a few in-context examples solves a surprising fraction of what people reach for fine-tuning to do, costs nothing, and can be changed in a minute rather than a day.

Doing it

```
from transformers import (AutoModelForCausalLM,
                          AutoTokenizer, TrainingArguments)
from peft import LoraConfig
from trl import SFTTrainer, DPOTrainer

# --- SFT ---
model = AutoModelForCausalLM.from_pretrained(
            "meta-llama/Llama-3.1-8B", torch_dtype="bfloat16",
            device_map="auto")
tok = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B")

args = TrainingArguments(
    output_dir="out",
    num_train_epochs=2,            # 1-3. MORE OVERFITS.
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8, # effective batch 32
    learning_rate=2e-5,            # LOW (chapter 11)
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    bf16=True,
    gradient_checkpointing=True,
    logging_steps=10,
    eval_strategy="steps", eval_steps=100,
)

trainer = SFTTrainer(
    model=model, args=args,
    train_dataset=train_ds, eval_dataset=eval_ds,
    peft_config=LoraConfig(r=16, lora_alpha=32),
    max_seq_length=2048,
)
trainer.train()

# --- DPO, on a dataset of {prompt, chosen, rejected} ---
dpo = DPOTrainer(
    model=sft_model,
    ref_model=None,        # LoRA: the base with adapters
                           # disabled IS the reference
    args=dpo_args,
    beta=0.1,              # the KL strength
    train_dataset=pref_ds,
)
dpo.train()

# --- MERGE AND EXPORT ---
merged = model.merge_and_unload()   # folds B@A into W
merged.save_pretrained("final")     # zero inference
                                    # overhead

/* --- EVALUATION: THE PART PEOPLE SKIP --- */
    · HOLD OUT a test set BEFORE you start (chapter 12)
    · CHECK FOR REGRESSION on general benchmarks, not only
      your task - catastrophic forgetting is invisible if
      you only measure what you tuned for
    · LLM-AS-JUDGE for open-ended quality, with the known
      biases accounted for: judges prefer LONGER answers,
      prefer their own family's style, and are sensitive to
      option ORDER (so randomise it)
    · AND READ THE OUTPUTS. A hundred generations, by hand.
      Every automated metric misses things that are obvious
```

```
      on inspection (chapter 12).

/* --- WHAT GOES WRONG --- */
    OVERFITTING       loss near zero, outputs degenerate and
                      repetitive. Fewer epochs.
    FORGETTING        general ability drops. Lower lr, mix
                      in general data, use LoRA.
    FORMAT COLLAPSE   every answer takes the same shape.
                      Diversify the data.
    SYCOPHANCY        preference training rewards agreement,
                      so the model agrees with incorrect
                      user claims. A known and actively
                      studied side effect of RLHF.
    VERBOSITY         reward models like long answers. A
                      length penalty helps.
    OVER-REFUSAL      too many refusal examples, or too
                      broad. The model declines benign
                      requests. Balance deliberately.     */
```

How it shows up

Symptom	Cause
Model invents the next user turn	Loss not masked on prompt tokens
Confidently wrong on domain facts	Fine-tuning taught style, not knowledge — use retrieval
General ability drops after tuning	Catastrophic forgetting — lower LR, fewer epochs, LoRA
Outputs repetitive and degenerate	Too many epochs on too little data
Answers grow steadily longer	Reward hacking on length
Model agrees with incorrect user claims	Sycophancy from preference training
Benign requests refused	Refusal data too broad
Policy diverges into nonsense during RL	KL penalty too weak or absent
LLM-judge scores disagree with human reading	Length and position bias — randomise and read outputs
LoRA has no effect	Wrong `target_modules`, or `alpha` too small
Fine-tuned model is slower	Adapters not merged

Check yourself

- ✓ Name the post-training stages and their relative compute cost.
- ✓ State the superficial alignment hypothesis and its practical corollary.
- ✓ Explain prompt-token loss masking and what happens without it.
- ✓ Give four properties of good SFT data.
- ✓ Write the LoRA update, say why B starts at zero, and compute the parameter saving.
- ✓ Explain QLoRA and what it makes possible.
- ✓ Describe the three RLHF steps and the reward-model loss.
- ✓ Explain the KL penalty and name four reward-hacking behaviours it prevents.
- ✓ Explain what DPO removes and why it is simpler.
- ✓ Describe RLAIF and RLVR, and the limit of each.
- ✓ Choose an approach for six different needs.

✓ List three known biases of LLM-as-judge evaluation.

Inference: sampling, the KV cache and context

Training happens once; inference happens billions of times. This is where the systems engineering lives, and where an embedded engineer's instincts about memory bandwidth turn out to be exactly the right ones.

The generation loop

```
A MODEL PRODUCES ONE DISTRIBUTION OVER THE VOCABULARY,
FOR THE NEXT TOKEN. Generating text means running it
repeatedly, feeding each choice back in.

  tokens = tokenise(prompt)
  loop:
     logits = model(tokens)[-1]      # last position only
     probs  = softmax(logits / T)
     next   = sample(probs)
     tokens.append(next)
     if next == EOS or len(tokens) >= max: break

THIS IS AUTOREGRESSIVE DECODING, and it has two distinct
phases with completely different performance characters:

PREFILL - process the whole prompt at once
  · one forward pass over n tokens, fully parallel
  · MATRIX-MATRIX products -> COMPUTE BOUND (chapter 04)
  · fast per token: thousands of tokens/second
  · determines TIME TO FIRST TOKEN

DECODE - generate one token at a time
  · one forward pass per token, inherently sequential
  · MATRIX-VECTOR products -> MEMORY BANDWIDTH BOUND
  · EVERY WEIGHT IN THE MODEL IS READ FROM MEMORY TO
    PRODUCE ONE TOKEN
  · determines TOKENS PER SECOND

THE ARITHMETIC THAT EXPLAINS EVERYTHING:

  70B model in fp16 = 140 GB of weights.
  At 2 TB/s of HBM bandwidth:
     140 / 2000 = 70 ms per token = ~14 tokens/second
  AND THE ARITHMETIC UNITS ARE ALMOST ENTIRELY IDLE.

SO EVERY INFERENCE OPTIMISATION IS AN ATTACK ON BYTES
MOVED, NOT ON FLOPS:
  · QUANTISATION - fewer bytes per weight (chapter 26)
  · BATCHING - amortise the weight read over many
    requests
  · SPECULATIVE DECODING - verify several tokens in one
    memory-bound pass
  · SMALLER MODELS
None of them make the arithmetic faster, because the
arithmetic was never the problem.
```

The KV cache

```
/* THE NAIVE LOOP RECOMPUTES EVERYTHING EVERY STEP.

   Generating token 100 re-runs attention over tokens 1..99,
   recomputing keys and values that have not changed. TOTAL
   COST IS O(n^2) FORWARD PASSES for n tokens - quadratic on
   top of attention's own quadratic.

   THE FIX: CACHE THE KEYS AND VALUES.

   Because of the causal mask (chapter 19), the K and V for
   position i NEVER CHANGE once computed. Store them. Each
   new token computes only its own Q, K, V and attends to
   the cache.

     WITHOUT CACHE   O(n) work per token, O(n^2) total
     WITH CACHE      O(1) work per token, O(n) total

   THIS IS NOT AN OPTIMISATION - IT IS THE DIFFERENCE
   BETWEEN USABLE AND UNUSABLE.                          */

    class KVCache:
        def __init__(self):
            self.k = None   # (B, n_kv_head, T, d_head)
            self.v = None

        def update(self, k_new, v_new):
            if self.k is None:
                self.k, self.v = k_new, v_new
            else:
                self.k = torch.cat([self.k, k_new], dim=2)
                self.v = torch.cat([self.v, v_new], dim=2)
            return self.k, self.v

    # in attention, during decode:
    #   q is (B, H, 1, d)  - ONE new position
    #   k, v are the FULL cache
    #   -> one row of the attention matrix, not the whole
    #      thing. And is_causal is unnecessary: there is
    #      nothing after the current position anyway.

/* --- THE COST, AND IT IS THE BINDING CONSTRAINT --- */

    per token = 2 (K and V)
              * n_layers
              * n_kv_heads
              * d_head
              * bytes_per_element

    Llama-3-8B, fp16:
        2 * 32 * 8 * 128 * 2 = 131 KB PER TOKEN
        8k context   -> 1.0 GB
        128k context -> 16 GB    <-- LARGER THAN THE MODEL

    WITHOUT GQA (32 KV heads instead of 8) that would be
    4x more. THIS IS WHY GQA EXISTS (chapter 19), and it is
    a pure inference-memory optimisation.

/* --- MANAGING IT: PAGED ATTENTION --- */
    Naively, you preallocate max_length per request and
    waste most of it. vLLM's PagedAttention borrows the
```

```
    idea from virtual memory (see the OS track): allocate
    the cache in fixed-size BLOCKS, keep a per-request
    BLOCK TABLE, and let blocks be non-contiguous.

      · near-zero internal fragmentation
      · COPY-ON-WRITE SHARING of a common prompt prefix
        across parallel samples
      · 2-4x more concurrent requests on the same hardware

    A PAGE TABLE, FOR ATTENTION. If you have read the OS
    track this will feel extremely familiar, and that is not
    a coincidence - it is the same problem.

/* --- CONTINUOUS BATCHING --- */
    Static batching waits for the longest sequence in the
    batch to finish, idling everything else. Continuous
    (in-flight) batching removes finished sequences and
    admits new ones EVERY STEP.
    Typically 2-3x higher throughput. Standard in vLLM,
    TGI, TensorRT-LLM.

/* --- PREFIX CACHING --- */
    A long system prompt shared across requests need only be
    prefilled ONCE. Cache the KV for the shared prefix and
    reuse it. Large latency win for agent and chat
    workloads, and the reason API providers charge less for
    cached input tokens.                                  */
```

Sampling

```
/* THE MODEL GIVES A DISTRIBUTION. HOW YOU DRAW FROM IT
   changes the output character completely.               */

/* --- GREEDY: always the argmax --- */
    next = logits.argmax()
    Deterministic and reproducible; frequently REPETITIVE,
    and it falls into loops ("and then, and then, and
    then"). Correct for tasks with one right answer -
    classification, extraction, structured output.

/* --- TEMPERATURE (chapter 08) --- */
    probs = softmax(logits / T)
      T -> 0    approaches greedy
      T = 0.7   a common default: coherent, some variety
      T = 1.0   the model's own calibrated distribution
      T > 1.2   creative, and increasingly incoherent

/* --- TOP-K --- */
    Keep the k highest-probability tokens, renormalise.
    k = 40 is typical.
    THE FLAW: a fixed k is wrong in both directions. When
    the model is CERTAIN, 40 candidates includes 39 bad
    ones. When it is genuinely uncertain, 40 may be too few.

/* --- TOP-P / NUCLEUS SAMPLING. THE DEFAULT. --- */
    Keep the smallest set of tokens whose cumulative
    probability exceeds p. p = 0.9 or 0.95.

    ADAPTIVE: a confident distribution yields 1-2
    candidates; a flat one yields hundreds. This is why it
    outperforms top-k, and it is what almost every API uses.

    def top_p(logits, p=0.9, T=1.0):
        logits = logits / T
        s, idx = torch.sort(logits, descending=True)
        probs = torch.softmax(s, dim=-1)
        cum = torch.cumsum(probs, dim=-1)
        # keep everything up to and including the crossing
        mask = cum - probs > p
        s[mask] = float('-inf')
        return idx[torch.multinomial(
                     torch.softmax(s, -1), 1)]

/* --- MIN-P: newer, and often better --- */
    Keep tokens with probability at least
    min_p * p_max. Scales the threshold with the model's own
    confidence, and tends to be more robust at high
    temperature.

/* --- REPETITION AND PRESENCE PENALTIES --- */
    Subtract from the logits of tokens already generated.
    Reduces loops. USE SPARINGLY: a strong penalty degrades
    text that legitimately repeats words - code, lists,
    technical writing where the same identifier recurs.

/* --- BEAM SEARCH --- */
    Keep the b highest-probability SEQUENCES rather than
    tokens.
    · better for translation and summarisation, where a
      single best output exists
```

```
    · WORSE for open-ended generation — it produces bland,
      high-probability, generic text, because the most
      probable sequence is rarely the most interesting one
    · b times the compute and b times the KV cache

/* --- STRUCTURED / CONSTRAINED DECODING --- */
    Force the output to match a grammar or JSON schema by
    MASKING INVALID TOKENS at each step — set their logits
    to -inf.

    THE OUTPUT IS THEN VALID BY CONSTRUCTION, not by hope.
    For any system that parses model output — which for an
    engineer is most of them — this is far more reliable
    than asking politely and retrying. Outlines, XGrammar
    and llama.cpp's GBNF all do this.

/* --- WHAT TO USE --- */
    factual Q&A, extraction   T=0 (greedy)
    code                      T=0.2, or greedy
    general chat              T=0.7, top_p=0.95
    creative writing          T=0.9–1.1, top_p=0.95
    anything parsed           CONSTRAINED DECODING
    reproducible tests        greedy + a fixed seed        */
```

SPECULATIVE DECODING

```
1. A SMALL DRAFT MODEL generates k tokens cheaply.
2. The BIG MODEL scores all k IN ONE FORWARD PASS.
3. Accept the longest prefix matching what the big
   model would have sampled; reject the rest.

Because decode is memory-bound, verifying k tokens costs
almost the same as generating one. Typical speedup
2–3x, and — the crucial property — THE OUTPUT
DISTRIBUTION IS MATHEMATICALLY IDENTICAL to sampling
from the big model alone. It is free.

Variants: Medusa (extra heads on the same model),
EAGLE (feature-level drafting), and n-gram lookup
(no draft model at all — just match against the
prompt, which works remarkably well for
summarisation and code editing).
```

Context length

```
/* --- WHAT LIMITS IT --- */
    1. ATTENTION IS QUADRATIC in compute (chapter 19)
    2. THE KV CACHE IS LINEAR in memory - and at long
       context it exceeds the model itself
    3. POSITIONAL ENCODING must extrapolate (chapter 21)
    4. AND TRAINING DATA: long documents are rare, so the
       model has seen few genuine long-range dependencies

/* --- HOW LONG CONTEXTS WERE ACHIEVED --- */
    · ROPE FREQUENCY SCALING (position interpolation, NTK
      scaling, YaRN) - extend an 8k model to 128k with a
      short fine-tune (chapter 21)
    · FLASH ATTENTION - removes the O(n^2) MEMORY
      (chapter 19)
    · SLIDING WINDOW attention for most layers
    · GQA and KV-cache quantisation
    · training on synthetically constructed long documents

/* --- AND THE HONEST LIMITATION: LOST IN THE MIDDLE --- */

    Retrieval accuracy across a long context is not
    uniform. Models attend well to the BEGINNING and the
    END and measurably worse to the MIDDLE:

      accuracy
        |  \                      /
        |   \                   /
        |     \_______________/
        +-----------------------------> position in context
         start        middle        end

    A 128k CONTEXT WINDOW IS NOT 128k OF RELIABLE WORKING
    MEMORY. The practical implications:
      · put the important material at the START or the END
      · RETRIEVE THE RELEVANT PARTS (chapter 17) rather than
        stuffing everything in - shorter context is usually
        MORE accurate as well as cheaper
      · and test retrieval at YOUR context length, rather
        than trusting the advertised number

/* --- COST SCALES WITH CONTEXT --- */
    prefill compute   O(n^2)
    KV cache memory   O(n)
    per-token decode  O(n) attention + O(1) weights
    So a 100k-token prompt is not 10x a 10k prompt; it is
    considerably worse for time-to-first-token, and it
    permanently occupies a large amount of cache memory for
    the whole session.

/* --- SERVING METRICS TO MEASURE --- */
    TTFT      time to first token - dominated by prefill
    TPOT      time per output token - dominated by
              bandwidth
    THROUGHPUT total tokens/second across all requests
    GOODPUT   requests meeting their latency target

    AND THE TRADE-OFF TO UNDERSTAND: LARGER BATCHES RAISE
    THROUGHPUT AND RAISE PER-REQUEST LATENCY. There is no
    setting that optimises both; pick according to whether
```

```
    you are serving an interactive product or a batch
    pipeline.                                          */
```

How it shows up

Symptom	Cause
Generation slows quadratically with length	No KV cache
Out of memory at long context	KV cache exceeds the model — quantise it, or use GQA
GPU utilisation near zero while generating	Memory bound — expected; batch or quantise
Output loops "and then, and then"	Greedy or very low temperature — add top-p
Output incoherent	Temperature too high
Code output degraded by a repetition penalty	Code legitimately repeats identifiers — reduce it
JSON output occasionally malformed	Not using constrained decoding
Facts in the middle of a long prompt ignored	Lost in the middle — retrieve instead of stuffing
Time to first token very high	Prefill is $O(n^2)$ — shorten the prompt or cache the prefix
Throughput good, latency terrible	Batch size too large for an interactive workload
Few concurrent requests fit	Static cache allocation — use paged attention
Results not reproducible	Sampling enabled, or non-deterministic kernels

Check yourself

- ✓ Distinguish prefill and decode, and say which is compute bound and which memory bound.
- ✓ Compute tokens per second for a 70B fp16 model at 2 TB/s.
- ✓ Explain why the KV cache is valid, and what it changes asymptotically.
- ✓ Compute the KV cache size per token for Llama-3-8B and at 128k context.
- ✓ Explain paged attention and name the OS concept it borrows.
- ✓ Explain continuous batching and prefix caching.
- ✓ Compare greedy, top-k, top-p and min-p, and say why top-p beats top-k.
- ✓ Say when beam search helps and when it hurts.
- ✓ Explain constrained decoding and why it beats retrying.
- ✓ Explain speculative decoding and why the output distribution is unchanged.
- ✓ Give the four limits on context length and how each was addressed.
- ✓ Explain "lost in the middle" and three practical responses.
- ✓ Name four serving metrics and the throughput/latency trade-off.

Quantisation and efficient inference

Representing weights in 8 or 4 bits instead of 16. This is fixed-point arithmetic, scale factors and dynamic range — territory an embedded engineer already knows, applied to a new problem.

Why it works

```
THE MOTIVATION IS BANDWIDTH, NOT ARITHMETIC
(chapters 04, 25).

  70B model in fp16   140 GB   -> ~14 tokens/s at
                                  2 TB/s
  the same in int8     70 GB   -> ~28 tokens/s
  the same in int4     35 GB   -> ~56 tokens/s

THE SPEEDUP IS PROPORTIONAL TO THE BYTE REDUCTION,
because generation reads every weight once per token and
the multipliers are idle anyway. It also determines
whether the model FITS AT ALL - which for a device is
usually the real question.

AND WHY IT DOES NOT DESTROY THE MODEL:

  · WEIGHTS ARE ROUGHLY GAUSSIAN and tightly clustered
    around zero. A well-chosen scale covers the range
    with little error.
  · NEURAL NETWORKS ARE ROBUST TO NOISE. They were
    trained with dropout, augmentation and stochastic
    gradients; small perturbations to weights behave
    like more of the same.
  · THE ERROR AVERAGES OUT. A dot product over 4096
    terms sums many independent rounding errors, which
    partially cancel (chapter 08).
  · AND THE INFORMATION IS REDUNDANT. Superposition
    (chapter 17) means features are spread across many
    weights, so no single weight is critical.

THE FORMATS, and note what varies:

  FORMAT   BITS   RANGE           WHERE USED
  fp32       32   +-3.4e38        training master
                                  weights
  fp16       16   +-65504         older training, some
                                  inference
  bf16       16   +-3.4e38        MODERN TRAINING -
                                  fp32's range, fewer
                                  mantissa bits, so NO
                                  LOSS SCALING NEEDED
  fp8         8   two variants    newest accelerators
                  (E4M3, E5M2)
  int8        8   -128..127       THE EMBEDDED WORKHORSE
  int4        4   -8..7           LLM weights
  int2/
  ternary   2-1.6 {-1,0,1}        research, and
                                  BitNet-style models

THE PATTERN: FEWER BITS MEANS LESS RANGE OR LESS
PRECISION, AND THE SCALE FACTOR IS WHAT BUYS THE RANGE
BACK.
```

The arithmetic

```
/* --- THE BASIC MAPPING --- */

    QUANTISE:    q = round(x / scale) + zero_point
    DEQUANTISE:  x = (q - zero_point) * scale

/* SYMMETRIC (zero_point = 0): */
    scale = max(|x|) / 127                # int8
    q     = clamp(round(x / scale), -128, 127)

    Simpler and faster - no zero-point term in the inner
    loop. USE FOR WEIGHTS, which are roughly symmetric
    about zero.

/* ASYMMETRIC: */
    scale = (max - min) / 255
    zero_point = round(-min / scale)

    Uses the full range for skewed distributions. USE FOR
    ACTIVATIONS after ReLU, which are all non-negative -
    symmetric quantisation would waste half the codes.

/* --- WHY THE ACCUMULATOR IS WIDER (chapter 02) --- */
    int8 * int8 -> up to 16 bits, and summing 4096 of them
    needs ~28 bits. So hardware accumulates int8 x int8 into
    INT32, then rescales once at the end:

        acc = 0                           # int32
        for i: acc += a_q[i] * b_q[i]     # int8 x int8
        out = acc * (scale_a * scale_b)   # ONE rescale

    THE SCALES FACTOR OUT OF THE SUM. That is the whole
    trick that makes quantised matmul exact-ish and fast:
    the inner loop is pure integer MAC, and the floating
    point appears once.

/* --- GRANULARITY: the single most important choice --- */
    PER-TENSOR    one scale for the entire weight matrix.
                  Simplest, worst accuracy.
    PER-CHANNEL   one scale per output channel/row. MUCH
                  better, negligible extra cost.
    PER-GROUP     one scale per group of 32-128 weights
                  ALONG a row. What int4 LLM quantisation
                  uses.
                  Costs ~0.5 bits/weight in scale overhead
                  and is worth every bit of it.

    A SINGLE OUTLIER WEIGHT FORCES A LARGE SCALE, which
    coarsens every other weight sharing it. Finer groups
    contain the damage. THIS IS THE ENTIRE REASON GROUPED
    QUANTISATION EXISTS.

/* --- THE OUTLIER PROBLEM IN ACTIVATIONS --- */
    Transformer activations contain a small number of
    dimensions with values 10-100x larger than the rest,
    consistently in the same channels. Quantising naively
    destroys everything else.

    THE SOLUTIONS:
      LLM.int8()   keep the outlier dimensions in fp16, the
                   rest in int8. A mixed-precision matmul.
```

```
      SMOOTHQUANT  MIGRATE the difficulty from activations
                   into weights by scaling:
                       Y = (X / s) @ (s * W)
                   mathematically identical, and the
                   scaling is folded into the previous
                   layer's weights so it is free at
                   runtime. Weights are much easier to
                   quantise than activations, so this is a
                   genuinely elegant fix.
      AWQ          identify the ~1% of weight channels that
                   matter most (by activation magnitude) and
                   protect them with per-channel scaling.  */
```

The methods

```
/* --- PTQ: POST-TRAINING QUANTISATION --- */
    Quantise an already-trained model. No retraining.

    DYNAMIC   weights quantised offline, activations
              quantised at runtime from their observed
              range. Easy, no calibration data, some
              runtime cost.
    STATIC    activation ranges determined in advance from
              a CALIBRATION SET of a few hundred
              representative samples. Faster at runtime.
              THE CALIBRATION DATA MUST RESEMBLE PRODUCTION
              DATA, or the ranges are wrong.

    THE MODERN LLM METHODS:
      GPTQ    layer-by-layer, using second-order information
              to choose roundings that minimise the output
              error. int4 with small degradation.
      AWQ     activation-aware; protects salient channels.
              Fast, and generally excellent at int4.
      GGUF /  llama.cpp's family (Q4_K_M, Q5_K_M, Q8_0...).
      K-QUANTS Mixed precision per tensor type - attention
              and the output layer get more bits. THE
              PRACTICAL DEFAULT FOR RUNNING MODELS LOCALLY.
      HQQ,
      SqueezeLLM, and a steady stream of others

/* --- QAT: QUANTISATION-AWARE TRAINING --- */
    Simulate quantisation DURING training so the model
    learns weights that survive it.

      forward:  x_q = dequant(quant(x))   # "fake quant"
      backward: STRAIGHT-THROUGH ESTIMATOR - pass the
                gradient through as if quantisation were
                the identity, because round() has zero
                gradient everywhere (chapter 07)

    · better accuracy than PTQ, especially below 8 bits
    · needs training infrastructure and data
    · ESSENTIAL FOR VERY SMALL MODELS on MCUs, where PTQ
      at int8 can lose several percent of accuracy that QAT
      recovers

/* --- WHAT ACCURACY ACTUALLY COSTS (large models) --- */
    fp16      baseline
    int8      essentially lossless with per-channel scales
    int4 GPTQ/AWQ    perplexity +1-3%; usually
                     imperceptible in use
    int3      noticeable degradation
    int2      substantial, though active research
              continues

    AND THE RULE THAT MATTERS FOR CHOOSING:
      A 4-BIT 70B MODEL BEATS AN 8-BIT 34B MODEL AT THE SAME
      MEMORY BUDGET. Given a fixed number of gigabytes,
      prefer a larger model more aggressively quantised -
      up to about 4 bits, below which the relationship
      inverts.

/* --- THE CODE --- */
    # llama.cpp - the practical local path
```

```
#   python convert_hf_to_gguf.py ./model
#   ./llama-quantize model.gguf out.gguf Q4_K_M

# PyTorch dynamic PTQ, for a small model
import torch.ao.quantization as q
qmodel = q.quantize_dynamic(
    model, {torch.nn.Linear}, dtype=torch.qint8)

# bitsandbytes, at load time
from transformers import BitsAndBytesConfig
cfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_quant_type="nf4",     # normal-float 4,
                                   # matched to a
                                   # Gaussian weight
                                   # distribution
    bnb_4bit_use_double_quant=True)  # quantise the
                                     # scales too
model = AutoModelForCausalLM.from_pretrained(
            name, quantization_config=cfg)

# AND THIS IS WHAT QLoRA USES (chapter 24): a 4-bit
# frozen base plus bf16 adapters.
```

QUANTISE THE KV CACHE TOO

At long context the KV cache exceeds the model itself (chapter 25). Quantising it to int8 halves that, and to int4 quarters it. Keys tolerate quantisation less well than values — keys feed a dot product whose result goes through a softmax, so errors are amplified — so a common configuration is int8 keys with int4 values. For long-context serving this is often a larger win than quantising the weights.

The other compression techniques

```
/* --- PRUNING --- */
    UNSTRUCTURED   zero individual weights. High sparsity
                   achievable (50-90%), and NO SPEEDUP on
                   normal hardware, because the memory
                   access pattern is destroyed (chapter 03).
    STRUCTURED     remove whole channels, heads
                   (chapter 21), or layers. Lower achievable
                   sparsity, REAL SPEEDUP - the matrices are
                   genuinely smaller.
    2:4 SEMI-      exactly 2 of every 4 weights zero.
    STRUCTURED     Supported in hardware by recent NVIDIA
                   parts for a ~2x gain. The compromise that
                   actually ships.

    THE LESSON, which generalises far beyond this: SPARSITY
    ONLY HELPS IF THE HARDWARE CAN EXPLOIT ITS PATTERN.

/* --- DISTILLATION --- */
    Train a small STUDENT to match a large TEACHER.

      L = alpha * CE(student, hard_labels)
        + (1-alpha) * T^2 * KL(student_soft || teacher_soft)

    THE SOFT TARGETS CARRY MORE INFORMATION than the labels:
    a teacher saying "80% cat, 15% dog, 5% fox" tells the
    student about the similarity structure, which a one-hot
    label cannot. This is called DARK KNOWLEDGE, and it is
    why distillation beats training the small model
    directly.

    (The T^2 factor compensates for the gradients shrinking
    as 1/T^2 when the logits are softened.)

    DistilBERT: 40% smaller, 60% faster, 97% of the
    performance. And most good small models today are
    distilled from larger ones - which is why the small-model
    frontier moves whenever the large-model frontier does.

/* --- ARCHITECTURE-LEVEL CHOICES --- */
    · GQA / MQA           smaller KV cache (chapter 19)
    · sliding-window attention
    · fewer, wider layers - better for parallel hardware
    · WEIGHT SHARING across layers (ALBERT-style) - large
      memory saving, some quality cost
    · and simply TRAINING A SMALL MODEL LONGER
      (chapter 23), which is now the mainstream answer

/* --- COMPILATION AND KERNELS --- */
    torch.compile, TensorRT, ONNX Runtime, OpenVINO,
    TVM, ExecuTorch.

    WHAT THEY ACTUALLY DO - and it will be familiar from any
    embedded optimisation work:
      · OPERATOR FUSION: LayerNorm + Linear + GELU as one
        kernel, so intermediates never leave registers or
        SRAM. This is the single biggest win, and it is a
        MEMORY TRAFFIC optimisation.
      · constant folding
      · memory planning and buffer reuse
      · selecting the best kernel per shape
```

```
      · removing framework overhead entirely

    2–5x SPEEDUP FOR NO ACCURACY LOSS is typical, and it
    should be the FIRST thing you try – before quantising,
    before pruning, before anything that costs accuracy. */
```

How it shows up

Symptom	Cause
int8 model loses several percent accuracy	Per-tensor scales — switch to per-channel
Accuracy collapses despite per-channel scales	Activation outliers — use SmoothQuant or LLM.int8()
Quantised model fine in tests, poor in production	Calibration data unrepresentative
int4 output is incoherent	Group size too large, or a naive round-to-nearest method
Quantisation gives no speedup	Compute bound, not memory bound — check the phase
Overflow in a quantised matmul	Accumulator too narrow — int8 needs int32
Pruning to 70% sparsity gives no speedup	Unstructured — hardware cannot exploit it
Long-context serving still runs out of memory	KV cache not quantised
Distilled student worse than expected	Temperature too low — soft targets carry the signal
QAT gradients are always zero	Straight-through estimator missing — `round()` has no gradient
Small model on MCU loses accuracy at int8	PTQ is not enough at that size — use QAT

Check yourself

- ✓ Explain why quantisation speeds up generation, in terms of bytes not FLOPs.
- ✓ Give four reasons neural networks tolerate quantisation.
- ✓ Write the quantise and dequantise formulas, and say when to use symmetric versus asymmetric.
- ✓ Explain why the accumulator is int32 and how the scales factor out.
- ✓ Compare per-tensor, per-channel and per-group granularity, and say why outliers force finer groups.
- ✓ Explain the activation outlier problem and how SmoothQuant fixes it.
- ✓ Contrast PTQ and QAT, and explain the straight-through estimator.
- ✓ State the rule for choosing model size against bit width at a fixed memory budget.
- ✓ Say why keys tolerate KV-cache quantisation worse than values.
- ✓ Explain why unstructured pruning gives no speedup and what 2:4 sparsity is.
- ✓ Explain dark knowledge and the T^2 factor in distillation.
- ✓ Say what operator fusion does and why it should be tried first.

Running models on embedded hardware

The chapter this whole track was heading toward. Everything above, constrained by 256 KB of SRAM, a battery, no operating system, and a part that has to cost three dollars.

Know which tier you are in

```
"EDGE AI" SPANS FOUR ORDERS OF MAGNITUDE. The right
answer depends entirely on which tier you are in, and
conflating them is the most common mistake.

TIER 1 - MCU CLASS      Cortex-M0/M4/M33, RISC-V
  RAM      16 KB - 1 MB
  flash    64 KB - 2 MB
  clock    16-400 MHz, often no FPU
  power    mW, battery for months or years
  RUNS     keyword spotting, anomaly detection,
           gesture recognition, simple vision
  MODELS   1 KB - 500 KB, int8, often QAT
  STACK    TFLite Micro, CMSIS-NN, microTVM,
           ExecuTorch, or hand-written C

TIER 2 - MCU + NPU      Cortex-M55 + Ethos-U55/U85,
                        ESP32-S3, nRF54, Kendryte
  +        1-512 GOPS of int8 accelerator
  RUNS     real-time vision at low frame rates, speech
  MODELS   up to a few MB
  STACK    Vela compiler (Ethos), vendor toolchains

TIER 3 - APPLICATION PROCESSOR CLASS
                        i.MX8, Jetson Orin Nano, RK3588,
                        Raspberry Pi 5, phones
  RAM      1-32 GB, running Linux or Android
  NPU      1-100 TOPS
  RUNS     full vision pipelines, speech, AND SMALL
           LANGUAGE MODELS (1-8B, quantised)
  STACK    ONNX Runtime, TensorRT, OpenVINO, llama.cpp,
           NPU vendor SDKs

TIER 4 - EDGE SERVER    a GPU in a cabinet
  Essentially datacentre inference with a latency and
  privacy requirement. Chapter 25 applies unchanged.

THE QUESTION THAT DECIDES EVERYTHING:

  DOES THE MODEL'S WORKING SET FIT IN THE MEMORY YOU
  HAVE?

Not the parameter count - THE WORKING SET: weights plus
the largest activation buffer plus the framework's
arena. On an MCU this is the binding constraint, before
accuracy, before speed, before anything.
```

The memory budget

```
/* --- WHAT MUST FIT --- */

    FLASH (persistent)
      model weights (int8: 1 byte per parameter)
      the inference runtime      20-100 KB
      your application

    RAM (working)
      the TENSOR ARENA: the largest set of activation
        buffers live at once
      framework state
      your application's stack and heap

/* THE ARENA IS THE ONE PEOPLE GET WRONG. It is NOT the sum
   of all activations - a good runtime reuses buffers, so it
   is the peak of a liveness analysis, exactly like register
   allocation in a compiler (see the compilers track).

    For a small CNN it is typically driven by ONE layer -
    usually an early one, where the spatial dimensions are
    still large:

      a 96x96x3 input with 16 channels after the first conv
        = 96*96*16 = 147 KB in int8
      ALREADY TOO BIG for many parts, before any further
      layers.

    THE FIXES: a smaller input resolution, striding early,
    and depthwise separable convolutions.

/* --- A WORKED EXAMPLE: KEYWORD SPOTTING --- */
    Target: Cortex-M4, 256 KB flash, 64 KB RAM.

      input        49 x 10 MFCC features from 1 second of
                   audio
      model        DS-CNN, 4 depthwise separable blocks
      parameters   ~24k -> 24 KB in int8
      arena        ~21 KB
      runtime      ~45 KB flash (TFLM + CMSIS-NN)
      inference    ~10 ms at 80 MHz
      accuracy     ~95% on 12 keywords
      energy       runs continuously on a coin cell for
                   months with duty cycling

    THAT FITS COMFORTABLY, and it is a real, shipping class
    of product.

/* --- AND THE ONE THAT DOES NOT --- */
    A 1B-parameter language model at int4 = 500 MB.
    NOT ON AN MCU. NOT WITH ANY AMOUNT OF CLEVERNESS.
    That is Tier 3, and being clear about it early saves
    months.

/* --- THE ARITHMETIC TO DO FIRST, BEFORE ANY WORK --- */
    weights_bytes  = params * bytes_per_param
    arena_bytes    = peak concurrent activations
    flash_needed   = weights + runtime + application
    ram_needed     = arena + stack + heap

    inference_time ~= MACs / (MACs_per_cycle * clock_hz)
```

```
    Cortex-M4 with CMSIS-NN: ~1-2 int8 MACs/cycle
    Cortex-M55 + Ethos-U55:  hundreds to thousands
    -> a 5 MMAC model on an M4 at 80 MHz is ~60 ms;
       on an M55+U55 it is under a millisecond.

  energy_per_inference = time * average_current * voltage
    AND FOR A BATTERY DEVICE, COMPUTE THIS EARLY. It
    frequently rules out an architecture that would
    otherwise fit.                                    */
```

The stacks

```
/* --- TFLITE MICRO: the MCU standard --- */
    No dynamic allocation, no OS, no file system. You supply
    a static arena and it plans everything inside it.

    #include "tensorflow/lite/micro/micro_interpreter.h"

    // the model as a C array, in flash
    extern const unsigned char g_model[];

    constexpr int kArenaSize = 32 * 1024;
    alignas(16) static uint8_t tensor_arena[kArenaSize];

    void setup() {
        const tflite::Model* model =
            tflite::GetModel(g_model);

        // ONLY the operators you actually use - each one
        // linked in costs flash
        static tflite::MicroMutableOpResolver<6> resolver;
        resolver.AddConv2D();
        resolver.AddDepthwiseConv2D();
        resolver.AddFullyConnected();
        resolver.AddSoftmax();
        resolver.AddReshape();
        resolver.AddMaxPool2D();

        static tflite::MicroInterpreter interpreter(
            model, resolver, tensor_arena, kArenaSize);
        interpreter.AllocateTensors();
        // ^ FAILS HERE if the arena is too small, and the
        //   error tells you the required size. Start large,
        //   then shrink to fit.
    }

    void loop(void) {
        TfLiteTensor* in = interpreter.input(0);
        // QUANTISE YOUR INPUT WITH THE MODEL'S OWN
        // PARAMETERS - not with values you guessed
        for (int i = 0; i < n; i++)
            in->data.int8[i] =
                (int8_t)(raw[i] / in->params.scale
                         + in->params.zero_point);

        if (interpreter.Invoke() != kTfLiteOk) return;

        TfLiteTensor* out = interpreter.output(0);
        float score = (out->data.int8[0]
                       - out->params.zero_point)
                      * out->params.scale;
    }

/* --- CMSIS-NN --- */
    ARM's hand-optimised kernels for Cortex-M. 4-5x faster
    than reference C, using SIMD (SMLAD), DSP instructions
    and Helium (MVE) on M55.
    TFLM uses it automatically when built for Cortex-M -
    verify it actually is, because the reference kernels are
    the silent fallback and the difference is enormous.

/* --- ETHOS-U AND THE VELA COMPILER --- */
```

```
    vela model_int8.tflite --accelerator-config ethos-u55-128

    Vela AHEAD-OF-TIME compiles the graph into NPU command
    streams. Operators the NPU cannot handle FALL BACK TO THE
    CPU, and every fallback costs a round trip through
    memory.

    READ VELA'S REPORT. It tells you exactly which
    operations ran where. A model that is 90% NPU-resident
    is fast; one that ping-pongs is often SLOWER than pure
    CPU. CHECKING THIS IS THE SINGLE HIGHEST-VALUE
    OPTIMISATION STEP on an NPU target.

/* --- THE OTHERS --- */
    ExecuTorch   PyTorch's edge runtime. Growing quickly.
    microTVM     compile-based, aggressive
                 auto-scheduling
    ONNX Runtime Tier 3, with vendor execution providers
    llama.cpp    Tier 3 language models. GGUF, extremely
                 portable, runs on a Raspberry Pi
    OpenVINO     Intel-based edge
    TensorRT     NVIDIA Jetson
    Vendor SDKs  Qualcomm QNN, Rockchip RKNN, Hailo,
                 Ambarella - usually the fastest path on
                 their own silicon, and the least
                 portable                                */
```

DESIGN FOR THE TARGET FROM THE START

Do not train a model and then ask whether it fits. Pick the hardware, compute the budget, choose an architecture that fits with margin, and quantise *during* training if you are anywhere near the limit. Retrofitting a model into a device that cannot hold it is the most common and most expensive failure in embedded machine learning — and the second most common is discovering at integration time that the preprocessing, not the model, is what does not fit.

Architectures that fit

```
/* --- DEPTHWISE SEPARABLE CONVOLUTION --- */
    Split a standard convolution into two cheaper steps:
      DEPTHWISE   one k x k filter PER INPUT CHANNEL
      POINTWISE   a 1x1 convolution mixing channels

    COST:  standard   k*k * C_in * C_out
           separable  k*k * C_in  +  C_in * C_out

    For k=3, C_in=C_out=64:  36,864 -> 4,672. 8x FEWER.

    THIS IS THE SINGLE MOST IMPORTANT ARCHITECTURAL IDEA IN
    EMBEDDED VISION, and it underlies MobileNet,
    EfficientNet-Lite and essentially every model that runs
    on a microcontroller.

/* --- THE MODEL FAMILIES WORTH KNOWING --- */
    VISION       MobileNetV2/V3, EfficientNet-Lite,
                 MCUNet, NanoDet, YOLO-Nano
    AUDIO        DS-CNN, TC-ResNet for keyword spotting
    TIME SERIES  small 1-D CNNs, or a GRU (chapter 18) -
                 constant memory, which for a continuous
                 stream is decisive
    ANOMALY      an autoencoder: reconstruct the input and
                 threshold the error. NO LABELS NEEDED,
                 which matters because you usually have no
                 examples of the fault you are looking for.
    LANGUAGE     Tier 3 only. Qwen 0.5B, Gemma 3 270M/1B,
                 Llama 3.2 1B, SmolLM, Phi - all at int4

/* --- THE TECHNIQUES, IN THE ORDER TO APPLY THEM --- */
  1. REDUCE THE INPUT. 96x96 instead of 224x224 is 5.4x less
     work AND a much smaller arena. Usually the biggest
     single win, and usually the first thing to try.
  2. QUANTISE to int8, with QAT if accuracy matters
     (chapter 26).
  3. USE A MOBILE-CLASS ARCHITECTURE rather than shrinking
     a large one.
  4. PRUNE STRUCTURALLY - whole channels (chapter 26).
  5. DISTIL from a larger model.
  6. NEURAL ARCHITECTURE SEARCH under explicit hardware
     constraints, if the volume justifies it (MCUNet does
     exactly this).

/* --- AND THE ONE PEOPLE FORGET: PREPROCESSING --- */
    An MFCC pipeline needs an FFT, mel filterbanks and a
    DCT. Image preprocessing needs resize, colour conversion
    and normalisation.

    THIS CAN COST MORE THAN THE MODEL, and it needs its own
    buffers in the same RAM. Budget it explicitly. CMSIS-DSP
    provides fixed-point FFT and filter functions; use them
    rather than pulling in a float pipeline.               */
```

Getting it into a product

```
/* --- THE WORKFLOW --- */
    1. define the task, the METRIC, and the acceptable
       error rate (chapter 12)
    2. COLLECT DATA ON THE TARGET HARDWARE, with the
       production sensor. Data from a different microphone,
       a different lens, or a development board with a
       different ground plane WILL NOT TRANSFER.
    3. train on a workstation, in PyTorch or TensorFlow
    4. quantise, ideally with QAT
    5. convert and compile for the target
    6. VALIDATE ON DEVICE - accuracy AND latency AND
       memory AND power
    7. integrate, with a fallback path
    8. monitor in the field

/* --- STEP 6 IS WHERE PROJECTS FAIL --- */
    Accuracy measured on a workstation with float inputs is
    NOT the accuracy you will get. The differences:
      · int8 quantisation error
      · a different preprocessing implementation - a
        fixed-point FFT is not bit-identical to numpy's
      · real sensor noise, DC offset, gain drift, and
        temperature effects
      · timing jitter and dropped samples

    ALWAYS MEASURE END-TO-END ON THE ACTUAL DEVICE, from
    sensor to decision. A test harness that replays recorded
    raw sensor data through the real firmware path is worth
    building early.

/* --- THE ENGINEERING CONCERNS --- */

    DETERMINISM       fixed-size arena, no dynamic
                      allocation, bounded worst-case
                      inference time. If the model runs in
                      a control loop, its WCET is a
                      real-time constraint like any other
                      (see the OS track).
    POWER             duty cycle. A cheap always-on
                      detector (an analogue threshold, or a
                      tiny first-stage model) that WAKES the
                      real model is the standard cascade,
                      and it is often a 100x energy saving.
    UPDATES           the model is data. Ship it as a
                      SIGNED, VERSIONED blob through your
                      normal firmware update path - and
                      verify the signature before loading
                      it, exactly as for firmware
                      (the cryptography track). A model file
                      loaded without verification is
                      arbitrary code's worth of influence
                      over your device's behaviour.
    SAFETY            never let a model make an unbounded
                      safety-critical decision alone. Bound
                      its authority, add a deterministic
                      supervisor, and define the behaviour
                      when it is uncertain.
    OBSERVABILITY     log inputs that produce low-confidence
                      or unusual outputs, if you can do so
                      privately. Field data is how the next
```

```
                    version gets better - and drift
                    (chapter 12) is invisible without it.
    PRIVACY         and the strongest argument for on-
                    device inference: THE DATA NEVER
                    LEAVES. For audio, video and health
                    sensing that is not a nice-to-have,
                    it is often the whole reason the
                    product is viable.                */
```

How it shows up

Symptom	Cause
`AllocateTensors` fails	Arena too small — the error reports the size needed
Model fits in flash but crashes at runtime	Arena plus stack exceeds RAM
Inference 5× slower than expected	Reference kernels instead of CMSIS-NN
NPU target slower than CPU	Operator fallbacks ping-ponging — read the Vela report
Accuracy far below the workstation figure	Preprocessing mismatch, or PTQ where QAT was needed
Works on the dev board, fails in the product	Different sensor — collect data on the target hardware
Battery life far below the estimate	No duty cycling — always-on inference
Occasional missed deadline in a control loop	Unbounded inference time — measure WCET
Preprocessing does not fit	FFT buffers not budgeted alongside the arena
Field accuracy degrades over months	Distribution drift with no monitoring
Someone loaded a modified model file	Model blob not signature-verified

Check yourself

- ✓ Describe the four edge tiers and what each can realistically run.
- ✓ Explain what a tensor arena is and why it is not the sum of activations.
- ✓ Compute the memory budget for a keyword-spotting model on a 64 KB part.
- ✓ Estimate inference time from MAC count and MACs per cycle.
- ✓ Write the TFLite Micro setup, including why the op resolver is explicit.
- ✓ Explain how to quantise an input using the model's own scale and zero point.
- ✓ Say what Vela does and why operator fallbacks matter so much.
- ✓ Derive the cost saving of depthwise separable convolution.
- ✓ List the six size-reduction techniques in the order to apply them.
- ✓ Explain why preprocessing must be budgeted explicitly.
- ✓ Give five reasons on-device accuracy differs from workstation accuracy.
- ✓ Name six engineering concerns for shipping a model, including update signing.

Project: a tiny transformer from scratch

Build, train and run a character-level language model end to end. Every line traces back to a chapter in this track, it runs on a laptop CPU in minutes, and at the end you export it to C and run it on a microcontroller.

What you will build

```
PART A   a character tokeniser              (chapter 16)
PART B   the model, in PyTorch          (chapters 19–22)
PART C   the training loop              (chapters 10, 11)
PART D   sampling and generation            (chapter 25)
PART E   quantise to int8 and export to C
                                       (chapters 26, 27)

TARGET: a model that learns the structure of a text
corpus well enough to produce recognisable output.

THE SPECIFICATION:
  vocabulary      ~65 characters
  d_model         128
  layers          4
  heads           4
  context         128 characters
  parameters      ~800k
  training        ~10 minutes on a laptop CPU,
                  ~30 seconds on any GPU
  int8 export     ~800 KB

THAT LAST NUMBER MATTERS: 800 KB does not fit on a
Cortex-M4 (chapter 27). Part E shrinks it to a version
that does, and the exercise of finding out WHY is more
instructive than being handed a model that already fits.

PREREQUISITES:
  pip install torch numpy

THE DATA: any plain text file of a megabyte or more.
The classic choice is the concatenated works of
Shakespeare (input.txt, ~1.1 MB, widely available), but
USE SOMETHING YOU KNOW WELL — your own logs, a codebase,
a protocol dump. You will judge the output by whether it
looks right, and that requires knowing what right looks
like.
```

Part A — data and tokenisation

```
# ============ prepare.py ============
import torch

text = open('input.txt', 'r', encoding='utf-8').read()
print(f"{len(text):,} characters")

# CHARACTER-LEVEL TOKENISATION (chapter 16).
# No BPE, no merge list, no vocabulary file - just the set
# of characters that occur. Small vocabulary, long
# sequences, and nothing to get wrong.
chars = sorted(set(text))
vocab_size = len(chars)
print(f"vocab: {vocab_size} -> {''.join(chars)}")

stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for i, ch in enumerate(chars)}
encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join(itos[i] for i in l)

data = torch.tensor(encode(text), dtype=torch.long)

# TRAIN/VALIDATION SPLIT (chapter 12).
# NOTE: split by POSITION, not randomly - this is sequential
# data, and shuffling would let the model see continuations
# of its own training text.
n = int(0.9 * len(data))
train_data, val_data = data[:n], data[n:]

BLOCK = 128        # context length
BATCH = 32

def get_batch(split):
    d = train_data if split == 'train' else val_data
    # random starting offsets, and stack into a batch
    ix = torch.randint(len(d) - BLOCK - 1, (BATCH,))
    x = torch.stack([d[i:i+BLOCK]     for i in ix])
    y = torch.stack([d[i+1:i+BLOCK+1] for i in ix])
    #                  ^^^ SHIFTED BY ONE. That is the
    #                  entire supervision signal
    #                  (chapter 23).
    return x, y

xb, yb = get_batch('train')
print(xb.shape, yb.shape)     # (32, 128) (32, 128)
print(repr(decode(xb[0][:40].tolist())))
print(repr(decode(yb[0][:40].tolist())))
# The second is the first, shifted left by one character.
```

Part B — the model

```
# ============ model.py ============
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class Config:
    vocab_size = 65
    block_size = 128
    n_layer    = 4
    n_head     = 4
    d_model    = 128
    dropout    = 0.1

class SelfAttention(nn.Module):
    """Chapters 19 and 21."""
    def __init__(self, c):
        super().__init__()
        assert c.d_model % c.n_head == 0
        self.n_head = c.n_head
        self.d_head = c.d_model // c.n_head
        # Q, K, V in one matmul (chapter 04)
        self.qkv  = nn.Linear(c.d_model, 3*c.d_model,
                              bias=False)
        self.proj = nn.Linear(c.d_model, c.d_model,
                              bias=False)
        self.drop = nn.Dropout(c.dropout)
        self.p = c.dropout

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        # (B,T,C) -> (B,n_head,T,d_head): heads become a
        # batch dimension (chapter 21)
        sp = lambda t: t.view(B, T, self.n_head,
                              self.d_head).transpose(1, 2)
        q, k, v = sp(q), sp(k), sp(v)

        # Flash Attention where available; is_causal gives
        # the mask (chapter 19)
        y = F.scaled_dot_product_attention(
                q, k, v, is_causal=True,
                dropout_p=self.p if self.training else 0.0)

        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.drop(self.proj(y))

class MLP(nn.Module):
    """Chapter 22. Two thirds of the parameters live here."""
    def __init__(self, c):
        super().__init__()
        self.fc   = nn.Linear(c.d_model, 4*c.d_model)
        self.proj = nn.Linear(4*c.d_model, c.d_model)
        self.drop = nn.Dropout(c.dropout)
    def forward(self, x):
        return self.drop(self.proj(F.gelu(self.fc(x))))

class Block(nn.Module):
    """PRE-NORM, with both residuals (chapter 22)."""
    def __init__(self, c):
```

```
        super().__init__()
        self.ln1, self.attn = nn.LayerNorm(c.d_model), SelfAttention(c)
        self.ln2, self.mlp  = nn.LayerNorm(c.d_model), MLP(c)
    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.mlp(self.ln2(x))
        return x

class TinyGPT(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.c = c
        self.wte = nn.Embedding(c.vocab_size, c.d_model)
        self.wpe = nn.Embedding(c.block_size, c.d_model)
        self.drop = nn.Dropout(c.dropout)
        self.blocks = nn.ModuleList(
            [Block(c) for _ in range(c.n_layer)])
        self.ln_f = nn.LayerNorm(c.d_model)
        self.head = nn.Linear(c.d_model, c.vocab_size,
                              bias=False)
        self.head.weight = self.wte.weight   # weight tying
                                             # (chapter 17)

        self.apply(self._init)
        # scale residual projections by 1/sqrt(2*n_layer)
        # (chapter 15)
        for name, p in self.named_parameters():
            if name.endswith('proj.weight'):
                nn.init.normal_(p, 0.0,
                    0.02 / math.sqrt(2 * c.n_layer))

    def _init(self, m):
        if isinstance(m, (nn.Linear, nn.Embedding)):
            nn.init.normal_(m.weight, 0.0, 0.02)
            if isinstance(m, nn.Linear) and m.bias is not None:
                nn.init.zeros_(m.bias)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        assert T <= self.c.block_size
        pos = torch.arange(T, device=idx.device)
        x = self.drop(self.wte(idx) + self.wpe(pos))
        for b in self.blocks:
            x = b(x)
        logits = self.head(self.ln_f(x))

        loss = None
        if targets is not None:
            # RAW LOGITS into cross_entropy (chapter 10)
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)),
                targets.view(-1))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new, T=0.8, top_k=None):
        """Chapter 25. No KV cache here - at this size it is
           unnecessary, and adding it is exercise 4."""
        for _ in range(max_new):
            # CROP to block_size: learned absolute positions
            # have a hard limit (chapter 21)
            ctx = idx[:, -self.c.block_size:]
            logits, _ = self(ctx)
```

```
            logits = logits[:, -1, :] / T       # last
                                                # position
            if top_k is not None:
                v, _ = torch.topk(logits,
                                  min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = -float('inf')
            probs = F.softmax(logits, dim=-1)
            nxt = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, nxt), dim=1)
        return idx

model = TinyGPT(Config())
print(f"{sum(p.numel() for p in model.parameters())/1e3:.0f}k "
      f"parameters")
```

Part C — training

```
# ============ train.py ============
import math, time, torch

cfg = Config(); cfg.vocab_size = vocab_size
model = TinyGPT(cfg)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model.to(device)

MAX_STEPS, WARMUP = 5000, 100
PEAK_LR, MIN_LR   = 1e-3, 1e-4

# AdamW, betas from chapter 11; no weight decay on
# 1-D parameters (chapter 15)
decay    = [p for p in model.parameters() if p.dim() >= 2]
no_decay = [p for p in model.parameters() if p.dim() < 2]
opt = torch.optim.AdamW(
    [{'params': decay,    'weight_decay': 0.1},
     {'params': no_decay, 'weight_decay': 0.0}],
    lr=PEAK_LR, betas=(0.9, 0.95))

def lr_at(step):                        # chapter 11
    if step < WARMUP:
        return PEAK_LR * step / WARMUP
    p = (step - WARMUP) / (MAX_STEPS - WARMUP)
    return MIN_LR + 0.5*(PEAK_LR-MIN_LR)*(1+math.cos(math.pi*p))

@torch.no_grad()
def estimate_loss(iters=50):
    model.eval()                        # <<< chapter 15
    out = {}
    for split in ('train', 'val'):
        losses = torch.zeros(iters)
        for k in range(iters):
            x, y = get_batch(split)
            _, loss = model(x.to(device), y.to(device))
            losses[k] = loss.item()
        out[split] = losses.mean().item()
    model.train()
    return out

# ---- SANITY CHECK BEFORE TRAINING (chapter 15) ----
x, y = get_batch('train')
_, loss0 = model(x.to(device), y.to(device))
print(f"step-0 loss {loss0.item():.4f}, "
      f"expected ~{math.log(vocab_size):.4f}")
# IF THESE DISAGREE, STOP AND FIND THE BUG. Do not train.

t0 = time.time()
for step in range(MAX_STEPS):
    for g in opt.param_groups:
        g['lr'] = lr_at(step)

    x, y = get_batch('train')
    _, loss = model(x.to(device), y.to(device))

    opt.zero_grad(set_to_none=True)        # chapter 07
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
```

```
    if step % 500 == 0 or step == MAX_STEPS - 1:
        l = estimate_loss()
        print(f"step {step:5d}  train {l['train']:.4f}  "
              f"val {l['val']:.4f}  "
              f"ppl {math.exp(l['val']):.1f}  "
              f"{time.time()-t0:.0f}s")

torch.save({'model': model.state_dict(),
            'stoi': stoi, 'itos': itos,
            'cfg': cfg.__dict__}, 'tiny.pt')

# ---- WHAT YOU SHOULD SEE ----
# step     0  train 4.1745  val 4.1748   ppl 65.0
# step  1000  train 1.9421  val 2.0134    ppl 7.5
# step  3000  train 1.5502  val 1.6789    ppl 5.4
# step  4999  train 1.4231  val 1.5993    ppl 4.9
#
# Loss starts at log(65) = 4.17 - a uniform distribution.
# Perplexity 4.9 means the model is choosing between about
# five characters at each step (chapter 10). For English
# text that is roughly the right order.
#
# TRAIN AND VAL SHOULD TRACK EACH OTHER. If val turns
# upward while train keeps falling, you are overfitting -
# more data, more dropout, or fewer steps (chapter 12).
```

Part D — generation

```
# ============ sample.py ============
ck = torch.load('tiny.pt')
cfg = Config()
cfg.__dict__.update(ck['cfg'])
model = TinyGPT(cfg); model.load_state_dict(ck['model'])
model.eval()
itos = ck['itos']
decode = lambda l: ''.join(itos[i] for i in l)

ctx = torch.zeros((1, 1), dtype=torch.long)   # start token
out = model.generate(ctx, max_new=500, T=0.8, top_k=40)
print(decode(out[0].tolist()))

# ---- EXPERIMENT WITH TEMPERATURE (chapters 08, 25) ----
for T in (0.2, 0.5, 0.8, 1.0, 1.5):
    print(f"--- T = {T} ---")
    print(decode(model.generate(ctx, 200, T=T)[0].tolist()))

# T=0.2  repetitive, often loops on common words
# T=0.8  the usual sweet spot - word-like, punctuated,
#        structurally plausible
# T=1.5  noise with occasional real words
#
# THE MODEL LEARNS, IN ROUGHLY THIS ORDER:
#   ~200 steps    character frequencies; spaces
#   ~500 steps    word-like letter clusters
#   ~1500 steps   real short words, line structure
#   ~3000 steps   punctuation, capitalisation, dialogue
#                 formatting
#   ~5000 steps   short grammatical phrases
#
# WATCHING THAT PROGRESSION IS THE POINT OF THIS PROJECT.
# Sample every 500 steps and keep the outputs.

# ---- CONDITIONED GENERATION ----
prompt = "ROMEO:"
ids = torch.tensor([[stoi[c] for c in prompt]])
print(decode(model.generate(ids, 300, T=0.8)[0].tolist()))
```

Part E — quantise and export to C

```
# ============ export.py ============
"""Chapters 26 and 27: int8 symmetric per-channel
   quantisation, and a C header the firmware can compile."""
import numpy as np, torch

ck = torch.load('tiny.pt')
sd = ck['model']

def quantise_per_channel(W):
    """W: (out, in) torch tensor -> int8 array + per-row
       scales. PER-CHANNEL, because per-tensor loses
       measurable accuracy (chapter 26)."""
    W = W.detach().float().numpy()
    scale = np.abs(W).max(axis=1, keepdims=True) / 127.0
    scale = np.maximum(scale, 1e-8)          # never divide
                                             # by zero
    q = np.clip(np.round(W / scale), -127, 127).astype(np.int8)
    return q, scale.squeeze(1).astype(np.float32)

total_fp32 = total_int8 = 0
with open('model_weights.h', 'w') as f:
    f.write("/* generated by export.py - do not edit */\n")
    f.write("#ifndef MODEL_WEIGHTS_H\n"
            "#define MODEL_WEIGHTS_H\n#include <stdint.h>\n\n")
    for name, W in sd.items():
        cname = name.replace('.', '_')
        if W.dim() == 2:                      # quantise
                                              # matrices
            q, s = quantise_per_channel(W)
            total_fp32 += W.numel() * 4
            total_int8 += q.size + s.size * 4
            f.write(f"/* {name} {tuple(W.shape)} */\n")
            f.write(f"static const int8_t {cname}[{q.size}] "
                    "= {\n  ")
            f.write(','.join(str(int(v)) for v in q.ravel()))
            f.write("\n};\n")
            f.write(f"static const float {cname}_scale"
                    f"[{s.size}] = {{")
            f.write(','.join(f"{v:.8e}f" for v in s))
            f.write("};\n\n")
        else:                                 # keep 1-D
                                              # params in
                                              # float
            v = W.detach().float().numpy().ravel()
            total_fp32 += v.size * 4
            total_int8 += v.size * 4
            f.write(f"static const float {cname}[{v.size}] "
                    "= {")
            f.write(','.join(f"{x:.8e}f" for x in v))
            f.write("};\n\n")
    f.write("#endif\n")

print(f"fp32 {total_fp32/1024:.0f} KB -> "
      f"int8 {total_int8/1024:.0f} KB  "
      f"({total_fp32/total_int8:.2f}x)")

# ---- AND THE C INFERENCE KERNEL ----
/*
 * The one function that matters. Everything else in a
 * transformer is bookkeeping around this (chapter 04).
```

```
 */
void qmatmul(float *out, const float *in,
             const int8_t *W, const float *scale,
             int n_out, int n_in)
{
    for (int o = 0; o < n_out; o++) {
        int32_t acc = 0;            /* WIDE ACCUMULATOR
                                       (chapter 26) */
        const int8_t *row = &W[o * n_in];
        for (int i = 0; i < n_in; i++)
            acc += (int32_t)row[i] * (int32_t)in[i];
        out[o] = (float)acc * scale[o];   /* ONE rescale */
    }
}
/*
 * NOTE: this takes float `in` and multiplies by int8 - a
 * simplification. A fully-integer pipeline quantises the
 * activations too, and the scales multiply
 * (chapter 26). Exercise 8 is to make it fully integer.
 *
 * ON A CORTEX-M4, replace the inner loop with CMSIS-NN's
 * arm_nn_mat_mult_nt_t_s8 and it gets 4-5x faster via SMLAD
 * (chapter 27).
 */
```

WHAT THE EXPORT TELLS YOU

```
800k parameters at int8  ~= 800 KB
plus the activation arena, plus the runtime.

THAT DOES NOT FIT ON A CORTEX-M4 (chapter 27).

To fit in 256 KB of flash:
  d_model 128 -> 64      (4x fewer parameters:
                          they scale as d²)
  layers  4   -> 3
  vocab   65  -> 65      (unchanged; already small)
  -> ~150k parameters, ~150 KB int8. FITS.

Retrain at that size and compare the samples. The
quality drop is real and visible - and measuring it
yourself is worth more than any table of numbers.
```

Exercises

```
/* IN ROUGHLY INCREASING DIFFICULTY. Each one exercises a
   specific chapter.                                      */

  1. WATCH IT LEARN. Sample every 500 steps and save the
     output. Plot the loss curve. (chapters 11, 25)

  2. SCALING LAWS AT SMALL SCALE. Train d_model in
     {32,64,128,256} at fixed steps and plot final loss
     against parameter count on log-log axes. YOU SHOULD SEE
     A STRAIGHT LINE. (chapter 23)

  3. BREAK IT DELIBERATELY, one at a time, and predict the
     symptom before you run it:
       · remove the positional embedding      (ch. 21)
       · remove `is_causal=True`              (ch. 19)
       · remove the residual connections      (ch. 22)
       · use post-norm instead of pre-norm    (ch. 22)
       · remove the 1/sqrt(d_k) scaling - implement
         attention by hand to do this         (ch. 19)
       · set the learning rate to 1e-1        (ch. 11)
     THIS IS THE MOST INSTRUCTIVE EXERCISE HERE.

  4. ADD A KV CACHE to generate(). Measure tokens/second
     before and after at 128 context. (chapter 25)

  5. SWAP LEARNED POSITIONS FOR ROPE. Then generate BEYOND
     block_size and see whether it degrades gracefully.
     (chapter 21)

  6. REPLACE THE MLP WITH SwiGLU, keeping the parameter
     count equal by setting hidden = 8/3 * d_model.
     (chapter 22)

  7. TRAIN A BPE TOKENISER on the same corpus (chapter 16's
     code), retrain, and compare BITS PER BYTE - not
     perplexity, which is not comparable across tokenisers.
     (chapters 10, 16)

  8. FULLY-INTEGER INFERENCE in C: quantise the activations
     too, so no float appears in the inner loop. Compare
     output against the PyTorch model on the same prompt.
     (chapter 26)

  9. RUN IT ON HARDWARE. Shrink to fit, export, and run on
     a Raspberry Pi Pico or any Cortex-M4 board. Measure
     inference time, flash usage and peak RAM.
     (chapter 27)

 10. FINE-TUNE IT. Train on Shakespeare, then continue
     training on a different corpus with a low learning
     rate. Observe CATASTROPHIC FORGETTING directly.
     (chapter 24)

 11. IMPLEMENT LoRA on the attention projections and
     fine-tune with the base frozen. Compare the forgetting
     against exercise 10. (chapter 24)

 12. CONSTRAINED DECODING: force the output to match a
     simple grammar - balanced brackets, or a fixed
     line format - by masking invalid tokens.
```

```
     (chapter 25)

/* --- WHERE TO GO NEXT --- */
    · nanoGPT (Karpathy) - the same design, production
      quality, and the reference this project follows
    · llm.c - GPT-2 training in plain C/CUDA. Read it after
      exercise 8.
    · llama.cpp - study the quantisation formats
      (chapter 26) and the GGUF layout
    · TFLite Micro examples - the keyword-spotting model
      from chapter 27, complete
    · and the papers: "Attention Is All You Need",
      "Language Models are Few-Shot Learners",
      "Training Compute-Optimal Large Language Models",
      "LoRA", "FlashAttention". You now have the background
      to read all of them.                              */
```

How it shows up

Symptom	Cause
Step-0 loss is not `log(vocab_size)`	A bug in the model, data or loss — fix before training
Loss drops below 0.5 quickly	Causal mask missing — the model is reading the answer
Loss flat at 4.17	Gradients not flowing — check `zero_grad` and the optimiser's parameters
Loss becomes `nan`	Learning rate too high, or warmup skipped
Validation loss rises while training falls	Overfitting — the corpus is too small for the model
Output is one character repeated	Temperature too low, or undertrained
Output is pure noise at 5000 steps	Positional embedding missing, or a shape bug
Generation crashes past 128 tokens	Context not cropped to `block_size`
C output differs from PyTorch	Quantisation applied per-tensor, or scales indexed wrongly
Exported model too large for the target	Parameters scale as `d_model²` — halve it
C inner loop overflows	Accumulator not int32

Check yourself

- ✓ Explain why the train/validation split is by position rather than random.
- ✓ Show that `y` is `x` shifted by one and say why that is the whole supervision signal.
- ✓ Explain each line of the `SelfAttention` forward pass, with shapes.
- ✓ Say what the residual-projection initialisation scaling does.
- ✓ Explain why `generate` crops the context.
- ✓ Predict the step-0 loss and explain the calculation.
- ✓ Describe the order in which the model learns structure.
- ✓ Explain the effect of temperature at 0.2, 0.8 and 1.5.
- ✓ Explain per-channel quantisation and why the accumulator is int32.
- ✓ Compute how to shrink the model to fit 256 KB, and why `d_model` is the lever.
- ✓ Do exercise 3 and predict every symptom before running it.

TRACK

DSA interview prep

Pattern-based data-structures and algorithms practice, basic to hard, with an in-browser judge that runs your solution against real test cases.

38 chapters

How to practise, and the pattern method

Interview problems look infinite. They are not — they are perhaps thirty recurring patterns in different costumes. This track teaches the patterns, and gives you a judge in the page so you write and run code rather than reading about it.

Why patterns, not problems

```
THE FAILING STRATEGY: grind 500 problems, one at a time,
memorising solutions.

  · you meet problem 501 in the interview and it is not
    one you memorised
  · you cannot explain WHY the solution works, and the
    follow-up question exposes that immediately
  · retention is poor, because nothing connects

THE WORKING STRATEGY: learn the PATTERN, then solve
several problems that wear it.

  "Find a pair summing to a target in a SORTED array"
  "Find the container holding the most water"
  "Remove duplicates in place"
  "Sort an array of 0s, 1s and 2s"
  "Trap rainwater"

FIVE DIFFERENT PROBLEMS. ONE PATTERN: two pointers moving
toward each other, or in the same direction, exploiting
an ordering invariant.

Once you SEE the pattern, problem 501 is recognisable
even though you have never met it, because you are not
matching the problem - you are matching its SHAPE.

THE RECOGNITION SIGNALS ARE LEARNABLE, and each chapter
in this track states them explicitly:

  "sorted array" + "pair/triplet"      -> two pointers
  "contiguous subarray" + "max/min"    -> sliding window
  "kth largest"                        -> heap
  "all permutations/combinations"      -> backtracking
  "count the ways" / "min cost"        -> DP
  "shortest path, unweighted"          -> BFS
  "cycle in a linked list"             -> fast/slow
  "next greater element"               -> monotonic stack
  "prefix/word lookup"                 -> trie
  "connected components"               -> union-find or DFS

THAT TABLE IS MOST OF THE INTERVIEW. The rest is
implementing it without bugs, which is what the runnable
problems here are for.
```

The method for a single problem

```
/* Follow this EVERY TIME, including in the interview. It is
   also exactly what the interviewer is scoring you on. */

  1. RESTATE THE PROBLEM IN YOUR OWN WORDS.
     "So I'm given a sorted array and a target, and I need
      the indices of two numbers that add to it. Is there
      always exactly one answer?"
     -> catches misunderstanding before you write anything,
        and it is the single highest-scoring 30 seconds of
        the whole interview.

  2. ASK ABOUT THE CONSTRAINTS AND THE EDGE CASES.
     · How large is n? (this DETERMINES the target
       complexity - chapter 02)
     · Sorted? Duplicates? Negative numbers? Empty input?
     · Can I modify the input?
     · What should I return if there is no answer?

  3. WORK A SMALL EXAMPLE BY HAND.
     On paper, with 4-5 elements. THIS IS WHERE THE PATTERN
     REVEALS ITSELF. Skipping this step is the most common
     cause of a wrong approach confidently pursued.

  4. STATE THE BRUTE FORCE, WITH ITS COMPLEXITY.
     "I could check every pair - O(n^2). Let me improve
      that."
     Never stay silent while thinking. And never refuse to
     state a brute force because it is "too obvious" - it
     establishes a baseline and buys you thinking time.

  5. FIND THE INSIGHT THAT REMOVES WORK.
     Usually one of: the input is SORTED (binary search or
     two pointers), I am RECOMPUTING something (memoise or
     prefix sum), I only need the TOP FEW (heap), or there
     is REDUNDANT STATE I can drop.

  6. STATE THE APPROACH AND ITS COMPLEXITY BEFORE CODING.
     "Two pointers from both ends, O(n) time, O(1) space.
      Shall I code that?"
     -> gives the interviewer a chance to redirect you
        BEFORE you spend ten minutes on the wrong thing.

  7. CODE IT. Talk while you type. Use real names.

  8. TRACE IT ON THE EXAMPLE, out loud, line by line.
     You will find your own bug about a third of the time,
     and finding your own bug scores far better than the
     interviewer finding it.

  9. STATE THE EDGE CASES and check them: empty, one
     element, all equal, negatives, overflow, the target
     absent.

 10. STATE THE FINAL COMPLEXITY, time AND space.

/* AND THE THING PEOPLE GET WRONG: THE INTERVIEW IS NOT A
   TEST OF WHETHER YOU KNOW THE ANSWER. It is a simulation
   of working with you. A candidate who reasons clearly to
```

```
a good-but-not-optimal solution usually outscores one
who silently types a memorised optimal answer.        */
```

How this track is built

```
EVERY CHAPTER HAS THE SAME SHAPE:

  THE PATTERN        what it is, as a diagram or template
  WHEN TO REACH      the recognition signals
  FOR IT             ("if you see X, try Y")
  THE TEMPLATE       code you can adapt, with the
                     invariant stated
  PROBLEMS           4-6, EASY -> MEDIUM -> HARD, each
                     with:
                       · statement and examples
                       · hints, if you want a nudge
                       · an APPROACH LADDER: brute force,
                         better, optimal, with complexity
                         for each
                       · AN EDITOR THAT RUNS YOUR CODE
                         against real test cases
                       · reference solutions in
                         JavaScript, C++, Python and Java
                       · a walkthrough of why it works
  HOW IT SHOWS UP    the bugs this pattern produces
  CHECK YOURSELF     recall questions

THE ORDER TO READ THEM:

  02        complexity - do not skip this one
  03        the playground
  04-08     arrays: two pointers, sliding window,
            prefix sums, Kadane, matrices
  09-12     searching and sorting
  13-14     hashing
  15-16     bit manipulation
  17-18     linked lists
  19-21     stacks and queues
  22        heaps
  23-26     trees and tries
  27-30     graphs
  31-32     recursion and backtracking
  33-37     dynamic programming
  38        the interview itself, plus a mixed set

ARRAYS AND HASHING FIRST is deliberate: they appear
inside every later topic. DP LAST is also deliberate -
it is the hardest, and it is much easier once recursion
(31) is comfortable.
```

THE JUDGE IN THE PAGE

Every problem has a real editor. Write a solution, press **RUN TESTS** (or Ctrl/⌘+Enter), and it executes against the test cases and shows you which ones fail, with the input, what was expected, and what you actually returned. It runs *in your browser* — no server, no account, no network. Your code is saved in local storage per problem, so you can close the tab and come back.

The runner executes **JAVASCRIPT**, because that is what a browser can run with zero dependencies. Reference solutions are given in **C++, Python and Java** as well, since those are what you will actually write in an interview. If your target language is C++, read the C++ solution and type the JavaScript — the *algorithm* is what you are practising, and the syntax difference for these problems is small.

A realistic schedule

```
/* --- IF YOU HAVE 8+ WEEKS --- */
    week 1     complexity, arrays (02-08)
    week 2     searching, sorting, hashing (09-14)
    week 3     bits, linked lists (15-18)
    week 4     stacks, queues, heaps (19-22)
    week 5     trees and tries (23-26)
    week 6     graphs (27-30)
    week 7     recursion, backtracking (31-32)
    week 8     dynamic programming (33-37)
    then       mixed practice under time pressure (38)

    3-5 problems a day, done PROPERLY, beats 15 skimmed.

/* --- IF YOU HAVE 2 WEEKS --- */
    Do the EASY and MEDIUM problems only, in this order:
      arrays/hashing (04,05,06,13) -> binary search (09,10)
      -> linked lists (17,18) -> trees (23,24) ->
      graphs (27,28) -> DP intro (33)
    Then chapter 38.
    THESE SIX AREAS COVER THE LARGE MAJORITY OF WHAT IS
    ACTUALLY ASKED.

/* --- IF YOU HAVE 2 DAYS --- */
    Read chapter 38, then do the EASY problems in 04, 05,
    09, 13, 17, 23, 27. Accept that you are optimising for
    "not freezing" rather than for solving a hard problem.

/* --- THE RULES THAT MAKE PRACTICE WORK --- */

  · ATTEMPT BEFORE READING. Give every problem an honest
    20-25 minutes before opening the hints. The struggle IS
    the learning; reading a solution you did not struggle
    with teaches almost nothing.

  · WHEN YOU DO LOOK, CLOSE IT AND RETYPE FROM MEMORY.
    Understanding a solution and being able to produce one
    are different skills, and only the second is tested.

  · REDO PROBLEMS. Anything you needed help with, redo in
    three days, then in a week. Spaced repetition works
    here exactly as it does for vocabulary.

  · SAY IT OUT LOUD. Practise narrating. Interviews are
    verbal, and the first time you try to think out loud
    should not be in the interview.

  · TIME YOURSELF eventually. 20 minutes easy, 30-35
    medium. But only after the pattern is solid - timing
    yourself while still learning just creates anxiety.

  · KEEP A LOG of every problem you got wrong and WHY:
    "off-by-one in the binary search bound", "forgot the
    empty-input case", "did not see it was a graph". After
    thirty entries the log tells you exactly what to drill,
    and it is almost never what you assumed.            */
```

How it shows up

Symptom	Cause
Solved 300 problems, still fail interviews	Memorising solutions instead of patterns — and not practising out loud
Freeze completely on an unfamiliar problem	No process to fall back on — use the ten steps
Jump to code and write the wrong algorithm	Skipped the hand-worked example (step 3)
Interviewer stops you halfway	You did not state the approach before coding (step 6)
Solution works, interviewer unconvinced	Never stated the complexity, or could not justify correctness
Can follow a solution but not produce one	Reading without retyping from memory
Same bug class again and again	No error log — you are not measuring your own weaknesses
Good at easy, collapse on medium	Pattern not yet recognised — go back to the chapter's template

Check yourself

- ✓ Explain why pattern-based practice beats problem-count practice.
- ✓ Recite at least six recognition signals and the pattern each implies.
- ✓ List the ten steps of the per-problem method in order.
- ✓ Say which step buys you the most credit for the least time, and why.
- ✓ Explain why you state the approach and complexity *before* coding.
- ✓ Say what the interview is actually measuring.
- ✓ Give the four practice rules: attempt first, retype from memory, redo, narrate.
- ✓ Describe what belongs in an error log and what it is for.

Complexity analysis and reading the constraints

Big-O is not academic decoration — the constraint line in the problem statement tells you which algorithm is expected, before you have had a single idea. Learn to read it and half the search is already done.

What the notation means

```
BIG-O IS AN UPPER BOUND ON GROWTH, ignoring constants
and lower-order terms.

  3n^2 + 100n + 5000   ->   O(n^2)

WHY THROW AWAY THE CONSTANTS: because for large n the
fastest-growing term dominates everything else, and the
constants depend on hardware you cannot predict.

  n = 1,000,000:
    3n^2   = 3,000,000,000,000
    100n   =       100,000,000    (0.003% of the total)
    5000   =             5,000    (negligible)

THE THREE NOTATIONS, since interviewers occasionally ask:
  O(f)      grows NO FASTER than f   (upper bound)
  Omega(f)  grows NO SLOWER than f   (lower bound)
  Theta(f)  both - the tight bound

In practice everyone says "O" and means Theta. Saying
"O(n^2)" for something that is actually O(n) is
technically true and will still be marked wrong.

THE GROWTH LADDER, memorise the order:

  O(1)         constant      hash lookup, array index
  O(log n)     logarithmic   binary search, balanced tree
  O(n)         linear        one pass
  O(n log n)   linearithmic  good sorting, divide+conquer
  O(n^2)       quadratic     nested loops, all pairs
  O(n^3)       cubic         all triples, Floyd-Warshall
  O(2^n)       exponential   all subsets
  O(n!)        factorial     all permutations

WHAT THOSE ACTUALLY COST at n = 1,000,000:

  O(1)          1 step
  O(log n)      20 steps
  O(n)          1,000,000
  O(n log n)    20,000,000
  O(n^2)        1,000,000,000,000       ~ hours
  O(2^n)        more steps than atoms in the observable
                universe

AND AT n = 20:  2^20 = 1,048,576 - perfectly fine.
EXPONENTIAL IS NOT AUTOMATICALLY DISQUALIFYING. It
depends entirely on n, which is why the constraints
matter so much.
```

Reading the constraints backwards

```
THIS IS THE SINGLE MOST USEFUL TABLE IN INTERVIEW PREP.

A judge runs roughly 10^8 simple operations per second.
So from the largest n, you can read off the intended
complexity:

  n up to          intended complexity     what that means
  ------------------------------------------------------
  10^18            O(log n) or O(1)        maths, bit tricks
  10^8             O(n)                    one pass only
  10^6 - 10^7      O(n) or O(n log n)      sort is fine
  10^5             O(n log n)              sort, heap, BS
  10^4             O(n^2) is borderline    nested loop ok
  500 - 1000       O(n^2) or O(n^2 log n)  DP on pairs
  100 - 200        O(n^3)                  interval DP
  40 - 50          O(2^(n/2))              meet in middle
  20 - 25          O(2^n)                  subset DP,
                                           bitmask
  10 - 12          O(n!)                   permutations

HOW TO USE IT, in the interview:

  "n is up to 10^5, so I need about O(n log n). That
   rules out checking every pair. Sorting is affordable,
   and so is a heap or a binary search. Let me look for
   one of those."

YOU HAVE JUST NARROWED THE ENTIRE SEARCH SPACE OF
ALGORITHMS, out loud, before having an idea. That is
worth saying even if you then need a minute of silence.

THE REVERSE READING IS EQUALLY USEFUL:
  · "n <= 20" almost always means BITMASK or
    BACKTRACKING (chapters 16, 32)
  · "n <= 100 and it asks for a minimum cost" -> DP
  · "answer is monotonic and the range is huge" ->
    BINARY SEARCH ON THE ANSWER (chapter 10)
  · "array values up to 10^9 but n only 10^5" -> the
    VALUES need hashing or coordinate compression, not
    an array indexed by value
```

Counting correctly

```
/* --- SEQUENTIAL: ADD, then keep the largest --- */
    for (i = 0; i < n; i++) {}     // O(n)
    for (j = 0; j < n; j++) {}     // O(n)
    -> O(n) + O(n) = O(2n) = O(n)

/* --- NESTED: MULTIPLY --- */
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {}
    -> O(n*m)        NOT O(n^2) unless n == m. Say "n times
                     m" - interviewers notice.

/* --- THE TRIANGLE, which people get wrong --- */
    for (i = 0; i < n; i++)
        for (j = i; j < n; j++) {}
    -> n + (n-1) + ... + 1 = n(n+1)/2 = O(n^2)
    Half the work of the full square, but the SAME class.

/* --- HALVING IS LOGARITHMIC --- */
    while (n > 1) n = n / 2;       // O(log n)
    Each step throws away half the remaining input. The
    number of halvings of n before reaching 1 is log2(n).

/* --- THE ONE THAT LOOKS QUADRATIC AND IS NOT --- */
    let j = 0;
    for (let i = 0; i < n; i++) {
        while (j < n && cond(i, j)) j++;   // j NEVER RESETS
    }
    -> O(n), not O(n^2).

    THE ARGUMENT: `j` only ever increases and is bounded by
    n, so the inner loop body executes at most n times IN
    TOTAL across all iterations of the outer loop. This is
    AMORTISED ANALYSIS, and it is exactly why the sliding
    window (chapter 05) is linear. Be ready to say this out
    loud - it is a common follow-up question.

/* --- RECURSION: draw the tree --- */
    T(n) = 2*T(n/2) + O(n)      -> O(n log n)   merge sort
      log n levels, O(n) work per level

    T(n) = T(n/2) + O(1)        -> O(log n)     binary search
      one branch survives per level

    T(n) = 2*T(n-1) + O(1)      -> O(2^n)       subsets
      the tree DOUBLES each level

    T(n) = T(n-1) + O(n)        -> O(n^2)       naive quicksort
                                                worst case

/* --- SPACE COMPLEXITY: COUNT THE STACK --- */
    Recursion depth counts as space. A recursive DFS on a
    skewed tree of n nodes is O(n) SPACE even though it
    allocates nothing - which is precisely why "can you do
    it iteratively?" is asked (chapter 23).

    AND: the output does not usually count. If a problem
    asks for all n! permutations, you are not penalised for
    the O(n!) output. Say "O(n) auxiliary space, excluding
    the output" to show you know the distinction.        */
```

THE COMPLEXITY OF THE OPERATIONS YOU USE

```
ARRAY        index O(1) · push/pop end O(1) amortised
             insert/delete middle O(n) · search O(n)
SORTED ARRAY binary search O(log n)
HASH MAP     insert/lookup/delete O(1) average, O(n)
             worst case (collisions)
BALANCED BST insert/lookup/delete O(log n), and it keeps
             the keys ORDERED — the reason to prefer it
             over a hash map
HEAP         push/pop O(log n) · peek O(1) · build O(n)
STACK/QUEUE  push/pop O(1)
LINKED LIST  insert/delete at a known node O(1)
             search O(n) · NO random access
TRIE         insert/search O(L), L = key length
UNION-FIND   near O(1) amortised with both optimisations
STRING       concatenation in a loop is O(n²) in most
             languages — build an array and join once
```

Knowing that `Array.shift()`, `list.pop(0)` and `vector.erase(begin())` are all O(n) is the difference between an O(n) solution and an accidental $O(n^2)$ one. This is a genuinely common silent failure.

Amortised, average and worst case

```
/* --- WORST CASE is what you quote unless asked otherwise.
       It is the guarantee.

/* --- AVERAGE CASE assumes a distribution of inputs.
       Hash maps are O(1) average, O(n) worst (everything
       collides). Quicksort is O(n log n) average, O(n^2)
       worst (already-sorted input with a bad pivot).

/* --- AMORTISED is the average over a SEQUENCE of
       operations, and it is a WORST-CASE guarantee for the
       sequence - a much stronger statement than "average".

    THE CLASSIC: a dynamic array that doubles when full.
      Most pushes are O(1).
      A push that triggers a resize copies everything: O(n).
      But doubling means resizes happen at sizes
      1, 2, 4, 8, … n, and the total copying is
        1 + 2 + 4 + … + n < 2n
      so n pushes cost O(n) TOTAL, i.e. O(1) AMORTISED PER
      PUSH.

    THE KEY INSIGHT: doubling (not adding a constant) is
    what makes it amortised O(1). Growing by a fixed 10
    elements gives O(n) amortised per push and an O(n^2)
    total - a real performance bug people ship.

/* --- WHERE AMORTISED SHOWS UP IN INTERVIEWS ---
     · dynamic array / vector push
     · the sliding window's inner pointer (chapter 05)
     · the monotonic stack: each element is pushed once and
       popped at most once, so the whole scan is O(n) even
       though a single step can pop many items (chapter 20)
     · union-find with path compression (chapter 29)

   IN EACH CASE THE ARGUMENT IS THE SAME: bound the TOTAL
   work across the whole run, not the work of one step.
   "Each element is pushed once and popped at most once,
   so the total is O(n)" is a sentence worth memorising.  */
```

Practice: predict the complexity

These are not algorithm problems — each asks you to *return the complexity* of a snippet as a string. Getting them right quickly is a real interview skill.

How it shows up

Symptom	Cause
Solution times out at n = 10^5	Wrote $O(n^2)$ when the constraints demanded O(n log n)
Accidentally $O(n^2)$ with a linear-looking loop	`shift()`, `pop(0)`, or string concatenation inside a loop
Said $O(n^2)$ for the triangle loop and were told it is right	Correct — half the work, same class
Said $O(n^2)$ for a sliding window	The inner pointer never resets — it is amortised O(n)
Stack overflow on a large input	Recursion depth is O(n) space — convert to iteration

Symptom	Cause
Hash solution slower than expected	Worst-case collisions, or hashing large keys is not O(1)
Quoted O(1) space but used a recursive helper	The call stack counts
Exponential solution accepted	Check n — at $n \le 20$, 2^n is fine

Check yourself

- ✓ Recite the growth ladder from O(1) to O(n!).
- ✓ Give the intended complexity for $n = 10^5$, 10^3, and 20.
- ✓ Explain why the triangle loop is $O(n^2)$ despite doing half the work.
- ✓ Give the amortised argument for why a non-resetting inner pointer is O(n).
- ✓ Explain why array doubling gives O(1) amortised push and fixed growth does not.
- ✓ State the recurrence for merge sort and binary search, and solve each.
- ✓ Say why recursion has a space cost even with no allocation.
- ✓ Name three operations that look O(1) but are O(n).

Problems

Classify: nested and sequential loops `Easy`

Pattern: Counting loops

Return the tightest complexity of this function as a string, one of: `"O(1)"`, `"O(log n)"`, `"O(n)"`, `"O(n log n)"`, `"O(n^2)"`, `"O(n^3)"`.

```
function f(n) {
  let c = 0;
  for (let i = 0; i < n; i++) c++;        // A
  for (let i = 0; i < n; i++)             // B
    for (let j = i; j < n; j++) c++;
  return c;
}
```

Implement `classify()` to return that string.

Input	Output	Note
–	`one of the strings above`	

HINTS

1. Sequential blocks ADD; you keep the largest.
2. Loop B is the triangle: n + (n−1) + … + 1.
3. n(n+1)/2 drops its constant factor.

Approach	Idea	Time	Space
Count each block, take the max	A is O(n). B is the triangle sum n(n+1)/2, which is O(n^2). The total is O(n) + O(n^2) = O(n^2).	–	–

Solution — js

```
function classify() {
  // A: O(n)
  // B: n + (n-1) + ... + 1 = n(n+1)/2 = O(n^2)
  // sequential -> take the larger
  return "O(n^2)";
}
```

WALKTHROUGH

The trap is assuming the triangle loop is cheaper *in class* because it does half the work. Constant factors are exactly what big-O discards, so `n(n+1)/2` and `n²` are the same class. The right thing to say aloud is "it is about half of n^2, so O(n^2)" — that shows you counted rather than pattern-matched.

Classify: halving with inner work `Medium`

Pattern: Logarithmic outer loop

Return the tightest complexity of:

```
function f(n) {
  let c = 0;
  for (let i = n; i > 0; i = Math.floor(i / 2))  // outer
    for (let j = 0; j < n; j++) c++;             // inner
  return c;
}
```

Input	Output	Note
–	`a complexity string`	

HINTS

1. How many times does the outer loop run before i reaches 0?
2. The inner loop does n work every time, independent of i.
3. Multiply: (number of outer iterations) × (work per iteration).

Approach	Idea	Time	Space
Multiply the two	The outer loop halves `i` each time, so it runs about $\log_2$ n times. The inner loop is O(n) and does *not* shrink with `i`. Total O(n log n).	–	–

Solution — js

```
function classify() {
  // outer: n, n/2, n/4, ... -> log2(n) iterations
  // inner: n work each time (does NOT depend on i)
  // -> n * log n
  return "O(n log n)";
}
```

WALKTHROUGH

The common error is answering `O(n)` by assuming the inner loop shrinks along with `i`. It does not — it is bounded by `n`, not `i`. Contrast with the version where the inner loop runs to `i`: then the total is `n + n/2 + n/4 + … = 2n`, which is **O(n)**. That geometric-series argument is worth knowing; it is why building a heap is O(n) rather than O(n log n).

Classify: the non-resetting pointer `Medium`

Pattern: Amortised analysis

Return the tightest complexity of:

```
function f(a) {
  const n = a.length;
  let j = 0, c = 0;
  for (let i = 0; i < n; i++) {
    while (j < n && a[j] < a[i] + 10) { j++; c++; }
  }
  return c;
}
```

Note that `j` is declared *outside* the outer loop and is never reset.

Input	Output	Note
`—`	`a complexity string`	

HINTS

1. It *looks* like a nested loop, so the instinct is $O(n^2)$.
2. Ask a different question: across the whole run, how many times in total can the line `j++` execute?
3. j only increases and is capped at n. That bounds the total inner work regardless of how the outer loop behaves.

Approach	Idea	Time	Space
Naive reading	Two nested loops → $O(n^2)$. Wrong, because the inner loop does not restart.	—	—
Amortised: bound the total	`j` is monotonically increasing and never exceeds `n`, so `j++` runs at most n times *in total* across every outer iteration. Outer loop O(n) + total inner work O(n) = **O(n)**.	—	—

Solution — js

```
function classify() {
  // j never resets and never decreases, and j < n.
  // So the inner body runs at most n times IN TOTAL,
  // not n times PER outer iteration.
  // O(n) outer + O(n) total inner = O(n)
  return "O(n)";
}
```

WALKTHROUGH

This is the single most important complexity argument in array interviews, because it is exactly why the sliding window is linear and why the monotonic stack is linear. The sentence to have ready is: *"each element is visited by the inner pointer at most once over the whole run, so the total inner work is O(n), giving O(n) overall."*

The distinguishing test: is the inner counter reset inside the outer loop? If yes, it multiplies ($O(n^2)$). If it is declared outside and never reset, it amortises ($O(n)$).

Classify: recursion trees `Hard`

Pattern: Solving recurrences

Given a recurrence as a string, return its complexity. Implement `solve(rec)` handling exactly these four:

- `"T(n) = 2T(n/2) + O(n)"`
- `"T(n) = T(n/2) + O(1)"`
- `"T(n) = 2T(n-1) + O(1)"`
- `"T(n) = T(n-1) + O(n)"`

Input	Output	Note
`"T(n) = 2T(n/2) + O(n)"`	`"O(n log n)"`	Merge sort: log n levels, O(n) work per level.
`"T(n) = T(n/2) + O(1)"`	`"O(log n)"`	Binary search: one branch survives per level.

HINTS

1. Draw the tree. How many LEVELS, and how much work per level?
2. Dividing by 2 gives log n levels; subtracting 1 gives n levels.
3. Two recursive calls means the tree doubles in width each level; one call means it stays a single path.

Approach	Idea	Time	Space
Recursion tree	Levels × work per level. `2T(n/2)+O(n)`: log n levels × O(n) = O(n log n). `T(n/2)+O(1)`: log n levels × O(1) = O(log n). `2T(n-1)+O(1)`: n levels, width doubles = $O(2^n)$. `T(n-1)+O(n)`: n levels × O(n) = $O(n^2)$.	–	–

Solution — js

```
function solve(rec) {
  const table = {
    // log n levels, O(n) work each -> merge sort
    "T(n) = 2T(n/2) + O(n)": "O(n log n)",
    // log n levels, O(1) work each -> binary search
    "T(n) = T(n/2) + O(1)":  "O(log n)",
    // n levels, width doubles each level -> subsets
    "T(n) = 2T(n-1) + O(1)": "O(2^n)",
    // n levels, O(n) work each -> naive quicksort worst case
    "T(n) = T(n-1) + O(n)":  "O(n^2)"
  };
  return table[rec];
}
```

WALKTHROUGH

THE GENERAL METHOD.

For `T(n) = a·T(n/b) + f(n)` the tree has `log_b n` levels and `a^k` nodes at level `k`. Compare the work at the leaves, `n^(log_b a)`, against `f(n)`: whichever dominates is the answer, and if they match you gain a `log n` factor. That is the Master Theorem, and for `2T(n/2)+O(n)` the two match exactly — hence `O(n log n)`.

For recurrences that *subtract* rather than divide, the tree has `n` levels instead of `log n`, which is why those land at `O(n²)` or `O(2ⁿ)`. Recognising "divide" versus "subtract" is most of the skill.

The playground: write and run code

A scratch pad that runs in this page. Use it to try an idea, trace an algorithm with `console.log`, or check a language detail without leaving the site.

Scratch pad

Write anything and press **Run** (or Ctrl/⌘+Enter). Output from `console.log` appears below. Your code is saved in this browser, so it survives a reload.

Built-in helpers

Linked lists and trees are pre-defined in the sandbox, on this page and inside every problem, so you never have to write the plumbing:

```
/* --- LINKED LISTS --- */
function ListNode(val, next)      // constructor
buildList([1,2,3])                // array  -> list, returns head
listToArray(head)                 // list   -> array

/* --- BINARY TREES (level-order, null for a missing child) --- */
function TreeNode(val, left, right)
buildTree([3,9,20,null,null,15,7])   // array -> tree, returns root
treeToArray(root)                    // tree  -> array

/* EXAMPLE - reverse a list and print it: */
const head = buildList([1,2,3,4,5]);
let prev = null, cur = head;
while (cur) { const nx = cur.next; cur.next = prev; prev = cur; cur = nx; }
console.log(listToArray(prev));     // [5,4,3,2,1]

/* EXAMPLE - the tree [3,9,20,null,null,15,7] is:
        3
       / \
      9  20
         / \
        15  7                                        */
const root = buildTree([3,9,20,null,null,15,7]);
function depth(n) { return n ? 1 + Math.max(depth(n.left), depth(n.right)) : 0; }
console.log('depth =', depth(root));      // 3
```

How the runner works

```
YOUR CODE ---> a Web Worker (a separate thread)
                    |
                    |  runs the test harness
                    |  captures console.log
                    |  4-second timeout
                    v
               results back to the page

WHY A WORKER:
  · an INFINITE LOOP cannot freeze the page — the worker
    is terminated after 4 seconds and you get a clear
    message instead of a dead tab
  · your code cannot touch the page
  · console.log is captured rather than lost

IF WORKERS ARE UNAVAILABLE — which happens when the site
is opened directly from disk with a file:// URL in some
browsers — the runner FALLS BACK to executing on the main
thread. Everything still works, with one caveat: an
infinite loop will freeze the tab and you will have to
close it. Serving the site over http (even
`python -m http.server`) avoids that entirely.

WHAT IS NOT HERE, and honestly:
  · NO C++, Python or Java execution. Running those needs
    a compiler toolchain or a multi-megabyte WebAssembly
    runtime downloaded from a CDN, which would break the
    offline, no-dependency design of this site.
    Reference solutions are given in all four languages
    on every problem; the JUDGE runs JavaScript.
  · no file I/O, no network, no npm packages.
  · no stdin. Problems pass arguments to a function
    rather than reading input, which is also how
    LeetCode-style interviews work.

IF YOUR TARGET LANGUAGE IS C++ OR JAVA: read the
reference solution in your language, then type the
JavaScript. For these problems the algorithm is the whole
content and the syntax gap is small. Practise the real
syntax separately in your own editor before the
interview.
```

JavaScript for people who write C++ or Java

```
/* Enough to solve every problem in this track. */

/* --- ARRAYS --- */
const a = [1, 2, 3];
a.length                      // size
a.push(4); a.pop();           // back:  O(1)
a.unshift(0); a.shift();      // front: O(n)  <-- AVOID IN LOOPS
a.slice(1, 3)                 // copy of [1,3)   - non-destructive
a.splice(1, 2)                // REMOVE 2 items at index 1 - mutates
new Array(n).fill(0)          // n zeros
Array.from({length: n}, (_, i) => i)          // [0..n-1]
Array.from({length: r}, () => new Array(c).fill(0))   // r x c grid

/* THE 2-D ARRAY TRAP: */
new Array(3).fill([])         // WRONG - three refs to the SAME array
Array.from({length: 3}, () => [])   // right - three distinct arrays

/* --- SORTING: THE #1 JAVASCRIPT BUG --- */
[10, 9, 1].sort()                  // ["1","10","9"] - LEXICOGRAPHIC!
[10, 9, 1].sort((x, y) => x - y)   // [1, 9, 10] - always pass a comparator
arr.sort((p, q) => p[0] - q[0] || q[1] - p[1]);   // by [0] asc, then [1] desc

/* --- MAP AND SET --- */
const m = new Map();
m.set(k, v); m.get(k); m.has(k); m.delete(k); m.size;
m.get(k) ?? 0                 // default when absent
for (const [k, v] of m) {}
const s = new Set([1, 2, 2]); // size 2
s.add(3); s.has(1); s.delete(1);
[...s]                        // Set -> array

/* A PLAIN OBJECT works for string keys, but Map keeps
   INSERTION ORDER, allows ANY key type, and has O(1) size.
   Prefer Map. */

/* --- STRINGS are immutable --- */
s[i]  s.charCodeAt(i)  s.split('')  arr.join('')
s.slice(1, 4)   s.toLowerCase()
// building a string in a loop with += is O(n^2) in the
// worst case - push to an array and join once.

/* --- NUMBERS --- */
Math.floor(7 / 2)   // 3   - / is FLOAT division, unlike C++ int/int
(7 / 2) | 0         // 3   - fast truncation, but 32-bit only
Math.trunc(-3.5)    // -3  - toward zero
Math.floor(-3.5)    // -4  - toward -infinity  (they DIFFER for
                    //       negatives - a real source of bugs)
Number.MAX_SAFE_INTEGER   // 2^53 - 1; beyond this use BigInt
Infinity, -Infinity       // handy initial values for min/max

/* --- DESTRUCTURING AND SWAP --- */
[a[i], a[j]] = [a[j], a[i]];
const [lo, hi] = [0, n - 1];

/* --- THE EQUALITY RULE: ALWAYS USE === --- */
0 == '0'    // true   - do not use
0 === '0'   // false  - use this

/* --- COMMON IDIOMS --- */
const freq = new Map();
```

```
for (const c of s) freq.set(c, (freq.get(c) ?? 0) + 1);

const seen = new Set();
if (!seen.has(x)) { seen.add(x); }

let best = -Infinity;
for (const v of a) best = Math.max(best, v);

// a grid of visited flags
const vis = Array.from({length: R}, () => new Array(C).fill(false));
```

KEYBOARD SHORTCUTS

```
Ctrl / ⌘ + Enter    run
Tab                 insert two spaces (does not leave
                    the editor)
Reset               restore the starter code and clear
                    your saved version
```

How it shows up

Symptom	Cause
Sorting gives `[1, 10, 9]`	Default sort is lexicographic — pass `(a,b)=>a-b`
All rows of a grid change together	`fill([])` shares one array — use `Array.from`
Loop is unexpectedly O(n^2)	`shift()` / `unshift()` or `+=` on strings inside the loop
Negative division off by one	`Math.floor` vs `Math.trunc` differ for negatives
Large integers lose precision	Past 2^{53} — use `BigInt`
"No function named X was defined"	Renamed the function, or wrapped it in a class
Timed out after 4 s	Infinite loop — a pointer that never advances
Tab freezes instead of timing out	Running from `file://` with no Worker support — serve over http
Output empty	Nothing was logged — the runner shows `console.log`, not return values

Check yourself

- ✓ Run something in the pad and see its output.
- ✓ Build a linked list from an array and convert it back.
- ✓ Build the tree `[3,9,20,null,null,15,7]` and compute its depth.
- ✓ Say why the runner uses a Worker and what happens without one.
- ✓ State what the judge does *not* run, and why.
- ✓ Give the correct way to sort numbers in JavaScript.
- ✓ Give the correct way to build a 2-D grid, and the trap.
- ✓ Name two array operations that are O(n) rather than O(1).

Arrays: the two-pointer pattern

Two indices walking the array under an invariant, turning an O(n^2) pair search into a single O(n) pass. The first pattern to learn, and the one that appears most often.

The pattern

```
THREE VARIANTS. Learn to tell them apart.

1. OPPOSITE ENDS - converging
     lo = 0, hi = n-1
     while (lo < hi) {
         evaluate(a[lo], a[hi])
         move ONE pointer inward, based on a rule
     }

     [ 1, 3, 5, 8, 11, 14 ]   target 16
       ^               ^        1+14=15 < 16 -> lo++
          ^            ^        3+14=17 > 16 -> hi--
          ^         ^           3+11=14 < 16 -> lo++
             ^      ^           5+11=16  FOUND

     REQUIRES A SORTED (or otherwise ordered) INPUT. The
     ordering is what makes "move which pointer?"
     decidable: if the sum is too small, only increasing
     the left value can help.

2. SAME DIRECTION - fast and slow (read/write)
     write = 0
     for (read = 0; read < n; read++)
         if (keep(a[read]))
             a[write++] = a[read]

     THE INVARIANT: everything before `write` is the
     answer so far. Used for IN-PLACE FILTERING -
     removing duplicates, moving zeroes, partitioning.

3. TWO SEQUENCES - one pointer in each
     i = 0, j = 0
     while (i < n && j < m)
         compare a[i] with b[j], advance the smaller

     Merging, intersection, subsequence checking.

WHY IT WORKS - THE ARGUMENT YOU MUST BE ABLE TO GIVE:

  Each step ELIMINATES a whole set of candidates.
  In variant 1, when a[lo]+a[hi] < target, no pair
  (lo, k) for any k <= hi can reach the target, because
  a[hi] is the largest remaining partner. So the entire
  row for `lo` is eliminated in one step, and lo can
  advance permanently.

  n candidates, each pointer moves at most n times, never
  backwards -> O(n) total.

THE COST: O(n) time, O(1) EXTRA SPACE. That O(1) is
frequently the reason the interviewer asks for it.
```

When to reach for it

Sorted array + find a pair/triplet

Two Sum II, 3Sum, 4Sum. Sort first if the input is not sorted *and* indices do not matter.

"In place" + "O(1) extra space"

Read/write pointers. Remove duplicates, move zeroes, remove element.

Palindrome or symmetry

Converge from both ends comparing as you go.

Maximise something between two indices

Container with most water, trapping rain water. Move the pointer that *limits* the result.

Partition by a predicate

Dutch national flag, sort colours, segregate even/odd.

Merge or intersect two sorted lists

One pointer per sequence, advance the smaller.

```
/* THE TEMPLATE, converging variant */
function twoPointer(a, target) {
    let lo = 0, hi = a.length - 1;
    while (lo < hi) {
        const sum = a[lo] + a[hi];
        if (sum === target) return [lo, hi];
        if (sum < target) lo++;      // need a BIGGER value
        else              hi--;      // need a SMALLER value
    }
    return [-1, -1];
}

/* THE TEMPLATE, read/write variant */
function filterInPlace(a, keep) {
    let write = 0;
    for (let read = 0; read < a.length; read++)
        if (keep(a[read])) a[write++] = a[read];
    return write;                    // new logical length
}

/* THE FOUR THINGS THAT GO WRONG, every time:
     · `lo < hi` versus `lo <= hi`. Use `<` when the two
       pointers must be DIFFERENT elements (a pair), `<=`
       when a single middle element is still a valid state.
     · forgetting to SKIP DUPLICATES when the problem wants
       unique tuples
     · moving BOTH pointers when only one should move
     · assuming sorted input when it is not              */
```

Problems

Six problems, easy to hard. Write your solution in the editor and press **Run tests** (or Ctrl/⌘+Enter). Give each one an honest attempt before opening the hints — the struggle is the part that teaches.

How it shows up

Symptom	Cause
Infinite loop	A branch where neither pointer moves
Misses the answer when the pair is adjacent	`lo < hi` vs `lo <= hi` chosen wrongly
Duplicate triplets in the output	Duplicates not skipped after a match
Wrong answer on unsorted input	The converging variant needs an ordering
Indices wrong after sorting	Sorting destroys original indices — use a hash map instead
In-place filter leaves stale tail values	Expected — only the first `write` elements are defined
Container-with-water answer too small	Moved the taller pointer; you must move the *shorter* one
Off-by-one on the last element	Loop bound excludes `hi` when it should include it

Check yourself

- ✓ Name the three two-pointer variants and give a problem for each.
- ✓ Give the elimination argument for why the converging variant is O(n).
- ✓ Say why the converging variant requires sorted input and the read/write variant does not.
- ✓ Write the read/write template and state its invariant.
- ✓ Explain when to use `lo < hi` and when `lo <= hi`.
- ✓ Explain why moving the shorter pointer is correct in Container With Most Water.
- ✓ Say why sorting is wrong for the original Two Sum but right for 3Sum.
- ✓ Describe how to skip duplicates without missing valid tuples.

Problems

Two Sum II — sorted input `Easy`

Pattern: Converging pointers

Given a **1-indexed** array `numbers` sorted in non-decreasing order, find two numbers that add up to `target`. Return their 1-based indices as `[i, j]` with `i < j`.

Exactly one solution exists, and you may not use the same element twice. Use **O(1) extra space**.

Input	Output	Note
`numbers = [2,7,11,15], target = 9`	`[1,2]`	2 + 7 = 9, at 1-based positions 1 and 2.
`numbers = [2,3,4], target = 6`	`[1,3]`	
`numbers = [-1,0], target = -1`	`[1,2]`	

HINTS

1. The array is sorted. What does that let you conclude when the current sum is too small?
2. If `a[lo] + a[hi] < target`, then `a[lo]` paired with *any* index at or below `hi` is also too small — so `lo` can never be part of the answer with those. Advance it.
3. Remember the answer is 1-indexed: return `[lo+1, hi+1]`.

Approach	Idea	Time	Space
Brute force — every pair	Two nested loops testing each pair.	`O(n²)`	`O(1)`
Hash map	Store each value as you scan and look up `target − x`. Works, but ignores the sorting and costs O(n) memory.	`O(n)`	`O(n)`
Converging two pointers	Start at both ends. Sum too small → move `lo` right; too large → move `hi` left. Each step eliminates a whole row of candidates.	`O(n)`	`O(1)`

Solution — js

```
function twoSum(numbers, target) {
  let lo = 0, hi = numbers.length - 1;
  while (lo < hi) {
    const sum = numbers[lo] + numbers[hi];
    if (sum === target) return [lo + 1, hi + 1];
    if (sum < target) lo++;   // need a larger value
    else              hi--;   // need a smaller value
  }
  return [-1, -1];
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int lo = 0, hi = numbers.size() - 1;
        while (lo < hi) {
            int sum = numbers[lo] + numbers[hi];
            if (sum == target) return {lo + 1, hi + 1};
            if (sum < target) ++lo;
            else              --hi;
        }
        return {-1, -1};
    }
};
```

Solution — python

```python
class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        lo, hi = 0, len(numbers) - 1
        while lo < hi:
            s = numbers[lo] + numbers[hi]
            if s == target:
                return [lo + 1, hi + 1]
            if s < target:
                lo += 1
            else:
                hi -= 1
        return [-1, -1]
```

Solution — java

```java
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int lo = 0, hi = numbers.length - 1;
        while (lo < hi) {
            int sum = numbers[lo] + numbers[hi];
            if (sum == target) return new int[]{lo + 1, hi + 1};
            if (sum < target) lo++;
            else              hi--;
        }
        return new int[]{-1, -1};
    }
}
```

WALKTHROUGH

WHY IT IS CORRECT.

The invariant is that the answer, if it exists, always lies within `[lo, hi]`. Suppose `a[lo] + a[hi] < target`. Since the array is sorted, `a[hi]` is the largest value still available to pair with `a[lo]`. If even that is too small, `a[lo]` cannot participate in any remaining pair, so discarding it preserves the invariant. The symmetric argument applies to `hi`. Each iteration discards exactly one index and neither pointer moves backwards, so the loop runs at most n times.

NOTE THE CONTRAST WITH PLAIN TWO SUM

(unsorted, return original indices): there you *cannot* sort, because sorting destroys the indices you must report. That version is a hash-map problem — see chapter 13.

Remove duplicates from a sorted array Easy

Pattern: Read / write pointers

Given a sorted array `nums`, remove duplicates **in place** so each element appears once, keeping the relative order. Return `k`, the number of unique elements.

The first `k` positions of `nums` must hold the unique values. Whatever is left beyond `k` does not matter.

Input	Output	Note
`nums = [1,1,2]`	`2, with nums starting [1,2,...]`	
`nums = [0,0,1,1,1,2,2,3,3,4]`	`5, with nums starting [0,1,2,3,4,...]`	

HINTS

1. Keep two indices: one that *reads* every element, and one that marks where the next unique value should be *written*.
2. Because the array is sorted, a value is new exactly when it differs from the last one you wrote.
3. Start `write` at 1 — the first element is always unique.

Approach	Idea	Time	Space
Build a new array	Copy uniques into a fresh array, then copy back. Simple but uses O(n) extra space, which the problem forbids.	`O(n)`	`O(n)`
Read / write pointers	One forward pass. `read` scans; whenever `nums[read] !== nums[write-1]`, write it at `write` and advance.	`O(n)`	`O(1)`

Solution — js

```
function removeDuplicates(nums) {
  if (nums.length === 0) return 0;
  let write = 1;
  for (let read = 1; read < nums.length; read++) {
    if (nums[read] !== nums[write - 1]) {
      nums[write] = nums[read];
      write++;
    }
  }
  return write;
}
```

Solution — cpp

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {
        if (nums.empty()) return 0;
        int write = 1;
        for (int read = 1; read < (int)nums.size(); ++read)
            if (nums[read] != nums[write - 1])
                nums[write++] = nums[read];
        return write;
    }
};
```

Solution — python

```
class Solution:
    def removeDuplicates(self, nums: list[int]) -> int:
        if not nums:
            return 0
        write = 1
        for read in range(1, len(nums)):
            if nums[read] != nums[write - 1]:
                nums[write] = nums[read]
                write += 1
        return write
```

Solution — java

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int write = 1;
        for (int read = 1; read < nums.length; read++)
            if (nums[read] != nums[write - 1])
                nums[write++] = nums[read];
        return write;
    }
}
```

WALKTHROUGH

THE INVARIANT:

`nums[0 .. write-1]` always holds the unique values seen so far, in order. Because the input is sorted, all copies of a value are adjacent, so comparing against `nums[write-1]` — the last value kept — is enough to decide whether the current value is new.

THE GENERALISATION WORTH KNOWING:

for "allow at most *k* copies of each value", compare against `nums[write-k]` instead and start `write` at `k`. That single change solves the "at most twice" variant, which is a common follow-up.

Valid palindrome `Easy`

Pattern: Converging pointers

Given a string `s`, return `true` if it is a palindrome after converting all uppercase letters to lowercase and removing every non-alphanumeric character.

Input	Output	Note
`s = "A man, a plan, a canal: Panama"`	`true`	Cleaned it reads "amanaplanacanalpanama".
`s = "race a car"`	`false`	
`s = " "`	`true`	The cleaned string is empty, which is a palindrome.

HINTS

1. You can clean the string first, but you can also skip junk characters *while* the pointers converge — that avoids the extra O(n) string.
2. Use inner `while` loops to advance `lo` past non-alphanumeric characters and retreat `hi` past them, before comparing.
3. Guard those inner loops with `lo < hi` so they cannot run off the ends.

Approach	Idea	Time	Space
Clean then reverse	Build a filtered lowercase string and compare it with its reverse. Very readable.	`O(n)`	`O(n)`
Two pointers, skipping in place	Converge from both ends, skipping non-alphanumerics as you go. No extra string.	`O(n)`	`O(1)`

Solution — js

```javascript
function isPalindrome(s) {
  const ok = c => /[a-z0-9]/i.test(c);
  let lo = 0, hi = s.length - 1;
  while (lo < hi) {
    while (lo < hi && !ok(s[lo])) lo++;
    while (lo < hi && !ok(s[hi])) hi--;
    if (s[lo].toLowerCase() !== s[hi].toLowerCase()) return false;
    lo++; hi--;
  }
  return true;
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool isPalindrome(string s) {
        int lo = 0, hi = s.size() - 1;
        while (lo < hi) {
            while (lo < hi && !isalnum((unsigned char)s[lo])) ++lo;
            while (lo < hi && !isalnum((unsigned char)s[hi])) --hi;
            if (tolower((unsigned char)s[lo]) != tolower((unsigned char)s[hi]))
                return false;
            ++lo; --hi;
        }
        return true;
    }
};
```

Solution — python

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        lo, hi = 0, len(s) - 1
        while lo < hi:
            while lo < hi and not s[lo].isalnum():
                lo += 1
            while lo < hi and not s[hi].isalnum():
                hi -= 1
            if s[lo].lower() != s[hi].lower():
                return False
            lo += 1
            hi -= 1
        return True
```

Solution — java

```java
class Solution {
    public boolean isPalindrome(String s) {
        int lo = 0, hi = s.length() - 1;
        while (lo < hi) {
            while (lo < hi && !Character.isLetterOrDigit(s.charAt(lo))) lo++;
            while (lo < hi && !Character.isLetterOrDigit(s.charAt(hi))) hi--;
            if (Character.toLowerCase(s.charAt(lo)) != Character.toLowerCase(s.charAt(hi)))
                return false;
            lo++; hi--;
        }
        return true;
    }
}
```

WALKTHROUGH

THE EDGE CASES THAT CATCH PEOPLE:

a string of only punctuation (the pointers cross without ever comparing — correctly returning `true`), and `"0P"`, which catches anyone who compared ASCII codes with a careless case conversion. Note also the `lo < hi` guard *inside* the skip loops: without it, a string of all punctuation walks `lo` past the end and `s[lo]` is undefined.

Container with most water `Medium`

Pattern: Converging pointers, greedy move

You are given an array `height` where `height[i]` is the height of a vertical line at position `i`. Pick two lines that together with the x-axis form a container holding the most water. Return that maximum area.

Area between `i` and `j` is `min(height[i], height[j]) × (j − i)`.

Input	Output	Note
`height = [1,8,6,2,5,4,8,3,7]`	`49`	Lines at index 1 and 8: min(8,7) × (8−1) = 49.
`height = [1,1]`	`1`	

HINTS

1. Start with the widest possible container: the two ends. Any other container is narrower, so it must be *taller* to beat it.
2. You must move one pointer inward. Which one can you move without risking the loss of a better answer?
3. Moving the taller line inward can never help: the area is capped by the *shorter* line, and the width only shrinks. So move the shorter one.

Approach	Idea	Time	Space
Brute force	Try all pairs and take the maximum.	`O(n²)`	`O(1)`
Converging pointers, move the shorter line	Start wide. Record the area, then discard the shorter line. Each step provably cannot discard the optimum.	`O(n)`	`O(1)`

Solution — js

```
function maxArea(height) {
  let lo = 0, hi = height.length - 1, best = 0;
  while (lo < hi) {
    const h = Math.min(height[lo], height[hi]);
    best = Math.max(best, h * (hi - lo));
    // discard the SHORTER line — it can never do better
    if (height[lo] < height[hi]) lo++;
    else                         hi--;
  }
  return best;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int maxArea(vector<int>& height) {
        int lo = 0, hi = height.size() - 1, best = 0;
        while (lo < hi) {
            int h = min(height[lo], height[hi]);
            best = max(best, h * (hi - lo));
            if (height[lo] < height[hi]) ++lo;
            else                         --hi;
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def maxArea(self, height: list[int]) -> int:
        lo, hi, best = 0, len(height) - 1, 0
        while lo < hi:
            h = min(height[lo], height[hi])
            best = max(best, h * (hi - lo))
            if height[lo] < height[hi]:
                lo += 1
            else:
                hi -= 1
        return best
```

Solution — java

```java
class Solution {
    public int maxArea(int[] height) {
        int lo = 0, hi = height.length - 1, best = 0;
        while (lo < hi) {
            int h = Math.min(height[lo], height[hi]);
            best = Math.max(best, h * (hi - lo));
            if (height[lo] < height[hi]) lo++;
            else                         hi--;
        }
        return best;
    }
}
```

WALKTHROUGH

THE EXCHANGE ARGUMENT — BE READY TO GIVE THIS ONE ALOUD.

Suppose `height[lo] < height[hi]`. Consider any container that uses `lo` as its left wall and some `j < hi` as its right wall. Its height is at most `height[lo]` (unchanged), and its width `j - lo` is strictly smaller than `hi - lo`. So its area is strictly less than the area we just recorded. Every remaining container involving `lo` is therefore already beaten, and `lo` can be discarded safely.

This is the key move: the pointer to advance is always the one that *limits* the objective. The same reasoning drives the harder Trapping Rain Water problem below.

3Sum `Medium`

Pattern: Sort + fix one + two pointers

Given an integer array `nums`, return all **unique** triplets `[a, b, c]` such that `a + b + c === 0`.

The solution set must not contain duplicate triplets. Order of the triplets, and of values within a triplet, does not matter for grading here.

Input	Output	Note
`nums = [-1,0,1,2,-1,-4]`	`[[-1,-1,2],[-1,0,1]]`	
`nums = [0,1,1]`	`[]`	
`nums = [0,0,0]`	`[[0,0,0]]`	

HINTS

1. Sort the array first. Indices are not required in the output, so sorting is free to do.
2. Fix the first element with an outer loop, then the problem reduces to Two Sum II on the remainder — find a pair summing to `-nums[i]`.
3. Duplicates are the whole difficulty. Skip a repeated value for the fixed element, and after recording a match skip repeats at *both* inner pointers.
4. Small optimisation: once `nums[i] > 0` you can stop — a sorted array cannot produce a zero sum from three positives.

Approach	Idea	Time	Space
Brute force	Three nested loops, deduplicate with a set of sorted triplets.	`O(n³)`	`O(n)`
Hash set on the inner pair	Fix one element, then use a hash set for the two-sum. Faster, but deduplication becomes awkward.	`O(n²)`	`O(n)`
Sort + two pointers	Sort once, fix the first element, converge two pointers on the rest. Sorting makes deduplication trivial and drops extra space to O(1).	`O(n²)`	`O(1)`

Solution — js

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const out = [];
  for (let i = 0; i < nums.length - 2; i++) {
    if (nums[i] > 0) break;                       // no zero sum possible
    if (i > 0 && nums[i] === nums[i - 1]) continue; // skip duplicate anchor
    let lo = i + 1, hi = nums.length - 1;
    while (lo < hi) {
      const sum = nums[i] + nums[lo] + nums[hi];
      if (sum < 0) lo++;
      else if (sum > 0) hi--;
      else {
        out.push([nums[i], nums[lo], nums[hi]]);
        lo++; hi--;
        while (lo < hi && nums[lo] === nums[lo - 1]) lo++;  // skip dups
        while (lo < hi && nums[hi] === nums[hi + 1]) hi--;
      }
    }
  }
  return out;
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> out;
        int n = nums.size();
        for (int i = 0; i + 2 < n; ++i) {
            if (nums[i] > 0) break;
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int lo = i + 1, hi = n - 1;
            while (lo < hi) {
                long sum = (long)nums[i] + nums[lo] + nums[hi];
                if (sum < 0) ++lo;
                else if (sum > 0) --hi;
                else {
                    out.push_back({nums[i], nums[lo], nums[hi]});
                    ++lo; --hi;
                    while (lo < hi && nums[lo] == nums[lo - 1]) ++lo;
                    while (lo < hi && nums[hi] == nums[hi + 1]) --hi;
                }
            }
        }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        out = []
        n = len(nums)
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            lo, hi = i + 1, n - 1
            while lo < hi:
                s = nums[i] + nums[lo] + nums[hi]
                if s < 0:
                    lo += 1
                elif s > 0:
                    hi -= 1
                else:
                    out.append([nums[i], nums[lo], nums[hi]])
                    lo += 1
                    hi -= 1
                    while lo < hi and nums[lo] == nums[lo - 1]:
                        lo += 1
                    while lo < hi and nums[hi] == nums[hi + 1]:
                        hi -= 1
        return out
```

Solution — java

```java
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> out = new ArrayList<>();
        for (int i = 0; i + 2 < nums.length; i++) {
            if (nums[i] > 0) break;
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int lo = i + 1, hi = nums.length - 1;
            while (lo < hi) {
                int sum = nums[i] + nums[lo] + nums[hi];
                if (sum < 0) lo++;
                else if (sum > 0) hi--;
                else {
                    out.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                    lo++; hi--;
                    while (lo < hi && nums[lo] == nums[lo - 1]) lo++;
                    while (lo < hi && nums[hi] == nums[hi + 1]) hi--;
                }
            }
        }
        return out;
    }
}
```

WALKTHROUGH

WHY SORTING IS ALLOWED HERE BUT NOT IN PLAIN TWO SUM:

the output is the *values*, not the indices. Sorting destroys indices, which matters only when they are the answer.

THE THREE DEDUPLICATION POINTS

, and you need all three: skip a repeated anchor (`nums[i] === nums[i-1]`), and after recording a triplet skip repeats at `lo` and at `hi` . Note the direction of the comparisons in those inner loops — `nums[lo] === nums[lo-1]` looks *backwards* because `lo` has already been incremented.

COMPLEXITY:

the outer loop is O(n) and each inner two-pointer scan is O(n), giving $O(n^2)$, which dominates the O(n log n) sort. This generalises: 4Sum is two nested anchors plus the same scan, $O(n^3)$.

Trapping rain water `Hard`

Pattern: Converging pointers with running maxima

Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water is trapped after raining.

Input	Output	Note
`height = [0,1,0,2,1,0,1,3,2,1,2,1]`	`6`	
`height = [4,2,0,3,2,5]`	`9`	

HINTS

1. Think per column, not per pool. Water above column `i` is `min(maxLeft[i], maxRight[i]) - height[i]`, floored at zero.
2. That gives an easy O(n) time, O(n) space solution with two precomputed arrays. Now remove the arrays.
3. Two pointers: if `leftMax < rightMax`, then for the left column the answer is already decided by `leftMax` — whatever is on the right, the minimum is `leftMax`. So you can settle the left column now and advance.

Approach	Idea	Time	Space
Brute force	For each column scan left and right for the tallest bar.	`O(n²)`	`O(1)`
Prefix maxima	Precompute `maxLeft` and `maxRight` arrays, then one pass summing `min(l,r) - h`. Very clear, and the one to state first.	`O(n)`	`O(n)`
Two pointers	Maintain `leftMax` and `rightMax` as you converge. Whichever side has the smaller maximum is the side whose answer is already determined.	`O(n)`	`O(1)`
Monotonic stack	Fill pools horizontally as bars are popped. Same complexity — see chapter 20.	`O(n)`	`O(n)`

Solution — js

```
function trap(height) {
  let lo = 0, hi = height.length - 1;
  let leftMax = 0, rightMax = 0, total = 0;
  while (lo < hi) {
    if (height[lo] < height[hi]) {
      // left side is the limiting one -> its answer is decided
      leftMax = Math.max(leftMax, height[lo]);
      total += leftMax - height[lo];
      lo++;
    } else {
      rightMax = Math.max(rightMax, height[hi]);
      total += rightMax - height[hi];
      hi--;
    }
  }
  return total;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int trap(vector<int>& height) {
        int lo = 0, hi = (int)height.size() - 1;
        int leftMax = 0, rightMax = 0, total = 0;
        while (lo < hi) {
            if (height[lo] < height[hi]) {
                leftMax = max(leftMax, height[lo]);
                total += leftMax - height[lo];
                ++lo;
            } else {
                rightMax = max(rightMax, height[hi]);
                total += rightMax - height[hi];
                --hi;
            }
        }
        return total;
    }
};
```

Solution — python

```python
class Solution:
    def trap(self, height: list[int]) -> int:
        lo, hi = 0, len(height) - 1
        left_max = right_max = total = 0
        while lo < hi:
            if height[lo] < height[hi]:
                left_max = max(left_max, height[lo])
                total += left_max - height[lo]
                lo += 1
            else:
                right_max = max(right_max, height[hi])
                total += right_max - height[hi]
                hi -= 1
        return total
```

Solution — java

```java
class Solution {
    public int trap(int[] height) {
        int lo = 0, hi = height.length - 1;
        int leftMax = 0, rightMax = 0, total = 0;
        while (lo < hi) {
            if (height[lo] < height[hi]) {
                leftMax = Math.max(leftMax, height[lo]);
                total += leftMax - height[lo];
                lo++;
            } else {
                rightMax = Math.max(rightMax, height[hi]);
                total += rightMax - height[hi];
                hi--;
            }
        }
        return total;
    }
}
```

WALKTHROUGH

THE REFRAMING THAT MAKES THIS TRACTABLE:

stop thinking about pools. Ask instead, for each column independently, how deep the water is above it. That depth is `min(tallest to the left, tallest to the right) − own height`. Summing over columns gives the answer, and suddenly it is a one-pass problem.

WHY THE TWO-POINTER VERSION IS CORRECT.

Suppose `height[lo] < height[hi]`. We know `rightMax ≥ height[hi] > height[lo]`, and we know `leftMax` exactly. Since the true right maximum for column `lo` is at least `height[hi]`, the minimum of the two maxima *must* be `leftMax` — the right side cannot be the binding constraint. So the water above `lo` is settled as `leftMax − height[lo]`, and we can advance without ever knowing the true right maximum.

NOTE THAT `LEFTMAX − HEIGHT[LO]` IS NEVER NEGATIVE

: `leftMax` was just updated to include `height[lo]`, so the subtraction is safe with no explicit `max(0, …)`.

Arrays: the sliding window

A contiguous range with two moving edges and a running summary of what is inside it. The answer to almost every question containing the word "substring" or "subarray".

The pattern

```
THE PROBLEM SHAPE: "find the longest / shortest / count
of CONTIGUOUS subarrays satisfying some condition".

THE BRUTE FORCE regenerates every window from scratch:
  for each start i:            O(n)
    for each end j:            O(n)
      summarise a[i..j]        O(n)   -> O(n^3)

THE INSIGHT: consecutive windows OVERLAP almost
completely. Sliding from [i..j] to [i..j+1] adds ONE
element. Sliding from [i..j] to [i+1..j] removes ONE.
So maintain the summary INCREMENTALLY instead of
recomputing it.

  [ 2, 1, 5, 1, 3, 2 ]   k = 3
    ^-----^                sum = 8
       ^-----^             sum = 8 - 2 + 1 = 7
          ^-----^          sum = 7 - 1 + 3 = 9   <- best
             ^-----^       sum = 9 - 5 + 2 = 6

TWO KINDS OF WINDOW, and telling them apart is the whole
skill:

FIXED SIZE - the window is always exactly k wide.
  add a[right]; if (right >= k) remove a[right-k];
  record the answer once the window is full.

VARIABLE SIZE - grow greedily, shrink when invalid.
  for right in 0..n-1:
      ADD a[right] to the window
      while (WINDOW IS INVALID):
          REMOVE a[left]; left++
      record the answer          <- position matters!

WHY IT IS O(n) AND NOT O(n^2): `left` only ever moves
FORWARD and is bounded by n, so the total number of
shrink steps across the entire run is at most n. Each
element enters the window once and leaves at most once.
That is the amortised argument from chapter 02, and you
should be ready to say it aloud.

  TIME  O(n)      SPACE  O(1), or O(k) for a frequency map
```

When to reach for it

"Contiguous" + longest/shortest/max/min

Maximum sum subarray of size k, longest substring without repeats, minimum window substring.

"Substring" or "subarray"

The word almost always means a window. If it says *subsequence* (non-contiguous), it is usually DP instead.

A constraint that is monotonic

Growing the window can only make it "more invalid", shrinking only "more valid" — that is what makes the greedy shrink correct.

Counting subarrays with a property

"At most K" is windowable; "exactly K" = atMost(K) − atMost(K−1).

```
/* --- TEMPLATE 1: FIXED WINDOW OF SIZE k --- */
function fixedWindow(a, k) {
    let sum = 0, best = -Infinity;
    for (let right = 0; right < a.length; right++) {
        sum += a[right];                        // ADD
        if (right >= k) sum -= a[right - k];    // REMOVE the one
                                                // that fell out
        if (right >= k - 1) best = Math.max(best, sum);
    }                                           // ^ only once full
    return best;
}

/* --- TEMPLATE 2: LONGEST VALID WINDOW --- */
function longestValid(a) {
    let left = 0, best = 0;
    for (let right = 0; right < a.length; right++) {
        add(a[right]);
        while (!valid()) { remove(a[left]); left++; }
        best = Math.max(best, right - left + 1);   // AFTER the fix
    }
    return best;
}

/* --- TEMPLATE 3: SHORTEST VALID WINDOW --- */
function shortestValid(a) {
    let left = 0, best = Infinity;
    for (let right = 0; right < a.length; right++) {
        add(a[right]);
        while (valid()) {                          // note: WHILE VALID
            best = Math.min(best, right - left + 1);  // record FIRST
            remove(a[left]); left++;
        }
    }
    return best === Infinity ? 0 : best;
}

/* THE DIFFERENCE THAT CATCHES EVERYONE:

     LONGEST   shrink while INVALID, record AFTER the loop
     SHORTEST  shrink while VALID,   record INSIDE the loop

   Get these the wrong way round and you will pass the easy
   tests and fail the rest. Write the loop, then ask
   yourself: "at the moment I record the answer, is the
   window guaranteed valid?"                          */
```

Problems

Five problems, easy to hard. Attempt each for twenty minutes before opening the hints.

How it shows up

Symptom	Cause
Answer is always the whole array	Recording the size before shrinking back to validity
Off by one in the window size	Size is `right - left + 1`, not `right - left`
Fixed window includes partial windows	Missing the `right >= k - 1` guard before recording
Infinite loop in the shrink	`left` not incremented, or the condition can never become false
Frequency map keeps zero entries	Delete the key when its count hits 0, or `size` is wrong
Works for positives, wrong with negatives	Sliding window needs monotonicity — with negatives use prefix sums (ch. 06)
"Exactly K" solution is quadratic	Use `atMost(K) - atMost(K-1)`
Shortest-window answer is 0 when none exists	Sentinel `Infinity` not converted back

Check yourself

- ✓ Give the two window kinds and a problem for each.
- ✓ State the amortised argument for O(n).
- ✓ Write the fixed-window template including both guards.
- ✓ State the difference between the longest and shortest templates.
- ✓ Explain why a sliding window fails when the array contains negative numbers.
- ✓ Explain the `atMost(K) - atMost(K-1)` trick.
- ✓ Say when "substring" implies a window and when it implies DP.
- ✓ Write the window size expression correctly.

Problems

Maximum average subarray I `Easy`

Pattern: Fixed window

Given an integer array `nums` and an integer `k`, find the contiguous subarray of length exactly `k` with the maximum average, and return that average.

Answers within 1e-5 of the correct value are accepted.

Input	Output	Note
`nums = [1,12,-5,-6,50,3], k = 4`	`12.75`	(12 − 5 − 6 + 50) / 4 = 12.75
`nums = [5], k = 1`	`5.0`	

HINTS

1. Maximising the average of a fixed-size window is the same as maximising its sum — the divisor is constant.
2. Do not recompute the sum for each window. Moving right by one adds one element and drops one.
3. Only start recording once the window actually holds k elements.

Approach	Idea	Time	Space
Recompute each window	For each start index, sum k elements.	`O(n·k)`	`O(1)`
Fixed sliding window	Keep a running sum; add the entering element and subtract the leaving one.	`O(n)`	`O(1)`

Solution — js

```
function findMaxAverage(nums, k) {
  let sum = 0, best = -Infinity;
  for (let right = 0; right < nums.length; right++) {
    sum += nums[right];                       // element enters
    if (right >= k) sum -= nums[right - k];   // element leaves
    if (right >= k - 1) best = Math.max(best, sum);
  }
  return best / k;
}
```

Solution — cpp

```
class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {
        long sum = 0, best = LONG_MIN;
        for (int right = 0; right < (int)nums.size(); ++right) {
            sum += nums[right];
            if (right >= k) sum -= nums[right - k];
            if (right >= k - 1) best = max(best, sum);
        }
        return (double)best / k;
    }
};
```

Solution — python

```
class Solution:
    def findMaxAverage(self, nums: list[int], k: int) -> float:
        s = 0
        best = float("-inf")
        for right in range(len(nums)):
            s += nums[right]
            if right >= k:
                s -= nums[right - k]
            if right >= k - 1:
                best = max(best, s)
        return best / k
```

Solution — java

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        long sum = 0, best = Long.MIN_VALUE;
        for (int right = 0; right < nums.length; right++) {
            sum += nums[right];
            if (right >= k) sum -= nums[right - k];
            if (right >= k - 1) best = Math.max(best, sum);
        }
        return (double) best / k;
    }
}
```

WALKTHROUGH

THE TWO GUARDS DO DIFFERENT JOBS.

`right >= k` controls *removal*: only once the window has more than k elements does something fall out. `right >= k - 1` controls *recording*: the window first reaches size k when `right` is `k-1`. Getting either boundary wrong is the classic bug here, and the fix is to test with `k === nums.length` — a single full-array window.

Note that dividing only at the very end avoids accumulating floating-point error, and in C++/Java it keeps the running sum in integer arithmetic where overflow is the real concern (hence `long`).

Longest substring without repeating characters `Medium`

Pattern: Variable window, longest

Given a string `s`, return the length of the longest substring that contains no repeated characters.

Input	Output	Note
`s = "abcabcbb"`	`3`	"abc"
`s = "bbbbb"`	`1`	"b"
`s = "pwwkew"`	`3`	"wke" — note "pwke" is a subsequence, not a substring.

HINTS

1. This is the "longest" template: grow the right edge, and shrink from the left while the window is invalid.
2. The window is invalid exactly when the character just added is already inside it. Track membership with a Set or a Map of last-seen indices.
3. With a Set, shrink one character at a time. With a Map of last indices you can jump `left` straight past the duplicate — but be careful never to move `left` backwards.

Approach	Idea	Time	Space
Check every substring	For each pair (i, j) test whether the substring has duplicates.	`O(n³)`	`O(n)`
Window + Set	Grow right; while the new character is already in the set, remove `s[left]` and advance. Record the size after the window is valid again.	`O(n)`	`O(min(n, alphabet))`
Window + last-index Map	Jump `left` to `max(left, lastIndex[c] + 1)` in one step. Same complexity, fewer operations.	`O(n)`	`O(min(n, alphabet))`

Solution — js

```
function lengthOfLongestSubstring(s) {
  const inWindow = new Set();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    // shrink while the window is INVALID
    while (inWindow.has(s[right])) {
      inWindow.delete(s[left]);
      left++;
    }
    inWindow.add(s[right]);
    best = Math.max(best, right - left + 1);  // record AFTER
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        vector<int> last(128, -1);
        int left = 0, best = 0;
        for (int right = 0; right < (int)s.size(); ++right) {
            if (last[s[right]] >= left) left = last[s[right]] + 1;
            last[s[right]] = right;
            best = max(best, right - left + 1);
        }
        return best;
    }
};
```

Solution — python

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        in_window = set()
        left = best = 0
        for right, ch in enumerate(s):
            while ch in in_window:
                in_window.remove(s[left])
                left += 1
            in_window.add(ch)
            best = max(best, right - left + 1)
        return best
```

Solution — java

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] last = new int[128];
        Arrays.fill(last, -1);
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (last[c] >= left) left = last[c] + 1;
            last[c] = right;
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

WALKTHROUGH

TRACE `"ABBA"`, WHICH IS THE TEST THAT SEPARATES CORRECT SOLUTIONS FROM NEARLY-CORRECT ONES.

With the Set version: `a` enters, `b` enters, the second `b` triggers shrinking until the first `b` is gone (left = 2), then `a` arrives — and the first `a` was already removed, so the window is `"ba"`, length 2. Correct.

With a last-index Map, the naive `left = lastIndex[c] + 1` would set `left = 1` when the final `a` arrives, *moving left backwards* from 2 to 1 and wrongly reporting 3. That is why the jump must be `left = max(left, lastIndex[c] + 1)` — visible as the `last[c] >= left` guard in the C++ and Java versions. `"tmmzuxt"` tests the same thing.

Minimum size subarray sum `Medium`

Pattern: Variable window, shortest

Given an array of **positive** integers `nums` and a positive integer `target`, return the minimal length of a contiguous subarray whose sum is `>= target`. If there is none, return `0`.

Input	Output	Note
`target = 7, nums = [2,3,1,2,4,3]`	`2`	[4,3] has sum 7 and length 2.
`target = 4, nums = [1,4,4]`	`1`	
`target = 11, nums = [1,1,1,1,1,1,1,1]`	`0`	

HINTS

1. This is the "shortest" template — the mirror image of the previous problem.
2. Grow the window until it is *valid* (sum ≥ target). Then shrink from the left *while it stays valid*, recording the length each time before you shrink.
3. All values are positive, which is what makes the sum monotonic: growing can only increase it and shrinking can only decrease it.

Approach	Idea	Time	Space
All subarrays	Try every start and extend until the sum reaches the target.	`O(n²)`	`O(1)`

Approach	Idea	Time	Space
Prefix sums + binary search	Prefix sums are increasing for positive input, so binary search for each start.	`O(n log n)`	`O(n)`
Shrinking window	Expand right, then shrink left while still valid, recording the length before each shrink.	`O(n)`	`O(1)`

Solution — js

```
function minSubArrayLen(target, nums) {
  let left = 0, sum = 0, best = Infinity;
  for (let right = 0; right < nums.length; right++) {
    sum += nums[right];
    // shrink WHILE VALID, recording BEFORE each shrink
    while (sum >= target) {
      best = Math.min(best, right - left + 1);
      sum -= nums[left];
      left++;
    }
  }
  return best === Infinity ? 0 : best;
}
```

Solution — cpp

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int left = 0, best = INT_MAX;
        long sum = 0;
        for (int right = 0; right < (int)nums.size(); ++right) {
            sum += nums[right];
            while (sum >= target) {
                best = min(best, right - left + 1);
                sum -= nums[left++];
            }
        }
        return best == INT_MAX ? 0 : best;
    }
};
```

Solution — python

```
class Solution:
    def minSubArrayLen(self, target: int, nums: list[int]) -> int:
        left = 0
        s = 0
        best = float("inf")
        for right, v in enumerate(nums):
            s += v
            while s >= target:
                best = min(best, right - left + 1)
                s -= nums[left]
                left += 1
        return 0 if best == float("inf") else best
```

Solution — java

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int left = 0, best = Integer.MAX_VALUE;
        long sum = 0;
        for (int right = 0; right < nums.length; right++) {
            sum += nums[right];
            while (sum >= target) {
                best = Math.min(best, right - left + 1);
                sum -= nums[left++];
            }
        }
        return best == Integer.MAX_VALUE ? 0 : best;
    }
}
```

WALKTHROUGH

COMPARE THIS LOOP WITH THE PREVIOUS PROBLEM'S AND THE SYMMETRY IS THE LESSON.

There: `while (invalid) shrink;` then record. Here: `while (valid) { record; shrink; }`. The rule that generates both: *record the answer only at a moment when the window is in the state you are measuring.* For "longest valid" that is after restoring validity; for "shortest valid" that is before destroying it.

WHY POSITIVITY MATTERS.

The greedy shrink assumes that removing a left element can only *decrease* the sum. With a negative number present that is false — removing it would increase the sum, so a window you discarded might have become valid again. Problems with negatives need prefix sums and a hash map instead (chapter 06). Interviewers ask "what if the numbers can be negative?" precisely to see whether you know this.

Longest subarray with at most K distinct `Medium`

Pattern: Variable window + frequency map

Given an array `nums` and an integer `k`, return the length of the longest contiguous subarray containing **at most `k` distinct** values.

This is the "Fruit into Baskets" problem with `k = 2`, and the engine behind "longest substring with at most k distinct characters".

Input	Output	Note
`nums = [1,2,1,2,3], k = 2`	`4`	[1,2,1,2]
`nums = [1,2,3,4], k = 1`	`1`	
`nums = [1,1,1,1], k = 3`	`4`	

HINTS

1. Track how many times each value occurs inside the window, in a Map.
2. The number of distinct values is the Map's `size` — but only if you *delete* keys whose count drops to zero.
3. Invalid means `map.size > k`. Shrink until it is not.

Approach	Idea	Time	Space
All subarrays	For every start, extend and count distinct values with a set.	`O(n²)`	`O(n)`
Window + frequency map	Grow right, incrementing counts. While more than k distinct, decrement from the left and delete zero counts. Record after restoring validity.	`O(n)`	`O(k)`

Solution — js

```
function longestAtMostKDistinct(nums, k) {
  const freq = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < nums.length; right++) {
    const v = nums[right];
    freq.set(v, (freq.get(v) ?? 0) + 1);

    while (freq.size > k) {                 // INVALID
      const u = nums[left];
      const c = freq.get(u) - 1;
      if (c === 0) freq.delete(u);          // MUST delete, or
      else freq.set(u, c);                  // size stays wrong
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int longestAtMostKDistinct(vector<int>& nums, int k) {
        unordered_map<int,int> freq;
        int left = 0, best = 0;
        for (int right = 0; right < (int)nums.size(); ++right) {
            ++freq[nums[right]];
            while ((int)freq.size() > k) {
                if (--freq[nums[left]] == 0) freq.erase(nums[left]);
                ++left;
            }
            best = max(best, right - left + 1);
        }
        return best;
    }
};
```

Solution — python

```python
from collections import defaultdict

class Solution:
    def longestAtMostKDistinct(self, nums: list[int], k: int) -> int:
        freq = defaultdict(int)
        left = best = 0
        for right, v in enumerate(nums):
            freq[v] += 1
            while len(freq) > k:
                u = nums[left]
                freq[u] -= 1
                if freq[u] == 0:
                    del freq[u]
                left += 1
            best = max(best, right - left + 1)
        return best
```

Solution — java

```java
class Solution {
    public int longestAtMostKDistinct(int[] nums, int k) {
        Map<Integer,Integer> freq = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < nums.length; right++) {
            freq.merge(nums[right], 1, Integer::sum);
            while (freq.size() > k) {
                int u = nums[left];
                if (freq.merge(u, -1, Integer::sum) == 0) freq.remove(u);
                left++;
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

WALKTHROUGH

THE BUG THIS PROBLEM EXISTS TO TEACH:

leaving zero-count keys in the map. `freq.size` is then a count of values ever seen in the window rather than currently in it, the shrink loop never terminates correctly, and the answer is too small. Deleting on zero is not an optimisation — it is what makes `size` mean "distinct".

THE EXTENSION WORTH KNOWING.

"Exactly K distinct" is not directly windowable, because the property is not monotonic. The standard trick is `exactly(K) = atMost(K) - atMost(K-1)`: every subarray with at most K distinct, minus those with at most K−1, leaves precisely those with exactly K. Two linear passes, still O(n), and it converts a whole family of "exactly" problems into this one.

Minimum window substring `Hard`

Pattern: Shortest window + need/have counting

Given strings `s` and `t`, return the shortest substring of `s` that contains every character of `t` **including duplicates**. If there is none, return the empty string.

Input	Output	Note
`s = "ADOBECODEBANC", t = "ABC"`	`"BANC"`	
`s = "a", t = "a"`	`"a"`	
`s = "a", t = "aa"`	`""`	Two a's are required but only one is available.

HINTS

1. Counts matter, not just presence: `t = "aa"` needs two a's.
2. Keep a `need` map from `t`, and a counter `formed` of how many *distinct* characters currently meet their required count.
3. The window is valid when `formed === need.size`. Then apply the "shortest" template: record, then shrink.
4. Only decrement `formed` when a count drops strictly *below* what is required — going from 3 copies to 2 when only 2 are needed does not break validity.

Approach	Idea	Time	Space
All substrings	Check each of the O(n^2) substrings against the required counts.	`O(n²·m)`	`O(m)`
Window with need/have counts	Expand right until valid, then shrink left while it stays valid, tracking the best. The `formed` counter makes the validity check O(1).	`O(n + m)`	`O(m)`

Solution — js

```javascript
function minWindow(s, t) {
  if (t.length === 0 || s.length < t.length) return "";

  const need = new Map();
  for (const c of t) need.set(c, (need.get(c) ?? 0) + 1);

  const have = new Map();
  let formed = 0;                    // chars meeting their quota
  let left = 0, bestLen = Infinity, bestStart = 0;

  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    if (need.has(c)) {
      const h = (have.get(c) ?? 0) + 1;
      have.set(c, h);
      if (h === need.get(c)) formed++;   // exactly meets it now
    }

    // shrink WHILE VALID, recording BEFORE shrinking
    while (formed === need.size) {
      if (right - left + 1 < bestLen) {
        bestLen = right - left + 1;
        bestStart = left;
      }
      const d = s[left];
      if (need.has(d)) {
        const h = have.get(d) - 1;
        have.set(d, h);
        if (h < need.get(d)) formed--;   // dropped BELOW quota
      }
      left++;
    }
  }
  return bestLen === Infinity ? "" : s.slice(bestStart, bestStart + bestLen);
}
```

Solution — cpp

```cpp
class Solution {
public:
    string minWindow(string s, string t) {
        if (t.empty() || s.size() < t.size()) return "";
        unordered_map<char,int> need, have;
        for (char c : t) ++need[c];
        int formed = 0, left = 0, bestLen = INT_MAX, bestStart = 0;
        for (int right = 0; right < (int)s.size(); ++right) {
            char c = s[right];
            if (need.count(c) && ++have[c] == need[c]) ++formed;
            while (formed == (int)need.size()) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestStart = left;
                }
                char d = s[left];
                if (need.count(d) && --have[d] < need[d]) --formed;
                ++left;
            }
        }
        return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
    }
};
```

Solution — python

```python
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not t or len(s) < len(t):
            return ""
        need = Counter(t)
        have = Counter()
        formed = 0
        left = 0
        best_len = float("inf")
        best_start = 0
        for right, c in enumerate(s):
            if c in need:
                have[c] += 1
                if have[c] == need[c]:
                    formed += 1
            while formed == len(need):
                if right - left + 1 < best_len:
                    best_len = right - left + 1
                    best_start = left
                d = s[left]
                if d in need:
                    have[d] -= 1
                    if have[d] < need[d]:
                        formed -= 1
                left += 1
        return "" if best_len == float("inf") else s[best_start:best_start + best_len]
```

Solution — java

```java
class Solution {
    public String minWindow(String s, String t) {
        if (t.isEmpty() || s.length() < t.length()) return "";
        Map<Character,Integer> need = new HashMap<>(), have = new HashMap<>();
        for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);
        int formed = 0, left = 0, bestLen = Integer.MAX_VALUE, bestStart = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (need.containsKey(c)) {
                int h = have.merge(c, 1, Integer::sum);
                if (h == need.get(c)) formed++;
            }
            while (formed == need.size()) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestStart = left;
                }
                char d = s.charAt(left);
                if (need.containsKey(d)) {
                    int h = have.merge(d, -1, Integer::sum);
                    if (h < need.get(d)) formed--;
                }
                left++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
    }
}
```

WALKTHROUGH

WHY `FORMED` EXISTS.

The naive validity check compares two maps entry by entry, which is O(alphabet) inside a loop that already runs n times. The `formed` counter reduces validity to a single integer comparison, and maintaining it costs O(1) per step — the whole reason this is O(n) rather than O(n·m).

THE TWO CONDITIONS MUST BE ASYMMETRIC, AND THIS IS THE SUBTLE PART.

On the way in, increment `formed` only when the count becomes *exactly* the requirement (`h === need`); using `>=` would count the same character repeatedly. On the way out, decrement only when the count falls *strictly below* (`h < need`); a surplus copy leaving does not invalidate anything. Test `"aa"/"aa"` and `"bba"/"ab"` — those catch both mistakes.

COMPLEXITY.

Each character enters and leaves the window at most once, so despite the nested `while` this is O(n + m) — the same amortised argument as every window in this chapter.

Arrays: prefix sums and difference arrays

Precompute once so every range query is O(1). The technique that rescues you when a sliding window fails because the array contains negative numbers.

The pattern

```
PREFIX SUM: P[i] = sum of the first i elements.

  a =      [ 3,  1,  4,  1,  5 ]
  P = [ 0,   3,  4,  8,  9, 14 ]
       ^
       P[0] = 0, the EMPTY prefix — always include it

THEN THE SUM OF ANY RANGE IS ONE SUBTRACTION:

  sum(a[l..r])  =  P[r+1] - P[l]

  sum(a[1..3]) = P[4] - P[1] = 9 - 3 = 6   (1+4+1) ✓

BUILD O(n) ONCE, then EVERY QUERY IS O(1). Without it,
q queries cost O(n·q).

THE OFF-BY-ONE IS THE WHOLE DIFFICULTY. Using a
1-INDEXED prefix array of length n+1 makes it disappear:
P[i] means "sum of the first i elements", so P[0] = 0
needs no special case and l = 0 works naturally.

------------------------------------------------------------

DIFFERENCE ARRAY — the mirror image. Use it for RANGE
UPDATES with a single query at the end.

  to add v to a[l..r]:
      D[l]   += v
      D[r+1] -= v          <-- one operation, not r-l+1

  after all updates, PREFIX-SUM D to recover the array.

  add 5 to [1..3] of a zero array of length 5:
     D = [0, 5, 0, 0, -5]
     prefix sums -> [0, 5, 5, 5, 0]  ✓

m updates in O(m), one reconstruction in O(n), instead of
O(m·n).

------------------------------------------------------------

WHICH TO USE:
  many RANGE QUERIES, no updates      -> PREFIX SUM
  many RANGE UPDATES, one final read  -> DIFFERENCE ARRAY
  both, interleaved                   -> a Fenwick tree
                                         or segment tree

AND THE COMBINATION THAT SOLVES THE HARD PROBLEMS:
PREFIX SUM + HASH MAP. When you need a range with a
particular property and the values can be negative, store
each prefix sum you have seen and look up the one you
need (chapter 13's Subarray Sum Equals K).
```

When to reach for it

Repeated range-sum queries

"Sum between i and j" asked many times. Build once, answer in O(1).

Subarray sum with negatives

A sliding window needs monotonicity. Negatives break it — use prefix sums plus a hash map.

"Divisible by k", "equal counts"

Map the property onto a prefix value: remainder, or +1/−1 balance.

Many range increments

Difference array: two writes per update, one reconstruction.

2-D region sums

Inclusion–exclusion on a 2-D prefix table.

Running product / XOR

Any associative, invertible operation works the same way.

```
/* --- BUILD --- */
const P = new Array(a.length + 1).fill(0);
for (let i = 0; i < a.length; i++) P[i + 1] = P[i] + a[i];
// range [l, r] inclusive:  P[r + 1] - P[l]

/* --- 2-D PREFIX SUM, inclusion-exclusion --- */
//  P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
//                                                  ^ added twice
//  sum of rectangle (r1,c1)..(r2,c2):
//    P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]

/* --- THE PREFIX + HASH MAP SHAPE --- */
const seen = new Map([[0, -1]]);   // prefix value -> earliest index
let running = 0;
for (let i = 0; i < a.length; i++) {
    running += a[i];
    const key = f(running);         // the property you care about
    if (seen.has(key)) best = Math.max(best, i - seen.get(key));
    else seen.set(key, i);          // KEEP THE EARLIEST index only
}
/* STORING ONLY THE FIRST OCCURRENCE gives the LONGEST range.
   Storing counts instead gives the NUMBER of ranges
   (chapter 13). Which one you need decides the map's value
   type - get this backwards and the answer is subtly wrong. */
```

Problems

How it shows up

Symptom	Cause
Off by one on every range	Mixing 0-indexed and 1-indexed prefix conventions
Ranges starting at 0 are wrong	Missing the `P[0] = 0` entry
Sliding window gives the wrong answer	Array has negatives — switch to prefix sums
Longest range too short	Overwrote the map entry — keep the *earliest* index
Count of ranges too small	Stored an index where a count was needed
Negative remainder in a modulo problem	`((x % k) + k) % k` — JS `%` keeps the sign
2-D sums double-count	Forgot to add back the doubly-subtracted corner
Difference array off by one at the end	`D` needs length `n+1` for `r+1`

Check yourself

- ✓ Write the prefix build and the range-sum formula.
- ✓ Say why `P[0] = 0` removes a special case.
- ✓ Explain when prefix sums beat a sliding window.
- ✓ Give the difference-array update and reconstruction.
- ✓ Write the 2-D range formula and explain the added-back corner.
- ✓ Say when to store an index versus a count in the map.
- ✓ Give the fix for negative remainders.
- ✓ Say what structure you need when queries and updates interleave.

Problems

Range sum query — immutable `Easy`

Pattern: Prefix sum build

Given an array `nums` and a list of `[left, right]` query pairs (inclusive), return an array of the sums of each range.

There may be very many queries, so each one must be answered in O(1) after preprocessing.

Input	Output	Note
`nums = [-2,0,3,-5,2,-1], queries = [[0,2],[2,5],[0,5]]`	`[1,-1,-3]`	

HINTS

1. Build `P` of length `n+1` where `P[i]` is the sum of the first `i` elements.
2. `P[0]` must be 0 so that ranges starting at index 0 need no special case.
3. The inclusive range `[l, r]` is `P[r+1] - P[l]`.

Approach	Idea	Time	Space
Sum per query	Loop over each range. Fine for one query, quadratic for many.	`O(n·q)`	`O(1)`
Prefix sums	Build once in O(n), then each query is one subtraction.	`O(n + q)`	`O(n)`

Solution — js

```
function rangeSums(nums, queries) {
  const n = nums.length;
  const P = new Array(n + 1).fill(0);
  for (let i = 0; i < n; i++) P[i + 1] = P[i] + nums[i];
  return queries.map(([l, r]) => P[r + 1] - P[l]);
}
```

Solution — cpp

```
class NumArray {
    vector<long> P;
public:
    NumArray(vector<int>& nums) : P(nums.size() + 1, 0) {
        for (size_t i = 0; i < nums.size(); ++i)
            P[i + 1] = P[i] + nums[i];
    }
    int sumRange(int l, int r) { return P[r + 1] - P[l]; }
};
```

Solution — python

```
from itertools import accumulate

class NumArray:
    def __init__(self, nums: list[int]):
        self.P = [0] + list(accumulate(nums))

    def sumRange(self, l: int, r: int) -> int:
        return self.P[r + 1] - self.P[l]
```

Solution — java

```
class NumArray {
    private final int[] P;
    public NumArray(int[] nums) {
        P = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) P[i + 1] = P[i] + nums[i];
    }
    public int sumRange(int l, int r) { return P[r + 1] - P[l]; }
}
```

WALKTHROUGH

THE CONVENTION IS THE WHOLE TRICK.

Defining `P[i]` as "the sum of the first `i` elements" — rather than "the sum up to index `i`" — makes `P[0] = 0` meaningful and the formula uniform. With the other convention you need `l === 0 ? P[r] : P[r] - P[l-1]`, and that branch is where the bug lives.

NOTE THE SIZES:

`P` has `n+1` entries, and `P[r+1]` with `r = n-1` reads the last one. Sizing `P` as `n` gives an out-of-bounds read on the full-array query.

Find the pivot index `Easy`

Pattern: Running prefix vs suffix

Given an array `nums`, return the leftmost index where the sum of everything strictly to the left equals the sum of everything strictly to the right. Return `-1` if there is none.

The sum of an empty side is 0.

Input	Output	Note
`nums = [1,7,3,6,5,6]`	`3`	left = 1+7+3 = 11, right = 5+6 = 11.
`nums = [1,2,3]`	`-1`	
`nums = [2,1,-1]`	`0`	left is empty (0), right = 1 + (−1) = 0.

HINTS

1. You do not need a full prefix array — one running total and the grand total are enough.
2. At index `i`, if `left` is the sum before `i`, then the right side is `total - left - nums[i]`.
3. Add `nums[i]` to `left` only *after* testing index `i`.

Approach	Idea	Time	Space
Recompute both sides	For each index sum both halves.	`O(n²)`	`O(1)`
Total minus running left	One pass with the grand total and a running left sum.	`O(n)`	`O(1)`

Solution — js

```
function pivotIndex(nums) {
  const total = nums.reduce((a, b) => a + b, 0);
  let left = 0;
  for (let i = 0; i < nums.length; i++) {
    // right side = everything except left and nums[i]
    if (left === total - left - nums[i]) return i;
    left += nums[i];          // AFTER the test
  }
  return -1;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        long total = accumulate(nums.begin(), nums.end(), 0L), left = 0;
        for (int i = 0; i < (int)nums.size(); ++i) {
            if (left == total - left - nums[i]) return i;
            left += nums[i];
        }
        return -1;
    }
};
```

Solution — python

```python
class Solution:
    def pivotIndex(self, nums: list[int]) -> int:
        total = sum(nums)
        left = 0
        for i, x in enumerate(nums):
            if left == total - left - x:
                return i
            left += x
        return -1
```

Solution — java

```java
class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int x : nums) total += x;
        int left = 0;
        for (int i = 0; i < nums.length; i++) {
            if (left == total - left - nums[i]) return i;
            left += nums[i];
        }
        return -1;
    }
}
```

WALKTHROUGH

TESTING BEFORE ACCUMULATING

is what makes "the sum strictly to the left" correct at index 0, where the left side is empty and must be 0. Moving the `left += nums[i]` above the test breaks `[2,1,-1]`, which expects index 0.

NEGATIVE NUMBERS MATTER HERE

— `[-1,-1,-1,-1,-1,0]` and `[1,-1,0]` both have pivots that a "sums only grow" assumption would miss. This is a small reminder of why the sliding window is not applicable to signed data.

Product of array except self `Medium`

Pattern: Prefix and suffix passes

Given an array `nums`, return an array `out` where `out[i]` is the product of every element *except* `nums[i]`.

Solve it **without division** and in O(n). The output array does not count toward extra space.

Input	Output	Note
`nums = [1,2,3,4]`	`[24,12,8,6]`	
`nums = [-1,1,0,-3,3]`	`[0,0,9,0,0]`	

HINTS

1. The answer at `i` is (product of everything to the left) × (product of everything to the right).
2. That is a prefix product and a suffix product — the same idea as prefix sums with × instead of +.
3. Division would be simpler, but it breaks on zeros. Two passes over the output array need no extra space beyond the answer.

Approach	Idea	Time	Space
Divide by each element	Compute the total product and divide. **Fails on zeros**, and the problem forbids it.	`O(n)`	`O(1)`
Two arrays	Build prefix and suffix product arrays, then multiply pairwise.	`O(n)`	`O(n)`
Two passes into the output	First pass writes prefix products into `out`; second pass multiplies by a running suffix product carried in a single variable.	`O(n)`	`O(1)` `extra`

Solution — js

```
function productExceptSelf(nums) {
  const n = nums.length;
  const out = new Array(n).fill(1);

  // pass 1: out[i] = product of everything LEFT of i
  let prefix = 1;
  for (let i = 0; i < n; i++) {
    out[i] = prefix;
    prefix *= nums[i];
  }

  // pass 2: multiply in the product of everything RIGHT of i
  let suffix = 1;
  for (let i = n - 1; i >= 0; i--) {
    out[i] *= suffix;
    suffix *= nums[i];
  }
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> out(n, 1);
        long prefix = 1;
        for (int i = 0; i < n; ++i) { out[i] = prefix; prefix *= nums[i]; }
        long suffix = 1;
        for (int i = n - 1; i >= 0; --i) { out[i] *= suffix; suffix *= nums[i]; }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        out = [1] * n
        prefix = 1
        for i in range(n):
            out[i] = prefix
            prefix *= nums[i]
        suffix = 1
        for i in range(n - 1, -1, -1):
            out[i] *= suffix
            suffix *= nums[i]
        return out
```

Solution — java

```java
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] out = new int[n];
        int prefix = 1;
        for (int i = 0; i < n; i++) { out[i] = prefix; prefix *= nums[i]; }
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) { out[i] *= suffix; suffix *= nums[i]; }
        return out;
    }
}
```

WALKTHROUGH

TRACE `[1,2,3,4]`.

Pass 1 writes the running prefix *before* multiplying it in, giving `[1, 1, 2, 6]` — each entry is the product of everything to its left. Pass 2 walks backwards doing the same with a suffix variable: index 3 gets ×1, index 2 gets ×4, index 1 gets ×12, index 0 gets ×24, producing `[24,12,8,6]`.

WRITING BEFORE UPDATING

is the same discipline as the pivot problem — it is what excludes `nums[i]` itself from its own answer.

WHY DIVISION IS DISALLOWED:

with one zero every other answer needs the product of the non-zero elements, and with two zeros everything is 0. Handling that takes a zero-count special case; the two-pass version has no special cases at all, which is why it is the expected answer.

Contiguous array of equal 0s and 1s `Medium`

Pattern: Prefix balance + earliest-index map

Given a binary array `nums`, return the length of the longest contiguous subarray containing an **equal number** of 0s and 1s.

Input	Output	Note
`nums = [0,1]`	`2`	

Input	Output	Note
`nums = [0,1,0]`	`2`	Either [0,1] or [1,0].
`nums = [0,1,1,1,0,0]`	`6`	

HINTS

1. Turn "equal counts" into a single number: treat 0 as −1 and 1 as +1, and keep a running balance.
2. A subarray has equal counts exactly when its balance is 0 — that is, when the running balance is the *same* at both ends.
3. So store each balance value with the earliest index where it occurred, and when you see it again the span between is a valid subarray.
4. Seed the map with balance 0 at index −1, so a valid subarray starting at index 0 is measured correctly.

Approach	Idea	Time	Space
All subarrays	Count 0s and 1s in every subarray.	`O(n²)`	`O(1)`
Balance + earliest index map	+1/−1 running balance; first occurrence of each balance recorded. Equal balances bracket a balanced subarray.	`O(n)`	`O(n)`

Solution — js

```
function findMaxLength(nums) {
  // balance -> EARLIEST index at which it occurred
  const first = new Map([[0, -1]]);
  let balance = 0, best = 0;

  for (let i = 0; i < nums.length; i++) {
    balance += nums[i] === 1 ? 1 : -1;

    if (first.has(balance)) {
      // same balance seen before -> the span between is balanced
      best = Math.max(best, i - first.get(balance));
    } else {
      first.set(balance, i);       // keep only the FIRST
    }
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int findMaxLength(vector<int>& nums) {
        unordered_map<int,int> first{{0, -1}};
        int balance = 0, best = 0;
        for (int i = 0; i < (int)nums.size(); ++i) {
            balance += nums[i] == 1 ? 1 : -1;
            auto it = first.find(balance);
            if (it != first.end()) best = max(best, i - it->second);
            else first[balance] = i;
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def findMaxLength(self, nums: list[int]) -> int:
        first = {0: -1}
        balance = best = 0
        for i, x in enumerate(nums):
            balance += 1 if x == 1 else -1
            if balance in first:
                best = max(best, i - first[balance])
            else:
                first[balance] = i
        return best
```

Solution — java

```java
class Solution {
    public int findMaxLength(int[] nums) {
        Map<Integer,Integer> first = new HashMap<>();
        first.put(0, -1);
        int balance = 0, best = 0;
        for (int i = 0; i < nums.length; i++) {
            balance += nums[i] == 1 ? 1 : -1;
            if (first.containsKey(balance)) best = Math.max(best, i - first.get(balance));
            else first.put(balance, i);
        }
        return best;
    }
}
```

WALKTHROUGH

THE REFRAMING IS THE ENTIRE PROBLEM.

"Equal numbers of 0s and 1s" is awkward to track as two counters, but as a single ±1 balance it becomes "the prefix balance is unchanged across this range" — and that is a prefix-sum question with a hash map, exactly like Subarray Sum Equals K.

EARLIEST INDEX, NOT LATEST.

Because you want the *longest* span, the map must keep the first time each balance appeared — hence the `else`. Overwriting on every occurrence yields the shortest span instead, and `[0,1,1,1,0,0]` would return 2 rather than 6.

THE `{0: -1}` SEED

handles a subarray that starts at index 0: its balance returns to 0, and the length is `i - (-1) = i + 1`. Without it, `[0,1]` returns 0.

Arrays: Kadane and maximum subarray

One decision, made at every element: extend the run, or start a new one. It is the smallest dynamic program there is, and recognising its shape unlocks a whole family of problems.

The pattern

```
THE PROBLEM: the contiguous subarray with the largest
sum.

THE INSIGHT — at each element there are only TWO choices:

  EXTEND the best run ending at the previous element, or
  START a fresh run at this element.

     cur = max(a[i],  cur + a[i])
               ^          ^
           start new    extend
     best = max(best, cur)

AND THE EQUIVALENT FORM THAT EXPLAINS IT BEST:

     if (cur < 0) cur = 0;    // a negative prefix can
     cur += a[i];             // only HURT what follows
     best = max(best, cur);

THE ARGUMENT: if the best run ending at i-1 has a
NEGATIVE sum, attaching it to a[i] makes things worse
than starting fresh. So drop it. That is the whole
algorithm.

  a = [ -2,  1, -3,  4, -1,  2,  1, -5,  4 ]
cur =   -2   1  -2   4   3   5   6   1   5
best =  -2   1   1   4   4   5   6   6   6
                               ^ answer

THE TRAP: INITIALISE best TO -Infinity (or a[0]), NOT 0.
With an all-negative array the answer is the largest
single element, and starting at 0 wrongly returns 0 for
an empty subarray the problem does not allow.

THE SHAPE GENERALISES. Kadane is a one-state DP:
  "the best answer ENDING HERE, given the best answer
   ending one step back."
Once you see that, the variants follow:

  MAXIMUM PRODUCT   track min AND max (negatives flip)
  CIRCULAR          answer = max(normal, total - minSubarray)
  MAX SUM WITH ONE
    DELETION        two states: no deletion used, one used
  BEST TIME TO
    BUY/SELL STOCK  Kadane on the difference array

COST: O(n) time, O(1) space, one pass.
```

When to reach for it

"Maximum/minimum contiguous sum"

The plain case. Also works for products and XOR with adjustments.

Best profit from one buy and one sell

Kadane over consecutive differences, or equivalently track the running minimum.

Values can be negative and it is contiguous

A sliding window fails; Kadane does not.

"Ending here" feels natural

Whenever the answer at `i` depends only on the answer at `i-1`, it is a one-state DP.

```
/* --- THE TEMPLATE --- */
function maxSubArray(a) {
    let cur = a[0], best = a[0];     // NOT 0
    for (let i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
}

/* --- RECOVERING THE ACTUAL SUBARRAY, a common follow-up --- */
let cur = a[0], best = a[0];
let s = 0, bestL = 0, bestR = 0;
for (let i = 1; i < a.length; i++) {
    if (cur + a[i] < a[i]) { cur = a[i]; s = i; }  // restarted
    else                   cur = cur + a[i];
    if (cur > best) { best = cur; bestL = s; bestR = i; }
}
// the subarray is a[bestL..bestR]

/* --- MAXIMUM PRODUCT: why one variable is not enough --- */
    A large NEGATIVE product becomes the largest POSITIVE
    product the moment it meets another negative. So you
    must carry BOTH extremes:
        newMax = max(x, max*x, min*x)
        newMin = min(x, max*x, min*x)
    and compute both from the OLD values — overwrite max
    first and newMin uses the wrong one.               */
```

Problems

How it shows up

Symptom	Cause
All-negative array returns 0	`best` initialised to 0 instead of `a[0]`
Maximum product wrong after a negative	Only tracking the maximum — carry the minimum too

Symptom	Cause
Product answer wrong by one step	Overwrote `max` before computing `min`
Circular answer wrong when all negative	`total - minSubarray` is 0 (empty) — special-case it
Stock profit negative	Profit must be floored at 0 when no profitable trade exists
Returned the subarray but the bounds are off	Start index not reset at the moment the run restarts
Crash on an empty array	`a[0]` read without a length check

Check yourself

- ✓ State the two choices Kadane makes at each element.
- ✓ Give the argument for discarding a negative running sum.
- ✓ Say why `best` must not start at 0.
- ✓ Write the version that recovers the subarray bounds.
- ✓ Explain why maximum product needs two tracked values.
- ✓ Give the circular-array formula and its edge case.
- ✓ Explain how the stock problem reduces to Kadane.
- ✓ Say what makes Kadane a one-state dynamic program.

Problems

Maximum subarray `Easy`

Pattern: Kadane

Given an integer array `nums`, find the contiguous subarray with the largest sum and return that sum. The subarray must contain at least one element.

Input	Output	Note
`nums = [-2,1,-3,4,-1,2,1,-5,4]`	`6`	[4,-1,2,1]
`nums = [1]`	`1`	
`nums = [-1]`	`-1`	All negative — the answer is the largest single element.

HINTS

1. At each element, either extend the run ending at the previous element or start a new run here.
2. If the running sum has gone negative, it can only reduce whatever comes next — drop it.
3. Initialise both the running sum and the best to `nums[0]`, not to 0, or an all-negative array returns 0.

Approach	Idea	Time	Space
All subarrays	Two nested loops with a running sum.	`O(n²)`	`O(1)`
Divide and conquer	Best in the left half, the right half, or crossing the middle. Worth naming as it shows the recursion.	`O(n log n)`	`O(log n)`

Approach	Idea	Time	Space
Kadane	One pass, one decision per element.	`O(n)`	`O(1)`

Solution — js

```
function maxSubArray(nums) {
  let cur = nums[0], best = nums[0];
  for (let i = 1; i < nums.length; i++) {
    // extend the previous run, or start fresh at nums[i]
    cur = Math.max(nums[i], cur + nums[i]);
    best = Math.max(best, cur);
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int cur = nums[0], best = nums[0];
        for (size_t i = 1; i < nums.size(); ++i) {
            cur = max(nums[i], cur + nums[i]);
            best = max(best, cur);
        }
        return best;
    }
};
```

Solution — python

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        cur = best = nums[0]
        for x in nums[1:]:
            cur = max(x, cur + x)
            best = max(best, cur)
        return best
```

Solution — java

```
class Solution {
    public int maxSubArray(int[] nums) {
        int cur = nums[0], best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

WALKTHROUGH

WHY `MAX(NUMS[I], CUR + NUMS[I])` IS EXACTLY RIGHT.

`cur` holds the best sum of any subarray ending at `i-1`. A subarray ending at `i` either includes that run or does not; there is no third option. Taking the larger of the two therefore gives the best subarray ending at `i`, and the global answer is the largest of those over all `i`.

THE ALL-NEGATIVE TESTS ARE THE ONES THAT MATTER.

`[-2,-1,-3]` must return `-1`. Initialising `best = 0` returns 0, which corresponds to the empty subarray the problem explicitly disallows. Interviewers reach for this case immediately.

Best time to buy and sell stock Easy

Pattern: Running minimum (Kadane in disguise)

You are given an array `prices` where `prices[i]` is the price of a stock on day `i`. Choose one day to buy and a **later** day to sell. Return the maximum profit, or 0 if no profitable trade exists.

Input	Output	Note
`prices = [7,1,5,3,6,4]`	`5`	Buy at 1, sell at 6.
`prices = [7,6,4,3,1]`	`0`	Prices only fall — do not trade.

HINTS

1. For each day, the best profit selling *today* is today's price minus the cheapest price seen so far.
2. So one pass tracking the running minimum is enough.
3. Alternatively: run Kadane over the array of day-to-day differences — the maximum subarray of differences is exactly the best trade.

Approach	Idea	Time	Space
All pairs	Try every buy day against every later sell day.	`O(n²)`	`O(1)`
Running minimum	Track the cheapest price so far; the answer is the largest gap.	`O(n)`	`O(1)`
Kadane on differences	Maximum subarray of `prices[i] - prices[i-1]`, floored at 0. Same complexity — useful for showing the connection.	`O(n)`	`O(1)`

Solution — js

```
function maxProfit(prices) {
  let minSoFar = Infinity, best = 0;
  for (const p of prices) {
    // best trade SELLING today = today - cheapest day so far
    best = Math.max(best, p - minSoFar);
    minSoFar = Math.min(minSoFar, p);
  }
  return best;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minSoFar = INT_MAX, best = 0;
        for (int p : prices) {
            if (minSoFar != INT_MAX) best = max(best, p - minSoFar);
            minSoFar = min(minSoFar, p);
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        min_so_far = float("inf")
        best = 0
        for p in prices:
            best = max(best, p - min_so_far)
            min_so_far = min(min_so_far, p)
        return best
```

Solution — java

```java
class Solution {
    public int maxProfit(int[] prices) {
        int minSoFar = Integer.MAX_VALUE, best = 0;
        for (int p : prices) {
            if (minSoFar != Integer.MAX_VALUE) best = Math.max(best, p - minSoFar);
            minSoFar = Math.min(minSoFar, p);
        }
        return best;
    }
}
```

WALKTHROUGH

UPDATING `BEST` BEFORE `MINSOFAR`

enforces "sell strictly after buy". On the first day `minSoFar` is still `Infinity`, so the profit is `-Infinity` and `best` stays 0 — correct, since you cannot sell on the day you first see a price. In C++ and Java the sentinel would overflow on subtraction, hence the explicit guard.

THE KADANE CONNECTION.

Buying at `i` and selling at `j` earns `prices[j] - prices[i]`, which telescopes to the sum of the daily differences from `i+1` to `j`. So the best trade is the maximum subarray of the difference array — and floored at 0 because declining to trade is allowed. Saying this aloud demonstrates you see the structure rather than a memorised trick.

Maximum product subarray `Medium`

Pattern: Kadane with two states

Given an integer array `nums`, find the contiguous subarray with the largest **product** and return that product. The subarray must be non-empty.

Input	Output	Note
`nums = [2,3,-2,4]`	`6`	[2,3]
`nums = [-2,0,-1]`	`0`	
`nums = [-2,3,-4]`	`24`	The whole array — two negatives make a positive.

HINTS

1. One tracked value is not enough: multiplying by a negative turns the largest product into the smallest and vice versa.
2. Carry both the maximum and the minimum product ending here.
3. Compute both new values from the *old* pair — if you overwrite the maximum first, the minimum is computed from the wrong number.
4. A zero resets both, which falls out naturally from including `x` itself as a candidate.

Approach	Idea	Time	Space
All subarrays	Multiply out every subarray.	`O(n²)`	`O(1)`
Kadane with max and min	Two running states, swapped in effect by every negative element.	`O(n)`	`O(1)`

Solution — js

```
function maxProduct(nums) {
  let maxHere = nums[0], minHere = nums[0], best = nums[0];

  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    // compute BOTH from the OLD pair before overwriting
    const candMax = Math.max(x, maxHere * x, minHere * x);
    const candMin = Math.min(x, maxHere * x, minHere * x);
    maxHere = candMax;
    minHere = candMin;
    best = Math.max(best, maxHere);
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int maxHere = nums[0], minHere = nums[0], best = nums[0];
        for (size_t i = 1; i < nums.size(); ++i) {
            int x = nums[i];
            int candMax = max({x, maxHere * x, minHere * x});
            int candMin = min({x, maxHere * x, minHere * x});
            maxHere = candMax; minHere = candMin;
            best = max(best, maxHere);
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        max_here = min_here = best = nums[0]
        for x in nums[1:]:
            cand_max = max(x, max_here * x, min_here * x)
            cand_min = min(x, max_here * x, min_here * x)
            max_here, min_here = cand_max, cand_min
            best = max(best, max_here)
        return best
```

Solution — java

```java
class Solution {
    public int maxProduct(int[] nums) {
        int maxHere = nums[0], minHere = nums[0], best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            int x = nums[i];
            int candMax = Math.max(x, Math.max(maxHere * x, minHere * x));
            int candMin = Math.min(x, Math.min(maxHere * x, minHere * x));
            maxHere = candMax; minHere = candMin;
            best = Math.max(best, maxHere);
        }
        return best;
    }
}
```

WALKTHROUGH

WHY THE MINIMUM IS WORTH TRACKING.

On `[-2,3,-4]`: after `-2` both states are `-2`. At `3`, max is 3 and min is `-6`. At `-4`, the *minimum* `-6` multiplied by `-4` gives `24` — the answer. A solution tracking only the maximum would have discarded `-6` as useless and returned 3.

THE SIMULTANEOUS UPDATE IS NOT STYLISTIC.

Writing `maxHere = …; minHere = Math.min(x, maxHere*x, minHere*x);` uses the *new* `maxHere` in the second line and silently produces wrong answers on inputs like `[2,-5,-2,-4,3]`. Compute both candidates first, then assign.

INCLUDING `X` ALONE AS A CANDIDATE

is what handles zeros: after a 0 both states restart from the next element rather than being stuck at 0.

Maximum sum circular subarray `Hard`

Pattern: Kadane twice + total complement

Given a **circular** integer array `nums`, return the maximum possible sum of a non-empty subarray. Circular means the element after the last is the first again, but a subarray may use each element at most once.

Input	Output	Note
`nums = [1,-2,3,-2]`	`3`	Just [3].
`nums = [5,-3,5]`	`10`	The wrapping subarray [5, 5].
`nums = [-3,-2,-3]`	`-2`	All negative — the largest single element.

HINTS

1. The answer either wraps around the end or it does not. Handle the two cases separately and take the larger.
2. The non-wrapping case is ordinary Kadane.
3. A wrapping subarray is the complement of a non-wrapping one — so its sum is `total - (minimum subarray sum)`.
4. Beware the all-negative case: the minimum subarray is then the whole array, and `total - min` is 0, corresponding to the empty subarray. Special-case it.

Approach	Idea	Time	Space
Try every start	Simulate the circle from each starting index.	`O(n²)`	`O(1)`
Duplicate the array	Concatenate `nums` with itself and run a length-limited sliding window with a deque.	`O(n)`	`O(n)`
Kadane max and min + total	Answer is `max(maxKadane, total - minKadane)`, with an all-negative guard.	`O(n)`	`O(1)`

Solution — js

```
function maxSubarraySumCircular(nums) {
  let total = nums[0];
  let curMax = nums[0], bestMax = nums[0];
  let curMin = nums[0], bestMin = nums[0];

  for (let i = 1; i < nums.length; i++) {
    const x = nums[i];
    total += x;
    curMax = Math.max(x, curMax + x);
    bestMax = Math.max(bestMax, curMax);
    curMin = Math.min(x, curMin + x);
    bestMin = Math.min(bestMin, curMin);
  }

  // all negative: the wrapping candidate would be the EMPTY
  // subarray, which is not allowed
  if (bestMax < 0) return bestMax;

  return Math.max(bestMax, total - bestMin);
}
```

Solution — cpp

```
class Solution {
public:
    int maxSubarraySumCircular(vector<int>& nums) {
        int total = nums[0];
        int curMax = nums[0], bestMax = nums[0];
        int curMin = nums[0], bestMin = nums[0];
        for (size_t i = 1; i < nums.size(); ++i) {
            int x = nums[i];
            total += x;
            curMax = max(x, curMax + x);  bestMax = max(bestMax, curMax);
            curMin = min(x, curMin + x);  bestMin = min(bestMin, curMin);
        }
        if (bestMax < 0) return bestMax;
        return max(bestMax, total - bestMin);
    }
};
```

Solution — python

```
class Solution:
    def maxSubarraySumCircular(self, nums: list[int]) -> int:
        total = cur_max = best_max = cur_min = best_min = nums[0]
        for x in nums[1:]:
            total += x
            cur_max = max(x, cur_max + x)
            best_max = max(best_max, cur_max)
            cur_min = min(x, cur_min + x)
            best_min = min(best_min, cur_min)
        if best_max < 0:
            return best_max
        return max(best_max, total - best_min)
```

Solution — java

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = nums[0];
        int curMax = nums[0], bestMax = nums[0];
        int curMin = nums[0], bestMin = nums[0];
        for (int i = 1; i < nums.length; i++) {
            int x = nums[i];
            total += x;
            curMax = Math.max(x, curMax + x); bestMax = Math.max(bestMax, curMax);
            curMin = Math.min(x, curMin + x); bestMin = Math.min(bestMin, curMin);
        }
        if (bestMax < 0) return bestMax;
        return Math.max(bestMax, total - bestMin);
    }
}
```

WALKTHROUGH

THE CASE SPLIT IS THE IDEA.

Any subarray of a circular array either occupies a contiguous stretch of the original array (no wrap) or wraps past the end. In the wrapping case, the elements it *excludes* form a contiguous non-wrapping block — so maximising the wrapping sum is the same as *minimising* the excluded block. Hence `total - minSubarray`, computed by running Kadane a second time with the comparisons flipped.

WHY THE ALL-NEGATIVE GUARD IS NECESSARY.

On `[-3,-2,-3]` the minimum subarray is the entire array, so `total - bestMin = 0` — which would be the sum of the empty subarray. The problem requires a non-empty answer, so when every element is negative (detected by `bestMax < 0`) the answer is simply the largest element. Without that line the function returns 0 and the test fails.

BOTH KADANES RUN IN THE SAME LOOP

, so this is still one pass and O(1) space.

Arrays: matrix traversal patterns

Two-dimensional index arithmetic. The algorithms are simple; the bugs are all boundaries, and the trick in most of these problems is finding a transformation that makes the boundaries disappear.

The patterns

```
THE CONVENTIONS. Fix them once and never think again:

  R = grid.length            rows
  C = grid[0].length         columns
  grid[r][c]                 ROW FIRST, always

  in bounds:  r >= 0 && r < R && c >= 0 && c < C

FLATTENING — a 2-D cell to a 1-D index and back:
  idx = r * C + c
  r   = Math.floor(idx / C)
  c   = idx % C
Useful for hashing a cell, or for binary search over a
row-sorted matrix as if it were one array.

THE FOUR NEIGHBOUR DIRECTIONS:
  const DIRS = [[-1,0],[1,0],[0,-1],[0,1]];
  for (const [dr, dc] of DIRS) {
      const nr = r + dr, nc = c + dc;
      if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
      ...
  }
Eight directions add the four diagonals. A DIRECTION
ARRAY IS ALWAYS BETTER THAN FOUR COPIES OF THE BODY —
fewer places to make a typo.

------------------------------------------------------------

ROTATION BY 90 DEGREES, in place:
  TRANSPOSE (swap across the main diagonal), then
  REVERSE EACH ROW.

    1 2 3      1 4 7      7 4 1
    4 5 6  ->  2 5 8  ->  8 5 2
    7 8 9      3 6 9      9 6 3
    original   transpose  reverse rows

  Anticlockwise: transpose, then reverse each COLUMN
  (equivalently, reverse the row order first).

  THE TRANSPOSE LOOP MUST START AT j = i, not j = 0 —
  otherwise every pair is swapped twice and you get the
  original back.

------------------------------------------------------------

SPIRAL — four moving boundaries:
    top, bottom, left, right
    walk right along `top`, then down `right`,
    then left along `bottom`, then up `left`,
    shrinking each boundary after use.
    CHECK top <= bottom AND left <= right BEFORE the
    last two walks, or a single remaining row/column is
    emitted twice.

------------------------------------------------------------

SEARCH IN A SORTED MATRIX — start from a CORNER where
the two directions disagree:
    start at TOP-RIGHT:
      value too big   -> move LEFT   (column decreases)
```

```
          value too small -> move DOWN   (row increases)
       Each step eliminates a whole row or column: O(R + C).
```

When to reach for it

"Rotate / transpose in place"

Compose two simple reversals rather than working out the rotation index formula.

Boundary walks

Spiral, diagonal, zigzag. Track explicit boundaries and shrink them.

Sorted rows and columns

Start at a corner with conflicting directions — top-right or bottom-left.

Fully row-sorted, rows chained

Treat as a flat array and binary search with `idx / C`, `idx % C`.

Flood fill / islands / shortest path

That is a graph on a grid — see chapter 28.

"O(1) extra space" on a grid

Encode markers in the matrix itself — first row/column as flags, or a sentinel value.

```
/* --- BUILDING A GRID, and the trap --- */
const g = Array.from({length: R}, () => new Array(C).fill(0));
// NOT new Array(R).fill(new Array(C).fill(0))
//     ^ every row is the SAME array; writing one writes all

/* --- COPYING --- */
const copy = g.map(row => row.slice());   // deep enough for
                                          // a grid of numbers

/* --- ITERATING --- */
for (let r = 0; r < R; r++)
    for (let c = 0; c < C; c++) { ... }

/* --- THE GUARD, as a helper --- */
const ok = (r, c) => r >= 0 && r < R && c >= 0 && c < C;

/* --- EMPTY-INPUT CHECK, first line of every solution --- */
if (!grid || grid.length === 0 || grid[0].length === 0) return ...;
```

Problems

How it shows up

Symptom	Cause
Writing one cell changes a whole column	`fill()` with a shared row array
Transpose returns the original	Inner loop started at 0 — every pair swapped twice
Spiral repeats the middle row or column	Missing the `top <= bottom` / `left <= right` guard
Index out of range on a non-square grid	Used `R` where `C` was needed
Rotation is anticlockwise	Reversed rows before transposing instead of after
Search misses values	Started from a corner where both directions agree
Zero-marking corrupts later rows	Marked with 0 and then read it as original data
Crash on `grid[0].length`	Empty outer array not checked first

Check yourself

- ✓ Write the bounds guard and the four-direction array.
- ✓ Give the flatten and unflatten formulas.
- ✓ Give the two-step in-place 90° rotation, both directions.
- ✓ Say why the transpose loop starts at `j = i`.
- ✓ Name the four spiral boundaries and the guard that prevents double emission.
- ✓ Explain why the sorted-matrix search starts at a corner.
- ✓ Give the correct way to build a 2-D array in JavaScript.
- ✓ Describe how to mark state in the matrix itself for O(1) space.

Problems

Rotate image 90° clockwise `Medium`

Pattern: Transpose then reverse rows

Given an `n × n` matrix, rotate it 90 degrees clockwise **in place**. Do not allocate another matrix.

Input	Output	Note
`matrix = [[1,2,3],[4,5,6],[7,8,9]]`	`[[7,4,1],[8,5,2],[9,6,3]]`	
`matrix = [[1,2],[3,4]]`	`[[3,1],[4,2]]`	

HINTS

1. Rather than deriving the index mapping, compose two operations you already know.
2. Transposing swaps `m[i][j]` with `m[j][i]`. What does the matrix look like afterwards?
3. Transpose, then reverse each row.
4. The transpose loop must start at `j = i`, or each pair is swapped twice and nothing changes.

Approach	Idea	Time	Space
Copy into a new matrix	`out[j][n-1-i] = m[i][j]`. Clear, but uses O(n^2) extra space, which the problem forbids.	`O(n²)`	`O(n²)`
Rotate four cells at a time	Cycle each group of four positions through a temporary. Correct and O(1), but the index expressions are easy to get wrong.	`O(n²)`	`O(1)`
Transpose then reverse rows	Two trivially correct passes.	`O(n²)`	`O(1)`

Solution — js

```
function rotate(matrix) {
  const n = matrix.length;

  // 1. transpose — swap across the main diagonal.
  //    j starts at i so each pair is swapped exactly ONCE.
  for (let i = 0; i < n; i++)
    for (let j = i; j < n; j++)
      [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];

  // 2. reverse each row
  for (const row of matrix) row.reverse();
}
```

Solution — cpp

```
class Solution {
public:
    void rotate(vector<vector<int>>& m) {
        int n = m.size();
        for (int i = 0; i < n; ++i)
            for (int j = i; j < n; ++j)
                swap(m[i][j], m[j][i]);
        for (auto& row : m) reverse(row.begin(), row.end());
    }
};
```

Solution — python

```
class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        n = len(matrix)
        for i in range(n):
            for j in range(i, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        for row in matrix:
            row.reverse()
```

Solution — java

```java
class Solution {
    public void rotate(int[][] m) {
        int n = m.length;
        for (int i = 0; i < n; i++)
            for (int j = i; j < n; j++) {
                int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
            }
        for (int[] row : m) {
            for (int l = 0, r = n - 1; l < r; l++, r--) {
                int t = row[l]; row[l] = row[r]; row[r] = t;
            }
        }
    }
}
```

WALKTHROUGH

WHY THE COMPOSITION WORKS.

Transposing reflects the matrix across the main diagonal; reversing each row reflects it across the vertical centre line. Two reflections about axes meeting at 45° compose to a rotation of 90°. That is also why the anticlockwise version is transpose-then-reverse-*columns* — the second reflection is horizontal instead.

THE `J = I` BOUND IS THE BUG THIS PROBLEM EXISTS TO TEACH.

With `j = 0` the loop visits both `(i,j)` and `(j,i)`, swapping each pair twice and leaving the matrix unchanged. Starting at `i` visits only the upper triangle — and including `j = i` is harmless since swapping a cell with itself does nothing.

Spiral matrix `Medium`

Pattern: Four shrinking boundaries

Given an `m × n` matrix, return all its elements in spiral order — right along the top, down the right side, left along the bottom, up the left side, then inward.

Input	Output	Note
`matrix = [[1,2,3],[4,5,6],[7,8,9]]`	`[1,2,3,6,9,8,7,4,5]`	
`matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]`	`[1,2,3,4,8,12,11,10,9,5,6,7]`	

HINTS

1. Keep four boundaries: `top`, `bottom`, `left`, `right`. Walk one edge, then shrink that boundary.
2. After walking the top row, increment `top`. After the right column, decrement `right`. And so on.
3. Before the leftward and upward walks, re-check that the boundaries have not crossed — otherwise a single remaining row or column is emitted twice.

Approach	Idea	Time	Space
Direction vector with a visited grid	Walk, turning right whenever the next cell is out of bounds or already visited. Uses O(m·n) extra space.	`O(m·n)`	`O(m·n)`
Four boundaries	No extra space; each element emitted exactly once.	`O(m·n)`	`O(1)` `extra`

Solution — js

```
function spiralOrder(matrix) {
  if (!matrix.length || !matrix[0].length) return [];
  const out = [];
  let top = 0, bottom = matrix.length - 1;
  let left = 0, right = matrix[0].length - 1;

  while (top <= bottom && left <= right) {
    for (let c = left; c <= right; c++) out.push(matrix[top][c]);
    top++;

    for (let r = top; r <= bottom; r++) out.push(matrix[r][right]);
    right--;

    if (top <= bottom) {                // still a row left?
      for (let c = right; c >= left; c--) out.push(matrix[bottom][c]);
      bottom--;
    }
    if (left <= right) {                // still a column left?
      for (let r = bottom; r >= top; r--) out.push(matrix[r][left]);
      left++;
    }
  }
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& m) {
        if (m.empty() || m[0].empty()) return {};
        vector<int> out;
        int top = 0, bottom = m.size() - 1, left = 0, right = m[0].size() - 1;
        while (top <= bottom && left <= right) {
            for (int c = left; c <= right; ++c) out.push_back(m[top][c]);
            ++top;
            for (int r = top; r <= bottom; ++r) out.push_back(m[r][right]);
            --right;
            if (top <= bottom) {
                for (int c = right; c >= left; --c) out.push_back(m[bottom][c]);
                --bottom;
            }
            if (left <= right) {
                for (int r = bottom; r >= top; --r) out.push_back(m[r][left]);
                ++left;
            }
        }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def spiralOrder(self, matrix: list[list[int]]) -> list[int]:
        if not matrix or not matrix[0]:
            return []
        out = []
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1
        while top <= bottom and left <= right:
            for c in range(left, right + 1):
                out.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1):
                out.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1):
                    out.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1):
                    out.append(matrix[r][left])
                left += 1
        return out
```

Solution — java

```java
class Solution {
    public List<Integer> spiralOrder(int[][] m) {
        List<Integer> out = new ArrayList<>();
        if (m.length == 0 || m[0].length == 0) return out;
        int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int c = left; c <= right; c++) out.add(m[top][c]);
            top++;
            for (int r = top; r <= bottom; r++) out.add(m[r][right]);
            right--;
            if (top <= bottom) {
                for (int c = right; c >= left; c--) out.add(m[bottom][c]);
                bottom--;
            }
            if (left <= right) {
                for (int r = bottom; r >= top; r--) out.add(m[r][left]);
                left++;
            }
        }
        return out;
    }
}
```

WALKTHROUGH

THE TWO MID-LOOP GUARDS ARE THE WHOLE DIFFICULTY.

Consider a single-row matrix `[[1,2,3]]`. The first walk emits 1, 2, 3 and sets `top = 1`, which now exceeds `bottom = 0`. Without the `if (top <= bottom)` check, the leftward walk would run along `matrix[bottom]` — the same row — and emit 3, 2, 1 again. The single-column test `[[1],[2],[3]]` catches the symmetric case.

WHY THE GUARDS ARE NOT NEEDED FOR THE FIRST TWO WALKS:

the `while` condition was just checked, so the top row and right column are known to exist. Only after two boundaries have moved can they cross mid-iteration.

Search a 2-D matrix II `Medium`

Pattern: Staircase search from a corner

Search a `m × n` matrix for a `target`. Each **row** is sorted left to right and each **column** is sorted top to bottom, but rows are *not* chained — the first element of a row may be smaller than the last element of the previous row.

Return `true` if the target is present.

Input	Output	Note
`matrix = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target = 5`	`true`	
`same matrix, target = 20`	`false`	

HINTS

1. Binary search on each row works but is O(m log n). There is a linear-in-(m+n) approach.
2. Look at the top-right corner: everything to its left is smaller, everything below it is larger. The two directions disagree, which makes the comparison decisive.
3. If the value is too large, the whole column can be discarded. If too small, the whole row can.
4. The bottom-left corner works equally well; the top-left does not, because both directions increase.

Approach	Idea	Time	Space
Scan everything	Check every cell.	`O(m·n)`	`O(1)`
Binary search each row	Uses the row ordering only.	`O(m log n)`	`O(1)`
Staircase from the top-right	Each comparison eliminates an entire row or column.	`O(m + n)`	`O(1)`

Solution — js

```javascript
function searchMatrix(matrix, target) {
  if (!matrix.length || !matrix[0].length) return false;

  // start at the TOP-RIGHT: left is smaller, down is larger
  let r = 0, c = matrix[0].length - 1;

  while (r < matrix.length && c >= 0) {
    const v = matrix[r][c];
    if (v === target) return true;
    if (v > target) c--;      // whole COLUMN is too big
    else            r++;      // whole ROW is too small
  }
  return false;
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& m, int target) {
        if (m.empty() || m[0].empty()) return false;
        int r = 0, c = m[0].size() - 1;
        while (r < (int)m.size() && c >= 0) {
            if (m[r][c] == target) return true;
            if (m[r][c] > target) --c;
            else                  ++r;
        }
        return false;
    }
};
```

Solution — python

```python
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False
        r, c = 0, len(matrix[0]) - 1
        while r < len(matrix) and c >= 0:
            v = matrix[r][c]
            if v == target:
                return True
            if v > target:
                c -= 1
            else:
                r += 1
        return False
```

Solution — java

```
class Solution {
    public boolean searchMatrix(int[][] m, int target) {
        if (m.length == 0 || m[0].length == 0) return false;
        int r = 0, c = m[0].length - 1;
        while (r < m.length && c >= 0) {
            if (m[r][c] == target) return true;
            if (m[r][c] > target) c--;
            else                  r++;
        }
        return false;
    }
}
```

WALKTHROUGH

WHY THE CORNER CHOICE MATTERS.

At the top-left, moving right increases the value and moving down also increases it — a comparison tells you nothing about which way to go. At the top-right the two available moves disagree: left decreases, down increases. That makes every comparison eliminate an entire row or column, which is what bounds the walk at `m + n` steps.

THE ELIMINATION ARGUMENT, STATED PROPERLY:

if `matrix[r][c] > target` at the top-right, then every element *below* it in column `c` is also greater (columns are sorted), so no cell in that column can be the target — discard the column. Symmetrically, if the value is too small, every element to its left in row `r` is smaller still, so the row is useless — discard it.

NOTE THIS IS A DIFFERENT PROBLEM FROM "SEARCH A 2-D MATRIX I"

, where the rows are chained end to end. That one really is a single sorted array and admits a true O(log(m·n)) binary search using `idx / C` and `idx % C`. Check which variant you have been given.

Searching: binary search done correctly

Everyone knows the idea. Almost nobody writes it without an off-by-one on the first attempt. This chapter gives you one template you can trust, and the discipline that makes the boundary cases obvious rather than guessed.

The pattern

```
HALVE THE SEARCH SPACE EVERY STEP. n candidates become 1
in log2(n) steps: a million-element array is 20 steps.

THE PRECONDITION IS NOT "SORTED". It is:

  THE ARRAY CAN BE SPLIT BY SOME PREDICATE INTO A FALSE
  REGION FOLLOWED BY A TRUE REGION.

  [ F, F, F, F, T, T, T, T ]
                ^
                the boundary — what you are looking for

Sortedness is just the most common way to get that
property. Once you see binary search as FINDING A
BOUNDARY rather than FINDING A VALUE, the rotated-array
and search-on-answer variants stop being separate tricks.

THE BUG EVERYONE WRITES:

  mid = (lo + hi) / 2        // OVERFLOWS in C++/Java
                             // when lo+hi > 2^31
  mid = lo + (hi - lo) / 2   // correct everywhere

In JavaScript numbers are doubles so overflow is not a
practical concern, but write the safe form anyway — the
interviewer is watching, and you will write C++ one day.

THE THREE DECISIONS THAT DETERMINE CORRECTNESS:

  1. IS hi AN INDEX (n-1) OR A BOUND (n)?
  2. IS THE LOOP `lo <= hi` OR `lo < hi`?
  3. DO YOU MOVE TO mid OR mid±1?

MIXING CONVENTIONS IS WHAT CAUSES INFINITE LOOPS AND
MISSED ELEMENTS. Pick one and never deviate.

THE TERMINATION ARGUMENT, worth being able to state:
the interval must strictly shrink every iteration. If a
branch can leave [lo, hi] unchanged, the loop hangs.
With `lo < hi` and `mid = lo + (hi-lo)/2`, mid can equal
lo but never hi, so `hi = mid` shrinks and `lo = mid+1`
shrinks. That is why the halves are asymmetric.
```

Two templates, and only two

```
/* ===== TEMPLATE A: EXACT MATCH =====
   "Is this value present, and where?"                    */
function find(a, target) {
    let lo = 0, hi = a.length - 1;          // hi is an INDEX
    while (lo <= hi) {                      // <= : lo==hi is
                                            //      still valid
        const mid = lo + ((hi - lo) >> 1);
        if (a[mid] === target) return mid;
        if (a[mid] < target) lo = mid + 1;  // EXCLUDE mid
        else                 hi = mid - 1;  // EXCLUDE mid
    }
    return -1;
}

/* ===== TEMPLATE B: FIND THE BOUNDARY =====
   "First index where predicate(i) is true."
   USE THIS ONE FOR EVERYTHING THAT IS NOT AN EXACT MATCH. */
function firstTrue(lo, hi, pred) {          // hi is a BOUND
    while (lo < hi) {                       // < : stop when
                                            //     they meet
        const mid = lo + ((hi - lo) >> 1);
        if (pred(mid)) hi = mid;            // KEEP mid - it
                                            // might be the
                                            // answer
        else           lo = mid + 1;        // mid is false,
                                            // discard it
    }
    return lo;          // == hi; equals `hi` if nothing is true
}

/* TEMPLATE B SOLVES ALL OF THESE, by changing only pred: */
    lowerBound(a, t)   // first index with a[i] >= t
      = firstTrue(0, a.length, i => a[i] >= t)
    upperBound(a, t)   // first index with a[i] >  t
      = firstTrue(0, a.length, i => a[i] >  t)
    countOf(a, t)      = upperBound(a,t) - lowerBound(a,t)
    insertPosition     = lowerBound(a, t)
    firstBadVersion    = firstTrue(1, n+1, v => isBad(v))

/* AND THE STANDARD LIBRARY EQUIVALENTS, worth knowing:
     C++     std::lower_bound / std::upper_bound
     Python  bisect.bisect_left / bisect.bisect_right
     Java    Arrays.binarySearch (returns -(insertion)-1
             when absent - read that signature carefully)

   THE DISCIPLINE THAT REMOVES THE BUGS:
     · state the INVARIANT before writing the loop:
       "the answer is always inside [lo, hi]"
     · every branch must SHRINK the interval
     · after the loop, say what lo and hi mean
     · TEST WITH: empty, one element, target smaller than
       everything, larger than everything, and duplicates  */
```

Problems

How it shows up

Symptom	Cause
Infinite loop	A branch that does not shrink the interval — usually `lo = mid` with `lo < hi`
Misses the target at index 0 or n−1	Mixed conventions: `hi = n` with `lo <= hi`
Overflow in C++/Java	`(lo + hi) / 2` — use `lo + (hi - lo) / 2`
Returns any duplicate, not the first	Exact-match template returns on the first hit — use lower bound
Off by one on the insert position	Confusing lower bound with upper bound
Works on odd sizes, fails on even	Mid rounding — check whether you need the upper mid
Fails on the rotated array at the pivot	Comparing against the wrong endpoint when deciding which half is sorted
Correct value, wrong index with duplicates	Rotated search is not O(log n) when duplicates are allowed

Check yourself

- ✓ State the real precondition for binary search, not just "sorted".
- ✓ Write both templates from memory.
- ✓ Explain why `hi = mid` and `lo = mid + 1` are asymmetric.
- ✓ Give the termination argument.
- ✓ Define lower bound and upper bound, and express "count of t" with them.
- ✓ Write the overflow-safe mid and say where it matters.
- ✓ Explain how to decide which half of a rotated array is sorted.
- ✓ Say why duplicates break the O(log n) guarantee on a rotated array.

Problems

Binary search `Easy`

Pattern: Exact match

Given a sorted array of distinct integers `nums` and a `target`, return its index, or `-1` if it is not present. Must run in O(log n).

Input	Output	Note
`nums = [-1,0,3,5,9,12], target = 9`	`4`	
`nums = [-1,0,3,5,9,12], target = 2`	`-1`	

HINTS

1. Use template A: `hi` is an index, the loop is `lo <= hi`, and both branches exclude `mid`.
2. Compute mid as `lo + ((hi - lo) >> 1)`.
3. Check the empty array and a single-element array explicitly.

Approach	Idea	Time	Space
Linear scan	Check each element.	`O(n)`	`O(1)`
Binary search	Halve the interval each step, excluding `mid` on both branches since it has been tested.	`O(log n)`	`O(1)`

Solution — js

```
function search(nums, target) {
  let lo = 0, hi = nums.length - 1;
  while (lo <= hi) {
    const mid = lo + ((hi - lo) >> 1);
    if (nums[mid] === target) return mid;
    if (nums[mid] < target) lo = mid + 1;
    else                    hi = mid - 1;
  }
  return -1;
}
```

Solution — cpp

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int lo = 0, hi = (int)nums.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1;
            else                    hi = mid - 1;
        }
        return -1;
    }
};
```

Solution — python

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1
        while lo <= hi:
            mid = lo + (hi - lo) // 2
            if nums[mid] == target:
                return mid
            if nums[mid] < target:
                lo = mid + 1
            else:
                hi = mid - 1
        return -1
```

Solution — java

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1;
            else                    hi = mid - 1;
        }
        return -1;
    }
}
```

WALKTHROUGH

WHY `LO <= HI` AND NOT `LO < HI`.

With `hi` as an inclusive index, the interval `[lo, hi]` is non-empty whenever `lo <= hi`. Using `<` would skip the case where a single candidate remains — and that candidate is exactly the answer when the target sits at an end. The empty-array test and `[5], 5` both catch this.

Search insert position Easy

Pattern: Lower bound

Given a sorted array of distinct integers and a target, return the index of the target if found. If not, return the index where it **would** be inserted to keep the array sorted.

Input	Output	Note
`nums = [1,3,5,6], target = 5`	`2`	
`nums = [1,3,5,6], target = 2`	`1`	
`nums = [1,3,5,6], target = 7`	`4`	Past the end — a valid answer.

HINTS

1. This is exactly the lower bound: the first index `i` with `nums[i] >= target`.
2. Use template B with `hi = nums.length` (a bound, not an index) so "insert at the end" is representable.
3. When the predicate is true, keep `mid` — it may be the answer.

Approach	Idea	Time	Space
Linear scan	Walk until you find an element at least as large.	`O(n)`	`O(1)`
Lower bound (template B)	Boundary search for the first index satisfying `nums[i] >= target`. Returns `n` naturally when the target exceeds everything.	`O(log n)`	`O(1)`

Solution — js

```
function searchInsert(nums, target) {
  let lo = 0, hi = nums.length;        // hi is a BOUND, not an index
  while (lo < hi) {
    const mid = lo + ((hi - lo) >> 1);
    if (nums[mid] >= target) hi = mid;  // might be the answer: KEEP
    else                     lo = mid + 1;
  }
  return lo;
}
```

Solution — cpp

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        return lower_bound(nums.begin(), nums.end(), target) - nums.begin();
        // or written out:
        // int lo = 0, hi = nums.size();
        // while (lo < hi) { int mid = lo + (hi-lo)/2;
        //   if (nums[mid] >= target) hi = mid; else lo = mid+1; }
        // return lo;
    }
};
```

Solution — python

```
import bisect

class Solution:
    def searchInsert(self, nums: list[int], target: int) -> int:
        return bisect.bisect_left(nums, target)
        # written out:
        # lo, hi = 0, len(nums)
        # while lo < hi:
        #     mid = lo + (hi - lo) // 2
        #     if nums[mid] >= target: hi = mid
        #     else: lo = mid + 1
        # return lo
```

Solution — java

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int lo = 0, hi = nums.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] >= target) hi = mid;
            else                     lo = mid + 1;
        }
        return lo;
    }
}
```

WALKTHROUGH

WHY `HI = N` RATHER THAN `N - 1`.

The answer can legitimately be `n` — "insert past the last element". An inclusive `hi = n - 1` cannot represent that, so you would need a special case afterwards. Choosing the half-open interval `[lo, hi)` makes the boundary case fall out for free, which is the general argument for preferring template B.

RECOGNISE THIS AS LOWER BOUND AND A FAMILY OF PROBLEMS COLLAPSES:

first occurrence of a duplicate, count of a value, "first bad version", and the insert position are all the same three lines with a different predicate.

First and last position of an element `Medium`

Pattern: Lower bound + upper bound

Given an array of integers sorted in non-decreasing order, find the starting and ending index of a given `target`. Return `[-1, -1]` if the target is not present. Must run in O(log n).

Input	Output	Note
`nums = [5,7,7,8,8,10], target = 8`	`[3,4]`	
`nums = [5,7,7,8,8,10], target = 6`	`[-1,-1]`	
`nums = [], target = 0`	`[-1,-1]`	

HINTS

1. Two boundary searches, not one search plus a linear scan — a linear scan of duplicates is O(n) in the worst case.
2. The first position is `lowerBound(target)`: first index with `nums[i] >= target`.
3. The last position is `upperBound(target) - 1`: first index with `nums[i] > target`, minus one.
4. Check presence by testing whether `lowerBound` is in range and actually holds the target.

Approach	Idea	Time	Space
Find one, then expand	Binary search for any occurrence, then walk outwards. Degrades to O(n) when the array is all one value.	`O(n)`	`O(1)`
Two boundary searches	Lower bound gives the start; upper bound minus one gives the end. Two independent O(log n) passes.	`O(log n)`	`O(1)`

Solution — js

```javascript
function searchRange(nums, target) {
  const bound = (pred) => {
    let lo = 0, hi = nums.length;
    while (lo < hi) {
      const mid = lo + ((hi - lo) >> 1);
      if (pred(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
  };

  const first = bound(i => nums[i] >= target);   // lower bound
  // not present at all?
  if (first === nums.length || nums[first] !== target) return [-1, -1];

  const afterLast = bound(i => nums[i] > target); // upper bound
  return [first, afterLast - 1];
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        auto lo = lower_bound(nums.begin(), nums.end(), target);
        if (lo == nums.end() || *lo != target) return {-1, -1};
        auto hi = upper_bound(nums.begin(), nums.end(), target);
        return {(int)(lo - nums.begin()), (int)(hi - nums.begin()) - 1};
    }
};
```

Solution — python

```python
import bisect

class Solution:
    def searchRange(self, nums: list[int], target: int) -> list[int]:
        first = bisect.bisect_left(nums, target)
        if first == len(nums) or nums[first] != target:
            return [-1, -1]
        return [first, bisect.bisect_right(nums, target) - 1]
```

Solution — java

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int first = bound(nums, target, true);
        if (first == nums.length || nums[first] != target)
            return new int[]{-1, -1};
        return new int[]{first, bound(nums, target, false) - 1};
    }
    // lower = true  -> first i with nums[i] >= target
    // lower = false -> first i with nums[i] >  target
    private int bound(int[] nums, int target, boolean lower) {
        int lo = 0, hi = nums.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            boolean ok = lower ? nums[mid] >= target : nums[mid] > target;
            if (ok) hi = mid; else lo = mid + 1;
        }
        return lo;
    }
}
```

WALKTHROUGH

THE PRESENCE CHECK MUST COME FIRST AND MUST BE IN THIS EXACT FORM.

`first === nums.length` handles a target larger than everything, and `nums[first] !== target` handles a target that falls in a gap. Testing only one of the two crashes or returns nonsense on the empty array and on `[1,2,3], 0` respectively.

WHY NOT SCAN OUTWARDS FROM A HIT?

On `[2,2,2,…,2]` with target 2 there are n occurrences, so expanding is O(n) and the required O(log n) is violated. The interviewer will construct exactly that input.

Search in a rotated sorted array `Medium`

Pattern: Binary search on a piecewise-sorted array

An ascending array of **distinct** values was rotated at some unknown pivot — for example `[0,1,2,4,5,6,7]` became `[4,5,6,7,0,1,2]`. Given the rotated array and a `target`, return its index or `-1`. Must run in O(log n).

Input	Output	Note
`nums = [4,5,6,7,0,1,2], target = 0`	`4`	
`nums = [4,5,6,7,0,1,2], target = 3`	`-1`	
`nums = [1], target = 0`	`-1`	

HINTS

1. Split at `mid`. At least one of the two halves is *fully sorted* — always.
2. Determine which half is sorted by comparing `nums[lo]` with `nums[mid]`.
3. In the sorted half you can test membership with two comparisons. If the target is in it, search there; otherwise search the other half.
4. Use `<=` when comparing `nums[lo] <= nums[mid]` so that a two-element interval is classified correctly.

Approach	Idea	Time	Space
Linear scan	Ignore the structure and scan.	`O(n)`	`O(1)`
Find the pivot, then search	Binary search for the rotation point, then binary search the correct segment. Two passes, easy to reason about.	`O(log n)`	`O(1)`
Single modified binary search	One pass. Each step identifies the sorted half and decides whether the target lies inside it.	`O(log n)`	`O(1)`

Solution — js

```
function search(nums, target) {
  let lo = 0, hi = nums.length - 1;
  while (lo <= hi) {
    const mid = lo + ((hi - lo) >> 1);
    if (nums[mid] === target) return mid;

    if (nums[lo] <= nums[mid]) {
      // LEFT half [lo..mid] is sorted
      if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
      else                                          lo = mid + 1;
    } else {
      // RIGHT half [mid..hi] is sorted
      if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
      else                                          hi = mid - 1;
    }
  }
  return -1;
}
```

Solution — cpp

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int lo = 0, hi = (int)nums.size() - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[lo] <= nums[mid]) {
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else                                          lo = mid + 1;
            } else {
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else                                          hi = mid - 1;
            }
        }
        return -1;
    }
};
```

Solution — python

```python
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1
        while lo <= hi:
            mid = lo + (hi - lo) // 2
            if nums[mid] == target:
                return mid
            if nums[lo] <= nums[mid]:            # left half sorted
                if nums[lo] <= target < nums[mid]:
                    hi = mid - 1
                else:
                    lo = mid + 1
            else:                                 # right half sorted
                if nums[mid] < target <= nums[hi]:
                    lo = mid + 1
                else:
                    hi = mid - 1
        return -1
```

Solution — java

```java
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[lo] <= nums[mid]) {
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else                                          lo = mid + 1;
            } else {
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else                                          hi = mid - 1;
            }
        }
        return -1;
    }
}
```

WALKTHROUGH

THE STRUCTURAL FACT THAT MAKES THIS WORK:

cutting a rotated sorted array at any point leaves at least one side unrotated, because there is only one "drop". So every step you can identify a fully sorted half, test membership in it with two comparisons, and discard half the array either way — preserving O(log n).

WHY `NUMS[LO] <= NUMS[MID]` NEEDS THE `<=`.

When the interval has two elements, `mid === lo`, so the comparison is `nums[lo] <= nums[lo]`. With strict `<` it would be false and you would take the "right half sorted" branch incorrectly. `[3,1]` searching for 1 is the test that catches it.

THE FOLLOW-UP:

"what if duplicates are allowed?" Then `nums[lo] === nums[mid] === nums[hi]` tells you nothing about which side is sorted — as in `[1,1,1,0,1,1]`. The fix is to shrink `lo++, hi--` in that case, which makes the worst case O(n). Say that explicitly; the point of the follow-up is to see whether you notice the guarantee breaks.

Searching: binary search on the answer

Binary search without a sorted array anywhere in sight. When the question is "what is the smallest X that works?", search the space of *answers* rather than the input — and this is the single highest-leverage pattern for medium and hard problems.

The pattern

```
THE RECOGNITION SIGNAL:

  "MINIMISE the maximum ..."
  "MAXIMISE the minimum ..."
  "the SMALLEST capacity / speed / size such that ..."
  "can we do it within K ...?"

plus a MONOTONIC feasibility: if an answer of X works,
then every answer larger than X also works (or every
smaller one, for a maximise problem).

THAT MONOTONICITY IS THE SORTED ARRAY. You never
materialise it — you just binary search over it.

  candidate answers:  1  2  3  4  5  6  7  8  9
  feasible(x):        N  N  N  N  Y  Y  Y  Y  Y
                                  ^
                                  the answer

THE RECIPE, four steps:

  1. GUESS the answer x.
  2. Write feasible(x): "can the task be done with x?"
     This is usually a SIMPLE GREEDY SCAN, O(n).
  3. Verify MONOTONICITY — argue why feasible is false
     then true (never alternating).
  4. Binary search the boundary with template B from
     chapter 09.

THE TEMPLATE:

  let lo = <smallest conceivable answer>;
  let hi = <largest conceivable answer>;
  while (lo < hi) {
      const mid = lo + Math.floor((hi - lo) / 2);
      if (feasible(mid)) hi = mid;      // might be it
      else               lo = mid + 1;  // definitely not
  }
  return lo;

PICKING THE BOUNDS IS HALF THE PROBLEM:
  lo = the answer that is trivially too small
       (often max(a), or 1)
  hi = the answer that trivially always works
       (often sum(a), or max value)
  GETTING lo TOO HIGH SILENTLY CUTS OFF VALID ANSWERS.
  When in doubt, make the range wider — it costs one
  extra iteration per doubling.

COST: O(n · log(range)). The log of a numeric range is
tiny — searching 10^9 values is 30 iterations — so this
turns "try every capacity" into something linear-ish.
```

When to reach for it

"Minimum largest / maximum smallest"

Split array largest sum, ship packages, allocate books, painters partition.

A rate or capacity to choose

Koko eating bananas, minimum speed to arrive on time.

"Within D days / K groups"

Feasibility is a greedy sweep counting groups.

Maximise the minimum distance

Aggressive cows, magnetic force between balls. Note the flipped monotonicity.

Real-valued answers

Iterate a fixed 100 times, or until `hi - lo < 1e-9`, instead of using integer bounds.

Kth smallest in a structured set

Binary search the value; feasibility counts how many are ≤ it.

```
/* --- THE MAXIMISE VARIANT, note the flip --- */
    // feasible(x) is TRUE for small x and FALSE for large x
    while (lo < hi) {
        const mid = lo + Math.ceil((hi - lo) / 2);   // UPPER mid
        if (feasible(mid)) lo = mid;                 // keep it
        else               hi = mid - 1;
    }
    return lo;

/* THE UPPER MID IS MANDATORY HERE. With the lower mid and
   `lo = mid`, a two-element range never shrinks and the
   loop hangs forever. This is the single most common bug in
   the maximise form.

/* --- A TYPICAL feasible(): count the groups greedily --- */
    function feasible(cap) {
        let groups = 1, load = 0;
        for (const w of weights) {
            if (w > cap) return false;   // impossible at all
            if (load + w > cap) { groups++; load = 0; }
            load += w;
        }
        return groups <= maxGroups;
    }
/* NOTE the `w > cap` guard - without it a single oversized
   item silently starts an infinite number of groups.     */
```

Problems

How it shows up

Symptom	Cause
Infinite loop in the maximise form	Lower mid with `lo = mid` — use the upper mid
Answer too large	`lo` started above the true answer
Answer too small / infeasible	`hi` not large enough to always work

Symptom	Cause
Works on small inputs, wrong on large	Feasibility is not actually monotonic — re-examine
Feasibility never returns false	Missing the "single item exceeds capacity" guard
Off by one on the boundary	Mixing `hi = mid` with `hi = mid - 1`
Float version never terminates	Exact equality on doubles — iterate a fixed count instead
Timeout	Feasibility check is worse than O(n log n)

Check yourself

- ✓ Give the four recognition signals.
- ✓ State the four-step recipe.
- ✓ Explain what plays the role of the sorted array.
- ✓ Write the minimise template and the maximise template.
- ✓ Explain why the maximise form needs the upper mid.
- ✓ Say how to choose `lo` and `hi`, and which error is silent.
- ✓ Write a greedy feasibility check and its impossible-item guard.
- ✓ Give the total complexity and why the log factor is cheap.

Problems

Koko eating bananas `Medium`

Pattern: Binary search the rate

There are `piles.length` piles of bananas and `h` hours available. At an eating speed of `k` bananas per hour, each hour Koko picks one pile and eats up to `k` from it; if the pile has fewer than `k` left, she still uses the whole hour.

Return the **minimum** integer `k` such that all piles are finished within `h` hours.

Input	Output	Note
`piles = [3,6,7,11], h = 8`	`4`	
`piles = [30,11,23,4,20], h = 5`	`30`	One pile per hour — speed must cover the largest.
`piles = [30,11,23,4,20], h = 6`	`23`	

HINTS

1. The hours needed for one pile at speed k is `ceil(pile / k)`.
2. If speed k finishes in time, so does every speed above k — the feasibility is monotonic.
3. The answer is at least 1 and at most `max(piles)`, since eating the largest pile in one hour each is always enough when `h ≥ piles.length`.

Approach	Idea	Time	Space
Try every speed	Test k = 1, 2, 3 … until one works.	`O(n · max)`	`O(1)`
Binary search the speed	Feasibility is monotonic, so binary search the boundary between too-slow and fast-enough.	`O(n log max)`	`O(1)`

Solution — js

```
function minEatingSpeed(piles, h) {
  // hours needed at speed k — monotonically DECREASING in k
  const hoursAt = (k) =>
    piles.reduce((sum, p) => sum + Math.ceil(p / k), 0);

  let lo = 1, hi = Math.max(...piles);
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (hoursAt(mid) <= h) hi = mid;   // fast enough: KEEP mid
    else                   lo = mid + 1;
  }
  return lo;
}
```

Solution — cpp

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        auto hoursAt = [&](long k) {
            long total = 0;
            for (int p : piles) total += (p + k - 1) / k;   // ceil
            return total;
        };
        int lo = 1, hi = *max_element(piles.begin(), piles.end());
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (hoursAt(mid) <= h) hi = mid;
            else                   lo = mid + 1;
        }
        return lo;
    }
};
```

Solution — python

```
import math

class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def hours_at(k: int) -> int:
            return sum(math.ceil(p / k) for p in piles)

        lo, hi = 1, max(piles)
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if hours_at(mid) <= h:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

Solution — java

```java
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int lo = 1, hi = 0;
        for (int p : piles) hi = Math.max(hi, p);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (hoursAt(piles, mid) <= h) hi = mid;
            else                          lo = mid + 1;
        }
        return lo;
    }
    private long hoursAt(int[] piles, int k) {
        long total = 0;
        for (int p : piles) total += (p + k - 1) / k;
        return total;
    }
}
```

WALKTHROUGH

WHERE IS THE SORTED ARRAY?

There is not one — the input order is irrelevant. The sorted structure is the sequence of candidate speeds 1, 2, 3, …, max, along which `hoursAt(k)` is non-increasing. That monotonicity is what licenses binary search, and stating it explicitly is the part interviewers listen for.

WHY `HI = MAX(PILES)` IS SAFE.

At that speed every pile takes exactly one hour, so the total is `piles.length` hours. The problem guarantees `h ≥ piles.length`, so the largest candidate is always feasible — which is exactly the requirement for `hi`.

INTEGER CEILING WITHOUT FLOATING POINT:

`(p + k - 1) / k`. With large values `Math.ceil(p / k)` in JavaScript is fine (doubles are exact to 2^{53}) but in C++ the division must stay in integers, and `total` needs to be 64-bit — the test with a billion-banana pile is there to make that concrete.

Capacity to ship packages within D days `Medium`

Pattern: Binary search the capacity + greedy check

Packages on a conveyor must be shipped within `days` days, **in the given order**. Each day the ship loads packages in sequence without exceeding its weight capacity.

Return the minimum capacity that allows all packages to ship within `days` days.

Input	Output	Note
`weights = [1,2,3,4,5,6,7,8,9,10], days = 5`	`15`	
`weights = [3,2,2,4,1,4], days = 3`	`6`	
`weights = [1,2,3,1,1], days = 4`	`3`	Days: [1,2] / [3] / [1,1] — capacity 2 cannot carry the package of weight 3.

HINTS

1. Given a capacity, greedily fill each day until the next package would overflow, then start a new day. Count the days.
2. The capacity must be at least `max(weights)` — a package that does not fit can never ship.
3. It need never exceed `sum(weights)` — that ships everything in one day.
4. More capacity never needs more days, so feasibility is monotonic.

Approach	Idea	Time	Space
Try every capacity	Increment from `max` to `sum`.	`O(n · sum)`	`O(1)`
Binary search + greedy count	Search capacities in `[max, sum]`; the check is one linear sweep.	`O(n log sum)`	`O(1)`

Solution — js

```
function shipWithinDays(weights, days) {
  const daysNeeded = (cap) => {
    let d = 1, load = 0;
    for (const w of weights) {
      if (load + w > cap) { d++; load = 0; }   // start a new day
      load += w;
    }
    return d;
  };

  // lo: a package must fit.  hi: everything in one day.
  let lo = Math.max(...weights);
  let hi = weights.reduce((a, b) => a + b, 0);

  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (daysNeeded(mid) <= days) hi = mid;
    else                         lo = mid + 1;
  }
  return lo;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        auto daysNeeded = [&](int cap) {
            int d = 1, load = 0;
            for (int w : weights) {
                if (load + w > cap) { ++d; load = 0; }
                load += w;
            }
            return d;
        };
        int lo = *max_element(weights.begin(), weights.end());
        int hi = accumulate(weights.begin(), weights.end(), 0);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (daysNeeded(mid) <= days) hi = mid;
            else                         lo = mid + 1;
        }
        return lo;
    }
};
```

Solution — python

```python
class Solution:
    def shipWithinDays(self, weights: list[int], days: int) -> int:
        def days_needed(cap: int) -> int:
            d, load = 1, 0
            for w in weights:
                if load + w > cap:
                    d += 1
                    load = 0
                load += w
            return d

        lo, hi = max(weights), sum(weights)
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if days_needed(mid) <= days:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

Solution — java

```
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int lo = 0, hi = 0;
        for (int w : weights) { lo = Math.max(lo, w); hi += w; }
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (daysNeeded(weights, mid) <= days) hi = mid;
            else                                  lo = mid + 1;
        }
        return lo;
    }
    private int daysNeeded(int[] weights, int cap) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d;
    }
}
```

WALKTHROUGH

SETTING `LO = MAX(WEIGHTS)` IS WHAT REMOVES THE IMPOSSIBLE-ITEM GUARD.

Because the search never considers a capacity below the largest package, the greedy loop can never encounter an item that does not fit on an empty day. Had `lo` started at 1, `daysNeeded` would loop forever adding a package that overflows even an empty ship — which is precisely the guard mentioned in the pattern section.

WHY THE GREEDY CHECK IS OPTIMAL.

Packages must ship in order, so at any point the only decision is whether to add the next package to today or start tomorrow. Adding it whenever it fits can never increase the day count: postponing a package that fits only shifts work later without freeing capacity. So the greedy count *is* the minimum number of days for that capacity.

THIS IS THE SAME PROBLEM AS "SPLIT ARRAY LARGEST SUM" AND "ALLOCATE MINIMUM PAGES"

, reworded. Recognising that family is worth more than memorising any one of them.

Split array largest sum `Hard`

Pattern: Minimise the maximum

Given an array `nums` and an integer `k`, split the array into `k` non-empty **contiguous** subarrays so as to **minimise the largest subarray sum**. Return that minimised largest sum.

Input	Output	Note
`nums = [7,2,5,10,8], k = 2`	`18`	Split as [7,2,5] and [10,8]: sums 14 and 18.
`nums = [1,2,3,4,5], k = 2`	`9`	[1,2,3] and [4,5].
`nums = [1,4,4], k = 3`	`4`	

HINTS

1. The phrase "minimise the maximum" is the signal. Do not try to construct the split directly.
2. Guess a value X for the largest allowed subarray sum. Can the array be split into at most k parts each summing to ≤ X? That is a greedy sweep.
3. If X works, every larger X works too — monotonic.
4. Bounds: `lo = max(nums)` (a single element must fit in some part), `hi = sum(nums)` (one part holds everything).

Approach	Idea	Time	Space
DP over splits	`dp[i][j]` = best split of the first `i` elements into `j` parts. Correct and a perfectly good answer.	`O(n²·k)`	`O(n·k)`
Binary search the answer	Search the value of the largest sum; feasibility is a greedy part count.	`O(n log sum)`	`O(1)`

Solution — js

```
function splitArray(nums, k) {
  // how many parts are needed if no part may exceed `limit`?
  const partsNeeded = (limit) => {
    let parts = 1, sum = 0;
    for (const x of nums) {
      if (sum + x > limit) { parts++; sum = 0; }
      sum += x;
    }
    return parts;
  };

  let lo = Math.max(...nums);              // a single element must fit
  let hi = nums.reduce((a, b) => a + b, 0); // one part holds everything

  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (partsNeeded(mid) <= k) hi = mid;   // feasible: try smaller
    else                       lo = mid + 1;
  }
  return lo;
}
```

Solution — cpp

```
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        auto partsNeeded = [&](long limit) {
            int parts = 1; long sum = 0;
            for (int x : nums) {
                if (sum + x > limit) { ++parts; sum = 0; }
                sum += x;
            }
            return parts;
        };
        long lo = *max_element(nums.begin(), nums.end());
        long hi = accumulate(nums.begin(), nums.end(), 0L);
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            if (partsNeeded(mid) <= k) hi = mid;
            else                       lo = mid + 1;
        }
        return (int)lo;
    }
};
```

Solution — python

```
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        def parts_needed(limit: int) -> int:
            parts, s = 1, 0
            for x in nums:
                if s + x > limit:
                    parts += 1
                    s = 0
                s += x
            return parts

        lo, hi = max(nums), sum(nums)
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if parts_needed(mid) <= k:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

Solution — java

```
class Solution {
    public int splitArray(int[] nums, int k) {
        long lo = 0, hi = 0;
        for (int x : nums) { lo = Math.max(lo, x); hi += x; }
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            if (partsNeeded(nums, mid) <= k) hi = mid;
            else                             lo = mid + 1;
        }
        return (int) lo;
    }
    private int partsNeeded(int[] nums, long limit) {
        int parts = 1; long sum = 0;
        for (int x : nums) {
            if (sum + x > limit) { parts++; sum = 0; }
            sum += x;
        }
        return parts;
    }
}
```

WALKTHROUGH

THIS IS THE ARCHETYPE OF THE PATTERN.

Constructing the optimal split directly is a two-dimensional DP; inverting the question — "given a ceiling, how many parts do I need?" — turns it into a one-line greedy plus a binary search, and drops the complexity from $O(n^2k)$ to $O(n \log \text{sum})$.

WHY `PARTSNEEDED ≤ K` RATHER THAN `=== K`.

If a limit allows the array to be split into fewer than `k` parts, you can always split one part further to reach exactly `k` — the largest sum only decreases. So "needs at most k parts" is the correct feasibility test, and testing for equality would wrongly reject valid limits. The test `[1,4,4], k=3` exercises this: at limit 4 the greedy uses exactly 3, but at limit 5 it uses 2, which is still feasible for k = 3.

NOTE THE IDENTICAL SHAPE TO THE SHIPPING PROBLEM ABOVE.

Same greedy, same bounds, different words. That is the point of learning patterns rather than problems.

Sorting: the algorithms and custom comparators

You will rarely implement a sort in an interview, but you will constantly *use* one — and you will be asked how merge sort and quicksort work, why one is stable, and when sorting is worth its log factor.

The algorithms worth knowing

Algorithm	Best	Average	Worst	Space	Stable
Merge sort	n log n	n log n	n log n	O(n)	**Yes**
Quicksort	n log n	n log n	**n^2**	O(log n)	No
Heapsort	n log n	n log n	n log n	**O(1)**	No
Insertion sort	**n**	n^2	n^2	O(1)	Yes
Counting sort	n + k	n + k	n + k	O(k)	Yes
Radix sort	d(n + k)	d(n + k)	d(n + k)	O(n + k)	Yes

```
MERGE SORT — divide and conquer, always n log n
  split in half, sort each half, MERGE the two sorted
  halves with two pointers (chapter 17's merge).
  T(n) = 2T(n/2) + O(n)  ->  O(n log n)

  + STABLE, predictable, parallelises, works on linked
    lists with O(1) extra space
  - needs O(n) scratch space for arrays

QUICKSORT — partition around a pivot
  choose a pivot, move smaller elements left and larger
  right, recurse on both sides.
  Average O(n log n); WORST O(n^2) when the pivot is
  always extreme (a sorted array with a first-element
  pivot).

  + in place, excellent cache behaviour, fastest in
    practice
  - unstable, and the worst case is real
  FIXES: random pivot, median-of-three, or introsort
  (switch to heapsort once recursion gets too deep) —
  which is what std::sort actually does.

WHAT YOUR LANGUAGE ACTUALLY USES:
  C++ std::sort        introsort — NOT stable
  C++ stable_sort      merge sort
  Java primitives      dual-pivot quicksort — not stable
  Java objects         TimSort — STABLE
  Python sorted()      TimSort — STABLE
  JavaScript .sort()   STABLE since ES2019

STABILITY MEANS equal elements keep their original
relative order. It matters whenever you sort by one key
and then by another: sort by the SECONDARY key first,
then STABLY by the primary, and the secondary order
survives inside each group.

NON-COMPARISON SORTS beat n log n by not comparing:
  COUNTING SORT  O(n + k) for integers in a small range k
  RADIX SORT     O(d(n + k)) digit by digit
  BUCKET SORT    O(n) for uniformly distributed values
The n log n lower bound applies only to COMPARISON sorts,
which is a good thing to say when someone asks whether
you can beat it.
```

Comparators — where the bugs are

```
/* --- THE JAVASCRIPT TRAP, first of all --- */
[10, 9, 1].sort()               // ["1","10","9"] LEXICOGRAPHIC
[10, 9, 1].sort((a, b) => a - b) // [1, 9, 10]

/* THE CONTRACT: return NEGATIVE if a comes first, POSITIVE
   if b comes first, ZERO if equivalent.                   */

/* --- MULTI-KEY: chain with || --- */
people.sort((a, b) =>
      a.age - b.age            // primary:   age ascending
   || b.score - a.score        // secondary: score DESCENDING
   || a.name.localeCompare(b.name));   // tertiary: name

/* `||` works because 0 is falsy — a tie on one key falls
   through to the next.

/* --- STRINGS --- */
words.sort((a, b) => a.localeCompare(b));   // locale-aware
words.sort((a, b) => (a < b ? -1 : a > b ? 1 : 0));  // by code unit

/* --- THE SUBTRACTION TRAP --- */
    a - b OVERFLOWS in C++/Java when the values are far
    apart. Use explicit comparison instead:
        Integer.compare(a, b)          // Java
        return a < b ? -1 : (a > b);   // C++ — or just a < b
    In JavaScript doubles do not overflow, but the habit is
    worth keeping.

/* --- SORTING PAIRS AND KEEPING INDICES --- */
    const idx = nums.map((v, i) => [v, i])
                    .sort((a, b) => a[0] - b[0]);
    // now idx[j] = [value, ORIGINAL index]
    // this is how you sort when the answer needs the
    // original positions (chapter 13's Two Sum problem)

/* --- SORT BY A COMPUTED KEY, computed ONCE --- */
    // BAD: recomputes the key O(n log n) times
    words.sort((a, b) => score(a) - score(b));
    // GOOD: Schwartzian transform — compute n times
    words.map(w => [score(w), w])
         .sort((x, y) => x[0] - y[0])
         .map(p => p[1]);

/* --- A COMPARATOR MUST BE CONSISTENT --- */
    It must be transitive and antisymmetric. An inconsistent
    comparator (for instance one using a random tiebreak, or
    `a.x > b.x ? 1 : -1` with no zero case) can crash
    Java's TimSort with "Comparison method violates its
    general contract".                                    */
```

Problems

How it shows up

Symptom	Cause
`[1, 10, 9]` from a numeric sort	No comparator in JavaScript — lexicographic default

Symptom	Cause
Secondary ordering lost	Sort is not stable, or the keys were applied in the wrong order
Negative numbers sorted wrongly	`a - b` overflowed in C++/Java
"Comparison method violates its general contract"	Comparator not transitive or missing the equal case
Quicksort times out	Adversarial input with a fixed pivot — randomise
Sorting destroys the answer	The problem wanted original indices — pair them first
Sort is slower than expected	Expensive key recomputed inside the comparator
Custom sort of objects loses ties randomly	Add an explicit tiebreak to make it deterministic

Check yourself

- ✓ Give the time, space and stability of merge sort, quicksort and heapsort.
- ✓ State quicksort's worst case and three ways to avoid it.
- ✓ Define stability and explain the sort-secondary-key-first technique.
- ✓ Say which sort your language uses and whether it is stable.
- ✓ Name three sorts that beat n log n and the condition each needs.
- ✓ Write a three-key comparator using `||`.
- ✓ Explain the subtraction overflow trap.
- ✓ Describe how to sort while preserving original indices.

Problems

Implement merge sort `Medium`

Pattern: Divide and conquer

Sort an integer array in ascending order. Implement **merge sort** yourself — do not call the built-in sort.

Your solution must run in O(n log n) and be stable.

Input	Output	Note
`nums = [5,2,3,1]`	`[1,2,3,5]`	
`nums = [5,1,1,2,0,0]`	`[0,0,1,1,2,5]`	

HINTS

1. Split the array in half, sort each half recursively, then merge the two sorted halves.
2. The merge is the two-pointer walk from chapter 17: repeatedly take the smaller front element.
3. Base case: an array of length 0 or 1 is already sorted.
4. Use `<=` when choosing from the left half so equal elements keep their order — that is what makes it stable.

Approach	Idea	Time	Space
Merge sort	Halve, sort both sides, merge. Always O(n log n) regardless of input.	`O(n log n)`	`O(n)`
Quicksort	Partition around a pivot. Faster in practice and O(1) extra space, but $O(n^2)$ worst case and not stable.	`O(n log n) avg`	`O(log n)`

Solution — js

```
function sortArray(nums) {
  if (nums.length <= 1) return nums;

  const mid = nums.length >> 1;
  const left  = sortArray(nums.slice(0, mid));
  const right = sortArray(nums.slice(mid));

  // merge two sorted halves
  const out = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    // <= keeps equal elements in their original order: STABLE
    if (left[i] <= right[j]) out.push(left[i++]);
    else                     out.push(right[j++]);
  }
  while (i < left.length)  out.push(left[i++]);
  while (j < right.length) out.push(right[j++]);
  return out;
}
```

Solution — cpp

```
class Solution {
    void mergeSort(vector<int>& a, int lo, int hi, vector<int>& buf) {
        if (hi - lo <= 1) return;
        int mid = lo + (hi - lo) / 2;
        mergeSort(a, lo, mid, buf);
        mergeSort(a, mid, hi, buf);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi)
            buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i < mid) buf[k++] = a[i++];
        while (j < hi)  buf[k++] = a[j++];
        copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
    }
public:
    vector<int> sortArray(vector<int>& nums) {
        vector<int> buf(nums.size());
        mergeSort(nums, 0, nums.size(), buf);
        return nums;
    }
};
```

Solution — python

```python
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:
        if len(nums) <= 1:
            return nums
        mid = len(nums) // 2
        left = self.sortArray(nums[:mid])
        right = self.sortArray(nums[mid:])

        out = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                out.append(left[i]); i += 1
            else:
                out.append(right[j]); j += 1
        out.extend(left[i:])
        out.extend(right[j:])
        return out
```

Solution — java

```java
class Solution {
    public int[] sortArray(int[] nums) {
        int[] buf = new int[nums.length];
        sort(nums, 0, nums.length, buf);
        return nums;
    }
    private void sort(int[] a, int lo, int hi, int[] buf) {
        if (hi - lo <= 1) return;
        int mid = lo + (hi - lo) / 2;
        sort(a, lo, mid, buf);
        sort(a, mid, hi, buf);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi) buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
        while (i < mid) buf[k++] = a[i++];
        while (j < hi)  buf[k++] = a[j++];
        System.arraycopy(buf, lo, a, lo, hi - lo);
    }
}
```

WALKTHROUGH

WHY THE COMPLEXITY IS EXACTLY N LOG N, ALWAYS.

The recursion halves the input, so the tree is `log₂ n` levels deep, and the merges at each level together touch every element once — O(n) per level. Unlike quicksort there is no pivot to choose badly, so the bound holds for every input including already-sorted and reverse-sorted ones. The test cases include both.

`<=` VERSUS `<` IS THE STABILITY SWITCH.

When the two fronts are equal, taking from the *left* half preserves the original relative order, since the left half came first in the input. Changing it to `<` still sorts correctly but silently loses stability — which breaks any multi-key sort built on top of it.

THE C++ AND JAVA VERSIONS SORT IN PLACE WITH ONE SHARED BUFFER

rather than allocating at every level, which is what a real implementation does. The JavaScript version uses `slice` for clarity; mention the difference if asked about memory.

Sort colors (Dutch national flag) `Medium`

Pattern: Three-way partition

Given an array `nums` containing only `0`, `1` and `2`, sort it **in place** so that same-valued elements are adjacent and in the order 0, 1, 2.

Do it in **one pass** with O(1) extra space, without using a library sort.

Input	Output	Note
`nums = [2,0,2,1,1,0]`	`[0,0,1,1,2,2]`	
`nums = [2,0,1]`	`[0,1,2]`	

HINTS

1. Counting sort in two passes is easy — count the 0s, 1s and 2s, then overwrite. Try for one pass.
2. Maintain three regions: everything before `low` is 0, everything after `high` is 2, and `i` scans the unknown middle.
3. When you swap a 2 down from the front, do
   NOT
   advance `i` — the value you just received is unexamined.
4. When you swap a 0 up, you may advance, because the value coming back is guaranteed to be a 1 already processed.

Approach	Idea	Time	Space
Library sort	Correct but ignores the structure and is O(n log n).	`O(n log n)`	`O(1)`
Counting sort, two passes	Count each value, then rewrite the array. Simple and O(n) — a good answer to state first.	`O(n)`	`O(1)`
Dutch national flag, one pass	Three pointers partition the array into 0s, 1s, unknown, 2s.	`O(n)`	`O(1)`

Solution — js

```javascript
function sortColors(nums) {
  let low = 0;                    // everything before low is 0
  let i = 0;                      // scanner
  let high = nums.length - 1;     // everything after high is 2

  while (i <= high) {
    if (nums[i] === 0) {
      [nums[low], nums[i]] = [nums[i], nums[low]];
      low++; i++;                 // safe to advance: the value
                                  // swapped back is a known 1
    } else if (nums[i] === 2) {
      [nums[high], nums[i]] = [nums[i], nums[high]];
      high--;                     // do NOT advance i — the new
                                  // nums[i] is unexamined
    } else {
      i++;                        // a 1 is already in place
    }
  }
}
```

Solution — cpp

```cpp
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int low = 0, i = 0, high = (int)nums.size() - 1;
        while (i <= high) {
            if (nums[i] == 0)      swap(nums[low++], nums[i++]);
            else if (nums[i] == 2) swap(nums[high--], nums[i]);
            else                   ++i;
        }
    }
};
```

Solution — python

```python
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        low, i, high = 0, 0, len(nums) - 1
        while i <= high:
            if nums[i] == 0:
                nums[low], nums[i] = nums[i], nums[low]
                low += 1
                i += 1
            elif nums[i] == 2:
                nums[high], nums[i] = nums[i], nums[high]
                high -= 1
            else:
                i += 1
```

Solution — java

```
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, i = 0, high = nums.length - 1;
        while (i <= high) {
            if (nums[i] == 0) {
                int t = nums[low]; nums[low] = nums[i]; nums[i] = t;
                low++; i++;
            } else if (nums[i] == 2) {
                int t = nums[high]; nums[high] = nums[i]; nums[i] = t;
                high--;
            } else i++;
        }
    }
}
```

WALKTHROUGH

THE INVARIANT, WHICH IS WHAT MAKES THE ASYMMETRY CORRECT:

at all times `nums[0..low-1]` are 0s, `nums[low..i-1]` are 1s, `nums[i..high]` are unexamined, and `nums[high+1..]` are 2s.

When `nums[i]` is 0, it is swapped with `nums[low]`. But `low ≤ i` and everything in `[low, i)` is a 1, so the value coming back is either a 1 (already classified) or the same element when `low === i`. Either way `i` may advance.

When `nums[i]` is 2, it is swapped with `nums[high]`, which lies in the *unexamined* region. The value arriving at `i` has never been looked at, so advancing `i` would skip it. `[2,0,1]` is the smallest test that catches this: advancing would leave the 0 unprocessed and produce `[0,1,2]` only by luck — try `[2,1,0,2,1,0]` to see it fail properly.

THE LOOP CONDITION IS `I <= HIGH`, NOT `I < HIGH`

, because `high` itself is unexamined until the moment it is passed.

Largest number `Medium`

Pattern: Custom comparator

Given a list of non-negative integers `nums`, arrange them so that they form the **largest possible number**, and return it as a string.

Input	Output	Note
`nums = [10,2]`	`"210"`	
`nums = [3,30,34,5,9]`	`"9534330"`	
`nums = [0,0]`	`"0"`	Not "00".

HINTS

1. Sorting by numeric value is wrong: 3 should come before 30, but 30 > 3.
2. Sorting by string value is also wrong: "9" vs "34" happens to work, but "3" vs "30" does not.
3. Compare two candidates by which *concatenation* is larger: put `a` first if `a + b > b + a` as strings.
4. After sorting, if the first character is `"0"` the whole array was zeros — return `"0"`.

Approach	Idea	Time	Space
Try every permutation	Generate all orderings and take the largest.	`O(n!)`	`O(n)`
Concatenation comparator	Sort with `(a,b) => (b+a).localeCompare(a+b)`, then join.	`O(n log n · L)`	`O(n)`

Solution — js

```
function largestNumber(nums) {
  const s = nums.map(String);

  // a should come first if a+b is the larger concatenation
  s.sort((a, b) => (b + a).localeCompare(a + b));

  // all zeros -> "0", not "000"
  if (s[0] === "0") return "0";

  return s.join("");
}
```

Solution — cpp

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> s;
        for (int x : nums) s.push_back(to_string(x));
        sort(s.begin(), s.end(), [](const string& a, const string& b) {
            return a + b > b + a;      // a first if a+b is larger
        });
        if (s[0] == "0") return "0";
        string out;
        for (auto& x : s) out += x;
        return out;
    }
};
```

Solution — python

```
from functools import cmp_to_key

class Solution:
    def largestNumber(self, nums: list[int]) -> str:
        s = list(map(str, nums))
        s.sort(key=cmp_to_key(lambda a, b: (a + b < b + a) - (a + b > b + a)))
        if s[0] == "0":
            return "0"
        return "".join(s)
```

Solution — java

```
class Solution {
    public String largestNumber(int[] nums) {
        String[] s = new String[nums.length];
        for (int i = 0; i < nums.length; i++) s[i] = String.valueOf(nums[i]);
        Arrays.sort(s, (a, b) -> (b + a).compareTo(a + b));
        if (s[0].equals("0")) return "0";
        StringBuilder sb = new StringBuilder();
        for (String x : s) sb.append(x);
        return sb.toString();
    }
}
```

WALKTHROUGH

WHY THE CONCATENATION COMPARATOR IS CORRECT.

For two adjacent items the only question is which order produces a larger result, and since both orderings have the same total length, comparing `a+b` against `b+a` as strings answers it exactly. The non-obvious part is that this *local* rule produces a globally optimal arrangement — it does, because the comparator is a valid total order (it is transitive), so sorting by it yields the unique maximum.

THE COMPARATOR DIRECTION.

`(b + a).localeCompare(a + b)` returns negative when `b+a < a+b`, i.e. when putting `a` first is better — and negative means "a comes first". Getting this backwards produces the smallest number instead, which the `[3,30,34,5,9]` test catches immediately.

THE ALL-ZEROS GUARD IS NOT COSMETIC.

`[0,0]` sorts to `["0","0"]` and joins to `"00"`, which is not a valid number. Checking only the first character suffices: if the largest arrangement starts with 0, every element must be 0.

Sorting: interval patterns

Meetings, bookings, ranges. Almost every interval problem starts with one decision — sort by start or sort by end — and that decision determines whether the rest is three lines or impossible.

The pattern

```
TWO INTERVALS [a1, a2] AND [b1, b2] OVERLAP when

     a1 <= b2  AND  b1 <= a2

equivalently, they do NOT overlap when one ends entirely
before the other starts. Get this predicate right once
and write it down.

-----------------------------------------------------------

THE DECISION THAT DETERMINES EVERYTHING:

SORT BY START  -> MERGING, and "can I attend all?"
     Walk left to right; each interval either extends the
     current block or begins a new one.

SORT BY END    -> MAXIMISING THE COUNT of
                  non-overlapping intervals
     The classic activity-selection greedy: always keep
     the interval that FINISHES EARLIEST, because it
     leaves the most room for everything after it.

SWEEP LINE     -> "how many are active at once?"
     Turn each interval into two EVENTS,
     (start, +1) and (end, -1), sort all events by time,
     and scan keeping a running counter. The maximum
     counter value is the answer.

-----------------------------------------------------------

MERGE, after sorting by start:

  [1,3] [2,6] [8,10] [15,18]
    |-----|              sorted by start
     overlap -> merge to [1,6]
                |---|    8 > 6, new block
                     |---|

  out = [];
  for (const [s, e] of sorted) {
      const last = out[out.length - 1];
      if (last && s <= last[1]) last[1] = Math.max(last[1], e);
      else out.push([s, e]);
  }

  THE Math.max IS NOT OPTIONAL. [1,10] followed by [2,3]
  must stay [1,10]; without it the end shrinks to 3.

-----------------------------------------------------------

THE TOUCHING QUESTION: do [1,2] and [2,3] overlap?
IT DEPENDS ON THE PROBLEM. Merging usually says yes
(use <=); meeting rooms usually says no, because a
meeting ending at 2 frees the room for one starting at 2
(use <). ASK, or state your assumption.
```

When to reach for it

Merge overlapping ranges

Sort by start, extend or append.

Minimum removals to make disjoint

Sort by end, greedily keep the earliest finisher.

Minimum rooms / max concurrent

Sweep line, or a min-heap of end times.

Insert into a sorted set of intervals

Three phases: before, overlapping (merge), after.

Free time / gaps

Merge first, then read the gaps between blocks.

Minimum arrows / points hitting all

Sort by end — the same greedy as activity selection.

```
/* --- SWEEP LINE, the general tool --- */
const events = [];
for (const [s, e] of intervals) {
    events.push([s, +1]);
    events.push([e, -1]);
}
// tie-break: process ENDS before STARTS when touching
// intervals do NOT count as overlapping
events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);
//                                  ^ -1 sorts before +1

let cur = 0, best = 0;
for (const [, delta] of events) {
    cur += delta;
    best = Math.max(best, cur);
}

/* THE TIE-BREAK IS THE WHOLE SUBTLETY. With events at the
   same time, processing the -1 first means a meeting ending
   at 10 releases the room before one starting at 10 claims
   it. Flip the tie-break and you need an extra room.     */
```

Problems

How it shows up

Symptom	Cause
Merged interval ends too early	Assigned `e` instead of `max(last[1], e)`

Symptom	Cause
Off by one on touching intervals	`<` vs `<=` — decide and state it
Greedy keeps too few intervals	Sorted by start instead of by end
Room count one too high	Sweep tie-break processes starts before ends
Wrong answer on unsorted input	Forgot to sort at all
Sort scrambles the intervals	Default JavaScript sort on arrays — pass a comparator
Crash on empty input	`out[out.length-1]` on an empty result
Mutating the input causes odd bugs	`last[1] = …` writes into the caller's array — copy first

Check yourself

- ✓ Write the overlap predicate for two intervals.
- ✓ Say when to sort by start and when by end, with a problem for each.
- ✓ Explain why "earliest finishing time" is the right greedy.
- ✓ Write the merge loop and explain the `Math.max`.
- ✓ Describe the sweep line and its tie-break rule.
- ✓ Say what the tie-break controls, concretely.
- ✓ Explain the touching-intervals ambiguity and how to resolve it.
- ✓ Give the three phases of inserting into a sorted interval list.

Problems

Merge intervals `Medium`

Pattern: Sort by start, extend or append

Given an array of intervals `[start, end]`, merge all overlapping intervals and return the non-overlapping intervals that cover all the input.

Touching intervals such as `[1,4]` and `[4,5]` **do** merge here.

Input	Output	Note
`intervals = [[1,3],[2,6],[8,10],[15,18]]`	`[[1,6],[8,10],[15,18]]`	
`intervals = [[1,4],[4,5]]`	`[[1,5]]`	

HINTS

1. Sort by start time first. Then any interval can only overlap the block you are currently building.
2. If the new start is at most the current block's end, they overlap — extend the block.
3. Extend to `max(currentEnd, newEnd)`, not to `newEnd`: a fully contained interval must not shrink the block.

Approach	Idea	Time	Space
Repeated pairwise merging	Scan for any overlapping pair, merge, repeat until stable.	`O(n²)`	`O(n)`
Sort by start, one pass	After sorting, one linear sweep suffices because overlaps can only be with the immediately preceding block.	`O(n log n)`	`O(n)`

Solution — js

```
function merge(intervals) {
  if (intervals.length === 0) return [];

  // copy before sorting so the caller's array is untouched
  const sorted = intervals.slice().sort((a, b) => a[0] - b[0]);
  const out = [sorted[0].slice()];

  for (let i = 1; i < sorted.length; i++) {
    const [s, e] = sorted[i];
    const last = out[out.length - 1];
    if (s <= last[1]) {
      // overlap - EXTEND, never shrink
      last[1] = Math.max(last[1], e);
    } else {
      out.push([s, e]);
    }
  }
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        if (intervals.empty()) return {};
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> out{intervals[0]};
        for (size_t i = 1; i < intervals.size(); ++i) {
            if (intervals[i][0] <= out.back()[1])
                out.back()[1] = max(out.back()[1], intervals[i][1]);
            else
                out.push_back(intervals[i]);
        }
        return out;
    }
};
```

Solution — python

```
class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        if not intervals:
            return []
        out = []
        for s, e in sorted(intervals):
            if out and s <= out[-1][1]:
                out[-1][1] = max(out[-1][1], e)
            else:
                out.append([s, e])
        return out
```

Solution — java

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length == 0) return new int[0][];
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> out = new ArrayList<>();
        out.add(intervals[0].clone());
        for (int i = 1; i < intervals.length; i++) {
            int[] last = out.get(out.size() - 1);
            if (intervals[i][0] <= last[1]) last[1] = Math.max(last[1], intervals[i][1]);
            else out.add(intervals[i].clone());
        }
        return out.toArray(new int[0][]);
    }
}
```

WALKTHROUGH

WHY SORTING BY START MAKES ONE PASS SUFFICIENT.

After sorting, every interval starts at or after the current block's start. So if it overlaps *anything* already emitted, it must overlap the most recent block — earlier blocks ended before that one began. That reduces an all-pairs question to a single comparison per element.

`[[1,10],[2,3]]` IS THE TEST THAT MATTERS.

The second interval is entirely inside the first. Writing `last[1] = e` would shrink the block to `[1,3]` and lose coverage. The `Math.max` is what makes containment work.

NOTE THE `.SLICE()` CALLS.

Sorting mutates the input array and writing `last[1]` would mutate the caller's interval objects. Copying is cheap and avoids a class of bug that only shows up when the caller reuses the data.

Non-overlapping intervals `Medium`

Pattern: Sort by END — activity selection

Given an array of intervals, return the **minimum number you must remove** so that the rest are non-overlapping.

Intervals that merely touch, such as `[1,2]` and `[2,3]`, are **not** considered overlapping.

Input	Output	Note
`intervals = [[1,2],[2,3],[3,4],[1,3]]`	`1`	Remove [1,3].
`intervals = [[1,2],[1,2],[1,2]]`	`2`	
`intervals = [[1,2],[2,3]]`	`0`	

HINTS

1. Removing the fewest is the same as *keeping* the most — the classic activity-selection problem.
2. Sort by
   END
   time, not start. Greedily keep an interval whenever it starts at or after the last kept end.
3. Intuition: finishing earliest leaves the maximum room for everything that follows.
4. The answer is `total - kept`.

Approach	Idea	Time	Space
Try every subset	Check all subsets for disjointness.	`O(2^n)`	`O(n)`
DP on sorted intervals	Longest non-overlapping chain by binary searching the previous compatible interval.	`O(n log n)`	`O(n)`
Greedy by earliest end	Sort by end and keep greedily. Provably optimal by an exchange argument.	`O(n log n)`	`O(1)`

Solution — js

```
function eraseOverlapIntervals(intervals) {
  if (intervals.length === 0) return 0;

  // sort by END time — the heart of the greedy
  const sorted = intervals.slice().sort((a, b) => a[1] - b[1]);

  let kept = 0, lastEnd = -Infinity;
  for (const [s, e] of sorted) {
    if (s >= lastEnd) {      // >= : touching is allowed
      kept++;
      lastEnd = e;
    }
  }
  return intervals.length - kept;
}
```

Solution — cpp

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty()) return 0;
        sort(intervals.begin(), intervals.end(),
             [](const vector<int>& a, const vector<int>& b) {
                 return a[1] < b[1];
             });
        int kept = 0; long lastEnd = LONG_MIN;
        for (auto& iv : intervals)
            if (iv[0] >= lastEnd) { ++kept; lastEnd = iv[1]; }
        return (int)intervals.size() - kept;
    }
};
```

Solution — python

```python
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        if not intervals:
            return 0
        kept = 0
        last_end = float("-inf")
        for s, e in sorted(intervals, key=lambda iv: iv[1]):
            if s >= last_end:
                kept += 1
                last_end = e
        return len(intervals) - kept
```

Solution — java

```java
class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        if (intervals.length == 0) return 0;
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
        int kept = 0;
        long lastEnd = Long.MIN_VALUE;
        for (int[] iv : intervals)
            if (iv[0] >= lastEnd) { kept++; lastEnd = iv[1]; }
        return intervals.length - kept;
    }
}
```

WALKTHROUGH

THE EXCHANGE ARGUMENT — BE READY TO GIVE THIS.

Suppose an optimal solution keeps some interval `X` as the first of its chain, and let `G` be the interval with the earliest end time overall. Since `G` ends no later than `X`, replacing `X` with `G` cannot conflict with anything the optimal solution kept after `X`. So there is an optimal solution that starts with the earliest finisher, and by induction the greedy is optimal.

SORTING BY START IS THE NATURAL INSTINCT AND IT IS WRONG.

On `[[1,100],[11,22],[1,11],[2,12]]`, start-order would consider `[1,100]` first and block everything else, keeping 1 and removing 3. End-order keeps `[1,11]` and `[11,22]`, removing only 2 — which is the answer.

`S >= LASTEND` ENCODES "TOUCHING IS FINE".

If the problem said touching counts as overlapping, this becomes `s > lastEnd`. That single character is the entire difference, which is why the pattern section insists on settling the question before writing code.

Meeting rooms II `Medium`

Pattern: Sweep line / min-heap of ends

Given an array of meeting time intervals `[start, end]`, return the **minimum number of conference rooms** required so that no two meetings in the same room overlap.

A meeting ending at time `t` frees the room for a meeting starting at `t`.

Input	Output	Note
`intervals = [[0,30],[5,10],[15,20]]`	`2`	

Input	Output	Note
`intervals = [[7,10],[2,4]]`	`1`	
`intervals = [[1,5],[2,6],[3,7]]`	`3`	

HINTS

1. The answer is the maximum number of meetings simultaneously in progress.
2. Turn each meeting into two events: a start (+1 room) and an end (−1 room). Sort all events by time and sweep.
3. When a start and an end share the same time, process the **END FIRST** — the room is freed before the next meeting claims it.
4. Equivalently: sort the start times and end times into two separate arrays and walk them with two pointers.

Approach	Idea	Time	Space
Count overlaps per meeting	For each meeting, count how many others overlap it.	`O(n²)`	`O(1)`
Min-heap of end times	Sort by start; keep a heap of the end times of active meetings, popping those that have finished. Heap size is the room count.	`O(n log n)`	`O(n)`
Sweep line on events	Two events per meeting, sorted, one linear scan of a counter.	`O(n log n)`	`O(n)`

Solution — js

```
function minMeetingRooms(intervals) {
  const events = [];
  for (const [s, e] of intervals) {
    events.push([s, +1]);   // a meeting starts: need a room
    events.push([e, -1]);   // a meeting ends: free a room
  }

  // same time -> process the END (-1) before the START (+1)
  events.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

  let cur = 0, best = 0;
  for (const [, delta] of events) {
    cur += delta;
    best = Math.max(best, cur);
  }
  return best;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        vector<pair<int,int>> events;
        for (auto& iv : intervals) {
            events.push_back({iv[0], +1});
            events.push_back({iv[1], -1});
        }
        sort(events.begin(), events.end());   // -1 sorts before +1
        int cur = 0, best = 0;
        for (auto& [t, d] : events) { cur += d; best = max(best, cur); }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        events = []
        for s, e in intervals:
            events.append((s, 1))
            events.append((e, -1))
        events.sort()          # (t, -1) sorts before (t, +1)
        cur = best = 0
        for _, d in events:
            cur += d
            best = max(best, cur)
        return best
```

Solution — java

```java
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        int[][] events = new int[intervals.length * 2][2];
        int k = 0;
        for (int[] iv : intervals) {
            events[k++] = new int[]{iv[0], 1};
            events[k++] = new int[]{iv[1], -1};
        }
        Arrays.sort(events, (a, b) ->
            a[0] != b[0] ? Integer.compare(a[0], b[0])
                         : Integer.compare(a[1], b[1]));
        int cur = 0, best = 0;
        for (int[] e : events) { cur += e[1]; best = Math.max(best, cur); }
        return best;
    }
}
```

WALKTHROUGH

WHY THE MAXIMUM CONCURRENCY IS THE ANSWER.

If at some instant `k` meetings are in progress they need `k` distinct rooms, so the answer is at least the peak. And it is achievable: process meetings in start order assigning any free room — a room is unavailable only if its meeting is still running, so you never need more than the peak. Hence peak concurrency is exactly the minimum.

THE TIE-BREAK IS THE ENTIRE SUBTLETY.

On `[[1,2],[2,3],[3,4]]` the answer is 1, because each meeting ends exactly as the next begins. Sorting `-1` before `+1` at equal times produces the sequence +1, −1, +1, −1, +1, −1 and a peak of 1. Reverse the tie-break and you get +1, +1, −1 … and a peak of 2. Note the comparator `a[1] - b[1]` achieves this for free, since `-1 < +1`.

THE HEAP ALTERNATIVE IS WORTH KNOWING

because it generalises: if you also need to know *which* room each meeting goes in, a min-heap of (end time, room id) gives you that, and the sweep does not.

Hashing: maps, sets and frequency

Trade memory for time. A hash map turns "have I seen this?" from an O(n) scan into an O(1) lookup, and that single substitution converts more $O(n^2)$ solutions into O(n) ones than any other technique.

The pattern

```
THE CORE MOVE: whenever an inner loop is SEARCHING for
something, ask whether a hash map could have remembered
it instead.

  for i in 0..n:
    for j in 0..n:            <-- this loop is a LOOKUP
      if a[j] == something(i)

  becomes

  seen = {}
  for i in 0..n:
    if something(i) in seen:  <-- O(1)
    seen[a[i]] = i

  O(n^2)  ->  O(n) time, O(n) space.

THE FOUR USES, and nearly every hashing problem is one
of them:

1. MEMBERSHIP - "have I seen x?"            Set
     duplicates, intersections, cycle detection

2. COUNTING - "how many times did x occur?"  Map -> int
     anagrams, majority element, frequency ranking

3. INDEXING - "where did x occur?"           Map -> index
     two sum, longest subarray with a property

4. GROUPING - "which items share a key?"     Map -> list
     group anagrams, bucket by some signature

THE KEY DESIGN DECISION IS THE KEY ITSELF. Most hashing
problems reduce to: WHAT IS THE RIGHT SIGNATURE?

  group anagrams        -> sorted letters, or a
                           26-length count vector
  find duplicates       -> the value itself
  subarray sums         -> the running prefix sum
  isomorphic strings    -> a pair of forward/backward maps
  board validation      -> "row3-val7", "col2-val7"

CHOOSING A GOOD KEY IS THE WHOLE PROBLEM. The map itself
is never the hard part.

THE COSTS:
  lookup/insert/delete   O(1) AVERAGE, O(n) worst case
  space                  O(n)
  ordering               NONE - if you need sorted keys,
                         you want a balanced BST or a
                         sort, not a hash map
```

Choosing the structure

Set

Membership only. `has`, `add`, `delete`. Use when you never need counts or positions.

Map → count

Frequency. The single most common shape. Remember to delete keys that reach zero if you rely on `size`.

Map → index

When the answer is a position, not a value. Two Sum returns indices, so it cannot sort.

Map → array

Grouping. Push into the bucket for a computed key.

Fixed array instead of a map

If keys are small integers or lowercase letters, `new Array(26).fill(0)` is faster and simpler than a Map.

Not a hash map

Need order, ranks, or "next larger key"? Use sorting, a heap, or a BST — hashing has no order.

```
/* --- FREQUENCY, the idiom to have automatic --- */
const freq = new Map();
for (const x of arr) freq.set(x, (freq.get(x) ?? 0) + 1);

// letters only? an array is better:
const cnt = new Array(26).fill(0);
for (const ch of s) cnt[ch.charCodeAt(0) - 97]++;

/* --- GROUPING --- */
const groups = new Map();
for (const w of words) {
    const key = signature(w);
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
}
return [...groups.values()];

/* --- SEEN-BEFORE WITH INDEX --- */
const seen = new Map();          // value -> index
for (let i = 0; i < a.length; i++) {
    if (seen.has(target - a[i])) return [seen.get(target - a[i]), i];
    seen.set(a[i], i);           // AFTER the check, so an element
}                                // cannot pair with itself

/* --- THE TRAPS ---
   · A JS OBJECT COERCES KEYS TO STRINGS: obj[1] and
     obj["1"] are the SAME entry. Map does not. Use Map.
   · ARRAYS AND OBJECTS AS KEYS compare by REFERENCE, so
     map.set([1,2],x) then map.get([1,2]) returns undefined.
     Serialise to a string first: a.join(',').
   · CHECK-THEN-INSERT ORDER matters (see above).
   · `map.get(k) || 0` breaks when the stored value is 0 or
     "". Use `?? 0`.
   · ITERATION ORDER of a Map is insertion order in JS, but
     do not rely on any order in C++ unordered_map.      */
```

Problems

How it shows up

Symptom	Cause
Element pairs with itself	Inserted into the map before checking for the complement
Numeric and string keys collide	Plain object coerces keys — use `Map`
Array key never found	Reference equality — serialise the key
Count of zero treated as missing	`|| 0` instead of `?? 0`
`map.size` wrong after removals	Zero counts not deleted
Answers come out in the wrong order	Hash maps have no order — sort explicitly if order is required
Memory blows up	O(n) space — check whether the constraints allow it
Solution is O(n) but slow in practice	Hashing long string keys is O(L), not O(1)

Check yourself

✓ Give the four uses of hashing and a problem for each.

- ✓ State the transformation that turns an $O(n^2)$ search loop into O(n).
- ✓ Explain why key design is the real problem, with two examples.
- ✓ Say why Two Sum cannot be solved by sorting but 3Sum can.
- ✓ Explain why the insert must come after the check.
- ✓ Give two signatures that group anagrams and their complexities.
- ✓ Say when a fixed array beats a hash map.
- ✓ Name three situations where a hash map is the wrong structure.

Problems

Two Sum `Easy`

Pattern: Map value → index

Given an array of integers `nums` and an integer `target`, return the **indices** of the two numbers that add up to the target.

Exactly one solution exists, and you may not use the same element twice. The array is **not sorted**.

Input	Output	Note
`nums = [2,7,11,15], target = 9`	`[0,1]`	
`nums = [3,2,4], target = 6`	`[1,2]`	
`nums = [3,3], target = 6`	`[0,1]`	Duplicate values, distinct indices.

HINTS

1. For each element you need to know whether `target - x` has already appeared, and where.
2. A map from value to index answers both in O(1).
3. Check for the complement *before* inserting the current element, or a value equal to half the target will pair with itself.

Approach	Idea	Time	Space
Brute force	Every pair.	`O(n²)`	`O(1)`
Sort + two pointers	Works for the values but **destroys the indices**, which are the required output. You would have to store and sort (value, index) pairs.	`O(n log n)`	`O(n)`
Hash map, one pass	Scan once, looking up `target - x` in a map of everything seen so far.	`O(n)`	`O(n)`

Solution — js

```javascript
function twoSum(nums, target) {
  const seen = new Map();          // value -> index
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);          // insert AFTER the check
  }
  return [];
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> seen;
        for (int i = 0; i < (int)nums.size(); ++i) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end()) return {it->second, i};
            seen[nums[i]] = i;
        }
        return {};
    }
};
```

Solution — python

```python
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        seen = {}                      # value -> index
        for i, x in enumerate(nums):
            need = target - x
            if need in seen:
                return [seen[need], i]
            seen[x] = i
        return []
```

Solution — java

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer,Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (seen.containsKey(need)) return new int[]{seen.get(need), i};
            seen.put(nums[i], i);
        }
        return new int[0];
    }
}
```

WALKTHROUGH

WHY THE ORDER OF CHECK-THEN-INSERT MATTERS.

On `[3,3], target 6`: at `i=0` the map is empty, so no match, and 3 is stored. At `i=1` we need 3 and find it at index 0 — correct. If you inserted first, `i=0` would find its own 3 and return `[0,0]`, using one element twice.

THE CONTRAST WITH TWO SUM II IS THE LESSON.

There the input is sorted and the two-pointer solution uses O(1) space. Here you cannot sort, because sorting permutes the indices you must return. *What the problem asks you to return determines which technique is legal* — a distinction worth stating aloud in an interview.

Valid anagram Easy

Pattern: Frequency count

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s` — the same characters with the same multiplicities, in any order.

Input	Output	Note
`s = "anagram", t = "nagaram"`	`true`	
`s = "rat", t = "car"`	`false`	
`s = "a", t = "ab"`	`false`	

HINTS

1. Different lengths can never be anagrams — check that first and return early.
2. Count each character of `s`, then decrement for each character of `t`.
3. If any count goes negative, `t` has a character `s` does not have enough of. If lengths match and nothing went negative, they are anagrams.

Approach	Idea	Time	Space
Sort both	Sort the characters of each and compare. Very short, and a fine answer to state.	`O(n log n)`	`O(n)`
Frequency map	Count up for `s`, down for `t`; fail on any negative.	`O(n)`	`O(k)`
Fixed 26-element array	Same idea, but for lowercase-only input an array beats a Map on constant factors.	`O(n)`	`O(1)`

Solution — js

```javascript
function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const freq = new Map();
  for (const c of s) freq.set(c, (freq.get(c) ?? 0) + 1);
  for (const c of t) {
    const n = (freq.get(c) ?? 0) - 1;
    if (n < 0) return false;        // more of c in t than in s
    freq.set(c, n);
  }
  return true;                       // lengths equal + none negative
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) return false;
        array<int,26> cnt{};
        for (char c : s) ++cnt[c - 'a'];
        for (char c : t) if (--cnt[c - 'a'] < 0) return false;
        return true;
    }
};
```

Solution — python

```python
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        return Counter(s) == Counter(t)
```

Solution — java

```java
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) cnt[c - 'a']++;
        for (char c : t.toCharArray()) if (--cnt[c - 'a'] < 0) return false;
        return true;
    }
}
```

WALKTHROUGH

WHY THE LENGTH CHECK MAKES THE REST SOUND.

If the lengths are equal and no count ever goes negative, then `t` never over-uses a character; since the totals are equal, it cannot under-use one either. That is why a single downward pass suffices with no final verification loop.

THE FOLLOW-UP TO EXPECT:

"what about Unicode?" The 26-element array assumes lowercase ASCII. For arbitrary Unicode use a Map, and be aware that comparing by code point ignores normalisation — "é" can be one code point or two. Mentioning that is a strong signal.

Group anagrams `Medium`

Pattern: Map key → bucket

Given an array of strings, group the anagrams together. Return the groups in any order, and the strings within each group in any order.

Input	Output	Note
`strs = ["eat","tea","tan","ate","nat","bat"]`	`[["eat","tea","ate"],["tan","nat"],["bat"]]`	
`strs = [""]`	`[[""]]`	
`strs = ["a"]`	`[["a"]]`	

HINTS

1. Anagrams need a *canonical form* — a signature that is identical for exactly the strings that belong together.
2. Sorting a word's letters gives one such signature: "eat", "tea" and "ate" all become "aet".
3. A 26-length count vector joined into a string is another, and it avoids the per-word sort.

Approach	Idea	Time	Space
Compare every pair	Test each pair for anagram-ness and union them.	`O(n²·k)`	`O(n)`
Sorted-letters key	Bucket by the sorted characters. Simple and usually fast enough.	`O(n·k log k)`	`O(n·k)`
Count-vector key	Bucket by a 26-slot count signature. Removes the log factor when words are long.	`O(n·k)`	`O(n·k)`

Solution — js

```javascript
function groupAnagrams(strs) {
  const groups = new Map();
  for (const w of strs) {
    const key = w.split("").sort().join("");   // canonical form
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
  }
  return [...groups.values()];
}

// O(n*k) variant using a count signature instead of sorting:
// const cnt = new Array(26).fill(0);
// for (const c of w) cnt[c.charCodeAt(0) - 97]++;
// const key = cnt.join(",");
```

Solution — cpp

```cpp
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> groups;
        for (auto& w : strs) {
            string key = w;
            sort(key.begin(), key.end());
            groups[key].push_back(w);
        }
        vector<vector<string>> out;
        for (auto& [k, v] : groups) out.push_back(move(v));
        return out;
    }
};
```

Solution — python

```python
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        groups = defaultdict(list)
        for w in strs:
            groups["".join(sorted(w))].append(w)
        return list(groups.values())
```

Solution — java

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> groups = new HashMap<>();
        for (String w : strs) {
            char[] c = w.toCharArray();
            Arrays.sort(c);
            groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
        }
        return new ArrayList<>(groups.values());
    }
}
```

WALKTHROUGH

THIS PROBLEM IS ENTIRELY ABOUT CHOOSING THE KEY.

Once you decide that two words belong together exactly when their canonical forms are equal, the code is four lines. The interview value is in articulating *why* sorted letters is a valid canonical form: sorting is a function of the multiset of characters and nothing else, so two words share a sorted form precisely when they are anagrams.

WHICH KEY TO CHOOSE.

With `k` the word length, sorting costs `O(k log k)` per word and the count vector costs `O(k)`. For short words sorting wins on constant factors; for long words or a large alphabet the count vector wins. Say both and pick one — that is the expected answer.

Subarray sum equals K `Medium`

Pattern: Prefix sum + hash map

Given an array of integers `nums` (which **may contain negatives**) and an integer `k`, return the total number of contiguous subarrays whose sum equals `k`.

Input	Output	Note
`nums = [1,1,1], k = 2`	`2`	The two overlapping [1,1] subarrays both count.
`nums = [1,2,3], k = 3`	`2`	[1,2] and [3].
`nums = [1,-1,0], k = 0`	`3`	[1,-1], [0] and [1,-1,0].

HINTS

1. Negatives are present, so a sliding window will not work — growing the window no longer monotonically increases the sum.
2. Let `P[i]` be the sum of the first `i` elements. The sum of `nums[j..i)` is `P[i] - P[j]`.
3. You want `P[i] - P[j] === k`, i.e. `P[j] === P[i] - k`. So count how many earlier prefix sums equal `P[i] - k`.
4. Seed the map with `{0: 1}` — the empty prefix — so that subarrays starting at index 0 are counted.

Approach	Idea	Time	Space
All subarrays	Two nested loops with a running sum.	`O(n²)`	`O(1)`
Sliding window	**Does not work.** Requires all-positive values for the sum to be monotonic in the window size.	–	–
Prefix sums + frequency map	One pass. For each prefix sum, add the number of earlier prefixes equal to `running - k`.	`O(n)`	`O(n)`

Solution — js

```javascript
function subarraySum(nums, k) {
  // prefix sum -> number of times it has occurred
  const count = new Map([[0, 1]]);   // the EMPTY prefix sums to 0
  let running = 0, total = 0;

  for (const x of nums) {
    running += x;
    // any earlier prefix equal to (running - k) closes a valid subarray
    total += count.get(running - k) ?? 0;
    count.set(running, (count.get(running) ?? 0) + 1);
  }
  return total;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<long,int> count{{0, 1}};
        long running = 0;
        int total = 0;
        for (int x : nums) {
            running += x;
            auto it = count.find(running - k);
            if (it != count.end()) total += it->second;
            ++count[running];
        }
        return total;
    }
};
```

Solution — python

```python
from collections import defaultdict

class Solution:
    def subarraySum(self, nums: list[int], k: int) -> int:
        count = defaultdict(int)
        count[0] = 1
        running = total = 0
        for x in nums:
            running += x
            total += count[running - k]
            count[running] += 1
        return total
```

Solution — java

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer,Integer> count = new HashMap<>();
        count.put(0, 1);
        int running = 0, total = 0;
        for (int x : nums) {
            running += x;
            total += count.getOrDefault(running - k, 0);
            count.merge(running, 1, Integer::sum);
        }
        return total;
    }
}
```

WALKTHROUGH

WHY THE SEED `{0: 1}` IS NOT A HACK.

`P[0] = 0` is a genuine prefix — the empty one. A subarray starting at index 0 has sum `P[i] - P[0] = P[i]`, so it is counted only if `0` is already in the map. Test `[1], k = 1`: without the seed the answer is 0 instead of 1.

COUNT OCCURRENCES, NOT PRESENCE.

The map stores *how many times* each prefix sum has appeared, because several earlier positions can produce the same prefix sum and each of them closes a distinct valid subarray. `[0,0,0], k=0` has six subarrays and only counting works.

THE GENERAL PRINCIPLE:

whenever a problem asks about contiguous ranges with a *sum* condition and negatives are allowed, prefix sums plus a hash map is the pattern. With only positives you can usually use a sliding window instead, which is O(1) space. Knowing which precondition each needs is the point — see chapter 06.

Longest consecutive sequence `Hard`

Pattern: Set membership + start detection

Given an unsorted array of integers, return the length of the longest sequence of **consecutive** integers present in the array. Elements need not be adjacent in the array.

Must run in O(n).

Input	Output	Note
`nums = [100,4,200,1,3,2]`	`4`	1, 2, 3, 4.
`nums = [0,3,7,2,5,8,4,6,0,1]`	`9`	0 through 8.
`nums = []`	`0`	

HINTS

1. Sorting solves it in O(n log n), but the requirement is O(n) — so the ordering must come from a Set instead.
2. Put everything in a Set. Now "is x+1 present?" is O(1).
3. The trap is walking a run from every element, which is $O(n^2)$ on a long run. Only start walking from an element that *begins* a run.
4. `x` begins a run exactly when `x - 1` is not in the set.

Approach	Idea	Time	Space
Sort then scan	Sort, then measure consecutive runs, skipping duplicates. Perfectly good — but not O(n).	`O(n log n)`	`O(1)`
Set, walk from every element	Correct but quadratic: a run of length L is re-walked from each of its L members.	`O(n²)`	`O(n)`
Set + start detection	Only extend from elements with no predecessor. Every element is visited at most twice overall.	`O(n)`	`O(n)`

Solution — js

```
function longestConsecutive(nums) {
  const s = new Set(nums);
  let best = 0;

  for (const x of s) {
    // only start counting from the BEGINNING of a run
    if (s.has(x - 1)) continue;

    let len = 1;
    while (s.has(x + len)) len++;
    best = Math.max(best, len);
  }
  return best;
}
```

Solution — cpp

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int best = 0;
        for (int x : s) {
            if (s.count(x - 1)) continue;   // not a run start
            int len = 1;
            while (s.count(x + len)) ++len;
            best = max(best, len);
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        s = set(nums)
        best = 0
        for x in s:
            if x - 1 in s:
                continue
            length = 1
            while x + length in s:
                length += 1
            best = max(best, length)
        return best
```

Solution — java

```java
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int x : nums) s.add(x);
        int best = 0;
        for (int x : s) {
            if (s.contains(x - 1)) continue;
            int len = 1;
            while (s.contains(x + len)) len++;
            best = Math.max(best, len);
        }
        return best;
    }
}
```

WALKTHROUGH

WHY THIS IS O(N) DESPITE THE NESTED `WHILE`.

The inner loop runs only for elements that start a run, and it walks each run exactly once. Every value is therefore touched at most twice in total — once by the outer iteration and once by the single walk over its run — so the whole thing is O(n). This is the same amortised reasoning as the sliding window in chapter 02, and stating it is what convinces the interviewer the bound is real.

WITHOUT THE `CONTINUE` GUARD THE ALGORITHM IS QUADRATIC.

On `[1..n]` shuffled, starting a walk from every element costs `n + (n−1) + … = O(n²)`. That single line is the whole optimisation.

ITERATING THE SET RATHER THAN THE ARRAY

handles duplicates for free — `[5,5,5,5]` visits 5 once and returns 1.

Hashing: cyclic sort and index-as-hash

When the values are a permutation of 1..n, the array *is* the hash table. This is the pattern behind every "find the missing/duplicate number in O(1) space" problem.

The pattern

```
THE RECOGNITION SIGNAL, and it is very specific:

  "an array of n integers where each value is in the
   range [1, n]"      (or [0, n-1])

plus a request for O(1) EXTRA SPACE.

THAT RANGE IS THE WHOLE HINT. Values and indices are
drawn from the same set, so each value has a NATURAL HOME:

     value v  belongs at index  v - 1     (for 1..n)
     value v  belongs at index  v         (for 0..n-1)

TWO WAYS TO EXPLOIT IT:

------------------------------------------------------------
1. CYCLIC SORT - put every value where it belongs

     i = 0
     while (i < n) {
         const home = a[i] - 1;
         if (a[i] !== a[home]) swap(a, i, home);  // not home
         else i++;                                 // settled
     }
     // now scan: the first index where a[i] !== i+1
     // exposes the missing / duplicate value

     [3, 1, 5, 4, 2]
      ^  3 belongs at index 2, swap with 5
     [5, 1, 3, 4, 2]
      ^  5 belongs at index 4, swap with 2
     [2, 1, 3, 4, 5]
      ^  2 belongs at index 1, swap with 1
     [1, 2, 3, 4, 5]  sorted, in O(n)

   WHY IT IS O(n) DESPITE THE NESTED-LOOKING LOOP: every
   swap places at least one value in its final position,
   so there are at most n swaps in total. The amortised
   argument from chapter 02, again.

   COMPARE `a[i] !== a[home]` RATHER THAN `i !== home` -
   with duplicates present, comparing indices loops
   forever when two equal values want the same slot.

------------------------------------------------------------
2. SIGN MARKING - use the sign bit as a "seen" flag

     for (const x of a) {
         const idx = Math.abs(x) - 1;
         if (a[idx] < 0) -> value idx+1 is a DUPLICATE
         a[idx] = -a[idx];        // mark as seen
     }
     // afterwards, any index still positive is MISSING

   Non-destructive if you restore the signs afterwards.
   Only works when the values are strictly positive, so
   the sign is free to carry information.

------------------------------------------------------------

THE OTHER O(1) TRICKS IN THIS FAMILY:
```

```
SUM FORMULA   expected n(n+1)/2 minus the actual sum
              gives one missing value. Simple, but
              overflows for large n and only finds one.
XOR           XOR all values with 1..n; pairs cancel and
              the survivor is the answer (chapter 16).
FLOYD'S CYCLE fast/slow pointers over the "value as next
              index" graph — finds a duplicate without
              modifying the array at all (chapter 18).
```

When to reach for it

"Values are 1..n" + O(1) space

Cyclic sort or sign marking. This phrasing is essentially a giveaway.

Find all missing / all duplicates

Sign marking finds every one in a single pass.

First missing positive

Cyclic sort, ignoring values outside 1..n.

Cannot modify the array

Floyd's cycle detection, or binary search on the value range.

Exactly one missing, no duplicates

Sum formula or XOR — one line each.

Problems

How it shows up

Symptom	Cause
Infinite loop in cyclic sort	Compared indices instead of values — duplicates never settle
Index out of range	Values outside 1..n not filtered before computing the home
Sign marking gives wrong answers	Forgot `Math.abs` when reading an already-flipped value
Marks a zero	Zero has no sign — shift to 1-based or use a different marker
Sum formula overflows	n(n+1)/2 exceeds 32 bits — use XOR or 64-bit
Works for one missing, fails for several	Sum/XOR find exactly one — use marking
Input array corrupted for the caller	These techniques mutate — restore or document it

Check yourself

- ✓ State the recognition signal precisely.
- ✓ Give the home-index formula for both 1..n and 0..n−1.
- ✓ Write the cyclic sort loop and justify O(n).

- ✓ Explain why the swap condition compares values, not indices.
- ✓ Describe sign marking and its precondition.
- ✓ Give three alternative O(1)-space tricks and each one's limit.
- ✓ Say what to do when the array may not be modified.

Problems

Missing number Easy

Pattern: Sum formula / XOR

Given an array `nums` containing `n` distinct numbers drawn from the range `[0, n]`, return the one number in that range which is missing.

Input	Output	Note
`nums = [3,0,1]`	`2`	
`nums = [0,1]`	`2`	
`nums = [9,6,4,2,3,5,7,0,1]`	`8`	

HINTS

1. The numbers 0..n sum to `n(n+1)/2`. Compare with the actual sum.
2. Alternatively, XOR everything in the array together with every index 0..n — each present value cancels itself and only the missing one survives.
3. The XOR version cannot overflow, which is the reason to prefer it.

Approach	Idea	Time	Space
Sort then scan	Sort and find the first gap.	`O(n log n)`	`O(1)`
Hash set	Insert everything, then test 0..n.	`O(n)`	`O(n)`
Sum formula	Expected total minus actual total.	`O(n)`	`O(1)`
XOR	Pairs cancel; no overflow risk.	`O(n)`	`O(1)`

Solution — js

```
function missingNumber(nums) {
  const n = nums.length;
  let x = n;                        // start with n itself
  for (let i = 0; i < n; i++) {
    x ^= i ^ nums[i];               // cancel index against value
  }
  return x;
}

// sum-formula alternative:
// const expected = n * (n + 1) / 2;
// return expected - nums.reduce((a, b) => a + b, 0);
```

Solution — cpp

```cpp
class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int n = nums.size(), x = n;
        for (int i = 0; i < n; ++i) x ^= i ^ nums[i];
        return x;
    }
};
```

Solution — python

```python
class Solution:
    def missingNumber(self, nums: list[int]) -> int:
        x = len(nums)
        for i, v in enumerate(nums):
            x ^= i ^ v
        return x
```

Solution — java

```java
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length, x = n;
        for (int i = 0; i < n; i++) x ^= i ^ nums[i];
        return x;
    }
}
```

WALKTHROUGH

WHY THE XOR WORKS.

XOR is associative and commutative, `a ^ a === 0`, and `a ^ 0 === a`. Folding together every index `0..n` and every array value pairs up each present number with its index, leaving only the number that has no partner — the missing one. Seeding `x = n` supplies the index `n` that the loop over `0..n-1` does not reach.

WHY XOR BEATS THE SUM FORMULA.

For `n` near 2^{31} the expected sum overflows a 32-bit integer, and in C++ or Java that is undefined or wraps. XOR never grows. Mentioning that trade-off is exactly the kind of detail interviewers reward.

Find all duplicates in an array `Medium`

Pattern: Sign marking

Given an array `nums` of length `n` where every value is in the range `[1, n]` and each appears **once or twice**, return all values that appear twice.

Use O(1) extra space (the output does not count) and one pass.

Input	Output	Note
`nums = [4,3,2,7,8,2,3,1]`	`[2,3]`	
`nums = [1,1,2]`	`[1]`	
`nums = [1]`	`[]`	

HINTS

1. Every value is a valid index (after subtracting 1). The array can record its own "seen" flags.
2. Negate `nums[|v| - 1]` to mark that value `|v|` has been seen.
3. If that slot is already negative, you are seeing `|v|` for the second time — record it.
4. Always read values through `Math.abs`, since earlier marking may have flipped them.

Approach	Idea	Time	Space
Hash set	Track seen values in a set. Clear, but O(n) extra space.	`O(n)`	`O(n)`
Sort then scan	Adjacent equal values are duplicates.	`O(n log n)`	`O(1)`
Sign marking	Use the sign bit of each slot as a seen flag.	`O(n)`	`O(1)`

Solution — js

```
function findDuplicates(nums) {
  const out = [];
  for (const v of nums) {
    const idx = Math.abs(v) - 1;      // v may already be negated
    if (nums[idx] < 0) {
      out.push(idx + 1);              // second sighting
    } else {
      nums[idx] = -nums[idx];         // mark as seen
    }
  }
  // (optional) restore: for (let i=0;i<nums.length;i++) nums[i]=Math.abs(nums[i]);
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> findDuplicates(vector<int>& nums) {
        vector<int> out;
        for (int v : nums) {
            int idx = abs(v) - 1;
            if (nums[idx] < 0) out.push_back(idx + 1);
            else               nums[idx] = -nums[idx];
        }
        return out;
    }
};
```

Solution — python

```
class Solution:
    def findDuplicates(self, nums: list[int]) -> list[int]:
        out = []
        for v in nums:
            idx = abs(v) - 1
            if nums[idx] < 0:
                out.append(idx + 1)
            else:
                nums[idx] = -nums[idx]
        return out
```

Solution — java

```java
class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> out = new ArrayList<>();
        for (int v : nums) {
            int idx = Math.abs(v) - 1;
            if (nums[idx] < 0) out.add(idx + 1);
            else               nums[idx] = -nums[idx];
        }
        return out;
    }
}
```

WALKTHROUGH

THE SIGN BIT IS FREE STORAGE.

Because every value is guaranteed positive and in range, the sign carries no information of its own — so it can be repurposed as one bit of scratch space per slot, giving n bits of working memory without allocating anything. That is the trick, and it generalises: whenever the data does not use the full range of its type, the spare bits are usable.

`MATH.ABS` ON THE READ IS NOT OPTIONAL.

By the time you reach index 5 in `[4,3,2,7,8,2,3,1]`, the value stored there may already have been negated by an earlier iteration. Reading it raw gives a negative index and either a crash or silent nonsense.

PUSH `IDX + 1`, NOT `V`.

`v` is the possibly-negated array value; `idx + 1` is the actual number being duplicated. They coincide only when `v` happens to still be positive.

THIS MUTATES THE INPUT.

Say so aloud, and offer the restore loop — some interviewers specifically ask whether you noticed.

First missing positive `Hard`

Pattern: Cyclic sort with filtering

Given an unsorted integer array `nums` — which may contain negatives, zeros and duplicates — return the smallest **positive** integer that is **not** present.

Must run in O(n) time and use O(1) extra space.

Input	Output	Note
`nums = [1,2,0]`	`3`	
`nums = [3,4,-1,1]`	`2`	
`nums = [7,8,9,11,12]`	`1`	

HINTS

1. The answer is always in the range `1 .. n+1`: with `n` slots you cannot hide all of 1..n+1.
2. So any value outside `1..n` is irrelevant and can be ignored.
3. Cyclic sort: repeatedly send each in-range value to index `v - 1`. Then the first index whose value is not `i + 1` gives the answer.
4. Guard the swap with `nums[i] !== nums[home]`, not `i !== home`, or duplicates spin forever.

Approach	Idea	Time	Space
Sort then scan	Sort and walk looking for the first gap. Violates the O(n) requirement.	`O(n log n)`	`O(1)`
Hash set	Insert everything, then test 1, 2, 3 … Violates the O(1) space requirement.	`O(n)`	`O(n)`
Cyclic sort in place	Place each in-range value at its home index, then scan for the first mismatch.	`O(n)`	`O(1)`

Solution — js

```
function firstMissingPositive(nums) {
  const n = nums.length;

  // 1. send every in-range value to its home index v-1
  for (let i = 0; i < n; i++) {
    while (nums[i] >= 1 && nums[i] <= n &&
           nums[nums[i] - 1] !== nums[i]) {
      const home = nums[i] - 1;
      [nums[i], nums[home]] = [nums[home], nums[i]];
    }
  }

  // 2. the first slot not holding i+1 reveals the answer
  for (let i = 0; i < n; i++) {
    if (nums[i] !== i + 1) return i + 1;
  }
  return n + 1;          // 1..n all present
}
```

Solution — cpp

```
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        int n = nums.size();
        for (int i = 0; i < n; ++i)
            while (nums[i] >= 1 && nums[i] <= n &&
                   nums[nums[i] - 1] != nums[i])
                swap(nums[i], nums[nums[i] - 1]);
        for (int i = 0; i < n; ++i)
            if (nums[i] != i + 1) return i + 1;
        return n + 1;
    }
};
```

Solution — python

```python
class Solution:
    def firstMissingPositive(self, nums: list[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                home = nums[i] - 1
                nums[i], nums[home] = nums[home], nums[i]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

Solution — java

```java
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; i++)
            while (nums[i] >= 1 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                int home = nums[i] - 1;
                int t = nums[i]; nums[i] = nums[home]; nums[home] = t;
            }
        for (int i = 0; i < n; i++)
            if (nums[i] != i + 1) return i + 1;
        return n + 1;
    }
}
```

WALKTHROUGH

THE BOUND ARGUMENT COMES FIRST AND IT IS WHAT MAKES O(1) SPACE POSSIBLE.

With `n` array slots, the values 1 through `n+1` are `n+1` distinct candidates, so at least one must be absent. The answer therefore lies in `[1, n+1]`, and every value outside `1..n` — negatives, zeros, huge numbers — can be ignored entirely. That is why no extra structure is needed: the array itself is exactly large enough to index the candidate space.

THE SWAP GUARD IS THE CRUX.

Writing `while (… && i !== home)` looks equivalent but is not: on `[2,2,2,2]`, index 0 holds 2 whose home is index 1, which also holds 2. Swapping changes nothing, `i !== home` stays true, and the loop spins forever. Comparing *values* — "is my home already occupied by my own value?" — terminates because it detects the duplicate. Test `[1,1]` and `[2,2,2,2]` to confirm.

WHY THE WHOLE THING IS O(N) DESPITE A `WHILE` INSIDE A `FOR`.

Each successful swap puts at least one value permanently into its correct home, and a value once home is never moved again. So there are at most `n` swaps across the entire run, and the outer loop contributes another `n` — linear overall.

Bit manipulation: the fundamentals

The operators, the idioms, and the two-line tricks worth memorising. Everything here is O(1) and appears constantly in embedded work as well as in interviews.

The operators

```
&   AND    1 only where BOTH are 1      1100 & 1010 = 1000
|   OR     1 where EITHER is 1          1100 | 1010 = 1110
^   XOR    1 where they DIFFER          1100 ^ 1010 = 0110
~   NOT    flip every bit               ~1100       = 0011
                                        (in two's
                                         complement,
                                         ~x === -x - 1)
<<  SHIFT LEFT    x << k  ==  x * 2^k
>>  SHIFT RIGHT   arithmetic — copies the SIGN bit
>>> SHIFT RIGHT   logical — fills with ZEROS (JavaScript;
                  Java uses >>> too, C++ depends on
                  signedness)

THE SIX IDIOMS TO KNOW COLD:

  TEST bit i        (x >> i) & 1
  SET bit i         x | (1 << i)
  CLEAR bit i       x & ~(1 << i)
  TOGGLE bit i      x ^ (1 << i)
  LOWEST SET BIT    x & -x        -> isolates it
  CLEAR LOWEST SET  x & (x - 1)   -> removes it

THE LAST TWO ARE THE ONES THAT WIN PROBLEMS.

  x       = 1011 0100
  x - 1   = 1011 0011      borrow flips the lowest 1 and
                           everything below it
  x&(x-1) = 1011 0000      lowest set bit REMOVED
  -x      = 0100 1100      two's complement = ~x + 1
  x & -x  = 0000 0100      lowest set bit ISOLATED

USEFUL CONSEQUENCES:
  x & (x - 1) === 0        ->  x is a POWER OF TWO
                               (or zero — check separately)
  counting with x &= x-1   ->  loops once per SET BIT,
                               not once per bit

THE XOR PROPERTIES, which power chapter 16:
  x ^ x = 0        a value cancels itself
  x ^ 0 = x        zero is the identity
  XOR is COMMUTATIVE and ASSOCIATIVE — order never matters
  a ^ b ^ b = a    XOR is its own inverse
```

JAVASCRIPT'S BITWISE OPERATORS ARE 32-BIT

All of `& | ^ ~ << >>` convert their operands to **signed 32-bit** integers, so `1 << 31` is negative and anything above 2^{31} loses its high bits. Use `>>> 0` to read a result as unsigned, and `BigInt` if you genuinely need more than 32 bits. In C++ and Java the same care applies: shifting by ≥ the width is *undefined* in C++, and in Java the shift count is taken modulo 32 (or 64 for `long`), so `1 << 32` is `1`, not 0.

```
/* --- COUNTING SET BITS (population count) --- */
function popcount(x) {
    let c = 0;
    while (x) { x &= x - 1; c++; }   // one iteration PER SET BIT
    return c;
}
// built-ins: __builtin_popcount (C++), Integer.bitCount (Java),
//            bin(x).count('1') (Python)

/* --- ITERATE THE SET BITS --- */
while (mask) {
    const bit = mask & -mask;        // isolate the lowest
    const i   = Math.log2(bit);      // its index
    mask ^= bit;                     // remove it
}

/* --- SWAP WITHOUT A TEMPORARY (a party trick, not
       production code - it breaks when a === b aliases) --- */
a ^= b; b ^= a; a ^= b;

/* --- USEFUL MASKS --- */
(1 << k) - 1        // k low bits set:  k=4 -> 0b1111
x & ((1 << k) - 1)  // keep the low k bits
x >> k & 1          // read bit k
~0                  // all ones (-1)

/* --- PARITY of the set-bit count --- */
x ^= x >> 16; x ^= x >> 8; x ^= x >> 4;
x ^= x >> 2;  x ^= x >> 1;  return x & 1;

/* --- MULTIPLY / DIVIDE BY POWERS OF TWO --- */
x << 3   // x * 8
x >> 3   // floor(x / 8) for NON-NEGATIVE x
// CAREFUL: -7 >> 1 === -4, but -7 / 2 truncates to -3.
// Arithmetic shift rounds toward NEGATIVE INFINITY.  */
```

Problems

How it shows up

Symptom	Cause
Result is negative unexpectedly	Bit 31 set — JavaScript bitwise ops are signed 32-bit; use `>>> 0`
Shift by 32 does nothing	Java/JS take the count mod 32; C++ it is undefined
Loop never terminates on a negative input	`>>` preserves the sign forever — use `>>>`
Zero reported as a power of two	`x & (x-1) === 0` is also true for 0 — add `x > 0`
Negative division off by one	Arithmetic shift rounds toward −∞, division truncates toward 0
Precedence bug	`&` and `|` bind looser than `==` — parenthesise
Wrong answer above 2^{31}	Exceeded 32 bits — use `BigInt` or 64-bit types

Check yourself

✓ Write test, set, clear and toggle for bit *i*.

- ✓ Explain what `x & (x-1)` and `x & -x` do, with a worked binary example.
- ✓ Give the power-of-two test including the zero case.
- ✓ Write popcount using `x &= x-1` and say why it is O(set bits).
- ✓ State the four XOR properties.
- ✓ Explain the difference between `>>` and `>>>`.
- ✓ Say why `-7 >> 1` is not the same as `-7 / 2` truncated.
- ✓ State JavaScript's 32-bit limitation and its workaround.

Problems

Number of 1 bits Easy

Pattern: Clear the lowest set bit

Given a non-negative integer `n`, return the number of `1` bits in its binary representation (its Hamming weight).

Input	Output	Note
`n = 11 (1011)`	`3`	
`n = 128 (10000000)`	`1`	
`n = 0`	`0`	

HINTS

1. The obvious loop shifts right 32 times checking the low bit. That works but always costs 32 iterations.
2. `n & (n - 1)` clears the lowest set bit. How many times can you do that before reaching zero?
3. Exactly once per set bit — so the loop runs only as many times as there are 1s.

Approach	Idea	Time	Space
Check all 32 bits	Shift and test each position.	`O(32)`	`O(1)`
Clear the lowest set bit	`n &= n - 1` until zero; count the iterations.	`O(set bits)`	`O(1)`
Built-in	`Integer.bitCount`, `__builtin_popcount`. Say you know it exists, then write the loop.	`O(1)`	`O(1)`

Solution — js

```
function hammingWeight(n) {
  let count = 0;
  while (n !== 0) {
    n &= n - 1;      // remove the LOWEST set bit
    count++;
  }
  return count;
}
```

Solution — cpp

```
class Solution {
public:
    int hammingWeight(uint32_t n) {
        int count = 0;
        while (n) { n &= n - 1; ++count; }
        return count;
    }
};
```

Solution — python

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        count = 0
        while n:
            n &= n - 1
            count += 1
        return count
```

Solution — java

```
class Solution {
    public int hammingWeight(int n) {
        int count = 0;
        while (n != 0) { n &= n - 1; count++; }
        return count;
    }
}
```

WALKTHROUGH

WHY `N & (N - 1)` REMOVES EXACTLY THE LOWEST SET BIT.

Subtracting 1 flips the lowest 1 to 0 and turns every 0 below it into 1; all higher bits are untouched. ANDing with the original therefore keeps every higher bit unchanged, kills the lowest 1, and zeroes everything below it. Worked: `1011 0100 - 1 = 1011 0011`, and `1011 0100 & 1011 0011 = 1011 0000`.

THE COMPLEXITY CLAIM MATTERS IN THE INTERVIEW.

The naive version is always 32 iterations; this one is exactly `popcount(n)` iterations, so for sparse numbers it is dramatically faster. Saying "O(number of set bits) rather than O(word size)" is the answer they are listening for.

WATCH THE SIGN IN JAVASCRIPT AND JAVA.

For `n` with bit 31 set, `n` is negative and `n !== 0` still terminates correctly because the AND strictly reduces the set bits. But writing the shift-based version with `>>` instead of `>>>` would loop forever on a negative value.

Power of two Easy

Pattern: Single-bit test

Given an integer `n`, return `true` if it is a power of two — that is, if `n === 2^k` for some non-negative integer `k`.

Input	Output	Note
`n = 1`	`true`	2^0

Input	Output	Note
`n = 16`	`true`	
`n = 3`	`false`	
`n = 0`	`false`	
`n = -16`	`false`	

HINTS

1. A power of two has exactly one bit set in its binary representation.
2. `n & (n - 1)` clears the lowest set bit — so for a power of two the result is 0.
3. That test is *also* true for `n = 0`, so require `n > 0` as well. Negatives must be rejected too.

Approach	Idea	Time	Space
Repeated division	Divide by 2 while even; check whether you land on 1.	`O(log n)`	`O(1)`
Popcount	Count set bits and test for exactly one.	`O(1)`	`O(1)`
Single AND	`n > 0 && (n & (n-1)) === 0`.	`O(1)`	`O(1)`

Solution — js

```
function isPowerOfTwo(n) {
  // n > 0 rules out zero and negatives;
  // (n & (n-1)) === 0 means exactly one bit is set
  return n > 0 && (n & (n - 1)) === 0;
}
```

Solution — cpp

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }
};
```

Solution — python

```
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
        return n > 0 and (n & (n - 1)) == 0
```

Solution — java

```
class Solution {
    public boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }
}
```

WALKTHROUGH

BOTH GUARDS ARE LOAD-BEARING.

`(0 & -1) === 0`, so zero passes the AND test and must be excluded by `n > 0`. Negative powers of two are not powers of two by the definition here, and in two's complement `-16 = …11110000` whose AND with `-17` is not zero anyway — but relying on that is fragile, and `n > 0` makes the intent explicit.

THE RELATED ONE-LINERS WORTH KNOWING:

`n & (n-1)` being zero means "at most one bit set"; `n & -n` isolates the lowest set bit, so `(n & -n) === n` is an equivalent power-of-two test. And `(n - 1) & n` appearing in a codebase is almost always a power-of-two mask for a ring buffer — see the embedded tracks on this site.

Reverse bits Easy

Pattern: Shift out, shift in

Reverse the bits of a given **32-bit unsigned** integer and return the result as an unsigned value.

For example, `43261596` (binary `00000010100101000001111010011100`) reverses to `964176192` (`00111001011110000010100101000000`).

Input	Output	Note
`n = 43261596`	`964176192`	
`n = 1`	`2147483648`	Bit 0 becomes bit 31.
`n = 0`	`0`	

HINTS

1. Process 32 bits. Each step: shift the answer left, pull the lowest bit off the input, and OR it in.
2. Shift the *input* right with `>>>` so the sign bit does not replicate.
3. The final result must be interpreted as unsigned — in JavaScript finish with `>>> 0`.

Approach	Idea	Time	Space
Build a binary string	Convert to a padded 32-character string, reverse it, parse back. Works, and is easy to explain.	`O(32)`	`O(32)`
Shift out, shift in	Thirty-two iterations of one shift each way.	`O(32)`	`O(1)`
Divide and conquer masks	Swap halves, then quarters, then bytes, then nibbles, then pairs — five masked steps, O(log 32).	`O(1)`	`O(1)`

Solution — js

```
function reverseBits(n) {
  let out = 0;
  for (let i = 0; i < 32; i++) {
    out = (out << 1) | (n & 1);   // make room, take the low bit
    n >>>= 1;                      // LOGICAL shift: no sign copy
  }
  return out >>> 0;                // read the result as unsigned
}
```

Solution — cpp

```cpp
class Solution {
public:
    uint32_t reverseBits(uint32_t n) {
        uint32_t out = 0;
        for (int i = 0; i < 32; ++i) {
            out = (out << 1) | (n & 1);
            n >>= 1;
        }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def reverseBits(self, n: int) -> int:
        out = 0
        for _ in range(32):
            out = (out << 1) | (n & 1)
            n >>= 1
        return out
```

Solution — java

```java
public class Solution {
    public int reverseBits(int n) {
        int out = 0;
        for (int i = 0; i < 32; i++) {
            out = (out << 1) | (n & 1);
            n >>>= 1;          // logical shift
        }
        return out;
    }
}
```

WALKTHROUGH

TWO PLACES THE SIGN BIT BITES.

First, `n >>= 1` in JavaScript or Java is an *arithmetic* shift: for a negative `n` it fills with 1s and the loop reads 1 bits forever. `>>>` fills with zeros, which is what "unsigned" requires. Second, `out` itself becomes negative once bit 31 is set, so the final `>>> 0` reinterprets the 32-bit pattern as an unsigned value — without it, `reverseBits(1)` returns `-2147483648` instead of `2147483648`.

THE MASK-BASED VERSION IS WORTH MENTIONING

as a follow-up: swap the two 16-bit halves, then the 8-bit groups within each, then 4, then 2, then 1 — five constant-masked steps instead of 32 iterations. It is the standard answer to "can you do better if this is called millions of times?", along with "cache results in a 256-entry byte lookup table".

Bit manipulation: XOR, subsets and tricks

The problem-solving half. XOR as a cancelling accumulator, integers as sets, and the handful of tricks that turn a hard-looking question into two lines.

XOR as a cancelling accumulator

```
THE FOUR PROPERTIES, and every XOR problem uses them:

  x ^ x = 0            a value CANCELS itself
  x ^ 0 = x            zero is the identity
  COMMUTATIVE          a ^ b = b ^ a
  ASSOCIATIVE          (a^b)^c = a^(b^c)

CONSEQUENCE: XOR-ing a whole collection together makes
every PAIR vanish, whatever the order. Whatever survives
is what appeared an ODD number of times.

  [4, 1, 2, 1, 2]  ->  4^1^2^1^2
                   =  4^(1^1)^(2^2)
                   =  4^0^0  =  4

THAT IS "SINGLE NUMBER" IN ONE LINE, in O(1) space, and
it is why the pattern is worth recognising instantly.

------------------------------------------------------------
TWO NUMBERS APPEARING ONCE — the two-pass trick

  xorAll = a ^ b            (everything else cancels)

  Any set bit in xorAll is a position where a and b
  DIFFER. Pick the lowest:

     diff = xorAll & -xorAll

  Now PARTITION the input by that bit. a and b land in
  different groups, and every duplicate pair lands in the
  SAME group (equal values have equal bits). XOR each
  group separately and you get a and b.

------------------------------------------------------------
EVERY NUMBER APPEARS THREE TIMES, one appears once

  XOR fails — three copies leave one behind. Instead,
  count each bit position modulo 3:

     ones = (ones ^ x) & ~twos;
     twos = (twos ^ x) & ~ones;

  `ones` holds bits seen 1 mod 3 times, `twos` bits seen
  2 mod 3. After the pass, `ones` is the answer.
  (The generic version: sum each of the 32 bit positions
   mod k. Easier to derive under pressure.)

------------------------------------------------------------
XOR PREFIX — the same idea as prefix sums (chapter 06)

  P[i] = a[0] ^ a[1] ^ ... ^ a[i-1]
  XOR of range [l, r]  =  P[r+1] ^ P[l]

  because the shared prefix cancels itself. Everything
  you know about prefix sums transfers, with ^ replacing
  + and its own inverse replacing subtraction.
```

Integers as sets

```
AN n-BIT INTEGER IS A SUBSET OF {0, 1, ..., n-1}.
Bit i set means element i is IN the subset.

  mask = 0b1011   ->   {0, 1, 3}

THE SET OPERATIONS:
  union            a | b
  intersection     a & b
  difference       a & ~b
  symmetric diff   a ^ b
  add element i    a | (1 << i)
  remove element i a & ~(1 << i)
  test element i   (a >> i) & 1
  size             popcount(a)
  is subset of b   (a & b) === a
  empty            a === 0
  full (n items)   a === (1 << n) - 1

ENUMERATE ALL 2^n SUBSETS:
  for (let mask = 0; mask < (1 << n); mask++) {
      for (let i = 0; i < n; i++)
          if (mask & (1 << i)) use(items[i]);
  }
  O(2^n · n). THIS IS WHY "n <= 20" IN A CONSTRAINT
  MEANS BITMASK (chapter 02).

ENUMERATE ALL SUBMASKS OF A MASK — a genuinely useful
idiom:
  for (let s = mask; s > 0; s = (s - 1) & mask) { ... }
  // then handle s = 0 separately
Total across all masks is 3^n, not 4^n.

THIS REPRESENTATION IS THE BASIS OF BITMASK DP
(chapter 37): the state "which items have I used?" is a
single integer, so the DP table is a flat array.
```

Problems

How it shows up

Symptom	Cause
XOR trick fails when values repeat 3×	XOR only cancels *pairs* — count bits mod 3
Two-single-numbers split puts both in one group	Partition bit not actually a differing bit
Subset loop misses the empty set	Submask loop stops at `s > 0` — handle 0 after
Subset enumeration overflows	`1 << n` with n ≥ 31 — JavaScript ints are 32-bit signed
Sum of two numbers loops forever	Carry not shifted, or negative handling in JS
Answer negative on the last bit	Bit 31 is the sign bit — finish with `>>> 0`
Gray code sequence has a jump	Formula must be `i ^ (i >> 1)`

Check yourself

- ✓ State the four XOR properties and derive "single number" from them.
- ✓ Explain the two-group partition for two unique numbers.
- ✓ Say why XOR fails when values repeat three times, and give the fix.
- ✓ Give the XOR-prefix range formula and why it works.
- ✓ Write the set operations on a bitmask: union, intersection, difference, subset test.
- ✓ Write the loop enumerating all 2^n subsets and give its complexity.
- ✓ Write the submask enumeration idiom and its total cost.
- ✓ Explain why "n ≤ 20" suggests a bitmask.

Problems

Single number Easy

Pattern: XOR cancellation

Every element in `nums` appears **twice** except for one, which appears once. Find that one.

Use O(1) extra space and linear time.

Input	Output	Note
`nums = [2,2,1]`	`1`	
`nums = [4,1,2,1,2]`	`4`	
`nums = [1]`	`1`	

HINTS

1. A hash set works but uses O(n) space, which the problem rules out.
2. What operation makes a value cancel itself?
3. XOR the whole array together — every pair vanishes and the loner survives.

Approach	Idea	Time	Space
Hash set	Add and remove; whatever remains is the answer.	`O(n)`	`O(n)`
Sort then scan	The loner breaks the pairing pattern.	`O(n log n)`	`O(1)`
XOR everything	One accumulator, one pass.	`O(n)`	`O(1)`

Solution — js

```
function singleNumber(nums) {
  let x = 0;
  for (const v of nums) x ^= v;   // pairs cancel to 0
  return x;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int x = 0;
        for (int v : nums) x ^= v;
        return x;
    }
};
```

Solution — python

```python
from functools import reduce
import operator

class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        return reduce(operator.xor, nums, 0)
```

Solution — java

```java
class Solution {
    public int singleNumber(int[] nums) {
        int x = 0;
        for (int v : nums) x ^= v;
        return x;
    }
}
```

WALKTHROUGH

ORDER DOES NOT MATTER

, which is the property doing the real work. Because XOR is commutative and associative, `4^1^2^1^2` can be regrouped as `4^(1^1)^(2^2) = 4^0^0 = 4` without touching the array. That is why no sorting or bookkeeping is needed.

IT WORKS FOR NEGATIVES TOO

, since XOR operates on the two's-complement bit pattern and identical patterns still cancel — the `[-1,-1,-2]` test confirms it.

Single number III — two loners `Medium`

Pattern: XOR then partition on a differing bit

Exactly **two** elements of `nums` appear once; every other element appears twice. Return the two singles in any order.

Linear time, constant extra space.

Input	Output	Note
`nums = [1,2,1,3,2,5]`	`[3,5]`	
`nums = [-1,0]`	`[-1,0]`	
`nums = [0,1]`	`[1,0]`	

HINTS

1. XOR everything: the pairs cancel and you are left with `a ^ b`, but mixed together.
2. Any set bit in `a ^ b` is a position where `a` and `b` differ. Isolate one with `x & -x`.
3. Split the array into two groups by that bit. Duplicates always land together; `a` and `b` land apart.
4. XOR each group separately — each collapses to one of the answers.

Approach	Idea	Time	Space
Hash map counts	Count occurrences and collect the ones. O(n) space.	`O(n)`	`O(n)`
XOR + partition bit	Two passes, one accumulator each, no extra memory.	`O(n)`	`O(1)`

Solution — js

```
function singleNumberIII(nums) {
  // pass 1: everything cancels except a ^ b
  let xorAll = 0;
  for (const v of nums) xorAll ^= v;

  // any set bit is a position where a and b DIFFER
  const diff = xorAll & -xorAll;   // isolate the lowest

  // pass 2: partition on that bit; each group holds one loner
  let a = 0, b = 0;
  for (const v of nums) {
    if (v & diff) a ^= v;
    else          b ^= v;
  }
  return [a, b];
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {
        long xorAll = 0;
        for (int v : nums) xorAll ^= v;
        int diff = (int)(xorAll & -xorAll);
        int a = 0, b = 0;
        for (int v : nums) {
            if (v & diff) a ^= v;
            else          b ^= v;
        }
        return {a, b};
    }
};
```

Solution — python

```python
class Solution:
    def singleNumber(self, nums: list[int]) -> list[int]:
        xor_all = 0
        for v in nums:
            xor_all ^= v
        diff = xor_all & -xor_all
        a = b = 0
        for v in nums:
            if v & diff:
                a ^= v
            else:
                b ^= v
        return [a, b]
```

Solution — java

```java
class Solution {
    public int[] singleNumber(int[] nums) {
        int xorAll = 0;
        for (int v : nums) xorAll ^= v;
        int diff = xorAll & -xorAll;
        int a = 0, b = 0;
        for (int v : nums) {
            if ((v & diff) != 0) a ^= v;
            else                 b ^= v;
        }
        return new int[]{a, b};
    }
}
```

WALKTHROUGH

WHY THE PARTITION IS GUARANTEED TO SEPARATE THEM.

`a ≠ b`, so `a ^ b ≠ 0` and it has at least one set bit. At that bit position `a` and `b` disagree by definition of XOR, so exactly one of them has it set — they must land in different groups. Meanwhile any duplicated value has the same bits in both copies, so both copies land in the same group and cancel there. Each group therefore reduces to exactly one loner.

`XORALL & -XORALL` ISOLATES THE LOWEST SET BIT

(chapter 15). Any set bit would do; the lowest is simply the cheapest to extract. In C++ the intermediate is widened to `long` to avoid undefined behaviour when negating `INT_MIN`.

Counting bits Easy

Pattern: DP on bit structure

Given an integer `n`, return an array `ans` of length `n + 1` where `ans[i]` is the number of `1` bits in `i`.

Do it in a single pass, O(n) total — not O(n log n).

Input	Output	Note
`n = 2`	`[0,1,1]`	
`n = 5`	`[0,1,1,2,1,2]`	

HINTS

1. Calling popcount on each number independently is O(n log n). You want to reuse earlier answers.
2. Relate `i` to a smaller number whose answer you already have.
3. `i >> 1` is `i` with the last bit dropped — so `bits[i] = bits[i >> 1] + (i & 1)`.
4. Alternatively `bits[i] = bits[i & (i - 1)] + 1`, since `i & (i-1)` removes one set bit.

Approach	Idea	Time	Space
popcount each number	Independent counting per value.	`O(n log n)`	`O(1)`
DP on <code>i >> 1</code>	Each answer is one table lookup plus the last bit.	`O(n)`	`O(1) extra`

Solution — js

```
function countBits(n) {
  const bits = new Array(n + 1).fill(0);
  for (let i = 1; i <= n; i++) {
    // i >> 1 is i without its last bit — already computed
    bits[i] = bits[i >> 1] + (i & 1);
  }
  return bits;
}

// equivalent: bits[i] = bits[i & (i - 1)] + 1;
```

Solution — cpp

```
class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> bits(n + 1, 0);
        for (int i = 1; i <= n; ++i)
            bits[i] = bits[i >> 1] + (i & 1);
        return bits;
    }
};
```

Solution — python

```
class Solution:
    def countBits(self, n: int) -> list[int]:
        bits = [0] * (n + 1)
        for i in range(1, n + 1):
            bits[i] = bits[i >> 1] + (i & 1)
        return bits
```

Solution — java

```
class Solution {
    public int[] countBits(int n) {
        int[] bits = new int[n + 1];
        for (int i = 1; i <= n; i++)
            bits[i] = bits[i >> 1] + (i & 1);
        return bits;
    }
}
```

WALKTHROUGH

THE RECURRENCE IS THE WHOLE SOLUTION.

Writing `i` in binary, dropping the last bit gives `i >> 1` — a strictly smaller number, so its answer is already in the table. The bit you dropped was `i & 1`. Hence `bits[i] = bits[i >> 1] + (i & 1)`, one lookup and one addition per value.

THIS IS DYNAMIC PROGRAMMING

, and a good one to recognise as such: an optimal-substructure relation on a smaller subproblem, with the table filled in increasing order so every dependency is ready. It is the smallest possible example of the technique in chapter 33.

THE ALTERNATIVE RECURRENCE

`bits[i] = bits[i & (i-1)] + 1` uses the lowest-set-bit clearing trick from chapter 15 instead, and is equally valid — mention both.

Sum of two integers without + `Medium`

Pattern: XOR as add, AND as carry

Given two integers `a` and `b`, return their sum **without using the `+` or `-` operators**.

Negative inputs must work correctly.

Input	Output	Note
`a = 1, b = 2`	`3`	
`a = 2, b = 3`	`5`	
`a = -1, b = 1`	`0`	

HINTS

1. Think about how binary addition works bit by bit: a sum bit and a carry bit.
2. The sum *ignoring carries* is exactly `a ^ b`. The carries are `(a & b) << 1`.
3. Repeat: add the carry into the sum, producing new carries, until there is no carry left.
4. Two's complement means the same loop handles negatives with no special cases.

Approach	Idea	Time	Space
Repeated increment	Add 1 `b` times. Correct only for tiny positive `b`.	`O(b)`	`O(1)`
XOR + carry loop	Simulate the ripple-carry adder. At most 32 iterations.	`O(1)`	`O(1)`

Solution — js

```javascript
function getSum(a, b) {
  while (b !== 0) {
    const carry = (a & b) << 1;   // where both bits are 1
    a = a ^ b;                    // sum ignoring carries
    b = carry;                    // fold the carry in next round
  }
  return a;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int getSum(int a, int b) {
        while (b != 0) {
            unsigned carry = (unsigned)(a & b) << 1;
            a = a ^ b;
            b = (int)carry;
        }
        return a;
    }
};
```

Solution — python

```python
class Solution:
    def getSum(self, a: int, b: int) -> int:
        MASK = 0xFFFFFFFF
        MAX = 0x7FFFFFFF
        while b != 0:
            carry = ((a & b) << 1) & MASK
            a = (a ^ b) & MASK
            b = carry
        # Python ints are unbounded — reinterpret as signed 32-bit
        return a if a <= MAX else ~(a ^ MASK)
```

Solution — java

```java
class Solution {
    public int getSum(int a, int b) {
        while (b != 0) {
            int carry = (a & b) << 1;
            a = a ^ b;
            b = carry;
        }
        return a;
    }
}
```

WALKTHROUGH

THE DECOMPOSITION OF BINARY ADDITION.

For each bit position, the result bit is the XOR of the inputs, and a carry is produced only where *both* inputs are 1 — that is the AND, shifted left one place to reach the next column. Adding the carry can itself produce new carries, so the process repeats; since each round pushes carries strictly leftwards, it terminates within the word width.

NEGATIVES WORK FOR FREE

because two's complement is designed so that the same bit-level addition is correct for signed values. `-1` is all ones, and `-1 + 1` ripples a carry off the top which is simply discarded — giving 0.

PYTHON NEEDS THE MASK

because its integers are arbitrary precision: the carry would keep growing left forever instead of falling off the end. Masking to 32 bits emulates a fixed-width register, and the final step reinterprets the pattern as signed. That contrast is a nice thing to be able to explain.

Linked lists: reversal and surgery

No random access, no length, only `next`. Every linked-list problem is pointer manipulation, and every linked-list bug is a pointer you overwrote before you had finished with it.

The pattern

```
THE THREE TECHNIQUES THAT SOLVE ALMOST EVERYTHING:

1. REVERSAL — three pointers, one pass
     prev = null, cur = head
     while (cur) {
         next = cur.next     <-- SAVE FIRST. Always.
         cur.next = prev     <-- rewire
         prev = cur          <-- advance
         cur = next
     }
     return prev             <-- the new head

     null <- 1    2 -> 3 -> null
             ^    ^
           prev  cur

2. DUMMY HEAD — removes every "what if it is the first
   node?" special case
     dummy = new ListNode(0, head)
     ... operate using `dummy` as a stable anchor ...
     return dummy.next

3. TWO POINTERS — fast/slow for the middle, cycles and
   "nth from the end" (chapter 18)

THE UNIVERSAL RULE:

  BEFORE YOU OVERWRITE A POINTER, SAVE WHAT YOU STILL
  NEED. Losing `cur.next` disconnects the rest of the
  list, and there is no way back — you cannot walk
  backwards.

THE FOUR-LINE REVERSAL IS WORTH MEMORISING COLD. It
appears inside palindrome checks, k-group reversal,
reordering, and addition problems. If you can write it
without thinking, a whole class of medium problems
becomes routine.

COSTS
  traverse / search      O(n)
  insert or delete at a KNOWN node   O(1)
  access the kth element O(n)     <-- no random access
  reverse                O(n) time, O(1) space
```

The helpers in this page

```
/* Pre-defined in the sandbox — you never write the
   plumbing (see chapter 03):                          */

function ListNode(val, next)     // the node constructor
buildList([1,2,3])               // array -> list, returns head
listToArray(head)                // list  -> array

/* Problems take and return ARRAYS; the judge converts to
   and from real lists for you. So you write real pointer
   code and get readable test output.                  */

/* --- THE DUMMY-HEAD IDIOM, in full --- */
function removeElements(head, val) {
    const dummy = new ListNode(0, head);
    let prev = dummy;                  // ALWAYS a valid node
    while (prev.next) {
        if (prev.next.val === val) prev.next = prev.next.next;
        else                       prev = prev.next;
    }
    return dummy.next;   // correct even if the head was removed
}
/* WITHOUT the dummy you need a separate branch for
   "the node to delete is the head", and that branch is
   where the bug lives. USE A DUMMY WHENEVER THE HEAD MIGHT
   CHANGE.

   NOTE the else: only advance when you did NOT delete.
   Advancing after a deletion skips the next node — the
   most common bug in this shape, caught by [7,7,7,7]
   removing 7.                                         */
```

Problems

How it shows up

Symptom	Cause
Only part of the list survives	Overwrote `cur.next` before saving it
Infinite loop when printing	Created a cycle — a node points back into the list
Consecutive duplicates survive	Advanced the pointer after a deletion — use `if/else`
Crash when the head is removed	No dummy node
Reversal returns the old head	Return `prev`, not `cur` (which is `null`)
Last node still points into the old list	Tail's `next` never set to `null`
Off by one on "nth from the end"	Gap between the pointers set to `n` instead of `n+1`
Works for odd lengths, fails for even	Middle definition — `slow` vs `slow.next`

Check yourself

- ✓ Write the three-pointer reversal from memory and say why `prev` is returned.
- ✓ State the universal rule about overwriting pointers.

- ✓ Explain what a dummy head removes, and when you need one.
- ✓ Explain why deletion uses `if/else` rather than always advancing.
- ✓ Give the complexity of access, search, and insert-at-known-node.
- ✓ Describe how to merge two sorted lists in O(1) extra space.
- ✓ Explain how to reverse only part of a list without breaking the joins.
- ✓ Say why recursion on a long list is a space risk.

Problems

Reverse a linked list `Easy`

Pattern: Three-pointer reversal

Given the `head` of a singly linked list, reverse it and return the new head.

The judge passes the list as an array and compares the returned list as an array.

Input	Output	Note
`head = [1,2,3,4,5]`	`[5,4,3,2,1]`	
`head = [1,2]`	`[2,1]`	
`head = []`	`[]`	

HINTS

1. You need three pointers: the node before, the current node, and the node after.
2. Save `cur.next` *before* you overwrite it — otherwise the rest of the list is lost.
3. When the loop ends `cur` is `null` and `prev` is the last node visited, which is the new head.

Approach	Idea	Time	Space
Copy to an array	Collect values, reverse, rebuild. Works, but O(n) extra space and misses the point.	`O(n)`	`O(n)`
Iterative three-pointer	Walk once, flipping each `next` as you go.	`O(n)`	`O(1)`
Recursive	Reverse the tail, then attach the head. Elegant, but O(n) stack — a real risk on long lists.	`O(n)`	`O(n)`

Solution — js

```
function reverseList(head) {
  let prev = null, cur = head;
  while (cur) {
    const next = cur.next;   // SAVE before overwriting
    cur.next = prev;         // flip the link
    prev = cur;              // advance both
    cur = next;
  }
  return prev;               // cur is null; prev is the new head
}
```

Solution — cpp

```cpp
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode *prev = nullptr, *cur = head;
        while (cur) {
            ListNode* next = cur->next;
            cur->next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }
};
```

Solution — python

```python
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        prev, cur = None, head
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        return prev
```

Solution — java

```java
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, cur = head;
        while (cur != null) {
            ListNode next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }
}
```

WALKTHROUGH

TRACE `[1,2,3]`.

Start `prev=null, cur=1`. Save `next=2`, set `1.next=null`, move to `prev=1, cur=2`. Save `next=3`, set `2.next=1`, move to `prev=2, cur=3`. Save `next=null`, set `3.next=2`, move to `prev=3, cur=null`. Loop ends; return 3. The list is `3→2→1→null`, and the original head correctly terminates the list.

RETURNING `CUR` IS THE CLASSIC SLIP

— it is always `null` when the loop exits. The empty-list test catches it only if you also handle `head = null`, which this code does for free since the loop body never runs and `prev` is already `null`.

Merge two sorted lists Easy

Pattern: Dummy head + two pointers

Given the heads of two sorted linked lists, splice them into one sorted list by rearranging the existing nodes, and return its head.

Input	Output	Note
`l1 = [1,2,4], l2 = [1,3,4]`	`[1,1,2,3,4,4]`	
`l1 = [], l2 = []`	`[]`	
`l1 = [], l2 = [0]`	`[0]`	

HINTS

1. Use a dummy head so you never special-case which list supplies the first node.
2. Keep a `tail` pointer at the end of the result and append the smaller front node each step.
3. When one list runs out, the other is already sorted — attach it wholesale rather than looping.

Approach	Idea	Time	Space
Collect and sort	Gather all values, sort, rebuild. Throws away the fact that the inputs are sorted.	`O(n log n)`	`O(n)`
Dummy head + merge	Classic two-pointer merge, relinking existing nodes.	`O(n + m)`	`O(1)`

Solution — js

```
function mergeTwoLists(l1, l2) {
  const dummy = new ListNode(0);
  let tail = dummy;

  while (l1 && l2) {
    if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
    else                  { tail.next = l2; l2 = l2.next; }
    tail = tail.next;
  }
  // exactly one list may remain; it is already sorted
  tail.next = l1 ?? l2;

  return dummy.next;
}
```

Solution — cpp

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
        ListNode dummy(0);
        ListNode* tail = &dummy;
        while (l1 && l2) {
            if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
            else                    { tail->next = l2; l2 = l2->next; }
            tail = tail->next;
        }
        tail->next = l1 ? l1 : l2;
        return dummy.next;
    }
};
```

Solution — python

```python
class Solution:
    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                tail.next, l1 = l1, l1.next
            else:
                tail.next, l2 = l2, l2.next
            tail = tail.next
        tail.next = l1 or l2
        return dummy.next
```

Solution — java

```java
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0), tail = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) { tail.next = l1; l1 = l1.next; }
            else                  { tail.next = l2; l2 = l2.next; }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

WALKTHROUGH

THE DUMMY EARNS ITS PLACE TWICE HERE:

it removes the "which list has the smaller head?" special case at the start, and it means the empty-input cases need no code at all — the loop never runs and `tail.next = l1 ?? l2` attaches whatever exists (possibly nothing).

`<=` RATHER THAN `<`

keeps the merge *stable*: equal elements from `l1` come first. It does not change correctness here but it is the right habit, and it matters in merge sort.

THIS IS THE MERGE STEP OF MERGE SORT

, which is why the same routine solves "merge k sorted lists" — either by repeated pairwise merging in a divide-and-conquer tree (O(N log k)) or with a heap (chapter 22).

Remove the nth node from the end `Medium`

Pattern: Dummy head + gap of n+1

Given the head of a linked list, remove the `n`-th node from the **end** and return the head. Do it in **one pass**.

Input	Output	Note
`head = [1,2,3,4,5], n = 2`	`[1,2,3,5]`	
`head = [1], n = 1`	`[]`	
`head = [1,2], n = 2`	`[2]`	Removes the head — the case a dummy makes free.

HINTS

1. To delete a node you must reach the node *before* it.
2. Advance a `fast` pointer `n+1` steps ahead of `slow`, starting both at the dummy. When `fast` falls off the end, `slow` is exactly one before the target.
3. The `+1` and starting at the dummy rather than the head are what make removing the first node work.

Approach	Idea	Time	Space
Two passes	Count the length, then walk `len - n` nodes. Perfectly correct and worth stating.	`O(n)`	`O(1)`
One pass, gap of n+1	Two pointers separated by `n+1`, both starting at the dummy.	`O(n)`	`O(1)`

Solution — js

```
function removeNthFromEnd(head, n) {
  const dummy = new ListNode(0, head);
  let fast = dummy, slow = dummy;

  // open a gap of n+1 so slow lands BEFORE the target
  for (let i = 0; i <= n; i++) fast = fast.next;

  while (fast) { fast = fast.next; slow = slow.next; }

  slow.next = slow.next.next;   // unlink
  return dummy.next;
}
```

Solution — cpp

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode dummy(0, head);
        ListNode *fast = &dummy, *slow = &dummy;
        for (int i = 0; i <= n; ++i) fast = fast->next;
        while (fast) { fast = fast->next; slow = slow->next; }
        slow->next = slow->next->next;
        return dummy.next;
    }
};
```

Solution — python

```
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        dummy = ListNode(0, head)
        fast = slow = dummy
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next
```

Solution — java

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head);
        ListNode fast = dummy, slow = dummy;
        for (int i = 0; i <= n; i++) fast = fast.next;
        while (fast != null) { fast = fast.next; slow = slow.next; }
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

WALKTHROUGH

WHY `N + 1` AND NOT `N`.

A gap of `n` leaves `slow` *on* the node to delete, and from there you cannot unlink it — you need its predecessor. One extra step puts `slow` one before the target. Starting both pointers at the `dummy` rather than at `head` is the other half: it gives the head itself a predecessor, so `[1], n=1` works with no special case.

CHECK THE LOOP BOUND.

`for (i = 0; i <= n; i++)` runs `n+1` times. Writing `<` is the off-by-one this problem is designed to expose; `[1,2], n=2` catches it.

Reverse a sublist `Medium`

Pattern: Dummy head + local reversal

Given the head of a list and two integers `left` and `right` with `1 <= left <= right <= n`, reverse the nodes from position `left` to position `right` (1-indexed) and return the head. One pass.

Input	Output	Note
`head = [1,2,3,4,5], left = 2, right = 4`	`[1,4,3,2,5]`	
`head = [5], left = 1, right = 1`	`[5]`	
`head = [3,5], left = 1, right = 2`	`[5,3]`	

HINTS

1. Walk to the node *before* position `left` and anchor there. A dummy makes this work when `left === 1`.
2. The head-insertion technique avoids tracking four separate pointers: repeatedly take the node after the current one and move it to the front of the reversed section.
3. Do exactly `right - left` moves.

Approach	Idea	Time	Space
Collect, reverse, rebuild	Extract the values in range, reverse them, write them back. Easy but O(n) extra space and not really a list problem.	`O(n)`	`O(n)`
Cut, reverse, rejoin	Remember the four boundary nodes, reverse the middle with the standard loop, then reconnect. Correct but easy to misjoin.	`O(n)`	`O(1)`

Approach	Idea	Time	Space
Head insertion in place	Anchor before `left`; move each following node to the front of the section. No rejoining needed.	`O(n)`	`O(1)`

Solution — js

```
function reverseBetween(head, left, right) {
  const dummy = new ListNode(0, head);
  let prev = dummy;

  // step to the node BEFORE position `left`
  for (let i = 1; i < left; i++) prev = prev.next;

  // head insertion: repeatedly move the node after `cur`
  // to the front of the reversed section
  const cur = prev.next;
  for (let i = 0; i < right - left; i++) {
    const moved = cur.next;      // the node to relocate
    cur.next = moved.next;       // unlink it
    moved.next = prev.next;      // put it at the front
    prev.next = moved;
  }
  return dummy.next;
}
```

Solution — cpp

```
class Solution {
public:
    ListNode* reverseBetween(ListNode* head, int left, int right) {
        ListNode dummy(0, head);
        ListNode* prev = &dummy;
        for (int i = 1; i < left; ++i) prev = prev->next;
        ListNode* cur = prev->next;
        for (int i = 0; i < right - left; ++i) {
            ListNode* moved = cur->next;
            cur->next = moved->next;
            moved->next = prev->next;
            prev->next = moved;
        }
        return dummy.next;
    }
};
```

Solution — python

```
class Solution:
    def reverseBetween(self, head: ListNode, left: int, right: int) -> ListNode:
        dummy = ListNode(0, head)
        prev = dummy
        for _ in range(left - 1):
            prev = prev.next
        cur = prev.next
        for _ in range(right - left):
            moved = cur.next
            cur.next = moved.next
            moved.next = prev.next
            prev.next = moved
        return dummy.next
```

Solution — java

```java
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode dummy = new ListNode(0, head), prev = dummy;
        for (int i = 1; i < left; i++) prev = prev.next;
        ListNode cur = prev.next;
        for (int i = 0; i < right - left; i++) {
            ListNode moved = cur.next;
            cur.next = moved.next;
            moved.next = prev.next;
            prev.next = moved;
        }
        return dummy.next;
    }
}
```

WALKTHROUGH

TRACE `[1,2,3,4,5], LEFT=2, RIGHT=4`.

`prev` stops at node 1, `cur` is node 2. Iteration 1: move node 3 to the front → `1→3→2→4→5`. Iteration 2: `cur` is still node 2, whose next is now 4; move it → `1→4→3→2→5`. Two iterations, which is `right - left`. Done.

WHY HEAD INSERTION BEATS CUT-AND-REJOIN.

The cut version needs four boundary pointers (before-left, the left node, the right node, after-right) and reconnecting them in the right order — a common source of lost tails. Head insertion keeps the list connected at every instant, so there is no rejoin step and nothing to lose. `cur` never moves; it simply drifts backwards as nodes are pulled in front of it, and ends up as the tail of the reversed section.

THE GENERALISATION:

reversing in groups of k ("reverse nodes in k-group") is this loop applied repeatedly, with a length check before each group.

Linked lists: fast and slow pointers

Two pointers moving at different speeds. It finds the middle in one pass, detects a cycle in O(1) space, and locates where the cycle begins — the last of which has a proof worth knowing.

The pattern

```
TWO POINTERS, ONE MOVING TWICE AS FAST.

  slow = head, fast = head
  while (fast && fast.next) {
      slow = slow.next;
      fast = fast.next.next;
  }

THE GUARD `fast && fast.next` IS BOTH CONDITIONS. Checking
only `fast` crashes on `fast.next.next` when fast is the
last node; checking only `fast.next` crashes when fast is
already null.

------------------------------------------------------------
USE 1 — FIND THE MIDDLE, one pass

  When fast reaches the end, slow is at the middle.

    1 -> 2 -> 3 -> 4 -> 5        odd:  slow lands on 3
    s    f
         s         f
              s              f=null -> slow = 3

    1 -> 2 -> 3 -> 4             even: slow lands on 3,
                                 the SECOND middle
  For the FIRST middle on even length, start
  `fast = head.next`. Decide which one the problem wants.

------------------------------------------------------------
USE 2 — CYCLE DETECTION (Floyd's tortoise and hare)

  If there is a cycle, fast eventually laps slow and they
  MEET. If there is none, fast runs off the end.

  WHY THEY MUST MEET: once both are inside the cycle,
  fast gains exactly one position on slow per step. The
  gap shrinks by 1 each step and cannot skip over zero,
  so it hits zero — they meet.

------------------------------------------------------------
USE 3 — FIND THE CYCLE'S START

  After they meet, reset one pointer to the head and
  advance BOTH ONE STEP AT A TIME. They meet at the entry.

     head ---- F ----> [ entry ]
                         |    ^
                         v    |
                         \____/   cycle length C

  THE PROOF, and it is short enough to give aloud:
    Let F = distance head -> entry,
        a = distance entry -> meeting point,
        C = cycle length.
    Slow travelled F + a. Fast travelled twice that, and
    is at the same place, so fast's distance is
    F + a + kC for some k >= 1.
      2(F + a) = F + a + kC
           F + a = kC
               F = kC - a
    So walking F steps from the head, and F steps forward
```

```
        from the meeting point, both land on the entry —
        because kC - a steps past the meeting point is exactly
        a whole number of laps back to the entry.

    COST: O(n) time, O(1) space — versus a hash set of visited
    nodes which is O(n) space. That space difference is the
    entire reason this pattern is asked.
```

When to reach for it

"Find the middle"

One pass instead of counting then walking half.

"Is there a cycle?" / "where does it start?"

Floyd's algorithm, O(1) space.

Palindrome linked list

Find the middle, reverse the second half, compare.

Duplicate number in an array

Treat `i → nums[i]` as a linked list; the duplicate creates a cycle.

Happy number

The digit-square sequence either reaches 1 or cycles.

Reorder / split a list

Middle first, then operate on each half.

Problems

How it shows up

Symptom	Cause
Crash reading `next` of null	Guard must be `fast && fast.next`, both
Middle is off by one on even length	Two valid definitions — start `fast` at `head` or `head.next`
Cycle detection never terminates	Moving both pointers at the same speed
Cycle start is wrong	Reset to head *after* the meeting, then move both by one
Reports a cycle on a clean list	Comparing values instead of node identity
Palindrome check fails on odd length	Middle node belongs to neither half — skip it
Input list left reversed	Restore the second half if the caller reuses the list

Check yourself

- ✓ Write the fast/slow loop and explain both halves of the guard.
- ✓ Say where `slow` lands for odd and even lengths, and how to shift it.
- ✓ Explain why fast and slow must meet inside a cycle.
- ✓ Give the proof that resetting to the head finds the cycle entry.
- ✓ State the space advantage over a visited set.
- ✓ Describe the palindrome algorithm in three steps.
- ✓ Explain how "find the duplicate number" becomes a cycle problem.

Problems

Middle of the linked list `Easy`

Pattern: Fast/slow

Given the head of a singly linked list, return the middle node. If there are two middle nodes, return the **second** one.

The judge compares the returned list, so returning the middle node means returning it *and everything after it*.

Input	Output	Note
`head = [1,2,3,4,5]`	`[3,4,5]`	
`head = [1,2,3,4,5,6]`	`[4,5,6]`	Second of the two middles.

HINTS

1. Counting the length then walking half is two passes. One pass is possible.
2. Move one pointer twice as fast as the other. When the fast one falls off the end, where is the slow one?
3. The loop guard must check both `fast` and `fast.next`.

Approach	Idea	Time	Space
Count then walk	Two passes: measure the length, then advance `n/2`.	`O(n)`	`O(1)`
Fast/slow	One pass; fast covers twice the ground.	`O(n)`	`O(1)`

Solution — js

```
function middleNode(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow;
}
```

Solution — cpp

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        ListNode *slow = head, *fast = head;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }
};
```

Solution — python

```
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

Solution — java

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

WALKTHROUGH

WHY THIS LANDS ON THE MIDDLE.

Each iteration advances `slow` by 1 and `fast` by 2, so `fast` is always at twice `slow`'s offset. When `fast` has travelled the whole list, `slow` has travelled exactly half of it.

THE TWO MIDDLE CONVENTIONS.

With an even count, starting both at `head` lands `slow` on the *second* middle (index 3 of 6, zero-based). Starting `fast = head.next` shifts it to the *first*. Neither is more correct — read the problem statement, and say which you chose.

Linked list cycle `Easy`

Pattern: Floyd's cycle detection

Given a linked list, determine whether it has a cycle.

Because the judge takes arrays, this problem is given as `values` plus `pos` — the index the tail connects to, or `-1` for no cycle. Build the list, wire the cycle, and return `true` or `false`.

Input	Output	Note
`values = [3,2,0,-4], pos = 1`	`true`	
`values = [1,2], pos = 0`	`true`	
`values = [1], pos = -1`	`false`	

HINTS

1. A visited-node hash set works and is O(n) space. The point of this problem is O(1) space.
2. Two pointers at different speeds: if there is a loop, the faster one laps the slower and they collide.
3. If there is no loop, the fast pointer simply reaches the end.
4. Compare *nodes*, not values — duplicate values do not mean a cycle.

Approach	Idea	Time	Space
Visited set	Record each node; a repeat means a cycle.	`O(n)`	`O(n)`
Floyd tortoise and hare	Fast gains one position per step, so it must catch up inside a loop.	`O(n)`	`O(1)`

Solution — js

```
function hasCycle(values, pos) {
  const head = buildList(values);
  if (head && pos >= 0) {
    let tail = head; while (tail.next) tail = tail.next;
    let target = head; for (let i = 0; i < pos; i++) target = target.next;
    tail.next = target;
  }

  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;   // node identity, not value
  }
  return false;                        // fast ran off the end
}
```

Solution — cpp

```
class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode *slow = head, *fast = head;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) return true;
        }
        return false;
    }
};
```

Solution — python

```python
class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:        # identity, not equality
                return True
        return False
```

Solution — java

```java
public class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

WALKTHROUGH

WHY THEY ARE GUARANTEED TO MEET.

Once both pointers are inside the cycle, consider the gap from `fast` to `slow` measured forwards around the loop. Each step `fast` advances 2 and `slow` advances 1, so the gap decreases by exactly 1 per step. A quantity that decreases by exactly one each step cannot jump over zero — so it reaches zero and the pointers coincide. That argument is the answer to "how do you know it terminates?"

IDENTITY, NOT EQUALITY.

`[1,1,1,1]` with no cycle must return `false`. Comparing `slow.val === fast.val` would report a cycle immediately. In Python this is the difference between `is` and `==`, which is exactly why the test is there.

THE CHECK GOES AFTER THE MOVES.

Both start at `head`, so testing before advancing would report a cycle on step zero for every list.

Linked list cycle II — find the entry `Medium`

Pattern: Floyd + reset to head

Given a linked list, return the **index** of the node where the cycle begins, or `-1` if there is no cycle.

As before the list is given as `values` and `pos`; return the index of the entry node, which should equal `pos`.

Input	Output	Note
`values = [3,2,0,-4], pos = 1`	`1`	
`values = [1,2], pos = 0`	`0`	
`values = [1], pos = -1`	`-1`	

HINTS

1. First detect the cycle exactly as before, and stop at the meeting point.
2. Then reset one pointer to the head and advance
   BOTH ONE STEP AT A TIME
   . They meet at the entry.
3. The reason is arithmetic: if F is head-to-entry and a is entry-to-meeting, then F = kC − a, so F steps from the head and F steps from the meeting point land on the same node.
4. Count steps from the head to get the index.

Approach	Idea	Time	Space
Visited set with indices	Record each node and its index; the first repeat is the entry.	`O(n)`	`O(n)`
Floyd's two phases	Detect the meeting point, then walk both pointers at speed 1 from head and meeting point.	`O(n)`	`O(1)`

Solution — js

```
function detectCycle(values, pos) {
  const head = buildList(values);
  if (head && pos >= 0) {
    let tail = head; while (tail.next) tail = tail.next;
    let target = head; for (let i = 0; i < pos; i++) target = target.next;
    tail.next = target;
  }

  // PHASE 1: detect, and stop at the meeting point
  let slow = head, fast = head, met = null;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) { met = slow; break; }
  }
  if (!met) return -1;

  // PHASE 2: head and meeting point advance at the SAME speed
  let p = head, idx = 0;
  while (p !== met) { p = p.next; met = met.next; idx++; }
  return idx;
}
```

Solution — cpp

```cpp
class Solution {
public:
    ListNode* detectCycle(ListNode* head) {
        ListNode *slow = head, *fast = head, *met = nullptr;
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
            if (slow == fast) { met = slow; break; }
        }
        if (!met) return nullptr;
        ListNode* p = head;
        while (p != met) { p = p->next; met = met->next; }
        return p;
    }
};
```

Solution — python

```python
class Solution:
    def detectCycle(self, head: ListNode) -> ListNode:
        slow = fast = head
        met = None
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                met = slow
                break
        if met is None:
            return None
        p = head
        while p is not met:
            p = p.next
            met = met.next
        return p
```

Solution — java

```java
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head, met = null;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) { met = slow; break; }
        }
        if (met == null) return null;
        ListNode p = head;
        while (p != met) { p = p.next; met = met.next; }
        return p;
    }
}
```

WALKTHROUGH

THE PROOF, WRITTEN OUT.

Let `F` be the number of steps from the head to the cycle entry, `a` the steps from the entry to the meeting point, and `C` the cycle length. When they meet, `slow` has taken `F + a` steps. `fast` has taken twice as many and is at the same node, so its distance is `F + a + kC` for some whole number of extra laps `k ≥ 1`. Setting them equal:

```
2(F + a) = F + a + kC
    F + a = kC
        F = kC − a
```

So advancing `F` steps from the *meeting point* covers `kC − a` steps, which combined with the `a` already travelled from the entry is exactly `k` whole laps — landing back on the entry. Meanwhile advancing `F` steps from the head also lands on the entry, by definition of `F`. Hence the two pointers meet there.

THE SINGLE-NODE SELF-LOOP

(`[1], pos = 0`) is the edge case: phase 1 meets immediately at the head, and phase 2 finds `p === met` without moving, returning index 0.

Palindrome linked list `Medium`

Pattern: Middle + reverse + compare

Given the head of a singly linked list, return `true` if it reads the same forwards and backwards.

Aim for O(n) time and **O(1) extra space**.

Input	Output	Note
`head = [1,2,2,1]`	`true`	
`head = [1,2]`	`false`	
`head = [1,2,3,2,1]`	`true`	

HINTS

1. Copying the values into an array and using two pointers is O(n) space — correct, and worth stating first.
2. For O(1) space: find the middle with fast/slow, reverse the second half, then walk the two halves in parallel.
3. With an odd length the middle node belongs to neither half; starting the comparison from `slow` handles it naturally.
4. Stop when the reversed half runs out — it is the shorter one.

Approach	Idea	Time	Space
Copy to an array	Then converge two pointers. Simple and clear.	`O(n)`	`O(n)`
Recursion	Compare the front against the unwinding stack. Still O(n) space, hidden in the call stack.	`O(n)`	`O(n)`

Approach	Idea	Time	Space
Reverse the second half	Find the middle, reverse from there, compare in parallel.	`O(n)`	`O(1)`

Solution — js

```
function isPalindrome(head) {
  if (!head || !head.next) return true;

  // 1. find the middle (slow ends on the second middle if even)
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
  }

  // 2. reverse from the middle onwards
  let prev = null, cur = slow;
  while (cur) {
    const next = cur.next;
    cur.next = prev;
    prev = cur;
    cur = next;
  }

  // 3. walk both halves; the reversed half is never longer
  let a = head, b = prev;
  while (b) {
    if (a.val !== b.val) return false;
    a = a.next;
    b = b.next;
  }
  return true;
}
```

Solution — cpp

```
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        if (!head || !head->next) return true;
        ListNode *slow = head, *fast = head;
        while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
        ListNode *prev = nullptr, *cur = slow;
        while (cur) { ListNode* nx = cur->next; cur->next = prev; prev = cur; cur = nx; }
        ListNode *a = head, *b = prev;
        while (b) {
            if (a->val != b->val) return false;
            a = a->next; b = b->next;
        }
        return true;
    }
};
```

Solution — python

```python
class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        if not head or not head.next:
            return True
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        prev, cur = None, slow
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        a, b = head, prev
        while b:
            if a.val != b.val:
                return False
            a, b = a.next, b.next
        return True
```

Solution — java

```java
class Solution {
    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        ListNode prev = null, cur = slow;
        while (cur != null) { ListNode nx = cur.next; cur.next = prev; prev = cur; cur = nx; }
        ListNode a = head, b = prev;
        while (b != null) {
            if (a.val != b.val) return false;
            a = a.next; b = b.next;
        }
        return true;
    }
}
```

WALKTHROUGH

WHY THE ODD CASE NEEDS NO SPECIAL HANDLING.

On `[1,2,3,2,1]`, `slow` stops on the `3`. Reversing from there gives `3 → 2 → 1` while the front half is still `1 → 2 → 3 …`. Comparing three pairs — 1/1, 2/2, 3/3 — succeeds, and the middle element compares against itself harmlessly. Looping while `b` is non-null rather than while both are, means the shorter reversed half controls termination and the front half's leftover nodes are ignored.

THE LIST IS LEFT PARTIALLY REVERSED.

That is a genuine side effect: if the caller uses the list afterwards it is corrupted. Interviewers often ask whether you noticed — the fix is to reverse the second half back before returning, which costs another O(n) pass and keeps the space at O(1).

Stacks: matching, expressions and design

Last in, first out. Whenever the most recent unresolved thing is the one you need next — a bracket, an operand, an undo step, a recursive call — the structure is a stack.

The pattern

```
THE OPERATIONS, all O(1):
  push(x)   put on top
  pop()     remove and return the top
  peek()    look at the top without removing
  isEmpty()

IN JAVASCRIPT an array IS a stack:
  st.push(x)      st.pop()      st[st.length - 1]

THE RECOGNITION SIGNALS:

  NESTING or MATCHING       brackets, tags, nested
                            structures
  "MOST RECENT" matters     undo, back button, call stack
  REVERSE something         a stack reverses order for free
  EXPRESSION EVALUATION     operands and operators
  RECURSION MADE ITERATIVE  the explicit stack IS the call
                            stack (chapter 23)
  NEXT GREATER / SMALLER    a MONOTONIC stack — chapter 20

------------------------------------------------------------
THE MATCHING TEMPLATE, which covers a whole family:

  for (const c of s) {
      if (isOpener(c)) st.push(c);
      else {
          if (st.length === 0) return false;   // nothing to
                                               // match
          if (!matches(st.pop(), c)) return false;
      }
  }
  return st.length === 0;      // NOTHING LEFT UNCLOSED

BOTH EMPTINESS CHECKS ARE NEEDED. Popping an empty stack
means a closer with no opener — ")(" fails here. A
non-empty stack at the end means an unclosed opener —
"(((" fails there. Miss either and half the tests pass.

------------------------------------------------------------
EXPRESSION EVALUATION

  POSTFIX (RPN): "3 4 + 2 *"
    push operands; on an operator, pop TWO, apply, push
    the result. ORDER MATTERS for - and /:
      b = pop(); a = pop(); push(a OP b);
    Popping in the wrong order silently breaks subtraction
    and division while addition still passes.

  INFIX with precedence: two stacks (operands and
  operators), or the shunting-yard algorithm. Rare in
  interviews; the postfix version is the common one.
```

When to reach for it

Balanced brackets / valid parentheses

The matching template.

Evaluate RPN, basic calculator

Operand stack; operator pops two.

Simplify a path, undo history

`..` pops, a name pushes.

Decode nested structures

`3[a2[c]]` — push the context on `[`, restore on `]`.

Iterative traversal

Replace recursion with an explicit stack (ch. 23).

Min stack / stack with extra queries

Store auxiliary data alongside each element.

Problems

How it shows up

Symptom	Cause
`"((("` reported valid	Missing the "stack empty at the end" check
`")("` reported valid	Missing the "pop from empty" check
Subtraction/division wrong in RPN	Popped operands in the wrong order
Crash on `pop()`	Empty stack — guard before popping
Min stack returns a stale minimum	Minimum not stored per element, or not popped alongside
Wrong mismatched pair accepted	Only counting brackets instead of checking the type
Stack overflow in recursion	Convert to an explicit stack

Check yourself

- ✓ Give the six recognition signals for a stack.
- ✓ Write the matching template and name both emptiness checks and what each catches.
- ✓ Explain the RPN pop order and which operators expose the bug.
- ✓ Say how a stack makes a recursive traversal iterative.
- ✓ Describe how to support O(1) `getMin` on a stack.
- ✓ Explain why an array is already a stack in most languages.

Problems

Valid parentheses Easy

Pattern: Matching template

Given a string containing only `()[]{}`, determine whether it is valid: every bracket is closed by the same type, and in the correct order.

Input	Output	Note
`s = "()[]{}"`	`true`	
`s = "(]"`	`false`	
`s = "([)]"`	`false`	Correct types but wrong nesting order.
`s = "{[]}"`	`true`	

HINTS

1. A closer must match the *most recent* unmatched opener — that is a stack.
2. Map each closer to its opener so the check is one lookup.
3. Two failure cases: a closer arriving with an empty stack, and openers left over at the end.

Approach	Idea	Time	Space
Repeatedly remove pairs	Delete `()`, `[]`, `{}` until nothing changes; valid if empty.	`O(n²)`	`O(n)`
Counters per type	**Wrong.** Counting cannot detect `([)]` — order is lost.	`O(n)`	`O(1)`
Stack	Push openers, match each closer against the top.	`O(n)`	`O(n)`

Solution — js

```
function isValid(s) {
  const pairs = { ")": "(", "]": "[", "}": "{" };
  const st = [];

  for (const c of s) {
    if (c === "(" || c === "[" || c === "{") {
      st.push(c);
    } else {
      // a closer: the top must be its matching opener
      if (st.pop() !== pairs[c]) return false;
    }
  }
  return st.length === 0;    // nothing left unclosed
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool isValid(string s) {
        unordered_map<char,char> pairs{{')','('}, {']','['}, {'}','{'}};
        stack<char> st;
        for (char c : s) {
            if (!pairs.count(c)) st.push(c);
            else {
                if (st.empty() || st.top() != pairs[c]) return false;
                st.pop();
            }
        }
        return st.empty();
    }
};
```

Solution — python

```python
class Solution:
    def isValid(self, s: str) -> bool:
        pairs = {")": "(", "]": "[", "}": "{"}
        st = []
        for c in s:
            if c not in pairs:
                st.append(c)
            else:
                if not st or st.pop() != pairs[c]:
                    return False
        return not st
```

Solution — java

```java
class Solution {
    public boolean isValid(String s) {
        Map<Character,Character> pairs = Map.of(')','(', ']','[', '}','{');
        Deque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (!pairs.containsKey(c)) st.push(c);
            else {
                if (st.isEmpty() || st.pop() != pairs.get(c)) return false;
            }
        }
        return st.isEmpty();
    }
}
```

WALKTHROUGH

THE JAVASCRIPT VERSION EXPLOITS A CONVENIENCE:

`[].pop()` on an empty array returns `undefined`, which can never equal a bracket character, so the empty check is implicit. In C++ and Java popping an empty stack is undefined or throws, so the explicit `st.empty()` guard is mandatory — note the difference between the versions.

WHY COUNTING FAILS.

A tempting solution counts openers and closers of each type and checks they balance. `([])` has one of each and balances perfectly, yet is invalid. Only a stack records the *order* of nesting, which is what the problem is really about — a good thing to say aloud when you dismiss the counting approach.

THE TWO FAILURE TESTS TO KEEP:

`"((("` for leftover openers and `")("` for a closer with nothing to match.

Min stack `Medium`

Pattern: Store auxiliary data per element

Design a stack supporting `push`, `pop`, `top` and `getMin` — where `getMin` returns the smallest element currently in the stack — **all in O(1)**.

Implement `minStack(ops, args)`: apply each operation in order and return an array of the results of the query operations (`"top"` and `"getMin"`), in order.

Input	Output	Note
`ops = ["push","push","push","getMin","pop","top","getMin"], args = [[-2],[0],[-3],[],[],[],[]]`	`[-3,0,-2]`	

HINTS

1. Scanning for the minimum on every call is O(n). You need it precomputed.
2. Alongside each pushed value, record the minimum of the stack *at that moment*.
3. Then popping automatically restores the previous minimum — no recomputation.
4. The new minimum on push is `min(value, currentMin)`.

Approach	Idea	Time	Space
Scan on getMin	Walk the stack each time. Simple but O(n) per query.	`O(n) per getMin`	`O(n)`
Second stack of minima	Push to the min-stack only when a new value is ≤ the current minimum. Slightly less memory.	`O(1)`	`O(n)`
Pair (value, minSoFar)	Store both in each entry. Simplest to reason about and impossible to desynchronise.	`O(1)`	`O(n)`

Solution — js

```javascript
function minStack(ops, args) {
  const st = [];        // each entry: [value, minSoFar]
  const out = [];

  for (let i = 0; i < ops.length; i++) {
    const op = ops[i];
    if (op === "push") {
      const v = args[i][0];
      const prevMin = st.length ? st[st.length - 1][1] : Infinity;
      st.push([v, Math.min(v, prevMin)]);
    } else if (op === "pop") {
      st.pop();                       // the old min comes back
    } else if (op === "top") {
      out.push(st[st.length - 1][0]);
    } else if (op === "getMin") {
      out.push(st[st.length - 1][1]);
    }
  }
  return out;
}
```

Solution — cpp

```cpp
class MinStack {
    vector<pair<int,int>> st;   // (value, minSoFar)
public:
    void push(int v) {
        int m = st.empty() ? v : min(v, st.back().second);
        st.push_back({v, m});
    }
    void pop()      { st.pop_back(); }
    int  top()      { return st.back().first; }
    int  getMin()   { return st.back().second; }
};
```

Solution — python

```python
class MinStack:
    def __init__(self):
        self.st = []          # (value, min_so_far)

    def push(self, v: int) -> None:
        m = v if not self.st else min(v, self.st[-1][1])
        self.st.append((v, m))

    def pop(self) -> None:
        self.st.pop()

    def top(self) -> int:
        return self.st[-1][0]

    def getMin(self) -> int:
        return self.st[-1][1]
```

Solution — java

```
class MinStack {
    private final Deque<int[]> st = new ArrayDeque<>();  // {value, minSoFar}
    public void push(int v) {
        int m = st.isEmpty() ? v : Math.min(v, st.peek()[1]);
        st.push(new int[]{v, m});
    }
    public void pop()    { st.pop(); }
    public int  top()    { return st.peek()[0]; }
    public int  getMin() { return st.peek()[1]; }
}
```

WALKTHROUGH

THE INSIGHT IS THAT THE MINIMUM IS A PROPERTY OF A *PREFIX*, NOT OF THE WHOLE STRUCTURE.

When the stack holds `[a, b, c]`, the minimum is `min(a,b,c)`; after popping `c` it is `min(a,b)` — a value that existed earlier and can simply be remembered rather than recomputed. Storing it per entry makes pop restore it for free.

WHY THE ALTERNATIVE TWO-STACK VERSION NEEDS `<=`.

If you only push to the min-stack when a value is *strictly* smaller, duplicates of the minimum break it: pushing `2, 2` then popping once would remove the only recorded 2 and report the wrong minimum. The test `[2,2,1]` is there to catch exactly that. Storing the pair per entry sidesteps the issue entirely, which is why it is the recommended version.

Evaluate reverse Polish notation `Medium`

Pattern: Operand stack

Evaluate an arithmetic expression in Reverse Polish Notation. Valid operators are `+ - * /`; each operand is an integer or another expression.

Division truncates **toward zero**. The input is always valid.

Input	Output	Note
`tokens = ["2","1","+","3","*"]`	`9`	(2 + 1) × 3
`tokens = ["4","13","5","/","+"]`	`6`	4 + (13 / 5) = 4 + 2
`tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]`	`22`	

HINTS

1. Push every number. When you meet an operator, pop two values, combine them, push the result.
2. The
   SECOND
   value popped is the left operand — the stack reverses the order.
3. Division must truncate toward zero, which is `Math.trunc`, not `Math.floor` (they differ for negatives).

Approach	Idea	Time	Space
Recursive parse	Parse right to left building a tree, then evaluate. Works, but more machinery than needed.	`O(n)`	`O(n)`

Approach	Idea	Time	Space
Operand stack	One linear pass; the stack holds pending operands.	`O(n)`	`O(n)`

Solution — js

```
function evalRPN(tokens) {
  const st = [];
  const ops = {
    "+": (a, b) => a + b,
    "-": (a, b) => a - b,
    "*": (a, b) => a * b,
    "/": (a, b) => Math.trunc(a / b)   // toward ZERO, not -inf
  };

  for (const t of tokens) {
    if (t in ops) {
      const b = st.pop();      // RIGHT operand pops first
      const a = st.pop();      // LEFT operand second
      st.push(ops[t](a, b));
    } else {
      st.push(Number(t));
    }
  }
  return st.pop();
}
```

Solution — cpp

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<long> st;
        for (auto& t : tokens) {
            if (t == "+" || t == "-" || t == "*" || t == "/") {
                long b = st.top(); st.pop();
                long a = st.top(); st.pop();
                if      (t == "+") st.push(a + b);
                else if (t == "-") st.push(a - b);
                else if (t == "*") st.push(a * b);
                else               st.push(a / b);   // C++ truncates toward 0
            } else st.push(stol(t));
        }
        return (int)st.top();
    }
};
```

Solution — python

```python
class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        st = []
        for t in tokens:
            if t in ("+", "-", "*", "/"):
                b = st.pop()
                a = st.pop()
                if   t == "+": st.append(a + b)
                elif t == "-": st.append(a - b)
                elif t == "*": st.append(a * b)
                else:          st.append(int(a / b))  # toward zero
            else:
                st.append(int(t))
        return st.pop()
```

Solution — java

```java
class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> st = new ArrayDeque<>();
        for (String t : tokens) {
            switch (t) {
                case "+": { int b = st.pop(), a = st.pop(); st.push(a + b); break; }
                case "-": { int b = st.pop(), a = st.pop(); st.push(a - b); break; }
                case "*": { int b = st.pop(), a = st.pop(); st.push(a * b); break; }
                case "/": { int b = st.pop(), a = st.pop(); st.push(a / b); break; }
                default:  st.push(Integer.parseInt(t));
            }
        }
        return st.pop();
    }
}
```

WALKTHROUGH

POP ORDER IS THE BUG THIS PROBLEM EXISTS TO EXPOSE.

For `["7","2","-"]` the stack is `[7, 2]`; the first pop yields 2 and the second yields 7. Since the expression means `7 - 2`, the *second* pop is the left operand. Getting it backwards returns `-5` — and addition and multiplication still pass, so half the tests look fine. Both orderings are in the test list deliberately.

`MATH.TRUNC` VERSUS `MATH.FLOOR`.

For `-7 / 2`, truncation toward zero gives `-3` while flooring gives `-4`. The problem specifies toward zero, matching C++ and Java integer division. In Python, `//` floors, so the idiomatic solution uses `int(a / b)` instead — a real cross-language trap worth knowing.

Stacks: the monotonic stack

A stack kept sorted by construction. It answers "what is the next larger element?" for every position in one linear pass, and it is the key to several problems that look inescapably quadratic.

The pattern

```
A MONOTONIC STACK holds elements in sorted order —
increasing or decreasing from the bottom. You maintain
that by POPPING anything that would break it before
pushing.

THE RECOGNITION SIGNAL:
  "NEXT GREATER" / "NEXT SMALLER" / "PREVIOUS GREATER" /
  "PREVIOUS SMALLER" element — for EVERY position.
  Also: largest rectangle, span problems, trapping water,
  and "how far until something bigger?"

------------------------------------------------------
THE TEMPLATE — next greater to the RIGHT

  const out = new Array(n).fill(-1);
  const st = [];                      // holds INDICES
  for (let i = 0; i < n; i++) {
      while (st.length && a[st[st.length-1]] < a[i]) {
          out[st.pop()] = a[i];       // a[i] IS the answer
      }                               // for the popped index
      st.push(i);
  }
  // anything still on the stack has no greater element

  THE STACK HOLDS INDICES, NOT VALUES. You almost always
  need the position — to write the answer, or to compute
  a width.

  [2, 1, 2, 4, 3]
   i=0  push 0                st=[0]        (values 2)
   i=1  1 < 2, push 1         st=[0,1]      (2,1)
   i=2  2 > 1 -> out[1]=2     st=[0]
        2 = 2 not >, push 2   st=[0,2]      (2,2)
   i=3  4 > 2 -> out[2]=4
        4 > 2 -> out[0]=4     st=[]
        push 3                st=[3]        (4)
   i=4  3 < 4, push 4         st=[3,4]
   leftover 3,4 -> out = [4,2,4,-1,-1]

------------------------------------------------------
THE FOUR VARIANTS — only the comparison changes

  next GREATER right   iterate LEFT->RIGHT, pop while
                       stack top <  current   (decreasing
                                               stack)
  next SMALLER right   pop while stack top >  current
  prev GREATER left    iterate LEFT->RIGHT, the stack top
                       AFTER popping is the answer
  prev SMALLER left    same, flipped comparison

  Equivalently, iterate right-to-left for the "next"
  versions and read the top instead. Pick one convention
  and keep it.

------------------------------------------------------
WHY IT IS O(n) DESPITE THE NESTED WHILE

  EACH INDEX IS PUSHED EXACTLY ONCE AND POPPED AT MOST
  ONCE. So the total number of pops across the whole run
  is at most n, no matter how many happen in a single
  iteration. Amortised O(1) per element (chapter 02).
```

```
  THAT SENTENCE IS THE ANSWER TO "what is the
  complexity?" — say it exactly.

-------------------------------------------------------
THE >= VERSUS > DECISION

  Popping on EQUAL values or not changes which duplicate
  gets credited. For "next STRICTLY greater", pop while
  top < current (leave equals on the stack). For
  rectangle problems either works, because equal-height
  bars produce the same area — but be able to say why.
```

When to reach for it

Next / previous greater or smaller

Daily temperatures, next greater element, stock span.

Largest rectangle in a histogram

Each bar extends until a shorter one on each side.

Maximal rectangle in a binary matrix

Row by row, each becomes a histogram problem.

Trapping rain water

An alternative to the two-pointer solution (ch. 04).

Remove k digits / smallest subsequence

Greedily pop larger digits to make the result smaller.

Sum of subarray minimums

Count how many subarrays each element is the minimum of.

Problems

How it shows up

Symptom	Cause
Cannot compute widths	Stack holds values instead of indices
Leftover elements have no answer	Correct — they have no next greater; initialise to −1 or 0
Duplicates credited to the wrong index	`<` vs `<=` in the pop condition
Histogram misses the last bars	No sentinel — append a height of 0, or drain after the loop
Width off by one	Width is `i - st.top() - 1` after popping, not `i - popped`
Claimed O(n^2) complexity	It is amortised O(n) — each index pushed and popped once
Answer array wrong length	Must be sized n and pre-filled with the "none" value

Check yourself

- ✓ State the recognition signal and name four problems that match it.
- ✓ Write the next-greater template and say why the stack holds indices.
- ✓ Give the amortised argument for O(n) in one sentence.
- ✓ Explain what stays on the stack at the end and what it means.
- ✓ Describe the four variants and what changes between them.
- ✓ Explain the sentinel trick for the histogram problem.
- ✓ Give the correct width formula after a pop.

Problems

Daily temperatures `Medium`

Pattern: Next greater element

Given an array `temperatures`, return an array `answer` where `answer[i]` is the number of days you must wait after day `i` to get a warmer temperature. If no warmer day exists, put `0`.

Input	Output	Note
`temperatures = [73,74,75,71,69,72,76,73]`	`[1,1,4,2,1,1,0,0]`	
`temperatures = [30,40,50,60]`	`[1,1,1,0]`	
`temperatures = [30,60,90]`	`[1,1,0]`	

HINTS

1. This is "next strictly greater element", reported as a *distance* rather than a value.
2. Keep a stack of indices whose answer is still unknown, in decreasing temperature order.
3. When today is warmer than the top of the stack, today resolves it — the wait is the index difference.
4. Anything left on the stack at the end never gets a warmer day, so it keeps its 0.

Approach	Idea	Time	Space
For each day scan forward	Look ahead until a warmer day.	`O(n²)`	`O(1)`
Monotonic decreasing stack	Each index is pushed once and popped at most once.	`O(n)`	`O(n)`

Solution — js

```javascript
function dailyTemperatures(temperatures) {
  const n = temperatures.length;
  const out = new Array(n).fill(0);
  const st = [];   // indices awaiting a warmer day

  for (let i = 0; i < n; i++) {
    // today resolves every colder pending day
    while (st.length && temperatures[st[st.length - 1]] < temperatures[i]) {
      const j = st.pop();
      out[j] = i - j;          // the WAIT, not the temperature
    }
    st.push(i);
  }
  return out;                  // leftovers keep their 0
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& t) {
        int n = t.size();
        vector<int> out(n, 0);
        stack<int> st;
        for (int i = 0; i < n; ++i) {
            while (!st.empty() && t[st.top()] < t[i]) {
                int j = st.top(); st.pop();
                out[j] = i - j;
            }
            st.push(i);
        }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
        n = len(temperatures)
        out = [0] * n
        st = []
        for i, t in enumerate(temperatures):
            while st and temperatures[st[-1]] < t:
                j = st.pop()
                out[j] = i - j
            st.append(i)
        return out
```

Solution — java

```
class Solution {
    public int[] dailyTemperatures(int[] t) {
        int n = t.length;
        int[] out = new int[n];
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && t[st.peek()] < t[i]) {
                int j = st.pop();
                out[j] = i - j;
            }
            st.push(i);
        }
        return out;
    }
}
```

WALKTHROUGH

WHY STORING INDICES IS ESSENTIAL HERE.

The answer is a *distance*, so when day `i` resolves day `j` you need `j` itself to compute `i - j`. A stack of temperatures could tell you *that* a warmer day arrived but not how long the wait was.

THE STRICT `<` MATTERS FOR TIES.

`[50,50,60]` must give `[2,1,0]`: neither 50 resolves the other, because "warmer" means strictly greater. Using `<=` would pop the first 50 when the second arrives and wrongly report a wait of 1.

COMPLEXITY, SAID PROPERLY:

the `while` looks like it could make this quadratic, but every index enters the stack exactly once and leaves at most once, so the total pop count over the whole run is bounded by `n`. The algorithm is O(n) amortised — the same argument as the sliding window in chapter 05.

Next greater element in a circular array `Medium`

Pattern: Monotonic stack over a doubled array

Given a **circular** integer array `nums`, return the next greater number for every element. The next greater of `nums[i]` is the first larger value encountered when moving forwards, wrapping around to the start. If none exists, return `-1` for that position.

Input	Output	Note
`nums = [1,2,1]`	`[2,-1,2]`	The last 1 wraps around to find 2.
`nums = [1,2,3,4,3]`	`[2,3,4,-1,4]`	

HINTS

1. Handle the wrap by pretending the array is traversed twice — indices `0 .. 2n-1`, using `i % n`.
2. Do not actually build a doubled array; the modulo is enough.
3. On the second pass, only *resolve* pending indices — do not push new ones, or leftovers get answers they should not have.

Approach	Idea	Time	Space
For each i scan around	Walk up to n−1 positions forwards from every index.	`O(n²)`	`O(1)`
Monotonic stack, two passes	Iterate `2n` times with `i % n`; push only during the first pass.	`O(n)`	`O(n)`

Solution — js

```
function nextGreaterElements(nums) {
  const n = nums.length;
  const out = new Array(n).fill(-1);
  const st = [];                   // indices in 0..n-1

  // walk the array TWICE to cover the wrap-around
  for (let i = 0; i < 2 * n; i++) {
    const v = nums[i % n];
    while (st.length && nums[st[st.length - 1]] < v) {
      out[st.pop()] = v;
    }
    // only push during the FIRST pass
    if (i < n) st.push(i);
  }
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        int n = nums.size();
        vector<int> out(n, -1);
        stack<int> st;
        for (int i = 0; i < 2 * n; ++i) {
            int v = nums[i % n];
            while (!st.empty() && nums[st.top()] < v) { out[st.top()] = v; st.pop(); }
            if (i < n) st.push(i);
        }
        return out;
    }
};
```

Solution — python

```
class Solution:
    def nextGreaterElements(self, nums: list[int]) -> list[int]:
        n = len(nums)
        out = [-1] * n
        st = []
        for i in range(2 * n):
            v = nums[i % n]
            while st and nums[st[-1]] < v:
                out[st.pop()] = v
            if i < n:
                st.append(i)
        return out
```

Solution — java

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] out = new int[n];
        Arrays.fill(out, -1);
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < 2 * n; i++) {
            int v = nums[i % n];
            while (!st.isEmpty() && nums[st.peek()] < v) out[st.pop()] = v;
            if (i < n) st.push(i);
        }
        return out;
    }
}
```

WALKTHROUGH

WHY TWO PASSES SUFFICE.

The next greater element, if it exists, is at most `n-1` positions ahead. Traversing the array twice therefore gives every index a full lap of lookahead, and a third pass could add nothing.

THE `IF (I < N)` GUARD IS THE WHOLE TRICK.

During the second pass the stack must only be *drained*, never refilled. Pushing again would let a second-pass index receive an answer from a third conceptual lap that does not exist, and would also leave stale entries. Elements still on the stack after both passes genuinely have no greater element anywhere in the circle, so they keep their `-1`.

NOTE THE ANSWER ARRAY STORES VALUES HERE

, unlike the previous problem which stored distances — but the *stack* still holds indices, because that is what you need to write into `out`.

Largest rectangle in a histogram `Hard`

Pattern: Monotonic increasing stack + width from neighbours

Given an array `heights` of bar heights, each of width 1, find the area of the largest rectangle that fits inside the histogram.

Input	Output	Note
`heights = [2,1,5,6,2,3]`	`10`	Bars of height 5 and 6 give 5 × 2 = 10.
`heights = [2,4]`	`4`	

HINTS

1. For each bar, the largest rectangle *using that bar as the limiting height* extends left and right until a strictly shorter bar.
2. So for every bar you need the previous smaller and the next smaller — both are monotonic-stack queries.
3. Keep an increasing stack. When a shorter bar arrives, the popped bar's rectangle is bounded on the right by the current index and on the left by the new stack top.
4. The width after popping index `j` is `i - st.top() - 1`, or `i` when the stack is empty.
5. Append a sentinel height of 0 so every remaining bar gets flushed.

Approach	Idea	Time	Space
All pairs	For every (left, right) pair, find the minimum height and multiply.	`O(n²)`	`O(1)`
Divide and conquer	Split at the minimum bar. O(n log n) average, $O(n^2)$ on a sorted histogram.	`O(n log n)`	`O(n)`
Monotonic increasing stack	One pass; each bar is popped once and its full rectangle computed then.	`O(n)`	`O(n)`

Solution — js

```
function largestRectangleArea(heights) {
  const st = [];       // indices with INCREASING heights
  let best = 0;
  const n = heights.length;

  // i === n acts as a sentinel bar of height 0, flushing the stack
  for (let i = 0; i <= n; i++) {
    const cur = i === n ? 0 : heights[i];

    while (st.length && heights[st[st.length - 1]] >= cur) {
      const h = heights[st.pop()];
      // left bound is the new top (exclusive); right bound is i
      const width = st.length ? i - st[st.length - 1] - 1 : i;
      best = Math.max(best, h * width);
    }
    st.push(i);
  }
  return best;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<int> st;
        int best = 0, n = heights.size();
        for (int i = 0; i <= n; ++i) {
            int cur = (i == n) ? 0 : heights[i];
            while (!st.empty() && heights[st.top()] >= cur) {
                int h = heights[st.top()]; st.pop();
                int width = st.empty() ? i : i - st.top() - 1;
                best = max(best, h * width);
            }
            st.push(i);
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def largestRectangleArea(self, heights: list[int]) -> int:
        st = []
        best = 0
        n = len(heights)
        for i in range(n + 1):
            cur = 0 if i == n else heights[i]
            while st and heights[st[-1]] >= cur:
                h = heights[st.pop()]
                width = i - st[-1] - 1 if st else i
                best = max(best, h * width)
            st.append(i)
        return best
```

Solution — java

```java
class Solution {
    public int largestRectangleArea(int[] heights) {
        Deque<Integer> st = new ArrayDeque<>();
        int best = 0, n = heights.length;
        for (int i = 0; i <= n; i++) {
            int cur = (i == n) ? 0 : heights[i];
            while (!st.isEmpty() && heights[st.peek()] >= cur) {
                int h = heights[st.pop()];
                int width = st.isEmpty() ? i : i - st.peek() - 1;
                best = Math.max(best, h * width);
            }
            st.push(i);
        }
        return best;
    }
}
```

WALKTHROUGH

THE REFRAMING.

Instead of enumerating rectangles, ask for each bar: "if *this* bar is the shortest one in the rectangle, how wide can the rectangle be?" It extends left until a strictly shorter bar and right until a strictly shorter bar. Every maximal rectangle has some shortest bar, so considering each bar in this role covers all of them.

WHY THE STACK GIVES BOTH BOUNDS AT ONCE.

A bar is popped exactly when a shorter bar arrives — that arrival index `i` is its right bound. And because the stack is increasing, whatever is beneath it is the nearest shorter bar to its left. So the width is `i - st.top() - 1`: the span strictly between the two shorter neighbours. When the stack empties, no shorter bar exists to the left, so the rectangle reaches index 0 and the width is simply `i`.

THE SENTINEL.

Running the loop to `i === n` with a virtual height of 0 forces every remaining bar to be popped and measured. Without it, a monotonically increasing histogram like `[1,2,3,4,5]` never pops anything and returns 0.

WHY `>=` RATHER THAN `>`.

With equal heights the popped bar computes a width that stops short, but the *later* equal bar will be popped afterwards and will measure the full span — so the maximum is still found. `[2,2,2]` returns 6 either way; using `>=` just keeps the stack smaller.

Queues: BFS, deques and the monotonic deque

First in, first out — the structure behind every shortest-path-in-an-unweighted-graph problem. Add the ability to push and pop at both ends and you get the sliding-window maximum in linear time.

The pattern

```
QUEUE — enqueue at the back, dequeue from the front.
Its defining property: elements come out in the order
they went in, so BFS explores strictly by DISTANCE.

  THE JAVASCRIPT TRAP: Array.shift() is O(n) because it
  reindexes the whole array. A "linear" BFS built on
  shift() is really O(n^2).

  THE FIX — a head index, never shrinking the array:
    const q = [start];
    let head = 0;
    while (head < q.length) {
        const node = q[head++];
        ...
        q.push(neighbour);
    }
  Memory is not reclaimed until the loop ends, which is
  fine for interview-sized inputs and is what a real
  implementation would do with a ring buffer.

------------------------------------------------------------
DEQUE — push/pop at BOTH ends, all O(1).

  pushFront  popFront
  pushBack   popBack

  C++    std::deque
  Java   ArrayDeque
  Python collections.deque
  JS     an array, using push/pop/shift/unshift —
         with the same O(n) caveat at the front

------------------------------------------------------------
THE MONOTONIC DEQUE — the pattern worth learning here.

PROBLEM: the maximum of every window of size k.
Recomputing the max per window is O(n·k). A heap gives
O(n log k). A deque gives O(n).

THE IDEA: keep indices in the deque so their VALUES are
DECREASING. Then the front is always the window maximum.

  for i in 0..n-1:
     1. DROP FROM THE FRONT anything that has slid out of
        the window:      while (dq[0] <= i - k) popFront
     2. DROP FROM THE BACK anything smaller than a[i] —
        it can never be the maximum again, because a[i]
        is both LARGER and LATER:
                         while (a[dq.back] <= a[i]) popBack
     3. pushBack(i)
     4. once i >= k-1, a[dq[0]] is the window maximum

  [1, 3, -1, -3, 5, 3, 6, 7]   k = 3
   i=0  dq=[0]                 (1)
   i=1  3>1 pop 0, dq=[1]      (3)
   i=2  -1<3 push, dq=[1,2]    (3,-1)     window max 3
   i=3  -3<-1 push, dq=[1,2,3]            window max 3
   i=4  5 pops 3,2,1, dq=[4]              window max 5
   ...                          -> [3,3,5,5,6,7]

WHY IT IS O(n): every index is pushed once and popped at
```

```
    most once. The same amortised argument as the monotonic
    stack (chapter 20).

    THE KEY INSIGHT, worth saying aloud: an element that is
    smaller than a later element is USELESS FOREVER — the
    later one outlives it in every future window and is
    bigger. So it can be discarded immediately.
```

When to reach for it

Shortest path, unweighted

BFS. The first time you reach a node is via a shortest path.

Level-by-level processing

Tree levels, rotting oranges, multi-source spread.

Sliding window max/min

Monotonic deque, O(n).

"Most recent k" / round robin

Plain queue or ring buffer.

Palindrome check on a stream

Deque, comparing both ends.

0-1 BFS

Deque: push weight-0 edges to the front, weight-1 to the back.

Problems

How it shows up

Symptom	Cause
BFS is unexpectedly slow	`shift()` is O(n) — use a head index
Window maximum is stale	Expired indices not dropped from the front
Deque grows without bound	Smaller elements not popped from the back
Off by one on the first window	Record only once `i >= k - 1`
Deque holds values, cannot expire them	Store **indices** so you can test the window bound
Duplicates dropped incorrectly	`<=` vs `<` when popping the back — either works for max, but be consistent
BFS revisits nodes forever	Mark visited when *enqueueing*, not when dequeuing

Check yourself

- ✓ Say why a queue gives shortest paths in an unweighted graph.
- ✓ Explain the `shift()` trap and the head-index fix.
- ✓ Write the monotonic deque loop and its four steps.
- ✓ State the insight that justifies popping from the back.
- ✓ Give the amortised O(n) argument.
- ✓ Say why the deque stores indices rather than values.
- ✓ Explain why visited is marked at enqueue time.

Problems

Sliding window maximum `Hard`

Pattern: Monotonic decreasing deque

Given an array `nums` and a window size `k`, return an array of the maximum value in each window as it slides from left to right.

Input	Output	Note
`nums = [1,3,-1,-3,5,3,6,7], k = 3`	`[3,3,5,5,6,7]`	
`nums = [1], k = 1`	`[1]`	

HINTS

1. Recomputing the maximum per window is O(n·k). A max-heap gets you O(n log k). Linear is possible.
2. Keep a deque of *indices* whose values are decreasing, so the front is always the current maximum.
3. Before using the front, drop indices that have slid out of the window.
4. Before pushing `i`, drop from the back every index whose value is ≤ `nums[i]` — they can never be the maximum again.

Approach	Idea	Time	Space
Max per window	Scan each window.	`O(n·k)`	`O(1)`
Max-heap with lazy deletion	Push (value, index); pop the top while its index is expired.	`O(n log n)`	`O(n)`
Monotonic deque	Each index enters and leaves once.	`O(n)`	`O(k)`

Solution — js

```javascript
function maxSlidingWindow(nums, k) {
  const out = [];
  const dq = [];        // indices; nums[dq] is DECREASING
  let head = 0;         // front index into dq (avoids shift())

  for (let i = 0; i < nums.length; i++) {
    // 1. drop indices that have slid out of the window
    while (head < dq.length && dq[head] <= i - k) head++;

    // 2. drop smaller values from the back — they are useless:
    //    nums[i] is both larger AND later
    while (head < dq.length && nums[dq[dq.length - 1]] <= nums[i]) dq.pop();

    // 3. this index is a candidate
    dq.push(i);

    // 4. once the first full window exists, the front is the max
    if (i >= k - 1) out.push(nums[dq[head]]);
  }
  return out;
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        vector<int> out;
        deque<int> dq;
        for (int i = 0; i < (int)nums.size(); ++i) {
            while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
            while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
            dq.push_back(i);
            if (i >= k - 1) out.push_back(nums[dq.front()]);
        }
        return out;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        out = []
        dq = deque()          # indices, values decreasing
        for i, v in enumerate(nums):
            while dq and dq[0] <= i - k:
                dq.popleft()
            while dq and nums[dq[-1]] <= v:
                dq.pop()
            dq.append(i)
            if i >= k - 1:
                out.append(nums[dq[0]])
        return out
```

Solution — java

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums.length == 0 || k == 0) return new int[0];
        int[] out = new int[nums.length - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        int w = 0;
        for (int i = 0; i < nums.length; i++) {
            while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
            while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast();
            dq.addLast(i);
            if (i >= k - 1) out[w++] = nums[dq.peekFirst()];
        }
        return out;
    }
}
```

WALKTHROUGH

THE DISCARD RULE IS THE WHOLE ALGORITHM.

If `nums[j] ≤ nums[i]` and `j < i`, then index `j` is worthless: every window that still contains `j` also contains `i` (since `i` is later), and `i` holds a value at least as large. So `j` can never be a window maximum again and is dropped immediately. What survives is a decreasing sequence, whose front is by construction the largest value still in play.

TWO DIFFERENT REMOVALS, TWO DIFFERENT REASONS.

The front is popped because of *time* — the index has aged out of the window. The back is popped because of *value* — it has been dominated. Conflating them is the usual bug.

WHY INDICES AND NOT VALUES.

Expiring the front requires knowing *where* an element came from; a deque of raw values cannot tell whether its front is still inside the window.

THE `HEAD` POINTER IN THE JAVASCRIPT VERSION

avoids `shift()`, keeping the whole thing genuinely O(n) rather than O(n·k) in disguise — see the pattern notes above.

Implement a stack using queues `Easy`

Pattern: Structure conversion

Implement a LIFO stack using only FIFO queue operations: push to the back, pop from the front, peek the front, size and empty.

Implement `stackOps(ops, args)`: apply each operation and return the results of the query operations (`"pop"`, `"top"`, `"empty"`) in order.

Input	Output	Note
`ops = ["push","push","top","pop","empty"], args = [[1],[2],[],[],[]]`	`[2,2,false]`	

HINTS

1. The problem is that a queue hands back the *oldest* element but a stack needs the newest.
2. You can make push do the work: after enqueuing the new element, rotate everything before it to the back.
3. Rotating `size - 1` elements puts the newest at the front, so pop and top become trivial.
4. That makes push O(n) and pop O(1) — the reverse trade is also valid.

Approach	Idea	Time	Space
Two queues, costly pop	On pop, move all but one element to the second queue, then take the last. O(n) pop.	`O(n) pop`	`O(n)`
One queue, costly push	Enqueue then rotate. Only one queue needed and pop is O(1).	`O(n) push`	`O(n)`

Solution — js

```
function stackOps(ops, args) {
  const q = [];        // FIFO: push() to back, shift() from front
  const out = [];

  for (let i = 0; i < ops.length; i++) {
    const op = ops[i];
    if (op === "push") {
      q.push(args[i][0]);
      // rotate everything that was already there to the back,
      // so the NEWEST element ends up at the front
      for (let r = 0; r < q.length - 1; r++) q.push(q.shift());
    } else if (op === "pop") {
      out.push(q.shift());
    } else if (op === "top") {
      out.push(q[0]);
    } else if (op === "empty") {
      out.push(q.length === 0);
    }
  }
  return out;
}
```

Solution — cpp

```
class MyStack {
    queue<int> q;
public:
    void push(int x) {
        q.push(x);
        for (int r = 0; r + 1 < (int)q.size(); ++r) { q.push(q.front()); q.pop(); }
    }
    int pop()    { int v = q.front(); q.pop(); return v; }
    int top()    { return q.front(); }
    bool empty() { return q.empty(); }
};
```

Solution — python

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q = deque()

    def push(self, x: int) -> None:
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())

    def pop(self) -> int:
        return self.q.popleft()

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return not self.q
```

Solution — java

```
class MyStack {
    private final Queue<Integer> q = new LinkedList<>();
    public void push(int x) {
        q.add(x);
        for (int r = 0; r < q.size() - 1; r++) q.add(q.remove());
    }
    public int pop()      { return q.remove(); }
    public int top()      { return q.peek(); }
    public boolean empty(){ return q.isEmpty(); }
}
```

WALKTHROUGH

THE ROTATION MAKES THE QUEUE LIE ABOUT ITS ORDER.

After pushing 1, the queue is `[1]`. Pushing 2 gives `[1,2]`, then one rotation moves the 1 to the back: `[2,1]`. The front is now the most recently pushed element, which is exactly stack semantics. Pushing 3 gives `[2,1,3]` and two rotations produce `[3,2,1]`.

THE TRADE-OFF IS THE INTERESTING PART.

This version is O(n) push and O(1) pop. The two-queue version reverses it: O(1) push and O(n) pop. Neither is better in the abstract — it depends on which operation dominates. Being able to state both, and pick based on the expected workload, is what the question is really testing.

NOTE THE DISCIPLINE:

only `push` (to the back) and `shift` (from the front) are used. Reaching into the middle of the array, or using `pop()` from the back, would be cheating — the point is to use nothing but FIFO operations.

Heaps: top-K, merging and two-heap median

A structure that gives you the smallest (or largest) element instantly and lets you replace it in log time. Whenever a problem says "kth" or "top K" or "median of a stream", this is the answer.

The pattern

```
A BINARY HEAP is a complete binary tree stored in a flat
array, with the HEAP PROPERTY: every parent is <= (min
heap) or >= (max heap) both of its children.

  index i:   parent  (i-1) >> 1
             left    2i + 1
             right   2i + 2

  min heap        [1, 3, 6, 5, 9, 8]
                       1
                     /   \
                    3     6
                   / \   /
                  5   9 8

NOTE WHAT IT DOES NOT GIVE YOU: the heap is NOT sorted.
Only the ROOT is guaranteed to be the extreme. Anything
asking for full order needs a sort.

OPERATIONS
  peek        O(1)
  push        O(log n)   - append, then SIFT UP
  pop         O(log n)   - swap root with last, shrink,
                           SIFT DOWN
  heapify     O(n)       - building from an array is
                           LINEAR, not n log n

------------------------------------------------------------
THE TOP-K PATTERN - the one that matters most

  "K LARGEST"  -> keep a MIN heap of size k
  "K SMALLEST" -> keep a MAX heap of size k

THAT INVERSION SURPRISES PEOPLE. For the k largest, the
min heap's root is the WEAKEST of your current winners -
the one to evict when something better arrives:

    for (const x of items) {
        heap.push(x);
        if (heap.size() > k) heap.pop();   // drop the
    }                                      // smallest
    // the heap now holds the k largest; its root is the
    // kth largest

  COST O(n log k), and SPACE O(k) - which beats sorting
  (O(n log n), O(n)) when k is small, and is the only
  option for a STREAM you cannot store.

------------------------------------------------------------
THE TWO-HEAP PATTERN - median of a stream

  MAX heap for the lower half, MIN heap for the upper.
  Keep the sizes within one of each other.

     lower (max heap)     upper (min heap)
       ... 3 4 |            | 5 6 ...
           ^ root             ^ root

  median = root of the bigger heap, or the mean of both
  roots when the sizes are equal.
```

```
     Insert: push to one, then move its root to the other to
     restore the ordering, then rebalance the sizes. Always
     push-then-transfer, or an element can end up on the
     wrong side.
```

> **JAVASCRIPT HAS NO BUILT-IN HEAP**
> C++ has `priority_queue`, Java has `PriorityQueue`, Python has `heapq` (a *min* heap — negate values for a max heap). JavaScript has nothing, so you write one. The implementation below is about 25 lines and is worth being able to produce from memory; the problems on this page provide it for you so you can focus on the pattern.

```
class MinHeap {
    constructor(cmp = (a, b) => a - b) { this.a = []; this.cmp = cmp; }
    size()  { return this.a.length; }
    peek()  { return this.a[0]; }

    push(v) {
        this.a.push(v);
        let i = this.a.length - 1;
        while (i > 0) {                       // SIFT UP
            const p = (i - 1) >> 1;
            if (this.cmp(this.a[i], this.a[p]) >= 0) break;
            [this.a[i], this.a[p]] = [this.a[p], this.a[i]];
            i = p;
        }
    }

    pop() {
        const top = this.a[0];
        const last = this.a.pop();
        if (this.a.length) {
            this.a[0] = last;
            let i = 0;                        // SIFT DOWN
            for (;;) {
                const l = 2*i + 1, r = l + 1;
                let m = i;
                if (l < this.a.length && this.cmp(this.a[l], this.a[m]) < 0) m = l;
                if (r < this.a.length && this.cmp(this.a[r], this.a[m]) < 0) m = r;
                if (m === i) break;
                [this.a[i], this.a[m]] = [this.a[m], this.a[i]];
                i = m;
            }
        }
        return top;
    }
}

/* A MAX HEAP is the same class with a flipped comparator: */
new MinHeap((a, b) => b - a);
```

When to reach for it

"Kth largest / smallest"

Size-k heap of the opposite polarity. O(n log k).

"Top K frequent"

Count with a map, then a size-k heap — or bucket sort for O(n).

Merge K sorted lists/arrays

Heap of the K current heads.

Median of a data stream

Two heaps, balanced.

Scheduling / "next event"

Task scheduler, meeting rooms, Dijkstra (ch. 30).

K closest points

Max heap of size k keyed on distance — no need to sort everything.

Problems

How it shows up

Symptom	Cause
Kth largest returns the kth smallest	Wrong heap polarity — k largest needs a *min* heap
Heap grows to n instead of k	Missing the "pop when size exceeds k" step
Expected sorted output, got heap order	A heap orders only the root — sort if full order is needed
Median wrong by one element	Heaps not rebalanced after insertion
Median puts an element on the wrong side	Pushed directly instead of push-then-transfer
Python max-heap misbehaves	`heapq` is min-only — negate on push and on pop
Timeout building a heap	Pushing n times is O(n log n); `heapify` is O(n)

Check yourself

- ✓ Give the parent and child index formulas.
- ✓ State the cost of peek, push, pop and heapify.
- ✓ Say which heap polarity to use for "k largest" and explain why.
- ✓ Compare the heap and sorting approaches to top-K in time and space.
- ✓ Describe the two-heap median invariant and the insertion procedure.
- ✓ Explain what a heap does *not* guarantee.
- ✓ Say how to get a max heap from a min-heap-only library.

Problems

Kth largest element in an array `Medium`

Pattern: Size-k min heap

Given an integer array `nums` and an integer `k`, return the `k`th largest element. Note this is the kth largest in **sorted order**, not the kth distinct value.

A `MinHeap` class is already defined for you — construct it with a comparator, e.g. `new MinHeap((a,b) => a - b)`, and use `push`, `pop`, `peek`, `size`.

Input	Output	Note
`nums = [3,2,1,5,6,4], k = 2`	`5`	
`nums = [3,2,3,1,2,4,5,5,6], k = 4`	`4`	

HINTS

1. Sorting gives the answer in O(n log n). A heap of size k gives O(n log k), which is better when k is small.
2. For the k *largest*, keep a **min** heap of size k — its root is the weakest of your current winners.
3. Push every element; whenever the heap exceeds size k, pop the smallest.
4. At the end the root is the kth largest.

Approach	Idea	Time	Space
Sort and index	Sort descending and take index `k-1`. Perfectly acceptable — state it first.	`O(n log n)`	`O(1)`
Size-k min heap	Keep only the k best seen so far. Works on a stream.	`O(n log k)`	`O(k)`
Quickselect	Partition around a pivot, recurse into one side only. O(n) average, $O(n^2)$ worst.	`O(n) avg`	`O(1)`

Solution — js

```
function findKthLargest(nums, k) {
  const h = new MinHeap((a, b) => a - b);   // MIN heap

  for (const v of nums) {
    h.push(v);
    if (h.size() > k) h.pop();   // evict the weakest winner
  }
  return h.peek();               // root = kth largest
}
```

Solution — cpp

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> h;  // MIN heap
        for (int v : nums) {
            h.push(v);
            if ((int)h.size() > k) h.pop();
        }
        return h.top();
    }
};
```

Solution — python

```
import heapq

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        h = []                       # heapq is a MIN heap
        for v in nums:
            heapq.heappush(h, v)
            if len(h) > k:
                heapq.heappop(h)
        return h[0]
        # or simply: return heapq.nlargest(k, nums)[-1]
```

Solution — java

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> h = new PriorityQueue<>();   // MIN heap
        for (int v : nums) {
            h.offer(v);
            if (h.size() > k) h.poll();
        }
        return h.peek();
    }
}
```

WALKTHROUGH

WHY A MIN HEAP FOR THE LARGEST.

The heap holds the k best elements seen so far. When a new candidate arrives you want to discard the *worst* of the current winners — and that is the smallest of them, which is exactly what a min heap exposes in O(1). A max heap would give you the strongest, which is the one you never want to remove.

WHEN TO PREFER THIS OVER SORTING.

Sorting is simpler and fine for a one-off array. The heap wins when `k ≪ n` (O(n log k) vs O(n log n)), when the data arrives as a *stream* that cannot be stored, or when memory is bounded — you hold k elements rather than n. Say all three; the streaming point is the one interviewers are usually fishing for.

QUICKSELECT IS THE O(N)-AVERAGE ANSWER

and worth naming, but its $O(n^2)$ worst case and the fiddly partition make the heap the safer thing to write under time pressure.

Top K frequent elements `Medium`

Pattern: Frequency map + size-k heap

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. Return them in any order.

Input	Output	Note
`nums = [1,1,1,2,2,3], k = 2`	`[1,2]`	
`nums = [1], k = 1`	`[1]`	

HINTS

1. First count occurrences with a hash map (chapter 13).
2. Then it is a top-K problem over the (value, count) pairs, keyed on count.
3. Keep a min heap of size k ordered by frequency.
4. There is also an O(n) solution: bucket by frequency, since no count can exceed n.

Approach	Idea	Time	Space
Sort by frequency	Count, then sort the entries. Simple.	`O(n log n)`	`O(n)`
Size-k min heap	Heap keyed on count, evicting the least frequent.	`O(n log k)`	`O(n)`
Bucket sort	Index buckets by frequency 1..n and read from the top. Linear.	`O(n)`	`O(n)`

Solution — js

```
function topKFrequent(nums, k) {
  const freq = new Map();
  for (const v of nums) freq.set(v, (freq.get(v) ?? 0) + 1);

  // MIN heap keyed on count: root is the least frequent kept
  const h = new MinHeap((a, b) => a[1] - b[1]);
  for (const entry of freq) {          // entry = [value, count]
    h.push(entry);
    if (h.size() > k) h.pop();
  }

  const out = [];
  while (h.size()) out.push(h.pop()[0]);
  return out;
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int,int> freq;
        for (int v : nums) ++freq[v];
        using P = pair<int,int>;                 // (count, value)
        priority_queue<P, vector<P>, greater<P>> h;
        for (auto& [v, c] : freq) {
            h.push({c, v});
            if ((int)h.size() > k) h.pop();
        }
        vector<int> out;
        while (!h.empty()) { out.push_back(h.top().second); h.pop(); }
        return out;
    }
};
```

Solution — python

```python
import heapq
from collections import Counter

class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        freq = Counter(nums)
        return [v for v, _ in freq.most_common(k)]
        # explicit heap version:
        # h = []
        # for v, c in freq.items():
        #     heapq.heappush(h, (c, v))
        #     if len(h) > k: heapq.heappop(h)
        # return [v for _, v in h]
```

Solution — java

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer,Integer> freq = new HashMap<>();
        for (int v : nums) freq.merge(v, 1, Integer::sum);
        PriorityQueue<int[]> h = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        for (var e : freq.entrySet()) {
            h.offer(new int[]{e.getKey(), e.getValue()});
            if (h.size() > k) h.poll();
        }
        int[] out = new int[h.size()];
        for (int i = 0; i < out.length; i++) out[i] = h.poll()[0];
        return out;
    }
}
```

WALKTHROUGH

TWO PHASES, TWO DATA STRUCTURES.

Counting is a hash-map job; selecting the top k is a heap job. Recognising that a problem decomposes into two familiar patterns — rather than being one new thing — is most of the skill.

THE HEAP IS KEYED ON THE COUNT, NOT THE VALUE

, which is why the entries are pairs. Getting the comparator to read `a[1] - b[1]` rather than `a[0] - b[0]` is the detail to check.

THE O(N) BUCKET ALTERNATIVE IS WORTH DESCRIBING.

Since a frequency can be at most `n`, make an array of `n+1` buckets and put each value into `buckets[count]`. Then walk from the highest bucket downwards collecting values until you have k. No comparisons, no log factor — the same trick as counting sort in chapter 11. Mentioning it after giving the heap solution is a strong finish.

Find median from a data stream `Hard`

Pattern: Two balanced heaps

Design a structure that supports adding numbers from a stream and querying the median so far.

Implement `medianStream(ops, args)` where each op is `"add"` (with one argument) or `"median"`; return the results of the `"median"` queries in order.

The median of an even-sized set is the mean of the two middle values.

Input	Output	Note
`ops = ["add","add","median","add","median"], args = [[1],[2],[],[3],[]]`	`[1.5,2]`	

HINTS

1. Keeping a sorted array costs O(n) per insertion. You only ever need the *middle*, not the whole order.
2. Split the data at the median: a max heap for the lower half and a min heap for the upper half.
3. The two roots are the two candidates for the median, both reachable in O(1).
4. To insert safely, always push into one heap and move its root across, then rebalance so the sizes differ by at most one.

Approach	Idea	Time	Space
Sorted array with insertion	Binary search the position, then shift. Query is O(1) but insert is O(n).	`O(n) add`	`O(n)`
Two heaps	Lower half in a max heap, upper half in a min heap, sizes balanced.	`O(log n) add, O(1) query`	`O(n)`

Solution — js

```
function medianStream(ops, args) {
  const lower = new MinHeap((a, b) => b - a);  // MAX heap: lower half
  const upper = new MinHeap((a, b) => a - b);  // MIN heap: upper half
  const out = [];

  const add = (v) => {
    // push to lower, then hand its largest to upper —
    // this guarantees v lands on the correct side
    lower.push(v);
    upper.push(lower.pop());
    // rebalance: lower may hold one extra, never fewer
    if (upper.size() > lower.size()) lower.push(upper.pop());
  };

  const median = () =>
    lower.size() > upper.size()
      ? lower.peek()
      : (lower.peek() + upper.peek()) / 2;

  for (let i = 0; i < ops.length; i++) {
    if (ops[i] === "add") add(args[i][0]);
    else out.push(median());
  }
  return out;
}
```

Solution — cpp

```
class MedianFinder {
    priority_queue<int> lower;                                  // max heap
    priority_queue<int, vector<int>, greater<int>> upper;       // min heap
public:
    void addNum(int v) {
        lower.push(v);
        upper.push(lower.top()); lower.pop();
        if (upper.size() > lower.size()) { lower.push(upper.top()); upper.pop(); }
    }
    double findMedian() {
        if (lower.size() > upper.size()) return lower.top();
        return (lower.top() + upper.top()) / 2.0;
    }
};
```

Solution — python

```python
import heapq

class MedianFinder:
    def __init__(self):
        self.lower = []   # max heap via negation
        self.upper = []   # min heap

    def addNum(self, v: int) -> None:
        heapq.heappush(self.lower, -v)
        heapq.heappush(self.upper, -heapq.heappop(self.lower))
        if len(self.upper) > len(self.lower):
            heapq.heappush(self.lower, -heapq.heappop(self.upper))

    def findMedian(self) -> float:
        if len(self.lower) > len(self.upper):
            return -self.lower[0]
        return (-self.lower[0] + self.upper[0]) / 2
```

Solution — java

```java
class MedianFinder {
    private final PriorityQueue<Integer> lower =
        new PriorityQueue<>(Comparator.reverseOrder());   // max heap
    private final PriorityQueue<Integer> upper =
        new PriorityQueue<>();                            // min heap

    public void addNum(int v) {
        lower.offer(v);
        upper.offer(lower.poll());
        if (upper.size() > lower.size()) lower.offer(upper.poll());
    }
    public double findMedian() {
        if (lower.size() > upper.size()) return lower.peek();
        return (lower.peek() + upper.peek()) / 2.0;
    }
}
```

WALKTHROUGH

THE INVARIANT:

every element of `lower` is ≤ every element of `upper`, and `lower.size()` equals `upper.size()` or exceeds it by exactly one. Under that invariant the median is either `lower`'s root (odd count) or the mean of the two roots (even count) — both O(1).

WHY PUSH-THEN-TRANSFER RATHER THAN CHOOSING A SIDE.

The naive version compares `v` against the current median and pushes to whichever heap seems right, then rebalances. It works, but the comparison has several edge cases (empty heaps, equal values) and is easy to get subtly wrong. Pushing unconditionally into `lower` and immediately moving its largest into `upper` is unconditionally correct: whatever the new element is, the largest of the lower half ends up in the upper half, so the ordering invariant is restored automatically. The only thing left to fix is the size balance, which is one comparison.

THE CONVENTION CHOICE.

Here `lower` is allowed to be the bigger heap, so an odd count means the median is its root. Choosing the opposite convention works equally well provided the query code agrees — mismatching them is the classic off-by-one in this problem, which the odd-count tests catch.

PYTHON HAS NO MAX HEAP

, so `lower` stores negated values and negates again on read. That is worth mentioning if you write Python in the interview.

Trees: traversals, iterative and recursive

Four ways to visit every node. Choosing the right one turns most tree problems into three lines — and the interviewer will ask you for the iterative version.

The four traversals

```
THE TREE USED THROUGHOUT:

          1
        /   \
       2     3
      / \
     4   5

DEPTH FIRST — the position of "visit" is the only
difference:

  PREORDER    node, left, right      1 2 4 5 3
              -> COPY a tree, serialise, root-first work
  INORDER     left, node, right      4 2 5 1 3
              -> SORTED order in a BST (chapter 25)
  POSTORDER   left, right, node      4 5 2 3 1
              -> DELETE a tree, compute sizes/heights,
                 any work needing CHILD RESULTS FIRST

BREADTH FIRST — level by level, with a queue:

  LEVEL ORDER                        1 2 3 4 5
              -> shortest path, level-by-level output,
                 "closest" anything

THE CHOICE IS DETERMINED BY WHEN YOU NEED INFORMATION:

  need PARENT info before children   -> PREORDER
    (depth, path so far, passing a value down)
  need CHILD results before parent   -> POSTORDER
    (height, subtree sums, "is this subtree valid?")
  need SORTED order in a BST         -> INORDER
  need DISTANCE from the root        -> BFS

THAT MAPPING SOLVES MOST TREE PROBLEMS BY ITSELF. If you
are stuck on a tree question, ask: does this node need
something from above, or something from below?

COSTS
  every traversal   O(n) time
  recursion         O(h) space — h = height
                    O(log n) balanced, O(n) SKEWED
  BFS               O(w) space — w = maximum width,
                    which is up to n/2 for a full tree
```

The templates

```
/* --- RECURSIVE: all three, one shape --- */
function dfs(node, out) {
    if (!node) return;
    // out.push(node.val);          <-- PREORDER here
    dfs(node.left, out);
    // out.push(node.val);          <-- INORDER here
    dfs(node.right, out);
    // out.push(node.val);          <-- POSTORDER here
}

/* --- ITERATIVE PREORDER: a stack, push RIGHT first --- */
function preorder(root) {
    if (!root) return [];
    const out = [], st = [root];
    while (st.length) {
        const n = st.pop();
        out.push(n.val);
        if (n.right) st.push(n.right);   // right first, so
        if (n.left)  st.push(n.left);    // LEFT pops first
    }
    return out;
}

/* --- ITERATIVE INORDER: go left, then visit, then right --- */
function inorder(root) {
    const out = [], st = [];
    let cur = root;
    while (cur || st.length) {
        while (cur) { st.push(cur); cur = cur.left; }  // dive left
        cur = st.pop();
        out.push(cur.val);                             // VISIT
        cur = cur.right;                               // then right
    }
    return out;
}

/* --- ITERATIVE POSTORDER: the trick --- */
    Do a MODIFIED PREORDER (node, RIGHT, left), then REVERSE
    the result. node-right-left reversed is left-right-node.
    Far easier than tracking whether each node's children
    have been processed.

/* --- BFS: a queue, one LEVEL per outer iteration --- */
function levelOrder(root) {
    if (!root) return [];
    const out = [], q = [root];
    while (q.length) {
        const size = q.length;        // FIX the level size FIRST
        const level = [];
        for (let i = 0; i < size; i++) {
            const n = q.shift();
            level.push(n.val);
            if (n.left)  q.push(n.left);
            if (n.right) q.push(n.right);
        }
        out.push(level);
    }
    return out;
}
/* CAPTURING `size` BEFORE THE INNER LOOP IS THE WHOLE
```

```
    TRICK — it separates the levels. Without it you get a
    flat list.

    NOTE: Array.shift() is O(n) in JavaScript, so a large BFS
    should use an index pointer instead of shift (chapter 03).
    For interview-sized inputs shift is fine, but say that
    you know.                                              */
```

Problems

How it shows up

Symptom	Cause
BFS returns one flat list	Level size not captured before the inner loop
Iterative preorder is mirrored	Pushed left before right — the stack reverses order
Stack overflow on a large tree	Skewed tree, O(n) recursion depth — go iterative
Null children crash	Missing the `if (!node) return` base case
Inorder of a BST is not sorted	Visiting in the wrong position, or it is not a valid BST
Height off by one	Decide whether a leaf has height 0 or 1, and stay consistent
BFS very slow on a wide tree	`shift()` is O(n) — use a head index
Result accumulates across calls	Output array declared outside and not reset

Check yourself

- ✓ Give all four traversals of the example tree from memory.
- ✓ State which traversal to use when a node needs information from above versus below.
- ✓ Write iterative preorder and explain the push order.
- ✓ Write iterative inorder and explain the dive-left loop.
- ✓ Give the postorder trick and why it works.
- ✓ Write level-order and explain why the level size is captured first.
- ✓ State the space complexity of DFS on a balanced versus a skewed tree.
- ✓ Say why `shift()` is a problem in a large BFS.

Problems

Binary tree inorder traversal `Easy`

Pattern: Inorder, iterative

Given the root of a binary tree, return the **inorder** traversal of its node values.

The recursive version is three lines — so write the **iterative** one, which is what gets asked as the follow-up.

Trees are given in level order with `null` for a missing child.

Input	Output	Note
`root = [1,null,2,3]`	`[1,3,2]`	
`root = []`	`[]`	
`root = [1,2,3,4,5]`	`[4,2,5,1,3]`	

HINTS

1. Use an explicit stack to simulate what recursion does for you.
2. From any node, first push everything down the left spine. Then pop, visit, and move to the right child.
3. The loop continues while there is a current node *or* the stack is non-empty — both conditions are needed.

Approach	Idea	Time	Space
Recursive	Recurse left, visit, recurse right.	`O(n)`	`O(h)`
Iterative with a stack	Dive left pushing as you go; pop, visit, then switch to the right subtree.	`O(n)`	`O(h)`
Morris traversal	Thread the tree using spare right pointers for O(1) space. Worth naming; rarely required.	`O(n)`	`O(1)`

Solution — js

```
function inorderTraversal(root) {
  const out = [], st = [];
  let cur = root;
  while (cur || st.length) {
    while (cur) {          // dive as far left as possible
      st.push(cur);
      cur = cur.left;
    }
    cur = st.pop();        // leftmost unvisited node
    out.push(cur.val);     // VISIT
    cur = cur.right;       // now handle its right subtree
  }
  return out;
}
```

Solution — cpp

```
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> out;
        stack<TreeNode*> st;
        TreeNode* cur = root;
        while (cur || !st.empty()) {
            while (cur) { st.push(cur); cur = cur->left; }
            cur = st.top(); st.pop();
            out.push_back(cur->val);
            cur = cur->right;
        }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def inorderTraversal(self, root: TreeNode) -> list[int]:
        out, st, cur = [], [], root
        while cur or st:
            while cur:
                st.append(cur)
                cur = cur.left
            cur = st.pop()
            out.append(cur.val)
            cur = cur.right
        return out
```

Solution — java

```java
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> out = new ArrayList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            out.add(cur.val);
            cur = cur.right;
        }
        return out;
    }
}
```

WALKTHROUGH

WHY BOTH LOOP CONDITIONS ARE NEEDED.

`cur` non-null means there is a subtree still to descend into. A non-empty stack means there are ancestors still waiting to be visited. After visiting a leaf, `cur` becomes `null` but the stack still holds its ancestors — dropping the `st.length` check would stop early. Conversely, at the very start the stack is empty but `cur` is the root.

WHY THIS ORDER GIVES SORTED OUTPUT ON A BST:

everything in the left subtree is smaller and everything in the right is larger, so visiting left-node-right emits values in increasing order. That single fact powers most BST problems (chapter 25).

Binary tree level order traversal `Medium`

Pattern: BFS with level separation

Given the root of a binary tree, return its level order traversal as an array of arrays — one inner array per level, left to right.

Input	Output	Note
`root = [3,9,20,null,null,15,7]`	`[[3],[9,20],[15,7]]`	
`root = [1]`	`[[1]]`	
`root = []`	`[]`	

HINTS

1. A queue gives you level order, but a plain queue loop produces one flat list.
2. Capture `q.length` into a variable *before* the inner loop — that is exactly the number of nodes on the current level.
3. Enqueue children during the inner loop; they belong to the next level and must not be counted in this one.

Approach	Idea	Time	Space
DFS with a depth parameter	Recurse carrying the depth and append into `out[depth]`. Works, and is O(h) space.	`O(n)`	`O(h)`
BFS with a fixed level size	Standard queue BFS; snapshot the queue length at the start of each level.	`O(n)`	`O(w)`

Solution — js

```
function levelOrder(root) {
  if (!root) return [];
  const out = [];
  const q = [root];
  let head = 0;                      // index instead of shift()

  while (head < q.length) {
    const size = q.length - head;    // FIX this level's size
    const level = [];
    for (let i = 0; i < size; i++) {
      const n = q[head++];
      level.push(n.val);
      if (n.left)  q.push(n.left);
      if (n.right) q.push(n.right);
    }
    out.push(level);
  }
  return out;
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (!root) return {};
        vector<vector<int>> out;
        queue<TreeNode*> q;
        q.push(root);
        while (!q.empty()) {
            int size = q.size();
            vector<int> level;
            for (int i = 0; i < size; ++i) {
                TreeNode* n = q.front(); q.pop();
                level.push_back(n->val);
                if (n->left)  q.push(n->left);
                if (n->right) q.push(n->right);
            }
            out.push_back(level);
        }
        return out;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def levelOrder(self, root: TreeNode) -> list[list[int]]:
        if not root:
            return []
        out, q = [], deque([root])
        while q:
            level = []
            for _ in range(len(q)):      # snapshot the size
                n = q.popleft()
                level.append(n.val)
                if n.left:  q.append(n.left)
                if n.right: q.append(n.right)
            out.append(level)
        return out
```

Solution — java

```java
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> out = new ArrayList<>();
        if (root == null) return out;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                level.add(n.val);
                if (n.left  != null) q.add(n.left);
                if (n.right != null) q.add(n.right);
            }
            out.add(level);
        }
        return out;
    }
}
```

WALKTHROUGH

THE `SIZE` SNAPSHOT IS THE ENTIRE TECHNIQUE.

The queue grows while you are draining it, so `q.length` is a moving target. Capturing it once fixes the boundary between "this level" and "the next".

NOTE THE JAVASCRIPT SOLUTION USES A HEAD INDEX RATHER THAN `SHIFT()`.

`Array.shift()` is O(n) because it reindexes the whole array, so a BFS built on it is $O(n^2)$ on a wide tree. Using `q[head++]` keeps it O(n) at the cost of not reclaiming memory — the right trade for an interview, and worth mentioning aloud.

THIS TEMPLATE GENERALISES DIRECTLY:

zigzag order (reverse alternate levels), right side view (take the last element of each level), maximum level sum, and level averages are all this loop with one line changed.

Maximum depth of a binary tree Easy

Pattern: Postorder — children first

Given the root of a binary tree, return its maximum depth: the number of nodes along the longest path from the root down to a leaf.

Input	Output	Note
`root = [3,9,20,null,null,15,7]`	`3`	
`root = [1,null,2]`	`2`	
`root = []`	`0`	

HINTS

1. The depth of a node depends on its children, so this is postorder: compute both subtrees, then combine.
2. An empty tree has depth 0. That is the base case and it makes everything else fall out.
3. depth(node) = 1 + max(depth(left), depth(right)).

Approach	Idea	Time	Space
Recursive postorder	Three lines. The definition is the algorithm.	`O(n)`	`O(h)`
BFS counting levels	Level-order traversal, incrementing a counter per level. Useful when recursion depth is a concern.	`O(n)`	`O(w)`

Solution — js

```
function maxDepth(root) {
  if (!root) return 0;                       // base case
  return 1 + Math.max(maxDepth(root.left),   // children FIRST
                      maxDepth(root.right));
}
```

Solution — cpp

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

Solution — python

```
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

Solution — java

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

WALKTHROUGH

THIS IS THE SHAPE OF NEARLY EVERY RECURSIVE TREE PROBLEM

(chapter 24): a base case for `null`, a recursive call on each child, and a combine step. Getting comfortable with "what does each child return, and how do I combine it?" is the whole skill.

THE SPACE COMPLEXITY IS O(H), NOT O(1).

The call stack holds one frame per level, so a balanced tree costs O(log n) and a completely skewed one costs O(n). The test `[1,2,null,3,null,4]` is a left-leaning chain — on a tree of a million such nodes this would overflow the stack, which is exactly why the BFS alternative is worth knowing.

Trees: the recursive pattern

Almost every tree problem is the same three lines: a base case, a call on each child, and a combine step. The skill is deciding what each child should return.

The pattern

```
THE UNIVERSAL SHAPE:

  function solve(node) {
      if (!node) return BASE;              // 1. base case
      const L = solve(node.left);          // 2. recurse
      const R = solve(node.right);
      return COMBINE(L, R, node.val);      // 3. combine
  }

THE ONLY QUESTION THAT MATTERS:

  WHAT SHOULD A SUBTREE REPORT TO ITS PARENT?

Answer that and the code writes itself. Examples:

  height            report the height below me
  sum               report the subtree sum
  "is this a BST?"  report (isBST, min, max)
  diameter          report the height, and record the
                    best diameter in an outer variable
  balanced?         report the height, or -1 for
                    "already unbalanced"

------------------------------------------------------------
THE TWO DIRECTIONS OF INFORMATION

  TOP-DOWN (preorder): the parent passes context DOWN as
  a parameter.
      dfs(node, depth)      dfs(node, pathSoFar)
      dfs(node, lowerBound, upperBound)

  BOTTOM-UP (postorder): the child returns a result UP.
      const h = 1 + max(dfs(left), dfs(right));

  MANY PROBLEMS NEED BOTH. Validating a BST passes bounds
  DOWN; computing the diameter sends heights UP. When you
  are stuck, ask which direction the missing information
  travels.

------------------------------------------------------------
THE "GLOBAL BEST + LOCAL RETURN" IDIOM

Some problems ask for the best value ANYWHERE in the tree,
but the recursion must return something DIFFERENT to be
composable. Keep the answer outside:

  let best = -Infinity;

  function gain(node) {
      if (!node) return 0;
      const L = Math.max(gain(node.left), 0);   // drop
      const R = Math.max(gain(node.right), 0);  // negatives
      best = Math.max(best, node.val + L + R);  // path
                                                // THROUGH me
      return node.val + Math.max(L, R);         // path
  }                                             // ENDING at me

THE TWO EXPRESSIONS ARE DIFFERENT AND BOTH ARE NEEDED.
A path through a node uses both children; a path handed
to a parent may use only one, or the parent could not
```

```
    extend it. Confusing them is the classic bug in maximum
    path sum and in diameter.
```

When to reach for it

Any aggregate over a subtree

Height, size, sum, count of something. Pure bottom-up.

Structural comparison

Same tree, symmetric tree, subtree of another — recurse on pairs of nodes.

Path questions

Root-to-leaf sums pass context down; any-to-any paths use the global-best idiom.

Validation with constraints

Pass the allowed range down (chapter 25).

Lowest common ancestor

Return the node found, or null; a node seeing both sides non-null is the answer.

Tree modification

Invert, flatten, prune — return the new subtree root.

Problems

How it shows up

Symptom	Cause
Crash on a null child	Missing base case
Diameter or path sum too large	Returned the "through me" value instead of the "ending at me" value
Negative values ruin a path sum	Child contributions not floored at 0
Answer only correct through the root	Global best never updated at non-root nodes
BST validation accepts a bad tree	Compared only against the immediate parent, not an inherited range
Stack overflow	Skewed tree, O(n) depth — convert to iterative (ch. 23)
Result accumulates between calls	Global mutable state not reset

Check yourself

- ✓ Write the universal three-step shape.
- ✓ State the single question that determines the recursion.
- ✓ Contrast information travelling down versus up, with an example of each.

- ✓ Explain the global-best idiom and why two different expressions are needed.
- ✓ Say why child contributions are clamped at zero in path-sum problems.
- ✓ Give the space complexity and when it becomes a problem.

Problems

Invert a binary tree Easy

Pattern: Bottom-up modification

Given the root of a binary tree, invert it — swap every left and right child — and return the root.

Input	Output	Note
`root = [4,2,7,1,3,6,9]`	`[4,7,2,9,6,3,1]`	
`root = [2,1,3]`	`[2,3,1]`	
`root = []`	`[]`	

HINTS

1. Invert each subtree, then swap the two children of the current node.
2. The base case is a null node — return it unchanged.
3. The order of "swap" and "recurse" does not matter here, as long as both happen.

Approach	Idea	Time	Space
Recursive	Swap children, recurse into both.	`O(n)`	`O(h)`
Iterative BFS or DFS	Same swap, driven by an explicit queue or stack. Avoids deep recursion.	`O(n)`	`O(n)`

Solution — js

```
function invertTree(root) {
  if (!root) return null;                 // base case

  const left  = invertTree(root.left);    // invert each side
  const right = invertTree(root.right);

  root.left  = right;                     // then swap them
  root.right = left;
  return root;
}
```

Solution — cpp

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (!root) return nullptr;
        TreeNode* l = invertTree(root->left);
        TreeNode* r = invertTree(root->right);
        root->left = r;
        root->right = l;
        return root;
    }
};
```

Solution — python

```
class Solution:
    def invertTree(self, root: TreeNode) -> TreeNode:
        if not root:
            return None
        left = self.invertTree(root.left)
        right = self.invertTree(root.right)
        root.left, root.right = right, left
        return root
```

Solution — java

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode l = invertTree(root.left);
        TreeNode r = invertTree(root.right);
        root.left = r;
        root.right = l;
        return root;
    }
}
```

WALKTHROUGH

THE SIMPLEST POSSIBLE INSTANCE OF THE PATTERN.

Base case: null returns null. Recurse: invert each side. Combine: attach them the other way round. Nothing is reported upward except the subtree root itself.

WHY THE ORDER DOES NOT MATTER HERE.

Swapping before recursing works equally well, because the recursion is applied to whatever ends up in each slot and the operation is symmetric. That is unusual — in most tree problems the combine step genuinely depends on the children's *results*, so it must come after. Do not generalise from this one.

THE ASYMMETRIC TESTS MATTER:

`[1,2]` has only a left child and must become `[1,null,2]`. A solution that only swaps when both children exist passes the symmetric tests and fails these.

Balanced binary tree Easy

Pattern: Height with an early-exit sentinel

Given a binary tree, determine whether it is **height-balanced**: for every node, the heights of its two subtrees differ by at most 1.

Input	Output	Note
`root = [3,9,20,null,null,15,7]`	`true`	
`root = [1,2,2,3,3,null,null,4,4]`	`false`	
`root = []`	`true`	

HINTS

1. The naive version computes the height at every node, which recomputes the same subtrees repeatedly — $O(n^2)$.
2. A single postorder pass can compute the height *and* detect imbalance at the same time.
3. Use a sentinel: return `-1` to mean "this subtree is already unbalanced" and propagate it upward immediately.

Approach	Idea	Time	Space
Height at every node	Check the balance condition at each node, recomputing heights. Correct but quadratic on a skewed tree.	`O(n²)`	`O(h)`
Height with a −1 sentinel	One postorder pass; imbalance short-circuits all the way up.	`O(n)`	`O(h)`

Solution — js

```
function isBalanced(root) {
  // returns the height, or -1 meaning "already unbalanced"
  function height(node) {
    if (!node) return 0;

    const l = height(node.left);
    if (l === -1) return -1;              // propagate failure

    const r = height(node.right);
    if (r === -1) return -1;

    if (Math.abs(l - r) > 1) return -1;   // fail here
    return 1 + Math.max(l, r);
  }
  return height(root) !== -1;
}
```

Solution — cpp

```
class Solution {
    int height(TreeNode* n) {
        if (!n) return 0;
        int l = height(n->left);
        if (l == -1) return -1;
        int r = height(n->right);
        if (r == -1) return -1;
        if (abs(l - r) > 1) return -1;
        return 1 + max(l, r);
    }
public:
    bool isBalanced(TreeNode* root) { return height(root) != -1; }
};
```

Solution — python

```
class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        def height(n):
            if not n:
                return 0
            l = height(n.left)
            if l == -1:
                return -1
            r = height(n.right)
            if r == -1:
                return -1
            if abs(l - r) > 1:
                return -1
            return 1 + max(l, r)
        return height(root) != -1
```

Solution — java

```
class Solution {
    public boolean isBalanced(TreeNode root) { return height(root) != -1; }
    private int height(TreeNode n) {
        if (n == null) return 0;
        int l = height(n.left);
        if (l == -1) return -1;
        int r = height(n.right);
        if (r == -1) return -1;
        if (Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }
}
```

WALKTHROUGH

THE SENTINEL DOES TWO JOBS AT ONCE.

A single return value carries both "how tall am I?" and "is anything below me broken?", which is what collapses two passes into one. Because `-1` can never be a real height, there is no ambiguity.

WHY THE NAIVE VERSION IS QUADRATIC.

Checking balance at a node calls `height` on both subtrees, and then the recursion repeats that work at every descendant. On a completely skewed tree of n nodes that is `n + (n−1) + … = O(n²)`. Being able to name that cost, and then fix it, is the point of the problem.

NOTE THE EARLY RETURNS ARE PLACED BETWEEN THE TWO RECURSIVE CALLS

, so a failure on the left avoids exploring the right subtree at all. Not required for correctness, but it is the natural way to write it and it does save work.

Diameter of a binary tree Easy

Pattern: Global best + local return

Given the root of a binary tree, return the length of its **diameter**: the number of edges on the longest path between any two nodes. The path need not pass through the root.

Input	Output	Note
`root = [1,2,3,4,5]`	`3`	The path 4 → 2 → 1 → 3 has 3 edges.
`root = [1,2]`	`1`	
`root = []`	`0`	

HINTS

1. For any node, the longest path *through* it is `leftHeight + rightHeight` edges.
2. But that value cannot be returned to the parent — a parent can only extend a path down **ONE** side.
3. So return the height, and record the best diameter in a variable outside the recursion.

Approach	Idea	Time	Space
Height at every node	Compute `left + right` at each node, recomputing heights each time.	`O(n²)`	`O(h)`
Global best + height return	One postorder pass; update the best while returning the height.	`O(n)`	`O(h)`

Solution — js

```javascript
function diameterOfBinaryTree(root) {
  let best = 0;

  // returns the number of EDGES on the longest downward path
  function height(node) {
    if (!node) return 0;

    const l = height(node.left);
    const r = height(node.right);

    // path THROUGH this node joins both sides
    best = Math.max(best, l + r);

    // path ENDING at this node can only use ONE side
    return 1 + Math.max(l, r);
  }

  height(root);
  return best;
}
```

Solution — cpp

```cpp
class Solution {
    int best = 0;
    int height(TreeNode* n) {
        if (!n) return 0;
        int l = height(n->left), r = height(n->right);
        best = max(best, l + r);
        return 1 + max(l, r);
    }
public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;
        height(root);
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        self.best = 0

        def height(n):
            if not n:
                return 0
            l = height(n.left)
            r = height(n.right)
            self.best = max(self.best, l + r)
            return 1 + max(l, r)

        height(root)
        return self.best
```

Solution — java

```
class Solution {
    private int best;
    public int diameterOfBinaryTree(TreeNode root) {
        best = 0;
        height(root);
        return best;
    }
    private int height(TreeNode n) {
        if (n == null) return 0;
        int l = height(n.left), r = height(n.right);
        best = Math.max(best, l + r);
        return 1 + Math.max(l, r);
    }
}
```

WALKTHROUGH

THE TWO EXPRESSIONS ARE THE ENTIRE LESSON.

`l + r` is the answer *at* this node — a path that comes up the left side, crosses the node, and goes down the right. `1 + max(l, r)` is what the parent needs — a path that continues downward through one side only. They are different quantities, and a solution that returns `l + r` produces wildly inflated answers because it lets a parent "extend" a path that already turned a corner.

WHY THE GLOBAL VARIABLE IS NOT LAZINESS.

The function can only return one value, and the parent needs the height. The diameter is a maximum over all nodes and is not composable from the children's heights alone, so it is accumulated on the side. This idiom recurs in maximum path sum, longest univalue path, and several others — recognising it saves a lot of flailing.

COUNTING EDGES, NOT NODES.

A single node has diameter 0 and a null child contributes height 0, which makes `1 + max(l, r)` count edges consistently. If a problem asks for the number of *nodes* on the path, add one.

Binary tree maximum path sum `Hard`

Pattern: Global best + clamped local return

A **path** is any sequence of nodes connected by edges, where each node appears at most once; it need not pass through the root. The **path sum** is the sum of its node values.

Return the maximum path sum in the tree. Values may be negative.

Input	Output	Note
`root = [1,2,3]`	`6`	2 + 1 + 3
`root = [-10,9,20,null,null,15,7]`	`42`	15 + 20 + 7 — the root is not used.
`root = [-3]`	`-3`	A single node is a valid path.

HINTS

1. Same shape as diameter: record the best path *through* each node, return the best path *ending* at it.
2. Negative values change things: a subtree whose best contribution is negative should simply be excluded.
3. Clamp each child's contribution at 0 — `Math.max(childGain, 0)` — which means "do not take this branch".
4. The best answer may be a single negative node, so initialise the global best to −Infinity, not 0.

Approach	Idea	Time	Space
Try every pair of nodes	Compute the path sum between all pairs.	`O(n²) or worse`	`O(n)`
Global best + clamped gain	One postorder pass, clamping negative branches to zero.	`O(n)`	`O(h)`

Solution — js

```
function maxPathSum(root) {
  let best = -Infinity;         // a single negative node is valid

  // returns the best sum of a path that STARTS at `node`
  // and goes downward through at most one child
  function gain(node) {
    if (!node) return 0;

    // a negative branch is worse than not taking it at all
    const l = Math.max(gain(node.left), 0);
    const r = Math.max(gain(node.right), 0);

    // best path THROUGH this node uses both sides
    best = Math.max(best, node.val + l + r);

    // but a path handed upward may use only ONE side
    return node.val + Math.max(l, r);
  }

  gain(root);
  return best;
}
```

Solution — cpp

```
class Solution {
    int best;
    int gain(TreeNode* n) {
        if (!n) return 0;
        int l = max(gain(n->left), 0);
        int r = max(gain(n->right), 0);
        best = max(best, n->val + l + r);
        return n->val + max(l, r);
    }
public:
    int maxPathSum(TreeNode* root) {
        best = INT_MIN;
        gain(root);
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def maxPathSum(self, root: TreeNode) -> int:
        self.best = float("-inf")

        def gain(n):
            if not n:
                return 0
            l = max(gain(n.left), 0)
            r = max(gain(n.right), 0)
            self.best = max(self.best, n.val + l + r)
            return n.val + max(l, r)

        gain(root)
        return self.best
```

Solution — java

```java
class Solution {
    private int best;
    public int maxPathSum(TreeNode root) {
        best = Integer.MIN_VALUE;
        gain(root);
        return best;
    }
    private int gain(TreeNode n) {
        if (n == null) return 0;
        int l = Math.max(gain(n.left), 0);
        int r = Math.max(gain(n.right), 0);
        best = Math.max(best, n.val + l + r);
        return n.val + Math.max(l, r);
    }
}
```

WALKTHROUGH
THREE DECISIONS, EACH OF WHICH IS A TEST CASE.

1. CLAMPING AT ZERO.
`Math.max(gain(child), 0)` encodes "including this branch is optional". On `[2,-1]` the left child contributes −1, so the best path is just the root: 2. Without the clamp you would be forced to include it and return 1.

2. THE BEST IS INITIALISED TO −INFINITY, NOT 0.
On `[-3]` the only path is a single negative node, and the answer is −3. Starting at 0 would return 0, which corresponds to the empty path that the problem does not allow.

3. TWO DIFFERENT EXPRESSIONS AGAIN.
`node.val + l + r` is a path that turns a corner at this node — it can be the global answer but cannot be extended by a parent. `node.val + max(l, r)` is a straight downward path that a parent *can* extend. Returning the first is the classic bug and it overcounts badly.

NOTE THE ROOT NEED NOT BE INVOLVED:
`[-10,9,20,null,null,15,7]` answers 42 from the right subtree alone, because `gain(root)` is negative and the global best was already recorded deeper down.

Trees: binary search trees

A binary tree with an ordering invariant, which turns search into a descent. Two facts do most of the work: comparison tells you which way to go, and an inorder traversal comes out sorted.

The pattern

```
THE BST INVARIANT, stated correctly:

  for EVERY node, ALL values in its left subtree are
  smaller, and ALL values in its right subtree are larger.

NOT "left child < node < right child". That weaker
condition is the source of the most common bug in this
chapter:

           5
          / \
         1   7          7 > 5 ✓, 4 < 7 ✓ locally
            / \         BUT 4 < 5 and it is on the
           4   8        RIGHT of 5 -> NOT A BST

------------------------------------------------------------
THE TWO FACTS TO EXPLOIT

1. COMPARISON GIVES DIRECTION
     target < node.val  ->  go LEFT
     target > node.val  ->  go RIGHT
   Search, insert and delete are all O(h) descents.

2. INORDER TRAVERSAL IS SORTED
     left, node, right  ->  ascending order
   So "kth smallest", "validate", "find the two swapped
   nodes" and "convert to a sorted list" are all inorder
   walks (chapter 23).

------------------------------------------------------------
THE COST IS THE HEIGHT, NOT log n

  balanced   h = log n   -> O(log n)
  SKEWED     h = n       -> O(n), no better than a list

  Inserting sorted data into a plain BST produces exactly
  the skewed case. Real implementations self-balance -
  AVL, red-black - which is why std::map and TreeMap are
  O(log n) guaranteed. Interviews rarely ask you to
  implement balancing, but DO ask you to state that a
  plain BST degrades.

------------------------------------------------------------
VALIDATION - pass bounds DOWN

  function valid(node, lo, hi) {
      if (!node) return true;
      if (node.val <= lo || node.val >= hi) return false;
      return valid(node.left,  lo, node.val)
          && valid(node.right, node.val, hi);
  }
  valid(root, -Infinity, Infinity);

  Each recursion NARROWS the allowed range. Going left
  tightens the upper bound; going right tightens the
  lower bound. That is how a distant ancestor still
  constrains a deep descendant.

------------------------------------------------------------
DELETION - the only fiddly operation
```

```
0 children   remove it
1 child      replace it with that child
2 children   replace its value with the INORDER
             SUCCESSOR (the smallest value in the right
             subtree), then delete that successor from
             the right subtree — it has at most one
             child, so the recursion terminates.
```

When to reach for it

Sorted order from a tree

Inorder traversal. Kth smallest, sorted output, validation.

Search / insert / delete by key

Descend by comparison. O(h).

Lowest common ancestor in a BST

Much easier than the general tree — the first node between the two values is the LCA.

Range queries

Prune whole subtrees using the bounds.

Need order *and* fast lookup

A hash map gives O(1) lookup but no order; a balanced BST gives both at O(log n).

Build a balanced BST from sorted data

Recursive midpoint selection.

Problems

How it shows up

Symptom	Cause
Invalid tree accepted	Only compared with the parent — must pass a range down
Duplicates break validation	Decide whether equal values are allowed, and use `<=` or `<` consistently
Overflow with extreme values	Use `null` bounds rather than `INT_MIN/MAX` sentinels
Kth smallest scans the whole tree	Stop early — an inorder walk can return as soon as k is reached
Operations degrade to O(n)	Skewed tree from sorted insertions — mention self-balancing
Deletion loses a subtree	Two-child case must reattach via the inorder successor
LCA wrong in a BST	Used the general-tree algorithm and ignored the ordering

Check yourself

- ✓ State the BST invariant precisely and give the tree that breaks the weak version.
- ✓ Say what an inorder traversal of a BST produces and name three problems that follow.
- ✓ Write the validation function with bounds and explain how they narrow.
- ✓ Give the complexity of search in terms of height and say when that is bad.
- ✓ Describe the three deletion cases.
- ✓ Explain why LCA is simpler in a BST than in a general tree.
- ✓ Say when to prefer a balanced BST over a hash map.

Problems

Validate a binary search tree `Medium`

Pattern: Pass bounds down

Given the root of a binary tree, determine whether it is a valid binary search tree: every node in the left subtree is **strictly less** than the node, every node in the right subtree is **strictly greater**, and both subtrees are themselves valid BSTs.

Input	Output	Note
`root = [2,1,3]`	`true`	
`root = [5,1,4,null,null,3,6]`	`false`	3 is in the right subtree of 5 but is smaller than 5.
`root = [5,4,6,null,null,3,7]`	`false`	

HINTS

1. Comparing each node only with its immediate children is not enough — a value can satisfy its parent yet violate a grandparent.
2. Carry an allowed `(low, high)` range down the recursion.
3. Going left tightens the upper bound to the current value; going right tightens the lower bound.
4. An inorder traversal is an equally valid approach: the values must come out strictly increasing.

Approach	Idea	Time	Space
Compare with children only	**Wrong.** Accepts trees where a deep node violates a distant ancestor.	`O(n)`	`O(h)`
Inorder traversal	Walk inorder and check each value exceeds the previous. Elegant and easy to justify.	`O(n)`	`O(h)`
Range bounds passed down	Each node must lie strictly inside an inherited interval.	`O(n)`	`O(h)`

Solution — js

```javascript
function isValidBST(root) {
  // lo and hi are EXCLUSIVE bounds this subtree must respect
  function valid(node, lo, hi) {
    if (!node) return true;
    if (node.val <= lo || node.val >= hi) return false;

    return valid(node.left,  lo, node.val)    // tighten the ceiling
        && valid(node.right, node.val, hi);   // tighten the floor
  }
  return valid(root, -Infinity, Infinity);
}
```

Solution — cpp

```cpp
class Solution {
    bool valid(TreeNode* n, long lo, long hi) {
        if (!n) return true;
        if (n->val <= lo || n->val >= hi) return false;
        return valid(n->left, lo, n->val) && valid(n->right, n->val, hi);
    }
public:
    bool isValidBST(TreeNode* root) {
        return valid(root, LONG_MIN, LONG_MAX);
    }
};
```

Solution — python

```python
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        def valid(n, lo, hi):
            if not n:
                return True
            if not (lo < n.val < hi):
                return False
            return valid(n.left, lo, n.val) and valid(n.right, n.val, hi)
        return valid(root, float("-inf"), float("inf"))
```

Solution — java

```java
class Solution {
    public boolean isValidBST(TreeNode root) {
        return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
    private boolean valid(TreeNode n, long lo, long hi) {
        if (n == null) return true;
        if (n.val <= lo || n.val >= hi) return false;
        return valid(n.left, lo, n.val) && valid(n.right, n.val, hi);
    }
}
```

WALKTHROUGH

THE TEST THAT SEPARATES RIGHT FROM NEARLY-RIGHT

is `[5,1,4,null,null,3,6]`. Node 3 sits under 4, and 3 < 4 so a parent-only check is satisfied. But 3 is in the *right subtree of 5*, and everything there must exceed 5. The range mechanism catches it: descending right from 5 sets the floor to 5, and 3 fails it immediately.

HOW THE BOUNDS NARROW.

Moving left means every node below must be smaller than the current value, so the ceiling becomes `node.val`. Moving right means everything below must be larger, so the floor becomes `node.val`. The interval only ever shrinks, and it carries the constraint from every ancestor at once.

THE OVERFLOW TRAP IN C++ AND JAVA.

If a node holds `INT_MIN` and you initialise the bound to `INT_MIN`, the strict comparison wrongly rejects it. Widening to `long` — or using nullable bounds — avoids that. JavaScript and Python have no such problem, but say you know about it.

DUPLICATES ARE REJECTED HERE

(`[1,1]` is false) because the problem says strictly less and strictly greater. Some variants allow duplicates on one side; ask.

Kth smallest element in a BST `Medium`

Pattern: Inorder with early exit

Given the root of a binary search tree and an integer `k`, return the `k`th smallest value (1-indexed).

Input	Output	Note
`root = [3,1,4,null,2], k = 1`	`1`	
`root = [5,3,6,2,4,null,null,1], k = 3`	`3`	

HINTS

1. An inorder traversal of a BST visits values in ascending order.
2. So the kth value emitted by an inorder walk is the answer.
3. Do not collect the whole traversal — stop as soon as you have counted k.
4. The iterative stack version makes the early exit natural.

Approach	Idea	Time	Space
Full inorder into an array	Traverse everything, index `k-1`. Simple, O(n) time and space.	`O(n)`	`O(n)`
Inorder with a counter and early exit	Stop after k visits — O(h + k).	`O(h + k)`	`O(h)`
Augment nodes with subtree sizes	If the tree is modified often and this is queried often, storing subtree counts gives O(h) per query.	`O(h)`	`O(n)`

Solution — js

```javascript
function kthSmallest(root, k) {
  const st = [];
  let cur = root;

  while (cur || st.length) {
    while (cur) { st.push(cur); cur = cur.left; }   // dive left
    cur = st.pop();
    if (--k === 0) return cur.val;                  // EARLY EXIT
    cur = cur.right;
  }
  return -1;   // k exceeded the tree size
}
```

Solution — cpp

```cpp
class Solution {
public:
    int kthSmallest(TreeNode* root, int k) {
        stack<TreeNode*> st;
        TreeNode* cur = root;
        while (cur || !st.empty()) {
            while (cur) { st.push(cur); cur = cur->left; }
            cur = st.top(); st.pop();
            if (--k == 0) return cur->val;
            cur = cur->right;
        }
        return -1;
    }
};
```

Solution — python

```python
class Solution:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
        st, cur = [], root
        while cur or st:
            while cur:
                st.append(cur)
                cur = cur.left
            cur = st.pop()
            k -= 1
            if k == 0:
                return cur.val
            cur = cur.right
        return -1
```

Solution — java

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> st = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            if (--k == 0) return cur.val;
            cur = cur.right;
        }
        return -1;
    }
}
```

WALKTHROUGH

WHY INORDER IS THE RIGHT TRAVERSAL.

In a BST everything left of a node is smaller and everything right is larger, so visiting left-node-right emits values in ascending order. The kth emitted value is therefore the kth smallest, with no sorting.

THE EARLY EXIT IS WHAT THE INTERVIEWER WANTS.

Collecting the full traversal and indexing works, but visits all n nodes and allocates O(n). Stopping at the kth visit costs O(h + k) — the descent to the smallest element, plus k steps. For small k on a large tree that is dramatically better, and the iterative form makes returning mid-traversal trivial.

THE FOLLOW-UP WORTH PREPARING:

"what if the tree is modified frequently and this is queried often?" Then augment each node with the size of its subtree; the kth smallest becomes a single O(h) descent, comparing k against the left subtree size at each step. Mention the maintenance cost on insert and delete.

Lowest common ancestor of a BST `Medium`

Pattern: Descend while both are on the same side

Given a BST and two values `p` and `q` that are both present, return the value of their lowest common ancestor — the deepest node having both as descendants (a node may be a descendant of itself).

Input	Output	Note
`root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8`	`6`	
`root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4`	`2`	A node can be its own ancestor.

HINTS

1. In a general tree this needs a full search. A BST gives you far more.
2. If both values are smaller than the current node, the answer must be in the left subtree. If both are larger, go right.
3. The moment they fall on opposite sides — or one equals the current node — that node is the answer.
4. This is an iterative descent; no recursion or extra space is needed.

Approach	Idea	Time	Space
General-tree LCA	Recurse both sides and combine. Works, but ignores the ordering and costs O(n).	`O(n)`	`O(h)`
Find both paths	Record the root-to-node path for each, then compare. O(h) space.	`O(h)`	`O(h)`
Ordered descent	One walk down, no extra memory.	`O(h)`	`O(1)`

Solution — js

```
function lowestCommonAncestor(root, p, q) {
  let cur = root;
  while (cur) {
    if (p < cur.val && q < cur.val)      cur = cur.left;   // both left
    else if (p > cur.val && q > cur.val) cur = cur.right;  // both right
    else return cur.val;   // they split here, or one IS this node
  }
  return -1;
}
```

Solution — cpp

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        TreeNode* cur = root;
        while (cur) {
            if (p->val < cur->val && q->val < cur->val)      cur = cur->left;
            else if (p->val > cur->val && q->val > cur->val) cur = cur->right;
            else return cur;
        }
        return nullptr;
    }
};
```

Solution — python

```
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        cur = root
        while cur:
            if p.val < cur.val and q.val < cur.val:
                cur = cur.left
            elif p.val > cur.val and q.val > cur.val:
                cur = cur.right
            else:
                return cur
        return None
```

Solution — java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode cur = root;
        while (cur != null) {
            if (p.val < cur.val && q.val < cur.val)      cur = cur.left;
            else if (p.val > cur.val && q.val > cur.val) cur = cur.right;
            else return cur;
        }
        return null;
    }
}
```

WALKTHROUGH

THE SPLITTING ARGUMENT.

If both targets are less than the current node, both live in its left subtree, so the current node cannot be the *lowest* common ancestor — something deeper on the left also contains both. The symmetric case sends you right. The first node where they do not both go the same way is the split point, and no deeper node can contain both. That node is the LCA.

THE "NODE IS ITS OWN ANCESTOR" CASE FALLS OUT FOR FREE.

If `p` equals the current value, neither the both-smaller nor the both-larger condition holds, so the `else` fires and returns it — exactly right, as in the `p=2, q=4` test.

CONTRAST WITH THE GENERAL-TREE VERSION

, which must search both subtrees and combine results in O(n). The BST version reads a comparison and discards half the tree, giving O(h) and O(1) space. Recognising that the ordering makes the problem easier is the entire point of asking the BST variant.

Trees: tries and prefix problems

A tree where the path spells the word. It makes prefix queries as cheap as exact lookups — something a hash map fundamentally cannot do.

The pattern

```
A TRIE (prefix tree) stores strings by CHARACTER PATH.
Each edge is a character; each node is a prefix.

Inserting "cat", "car", "dog":

           (root)
           /    \
          c       d
          |       |
          a       o
         / \      |
        t   r     g*
        *   *
                  * = end of a word

THE KEY PROPERTY: WORDS SHARING A PREFIX SHARE A PATH.
"cat" and "car" traverse the same first two nodes, so
prefix questions are answered by a single descent.

------------------------------------------------------------
THE NODE

  class TrieNode {
      children = new Map();   // char -> TrieNode
      isWord = false;         // does a word END here?
  }

  `isWord` IS ESSENTIAL AND SEPARATE FROM "has children".
  Without it you cannot distinguish the stored word "car"
  from the mere prefix "car" of "cart".

  Use a Map for arbitrary alphabets; a fixed
  `new Array(26)` is faster when the input is lowercase
  a-z only.

------------------------------------------------------------
THE THREE OPERATIONS - all O(L), L = key length

  insert(word)     walk, creating nodes as needed, mark
                   the last node isWord = true
  search(word)     walk; return node exists AND isWord
  startsWith(pre)  walk; return the node merely EXISTS

  Search and startsWith differ by ONE LINE. That is the
  whole point of the structure.

------------------------------------------------------------
WHY NOT A HASH SET?

  exact lookup       hash set O(L)     trie O(L)   - tie
  "any word with
   prefix 'app'?"    hash set O(n·L)   trie O(L)   - trie
  "all words with
   prefix 'app'"     hash set O(n·L)   trie O(L + k)
  wildcard '.'       hash set: no      trie: DFS branching
  autocomplete       hash set: no      trie: natural

A HASH SET CANNOT ANSWER PREFIX QUESTIONS WITHOUT
SCANNING EVERYTHING. That is the sentence to say when
asked "why not just use a set?"
```

```
    COST: O(total characters) space — which can be large.
    The trade is memory for prefix speed.
```

When to reach for it

"Prefix", "starts with", autocomplete

The defining signal.

Wildcard or regex-lite matching

`.` matches any child — DFS over all branches.

Word search in a grid, many words

Build a trie of the dictionary and prune the DFS the moment a path leaves the trie.

Longest common prefix

Descend while there is exactly one child and no word ends.

Replace words / dictionary root matching

Walk the word until a stored root is hit.

Maximum XOR pair

A *binary* trie over bits — greedily take the opposite bit at each level.

Problems

How it shows up

Symptom	Cause
Prefix reported as a stored word	Missing `isWord` flag — checked node existence only
`search("app")` true after inserting only "apple"	Same cause
Wildcard search misses matches	Must try *every* child on `.`, not just the first
Wildcard search too slow	Expected — `.` branches; bound it by depth
Memory blowup	26-slot arrays at every node — use a Map for sparse alphabets
Deletion corrupts other words	Only unset `isWord`; prune a node only if it has no children
Case-sensitivity bugs	Normalise before inserting and before querying

Check yourself

- ✓ Draw the trie for "cat", "car", "dog".
- ✓ Say what `isWord` is for and what breaks without it.
- ✓ Give the three operations and their complexity.

- ✓ Say precisely how `search` and `startsWith` differ.
- ✓ Explain why a hash set cannot replace a trie.
- ✓ Describe how a wildcard `.` is handled.
- ✓ Say what the space cost is and what it buys.

Problems

Implement a trie `Medium`

Pattern: Prefix tree construction

Implement a trie supporting `insert(word)`, `search(word)` (is this exact word stored?) and `startsWith(prefix)` (is any stored word prefixed by this?).

Implement `trieOps(ops, args)`: apply each operation and return the boolean results of the `search` and `startsWith` queries, in order.

Input	Output	Note
`ops = ["insert","search","search","startsWith","insert","search"], args = [["apple"],["apple"],["app"],["app"],["app"],["app"]]`	`[true,false,true,true]`	

HINTS

1. Each node holds a map from character to child node, plus a flag saying whether a word ends there.
2. Insert walks the word, creating missing children, and sets the flag on the final node.
3. Search and startsWith share the same walk; they differ only in what they check at the end.
4. Without the flag, `search("app")` would wrongly return true after inserting only "apple".

Approach	Idea	Time	Space
Hash set of words	Exact search is fine, but `startsWith` requires scanning every stored word.	`O(n·L)` `prefix`	`O(n·L)`
Sorted array + binary search	Prefix queries become a range search. Works, but insertion is O(n).	`O(L log n)`	`O(n·L)`
Trie	All three operations are one descent of length L.	`O(L)`	`O(total chars)`

Solution — js

```javascript
function trieOps(ops, args) {
  const root = { children: new Map(), isWord: false };
  const out = [];

  // walk the characters; return the node reached, or null
  function walk(s) {
    let node = root;
    for (const c of s) {
      if (!node.children.has(c)) return null;
      node = node.children.get(c);
    }
    return node;
  }

  for (let i = 0; i < ops.length; i++) {
    const s = args[i][0];
    if (ops[i] === "insert") {
      let node = root;
      for (const c of s) {
        if (!node.children.has(c))
          node.children.set(c, { children: new Map(), isWord: false });
        node = node.children.get(c);
      }
      node.isWord = true;              // mark the END of a word
    } else if (ops[i] === "search") {
      const node = walk(s);
      out.push(node !== null && node.isWord);   // must be a WORD
    } else if (ops[i] === "startsWith") {
      out.push(walk(s) !== null);               // existing is enough
    }
  }
  return out;
}
```

Solution — cpp

```
class Trie {
    struct Node {
        unordered_map<char, Node*> ch;
        bool isWord = false;
    };
    Node* root = new Node();

    Node* walk(const string& s) {
        Node* n = root;
        for (char c : s) {
            auto it = n->ch.find(c);
            if (it == n->ch.end()) return nullptr;
            n = it->second;
        }
        return n;
    }
public:
    void insert(string word) {
        Node* n = root;
        for (char c : word) {
            if (!n->ch.count(c)) n->ch[c] = new Node();
            n = n->ch[c];
        }
        n->isWord = true;
    }
    bool search(string word)      { Node* n = walk(word); return n && n->isWord; }
    bool startsWith(string prefix){ return walk(prefix) != nullptr; }
};
```

Solution — python

```
class TrieNode:
    __slots__ = ("children", "is_word")
    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def _walk(self, s):
        node = self.root
        for c in s:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.is_word = True

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return node is not None and node.is_word

    def startsWith(self, prefix: str) -> bool:
        return self._walk(prefix) is not None
```

Solution — java

```
class Trie {
    private static class Node {
        Map<Character, Node> ch = new HashMap<>();
        boolean isWord = false;
    }
    private final Node root = new Node();

    private Node walk(String s) {
        Node n = root;
        for (char c : s.toCharArray()) {
            n = n.ch.get(c);
            if (n == null) return null;
        }
        return n;
    }
    public void insert(String word) {
        Node n = root;
        for (char c : word.toCharArray())
            n = n.ch.computeIfAbsent(c, k -> new Node());
        n.isWord = true;
    }
    public boolean search(String word)       { Node n = walk(word); return n != null &&
n.isWord; }
    public boolean startsWith(String prefix) { return walk(prefix) != null; }
}
```

WALKTHROUGH

`ISWORD` IS THE WHOLE DESIGN.

A node's existence means "some word passes through this prefix"; the flag means "a word *ends* exactly here". After inserting only "apple", the node for "app" exists but is not a word — so `search("app")` is false while `startsWith("app")` is true. The first test case exercises exactly that distinction, then inserts "app" and re-searches to confirm the flag flips.

SEARCH AND STARTSWITH ARE THE SAME WALK.

Factoring the descent into `walk` makes that explicit and removes duplicated code — a small thing that reads well in an interview.

MAP VERSUS FIXED ARRAY.

A 26-element array per node is faster and simpler for lowercase input, but wastes memory on sparse tries and cannot handle arbitrary characters. A hash map is general. Mention the trade; for a dictionary of English words the array version is usually the right call.

Design add and search words — with wildcards `Medium`

Pattern: Trie + branching DFS

Design a structure supporting `addWord(word)` and `search(word)`, where the search string may contain `.` as a wildcard matching **any single character**.

Implement `wordDictionary(ops, args)` returning the boolean results of the `search` calls, in order.

Input	Output	Note
`ops = ["addWord","addWord","addWord","search","search","search","search"], args = [["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]`	`[false,true,true,true]`	

HINTS

1. A plain trie walk assumes one child per character. A wildcard has to try them all.
2. Make the search recursive: at a normal character descend into that one child; at `.` recurse into **every** child.
3. The base case is reaching the end of the pattern — then check `isWord`.
4. Return true as soon as any branch succeeds.

Approach	Idea	Time	Space
Store words, match each	Compare the pattern against every stored word.	`O(n·L)`	`O(n·L)`
Trie + DFS with branching	Exact characters prune to one child; wildcards branch. Fast in practice because the trie shares prefixes.	`O(L) typical, O(26^d) worst`	`O(total chars)`

Solution — js

```
function wordDictionary(ops, args) {
  const root = { children: new Map(), isWord: false };
  const out = [];

  function dfs(node, s, i) {
    if (i === s.length) return node.isWord;      // pattern consumed

    const c = s[i];
    if (c === ".") {
      // wildcard: ANY child may work
      for (const child of node.children.values()) {
        if (dfs(child, s, i + 1)) return true;
      }
      return false;
    }

    const child = node.children.get(c);
    return child ? dfs(child, s, i + 1) : false;
  }

  for (let i = 0; i < ops.length; i++) {
    const s = args[i][0];
    if (ops[i] === "addWord") {
      let node = root;
      for (const c of s) {
        if (!node.children.has(c))
          node.children.set(c, { children: new Map(), isWord: false });
        node = node.children.get(c);
      }
      node.isWord = true;
    } else {
      out.push(dfs(root, s, 0));
    }
  }
  return out;
}
```

Solution — cpp

```
class WordDictionary {
    struct Node {
        unordered_map<char, Node*> ch;
        bool isWord = false;
    };
    Node* root = new Node();

    bool dfs(Node* n, const string& s, int i) {
        if (i == (int)s.size()) return n->isWord;
        char c = s[i];
        if (c == '.') {
            for (auto& [k, child] : n->ch)
                if (dfs(child, s, i + 1)) return true;
            return false;
        }
        auto it = n->ch.find(c);
        return it != n->ch.end() && dfs(it->second, s, i + 1);
    }
public:
    void addWord(string word) {
        Node* n = root;
        for (char c : word) {
            if (!n->ch.count(c)) n->ch[c] = new Node();
            n = n->ch[c];
        }
        n->isWord = true;
    }
    bool search(string word) { return dfs(root, word, 0); }
};
```

Solution — python

```
class WordDictionary:
    def __init__(self):
        self.root = {}          # nested dicts; "$" marks a word end

    def addWord(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node["$"] = True

    def search(self, word: str) -> bool:
        def dfs(node, i):
            if i == len(word):
                return "$" in node
            c = word[i]
            if c == ".":
                return any(dfs(child, i + 1)
                           for k, child in node.items() if k != "$")
            return c in node and dfs(node[c], i + 1)
        return dfs(self.root, 0)
```

Solution — java

```
class WordDictionary {
    private static class Node {
        Map<Character, Node> ch = new HashMap<>();
        boolean isWord = false;
    }
    private final Node root = new Node();

    public void addWord(String word) {
        Node n = root;
        for (char c : word.toCharArray())
            n = n.ch.computeIfAbsent(c, k -> new Node());
        n.isWord = true;
    }
    public boolean search(String word) { return dfs(root, word, 0); }

    private boolean dfs(Node n, String s, int i) {
        if (i == s.length()) return n.isWord;
        char c = s.charAt(i);
        if (c == '.') {
            for (Node child : n.ch.values())
                if (dfs(child, s, i + 1)) return true;
            return false;
        }
        Node child = n.ch.get(c);
        return child != null && dfs(child, s, i + 1);
    }
}
```

WALKTHROUGH

THE WILDCARD TURNS A WALK INTO A SEARCH.

Without `.`, each character determines exactly one child and the descent is a straight line of length L. A `.` means every child is a candidate, so the traversal fans out and needs backtracking — which is what makes this a DFS rather than a loop.

THE BASE CASE MUST CHECK `ISWORD`, NOT JUST ARRIVAL.

Test `addWord("ab")` then `search("a")`: the walk reaches the node for "a" with the pattern exhausted, but no word ends there, so the answer is false. Returning `true` on arrival is the classic bug.

LENGTH MISMATCHES ARE HANDLED IMPLICITLY.

`search("abc")` after adding only "ab" fails because the node for "b" has no "c" child. Nothing special is needed.

COMPLEXITY HONESTLY.

With no wildcards it is O(L). A pattern of all dots explores every path of that length, which is O(26^L) in the worst case — but the trie only contains real words, so the branching is bounded by what was actually inserted. Saying "worst case exponential in the number of wildcards, but bounded by the trie's actual branching" is the right level of precision.

Graphs: representation, BFS and DFS

Nodes and edges. Half of graph problems are solved by picking the right traversal and remembering to mark visited nodes at the right moment.

Representation

```
ADJACENCY LIST — the default, and almost always correct.

  const g = new Map();          // node -> array of neighbours
  // or, for nodes 0..n-1:
  const g = Array.from({length: n}, () => []);

  space O(V + E), iterate a node's neighbours in O(deg)

ADJACENCY MATRIX — only for dense graphs or O(1) edge
tests.

  const g = Array.from({length: n}, () => new Array(n).fill(0));
  space O(V^2), "is there an edge u-v?" in O(1)

EDGE LIST — just [u, v, w] triples. Used by Kruskal and
Bellman-Ford (chapter 30).

BUILDING FROM AN EDGE LIST — the first three lines of
most graph solutions:

  const g = Array.from({length: n}, () => []);
  for (const [u, v] of edges) {
      g[u].push(v);
      g[v].push(u);       // OMIT for a DIRECTED graph
  }

THE VOCABULARY YOU WILL BE ASKED FOR:
  DIRECTED / UNDIRECTED    do edges have a direction?
  WEIGHTED / UNWEIGHTED    do edges carry a cost?
  CYCLIC / ACYCLIC         a DAG is a directed acyclic
                           graph
  CONNECTED                every node reachable from every
                           other
  DEGREE                   number of incident edges;
                           in-degree / out-degree when
                           directed
  A TREE is a connected acyclic undirected graph with
  exactly V-1 edges.
```

BFS and DFS

```
BOTH VISIT EVERY REACHABLE NODE ONCE, in O(V + E).
THE DIFFERENCE IS THE ORDER, and it decides which
problems each solves.

BFS — a QUEUE, explores by DISTANCE
  · SHORTEST PATH in an UNWEIGHTED graph
  · level-by-level processing
  · "minimum number of steps"

DFS — a STACK (or recursion), explores by DEPTH
  · connectivity, components, cycle detection
  · topological order
  · path enumeration, backtracking
  · anything needing "finish the subtree first"

------------------------------------------------------------
BFS TEMPLATE

  const q = [start];
  const seen = new Set([start]);      // MARK ON ENQUEUE
  let head = 0;                       // avoid shift()
  let dist = 0;

  while (head < q.length) {
      const size = q.length - head;   // one LEVEL
      for (let i = 0; i < size; i++) {
          const u = q[head++];
          for (const v of g[u]) {
              if (seen.has(v)) continue;
              seen.add(v);            // <-- HERE, not on
              q.push(v);              //     dequeue
          }
      }
      dist++;
  }

  MARKING ON DEQUEUE INSTEAD OF ENQUEUE lets the same node
  be queued many times before it is first processed. It
  still terminates, but the queue can blow up
  exponentially on dense graphs. THIS IS THE MOST COMMON
  BFS BUG.

------------------------------------------------------------
DFS TEMPLATE — recursive

  const seen = new Set();
  function dfs(u) {
      seen.add(u);
      for (const v of g[u]) if (!seen.has(v)) dfs(v);
  }

DFS TEMPLATE — iterative, when recursion may overflow

  const st = [start];
  const seen = new Set();
  while (st.length) {
      const u = st.pop();
      if (seen.has(u)) continue;   // may be queued twice
      seen.add(u);
      for (const v of g[u]) if (!seen.has(v)) st.push(v);
  }
```

```
  Note the iterative version checks on POP, because a node
  can be pushed by several neighbours before being
  processed. That is fine here — unlike BFS, it does not
  break the distance guarantee, because DFS has none.

------------------------------------------------------------
WHY BFS GIVES SHORTEST PATHS

  The queue processes nodes in non-decreasing distance
  order. So the first time a node is reached, it is
  reached by a shortest path — and marking it seen
  immediately locks that in. WITH WEIGHTS THIS FAILS, and
  you need Dijkstra (chapter 30).
```

Problems

How it shows up

Symptom	Cause
Infinite loop	No visited set, or a cycle in an undirected graph
Queue grows enormously	Marked visited on dequeue instead of enqueue
Shortest path too long	Used DFS, or the graph is weighted — use BFS or Dijkstra
Only part of the graph visited	Disconnected — loop over all nodes as start points
Stack overflow	Deep recursion — convert DFS to iterative
Undirected graph behaves as directed	Forgot to add the reverse edge when building
Levels merged together	Level size not snapshot before the inner loop
Self-loops or duplicate edges break it	The visited check handles them — do not assume simplicity

Check yourself

- ✓ Build an adjacency list from an edge list, directed and undirected.
- ✓ Say when a matrix beats a list.
- ✓ Give the complexity of both traversals.
- ✓ State which problems suit BFS and which suit DFS.
- ✓ Explain why BFS marks visited on enqueue and what goes wrong otherwise.
- ✓ Explain why BFS finds shortest paths and when that guarantee fails.
- ✓ Write the iterative DFS and say why it checks on pop.
- ✓ Say how to handle a disconnected graph.

Problems

Number of connected components `Medium`

Pattern: DFS from every unvisited node

You have a graph of `n` nodes labelled `0` to `n-1`, given as an **undirected** edge list. Return the number of connected components.

Input	Output	Note
`n = 5, edges = [[0,1],[1,2],[3,4]]`	`2`	
`n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]`	`1`	
`n = 4, edges = []`	`4`	Isolated nodes are components too.

HINTS

1. Build an adjacency list first — remember to add *both* directions.
2. Start a traversal from every node that has not been visited yet.
3. Each traversal you start marks one whole component, so count the starts.
4. Union-find also solves this — see chapter 29.

Approach	Idea	Time	Space
DFS or BFS per unvisited node	Each new start discovers exactly one component.	`O(V + E)`	`O(V + E)`
Union-find	Union the endpoints of each edge; count distinct roots. Better when edges arrive as a stream.	`O(E·α(V))`	`O(V)`

Solution — js

```
function countComponents(n, edges) {
  // adjacency list — BOTH directions for an undirected graph
  const g = Array.from({length: n}, () => []);
  for (const [u, v] of edges) { g[u].push(v); g[v].push(u); }

  const seen = new Array(n).fill(false);
  let count = 0;

  for (let s = 0; s < n; s++) {
    if (seen[s]) continue;
    count++;                      // a new component starts here

    // iterative DFS to avoid recursion depth limits
    const st = [s];
    seen[s] = true;
    while (st.length) {
      const u = st.pop();
      for (const v of g[u]) {
        if (!seen[v]) { seen[v] = true; st.push(v); }
      }
    }
  }
  return count;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int countComponents(int n, vector<vector<int>>& edges) {
        vector<vector<int>> g(n);
        for (auto& e : edges) { g[e[0]].push_back(e[1]); g[e[1]].push_back(e[0]); }
        vector<bool> seen(n, false);
        int count = 0;
        for (int s = 0; s < n; ++s) {
            if (seen[s]) continue;
            ++count;
            stack<int> st; st.push(s); seen[s] = true;
            while (!st.empty()) {
                int u = st.top(); st.pop();
                for (int v : g[u]) if (!seen[v]) { seen[v] = true; st.push(v); }
            }
        }
        return count;
    }
};
```

Solution — python

```python
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)

        seen = [False] * n
        count = 0
        for s in range(n):
            if seen[s]:
                continue
            count += 1
            st = [s]
            seen[s] = True
            while st:
                u = st.pop()
                for v in g[u]:
                    if not seen[v]:
                        seen[v] = True
                        st.append(v)
        return count
```

Solution — java

```java
class Solution {
    public int countComponents(int n, int[][] edges) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        for (int[] e : edges) { g.get(e[0]).add(e[1]); g.get(e[1]).add(e[0]); }

        boolean[] seen = new boolean[n];
        int count = 0;
        for (int s = 0; s < n; s++) {
            if (seen[s]) continue;
            count++;
            Deque<Integer> st = new ArrayDeque<>();
            st.push(s); seen[s] = true;
            while (!st.isEmpty()) {
                int u = st.pop();
                for (int v : g.get(u))
                    if (!seen[v]) { seen[v] = true; st.push(v); }
            }
        }
        return count;
    }
}
```

WALKTHROUGH

WHY THE OUTER LOOP IS NECESSARY.

A single traversal only reaches nodes connected to its start. Iterating over every node and launching a traversal whenever one is still unvisited guarantees complete coverage, and each launch corresponds to exactly one component — which is why counting launches counts components.

ISOLATED NODES COUNT.

`n = 4` with no edges is four components, and the code handles it because each node is its own unvisited start. Forgetting this is a common off-by-several.

THE DUPLICATE-EDGE TEST

`[[0,1],[0,1]]` confirms the visited check tolerates a multigraph. Real inputs are not always simple graphs, and assuming otherwise is a silent failure.

MARKING `SEEN` WHEN PUSHING

rather than when popping keeps the stack small here — a node cannot be pushed twice. It is not strictly required for DFS correctness, but it is tidier and mirrors the BFS discipline.

Shortest path in an unweighted graph `Medium`

Pattern: BFS by levels

Given `n` nodes labelled `0..n-1`, an **undirected** edge list, and two nodes `src` and `dst`, return the minimum number of edges on a path from `src` to `dst`, or `-1` if unreachable.

Input	Output	Note
`n = 6, edges = [[0,1],[1,2],[2,5],[0,3],[3,4],[4,5]], src = 0, dst = 5`	`3`	
`n = 3, edges = [[0,1]], src = 0, dst = 2`	`-1`	
`n = 1, edges = [], src = 0, dst = 0`	`0`	

HINTS

1. DFS finds *a* path but not necessarily the shortest. BFS explores in order of distance.
2. Track the distance either by processing a whole level per iteration, or by storing a distance per node.
3. Mark nodes visited when you enqueue them, not when you dequeue them.
4. The source is at distance 0 — handle `src === dst` before anything else.

Approach	Idea	Time	Space
DFS all paths	Enumerate every path and take the shortest. Exponential.	`O(V!)`	`O(V)`
BFS	The first arrival at a node is via a shortest path.	`O(V + E)`	`O(V)`
Bidirectional BFS	Search from both ends and meet in the middle; roughly halves the explored frontier.	`O(V + E)`	`O(V)`

Solution — js

```
function shortestPath(n, edges, src, dst) {
  const g = Array.from({length: n}, () => []);
  for (const [u, v] of edges) { g[u].push(v); g[v].push(u); }

  if (src === dst) return 0;

  const q = [src];
  const seen = new Array(n).fill(false);
  seen[src] = true;                 // MARK ON ENQUEUE
  let head = 0, dist = 0;

  while (head < q.length) {
    const size = q.length - head;   // process one whole LEVEL
    dist++;
    for (let i = 0; i < size; i++) {
      const u = q[head++];
      for (const v of g[u]) {
        if (seen[v]) continue;
        if (v === dst) return dist; // first arrival = shortest
        seen[v] = true;
        q.push(v);
      }
    }
  }
  return -1;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int shortestPath(int n, vector<vector<int>>& edges, int src, int dst) {
        vector<vector<int>> g(n);
        for (auto& e : edges) { g[e[0]].push_back(e[1]); g[e[1]].push_back(e[0]); }
        if (src == dst) return 0;
        vector<bool> seen(n, false);
        queue<int> q; q.push(src); seen[src] = true;
        int dist = 0;
        while (!q.empty()) {
            int size = q.size();
            ++dist;
            for (int i = 0; i < size; ++i) {
                int u = q.front(); q.pop();
                for (int v : g[u]) {
                    if (seen[v]) continue;
                    if (v == dst) return dist;
                    seen[v] = true;
                    q.push(v);
                }
            }
        }
        return -1;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def shortestPath(self, n, edges, src, dst):
        g = [[] for _ in range(n)]
        for u, v in edges:
            g[u].append(v)
            g[v].append(u)
        if src == dst:
            return 0
        seen = [False] * n
        seen[src] = True
        q = deque([src])
        dist = 0
        while q:
            dist += 1
            for _ in range(len(q)):
                u = q.popleft()
                for v in g[u]:
                    if seen[v]:
                        continue
                    if v == dst:
                        return dist
                    seen[v] = True
                    q.append(v)
        return -1
```

Solution — java

```java
class Solution {
    public int shortestPath(int n, int[][] edges, int src, int dst) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        for (int[] e : edges) { g.get(e[0]).add(e[1]); g.get(e[1]).add(e[0]); }
        if (src == dst) return 0;
        boolean[] seen = new boolean[n];
        Deque<Integer> q = new ArrayDeque<>();
        q.add(src); seen[src] = true;
        int dist = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            dist++;
            for (int i = 0; i < size; i++) {
                int u = q.poll();
                for (int v : g.get(u)) {
                    if (seen[v]) continue;
                    if (v == dst) return dist;
                    seen[v] = true;
                    q.add(v);
                }
            }
        }
        return -1;
    }
}
```

WALKTHROUGH

WHY THE FIRST ARRIVAL IS OPTIMAL.

BFS dequeues nodes in non-decreasing order of distance from the source. So when a node is first discovered, no shorter route to it can exist — anything shorter would have been explored in an earlier level. Marking it seen at that moment permanently records the optimal distance.

MARK ON ENQUEUE, NOT ON DEQUEUE.

If you defer the mark until a node is popped, several neighbours can each enqueue the same node before any of them is processed. The result is still correct but the queue can grow far beyond V, and on a dense graph that is a real blow-up. This is the single most common BFS bug.

LEVEL COUNTING VERSUS A DISTANCE ARRAY.

Here the level loop increments `dist` once per frontier. The alternative — storing `dist[v] = dist[u] + 1` per node — is equally valid and is what you want if you need *all* distances rather than one. Both are O(V + E).

THE `SRC === DST` GUARD

is needed because the loop only returns when it *discovers* `dst` as a neighbour, and the source is never discovered that way. The `n = 1` test catches it.

Graphs: grids, topological sort and cycles

Three applications that cover most graph questions asked in interviews: a grid is a graph, dependencies need an ordering, and detecting cycles is what tells you whether one exists.

A grid is a graph

```
EVERY CELL IS A NODE. Its neighbours are the adjacent
cells. Nothing else changes — BFS and DFS work unaltered.

  const DIRS = [[-1,0],[1,0],[0,-1],[0,1]];
  for (const [dr, dc] of DIRS) {
      const nr = r + dr, nc = c + dc;
      if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
      ...
  }

THE STANDARD PROBLEMS:
  COUNT ISLANDS       DFS/BFS from each unvisited land
                      cell; each launch is one island
  FLOOD FILL          same traversal, writing a new value
  ROTTING ORANGES     MULTI-SOURCE BFS — push every
                      initial source before starting, and
                      the level count is the answer
  SHORTEST PATH       BFS, because every step costs 1
  SURROUNDED REGIONS  traverse from the BORDER inwards to
                      find what is NOT enclosed

MARKING VISITED: either a separate boolean grid, or
overwrite the cell itself (turning '1' into '0'). The
second is O(1) space but MUTATES THE INPUT — say which
you chose.

MULTI-SOURCE BFS IS THE PATTERN PEOPLE MISS. If several
starting points spread simultaneously, seed the queue
with all of them at distance 0. One BFS then computes
the distance to the NEAREST source for every cell.
```

Topological sort

```
AN ORDERING OF A DIRECTED ACYCLIC GRAPH such that every
edge u -> v puts u before v. "Do this before that."

IT EXISTS IF AND ONLY IF THE GRAPH IS ACYCLIC — which is
why the same algorithm detects cycles.

------------------------------------------------------------
KAHN'S ALGORITHM (BFS on in-degrees) — the one to write

  1. compute in-degree for every node
  2. queue every node with in-degree 0
  3. pop u, append to the order, and for each edge u -> v
     decrement in-degree[v]; if it reaches 0, enqueue v
  4. IF THE ORDER IS SHORTER THAN n, A CYCLE EXISTS

  THAT LAST LINE IS THE CYCLE TEST. Nodes inside a cycle
  never reach in-degree 0, so they are never emitted.

------------------------------------------------------------
DFS ALTERNATIVE

  Postorder DFS, then REVERSE the finishing order.
  Detect cycles with THREE COLOURS:
     0 unvisited   1 IN PROGRESS   2 done
  Meeting a node in state 1 means a BACK EDGE — a cycle.

  TWO COLOURS ARE NOT ENOUGH for a directed graph:
  revisiting a FINISHED node is fine (a diamond), but
  revisiting an IN-PROGRESS one is a cycle. Conflating
  them reports false cycles on any DAG with two paths to
  the same node.

------------------------------------------------------------
CYCLE DETECTION, and the directed/undirected difference

  DIRECTED    three-colour DFS, or Kahn's count
  UNDIRECTED  DFS tracking the PARENT — seeing a visited
              node that is not your parent is a cycle.
              (Union-find also works: an edge whose
              endpoints already share a root closes a
              cycle — chapter 29.)

  Using the directed method on an undirected graph
  reports a "cycle" for every single edge, because u-v is
  stored as both u->v and v->u.
```

Problems

How it shows up

Symptom	Cause
Island count too high	Not marking cells visited during the traversal
Grid BFS gives the wrong distance	Not multi-source, or level size not snapshot
Topological sort returns too few nodes	Correct — that is the cycle signal
False cycle in a DAG	Two-colour DFS instead of three

Symptom	Cause
Every undirected edge reported as a cycle	Used the directed algorithm — track the parent instead
Course schedule accepts an impossible plan	Never compared the emitted count with `n`
Index out of bounds in a grid	Bounds check missing or applied after the array access
Input grid corrupted	Marked visited by overwriting — document or copy

Check yourself

- ✓ Write the four-direction neighbour loop with its bounds guard.
- ✓ Explain multi-source BFS and a problem it solves.
- ✓ Give the two ways to mark grid cells visited and their trade-off.
- ✓ Write Kahn's algorithm and state the cycle test.
- ✓ Explain why three colours are needed for directed cycle detection.
- ✓ Say how undirected cycle detection differs and why.
- ✓ State the condition for a topological order to exist.

Problems

Number of islands `Medium`

Pattern: Grid DFS per unvisited land cell

Given a 2-D grid of `"1"` (land) and `"0"` (water), return the number of islands. An island is land connected horizontally or vertically, and the grid is surrounded by water.

Input	Output	Note
`[["1","1","0"],["1","0","0"],["0","0","1"]]`	`2`	
`[["1","1","1"],["1","1","1"]]`	`1`	

HINTS

1. Whenever you find an unvisited land cell, it belongs to a new island — so count it and then erase the whole island.
2. "Erasing" means flooding the connected land with a traversal, marking each cell so it is never counted again.
3. You can mark by writing `"0"` into the grid, which needs no extra memory but mutates the input.
4. Guard the bounds *before* indexing, not after.

Approach	Idea	Time	Space
DFS flood fill	Each unvisited land cell launches a traversal that sinks its whole island.	`O(R·C)`	`O(R·C) worst-case stack`
BFS flood fill	Same idea with a queue; bounded frontier instead of deep recursion.	`O(R·C)`	`O(min(R,C))`

Approach	Idea	Time	Space
Union-find	Union adjacent land cells and count roots. Natural if the grid changes over time.	`O(R·C·α)`	`O(R·C)`

Solution — js

```
function numIslands(grid) {
  if (!grid.length || !grid[0].length) return 0;
  const R = grid.length, C = grid[0].length;
  const DIRS = [[-1,0],[1,0],[0,-1],[0,1]];
  let count = 0;

  // sink the whole island reachable from (sr, sc)
  function sink(sr, sc) {
    const st = [[sr, sc]];
    grid[sr][sc] = "0";               // mark immediately
    while (st.length) {
      const [r, c] = st.pop();
      for (const [dr, dc] of DIRS) {
        const nr = r + dr, nc = c + dc;
        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
        if (grid[nr][nc] !== "1") continue;
        grid[nr][nc] = "0";           // mark on PUSH
        st.push([nr, nc]);
      }
    }
  }

  for (let r = 0; r < R; r++)
    for (let c = 0; c < C; c++)
      if (grid[r][c] === "1") { count++; sink(r, c); }

  return count;
}
```

Solution — cpp

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        if (grid.empty() || grid[0].empty()) return 0;
        int R = grid.size(), C = grid[0].size(), count = 0;
        const int DR[4] = {-1,1,0,0}, DC[4] = {0,0,-1,1};
        for (int r = 0; r < R; ++r)
            for (int c = 0; c < C; ++c) {
                if (grid[r][c] != '1') continue;
                ++count;
                stack<pair<int,int>> st;
                st.push({r, c}); grid[r][c] = '0';
                while (!st.empty()) {
                    auto [cr, cc] = st.top(); st.pop();
                    for (int d = 0; d < 4; ++d) {
                        int nr = cr + DR[d], nc = cc + DC[d];
                        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                        if (grid[nr][nc] != '1') continue;
                        grid[nr][nc] = '0';
                        st.push({nr, nc});
                    }
                }
            }
        return count;
    }
};
```

Solution — python

```
class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        if not grid or not grid[0]:
            return 0
        R, C = len(grid), len(grid[0])
        DIRS = ((-1,0),(1,0),(0,-1),(0,1))
        count = 0

        for r in range(R):
            for c in range(C):
                if grid[r][c] != "1":
                    continue
                count += 1
                st = [(r, c)]
                grid[r][c] = "0"
                while st:
                    cr, cc = st.pop()
                    for dr, dc in DIRS:
                        nr, nc = cr + dr, cc + dc
                        if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == "1":
                            grid[nr][nc] = "0"
                            st.append((nr, nc))
        return count
```

Solution — java

```
class Solution {
    public int numIslands(char[][] grid) {
        if (grid.length == 0 || grid[0].length == 0) return 0;
        int R = grid.length, C = grid[0].length, count = 0;
        int[] DR = {-1,1,0,0}, DC = {0,0,-1,1};
        for (int r = 0; r < R; r++)
            for (int c = 0; c < C; c++) {
                if (grid[r][c] != '1') continue;
                count++;
                Deque<int[]> st = new ArrayDeque<>();
                st.push(new int[]{r, c}); grid[r][c] = '0';
                while (!st.isEmpty()) {
                    int[] cur = st.pop();
                    for (int d = 0; d < 4; d++) {
                        int nr = cur[0] + DR[d], nc = cur[1] + DC[d];
                        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                        if (grid[nr][nc] != '1') continue;
                        grid[nr][nc] = '0';
                        st.push(new int[]{nr, nc});
                    }
                }
            }
        return count;
    }
}
```

WALKTHROUGH

COUNT-THEN-ERASE IS THE SHAPE.

The outer double loop finds any land cell that has not been absorbed into an earlier island; that cell must begin a new one, so increment and then flood the entire connected region to `"0"`. Because every land cell is erased exactly once, the total work is O(R·C) despite the nested traversal.

MARK ON PUSH, NOT ON POP.

Exactly as in BFS (chapter 27), deferring the mark lets the same cell be pushed by several neighbours. Here it would also let a cell be counted while still on the stack, which can inflate the answer.

OVERWRITING THE GRID IS O(1) EXTRA SPACE BUT DESTROYS THE INPUT.

That is usually acceptable and worth stating explicitly; if the caller needs the grid intact, use a parallel boolean array at O(R·C) space, or restore afterwards.

THE BOUNDS CHECK PRECEDES THE ARRAY ACCESS.

Writing `if (grid[nr][nc] !== "1" || nr < 0)` reads out of range before testing — in JavaScript that silently yields `undefined` and in C++ it is undefined behaviour.

Course schedule — can all courses be finished? `Medium`

Pattern: Kahn's topological sort

There are `numCourses` courses labelled `0` to `numCourses-1`. `prerequisites[i] = [a, b]` means you must take `b` before `a`.

Return `true` if it is possible to finish all courses.

Input	Output	Note
`numCourses = 2, prerequisites = [[1,0]]`	`true`	
`numCourses = 2, prerequisites = [[1,0],[0,1]]`	`false`	A circular dependency.

HINTS

1. This asks whether the dependency graph is acyclic — a topological order exists exactly when it is.
2. Build edges `b → a` (prerequisite points to dependant) and count each node's in-degree.
3. Repeatedly take a course with in-degree 0 and decrement its dependants.
4. If you cannot emit all `numCourses` nodes, the remainder are stuck in a cycle.

Approach	Idea	Time	Space
Try every ordering	Permute the courses and check. Factorial.	`O(n!)`	`O(n)`
Three-colour DFS	Detect a back edge to a node still in progress.	`O(V + E)`	`O(V)`
Kahn's algorithm	Peel off in-degree-0 nodes; a short result means a cycle.	`O(V + E)`	`O(V + E)`

Solution — js

```
function canFinish(numCourses, prerequisites) {
  const g = Array.from({length: numCourses}, () => []);
  const indeg = new Array(numCourses).fill(0);

  // [a, b] means b must come BEFORE a, so the edge is b -> a
  for (const [a, b] of prerequisites) {
    g[b].push(a);
    indeg[a]++;
  }

  // start with everything that has no prerequisites
  const q = [];
  for (let i = 0; i < numCourses; i++) if (indeg[i] === 0) q.push(i);

  let head = 0, taken = 0;
  while (head < q.length) {
    const u = q[head++];
    taken++;
    for (const v of g[u]) {
      if (--indeg[v] === 0) q.push(v);   // all prereqs satisfied
    }
  }

  // anything never emitted is trapped in a cycle
  return taken === numCourses;
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> g(numCourses);
        vector<int> indeg(numCourses, 0);
        for (auto& p : prerequisites) { g[p[1]].push_back(p[0]); ++indeg[p[0]]; }
        queue<int> q;
        for (int i = 0; i < numCourses; ++i) if (!indeg[i]) q.push(i);
        int taken = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            ++taken;
            for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
        }
        return taken == numCourses;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def canFinish(self, numCourses: int, prerequisites: list[list[int]]) -> bool:
        g = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1

        q = deque(i for i in range(numCourses) if indeg[i] == 0)
        taken = 0
        while q:
            u = q.popleft()
            taken += 1
            for v in g[u]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    q.append(v)
        return taken == numCourses
```

Solution — java

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) g.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p : prerequisites) { g.get(p[1]).add(p[0]); indeg[p[0]]++; }

        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.add(i);

        int taken = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            taken++;
            for (int v : g.get(u)) if (--indeg[v] == 0) q.add(v);
        }
        return taken == numCourses;
    }
}
```

WALKTHROUGH

THE EDGE DIRECTION IS THE FIRST THING TO GET RIGHT.

`[a, b]` reads "to take `a` you must first take `b`", so `b` is the prerequisite and the edge points `b → a`. Reversing it produces a graph whose topological order is backwards; on symmetric test cases it still passes, which makes the mistake hard to spot. State the direction aloud before coding.

WHY THE COUNT DETECTS CYCLES.

A node only enters the queue when its in-degree reaches zero — that is, when every prerequisite has already been emitted. A node inside a cycle always has at least one unemitted prerequisite (another cycle member), so its in-degree never reaches zero and it is never taken. Hence `taken < numCourses` exactly when a cycle exists.

THE VARIANT TO EXPECT:

"return the actual order". Collect the emitted nodes into an array and return it, or return an empty array when the count falls short. Same algorithm, one extra line.

Rotting oranges `Medium`

Pattern: Multi-source BFS

In a grid, `0` is empty, `1` is a fresh orange and `2` is rotten. Every minute, any fresh orange **adjacent** (4-directionally) to a rotten one becomes rotten.

Return the number of minutes until no fresh orange remains, or `-1` if that is impossible.

Input	Output	Note
`[[2,1,1],[1,1,0],[0,1,1]]`	`4`	
`[[2,1,1],[0,1,1],[1,0,1]]`	`-1`	The bottom-left orange is unreachable.
`[[0,2]]`	`0`	No fresh oranges — zero minutes.

HINTS

1. All rotten oranges spread *simultaneously*, so this is not one BFS per source.
2. Seed the queue with
   EVERY
   rotten orange before starting — that is multi-source BFS.
3. Each BFS level is one minute. Count the levels, not the cells.
4. Count the fresh oranges up front; if any remain at the end, return −1.

Approach	Idea	Time	Space
BFS from each rotten orange	Run a separate BFS per source and take the minimum per cell. Correct but wasteful.	`O((R·C)²)`	`O(R·C)`
Simulate minute by minute	Scan the whole grid each minute looking for spread. O(R·C) per minute.	`O((R·C)²)`	`O(1)`
Multi-source BFS	One traversal from all sources at once; level count is the answer.	`O(R·C)`	`O(R·C)`

Solution — js

```
function orangesRotting(grid) {
  const R = grid.length, C = R ? grid[0].length : 0;
  const DIRS = [[-1,0],[1,0],[0,-1],[0,1]];

  const q = [];
  let fresh = 0;

  // MULTI-SOURCE: every rotten orange starts at minute 0
  for (let r = 0; r < R; r++)
    for (let c = 0; c < C; c++) {
      if (grid[r][c] === 2) q.push([r, c]);
      else if (grid[r][c] === 1) fresh++;
    }

  if (fresh === 0) return 0;      // nothing to rot

  let head = 0, minutes = 0;
  while (head < q.length && fresh > 0) {
    const size = q.length - head;  // one MINUTE = one level
    minutes++;
    for (let i = 0; i < size; i++) {
      const [r, c] = q[head++];
      for (const [dr, dc] of DIRS) {
        const nr = r + dr, nc = c + dc;
        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
        if (grid[nr][nc] !== 1) continue;
        grid[nr][nc] = 2;          // mark on PUSH
        fresh--;
        q.push([nr, nc]);
      }
    }
  }
  return fresh === 0 ? minutes : -1;
}
```

Solution — cpp

```
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int R = grid.size(), C = R ? grid[0].size() : 0;
        const int DR[4] = {-1,1,0,0}, DC[4] = {0,0,-1,1};
        queue<pair<int,int>> q;
        int fresh = 0;
        for (int r = 0; r < R; ++r)
            for (int c = 0; c < C; ++c) {
                if (grid[r][c] == 2) q.push({r, c});
                else if (grid[r][c] == 1) ++fresh;
            }
        if (fresh == 0) return 0;
        int minutes = 0;
        while (!q.empty() && fresh > 0) {
            int size = q.size();
            ++minutes;
            for (int i = 0; i < size; ++i) {
                auto [r, c] = q.front(); q.pop();
                for (int d = 0; d < 4; ++d) {
                    int nr = r + DR[d], nc = c + DC[d];
                    if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                    if (grid[nr][nc] != 1) continue;
                    grid[nr][nc] = 2; --fresh;
                    q.push({nr, nc});
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def orangesRotting(self, grid: list[list[int]]) -> int:
        R = len(grid)
        C = len(grid[0]) if R else 0
        DIRS = ((-1,0),(1,0),(0,-1),(0,1))

        q = deque()
        fresh = 0
        for r in range(R):
            for c in range(C):
                if grid[r][c] == 2:
                    q.append((r, c))
                elif grid[r][c] == 1:
                    fresh += 1

        if fresh == 0:
            return 0

        minutes = 0
        while q and fresh > 0:
            minutes += 1
            for _ in range(len(q)):
                r, c = q.popleft()
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        q.append((nr, nc))
        return minutes if fresh == 0 else -1
```

Solution — java

```
class Solution {
    public int orangesRotting(int[][] grid) {
        int R = grid.length, C = R > 0 ? grid[0].length : 0;
        int[] DR = {-1,1,0,0}, DC = {0,0,-1,1};
        Deque<int[]> q = new ArrayDeque<>();
        int fresh = 0;
        for (int r = 0; r < R; r++)
            for (int c = 0; c < C; c++) {
                if (grid[r][c] == 2) q.add(new int[]{r, c});
                else if (grid[r][c] == 1) fresh++;
            }
        if (fresh == 0) return 0;
        int minutes = 0;
        while (!q.isEmpty() && fresh > 0) {
            int size = q.size();
            minutes++;
            for (int i = 0; i < size; i++) {
                int[] cur = q.poll();
                for (int d = 0; d < 4; d++) {
                    int nr = cur[0] + DR[d], nc = cur[1] + DC[d];
                    if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                    if (grid[nr][nc] != 1) continue;
                    grid[nr][nc] = 2; fresh--;
                    q.add(new int[]{nr, nc});
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

WALKTHROUGH

MULTI-SOURCE IS THE WHOLE IDEA.

Rot spreads from every rotten orange at once, so seeding the queue with all of them makes a single BFS compute, for every cell, the time at which the *nearest* source reaches it. Running one BFS per source and taking minima gives the same answer at far greater cost.

COUNTING MINUTES, NOT CELLS.

Each level of the BFS is one minute of spreading, so `minutes` is incremented once per frontier rather than once per orange.

THE `FRESH > 0` GUARD PREVENTS AN OFF-BY-ONE.

Without it, the loop runs one extra level after the last orange rots — that final level pushes nothing, but `minutes` has already been incremented, returning 5 instead of 4 on the first example. Checking `fresh` both in the loop condition and at the end is what keeps it exact.

THE EARLY RETURN FOR ZERO FRESH ORANGES

handles `[[0,2]]` and `[[0]]`, where the answer is 0 minutes. Without it the function would fall through and still return 0 — but only by accident; being explicit is clearer.

UNREACHABLE ORANGES GIVE −1

, detected by `fresh` still being positive when the queue drains. That is the same "did we reach everything?" check as Kahn's `taken === n`.

Graphs: union-find

A structure that tracks "are these two things in the same group?" while groups keep merging. Thirty lines of code, effectively constant time, and the right answer whenever connectivity arrives incrementally.

The pattern

```
DISJOINT SET UNION (DSU). Each group is a tree; the root
identifies the group.

  find(x)      which group is x in?  (returns the root)
  union(a, b)  merge the two groups
  connected(a, b)  find(a) === find(b)

NAIVE VERSION: parent[] pointers, find walks to the root.
Without optimisation the trees degenerate into chains and
find becomes O(n).

TWO OPTIMISATIONS MAKE IT NEARLY O(1). Use BOTH.

1. PATH COMPRESSION — flatten as you search
     function find(x) {
         while (parent[x] !== x) {
             parent[x] = parent[parent[x]];  // point at
             x = parent[x];                  // grandparent
         }
         return x;
     }
   Every node on the path ends up much closer to the root.

2. UNION BY SIZE (or rank) — attach the SMALLER tree to
   the LARGER, so depth grows slowly.
     if (size[ra] < size[rb]) swap(ra, rb);
     parent[rb] = ra;
     size[ra] += size[rb];

TOGETHER: O(α(n)) amortised per operation, where α is the
inverse Ackermann function. α(n) < 5 for any n you will
ever see, so treat it as constant — but say "amortised
near-constant", not "O(1)".

------------------------------------------------------------
WHEN UNION-FIND BEATS DFS

  DFS/BFS need the WHOLE graph before they can answer.
  Union-find answers connectivity queries INCREMENTALLY,
  as edges arrive, and supports interleaved queries.

  use DFS        one static graph, need paths or order
  use UNION-FIND edges arriving over time, only need
                 "same group?", or you need a component
                 COUNT that updates as you merge

------------------------------------------------------------
THE COMPONENT-COUNT TRICK

  Start with count = n (every node its own group).
  DECREMENT ONLY WHEN A UNION ACTUALLY MERGES:

    function union(a, b) {
        const ra = find(a), rb = find(b);
        if (ra === rb) return false;   // ALREADY together
        ...attach...
        count--;
        return true;                    // a real merge
    }

  THE BOOLEAN RETURN IS THE USEFUL PART. `false` means the
```

```
    edge closed a CYCLE — which is how you detect cycles in
    an undirected graph, and how Kruskal rejects edges
    (chapter 30).
```

When to reach for it

Connected components, incrementally

Number of provinces, friend circles, accounts merge.

Cycle detection, undirected

An edge whose endpoints already share a root closes a cycle.

"Is this a valid tree?"

Exactly `n−1` edges, no cycle, one component.

Kruskal's MST

Sort edges, add any that merges two groups (ch. 30).

Redundant connection

The first edge that fails to merge is the answer.

Equations / equivalence classes

Group things declared equal, then test the inequalities.

Problems

How it shows up

Symptom	Cause
Operations degrade to O(n)	No path compression, or no union by size
Component count too low	Decremented on every union, not only on real merges
Cycle never detected	Ignored the `find(a) === find(b)` case
Union attaches roots to non-roots	Must call `find` on both before attaching
Wrong groups after many unions	Compared `parent[a] === parent[b]` instead of roots
Stack overflow in `find`	Recursive compression on a deep chain — use the iterative form
Cannot recover the actual path	Union-find tracks membership only — use DFS if you need paths

Check yourself

- ✓ Write `find` with path compression, iteratively.
- ✓ Write `union` with union by size, returning whether it merged.
- ✓ State the amortised complexity and phrase it correctly.

- ✓ Say when union-find beats DFS and when it does not.
- ✓ Explain the component-count trick and why the guard matters.
- ✓ Explain how a failed union detects a cycle.
- ✓ Give the three conditions for a graph to be a valid tree.

Problems

Number of provinces `Medium`

Pattern: Union every edge, count roots

You are given an `n × n` matrix `isConnected` where `isConnected[i][j] === 1` means city `i` and city `j` are directly connected. A **province** is a maximal group of directly or indirectly connected cities.

Return the number of provinces.

A `DSU` class is provided — `new DSU(n)`, with `find`, `union`, `connected` and a `count` field.

Input	Output	Note
`isConnected = [[1,1,0],[1,1,0],[0,0,1]]`	`2`	
`isConnected = [[1,0,0],[0,1,0],[0,0,1]]`	`3`	

HINTS

1. Start with every city as its own province and merge whenever an edge connects two.
2. The matrix is symmetric, so only the upper triangle needs scanning.
3. The DSU's `count` field already tracks the number of groups — read it at the end.

Approach	Idea	Time	Space
DFS per unvisited city	Each traversal marks one province.	`O(n²)`	`O(n)`
Union-find	Union every connected pair; `count` is the answer.	`O(n²·α)`	`O(n)`

Solution — js

```
function findCircleNum(isConnected) {
  const n = isConnected.length;
  const dsu = new DSU(n);

  // symmetric matrix: only the upper triangle is needed
  for (let i = 0; i < n; i++)
    for (let j = i + 1; j < n; j++)
      if (isConnected[i][j] === 1) dsu.union(i, j);

  return dsu.count;
}
```

Solution — cpp

```
class Solution {
    vector<int> parent, sz;
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
public:
    int findCircleNum(vector<vector<int>>& isConnected) {
        int n = isConnected.size(), count = n;
        parent.resize(n); sz.assign(n, 1);
        iota(parent.begin(), parent.end(), 0);
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                if (!isConnected[i][j]) continue;
                int ra = find(i), rb = find(j);
                if (ra == rb) continue;
                if (sz[ra] < sz[rb]) swap(ra, rb);
                parent[rb] = ra; sz[ra] += sz[rb]; --count;
            }
        return count;
    }
};
```

Solution — python

```
class Solution:
    def findCircleNum(self, isConnected: list[list[int]]) -> int:
        n = len(isConnected)
        parent = list(range(n))
        size = [1] * n
        count = n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for i in range(n):
            for j in range(i + 1, n):
                if not isConnected[i][j]:
                    continue
                ra, rb = find(i), find(j)
                if ra == rb:
                    continue
                if size[ra] < size[rb]:
                    ra, rb = rb, ra
                parent[rb] = ra
                size[ra] += size[rb]
                count -= 1
        return count
```

Solution — java

```
class Solution {
    private int[] parent, size;
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, count = n;
        parent = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++) {
                if (isConnected[i][j] == 0) continue;
                int ra = find(i), rb = find(j);
                if (ra == rb) continue;
                if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
                parent[rb] = ra; size[ra] += size[rb]; count--;
            }
        return count;
    }
}
```

WALKTHROUGH

THE COUNT MAINTAINS ITSELF.

Starting at `n` and decrementing on each genuine merge means the answer is available at any moment, without a final pass to collect distinct roots. That incremental property is what makes union-find attractive when the graph is built up over time.

ONLY THE UPPER TRIANGLE.

The matrix is symmetric and the diagonal is always 1 (a city is connected to itself), so scanning `j > i` halves the work and skips the meaningless self-edges. Unioning a node with itself is harmless — `find` returns the same root and the union is rejected — but it is wasted work.

EITHER APPROACH IS ACCEPTABLE HERE.

DFS is equally $O(n^2)$ on a dense matrix. Union-find wins when the connections arrive as a stream of edges rather than a fixed matrix, which is worth saying when you choose it.

Redundant connection `Medium`

Pattern: A failed union closes a cycle

A tree on `n` nodes labelled `1..n` has had one extra edge added, creating exactly one cycle. Given the `n` edges in the order they were added, return the edge that can be removed so the result is a tree.

If several answers exist, return the one that appears **last** in the input.

Input	Output	Note
`edges = [[1,2],[1,3],[2,3]]`	`[2,3]`	
`edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]`	`[1,4]`	

HINTS

1. Process the edges in order, unioning each pair.
2. An edge whose endpoints are *already* in the same group would close a cycle.
3. Because you scan in order, the first such edge encountered is the last one that could be removed — return it immediately.
4. Node labels start at 1, so size the DSU `n + 1`.

Approach	Idea	Time	Space
Remove each edge and test	For every edge, delete it and check whether the rest is a tree.	`O(n²)`	`O(n)`
Union-find	The first union that fails is the redundant edge.	`O(n·α)`	`O(n)`

Solution — js

```
function findRedundantConnection(edges) {
  const dsu = new DSU(edges.length + 1);   // 1-indexed labels

  for (const [u, v] of edges) {
    // union returns false when u and v are ALREADY connected,
    // which means this edge closes a cycle
    if (!dsu.union(u, v)) return [u, v];
  }
  return [];
}
```

Solution — cpp

```
class Solution {
    vector<int> parent, sz;
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
public:
    vector<int> findRedundantConnection(vector<vector<int>>& edges) {
        int n = edges.size() + 1;
        parent.resize(n); sz.assign(n, 1);
        iota(parent.begin(), parent.end(), 0);
        for (auto& e : edges) {
            int ra = find(e[0]), rb = find(e[1]);
            if (ra == rb) return e;              // closes a cycle
            if (sz[ra] < sz[rb]) swap(ra, rb);
            parent[rb] = ra; sz[ra] += sz[rb];
        }
        return {};
    }
};
```

Solution — python

```python
class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        n = len(edges) + 1
        parent = list(range(n))
        size = [1] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for u, v in edges:
            ra, rb = find(u), find(v)
            if ra == rb:
                return [u, v]
            if size[ra] < size[rb]:
                ra, rb = rb, ra
            parent[rb] = ra
            size[ra] += size[rb]
        return []
```

Solution — java

```java
class Solution {
    private int[] parent, size;
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    public int[] findRedundantConnection(int[][] edges) {
        int n = edges.length + 1;
        parent = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        for (int[] e : edges) {
            int ra = find(e[0]), rb = find(e[1]);
            if (ra == rb) return e;
            if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
            parent[rb] = ra; size[ra] += size[rb];
        }
        return new int[0];
    }
}
```

WALKTHROUGH

WHY A FAILED UNION MEANS A CYCLE.

If `u` and `v` already share a root, a path between them exists using earlier edges. Adding the direct edge `u–v` therefore creates a second path, which is precisely a cycle. Union-find detects this in near-constant time without ever tracing the path.

WHY THE FIRST FAILURE IS THE ANSWER THE PROBLEM WANTS.

The graph has exactly one cycle, and edges are processed in input order. Every edge before the failure was necessary to build the tree so far; the failing edge is the one that closed the loop, and by construction it is the last edge of the cycle in input order — which is what "return the one appearing last" asks for.

SIZE THE DSU FOR 1-INDEXED LABELS.

Nodes run `1..n` where `n === edges.length`, so the arrays need `n + 1` slots and index 0 goes unused. Off-by-one here produces an out-of-range write that is easy to miss.

CONTRAST WITH THE DIRECTED VERSION

("Redundant Connection II"), where a node can also gain two parents — that variant needs extra case analysis and is a common follow-up.

Graph valid tree `Medium`

Pattern: Edge count + no cycle + connected

Given `n` nodes labelled `0..n-1` and a list of undirected edges, determine whether the graph forms a **valid tree**.

A tree is connected and acyclic.

Input	Output	Note
`n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]`	`true`	
`n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]`	`false`	Contains a cycle.
`n = 4, edges = [[0,1],[2,3]]`	`false`	Two components.

HINTS

1. A tree on `n` nodes has exactly `n - 1` edges. Check that first — it is O(1) and rules out most bad inputs.
2. Given the right edge count, "acyclic" and "connected" become equivalent: either one implies the other.
3. So union every edge and reject if any union fails (a cycle).
4. Alternatively check `dsu.count === 1` at the end.

Approach	Idea	Time	Space
DFS with parent tracking	Detect a cycle by revisiting a non-parent node, then verify all nodes were reached.	`O(V + E)`	`O(V + E)`
Union-find	Edge count check, then reject any union that closes a cycle.	`O(E·α)`	`O(V)`

Solution — js

```
function validTree(n, edges) {
  // a tree on n nodes has EXACTLY n-1 edges
  if (edges.length !== n - 1) return n === 0 && edges.length === 0;

  const dsu = new DSU(n);
  for (const [u, v] of edges) {
    if (!dsu.union(u, v)) return false;   // a cycle
  }
  // n-1 edges and no cycle => necessarily connected
  return true;
}
```

Solution — cpp

```
class Solution {
    vector<int> parent, sz;
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
public:
    bool validTree(int n, vector<vector<int>>& edges) {
        if (n == 0) return edges.empty();
        if ((int)edges.size() != n - 1) return false;
        parent.resize(n); sz.assign(n, 1);
        iota(parent.begin(), parent.end(), 0);
        for (auto& e : edges) {
            int ra = find(e[0]), rb = find(e[1]);
            if (ra == rb) return false;
            if (sz[ra] < sz[rb]) swap(ra, rb);
            parent[rb] = ra; sz[ra] += sz[rb];
        }
        return true;
    }
};
```

Solution — python

```python
class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0:
            return not edges
        if len(edges) != n - 1:
            return False

        parent = list(range(n))
        size = [1] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for u, v in edges:
            ra, rb = find(u), find(v)
            if ra == rb:
                return False
            if size[ra] < size[rb]:
                ra, rb = rb, ra
            parent[rb] = ra
            size[ra] += size[rb]
        return True
```

Solution — java

```java
class Solution {
    private int[] parent, size;
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    public boolean validTree(int n, int[][] edges) {
        if (n == 0) return edges.length == 0;
        if (edges.length != n - 1) return false;
        parent = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        for (int[] e : edges) {
            int ra = find(e[0]), rb = find(e[1]);
            if (ra == rb) return false;
            if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
            parent[rb] = ra; size[ra] += size[rb];
        }
        return true;
    }
}
```

WALKTHROUGH

THE EDGE-COUNT CHECK DOES MOST OF THE WORK.

A tree on `n` nodes has exactly `n - 1` edges — fewer means it cannot be connected, more means it must contain a cycle. Testing that first is O(1) and eliminates `[[0,1],[2,3]]` with `n = 4` before any traversal.

GIVEN THE RIGHT COUNT, ONE CHECK SUFFICES.

With exactly `n - 1` edges, connectivity and acyclicity are equivalent: a graph with `n - 1` edges and no cycle is a spanning tree, hence connected; and if it were disconnected it would need a cycle somewhere to use up the edges. So verifying "no union fails" is enough — you do not also need to check `dsu.count === 1`, though doing so is harmless and arguably clearer.

THE `N = 0` CASE

is a convention question. An empty graph is usually accepted as a (degenerate) tree; `edges.length !== n - 1` would be `0 !== -1`, which is true, so the guard is needed. Ask the interviewer rather than guessing.

Graphs: shortest paths and spanning trees

Once edges have weights, BFS stops working. Four algorithms cover everything you will be asked, and choosing between them is mostly about negative edges and how many sources you need.

Choosing the algorithm

Situation	Algorithm	Time
Unweighted, one source	**BFS**	O(V + E)
Weights 0 or 1	**0-1 BFS** (deque)	O(V + E)
Non-negative weights, one source	**Dijkstra**	O((V + E) log V)
Negative weights allowed	**Bellman–Ford**	O(V · E)
All pairs, small V	**Floyd–Warshall**	$O(V^3)$
Cheapest set of edges connecting everything	**Kruskal / Prim** (MST)	O(E log E)

```
THE DECISIVE QUESTION IS ALWAYS: ARE THERE NEGATIVE
EDGES?

  no  -> Dijkstra (or BFS if unweighted)
  yes -> Bellman-Ford, which also DETECTS negative cycles

WHY DIJKSTRA BREAKS ON NEGATIVE EDGES: it finalises a
node the moment it is popped, assuming no cheaper route
can appear later. A negative edge can make a later route
cheaper, invalidating a decision already made.

     A --1--> B --1--> C
     A --5--------------> C ... and C --(-10)--> B

  Dijkstra finalises B at cost 1 and never revisits it,
  missing the cost -5 route through C.

------------------------------------------------------------
DIJKSTRA — BFS with a priority queue

  dist = array filled with Infinity;  dist[src] = 0
  pq = min-heap of [distance, node], seeded with [0, src]

  while (pq not empty) {
      [d, u] = pq.pop()
      if (d > dist[u]) continue;        // STALE entry
      for ([v, w] of adj[u]) {
          const nd = d + w;
          if (nd < dist[v]) {           // RELAXATION
              dist[v] = nd;
              pq.push([nd, v]);
          }
      }
  }

  THE STALE CHECK IS ESSENTIAL. A node can be pushed
  several times with different tentative distances; when
  an out-of-date entry surfaces, skip it. Without that
  line the algorithm still terminates but wastes work and
  can relax from a distance that is no longer current.

  (Deleting or decreasing keys inside a heap is the
  alternative, and it is more trouble than it is worth.)

------------------------------------------------------------
MINIMUM SPANNING TREE — a different question

  Dijkstra minimises the distance FROM ONE SOURCE.
  An MST minimises the TOTAL WEIGHT of edges connecting
  everything. They are not the same and one does not
  solve the other.

  KRUSKAL   sort all edges ascending; add an edge if it
            joins two different components (union-find,
            chapter 29). Stop at V-1 edges.
  PRIM      grow one tree: repeatedly take the cheapest
            edge leaving the current set. Dijkstra's shape
            with "edge weight" instead of "path length".

  Kruskal is easier to write when you already have DSU.
```

Problems

How it shows up

Symptom	Cause
Wrong distances with negative edges	Dijkstra is invalid there — use Bellman–Ford
Dijkstra is slow	Missing the stale-entry check, or scanning for the minimum instead of using a heap
Unreachable node reported as 0	Distances must start at Infinity, not 0
Shortest path wrong on an unweighted graph	Used DFS — BFS is required
MST has a cycle	Kruskal must reject edges whose endpoints already share a root
MST incomplete	Graph is disconnected — no spanning tree exists
Used Dijkstra for an MST	Different objectives — minimise edge weight, not path length
Overflow adding distances	`Infinity + w` in fixed-width types — guard before relaxing

Check yourself

- ✓ Pick the right algorithm for six situations.
- ✓ Explain why Dijkstra fails on negative edges, with a concrete graph.
- ✓ Write Dijkstra and explain the stale-entry check.
- ✓ Say what "relaxation" means.
- ✓ Contrast a shortest-path tree with a minimum spanning tree.
- ✓ Describe Kruskal and the role of union-find in it.
- ✓ Say what Bellman–Ford gives you that Dijkstra does not.

Problems

Network delay time `Medium`

Pattern: Dijkstra

You are given `times[i] = [u, v, w]`, a **directed** edge from `u` to `v` taking `w` time. Nodes are labelled `1..n`.

A signal is sent from node `k`. Return the time for **all** nodes to receive it, or `-1` if some node never does.

A `MinHeap` class is provided.

Input	Output	Note
`times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2`	`2`	
`times = [[1,2,1]], n = 2, k = 1`	`1`	
`times = [[1,2,1]], n = 2, k = 2`	`-1`	

HINTS

1. The answer is the *maximum* shortest-path distance from `k` to any node.
2. Weights are positive, so Dijkstra applies.
3. Push `[distance, node]` pairs and skip entries whose distance is worse than the best already recorded.
4. If any node still has distance Infinity at the end, return −1.

Approach	Idea	Time	Space
Bellman–Ford	Relax all edges V−1 times. Simpler to write and handles negatives, but O(V·E).	`O(V·E)`	`O(V)`
Dijkstra with a heap	Finalise nodes in increasing distance order.	`O((V+E) log V)`	`O(V+E)`

Solution — js

```
function networkDelayTime(times, n, k) {
  const adj = Array.from({length: n + 1}, () => []);
  for (const [u, v, w] of times) adj[u].push([v, w]);

  const dist = new Array(n + 1).fill(Infinity);
  dist[k] = 0;

  const pq = new MinHeap((a, b) => a[0] - b[0]);   // [dist, node]
  pq.push([0, k]);

  while (pq.size()) {
    const [d, u] = pq.pop();
    if (d > dist[u]) continue;         // STALE entry, skip

    for (const [v, w] of adj[u]) {
      const nd = d + w;
      if (nd < dist[v]) {              // RELAX
        dist[v] = nd;
        pq.push([nd, v]);
      }
    }
  }

  let worst = 0;
  for (let i = 1; i <= n; i++) {
    if (dist[i] === Infinity) return -1;   // unreachable
    worst = Math.max(worst, dist[i]);
  }
  return worst;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int,int>>> adj(n + 1);
        for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});

        vector<int> dist(n + 1, INT_MAX);
        dist[k] = 0;
        priority_queue<pair<int,int>, vector<pair<int,int>>,
                       greater<pair<int,int>>> pq;
        pq.push({0, k});

        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (auto& [v, w] : adj[u]) {
                if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
            }
        }
        int worst = 0;
        for (int i = 1; i <= n; ++i) {
            if (dist[i] == INT_MAX) return -1;
            worst = max(worst, dist[i]);
        }
        return worst;
    }
};
```

Solution — python

```python
import heapq

class Solution:
    def networkDelayTime(self, times, n, k):
        adj = [[] for _ in range(n + 1)]
        for u, v, w in times:
            adj[u].append((v, w))

        dist = [float("inf")] * (n + 1)
        dist[k] = 0
        pq = [(0, k)]

        while pq:
            d, u = heapq.heappop(pq)
            if d > dist[u]:
                continue
            for v, w in adj[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heapq.heappush(pq, (dist[v], v))

        worst = 0
        for i in range(1, n + 1):
            if dist[i] == float("inf"):
                return -1
            worst = max(worst, dist[i])
        return worst
```

Solution — java

```java
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i <= n; i++) adj.add(new ArrayList<>());
        for (int[] t : times) adj.get(t[0]).add(new int[]{t[1], t[2]});

        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.add(new int[]{0, k});

        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int d = cur[0], u = cur[1];
            if (d > dist[u]) continue;
            for (int[] e : adj.get(u)) {
                if (d + e[1] < dist[e[0]]) {
                    dist[e[0]] = d + e[1];
                    pq.add(new int[]{dist[e[0]], e[0]});
                }
            }
        }
        int worst = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            worst = Math.max(worst, dist[i]);
        }
        return worst;
    }
}
```

WALKTHROUGH

REFRAMING THE QUESTION.

"When has everyone received the signal?" is the maximum over all nodes of the shortest time to reach that node — so it is a single-source shortest-path computation followed by a max. Spotting that the aggregate is a max over shortest paths, not something new, is the step people miss.

THE STALE CHECK EARNS ITS LINE.

A node can be pushed several times as better routes are discovered; the heap has no way to remove the obsolete entries. When one surfaces, `d > dist[u]` is true and it is discarded. Omitting the check does not break correctness — the relaxations would simply fail — but it does waste work, and it is the idiomatic form every reviewer expects.

WHY DIJKSTRA IS VALID HERE.

All times are positive. Popping the smallest tentative distance means no unexplored route can be shorter, because every remaining path costs at least as much and can only grow. That argument is exactly what negative edges destroy.

THE EMPTY-GRAPH CASE

(`n = 1`, no edges) returns 0: the source is reached at time 0 and there is nobody else.

Cheapest flights within K stops `Medium`

Pattern: Bellman–Ford with a bounded relaxation count

Given `n` cities and flights `[from, to, price]`, find the cheapest price from `src` to `dst` using **at most** `k` **stops** (so at most `k + 1` flights). Return `-1` if there is no such route.

Input	Output	Note
`n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src=0, dst=3, k=1`	`700`	0→1→3. The cheaper 0→1→2→3 needs 2 stops.
`n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=1`	`200`	
`n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=0`	`500`	

HINTS

1. Dijkstra optimises price alone and ignores the stop limit, so the cheapest route it finds may use too many hops.
2. Bellman–Ford relaxes all edges repeatedly. After `i` rounds, `dist` holds the best cost using at most `i` edges — exactly the bound you need.
3. Run `k + 1` rounds.
4. Crucially, each round must relax from a
   SNAPSHOT
   of the previous round, or one round can chain several flights.

Approach	Idea	Time	Space
Dijkstra ignoring stops	**Wrong.** Finds the cheapest route regardless of hop count.	`—`	`—`
Dijkstra on (node, stops) states	Correct: treat the stop count as part of the state. More bookkeeping.	`O(E·k log(V·k))`	`O(V·k)`
Bellman–Ford, k+1 rounds	The round count naturally bounds the number of edges used.	`O(k·E)`	`O(V)`

Solution — js

```
function findCheapestPrice(n, flights, src, dst, k) {
  let dist = new Array(n).fill(Infinity);
  dist[src] = 0;

  // after round i, dist[v] = cheapest cost using AT MOST i edges
  for (let round = 0; round <= k; round++) {
    // relax from a SNAPSHOT so one round cannot chain two flights
    const prev = dist.slice();
    for (const [u, v, w] of flights) {
      if (prev[u] === Infinity) continue;      // unreachable so far
      if (prev[u] + w < dist[v]) dist[v] = prev[u] + w;
    }
  }

  return dist[dst] === Infinity ? -1 : dist[dst];
}
```

Solution — cpp

```cpp
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights,
                          int src, int dst, int k) {
        const int INF = 1e9;
        vector<int> dist(n, INF);
        dist[src] = 0;
        for (int round = 0; round <= k; ++round) {
            vector<int> prev = dist;
            for (auto& f : flights) {
                if (prev[f[0]] == INF) continue;
                dist[f[1]] = min(dist[f[1]], prev[f[0]] + f[2]);
            }
        }
        return dist[dst] == INF ? -1 : dist[dst];
    }
};
```

Solution — python

```python
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        INF = float("inf")
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            prev = dist[:]                # snapshot
            for u, v, w in flights:
                if prev[u] == INF:
                    continue
                if prev[u] + w < dist[v]:
                    dist[v] = prev[u] + w
        return -1 if dist[dst] == INF else dist[dst]
```

Solution — java

```java
class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        final int INF = 1_000_000_000;
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 0; round <= k; round++) {
            int[] prev = dist.clone();
            for (int[] f : flights) {
                if (prev[f[0]] == INF) continue;
                dist[f[1]] = Math.min(dist[f[1]], prev[f[0]] + f[2]);
            }
        }
        return dist[dst] == INF ? -1 : dist[dst];
    }
}
```

WALKTHROUGH

THE SNAPSHOT IS THE ENTIRE PROBLEM.

Relaxing in place lets a single round use an edge whose source was itself improved earlier *in that same round*, chaining two flights into one hop count. Copying `dist` before each round forces every relaxation to start from the previous round's values, so after round `i` the array genuinely means "cheapest cost using at most `i` edges". Remove the snapshot and the first example returns 500 instead of 700, because it silently allows the two-stop route.

WHY BELLMAN–FORD SUITS A HOP LIMIT.

Its rounds correspond exactly to edge counts, so bounding the rounds bounds the path length for free. Dijkstra has no such notion — it would need `(node, stopsUsed)` as a composite state, which works but is more code.

ROUNDS, NOT STOPS.

"At most k stops" means at most `k + 1` flights, hence `k + 1` rounds. The `k = 0` test (direct flights only) and the `k = 2` vs `k = 3` pair are there to pin this down.

THE UNREACHABLE GUARD

avoids adding a weight to `Infinity`. In JavaScript that is harmless, but in C++ or Java a sentinel like `INT_MAX` would overflow — hence the smaller `1e9` sentinel and the explicit skip.

Minimum spanning tree cost (Kruskal) `Medium`

Pattern: Sort edges + union-find

Given `n` nodes labelled `0..n-1` and a list of weighted undirected edges `[u, v, w]`, return the total weight of a **minimum spanning tree**, or `-1` if the graph is not connected.

Input	Output	Note
`n = 4, edges = [[0,1,1],[1,2,2],[2,3,3],[0,3,10]]`	`6`	Take the three cheapest: 1 + 2 + 3.
`n = 3, edges = [[0,1,1]]`	`-1`	Node 2 is unreachable.

HINTS

1. Sort the edges by weight ascending, then take each edge only if it joins two currently separate components.
2. Union-find (chapter 29) answers "separate components?" in near-constant time.
3. A spanning tree on `n` nodes has exactly `n - 1` edges — if you finish with fewer, the graph is disconnected.

Approach	Idea	Time	Space
Try all edge subsets	Exponential.	`O(2^E)`	`O(E)`
Prim's algorithm	Grow one tree, always taking the cheapest edge leaving it. Uses a heap; better on dense graphs.	`O(E log V)`	`O(V+E)`
Kruskal's algorithm	Sort edges, accept those that merge components.	`O(E log E)`	`O(V)`

Solution — js

```
function mstCost(n, edges) {
  const parent = Array.from({length: n}, (_, i) => i);
  const size = new Array(n).fill(1);

  function find(x) {
    while (parent[x] !== x) { parent[x] = parent[parent[x]]; x = parent[x]; }
    return x;
  }

  // cheapest edges first
  const sorted = edges.slice().sort((a, b) => a[2] - b[2]);

  let total = 0, used = 0;
  for (const [u, v, w] of sorted) {
    let ru = find(u), rv = find(v);
    if (ru === rv) continue;          // would close a cycle

    if (size[ru] < size[rv]) { const t = ru; ru = rv; rv = t; }
    parent[rv] = ru;
    size[ru] += size[rv];

    total += w;
    if (++used === n - 1) break;      // tree complete
  }

  return used === n - 1 ? total : -1; // fewer edges => disconnected
}
```

Solution — cpp

```
class Solution {
    vector<int> parent, sz;
    int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
public:
    int mstCost(int n, vector<vector<int>>& edges) {
        parent.resize(n); sz.assign(n, 1);
        iota(parent.begin(), parent.end(), 0);
        sort(edges.begin(), edges.end(),
             [](const vector<int>& a, const vector<int>& b){ return a[2] < b[2]; });
        long total = 0; int used = 0;
        for (auto& e : edges) {
            int ru = find(e[0]), rv = find(e[1]);
            if (ru == rv) continue;
            if (sz[ru] < sz[rv]) swap(ru, rv);
            parent[rv] = ru; sz[ru] += sz[rv];
            total += e[2];
            if (++used == n - 1) break;
        }
        return used == n - 1 ? (int)total : -1;
    }
};
```

Solution — python

```python
class Solution:
    def mstCost(self, n: int, edges: list[list[int]]) -> int:
        parent = list(range(n))
        size = [1] * n

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        total = used = 0
        for u, v, w in sorted(edges, key=lambda e: e[2]):
            ru, rv = find(u), find(v)
            if ru == rv:
                continue
            if size[ru] < size[rv]:
                ru, rv = rv, ru
            parent[rv] = ru
            size[ru] += size[rv]
            total += w
            used += 1
            if used == n - 1:
                break
        return total if used == n - 1 else -1
```

Solution — java

```java
class Solution {
    private int[] parent, size;
    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }
    public int mstCost(int n, int[][] edges) {
        parent = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        int[][] sorted = edges.clone();
        Arrays.sort(sorted, (a, b) -> Integer.compare(a[2], b[2]));
        long total = 0; int used = 0;
        for (int[] e : sorted) {
            int ru = find(e[0]), rv = find(e[1]);
            if (ru == rv) continue;
            if (size[ru] < size[rv]) { int t = ru; ru = rv; rv = t; }
            parent[rv] = ru; size[ru] += size[rv];
            total += e[2];
            if (++used == n - 1) break;
        }
        return used == n - 1 ? (int) total : -1;
    }
}
```

WALKTHROUGH

WHY GREEDY IS OPTIMAL — THE CUT PROPERTY.

For any partition of the nodes into two sets, the cheapest edge crossing that partition belongs to some MST. Kruskal processes edges in increasing weight, so when it accepts an edge, that edge is the cheapest one crossing the cut between the two components it joins. Every accepted edge is therefore safe, and the result is a minimum spanning tree.

REJECTING AN EDGE IS EXACTLY THE UNION-FIND CYCLE TEST.

If both endpoints already share a root they are already connected, so adding the edge would create a cycle without connecting anything new — and since it is no cheaper than the edges already linking them, skipping it loses nothing.

COUNTING EDGES DETECTS DISCONNECTION.

A spanning tree needs exactly `n - 1` edges. Finishing the sorted list with fewer means some component was never joined, so no spanning tree exists. The `n = 1` case needs 0 edges and returns 0 — worth checking, since `used === n - 1` is `0 === 0`.

CONTRAST WITH DIJKSTRA ONE MORE TIME.

Dijkstra would connect everything to node 0 by shortest *path*, which is generally a different and more expensive edge set. The objectives differ, and confusing them is the classic error on this topic.

Recursion and divide and conquer

Solve a problem by solving smaller copies of itself. Get comfortable here and backtracking, trees and dynamic programming all become variations rather than new topics.

The pattern

```
EVERY RECURSIVE FUNCTION HAS EXACTLY THREE PARTS:

  1. BASE CASE      the smallest input, answered directly
  2. RECURSIVE CASE reduce toward the base case
  3. COMBINE        build the answer from the sub-answers

  function f(n) {
      if (n <= 1) return 1;        // 1. base
      return n * f(n - 1);         // 2 and 3
  }

THE TWO FAILURES:
  NO BASE CASE           -> infinite recursion, stack
                            overflow
  RECURSION THAT DOES
  NOT SHRINK THE INPUT   -> same thing

ALWAYS CHECK: does every recursive call move strictly
closer to the base case?

------------------------------------------------------------
THE LEAP OF FAITH

Do NOT trace the whole call tree in your head. Instead:

  ASSUME the recursive call already returns the correct
  answer for a smaller input, and ask only "given that,
  how do I build MY answer?"

For reversing a list: assume `reverse(head.next)` has
correctly reversed everything after the head. Then you
only have to attach the head to the end. That is one
line, and you never think about the depth.

THIS IS THE SINGLE MOST USEFUL MENTAL HABIT for
recursion, and it is what makes tree problems (chapter 24)
feel easy.

------------------------------------------------------------
DIVIDE AND CONQUER — a shape of recursion

  1. DIVIDE the problem into independent parts
  2. CONQUER each part recursively
  3. COMBINE the results

  merge sort      split in half, sort each, merge
  quicksort       partition, sort each side
  binary search   discard half (only ONE side survives)
  maximum subarray left, right, or crossing the middle

THE RECURRENCE TELLS YOU THE COST (chapter 02):
  T(n) = 2T(n/2) + O(n)   -> O(n log n)   merge sort
  T(n) = T(n/2)  + O(1)   -> O(log n)     binary search
  T(n) = 2T(n-1) + O(1)   -> O(2^n)       subsets

------------------------------------------------------------
RECURSION VERSUS ITERATION

  Recursion costs O(depth) STACK SPACE, always. A
  recursive walk of a skewed tree or a long list is O(n)
  space and can overflow — typically around 10^4 frames in
```

```
JavaScript, more in C++.

TAIL RECURSION (the recursive call is the last thing
that happens) can in principle be optimised into a loop.
C++ compilers usually do this; JavaScript engines
generally do NOT, despite the specification allowing it.
Do not rely on it.

CONVERT TO ITERATION with an explicit stack when depth
is a risk (chapter 23).
```

When to reach for it

The problem is self-similar

Trees, nested structures, "the same thing but smaller".

Divide into independent halves

Sorting, searching, closest pair.

Enumerate all possibilities

Subsets, permutations — backtracking (ch. 32).

Overlapping subproblems

Add memoisation and it becomes DP (ch. 33).

Do NOT use it

When depth is O(n) and n is large, or a simple loop is clearer.

Problems

How it shows up

Symptom	Cause
Stack overflow	Missing base case, or depth is O(n) on a large input
Returns `undefined`	A branch with no `return`
Exponentially slow	Recomputing the same subproblem — memoise
Wrong answer only for the smallest input	Base case handles the wrong value or is off by one
Works for one call, wrong on the next	Shared mutable state not reset between calls
Result accumulates across recursion	Passing a shared array without copying at the right moment
Infinite recursion on equal inputs	The recursive call does not strictly shrink the input

Check yourself

✓ Name the three parts of a recursive function.

- ✓ State the two ways recursion fails to terminate.
- ✓ Explain the leap of faith and apply it to reversing a linked list.
- ✓ Give the three steps of divide and conquer with two examples.
- ✓ Solve the recurrences for merge sort and binary search.
- ✓ Say what recursion costs in space and when that matters.
- ✓ Explain tail recursion and whether you can rely on it.

Problems

Pow(x, n) — fast exponentiation `Medium`

Pattern: Divide and conquer by halving

Implement `myPow(x, n)` computing `x` raised to the power `n`. `n` may be negative.

Answers within 1e-5 of the correct value are accepted.

Input	Output	Note
`x = 2.0, n = 10`	`1024.0`	
`x = 2.1, n = 3`	`9.261`	
`x = 2.0, n = -2`	`0.25`	$1 / 2^2$

HINTS

1. Multiplying `x` by itself `n` times is O(n). Halving is O(log n).
2. $x^n = (x^{n/2})^2$ for even `n`, and $x \cdot (x^{(n-1)/2})^2$ for odd.
3. Compute the half **ONCE** and square it — calling the recursion twice throws the saving away.
4. For negative `n`, compute the positive power and take the reciprocal.

Approach	Idea	Time	Space
Multiply n times	A simple loop.	`O(n)`	`O(1)`
Recursive halving	Square the half-power. Depth log n.	`O(log n)`	`O(log n)`
Iterative binary exponentiation	Walk the bits of `n`, squaring the base each step. Same speed, O(1) space.	`O(log n)`	`O(1)`

Solution — js

```javascript
function myPow(x, n) {
  if (n < 0) return 1 / myPow(x, -n);   // negative -> reciprocal
  if (n === 0) return 1;                // base case

  // compute the half ONCE, then square it
  const half = myPow(x, Math.floor(n / 2));

  return (n % 2 === 0) ? half * half
                       : half * half * x;
}
```

Solution — cpp

```cpp
class Solution {
public:
    double myPow(double x, int n) {
        long long m = n;                 // -INT_MIN overflows int
        if (m < 0) return 1.0 / power(x, -m);
        return power(x, m);
    }
private:
    double power(double x, long long n) {
        if (n == 0) return 1.0;
        double half = power(x, n / 2);
        return (n % 2 == 0) ? half * half : half * half * x;
    }
};
```

Solution — python

```python
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n == 0:
            return 1.0
        half = self.myPow(x, n // 2)
        return half * half if n % 2 == 0 else half * half * x
```

Solution — java

```java
class Solution {
    public double myPow(double x, int n) {
        long m = n;                      // avoid -Integer.MIN_VALUE
        if (m < 0) return 1.0 / power(x, -m);
        return power(x, m);
    }
    private double power(double x, long n) {
        if (n == 0) return 1.0;
        double half = power(x, n / 2);
        return (n % 2 == 0) ? half * half : half * half * x;
    }
}
```

WALKTHROUGH

COMPUTING THE HALF ONCE IS THE ENTIRE OPTIMISATION.

Writing `myPow(x, n/2) * myPow(x, n/2)` looks equivalent but makes two recursive calls per level, so the call tree doubles at every step and the whole thing collapses back to O(n). Binding the result to a variable and squaring it keeps exactly one call per level, giving depth log n.

NEGATIVE EXPONENTS ARE HANDLED ONCE, AT THE TOP.

Recursing with a negative `n` would never reach the `n === 0` base case. Converting to the positive power and taking the reciprocal at the outermost call keeps the recursion monotone.

THE OVERFLOW TRAP IN C++ AND JAVA.

Negating `INT_MIN` overflows, because its magnitude exceeds `INT_MAX`. Widening to `long` before negating is the standard fix — a detail JavaScript and Python do not have, and one interviewers do probe.

Reverse a string in place, recursively Easy

Pattern: Two pointers via recursion

Reverse an array of characters **in place**, using recursion rather than a loop.

The judge passes an array of single-character strings and checks the array after the call.

Input	Output	Note
`s = ["h","e","l","l","o"]`	`["o","l","l","e","h"]`	
`s = ["H","a","n","n","a","h"]`	`["h","a","n","n","a","H"]`	

HINTS

1. Swap the outermost pair, then recurse on everything between them.
2. The base case is when the two pointers meet or cross.
3. Pass the two indices as parameters — that is what shrinks the problem.

Approach	Idea	Time	Space
Two-pointer loop	Converge from both ends swapping. The obvious answer, O(1) space.	`O(n)`	`O(1)`
Recursive with index parameters	Same swaps, expressed recursively. O(n) stack — worse, but it is what the question asks for.	`O(n)`	`O(n)`

Solution — js

```
function reverseString(s) {
  function helper(lo, hi) {
    if (lo >= hi) return;                 // met or crossed: done
    [s[lo], s[hi]] = [s[hi], s[lo]];      // swap the outer pair
    helper(lo + 1, hi - 1);               // recurse inward
  }
  helper(0, s.length - 1);
}
```

Solution — cpp

```
class Solution {
    void helper(vector<char>& s, int lo, int hi) {
        if (lo >= hi) return;
        swap(s[lo], s[hi]);
        helper(s, lo + 1, hi - 1);
    }
public:
    void reverseString(vector<char>& s) { helper(s, 0, (int)s.size() - 1); }
};
```

Solution — python

```
class Solution:
    def reverseString(self, s: list[str]) -> None:
        def helper(lo, hi):
            if lo >= hi:
                return
            s[lo], s[hi] = s[hi], s[lo]
            helper(lo + 1, hi - 1)
        helper(0, len(s) - 1)
```

Solution — java

```
class Solution {
    public void reverseString(char[] s) { helper(s, 0, s.length - 1); }
    private void helper(char[] s, int lo, int hi) {
        if (lo >= hi) return;
        char t = s[lo]; s[lo] = s[hi]; s[hi] = t;
        helper(s, lo + 1, hi - 1);
    }
}
```

WALKTHROUGH

THE BASE CASE MUST BE `LO >= HI`, NOT `LO === HI`.

With an even length the pointers cross without ever being equal — `["a","b"]` goes from `(0,1)` straight to `(1,0)`. Testing only for equality would recurse past each other forever. The empty-array test also depends on it, since `hi` starts at `-1`.

THE PARAMETERS ARE WHAT SHRINK.

The array itself never gets smaller; the *window* does. That is the usual way to recurse over an array without copying, and it is the same technique backtracking uses (chapter 32).

BE HONEST ABOUT THE TRADE.

The iterative version is O(1) space; this is O(n) stack for no benefit. It is worth writing once to internalise the shape, but in an interview say plainly that the loop is better unless recursion was explicitly requested.

Count inversions with merge sort `Hard`

Pattern: Divide and conquer with a combine-step count

An **inversion** is a pair of indices `i < j` with `nums[i] > nums[j]`. Return the number of inversions in the array.

Must be better than $O(n^2)$.

Input	Output	Note
`nums = [2,4,1,3,5]`	`3`	(2,1), (4,1), (4,3)
`nums = [5,4,3,2,1]`	`10`	Every pair is inverted.
`nums = [1,2,3]`	`0`	

HINTS

1. Counting pairs directly is O(n^2). Merge sort already compares elements across a split — reuse that.
2. Sort each half, then count the inversions *between* the halves during the merge.
3. When you take an element from the right half while `i` elements remain unconsumed in the left half, that element is smaller than all of them — so it forms `i` inversions at once.
4. Total = inversions in the left + in the right + across the merge.

Approach	Idea	Time	Space
Check every pair	Two nested loops.	`O(n²)`	`O(1)`
Merge sort with counting	The merge step reveals cross-half inversions in bulk.	`O(n log n)`	`O(n)`
Fenwick tree	Sweep right to left counting how many smaller values were already seen. Same complexity, different machinery.	`O(n log n)`	`O(n)`

Solution — js

```
function countInversions(nums) {
  // returns [sorted, inversions]
  function sortCount(a) {
    if (a.length <= 1) return [a, 0];

    const mid = a.length >> 1;
    const [left,  cl] = sortCount(a.slice(0, mid));
    const [right, cr] = sortCount(a.slice(mid));

    const merged = [];
    let i = 0, j = 0, cross = 0;

    while (i < left.length && j < right.length) {
      if (left[i] <= right[j]) {
        merged.push(left[i++]);          // no inversion
      } else {
        // right[j] is smaller than EVERY remaining left element
        cross += left.length - i;
        merged.push(right[j++]);
      }
    }
    while (i < left.length)  merged.push(left[i++]);
    while (j < right.length) merged.push(right[j++]);

    return [merged, cl + cr + cross];
  }

  return sortCount(nums.slice())[1];
}
```

Solution — cpp

```cpp
class Solution {
    long long sortCount(vector<int>& a, vector<int>& buf, int lo, int hi) {
        if (hi - lo <= 1) return 0;
        int mid = lo + (hi - lo) / 2;
        long long count = sortCount(a, buf, lo, mid) + sortCount(a, buf, mid, hi);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi) {
            if (a[i] <= a[j]) buf[k++] = a[i++];
            else { count += mid - i; buf[k++] = a[j++]; }
        }
        while (i < mid) buf[k++] = a[i++];
        while (j < hi)  buf[k++] = a[j++];
        copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
        return count;
    }
public:
    long long countInversions(vector<int>& nums) {
        vector<int> a = nums, buf(nums.size());
        return sortCount(a, buf, 0, a.size());
    }
};
```

Solution — python

```python
class Solution:
    def countInversions(self, nums: list[int]) -> int:
        def sort_count(a):
            if len(a) <= 1:
                return a, 0
            mid = len(a) // 2
            left, cl = sort_count(a[:mid])
            right, cr = sort_count(a[mid:])

            merged = []
            i = j = cross = 0
            while i < len(left) and j < len(right):
                if left[i] <= right[j]:
                    merged.append(left[i]); i += 1
                else:
                    cross += len(left) - i
                    merged.append(right[j]); j += 1
            merged.extend(left[i:])
            merged.extend(right[j:])
            return merged, cl + cr + cross

        return sort_count(list(nums))[1]
```

Solution — java

```
class Solution {
    public long countInversions(int[] nums) {
        int[] a = nums.clone(), buf = new int[nums.length];
        return sortCount(a, buf, 0, a.length);
    }
    private long sortCount(int[] a, int[] buf, int lo, int hi) {
        if (hi - lo <= 1) return 0;
        int mid = lo + (hi - lo) / 2;
        long count = sortCount(a, buf, lo, mid) + sortCount(a, buf, mid, hi);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi) {
            if (a[i] <= a[j]) buf[k++] = a[i++];
            else { count += mid - i; buf[k++] = a[j++]; }
        }
        while (i < mid) buf[k++] = a[i++];
        while (j < hi)  buf[k++] = a[j++];
        System.arraycopy(buf, lo, a, lo, hi - lo);
        return count;
    }
}
```

WALKTHROUGH

THE BULK COUNT IS THE INSIGHT.

During the merge, both halves are already sorted. If `right[j] < left[i]`, then `right[j]` is smaller than `left[i]` *and* every element after it in the left half — because the left half is sorted. All of those are inversions, and there are exactly `left.length - i` of them. Counting them one at a time would be O(n^2); counting them as a group is what makes the whole thing O(n log n).

WHY THE THREE TERMS ARE EXHAUSTIVE.

Any inversion has its two indices either both in the left half, both in the right half, or one in each. The first two are counted by the recursive calls, the third by the merge. No pair is counted twice, because the halves are disjoint.

`<=` KEEPS EQUAL ELEMENTS NON-INVERTED.

An inversion requires `nums[i] > nums[j]` strictly, so ties must take from the left and add nothing. The `[3,3,3]` test returns 0 only if you get this right; using `<` would count three phantom inversions.

THIS IS THE SAME TECHNIQUE AS "COUNT OF SMALLER NUMBERS AFTER SELF" AND "REVERSE PAIRS"

— recognising that a sorting algorithm can carry a counter through its combine step is the transferable idea.

Backtracking

Build a candidate one choice at a time, abandon it the moment it cannot work, and undo the choice on the way out. One template covers subsets, permutations, combinations and every puzzle-solver.

The pattern

```
THE TEMPLATE — learn this shape, not the individual
problems:

  function backtrack(path, choices) {
      if (isComplete(path)) {
          result.push([...path]);      // COPY, not the ref
          return;
      }
      for (const c of choices) {
          if (!isValid(c, path)) continue;   // PRUNE

          path.push(c);                // 1. CHOOSE
          backtrack(path, next(c));    // 2. EXPLORE
          path.pop();                  // 3. UN-CHOOSE
      }
  }

THE THREE STEPS ARE ALWAYS CHOOSE / EXPLORE / UN-CHOOSE.
The un-choose is what makes it "backtracking" rather than
plain recursion — it restores the state so the next branch
starts clean.

------------------------------------------------------------
THE TWO MISTAKES THAT ACCOUNT FOR MOST BUGS

1. PUSHING THE PATH WITHOUT COPYING
     result.push(path);      // WRONG — stores a reference
     result.push([...path]); // right — snapshot

   `path` keeps mutating, so every stored reference ends
   up pointing at the same (eventually empty) array.

2. FORGETTING TO UNDO
     Every mutation before the recursive call needs a
     matching undo after it — the array push, a `used[]`
     flag, a board cell, a running sum.

------------------------------------------------------------
SUBSETS VERSUS PERMUTATIONS VERSUS COMBINATIONS

  SUBSETS       order does NOT matter, each item optional
                -> pass `start` so you never look back
                -> 2^n results
  COMBINATIONS  subsets of a fixed size k
                -> same `start`, plus a size check
                -> C(n, k) results
  PERMUTATIONS  order DOES matter, all items used
                -> no `start`; a `used[]` array instead
                -> n! results

  THE `start` PARAMETER IS THE WHOLE DIFFERENCE between
  subsets and permutations. `start` forbids revisiting
  earlier indices, which is exactly what makes [1,2] and
  [2,1] the same subset but different permutations.

------------------------------------------------------------
HANDLING DUPLICATE INPUT

  SORT FIRST, then inside the loop skip a value equal to
  the previous one at the SAME LEVEL:
```

```
        if (i > start && nums[i] === nums[i-1]) continue;

    `i > start` is essential: the first occurrence at each
    level must be allowed; only repeats are skipped.

  ------------------------------------------------------
  COST: exponential by nature. Backtracking is what you use
  when the answer genuinely requires enumeration —
  which is why constraints of n <= 20 or so point here
  (chapter 02). PRUNING is what makes it survivable in
  practice: the earlier you can reject a partial candidate,
  the more of the tree you never build.
```

When to reach for it

"All possible ..."

Subsets, permutations, combinations, partitions.

"Generate every valid ..."

Parentheses, IP addresses, palindrome partitions.

Constraint puzzles

N-Queens, Sudoku, word search.

Path enumeration

All root-to-leaf paths, all routes in a maze.

Small n with no better structure

n ≤ 20 and the problem resists DP or greedy.

Do NOT use it

When you only need a *count* or an *optimum* — that is usually DP (ch. 33).

Problems

How it shows up

Symptom	Cause
All results identical or empty	Pushed the path by reference instead of copying
Results contaminated across branches	Missing the un-choose step
Permutations appear as subsets	Used `start` where a `used[]` array was needed
Duplicate results	Input not sorted, or the same-level skip missing
Valid first occurrence skipped	Skip condition missing the `i > start` guard
Times out	No pruning — reject invalid partials earlier
Stack overflow	Recursion never reaches the completion check

Check yourself

- ✓ Write the template and name the three steps.
- ✓ Explain why the path must be copied when recorded.
- ✓ Say what the `start` parameter does and which problems need it.
- ✓ Contrast subsets, combinations and permutations in result count and mechanism.
- ✓ Give the duplicate-skip line and explain the `i > start` guard.
- ✓ Say what pruning buys and give an example.
- ✓ State when backtracking is the wrong tool.

Problems

Subsets `Medium`

Pattern: Backtracking with a start index

Given an array of **distinct** integers, return all possible subsets (the power set). The solution must not contain duplicate subsets; order does not matter.

Input	Output	Note
`nums = [1,2,3]`	`[[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]`	
`nums = [0]`	`[[],[0]]`	

HINTS

1. Every element is either in a subset or not — so there are 2^n results.
2. Record the current path at *every* node of the recursion, not only at the leaves. Each partial path is itself a valid subset.
3. Pass a `start` index so each branch only considers elements after the one just taken; that prevents [1,2] and [2,1] both appearing.

Approach	Idea	Time	Space
Bitmask enumeration	For each mask $0..2^n-1$, include element `i` when bit `i` is set (chapter 16).	`O(2ⁿ·n)`	`O(n)`
Iterative doubling	Start with [[]]; for each element, append it to a copy of every subset so far.	`O(2ⁿ·n)`	`O(2ⁿ·n)`
Backtracking	Choose / explore / un-choose with a start index.	`O(2ⁿ·n)`	`O(n)` `recursion`

Solution — js

```javascript
function subsets(nums) {
  const out = [], path = [];

  function backtrack(start) {
    out.push([...path]);            // COPY — every node is a subset

    for (let i = start; i < nums.length; i++) {
      path.push(nums[i]);           // 1. CHOOSE
      backtrack(i + 1);             // 2. EXPLORE (never look back)
      path.pop();                   // 3. UN-CHOOSE
    }
  }

  backtrack(0);
  return out;
}
```

Solution — cpp

```cpp
class Solution {
    vector<vector<int>> out;
    vector<int> path;
    void backtrack(vector<int>& nums, int start) {
        out.push_back(path);
        for (int i = start; i < (int)nums.size(); ++i) {
            path.push_back(nums[i]);
            backtrack(nums, i + 1);
            path.pop_back();
        }
    }
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        backtrack(nums, 0);
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        out, path = [], []

        def backtrack(start):
            out.append(path[:])          # copy
            for i in range(start, len(nums)):
                path.append(nums[i])
                backtrack(i + 1)
                path.pop()

        backtrack(0)
        return out
```

Solution — java

```java
class Solution {
    private final List<List<Integer>> out = new ArrayList<>();
    private final List<Integer> path = new ArrayList<>();
    public List<List<Integer>> subsets(int[] nums) {
        backtrack(nums, 0);
        return out;
    }
    private void backtrack(int[] nums, int start) {
        out.add(new ArrayList<>(path));      // copy
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            backtrack(nums, i + 1);
            path.remove(path.size() - 1);
        }
    }
}
```

WALKTHROUGH

RECORDING AT EVERY NODE, NOT JUST THE LEAVES.

Unlike permutations — where only a complete arrangement counts — every partial path here is already a valid subset. So the `push` happens on entry, before the loop, and there is no separate completion check. That is why the empty array appears first: it is the path at the root.

`BACKTRACK(I + 1)`, NOT `BACKTRACK(START + 1)`.

The next level must resume after the element just chosen, which is `i`. Using `start + 1` would allow the same element to be picked again at a deeper level and produce duplicates.

THE COPY IS MANDATORY.

`path` is a single array mutated throughout; storing it by reference gives 2^n pointers to the same object, which is empty by the time the recursion unwinds. This is the most common backtracking bug and it produces spectacularly wrong output.

Permutations `Medium`

Pattern: Backtracking with a used array

Given an array of **distinct** integers, return all possible permutations. Order of the results does not matter.

Input	Output	Note
`nums = [1,2,3]`	`[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`	
`nums = [1]`	`[[1]]`	

HINTS

1. Unlike subsets, order matters and every element must be used — so no `start` index.
2. Track which indices are already in the path with a boolean array.
3. Only record when the path has full length.
4. Remember to clear the `used` flag in the un-choose step as well as popping the path.

Approach	Idea	Time	Space
Backtracking with <code>used[] </code>	Try every unused element at each position.	`O(n! ·n)`	`O(n)`
Swap in place	Swap element `i` into position, recurse, swap back. Avoids the extra array.	`O(n! ·n)`	`O(n)`
Insert into every gap	Build permutations of the first k elements, then insert the next into every position.	`O(n! ·n)`	`O(n! ·n)`

Solution — js

```
function permute(nums) {
  const out = [], path = [];
  const used = new Array(nums.length).fill(false);

  function backtrack() {
    if (path.length === nums.length) {   // COMPLETE
      out.push([...path]);
      return;
    }
    for (let i = 0; i < nums.length; i++) {
      if (used[i]) continue;            // already in the path

      used[i] = true;  path.push(nums[i]);   // CHOOSE
      backtrack();                            // EXPLORE
      path.pop();      used[i] = false;       // UN-CHOOSE (both!)
    }
  }

  backtrack();
  return out;
}
```

Solution — cpp

```
class Solution {
    vector<vector<int>> out;
    vector<int> path;
    vector<bool> used;
    void backtrack(vector<int>& nums) {
        if (path.size() == nums.size()) { out.push_back(path); return; }
        for (int i = 0; i < (int)nums.size(); ++i) {
            if (used[i]) continue;
            used[i] = true; path.push_back(nums[i]);
            backtrack(nums);
            path.pop_back(); used[i] = false;
        }
    }
public:
    vector<vector<int>> permute(vector<int>& nums) {
        used.assign(nums.size(), false);
        backtrack(nums);
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def permute(self, nums: list[int]) -> list[list[int]]:
        out, path = [], []
        used = [False] * len(nums)

        def backtrack():
            if len(path) == len(nums):
                out.append(path[:])
                return
            for i, v in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(v)
                backtrack()
                path.pop()
                used[i] = False

        backtrack()
        return out
```

Solution — java

```java
class Solution {
    private final List<List<Integer>> out = new ArrayList<>();
    private final List<Integer> path = new ArrayList<>();
    private boolean[] used;
    public List<List<Integer>> permute(int[] nums) {
        used = new boolean[nums.length];
        backtrack(nums);
        return out;
    }
    private void backtrack(int[] nums) {
        if (path.size() == nums.length) { out.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true; path.add(nums[i]);
            backtrack(nums);
            path.remove(path.size() - 1); used[i] = false;
        }
    }
}
```

WALKTHROUGH

NO `START` IS THE WHOLE DIFFERENCE FROM SUBSETS.

Every level scans the full array, because a later element may need to appear before an earlier one. What prevents reuse is the `used` array, not a positional bound — which is precisely why `[1,2]` and `[2,1]` are both produced here and only one of them in subsets.

TWO THINGS ARE CHOSEN, SO TWO THINGS MUST BE UN-CHOSEN.

The `path.pop()` and the `used[i] = false` must both happen. Forgetting the flag reset means an element is consumed permanently after its first use and only one permutation is ever produced.

RECORDING ONLY AT FULL LENGTH.

A partial arrangement is not a permutation, so unlike subsets the push is guarded by the completion check.

THE FOLLOW-UP:

with duplicate input, sort first and skip `i` when `nums[i] === nums[i-1] && !used[i-1]` — the `!used[i-1]` ensures duplicates are consumed in a fixed left-to-right order, which yields each distinct permutation exactly once.

Combination sum `Medium`

Pattern: Backtracking with reuse and pruning

Given an array of **distinct** positive integers `candidates` and a `target`, return all unique combinations that sum to the target. The **same number may be used unlimited times**, and two combinations are the same if they contain the same multiset of numbers.

Input	Output	Note
`candidates = [2,3,6,7], target = 7`	`[[2,2,3],[7]]`	
`candidates = [2,3,5], target = 8`	`[[2,2,2,2],[2,3,3],[3,5]]`	
`candidates = [2], target = 1`	`[]`	

HINTS

1. Because numbers may repeat, the recursive call passes `i` rather than `i + 1`.
2. Still use a `start` index so combinations are generated in non-decreasing order — that is what stops [2,3] and [3,2] both appearing.
3. Track the remaining target. Zero means success; negative means abandon this branch.
4. Sorting the candidates lets you `break` out of the loop as soon as a candidate exceeds the remainder.

Approach	Idea	Time	Space
Generate then filter	Enumerate everything and keep those that sum correctly. Hopeless.	`huge`	`huge`
Backtracking with a remainder	Subtract as you go; prune when it goes negative.	`exponential`	`O(target/min)`
Sort + break early	Same, but the sorted order lets you stop the loop rather than skipping.	`exponential, much faster in practice`	`O(target/min)`

Solution — js

```
function combinationSum(candidates, target) {
  const out = [], path = [];
  // sorting enables the early break below
  const sorted = candidates.slice().sort((a, b) => a - b);

  function backtrack(start, remain) {
    if (remain === 0) {
      out.push([...path]);        // COPY
      return;
    }

    for (let i = start; i < sorted.length; i++) {
      // sorted, so every later candidate is also too big
      if (sorted[i] > remain) break;          // PRUNE

      path.push(sorted[i]);                   // CHOOSE
      backtrack(i, remain - sorted[i]);       // EXPLORE - i, so
                                              // reuse is allowed
      path.pop();                             // UN-CHOOSE
    }
  }

  backtrack(0, target);
  return out;
}
```

Solution — cpp

```
class Solution {
    vector<vector<int>> out;
    vector<int> path;
    void backtrack(vector<int>& c, int start, int remain) {
        if (remain == 0) { out.push_back(path); return; }
        for (int i = start; i < (int)c.size(); ++i) {
            if (c[i] > remain) break;
            path.push_back(c[i]);
            backtrack(c, i, remain - c[i]);
            path.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end());
        backtrack(candidates, 0, target);
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def combinationSum(self, candidates: list[int], target: int) -> list[list[int]]:
        out, path = [], []
        c = sorted(candidates)

        def backtrack(start, remain):
            if remain == 0:
                out.append(path[:])
                return
            for i in range(start, len(c)):
                if c[i] > remain:
                    break
                path.append(c[i])
                backtrack(i, remain - c[i])   # i: reuse allowed
                path.pop()

        backtrack(0, target)
        return out
```

Solution — java

```java
class Solution {
    private final List<List<Integer>> out = new ArrayList<>();
    private final List<Integer> path = new ArrayList<>();
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        int[] c = candidates.clone();
        Arrays.sort(c);
        backtrack(c, 0, target);
        return out;
    }
    private void backtrack(int[] c, int start, int remain) {
        if (remain == 0) { out.add(new ArrayList<>(path)); return; }
        for (int i = start; i < c.length; i++) {
            if (c[i] > remain) break;
            path.add(c[i]);
            backtrack(c, i, remain - c[i]);
            path.remove(path.size() - 1);
        }
    }
}
```

WALKTHROUGH

`BACKTRACK(I, …)` IS WHAT PERMITS REUSE.

Subsets pass `i + 1` to move past the chosen element; here passing `i` lets the same candidate be chosen again at the next level, giving `[2,2,3]`. Keeping `start` at all is still necessary — it forces non-decreasing order, so `[2,3]` is generated but `[3,2]` never is, which is exactly the "no duplicate combinations" requirement.

SORTING TURNS A `CONTINUE` INTO A `BREAK`.

Once `sorted[i] > remain`, every subsequent candidate is larger still and equally hopeless, so the entire rest of the loop can be abandoned. That single change prunes a large fraction of the tree and is the difference between comfortable and slow on bigger inputs. Note the test `[7,3,2]` is deliberately unsorted to confirm the sort actually happens.

TRACKING THE REMAINDER RATHER THAN A RUNNING SUM

makes the base case a clean `remain === 0` and the prune a clean comparison, with no need to also carry the target around.

N-Queens `Hard`

Pattern: Backtracking with constraint sets

Place `n` queens on an `n × n` board so that no two attack each other (no shared row, column or diagonal). Return the **number** of distinct solutions.

Input	Output	Note
`n = 4`	`2`	
`n = 1`	`1`	
`n = 8`	`92`	

HINTS

1. Place one queen per row, so rows are automatically distinct — the choice at each level is just the column.
2. Track occupied columns in a set. For the diagonals, notice that `r - c` is constant along one diagonal and `r + c` along the other.
3. Use `r - c + n` as the index if you want array indices rather than a set, to keep it non-negative.
4. Choose, recurse, then remove all three markers on the way out.

Approach	Idea	Time	Space
Try every placement	Choose n cells from n^2 and validate. Astronomically wasteful.	`C(n², n)`	`O(n)`
Row by row with a validity scan	One queen per row; check each candidate against all placed queens. O(n) per check.	`O(n!·n)`	`O(n)`
Row by row with O(1) constraint sets	Column and both diagonal keys in sets, so validity is a constant-time test.	`O(n!)`	`O(n)`

Solution — js

```
function totalNQueens(n) {
  const cols  = new Set();   // occupied columns
  const diag1 = new Set();   // r - c  (constant on one diagonal)
  const diag2 = new Set();   // r + c  (constant on the other)
  let count = 0;

  function backtrack(row) {
    if (row === n) { count++; return; }   // all rows filled

    for (let c = 0; c < n; c++) {
      const d1 = row - c, d2 = row + c;
      if (cols.has(c) || diag1.has(d1) || diag2.has(d2)) continue;

      cols.add(c); diag1.add(d1); diag2.add(d2);      // CHOOSE
      backtrack(row + 1);                             // EXPLORE
      cols.delete(c); diag1.delete(d1); diag2.delete(d2); // UNDO
    }
  }

  backtrack(0);
  return count;
}
```

Solution — cpp

```
class Solution {
    int n, count = 0;
    vector<bool> cols, d1, d2;
    void backtrack(int row) {
        if (row == n) { ++count; return; }
        for (int c = 0; c < n; ++c) {
            int i1 = row - c + n, i2 = row + c;
            if (cols[c] || d1[i1] || d2[i2]) continue;
            cols[c] = d1[i1] = d2[i2] = true;
            backtrack(row + 1);
            cols[c] = d1[i1] = d2[i2] = false;
        }
    }
public:
    int totalNQueens(int nn) {
        n = nn; count = 0;
        cols.assign(n, false);
        d1.assign(2 * n, false);
        d2.assign(2 * n, false);
        backtrack(0);
        return count;
    }
};
```

Solution — python

```
class Solution:
    def totalNQueens(self, n: int) -> int:
        cols, diag1, diag2 = set(), set(), set()
        self.count = 0

        def backtrack(row):
            if row == n:
                self.count += 1
                return
            for c in range(n):
                d1, d2 = row - c, row + c
                if c in cols or d1 in diag1 or d2 in diag2:
                    continue
                cols.add(c); diag1.add(d1); diag2.add(d2)
                backtrack(row + 1)
                cols.remove(c); diag1.remove(d1); diag2.remove(d2)

        backtrack(0)
        return self.count
```

Solution — java

```
class Solution {
    private int n, count;
    private boolean[] cols, d1, d2;
    public int totalNQueens(int n) {
        this.n = n; this.count = 0;
        cols = new boolean[n];
        d1 = new boolean[2 * n];
        d2 = new boolean[2 * n];
        backtrack(0);
        return count;
    }
    private void backtrack(int row) {
        if (row == n) { count++; return; }
        for (int c = 0; c < n; c++) {
            int i1 = row - c + n, i2 = row + c;
            if (cols[c] || d1[i1] || d2[i2]) continue;
            cols[c] = d1[i1] = d2[i2] = true;
            backtrack(row + 1);
            cols[c] = d1[i1] = d2[i2] = false;
        }
    }
}
```

WALKTHROUGH

ONE QUEEN PER ROW REMOVES AN ENTIRE DIMENSION.

Since two queens can never share a row, fixing "row `r` gets exactly one queen" means the only decision at depth `r` is which column — reducing the search from choosing cells to choosing a permutation of columns.

THE DIAGONAL KEYS ARE THE NEAT PART.

Along a top-left-to-bottom-right diagonal, moving one step down and one right increases both `r` and `c` by 1, so `r - c` is invariant. Along the other diagonal `r` increases as `c` decreases, so `r + c` is invariant. Two integers therefore identify the two diagonals through any cell, making the attack test three O(1) set lookups instead of an O(n) scan over placed queens.

THREE MARKERS SET, THREE MARKERS CLEARED.

The undo must mirror the choose exactly; leaving one diagonal marked poisons every sibling branch and silently undercounts. Testing `n = 8` against the known answer of 92 catches that immediately, which is why it is in the list.

`N = 2` AND `N = 3` RETURN 0

— genuinely unsolvable, and a good check that the code does not assume a solution exists. In C++ and Java the diagonal index is shifted by `+n` to keep it non-negative for array indexing; a set-based language does not need that.

DP: the method, memoisation and tabulation

Recursion that stops recomputing. Dynamic programming has a reputation for difficulty because people try to leap straight to the table — the reliable route is to write the recursion first and then cache it.

The method

```
DP APPLIES WHEN BOTH OF THESE HOLD:

  OPTIMAL SUBSTRUCTURE   the answer is built from answers
                         to smaller subproblems
  OVERLAPPING SUBPROBLEMS the same subproblem is reached
                         many times

The second is what distinguishes DP from plain divide and
conquer. Merge sort has optimal substructure but its
halves never overlap, so caching would gain nothing.

------------------------------------------------------
THE FOUR-STEP RECIPE. FOLLOW IT IN ORDER.

  1. WRITE THE BRUTE-FORCE RECURSION.
     What is the smallest decision? What does it leave?
     Do not think about efficiency yet.

  2. IDENTIFY THE STATE.
     Which parameters actually change between calls?
     Those, and only those, are the memo key.

  3. ADD MEMOISATION.
     One dictionary or array. This alone usually takes
     the solution from exponential to polynomial.

  4. (optional) CONVERT TO TABULATION.
     Fill the table in dependency order with loops.
     Removes the recursion depth risk and often allows
     space to be reduced.

  MOST INTERVIEWERS ACCEPT STEP 3. Do not skip 1 and 2 to
  get there faster - writing the recursion first is what
  makes the state obvious.

------------------------------------------------------
TOP-DOWN (memoised recursion)

  const memo = new Map();
  function f(state) {
      if (isBase(state)) return baseValue;
      if (memo.has(state)) return memo.get(state);

      const result = combine(f(smaller1), f(smaller2));
      memo.set(state, result);
      return result;
  }

  + follows the natural recursion
  + only computes states actually REACHED
  - recursion depth, and hashing overhead

BOTTOM-UP (tabulation)

  const dp = new Array(n + 1);
  dp[0] = base;
  for (let i = 1; i <= n; i++)
      dp[i] = combine(dp[i - 1], dp[i - 2]);
  return dp[n];

  + no recursion, usually faster constants
```

```
  + SPACE OPTIMISATION becomes visible
  - you must work out the fill order yourself
  - computes every state, even unreachable ones

------------------------------------------------------------
SPACE OPTIMISATION — the standard follow-up

  If dp[i] depends only on dp[i-1] and dp[i-2], you never
  need the whole array:

    let prev2 = base0, prev1 = base1;
    for (let i = 2; i <= n; i++) {
        const cur = f(prev1, prev2);
        prev2 = prev1;
        prev1 = cur;
    }

  O(n) space becomes O(1). The same idea reduces a 2-D
  table to two rows, or sometimes one (chapter 34).

------------------------------------------------------------
HOW TO SPOT A DP PROBLEM

  "how many ways ..."          COUNT
  "minimum / maximum cost"     OPTIMISE
  "is it possible to ..."      DECIDE
  "longest / shortest ..."     OPTIMISE

  plus: choices at each step, and the same situation
  reachable by several routes.

  IF THE PROBLEM ASKS FOR THE ACTUAL LIST OF SOLUTIONS,
  it is backtracking, not DP (chapter 32). DP gives you
  a count or an optimum, not an enumeration.
```

Problems

How it shows up

Symptom	Cause
Exponentially slow	Memoisation missing, or the key does not match the real state
Memo never hits	Key includes a parameter that always differs — narrow the state
Wrong answer, right shape	Base case wrong or off by one
Stack overflow	Deep top-down recursion — convert to tabulation
Table reads uninitialised entries	Fill order does not respect dependencies
Space-optimised version wrong	Overwrote a value still needed — check the iteration direction
Cached results leak between calls	Memo declared outside the function and not reset

Check yourself

- ✓ State the two conditions for DP and say which one excludes merge sort.
- ✓ Give the four-step recipe in order.
- ✓ Write the memoisation template and the tabulation template.

- ✓ Give one advantage and one drawback of each direction.
- ✓ Explain how to reduce O(n) space to O(1) and when it is possible.
- ✓ Name the four question shapes that signal DP.
- ✓ Say when a problem is backtracking rather than DP.

Problems

Climbing stairs Easy

Pattern: One-dimensional DP

You are climbing a staircase of `n` steps. Each move you may climb 1 or 2 steps. In how many distinct ways can you reach the top?

Input	Output	Note
`n = 2`	`2`	1+1, or 2
`n = 3`	`3`	1+1+1, 1+2, 2+1
`n = 5`	`8`	

HINTS

1. To arrive at step `n` your last move came from step `n-1` or step `n-2`.
2. So `ways(n) = ways(n-1) + ways(n-2)` — the Fibonacci recurrence.
3. Base cases: one way to stand at step 0 (do nothing), one way to reach step 1.
4. Only the last two values are ever needed, so O(1) space is achievable.

Approach	Idea	Time	Space
Plain recursion	Recompute both branches every time. The call tree is a Fibonacci tree.	`O(2ⁿ)`	`O(n)`
Memoised recursion	Cache each `n`. Linear, but O(n) stack.	`O(n)`	`O(n)`
Tabulation, two variables	Roll forward keeping only the previous two.	`O(n)`	`O(1)`

Solution — js

```
function climbStairs(n) {
  // one way to be at step 0 (stand still), one way to reach step 1
  let prev2 = 1, prev1 = 1;

  for (let i = 2; i <= n; i++) {
    const cur = prev1 + prev2;   // arrive from i-1 or i-2
    prev2 = prev1;
    prev1 = cur;
  }
  return prev1;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int climbStairs(int n) {
        long prev2 = 1, prev1 = 1;
        for (int i = 2; i <= n; ++i) {
            long cur = prev1 + prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        return (int)prev1;
    }
};
```

Solution — python

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        prev2 = prev1 = 1
        for _ in range(2, n + 1):
            prev2, prev1 = prev1, prev1 + prev2
        return prev1
```

Solution — java

```java
class Solution {
    public int climbStairs(int n) {
        int prev2 = 1, prev1 = 1;
        for (int i = 2; i <= n; i++) {
            int cur = prev1 + prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

WALKTHROUGH

DERIVING THE RECURRENCE BY LOOKING BACKWARDS.

Ask what the *final* move must have been: either a single step from `n-1` or a double step from `n-2`. Those two cases are disjoint and cover everything, so the counts simply add. Looking at the last decision rather than the first is the habit that makes most DP recurrences fall out.

WHY `N = 0` IS 1, NOT 0.

There is exactly one way to be at the bottom: do nothing. That convention is what makes `n = 2` come out as 2 rather than 1. Base cases in counting problems are almost always "one empty way", not "no ways".

THE SPACE OPTIMISATION IS VISIBLE ONLY AFTER TABULATION.

Once written as `dp[i] = dp[i-1] + dp[i-2]`, it is obvious that entries older than two steps are never read — so two scalars suffice. That recognition is exactly the follow-up interviewers ask for, and it generalises to every DP whose dependencies reach back a fixed distance.

House robber `Medium`

Pattern: Take-or-skip DP

Each house holds some money, but robbing two **adjacent** houses triggers the alarm. Given an array `nums`, return the maximum you can rob.

Input	Output	Note
`nums = [1,2,3,1]`	`4`	Houses 0 and 2.
`nums = [2,7,9,3,1]`	`12`	Houses 0, 2 and 4.

HINTS

1. At each house you have exactly two options: rob it (and skip the previous) or skip it.
2. So `best(i) = max(nums[i] + best(i-2), best(i-1))`.
3. Greedily taking the largest houses does not work — try `[2,1,1,2]`.
4. Only the previous two answers are needed, so O(1) space again.

Approach	Idea	Time	Space
Try every valid subset	Enumerate non-adjacent subsets. Exponential.	`O(2ⁿ)`	`O(n)`
Greedy on largest values	**Wrong.** `[2,1,1,2]` defeats it — the answer is 4, not 3.	`—`	`—`
DP, two rolling variables	Take-or-skip at each index.	`O(n)`	`O(1)`

Solution — js

```
function rob(nums) {
  // best if we stop before the previous house / at the previous house
  let prev2 = 0, prev1 = 0;

  for (const v of nums) {
    // rob this one (adding prev2) or skip it (keeping prev1)
    const cur = Math.max(v + prev2, prev1);
    prev2 = prev1;
    prev1 = cur;
  }
  return prev1;
}
```

Solution — cpp

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int prev2 = 0, prev1 = 0;
        for (int v : nums) {
            int cur = max(v + prev2, prev1);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
};
```

Solution — python

```python
class Solution:
    def rob(self, nums: list[int]) -> int:
        prev2 = prev1 = 0
        for v in nums:
            prev2, prev1 = prev1, max(v + prev2, prev1)
        return prev1
```

Solution — java

```java
class Solution {
    public int rob(int[] nums) {
        int prev2 = 0, prev1 = 0;
        for (int v : nums) {
            int cur = Math.max(v + prev2, prev1);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

WALKTHROUGH

WHY GREEDY FAILS, CONCRETELY.

On `[2,1,1,2]` a greedy rule that grabs the largest available house takes a 2, then must skip its neighbour, and ends at 3. The DP takes both 2s — houses 0 and 3 are not adjacent — for 4. Being able to produce a counterexample on demand is worth more than asserting "greedy does not work".

THE TWO VARIABLES HAVE PRECISE MEANINGS.

`prev1` is the best total considering all houses up to and including the previous one; `prev2` is the best up to the one before that. Robbing the current house is only legal on top of `prev2`, which is exactly what enforces non-adjacency.

BOTH START AT 0

, which handles the empty array and the single-house case without special-casing: with one house the loop runs once and returns `max(v, 0)`.

THE VARIANTS WORTH KNOWING:

"House Robber II" puts the houses in a circle, solved by running this twice — once excluding the first house, once excluding the last — and taking the better. "House Robber III" puts them in a tree, solved with the tree-recursion pattern from chapter 24 returning a (robbed, skipped) pair.

Coin change — fewest coins `Medium`

Pattern: Unbounded choice DP

Given coin denominations `coins` and an `amount`, return the fewest coins needed to make that amount. You have an unlimited supply of each coin. Return `-1` if it cannot be made.

Input	Output	Note
`coins = [1,2,5], amount = 11`	`3`	5 + 5 + 1
`coins = [2], amount = 3`	`-1`	

Input	Output	Note
`coins = [1], amount = 0`	`0`	

HINTS

1. Greedily taking the largest coin fails — try `coins = [1,3,4]`, `amount = 6`.
2. For each amount `a`, the answer is `1 + min(best(a − c))` over every coin `c` that fits.
3. Fill a table from 0 up to the target so every dependency is already computed.
4. Use `Infinity` for "not makeable" and convert it to −1 at the end.

Approach	Idea	Time	Space
Greedy largest first	**Wrong.** With `[1,3,4]` and amount 6, greedy gives 4+1+1 = 3 coins; the optimum is 3+3 = 2.	`—`	`—`
Backtracking over combinations	Try every combination and keep the shortest. Exponential.	`exponential`	`O(amount)`
Bottom-up DP	One table entry per amount, each computed from smaller amounts.	`O(amount · coins)`	`O(amount)`

Solution — js

```
function coinChange(coins, amount) {
  // dp[a] = fewest coins to make amount a
  const dp = new Array(amount + 1).fill(Infinity);
  dp[0] = 0;                        // zero coins make zero

  for (let a = 1; a <= amount; a++) {
    for (const c of coins) {
      if (c > a) continue;                     // does not fit
      if (dp[a - c] === Infinity) continue;    // unreachable
      dp[a] = Math.min(dp[a], dp[a - c] + 1);
    }
  }

  return dp[amount] === Infinity ? -1 : dp[amount];
}
```

Solution — cpp

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        const int INF = 1e9;
        vector<int> dp(amount + 1, INF);
        dp[0] = 0;
        for (int a = 1; a <= amount; ++a)
            for (int c : coins) {
                if (c > a || dp[a - c] == INF) continue;
                dp[a] = min(dp[a], dp[a - c] + 1);
            }
        return dp[amount] == INF ? -1 : dp[amount];
    }
};
```

Solution — python

```python
class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        INF = float("inf")
        dp = [INF] * (amount + 1)
        dp[0] = 0
        for a in range(1, amount + 1):
            for c in coins:
                if c <= a and dp[a - c] != INF:
                    dp[a] = min(dp[a], dp[a - c] + 1)
        return -1 if dp[amount] == INF else dp[amount]
```

Solution — java

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        final int INF = 1_000_000_000;
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, INF);
        dp[0] = 0;
        for (int a = 1; a <= amount; a++)
            for (int c : coins) {
                if (c > a || dp[a - c] == INF) continue;
                dp[a] = Math.min(dp[a], dp[a - c] + 1);
            }
        return dp[amount] == INF ? -1 : dp[amount];
    }
}
```

WALKTHROUGH

THE GREEDY COUNTEREXAMPLE IS THE POINT OF THIS PROBLEM.

With `[1,3,4]` and amount 6, taking the largest coin first gives 4, then 1, then 1 — three coins. The optimum is 3 + 3, two coins. Greedy happens to work for real-world currency systems because they are designed to be "canonical", but it is not valid in general, and the interviewer will supply a counterexample if you do not.

FILLING UPWARDS GUARANTEES THE DEPENDENCIES.

`dp[a]` reads only `dp[a - c]` for positive `c`, so every value it needs has a smaller index and is already final. That is the fill-order reasoning tabulation always requires.

`DP[0] = 0` IS THE BASE CASE AND IT IS DOING REAL WORK

— it is what lets a single coin equal to the amount produce an answer of 1.

THE UNREACHABLE GUARD.

Skipping when `dp[a - c]` is `Infinity` keeps the arithmetic clean; in C++ or Java adding 1 to a sentinel like `INT_MAX` would overflow to a negative number and silently corrupt the minimum. Using `1e9` instead of `INT_MAX` is the usual defence.

NOTE THE CONTRAST WITH THE COUNTING VARIANT.

"In how many ways can you make the amount?" is the same table with `+=` instead of `min` — and it needs the loop order reversed to avoid counting permutations. That is chapter 34.

DP: knapsack and subset problems

A family of problems all reducible to one question: which items do I take, subject to a capacity? The loop order tells you whether each item may be reused, and that single detail decides correctness.

The pattern

```
THE 0/1 KNAPSACK: n items with weights and values, a
capacity W, each item taken AT MOST ONCE. Maximise value.

  dp[i][w] = best value using the first i items with
             capacity w

  dp[i][w] = max( dp[i-1][w],                    // skip
                  dp[i-1][w - wt[i]] + val[i] )  // take

THE SHAPE - "for each item, take it or skip it" - is the
whole family.

-----------------------------------------------------------
THE SPACE OPTIMISATION AND THE LOOP-ORDER RULE

Each row depends only on the previous row, so one array
suffices. But the ITERATION DIRECTION now carries meaning:

  0/1 KNAPSACK (each item once):
     for (item of items)
         for (w = W; w >= wt; w--)     // BACKWARDS
             dp[w] = max(dp[w], dp[w - wt] + val);

  UNBOUNDED (unlimited copies):
     for (item of items)
         for (w = wt; w <= W; w++)     // FORWARDS
             dp[w] = max(dp[w], dp[w - wt] + val);

WHY. Going backwards, dp[w - wt] still holds the PREVIOUS
row's value - the item has not been used yet at that
capacity - so it can be taken at most once. Going
forwards, dp[w - wt] has ALREADY been updated with this
item, so taking it again is allowed.

ONE CHARACTER SEPARATES 0/1 FROM UNBOUNDED. If a knapsack
answer is wrong and the structure looks right, check the
direction first.

-----------------------------------------------------------
THE OTHER LOOP-ORDER RULE: COMBINATIONS VS PERMUTATIONS

For COUNTING problems the nesting order matters too:

  items OUTSIDE, capacity inside   -> COMBINATIONS
                                      (1+2 and 2+1 count
                                       once)
  capacity OUTSIDE, items inside   -> PERMUTATIONS
                                      (counted separately)

"Coin change 2" wants combinations; "combination sum IV"
(badly named) wants permutations. Same table, swapped
loops.

-----------------------------------------------------------
THE FAMILY, all the same table

  subset sum          "can I hit exactly T?"   boolean dp
  partition equal
    subset sum        subset sum with T = total/2
  target sum          assign +/- to each number
  coin change         unbounded, minimise count
```

```
coin change 2       unbounded, count combinations
last stone weight II minimise |difference| = subset sum
                     near total/2
ones and zeroes      knapsack with TWO capacities
```

Problems

How it shows up

Symptom	Cause
Items used more than once in 0/1	Inner loop runs forwards — must go backwards
Unbounded version misses reuse	Inner loop runs backwards — must go forwards
Count too high	Loops nested the wrong way — counting permutations, not combinations
Odd total reported as partitionable	Missing the parity check before halving
Off by one in the capacity array	Table needs `W + 1` entries, indices 0..W
Base case wrong	`dp[0]` must be `true`/0 — the empty selection
Timeout on large capacity	Pseudo-polynomial: O(n·W) grows with the *value* of W

Check yourself

- ✓ Write the 0/1 knapsack recurrence and say what each branch means.
- ✓ Give the loop direction for 0/1 versus unbounded, and explain why.
- ✓ Explain how the nesting order distinguishes combinations from permutations.
- ✓ Reduce "partition into two equal subsets" to subset sum.
- ✓ Say why the base case is the empty selection.
- ✓ Explain what "pseudo-polynomial" means and why it matters.
- ✓ Name four problems in this family.

Problems

Partition equal subset sum `Medium`

Pattern: 0/1 subset sum

Given an array of positive integers, determine whether it can be split into two subsets with **equal sums**.

Input	Output	Note
`nums = [1,5,11,5]`	`true`	[1,5,5] and [11]
`nums = [1,2,3,5]`	`false`	

HINTS

1. Two equal halves means each sums to `total / 2` — so if the total is odd, stop immediately.
2. The question becomes: is there a subset summing to exactly `total / 2`?
3. That is 0/1 subset sum: a boolean table over achievable sums.
4. Iterate the capacity
   BACKWARDS
   so each number is used at most once.

Approach	Idea	Time	Space
Try every subset	Enumerate all 2^n subsets and check the sums.	`O(2ⁿ)`	`O(n)`
2-D boolean table	`dp[i][s]` = can the first `i` items reach sum `s`?	`O(n·S)`	`O(n·S)`
1-D backwards	One boolean array over sums, filled right to left per item.	`O(n·S)`	`O(S)`

Solution — js

```
function canPartition(nums) {
  const total = nums.reduce((a, b) => a + b, 0);
  if (total % 2 !== 0) return false;     // cannot halve an odd total

  const target = total / 2;
  const dp = new Array(target + 1).fill(false);
  dp[0] = true;                          // empty subset sums to 0

  for (const v of nums) {
    // BACKWARDS: dp[s - v] must still be the PREVIOUS row,
    // so v is used at most once
    for (let s = target; s >= v; s--) {
      if (dp[s - v]) dp[s] = true;
    }
  }
  return dp[target];
}
```

Solution — cpp

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int total = accumulate(nums.begin(), nums.end(), 0);
        if (total % 2) return false;
        int target = total / 2;
        vector<char> dp(target + 1, false);
        dp[0] = true;
        for (int v : nums)
            for (int s = target; s >= v; --s)
                if (dp[s - v]) dp[s] = true;
        return dp[target];
    }
};
```

Solution — python

```python
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        total = sum(nums)
        if total % 2:
            return False
        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True
        for v in nums:
            for s in range(target, v - 1, -1):   # backwards
                if dp[s - v]:
                    dp[s] = True
        return dp[target]
```

Solution — java

```java
class Solution {
    public boolean canPartition(int[] nums) {
        int total = 0;
        for (int v : nums) total += v;
        if (total % 2 != 0) return false;
        int target = total / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int v : nums)
            for (int s = target; s >= v; s--)
                if (dp[s - v]) dp[s] = true;
        return dp[target];
    }
}
```

WALKTHROUGH

THE REDUCTION IS THE INSIGHT.

"Split into two equal halves" sounds like a partitioning problem, but the second half is determined by the first — so it is enough to find *one* subset summing to `total / 2`. That turns an unfamiliar question into textbook subset sum.

THE PARITY CHECK IS NOT AN OPTIMISATION, IT IS CORRECTNESS.

An odd total cannot be halved into integers at all, and without the check `target` would be fractional and the array size nonsensical. `[1,2,5]` has total 8 and is still false, so parity alone is not sufficient — but it is necessary.

BACKWARDS IS WHAT MAKES IT 0/1.

Reading `dp[s - v]` at a *lower* index that has not yet been touched in this pass means it still reflects the state before `v` was available. Iterating forwards would let `dp[s - v]` already include `v`, effectively allowing the same number to be used twice — and `[1]` would wrongly report `true` for target 0.5-style reasoning, or more visibly `[1,1]` vs `[2]` cases would diverge. Try flipping the direction in the editor and watch which tests break.

Coin change 2 — count the combinations `Medium`

Pattern: Unbounded knapsack, counting combinations

Given coin denominations and an `amount`, return the **number of distinct combinations** that make up that amount. You have unlimited coins of each kind, and combinations differing only in order count as **one**.

Input	Output	Note
`amount = 5, coins = [1,2,5]`	`4`	5; 2+2+1; 2+1+1+1; 1×5
`amount = 3, coins = [2]`	`0`	
`amount = 10, coins = [10]`	`1`	

HINTS

1. This is a counting problem, so the table accumulates with `+=` rather than `min`.
2. Coins may repeat, so the capacity loop runs **FORWARDS** .
3. To count combinations rather than permutations, put the **COIN LOOP OUTSIDE** and the amount loop inside.
4. Swapping the loops counts 1+2 and 2+1 separately — which is a different problem.

Approach	Idea	Time	Space
Backtracking	Enumerate every combination and count. Exponential.	`exponential`	`O(amount)`
2-D table over (coin, amount)	Explicit rows per coin. Clear, more memory.	`O(n·amount)`	`O(n·amount)`
1-D, coins outside	One array; the outer coin loop enforces combination counting.	`O(n·amount)`	`O(amount)`

Solution — js

```
function change(amount, coins) {
  const dp = new Array(amount + 1).fill(0);
  dp[0] = 1;                 // exactly one way to make 0: take nothing

  // COIN LOOP OUTSIDE -> each combination is counted once,
  // because coins are always considered in a fixed order
  for (const c of coins) {
    // FORWARDS -> the same coin may be reused
    for (let a = c; a <= amount; a++) {
      dp[a] += dp[a - c];
    }
  }
  return dp[amount];
}
```

Solution — cpp

```
class Solution {
public:
    int change(int amount, vector<int>& coins) {
        vector<unsigned long long> dp(amount + 1, 0);
        dp[0] = 1;
        for (int c : coins)
            for (int a = c; a <= amount; ++a)
                dp[a] += dp[a - c];
        return (int)dp[amount];
    }
};
```

Solution — python

```
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [0] * (amount + 1)
        dp[0] = 1
        for c in coins:                      # coins outside
            for a in range(c, amount + 1):    # forwards
                dp[a] += dp[a - c]
        return dp[amount]
```

Solution — java

```
class Solution {
    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int c : coins)
            for (int a = c; a <= amount; a++)
                dp[a] += dp[a - c];
        return dp[amount];
    }
}
```

WALKTHROUGH

TWO LOOP DECISIONS, TWO DIFFERENT MEANINGS — THIS PROBLEM IS THE CLEAREST PLACE TO LEARN THEM.

FORWARDS VERSUS BACKWARDS

controls reuse. Reading `dp[a - c]` after it has already been updated with coin `c` in this pass means `c` can be used again, which is exactly what "unlimited supply" requires. Backwards would make each coin single-use.

COINS OUTSIDE VERSUS AMOUNT OUTSIDE

controls what counts as distinct. With coins outside, every combination is built in a fixed coin order — all the 1s are considered before any 2s — so `1+2` and `2+1` are the same path through the table and are counted once. Putting the amount loop outside would let each amount consider every coin as the "last" one added, counting orderings separately. Swap the loops on the `amount = 5, coins = [1,2,5]` test: you get 9 instead of 4.

`DP[0] = 1` IS THE SEED FOR THE WHOLE COUNT.

Every combination ultimately bottoms out at "make zero from nothing", and that must count as one way, not zero — otherwise the entire table stays at 0.

THE LARGE TEST EXISTS TO CHECK OVERFLOW.

35 million fits in 32 bits here, but in C++ intermediate sums can exceed it, hence the wider accumulator.

Target sum `Medium`

Pattern: Sign assignment reduced to subset sum

Given an array of non-negative integers `nums` and an integer `target`, assign either `+` or `-` to each number so the resulting expression equals the target. Return the number of ways to do so.

Input	Output	Note
`nums = [1,1,1,1,1], target = 3`	`5`	
`nums = [1], target = 1`	`1`	
`nums = [1], target = 2`	`0`	

HINTS

1. Let `P` be the numbers given a plus and `N` those given a minus. Then `P - N = target` and `P + N = total`.
2. Adding those: `2P = target + total`, so `P = (target + total) / 2`.
3. So the question becomes: how many subsets sum to `P`? That is counting subset sum.
4. Reject impossible cases first: if `target + total` is odd or negative, the answer is 0.

Approach	Idea	Time	Space
Try all 2^n sign assignments	Backtracking over the signs.	`O(2^n)`	`O(n)`
Memoised (index, runningSum)	Cache on the pair. Works, but the state is larger than needed.	`O(n·S)`	`O(n·S)`

Approach	Idea	Time	Space
Reduce to counting subset sum	Algebra turns it into a 1-D 0/1 knapsack count.	`O(n·P)`	`O(P)`

Solution — js

```
function findTargetSumWays(nums, target) {
  const total = nums.reduce((a, b) => a + b, 0);

  // P − N = target,  P + N = total  =>  P = (target + total) / 2
  const twiceP = target + total;
  if (twiceP < 0 || twiceP % 2 !== 0) return 0;   // impossible
  const P = twiceP / 2;
  if (P > total) return 0;

  // count subsets summing to P − 0/1, so iterate BACKWARDS
  const dp = new Array(P + 1).fill(0);
  dp[0] = 1;                        // one way: the empty subset

  for (const v of nums) {
    for (let s = P; s >= v; s--) {
      dp[s] += dp[s − v];
    }
  }
  return dp[P];
}
```

Solution — cpp

```
class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int total = accumulate(nums.begin(), nums.end(), 0);
        int twiceP = target + total;
        if (twiceP < 0 || twiceP % 2) return 0;
        int P = twiceP / 2;
        if (P > total) return 0;
        vector<int> dp(P + 1, 0);
        dp[0] = 1;
        for (int v : nums)
            for (int s = P; s >= v; --s)
                dp[s] += dp[s − v];
        return dp[P];
    }
};
```

Solution — python

```python
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        total = sum(nums)
        twice_p = target + total
        if twice_p < 0 or twice_p % 2:
            return 0
        P = twice_p // 2
        if P > total:
            return 0
        dp = [0] * (P + 1)
        dp[0] = 1
        for v in nums:
            for s in range(P, v - 1, -1):
                dp[s] += dp[s - v]
        return dp[P]
```

Solution — java

```java
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int total = 0;
        for (int v : nums) total += v;
        int twiceP = target + total;
        if (twiceP < 0 || twiceP % 2 != 0) return 0;
        int P = twiceP / 2;
        if (P > total) return 0;
        int[] dp = new int[P + 1];
        dp[0] = 1;
        for (int v : nums)
            for (int s = P; s >= v; s--)
                dp[s] += dp[s - v];
        return dp[P];
    }
}
```

WALKTHROUGH

THE ALGEBRA IS WORTH DOING ON THE WHITEBOARD.

Splitting the numbers into a plus-set `P` and a minus-set `N` gives two equations — `P - N = target` and `P + N = total` — and adding them eliminates `N` entirely, leaving `P = (target + total) / 2`. A problem about signs becomes a problem about choosing a subset, which you already know how to count.

THE REJECTION TESTS MATTER.

If `target + total` is odd there is no integer subset sum, and if it is negative (or `P` exceeds `total`) the target is unreachable. Both return 0 immediately. `[1], target = 2` exercises the first path.

ZEROS DOUBLE THE COUNT, AND THE TABLE HANDLES IT FOR FREE.

A zero can be signed either way without changing the sum, so `[0,0,…,0,1]` with target 1 has $2^8 = 256$ answers. In the subset formulation, `v = 0` makes the inner loop run with `s - v === s`, doubling `dp[s]` exactly as required. That test is there because naive backtracking-with-pruning solutions often get it wrong.

BACKWARDS AGAIN

, because each number is signed exactly once — this is 0/1, not unbounded.

DP: string problems

Two strings, one table. Nearly every two-string problem is the same grid with a different combine rule, and recognising that turns edit distance, LCS and pattern matching into one technique.

The pattern

```
THE STATE IS ALWAYS A PAIR OF PREFIX LENGTHS:

  dp[i][j] = answer for the first i characters of s
             and the first j characters of t

The table is (m+1) x (n+1) — the extra row and column are
the EMPTY prefixes, and they are where the base cases live.

THE RECURRENCE ALWAYS SPLITS ON ONE QUESTION:
DO THE LAST TWO CHARACTERS MATCH?

  if (s[i-1] === t[j-1])
      dp[i][j] = f(dp[i-1][j-1])          // consume both
  else
      dp[i][j] = g(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

NOTE THE INDEX SHIFT: dp[i] refers to the first i
characters, so the LAST one is s[i-1]. Mixing this up is
the single most common bug in this chapter.

------------------------------------------------------------
THE THREE NEIGHBOURS AND WHAT THEY MEAN

     dp[i-1][j-1]   dp[i-1][j]
     dp[i][j-1]     dp[i][j]

  dp[i-1][j-1]  both consumed  -> match / replace
  dp[i-1][j]    only s consumed -> delete from s
  dp[i][j-1]    only t consumed -> insert into s

EDIT DISTANCE IS EXACTLY THAT READING:
  match   -> dp[i-1][j-1]
  replace -> dp[i-1][j-1] + 1
  delete  -> dp[i-1][j]   + 1
  insert  -> dp[i][j-1]   + 1

------------------------------------------------------------
THE FAMILY

  LCS                     longest common subsequence
  edit distance           insert / delete / replace
  is subsequence          greedy two-pointer, no DP needed
  longest common substring  must be CONTIGUOUS: reset to
                          0 on a mismatch, and the answer
                          is the table MAXIMUM, not the
                          corner
  distinct subsequences   count instead of maximise
  regex / wildcard match  extra cases for * and ?
  shortest common
    supersequence         m + n - LCS

------------------------------------------------------------
SUBSEQUENCE VERSUS SUBSTRING — the distinction that
decides the recurrence:

  SUBSEQUENCE  characters need not be adjacent; on a
               mismatch you carry the best so far forward
  SUBSTRING    must be contiguous; on a mismatch the run
               is BROKEN and the entry resets to 0

------------------------------------------------------------
```

```
    SPACE: each row depends only on the row above, so two rows
    suffice — O(min(m, n)) after swapping so the shorter
    string indexes the columns.
```

Problems

How it shows up

Symptom	Cause
Off by one everywhere	`dp[i]` means *i characters*, so the last is `s[i-1]`
Empty-string cases wrong	Row 0 and column 0 not initialised
Substring answer too long	Did not reset to 0 on a mismatch
Substring answer read from the corner	For substrings the answer is the table maximum
Edit distance one too high	Counted a replace when the characters already matched
Rolling-array version wrong	Overwrote `dp[i-1][j-1]` before using it — keep a diagonal temp
Counting version overflows	Distinct-subsequence counts grow fast — use a wide type

Check yourself

- ✓ State what `dp[i][j]` means and why the table is `(m+1)×(n+1)`.
- ✓ Explain the index shift between `dp[i]` and `s[i-1]`.
- ✓ Give the meaning of each of the three neighbouring cells.
- ✓ Map the four edit-distance operations onto those cells.
- ✓ Contrast the subsequence and substring recurrences.
- ✓ Say where the answer is read from in each case.
- ✓ Explain how to reduce the table to two rows and the trap in doing so.

Problems

Longest common subsequence `Medium`

Pattern: Two-string grid DP

Given two strings, return the length of their longest common **subsequence** — a sequence of characters appearing in both, in the same order but not necessarily contiguously. Return 0 if there is none.

Input	Output	Note
`text1 = "abcde", text2 = "ace"`	`3`	"ace"
`text1 = "abc", text2 = "abc"`	`3`	
`text1 = "abc", text2 = "def"`	`0`	

HINTS

1. Let `dp[i][j]` be the LCS length of the first `i` characters of one string and the first `j` of the other.
2. If the last characters match, they can both be used: `1 + dp[i-1][j-1]`.
3. If they do not, drop one character from either string and take the better: `max(dp[i-1][j], dp[i][j-1])`.
4. Row 0 and column 0 are all zeros — an empty string shares nothing.

Approach	Idea	Time	Space
Enumerate subsequences	Generate all 2^m subsequences of one string and test each.	`O(2^m·n)`	`O(m)`
Memoised recursion	Cache on `(i, j)`. Same complexity as the table, plus stack.	`O(m·n)`	`O(m·n)`
Bottom-up table	Fill row by row; the answer is the bottom-right corner.	`O(m·n)`	`O(m·n)`

Solution — js

```
function longestCommonSubsequence(text1, text2) {
  const m = text1.length, n = text2.length;
  // dp[i][j] = LCS of the first i chars of text1 and first j of text2
  const dp = Array.from({length: m + 1}, () => new Array(n + 1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (text1[i - 1] === text2[j - 1]) {
        dp[i][j] = 1 + dp[i - 1][j - 1];      // use both characters
      } else {
        dp[i][j] = Math.max(dp[i - 1][j],      // drop from text1
                            dp[i][j - 1]);     // drop from text2
      }
    }
  }
  return dp[m][n];
}
```

Solution — cpp

```
class Solution {
public:
    int longestCommonSubsequence(string a, string b) {
        int m = a.size(), n = b.size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                dp[i][j] = (a[i-1] == b[j-1])
                         ? 1 + dp[i-1][j-1]
                         : max(dp[i-1][j], dp[i][j-1]);
        return dp[m][n];
    }
};
```

Solution — python

```python
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j] = 1 + dp[i-1][j-1]
                else:
                    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
        return dp[m][n]
```

Solution — java

```java
class Solution {
    public int longestCommonSubsequence(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++)
                dp[i][j] = (a.charAt(i-1) == b.charAt(j-1))
                         ? 1 + dp[i-1][j-1]
                         : Math.max(dp[i-1][j], dp[i][j-1]);
        return dp[m][n];
    }
}
```

WALKTHROUGH

WHY THE MATCH CASE TAKES THE DIAGONAL AND ADDS ONE.

If the last characters agree, there is always an optimal LCS that uses them as a matched pair — so the rest of the answer is the LCS of both prefixes with that pair removed, which is `dp[i-1][j-1]`. Taking `max(dp[i-1][j], dp[i][j-1])` in the match case is not *wrong*, just never better, which is why the simpler form is used.

THE MISMATCH CASE IS A GENUINE CHOICE.

The final characters cannot both be in the subsequence, so at least one must be discarded — try both and keep the better.

THE ZERO ROW AND COLUMN ARE THE BASE CASES

, and they need no explicit code because the array is initialised to 0: an empty prefix shares nothing with anything.

THIS ONE TABLE ANSWERS SEVERAL QUESTIONS.

The shortest common supersequence is `m + n − LCS`. The minimum deletions to make two strings equal is `m + n − 2·LCS`. And the longest palindromic subsequence of a string is the LCS of that string with its own reverse — the `"abcba"` test hints at that.

Edit distance `Hard`

Pattern: Grid DP with three operations

Given two words, return the minimum number of operations to convert the first into the second. The permitted operations are **insert** a character, **delete** a character, and **replace** a character.

Input	Output	Note
`word1 = "horse", word2 = "ros"`	`3`	horse → rorse → rose → ros
`word1 = "intention", word2 = "execution"`	`5`	

HINTS

1. Same table as LCS, but minimising a cost rather than maximising a length.
2. If the last characters match, no operation is needed: carry `dp[i-1][j-1]` through unchanged.
3. Otherwise take the cheapest of replace (`dp[i-1][j-1]`), delete (`dp[i-1][j]`) and insert (`dp[i][j-1]`), plus one.
4. The base cases are not zero here: turning a prefix into an empty string costs one deletion per character.

Approach	Idea	Time	Space
Try every edit sequence	Explore all operation orders. Exponential.	`exponential`	`O(m+n)`
Memoised recursion on (i, j)	Natural top-down form of the same recurrence.	`O(m·n)`	`O(m·n)`
Bottom-up table	Explicit base rows, then fill.	`O(m·n)`	`O(m·n)`

Solution — js

```
function minDistance(word1, word2) {
  const m = word1.length, n = word2.length;
  const dp = Array.from({length: m + 1}, () => new Array(n + 1).fill(0));

  // base cases: i deletions to empty, j insertions from empty
  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (word1[i - 1] === word2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1];        // FREE — no operation
      } else {
        dp[i][j] = 1 + Math.min(
          dp[i - 1][j - 1],                 // replace
          dp[i - 1][j],                     // delete from word1
          dp[i][j - 1]                      // insert into word1
        );
      }
    }
  }
  return dp[m][n];
}
```

Solution — cpp

```cpp
class Solution {
public:
    int minDistance(string a, string b) {
        int m = a.size(), n = b.size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
        for (int i = 0; i <= m; ++i) dp[i][0] = i;
        for (int j = 0; j <= n; ++j) dp[0][j] = j;
        for (int i = 1; i <= m; ++i)
            for (int j = 1; j <= n; ++j)
                dp[i][j] = (a[i-1] == b[j-1])
                         ? dp[i-1][j-1]
                         : 1 + min({dp[i-1][j-1], dp[i-1][j], dp[i][j-1]});
        return dp[m][n];
    }
};
```

Solution — python

```python
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i-1] == word2[j-1]:
                    dp[i][j] = dp[i-1][j-1]
                else:
                    dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
        return dp[m][n]
```

Solution — java

```java
class Solution {
    public int minDistance(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++)
                dp[i][j] = (a.charAt(i-1) == b.charAt(j-1))
                         ? dp[i-1][j-1]
                         : 1 + Math.min(dp[i-1][j-1],
                               Math.min(dp[i-1][j], dp[i][j-1]));
        return dp[m][n];
    }
}
```

WALKTHROUGH

THE BASE CASES CARRY REAL INFORMATION HERE, UNLIKE LCS.

`dp[i][0] = i` says that turning an `i`-character prefix into the empty string costs `i` deletions, and `dp[0][j] = j` says building a `j`-character string from nothing costs `j` insertions. Leaving them at zero — as LCS can — silently makes every conversion look free from one direction, and the `["", "abc"]` tests catch it.

EACH NEIGHBOUR IS ONE OPERATION, AND THE MAPPING IS WORTH STATING ALOUD.

Moving diagonally consumes a character from both strings, which is a replacement. Moving up consumes only from `word1`, which is a deletion. Moving left consumes only from `word2`, which is an insertion into `word1`. The recurrence is just "pick the cheapest predecessor and pay one".

THE MATCH CASE COSTS NOTHING.

Writing `1 + min(...)` unconditionally is the classic off-by-one: identical strings would then cost their full length instead of 0.

THIS IS LEVENSHTEIN DISTANCE

, used in spell-checkers, diff tools and DNA alignment. Weighted variants — where a replacement costs 2, or different characters cost differently — change only the constants in the same recurrence.

Longest palindromic substring `Medium`

Pattern: Expand around centre

Given a string `s`, return the longest **substring** (contiguous) that is a palindrome.

If several are tied for longest, any one of them is acceptable — the tests use inputs with a unique answer.

Input	Output	Note
`s = "babad"`	`"bab"`	"aba" is also valid; the tests avoid such ties.
`s = "cbbd"`	`"bb"`	
`s = "a"`	`"a"`	

HINTS

1. Checking every substring for palindromicity is $O(n^3)$.
2. A palindrome is defined by its *centre*. There are `2n - 1` centres: one at each character, and one between each adjacent pair.
3. From each centre, expand outwards while the characters match.
4. The even-length centres are the ones people forget — "cbbd" has no odd-length answer longer than 1.

Approach	Idea	Time	Space
Check every substring	$O(n^2)$ substrings, O(n) to verify each.	`O(n³)`	`O(1)`
DP table of palindromic ranges	`dp[i][j]` = is `s[i..j]` a palindrome? Fill by increasing length.	`O(n²)`	`O(n²)`
Expand around centre	Same time, constant space, and much shorter to write.	`O(n²)`	`O(1)`
Manacher's algorithm	Linear time. Worth naming; almost never required.	`O(n)`	`O(n)`

Solution — js

```
function longestPalindrome(s) {
  if (s.length === 0) return "";

  let bestStart = 0, bestLen = 1;

  // grow outwards while the ends match; return the span found
  function expand(lo, hi) {
    while (lo >= 0 && hi < s.length && s[lo] === s[hi]) {
      lo--; hi++;
    }
    // the loop overshot by one on each side
    const len = hi - lo - 1;
    if (len > bestLen) { bestLen = len; bestStart = lo + 1; }
  }

  for (let i = 0; i < s.length; i++) {
    expand(i, i);       // ODD  length centre: one character
    expand(i, i + 1);   // EVEN length centre: between two
  }

  return s.slice(bestStart, bestStart + bestLen);
}
```

Solution — cpp

```
class Solution {
    int bestStart = 0, bestLen = 1;
    void expand(const string& s, int lo, int hi) {
        while (lo >= 0 && hi < (int)s.size() && s[lo] == s[hi]) { --lo; ++hi; }
        int len = hi - lo - 1;
        if (len > bestLen) { bestLen = len; bestStart = lo + 1; }
    }
public:
    string longestPalindrome(string s) {
        if (s.empty()) return "";
        bestStart = 0; bestLen = 1;
        for (int i = 0; i < (int)s.size(); ++i) {
            expand(s, i, i);
            expand(s, i, i + 1);
        }
        return s.substr(bestStart, bestLen);
    }
};
```

Solution — python

```python
class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""
        self.best_start, self.best_len = 0, 1

        def expand(lo, hi):
            while lo >= 0 and hi < len(s) and s[lo] == s[hi]:
                lo -= 1
                hi += 1
            length = hi - lo - 1
            if length > self.best_len:
                self.best_len = length
                self.best_start = lo + 1

        for i in range(len(s)):
            expand(i, i)
            expand(i, i + 1)
        return s[self.best_start:self.best_start + self.best_len]
```

Solution — java

```java
class Solution {
    private int bestStart = 0, bestLen = 1;
    public String longestPalindrome(String s) {
        if (s.isEmpty()) return "";
        bestStart = 0; bestLen = 1;
        for (int i = 0; i < s.length(); i++) {
            expand(s, i, i);
            expand(s, i, i + 1);
        }
        return s.substring(bestStart, bestStart + bestLen);
    }
    private void expand(String s, int lo, int hi) {
        while (lo >= 0 && hi < s.length() && s.charAt(lo) == s.charAt(hi)) {
            lo--; hi++;
        }
        int len = hi - lo - 1;
        if (len > bestLen) { bestLen = len; bestStart = lo + 1; }
    }
}
```

WALKTHROUGH

BOTH CENTRE KINDS ARE REQUIRED.

An odd-length palindrome is centred on a character; an even-length one is centred between two. Calling `expand(i, i)` only would return `"c"` for `"cbbd"` instead of `"bb"`. That is why the loop makes two calls per index, giving `2n - 1` centres in total.

THE OFF-BY-ONE AFTER THE LOOP.

The `while` exits having already moved `lo` one too far left and `hi` one too far right, so the valid span is `[lo+1, hi-1]` and its length is `(hi-1) - (lo+1) + 1 = hi - lo - 1`. Deriving that on the spot rather than guessing is the difference between this working first time and not.

WHY EXPANSION BEATS THE DP TABLE HERE.

Both are $O(n^2)$ time, but expansion is O(1) space and about eight lines. The table version is worth mentioning because it generalises to counting palindromic substrings, but for this question expansion is strictly better.

NOTE THIS IS A *SUBSTRING* PROBLEM

, so contiguity matters — contrast with longest palindromic *subsequence*, which is the LCS of the string with its reverse and uses the grid from the first problem on this page.

DP: grids and sequences

Two more shapes: paths through a grid, where each cell depends on its neighbours, and increasing subsequences, where the classic $O(n^2)$ has a genuinely clever $O(n \log n)$ alternative.

Grid DP

```
THE STATE IS THE CELL. Movement is restricted — usually
right and down only — which is what makes it acyclic and
therefore a DP rather than a graph search.

  dp[r][c] = answer for the subproblem ending at (r, c)

  dp[r][c] = f(dp[r-1][c], dp[r][c-1])
                ^ from above   ^ from the left

COUNT PATHS       dp[r][c] = dp[r-1][c] + dp[r][c-1]
MINIMUM PATH SUM  dp[r][c] = grid[r][c]
                           + min(dp[r-1][c], dp[r][c-1])
MAXIMUM SQUARE    dp[r][c] = 1 + min(up, left, diagonal)
                           when grid[r][c] === 1

BASE CASES ARE THE FIRST ROW AND FIRST COLUMN — they have
only one predecessor each, so they accumulate in a
straight line.

THE OBSTACLE VARIANT: a blocked cell gets dp = 0 and is
skipped, which propagates naturally. Blocking the START
makes the whole answer 0.

SPACE: each row depends only on the row above, so one
row updated left to right suffices — O(n) instead of
O(m·n).

IF MOVEMENT IS UNRESTRICTED (any direction, or weights
vary), it is no longer a DP — it is a shortest-path
problem and needs BFS or Dijkstra (chapter 30).

————————————————————————————————————————
LONGEST INCREASING SUBSEQUENCE

THE O(n^2) DP:
  dp[i] = length of the longest increasing subsequence
          ENDING at index i

  dp[i] = 1 + max(dp[j])  over all j < i with a[j] < a[i]
  answer = max(dp)

  Note the answer is the MAXIMUM of the table, not
  dp[n-1] — the best subsequence need not use the last
  element.

THE O(n log n) VERSION — patience sorting:

  tails[k] = the SMALLEST possible tail value of an
             increasing subsequence of length k+1

  for (x of a) {
      i = lowerBound(tails, x);   // first tail >= x
      if (i === tails.length) tails.push(x);
      else tails[i] = x;          // REPLACE, keep it small
  }
  return tails.length;

  `tails` IS NOT THE SUBSEQUENCE. It is only guaranteed to
  have the right LENGTH. Recovering the actual sequence
  needs parent pointers.
```

```
    THE INTUITION: a shorter tail leaves more room for
    future elements, so always keep the smallest tail for
    each length. Binary searching that array is what
    removes the inner loop.
```

Problems

How it shows up

Symptom	Cause
Grid answer off by a row or column	First row and column not seeded before the main loops
Obstacle grid returns a positive count when blocked at the start	Start cell not checked
LIS returns the last table entry	The answer is the *maximum* over the table
LIS counts equal values as increasing	Use strict `<`; for non-decreasing use upper bound instead
Patience version returns a wrong sequence	`tails` holds lengths, not the actual subsequence
Rolling row gives wrong results	Updated in the wrong direction, clobbering a needed value
Path count overflows	Binomial growth — use a wide type

Check yourself

- ✓ Give the grid recurrence for path counting and for minimum path sum.
- ✓ Say why the first row and column are the base cases.
- ✓ Explain when a grid problem stops being DP and becomes a graph search.
- ✓ Write the $O(n^2)$ LIS recurrence and say where the answer is read.
- ✓ Explain what `tails[k]` means in the O(n log n) version.
- ✓ Say why keeping the smallest tail is correct.
- ✓ State what `tails` does *not* give you.

Problems

Unique paths with obstacles `Medium`

Pattern: Grid path counting

A robot starts at the top-left of an `m × n` grid and must reach the bottom-right, moving only **right or down**. Cells containing `1` are obstacles and cannot be entered.

Return the number of distinct paths.

Input	Output	Note
`[[0,0,0],[0,1,0],[0,0,0]]`	`2`	
`[[0,1],[0,0]]`	`1`	
`[[1]]`	`0`	The start itself is blocked.

HINTS

1. Every cell is reached only from above or from the left, so the counts simply add.
2. An obstacle contributes zero paths — set its cell to 0 and the zero propagates.
3. Seed the start at 1, then fill row by row.
4. Check the start and end cells for obstacles; a blocked start makes the answer 0.

Approach	Idea	Time	Space
Backtracking	Explore every route. Exponential.	`O(2^(m+n))`	`O(m+n)`
Full 2-D table	One entry per cell, filled top-left to bottom-right.	`O(m·n)`	`O(m·n)`
Rolling single row	Each row needs only the one above, so a single array updated left to right suffices.	`O(m·n)`	`O(n)`

Solution — js

```
function uniquePathsWithObstacles(grid) {
  const m = grid.length, n = m ? grid[0].length : 0;
  if (!m || !n || grid[0][0] === 1) return 0;   // blocked start

  const dp = Array.from({length: m}, () => new Array(n).fill(0));
  dp[0][0] = 1;

  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      if (grid[r][c] === 1) { dp[r][c] = 0; continue; }   // obstacle
      if (r > 0) dp[r][c] += dp[r - 1][c];   // from above
      if (c > 0) dp[r][c] += dp[r][c - 1];   // from the left
    }
  }
  return dp[m - 1][n - 1];
}
```

Solution — cpp

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& grid) {
        int m = grid.size(), n = m ? grid[0].size() : 0;
        if (!m || !n || grid[0][0] == 1) return 0;
        vector<vector<long long>> dp(m, vector<long long>(n, 0));
        dp[0][0] = 1;
        for (int r = 0; r < m; ++r)
            for (int c = 0; c < n; ++c) {
                if (grid[r][c] == 1) { dp[r][c] = 0; continue; }
                if (r > 0) dp[r][c] += dp[r-1][c];
                if (c > 0) dp[r][c] += dp[r][c-1];
            }
        return (int)dp[m-1][n-1];
    }
};
```

Solution — python

```
class Solution:
    def uniquePathsWithObstacles(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0]) if m else 0
        if not m or not n or grid[0][0] == 1:
            return 0
        dp = [[0] * n for _ in range(m)]
        dp[0][0] = 1
        for r in range(m):
            for c in range(n):
                if grid[r][c] == 1:
                    dp[r][c] = 0
                    continue
                if r > 0:
                    dp[r][c] += dp[r-1][c]
                if c > 0:
                    dp[r][c] += dp[r][c-1]
        return dp[m-1][n-1]
```

Solution — java

```
class Solution {
    public int uniquePathsWithObstacles(int[][] grid) {
        int m = grid.length, n = m > 0 ? grid[0].length : 0;
        if (m == 0 || n == 0 || grid[0][0] == 1) return 0;
        int[][] dp = new int[m][n];
        dp[0][0] = 1;
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 1) { dp[r][c] = 0; continue; }
                if (r > 0) dp[r][c] += dp[r-1][c];
                if (c > 0) dp[r][c] += dp[r][c-1];
            }
        return dp[m-1][n-1];
    }
}
```

WALKTHROUGH

WHY THE COUNTS ADD RATHER THAN COMBINE SOME OTHER WAY.

The last move into `(r, c)` was either downward from `(r-1, c)` or rightward from `(r, c-1)`. Those two sets of paths are disjoint — they differ in their final step — and together they are all of them. So the total is the sum, and the same reasoning as in climbing stairs applies, one dimension up.

OBSTACLES NEED NO SPECIAL PROPAGATION.

Setting a blocked cell to 0 means it contributes nothing to its right and lower neighbours, and the effect ripples outward automatically. The only case that needs an explicit check is a blocked *start*, because `dp[0][0]` is seeded before the loop runs.

SEEDING THEN ADDING HANDLES THE FIRST ROW AND COLUMN.

`dp[0][0] = 1` is set outside, and the `r > 0` / `c > 0` guards mean edge cells accumulate from their single predecessor. Note that `dp[0][0]` is not overwritten in the loop because both guards fail there.

THE ROLLING-ROW VERSION

updates one array left to right: `dp[c] += dp[c-1]`, where `dp[c]` already holds the value from the row above. That is the O(n)-space follow-up, and the direction matters — going right to left would use the wrong neighbour.

Minimum path sum `Medium`

Pattern: Grid DP, minimising

Given an `m × n` grid of non-negative numbers, find a path from top-left to bottom-right that minimises the sum of the numbers along it. You may move only **right or down**.

Input	Output	Note
`[[1,3,1],[1,5,1],[4,2,1]]`	`7`	1→3→1→1→1
`[[1,2,3],[4,5,6]]`	`12`	

HINTS

1. Same grid, same movement — only the combine rule changes from `+` to `min`.
2. Each cell costs its own value plus the cheaper of the two ways to arrive.
3. The first row and first column have only one predecessor, so they are running totals.
4. You can write the result into the input grid for O(1) extra space, if mutation is acceptable.

Approach	Idea	Time	Space
Explore every path	Exponentially many routes.	`O(2^(m+n))`	`O(m+n)`
Dijkstra	Works, but the movement restriction makes the graph acyclic — the log factor is wasted.	`O(m·n·log)`	`O(m·n)`
Grid DP	Fill top-left to bottom-right taking the cheaper predecessor.	`O(m·n)`	`O(1) if in place`

Solution — js

```
function minPathSum(grid) {
  const m = grid.length, n = m ? grid[0].length : 0;
  if (!m || !n) return 0;

  const dp = Array.from({length: m}, () => new Array(n).fill(0));
  dp[0][0] = grid[0][0];

  // first column: only one way in — from above
  for (let r = 1; r < m; r++) dp[r][0] = dp[r - 1][0] + grid[r][0];
  // first row: only one way in — from the left
  for (let c = 1; c < n; c++) dp[0][c] = dp[0][c - 1] + grid[0][c];

  for (let r = 1; r < m; r++)
    for (let c = 1; c < n; c++)
      dp[r][c] = grid[r][c] + Math.min(dp[r - 1][c], dp[r][c - 1]);

  return dp[m - 1][n - 1];
}
```

Solution — cpp

```cpp
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int m = grid.size(), n = m ? grid[0].size() : 0;
        if (!m || !n) return 0;
        vector<vector<int>> dp(m, vector<int>(n, 0));
        dp[0][0] = grid[0][0];
        for (int r = 1; r < m; ++r) dp[r][0] = dp[r-1][0] + grid[r][0];
        for (int c = 1; c < n; ++c) dp[0][c] = dp[0][c-1] + grid[0][c];
        for (int r = 1; r < m; ++r)
            for (int c = 1; c < n; ++c)
                dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1]);
        return dp[m-1][n-1];
    }
};
```

Solution — python

```python
class Solution:
    def minPathSum(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0]) if m else 0
        if not m or not n:
            return 0
        dp = [[0] * n for _ in range(m)]
        dp[0][0] = grid[0][0]
        for r in range(1, m):
            dp[r][0] = dp[r-1][0] + grid[r][0]
        for c in range(1, n):
            dp[0][c] = dp[0][c-1] + grid[0][c]
        for r in range(1, m):
            for c in range(1, n):
                dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1])
        return dp[m-1][n-1]
```

Solution — java

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = m > 0 ? grid[0].length : 0;
        if (m == 0 || n == 0) return 0;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
        for (int r = 1; r < m; r++) dp[r][0] = dp[r-1][0] + grid[r][0];
        for (int c = 1; c < n; c++) dp[0][c] = dp[0][c-1] + grid[0][c];
        for (int r = 1; r < m; r++)
            for (int c = 1; c < n; c++)
                dp[r][c] = grid[r][c] + Math.min(dp[r-1][c], dp[r][c-1]);
        return dp[m-1][n-1];
    }
}
```

WALKTHROUGH

ONLY THE COMBINE RULE CHANGED.

Counting paths added the two predecessors; minimising takes the smaller and adds the current cell's cost. The traversal order, the base cases and the state are identical — which is the point of recognising a problem *family* rather than memorising individual solutions.

SEEDING THE EDGES EXPLICITLY

is slightly more verbose than the guard-based style used in the previous problem, but it reads more clearly for a minimisation: an edge cell has no choice to make, so writing it as a running total says so directly. Either style is fine; be consistent.

WHY DIJKSTRA IS OVERKILL.

Because movement is restricted to right and down, the grid is a DAG and every cell's predecessors are guaranteed to be computed before it. That removes any need for a priority queue. If diagonal or upward movement were allowed the graph would have cycles and Dijkstra would become the right tool — that is the follow-up question to be ready for.

Longest increasing subsequence `Medium`

Pattern: Patience sorting with binary search

Given an integer array `nums`, return the length of the longest **strictly increasing subsequence**. Elements need not be contiguous.

Aim for O(n log n).

Input	Output	Note
`nums = [10,9,2,5,3,7,101,18]`	`4`	[2,3,7,101]
`nums = [0,1,0,3,2,3]`	`4`	
`nums = [7,7,7,7]`	`1`	Strictly increasing, so equal values do not extend.

HINTS

1. The O(n^2) version is `dp[i] = 1 + max(dp[j])` over earlier smaller elements — write that first.
2. For O(n log n): maintain `tails`, where `tails[k]` is the smallest tail among all increasing subsequences of length `k+1`.
3. `tails` is always sorted, so binary search for the first entry ≥ the current value.
4. If nothing is ≥ it, the value extends the longest run — append. Otherwise replace that entry, keeping tails as small as possible.

Approach	Idea	Time	Space
Try every subsequence	Exponential.	`O(2ⁿ)`	`O(n)`
O(n^2) DP	`dp[i]` = LIS ending at `i`; answer is the table maximum. Easy to derive and to explain.	`O(n²)`	`O(n)`

Approach	Idea	Time	Space
Patience sorting	Binary search into a tails array; length of the array is the answer.	`O(n log n)`	`O(n)`

Solution — js

```
function lengthOfLIS(nums) {
  // tails[k] = smallest possible tail of an increasing
  // subsequence of length k+1.  Always sorted ascending.
  const tails = [];

  for (const x of nums) {
    // lower bound: first index whose tail is >= x
    let lo = 0, hi = tails.length;
    while (lo < hi) {
      const mid = lo + ((hi - lo) >> 1);
      if (tails[mid] >= x) hi = mid;
      else                 lo = mid + 1;
    }

    if (lo === tails.length) tails.push(x);  // extends the longest
    else                     tails[lo] = x;  // better tail, same length
  }

  return tails.length;
}
```

Solution — cpp

```
class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        vector<int> tails;
        for (int x : nums) {
            auto it = lower_bound(tails.begin(), tails.end(), x);
            if (it == tails.end()) tails.push_back(x);
            else                   *it = x;
        }
        return tails.size();
    }
};
```

Solution — python

```
import bisect

class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        tails = []
        for x in nums:
            i = bisect.bisect_left(tails, x)   # first tail >= x
            if i == len(tails):
                tails.append(x)
            else:
                tails[i] = x
        return len(tails)
```

Solution — java

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;
        for (int x : nums) {
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                if (tails[mid] >= x) hi = mid;
                else                 lo = mid + 1;
            }
            tails[lo] = x;
            if (lo == size) size++;
        }
        return size;
    }
}
```

WALKTHROUGH

WHY KEEPING THE SMALLEST TAIL IS CORRECT.

Suppose two increasing subsequences both have length 3, ending in 9 and in 5. Any future element that can extend the one ending in 9 can also extend the one ending in 5, but not conversely. So the larger tail is never useful and can be discarded — the array only ever needs the smallest tail per length. That greedy exchange argument is the whole justification, and it is what an interviewer will ask for.

`TAILS` IS SORTED, WHICH IS WHAT PERMITS THE BINARY SEARCH.

It stays sorted because each operation either appends a value larger than everything (extending the longest run) or replaces an entry with something smaller than it but larger than its predecessor.

LOWER BOUND VERSUS UPPER BOUND IS THE STRICTNESS SWITCH.

Searching for the first tail `>= x` means an equal value *replaces* rather than extends, giving strictly increasing — which is why `[7,7,7,7]` returns 1. For a non-decreasing LIS, search for the first tail `> x` instead. One comparison, two different problems.

THE HONEST CAVEAT:

`tails` ends up with the right *length* but is generally not itself a valid subsequence of the input. Reconstructing the actual sequence requires recording, for each element, the index it was placed at and its predecessor — worth mentioning if asked to return the subsequence rather than its length.

DP: interval, tree and bitmask

Three state shapes beyond the linear ones. Each is recognisable from the constraints, and each has a fill order that is the whole difficulty.

Interval DP

```
STATE: a RANGE.   dp[i][j] = answer for the segment
                             a[i..j]

RECURRENCE: split the range at every possible point and
take the best.

  dp[i][j] = best over k in (i, j) of
               dp[i][k] + dp[k][j] + cost(i, k, j)

THE FILL ORDER IS THE WHOLE TRICK: iterate by INCREASING
LENGTH, not by increasing i.

  for (len = 2; len <= n; len++)
      for (i = 0; i + len <= n; i++) {
          j = i + len - 1;
          ...
      }

Filling by rows would read dp[i][k] and dp[k][j] before
they exist, because both are SHORTER ranges that live on
later rows or earlier columns.

COST: O(n^3) - n^2 states, O(n) split points each. Which
is exactly why the constraint is typically n <= 100..500
(chapter 02).

THE FAMILY: burst balloons, matrix chain multiplication,
minimum cost to merge stones, palindrome partitioning II,
optimal BST.

------------------------------------------------------------
TREE DP

STATE: a NODE, usually with a small extra flag.
The children are the subproblems, so a POSTORDER
traversal is automatically a valid fill order - no
ordering work needed (chapter 24).

  function dfs(node) {
      if (!node) return baseCase;
      const L = dfs(node.left), R = dfs(node.right);
      return combine(L, R, node.val);
  }

THE COMMON SHAPE returns a PAIR: "best if I take this
node" and "best if I do not". House Robber III is exactly
that, and so are most tree-selection problems.

------------------------------------------------------------
BITMASK DP

STATE: a SUBSET, encoded as an integer (chapter 16).

  dp[mask] = answer having already used the items in mask

2^n states, so it is only viable for n <= 20 or so.
THAT CONSTRAINT IS THE SIGNAL: if you see n <= 20 and the
problem is about assignments or orderings, think bitmask.

THE TWO STANDARD FORMS:
```

```
    ITERATE OVER SUBSETS
      for (mask = 0; mask < (1 << n); mask++)
          for (i = 0; i < n; i++)
              if (!(mask & (1 << i)))
                  dp[mask | (1<<i)] = best(dp[mask] + cost)

    TRAVELLING-SALESMAN STYLE — dp[mask][last]
      state also records WHERE you currently are.
      O(2^n * n^2).

  THE FAMILY: assignment problems, TSP, partition into k
  equal subsets, shortest superstring, "can I win this
  game?".
```

Problems

How it shows up

Symptom	Cause
Interval DP reads zeros	Filled by row instead of by increasing length
Burst balloons double-counts	Think of the balloon burst *last*, not first
Out-of-range index at the interval edges	Pad the array with sentinel 1s and use an exclusive range
Tree DP returns one value where two are needed	Return a (take, skip) pair
Bitmask table too large	2^n only survives to about n = 20–25
Bitmask transitions skip states	Iterate masks in increasing order so predecessors are ready
Popcount recomputed in the inner loop	Precompute it, or derive the index from the mask

Check yourself

- ✓ Give the interval DP recurrence and its fill order, and say why the order matters.
- ✓ State the complexity of interval DP and the constraint that signals it.
- ✓ Say why a postorder traversal is automatically a valid tree-DP fill order.
- ✓ Describe the (take, skip) pair pattern.
- ✓ Say what `dp[mask]` means and the n at which bitmask DP stops being viable.
- ✓ Give the two standard bitmask iteration forms.
- ✓ Explain the "burst last" reframing.

Problems

Burst balloons `Hard`

Pattern: Interval DP

You have `n` balloons, each painted with a number. Bursting balloon `i` earns `nums[left] × nums[i] × nums[right]`, where `left` and `right` are its **current** neighbours; balloons outside the array count as 1.

Return the maximum coins obtainable by bursting all of them.

Input	Output	Note
`nums = [3,1,5,8]`	`167`	3·1·5 + 3·5·8 + 1·3·8 + 1·8·1
`nums = [1,5]`	`10`	

HINTS

1. Thinking about which balloon to burst *first* fails — the array changes and the subproblems overlap in a way that does not decompose.
2. Instead ask: which balloon in this range is burst **LAST** ?
3. If `k` is burst last in the open range `(i, j)`, then everything to its left and right is already gone, so its neighbours are exactly `i` and `j`.
4. That makes the two sides independent: `dp[i][k] + dp[k][j] + nums[i]·nums[k]·nums[j]`.
5. Pad the array with 1 at both ends so the boundary balloons need no special case.

Approach	Idea	Time	Space
Try every burst order	n! orderings.	`O(n!)`	`O(n)`
DP on "burst first"	**Does not work.** After bursting one balloon the two sides are *not* independent — the surviving neighbours change.	`—`	`—`
Interval DP on "burst last"	Fix the last balloon in each range; the sides become independent.	`O(n³)`	`O(n²)`

Solution — js

```
function maxCoins(nums) {
  const a = [1, ...nums, 1];        // virtual 1s at both ends
  const n = a.length;
  // dp[i][j] = max coins from bursting everything STRICTLY
  // between i and j (an OPEN range)
  const dp = Array.from({length: n}, () => new Array(n).fill(0));

  // fill by increasing span so shorter ranges are ready
  for (let span = 2; span < n; span++) {
    for (let i = 0; i + span < n; i++) {
      const j = i + span;

      for (let k = i + 1; k < j; k++) {
        // k is burst LAST, so its neighbours are i and j
        const coins = a[i] * a[k] * a[j] + dp[i][k] + dp[k][j];
        if (coins > dp[i][j]) dp[i][j] = coins;
      }
    }
  }
  return dp[0][n - 1];
}
```

Solution — cpp

```cpp
class Solution {
public:
    int maxCoins(vector<int>& nums) {
        vector<int> a;
        a.push_back(1);
        for (int v : nums) a.push_back(v);
        a.push_back(1);
        int n = a.size();
        vector<vector<int>> dp(n, vector<int>(n, 0));
        for (int span = 2; span < n; ++span)
            for (int i = 0; i + span < n; ++i) {
                int j = i + span;
                for (int k = i + 1; k < j; ++k)
                    dp[i][j] = max(dp[i][j],
                                   a[i]*a[k]*a[j] + dp[i][k] + dp[k][j]);
            }
        return dp[0][n-1];
    }
};
```

Solution — python

```python
class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        a = [1] + nums + [1]
        n = len(a)
        dp = [[0] * n for _ in range(n)]
        for span in range(2, n):
            for i in range(0, n - span):
                j = i + span
                for k in range(i + 1, j):
                    dp[i][j] = max(dp[i][j],
                                   a[i]*a[k]*a[j] + dp[i][k] + dp[k][j])
        return dp[0][n-1]
```

Solution — java

```java
class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length + 2;
        int[] a = new int[n];
        a[0] = a[n-1] = 1;
        for (int i = 0; i < nums.length; i++) a[i+1] = nums[i];
        int[][] dp = new int[n][n];
        for (int span = 2; span < n; span++)
            for (int i = 0; i + span < n; i++) {
                int j = i + span;
                for (int k = i + 1; k < j; k++)
                    dp[i][j] = Math.max(dp[i][j],
                               a[i]*a[k]*a[j] + dp[i][k] + dp[k][j]);
            }
        return dp[0][n-1];
    }
}
```

WALKTHROUGH

WHY "FIRST" FAILS AND "LAST" WORKS.

If you burst `k` first, the two remaining halves are not independent — a balloon in the left half may later become adjacent to one in the right half, so their scores are coupled and the problem does not decompose. If instead `k` is burst *last* within the open range `(i, j)`, then by definition everything strictly between `i` and `k`, and between `k` and `j`, has already gone. At the moment `k` pops, its neighbours are exactly `i` and `j` — fixed and known — and the two sides never interact. That reframing is the entire problem.

THE PADDING REMOVES EVERY BOUNDARY CASE.

Prepending and appending 1 means the outermost real balloons have well-defined neighbours worth 1, matching the problem statement, with no `if (i === 0)` anywhere.

THE OPEN-RANGE CONVENTION.

`dp[i][j]` covers the balloons strictly between `i` and `j`, so `dp[i][i+1]` is empty and worth 0 — which is why `span` starts at 2 and the answer is `dp[0][n-1]`, the range strictly inside both sentinels.

FILL BY SPAN, NOT BY ROW.

`dp[i][k]` and `dp[k][j]` are both strictly shorter ranges. Iterating `i` in the outer loop would reach `dp[0][n-1]` while its shorter sub-ranges are still zero, silently returning a wrong answer rather than crashing — which makes it a nasty bug to spot.

House robber III — on a tree `Medium`

Pattern: Tree DP with a (take, skip) pair

Houses are arranged as a binary tree. Robbing two **directly connected** houses — a parent and its child — triggers the alarm.

Return the maximum amount that can be robbed.

Input	Output	Note
`root = [3,2,3,null,3,null,1]`	`7`	3 + 3 + 1 (the root and both grandchildren).
`root = [3,4,5,1,3,null,1]`	`9`	4 + 5.

HINTS

1. A single number per node is not enough — the parent needs to know both outcomes.
2. Return a pair: the best total for this subtree *if this node is robbed*, and *if it is not*.
3. If robbed: `node.val` plus the "skipped" values of both children.
4. If skipped: each child independently contributes `max(itsRobbed, itsSkipped)`.

Approach	Idea	Time	Space
Memoised on the node	Recurse over children and grandchildren, caching per node. Works, but recomputes grandchildren.	`O(n)`	`O(n)`
(take, skip) pair per node	One postorder pass, constant work per node, no memo needed.	`O(n)`	`O(h)`

Solution — js

```
function rob(root) {
  // returns [best if this node IS robbed, best if it is NOT]
  function dfs(node) {
    if (!node) return [0, 0];

    const [lRob, lSkip] = dfs(node.left);
    const [rRob, rSkip] = dfs(node.right);

    // rob here -> both children MUST be skipped
    const robbed = node.val + lSkip + rSkip;

    // skip here -> each child does whatever is best for it
    const skipped = Math.max(lRob, lSkip) + Math.max(rRob, rSkip);

    return [robbed, skipped];
  }

  const [a, b] = dfs(root);
  return Math.max(a, b);
}
```

Solution — cpp

```
class Solution {
    pair<int,int> dfs(TreeNode* n) {
        if (!n) return {0, 0};
        auto [lRob, lSkip] = dfs(n->left);
        auto [rRob, rSkip] = dfs(n->right);
        int robbed  = n->val + lSkip + rSkip;
        int skipped = max(lRob, lSkip) + max(rRob, rSkip);
        return {robbed, skipped};
    }
public:
    int rob(TreeNode* root) {
        auto [a, b] = dfs(root);
        return max(a, b);
    }
};
```

Solution — python

```
class Solution:
    def rob(self, root: TreeNode) -> int:
        def dfs(node):
            if not node:
                return (0, 0)
            l_rob, l_skip = dfs(node.left)
            r_rob, r_skip = dfs(node.right)
            robbed = node.val + l_skip + r_skip
            skipped = max(l_rob, l_skip) + max(r_rob, r_skip)
            return (robbed, skipped)

        return max(dfs(root))
```

Solution — java

```
class Solution {
    public int rob(TreeNode root) {
        int[] r = dfs(root);
        return Math.max(r[0], r[1]);
    }
    // returns {robbedHere, skippedHere}
    private int[] dfs(TreeNode n) {
        if (n == null) return new int[]{0, 0};
        int[] l = dfs(n.left), r = dfs(n.right);
        int robbed  = n.val + l[1] + r[1];
        int skipped = Math.max(l[0], l[1]) + Math.max(r[0], r[1]);
        return new int[]{robbed, skipped};
    }
}
```

WALKTHROUGH

WHY ONE NUMBER PER NODE IS INSUFFICIENT.

If a child reports only "the best I can do", the parent cannot tell whether that best involved robbing the child — which is exactly what determines whether the parent may rob itself. Returning both outcomes gives the parent the information it needs, and is the tree analogue of the two rolling variables in the array version (chapter 33).

THE ASYMMETRY IN THE TWO BRANCHES.

Robbing here *forces* both children to be skipped, so it uses their skip values specifically. Skipping here *frees* each child to choose, so it takes each child's maximum independently. Using `max` in the robbed branch is the classic bug and it silently overcounts adjacent houses.

`[1,10,10]` **IS THE DISCRIMINATING TEST:**

the root is worth 1 and its two children are worth 10 each, so the answer is 20 — skipping the root. A greedy or root-biased solution returns 1 or 11.

POSTORDER IS THE FILL ORDER, FOR FREE.

Because both recursive calls complete before the combine, every dependency is satisfied without any explicit ordering logic — the recursion structure *is* the topological sort.

Partition into k equal-sum subsets `Hard`

Pattern: Bitmask DP over subsets

Given an array `nums` and an integer `k`, decide whether it can be partitioned into `k` non-empty subsets all having the **same sum**.

`nums.length` is at most 16.

Input	Output	Note
`nums = [4,3,2,3,5,2,1], k = 4`	`true`	(5), (1,4), (2,3), (2,3)
`nums = [1,2,3,4], k = 3`	`false`	

HINTS

1. `n ≤ 16` is the signal: 2^n states are affordable, so encode "which elements are used" as a bitmask.
2. If the total is not divisible by `k`, or any element exceeds the target, the answer is immediately false.
3. Track the running sum *within the current subset* as `usedSum(mask) % target` — you never need to know which subset you are on.
4. Only add an element if it fits in the current partial subset; that single rule keeps every completed subset at exactly the target.

Approach	Idea	Time	Space
Backtracking over k buckets	Assign each number to a bucket. Works with good pruning, but exponential and fiddly.	`O(kⁿ)`	`O(n)`
Bitmask DP	`dp[mask]` = is this set of elements reachable as a sequence of complete subsets plus a valid partial one?	`O(2ⁿ·n)`	`O(2ⁿ)`

Solution — js

```
function canPartitionKSubsets(nums, k) {
  const total = nums.reduce((a, b) => a + b, 0);
  if (k <= 0 || total % k !== 0) return false;

  const target = total / k;
  if (Math.max(...nums) > target) return false;

  const n = nums.length;
  const full = (1 << n) - 1;

  const dp   = new Array(1 << n).fill(false);  // reachable?
  const used = new Array(1 << n).fill(0);      // sum of mask
  dp[0] = true;

  for (let mask = 0; mask <= full; mask++) {
    if (!dp[mask]) continue;

    // how full the CURRENT partial subset is
    const partial = used[mask] % target;

    for (let i = 0; i < n; i++) {
      if (mask & (1 << i)) continue;          // already used
      if (partial + nums[i] > target) continue; // would overflow

      const next = mask | (1 << i);
      if (!dp[next]) {
        dp[next] = true;
        used[next] = used[mask] + nums[i];
      }
    }
  }
  return dp[full];
}
```

Solution — cpp

```cpp
class Solution {
public:
    bool canPartitionKSubsets(vector<int>& nums, int k) {
        int total = accumulate(nums.begin(), nums.end(), 0);
        if (k <= 0 || total % k) return false;
        int target = total / k;
        if (*max_element(nums.begin(), nums.end()) > target) return false;
        int n = nums.size(), full = (1 << n) - 1;
        vector<char> dp(1 << n, false);
        vector<int> used(1 << n, 0);
        dp[0] = true;
        for (int mask = 0; mask <= full; ++mask) {
            if (!dp[mask]) continue;
            int partial = used[mask] % target;
            for (int i = 0; i < n; ++i) {
                if (mask & (1 << i)) continue;
                if (partial + nums[i] > target) continue;
                int nxt = mask | (1 << i);
                if (!dp[nxt]) { dp[nxt] = true; used[nxt] = used[mask] + nums[i]; }
            }
        }
        return dp[full];
    }
};
```

Solution — python

```python
class Solution:
    def canPartitionKSubsets(self, nums: list[int], k: int) -> bool:
        total = sum(nums)
        if k <= 0 or total % k:
            return False
        target = total // k
        if max(nums) > target:
            return False

        n = len(nums)
        full = (1 << n) - 1
        dp = [False] * (1 << n)
        used = [0] * (1 << n)
        dp[0] = True

        for mask in range(full + 1):
            if not dp[mask]:
                continue
            partial = used[mask] % target
            for i, v in enumerate(nums):
                if mask & (1 << i):
                    continue
                if partial + v > target:
                    continue
                nxt = mask | (1 << i)
                if not dp[nxt]:
                    dp[nxt] = True
                    used[nxt] = used[mask] + v
        return dp[full]
```

Solution — java

```java
class Solution {
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int total = 0, mx = 0;
        for (int v : nums) { total += v; mx = Math.max(mx, v); }
        if (k <= 0 || total % k != 0) return false;
        int target = total / k;
        if (mx > target) return false;
        int n = nums.length, full = (1 << n) - 1;
        boolean[] dp = new boolean[1 << n];
        int[] used = new int[1 << n];
        dp[0] = true;
        for (int mask = 0; mask <= full; mask++) {
            if (!dp[mask]) continue;
            int partial = used[mask] % target;
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0) continue;
                if (partial + nums[i] > target) continue;
                int nxt = mask | (1 << i);
                if (!dp[nxt]) { dp[nxt] = true; used[nxt] = used[mask] + nums[i]; }
            }
        }
        return dp[full];
    }
}
```

WALKTHROUGH

THE MODULO TRICK IS WHAT KEEPS THE STATE SMALL.

You might expect to need `dp[mask][subsetIndex][partialSum]`. But if every completed subset sums to exactly `target`, then `used[mask] % target` *is* the partial sum of the subset currently being filled, and `used[mask] / target` is how many are finished. Both are derivable from the mask, so the state collapses to a single dimension of size 2^n.

THE FIT CHECK ENFORCES THE INVARIANT.

Refusing to add an element that would push the partial sum past `target` means a subset can only ever be closed by hitting `target` exactly — at which point the modulo wraps to 0 and the next element begins a fresh subset. No explicit "start a new subset" step is needed.

MASKS ARE VISITED IN INCREASING NUMERICAL ORDER

, which is a valid topological order because `mask | (1 << i)` is always strictly greater than `mask`. Every predecessor is therefore final before it is read.

THE EARLY REJECTIONS ARE NOT JUST OPTIMISATIONS.

If any element exceeds `target` it can never be placed, and the loop would simply never reach the full mask — returning the right answer, but after doing 2^n pointless work. Checking divisibility first also avoids a fractional `target` and a meaningless modulo.

The interview itself, and a mixed final set

The last chapter. What actually happens in the room, how you are scored, and a final set of problems presented without telling you which pattern they use — which is the only realistic test.

What you are actually scored on

```
MOST CANDIDATES BELIEVE THE SCORE IS "did you solve it?"
It is not. A typical rubric has four axes:

  PROBLEM SOLVING   did you make progress systematically?
                    did you use hints well?
  CODING            is the code clean, correct, and does
                    it match what you said?
  VERIFICATION      did you test it yourself, or wait to
                    be told?
  COMMUNICATION     could the interviewer follow your
                    reasoning throughout?

A CANDIDATE WHO REASONS CLEARLY TO AN O(n log n) SOLUTION
USUALLY OUTSCORES ONE WHO SILENTLY TYPES A MEMORISED
O(n) ANSWER. Two of the four axes are not about the
algorithm at all.

------------------------------------------------------------
THE TIMELINE OF A 45-MINUTE INTERVIEW

  0-5    introductions, the problem is stated
  5-10   YOU CLARIFY. Restate it. Ask about constraints,
         edge cases, input size, whether you may modify
         the input.
  10-15  YOU DISCUSS APPROACHES. Brute force first, with
         its complexity. Then improve it. State the plan
         and the target complexity BEFORE coding.
  15-32  YOU CODE, narrating.
  32-40  YOU TEST. Trace the example by hand. Then the
         edge cases.
  40-45  follow-ups, and your questions for them.

THE MOST COMMON TIMING FAILURE is starting to code at
minute 6. The five minutes of clarification are the
highest-value part of the whole interview, and skipping
them is how people solve the wrong problem.

------------------------------------------------------------
WHAT TO SAY WHEN YOU ARE STUCK

Silence is the only genuinely bad option. Say one of:

  "Let me work through a small example by hand."
  "The brute force is O(n^2) because ... - let me look
   for what work is being repeated."
  "n is up to 10^5, so I need about O(n log n). That
   rules out checking every pair."
  "This feels like it might be a sliding window because
   the subarray is contiguous - let me test that on the
   example."
  "I'm stuck between two approaches. Can I talk through
   both?"

ASKING FOR A HINT IS NOT A FAILURE. Using one well -
taking it and running with it - scores better than
grinding silently for ten minutes.

------------------------------------------------------------
THE THINGS THAT LOSE POINTS UNNECESSARILY

  · coding before agreeing the approach
```

```
    · never stating a complexity
    · claiming it works without tracing anything
    · ignoring the empty / single-element / all-equal cases
    · going silent for more than about twenty seconds
    · arguing when told there is a better solution
    · a variable named `x` doing three different jobs

  AND THE ONE THAT SURPRISES PEOPLE: FINDING YOUR OWN BUG
  SCORES POSITIVELY. Tracing your code and saying "this is
  wrong when the array is empty — let me fix it" is a
  strong signal, not a weak one.

  ------------------------------------------------------
  THE EDGE CASES TO CHECK, EVERY TIME

    empty input                []  ""  null
    one element
    two elements
    all identical
    already sorted / reverse sorted
    negatives and zero
    the target absent
    duplicates
    maximum size (does it overflow? is it fast enough?)

  Say them aloud as a list. It takes fifteen seconds and it
  is the cheapest scoring opportunity in the interview.
```

THE ONE-MINUTE OPENING, VERBATIM

```
"So, to make sure I understand: I'm given [restate],
 and I need to return [restate]. A few questions —
 how large can n be? Can the values be negative? Is
 the input sorted? Am I allowed to modify it? And
 what should I return if [edge case]?"

 ... then ...

"The brute force is [X], which is O(n²). I think I
 can do better by [insight] — that would be
 O(n log n). Shall I code that?"
```

That is ninety seconds and it hits three of the four rubric axes before you have written a line.

The pattern cheat sheet

If you see ...	Try ...	Chapter
Sorted array, find a pair	Two pointers	04
Contiguous subarray, max/min/longest	Sliding window	05
Range sums, negatives present	Prefix sums + hash map	06
Max contiguous sum	Kadane	07
Sorted input, O(log n) wanted	Binary search	09
"Minimise the maximum"	Binary search the answer	10
Overlapping ranges	Sort by start or end	12

If you see ...	Try ...	Chapter
"Have I seen this?", counting	Hash map / set	13
Values are 1..n, O(1) space	Cyclic sort / sign marking	14
Pairs cancel, single element	XOR	16
Cycle, middle, nth from end	Fast/slow pointers	18
Matching, nesting, "most recent"	Stack	19
Next greater / smaller element	Monotonic stack	20
Sliding window max	Monotonic deque	21
"Kth largest", "top K", median stream	Heap	22
Sorted order from a tree	Inorder traversal	25
Prefix, autocomplete	Trie	26
Shortest path, unweighted	BFS	27
Dependencies, ordering	Topological sort	28
Connectivity as edges arrive	Union-find	29
Weighted shortest path	Dijkstra	30
"All possible ..."	Backtracking	32
"How many ways", "min cost"	Dynamic programming	33
n ≤ 20, subsets or assignments	Bitmask DP	37

The mixed set

Four problems, **no pattern label**. Work out which technique applies before you start — that identification step is the one you have been practising, and it is the one the interview actually tests. Give each a full twenty minutes before opening the hints.

After the interview

```
/* --- WRITE IT DOWN, THE SAME DAY --- */
    the problem, your approach, where you got stuck,
    what the interviewer nudged you toward, and what you
    would do differently.

    After five interviews this log tells you your actual
    weakness, which is almost never what you assumed. Most
    people discover it is communication or edge cases, not
    algorithms.

/* --- ASK THEM QUESTIONS --- */
    You are also evaluating them. Reasonable questions:
      what does the team own end to end?
      what does the on-call rotation look like?
      how do changes get from a laptop to production?
      what is the most frustrating part of the codebase?

    The last one gets unusually honest answers.

/* --- IF YOU ARE REJECTED --- */
    Ask for feedback; sometimes you get it. Most companies
    allow a reapplication after 6-12 months, and people do
    get in on the second attempt regularly. A rejection is
    a sample of one interviewer on one day, not a
    measurement of your ability.

/* --- AND THE LAST PIECE OF ADVICE --- */

    THE STRONGEST PREDICTOR OF PASSING IS HAVING PRACTISED
    OUT LOUD. Not problem count. Not language. Talking
    through your reasoning while typing is a separate skill
    from solving, it feels unnatural at first, and it is
    almost entirely learnable in a couple of weeks.

    Explain three problems out loud to an empty room before
    your next interview. It is the highest-value hour left
    available to you.                                    */
```

Check yourself

- ✓ Name the four rubric axes and say which two are not about the algorithm.
- ✓ Give the timeline of a 45-minute interview and the most common timing failure.
- ✓ Recite the one-minute opening.
- ✓ Give five things to say when stuck.
- ✓ List the nine edge cases to check every time.
- ✓ Explain why finding your own bug scores positively.
- ✓ Recall ten rows of the pattern cheat sheet from the "if you see" side.
- ✓ Say what the strongest predictor of passing is.

Problems

Mixed 1 — rearrange without division `Medium`

Pattern: Identify it yourself

Given an integer array `nums`, return an array `answer` where `answer[i]` is the product of every element except `nums[i]`.

You may not use division, and it must run in O(n).

Input	Output	Note
`nums = [1,2,3,4]`	`[24,12,8,6]`	
`nums = [-1,1,0,-3,3]`	`[0,0,9,0,0]`	

HINTS

1. What does the answer at index `i` decompose into, if you split the array at `i`?
2. Everything to the left, times everything to the right.
3. Those are a prefix product and a suffix product — the pattern from chapter 06 with × instead of +.
4. Two passes over the output array need no extra storage beyond the answer itself.

Approach	Idea	Time	Space
Total product ÷ each element	Forbidden, and broken by zeros anyway.	`O(n)`	`O(1)`
Recompute per index	Multiply the other n−1 elements each time.	`O(n²)`	`O(1)`
Prefix and suffix passes	Write prefix products into the output, then multiply by a running suffix.	`O(n)`	`O(1) extra`

Solution — js

```
function productExceptSelf(nums) {
  const n = nums.length;
  const out = new Array(n).fill(1);

  // pass 1: out[i] = product of everything LEFT of i
  let prefix = 1;
  for (let i = 0; i < n; i++) {
    out[i] = prefix;
    prefix *= nums[i];
  }

  // pass 2: multiply in the product of everything RIGHT of i
  let suffix = 1;
  for (let i = n - 1; i >= 0; i--) {
    out[i] *= suffix;
    suffix *= nums[i];
  }
  return out;
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> out(n, 1);
        long prefix = 1;
        for (int i = 0; i < n; ++i) { out[i] = prefix; prefix *= nums[i]; }
        long suffix = 1;
        for (int i = n - 1; i >= 0; --i) { out[i] *= suffix; suffix *= nums[i]; }
        return out;
    }
};
```

Solution — python

```python
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        out = [1] * n
        prefix = 1
        for i in range(n):
            out[i] = prefix
            prefix *= nums[i]
        suffix = 1
        for i in range(n - 1, -1, -1):
            out[i] *= suffix
            suffix *= nums[i]
        return out
```

Solution — java

```java
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] out = new int[n];
        int prefix = 1;
        for (int i = 0; i < n; i++) { out[i] = prefix; prefix *= nums[i]; }
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) { out[i] *= suffix; suffix *= nums[i]; }
        return out;
    }
}
```

WALKTHROUGH

THE IDENTIFICATION STEP:

"answer at `i` = something about everything before `i`, combined with something about everything after" is the prefix/suffix signature. Once you name it, the code is two loops.

WRITING BEFORE UPDATING

is what excludes `nums[i]` from its own answer, exactly as in the pivot-index problem in chapter 06.

WHY THE DIVISION SHORTCUT IS BANNED:

with one zero, every other answer needs the product of the non-zero elements; with two zeros, everything is 0. Handling that requires counting zeros as a special case. The two-pass version has no special cases at all — which is the real reason it is the expected answer.

Mixed 2 — longest run of consecutive integers `Medium`

Pattern: Identify it yourself

Given an unsorted array of integers, return the length of the longest sequence of **consecutive** integers present. The elements need not be adjacent in the array.

Must run in O(n).

Input	Output	Note
`nums = [100,4,200,1,3,2]`	`4`	1, 2, 3, 4
`nums = [0,3,7,2,5,8,4,6,0,1]`	`9`	

HINTS

1. Sorting gives it in O(n log n), but the requirement is O(n) — so where does the ordering come from instead?
2. A hash set makes "is `x+1` present?" an O(1) question.
3. The trap is walking a run from every element, which is $O(n^2)$ on a long run.
4. Only start counting from an element that *begins* a run — one with no predecessor in the set.

Approach	Idea	Time	Space
Sort then scan	Sort and measure runs, skipping duplicates. Correct, but O(n log n).	`O(n log n)`	`O(1)`
Set, walk from everywhere	Quadratic: a run of length L is re-walked from each of its members.	`O(n²)`	`O(n)`
Set + run-start detection	Extend only from elements with no predecessor. Each value touched at most twice.	`O(n)`	`O(n)`

Solution — js

```
function longestConsecutive(nums) {
  const s = new Set(nums);
  let best = 0;

  for (const x of s) {
    // skip anything that is NOT the start of a run
    if (s.has(x - 1)) continue;

    let len = 1;
    while (s.has(x + len)) len++;
    best = Math.max(best, len);
  }
  return best;
}
```

Solution — cpp

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int best = 0;
        for (int x : s) {
            if (s.count(x - 1)) continue;
            int len = 1;
            while (s.count(x + len)) ++len;
            best = max(best, len);
        }
        return best;
    }
};
```

Solution — python

```python
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        s = set(nums)
        best = 0
        for x in s:
            if x - 1 in s:
                continue
            length = 1
            while x + length in s:
                length += 1
            best = max(best, length)
        return best
```

Solution — java

```java
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int x : nums) s.add(x);
        int best = 0;
        for (int x : s) {
            if (s.contains(x - 1)) continue;
            int len = 1;
            while (s.contains(x + len)) len++;
            best = Math.max(best, len);
        }
        return best;
    }
}
```

WALKTHROUGH

THE IDENTIFICATION STEP:

"O(n) but the answer depends on order" almost always means a hash set is supplying the ordering implicitly, rather than a sort supplying it explicitly.

THE `CONTINUE` IS THE WHOLE ALGORITHM.

Without it, a shuffled `[1..n]` walks a run from every one of its members, costing `n + (n−1) + … = O(n²)`. With it, the inner loop runs only once per run, so every value is touched at most twice overall — once by the outer iteration, once by the single walk. That amortised argument (chapter 02) is what you must state to justify the O(n) claim.

ITERATING THE SET RATHER THAN THE ARRAY

handles duplicates for free: `[5,5,5,5]` visits 5 once and returns 1.

Mixed 3 — smallest covering substring `Hard`

Pattern: Identify it yourself

Given strings `s` and `t`, return the shortest substring of `s` containing every character of `t`, **including duplicates**. Return the empty string if none exists.

Input	Output	Note
`s = "ADOBECODEBANC", t = "ABC"`	`"BANC"`	
`s = "a", t = "aa"`	`""`	

HINTS

1. "Substring" means contiguous, and you want the shortest valid one — that is a window problem.
2. Grow the right edge until the window is valid, then shrink the left edge while it *stays* valid, recording the length before each shrink.
3. Counts matter, not just presence. Track how many distinct characters currently meet their required count.
4. Increment that counter only when a count becomes *exactly* the requirement; decrement only when it drops *strictly below*.

Approach	Idea	Time	Space
Check every substring	$O(n^2)$ substrings, each validated against the required counts.	`O(n²·m)`	`O(m)`
Sliding window with need/have counts	Expand, then contract while valid; a `formed` counter makes validity O(1).	`O(n + m)`	`O(m)`

Solution — js

```javascript
function minWindow(s, t) {
  if (!t.length || s.length < t.length) return "";

  const need = new Map();
  for (const c of t) need.set(c, (need.get(c) ?? 0) + 1);

  const have = new Map();
  let formed = 0;                 // chars meeting their quota
  let left = 0, bestLen = Infinity, bestStart = 0;

  for (let right = 0; right < s.length; right++) {
    const c = s[right];
    if (need.has(c)) {
      const h = (have.get(c) ?? 0) + 1;
      have.set(c, h);
      if (h === need.get(c)) formed++;      // EXACTLY met
    }

    // shrink WHILE VALID, recording BEFORE each shrink
    while (formed === need.size) {
      if (right - left + 1 < bestLen) {
        bestLen = right - left + 1;
        bestStart = left;
      }
      const d = s[left];
      if (need.has(d)) {
        const h = have.get(d) - 1;
        have.set(d, h);
        if (h < need.get(d)) formed--;      // dropped BELOW
      }
      left++;
    }
  }
  return bestLen === Infinity ? "" : s.slice(bestStart, bestStart + bestLen);
}
```

Solution — cpp

```cpp
class Solution {
public:
    string minWindow(string s, string t) {
        if (t.empty() || s.size() < t.size()) return "";
        unordered_map<char,int> need, have;
        for (char c : t) ++need[c];
        int formed = 0, left = 0, bestLen = INT_MAX, bestStart = 0;
        for (int right = 0; right < (int)s.size(); ++right) {
            char c = s[right];
            if (need.count(c) && ++have[c] == need[c]) ++formed;
            while (formed == (int)need.size()) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1; bestStart = left;
                }
                char d = s[left];
                if (need.count(d) && --have[d] < need[d]) --formed;
                ++left;
            }
        }
        return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
    }
};
```

Solution — python

```python
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not t or len(s) < len(t):
            return ""
        need, have = Counter(t), Counter()
        formed = left = 0
        best_len, best_start = float("inf"), 0
        for right, c in enumerate(s):
            if c in need:
                have[c] += 1
                if have[c] == need[c]:
                    formed += 1
            while formed == len(need):
                if right - left + 1 < best_len:
                    best_len, best_start = right - left + 1, left
                d = s[left]
                if d in need:
                    have[d] -= 1
                    if have[d] < need[d]:
                        formed -= 1
                left += 1
        return "" if best_len == float("inf") else s[best_start:best_start + best_len]
```

Solution — java

```java
class Solution {
    public String minWindow(String s, String t) {
        if (t.isEmpty() || s.length() < t.length()) return "";
        Map<Character,Integer> need = new HashMap<>(), have = new HashMap<>();
        for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);
        int formed = 0, left = 0, bestLen = Integer.MAX_VALUE, bestStart = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (need.containsKey(c)) {
                int h = have.merge(c, 1, Integer::sum);
                if (h == need.get(c)) formed++;
            }
            while (formed == need.size()) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1; bestStart = left;
                }
                char d = s.charAt(left);
                if (need.containsKey(d)) {
                    int h = have.merge(d, -1, Integer::sum);
                    if (h < need.get(d)) formed--;
                }
                left++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
    }
}
```

WALKTHROUGH

THE IDENTIFICATION STEP:

contiguous, plus "shortest satisfying a condition", is the shrinking-window template from chapter 05 — `while (valid) { record; shrink; }`, the mirror image of the "longest" form.

THE TWO CONDITIONS MUST BE ASYMMETRIC.

On the way in, increment `formed` only when a count becomes *exactly* the requirement; using `>=` would count the same character repeatedly. On the way out, decrement only when it falls *strictly below*; a surplus copy leaving invalidates nothing. `"aa"/"aa"` and `"bba"/"ab"` catch the two mistakes respectively.

WHY `FORMED` EXISTS AT ALL.

Comparing the two maps entry by entry inside a loop that already runs n times would make this O(n·m). A single integer reduces the validity test to one comparison, which is what keeps it linear.

Mixed 4 — a valid ordering of dependencies `Medium`

Pattern: Identify it yourself

There are `numCourses` courses labelled `0..numCourses-1`. `prerequisites[i] = [a, b]` means `b` must be taken before `a`.

Return **any valid order** in which all courses can be taken, or an empty array if none exists.

Input	Output	Note
`numCourses = 2, prerequisites = [[1,0]]`	`[0,1]`	
`numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`	`[0,1,2,3]`	[0,2,1,3] is equally valid.
`numCourses = 2, prerequisites = [[1,0],[0,1]]`	`[]`	

HINTS

1. "X before Y" over a set of items is a dependency graph, and the question is asking for a topological order.
2. Build edges from prerequisite to dependant, and count each node's in-degree.
3. Repeatedly take any course whose in-degree has reached zero, and decrement its dependants.
4. If you cannot emit all the courses, the remainder are in a cycle — return an empty array.

Approach	Idea	Time	Space
Try every permutation	Check each ordering for validity.	`O(n!·E)`	`O(n)`
DFS postorder, reversed	Also correct; needs three-colour marking to detect cycles.	`O(V+E)`	`O(V)`
Kahn's algorithm	Peel off zero-in-degree nodes; a short result signals a cycle.	`O(V+E)`	`O(V+E)`

Solution — js

```javascript
function findOrder(numCourses, prerequisites) {
  const g = Array.from({length: numCourses}, () => []);
  const indeg = new Array(numCourses).fill(0);

  // [a, b] means b BEFORE a, so the edge points b -> a
  for (const [a, b] of prerequisites) {
    g[b].push(a);
    indeg[a]++;
  }

  // everything with no outstanding prerequisites
  const order = [];
  for (let i = 0; i < numCourses; i++) if (indeg[i] === 0) order.push(i);

  for (let head = 0; head < order.length; head++) {
    const u = order[head];
    for (const v of g[u]) {
      if (--indeg[v] === 0) order.push(v);   // now unblocked
    }
  }

  // anything never emitted is stuck in a cycle
  return order.length === numCourses ? order : [];
}
```

Solution — cpp

```cpp
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> g(numCourses);
        vector<int> indeg(numCourses, 0);
        for (auto& p : prerequisites) { g[p[1]].push_back(p[0]); ++indeg[p[0]]; }
        vector<int> order;
        for (int i = 0; i < numCourses; ++i) if (!indeg[i]) order.push_back(i);
        for (size_t head = 0; head < order.size(); ++head)
            for (int v : g[order[head]])
                if (--indeg[v] == 0) order.push_back(v);
        if ((int)order.size() != numCourses) return {};
        return order;
    }
};
```

Solution — python

```python
from collections import deque

class Solution:
    def findOrder(self, numCourses: int, prerequisites: list[list[int]]) -> list[int]:
        g = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1

        order = [i for i in range(numCourses) if indeg[i] == 0]
        head = 0
        while head < len(order):
            u = order[head]
            head += 1
            for v in g[u]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    order.append(v)
        return order if len(order) == numCourses else []
```

Solution — java

```java
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) g.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p : prerequisites) { g.get(p[1]).add(p[0]); indeg[p[0]]++; }

        int[] order = new int[numCourses];
        int size = 0;
        for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) order[size++] = i;
        for (int head = 0; head < size; head++)
            for (int v : g.get(order[head]))
                if (--indeg[v] == 0) order[size++] = v;

        return size == numCourses ? order : new int[0];
    }
}
```

WALKTHROUGH

THE IDENTIFICATION STEP:

any problem phrased as "X must come before Y" over a set of items is a topological sort. Build the graph, run Kahn, done — the difficulty is recognising it, not implementing it.

THE EDGE DIRECTION IS THE FIRST THING TO STATE ALOUD.

`[a, b]` means "to take `a`, first take `b`", so `b` is the prerequisite and the edge runs `b → a`. Reversing it produces a backwards order that still passes symmetric test cases, which makes the mistake hard to catch.

USING THE OUTPUT ARRAY AS THE QUEUE

avoids a second data structure: `order` is both the answer being built and the frontier being consumed, indexed by `head`. That also avoids `shift()`, which is O(n) in JavaScript (chapter 21).

THE CYCLE TEST IS THE LENGTH CHECK.

A node in a cycle always has an unemitted prerequisite, so its in-degree never reaches zero and it never enters the order — hence `order.length < numCourses` exactly when a cycle exists.

NOTE THE TESTS EXPECT ONE SPECIFIC ORDERING

even though several are valid; that is an artefact of automated grading. In an interview, say that any valid order is acceptable and that the greedy naturally produces the smallest-index-first one.

TRACK

Debugging & reference

The debugging discipline, the tools on both platforms, and the reference material worth keeping open.

9 chapters

The debugging method

Debugging is not a talent. It is a procedure, and the engineers who look fast at it are following it deliberately while everyone else is guessing.

The loop

```
1. OBSERVE          what ACTUALLY happens, precisely
2. HYPOTHESISE      one falsifiable explanation
3. PREDICT          "if this is true, then X will be
                     true"
4. TEST             the cheapest experiment that
                     distinguishes
5. CONCLUDE         confirmed, or eliminated
6. REPEAT

THE SCIENTIFIC METHOD, applied to a machine that cannot
lie to you.

THE FAILURE MODE THAT WASTES DAYS - "SHOTGUN DEBUGGING":
  change something, rebuild, see if it helps, change
  something else.

Why it fails:
  · you learn NOTHING when it does not help
  · when it DOES help you do not know why, so you cannot
    be sure it is fixed
  · you accumulate unexplained changes, each a future bug
  · and you cannot tell whether you fixed the problem or
    merely moved it

THE DISCIPLINE THAT REPLACES IT:

  CHANGE ONE THING. PREDICT THE RESULT FIRST. WRITE THE
  PREDICTION DOWN.

Predicting first is what converts an experiment into
information. If the result matches, you have confirmed a
model. IF IT DOES NOT, YOU HAVE LEARNED SOMETHING FAR
MORE VALUABLE: your mental model of the system is wrong,
and you now know approximately where.

THE FOUR QUESTIONS THAT START EVERY INVESTIGATION:

  WHAT did you expect to happen?
  WHAT actually happened?
  WHAT is the smallest difference between the working
    case and the broken one?
  WHEN did it last work?

That fourth one is often the whole answer. Something
changed.
```

Reproduce it first

```
/* A BUG YOU CANNOT REPRODUCE IS A BUG YOU CANNOT FIX AND -
   more importantly - CANNOT PROVE YOU FIXED.

   THE PROGRESSION:

   1. REPRODUCE IT AT ALL.
      Even at 1 in 1000. Automate the attempt and let it run
      overnight.

   2. REPRODUCE IT RELIABLY.
      Find the conditions. Then make a script.

   3. MINIMISE THE REPRODUCTION.
      Cut everything that is not required. Halve the input.
      Remove modules. Delete configuration.
      Each cut either preserves the bug (keep cutting) or
      kills it (you just found something relevant).

   4. AUTOMATE IT.
      A one-command reproduction is worth an hour of setup,
      because you will run it fifty times.

/* --- WHEN IT IS INTERMITTENT --- */
    INCREASE THE RATE, don't chase it:
      · run it in a loop, thousands of times
      · add STRESS: other load, memory pressure, thermal
      · vary TIMING: add random delays, change thread
        priorities, run on fewer cores
      · run on the SLOWEST and the FASTEST hardware you
        have - races often appear only at one extreme
      · disable optimisations, or ENABLE them - both change
        timing and both find different bugs
      · use the sanitisers and analysis tools that turn
        latent problems into deterministic failures
        (chapter 04 for Linux, 05 for Windows)

    AND CAPTURE STATE WHEN IT HAPPENS. If it takes six
    hours to reproduce, you cannot afford to catch it and
    learn nothing. Set up the logging, the core dump, the
    trace buffer BEFORE the long run.

/* --- WHEN IT ONLY HAPPENS IN THE FIELD --- */
    · what is different? hardware revision, temperature,
      supply, EMI, network, uptime, data volume, locale
    · get the LOGS and the CONFIGURATION
    · ask what the user was actually doing, not what they
      think they were doing
    · and consider whether your development environment is
      hiding it: a debugger attached changes timing, a
      development build has different optimisation, a bench
      supply is cleaner than a battery.               */
```

Bisection

```
/* THE MOST POWERFUL DEBUGGING TECHNIQUE, IN ANY DIMENSION.
   Halve the search space repeatedly: 1000 candidates becomes
   10 tests.                                                  */

/* --- BISECT IN TIME: git bisect --- */
    git bisect start
    git bisect bad                # current commit fails
    git bisect good v1.2.0        # this one worked
    # git checks out the midpoint; test and mark it
    git bisect good     # or: git bisect bad
    # ... repeat ~log2(n) times
    git bisect reset

    AUTOMATED, which is how you should do it:
      git bisect run ./test.sh
      # exit 0 = good, 1 = bad, 125 = skip (cannot build)

    10,000 commits -> 14 tests. If your test takes a minute,
    that is a quarter of an hour to find the exact commit
    that introduced a bug, unattended.

/* --- BISECT IN SPACE: the code --- */
    Comment out half the initialisation. Still broken?
    The cause is in the other half.
    Crude, and extremely effective.

/* --- BISECT IN CONFIGURATION --- */
    A working config and a broken one that differ in forty
    settings: change twenty, test, then ten. Six tests.

/* --- BISECT IN DATA --- */
    A parser crashes on a 10 MB file. Halve it. Still
    crashes? Halve again. You will have a 40-byte
    reproduction in twenty steps.
    (And this is exactly what a fuzzer's test-case minimiser
    does automatically.)

/* --- BISECT IN HARDWARE --- */
    Two boards, one works. Swap the module. Swap the cable.
    Swap the power supply. Swap the SD card. Each swap
    halves the space.

/* --- BISECT ACROSS THE STACK --- */
    Is the problem above or below this layer? Instrument the
    boundary. Application, library, syscall, driver,
    hardware register, bus, device.

    THE SINGLE MOST USEFUL QUESTION IN EMBEDDED DEBUGGING:

      DID THE BYTES ACTUALLY GO OUT ON THE WIRE?

    Put a logic analyser or a scope on it and find out.
    Half of all "the driver is broken" investigations end
    at "the pin was never configured as an output" or "the
    device NAKed and nobody checked".                     */
```

READ THE ERROR MESSAGE

Actually read it. All of it. Then read the line before it, and the line after. An enormous fraction of debugging time is spent on problems whose exact cause, file and line number were printed on screen and skimmed past. The same applies to the compiler's *first* error — the ones after it are usually consequences, and fixing the first often fixes forty.

Question the assumptions

```
/* WHEN YOU ARE GENUINELY STUCK, THE BUG IS IN SOMETHING YOU
   ARE CERTAIN ABOUT. That is definitionally where it must
   be - you have already checked everything you doubt.

   SO GO AND VERIFY THE CERTAINTIES:

   "The code I'm editing is the code that's running."
     -> Is it? Check the timestamp. Check the build actually
        succeeded. Check you flashed the right image to the
        right board. Add a deliberate crash and confirm it
        crashes. THIS IS NOT A JOKE - it is one of the most
        common causes of an impossible bug.

   "The compiler generated what I wrote."
     -> Read the disassembly (chapter 03, and the compilers
        track). Undefined behaviour licenses the optimiser
        to do surprising things, and a `volatile` you forgot
        means your register read was hoisted out of the loop.

   "This function returns what I think."
     -> Check the return value. Every time. The number of
        bugs that are an unchecked error code is
        extraordinary.

   "The hardware works."
     -> Usually true, and not always. Check the supply rail
        under load. Check the crystal is oscillating. Check
        the reset line. Check the part number matches the
        datasheet you are reading. Check the errata sheet -
        vendors publish them because the silicon has bugs.

   "The data is what I expect."
     -> Print it. In hex. With its length. Endianness,
        alignment, sign extension, off-by-one, and a
        stray 0x0D from a Windows line ending have all
        cost people days.

   "The clock is configured."
     -> A peripheral with no clock reads back zeros or
        retains defaults, and writes silently do nothing.
        A gated clock is the single most common cause of
        "the peripheral doesn't respond".

   "The other side is doing its part."
     -> Instrument the boundary and confirm.

/* AND THE META-ASSUMPTION:

     "This cannot possibly be related."

   Note it anyway. Coincidences in a deterministic machine
   are usually causation you have not traced yet.       */
```

Tactics that work

```
/* --- RUBBER DUCK DEBUGGING --- */
    Explain the problem, out loud, in complete sentences, to
    something that cannot help you. The bug frequently
    surfaces mid-sentence, because articulating forces you
    to state assumptions you had been skipping over.
    THIS SOUNDS SILLY AND IT WORKS.

/* --- WRITE IT DOWN --- */
    A running log of what you tried, what you predicted, and
    what happened. Three benefits: you stop repeating
    experiments, you can hand it to someone else, and the
    act of writing forces precision.

/* --- CHANGE ONE THING --- */
    Two changes at once and a fixed bug tells you nothing.

/* --- REVERT TO A KNOWN-GOOD STATE --- */
    When you have made twenty changes and it is worse than
    when you started: revert everything, reproduce, and
    start again with what you have learned. Sunk cost is
    not a debugging strategy.

/* --- TAKE A BREAK --- */
    Genuinely effective, for a real reason: you get stuck in
    a search path and stop generating new hypotheses. A
    walk, or sleep, resets it. Many hard bugs are solved in
    the first ten minutes of the next morning.

/* --- ASK SOMEONE --- */
    Not because they know more, but because explaining it
    (see rubber duck) plus one outside assumption is often
    all it takes. Set a time limit on solo effort - two
    hours, say - and then ask.

/* --- FIND THE SIMPLEST THING THAT FAILS --- */
    Strip until it works, then add back until it breaks.
    The last thing you added is where to look.

/* --- AND WHEN YOU FIX IT --- */
    1. UNDERSTAND WHY THE FIX WORKS. A change that makes the
       symptom disappear for a reason you cannot state is
       not a fix - it is a timing change that will come
       back.
    2. WRITE A TEST that fails without it.
    3. LOOK FOR THE SAME BUG ELSEWHERE. Bugs come in
       families; the same mistake is usually in three other
       places, made by the same person on the same
       afternoon.
    4. ASK WHY IT WAS POSSIBLE. Could a type, an assertion,
       a lint rule, or an API change make this class of bug
       unrepresentable? That question is worth more than the
       fix.
    5. WRITE IT DOWN somewhere the next person will find
       it.                                                */
```

How it shows up

Symptom	What it usually means
"It works on my machine"	An environment difference — enumerate them
"It's random"	It is not. Uninitialised memory, a race, or an input you have not identified
Adding a `printf` makes it go away	A race, or a timing-dependent bug — a heisenbug
Works in debug, fails in release	Undefined behaviour, uninitialised memory, or a missing `volatile`
Fails only under the debugger	Timing, or a breakpoint changing interrupt latency
Fixed itself after a rebuild	It did not. A stale object file, or the bug moved
Impossible state observed	Memory corruption, a stack overflow, or the wrong binary is running
Only fails after hours	A leak, a counter wrap, or fragmentation
Only fails on one unit	Hardware, or a per-unit configuration or calibration value
Only fails when cold or hot	Timing margin, crystal drift, or a marginal supply
Peripheral reads back all zeros	Clock not enabled

Check yourself

- ✓ State the six steps of the debugging loop.
- ✓ Explain why predicting before testing is what makes an experiment informative.
- ✓ Give the four opening questions.
- ✓ Describe the four stages from "reproduce at all" to "automate it".
- ✓ List six ways to raise the reproduction rate of an intermittent bug.
- ✓ Run a `git bisect`, including the automated form.
- ✓ Give five dimensions other than time that you can bisect in.
- ✓ Name six "certainties" that are worth verifying when stuck.
- ✓ Explain why a peripheral reading back zeros usually means a clock problem.
- ✓ Give the five things to do after a fix works.

Print-based debugging and tracing

The most-used debugging technique in systems programming, and the one most often done badly. A well-designed trace answers questions a debugger cannot — because it does not stop time.

Why printing beats breakpoints

```
A DEBUGGER STOPS THE WORLD. That is exactly wrong for:

  · REAL-TIME systems - stop for 200 ms and the motor
    controller has already faulted, the watchdog has
    fired, or the USB host has dropped the device
  · INTERRUPT HANDLERS - a breakpoint in an ISR changes
    the latency you are trying to measure
  · RACES - stopping one thread changes the interleaving,
    and the bug disappears
  · PROTOCOL TIMING - the peer times out while you look
    at a variable
  · PRODUCTION - you cannot attach a debugger to a
    customer's device in the field
  · and ANY BUG THAT TAKES HOURS TO APPEAR - you cannot
    single-step for six hours

A TRACE RECORDS WHAT HAPPENED AND LETS THE SYSTEM RUN.
You get the SEQUENCE, which is usually the actual
question:

  what happened before the failure?
  in what ORDER did these events occur?
  how LONG did that take?
  how many times did that path execute?
  what did the state look like over TIME?

A DEBUGGER ANSWERS "what is true NOW". A TRACE ANSWERS
"what happened, and in what order". MOST BUGS ARE
SEQUENCE BUGS.

THE OTHER SIDE OF THE TRADE, honestly:
  · a trace only shows what you thought to record
  · you must rebuild and re-run to add a field
  · printing PERTURBS TIMING - the classic heisenbug,
    where adding a printf makes the race disappear
  · and it costs code space and bandwidth

USE BOTH. A debugger to inspect a stopped state
(chapter 03); a trace to understand a sequence.
```

What a good trace line contains

```
/* BAD - and every codebase is full of these: */
    printf("here\n");
    printf("value = %d\n", x);
    printf("error!\n");

/* Useless in a log with a thousand lines from twelve
   modules, and completely useless six months later.

/* GOOD: */
    [00123.456] [INFO ] [i2c    ] i2c.c:214 xfer addr=0x50
                len=32 -> OK (1.2ms)
    [00123.458] [WARN ] [sensor ] sen.c:88  temp=87.5C
                exceeds limit 85.0C
    [00123.461] [ERROR] [net    ] net.c:512 tx failed
                errno=110 (ETIMEDOUT) retry=3/5

/* THE SEVEN FIELDS THAT EARN THEIR SPACE:

   1. TIMESTAMP. Monotonic, high resolution. THE SINGLE
      MOST VALUABLE FIELD - it gives you ordering and
      duration for free. Use a monotonic clock, never
      wall-clock time, which jumps.

   2. LEVEL. So you can filter without rebuilding.

   3. SUBSYSTEM. So you can filter to one module.

   4. LOCATION. File and line, or the function name.
      __FILE__, __LINE__, __func__ cost nothing to add.

   5. THE OPERATION, named in a way you can grep for.

   6. THE VALUES, with UNITS and in a consistent radix.
      "temp=87.5C" not "temp is high". "addr=0x50" not
      "addr=80".

   7. THE OUTCOME, including the ERROR CODE AND ITS NAME.
      "errno=110 (ETIMEDOUT)" saves the reader a lookup.

/* --- THE PATTERN THAT MAKES IT GREPPABLE --- */
    Log KEY=VALUE pairs, consistently. Then:

      grep 'i2c' log.txt | grep 'ERROR'
      awk '/xfer/ {print $NF}' log.txt | sort | uniq -c

    Structured logging (JSON lines) goes further and is
    worth it on Linux-class systems. On an MCU, consistent
    key=value text is the right level.                    */
```

The logging layer

```
/* ------------------------------------------------------
 * log.h - a complete, compile-time-filtered logging layer
 * ------------------------------------------------------ */
#ifndef LOG_H
#define LOG_H

#include <stdint.h>

#define LOG_LEVEL_NONE   0
#define LOG_LEVEL_ERROR  1
#define LOG_LEVEL_WARN   2
#define LOG_LEVEL_INFO   3
#define LOG_LEVEL_DEBUG  4
#define LOG_LEVEL_TRACE  5

/* Per-module override: define LOG_LEVEL before including,
   so one noisy driver can be turned up without drowning in
   everything else. */
#ifndef LOG_LEVEL
#define LOG_LEVEL LOG_LEVEL_INFO
#endif

#ifndef LOG_TAG
#define LOG_TAG "app"
#endif

uint32_t log_time_ms(void);
void log_write(const char *fmt, ...);

/* THE do/while(0) WRAPPER: makes the macro a single
   statement, so `if (x) LOG_INFO(...); else ...` compiles
   correctly (see the C track). */
#define LOG_AT(lvl, tag_str, fmt, ...)                    \
    do {                                                  \
        log_write("[%8lu] [%-5s] [%-7s] %s:%d " fmt "\n", \
                  (unsigned long)log_time_ms(), tag_str,  \
                  LOG_TAG, __FILE__, __LINE__,            \
                  ##__VA_ARGS__);                         \
    } while (0)

/* COMPILED OUT ENTIRELY when the level is too low - the
   string literals do not even reach the binary, which
   matters when flash is 256 KB. */
#if LOG_LEVEL >= LOG_LEVEL_ERROR
#define LOG_ERROR(f, ...) LOG_AT(1,"ERROR",f,##__VA_ARGS__)
#else
#define LOG_ERROR(f, ...) ((void)0)
#endif

#if LOG_LEVEL >= LOG_LEVEL_WARN
#define LOG_WARN(f, ...)  LOG_AT(2,"WARN", f,##__VA_ARGS__)
#else
#define LOG_WARN(f, ...)  ((void)0)
#endif

#if LOG_LEVEL >= LOG_LEVEL_INFO
#define LOG_INFO(f, ...)  LOG_AT(3,"INFO", f,##__VA_ARGS__)
#else
#define LOG_INFO(f, ...)  ((void)0)
#endif
```

```
#if LOG_LEVEL >= LOG_LEVEL_DEBUG
#define LOG_DEBUG(f, ...) LOG_AT(4,"DEBUG",f,##__VA_ARGS__)
#else
#define LOG_DEBUG(f, ...) ((void)0)
#endif

#endif /* LOG_H */

/* --- USE --- */
#define LOG_TAG   "i2c"
#define LOG_LEVEL LOG_LEVEL_DEBUG
#include "log.h"

int i2c_xfer(uint8_t addr, const uint8_t *buf, size_t len)
{
    uint32_t t0 = log_time_ms();
    LOG_DEBUG("xfer addr=0x%02X len=%u", addr,
              (unsigned)len);

    int rc = hw_xfer(addr, buf, len);
    if (rc != 0) {
        LOG_ERROR("xfer addr=0x%02X failed rc=%d (%s)",
                  addr, rc, i2c_strerror(rc));
        return rc;
    }
    LOG_DEBUG("xfer addr=0x%02X ok (%lums)", addr,
              (unsigned long)(log_time_ms() - t0));
    return 0;
}

/* --- A HEX DUMP, which you will want constantly --- */
void log_hexdump(const char *label, const void *data,
                 size_t len)
{
    const uint8_t *p = data;
    LOG_DEBUG("%s (%u bytes):", label, (unsigned)len);
    for (size_t i = 0; i < len; i += 16) {
        char line[80];
        int n = snprintf(line, sizeof line, "  %04X: ",
                         (unsigned)i);
        for (size_t j = 0; j < 16 && i + j < len; j++)
            n += snprintf(line + n, sizeof line - n,
                          "%02X ", p[i + j]);
        /* pad, then the ASCII column */
        while (n < 8 + 48)
            line[n++] = ' ';
        for (size_t j = 0; j < 16 && i + j < len; j++) {
            uint8_t c = p[i + j];
            line[n++] = (c >= 32 && c < 127) ? (char)c : '.';
        }
        line[n] = '\0';
        LOG_DEBUG("%s", line);
    }
}
```

NEVER LOG A SECRET

Keys, passwords, tokens, personal data. A UART pad on the board is accessible to anyone with a soldering iron, and a log file gets emailed to support (cryptography chapter 18). Log a *fingerprint* — the first four bytes of a hash — if you need to confirm which key is in use. Add a review check for it; this is one of the most common ways a shipped product leaks its own credentials.

When printing is too slow

```
/* A 115200-BAUD UART CARRIES ~11 KB/S. One 80-character
   line takes 7 MILLISECONDS. In an interrupt handler that
   runs every 100 microseconds, that is not debugging - it
   is a denial of service against your own system.

/* --- 1. RING BUFFER, DEFERRED OUTPUT --- */
    The ISR writes into a RAM buffer and returns. The idle
    loop drains it to the UART.

    static char logbuf[4096];
    static volatile uint32_t head, tail;

    void log_put(const char *s, size_t n) {
        /* single producer here; for multiple producers use
           an atomic head increment (see the OS track) */
        for (size_t i = 0; i < n; i++) {
            uint32_t next = (head + 1) % sizeof logbuf;
            if (next == tail) break;   /* FULL: DROP.
                                          Never block. */
            logbuf[head] = s[i];
            head = next;
        }
    }
    /* AND COUNT THE DROPS. A log that silently discards
       lines will mislead you at the worst moment; print
       "[dropped 412]" when it recovers. */

/* --- 2. BINARY / DEFERRED-FORMAT LOGGING --- */
    Do not format on the device. Log an EVENT ID and raw
    arguments; a host tool formats them using the same
    string table extracted from the ELF.

      · a trace point becomes ~8 bytes instead of 80
      · no snprintf on the target - which is often the
        single largest and slowest function in a small
        firmware image
      · format strings live in the ELF, not in flash

    THIS IS WHAT SEGGER RTT + SystemView, TraceX, Zephyr's
    dictionary logging and the Linux kernel's tracepoints
    all do.

/* --- 3. SEGGER RTT --- */
    Uses the debug probe's memory access to read a RAM ring
    buffer WHILE THE TARGET RUNS. No UART, no pins,
    megabytes per second, and microseconds per call. IF YOU
    HAVE A J-LINK, USE IT - it removes the timing
    perturbation almost entirely.

/* --- 4. ITM / SWO --- */
    ARM Cortex-M's instrumentation trace: write a byte to a
    register, the trace unit streams it out the SWO pin.
    Single-cycle cost.

      ITM_SendChar(c);          /* CMSIS */

    32 stimulus ports, so you can separate channels. Needs
    the SWO pin and a probe that supports it.

/* --- 5. A GPIO PIN AND A SCOPE --- */
```

```
    THE FASTEST INSTRUMENTATION THAT EXISTS, and frequently
    the best:

      GPIOA->BSRR = PIN;      /* one store */
      do_the_thing();
      GPIOA->BSRR = PIN << 16;

    A few nanoseconds. Put a scope or logic analyser on it
    and you get EXACT timing, jitter, and the relationship
    to other signals - things no software trace can give
    you. Toggle a different pin in each ISR and you can SEE
    your system's scheduling.

    THIS IS THE TECHNIQUE THAT SEPARATES EMBEDDED DEBUGGING
    FROM APPLICATION DEBUGGING. Use it.

/* --- 6. TRACE BUFFER IN NON-VOLATILE MEMORY --- */
    For crashes: a small circular buffer in a RAM region
    excluded from initialisation, or in flash. After a
    watchdog reset, dump the last events that preceded it.
    Combined with a reset-reason register, this is how you
    debug field failures you cannot reproduce.          */
```

Tracing on Linux and Windows

```
/* --- LINUX KERNEL --- */
    printk(KERN_INFO "mydrv: probe %s\n", dev_name(dev));
    dev_info(dev, "probe ok\n");        /* PREFERRED -
                                           includes the
                                           device name */
    dev_err(dev, "failed: %d\n", ret);
    dev_dbg(dev, "reg=0x%08x\n", val);  /* dynamic debug */

    /* enable dev_dbg at runtime, no rebuild: */
    # echo 'module mydrv +p' > \
        /sys/kernel/debug/dynamic_debug/control
    # echo 'file mydrv.c line 100-200 +p' > \
        /sys/kernel/debug/dynamic_debug/control

    /* TRACEPOINTS - near-zero cost when disabled */
    # cd /sys/kernel/tracing
    # echo 1 > events/irq/enable
    # cat trace_pipe

    /* trace_printk() for temporary work: goes to the ftrace
       ring buffer instead of the console, so it is
       thousands of times cheaper than printk. Remove it
       before you submit anything. */

    /* AND THE BIG ONES, covered in chapter 04:
       ftrace, perf, eBPF/bpftrace, LTTng. */

/* --- WINDOWS KERNEL --- */
    DbgPrintEx(DPFLTR_IHVDRIVER_ID, DPFLTR_INFO_LEVEL,
               "mydrv: probe\n");

    /* WPP SOFTWARE TRACING is the production answer: */
    DoTraceMessage(TRACE_LEVEL_INFORMATION,
                   "xfer addr=%!HEXDUMP! len=%d", ...);

    · formatted OFF the device by a trace consumer
    · negligible runtime cost
    · always compiled in, enabled at runtime
    · integrated with ETW (Event Tracing for Windows), so
      it correlates with the rest of the system

    Chapter 05 covers the tooling.

/* --- THE COMMON PRINCIPLE ACROSS ALL THREE PLATFORMS --- */
    THE BEST TRACING SYSTEMS ARE ALWAYS COMPILED IN,
    NEGLIGIBLE WHEN DISABLED, AND ENABLED AT RUNTIME.

    A debug build you have to install to diagnose a problem
    is not available when you need it - the problem is
    happening on a production unit, now. Design so that the
    instrumentation is already there and only needs turning
    on.                                                    */
```

How it shows up

Symptom	Cause
Adding a `printf` makes the bug vanish	Timing perturbation — use RTT, ITM or a GPIO

Symptom	Cause
System misses deadlines with logging on	Blocking UART writes — defer to a ring buffer
Log stops partway through a failure	Buffer full and dropping, or the crash beat the drain
Log lines interleaved and corrupted	Multiple producers with no locking or atomic head
Cannot tell what order events occurred in	No timestamps, or wall-clock time that jumped
Flash full after adding logging	Format strings — use deferred-format or dictionary logging
Cannot reproduce in a debug build	The bug is timing-dependent — trace instead
Field unit fails, no information	No persistent trace buffer across reset
Credentials found in a support log	Secrets logged — log a fingerprint instead
Log too noisy to read	No per-module levels or subsystem tags

Check yourself

- ✓ Give six situations where a debugger is the wrong tool.
- ✓ State what a trace answers that a stopped debugger cannot.
- ✓ List the seven fields of a good trace line and what each is for.
- ✓ Explain why `do { } while (0)` wraps a logging macro.
- ✓ Explain compile-time level filtering and why it matters on an MCU.
- ✓ Compute how long an 80-character line takes at 115200 baud.
- ✓ Describe deferred output and why dropped lines must be counted.
- ✓ Explain deferred-format logging and what it saves.
- ✓ Say why a GPIO toggle and a scope is sometimes the best instrument.
- ✓ Enable a Linux `dev_dbg` at runtime without rebuilding.
- ✓ State the principle shared by ftrace, WPP and RTT.

JTAG, SWD and hardware debug

A side channel into the processor that works before any software runs. When the board is dead, the boot loops, or the fault handler fires before a single character reaches the UART, this is the only tool you have.

What the debug port is

```
THE DEBUG PORT IS HARDWARE INSIDE THE CHIP, independent
of the CPU's execution. It can:

  · HALT and RESUME the core
  · READ AND WRITE registers and memory - THROUGH THE
    BUS, so it works even when the CPU is halted or
    stuck
  · SET BREAKPOINTS in flash (hardware comparators)
  · SET WATCHPOINTS on data addresses
  · STEP single instructions
  · RESET the chip, and HALT BEFORE THE FIRST
    INSTRUCTION
  · program flash
  · and stream TRACE

THAT LAST-BUT-ONE POINT IS THE IMPORTANT ONE: "connect
under reset" lets you attach to a chip whose firmware
immediately disables the debug pins, sleeps, or crashes.
It is how you recover a board you have bricked with your
own code.

JTAG (IEEE 1149.1) - the original, 4-5 pins:
  TCK   clock
  TMS   mode select (drives a state machine)
  TDI   data in
  TDO   data out
  TRST  reset (optional)
DAISY-CHAINABLE: several devices on one chain, which is
why it is still used on FPGAs and multi-chip boards. It
also does BOUNDARY SCAN - toggling and reading individual
package pins to test solder joints, independently of the
CPU.

SWD (Serial Wire Debug) - ARM's 2-pin replacement:
  SWDIO  bidirectional data
  SWCLK  clock
  (+ SWO, optional, for trace output)

SAME CAPABILITY, TWO PINS. Universal on Cortex-M, and
what you will use in practice.

THE CONNECTOR, and it matters when the board arrives:
  10-pin 1.27mm Cortex Debug     the modern standard
  20-pin 2.54mm ARM JTAG         older, bulky
  TC2050 "tag-connect"           no connector at all -
                                 spring pins on pads.
                                 COSTS NOTHING IN BOM
                                 AND SAVES YOU EVERY
                                 TIME.

PUT A DEBUG CONNECTOR ON EVERY BOARD YOU DESIGN. Even
production ones - use tag-connect pads, and disable the
port in firmware for shipping units (see below).
```

Probes and the software stack

```
/* --- THE PROBES --- */
    SEGGER J-Link       the reference. Fast flash
                        programming, RTT (chapter 02),
                        excellent target support. The EDU
                        version is inexpensive for
                        non-commercial use.
    ST-LINK             on every Nucleo and Discovery
                        board. Fine for STM32.
    CMSIS-DAP / DAPLink open standard, on many dev boards.
                        Slower, universal.
    Black Magic Probe   runs the GDB SERVER ON THE PROBE -
                        connect GDB directly to a serial
                        port, no OpenOCD needed.
    Vendor probes       Lauterbach TRACE32 and iSYSTEM at
                        the high end: full instruction
                        trace, expensive, and worth it for
                        a team debugging hard timing
                        problems.

/* --- THE SOFTWARE STACK --- */

    GDB  <--TCP:3333-->  OpenOCD / JLinkGDBServer
                              |
                              v  (USB)
                           PROBE
                              |
                              v  (SWD/JTAG)
                           TARGET

    THE GDB SERVER translates GDB's remote protocol into
    probe commands. Understanding that there are three
    separate pieces is most of what makes connection
    failures diagnosable.

/* --- OPENOCD --- */
    $ openocd -f interface/stlink.cfg \
              -f target/stm32f4x.cfg

    Open On-Chip Debugger. Supports hundreds of targets and
    dozens of probes. Its telnet interface (port 4444) is
    useful on its own:

      $ telnet localhost 4444
      > reset halt
      > mdw 0x40021000 4       # read 4 words
      > mww 0x40020014 0x20    # write a word
      > flash write_image erase firmware.elf
      > reg                    # dump all registers

    READING AND WRITING PERIPHERAL REGISTERS BY HAND, with
    the CPU halted, is an extremely direct way to answer
    "is this peripheral configured the way I think?"

/* --- A CONNECTION THAT FAILS --- */
    "Error: init mode failed (unable to connect to the
     target)"

    THE CHECKLIST, in order:
      1. POWER. Is the target powered? Measure it.
      2. GROUND. Is the probe's ground connected? A missing
```

```
   ground is the most common cause.
3. WIRING. SWDIO and SWCLK to the right pins, not
   swapped.
4. RESET. Try "connect under reset" - firmware may be
   disabling the debug pins or entering sleep.
5. PIN REUSE. Did your code reconfigure PA13/PA14 as
   GPIO? That kills SWD. Connect under reset and
   erase.
6. LOW POWER MODE. Deep sleep gates the debug clock.
   Set the DBGMCU standby/stop bits to keep it alive.
7. READOUT PROTECTION. If RDP is set, the port is
   disabled by design. A mass erase may be the only
   way back - and at RDP level 2, there is no way
   back.
8. TARGET SPEED. Drop the adapter clock to 100 kHz
   and retry; long wires and slow crystals both
   matter.
9. CLOCK. Is the target's crystal actually
   oscillating? Scope it.                          */
```

GDB, the parts you need

```
$ arm-none-eabi-gdb firmware.elf
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt     # "monitor" passes to OpenOCD
(gdb) load                   # flash the image
(gdb) break main
(gdb) continue

# ---- BREAKPOINTS ----
break main                   # by function
break i2c.c:214              # by file:line
break *0x08001234            # by address
break i2c_xfer if addr==0x50 # CONDITIONAL - and note it
                             # is evaluated by GDB, so it
                             # halts EVERY time and is slow
tbreak setup                 # temporary, one shot
info breakpoints
delete 2

# NOTE: A CORTEX-M HAS ONLY 4-8 HARDWARE BREAKPOINTS,
# because code is in flash and cannot be patched with a
# software trap. Exceed the limit and GDB reports a failure
# to set one. Delete the ones you are not using.

# ---- WATCHPOINTS: the most underused feature ----
watch g_state               # break when it is WRITTEN
rwatch g_config             # when READ
awatch g_flag               # either
watch *(uint32_t*)0x20001000

# "SOMETHING IS CORRUPTING THIS VARIABLE AND I CANNOT FIND
# WHAT" IS EXACTLY WHAT A WATCHPOINT IS FOR. Set it, run,
# and the debugger stops at the instruction that did it and
# shows you the backtrace. This turns a two-day bug into a
# two-minute one, and most people never use it.
#
# Hardware watchpoints are also limited (typically 4) and
# they watch an ADDRESS RANGE, so watching a whole struct
# may need several.

# ---- EXECUTION ----
continue / c
next / n           # step over calls
step / s           # step into
finish             # run until the current function returns
until 250          # run until line 250

# ---- STATE ----
print var
print/x reg                 # hex
print *ptr
print arr[0]@16             # ARTIFICIAL ARRAY: 16 elements
                            # from arr[0]. Extremely useful.
print sizeof(struct foo)
x/16xb 0x20000000           # 16 hex bytes
x/8xw 0x40021000            # 8 hex words - a peripheral
x/i $pc                     # the instruction at PC
x/20i $pc                   # disassemble forward
info registers
info locals
info args
```

```
backtrace / bt
bt full                     # with locals at each frame
frame 2                     # switch frames
set var x = 5               # MODIFY state and continue

# ---- WHEN THERE IS NO BACKTRACE ----
# After a hard fault the stack may be unwindable only
# manually. See chapter 06 for reading the fault registers
# and the stacked frame.

# ---- SCRIPTING: put this in .gdbinit ----
define reload
  monitor reset halt
  load
  monitor reset halt
end

define regs
  printf "PC=%08x LR=%08x SP=%08x\n", $pc, $lr, $sp
end

# ---- TUI ----
# Ctrl-X A, or start with `gdb -tui`. Source, registers and
# disassembly in one terminal. layout split, layout regs.
```

THE THREE THINGS A DEBUGGER DOES THAT NOTHING ELSE CAN

```
1. WATCHPOINTS. Catch the instruction that corrupts
   a variable, with a backtrace.
2. CONNECT UNDER RESET. Reach a chip whose firmware
   is hostile to debugging — including your own
   bricked board.
3. READ PERIPHERAL REGISTERS ON A HALTED CORE.
   Answer "is this actually configured?" directly,
   without trusting any code you wrote.
```

Trace, and what it buys

```
/* ARM CoreSight, the units and what each is for: */

    DWT   Data Watchpoint and Trace
            · a CYCLE COUNTER - free, precise timing
            · watchpoints
            · PC sampling for a statistical profile
            · exception tracing
    ITM   Instrumentation Trace Macrocell
            · software-generated messages, single-cycle
              cost (chapter 02)
    ETM   Embedded Trace Macrocell
            · FULL INSTRUCTION TRACE. Every instruction
              executed, reconstructible offline.
    TPIU  Trace Port Interface Unit
            · streams it out, over SWO (1 pin, slow) or a
              4-bit parallel TRACE port (fast, needs a
              proper probe)

/* --- THE CYCLE COUNTER: use this constantly --- */
    /* enable once */
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL  |= DWT_CTRL_CYCCNTENA_Msk;

    /* measure anything */
    uint32_t t0 = DWT->CYCCNT;
    function_under_test();
    uint32_t cycles = DWT->CYCCNT - t0;
    /* the subtraction is correct across wraparound,
       for unsigned arithmetic (see the C track) */

    NANOSECOND-RESOLUTION TIMING, no probe required, a few
    lines of setup. IF YOU MEASURE ANYTHING ON A CORTEX-M,
    MEASURE IT WITH THIS.

/* --- FULL INSTRUCTION TRACE (ETM) --- */
    Records every branch, so a host tool can reconstruct the
    exact execution path - thousands of instructions before
    a crash, with no instrumentation and no timing
    perturbation.

    WHAT IT SOLVES THAT NOTHING ELSE DOES:
      · "how did we get to this address?"
      · a corrupted stack with no backtrace
      · a race whose interleaving you cannot otherwise see
      · code coverage on the real target
      · exact cycle-accurate profiling

    THE COST: a probe that supports it (J-Trace,
    Lauterbach, iSYSTEM) and a board that routes the trace
    pins. ROUTE THEM. Adding four pins at schematic time
    costs nothing; adding them after a failure costs a board
    spin.

/* --- ETW/perf EQUIVALENTS --- */
    On Linux and Windows you have perf and ETW instead
    (chapters 04, 05) - same idea, done in software with
    hardware performance counters.                        */
```

Debug ports and security

```
/* A DEBUG PORT IS COMPLETE ACCESS TO THE DEVICE. Read all
   memory, read the keys, modify execution, dump the
   firmware. On a shipped product that is a serious
   vulnerability, and leaving it open is one of the most
   common findings in any hardware teardown (see the
   cryptography track, chapter 21).

/* --- THE LIFECYCLE APPROACH --- */
    DEVELOPMENT   debug open, test keys only
    PRODUCTION    debug DISABLED via a one-way fuse or a
                  protection level, production keys
                  provisioned
    RMA           if debug can be re-enabled at all, THE
                  TRANSITION MUST ERASE ALL SECRETS FIRST

/* --- THE MECHANISMS, by vendor --- */
    STM32          RDP levels 0/1/2. Level 2 is PERMANENT -
                   there is no way back, and no way to
                   debug a returned unit.
    nRF            APPROTECT. Note that early revisions were
                   defeated by a voltage glitch, which is a
                   useful reminder that a protection bit is
                   only as strong as its enforcement
                   (cryptography chapter 19).
    ARMv8-M /
    TrustZone-M    debug authentication with SEPARATE
                   secure and non-secure permissions - you
                   can allow non-secure debug while keeping
                   the secure world closed. This is the
                   right model.
    ADAC           an ARM standard for AUTHENTICATED debug:
                   the port stays physically present but
                   requires a signed challenge-response to
                   unlock, so the vendor can debug a
                   returned unit and an attacker cannot.

/* --- AND THE PRACTICAL ADVICE --- */
      · DO put a debug connector on production boards -
        tag-connect pads cost nothing
      · DO disable the port in the production firmware
        configuration
      · DO verify on an actual production unit that it is
        disabled. "We set the fuse in the build script" is
        not evidence; probing a shipping unit is.
      · DO plan the RMA path before you ship, and make it
        erase
      · and prefer AUTHENTICATED debug over permanent
        lockout if the silicon supports it - a fleet you
        cannot diagnose is its own kind of problem.    */
```

How it shows up

Symptom	Cause
Cannot connect to the target	Work the nine-step checklist — ground and connect-under-reset first
Connects once, then never again	Firmware reconfigured the SWD pins — connect under reset and erase
Connection drops when the device sleeps	Debug clock gated — set the DBGMCU low-power bits

Symptom	Cause
"Cannot set breakpoint"	Hardware breakpoints exhausted — delete unused ones
Watchpoint has no effect	Watching a range larger than the comparator covers
Backtrace is garbage	Stack corrupted, or the frame pointer optimised away — see chapter 06
Variables show `<optimized out>`	Build with `-Og -g3` for debugging
Single-stepping breaks the behaviour	Real-time constraint — use a trace instead (chapter 02)
Flash programming fails intermittently	Adapter speed too high, or a marginal supply
Cannot debug a returned production unit	Permanent readout protection — should have been authenticated debug
Firmware dumped from a shipped product	Debug port left enabled

Check yourself

- ✓ List what the debug port can do independently of the CPU.
- ✓ Compare JTAG and SWD, and say what boundary scan is for.
- ✓ Draw the GDB / server / probe / target stack.
- ✓ Work the nine-step connection checklist from memory.
- ✓ Explain why "connect under reset" recovers a bricked board.
- ✓ Set a conditional breakpoint and say why it is slow.
- ✓ Explain why Cortex-M has only a handful of hardware breakpoints.
- ✓ Explain what a watchpoint solves and why it is underused.
- ✓ Use `print arr[0]@16` and `x/8xw` correctly.
- ✓ Enable and use the DWT cycle counter.
- ✓ Say what full instruction trace gives you that nothing else does.
- ✓ Describe the debug lifecycle and why authenticated debug beats permanent lockout.

The Linux debugging toolkit

A reference for the tools that actually get used, organised by the question you are trying to answer rather than by tool name.

Which tool for which question

Question	Tool
Why did it crash?	`gdb` on the core dump, `dmesg`, `decode_stacktrace.sh`
Is it a memory bug?	ASan, UBSan, Valgrind, KASAN
Where is the time going?	`perf`, flame graphs
What syscalls is it making?	`strace`
What library calls?	`ltrace`
What is the kernel doing?	`ftrace`, `bpftrace`, `/proc`
Why is it stuck?	`gdb` attach, `/proc/PID/stack`, `wchan`
What is on the wire?	`tcpdump`, `wireshark`
What files is it touching?	`lsof`, `fatrace`, `inotifywait`
Is the device present?	`lspci`, `lsusb`, `/sys`, `udevadm`
Where is the memory going?	`/proc/meminfo`, `smem`, `heaptrack`
Is it a race?	TSan, lockdep, `KCSAN`

Crashes

```
# ---- ENABLE CORE DUMPS (they are usually off) ----
$ ulimit -c unlimited
$ cat /proc/sys/kernel/core_pattern
$ sudo sysctl -w kernel.core_pattern=/tmp/core.%e.%p.%t
# systemd systems: coredumpctl list; coredumpctl gdb PID

$ gdb ./myapp /tmp/core.myapp.1234.1699
(gdb) bt full          # backtrace with locals
(gdb) info registers
(gdb) frame 3
(gdb) print *ptr
(gdb) info sharedlibrary
(gdb) thread apply all bt      # ALL threads - and for a
                               # deadlock this is the
                               # first thing to run

# ---- BUILD FOR DEBUGGABILITY ----
CFLAGS += -g3 -Og -fno-omit-frame-pointer
# -g3   includes macro definitions
# -Og   optimised but debuggable. NOT -O0, which changes
#       timing and hides optimiser-dependent bugs, and NOT
#       -O2, which optimises away the variables you want.
# -fno-omit-frame-pointer   makes perf and backtraces work.
#       Costs ~1% and is worth it in production too.

# KEEP THE UNSTRIPPED BINARY for every release you ship.
# A stripped binary plus a core dump is nearly useless, and
# the matching .debug file is the difference between a
# five-minute diagnosis and a week.

# ---- KERNEL OOPS ----
$ dmesg -T | tail -50
$ journalctl -k -b -1        # PREVIOUS boot - after a
                             # crash this is where it is

# decode a raw oops:
$ ./scripts/decode_stacktrace.sh vmlinux < oops.txt

# READING AN OOPS - the fields that matter:
#   "BUG: kernel NULL pointer dereference at 0000000000000018"
#        -> a null struct pointer; 0x18 is the field offset
#   "RIP: 0010:my_driver_probe+0x42/0x120"
#        -> the FUNCTION, the OFFSET INTO IT, and its SIZE
#   "Call Trace:"  -> who called whom
#   "Tainted: G  O"  -> an out-of-tree module is loaded
#
# $ addr2line -e mydriver.ko -f 0x42     # offset -> line

# ---- PANIC / HANG CONFIGURATION ----
$ sudo sysctl -w kernel.panic_on_oops=1
$ sudo sysctl -w kernel.hung_task_timeout_secs=60
# SysRq, when the console still responds:
#   Alt+SysRq+t   all task stacks
#   Alt+SysRq+w   BLOCKED tasks - the fastest way to find a
#                 deadlock
#   Alt+SysRq+l   all CPU backtraces
#   Alt+SysRq+c   deliberate crash, to test your crash dump
#                 path
# $ echo t > /proc/sysrq-trigger      # same, from a shell
```

Memory bugs

```
# ---- SANITISERS: use these first, always ----
$ gcc -fsanitize=address,undefined -g -Og prog.c
$ ./a.out
# =================================================
# ERROR: AddressSanitizer: heap-use-after-free
#   READ of size 4 at 0x602000000010 thread T0
#     #0 0x4008f1 in main prog.c:12
#   freed by thread T0 here:
#     #0 0x7f... in free
#     #1 0x4008d5 in main prog.c:10
#   previously allocated by thread T0 here: ...

# ASAN GIVES YOU THREE STACKS: where the bad access
# happened, where it was freed, and where it was allocated.
# That is usually the entire diagnosis.

# WHAT EACH CATCHES:
#   ASan   buffer overflow, use-after-free, double free,
#          use-after-return, leaks. ~2x slowdown, ~3x memory.
#   UBSan  signed overflow, null deref, misaligned access,
#          shifts out of range, bad enum values. Near-free.
#          (see the C track on undefined behaviour)
#   TSan   DATA RACES. ~10x slowdown, and worth every cycle.
#   MSan   reads of UNINITIALISED memory. Needs ALL code
#          instrumented, including libraries.
#
# ASan and TSan CANNOT be combined. Run separately.
# RUN THE SANITISERS IN CI. They find bugs that have not
# manifested yet, which is the only cheap time to find them.

# ---- VALGRIND: no rebuild needed ----
$ valgrind --leak-check=full --show-leak-kinds=all \
           --track-origins=yes ./myapp
# 20-50x slower than ASan but requires no recompilation -
# which matters when you have a binary and no source, or a
# build system you cannot change.
$ valgrind --tool=helgrind ./myapp     # races
$ valgrind --tool=massif ./myapp       # heap profile

# ---- WHERE IS THE MEMORY GOING ----
$ cat /proc/meminfo
$ cat /proc/PID/status | grep -E 'VmRSS|VmSize|Threads'
$ cat /proc/PID/smaps_rollup      # per-process summary
$ smem -tk                        # PSS - the honest
                                  # shared-memory number
$ heaptrack ./myapp && heaptrack_gui heaptrack.myapp.*

# ---- KERNEL-SIDE ----
# CONFIG_KASAN     ASan for the kernel. Use it on any
#                  driver you write.
# CONFIG_KMEMLEAK  kernel leak detector:
$ echo scan > /sys/kernel/debug/kmemleak
$ cat /sys/kernel/debug/kmemleak
# CONFIG_DEBUG_KMEMLEAK, CONFIG_SLUB_DEBUG=FZPU
# CONFIG_KCSAN     kernel race detector
$ cat /proc/slabinfo               # kernel object caches
$ cat /proc/vmallocinfo
```

Performance

```
# ---- THE ONE COMMAND TO KNOW ----
$ perf record -g --call-graph dwarf -F 999 ./myapp
$ perf report --stdio

# -g / --call-graph   capture stacks. dwarf works without
#                     frame pointers; fp is cheaper if you
#                     built with -fno-omit-frame-pointer
# -F 999              999 Hz, deliberately not 1000, to
#                     avoid aliasing with periodic activity

$ perf top                      # live, system-wide
$ perf stat ./myapp             # counters: IPC, cache
                                # misses, branch misses
$ perf stat -e cache-misses,cache-references,\
instructions,cycles ./myapp

# READING perf stat:
#   IPC below ~1.0        stalling - memory or branches
#   high cache-miss ratio poor locality (see the computer
#                         architecture track)
#   high branch-misses    unpredictable control flow

# ---- FLAME GRAPHS: the best way to read a profile ----
$ perf record -F 99 -g -p PID -- sleep 30
$ perf script | stackcollapse-perf.pl | flamegraph.pl \
    > out.svg
# WIDTH = time spent. Look for wide plateaus, not tall
# towers. Height is just stack depth and means nothing.

# ---- WHAT IS IT DOING RIGHT NOW ----
$ strace -f -T -tt -p PID          # syscalls, with timing
$ strace -c ./myapp                # SUMMARY - start here
$ strace -e trace=openat,read ./myapp
$ ltrace ./myapp                   # library calls

# strace -c IS THE FASTEST WAY TO ANSWER "why is this
# slow?" for anything I/O bound. If 90% of the time is in
# one syscall, you are done.

# ---- KERNEL TRACING ----
$ cd /sys/kernel/tracing
# echo function_graph > current_tracer
# echo my_driver_probe > set_graph_function
# echo 1 > tracing_on ; cat trace_pipe

$ trace-cmd record -p function_graph -g my_func ./app
$ kernelshark trace.dat

# ---- bpftrace: the modern answer ----
$ bpftrace -e 'tracepoint:syscalls:sys_enter_openat {
                 printf("%s %s\n", comm, str(args->filename));
               }'
$ bpftrace -e 'kprobe:vfs_read { @[comm] = count(); }'
$ bpftrace -e 'kretprobe:vfs_read /pid == 1234/ {
                 @bytes = hist(retval); }'

# ARBITRARY INSTRUMENTATION OF A RUNNING PRODUCTION KERNEL,
# safely, with no rebuild and no reboot. This is the single
# biggest advance in Linux debugging in fifteen years, and
# the bcc tools (execsnoop, opensnoop, biolatency,
```

```
# runqlat, tcpconnect) are ready-made scripts worth
# knowing.

# ---- LATENCY AND SCHEDULING ----
$ cyclictest -t4 -p 80 -n -i 200 -l 100000   # RT latency
$ perf sched record -- sleep 5 ; perf sched latency
$ chrt -f 80 ./myapp                          # run at RT
                                              # priority
$ taskset -c 2 ./myapp                        # pin to a
                                              # CPU
```

THE ORDER TO REACH FOR THESE

1. dmesg / journalctl — did the kernel already tell you?
2. strace -c — is it syscall bound?
3. perf stat — is it compute or memory bound?
4. perf record + flame graph — where exactly?
5. sanitisers — is it a latent memory or race bug?
6. bpftrace — a question none of the above answered.

Most investigations end at step 1 or 2. Reaching for a profiler before checking whether the kernel logged the answer is the most common wasted hour.

Devices, drivers and the system

```
# ---- IS THE DEVICE THERE ----
$ lspci -nnk                  # -k shows the bound DRIVER
$ lsusb -t                    # tree view, with drivers
$ ls /sys/bus/i2c/devices/
$ ls /sys/bus/platform/drivers/mydriver/
$ udevadm info -a -n /dev/ttyUSB0   # full attribute chain
$ udevadm monitor --property        # live hotplug events
$ cat /proc/device-tree/soc/i2c@40005400/status  # DT
$ dtc -I fs -O dts /proc/device-tree | less      # decompile
                                                 # the live
                                                 # DT

# "THE DRIVER DOESN'T LOAD" - THE CHECKLIST:
#   1. is the module present?        lsmod | grep mydrv
#   2. does the ID table match?      modinfo mydrv | grep alias
#   3. did probe run?                dmesg | grep mydrv
#   4. is the device in the DT/ACPI, and status = "okay"?
#   5. is its clock enabled?         /sys/kernel/debug/clk/clk_summary
#   6. is its regulator on?          /sys/kernel/debug/regulator/
#   7. did another driver claim it?  lspci -k, /proc/iomem
#   8. what did probe return?        dev_err messages, or
#                                    -EPROBE_DEFER (which is
#                                    normal - it will retry)

# ---- INTERRUPTS AND RESOURCES ----
$ cat /proc/interrupts             # WATCH THE COUNT CHANGE.
                                   # Not incrementing means
                                   # the IRQ never fires.
$ cat /proc/iomem
$ cat /proc/ioports
$ cat /proc/dma

# ---- LOCKING AND DEADLOCK ----
# CONFIG_PROVE_LOCKING (lockdep) - reports a potential
# deadlock the FIRST TIME an inconsistent lock order is
# observed, even if it never actually deadlocks. Enable it
# in any kernel you are developing against.
$ cat /proc/lockdep_stats
$ cat /proc/PID/stack             # kernel stack of a stuck
                                  # task
$ cat /proc/PID/wchan             # what it is waiting on

# ---- NETWORK ----
$ tcpdump -i eth0 -w cap.pcap 'port 443'
$ ss -tulpn                       # sockets and owning
                                  # processes
$ ip -s link ; ethtool -S eth0    # error counters

# ---- FILES AND I/O ----
$ lsof -p PID
$ ls -l /proc/PID/fd/
$ iostat -x 1 ; iotop
$ fatrace                         # every file access,
                                  # system-wide

# ---- USEFUL /proc AND /sys ENTRIES ----
/proc/PID/maps       memory map - find which library an
                     address is in
/proc/PID/smaps      per-mapping detail
/proc/PID/limits     the limits actually in effect
```

```
/proc/PID/environ    the environment it actually got
/proc/cmdline        kernel boot arguments
/proc/config.gz      the running kernel's config
/sys/kernel/debug/   debugfs - drivers expose their state
                     here, and yours should too
```

How it shows up

Symptom	Cause / tool
No core dump after a crash	`ulimit -c` is 0, or `core_pattern` pipes elsewhere
Backtrace shows only `??`	Stripped binary — keep the unstripped one
Variables `<optimized out>`	Rebuild with `-Og -g3`
perf shows no call graph	Frame pointers omitted — use `--call-graph dwarf`
Crash only in release builds	Undefined behaviour — run UBSan
Intermittent corruption under load	Data race — run TSan or enable KCSAN
Driver never probes	Work the eight-step checklist; often an ID or device-tree mismatch
`-EPROBE_DEFER` in dmesg	Normal — a dependency is not ready; it will retry
IRQ count never increments	Wrong IRQ number, wrong trigger type, or the pin is not muxed
Peripheral registers read as zero	Clock or regulator not enabled — check the debugfs summaries
Task stuck forever	`/proc/PID/stack` and SysRq-w
Memory grows without bound	`heaptrack`, or kmemleak for kernel allocations

Check yourself

- ✓ Match each of the twelve questions to its tool.
- ✓ Enable core dumps and open one in GDB.
- ✓ Explain why `-Og` is preferred over both `-O0` and `-O2`.
- ✓ Read a kernel oops and identify the function, offset and cause.
- ✓ Say what each of ASan, UBSan, TSan and MSan catches, and which cannot be combined.
- ✓ Explain what Valgrind offers that ASan does not.
- ✓ Record a profile and read a flame graph correctly.
- ✓ Interpret `perf stat`'s IPC and cache-miss figures.
- ✓ Write a `bpftrace` one-liner and say why it is safe on production.
- ✓ Work the eight-step "driver does not load" checklist.
- ✓ Explain what lockdep detects that a deadlock test would not.
- ✓ Give the six-step escalation order and say where most investigations end.

The Windows debugging toolkit

WinDbg is the most capable debugger in common use and the one with the steepest learning curve. This is the subset that does the work, plus the driver-specific tools that have no Linux equivalent.

Setting up

```
REM ---- INSTALL ----
REM Windows SDK "Debugging Tools for Windows", or:
winget install Microsoft.WinDbg

REM ---- SYMBOLS: DO THIS FIRST OR NOTHING WORKS ----
setx _NT_SYMBOL_PATH srv*C:\symbols*https://msdl.microsoft.com/download/symbols

REM add your own build output IN FRONT:
setx _NT_SYMBOL_PATH
"C:\myproject\out;srv*C:\symbols*https://msdl.microsoft.com/download/symbols"

REM and source, so you get source-level stepping:
.srcpath C:\myproject\src

REM ---- WITHOUT SYMBOLS, EVERY STACK IS HEXADECIMAL AND
REM      USELESS. If a stack looks like nonsense, run
REM      !sym noisy and .reload /f and read what it says
REM      it could not find.

REM ---- KERNEL DEBUGGING: two machines, or a VM ----
REM On the TARGET (an admin prompt), for a network
REM connection:
bcdedit /debug on
bcdedit /dbgsettings net hostip:192.168.1.10 port:50000 key:1.2.3.4
REM ^ it prints a KEY. Save it - the host needs it.

REM Then on the HOST:
windbg -k net:port=50000,key=1.2.3.4

REM Other transports:
bcdedit /dbgsettings serial debugport:1 baudrate:115200
bcdedit /dbgsettings usb targetname:mytarget

REM ---- TEST SIGNING, for driver development ----
bcdedit /set testsigning on
REM reboot. Without this an unsigned driver will not load,
REM and the failure message is not always obvious.

REM ---- CRASH DUMP CONFIGURATION ----
REM System Properties -> Advanced -> Startup and Recovery
REM   Kernel memory dump   the usual choice
REM   Complete memory dump needs a page file at least RAM
REM                        size; use it when you need user
REM                        memory too
REM Dumps land in C:\Windows\MEMORY.DMP

REM ---- FORCE A CRASH, to test the dump path ----
REM Enable the keyboard crash: HKLM\SYSTEM\CurrentControlSet\
REM   Services\kbdhid\Parameters\CrashOnCtrlScroll = 1
REM then Ctrl + ScrollLock + ScrollLock.
REM TEST THIS BEFORE YOU NEED IT.
```

The commands that matter

```
* ---- THE THREE-COMMAND CRASH ANALYSIS ----
!analyze -v          * START HERE, ALWAYS. Identifies the
                     * bugcheck, the faulting module, the
                     * probable cause and the stack.
                     * It is right most of the time.
k                    * the stack
lm                   * loaded modules; lm v m mydrv for
                     * detail

* ---- STACKS ----
k                    * current thread
kb                   * with the first three arguments
kv                   * verbose: frame pointers, calling
                     * convention
kP                   * full parameters with types - the
                     * most readable
!thread              * the current thread's full state
!process 0 7         * ALL processes with ALL thread
                     * stacks. Huge, and the right tool for
                     * a hang.
~*k                  * all threads in the current process
                     * (user mode)

* ---- MEMORY ----
dd addr              * dwords
dq addr              * qwords
db addr              * bytes + ASCII
dc addr              * dwords + ASCII
du addr              * UNICODE string
da addr              * ASCII string
dt nt!_EPROCESS addr * DUMP A TYPED STRUCTURE - the single
                     * most useful WinDbg command
dt mydrv!_DEVICE_CONTEXT addr
dt -r nt!_IRP addr   * recursive, expanding substructures
!pool addr           * which pool allocation an address is
                     * in
!address addr        * what kind of memory this is
ed / eb / eq         * EDIT memory
s -b 0 L?ffffffff 41 42 43   * SEARCH memory

* ---- DRIVER-SPECIFIC, and these have no Linux
*      equivalent ----
!irp addr            * decode an IRP: which stack location,
                     * which driver, what major function
!devobj addr         * a device object
!drvobj mydrv 7      * a driver object with all its
                     * dispatch routines - shows instantly
                     * whether you registered the handler
                     * you think you did
!devstack addr       * the whole device stack - who is
                     * above and below you
!devnode 0 1         * the PnP device tree
!wdfkd.wdfdriverinfo mydrv   * KMDF state
!wdfkd.wdfqueue addr
!wdfkd.wdfdevice addr
!wdflogdump mydrv    * THE KMDF INTERNAL LOG. Read this
                     * before anything else when a KMDF
                     * driver misbehaves.

* ---- POOL AND MEMORY LEAKS ----
```

```
!poolused 2          * per-tag, sorted by usage. USE A
                     * UNIQUE POOL TAG PER ALLOCATION SITE
                     * IN YOUR DRIVER and this finds your
                     * leak in one command.
!poolfind MyTg
!vm                  * system memory summary
!heap -s             * user-mode heaps

* ---- BREAKPOINTS ----
bp mydrv!MyDispatch          * software breakpoint
bu mydrv!MyDispatch          * UNRESOLVED - set it before
                             * the driver loads, and it
                             * arms when it does. THIS IS
                             * THE ONE YOU WANT for
                             * DriverEntry.
ba w4 addr                   * hardware WRITE breakpoint on
                             * 4 bytes - the watchpoint
                             * equivalent
bp mydrv!Fn ".printf \"len=%d\\n\", @rcx; gc"
                             * print and continue, without
                             * stopping
bl                           * list
bc *                         * clear all

* ---- EXECUTION ----
g        * go
p        * step over
t        * step into
gu       * go up (finish)
.restart

* ---- SESSION ----
.reload /f           * force symbol reload
.sympath+ C:\mysyms
.logopen C:\log.txt  * LOG THE WHOLE SESSION. Do this.
.dump /ma C:\my.dmp  * write a full dump
!sym noisy           * diagnose symbol loading
.hh command          * open help for a command
```

Reading a bugcheck

```
* !analyze -v output, and what to actually read:

  DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
  Arguments:
    Arg1: ffffb00000000000   memory referenced
    Arg2: 0000000000000002   IRQL at the time
    Arg3: 0000000000000001   0 = read, 1 = WRITE
    Arg4: fffff80512345678   address that referenced it

  READ THE FOUR ARGUMENTS. They usually contain the whole
  answer: what was touched, at what IRQL, read or write,
  and from where.

* ---- THE BUGCHECKS YOU WILL ACTUALLY MEET ----

  0x0A  IRQL_NOT_LESS_OR_EQUAL
        Touched pageable memory, or a bad pointer, at
        DISPATCH_LEVEL or above.
  0xD1  DRIVER_IRQL_NOT_LESS_OR_EQUAL
        The same thing, and it was a driver. THE MOST
        COMMON DRIVER BUGCHECK BY A LARGE MARGIN.
        Cause: paged memory accessed at raised IRQL, or a
        freed pointer.
  0x1E  KMODE_EXCEPTION_NOT_HANDLED
  0x50  PAGE_FAULT_IN_NONPAGED_AREA
        Freed memory, or a wild pointer.
  0x7E  SYSTEM_THREAD_EXCEPTION_NOT_HANDLED
  0x9F  DRIVER_POWER_STATE_FAILURE
        A driver did not complete a power IRP. Arg1 = 3
        means a device object blocked the transition.
  0xC4  DRIVER_VERIFIER_DETECTED_VIOLATION
        Driver Verifier caught you. The subcode says
        exactly what.
  0xC9  DRIVER_VERIFIER_IOMANAGER_VIOLATION
  0xC5  DRIVER_CORRUPTED_EXPOOL
        Pool corruption - you wrote past an allocation.
  0x139 KERNEL_SECURITY_CHECK_FAILURE
        A stack cookie or list-entry corruption check.
  0x133 DPC_WATCHDOG_VIOLATION
        A DPC ran too long, or IRQL stayed high. Usually a
        loop that should have been in a work item.

* ---- THE IRQL RULE THAT CAUSES MOST OF THESE ----
    PASSIVE_LEVEL (0)   may touch paged memory, may block
    APC_LEVEL (1)
    DISPATCH_LEVEL (2)  NO PAGED MEMORY. NO BLOCKING. NO
                        WAITING.
    DIRQL (3+)          device interrupt level

    At DISPATCH_LEVEL or above, touching pageable memory
    faults, and a page fault cannot be serviced because
    the page-fault handler needs to wait. THAT IS BUGCHECK
    0xD1, and it is why the rule exists.

    !irql               * check where you are
    KeGetCurrentIrql()  * from code

* ---- A HANG RATHER THAN A CRASH ----
    !analyze -v -hang
    !process 0 7        * every thread stack
```

```
!locks              * held resources and waiters
!stacks 2           * all kernel stacks, filtered
Look for a cycle: A waits on a lock held by B, which
waits on a lock held by A.
```

DRIVER VERIFIER IS NOT OPTIONAL

```
verifier /standard /driver mydrv.sys
shutdown /r /t 0

It injects allocation failures, checks IRQL rules,
detects pool corruption immediately rather than
later, catches leaked resources at unload, and forces
DMA and IRP misuse to fail loudly at the point of the
mistake.

verifier /flags 0x209BB /driver mydrv.sys   (more)
verifier /volatile /flags 0x8 /adddriver mydrv.sys
verifier /querysettings
verifier /reset                             (turn off)
```

It will crash your driver on bugs that would otherwise corrupt memory silently and fail a week later in someone else's code. **Run every driver under Verifier before shipping it** — and expect the first run to find something.

The rest of the toolkit

```
REM ---- SYSINTERNALS: download the whole suite ----
Process Explorer   handles, DLLs, threads, stacks. The
                   better Task Manager.
Process Monitor    EVERY file, registry, network and
                   process event, with a stack for each.
                   THE FIRST TOOL for "why can't it find
                   this file" and "why is this failing with
                   access denied".
                   FILTER HARD or you will drown - by
                   process name first.
WinObj             the kernel object namespace: device
                   objects, symbolic links, sections
DebugView          captures DbgPrint output from user AND
                   kernel mode, without a debugger
                   attached. Enable kernel capture in the
                   menu.
Autoruns           what starts, and from where
RAMMap / VMMap     physical and virtual memory breakdown
Handle             which process has that file open
LiveKD             kernel debugging on a LIVE system with
                   no second machine - read-only, but
                   frequently enough

REM ---- ETW: the platform's tracing infrastructure ----
wpr -start GeneralProfile -filemode
REM ... reproduce ...
wpr -stop trace.etl
wpa trace.etl                REM Windows Performance
                             REM Analyzer

logman query providers       REM everything that can be
                             REM traced
tracelog -start mytrace -guid #GUID -f out.etl -flag 0xff
tracefmt out.etl -o out.txt  REM decode WPP (chapter 02)

REM ETW IS THE SAME IDEA AS ftrace/eBPF: always compiled
REM in, negligible when off, enabled at runtime. It covers
REM the kernel, drivers, .NET and applications in ONE
REM correlated timeline, which is genuinely better than
REM anything on Linux for whole-system analysis.

REM ---- DEVICE AND DRIVER STATE ----
pnputil /enum-devices /connected
pnputil /enum-drivers
devcon status *              REM from the WDK
devmgmt.msc                  REM and turn on
                             REM View -> Devices by
                             REM connection

REM ---- INSTALLATION FAILURES ----
REM C:\Windows\INF\setupapi.dev.log
REM IT LOGS EXACTLY WHY AN INSTALL FAILED - signature,
REM ranking, a missing file, a hardware ID mismatch. Search
REM it before guessing.

REM ---- DRIVER TESTING ----
REM HLK (Hardware Lab Kit) - required for WHQL
REM certification
REM Static Driver Verifier, at build time:
msbuild /p:RunCodeAnalysis=true
```

```
REM SDV does formal verification of DDI usage rules and
REM finds real bugs on paths your tests never reach.
REM Code Analysis with driver-specific rules:
REM   /analyze plus the WDK ruleset

REM ---- USER-MODE CRASHES ----
REM Configure Windows Error Reporting local dumps:
REM HKLM\SOFTWARE\Microsoft\Windows\
REM   Windows Error Reporting\LocalDumps
REM     DumpFolder  = C:\dumps
REM     DumpType    = 2   (full)
windbg -z C:\dumps\myapp.exe.1234.dmp
!analyze -v
```

How it shows up

Symptom	Cause
Stacks are all hex addresses	Symbols not configured — set `_NT_SYMBOL_PATH`, run `!sym noisy`
Bugcheck 0xD1	Paged memory or a bad pointer at raised IRQL
Bugcheck 0x133	A DPC ran too long — move the work to a work item
Bugcheck 0xC4	Driver Verifier caught a rule violation — read the subcode
Bugcheck 0x9F	A power IRP was never completed
Pool corruption reported far from the cause	Run under Verifier — it catches it at the point of the write
Driver will not load, no error	`setupapi.dev.log` has the reason; check test signing
Breakpoint on `DriverEntry` never hits	Use `bu`, not `bp` — the module is not loaded yet
Cannot find why a file open fails	Process Monitor, filtered by process name
System hangs, no crash	`!process 0 7` and `!locks` — look for a wait cycle
KMDF request handled by the wrong queue	`!wdflogdump` and `!wdfqueue`
Leak with no obvious owner	`!poolused 2` — needs unique pool tags

Check yourself

- ✓ Configure a symbol path and explain why it must come first.
- ✓ Set up kernel debugging over the network, including the key.
- ✓ Name the three commands you run first on any crash dump.
- ✓ Use `dt` to dump a typed structure, recursively.
- ✓ Explain what `!drvobj`, `!devstack` and `!irp` each tell you.
- ✓ Say why `!wdflogdump` comes first for a KMDF driver.
- ✓ Read the four arguments of a 0xD1 bugcheck.
- ✓ State the IRQL rule and explain why violating it causes 0xD1.
- ✓ Explain the difference between `bp` and `bu`.
- ✓ Say what Driver Verifier injects and checks, and why you run it before shipping.
- ✓ Describe what ETW has in common with ftrace and WPP.
- ✓ Say where to look when a driver installation fails.

Bug patterns and their signatures

Most bugs are one of about twenty recurring shapes. Recognising the shape from the symptom takes you from "something is wrong" to a short list of causes in seconds, which is most of what experience buys.

Memory bugs

```
BUFFER OVERFLOW
  SIGNATURE   corruption of a variable ADJACENT to the
              one you wrote; a smashed stack canary;
              crash in free() far from the real bug;
              works until a struct layout changes
  CAUSES      off-by-one on a bound, strcpy, sprintf,
              a length in elements used as bytes
  FIND IT     ASan, Driver Verifier, a canary pattern
              around buffers

USE AFTER FREE
  SIGNATURE   works most of the time, then garbage; the
              data is plausible but WRONG; often
              timing-dependent
  CAUSES      a second pointer to freed memory; a
              callback firing after teardown; refcount
              dropped early
  FIND IT     ASan; set pointers to NULL after free; use
              a poison pattern

DOUBLE FREE
  SIGNATURE   heap corruption, a crash in the allocator,
              an abort from glibc
  CAUSES      two error paths both freeing; ownership
              unclear

MEMORY LEAK
  SIGNATURE   RSS grows monotonically; fails after hours
              or days; degrades then dies
  CAUSES      an error path that returns without
              freeing; a missing put() on a refcounted
              object
  FIND IT     heaptrack, Valgrind, kmemleak,
              !poolused

UNINITIALISED READ
  SIGNATURE   works in debug, fails in release; the
              value is different every run; adding a
              printf changes it
  CAUSES      a missing initialiser; a struct field
              added and not set in one construction site
  FIND IT     MSan, Valgrind, and simply initialising
              everything

STACK OVERFLOW
  SIGNATURE   corruption with no obvious writer;
              hard fault; crash on entering a function;
              only under deep recursion or a large local
  CAUSES      a large local array, deep recursion, an
              ISR stack too small, alloca
  FIND IT     paint the stack with a pattern at boot and
              measure the high-water mark. DO THIS ON
              EVERY EMBEDDED PROJECT.

THE UNIFYING CHARACTERISTIC OF ALL SIX: THE SYMPTOM
APPEARS FAR FROM THE CAUSE, in space and in time. That
is exactly why the sanitisers are worth their slowdown -
they move the report back to the moment of the mistake.
```

Concurrency bugs

```
/* --- DATA RACE --- */
    SIGNATURE   rare, load-dependent, disappears under a
                debugger, differs between machines, worse
                or better on more cores
    THE CLASSIC:
        if (!ptr)              /* thread A checks */
            ptr = alloc();     /* thread B also checks,
                                  both allocate */
    ALSO:  count++ is READ, MODIFY, WRITE - three
           operations, interruptible between any two.
    FIND IT     TSan, KCSAN, helgrind
    FIX IT      a lock, or an ATOMIC operation - not a
                volatile, which orders nothing (see the C
                track)

/* --- DEADLOCK --- */
    SIGNATURE   total hang, no CPU usage, threads blocked
                on locks
    CAUSE       inconsistent lock ORDER. A takes L1 then
                L2; B takes L2 then L1.
    FIND IT     lockdep, !locks, all-thread backtraces
    FIX IT      A GLOBAL LOCK ORDERING, documented, and
                enforced. Never acquire a lock while
                holding one you did not plan for.

/* --- LOST WAKEUP --- */
    SIGNATURE   a thread waits forever even though the
                condition became true
    CAUSE       the condition was signalled between the
                check and the wait
    FIX         the condition-variable pattern, ALWAYS with
                a loop:
                    while (!condition)
                        wait(cv, lock);
                The `while` also handles SPURIOUS WAKEUPS,
                which are permitted by the specification and
                do occur.

/* --- PRIORITY INVERSION --- */
    SIGNATURE   a high-priority task misses its deadline
                for no visible reason; intermittent and
                load-dependent
    CAUSE       high waits on a lock held by low, and
                MEDIUM preempts low so it never releases
    FIX         priority inheritance, or a priority
                ceiling. (This is the Mars Pathfinder bug,
                and it is worth knowing the story.)

/* --- ABA --- */
    SIGNATURE   a lock-free structure corrupts under load
    CAUSE       a value read as A, changed to B and back to
                A; the compare-and-swap succeeds but the
                world moved
    FIX         a tagged pointer with a version counter, or
                hazard pointers. And seriously consider not
                writing lock-free code.

/* --- TORN READ/WRITE --- */
    SIGNATURE   a 64-bit value observed half-updated on a
                32-bit machine
```

```
    FIX         atomics of the right width, or a lock

/* --- MISSING MEMORY BARRIER --- */
    SIGNATURE   works on x86, fails on ARM. Reordering
                visible only on weakly-ordered
                architectures.
    CAUSE       the compiler or the CPU reordered stores;
                a peripheral write was buffered
    FIX         the right barrier, or the right atomic
                ordering (see the ARM track on memory
                ordering)
    AND NOTE: THIS IS WHY x86 EXPERIENCE MISLEADS ON ARM.
    x86's strong ordering hides bugs that ARM exposes
    immediately.

/* --- REENTRANCY --- */
    SIGNATURE   corruption when an ISR fires during a
                function that the main loop also calls
    CAUSE       shared state, a static local, a non-atomic
                read-modify-write on a peripheral register
    FIX         critical sections, or lock-free by design  */
```

C and arithmetic

```
/* --- INTEGER OVERFLOW --- */
    SIGNATURE   a negative length; an enormous allocation;
                a loop that never terminates; a bounds
                check that passes when it should not
        if (offset + len > size)    /* OVERFLOWS -> passes */
        if (len > size - offset)    /* correct, given
                                       offset <= size    */
    AND SIGNED OVERFLOW IS UNDEFINED BEHAVIOUR, so the
    optimiser may delete the check entirely (see the C and
    compilers tracks).

/* --- IMPLICIT CONVERSION --- */
    SIGNATURE   a comparison that is always true or always
                false
        if (strlen(s) - 1 >= 0)    /* size_t: ALWAYS TRUE */
        int c = getchar();
        if (c == EOF)              /* correct - and `char c`
                                      would be a bug */
    FIND IT     -Wconversion, -Wsign-compare

/* --- SIZEOF ON A POINTER --- */
        void f(int arr[]) { sizeof(arr); }   /* = 8, not
                                                the array */
    Arrays DECAY to pointers when passed. Pass the length.

/* --- UNINITIALISED STRUCT PADDING --- */
    memcmp on structs compares PADDING, which is
    indeterminate. Two logically equal structs compare
    unequal.
    FIX: compare fields, or memset the struct first.

/* --- STRICT ALIASING --- */
        float f = *(float*)&i;     /* UB */
    The optimiser assumes an int* and a float* cannot alias
    and reorders accordingly. Use memcpy - it compiles to
    the same instruction and is defined.

/* --- MISSING volatile --- */
    SIGNATURE   a loop reading a hardware register or an
                ISR-modified flag never exits; works at -O0
                and hangs at -O2
    FIX         volatile on the variable. And note that
                volatile provides NO atomicity and NO
                ordering between different objects.

/* --- FORMAT STRING MISMATCH --- */
        printf("%d", (long)x);      /* UB */
        printf("%d", size_t_value); /* wrong on 64-bit */
    Use the macros: %zu for size_t, PRIu32 from inttypes.h.
    FIND IT     -Wformat=2, which catches all of these.

/* --- THE ERROR PATH NOBODY TESTED --- */
    SIGNATURE   fine until something fails, then a leak, a
                double free, or a crash
    CAUSE       cleanup paths that are never exercised
    FIND IT     FAULT INJECTION. Make every allocation fail
                once. Driver Verifier does this for you on
                Windows; on Linux, failslab and
```

```
                fail_page_alloc do the same. THIS FINDS
                REAL BUGS EVERY TIME IT IS RUN.           */
```

> **THE DEBUG/RELEASE DIVIDE**
> "Works in debug, fails in release" is almost always one of five things: **uninitialised memory** (debug builds often zero it), **a missing `volatile`**, **undefined behaviour** the optimiser exploited, **a timing-dependent race** that the slower debug build hid, or **an assertion that had a side effect** and was compiled out. Check those five before anything else.

Hardware and embedded

```
/* --- CLOCK NOT ENABLED --- */
    SIGNATURE   the peripheral reads back all zeros or
                reset values; writes have no effect; no
                error reported
    THE SINGLE MOST COMMON EMBEDDED BRING-UP BUG. Enable
    the peripheral clock before touching any of its
    registers.

/* --- PIN NOT MUXED --- */
    SIGNATURE   the peripheral looks correctly configured
                and nothing appears on the pin
    A pin has one function at a time. Check the alternate
    function register, and check with a scope.

/* --- READ-MODIFY-WRITE ON A REGISTER --- */
        REG |= BIT;      /* three operations */
    An interrupt between them loses the other write. And
    some registers are WRITE-1-TO-CLEAR, where |= clears
    every pending flag, not just yours. READ THE REGISTER
    DESCRIPTION.

/* --- ALIGNMENT FAULT --- */
    SIGNATURE   hard fault on a cast from a byte buffer;
                works on x86, faults on Cortex-M0
    FIX         memcpy into a properly aligned object

/* --- ENDIANNESS --- */
    SIGNATURE   values byte-swapped over a network or a bus
    FIX         explicit conversion at every boundary;
                never cast a buffer to a struct

/* --- DMA AND CACHE COHERENCY --- */
    SIGNATURE   DMA'd data is stale or partially correct;
                works with the cache off
    CAUSE       the CPU wrote to a cached buffer the device
                then read, or the device wrote to memory the
                CPU has cached
    FIX         clean before the device reads, invalidate
                after it writes - or use the DMA API and
                let it do this (Linux track), or a
                non-cacheable buffer.
    AND WATCH THE CACHE LINE: an invalidate affects a whole
    line, so an adjacent variable sharing the line gets
    discarded too. Align and pad DMA buffers to a cache
    line.

/* --- BUFFER NOT IN DMA-CAPABLE MEMORY --- */
    A stack buffer, or memory the device's address space
    cannot reach. Silently transfers garbage.

/* --- ISR TOO LONG --- */
    SIGNATURE   missed interrupts, jitter, a watchdog
                reset, a DPC watchdog bugcheck
    FIX         do the minimum in the ISR; defer the rest

/* --- MISSING FLAG CLEAR --- */
    SIGNATURE   the ISR fires continuously and the system
                appears hung
    FIX         clear the interrupt source, and re-read the
                register afterwards to ensure the write has
```

```
                landed before returning.

/* --- FLOATING INPUT --- */
    SIGNATURE   a GPIO reads randomly, and touching the
                board changes it
    FIX         enable a pull-up or pull-down

/* --- POWER AND DECOUPLING --- */
    SIGNATURE   resets under load, corruption during radio
                transmission or motor start, only on some
                units
    FIX         scope the rail WITH THE LOAD APPLIED, at a
                fast timebase. A brownout of a few
                microseconds does not show on a multimeter.

/* --- SILICON ERRATA --- */
    SIGNATURE   the hardware does not do what the reference
                manual says
    THE FIX     READ THE ERRATA SHEET. Vendors publish
                them. Some of them are extraordinary, and
                the workaround is usually documented.    */
```

Symptom to cause

Symptom	Look at first
Works in debug, fails in release	Uninitialised memory, `volatile`, UB, a race, an assertion with a side effect
Works on x86, fails on ARM	Memory ordering, alignment, signedness of `char`
Fails only under load	A race, a resource limit, or a timing margin
Fails after hours or days	A leak, a counter wrap, fragmentation
Fails on one unit only	Hardware, calibration data, or a per-unit configuration
A `printf` makes it go away	A race, or a timing-dependent bug
Crash in `free` or the allocator	Heap corruption from an earlier overflow
Corrupted value with no writer	Buffer overflow, stack overflow, or DMA
Total hang, no CPU use	Deadlock — get all thread stacks
Total hang, 100% CPU	Infinite loop, or an interrupt flag never cleared
Peripheral reads all zeros	Clock not enabled
Data byte-swapped	Endianness at a boundary
Stale DMA data	Cache maintenance missing
Fails only when cold or hot	Timing margin, crystal drift, marginal supply
Bounds check passes wrongly	Integer overflow or an implicit conversion

Check yourself

- ✓ Name six memory bug patterns and the signature of each.
- ✓ Say what all memory bugs have in common and why sanitisers help.
- ✓ Describe the check-then-act race and why `volatile` does not fix it.
- ✓ Explain deadlock's cause and the one discipline that prevents it.
- ✓ Write the correct condition-variable wait and give two reasons for the loop.

- ✓ Explain priority inversion and its fix.
- ✓ Rewrite `offset + len > size` safely and say why the original fails.
- ✓ Give the five causes of "works in debug, fails in release".
- ✓ Say why x86 experience misleads about memory ordering on ARM.
- ✓ Explain the DMA cache-coherency failure in both directions, and the cache-line trap.
- ✓ Say what a peripheral reading all zeros almost always means.
- ✓ Describe fault injection and why it finds bugs every time.

The board bring-up checklist

A new board arrives and nothing works. This is the order to attack it in — strictly bottom up, verifying each layer before trusting anything above it.

The principle

```
BRING UP IN LAYERS. VERIFY EACH ONE BEFORE MOVING UP.

  7  APPLICATION
  6  DRIVERS AND PERIPHERALS
  5  CONSOLE / STDIO
  4  RAM AND THE C RUNTIME
  3  CLOCKS
  2  DEBUG CONNECTION
  1  POWER
  0  THE PHYSICAL BOARD

THE MISTAKE EVERYONE MAKES: flashing the full application
onto a brand-new board and wondering why nothing happens.
You have just introduced seven layers of possible failure
at once, with no way to distinguish them.

START AT LAYER 0. GET A BLINKING LED BEFORE ANYTHING
ELSE.

A BLINKING LED PROVES A REMARKABLE AMOUNT:
  · power is present and stable enough
  · the clock configuration works
  · reset released correctly
  · the vector table is where the CPU expects it
  · the C runtime initialised
  · main() was reached
  · GPIO configuration works
  · your build and flash toolchain are correct

That is layers 0 through 4 in one test, and it takes ten
minutes. NEVER SKIP IT.

AND THE SECOND PRINCIPLE: ON A NEW BOARD, DO NOT TRUST
THE SCHEMATIC OR THE ASSEMBLY. Both contain mistakes.
Measure.
```

Layers 0-2: power and access

```
/* --- 0. BEFORE APPLYING POWER --- */
  [ ] VISUAL INSPECTION under magnification: solder
      bridges, tombstoned parts, missing components,
      COMPONENTS FITTED THE WRONG WAY AROUND (check pin 1
      on every polarised part)
  [ ] CHECK FOR SHORTS with a meter: every supply rail to
      ground, and rail to rail. A short here saves the
      board.
  [ ] verify the assembly matches the BOM - wrong-value
      resistors in a divider are common and invisible
  [ ] confirm the input polarity and voltage before
      connecting anything

/* --- 1. POWER --- */
  [ ] apply power through a CURRENT-LIMITED SUPPLY. Set the
      limit to a little above the expected draw. If it
      immediately limits, power down and find the short.
  [ ] MEASURE EVERY RAIL with a meter. Tolerance, typically
      +-5%.
  [ ] check the POWER SEQUENCING if the datasheet requires
      an order - many SoCs do, and violating it can damage
      the part or leave it wedged
  [ ] SCOPE EVERY RAIL, AC-coupled, at a fast timebase.
      Ripple, and any dip at start-up. A multimeter will
      not show a 2 microsecond brownout that resets your
      MCU.
  [ ] check the current draw against expectation. Far too
      high means a short or a mis-stuffed part; far too low
      means something is not powered at all.
  [ ] check thermals - anything unusually hot after a
      minute is wrong

/* --- 2. THE DEBUG CONNECTION --- */
  [ ] RESET pin: is it released? Scope it at power-up. A
      held reset is a very common cause of "the board is
      dead".
  [ ] BOOT MODE pins: strapped correctly for the boot
      source you intend?
  [ ] connect the probe. Ground first (chapter 03).
  [ ] read the CHIP ID register. This proves SWD/JTAG
      works, the core is alive, and the part is what you
      think it is.
        > mdw 0xE0042000       # DBGMCU_IDCODE on STM32
  [ ] if it will not connect, work the nine-step checklist
      in chapter 03. Try "connect under reset" early.
  [ ] MASS ERASE the part. Factory-programmed content and
      leftover test firmware both cause confusing
      behaviour.

/* AT THIS POINT YOU HAVE A CONNECTION TO A LIVE CHIP. Every
   remaining problem is now diagnosable, which is a
   qualitatively different situation from ten minutes
   ago.                                                  */
```

Layers 3-5: clocks, RAM, console

```
/* --- 3. CLOCKS --- */
  [ ] FIRST, RUN FROM THE INTERNAL OSCILLATOR. It always
      works. Do not configure the PLL yet.
  [ ] blink an LED with a crude delay loop. THE FIRST
      MILESTONE.
  [ ] now switch to the external crystal, ONE STEP AT A
      TIME:
        · is the crystal oscillating? Scope it with a 10x
          probe - a 1x probe's capacitance can stop a
          crystal, which is a genuinely confusing failure
        · are the load capacitors the value the crystal
          datasheet specifies?
        · does the ready flag set? DO NOT SPIN FOREVER
          WAITING:
              uint32_t to = 100000;
              while (!(RCC->CR & RCC_CR_HSERDY) && --to)
                      ;
              if (!to) fall_back_to_internal_osc();
          A firmware that hangs in a clock wait loop looks
          exactly like a dead board.
  [ ] configure the PLL. CHECK THE FLASH WAIT STATES FIRST
      - at a higher core clock the flash needs more wait
      states, and setting the clock before the wait states
      is an instant hard fault.
  [ ] VERIFY THE ACTUAL FREQUENCY. Output the system clock
      on the MCO pin and measure it, or time a known delay
      loop against a stopwatch. DO NOT ASSUME THE
      CONFIGURATION WORKED - a wrong clock makes every
      subsequent baud rate and timer wrong, and the symptom
      is "the UART outputs garbage".

/* --- 4. RAM AND THE C RUNTIME --- */
  [ ] internal RAM: the startup code has already used it if
      you reached main()
  [ ] EXTERNAL RAM (SDRAM/DDR), if fitted, needs its
      controller configured, and then TESTED:
        · walking ones and walking zeros - catches stuck
          and shorted data lines
        · an address test (write the address to itself) -
          catches ADDRESS line faults, which present as
          aliasing rather than corruption
        · a pattern test over the full range
        · AND TEST AT TEMPERATURE. Marginal timing passes
          at 25C and fails at 70C.
  [ ] check the LINKER SCRIPT regions match the actual
      memory map
  [ ] check .data was copied and .bss was zeroed - a global
      that is unexpectedly nonzero means the startup code
      is wrong
  [ ] SET UP STACK PAINTING NOW, while it is easy:
        fill the stack with 0xA5 at boot; a later
        high-water-mark check tells you the real usage
        (chapter 06)

/* --- 5. THE CONSOLE --- */
  [ ] configure the UART and print something
  [ ] IF IT IS GARBAGE, THE CLOCK IS WRONG. Garbled output
      at a consistent wrong rate is almost always a system
      clock that is not what you configured. Go back to
      step 3.
```

```
  [ ] if there is nothing at all: check the pin mux, check
      TX and RX are not swapped, check the peripheral clock
      is enabled, and scope the TX pin to see whether
      anything is coming out at all.
  [ ] print the clock frequencies, the reset reason, and
      the build ID at boot. YOU WILL WANT ALL THREE
      CONSTANTLY.
  [ ] install a fault handler that prints the fault
      registers (chapter 06) - do this before you have
      faults, not after

/* NOW YOU HAVE A CONSOLE. Everything from here is ordinary
   debugging.                                              */
```

THE THREE MILESTONES

```
1. THE CHIP ID READS BACK.
   Power, reset and debug access all work.

2. AN LED BLINKS.
   Clocks, reset, vector table, C runtime, GPIO and
   your entire toolchain work.

3. THE CONSOLE PRINTS.
   You now have observability, and every remaining
   problem is a normal debugging problem.

Do not attempt anything else until all three pass. Every
hour spent trying to debug an application on a board that
has not reached milestone 2 is wasted.
```

Layer 6: peripherals

```
/* ONE AT A TIME. Never bring up two peripherals at once -
   you will not know which one is failing.               */

/* --- THE UNIVERSAL SEQUENCE FOR ANY PERIPHERAL --- */
  1. ENABLE ITS CLOCK. Then read a register back. If it
     reads zero or the reset value and writes do not stick,
     the clock is off. (chapter 06 - the most common bug in
     this whole chapter.)
  2. CONFIGURE THE PINS: mux, direction, speed, pull,
     open-drain where required.
  3. CONFIGURE THE PERIPHERAL, in the ORDER THE REFERENCE
     MANUAL SPECIFIES. Some registers can only be written
     while the peripheral is disabled.
  4. READ BACK EVERY CONFIGURATION REGISTER and compare it
     to what you wrote. Reserved bits, write-protected
     registers and clock-gated writes all silently discard.
  5. THE SIMPLEST POSSIBLE TRANSACTION. One byte. Polled,
     not interrupt-driven, not DMA.
  6. SCOPE IT. Confirm the signal is physically present and
     correct.
  7. ONLY THEN add interrupts.
  8. ONLY THEN add DMA.

/* --- GPIO --- */
  [ ] output: set a pin, measure it. Toggle it, scope it.
  [ ] input: tie the pin high and low, read it. CHECK THE
      PULL CONFIGURATION - a floating input reads randomly.

/* --- I2C --- */
  [ ] pull-ups fitted? Value appropriate for the bus
      capacitance? (2.2k-10k typically; too weak gives
      slow rising edges and a bus that half-works)
  [ ] SCAN THE BUS - address every device from 0x08 to
      0x77 and see who ACKs. This immediately proves
      wiring, pull-ups and addressing.
  [ ] SCOPE SDA AND SCL. Look at the RISE TIME - a slow
      rise is the classic weak-pull-up signature. Look for
      the ACK bit.
  [ ] check the address: 7-bit versus 8-bit conventions
      differ between datasheets and APIs, and a factor-of-
      two address error is the most common I2C mistake.
  [ ] a stuck bus (SDA held low by a confused slave)
      recovers by clocking SCL manually nine times

/* --- SPI --- */
  [ ] MODE: CPOL and CPHA must match the slave. Four
      combinations, and the wrong one gives plausible
      garbage rather than nothing.
  [ ] chip select polarity and timing
  [ ] MOSI/MISO not swapped
  [ ] read a known register - most devices have an ID
      register, and that is your test
  [ ] scope all four signals together

/* --- UART --- */
  [ ] baud rate, and therefore the clock (step 3)
  [ ] TX/RX crossed correctly
  [ ] voltage levels - 3.3V versus 5V versus RS-232, which
      is inverted and bipolar
  [ ] flow control configuration matching both ends
```

```
/* --- ADC --- */
  [ ] reference voltage present and correct
  [ ] measure a known voltage; check the reading matches
  [ ] sampling time long enough for the source impedance -
      too short reads low and the error scales with
      impedance

/* --- FLASH / STORAGE --- */
  [ ] read the JEDEC ID first
  [ ] write and read back a page
  [ ] erase and confirm it reads all 0xFF
  [ ] check timing at the temperature extremes

/* --- USB / ETHERNET --- */
  [ ] clock accuracy - USB needs 0.25%, so the internal RC
      oscillator is usually not good enough
  [ ] termination, and the magnetics for Ethernet
  [ ] enumerate; check the descriptors with a host tool
  [ ] and a PROTOCOL ANALYSER is worth its cost here - USB
      problems are almost undiagnosable without one     */
```

Record what you find

```
/* A BRING-UP LOG IS A DELIVERABLE, NOT A PERSONAL NOTE.

   FOR EACH BOARD REVISION, RECORD:
     · every measured rail voltage and current
     · the actual clock frequencies measured
     · every deviation from the schematic - blue wires,
       lifted pins, changed resistor values, DNF parts
     · which peripherals were verified and how
     · every errata that applied and the workaround used
     · every mistake found in the schematic or layout, with
       enough detail for the next revision to fix it
     · photographs of any rework

   WHY IT MATTERS:
     · the next revision's fixes come from this list
     · when board #7 behaves differently from board #1, the
       log tells you what is different
     · a colleague debugging at 2am can read it
     · and PRODUCTION NEEDS TO KNOW which reworks are
       required on which units

/* --- AND THE THINGS TO BUILD INTO THE FIRMWARE NOW --- */
     · a BOOT BANNER printing the build ID, the git hash,
       the measured clock frequencies and the reset reason
     · a RESET REASON DECODER - power-on, watchdog,
       software, brownout, pin. When a unit reboots in the
       field this is the first thing you will want.
     · a FAULT HANDLER that prints the fault registers and
       the stacked frame (chapter 06)
     · STACK PAINTING and a high-water-mark report
     · a SELF-TEST command that exercises every peripheral
       and prints pass/fail - which then doubles as the
       production test
     · a persistent TRACE BUFFER surviving reset
       (chapter 02)

   ALL OF THIS COSTS A DAY DURING BRING-UP AND SAVES WEEKS
   OVER THE PRODUCT'S LIFE. It is also far harder to
   retrofit later, because by then the board is "working"
   and nobody wants to touch the startup code.          */
```

How it shows up

Symptom	Cause
Board completely dead	Reset held, no power, or a short — measure before assuming
Supply limits immediately	Short — power down and find it
Debugger will not connect	Chapter 03's checklist; try connect-under-reset
LED blinks at the wrong rate	Clock is not what you configured
Hangs before `main`	Spinning in a clock-ready loop, or flash wait states wrong
UART prints garbage	Clock wrong — go back to layer 3
UART prints nothing	Pin mux, TX/RX swapped, or peripheral clock off
Peripheral registers read zero	Clock not enabled

Symptom	Cause
Configuration register reads back different	Write-protected, reserved bits, or written while enabled
I2C never ACKs	Pull-ups, address convention, or wiring — scan the bus
SPI returns plausible garbage	Wrong CPOL/CPHA mode
External RAM fails only when warm	Marginal timing — test at temperature
Works on board 1, not board 7	Rework difference — read the bring-up log
Crystal will not start	Wrong load capacitors, or a 1× scope probe stopping it

Check yourself

- ✓ List the eight layers and explain why bring-up is strictly bottom up.
- ✓ Say what a blinking LED proves — all eight things.
- ✓ Give the pre-power checks, in order.
- ✓ Explain why a rail must be scoped as well as measured.
- ✓ Say why reading the chip ID is the right first debug step.
- ✓ Explain why you start on the internal oscillator.
- ✓ Write a clock-ready wait with a timeout and say why the timeout matters.
- ✓ Explain why flash wait states must be set before raising the clock.
- ✓ Describe how to verify the actual clock frequency.
- ✓ Give the three memory tests for external RAM and what each catches.
- ✓ Give the eight-step universal peripheral bring-up sequence.
- ✓ Say why garbled UART output points at the clock.
- ✓ List the six things to build into the firmware during bring-up.

Glossary

The terms and acronyms used across this site, with the chapter that explains each one properly. Definitions here are short on purpose — they are for recognition, not for learning from.

A – C

Term	Meaning
AArch64	The 64-bit ARM execution state. The instruction set is A64.
ABI	Application Binary Interface — register usage, stack layout, struct layout and calling convention. What lets separately compiled code link.
AEAD	Authenticated Encryption with Associated Data. Confidentiality and integrity in one primitive. cr-09
APIC	x86 interrupt controller. ARM's equivalent is the GIC.
ASan	AddressSanitizer — compile-time instrumentation catching memory errors. ref-04
Atomic	An operation that completes indivisibly with respect to other observers.
Attention	Weighted averaging of values, with weights from query–key similarity. ai-19
Barrier	An instruction constraining the ordering of memory operations. `DMB`, `DSB`, `ISB` on ARM.
BSP	Board Support Package — the vendor's port of an OS or RTOS to a board.
Bugcheck	A Windows kernel panic. The blue screen. ref-05
Cache coherency	Keeping multiple caches consistent. A DMA device outside the coherency domain requires explicit maintenance.
CFI	Control Flow Integrity — restricting indirect branches to valid targets.
Chinchilla	The scaling result: about 20 training tokens per parameter is compute-optimal. ai-23
Constant time	Execution whose timing does not depend on secret data. cr-19
Context switch	Saving one execution context and restoring another.
CoreSight	ARM's debug and trace architecture: DWT, ITM, ETM, TPIU. ref-03
Cross-entropy	The loss function of essentially every model here; equals negative log likelihood. ai-10

D – H

Term	Meaning
DMA	Direct Memory Access — a device reading or writing memory without the CPU. The reason SMMUs exist.
DPC	Deferred Procedure Call — Windows' deferred interrupt work, at DISPATCH_LEVEL. Linux's equivalent is a softirq or tasklet.
DPA	Differential Power Analysis — key recovery from many power traces. Constant-time code does not stop it. cr-19
DTB / Device tree	A data structure describing non-discoverable hardware to the kernel.
ECDH / ECDSA	Elliptic-curve key exchange and signatures. cr-14, cr-15
EL0–EL3	ARM Exception Levels: application, OS, hypervisor, firmware.
Embedding	A learned dense vector representing a token. ai-17
Errata	Documented silicon bugs. Read the sheet. ref-06

Term	Meaning
ETW	Event Tracing for Windows — always-compiled-in, runtime-enabled tracing. ref-05
Flash Attention	Tiled attention that never materialises the n×n matrix. Exact, not approximate. ai-19
Forward secrecy	Past sessions stay secret after a long-term key is stolen. Requires ephemeral keys. cr-13
ftrace	The Linux kernel's built-in function tracer. ref-04
GIC	Generic Interrupt Controller — ARM's interrupt distribution and prioritisation.
GQA	Grouped-Query Attention — query heads share key/value heads, shrinking the KV cache. ai-19
Hard fault	Cortex-M's catch-all exception for memory, bus and usage faults.
HKDF	The standard key derivation function: extract then expand, with a context string. cr-10
HMAC	The standard hash-based MAC. Its nesting defeats length extension. cr-08
HSM	Hardware Security Module — where signing keys belong. cr-18
Hypervisor	Software running at EL2 that virtualises the machine for guest operating systems.

I – M

Term	Meaning
IOMMU / SMMU	Address translation and access control for device DMA. The MMU for peripherals.
IRP	I/O Request Packet — the unit of I/O in the Windows kernel. `!irp` decodes one.
IRQL	Windows' interrupt request level. At DISPATCH_LEVEL and above you may not touch paged memory or block. ref-05
ISR	Interrupt Service Routine. Keep it short; defer the rest.
ITM / SWO	Cortex-M instrumentation trace and its output pin. Single-cycle logging. ref-02
KASAN	AddressSanitizer for the Linux kernel.
KMDF / UMDF	Windows Driver Framework, kernel and user mode. Preferred over raw WDM.
KV cache	Cached keys and values, making generation linear instead of quadratic. Often larger than the model. ai-25
LayerNorm / RMSNorm	Per-sample feature normalisation; RMSNorm drops the mean subtraction. ai-22
Lockdep	Linux's lock-order validator — reports potential deadlocks before they happen.
LoRA	Low-Rank Adaptation — fine-tuning via a small low-rank weight update. ai-24
MAC	Either Message Authentication Code (cr-08) or Multiply-Accumulate (ai-04). Context disambiguates.
Memory-bound	Limited by bandwidth rather than arithmetic. Token generation is memory-bound. ai-04
MFU	Model FLOPs Utilisation — achieved over peak. 40–55% is good. ai-23
MMIO	Memory-Mapped I/O — device registers in the address space. Always `volatile`.
MMU	Memory Management Unit — virtual to physical translation and permissions.
MoE	Mixture of Experts — many FFNs, a router picks a few per token. Saves compute, not memory. ai-22
MPU	Memory Protection Unit — region-based permissions without translation. Cortex-M.

N – R

Term	Meaning
NEON / SVE / Helium	ARM SIMD: Cortex-A, scalable vector, and Cortex-M (MVE) respectively.
Nonce	A number used once. Reuse breaks GCM catastrophically. cr-09
NPU	Neural Processing Unit — a fixed-function accelerator, usually int8 matmul. ai-27
NVIC	Cortex-M's nested vectored interrupt controller.
OTP / eFuse	One-time programmable bits. Where root key hashes and anti-rollback counters live. cr-18
Paged attention	Block-allocated KV cache with a per-request block table. Virtual memory, for attention. ai-25
Perplexity	exp of the cross-entropy — the effective number of options the model is choosing between. Not comparable across tokenisers. ai-10
PKI	Public Key Infrastructure — certificates chaining to a trusted root. cr-16
Pre-norm	Normalisation inside the residual branch. What makes deep transformers trainable. ai-22
Priority inversion	A high-priority task blocked by a low-priority lock holder that a medium-priority task preempts. ref-06
PSA Crypto	ARM's crypto API abstracting hardware and software implementations. cr-21
PUF	Physically Unclonable Function — a key derived from manufacturing variation, never stored.
Quantisation	Representing weights in fewer bits, with a scale factor. ai-26
RAII	Resource Acquisition Is Initialisation — C++ ownership tied to object lifetime.
Residual stream	The un-transformed path through a transformer; a shared bus every layer reads and adds to. ai-22
RLHF / DPO	Preference-based post-training, with and without a reward model. ai-24
RoPE	Rotary position embedding — rotate Q and K by angle ∝ position, giving relative position for free. ai-21
RTT	SEGGER Real-Time Transfer — logging via a RAM buffer read by the probe while running. ref-02

S – Z

Term	Meaning
Scaling laws	Loss falls as a power law in parameters, data and compute. Lets you predict a large run from small ones. ai-23
Secure boot	Each stage verifies the next before transferring control, rooted in immutable ROM. cr-15
Secure element	A separate certified chip holding keys that never leave it. cr-18
SFT	Supervised Fine-Tuning — instruction pairs, with the prompt masked out of the loss. ai-24
SMC	Secure Monitor Call — the instruction that enters EL3 / the secure world.
Softmax	Turns scores into a distribution. Always subtract the maximum first. ai-06
Speculative decoding	A small model drafts, the big model verifies in one pass. Identical output distribution. ai-25
Straight-through estimator	Passing the gradient through a non-differentiable step such as rounding. ai-26
SWD	Serial Wire Debug — ARM's two-pin debug port. ref-03
SwiGLU	The gated FFN used by most recent models; `d_ff` becomes 8/3·d. ai-22
TEE	Trusted Execution Environment — an isolated world, typically TrustZone plus a trusted OS.
Tokeniser	Text to integers, usually by BPE. Matched permanently to its model. ai-16

Term	Meaning
TPM	Trusted Platform Module — PCRs, sealing, attestation, monotonic counters.
Transformer	Attention plus FFN, each wrapped in a norm and a residual, stacked. ai-20
TrustZone	ARM's secure/non-secure partitioning, propagated across the interconnect by the NS bit.
TSan	ThreadSanitizer — data race detection. ~10× slowdown, worth it.
UB	Undefined Behaviour — the optimiser may assume it never happens, and does. ref-06
volatile	Prevents the compiler caching a value in a register. Provides *no* atomicity and *no* ordering between objects.
WCET	Worst-Case Execution Time — the number that matters in a real-time system, not the average.
WDM	Windows Driver Model — the raw IRP-based interface beneath KMDF.
WPP	Windows software tracing: formatted off-device, negligible cost, runtime-enabled. ref-02
Watchpoint	A breakpoint on *data* access. The fastest way to find what corrupts a variable. ref-03
X25519	Curve25519 key exchange. 32-byte keys, constant time, no validation to get wrong. cr-13
XPU	Access-control hardware gating which masters may reach which address ranges.
XTS	The disk-encryption mode. Confidentiality only — *no* integrity. cr-18
ZeRO / FSDP	Sharding optimiser state, gradients and parameters across devices. ai-23

TERMS THAT MEAN DIFFERENT THINGS IN DIFFERENT TRACKS

```
MAC       Message Authentication Code / Multiply-Accumulate
          / Media Access Control
KEY       a cryptographic key / an attention key vector
BLOCK     a cipher block / a storage block / a transformer
          block / a scheduling state
CACHE     CPU cache / KV cache / page cache
TENSOR    an n-dimensional array (the physics meaning is
          not implied)
TRACE     execution trace / matrix trace
DECODE    instruction decode / token generation
PAGE      a memory page / a flash page
ATTENTION the mechanism / what you should pay to the
          errata sheet
```

Check yourself

- ✓ Define IRQL and state the rule that causes most Windows driver bugchecks.
- ✓ Explain what a nonce is and what its reuse costs in GCM.
- ✓ Say what the KV cache is and why it can exceed the model.
- ✓ Explain what `volatile` does and the two things it does not.
- ✓ Distinguish MMU, MPU and SMMU.
- ✓ Say what forward secrecy requires.
- ✓ Explain why perplexity is not comparable across tokenisers.
- ✓ Give three meanings of "MAC" and say how context disambiguates.
- ✓ Say what XTS provides and what it conspicuously does not.
- ✓ Explain what a watchpoint does and why it is underused.

Books, manuals and practice environments

Where to go after this site. The primary sources, the books worth owning, and — more importantly — the hardware and exercises that turn reading into competence.

Primary sources

```
READ THE SPECIFICATION. Not a blog post about the
specification.

This is the single habit that separates engineers who
can solve novel problems from those who can only apply
remembered patterns. Secondary sources are for
orientation; the primary source is where the answer
actually is, and it is usually free.

ARM
  ARM Architecture Reference Manual (ARM ARM) for
    Armv8-A / Armv9-A - the definitive text. Enormous.
    Use it for lookup, never linearly.
  Armv8-M Architecture Reference Manual
  Cortex-M Technical Reference Manuals - per-core
  Procedure Call Standard (AAPCS) - the ABI
  ARM Cortex-M Generic User Guide - the practical
    subset, and the right place to start
  TrustZone, SMMU and GIC architecture specifications
  -> developer.arm.com/documentation

YOUR SILICON VENDOR
  The REFERENCE MANUAL - every register, every bit
  The DATASHEET - electrical, timing, pinout
  The ERRATA SHEET - READ THIS ONE. It explains
    behaviour the reference manual does not.
  Application notes - often excellent and overlooked

LINUX
  The kernel source itself. Documentation/ is
    substantial and current.
  Documentation/process/ - how to contribute
  lore.kernel.org - mailing list archives, where the
    reasoning lives
  elixir.bootlin.com - the best kernel source browser
    there is
  LWN.net - the best technical journalism in the field.
    Subscribe; it funds the writing.

WINDOWS
  learn.microsoft.com/windows-hardware/drivers
  The WDK samples on GitHub - genuinely good starting
    points
  The Windows Internals books (below)

STANDARDS
  ISO C (N3220 for C23 drafts; the drafts are free)
  ISO C++ (eel.is/c++draft, and cppreference.com for
    daily use)
  RFCs - rfc-editor.org. TLS is RFC 8446; ChaCha20-
    Poly1305 is RFC 8439.
  NIST FIPS and SP 800 series for cryptography
```

Books worth owning

Area	Book
Architecture	Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* — the reference. Patterson & Hennessy, *Computer Organization and Design* — the accessible one, start here.
Architecture	Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* — **if you buy one book on this list, buy this one.**
C	Kernighan & Ritchie, *The C Programming Language* — dated, still the clearest. Klemens, *21st Century C* for modern practice.
C	Seacord, *Effective C* — current, careful about undefined behaviour.
C++	Meyers, *Effective Modern C++*. Stroustrup, *A Tour of C++* for a fast survey.
Rust	*The Rust Programming Language* (free online). *The Rustonomicon* for unsafe. *The Embedded Rust Book*.
Operating systems	Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* — free, and better than most paid textbooks.
Linux kernel	Love, *Linux Kernel Development* (dated but clear). Corbet et al., *Linux Device Drivers* (dated; read alongside current source). Kaiwan Billimoria, *Linux Kernel Programming* — current.
Windows	Russinovich et al., *Windows Internals* parts 1 and 2. Orwick & Smith, *Developing Drivers with the WDF*.
Embedded	White, *Making Embedded Systems* — the best book on the discipline rather than a chip. Yiu, *The Definitive Guide to ARM Cortex-M*.
Real-time	Liu, *Real-Time Systems*. Buttazzo, *Hard Real-Time Computing Systems*.
Compilers	Aho et al., *Compilers* (the dragon book). Cooper & Torczon, *Engineering a Compiler* — more practical. Nystrom, *Crafting Interpreters* — free, and the most enjoyable.
Cryptography	Aumasson, *Serious Cryptography* — **the right book for an engineer.** Ferguson, Schneier & Kohno, *Cryptography Engineering*. Boneh & Shoup, *A Graduate Course in Applied Cryptography* (free) for the theory.
Security	Dowd et al., *The Art of Software Security Assessment*. Huang, *The Hardware Hacker*.
ML foundations	Bishop, *Pattern Recognition and Machine Learning*. Goodfellow et al., *Deep Learning* (free online). Prince, *Understanding Deep Learning* (free) — the most current.
ML practice	Géron, *Hands-On Machine Learning*. Warden & Situnayake, *TinyML* — directly relevant to ai-27.
Mathematics	Strang, *Introduction to Linear Algebra* (and his MIT lectures). Deisenroth et al., *Mathematics for Machine Learning* (free).
Debugging	Agans, *Debugging: The Nine Indispensable Rules* — short, and it will change how you work. Zeller, *Why Programs Fail*.
Craft	Hunt & Thomas, *The Pragmatic Programmer*. Ousterhout, *A Philosophy of Software Design* — short and unusually good on complexity.

Hardware to learn on

```
/* YOU CANNOT LEARN EMBEDDED SYSTEMS WITHOUT HARDWARE. Buy
   something. The total cost of a genuinely capable bench is
   under three hundred dollars.                          */

/* --- BOARDS, roughly in order of what to get first --- */
    RASPBERRY PI PICO / PICO 2       ~$5
      Dual Cortex-M0+/M33, and the RP2040/RP2350's PIO is a
      genuinely novel and instructive peripheral. Excellent
      documentation. START HERE if you have nothing.

    ST NUCLEO or DISCOVERY           ~$15-30
      Cortex-M0 to M7, ON-BOARD ST-LINK, huge community.
      The most practical general-purpose learning platform.

    NORDIC nRF52840-DK               ~$50
      Bluetooth, TrustZone-M on the nRF5340/nRF54, and
      Zephyr support. For wireless and security work.

    ESP32-S3                         ~$10
      Wi-Fi, Bluetooth, an NPU-ish vector unit, and a
      completely different ecosystem worth seeing.

    RASPBERRY PI 4 or 5              ~$60
      A full Cortex-A Linux machine you can compile kernels
      for and reflash without fear. THE RIGHT PLATFORM FOR
      THE LINUX DRIVER TRACK.

    BEAGLEBONE BLACK / AI-64
      Excellent for device tree, PRU real-time cores, and
      mainline kernel work.

    QEMU                             free
      Boot a kernel, attach GDB, and never brick anything.
      -M virt with a Cortex-A53 runs the Linux track's
      examples, and it is what the PL031 driver project
      targets.

    AN OLD PC or a VM                free
      For the Windows driver track: a second machine or a
      Hyper-V guest for kernel debugging (ref-05).

/* --- INSTRUMENTS, in order of value per dollar --- */
    LOGIC ANALYSER                   $10-400
      A cheap 8-channel clone with sigrok/PulseView is
      $10 and will transform your debugging. A Saleae is
      $400 and better. GET ONE BEFORE YOU GET AN
      OSCILLOSCOPE - most embedded problems are digital
      timing, and a logic analyser decodes I2C, SPI and
      UART for you.

    OSCILLOSCOPE                     $150-400
      Rigol DHO800 or similar. For analogue questions:
      supply ripple, rise times, crystal oscillation
      (ref-07).

    MULTIMETER                       $30
      Continuity, voltage, current. The most-used
      instrument on any bench.

    BENCH SUPPLY WITH CURRENT LIMIT  $60
```

```
      The current limit is the feature. It has saved more
      boards than any other tool.

    USB-SERIAL ADAPTER (3.3V)        $5
      Buy several. You will lose them.

    J-LINK EDU                       $60
      For RTT (ref-02) and reliable flashing across
      vendors.

    USB PROTOCOL ANALYSER            $100+
      Only if you do USB work - and if you do, essential.

/* --- AND THE THING PEOPLE SKIP --- */
    A SOLDERING IRON AND SOME FLUX. Being able to fit a
    tag-connect header, tack a wire onto a test point, or
    lift a pin is a genuine debugging capability
    (ref-07).                                          */
```

> **THE SINGLE MOST VALUABLE PURCHASE**
>
> A cheap logic analyser. Ten dollars, and it converts "the driver doesn't work" into "the bytes are going out but the ACK never comes back". Almost every embedded problem is a question about what is actually happening on a wire, and until you can see the wire you are guessing.

Projects that teach

```
/* READING IS NOT LEARNING. Build these, in roughly this
   order. Each one forces a specific set of chapters to
   become concrete.                                      */

  1. BLINK AN LED WITH NO LIBRARIES.
     Registers only, from the reference manual. Then write
     the startup code and linker script yourself.
     -> the C track, computer architecture, ref-07

  2. A UART DRIVER, polled, then interrupt-driven, then
     with a ring buffer, then DMA.
     -> interrupts, concurrency, ref-02

  3. A BOOTLOADER that verifies and jumps to an
     application image.
     -> linkers, memory maps, and then signatures
        (cr-15)

  4. PORT AN RTOS, or write a cooperative scheduler with a
     context switch in assembly. Writing the context switch
     yourself teaches more than a term of reading.
     -> the OS track

  5. A LINUX CHARACTER DRIVER, then a platform driver with
     device tree, then one with interrupts and DMA.
     -> the Linux track; lx-28 and lx-29 are exactly this

  6. A WINDOWS KMDF DRIVER and its user-mode client.
     -> the Windows track; win-23 and win-24

  7. IMPLEMENT AES AND SHA-256 FROM THE SPECIFICATION.
     Then make them constant time. Then check with dudect.
     -> the cryptography track, cr-19

  8. A TLS CLIENT on an MCU, with a pinned root and a
     device certificate.
     -> cr-16, cr-17, cr-21

  9. A COMPILER for a small language, following Crafting
     Interpreters. Then emit real machine code.
     -> the compilers track

 10. THE TINY TRANSFORMER (ai-28), then shrink it, quantise
     it, and run it on the Pico.
     -> the AI track, end to end

 11. REVERSE-ENGINEER SOMETHING YOU OWN. Dump the firmware
     from a cheap device, find the UART, identify the
     architecture, disassemble it. Legally, on your own
     hardware. This teaches more about how real products
     are built - and how badly - than any number of
     tutorials.

 12. FIX A BUG IN AN OPEN-SOURCE PROJECT, and get the patch
     merged. The review is the education.
     -> lx-30 covers the kernel process specifically

/* --- HOW TO PRACTISE, not just what --- */
    · BUILD SOMETHING YOU WANT. Motivation carries you
      through the tedious parts, and there are many.
```

```
    · FINISH THINGS. A finished small project teaches more
      than three abandoned ambitious ones, because the last
      20% is where the real problems live.
    · READ OTHER PEOPLE'S CODE. The kernel, FreeRTOS,
      Zephyr, mbedTLS, llama.cpp. Pick a subsystem and read
      it properly.
    · EXPLAIN IT. Write it down, or teach someone. You will
      discover what you do not actually understand.
    · AND KEEP A LOG. Of bugs you found, and how. It
      compounds.                                          */
```

Staying current

```
/* --- WORTH FOLLOWING --- */
    LWN.net                  Linux and systems. Subscribe.
    lore.kernel.org          the mailing lists themselves
    Hacker News, lobste.rs   noisy, occasionally excellent
    Interrupt (Memfault)     one of the few genuinely good
                             embedded engineering blogs
    Embedded.fm              podcast, long-running
    The Amp Hour             hardware
    /r/embedded, /r/rust_embedded
    Zephyr and Rust-embedded matrix/discord channels

/* --- CONFERENCES, most with free recorded talks --- */
    Embedded Linux Conference, Linux Plumbers, FOSDEM,
    Embedded World, CppCon, RustConf, Chaos Communication
    Congress (excellent hardware security work), Black Hat
    and DEF CON, NeurIPS and ICML for machine learning.

    WATCH THE TALKS. Many are better than the equivalent
    book chapter and are forty minutes long.

/* --- CERTIFICATIONS, honestly --- */
    Largely not valued in this field. A portfolio of things
    you have built, and contributions people can read, are
    worth considerably more. The exceptions are
    domain-specific safety certifications (IEC 61508,
    ISO 26262, DO-178C) where the process qualification
    genuinely matters to employers in those industries.

/* --- AND A CLOSING NOTE ON THIS SITE --- */

    281 chapters is a great deal of reading and none of it
    is a substitute for the work. The tracks were written
    to be a MAP - to tell you what exists, what it is for,
    what the failure modes look like, and where the primary
    source is.

    THE ACTUAL LEARNING HAPPENS WHEN YOU BUILD SOMETHING
    THAT DOES NOT WORK AND YOU FIND OUT WHY.

    Everything in ref-01 through ref-07 exists to make that
    process faster. Everything else exists so you know what
    you are looking at when you get there.

    Go and build something.                               */
```

Check yourself

- ✓ Name the primary source you would consult for an ARM exception-level question.
- ✓ Say which vendor document explains behaviour the reference manual does not.
- ✓ Name one book per track you intend to read.
- ✓ List the minimum hardware bench and its total cost.
- ✓ Say why a logic analyser comes before an oscilloscope.
- ✓ Explain why a current-limited bench supply matters.
- ✓ Pick three projects from the list and say which chapters each exercises.
- ✓ Give the four rules for how to practise, not just what to build.
- ✓ Say what this site is for, and what it is not a substitute for.